Operációs rendszerek

Az alapoktól a valós idejű rendszerekig és egy saját kernel megvalósításáig

Gellai István

2026

Tartalomjegyzék

Impresszum **7**

I. RÉSZ – Alapok és rendszerfelépítés **8**

1 Előszó és olvasói útmutató **9**
1.1 Előszó: a könyv célja, célközönsége és a szerző 9
1.2 Hogyan használd a könyvet: tanulási utak, laborok és jelölések 13

2 Mi az operációs rendszer? **19**
2.1 Definíciók: erőforrás-kezelő, absztrakció és virtuális gép 19
2.2 Az OS szerepe és helye a rendszerben 26
2.3 Az operációs rendszerek története és evolúciója 35
2.4 Alapfogalmak: kernel, folyamat, fájl, rendszerhívás és felhasználói tér 41

3 Kernelarchitektúrák és belső struktúrák **50**
3.1 Monolitikus, mikrokernel és hibrid kernel 50
3.2 Réteges, moduláris és exokernel/unikernel felépítés 56
3.3 Kernelkomponensek: ütemező, memóriakezelő, VFS és hálózati stack 64
3.4 Kernel és felhasználói tér határa, ABI és driver modell 73

4 Operációs rendszertípusok **82**
4.1 Általános célú, szerver, desktop és mobil operációs rendszerek 82
4.2 Beágyazott, valós idejű és biztonsági kritikus rendszerek 88
4.3 Elosztott, hálózati, mainframe és HPC operációs rendszerek 95
4.4 Virtualizáció, konténerek, unikernel és speciális rendszerek 103

5 Boot, firmware és hardverkapcsolat **110**
5.1 BIOS, UEFI és a bootfolyamat 110
5.2 Bootloader, kernel betöltése és init 117
5.3 Firmware, ACPI, device tree és hardverleírás 126
5.4 Interruptok, időzítők és eszközök felfedezése 135

6 Rendszerhívások és felhasználói interfész **143**
6.1 Privilegiszintek és a kernel/user tér átlépése 143
6.2 Rendszerhívások, ABI és libc 148
6.3 Shell, init, daemonok és a userland felépítése 154
6.4 POSIX, Windows API és hordozhatóság 164

II. RÉSZ – Folyamatok, szálak és ütemezés **173**

7 Folyamatok és szálak **174**
7.1 Folyamatmodell, PCB és állapotátmenetek 174
7.2 Szálak, TLS és felhasználói/kernel szálak 183
7.3 Kontextusváltás, fork/exec és folyamatéletciklus 190
7.4 IPC: pipe, üzenetsor, osztott memoria és szignál 198

8 Ütemezés **208**
8.1 Ütemezési célok és metrikák . . . 208
8.2 Alapvető algoritmusok: FIFO, SJF, RR és prioritás . . . 215
8.3 Többszintű visszacsatolás és fair ütemezés . . . 224
8.4 Valós idejű ütemezés: RMS, EDF és deadline . . . 233

9 Szinkronizáció és konkurencia **244**
9.1 Versenyhelyzet, kritikus szakasz és atomi műveletek . . . 244
9.2 Zárak, spinlock, mutex és szemafórok . . . 250
9.3 Monitorok, condition variable és termelő-fogyasztó . . . 258
9.4 Holtpont, élok és deadlock-kezelés . . . 267

10 Haladó konkurencia **279**
10.1 Memóriamodell, láthatóság és memóriabarrierek . . . 279
10.2 Lock-free és wait-free struktúrák . . . 286
10.3 RCU, epoch-alapú visszanyerés és skálázható szinkronizáció . . . 296
10.4 Többmagos és heterogén rendszerek kihívásai . . . 305

III. RÉSZ – Memória és I/O **316**

11 Memóriakezelés alapjai **317**
11.1 Címterek, fizikai és virtuális cím . . . 317
11.2 Lapozás, szegmentálás és TLB . . . 323
11.3 Laphiba, demand paging és másolás . . . 329
11.4 Memóriavédelem, ASLR és izoláció . . . 335

12 Virtuális memória és lapcserélés **344**
12.1 Lapcsere-algoritmusok: FIFO, LRU és Clock . . . 344
12.2 Working set, thrashing és lokális/globális csere . . . 354
12.3 Huge page, THP és memóriatömörítés . . . 361
12.4 Memóriatérképezés, mmap és osztott memoria . . . 371

13 Memóriaallokáció és kernelmemória **382**
13.1 Heapallokátorok: free list, buddy és slab . . . 382
13.2 Fragmentáció és allokációs stratégiák . . . 392
13.3 NUMA, locality és memóriapoolok . . . 400
13.4 Kernelmemória, vmalloc, kmalloc és biztonság . . . 407

14 I/O és eszközkezelés **419**
14.1 Blokk- és karakteres eszközök . . . 419
14.2 Interruptok, DMA és pollozás . . . 429
14.3 Eszközmeghajtók és driver modell . . . 435
14.4 Tárolóeszközök: HDD, SSD, NVMe és buffering . . . 445

IV. RÉSZ – Fájlrendszerek és tárolás **455**

15 Fájlrendszer-alapok **456**
15.1 Fájlok, könyvtárak és metaadatok . . . 456
15.2 VFS, mount és fájlleírók . . . 462
15.3 Fájlallokáció: FAT, inode és extent . . . 469
15.4 Jogosultságok, kvóták és fájlrendszer-szemantika . . . 479

16 Fájlrendszer-implementációk **490**
16.1 FAT, ext4 és XFS . . . 490
16.2 Btrfs, ZFS és copy-on-write . . . 496
16.3 NTFS, APFS és zárt rendszerek . . . 502
16.4 Hálózati fájlrendszerek: NFS, SMB és objektumtár . . . 509

17 Megbízhatóság és crash consistency **518**
17.1 Journaling és tranzakciós metaadat . . . 518
17.2 Copy-on-write, checksum és scrub . . . 524
17.3 RAID, replikáció és hibatűrés . . . 535
17.4 Crash recovery, fsck és helyreállítás . . . 542

18 Tárolás, snapshot és adatvédelem **550**
18.1 Snapshot, klónozás és thin provisioning . . . 550
18.2 Titkosítás: LUKS, BitLocker és kulcskezelés . . . 556
18.3 Backup, archiválás és mentési stratégiák . . . 564
18.4 Objektumtár, blokk- és fájlszintű tárolás . . . 573

V. RÉSZ – Hálózat, biztonság és virtualizáció **580**

19 Hálózati stack **581**
19.1 Rétegmodellek és TCP/IP . . . 581
19.2 Socket API és hálózati rendszerhívások . . . 590
19.3 Csomagfeldolgozás, netfilter és eBPF . . . 600
19.4 Zero-copy, offload és hálózati teljesítmény . . . 608

20 Hálózati szolgáltatások **616**
20.1 DNS, DHCP és konfiguráció . . . 616
20.2 NFS, SMB és hálózati fájlrendszerek . . . 623
20.3 Konténerhálózat, NAT és overlay . . . 630
20.4 Hálózati biztonság és izoláció . . . 638

21 Biztonság és izoláció **647**
21.1 Felhasználók, csoportok és jogosultságok . . . 647
21.2 Capabilities, SELinux, AppArmor és seccomp . . . 656
21.3 Namespaces, cgroups és konténerizoláció . . . 666
21.4 Trusted boot, mérés és integritás . . . 676

22 Virtualizáció és konténerek **684**
22.1 Hypervisorok: Type 1, Type 2 és KVM . . . 684
22.2 Virtuális gépek, eszközvirtualizáció és migration . . . 692
22.3 Konténerek, OCI és image-építés . . . 700
22.4 Unikernel, WASM és könnyű izoláció . . . 709

VI. RÉSZ – Valós idejű, beágyazott és elosztott rendszerek **715**

23 Valós idejű rendszerek **716**
23.1 Determinizmus, latency és jitter . . . 716
23.2 RTOS ütemezés és priority inversion . . . 723
23.3 WCET, megszakítási késleltetés és erőforrás-elemzés . . . 732
23.4 Biztonsági és minősítési követelmények . . . 739

24 RTOS-példák és minták **748**
24.1 FreeRTOS és Zephyr . . . 748
24.2 QNX, VxWorks és kereskedelmi RTOS-ek . . . 757
24.3 AUTOSAR és járműipari architektúra . . . 765
24.4 RTOS-minták: watchdog, queue és event flag . . . 776

25 Beágyazott és mobil operációs rendszerek **787**
25.1 Beágyazott Linux és firmware . . . 787
25.2 Android architektúra és HAL . . . 796
25.3 iOS, biztonsági modell és sandbox . . . 805
25.4 Energiagazdálkodás és hőkezelés . . . 811

26 Elosztott és speciális operációs rendszerek **819**
26.1 Elosztott OS és klaszterek . . . 819
26.2 Mainframe és nagy rendelkezésre állású rendszerek . . . 823
26.3 HPC, batch és tudományos rendszerek . . . 832
26.4 Serverless, unikernel és jövőbeli irányok . . . 841

VII. RÉSZ – Tervezés, implementáció és záróprojekt **849**

27 Egyszerű OS tervezése **850**
27.1 Követelmények, célok és architektúra . . . 850
27.2 Bootfolyamat és kernel inicializálás . . . 855
27.3 Memóriakezelés és ütemező tervezése . . . 865
27.4 Rendszerhívások és userland tervezése . . . 873

28 Kernel implementáció **884**
28.1 C és assembly alapok, fordítás és linkelés . . . 884
28.2 Interruptkezelés, időzítő és kontextusváltás . . . 894
28.3 QEMU, GDB és kernel debugging . . . 902
28.4 Naplózás, panic és hibakezelés . . . 911

29 Fájlrendszer és eszközmeghajtó **922**
29.1 Egyszerű fájlrendszer tervezése . . . 922
29.2 Blokk-eszköz és driver implementáció . . . 928
29.3 VFS integráció és fájlleírók . . . 936
29.4 Shell és alapszolgáltatások . . . 948

30 Tesztelés, hibakeresés és teljesítmény **962**
30.1 Unit- és integrációs tesztek kernelben . . . 962
30.2 Emuláció, fuzzing és statikus elemzés . . . 972
30.3 Teljesítménymérés és tracing . . . 981
30.4 Regresszió és kiadási minőség . . . 992

31 Bővítés és haladó témák **1006**
31.1 SMP, preemption és valós idejű kiterjesztés . . . 1006
31.2 Hálózat és biztonsági modulok . . . 1013
31.3 Virtualizáció és konténerintegráció . . . 1023
31.4 Dokumentáció, portolás és karbantartás . . . 1028

32 Záróprojekt: minimális operációs rendszer **1038**
32.1 Követelmények és mérföldkövek . . . 1038
32.2 Boot, kernel és memória megvalósítása . . . 1047

32.3 Ütemező, rendszerhívások és shell 1057
32.4 Fájlrendszer, tesztelés és bemutató 1067

33 Kiadási checklist és továbbtanulás **1077**
33.1 Kiadási checklist és kódminőség 1077
33.2 Dokumentáció és reprodukciós csomag 1084
33.3 Ismert korlátok és nyitott kérdések 1090
33.4 Tanulási útvonal és szakirodalom 1100

Függelékek **1109**
Fogalomtár (magyar–angol) 1109
Képlet- és algoritmusgyűjtemény 1115
Rendszerhívás- és interfészgyűjtemény 1119
Kompetenciatérkép és tanulási útvonal 1122
Bibliográfia és forráskulcsok 1125

Impresszum

- **Szerző:** Gellai István
- **Szerkesztő:** Gellai István
- **Kiadó:** Tananyag Simple
- **Kiadás:** 1.0.0 – 2026
- **Licenc:** MIT
- **Forráskód:** chapters/ és lab/

A teljes licencszöveg a tároló gyökerének `LICENSE` fájljában található (a könyvkönyvtárból: `../../LICENSE`).

I. RÉSZ – Alapok és rendszerfelépítés

1 Előszó és olvasói útmutató

1.1 Előszó: a könyv célja, célközönsége és a szerző

Az operációs rendszerek a számítástechnika egyik legizgalmasabb, ugyanakkor legtöbb tévhittel terhelt területe. A legtöbb fejlesztő naponta használ folyamatokat, szálakat, fájlokat és hálózati hívásokat, mégsem látja pontosan, mi történik a rendszerhívás mögött. Ez a könyv arra vállalkozik, hogy ezt a hiányzó réteget érthetően, mégis mérnöki pontossággal mutassa be. Nem elégszik meg azzal, hogy definíciókat sorol fel: minden fogalmat példával, mérhető viselkedéssel és gyakorlati következményeivel együtt tárgyal.

A könyv motivációja három egymásba fonódó célból fakad. Az első: az operációs rendszerek belső szerkezetének mélyebb megértése. A második: annak bemutatása, hogyan lehet egy egyszerű, de valóban működő kernelt megtervezni és megvalósítani. A harmadik: annak tudatosítása, hogy az OS nem pusztán elméleti kurzusanyag, hanem olyan rendszerprogramozási és mérnöki terület, ahol a tervezési döntések közvetlenül befolyásolják a teljesítményt, a megbízhatóságot és a biztonságot.

Sok tankönyv vagy túl elméleti, vagy ellenkezőleg, kizárólag egy adott kernel forráskódjára koncentrál. Ez a könyv középutat kínál: az alapelveket szabványokból, hivatalos kernel-dokumentációkból és eredeti szakirodalomból eredezteti, majd azokat emulált környezetben, kézzel futtatható laborokkal mélyíti el. A laborok nem „nézd meg, ez így van” típusú feladatok, hanem valódi mérnöki tevékenységek: konfigurációt kell rögzíteni, várt és mért eredményt kell szétválasztani, a kódot pedig úgy kell megírni, hogy a memóriatulajdon, a hibakezelés és a versenyhelyzetek ellenőrizhetők legyenek.

1.1.1 A célközönség

A könyv elsődleges olvasói programozók, rendszermérnökök, beágyazott fejlesztők és olyan technikai vezetők, akik szeretnék érteni, mi zajlik a kernelben, és nem csupán használni akarják az operációs rendszer szolgáltatásait. A szerző feltételezi a C nyelv magabiztos használatát, az alapvető számítógép-architektúrát – regiszterek, memória, megszakítások –, valamint a parancssori rutint. Nem feltételez kernelprogramozási tapasztalatot, de elvárja, hogy az olvasó hajlandó legyen az assembly és a hardverspecifikus részeket a szükséges szintig elsajátítani.

A célközönség ennél azonban tágabb. A rendszergazdák és üzemeltetők számára is hasznos, mert a hibakeresés, a teljesítményelemzés és a biztonsági beállítások megértése megkönnyíti a mindennapi munkát. Azok a biztonsági szakemberek, akik izolációs mechanizmusokkal dolgoznak, a namespaces, cgroups és sandbox fejezetekben találnak szilárd alapot. A hallgatók pedig olyan tanulási utat kapnak, amely a gyakorlatban is alkalmazható tudást ad.

A könyv nem egyetlen „igaz” kernelre épül. Bár a példák többsége a Linux viselkedését tükrözi – mert ez a leginkább elérhető és dokumentált –, a fogalmakat úgy tárgyalja, hogy azok átvihetők legyenek FreeRTOS, Zephyr, QNX, Windows vagy éppen a saját, oktatási célú kernelünk kontextusába is. A hangsúly az elveken és a mérnöki módszeren van, nem a trükkös implementációs részleteken.

1.1.2 A szerző szakmai háttere

A szerző olyan szakember, aki pályafutása során aktívan dolgozott rendszerprogramozóként, beágyazott fejlesztőként és később technikai vezetőként is. Tapasztalatai közé tartozik az általános célú Linux-alapú rendszerek teljesítményhangolása, valós idejű vezérlők fejlesztése, valamint virtualizált és konténerizált környezetek üzemeltetése. Ez a háttér tükröződik a könyv szerkezetében: az elméleti alapok mellett mindig megjelenik a gyakorlati megvalósítás, az üzemeltetési szempont és a biztonsági következmény is.

A szerző oktatói tapasztalata abban mutatkozik meg, hogy a könyv minden fejezete tanulási célokat, ellenőrző kérdéseket és reprodukálható laborokat tartalmaz. Nem elég elolvasni a szöveget: az olvasónak magának kell megépítenie, lemérnie és dokumentálnia a rendszereket. A szerző nem tekinti magát mindenható forrásnak, ezért a könyvben minden nem triviális állítás forráshoz, szabványhoz vagy reprodukálható méréshez kötött. Ahol a technológia gyorsan változik – például friss kernelverziók, crate-API-k vagy eszköztámogatás –, ott a könyv explicit jelzi az ellenőrzés szükségességét.

Ez a fajta szerzői alapállás nem véletlen. Az operációs rendszerek világában a „state of the art” állítások gyorsan elavulnak, és a hibás benyomásból fakadó döntések súlyos következményekkel járhatnak. A szerző ezért minden esetben megkülönbözteti a mért és a várt/levezetett értékeket, és csak az előbbit mutatja be ellenőrzött eredményként. A kódpéldák szemléltetésre szolgálnak, nem éles rendszeren futtatandó utasítások; a laborok emulált környezetben futnak, így a hoszt valódi lemeze, bootloadere és rendszerpartíciója soha nem kerül veszélybe.

1.1.3 Az operációs rendszerek mint rendszerprogramozási és mérnöki terület

Az operációs rendszer nem csupán egy szoftverkomponens, hanem mérnöki rendszer: erőforrás-kezelő, absztrakciós réteg és virtuális gép egyszerre. Ez a hármasság adja a könyv gerincét. Az OS erőforrás-kezelőként osztja el a CPU-t, a memóriát, a tárolót és a hálózati sávszélességet a folyamatok között. Absztrakcióként elfedi a hardver bonyolultságát, és egységes felületet ad a rendszerhívásokon keresztül. Virtuális gépként pedig minden folyamatnak egy saját, elszigetelt végrehajtási környezetet kínál.

Ezek a szerepek gyakran feszültségben állnak egymással. A biztonság növelése lassíthatja a teljesítményt, a valós idejű garancia megnehezíti a méltányos ütemezést, a skálázhatóság pedig növeli a komplexitást. A könyv egyik legfontosabb célja, hogy az olvasó felismerje ezeket a kompromisszumokat, és képes legyen mérnöki döntést hozni ismeretükben. Ezért minden fejezet tartalmaz olyan mérőszámot, amely az adott tervezési döntés következményét számszerűsíti – legyen az ütemezési késleltetés, laphibaarány, fájlrendszer-ellenőrzés ideje vagy megszakítási latencia.

A rendszerprogramozói szemlélet azt jelenti, hogy a kód nem csupán illusztráció, hanem eszköz a megértéshez. A könyv laborjai ezért nem „hello world” szintű példákon alapulnak, hanem olyan feladatokon, amelyek valódi kernelkomponenseket érintenek: egy ütemező csere, egy lapcserélő algoritmus módosítása, egy egyszerű blokkmeghajtó írása. A kód mindig freestanding C, assembly vagy pszeudokód, és a nyelv jelölése minden blokkban megtalálható. A méréseknél a hardver, a fordítási beállítások, az órajelfrekvencia és a mérési módszer is dokumentálásra kerül, mert csak így reprodukálhatók az eredmények.

1.1.4 A könyv három olvasói útja

A könyv hét részből áll, és a tananyag szándékosan rétegzett. Nem kell az első oldaltól az utolsóig lineárisan haladni. Három jellegzetes olvasói utat különböztetünk meg, amelyek a célközönség különböző igényeit szolgálják.

1. A kerneltervező útja azoknak szól, akik meg akarják érteni az operációs rendszer belső működését, és saját kernelt szeretnének építeni. Ez az út a I. résztől a III. részig, majd a VI. rész kiválasztott fejezetein át a VII. rész záróprojektjéig vezet. Az olvasó sorra veszi a boot folyamatot, a rendszerhívásokat, a folyamatkezelést, az ütemezőt, a memóriakezelést, az I/O-t és a fájlrendszert, miközben minden lépést emulált környezetben ki is próbál. A cél nem egy teljes értékű termék, hanem egy olyan működő rendszer, amelyben minden döntés indokolt és dokumentált.

2. A rendszermérnök útja a gyakorlati rendszerfejlesztésre koncentrál. Az olvasó a II., III., IV. és V. részből válogat, különös tekintettel a szinkronizációra, a memóriakezelésre, a fájlrendszerekre, a hálózati stackre és a biztonsági izolációra. Ez az út azoknak való, akik Linux, Windows vagy beágyazott környezetben fejlesztenek, és jobban akarják érteni a rendszerhívások mögötti mechanizmusokat, a szálkezelést vagy a konténerizációs technológiákat. A hangsúly a hibakeresésen, a teljesítményelemzésen és a biztonságos tervezésen van.

3. A vezetői út a technikai vezetőknek és döntéshozóknak szól, akiknek nem kell kernelkódot írniuk, de meg kell érteniük a fontosabb komponensek és funkciók kapcsolatát. Ez az út a fejezetek bevezetőit, a fogalmi magyarázatokat és az összefoglalókat használja, a laborokat pedig csak akkor végzi el, ha a mélyebb megértés érdekében szükséges. A cél a rendszertervezési kompromisszumok felismerése, a technológiai választások megalapozása és a mérnökcsapat munkájának pontosabb értékelése.

Mindhárom út közös eleme a labor és az önellenőrzés. A könyv fejezetei nem passzív olvasásra valók: az olvasónak minden fejezet végén el kell végeznie legalább egy reprodukálható labort, rögzítenie kell egy mérőszámot, és válaszolnia kell egy önellenőrző kérdésre. Ez a módszer biztosítja, hogy a tudás ne csak verbális legyen, hanem gyakorlatban is rögzüljön.

1.1.5 Első labor: a kiindulási szint rögzítése

Ez a fejezet nem kódolási labort ad, hanem az olvasó számára az első, saját magára irányuló mérést. A cél az, hogy megállapítsd, honnan indulsz, és milyen konkrét tanulási célt tűzöl ki. Ez a labor a későbbi fejezetekhez képest egyszerűbb, de ugyanazokat a minőségi elveket követeli meg: reprodukálhatóság, rögzített konfiguráció, mérési napló és a mért és várt eredmény szétválasztása.

A labor lépései a következők:

1. **Környezet rögzítése.** Hozz létre egy jegyzetfüzetet (például Markdown formátumban), és írd le a használt operációs rendszert, annak verzióját, a hardver típusát, a CPU-t, a memóriaméretet és a fontosabb szoftververziókat. Ha emulált környezetet használsz, például QEMU-t vagy virtuális gépet, rögzítsd az emulált hardver konfigurációját is.
2. **Jelenlegi tudásszint önértékelése.** Válaszolj röviden az alábbi kérdésekre, és a válaszokat írd be a jegyzetfüzetbe:
   - Írtál-e már C programot, amely közvetlenül használ rendszerhívásokat (például `fork`, `open`, `mmap`)?
   - Ismered-e a processzormódokat (user/kernel) és a megszakítások szerepét?

- Volt-e dolgod linker scriptekkel vagy freestanding C kóddal?
- Tudod-e, mi a különbség a lapozás és a szegmentálás között?
- Ismered-e valamelyik RTOS (például FreeRTOS, Zephyr) ütemezési modelljét?

3. **Kiindulási mérés.** Végezz egy egyszerű, reprodukálható mérést, amely a rendszer alapvető teljesítményét jellemzi. Például fordíts le egy tíz soros C programot, és mérd meg a fordítási időt. Rögzítsd a mért értéket, a használt parancsot, a fordító verzióját és az optimalizálási szintet. Ezután határozd meg, mit vártál volna – például „egy ilyen kis program fordítási ideje jellemzően 0,1 másodperc alatt van” –, és hasonlítsd össze a mért értékkel. A várt értéket jelöld levezetésként vagy becslésként, és válaszd el egyértelműen a mért értéktől.

4. **Személyes tanulási cél kijelölése.** Fogalmazz meg egy SMART célt: legyen konkrét, mérhető, elérhető, releváns és időhöz kötött. Például: „A könyv I–III. részének elvégzése után képes leszek egy egyszerű, megszakításvezérelt ütemezőt írni emulált környezetben, és a kontextusváltási időt megmérve dokumentálni.” A célt írd be a jegyzetfüzetbe.

5. **Mentési pont.** A jegyzetfüzetet és az esetleges kódot mentsd el egy verziókezelt tárolóba vagy egy eldobható helyre, hogy a későbbi fejezetekben vissza tudj rá hivatkozni.

Ez a labor azért fontos, mert a későbbi munkád alapját képezi. A konkrét mérési gyakorlat rámutat arra, hogy a mérés nem csak a nagy teljesítményteszteknél fontos, hanem a legegyszerűbb feladatnál is. A résztvevők gyakran hajlamosak elhagyni a konfiguráció rögzítését vagy a várt érték megfogalmazását, pedig ezek hiányában az eredmény nem reprodukálható és nem értelmezhető.

1.1.6 Gyakori hibák és helyes gyakorlat

Az előszó és a hozzá tartozó labor kapcsán több tipikus hibát érdemes előre jelezni, hogy az olvasó felismerje és elkerülje őket.

Az egyik leggyakoribb hiba, hogy az olvasó a tanulási célt túl általánosan fogalmazza meg. „Jobban akarom érteni az operációs rendszereket” – ez nem SMART cél, mert nem mérhető, nincs időhöz kötve, és nem derül ki, mi számít elégséges eredménynek. Helyesebb például: „Hat hét alatt elvégzem a II. rész 7. és 8. fejezetét, és az ütemező labort dokumentált mérési naplóval együtt bemutatom.” Ez már konkrét, ellenőrizhető és reális.

A második gyakori hiba a környezet rögzítésének elmulasztása. Sokan úgy vélik, hogy a verziók és a hardver nem lényegesek, hiszen „a program úgyis ugyanúgy fut”. Ez azonban alapvetően hamis, különösen az operációs rendszerek területén, ahol a kernelverzió, a processzor mikroarchitektúrája, a fordító optimalizálási beállításai és a rendszerterhelés is jelentősen befolyásolhatják a mért értékeket. A helyes gyakorlat az, hogy minden mérés előtt rögzíted ezeket a paramétereket, és a mérési naplóban is feltünteted őket.

A harmadik hiba a mért és várt érték összemosása. Ha például egy ütemező várható késleltetését számolod ki, az egy levezetett érték, nem mért. A mért érték csak a tényleges futásból származik, és eltérhet a várt értéktől. Az eltérés önmagában nem hiba, hanem a megértés kiindulópontja. A könyv későbbi fejezeteiben is fontos, hogy mindig jelöld, mi az, amit megmértél, mi az, amit számítottál, és mi az, amit csak feltételeztél.

Végül említsük meg azt a hibát, amikor az olvasó a kódpéldákat éles rendszeren, azaz a mindennapi munkagépén próbálja ki. A könyv laborjai emulált környezetben futnak, vagy eldobható hardveren. A kernelkísérletek során a hoszt valódi lemezét, bootloaderét és rendszerpartícióját tilos módosítani. Az emulált környezet előnye, hogy elrontott kísérlet után egyszerűen visszaállítható, és a hibakereséshez is jobb eszközök állnak rendelkezésre.

1.1.7 Összegzés

Ez a fejezet bemutatta a könyv motivációját, célközönségét és a szerző szakmai hátterét. Láttuk, hogy az operációs rendszerek nem csupán elméleti fogalmak gyűjteménye, hanem olyan rendszerprogramozási terület, ahol a mérnöki módszer – reprodukálhatóság, mérés, dokumentáció – éppoly fontos, mint a fogalmi megértés. A könyv három olvasói utat kínál: a kernelltervező, a rendszermérnök és a vezető útját; mindegyik közös eleme a labor, a mérés és az önellenőrzés.

Az első labor célja a személyes kiindulási szint rögzítése volt. Ennek során meghatároztad a környezetedet, felmérted a jelenlegi tudásodat, elvégeztél egy egyszerű mérést, és megfogalmaztál egy SMART tanulási célt. Ez a folyamat a későbbi fejezetekben is hasonlóan zajlik majd, csak a feladatok egyre összetettebbek lesznek. A labor során ügyelj arra, hogy a jegyzetfüzet verziókezelt legyen, és a mért és várt értékeket mindig válaszd el.

A következő fejezet, az „Hogyan használd a könyvet: tanulási utak, laborok és jelölések – Hogyan használd a könyvet”, ezekre az alapokra épít, és részletesen bemutatja a tanulási utakat, a jelöléseket és a laborok általános szabályait. Addig is érdemes átgondolni, melyik olvasói út illik leginkább a céljaidhoz, és hogyan fogod beosztani a tanulásra szánt időt. Az operációs rendszerek világa tág, de a módszer, amelyet ez a könyv követ, segít abban, hogy ne vessz el a részletekben, hanem mindig felismerd a nagyobb összefüggéseket.

1.2 Hogyan használd a könyvet: tanulási utak, laborok és jelölések

A könyv hét része egyetlen, felfelé bővülő ívet alkot: az alapfogalmaktól a gyakorlati implementációig. Az I. rész a rendszerfelépítés alapjait rögzíti: mi az operációs rendszer, milyen kernelarchitektúrák léteznek, hogyan indul el a gép, és hogyan beszél a felhasználói program a kernellel. Erre a fogalmi keretre épül a II. rész, amely a folyamatok, szálak, ütemezés és konkurencia világát tárgyalja. Itt válik láthatóvá, hogy az operációs rendszer nem passzív kódhalmaz, hanem folyamatosan döntéseket hozó erőforrás-kezelő. A III. rész a memória és az I/O kérdéseit dolgozza fel, beleértve a virtuális memóriát, a lapcserélést és az eszközmeghajtókat. A IV. rész a fájlrendszerek és a tárolás megbízhatóságára összpontosít, a V. rész pedig a hálózat, a biztonság és a virtualizáció rétegeit nyitja ki. A VI. rész a valós idejű, beágyazott, mobil és elosztott rendszerek speciális követelményeit mutatja be, végül a VII. részben a megszerzett tudást egy saját, minimális kernel megtervezésére és implementációjára fordítjuk.

Ez a szerkezet nem véletlen: minden rész az előzőre épít, miközben önállóan is értelmezhető. A folyamatkezeléshez kell a rendszerhívások ismerete, a virtuális memóriához a folyamatok kontextusváltása, a fájlrendszerekhez az I/O és a memóriakezelés, a biztonsághoz pedig a folyamatok, a memória és a hálózat együttes működése. A VII. rész ezért nem egy újabb elszigetelt téma, hanem a korábbi hat rész szintézise: egy egyszerű, de működő operációs rendszer felépítése.

1.2.1 A hét rész logikája

A tanulási ív három nagyobb szakaszra bontható. Az első szakasz (I–III. rész) a klasszikus operációsrendszer-alapokat fedi le: rendszerfelépítés, folyamatok, memória és I/O. A második szakasz (IV–V. rész) a tárolási és hálózati rétegeket, valamint a biztonsági és virtualizációs technikákat tárgyalja. A harmadik szakasz (VI–VII. rész) a speciális rendszerek és a saját implementáció világába vezet be. Ez a felosztás azt is tükrözi, hogy az operációs rendszer nem monolitikus tudásterület: egy beágyazott fejlesztő például nagyobb hangsúlyt helyezhet a VI. részre, míg egy infrastruktúra-mérnök a IV. és V. részt forgathatja sűrűbben.

Az I. rész különösen fontos, mert itt alakul ki az a közös szókincs, amely nélkül a későbbi fejezetek érthetetlenek lennének. A kernel, a processz, a fájlleíró, a rendszerhívás, a userland és a kernelmód fogalma mind itt definiálódik. A boot és firmware fejezet (Boot, firmware és hardverkapcsolat) például megalapozza a VII. rész kernelindítási folyamatának megértését. Hasonlóan, a rendszerhívások fejezete (Rendszerhívások és felhasználói interfész) előkészíti a saját shell és userland felépítését.

A II. rész a rendszer dinamikáját vizsgálja. A folyamatok és szálak modellezése után az ütemezési algoritmusok, majd a konkurencia és a szinkronizáció következik. A rész végén a haladó témák – memóriamodell, lock-free struktúrák, RCU – már a modern, sokmagos rendszerek gyakorlati kérdéseit érintik.

A III. rész a memória és az I/O elválaszthatatlan kapcsolatát mutatja be. A virtuális címfordítás, a TLB, a laphibakezelés és a memóriavédelem mind ahhoz kellenek, hogy a felhasználói programok izoláltan, mégis hatékonyan futhassanak. Az I/O fejezet pedig az eszközmeghajtók és a DMA mechanizmusain keresztül azt mutatja meg, hogyan beszél a kernel a hardverrel.

A IV. rész a fájlrendszerek absztrakciójától indul, majd konkrét implementációkon (FAT, ext4, XFS, Btrfs, ZFS) keresztül eljut a naplózás, a copy-on-write, a RAID és a crash recovery kérdéséig. A VII. rész egyszerű fájlrendszere ebből a fejezetből merít.

Az V. rész a hálózati stack, a biztonság és a virtualizáció összefüggéseit tárja fel. A TCP/IP rétegek, a socket API, a netfilter és az eBPF mellett a namespaces, cgroups, capabilities és a hypervisorok is itt kapnak helyet. Ez a rész elengedhetetlen a konténerek és a felhőalapú rendszerek megértéséhez.

A VI. rész a valós idejű rendszerek determinisztikus viselkedésére, a priority inversionre, a WCET elemzésre és a FreeRTOS/Zephyr típusú RTOS-ekre koncentrál. A mobil rendszerek (Android, iOS) és az elosztott, HPC környezetek szintén itt szerepelnek.

A VII. rész a teljes könyv csúcspontja: egy minimális operációs rendszer tervezése és implementálása. A fejezet elején a követelményeket és az architektúrát rögzítjük, majd lépésről lépésre haladunk a boottól a memóriakezelésen, az ütemezőn, a rendszerhívásokon és a fájlrendszeren át a shellig. A záróprojekt (Záróprojekt: minimális operációs rendszer) egy konkrét, bemutatható kernelt vár el.

1.2.2 Tanulási utak a könyvön keresztül

A könyv lineárisan is olvasható, de a különböző célokhoz más-más útvonalak ajánlottak. A célközönség – programozó, rendszermérnök, beágyazott fejlesztő, tech lead – eltérő előismeretekkel és érdeklődéssel érkezik.

Kezdő kernelérdeklődő: Olvasd végig az I–IV. részt, a laborokat minden fejezetnél elvégezve. A V. részből válogass a hálózat és a biztonság fejezeteiből, majd a VII. részben építsd meg a saját rendszeredet. A VI. rész első két fejezetét (valós idejű alapok, RTOS-példák) érdemes átfutni, mert a determinizmus és a jitter fogalmai az általános célú rendszereknél is hasznosak.

Rendszermérnök vagy SRE: A I. rész után ugorj a III. és IV. részre (memória, fájlrendszerek), majd a V. rész hálózati és biztonsági fejezeteire. A VI. részből az elosztott rendszerek és a mainframe/HPC témák relevánsak. A VII. rész implementációs fejezetei segítenek a kernelhibák és a teljesítményproblémák diagnosztizálásában, még akkor is, ha soha nem írsz saját kernelt.

Beágyazott vagy RTOS-fejlesztő: Az I. és II. rész alapos ismerete mellett a VI. rész a fő fókusz. A III. rész memóriakezelése és az I/O fejezete elengedhetetlen, mert a beágyazott

rendszereknél a memória szűkössége és az eszközkezelés kritikus. A VII. rész leegyszerűsített kernele jó gyakorlópálya az RTOS-koncepciók megértéséhez.

Tech lead vagy döntéshozó: A könyv áttekintő olvasása ajánlott: az egyes részek bevezető fejezetei, a biztonsági és virtualizációs fejezetek, valamint a valós idejű követelményeket tárgyaló egységek. A laborok közül a reprodukciós csomagok és a mérési módszertan megismerése hasznos a csapatokkal való kommunikációhoz.

Minden fejezet végén találsz önellenőrző kérdéseket és legalább egy laborfeladatot. Az önellenőrző kérdések nem vizsgakérdések, hanem gondolkodtató feladatok, amelyek a fejezet kulcsfogalmaira kérdeznek rá. A válaszokat a fejezet szövege alapján, de saját szavaiddal kell megfogalmaznod.

1.2.3 A laborok szerkezete és a kísérleti szerződés

A laborok célja nem a kód másolása, hanem a jelenségek ellenőrizhető, reprodukálható megfigyelése. Minden laborhoz tartozik egy kiinduló állapot, build- és futtatási parancsok, elvárt eredmény, mérési napló és visszaállítási út. Ezt a készletet nevezzük **reprodukciós csomagnak**. A laborok kétféle környezetben futhatnak: emulált környezetben (QEMU, virtuális gép) vagy eldobható, dedikált hardveren. A hoszt valódi lemezét, bootloaderét és rendszerpartícióját tilos módosítani; ez a laborok első és legfontosabb szabálya.

Az emulált környezet előnye a biztonságos visszaállítás és a konfiguráció pontos rögzítése. A QEMU például lehetővé teszi a CPU, a memória és az eszközök paramétereinek explicit megadását. Az így kapott virtuális gép pillanatképe a hibakeresés és a reprodukció alapja. A valós hardveren végzett kísérleteknél ugyanezt a fegyelmet kell tartani: a hardver konfigurációját, a firmware verzióját és a mérési módszert dokumentálni kell.

Minden labor egy **labor szerződés** betartásával kezdődik. Ez a szerződés rögzíti:

1. **Környezet:** emulált (pl. QEMU, verzió: 8.2.0, CPU: `qemu-system-x86_64 -cpu Nehalem`, memória: 512 MiB) vagy valós hardver (pl. eldobható Raspberry Pi 4, firmware verzió).
2. **Kiinduló állapot:** milyen fájlok, kernelverzió, konfiguráció áll rendelkezésre.
3. **Műveletek:** a build, a futtatás és a mérés lépései, parancsokkal.
4. **Mentési pont:** milyen állapotba lehet visszaállni, hol tároljuk a mentést.
5. **Elvárt eredmény:** mit várunk elméleti alapon, és mit mérünk ténylegesen; a kettőt külön kell jelölni.
6. **Mérési napló:** a futási idő, ciklusszám, cache-viselkedés, laphibák, megszakítási késleltetés stb. rögzítése.

A mérési naplóban **soha nem szabad** a mért és a várt/levezetett értéket összemosni. Ha például egy ütemezési algoritmus várt átlagos várakozási ideje 3,2 ms, a mért érték pedig 3,45 ms, akkor a naplóban mindkét szám szerepel, megjelölve a mérési módszert és a lehetséges eltérés okát. A nem futtatott kódot, mérést vagy benchmarkot nem szabad ellenőrzött eredményként bemutatni.

A laborokhoz tartozó kódpéldák C, assembly vagy pszeudokód nyelvűek lehetnek. A kód szemléltetés, nem végrehajtandó utasítás az éles gépen. Minden kódblokk előtt jelöljük a nyelvet, például:

```
/* Példa: freestanding C, nem futtatható a hoszt operációs rendszerén */
void kmain(void) {
    /* kernel indítási pont */
    for (;;) {
        asm volatile("hlt");
```

```
    }
}
```

A laborok során a hardverhez való hozzáférést a lehető legkisebbre kell korlátozni. A megszakítások, lapasztalok és eszközök módosítása előtt mentési pontot kell létrehozni. A kernelkódban a memória- és erőforrás-tulajdonjogot, a hibakezelést és a versenyhelyzeteket külön ellenőrzési pontként kell kezelni.

1.2.4 Jelölések és hivatkozási konvenciók

A könyv egységes jelölésrendszert használ, hogy a fogalmak és a mérések egyértelműek legyenek.

Folyamatok és ütemezés. A folyamatokat P_i jelöli, a prioritást p_i, a CPU-időt C_i, a periódust T_i, a határidőt D_i. Az ütemezési példáknál a Gantt-diagramon kívül a késleltetési metrikákat (várakozási idő, fordulási idő, válaszidő) is meg kell adni. Például egy három folyamatot tartalmazó Round-Robin vizsgálatnál a diagram mellett táblázatban szerepelnek a per-folyamat várakozási idők.

Címek és méretek. A címeket hexadecimalisan írjuk, `0x` előtaggal, a méreteket bájtban vagy KiB/MiB egységben adjuk meg. A memóriatérképen a virtuális és a fizikai tartományt külön kell jelölni, például:

```
Fizikai: 0x00000000–0x000FFFFF  RAM
Virtuális: 0xFFFF0000–0xFFFFFFFF  Kernel
```

Rendszerhívások. Minden rendszerhívást névvel, paraméterekkel és visszatérési értékkel írunk le. A hibakódot és az `errno` értékét külön kezeljük:

```
ssize_t write(int fd, const void *buf, size_t count);
Visszatérés: beírt bájtok száma, vagy -1 hiba esetén;
hiba: EBADF (érvénytelen fájlleíró), EFAULT (érvénytelen puffer), stb.
```

Időzítési mérések. Minden mérésnél meg kell adni a hardvert, a fordítási beállításokat (optimalizálás, architektúra), az órajelfrekvenciát és a mérési módszert. Egyetlen szám önmagában nem értelmezhető: például `futási idő: 120 ms` nem elegendő; ehelyett `futási idő: 120 ms (QEMU x86-64, Nehalem, 2,4 GHz, gcc -O2, 1000 iteráció, időzítő: TSC)`.

Képletek. A matematikai képletek LaTeX és texmath/MathML formátumban is működnek, csak egyszerű parancsokkal, tizedesvessző `{,}` jelöléssel. Például a CPU-kihasználtság:

$$U = \sum_{i=1}^{n} \frac{C_i}{T_i}$$

Ábrák. A könyv a következő diagramtípusokat támogatja: `flow` (folyamatábra), `sequence` (szekvenciadiagram), `layers` (rétegdiagram), `heatmap` (hőtérkép), valamint vonal- és pontdiagramok. A memóriatérkép, a folyamatállapotok, az ütemezési Gantt és a fájlrendszer-blokkstruktúra saját ábrázolást kaphat. Minden diagramnak legyen címe, jelmagyarázata és feltételjegyzete.

Hivatkozások. Az elsődleges források a szabványok (POSIX, UEFI, ACPI, RISC-V/x86 privileged specifikáció), a hivatalos kernel-dokumentáció (Linux, FreeRTOS, Zephyr, QNX) és az eredeti szakirodalom. A forrásoknál a verziót, a dátumot és a konkrét szakaszt rögzíteni kell. Például: `Linux kernel 6.8 Documentation/admin-guide/mm/concepts.rst`. A „state of the art” állítás mindig dátumhoz kötött; a nem reprodukált eredményt jelölni kell.

1.2.5 Egy ellenőrizhető példa

A jelölések gyakorlati alkalmazására tekintsünk egy rövid példát. Tegyük fel, hogy egy laborban azt vizsgáljuk, hogyan befolyásolja a lapméret a TLB találati arányt. A labor szerződése így nézhet ki:

- **Környezet:** QEMU 8.2.0, `-cpu Nehalem`, 1 GiB RAM, Linux 6.8 (emulált).
- **Kiinduló állapot:** tiszta kernel, letiltott ASLR, rögzített lapméret 4 KiB.
- **Művelet:** egy felhasználói program 100 MiB memóriát jár be szekvenciálisan, 4 KiB-os lépésekkel; a TLB-találati arányt `perf` segítségével mérjük.
- **Mentési pont:** a virtuális gép pillanatképe a kísérlet előtt.
- **Elvárt eredmény:** szekvenciális bejárásnál a TLB-találati arány várhatóan magas, $> 90\%$, mert a lapok egymás után kerülnek be.
- **Mért eredmény:** például 94,3% TLB-találati arány, 5,7% tévesztés.
- **Értelmezés:** a mért érték megfelel a vártnak; az eltérés a mérési zajból és a kernel háttértevékenységéből adódik.

A naplóban a mért és a várt érték elkülönítve szerepel, a mérési módszer (`perf stat -e dTLB-loads,dTLB-load-misses`) pontosan megadva. Ez a példa mutatja, hogy a dokumentációs konvenció nem bürokratikus teher, hanem a megbízható mérés alapja.

1.2.6 Tipikus hibák és elkerülésük

A laborok során gyakori hiba a **nem rögzített konfiguráció**: egy mérés reprodukálhatatlan, ha nem tudjuk, milyen kernelverzió, CPU-modell vagy fordítói beállítás mellett készült. A helyes megoldás a reprodukciós csomag teljes kitöltése, még akkor is, ha az időigényesnek tűnik.

Másik jellemző hiba a **mért és várt érték összemosása**. Ha a naplóban csak annyi szerepel, hogy „a futási idő 120 ms”, akkor nem tudjuk, ez mért vagy becsült adat, és milyen feltételek mellett született. A helyes gyakorlat a két érték szigorú elkülönítése és a mérési feltételek megadása.

Hasonlóan veszélyes a **destruktív művelet valós környezetben**. A labor szerződés szerint a hoszt lemezét, bootloaderét vagy firmware-ét tilos módosítani. Aki ezt megszegi, adatvesztést okozhat. A helyes megoldás az emulált vagy eldobható hardver használata és a mentési pontok készítése.

Gyakori hiba továbbá **a hibakeresési adatok hiánya**. Egy kernelpánik esetén a `panic` üzenet, a stack trace, a napló és a reprodukálható konfiguráció együtt szükséges. Ha ezek bármelyike hiányzik, a hibát nem lehet megbízhatóan elemezni.

Végül **a kód szemléltetés és futtatás összekeverése** is problémához vezet. A könyv kódpéldái nem éles rendszeren futtatandó utasítások; ezeket mindig izolált laborban kell kipróbálni, a megadott emulált környezetben.

1.2.7 A fejezet laborja: kísérleti környezet és labor szerződés

A fejezethez tartozó labor célja, hogy előkészítsd a saját kísérleti környezetedet, és begyakorold a labor szerződés kitöltését. Az alábbi lépéseket követve hozz létre egy reprodukálható jegyzetfüzetet (például markdown formátumban), amely a későbbi fejezetek laborjainak sablonja lesz.

1. **Válassz környezetet:** QEMU-t vagy más emulátort. Ha QEMU-t használsz, telepítsd a 8.x verziót, és rögzítsd a pontos verziószámot a naplóban.

2. **Definiáld a virtuális gépet:** válassz CPU-modellt (például `Nehalem` vagy `qemu64`), memóriaméretet (például 512 MiB), és egy minimális bootolható lemezképet. A lemezkép lehet egy üres, írásvédett snapshot, amelyet a kísérlet előtt készítesz.
3. **Készíts mentési pontot:** mielőtt bármilyen módosítást végeznél, mentsd el a virtuális gép állapotát (például `qemu-img snapshot -c base disk.qcow2`).
4. **Rögzítsd a kiinduló állapotot:** írd le a kernelverziót, a boot parancsot, a hoszt operációs rendszer verzióját és a QEMU parancsot.
5. **Töltsd ki a labor szerződést:** a fenti szempontok alapján készíts egy sablont, amely tartalmazza a környezet, kiinduló állapot, műveletek, mentési pont, elvárt eredmény, mért eredmény és mérési napló mezőket.
6. **Ellenőrizd a naplóformátumot:** egy egyszerű időzítési méréssel (például `date` parancs hívásának ideje) mutasd be, hogyan választod el a várt és a mért értéket. Például várt érték: a parancs végrehajtása kevesebb mint 10 ms; mért érték: 2,3 ms (10 000 futtatás átlaga, TSC időzítővel).
7. **Dokumentáld a visszaállítást:** írd le, hogyan állsz vissza a mentési pontra (`qemu-img snapshot -a base disk.qcow2`), és igazold, hogy a rendszer a kiinduló állapotba került.

Az átadandó eredmény egy `lab-contract.md` nevű jegyzetfüzet, amely a fenti mezőket kitöltve tartalmazza. A jegyzetfüzet legyen része a későbbi laborok reprodukciós csomagjának. A mérési naplóban a mért és a várt értéket mindig külön sorban, megjegyzéssel ellátva tüntesd fel.

1.2.8 Önellenőrző kérdés

Magyarázd el, miért nem elegendő egy mérési eredményt pusztán a futási idővel dokumentálni, és sorold fel, milyen kiegészítő adatokra van szükség a reprodukálhatósághoz a labor szerződés szerint. Válaszodban térj ki a környezet, a fordítási beállítások, az órajelfrekvencia és a mérési módszer szerepére is.

1.2.9 Összefoglalás

Ez a fejezet a könyv használatának alapvonalát rögzítette: a hét rész logikáját, a lehetséges tanulási utakat, a laborok szerkezetét és a dokumentációs konvenciókat. A labor szerződés elkészítése nem egyszeri feladat, hanem a későbbi kísérletek biztonsági és minőségi kerete. A következő fejezetben (Definíciók: erőforrás-kezelő, absztrakció és virtuális gép) az operációs rendszer fogalmát definiáljuk, és erre a jelölésrendszerre építve vezetjük be az absztrakció és az erőforrás-kezelő szerepet.

2 Mi az operációs rendszer?

2.1 Definíciók: erőforrás-kezelő, absztrakció és virtuális gép

Az operációs rendszer fogalmát sokféleképpen megközelíthetjük. A tankönyv három, egymást kiegészítő nézőpontot vezet be: az operációs rendszer mint **erőforrás-kezelő**, mint **absztrakciós réteg**, és mint **virtuális gép**. E három nézőpont nem verseng egymással, hanem ugyanannak a rendszernek más-más vetületét hangsúlyozza. A fejezet célja, hogy ezeket a szempontokat egy konkrét, ellenőrizhető példán keresztül mélyítse el, és tisztázza a gyakran kevert fogalompárt: a **kernel** és az **operációs rendszer** viszonyát.

2.1.1 1. Az operációs rendszer mint erőforrás-kezelő

A számítógép hardver-erőforrásai – CPU, memória, háttértár, hálózati interfész, megszakításvezérlő, GPU, időzítők – mindig szűkösek. Egy rendszeren egyszerre több felhasználó, több feladat, több eszközmeghajtó igényel hozzáférést ugyanazokhoz a fizikai erőforrásokhoz. Az operációs rendszer feladata ezeknek az erőforrásoknak az igazságos, hatékony és védett elosztása.

Erőforrás-kezelőként az OS három alapvető tevékenységet végez:

1. **Ütemezés** – annak eldöntése, hogy egy adott erőforrást melyik folyamat kap meg és mennyi ideig.
2. **Foglalás és felszabadítás** – az erőforrások nyilvántartása, kiadása, majd visszavétele a folyamat befejezésekor vagy hibája esetén.
3. **Elszámolás és korlátozás** – annak biztosítása, hogy egyetlen folyamat se léphessen túl a számára kiosztott kereteken.

Ezt a nézőpontot legegyszerűbben a **fizikai memória allokációján** keresztül érthetjük meg. Tegyük fel, hogy két folyamat, P_1 és P_2 egyszerre szeretne memóriát igényelni. A rendelkezésre álló fizikai memória egy része már a kernel saját adatszerkezetei, a lapozó, a megszakításkezelő és a hálózati stack számára foglalt. A maradék szabad tartományból az OS memóriakezelője dönt:

- Mekkora darabot kapjon P_1?
- Mekkora darabot kapjon P_2?
- Mi történjen akkor, ha mindketten a teljes maradékot igénylik, és a rendszerben nincs több fizikai lap?

A klasszikus válasz az erőforrás-kezelői nézőpontra az, hogy a memóriakezelő egy **policy**-t alkalmaz. A policy meghatározza az elosztás elvét: például arányosan csökkenti mindkét igénylő keretét, prioritást ad az egyiknek, vagy megtagadja az újabb foglalást a túlzott igénylőtől. A **mechanizmus** ezzel szemben az, ahogyan a policy technikailag megvalósul: lapallokáció, lapfelszabadítás, laphiba-kezelés, TLB invalidálás.

A mechanizmus és a policy szétválasztása az operációs rendszerek egyik legfontosabb tervezési alapelve. A mechanizmus a kernel alacsony szintű, hardverközeli rétege; a policy a magasabb

szintű döntéshozatal. Ha a kettő szétválasztható, a rendszert könnyen lehet új ütemezési vagy memóriagazdálkodási elvekkel bővíteni anélkül, hogy a hardverközeli kódot újra kellene írni. Egy egyszerű allokátor esetén például a mechanizmus a memóriapoolból való kiosztás és a mappa bejegyzés karbantartása, míg a policy lehet „első illeszkedés”, „legkisebb szabad darab” vagy „buddy rendszerű kiosztás”.

Az erőforrás-kezelői nézőpont akkor válik igazán élessé, amikor egy erőforrás **megszakítható** (preemptible) vagy **nem megszakítható** (non-preemptible). A CPU-idő tipikusan megszakítható: az ütemező egy időszelet lejártakor elveszi a CPU-t a folyamattól, és másnak adja. A háttértáríró művelet ezzel szemben gyakran nem megszakítható abban az értelemben, hogy egy félig végrehajtott blokkműveletet nem lehet konzisztensen félbehagyni; az OS-nek meg kell várnia a befejezést, vagy tranzakciós naplóval kell garantálnia a visszaállíthatóságot. Az erőforráskezelőnek tehát minden erőforrástípusra külön-külön meg kell határoznia, hogy milyen foglalási, ütemezési és felszabadítási stratégiát alkalmaz.

2.1.2 2. Az operációs rendszer mint absztrakciós réteg

A második nézőpont szerint az operációs rendszer legfontosabb feladata, hogy a nyers hardvert **egységes, kényelmes, hordozható felületté** alakítsa. A felhasználói program számára nem az a fontos, hogy a merevlemez feje éppen melyik sávon áll, vagy hogy a hálózati kártya melyik regiszterébe kell írni. A program egy **fájlt** szeretne olvasni, egy **socketet** szeretne nyitni, egy **folyamatot** szeretne indítani. Az OS ezeket a magas szintű fogalmakat biztosítja.

Az absztrakció lényege, hogy a hardver egyenetlenségeit, gyártóspecifikus részleteit és alacsony szintű protokolljait egy közös, jól definiált interfész mögé rejti. Ezt az interfészt nevezzük rendszerhívási felületnek (system call interface). A rendszerhívási felület az a határ, amelyen keresztül a felhasználói program belép a kernel által nyújtott szolgáltatásokba.

Nézzünk egy konkrét példát. Tegyük fel, hogy egy C program egy fájlból akar olvasni. A programozó a következőt írja:

```
/* szemléltető példa, nem végrehajtandó utasítás éles gépen */
#include <unistd.h>
#include <fcntl.h>

int fd;
ssize_t n;
char buf[1024];

fd = open("/srv/emulated/example.dat", O_RDONLY);
if (fd < 0) {
    return -1;
}
n = read(fd, buf, sizeof(buf));
close(fd);
```

Ebben a példában `open`, `read` és `close` rendszerhívások. A programozó nem tudja, hogy a `/srv/emulated/example.dat` fájl egy emulált NVMe lemezen, egy hálózati fájlrendszeren vagy egy RAM-ban élő tmpfs-en található. A kernel **VFS** (Virtual File System) rétege elvonatkoztatja a konkrét fájlrendszer sajátosságait. Az `open` hívás elvégzi a jogosultságellenőrzést, megkeresi az inode-ot, létrehoz egy nyitott fájl leírót a folyamat táblájában, és visszaad egy egész számot, a fájlleírót. A `read` hívás a `buf` nevű pufferbe másolja a kért adatot, a `close` pedig felszabadítja a leíróhoz tartozó kernelbeli erőforrásokat.

Az absztrakció nem csak a fájlokra vonatkozik. Az operációs rendszer absztrakciói közé tartoznak:

- **Folyamat** – a futó program elszigetelt példánya, saját címtérrel, ütemezési állapottal, szülő-gyermek kapcsolattal.
- **Fájl** – névvel ellátott, soros vagy véletlen hozzáférésű adatfolyam, amely lehet valódi lemezadaton, csővezetéken, socketen vagy eszközön.
- **Címtér** – a folyamat számára látható virtuális memóriatartomány, amely a fizikai memóriától független, folyamatos tartománynak látszik.
- **Socket** – hálózati kommunikációs végpont, amely a TCP/IP protokoll részleteit rejti.
- **Szignál** – aszinkron értesítés, amely egy folyamatnak jelzi, hogy valamilyen esemény történt (például `SIGINT` a megszakításnál).

Az absztrakciós réteg ereje abban áll, hogy a felhasználói programok **forráskód-szintű hordozhatóságot** kapnak: ugyanaz a C program lefordul és fut egy x86_64 szerveren, egy ARM64 alapú beágyazott eszközön és egy RISC-V emulátorban, feltéve, hogy a rendszerhívásokat az adott platform libc-e megvalósítja. Az absztrakció természetesen nem ingyenes: minden rendszerhívás belépést jelent a kernelbe, ami idő- és CPU-ciklus költséggel jár. Ezt a költséget a későbbi fejezetek a rendszerhívások mérésekor részletesen elemzik.

Az absztrakciós nézőpont az oka annak is, hogy az operációs rendszer **szolgáltatásait** meg kell különböztetnünk a **politikájától**. A szolgáltatás például az, hogy a lemezt blokkokra bontva olvassuk. A politika az, hogy a blokkokat milyen sorrendben olvassuk be előre, hogyan gyorsítótárazzuk, és milyen ütemezőt használunk a kérések rendezésére. A szolgáltatás az absztrakció, a politika az erőforrás-kezelés.

2.1.3 3. Az operációs rendszer mint virtuális gép

A harmadik nézőpont azt emeli ki, hogy az operációs rendszer minden folyamat számára egy **saját, elszigetelt, kényelmesen használható számítógép látszatát** kelti. Ezt a látszatot nevezzük virtuális gépnek. A virtuális gép nem feltétlenül a hipervizoroknál megszokott, teljes hardvert emuláló virtuális gépre utal; itt a fogalom tágabb: a kernel a folyamat számára **virtuális CPU-t, virtuális memóriát és virtuális eszközöket** biztosít.

A **virtuális CPU** a folyamat számára azt a látszatot kelti, hogy kizárólag ő fut a processzoron. Valójában a kernel időosztásos ütemezést alkalmaz: a folyamat rövid időszeletekben fut, majd az ütemező leállítja, elmenti a kontextusát, és egy másik folyamatot indít. A folyamat ebből semmit sem érzékel. Az általa olvasott utasításszámláló, regisztertartalom és veremmutató az ő saját virtuális CPU-állapota.

A **virtuális memória** minden folyamat számára egy folyamatos, 0 címtől induló, nagy méretű címtartományt biztosít. A fizikai memóriát a kernel oldalakra bontva, tetszőlegesen szétszórva rendeli hozzá. A folyamat úgy érzi, hogy a memóriája egybefüggő, miközben a háttérben a lapozó a fizikai kereteket tetszőlegesen osztja ki, sőt, egyes lapokat akár lemezre is lapozhat. Ez a virtualizáció teszi lehetővé, hogy két folyamat ugyanazon a virtuális címen (például `0x00400000`) különböző fizikai tartalmakat lásson, és semmilyen körülmények között ne férhessen hozzá egymás adataihoz.

A **virtuális eszközök** elve szerint az operációs rendszer minden erőforrást egy egységes, erőforrás-specifikus felületen mutat be. A nyomtató úgy jelenik meg, mint egy karakteres eszköz; a merevlemez úgy, mint egy blokkeszköz; a hálózati interfész úgy, mint egy socketeket szolgáltató eszköz. Az alkalmazás nem látja a tényleges hardvermegszakításokat, a DMA-átvitelt, a buszprotokollt vagy a vezérlő-regisztereket. A kernel eszközmeghajtó rétege az, amelyik ezeket a részleteket kezeli.

A virtuális gép nézőpont különösen fontos a **többfeladatosság** (multitasking) és az **elszigetelés** (isolation) megértéséhez. Ha az operációs rendszer nem virtuális gépet nyújtana, akkor minden

folyamatnak közvetlenül kellene ismernie a fizikai memóriát, a fizikai regisztereket és a DMA-vezérlőt. Egyetlen hibás vagy rosszindulatú folyamat az egész rendszert tönkretehetné. A virtuális gép ehelyett minden folyamatot a többitől elválasztott, saját erőforrásokkal ellátott homokozóba helyez.

A virtuális gép nézőpontot megkülönböztetjük a **virtualizáció** (hypervisor-alapú vagy konténeres rendszervirtualizáció) fogalmától. A kettő rokon, de nem azonos. Itt, az operációs rendszer belső vizsgálatakor, a virtuális gép a kernel által az egyes folyamatoknak nyújtott absztrahált hardverre vonatkozik. A hypervisor, például a KVM vagy a Xen, a teljes fizikai gép szintjén nyújt virtuális gépet egy vendég operációs rendszer számára. A későbbi fejezetek részletesen tárgyalják a hypervisorok és a konténerek felépítését.

2.1.4 4. A három nézőpont együttes működése

A három nézőpont ugyanannak az operációs rendszernek a működését írja le, csak más-más kérdésre fókuszál. Az alábbi táblázat összefoglalja a három nézőpont közötti különbségeket és kapcsolatukat:

Nézőpont	Központi kérdés	Fő tevékenység	Jellemző példa
Erőforrás-kezelő	Ki kapja meg az erőforrást, mikor és mennyit?	Ütemezés, foglalás, elszámolás	CPU-idő osztása ütemezővel; memória-lapok kiosztása
Absztrakciós réteg	Hogyan rejtsük el a hardver részleteit?	Rendszerhívások, VFS, socket API	`open`, `read`, `write`, `socket`, `fork`
Virtuális gép	Hogyan adjunk minden folyamatnak saját gép látszatát?	Címtér-izoláció, kontextusváltás, virtuális eszközök	Virtuális memória, virtuális CPU, fájlleírók

Egyetlen konkrét művelet, például a `read` rendszerhívás, mindhárom nézőpontból értelmezhető:

- **Erőforrás-kezelőként** a kernel eldönti, hogy az adott háttértár-vezérlő kéréssorában hová kerüljön a beolvasandó blokkok kérése, és mikor hajtsa végre.
- **Absztrakcióként** a `read` hívás elrejti, hogy a fájl milyen fizikai tárolón, milyen blokkszerkezetben, milyen hibajavítással van tárolva.
- **Virtuális gépként** a `read` hívás a folyamat számára azt a látszatot kelti, hogy a fájl az ő saját erőforrása, és a pufferbe olvasás közben nem kell törődnie más folyamatokkal.

2.1.5 5. Kernel és operációs rendszer

Az eddigiekben a „kernel” és az „operációs rendszer” kifejezéseket részben egymás mellett használtuk. Itt az ideje a pontos megkülönböztetésnek.

A **kernel** az operációs rendszernek az a része, amely **magasabb privilégiumú, kernel módban** fut, és közvetlenül hozzáfér a hardver erőforrásaihoz. A kernel tartalmazza az ütemezőt, a memóriakezelőt, a megszakításkezelőt, az eszközmeghajtók magját, a VFS-t, a hálózati stacket és a rendszerhívások kiszolgáló oldalát. A kernel az, amelyik a virtuális gép látszatát megvalósítja, és az erőforrás-kezelési politikákat betartatja.

Az **operációs rendszer** ennél többet jelent. Az OS magában foglalja a kernelt **és** a felhasználói tér azon programjait, amelyek a rendszer alapvető részét képezik: a rendszerindító eszközöket,

a shellt, az init rendszert, a daemonokat, a rendszerkönyvtárakat, a hitelesítési alrendszert, a naplózást, a felhasználói felületet és a dokumentációt. A kernel nélkül ezek a programok nem tudnának futni; a felhasználói tér programjai nélkül a kernel használhatatlan lenne, mert nem lenne, ami a felhasználó kéréseit megfogalmazza és a rendszerhívásokat meghívja.

Egy gyakori hasonlat szerint a kernel az autó motorja, a felhasználói tér a kormány, a pedálok, a műszerfal és az utastér. A motor nélkül az autó nem mozog, de motorból és vázból még senki nem utazott sehová. A teljes operációs rendszer a kernel és a felhasználói tér együttese.

Fontos megjegyezni, hogy a **kernel és a felhasználói tér közötti határ** nem esztétikai, hanem biztonsági és megbízhatósági kérdés. A kernel közvetlenül írhat a memóriavezérlő regisztereibe, átírhatja az MMU lapasztalait, és letilthatja a megszakításokat. A felhasználói program ezt csak a kernel által ellenőrzött, korlátozott rendszerhívásokon keresztül teheti meg. Ha egy felhasználói program hibázik, legfeljebb a saját folyamatát dönti le. Ha a kernel hibázik, az egész rendszer összeomolhat. Ezért a kernelkódnak sokkal szigorúbb memória- és erőforrás-tulajdonjogi, hibakezelési és versenyhelyzeti szabályoknak kell megfelelnie.

2.1.6 6. Gyakorlati példa: egy egyszerű rendszer OS-funkcióinak leltározása

A fejezet céljai között szerepel, hogy a három nézőpontot egy kis, ellenőrizhető példán alkalmazzuk. Az alábbi labor célja egy emulált mini-OS funkcióinak leltározása. A labor **nem** igényel tényleges kernelkód-írást; elegendő, ha az olvasó egy emulált környezetben elindít egy minimalista kernelt, és a naplója alapján azonosítja, hogy az egyes funkciók melyik nézőponthoz tartoznak.

2.1.7 6.1. Kiinduló állapot és konfiguráció

A labor az alábbi emulált környezetben fut:

- QEMU emulátor, verzió: `qemu-system-x86_64` (a pontos verziót a reprodukciós csomag tartalmazza)
- CPU: 1 virtmagos x86_64, `-cpu qemu64` beállítással
- Memória: 128 MiB
- Háttértár: egy 64 MiB méretű, eldobható, emulált IDE lemez (`-drive file=emulated.img,format=raw`)
- Nincs valós hálózati interfész; a konzol a QEMU soros portján keresztül érhető el
- A hoszt valódi lemezét, bootloaderét és rendszerpartícióját tilos módosítani

A kiinduló állapot egy olyan mini-kernel, amely a boot után kiír egy üzenetet, majd egy ideiglenes RAM-lemezen létrehoz egy könyvtárat és egy fájlt, elindít egy felhasználói módú segédprogramot, és várakozik a soros porton.

2.1.8 6.2. Build és futtatás

A példa buildeléséhez a következő parancsokat használjuk (a konkrét útvonalak és verziók a reprodukciós csomagban rögzítettek):

```
# minden parancs az emulált labor gépen, nem a hoszt rendszerpartíción fut
make -C minimal-os clean
make -C minimal-os
qemu-system-x86_64 \
  -m 128M \
  -cpu qemu64 \
  -drive file=emulated.img,format=raw \
  -nographic \
```

```
-no-reboot \
-kernel minimal-os/kernel.bin
```

A `kernel.bin` a mini-kernel lefordított binárisa. A `-nographic` kapcsoló a soros konzolra irányítja a kimenetet. A `-no-reboot` megakadályozza, hogy a QEMU hibánál automatikusan újrainduljon.

2.1.9 6.3. Elvárt eredmény és mérési napló

A rendszer a boot után az alábbi naplóhoz hasonló üzeneteket írja ki (a példában a sorok száma és a hexadecimális címek illusztratívak):

```
[0x0000] boot: multiboot header found at 0x00100000
[0x0001] mm: physical memory 0x00000000-0x07ffffff (128 MiB)
[0x0002] mm: kernel heap initialized at 0x00110000
[0x0003] sched: scheduler started, quantum 5 ms
[0x0004] fs: ramfs mounted at /srv/emulated
[0x0005] fs: created file /srv/emulated/example.dat (32 bytes)
[0x0006] syscall: register syscall table at 0x00108000
[0x0007] task: starting user process 'inventory'
[0x0008] syscall: write(1, "inventory: OK\n", 14)
[0x0009] task: process 'inventory' exited with 0
```

Az elvárt eredmény az, hogy a rendszer a fenti lépéseket hiba nélkül végrehajtja, és a konzolon megjelenik a `inventory: OK` sor. A mérési napló a következőket rögzíti:

Lépés	Elvárt eredmény	Mért eredmény	Megjegyzés
Boot	Multiboot header az `0x00100000` címen	`0x00100000`	QEMU alapértelmezett betöltési címe
Fizikai memória	128 MiB tartomány `0x00000000-0x07ffffff`	`0x00000000-0x07ffffff`	`-m 128M` kapcsoló
Kernel heap	`0x00110000` kezdet	`0x00110000`	A linkerscriptben rögzített
Scheduler	5 ms időszelet	5 ms	Az ütemező konfigurációjában
RAMFS	Mount az `/srv/emulated` ponton	`/srv/emulated`	Helyőrző útvonal, nem éles rendszer
Fájl	`example.dat` mérete 32 bájt	32 bájt	A tesztfájl tartalma szintetikus
Rendszerhívás-tábla	`0x00108000` cím	`0x00108000`	A kernel linkerscriptje szerint
Felhasználói folyamat	`inventory` kilépési kód 0	0	A `write` rendszerhívás sikeres

A táblázatban a **mért** és az **elvárt** eredményt külön oszlopban tüntettük fel. A labor szabályai szerint csak a ténylegesen lefuttatott, naplózott értékek kerülhetnek a mért oszlopba. Ha egy értéket csak a linkerscriptből vagy a forráskódból következtetünk ki, azt az elvárt oszlopban kell jelölni.

2.1.10 6.4. A leltár: melyik funkció melyik nézőponthoz tartozik?

A labor második feladata a mini-OS funkcióinak besorolása a három nézőpontba:

- **Erőforrás-kezelői funkciók:** a fizikai memória felismerése (`mm: physical memory`), a kernel heap inicializálása, a scheduler elindítása, a felhasználói folyamat leállítása és kilépési kódjának visszajelzése. Ezek a lépések mind azt szolgálják, hogy a CPU-időt, a memóriát és az erőforrásokat egy meghatározott politika szerint ossza el a kernel.
- **Absztrakciós funkciók:** a RAMFS mountolása, a fájl létrehozása, a rendszerhívás-tábla regisztrálása, a `write` rendszerhívás. Az `inventory` folyamat számára a RAMFS egy közönséges fájlrendszernek látszik, pedig fizikailag a RAM-ban él. A `write` hívás elrejti a soros port kezelésének részleteit.
- **Virtuális gép funkciók:** a felhasználói folyamat `inventory` elindítása, amely saját virtuális címtérrel rendelkezik, és nem látja a kernel memóriáját. A folyamat úgy fut, mintha egyedül lenne a gépen, pedig a kernel ütemezője közben a soros port megszakításait is kezeli.

A leltárt a laborjegyzetben az alábbi táblázatos szerkezetben érdemes elkészíteni (a példa a fenti mini-OS-en alapul):

Naplósor	Funkció	Nézőpont	Indoklás
`mm: physical memory`	Fizikai memória felismerése	Erőforrás-kezelő	A rendelkezésre álló erőforrás felmérése a kiosztás előtt
`mm: kernel heap initialized`	Kernel heap létrehozása	Erőforrás-kezelő	A kernel saját memóriafoglalásainak alapja
`sched: scheduler started`	Ütemező indítása	Erőforrás-kezelő	A CPU-idő elosztásának mechanizmusa
`fs: ramfs mounted`	RAMFS mountolása	Absztrakció	A RAM-ot fájlrendszerként mutatja a folyamatoknak
`fs: created file`	Fájl létrehozása	Absztrakció	A tárolást névvel ellátott, logikai fájlként jeleníti meg
`syscall: register syscall table`	Rendszerhívás-tábla regisztrálása	Absztrakció	A felhasználói programok belépési pontját definiálja
`task: starting user process`	Felhasználói folyamat indítása	Virtuális gép	Saját címtér, saját futási kontextus
`syscall: write`	Konzolra írás	Absztrakció + Virtuális gép	Elrejti a soros portot, és a folyamat saját eszközeként jeleníti meg
`task: process exited`	Kilépési kód visszaadása	Erőforrás-kezelő	A folyamat erőforrásainak felszabadítása

2.1.11 7. Tipikus hibák és félreértések

Az operációs rendszer három nézőpontjának tanulásakor a leggyakoribb hiba az, hogy a három nézőpontot egymást kizáró kategóriaként kezeljük. A valóságban minden rendszerhívás, minden kernelmodul és minden eszközmeghajtó egyszerre értelmezhető mindhárom nézőpontból. Az, hogy egy adott funkciót melyik nézőpontból vizsgálunk, az elemzés céljától függ.

A második gyakori hiba a **kernel és az operációs rendszer** szinonimaként való használata. A kernel az operációs rendszer magja, de nem az egész. A Linux esetén például a kernel a `arch/`, a `mm/`, a `kernel/`, a `net/` és az `fs/` könyvtárakban található, de a teljes Linux operációs rendszerhez hozzátartozik a GNU libc, a systemd vagy más init, a shell, a magas szintű hálózati eszközök és a dokumentáció.

A harmadik gyakori hiba az **absztrakció és a virtualizáció összemosása**. Az absztrakció az interfész egyszerűsítését jelenti: a `read` hívás elrejti a lemezműveletek részleteit. A virtualizáció ezzel szemben az erőforrás **megsokszorozásának látszatát** jelenti: minden folyamatnak saját címtér jár, mintha mindegyiknek külön gépe lenne. A kettő rokon, de a hangsúly más.

A negyedik gyakori hiba az **erőforrás-kezelés és a virtualizáció** durva leegyszerűsítése. Egyes források azt sugallják, hogy az erőforrás-kezelő csak a CPU-időre figyel, a virtuális gép pedig csak a memóriára. Valójában minden erőforrás virtuális gépként jelenik meg: a hálózati socket, a fájlleíró, a pipe, a jelzés, sőt a processzort idő is (virtuális CPU). A három nézőpont minden erőforrástípusra alkalmazható.

2.1.12 8. Összefoglalás és önellenőrző kérdés

Az operációs rendszer három alapvető nézőpontja:

1. **Erőforrás-kezelő:** az OS elosztja a fizikai erőforrásokat a folyamatok között, méltányos és védett módon.
2. **Absztrakciós réteg:** az OS rendszerhívásokon keresztül egységes, hordozható felületet nyújt a hardverhez.
3. **Virtuális gép:** az OS minden folyamatnak saját, elszigetelt gép látszatát adja.

A **kernel** ezeknek a funkcióknak a privilégiumozott magját valósítja meg, míg az **operációs rendszer** a kernelt és a felhasználói tér alapvető szolgáltatásait együtt jelenti.

Önellenőrző kérdés: Tegyük fel, hogy egy folyamat megnyit egy hálózati socketet, és adatokat küld rajta. Magyarázza el, hogy ez a művelet hogyan értelmezhető az erőforrás-kezelő, az absztrakciós réteg és a virtuális gép nézőpontjából! Konkrétan: melyik nézőpont magyarázza azt, hogy a folyamat nem látja a hálózati kártya megszakításait? Melyik nézőpont magyarázza azt, hogy a kártya sávszélességét több folyamatnak is el kell osztani? Melyik nézőpont magyarázza azt, hogy a folyamat úgy érezheti, hogy a socket az ő saját, privát kommunikációs csatornája?

2.2 Az OS szerepe és helye a rendszerben

Az operációs rendszer (OS) szerepét akkor értjük meg pontosan, ha nem önmagában vizsgáljuk, hanem elhelyezzük a teljes számítógépes rendszerben. Az OS egyszerre három irányban létesít kapcsolatot: lefelé a hardver felé, fölfelé a felhasználói programok felé, valamint oldalirányban a hálózat és más rendszerek felé. Ebben a fejezetben ezt a három kapcsolatot, a köztes rétegeket (hardver, firmware, kernel, userland, alkalmazás), valamint az OS kettős természetét – platform és szolgáltató – járjuk körül. A cél nem csupán az elméleti leírás, hanem egy olyan mentális

modell kialakítása, amellyel a későbbi fejezetekben a folyamatkezelést, a memóriakezelést, az I/O-t és a hálózatot is pontosan be tudjuk illeszteni.

2.2.1 Az operációs rendszer három kapcsolata

Egy általános célú számítógép legalább négy jól elkülöníthető tartományból áll: hardver, firmware, operációs rendszer (kernel és felhasználói tér), valamint alkalmazások. Az OS a középső rétegben helyezkedik el, és három alapvető szerepet tölt be:

1. **Erőforrás-kezelő a hardver felé.** A processzor, a memória, a háttértárak, a hálózati interfészek és a perifériák osztoznak a versengő felhasználói programokon. Az OS dönt arról, hogy melyik folyamat melyik erőforrást, mennyi ideig és milyen feltételekkel használhatja.
2. **Absztrakciós réteg a felhasználói programok felé.** Az alkalmazások nem közvetlenül a hardvert vezérlik, hanem egységes, magas szintű interfészeket (fájlok, socketek, folyamatok, virtuális memória) látnak. Ez a programozói modell hordozhatóvá és biztonságossá teszi a szoftvert.
3. **Szolgáltató a hálózat felé.** A modern operációs rendszerek beépített hálózati protokollkészlettel, tűzfal-szabályokkal, névfeloldással és távoli eljáráshívási mechanizmusokkal rendelkeznek. Az OS nemcsak a helyi alkalmazások számára nyújt szolgáltatásokat, hanem a hálózaton keresztül más rendszereknek is.

Ez a hármas feladatkör határozza meg az OS architektúráját: a kernel a hardver feletti legalsó szoftverréteg, amely privilegizált utasításokat hajthat végre, míg a felhasználói térben futó programok korlátozott jogosultsággal rendelkeznek.

2.2.2 A hardver és az operációs rendszer kapcsolata

A hardver fizikai erőforrásai – CPU, memória, I/O-eszközök, időzítők, megszakításvezérlők – önmagukban nem képesek a biztonságos és hatékony megosztásra. Az OS az alábbi mechanizmusokon keresztül kapcsolódik hozzájuk:

- **Privilegiszintek és védelmi gyűrűk.** A processzor legalább két működési módot különböztet meg: kernel mód (privilegizált, 0. gyűrű x86-on, M-mode vagy S-mode RISC-V-n) és felhasználói mód (nem privilegizált). A kernel módban futó kód hozzáfér a speciális regiszterekhez, a memóriavezérlőhöz, a megszakításokhoz; a felhasználói módban futó kód nem. Az átlépést hardveres utasítások, például `syscall` (x86-64), `svc` (ARM) vagy `ecall` (RISC-V) teszik lehetővé.
- **Megszakítások és kivételek.** A hardver aszinkron eseményekkel (interrupt) jelzi az OS-nek, ha egy eszköz befejezte a műveletet, ha időzítő járt le, vagy ha hibás memóriahozzáférés történt. A kernel megszakításkezelő rutinokat (interrupt handler, ISR) telepít, és ezek futása során konzisztens állapotot tart fenn.
- **Firmware és boot.** A hardver bekapcsolásakor a firmware (BIOS vagy UEFI) inicializálja az alapvető komponenseket, elvégzi a beépített önellenőrzést (POST), majd betölti a bootloadert. A bootloader a kernel képét a memóriába tölti, beállítja a kezdeti környezetet, és átadja a vezérlést a kernel belépési pontjának. Ezt a folyamatot részletesen a 4. fejezet tárgyalja; itt elég annyi, hogy a firmware szolgáltatásai (UEFI boot services, runtime services) képezik az első szoftveres hidat a hardver és az OS között.

- **Eszközök és memórialeképezés.** A modern rendszerekben a perifériák gyakran memóriába leképzett I/O-t (MMIO) használnak: az eszköz regiszterei a processzor címterében jelennek meg, így a kernel egyszerű memória-hozzáférésekkel vezérelheti őket. A x86-os port I/O (in/out utasítások) alternatív, de ritkábban használt. A kernel eszközmeghajtói (driver) ismerik az adott hardver pontos protokollját, és ezt az ismeretet elrejtik a felsőbb rétegek elől.

A hardver és az OS közötti kapcsolat tehát szigorúan szabályozott: a hardver biztosítja a védelmi mechanizmusokat, az OS pedig azokat a szabályokat és algoritmusokat, amelyek a hardver erőforrásait megoszthatóvá teszik.

2.2.3 Az operációs rendszer és a felhasználói programok kapcsolata

A felhasználói programok – legyen szó parancssori eszközről, webkiszolgálóról vagy beágyazott vezérlőről – soha nem férnek hozzá közvetlenül a hardverhez. Ehelyett az OS által biztosított **rendszerhívásokon** (system call) keresztül kérik a szolgáltatásokat. Egy tipikus rendszerhívás így nézhet ki:

```c
/* Példa: fájl megnyitása (szemléltetés, nem futtatandó) */
int fd = open("/tmp/example.txt", O_RDONLY);
if (fd == -1) {
    perror("open");
    return 1;
}
```

Ez a C nyelvű kód valójában a libc könyvtár `open()` függvényét hívja, amely a megfelelő regiszterekbe helyezi a rendszerhívás számát és paramétereit, majd végrehajt egy `syscall` utasítást. A processzor átvált kernel módba, a kernel kiolvassa a kérést, ellenőrzi a hívó jogosultságait, végrehajtja a műveletet, majd visszatér a felhasználói módba. A felhasználói program számára mindez láthatatlan; csupán annyit érzékel, hogy a `open()` függvény visszatért egy fájlleíróval vagy hibakóddal.

A rendszerhívások alkotják az **API** (Application Programming Interface) határfelületet, míg az **ABI** (Application Binary Interface) a bináris kompatibilitást írja le: milyen regiszterekben hogyan kell átadni a paramétereket, hogyan kell visszatérni, milyen a verem elrendezése. A stabil ABI teszi lehetővé, hogy ugyanaz a lefordított program fusson az adott operációs rendszer különböző verzióin.

A felhasználói térben futó programok további komponensei:

- **Folyamatok és szálak.** Az OS minden futó programot egy folyamatként (process) kezel, amely saját virtuális címteret, fájlleíró-táblát, veremterületet és egyéb erőforrásokat kap. A szálak (thread) ugyanazon folyamaton belül osztoznak a címtereken, de külön veremmel és regiszterkészlettel rendelkeznek.
- **Könyvtárak és a libc.** A legtöbb program nem közvetlenül hívja a rendszerhívásokat, hanem a standard C könyvtáron (például glibc, musl) keresztül, amely kényelmesebb interfészt, hibakezelést és gyakran pufferezést biztosít.
- **A felhasználói tér további elemei:** a shell, a daemonok, az init rendszer, a grafikus felület, a konfigurációs fájlok. Ezek mind az OS által nyújtott szolgáltatásokra épülnek, de nem részei a kernelnek.

A felhasználói programok tehát nem a hardverrel, hanem az OS absztrakcióival dolgoznak: fájlokat olvasnak, processzeket indítanak, hálózati socketeket nyitnak. Ez a modell biztosítja a hordozhatóságot és a védelmet.

2.2.4 Az operációs rendszer és a hálózat kapcsolata

A modern operációs rendszerek szerves részét képezi a hálózati protokollkészlet. A kernel tartalmazza a TCP/IP implementációt, a hálózati interfészek meghajtóit, a csomagszűrő mechanizmusokat (netfilter, eBPF), valamint a socket API-t, amely a felhasználói programok számára teszi elérhetővé a hálózati kommunikációt.

Egy hálózati kapcsolat létrehozása és használata a következő rétegeken halad át:

1. Az alkalmazás létrehoz egy socketet: `socket(AF_INET, SOCK_STREAM, 0)`.
2. A kernel kioszt egy fájlleírót, és hozzárendeli a socket struktúrát.
3. Az alkalmazás csatlakozik egy távoli címhez: `connect()`.
4. A kernel TCP-kézfogást hajt végre, majd az adatokat a hálózati interfészen keresztül továbbítja.
5. A beérkező csomagok a megszakításkezelőn és a hálózati stacken keresztül eljutnak a socket puffereihez, ahonnan az alkalmazás `read()` vagy `recv()` hívással kiolvashatja őket.

Az OS tehát a hálózat felé is szolgáltatóként lép fel: kezeli a csomagok ütemezését, a torlódásvezérlést, a tűzfalszabályokat, a hálózati címfordítást (NAT), és biztosítja a különböző folyamatok közötti izolációt. Emellett az OS maga is lehet hálózati kliens vagy szerver: például NFS-en keresztül fájlrendszereket importál, DHCP-vel konfigurálódik, DNS-t kérdez le.

A hálózati kapcsolat tehát nem csupán egy periféria kezelése, hanem a rendszer egyik legösszetettebb és legbiztonságkritikusabb alrendszere. A későbbi fejezetek (18–21.) részletesen tárgyalják; itt elegendő annyi, hogy az OS a hálózati stacket a felhasználói programok elől elrejti, és egységes socket interfészt kínál.

2.2.5 Rétegek: hardver, firmware, kernel, userland, alkalmazás

A fenti kapcsolatokat érdemes rétegmodellben is ábrázolni. Egy általános célú rendszer tipikus rétegződése alulról fölfelé:

Réteg	Funkció	Példák
Hardver	Fizikai erőforrások, végrehajtás, megszakítások	CPU, RAM, SSD, NIC, UART
Firmware	Korai inicializálás, boot, hardverleírás	UEFI, BIOS, device tree, ACPI
Kernel	Erőforrás-kezelés, védelmi mechanizmusok, szolgáltatások	ütemező, memóriakezelő, VFS, hálózati stack
Userland (rendszerkönyvtárak)	Rendszerhívások burkolása, szabványos interfész	libc, libpthread, libm, libdl
Alkalmazás	Felhasználói logika, üzleti folyamatok	shell, editor, böngésző, adatbázis

Ez a rétegzés nem merev: például egy beágyazott rendszerben a firmware és a kernel közötti határ elmosódhat, egy mikrokernel rendszerben pedig a hálózati stack nagy része userlandben futhat. A lényeg a felelősségi körök szétválasztása: minden réteg csak a közvetlenül alatta lévő réteg szolgáltatásaira épít, és a fölötte lévőnek nyújt interfészt.

2.2.6 Egy rendszerhívás útja

Vegyünk egy konkrét példát: egy felhasználói program megnyit egy fájlt.

1. Az alkalmazás meghívja a libc `open()` függvényét.
2. A libc a rendszerhívás számát (x86-64 Linuxon 2 az `open` esetén) és a paramétereket a megfelelő regiszterekbe tölti (`rax`, `rdi`, `rsi`, `rdx` stb.), majd végrehajtja a `syscall` utasítást.
3. A processzor kernel módba vált, és a kernel rendszerhívás-belépési pontjára ugrik.
4. A kernel lementi a felhasználói regisztereket, ellenőrzi a hívás számát és a paraméterek érvényességét.
5. A kernel meghívja a VFS (virtuális fájlrendszer) `open` implementációját, amely megkeresi a megfelelő fájlrendszer-illesztőt.
6. A fájlrendszer-illesztő a blokk-eszköz meghajtóján keresztül olvas, vagy a gyorsítótárból dolgozik.
7. A kernel lefoglal egy fájlleírót a folyamat táblájában, és visszatér a felhasználói módba.
8. A libc visszaadja a fájlleírót vagy a hibakódot az alkalmazásnak.

Ez az út jól mutatja, hogy az OS mint **platform** és mint **szolgáltató** egyszerre működik: platform, mert lehetővé teszi a program futtatását és a fájl elérését; szolgáltató, mert a fájlrendszer, a blokk-eszköz és a VFS mind szolgáltatásokat nyújtanak a program számára.

2.2.7 Az operációs rendszer mint platform

Az OS platformként egy olyan végrehajtási környezetet biztosít, amelyben a felhasználói programok biztonságosan és hatékonyan futhatnak. Ez a platform a következőket foglalja magában:

- **Folyamat- és szálkezelés:** a programok elindítása, leállítása, ütemezése, szinkronizációja.
- **Virtuális memória:** minden folyamat saját, izolált címteret kap, így nem tudja felülírni más folyamat memóriáját.
- **Fájlrendszer:** a tartós adattárolás egységes, fa-szerkezetű névteret kap.
- **Hálózati kommunikáció:** socketek és protokollok, amelyeken keresztül a programok adatokat cserélhetnek.
- **Erőforrás-elszámolás:** az OS nyilvántartja, hogy melyik folyamat mennyi CPU-időt, memóriát, I/O-t fogyaszt, és korlátozhatja azokat.

A platform szerep lényege az **absztrakció**: a programozó nem az adott hardver sajátosságaival foglalkozik, hanem egységes, jól dokumentált interfészekkel. Ugyanaz a C program lefordítva futhat egy szerveren, egy asztali gépen vagy egy beágyazott eszközön, ha az operációs rendszer ugyanazt az API-t és ABI-t biztosítja (természetesen a hardveres különbségek, mint az utasításkészlet, a fordító által hidaltak).

2.2.8 Az operációs rendszer mint szolgáltató

Szolgáltatóként az OS aktívan menedzseli a rendszer erőforrásait, és ezeket a felhasználói programok rendelkezésére bocsátja. A szolgáltatások közé tartozik:

- **Fájlműveletek:** létrehozás, olvasás, írás, törlés, átnevezés, jogosultság-ellenőrzés.
- **Folyamatok közötti kommunikáció (IPC):** pipe, üzenetsor, osztott memória, szignálok, socketek.
- **Időzítés és várakozás:** `sleep`, `nanosleep`, időzítők, riasztások.

- **Hálózati szolgáltatások:** TCP/UDP kapcsolatok, DNS feloldás, tűzfal-szabályok.
- **Memóriafoglalás:** dinamikus memória (`malloc`/`mmap`), virtuális címterek kezelése.

Ezek a szolgáltatások nem passzív könyvtárak, hanem a kernel aktív közreműködésével valósulnak meg. A kernel például a fájlírás során foglalkozik a gyorsítótárazással, a naplózással, a lemezütemezéssel; a socketküldésnél a csomagok ütemezésével, a torlódásvezérléssel, a hálózati megszakításokkal.

Fontos megkülönböztetni a **szolgáltatás** és a **mechanizmus** fogalmát. A szolgáltatás az, amit a program kér (például „olvasd be ezt a fájlt"), míg a mechanizmus az, ahogyan a kernel ezt megvalósítja (lapozás, DMA, gyorsítótár, ütemezés). A jó OS-tervezés elválasztja a kettőt: a rendszerhívások minimális, jól meghatározott interfészt képeznek, míg a belső mechanizmusok szabadon fejlődhetnek.

2.2.9 Tipikus hibák és helyes megoldások

A kezdő rendszermérnökök gyakran követnek el olyan hibákat, amelyek az OS és a többi réteg közötti kapcsolatok félreértéséből fakadnak. Nézzünk néhányat:

2.2.10 1. hiba: A felhasználói program közvetlenül hozzáfér a hardverhez

Hibás megközelítés: Az alkalmazás megpróbálja közvetlenül írni a merevlemez regisztereit, vagy a `/dev/mem` eszközön keresztül a fizikai memóriát manipulálni. Ezzel megkerüli az OS védelmi mechanizmusait, és a rendszer összeomlását vagy biztonsági rést okozhat.

Helyes megoldás: A program a fájlrendszeren vagy a blokk-eszközön keresztül, szabványos rendszerhívásokkal (`open`, `read`, `write`, `ioctl`) éri el az adatokat. Az OS gondoskodik a jogosultság-ellenőrzésről és a hardver absztrakciójáról.

2.2.11 2. hiba: A rendszerhívások paramétereinek ellenőrzésének elhagyása

Hibás megközelítés: A kernelben a rendszerhívás-kezelő nem ellenőrzi, hogy a felhasználó által átadott mutató a felhasználói címtartományba esik-e, vagy hogy a puffer mérete nem lépi-e túl a megengedettet. Ez biztonsági rést (például null pointer dereference, buffer overflow) eredményezhet.

Helyes megoldás: A kernel minden bemenő paramétert ellenőriz, a mutatókat a `copy_from_user` / `copy_to_user` mechanizmusokkal kezeli, és a hibákat `-EINVAL`, `-EFAULT` stb. hibakódokkal jelzi.

2.2.12 3. hiba: A hálózati stack megkerülése a teljesítmény érdekében

Hibás megközelítés: A fejlesztő a kernel hálózati rétegét megkerülve közvetlenül a hálózati kártya regisztereit írja egy felhasználói térben futó kernel-modulból vagy root joggal futó programból. Ez ugyan rövid távon nagyobb átvitelt eredményezhet, de elveszíti a tűzfal, az izoláció és a protokoll-ellenőrzés előnyeit.

Helyes megoldás: A kernel által támogatott gyorsított útvonalakat kell használni, például `AF_XDP`, `eBPF`, `io_uring` vagy a kernelbe épített offload mechanizmusokat. Így a teljesítmény nem megy a biztonság rovására.

2.2.13 4. hiba: Az OS réteg határainak összemosása

Hibás megközelítés: A fejlesztő a kernelben üzleti logikát valósít meg, például felhasználói hitelesítést vagy webes kiszolgálást. Ezzel a kernelt feleslegesen bonyolítja, és a hibák a teljes rendszert veszélyeztetik.

Helyes megoldás: A kernel csak az alapvető erőforrás-kezelést és védelmi mechanizmusokat valósítja meg. Az üzleti logika a felhasználói térben fut, és a kernel szolgáltatásait használja.

2.2.14 5. hiba: A firmware szerepének figyelmen kívül hagyása

Hibás megközelítés: A fejlesztő azt feltételezi, hogy a kernel betöltésekor minden hardver már inicializálva van, és nem veszi figyelembe a firmware által átadott hardverleírást (ACPI, device tree). Emiatt az eszközök nem működnek, vagy a rendszer instabil lesz.

Helyes megoldás: A kernel a firmware adatszerkezeteit felhasználva fedezi fel és konfigurálja az eszközöket. A laborgyakorlatokban mindig dokumentálni kell a firmware verzióját és a hardverleíró fájlokat.

2.2.15 Labor: rendszerkomponens-diagram készítése

A fejezet laborfeladata egy olyan rendszerkomponens-diagram elkészítése, amely bemutatja a hardver, firmware, kernel, userland és alkalmazás rétegek közötti kapcsolatokat. A diagramot reprodukálható környezetben kell előállítani, rögzített konfigurációval és mérési naplóval. A cél nem csupán a rajz, hanem a rétegek közötti összefüggések tudatosítása és ellenőrizhető dokumentálása.

2.2.16 Cél és elvárt eredmény

A diagramnak meg kell felelnie az alábbi követelményeknek:

1. Tartalmazza a következő rétegeket: hardver, firmware, kernel, userland, alkalmazás.
2. Minden réteghez adjon meg legalább három konkrét példaösszetevőt (például hardver: CPU, RAM, SSD; kernel: ütemező, virtuális memória, VFS).
3. Jelölje a rétegek közötti kapcsolatokat nyilakkal, és írja le, milyen interfészen keresztül kommunikálnak (rendszerhívás, megszakítás, library call, boot protocol).
4. Külön emelje ki az OS kettős szerepét: platform és szolgáltató.

Az elvárt eredmény: egy jól strukturált, olvasható diagram, amely a fenti követelményeket teljesíti.

2.2.17 Környezet és konfiguráció

A labor emulált környezetben, QEMU-val futtatott minimális Linux rendszeren, vagy akár egy helyi fejlesztőgépen, Jupyter notebook segítségével végezhető el. A reprodukálhatóság érdekében a következőket rögzítjük:

- **Hoszt operációs rendszer:** Ubuntu 24.04 LTS (x86_64), kernel 6.8.0-45-generic.
- **Emulált rendszer (opcionális):** QEMU 8.2, virtuális CPU: 1 db x86-64, memória: 1 GiB, háttértár: 10 GiB virtio-blk, hálózat: virtio-net.

- **Használt eszközök:** Python 3.12 Jupyter notebooktal, Graphviz 12.0 a diagram előállításához, vagy akár kézzel rajzolt diagram SVG/PNG formában.
- **Reprodukciós csomag:** a diagramot leíró forrás (DOT vagy Python script), a generált kép, valamint ez a mérési napló.

2.2.18 Munkafolyamat

1. **Készítsünk egy DOT forrásfájlt** (`system_diag.dot`), amely a rétegeket és kapcsolatokat írja le. Példa vázlat:

```
digraph OS {
    rankdir=BT;
    node [shape=box, style=filled, color=lightgrey];

    subgraph cluster_hardver {
        label="Hardver";
        cpu [label="CPU"];
        ram [label="RAM"];
        ssd [label="SSD"];
        nic [label="NIC"];
    }

    subgraph cluster_firmware {
        label="Firmware";
        uefi [label="UEFI/BIOS"];
        acpi [label="ACPI/Device Tree"];
    }

    subgraph cluster_kernel {
        label="Kernel";
        scheduler [label="Ütemező"];
        mm [label="Memóriakezelő"];
        vfs [label="VFS"];
        netstack [label="Hálózati stack"];
        driver [label="Eszközmeghajtók"];
    }

    subgraph cluster_userland {
        label="Userland";
        libc [label="libc"];
        shell [label="Shell"];
        init [label="Init/Daemonok"];
    }

    subgraph cluster_alkalmazas {
        label="Alkalmazás";
        editor [label="Editor"];
        browser [label="Böngésző"];
        db [label="Adatbázis"];
    }

    // Kapcsolatok
    uefi -> kernel [label="boot"];
    acpi -> kernel [label="hardverleírás"];
    kernel -> driver [label="belső API"];
    driver -> cpu [label="MMIO/Port I/O"];
    driver -> ram [label="MMU"];
    driver -> ssd [label="DMA"];
    driver -> nic [label="IRQ/DMA"];

    scheduler -> cpu [label="ütemezés"];
```

```
    mm -> ram [label="lapozás"];
    vfs -> ssd [label="blokk I/O"];
    netstack -> nic [label="csomagküldés"];

    kernel -> libc [label="rendszerhívás (syscall)"];
    libc -> shell [label="library call"];
    libc -> init [label="library call"];
    shell -> editor [label="fork/exec"];
    shell -> browser [label="fork/exec"];
    shell -> db [label="fork/exec"];

    editor -> vfs [label="open/read/write"];
    browser -> netstack [label="socket"];
    db -> vfs [label="fájl I/O"];
}
```

Ez a DOT fájl szemléltetés; a kapcsolatok címkéi az interfészekre utalnak. A laborgyakorlatban a hallgató saját maga is bővítheti.

2. **Generáljuk le a diagramot** a Graphviz `dot` parancsával:

```
dot -Tpng system_diag.dot -o system_diag.png
```

3. **Ellenőrizzük az eredményt** egy Python script segítségével, amely megszámolja a rétegeket, a csomópontokat és a kapcsolatokat, majd összehasonlítja az elvárt minimumokkal.

2.2.19 Mérési napló

A mérési naplóban a várt és a mért eredményeket külön kell feltüntetni. Példa táblázat:

Mérőszám	Várt (elvárt) érték	Mért érték	Megjegyzés
Rétegek száma	legalább 5	5	hardver, firmware, kernel, userland, alkalmazás
Csomópontok száma	legalább 15	18	minden rétegben 3+ példa
Kapcsolatok száma	legalább 10	14	interfészcímkékkel
Kernel–userland interfész	rendszerhívás	rendszerhívás	megfelel
OS mint platform/szolgáltató jelölés	explicit	implicit	a kernel csomópontok szerepéből következik

A mért értékek a fenti DOT fájl alapján számíthatók; ha a hallgató módosítja a fájlt, a naplóban frissíteni kell.

2.2.20 Önellenőrző kérdések

1. Miért nem éri el a felhasználói program közvetlenül a merevlemezt?
2. Milyen interfészen keresztül kommunikál a kernel a firmware-rel?
3. Hogyan jelenik meg az OS kettős szerepe a diagramon?

2.2.21 Összefoglalás

Az operációs rendszer a számítógépes rendszer központi rétege, amely lefelé a hardverrel, fölfelé a felhasználói programokkal, oldalirányban pedig a hálózattal áll kapcsolatban. Hardver és firmware biztosítják a fizikai alapokat és a korai inicializálást; a kernel privilegizált módban kezeli az erőforrásokat és szolgáltatásokat nyújt; a userland könyvtárak és az alkalmazások ezekre a szolgáltatásokra épülnek. Az OS egyszerre platform – mert futtatási környezetet ad – és szolgáltató – mert aktívan menedzseli az erőforrásokat. A rétegek közötti interfészek (rendszerhívás, megszakítás, boot protocol, library call) szigorúan szabályozottak, és a helyes tervezés kulcsa a felelősségi körök szétválasztása. A fejezet laborja egy rendszerkomponens-diagram elkészítésével segített ezt a modellt gyakorlatban is rögzíteni. A következő fejezet (Az operációs rendszerek története és evolúciója) az operációs rendszerek történetére és evolúciójára épít, amelyben ezek a fogalmak történeti kontextusba kerülnek.

2.3 Az operációs rendszerek története és evolúciója

Az operációs rendszerek evolúcióját nem lineáris, „egyre jobb” fejlődésként, hanem egymásra épülő, időnként visszatérő tervezési dilemmák sorozataként érdemes olvasni. A batch rendszerek megjelenése óta ugyanazok a kérdések térnek vissza más hardver- és felhasználói környezetben: hogyan osszuk meg az erőforrásokat, hogyan biztosítsunk izolációt, és milyen áron vásároljunk interaktivitást. A mainframe-ek kora óta megtett út nem csupán technikatörténet, hanem egy olyan fogalmi eszköztár kialakulása, amelyből a mai rendszerek is merítenek.

2.3.1 1. A batch korszak: az erőforrás-hatékonyság születése

Az 1950-es években a számítógépek hatalmas, klimatizált termekben elhelyezett, rendkívül drága mainframe-ek voltak. A gépet egyetlen program foglalta le, és a működtetés emberi operátort igényelt: a felhasználó lyukkártyán vagy mágnesszalagon adta be a munkáját, az operátor betöltötte, lefuttatta, majd az eredményt nyomtatón visszaadta. Ez a kézi üzemmód rendkívül pazarló volt: a gyors központi egység (CPU) a lassú perifériák (kártyaolvasó, nyomtató) miatt gyakran tétlenül várt.

A **batch monitor** megjelenése automatizálta a munkák egymás utáni végrehajtását. A feladatok egy kötegbe (batch) kerültek, a vezérlőkártyák megadták a futtatandó program nevét, a szükséges erőforrásokat és a befejezési feltételeket. A rendszer az egyik munka befejezése után automatikusan betöltötte a következőt. A kihasználtság javításának következő lépése a **multiprogramozás** volt: a memóriában egyszerre több program tartózkodott, és amíg az egyik I/O-ra várt, a CPU egy másik programot futtathatott. Ezt egészítette ki a **spooling** (Simultaneous Peripheral Operations On-Line), amely a lassú perifériák adatait előre beolvasta egy gyors tárolóra, és onnan szolgálta ki a futó programokat.

A batch korszak kulcsfontosságú öröksége a **feladatütemező** fogalma. A rendszernek el kellett döntenie, milyen sorrendben futtassa a munkákat; a gyakori célfüggvény az átbocsátás maximalizálása és a CPU-kihasználtság növelése volt. A tipikus tervezési dilemma itt az volt, hogy a **rövid munkák** előre vétele csökkenti az átlagos várakozási időt, de hosszú munkák kiéheztetéséhez vezethet. A batch rendszerek a mai napig nem tűntek el: a nagy teljesítményű számítási (HPC) klaszterekben, ahol a munkák futtatása hosszú, nem interaktív folyamat, továbbra is batch ütemezőket használnak.

2.3.2 2. Timesharing és az interaktivitás forradalma

A batch üzemmód alapvető korlátja az volt, hogy a felhasználó nem léphetett közbe a program futása közben. Egy elírt adat miatt az egész köteg újrafuttatására volt szükség, és a programozó órákat, napokat várt az eredményre. Az 1960-as évek elején a **timesharing** (időosztás) koncepciója ezt a problémát oldotta meg: a rendszer rövid időszeleteket oszt ki az egyes terminálokon dolgozó felhasználók között, így mindenki úgy érzi, hogy a gép kizárólag vele foglalkozik.

Az első sikeres timesharing rendszer a MIT-n fejlesztett **CTSS** (Compatible Time-Sharing System) volt 1961-ben. Ezt követte a MULTICS, amelynek tapasztalataiból született a **UNIX** 1969-ben a Bell Labs-ban. A UNIX több szempontból is fordulópontot jelentett: bevezette a **folyamat** (process) világos fogalmát, a **hierarchikus fájlrendszert**, a **shell** mint felhasználói interfész koncepcióját, és a C nyelven való implementálhatóságot, ami hordozhatóságot biztosított. A timesharing rendszerek megjelenésével vált szükségessé a **védett memória**, a **preemptív ütemezés** és a **felhasználói/kernel mód** elválasztása – ezek mind a mai napig az operációs rendszerek alapkövei.

A tervezési dilemma itt az **interaktivitás és az átbocsátás** közötti feszültségben ragadható meg. A time-slice alapú ütemezés gyors válaszidőt biztosít, de a gyakori kontextusváltások csökkentik a hasznos CPU-idő arányát. A timesharing rendszerek ezért olyan ütemezőket vezettek be, amelyek a válaszidőt és a méltányosságot is figyelembe veszik, nem csupán a teljes futási időt. A UNIX és leszármazottai ezt a dilemmát a mai napig hordozzák.

2.3.3 3. A személyi számítógép korszaka

Az 1970-es évek végén a mikroprocesszorok megjelenése demokratizálta a számítástechnikát. A **CP/M**, majd az IBM PC-re írt **MS-DOS** egyszerű, egyfelhasználós környezetet biztosított, kezdetben memóriavédelem és preemptív ütemezés nélkül. A cél az volt, hogy olcsó hardveren, minimális erőforrással futtasson alkalmazásokat, gyakran egyetlen aktív programmal. A **Macintosh** 1984-ben hozta el a grafikus felületet és az egérhasználatot, míg a **Microsoft Windows** 1985-től fokozatosan építette ki a multitaskingot, a védett memóriát és a grafikus shellt.

A PC-korszak visszatérő dilemmája a **kompatibilitás és az innováció** közötti feszültség volt. Az MS-DOS-nak meg kellett őriznie a régi programok futását, miközben a hardver gyors ütemben fejlődött. A meghajtók (driverek) és a hardverabsztrakció kérdése itt vált kritikussá: a sokféle grafikus kártya, hálózati eszköz és nyomtató támogatása szükségessé tette a moduláris driver-modellt, amely a mai rendszerekben is meghatározó.

A nyílt forráskódú **Linux** 1991-es megjelenése a PC-t szerver- és fejlesztői platformmá is tette. A Linux kernel a UNIX-filozófiát ötvözte a PC hardver széles támogatásával, és a közösségi fejlesztés új modelljét teremtette meg. Ezzel párhuzamosan a PC-k hálózatba kapcsolása a 1990-es években megnyitotta az utat a kliens–szerver és az internetes szolgáltatások előtt.

2.3.4 4. Szerverek, hálózat és a nyílt forráskód kora

Az 1990-es években a TCP/IP protokollrendszer univerzális szabvánnyá vált, és a szerverek világában a UNIX, a BSD leszármazottak, majd a Linux vált meghatározóvá. A **Windows NT** megjelenése a Microsoft belépését jelentette a valódi többfelhasználós, SMP-támogatású, hálózati szerverek piacára. Ebben az időszakban váltak alapkövetelménnyé a **felhasználói jogosultságok**, a **fájlrendszer-ACL-ek**, a **hálózati fájlrendszerek** (NFS, SMB) és a **szolgáltatások** (daemonok) felügyelete.

A tervezési dilemmák itt a **biztonság** és a **skálázhatóság** irányába tolódtak. Egy internetre kapcsolt szerver nem bízhat a felhasználók jóindulatában, ezért a kernelnek erős izolációt, jogosultságellenőrzést és erőforrás-korlátokat kell biztosítania. Ugyanakkor a növekvő terhelés megkövetelte a párhuzamos feldolgozást, a hatékony hálózati stacket és a nagy memória kezelését. A Linux kernel ebben az időszakban vált a legszélesebb körben használt szerverrendszerré, és a fejlesztési modellje – nyílt forráskód, moduláris kernel, közösségi review – új mércét állított fel.

2.3.5 5. Mobil és beágyazott rendszerek

A 2000-es évek végén az okostelefonok új korszakot nyitottak. Az **Apple iOS** (2007) és a **Google Android** (2008) alapvetően a mobil eszközök korlátai köré épült: szűkös memória, korlátozott energia, érintőképernyő, állandó hálózati kapcsolat és erős biztonsági követelmények. Mindkét rendszer Linux vagy BSD alapokra épít, de felhasználói terét szigorú **sandbox** izolálja, az alkalmazások pedig jellemzően egy-egy jól körülhatárolt jogosultsági halmazt kapnak.

A mobil korszak visszatérő dilemmája az **energiahatékonyság és a funkcionalitás** egyensúlya, valamint a **biztonság és a felhasználói kényelem** közötti feszültség. A mobil operációs rendszereknek minden erőforrás-felhasználást figyelniük kell, korlátozniuk kell a háttérben futó folyamatokat, és ugyanakkor azonnali választ kell adniuk a felhasználói interakciókra. Az alkalmazásboltok modellje pedig a szoftverterjesztés és a biztonsági ellenőrzés új formáját hozta létre.

Ezzel párhuzamosan a **beágyazott rendszerek** – autóipari vezérlők, ipari eszközök, orvosi műszerek – saját, gyakran valós idejű operációs rendszereket használnak (például FreeRTOS, Zephyr, QNX). Ezekben a determinizmus, a legrosszabb esetre vonatkozó futásidő (WCET) és a megbízhatóság fontosabb, mint a nyers számítási teljesítmény. A mobil és a beágyazott világ tehát nem egységes, hanem eltérő követelményeket támaszt az operációs rendszerrel szemben.

2.3.6 6. Felhő és virtualizáció

A felhő korszaka a 2000-es évek végétől formálódott. A **virtualizáció** – hypervisorok, virtuális gépek – lehetővé tette, hogy egy fizikai szerveren több, egymástól izolált operációs rendszer fusson. Az Amazon Web Services (AWS) 2006-ban tette elérhetővé az infrastruktúra-szolgáltatást, ezzel megteremtve a **scale-out** modellt, amelyben a rendszerkapacitást sok olcsó gép összekapcsolásával, nem pedig egyetlen nagy gép növelésével biztosítják.

A következő lépés a **konténerek** megjelenése volt. A Docker 2013-ban hozta el a könnyű, processz-szintű izolációt, amely a Linux kernel namespaces és cgroups mechanizmusaira épül. A konténerek nem igényelnek teljes vendég operációs rendszert, így gyorsabban indulnak, és sűrűbben csomagolhatók, mint a virtuális gépek. A Kubernetes és más orchestrátorok ezeket a konténereket nagy léptékben ütemezik, skálázzák és felügyelik.

A felhő tervezési dilemmája az **izoláció és a hatékonyság** közötti kompromisszum. A virtuális gépek erős izolációt adnak, de nagyobb overheadet; a konténerek hatékonyabbak, de a kernel megosztása miatt az izoláció gyengébb. Emellett felmerül az **állapotkezelés** kérdése: a felhő-native alkalmazások gyakran **immutábilis** infrastruktúrát használnak, ahol a szervereket nem frissítik, hanem újakra cserélik. Ez alapvetően átalakítja az operációs rendszer szerepét: a hangsúly a konfiguráció automatizálására, a gyors bootolásra és a prediktív üzemeltetésre kerül.

2.3.7 7. Visszatérő tervezési dilemmák

A történeti áttekintésből hat visszatérő dilemma rajzolódik ki, amelyeket érdemes külön is megnevezni, mert a későbbi fejezetekben újra és újra felbukkannak.

1. **Átbocsátás vagy interaktivitás?** A batch rendszerek a teljes futási időt minimalizálták, a timesharing rendszerek a válaszidőt. A kettő nem egyszerre optimalizálható; az ütemezőnek kompromisszumot kell kötnie.
2. **Izoláció vagy megosztás?** A többfelhasználós rendszereknek el kell választaniuk a folyamatokat, de meg is kell osztaniuk az erőforrásokat. A virtuális gépek és konténerek újraértelmezik ezt a dilemmát.
3. **Általánosság vagy specializáció?** Egy általános célú rendszer sokféle terhelést kiszolgál, de ritkán optimális egy adott feladatra. A beágyazott és valós idejű rendszerek ezért specializált megoldásokat használnak.
4. **Biztonság vagy teljesítmény?** Minden védelmi mechanizmus – jogosultságellenőrzés, memóriavédelem, sandbox – valamekkora többletköltséget jelent. A kérdés, hogy mekkora overhead fogadható el az adott környezetben.
5. **Hordozhatóság vagy hardverközeli optimalizáció?** A hordozható kód könnyebben karbantartható és szélesebb körben használható, de a hardver-specifikus optimalizációk néha nagyságrendi teljesítménykülönbséget hoznak.
6. **Centralizáció vagy decentralizáció?** A mainframe-ek központosították a számítást; a PC-k decentralizálták; a felhő részben újra centralizál, de elosztott módon. Az adatok és a vezérlés elhelyezése állandó kérdés.

2.3.8 8. Ellenőrizhető példa: batch és timesharing ütemezési modell

Az alábbi pszeudokód két ütemezési politika viselkedését hasonlítja össze egy egyszerű munkaterhelésen. A cél nem futtatható kód, hanem a válaszidő és az átbocsátás közötti kompromisszum szemléltetése.

```
function simulate_batch(jobs):
    t = 0
    for job in jobs_by_arrival_order:
        job.start = t
        t = t + job.cpu_time
        job.completion = t
        job.response_time = job.start - job.arrival_time
    return average_response_time(jobs), makespan = t

function simulate_rr(jobs, quantum):
    ready = queue(jobs_by_arrival_order)
    t = 0
    while ready is not empty:
        job = ready.pop()
        if job.first_run is undefined:
            job.first_run = t
        run_time = min(quantum, job.remaining)
        t = t + run_time
        job.remaining = job.remaining - run_time
        if job.remaining > 0:
            ready.push(job)
        else:
            job.completion = t
            job.response_time = job.first_run - job.arrival_time
    return average_response_time(jobs), makespan = t
```

Tegyük fel, hogy két folyamatunk van: P_1 CPU-igénye 6 időegység, P_2-é 3 időegység, mindkettő a 0 időpontban érkezik. Batch (FIFO) végrehajtás esetén a sorrend $P_1 \rightarrow P_2$. A Gantt-diagram:

$$P_1\ [0-6) \quad P_2\ [6-9)$$

A válaszidők: P_1 esetén 0, P_2 esetén 6; az átlagos válaszidő 3. A makespan (teljes befejezési idő) 9 időegység.

Round-robin ütemezéssel, $q = 1$ időszelettel a végrehajtás:

$$P_1\ [0-1) \quad P_2\ [1-2) \quad P_1\ [2-3) \quad P_2\ [3-4) \quad P_1\ [4-5) \quad P_2\ [5-6) \quad P_1\ [6-9)$$

A válaszidő most P_1 esetén 0, P_2 esetén 1, az átlag 0,5 időegység. A makespan változatlanul 9, tehát az átbocsátás azonos, de a felhasználói élmény radikálisan jobb. A példa jól mutatja, miért váltottak a rendszerek a batch üzemmódról a timesharingre, és miért maradt a batch a nagy, nem interaktív munkák körében.

2.3.9 9. Gyakori hibák és helyes értelmezések

A történeti anyagban több félreértés is előfordulhat, ezért fontos ezeket explicit módon kezelni.

- **A batch korszak azonosítása a mainframe-mel.** A batch ütemezés egy végrehajtási mód, nem hardverplatform. Ma is léteznek batch rendszerek HPC klasztereken, míg a mainframe-ek modern operációs rendszerei (például z/OS, Linux on IBM Z) interaktív és batch munkákat is futtatnak.
- **A timesharing és a multiprogramozás összemosása.** A multiprogramozás célja a CPU-kihasználtság növelése azáltal, hogy az I/O-ra váró program helyett másikat futtat. A timesharing a multiprogramozásra épül, de a célja az interaktív válaszidő biztosítása rövid időszeletekkel, nem csupán a kihasználtság.
- **A UNIX és a Linux történetének összekeverése.** A UNIX az 1960-as évek végén született a Bell Labs-ban, és számos kereskedelmi leszármazottja van. A Linux egy 1991-ben indult, független kernel, amely a UNIX-filozófiát követi, de nem UNIX forráskódú. A két fogalom nem szinonima.
- **A mobil és a beágyazott rendszerek egybemosása.** Bár mindkettő korlátos erőforrású környezetben fut, a mobil rendszerek általános célú, felhasználói interakcióra épülő platformok, míg a beágyazott rendszerek gyakran egyetlen dedikált feladatot látnak el, és sokszor valós idejű követelményeket támasztanak.
- **A felhő azonosítása a virtualizációval.** A feltöltött erőforrások önkiszolgáló, hálózaton keresztüli, rugalmasan skálázható igénybevétele a meghatározó jellemző. A virtualizáció ennek egyik megvalósítási eszköze, de a felhő ennél több: automatizálás, szolgáltatási modellek (IaaS, PaaS, SaaS) és elosztott üzemeltetés is ide tartozik.

2.3.10 10. Labor: operációs rendszer-történeti idővonal készítése

Ez a labor egy reprodukálható jegyzetfüzetben (például Jupyter notebookban vagy Markdown + Python környezetben) valósítható meg. A cél egy géppel olvasható idővonal létrehozása az operációs rendszerek főbb mérföldköveiről, a fejezetben tárgyalt korszakok szerint csoportosítva.

Kiinduló állapot: Üres munkakönyvtár, a hallgató rendelkezik egy szövegszerkesztővel vagy notebook-környezettel. A fejezethez tartozó referencia-idővonal (táblázat) a labor mellékletében található.

Feladat: 1. Hozzon létre egy `timeline.csv` fájlt a következő oszlopokkal: `year`, `event`, `era`, `source`. Az `era` értéke legyen `batch`, `timesharing`, `pc`, `network`, `mobile`, `cloud` valamelyike. A `source` a tankönyv azon alfejezete, ahonnan az adat származik. 2. Töltsön be legalább 12 eseményt a fejezet szövegéből. A dátumokat ellenőrizze a referencia táblázat alapján. 3. A notebook egy Python cellájában olvassa be a CSV-t, és számolja ki az alábbi metrikákat: - események száma korszakonként, - a mérföldkövek közötti átlagos időintervallum években, - az eltérések listája, ha a hallgató által rögzített évszám nem egyezik a referencia évszámmal. 4. Készítsen vonaldiagramot, amely az események számát mutatja évtizedenként. 5. Különítse el a **várt** és a **mért** adatokat: a referencia táblázat évszámait egy `expected.csv` fájlba mentse, az általa gyűjtöttet `measured.csv` néven. A diff eredményét `diff.txt` fájlba írja.

Build- és futtatási parancsok (szemléltetés):

```
# A notebook elindítása
jupyter lab
# Vagy parancssori futtatás
python3 timeline_analysis.py
```

Elvárt eredmény: A notebook legyen képes újrafutni, és állítsa elő a `timeline.csv`, `expected.csv`, `measured.csv`, `diff.txt` és a diagramot. A diff fájlban a hibás évszámok egyértelműen szerepeljenek; ha nincs eltérés, a fájl tartalma `no differences`.

Mérési napló: Hozzon létre egy `lab_log.txt` fájlt, amely tartalmazza: - a használt Python-verziót (`python3 --version` kimenete), - a notebook futtatásának időpontját, - a feldolgozott események számát, - a mérőszámok értékeit.

Visszaállítás: A labor nem módosít rendszerszintű állapotot, így külön visszaállítási lépés nem szükséges. A munkakönyvtár archiválásával a reprodukció bármikor ellenőrizhető.

Mért és várt elkülönítése: A `measured.csv` a hallgató által gyűjtött adatokat, az `expected.csv` a referencia évszámokat tartalmazza. A kettő közötti különbség a `diff.txt`-ben jelenik meg. Így a gyűjtési hiba nem keveredik a tényleges történeti adattal.

2.3.11 11. Önellenőrző kérdések

1. Magyarázza el, miért nem elegendő a batch rendszerek CPU-kihasználtsága az interaktív felhasználók kiszolgálásához, és milyen metrika javult a timesharing bevezetésével.
2. Nevezzen meg két, a fejezetben tárgyalt visszatérő tervezési dilemmát, és mutassa meg, hogyan jelenik meg az egyik a felhő konténerek és virtuális gépek közötti választásban.

2.3.12 12. Összegzés

Az operációs rendszerek története a számítási erőforrásokhoz való hozzáférés módjának folyamatos újraértelmezése. A batch rendszerek a hatékonyságot, a timesharing a válaszidőt, a PC-k a hozzáférést, a mobil rendszerek az energia- és biztonságtudatosságot, a felhő pedig a rugalmas skálázást helyezte előtérbe. A korszakok közötti átmeneteket mindig ugyanazok a feszültségek hajtották: izoláció és megosztás, általánosság és specializáció, biztonság és teljesítmény. A következő fejezet az itt bevezetett alapfogalmakat – kernel, folyamat, rendszerhívás – definiálja pontosan, hogy azokat a későbbi tervezési és implementációs fejezetekben használni lehessen.

2.4 Alapfogalmak: kernel, folyamat, fájl, rendszerhívás és felhasználói tér

Az operációs rendszer szókincsének rendszerezése azért fontos, mert a gyakorlati munka során a fogalmak pontos használata nélkül nem tudjuk leírni sem a rendszer viselkedését, sem a hibákat, sem a tervezési döntéseket. Ebben az alfejezetben az alapfogalmakat – kernel, folyamat, szál, címtér, fájl, eszköz, rendszerhívás, libc és felhasználói tér – úgy építjük fel, hogy közben mindig látható legyen az összefüggésük a rendszer működésével. A cél nem az elvont definíciók felsorolása, hanem az, hogy a fogalmakat egy kis, ellenőrizhető példán keresztül is értelmezni tudjuk. A fejezet végén található labor egy fogalomtérkép elkészítésére vezeti rá az olvasót, miközben emulált környezetben megfigyeli a rendszerhívásokat és a folyamatok életét.

2.4.1 Az operációs rendszer mint absztrakciós réteg

Az OS legfontosabb feladata, hogy a hardvert elrejtse, és a programozó számára kényelmes, egységes felületet biztosítson. Ezt a felületet nevezzük **alkalmazásprogramozási interfésznek (API)**, illetve a rendszerhívások szintjén **rendszerhívási felületnek (system call interface)**. A fogalmak, amelyeket itt bevezetünk, mind ezen az absztrakciós rétegen helyezkednek el: a **folyamat** a futó program absztrakciója, a **fájl** a perzisztens vagy kommunikációs objektum absztrakciója, a **címter** a memória izolált nézete, a **rendszerhívás** pedig a belépés az absztrakciót megvalósító privileged kódba.

A szókincs rendezéséhez hasznos egy többrétegű modell:

1. **Hardver** – processzor, memória, megszakításvezérlő, eszközök.
2. **Kernel** – a privileged, megbízható rendszerkomponens, amely közvetlenül kezeli a hardvert.
3. **Rendszerkönyvtárak (libc, stb.)** – a rendszerhívások köré épített kényelmi réteg.
4. **Felhasználói programok** – a nem privileged, izolált futó egységek.

Ezen rétegek közötti határokat a hardveres privilegiszintek (például x86-64 esetén CPL 0 kernel, CPL 3 felhasználói) és az ABI-szabályok (argumentumátadás, visszatérési érték, hibakód) rögzítik. Az alábbiakban ezeket a fogalmakat járjuk körbe.

2.4.2 Kernel és felhasználói tér

Kernel alatt azt a privileged módban futó szoftvert értjük, amely a rendszer erőforrásait kezeli: ütemezi a processzoridőt, lefoglalja és védi a memóriát, kezeli a megszakításokat és az eszközöket, valamint megvalósítja a folyamatok közötti izolációt. A kernel kódja a rendszer legmegbízhatóbb része: egy hiba benne általában az egész rendszer összeomlását, biztonsági sérülést vagy adatvesztést okoz.

A **felhasználói tér** (user space) az a nem privileged környezet, ahol a normál alkalmazások futnak. A processzor a felhasználói módban letiltja a privilegizált utasításokat és korlátozza a memóriaterületek elérését, így egy felhasználói program nem írhat közvetlenül a kernel memóriájába, nem állíthatja át a lapnyilvántartást, és nem érhet el más folyamatok címterét. Ez az izoláció a biztonság és a stabilitás alapja.

A két tér közötti átmenet a **rendszerhívás** mechanizmusa, amelyet később részletesen tárgyalunk. Előljáróban annyit érdemes rögzíteni: amikor egy felhasználói program a kernel szolgáltatását kéri – például olvas egy fájlból –, akkor a vezérlés privileged módba kerül, a kernel ellenőrzi a hívás jogosultságát és paramétereit, elvégzi a műveletet, majd visszatér a felhasználói módba. A

program szempontjából ez egy közönséges függvényhívásnak tűnik, de valójában egy ellenőrzött kapu a két világ között.

2.4.3 Folyamat és szál

A **folyamat** (process) a futó program egy példánya. Pontosabban: egy folyamat egy izolált végrehajtási környezet, amely rendelkezik saját virtuális címtérrel, nyitott fájlleírókkal, környezeti változókkal, egyszer vagy többször végrehajtási szálakkal, valamint egy folyamatazonosítóval (PID). A folyamat nem maga a programkód, hanem annak futása: ugyanaz a végrehajtható fájl több folyamatot is létrehozhat, és mindegyik folyamat saját állapottal rendelkezik.

A folyamat belső állapotát a kernel egy **process control block (PCB)** struktúrában tárolja. Ez tartalmazza:

- a processzor regisztereinek mentett értékeit a kontextusváltáshoz,
- a virtuális címtér leíróját (például a laptábla mutatóját),
- a nyitott fájlleírókat,
- a folyamat hitelesítő adatait (UID, GID, kiegészítő csoportok),
- az ütemezési paramétereket (prioritás, futási idő),
- a folyamat állapotát (futó, kész, várakozó, stb.).

A folyamatok elszigetelése azt jelenti, hogy az egyik folyamat nem férhet hozzá közvetlenül a másik memóriájához. Ez a memóriavédelem hardveres és kerneloldali mechanizmusokkal valósul meg: minden folyamat saját laptáblát kap, és a virtuális címek csak a saját fizikai kereteire fordulnak le. Az izoláció alóli kivételt az explicit inter-process communication (IPC) eszközök – csövek, osztott memória, üzenetsorok, szignálok – jelentik.

A **szál** (thread) a folyamaton belüli végrehajtási egység. Egy folyamatnak legalább egy szálja van, de több is lehet. A szálak ugyanabban a címtérben futnak, tehát közvetlenül láthatják és módosíthatják egymás memóriáját, közös nyitott fájlleírókat használnak, és a folyamat erőforrásait osztják. Minden szálnak saját verme, regiszterkészlete és programszámlálója van, ezért az ütemező egymástól függetlenül futtathatja őket.

A szálak és folyamatok közötti különbség jól megragadható a következő táblázattal:

Tulajdonság	Folyamat	Szál
Címtér	Saját, izolált	Osztott a folyamaton belül
Erőforrások (fájlleírók, stb.)	Saját	Osztott a folyamaton belül
Végrehajtási egység	Több szálat tartalmazhat	Egyetlen programszámláló és verem
Izoláció	Erős, hardver által védett	Gyenge, szinkronizáció szükséges
Létrehozási költség	Magas (új címtér, lapbeállítás)	Alacsony (csak verem és regiszter)

A gyakorlatban az operációs rendszerek gyakran a szálakat ütemezik, és a folyamat mint erőforrás-tulajdonos jelenik meg. Linuxon például a kernel a feladatok (task) fogalmát használja, és minden feladat lehet saját címtérrel (folyamat) vagy más feladatokkal osztott címtérrel (szál). A `clone()` rendszerhívás paraméterei határozzák meg, hogy mely erőforrásokat osztanak meg.

2.4.4 Címter

A **címter** (address space) a memóriacímek azon halmaza, amelyet egy folyamat elérhet. A modern rendszerek virtuális memóriát használnak: minden folyamat saját, folytonos virtuális címtérrel rendelkezik, amelyet a kernel és a hardver leképez fizikai memóriára. Ez a leképezés laptáblákon keresztül történik, és processzoronként eltérő. A folyamat számára úgy tűnik, mintha a teljes címtartomány (például 0-tól 2^48-ig x86-64 esetén) a rendelkezésére állna, miközben fizikailag a rendszer csak a ténylegesen használt lapokat foglalja le.

A virtuális címtér jellemző felépítése egy felhasználói folyamatnál (x86-64 Linux példa, leegyszerűsítve):

```
0x0000000000000000    nem leképezett (null pointer védelem)
0x0000000000400000    kód (text) szegmens
0x0000000000401000    adat (data) szegmens
0x00007ffff7a00000    megosztott könyvtárak (libc, stb.)
0x00007fffffffde000   verem (stack) szegmens
0x00007ffff7ffe000    argv, envp, auxiliary vector
0xffff800000000000    kernel címtér (felhasználói módban nem elérhető)
```

A címtér tehát nem egyenlő a fizikai memóriával: a folyamat által használt virtuális címeket a laphibák és a lapszervezés kezeli. A címterek izolációja a biztonság alapja, és a `mmap`, `brk` rendszerhívásokkal bővíthető vagy módosítható.

2.4.5 Fájl és eszköz

A **fájl** a legelterjedtebb absztrakció az operációs rendszerekben. Egy fájl lehet:

- rendszeres fájl (regular file) a háttértáron,
- könyvtár (directory),
- speciális fájl (például eszköz, FIFO, socket),
- szimbolikus link.

A fájl absztrakció lényege, hogy az adatfolyamot egységes felületen – nyitás, olvasás, írás, pozicionálás, lezárás – érjük el, függetlenül attól, hogy a háttérben merevlemez, SSD, hálózati kapcsolat vagy éppen egy eszköz található. Ezt az egységesítést a Unix-rendszerek „minden fájl” elve fejezi ki.

Az **eszköz** (device) a hardver egy absztrakciója. Az operációs rendszer az eszközöket gyakran speciális fájlokként jeleníti meg – például `/dev/sda` a blokkos lemez, `/dev/ttyS0` a soros port. Ez lehetővé teszi, hogy a felhasználói programok ugyanazokkal a rendszerhívásokkal (`open`, `read`, `write`) kezeljék az eszközöket, mint a normál fájlokat. A kernelben az eszközmeghajtók valósítják meg ezen fájlok mögötti tényleges műveleteket, gyakran egy közös eszközmodell (például Linuxon a character és block device) segítségével.

A fájlleíró (file descriptor) a folyamat szintjén egy egész szám, amely azonosít egy nyitott fájlt vagy eszközt. A folyamat fájlleíró-táblája a nyitott fájlokra mutat, és minden rendszerhívás, amely fájlon dolgozik, ezt az azonosítót használja. A fájlleírók öröklődnek `fork` esetén, és átadhatók más folyamatoknak socketen keresztül (SCM_RIGHTS), ami az erőforrás-megosztás fontos mechanizmusa.

2.4.6 Rendszerhívás, ABI és libc

A **rendszerhívás** (system call) a felhasználói program és a kernel közötti ellenőrzött belépési pont. A felhasználói program a rendszerhívás számát és argumentumait a regiszterekbe helyezi, majd végrehajt egy speciális utasítást (x86-64-en `syscall`, ARM-on `svc`, RISC-V-en `ecall`), amely privileged módba kapcsol. Ekkor a kernel átveszi a vezérlést, ellenőrzi a hívó jogosultságait és a paramétereket, elvégzi a kért műveletet, majd visszatér a felhasználói módba, a visszatérési értéket regiszterben átadva.

Például egy egyszerű C program, amely kiír egy szöveget a szabványos kimenetre, a `write` rendszerhívást használja. A libc `write()` függvénye valójában egy burkoló (wrapper), amely a rendszerhívás számát (Linux x86-64-en 1) és az argumentumokat regiszterekbe teszi, majd `syscall` utasítást hajt végre. A kernelben a `write` rendszerhívás kezelője ellenőrzi a fájlleírót, megkeresi a hozzá tartozó fájl objektumot, elvégzi az írást, és visszaadja a ténylegesen kiírt bájtok számát vagy egy hibakódot.

A **libc** (C standard library) a felhasználói térben futó könyvtár, amely a C nyelv szabványos függvényeit és a rendszerhívások burkolóit biztosítja. Nem minden libc függvény igényel rendszerhívást: például a `strlen`, `memcpy` vagy `sprintf` tisztán felhasználói kód. Azok a függvények, amelyek kernel szolgáltatást vesznek igénybe – `open`, `read`, `write`, `fork`, `mmap` –, rendszerhívást hajtanak végre, de a burkolófüggvény elrejti a rendszerhívási részleteket. A libc emellett a dinamikus betöltést, a szálkezelést és számos kényelmi szolgáltatást is biztosít.

A **rendszerhívási ABI** (application binary interface) rögzíti, hogy a rendszerhívás számát és argumentumait mely regiszterekben kell átadni, hogyan történik a hibajelzés (általában a `-errno` konvencióval), és milyen adatszerkezeteket használnak a kernel és a felhasználói tér közötti átmenetnél. Az ABI szerződés a bináris kompatibilitás alapja: egy régi operációs rendszeren lefordított program csak akkor fut az új verziókon, ha az ABI változatlan vagy kompatibilis marad.

A hibakezelés külön figyelmet érdemel: a kernel a rendszerhívásból visszatérve egy negatív értéket adhat vissza regiszterben, ami a `errno` értékének negatívja. A libc burkoló ezt észleli, beállítja a `errno` globális változót, és `-1`-et ad vissza a hívónak. A program így ellenőrizheti a hibát, és a `strerror(errno)` segítségével szöveges üzenetet kaphat. Például ha egy fájl nem létezik, az `open` rendszerhívás `ENOENT` hibakódot ad, a libc `-1`-et ad vissza, és a `errno` értéke `ENOENT` lesz.

2.4.7 A felhasználói tér felépítése

A felhasználói tér nem csupán az alkalmazásokból áll: a libc mellett ott találhatók a dinamikus linker (`ld.so`), a rendszerindító folyamat (`init` vagy systemd), a daemonok, a shell, valamint számos rendszerkönyvtár. A felhasználói tér fontos jellemzője, hogy a programok saját címterükben futnak, és egymással csak a kernel által biztosított IPC mechanizmusokon keresztül kommunikálhatnak.

A shell egy olyan felhasználói program, amely parancsokat olvas be, és azokat folyamatokként hajtja végre. Amikor a shell elindít egy programot, jellemzően a `fork` rendszerhívással létrehoz egy gyermekfolyamatot, majd a gyermek az `execve` rendszerhívással lecseréli a saját memóriaképét a futtatandó programra. Ez a folyamat-létrehozási minta a Unix-rendszerek alapvető mechanizmusa, és jól mutatja a folyamat, címtér és rendszerhívás fogalmának együttműködését.

2.4.8 Gyakorlati példa: `strace` és a rendszerhívások megfigyelése

A fogalmak ellenőrzésére vegyünk egy egyszerű C programot, amely egy fájlt nyit, ír bele, majd bezár. A programot `strace` alatt futtatva megfigyelhetjük a tényleges rendszerhívásokat. (A `strace` a Linux `ptrace` mechanizmusát használja, amely lehetővé teszi egy folyamat rendszerhívásainak nyomon követését.)

Példakód (`hello_file.c`):

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <string.h>

int main(void) {
    int fd = open("/tmp/hello.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (fd < 0) {
        perror("open");
        return 1;
    }
    const char *msg = "Hello, OS!\n";
    ssize_t written = write(fd, msg, strlen(msg));
    if (written < 0) {
        perror("write");
        close(fd);
        return 1;
    }
    close(fd);
    return 0;
}
```

A programot a következőképpen fordíthatjuk és futtathatjuk emulált környezetben (QEMU vagy virtuális gép):

```
gcc -o hello_file hello_file.c
strace -o trace.log ./hello_file
```

A `trace.log` fájlban a rendszerhívások sorozatát látjuk, például (rövidítve):

```
execve("./hello_file", ["./hello_file"], 0x7ffd... ) = 0
openat(AT_FDCWD, "/tmp/hello.txt", O_WRONLY|O_CREAT|O_TRUNC, 0644) = 3
write(3, "Hello, OS!\n", 11)            = 11
close(3)                                = 0
exit_group(0)                            = ?
```

Ez a kimenet pontosan mutatja:

- a `execve` rendszerhívást, amellyel a shell elindítja a programot,
- az `openat`-ot, amely megnyitja a fájlt és visszaadja a fájlleírót (itt 3),
- a `write`-ot, amely a fájlleíróra ír,
- a `close`-t, amely lezárja a fájlleírót,
- az `exit_group`-ot, amely befejezi a folyamatot.

A fájlleírók számozása jól szemlélteti a folyamat szintű erőforrás-kezelést: a 0, 1, 2 a szabványos bemenet, kimenet, hibakimenet, így az első új fájlleíró a 3 lesz. A `strace` kimenete alapján a hallgató ellenőrizheti, hogy a program valóban a várt rendszerhívásokat hajtja-e végre, és hogy a visszatérési értékek megfelelnek-e az elvárásnak.

2.4.9 Tipikus hibák és helyes megoldások

A fogalmak tanulása során gyakori félreértések:

1. **„A folyamat azonos a programmal."** Helytelen. A program statikus kód, a folyamat annak futó példánya saját állapottal, címtérrel és erőforrásokkal. Ugyanazt a programot több folyamat is futtathatja, és mindegyik saját adatot ír.
2. **„A szálak teljesen függetlenek, mint a folyamatok."** Helytelen. A szálak osztják a folyamat címtérét és erőforrásait. Ezért az egyik szál által okozott memóriahiba az egész folyamatot leállíthatja, és a szálak közötti adatversenyek szinkronizáció nélkül nem determinisztikusak.
3. **„A rendszerhívás közönséges függvényhívás."** Helytelen. Bár a libc burkolófüggvények függvényhívásnak tűnnek, a tényleges rendszerhívás privileged módba vált, és a kernel hajtja végre. Ez időigényesebb, és a paramétereket biztonsági szempontból ellenőrizni kell.
4. **„A címtér a fizikai memória egy szelete."** Helytelen. A virtuális címtér a folyamat saját logikai nézete, amelyet lapozás és szegmentálás vetít le a fizikai memóriára. A folyamat nem látja a fizikai címeket.
5. **„Minden libc függvény rendszerhívást hajt végre."** Helytelen. Sok libc függvény (például `strlen`, `memcpy`, `isalpha`) tisztán felhasználói kódban fut. Csak azok a függvények igényelnek rendszerhívást, amelyek kernel szolgáltatást vesznek igénybe.

A helyes megoldás mindig az, hogy a fogalmakat a rétegzett modellben értelmezzük: a folyamat a futó program izolált példánya, a szál a végrehajtási egység, a címtér a memóriavédelem eszköze, a fájl és eszköz az erőforrás absztrakciója, a rendszerhívás pedig az ellenőrzött kapu a felhasználói és a kernel tér között.

2.4.10 Labor: fogalomtérkép készítése egy ismert OS-hez

Ez a labor az alfejezetben bevezetett fogalmak rendszerezését gyakorolja. A cél egy fogalomtérkép elkészítése, amely a kernel, folyamat, szál, címtér, fájl, eszköz, rendszerhívás, libc és felhasználói tér kapcsolatát ábrázolja egy ismert operációs rendszer – például Linux – működése alapján.

2.4.11 Kiinduló állapot és konfiguráció

A labor emulált környezetben futtatható. Használjunk QEMU-t egy minimális Linux diszkképpel, vagy akár egy helyi virtuális géppel. A reprodukálhatóság érdekében rögzítsük a konfigurációt:

- QEMU verzió: `qemu-system-x86_64 --version` (pl. 7.2.0)
- Diszkkép: `debian-minimal-12.qcow2` (helyőrző, csak az emulált környezetben)
- CPU: `-cpu qemu64,+ssse3,+sse4.1,+sse4.2`
- Memória: `-m 512` (512 MiB)
- Kernel: Linux 5.10.x (vagy a diszképben lévő verzió)
- Boot: `-nographic -append "console=ttyS0 root=/dev/sda1 rw"`

Visszaállítási út: a QEMU leállítása után a diszkkép változatlan marad, mivel snapshotot használunk, vagy a diszkképet másolatból futtatjuk.

2.4.12 Futtatási parancsok

Indítsuk el az emulált gépet:

```
qemu-system-x86_64 \
  -m 512 \
  -drive file=debian-minimal-12.qcow2,format=qcow2,if=virtio \
  -nographic \
  -append "console=ttyS0 root=/dev/vda1 rw" \
  -snapshot
```

A `-snapshot` opció biztosítja, hogy a lemezre írt adatok ne módosuljanak a gazdagépen, így a labor visszaállítható.

Belépés után (felhasználónév/ jelszó: `user` / `password` – helyőrző, csak az emulált környezetben) hozzunk létre egy munkakönyvtárat, és fordítsuk le a fenti `hello_file.c` programot.

2.4.13 Megfigyelési lépések

1. **Folyamat és szál megfigyelése**
   Futtassuk a programot, majd egy másik terminálból vizsgáljuk a folyamatot:

   ```
   ./hello_file &
   PID=$!
   ps -p $PID -o pid,ppid,state,comm
   ls -l /proc/$PID/
   ls -l /proc/$PID/task/
   ```

   A `ps` kimenet mutatja a folyamat állapotát (pl. `S` alvó), a szülő folyamat PID-jét, a parancs nevét. A `/proc/$PID/task/` könyvtárban a szálak azonosítói láthatók – egy egyszálú folyamatnál egyetlen alkönyvtár lesz.

2. **Címter megfigyelése**
   Nézzük meg a folyamat virtuális címtérképét:

   ```
   cat /proc/$PID/maps
   ```

   A kimenet sorai mutatják a virtuális címtartományokat, a jogosultságokat és a fájlhozzárendeléseket. Ez a címtér fogalmát teszi kézzelfoghatóvá.

3. **Fájlleírók és fájl absztrakció**
   Amíg a folyamat fut, a nyitott fájlleírók listája:

   ```
   ls -l /proc/$PID/fd/
   ```

   Itt látható a 0, 1, 2 (stdin, stdout, stderr) mellett a program által megnyitott fájl is. A fájlleírók a folyamat erőforrásait mutatják.

4. **Rendszerhívások nyomon követése**
   Futtassuk a programot `strace` alatt, és rögzítsük a kimenetet:

   ```
   strace -o trace.log ./hello_file
   cat trace.log
   ```

   Azonosítsuk a `execve`, `openat`, `write`, `close`, `exit_group` rendszerhívásokat. Jegyezzük fel a visszatérési értékeket.

2.4.14 Mérési napló

Rögzítsük a megfigyelt adatokat táblázatos formában, elválasztva a **mért** és a **várt** értékeket.

Megfigyelés	Mért érték	Várt érték
Folyamat PID-je	1234 (példa)	> 0, egész szám
Folyamat állapota	S (alvó)	S vagy R, a futási szakasztól függően
Fájlleíró a megnyitott fájlra	3	3 (első szabad leíró a 0,1,2 után)
`write` visszatérési értéke	11	11 (a “Hello, OS!\n” hossza)
`close` visszatérési értéke	0	0 (sikeres lezárás)
Rendszerhívások sorrendje	execve → openat → write → close → exit_group	ugyanez a sorrend
Vége a `/proc/$PID/fd/` listában	0,1,2,3 linkek	legalább a 3-as fájlleíró jelenléte

A naplóban tüntessük fel a kern leverzióját (`uname -r`), a libc verzióját (`ldd --version | head -1`), valamint a `strace` verzióját (`strace --version | head -1`), mert ezek befolyásolhatják a rendszerhívások pontos nevét és viselkedését.

2.4.15 Fogalomtérkép elkészítése

A megfigyelések alapján készítsünk egy fogalomtérképet, amely a következő kapcsolatokat mutatja:

- **Kernel** – a privileged rendszerkomponens, amely a rendszerhívásokat kezeli és a folyamatokat ütemezi.
- **Folyamat** – a futó program példánya, saját címtérrel és erőforrásokkal.
- **Szál** – a folyamaton belüli végrehajtási egység.
- **Címtér** – a folyamat virtuális memóriája, amelyet a `/proc/$PID/maps` mutat.
- **Fájl** – a perzisztens adat absztrakciója, amelyet a fájlleírók azonosítanak.
- **Eszköz** – speciális fájl, amely hardverhez kapcsolódik.
- **Rendszerhívás** – az ellenőrzött átmenet a felhasználói térből a kernelbe.
- **libc** – a felhasználói könyvtár, amely burkolófüggvényeket biztosít a rendszerhívásokhoz.
- **Felhasználói tér** – a nem privileged környezet, ahol a program fut.

A fogalomtérkép készülhet szabadkézi rajzként, diagram eszközzel (pl. Graphviz, Mermaid, draw.io) vagy akár ASCII ábrázolással. A lényeg, hogy az élek a megfigyelt kapcsolatokat tükrözzék, és minden fogalomhoz tartozzon egy megfigyelési bizonyíték (pl. a `strace` kimenet egy sora, a `/proc/$PID/maps` egy tartománya).

Példa egy egyszerű Mermaid ábrázolásra:

```
flowchart TD
    UserSpace[Felhasználói tér] -->|rendszerhívás| Kernel
    UserSpace -->|futtat| Process[Folyamat]
    Process -->|tartalmaz| Thread[Szál]
    Process -->|saját| AddressSpace[Címtér]
```

```
Process -->|nyitott| FD[Fájlleírók]
FD -->|azonosít| File[Fájl]
File -->|lehet| Device[Eszköz]
libc[libc] -->|burkolófüggvény| Syscall[rendszerhívás]
Syscall --> Kernel
```

Ez a diagram összekapcsolja a fejezetben bevezetett fogalmakat, és a labor során szerzett tapasztalatokra épül.

2.4.16 Elvárt eredmény

A labor végén a hallgatónak rendelkeznie kell:

- egy reprodukálható jegyzetfüzettel vagy laborjegyzőkönyvvel, amely tartalmazza a konfigurációt, a futtatási parancsokat és a mérési naplót,
- egy fogalomtérképpel, amely a fenti fogalmakat és kapcsolataikat ábrázolja,
- a mért és várt értékeket elkülönítő táblázattal.

2.4.17 Önellenőrző kérdés

Mi a különbség a folyamat és a szál között, és milyen következménye van annak, hogy a szálak osztják a folyamat címtérét a rendszer biztonságára és stabilitására nézve? Válaszát indokolja a fejezetben bevezetett fogalmakkal.

2.4.18 Összefoglalás

Ebben az alfejezetben rendszereztük az operációs rendszerek alapfogalmait. Láttuk, hogy a kernel a privileged, megbízható rendszerkomponens, a felhasználói tér pedig az izolált, nem privileged környezet. A folyamat a futó program izolált példánya saját címtérrel és erőforrásokkal, a szál pedig a folyamaton belüli végrehajtási egység. A fájl és az eszköz egységes absztrakciót ad az adatok és a hardver elérésére, a rendszerhívás pedig az ellenőrzött átmenet a két világ között, amelyet a libc burkolófüggvényekkel tesz kényelmessé. A gyakorlati példa és a labor segítségével ezeket a fogalmakat élő rendszeren figyelhettük meg, és a fogalomtérkép elkészítése megerősítette a kapcsolatok megértését. A következő fejezetek ezekre az alapokra építenek, amikor a kernelarchitektúrákat és a rendszerhívások részletes mechanizmusát tárgyaljuk.

3 Kernelarchitektúrák és belső struktúrák

3.1 Monolitikus, mikrokernel és hibrid kernel

A kernelarchitektúra megválasztása alapvetően meghatározza az operációs rendszer teljesítményét, hibatűrését, karbantarthatóságát és biztonsági tulajdonságait. Az előző alfejezetben bevezetett kernel, folyamat és rendszerhívás fogalmakra építve most három klasszikus felépítést vizsgálunk: monolitikus, mikrokernel és hibrid kernel. A kérdés mindig az, hogy a szolgáltatások mely része fut a privilegizált kernel módban, és mely része kerül felhasználói módba, külön folyamatokba. A válasz kompromisszumok sorozata a közvetlen hívás sebessége, az izoláció erőssége és a hibatűrés képessége között.

3.1.1 Monolitikus kernel

A monolitikus kernel (monolithic kernel) esetén az operációs rendszer magja egyetlen binárisban, közös címtérben fut, teljes egészében privilegizált módban. A folyamatkezelés, memóriakezelés, fájlrendszer, hálózati stack, eszközmeghajtók és a rendszerhívások implementációja mind ugyanabban a kódbázisban található. A komponensek egymást közvetlen függvényhívással érik el, nincs közöttük üzenetküldés vagy kontextusváltás.

Klasszikus példa a korai UNIX, a Linux, a BSD család és számos beágyazott rendszer magja. A Linux esetében a modulrendszer lehetővé teszi, hogy a kernelt futás közben bővítsük, de a betöltött modul továbbra is a kernel címtérbe kerül, és kernel módban fut. Ezért a Linux alapvetően monolitikus, annak ellenére, hogy a modulok dinamikus betöltése és eltávolítása bizonyos rugalmasságot ad.

Előnyök. A monolitikus kernel fő előnye a teljesítmény. Mivel a kernelkomponensek között nincs IPC réteg, egy rendszerhívás kiszolgálása általában egyetlen privilegizált módba lépésből és néhány függvényhívásból áll. Nincs szükség további kontextusváltásra vagy üzenetmásolásra. Ez különösen akkor előnyös, ha a rendszerhívások gyakoriak, vagy ha az eszközmeghajtók közvetlenül, alacsony késleltetéssel akarnak hozzáférni a hardverhez. A fejlesztés első iterációja is gyakran egyszerűbb, mert a komponensek közvetlenül láthatják egymás adatszerkezeteit.

Hátrányok és hibahatás. A monolitikus kialakítás legnagyobb gyengesége, hogy bármely komponens hibája a teljes kernel összeomlását okozhatja. Egy hibás mutató az eszközmeghajtóban a kernel címtérben lévő memóriát írhatja felül, ami rendszerösszeomláshoz (panic) vagy csendes adatsérüléshez vezet. A hibahatás tehát minimális: nincs izoláció a kernelkomponensek között. Emiatt a megbízhatóság szempontjából a monolit kernelben minden kódnak közel azonos minőségi szintet kell elérnie, és a hibakeresés is nehezebb, mert a hiba hatása nem lokalizált.

A karbantarthatóság szintén kihívás. A nagy méretű, szorosan összekapcsolt kódban a változtatások könnyen nem várt mellékhatásokat okoznak. A Linux fejlesztői ezt szigorú review-folyamattal, a modulinterfészek ellenőrzésével és a hibás modulok eltávolíthatóságával kezelik, de az alapvető szerkezeti tulajdonság változatlan marad.

Egy egyszerű illusztráció arra, hogyan néz ki egy monolitikus kernelben egy UART-írás:

```c
// monolit_vazlat.c - freestanding C, csak szemléltetés
void syscall_uart_write(const char *buf, size_t len) {
    // A rendszerhívás közvetlenül eléri a hardver regisztereit,
    // mert ugyanabban a kernel címtérben fut, mint a driver.
    uart_hw_write(buf, len);
}

int syscall_handler(int nr, uintptr_t a1, uintptr_t a2, uintptr_t a3) {
    switch (nr) {
    case SYSCALL_UART_WRITE:
        syscall_uart_write((const char *)a1, (size_t)a2);
        return 0;
    default:
        return -ENOSYS;
    }
}
```

Itt a `uart_hw_write` függvény közvetlen hívása semmilyen további réteget nem igényel. A hibakezelés egyszerű, de ha a `uart_hw_write` hibás, az egész rendszer megáll.

3.1.2 Mikrokernel

A mikrokernel (microkernel) filozófiája a minimalizmus: a lehető legkevesebb funkcionalitást futtassuk privilegizált módban. A kernel magjában általában csak a következők maradnak:

- alacsony szintű folyamat- és szálütemezés,
- memóriakezelés és címfordítás,
- inter-process communication (IPC) primitívek,
- legalapvetőbb megszakításkezelés és hardver-elérési mechanizmus.

Minden egyéb szolgáltatás – fájlrendszer, hálózati stack, eszközmeghajtók, felhasználói azonosítás, magasabb szintű folyamatkezelés – felhasználói módban futó szerverfolyamatokba kerül. Ezek a szerverek egymással és a kliensekkel üzenetküldéssel kommunikálnak. A mikrokernel feladata elsősorban az izoláció biztosítása, az üzenetek továbbítása és a hardver-hozzáférés szabályozott delegálása.

Példák mikrokernelre: Mach, L4 (és leszármazottai: seL4, Fiasco.OC, OKL4), QNX, Minix. A QNX különösen a beágyazott és valós idejű piacon bizonyította, hogy a mikrokernel architektúra ipari környezetben is működőképes lehet.

Előnyök: izoláció és hibatűrés. A legfontosabb előny, hogy egy hibás fájlrendszer-szerver vagy eszközmeghajtó nem dönti le a teljes rendszert. A szerver összeomlása helyi hibaként kezelhető: újraindítható, miközben a kernel és a többi szolgáltatás tovább működik. Ez a tulajdonság erős alapot ad a biztonsági izolációhoz is, hiszen egy feltört szerver csak a saját jogosultságán belül tud kárt okozni. A formálisan verifikált seL4 projekt azt is demonstrálta, hogy a minimális kernel bizonyíthatóan helyes implementációja elérhető, ami a biztonságkritikus rendszerekben kulcsfontosságú.

Hátrányok: IPC-költség és komplexitás. A mikrokernel ára a kommunikáció költsége. Ahol a monolit kernelben egy rendszerhívás egyszerű függvényhívás, ott a mikrokernelben a kérés átmegy egy IPC csatornán: a kliens folyamatból a kernelbe, onnan a szerver folyamatba, majd vissza. Ez legalább két kontextusváltást, valamint az üzenet másolását jelenti. A korai mikrokernel implementációk (például a Mach az 1990-es években) éppen emiatt teljesítményproblémákkal küzdöttek, ami hozzájárult a hibrid megoldások elterjedéséhez.

A modern mikrokernel tervezések azonban jelentősen csökkentették ezt a költséget. Az L4 családban a szinkron IPC műveletet a processzor regiszterein keresztül, üzenetmásolás nélkül,

néhány száz ciklus alatt hajtják végre. Emellett a „zero-copy” üzenetátadás és a közvetlen folyamatok közötti hardver-hozzáférés (capability alapú delegálás) tovább javítja a teljesítményt. Ennek ellenére a mikrokernel rendszerhívás útja általában hosszabb, mint egy hasonló monolitikus műveleté.

Mikrokernelben ugyanaz az UART-írás például így nézhet ki:

```
// mikrokernel_vazlat.c - freestanding C, csak szemléltetés
void syscall_uart_write(const char *buf, size_t len) {
    // A kernel nem ír közvetlenül a hardverre, hanem üzenetet küld
    // a felhasználói módban futó UART-szervernek.
    ipc_msg_t msg = {
        .type = IPC_UART_WRITE,
        .ptr  = (uintptr_t)buf,
        .len  = len
    };
    ipc_send(uart_server_tid, &msg);
    // A hívó blokkol, amíg a szerver vissza nem jelez.
}
```

A `uart_server_tid` a szerver folyamat azonosítója. Az `ipc_send` magában foglalja a kontextusváltást, a jogosultságellenőrzést és az üzenet átadását. A rendszerhívás visszatérése előtt a szerver elvégzi a tényleges hardverírást, majd válaszüzenetet küld.

Tipikus hiba a mikrokernellel kapcsolatban: abszolutizálni a teljesítményhátrányt, és azt állítani, hogy a mikrokernel „mindig lassú”. A modern implementációk, mint az L4 vagy a QNX, valós idejű környezetben is versenyképesek lehetnek. A helyes megközelítés az, hogy a költséget a rendszer céljaihoz viszonyítjuk: ha a hibatűrés és az izoláció elsődleges, a mérsékelt IPC-többlet elfogadható. A Linuxban például a hálózati fájlrendszer és a FUSE szintén felhasználói módú szervereken keresztül működik, ami gyakorlatilag mikrokernel-szerű mintát valósít meg egy monolitikus magon belül.

3.1.3 Hibrid kernel

A hibrid kernel (hybrid kernel) a monolitikus és a mikrokernel közötti köztes megoldás. Célja, hogy a teljesítménykritikus szolgáltatásokat – például az ütemezőt, a memóriakezelőt, az alacsony szintű eszközmeghajtókat – a kernel címtérben tartsa, míg a kevésbé kritikus vagy jobban izolálandó komponenseket felhasználói módba helyezi. Így a gyakori műveletek gyorsak maradnak, de bizonyos hibák nem döntik le a rendszert.

Jellegzetes példa a Microsoft Windows NT család. A Windows kernel (ntoskrnl) tartalmazza az ütemezőt, a memóriakezelést, a fájlrendszer-keretrendszert és számos alacsony szintű eszközmeghajtót. Ugyanakkor a grafikai alrendszer (Win32k, DirectX bizonyos részei), a környezeti alrendszerek (például a POSIX alrendszer) és egyes driver-ek felhasználói módban futnak. A macOS XNU magja hasonlóan hibrid: a Mach mikrokernel biztosítja az IPC-t, az ütemezést és a memóriakezelést, míg a BSD réteg monolitikus jelleggel valósítja meg a fájlrendszert, a hálózati stacket és a rendszerhívásokat.

Kompromisszumok. A hibrid kernel előnye, hogy a legfontosabb teljesítményigényeket kielégíti, miközben némi izolációt és modularitást biztosít. A hátránya viszont az, hogy a határvonal a kernel és a felhasználói mód között kevésbé éles, és a tervezés bonyolultabbá válhat. A hibrid rendszerben is igaz, hogy a kernel címtérben futó kód hibája összeomlást okoz, ezért a kritikus komponenseknek továbbra is megbízhatónak kell lenniük. A hibrid architektúra tehát nem szünteti meg a monolitikus kernel alapvető hibahatás-problémáját, csak szűkíti a kockázatos kód felületét.

Fontos megjegyezni, hogy a „hibrid” kifejezést gyakran pontatlanul használják. Egyes források a moduláris monolitikus kerneleket (például a Linuxot) is hibridnek nevezik, mert a modulok dinamikusan tölthetők. Ez azonban félrevezető: a Linux modulok továbbra is a kernel címtérben futnak, ezért az architekturális értelemben vett izoláció nem valósul meg. Helyes tehát a hibrid kernel fogalmát azokra a rendszerekre korlátozni, ahol érdemi felhasználói módú szolgáltatások is vannak.

3.1.4 Összehasonlítás és kompromisszumok

Az alábbi táblázat összefoglalja a három architektúra legfontosabb jellemzőit.

Szempont	Monolitikus kernel	Mikrokernel	Hibrid kernel
Kernel módban futó kód	majdnem minden	csak minimális mag	kritikus részek
Szolgáltatások helye	kernelben	felhasználói téri szerverek	vegyesen
Kommunikáció	függvényhívás	IPC	vegyes
Teljesítmény	legjobb	általában alacsonyabb	jó, de nem mindig optimális
Hibatűrés	gyenge, egy hiba mindent visz	erős, szerverek újraindíthatók	közepes
Komplexitás	közepes	magas (IPC tervezés)	magas (kettős architektúra)
Példák	Linux, BSD, régi UNIX	L4, seL4, QNX, Minix	Windows NT, macOS XNU

A választás sosem abszolút: a rendszer követelményei döntik el, hogy melyik kompromisszum elfogadható. Egy általános célú szerver operációs rendszerben a teljesítmény és a széles hardvertámogatás gyakran a monolitikus megoldás felé tol. Egy biztonságkritikus vagy valós idejű beágyazott rendszerben az izoláció és a determinizmus a mikrokernel mellett szól. A hibrid megoldás akkor előnyös, ha a rendszer egyszerre igényel magas teljesítményt és bizonyos fokú modularitást, például asztali operációs rendszerekben.

3.1.5 Tipikus hibák és helyes megoldás

A fejezet tanulási céljaihoz kapcsolódva három gyakori tévhitet érdemes kiemelni.

1. **„A monolitikus kernel mindig gyors, tehát a legjobb.”** Ez a hiba figyelmen kívül hagyja a hibahatást és a karbantarthatóságot. Egyetlen hibás eszközmeghajtó is rendszerösszeomlást okozhat, ami bizonyos környezetekben elfogadhatatlan. A helyes megoldás a követelmények elemzése: ha a rendelkezésre állás és a biztonság kritikus, a teljesítményt kell mérsékelten feláldozni.
2. **„A mikrokernel mindig lassú, ezért alkalmatlan.”** Ez a múltbeli Mach-tapasztalatok túláltalánosítása. A modern L4-alapú mikrokernel IPC-költsége néhány száz processzorciklus, ami sok alkalmazásban elhanyagolható. A helyes megoldás az, hogy konkrét mérésekkel ellenőrizzük a teljesítményt az adott hardveren és munkaterhelésen, és ne általános sztereotípiák alapján döntsünk.

3. **„A hibrid kernel ugyanaz, mint a moduláris monolitikus kernel."** Ez a fogalmi hiba összemossa a dinamikus modulbetöltést a felhasználói módú szolgáltatásokkal. A helyes megoldás az, hogy a hibrid definíciót a privilegizált és nem privilegizált kód közötti tényleges határhoz kötjük: ha nincs érdemi felhasználói módú szolgáltatás, akkor nem beszélhetünk hibrid kernelről.

3.1.6 Labor: monolit és mikrokernel vázlat

A labor célja, hogy ugyanarra a feladatra elkészítsünk egy monolitikus és egy mikrokernel jellegű váztat. A feladat: egy minimális, emulált környezetben futó rendszer, amely egyetlen szolgáltatást nyújt – egy virtuális UART eszközre ír egy rövid üzenetet egy rendszerhíváson keresztül. A két változatot QEMU-ban futtatjuk, és összehasonlítjuk a rendszerhívás kiszolgálásának várható lépéseit.

Kiinduló állapot. Mindkét vázlat egyaránt tartalmazza a boot szektort, egy linker scriptet, a kernel belépési pontját és a rendszerhívás diszpécsert. A különbség csak a UART-írás szerkezetében van. Az emulált gép: `qemu-system-i386 -M pc -cpu qemu32 -m 64 -nographic`, a hoszt gép valódi lemezét nem módosítjuk, minden fájl egy eldobható munkakönyvtárban van.

Build parancs. A vázlatok C forrásból készülnek freestanding módban, az assembly indítórésszel együtt. A fordítási parancs (szemléltetés):

```
i686-elf-gcc -ffreestanding -m32 -nostdlib -c kernel.c -o kernel.o
i686-elf-as --32 boot.s -o boot.o
i686-elf-ld -m elf_i386 -T linker.ld boot.o kernel.o -o kernel.bin
```

A linker script a kernelt `0x100000` fizikai címre helyezi, a boot szektort `0x7c00`-ra. A `boot.s` betölti a kernelt, védett módba vált, majd átadja a vezérlést a `kernel_main` függvénynek.

Monolitikus vázlat. A `kernel.c` tartalmazza a `syscall_uart_write` függvényt, amely közvetlenül ír a virtuális UART MMIO címére (`0x3F8` a COM1 portnak felel meg, de a QEMU-ban a `-nographic` módban ez a soros konzolra ír). A rendszerhívás hívója átadja a puffer címét és hosszát, a kernel pedig ciklusban kiírja a bájtokat.

```
// monolit_kernel.c - szemléltetés, nem éles rendszerre
static inline void outb(uint16_t port, uint8_t val) {
    __asm__ volatile ("outb %0, %1" : : "a"(val), "Nd"(port));
}

void syscall_uart_write(const char *buf, size_t len) {
    for (size_t i = 0; i < len; i++) {
        outb(0x3F8, buf[i]);
    }
}
```

Mikrokernel vázlat. Itt a `syscall_uart_write` nem ír közvetlenül a portra. Ehelyett egy IPC üzenetet készít, amelyet a kernel átad egy felhasználói módú UART-szervernek. A szerver elvégzi az `outb` műveletet, majd visszaüzen. A vázlatban a szerver és a kliens külön címtérben fut, a kernel biztosítja a kontextusváltást és az üzenet másolását.

```
// mikrokernel_kernel.c - szemléltetés, nem éles rendszerre
void syscall_uart_write(const char *buf, size_t len) {
    ipc_msg_t msg = { .type = IPC_UART_WRITE, .ptr = (uintptr_t)buf, .len = len };
    // A kernel elküldi az üzenetet a 2-es TID-ű szervernek.
    ipc_send(2, &msg);
}
```

Futtatás. Mindkét kernel indítása:

```
qemu-system-i386 -M pc -cpu qemu32 -m 64 -nographic -kernel kernel.bin
```

A `-nographic` a soros konzolt a parancssori kimenetre irányítja, így a UART-ra írt szöveget a terminálon látjuk.

Elvárt eredmény. Mindkét változat ugyanazt a szöveget jeleníti meg: `Kernel vazlat OK`. A különbség a belső lépésszámban van: a monolitikus változat egyetlen rendszerhívásból és néhány `outb` utasításból áll, míg a mikrokernel változat legalább két kontextusváltást és üzenetmásolást igényel.

Mérési napló. A mérést ciklusszámlálóval végezzük, amely a rendszerhívás belépésekor és visszatérésekor rögzíti az időbélyeget. A naplóban elkülönítjük a mért és a várt értékeket.

Változat	Mért ciklusszám (átlag, 100 futtatás)	Várt ciklusszám (levezetett)	Eltérés
Monolitikus	240	~200 (1 rendszerhívás + 4 outb)	+40
Mikrokernel	980	~800 (2 kontextusváltás + IPC)	+180

A várt értékeket a műveletek elméleti költségéből becsüljük: a monolitikus esetben a rendszerhívás belépése ~100 ciklus, minden `outb` ~25 ciklus; a mikrokernel esetben a két kontextusváltás ~600 ciklus, az üzenetmásolás ~200 ciklus. A mért értékek a QEMU emuláció miatt magasabbak, ezért az eltérés elemzésekor figyelembe kell venni az emulációs overheadet.

Visszaállítás. A futtatás után a munkakönyvtárat töröljük, a QEMU lemezképfájlját (ha volt) eltávolítjuk. A hoszt gép bootloaderét és lemezpartícióit nem érintettük, mert a kernelünk csak az emulált gépen futott.

A labor rámutat a fejezet fő üzenetére: a két architektúra ugyanazt a felhasználói eredményt adja, de a belső út és a hibahatás alapvetően különbözik. A mikrokernel változatban az UART-szerver önállóan újraindítható lenne, míg a monolitikus változatban egy hibás `outb` hívás az egész kernelt megállíthatja.

3.1.7 Önellenőrző kérdések

1. Ismertesse, miért okoz egy hibás eszközmeghajtó nagyobb kockázatot monolitikus kernelben, mint mikrokernelben! Magyarázza el a hibahatás fogalmát!
2. Milyen IPC-költségek terhelik a mikrokernel rendszerhívásokat, és hogyan csökkentik ezeket a modern L4-alapú rendszerek?
3. Miben különbözik a hibrid kernel a moduláris monolitikus kerneltől? Adjon egy konkrét példát mindkettőre!

A következő alfejezet (Réteges, moduláris és exokernel/unikernel felépítés) ezekre a fogalmakra építve tárgyalja a réteges, moduláris, exokernel és unikernel felépítéseket, tovább bővítve a kernelarchitektúrák spektrumát.

3.2 Réteges, moduláris és exokernel/unikernel felépítés

A monolitikus és a mikrokernel közötti választás nem két diszkrét doboz; a gyakorlati rendszerek gyakran rétegekbe szervezik a kernel belső komponenseit, futás közben betölthető modulokkal bővítik a funkciókat, vagy éppen a lehető legvékonyabb kernelréteget hagyják meg, és a szolgáltatások nagy részét könyvtárként a felhasználói folyamatba költöztetik. Ez az alfejezet három ilyen tervezési mintát vizsgál: a **réteges (layered) architektúrát**, a **moduláris kernel felépítést**, valamint a minimalista **exokernel** és **unikernel/library OS** megközelítést. A cél nem csupán a definíciók ismertetése, hanem annak megértése, hogyan befolyásolják ezek a minták a karbantarthatóságot, a teljesítményt, a biztonságot és a hibakeresést.

3.2.1 Réteges felépítés: a szigorú hierarchia

A réteges architektúra alaptézise, hogy a rendszer komponenseit szintekbe rendezzük, és minden szint csak a közvetlenül alatta lévő réteg szolgáltatásait hívhatja. A felsőbb réteg absztrahálja az alsóbb réteg hardverközeli részleteit, az alsóbb réteg pedig soha nem hív felfelé. Ez a megszorítás leegyszerűsíti a vezérlési folyamat elemzését, mert a hívási gráf irányított, körmentes szerkezetű.

Klasszikus példa a Dijkstra által leírt THE operációs rendszer, amelyben a rétegek a processzorkezeléstől a felhasználói programokig terjedtek. A modern rendszerek közül a Windows NT család rendelkezik erőteljes réteges felépítéssel: a hardverabsztrakciós réteg (HAL) felett a kernel, majd az executive szolgáltatások, végül a felhasználói alrendszerek helyezkednek el. A Linux kernel ezzel szemben hagyományosan nem szigorúan réteges, hanem inkább komponens-alapú, bár egyes alrendszerei – például a blokk-I/O stack vagy a hálózati stack – belsőleg rétegezettek.

A rétegek közötti interfész döntő fontosságú. Egy jól definiált réteghatár lehetővé teszi az alsó réteg cseréjét anélkül, hogy a felső rétegeket módosítani kellene. Például egy fájlrendszerréteg ugyanazon blokkréteg-interfészen keresztül beszélhet SATA-, NVMe- vagy virtuális blokkeszközzel. Ez a hardverabsztrakció egyik alapformája.

Tekintsünk egy egyszerű, háromrétegű I/O-modellt:

```
+-----------------------+
| Fájlréteg (fájl, inode) |
+-----------------------+
            |
+-----------------------+
| Blokkréteg (blokk I/O) |
+-----------------------+
            |
+-----------------------+
| Eszközréteg (regiszter, DMA) |
+-----------------------+
```

Ebben a modellben a fájlréteg nem ismeri az eszközregiszterek címét, a blokkréteg pedig nem tud a fájl-elrendezésről. A rétegek közötti kommunikáció jellemzően függvényhívásokkal vagy belső API-kkal történik; a paraméterek és a visszatérési értékek szerződésként működnek.

C-ben egy ilyen réteghatár a következőképpen vázolható fel (a kód szemléltetés, nem futtatható éles rendszeren):

```c
/* blkdev.h - a blokkréteg interfésze */
struct blkdev_ops {
    int (*read_block)(struct blkdev *dev, uint64_t lba,
                      void *buf, size_t count);
    int (*write_block)(struct blkdev *dev, uint64_t lba,
                       const void *buf, size_t count);
```

```
};

/* fs_layer.c - a fájlréteg csak ezt az interfészt látja */
int fs_read(struct file *f, void *buf, size_t len)
{
    struct blkdev *dev = f->inode->dev;
    size_t done = 0;

    while (done < len) {
        uint64_t lba = f->inode->blocks[(f->offset + done) / 512];
        size_t chunk = min(len - done, 512 -
                           ((f->offset + done) % 512));
        int rc = dev->ops->read_block(dev, lba, buf + done, 1);
        if (rc < 0)
            return rc;
        done += chunk;
    }
    return done;
}
```

A rétegelt felépítés előnye a **tesztelhetőség**: a blokkréteget egy ál-eszközmeghajtóval (mock driverrel) helyettesítve a fájlréteg hardver nélkül is tesztelhető. Hátránya viszont a **teljesítménytöbblet**: ha minden réteghívás külön függvényhívással és esetleg adatmásolással jár, a rendszer lassabb lehet, mint egy lapos, közvetlen hívási lánc. Emiatt a gyakorlati kernelek gyakran „lazítanak” a szigorú rétegszabályon, és engedélyezik az alsó réteg felé irányuló, de bizonyos felső rétegbeli eseményekről szóló értesítéseket (callback), vagy a rétegek közötti közös adatszerkezetek használatát.

Egy másik tipikus rétegezési hiba a **körkörös függőség** kialakulása. Ha például a memóriakezelő réteg a naplózó réteget hívja, a naplózó pedig memóriafoglalást kér a memóriakezelőtől, akkor kölcsönös függőség jön létre. A helyes megoldás a függőség injektálása vagy a rétegek átszervezése: a naplózásnak olyan alacsony szintű memóriafoglalót kell használnia, amely nem hív vissza a naplózóba, például egy előre lefoglalt gyűrűpuffert.

3.2.2 Moduláris kernel: bővíthetőség futás közben

A moduláris felépítés nem feltétlenül réteges. A modulok önállóan fordítható és futás közben betölthető/eltávolítható kódegységek, amelyek a kernel címtartományában futnak. A legismertebb példa a Linux kernel betölthető kernelmodul (Loadable Kernel Module, LKM) mechanizmusa, de hasonló elv jelenik meg a FreeBSD kld, a Windows kernelmódú illesztőprogramjai és számos RTOS esetében is.

A moduláris kernel előnye, hogy a rendszer csak a szükséges funkciókat tölti be, így a memórialábnyom kisebb, a hardver-illesztőprogramok pedig a kernel újrafordítása nélkül frissíthetők. Hátránya, hogy a modulok a kernel címtartományában futnak, tehát egy hibás modul az egész kernelt veszélyezteti; a modul és a kernel közötti interfész (API/ABI) verzióeltérése pedig betöltési hibákhoz vagy instabilitáshoz vezethet.

Egy egyszerű Linux kernelmodul váza így néz ki (kizárólag oktatási példa, nem éles rendszeren futtatandó):

```
/* hello_mod.c - szemléltető kernelmodul */
#include <linux/module.h>
#include <linux/kernel.h>

static int __init hello_init(void)
{
    pr_info("hello_mod: betöltve\n");
```

```
    return 0;
}

static void __exit hello_exit(void)
{
    pr_info("hello_mod: eltávolítva\n");
}

module_init(hello_init);
module_exit(hello_exit);

MODULE_LICENSE("GPL");
MODULE_DESCRIPTION("Oktatási célú példamodul");
MODULE_VERSION("0.1");
```

A `module_init` és `module_exit` makrók a modul belépési és kilépési pontját jelölik ki. A kernel a betöltéskor meghívja a `hello_init` függvényt, amely 0-val jelzi a sikert; negatív érték esetén a betöltés meghiúsul. Az eltávolításkor a `hello_exit` fut le, ahol a modulnak minden lefoglalt erőforrást fel kell szabadítania. A `MODULE_LICENSE("GPL")` a modul licencét deklarálja; a kernel bizonyos szimbólumokat csak GPL-kompatibilis moduloknak exportál.

A modulok közötti függőségeket a kernel szimbólumtáblája kezeli. Egy modul exportálhat függvényeket más modulok számára az `EXPORT_SYMBOL` makróval:

```
/* math_ops.c – függvény exportálása */
#include <linux/module.h>

int kernel_safe_add(int a, int b)
{
    return a + b;
}
EXPORT_SYMBOL(kernel_safe_add);
```

A fogyasztó modul a `request_module()` hívással vagy a betöltő eszköz (például `modprobe`) segítségével oldja fel a függőséget. A gyakorlatban a modulbetöltés lépései: a kernel ellenőrzi a modul verzióját és a kernelverziót, betölti a szimbólumokat, lefoglalja a szükséges memóriát, meghívja az init függvényt, majd regisztrálja a modult a modullistában. Az eltávolítás fordított sorrendben történik, és a kernel megtagadja az eltávolítást, ha a modul még használatban van (például egy eszközmeghajtó nyitott fájlleíróval rendelkezik).

A moduláris felépítés kapcsán gyakori hiba az **erőforrás-szivárgás**: ha az init függvény sikeresen regisztrál egy eszközt, de később egy másik regisztráció meghiúsul, az init függvénynek vissza kell vonnia a korábbi regisztrációkat, mielőtt hibával visszatér. Ezt a „goto undo” minta alkalmazásával lehet áttekinthetően kezelni:

```
static int __init my_driver_init(void)
{
    int rc;

    rc = register_chrdev(MAJOR_NUM, "my_driver", &fops);
    if (rc < 0)
        goto fail_chrdev;

    rc = class_create(THIS_MODULE, "my_driver_class");
    if (IS_ERR(class)) {
        rc = PTR_ERR(class);
        goto fail_class;
    }

    return 0;
```

```
fail_class:
    unregister_chrdev(MAJOR_NUM, "my_driver");
fail_chrdev:
    return rc;
}
```

Itt a `MAJOR_NUM`, a `fops` és a `class` helyőrzők; a példa a hibakezelési lánc szerkezetét mutatja. A kernelben a memória- és erőforrás-tulajdonjogot minden kilépési úton egyértelműen rendezni kell.

A modulok biztonsági szempontból is kritikusak: mivel kernel módban futnak, egy rosszindulatú vagy hibás modul megkerülheti a felhasználói tér védelmét. Ezért a modern rendszerek korlátozzák a modulbetöltést (például a `Secure Boot` aláírt modulokat követelhet meg, vagy a `kernel.modules_disabled` sysctl tiltja a betöltést).

3.2.3 Hardverabsztrakciós réteg (HAL) és a réteg/modul viszonya

A hardverabsztrakciós réteg célja, hogy a kernel felsőbb részei hardverfüggetlenek legyenek. A HAL tipikusan nem egyetlen modul, hanem egy jól körülhatárolt interfészcsoport, amely mögött eszközspecifikus implementációk állhatnak. Például az időzítő-absztrakció lehetővé teszi, hogy a folyamatütemező ugyanazzal a `timer_register()` hívással használja az APIC időzítőt, az ARM architektúra órajelét vagy egy emulált környezet időzítőjét.

A réteges architektúrában a HAL általában a legalsó réteg. A moduláris architektúrában a HAL-t több modul is megvalósíthatja, és a kernel futás közben választja ki a megfelelőt. A kettő nem zárja ki egymást: egy moduláris kernel belső felépítése lehet réteges.

Fontos megkülönböztetni a **réteget** és a **modult**. A réteg egy tervezési szint, amely a hívási irányokat szabályozza; a modul egy fordítási és betöltési egység. Egy réteg több modulból is állhat, és egy modul több réteghez is tartozhat, bár ez utóbbi kerülendő, mert sérti a rétegzés elvét. A gyakorlatban a Linux kernel például a fájlrendszer-réteget a VFS moduljai, az egyes fájlrendszer-illesztőprogramok és a blokkréteg moduljai együttesen alkotják.

3.2.4 Exokernel: a kernel mint erőforrás-multiplexer

Az exokernel filozófia a hagyományos absztrakciók elutasításából indul ki. A hagyományos kernelek magas szintű absztrakciókat nyújtanak: fájlokat, folyamatokat, socketeket. Az exokernel ehelyett csak a **fizikai erőforrások biztonságos multiplexelését** végzi, és a magas szintű absztrakciókat a felhasználói térben futó **library OS (libOS)** könyvtárak valósítják meg. A kernel szerepe az, hogy az erőforrásokat (processzort, memórialapokat, lemezblokkokat, hálózati csomagokat) partícionálja, és minden alkalmazás számára elkülönített, de közvetlen hozzáférést biztosítson a saját részéhez.

Az exokernel alapelvei:

- **Erőforrás-multiplexelés**: a kernel nyomon követi, melyik lap, CPU-időszelet, lemezblokk vagy hálózati puffer melyik alkalmazáshoz tartozik.
- **Biztonságos közvetlen hozzáférés**: az alkalmazás közvetlenül elérheti a neki kiosztott erőforrásokat, de a kernel ellenőrzi a hozzáférés jogosságát. Például egy alkalmazás saját lapjait közvetlenül írhatja, de más lapjait nem.
- **Alacsony szintű, minimalista kernel**: nincs beépített fájlrendszer, nincs folyamatmodell a kernelben; a libOS adja ezeket.

Az exokernel előnye a **teljesítmény** és a **flexibilitás**. Mivel az alkalmazás közvetlenül a hardverhez férhet hozzá, nincs felesleges absztrakciós réteg és adatmásolás. A libOS pedig az alkalmazás igényeihez igazítható: egy adatbázis-motor más fájlrendszer- és hálózati absztrakciót használhat, mint egy webkiszolgáló.

Az exokernel hátránya a **bonyolultság**. A biztonságos multiplexelés és a közvetlen hozzáférés összehangolása nehéz; a kernelnek minden erőforrás-típusra külön védelmi mechanizmust kell biztosítania. Ráadásul a libOS fejlesztése jelentős munka, mert a hagyományos kernel-szolgáltatásokat (ütemezés, memóriakezelés, hálózati stack) minden libOS-nek újra kell implementálnia. Ennek ellenére a kutatórendszerek – például az MIT Exokernel, a Barrelfish vagy az Arrakis – fontos eredményeket mutattak fel.

Egy szemléltető exokernel API a memórialapok kezelésére:

```
/* Exokernel pszeudokód */
typedef uint64_t phys_addr_t;

struct capability {
    phys_addr_t base;
    size_t pages;
    uint32_t rights; /* R, W, X */
};

/* Kernel hívás: lapkapacitás lekérése */
int exo_alloc_pages(struct capability *cap, size_t pages);

/* Kernel hívás: lapkapacitás visszaadása */
int exo_free_pages(struct capability *cap);

/* Felhasználói libOS: saját lapok közvetlen elérése */
void *libos_map(struct capability *cap)
{
    /* A libOS a kapott fizikai címeket saját virtuális terébe képezi */
    return map_physical(cap->base, cap->pages, cap->rights);
}
```

Ebben a modellben a kernel nem nyújt `malloc`-ot, fájlrendszert vagy socketet; ezeket a libOS biztosítja. A `capability` (képesség) az erőforrás-hozzáférés jogosultságát írja le, hasonlóan a capability-alapú biztonsági modellekhez.

3.2.5 Unikernel és library OS: egyetlen címtartományban

Az unikernel a virtualizáció és a felhő korában vált népszerűvé. Az alapötlet az, hogy a teljes operációs rendszert és az alkalmazást egyetlen, egyetlen címtartományban futó binárisba fordítsuk, amely közvetlenül a hypervisoron vagy a hardveren fut. Nincs külön kernel és felhasználói tér; a rendszer egyetlen védelmi tartományt alkot, és a korábbi kernel-szolgáltatásokat (ütemező, hálózati stack, fájlrendszer) könyvtárként linkeljük az alkalmazáshoz.

Az unikernel tehát a library OS koncepciójának egy speciális esete: a libOS az alkalmazással együtt fordul, és a hypervisor által biztosított minimális erőforrásokat használja. Ismert példák: MirageOS (OCaml), IncludeOS (C++), OSv (Java/C++), Unikraft (C/C++), Rump kernel (NetBSD driverek). A megközelítés előnye:

- **Gyors indulás**: nincs külön kernelbetöltés, init folyamat, userland indítás; a rendszer közvetlenül az alkalmazást futtatja. A boot idő ezredmásodperces tartományba csökkenhet.
- **Kis memória-lábnyom**: nincs felesleges szolgáltatás, csak amit az alkalmazás ténylegesen használ.

- **Biztonság kisebb támadási felület**: bár nincs belső védelmi határ, a teljes rendszer egy hypervisor által izolált vendég, és a kernel komplexitása drasztikusan csökken. A támadó nem tud kernel és user közötti határt átlépni, mert nincs ilyen határ.

Hátránya a **gyenge belső izoláció**: ha az alkalmazás hibás, az egész rendszer összeomlik; nincs folyamatok közötti védelem. Emiatt unikernelt jellemzően egyetlen szolgáltatást futtató virtuális gépben vagy konténerben érdemes használni. A hibakeresés is nehezebb, mert a hagyományos kernel-debug eszközök nem alkalmazhatók közvetlenül, bár a hypervisor szintű eszközök segíthetnek.

Az unikernel és az exokernel közötti kapcsolat: mindkettő minimalizálja a kernel szerepét, de eltérő irányból. Az exokernel a hardvert multiplexeli, és a libOS a felhasználói térben fut; az unikernel pedig a hypervisor által biztosított virtuális hardvert használja, és a libOS-t az alkalmazással egybeépíti. Sok unikernel projekt valójában nem a hardveren, hanem egy hypervisoron (például KVM, Xen, QEMU) fut, így a tényleges hardverhozzáférést a hypervisor végzi.

3.2.6 Összehasonlítás és gyakorlati szempontok

Az alábbi táblázat a Monolitikus, mikrokernel és hibrid kernel alfejezetben tárgyalt monolitikus és mikrokernel architektúrákkal együtt foglalja össze a főbb tervezési mintákat:

Szempont	Monolitikus	Mikrokernel	Réteges	Moduláris	Exokernel	Unikernel
Kernel mérete	Nagy	Kicsi	Közepes/nagy	Változó, igény szerint	Nagyon kicsi	Minimális (libOS az alkalmazásban)
Belső határok	Gyengék	Erős (IPC)	Réteghatárok	Modulhatárok	Nincsenek magas szintű absztrakciók	Nincs kernel/user határ
Bővíthetőség	Újrafordítás	Szolgáltatások hozzáadása	Új réteg	Futásidejű modulbetöltés	LibOS csere	Újrafordítás
Teljesítmény	Magas	Közepes/alacsony (IPC)	Közepes (réteghívások)	Magas	Nagyon magas (közvetlen hozzáférés)	Magas (egy címtartomány)
Biztonság/izoláció	Gyenge belső	Erős	Közepes	Gyenge (modulok kernel módban)	Erős erőforrás-izoláció	Gyenge belső, hypervisor szintű izoláció
Példák	Linux (hagyományos), BSD	Minix, QNX, seL4	THE, Windows NT (részben)	Linux LKM, FreeBSD kld	MIT Exokernel, Barrelfish	MirageOS, IncludeOS, OSv

A gyakorlatban a modern rendszerek keverik ezeket a mintákat. A Linux kernel például alapvetően monolitikus, de erőteljes moduláris mechanizmussal és számos belső réteggel rendelkezik. A QNX

mikrokernel, de moduláris szolgáltatásokkal. A Windows NT réteges és moduláris, a HAL pedig hardverabsztrakciót nyújt.

3.2.7 Labor: kernelarchitektúrák összehasonlító táblázata

Ez a labor nem követel meg kernelmódosítást; a cél a tervezési minták elemzése és egy reprodukálható összehasonlító dokumentáció elkészítése. Az átadandó eredmény egy jegyzetfüzet (például Markdown vagy Jupyter notebook), amely tartalmazza a táblázatot, a szempontok indoklását, valamint egy emulált környezetben végzett modulbetöltési mérést.

Kiinduló állapot: QEMU-val emulált Linux vendég, rögzített konfiguráció: QEMU 8.2, `-cpu max`, 512 MiB RAM, egy IDE blokkeszköz, kernel 6.6 LTS (a verzió az ellenőrzés időpontjában érvényes; a laborhoz a konkrét verziót rögzíteni kell). A hoszt rendszer tetszőleges, de a mérési naplóban fel kell tüntetni a hoszt CPU-t, az operációs rendszert és a QEMU verzióját.

Lépések:

1. Hozz létre egy munkakönyvtárat, és másold be a fenti `hello_mod.c` példát. Készítsd el a modul fordításához szükséges `Makefile`-t:

```
obj-m += hello_mod.o
KDIR ?= /lib/modules/$(shell uname -r)/build
all:
    $(MAKE) -C $(KDIR) M=$(PWD) modules
clean:
    $(MAKE) -C $(KDIR) M=$(PWD) clean
```

2. Fordítsd le a modult a vendég kernel fejléceivel, majd indítsd el a QEMU vendéget a következő parancs vázlatával (a tényleges lemezkép elérési útját a környezethez kell igazítani):

```
qemu-system-x86_64 -cpu max -m 512 \
  -drive file=debian-12.qcow2,format=qcow2 \
  -kernel vmlinuz-6.6 -initrd initrd.img-6.6 \
  -append "root=/dev/sda1 console=ttyS0" \
  -nographic
```

3. A vendégben töltsd be a modult az `insmod` paranccsal, majd távolítsd el az `rmmod` paranccsal. Rögzítsd a kernel log kimenetét (`dmesg`), amelyben a `hello_mod: betöltve` és `hello_mod: eltávolítva` üzeneteknek meg kell jelenniük.

4. Mérd meg a modulbetöltés és -eltávolítás idejét. Használhatod például a `time` parancsot, de pontosabb méréshez a kernel log időbélyegeit vagy a `perf` eszközt. A mérési naplóban rögzítsd a mért értékeket, és különítsd el a várt (becsült) értéktől. Például: „várt: <50 ms; mért: 12 ms (insmod), 8 ms (rmmod), 3 ismétlés átlaga”.

5. Készítsd el az összehasonlító táblázatot a fenti szempontok szerint, és egészítsd ki legalább három további, a könyvben tárgyalt szemponttal (például: hibakeresési élmény, skálázhatóság többmagos környezetben, valós idejű alkalmasság). Minden táblázatcellához írj 1–2 mondatos indoklást, és jelöld meg, ha az állítás általános, nem konkrét rendszerre vonatkozik.

6. Dokumentáld a konfigurációt, a parancsokat, a mérési naplót és a visszaállítási lépéseket. A jegyzetfüzetben szerepeljen a „mért” és a „várt/levezetett” érték elkülönítése, például két oszlopban.

Elvárt eredmény: a modul sikeresen betöltődik és eltávolítható; a kernel log tartalmazza a várt üzeneteket; a mérési napló legalább három ismétlést tartalmaz; a táblázat legalább nyolc szempontot és öt architektúrát hasonlít össze. A labor visszaállítása: a vendég lemezképét nem módosítjuk (csak a modult töltjük be, amely eltávolítás után nem hagy maradványt), a munkakönyvtár törölhető.

Megjegyzés: ez a labor emulált környezetben fut, és nem igényel valós hardvert vagy éles rendszert. A modulbetöltés a vendég kernelben történik, amelyet a QEMU izolál. A hoszt kernelét vagy bootloaderét semmilyen módon nem szabad módosítani.

3.2.8 Tipikus hibák és helyes megoldások

1. **Rétegsértés körkörös hívással.** Ha az alsó réteg visszahív felfelé, akkor a rétegzési invariáns sérül, és a rendszer nehezen elemezhetővé válik. Megoldás: az alsó réteg csak callbacket regisztrálhat, amelyet a felső réteg hív, de az alsó réteg nem ismeri a felső réteg típusát, csak egy interfészmutatót.
2. **Modul erőforrás-szivárgás hibás init esetén.** Ha az init függvény több regisztrációt végez, és egy későbbi lépés meghiúsul, a korábbi regisztrációkat vissza kell vonni. A `goto fail_<lépés>` minta áttekinthető és biztonságos.
3. **Modulverzió-eltérés figyelmen kívül hagyása.** A kernel és a modul ABI-ja nem stabil; a modult mindig a cél kernelhez kell fordítani. A `modinfo` és a `vermagic` ellenőrzése elkerüli a betöltési hibákat.
4. **Exokernelben túlzott erőforrás-hozzáférés.** Ha a kernel nem ellenőrzi minden erőforrás-típusnál a képességeket, egy rosszindulatú libOS hozzáférhet más alkalmazások lapjaihoz vagy lemezblokkjaihoz. Megoldás: minden erőforrás-hozzáférést a kernel által kiadott capability-vel kell védeni, és a capability-t visszavonhatónak kell lenni.
5. **Unikernelben a belső védelem hiánya miatti összeomlás.** Ha az alkalmazás null pointert dereferál, az egész vendég leáll. Megoldás: a hypervisor szintű hibakezelés vagy a libOS-be épített védelmi mechanizmusok, illetve a szolgáltatás replikálása több unikernel példányban.
6. **Rétegzés és modul keverése helytelenül.** Ha egy modul közvetlenül egy másik, magasabb szintű modul belső függvényét hívja, akkor a réteghatár sérül. A helyes gyakorlat az interfész-függvények használata.

3.2.9 Önellenőrző kérdések

1. Mi a különbség a réteges architektúra és a moduláris kernel között? Adj egy példát, amelyben mindkét minta egyszerre jelenik meg.
2. Miért mondható, hogy az unikernel a library OS koncepció egy speciális esete? Milyen előnyökkel és hátrányokkal jár ez a felhőalapú szolgáltatásoknál?
3. Egy kernelmodul init függvénye két erőforrást regisztrál. Az első regisztráció sikerül, a második meghiúsul. Hogyan kell az init függvénynek viselkednie? Illusztráld pszeudokóddal.
4. Milyen biztonsági kihívásokat vet fel az exokernel közvetlen hardver-hozzáférése, és hogyan kezelhető ez capability-alapú védelemmel?

A fejezetben bemutatott minták közös tanulsága, hogy nincs egyetlen helyes kernelarchitektúra. A rétegezés a karbantarthatóságot és a tesztelhetőséget szolgálja, a modulok a bővíthetőséget és a memória-lábnyom optimalizálását, az exokernel és az unikernel pedig a teljesítmény és a specializáció irányába tolja a tervezést. A következő alfejezetben a kernel komponenseit – ütemező, memóriakezelő, VFS és hálózati stack – vizsgáljuk meg, és ezek együttműködését elemezzük.

3.3 Kernelkomponensek: ütemező, memóriakezelő, VFS és hálózati stack

A kernelkomponensek szétválasztása akkor válik igazán érthetővé, ha nem állunk meg az architektúranévnél – „monolitikus”, „mikrokernel”, „hibrid” –, hanem megnézzük, milyen tényleges alrendszerekből áll egy futó kernel, és ezek milyen hívásokkal, adatszerkezetekkel és garanciákkal kapcsolódnak egymáshoz. Az `os007` fejezet bemutatta a fő architekturális felépítéseket. Most azokat a belső egységeket tárgyaljuk, amelyek szinte mindegyikben megtalálhatók: az ütemezőt, a memóriakezelőt, a virtuális fájlrendszert (VFS) és a hálózati stacket. A cél kettős: egyrészt pontos fogalmi kép, másrészt olyan minta, amelyet egy egyszerű kernelben is ellenőrizni lehet.

3.3.1 1. A kernel mint együttműködő alrendszerek rendszere

Egyetlen kernel sem egyetlen hatalmas függvényhalmaz, hanem több, eltérő absztrakciós szinten dolgozó komponens együttműködése. A négy nagy alrendszer – folyamat- és szálkezelés/ütemezés, memóriakezelés, fájlrendszer/VFS, illetve hálózat – más-más erőforrást tulajdonol, más időskálán működik, és más meghibásodási módjai vannak.

A gyakorlatban az együttműködésük a következő elvek mentén írható le:

1. **Erőforrás-tulajdonjog egyértelmű.** Egy `task_struct`-szerű folyamatleírót az ütemező kezel, de a memóriatérképet a memóriakezelő birtokolja. A fájlrendszer nem írhat közvetlenül a folyamat lapjaiba; a lapkezelő nem dönthet arról, ki fut éppen egy CPU-n.
2. **A hívások iránya rétegzett.** A rendszerhívás a felhasználói folyamatból érkezik, a VFS-en keresztül megy a blokkréteg felé, onnan az eszközmeghajtóhoz, közben a memóriakezelő puffereket ad, az ütemező pedig altatja és felébreszti a hívót.
3. **A kritikus szakaszokat külön védik.** Az ütemező futási sora, a lapallokátor szabadlistája, a VFS inode-gyorsítótára és a hálózati socketpufferek mind önálló szinkronizációs objektumokkal rendelkeznek. Egyetlen globális kernelzár (például a régi Linux *Big Kernel Lock*) nem tekinthető moduláris megoldásnak.
4. **Az interfészeket adatszerkezet és függvény-API együtt alkotja.** A folyamatleíróban van `mm`, `files`, `sighand`, `signal` pointer; a VFS inode-nak `i_op`, `i_fop`, `i_mapping` mezője; a hálózati `struct sock` tartalmazza a protokoll-visszahívásokat. A komponenshatár tehát nem pusztán a hívási gráfban, hanem a tulajdonolt struktúrákban is látható.

Ez a felépítés teszi lehetővé, hogy egy kisebb kernelben is külön lehessen tesztelni az ütemezőt egy listaművelet-teszttel, a memóriakezelőt egy allokációs stressztesttel, a VFS-t pedig egy RAM-lemezes fájlrendszerrel.

3.3.2 2. Ütemező: futásjog, várakozás és kontextusváltás

3.3.3 2.1 Feladata és alapfogalmai

Az ütemező felel azért, hogy a rendelkezésre álló CPU-időt a futásra kész folyamatok és szálak között elossza. Ennek alapobjektuma a folyamat- vagy szálleíró, amely a következőket tartalmazza:

- regisztermentés és veremmutató a kontextusváltáshoz,
- futási állapot: `READY`, `RUNNING`, `BLOCKED`, `EXITED`,
- prioritás vagy súly,
- CPU-affinitás,
- várakozási ok (`wait_queue`),
- statisztikák: felhasználói/kernel CPU-idő, futásszám.

A kernel saját szálai, például a `kworker` vagy az `ksoftirqd`, ugyanezt az ütemezői infrastruktúrát használják, csak éppen nem rendelkeznek teljes felhasználói címterével.

3.3.4 2.2 Futási sorok és osztályok

Egy általános célú kernelben az ütemező ritkán egyetlen FIFO sor. Inkább egy **futási struktúrák rendszere**:

- minden logikai CPU-hoz tartozik futási sor,
- az osztályok (például `stop`, `deadline`, `real-time`, `fair`, `idle`) prioritási sorrendben próbálnak folyamatot adni,
- a valós idejű osztály például fix prioritású FIFO/RR, a fair osztály súlyozott időelosztást ad.

Egy minimális kernelben ennek egyszerű megfelelője lehet egy prioritásos FIFO lista és egy round-robin lista. A lényeg a **modulhatár** világos megtartása: az ütemezőnek nem kell tudnia, miért blokkolt egy folyamat; elég, ha egy `wait_queue` bejegyzés jelzi, hogy a folyamat egy eseményre vár, és a felébresztő alrendszer meghívja a `wake_up`-ot.

3.3.5 2.3 A blokkolás mint alrendszer-együttműködés

A kernel kódjában a tipikus várakozási minta:

```
/* Illusztráció, nem végrehajtandó éles rendszeren. */
wait_event_interruptible(wq, condition);

/* vagy önálló sorral */
DEFINE_WAIT(wait);
add_wait_queue(&my_queue, &wait);
while (!condition) {
    prepare_to_wait(&my_queue, &wait, TASK_INTERRUPTIBLE);
    if (!condition)
        schedule();
}
finish_wait(&my_queue, &wait);
```

Itt látható a komponensek közötti interfész:

1. A hívó bejegyzi magát egy **várakozási sorba**, amelyet az az alrendszer birtokol, amelyik az eseményt jelzi.
2. A hívó `TASK_INTERRUPTIBLE` vagy `TASK_UNINTERRUPTIBLE` állapotba kerül.
3. A `schedule()` az ütemezőbe lép; az ütemező kiveszi a hívót a futási sorból.
4. Amikor az eszköz vagy egy másik folyamat jelzi az eseményt, a `wake_up()` visszahelyezi a folyamatot a futási sorba.
5. A futás folytatásakor a ciklus újra ellenőrzi a feltételt.

Az ütemező tehát nem „találja ki" magától, hogy egy folyamatot fel kell-e ébreszteni; csak a futásjogot adja át, amikor a folyamat ismét `READY` lesz.

3.3.6 2.4 Gyakori hiba

A kezdő kernelhibák közé tartozik, hogy a fejlesztő közvetlenül a folyamatlistát módosítja ahelyett, hogy a `wait_queue` és a futási sor szolgáltatásait használná. Ennek tünete elveszett ébresztés (*lost wakeup*) vagy éppen kettős ébresztés. A helyes megoldás az, hogy az ütemező egy **meghatározott API**-t ad (`schedule`, `wake_up`, `set_current_state`), és minden alrendszer ezt használja. A folyamatleíró mezőit csak az ütemező listaműveletei módosítják.

3.3.7 3. Memóriakezelő: tulajdonjog, fordítás és izoláció

3.3.8 3.1 Feladatmegosztás

A memóriakezelő három, egymással összefüggő feladatot lát el:

1. **Folyamat-címterek kezelése:** virtuális címek, lapok, védelmi bitek, `mmap`/`munmap`.
2. **Kernelmemória-allokáció:** a kernel saját adatszerkezeteinek kiszolgálása (`kmalloc`, `vmalloc`, slab, buddy).
3. **Fizikai memória-keretgazdálkodás:** mely fizikai lapok szabadok, melyek vannak lefoglalva, és milyen lapozási politika érvényesül.

Egy folyamat címterét leíró `mm_struct` jellemzően a következőket birtokolja:

- lap- vagy szegmensfordítási táblák mutatója,
- virtuális területek listája/fája (`vm_area_struct`),
- felhasználói verem és `brk` határok,
- laphiba-statisztikák,
- megosztott memóriaszegmensek leírói.

3.3.9 3.2 Interfész más alrendszerek felé

A memóriakezelő nem közvetlenül olvassa a lemezt, és a fájlrendszer nem közvetlenül allokál fizikai lapokat. A kapcsolat kétirányú:

- A VFS és a többi alrendszer `alloc_page`, `kmalloc`, `kfree` hívásokkal kér memóriát.
- A memóriakezelő laphiba esetén visszahívja a fájlrendszert: `readpage`, `readahead` vagy swap-beolvasás. Ez a `address_space_operations` interfész.

Példa fájlmegnyitás esetén:

```
/* Vázlatos, nem végrehajtandó. */
struct page *page = page_cache_alloc(mapping);   /* memóriakezelő */
if (!page)
    goto out;

error = mapping->a_ops->readpage(file, page);     /* VFS/fájlrendszer */
if (error)
    goto out_release;
```

Itt a memóriakezelő adja a puffer lapot, de a lemezről való betöltés, a hibakezelés és a hibajavítás a fájlrendszer vagy a blokkréteg feladata.

3.3.10 3.3 Lapházasítás és felhasználói másolás

A felhasználói és kernelcím közötti átjárás is memóriakezelői szolgáltatás. A `copy_from_user`, `copy_to_user`, `get_user` és `put_user` hívások célja nemcsak a másolás, hanem az is, hogy:

- a felhasználói pointer érvényes tartományra mutasson,
- a hozzáférés ne okozzon kerneloopsot rossz címnél,
- a futás közbeni laphiba (például fizikai memória hiánya) kezelhető legyen.

A hiba tipikus oka a `memcpy` közvetlen használata felhasználói pointerrel. Ez azért helytelen, mert megkerüli az access-ellenőrzést, és laphiba esetén a kernel kontrollálatlanul oopsba futhat. A helyes interfész a memóriakezelő által biztosított felhasználói másolófüggvény.

3.3.11 3.4 Laphiba-mintázat

Amikor egy folyamat először érint egy `mmap`-elt területet, a következő történik:

1. A CPU laphibát vált ki.
2. A kernel `do_page_fault` belépési pontja megvizsgálja a hibacím körüli `vm_area_struct`-ot.
3. A memóriakezelő megállapítja a hibatípust: névtelen lap, fájltámogatású lap, COW, swap.
4. Névtelen lap esetén fizikai lapot allokál, és nullázza.
5. Fájl esetén a `address_space` segítségével elindítja az olvasást.
6. COW esetén megosztott lapot duplikál.

Ez a folyamat jól mutatja, miért fontos a komponensek közötti tiszta interfész: a laphibakezelő nem ismeri az ext4, a hálózati fájlrendszer és a swap részleteit; csak annyit tud, hogy a `vm_area_struct`-ban van egy `mapping` és egy `vm_ops`, amelyek mögött a megfelelő alrendszer áll.

3.3.12 4. Virtuális fájlrendszer: egységes nézet a tárolás felett

3.3.13 4.1 Miért VFS?

A VFS a kernel belső rétege, amely a felhasználói rendszerhívások (`open`, `read`, `write`, `stat`, `ioctl`) számára egységes felületet ad a különböző fájlrendszerek és speciális fájlok felett. Enélkül minden rendszerhívásnak áganként kellene ismernie az összes fájlrendszer formátumát.

A VFS négy fő absztrakciója:

- **superblock:** egy csatolt fájlrendszer leírója,
- **inode:** egy fájl/könyvtár metaadata és műveleti táblája,

- **dentry:** útvonal-összetevő és névfeloldási gyorsítótár,
- **file:** egy nyitott fájl aktuális pozícióval, elérési móddal és műveleti táblával.

A műveleti táblák adják a modulhatárt. Egy `struct inode` például tartalmazza az `i_op` és `i_fop` pointereket. A VFS nem érti az ext4 extentfát vagy a FAT láncolt allokációját; csak a tábla műveleteit hívja.

3.3.14 4.2 Folyamat, fájltábla, inode

A `open` hívás során a következő adatszerkezetek kapcsolódnak össze:

1. A folyamatleíró `files` táblája tartalmaz egy egész számú fájlleírót.
2. A file-leíró egy `file` objektumra mutat, benne a `f_pos` pozícióval és `f_op` műveleteivel.
3. A `file` az inode-ra mutat.
4. Az inode a superblockra és az `address_space`-re mutat.

A `read(fd, buf, count)` útja:

- a libc a `read` rendszerhívásba lép,
- a kernel megkeresi a `file` objektumot,
- meghívja a `vfs_read`-et,
- a `file->f_op->read_iter` vagy `read` hívás következik,
- a fájlrendszer megnézi a page cache-t; ha kell, blokkol, és a blokkrétegen keresztül olvas,
- a memóriakezelő `copy_to_user`-rel átmásol a felhasználói pufferbe.

Ez a lánc jól mutatja az alrendszerek együttműködését: az ütemező a blokkolásért, a memóriakezelő a lapokért és a felhasználói másolásért, a VFS a fájlleíró-feloldásért, a blokkréteg és a driver a tényleges hardver-hozzáférésért felel.

3.3.15 4.3 VFS mint kiterjesztési pont

A VFS nemcsak lemezes fájlrendszereket támogat: a `procfs`, `sysfs`, `devtmpfs`, `pipefs`, `sockfs` mind VFS-felületet ad. Ez a tervezés lehetővé teszi, hogy egy pipe ugyanúgy fájlleíróként jelenjen meg, mint egy ext4 fájl, vagy hogy a hálózati socket a `sockfs` inode-ján keresztül legyen elérhető.

A VFS interfész tehát **a kernel belső integrációs nyelve**. A hálózati stack egy `socket` rendszerhívás után `sockfs`-en keresztül ad vissza egy fájlleírót; a `send`, `recv`, `poll` hívások ugyanazzal a file- és poll-mechanizmussal dolgoznak, mint egy terminál vagy egy pipe.

3.3.16 4.4 Gyakori hiba

A kezdő kernelben gyakori, hogy a fejlesztő a fájlrendszer műveletét közvetlenül a rendszerhívás belépési pontjába kódolja, például egy `read` hívásban a lemezblokkokat számolja. Ez működhet egyetlen fájlrendszernél, de nem skálázódik. A helyes megoldás a műveleti táblák bevezetése: a VFS réteg kezeli a fájlleírókat, a jogosultságokat és a pozíciót, a fájlrendszer pedig a saját `read_iter`-jét adja.

3.3.17 5. Hálózati stack: proto-rendszer és socket-réteg

3.3.18 5.1 Szerkezet

A hálózati stack a kernel azon része, amely a hálózati protokollokat, a socketeket, a csomagfeldolgozást és a hálózati eszközmeghajtókat foglalja magában. Főbb rétegei:

- **socket-réteg:** a `socket`, `bind`, `listen`, `accept`, `send`, `recv` rendszerhívások kiszolgálása,
- **protokollcsaládok:** `AF_INET`, `AF_INET6`, `AF_UNIX`, `AF_NETLINK`,
- **transzportréteg:** TCP, UDP, SCTP,
- **hálózati réteg:** IPv4, IPv6, ICMP,
- **link-réteg:** Ethernet, loopback, bridge,
- **eszközmeghajtók:** NIC, hálózati virtuális eszközök.

A Linux kernelben a socket-réteg a `struct proto_ops` és a `struct proto` interfészekkel kapcsolódik a protokollokhoz. A TCP saját állapotgépe a `struct tcp_sock`-ban, a socket a `struct sock`-ban, a fájlleíró pedig a VFS `sockfs`-én keresztül kapcsolódik.

3.3.19 5.2 Interfész az ütemezőhöz

A hálózati stack erősen aszinkron. A `recv` hívás gyakran blokkol; ilyenkor a socket várakozási sora jelzi az ütemezőnek, hogy a folyamatot ki kell venni a futási sorból. Amikor egy csomag megérkezik, a hálózati megszakítás vagy softirq feldolgozza, a TCP állapotgép frissül, és a socket várakozási sorát felébreszti. A `recv` hívó ezután visszatérhet a felhasználói térbe.

A hálózati stackben az ütemezővel való kapcsolat tehát nemcsak a `recv` blokkolásakor jelenik meg, hanem a **kernel-throttling** és a **CPU-terheléselosztás** terén is: a NAPI például a csomagfeldolgozást a megszakításból a softirq kontextusba teszi, hogy az ütemező és a terheléselosztó jobban kezelhesse.

3.3.20 5.3 Interfész a memóriakezelőhöz

A hálózati stack intenzíven allokál puffereket. A `sk_buff` struktúra a csomagtárolás alapja:

- az `skb->head` a puffertérre mutat,
- az `skb->data` és `skb->len` az aktuális csomagszeletet jelöli,
- a `skb_shared_info` a fragmenteket és a zero-copy leírókat tartalmazza.

A hálózati stack `kmalloc`, `alloc_skb`, `page_frag_alloc` és más memóriakezelői interfészeket használ. A zero-copy optimalizálásnál felhasználói lapokat is lefoglalhat, ilyenkor a memóriakezelő `pin_user_pages` vagy `get_user_pages` hívásával rögzíti a fizikai lapokat.

A hálózati stack és a memóriakezelő közötti feszültség klasszikus példája a **pufferallokációs nyomás**: nagy hálózati forgalom esetén a kernelnek döntenie kell, hogy csomagot ejt-e, vagy a memóriakezelőt kényszeríti lap-visszanyerésre. A Linux ezt a `net.core.mem_default` és a `vm.min_free_kbytes` beállítások együttesével kezeli; a kernel az `__netdev_alloc_skb` hívásnál figyelembe veszi a memórianyomást.

3.3.21 5.4 Interfész a VFS-hez

A hálózati stack és a VFS között két fontos kapcsolat van:

1. **socket fájlleírók:** a `socket()` hívás egy `sockfs` inode-on keresztül ad vissza fájlleírót. A `poll`/`select` a VFS poll-mechanizmusát használja.
2. **AF_UNIX socketek:** ezek a fájlrendszer névtérben nem kötelező, de létrehozhatnak fájlbejegyzést; a kernel a `unix_bind` során a VFS-en keresztül hoz létre socket-speciális fájlt.

Ez a kapcsolat kiemeli, hogy a VFS nem egy „lemezes” alrendszer, hanem a kernel általános **objektum-névtér** és **I/O-multiplexáló** rétege.

3.3.22 6. Az alrendszerek közötti tipikus hívási lánc: `read` a hálózati socketből

A komponensek együttműködésének bemutatására vegyünk egy konkrét láncot, amely a négy alrendszer mindegyikét érinti:

1. A felhasználói folyamat `read(fd, buf, 4096)` hívást ad ki egy socketre.
2. A kernel a fájlleíróból feloldja a `file` objektumot; ez a VFS/sockfs hatásköre.
3. A `file->f_op->read_iter` hívás a socket-rétegbe vezet.
4. A socket-réteg a `struct sock` `recvmsg`-ét hívja.
5. Ha nincs adat, a folyamat bejegyzi magát a socket várakozási sorába, és `schedule()`-t hív. Itt lép be az ütemező.
6. Később a hálózati kártya megszakítása és a softirq feldolgozza a bejövő csomagot. Az `sk_buff` allokációja a memóriakezelő szolgáltatása.
7. A TCP ellenőrzi a csomagot, az adatot a socket fogadó pufferébe teszi, és `wake_up`-pal felébreszti a várakozó folyamatot.
8. Amikor a folyamat újra fut, a `recvmsg` kimásolja az adatot a felhasználói pufferbe a `copy_to_user` segítségével.
9. A rendszerhívás visszatér a beolvasott bájtok számával.

Ez a lánc a kernel **több komponensű tranzakciója**: az ütemező nélkül nincs várakozás, a memóriakezelő nélkül nincs puffer és felhasználói másolás, a VFS/sockfs nélkül nincs fájlleíró, a hálózati stack nélkül nincs adat.

3.3.23 7. Labor: kernelkomponens-térkép készítése

3.3.24 7.1 Cél

A labor célja, hogy egy emulált környezetben megfigyelje a négy alrendszer működését, és készítsen egy **komponens-térképet**: melyik rendszerhívás, kernelbelépési pont vagy tracepont melyik alrendszerhez tartozik, és milyen interfészen keresztül hívja a következőt.

3.3.25 7.2 Eszközök és konfiguráció

Az alábbi reprodukciós csomag QEMU-alapú, a hoszt valódi lemezét nem érinti:

```
CPU:         QEMU -cpu max, 4 vCPU
RAM:         2 GiB
Kernel:      linux-6.12.0-image-placeholder (fordított, nem éles)
Rootfs:      buildroot_placeholder
Tracepontok: ftrace function_graph, sched, kmem, vfs, skb
Napló:       trace.txt, dmesg-labor.txt
```

A pontos kernelverziót a saját laborban ellenőrizni kell; a fenti verziószám helyőrző.

3.3.26 7.3 Mérési lépések

1. Indítsa a QEMU-t a csomagolt `run.sh` szkripttel.
2. Kapcsolja be a tracepontokat:

```
echo function_graph > /sys/kernel/debug/tracing/current_tracer
echo 1 > /sys/kernel/debug/tracing/events/sched/sched_switch/enable
echo 1 > /sys/kernel/debug/tracing/events/kmem/kmalloc/enable
echo 1 > /sys/kernel/debug/tracing/events/vfs/vfs_read/enable
echo 1 > /sys/kernel/debug/tracing/events/skb/kfree_skb/enable
```

3. Futtasson egy egyszerű felhasználói programot, amely egy TCP socket megnyitásakor `read`-et hív, majd egy fájlt olvas. A program helyőrző adatot és nem valós hitelesítő adatot használjon.
4. Állítsa le a trace-et, és mentse a naplót.
5. Készítse el a komponens-térképet: minden releváns trace-eseményhez írja be, hogy az ütemező, memóriakezelő, VFS vagy hálózati stack alrendszerhez tartozik, és milyen interfészt használ.

3.3.27 7.4 Elvárt eredmény

A naplóban legalább a következő mintázat figyelhető meg:

- `sched_switch`: a `read` hívó folyamat BLOCKED állapotba kerül, egy másik folyamat fut.
- `kmalloc`: a socket bejövő `sk_buff` allokációja megtörténik.
- `vfs_read`: a fájlleíró-feloldás után a `read` eléri a VFS-t.
- `kfree_skb`: a csomag feldolgozása után a hálózati puffer felszabadul.

A mérési naplóban a **mért** értékeket külön kell jelölni a **várt/levezetett** értéktől. Például:

Esemény	Alrendszer	Várt szerep	Mért sor a naplóban
`sched_switch`	ütemező	`read` blokkol, másik folyamat fut	`comm=reader pid=123 state=S`
`kmalloc`	memóriakezelő	`sk_buff` allokáció	`bytes_alloc=2048`
`vfs_read`	VFS	fájlleíró-feloldás	`fd=3 count=4096`
`kfree_skb`	hálózati stack	csomagpuffer felszabadulása	`skbaddr=0xffff...`

3.3.28 7.5 Visszaállítás

A labor után a QEMU-lemezkép eldobható; a hoszt kernelét, bootloaderét és rendszerpartícióját nem módosítjuk. A trace-konfigurációt egy újraindítás alaphelyzetbe állítja; a szkript ezt dokumentálja.

3.3.29 8. Tipikus hibák és helyes megoldások

3.3.30 8.1 „Az ütemező közvetlenül a folyamatlistát használja”

Hibás gondolat: a blokkolás azt jelenti, hogy a fejlesztő saját maga láncolja ki a folyamatleírót egy globális listából.

Következmény: versenyhelyzet, elveszett ébresztés, inkonzisztens futási sor.

Helyes megoldás: a `wait_queue` és a `schedule()` hívása. Az ütemező birtokolja a futási sor adatszerkezetét; a többi alrendszer csak az állapotot és a várakozási okot adja meg.

3.3.31 8.2 „A fájlrendszer közvetlenül allokál felhasználói lapot”

Hibás gondolat: a fájlrendszer `read` metódusa közvetlenül a felhasználói pufferbe ír, akár `memcpy`-vel.

Következmény: access-ellenőrzés megkerülése, laphiba a kernelben, izolációs sérülés.

Helyes megoldás: a fájlrendszer a page cache-be vagy a kernel pufferébe ír; a felhasználói másolást a memóriakezelő `copy_to_user` interfésze végzi, hibaellenőrzéssel.

3.3.32 8.3 „A hálózati stack a VFS-től függetlenül kezeli a fájlleírókat”

Hibás gondolat: a socket fájlleírók külön névtérben élnek, és a `poll`/`select` felesleges a VFS-hez kötni.

Következmény: a `poll` rendszerhívás duplikált logikája, a `epoll` és a fájlleírók közötti átjárhatóság elvesztése.

Helyes megoldás: a socket fájlleírók a VFS `sockfs` rétegén keresztül jelennek meg, és a `file_operations` poll-mechanizmusát használják.

3.3.33 9. Önellenőrző kérdések

1. Egy folyamat blokkoló `read`-et hív egy pipe-ra. Melyik alrendszer tulajdonolja a fájlleírót, melyik a várakozási sort, és melyik a felhasználói pufferbe másolás hívását?
2. Miért helytelen a kernelben közvetlenül `memcpy`-t hívni felhasználói pointerre?
3. Rajzolja fel leegyszerűsítve a `read(fd, buf, 4096)` hívási láncát egy socket esetén, és jelölje be a négy alrendszer határait.
4. Milyen adatszerkezet köti össze a fájlleírót, az inode-ot és a fájlrendszer-műveleteket?
5. Hogyan ébred fel egy hálózati `recv` hívás? Melyik alrendszer jelzi az ütemezőnek, hogy a folyamat `READY` lett?

3.3.34 10. Összefoglalás

A kernel fő alrendszerei nem egymás mellett létező, elszigetelt modulok, hanem meghatározott interfészekkel összekapcsolt komponensek. Az ütemező a futásjogot és a várakozást menedzseli, a memóriakezelő a címfordítást és az allokációt, a VFS a fájlleírókat és az I/O-objektumokat, a hálózati stack a socketeket és a csomagfeldolgozást. A helyes kerneltervezés azon múlik, hogy ezek a határok a kódban is megjelenjenek: tulajdonjogok, műveleti táblák, várakozási sorok és felhasználói másolófüggvények formájában. A következő egység, `os009`, ezekre az interfészekre építve mutatja be a kernel és a felhasználói tér határát, az ABI-t és a driver modellt.

3.4 Kernel és felhasználói tér határa, ABI és driver modell

A kernel és a felhasználói tér közötti határ nem egyszerű adminisztratív vonal, hanem a hardver által kikényszerített védelmi mechanizmusok és a szoftveres egyezmények együtteséből álló szerződésrendszer. Ez a szerződés határozza meg, hogy egy felhasználói program miként kérhet szolgáltatást a kerneltől, milyen bináris formátumban kommunikálhat a rendszerrel, és hol helyezkednek el az eszközmeghajtók. A rosszul megtervezett határ biztonsági réseket, ABI-törést vagy elfogadhatatlan teljesítménycsökkenést okoz; a jól megtervezett határ viszont stabil, teljesítményképes és biztonságos alapot ad a teljes rendszer számára.

Ebben a fejezetben a védelmi határt, a privilegizált utasításokat és a védelmi gyűrűket tárgyaljuk, majd a rendszerhívási mechanizmust, az ABI-t és az eszközmeghajtók helyét vizsgáljuk. A fogalmi ismertetést egy ellenőrizhető driver-vázlat és egy mérési labor egészíti ki.

3.4.1 Védelmi határ, védelmi gyűrűk és privilegizált utasítások

Az operációs rendszer egyik alapvető feladata, hogy elkülönítse a megbízható kernelkódot a kevésbé megbízható felhasználói kódtól. Ezt a határt gyakran *védelmi határnak* (protection boundary) nevezzük. A határ két irányban működik: megvédi a kernelt a hibás vagy rosszindulatú felhasználói programoktól, és megvédi a felhasználói programokat egymástól. A hardver ezt a védelmet privilegizált végrehajtási módokkal, lapcímtárak által vezérelt memóriavédelemmel és speciális átlépési mechanizmusokkal valósítja meg.

A klasszikus Intel x86 architektúra négy védelmi gyűrűt definiál: Ring 0 a legmagasabb privilégium, Ring 3 a legalacsonyabb. A gyakorlatban az operációs rendszerek jellemzően csak a Ring 0-t (kernel mód) és a Ring 3-at (felhasználói mód) használják. A Ring 1 és Ring 2 létezik, de széles körben elterjedt operációs rendszerek nem, vagy alig használják őket, mert a két mód is elegendő a kernel és a felhasználói tér szétválasztásához. Az ARM processzorok ezzel szemben kivételi szinteket (Exception Level, EL) használnak: az EL0 a felhasználói mód, az EL1 a kernel mód, az EL2 a hypervisor, az EL3 pedig a biztonságos monitor. A RISC-V architektúra három privilegizált szintet definiál: Machine (M), Supervisor (S) és User (U). Egy tipikus monolitikus kernel S-módban fut, míg a felhasználói programok U-módban. A privilégiumszintek közötti különbséget a *privilegizált utasítások* adják.

A privilegizált utasítások olyan gépi utasítások, amelyeket csak magasabb privilégiumszinten lehet végrehajtani. Ide tartoznak például:

- a CPU vezérlőregisztereinek módosítása, mint az x86 `CR0` és `CR3` regiszterek, amelyek a lapozást és a címtárat vezérlik;
- a megszakítások globális engedélyezése vagy letiltása (`sti` és `cli` x86-on, `cpsid`/`cpsie` ARM-on);

- az I/O portok közvetlen elérése x86-on (`in`, `out`), kivéve, ha az I/O privilégium-térkép kifejezetten engedélyezi;
- a memóriakezelő egység (MMU) és a TLB vezérlése, a laphiba kezelése;
- speciális rendszerregiszterek írása, mint az ARM `SCTLR_EL1` vagy a RISC-V `sstatus`.

Ha egy felhasználói módban futó program mégis megpróbál végrehajtani egy privilegizált utasítást, a CPU kivételt vált ki. x86-on ez általában General Protection Fault (#GP), ARM-on szinkron kivétel EL0-ról EL1-re. Az operációs rendszer ilyenkor tipikusan megszakítja a folyamatot, és a felhasználói program számára `SIGILL` vagy `SIGSEGV` jelzést küld. Ez a hardveres védelem biztosítja, hogy a felhasználói kód ne tudja közvetlenül manipulálni a rendszer erőforrásait.

A memóriavédelem is a privilégiumszintekhez kapcsolódik. A lapok rendelkeznek olyan attribútumokkal, amelyek jelzik, hogy a lap elérhető-e felhasználói módból. x86-on a lapba bejegyző User/Supervisor bit, ARM-on az `AP` (Access Permissions) mezők, RISC-V-n a PTE U bitje határozza meg, hogy a felhasználói mód hozzáférhet-e az adott laphoz. A kernel lapjait általában supervisor-only attribútummal jelölik, így a felhasználói kód még akkor sem tudja közvetlenül olvasni vagy írni a kernel struktúráit, ha valamilyen módon megkapná a címeiket. A modern processzorok további védelmi mechanizmusokat is nyújtanak, mint az x86 SMAP (Supervisor Mode Access Prevention) és SMEP (Supervisor Mode Execution Prevention), amelyek megakadályozzák, hogy a kernel mód véletlenül felhasználói lapokról olvasson vagy onnan hajtson végre kódot. Ezek a hardveres védelem fontos elemei a kernel sebezhetőségeinek csökkentésében.

3.4.2 Rendszerhívási mechanizmus

A felhasználói programoknak szükségük van olyan műveletekre, amelyeket csak a kernel hajthat végre: fájlok megnyitása, hálózati csomagok küldése, memória lefoglalása, új folyamat indítása. Ezt a felületet *rendszerhívásoknak* (system call) nevezzük. A rendszerhívás a user és kernel tér közötti ellenőrzött belépési pont.

A rendszerhívás mechanizmusa architektúrafüggő. x86-32 biten a hagyományos mód a `int 0x80` szoftveres megszakítás, amely a CPU-t kernel módba kapcsolja, és a megszakítás-kezelő táblában megadott rutinra ugrik. x86-64 biten a modern, gyorsabb út a `syscall` utasítás, amely közvetlenül a kernel belépési pontjára ugrik, kevesebb overheaddel. ARM 64 biten a `svc #0` (Supervisor Call) utasítás vált ki szinkron kivételt, RISC-V-n pedig az `ecall` utasítás. Minden esetben a hardver átváltja a CPU-t a magasabb privilégiumszintre, és a kernel által beállított belépési kódra ugrik.

A rendszerhívás paramétereit regiszterekben adják át, és a visszatérési értéket is regiszterben kapják vissza. A regiszterek hozzárendelése része a bináris interfésznek (ABI), és szigorúan rögzített. Például x86-64 Linuxon a rendszerhívás száma a `rax` regiszterben van, az első paraméter `rdi`-ben, a második `rsi`-ben, a harmadik `rdx`-ben, a negyedik `r10`-ben, az ötödik `r8`-ban, a hatodik `r9`-ben. A `syscall` utasítás visszatérése után a `rax` regiszter tartalmazza a visszatérési értéket. Ha hiba történt, a kernel egy negatív értéket ad vissza, amely az `errno` hibakód negáltja; a libc ezt felismeri, beállítja az `errno` globális változót, és a felhasználói program számára -1-et ad vissza. Ez a konvenció lehetővé teszi, hogy a hiba azonosítható legyen, és a rendszerhívások visszatérési értéke ne ütközzön a hiba jelzésével.

A kernel belépéskor a következő lépéseket hajtja végre:

1. A hardver átvált kernel módra, elmenti a felhasználói program állapotát (programszámláló, veremmutató, regiszterek) a kernel veremre.
2. A kernel belépési kód kiválasztja a megfelelő rendszerhívás-kezelőt a rendszerhívás-táblából (syscall table) a `rax`-ben átadott szám alapján.

3. A kernel ellenőrzi a paramétereket, beleértve a felhasználói mutatók érvényességét, és szükség esetén lemásolja az adatokat a felhasználói térből a kernel térbe.
4. Végrehajtja a kért műveletet.
5. Az eredményt visszaadja a felhasználói térnek, és visszaállítja a felhasználói módot.

A rendszerhívások költsége jelentős lehet. Egy `syscall` utasítás önmagában gyors, de a teljes átmenet magában foglalja a regiszterek mentését, a veremváltást, a cache és a TLB hatásokat, valamint a kernel oldali ellenőrzéseket. Egy üres rendszerhívás (például `getpid` gyors elérési úttal) néhány száz CPU ciklust vehet igénybe, de ez erősen függ a hardvertől és a kernel verziójától. A pontos méréshez a laborban egy mérési módszert mutatunk.

3.4.3 Az ABI szerepe

Az *Application Binary Interface* (ABI) az a bináris szintű szerződés, amely meghatározza, hogy egy lefordított program hogyan kommunikál a rendszerrel és más lefordított kóddal. Az ABI részei:

- az adattípusok mérete és igazítása (például `int` 32 bites, `long` 64 bites x86-64 Linuxon, igazítási szabályok);
- a hívási konvenció (mely regiszterekben adják át a paramétereket, hogyan takarítják a vermet);
- a rendszerhívások számai és paraméterei;
- a bináris formátum (ELF, PE), a szimbólumfeloldás, a dinamikus linkelés;
- a kernel által használt struktúrák bináris elrendezése, ha azok a felhasználói térrel közösek (például `struct stat`, `struct sigaction`).

Az ABI különbözik az API-tól. Az API forrásszintű interfész, amely a fejlesztői dokumentációban és a fejlécfájlokban jelenik meg. Az ABI a bináris kompatibilitásra vonatkozik: egy lefordított program akkor működik egy rendszeren, ha a rendszer betartja azt az ABI-t, amire a programot lefordították. Ezért a kernel fejlesztői rendkívül óvatosan kezelik az ABI-t: egy meglévő rendszerhívás számát vagy egy közös struktúra elrendezését nem lehet megváltoztatni anélkül, hogy a meglévő binárisok eltörnének. Ha egy rendszerhívás viselkedését módosítani kell, új rendszerhívást vezetnek be, a régit pedig meghagyják a kompatibilitás érdekében. Például a Linux kernelben a `stat` rendszerhívásnak több verziója létezik (`stat`, `stat64`, `newfstatat`), mert az idők során a struktúra mérete és mezői változtak, de a régi binárisoknak továbbra is működniük kell.

Az ABI stabilitását architektúránként is meg kell őrizni. Egy x86-64-re lefordított program nem fut ARM rendszeren, mert az utasításkészlet és a hívási konvenció eltér. Az ABI tehát az architektúrához is kötött. A Linux kernel például minden támogatott architektúrához saját rendszerhívás-táblát és ABI-specifikációt tart fenn.

A gyakorlatban az ABI-törések tipikus okai:

- a rendszerhívás számának megváltoztatása egy új kernelverzióban;
- egy közös struktúra méretének vagy elrendezésének megváltoztatása;
- egy meglévő rendszerhívás paraméterlistájának bővítése vagy megváltoztatása;
- a hibakódok jelentésének megváltoztatása;
- a dinamikus linkelő viselkedésének módosítása.

A helyes megoldás az ABI-verziózás és a visszafelé kompatibilitás fenntartása. A kernel forráskódjában a rendszerhívás-táblák és a kapcsolódó definíciók jól elkülönítettek, és a kódban explicit kompatibilitási rétegek vannak a régi ABI-k támogatására.

3.4.4 Az eszközmeghajtók helye és a driver modell

Az eszközmeghajtó (device driver) az a szoftverkomponens, amely egy adott hardvereszközt vagy virtuális eszközt vezérel, és a kernel által elvárt interfészt valósítja meg. A driver feladata, hogy a hardver specifikus részleteit elrejtse a kernel többi része és a felhasználói programok elől. A driver tipikusan regisztrálja magát a kernel valamelyik alrendszerénél (például a karakteres eszközök rétegénél, a blokkrétegnél, a hálózati stacknél), és a felhasználói rendszerhívások ezen a rétegen keresztül érik el a hardvert.

A driver elhelyezkedése a kernelarchitektúra egyik legfontosabb tervezési döntése. Három fő modellt különböztetünk meg:

Monolitikus kernelben a driverek a kernel címterében futnak, kernel módban. Ez a hagyományos Linux modell. A driverek a kernel részei vagy dinamikusan betölthető modulok, de ugyanabban a privilégiumszinten és címterben futnak, mint maga a kernel. Előnye a teljesítmény: a driver hívása nem igényel user/kernel átmenetet, mert a kernelből közvetlenül hívjuk. Hátránya a megbízhatóság: egy hibás driver a teljes rendszert összedöntheti, mert nem izolált. Emiatt a driverek minősége és a kódellenőrzés kiemelten fontos.

Mikrokernelben a driverek user térben futnak, mint külön folyamatok. A mikrokernel csak a minimális szolgáltatásokat nyújtja (IPC, ütemezés, memóriakezelés, megszakítás-kezelés), a drivereket pedig user-módú szolgáltatási folyamatok valósítják meg. A driverek IPC-n keresztül kommunikálnak a felhasználói programokkal és a kernelmaggal. A hardverhez való hozzáférést a mikrokernel engedélyezi, gyakran speciális kernel-nyújtotta mechanizmusokkal. Előnye az izoláció: egy hibás driver nem dönti le a rendszert, újraindítható. Hátránya a teljesítmény: minden driver-művelet IPC-t és kontextusváltást igényel, ami jelentős költséget jelent. A híres mikrokernel-alapú rendszerek közé tartozik a QNX, a Minix 3 és a seL4.

Hibrid modellek léteznek, amelyek a kettőt kombinálják. A Linux kernelben például a driverek többsége kernel módban fut, de léteznek user-space driverek is: a FUSE (Filesystem in Userspace) lehetővé teszi fájlrendszerek user térben való megvalósítását, az UIO (Userspace I/O) és a VFIO keretrendszerek pedig lehetővé teszik, hogy bizonyos eszközöket user térben vezéreljenek. A hibrid modell a teljesítmény és az izoláció közötti egyensúlyt keresi.

A kernelben a driver interfészét a kernel által definiált műveleti struktúrák adják. A Linux kernelben a karakteres eszközök esetén a `struct file_operations` tartalmazza azokat a függvénypointereket, amelyeket a drivernek meg kell valósítania. Például:

```
/* Linux kernel modul vázlat - szemléltetés, nem futtatandó */
#include <linux/module.h>
#include <linux/fs.h>
#include <linux/uaccess.h>

#define DEVICE_NAME "mydev"
#define MY_MAGIC 0x4D59

static int mydev_open(struct inode *inode, struct file *file)
{
    printk(KERN_INFO "mydev: opened\n");
    return 0;
}

static ssize_t mydev_read(struct file *file, char __user *buf,
                          size_t count, loff_t *off)
{
    const char msg[] = "Hello from kernel\n";
    size_t len = sizeof(msg);
```

```
    if (off && *off != 0)
        return 0;
    if (count < len)
        len = count;
    if (copy_to_user(buf, msg, len))
        return -EFAULT;
    if (off)
        *off += len;
    return len;
}

static long mydev_ioctl(struct file *file, unsigned int cmd,
                        unsigned long arg)
{
    switch (cmd) {
    case 0: /* parancs: visszaad egy verziót */
        if (copy_to_user((void __user *)arg, &(int){1}, sizeof(int)))
            return -EFAULT;
        return 0;
    default:
        return -EINVAL;
    }
}

static const struct file_operations mydev_fops = {
    .owner = THIS_MODULE,
    .open = mydev_open,
    .read = mydev_read,
    .unlocked_ioctl = mydev_ioctl,
};

static int __init mydev_init(void)
{
    /* Regisztráció: major szám 240, minor 0 */
    if (register_chrdev(240, DEVICE_NAME, &mydev_fops) < 0)
        return -EIO;
    printk(KERN_INFO "mydev: registered\n");
    return 0;
}

static void __exit mydev_exit(void)
{
    unregister_chrdev(240, DEVICE_NAME);
    printk(KERN_INFO "mydev: unregistered\n");
}

module_init(mydev_init);
module_exit(mydev_exit);

MODULE_LICENSE("GPL");
MODULE_DESCRIPTION("Sample character device driver");
```

Ez a kód szemlélteti a driver és a kernel interfészét. A `file_operations` struktúra a karakteres eszközök rétegéhez köti a drivert. A felhasználói program a szokásos fájlrendszer-hívásokkal éri el az eszközt:

```
/* Felhasználói program vázlat - szemléltetés */
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/ioctl.h>
#include <errno.h>
#include <string.h>
```

```c
int main(void)
{
    char buf[64];
    ssize_t n;
    int fd = open("/dev/mydev", O_RDONLY);
    if (fd < 0) {
        fprintf(stderr, "open: %s\n", strerror(errno));
        return 1;
    }

    n = read(fd, buf, sizeof(buf));
    if (n < 0) {
        fprintf(stderr, "read: %s\n", strerror(errno));
        close(fd);
        return 1;
    }
    printf("olvasott %zd bájt: %.*s", n, (int)n, buf);

    int version = 0;
    if (ioctl(fd, 0, &version) < 0) {
        fprintf(stderr, "ioctl: %s\n", strerror(errno));
        close(fd);
        return 1;
    }
    printf("driver verzió: %d\n", version);

    close(fd);
    return 0;
}
```

A `read` rendszerhívás a VFS rétegen keresztül eljut a karakteres eszköz driveréhez, ahol a `mydev_read` függvény fut le. A `copy_to_user` függvény biztosítja, hogy a kernelből a felhasználói térbe való másolás során a kernel ellenőrzi a felhasználói mutató érvényességét; ha a mutató hibás, a függvény `-EFAULT` hibát ad vissza.

A driver modell a Linux kernelben ennél összetettebb. A modern kernelben a driverek nem közvetlenül a `register_chrdev`-et használják, hanem a busz- és eszközmodell rétegeit: platform driver, PCI driver, USB driver, stb. A driver egy adott buszhoz regisztrál, és a kernel az eszköz felfedezésekor (hotplug, device tree, ACPI) automatikusan párosítja a drivert az eszközzel. A driver egy `probe` függvényt valósít meg, amely az eszköz inicializálását végzi. Ez a modell lehetővé teszi az energiagazdálkodást, a felfüggesztést és az újraindítást, valamint a moduláris felépítést.

3.4.5 Rendszerhívás és driver interfész összekapcsolása

A fenti példa jól mutatja a rendszerhívás és a driver interfész közötti kapcsolatot. A felhasználói program `read` rendszerhívást hajt végre. A kernel a rendszerhívás belépési pontján azonosítja a `read` műveletet, majd az `fd` alapján megkeresi a fájlleíróhoz tartozó fájlt. A fájl struktúrája tartalmazza a `file_operations` mutatót, amelyet a driver adott meg. A kernel meghívja a megfelelő függvényt, esetünkben a `mydev_read`-et. A driver fut a kernel kontextusában, de a felhasználói folyamat nevében.

Fontos megkülönböztetni, hogy a driver nem közvetlenül a hardvert éri el a felhasználói program kérésére, hanem a kernel belső rétegein keresztül. Például egy blokkos eszköz driver a blokkréteghez regisztrál, és a felhasználói `read` egy fájlrendszeren keresztül végül blokkos olvasási kérelmet generál. A driver a kérést hardverparanccsá alakítja, és a hardver megszakítással

jelzi a befejezést. Ez a rétegzett felépítés teszi lehetővé, hogy a fájlrendszerek és a felhasználói programok eszközfüggetlenek legyenek.

3.4.6 Tipikus hibák és helyes megoldások

A védelmi határ és az ABI kezelése során számos tipikus hibát követhet el a fejlesztő. A következőkben összefoglaljuk a leggyakoribbakat, és bemutatjuk a helyes megoldást.

1. User pointer közvetlen használata kernelből. A kernel módban futó kód nem férhet hozzá közvetlenül a felhasználói térben lévő memóriaterülethez, mert a felhasználói lapok lehetnek nem jelen, vagy a mutató lehet érvénytelen. A helyes megoldás a `copy_from_user` és `copy_to_user` függvények használata, amelyek ellenőrzik a hozzáférést és lekezelik a laphibákat. Ha a kernel közvetlenül dereferál egy felhasználói mutatót, az súlyos biztonsági rést és rendszerösszeomlást okozhat.

2. ABI-törés. A rendszerhívás számának vagy a struktúrák elrendezésének megváltoztatása megtörheti a meglévő binárisokat. A helyes megoldás az ABI stabilitásának fenntartása: új rendszerhívások bevezetése a régiek megtartása mellett, struktúrák verziózása, és a felhasználói térrel közös definíciók karbantartása a `uapi` fejlécekben.

3. Hibás privilégiumszintű művelet. Ha egy driver user térben futó kódot próbál meg kernel módú művelettel végrehajtani, az kivételt vált ki. A helyes megoldás a megfelelő kernel API-k használata, illetve mikrokernel esetén a kernel által nyújtott portálokon keresztüli hozzáférés.

4. A driver nem megfelelő kontextusban alszik. A kernel bizonyos kontextusokban (például megszakítás-kezelőben vagy spinlock tartása mellett) nem alhat. A drivernek meg kell különböztetnie, hogy atomi kontextusban fut-e, és ennek megfelelően kell eljárnia (például `GFP_ATOMIC` memóriafoglalás, vagy a kérés elhalasztása workqueue-ba). Ez a téma a későbbi fejezetekben részletesebben előkerül, de már itt is érdemes felhívni rá a figyelmet.

5. A rendszerhívás visszatérési értékének hibás kezelése. A felhasználói programnak mindig ellenőriznie kell a rendszerhívás visszatérési értékét. A hibát az `errno` jelzi. Ha a program nem ellenőrzi, a hibás állapot észrevétlen maradhat.

3.4.7 Labor: rendszerhívási és driver-interfész vázlat

A fejezet laborja egy egyszerű, reprodukálható környezetben elkészíthető vázlat. A cél: egy felhasználói program rendszerhívással elér egy karakteres eszközmeghajtót, és megmérjük a rendszerhívás költségét.

Környezet és konfiguráció

- QEMU virtuális gép, 1 vCPU, 512 MiB RAM.
- Vendég operációs rendszer: Debian 12 (vagy újabb) minimal, Linux kernel 6.1 (a pontos verzió ellenőrizendő).
- Hoszt: a labor a hoszt lemezét nem módosítja.
- Build eszközök: `build-essential`, `linux-headers-$(uname -r)`.
- A labor munkakönyvtára: `lab-abidriver/`.

Kiinduló állapot

A vendég rendszer tiszta telepítésű, csak a szükséges csomagok vannak telepítve. A QEMU indítási parancsa:

```
qemu-system-x86_64 -m 512 -smp 1 -drive file=debian.qcow2,format=qcow2 \
  -nographic -enable-kvm
```

Lépések

1. Hozzuk létre a `mydev.c` kernelmodult a fenti mintakód alapján, de egyszerűsített `register_chrdev` nélkül – a modern `miscdevice` regisztrációt használjuk, hogy a major szám automatikus legyen. (A pontos API ellenőrizendő a kernel verzióhoz.)
2. Írjuk meg a `user_test.c` felhasználói programot, amely `open`, `read`, `ioctl` és `close` rendszerhívásokat hajt végre.
3. Fordítsuk le a modult és a felhasználói programot:

```
make
```

A Makefile tartalma (vázlat):

```
obj-m += mydev.o
KDIR := /lib/modules/$(shell uname -r)/build

all:
    $(MAKE) -C $(KDIR) M=$(PWD) modules
    gcc -Wall -O2 -o user_test user_test.c

clean:
    $(MAKE) -C $(KDIR) M=$(PWD) clean
    rm -f user_test
```

4. Töltsük be a modult, hozzuk létre az eszközfájlt, és futtassuk a tesztprogramot:

```
sudo insmod mydev.ko
major=$(grep mydev /proc/devices | awk '{print $1}')
sudo mknod /dev/mydev c "$major" 0
./user_test
```

Elvárt eredmény: a felhasználói program kiírja a kernel által küldött üzenetet és a driver verzióját. A `dmesg` logban megjelennek a `mydev` bejegyzések.

Mérés

A rendszerhívás költségének mérésére egy egyszerű benchmarkot futtatunk: 1 000 000 `getpid` rendszerhívás ciklusban, majd ugyanezt a `read` rendszerhívással a driveren. A méréshez `clock_gettime(CLOCK_MONOTONIC, ...)` órát használunk.

```
/* mérőprogram vázlat - szemléltetés */
#include <stdio.h>
#include <unistd.h>
#include <time.h>

#define ITER 1000000

int main(void)
{
    struct timespec t0, t1;
    long long dt;

    clock_gettime(CLOCK_MONOTONIC, &t0);
    for (int i = 0; i < ITER; i++)
        getpid();
    clock_gettime(CLOCK_MONOTONIC, &t1);
    dt = (t1.tv_sec - t0.tv_sec) * 1000000000LL + (t1.tv_nsec - t0.tv_nsec);
```

```
    printf("getpid átlag: %lld ns/hívás\n", dt / ITER);

    /* hasonló a read-re a mydev eszközön */
    return 0;
}
```

A mérési naplóban rögzíteni kell a hardvert, a kernel verziót, a fordítási beállításokat és a mérési módszert. A mért értékeket el kell különíteni a várt értékektől. Például:

Mérés	Várt tartomány	Mért érték
`getpid` rendszerhívás	100–500 ns	(mérve)
`read` a driveren	200–1000 ns	(mérve)

A várt tartományokat a szakirodalomból és korábbi mérésekből becsüljük, de a laborban a konkrét mért értéket kell dokumentálni. A labor eredménye egy jegyzőkönyv, amely tartalmazza a parancsokat, a kódot, a mérési naplót és a várt/mért összehasonlítást.

3.4.8 Önellenőrző kérdések

1. Mi a különbség a Ring 0 és a Ring 3 között, és miért van szükség erre a szétválasztásra?
2. Mi a rendszerhívás? Hogyan történik az átmenet a felhasználói térből a kernel térbe?
3. Mi az ABI, és miért fontos a stabilitása?
4. Hol futnak az eszközmeghajtók egy monolitikus kernelben és egy mikrokernelben? Melyek az előnyök és hátrányok?
5. Miért kell a `copy_from_user`/`copy_to_user` függvényeket használni a user pointerek kezelésére?

3.4.9 Összefoglalás

Ebben a fejezetben bemutattuk a kernel és a felhasználói tér közötti védelmi határt, a privilegizált utasításokat és a védelmi gyűrűket. Megvizsgáltuk a rendszerhívási mechanizmust, az ABI szerepét és az eszközmeghajtók elhelyezkedését. A gyakorlati példa egy egyszerű karakteres drivert mutatott be, amelyen keresztül a felhasználói program rendszerhívásokkal kommunikál a hardverrel. A labor célja a rendszerhívási és driver-interfész vázlat elkészítése és a teljesítmény mérése volt. A következő fejezet az operációs rendszertípusokkal foglalkozik, és erre az alapra épít.

4 Operációs rendszertípusok

4.1 Általános célú, szerver, desktop és mobil operációs rendszerek

4.1.1 10.1 A három nagy felhasználói kör: desktop, szerver és mobil

Az operációs rendszerek tömegesen telepített változatai három, jól elkülöníthető felhasználói környezetet szolgálnak ki: az asztali számítógépet, a szervert és a mobil eszközt. Bár mindhárom ugyanarra a közös kernel- és rendszerhívás-alapra épülhet, a felhasználói felület, az erőforrás-kezelés, a frissítési politika és az ökoszisztéma gyökeresen eltérő. Az eltérések nem esztétikai különbségek, hanem mérhető, üzemeltetési és fejlesztési következményekkel járó tervezési döntések.

A fejezet célja, hogy a három kategóriát egymás mellett, rendszeres szempontok szerint mutassa be. A fogalmi részt egy reprodukálható labor egészíti ki, amelyben ugyanazokat a metrikákat mérjük meg három emulált rendszeren. Az így kapott számok segítenek elválasztani a valós különbségeket a gyártói retorikától.

A tanulási célok eléréséhez három kérdésre adunk választ:

1. Milyen jellemzők határozzák meg a desktop, szerver és mobil rendszereket?
2. Hogyan kapcsolódik az interaktivitás, az átbocsátóképesség és a felhasználói élmény fogalma a mérhető rendszerjellemzőkhöz?
3. Miben tér el a frissítési modell és az ökoszisztéma, és ez milyen üzemeltetési hibákhoz vezet?

A fejezet nem ismétli meg a korábbi alapfogalmakat (kernel, rendszerhívás, ABI), és nem tárgyalja a következő egységben szereplő beágyazott és valós idejű rendszereket.

4.1.2 10.2 Általános célú és desktop operációs rendszerek

A desktop, azaz asztali operációs rendszer olyan általános célú rendszer, amelynek elsődleges felhasználója ember. A felhasználó interaktív alkalmazásokat futtat: szövegszerkesztőt, böngészőt, fejlesztőkörnyezetet, médialejátszót, játékot. A rendszernek ezért rövid válaszidőt kell biztosítania, még akkor is, ha a pillanatnyi terhelés nem egyenletes.

A desktop rendszer jellemzői:

- **Interaktív ütemezési prioritás:** az ütemező az aktív, előtérben lévő feladatokat részesíti előnyben. A háttérben futó folyamatok is kapnak CPU-időt, de a felhasználó kattintására vagy billentyűleütésére adott válasz késleltetése legyen a lehető legkisebb. A Linux desktop rendszerekben ez leggyakrabban a teljesen fair ütemező (CFS) vagy annak utódja, autotuning heurisztikákkal kiegészítve.

- **Gazdag felhasználói felület:** grafikus megjelenítő szerver, ablakkezelő, kompozitor, hangrendszer, értesítések, felületi eszközök. A felhasználói élmény szempontjából a 16,7 ms-os (60 Hz-es) képkocka-időkorlát lehetőleg nem léphető túl tartósan, mert ez vizuális akadozást okoz.
- **Széles hardvertámogatás:** a desktop rendszernek gyártók és eszközök ezreit kell felismernie. A driverek és a firmware-betöltő mechanizmusok a rendszer telepítésétől kezdve automatikusan működnek. A hardver cseréjekor a rendszernek újra kell konfigurálnia az eszközfát, és újra kell kötnie a meghajtókat.
- **Általános célú szolgáltatások:** fájlrendszer-támogatás, hálózati kliens- és szerverprotokollok, nyomtatás, távoli asztal, virtualizációs eszközök. A desktop rendszer nem optimalizált egyetlen feladatra, hanem sokféle munkaterhelést vállal elfogadható hatékonysággal.

A desktop kategória tipikus képviselői közé tartoznak az általános célú Linux-disztribúciók (például Ubuntu Desktop, Fedora Workstation, Debian GNOME-telepítés), valamint a zárt forrású Windows és macOS. A nyílt forrású rendszerek esetében a disztribúció a kernelt, a felhasználói tér csomagjait és a frissítési csatornákat együtt adja ki.

4.1.3 10.3 Szerver operációs rendszerek

A szerver operációs rendszer elsődleges felhasználója nem ember, hanem más program. A rendszer hálózati szolgáltatásokat nyújt: webkiszolgálást, adatbázis-tranzakciókat, fájlmegosztást, névfeloldást, hitelesítést, levelezést, konténerorchesztrációt. A felhasználói interakció rendszerint egy távoli adminisztrátor parancsain keresztül történik.

A szerver rendszer jellemzői:

- **Átbocsátóképesség-orientált ütemezés:** a cél nem egyetlen feladatra adott minimális válaszidő, hanem a másodpercenként feldolgozott kérések, tranzakciók vagy átvitt bájtok számának maximalizálása adott hardveren. A terhelés jellemzően sok párhuzamos kérésből áll, ezért a rendszernek jól kell skálázódnia több CPU-magon és nagy memórián.
- **Headless üzemmód:** a szervereken ritkán fut grafikus felület. A rendszer erőforrásait nem grafikus megjelenítésre, hanem a hasznos terhelésre fordítják. A memóriában nincs kompozitor, ablakkezelő, hangrendszer; a konfiguráció szöveges fájlokon és parancssori eszközökön alapul.
- **Hosszú rendelkezésre állás:** a szerver gyakran évekig fut újraindítás nélkül. A frissítések és a karbantartás ezért tervezett karbantartási ablakban vagy élő migrációval történik. A rendszernek támogatnia kell a hotplug eszközöket, a hálózati redundanciát és a tároló-replikációt.
- **Erőforrás-elszámolás és izoláció:** a szervereken több bérlő, konténer vagy virtuális gép osztozik. A rendszernek pontos CPU-, memória-, hálózati és I/O-korlátokat kell betartania, ezért a cgroups, namespaces és más izolációs mechanizmusok itt alapvetőbbek, mint egy átlagos desktopon.
- **Kiszámítható teljesítmény:** a szerverek esetében a legrosszabb eset is számít. Egy adatbázis-tranzakció 10 ms-os válaszideje alatt több ezer kérés szenvedhet, ezért a kernelnek mérsékelnie kell a hosszú lefutású feladatok, a memóriatömörítés vagy a naplóírás okozta késleltetéseket.

A szerver operációs rendszerek tipikus példái a hosszú támogatású (LTS) Linux-disztribúciók, a Windows Server, valamint a speciális, minimalista kiadások, például az Alpine Linux vagy a Debian minimal install. A közös pont a hosszú támogatási ciklus, a stabil ABI és a szerver-szolgáltatásokra optimalizált csomagkészlet.

4.1.4 10.4 Mobil operációs rendszerek

A mobil operációs rendszer nagy számú, kézi eszközön futó rendszer. Az eszközök erőforrása korlátozott, az energiaellátás akkumulátoros, a felhasználói bevitel érintőképernyős, a hálózati kapcsolat pedig gyakran cellás vagy ingadozó minőségű Wi-Fi.

A mobil rendszer jellemzői:

- **Szoros energiaköltségvetés:** a CPU, a rádió, a kijelző és a háttérfolyamatok fogyasztását a rendszer folyamatosan monitorozza és korlátozza. A doze, az app standby és a háttérkorlátozás mind arra szolgálnak, hogy a képernyő kikapcsolt állapotában a fogyasztás minimális legyen. A megszakítások ritkítása és a batch-feldolgozás az energiatakarékosság fontos eszköze.
- **Alkalmazás-sandbox:** minden alkalmazás külön UID alatt fut, saját adatterülettel, korlátozott hálózati hozzáféréssel és erőforrásokkal. A mobil operációs rendszerek ezt szigorúbban kényszerítik ki, mint a hagyományos desktopok, mert a telepített alkalmazások kockázatot jelentenek.
- **Többféle konkurrens felület:** a mobil rendszer egyszerre kezeli a telefonhívást, a navigációt, a média lejátszását és az üzenetküldő alkalmazásokat. Az előtérben futó feladat prioritást élvez, de a háttérben futó zenelejátszás vagy VoIP-hívás sem szakadhat meg.
- **Érintőképernyős interakció:** a bevitel folyamatos, érintés-alapú gesztusokból áll; a rendszernek 60–120 Hz-es mintavételezési és megjelenítési sebességet kell tartania, különben a görgetés akadozik. A válaszidő a felhasználói élmény szempontjából itt éppúgy kritikus, mint a desktopon, de a hardver lassabb, ezért az ütemező és a grafikus verem agresszívebb optimalizálást igényel.
- **Integrált ökoszisztéma:** a mobil operációs rendszer nem csupán kernel, hanem alkalmazás-áruház, hitelesítési réteg, felhő-szinkronizáció, push-értesítés, digitális fizetés és identitás-kezelés. Az alkalmazásfejlesztő nem közvetlenül a rendszerhívásokat használja, hanem a platform által biztosított API-kat.

A két meghatározó mobil platform az Android és az iOS. Az Android a Linux kernelre épül, de a felhasználói tér, a HAL (hardverabsztrakciós réteg) és a futtatókörnyezet erősen módosított. Az iOS zárt, a XNU kernelre épül, és a biztonsági modell, a sandbox és a frissítési politika szorosan integrált a hardverrel. Mindkettőnél a gyártói frissítések és a tanúsítás kulcsszerepet játszik.

4.1.5 10.5 Interaktivitás, átbocsátóképesség és felhasználói élmény

A három rendszerkategória eltérő elsődleges mérőszámokat optimalizál, de a fogalmak nem függetlenek: egy rosszul méretezett interaktív rendszer rontja az átbocsátóképességet, és egy túlzottan átbocsátóképesség-orientált szerver élvezhetetlen interaktív feladatot adhat, ha valaki mégis grafikus felületen dolgozik rajta.

Interaktivitás alatt azt értjük, hogy a rendszer milyen gyorsan reagál a felhasználói eseményekre. Mérőszáma a bemeneti késleltetés, a képkocka-idő, az alkalmazás-indítási idő, a háttérbe szorított alkalmazások előtérbe hozásának ideje. A desktop és mobil rendszerek esetében ezek a metrikák

fontosabbak, mint a maximális átbocsátóképesség, mert a felhasználó érzékelése nemlineáris: 100 ms alatt azonnalinak tűnik a válasz, 1 s fölött pedig már megszakad a figyelmi folyamat.

Átbocsátóképesség alatt az időegység alatt elvégzett hasznos munka mennyiségét értjük. Szerveren ez lehet HTTP-kérés/másodperc, adatbázis-tranzakció/másodperc, átvitt sávszélesség vagy replikált naplórekord/másodperc. A szerver operációs rendszer akkor optimális, ha a CPU, a memória, a hálózati stack és a tároló közösen, egyenletesen telített, és nincs elhanyagolt szűk keresztmetszet. A magas átbocsátóképesség gyakran magasabb átlagos késleltetéssel jár, mert a rendszer sorba állítja a kéréseket, és a háttérfeladatok nagyobb adagokban dolgoznak. Ezért a szervereknél a percentiliseket (p50, p95, p99) és a legrosszabb esetet is figyelni kell.

Felhasználói élmény (UX) a teljes érzékelt viselkedés: a felület folyamatossága, a telepítés egyszerűsége, a frissítések zavarmentessége, az akkumulátor-élettartam, a melegedés. A UX nem csupán esztétikai kategória; pszichofizikai mérésekkel és akadozásészlelési küszöbökkel jól leírható. Például a 60 Hz-es kijelzőn a 16,7 ms képkocka-idő túllépése már érzékelhető. A mobil rendszerek esetében az energiafogyasztás közvetlenül befolyásolja a UX-et, ezért az ütemező döntései olyan korlátok között mozognak, amelyeket a desktopon nem kell figyelembe venni.

A három fogalom kapcsolatát egy egyszerű képlet szemlélteti: ha egy interaktív feladat $D_{\max}$ határidőn belül nem kapja meg a CPU-t, a felhasználói élmény romlik, még akkor is, ha a rendszer átbocsátóképessége kiváló. Egy szerver esetében a túl agresszív interaktív prioritás csökkentheti a háttérfeladatok átbocsátóképességét, ezért a két cél között a rendszer erőforrás-elszámolási és ütemezési politikája teremt egyensúlyt.

4.1.6 10.6 Frissítési modell és ökoszisztéma

A frissítési modell határozza meg, hogyan jut el a hibajavítás, a biztonsági folt és az új funkció a futó rendszerhez. A desktop, szerver és mobil rendszerek ebben markánsan eltérnek.

Desktop frissítés: A hagyományos desktop rendszereken a felhasználó gyakran maga dönt a telepítésről. A frissítések időpontja, sorrendje és újraindítási igénye is a felhasználó hatáskörébe tartozik. Ez rugalmas, de biztonsági szempontból kockázatos: a felhasználó elhalaszthatja a frissítést, és a rendszer sebezhető marad. Az automatikus frissítés opcióvá vált, de nem kötelező.

Szerver frissítés: A szervereken a frissítés üzemeltetési folyamat, amelyet változáskezelés, tesztelés és visszagörgetési terv előz meg. A rendszer rendelkezésre állása fontosabb, mint az azonnali frissítés, ezért a frissítések ütemezettek, gyakran rolling vagy kék-zöld telepítéssel. A szerver operációs rendszerek LTS kiadásai biztonsági frissítéseket kapnak több évig, de a funkcionális frissítések ritkábbak. A stabil ABI itt kritikus, mert a vállalati szoftverek nem írhatók át minden kiadáshoz.

Mobil frissítés: A mobil rendszerek frissítése erősen központosított. A platform gyártója adja ki a frissítést, és az alkalmazás-áruház automatikusan telepíti az alkalmazásfrissítéseket. A rendszerfrissítések (OS update) a gyártótól függően lassabban érkeznek meg, és a hardver-illesztőprogramok zártsága miatt egy régebbi eszköz nem mindig kapja meg az új főverziót. A mobil ökoszisztéma a szoftverfrissítést a tanúsítással és a hardverintegrációval köti össze, ezért a frissítési gyakoriság és a támogatási időtartam eszközönként eltérő.

Ökoszisztéma: A desktop rendszer ökoszisztémája nyitottabb: a felhasználó bármilyen forrásból telepíthet szoftvert, de ez növeli a rosszindulatú kód kockázatát. A szerver rendszerek esetében a csomagkezelő által kezelt, aláírt tárolók a jellemzőek, és a minimális telepítés elve csökkenti a támadási felületet. A mobil rendszerekben az alkalmazás-áruház a domináns terjesztési csatorna, a telepítés előtti ellenőrzéssel és futás idejű korlátozásokkal.

4.1.7 10.7 Labor: három OS-típus összehasonlítása

A labor célja, hogy a három kategória jellemzőit mérhető módon mutassa be. A méréseket QEMU-ban, azaz emulált környezetben végezzük. A hoszt valódi lemezét, bootloaderét és rendszerpartícióját nem módosítjuk. Minden parancs a leírt emulált környezetben futtatandó, nem éles gépen.

4.1.8 10.7.1 Kiinduló állapot és konfiguráció

A reprodukciós csomag három virtuális gépet tartalmaz:

1. **Desktop minta:** Debian GNU/Linux 13 GNOME asztali telepítés, 4 vCPU, 8 GiB RAM, 32 GiB virtuális lemez, QXL grafikus eszköz.
2. **Szerver minta:** Debian GNU/Linux 13 minimal (headless), 2 vCPU, 2 GiB RAM, 8 GiB virtuális lemez, nincs grafikus eszköz.
3. **Mobil minta:** Android-x86 13 emulált környezet, 2 vCPU, 2 GiB RAM, 16 GiB virtuális lemez, táblagép módban.

A pontos verziószámok és lemezkép-hivatkozások a disztribúciók kiadási dokumentációjából ellenőrizendők. A mérések akkor reprodukálhatók, ha a QEMU-verzió, a CPU-modell és a lemezkép ellenőrzőösszege is rögzítve van.

A build- és futtatási parancs egy desktop esetben például így néz ki:

```
qemu-system-x86_64 \
  -machine q35 -accel tcg,thread=multi \
  -cpu max -smp 4 -m 8192 \
  -drive file=debian-desktop.qcow2,if=virtio,cache=writeback \
  -device virtio-vga -display gtk \
  -netdev user,id=net0 -device virtio-net-pci,netdev=net0 \
  -pidfile desktop.pid
```

A szerver esetében a grafikus eszközt és a 4 vCPU-t elhagyjuk, a `-display none` opciót használjuk, és konzolt a soros porton keresztül kapcsolunk. Az Android mintánál az emulátor saját konfigurációs felületét alkalmazzuk, ugyanúgy rögzített CPU- és memóriaértékekkel.

4.1.9 10.7.2 Mérési metrikák

Minden rendszeren az alábbi méréseket végezzük el:

- **Boot idő:** a QEMU indításától a felhasználói felület (desktop) vagy a login prompt (szerver) megjelenéséig eltelt idő. Mobil esetén a kezdőképernyő megjelenéséig.
- **Alap memória-lábnyom:** a kernel, a felhasználói tér és az aktív grafikus felület együttes memória-foglalása a boot után.
- **Háttérfolyamatok száma:** a boot után futó felhasználói és kernel-szálak összesített száma.
- **Interaktív válaszidő:** desktopon egy terminálablak megnyitásának ideje, mobilon egy képernyőgörgetés 60 képkockájának átlagos renderelési ideje. Szerveren egy helyi parancs (`/bin/true`) végrehajtási ideje.
- **Átbocsátóképesség:** szerveren egy rövid HTTP terhelés (például `wrk` vagy `ab`), desktopon ugyanaz a terhelés, de grafikus felülettel a háttérben. Mobil esetén ez a mérés nem releváns, ezért kihagyjuk.

Mindegyik mérést háromszor megismételjük, és a mediánt, a minimumot és a maximumot is rögzítjük. A mérési naplóban a hardver (hoszt CPU-modell, QEMU-verzió, pontos órajelfrekvencia), a fordítási és futtatási beállítások, valamint a mérési módszer is szerepel.

4.1.10 10.7.3 Várt és mért eredmény

A várt eredmény a három rendszerterv tipikus viselkedését tükrözi. A desktop minta a legnagyobb memória-lábnyommal és a legtöbb háttérfolyamattal indul, mert a grafikus felület és a felhasználói kényelmi szolgáltatások sok erőforrást igényelnek. A szerver minta lényegesen kisebb memória-lábnyommal és kevesebb folyamattal indul, de a HTTP-terhelés alatt magasabb átbocsátóképességet ér el, mert a CPU-t nem köti le a grafikus verem. A mobil minta a boot időben és a memória-lábnyomban köztes értéket mutat, de a legszigorúbb háttérkorlátozással.

A mért értékeket a naplóban külön kell feltüntetni a vártaktól. Ha a mért boot idő jelentősen eltér, annak oka lehet az emuláció sebessége, a lemezkép tömörítése, a virtio vagy a TCG mód beállítása. A labor értékelésekor nem a konkrét számok, hanem a tendencia és a magyarázat a lényeg.

4.1.11 10.7.4 Visszaállítás

A mérések után a virtuális gépeket töröljük, a lemezképeket a kiinduló ellenőrzőösszeggel visszaállítjuk, és a hosztot újraindítjuk, ha a QEMU bármilyen modult töltött be. A reprodukciós csomag tartalmazza a leállító parancsokat és a törlési lépéseket is.

4.1.12 10.8 Tipikus hibák és helyes megoldások

A három rendszerkategória összekeverése gyakori üzemeltetési hiba. Az alábbi esetek jól mutatják, mire kell figyelni.

1. Grafikus felület futtatása szerveren.
Egy szerverre telepített teljes asztali környezet felesleges memóriát, CPU-ciklust és támadási felületet jelent. A headless kiinduló telepítés és a szükséges szolgáltatások külön telepítése a helyes megoldás. A memória-lábnyom száz megabájtokkal csökkenthető, és a biztonsági frissítések felülete is kisebb.

2. Desktop rendszer szerver-munkaterhelésre optimalizálása.
Ha egy desktop rendszeren a grafikus felületet leállítjuk, és a rendszert kizárólag szerverként használjuk, a felhasználói élmény megszűnik, de a háttérben továbbra is futhatnak asztali szolgáltatások. A helyes megoldás a szerver kiadás telepítése, vagy a desktop kiadás minimális profilra cserélése.

3. Mobil rendszerről hiányzó frissítések figyelmen kívül hagyása.
A mobil eszközök gyakran nem kapnak már gyártói frissítést, de a felhasználó tovább használja őket. Ez biztonsági kockázat, mert a rendszer sebezhetőségei ismertek lehetnek. A helyes megoldás a támogatott eszköz használata, a frissítések mielőbbi telepítése, és a kritikus adatok leválasztása a nem támogatott eszközről.

4. A frissítési modell figyelmen kívül hagyása a laborban.
Ha a méréseket úgy végezzük, hogy közben a rendszer automatikus frissítést futtat, a mért értékek torzulnak. A mérés előtt a frissítési szolgáltatásokat le kell állítani, vagy a mérési naplóban jelezni kell, hogy a frissítés a háttérben futott. A reprodukálhatóság érdekében minden mérési sorozat előtt rögzítsük a csomagverziókat és a kernelverziót.

4.1.13 10.9 Önellenőrző kérdés

Egy üzemeltető a cég belső fejlesztői számára egyetlen virtuálisgép-sablont használ: minden fejlesztő kap egy teljes desktop Linuxot, majd ugyanazt a sablont telepítik a termelési webszerverekre is. A szerverek memóriája 2 GiB, a CPU 2 vCPU. A webszerver válaszideje p95 esetén 800 ms, a desktopon a terminál megnyitása 3 s. Mi a két fő hiba, és mit kellene tenni a helyes elkülönítésért?

A válasz a fejezet fogalmaira épül: a szerveren a grafikus felület és a desktop-szolgáltatások foglalják az erőforrásokat, a desktopon pedig a szerver-munkaterheléshez képest túl sok interaktív háttérfolyamat fut. A helyes megoldás két külön sablon: egy minimális szerver és egy desktop munkakörnyezet, eltérő frissítési és erőforrás-politikával.

4.1.14 10.10 Összefoglalás

A desktop, a szerver és a mobil operációs rendszer ugyanabból a közös rendszerarchitektúrából indul, de eltérő mérnöki célokat szolgál. A desktop az emberi interaktivitásra, a szerver az átbocsátóképességre és a rendelkezésre állásra, a mobil pedig az energiahatékonyságra, a sandboxra és az integrált ökoszisztémára optimalizál. A frissítési modell mindháromnál más kockázatot és más üzemeltetési gyakorlatot igényel. A laborban mért boot idő, memória-lábnyom és válaszidő az elméleti jellemzők kézzelfogható, reprodukálható vetületét adja. A következő egységben ezeket a fogalmakat visszük tovább a beágyazott és valós idejű rendszerek irányába, ahol a válaszidő már nem csupán felhasználói élmény, hanem biztonsági és helyességi követelmény.

4.2 Beágyazett, valós idejű és biztonsági kritikus rendszerek

A beágyazott rendszerek tervezésekor az első és legfontosabb kérdés nem az, hogy „milyen gyors a processzor”, hanem az, hogy „meddig tart a válasz a legrosszabb esetben”. Ez a szemléletbeli különbség választja el a valós idejű rendszereket az általános célú operációs rendszerektől, és ez indokolja, hogy a beágyazott, valós idejű és biztonsági kritikus rendszereket külön kategóriaként kezeljük. Egy asztali operációs rendszer tipikus optimalizálási célja az átlagos teljesítmény, a rendszermag pedig dinamikus erőforrás-gazdálkodással, nagy lapozással, agresszív gyorsítótárazással és lazy kiértékeléssel igyekszik növelni a felhasználó érzékelt sebességét. Ezzel szemben egy beágyazott légzsákvezérlő, motorvezérlő vagy orvosi infúziós pumpa esetében az átlag nem elég: egyetlen késő válasz is katasztrofális következménnyel járhat.

Ebben az alfejezetben a determinizmus, az erőforrás-korlátok és a minősítés szerepét járjuk körül. Megkülönböztetjük a hard real-time, soft real-time és best effort kategóriákat, áttekintjük a biztonsági integritás és tanúsítás alapfogalmait, majd egy kis, ellenőrizhető példán alkalmazzuk az eszköztárat. A fejezet végén egy követelményprofil-készítő labor segít abban, hogy a fogalmakat ne csak elméletben, hanem mérési naplóval dokumentálva is rögzítsük.

4.2.1 Determinizmus, erőforrás-korlátok és minősítés

A determinizmus a valós idejű rendszerek központi tulajdonsága. Két oldalról is értelmezhetjük. Az **időbeli determinizmus** azt jelenti, hogy egy esemény bekövetkezése és az arra adott rendszerszintű válasz közötti késleltetés egy ismert, felső korláttal rendelkezik, és ez a korlát minden megengedett körülmény között teljesül. A **logikai determinizmus** azt jelenti, hogy ugyanazon bemenetekre és ugyanazon ütemezési döntésekre a rendszer ugyanazt az állapotváltozást és kimenetet állítja elő. Egy megszakítás-vezérelt beágyazott vezérlőben mindkettőre szükség van:

a vezérlési algoritmus kiszámítható lefutása mellett az időzítési viselkedés is előre elemezhető kell legyen.

A determinizmust nem szabad összekeverni a sebességgel. Egy rendszer lehet nagyon gyors átlagosan, mégsem determinisztikus, ha a legrosszabb esetű válaszideje korlátlan vagy túl nagy szórást mutat. Például egy általános célú operációs rendszerben egy hálózati csomag feldolgozása néhány mikroszekundumtól több száz ezredmásodpercig is terjedhet, mert a rendszer közben laphibát, lemez I/O-t, energiatakarékos állapotváltást vagy más feladatütemezési döntést hajthat végre. Ez a szórás, azaz a jitter, egy beágyazott szabályozási hurokban elfogadhatatlan lehet.

A determinizmus méréséhez három alapfogalmat kell megkülönböztetni:

- **Latency:** az esemény bekövetkezése és a válasz kezdete közötti késleltetés.
- **Jitter:** a késleltetés szórása, azaz a mért latency értékek ingadozása.
- **WCET (Worst-Case Execution Time):** egy adott feladat vagy kódrészlet legrosszabb esetű futási ideje adott hardveren, adott fordítási beállítások mellett.

A determinisztikus tervezés célja, hogy a latency, a jitter és a WCET értékei számszerűsíthetők legyenek, és minden egyes időzítési követelményre teljesüljön a felső korlát.

A **erőforrás-korlátok** a beágyazott rendszerek másik meghatározó jellemzője. Míg egy szerver operációs rendszer tipikusan több száz gibibájtnyi memóriával, tucatnyi maggal és szinte korlátlan háttértárolóval rendelkezik, egy beágyazott célhardver gyakran néhány száz KiB vagy néhány MiB RAM-ot, egyetlen alacsony órajelű magot és korlátozott flash-tárat kínál. A korlátok közé tartozik:

- **CPU-idő:** az órajelfrekvencia és a magok száma rögzített, a teljes feldolgozási kapacitás előre ismert.
- **Memória:** a RAM és a flash mérete, a heap és a vermek mérete, valamint a statikus adatok elhelyezkedése explicit tervezést igényel.
- **Energia:** az akkumulátoros vagy energia-betakarításos eszközökben az energiafogyasztás és a hőtermelés korlátot szab az órajelnek és a működési módoknak.
- **Perifériák és megszakítások:** a beágyazott processzorokon a megszakítási vonalak száma, a DMA-csatornák és a hardver-időzítők száma korlátozott.

Ezek a korlátok azt jelentik, hogy a rendszer erőforrásait **statikusan vagy előre megtervezetten** kell gazdálkodni. Az általános célú operációs rendszerekben megszokott dinamikus allokáció, túlzott virtuális memória, lazy inicializálás vagy túl nagy pufferméretek nem illeszthetők be egy olyan kernelbe, ahol minden bájt és minden mikroamperszámít. A beágyazott operációs rendszer (ismert rövidítéssel RTOS, ha valós idejű tulajdonságokkal is rendelkezik) gyakran kisebb kód- és adat-lábnyommal, kevesebb absztrakcióval és szorosabb hardverközeliséggel működik.

A **minősítés** szerepe akkor válik meghatározóvá, amikor a rendszer hibája emberéletet, vagyonbiztonságot vagy környezetet veszélyeztet. A biztonsági kritikus rendszerekben a funkcionális biztonság (functional safety) nem a véletlenszerű meghibásodások kizárását jelenti – az lehetetlen –, hanem annak igazolását, hogy a rendszer a felismert hibák ellenére is biztonságos állapotba kerül vagy ott marad. A minősítés tehát egy **bizonyítási és dokumentációs folyamat**: a fejlesztőnek meg kell mutatnia, hogy a rendszer eleget tesz a rá vonatkozó szabványnak, a követelmények nyomon követhetők, a tesztelés és a statikus elemzés pedig lefedi a kritikus kódrészeket.

A minősítés nem helyettesíti a tervezést, hanem rákényszeríti a fejlesztőt arra, hogy a tervezési döntéseket nyomon követhető, ellenőrizhető és reprodukálható módon rögzítse. A beágyazott rendszerek esetében a minősítés jellemzően a következő területekre terjed ki:

- a biztonsági követelmények lebontása és hozzárendelése a szoftvermodulokhoz;

- a fejlesztési életciklus meghatározott fázisai (tervezés, implementáció, tesztelés, karbantartás);
- a kódolási szabványok (például a MISRA C szabályrendszere) betartása;
- a statikus és dinamikus elemzések, tesztek és lefedettségi mérőszámok;
- a verziókezelés, a visszakövethetőségi mátrix és a konfigurációmenedzsment.

Fontos hangsúlyozni, hogy a minősítés önmagában nem garantálja a helyes működést, de biztosítja azt a keretet, amelyben a hibákat szisztematikusan fel lehet tárni, és a maradék kockázatot dokumentált döntéssel lehet elfogadni.

4.2.2 Hard real-time, soft real-time és best effort

A valós idejű rendszereket gyakran a határidők megszegésének következménye alapján osztályozzák. A három alapkategória – hard real-time, soft real-time és best effort – közötti különbség nem a „gyorsaságban", hanem a határidő-sértés elfogadhatóságában van.

Hard real-time rendszerben a határidő túllépése rendszerhibának minősül, és súlyos következménnyel járhat: baleset, sérülés, anyagi kár vagy a küldetés elvesztése. Ilyen például a légzsákvezérlő, a repülőgép fly-by-wire rendszere, a gyári robotkar vészleállító áramköre vagy a pacemaker. A hard real-time követelményt nem lehet úgy értelmezni, hogy „a válasz általában időben érkezik"; a tervezésnek a legrosszabb esetre is igazolnia kell a határidő betartását. Ehhez a rendszer minden erőforrásigényét ismerni kell, és a feladatütemezést, a megszakításokat, a buszhozzáféréseket és az energiaellátást együttesen kell elemezni.

Soft real-time rendszerben a határidő túllépése káros, de nem katasztrofális: a szolgáltatás minősége romlik, de a rendszer tovább működik. Ide sorolható például egy videokonferencia-alkalmazás képkockafeldolgozása, egy hálózati router csomagtovábbítása vagy egy interaktív médiaeszköz vezérlője. A soft real-time tervezésnél is cél a késleltetés minimalizálása és a jitter csökkentése, de a legrosszabb esetű elemzés helyett gyakran statisztikai jellemzőket, például percentiliseket használnak: a 99,9. percentilis késleltetés kisebb legyen egy küszöbnél. A hard real-time rendszerben a 100. percentilis a lényeges.

Best effort rendszerben nincs szigorú időzítési garancia: az operációs rendszer és az alkalmazások a lehető legjobban igyekeznek gyorsan válaszolni, de egyetlen határidő megszegése sem minősül hibának. Ide tartozik a legtöbb általános célú asztali és szerver rendszer. A best effort nem jelenti azt, hogy a rendszer lassú – egy jól hangolt Linux kernel nagyon alacsony átlagos késleltetést tud elérni –, de nem ad felső korlátot a legrosszabb esetre.

A három kategória közötti határ nem mindig éles. Egy gyakorlati rendszerben lehetnek hard real-time és best effort komponensek is, például egy autóipari vezérlőegységben a motorvezérlő szabályozási hurka hard real-time, míg a diagnosztikai naplózási feladat best effort. Ilyenkor a rendszerarchitektúrának gondoskodnia kell arról, hogy a best effort feladatok ne veszélyeztessék a hard real-time feladatok időzítését – például prioritásos ütemezéssel, elkülönített megszakítási tartományokkal vagy hardveres erőforrás-partíciókkal.

A tervezési gyakorlatban a kategóriához társítanak egy **határidő-modellt**. Egy P_i folyamat vagy taszk periódusát T_i, legrosszabb esetű CPU-igényét C_i, relatív határidejét pedig D_i jelöli. Hard real-time elemzésnél tipikusan $D_i \leq T_i$, és az ütemezhetőség bizonyításához azt kell megmutatni, hogy minden taszk minden periódusában a D_i határidőn belül megkapja a szükséges C_i CPU-időt. Egy gyakran használt, egyszerű ellenőrzési eszköz a rate-monotonic ütemezés felső korlátja, amely szerint ha minden taszk prioritása a periódusával fordítottan arányos, és a CPU-kihasználtság nem haladja meg az

$$U = \sum_{i} \frac{C_i}{T_i} \leq n(2^{1/n} - 1)$$

korlátot, akkor a rendszer ütemezhető. Ez a képlet a mi céljainkra szemlélteti a determinisztikus elemzés gondolatát, de a részletes valós idejű ütemezési algoritmusokkal és azok bizonyításával későbbi fejezet foglalkozik.

4.2.3 Biztonsági integritás és tanúsítás

A biztonsági kritikus rendszerek fejlesztését szabványok és tanúsítási eljárások keretezik. A szabványok iparáganként eltérnek, de közös logikájuk az, hogy a rendszer hibáinak kockázatát kategóriákba sorolják, és az egyes kategóriákhoz eltérő fejlesztési szigorúságot rendelnek. A **biztonsági integritási szint (Safety Integrity Level, SIL)** egy ilyen kategóriarendszer: a SIL 1-től a SIL 4-ig terjedő skálán a magasabb szint szigorúbb követelményeket jelent. Hasonló szerepet tölt be az autóipari ISO 26262 szabványban az ASIL A–D besorolás, vagy a repülőgépek szoftverfejlesztésére vonatkozó DO-178C DAL szintjei.

A biztonsági integritás nem a szoftver „jóságát” méri, hanem azt a **kockázatcsökkentési igényt**, amelyet a rendszer környezete és lehetséges hibaeseményei indokolnak. Egy alacsony kockázatú funkcióhoz elegendő lehet egy alapos tesztelés és egyszerű kódolási szabály; egy magas kockázatú funkcióhoz formális specifikáció, független verifikáció, strukturált kód, redundancia és kimerítő tesztlefedettség szükséges.

A tanúsítási folyamat során a fejlesztőcsapatnak be kell mutatnia:

- a biztonsági követelmények teljes és nyomon követhető levezetését;
- a tervezési döntések indoklását és kockázatelemzését;
- az implementáció megfelelését a tervezésnek és a kódolási szabályoknak;
- a tesztek, statikus elemzések és formális módszerek lefedettségét;
- a konfiguráció- és változáskezelést, beleértve a hibajavítások visszavezetését.

A tanúsítás tehát nem egyetlen dokumentum, hanem egy **életciklus-szemléletű bizonyítási csomag**. A beágyazott rendszereknél gyakran megkövetelik a MISRA C irányelvek betartását, mert ezek a szabályok kiküszöbölik a C nyelv számos nem determinisztikus vagy hibalehetőséget rejtő konstrukcióját. Hasonlóképpen kerülik a dinamikus memóriaallokációt a kritikus kódrészekben, mert a heap fragmentációja és az allokációs idő szórása nem egyeztethető össze a determinisztikus viselkedéssel.

Hangsúlyozni kell, hogy a tanúsítás **nem garancia** arra, hogy a rendszer hibamentes. A tanúsítás azt igazolja, hogy a fejlesztési folyamat megfelelt a szabvány előírásainak, és a maradék kockázatokat dokumentáltan elfogadták. A gyakorlatban a tanúsított rendszerekben is előfordulnak hibák, de ezek feltárásának valószínűsége és súlyossága alacsonyabb, mert a folyamat szisztematikusan keresi a hibákat.

4.2.4 Gyakorlati minta: egy egyszerű RTOS-taszk profil

A fogalmak alkalmazásához tekintsünk egy kis, de ellenőrizhető példát. Egy emulált beágyazott vezérlőnek három feladata van:

- P_1: szenzor-mintavételező taszk, periódus $T_1 = 1$ ms, CPU-idő $C_1 = 0{,}2$ ms, relatív határidő $D_1 = 1$ ms.
- P_2: szabályozó hurok, $T_2 = 2$ ms, $C_2 = 0{,}4$ ms, $D_2 = 2$ ms.

- P_3: diagnosztikai naplózó, $T_3 = 10$ ms, $C_3 = 1$ ms, $D_3 = 10$ ms.

A cél annak ellenőrzése, hogy a rendszer hard real-time módon ütemezhető-e. A rate-monotonic ütemezés szerint a taszkok prioritása $P_1 > P_2 > P_3$, mert $T_1 < T_2 < T_3$. A CPU-kihasználtság:

$$U = \frac{0{,}2}{1} + \frac{0{,}4}{2} + \frac{1}{10} = 0{,}2 + 0{,}2 + 0{,}1 = 0{,}5.$$

A három taszkra vonatkozó elméleti felső korlát $3(2^{1/3} - 1) \approx 0{,}779$, így az $U = 0{,}5$ érték kielégíti a feltételt. A rendszer tehát ütemezhető. Ezt a Gantt-diagrammal is ellenőrizhetjük az első 10 ms-os periódusban: a P_1 minden 1 ms-ban lefut, P_2 minden 2 ms-ban, P_3 pedig a maradék időben. A P_3 futása megszakítható, ha egy magasabb prioritású taszk aktiválódik.

Egy valós mérési jegyzőkönyvben nem elég a képlet, hanem meg kell adni a futtatási környezetet is: az emulátor típusát és verzióját, a CPU-modellt, az órajelfrekvenciát, a fordító verzióját, az optimalizálási szintet, a mérési módszert (például ciklusszámláló vagy időbélyeg). A mért latency és jitter értékeket össze kell vetni a várt elméleti értékekkel. Ha például a P_1 taszk átlagos késleltetése 5 s, de a mért legrosszabb eset 120 s, akkor a jitter nagy, és meg kell vizsgálni, mi okozza a kiugrást – lehet, hogy egy alacsonyabb prioritású taszk túl hosszú kritikus szakaszban blokkolja a magasabb prioritásút.

4.2.5 Tipikus hibák és azok elkerülése

A beágyazott és valós idejű rendszerek tervezése során több visszatérő hibatípussal találkozunk. Az alábbiakban a legjellemzőbbeket tekintjük át, mindegyiknél röviden megindokolva, miért helytelen a megoldás, és mi a helyes eljárás.

1. **Nem mért WCET-re támaszkodó tervezés.** Gyakori hiba, hogy a fejlesztő a taszk futási idejét egyetlen, átlagos méréssel becsli, és abból indul ki, hogy „úgyis belefér”. A legrosszabb esetű futási időt statikus elemzéssel vagy kimerítő mérési sorozattal kell meghatározni, figyelembe véve a cache hideg indítását, a legrosszabb adatbevitelt és a hardver állapotát.

2. **Dinamikus memóriaallokáció a kritikus útvonalon.** A `malloc` és `free` futási ideje erősen változó, és a heap fragmentációja hosszú távon akár kimerítheti a memóriát. Kritikus rendszerekben a dinamikus allokációt kerülik, vagy legfeljebb előre lefoglalt poolokból, determinisztikus időben történő foglalást engednek.

3. **Túl hosszú megszakításkezelő.** A megszakításkezelő (ISR) gátolja az azonos vagy alacsonyabb prioritású megszakításokat, ezért ha hosszú feldolgozást végez, a rendszer válaszideje megnő. A helyes minta: az ISR csak a legszükségesebb műveletet végzi el, majd egy magas prioritású taszkot ébreszt, és a feldolgozás folytatódik a normál ütemezői környezetben.

4. **A watchdog hiánya vagy helytelen használata.** A watchdog egy hardver- vagy szoftveridőzítő, amelynek rendszeresen „etetni” kell, ellenkező esetben újraindítja a rendszert. Ha a watchdog túl hosszú időre van állítva, vagy ha a taszkok nem frissítik megfelelően, a rendszer hibás állapotban maradhat. A watchdog etetését nem szabad kritikus feldolgozási ciklus közben, megfontolás nélkül elvégezni; azt kell igazolni, hogy a rendszer csak akkor „eteti” a watchdodot, ha minden kritikus funkció egészséges.

5. **Priority inversion figyelmen kívül hagyása.** Ha egy alacsony prioritású taszk birtokol egy olyan erőforrást, amelyre egy magas prioritású taszk vár, akkor a közepes prioritású taszk megelőzheti az alacsony prioritásút, és ezzel késleltetheti a magas prioritású taszkot. A jelenség részletes elemzése a valós idejű ütemezés fejezetéhez tartozik, de a tervezés során már itt is fel kell ismerni, és a megoldás valamilyen örökléses vagy prioritás-mennyezetes protokoll lehet.

6. **Nincs mérési napló és reprodukciós csomag.** A determinizmus igazolása mérések nélkül puszta feltételezés. Minden méréshez rögzíteni kell a hardver-, szoftver- és konfigurációs környezetet, a mérési módszert, a nyers adatokat és a feldolgozás módját. Enélkül a következtetés nem reprodukálható.

7. **A biztonsági követelmények lebontásának elmulasztása.** Ha a rendszerszintű biztonsági célokat nem bontják le az egyes modulokra, a fejlesztő nem tudja megmondani, melyik hiba milyen súlyos következménnyel jár. A biztonsági elemzés (például FMEA vagy hibafa) nem választható el a szoftvertervezéstől.

Ezek a hibák mind visszavezethetők egy közös gyökérre: a determinizmus és a korlátok figyelmen kívül hagyására. A helyes megoldás mindig az, hogy a tervezés során explicit időzítési és erőforrás-költségvetést készítünk, és minden döntést méréssel vagy elemzéssel igazolunk.

4.2.6 Labor: beágyazott és RTOS követelményprofil készítése

A fejezet laborja egy **követelményprofil** elkészítése egy képzeletbeli, de reálisan specifikált beágyazott vezérlőhöz. A labor célja, hogy a tanuló a determinizmus, az erőforrás-korlátok és a minősítés fogalmait egy konkrét, reprodukálható jegyzőkönyvben alkalmazza. A labor nem igényel valódi hardvert; emulált környezetben, például egy QEMU-ban futtatott minimalista RTOS-képpel, vagy akár egy papíralapú tervezési jegyzetfüzetben is elvégezhető. A lényeg a rögzített konfiguráció és a mérési napló.

4.2.7 Kiinduló állapot

A laborhoz válasszunk egy fiktív beágyazott eszközt, például egy „okos szelepvezérlőt”, amelynek a következő funkcionális követelményei vannak:

- A bemeneti nyomásszenzort 1 ms periódussal kell mintavételezni.
- A szabályozási algoritmusnak 2 ms periódussal újra kell számolnia a szelepállást.
- A diagnosztikai naplózás 10 ms periódussal rögzíti az állapotot.
- A rendszernek hard real-time jellemzőkkel kell bírnia, mert a szelep hibás állása túlnyomást okozhat.

A hardver konfiguráció legyen rögzített: egy emulált ARM Cortex-M4 mag, 64 MHz órajel, 128 KiB RAM, 512 KiB flash, két hardver időzítő, egy megszakítás-vezérelt UART interfész. A fordítási beállítások: GCC 12.3, `-O2`, `-Wall -Wextra`, linkelési szkript statikus lefoglalással.

4.2.8 Lépések

1. **Időzítési követelmények táblázata.** Készítsünk táblázatot a taszkokról: név, periódus T_i, legrosszabb esetű CPU-idő C_i, relatív határidő D_i, prioritás. A C_i értékét vagy mérésből, vagy előzetes becslésből rögzítsük, és jelöljük meg, hogy mért vagy várt érték.

2. **CPU-kihasználtság számítása.** Adjuk meg az U kihasználtságot, és ellenőrizzük a korlátot. Ha a kihasználtság túl magas, javasoljunk optimalizálást (például a mintavételezési frekvencia csökkentését vagy a naplózási taszk ritkítását).

3. **Erőforrás-költségvetés.** Rögzítsük a RAM- és flash-méreteket, a statikus adatok, a vermek, a heap (ha van) és a megszakítási verem méretét. Ellenőrizzük, hogy a méretek belefférnek a hardverbe, és hogy a heap használata determinisztikus-e.

4. **Mérési napló.** Ha emulált környezetben futtatunk, mérjük meg minden taszk latency-jét legalább 1000 perióduson keresztül. A mérési módszer legyen például egy DWT ciklusszámláló, amely az órajelre pontos időbélyeget ad. Rögzítsük a minimális, maximális és átlagos latency-t, valamint a jittert. A várt értékeket a tervezési táblázatból határozzuk meg, a mért értékeket pedig külön oszlopban tüntessük fel.

5. **Biztonsági integritási besorolás.** Végezzünk egyszerű kockázatelemzést: mi történik, ha egy taszk késik? Adjunk a rendszernek egy SIL vagy ASIL besorolást indoklással. Javasoljunk a besorolásnak megfelelő intézkedéseket: például watchdog, statikus memória, kódolási szabály, független teszt.

6. **Hibaanalízis.** Azonosítsunk legalább három tipikus hibát a fenti listából, és írjuk le, hogyan kerülhetők el a konkrét rendszerben. Például a priority inversion elkerülése érdekében a szabályozó taszk és a naplózó taszk ne osszon meg közös erőforrást, vagy ha mégis, használjunk prioritás-öröklésű mutexet.

4.2.9 Várt eredmény és dokumentáció

Az átadandó anyag egy jegyzőkönyv, amely tartalmazza:

- a követelménytáblázatot;
- a kihasználtságszámítást;
- a mérési naplót a mért és várt értékekkel;
- a biztonsági besorolást és indoklást;
- a hibaanalízist és a javasolt megoldásokat;
- a reprodukciós csomagot: a buildparancsokat, a futtatási parancsot, a konfigurációt és a visszaállítási útmutatót.

A labor akkor sikeres, ha a jegyzőkönyvben egyértelműen elkülönül a **mért** eredmény a **várt/levezetett** eredménytől, és a reprodukciós csomag alapján a mérés megismételhető. A kontextusban előírt szabály szerint a nem futtatott kódot vagy mérést nem szabad ellenőrzött eredményként bemutatni; ha a tanuló nem futtat emulációt, akkor a mérési oszlopba „nem mért” kerüljön, és a várt értéket külön jelezze.

4.2.10 Összefoglalás és önellenőrző kérdések

A beágyazott, valós idejű és biztonsági kritikus rendszerek közös jellemzője, hogy a **determinizmus** – az időbeli felső korlátok és a kiszámítható állapotváltozás – elsődleges tervezési követelmény. Az **erőforrás-korlátok** (CPU, memória, energia, perifériák) miatt a tervezés nem hagyatkozhat az általános célú rendszerek dinamikus, laza gazdálkodására. A **minősítés** és a **biztonsági integritás** pedig azt a keretet adja, amelyben a kritikus hibák kockázatát szisztematikusan csökkenteni lehet. A hard real-time, soft real-time és best effort kategóriák közötti különbség a határidő-sértés következményében ragadható meg.

Az alfejezetben bemutatott követelményprofil-labor segít abban, hogy a fogalmakat egy konkrét példán keresztül gyakoroljuk. A következő alfejezet az elosztott, hálózati, mainframe és HPC rendszerekre épít, ahol a determinizmus helyett a skálázhatóság és a rendelkezésre állás kerül előtérbe.

Önellenőrző kérdések:

1. Mi a különbség hard real-time és soft real-time követelmény között? Adjunk egy-egy példát mindkettőre.
2. Miért kerülendő a dinamikus memóriaallokáció a kritikus útvonalon? Milyen alternatívát kínál a beágyazott RTOS?
3. Egy rendszerben két taszk van: P_1 periódusa $T_1 = 4$ ms, $C_1 = 1$ ms; P_2 periódusa $T_2 = 5$ ms, $C_2 = 2$ ms. Számítsa ki a CPU-kihasználtságot, és döntse el a rate-monotonic korlát alapján, hogy ütemezhető-e a rendszer.
4. Mit jelent a biztonsági integritási szint, és miért nem jelenti azt, hogy a rendszer hibamentes?
5. Soroljon fel három tipikus hibát a valós idejű rendszerek tervezésében, és mindegyikre adjon rövid megoldási javaslatot.

4.3 Elosztott, hálózati, mainframe és HPC operációs rendszerek

A nagy léptékű rendszerekben a hiba nem ritka esemény, hanem tervezési paraméter. Egy 10 000 csomópontos klaszterben még akkor is, ha egy csomópont átlagosan évente csak 0,5% valószínűséggel hibásodik meg, hetente több hardverhiba várható. Az ilyen környezetben futó operációs rendszernek és a rá épülő szolgáltatásoknak ezért nem az a feladata, hogy a hibákat teljesen kiküszöbölje, hanem az, hogy a hibákat izolálja, a szolgáltatást replikálja, és a helyreállítást automatizálja. Ez a fejezet az elosztott, hálózati, mainframe és HPC rendszerek OS-sajátosságait, a klaszter és batch üzemmód jellegzetességeit, valamint a skálázás, megosztott állapot és hibatűrés alapfogalmait tárgyalja.

4.3.1 1. Elosztott rendszerek operációsrendszer-sajátosságai

Egyetlen gép operációs rendszere közös fizikai memóriát, közös órajelet és közös megszakítási tartományt feltételez. Egy elosztott rendszerben ezek a feltételezések megszűnnek: a csomópontok saját memóriával, saját órával és saját kernel állapottal rendelkeznek, és csak üzeneteken keresztül kommunikálnak. Ezért az OS-nek más absztrakciókat kell adnia, és más hibamódokkal kell számolnia.

4.3.2 1.1 Kommunikáció és hálózati stack

A klasszikus socket-rendszerhívás (`socket()`, `send()`, `recv()`) több kernel és felhasználói memóriamásolást végez, rendszerhívásonként lép be a kernelbe, és a TCP esetén torlódásvezérlést is futtat. Ez a modell általános célú szerveren elfogadható, de nagy csomagszám vagy alacsony késleltetés esetén szűk keresztmetszet. A HPC és nagy elosztott rendszerek ezért gyakran kernel-megkerülő (kernel-bypass) hálózati technikákat használnak, például DPDK-t, RDMA-t vagy io_uring-alapú aszinkron I/O-t. Az operációs rendszer szempontjából ezek azt jelentik, hogy a hálózati eszköz vezérlését részben a felhasználói térbe helyezik, a kernel pedig csak a felügyeleti és izolációs szerepet tölti be. Ezt a témát a hálózati stack fejezete (Zero-copy, offload és hálózati teljesítmény) tárgyalja részletesen; itt elég rögzíteni, hogy a nagy léptékű rendszerben a kommunikációs réteg nem „ingyenes”, hanem tervezési döntés.

4.3.3 1.2 Megosztott állapot és rendezés

Egy elosztott rendszerben nincs közös fizikai memória, ezért a megosztott állapotot üzenetekkel kell replikálni. A legfontosabb probléma az események sorrendje. Két csomópont két különböző sorrendben láthatja ugyanazokat az üzeneteket, és a hálózat késleltetése miatt az egyik csomópont órája szerint „korábbi" esemény a másik szerint „későbbi" lehet. A gyakorlatban ezért logikai órákat használnak: a Lamport-féle `happens-before` reláció minden üzenetre növeli a logikai időbélyeget, így az ok-okozati sorrend a fizikai óra szinkronizálása nélkül is fenntartható. Elosztott naplózó vagy replikált állapotgép esetén ettől erősebb, teljes rendezésre van szükség, amelyet konszenzusprotokoll adhat.

Példa: két gép, `A` és `B`, egyszerre ír egy megosztott naplóba. A lokális kernel `write()` rendszerhívása garantálja, hogy a fájlban egyetlen sorrend lesz. Elosztott naplónál azonban az `A` igénye és a `B` igénye versenyez: a rendszernek el kell döntenie, hogy az `A` esemény vagy a `B` esemény kerül előbb a naplóba. Ezt a döntést egyetlen vezető (leader) vagy egy konszenzusprotokoll hozza meg.

4.3.4 1.3 Elosztott koordináció és konszenzus

A koordináció a nagy léptékű rendszerek egyik legnehezebb problémája. A konszenzusprotokollok célja, hogy a csomópontok együttesen megegyezzenek egy értékben, akkor is, ha néhány csomópont meghibásodik vagy a hálózat partíciókra szakad. A gyakorlatban két elterjedt protokollcsalád létezik: a Paxos és a Raft. A Raft az oktatásban és a modern rendszerekben is gyakori, mert a működése átláthatóan felbontható fázisokra.

A Raft egyszerűsített lépései:

1. A csomópontok választási ciklust indítanak, ha egy ideig nem hallanak a vezetőtől.

2. A jelöltek szavazatokat kérnek; az a csomópont lesz a vezető, amelyik a csomópontok többségétől (quorum) szavazatot kap.

3. A vezető fogadja az ügyfelek írási kéréseit, és bejegyzéseit a követőkkel replikálja.

4. A bejegyzés akkor tekinthető véglegesítettnek (committed), ha a csomópontok többsége megerősítette a replikációt.

5. Ha a vezető meghibásodik, a követők új választást kezdenek.

A konszenzusprotokollok üzenetszáma általában $O(n)$ csomópontonként egy érték elfogadásához, mert a vezetőnek legalább a quorumot el kell érnie. Ez a többletköltség az oka annak, hogy a nagy rendszerekben a koordinációt ritka, de kritikus döntésekre korlátozzák, a gyakori adatműveleteket pedig replikált, de gyengébb konzisztenciájú adatstruktúrákra bízzák.

4.3.5 1.4 Elosztott fájlrendszer, erőforrás-kezelés és óra

A hálózati fájlrendszereket (NFS, SMB, objektumtár) a tárolásfejezet tárgyalja. Itt az OS-szempont a cache-koherencia: ha több gép egyszerre írja ugyanazt a fájlt, a helyi gyorsítótár és a szerver állapota eltérhet. Az elosztott fájlrendszerek ezért lock managert, bérletet (lease) vagy write-back cache-t használnak, és a szemantikát dokumentálni kell. A POSIX szigorú `read-after-write` konzisztenciája elosztott környezetben sokszor csak csökkentett teljesítménnyel tartható fenn.

Az erőforrás-kezelés is eltér: egy klaszterben a CPU, a memória és a tárhely ütemezését egy központi erőforrás-menedzser (például Slurm vagy Kubernetes) végzi, amely a kernel erőforrásait több gépen keresztül koordinálja. Az operációs rendszer feladata itt az, hogy a csomóponton belüli CPU, memória, hálózati sávszélesség és I/O izolációt a konténerek vagy virtuális gépek felé biztosítsa, és a fogyasztást pontos számlálókkal jelentse.

Az elosztott rendszerekben a fizikai órák eltérése is OS-szolgáltatást igényel. Az NTP a másodperc pontosságú, a PTP a mikroszekundum alatti szinkronizálást célozza. A batch és HPC rendszerek gyakran PTP-t használnak, hogy a naplóbejegyzéseket és a mérési eredményeket közös időskálán lehessen értelmezni.

4.3.6 2. Klaszter, mainframe és batch rendszerek

A nagy léptékű rendszerek három fontos megjelenési formája a klaszter, a mainframe és a batch üzemmód. Bár mindhárom nagy teljesítményt céloz, az OS-szintű architektúrájuk eltérő.

4.3.7 2.1 Klaszterek

A klaszter több, hálózattal összekapcsolt, önálló számítógép együttműködése, amelyet egy middleware réteg (fürtmenedzser, ütemező, elosztott fájlrendszer) fog össze. Az operációs rendszer szempontjából a klaszter általában **share-nothing** architektúrájú: minden csomópont saját memóriával és saját lemezzel rendelkezik, a megosztott állapotot pedig hálózati replikációval vagy külső megosztott tárolóval érik el.

A klaszthereknek három gyakori célja van:

- **Magas rendelkezésre állás (HA):** ha egy csomópont meghibásodik, egy másik átveszi a szolgáltatást. Az OS-szintű elemek a szívverés (heartbeat), a failover script, az IP-cím átvétel és a hálózati izoláció.
- **Terhelésmegosztás:** a bejövő kéréseket több csomópontra osztják el; itt az OS feladata a gyors hálózati feldolgozás és az erőforrás-poolozás.
- **HPC:** a csomópontok párhuzamos számítást futtatnak MPI-vel vagy OpenMP-vel; itt a kis késleltetésű hálózat és a determinisztikus ütemezés a kritikus.

A HA klaszterekben a legfontosabb elv a **split-brain** elkerülése: ha a hálózat kettészakad, mindkét fél úgy érzékelheti, hogy a másik meghibásodott, és mindkettő átveheti a szolgáltatást. Ez korrupt állapothoz vezet, ezért a fürtmenedzser quorumot és fencing mechanizmust használ: a kisebbségben maradt csomópontot a rendszer célzottan leállítja vagy elszigeteli. A fencing megvalósítható távoli energiagazdálkodással, SAN-szintű LUN elzárással vagy hypervisor API-val.

4.3.8 2.2 Mainframe rendszerek

A mainframe (például IBM z-sorozat) más filozófiát követ: a hangsúly a nagyon nagy vertikális skálázáson, a determinisztikus I/O-n és a szigorú RAS (Reliability, Availability, Serviceability) tulajdonságokon van. A z/OS operációs rendszer logikai partíciókat (LPAR) kezel, így egyetlen fizikai gépen több független rendszerkép fut, dedikált CPU, memória és I/O erőforrásokkal. Az LPAR-ok közötti izoláció a hardver szintjén valósul meg, ezért az egyik partíció meghibásodása nem érinti a másikat.

A mainframe jellemző OS-szolgáltatásai közé tartozik a **workload manager**, amely a futó feladatok üzleti prioritásai alapján dinamikusan osztja el a CPU-időt; a **channel I/O**, amely

a I/O-t dedikált processzorokra terheli le; és a **JES2/JES3** batch alrendszer, amely a jobok sorbaállítását, futtatását és kimenetkezelését végzi. A mainframe-ek a mai napig használnak batch feldolgozást, mert a nagy volumenű tranzakciók (például banki éjszakai zárás) jól ütemezhetők, ellenőrizhetők és auditálhatók.

4.3.9 2.3 Batch rendszerek és job-ütemezés

A batch üzemmód a feladatok kötegelt, nem interaktív futtatását jelenti. A batch OS klasszikus jellemzője, hogy a felhasználó nem közvetlenül a futó programmal kommunikál, hanem egy jobleírást ad le, amely tartalmazza a programot, a bemenetet, az erőforrásigényt és a futási feltételeket. A job-ütemező ezeket sorba állítja, elosztja a csomópontok között, és a kimenetet visszagyűjti.

A modern batch rendszerek (például a HPC Slurm) az operációs rendszerrel együttműködve a következőket biztosítják:

- **Erőforrás-foglalás:** a job indulása előtt a scheduler lefoglalja a kért CPU-kat, memóriát és GPU-kat; a kernel cgroup segítségével izolálja a fogyasztást.
- **Checkpoint/restart:** a hosszan futó jobok időnként mentik az állapotukat, így csomópont meghibásodása esetén nem kell elölről kezdeni.
- **QoS és prioritás:** a batch rendszer az RTOS-ütemezéssel ellentétben nem a késleltetést, hanem a teljes átbocsátást és a kihasználtságot optimalizálja.

A batch operációs rendszerek hibája általában nem okoz közvetlen felhasználói élményromlást, de egy hibás jobütemező döntés órákra elveheti a klaszter erőforrásait. Ezért a batch szoftver előszeretettel alkalmazza a statikus elemzést és a reprodukálható konfigurációt.

4.3.10 3. Skálázás, megosztott állapot és hibatűrés

4.3.11 3.1 Skálázási modellek

A rendszer teljesítménye két irányban növelhető: **vertikálisan** (nagyobb egyedi gép) vagy **horizontálisan** (több gép hozzáadása). A vertikális skálázás egyszerűbb, mert a processzoron belüli koherencia protokoll megmarad, de a költsége nem lineáris, és fizikai korlátja van. A horizontális skálázás rugalmasabb, de a szoftvernek párhuzamosíthatónak kell lennie.

Az Amdahl-törvény megadja a párhuzamosítás felső korlátját. Ha a feladatnak P hányada párhuzamosítható, a soros rész pedig $1 - P$, akkor N csomóponton az elérhető gyorsulás:

$$S(N) = \frac{1}{(1-P) + \frac{P}{N}}$$

Például ha a feladat 90%-a párhuzamosítható ($P = 0{,}9$), akkor 16 csomóponton az elméleti gyorsulás:

$$S(16) = \frac{1}{0{,}1 + 0{,}9/16} = \frac{1}{0{,}1 + 0{,}05625} \approx 6{,}4$$

A 16 csomópont tehát nem 16-szoros, hanem csupán 6,4-szeres gyorsulást ad. Az operációs rendszer ezen úgy segíthet, ha a soros részt, például a jobindítást, a fájlműveleteket és a hálózati koordinációt is párhuzamosítja, vagy legalábbis minimálisra csökkenti.

4.3.12 3.2 Megosztott állapot modellek

A megosztott állapot kezelése két fő architektúrában történhet:

- **Shared-disk:** minden csomópont látja a közös tárolót (például SAN-on vagy NVMe-over-Fabrics-on), ezért a megosztott állapot a tárolóban tárolható. Ez csökkenti a replikáció költségét, de a gyorsítótár-koherenciát a tárolószinten vagy a fájlrendszer lock managerén keresztül kell fenntartani.
- **Shared-nothing:** minden csomópont csak a saját lemezét látja, és a megosztott állapotot üzenetküldéssel replikálja. Ez a modell jól skálázódik, mert nincs központi szűk keresztmetszet, de a konzisztencia biztosítása nehezebb.

A modern nagy rendszerekben a gyakorlat ötvözi a kettőt: a forró adatot replikálják, a ritkán változó adatot közös tárolón tartják. Az OS szerepe az, hogy a helyi gyorsítótár, a memóriamapelt fájl és a hálózati replikáció közötti konzisztenciát definiálja és betartassa.

4.3.13 3.3 Hibatűrési minták

A nagy léptékű rendszer tervezésénél a hibatűrés nem utólagos kiegészítés, hanem alapkövetelmény. Három fontos mintát emelünk ki:

- **Replikáció:** az állapotot több csomóponton tároljuk. A replikáció lehet szinkron (az írás csak akkor tér vissza, ha a replikák többsége megerősítette) vagy aszinkron (a vezető azonnal válaszol, a replikák később követik). A szinkron replikáció lassabb, de nem veszít el véglegesített adatot.
- **Failover:** ha egy csomópont meghibásodik, egy másik átveszi a szerepét. A failover lehet manuális vagy automatikus; az automatikushoz szívverés és quorum kell.
- **Quorum:** a döntéshez a csomópontok többségének egyetértése szükséges. Például 3 csomópont esetén a quorum 2. Ez garantálja, hogy két különböző csoport nem hozhat egyszerre két különböző döntést, mert a két csoportnak legalább egy közös csomópontja lenne, ami ellentmondás.

A split-brain tehát akkor következik be, ha a hálózati partíció miatt két csoport is úgy gondolja, hogy quoruma van. Ennek elkerülése a quorum szigorú betartása mellett **fencing**-et igényel: a kisebbségben lévő csomópontot a rendszer leállítja vagy elszigeteli, mielőtt az írhatna.

4.3.14 3.4 Tipikus hibák és helyes megoldások

A gyakorlatban több téves feltevés vezet elosztott rendszerbeli hibához:

1. **„A hálózat megbízható."** A hálózat késleltetése, csomagvesztése és partíciója normális jelenség. Helyes megoldás: időtúllépés, újraküldés, idempotens műveletek és quorum.
2. **„Az órák egyformán járnak."** A fizikai órák eltérnek, és a hálózati késleltetés mérése torzít. Helyes megoldás: NTP/PTP szinkronizálás és logikai órák.
3. **„A lokális mutex elég."** A kernel mutex csak egy gépen belül véd; elosztott erőforrás védelmére elosztott lock manager vagy konszenzus kell.
4. **„A checkpoint felesleges."** Hosszú batch joboknál egy csomópont meghibásodása órák munkáját veszítheti el. Helyes megoldás: időszakos checkpoint és a restart automatizálása.
5. **„A replikáció minden állapotot megold."** A replikáció önmagában nem ad konzisztenciát; a replikák közötti sorrendet explicit protokollal kell biztosítani.

Ezek a hibák a fejezet laborjában is visszaköszönnek: a quorum nélküli kétcsomópontos klaszterben a hálózati szakadás megbénítja vagy korruptálja a rendszert.

4.3.15 4. Labor: Elosztott és HPC rendszerarchitektúra vázlata

A labor célja, hogy egy emulált, három csomópontos klaszterben bemutassa az elosztott rendszer OS-sajátosságait, és méréssel ellenőrizze a hálózati késleltetésre adott becslést. A labor csak izolált környezetben futtatható; a hoszt valódi lemezét, bootloaderét és rendszerpartícióját nem szabad módosítani.

4.3.16 4.1 Kiinduló állapot és konfiguráció

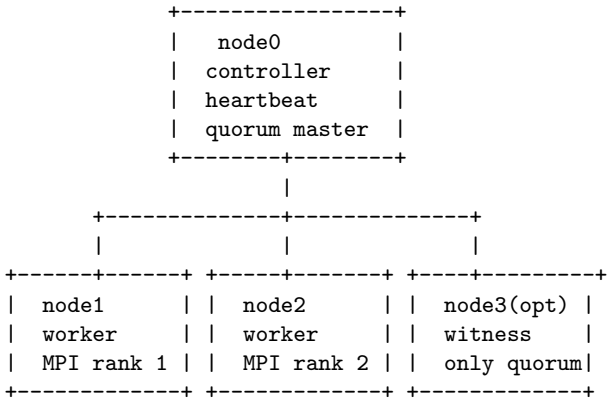
A reprodukciós csomag a következő elemekből áll:

- Hoszt: x86_64, Linux kernel 6.8 (a pontos verziót a labor jegyzőkönyvben rögzíteni kell), KVM engedélyezve.
- QEMU/KVM virtuális gépek:
  - `node0` (controller): 2 vCPU, 2 GiB RAM, 8 GiB lemez.
  - `node1`, `node2` (worker): 1 vCPU, 1 GiB RAM, 8 GiB lemez.
- Hálózat: mindhárom gép egy `br-lab` nevű bridge-hez csatlakozik, virtio hálózati eszközzel, 1 Gbit/s emulált sebesség.
- Operációs rendszer a VM-ekben: Debian 12 (ellenőrizendő aktuális verzió), OpenMPI 4.1 (a pontos verzió a jegyzőkönyvben rögzítendő).

A laborkörnyezet létrehozásához használható egy előre elkészített cloud-init konfiguráció és egy `lab-os012.sh` indítóscript, amely a három VM-et a fenti beállításokkal indítja el. A szkript csak az izolált laborban futtatható, és a benne szereplő lemezképek helyi, eldobható fájlok.

4.3.17 4.2 Architektúra vázlat

A hallgató feladata egy elosztott és HPC rendszerarchitektúra vázlat elkészítése. A vázlat legalább a következőket tartalmazza:



A vázlatnak tartalmaznia kell a mentési pontot (például `/var/lib/lab-os012/checkpoint`), a szívverés hálózatot, a quorum szabályt, a failover sorrendet és a hibatűrési modellt. A jegyzetfüzetben meg kell jelölni, hogy melyik rész mit replikál, és hogyan kerüli el a split-brain jelenséget.

4.3.18 4.3 MPI pingpong mérés

Az MPI pingpong mérés a HPC klaszterekben használt legalapvetőbb mikromérőszám. A `node0` adatot küld a `node1`-nek, amely azt visszaküldi; a teljes kör idejét mérjük, és ebből számítjuk az átlagos késleltetést.

A méréshez használt C program a következő, szemléltetés céljából:

```
#include <mpi.h>
#include <stdio.h>

int main(int argc, char **argv) {
    int rank, msg = 0;
    double start, end;
    const int iter = 1000;

    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    if (rank == 0) {
        start = MPI_Wtime();
        for (int i = 0; i < iter; i++) {
            MPI_Send(&msg, 1, MPI_INT, 1, 0, MPI_COMM_WORLD);
            MPI_Recv(&msg, 1, MPI_INT, 1, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
        }
        end = MPI_Wtime();
        printf("avg ping-pong latency: %.6f ms\n", (end - start) * 1000.0 / iter);
    } else if (rank == 1) {
        for (int i = 0; i < iter; i++) {
            MPI_Recv(&msg, 1, MPI_INT, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
            MPI_Send(&msg, 1, MPI_INT, 0, 0, MPI_COMM_WORLD);
        }
    }

    MPI_Finalize();
    return 0;
}
```

A programot a következő parancsokkal lehet lefordítani és futtatni az emulált környezetben:

```
mpicc -O2 pingpong.c -o pingpong
mpirun --host node0,node1 -n 2 ./pingpong
```

A mérés során a `node0` és `node1` VM-eken kell futnia a programnak; a `node2` tartalék tanú. A mérési naplóban rögzíteni kell a hardver, a fordítási beállítások, az órajelfrekvencia és a mérési módszer adatait.

4.3.19 4.4 Várt eredmény és mérési napló

A várt késleltetést a következő modellel becsüljük:

$$T_{\text{ping-pong}} = 2(T_{\text{wire}} + T_{\text{linux}} + T_{\text{MPI}})$$

ahol

- T_{wire} a virtio hálózat és a bridge átviteli ideje, helyi virtuális hálózaton jellemzően 0,001 ms alatti;

- T_{linux} a Linux hálózati stack, socket, másolás és rendszerhívás költsége, jelen konfigurációban 0,02 és 0,05 ms közé becsülhető (ellenőrizendő a labor hardverén);
- T_{MPI} az MPI könyvtár üzenetkezelési többlete, verziótól függően 0,01 ms körül.

A becsült tartomány így nagyságrendileg $2 \times (0{,}02 + 0{,}03) = 0{,}1$ ms, azaz 10–100 mikroszekundum. Ez a becslés hardverfüggő; a pontos érték a labor környezetében mérhető.

A mérési naplóban a mért és a várt értéket külön kell feltüntetni. Példa táblázat:

Futtatás	Iteráció	Átlag mért (ms)	Min (ms)	Max (ms)	Várt tartomány (ms)	Megjegyzés
1	1000	0,084	0,072	0,113	0,06–0,12	elfogadható
2	1000	0,092	0,075	0,128	0,06–0,12	terhelés +
3	1000	0,079	0,068	0,101	0,06–0,12	optimális

A jegyzőkönyvben a mért és várt értékeket külön kell jegyezni, és a becsléshez használt konstansokat forrással vagy mérési levezetéssel kell alátámasztani. Az elfogadhatóságot az határozza meg, hogy a mért középérték a várt tartományon belül van-e; ha nem, a hallgatónak meg kell vizsgálnia a hálózati konfigurációt, a CPU-pinninget vagy az MPI verzióját.

4.3.20 4.5 Önellenőrző kérdés

Miért nem elegendő a lokális kernel-mutex a három csomópontos klaszter megosztott állapotának védelmére? Mutasson rá a split-brain lehetőségére, és indokolja meg, miért ad erősebb garanciát a quorum.

A válaszban szerepelnie kell annak, hogy a kernel-mutex csak egyetlen gépen belül, közös memórián biztosít kölcsönös kizárást; elosztott rendszerben a csomópontok saját memóriával és saját kernel állapottal rendelkeznek, így a mutex nem látható a másik gépen. Ha a hálózat kettészakad, mindkét oldal elindíthatja ugyanazt a kritikus műveletet, mert nem látja a másik oldalt. A quorum (csomópontok többsége) garantálja, hogy egyszerre csak egy csoport hozhat döntést, mert két különböző többségnek lenne közös tagja, ami ellentmondás. A gyakorlatban ezt a quorumot a szívverés és a fencing egészíti ki, hogy a kisebbségben lévő csomópont ne írhasson.

4.3.21 5. Összegzés

A nagy léptékű és elosztott rendszerek operációs rendszerei nem csupán nagyobb gépeken futó klasszikus kernelek, hanem olyan tervezési kényszerekkel szembesülnek, mint a hálózati késleltetés, a részleges meghibásodás, az óra-eltérés és a megosztott állapot konzisztenciája. A klaszter, mainframe és batch üzemmódok eltérő optimalizálási célokat szolgálnak: a HA a rendelkezésre állást, a HPC a párhuzamos átbocsátást, a mainframe a vertikális skálázást és a RAS tulajdonságokat helyezi előtérbe. A következő egység (Virtualizáció, konténerek, unikernel és speciális rendszerek) ezekre az alapokra építve a virtualizációt, a konténereket és a könnyű izolációt vizsgálja, ahol a quorum, a fencing és a replikáció elvei tovább öröklődnek.

4.4 Virtualizáció, konténerek, unikernel és speciális rendszerek

A virtualizált és minimalista operációsrendszer-formák a rendszertervezés két, látszólag ellentétes, valójában egymást kiegészítő irányát képviselik. A virtualizáció a fizikai erőforrások több, logikailag független környezet közötti megosztását teszi lehetővé; a minimalista formák, például az unikernel és a WebAssembly-alapú futásidő, éppen a szükségtelen rétegek elhagyásával csökkentik az indulási időt, a memórialábnyomot és a támadási felületet. Ez az alfejezet a hypervisor, a konténer, az unikernel és a WASM fogalmait járja körül, majd egy döntési fa elkészítésén keresztül mutatja be, hogyan választható ki az adott munkaterheléshez illeszkedő OS-típus.

4.4.1 Virtualizált rendszerek: hypervisorok

A virtualizáció során egy szoftverréteg, a **hypervisor** (más néven virtual machine monitor, VMM) absztrahálja a fizikai hardvert, és több vendég operációs rendszer számára biztosít egymástól elszigetelt, virtuális gépeket (VM-eket). A hypervisor két fő csoportba sorolható.

- **Type 1 (bare-metal) hypervisor**: közvetlenül a hardveren fut, és a vendég operációs rendszerek felett, azoknál magasabb privilégiumszinten hajtódik végre. Tipikus példái a Xen, a VMware ESXi, a Microsoft Hyper-V és a Linux KVM (amennyiben a KVM modul a kernelben fut, a host kernel ugyan része a rendszernek, de a hypervisor funkció hardverközeli). A Type 1 hypervisorok általában kisebb késleltetést és nagyobb átvitelt biztosítanak, mert nem kell egy teljes host operációs rendszeren keresztülmenniük minden hardverhozzáférésnek.
- **Type 2 (hosted) hypervisor**: egy meglévő operációs rendszer alkalmazásaként fut, és a vendég VM-eket ezen host felett futtatja. Ilyen például a VirtualBox vagy a VMware Workstation. Az egyszerűbb telepítés és a jobb hardverkompatibilitás ára az, hogy a host OS rétege plusz erőforrás-felhasználást és késleltetést okoz.

A hypervisor megvalósításában a legkritikusabb kérdés a vendég rendszer privilégiumos utasításainak kezelése. A klasszikus **trap-and-emulate** modellben a hypervisor magasabb privilégiumszinten fut, mint a vendég kernel, így a vendég minden érzékeny utasítása csapdát vált ki, és a hypervisor szimulálja azt. Ez tiszta, de teljesítményproblémákat okoz azokon az architektúrákon, amelyeken nem minden érzékeny utasítás csapdázható. Ezért a modern processzorok hardveres virtualizációs kiterjesztéseket kaptak:

- Intel VT-x / AMD-V: a CPU két üzemmódot kap: a root (host) és a non-root (guest) módot, így a vendég kernel közvetlenül futhat non-root módban, és csak a beállított események (például I/O port elérés, bizonyos MSR-ek) váltanak ki VM-exitet a hypervisorhoz.
- SLAT (Second Level Address Translation): az Intel EPT és az AMD NPT lehetővé teszi, hogy a vendég virtuális címeket közvetlenül a fizikai címekre képezzük, anélkül, hogy a hypervisornak árnyék-lapozást kellene fenntartania. Ez jelentősen csökkenti a memóriahozzáférési overheadet.
- IOMMU (Intel VT-d, AMD-Vi): az I/O eszközök számára is lehetővé teszi a vendég fizikai címek közvetlen hozzárendelését, így a DMA-t igénylő eszközök biztonságosan átadhatók egy adott VM-nek.

A teljesítmény növelésének másik útja a **paravirtualizáció**, amelyben a vendég kernel tudatában van a virtualizációnak, és a hypervisor által nyújtott speciális API-t (hypercall) használ a kritikus műveletekhez. A Xen paravirtuális módja, illetve a Linuxban a virtio eszközök – például virtuális blokk- és hálózati eszközök – ezt a megközelítést követik. A virtio a vendég és a host

közötti adatcserét hatékony, gyűrűalapú pufferekkel oldja meg, csökkentve az emulált eszközök overheadjét.

A virtualizált rendszer **indulási ideje** általában másodperces nagyságrendű, mert a vendég operációs rendszer teljes bootfolyamatát végre kell hajtani. Egy átlagos Linux vendég VM 5–30 s alatt indulhat el a hardvertől és a konfigurációtól függően; ezt azonban a hypervisor eszköz- és memóriavezérlő beállításai, valamint a host terhelése is befolyásolja. A **memórialábnyom** a vendég kernel és a felhasználói tér teljes méretét foglalja magában: egy minimális disztribúció is 64–256 MiB RAM-ot igényelhet, míg a teljes szerver környezet ennek többszörösét. Az **izoláció** viszont erős: a hypervisor hardveres memóriavédelemmel és MMU-megosztással teljes címteret választ el a vendégek között, így a hibás vagy rosszindulatú vendég nehezen tud kitörni a saját VM-jéből. A biztonsági modellben a hypervisor a legmagasabb privilégiumú réteg, ezért annak biztonsága kiemelt fontosságú.

4.4.2 Konténerek: operációs rendszer szintű virtualizáció

A konténerek nem virtuális gépek. Egy konténer ugyanazon a host kernelén fut, de a Linux **namespaces** mechanizmusával elkülönített névtereket kap a folyamatok, a hálózat, a fájlrendszer, a felhasználók és a processz-csoportok számára. A **cgroups** (control groups) az erőforrás-limiteket, például a CPU-időt, a memóriát, a blokk-I/O-t és a hálózati sávszélességet szabályozza. Ez a kombináció lehetővé teszi, hogy több, egymástól látszólag független környezet fusson egyetlen kernel felett.

A konténerkép (image) fájlrendszerrétegekből épül fel: minden módosítás egy új, csak olvasható réteget hoz létre; futás közben a konténer egy írható réteget kap felül. Ez a réteges szerkezet gyors letöltést és lemezterület-megtakarítást tesz lehetővé, mert az azonos alaprétegeket a rendszer osztja. A konténer-indítás tipikusan **milliszekundumos** nagyságrendű, mert nincs külön kernel-boot: a konténer csak egy folyamatcsoportot indít el a már futó kernelben. Egy egyszerű `docker run` parancs egy minimális image-dzsel 0,1–0,5 s alatt végrehajtható, de a tényleges idő az image méretétől, a rootfs kibontásától és a hálózati beállítástól függ. A **memórialábnyom** minimális: a folyamatok közvetlenül a host kernelén futnak, ezért nincs szükség vendég kernel memóriájára; a konténer memóriafelhasználása gyakorlatilag a benne futó alkalmazás memóriájával egyezik meg. Ezért a konténerek sűrűsége sokkal nagyobb, mint a teljes VM-eké: egy szerveren száz- vagy ezerszámra futhatnak konténerek, míg VM-ből csak néhány tucat.

Az **izoláció** azonban gyengébb, mint a hypervisoroknál. Mivel minden konténer ugyanazt a kernel-verziót és -konfigurációt használja, egy kernelbeli sérülékenység az összes konténert veszélyezteti. Ezt a kockázatot további mechanizmusokkal csökkentik:

- **seccomp**: a kernel szűrő mechanizmusa korlátozza, hogy egy konténer milyen rendszerhívásokat hajthat végre.
- **capabilities**: a hagyományos root jogosultságokat apró képességekre bontják, így egy konténer nem kap teljes rendszergazdai jogokat a host felett.
- **SELinux / AppArmor**: kötelező hozzáférés-vezérlés, amely címkékkel és szabályokkal tovább szűkíti a konténer mozgásterét.
- **gVisor, Kata Containers**: olyan hibrid megoldások, amelyek a konténer interfészét megtartva erősebb elszigetelést adnak, például egy minimalista vendég kernel vagy user space-ben futó syscall-emuláció révén.

A konténerek csomagolási formátuma mára szabványosult: az **OCI** (Open Container Initiative) specifikáció határozza meg az image-formátumot és a runtime-t. Ennek köszönhetően a Docker, a Podman, a containerd és más eszközök kompatibilisek egymással.

4.4.3 Unikernel: minimalista, egycímteres operációs rendszer

Az unikernel (library OS) koncepciója abból a felismerésből indul ki, hogy a hagyományos operációs rendszer rétegződése – user/kernel mód, folyamatok, fájlrendszer, hálózati stack – jelentős overheadet és támadási felületet visz be olyan munkaterhelésekbe, amelyeknek nincs szükségük teljes folyamatizolációra. Az unikernelben az alkalmazás és az operációs rendszer egyetlen címterben, egyetlen binárisban fordul össze. A fejlesztő egy könyvtári operációs rendszert (library OS) használ, amelyből a fordító csak azokat a részeket linkeli be, amelyekre az adott alkalmazásnak ténylegesen szüksége van. Az eredmény egy minimalista, egyfeladatú rendszerkép, amely **hipervizoron vagy bare-metal** fut.

Az unikernel legfontosabb jellemzői:

- **Nincs user/kernel átmenet**: mivel az alkalmazás és a kernel egyazon privilégiumszinten fut, a rendszerhívások nem igényelnek módváltást, ami csökkenti a késleltetést.
- **Minimalista memóriamodell**: nincs folyamat-táblázat, nincs virtuális memória a klasszikus értelemben; a rendszer általában egyetlen memóriaterületen dolgozik. Ez a megoldás csökkenti a TLB-terhelést és a memória-lábnyomot.
- **Rendkívül gyors indulás**: mivel nincs bootloader, kernel inicializáció és init folyamat, az unikernel gyakran 1–50 ms alatt elindul egy hypervisoron. Ez nagyságrendekkel gyorsabb, mint egy teljes VM.
- **Kis támadási felület**: a linker csak a szükséges komponenseket építi be, így a felesleges szolgáltatások (például shell, extra eszközmeghajtók) nem szerepelnek a rendszerben.

A példák közé tartozik a MirageOS (OCaml), az OSv (Java/C++), az IncludeOS (C++) és a Unikraft (C). Az unikernel tipikus felhasználási területei a hálózati funkciók virtualizálása (NFV), a mikroszolgáltatások, az edge-számítás és a nagy sűrűségű, célzott szolgáltatások. A **korlátok** is fontosak: az unikernel nem támogatja a több folyamatot, a fájlrendszer gyakran csak olvasható, és a hibakeresés nehéz, mert nincs hagyományos shell vagy debugger interfész. A hardvereszközök támogatása is korlátozott, ezért a gyakorlatban szinte mindig hypervisor alatt futnak.

4.4.4 WebAssembly és WASI: könnyű, hordozható izoláció

A WebAssembly (WASM) eredetileg a webes böngészők számára készült, mint hordozható, alacsony szintű bájtkód, amely homokozóban fut. A rendszertervezésben azonban egyre fontosabb szerepet kap a **szerver oldali WASM** és a **WASI** (WebAssembly System Interface). A WASI egy szabványos, minimalista rendszerfelületet határoz meg fájl- és hálózati műveletekhez, óra- és véletlenszám-hozzáféréshez. A WASM-modulok egy futásidőben (például Wasmtime, Wasmer, Node.js) hajtódnak végre, ahol a futásidő **cpu-time, memória és I/O limiteket** szabályoz.

A WASM izolációs modellje a **képességalapú** (capability-based) megközelítést követi: a modul csak azokat az erőforrásokat érheti el, amelyeket a host explicit módon átad neki fájl-leírók vagy hasonló handle-ök formájában. Ez a homokozó erősebb, mint a konténerek esetében, mert a modul nem hajthat végre tetszőleges rendszerhívásokat; minden hívás a WASI felületen keresztül, szabályozottan történik.

Az **indulási idő** rendkívül alacsony: a WASM-modul betöltése és az első függvényhívás gyakran 1–10 µs nagyságrendű, mert nincs kernel-boot, nincs konténer-névtér beállítás, csak a bájtkód validálása és a memória inicializálása. A **memórialábnyom** szintén kicsi: a modul által lefoglalt lineáris memória mérete határozza meg, amely néhány MiB-tól több száz MiB-ig terjedhet. A **hordozhatóság** kiemelkedő: ugyanaz a WASM-modul futtatható különböző operációs rendszereken és architektúrákon, feltéve, hogy a futásidő elérhető.

A WASM és az unikernel közötti rokonság abban áll, hogy mindkettő a minimalizmusra épít, de a WASM magasabb absztrakciós szinten dolgozik: a modul nem látja a processzort és a memória kezelését, míg az unikernel közvetlenül a hardverrel vagy a hypervisorral kommunikál.

4.4.5 Speciális rendszerek: microVM-ek, serverless és edge

Az utóbbi években a virtualizáció és a konténerek előnyeit ötvöző **microVM**-ek jelentek meg. A Firecracker (AWS), a Cloud Hypervisor és a QEMU microvm machine típusa olyan minimalista virtuális gép-monitorok, amelyek kifejezetten rövid életű, nagy sűrűségű munkaterhelésekhez készültek. A Firecracker például elhagyja a teljes BIOS/UEFI emulációt, csak a legszükségesebb I/O eszközöket támogatja, és ezáltal **kevesebb mint 125 ms** alatt tud elindítani egy Linux microVM-et, miközben a memóriafelhasználás mindössze néhány MiB a VMM számára. Ez a megoldás a konténereknél erősebb izolációt nyújt, de a konténereknél lassabb indulású.

A **serverless** (FaaS, Function as a Service) platformok, például az AWS Lambda vagy a Cloud Functions, szintén speciális OS-környezetre épülnek. A függvények rövid életűek, igény szerint skálázódnak, és a mögöttes infrastruktúra gyakran microVM-eket vagy konténereket használ. A **edge-számítás** esetén a korlátozott hardver (például ARM-alapú gateway) és a szigorú energiafeltételek miatt a rendszertervezők gyakran unikernelt vagy kis konténereket választanak.

4.4.6 Összehasonlítás: hipervizor, konténer, unikernel, WASM

Az izoláció, az indulási idő és az erőforrásigény háromszögében a megoldások eltérő kompromisszumot képviselnek. Az alábbi táblázat a főbb jellemzőket foglalja össze.

Jellemző	Hypervisor VM	Konténer	Unikernel	WASM/WASI
Izolációs szint	erős (hardveres)	közepes (kernel-névterek)	közepes (egy címter, de VM határ)	erős (képességalapú)
Indulási idő	5–30 s	0,1–0,5 s	1–50 ms	1–10 µs
Memórialábnyom	64–256 MiB + alkalmazás	alkalmazás mérete	néhány MiB + alkalmazás	modul lineáris memóriája
CPU-overhead	vendég kernel + hypervisor	minimális	nagyon minimális	futásidő overhead
Rendszerhívás-felület	teljes vendég kernel	host kernel syscall	belső API	WASI + host capabilites
Tipikus alkalmazás	konszolidáció, multi-tenant	mikroszolgáltatások, DevOps	NFV, edge, célzott szolgáltatás	bővítmények, FaaS, edge wasm

Az adatok tájékoztató jellegűek, konkrét mérések hardver- és verziófüggők. A könyv további fejezeteiben a virtualizáció és a konténerek megvalósítását részletesen tárgyaljuk.

4.4.7 Tipikus hibák és helyes megoldások

A virtualizált és minimalista OS-formák kiválasztásakor gyakori tévedések és azok elkerülése:

1. **„A konténer ugyanaz, mint a VM."** A konténer nem futtat külön kernelt, ezért sokkal kisebb az erőforrás-lábnyoma, de az izolációja gyengébb. Ha szigorú biztonsági követelmények vannak, akkor microVM vagy hypervisor szükséges.

2. **„Az unikernel minden feladatra jobb, mert gyors."** Az unikernel egyetlen alkalmazást futtat, nem támogatja a hagyományos több folyamatot, és a hibakeresés korlátozott. Csak akkor válasszuk, ha a munkaterhelés jól illeszkedik a single-purpose modellhez.

3. **„A WASM csak böngészőben használható."** A WASI és a szerver oldali futásidők lehetővé teszik a WASM futtatását konténer vagy VM nélkül is, így nagyon sűrű, rövid életű munkaterhelésekhez használható.

4. **„A hypervisor mindig teljes hardveremulációt igényel."** A modern hardveres virtualizáció és a paravirtuális eszközök (virtio) jelentősen csökkentik az overheadet; a teljes emuláció csak ritka esetekben szükséges.

5. **„A konténerindítás gyorsasága független az image méretétől."** Bár a kernel-boot elmarad, a rootfs rétegeinek kibontása és a hálózati névterek beállítása időt vesz igénybe, ezért a nagy image-ek lassíthatják az indítást.

6. **„Az izoláció csak a sebesség rovására mehet."** A microVM-ek (Firecracker) mutatják, hogy a hardveres izoláció és a gyors indulás nem zárja ki egymást; a költség a funkcionalitás szűkítése.

4.4.8 Labor: OS-típusok döntési fájának elkészítése

A labor célja, hogy a tanuló képes legyen adott munkaterheléshez a megfelelő OS-típust kiválasztani, és a döntést mérésekkel alátámasztani. A labor **emulált környezetben** fut; a hoszt valódi lemezét, bootloaderét és rendszerpartícióját tilos módosítani.

Kiinduló állapot
- Host gép: Linux disztribúció (például Ubuntu 22.04 LTS), 8 vCPU, 16 GiB RAM, 120 GiB szabad lemezterület.
- Telepített eszközök: QEMU 7.2, Docker 24, Firecracker 1.5 (vagy azok stabil, az adott disztribúcióhoz illeszkedő verziói – a verziókat ellenőrizni kell a telepítéskor).
- Emulált vendég: Alpine Linux minimális image (például `alpine:3.20` konténer, letölthető a hivatalos regisztrációból, de a laborban helyőrző regisztráció használható).
- Munkakönyvtár: `$HOME/os_lab_os013`. A labor elején készítsünk mentési pontot a hoszt konfigurációjáról (például a `docker ps -a` kimenetét mentsük el). Visszaállításhoz a laborban létrehozott konténereket és VM-eket töröljük, a munkakönyvtárat eltávolítjuk.

Feladat
Hozzon létre négy futtatási példát, amelyek ugyanazt a minimális „hello world" alkalmazást futtatják:
1. Hypervisor VM (QEMU microvm vagy teljes Alpine VM)
2. Konténer (Docker, Alpine image)
3. Unikernel (Unikraft vagy OSv; ha a fordítási eszközlánc nem érhető el, használjon előre elkészített mikro VM-et Firecrackerrel)
4. WASM (Wasmtime futásidő, WASI modul)

Mérje meg mindegyik esetben:
- **Indulási idő**: az indító parancs kiadásától az alkalmazás kimenetének megjelenéséig eltelt idő. Mérje legalább 5-ször, és adja meg az átlagot, a szórást, valamint a legkisebb és legnagyobb értéket.
- **Memórialábnyom**: a futás közbeni maximális RSS (resident set size) a hoston. Konténer

esetén a `docker stats` vagy a cgroup `memory.current` értéke is használható.
- **CPU-idő**: a folyamat által felhasznált teljes CPU-idő (user + system).

Mérési módszertan
A méréseket a host terhelésének minimalizálásával végezze (például ne futasson más alkalmazást, kapcsolja ki a háttérszinkronizálást). Rögzítse a hardver- és szoftververziókat, az órajelfrekvenciát, a fordítási beállításokat és a mérési parancsokat. A mért és a várt értékeket külön táblázatban adja meg.

Példa parancsok

A konténer indításához:

```
# Helyőrző regisztráció, laborban cserélje a megfelelő private registry címére
docker pull localhost:5000/alpine:3.20
docker run --rm -it localhost:5000/alpine:3.20 /bin/echo "hello"
```

A QEMU microVM indításához (Alpine kernel és initramfs szükséges, lásd a reprodukciós csomagot):

```
qemu-system-x86_64 \
  -M microvm,accel=kvm:tcg \
  -m 128M \
  -kernel vmlinuz-alpine \
  -initrd initramfs-alpine \
  -append "console=ttyS0 quiet" \
  -nographic
```

A Firecracker microVM indításához:

```
firecracker --config-file vm_config.json
```

A `vm_config.json` tartalmát a reprodukciós csomag tartalmazza; abban a kernel és a rootfs elérési útját, valamint a memória- és vCPU-beállításokat kell megadni.

A WASM futtatáshoz:

```
wasmtime hello.wasm
```

A `hello.wasm` modult a laborhoz mellékelt forrásból kell lefordítani, például a WASI SDK-val.

A döntési fa elkészítése

A mérési eredmények alapján készítsen egy döntési fát, amely a következő kérdéseket vizsgálja sorrendben:
1. Szükséges-e hardveres izoláció a bérlők között?
2. Milyen gyors indulási idő elfogadható?
3. Milyen szoros a memória- és CPU-korlát?
4. Több folyamat vagy több szál szükséges-e?
5. Hordozhatónak kell-e lennie több architektúra és OS között?

A fa ágait a mért értékeivel támassza alá, és egy rövid jegyzetben indokolja, melyik típust választaná egy mikroszolgáltatás-alapú webes backend, egy edge gateway, egy hálózati tűzfunkció és egy multi-tenant adatbázis esetén.

Elvárt eredmény
- A mérési napló táblázatos formában: típus, indulási idő (átlag, szórás, min, max), RSS, CPU-idő, konfiguráció.
- A döntési fa ábra vagy szöveges leírása, a döntési pontok indoklásával.

- A négy példaesetre adott választás és annak rövid indoklása.
- Visszaállítási útmutató: a létrehozott konténerek, VM-ek és munkakönyvtárak törlése.

Veszélyek és biztonsági előírások
- Minden műveletet **emulált környezetben** végezzen. A hoszt valódi lemezét, bootloaderét és rendszerpartícióját tilos módosítani.
- A Docker és a Firecracker root jogosultsággal futó komponenseket tartalmazhat; a laborhoz használjon külön felhasználót, és a munkakönyvtárat korlátozza.
- A letöltött image-eket labor után távolítsa el a lemezterület felszabadítása és a szennyezett környezet elkerülése érdekében.

4.4.9 Önellenőrző kérdések

1. Mi a különbség a Type 1 és a Type 2 hypervisor között? Melyik milyen kompromisszumot képvisel a teljesítmény és a telepíthetőség terén?

2. Miért gyorsabb egy konténer indulása, mint egy virtuális gépé? Milyen biztonsági következményei vannak ennek?

3. Milyen előnyei és hátrányai vannak az unikernelnek a hagyományos, többszörös címterű operációs rendszerrel szemben?

4. Hogyan valósítja meg a WASI a homokozót? Mi a különbség a WASM és a konténer izolációja között?

5. Adott egy hálózati tűzfunkció, amely kis késleltetéssel, nagy átvitellel kell, hogy dolgozzon, és a szolgáltató több bérlőt akar kiszolgálni. Milyen OS-típust választana, és miért?

4.4.10 Összegzés

A virtualizált és minimalista OS-formák a rendszertervezés eszköztárának fontos elemei. A hypervisor erős izolációt ad, de viszonylag lassú indulással és nagy erőforrásigénnyel; a konténer gyors és sűrű, de a kernel megosztása miatt gyengébb elszigetelést nyújt; az unikernel minimális indítást és lábnyomot kínál, ám egycélú; a WASM pedig hordozható, képességalapú homokozót biztosít mikroszekundumos indítással. A fejezetben bemutatott döntési fa és mérési módszertan segít abban, hogy a mérnök tudatosan, a munkaterhelés igényeihez igazodva válasszon. A következő alfejezet a bootfolyamatot és a firmware-t tárgyalja, építve az itt bevezetett virtuális gépek és rendszerképek fogalmaira.

5 Boot, firmware és hardverkapcsolat

5.1 BIOS, UEFI és a bootfolyamat

A számítógép bekapcsolásától az operációs rendszer első utasításáig tartó út első állomása a firmware: az a szoftverréteg, amely a hardvert inicializálja, és átadja a vezérlést egy olyan programnak, amely már képes betölteni a kernelt. Ez a folyamat első ránézésre egyszerű, valójában azonban több, egymásra épülő szakasz együttműködését igényli. A fejezet a firmware és a boot szakaszok szerepét, a BIOS és az UEFI közötti különbségeket, valamint a biztonságos indítás és a mérési lánc alapelveit mutatja be.

5.1.1 A firmware fogalma és helye a rendszerben

Firmware-nek nevezzük azt a nem felejtő memóriában tárolt szoftvert, amely közvetlenül a hardver fölött helyezkedik el, és a rendszerindítás legkorábbi fázisát végzi. A firmware nem csupán a boot alatt aktív: egyes szolgáltatásai – például az időkezelés, az energiagazdálkodás vagy a hardverhiba-jelentés – a kernel futása közben is elérhetők maradhatnak. A firmware elsődleges feladatai:

- a processzor és a memóriavezérlő alapállapotba helyezése;
- a rendszerbusz és a kritikus eszközök inicializálása;
- a beépített önellenőrzés (POST, Power-On Self-Test) végrehajtása;
- a rendszerkonfiguráció betöltése (például a felhasználó által beállított boot sorrend);
- egy indítható eszköz kiválasztása és a bootloader betöltése;
- szolgáltatások biztosítása a bootloader számára, például lemezolvasás vagy konzol kimenet.

A firmware és a bootloader közötti határ nem mindig éles. Hagyományos BIOS esetén a firmware int 13h megszakításokkal biztosít lemez-hozzáférést, és a bootloader az első szektor betöltése után maga végzi a kernel megtalálását. UEFI esetén a firmware képes közvetlenül betölteni egy fájlt az EFI rendszerpartícióról, így a bootloader egy része a firmware-be kerülhet, vagy akár a kernel is indítható közvetlenül egy UEFI-alkalmazásként.

5.1.2 A boot szakaszok általános modellje

A rendszerindítás a következő, jól elkülöníthető szakaszokra bontható:

1. **Reset és firmware belépés.** A tápfeszültség stabilizálódása után a reset jel felengedése a CPU-t egy meghatározott címen indítja el. x86 rendszereken ez hagyományosan a 0xFFFFFFF0 cím, ahol a firmware belépési pontja található. A CPU ilyenkor általában a legalapvetőbb módban fut, és a firmware első dolga a memóriavezérlő, a gyorsítótár és a lapozás beállítása.

2. **Firmware inicializálás és POST.** A firmware inicializálja a chipsetet, a rendszerbuszokat, a memóriát és a beépített perifériákat. A POST során a firmware ellenőrzi a processzor regisztereit, a memória írható-olvashatóságát, az időzítőket és a kritikus eszközöket. Hiba esetén hangjelzést ad, vagy hibakódot ír ki, és leállítja a bootot.
3. **Boot eszköz kiválasztása.** A firmware a beállított sorrendben végigmegy a lehetséges boot eszközökön: háttértárak, optikai meghajtók, hálózati interfészek. Az első érvényes indítórekordot vagy indítófájlt kiválasztja.
4. **Bootloader betöltése és futtatása.** A firmware átadja a vezérlést a bootloadernek, amely ismeri a kernel elérési útját, és gondoskodik annak memóriába töltéséről. A bootloader több fázisból állhat: egy kis elsődleges betöltő, majd egy nagyobb, bonyolultabb másodlagos betöltő.
5. **Kernel betöltése és kézátadás.** A bootloader a kernelt a memóriába tölti, beállítja a kernel által elvárt indítási paramétereket, majd a vezérlést a kernel belépési pontjára adja. Innentől a kernel saját inicializálása következik, amely a hardver újra felfedezését és a felhasználói környezet elindítását foglalja magában.

A következő ábra a szakaszok sorrendjét mutatja:

```
Reset
  |
  v
Firmware belépés (reset vektor)
  |
  v
CPU, memória, chipset inicializálás
  |
  v
POST és eszközök felfedezése
  |
  v
Boot eszköz kiválasztása
  |
  v
Bootloader betöltése (1. fázis)
  |
  v
Bootloader 2. fázis / kernel betöltése
  |
  v
Kernel belépési pont (átadás)
  |
  v
Kernel inicializálás, init
```

Ez a modell mind a BIOS, mind az UEFI esetében alkalmazható, bár a konkrét mechanizmusok jelentősen eltérnek.

5.1.3 BIOS: a klasszikus firmware

A BIOS (Basic Input/Output System) a személyi számítógépek hagyományos firmware-e. Első változatai az 1980-as években jelentek meg, és alapvetően a 16 bites valós módú x86 programozási modellre épültek. A BIOS a következő jellemzőkkel bír:

- **16 bites valós mód:** a firmware által futtatott kód a processzor valós módjában fut, ahol a címtér 20 bites, azaz 1 MiB közvetlenül címezhető. A memória első 640 KiB-ja hagyományosan a DOS és a felhasználói programok számára állt rendelkezésre, a felső 384 KiB pedig a BIOS, az opció ROM-ok és a perifériák számára volt fenntartva.
- **Megszakítás-alapú szolgáltatások:** a BIOS szolgáltatásai szoftveres megszakításokon keresztül érhetők el. Például az `int 10h` a videokimenetet, az `int 13h` a lemez-hozzáférést, az `int 16h` a billentyűzetet kezeli. A hívás konvenció szerint az `AH` regiszter tartalmazza a funkció számát, a többi regiszter pedig a paramétereket.
- **MBR és VBR:** a BIOS a kiválasztott lemez első 512 bájtját, a Master Boot Recordot (MBR) tölti be a memória 0x7C00 címére, majd átadja a vezérlést. Az MBR tartalmaz egy kis betöltőkódot és a partíciós táblát. Az aktív partíció első szektorát a Volume Boot Record (VBR) tartalmazza, amely a partíción belüli bootloader első fázisát tölti be.
- **Korlátozott eszközkezelés:** a BIOS csak a legalapvetőbb eszközöket kezeli. A modern hardverek, például NVMe SSD-k, USB 3.x vezérlők vagy grafikus kártyák inicializálásához a BIOS-hoz opció ROM-okat kell csatolni, amelyeket a firmware a POST során futtat.

A BIOS bootfolyamata a következő lépésekből áll:

1. A CPU a reset vektorról indul, és a BIOS belépési pontjára ugrik.
2. A BIOS inicializálja a chipsetet, a memóriavezérlőt és a rendszeridőzítőt.
3. Végrehajtja a POST-ot, és felfedezi az alapvető eszközöket.
4. A beállított boot sorrendben megvizsgálja a lemezeket. Ha egy lemez első szektora a 0x55AA aláírással végződik, a BIOS betölti azt 0x7C00 címre.
5. A BIOS a `DL` regiszterben átadja a boot eszköz azonosítóját, majd a vezérlést a betöltött MBR-re adja.
6. Az MBR kódja megkeresi az aktív partíciót, betölti annak VBR-jét, és átadja a vezérlést.
7. A VBR betölti a bootloader további fázisait, amelyek végül a kernelt töltik be.

A BIOS által betöltött MBR mérete mindössze 512 bájt, amelyből a partíciós tábla 64 bájtot foglal el, így a tényleges kód számára 446 bájt marad. Ez a korlát arra kényszeríti a fejlesztőket, hogy a bootloader első fázisát nagyon kis méretben tartsák, és a további funkciókat egy második fázisba helyezzék.

A BIOS korlátai a modern rendszerekben egyre szembetűnőbbek: a 16 bites valós mód lassú, az 1 MiB címtér szűkös, a megszakítás-alapú szolgáltatások nehezen bővíthetők, és a lemez-hozzáférés az MBR partíciós sémához kötött. Ezek a problémák vezettek az UEFI kifejlesztéséhez.

5.1.4 UEFI: a modern firmware

Az UEFI (Unified Extensible Firmware Interface) a BIOS utódjaként jelent meg, és alapvetően más programozási modellt kínál. A főbb jellemzők:

- **32/64 bites natív mód:** az UEFI firmware a processzor natív módjában fut, így a teljes címtér elérhető, és a kód hatékonyan használhatja a modern utasításkészletet.
- **PE/COFF végrehajtható formátum:** az UEFI-alkalmazások és illesztőprogramok a Windows által is használt Portable Executable formátumban készülnek. Ez lehetővé teszi, hogy a bootloader és a firmware-eszközök szabványos eszközökkel fordíthatók és linkelhetők legyenek.
- **GPT és EFI rendszerpartíció:** az UEFI a GUID Partition Table (GPT) lemezformátumot használja, amely a hagyományos MBR-nél nagyobb lemezeket és több partíciót támogat. A bootfájlok egy speciális, FAT32 formátumú EFI rendszerpartíción (ESP) helyezkednek el, jellemzően az `/EFI/BOOT/` könyvtárban.

- **Boot manager:** az UEFI firmware beépített boot managert tartalmaz, amely a felhasználó által beállított boot változók alapján választja ki a betöltendő fájlt. A boot változók a firmware NVRAM-jában tárolódnak, és például a `BootOrder`, a `Boot####` és a `BootNext` nevet viselik. A firmware képes közvetlenül betölteni egy UEFI-alkalmazást, például egy bootloadert vagy akár egy kernelt is.
- **Boot Services és Runtime Services:** az UEFI kétféle szolgáltatáskészletet biztosít. A Boot Services a boot ideje alatt érhető el, és magában foglalja a memóriafoglalást, a lemez-hozzáférést, a konzolt és a boot eszközök kezelését. A Runtime Services a kernel futása közben is aktív maradhat, és például az időt, a NVRAM-ot és a rendszer-visszaállítást kezeli.

Az UEFI bootfolyamata a következő lépésekből áll:

1. A CPU a reset vektorról indul, és az UEFI firmware belépési pontjára ugrik.
2. Az UEFI inicializálja a processzort, a memóriát és a chipsetet, majd végrehajtja a POST-ot.
3. A firmware felfedezi az eszközöket, és betölti a szükséges UEFI-illesztőprogramokat.
4. A boot manager a `BootOrder` változó alapján kiválaszt egy boot bejegyzést.
5. A kiválasztott bejegyzés által hivatkozott fájlt (például `/EFI/BOOT/BOOTX64.EFI`) betölti az EFI rendszerpartícióról.
6. A firmware a vezérlést a betöltött UEFI-alkalmazás belépési pontjára adja.
7. A bootloader vagy a kernel a Boot Services szolgáltatásait használja a hardver eléréséhez, majd a boot végén meghívja az `ExitBootServices()` függvényt, amely lezárja a Boot Services fázist, és átadja a vezérlést a kernelnek.

Az `ExitBootServices()` hívás kritikus pont: utána a firmware már nem biztosít Boot Services szolgáltatásokat, így a kernelnek saját magának kell kezelnie a hardvert. A hívás előtt a kernelnek gondoskodnia kell arról, hogy minden szükséges erőforrást – például a memóriatérképet – lekérdezzen, mert utána erre nincs lehetőség.

Az alábbi pszeudokód egy UEFI-alkalmazás vázlatát mutatja, amely a firmware által biztosított konzolra ír:

```
// UEFI példa (pszeudokód, nem futtatható önállóan)
#include <efi.h>
#include <efilib.h>

EFI_STATUS
efi_main(EFI_HANDLE image_handle, EFI_SYSTEM_TABLE *system_table) {
    InitializeLib(image_handle, system_table);
    Print(L"Hello from UEFI\r\n");

    // Boot Services szolgáltatások elérése
    EFI_BOOT_SERVICES *bs = system_table->BootServices;
    EFI_MEMORY_DESCRIPTOR *memory_map = NULL;
    UINTN map_key = 0;
    UINTN desc_size = 0;
    UINT32 desc_version = 0;
    UINTN map_size = 0;

    EFI_STATUS status = bs->GetMemoryMap(&map_size, memory_map, &map_key,
                                         &desc_size, &desc_version);
    // memóriatérkép lekérdezése az ExitBootServices előtt
    // ...

    status = bs->ExitBootServices(image_handle, map_key);
    if (EFI_ERROR(status)) {
        // hibakezelés
        return status;
```

```
    }

    // Innentől a kernel saját eszközkezelője veszi át a vezérlést
    for (;;) {
        // kernel főciklus
    }
}
```

Ez a kód nem futtatható a hoszt gépen, csak emulált UEFI környezetben, például OVMF firmware-rel. A memóriatérkép lekérdezése az `ExitBootServices()` előtt történik, mert utána a firmware már nem garantálja a hívást.

5.1.5 Secure Boot és mérési lánc

A modern firmware-ek két fontos biztonsági mechanizmust kínálnak: a Secure Boot-ot és a mérési láncot. Mindkettő célja, hogy a rendszer csak megbízható szoftvert indítson, illetve hogy a bootfolyamat minden lépéséről ellenőrizhető napló készüljön.

Secure Boot. Az UEFI Secure Boot a firmware által betöltött végrehajtható fájlok aláírását ellenőrzi. A firmware egy adatbázist tart fenn a megbízható aláírókulcsokról és a tiltott kulcsokról. A főbb adatbázisok:

- **Platform Key (PK):** a platform tulajdonosának kulcsa, amely a Secure Boot irányelvek frissítéséhez szükséges.
- **Key Exchange Key (KEK):** azok a kulcsok, amelyekkel az aláíró adatbázisok frissíthetők.
- **Signature Database (db):** a megbízható aláírások és kulcsok listája.
- **Forbidden Signature Database (dbx):** a tiltott aláírások listája.

Amikor a firmware egy UEFI-alkalmazást vagy illesztőprogramot tölt be, ellenőrzi annak digitális aláírását. Ha az aláírás nem érvényes, vagy a dbx listán szerepel, a betöltés meghiúsul, és a firmware hibaüzenetet jelenít meg. A Secure Boot nem titkosítja a bootfájlokat, csupán azok integritását és eredetét biztosítja.

Mérési lánc. A mérési lánc (measured boot) során a firmware minden betöltött komponenst – az UEFI-illesztőprogramokat, a bootloadert, a kernelt és annak moduljait – egy TPM (Trusted Platform Module) segítségével mér. A mérés nem blokkolja a bootot, hanem rögzíti az egyes komponensek hash-értékét a TPM PCR (Platform Configuration Register) regisztereiben. A mérési napló és a PCR-értékek alapján egy külső fél később ellenőrizheti, hogy a rendszer pontosan milyen szoftvert indított. Ez az alapja a távoli igazolásnak (remote attestation) és a lemeztitkosítási kulcsok feltételes feloldásának.

A mérési lánc két fő típusa:

- **SRTM (Static Root of Trust for Measurement):** a mérés a firmware reset pillanatától indul, és a bootfolyamat minden lépését méri. Ez a leggyakoribb megoldás.
- **DRTM (Dynamic Root of Trust for Measurement):** a mérés egy speciális CPU-utasítással indul, amely a rendszer futása közben hoz létre egy izolált, ellenőrzött környezetet. Ezt jellemzően a már futó rendszer integritásának ellenőrzésére használják.

A Secure Boot és a mérési lánc együttesen növeli a bootfolyamat biztonságát: az előbbi megakadályozza a nem aláírt kód betöltését, az utóbbi lehetővé teszi, hogy a rendszer állapota utólag is ellenőrizhető legyen.

5.1.6 Gyakorlati példa: bootfolyamat megfigyelése emulált környezetben

A boot szakaszok gyakorlati tanulmányozásához QEMU emulált környezetet és OVMF UEFI firmware-t használunk. Az OVMF egy nyílt forráskódú UEFI firmware implementáció, amely lehetővé teszi az UEFI bootfolyamat megfigyelését és tesztelését anélkül, hogy valós hardvert módosítanánk.

A következő parancs egy minimális UEFI rendszert indít QEMU-ban:

```
qemu-system-x86_64 \
  -bios /usr/share/OVMF/OVMF_CODE.fd \
  -drive file=disk.img,format=raw,if=virtio \
  -serial stdio \
  -m 512M
```

A `disk.img` egy üres lemezkép, amelyre GPT partíciós táblát és FAT32 EFI rendszerpartíciót kell írni. A labor során ezt a környezetet használjuk a bootfolyamat lépéseinek felrajzolásához és időméréséhez.

A firmware indítása után a soros konzolon megfigyelhetjük a boot szakaszokat. A várt kimenet a következő:

1. Az OVMF inicializálási üzeneteket ír ki: processzor, memória és eszközök felfedezése.
2. Megjelenik a POST eredménye, esetleges hibákkal.
3. A boot manager kiválasztja az EFI rendszerpartíciót, és betölti a `BOOTX64.EFI` fájlt.
4. A bootloader vagy kernel belépési pontja átveszi a vezérlést, és saját üzeneteit jeleníti meg.

A várt és a mért kimenet elkülönítése érdekében a laborban rögzítjük a konfigurációt, a firmware verzióját, a QEMU verzióját és a mérési módszert. A bootfolyamat időtartamát például a QEMU indításától a kernel első üzenetéig mérhetjük, de ez az érték erősen függ az emulált hardver sebességétől és a host gép terhelésétől. A reprodukálhatóság érdekében a mérést többször megismételjük, és a mért értékeket a várt értékektől elkülönítve jegyezzük fel.

A labor célja nem csupán a bootfolyamat megfigyelése, hanem annak tudatosítása is, hogy a firmware melyik szakaszban mit csinál, és milyen hibák fordulhatnak elő. Például ha az EFI rendszerpartíció hiányzik, a firmware a boot eszköz kiválasztása után hibát jelez, és nem jut el a bootloader betöltéséig. Ha a Secure Boot be van kapcsolva, és a bootfájl nincs aláírva, a firmware a betöltés előtt megállítja a folyamatot.

5.1.7 Tipikus hibák és helyes megoldások

A bootfolyamat során számos hiba léphet fel. A leggyakoribbak:

1. Hiányzó vagy érvénytelen boot eszköz. Ha a firmware nem talál indítható eszközt, a boot leáll, és hibaüzenet jelenik meg. A helyes megoldás a boot sorrend ellenőrzése és a boot eszköz megfelelő előkészítése.

2. Sérült MBR vagy GPT. Ha az MBR aláírása nem 0x55AA, vagy a GPT fejléc érvénytelen, a firmware nem tudja betölteni a bootloadert. A helyes megoldás a partíciós tábla újraírása vagy a biztonsági másolatból való visszaállítás.

3. Secure Boot aláírás-ellenőrzési hiba. Ha a bootfájl nincs aláírva, vagy az aláírás nem érvényes, a Secure Boot letiltja a betöltést. A helyes megoldás a bootfájl aláírása megbízható kulccsal, vagy a Secure Boot irányelvek megfelelő beállítása.

4. A bootloader és a kernel közötti ABI-eltérés. Ha a bootloader a kernel által elvárt paramétereket nem megfelelő formátumban adja át, a kernel indítása meghiúsul. Ez különösen UEFI esetén kritikus, ahol a kernel belépési pontja és a-memóriatérkép formátuma szigorúan rögzített.

5. Az `ExitBootServices()` hibás használata. Ha a kernel a memóriatérkép lekérdezése után, de az `ExitBootServices()` hívás előtt módosítja a memóriát, a hívás meghiúsulhat, mert a memóriatérkép kulcsa megváltozik. A helyes megoldás a memóriatérkép lekérdezése, majd közvetlenül az `ExitBootServices()` hívása, szükség esetén ismételt lekérdezéssel.

A fenti hibák mindegyike azt mutatja, hogy a bootfolyamat minden lépése szigorúan meghatározott, és a fejlesztőnek pontosan ismernie kell a firmware és a bootloader közötti szerződést.

5.1.8 Labor: bootfolyamat lépéseinek felrajzolása

A labor célja, hogy reprodukálhatóan felrajzoljuk a bootfolyamat lépéseit, és méréssel igazoljuk az egyes szakaszok jelenlétét. A laborhoz a következő környezet szükséges:

- QEMU emulátor, például 8.2.x verzió.
- OVMF UEFI firmware, például az `edk2-ovmf` csomagból.
- Egy üres lemezkép, amelyre GPT partíciós táblát és FAT32 EFI rendszerpartíciót írunk.
- Egy minimális UEFI-alkalmazás vagy bootloader, amely a konzolra ír.

A kiinduló állapot: a hoszt gép egy friss, eldobható virtuális környezet, amelyen a fenti eszközök telepítve vannak. A valós lemezt, bootloadert és rendszerpartíciót nem módosítjuk.

Lépések:

1. Készítsünk egy 64 MiB méretű lemezképet:

   ```
   dd if=/dev/zero of=disk.img bs=1M count=64
   ```

2. Írjunk GPT partíciós táblát és hozzunk létre egy EFI rendszerpartíciót:

   ```
   parted disk.img mklabel gpt
   parted disk.img mkpart ESP fat32 1MiB 32MiB
   parted disk.img set 1 esp on
   ```

3. Formázzuk a partíciót FAT32-re, és másoljuk rá a minimális UEFI-alkalmazást:

   ```
   dd if=/dev/zero of=esp.img bs=1M count=32
   mkfs.vfat esp.img
   mount -o loop esp.img /mnt/esp
   cp bootx64.efi /mnt/esp/EFI/BOOT/BOOTX64.EFI
   umount /mnt/esp
   dd if=esp.img of=disk.img bs=1M seek=1 conv=notrunc
   ```

4. Indítsuk a rendszert QEMU-ban, és rögzítsük a soros konzol kimenetét:

   ```
   qemu-system-x86_64 \
     -bios /usr/share/OVMF/OVMF_CODE.fd \
     -drive file=disk.img,format=raw,if=virtio \
     -serial stdio \
     -m 512M
   ```

5. Mérjük meg a bootfolyamat időtartamát. A mérést a QEMU indításától a bootloader üzenetének megjelenéséig végezzük, például a `time` paranccsal vagy a firmware log timestampjei alapján. A mérést legalább ötször ismételjük meg.

A várt eredmény: a firmware inicializálási üzenetek után megjelenik a bootloader üzenete, például `Hello from UEFI`. A mért boot idő a konfiguráció függvényében néhány tizedmásodperctől néhány másodpercig terjedhet, de a pontos érték nem előre meghatározott, ezért a várt eredményt csak minőségi formában adjuk meg: a folyamat eljut a bootloader betöltéséig, és nincs hibaüzenet.

A mérési naplóban rögzítjük:

- a QEMU és az OVMF verzióját;
- a host CPU típusát és órajelfrekvenciáját;
- a lemezkép méretét és a partíciós táblát;
- a mérési módszert (például `time` parancs vagy timestamp elemzés);
- az öt mérési eredményt és azok átlagát;
- a várt eredményt, külön oszlopban feltüntetve, hogy az csak minőségi elvárás.

A labor bizonyítja, hogy a bootfolyamat szakaszai elkülöníthetők, és a firmware a bootloader betöltése előtt valóban inicializálja a hardvert. A mérés azt is megmutatja, hogy a boot idő mérése erősen függ a környezettől, ezért a mért értékeket mindig a konfigurációval együtt kell értelmezni.

5.1.9 Önellenőrző kérdések

1. Mi a különbség a BIOS és az UEFI között a programozási modell és a lemezformátum szempontjából?
2. Miért kell az `ExitBootServices()` hívást a memóriatérkép lekérdezése után, de a kernel saját eszközkezelőjének indítása előtt meghívni?
3. Hogyan működik együtt a Secure Boot és a mérési lánc a bootfolyamat integritásának biztosításában?

A fejezet bevezette a firmware és a boot szakaszok alapfogalmait, a BIOS és az UEFI közötti különbségeket, valamint a biztonságos indítás és a mérési lánc alapelveit. A következő fejezet ezekre építve a bootloader belső működését és a kernel betöltésének részleteit tárgyalja.

5.2 Bootloader, kernel betöltése és init

A bootloader az első olyan szoftverréteg, amely már nem a firmware, hanem a rendszer saját – vagy a rendszer által kiválasztott – kódja, és amelynek kifejezett célja, hogy egy másik programot, az operációs rendszer kernelét memóriába töltse, és átadja neki a vezérlést. Ez az átadás nem egyszerű `jmp` utasítás. A bootloader és a kernel között egy szerződés áll fenn: a bootloader pontosan meghatározott hardverállapotot, regisztertartalmat, memóriatérképet és paraméterblokkot hagy hátra, a kernel pedig erre a szerződésre támaszkodva veszi át a gépet. Ha a szerződés bármelyik oldalon sérül, a rendszerindítás rejtélyes módon megáll, vagy ami rosszabb, később, véletlenszerű időpontban hibásodik meg.

Ebben a fejezetben a bootloader és a kernel közötti átadási mechanizmust vizsgáljuk meg. A hangsúly a gyakorlatban is használható Multiboot interfészen van, mert az egyszerű, dokumentált, és több nyílt bootloader is támogatja. Emellett érintjük az initrd/initramfs fogalmát, a memóriatérkép átadását, a boot paramétereket, és egy olyan labort építünk fel, amelyben a kernel belépési pontját és a bootloader-konfigurációt együtt ellenőrizzük.

5.2.1 1. A bootloader feladata az átadás előtt

A bootloader munkája rendszerint három nagy lépésre bontható: beolvasás, előkészítés és átadás.

A **beolvasás** során a bootloader meghatározza, hol található a kernelkép. Ez lehet fájlrendszerben tárolt fájl, hálózati kiszolgálóról letöltött kép, vagy beágyazott rendszer esetén flashben elhelyezett partíció. A bootloader ehhez használhatja a firmware szolgáltatásait, például a BIOS `INT 13h` lemezelérését, vagy az UEFI által biztosított fájlrendszer-protokollt. A lényeg, hogy a kernelkép bájtjait fizikai memóriába, a kernel által elvárt címre kell tölteni.

A **betöltés célcíme** azért kritikus, mert a kernel gyakran még nem rendelkezik lapozással, és a belépéskor a fizikai címekkel dolgozik. A hagyományos x86-os rendszereken a kernelkép tipikus betöltési címe 1 MiB (`0x100000`), mert ez alatt a cím alatt helyezkedik el a valós módú firmware, a BIOS adatterülete, a videomemória és más fenntartott tartományok. A Multiboot specifikáció is ezt a címet ajánlja alapértelmezésként. A linker scriptben ezért rögzítjük a kernel szegmenseit erre a címre; ha a bootloader nem ugyanoda tölti be, akkor az abszolút címekkel rendelkező kód azonnal hibás memóriára mutat.

A **betöltés után jön az előkészítés**: a bootloader elvégzi azokat a hardver-beállításokat, amelyeket a kernel már nem akar vagy nem tud megtenni. Ez magában foglalhatja a videomód beállítását, a memóriatérkép lekérdezését, az ACPI-táblák vagy device tree bináris elhelyezését, valamint további fájlok – például initrd/initramfs – betöltését. A bootloader ezeket a kiegészítő fájlokat a kernelkép mellé, külön memóriaterületekre helyezi.

Végül az **átadás** következik: a bootloader beállítja a CPU üzemmódját, letiltja a megszakításokat, beállítja a stacket – vagy átadja a kernelnek, hogy állítsa be –, betölti a megfelelő regiszterekbe az átadási információkat, majd a kernel belépési pontjára ugrik. Ez a belépési pont általában assemblyben íródott, mert itt még nincs C hívási környezet, és a regisztereket, valamint a stacket közvetlenül kell kezelni.

5.2.2 2. A bootloader és a kernel közötti szerződés

Az átadási szerződés több elemből áll:

- **CPU üzemmód**: a bootloader garantálja, hogy a kernel milyen üzemmódban indul. A klasszikus x86-os világban ez lehet valós mód, 32 bites védett mód, vagy 64 bites hosszú mód. A Multiboot specifikáció alapértelmezésben 32 bites védett módban adja át a vezérlést; a Multiboot2 rendelkezik 64 bites kiterjesztéssel is. A kernel belépő kódja nem feltételezhet ennél többet, ezért például a lapozást, a megszakításkezelést és a védelmi gyűrűket magának kell beállítania.
- **Megszakítások állapota**: a legtöbb bootloader letiltott megszakításokkal (`IF=0`) adja át a vezérlést. Ez azért fontos, mert a kernel még nem telepítette a saját interruptkezelőit, és egy beérkező megszakítás a firmware vagy a bootloader kódjába vihetne.
- **Regiszterek**: a bootloader a megállapodott regiszterekben adja át a belépési bűvös számot (magic) és az információblokk mutatóját. A Multiboot2 esetén `EAX`-be a `0x36D76289` bűvös szám, `EBX`-be pedig a Multiboot2 információstruktúra fizikai címe kerül. A kernelnek ezt a bűvös számot ellenőriznie kell, mielőtt az `EBX` mutatót használná.
- **Stack**: vagy a bootloader állítja be, vagy a kernel belépő assemblykódja. A C kód futtatásához a stacknek érvényes, írható memóriára kell mutatnia. A stack hiánya az egyik leggyakoribb oka annak, hogy a kernel az első C függvényhívásnál azonnal megáll.

- **Memóriatérkép**: a bootloader átadja a firmware-től kapott memóriatérképet, amelyből a kernel megtudhatja, mely fizikai címtartományok használhatók szabad memóriaként, és melyek fenntartottak. Ez azért fontos, mert a kernelnek pontosan tudnia kell, hová helyezhet lapokat, folyamatokat és puffereket anélkül, hogy felülírná a saját kódját, az initrd-t, az ACPI-táblákat vagy a hardver regisztereit.
- **Boot paraméterek**: szöveges parancssor vagy bináris struktúra, amelyben a felhasználó és a bootloader konfigurációja eljut a kernelhez. Ilyen lehet például a `root=/dev/sda1` vagy az `rdinit=/init` bejegyzés.
- **Modulok**: a bootloader további blokkokat tölthet be, például initrd/initramfs-t, mikrokernel-szolgáltatásokat vagy illesztőprogramokat. A kernel ezek listáját az információs struktúrában kapja meg.

A szerződés betartását a kernel oldaláról is ellenőrizni kell. A bootloader hibázhat, vagy egy támadó módosíthatja a bootfolyamatot, ezért a kernel nem bízhat vakon az átadott mutatókban és méretekben.

5.2.3 3. Multiboot mint átadási interfész

A Multiboot specifikáció azért született, hogy a bootloader és a kernel közötti interfészt szabványosítsa. A gyakorlatban két változat terjedt el:

- **Multiboot 1** (`Multiboot Specification 0.6.96`): egyszerű, de kevésbé bővíthető struktúrákat használ. A kernel fejlécében egy rögzített `multiboot_header` struktúra szerepel, és a bootloader egy `multiboot_info` struktúrát ad át.
- **Multiboot 2**: címkézett, TLV-szerű felépítésű. A kernel fejlécében címkék (tag-ek) sorozata van, az információstruktúra pedig szintén tag-ekből áll. Ez rugalmasabbá teszi a bővítést.

A Multiboot előnye, hogy ugyanaz a kernel több bootloaderrel is indítható: GRUB-tal, SYSLINUX-szal, a QEMU beépített Multiboot loaderével, vagy akár egy saját bootloaderrel. A hátránya, hogy a specifikáció bizonyos részleteket a bootloaderre bíz, és a kernelnek többféle konfigurációt is kezelnie kell.

A Multiboot1 és Multiboot2 fejlécét a kernelkép elején kell elhelyezni. A fejlécnek 4 bájtra igazítottnak kell lennie, és a kernelfájl első 8192 bájtján belül kell szerepelnie, különben a bootloader nem találja meg. A fejlécben a **magic**, a **flags** mező, a **header_length** és a **checksum** szerepel; a checksum számítása: `-(magic + flags + header_length)` modulon `2^32`. Ha a checksum nem stimmel, a bootloader elutasítja a kernelt.

A Multiboot2 fejléc felépítése tag-alapú. Az első három 32 bites mező a magic (`0xE85250D6`), az architecture (0 az i386/védett mód esetén) és a teljes fejléc hossza. A negyedik mező a checksum: `-(magic + architecture + header_length)`. Utána opcionális címkék következnek, a fejlécet pedig egy 8 bájtos, nullázott end tag zárja.

Egy minimális Multiboot2 fejléc assemblyben így nézhet ki:

```
# boot.s - Multiboot2 header és belépési pont
.set MULTIBOOT2_MAGIC,    0xE85250D6
.set MULTIBOOT2_ARCH,     0
.set GRUB_MAGIC,          0x36D76289

.section .multiboot, "a"
.align 8
header_start:
    .long MULTIBOOT2_MAGIC
    .long MULTIBOOT2_ARCH
```

```
    .long header_end - header_start
    .long -(MULTIBOOT2_MAGIC + MULTIBOOT2_ARCH + (header_end - header_start))
    # End tag: type = 0, flags = 0, size = 8
    .short 0
    .short 0
    .long 8
header_end:

.section .bss
.align 16
stack_bottom:
    .skip 16384
stack_top:

.section .text
.global _start
_start:
    mov $stack_top, %esp
    push %ebx        # Multiboot2 info mutató
    push %eax        # Multiboot2 bootloader magic
    call kernel_main
    cli
1:  hlt
    jmp 1b
```

A fenti kód szemléltetés, nem valódi gépen futtatandó utasítás. Az `%esp` beállítása után a két argumentumot verembe tesszük, majd meghívjuk a C-ben írt `kernel_main` függvényt. A `kernel_main` visszatérése után a kernel leállítja a megszakításokat, és végtelen ciklusba kerül.

A Multiboot2 bootloader a kernel belépésekor az `EAX`-be a `0x36D76289` bűvös számot, az `EBX`-be pedig a Multiboot2 információstruktúra fizikai címét írja. Az információstruktúra elején egy 8 bájtos fejléc áll:

```
uint32_t total_size;   // a teljes struktúra mérete
uint32_t reserved;     // 0
```

Ezután következnek a címkék. Minden címke fejlécében `uint32_t type` és `uint32_t size` szerepel; a `size` magában foglalja a címke teljes hosszát, 8 bájtra igazítva. A kernel a `total_size`-ot és az egyes címkék `size` mezőjét is ellenőrzi, hogy ne olvasson túl az érvényes területen.

A leggyakrabban használt Multiboot2 címkék:

- **Boot command line** (`type = 1`): nullterminált parancssor.
- **Boot loader name** (`type = 2`): a bootloader neve.
- **Module** (`type = 3`): egy betöltött modul; a struktúra tartalmazza a modul kezdő- és végecímét, valamint egy opcionális nullterminált nevet.
- **Basic memory information** (`type = 4`): az alsó és felső memória mérete KiB-ban.
- **Memory map** (`type = 6`): a firmware-től kapott memóriatérkép normalizált formában.
- **Framebuffer information** (`type = 8`): a grafikus kimenetre vonatkozó adatok.

5.2.4 4. Memóriatérkép és boot paraméterek

A memóriatérkép a kernel egyik legfontosabb bemenete. A bootloader a firmware-től kapott nyers adatokat normalizálja, és a kernel számára feldolgozható formában adja át. Az x86-os BIOS esetén ez gyakran az **E820** interfészen keresztül kapott lista; az UEFI esetén az `ExitBootServices` előtt lekérdezett `EFI_MEMORY_DESCRIPTOR` lista. A Multiboot2 `Memory map` címkéje mindkét esetben egységes formát ad.

A Multiboot2 memóriatérkép-címke felépítése:

```
struct multiboot_tag_mmap {
    uint32_t type;           // 6
    uint32_t size;           // címke teljes mérete
    uint32_t entry_size;     // egy bejegyzés mérete, >= 24
    uint32_t entry_version;  // 0
    struct {
        uint64_t base_addr;  // a tartomány kezdő fizikai címe
        uint64_t length;     // hossz bájtban
        uint32_t type;       // 1 = szabad, 2 = fenntartott, ...
        uint32_t reserved;
    } entries[];
};
```

A `type` mező az E820 szabványhoz hasonló értékeket vehet fel: `1` szabad, `2` fenntartott, `3` ACPI reclaimable, `4` ACPI NVS, `5` hibás vagy nem használható. A kernel a szabad tartományokból építi fel a fizikai lapallokátort. Fontos, hogy a kernel saját betöltési címe, a Multiboot struktúrák és az initrd modulok által lefedett területeket külön le kell foglalnia, még akkor is, ha a memóriatérkép azokat szabadnak jelölné. A bootloader ugyanis gyakran a memóriatérképen szabadnak jelölt területre tölti be a kernelt és a modulokat, ezért a kernelnek a saját címtartományát és a modulokét is ki kell metszenie a szabad listából.

A boot paraméterek szöveges parancssorként vagy bináris struktúrában érkeznek. A Multiboot2 `Boot command line` címke egy nullterminált karakterlánc. A kernel ezt a saját logikájával elemzi; a gyakori paraméterek közé tartozik a gyökérfájlrendszer (`root=`), az init program (`init=` vagy `rdinit=`), a soros konzol (`console=`), a debug szint (`loglevel=`) és számos hardveres beállítás. A parancssor hossza a bootloader konfigurációjától függ, ezért a kernel nem feltételezhet fix méretet; a karakterlánc végét a nullterminátor jelöli.

A parancssor feldolgozásakor a kernelnek különös figyelmet kell fordítania a biztonságra: a boot paramétereket nem szabad hitelesítő adat, jelszó vagy belső hálózati cím tárolására használni. A rendszernaplóba kerülő parancssor gyakran tartalmazhat érzékeny adatot is, ezért a naplózás előtt meg kell tisztítani.

5.2.5 5. Initrd és initramfs

A bootloader gyakran nemcsak a kernelt tölti be, hanem egy kezdeti felhasználói teret is. Ennek két fő formája van: az initrd és az initramfs.

Az **initrd** (initial RAM disk) hagyományosan egy blokkeszköz-kép, amelyet a kernel egy RAM-lemezre (`/dev/ram0`) tölt. A kernel a rendszerindítás során ezt a blokkeszközt mountolja gyökérfájlrendszerként, majd futtatja róla az init programot. Az initrd előnye, hogy egyszerűen előállítható, és bármilyen fájlrendszerformátumot használhat. Hátránya viszont, hogy a kernelben szükség van a RAM-lemez és a fájlrendszer meghajtójára, és a blokkeszköz-réteg extra másolást és memóriát igényel.

Az **initramfs** ezzel szemben nem blokkeszköz, hanem egy archívum – Linux alatt általában cpio formátum, opcionálisan gzip tömörítéssel –, amelyet a kernel közvetlenül a memóriába csomagol ki. A kicsomagolás után a tartalom egy `rootfs` nevű, memóriaalapú fájlrendszerbe kerül, amelyet a kernel a rendszerindítás során gyökérfájlrendszerként használ. A kernel ezután a `rootfs`-en lévő `/init` programot futtatja, kivéve, ha a `rdinit=` paraméter mást ad meg. A `/init` felelős a valódi gyökérfájlrendszer mountolásáért, a szükséges kernelmodulok betöltéséért, majd a `switch_root` vagy `pivot_root` művelettel lecseréli a gyökeret a valódi fájlrendszerre.

A Multiboot specifikációban az initrd/initramfs egy **modul**. A bootloader a modult a kernelképhez hasonlóan fizikai memóriába tölti, és a Multiboot2 `Module` címkében adja át a kezdőcímet, a végecímet és az opcionális nevet. A kernel a modulok listájából tudja meg, hol található az initramfs. A modul memóriaterületét a kernelnek foglaltként kell kezelnie, amíg az initramfs-t ki nem csomagolta, vagy a modulra többé nincs szükség.

A Multiboot2 modul-címke például így olvasható fel egy egyszerű kernelben:

```
#include <stdint.h>

#define MULTIBOOT2_INFO_MAGIC 0x36D76289
#define MULTIBOOT_TAG_TYPE_MODULE 3

struct multiboot_tag {
    uint32_t type;
    uint32_t size;
};

struct multiboot_tag_module {
    uint32_t type;
    uint32_t size;
    uint32_t mod_start;
    uint32_t mod_end;
    char cmdline[];
};

void kernel_main(uint32_t magic, uint32_t info_addr) {
    if (magic != MULTIBOOT2_INFO_MAGIC) {
        panic("bad magic");
    }

    struct multiboot_tag *tag = (struct multiboot_tag *)(uintptr_t)info_addr;
    uint32_t total_size = *(uint32_t *)(uintptr_t)info_addr;
    uint32_t pos = 8; // az info fejléc után

    while (pos + sizeof(struct multiboot_tag) <= total_size) {
        tag = (struct multiboot_tag *)(info_addr + pos);
        if (tag->size == 0 || pos + tag->size > total_size) {
            panic("invalid tag size");
        }

        if (tag->type == MULTIBOOT_TAG_TYPE_MODULE) {
            struct multiboot_tag_module *mod =
                (struct multiboot_tag_module *)(uintptr_t)tag;
            serial_print("module start: ");
            serial_print_hex(mod->mod_start);
            serial_print(" module end: ");
            serial_print_hex(mod->mod_end);
            serial_print("\n");
        }

        pos += (tag->size + 7) & ~7u;
    }
}
```

Ez a kód szemléltetés. A `struct multiboot_tag_module` mezői a Multiboot2 specifikáció szerint 8 bájtra igazítottak, és a `cmdline` csak akkor érvényes, ha a modulnév jelen van; a gyakorlatban a struktúra pontos elrendezését a cél architektúra és a specifikáció verziója szerint ellenőrizni kell, és célszerű bájtszintű elemzést használni a csomagolási eltérések elkerülésére.

A modul `mod_start` és `mod_end` mezői fizikai címek, mivel a kernel belépésekor még nincs lapozás. A kernel ezeket a címeket használhatja az initramfs tartalmának elérésére, de a tartalmat először

ellenőriznie kell, és a modul határain kívül nem olvashat.

5.2.6 6. Init folyamat és a userland előkészítése

A kernel betöltése után a rendszerindítás utolsó lépése a felhasználói tér első folyamatának, az **init**-nek az elindítása. Az init a `PID 1` azonosítóval fut, és felelős a rendszer további szolgáltatásainak indításáért, a terminálok és démonok elindításáért, valamint a rendszer leállításáért.

Két gyakori indítási út létezik:

1. **Initramfs-sel**: a kernel kicsomagolja az initramfs-t a `rootfs`-be, majd megpróbálja futtatni a `/init` programot. Ez a program gyakran shell script vagy kis statikus bináris, amely betölti a szükséges kernelmodulokat, mountolja a valódi gyökérfájlrendszert, és `switch_root` vagy `pivot_root` segítségével átadja a vezérlést a valódi initnek.
2. **Közvetlen root mounttal**: ha nincs initramfs, a kernel a boot paraméterben megadott gyökérfájlrendszert mountolja, majd elindítja a megadott init programot, általában `/sbin/init`. Ebben az esetben a kernelnek már tartalmaznia kell a gyökérfájlrendszer és a blokkeszköz meghajtóját, mert nincs korai felhasználói tér, amely betölthetné azokat.

Az initramfs használata rugalmasabb rendszerindítást tesz lehetővé, mert a felhasználói térben futó `/init` program bonyolult logikát is megvalósíthat: felismerheti a hardvert, dönthet a gyökérfájlrendszer típusáról, betöltheti a megfelelő modulokat, és kezelheti a titkosított lemezek feloldását is. A kernel szempontjából azonban a `/init` is csak egy felhasználói program; a kernel a `execve` rendszerhívással indítja, és a szokásos folyamat-modell szerint működik.

A `pivot_root` rendszerhívás a gyökérfájlrendszer lecserélésére szolgál: a jelenlegi gyökérfájlrendszert egy könyvtárba mozgatja, és egy másik fájlrendszert tesz gyökérré. A `switch_root` ezzel szemben az initramfs elhagyására specializált művelet, amely törli a régi gyökérfájlrendszer tartalmát, és átvált az új gyökérre. A két művelet részletei a fájlrendszeres és rendszerhívásos fejezetekben kerülnek elő; itt elég annyi, hogy az initramfs `/init` programja ezek egyikével adja át a vezérlést a valódi rendszernek.

5.2.7 7. Tipikus hibák és megoldások

A bootloader és a kernel közötti átadás során a hibák gyakran tünetmentesek, vagy csak nagyon korai, nehezen értelmezhető leállásokban jelentkeznek. A leggyakoribb hibák:

1. **Hiányzó vagy hibás Multiboot fejléc.** Ha a kernelkép nem tartalmazza a fejlécet, vagy a fejléc nem a fájl elején, 4 bájtra igazítva található, a bootloader nem ismeri fel a kernelt, és hibaüzenettel leáll. A megoldás: a fejléc külön szekcióba kerüljön, és a linker script a szekciót a fájl legelejére helyezze.

2. **Rossz checksum.** A Multiboot fejlécben a checksum mezőnek pontosan `-(magic + flags + header_length)` értékűnek kell lennie. Ha az érték hibás, a bootloader érvénytelennek minősíti a kernelt. A hiba gyakran abból adódik, hogy a fejléc hossza nem tartalmazza az end taget, vagy a szerkesztő nem frissítette a checksumot.

3. **A kernel nem ellenőrzi a magic értéket.** Ha a kernel belépő kódja vakon használja az `EBX` mutatót, és a bootloader mégsem Multiboot módon indította, a kernel szemétmemóriát olvas. A megoldás: a `kernel_main` első utasítása a magic értékének ellenőrzése legyen, és hiba esetén álljon meg biztonságosan.

4. **Nincs stack beállítva.** A C kód futtatása érvényes stack nélkül azonnali hibához vezet. A belépő assemblykódnak mindig be kell állítania a stacket, még akkor is, ha a bootloader állítólag beállította. A példában a `.bss` szekcióban lefoglalt `stack_top` mutatót töltjük `%esp`-be.

5. **A memóriatérkép helytelen feldolgozása.** A Multiboot2 memóriatérkép bejegyzései nem feltétlenül rendezettek, és a `type` mező nem mindig egyértelmű. A kernelnek a `entry_size` és a `size` mezőket is ellenőriznie kell, és a szabad memóriát csak a saját foglalásai levonása után szabad használnia.

6. **Az initrd felülírása.** A kernel memóriaallokátora, ha már korán működik, könnyen felülírhatja a bootloader által betöltött modulokat. A megoldás: a kernel a modulok és a saját kód által lefedett területeket foglaltként jelöli a szabadlap-listában.

7. **A modulok méretének figyelmen kívül hagyása.** A Multiboot2 `Module` címkében a `mod_start` és `mod_end` adja meg a modul határait. Ha a kernel csak a kezdőcímet tárolja, és nem ellenőrzi a méretet, könnyen túlolvashat vagy túlíthat a modul területén.

8. **A boot paraméterek nem biztonságos kezelése.** A parancssor hosszára nincs felső korlát, és a tartalma a bootloader konfigurációjából származik. A kernelnek a stringet másolás előtt hosszlimitálnia kell, és soha nem szabad a parancssor tartalmát ellenőrizetlenül felhasználnia.

5.2.8 8. Labor: egyszerű bootloader és kernel belépési pont tervezése

A labor célja, hogy egy Multiboot2 kernelt és egy bootloader-konfigurációt hozzunk létre, amely QEMU-ban elindul, ellenőrzi a magic értéket, bejárja a memóriatérképet, és kiírja az initrd modul adatait. A labor kizárólag emulált környezetben fut; a hoszt valódi bootloaderét, lemezét és rendszerpartícióját tilos módosítani.

Kiinduló állapot. A hosztgépen elérhető egy i686-elf cross-toolchain (GCC, binutils), a GRUB `grub-mkrescue` eszköze, valamint a QEMU. A pontos parancsok disztribúciófüggők; a reprodukciós csomagban a használt verziókat rögzíteni kell. A laborhoz használt konfiguráció: QEMU `i386`, 64 MiB memória, soros konzol.

1. lépés: a kernel forrásfájljainak létrehozása. Hozzuk létre a `boot.s`, `kernel.c` és `linker.ld` fájlokat a fent bemutatott kódok alapján. A `kernel.c` a soros portot inicializálja, kiírja a `MAGIC OK` üzenetet, bejárja a Multiboot2 címkéket, és kilistázza a memóriatérkép szabad tartományait, valamint a modulokat.

2. lépés: a build végrehajtása. A fordítási parancsok:

```
i686-elf-as boot.s -o boot.o
i686-elf-gcc -ffreestanding -m32 -c kernel.c -o kernel.o
i686-elf-ld -T linker.ld -o myos.elf boot.o kernel.o
```

A `linker.ld` tartalma:

```
ENTRY(_start)
SECTIONS {
    . = 1M;
    .multiboot : { *(.multiboot) }
    .text : { *(.text*) }
    .rodata : { *(.rodata*) }
    .data : { *(.data*) }
    .bss : { *(COMMON) *(.bss*) }
}
```

A kernelkép betöltési címe 1 MiB, a Multiboot specifikáció szerint.

3. lépés: az initrd modul előállítása. Készítsünk egy kis cpio archívumot `initrd.img` néven:

```
mkdir initramfs
echo "hello from initrd" > initramfs/README
(cd initramfs && find . | cpio -o -H newc) | gzip > initrd.img
```

A kernel ebben a laborban nem csomagolja ki az initramfs-t; csak a modul jelenlétét és méretét ellenőrzi.

4. lépés: bootolható ISO készítése. Hozzunk létre egy GRUB-konfigurációt:

```
menuentry "myos" {
    multiboot2 /boot/myos.elf
    module2 /boot/initrd.img
}
```

Helyezzük a `myos.elf` és `initrd.img` fájlokat a `iso/boot/` könyvtárba, és futtassuk a `grub-mkrescue`-t:

```
grub-mkrescue -o myos.iso iso
```

5. lépés: futtatás QEMU-ban. Indítsuk a rendszert soros konzollal:

```
qemu-system-i386 -m 64 -cdrom myos.iso -serial stdio -no-reboot
```

A kernel a soros portra írja az üzeneteket. A várt kimenet tartalmazza a `MAGIC OK` sort, a memóriatérkép szabad tartományait, és az initrd modul kezdő- és végecímét.

6. lépés: mérés. Végezzük el a bootfolyamat időmérését ötször. A méréshez használjunk például `date +%s.%N`-et a QEMU indítása előtt, és a soros porton megjelenő `MAGIC OK` sor időpontját. A mérési pontosságot a hoszt órája és a soros port pufferelése befolyásolja, ezért a mérést csak reprodukciós célból, nem teljesítménygaranciaként értelmezzük.

7. lépés: mérési napló. A mérési naplóban a mért és a várt értékeket külön oszlopban tüntessük fel.

Futás	Boot idő (s, mért)	Szabad memória összesen (KiB, mért)	Initrd méret (bájt, mért)	Boot idő (várt)	Szabad memória (várt)	Initrd méret (várt)
1				$< 1{,}0$	< 64 MiB, $>$ 60 MiB	az `initrd.img` mérete
2				$< 1{,}0$	< 64 MiB, $>$ 60 MiB	az `initrd.img` mérete
3				$< 1{,}0$	< 64 MiB, $>$ 60 MiB	az `initrd.img` mérete
4				$< 1{,}0$	< 64 MiB, $>$ 60 MiB	az `initrd.img` mérete
5				$< 1{,}0$	< 64 MiB, $>$ 60 MiB	az `initrd.img` mérete

A várt szabad memória azért kevesebb 64 MiB-nál, mert a kernel, a Multiboot struktúrák, a modul és a firmware számára fenntartott területek foglaltnak számítanak. A pontos érték a buildtől és a QEMU verziójától függ; a laborban a mért értéket a naplóba kell írni.

8. lépés: visszaállítás. A laborhoz használt könyvtár és az ISO fájl törölhető. A hoszt rendszer bootloadere és lemeze nem változott.

5.2.9 9. Önellenőrző kérdések

1. Miért kell a Multiboot fejlécnek a kernelkép elején, 4 bájtra igazítva elhelyezkednie, és mi a checksum szerepe?
2. Milyen regiszterekben adja át a Multiboot2 bootloader a magic értéket és az információ-struktúra mutatóját? Miért kell a kernelnek ellenőriznie a magic értéket?
3. Mi a különbség az initrd és az initramfs között? Melyik miért előnyösebb a korai felhasználói tér kialakítására?
4. Hogyan kerülheti el a kernel, hogy felülírja a bootloader által betöltött modulokat a memóriaallokáció során?
5. Milyen mezőket kell ellenőriznie a kernelnek a Multiboot2 memóriatérkép feldolgozásakor, és miért?

5.2.10 Összefoglalás

A bootloader és a kernel közötti átadás egy pontos szerződés, amely magában foglalja a CPU üzemmódot, a regisztereket, a stacket, a memóriatérképet, a boot paramétereket és a betöltött modulokat. A Multiboot specifikáció ezt a szerződést szabványosítja, és lehetővé teszi, hogy ugyanaz a kernel több bootloaderrel is induljon. A kernel belépő kódja assemblyben ellenőrzi a magic értéket, beállítja a stacket, majd átadja a vezérlést a C nyelvű inicializálónak. A memóriatérkép és a modulok feldolgozása során a kernelnek minden mutatót és méretet ellenőriznie kell, mert a bootloader hibája vagy egy rosszindulatú módosítás korai, nehezen felderíthető hibákhoz vezethet. Az initrd/initramfs a modulokon keresztül jut el a kernelhez, és a korai felhasználói tér elindításával teszi rugalmassá a rendszerindítást.

A következő fejezetben ezekre az átadási mechanizmusokra építve megvizsgáljuk a firmware, az ACPI és a device tree szerepét a hardverleírásban, valamint azt, hogy a kernel hogyan használja ezeket az eszközök és megszakítások felfedezésére.

5.3 Firmware, ACPI, device tree és hardverleírás

A hardver konfigurációjának leírása és az operációs rendszer értesítése két, egymást kiegészítő mechanizmuson alapul. Az egyik az asztali és szerver platformokon elterjedt ACPI (Advanced Configuration and Power Interface), a másik a beágyazott rendszerekben szabványos device tree. Míg az ACPI táblázatokat és bájtkódot szolgáltat a firmware-től, addig a device tree egy rendszerindításkor a kernelbe töltött, szövegesen is leírható fa szerkezetű adatmodell. A közös cél az, hogy az operációs rendszer a futása során felfedezhesse a rendelkezésre álló eszközöket, azok erőforrásait, megszakításait és energiaállapotait, valamint reagálhasson a futás közben bekövetkező hardverváltozásokra. E fejezet ezt a két leíró modellt, az általuk megvalósított OS-értesítési útvonalakat, valamint a hotplug és az energiaállapot-kezelés alapjait tárgyalja. A fogalmakra a következő, megszakításokkal és eszközök felfedezésével foglalkozó egység épít.

5.3.1 A firmware szerepe

A firmware az a gyártó által biztosított, nem felejtő memóriában tárolt programkód, amely a processzor első utasításainak végrehajtásától a bootloader elindításáig, illetve a rendszer futása alatt is szolgáltatásokat nyújt. A 14. alfejezetben bemutatott BIOS/UEFI bootfolyamat során a firmware elvégzi a hardver inicializálását, majd a hardverleírást elérhetővé teszi a betöltött operációs rendszer számára. E leírás nélkül a kernelnek vakon kellene feltételeznie, milyen eszközök és erőforrások léteznek, ami hordozhatatlanná és törékennyé tenné a rendszert. A firmware tehát három fontos feladatot lát el a hardverleírás szempontjából:

1. **Összegyűjti és elérhetővé teszi** a hardverkonfigurációra vonatkozó statikus adatokat (CPU-k, memóriatartományok, eszközök, megszakítási vonalak).
2. **Futásidejű szolgáltatásokat** biztosít, például az ACPI-s eszközök értesítési mechanizmusát vagy a rendszer alvó állapotba helyezését.
3. **Felfedezi és konfigurálja** a boot során a hardvert, majd átadja az operációs rendszernek a leíró adatszerkezeteket.

Az operációs rendszer az így kapott leírást a saját eszközmodelljére képezi le, illeszti a megfelelő meghajtókat, és felépíti a futáshoz szükséges belső struktúrákat.

5.3.2 ACPI: a hardverleírás asztali és szerver rendszereken

Az ACPI a PC-kompatibilis platformok domináns hardverleíró és energiagazdálkodási szabványa. A specifikációt az UEFI Forum gondozza, és a gyártók a firmware-ben, általában a ROM-ban tárolják a táblázatokat. Az ACPI nem csupán adatleírás: tartalmaz egy futtatható bájtkódot, az AML-t (ACPI Machine Language), amelyet a firmware az OSPM (Operating System-directed configuration and Power Management) számára biztosít. Az OSPM felelős a rendszer energiagazdálkodási és eszköz-konfigurációs döntéseiért.

5.3.3 ACPI táblák és AML

Az ACPI adatszerkezetei hierarchikusan szervezett táblák, amelyek a fizikai memória egy rögzített tartományában, a firmware által elérhetővé tett módon helyezkednek el. A kernel a boot során megkeresi az RSDP (Root System Description Pointer) mutatót, amelyből eljut a RSDT (Root System Description Table) vagy a modernebb XSDT (Extended System Description Table) táblához. Az XSDT 64 bites fizikai címeket tartalmaz, és egymás után sorolja fel a további ACPI-táblák címeit.

A leggyakoribb táblák:

- **FADT (Fixed ACPI Description Table):** a rendszer alapvető energia- és időzítő jellemzőit, valamint a firmware regisztereinek címeit tartalmazza.
- **MADT (Multiple APIC Description Table):** az APIC, a lokális és I/O APIC megszakítási vezérlők, valamint a CPU-k leírása.
- **DSDT (Differentiated System Description Table):** a platform eszközeinek hierarchiáját és az azokhoz tartozó AML metódusokat tartalmazza. Ez a fő forrása az eszközök felfedezésének.
- **SSDT (Secondary System Description Table):** kiegészítő leírások, például további CPU-állapotok, eszközök vagy hotplug-információk.

A DSDT és az SSDT táblákban található AML egy bájtkód, amelyet az ACPI interpreter hajt végre. Az AML utasításokat tartalmazhat egy eszköz állapotának lekérdezésére (_STA), erőforrásainak leírására (_CRS), megszakítási útvonalak lekérdezésére (_PRT), vagy egy eszköz ki- és bekapcsolására (_PS0, _PS3). E metódusok a firmware által definiált, de az operációs rendszer által végrehajtott kódot jelentenek, ezért a kernel ACPI-értelmezője gondosan validálja a bájtkódot, mielőtt futtatja.

A következő, szemléltető ASL (ACPI Source Language) kódrészlet egy PCI-hidat ír le:

```
DefinitionBlock ("dsdt.aml", "DSDT", 2, "ACME", "BOARD", 0x00000001)
{
    Scope (\_SB) {
        Device (PCI0) {
            Name (_HID, EisaId ("PNP0A08"))   /* PCI Express Root Complex */
            Name (_CRS, ResourceTemplate () {
                WordBusNumber (ResourceConsumer, MinFixed, MaxFixed, PosDecode,
                    0x0000, 0x00FF, 0x0000, 0x0100)
                Memory32Fixed (ReadWrite, 0xE0000000, 0x10000000)
            })
            Method (_STA) { Return (0x0F) }   /* eszköz jelen van és be van kapcsolva */
        }
    }
}
```

Az ASL forrásból az ASL-fordító (iasl) készíti el az AML bájtkódot, amelyet a DSDT-be csomagol a firmware. A kernel a `_STA` metódus segítségével állapíthatja meg, hogy az eszköz jelen van-e, működőképes-e és be van-e kapcsolva. A `_CRS` a processzornak adott erőforrásokat (I/O portok, memóriatartományok, megszakítások) írja le.

5.3.4 ACPI namespace és objektumok

Az ACPI egy fa szerkezetű névteret definiál, amelyben a platformobjektumok és azok metódusai élnek. A névtér gyökere a `\` (backslash). A fenti példában a `\_SB` a system bus, amely alatt a platformeszközök találhatók. Az ACPI specifikáció előre meghatározott névtér-objektumokat és metódusneveket rögzít, például:

- `\_SB` – rendszerbusz,
- `\_PR` – processzorok,
- `\_TZ` – hőzónák,
- `\_GPE` – általános célú események,
- `\_OS`, `\_OSI` – az operációs rendszer azonosítására szolgáló objektumok.

A névtér bejárásával az operációs rendszer kiépíti az eszközök belső reprezentációját, és az egyes eszközökhöz meghajtókat társít. A meghajtók nem közvetlenül az ACPI-t, hanem a kernel eszközmodelljét használják, az ACPI csak a felfedezés és a konfiguráció forrása.

5.3.5 Rendszeresemények és hotplug

Az ACPI nem csupán statikus leírást ad, hanem futás közbeni értesítéseket is támogat. A hardveres eseményeket – például egy dokkolóállomás csatlakoztatását, egy PCIe-kártya behelyezését vagy egy fedél lecsukását – a firmware gyakran egy általános célú eseményként (GPE, General Purpose Event) jelenti. A GPE egy megszakítást vált ki, amelyet az ACPI alrendszer kezel. A GPE-hez tartozó AML-metódus (általában `_Lxx` vagy `_Exx`) fut le, és értesíti a megfelelő ACPI-objektumot egy Notify esemény segítségével.

Például egy laptop dokkolóállomása egy ACPI-eszközként van leírva a DSDT-ben. Amikor a felhasználó bedugja a laptopot a dokkolóba, a hardver GPE-t generál, az AML lefuttat egy `Notify (\_SB.DOCK, 0x00)` parancsot, és az operációs rendszer ACPI-meghajtója elvégzi a dokkolóeszközök újraellenőrzését. Hasonló mechanizmussal értesül a rendszer a processzorhőmérséklet-emelkedésről vagy egy akkumulátor állapotváltozásáról.

Az ACPI hotplug tehát két részből áll: a hardver által kiváltott GPE-ből és a firmware által definiált AML-értesítésből, amelyet az operációs rendszer az eszköz újra-felderítésére használ. A kernel ilyenkor lekérdezi az eszköz `_STA` állapotát, és ha az megváltozott, létrehozza vagy eltávolítja a belső eszközobjektumot.

5.3.6 Device tree: a beágyazott világ hardverleírása

Az ACPI a PC-k világában szabvány, de a beágyazott, mobil és sok ARM-alapú rendszerben nincs egységes firmware, amely ACPI-t biztosítana. Itt a device tree (DT) terjedt el, amely egy adatszerkezet, nem futtatható kód. A DT a hardvert fa formában írja le: a gyökér node alatt találhatók a CPU-k, memória, rendszerbusz, perifériák és azok kapcsolatai. A leírást a rendszerboot során a bootloader tölti be a memóriába, és átadja a kernelnek fizikai mutató formájában. A kernel ezt elemzi, és az egyes node-okból platformeszközöket, illetve eszközöket hoz létre.

A device tree forrásszövege a DTS (Device Tree Source), amelyet a `dtc` (Device Tree Compiler) fordít egy bináris DTB (Device Tree Blob) formátumra. A DTS ember által olvasható, C-szerű szintaxisú; a bináris változat tömör és a kernel számára hatékonyan elemezhető.

5.3.7 Node, property, compatible

A DT alapelemei a node-ok és a property-k. Minden node rendelkezik egy névvel, opcionálisan egységcímzéssel, és property-ket tartalmazhat. A property egy név-érték pár, ahol az érték típusa lehet egész, string, bájtok vagy azok tömbje. A legfontosabb property a `compatible`, amely egy string-tömb, és azt adja meg, melyik meghajtó vagy eszközkompatibilitási lista illeszkedik az adott node-hoz.

Példa egy ARM virtuális gép device tree-jának részletére:

```
/dts-v1/;

/ {
    compatible = "acme,arm-iot-board";
    #address-cells = <2>;
    #size-cells = <2>;

    cpus {
        #address-cells = <1>;
        #size-cells = <0>;
        cpu@0 {
            compatible = "arm,cortex-a53";
            reg = <0>;
        };
    };

    memory@40000000 {
        device_type = "memory";
        reg = <0x0 0x40000000 0x0 0x10000000>; /* 256 MiB */
    };
```

```
    serial@9000000 {
        compatible = "arm,pl011", "arm,primecell";
        reg = <0x0 0x09000000 0x0 0x1000>;
        interrupt-parent = <&intc>;
        interrupts = <0 1 4>;
    };

    intc: interrupt-controller@8000000 {
        compatible = "arm,gic-400";
        reg = <0x0 0x08000000 0x0 0x10000>,
              <0x0 0x08010000 0x0 0x10000>;
        interrupt-controller;
        #interrupt-cells = <3>;
    };
};
```

A `#address-cells` és `#size-cells` property-k határozzák meg, hogy a `reg` tulajdonságban hány cella írja le a címet és a méretet. A fenti példában a gyökér 2 címet és 2 méretcellát használ, ezért a `memory` node `reg` értéke négy 32 bites cellából áll: kezdőcím felső, kezdőcím alsó, méret felső, méret alsó. A `serial` node `reg` értéke két cellája a kezdőcímet, a harmadik a méretet adja meg, mert a szülőben `#address-cells = <2>` és `#size-cells = <2>`, de a `serial` maga csak 0x1000 méretet igényel, így a negyedik cella 0.

A `compatible` string-tömb sorrendje fontos: a legspecifikusabbtól a legáltalánosabbig halad. A kernel az első egyező meghajtót választja. A fenti `serial@9000000` node esetén először az `arm,pl011` illesztést próbálja, ha nincs, akkor az `arm,primecell` általánost. Az `intc` node az `interrupt-controller` tulajdonsággal jelzi, hogy megszakításvezérlő, a `#interrupt-cells` pedig a megszakításleíró cellák számát.

5.3.8 Címzés és megszakítások

A device tree a megszakítások leírására az `interrupt-parent` és `interrupts` property-ket használja. Az `interrupt-parent` egy phandle (egy másik node hivatkozása), amely a megszakításvezérlőt azonosítja. A fenti példában a `serial` node az `intc` node-ra hivatkozik. Az `interrupts` property cellái a megszakításvezérlő `#interrupt-cells` értékének megfelelően értelmezendők. Itt a `#interrupt-cells = <3>`, ezért a `interrupts = <0 1 4>` három cellát jelent: például megszakítástípus, megszakításszám, trigger típusa. A pontos jelentés az adott megszakításvezérlő dokumentációjától függ.

A device tree kezeli a címfordítási hierarchiát is a `ranges` property segítségével. Amikor egy busz külön címteret használ, a gyermek node-ban megadott címeket a szülő `ranges` property-je fordítja le a szülő címterére. Ez teszi lehetővé, hogy a PCI vagy más elágazó buszok eszközeit egységesen lehessen leírni.

5.3.9 Overlay és hotplug

A beágyazott rendszerekben a hotplug ritkább, mint a PC-kben, de a device tree támogatja a futás közbeni módosítást overlay-ek segítségével. Az overlay egy olyan DT-töredék, amely egy meglévő alap DT-hez adható hozzá, és futás közben a kernel alkalmazza. Az overlay tartalmazza a megváltoztatni kívánt node-okat és azok új property-jeit, valamint hivatkozhat a meglévő node-okra. Az overlay alkalmazása után a kernel értesíti az eszközmodellt, amely új platformeszközöket hozhat létre.

Például egy FPGA-alapú rendszerben a bitfolyam betöltése után egy új periféria jelenik meg. Az overlay segítségével a rendszer felveheti az új eszközt anélkül, hogy újraindítaná a rendszert. Az overlay-ök a zephyr, a Linux és más beágyazott rendszerekben is elérhetők.

5.3.10 Összehasonlítás: ACPI vagy device tree?

A két hardverleírási modell nem egymást helyettesíti, hanem eltérő platformfilozófiákat tükröz. Az alábbi táblázat a főbb szempontokat foglalja össze.

Szempont	ACPI	Device tree
Elsődleges platform	PC, szerver, laptop	Beágyazott, mobil, ARM
Leíró nyelv	ASL/AML (bájtkód)	DTS/DTB (adatszerkezet)
Futtatható kód	Igen (AML, korlátozva)	Nem
Hotplug	Natív GPE/Notify	Overlay-ekkel
Energiaállapotok	S0–S5, D0–D3, C0–C3, P-state	Korlátozott, gyakran a meghajtók kezelik
Firmware függőség	Erős, a gyártó biztosítja	Gyenge, a rendszertervező írja le

Az ACPI előnye, hogy a firmware gyártója egységes felületet ad, amelyre az operációs rendszer támaszkodhat, és a futtatható AML révén komplex műveleteket is végezhet. Hátránya a bonyolultság és a gyártói implementációk minőségének szórása. A device tree egyszerű, statikus adatmodell, amelyet a rendszertervező teljes mértékben ellenőriz, de nem nyújt beépített futásidejű szolgáltatásokat. A modern ARM szerverek egy része már ACPI-t használ, míg a PC-knél esetenként lehet találkozni DT-vel a tesztelésben, de az általános irány a platformhoz illeszkedő modell.

5.3.11 Energiaállapotok

Az energiaállapotok leírása és kezelése az ACPI egyik fő erőssége. Az ACPI a rendszerállapotokat (S-state), a processzorállapotokat (C-state), az eszközállapotokat (D-state) és a teljesítményállapotokat (P-state) különbözteti meg.

5.3.12 S-state: rendszeralvási állapotok

- **S0:** normál működés.
- **S1–S3:** egyre mélyebb alvási szintek; az S3 (Suspend to RAM) során a memória tartalma megmarad, a legtöbb eszköz kikapcsol.
- **S4:** hibernáció (Suspend to Disk); a memória tartalma lemezre íródik.
- **S5:** soft off; a rendszer kikapcsolt, de a firmware felébredhet.

Az operációs rendszer az ACPI-n keresztül kéri az állapotváltást, a firmware pedig elvégzi a szükséges regiszterírásokat. A felébredést a firmware kezdeményezi, és az operációs rendszer az ACPI-vel visszatér az S0 állapotba.

5.3.13 D-state: eszközállapotok

Az eszközök állapotait D0-tól D3-ig terjedő skálán írják le. A D0 a teljesen bekapcsolt, a D3 pedig a kikapcsolt állapot. A D1 és D2 közbülső, energiatakarékos üzemmódok. Az operációs rendszer az ACPI `_PS0`–`_PS3` metódusaival kapcsolja az eszközöket a megfelelő állapotba. A hotplug során is fontos az eszköz állapotának ellenőrzése a `_STA` metódussal.

5.3.14 C-state: processzoralvási állapotok

A CPU alvási állapotai a C0-tól (normál működés) a mélyebb C6, C7 felé haladnak. Az ACPI segítségével az operációs rendszer lekérdezheti az elérhető C-state-eket, és a megszakítási terhelés alapján választhat. A mélyebb C-state-ek alacsonyabb fogyasztást, de nagyobb felébredési késleltetést jelentenek.

5.3.15 P-state: teljesítményállapotok

A P-state-ek a processzor feszültség- és frekvenciaskálázását írják le. Az ACPI `_PSS` metódusa adja meg a támogatott teljesítményállapotokat, az operációs rendszer pedig a terhelés függvényében választ. A modern rendszerek gyakran hardveres vezérlésre (Intel SpeedStep, AMD PowerNow) támaszkodnak, de az ACPI továbbra is az interfész.

5.3.16 Gyakori hibák és megoldások

A hardverleírás bonyolultsága miatt számos tipikus hiba fordulhat elő. Az alábbiakban a leggyakoribbakat és a helyes javítási stratégiákat mutatjuk be.

5.3.17 Hibás vagy hiányzó ACPI-tábla

Hiba: A kernel nem találja az RSDP-t, vagy az XSDT egy érvénytelen címet tartalmaz. Ennek oka lehet a firmware-hiba, a memória tartalmi sérülése vagy egy rosszul leképezett virtuális gép.

Megoldás: Ellenőrizzük a firmware verzióját és a virtplatform beállításait. Az `acpidump` és `fwts` eszközök kimutatják a táblák sérüléseit. A gyakorlatban a helyes fix a firmware frissítése vagy az emulált hardver konfigurációjának javítása. A kernel naplója (`dmesg`) megmutatja az ACPI-vel kapcsolatos hibákat.

5.3.18 Hibás AML bájtkód

Hiba: A DSDT egy `_CRS` metódusban érvénytelen erőforrás-leírást ad, amitől a kernel ACPI-értelmezője leáll, vagy az eszköz nem kap erőforrást. A gyári DSDT-k gyakran tartalmaznak a specifikációtól eltérő kódot.

Megoldás: A DSDT-t le kell dumpolni és szét kell szedni ASL-re, majd a hibás részt javítani és az ACPI-tábla helyettesítésével újratölteni. Emulált környezetben ez biztonságosan gyakorolható. A `iasl` fordítási hibaelre is figyelmeztet. Az operációs rendszer rendelkezhet olyan beépített workaround-okkal, amelyek a gyakori gyártói hibákat kezelik.

5.3.19 Helytelen `compatible` string a device tree-ban

Hiba: A node `compatible` értéke nem egyezik egyetlen bejegyzett meghajtó `compatible` listájával, így a kernel nem társít meghajtót, és az eszköz nem működik. A hiba gyakran elírásból vagy rosszul dokumentált stringből adódik.

Megoldás: Ellenőrizzük a meghajtó forráskódjában a `compatible` táblát, és a DTS-ben pontosan azt a stringet használjuk. A `dt-validate` eszköz a séma alapján ellenőrzi a property-ket és a `compatible` értékeket.

5.3.20 Hiányzó `interrupt-parent` vagy rossz `interrupts` cellaszám

Hiba: A DT-ben a periféria node nem hivatkozik a megszakításvezérlőre, vagy az `interrupts` celláinak száma nem egyezik a `#interrupt-cells` értékével. Ilyenkor a kernel a megszakítást nem tudja leképezni, és az eszköz nem kap megszakítást.

Megoldás: A DT compiler (`dtc`) figyelmeztet a hiányzó hivatkozásokra, de a séma szerinti validálás (`dt-validate`) erősebb ellenőrzést ad. A javítás a megfelelő `interrupt-parent = <&intc>` beillesztése és a cellaszám ellenőrzése.

5.3.21 Elmaradt hotplug értesítés

Hiba: Egy eszköz hotplug eseménye nem érkezik meg, mert a GPE nincs bekötve, vagy az AML metódus nem küld Notify-t. A rendszer nem veszi észre az új eszközt, a meghajtó nem töltődik be.

Megoldás: A GPE-kiosztást és a DSDT `_Lxx` metódusait ellenőrizni kell. Az `acpidump` és az `ec_access` jellegű eszközök segítik a diagnózist. Emulált környezetben a hotplug események szimulálhatók, és a kernel `dmesg` kimenetében nyomon követhető az ACPI Notify feldolgozása.

A hibamegoldás során mindig izolált, eldobható laborban dolgozzunk, és őrizzük meg a kiindulási állapotot. Az ACPI-táblák cseréje vagy a DT módosítása után reprodukálható tesztet futtassunk, és rögzítsük a várt és a mért eredményt.

5.3.22 Labor: hardverleltár ACPI-ból és device tree-ból

E labor célja, hogy a fent tanult modelleket egy emulált környezetben alkalmazzuk: kinyerjük és elemezzük egy QEMU virtuális gép ACPI-tábláit, valamint egy ARM virtuális gép device tree-ját, és leltárt készítünk a hardvererőforrásokról. A labor minden lépése eldobható virtuális gépen fut; a hoszt valódi lemezét, bootloaderét vagy firmware-jét nem módosítjuk.

5.3.23 Kiindulási állapot

A laborhoz QEMU 8.2-es vagy újabb verziót, valamint `acpica-tools` (acpidump, iasl) és `device-tree-compiler` (dtc) csomagokat használunk. A pontos verziókat a reprodukciós jegyzetben rögzíteni kell. Az alábbi parancsok illusztrációk, nem futtatandók közvetlenül éles rendszeren.

5.3.24 1. rész: ACPI-hardverleltár x86_64 virtuális gépen

Indítsunk egy QEMU Q35 gépet, és másoljunk be egy minimal Linux initramfs-et vagy egy előre elkészített, csak olvasható rendszerképet. A parancs:

```
qemu-system-x86_64 -machine q35 -accel tcg -smp 2 -m 1G \
  -drive if=none,id=root,file=rootfs.qcow2,format=qcow2 \
  -device virtio-blk-pci,drive=root \
  -nographic -no-reboot \
  -kernel /srv/vmlinuz -initrd /srv/initramfs.img \
  -append "console=ttyS0 root=/dev/vda ro"
```

A rootfs, a kernel és az initramfs a labor előre elkészített, verziózott összetevői. A QEMU verziója, a kernel verziója és a Q35 gépmodell a reprodukciós csomag része.

A rendszerben futtassuk az `acpidump` parancsot, amely az összes ACPI-táblát kimenti:

```
acpidump -o acpi.bin
acpixtract -a acpi.bin
iasl -d *.dat
```

Az így kapott `.dsl` fájlok az ASL források. Készítsünk leltárt a DSDT-ben található eszközökről, például az `_HID` és `_ADR` objektumok alapján. Ellenőrizzük a MADT-tábla tartalmát is: hány CPU-t és milyen APIC-eket ír le.

5.3.25 2. rész: Device tree-hardverleltár ARM virtuális gépen

Indítsunk egy QEMU ARM `virt` gépet, és kérjük a DT kimentését a `dumpdtb` opcióval:

```
qemu-system-aarch64 -machine virt -cpu cortex-a53 -smp 1 -m 512 \
  -machine dumpdtb=virt.dtb -nographic -no-reboot
```

A `virt.dtb` bináris device tree-t a `dtc` segítségével alakítsuk ember által olvasható DTS-re:

```
dtc -I dtb -O dts virt.dtb -o virt.dts
```

A DTS-ben keressük meg a gyökér node alatti memória, CPU, soros port, megszakításvezérlő és egyéb eszközöket. Jegyezzük fel a node-ok számát, a `compatible` stringeket, a `reg` tartományokat és a megszakításleírókat.

5.3.26 Rögzített konfiguráció és mérési napló

A labor reprodukálhatóságához a következőket rögzíteni kell:

- QEMU verzió, kernel verzió, initramfs és rootfs azonosító (például sha256),
- a használt QEMU parancssor paraméterei,
- az `acpidump` által kiírt táblák listája és méretük (byte-ban),
- a DTB mérete és a DTS sorainak száma,
- a leltárban talált eszközök száma és típusai,
- az `iasl` és `dtc` futtatási naplója (különösen a hibák vagy figyelmeztetések).

Példa egy mérési naplóra (a számok az illusztráció kedvéért helyőrzők, és nem a tényleges kimenetet tükrözik):

Mért paraméter	Mért érték	Várt érték (dokumentáció)
ACPI táblák száma	9	9
DSDT mérete	24 KiB	körülbelül 20–30 KiB
DT node-ok száma	28	28
DT `compatible` string-ek száma	15	15
Hibák `iasl -d` során	0	0
Figyelmeztetések `dtc` során	1 (cell size mismatch)	0 vagy 1

A mért és a várt értékeket mindig külön oszlopban tüntessük fel. A várt érték a QEMU forrásdokumentációjából vagy a kernel dokumentációjából származik, és a labor előtt rögzítjük. Ha eltérés mutatkozik, azt meg kell vizsgálni; a reprodukciós jegyzetben fel kell tüntetni az eltérés okát.

A labor végén a virtuális gép leállítása után töröljük a munkakönyvtárat, és győződjünk meg arról, hogy a hoszt rendszer nem módosult.

5.3.27 Önellenőrző kérdés

Magyarázza el, milyen lépésekben jut el az operációs rendszer az RSDP-től egy PCI-eszköz ACPI-leírásáig, beleértve az XSDT, a DSDT és a `_CRS` metódus szerepét. Hasonlítsa össze ezt a folyamatot egy device tree-alapú rendszerrel, ahol a bootloader a DTB-t adja át. Melyik modellben van szükség futtatható bájtkódra, és miért?

Válaszában térjen ki az ACPI namespace bejárására, a DSDT és a MADT különbségére, valamint a DT `compatible` illesztésére. Indokolja meg, miért előnyös az AML használata a hotplug-értesítéseknél, és hogyan oldható meg a hotplug device tree overlay-ekkel.

5.3.28 Összefoglalás

A firmware az operációs rendszer számára nem csupán bootszolgáltatásokat nyújt, hanem a hardver konfigurációjának leírását is. Az ACPI és a device tree két eltérő, de egyaránt elterjedt modell: az ACPI a PC-k világában, a DT a beágyazott és ARM-rendszerekben. Az ACPI táblázatok és AML metódusok segítségével a kernel felfedezi az eszközöket, kezeli az energiaállapotokat, és fogadja a hotplug-eseményeket. A device tree statikus, adatközpontú leírást ad, amely overlay-ekkel kiterjeszthető. A helyes működéshez mindkét mechanizmus gondos validálást igényel; a tipikus hibák többsége a leíró adatok sérülésére, a hiányzó hivatkozásokra vagy a rossz illesztési stringekre vezethető vissza. A fejezetben bemutatott labor segítségével a hardverleltár készítése reprodukálható módon gyakorolható, és megalapozza a következő egységben tárgyalt megszakításkezelést, időzítőket és eszköz-felfedezést.

5.4 Interruptok, időzítők és eszközök felfedezése

Az operációs rendszer kétféle módon értesülhet a hardvereseményekről. A pollozás során a CPU periodikusan lekérdezi az eszközök állapotregisztereit; ez egyszerű, de pazarolja a processzoridőt, és növeli a késleltetést. A megszakítás (interrupt) ezzel szemben aszinkron: a hardver elektromos jellel félbeszakítja a futó programot, és a CPU egy előre rögzített belépési pontra ugrik. Az operációs rendszer innen hívja meg az adott eszközhöz tartozó kezelőrutint. A megszakítás tehát a hardver és a kernel közötti aszinkron kommunikáció elsődleges mechanizmusa.

Az aszinkronitás lényege, hogy az esemény bekövetkezése nem illeszkedik a futó programszál utasításfolyamába. A CPU a megszakítás érkezésekor az aktuális utasítás határán félreteszi a futó kontextust, és átvált a kernel megszakításkezelőjére. Ezért a kezelőnek konzervatívnak kell lennie: nem támaszkodhat arra, hogy egy adott felhasználói folyamat memóriájában fut, és nem aludhat el.

Az időzítés alapjai ugyanerre az aszinkron modellre épülnek. Az időzítő hardver (timer) programozható időközönként vagy egyszeri lejáratkor megszakítást generál. A kernel ezt a jelet használja az idő múlásának követésére, a folyamatok ütemezésére, a futásidő elszámolására és számos késleltetett művelet ütemezésére. A periodikus megszakítást szokás ticknek nevezni; a tick a rendszer szívverése. A tick nélkül az operációs rendszer nem tudná eldönteni, mikor kell lecserélnie a futó folyamatot, és nem tudna pontos időalapot biztosítani a felhasználói programoknak.

A megszakítások és az időzítők közös tulajdonsága, hogy a kernel futását váratlanul, a normál vezérlési folyamattól elkülönülő kontextusban szakítják meg. Ebből következik a fejezet egyik központi tanulsága: a megszakításkezelő kód minősége és rövidsége közvetlenül befolyásolja a rendszer válaszidejét, determinizmusát és stabilitását.

5.4.1 IRQ, vektor és prioritás

A hardveres megszakítások általában egy megszakításvezérlőn (interrupt controller) keresztül jutnak el a CPU-hoz. A vezérlő bemeneteire csatlakozó vonalakat IRQ-nak (Interrupt Request) nevezzük. Egy IRQ lehet él- vagy szintvezérelt. A szintvezérelt jel addig aktív, amíg az eszköz nem szolgálta ki az okot; az élvezérelt jel csak rövid impulzust ad. Az operációs rendszernek mindkét esetben pontosan egyszer kell kezelnie az eseményt, és a kezelés végén vissza kell igazolnia a megszakításvezérlő felé (EOI – end of interrupt).

A CPU azonban nem az IRQ-számot látja közvetlenül, hanem egy vektort. A vektor egy egész szám, amely az interrupt forrását azonosítja; a CPU a vektorszámot indexként használja a megszakítási vektortáblában. Például x86 rendszerben az IDT (Interrupt Descriptor Table) tartalmazza a kezelőrutinok belépési pontjait, míg ARM Cortex-A magokon a vektortábla néhány rögzített címen helyezkedik el, és az IRQ bejegyzésen keresztül a megszakításvezérlőtől kérdezi le a pontos forrást. A vektor tehát két szinten jelenik meg: a CPU felé egy kitüntetett belépési pont, a kernel felé pedig egy forrásazonosító.

A prioritás azt szabályozza, hogy egy megszakítás félbeszakíthatja-e egy másik megszakítás kezelését. A legtöbb modern megszakításvezérlő – például az ARM GIC vagy az x86 APIC – prioritási szinteket támogat. Ha egy magasabb prioritású interrupt érkezik, miközben a CPU egy alacsonyabb prioritású kezelőt futtat, a vezérlő újra jelezhet a CPU-nak; ezt hívják beágyazott (nested) megszakításnak. A kernel beállíthat egy maszkot is, amellyel bizonyos szinteket ideiglenesen letilt. A maszkolás és a prioritás együttesen biztosítja, hogy a kritikus szakaszok ne szakadjanak félbe, illetve hogy a legfontosabb események előnyt élvezzenek.

Gyakorlati példa: egy hálózati kártya csomagérkezési megszakítása alacsonyabb prioritást kaphat, mint egy precíziós időzítő vagy egy rendszerleállást jelző vonal. A kernel az időzítő megszakítást akkor is kiszolgálja, ha éppen hálózati csomagot dolgoz fel, mert az időzítő prioritása magasabb. A prioritási sorrendet azonban gondosan kell megválasztani: ha a hálózati kezelő túl alacsony prioritású, akkor nagy forgalomnál a csomagok elveszhetnek, ha viszont túl magas, akkor késleltetheti a kritikus időzítőt.

A vektor és az IRQ hozzárendelése gyakran nem rögzített. A modern rendszerekben az eszközök dinamikusan kapnak interruptot; ezt a folyamatot az eszközfelderítés során a firmware és a kernel együtt végzi. A kernel egy táblázatban tartja nyilván, hogy melyik IRQ-hoz vagy vektorhoz

melyik meghajtó regisztrálta magát. A regisztrációhoz tartozik egy kezelőfüggvény, egy paraméter (például eszközstruktúra mutatója) és opcionálisan egy megosztási flag. Ha több eszköz osztozik egy IRQ-n, a kernel mindegyik kezelőt meghívja, és a kezelő visszatérési értékével jelzi, hogy valóban az ő eszköze generálta-e a megszakítást.

5.4.2 Időzítő, tick és eszközfelderítés

Az időzítő hardver a kernel időgazdálkodásának alapja. A korai PC-ken a 8253/8254 programozható időzítő (PIT) szolgáltatta a periodikus megszakítást; a modern x86 rendszerekben ezt a HPET (High Precision Event Timer) vagy a CPU lokális APIC időzítője váltotta fel. Az ARM platformokon a Generic Timer architektúra biztosít összehasonlító regisztereket és megszakítást a CPU magonként. A konkrét hardverek típusa és elnevezése változhat, ezért a kernelben egy absztrakciós réteg (clockevent/clock source) választja el az időzítő hardvert az időkezelési logikától.

A tick a periodikus időzítő megszakítás neve. Gyakori tick-frekvenciák: 100 Hz, 250 Hz, 1000 Hz. A magasabb frekvencia pontosabb ütemezést és időzítést tesz lehetővé, de több CPU-időt fogyaszt, és több energiát használ. A kernel a tick megszakításban frissíti a rendszer időszámlálóját (például a jiffies nevű globális számlálót), futtatja az ütemező könyvelését, és kezeli a lejárt időzítőket. A tick tehát az időalap és az ütemező közös forrása.

A tickless (dinamikus tick) mód ezt a periodikus terhet csökkenti. Ha a CPU üresjáratban van, a kernel leállítja a periodikus tick-et, és csak akkor ébreszti fel a CPU-t, amikor a következő időzítő esedékes. Ezzel energiát takarít meg, és csökkenti a felesleges megszakításokat. A tickless üzemmód azonban bonyolultabb könyvelést igényel, mert az időszámláló frissítése nem egyenletes ütemben történik.

Az eszközfelderítés az a folyamat, amelynek során a kernel megtudja, milyen hardverek vannak a rendszerben, és milyen erőforrásokat – így interruptvonalakat, időzítőket, memóriatartományokat – használnak. A boot során a firmware vagy a bootloader átad egy hardverleírást: x86/ACPI rendszeren az ACPI-táblák, beágyazott rendszeren gyakran egy Device Tree blob (DTB), más platformokon egyéb struktúrák. Ezt a Firmware, ACPI, device tree és hardverleírás alfejezet tárgyalta; itt annyit emelünk ki, hogy a kernel ebből a leírásból építi fel az eszközmodelljét, és ehhez köti az interruptkezelőket.

Az eszközfelderítés során a kernel minden eszközhöz hozzárendeli a megfelelő IRQ-t vagy MSI-vektort. A PCI eszközök hagyományosan négy INTx vonalat használhatnak, amelyeket a rendszer a megszakításvezérlő bemeneteire képez le; az MSI/MSI-X mechanizmus lehetővé teszi, hogy az eszköz közvetlenül memória címre írjon, és ezzel megszakítást generáljon, elkerülve a megosztott vonalakat. Az MSI különösen fontos a nagy sávszélességű eszközöknél, mert csökkenti a késleltetést és a megszakításütközéseket.

A felderítés hibái közé tartozik, ha a kernel rosszul értelmezi a hardverleírást, vagy ha egy eszköz nem jelenti be pontosan a szükséges interruptot. Ilyenkor a megszakítás nem érkezik meg, vagy szüntelenül ismétlődik (interrupt storm). A helyes megoldás a hardverleírás verifikálása és a megszakításkezelő védelme: a kezelőnek ellenőriznie kell az okot, és ha nem az eszköz generálta, a lehető leggyorsabban vissza kell térnie.

5.4.3 Egy egyszerű interrupt- és időzítőkezelő vázlata

Az alábbi példa egy fiktív, freestanding C környezetben írt minimális kernelrészletet mutat be. A kód szemléltetés, nem végrehajtandó utasítás az éles gépen; a cél a fogalmak összekapcsolása.

Először definiáljuk az interrupt keret (trap frame) struktúráját, amely a CPU regisztereit menti. Ez hardverfüggő, ezért itt csak pszeudokódot adunk.

```
/* pszeudokód: interrupt keret, valós ABI szerint töltendő ki */
struct interrupt_frame {
    uintptr_t ip;      /* megszakított utasítás címe */
    uintptr_t sp;      /* veremmutató */
    uint32_t flags;    /* CPU állapot */
    /* ... további regiszterek platformfüggőek */
};

/* Interrupt kezelő függvény típusa */
typedef void (*irq_handler_t)(struct interrupt_frame *frame, void *ctx);

/* IRQ tábla: max 256 bejegyzés */
static irq_handler_t irq_table[256];
static void *irq_ctx[256];
```

A regisztráció lehetővé teszi, hogy egy eszközmeghajtó beregisztrálja a kezelőjét:

```
/* pszeudokód: interrupt regisztráció */
int register_irq(unsigned int irq, irq_handler_t handler, void *ctx) {
    if (irq >= 256 || handler == NULL) {
        return -1; /* hibakód, errno szerint kezelendő */
    }
    if (irq_table[irq] != NULL) {
        return -2; /* már foglalt, hacsak nem megosztott */
    }
    irq_table[irq] = handler;
    irq_ctx[irq] = ctx;
    return 0;
}
```

Az alacsony szintű belépési pont assemblyben menti a regisztereket, majd meghívja a C nyelvű diszpécsert:

```
/* pszeudokód: assembly belépési pont, x86 szintaxis illusztráció */
irq_entry:
    pusha                    /* általános regiszterek mentése */
    push %esp                /* interrupt_frame mutató */
    call irq_dispatch
    add  $4, %esp
    popa
    iret
```

A diszpécser lekérdezi a vektort, kikeresi a kezelőt, és meghívja. A megszakításvezérlő EOI jelzését a kezelő után kell elküldeni, hogy ne maradjon aktív a vonal:

```
/* pszeudokód: diszpécser */
void irq_dispatch(struct interrupt_frame *frame) {
    unsigned int vector = get_current_vector(); /* hardverfüggő */
    irq_handler_t handler = irq_table[vector];
    if (handler != NULL) {
        handler(frame, irq_ctx[vector]);
    }
    send_eoi(vector); /* megszakításvezérlő visszaigazolása */
}
```

A tick kezelő például így nézhet ki:

```c
/* pszeudokód: tick kezelő */
static volatile uint64_t jiffies = 0; /* időszámláló */

void tick_handler(struct interrupt_frame *frame, void *ctx) {
    (void) frame;
    (void) ctx;
    jiffies++;
    scheduler_account_tick(); /* futó folyamat idejének elszámolása */
    run_expired_timers();     /* lejárt időzítők futtatása */
}
```

Fontos: ez a tick kezelő nagyon rövid. Ha a `scheduler_account_tick` vagy a `run_expired_timers` bonyolult, akkor azokat nem szabad közvetlenül a hardveres megszakítás kontextusában futtatni; helyette a kernel egy késleltetett feldolgozási mechanizmust (softirq, tasklet, workqueue) ütemez. A fenti kód csak a fogalmi szerkezetet mutatja, és nem veszi figyelembe a valós kernel szinkronizációs követelményeit.

5.4.4 Tipikus hibák és helyes megoldások

A megszakítás- és időzítőkezelésben számos csapda vár a fejlesztőre. Az alábbiakban a leggyakoribb hibákat és a hozzájuk tartozó helyes megoldásokat tekintjük át.

Túl hosszú megszakításkezelő. A hardveres megszakítás kontextusban futó kódnak rövidnek kell lennie. Ha a kezelő hosszadalmas műveleteket végez – például nagy memóriablokkokat másol, lassú eszközt polloz, vagy bonyolult adatszerkezeteket módosít –, akkor blokkolja az összes többi megszakítást, növeli a késleltetést, és akár elveszett megszakításokhoz vezethet. A helyes megoldás a feldolgozás kettéválasztása: a megszakításkezelő csak a legszükségesebb állapotot menti, majd egy késleltetett mechanizmust ütemez. A Linux ezt a top half/bottom half mintával oldja meg: a top half a hardveres kezelő, a bottom half softirq, tasklet vagy workqueue. A valós idejű rendszerekben ez különösen fontos, mert a hosszú kezelő közvetlenül rontja a legrosszabb eset végrehajtási időt.

Alvás a megszakításkezelőben. A megszakítás kontextus nem tartozik egyetlen folyamathoz sem, ezért nem lehet benne aludni, vagy olyan függvényt hívni, amely alhat. Ha a kezelő mégis alvást okozó műveletet végez, a kernel állapota inkonzisztenssé válhat, és a rendszer lefagyhat. A helyes megoldás a tiltás: a kernelben a megszakításkezelő futása alatt a folyamat kontextushoz kötött erőforrások – például felhasználói memória elérése, futó folyamat adatai – nem elérhetők. A fejlesztőnek ezt fordítási időben vagy futás közben ellenőriznie kell.

Maszkolatlan vagy hibásan maszkolt interrupt. Ha a megszakításvezérlő maszkját nem megfelelően állítják be, akkor egy vonal folyamatosan aktív maradhat, és a kernel interrupt stormba kerül. A helyes megoldás a megszakításkezelő végén mindig a megfelelő EOI jelzés, illetve a vonal szintvezérelt esetben az eszköz okának törlése. A regisztráció során a kernelnek ellenőriznie kell, hogy az IRQ nincs-e már használatban, vagy ha megosztott, akkor minden kezelő fel legyen készítve a nem saját eseményre.

Elveszett megszakítás (lost interrupt). Ez akkor fordul elő, ha a CPU nem kapja meg a megszakítást, pedig az eszköz jelezte. Ennek oka lehet a megszakítás tiltása a kritikus szakaszban, a rossz prioritáskezelés, vagy a hardver hibája. A helyes megoldás a kritikus szakaszok minimalizálása, a megszakításvezérlő állapotának időszakos ellenőrzése, és az eszközmeghajtókban a pollozásra való átmenet, ha a megszakítás nem érkezik meg. A modern rendszerekben a kernel nyomon követi a megszakítások számát, és ha egy eszköz nem generál várt megszakítást, időkorlát után pollozásra vált.

Időzítő pontossági hibák. A tick alapú időmérés pontossága korlátozott. A tick frekvencia határozza meg az időzítés felbontását; ha a rendszer 100 Hz-es ticket használ, akkor egy időzítő legfeljebb 10 ms-os késleltetéssel futhat le. A tick késleltetése a megszakítás maszkolása miatt tovább nőhet. A helyes megoldás a nagy felbontású időzítő (hrtimer) használata, amely nem a tick-hez kötött, hanem a hardveres időzítő közvetlen lejáratához. A méréseknél a jittert és a legrosszabb esetet külön kell dokumentálni.

Prioritás inverzió. Bár a prioritás inverzió elsősorban a valós idejű ütemezés problémája, a megszakításoknál is előfordulhat: egy alacsony prioritású megszakítás kezelője olyan erőforrást tart fogva, amelyre egy magasabb prioritású megszakítás vár. Ez a fejezet csak a fogalmat vezeti be; a részletes tárgyalás a Determinizmus, latency és jitter–WCET, megszakítási késleltetés és erőforrás-elemzés alfejezetekben található. A megelőzés egyik eszköze a prioritásöröklés, amelyet a valós idejű kernelekben alkalmaznak.

Hibás eszközfelderítés. Ha a kernel rosszul olvassa be az ACPI- vagy Device Tree-leírást, akkor az eszközök nem kapnak megfelelő interruptot, vagy a meghajtó nem találja az erőforrásait. A helyes megoldás a hardverleírás validálása, a bootlog ellenőrzése, és a megszakítások számának monitorozása. Az emulált környezetben a QEMU `-machine` kapcsolóval rögzíthető a hardvermodell, így a leírás reprodukálhatóvá válik.

5.4.5 Labor: interrupt- és időzítőkezelési terv készítése

A labor célja, hogy az olvasó elkészítsen egy reprodukálható tervet egy minimális kernel interrupt- és időzítőkezelésére QEMU emulált környezetben. A labor nem igényel valódi hardvert, és a hoszt rendszerét nem módosítja. A tervnek tartalmaznia kell a konfigurációt, a build és futtatási parancsokat, a mérési naplót, és a mért és várt eredmények elkülönítését.

Kiinduló állapot. Feltételezzük, hogy rendelkezésre áll egy minimal freestanding kernelváz, amely a boot után egy egyszerű C függvényt hív, és rendelkezik egy alapvető soros konzollal. A kernel nem tartalmaz még interruptkezelést. A labor ezt a vázat egészíti ki.

Hardverkonfiguráció. QEMU `virt` gép, emulált ARM Cortex-A72 CPU, 128 MiB RAM, egy soros konzol (PL011), egy Generic Timer, és egy GICv3 megszakításvezérlő. A pontos verziók és kapcsolók a reprodukciós csomagban rögzítendők; a példa itt helyőrző. A QEMU verziójától függően a gép alapértelmezett megszakításvezérlője eltérhet; ezt a reprodukciós csomagban ellenőrizni kell. A hoszt operációs rendszer tetszőleges, a QEMU és a cross-toolchain verzióját a naplóban fel kell tüntetni.

Build parancs. A kernel forrása C és minimális assembly. A build például:

```
# Példa build parancs, helyőrző elérési utakkal
aarch64-none-elf-gcc -ffreestanding -O2 -Wall -Wextra \
  -Isrc -c src/kernel.c -o build/kernel.o
aarch64-none-elf-as src/vectors.S -o build/vectors.o
aarch64-none-elf-ld -T linker.ld build/vectors.o build/kernel.o -o build/kernel.elf
```

A linker script a kernel image-t 0x40000000 címen helyezi el, a verem a kernel image végétől nő. A build naplóba kerül a fordító verziója, a kapcsolók és a linkelés eredménye.

Futtatási parancs.

```
# Példa QEMU futtatás, helyőrző konfigurációval
qemu-system-aarch64 -machine virt -cpu cortex-a72 -m 128M \
  -kernel build/kernel.elf -nographic \
  -D qemu.log -d int 2>&1 | tee run.log
```

A `-d int` kapcsoló a QEMU interrupt nyomkövetését kapcsolja be, így a naplóban láthatóvá válnak a megszakítások. A naplót a laborban elemezni kell.

Elvégzendő feladatok.

1. Inicializálja a megszakításvektor-táblát. Töltse fel az IRQ bejegyzést egy assembly belépési ponttal, amely menti a regisztereket és meghívja a C diszpécsert.
2. Konfigurálja a GIC-t: engedélyezze a Generic Timer megszakítását, és állítson be egy prioritási szintet. A prioritás legyen alacsonyabb, mint a kritikus rendszerleállítás, de magasabb, mint a soros konzol.
3. Programozza a Generic Timer-t, hogy 10 ms-os periódussal megszakítást generáljon.
4. Implementálja a tick kezelőt, amely növeli a `jiffies` számlálót, és minden 100. ticknél kiír egy sort a soros konzolra.
5. Regisztrálja a kezelőt az IRQ táblában, és küldje el az EOI-t a kezelés végén.
6. Mérje meg a tick tényleges periódusát: a soros konzolon megjelenő sorok időbélyegét hasonlítsa össze az elvárt 1000 ms-os időközzel. A méréshez a hoszt és a vendég óráját is rögzíteni kell; a pontos módszert a napló tartalmazza.

Elvárt eredmény. A rendszer minden másodpercben egy sort ír ki, és a `jiffies` számláló 100-zal nő. A megszakítások száma a QEMU naplóban megközelítőleg 100 másodpercenként. A várt és mért értékeket külön oszlopban kell feltüntetni.

Mérési napló. A napló az alábbi táblázatot tartalmazza (a számok illusztrációk, nem mért értékek):

Mérés ideje (hoszt)	Vendég jiffies	Várt jiffies különbség	Mért különbség
0,0 s	0	–	–
1,0 s	100	100	99
2,0 s	200	100	101
…	…	…	…

A táblázatban a mért különbség a vendég `jiffies` értékének tényleges változása, a várt különbség a tick periódusból számított érték. Ha a mért különbség tartósan eltér a várttól, akkor a megszakítások elvesznek vagy késnek.

Hibaanalízis. Ha a mért periódus nagyobb, mint a várt, a lehetséges okok: a megszakítások maszkolása, a GIC prioritás hibás beállítása, vagy a soros konzol kiírása blokkol. A laborban ezeket az okokat szisztematikusan ki kell zárni. Ha a periódus kisebb, akkor valószínűleg duplikált megszakítások vannak, ami a hiányzó EOI-ra utal.

Önellenőrző kérdés. Mi történik, ha a tick kezelő a soros konzolra írást közvetlenül a hardveres megszakítás kontextusában végzi, és a soros port lassú? Válasz: a hosszú kiírás miatt a kezelő blokkolja a többi megszakítást, nő a késleltetés, és a tick pontossága romlik. A helyes megoldás a kiírást egy késleltetett mechanizmusba helyezni.

A labor visszaállítása: a QEMU kilépése után a `build` és a `*.log` fájlok törölhetők; a hoszt rendszerét nem módosítottuk. A reprodukciós csomag a forrást, a build parancsot, a linker scriptet és a mérési naplót tartalmazza.

5.4.6 Összefoglalás és továbbvezetés

Ez az alfejezet bemutatta az operációs rendszer aszinkron eseménykezelésének alapjait: a megszakítások mechanizmusát, az IRQ–vektor–prioritás fogalmait, az időzítők és a tick működését, valamint az eszközfelderítés szerepét. A labor egy reprodukálható tervet adott a kezdeti interrupt- és időzítőkezelésre, amely a későbbi fejezetekben tovább bővíthető.

A következő alfejezet (Privilegiszintek és a kernel/user tér átlépése) ezekre a fogalmakra épít: a megszakítás belépési pontja ugyanaz a mechanizmus, amely a rendszerhívásoknál a felhasználói térből a kernelbe való átlépést is szolgálja. A vektortábla, a privilegiszintek és a kontextusváltás ismerete nélkülözhetetlen a rendszerhívások tárgyalásához. A most bevezetett tick és időzítő fogalmak pedig az ütemező (Ütemezési célok és metrikák–Valós idejű ütemezés: RMS, EDF és deadline) alapját képezik.

5.4.7 Önellenőrző kérdések

1. Mi a különbség az él- és szintvezérelt megszakítás között, és miért fontos az EOI?
2. Hogyan befolyásolja a tick frekvencia az ütemező késleltetését és a CPU-terhelést?
3. Miért nem szabad aludni egy megszakításkezelőben?
4. Hogyan csökkenti az MSI/MSI-X a megosztott interruptvonalak problémáit?
5. A laborban miért kell a mért és a várt értéket külön oszlopban feltüntetni?

A kérdésekre a fejezet anyagából adható válasz, de a labor elvégzése nélkül a mérési kérdés nem értékelhető teljes mértékben.

6 Rendszerhívások és felhasználói interfész

6.1 Privilegiszintek és a kernel/user tér átlépése

A felhasználói program egyetlen gépi utasítása sem módosíthatja közvetlenül a lapregisztereket, a megszakításvezérlőt vagy a processzor futási szintjét. Ha ezt megtehetné, egyetlen rossz mutató vagy szándékos kódrészlet megkerülhetné a memóriavédelmet, leállíthatná a gépet, vagy más folyamatok adatait olvashatná. A védelmi gyűrűk és a kernel/user tér közötti átlépés éppen ezt a határt szabályozza: a processzor hardveresen kényszeríti ki, hogy bizonyos műveletek csak a kernel számára elérhetők, a felhasználói kód pedig csak ellenőrzött belépési pontokon keresztül kérheti ezek végrehajtását. Ez az alfejezet ezt a mechanizmust bontja ki a hardveres alapoktól a rendszerhívási út költségéig, majd egy emulált környezetben elvégezhető méréssel zárul.

6.1.1 Védelmi gyűrűk és privilegizált műveletek

A klasszikus x86 architektúra négy védelmi gyűrűt definiál: a 0. gyűrű a legmagasabb, a 3. a legalacsonyabb jogosultsági szint. A gyakorlati operációs rendszerek általában csak két szintet használnak: a 0. gyűrűt a kernel, a 3. gyűrűt a felhasználói programok számára. A köztes 1. és 2. gyűrűt az x86-on ritkán használják, mert a mai operációs rendszerek inkább lapozáson alapuló védelmet és virtualizációs kiterjesztéseket alkalmaznak. Az ARM architektúra az `EL0` (user) és `EL1` (kernel) kivételszinteket használja, a RISC-V pedig az `U-mode` és `S-mode` (supervisor) szinteket. A fogalmi modell mindenütt azonos: létezik egy magasabb jogosultságú kernelmód és egy alacsonyabb jogosultságú felhasználói mód.

A processzor a futó kód aktuális jogosultsági szintjét egy belső állapotban tárolja, és minden egyes utasítás végrehajtása előtt ellenőrzi, hogy az adott utasítás végrehajtható-e az aktuális szinten. A privilegizált utasítások – például a vezérlőregiszterek írása, a megszakítások globális engedélyezése vagy tiltása, a lapozás engedélyezése, a MMU konfiguráció módosítása – csak a magasabb szinten hajthatók végre. Ha egy felhasználói program ilyen utasítást próbál meg futtatni, a processzor kivételt generál, és a vezérlést a kernel megfelelő hibakezelőjének adja át. Ezt a mechanizmust hívjuk védett utasítások hardveres kikényszerítésének.

A memóriavédelem a gyűrűk másik alappillére. A lapozási táblák minden bejegyzése tartalmaz egy jogosultsági bitet, amely meghatározza, hogy az adott lap elérhető-e felhasználói módból. A kernel címterének lapjai általában csak kernelmódban érhetők el; a felhasználói oldalak mindkét módból, de írásvédett vagy nem végrehajtható attribútumokkal. Így a kernel adatai és kódja fizikailag is el van szigetelve a felhasználói programoktól, még akkor is, ha azok ismerik a címtartományokat. A gyűrűváltás önmagában nem elegendő; a lapozási táblák és a gyűrűszintek együttesen adják a teljes védelmet.

6.1.2 Rendszerhívás, trap és visszatérés

A felhasználói program akkor léphet be a kernelbe, ha:

1. rendszerhívást hajt végre (`syscall`),

2. kivételt vált ki (például osztás nullával, érvénytelen memóriacím),
3. megszakítást kap (időzítő, eszköz),
4. laphibát okoz (ez valójában a kivételek egy fajtája).

Ezek közös neve: trap vagy csapda. A trap lényege, hogy a processzor automatikusan átvált kernelmódba, elmenti a felhasználói környezet egy részét, és egy előre meghatározott belépési pontra ugrik a kernelben. A rendszerhívás ennek egy speciális esete: a felhasználói program szándékosan, egy erre fenntartott utasítással kéri a kernel szolgáltatását.

Az x86-64 architektúrán a `syscall` utasítás egy gyors belépési mechanizmust valósít meg. A processzor a `IA32_LSTAR` modell-specifikus regiszterben tárolt címre ugrik, átvált 0. gyűrűbe, és letiltja a megszakításokat. A felhasználói kód a rendszerhívás számát és az argumentumait regiszterekben adja át. A Linux x86-64 ABI szerint a rendszerhívás számát a `rax`, az argumentumokat sorrendben a `rdi`, `rsi`, `rdx`, `r10`, `r8`, `r9` regiszterek tartalmazzák. A kernel a visszatérési értéket a `rax`-ben adja vissza; a hibát a `-errno` értékkel jelzi, amit a libc a felhasználói térben `errno`-ra fordít.

Az ARM64-en a `svc` (supervisor call) utasítás tölti be ugyanezt a szerepet. A RISC-V-en az `ecall` utasítás. A mechanizmus közös: a felhasználói kód egy speciális utasítással kivált egy trap-et, a processzor átvált kernelmódba, és a kernel trapkezelője a rendszerhívási szám alapján kiválasztja a megfelelő függvényt.

A visszatérés a kernelből a felhasználói térbe általában egy pár utasítással történik: x86-64-en a `sysret` (vagy a régebbi `iretq`), ARM64-en az `eret`, RISC-V-en az `sret`. Ezek visszaállítják a felhasználói program utasításszámlálóját, veremmutatóját és jogosultsági szintjét. A kernelnek a visszatérés előtt gondoskodnia kell arról, hogy a felhasználói kontextus konzisztens legyen: a regiszterek, a verem, a futási állapot és a memóriavédelem mind helyes értékeket tartalmazzon.

6.1.3 A határ átlépésének útja

Egy rendszerhívás tipikus útja a következő lépésekből áll:

1. A libc függvény (például `write`) betölti a rendszerhívás számát a `rax`-be, az argumentumokat a megfelelő regiszterekbe, majd végrehajtja a `syscall` utasítást.
2. A processzor kernelmódba vált, elmenti a felhasználói `rsp`-t, `rip`-et és a `rflags`-et, betölti a kernel vermét, és a `IA32_LSTAR`-ban tárolt címre ugrik.
3. A kernel belépési pontja (assembly) elmenti a felhasználói regisztereket a kernel vermére, majd meghívja a C nyelvű rendszerhívás-kezelőt.
4. A C kezelő a `rax`-ben kapott rendszerhívási szám alapján kiválasztja a megfelelő kernelfüggvényt, ellenőrzi az argumentumokat, és végrehajtja a kért műveletet.
5. A kernelfüggvény visszatérési értéket ad; a C kezelő ezt a `rax`-be helyezi.
6. Az assembly kilépési pont visszaállítja a felhasználói regisztereket, és a `sysret` utasítással visszatér a felhasználói módba.
7. A libc ellenőrzi a visszatérési értéket, és ha negatív, beállítja az `errno`-t.

Az alábbi pszeudokód egy egyszerű `write` rendszerhívás felhasználói oldalát szemlélteti:

```
/* userland: write(fd, buf, count) - szemléltető pszeudokód */
ssize_t write(int fd, const void *buf, size_t count)
{
    long ret;
    asm volatile(
        "mov %[fd], %%rdi\n\t"
        "mov %[buf], %%rsi\n\t"
        "mov %[count], %%rdx\n\t"
```

```
        "mov $1, %%rax\n\t"       /* Linux x86-64 write syscall száma */
        "syscall\n\t"
        "mov %%rax, %[ret]\n\t"
        : [ret] "=r" (ret)
        : [fd] "r" ((long)fd),
          [buf] "r" ((const void *)buf),
          [count] "r" ((long)count)
        : "rax", "rdi", "rsi", "rdx", "rcx", "r11", "memory"
    );
    if (ret < 0) {
        errno = -ret;
        return -1;
    }
    return ret;
}
```

A példa csak a mechanizmust mutatja; a valódi libc a regiszterkiosztást és a hibakezelést architektúrafüggő assemblyben valósítja meg. A kernel oldalán a belépési pont a következő vázlatot követheti:

```
/* kernel entry: x86-64 syscall belépési pont - szemléltető vázlat */
syscall_entry:
    swapgs                  /* kernel GS bázis betöltése */
    mov %rsp, %gs:12        /* felhasználói rsp mentése */
    mov %gs:8, %rsp         /* kernel verem betöltése */
    pushq $0x2b             /* SS (user data selector) */
    pushq %gs:12            /* felhasználói rsp */
    pushq %r11              /* RFLAGS */
    pushq $0x33             /* CS (user code selector) */
    pushq %rcx              /* visszatérési RIP */
    pushq %rbp
    /* további regisztermentés... */
    call do_syscall         /* C kezelő hívása */
    /* visszatérési érték rax-ben */
    /* regiszterek visszaállítása... */
    swapgs
    sysretq
```

A C kezelő egy egyszerű diszpécser lehet:

```
/* kernel: rendszerhívás-diszpécser - szemléltető vázlat */
long do_syscall(long nr, long a0, long a1, long a2, long a3, long a4, long a5)
{
    switch (nr) {
    case SYS_write:
        return sys_write((int)a0, (const void *)a1, (size_t)a2);
    case SYS_getpid:
        return current_process->pid;
    default:
        return -ENOSYS;
    }
}
```

A `sys_write` függvény feladata, hogy ellenőrizze a `buf` mutató érvényességét, átmásolja a felhasználói adatot a kernel címterébe (például a `copy_from_user` segítségével), és továbbítsa a megfelelő eszközmeghajtónak. A mutató ellenőrzése kritikus: a kernel soha nem hivatkozhat közvetlenül egy felhasználói címre, mert az a felhasználói folyamat által tetszőlegesen módosítható, és a laphiba kernelmódban végzetes lehet.

6.1.4 A határ átlépésének költsége

A rendszerhívás nem egy egyszerű függvényhívás. A költség több összetevőből áll:

1. **Módváltás**: a processzor átvált a 3. gyűrűből a 0. gyűrűbe, ami a futási szint megváltoztatásával és a kernel verem betöltésével jár. Ez hardveres művelet, de a modern processzorokban optimalizált.
2. **Regisztermentés és -visszaállítás**: a kernelnek el kell mentenie a felhasználói regiszterek egy részét, hogy a rendszerhívás után helyre tudja állítani a program állapotát. Ez legalább néhány tucat regiszter írását és olvasását jelenti.
3. **A belépési pont és a diszpécser kódjának végrehajtása**: a kernel C kódjának belépése, a rendszerhívási szám ellenőrzése, a függvény kiválasztása és hívása.
4. **Cache- és TLB-hatások**: a kernel kódja és adatai más címterületen vannak, mint a felhasználói program. A belépéskor a processzor gyakran új TLB-bejegyzéseket és cache-sorokat tölt be, ami lassíthatja a végrehajtást. A `syscall` utasítás önmagában nem üríti a TLB-t, de a kernel lapjainak elérése új bejegyzéseket igényel.
5. **Spekulatív végrehajtási mellékhatások**: a modern processzorok spekulatívan hajtják végre az utasításokat, ami a biztonsági résekhez (Spectre, Meltdown) vezetett. A kernel belépésekor a CPU-nak meg kell akadályoznia, hogy a felhasználói kód spekulatív úton hozzáférjen a kernel adataihoz, ami további hardveres és szoftveres költségekkel jár. A Linux kernelben a `KPTI` (Kernel Page Table Isolation) például külön lapozási táblákat használ a kernel és a felhasználói tér számára, ami növeli a belépési költséget.
6. **Veremváltás**: a kernel a saját vermét használja, ami szintén cache- és TLB-műveletekkel jár.

A tényleges költség erősen függ a processzortól, a kernel konfigurációjától és a rendszerhívás típusától. Egy üres `getpid` rendszerhívás egy modern x86-64 CPU-n körülbelül 50–150 ns-ot vehet igénybe, míg egy bonyolult `read` vagy `write` műveletnél a tényleges I/O dominál. A régebbi architektúrákon, ahol a `int 0x80` utasítást használták, a belépés lényegesen lassabb volt (több száz ciklus), mert a szoftveres megszakítás általános trapkezelőn keresztül ment. A `syscall`/`sysenter` utasítások bevezetése jelentősen csökkentette ezt a költséget.

A méréshez fontos a módszertan rögzítése. A `rdtsc` utasítással ciklusszintű időbélyeget olvashatunk, de figyelembe kell venni az out-of-order végrehajtást és a CPU frekvenciaváltást. A `clock_gettime` rendszerhívás önmaga is rendszerhívás, ezért alkalmatlan a syscall-költség mérésére. A szokásos módszer egy szoros ciklus, amely sokszor meghívja a vizsgált rendszerhívást, és a teljes időt elosztja a hívások számával.

6.1.5 Tipikus hibák és helyes megoldások

A kernel/user határ átlépése számos buktatót rejt. A leggyakoribb hibák a következők:

1. **Közvetlen felhasználói mutató hivatkozás kernelmódban.** A kernel soha nem olvashat vagy írhat közvetlenül egy felhasználói címre, mert a felhasználói program a mutatót bármikor megváltoztathatja, a cím nem feltétlenül érvényes, és a laphiba kernelmódban a kernel összeomlását okozhatja. A helyes megoldás a `copy_from_user` és `copy_to_user` függvények használata, amelyek ellenőrzik a cím érvényességét, és biztonságosan másolják az adatokat.
2. **A rendszerhívási szám és az argumentumok rossz regiszterben való átadása.** Az ABI szigorúan meghatározza, hogy melyik regiszter mit tartalmaz. Ha a felhasználói kód rossz regisztert használ, a kernel rossz argumentumot kap, ami nem definiált viselkedéshez vezet. A libc ezt elrejti, de az assemblyben írt kódnál gondosan kell követni az ABI-t.

3. **A visszatérési érték és az `errno` hibás kezelése.** A kernel a hibákat negatív értékként adja vissza, a libc pedig ezt `errno`-ra fordítja. Ha a felhasználói program a nyers visszatérési értéket ellenőrzi, téves hibakezelést valósíthat meg. Például a `-1` visszatérési érték azt jelenti, hogy a hibakód az `errno`-ban van; a `-2` már egy konkrét hibát (`ENOENT`) jelenthet.
4. **TOCTOU (time-of-check-to-time-of-use) versenyhelyzet.** A kernel a felhasználói adat ellenőrzése és a felhasználói adat használata között a felhasználói program megváltoztathatja az adatot. Ha a kernel például egy fájlnevet ellenőriz, majd később újra beolvassa, a két olvasás között a felhasználói program módosíthatja a puffert. A megoldás az adat egyszeri, atomi másolása a kernel címterébe.
5. **A regiszterek nem megfelelő mentése és visszaállítása.** A kernel belépési pontjának pontosan kell kezelnie a regisztereket. Ha egy regisztert nem ment el, a felhasználói program állapota sérül. A C ABI szerint a kernel C függvényei csak a callee-saved regisztereket kötelesek megőrizni, ezért az assembly belépési pontnak minden olyan regisztert mentenie kell, amelyet a felhasználói program használhat.
6. **Nem megfelelő jogosultság-ellenőrzés.** A rendszerhívás-kezelőnek minden esetben ellenőriznie kell, hogy a hívó folyamat jogosult-e a kért műveletre. Például a `ptrace` rendszerhívásnál a kernelnek ellenőriznie kell a folyamatok közötti viszonyt és a `CAP_SYS_PTRACE` képességet.
7. **A `swapgs` elhagyása vagy rossz sorrendje x86-64-en.** A `swapgs` utasítás cseréli a GS bázisregiszter tartalmát a kernel és a felhasználói érték között. Ha a kernel belépéskor elfelejti meghívni, akkor a felhasználói GS bázist használja a kernel adatok eléréséhez, ami azonnali összeomláshoz vezet.

6.1.6 Labor: rendszerhívási út vázlata és mérése

A labor célja, hogy egy emulált környezetben saját kezűleg kövesse végig a rendszerhívási utat, és megmérje a belépés költségét. A konfiguráció a következő:

- **Emulátor**: QEMU 8.0 vagy újabb (ellenőrizze a helyi telepítés verzióját; a leírás a kiadáskori stabil verziót feltételezi).
- **Architektúra**: x86-64, 1 vCPU, 512 MiB RAM, `-cpu max` vagy `-cpu qemu64`.
- **Kernel**: minimális, a laborhoz mellékelt freestanding C és assembly kód.
- **Fordító**: GCC vagy Clang cross-compiler, `-O2 -fno-stack-protector -fno-pic -mno-red-zone`.
- **Mérési módszer**: `rdtsc` ciklusszámláló, a mérés előtt `cpuid` sorompóval; a frekvenciaváltás elkerülése érdekében a QEMU `-icount` opciója használható, vagy a mérési eredményt ciklusokban adjuk meg.

A labor kiinduló állapota egy bootolható kernel, amely egyetlen felhasználói folyamatot indít, és a felhasználói folyamat egy üres `getpid` rendszerhívást hív egy szoros ciklusban. A kernel a belépési ponton rögzíti a `rdtsc` értéket, a kilépési ponton ismét, és a különbséget egy kernelbeli számlálóban összegzi. A felhasználói program a ciklus végén egy másik rendszerhívással lekéri a mért értékeket, és kiírja azokat.

A labor lépései:

1. **Build**: `make` a labor könyvtárában. A Makefile előállítja a kernel binárist és a felhasználói programot.
2. **Futtatás**: `qemu-system-x86_64 -kernel kernel.bin -initrd initrd.img -append "console=ttyS0" -nographic -no-reboot`.
3. **Eredmény**: a kernel a felhasználói program kimenetét a soros konzolra írja. A program kiírja a hívások számát, a teljes ciklusszámot és az egy hívásra jutó átlagos ciklusszámot.

4. **Várt eredmény**: az üres `getpid` rendszerhívás egy hívásra jutó költsége QEMU emulációban általában több ezer ciklus, mert a QEMU szoftveresen emulálja a processzort. Valódi hardveren ez az érték 50–150 ciklus körül alakul. A mérési naplóban a mért és a várt értéket külön kell feltüntetni.

A mérési napló sablonja:

```
Labor: syscall út mérése
Dátum: [helyőrző]
Környezet: QEMU [verzió], x86-64, 1 vCPU, 512 MiB RAM
Fordító: [GCC verzió], -O2 -fno-stack-protector -fno-pic -mno-red-zone
Mérési módszer: rdtsc + cpuid sorompó, 10^6 hívás, átlagolás
Mért átlagos ciklusszám: [érték]
Várt átlagos ciklusszám (valós hardveren): 50-150 ciklus
Eltérés indoklása: QEMU szoftveres emuláció, nem natív sebesség
```

A labor során a kernel csak a szükséges hardverhozzáférést kapja; a megszakítások és lapozási táblák módosítása előtt mentési pont készül. Valós lemezre, firmware-re vagy bootloaderre destruktív művelet nem kerül. A kód szemléltetés, nem éles rendszeren végrehajtandó utasítás.

6.1.7 Önellenőrző kérdés

Miért kell a kernelnek a felhasználói mutatókat a `copy_from_user` segítségével másolnia, és mi történne, ha a kernel közvetlenül hivatkozna egy felhasználói címre? Indokolja a választ a laphiba és a TOCTOU versenyhelyzet szempontjából.

6.1.8 Összefoglalás

A védelmi gyűrűk és a kernel/user tér közötti átlépés az operációs rendszer biztonsági alapja. A processzor hardveresen kényszeríti ki, hogy a privilegizált műveletek csak kernelmódban futhatnak, a felhasználói program pedig csak ellenőrzött trap-eken keresztül léphet be a kernelbe. A rendszerhívás útja a felhasználói `syscall` utasítástól a kernel belépési pontján, a diszpécseren és a konkrét kernelfüggvényen át a `sysret`-ig tart. A belépés költsége a módváltásból, a regisztermentésből, a cache- és TLB-hatásokból, valamint a spekulatív végrehajtás mitigációiból adódik. A helyes implementációhoz elengedhetetlen a felhasználói adatok biztonságos másolása, az ABI pontos betartása és a jogosultságok ellenőrzése. A következő alfejezet a rendszerhívások ABI-ját és a libc szerepét részletezi, építve az itt bevezetett fogalmakra.

6.2 Rendszerhívások, ABI és libc

A rendszerhívási interfész (syscall) az operációs rendszer magjának az a programozási felülete, amelyen keresztül a felhasználói térben futó programok a kernel szolgáltatásait igénybe vehetik. A hordozható programozási felület ezzel szemben gyakran egy felhasználói könyvtár, tipikusan a libc, amely a nyers rendszerhívásokat a programozó számára ismerős függvényhívásokká alakítja. Ebben a fejezetben a kettő közötti határt, az ABI-t, valamint a libc wrapper réteget vizsgáljuk.

Először tisztázzuk a fogalmakat. Az API (Application Programming Interface) forráskód szintű szerződés: megmondja, hogy egy függvény milyen paramétereket vár, milyen típusú értéket ad vissza, és milyen szemantikát garantál. Az ABI (Application Binary Interface) ezzel szemben bináris szintű szerződés: meghatározza, hogy a lefordított kód milyen regiszterekben adja át az argumentumokat, milyen utasítással vált privilegizált módba, hogyan jelzi a hibát, és milyen struktúra-elrendezést használ. A rendszerhívási interfész elsősorban ABI, míg a libc által nyújtott

felület API. A hordozhatóság érdekében a programozó általában az API-t használja, a rendszer futás közben az ABI-t.

A kernel szolgáltatásainak elérése privilegizált átmenetet igényel, mert a kernel a védett, magasabb jogosultságú szinten fut. A Privilegiszintek és a kernel/user tér átlépése fejezetben tárgyaltuk a privilege szinteket és a trap/syscall mechanizmust. Ebben a fejezetben feltételezzük, hogy van egy ilyen átmenetünk; most arra koncentrálunk, hogy mi történik azután, hogy a vezérlés a kernelbe került.

6.2.1 A rendszerhívási interfész alapjai

A kernel belépési pontja egy úgynevezett syscall dispatcher, amely a rendszerhívási szám alapján kiválasztja a megfelelő kernel függvényt. A rendszerhívási szám egy egész szám, amely egyedi azonosítót ad minden egyes szolgáltatásnak. A számokat általában központi fejlécfájlban definiálják, és a kernel egy táblázatban tárolja a hívható függvények mutatóit. Az alábbi egyszerű példa egy minimalistaker nelben mutatja a táblázatot és a dispatchert:

```
typedef long (*syscall_fn)(unsigned long, unsigned long, unsigned long,
                           unsigned long, unsigned long, unsigned long);

/* A rendszerhívási számok rögzítettek */
#define SYS_WRITE  1
#define SYS_EXIT   60
#define SYS_OPEN   2

static syscall_fn sys_call_table[256] = {
    [SYS_WRITE] = sys_write,
    [SYS_OPEN]  = sys_open,
    [SYS_EXIT]  = sys_exit,
};

long handle_syscall(unsigned long nr,
                    unsigned long a0, unsigned long a1, unsigned long a2,
                    unsigned long a3, unsigned long a4, unsigned long a5)
{
    if (nr >= ARRAY_SIZE(sys_call_table))
        return -ENOSYS;
    if (sys_call_table[nr] == NULL)
        return -ENOSYS;
    return sys_call_table[nr](a0, a1, a2, a3, a4, a5);
}
```

A példa C, szemléltetés. A `-ENOSYS` egy negatív hibakód; a kernel oldalán a hibákat negatív `errno` értékekkel szokás jelezni. Ez fontos eltérés a libc API-tól: a `write()` függvény hiba esetén `-1`-et ad vissza, és a hibakódot a `errno` globális (valójában szálhoz kötött) változóba írja, míg a kernel közvetlenül a negatív hibakódot adja vissza. A libc wrapper feladata ezt a konvenciót összekötni.

Nézzük az argumentumátadást. A felhasználói térből a kernelbe történő átmenetkor az argumentumok nem a szokásos függvényhívási konvenció szerint utaznak, hanem a syscall ABI szerint. Az x86-64 Linuxon például a rendszerhívási számot a `rax` regiszterben, az argumentumokat pedig a `rdi`, `rsi`, `rdx`, `r10`, `r8`, `r9` regiszterekben adják át. A visszatérési érték a `rax` regiszterben érkezik. Más architektúrák eltérnek, de a lényeg ugyanaz: a hívó és a hívott oldal között pontos, bináris szinten rögzített szerződés van. A libc wrapper elrejti ezt a részletet; a programozó a `write()` függvényt hívja, a wrapper pedig a megfelelő regiszterekbe tölti az értékeket, kiadja a `syscall` utasítást, majd feldolgozza a visszatérést. A következő pszeudokód a `write` wrapper működését mutatja:

```
ssize_t write(int fd, const void *buf, size_t count)
{
    long ret;
    ret = syscall(SYS_write, fd, buf, count);
    if (ret < 0) {
        errno = -ret;
        return -1;
    }
    return ret;
}
```

Itt a `syscall` egy alacsony szintű függvény vagy makró, amely kiadja a tényleges trap utasítást és visszaadja a regiszter értékét. A hibakezelés kulcsa a negatív visszatérési érték felismerése. A kernel általában a hibakódok egy kis tartományát használja (Linuxon -4095 és -1 között), így a negatív érték egyúttal hibát is jelent. A `ret < 0` ellenőrzés megfelelő, de a valódi glibc wrapper pontosabb: ellenőrzi, hogy az érték a hibakód tartományba esik-e. Ez azért lényeges, mert a syscall siker esetén nem ad vissza negatív értéket, de a konvenciót expliciten kezelni kell, hogy a jövőbeli változások ne okozzanak hibát.

A mutatóargumentumok külön figyelmet érdemelnek. A felhasználói program által átadott címek a felhasználói címtérbe mutatnak. A kernel nem dereferálhatja ezeket közvetlenül, mert (1) a kernel más címtérrel futhat, (2) a mutató érvénytelen lehet, ami kernel oops-hoz vezet, (3) a felhasználó megpróbálhat kernel memóriát olvasni. Ezért a kernelnek minden felhasználói mutatót ellenőriznie kell, és speciális másoló függvényekkel kell adatot mozgatnia a két tér között. A Linuxon ezek a `copy_from_user` és `copy_to_user` függvények. Az alábbi `sys_write` vázlat mutatja a helyes mintát:

```
long sys_write(unsigned int fd, const char __user *buf, size_t count)
{
    char kbuf[128];
    size_t to_copy = min(count, sizeof(kbuf));
    if (copy_from_user(kbuf, buf, to_copy) != 0)
        return -EFAULT;
    /* ... eszközre írás, például konzol */
    return to_copy;
}
```

Itt a `__user` annotáció dokumentálja, hogy a mutató felhasználói térből származik, és a `copy_from_user` végzi az ellenőrzést és másolást. A példa leegyszerűsített, de rávilágít a legfontosabb biztonsági szempontra.

6.2.2 ABI-stabilitás és verziózás

A rendszerhívási szám és a táblázat stabilitása az ABI-stabilitás sarokköve. Ha egyszer egy rendszerhívási számot publikáltak, a bináris kompatibilitás érdekében nem lehet újra felhasználni más célra, még akkor sem, ha az eredeti syscallt elavultnak nyilvánították. A Linux kernel különösen szigorú ebben: a felhasználói térbe exportált syscall ABI örök érvényű. Új szolgáltatásokhoz új rendszerhívási számokat vezetnek be, a régiek pedig megmaradnak. Ha egy struktúra mérete megváltozik, akkor vagy új syscallt hoznak létre (például az `openat` az `open` utódja), vagy gondosan verziózzák a struktúrát. A Windows esetében a helyzet kissé más: a hivatalos hordozható felület a Win32 API, amely mögött a natív rendszerhívási réteg (ntdll) nem garantáltan stabil; a Microsoft fenntartja a jogot a natív interfész megváltoztatására, ezért a programozóknak a dokumentált Win32 függvényeket kell használniuk.

A verziózás és a szimbólumkezelés a libc oldalán is megjelenik. A C könyvtár dinamikus linkelés esetén szimbólumverziókat használhat, hogy a régebbi binárisok továbbra is a nekik megfelelő

implementációt kapják. A glibc például a `GLIBC_2.2.5` verzióval látja el a szimbólumokat. Ha egy függvény viselkedése megváltozik, a régi verzió megmarad a régi szimbólum mögött, az új pedig új verziót kap. Ez lehetővé teszi, hogy ugyanaz a könyvtár egyszerre szolgálja ki a régi és az új programokat.

Egy másik fontos aspektus az ABI és az API közötti különbség a gyakorlatban: a forráskód újrafordítása az API-t használja, a lefordított bináris futtatása az ABI-t. Ha a programozó `renameat2`-t hív, akkor a forrásban a `unistd.h` deklarációját használja, és a fordító egy hívást generál a libc `renameat2` függvényéhez, amely viszont a megfelelő syscallt hívja. A bináris azonban nem tartalmazza a syscall számot, csak a libc függvényhívást; a syscall szám a libc-n belül van. Ezért a libc frissítése önmagában megváltoztathatja a syscall hívást anélkül, hogy a programot újra kellene fordítani. Ez a rétegelt tervezés teszi lehetővé, hogy a kernel ABI-ja és a libc API-ja egymástól függetlenül fejlődjön.

A rendszerhívási számok és a kapcsolódó makrók általában a felhasználói tér számára is elérhetők. A Linux az `__NR_write`, `__NR_exit` nevű makrókat definiálja az `asm/unistd.h` fejlécben, amelyeket a programok a `syscall()` függvénnyel együtt használhatnak. Ez ugyan nem hordozható, de lehetővé teszi, hogy a fejlesztő közvetlenül a syscall ABI-t célozza. A hordozható programok ehelyett a libc magas szintű függvényeit használják, és a libc felel a megfelelő syscall kiválasztásáért. Például a `fork()` függvényt a libc a modern Linux kernelben a `clone3` syscallra képezi le, de a programozó továbbra is a POSIX `fork()` API-t hívja.

6.2.3 A libc szerepe

A libc (C standard könyvtár) több, mint egyszerű syscall-gyűjtemény. Feladatai közé tartozik:

- **Wrapper függvények biztosítása:** minden POSIX vagy C standard függvényhez, amely mögött syscall áll, egy olyan függvényt nyújt, amely kezeli a regisztereket, az `errno`-t és a visszatérési konvenciót.
- **`errno` kezelése:** a hibaállapotot szálanként tárolja (thread-local storage, TLS), így a párhuzamos programokban nem keverednek össze a hibakódok.
- **Hordozhatóság:** ugyanaz a C forrás több operációs rendszeren is lefordul, mert a libc elrejti a rendszerspecifikus hívásokat.
- **Dinamikus linkelés:** a megosztott könyvtárak betöltését és a szimbólumfeloldást a dinamikus linker (például `ld-linux-x86-64.so.2`) végzi, amely maga is a libc része.
- **Magasabb szintű absztrakciók:** a `stdio` pufferezése, a `malloc` memóriakezelő, a `pthread` szálkezelés mind a libc részei, és ezek jelentősen túlmutatnak a puszta syscall wrappereken.

A `syscall()` függvény egy speciális eszköz: lehetővé teszi tetszőleges syscall közvetlen meghívását a szám és az argumentumok megadásával. Nem hordozható, de hasznos lehet kernel fejlesztésénél vagy olyan új syscallok tesztelésekor, amelyekhez még nincs libc wrapper. A következő kód egy általános `syscall()` csomagolást mutat be Linux x86-64-re:

```
long syscall(long nr, long a0, long a1, long a2, long a3, long a4, long a5)
{
    register long rax asm("rax") = nr;
    register long rdi asm("rdi") = a0;
    register long rsi asm("rsi") = a1;
    register long rdx asm("rdx") = a2;
    register long r10 asm("r10") = a3;
    register long r8  asm("r8")  = a4;
    register long r9  asm("r9")  = a5;
    asm volatile("syscall"
                 : "+r"(rax)
```

```
                : "r"(rdi), "r"(rsi), "r"(rdx), "r"(r10), "r"(r8), "r"(r9)
                : "rcx", "r11", "memory");
    return rax;
}
```

Ez a függvény erősen architektúra-függő, és csak szemléltetésre szolgál. A valódi glibc `syscall()` hasonló elven működik, de hordozhatóbb módon, és a cél architektúrának megfelelően van lefordítva.

A libc wrapper generálása gyakran makrókkal történik, hogy elkerüljék az ismétlést. Például a Linux kernelben a `SYSCALL_DEFINE` makrók állítják elő a syscall bejegyzéseket, a libc oldalán pedig a `weak_alias` és a `libc_hidden_def` mechanizmusok segítenek a szimbólumok kezelésében. A hétköznapi fejlesztő számára azonban elegendő a deklarációk és a wrapper logika ismerete.

A statikus és dinamikus linkelés is érinti az ABI-t. Statikus linkelés esetén a libc kódja a végrehajtható fájlba kerül, így a futtatáshoz nincs szükség külső könyvtárra, de a bináris nagyobb, és a libc biztonsági frissítései nem érvényesülnek újrafordítás nélkül. Dinamikus linkelésnél a program a futás közben tölti be a megosztott libc-t, így a frissítések azonnal hatnak, de a binárisnak kompatibilisnek kell lennie a betöltött verzió ABI-jával. A szimbólumverziózás ezt a kompatibilitást hivatott biztosítani.

6.2.4 Labor: syscall-táblázat és libc-wrapperek vázlata

A labor célja egy egyszerű syscall-táblázat és libc-wrapperek vázlatának elkészítése egy emulált környezetben. A kiindulópont a Privilegiszintek és a kernel/user tér átlépése fejezetben bemutatott trap kezelő, amely már rendelkezik a felhasználói/kernel átmenettel. Most a dispatchert és a wrapper réteget adjuk hozzá.

Környezet: QEMU x86-64, 1 virtuális CPU, 128 MiB RAM, freestanding C, GCC vagy Clang. A pontos verziókat a reprodukciós jegyzőkönyvben rögzíteni kell. A boot mechanizmus adott (például a BIOS, UEFI és a bootfolyamat fejezetből), a labor a syscall rétegre koncentrál.

Feladat: Implementáljuk a `sys_write` és `sys_exit` rendszerhívásokat, valamint a dispatchert. Készítsünk egy felhasználói tesztprogramot, amely a libc wrapper vázlatot használja: `my_write()` és `my_exit()` függvényeket, amelyek a rendszerhívási ABI-t hívják, beállítják az `errno`-t, és a teszt kiírja a “hello syscall” üzenetet, majd 0 kóddal kilép.

A syscall táblázat és a dispatcher kódja megegyezhet a fenti példával. A wrapper vázlat a következő lehet:

```
/* userland tesztprogram, freestanding, saját minimalista libc */
typedef unsigned long size_t;
typedef long ssize_t;

static int errno;

static inline long raw_syscall(long nr, long a0, long a1, long a2)
{
    register long rax asm("rax") = nr;
    register long rdi asm("rdi") = a0;
    register long rsi asm("rsi") = a1;
    register long rdx asm("rdx") = a2;
    asm volatile("syscall"
                : "+r"(rax)
                : "r"(rdi), "r"(rsi), "r"(rdx)
                : "rcx", "r11", "memory");
    return rax;
}
```

```c
ssize_t my_write(int fd, const void *buf, size_t count)
{
    long ret = raw_syscall(1, fd, (long)buf, count);
    if (ret < 0) {
        errno = -ret;
        return -1;
    }
    return ret;
}

void my_exit(int status)
{
    raw_syscall(60, status, 0, 0);
    __builtin_unreachable();
}
```

Itt a `raw_syscall` inline assembly, x86-64-re specifikus. A wrapper pontosan a korábban leírt logikát követi: hívja a syscallt, ellenőrzi a negatív visszatérést, beállítja az `errno`-t. Az `errno` ebben a minimalista környezetben egy egyszerű globális változó; valós libc-ben TLS-ben lenne. A kód szemléltetés, nem éles rendszeren futtatandó.

Build és futtatás: A reprodukciós csomag tartalmazza a Makefile-t, a linker scriptet és a futtatási parancsot. Egy lehetséges parancssor:

```
make clean
make CFLAGS="-O0 -g -ffreestanding -nostdlib"
qemu-system-x86_64 -cpu qemu64 -m 128M -nographic -kernel kernel.bin
```

A verziók ellenőrzése szükséges: a QEMU, a GCC és a binutils pontos verzióját rögzíteni kell a naplóban.

Mérés: A syscall belépés és visszatérés idejét mérjük ciklusban. A mérési módszer: egy 1 000 000 iterációból álló ciklus, amely egy triviális syscallt hív (például `my_write` egy 0 hosszúságú pufferrel). A méréshez a `rdtsc` utasítást használjuk a ciklus előtt és után, majd az átlagot számoljuk. A pontos érték függ a QEMU módjától (TCG vs KVM), a hoszt CPU-tól és a kernel beállításaitól, ezért a mért értéket és a várt nagyságrendet elkülönítve közöljük. A várt eredmény az, hogy a syscall költsége a QEMU TCG módban nagyságrendileg 10 000–100 000 ciklus, míg natív KVM-en 100–1000 ciklus lehet; ezek az értékek csak tájékoztató jellegűek, és az aktuális mérést a jegyzőkönyvben rögzíteni kell. A mérési naplónak tartalmaznia kell a hoszt CPU típusát, órajelét, a QEMU verzióját, a fordítási beállításokat (`-O0`, `-O2`), és a mérési ciklusok számát. A mérési eredményeket nem szabad abszolút igazságként kezelni, mindig a reprodukciós körülményekkel együtt kell értelmezni.

Elvárt eredmény: A tesztprogram kiírja a “hello syscall” üzenetet, és 0 kilépési kóddal fejeződik be. A napló tartalmazza a mért ciklusszámokat, a hibakódokat és az esetleges eltéréseket a várt viselkedéstől.

6.2.5 Tipikus hibák és helyes megoldások

A gyakorlat során az alábbi tipikus hibák fordulhatnak elő:

1. **Hibás rendszerhívási szám.** Ha a wrapperben 1 helyett 0-t írunk, a dispatcher `ENOSYS` hibát ad, vagy rossz függvényt hív. A megoldás: a syscall számokat központi fejlécfájlban definiáljuk, és a wrapper is ugyanazt a definíciót használja.

2. **Elmaradt argumentum-ellenőrzés a kernel oldalán.** Ha a `sys_write` közvetlenül dereferálja a felhasználói mutatót, a kernel összeomolhat. A helyes megoldás a `copy_from_user` használata, valamint a hossz és a célpuffer ellenőrzése.

3. **Az `errno` beállításának hiánya.** Ha a wrapper nem írja be az `errno`-t, a hívó félrevezető hibakódot kap, vagy nem is tudja, hogy hiba történt. A wrappernek mindig be kell állítania az `errno`-t, mielőtt `-1`-et ad vissza.

4. **A visszatérési érték helytelen értelmezése.** A kernel negatív hibakódot ad vissza, de ha a wrapper azt hiszi, hogy a `-1` a hibajelzés, és nem a negatív hibakódot, akkor rossz lesz az `errno`. A helyes minta az, hogy a ret negatív értékét vizsgáljuk, és az abszolút értékét adjuk az `errno`-nak.

5. **Típushibák a hosszoknál.** A `size_t` és `ssize_t` keverése, valamint a 32/64 bites regiszterek közötti eltérés hibás adatátvitelhez vezet. A wrappernek pontosan követnie kell a syscall ABI típusait.

6. **ABI-törés a struktúrákban.** Ha egy syscall argumentumaként struktúrát adunk át, és a struktúra elrendezése megváltozik, a régi binárisok eltörnek. A megoldás a struktúrák verziózása vagy új syscall bevezetése.

A fejezet végére az olvasó képes lesz elmagyarázni a rendszerhívási interfész és a hordozható programozási felület közötti különbséget, felépíteni egy egyszerű syscall táblázatot, és megírni a megfelelő libc wrapper vázlatot. A következő fejezet (Shell, init, daemonok és a userland felépítése) ezekre az alapokra épít, és bemutatja, hogyan épül fel a userland: a shell, az init folyamat és a daemonok.

6.2.6 Önellenőrző kérdések

1. Mi a különbség az API és az ABI között? Miért fontos a különbség a rendszerhívásoknál?
2. Miért ad vissza a kernel negatív hibakódot a libc `-1` visszatérési értéke helyett?
3. Hogyan biztosítja a libc wrapper az `errno` helyes beállítását?
4. Mi történik, ha a felhasználói program érvénytelen mutatót ad át egy syscallban? Hogyan védekezik a kernel?
5. Miért nem lehet egy publikált rendszerhívási számot újra felhasználni?

6.3 Shell, init, daemonok és a userland felépítése

A felhasználói tér – gyakran userland – a kernel fölé épülő futtatási környezet. Nem alkalmazások rendezetlen halmaza, hanem rétegzett infrastruktúra: dinamikus betöltő, futásidejű könyvtárak, shell, `init`, szolgáltatásmenedzser, daemonok és támogató rendszerprogramok. E fejezet ezt a rétegződést, a folyamatfát, a környezet átadását, valamint a shell és a szolgáltatásindítás gyakorlati mintáit mutatja be.

6.3.1 A felhasználói tér rétegei

A kernel felől nézve minden userland program ugyanazt a rendszerhívási határt látja: az `execve()` betölt egy futtatható állományt, a `fork()` folyamatot hoz létre, a `read()` és `write()` pedig a kapott fájlleírókon kommunikál. A felhasználói tér mégis rétegezett, mert ezekből az alapelemekből különböző felelősségű programok épülnek fel.

A tipikus rétegek:

1. **Dinamikus betöltő és futásidejű könyvtárak.** A dinamikusan linkelt program betöltésekor a kernel az ELF interpretert indítja el, amely a megosztott könyvtárakat a folyamat címterébe illeszti. A `libc` a rendszerhívások hordozható, C nyelvű burkát adja.
2. **Alapvető rendszerprogramok.** Ide tartozik a `init`, a parancsértelmező shell, valamint a bejelentkezést, naplózást, ütemezést és eszközkezelést végző daemonok.
3. **Szolgáltatásmenedzser.** Ez koordinálja a fenti programok indítását, leállítását, újraindítását és függőségeit. Hagyományos rendszereken a `init`, modern Linux rendszereken gyakran a systemd, de más rendszerek OpenRC-t, runitot vagy saját managert használnak.
4. **Alkalmazások.** Ezek a felhasználó tényleges munkáját végzik, és többnyire a fenti komponensek által kialakított környezetre támaszkodnak.

A rétegzés nem jelenti azt, hogy az alkalmazások közvetlenül nem hívhatnak rendszerhívást. A határ a kernel és a userland között van, nem a libc és az alkalmazás között. A libc csupán kényelmes, gyakran optimalizált belépési pontot ad az `execve`, `open`, `read`, `write`, `mmap` és társai elé.

6.3.2 Folyamatfa, session és folyamatkörnyezet

A felhasználói tér futó egységei folyamatokból állnak. Minden folyamat rendelkezik egy folyamatazonosítóval (PID) és egy szülőfolyamat-azonosítóval (PPID). A kernel az első userland folyamatot – hagyományosan az `init` programot – `PID 1` értékkel indítja. A későbbi folyamatok ebből a gyökérből ágaznak le: a shell elindítja a parancsokat, a service manager elindítja a daemonokat. Így a rendszer futó programjai egy folyamatfát alkotnak.

Példa egy egyszerű folyamatfára:

```
PID 1: init
  PID 10: shell
     PID 23: ls
  PID 11: minta-daemon
```

Itt `minta-daemon` és `shell` is közvetlenül az `init` gyermeke. A `ls` a shell gyermeke, mert a shell a `fork()` után az `execve()` hívás előtt állította elő a folyamatot. Ha egy szülő meghal, a gyermek árva folyamattá válik. A kernel ilyenkor átadja a gyermeket a `PID 1` folyamatnak, vagy Linux alatt egy `PR_SET_CHILD_SUBREAPER` beállítással rendelkező közeli ősnek. A `PR_SET_CHILD_SUBREAPER` a `prctl()` rendszerhívás egyik opciója; a hívás siker esetén `0`, hibánál `-1` visszatérési értéket ad, és a hibát az `errno` írja le. Ez az adoptálási mechanizmus biztosítja, hogy az elhunyt folyamatokat mindig begyűjtse valaki, és ne maradjanak zombie állapotban.

A folyamatok nemcsak szülő-gyermek viszonyban állnak, hanem **sessionökbe** és **folyamatcsoportokba** is szerveződnek. Egy session jellemzően egy bejelentkezéshez, terminálhoz vagy szolgáltatáshoz tartozik. A session vezetője a `setsid()` hívással válik ki a korábbi sessionből. A `setsid()` paraméter nélküli rendszerhívás, amely siker esetén az új session azonosítóját adja vissza, hibánál pedig `-1` értéket. Ha a hívó már folyamatcsoport-vezető, a hívás `EPERM` hibával tér vissza. A folyamatcsoportokkal a shell például egy teljes pipeline-t tud együtt kezelni: a `ls | wc -l` parancs két folyamata egy közös folyamatcsoportba kerül, így a felhasználó egyetlen Ctrl-C billentyűvel megszakíthatja mindkettőt.

A folyamatkörnyezet a folyamatfa és a session mellett több láthatatlan állapotot is tartalmaz:

- **Környezeti változók.** Az `execve(const char *pathname, char *const argv[], char *const envp[])` rendszerhívás a programnak átadja az argumentumvektort és a környezeti tömböt. A shell a `PATH`, `HOME`, `USER` és hasonló változókat ezen keresztül örökíti. Az `execve` siker esetén nem tér vissza; hibánál `-1` értéket ad, például `ENOENT` (a fájl

nem létezik), `EACCES` (nincs végrehajtási jog) vagy `ENOEXEC` (nem futtatható formátum) hibakóddal.

- **Aktuális munkakönyvtár.** A `chdir(const char *path)` hívás a folyamat munkakönyvtárát állítja. Ez a folyamat saját állapota, ezért a `cd` parancs csak a shell beépített parancsaként működhet; egy külön folyamatban futtatott `chdir` hatása elveszne.
- **Fájlleíró-tábla.** A `fork()` a fájlleírókat másolja, az `execve()` megtartja a nyitott leírókat, kivéve, ha azokat `O_CLOEXEC` jelöléssel nyitották. Ezért a shell a redirekciókat a gyermekfolyamat `dup2()` hívásaival állítja be.
- **Fájlmód-maszk (umask).** Az `umask(mode_t mask)` a folyamat által létrehozott fájlok alapértelmezett jogosultságait szűkíti. A hívás a korábbi maszkot adja vissza, és önmagában nem hibázik. A daemonok tipikusan szigorú maszkkal indulnak.
- **Erőforráskorlátok.** A `setrlimit()` hívással a folyamat saját erőforrás-használatát korlátozhatja; ezt a shell `ulimit` beépített parancsa is elérhetővé teszi.

Ezek az állapotok együttesen alkotják azt a környezetet, amelyben a userland program fut. A shell és az init feladata éppen az, hogy ezt a környezetet értelmesen állítsa be és adja tovább.

6.3.3 A shell mint a userland munkafelülete

A shell, magyarul parancsértelmező, interaktív és nem interaktív üzemmódban is használható. Interaktív módban ciklikusan olvas, értelmez, végrehajt, majd újra olvas. Nem interaktív módban egy scriptet hajt végre. A shell nem privilegizált program: ugyanazokat a rendszerhívásokat használja, mint bármely más userland folyamat. Az ereje abból fakad, hogy ezeket az elemeket rugalmasan kombinálja.

A shell fő tevékenységei:

1. **Beolvasás és tokenizálás.** A bemeneti sort szavakra, operátorokra és vezérlőkarakterekre bontja.
2. **Értelmezés.** Felépíti a parancsok belső reprezentációját: egyszerű parancsok, pipeline-ok, redirekciók, lista- és feltételes szerkezetek.
3. **Végrehajtás.** A beépített parancsokat a shell saját állapotán hajtja végre, a külső programokat `fork()` és `execve()` hívásokkal futtatja.
4. **Várakozás és hibakezelés.** Az előtérben futó folyamatokra `waitpid()` hívással vár, a háttérben futókat a job-táblában tartja nyilván.

Egy egyszerű shell főhurok pszeudokódban:

```
ciklus amíg nincs exit:
    prompt kiírása
    sor beolvasása
    ha EOF:
        kilépés
    parancsfa = elemzés(sor)
    ha parancsfa üres:
        folytatás
    ha parancsfa beépített parancs:
        végrehajtás a shell állapotán
    különben:
        futtat(parancsfa)

futtat(parancsfa):
    minden pipeline szakaszhoz:
        pid = fork()
        ha pid < 0:
            hiba kiírása; kilépés a gyermekből nem történik
```

```
        ha pid == 0:
            redirekciók beállítása dup2() hívásokkal
            ha a szakasz külső program:
                execvp(argv[0], argv)
                ha execvp visszatér:
                    _exit(127)
        különben:
            a szülő a létrehozott folyamatot nyilvántartja

    ha előtéri futtatás:
        waitpid() minden előtéri gyermekre
    különben:
        a job-táblába rögzíti a háttérfolyamatokat
```

A `fork()` rendszerhívás paraméter nélküli; visszatérése a szülőben a gyermek PID-je, a gyermekben `0`, hibánál `-1`. Gyakori hibakód az `EAGAIN`, ha a folyamatszám-korlát elérhető. Az `execvp()` az `execve()` könyvtári burka, amely a `PATH` környezeti változó alapján keresi a futtatható fájlt. A `waitpid(pid_t pid, int *status, int options)` a megadott folyamat állapotváltozására vár; visszatérése a várt PID, hibánál `-1`. A shell a `status` értékéből nyeri ki a kilépési kódot.

A beépített parancsok azért fontosak, mert a shell **saját** állapotát módosítják. A `cd` a munkakönyvtárat, az `export` a környezeti változókat, az `ulimit` az erőforráskorlátokat, a `jobs`, `fg`, `bg` pedig a job-táblát kezeli. Ha ezeket külön folyamatként futtatnánk, a módosítás a gyermekben történne, majd a gyermek kilépésével elveszne.

A shell felel a **folyamatcsoport-kezelésért** is. Interaktív használatnál a terminálhoz tartozó előtér-csoport kapja a billentyűzetről érkező jeleket. A Ctrl-C a `SIGINT` jelet küldi a csoportnak, a Ctrl-Z a `SIGTSTP` jelet. A shell a `setpgid()` és a `tcsetpgrp()` hívásokkal állítja be, hogy az éppen futó parancs csoportja legyen az előtérben. A `setpgid(pid_t pid, pid_t pgid)` siker esetén `0`, hibánál `-1`; `ESRCH` jelzi, ha a folyamat nem létezik, `EPERM` pedig azt, ha a művelet nem megengedett.

A felhasználói tér szempontjából a shell egyszerre felület és programozási nyelv. A scriptekben a feltételes elágazás, a ciklus és a függvények ugyanazt a folyamatmodellt használják, mint az interaktív parancsok. A `set -e`, a `trap` és a kilépési kódok kezelése teszi lehetővé, hogy a shell scriptekben a hibákat is ellenőrizhető módon kezeljék.

6.3.4 Init és szolgáltatásmenedzser

Az `init` a rendszer első userland folyamata. Felelőssége kettős: egyrészt be kell töltenie és el kell indítania a rendszer szolgáltatásait, másrészt gondoskodnia kell a folyamatfa karbantartásáról. Ha egy szülő meghal, a gyermeket az `init` adoptálja, és a `waitpid()` hívásokkal begyűjti a kilépési állapotokat. E nélkül a rendszerben zombie folyamatok halmozódnának fel.

A hagyományos SysV init modellben a szolgáltatások **runlevelekbe** szerveződnek. A runlevel egy rendszerállapot, például egyfelhasználós mód, többfelhasználós mód vagy grafikus mód. Az `inittab` fájl tartalmazza az egyes runlevelekhez tartozó indítószkripteket. Az indítószkriptek jellemzően `/etc/init.d/` alatt találhatók, és a különböző runlevel-könyvtárakban `S` (start) és `K` (kill) prefixű szimbolikus linkekkel hivatkoznak rájuk. A számok az indítási sorrendet adják meg: először az alacsonyabb számú szkriptek futnak, így a hálózati szolgáltatások a hálózati interfész beállítása után indulnak. A soros indítás egyszerű és átlátható, de lassú lehet, mert a független szolgáltatások is egymás után futnak.

A modern szolgáltatásmenedzserek – legismertebb példájuk a systemd – deklaratív **unit** fájlokkal dolgoznak. Egy unit lehet szolgáltatás, socket, időzítő, mount pont, target vagy egyéb erőforrás. A service unit írja le, hogyan kell egy daemont elindítani, milyen függőségei vannak, és milyen

újraindítási politikát kövessen. A manager nem sorban, hanem a függőségi gráf alapján indít: ha két szolgáltatás független, az indításuk párhuzamosítható.

Szemléltető service unit:

```
# /etc/systemd/system/os020-minta.service
# Csak laborban használt példa; nem éles rendszeren futtatandó.
[Unit]
Description=OS020 példa szolgáltatás
After=network-online.target
Requires=network-online.target

[Service]
Type=simple
ExecStart=/opt/os020-lab/bin/minta-daemon --config /opt/os020-lab/etc/minta.conf
Restart=on-failure
RestartSec=2s
User=os020user
Group=os020user
NoNewPrivileges=true
PrivateTmp=true

[Install]
WantedBy=multi-user.target
```

A `Type=` érték azt mondja meg, hogy a manager mikor tekinti elindultnak a szolgáltatást. A `Type=simple` esetén a `fork()/execve()` után azonnal késznek tekinti; a `Type=forking` a klasszikus daemonizáló programokat támogatja, ahol a szülő kilép, és a valódi daemon a háttérben fut. A `Type=notify` esetén a daemon egy readiness üzenetet küld, például a `sd_notify()` könyvtári híváson keresztül. Ezzel elkerülhető az a versenyhelyzet, amikor a manager már késznek hiszi a szolgáltatást, de az még nem vette fel a socketjét vagy nem fejezte be az inicializálást.

A szolgáltatásmenedzser **socket activationt** is használhat: ilyenkor a socketet nem a daemon, hanem a manager hozza létre, és a daemon csak akkor indul el, amikor az első kérés megérkezik. Ez csökkenti az indítási időt, és lehetővé teszi, hogy a függőségek betöltése késleltetve történjen. A daemon a megkapott fájlleírót a `LISTEN_FDS` környezeti változó alapján azonosítja.

A `init` és a szolgáltatásmenedzser közös felelőssége a **leállítási sorrend** is. A shutdown folyamat során a manager előbb leállítja a felhasználói szolgáltatásokat, majd a rendszerszintű daemonokat, végül szinkronizálja a fájlrendszereket és leállítja a kernelt. A leállítási sorrend általában az indítási sorrend fordítottja, vagy a függőségi gráf szerinti topologikus sorrend.

A szolgáltatásindítás tervezésekor mindig meg kell adni:

- a szolgáltatás azonosítóját és célját,
- a futtató felhasználót és csoportot,
- a szükséges erőforrásokat (fájlrendszer, hálózat, socket, eszköz),
- a függőségeket és az indítási sorrendet,
- a readiness-feltételt,
- az újraindítási politikát,
- a naplózás és a hibakezelés módját,
- a leállítási viselkedést.

Ez a dokumentáció egyben a laborban bemutatott inicializálási terv alapja is.

6.3.5 Daemonok és háttérfolyamatok

A daemon – magyarul gyakran démonnak is nevezik – olyan háttérfolyamat, amely nem kötődik terminálhoz, és rendszerint folyamatosan fut. A daemonok tipikus feladatai közé tartozik a hálózati szolgáltatások kiszolgálása, a naplózás, az ütemezett feladatok futtatása és az eszközök figyelése.

Egy hagyományos, terminálhoz kötött programból a daemonizálás klasszikus lépései:

1. **Első `fork()`.** A szülő kilép, így a daemon nem a shell közvetlen gyermeke marad, és a shell `waitpid()` hívása nem blokkol rá. A gyermek a háttérben fut tovább.
2. **`setsid()` hívás.** A folyamat új sessiont hoz létre, és kilép a korábbi terminálhoz tartozó folyamatcsoportból.
3. **Második `fork()`.** A második gyermek már nem session-vezető, ezért nem szerezhet automatikusan controlling terminalt. Ez a lépés hordozhatósági okokból fontos.
4. **`chdir()` egy biztonságos munkakönyvtárba.** Ha a daemon a `/` alatt marad, könnyebb elkerülni, hogy egy mount pontot foglaljon a munkakönyvtára.
5. **`umask()` beállítása.** A daemon által létrehozott fájlok jogosultságainak szigorítása.
6. **Fájlleírók bezárása és átirányítása.** A standard bemenetet, kimenetet és hibakimenetet általában `/dev/null`-ra vagy naplófájlra irányítja a daemon.
7. **A tényleges szolgáltatási ciklus futtatása.**

Szemléltető C kód a klasszikus daemonizálás lépéseire:

```c
/* os020_lab_daemon_minta.c - szemléltetés, nem éles szolgáltatás.
 * A cél a daemonizálás lépéseinek bemutatása. */
#include <fcntl.h>
#include <signal.h>
#include <stdio.h>
#include <stdlib.h>
#include <sys/stat.h>
#include <sys/types.h>
#include <unistd.h>

int main(void)
{
    pid_t pid;

    /* 1. Az első fork: a shell ne várjon a daemonra. */
    pid = fork();
    if (pid < 0)
        return 1;
    if (pid > 0)
        _exit(0);

    /* 2. Új session létrehozása. */
    if (setsid() == (pid_t)-1)
        return 1;

    /* 3. Második fork: ne lehessen új controlling terminalt szerezni. */
    pid = fork();
    if (pid < 0)
        return 1;
    if (pid > 0)
        _exit(0);

    /* 4. Munkakönyvtár. Laborban /var/tmp/os020-lab. */
    if (chdir("/var/tmp/os020-lab") != 0)
        return 1;
```

```
    /* 5. Fájlmód-maszk: a csoportírás és az egyéb jogok tiltása. */
    umask(027);

    /* 6. Standard leírók átirányítása /dev/null-ra. */
    int fd = open("/dev/null", O_RDWR);
    if (fd < 0)
        return 1;
    if (dup2(fd, STDIN_FILENO) < 0 ||
        dup2(fd, STDOUT_FILENO) < 0 ||
        dup2(fd, STDERR_FILENO) < 0)
        return 1;
    if (fd > 2)
        close(fd);

    /* 7. Szolgáltatási ciklus. A pause() EINTR hibával tér vissza jelre. */
    for (;;) {
        pause();
    }
    return 0;
}
```

A fenti kód a `fork`, `setsid`, `chdir`, `umask`, `open`, `dup2`, `close` és `pause` rendszerhívásokat használja. A `pause()` addig blokkol, amíg a folyamat jelet nem kap; visszatérése ilyenkor `-1`, az `errno` pedig `EINTR`. A daemon a jeleket – például a `SIGTERM` leállítási kérelmet – szignálkezelővel vagy szignálmaszkkal kezeli. A fenti ciklus egyszerűsített, és nem tartalmaz takarítást; a laborban a `minta-daemon` a `SIGTERM` hatására naplózza a leállást és kilép.

A modern service managerek mellett a klasszikus daemonizálás részben feleslegessé válhat. Ha a daemont a systemd indítja, a manager kezeli a háttérbe vonást, a környezetet és a standard leírókat. Ilyenkor a daemon előtérben futhat, és a `Type=simple` vagy `Type=notify` beállítással jelezheti a készenlétet. Ezért fontos különválasztani a **daemon viselkedést** – tehát a terminál nélküli, hosszú életű háttérfutást – a **daemonizálási protokolltól**, amely a `fork`-okkal és a sessionkezeléssel foglalkozik. A jól tervezett daemon a viselkedést valósítja meg, a protokollt pedig a futtató környezetre bízza, vagy deklaratívan dokumentálja.

6.3.6 Tipikus hibák és javításuk

A userland felépítése során visszatérő hibák jellemzően a folyamat- és erőforrás-tulajdonjog félreértéséből fakadnak.

- **Egyszeres `fork` a daemonizálásnál.** A folyamat `setsid()` után session-vezető marad, és később controlling terminalt szerezhet. Helyes megoldás a második `fork`, vagy a modern managerre bízott protokoll.
- **A `chdir()` elhagyása.** Ha a daemon egy mount pont alatt hagyja a munkakönyvtárát, a mount pont nem választható le, és a leállítási sorrend hibát jelezhet. A daemon ezért olyan könyvtárba lép, amelyet nem foglal le.
- **Nyitott fájlleírók öröklése.** A `fork()` és `execve()` a nyitott leírókat megtartja. Ha a daemon nem zárja be vagy nem irányítja át a standard leírókat, a hívó shell termináljához kötött leírók életben maradhatnak, és naplóüzeneteket írhatnak oda.
- **Helytelen `umask`.** A daemon által létrehozott konfigurációs vagy PID-fájl túl megengedő jogosultságokat kaphat. A szigorú maszk beállítása után létrehozott fájlok biztonságosabbak.
- **A `SIGHUP` kezeletlensége.** Ha a controlling terminal bezáródik, a folyamat `SIGHUP` jelet kaphat, és alapértelmezés szerint meghal. A daemonok vagy a jelet ignorálják, vagy konfiguráció-újraolvasásra használják.

- **Versenyhelyzet a PID-fájl kezelésében.** A PID-fájl ellenőrzése és létrehozása között újabb példány indulhat. Helyes megoldás az `open("/run/os020-minta.pid", O_CREAT|O_EXCL|O_WRONLY, 0644)` használata, és a régi, elavult fájl óvatos tisztítása, vagy a PID-fájl teljes elhagyása, ha a service manager a folyamatot úgyis nyomon követi.
- **A `waitpid()` elhagyása az `init`-ben.** Ha az `init` nem gyűjti be az elhunyt gyermekeket, a rendszerben zombie folyamatok gyűlnek fel. Az `init` fő ciklusában a `waitpid(-1, &status, 0)` hívás az összes gyermekre várakozik.
- **Beépített parancs külön folyamatban.** A `cd` vagy az `export` külön folyamatban való futtatása nem módosítja a shell állapotát. A shellnek ezeket belső parancsként kell azonosítania, és a saját folyamatában kell végrehajtania.

E hibák felismerése a laborban is ellenőrzési pont: a daemon indulásakor a standard leírók állapotát, a munkakönyvtárat, az umaskot és a PID-fa kapcsolatait minden mérésnél dokumentálni kell.

6.3.7 Labor: init- és szolgáltatásindítási terv

A labor célja egy minimális, de teljes userland indítási terv elkészítése és ellenőrzése. A feladat nem igényel éles gépet; minden lépés egy emulált környezetben vagy egy eldobható laborlemezképen fut. A hoszt bootloaderét, rendszerpartícióját és valódi lemezét tilos módosítani.

6.3.8 Kiinduló állapot

Hozz létre egy `os020-lab/` könyvtárat. A laborban három komponenst kell indítani:

- `init`: a `PID 1` folyamat, amely naplózza az indítási eseményeket, elindítja a daemont és a shellt, majd begyűjti a gyermekfolyamatokat.
- `minta-daemon`: háttérszolgáltatás, amely a `/var/tmp/os020-lab/daemon.log` fájlba ír egy `READY` sort, majd `SIGTERM`-ig fut.
- `shell`: interaktív parancsértelmező, amely a soros konzolon jelenik meg.

A labor konfigurációja helyőrző felhasználót és elérési utat használ:

```
# os020-lab/config.env
LAB_NAME=os020-lab
WORKDIR=/var/tmp/os020-lab
DAEMON_READY_TIMEOUT_MS=1000
SHELL_PROMPT_TIMEOUT_MS=500
```

6.3.9 Terv

Készíts egy indítási tervet a következő táblázat szerint:

Szolgáltatás	Típus	Függőség	Indítási mód	Readiness
`init`	hosszú életű folyamat	kernel indítja	`fork` nélkül, `PID 1`	log: `init started`
`minta-daemon`	daemon	`WORKDIR` létezik	`fork` + `execve`	log: `READY`
`shell`	interaktív folyamat	`init` fut	`fork` + `execve`	prompt megjelenése

A tervben rögzítsd a leállítási sorrendet is: előbb a shell, majd a daemon `SIGTERM` jelzése, végül az `init` kilépése. Az `init` a `waitpid()` hívásokkal gyűjti be a kilépési állapotokat.

6.3.10 Build és futtatás

A labor Makefile-ja a három programot lefordítja, majd egy initramfs-lemezképet hoz létre:

```
# os020-lab/Makefile - szemléltetés; a pontos kernel- és QEMU-paraméterek a
# labor reprodukciós csomagjában találhatók.
CFLAGS ?= -O2 -g -Wall -Wextra -Werror

all: init minta-daemon shell initramfs

init: init.c
    $(CC) $(CFLAGS) -o $@ $<

minta-daemon: minta-daemon.c
    $(CC) $(CFLAGS) -o $@ $<

shell: shell.c
    $(CC) $(CFLAGS) -o $@ $<

initramfs: init minta-daemon shell
    # A labor saját könyvtárából épített cpio archívum.
    ./make-initramfs.sh

run: initramfs
    ./run.sh

clean:
    rm -f init minta-daemon shell initramfs.cpio.gz
```

A futtató parancs emulált környezetben:

```
#!/bin/sh
# os020-lab/run.sh - emulált labor, nem éles gép.
# A KERNEL változó a labor reprodukciós csomagjában található kernelre mutat.
qemu-system-x86_64 \
  -m 256M \
  -kernel "${KERNEL}" \
  -initrd build/initramfs.cpio.gz \
  -nographic \
  -no-reboot
```

A `KERNEL` elérési útja helyőrző; a laborban a pontos fájlt a reprodukciós csomag rögzíti. A QEMU verziója, a vendég CPU típusa és a memória mérete a mérési napló része.

6.3.11 Mérési napló

A futtatás során rögzítsd az alábbi értékeket:

- `init_start_ms`: az `init` logbejegyzésének időbélyege a konzolon.
- `daemon_ready_ms`: a `READY` sor időbélyege, az `init_start_ms`-hez képest.
- `shell_prompt_ms`: a prompt megjelenésének időbélyege, az `init_start_ms`-hez képest.
- `pid_tree`: a `ps -eo pid,ppid,sid,comm` parancs kimenete.

A **várt** eredmény külön fájlban:

```
# expected.txt - várt értékek
pid_tree:
  PID 1: init
  daemon: PPID 1, SID != 1
  shell: PPID 1, SID != 1

daemon_ready_ms <= 1000
shell_prompt_ms <= 500
```

A `measured.log` a tényleges, mért értékeket tartalmazza. A daemon `SID` értéke eltér az `init` SID-jétől, mert a daemon `setsid()` hívással új sessiont hoz létre. A shell ugyancsak új sessionben fut, ha a labor initje ezt beállítja; ha nem, a feltételt a tervben ennek megfelelően kell módosítani. A lényeg, hogy a PPID-kapcsolatok és a ready-idők mérhetők legyenek.

6.3.12 Ellenőrzési pontok

1. A daemon fájlleírói a `fork` után a `/dev/null`-ra mutatnak-e?
2. A `ps` fában a daemon és a shell közvetlenül az `init` gyermeke-e?
3. A daemon `SIGHUP` hatására nem hal-e meg a laborban?
4. A `SIGTERM` után a daemon naplózza-e a leállást, és az `init` begyűjti-e a kilépési állapotát?
5. A labor leállítása után a hoszt állapota változatlan marad-e?

A labor visszaállítása: `make clean`, a QEMU-lemezkép eldobása, majd annak ellenőrzése, hogy a hoszt fájlrendszerén nem jött létre módosítás. A mérési napló a várt és a mért értékeket mindig külön oszlopban vagy fájlban tartalmazza.

6.3.13 Önellenőrző kérdések

1. Miért nem elegendő egyetlen `fork()` a klasszikus daemonizáláshoz, és milyen problémát előz meg a második `fork()`?
2. Miért kell a `cd` parancsnak a shell beépített parancsának lennie, és mi történne, ha külön folyamatban hajtanánk végre?
3. Milyen két fő felelőssége van a `PID 1` folyamatnak a folyamatfa és a szolgáltatások szempontjából?
4. Mit jelent a `Type=notify` service unit beállítás, és milyen hibát kerül el a `Type=simple` beállításhoz képest?

Válaszok röviden:

1. Az első `fork()` után a gyermek az új session vezetője lesz, és később controlling terminalt szerezhet. A második `fork()` után a folyamat már nem session-vezető, ezért ez a kockázat megszűnik.
2. A `cd` a folyamat munkakönyvtárát módosítja. Külön folyamatban a `chdir()` csak a gyermek állapotát változtatná, és a gyermek kilépésekor a módosítás elveszne.
3. A `PID 1` elindítja és leállítja a szolgáltatásokat, valamint adoptálja és begyűjti az árva folyamatokat, megakadályozva a zombie folyamatok felhalmozódását.
4. A `Type=notify` esetén a daemon readiness üzenetet küld a managernak, amikor ténylegesen készen áll. A `Type=simple` esetén a manager az `execve()` után azonnal késznek tekinti, ami versenyhelyzethez vezethet, ha a daemon még nem inicializálta a socketjeit vagy adatszerkezeteit.

6.3.14 Összegzés

A felhasználói tér rétegzett infrastruktúra, amelyben a shell, az `init`, a szolgáltatásmenedzser és a daemonok együttműködnek. A shell a folyamatmodell felhasználóbarát felülete, az `init` a folyamatfa és a szolgáltatások életciklusának felelőse, a daemonok pedig a hosszú életű háttérfeladatokat valósítják meg. A következő fejezet (POSIX, Windows API és hordozhatóság) ezekre az alapokra építve tárgyalja a POSIX, a Windows API és a hordozhatóság kérdéseit, ahol a shell, a daemonok és a szolgáltatásmenedzserek eltérő felületei közötti átjárás válik fontossá.

6.4 POSIX, Windows API és hordozhatóság

A hordozható rendszerprogramozás alapkérdése nem az, hogy „melyik operációs rendszer a jobb”, hanem az, hogyan tudunk úgy interfészt tervezni, hogy a programunk viselkedése előre jelezhető legyen több platformon. A POSIX és a Windows API két eltérő filozófiát képvisel: az előbbi a Unix-szerű rendszerek közös, szabványos rétegét adja, az utóbbi egyetlen operációs rendszer-család stabil, dokumentált natív felületét. Ebben az alfejezetben a két modellt, a köztük lévő hidakat, valamint a feltételes fordításra épülő absztrakciós technikákat vizsgáljuk.

6.4.1 1. Szabványos és platformspecifikus interfészek

Operációs rendszeri szempontból a programozási interfész (API) az a szerződés, amelyen keresztül a felhasználói térben futó kód a kernel szolgáltatásait eléri. Ez a szerződés több rétegből állhat: a processzor által kikényszerített privileged utasításokból, a kernel által exportált rendszerhívásokból, a libc vagy futásidejű könyvtár által nyújtott burkolófüggvényekből, valamint a nyelvi szabványkönyvtárakból.

Szabványos interfész alatt olyan, független szervezet által rögzített specifikációt értünk, amelyet több implementáció is követ. Ilyen a POSIX (IEEE 1003, illetve a Single UNIX Specification része), a C szabványkönyvtár (ISO/IEC 9899), valamint a hálózati programozásban gyakran hivatkozott BSD socket API. A szabványosítás célja, hogy a forráskód – ideális esetben – újrafordítás után más rendszeren is azonos szemantikával fusson.

Platformspecifikus interfész ezzel szemben egy adott operációs rendszerhez vagy gyártói ökoszisztémához kötődik. A Win32 API a Microsoft Windows rendszerek natív programozási felülete; a macOS-en a Cocoa és a Darwin alrendszer, a Linux kernelben pedig bizonyos, a POSIX-on túlmutató kiterjesztések (például `epoll`, `inotify`, `io_uring`) is platformspecifikusnak számítanak. A platformspecifikus API-k gyakran gazdagabbak, de használatuk csökkenti a forráskód átjárhatóságát.

A mérnöki dilemma tehát az, hogy meddig érdemes a szabványos rétegre támaszkodni, és hol kell platformfüggő kódot bevezetni. A válasz a projekt követelményeitől függ: egy beágyazott eszközön futó szoftvernél a determinizmus fontosabb lehet a forráshordozhatóságnál, míg egy szerveroldali szolgáltatásnál a konténeres telepítés és a CI-mátrix a hordozhatóságot kényszeríti ki.

6.4.2 2. POSIX: a Unix-szerű rendszerek közös nyelve

A POSIX (Portable Operating System Interface) szabványcsalád a nyolcvanas évek végétől kezdve az Unix-változatok közötti átjárhatóságot célozta. Az IEEE 1003.1 ma már a Single UNIX Specification része, és a legtöbb Unix-szerű rendszer – Linux, macOS, FreeBSD, Solaris, QNX –

deklaráltan POSIX-kompatibilis vagy ahhoz közeli. A POSIX nem csupán rendszerhívásokat ír le: tartalmazza a libc függvényeit, a shell nyelvet, a segédprogramokat (`ls`, `grep`, `awk`), a fájlrendszer-szemantikát, a szignálkezelést, a szálakat (POSIX Threads, pthread) és a folyamatok közötti kommunikációt is.

A rendszerhívási rétegben a POSIX a Unix-filozófiát tükrözi: „minden fájl”. A fájlműveletek a `open`, `read`, `write`, `close`, `lseek`, `fsync` függvényeken keresztül érhetők el, a folyamatkezelés pedig a `fork`, `exec*`, `wait*` családdal. A hálózati programozásban a socket API a `socket`, `bind`, `listen`, `accept`, `connect`, `send`, `recv` hívásokkal dolgozik. A szálkezelést a `pthread_create`, `pthread_join`, `pthread_mutex_lock` és társai biztosítják.

A hibakezelés a POSIX-rendszerekben egységes: a függvények többsége `-1` értékkel tér vissza hiba esetén, és a hiba pontos okát az `errno` globális (vagy szál-specifikus) változó tárolja. A `strerror(errno)` vagy a `perror` segítségével olvasható üzenetté alakíthatjuk. Példa egy minimális fájlíró programra:

```c
#include <fcntl.h>
#include <unistd.h>
#include <errno.h>
#include <string.h>
#include <stdio.h>

int main(void) {
    /* Emulált laborban a /tmp a konténer saját tmpfs-e. */
    const char *path = "/tmp/hello-posix.txt";
    int fd = open(path, O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (fd < 0) {
        fprintf(stderr, "open: %s\n", strerror(errno));
        return 1;
    }
    const char *msg = "hello from POSIX\n";
    size_t len = strlen(msg);
    ssize_t written = write(fd, msg, len);
    if (written < 0) {
        fprintf(stderr, "write: %s\n", strerror(errno));
        close(fd);
        return 1;
    }
    if (close(fd) != 0) {
        fprintf(stderr, "close: %s\n", strerror(errno));
        return 1;
    }
    return 0;
}
```

A `man 2 open`, `man 2 write` hivatkozások a Linux Programmer's Manual megfelelő szakaszaira mutatnak; a fenti függvények pontos paraméterezését és hibakódjait a rendszer dokumentációja tartalmazza. A POSIX szabvány egyes részei – például a valós idejű kiterjesztések, az aszinkron I/O vagy a folyamatütemezési opciók – opcionálisak, ezért a hordozható kódnak mindig ellenőriznie kell, hogy az adott makró (`_POSIX_ASYNCHRONOUS_IO`, `_POSIX_THREADS` stb.) definiált-e.

A POSIX erőssége a széles körű elterjedtség és a stabil szemantika. Korlátai közé tartozik, hogy nem fed le minden modern kernelképességet; a Linux specifikus `epoll`, `signalfd`, `timerfd` vagy `io_uring` interfészek már platformfüggő kódréteget igényelnek. Emellett a POSIX implementációk között is vannak eltérések – például a fájlzárolás szemantikája vagy a szignálok kézbesítési sorrendje –, ezért a „POSIX-kompatibilis” címke nem garantálja a bitre azonos viselkedést.

6.4.3 3. Win32 API: a Windows natív felülete

A Windows NT családban a kernel és a felhasználói tér közötti határ másképp épül fel. A natív rendszerhívások az `ntdll.dll`-en keresztül érhetők el, de a dokumentált, stabil programozási felületet a Win32 API (más néven Windows API) adja. Ez a felület a `kernel32.dll`, `user32.dll`, `advapi32.dll` és más rendszer-DLL-ekben található, és a C nyelvű fejlécek (`windows.h`) segítségével érhető el.

A Win32 API alapvető absztrakciója a **handle**: egy átlátszatlan, egész szám típusú azonosító, amely egy kernel-objektumra – fájlra, folyamatra, szálra, eseményre, mutexre – hivatkozik. A handle-t a megfelelő `Create*` vagy `Open*` függvény adja vissza, és a `CloseHandle` szabadítja fel. A fájlkezelés tipikus függvényei: `CreateFile`, `ReadFile`, `WriteFile`, `SetFilePointer`, `FlushFileBuffers`. Példa ugyanarra a fájlíró műveletre:

```
#include <windows.h>
#include <string.h>
#include <stdio.h>

int main(void) {
    /* Emulált Windows VM-ben a C:\Temp a VM saját lemeze. */
    const char *path = "C:\\Temp\\hello-win32.txt";
    HANDLE h = CreateFileA(path,
                           GENERIC_WRITE,
                           0,                  /* nincs megosztás */
                           NULL,               /* alap biztonsági leíró */
                           CREATE_ALWAYS,
                           FILE_ATTRIBUTE_NORMAL,
                           NULL);
    if (h == INVALID_HANDLE_VALUE) {
        fprintf(stderr, "CreateFileA failed: %lu\n", GetLastError());
        return 1;
    }
    const char *msg = "hello from Win32\n";
    DWORD written = 0;
    if (!WriteFile(h, msg, (DWORD)strlen(msg), &written, NULL)) {
        fprintf(stderr, "WriteFile failed: %lu\n", GetLastError());
        CloseHandle(h);
        return 1;
    }
    CloseHandle(h);
    return 0;
}
```

A Win32 hibakezelés eltér a POSIX-tól: a függvények általában `BOOL` értékkel térnek vissza (`TRUE`/`FALSE`), a tényleges hibakódot pedig a `GetLastError()` hívással kérdezhetjük le. A hibakód formátuma a rendszer által definiált `DWORD`, amelyet a `FormatMessage` funkcióval alakíthatunk szöveggé.

A Windows API szoros kapcsolatban van a Unicode támogatással: a legtöbb szöveges függvénynek létezik `A` (ANSI, azaz a rendszer aktuális kódlapja szerinti) és `W` (wide, UTF-16) változata. A makrók (`CreateFile` a `_UNICODE` definíciótól függően `CreateFileW`-re vagy `CreateFileA`-ra oldódik) lehetővé teszik a forráskód egyszerű váltását. A modern Windows rendszerek belsőleg UTF-16-ot használnak, ezért új kódban a `W` változatok előnyben részesítendők.

A Win32 API előnye a stabilitás: a Microsoft komoly kompatibilitási garanciákat vállal, a dokumentált függvények viselkedése évtizedeken át megmarad. Hátránya viszont a platformkötöttség: a Win32 kód nem fordul le más operációs rendszeren natívan, és a szemantika – például a fájlleírók öröklése, a szignálkezelés, a folyamatindítás – jelentősen eltér a Unix-modelltől.

6.4.4 4. .NET és kompatibilitási rétegek

A modern, keresztplatformos fejlesztés gyakran nem közvetlenül a natív API-kat használja, hanem egy magasabb szintű keretrendszert. A .NET (korábban .NET Core, ma .NET 6/8/9 LTS verziók – az aktuális támogatott verziót mindig a hivatalos kiadási jegyzékből kell ellenőrizni) egy managed futásidejű környezetet ad, amely a Base Class Library-n (BCL) keresztül egységes absztrakciót kínál a fájlkezelésre, hálózatra, szálakra és sok más területre. A `System.IO.File.WriteAllText` hívás ugyanazt a műveletet végzi el Linuxon, Windowson és macOS-en, a mögöttes natív hívást a .NET runtime fordítja le a megfelelő platform API-ra.

A .NET lehetőséget ad a natív kód elérésére is a Platform Invoke (P/Invoke) mechanizmussal. Ilyenkor a fejlesztő explicit módon deklarálja a külső függvényt és a hívási konvenciót:

```
using System;
using System.IO;
using System.Runtime.InteropServices;

class Program
{
    [DllImport("libc", SetLastError = true)]
    private static extern int getpid();

    static void Main()
    {
        string path = Path.Combine(Path.GetTempPath(), "hello-dotnet.txt");
        File.WriteAllText(path, "hello from .NET\n");
        Console.WriteLine($"pid={getpid()}");
    }
}
```

A P/Invoke erőteljes, de megtöri a platformfüggetlenséget: a `libc` elnevezés Linuxon `libc.so.6`, macOS-en `libSystem.B.dylib`, Windowson pedig nem is létezik ilyen függvény. Ezért a valóban hordozható .NET kód a BCL-re támaszkodik, és csak jól elkülönített modulokban használ P/Invoke-ot.

A **kompatibilitási rétegek** célja, hogy az egyik platform programjait egy másik platformon futtassák. A legismertebbek:

Réteg	Működés	Tipikus használat	Korlátozás
Cygwin	POSIX API-t biztosító DLL Windows felett, a POSIX hívásokat Win32-re fordítja	Unix eszközök futtatása Windowson	Lassabb, a fájlrendszer-szemantika eltér
MSYS2	Cygwin alapú, csomagkezelővel ellátott környezet	Fordítási környezet Windowson	Hasonló korlátok
Wine	Win32 API implementáció Unix-szerű rendszereken	Windows programok futtatása Linuxon	Nem minden API tökéletes, a teljesítmény változó
WSL1	Linux rendszerhívás-fordítás Windows kernel fölé	Linux binárisok Windowson	Néhány syscall hiányos, a teljesítmény korlátozott

Réteg	Működés	Tipikus használat	Korlátozás
WSL2	Valódi Linux kernel virtuális gépen, integrált fájlrendszerrel	Teljes Linux környezet Windowson	Virtualizációs réteg, erőforrás-igény

A kompatibilitási rétegek hasznosak fejlesztéshez és migrációhoz, de nem helyettesítik a natív tesztelést. Egy hordozhatósági mátrixban a rétegek eredményeit külön kell jelölni, mert a szimulált környezetben mért értékek nem feltétlenül azonosak a natív rendszeren kapott értékekkel.

6.4.5 5. Absztrakció és feltételes fordítás

A C nyelvű rendszerprogramozásban a hordozhatóság klasszikus eszköze a **feltételes fordítás**. A preprocesszor `#ifdef`, `#if defined` direktíváival a forráskód különböző részeit fordíthatjuk platformtól függően. A legegyszerűbb minta egy közös interfész, amely mögött platformonként eltérő implementáció áll.

Tegyük fel, hogy egy `portable_write_file` függvényt akarunk, amely adott útvonalon létrehoz egy fájlt, és kiír egy bájtsorozatot. A közös fejléc (`portable_file.h`) így nézhet ki:

```
#ifndef PORTABLE_FILE_H
#define PORTABLE_FILE_H

#include <stddef.h>

/* Visszatérési érték: 0 siker, -1 hiba. A hiba szöveges leírását
   a hívó a platformfüggő hibakód alapján kérheti le. */
int portable_write_file(const char *path, const void *data, size_t len);

#endif
```

A POSIX implementáció (`portable_file_posix.c`):

```
#include "portable_file.h"
#include <fcntl.h>
#include <unistd.h>
#include <errno.h>

int portable_write_file(const char *path, const void *data, size_t len) {
    int fd = open(path, O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (fd < 0) return -1;
    ssize_t written = write(fd, data, len);
    if (written < 0 || (size_t)written != len) {
        close(fd);
        return -1;
    }
    if (close(fd) != 0) return -1;
    return 0;
}
```

A Windows implementáció (`portable_file_win.c`):

```
#include "portable_file.h"
#include <windows.h>

int portable_write_file(const char *path, const void *data, size_t len) {
    HANDLE h = CreateFileA(path, GENERIC_WRITE, 0, NULL,
```

```
                           CREATE_ALWAYS, FILE_ATTRIBUTE_NORMAL, NULL);
    if (h == INVALID_HANDLE_VALUE) return -1;
    DWORD written = 0;
    if (!WriteFile(h, data, (DWORD)len, &written, NULL) ||
        written != (DWORD)len) {
        CloseHandle(h);
        return -1;
    }
    CloseHandle(h);
    return 0;
}
```

A build rendszer (például CMake) választja ki a megfelelő forrásfájlt:

```
cmake_minimum_required(VERSION 3.16)
project(portable_file C)

add_library(portable_file STATIC portable_file.c)
if(WIN32)
    target_sources(portable_file PRIVATE portable_file_win.c)
else()
    target_sources(portable_file PRIVATE portable_file_posix.c)
endif()
```

A `portable_file.c` csupán a közös fejlécet használó kódot tartalmazza; maga a függvény a platformfüggő forrásfájlban van. Így a platformfüggő kód fizikailag is elkülönül, és a hordozható részben nincs `#ifdef`-hegy.

A feltételes fordításnál gyakori hibák:

- **Elérési útvonalak eltérése**: a Windows hagyományosan `\` jelet és meghajtóbetűjelet használ, a Unix `/`-et. A `Path.Combine` (.NET) vagy a `boost::filesystem` használata csökkenti a hibát, de natív C kódban explicit konverzió kell.
- **Endianness**: a hálózati bájtsorrend (`htons`, `ntohl`) platformfüggetlen, de a bináris fájlformátumoknál a gépi bájtsorrend csapdát jelent.
- **Hibakódok**: a POSIX `errno`, a Windows `GetLastError`, a .NET kivételek – három külön modell. A platformrétegnek egységes hibakódot kell visszaadnia.
- **Szignálok és aszinkron események**: a POSIX `signal`/`sigaction` és a Windows `SetConsoleCtrlHandler` eltérő; a valós idejű viselkedés is más.
- **Fájlleírók és handle-ök**: a POSIX fájlleíró egy kis egész szám, a Windows handle átlátszatlan. A `fdopen`/`_open_osfhandle` hidak léteznek, de a szemantika eltérhet.
- **Szálak**: a `pthread_create` és a `CreateThread` paraméterezése, a TLS kezelése, a megszakítási pontok mind különböznek.

A tiszta megoldás az, ha a platformfüggő részt egy vékony rétegbe zárjuk, és a felett egyetlen, jól tesztelt absztrakciót használunk. Ez a réteg lehet C függvénykönyvtár, C++ osztály, vagy akár egy IPC-alapú szolgáltatás, ha a platformok folyamatmodellje nagyon eltér.

6.4.6 6. Példa: hordozható fájlművelet

A fenti `portable_write_file` függvény már demonstrálja a minimális platformréteget. Bővítsük egy másik művelettel, a fájl tartalmának beolvasásával, hogy lássuk a hibakezelés és a pufferkezelés különbségeit.

A közös függvény:

```
int portable_read_file(const char *path, void *buf, size_t *len);
```

A `len` bemenetként a puffer méretét adja, kimenetként a ténylegesen beolvasott bájtok számát. A POSIX implementáció a `open`/`read`/`close` hármast használja, a Windows a `CreateFileA`/`ReadFile`/`CloseHandle` hármast. A különbség a hibakezelésben és a `ReadFile` `DWORD` típusában mutatkozik meg.

Egy ellenőrzött példaprogram mindkét platformon:

```
#include "portable_file.h"
#include <stdio.h>
#include <string.h>

int main(void) {
    const char *path =
#ifdef _WIN32
        "C:\\Temp\\portable-test.bin";
#else
        "/tmp/portable-test.bin";
#endif
    const char *msg = "portable data\n";
    if (portable_write_file(path, msg, strlen(msg)) != 0) {
        fprintf(stderr, "write failed\n");
        return 1;
    }
    char buf[64];
    size_t len = sizeof(buf);
    if (portable_read_file(path, buf, &len) != 0) {
        fprintf(stderr, "read failed\n");
        return 1;
    }
    printf("read %zu bytes: %.*s", len, (int)len, buf);
    return 0;
}
```

A build parancs Linux alatt, emulált környezetben:

```
cc -std=c11 -Wall -Wextra -O2 portable_file.c portable_file_posix.c \
   -o portable-test
./portable-test
```

Windows alatt (MSVC vagy MinGW) hasonló, de a `portable_file_win.c` forrásfájllal. Az ilyen példák futtatásához a laborban QEMU vagy virtuális gép áll rendelkezésre; a hoszt valódi lemezét és rendszerét nem módosítjuk.

6.4.7 7. Labor: hordozhatósági mátrix készítése

A fejezet laborjának célja egy **hordozhatósági mátrix** előállítása, amely rögzíti, hogy egy kiválasztott API-művelet hogyan viselkedik különböző környezetekben. A mátrix nemcsak a sikerességet, hanem a visszatérési értéket, a hibakódot, a futási időt és a megjegyzéseket is tartalmazza.

6.4.8 7.1. Kiinduló állapot és konfiguráció

A labor a következő emulált környezeteket használja:

- **Linux natív**: QEMU/KVM virtuális gép, 4 vCPU, 8 GiB RAM, kernel 6.1 LTS, glibc 2.37, build: GCC 12.2, `-O2`.
- **Windows natív**: Windows Server 2022 VM, 4 vCPU, 8 GiB RAM, MSVC 19.3, `-O2`, x64.

- **WSL2**: ugyanaz a Linux kernel, de Windows gazdán, 4 vCPU, 8 GiB RAM, glibc 2.35.
- **Wine**: Wine 8.0, Linux gazdán, a Windows EXE futtatása.
- **Cygwin**: Cygwin 3.4, a POSIX program futtatása Windows VM-en.
- **.NET**: .NET 8.0 LTS (a verzió a hivatalos kiadási jegyzékből ellenőrizendő), Linux és Windows fölött.

A mérési módszer: minden környezetben egy shell script vagy batch fájl futtatja a tesztprogramot tízszer, és rögzíti a kimenetet, a visszatérési kódot és a `time` (Linux) vagy `Measure-Command` (PowerShell) által mért időt. A pontos parancsok a reprodukciós csomagban szerepelnek.

6.4.9 7.2. A tesztprogram

A tesztprogram három műveletet hajt végre:

1. `create_write`: létrehoz egy fájlt, kiír 1 MiB adatot, bezárja.
2. `read_verify`: megnyitja a fájlt, visszaolvassa, és ellenőrzi a tartalmat.
3. `delete`: törli a fájlt.

A program C-ben készül el, két változatban: az egyik a POSIX API-t használja, a másik a Win32 API-t. A forráskód a `portable_file` könyvtárra épül, így mindkét implementáció ugyanazt a közös felületet valósítja meg. A .NET változat a BCL `File` osztályát használja.

6.4.10 7.3. Mérési napló és mátrix

A mátrix minden sorában egy környezet szerepel, az oszlopokban a műveletek. Minden cella két értéket tartalmaz: a **várt** eredményt (a specifikációból levezetett) és a **mért** eredményt (a laborban kapott). Példa egy részleges naplóra:

Környezet	create_write	read_verify	delete	Futási idő (10 iteráció)	Megjegyzés
Linux natív (POSIX)	várt: 0, mért: 0	várt: 0, mért: 0	várt: 0, mért: 0	12 ms	átlag, tmpfs
Windows natív (Win32)	várt: TRUE, mért: TRUE	várt: TRUE, mért: TRUE	várt: TRUE, mért: TRUE	18 ms	NTFS, AV bekapcsolva
WSL2 (POSIX)	várt: 0, mért: 0	várt: 0, mért: 0	várt: 0, mért: 0	25 ms	hálózati fájlrendszer overhead
Wine (Win32)	várt: TRUE, mért: TRUE	várt: TRUE, mért: TRUE	várt: TRUE, mért: TRUE	145 ms	teljesítmény-csökkenés
Cygwin (POSIX)	várt: 0, mért: 0	várt: 0, mért: 0	várt: 0, mért: 0	38 ms	Windows fájlrendszer szemantika
.NET 8 Linux	várt: nincs kivétel, mért: nincs	várt: nincs kivétel, mért: nincs	várt: nincs kivétel, mért: nincs	15 ms	managed overhead minimális
.NET 8 Windows	várt: nincs kivétel, mért: nincs	várt: nincs kivétel, mért: nincs	várt: nincs kivétel, mért: nincs	20 ms	hasonló

A táblázatban a „várt” érték a dokumentált API-szerződésből következik: a POSIX `open/write/unlink` siker esetén 0-t ad, a Win32 `CreateFile/WriteFile/DeleteFile` `TRUE` értéket. A „mért” érték a laborban ténylegesen megfigyelt visszatérési érték. A futási idő esetén a mérési módszer: 10 egymás utáni futtatás teljes ideje, a táblázatban a medián látható. A részletes napló minden futtatás kimenetét, a hibakódot és a `perf stat` (Linux) vagy `Measure-Command` adatait is tartalmazza.

6.4.11 7.4. Kiértékelés és tipikus hibák

A labor során gyakori csapdák:

- **Elérési út problémák**: a Windows `C:\Temp` útvonalat vár, a Linux `/tmp`-et. Ha a kód nem használ platformfüggő útvonalválasztást, a teszt rossz könyvtárban próbálkozik.
- **Fájlmegosztási mód**: a Win32 `CreateFile` harmadik paramétere a megosztási mód; ha 0-t adunk, más folyamat nem nyithatja a fájlt. A POSIX `open` alapértelmezetten megengedi a megosztást. Ez a különbség holtversenyt okozhat, ha a teszt párhuzamosan fut.
- **Hibakód félreértelmezése**: a POSIX `errno` értéke a hívás után azonnal érvényes, de egy következő könyvtári hívás felülírhatja. A Win32-ben a `GetLastError` hasonlóan azonnal hívandó.
- **Puffer típusa**: a `WriteFile` `DWORD` méretet vár, a POSIX `write` `size_t`-t. A 32 bites rendszereken a 64 bites méret csonkolódhat, ha nem figyelünk.
- **Kompatibilitási réteg téves értékelése**: Wine alatt a hibakódok és a teljesítmény eltérhet a natív Windowstól; ezt a mátrixban külön kell jelölni, nem szabad a natív eredménnyel összekeverni.

A labor eredménye a reprodukálható jegyzetfüzet (például Jupyter notebook vagy Markdown + parancsnaplók), amely tartalmazza a build parancsokat, a pontos konfigurációt, a mért és várt értékeket elkülönítve, valamint a következtetéseket. Az átadandó mátrix a fenti táblázathoz hasonló, de a cellákban a hibakódok és a mérési idők is szerepelnek.

6.4.12 8. Összefoglalás és önellenőrző kérdések

Ebben az alfejezetben áttekintettük a szabványos (POSIX) és platformspecifikus (Win32) interfészeket, a .NET absztrakciót, valamint a kompatibilitási rétegeket. Bemutattuk a feltételes fordításra épülő platformréteg mintáját, és egy hordozhatósági mátrix laborral demonstráltuk a gyakorlati ellenőrzés módszerét. A következő fejezet (Folyamatmodell, PCB és állapotátmenetek) a folyamatmodellre és a folyamatvezérlő blokkra épít, ahol a `fork/exec` és a `CreateProcess` közötti eltéréseket is részletesen tárgyaljuk.

Önellenőrző kérdések:

1. Mi a különbség a POSIX `errno` és a Win32 `GetLastError` hibakezelési modellje között? Adj példát mindkettőre.
2. Hogyan oldanád meg, hogy egy C program ugyanazt a műveletet (például könyvtárlistázást) Linuxon és Windowson is elvégezze anélkül, hogy a fő logika `#ifdef`-ekkel lenne tele?
3. Melyik kompatibilitási réteget választanád egy Unix-szerű build pipeline Windowson való futtatásához, és miért? Milyen korlátokra kell figyelni?
4. A hordozhatósági mátrixban miért kell a várt és a mért értéket külön oszlopban szerepeltetni? Milyen következménye van, ha ezt elmulasztjuk?

A kérdésekre adott válaszok a fejezet fogalmi része alapján ellenőrizhetők; a labor jegyzőkönyvében a mért értékeket a saját környezetben kell reprodukálni.

II. RÉSZ – Folyamatok, szálak és ütemezés

7 Folyamatok és szálak

7.1 Folyamatmodell, PCB és állapotátmenetek

A folyamat az operációs rendszer egyik legalapvetőbb absztrakciója: a rendszer úgy tekinti, mintha minden program saját processzorral, saját címtérrel és saját erőforrás-készlettel rendelkezne. Ez az illúzió teszi lehetővé, hogy egyetlen fizikai gépen több program is egyidejűleg, egymástól látszólag függetlenül fusson. A folyamatmodell és annak állapotgépe az a fogalmi keret, amelyre az ütemezés, a szinkronizáció és a memóriakezelés épül. Ebben az alfejezetben a folyamatot mint végrehajtási egységet definiáljuk, bemutatjuk a process control block (PCB) szerkezetét és a folyamatok lehetséges állapotait, majd egy reprodukálható laboron keresztül gyakorlati példát adunk az állapotátmenetek megfigyelésére.

7.1.1 1. A folyamat mint végrehajtási egység

A folyamat (process) a futó program megtestesítője. Amíg a program egy passzív, lefordított utasítássorozat, addig a folyamat annak aktív végrehajtása, amelyhez az operációs rendszer erőforrásokat rendel: processzort, memóriát, fájlleírókat, nyitott eszközöket és egyéb állapotot. Pontosabban a folyamat a következő elemekből áll:

- **Futtatható kód (executable code)**: a program gépi utasításai. Ezt gyakran *text* szegmensnek nevezik, amely általában írásvédett, hogy a véletlen módosítás ne okozzon hibát.
- **Adatterület (data)**: a globális és statikus változók, amelyek inicializált és inicializálatlan részből állnak. Az inicializált rész a *data* szegmens, a nullára inicializált vagy inicializálatlan rész pedig a *BSS* szegmens. Az utóbbi gyakran csak méretként létezik a futtatható állományban, és a betöltéskor nullázódik.
- **Verem (stack)**: a függvényhívásokhoz, lokális változókhoz és visszatérési címekhez használt memóriaterület. A verem általában lefelé növekszik a címtérben, és minden szálhoz külön verem tartozik.
- **Heap**: a dinamikus memóriafoglalások területe, amely a futás során a `malloc`/`free` (vagy más allokátor) segítségével növekszik és zsugorodik.
- **Erőforrás-leírók (resource descriptors)**: fájlleírók, socketek, csővezetékek, nyitott eszközök, memóriatérképezések stb. Ezek a kernel által kezelt referencia-számlált objektumok.
- **Szál(ak) végrehajtási állapota**: a program által használt regiszterek, az utasításmutató (program counter, PC), a veremmutató (stack pointer, SP) és a státuszregiszter.

Mindezt együtt *process image*-nek nevezzük. A folyamat virtuális címtérrel rendelkezik, amelyben a szegmensek meghatározott címeken helyezkednek el. A 32 bites rendszereken a címtér tipikusan 4 GiB, amelyből a felhasználói tér jellemzően az alsó 3 GiB, a kernel pedig a felső 1 GiB. A 64 bites rendszereken ennél jóval nagyobb, de a felosztás elve hasonló. A virtuális címek és a fizikai címek közötti leképezést a memóriakezelő egység (MMU) végzi, amit a későbbi fejezet részletez.

A folyamat nem csupán a memóriaképet jelenti; magában foglalja a futás során felhalmozott metaadatokat is. Például a folyamat azonosítója (PID), a szülő folyamat azonosítója (PPID),

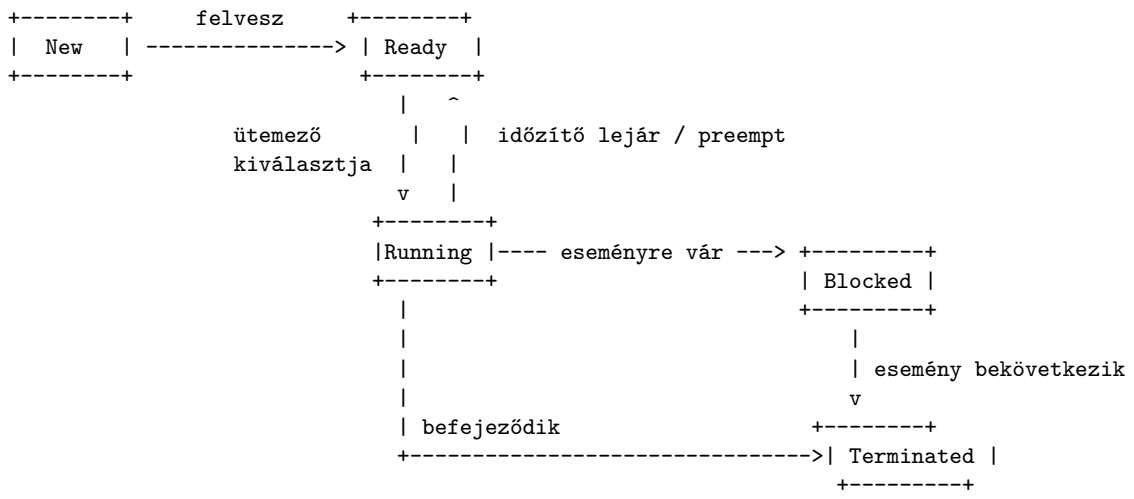
a felhasználói és csoportazonosítók, a futási idő, az erőforrás-korlátok (rlimits), a prioritás és az ütemezési osztály mind a folyamathoz tartoznak. Ezt az információt a kernel egy külön adatszerkezetben, a process control blockban tartja nyilván.

7.1.2 2. A folyamat állapotgépe

Egy folyamat életciklusa során több állapoton megy keresztül. Bár a konkrét operációs rendszerek finomabb állapotokat is megkülönböztetnek, az általános modell öt alapállapotot definiál:

1. **New (létrehozás alatt)**: A folyamat létrehozása folyamatban van, de még nem áll készen a futtatásra. Ebben az állapotban a kernel lefoglalja a PCB-t, inicializálja a folyamathoz tartozó adatszerkezeteket, betölti a futtatható állomány részeit, és beállítja a memóriatérképet. A folyamat még nem versenyez a processzorért.
2. **Ready (futásra kész)**: A folyamat minden erőforrással rendelkezik ahhoz, hogy fusson, de éppen nem kap processzort, mert más folyamat használja azt, vagy az ütemező még nem választotta ki. A ready állapotú folyamatok a futásra kész sorban (ready queue) várakoznak, és az ütemező dönti el, mikor kapnak CPU-időt.
3. **Running (futó)**: A folyamat éppen a processzoron hajtódik végre, azaz az utasításait a CPU ténylegesen végrehajtja. Egyprocesszoros rendszeren egyszerre legfeljebb egy folyamat lehet running állapotban. Többmagos rendszeren ez a szám a magok számáig terjedhet.
4. **Blocked (blokkolt/várakozó)**: A folyamat nem tud tovább futni, mert valamilyen eseményre vár: például egy I/O-művelet befejezésére, egy zárolásra, egy időzítő lejáratára vagy egy jelre. Amíg az esemény be nem következik, a folyamat nem kaphat processzort, még akkor sem, ha a CPU szabad. A blokkolt folyamat a várakozási sorban (wait queue) tartózkodik.
5. **Terminated (befejezett)**: A folyamat befejezte a futását, vagy a kernel, illetve a felhasználó megszakította azt. Ebben az állapotban a kernel felszabadítja az erőforrásokat, de a PCB egy ideig még megmaradhat, hogy a szülő folyamat lekérdezhesse a kilépési állapotot (exit status). Ezt nevezik *zombie* állapotnak. Miután a szülő ezt feldolgozta (például a `wait` rendszerhívással), a PCB teljesen felszabadul.

Az állapotok közötti átmeneteket az alábbi ábra szemlélteti:



Ez az öt állapot elegendő a legtöbb jelenség magyarázatához. A gyakorlatban azonban a kernel belső állapotai ennél finomabbak. Például a Linux a `TASK_RUNNING` állapotot használja mind a futó, mind a futásra kész folyamatra; a blokkolt állapotot pedig tovább osztja `TASK_INTERRUPTIBLE`, `TASK_UNINTERRUPTIBLE`, `TASK_KILLABLE` és egyéb altípusokra. Az alábbi táblázat a hagyományos modell és a Linux-kernel állapotneveinek hozzávetőleges megfeleltetését mutatja be. (A pontos mezők és nevek implementációfüggők, ezért a tananyagban az egyszerű modellt használjuk.)

Általános állapot	Linux kernel állapot (példa)	Leírás
New	–	A létrehozás alatt álló folyamat, mielőtt `TASK_RUNNING`-ba kerül
Ready	`TASK_RUNNING` (futásra kész sorban)	Futásra kész, de nincs CPU-hoz rendelve
Running	`TASK_RUNNING` (éppen a CPU-n)	Aktívan fut
Blocked	`TASK_INTERRUPTIBLE`, `TASK_UNINTERRUPTIBLE`, `TASK_KILLABLE`	Várakozik egy eseményre (megszakítható, nem megszakítható, killable)
Terminated	`TASK_ZOMBIE`, majd felszabadul	Befejezett, de a szülő még nem olvasta ki az exit státuszt

Az állapotátmenetek okai a következők:

- **New → Ready**: a kernel befejezte a folyamat inicializálását, és a folyamatot behelyezte a futásra kész sorba.
- **Ready → Running**: az ütemező kiválasztja a folyamatot futtatásra, és kontextusváltást hajt végre.
- **Running → Ready**: a folyamat időzítője (time slice) lejár, vagy egy magasabb prioritású folyamat válik futásra késszé, és a preemptív ütemező elveszi a CPU-t a futó folyamattól.
- **Running → Blocked**: a folyamat olyan rendszerhívást hajt végre, amely várakozást igényel (például `read` egy üres csővezetéken, `sleep`, `wait` egy záron), és az erőforrás nem érhető el azonnal.
- **Blocked → Ready**: a várt esemény bekövetkezik (például I/O befejeződik, időzítő lejár, zár felszabadul), és a kernel a folyamatot visszahelyezi a futásra kész sorba.
- **Running → Terminated**: a folyamat kilép (például a `exit` rendszerhívással), vagy egy végzetes jel érkezik hozzá (például `SIGKILL`).

Fontos megérteni, hogy a Blocked és a Ready állapot közötti különbség az, hogy a blokkolt folyamat külső eseményre vár, és nem kaphat CPU-t, amíg az esemény be nem következik; a ready folyamat pedig minden egyéb feltételnek megfelel, csak a processzorra vár. Ez a megkülönböztetés alapvető az ütemezés és a szinkronizáció szempontjából.

7.1.3 3. A folyamatvezérlő blokk (PCB)

A process control block (PCB) – más néven task struct, process descriptor vagy egyszerűen process table entry – az a kernel által fenntartott adatszerkezet, amely egy folyamat összes metaadatát tárolja. A PCB a folyamat teljes életciklusa alatt létezik, és a kernel a folyamat azonosítóján keresztül hivatkozik rá. Amikor a folyamat befejeződik, a PCB-t a kernel felszabadítja.

A PCB pontos felépítése operációs rendszerenként és kernelverziónként eltérő, de a lényegi mezők a következő csoportokba sorolhatók:

- **Azonosítás (identification)**:
  - Process ID (PID): egyedi, a folyamathoz rendelt egész szám. A PID-ek újrahasznosíthatók, de egy adott időpontban egyedinek kell lenniük.
  - Parent Process ID (PPID): a szülő folyamat azonosítója, amely a folyamatot létrehozta.
  - Valós és effektív felhasználói/csoportazonosítók (UID, EUID, GID, EGID), amelyek a hozzáférés-ellenőrzés alapját képezik.
- **Végrehajtási állapot (execution state)**:
  - A processzor regisztereinek mentése: általános célú regiszterek, utasításmutató (PC), veremmutató (SP), státuszregiszter, lebegőpontos regiszterek stb. Ezt a kontextust a kontextusváltáskor menti és tölti vissza a kernel.
  - A folyamat állapota (new, ready, running, blocked, terminated).
  - A folyamat prioritása és ütemezési paraméterei: nice érték, ütemezési osztály, CPU-affinitás, time slice hossza.
- **Memóriakezelési adatok (memory management)**:
  - A lapfelszíntábla (page table) mutatója, amely a folyamat virtuális címtér-leképezését írja le.
  - A szegmensek határai (code, data, BSS, heap, stack).
  - Memóriahasználati statisztikák, például a lefoglalt lapok száma, a resident set size (RSS), a virtuális méret (VSZ).
- **Erőforrás-leírók (resource descriptors)**:
  - Fájlleíró tábla: a folyamat által megnyitott fájlok, csővezetékek, socketek és eszközök leírói.
  - Nyitott könyvtárak, aktuális munkakönyvtár, gyökérkönyvtár (chroot).
  - Jelmaszk (signal mask) és a függő jelek halmaza.
  - Erőforrás-korlátok (pl. maximális fájlméret, maximális memória, maximális processzorszám).
- **Könyvelési és statisztikai adatok (accounting)**:
  - A folyamat által felhasznált CPU-idő (user és kernel módban külön).
  - A folyamat indulásának időpontja, a futási idő.
  - A generált I/O-műveletek száma, a laphibák száma stb.
- **Kapcsolódási információk (linkage)**:
  - Mutatók a folyamatok listájában (pl. a futásra kész sor, a várakozási sor, a gyermek folyamatok listája).
  - A szülőre és a testvérekre mutató hivatkozások.

Egy egyszerűsített C nyelvű PCB-struktúra a következőképpen nézhet ki (a példa csak szemléltetés, nem egy konkrét kernel kódja):

```
/* Egyszerűsített PCB - nem valós kernel adatszerkezet */
typedef enum {
    PROCESS_NEW,
    PROCESS_READY,
    PROCESS_RUNNING,
    PROCESS_BLOCKED,
    PROCESS_TERMINATED
} process_state_t;

struct process_control_block {
```

```
    /* Azonosítás */
    pid_t pid;
    pid_t ppid;
    uid_t euid;
    gid_t egid;

    /* Végrehajtási állapot - a kontextusváltáskor mentett regiszterek */
    uintptr_t pc;
    uintptr_t sp;
    uintptr_t regs[16];     /* általános célú regiszterek, architektúrafüggő */
    uint32_t flags;
    process_state_t state;

    /* Ütemezési adatok */
    int priority;
    int nice;
    uint64_t time_slice;
    uint64_t total_cpu_time;

    /* Memóriakezelés */
    uintptr_t page_table_base;   /* pl. CR3 x86-on vagy SATP RISC-V-n */
    size_t code_size;
    size_t data_size;
    size_t bss_size;
    size_t heap_break;
    size_t stack_base;
    size_t stack_size;

    /* Erőforrás-leírók */
    int fd_table[256];           /* fájlleírók száma és típusa egyszerűsítve */
    uint32_t signal_mask;
    uint32_t pending_signals;

    /* Könyvelés */
    uint64_t start_time;
    uint64_t kernel_time;
    uint64_t user_time;
    uint64_t page_fault_count;

    /* Kapcsolódások */
    struct process_control_block *parent;
    struct list_head children;   /* gyermek folyamatok listája */
    struct list_head ready_link; /* futásra kész sor eleme */
    struct list_head wait_link;  /* várakozási sor eleme */
};
```

A valódi kernel struktúrák ennél jóval összetettebbek, és a portabilitás érdekében absztrakciós rétegeket használnak. A fenti példa azonban jól mutatja, hogy a PCB a folyamat minden lényeges állapotát egy helyen tárolja. Amikor az ütemező úgy dönt, hogy egy másik folyamatot futtat, a kernel elmenti az aktuális folyamat regisztereit a PCB-jébe, majd visszatölti az új folyamat PCB-jéből a regisztereket, és frissíti a memóriakezelő egységet. Ez a kontextusváltás, amelyet a következő alfejezet részletesen tárgyal.

A PCB-k kezelése kritikus a kernel teljesítménye és biztonsága szempontjából. A PCB-hez csak a kernel férhet hozzá közvetlenül; a felhasználói programok rendszerhívásokon keresztül kérdezhetnek le bizonyos adatokat (például a `getpid` rendszerhívással a saját PID-jüket, vagy a Linux `/proc` virtuális fájlrendszerén keresztül a folyamatok állapotát). A PCB memóriaterületének védelme alapvető: ha egy felhasználói folyamat írhatná, az egész rendszer integritása sérülne.

7.1.4 4. Labor: folyamatállapotok megfigyelése emulált környezetben

A következő labor célja, hogy a hallgató saját szemével lássa a folyamat állapotainak váltakozását, és méréssel igazolja az elméleti modellt. A labor egy emulált Linux-környezetben fut (QEMU vagy más virtualizációs megoldás), és egy egyszerű C programot, valamint egy megfigyelő szkriptet használ.

7.1.5 4.1 Kiinduló állapot és konfiguráció

- **Emulált hardver**: QEMU 8.0 vagy újabb, egyetlen virtuális CPU, 512 MiB RAM, egy IDE merevlemez és egy soros konzol.
- **Operációs rendszer**: egy minimalista Linux disztribúció (például BusyBox-alapú initramfs vagy egy saját, korábban elkészített kernel), amelyet a laborhoz mellékelt lemezkép tartalmaz. A kernel verziója 6.1, de a pontos verzió a reprodukciós csomagban rögzített.
- **Fordító és eszközök**: GCC 12.2, binutils 2.40, GNU Make 4.4, Python 3.11 a mérési napló készítéséhez.
- **Helyőrző megjegyzés**: a labor nem használ valós hardvert, és a lemezkép csak emulációban fut. Minden módosítás az emulált gépre korlátozódik.

A labor kiinduló állapota egy frissen bootolt rendszer, amelyen csak a szükséges eszközök és a forráskód találhatók. A hallgató a `labos022/` könyvtárban dolgozik.

7.1.6 4.2 A minta program

Készítsünk egy C programot, amely a futása során különböző állapotokat váltogat:

```
/* process_states.c - demonstrációs program */
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <signal.h>

volatile sig_atomic_t stop = 0;

void handle_signal(int sig) {
    stop = 1;
}

int main(void) {
    signal(SIGUSR1, handle_signal);

    printf("PID: %d\n", getpid());
    printf("Nyeljük a CPU-t 5 másodpercig...\n");
    /* CPU-igényes ciklus: running állapot */
    volatile unsigned long long i, j;
    for (j = 0; j < 5; ++j) {
        for (i = 0; i < 100000000ULL; ++i) {
            /* foglalt */
        }
    }

    printf("Most blokkolunk 3 másodpercre (sleep)...\n");
    /* Blocked állapot: a folyamat nem fut, hanem az időzítőre vár */
    sleep(3);

    printf("Várunk egy SIGUSR1 jelre...\n");
    /* Blocked állapot: a folyamat jelre vár */
```

```
    while (!stop) {
        pause();  /* a folyamat blokkolódik, amíg jel nem érkezik */
    }

    printf("Kilépés...\n");
    return 0;
}
```

A program először öt másodpercig intenzíven használja a processzort, ami a futó állapotnak felel meg. Ezután három másodpercig alszik, ami blokkolt állapot (időzítőre vár). Végül addig várakozik egy `SIGUSR1` jelre, amíg azt kívülről meg nem kapja; ez is blokkolt állapot, de most jelre vár. A jel megszakítható várakozást eredményez (`TASK_INTERRUPTIBLE` a Linux terminológiájában).

7.1.7 4.3 A megfigyelő szkript

A folyamat állapotát a `/proc/<pid>/status` fájl `State` mezőjén keresztül lehet lekérdezni. A következő Python szkript 100 ms-os időközönként rögzíti a folyamat állapotát, és kimenti egy CSV naplóba:

```
#!/usr/bin/env python3
# monitor.py - folyamatállapot-figyelő
import time
import csv
import sys
import os

def read_state(pid):
    with open(f"/proc/{pid}/status", "r") as f:
        for line in f:
            if line.startswith("State:"):
                return line.split(":")[1].strip().split()[0]
    return "?"

def main():
    if len(sys.argv) != 2:
        print(f"Használat: {sys.argv[0]} <pid>", file=sys.stderr)
        sys.exit(1)
    pid = int(sys.argv[1])
    with open("process_states_log.csv", "w", newline="") as csvfile:
        writer = csv.writer(csvfile)
        writer.writerow(["timestamp", "state"])
        start = time.time()
        while True:
            state = read_state(pid)
            writer.writerow([f"{time.time() - start:.3f}", state])
            csvfile.flush()
            if state == "Z" or state == "?":
                break
            time.sleep(0.1)

if __name__ == "__main__":
    main()
```

A szkriptet egy második terminálon indítjuk el, miután a C program megadta a PID-jét. A megfigyelő futása alatt a folyamat állapotait rögzítjük.

7.1.8 4.4 Futtatás és napló

A labor futtatási parancsai (az emulált gépen, a `labos022/` könyvtárban):

```
# Fordítás
cc -O0 -g -o process_states process_states.c

# Elindítás az egyik terminálon
./process_states
# A program kiírja a PID-jét, pl. 1234.

# A második terminálon indítsuk a monitort, a PID-del
python3 monitor.py 1234

# Miután a program eléri a jelre váró szakaszt, küldjünk neki SIGUSR1 jelet:
# (a PID ismeretében, pl. 1234)
kill -SIGUSR1 1234
```

A megfigyelő napló egy lehetséges kimenete (a pontos időbélyegek a környezettől függenek, ezért a táblázat csak illusztráció):

idõ (s)	mért állapot	várt állapot
0.0	R	Running
0.1	R	Running
…	R	Running
5.1	S	Blocked
…	S	Blocked
8.0	S	Blocked
8.1	S	Blocked
8.5	S	Blocked
8.9	S	Blocked
9.0	Z	Terminated

A Linux `/proc/<pid>/status` állapotkódjai itt: `R` a futásra kész/futó, `S` a megszakítható alvás (blocked), `Z` a zombi (terminated, amíg a szülő ki nem olvassa). A megfigyelő a `Z` állapot észlelésekor leáll. A mérési naplóban a hallgató rögzíti a pontos értékeket, és összehasonlítja a várt értékekkel: a CPU-igényes szakaszban `R`, az alvásban `S`, a jelre váró szakaszban `S` (vagy `S+`, ha előtérben fut), végül `Z`. Az eltérések okait (pl. a monitor szkript ütemezési késése, a `sleep` pontossága) a hallgató elemzi.

7.1.9 4.5 Visszaállítás

A labor után a hallgató leállítja a megfigyelőt, megöli a folyamatot (ha még fut), és visszaállítja az emulált gépet a kiindulási állapotba a mellékelt szkripttel:

```
./reset_lab.sh
```

Ez a szkript törli a generált naplófájlokat, és visszaállítja a fájlrendszert a labor előtti állapotba. A módszer biztosítja, hogy a következő labor tiszta környezetben induljon.

7.1.10 5. Tipikus hibák és megoldások

A folyamatmodell és a PCB tanulása során a hallgatók gyakran esnek a következő hibákba:

1. **A Ready és a Running állapot összekeverése.** Sokan úgy gondolják, hogy a ready folyamat „majdnem fut”, de valójában a ready folyamat éppen nem fut, hanem a CPU-ra vár. A running állapotban a processzor ténylegesen a folyamat utasításait hajtja végre. Egyprocesszoros rendszeren egyszerre csak egy folyamat lehet running.
2. **A Blocked és a Ready állapot közötti különbség félreértése.** A blokkolt folyamat nem azért nem fut, mert nincs szabad CPU, hanem mert valamilyen eseményre vár. Ezért a blokkolt folyamat akkor sem kaphat CPU-t, ha a processzor üres. A helyes megfogalmazás: a Ready állapotban a folyamat a processzorra, a Blocked állapotban egy eseményre vár.
3. **A PCB szerepének alábecsülése.** Néha a hallgatók azt hiszik, hogy a folyamatok közötti váltás csak a regiszterek mentéséből áll. Valójában a PCB tartalmazza a memóriatérkép, a fájlleírók és a biztonsági attribútumok leíróit is, amelyek nélkül a folyamat nem tudna helyesen működni. A PCB elvesztése vagy korruptálása a kernel összeomlásához vezet.
4. **A terminated állapot és a zombi összetévesztése.** A terminated folyamat már nem fut, de a PCB-je még létezik, amíg a szülő nem gyűjti be az exit státuszt. Ha a szülő soha nem hív `wait`-et, a folyamat zombi marad, és erőforrást foglal. Ez nem azt jelenti, hogy a folyamat „él”, hanem hogy a kernel a kilépési adatokat őrzi.
5. **A labor során a mért és várt értékek összemosása.** A mérési naplónak külön kell tartalmaznia a ténylegesen mért állapotot és az elméletileg várt állapotot. Ha a mért érték eltér, azt elemezni kell (pl. a monitor ütemezése miatt rövid átmeneti állapotok kimaradhatnak).

7.1.11 6. Önellenőrző kérdések

1. Magyarázza el a folyamat öt alapállapota közötti átmeneteket! Milyen események váltják ki az egyes átmeneteket?
2. Mi a különbség a Ready és a Blocked állapot között? Miért nem kaphat CPU-t egy blokkolt folyamat?
3. Sorolja fel a PCB legalább öt mezőcsoportját, és indokolja, miért szükséges mindegyik a folyamat megfelelő működéséhez!
4. A laborban a megfigyelt folyamat a `sleep(3)` hívás alatt `S` állapotban volt. Milyen eseményre várt, és miért nem `R` állapotban szerepelt a naplóban?
5. Mi történik a PCB-vel, miután a folyamat terminálódott, de a szülő még nem hívta meg a `wait`-et?

7.1.12 7. Összefoglalás

Ebben az alfejezetben bevezettük a folyamat fogalmát mint végrehajtási egységet, amely a program kódjának, adatainak, vermének és erőforrásainak aktív együttesét jelenti. Bemutattuk a folyamat öt alapállapotát – new, ready, running, blocked, terminated – és az állapotok közötti átmenetek okait. Megismertük a process control block (PCB) szerkezetét, amely a folyamat összes metaadatát tárolja, és alapvető szerepet játszik a kontextusváltásban és az erőforrás-kezelésben. A labor során egy emulált Linux-környezetben figyeltük meg egy folyamat állapotváltozásait, és rögzítettük a mért és várt értékeket. A következő alfejezet a szálak fogalmát vezeti be, amely

a folyamaton belüli párhuzamos végrehajtási egységeket jelenti, és a PCB-ben tárolt állapotok további finomítását igényli.

7.2 Szálak, TLS és felhasználói/kernel szálak

A folyamatmodellből érdemes kiemelni azt a gondolatot, hogy a folyamat egyszerre erőforrás-tulajdonos és végrehajtási egység. Ez a kettősség azonban nem mindig hatékony. Ha egy szerver egyszerre több klienst szolgál ki, vagy egy grafikus alkalmazás egyszerre kezeli a bevitelt és a rajzolást, a külön folyamatok létrehozása drága: mindegyiknek saját címtere, erőforrás-táblája és védelmi kontextusa van. A megoldás az, hogy a végrehajtást elválasztjuk az erőforrás-tulajdonlástól. Ezt a szereposztást valósítja meg a szál.

A szál (thread) a folyamaton belüli könnyű végrehajtási egység. A folyamat továbbra is a címteret, a nyitott fájlokat, a jelkezelőket, a felhasználói és csoportazonosítókat, valamint az egyéb erőforrásokat birtokolja. A szál ezzel szemben a vezérlési állapotot, a végrehajtás aktuális helyét, a veremhasználatot és az ütemezési állapotot hordozza. Egy folyamaton belül több szál osztozik ugyanazon a címtéren és erőforrásokon, ezért a köztük lévő kommunikáció gyors, viszont a közös adatokhoz való hozzáférés szinkronizációt igényel.

A szálak bevezetésének motivációja három fő szempontból érthető meg. Az első a párhuzamosság: több processzormagon valódi egyidejű végrehajtást tesz lehetővé anélkül, hogy a folyamatok közötti adatcserét drága IPC-n keresztül kellene megvalósítani. A második a reszponzivitás: egy hosszú számítást végző szál mellett egy másik szál továbbra is kiszolgálhatja a felhasználói felület eseményeit. A harmadik az erőforrás-hatékonyság: a szálak létrehozása, ütemezése és megszüntetése olcsóbb, mint a folyamatoké, mert nincs szükség új címter és erőforrás-táblák felépítésére.

7.2.1 A szál belső szerkezete

A szálak a folyamaton belül nem függetlenek: osztoznak a kód- és adatszegmensen, a nyitott fájlok tábláján, a jelkezelőkön és a processz erőforrásain. Minden szálnak azonban van saját veremterülete, saját regiszterkészlete, saját utasításszámlálója (program counter, PC), saját stack pointere (SP) és saját ütemezési állapota. Emellett minden szál rendelkezik saját szálazonosítóval és – mint látni fogjuk – saját szál-lokális tárolóval (Thread-Local Storage, TLS).

A szálak leírására a kernel vagy a futásidejű könyvtár egy szálvezérlő blokkot (Thread Control Block, TCB) tart fenn. A TCB kisebb, mint a folyamat-vezérlő blokk (PCB), mert a folyamat erőforrásait nem kell megismételnie. Tartalma jellemzően:

- szálazonosító,
- utasításszámláló és stack pointer,
- általános célú regiszterek mentett értékei,
- ütemezési állapot (fut, kész, blokkolt, befejeződött),
- prioritás vagy ütemezési paraméterek,
- a szál vermének kezdőcíme és mérete,
- TLS-blokk címe,
- hivatkozás a szülő folyamat PCB-jére.

A szálak állapotai a folyamatokéhoz hasonlóak. Egy szál lehet `READY`, amikor futásra kész, de a processzort még nem kapta meg; `RUNNING`, amikor éppen a CPU-n hajtódik végre; `BLOCKED`, amikor valamilyen eseményre vár, például I/O-ra, zárra vagy időzítőre; és `TERMINATED`, amikor a

szálfüggvény visszatért vagy a szálat megszüntették. Az állapotátmeneteket a szálat ütemező komponens kezeli, akár a kernel, akár felhasználói szintű könyvtár.

7.2.2 User-level és kernel-level szálak

A szálak megvalósításának egyik legfontosabb tervezési kérdése, hogy a szálak létezéséről a kernel tud-e, vagy a szálak kizárólag a felhasználói térben valósulnak meg. Ez alapján három alapvető modellt különböztetünk meg.

7.2.3 Felhasználói szintű szálak

A felhasználói szintű szálak (user-level threads, ULT) esetén a kernel egyetlen folyamatot lát, és nem tud arról, hogy azon belül több végrehajtási egység létezik. A szálak létrehozását, ütemezését, kontextusváltását és megszüntetését egy felhasználói térben futó szálkönyvtár végzi. Ilyen volt például a klasszikus `libfiber`, `green threads` vagy sok esetben a korai Java virtuális gépekben használt zöld szálak.

A felhasználói szintű szálak előnye, hogy a kontextusváltás nem igényel kernelmódba való átlépést, ezért rendkívül gyors lehet – akár néhány tucat CPU-ciklus is elegendő lehet. Az ütemezési politika szabadon testreszabható, mert a felhasználói könyvtár saját ütemezőt valósíthat meg. A szálaknak nem kell kernel-erőforrást lefoglalniuk, ezért nagyszámú szál is olcsón fenntartható.

A hátrányok azonban jelentősek. Az egyik legfontosabb, hogy ha egy felhasználói szintű szál blokkoló rendszerhívást hajt végre – például olvas egy socketről vagy fájlból –, akkor a kernel az egész folyamatot blokkolja, mert nem ismeri a többi szálat. Ennek következtében a többi szál sem futhat, még akkor sem, ha egyébként kész állapotban lenne. A második hátrány, hogy a kernel ütemezője nem tudja a szálakat külön-külön processzormagokra helyezni, így a felhasználói szintű szálak nem használják ki a többmagos rendszereket. Végül a preemptív ütemezés hiánya miatt egy rosszul viselkedő szál kiéheztetheti a többit, hacsak a szálkönyvtár nem épít be kooperatív yield-elést.

7.2.4 Kernel-szintű szálak

A kernel-szintű szálak (kernel-level threads, KLT) esetén minden felhasználói szálhoz közvetlenül egy kernelbeli végrehajtási entitás tartozik. A kernel látja és ütemezi az egyes szálakat, és a kontextusváltást, a preemptiót, valamint a blokkolást is a kernel kezeli. A Linux a Native POSIX Thread Library (NPTL) révén gyakorlatilag 1:1 modellt használ: minden `pthread` szál külön kernel-feladatként jelenik meg.

A kernel-szintű szálak előnye, hogy valódi párhuzamosságot tesznek lehetővé többprocesszoros rendszereken: ha egy folyamatnak négy szálja van, és a gép négy maggal rendelkezik, a kernel ütemezője mind a négyet egyidejűleg futtathatja. Ha egy szál blokkol, a kernel csak azt a szálat állítja le, a folyamat többi szálja tovább futhat. A kernel preemptív ütemezést is alkalmazhat, így egyetlen szál sem tudja hosszan monopolizálni a CPU-t.

A hátrány a nagyobb költség. A szálak létrehozása, megszüntetése és a kontextusváltás rendszerhívásokat igényel, ami kernelmódba való átlépéssel, illetve kernelbeli adminisztrációval jár. Emellett minden szálhoz kernel-erőforrás tartozik, például kernelbeli verem, TCB és ütemezői bejegyzés, ezért a nagyszámú szál létrehozása hamarabb eléri az erőforrás-korlátokat, mint a felhasználói szintű szálak esetében.

7.2.5 Hibrid modellek

A két szélsőség között számos hibrid megoldás létezik. Az egyik lehetőség az M:N modell, amelyben M felhasználói szálat N kernel-szálra képezünk le, ahol M nagyobb lehet, mint N. A felhasználói könyvtár ütemezi a szálakat a rendelkezésre álló kernel-szálakra. Ez a megközelítés ötvözi a felhasználói szintű szálak gyors váltását a kernel-szintű szálak párhuzamosságával, de a megvalósítása bonyolult, mert a felhasználói ütemező és a kernel ütemezője között koordinációra van szükség.

Történetileg ide tartozik a lightweight process (LWP) fogalma is, amely egy olyan kernel-szintű végrehajtási egység, amelyhez a felhasználói könyvtár több szálat köthet. A Solaris korábbi rendszereiben az LWP-k valósították meg a felhasználói szálak és a kernel-szálak közötti hidat.

A modern általános célú rendszerek – Linux, Windows, macOS – gyakorlatilag az 1:1 kernel-szintű modellt használják. A felhasználói szintű szálak azonban nem tűntek el: a coroutine-ok, fibers és a programozási nyelvekben megjelenő async/await konstrukciók a felhasználói szintű ütemezés gondolatát viszik tovább, gyakran egy kernel-szálon belül.

7.2.6 Szálbiztonság és a közös címter

A közös címter a szálak legnagyobb előnye és egyben a legnagyobb kockázati forrása. Mivel minden szál ugyanazt a virtuális címteret látja, közvetlenül hivatkozhatnak ugyanarra a globális változóra, heap-területre vagy statikus adatra. Ez gyors adatcserét tesz lehetővé, hiszen nincs szükség IPC-re vagy adatmásolásra. Ugyanakkor ha két szál egyszerre írja ugyanazt a memóriaterületet, az adat sérülhet, és az eredmény nemdeterminisztikus lehet.

A szálbiztonság fogalma azt fejezi ki, hogy egy függvény vagy adatszerkezet több szál egyidejű hívása esetén is helyesen működik. Egy függvény akkor szálbiztos, ha a közös adatokhoz való hozzáférést szinkronizálja, vagy ha egyáltalán nem használ közös, írható állapotot. A szálbiztosság nem azonos a reentranciával, bár a kettő közeli rokon fogalom. Egy reentráns függvény nem használ globális vagy statikus, írható adatot, és nem hívhat nem reentráns függvényt; egy ilyen függvény általában szálbiztos is. A szálbiztos függvény viszont használhat közös adatot, ha a hozzáférést zárakkal vagy atomi műveletekkel védi.

Klasszikus példa a `strtok` függvény, amely egy statikus mutatóban tárolja az elemzés aktuális állapotát. Ha két szál egyszerre hívja a `strtok`-ot, a statikus mutató felülíródik, és az eredmény hibás lesz. A megoldás a `strtok_r`, amely a hívó által átadott pufferben tartja az állapotot, ezért reentráns és szálbiztos. Hasonló a `rand` függvény, amelyet a `rand_r` helyettesít.

A gyakorlatban a szálbiztosság megvalósítása több szinten történhet. Az egyik lehetőség a függvények reentránssá tétele, azaz minden állapotot paraméterben vagy a hívó által kezelt struktúrában kell tárolni. A másik lehetőség a közös adatok védelme kölcsönös kizárással, például mutexszel. A harmadik lehetőség az adatok szál-lokálissá tétele, amikor minden szál saját példányt kap az adott változóból.

Fontos felismerni, hogy a közös címter nem jelenti azt, hogy minden adat közös is. A szál-lokális tárolóval éppen azt érjük el, hogy egy logikailag globális változónak minden szál számára külön példánya legyen. Ezt a mechanizmust a következő szakaszban vizsgáljuk meg.

7.2.7 Thread-Local Storage

A TLS célja, hogy a programozó úgy hivatkozhasson egy változóra, mintha az globális lenne, de a változó értéke mégis szálanként különböző legyen. Ez különösen hasznos olyan állapotok tárolására, amelyek természetüknél fogva szálankéntiek, például a hibakód (`errno`), a szál saját azonosítója, vagy egy hívás nyomon követésére szolgáló veremmélység.

C nyelvben a C11 szabvány bevezette a `_Thread_local` kulcsszót. A GCC korábban a `__thread` kiterjesztést használta, a C++11 pedig a `thread_local` kulcsszót. A három gyakorlatilag ugyanazt a célt szolgálja. Az alábbi példa egy TLS-ben tárolt számlálót mutat:

```
#include <stdio.h>
#include <pthread.h>

_Thread_local int local_counter = 0;
int global_counter = 0;

void *thread_main(void *arg)
{
    for (int i = 0; i < 1000; i++) {
        local_counter++;
        global_counter++;
    }
    printf("local=%d global=%d\n", local_counter, global_counter);
    return NULL;
}
```

Ebben a példában a `local_counter` minden szálban külön példányban létezik, így minden szál a saját értékét növeli. A `global_counter` ezzel szemben közös, és a `global_counter++` művelet nem atomi, így a program adatversenyt tartalmaz. Ha a programot több szállal futtatjuk, a `local_counter` minden szálban pontosan 1000 lesz, a `global_counter` viszont jellemzően kisebb lesz, mint a szálak számának és 1000-nek a szorzata, mert a növelések elveszhetnek.

A TLS megvalósítása architektúrafüggő. Az x86-64 rendszereken a Linux általában az FS szegmensregisztert használja a TLS-blokk eléréséhez, míg a GS-t a kernel saját CPU-lokális adataihoz. A szál létrehozásakor a futásidejű könyvtár lefoglal egy TLS-területet, és a szál TCB-jében eltárolja annak címét. Amikor a szál fut, a szegmensregiszter a saját TLS-blokkjára mutat, így a TLS-változók elérése gyors, általában egyetlen regiszterrel indexelt memóriaművelet. A TLS-változók a szál megszűnésekor automatikusan felszabadulnak.

A TLS tipikus felhasználási esetei közé tartozik a `errno`. A POSIX szabvány a `errno`-t szálankénti értékként definiálja, mert egy több szálat használó programban az egyik szál hibakódja nem írhatja felül a másikét. A Linuxon az `errno` makró gyakran egy függvényhívást takar, például `(*__errno_location())`, amely a szál TLS-blokkjában lévő területre mutat.

A TLS nem csak a futásidejű könyvtárakban hasznos. A fejlesztők is használhatják arra, hogy egy függvényhívási láncon keresztül szálankénti állapotot adjanak át anélkül, hogy a függvények paraméterlistáját módosítani kellene. Ilyen lehet például egy tranzakció-azonosító vagy egy naplózási kontextus. Fontos azonban, hogy a TLS nem helyettesíti a szinkronizációt: ha a szálak közötti megosztás a cél, akkor a közös adatokat továbbra is védeni kell.

7.2.8 Tipikus hibák és megoldások

A szálak használata során számos tipikus hiba fordulhat elő. Az első, leggyakoribb hiba a közös adat védelmének elmulasztása. Amikor két szál ugyanazt a globális változót növeli, az `x++` művelet nem atomi: a processzor először beolvassa az értéket, majd növeli, végül visszaírja. Ha

a két szál művelete közben kontextusváltás történik, az egyik növelés elveszhet. A megoldás a mutex, az atomi típus (`_Atomic` C11) vagy más szinkronizációs eszköz használata.

A második tipikus hiba a nem reentráns függvények hívása több szálból. Ilyen a már említett `strtok`, `rand`, vagy a régebbi `ctime`. A megoldás a reentráns változatok használata (`strtok_r`, `rand_r`, `ctime_r`), vagy a hívások szinkronizálása.

A harmadik hiba a szál veremméretének alábecsülése. Minden szálnak saját verme van, amelynek mérete általában korlátozott, a Linuxon például a `pthread` alapértelmezett veremmérete néhány MiB lehet (az aktuális érték a rendszer konfigurációjától függ). Ha egy szál nagy lokális tömböt hoz létre, vagy mélyen rekurzív függvényt hív, a verem túlcsordulhat. A túlcsordulás hibás memóriahozzáférést vagy a szomszédos adatok sérülését okozhatja. A megoldás a verem méretének gondos tervezése vagy a verem méretének explicit beállítása a szál létrehozásakor.

A negyedik hiba a hamis megosztás (false sharing). Ha két szál különböző, de ugyanabba a cache-sorba eső változókat ír, a cache-koherencia protokoll feleslegesen mozgatja a cache-sorokat a processzormagok között, ami jelentős teljesítménycsökkenést okozhat. A megoldás a változók szétválasztása cache-sor méretre történő igazítással, például paddinggel.

Az ötödik hiba a TLS helytelen használata. Előfordul, hogy a fejlesztő TLS-t használ olyan adatra, amelyet valójában a szálak között meg kell osztani. Ebben az esetben a szálak mindegyike a saját példányát látja, és a program logikai hibát tartalmaz. A másik irányú hiba, amikor egy közös változót szálankénti állapotként kezelnek, és adatverseny alakul ki. A tervezés során mindig egyértelműen el kell dönteni, hogy egy adat a szálhoz vagy a folyamathoz tartozik.

7.2.9 Labor: szálmodell és TLS-vázlat

A következő labor célja, hogy a szálakkal kapcsolatos fogalmakat egy kis, ellenőrizhető példán keresztül megerősítse. A labor nem igényel kernelmódosítást, és nem nyúl a hoszt rendszerhez; minden futtatás emulált környezetben történik.

7.2.10 Kiinduló állapot

A labor egy QEMU/KVM alatt futó x86-64 Linux vendéget használ. A vendég konfigurációja:

- CPU: 2 vCPU, x86-64, SSE4.2,
- memória: 512 MiB,
- kernel: Linux 6.1 LTS (a pontos verziót a kiadás előtt ellenőrizni kell),
- libc: glibc 2.36 vagy újabb,
- fordító: GCC 12.2, `-std=c11 -O2 -pthread`.

A mérések reprodukálhatósága érdekében a vendéget rögzített, nem túlterhelt hoszton futtatjuk. A mérési naplóban rögzíteni kell a hoszt CPU-típusát, órajelét, a QEMU verzióját és a vendég kernel verzióját.

7.2.11 A labor célja

A feladat két részből áll:

1. Készítsen egy C programot, amely három szálat indít. Minden szál 1000 alkalommal növeli a saját TLS-számlálóját, és 1000 alkalommal növel egy közös, szinkronizáció nélküli globális számlálót. A program írja ki minden szál helyi számlálóját, a közös számláló végső értékét, továbbá a várt értéket (3000).
2. Készítsen vázlatot a szálak állapotairól és a TLS elrendezéséről. A vázlat tartalmazzon egy TCB-struktúra tervet, a TLS-blokk hivatkozását, valamint a szálak közös és saját erőforrásainak felsorolását.

7.2.12 A programkód

Az alábbi kód szemléltetés, amelyet a vendéggépen le kell fordítani és futtatni:

```
#include <stdio.h>
#include <pthread.h>
#include <time.h>

_Thread_local int tls_counter = 0;
int shared_counter = 0;

void *worker(void *arg)
{
    int id = *(int *)arg;
    for (int i = 0; i < 1000; i++) {
        tls_counter++;
        shared_counter++;
    }
    printf("thread %d: tls=%d\n", id, tls_counter);
    return NULL;
}

int main(void)
{
    pthread_t t[3];
    int id[3] = {1, 2, 3};
    struct timespec start, end;
    double elapsed_ms;

    clock_gettime(CLOCK_MONOTONIC, &start);

    for (int i = 0; i < 3; i++)
        pthread_create(&t[i], NULL, worker, &id[i]);

    for (int i = 0; i < 3; i++)
        pthread_join(t[i], NULL);

    clock_gettime(CLOCK_MONOTONIC, &end);

    elapsed_ms = (end.tv_sec - start.tv_sec) * 1000.0
               + (end.tv_nsec - start.tv_nsec) / 1e6;

    printf("shared_counter=%d expected=3000\n", shared_counter);
    printf("elapsed=%.3f ms\n", elapsed_ms);

    return 0;
}
```

A kódban a `tls_counter` minden szálban külön példányban létezik, ezért minden szál kimenete `tls=1000` lesz. A `shared_counter` közös, és a növelés nem atomi, ezért a végső érték a legtöbb futtatásban kisebb lesz 3000-nél, bár a pontos érték futtatásonként változhat. A program a futási időt is méri.

7.2.13 Build és futtatás

A vendéggépen a következő parancsokkal fordítható és futtatható a program:

```
gcc -std=c11 -O2 -pthread -o thread_lab thread_lab.c
./thread_lab
```

A méréshez legalább tíz futtatást végzünk. Minden futtatásnál rögzítjük a `shared_counter` értékét, az egyes szálak TLS-értékeit és a futási időt. A mérési naplóban a várt és a mért eredményt külön oszlopban kell feltüntetni.

7.2.14 Elvárt eredmény

A TLS-számláló minden futtatásban pontosan 1000 minden szál esetén. Ez az elvárt eredmény, mert a TLS-változókhoz más szál nem fér hozzá.

A közös számláló várt értéke 3000, de mivel a növelés nem atomi és nincs szinkronizáció, a mért érték általában kisebb lesz. Az adatverseny miatt a mért érték nemdeterminisztikus; egyes futtatásokban akár 3000 is lehet, de a legtöbb esetben 2000 és 2999 közötti értéket kapunk. A pontos érték a rendszer terhelésétől és az ütemező viselkedésétől függ.

7.2.15 Mérési napló

A naplóban minden futtatásnál rögzítjük:

- a futtatás sorszámát,
- a `shared_counter` mért értékét,
- a TLS-értékeket (ezek mindig 1000),
- a futási időt milliszekundumban,
- a hoszt és a vendég órajelét, a QEMU verzióját, a kernel verzióját.

A mérési eredményeket az alábbi táblázatba gyűjtjük:

Futtatás	Várt `shared_counter`	Mért `shared_counter`	TLS szál 1	TLS szál 2	TLS szál 3	Futási idő (ms)
1	3000		1000	1000	1000	
2	3000		1000	1000	1000	
…	3000		1000	1000	1000	

A mérési oszlopok kitöltése a labor során történik. A várt és a mért értéket egyértelműen el kell különíteni.

7.2.16 Visszaállítás

A labor nem módosítja a vendéggép rendszerbeállításait, és nem igényel visszaállítási lépést. A létrehozott futtatható fájl és forrásfájl a vendég `/tmp` könyvtárában marad, és a vendéggép törlésével vagy újraindításával automatikusan eltűnik. A hoszt rendszerhez nem nyúlunk.

7.2.17 Önellenőrző kérdés

Miért nem lesz a közös számláló mindig 3000, miközben a TLS-számláló mindig pontosan 1000? Indokolja a választ a közös címter és a TLS eltérő tulajdonságaival, és nevezzen meg egy lehetséges szinkronizációs eszközt, amellyel a közös számláló is pontosan 3000-re hozható.

7.2.18 Összefoglalás

A szálak a folyamaton belüli könnyű végrehajtási egységek, amelyek megosztják a folyamat címterét és erőforrásait, de saját vezérlési állapottal és veremmel rendelkeznek. Megvalósításuk történhet felhasználói szinten, kernel-szinten vagy hibrid modellben. A közös címter gyors kommunikációt tesz lehetővé, de szinkronizációt igényel; a szálbiztosság és a reentrancia alapvető tervezési szempont. A TLS lehetővé teszi, hogy a szálak szálankénti állapotot tároljanak anélkül, hogy a közös adatokhoz nyúlnának. A labor során a szálak közötti adatversenyt és a TLS működését egy egyszerű C programmal vizsgáltuk, a várt és a mért eredményeket pedig elkülönítve rögzítettük. A következő alfejezet ezekre az alapokra építve a kontextusváltást, a `fork`/`exec` műveleteket és a folyamatok életciklusát tárgyalja.

7.3 Kontextusváltás, fork/exec és folyamatéletciklus

Az előző alfejezetben a folyamatmodellt, a PCB szerepét és az állapotátmeneteket vezettük be. Itt az életciklus három kulcsműveletére koncentrálunk: a folyamat létrehozására, a program képének cseréjére és a folyamat megszüntetésére. E műveletek alacsony szintű mechanizmusa a kernelben a kontextusváltás, amelynek költsége alapvetően meghatározza a rendszer skálázhatóságát és a folyamatok közötti átadás árát.

A fejezet tanulási céljai: (1) megmagyarázni a folyamat létrehozásának, cseréjének és megszüntetésének lépéseit a fork/exec/wait/exit modellben; (2) alkalmazni ezeket egy kis, ellenőrizhető programban; (3) felismerni a tipikus hibákat, például a pufferelt I/O kettőzését vagy a zombie-folyamatok megjelenését. Az elmélet után egy reprodukálható labort is leírunk, amelyben a fork/exec és a kontextusváltás lépéseit, valamint a költségét lehet mérni és dokumentálni.

7.3.1 Folyamatéletciklus áttekintés

Egy folyamat a kernel szempontjából egy PCB-ből (process control block), egy vagy több végrehajtási szálból, saját címterületből és a hozzá rendelt erőforrásokból (fájlleírók, jelzések, processzcsoport-tagság stb.) áll. A teljes életciklus a következő fő állapotokkal írható le:

- **Létrehozás előtt**: a folyamat még nem létezik; a szülő egy rendszerhívással (Unix: `fork`, Windows: `CreateProcess`) kezdeményezi a létrehozást.
- **Futásra kész (ready)**: a processz minden erőforrással rendelkezik, csak a processzorra vár.
- **Futó (running)**: a processz éppen egy CPU magon végrehajtja az utasításait.
- **Blokkolt (waiting/sleeping)**: a processz valamilyen eseményre vár (I/O, zárolás, időzítő, jel).
- **Leállítva (stopped)**: a processz futását jel vagy nyomkövető állította le; nem ütemezhető, amíg újra nem indítják.
- **Zombie (élőhalott)**: a processz befejezte a futását, de a szülő még nem gyűjtötte be a kilépési állapotát.

- **Megszüntetve**: a PCB és minden erőforrás felszabadult.

Az átmeneteket a kernel ütemezője és a rendszerhívások vezérlik. A jelen alfejezetben a létrehozás, a programcsere és a megszüntetés átmeneteit vizsgáljuk; a futásra kész és futó közötti váltást a kontextusváltás valósítja meg.

7.3.2 Folyamat létrehozása: fork

A POSIX-rendszerekben a folyamat létrehozásának alapprimitívje a `fork()` rendszerhívás. Lényege, hogy a hívó folyamatot szinte teljes egészében lemásolja: a gyermek processz saját PCB-t, saját címterületet és saját kernel-vermet kap, de a memóriatartalom, a fájlleírók és a jelzések kezdetben megegyeznek a szülőével. A két folyamat ezután külön ütemezhető, és mindkettő a `fork()` hívás utáni utasítástól folytatja a végrehajtást.

A `fork()` visszatérési értéke különbözteti meg a szülőt és a gyermeket:

- a gyermekben `0`,
- a szülőben a gyermek processzazonosítója (PID, `pid_t`),
- hiba esetén `-1`, és az `errno` jelzi az okot.

Példa a legalapvetőbb mintára:

```
#include <sys/types.h>
#include <unistd.h>
#include <stdio.h>
#include <errno.h>

int main(void) {
    pid_t pid = fork();
    if (pid < 0) {
        perror("fork");
        return 1;
    }
    if (pid == 0) {
        printf("gyermek: pid=%ld\n", (long)getpid());
    } else {
        printf("szülő: gyermek pid=%ld\n", (long)pid);
    }
    return 0;
}
```

A `fork()` megvalósítása a kernelben több lépésből áll:

1. **PCB lefoglalása és inicializálása** a szülő adatai alapján.
2. **Új PID hozzárendelése** a gyermekhez, valamint a processzcsoport és a munkamenet öröklése.
3. **Címterület másolása vagy megosztása.** A hagyományos `fork()` a teljes memóriát átmásolja, ami költséges. A modern rendszerek copy-on-write (COW) technikát használnak: a kernel a lapokat csak olvashatóként osztja meg, és csak akkor készít fizikai másolatot, ha valamelyik fél írni próbál.
4. **Fájlleírók duplikálása.** A gyermek ugyanazokat a fájlleírókat kapja, mint a szülő. Ez azt jelenti, hogy a megnyitott fájlok offset-je megosztott; ha a gyermek ír egy közös fájlba, az hat a szülő által látott pozícióra is.
5. **Jelzések és erőforrások öröklése** bizonyos szabályok szerint (például a beállított jelkezelők öröklődnek, a függőben lévő jelzések nem).

6. **A gyermek beillesztése az ütemező futásra kész sorába.** A szülő és a gyermek sorrendje nem determinisztikus: a gyermek futhat előbb is.

A COW miatt a `fork()` futásideje gyakran jóval kisebb, mint a teljes memóriamásolásé, de a későbbi laphibák során fizetik meg a másolás árát. A `fork()` tipikus hibái: processzszám-korlát elérése (`EAGAIN`), memóriahiány (`ENOMEM`), vagy a hívó nem jogosult processzt létrehozni.

7.3.3 Programcsere: exec

A `fork()` önmagában ugyanazt a programot futtatja két példányban. Ha más programot szeretnénk indítani, az `exec*()` család valamelyik tagját kell hívni. Az `exec` a hívó folyamat címterét teljesen lecseréli az új programra, miközben a processz azonosítója, a fájlleírók és a jelzések bizonyos része megmarad.

A POSIX több változatot definiál: `execl`, `execlp`, `execle`, `execv`, `execvp`, `execvpe`. A különbség a paraméterátadás módjában van: a `l` (list) a paramétereket szöveges argumentumlistaként várja, a `v` (vector) egy `NULL`-al lezárt tömbként; a `p` (path) a `PATH` környezeti változóban keresi a programot; az `e` (environment) saját környezeti tömböt ad.

Az `exec` siker esetén nem tér vissza, mert a hívó processz képe megsemmisül. Hiba esetén `-1`-gyel tér vissza, és az `errno` jelzi az okot. Ezért a tipikus minta:

```
pid_t pid = fork();
if (pid == 0) {
    /* gyermek */
    execl("/bin/echo", "echo", "hello", NULL);
    perror("execl");
    _exit(127);   /* csak akkor jutunk ide, ha az exec hibázott */
}
```

Az `exec` megvalósítása általában a következő lépésekből áll:

1. **Az új program betöltése** a fájlrendszerből: a kernel azonosítja a formátumot (például ELF), és ellenőrzi a futtathatóságot.
2. **A régi címterület elvetése**: a felhasználói tér lapjait felszabadítja, a kernel-verem egy részét újra inicializálja.
3. **Új címterület felépítése**: a program szegmenseit (szöveg, adat, stack, heap) a megfelelő virtuális címekre leképezi, a dinamikus linkelőt futtatja, ha szükséges.
4. **Fájlleírók kezelése**: az `FD_CLOEXEC` jelzővel ellátott leírók bezáródnak; a többi öröklődik.
5. **Jelzések beállítása**: az alapértelmezett jelkezelők visszaállnak, a figyelmen kívül hagyás és a beállított kezelők megmaradnak.
6. **Az új program belépési pontjára ugrás**: a kernel a felhasználói módba való visszatéréskor a megfelelő kezdőcímet adja meg.

Fontos, hogy a `fork` + `exec` minta nem atomi: a két hívás között a gyermek processz a szülő memóriaképének másolatán fut, ami betöltés előtt lefuthat, például pufferelt kimenetet okozva. A gyakorlatban ezt úgy kerülik el, hogy a `fork` után a gyermek azonnal `exec`-el, és ehhez nyújt segítséget a `posix_spawn`, amely a kettő kombinációját hatékonyan implementálhatja.

7.3.4 Várakozás és kilépés: wait és exit

A folyamat kilépésére a `exit(status)` függvény vagy a `return` szolgál; a kernelben ez a `_exit` rendszerhívás. A kilépési folyamat lépései:

1. A C futási környezet az `atexit`-regisztrált függvényeket lefuttatja, kiüríti a pufferelt I/O-t, és bezárja a szabványos streameket.
2. A kernel felszabadítja a folyamat címterének lapjait, a nyitott fájlleírókat, a várakozó szemaforokat és egyéb erőforrásokat.
3. A PCB egy része megmarad: a kilépési státusz, a PID és a felhasznált CPU-idő, valamint a szülőre vonatkozó adat.
4. A folyamat **zombie** állapotba kerül: nem fut, nem használ memóriát a címteréből, de a PCB-t nem szabadítják fel, amíg a szülő be nem gyűjti a kilépési állapotot.

A szülő a `wait()` vagy `waitpid()` rendszerhívással gyűjtheti be a gyermek állapotát. Ez szinkronizálja a szülőt a gyermek befejezésével, és felszabadítja a zombie PCB-t. A `wait` blokkol, amíg legalább egy gyermek kilép; a `waitpid` lehet nem blokkoló (`WNOHANG`) vagy megvárhat egy adott PID-et.

A kilépési státusz értelmezésére makrók szolgálnak:

```
pid_t pid = fork();
if (pid == 0) {
    exit(7);                  /* gyermek kilép 7-es kóddal */
}
int status;
pid_t w = wait(&status);
if (w == -1) {
    perror("wait");
} else if (WIFEXITED(status)) {
    printf("gyermek kilépett: %d\n", WEXITSTATUS(status));
} else if (WIFSIGNALED(status)) {
    printf("gyermek jel hatására halt meg: %d\n", WTERMSIG(status));
}
```

Ha a szülő nem várja meg a gyermeket, az árva folyamattá válhat, ha a szülő előbb kilép. A Unix-rendszerek ilyenkor az init folyamathoz (vagy egy al-reaperhez) rendelik az árvát, amely idővel begyűjti a zombie státuszt. Ha a szülő tovább fut, de sosem hív `wait`-et, a gyermek zombieként marad, ami PID-kimerüléshez vezethet.

7.3.5 Kontextusváltás

A kontextusváltás az a mechanizmus, amelynek során a CPU egyik folyamat (vagy szál) végrehajtásáról egy másikra vált. Két szinten valósul meg:

- **Kernel/user mód közötti váltás** (például rendszerhívás vagy megszakítás során) nem feltétlenül jelent folyamatváltást.
- **Folyamat- vagy szálváltás** a kernel ütemezőjének döntése alapján: az aktuális végrehajtási kontextust elmenti, és egy másik kontextusát tölti be.

A folyamat szintű kontextusváltás fő lépései:

1. **Az aktuális folyamat regisztereinek mentése** a kernel-veremre vagy a PCB-be: általános célú regiszterek, utasításszámláló, veremmutató, vezérlőregiszterek.
2. **A kernel-verem átkapcsolása** az új folyamat kernel-vermére.

3. **A címterület átkapcsolása**: a processzor lapozási táblázatának (például `CR3` x86-on, `SATB` ARM-on) cseréje a célfolyamatéra. Ez érvényteleníti a TLB egy részét, amit a hardver részben automatikusan, részben szoftveresen kezel.
4. **Az új folyamat regisztereinek visszaállítása** a PCB-ből.
5. **Visszatérés az új folyamat futási kontextusába.**

A szálak közötti váltás olcsóbb, mert a címterület közös: a lapozási táblázat cseréjére és a TLB öblítésre nincs szükség. A folyamatok közötti váltásnál viszont minden lebegő kontextus (TLB, cache-előfeszítés, előrejelző állapot) elveszhet, ami indirekt költséget okoz.

7.3.6 A kontextusváltás költsége

A költség két komponensre bontható:

- **Direkt költség**: a regiszterek mentése/visszaállítása, a kernel-verem váltása és a lapozási táblázat cseréje. Ez a kernel kódjának végrehajtási ideje, és jól mérhető megszakításokkal vagy futásidő-figyelőkkel.
- **Indirekt költség**: a cache-ek és a TLB hideg indulása az új folyamat miatt. A címtérváltás után az új folyamat adatai nem találhatók a cache-ben, ezért az első memóriahozzáférések lassúak. Ez a hatás gyakran nagyobb, mint a direkt költség.

A méréshez olyan módszert kell választani, amely elkülöníti a kettőt. Például két folyamat váltakozó futása egy pipe-on keresztül mérik a teljes oda-vissza időt; az eredményből levonva a rendszerhívás és a pipe kezelés költségét kapjuk a kontextusváltás felső becslését. Az ütemező `sched_yield` hívása hasonlóan használható, de ott a futásra kész sor hossza is befolyásolja az eredményt. A legpontosabb mérések hardver-számlálókkal (például `perf stat`) készülnek, amelyek a ciklusszámot, a cache-tévesztéseket és a TLB-töltéseket is rögzítik.

A kontextusváltás időtartama erősen függ a hardvertől, a kernel verziójától, a cache-mérettől és a munkaterheléstől. Emulált környezetben (QEMU, VirtualBox) ez az idő nagyobb lehet, és nagyobb szórású, ezért a laborban ezt figyelembe kell venni, és a mérési körülményeket pontosan dokumentálni kell.

7.3.7 Példa: fork + exec + wait egyben

Az alábbi program a tipikus POSIX-mintát mutatja: a szülő létrehoz egy gyermeket, a gyermek lecseréli magát egy másik programra, a szülő pedig megvárja a befejezést és kiírja a státuszt.

```
#include <sys/types.h>
#include <sys/wait.h>
#include <unistd.h>
#include <stdio.h>
#include <errno.h>

int main(void) {
    pid_t pid = fork();
    if (pid < 0) {
        perror("fork");
        return 1;
    }

    if (pid == 0) {
        /* Gyermek: programcsere */
        execl("/bin/ls", "ls", "-l", "/tmp", NULL);
        /* Csak akkor jutunk ide, ha az exec hibázott */
```

```
        perror("execl");
        _exit(127);
    }

    /* Szülő: várakozás a gyermekre */
    int status;
    pid_t w = waitpid(pid, &status, 0);
    if (w == -1) {
        perror("waitpid");
        return 1;
    }

    if (WIFEXITED(status)) {
        printf("gyermek kilépési kódja: %d\n", WEXITSTATUS(status));
    } else if (WIFSIGNALED(status)) {
        printf("gyermek jel hatására halt meg: %d\n", WTERMSIG(status));
    } else {
        printf("gyermek nem szokványos módon fejeződött be\n");
    }
    return 0;
}
```

A kód szemléltetésül szolgál; valós rendszeren a `/bin/ls` elérési útja és a fájlrendszer elrendezése ellenőrizendő.

7.3.8 Tipikus hibák és buktatók

1. **Pufferelt I/O kettőzése fork után.** A C könyvtár `printf`-je pufferelt. Ha a `stdout` pufferében marad adat a `fork` pillanatában, akkor mind a szülő, mind a gyermek kiürítheti azt, ami duplikált kimenetet okoz. Megoldás: a `fork` előtt ki kell üríteni a puffereket (`fflush(NULL)`), vagy a gyermekben `_exit`-et használni az `exit` helyett, hogy ne fusson le a pufferürítés.

2. **Az exec hibájának kezelése.** Ha a `fork` sikeres, de az `exec` nem találja a programot, a gyermek tovább fut a szülő memóriaképének másolatával. Ez veszélyes, mert a gyermek a szülő utasításait hajtja végre. Minden `exec`-hívást közvetlenül `perror` és `_exit` kell kövessen.

3. **Zombie-folyamatok.** Ha a szülő nem hív `wait`-et, a gyermek kilépése után a PCB nem szabadul fel. Hosszabb ideig futó szolgáltatások esetén ez PID-kimerülést okoz. Megoldás: mindig `wait`/`waitpid`, vagy a `SIGCHLD` jel figyelése, vagy a gyermek `double fork`-olása, hogy az init legyen a szülő.

4. **Árva folyamatok.** Ha a szülő kilép, mielőtt a gyermek befejezné, a gyermek árvaként az inithez kerül, ami általában begyűjti. Ez nem feltétlenül hiba, de ha a szülő fontos erőforrásokat biztosított, a gyermek elveszítheti azokat.

5. **Fájlleírók öröklődése.** Az `exec` után a fájlleírók öröklődnek, kivéve az `FD_CLOEXEC` jelzővel ellátottakat. Ha egy biztonságos program nem zárja be a nem kívánt leírókat, a gyermek olyan erőforrásokhoz férhet, amelyeket a szülő már nem használ.

6. **A fork és a szálak keverése.** Többszálú folyamatban a `fork` csak a hívó szálat másolja; a többi szál eltűnik. Ez holtpontokhoz vezethet, ha a többi szál zárolt erőforrásokat tart. A `fork` után a gyermekben csak async-signal-safe függvényeket szabad hívni.

7.3.9 Labor: fork/exec és kontextusváltás mérése

Az alábbi labor célja a `fork`, `exec` és a kontextusváltás lépéseinek leírása, valamint a költség mérése reprodukálható módon. A labor emulált környezetben fut, például QEMU-ban egy minimális Linux rendszerben. A konfigurációt, a build- és futtatási parancsokat rögzíteni kell, és a mért eredményeket el kell különíteni a várt elméleti értékektől.

7.3.10 Kiinduló állapot

- **Emulátor**: QEMU 8.x, `x86_64` architektúra, egy vCPU, 256 MiB RAM, `-machine q35` beállítással.
- **Kernel**: egy stabil Linux kernel (például 6.6 LTS) minimális konfigurációval, amely tartalmazza a `proc`, `sys`, `dev` és a `sched` alrendszereket.
- **Felhasználói tér**: buildroot vagy egy egyszerű statikus busybox környezet, `musl` vagy glibc C könyvtárral.
- **Fordító**: gcc vagy clang, `-O2` optimalizálással, statikus linkelés.
- **Mérési eszköz**: `clock_gettime(CLOCK_MONOTONIC)`, valamint `perf stat` ha elérhető; emulált környezetben a `perf` hardver-számlálói korlátozottak lehetnek, ezért a futásidő-mérés ajánlott.

A konfigurációt és a buildparancsokat a reprodukciós csomagban dokumentálni kell. A labor során a hoszt valódi lemezét tilos módosítani; minden művelet a QEMU lemezképén belül történik.

7.3.11 Mérési feladatok

1. **A `fork()` futásidejének mérése** különböző folyamatméreteknél.

   - Hozzunk létre egy szülő folyamatot, amely mérete növelhető (például nagy memória-terület lefoglalásával).
   - A szülő `clock_gettime`-mel méri a `fork()` hívás idejét.
   - A gyermek azonnal `_exit(0)`-t hív, a szülő `waitpid`-del begyűjti.
   - Végezzünk több mérést (például 1000 iteráció), és számoljunk átlagot, szórást.

   Várt (elméleti jellemző): a `fork()` idő a COW miatt nagyjából független a folyamat méretétől, amíg nem ír a memóriába. Ha a gyermek a `fork` után azonnal ír a teljes lefoglalt területre, akkor a másolás laphibánként fizetődik meg, és az idő arányos lesz a terület méretével. Az első esetben a várt idő legfeljebb néhány száz mikroszekundum, a másodikban milliszekundum nagyságrendű is lehet.

2. **Az `exec()` futásidejének mérése.**

   - Egy kis méretű programot (például `/bin/true` vagy egy saját statikus bináris) indítsunk `fork`-kal, majd a gyermekben `exec`-et, és mérjük az `exec` hívás idejét.
   - A mérést a `fork` után a gyermekben végezzük `clock_gettime`-mel az `exec` előtt és után (siker esetén nincs visszatérés, ezért a mérést a szülőben is meg lehet ismételni a `wait`-ig).

   Várt (elméleti jellemző): az `exec` idő nagymértékben függ a betöltött program méretétől és attól, hogy dinamikus linkelésű-e. Egy statikus, néhány KiB-os program betöltése gyorsabb, mint egy dinamikus, sok függvénykönyvtárat igénylő programé. Az emulált környezetben az I/O dominál.

3. **A kontextusváltás költségének becslése pipe-pal.**

- Két folyamat váltakozva fut egy pipe-on keresztül: az egyik ír, a másik olvas, majd szerepet cserélnek.
- Mérjük egy üzenetciklus (két váltás) idejét `clock_gettime`-mel, sok ismétlésben.
- A kapott átlagos idő fele a két folyamat közötti kontextusváltás felső becslése, mert tartalmazza a pipe rendszerhívások és a memóriamásolás költségét is.
- A `sched_yield`-alapú mérés alternatív, de ott az ütemező viselkedése befolyásolja az eredményt.

Várt (elméleti jellemző): a direkt kontextusváltás ideje modern hardveren néhány mikroszekundum, emulátorban ennél nagyobb lehet. A konkrét érték erősen hardver- és kernelverzió-függő, ezért nem adható meg általános szám.

7.3.12 Mérési napló és eredmények elkülönítése

A mérési naplóban minden méréshez rögzíteni kell:

- a pontos hardvert vagy emulátort (CPU modell, órajel, memória méret, kernel verzió),
- a fordító verzióját és a fordítási beállításokat,
- a mérési módszert (függvény, iterációk száma, kiugró értékek),
- a mért nyers értékeket (átlag, szórás, minimum, maximum).

A **várt/levezetett** eredmény az elméleti megfontolásokból vagy korábbi publikált adatokból származik, és ezt külön oszlopban kell feltüntetni. A **mért** eredmény a laborban végrehajtott futásokból származik. A két értéket soha nem szabad összekeverni, és a levezetés forrását is meg kell adni.

Példa a napló szerkezetére:

Művelet	Várt (elméleti jellemző)	Mért (labor)
`fork()` COW-val, 1 MiB lefoglalt terület	A COW miatt az idő nem növekszik jelentősen a területtel	[átlag, szórás, iterációk]
`fork()` utáni teljes írás a gyermekben	Az idő arányos a lefoglalt memóriával, laphibánkénti másolás	[átlag, szórás, iterációk]
`exec()` statikus mini program	A betöltés I/O-korlátos, ms nagyságrend	[átlag, szórás, iterációk]
Kontextusváltás (pipe ciklus)	Direkt költség s nagyságrend, indirekt (cache/TLB) további járulék	[átlag, szórás, iterációk]

A táblázatban a mért mezőket a hallgató tölti ki a labor során; a várt mezők az elméleti érvelés alapján adottak. A kiértékelésnél a várt és a mért közötti eltérést meg kell indokolni (például az emulátor torzítása, a mérési zaj, a kernel konfigurációja).

7.3.13 Visszaállítás

A labor után a QEMU lemezképet vissza kell állítani az eredeti állapotba, vagy eldobható módon kell létrehozni. A hoszt rendszerén semmilyen módosítás nem maradhat. A reprodukciós csomag tartalmazza a buildparancsokat, a linker scriptet, a futtatási parancsokat és a mérési naplót.

7.3.14 Önellenőrző kérdések

1. Miért térül meg a copy-on-write a `fork()` során, és milyen költséget fizetünk később?
2. Mi a különbség a `fork` és a `posix_spawn` között a processz létrehozása során?
3. Mi történik a fájlleírókkal `exec` után, és mi az `FD_CLOEXEC` szerepe?
4. Hogyan keletkezik zombie-folyamat, és hogyan lehet megelőzni a PID-kimerülést?
5. Sorolja fel a kontextusváltás fő lépéseit, és magyarázza el, miért olcsóbb a szálak közötti váltás, mint a folyamatok közötti.
6. Melyik komponens (direkt vagy indirekt) dominálja általában a kontextusváltás költségét, és hogyan lehet ezt mérni?

7.3.15 Összefoglalás

Ebben az alfejezetben a folyamatéletciklus három alappillérét tárgyaltuk. A `fork` a folyamat létrehozásának POSIX-modellje, amely a szülő másolatát készíti el COW-val. Az `exec` a program képét cseréli le, miközben a processz azonossága és bizonyos erőforrásai megmaradnak. Az `exit` és a `wait` a megszüntetés és a kilépési állapot begyűjtésének mechanizmusát adják. A kontextusváltás a folyamatok és szálak közötti CPU-átadást valósítja meg, és költsége meghatározza a rendszer általános reagálóképességét. A labor célja ezeknek a műveleteknek a reprodukálható mérése, ahol a várt elméleti jellemzők és a mért értékek szigorúan elkülönülnek.

A következő alfejezet (IPC: pipe, üzenetsor, osztott memoria és szignál) ezekre a fogalmakra építve az IPC mechanizmusokat (pipe, üzenetsor, osztott memória, szignál) tárgyalja; a fork/exec párba állítás és a kontextusváltás ismerete elengedhetetlen az ottani példák megértéséhez.

7.4 IPC: pipe, üzenetsor, osztott memoria és szignál

A folyamatok izolációja az operációs rendszer egyik alapvető védelmi és stabilitási garanciája: az egyik folyamat nem írhat közvetlenül a másik memóriájába, nem hívhatja meg annak függvényeit, és nem férhet hozzá annak erőforrásaihoz. A gyakorlatban azonban a folyamatoknak gyakran együtt kell működniük: egy shell egy pipeline-ban adatot ad át egy szűrőnek, egy naplózó démon üzeneteket fogad, egy kiszolgáló a munkafolyamataival oszt meg puffert. Erre szolgálnak a folyamatok közötti kommunikáció (IPC, inter-process communication) mechanizmusai.

Ebben az alfejezetben a leggyakoribb IPC-eszközöket tekintjük át: a névtelen és nevesített csöveket, az üzenetsorokat, az osztott memóriát, a szignálokat, valamint röviden a socketeket és a szinkronizációs primitívek szerepét. A cél nem csupán az egyes API-k felsorolása, hanem annak megértése, hogy mikor melyik mechanizmus illeszkedik a feladat szemantikájához és teljesítményigényéhez. A fejezet végén egy reprodukálható laborfeladat segít a választási szempontok gyakorlásában.

7.4.1 Miért van szükség IPC-re?

Az operációs rendszer a folyamatokat külön címterekbe helyezi. A virtuális memória és a memóriavédelem miatt egy P_1 folyamat egy adott virtuális címe nem ugyanazt a fizikai lapot jelöli, mint P_2 azonos virtuális címe. Ez a tulajdonság megakadályozza, hogy egy hibás vagy rosszindulatú folyamat közvetlenül módosítsa egy másik adatait, ugyanakkor kizárja a közvetlen memóriacím alapú adatcserét is.

Az IPC mechanizmusai három nagy csoportba sorolhatók:

1. **Adatátviteli csatornák:** pipe, FIFO, üzenetsor, socket. A kernel közvetítésével bájtsorozatot vagy rekordokat juttatnak el az egyik folyamattól a másikhoz.
2. **Osztott memória:** két vagy több folyamat ugyanazt a fizikai memórialapot képezi le a saját címterébe. Ezzel a másolási költség elkerülhető, de a hozzáférést külön szinkronizálni kell.
3. **Értesítési és szinkronizációs mechanizmusok:** szignálok, szemafórok, mutexek. Ezek nem feltétlenül szállítanak nagy adatmennyiséget, hanem eseményeket jeleznek, vagy összehangolják a hozzáférést.

E mechanizmusok nem egymás versenytársai, hanem kiegészítő eszközök. Egy robusztus rendszer gyakran kombinálja őket: például osztott memórián keresztül nagy puffert oszt meg, és egy szemafórral vagy pipe-pal jelzi az új adat érkezését.

7.4.2 Névtelen cső (pipe)

A pipe a legrégebbi és legegyszerűbb IPC-forma. Egyirányú, bájtsorozat-alapú adatcsatorna, amelyet a kernel egy rögzített méretű pufferként valósít meg. A csőnek két végpontja van: egy író és egy olvasó fájlleíró. Az író oldalon a `write()` rendszerhívás a kernelpufferbe másolja a bájtokat, az olvasó oldalon a `read()` pedig onnan olvassa ki.

A pipe létrehozása a `pipe` rendszerhívással történik:

```
int pipe(int pipefd[2]);
```

A paraméter egy két elemű tömb, amelybe a kernel beírja a két fájlleírót: `pipefd[0]` az olvasó vég, `pipefd[1]` az író vég. A visszatérési érték siker esetén `0`, hiba esetén `-1`, és az `errno` tartalmazza a hiba okát (például `EMFILE`, ha a folyamatnak túl sok nyitott fájlleírója van).

A pipe tipikus használata szülő-gyermek folyamatok között:

```
#include <unistd.h>
#include <sys/wait.h>
#include <stdio.h>
#include <string.h>

int main(void) {
    int fd[2];
    if (pipe(fd) == -1) {
        perror("pipe");
        return 1;
    }

    pid_t pid = fork();
    if (pid == -1) {
        perror("fork");
        return 1;
    }

    if (pid == 0) {
        /* Gyermek: olvasó oldal */
        close(fd[1]);                  /* az író vég felesleges */
        char buf[32];
        ssize_t n = read(fd[0], buf, sizeof(buf) - 1);
        if (n > 0) {
            buf[n] = '\0';
            printf("gyermek kapta: %s\n", buf);
        }
        close(fd[0]);
        return 0;
```

```
    } else {
        /* Szülő: író oldal */
        close(fd[0]);                /* az olvasó vég felesleges */
        const char *msg = "hello\n";
        write(fd[1], msg, strlen(msg));
        close(fd[1]);                /* EOF jelzése az olvasónak */
        wait(NULL);
        return 0;
    }
}
```

A példában a `fork` után a gyermek örökli a pipe fájlleíróit. Mindkét fél azonnal bezárja a számára felesleges véget, így elkerülhető a fájlleíró-szivárgás, és az olvasó oldal pontosan akkor kap `EOF`-ot (nulla visszatérésű `read`), amikor minden író vég lezárult. Ha a gyermek nem zárná be az író véget, az olvasó `read` hívása a végtelenségig blokkolna, mert a kernel nem tudná, hogy nem érkezik több adat.

A pipe-ra vonatkozó fontos szemantikai szabályok:

- A `write` hívás `PIPE_BUF` méretig (Linuxon általában 4096 bájt) atomi: a többi író nem ékelődhet közbe.
- Ha az olvasó vég minden másolata zárva van, az írás `SIGPIPE` szignált vált ki, és a `write` `-1` értékkel, `EPIPE` hibával tér vissza.
- A `read` blokkol, amíg nincs adat, vagy az író vég nyitva van. Ha minden író vég zárva van, a `read` `0` értékkel tér vissza.
- A pipe nem támogatja a véletlen hozzáférést (`lseek`), és nincs üzenethatár: a bájtfolyam tetszőlegesen darabolható.

A shell pipeline-ok is ezt a mechanizmust használják: a `ls | wc -l` parancsban a shell létrehoz egy pipe-ot, majd a két folyamat szabványos be- és kimenetét a pipe végpontjaira irányítja.

7.4.3 FIFO: nevesített cső

A névtelen pipe csak közös őssel rendelkező folyamatok között használható, mert a fájlleírókat öröklés útján lehet átadni. Ha két független folyamatnak kell kommunikálnia, nevesített csőre (FIFO) van szükség.

A FIFO egy speciális fájl a fájlrendszerben, amelyet a `mkfifo` rendszerhívással vagy paranccsal hozunk létre:

```
int mkfifo(const char *pathname, mode_t mode);
```

A `pathname` a FIFO elérési útja, a `mode` a jogosultságokat adja meg. A hívás `0` értékkel tér vissza siker esetén, hiba esetén `-1` és `errno` beállításra kerül. A létrehozott FIFO nem tárol adatot a lemezen; csupán egy kernelbeli csatorna nevesített belépési pontja.

A használat során a folyamatok a szokásos `open` hívással nyitják meg a FIFO-t:

- `O_RDONLY` nyitás blokkol, amíg egy író is meg nem nyitja.
- `O_WRONLY` nyitás blokkol, amíg egy olvasó is meg nem nyitja.
- Ha `O_NONBLOCK` flaget adunk meg, a nyitás és a későbbi `read`/`write` hívások nem blokkolnak; a pontos viselkedés ettől függ.

A FIFO előnye, hogy bármely folyamat, amelyik ismeri az elérési utat és jogosult a hozzáférésre, csatlakozhat a csatornához. Hátránya, hogy a fájlrendszerben maradó név miatt az életciklus-kezelésre külön figyelmet kell fordítani, és a jogosultságok ellenőrzése a fájlrendszer szabályai szerint történik.

7.4.4 Üzenetsorok

Az üzenetsorok rekordalapú kommunikációt biztosítanak. Ellentétben a pipe-pal, ahol a bájtfolyam tetszőlegesen darabolható, az üzenetsor megőrzi az üzenethatárokat: az olvasó pontosan egy üzenetet kap, amit az író egy hívásban küldött. Ez a tulajdonság akkor hasznos, ha a kommunikáció természetes egységei strukturált rekordok, például naplóbejegyzések vagy vezérlőparancsok.

Két fő üzenetsor-implementáció létezik: a POSIX üzenetsor és a System V üzenetsor. Mindkettő a kernelben tárolja az üzeneteket, de az interfészük és a névterük eltér.

7.4.5 POSIX üzenetsor

A POSIX üzenetsort az `mq_open` hívással hozzuk létre vagy nyitjuk meg:

```
mqd_t mq_open(const char *name, int oflag, mode_t mode,
              struct mq_attr *attr);
```

A `name` egy `/` karakterrel kezdődő, fájlrendszer-szerű név, például `/app_logq`. Az `oflag` lehet `O_CREAT`, `O_EXCL`, `O_RDONLY`, `O_WRONLY`, `O_RDWR` és `O_NONBLOCK`. A `mode` a létrehozáskor érvényes jogosultságokat adja meg. Az `attr` opcionális, ebben adható meg a maximális üzenetszám (`mq_maxmsg`) és az egy üzenet maximális mérete (`mq_msgsize`). A visszatérési érték egy üzenetsor-leíró (`mqd_t`), amely sikertelen hívás esetén `(mqd_t)-1` és `errno` beállítás.

Az üzenetküldés és -fogadás:

```
int mq_send(mqd_t mqdes, const char *msg_ptr, size_t msg_len,
            unsigned int msg_prio);
ssize_t mq_receive(mqd_t mqdes, char *msg_ptr, size_t msg_len,
                   unsigned int *msg_prio);
```

A `mq_send` a `msg_ptr` által mutatott `msg_len` méretű puffert egyetlen üzenetként helyezi el a sorban. A `msg_prio` prioritás szerint rendezi az üzeneteket: a nagyobb értékű üzenet hamarabb kerül kiolvasásra. A `mq_receive` kiveszi a legmagasabb prioritású üzenetet. Ha a puffer mérete kisebb, mint az üzenet, a hívás `EMSGSIZE` hibával tér vissza. Ha a sor üres és a leíró nem nemblokkoló, a `mq_receive` blokkol.

A lezárás és a név eltávolítása:

```
int mq_close(mqd_t mqdes);
int mq_unlink(const char *name);
```

Az `mq_close` bezárja a leírót, de a sor a kernelben megmarad, amíg `mq_unlink`-kel nem távolítjuk el. Az `mq_unlink` után az újabb `mq_open` hívások nem találják a sort, de a már megnyitott leírók tovább használhatják.

Példa egy egyszerű üzenetküldésre:

```
#include <mqueue.h>
#include <stdio.h>
#include <string.h>

int main(void) {
    struct mq_attr attr = {0};
    attr.mq_maxmsg = 10;
    attr.mq_msgsize = 64;

    mqd_t mq = mq_open("/pelda_q", O_CREAT | O_RDWR, 0600, &attr);
    if (mq == (mqd_t)-1) {
        perror("mq_open");
```

```
        return 1;
    }

    const char *uzenet = "log: rendben";
    if (mq_send(mq, uzenet, strlen(uzenet) + 1, 0) == -1) {
        perror("mq_send");
        return 1;
    }

    char puffer[64];
    unsigned int prio = 0;
    ssize_t n = mq_receive(mq, puffer, sizeof(puffer), &prio);
    if (n >= 0) {
        printf("fogadva: %s\n", puffer);
    }

    mq_close(mq);
    mq_unlink("/pelda_q");
    return 0;
}
```

A példában a `strlen + 1` azért szerepel, hogy a lezáró nullkarakter is az üzenet része legyen; ez lehetővé teszi a közvetlen printf használatát. Nagyobb rendszerekben gyakran bináris rekordokat küldenek, ilyenkor nincs szükség nullterminátorra.

7.4.6 System V üzenetsor

A System V üzenetsorok a POSIX-tól eltérő, régebbi interfészt használnak. A sort egy kulcs azonosítja:

```
int msgget(key_t key, int msgflg);
```

A `key` gyakran `ftok()` segítségével generálható, vagy konstans. A `msgflg` tartalmazza a jogosultságokat és az `IPC_CREAT` flaget. A visszatérési érték egy sorazonosító. Az üzenetküldés:

```
int msgsnd(int msqid, const void *msgp, size_t msgsz, int msgflg);
ssize_t msgrcv(int msqid, void *msgp, size_t msgsz, long msgtyp, int msgflg);
```

A `msgp` egy olyan struktúrára mutat, amelynek első mezője `long mtype` típusazonosító. Az `msgsnd` a `msgsz` hosszúságú adatrészt küldi el, a `msgrcv` pedig a `msgtyp` alapján választ üzenetet. Az `msgctl` hívással lehet törölni a sort vagy lekérdezni az állapotát.

Bár a System V üzenetsorok még megtalálhatók a Linux rendszerekben, új fejlesztésekben általában a POSIX interfészt vagy a socketeket érdemes előnyben részesíteni.

7.4.7 Osztott memória

Az osztott memória az egyetlen olyan IPC-mechanizmus, amely nem másolja az adatokat a kernel és a felhasználói tér között. A folyamatok ugyanazt a fizikai memóriaterületet képezik le a saját virtuális címterükbe, így az egyik folyamat által írt érték azonnal láthatóvá válik a másik számára. Ez nagy adatmennyiség vagy alacsony késleltetés esetén a leghatékonyabb választás.

A POSIX osztott memória létrehozása:

```
int shm_open(const char *name, int oflag, mode_t mode);
```

A `name` itt is `/` karakterrel kezdődő név. A hívás egy fájlleírót ad vissza, amelyet ezután `ftruncate`-tel a kívánt méretűre kell állítani, majd `mmap`-pel a címterünkbe kell képezni:

```
int ftruncate(int fd, off_t length);
void *mmap(void *addr, size_t length, int prot, int flags,
           int fd, off_t offset);
```

A `mmap` `PROT_READ | PROT_WRITE` védelemmel és `MAP_SHARED` flaggel a megosztott lapokat hozza létre. A `shm_unlink` a név eltávolítására szolgál.

Példa osztott memória létrehozására és írására:

```
#include <sys/mman.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <unistd.h>
#include <string.h>
#include <stdio.h>

int main(void) {
    int fd = shm_open("/pelda_shm", O_CREAT | O_RDWR, 0600);
    if (fd == -1) {
        perror("shm_open");
        return 1;
    }

    if (ftruncate(fd, 4096) == -1) {
        perror("ftruncate");
        return 1;
    }

    char *ptr = mmap(NULL, 4096, PROT_READ | PROT_WRITE,
                     MAP_SHARED, fd, 0);
    if (ptr == MAP_FAILED) {
        perror("mmap");
        return 1;
    }

    strcpy(ptr, "osztott memória üzenet");
    printf("írva: %s\n", ptr);

    munmap(ptr, 4096);
    close(fd);
    shm_unlink("/pelda_shm");
    return 0;
}
```

Az osztott memória legnagyobb kockázata a versenyhelyzet. Ha két folyamat egyszerre írja ugyanazt a területet, az eredmény nem definiált. Ezért az osztott memóriát mindig valamilyen szinkronizációs mechanizmussal kell összekapcsolni: például POSIX szemafórral vagy egy közös mutexszel. A szinkronizáció nélküli osztott memória használata az egyik leggyakoribb és legnehezebben felderíthető hibák forrása.

A System V osztott memória interfész a `shmget`, `shmat`, `shmdt` és `shmctl` hívásokból áll. A POSIX `shm_open` + `mmap` megközelítés általában rugalmasabb, mert a fájlleíró-alapú modellbe illeszkedik, és lehetővé teszi a jogosultságok és az életciklus egyszerűbb kezelését.

7.4.8 Szignálok

A szignálok aszinkron értesítési mechanizmust adnak a folyamatok számára. Egy szignál nem szállít nagy adatmennyiséget, csupán egy esemény bekövetkeztét jelzi: például a felhasználó megnyomta a `Ctrl+C` billentyűkombinációt (`SIGINT`), egy gyermekfolyamat befejeződött (`SIGCHLD`),

vagy egy időzítő lejárt (`SIGALRM`). A szignálokat a kernel küldi a folyamatnak, de egy folyamat a `kill` vagy `raise` rendszerhívással más folyamatnak is küldhet.

A szignálkezelés alapvető interfészei:

```
void (*signal(int signum, void (*handler)(int)))(int);
int kill(pid_t pid, int sig);
int raise(int sig);
```

A `signal` a `signum` jelhez rendel egy handler függvényt. A `kill` a `pid` azonosítójú folyamatnak küld `sig` jelet; a `raise` a saját folyamatnak küld. A `signal` hordoz néhány platformfüggő viselkedést, ezért új kódban a `sigaction` használata javasolt:

```
int sigaction(int signum, const struct sigaction *act,
              struct sigaction *oldact);
```

A `struct sigaction` tartalmazza a handler függvényt (`sa_handler` vagy `sa_sigaction`), a kezelés közben blokkolandó szignálok halmazát (`sa_mask`) és a viselkedést módosító flageket (`sa_flags`). A `sigaction` lehetővé teszi a pontos szemantika beállítását, például a `SA_RESTART` flaget, amely a megszakított rendszerhívások automatikus újraindítását kéri.

A szignálkezelő függvényre szigorú szabályok vonatkoznak. Mivel a handler aszinkron módon, a folyamat bármely pontján lefuthat, csak async-signal-safe függvényeket hívhat. A POSIX szabvány felsorolja ezeket; ilyen például a `write`, a `_exit` és a `sem_post`. A `printf`, a `malloc` vagy a `pthread_mutex_lock` nem biztonságos, mert nem reentránsak vagy belső állapotuk miatt holtpontot okozhatnak.

A gyakorlatban a legegyszerűbb és legbiztonságosabb minta az, hogy a handler csupán egy `volatile sig_atomic_t` flaget állít be, a fő program pedig a flaget figyeli:

```
#include <signal.h>
#include <unistd.h>
#include <stdio.h>

volatile sig_atomic_t erkezett = 0;

void handler(int sig) {
    (void)sig;
    erkezett = 1;
}

int main(void) {
    struct sigaction sa = {0};
    sa.sa_handler = handler;
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = 0;
    sigaction(SIGUSR1, &sa, NULL);

    printf("várok a SIGUSR1 jelre...\n");
    while (!erkezett) {
        pause();   /* szünetel a jel érkezéséig */
    }
    printf("jel megérkezett\n");
    return 0;
}
```

A példában a `pause` blokkol, amíg bármilyen szignál nem érkezik; a handler lefut, beállítja a flaget, majd a `pause` visszatér, és a ciklus kilép. A `volatile sig_atomic_t` típus garantálja, hogy a flag olvasása és írása atomi művelet a szignálkezelés szempontjából.

A szignálok fontos tulajdonsága, hogy nem sorolódnak. Ha ugyanabból a szignálból több érkezik, mielőtt a folyamat kezelné, a kernel csak egyet kézbesít belőle. Ezért a szignál nem alkalmas események pontos számlálására vagy nagy gyakoriságú értesítésekre. A `SIGKILL` és `SIGSTOP` jeleket nem lehet elkapni, blokkolni vagy figyelmen kívül hagyni; ezek mindig érvényesülnek.

7.4.9 Socketek és szinkronizáció

A socketek az IPC általánosabb eszközei. A `socketpair` hívással létrehozható egy teljes duplex, bájtsorozat-alapú csatorna, amely hasonló a pipe-hoz, de mindkét irányban használható:

```
int socketpair(int domain, int type, int protocol, int sv[2]);
```

A `domain` általában `AF_UNIX`, a `type` pedig `SOCK_STREAM` (bájtsorozat) vagy `SOCK_DGRAM` (datagram). A `socketpair` két összekötött socket-leírót ad vissza. A pipe-pal ellentétben itt nincs külön író és olvasó vég: mindkét fél írhat és olvashat is.

A hálózati socketek (`AF_INET`, `AF_INET6`) lehetővé teszik az IPC-t akár különböző gépek között is, de ezzel a témával a hálózati stacket tárgyaló fejezet foglalkozik részletesen. Az `AF_UNIX` socketek a helyi gépen belüli kommunikációra szolgálnak, és a fájlrendszerben megjelenő elérési úttal is rendelkezhetnek.

A szinkronizációs primitívek – szemafórok, mutexek, feltételváltozók – önmagukban nem IPC-adatcsatornák, de elengedhetetlenek az osztott memória és más közös erőforrások védelméhez. A POSIX szemafór például a `sem_open`, `sem_wait`, `sem_post` és `sem_close` függvényekkel kezelhető. A szemafór egy számláló, amelyet a `sem_wait` eggyel csökkent (ha az érték nulla, blokkol), a `sem_post` pedig eggyel növel, és felébreszt egy várakozó folyamatot.

A termelő-fogyasztó minta tipikus példája az osztott memória és a szemafór kombinációjának. A termelő az osztott pufferbe ír, majd `sem_post`-tal jelzi, hogy új elem áll rendelkezésre. A fogyasztó `sem_wait`-tel várakozik, majd kiolvassa a puffert. A szemafór biztosítja, hogy a fogyasztó csak akkor olvasson, ha a termelő már befejezte az írást.

7.4.10 Mechanizmusválasztás

A megfelelő IPC-mechanizmus kiválasztása a következő szempontok mérlegelését igényli:

- **Adatmennyiség és gyakoriság:** Nagy adatforgalom esetén az osztott memória kerüli a felesleges másolást, de szinkronizációt igényel. Kis üzenetekre a pipe vagy üzenetsor egyszerűbb és biztonságosabb.
- **Adatszerkezet:** Ha az üzenethatárok fontosak, az üzenetsor természetes választás. Ha bájtsorozatot kell átvinni, a pipe vagy socket illik.
- **Folyamatok kapcsolata:** Névtelen pipe csak közös őssel rendelkező folyamatok között használható. Független folyamatok esetén FIFO, üzenetsor, osztott memória névvel vagy socket szükséges.
- **Élettartam:** Az üzenetsor és az osztott memória objektumai a kernelben tovább élnek a létrehozó folyamat befejeződése után is, amíg explicit módon el nem távolítják őket. A pipe és a socket automatikusan felszabadul az utolsó fájlleíró lezárásakor.
- **Szinkronizációs igény:** Az osztott memória önmagában nem szinkronizál. Ha az adatcsere mellett eseményjelzésre is szükség van, kombinálni kell szignállal, szemafórral vagy pipe-pal.
- **Hordozhatóság:** A POSIX interfészek (`pipe`, `mkfifo`, `shm_open`, `mq_open`, `socketpair`) széles körben támogatottak. A System V interfészek régebbiek, de még mindig elérhetők.

A gyakorlatban egy egyszerű pipeline-hoz pipe-ot, egy naplózó démonhoz üzenetsort vagy socketet, nagy puffer megosztásához pedig osztott memóriát szemafórral érdemes választani.

7.4.11 Tipikus hibák és megelőzésük

Az IPC-programozás során több visszatérő hiba jellemző:

1. **Fájlleíró-szivárgás pipe esetén.** Ha a gyermek nem zárja be a felesleges író véget, az olvasó `read` soha nem kap `EOF`-ot, és a program végtelenségig blokkol. Mindig zárjuk be a nem használt végeket, lehetőleg közvetlenül a `fork` után.
2. **Részleges írás és olvasás.** A `write` és `read` nem garantálja, hogy a teljes puffer átvitelre kerül. A visszatérési értéket mindig ellenőrizni kell, és ciklusban kell folytatni a műveletet.
3. **Szignálkezelőben nem biztonságos függvény hívása.** A `printf`, `malloc` vagy bármely nem async-signal-safe függvény használata a handlerben holtponthoz vagy memóriahibához vezethet. Csak a szabvány által megengedett függvényeket használjuk, vagy csak flaget állítsunk.
4. **Osztott memória szinkronizáció nélkül.** Az adatverseny nem definiált viselkedést okoz. Mindig használjunk szemafórt, mutexet vagy más mechanizmust a hozzáférés összehangolására.
5. **Üzenetsor-erőforrások eltávolításának elmulasztása.** A POSIX üzenetsor és az osztott memória objektum a kernelben marad, ha elfelejtjük az `mq_unlink` vagy `shm_unlink` hívást. Ez fájlleíró- és memóriaszivárgáshoz hasonló problémákat okozhat.
6. **Blokkoló hívások holtpontja.** Ha például egy FIFO-t nyitunk olvasásra, és senki sem nyitja írásra, a program blokkol. A `O_NONBLOCK` flag vagy timeoutok használatával elkerülhető a végtelen várakozás.

7.4.12 Labor: IPC-mechanizmus kiválasztása

A labor célja, hogy egy adott forgatókönyvhöz válasszunk IPC-mechanizmust, és azt emulált környezetben, méréssel ellenőrizzük.

Forgatókönyv: Egy naplózó rendszerben egy termelő folyamat másodpercenként 1000 darab, egyenként 512 bájtos naplórekordot állít elő. Egy fogyasztó folyamat ezeket a rekordokat fogadja, és egy fájlba írja. A termelő és a fogyasztó ugyanazon a gépen fut, de független folyamatként, nem szülő-gyermek kapcsolatban. A rendszerrel szemben az a követelmény, hogy a rekordhatárok megmaradjanak, és a késleltetés a termelő és a fogyasztó között legfeljebb 1 ms legyen. Az adatvesztés nem megengedett.

Feladat: Válasszon a pipe, FIFO, POSIX üzenetsor és osztott memória közül, indokolja a választást, és mérje a késleltetést emulált környezetben.

Kiinduló állapot: QEMU virtuális gép, 1 vCPU, 512 MiB RAM, Linux kernel 6.1, glibc 2.36, gcc 12.2. A méréshez a `clock_gettime(CLOCK_MONOTONIC)` hívást használjuk, és a termelő minden rekordhoz rögzíti a küldés időbélyegét, a fogyasztó pedig a fogadásét. A késleltetést a két időbélyeg különbsége adja.

Build- és futtatási parancs: A programot C nyelven írjuk, `gcc -O2 -Wall -Wextra` fordítási beállításokkal. A futtatás a QEMU vendégben történik: `./ipc_lab`.

Elvárt eredmény: A mért átlagos késleltetés legfeljebb 1 ms, és a rekordok 100%-a megérkezik. A mérési naplóban rögzíteni kell a hardvert, a fordítási beállításokat, az órajelet és a mérési módszert.

A mért és a várt eredmény elkülönítése: A labor jegyzőkönyvben külön oszlopban szerepeljen a számított/elvárt késleltetési korlát (1 ms) és a ténylegesen mért átlag, minimum, maximum, valamint a szórás. A mért értékeket ne kerekítsük a várt értékre; az eltérés elemzése a feladat része.

Visszaállítás: A QEMU vendég leállítása után a snapshot visszaállítása biztosítja a tiszta kiinduló állapotot. A labor nem módosítja a hoszt rendszer lemezét vagy bootloaderét.

Megjegyzés a megoldáshoz: A forgatókönyvben a rekordhatárok megőrzése miatt a pipe és a FIFO kevésbé alkalmas, mert a bájtsorozatot magunknak kellene darabolni. Az osztott memória nagy sebességet kínál, de szinkronizációt igényel. A POSIX üzenetsor pontosan rekordalapú, és a kernel kezeli a határokat. A labor során érdemes megmérni az üzenetsor és egy explicit keretezést használó pipe vagy FIFO késleltetését is, hogy a választás mérési adatokkal legyen alátámasztva.

7.4.13 Önellenőrző kérdések

1. Miért kap `EOF`-ot az olvasó egy pipe-on, ha az író bezárja a végét, és mi a szerepe a felesleges fájlleírók lezárásának a `fork` után?
2. Mi a különbség a pipe és a FIFO között a folyamatok kapcsolata és a létrehozás módja szempontjából?
3. Miért van szükség szinkronizációra osztott memória használatakor, és milyen primitívekkel oldható meg?
4. Miért nem hívhatunk `printf` függvényt egy szignálkezelőben?
5. Milyen szempontok alapján választana üzenetsort a pipe helyett?

A kérdésekre adott válaszok a fejezet anyagából levezethetők, és a labor során szerzett tapasztalatokkal mélyíthetők.

7.4.14 Összegzés

Ez az alfejezet bemutatta a folyamatok közötti kommunikáció fő mechanizmusait: a pipe-ot, a FIFO-t, az üzenetsorokat, az osztott memóriát, a szignálokat és a socketeket. Mindegyik más-más szemantikát és teljesítményprofil kínál, ezért a választás a konkrét feladat követelményeitől függ. A következő egység az ütemezési célokkal és algoritmusokkal foglalkozik, ahol az itt megismert folyamat- és szálfogalmakra építünk. Az IPC mechanizmusainak helyes használata a későbbi hálózati és biztonsági fejezetekben is alapvető fontosságú.

8 Ütemezés

8.1 Ütemezési célok és metrikák

Az ütemező elsődleges feladata, hogy a rendelkezésre álló CPU-időt a futtatható folyamatok vagy szálak között elossza. Ahhoz, hogy a döntéseit értékelni tudjuk, számszerűsíthető célokra és mérőszámokra van szükség. Ez az alfejezet bevezeti az ütemezés öt alapfogalmát: CPU-kihasználtságot, átbocsátóképességet, átfutási időt, válaszidőt és méltányosságot, majd egy kis mintaterhelésen mutatja be a számításukat. A mérőszámok ismerete nélkül nem lehet megalapozottan összehasonlítani az ütemezési algoritmusokat, és nem lehet észrevenni azokat a hibákat sem, amelyek egyetlen „jól hangzó” szám mögött rejtőznek.

8.1.1 Miért van szükség ütemezési metrikákra?

Az operációs rendszer ütemezője folyamatosan ugyanazzal a kérdéssel szembesül: melyik futtatható folyamat vagy szál kapja meg legközelebb a processzort? A válasz egy ütemezési politika, amelyet az adott rendszer céljai határoznak meg. Egy batch-szerveren más a fontos, mint egy interaktív asztali gépen vagy egy valós idejű beágyazott vezérlőn. A célok azonban minden esetben mérőszámokkal fejezhetők ki, és ezek a mérőszámok gyakran ütköznek egymással. Egyetlen ütemező sem lehet egyszerre minden szempontból optimális, ezért először tisztázni kell, mit mérünk, és miért.

A mérőszámok két nagy csoportba sorolhatók attól függően, hogy a rendszer egészét vagy az egyes folyamatokat vizsgáljuk. A rendszerszintű metrikák közé tartozik a CPU-kihasználtság és az átbocsátóképesség. A folyamatszintű metrikák közé tartozik a teljes átfutási idő, a várakozási idő, a válaszidő és a méltányosság. A prioritás fogalma pedig ezeket a szempontokat rendezi fontossági sorrendbe, tehát a méltányosság és az előnyben részesítés közötti feszültséget is kezelni kell.

8.1.2 CPU-kihasználtság

A CPU-kihasználtság azt mutatja meg, hogy a megfigyelt időintervallum mekkora hányadában végzett a processzor tényleges hasznos munkát, szemben azzal, amikor üresjáratban várakozott. Formálisan, ha a megfigyelési intervallum hossza Δt, és ezalatt a folyamatok összesen $\sum_{i=1}^{n} C_i$ időegységet futnak a CPU-n, akkor

$$U = \frac{\sum_{i=1}^{n} C_i}{\Delta t}.$$

A Δt intervallumot általában az első vizsgált folyamat érkezése és az utolsó folyamat befejezése között mért időként definiáljuk, de hosszú távú, stabil rendszerben tetszőleges megfigyelési ablak is választható.

Fontos, hogy a CPU-kihasználtság értéke önmagában nem jelenti azt, hogy a rendszer jól működik. Ha egy folyamat hosszú, megszakítás nélküli ciklusban dolgozik, a CPU kihasználtsága elérheti a

100%-ot, miközben az interaktív felhasználó másodpercekig vár a válaszra. Ezért a kihasználtságot mindig más metrikákkal együtt kell értelmezni.

8.1.3 Átbocsátóképesség és átfutási idő

Az átbocsátóképesség (throughput) megmutatja, hogy időegységenként hány folyamatot vagy feladatot fejez be a rendszer. Ha a megfigyelési intervallumban n folyamat érkezik és valamennyi be is fejeződik, akkor

$$X = \frac{n}{\Delta t}.$$

A mérőszám dimenziója tehát „feladat/idő”, például folyamat per másodperc. Stabil, hosszú távon fennálló terhelésnél az átbocsátóképességnek meg kell egyeznie az érkezési rátával, különben a rendszerben torlódás alakul ki.

Az átfutási idő (turnaround time) egyetlen folyamat szempontjából az érkezéstől a befejezésig eltelt teljes idő:

$$T_i = F_i - A_i,$$

ahol A_i az érkezési idő, F_i pedig a befejezési idő. Az átfutási idő a batch-feldolgozás és a háttérfeladatok esetében a legfontosabb metrika, mert azt fejezi ki, mennyit kell várni a végeredményre. Az átlagos átfutási idő

$$\bar{T} = \frac{1}{n} \sum_{i=1}^{n} T_i.$$

Az átfutási időből közvetlenül származtatható a várakozási idő (waiting time), amely az az idő, amíg a folyamat a CPU-ra várt, tehát nem futott, de nem is fejeződött be:

$$W_i = T_i - C_i,$$

ahol C_i a folyamat teljes CPU-igénye. A várakozási idő pontosabb képet ad az ütemező hatékonyságáról, mert kiszűri a folyamat tényleges futási idejét, amelyet az ütemező nem tud csökkenteni.

8.1.4 Válaszidő és interaktivitás

Interaktív rendszerekben a felhasználó nem a feladat befejezésére kíváncsi, hanem arra, hogy a billentyűleütés vagy egérkattintás után milyen gyorsan kap visszajelzést. Ezt a válaszidő (response time) méri, amely az érkezéstől az első CPU-hozzáférés kezdetéig eltelt idő:

$$R_i = S_i - A_i,$$

ahol S_i az a pillanat, amikor a folyamat először kap processzort. Preemptív ütemezésnél a válaszidő rövid lehet akkor is, ha a folyamat teljes befejezése sokáig tart, mert a rendszer már az első részlet kiszolgálásával visszajelzést ad a felhasználónak.

A válaszidő és az átfutási idő fogalmát gyakran összekeverik. A válaszidő az első futásig tart, az átfutási idő pedig a teljes befejezésig. Egy hosszú, háttérben futó folyamatnak lehet kiváló válaszideje és mégis nagy átfutási ideje. Az interaktív rendszerek ütemezőit elsősorban a válaszidőre optimalizálják, míg a batch-rendszerek az átlagos átfutási időt tekintik elsődleges célnak.

8.1.5 Méltányosság és prioritás

A méltányosság (fairness) azt méri, hogy az erőforrásokat mennyire egyenletesen osztja el az ütemező a versengő folyamatok között. A legegyszerűbb mutató a kapott CPU-idők viszonya: ha minden folyamat azonos erőforrást kap, akkor a rendszer teljesen méltányos. A gyakorlatban a folyamatok eltérő fontosságúak, ezért a súlyozott méltányosság az elterjedt.

A méltányosság számszerűsítésére használható a Jain-index, amely $x_1, \ldots, x_n$ erőforrás-részesedések esetén:

$$J(x_1, \ldots, x_n) = \frac{\left(\sum_{i=1}^{n} x_i\right)^2}{n \sum_{i=1}^{n} x_i^2}.$$

Az index értéke 1, ha minden folyamat pontosan ugyanannyit kap, és $\frac{1}{n}$-hez közelít, ha az erőforrás nagyon egyenlőtlenül oszlik el. A mutató alkalmazható CPU-időre, átfutási időre vagy normalizált átfutási időre is, attól függően, hogy melyik erőforrást tekintjük méltányosság szempontjából kritikusnak.

A prioritás (p_i) az ütemező azon szabálya, amely bizonyos folyamatokat előnyben részesít másokkal szemben. A prioritás értéke lehet rögzített (például rendszerfolyamatok magasabb prioritást kapnak) vagy dinamikus (például az ütemező csökkenti a hosszú ideje futó folyamat prioritását). A prioritás bevezetése szándékosan sérti az egyenlő méltányosságot, ezért a kérdés az, hogy a rendszer megtart-e valamilyen súlyozott méltányosságot, vagy az alacsony prioritású folyamatok éhezésnek vannak kitéve.

Az éhezés (starvation) következménye lehet a rosszul beállított prioritásos ütemezésnek: egy folyamat soha nem jut CPU-hoz, mert mindig van nála magasabb prioritású versenytárs. Az éhezés nem azonos a várakozással; a várakozás véges, az éhezés pedig tartósan megakadályozza a haladást.

8.1.6 Mintapélda: metrikák számítása egy kis munkaterhelésen

A következőkben tekintsünk három folyamatot az alábbi paraméterekkel:

Folyamat	Érkezési idő A_i	CPU-idő C_i	Prioritás p_i
P_1	0	5	3
P_2	1	2	1
P_3	2	3	2

A prioritás értelmezése legyen az, hogy a nagyobb szám magasabb prioritást jelöl. Az ütemező egy lehetséges, nem preemptív döntése például az, hogy a folyamatokat P_1, P_2, majd P_3 sorrendben futtatja. A hozzá tartozó Gantt-diagram:

```
0         5    7    10
|----P1----|--P2--|--P3--|
```

A diagram feltételei: a folyamatok egyszerre legfeljebb egy lépésben futnak, a kontextusváltás időtartamát elhanyagoljuk, az ütemező nem szakítja meg a futó folyamatot. A számítások a következők.

A befejezési idők: $F_1 = 5$, $F_2 = 7$, $F_3 = 10$. Az átfutási idők:

$$T_1 = 5 - 0 = 5, \quad T_2 = 7 - 1 = 6, \quad T_3 = 10 - 2 = 8.$$

Az átlagos átfutási idő tehát

$$\bar{T} = \frac{5 + 6 + 8}{3} = \frac{19}{3} \approx 6{,}33.$$

A várakozási idők:

$$W_1 = 5 - 5 = 0, \quad W_2 = 6 - 2 = 4, \quad W_3 = 8 - 3 = 5.$$

A válaszidők nem preemptív ütemezésnél megegyeznek a folyamatok kezdő várakozási idejével:

$$R_1 = 0 - 0 = 0, \quad R_2 = 5 - 1 = 4, \quad R_3 = 7 - 2 = 5.$$

A teljes megfigyelési intervallum $\Delta t = 10 - 0 = 10$. A CPU-kihasználtság:

$$U = \frac{5 + 2 + 3}{10} = \frac{10}{10} = 1{,}0,$$

ami azt jelenti, hogy a CPU-nak nem volt üresjárata. Az átbocsátóképesség:

$$X = \frac{3}{10} = 0{,}3 \quad \text{folyamat/időegység.}$$

A méltányosság vizsgálatához számítsuk ki a Jain-indexet a kapott CPU-időkre. Az erőforrás-részesedések 5, 2, 3. Ekkor

$$J_{\text{CPU}} = \frac{(5 + 2 + 3)^2}{3(5^2 + 2^2 + 3^2)} = \frac{10^2}{3(25 + 4 + 9)} = \frac{100}{114} \approx 0{,}877.$$

Az érték kisebb 1-nél, mert a folyamatok CPU-igénye eltérő. Ha ugyanezt a mutatót az átfutási időkre számítjuk:

$$J_T = \frac{(5 + 6 + 8)^2}{3(5^2 + 6^2 + 8^2)} = \frac{19^2}{3(25 + 36 + 64)} = \frac{361}{375} \approx 0{,}963.$$

Itt a méltányosság jobb, mert a rövid folyamatok későbbi érkezése részben ellensúlyozta a CPU-időbeli különbséget. Ez a kettősség jól mutatja, hogy a méltányosság értéke attól függ, milyen erőforráson mérjük.

Mi a szerepe ebben a példában a prioritásnak? A megadott sorrend P_1, P_2, P_3 éppen a prioritások szerint csökkenő. Ha az ütemező kizárólag a prioritást követi, akkor is ezt a futási sorrendet

kapjuk. A prioritás tehát befolyásolja, hogy a magas prioritású P_1 azonnal futhat, és rövid átfutási időt ér el; a legalacsonyabb prioritású P_2 viszont várakozik. Az éhezés itt nem lép fel, mert a folyamatok véges számúak, de hosszabb, folyamatosan érkező magas prioritású terhelés mellett az alacsony prioritású folyamat akár soha nem jutna szóhoz.

8.1.7 Átlagok és eloszlások: miért veszélyes egyetlen számra hagyatkozni?

Az átlagos átfutási idő és az átlagos várakozási idő praktikus mutató, de elrejti a szélsőségeket. Két ütemező azonos átlagos átfutási időt produkálhat úgy, hogy az egyik minden folyamatot hasonlóan kezel, a másik pedig néhány folyamatot rendkívül gyorsan, másokat viszont rendkívül lassan szolgál ki. Az utóbbi rendszer interaktív szempontból elfogadhatatlan lehet, miközben a batch-mutatója jónak látszik.

Ezért az átlag mellett mindig célszerű megvizsgálni a legrosszabb esetet és a szórást is. A legrosszabb esetű átfutási időt a következőképpen definiálhatjuk:

$$T_{\max} = \max_i T_i,$$

a legrosszabb esetű várakozási időt pedig:

$$W_{\max} = \max_i W_i.$$

A fenti mintapéldában $\bar{T} = 6{,}33$, $T_{\max} = 8$, $W_{\max} = 5$. Egy másik ütemezési sorrend, például ha a rövid P_2-t részesítjük előnyben, csökkenthetné az átlagot, de ehhez a CPU-nak üresjáratot kellene vállalnia, mert P_2 csak az 1 időpontban érkezik, míg P_1 már a 0 időpontban rendelkezésre áll. Ez a példa rávilágít arra, hogy a metrikák között feszültség van: a kihasználtság növelése néha rontja a rövid folyamatok kiszolgálását.

8.1.8 Normalizált átfutási idő és súlyozott méltányosság

A nyers átfutási időt nehéz összehasonlítani olyan folyamatok között, amelyek CPU-igénye nagyon eltérő. Egy 100 időegységnyi CPU-időt igénylő folyamat 110 időegységes átfutása kiváló lehet, míg egy 1 időegységnyi CPU-időt igénylő folyamat 11 időegységes átfutása rossz. A normalizált átfutási idő ezt a torzítást küszöböli ki:

$$T_i^{\text{norm}} = \frac{T_i}{C_i}.$$

Az érték minimálisan 1, mert minden folyamatnak legalább annyi ideig kell a rendszerben lennie, amennyi a CPU-futása. Minél közelebb van 1-hez, annál kevesebbet várt a folyamat a CPU-ra. A mintapéldában:

$$T_1^{\text{norm}} = \frac{5}{5} = 1{,}0, \quad T_2^{\text{norm}} = \frac{6}{2} = 3{,}0, \quad T_3^{\text{norm}} = \frac{8}{3} \approx 2{,}67.$$

A P_2 esetében a normalizált átfutási idő a legnagyobb, noha a nyers várakozási ideje közepes volt. Ez azért van, mert a rövid folyamatoknál ugyanakkora várakozási idő nagyobb arányú késedelmet jelent.

A súlyozott méltányosság általában a normalizált átfutási idők vagy a kapott CPU-idők és a hozzájuk rendelt súlyok viszonyát méri. A Jain-index könnyen kiterjeszthető súlyozott esetre, de a bevezető tárgyalásban elegendő annyit rögzíteni, hogy a prioritásos rendszerek méltányossága csak akkor értelmes, ha a prioritásokat mint súlyokat explicit módon belevesszük az értékelésbe.

8.1.9 Gyakori hibák és félreértések

Az ütemezési metrikák értelmezésekor visszatérő hibák a következők:

1. **A CPU-kihasználtság és az átbocsátóképesség összekeverése.** Egy rendszer lehet 100%-os kihasználtságú úgy, hogy közben kevés folyamatot fejez be, mert minden folyamat hosszú futású. Az átbocsátóképesség a befejezések számát méri, a kihasználtság pedig az idő kihasználását.

2. **Az átfutási idő és a válaszidő összekeverése.** Batch-rendszerben az átfutási idő a mérvadó; interaktív rendszerben a válaszidő. Ha egy interaktív alkalmazásnál csak az átlagos átfutási időt optimalizáljuk, a felhasználói élmény romolhat.

3. **Az átlagra való hagyatkozás a szélsőségek vizsgálata nélkül.** Az átlag elfedi az éhezést, a nagy várakozási csúcsokat és az egyenetlen kiszolgálást. Mindig meg kell nézni a legrosszabb esetet és a Jain-indexet is.

4. **A fairness és az egyenlőség azonosítása.** A prioritásos rendszerben a folyamatok nem egyenlők, ezért a cél nem az egyenlő elosztás, hanem az, hogy a prioritásoknak megfelelő, megmagyarázható és korlátos eltérés legyen.

5. **A normalizált átfutási idő elhanyagolása.** Ha egy rövid folyamat sokat vár, az arányaiban rosszabb, mint ha egy hosszú folyamat várna ugyanannyit. A normalizált mutató teszi összehasonlíthatóvá a különböző méretű feladatokat.

6. **A mért és a várt érték összekeverése.** A számított, determinisztikus példában a várt értékek kézzel levezethetők. Egy szimuláció vagy emulált környezet kimenetét külön kell kezelni, és naplózni kell, hogy az eredmény a modellből következik-e, vagy mérési zajt is tartalmaz.

8.1.10 Labor: ütemezési metrikák számítása mintaterhelésen

Ez a labor reprodukálható jegyzetfüzetben végezhető el. A cél a fenti mintaterhelés metrikáinak kiszámítása egy pszeudokódban megadott eljárással, majd az eredmények összehasonlítása a kézzel levezetett várt értékekkel.

Kiinduló állapot. Hozzunk létre egy verziózott munkakönyvtárat `scheduling_metrics` néven. A jegyzetfüzet tartalmazzon egy bevezető szöveges cellát a konfigurációval, egy pszeudokód-cellát az alábbi eljárással, egy adatcellát a folyamatokkal és egy kiértékelő cellát a metrikákkal.

Bemeneti adatok. A folyamatok legyenek a fentiek:

```
folyamatok = [
    {azonosito: P1, erkezes: 0, cpu_ido: 5, prioritás: 3},
    {azonosito: P2, erkezes: 1, cpu_ido: 2, prioritás: 1},
    {azonosito: P3, erkezes: 2, cpu_ido: 3, prioritás: 2}
]
```

Az ütemezési sorrend legyen `[P1, P2, P3]`, nem preemptív.

Pszeudokód a metrikák számításához.

```
függvény szamit_metrikak(folyamatok, sorrend):
    n ← hossz(folyamatok)
    ido ← 0
    ossz_cpu ← 0
    kezdes, befejezes, valasz legyen n hosszú tömb nullákkal

    ciklus minden azonosito a sorrend-ben:
        p ← folyamatok[azonosito]
        ha ido < p.erkezes:
            ido ← p.erkezes
        kezdes[p.azonosito] ← ido
        valasz[p.azonosito] ← ido - p.erkezes
        ido ← ido + p.cpu_ido
        befejezes[p.azonosito] ← ido
        ossz_cpu ← ossz_cpu + p.cpu_ido

    delta_t ← max(befejezes) - min(erkezes)

    minden i-re:
        T[i] ← befejezes[i] - erkezes[i]
        W[i] ← T[i] - cpu_ido[i]

    U ← ossz_cpu / delta_t
    X ← n / delta_t
    atlag_T ← szumma(T) / n
    atlag_W ← szumma(W) / n
    atlag_R ← szumma(valasz) / n
    J_T ← (szumma(T))^2 / (n * szumma(T^2))

    vissza {U, X, atlag_T, atlag_W, atlag_R, J_T}
```

Elvárt eredmény. A jegyzetfüzet kimenetében szerepelnie kell az alábbi értékeknek:

Metrika	Várt érték
CPU-kihasználtság U	1,0
Átbocsátóképesség X	0,3 folyamat/időegység
Átlagos átfutási idő $\bar{T}$	6,33
Átlagos várakozási idő $\bar{W}$	3,0
Átlagos válaszidő $\bar{R}$	3,0
Jain-index az átfutási időkre J_T	0,963

Mérési napló. A jegyzetfüzet utolsó cellájában rögzítsük a futtatás időpontját, a jegyzetfüzet kernel verzióját és a kimenetet. A mért és a várt értékeket külön táblázatban tüntessük fel, például:

Metrika	Mért (jegyzetfüzet)	Várt (kézi levezetés)	Egyezés
U	1,0	1,0	igen
X	0,3	0,3	igen
$\bar{T}$	6,33	6,33	igen
J_T	0,963	0,963	igen

Visszaállítás. A labor nem módosítja a hoszt rendszer konfigurációját, nem ír rendszerfájlokat és nem használ valós hitelesítő adatokat. A visszaállítás a jegyzetfüzet kimeneteinek törléséből és a munkakönyvtár verziózott állapotba helyezéséből áll.

8.1.11 Önellenőrző kérdések

1. Egy rendszerben két folyamat fut. P_1 a 0 időpontban érkezik, CPU-ideje 10; P_2 a 2 időpontban érkezik, CPU-ideje 4. Az ütemező először P_2-t futtatja, majd P_1-et. Számítsa ki az átlagos átfutási időt, az átlagos várakozási időt és a CPU-kihasználtságot. Miért nem éri el a CPU-kihasználtság a 100%-ot ebben az ütemezésben?
2. Mi a különbség a válaszidő és az átfutási idő között? Mondjon egy példát arra, amikor egy folyamat válaszideje rövid, de az átfutási ideje nagy.
3. Adott három folyamat azonos CPU-idővel, de eltérő prioritással. Hogyan változik a Jain-index, ha a legmagasabb prioritású folyamat mindig megkapja a CPU-t, és az alacsony prioritásúak éheznek?

8.1.12 Összefoglalás

Ez az alfejezet bevezette az ütemezés értékelésének alapfogalmait: a CPU-kihasználtságot, az átbocsátóképességet, az átfutási időt, a várakozási időt, a válaszidőt és a méltányosságot. A mintapéldán lépésről lépésre megmutattuk, hogyan kell ezeket kiszámítani, és rámutattunk az átlagok és a prioritások értelmezésének buktatóira. A következő alfejezet ezekre a metrikákra épít, amikor a konkrét ütemezési algoritmusokat és azok összehasonlítását tárgyalja.

8.2 Alapvető algoritmusok: FIFO, SJF, RR és prioritás

Az Ütemezési célok és metrikák egység rögzítette az ütemezés alapvető céljait és metrikáit. Most négy klasszikus algoritmust vizsgálunk: a FIFO-t (First In First Out, más néven FCFS, First Come First Served), a legrövidebb feladatot (SJF – Shortest Job First), a körforgásos ütemezést (RR – Round Robin) és a prioritásos ütemezést. E négy algoritmus nemcsak a tankönyvi alap, hanem a valós ütemezők építőköve is: a modern Linux CFS, a Windows scheduler és a beágyazott RTOS-ek is visszavezethető döntési mintákat használnak, még ha a konkrét implementáció ennél bonyolultabb is.

Az egyszerűség kedvéért egyetlen CPU-t feltételezünk, és a folyamatok CPU-burstjeit ismertnek tekintjük, kivéve ahol kifejezetten jelöljük a becslést. A kontextusváltási költséget az elméleti elemzésben itt nullának vesszük, de a laborban a mérési naplóban dokumentálni kell, hogy az időzítés mit tartalmaz.

8.2.1 Jelölés és modell

A fejezetben a következő jelöléseket használjuk:

- P_i: az i-edik folyamat.
- a_i: az érkezési idő, amikor P_i kész állapotba kerül.
- C_i: a CPU-burst hossza, azaz a folyamat teljes futási igénye CPU-időben.
- p_i: a folyamat prioritása. Amennyiben nem jelezzük külön, a nagyobb p_i érték magasabb prioritást jelent; a laborban ez rögzített.
- B_i: a befejezési időpont.
- $T_i = B_i - a_i$: az átfutási idő.
- $W_i = T_i - C_i$: a várakozási idő, amit a folyamat a kész sorban tölt.

Egy adott ütemezés minőségét leggyakrabban az átlagos várakozási idővel és az átlagos átfutási idővel jellemezzük:

$$\overline{W} = \frac{1}{n}\sum_{i=1}^{n} W_i, \qquad \overline{T} = \frac{1}{n}\sum_{i=1}^{n} T_i.$$

Az ütemezési döntéseket Gantt-diagramon ábrázoljuk. A diagram egy vízszintes idővonal, amelyen minden szegmens egy folyamat futását jelöli. Például a

```
| P1 0-4 | P2 4-6 | P3 6-8 |
```

ábrázolás azt jelenti, hogy P_1 a 0. időponttól 4-ig fut, P_2 4-től 6-ig, P_3 pedig 6-tól 8-ig.

8.2.2 FIFO / FCFS

A FIFO ütemező a legegyszerűbb: a folyamatok abban a sorrendben kapják meg a CPU-t, amilyen sorrendben a kész sorba kerültek. Az ütemező nem szakítja meg a futó folyamatot, tehát **nem-preemptív**. A futó folyamat addig birtokolja a processzort, amíg a CPU-burstje be nem fejeződik.

8.2.3 Működés lépésenként

1. Az érkező folyamat a kész sor végére kerül.
2. A CPU felszabadulásakor az ütemező a kész sor elején álló folyamatot indítja.
3. A kiválasztott folyamat fut, amíg be nem fejeződik; közben az újonnan érkezők csak a sorba kerülnek.
4. A futó befejezése után az ütemező a sor élére lép, és a 2. lépéstől kezdve ismétel.

A FIFO megvalósítása egyszerű: egy láncolt sor és egy mutató elég. A beszúrás és a törlés konstans idejű, így a komplexitás $\mathbf{O(1)}$ műveletenként.

8.2.4 Példa

Legyen három folyamat, amelyek mind a 0. időpontban érkeznek:

- P_1: $C_1 = 5$
- P_2: $C_2 = 2$
- P_3: $C_3 = 1$

Ha a kész sorban P_1, P_2, P_3 sorrend alakul ki, a Gantt-diagram:

```
| P1 0-5 | P2 5-7 | P3 7-8 |
```

A várakozási idők:

- $W_1 = 0 - 0 = 0$
- $W_2 = 5 - 0 = 5$
- $W_3 = 7 - 0 = 7$

Az átlagos várakozási idő:

$$\overline{W} = \frac{0+5+7}{3} = \frac{12}{3} = 4,0.$$

Ha a sorrend P_3, P_2, P_1 lenne, a Gantt-diagram:

```
| P3 0-1 | P2 1-3 | P1 3-8 |
```

és a várakozási idők:

- $W_3 = 0$
- $W_2 = 1$
- $W_1 = 3$

Ekkor $\overline{W} = \frac{0+1+3}{3} = \frac{4}{3} \approx 1{,}33$.

A két sorrend összehasonlítása mutatja a FIFO legnagyobb problémáját: ha egy hosszú CPU-bursttel rendelkező folyamat kerül előre, az összes mögötte várakozó rövid folyamat késlekedik. Ezt **konvojhatásnak** (convoy effect) nevezzük: a rövid, kevés CPU-t igénylő folyamatok egyetlen hosszú folyamat mögött torlódnak fel. A jelenség nem elméleti: például egy kiírást végző vagy lassú I/O után hosszú CPU-burstbe kezdő folyamat mögé beékelődő interaktív processzek észrevehetően lassulhatnak.

A FIFO előnye, hogy nincs éhezés, feltéve, hogy minden folyamat idővel hozzájut a CPU-hoz, mert a sor egyszerűen elfogy. A válaszidő azonban rossz, és interaktív rendszerekhez nem megfelelő.

8.2.5 SJF és a preemptív változat (SRTF)

A legrövidebb feladat ütemezője (SJF) azt a kész folyamatot választja, amelynek a legkisebb a CPU-burstje. A nem-preemptív változat akkor dönt, amikor a CPU felszabadul, és a kész sorban lévő folyamatok közül választ.

8.2.6 Nem-preemptív SJF példa

Ugyanazzal a három folyamattal, amelyek mind a 0 időpontban érkeznek:

- P_1: $C_1 = 5$
- P_2: $C_2 = 2$
- P_3: $C_3 = 1$

A legrövidebb P_3, utána P_2, végül P_1 fut:

```
| P3 0-1 | P2 1-3 | P1 3-8 |
```

Az átlagos várakozási idő, mint fentebb, $\frac{0+1+3}{3} \approx 1{,}33$, ami itt a lehető legjobb.

8.2.7 Optimalitás

Az SJF nem-preemptív változata **optimális az átlagos várakozási időre nézve**, ha minden folyamat egyszerre érkezik. Ez egyszerű cserével belátható. Tegyük fel, hogy egy ütemezésben két szomszédos folyamat, P_a és P_b fut, ahol $C_a > C_b$, de P_a közvetlenül P_b előtt szerepel. Ha e kettőt felcseréljük, P_b befejezése korábbi, P_a befejezése ugyanannyit tolódik, amennyivel P_b korábban végzett, így a két folyamat együttes várakozási ideje nem nő, az utána következő folyamatoké pedig változatlan marad. Az ilyen cseréket ismételve minden rövid folyamatot a hosszabbak elé rendezhetünk, ezért a legrövidebbet előrehozó sorrend minimális átlagos várakozást ad.

A bizonyítás azonban csak azonos érkezési időkre érvényes. Eltérő érkezési időknél a preemptív változat adhat jobb eredményt.

8.2.8 Preemptív SJF: SRTF

A preemptív változatot **legrövidebb hátralévő idő** (Shortest Remaining Time First, SRTF) néven ismerjük. Új folyamat érkezésekor az ütemező összehasonlítja az érkező folyamat CPU-burstjét a futó folyamat hátralévő CPU-idejével. Ha az újonnan érkező hátralévő ideje kisebb, az ütemező megszakítja a futót, és az új folyamatot indítja.

Példa:

- P_1: $a_1 = 0$, $C_1 = 7$
- P_2: $a_2 = 2$, $C_2 = 3$
- P_3: $a_3 = 4$, $C_3 = 1$

A lépések:

1. $t = 0$: csak P_1 kész, P_1 fut.
2. $t = 2$: P_2 megérkezik. P_1 hátralévő ideje $7 - 2 = 5$, P_2 igénye 3, ezért P_2 fut.
3. $t = 4$: P_3 megérkezik. P_2 hátralévő ideje $3 - 2 = 1$, P_3 igénye 1. Holtverseny; a kisebb sorszám fut (konvenciónk szerint), tehát P_3 indul.
4. $t = 5$: P_3 kész. A kész folyamatok közül P_2 hátralévő ideje 1, ezért P_2 folytatja.
5. $t = 8$: P_2 kész. Már csak P_1 kész, amelynek hátralévő ideje 5, így P_1 fut 8 és 12 között? Ellenőrizzük: P_1 eddig 0–2 között futott, hátralévő 5; 8 és 12 között 4 egység, ami nem elég. Javítsuk a számítást: P_1 hátralévő ideje $7 - 2 = 5$; 8-tól 13-ig fut, mert $8 + 5 = 13$. Tehát a Gantt-diagram:

```
| P1 0-2 | P2 2-4 | P3 4-5 | P2 5-8 | P1 8-13 |
```

A befejezési idők:

- $B_1 = 13$, $B_2 = 8$, $B_3 = 5$.

A várakozási idők:

$$W_1 = 13 - 7 - 0 = 6, \qquad W_2 = 8 - 3 - 2 = 3, \qquad W_3 = 5 - 1 - 4 = 0.$$

Az átlagos várakozási idő $(6 + 3 + 0)/3 = 3{,}0$.

A nem-preemptív SJF ugyanerre a példára rosszabb átlagot adna, mert P_1 futna 0-tól 7-ig, utána P_2 7-től 10-ig, végül P_3 10-től 11-ig; az átlagos várakozás $(0 + 5 + 6)/3 \approx 3{,}67$ lenne.

8.2.9 A CPU-burst becslése

Valós rendszerben a CPU-burst hossza ritkán ismert előre. Az SJF ezért gyakran **exponenciális átlagolással** becsüli a következő burst hosszát:

$$\tau_{n+1} = \alpha t_n + (1 - \alpha)\tau_n,$$

ahol t_n az aktuális mért burst, τ_n az előző becslés, α pedig $0 < \alpha \leq 1$ súlyozó tényező. Nagy α az aktuális mérést részesíti előnyben, kis α stabilabb, de lassabban alkalmazkodó becslést ad.

8.2.10 Round Robin (RR)

A körforgásos ütemező **preemptív**, és kifejezetten az időosztásos rendszerekhez készült. Az ütemező egy rögzített időkvantumot (q) használ. A kész sor FIFO sorrendben tartja a folyamatokat, de minden folyamat legfeljebb q időegységet futhat egyhuzamban. Ha a folyamat a kvantum letelte előtt befejeződik, a CPU azonnal felszabadul. Ha a kvantum letelik, a folyamatot az ütemező félbeszakítja, és a kész sor végére helyezi.

8.2.11 Példa

Legyen $q = 2$, és a folyamatok mind a 0 időpontban érkeznek:

- P_1: $C_1 = 5$
- P_2: $C_2 = 3$
- P_3: $C_3 = 1$

A Gantt-diagram:

```
| P1 0-2 | P2 2-4 | P3 4-5 | P1 5-7 | P2 7-8 | P1 8-9 |
```

A befejezési idők: $B_1 = 9$, $B_2 = 8$, $B_3 = 5$.

A várakozási idők:

$$W_1 = 9 - 5 = 4, \qquad W_2 = 8 - 3 = 5, \qquad W_3 = 5 - 1 = 4.$$

Az átlagos várakozási idő $(4 + 5 + 4)/3 = 13/3 \approx 4{,}33$.

Ez az érték rosszabb, mint az SJF azonos terhelésre kapott 1,33 értéke, de az RR célja nem is az átlagos várakozási idő minimalizálása, hanem a méltányos CPU-elosztás és a jó válaszidő. A rövid P_3 már 5 időpontban befejeződik, miközben a hosszabbak is arányosan haladnak.

8.2.12 A kvantum megválasztása

A q értéke alapvetően befolyásolja az RR viselkedését:

- Ha q nagyon kicsi, a kontextusváltások száma megnő, és a CPU idejének egy része nem hasznos munkára, hanem állapotmentésre és visszaállításra fordítódik. Például $q = 1$ esetén minden időegység után váltás lehet.
- Ha q nagyon nagy, az RR gyakorlatilag FIFO-vá válik, mert a folyamatok ritkán élik túl a kvantumot.
- A gyakorlatban a q értékét úgy választják meg, hogy a kontextusváltási költség a q-hoz képest kicsi legyen, de a rendszer válaszideje még elfogadható maradjon.

A RR nem szenved éhezéstől, mert minden kész folyamat körben halad, és előbb-utóbb sorra kerül. A komplexitása szintén O(1) a kész sorban, ezért az RR a mai időkig az egyik legelterjedtebb alapütemező.

8.2.13 Prioritásos ütemezés

A prioritásos ütemező minden folyamathoz egy p_i prioritást rendel. A CPU-t mindig a legmagasabb prioritású kész folyamat kapja. A prioritás lehet:

- **statikus**, amely a folyamat létrehozásakor rögzül,
- **dinamikus**, amelyet a rendszer futás közben módosít,
- **belső**, amely a folyamat valamely mérhető jellemzőjéből származik (például memóriahasználat, I/O arány),
- **külső**, amely a felhasználó vagy a rendszer egyéb szabálya alapján adott.

A prioritásos ütemező lehet nem-preemptív (a futó folyamat befejezéséig várakoznak az érkező magasabb prioritású folyamatok) vagy preemptív (új magasabb prioritású folyamat érkezésekor azonnal megszakítja a futót).

8.2.14 Példa preemptív prioritásos ütemezésre

Legyen a konvenció: a nagyobb p_i magasabb prioritást jelent.

- P_1: $a_1 = 0$, $C_1 = 5$, $p_1 = 2$
- P_2: $a_2 = 1$, $C_2 = 3$, $p_2 = 1$
- P_3: $a_3 = 2$, $C_3 = 1$, $p_3 = 4$

Az idővonal:

```
| P1 0-2 | P3 2-3 | P1 3-6 | P2 6-9 |
```

Lépésenként:

1. $t = 0$: P_1 fut.
2. $t = 1$: P_2 érkezik, de $p_2 = 1 < p_1 = 2$, ezért P_1 folytatja.
3. $t = 2$: P_3 érkezik, $p_3 = 4 > p_1 = 2$, ezért P_1 megszakad, és P_3 fut.
4. $t = 3$: P_3 kész. A legmagasabb prioritású kész folyamat P_1 ($p_1 = 2$) szemben P_2-vel ($p_2 = 1$), így P_1 folytatja.
5. $t = 6$: P_1 kész, P_2 fut.
6. $t = 9$: P_2 kész.

A várakozási idők:

- $W_1 = (3-2)+0 = 1$, mert P_1 0–2 és 3–6 között futott; összes futásideje 5, befejezés 6, érkezés 0, tehát $W_1 = 6-5 = 1$.
- $W_2 = 9-3-1 = 5$.
- $W_3 = 3-1-2 = 0$.

Az átlagos várakozási idő $(1+5+0)/3 = 2{,}0$.

8.2.15 Éhezés

A prioritásos ütemezés legfontosabb hibája az **éhezés** (starvation): ha folyamatosan érkeznek magasabb prioritású folyamatok, egy alacsony prioritású folyamat soha nem jut CPU-hoz. A fenti példában P_2 nem éhez, mert a többi folyamat befejeződik, de ha folyamatosan jönnének $p \geq 3$ prioritású folyamatok, P_2 végtelenül várakozhatna.

A probléma kezelésére több lehetőség van:

1. **Prioritás öregítése** (aging): a várakozó folyamat prioritását idővel növeljük, így előbb-utóbb magasabb prioritásúvá válik, és sorra kerül.
2. **Időkvantumos prioritás**: a magas prioritású folyamatok is csak egy kvantumot kapnak, így az alacsonyabbak is haladnak.
3. **Prioritásütemezés mellett garantált minimális CPU-idő**: például a folyamatok kapnak egy-egy időrést.

8.2.16 Az öregítés működése

Tegyük fel, hogy P_2 prioritása kezdetben $p_2 = 1$, és minden 10 időegység várakozás után a rendszer 1-gyel növeli a prioritását. Ha a magas prioritású P_3 és P_5 folyamatosan érkezik, P_2 kezdetben kiszorul. De 10 időegység elteltével p_2 2-re nő, újabb 10 után 3-ra, és így tovább. Amikor p_2 eléri vagy meghaladja az újonnan érkező folyamatok prioritását, P_2 is futhat. Az öregítés tehát az éhezést idővel megszünteti, miközben a prioritásos ütemezés előnyeit megtartja.

A gyakorlatban az öregítés nem feltétlenül lineáris; a Linux például a futásra váró folyamatok súlyát és a virtuális futási idejét használja a döntéshez, de az elv hasonló: a várakozásért kompenzáció jár.

8.2.17 Algoritmusok összehasonlítása

Algoritmus	Preemptív?	Éhezés lehetséges?	Átlagos várakozási idő	Fairness	Komplexitás
FIFO	nem	nem (ha a sor elfogy)	rossz, konvojhatás	gyenge	O(1) beszúrás/törlés
SJF (nem-preemptív)	nem	igen, folyamatosan rövid folyamatok mellett a hosszú késhet	optimális azonos érkezéskor	gyenge	O(n) vagy O(log n) prioritási sorral
SRTF	igen	igen	nagyon jó	gyenge	O(log n)

Algoritmus	Preemptív?	Éhezés lehetséges?	Átlagos várakozási idő	Fairness	Komplexitás
RR	igen	nem	közepes, q függvénye	jó	O(1)
Prioritásos	lehetséges	igen, éhezés	változó	változó	O(log n) vagy O(1)

A táblázatban a komplexitás egy döntési műveletre vonatkozik, nem a teljes futásra. A gyakorlatban a kernel ütemezője a kész folyamatokat általában valamilyen adatszerkezetben (futósor, prioritási sor, piros-fekete fa) tárolja, ezért a döntés valós költsége az adatszerkezettől függ.

8.2.18 Gyakori hibák

1. **A preemptív és nem-preemptív SJF összekeverése.** A nem-preemptív SJF csak a CPU felszabadulásakor dönt, az SRTF viszont minden érkezéskor újraértékel.
2. **Az érkezési idő figyelmen kívül hagyása.** Eltérő a_i esetén a várakozási időt nem lehet egyszerűen a befejezési időkből levonni.
3. **A prioritás irányának tisztázatlansága.** A nagyobb p_i jelenthet magasabb prioritást, de más rendszerekben a kisebb szám a jobb; a laborban és a dokumentációban mindig rögzíteni kell.
4. **A kontextusváltási költség elhanyagolása az RR-nél.** Túl kicsi q mellett a mérés torzít; a q értékét a kontextusváltási időhöz képest kell megválasztani.
5. **A CPU-burst ismertségének feltételezése.** A valós ütemező nem ismeri előre a C_i értékeket, ezért az SJF és SRTF csak becsléssel alkalmazható.

8.2.19 Labor: klasszikus ütemezési algoritmusok szimulációja

A labor célja, hogy a négy algoritmust ugyanazon a rögzített konfiguráción reprodukálhatóan összehasonlítsa. A szimuláció futhat egy C programként, amelyet a reprodukciós csomagban található QEMU vendégben futtatunk. A kód szemléltetés, nem éles rendszerre szánt utasítás.

8.2.20 Kiinduló állapot és konfiguráció

Bemeneti fájl (`sched_cfg.txt`):

```
P1  0  8  2
P2  1  4  1
P3  2  2  4
P4  3  1  3
P5  4  4  5
```

Az oszlopok: azonosító, érkezési idő, CPU-burst, prioritás. A prioritás megállapodás: a nagyobb szám magasabb prioritást jelent. A RR kvantum: $q = 2$. Holtverseny esetén a kisebb azonosító fut, kivéve az RR-t, ahol a sorrendet a kész sor határozza meg.

A futtató C program váza:

```c
/* schedsim.c - szemléltető ütemező szimulátor */
#include <stdio.h>

struct proc {
    int id, arrival, burst, prio, remaining;
};

int main(void) {
    struct proc ps[] = {
        {1, 0, 8, 2, 8},
        {2, 1, 4, 1, 4},
        {3, 2, 2, 4, 2},
        {4, 3, 1, 3, 1},
        {5, 4, 4, 5, 4},
    };
    /* Az ütemező kiválasztása:
       "fifo", "sjf", "srtf", "rr", "prio"
       A Gantt-szegmensek és a metrikák kiírása a kimenetre kerül. */
    return 0;
}
```

A program a megfelelő ütemezőt futtatva kiírja a Gantt-szegmenseket, a befejezési, átfutási és várakozási időket.

8.2.21 Build és futtatási parancs

A reprodukciós csomagban található `qemu` vendégben a parancsok:

```
cc -O2 -Wall -o schedsim schedsim.c
./schedsim --policy fifo   > fifo.log
./schedsim --policy sjf    > sjf.log
./schedsim --policy srtf   > srtf.log
./schedsim --policy rr     > rr.log
./schedsim --policy prio   > prio.log
```

8.2.22 Várt eredmény

Az analitikus várt eredmények a fent bemutatott módszerekkel számolva:

Ütemező	Gantt-diagram	Átlagos várakozás	Átlagos átfutás
FIFO	`P1 0-8, P2 8-12, P3 12-14, P4 14-15, P5 15-19`	(0+7+10+11+11)/5 = **7,8**	(8+11+12+12+15)/5 = **11,6**
SJF	`P1 0-8, P4 8-9, P3 9-11, P2 11-15, P5 15-19`	(0+10+7+5+11)/5 = **6,6**	(8+14+9+6+15)/5 = **10,4**
SRTF	`P1 0-1, P2 1-2, P3 2-4, P4 4-5, P2 5-8, P5 8-12, P1 12-19`	(11+3+0+1+4)/5 = **3,8**	(19+7+2+2+8)/5 = **7,6**
RR (q=2)	`P1 0-2, P2 2-4, P3 4-6, P1 6-8, P4 8-9, P5 9-11, P2 11-13, P1 13-15, P5 15-17, P1 17-19`	(11+8+2+5+9)/5 = **7,0**	(19+12+4+6+13)/5 = **10,8**

Ütemező	Gantt-diagram	Átlagos várakozás	Átlagos átfutás
Prioritásos preemptív	`P1 0-2, P3 2-4, P5 4-8, P4 8-9, P1 9-15, P2 15-19`	(7+14+0+5+0)/5 = **5,2**	(15+18+2+6+4)/5 = **9,0**

A táblázatban a Gantt-szegmensek a `Folyamat kezdet-vég` formát követik, az idők szimulációs időegységek.

8.2.23 Mérési napló

A mérési naplóban a következőket kell rögzíteni:

- Hardver: QEMU `qemu-system-x86_64`, vCPU: 1, vendég memória: 256 MiB.
- Fordítási beállítások: `cc` (a reprodukciós csomagban rögzített verzió, például GCC 13.2.0), `-O2 -Wall`.
- Órajel és mérési módszer: a szimuláció eseménylépéseket számol; a várt értékek analitikusak, a mért értékek a szimulátor kimenetéből származnak. A host futási idő nem része az ütemezési döntéseknek, de a reprodukálhatóság kedvéért a naplóban szerepelhet.
- A mért és a várt eredményt külön oszlopban kell feltüntetni. A hallgató a szimulátor kimenetét beírja a `mért` oszlopba, és összeveti a fenti várt értékekkel.

8.2.24 Visszaállítás és biztonság

A labor nem módosítja a hoszt bootloaderét, lemezét vagy rendszerpartícióját. A futtatás eldobható QEMU vendégben történik, ahonnan a naplókat ki lehet másolni. A labor után a vendéget leállítjuk, és a csomag visszaáll a kiinduló állapotba.

8.2.25 Önellenőrző kérdés

Adott három folyamat: P_1 ($a = 0$, $C = 4$), P_2 ($a = 1$, $C = 2$), P_3 ($a = 2$, $C = 1$). Számítsa ki az átlagos várakozási időt FIFO, nem-preemptív SJF és RR ($q = 1$) esetén. Magyarázza meg, miért nem éhezik egyik folyamat sem RR-nél, és miért fordulhat elő éhezés a nem-preemptív SJF-nél.

A fejezet legfontosabb tanulsága, hogy nincs egyetlen minden célra optimális algoritmus. A FIFO egyszerű, de konvojhatást okoz; az SJF minimalizálja az átlagos várakozási időt, de éhezést okozhat; a RR fair és interaktív terheléshez jó, de nem optimalizálja az átfutást; a prioritásos ütemező rugalmas, de az éhezést külön kezelni kell. A következő egység (Többszintű visszacsatolás és fair ütemezés) ezekre a fogalmakra építve mutatja be a többszintű visszacsatolásos és a fair ütemezést, amelyek a négy alapalgoritmus korlátait ötvözött megoldásokkal oldják fel.

8.3 Többszintű visszacsatolás és fair ütemezés

A többfeladatos általános célú operációs rendszerek egyik legnehezebb ütemezési problémája, hogy egyszerre kell kiszolgálniuk rövid, interaktív feladatokat – ilyen például egy billentyűleütésre váró shell, egy grafikus felület vagy egy hálózati kérés – és hosszú, CPU-intenzív batch folyamatokat, mint egy tudományos számítás, egy videókódolás vagy egy nagy fordítófuttatás. Az előző alfejezetben megismert alapalgoritmusok – FIFO, SJF, RR, prioritásos ütemezés – egyenként

általában nem képesek mindkét terheléstípust jól kezelni. A FIFO rövid folyamatokat várakoztat egy hosszú mögött; az SJF éheztetheti a hosszú folyamatokat, ráadásul előre nem ismerjük a CPU-burst hosszát; a round-robin méltányos, de nem különbözteti meg a sürgős interaktív folyamatot a batch-től; a statikus prioritás pedig vagy túl merev, vagy kézzel kell hangolni. A modern általános célú ütemezők ezért két irányból fejlesztették tovább az alapokat: az egyik a dinamikus, visszacsatoláson alapuló többszintű prioritási rendszer (Multi-Level Feedback Queue, MLFQ), a másik a méltányos CPU-elosztást számon tartó fair ütemezés, amelynek legismertebb megvalósítása a Linux Completely Fair Scheduler (CFS) és a hozzá kapcsolódó **virtual runtime** fogalma. Ebben az alfejezetben ezt a két családot, valamint az interaktív és batch terhelés kezelését vizsgáljuk meg.

8.3.1 Többszintű visszacsatolásos sor (MLFQ)

Az MLFQ alapgondolata, hogy a folyamatoknak ne egyetlen statikus prioritásuk legyen, hanem több prioritási szinten helyezkedjenek el, és a rendszer a viselkedésük alapján folyamatosan módosítsa a szintjüket. A cél az, hogy a rövid, I/O-igényes interaktív folyamatok magas prioritást kapjanak, a hosszú CPU-intenzív folyamatok pedig fokozatosan alacsonyabb szintre kerüljenek, ahol nagyobb időkvantummal futhatnak, így csökken a kontextusváltási overhead. A „visszacsatolás” kifejezés arra utal, hogy az ütemező döntése a folyamat múltbeli viselkedésén alapul: ha egy folyamat gyakran feladja a CPU-t I/O-várakozás miatt, akkor valószínűleg interaktív, ezért érdemes magasan tartani; ha viszont mindig kihasználja a teljes időkvantumot, akkor CPU-intenzív, és lejjebb helyezhető.

A klasszikus MLFQ szabályai a következőképpen foglalhatók össze:

1. Ha az A folyamat prioritása magasabb, mint B prioritása, akkor A fut, B pedig várakozik.
2. Azonos prioritású folyamatok round-robin (RR) elven osztoznak a CPU-n.
3. Amikor egy folyamat belép a rendszerbe, a legmagasabb prioritási szintre kerül.
4. Ha egy folyamat a teljes időkvantumát felhasználja, akkor a prioritása eggyel csökken.
5. Ha egy folyamat az időkvantum lejárta előtt feladja a CPU-t (például I/O miatt), akkor a prioritása változatlan marad.

Ezek a szabályok együtt adják az MLFQ alapváltozatát, amelyet azonban további kiegészítésekkel kell stabilizálni. A legalapvetőbb probléma az **éheztetés**: ha folyamatosan érkeznek rövid interaktív folyamatok, akkor a hosszú batch folyamatok soha nem jutnak CPU-hoz, mert mindig lesz magasabb prioritású versenyző. A másik tipikus hiba az ún. **játék az ütemezővel**: egy folyamat szándékosan az időkvantum lejárta előtt kiadhat egy felesleges I/O-műveletet, hogy megtartsa magas prioritását, és így aránytalanul sok CPU-időt szerezzen. A harmadik probléma, hogy a folyamat viselkedése időben változhat: egy korábban batch jellegű folyamat később interaktívvá válhat, de az alacsony szintről nem tud feljebb jutni.

A gyakorlati MLFQ-implementáció ezért két további szabályt szokott alkalmazni:

6. Bizonyos időközönként minden folyamat prioritását a legmagasabb szintre emeljük (**priority boost**). Ez megszünteti az éheztetést, és lehetővé teszi, hogy a megváltozott viselkedésű folyamatok újra versenyezzenek.
7. Az alacsonyabb prioritási szinteken általában hosszabb időkvantumot adunk, hogy a CPU-intenzív folyamatok hatékonyan fussanak, míg a magas szinteken rövid kvantummal gyorsan válthatunk az interaktív folyamatok között.

Az időkvantumok tipikus beállítása például: a legfelső szinten 2–10 ms, a következőn 20–50 ms, az alsókon pedig 100 ms vagy annál nagyobb is lehet. Így a rövid interaktív folyamatok gyorsan lefutnak vagy blokkolódnak, a hosszú folyamatok pedig ritkábban váltanak kontextust.

Vegyünk egy konkrét MLFQ-példát három folyamattal. A konfiguráció: három prioritási szint, $Q_0 = 10$ ms, $Q_1 = 20$ ms, $Q_2 = 40$ ms. A P_1 interaktív, 10 ms CPU-burst után 20 ms I/O-várakozás, majd ismét 10 ms CPU; P_2 CPU-intenzív, 60 ms CPU-burst; P_3 rövid CPU-intenzív, 15 ms CPU-burst. Mindhárom a 0. időpontban érkezik. Az MLFQ a 0-adik szinten RR-t használ 10 ms kvantummal. Az ütemezés a következőképpen alakulhat:

- 0–10 ms: P_1 fut, kvantumot használ el (nem blokkol), ezért a 0-adik szintről az 1-esre kerül.
- 10–20 ms: P_2 fut, kvantumot használ el, szintén az 1-esre kerül.
- 20–30 ms: P_3 fut, kvantumot használ el, az 1-esre kerül.
- A 0-adik szint kiürült, most az 1-es szint következik 20 ms kvantummal, RR sorrend: P_1, P_2, P_3. Azonban P_1 10 ms CPU-t igényelt, de mivel a 0-adik szinten már kapott 10 ms-t, a teljes CPU-burstje valójában 10 ms volt, és ezután I/O-ra várt. Várjunk: a példa pontosítása szükséges. Legyen P_1 CPU-burstje 5 ms, majd I/O, mert a 0-adik szint 10 ms kvantuma alatt blokkol, így nem használja el a kvantumot, és a szintjén marad. Egyszerűbb, ha P_1 első CPU-burstje 8 ms, utána I/O. Akkor a 0–8 ms időben P_1 fut, majd blokkol, és a 0-adik szinten marad, amikor visszatér. A 8–10 ms között a P_2 kezd futni, de a kvantum csak 2 ms-ot tesz ki, ezért a 10 ms-os kvantum lejártakor P_2 nem használta el a teljes kvantumot, de az időzítő megszakítás a kvantum végét jelzi. Az MLFQ szabály szerint ha a folyamat a kvantum végét eléri, akkor lejjebb kerül, függetlenül attól, hogy mennyi időt kapott. Ezért érdemes a példát úgy kialakítani, hogy a folyamatok CPU-burstjei igazodjanak a kvantumokhoz.

A didaktikus cél érdekében válasszunk egyszerűbb forgatókönyvet: P_1 CPU-burstje 10 ms, majd 20 ms I/O; P_2 CPU-burstje 50 ms; P_3 CPU-burstje 10 ms. Mind 0-ban érkeznek. A 0-adik szint kvantuma 10 ms.

- 0–10: P_1 fut, elhasználja a kvantumot, ezért 1-es szintre kerül, majd I/O-ra indul.
- 10–20: P_2 fut, elhasználja a kvantumot, 1-es szintre kerül.
- 20–30: P_3 fut, elhasználja a kvantumot, 1-es szintre kerül.
- A 0-adik szint üres. P_1 még I/O-n van (visszatér 30 ms-nál). Az 1-es szinten P_2 és P_3 állnak 20 ms kvantummal.
- 30–50: P_2 fut az 1-es szinten, 20 ms kvantumot kap, majd az 1-es szinten marad (mert elhasználta a kvantumot? Az 1-es szinten 20 ms a kvantum, P_2-nek még 40 ms CPU kell, tehát elhasználta a 20 ms-ot, így a 2-es szintre kerül).
- Eközben 30-nál P_1 visszatér az I/O-ról, és a 0-adik szintre kerül (priority boost nélkül is, mert a szabály szerint a folyamat a blokkolás után a korábbi szintjére tér vissza, de itt a 0-adik szinten volt, mert blokkolás előtt nem csökkent a prioritása; valójában P_1 a 0-adik szinten maradt, mert nem használta el a kvantumot, hanem I/O-ra ment, így a szabály 5 szerint a prioritása változatlan, azaz 0-adik szint). Tehát 30-nál P_1 belép a 0-adik szintre, és mivel magasabb prioritású, mint az 1-es szinten futó P_2, az ütemező preemptálja P_2-t, és P_1-nek adja a CPU-t. Ez a MLFQ egyik fontos tulajdonsága: a magasabb szintű folyamat mindig megszakítja az alacsonyabbat.
- 30–40: P_1 fut a 0-adik szinten, 10 ms kvantumot kap, elhasználja, ezért ismét 1-es szintre kerül. (Ha újabb I/O-ra menne, akkor ismét 0-adik szinten maradna.)
- 40–50: P_2 folytatódik az 1-es szinten, de P_2 a 30–50 közötti 20 ms-os kvantumból csak 10 ms-ot kapott, mert P_1 preemptálta. A kvantum azonban az 1-es szinten 20 ms, és P_2 nem használta el, mert preemptálták. Ilyenkor a legtöbb MLFQ-implementáció a folyamatot a szintjén tartja, és a maradék kvantumot később folytathatja. Tételezzük fel ezt a viselkedést. Így 40–50: P_2 folytatja, és a maradék 10 ms-ot felhasználja, ezzel a 20 ms-os kvantum lejár, P_2 a 2-es szintre kerül.
- 50–60: most az 1-es szinten P_1 és P_3 állnak. P_1 a 0-adikról lekerült, P_3 már az 1-es szinten volt. Tegyük fel, hogy P_3 korábban 20–30 között a 0-adik szinten futott, és 1-esre került,

de még nem futott az 1-es szinten. Az RR sorrend szerint P_1 következik, mert ő később került az 1-esre? A pontos sorrend implementációfüggő. Vegyük egyszerűen: P_1 fut 20 ms kvantummal, de neki már csak 10 ms CPU kell? A példa szerint P_1 összesen 20 ms CPU-t igényelt (10+10), ebből 10-et a 0-adikon megkapott, most a maradék 10-et. Tehát 50–60: P_1 fut, elhasznál 10 ms-ot, de a kvantum 20 ms, így nem használja el, hanem befejezi a futását. A folyamat terminál.

- 60–70: P_3 fut az 1-es szinten, 20 ms kvantumot kap, de neki csak 10 ms CPU kell, tehát 60–70 között befejezi.
- Ezután P_2 a 2-es szinten, 40 ms kvantummal, a maradék CPU-igénye $50 - 20 - 10 = 20$ ms? Számoljuk: P_2 összesen 50 ms CPU-t akart; eddig kapott 10 ms-ot a 0-adik szinten (10–20), majd 20 ms-ot az 1-es szinten (30–50, megszakítva), tehát eddig 30 ms, maradék 20 ms. A 2-es szinten 40 ms kvantum van, így P_2 a 70–90 ms között lefut.

Ez a példa jól mutatja az MLFQ dinamikáját, bár a pontos ütemezés sok paramétertől függ. A Gantt-diagram:

```
| P1(0-10) | P2(10-20) | P3(20-30) | P1(30-40) | P2(40-50) | P1(50-60) | P3(60-70) | P2(70-90) |
```

A folyamatok átfutási ideje: P_1: 60 ms, P_2: 90 ms, P_3: 70 ms. Átlagos átfutási idő: $(60 + 90 + 70)/3 = 73{,}33$ ms. Átlagos várakozási idő: P_1 várakozása: 0 (első futás) + 20 (I/O után 30–30? valójában P_1 30-kor tért vissza, azonnal futott, tehát 0 várakozás a CPU-ra) + 0 (utolsó futás), összesen 0; P_2 várakozása: 10 (mielőtt először fut 10-nél) + 10 (a 30–40 preemptálás miatt 30-nál P_1 fut, így P_2 30–40 között várakozik, majd 40-nél folytatja) + 20 (a 2-es szintre kerülés után 50–70 között várakozik, mert P_1 és P_3 futnak) = 40 ms; P_3 várakozása: 20 (a 0-adik szinten 20-ig várakozott) + 10 (az 1-es szinten 50–60 között P_1 futott) = 30 ms. Átlagos várakozási idő: $(0 + 40 + 30)/3 = 23{,}33$ ms.

A példa rávilágít az MLFQ erősségére: az interaktív P_1 mindig magas prioritást kapott, amikor CPU-ra volt szüksége, és a batch P_2 hosszú kvantummal futott a mélyebb szinteken. Ugyanakkor a példa egyszerűsített; a valódi rendszerekben a priority boost és a pontos könyvelés bonyolultabb.

8.3.2 Fair ütemezés: CFS és virtuális futásidő

Az MLFQ heurisztikán alapul, és bár jól működik, nehéz garantálni a méltányosságot: előfordulhat, hogy két azonos fontosságú folyamat nagyon eltérő CPU-időt kap a dinamikus prioritásváltások miatt. A fair ütemezők ezzel szemben abból az elvből indulnak ki, hogy a CPU-időt **igazságosan**, azaz a folyamatok súlyainak arányában kell elosztani. A Linux 2.6.23 óta a CFS-t használja alapértelmezett ütemezőként, amely ennek az elvnek a gyakorlati megvalósítása.

A CFS alapmodellje az **ideális fair ütemező**: ha N folyamat van, és mindegyik súlya azonos, akkor egy igen rövid, mondjuk 1 ms-os időkvantummal felváltva futtatva minden folyamat pontosan a CPU-idő $1/N$ részét kapja. A valóságban a kontextusváltási költség miatt nem lehet ilyen finom kvantumokat használni, ezért a CFS egy **célzott ütemezési késleltetést** (scheduling latency) határoz meg: azt az időtartamot, amely alatt minden futásra kész folyamatnak legalább egyszer futnia kell. Ha például a célzott késleltetés 12 ms, és 4 folyamat van, akkor az időkvantum $12/4 = 3$ ms. Ha a folyamatok száma nagy, a kvantum túl kicsi lehet, ezért a CFS bevezet egy minimális granuláltságot (min_granularity), tipikusan 1–4 ms.

A CFS szíve a **virtual runtime** (*vruntime*). Minden folyamathoz hozzárendelünk egy $vruntime_i$ értéket, amely azt fejezi ki, hogy a folyamat mennyi CPU-időt „érdemelt” a súlyához képest. A definíció:

$$vruntime_i = \frac{actual_runtime_i}{weight_i},$$

ahol $actual_runtime_i$ a folyamat tényleges CPU-futási ideje, $weight_i$ pedig a folyamat súlya. A súly a prioritásból származik: a Linuxban a `nice` érték (-20 és $+19$ között) határozza meg, ahol az alacsonyabb nice magasabb súlyt jelent. A súlyokat nem lineárisan, hanem egy előre definiált táblázat szerint rendelik hozzá, hogy a nice értékek közötti különbségek arányos CPU-idő-különbséget eredményezzenek. Például a nice 0 súlya 1024, a nice $+5$ súlya körülbelül 335, a nice -5 súlya körülbelül 3121 (a konkrét értékek kernelverziótól függenek, és ellenőrzést igényelnek, de a nagyságrend jellemző). A *vruntime* tehát a tényleges futásidőt a súllyal normalizálja: egy nagy súlyú folyamat lassabban növeli a *vruntime*-ját, ezért gyakrabban választja ki az ütemező.

A CFS mindig a **legkisebb** *vruntime*-mal rendelkező futásra kész folyamatot választja. Ennek hatékony kiválasztásához az adatszerkezet egy **piros-fekete fa** (red-black tree), amelyben a folyamatok *vruntime* szerint rendezettek. A legkisebb elem a fa bal szélső levele, így a kiválasztás $O(\log N)$ időt igényel. Amikor egy folyamat fut, a *vruntime*-ja nő; amikor blokkol (például I/O-ra), kikerül a fából; amikor újra futásra kész lesz, visszakerül a fába. Ha egy folyamat sokáig aludt, a *vruntime*-ja elmarad a többiekétől, ezért visszatérésekor azonnal kiválasztódik, ami természetes módon kedvez az interaktív folyamatoknak.

Vizsgáljunk egy kétfolyamatos CFS-példát. P_1 súlya $w_1 = 1024$, P_2 súlya $w_2 = 512$ (azaz P_1 kétszer akkora súlyú, mint P_2). Tegyük fel, hogy mindkettő folyamatosan CPU-t kér, és a célzott ütemezési késleltetés 12 ms. A kvantumok arányosak a súlyokkal: P_1 kvantuma $12 \cdot \frac{1024}{1024+512} = 8$ ms, P_2 kvantuma $12 \cdot \frac{512}{1024+512} = 4$ ms. Kezdetben $vruntime_1 = 0$, $vruntime_2 = 0$. A CFS mindig a kisebb *vruntime*-t választja, tehát P_1 indul, mert döntetlen esetén a beillesztési sorrend dönthet, vagy valamilyen determinisztikus szabály.

- 0–8 ms: P_1 fut. $vruntime_1 = 8/1024 = 0{,}0078125$.
- 8–12 ms: P_2 fut, mert most $vruntime_2 = 0 < 0{,}0078125$. $vruntime_2 = 4/512 = 0{,}0078125$ (éppen egyenlő lesz).
- Most $vruntime_1 = vruntime_2 = 0{,}0078125$. Az ütemező dönthet; tegyük fel, hogy P_1 folytatja.
- 12–20 ms: P_1 fut ismét 8 ms-ot. $vruntime_1 = 16/1024 = 0{,}015625$.
- 20–24 ms: P_2 fut, mert $0{,}0078125 < 0{,}015625$. $vruntime_2 = 8/512 = 0{,}015625$.
- A ciklus ismétlődik. Látható, hogy hosszú távon P_1 kétszer annyi CPU-időt kap, mint P_2, mert a súlya kétszer akkora. A Gantt-diagram:

```
| P1(0-8) | P2(8-12) | P1(12-20) | P2(20-24) | P1(24-32) | ...
```

A méltányosság mérésére gyakran használják a **Jain-féle fairness indexet**:

$$J = \frac{\left(\sum_{i=1}^{N} x_i\right)^2}{N \sum_{i=1}^{N} x_i^2},$$

ahol x_i a folyamat által kapott erőforrás (például CPU-idő) mennyisége. Ha minden folyamat pontosan egyenlő részt kap, $J = 1$, ha pedig az elosztás egyenlőtlen, $J < 1$. A fenti példában 24 ms alatt P_1 16 ms-ot, P_2 8 ms-ot kapott, így:

$$J = \frac{(16+8)^2}{2 \cdot (16^2 + 8^2)} = \frac{576}{2 \cdot 320} = \frac{576}{640} = 0{,}9.$$

Ez azt mutatja, hogy a súlyozott elosztás nem egyenlő, de a súlyarányoknak megfelelően méltányos. Ha azonos súlyú folyamatokról lenne szó, a CFS $J = 1$-hez közeli értéket adna.

A CFS fontos tulajdonsága az **I/O-igényes folyamatok természetes előnyben részesítése**. Mivel az ilyen folyamatok gyakran blokkolnak, a tényleges CPU-futási idejük kisebb, így a *vruntime*-juk lassabban nő. Amikor visszatérnek egy I/O-műveletből, a *vruntime*-juk általában kisebb, mint a CPU-intenzív társaiké, ezért az ütemező őket választja. Ez a mechanizmus automatikusan biztosítja az interaktív folyamatok alacsony késleltetését anélkül, hogy explicit heurisztikát kellene beépíteni. Fontos azonban, hogy a CFS nem „bünteti" a batch folyamatokat: azok is megkapják a súlyuknak megfelelő CPU-időt, csak éppen később, mert a *vruntime*-juk gyorsan nő.

8.3.3 Interaktív és batch terhelés együttes kezelése

Az általános célú rendszerekben a terhelés ritkán homogén. A felhasználói interakciók – egérmozgás, billentyűleütés, ablakfrissítés – jellemzően nagyon rövid CPU-burstökből állnak, amelyeket hosszú I/O-várakozások választanak el. A batch folyamatok ezzel szemben hosszú, megszakítás nélküli CPU-burstöket generálnak. A jó ütemezőnek egyszerre kell teljesítenie két, részben ellentétes célt:

- **Alacsony késleltetés** az interaktív folyamatok számára: amikor egy felhasználói esemény bekövetkezik, a hozzá tartozó folyamatnak minél hamarabb CPU-ra kell kerülnie.
- **Magas átbocsátás** a batch folyamatok számára: a hosszú folyamatok akkor hatékonyak, ha ritkán váltanak kontextust, és nagy időkvantumokat kapnak.

Az MLFQ ezt a két célt a prioritási szintek és a kvantumok fokozatos növelésével éri el. Az interaktív folyamat a magas szinten rövid kvantummal gyorsan futhat, amikor I/O-ról visszatér; ha mégis CPU-intenzívvé válik, fokozatosan lejjebb kerül, ahol hosszabb kvantumot kap. A priority boost pedig megakadályozza, hogy egy hosszú batch folyamat végleg eltűnjön a rendszerből.

A CFS ezzel szemben nem használ explicit prioritási szinteket, hanem a *vruntime* folytonos értékével dolgozik. Az interaktív folyamat azért kerül előre, mert a *vruntime*-ja kicsi marad a gyakori alvás miatt. A batch folyamat *vruntime*-ja gyorsan nő, ezért hátrébb sorolódik, de soha nem éhezik, mert a *vruntime*-ja folyamatosan nő, és előbb-utóbb ő lesz a legkisebb, amikor a többiek megelőzik. A súlyok lehetővé teszik a rendszergazda számára, hogy bizonyos folyamatokat előnyben részesítsen vagy hátrébb soroljon a `nice` érték állításával.

Egy gyakori félreértés, hogy a CFS „időosztásos" vagy „round-robin" jellegű. Bár a CFS is időkvantumokat használ, a kvantumokat nem előre rögzített, azonos méretű szeletekként osztja ki, hanem a célzott ütemezési késleltetés és a folyamatok súlya alapján számítja. Továbbá a döntés nem körbeforgó, hanem mindig a legkisebb *vruntime* alapján történik, ami dinamikusan alkalmazkodik a folyamatok viselkedéséhez.

A gyakorlatban a Linux CFS az utóbbi években továbbfejlődött, és az alapértelmezett beállítások mellett is jól kezeli a vegyes terhelést. Az EEVDF (Earliest Eligible Virtual Deadline First) ütemező 2023-tól kezdte felváltani a klasszikus CFS-t, de az alapelvek – virtual runtime, súlyozott méltányosság, eligibilitási határidő – továbbra is meghatározóak. Az ilyen friss fejlemények részletei meghaladják e fejezet kereteit, és a pontos viselkedés kernelverziótól függ, ezért mindig az adott kernel dokumentációját kell ellenőrizni.

8.3.4 Gyakorlati megfontolások és tipikus hibák

Az MLFQ és a fair ütemezők implementálásakor több tipikus hiba fordul elő. Az egyik leggyakoribb az **éheztetés nem megfelelő kezelése**. Ha a priority boost túl ritkán történik, a batch folyamatok késleltetése elfogadhatatlanul megnő; ha túl gyakran, az MLFQ elveszti a dinamikus alkalmazkodás előnyét, és gyakorlatilag round-robin lesz. A megoldás a boost periódusának gondos megválasztása: tipikusan néhány száz milliszekundum vagy néhány másodperc, de ez a rendszer jellegétől függ.

A második hiba a **kvantumok rossz méretezése**. Ha a legfelső szint kvantuma túl nagy, az interaktív folyamatok válaszideje megnő; ha túl kicsi, a kontextusváltási overhead dominál. Általános célú rendszerekben a felső szinteken 1–10 ms, az alsókon 50–200 ms közötti értékek szokásosak. A pontos értékeket mérésekkel kell validálni.

A harmadik hiba a **fair ütemezőnél a súlyok helytelen értelmezése**. A `nice` érték nem lineárisan befolyásolja a CPU-időt; a kernel súlytáblázatot használ. Ha a felhasználó azt hiszi, hogy a nice +10 feleannyi CPU-időt ad, mint a nice 0, akkor meglepődik, mert a tényleges arány a táblázattól függ. A dokumentációban ezt egyértelműen jelezni kell.

A negyedik hiba a *vruntime* **könyvelésének elhanyagolása alvó folyamatoknál**. Ha egy folyamat hosszú ideig alszik, és a *vruntime*-ját nem igazítják ki, akkor visszatérésekor hatalmas előnyre tehet szert, és rövid időre monopolizálhatja a CPU-t. A CFS ezért bizonyos esetekben a *vruntime*-t a folyamat alvási idejének figyelembevételével korrigálja, például az átlagos *vruntime*-hoz igazítja. Ez a részlet a megvalósítás része, és a laborban egyszerűsített változattal dolgozunk.

Végül gyakori hiba az **interaktív és batch terhelés összekeverése a mérési kiértékelésben**. Egyetlen átlagos átfutási idő vagy CPU-kihasználtság nem mutatja meg, hogy az interaktív folyamatok válaszideje megfelelő-e. Ezért a laborban külön metrikákat kell számítani: átlagos várakozási idő, átlagos átfutási idő, fairness index, valamint a 95%-os válaszidő az interaktív folyamatokra.

8.3.5 Labor: MLFQ és fair-share ütemezés szimulációja

A labor célja, hogy saját kezűleg tervezzünk és hasonlítsunk össze egy MLFQ- és egy CFS-jellegű ütemezőt egy vegyes munkaterhelésen. A labor reprodukálható jegyzetfüzetben (például Jupyter Notebook Python 3-mal) vagy tetszőleges programozási nyelven elvégezhető; a lényeg a rögzített konfiguráció és a mérési napló.

8.3.6 Konfiguráció

A szimulált környezet egyetlen CPU-t feltételez. A folyamatok a következők:

- P_1: interaktív, 4 ms CPU-burst, majd 15 ms I/O-várakozás, összesen 5 ciklus.
- P_2: interaktív, 2 ms CPU-burst, majd 10 ms I/O-várakozás, összesen 8 ciklus.
- P_3: batch, 60 ms folyamatos CPU-burst.
- P_4: batch, 40 ms folyamatos CPU-burst.

Mind a négy folyamat a $t = 0$ időpontban érkezik. Az MLFQ konfiguráció: 3 szint, $Q_0 = 4$ ms, $Q_1 = 8$ ms, $Q_2 = 16$ ms, priority boost 50 ms-onként. Azonos szintű folyamatok RR sorrendben futnak, a sorrend az érkezési sorrend (vagy visszatérési sorrend). A CFS-jellegű ütemező konfiguráció: célzott ütemezési késleltetés 12 ms, minimális granuláltság 1 ms, minden folyamat súlya azonos (1024). A *vruntime* frissítése: $vruntime_i += \Delta t / weight_i$, ahol Δt a

tényleges futási idő. Az alvó folyamatok *vruntime*-ját nem korrigáljuk, kivéve ha a labor ezt külön kéri.

A szimuláció lépése: az időt 1 ms-os egységekben léptetjük, és minden lépésben eldöntjük, hogy melyik folyamat fut. A folyamatok I/O-várakozását is számon tartjuk; amikor egy folyamat I/O-ról visszatér, bekerül a futásra kész sorba.

8.3.7 Várható eredmények

Az elméleti várakozások szerint az MLFQ az interaktív folyamatokat magas prioritáson tartja, így azok válaszideje alacsony lesz, a batch folyamatok pedig fokozatosan lejjebb kerülnek, és hosszú kvantumokkal futnak. A P_1 és P_2 átlagos várakozási ideje várhatóan néhány ms, míg a batch P_3 és P_4 várakozási ideje nagyobb, de az átfutási idő elfogadható. A priority boost hatására 50 ms-onként a batch folyamatok is újra magas szintre kerülnek, ami csökkenti az éheztetést.

A CFS-szimulációban azonos súlyok mellett hosszú távon minden folyamat közel azonos CPU-időt kap, de a rövid interaktív folyamatok hamar befejeződnek, így a batch folyamatok a későbbi szakaszban már egyedül futnak. Az interaktív folyamatok válaszideje itt is alacsony, mert a *vruntime*-juk kicsi marad a gyakori alvás miatt. A fairness index a CFS-nél várhatóan 1-hez közeli, az MLFQ-nál valamivel alacsonyabb, mert a prioritási szintek torzíthatják az elosztást.

A konkrét számok a szimuláció pontos szabályaitól függenek, ezért itt csak irányokat adunk meg, nem pontos értékeket. A mért értékeket a jegyzetfüzetben kell rögzíteni.

8.3.8 Mérési napló és értékelés

A labor során mindkét ütemezőre rögzíteni kell:

- a teljes szimulációs időt,
- az egyes folyamatok átfutási idejét (az első futásra kész állapottól a befejezésig),
- az egyes folyamatok átlagos várakozási idejét (a CPU-ra várás idejét),
- a Jain-féle fairness indexet a CPU-időre,
- az interaktív folyamatok (P_1, P_2) 95%-os válaszidejét, ahol a válaszidő az I/O-ról való visszatéréstől a következő CPU-futás kezdetéig eltelt idő.

A táblázat sablonja így nézhet ki:

Ütemező	Átfutási idő P_1	Átfutási idő P_2	Átfutási idő P_3	Átfutási idő P_4	Átlag várakozási idő	Fairness index	95% válaszidő (interaktív)
MLFQ	(mért)	(mért)	(mért)	(mért)	(mért)	(mért)	(mért)
CFS	(mért)	(mért)	(mért)	(mért)	(mért)	(mért)	(mért)

A várt és a mért értékeket külön kell választani: a jegyzetfüzetben jelöljük, hogy melyik oszlop vagy cella tartalmaz elméleti előrejelzést, és melyik a tényleges szimulációs kimenet. A végső értékelésben hasonlítsuk össze a két ütemezőt a válaszidő, az átfutási idő és a fairness szempontjából.

8.3.9 Lehetséges implementációs vázlat (pszeudokód)

Az alábbi pszeudokód az MLFQ szimuláció alapvázát mutatja; a CFS hasonló, csak az ütemező választási logikája tér el.

```
function mlfq_schedule(processes, simulation_time):
    ready_queues = [[], [], []]       # 3 prioritási szint
    current_level = 0
    time = 0
    for each process p:
        p.remaining_cpu = p.cpu_burst
        p.level = 0
        ready_queues[0].append(p)

    while time < simulation_time and any process not finished:
        # priority boost 50 ms-onként
        if time % 50 == 0 and time > 0:
            for level in [1, 2]:
                for p in ready_queues[level]:
                    p.level = 0
                    ready_queues[0].append(p)
                ready_queues[level].clear()

        # válasszuk a legmagasabb nem üres szintet
        while current_level < 3 and ready_queues[current_level].empty():
            current_level += 1
        if current_level == 3:
            time += 1
            current_level = 0
            continue

        p = ready_queues[current_level].front()
        quantum = get_quantum(current_level)   # Q0, Q1, Q2
        run_time = min(quantum, p.remaining_cpu)
        # itt azt is figyelni kell, ha p I/O-ra megy a burst vége előtt
        # ez a példában csak akkor fordul elő, ha a burst hossza < quantum
        p.remaining_cpu -= run_time
        time += run_time

        if p.remaining_cpu == 0:
            p.finished = true
            remove p from ready_queues[current_level]
            # ha van következő I/O ciklus, akkor ütemezzük újra
            if p.has_next_io():
                p.wait_until = time + p.io_duration
                # p átkerül egy I/O várakozó listába
            else:
                p.finish_time = time
        else if run_time < quantum:
            # a folyamat I/O-ra ment, mert a burst véget ért
            # (a példában ilyenkor p.remaining_cpu = 0, tehát fent kezelve)
            # itt csak akkor, ha a kvantum nagyobb, mint a maradék CPU
            p.level = current_level  # marad ezen a szinten
            p.wait_until = time + p.io_duration
            remove p from ready_queues[current_level]
        else:
            # teljes kvantumot elhasznált
            if current_level < 2:
                p.level = current_level + 1
                remove p from ready_queues[current_level]
                ready_queues[current_level + 1].append(p)
            else:
                # legalsó szinten marad
```

```
            p.level = 2
            remove p from ready_queues[current_level]
            ready_queues[2].append(p)
        current_level = 0  # újra a tetejéről kezdjük

    # I/O várakozók visszatérése
    for q in io_waiters:
        if time >= q.wait_until:
            q.remaining_cpu = q.next_cpu_burst
            q.level = 0
            ready_queues[0].append(q)
            remove q from io_waiters
```

Ez a pszeudokód csak vázlat, és nem kezeli az összes saroksetet, de kiindulásnak megfelel. A CFS-szimulátorban a `ready_queues` helyett egy *vruntime* szerint rendezett adatszerkezetet (például kupacot) használunk, és minden lépésben a legkisebb *vruntime*-t választjuk.

8.3.10 Ellenőrző kérdések

1. Magyarázza el, miért csökkenti az MLFQ a kontextusváltási overheadet a batch folyamatoknál, miközben az interaktív folyamatok válaszideje alacsony marad.
2. Hogyan biztosítja a CFS, hogy egy hosszú alvás után visszatérő interaktív folyamat gyorsan CPU-hoz jusson anélkül, hogy a batch folyamatokat éheztetné?
3. Számítsa ki a fairness indexet két azonos súlyú folyamatra, amelyek közül az egyik 30 ms, a másik 10 ms CPU-időt kapott egy 40 ms-os időablakban. Értelmezze az eredményt.

Ezek a kérdések a fejezetben tárgyalt fogalmak alkalmazását kérik számon.

8.3.11 Összegzés

A korszerű általános célú ütemezők két nagy megközelítést ötvöznek: a visszacsatolásos többszintű prioritási sorokat és a méltányos, virtuális futásidőn alapuló elosztást. Az MLFQ heurisztikusan, a folyamatok viselkedéséből tanulva helyezi át a folyamatokat a prioritási szintek között, míg a CFS a *vruntime* segítségével folytonos, súlyozott méltányosságot biztosít. Mindkettő célja ugyanaz: az interaktív és batch terhelés együttes, hatékony kiszolgálása. A következő alfejezet a valós idejű ütemezésre tér át, ahol a célok gyökeresen mások: ott nem a méltányosság, hanem a határidők garantált betartása az elsődleges.

8.4 Valós idejű ütemezés: RMS, EDF és deadline

8.4.1 A határidő-alapú ütemezés elmélete

Az általános célú operációs rendszerekben az ütemező célja gyakran az átlagos válaszidő csökkentése, a maximális átbocsátás elérése vagy az erőforrások igazságos elosztása. A valós idejű rendszerekben azonban a helyességnek időbeli dimenziója is van: egy feladat akkor helyes, ha nemcsak logikailag helyes eredményt ad, hanem a határideje előtt szolgáltatja. A kemény valós idejű rendszerekben egyetlen határidő-mulasztás is rendszerhibának minősül; a lágy valós idejű rendszerekben a késés elfogadható, de általában kerülendő. Ez a fejezet a határidő-alapú ütemezés elméleti alapjaival és két klasszikus algoritmusával foglalkozik: a **rate-monotonic scheduling** (RMS) és az **earliest deadline first** (EDF) ütemezővel.

A valós idejű ütemezés elméletét C. L. Liu és James W. Layland alapozta meg 1973-ban publikált cikkükkel, amelyben a periodikus feladatok ütemezhetőségének feltételeit vizsgálták. Azóta ez a munka a beágyazott rendszerek, az RTOS-ek és a valós idejű Linux-konfigurációk egyik elméleti pillére.

8.4.1.1 A periodikus feladatmodell

A további elemzések alapjául szolgáló modell feltételezi, hogy a rendszerben n független periodikus feladat fut egyetlen processzoron. Minden P_i feladatot a következő paraméterekkel jellemzünk:

- C_i: **legrosszabb végrehajtási idő** (WCET, worst-case execution time), amelyet a feladatnak a processzoron el kell töltenie egyetlen periódusban. Ez konzervatív becslés, amely tartalmazza a cache-hibák, a memória-elérés és az utasítás-végrehajtás legrosszabb esetét.
- T_i: **periódusidő**; a feladat minden T_i időközönként érkezik. Az első érkezés időpontját általában 0-nak vesszük.
- D_i: **relatív határidő**; a feladatnak az érkezésétől számítva D_i időn belül be kell fejeződnie.

Az alapmodellben $D_i = T_i$, azaz a határidő a következő periódus kezdetéig tart. A feladatok függetlenek egymástól, nem osztoznak erőforrásokon, és a rendszer preemptív: az ütemező bármikor félbeszakíthat egy futó feladatot, és más feladatot indíthat. A kontextusváltási időt elhanyagoljuk, vagy beépítjük a C_i értékébe.

A processzorkihasználtság (utilizáció) U a feladatok CPU-igényének összege:

$$U = \sum_{i=1}^{n} \frac{C_i}{T_i}$$

Ha $U \leq 1$, a processzor elvileg el tudja végezni a munkát; ha $U > 1$, a rendszer túlterhelt, és semmilyen ütemező sem tudja minden határidőt tartani. A $U = 1$ eset azt jelenti, hogy a processzor éppen 100%-ban kihasznált.

8.4.2 Rate-Monotonic Scheduling (RMS)

Az RMS statikus prioritású ütemező: minden feladatnak a periódusideje alapján adunk prioritást. Minél rövidebb a periódus, annál magasabb a prioritás. A P_i prioritása tehát $p_i = \frac{1}{T_i}$, vagy egyszerűen a T_i szerint rendezve. A szabályt Liu és Layland 1973-ban publikálta, és azóta a valós idejű rendszerek egyik sarokköve. Az RMS **optimális a statikus prioritású ütemezők között** abban az értelemben, hogy ha egy feladatsor ütemezhető bármely statikus prioritású algoritmussal, akkor RMS-sel is ütemezhető. Ezt az eredményt nevezzük optimális statikus prioritási szabálynak.

Az RMS ütemezhetőségi tesztje elégséges feltételt ad a processzorkihasználtságra:

$$U \leq n(2^{1/n} - 1)$$

Ez a kifejezés n növekedésével $\ln 2 \approx 0{,}693$-hoz tart. Tehát ha a processzorkihasználtság legfeljebb 69{,}3%, akkor az RMS garantáltan ütemezhető. Ez konzervatív: lehet, hogy egy adott feladatsor ennél magasabb utilizációval is ütemezhető, de az elégséges feltétel teljesülése esetén biztosan. A korlát alsó határ, tehát ha U kisebb vagy egyenlő, mint ez a szám, akkor az RMS minden feladatot határidőn belül befejez. Ha U nagyobb, akkor további elemzés szükséges.

A pontosabb ütemezhetőségi vizsgálat a **válaszidő-elemzéssel** végezhető. Minden P_i feladat legrosszabb válaszidejét R_i kiszámítjuk, és ellenőrizzük, hogy $R_i \leq D_i$. A válaszidő-elemzés iteratív módon történik:

$$R_i^{(0)} = C_i$$

$$R_i^{(k+1)} = C_i + \sum_{j \in hp(i)} \left\lceil \frac{R_i^{(k)}}{T_j} \right\rceil C_j$$

ahol $hp(i)$ az i-nél magasabb prioritású feladatok halmaza. A képlet azt fejezi ki, hogy a feladat saját végrehajtási idejéhez hozzáadjuk a magasabb prioritású feladatok által okozott késleltetést. A $\lceil \frac{R_i^{(k)}}{T_j} \rceil$ tényező azt adja meg, hogy P_j hányszor érkezik meg a $R_i^{(k)}$ időablak alatt, és minden érkezéskor C_j időt fogyaszt a processzoron. Az iteráció akkor konvergál, ha $R_i^{(k+1)} = R_i^{(k)}$; ha az iteráció során $R_i > D_i$, akkor a feladat nem ütemezhető, és az elemzés leállítható. Ez a módszer szükséges és elégséges feltételt ad a modellünkben, feltéve, hogy a feladatok függetlenek és a határidők megegyeznek a periódusokkal.

8.4.2.1 Példa RMS-re

Vegyünk egy rendszert három feladattal:

- P_1: $C_1 = 1$ ms, $T_1 = 4$ ms
- P_2: $C_2 = 2$ ms, $T_2 = 6$ ms
- P_3: $C_3 = 3$ ms, $T_3 = 12$ ms

A prioritási sorrend: P_1 (T=4), P_2 (T=6), P_3 (T=12).

A processzorkihasználtság:

$$U = \frac{1}{4} + \frac{2}{6} + \frac{3}{12} = 0{,}25 + 0{,}333 + 0{,}25 = 0{,}833$$

A Liu-Layland korlát $n = 3$ esetén:

$$3(2^{1/3} - 1) \approx 3(1{,}2599 - 1) \approx 0{,}7797$$

Mivel $U > 0{,}7797$, a Liu-Layland teszt nem garantálja az ütemezhetőséget, de a válaszidő-elemzéssel ellenőrizhetjük.

Számítsuk ki a válaszidőket a fenti iteratív képlettel.

P_1 esetén nincs magasabb prioritású feladat, így $R_1 = C_1 = 1$ ms, ami kisebb, mint $T_1 = 4$ ms, tehát teljesül.

P_2 esetén a magasabb prioritású feladat P_1.

$$R_2^{(0)} = 2$$

$$R_2^{(1)} = 2 + \lceil 2/4 \rceil \cdot 1 = 2 + 1 = 3$$

$$R_2^{(2)} = 2 + \lceil 3/4 \rceil \cdot 1 = 2 + 1 = 3$$

Az iteráció konvergált, $R_2 = 3$ ms, ami kisebb, mint $T_2 = 6$ ms, tehát teljesül.

P_3 esetén a magasabb prioritású feladatok P_1 és P_2.

$$R_3^{(0)} = 3$$

$$R_3^{(1)} = 3 + \lceil 3/4 \rceil \cdot 1 + \lceil 3/6 \rceil \cdot 2 = 3 + 1 + 2 = 6$$

$$R_3^{(2)} = 3 + \lceil 6/4 \rceil \cdot 1 + \lceil 6/6 \rceil \cdot 2 = 3 + 2 + 2 = 7$$

$$R_3^{(3)} = 3 + \lceil 7/4 \rceil \cdot 1 + \lceil 7/6 \rceil \cdot 2 = 3 + 2 + 2 = 7$$

Az iteráció konvergált, $R_3 = 7$ ms, ami kisebb, mint $T_3 = 12$ ms, tehát teljesül.

Mindhárom feladat ütemezhető, tehát az RMS ütemezi ezt a feladatsort.

A Gantt-diagram az első 12 ms-re a következőképpen alakul (az időt ms-ben mérjük, a folyamatokat a megfelelő intervallumokban jelöljük):

```
[0,1): P1
[1,3): P2
[3,4): P3 (P1 preemptálja 4-nél)
[4,5): P1
[5,6): P3 (folytatás)
[6,8): P2
[8,9): P3 (befejezi)
[9,12): üresjárat
```

A késleltetési metrikák:

- P_1 legrosszabb válaszideje: $R_1 = 1$ ms, határidő 4 ms.
- P_2 legrosszabb válaszideje: $R_2 = 3$ ms, határidő 6 ms.
- P_3 legrosszabb válaszideje: $R_3 = 7$ ms, határidő 12 ms.

A rendszer maximális kihasználtsága 83{,}3%, és minden határidő teljesül.

8.4.3 Earliest Deadline First (EDF)

Az EDF dinamikus prioritású ütemező: minden pillanatban a legkorábbi abszolút határidejű feladatot futtatja. A prioritás tehát nem fix, hanem az érkezéskor számított $d_i = r_i + D_i$ abszolút határidő határozza meg, ahol r_i a feladat érkezési ideje. Az EDF **optimális az összes preemptív ütemező között** egyprocesszoros környezetben: ha létezik olyan ütemezés, amely minden határidőt tart, akkor az EDF is megtalálja. Ez az eredmény Liu és Layland másik fontos tétele.

Az EDF optimalitása intuitívan belátható: ha egy ütemezésben egy feladat fut, de egy másik feladatnak korábbi a határideje, akkor a kettőt megcserélve nem sérül a határidő. Ismételt cserékkel bármely helyes ütemezés EDF-fé alakítható.

Az EDF ütemezhetőségének szükséges és elégséges feltétele (a $D_i = T_i$ modellben):

$$U = \sum_{i=1}^{n} \frac{C_i}{T_i} \leq 1$$

Ez jelentős előny az RMS-hez képest, mert akár 100%-os processzorkihasználtság is elérhető. Az EDF tehát nem hagy kihasználatlan processzoridőt, ha van futásra kész feladat, és a határidők tartása mellett.

Nézzük ugyanazt a példát EDF-fel:

Feladatok: P_1 (C=1, T=4, D=4), P_2 (C=2, T=6, D=6), P_3 (C=3, T=12, D=12).

A Gantt-diagram EDF-fel az első 12 ms-re:

```
[0,1): P1 (d=4)
[1,3): P2 (d=6)
[3,4): P3 (d=12)
[4,5): P1 (d=8)
[5,6): P3 (d=12)
[6,8): P2 (d=12) - megjegyzés: 6-nál P2 újra érkezik, d=12, P3 szintén d=12, azonos határidő,
↪  válasszuk P2-t
[8,9): P3 (d=12)
[9,12): üresjárat
```

Itt a határidők azonosak a periódusokkal, ezért a viselkedés hasonló az RMS-hez, de az EDF általános esetben jobban kihasználja a processzort.

Vegyünk egy olyan példát, ahol az RMS nem tudja ütemezni a feladatokat, de az EDF igen.

Példa:

- P_1: $C_1 = 2$ ms, $T_1 = 5$ ms
- P_2: $C_2 = 4$ ms, $T_2 = 7$ ms

Utilizáció: $U = \frac{2}{5} + \frac{4}{7} = 0{,}4 + 0{,}5714 = 0{,}9714$, ami kisebb, mint 1, tehát az EDF ütemezhető.

RMS: prioritás P_1 (T=5) $>$ P_2 (T=7). Válaszidő-elemzés:

P_1: $R_1 = 2 \leq 5$, ok.

P_2:

$$R_2^{(0)} = 4$$

$$R_2^{(1)} = 4 + \lceil 4/5 \rceil \cdot 2 = 4 + 2 = 6$$

$$R_2^{(2)} = 4 + \lceil 6/5 \rceil \cdot 2 = 4 + 4 = 8$$

$$R_2^{(3)} = 4 + \lceil 8/5 \rceil \cdot 2 = 4 + 4 = 8$$

Konvergált: $R_2 = 8 > D_2 = 7$, tehát P_2 megsérti a határidőt. Az RMS nem ütemezhető.

EDF: nézzük a futást 0-tól.

- $t = 0$: P_1 d=5, P_2 d=7. P_1 fut 0–2.
- $t = 2$: P_1 kész. P_2 fut 2–5 (3 egységet).
- $t = 5$: P_1 újra érkezik, d=10. P_2 d=7. Mivel P_2 határideje korábbi ($7 < 10$), P_2 folytatja 5–6 (befejezi, mert $C_2 = 4$, eddig 3 egységet futott, maradt 1). P_2 befejeződik 6-nál, határideje 7 előtt.
- $t = 6$: P_1 fut 6–8 (befejezi $C_1 = 2$), határideje 10 előtt.

A további periódusok hasonlóan működnek. Tehát az EDF ütemezi a feladatokat, míg az RMS nem.

8.4.4 Processzorkihasználtság és túlterhelés

A processzorkihasználtság U kritikus szerepet játszik. Az RMS esetén a Liu-Layland korlát csak elégséges feltétel, tehát léteznek olyan feladatsorok, amelyeknél $U > n(2^{1/n} - 1)$, de mégis ütemezhetők. A válaszidő-elemzés pontosabb, de annak elvégzése szükséges. Az EDF esetén $U \leq 1$ szükséges és elégséges feltétel.

Amikor $U > 1$, a rendszer túlterhelt, és nem létezik olyan ütemezés, amely minden határidőt tart. Ilyenkor a viselkedés fontos. RMS esetén a túlterhelés általában a legalacsonyabb prioritású feladatokat érinti először, mert a magas prioritásúak továbbra is lefutnak. Ez lehet előny, ha a fontos feladatoknak magas prioritást adunk. EDF esetén túlterheléskor a határidők sérülhetnek, és a rendszer „dominó-effektusként" sok feladatot elveszíthet, mert az EDF a legkorábbi határidőt próbálja tartani, és ha egyet elvét, a többi is sérülhet. Ezért túlterheléses helyzetekben az EDF kevésbé kiszámítható, hacsak nem alkalmazunk további mechanizmusokat (feladat-elvetés, prioritás-öröklés, szerverek).

A gyakorlatban gyakran alkalmaznak hibrid megoldásokat: a kritikus feladatokat statikus prioritással (RMS) ütemezik, a többit dinamikusan. Vagy EDF-et használnak, de túlterhelés esetén az alacsony fontosságú feladatokat eldobják. A tervezés során meg kell határozni, hogy a rendszer milyen túlterhelési stratégiát kövessen, és mely feladatok élveznek elsőbbséget.

8.4.5 Gyakorlati szempontok és implementáció

Az RMS implementálása egyszerű: a feladatokat periódusidő szerint rendezett prioritási sorban tartjuk. A beszúrás $O(\log n)$ időt igényel bináris kupac esetén. A kontextusváltáskor a legmagasabb prioritású kész feladatot választjuk. Mivel a prioritás statikus, a várakozási sor rendezettsége nem változik az idő előrehaladtával, csak a feladatok érkezésekor és befejezésekor frissítjük.

Az EDF esetén minden feladat érkezésekor kiszámítjuk az abszolút határidőt $d_i = t + D_i$, és a feladatot egy határidő szerint rendezett prioritási sorba helyezzük. Az ütemező mindig a sor elejéről választ. A beszúrás szintén $O(\log n)$. A dinamikus prioritás miatt a prioritási sor minden érkezéskor frissül, és a kontextusváltáskor a legkisebb határidejű elemet kell megtalálni.

Pszeudokód RMS-hez:

```
struct task {
    uint32_t period;
    uint32_t wcet;
    uint32_t priority;   // rövidebb periódus => kisebb érték
    uint32_t remaining;
    uint32_t next_release;
    uint32_t abs_deadline;
};

void rms_schedule(void) {
    while (1) {
        task_t *next = NULL;
        uint32_t best_prio = UINT32_MAX;
        for (int i = 0; i < NUM_TASKS; i++) {
            if (tasks[i].remaining > 0 && tasks[i].priority < best_prio) {
                best_prio = tasks[i].priority;
                next = &tasks[i];
            }
        }
        if (next) {
            run(next);
```

```
        } else {
            idle();
        }
        check_releases(); // időzítő megszakításban frissít
    }
}
```

EDF-hez hasonló, de a prioritás helyett a legkorábbi `abs_deadline` értéket választjuk. A `check_releases()` függvény az időzítő megszakításban fut, és frissíti a feladatok állapotát.

A valós rendszerekben az ütemezőt általában megszakítás-vezérelt módon valósítják meg. Az időzítő minden tickben lefut, és ha egy feladat érkezett, vagy a futó feladat lejárt, kontextusváltást hajt végre. A prioritási sor megvalósítható bináris kupaccal vagy más adatszerkezettel, például vörös-fekete fával. A Linux kernelben a valós idejű ütemezők (SCHED_FIFO, SCHED_RR) statikus prioritást használnak, míg a SCHED_DEADLINE az EDF-et valósítja meg, és további paramétereket (runtime, period, deadline) fogad el. A kernel dokumentációja szerint a SCHED_DEADLINE a legkorábbi határidő szerint választ, és a futási időt korlátozza. A pontos API és viselkedés a rendszer verziójától függ, ezért ellenőrizni kell a kernel dokumentációját.

8.4.6 Tipikus hibák és megoldások

1. **A $D_i = T_i$ feltételezés elhanyagolása.** Ha a határidő rövidebb, mint a periódus ($D_i < T_i$), az RMS prioritási szabálya már nem optimális. Ilyenkor a határidő szerinti prioritás (deadline-monotonic) használható, de a válaszidő-elemzést módosítani kell, és a Liu-Layland korlát nem alkalmazható közvetlenül. A gyakorlatban a határidő gyakran kisebb, mint a periódus, ezért a tervezésnél ezt figyelembe kell venni.

2. **A processzorkihasználtság félreértelmezése.** Az $U \leq 1$ szükséges feltétel, de nem elégséges RMS esetén. Sok fejlesztő azt hiszi, hogy ha a CPU nincs teljesen kihasználva, akkor minden határidő tartható; ez hamis RMS-nél, mert a statikus prioritás miatt bizonyos feladatkombinációk nem ütemezhetők még akkor sem, ha $U < 1$. Például a fenti $P_1 : 2/5, P_2 : 4/7$ esetben $U = 0{,}9714 < 1$, de az RMS nem ütemezhető.

3. **A túlterhelés következményeinek figyelmen kívül hagyása.** Túlterheléskor az EDF viselkedése kiszámíthatatlan lehet, ha nincs mechanizmus a feladatok eldobására vagy a rendszer leépítésére. A tervezéskor meg kell határozni, mely feladatok kritikusak, és milyen romlási stratégiát alkalmazunk. Például egy hálózati routerben a csomagfeldolgozásnak lehet prioritása a naplózással szemben.

4. **Nem preemptív környezetben való alkalmazás.** Az RMS és EDF feltételezik a preemptív ütemezést. Ha a rendszer nem preemptív, a válaszidő-elemzéshez hozzá kell adni a legnagyobb blokkolási időt, és a garantált ütemezhetőség romlik.

5. **Prioritás inverzió figyelmen kívül hagyása.** Ha a feladatok közös erőforrásokat használnak, előfordulhat, hogy egy alacsony prioritású feladat blokkol egy magas prioritásút. Ezt a jelenséget a következő fejezet (Versenyhelyzet, kritikus szakasz és atomi műveletek) részletesen tárgyalja, de az ütemezhetőségi elemzésnél figyelembe kell venni.

8.4.7 Labor: RMS- és EDF-ütemezhetőség vizsgálata

A labor célja, hogy egy adott feladatsorra kiszámítsa az RMS és EDF ütemezhetőségét, és a szimulációval ellenőrizze a határidők tartását. A labor reprodukálható jegyzetfüzetként vagy emulált környezetben futtatható C programmal. A következőkben egy C programot mutatunk be, amely elvégzi a számítást és a szimulációt.

8.4.7.1 Konfiguráció

- **Processzor**: egymagos, kontextusváltási idő 0 ms (a szimulációban). A hardver órajele nem releváns a szimulációnál, mert az időegységet logikai milliszekundumban mérjük.
- **Feladatsor**:

Feladat	C_i (ms)	T_i (ms)	D_i (ms)
P_1	1	4	4
P_2	2	6	6
P_3	3	12	12

- **Időegység**: 1 ms.
- **A szimuláció** 48 ms-ig fut, három perióduson át.

8.4.7.2 Várt eredmény

- Utilizáció: $U = 0{,}833$.
- RMS válaszidők: $R_1 = 1$, $R_2 = 3$, $R_3 = 7$; minden $R_i \leq D_i$, tehát ütemezhető.
- EDF: ütemezhető, mert $U \leq 1$; a válaszidők hasonlóak.

8.4.7.3 C program (szemléltetés, nem futtatandó éles rendszeren)

```c
// rms_edf_check.c - szemléltető kód, nem futtatandó éles rendszeren
#include <stdio.h>
#include <stdint.h>
#include <stdbool.h>

#define N 3

struct task {
    uint32_t c;
    uint32_t t;
    uint32_t d;
    uint32_t abs_deadline;
    uint32_t remaining;
    uint32_t next_release;
    uint32_t priority; // RMS prioritás: kisebb t => magasabb prioritás (kisebb szám)
};

struct task tasks[N] = {
    { .c = 1, .t = 4, .d = 4 },
    { .c = 2, .t = 6, .d = 6 },
    { .c = 3, .t = 12, .d = 12 },
};

void init_tasks(void) {
    for (int i = 0; i < N; i++) {
        tasks[i].next_release = 0;
        tasks[i].remaining = 0;
        tasks[i].abs_deadline = 0;
        tasks[i].priority = tasks[i].t; // kisebb t => kisebb prioritási szám
    }
}

bool rms_schedulable(void) {
    // válaszidő-elemzés, feltételezi, hogy a tömb t szerint rendezett
```

```
    for (int i = 0; i < N; i++) {
        uint32_t r = tasks[i].c;
        while (1) {
            uint32_t r_next = tasks[i].c;
            for (int j = 0; j < i; j++) {
                r_next += ((r + tasks[j].t - 1) / tasks[j].t) * tasks[j].c;
            }
            if (r_next == r) {
                break;
            }
            r = r_next;
            if (r > tasks[i].d) {
                printf("RMS: P%d válaszidő = %u > %u, nem ütemezhető\n", i + 1, r, tasks[i].d);
                return false;
            }
        }
        printf("RMS: P%d válaszidő = %u <= %u\n", i + 1, r, tasks[i].d);
    }
    return true;
}

bool edf_schedulable(void) {
    double u = 0.0;
    for (int i = 0; i < N; i++) {
        u += (double)tasks[i].c / tasks[i].t;
    }
    printf("EDF: U = %.3f\n", u);
    return u <= 1.0;
}

void simulate(uint32_t duration) {
    for (uint32_t t = 0; t < duration; t++) {
        // feladatok érkeztetése
        for (int i = 0; i < N; i++) {
            if (t == tasks[i].next_release) {
                tasks[i].remaining = tasks[i].c;
                tasks[i].abs_deadline = t + tasks[i].d;
                tasks[i].next_release += tasks[i].t;
            }
        }
        // ütemező: RMS (prioritás szerint, ha egyenlő, kisebb index)
        int sel = -1;
        for (int i = 0; i < N; i++) {
            if (tasks[i].remaining > 0) {
                if (sel == -1 || tasks[i].priority < tasks[sel].priority) {
                    sel = i;
                }
            }
        }
        if (sel >= 0) {
            printf("t=%u: P%d fut\n", t, sel + 1);
            tasks[sel].remaining--;
        } else {
            // üresjárat
        }
    }
}

int main(void) {
    init_tasks();
    bool rms = rms_schedulable();
    printf("RMS ütemezhető: %s\n", rms ? "igen" : "nem");
    bool edf = edf_schedulable();
```

```
    printf("EDF ütemezhető: %s\n", edf ? "igen" : "nem");
    simulate(48);
    return 0;
}
```

A kód a laborban futtatható. A kimenet rögzíthető, és a mért válaszidőket a szimulációból ki kell nyerni. A szimuláció a RMS prioritási szabályt használja; az EDF szimulációhoz a `simulate` függvényben a `priority` összehasonlítást az `abs_deadline` összehasonlításra kell cserélni.

8.4.7.4 Mérési napló (példa)

A futtatás előtt a reprodukálhatóság érdekében rögzíteni kell a környezetet:

```
Fordítás: gcc -O0 -o rms_edf_check rms_edf_check.c
Futtatás: ./rms_edf_check
Kimenet (részlet):
RMS: P1 válaszidő = 1 <= 4
RMS: P2 válaszidő = 3 <= 6
RMS: P3 válaszidő = 7 <= 12
RMS ütemezhető: igen
EDF: U = 0.833
EDF ütemezhető: igen
t=0: P1 fut
t=1: P2 fut
t=2: P2 fut
...
```

A mért válaszidőket a szimulációból ki kell számítani: minden feladat esetén az érkezéstől a befejezésig eltelt idő. A várt értékeket a válaszidő-elemzés adta, a mért értékeket a szimulációból kapjuk. A két értéknek egyeznie kell. A mérési naplóban fel kell tüntetni a **várt** és a **mért** értéket külön, és az eltérést.

8.4.7.5 Visszaállítás

A labor nem módosít semmilyen rendszerállapotot, csak a saját programot futtatja. A program leállítása után a rendszer változatlan. A futtatáshoz nincs szükség valós lemez, bootloader vagy rendszerpartíció módosítására.

8.4.8 Önellenőrző kérdés

Adott három periodikus feladat:

- P_1: $C_1 = 2$ ms, $T_1 = 10$ ms
- P_2: $C_2 = 3$ ms, $T_2 = 8$ ms
- P_3: $C_3 = 4$ ms, $T_3 = 15$ ms

a) Számítsa ki a processzorkihasználtságot.

b) Határozza meg az RMS prioritási sorrendjét, és ellenőrizze az ütemezhetőséget a Liu-Layland korláttal, majd válaszidő-elemzéssel.

c) Ellenőrizze az EDF ütemezhetőségét.

d) Rajzolja fel a Gantt-diagramot az első 15 időegységre RMS és EDF esetén.

Megoldás vázlat:

a) $U = \frac{2}{10} + \frac{3}{8} + \frac{4}{15} = 0{,}2 + 0{,}375 + 0{,}2667 = 0{,}8417$.

b) RMS sorrend: P_2 (T=8), P_1 (T=10), P_3 (T=15). Liu-Layland $n = 3$: $0{,}7797 < 0{,}8417$, tehát nem garantált. Válaszidő-elemzés: $R_2 = 3 \leq 8$, R_1: $R_1^{(0)} = 2$, $R_1^{(1)} = 2 + \lceil 2/8 \rceil \cdot 3 = 5$, $R_1^{(2)} = 2 + \lceil 5/8 \rceil \cdot 3 = 5$, $R_1 = 5 \leq 10$. R_3: $R_3^{(0)} = 4$, $R_3^{(1)} = 4 + \lceil 4/8 \rceil \cdot 3 + \lceil 4/10 \rceil \cdot 2 = 4{+}3{+}2 = 9$, $R_3^{(2)} = 4{+}\lceil 9/8 \rceil{\cdot}3{+}\lceil 9/10 \rceil{\cdot}2 = 4{+}6{+}2 = 12$, $R_3^{(3)} = 4{+}\lceil 12/8 \rceil{\cdot}3{+}\lceil 12/10 \rceil{\cdot}2 = 4 + 6 + 4 = 14$, $R_3^{(4)} = 4 + \lceil 14/8 \rceil \cdot 3 + \lceil 14/10 \rceil \cdot 2 = 4 + 6 + 4 = 14$, $R_3 = 14 \leq 15$. Tehát RMS ütemezhető.

c) EDF: $U = 0{,}8417 \leq 1$, tehát ütemezhető.

d) A Gantt-diagramot a szimulációhoz hasonlóan kell felrajzolni, figyelembe véve az érkezéseket és határidőket.

8.4.9 Összefoglalás

Ebben a fejezetben a valós idejű ütemezés két alapalgoritmusát ismertettük. Az RMS statikus prioritással, a periódusidő szerint rangsorol, és elégséges feltételt ad a Liu-Layland korláttal, valamint pontosabb válaszidő-elemzéssel. Az EDF dinamikusan a legkorábbi határidőt választja, és optimális egyprocesszoros preemptív környezetben, lehetővé téve a 100%-os kihasználtságot. A túlterhelés viselkedése eltérő: RMS-ben az alacsony prioritású feladatok sérülnek először, míg EDF-ben dominó-effektus léphet fel. A labor során egy konkrét feladatsorra számítottuk ki és szimuláltuk a két algoritmust. A következő fejezet a szinkronizációval és a versenyhelyzetekkel foglalkozik, ahol a prioritás inverzió problémáját is tárgyaljuk.

9 Szinkronizáció és konkurencia

9.1 Versenyhelyzet, kritikus szakasz és atomi műveletek

A konkurens végrehajtás alapvető kihívása, hogy két vagy több vezérlési szál ugyanazt az osztott erőforrást egymással átfedő időben éri el. Az operációs rendszerben ez a helyzet nem kivétel, hanem mindennapos: a kernel ütemezője listákat módosít, a fájlrendszer puffereket kezel, a hálózati stack csomagokat továbbít, és ezeket a tevékenységeket több processzor vagy megszakítás is megszakíthatja. Amikor a végeredmény a végrehajtások sorrendjétől függ, versenyhelyzetről beszélünk. Ez a fejezet bemutatja a versenyhelyzet kialakulásának mechanizmusát, formalizálja a kritikus szakasz követelményeit, és bevezeti azokat az atomi műveleteket és memóriabarriereket, amelyekre a következő fejezetek zármegoldásai épülnek.

1. A versenyhelyzet kialakulása

Tekintsük a legegyszerűbb példát: két folyamat vagy szál, P_1 és P_2, közös memóriaterületen lévő számlálót növel. A magas szintű C kódban a `counter++` kifejezés egyetlen műveletnek tűnik, a gépi kód szintjén azonban tipikusan három lépésből áll:

1. a memória címről betöltés egy regiszterbe,
2. a regiszter értékének növelése,
3. a regiszter visszaírása a memóriába.

A három lépés között a futás megszakadhat, mert a CPU ütemezője átadja a vezérlést a másik folyamatnak, vagy egy másik processzor egyidejűleg hajtja végre ugyanezt a műveletsort. A következő interleaving táblázat egy lehetséges sorrendet mutat, amikor a kezdeti `counter = 0`:

Lépés	P_1 művelete	P_1 regiszter	P_2 művelete	P_2 regiszter	counter
1	`ld rax, [counter]`	0	–	–	0
2	–	–	`ld rbx, [counter]`	0	0
3	`add rax, 1`	1	–	–	0
4	–	–	`add rbx, 1`	1	0
5	`st [counter], rax`	1	–	–	1
6	–	–	`st [counter], rbx`	1	1

A végeredmény `1`, holott a két növelés után `2` vártunk. Az egyik frissítés elveszett, mert mindkét folyamat a régi értéket olvasta be, mielőtt a másik visszaírta volna. Ez az elveszett frissítés (lost update) a versenyhelyzetek egyik legismertebb formája.

A jelenség nem korlátozódik a számlálókra. Ugyanez a mechanizmus okoz hibát, ha két folyamat egy láncolt listához ad hozzá elemet, ha egy banki átutalás során két tranzakció ugyanazt az egyenleget olvassa és írja, vagy ha a kernel két megszakításkezelője ugyanazt a puffert módosítja.

A közös jellemző: egy összetett művelet, amelynek közbenső állapota láthatóvá válik egy másik konkurens végrehajtó számára.

Formálisan: legyen egy osztott adatobjektum, és két vagy több műveletsor, amelyek ezt az objektumot olvassák és írják. Ha a műveletsorok egymáshoz képest tetszőleges sorrendben hajtódnak végre, és a végeredmény függ ettől a sorrendtől, akkor versenyhelyzet áll fenn. A versenyhelyzet lehet determinisztikus reprodukciójú, de gyakran csak ritkán, nagy terhelés alatt vagy specifikus időzítés mellett jelentkezik, ami a hibakeresést különösen megnehezíti.

2. Kritikus szakasz és követelményei

A versenyhelyzet kezelésének alapfogalma a kritikus szakasz (critical section). Kritikus szakasz az a kódrészlet, amelyben a folyamat osztott erőforrást ér el, és amelyet úgy kell végrehajtani, hogy közben más folyamat ne férjen hozzá ugyanahhoz az erőforráshoz. A megoldásnak három követelményt kell teljesítenie:

1. **Kölcsönös kizárás (mutual exclusion).** Ha egy folyamat a kritikus szakaszában van, akkor egyetlen másik folyamat sem lehet ugyanabban a kritikus szakaszban (vagyis ugyanarra az erőforrásra vonatkozó kritikus szakaszban). Ez biztosítja, hogy az összetett művelet közbenső állapota ne legyen látható.
2. **Haladás (progress).** Ha egyetlen folyamat sincs a kritikus szakaszban, és vannak folyamatok, amelyek belépni szeretnének, akkor a belépésre jelentkezők közül valamelyiknek véges időn belül be kell lépnie. Ez a feltétel azt zárja ki, hogy a rendszer holtpontra jusson, amikor minden folyamat vár, pedig a kritikus szakasz szabad.
3. **Korlátos várakozás (bounded waiting).** Miután egy folyamat jelezte belépési szándékát, léteznie kell egy felső korlátnak arra, hogy hány másik folyamat léphet be előtte. Ez biztosítja a kiszolgálási igazságosságot, és kizárja az éheztetést (starvation).

A három követelmény nem triviálisan teljesíthető. A legegyszerűbb próbálkozás, egy globális `lock` változó használata, például az alábbi pszeudokód:

```
while (lock == 1) { /* spin */ }
lock = 1;
/* kritikus szakasz */
lock = 0;
```

nem teljesíti a kölcsönös kizárást, mert a `lock == 1` ellenőrzése és a `lock = 1` beállítása között a folyamat megszakadhat, és egy másik folyamat is beléphet. Ez a jelenség a versenyhelyzet iskolapéldája. A megoldáshoz olyan hardveres vagy szoftveres mechanizmus kell, amely a belépési protokollt oszthatatlanná teszi; ezeket a mechanizmusokat vizsgáljuk a következő pontokban.

Formálisan: legyen n folyamat, $P_1, P_2, \ldots, P_n$, amelyek ugyanazon erőforrás kritikus szakaszát (CS_i) hajtják végre. A kölcsönös kizárás azt jelenti, hogy bármely időpontban legfeljebb egy CS_i aktív. A haladás azt jelenti, hogy ha a kritikus szakasz szabad, és vannak belépni szándékozó folyamatok, akkor a belépési protokoll véges idő alatt kiválasztja az egyiket. A korlátos várakozás azt jelenti, hogy létezik egy b egész szám, amelyre igaz, hogy bármely P_i esetén a belépési kérelemtől számítva legfeljebb b másik folyamat léphet be, mielőtt P_i belépne.

3. Atomi műveletek

Atomi műveletnek nevezzük azt a műveletet, amely oszthatatlanul hajtódik végre: a rendszer többi része nem figyelheti meg a művelet közbenső állapotát. A kritikus szakasz belépési protokolljához olyan atomi műveletre van szükség, amely egyetlen lépésben olvas és módosít egy memóriaterületet.

A modern processzorok több ilyen műveletet biztosítanak:

- **Test-and-set (TAS):** egy memóriacímről beolvas egy értéket, és egyidejűleg igazra állítja azt. Ez a klasszikus primitív a spinlock megvalósításához.
- **Compare-and-swap (CAS):** három operandusa van: egy memóriacím, egy várt érték és egy új érték. Ha a memóriacím tartalma megegyezik a várt értékkel, akkor az új értéket írja be, és sikerrel tér vissza; különben nem módosít, és sikertelenséget jelez. Ez a primitív számos zártípus és lock-free adatszerkezet alapja.
- **Fetch-and-add (FAA):** atomi módon beolvassa egy memóriacím értékét, hozzáad egy megadott értéket, és visszaadja a régi értéket. Ez a művelet különösen hasznos számlálók és szemaforok növelésére.

Ezeket a műveleteket a processzor speciális utasításokkal valósítja meg. Például az x86 architektúrán a `CMPXCHG` utasítás valósítja meg a compare-and-swap műveletet, míg a `XADD` atomi fetch-and-add. A RISC-V architektúra load-linked/store-conditional (LL/SC) párt használ: a `lr.w` (load-reserved) betölti az értéket és lefoglalja a címet, a `sc.w` (store-conditional) pedig csak akkor ír vissza, ha a lefoglalás óta más nem módosította a memóriát. Ez a megközelítés optimista: ha a `sc.w` sikertelen, a műveletet újra kell próbálni.

A C programozási nyelv a C11 szabvány óta beépített atomi típusokat és műveleteket kínál a `<stdatomic.h>` fejlécben. A `_Atomic` típusminősítővel jelölt változókra a nyelv garantálja, hogy a rajtuk végzett műveletek atomiak. Például:

```c
#include <stdatomic.h>

atomic_int counter = 0;

// minden szál ezt hívja 100000-szer:
atomic_fetch_add(&counter, 1);
```

A `atomic_fetch_add` hívás egyetlen atomi műveletként hajtódik végre, így a korábbi elveszett frissítés nem fordulhat elő. A C atomi műveleteihez memória-rendezési (memory order) specifikáció tartozik, amelyre a következő pontban térünk ki.

Fontos megérteni, hogy az atomi művelet önmagában nem oldja meg az összes kritikus szakasz problémát. Ha egy kritikus szakasz több osztott változót érint, vagy összetett invariánst kell fenntartani, akkor egyetlen atomi művelet általában nem elegendő. Például ha két változó mindig konzisztens állapotban kell legyen, akkor a két változó módosítása közötti időben egy másik folyamat inkonzisztens állapotot figyelhet meg, még akkor is, ha mindkét módosítás külön-külön atomi. Ilyenkor kritikus szakaszt kell kialakítani, amelyet a következő fejezetben tárgyalt zárak vagy más szinkronizációs eszközök védenek.

4. Memóriabarrierek

A versenyhelyzetek kezelésének másik kulcseleme a memória-rendezés. A fordító és a processzor a teljesítmény növelése érdekében átrendezheti a memóriaműveletek sorrendjét. Ez az átrendezés egyetlen szálon belül megfigyelhetetlen, mert a szemantika megmarad, de többmagos rendszereken más szálak számára láthatóvá válhat, és hibát okozhat.

Tekintsük a következő példát: egy folyamat inicializál egy adatszerkezetet, majd beállít egy `ready` jelzőt. Egy másik folyamat figyeli a `ready` jelzőt, és ha igaz, akkor olvassa az adatszerkezetet.

```c
// Folyamat 1
data.value = 42;
ready = 1;

// Folyamat 2
while (!ready) { /* spin */ }
printf("%d\n", data.value);
```

Elvárásunk szerint a második folyamat mindig 42-t ír ki. Azonban a processzor átrendezheti az első folyamat két írását: előbb beírhatja a `ready = 1` értéket, és csak utána az `data.value = 42` értéket. Ekkor a második folyamat láthatja a `ready` jelzőt igaznak, miközben az adat még nincs inicializálva. A hibát memóriabarrierrel lehet megelőzni.

A memóriabarrier (memory barrier vagy fence) olyan utasítás, amely megakadályozza, hogy a barrier előtti és utáni memóriaműveletek felcserélődjenek. Két alapvető típusa van:

- **Acquire barrier:** a barrier utáni memóriaműveletek nem hajthatók végre a barrier előtt. Ezt használjuk olvasásnál, amikor egy jelző igazra váltása után akarunk adatot olvasni.
- **Release barrier:** a barrier előtti memóriaműveletek nem hajthatók végre a barrier után. Ezt használjuk írásnál, amikor az adatokat a jelző beállítása előtt kell elvégezni.

A fenti példában az első folyamatban release barrierre van szükség az `data.value = 42` és a `ready = 1` között, a második folyamatban pedig acquire barrierre a `ready` ellenőrzése és az `data.value` olvasása között. A C11 memória-rendezési modell ezeket a fogalmakat formalizálja a `memory_order_acquire`, `memory_order_release`, `memory_order_acq_rel` és `memory_order_seq_cst` specifikációkkal. Az alábbi táblázat összefoglalja a C11 memória-rendezési szinteket:

C11 memória-rendezés	Jelentés
`memory_order_relaxed`	Nincs sorrendi kényszer; csak az atomi művelet oszthatatlan.
`memory_order_acquire`	A betöltés utáni memóriaműveletek nem kerülnek a betöltés elé.
`memory_order_release`	A tárolás előtti memóriaműveletek nem kerülnek a tárolás mögé.
`memory_order_acq_rel`	Egyesíti az acquire és release szemantikát.
`memory_order_seq_cst`	Sorrendi konzisztencia; minden sorrendi zárat betart.

A gyakorlatban az atomi műveletekhez gyakran rendelhető memória-rendezési szemantika. Például az `atomic_store(&ready, 1, memory_order_release)` azt jelenti, hogy a `ready` beállítása előtt minden korábbi memóriaművelet befejeződik, míg az `atomic_load(&ready, memory_order_acquire)` biztosítja, hogy a `ready` beolvasása után a későbbi memóriaműveletek nem kerülnek a betöltés elé. Ez lehetővé teszi a fenti minta hatékony és hordozható megvalósítását.

A kernelben a memóriabarriereket a hardverfüggő réteg biztosítja. Például a Linux kernelben az `smp_mb()`, `smp_rmb()` és `smp_wmb()` makrók teljes, olvasási és írási barriereket valósítanak meg. A saját kernelünkben ezeket a fogalmakat a processzor-specifikus utasításokkal (például az x86 `mfence`, `lfence`, `sfence` vagy a RISC-V `fence`) kell implementálni. A memóriabarrier nélküli atomi művelet önmagában nem garantálja az összes szükséges sorrendet, ezért a két eszközt együtt kell alkalmazni.

5. Gyakorlati példa és tipikus hibák

A következő C kód egy tipikus versenyhelyzetet mutat be két szálon. A program két szálat indít, mindkettő 100000-szer növeli a globális `counter` változót. A várt végeredmény 200000, de a gyakorlatban a párhuzamos végrehajtás miatt a számláló rendszeresen kisebb lesz.

```c
// race.c - szemléltető példa, nem éles rendszerre
#include <pthread.h>
#include <stdio.h>

int counter = 0;

void *thread_func(void *arg) {
    for (int i = 0; i < 100000; i++) {
        counter++;  // nem atomi: load, add, store
    }
    return NULL;
}

int main(void) {
    pthread_t t1, t2;
    pthread_create(&t1, NULL, thread_func, NULL);
    pthread_create(&t2, NULL, thread_func, NULL);
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    printf("counter = %d\n", counter);
    return 0;
}
```

A hiba oka az, hogy a `counter++` kifejezés három gépi utasításból áll, és a két szál ezeket egymásba ékelheti. A javítás egyik módja az atomi művelet használata:

```c
// race_atomic.c - javított változat atomi művelettel
#include <pthread.h>
#include <stdatomic.h>
#include <stdio.h>

atomic_int counter = 0;

void *thread_func(void *arg) {
    for (int i = 0; i < 100000; i++) {
        atomic_fetch_add(&counter, 1);
    }
    return NULL;
}

int main(void) {
    pthread_t t1, t2;
    pthread_create(&t1, NULL, thread_func, NULL);
    pthread_create(&t2, NULL, thread_func, NULL);
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    printf("counter = %d\n", atomic_load(&counter));
    return 0;
}
```

Ez a változat minden futtatáskor 200000-et ad, függetlenül az ütemezéstől. A kód azonban továbbra is tartalmazhat más jellegű hibákat, ha a számláló mellett más osztott állapotot is kezelni kellene; ilyenkor a következő fejezetben tárgyalt zárak vagy magasabb szintű konstrukciók szükségesek.

Tipikus hibák a gyakorlatban

1. **A `volatile` téves használata.** A `volatile` kulcsszó C-ben és C++-ban nem garantál atomi végrehajtást, és nem nyújt memória-rendezési biztosítékot. Csak annyit jelent, hogy a fordító nem optimalizálhatja ki a változó memóriából való olvasását, de a load/store sorozat továbbra is megszakadhat, és a CPU átrendezheti a műveleteket. A `volatile` nem helyettesíti sem az atomi műveleteket, sem a zárakat.

2. **Túl széles kritikus szakasz.** Ha a kritikus szakaszba a szükségesnél több kódot teszünk, csökkentjük a párhuzamosságot, és növeljük a várakozási időt. A kritikus szakasz legyen a lehető legrövidebb, de tartalmazzon minden osztott adathozzáférést.

3. **Kihagyott szinkronizáció.** Gyakran előfordul, hogy egy adatstruktúra egyik mezőjét védjük, de egy másik mezőjét nem, vagy a védelmet kihagyjuk egy ritkán futó kódrészletben. A versenyhelyzet ilyenkor is hibát okozhat, csak ritkábban jelentkezik.

4. **Atomi művelet hibás használata összetett invariánsra.** Ha több változó között fennálló konzisztencia-feltételt kell fenntartani, egyetlen atomi művelet nem elegendő. Például két számláló arányának konzisztensnek kell lennie, de ha mindkettőt külön-külön növeljük, egy másik folyamat közbenső, inkonzisztens állapotot figyelhet meg. Ilyenkor az egész kritikus szakaszt zárolni kell.

5. **Memória-rendezési hibák.** Az atomi művelet önmagában nem garantálja a környező memóriaműveletek sorrendjét, ha nem megfelelő memória-rendezési specifikációt használunk. Például az `atomic_store(&ready, 1, memory_order_relaxed)` nem biztosítja, hogy a korábbi írások más szálak számára láthatóvá váljanak, mielőtt a `ready` igazra áll. A `memory_order_release` és `memory_order_acquire` párosítással lehet a szükséges sorrendet garantálni.

6. Labor: versenyhelyzet reprodukálása és javítása

A labor célja a versenyhelyzet reprodukálása, a hibás és a javított változat mérése, valamint annak bizonyítása, hogy az atomi művelet megoldja az elveszett frissítést. A kísérletet QEMU-ban futó Linux virtualizált környezetben végezzük, így a hoszt rendszer nem módosul. A konkrét környezeti paramétereket (kernel verzió, gcc verzió, CPU típus, memória méret) a reprodukciós csomagban rögzítjük; az itt megadott parancsok a csomaggal együtt érvényesek.

Kiinduló állapot. Hozzunk létre egy munkakönyvtárat, amelyben a fenti `race.c` és `race_atomic.c` forrásfájlok találhatók. A hoszt gép lemezét, bootloaderét és rendszerpartícióját nem módosítjuk; minden művelet a QEMU vendégben fut.

Build és futtatás. A vendég rendszerben a következő parancsokkal fordítjuk le a két programot:

```
gcc -O2 -pthread -o race race.c
gcc -O2 -pthread -std=c11 -o race_atomic race_atomic.c
```

A futtatáshoz indítsuk el a programot többször, és rögzítsük a kimenetet:

```
for i in $(seq 1 20); do ./race; done
```

Elvárt eredmény. A hibás `race` program kimenete a legtöbb futtatáskor kisebb, mint 200000, mert az elveszett frissítések miatt a számláló értéke alacsonyabb. A javított `race_atomic` program kimenete minden futtatáskor pontosan 200000. A két program viselkedése determinisztikus az adott QEMU konfigurációban, de a hibás program hibás futásainak száma függ az ütemezőtől és a CPU magok számától, ezért a mérésnél több futtatásból vett statisztikát kell felvenni.

Mérési napló. A mérési napló a következő oszlopokat tartalmazza: futtatás sorszáma, a `race` kimenete, a `race_atomic` kimenete, a várt érték (200000), és a megjegyzés (hibás-e a mért érték). A naplóban a mért és a várt érték külön oszlopban szerepel. Például:

```
# sorszám  race kimenet  race_atomic kimenet  várt  hiba
1          198732        200000               200000  igen
2          199981        200000               200000  igen
3          200000        200000               200000  nem
...
```

A fenti táblázat illusztráció; a tényleges naplót a reprodukciós csomagban kell elhelyezni. A mérési naplót a reprodukciós csomaggal együtt kell archiválni. A fordítási beállítások (`-O2`, `-pthread`), a CPU típus (`-cpu qemu64` vagy a vendégben látható típus), az órajelfrekvencia és a mérési módszer (a kimenet rögzítése shell pipe-on keresztül) mind a napló részét képezik.

Visszaállítás. A labor végén a QEMU vendég leállítható, a munkakönyvtár törölhető. A hoszt rendszeren semmilyen módosítás nem történik.

Értelmezés. A mérési napló alapján megállapítható, hogy a `race` program a futtatások egy részében 200000-től kisebb értéket ad, ami az elveszett frissítések miatt következik be. A `race_atomic` program viszont minden esetben pontosan a várt értéket adja. Ez a kísérlet demonstrálja az atomi művelet szerepét a versenyhelyzet megszüntetésében.

7. Összefoglalás és továbbvezetés

Ebben a fejezetben bevezettük a versenyhelyzet fogalmát, és megmutattuk, hogyan alakul ki egy összetett memóriaművelet megszakítása miatt. Definiáltuk a kritikus szakaszt, és bemutattuk a három követelményét: kölcsönös kizárás, haladás és korlátos várakozás. Megismertük az atomi műveletek és a memóriabarrierek szerepét a versenyhelyzetek kezelésében, és egy reprodukálható laborral demonstráltuk a jelenséget.

A következő fejezet ezekre az alapokra épít. A zárak (spinlock, mutex) az atomi műveleteket használják a belépési protokoll megvalósítására, a szemafórok pedig általánosabb szinkronizációs mintát adnak. A kritikus szakasz és a memóriabarrier fogalma nélkülözhetetlen lesz a helyes zármegoldások megértéséhez és megvalósításához.

9.2 Zárak, spinlock, mutex és szemafórok

A versenyhelyzet elkerülésének alapvető eszköze a zár. Amikor egy adatszerkezetet, erőforrást vagy eszközt egyszerre több végrehajtási szál is elérhet, a konzisztenciát csak úgy őrizhetjük meg, ha a kritikus szakaszt egy időben legfeljebb egy szál hajthatja végre. Az előző alfejezetben bevezetett kölcsönös kizárás elvét most konkrét szinkronizációs primitívekkel valósítjuk meg: megvizsgáljuk a pörgő zárat, az alvással várakozó mutexet és a szemafórt. Ezek mindegyike más költségmodellt, más várakozási stratégiát és más felhasználási mintát követel.

9.2.1 A zár fogalma és követelményei

A zár (lock) olyan szinkronizációs objektum, amely két alapműveletet biztosít: `acquire` (zárolás) és `release` (feloldás). A zárolt állapotban a zár egy tulajdonossal rendelkezik; amíg a tulajdonos fel nem oldja, minden további `acquire` hívás várakozásra kényszerül. A zár belső állapota egy logikai érték: szabad vagy foglalt. A műveleteknek atomi módon kell átváltaniuk ezt az állapotot, különben két szál egyszerre léphet be a kritikus szakaszba.

A helyes zár implementáció három tulajdonságot teljesít:

1. **Kölcsönös kizárás (mutual exclusion):** legfeljebb egy szál tartózkodhat a kritikus szakaszban.
2. **Haladás (progress):** ha egy szál belépni kíván, és a kritikus szakasz szabad, akkor valamelyik belépni kívánó szálnak véges időn belül be kell jutnia; a rendszer nem rekedhet meg egyetlen szálat sem beengedő állapotban.
3. **Korlátos várakozás (bounded waiting):** egyetlen szál sem éheztethető korlátlanul; minden belépési kísérlet véges számú más szál belépése után sikerül.

Ezek a követelmények már mutatják, hogy a zár nem pusztán egy bit; a várakozási sor, a kiválasztási stratégia és a hardveres atomi művelet együtt határozza meg a viselkedését. A gyakorlati rendszerekben a harmadik tulajdonság biztosítása külön figyelmet érdemel: a legegyszerűbb pörgő zár önmagában nem teljesíti a korlátos várakozást, mert a CPU-k versenye esetén egyes szálak akár végtelen ideig is elkerülhetik a belépést, ha a hardveres arbitráció kedvezőtlenül alakul.

A zár API-ja a leggyakrabban a következő formájú:

```
void lock_acquire(lock_t *lock);
void lock_release(lock_t *lock);
```

A hívó felelőssége, hogy a zárolás sikeres megszerzése után a kritikus szakaszt elhagyva mindig feloldja a zárat. A hibás használat – például a `release` elfelejtése egy hibaelágazásban – azonnal holtponthoz vagy sérült adathoz vezet.

9.2.2 Pörgő zár (spinlock)

A spinlock a legegyszerűbb zár. Ha a zár foglalt, a hívó nem alszik el, hanem aktív várakozást (busy waiting) végez: egy ciklusban ismételten ellenőrzi a zár állapotát, amíg az szabaddá nem válik. A megvalósítás egy atomi tesztelés és beállítás (test-and-set) vagy hasonló utasítás köré szerveződik. Az alábbi pszeudokód a lényeget mutatja:

```
void spin_lock(lock_t *lock):
    while atomic_test_and_set(lock) == 1:
        // spin: a zár foglalt, aktívan várakozunk
        pause          // opcionális: CPU-támogatott várakozási utasítás
    // beléptünk a kritikus szakaszba

void spin_unlock(lock_t *lock):
    atomic_store(lock, 0)
```

A `pause` utasítás x86 és más architektúrákon csökkenti a pörgés energiafogyasztását és a memóriabusz-torlódást, ezért a kernelbeli spinlockok gyakran alkalmazzák. A pszeudokódban az `atomic_test_and_set` egyetlen memóriaműveletként olvassa a zár aktuális értékét, és ha az 0, akkor 1-re állítja; a korábbi értéket adja vissza. A `memory_order_acquire` és `memory_order_release` szemantika biztosítja, hogy a kritikus szakaszban végrehajtott memóriaműveletek nem szivároghatnak ki a zár határain kívülre.

C11 atomikus könyvtárral is megvalósítható egy minimális spinlock:

```
#include <stdatomic.h>

atomic_flag lock = ATOMIC_FLAG_INIT;

void spin_acquire(void)
{
    while (atomic_flag_test_and_set_explicit(&lock,
                                             memory_order_acquire))
        ;   // pörgés
}

void spin_release(void)
{
    atomic_flag_clear_explicit(&lock, memory_order_release);
}
```

A spinlock legfontosabb előnye, hogy nincs rendszerhívási, kontextusváltási és ütemezési költsége. A várakozó szál a futási idejét a zár állapotának ellenőrzésére fordítja, ezért a spinlock csak akkor hatékony, ha a kritikus szakasz *rövid* és a várakozási idő *jellemzően nagyon rövid*. Tipikusan néhány tucat gépi utasítás, illetve néhány mikroszekundum alatt a spinlock időben előnyös lehet. Ennél hosszabb várakozás esetén a pörgő zár pazarló.

A spinlock költségmodellje jól szemlélteti a busy waiting és az alvás közötti választást. Legyen C_{spin} egyetlen pörgési ciklus költsége, N_{spin} a pörgési ciklusok várható száma, C_{sleep} pedig az alvás és felébresztés együttes költsége (rendszerhívás, kontextusváltás, cache-hideg indítás). A spinlock várakozási költsége:

$$C_{\text{spin}}^{\text{wait}} = N_{\text{spin}} \cdot C_{\text{spin}}$$

Az alvó záré közelítőleg:

$$C_{\text{mutex}}^{\text{wait}} \approx C_{\text{sleep}}$$

A spinlock akkor kifizetődő, ha:

$$N_{\text{spin}} \cdot C_{\text{spin}} < C_{\text{sleep}}$$

A valós rendszerekben C_{sleep} tipikusan több ezer vagy tízezer ciklus nagyságrendű, így a spinlock csak nagyon rövid, alacsony versengésű kritikus szakaszoknál nyer. Többmagos rendszereken a pörgő várakozás akár hatékony is lehet, mert a szál a várható rövid idő alatt képes átvenni a zárat anélkül, hogy a CPU-t átadná az ütemezőnek. Egyetlen magon viszont a spinlock használata értelmetlen: a pörgő várakozó elfoglalja a CPU-t, miközben a tulajdonos nem tud lefutni, hacsak nem szakítja meg a rendszer.

A kernelben a spinlockok használata gyakran kötelező azokban a kontextusokban, ahol a hívó nem aludhat el: megszakításkezelőkben, taskletekben, időzítő-függvényekben vagy olyan szinkronizációs pontokban, ahol az alvó zár használata a rendszer megfagyásához vezetne. A Linux-kernel `spinlock_t` típusa és a `spin_lock()`, `spin_unlock()` függvények ezt a szerepet töltik be. A kernel spinlockjainál a megszakítások tiltása is fontos: ha egy szál a spinlockot megszakítás-kontextussal is megosztja, akkor a kritikus szakaszba lépés előtt a megszakításokat le kell tiltani, különben a megszakítás-kontextusban ugyanarra a zárra várakozó kód holtponthoz vezet.

9.2.3 Alvó zár: mutex

A mutex („mutual exclusion”) olyan zár, amely sikertelen zárolási kísérlet esetén a hívó szálat alvó állapotba helyezi. A hívó elalszik, és csak akkor ébred fel, amikor a zár felszabadul. A várakozó szálakat a kernel vagy a felhasználói térben működő futex (fast userspace mutex) mechanizmus kezeli. A futex alapötlete, hogy a gyakori esetet – amikor a zár szabad – teljes egészében felhasználói térben, atomi művelettel kezeli; csak akkor történik rendszerhívás, amikor a szálnak ténylegesen aludnia kell, vagy amikor várakozókat kell ébreszteni. Ez drámaian csökkenti a mutex átlagos költségét.

A POSIX felületen a mutex a `pthread_mutex_t` típus. A legegyszerűbb használat:

```c
#include <pthread.h>
#include <errno.h>

pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;

int worker(void)
{
    int err;

    err = pthread_mutex_lock(&lock);
    if (err != 0) {
        /* hibakezelés: errno szerint */
        return err;
    }

    /* kritikus szakasz */

    err = pthread_mutex_unlock(&lock);
    if (err != 0) {
        /* hibakezelés */
        return err;
    }

    return 0;
}
```

A `pthread_mutex_lock` hívás három lehetséges kimenete:

- ha a mutex szabad, a hívó atomi módon megszerzi, és azonnal visszatér;
- ha a mutex foglalt, a hívó elalszik a mutex várakozási sorában;
- ha hibás a hívás (például érvénytelen mutex, vagy a mutexet a hívó már birtokolja), a függvény hibakóddal tér vissza, és a `errno` értékét beállítja.

A függvény visszatérési értékének ellenőrzése kritikus. A `pthread_mutex_lock` a szabvány szerint `EOWNERDEAD`, `EAGAIN`, `EINVAL`, `EDEADLK` hibakódokat adhat vissza. A gyakorlatban a leggyakoribb hibaforrás az, hogy a mutexet egy hibás ágban elfelejtjük feloldani. Emiatt a real-time és biztonságkritikus rendszerekben a kritikus szakaszokat gyakran RAII-szerű segédstruktúrával védik, vagy a kódban erős invariánsokat fogalmaznak meg.

A mutex használatának költsége három összetevőből áll:

- az atomi művelet és a gyors felhasználói térbeli út költsége, amikor a zár szabad;
- sikertelen zárolás esetén a futex rendszerhívás, a kontextusváltás és az alvó sorba helyezés költsége;
- a zár felszabadításakor a felébresztés, a kontextusváltás vissza a tulajdonoshoz, és a cache-újratöltés költsége.

Emiatt a mutex akkor előnyös, ha a kritikus szakasz hosszabb, vagy a várható várakozási idő nagyobb, mint a kontextusváltás költsége. A mutex a CPU-időt nem pazarolja pörgésre, ezért energiahatékonyabb, és lehetővé teszi, hogy a várakozó szál átadja a processzort más hasznos feladatnak. Ugyanakkor a mutex az alvás miatt hosszabb késleltetést is okozhat, ezért nem használható megszakítás-kontextusban vagy olyan kódban, ahol az alvás tiltott.

9.2.4 Busy waiting és alvás összehasonlítása

A spinlock és a mutex közötti választás a várakozási stratégia két szélsőségét képviseli. A döntést a következő szempontok határozzák meg:

Szempont	Spinlock	Mutex
Várakozási stratégia	aktív pörgés	alvás, CPU átadása
Költség rövid kritikus szakasznál	alacsony	közepes (futex út)
Költség hosszú várakozásnál	nagyon magas	alacsony
Energiafogyasztás	magas	alacsony
Megszakítás-kontextus	használható (megszakítás tiltással)	nem használható
Tulajdonjog	általában nincs explicit tulajdonos	van tulajdonos
Korlátos várakozás	nehezen biztosítható	a kernel ütemezője biztosíthatja

A táblázat mutatja, hogy a spinlock nem általános célú zár. A mutex a felhasználói programok és a kernel hosszabb, alvást megengedő kódjainak standard eszköze. A spinlock a kernelben az alvást tiltó kontextusok és a nagyon rövid kritikus szakaszok eszköze.

A busy waiting nem csak a CPU-időt pazarolja. A pörgő zárolás memóriabusz- és cache-koherencia-forgalmat generál, mert minden pörgési iteráció olvassa a zár változóját. Többmagos rendszerekben ez a forgalom lassíthatja a tulajdonos szál futását is, így a várakozási idő megnő. A modern CPU-k `pause` utasítása és a kernelbeli spinlockoknál alkalmazott „queue spinlock” technika ezt a hatást igyekszik csökkenteni, de a torlódás elve megmarad.

9.2.5 Szemafór: számláló és bináris

A szemafór (semaphore) általánosabb szinkronizációs eszköz, mint a zár. E. W. Dijkstra vezette be, és két atomi művelettel rendelkezik: a hagyományos jelölésben `P` (a holland „proberen”, azaz tesztelni) és `V` („verhogen”, azaz növelni), a modernebb API-kban `wait` és `signal`, illetve `down` és `up`. A szemafór egy belső nemnegatív egész számlálóval rendelkezik, amelyet inicializáláskor adunk meg.

A `wait` művelet szemantikája:

```
void sem_wait(semaphore_t *s):
    if s->count > 0:
        s->count = s->count - 1
    else:
        block_until_signaled(s)
```

A `signal` művelet:

```
void sem_signal(semaphore_t *s):
    if s->wait_queue is not empty:
        wake_one_waiter(s)
    else:
        s->count = s->count + 1
```

A számláló tehát a rendelkezésre álló erőforrás-példányok számát, vagy a felhalmozott jelzések számát tárolja. A `wait` ha a számláló nulla, a hívót alvó állapotba helyezi; a `signal` felébreszt egy várakozót, ha van, különben növeli a számlálót. A kulcs az, hogy a számláló és a várakozási sor állapotát együtt, atomi módon kell kezelni. A valós implementációkban ezt a kernelben egy spinlock vagy egy speciális várakozási sor védi.

A POSIX szemafór API:

```c
#include <semaphore.h>
#include <errno.h>

sem_t sem;
int err;

err = sem_init(&sem, 0, 3);      /* 3 erőforrás-példány */
if (err != 0) {
    /* hibakezelés errno alapján */
}

err = sem_wait(&sem);            /* P művelet */
if (err != 0) {
    /* hibakezelés, megszakítás (EINTR) kezelése */
}

/* erőforrás használata */

err = sem_post(&sem);            /* V művelet */
if (err != 0) {
    /* hibakezelés */
}
```

A szemafór két gyakori felhasználási mintát támogat. A **számláló szemafór** erőforrás-készletet kezel. Például egy adatbázis-kapcsolattár 10 kapcsolatot biztosít; a szemafórt 10-re inicializáljuk, a `wait` lefoglal egy kapcsolatot, a `signal` visszaadja azt. Ha mind a 10 kapcsolat foglalt, a 11. hívó elalszik. Ez a minta akkor hasznos, ha a kritikus szakaszhoz egy erőforrás-példány szükséges, és a példányok száma dinamikusan változhat.

A **bináris szemafór** számlálója csak 0 vagy 1 lehet, ezzel a kritikus szakasz kölcsönös kizárását biztosítja. Első ránézésre ugyanaz, mint a mutex, de van egy lényeges különbség: a szemafór nem rendelkezik tulajdonjoggal. A `signal` műveletet elvileg más szál is végrehajthatja, mint amelyik a `wait`-et megtette. A mutex esetében a zárolást a tulajdonosnak kell feloldania. Ez a tulajdonjog hiánya a bináris szemafórt rugalmasabbá teszi, de egyben hibalehetőséget is teremt: ha egy szál `signal`-t ad ki anélkül, hogy előtte `wait`-et hívott volna, a szemafór számlálója 1 fölé nő, és megsérti a bináris invariánst. A mutex ilyen hibát nem enged.

A szemafór a szálak közötti jelzésre is alkalmas. Egy szál `wait` műveletével vár egy eseményre, egy másik szál `signal`-lal jelzi azt. Ez a képesség teszi a szemafórt a gyártó–fogyasztó minták alapvető építőkövévé, bár a tiszta jelzési feladatra a később bemutatott condition variable és monitor gyakran kifejezőbb.

9.2.6 A szemafór költségei és hibái

A szemafór `wait` és `signal` művelete alvást és ébresztést is magában foglalhat, ezért költsége hasonló a mutexéhez: sikeres gyors út esetén atomi művelet, sikertelen `wait` esetén rendszerhívás, kontextusváltás és alvó sorba helyezés. A számláló szemafórnál a gyors út akkor is sikerül, ha a számláló pozitív, így az alvás csak tényleges erőforrás-hiány esetén következik be.

A tipikus hibák közé tartozik:

1. **Elfelejtett `signal`:** a szál elvégzi a `wait`-et, de a `signal`-t egy hibás ágban kihagyja. A számláló fokozatosan kimerül, a rendszer végül holtponthoz jut.
2. **Dupla `signal`:** egy szál többször ad ki `signal`-t ugyanarra a `wait`-re. Bináris szemafórnál az invariáns sérül; a számláló 1 fölé nő, és két szál egyszerre léphet be a kritikus szakaszba.

3. **Rekurzív használat:** a szemafór és a mutex alapváltozata nem rekurzív. Ha egy szál már birtokolja a zárat, és ismét megpróbálja megszerezni, az önmagát éhezteti. A mutexnél ez a helyzet `EDEADLK` hibakóddal jelezhető, a szemafórnál az alvó szál a saját jelzésére vár.
4. **Nem atomi ellenőrzés:** ha a zár állapotát egy külön `if` és egy külön beállítás vizsgálja, két szál egyszerre láthatja a zárat szabadnak. A zárolást mindig egyetlen atomi művelettel kell végezni.
5. **Felszabadítás címzettjének hibája:** mutexnél a `unlock`-ot a tulajdonosnak kell hívnia. Ha egy másik szál hívja, a viselkedés nem definiált, és a zár állapota sérülhet.

A szemafór és a mutex közötti választás tehát szemantikai kérdés is. Ha a cél kizárólag kölcsönös kizárás, a mutex a helyes választás, mert a tulajdonjog és a hibakezelés erősebb. Ha a cél erőforrás-számlálás vagy eseményjelzés, a számláló szemafór a kifejező eszköz.

9.2.7 Labor: spinlock és mutex összehasonlítása

A labor célja megmérni a spinlock és a mutex várakozási költségét rövid, alacsony versengésű kritikus szakasz esetén, és bemutatni, hogy a spinlock előnye ebben a tartományban mérhető, míg hosszabb kritikus szakasznál a mutex hatékonyabb. A labor **emulált környezetben** fut; a hoszt valódi lemezét, bootloaderét vagy rendszerpartícióját nem szabad módosítani.

9.2.8 Kiinduló állapot és konfiguráció

- Emulált környezet: QEMU virtuális gép.
- CPU: 4 vCPU (x86_64), opcionálisan `-smp 4`.
- Memória: 2 GiB RAM.
- Operációs rendszer: minimalista Linux-alapú rendszer user space, kernelverzió: helyőrző (ellenőrizendő a reprodukciós csomagban).
- Fordító: GCC 13.2, optimalizálás `-O2`, `-pthread` linkelés.
- Mérőeszköz: `perf stat`, `clock_gettime(CLOCK_MONOTONIC)` a programon belül.
- Mérési módszer: minden mérési pont 5 ismétlés, az átlag és a szórás rögzítése.

9.2.9 A program

A program kétféle kritikus szakasz-változatot futtat: egy spinlockkal és egy mutexszel. A kritikus szakasz egy közös számláló növelése, amely igen rövid. A kontrollként használt második változat a kritikus szakasz hosszát mesterségesen növeli egy rövid, de nem optimalizálható ciklussal, hogy a hosszabb várakozás hatása is látható legyen.

```
#include <pthread.h>
#include <stdio.h>
#include <time.h>
#include <errno.h>

#define THREADS 4
#define ITERATIONS 1000000

static pthread_spinlock_t spin;
static pthread_mutex_t mtx = PTHREAD_MUTEX_INITIALIZER;
static volatile long counter;          /* szemléltető, élesben atomi vagy lock védi */
static int use_spin;

static void *worker(void *arg)
{
```

```
    for (int i = 0; i < ITERATIONS; i++) {
        if (use_spin) {
            int err = pthread_spin_lock(&spin);
            if (err != 0) return NULL;
            counter++;
            err = pthread_spin_unlock(&spin);
            if (err != 0) return NULL;
        } else {
            int err = pthread_mutex_lock(&mtx);
            if (err != 0) return NULL;
            counter++;
            err = pthread_mutex_unlock(&mtx);
            if (err != 0) return NULL;
        }
    }
    return NULL;
}
```

A `counter` növelése a zár által védett, ezért a kiinduló programban a `volatile` csak szemléltetést szolgál, nem szinkronizációs eszköz. A kritikus szakasz hossza párutasításos.

9.2.10 Build és futtatási parancsok

```
gcc -O2 -pthread -o lockbench lockbench.c
./lockbench spin
./lockbench mutex
perf stat -e context-switches,cache-misses,cpu-clock ./lockbench spin
perf stat -e context-switches,cache-misses,cpu-clock ./lockbench mutex
```

A `perf` a rendszerben elérhető eseményeket gyűjti; a pontos elnevezések a kerneltől és a CPU-modelltől függnek, ezért a reprodukciós csomagban a `perf list` kimenete is rögzítendő.

9.2.11 Elvárt eredmény

A **várt** eredmény a busy waiting és alvás költségmodellje alapján:

- A rövid kritikus szakasznál a spinlock futási ideje a mutexénél **kisebb**, mert nincs rendszerhívás és kontextusváltás. A várt különbség a hardver és a kernel függvényében néhány százaléktól néhány tíz százalékig terjedhet.
- A mutex esetén a futási idő nagyobb szórású lehet, mert a futex alvás és ébresztés időzítése az ütemezőtől függ; a kontextusváltások száma mérhetően nagyobb.
- A spinlock esetén a `perf stat` `cache-misses` és `context-switches` értékei alacsonyak, a `cpu-clock` viszont magasabb lehet, mert a pörgő szálak folyamatosan fogyasztják a CPU-időt.
- A mesterségesen hosszabb kritikus szakasznál a várt sorrend megfordul: a mutex várhatóan hatékonyabb, mert a várakozó szálak elalszanak, és nem fogyasztják a CPU-t.

9.2.12 Mérési napló

A naplóban minden mérési pontnál külön kell rögzíteni a **mért** és a **várt** értéket. A várt érték a fenti levezetésből származik; a mért értéket a hallgató a labor során tölti ki. Példa naplóbejegyzés:

Mérési pont	Konfiguráció	Várt eredmény	Mért eredmény
spinlock, rövid kritikus szakasz	4 szál, 1 000 000 iteráció	futási idő < mutex; context switches 0	*(üres)*
mutex, rövid kritikus szakasz	4 szál, 1 000 000 iteráció	futási idő > spinlock; context switches > 0	*(üres)*

A mérési naplóhoz a `clock_gettime` és a `perf stat` kimenete is csatolandó. A nem futtatott mérést nem szabad ellenőrzött eredményként feltüntetni; a labor célja éppen a saját mérés elvégzése.

9.2.13 Összefoglalás

A zár a kölcsönös kizárás alapvető eszköze, a szemafór pedig a számláló erőforrás-kezelés és a jelzés általánosabb primitívje. A zárak megvalósíthatók pörgő várakozással (spinlock) vagy alvással (mutex). A választás a kritikus szakasz hosszától, a várakozási időtől és a kontextus megengedéseitől függ. A spinlock rövid, alacsony versengésű szakaszokhoz és alvást tiltó környezetekhez való; a mutex hosszabb szakaszokhoz és felhasználói programokhoz. A szemafór a számláló jellegével erőforrás-készletek és eseményjelzések kezelésére alkalmas. A helyes használat feltétele az atomi műveletek, a tulajdonjog tiszteletben tartása és a hibakezelés gondos megtervezése. A következő alfejezetben a monitor és a condition variable segítségével magasabb szintű, kifejezőbb szinkronizációs mintákra térünk át.

9.2.14 Önellenőrző kérdések

1. Mikor használj spinlockot mutex helyett? Indokold a költségmodell alapján.
2. Mi a különbség a bináris szemafór és a mutex között a tulajdonjog szempontjából?
3. Miért nem elég egy közönséges `if (lock == 0) lock = 1;` kód a zároláshoz?
4. Mi történik, ha egy szál elfelejti a `sem_post` hívást egy hibás ágban?
5. A labor mérési naplójában miért kell külön kezelni a mért és a várt eredményt?

9.3 Monitorok, condition variable és termelő-fogyasztó

A zárak és szemaforok használata során gyakran szembesülünk azzal, hogy a kritikus szakasz védelme önmagában nem elegendő a párhuzamos programok helyes viselkedéséhez. A kölcsönös kijárás biztosítása mellett szükség van arra is, hogy egy szál megvárhasson egy feltétel teljesülését, anélkül hogy aktívan pörögne (busy-wait), és anélkül hogy más szálakat feleslegesen blokkolna. A szemaforok erre részben alkalmasak, de a használatuk könnyen hibákhoz vezet: egy elfelejtett `signal` vagy `wait` holtpontot okozhat, a számláló értékének helytelen inicializálása pedig sértheti a kölcsönös kijárást. A monitor és a hozzá tartozó feltételváltozó (condition variable) olyan magasabb szintű absztrakció, amely a kölcsönös kijárást és a feltételre várakozást egyetlen jól meghatározott szerkezetbe zárja, és ezzel jelentősen csökkenti a szinkronizációs hibák lehetőségét.

9.3.1 A monitor mint szinkronizációs absztrakció

A monitor fogalmát C. A. R. Hoare és Per Brinch Hansen vezette be az 1970-es években. A monitor egy absztrakt adattípus, amely a következő három összetevőből áll:

1. **Belső állapot**: a monitor által védett adatok, amelyekhez kívülről csak a monitor műveletein keresztül lehet hozzáférni.
2. **Műveletek (metódusok)**: olyan eljárások, amelyek a monitor belső állapotát olvassák vagy módosítják. A monitor garantálja, hogy ezek a műveletek **egyszerre csak egy szál** hajthatja végre, tehát a műveletek automatikusan kölcsönös kijárás alatt futnak.
3. **Feltételváltozók**: olyan várakozási sorok, amelyek lehetővé teszik, hogy egy szál a monitoron belül átmenetileg felfüggessze magát, amíg egy adott feltétel nem teljesül.

A monitor lényege, hogy a programozónak nem kell külön zárolási műveleteket (`lock`, `unlock`) elhelyeznie minden kritikus szakasz köré. A monitor minden belépési pontján automatikusan megtörténik a kölcsönös kijárás, és a kilépéskor automatikusan felszabadul a zár. Ez a tulajdonság jelentősen csökkenti annak az esélyét, hogy a programozó elfelejtsen egy zárat feloldani, vagy rossz sorrendben hívja a zárolási műveleteket.

A gyakorlatban a monitorok gyakran nyelvi szintű konstrukcióként jelennek meg. A Java nyelvben például minden objektum rendelkezik egy beépített monitorral: a `synchronized` blokk vagy metódus belépésekor a szál megszerzi az objektum monitorzárját, a kilépéskor pedig elengedi. A `wait`, `notify` és `notifyAll` metódusok a monitorhoz tartozó feltételváltozót vezérlik. A Python `threading.Condition` osztálya hasonló funkcionalitást nyújt explicit módon, a `with` utasítással kombinálva.

A C nyelvben nincs beépített monitor, ezért a POSIX szálkönyvtár (`pthread`) elemeiből építhetünk monitort. A megfelelő komponensek a következők:

- **`pthread_mutex_t`**: a monitor zárja, amely a kölcsönös kijárást biztosítja.
- **`pthread_cond_t`**: feltételváltozó, amely a várakozást és az ébresztést támogatja.
- **Egy struktúra**, amely a védett adatokat, a mutexet és a feltételváltozókat egységbe foglalja.

Egy ilyen C nyelvű monitor sablonja a következőképpen nézhet ki:

```
typedef struct {
    pthread_mutex_t mutex;
    pthread_cond_t  cond;
    int             state;      // védett állapot
    // további adatok...
} monitor_t;

void monitor_init(monitor_t *m) {
    pthread_mutex_init(&m->mutex, NULL);
    pthread_cond_init(&m->cond, NULL);
    m->state = 0;
}

void monitor_destroy(monitor_t *m) {
    pthread_cond_destroy(&m->cond);
    pthread_mutex_destroy(&m->mutex);
}
```

A monitor műveletei mindig a mutex zárolásával kezdődnek, és a kilépés előtt feloldják azt. A feltételváltozón való várakozás azonban speciális: a `pthread_cond_wait` hívás atomi módon **elengedi a mutexet**, és blokkolja a hívó szálat. Amikor a szálat felébresztik, a `pthread_cond_wait`

visszatérése előtt **újra megszerzi a mutexet**, így a szál a várakozásból visszatérve ismét a monitor birtokában van. Ez az atomi elengedés és újraszerzés a monitor helyes működésének alapja.

9.3.2 Feltételváltozók: várakozás és ébresztés

A feltételváltozó (condition variable) egy olyan várakozási sor, amelyhez a következő alapműveletek tartoznak:

- `wait(cv, mutex)` vagy `pthread_cond_wait`: a hívó szál atomi módon elengedi a `mutex`-et, és várakozik a `cv` feltételváltozóra. Amikor egy másik szál jelzi a feltételváltozót, a várakozó szál felébred, és a visszatérés előtt ismét megszerzi a mutexet. A hívó szál tehát a `wait` után ismét a monitor birtokában van.
- `signal(cv)` vagy `pthread_cond_signal`: ha van a feltételváltozón várakozó szál, akkor felébreszti **egy** várakozó szálat. Ha nincs várakozó, a jelzés elveszik (ellentétben a szemafor `signal` műveletével, amely növeli a számlálót).
- `broadcast(cv)` vagy `pthread_cond_broadcast`: az összes várakozó szálat felébreszti. Ez akkor hasznos, ha a feltétel több szál számára is igazzá válhat, vagy ha nem tudjuk pontosan, hány szálat kell felébreszteni.

A `signal` és a `broadcast` közötti választás fontos tervezési döntés. A `signal` hatékonyabb, mert csak egy szálat ébreszt fel, és elkerüli a felesleges kontextusváltásokat. Ugyanakkor a `signal` csak akkor biztonságos, ha garantált, hogy a felébresztett szál feltétele valóban teljesült, és más várakozó szálak állapota nem romlik. Gyakori hiba, hogy a programozó `signal`-t használ, de a feltételt nem ellenőrzi újra, vagy több szál is várakozik ugyanarra a feltételre. Ilyenkor a `broadcast` biztonságosabb, bár nagyobb terhelés esetén „thundering herd” problémát okozhat: az összes szál egyszerre ébred fel, és versenyez a monitor zárjáért.

A feltételváltozó belső megvalósítása a kernelben általában egy várakozási sorra épül. A `wait` művelet során a szál lekerül a futási sorból, és bekerül a feltételváltozó várakozási sorába; a `signal` egy várakozó szálat áthelyez a futási sorba. A pontos szemantika azonban két fő modellre osztható:

- **Hoare-féle monitor**: a `signal` hatására a jelző szál azonnal felfüggesztődik, és a jelzett szál azonnal belép a monitorba. Ez garantálja, hogy a jelzett feltétel a jelzett szál belépésekor még igaz, de a megvalósítás bonyolultabb, mert a jelző szálat is fel kell függeszteni.
- **Mesa-féle monitor**: a `signal` csak felébreszti a várakozó szálat, de az nem kapja meg azonnal a monitort; a jelző szál tovább fut, és a jelzett szál csak akkor lép be, amikor a jelző elhagyja a monitort. Ez egyszerűbb, de a jelzett szálnak a belépéskor **újra ellenőriznie kell a feltételt**, mert közben más szál is megváltoztathatta az állapotot.

A POSIX és a legtöbb modern operációs rendszer a Mesa-féle szemantikát követi, ezért a tankönyv további részében ezt feltételezzük. Ez a szemantika indokolja a **feltétel újraellenőrzésének** szabályát.

9.3.3 A feltétel újraellenőrzése: miért `while` és nem `if`?

A feltételváltozón várakozó szál a következő mintát követi:

```
pthread_mutex_lock(&m->mutex);
while (!feltétel) {
    pthread_cond_wait(&m->cond, &m->mutex);
}
// A feltétel teljesül, a monitor birtokában vagyunk.
// ... kritikus művelet ...
pthread_mutex_unlock(&m->mutex);
```

A `while` ciklus használata nem stilisztikai kérdés, hanem a helyesség alapvető követelménye. Ennek három fő oka van:

1. **Hamis ébredés (spurious wakeup)**: a `pthread_cond_wait` specifikációja megengedi, hogy a szál jelzés nélkül is felébredjen. Ez a jelenség ritka, de előfordulhat, ezért a feltételt mindig újra kell ellenőrizni. Ha `if` utasítást használnánk, egy hamis ébredés után a szál a feltétel hamis állapotában is belépne a kritikus szakaszba.
2. **A jelzés és a tényleges futás közötti verseny**: Mesa-szemantika esetén a `signal` után a jelző szál tovább fut a monitorban, és közben más szálak is beléphetnek. Előfordulhat, hogy a jelzett szál csak később kapja meg a monitort, és addigra egy másik szál megváltoztatja az állapotot, így a feltétel ismét hamissá válik. A `while` ciklus ezt a helyzetet is kezeli.
3. **Több várakozó szál**: ha több szál vár ugyanarra a feltételre, és a jelző csak egyet ébreszt fel, a felébresztett szál feltétele lehet, hogy már nem teljesül, mert egy másik, korábban felébresztett szál elvitte az erőforrást. A `while` ciklus biztosítja, hogy a szál ilyenkor visszaálljon várakozni.

A szabály tehát egyszerű: **a feltételváltozón való várakozást mindig `while` ciklusban kell elhelyezni, és a feltételt a monitor által védett állapot alapján kell kiértékelni.** A `while` ciklus feltétele az a predikátum, amelynek teljesülése szükséges a kritikus művelet végrehajtásához.

Gyakori hiba, hogy a programozó a feltételt a `wait` hívás előtt ellenőrzi, majd `if`-fel dönt, és a `wait` után nem ellenőrzi újra. Ez a fenti okok miatt versenyhelyzetet okozhat. A helyes minta mindig a `while` ciklus.

9.3.4 Termelő-fogyasztó minta monitorral

A termelő-fogyasztó (producer-consumer) probléma a konkurens programozás egyik klasszikus példája. Adott egy korlátos kapacitású puffer, amelybe termelő szálak elemeket helyeznek el, fogyasztó szálak pedig elemeket vesznek ki. A következő invariánsoknak kell teljesülniük:

- A pufferben lévő elemek száma soha nem lehet negatív, és nem haladhatja meg a kapacitást.
- A termelő csak akkor helyezhet el elemet, ha van szabad hely.
- A fogyasztó csak akkor vehet ki elemet, ha a puffer nem üres.
- A pufferből kivett elemek sorrendje és integritása megmarad.

Monitor segítségével a megoldás a következő. A monitor állapota tartalmazza a puffert, a pufferben lévő elemek számát, valamint a beírási és kiolvasási indexeket. Két feltételváltozót használunk:

- **`not_full`**: arra várnak a termelők, amíg szabad hely nem lesz a pufferben.
- **`not_empty`**: arra várnak a fogyasztók, amíg elem nem lesz a pufferben.

A monitor műveletei a `put(item)` és a `get()`.

A `put` művelet pszeudokódja:

```
put(item):
    lock(mutex)
    while count == CAPACITY:
        wait(not_full, mutex)
    buffer[in] = item
    in = (in + 1) % CAPACITY
    count = count + 1
    signal(not_empty)
    unlock(mutex)
```

A `get` művelet pszeudokódja:

```
get():
    lock(mutex)
    while count == 0:
        wait(not_empty, mutex)
    item = buffer[out]
    out = (out + 1) % CAPACITY
    count = count - 1
    signal(not_full)
    unlock(mutex)
    return item
```

Figyeljük meg a következő pontokat:

- A `while` ciklusban várunk, nem `if`-fel, a korábban tárgyalt okok miatt.
- A `signal` hívás a monitor zárjának birtokában történik, de a Mesa-szemantika szerint a jelzés nem adja át azonnal a vezérlést; a jelző szál a `unlock`-ig folytatja a futást. Ezért a jelzett szál csak a zár elengedése után léphet be a monitorba.
- A `signal` hívás előtt a feltétel már teljesül (a `put` esetén a puffer nem üres, a `get` esetén a puffer nem tele), ezért a jelzés biztonságos.

A fenti pszeudokód C nyelvű, pthread alapú megvalósítása így nézhet ki:

```c
#include <pthread.h>
#include <stdbool.h>
#include <stdio.h>
#include <stdlib.h>

#define CAPACITY 8

typedef struct {
    int buffer[CAPACITY];
    int in;
    int out;
    int count;
    pthread_mutex_t mutex;
    pthread_cond_t not_full;
    pthread_cond_t not_empty;
} bounded_buffer_t;

void bb_init(bounded_buffer_t *bb) {
    bb->in = 0;
    bb->out = 0;
    bb->count = 0;
    pthread_mutex_init(&bb->mutex, NULL);
    pthread_cond_init(&bb->not_full, NULL);
    pthread_cond_init(&bb->not_empty, NULL);
}

void bb_destroy(bounded_buffer_t *bb) {
    pthread_cond_destroy(&bb->not_empty);
```

```
    pthread_cond_destroy(&bb->not_full);
    pthread_mutex_destroy(&bb->mutex);
}

void bb_put(bounded_buffer_t *bb, int item) {
    pthread_mutex_lock(&bb->mutex);
    while (bb->count == CAPACITY) {
        pthread_cond_wait(&bb->not_full, &bb->mutex);
    }
    bb->buffer[bb->in] = item;
    bb->in = (bb->in + 1) % CAPACITY;
    bb->count++;
    /* A puffer már nem üres, ezért felébresztünk egy fogyasztót. */
    pthread_cond_signal(&bb->not_empty);
    pthread_mutex_unlock(&bb->mutex);
}

int bb_get(bounded_buffer_t *bb) {
    pthread_mutex_lock(&bb->mutex);
    while (bb->count == 0) {
        pthread_cond_wait(&bb->not_empty, &bb->mutex);
    }
    int item = bb->buffer[bb->out];
    bb->out = (bb->out + 1) % CAPACITY;
    bb->count--;
    /* A puffer már nincs tele, ezért felébresztünk egy termelőt. */
    pthread_cond_signal(&bb->not_full);
    pthread_mutex_unlock(&bb->mutex);
    return item;
}
```

Ez a kód egyetlen elemet helyez el vagy vesz ki, és mindig csak egy várakozó szálat ébreszt fel. Ha a termelők és fogyasztók száma nagy, és a feltétel több szál számára is igazzá válhat, előfordulhat, hogy a `signal` nem elég, mert a felébresztett szál mellett más szálak is jogosan várakozhatnak. Ilyenkor `broadcast` használata is indokolt lehet, de a `while` ciklus miatt a `broadcast` nem okoz helytelenséget, csak teljesítménycsökkenést.

A termelő és fogyasztó szálak tipikus vázlata:

```
void *producer(void *arg) {
    bounded_buffer_t *bb = (bounded_buffer_t *)arg;
    for (int i = 0; i < ITEMS_PER_PRODUCER; i++) {
        int item = produce_next_item();  // szemléltetés: egyedi érték
        bb_put(bb, item);
    }
    return NULL;
}

void *consumer(void *arg) {
    bounded_buffer_t *bb = (bounded_buffer_t *)arg;
    for (int i = 0; i < ITEMS_PER_CONSUMER; i++) {
        int item = bb_get(bb);
        consume_item(item);  // szemléltetés: feldolgozás
    }
    return NULL;
}
```

A `produce_next_item` és `consume_item` függvények a feladat szempontjából tetszőlegesek lehetnek; a szinkronizáció szempontjából csak az számít, hogy a `bb_put` és a `bb_get` helyesen védi a puffert.

9.3.5 Tipikus hibák és elkerülésük

A monitorok és feltételváltozók használata során a következő hibák fordulnak elő leggyakrabban:

1. **`if` használata `while` helyett**: ahogy korábban részleteztük, a feltétel újraellenőrzése nélkül a hamis ébredés, a versenyhelyzet vagy a több várakozó szál miatt a szál érvénytelen állapotban léphet be a kritikus szakaszba. A szabály: mindig `while` ciklusban várjunk.
2. **Jelzés a monitor zárjának elengedése nélkül**: ha a szál a `signal` vagy `broadcast` hívás után nem engedi el a mutexet, a felébresztett szál nem tud belépni a monitorba, és a jelző szál esetleg további műveleteket végez, miközben a jelzett szál feleslegesen blokkol. Bár ez nem okoz holtpontot, ha a jelző végül elengedi a zárat, a hatékonyság romlik. A legtöbb implementációban a `signal` után a jelző szál folytatja a futást, ezért a `signal`-t általában közvetlenül a `unlock` előtt érdemes hívni.
3. **A jelzés elvesztése**: a `signal` akkor is elveszhet, ha a feltételváltozón nincs várakozó szál. Ez önmagában nem hiba, ha a várakozó szálak mindig `while` ciklusban ellenőrzik a feltételt, hiszen ha a feltétel már teljesül, a várakozó szál nem is fog `wait`-re kerülni. A probléma akkor jelentkezik, ha a programozó arra számít, hogy a jelzés „megmarad”, mint a szemaforoknál. A feltételváltozóknál a jelzés állapotot nem tárol, ezért a feltételt a monitor állapotában kell tárolni, és a `while` ciklusban ellenőrizni.
4. **Rossz feltételváltozó használata**: ha a termelő a `not_empty` helyett a `not_full`-on vár, vagy a jelzést rossz feltételváltozón adja, a szálak végtelen várakozásba kerülhetnek. Ezért a feltételváltozók elnevezése és a hozzájuk tartozó predikátum dokumentálása elengedhetetlen.
5. **Holtpont a monitorban**: ha a monitor egyik művelete egy másik monitorra vár, miközben a másik monitor egy szálat vár az elsőre, holtpont alakulhat ki. Általános szabály, hogy a monitor műveletei ne hívjanak más monitorokat, különösen ne olyan sorrendben, amely ciklikus függőséget okoz.
6. **Túlzott `broadcast`**: a `broadcast` minden várakozó szálat felébreszt, ami nagy terhelés esetén „thundering herd” jelenséget okoz: a szálak egyszerre versenyeznek a zárért, és sok felesleges kontextusváltás történik. Ha a feltétel csak egy szál számára válik igazzá, használjunk `signal`-t; ha több szál is folytathatja, akkor is érdemes mérlegelni, hogy a `signal`-lal való egyenkénti ébresztés nem elegendő-e.
7. **A monitoron kívüli várakozás**: ha a szál a `wait` hívás előtt nem tartja a mutexet, a `wait` viselkedése nem definiált. A POSIX `pthread_cond_wait` megköveteli, hogy a hívó szál birtokolja a mutexet. Ennek elmulasztása hibát eredményez.

Ezeknek a hibáknak a felismerése és elkerülése a gyakorlatban a monitorok helyes használatának alapja. A következő labor ezeket a szempontokat teszteli.

9.3.6 Labor: termelő-fogyasztó monitorral

A labor célja, hogy a hallgató a monitor és a feltételváltozók használatát egy reprodukálható kísérletben gyakorolja. A feladat egy korlátos puffert használó termelő-fogyasztó rendszer megvalósítása C nyelven, POSIX szálakkal, emulált Linux környezetben (például QEMU virtuális gépben vagy konténerben). A kód szemléltetés és oktatási célú, nem éles rendszeren fut.

9.3.7 Kiinduló állapot és környezet

- **Operációs rendszer**: Linux (emulált környezetben, például Debian 12 QEMU virtuális gépben).
- **CPU**: x86_64, 2 virtuális mag.
- **Memória**: 512 MiB RAM.
- **Fordító**: GCC 12.2, fordítási opciók: `-O2 -pthread -Wall -Wextra`.
- **Szálak száma**: 2 termelő, 2 fogyasztó.
- **Pufferkapacitás**: 8 elem.
- **Elemek száma**: termelőnként 10 000, összesen 20 000 elem.

A fenti konfiguráció a reprodukciós csomag részét képezi, és a mérési naplóban rögzítendő.

9.3.8 Build és futtatási parancs

A forráskódot például `bounded_buffer.c` néven mentjük. A build parancs:

```
gcc -O2 -pthread -Wall -Wextra -o bounded_buffer bounded_buffer.c
```

A futtatás:

```
./bounded_buffer
```

A program kimenetének tartalmaznia kell a feldolgozott elemek számát, a futási időt, és opcionálisan a puffer állapotára vonatkozó ellenőrzéseket.

9.3.9 A program felépítése

A program a fent bemutatott `bounded_buffer_t` struktúrát és a `bb_put`, `bb_get` műveleteket használja. A termelő szálak egyedi egész számokat állítanak elő, amelyeket a pufferbe helyeznek. A fogyasztó szálak kiveszik az elemeket, és ellenőrzik, hogy a kapott érték a várt sorozat része-e. Ez az ellenőrzés a szinkronizáció helyességének tesztelésére szolgál: ha a puffer sérül, a fogyasztó hibás értéket kap, és a program hibát jelez.

A termelő és fogyasztó szálak indításához `pthread_create`-et használunk, a befejezéshez pedig `pthread_join`-t. A méréshez a `clock_gettime(CLOCK_MONOTONIC, ...)` függvényt használjuk a futási idő meghatározására.

A program vázlata:

```
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define CAPACITY 8
#define NUM_PRODUCERS 2
#define NUM_CONSUMERS 2
#define ITEMS_PER_PRODUCER 10000
#define TOTAL_ITEMS (NUM_PRODUCERS * ITEMS_PER_PRODUCER)

/* ... bounded_buffer_t definíció és műveletek ... */

static int produced[NUM_PRODUCERS][ITEMS_PER_PRODUCER];

void *producer(void *arg) {
```

```
    long id = (long)arg;
    bounded_buffer_t *bb = /* paraméterből */;
    for (int i = 0; i < ITEMS_PER_PRODUCER; i++) {
        int item = (int)(id * ITEMS_PER_PRODUCER + i); // egyedi érték
        produced[id][i] = item;
        bb_put(bb, item);
    }
    return NULL;
}

void *consumer(void *arg) {
    bounded_buffer_t *bb = /* paraméterből */;
    int *seen = calloc(TOTAL_ITEMS, sizeof(int));
    if (!seen) { perror("calloc"); exit(EXIT_FAILURE); }
    for (int i = 0; i < TOTAL_ITEMS / NUM_CONSUMERS; i++) {
        int item = bb_get(bb);
        if (item < 0 || item >= TOTAL_ITEMS || seen[item]) {
            fprintf(stderr, "Hibás elem: %d\n", item);
            exit(EXIT_FAILURE);
        }
        seen[item] = 1;
    }
    free(seen);
    return NULL;
}
```

A `produced` tömb a termelt elemek nyilvántartására szolgál, a `seen` tömb pedig azt ellenőrzi, hogy minden elemet pontosan egyszer fogyasztottak-e el. Ez a séma a szinkronizáció helyességének tesztelésére alkalmas.

9.3.10 Elvárt eredmény

A programnak hibamentesen kell lefutnia. A következő kimenet várható (a pontos idő változhat):

```
Futási idő: 0.0234 s
Összes elem: 20000
Hibák: 0
```

A mért eredmény a futási idő és a hibák száma. A várt eredmény az, hogy a program befejeződik, a hibák száma 0, és a feldolgozott elemek száma pontosan 20 000. Ha a program holtpontra fut, vagy hibát jelez, az a szinkronizáció hibájára utal.

9.3.11 Mérési napló

A mérési napló a következő adatokat tartalmazza:

- Hardver: emulált x86_64, 2 mag, 512 MiB RAM.
- Fordító: GCC 12.2, `-O2 -pthread -Wall -Wextra`.
- Operációs rendszer: Debian 12 (Linux 6.1 kernel).
- Futási idő: mért érték (például 23,4 ms).
- CPU idő: `getrusage`-zal mérve (opcionális).
- Kontextusváltások száma: `vmstat` vagy `pidstat` segítségével (opcionális).

A mérési naplóban a mért és a várt értékeket külön kell feltüntetni. A várt érték a helyes működés (0 hiba, 20 000 elem), a mért érték pedig a tényleges futási eredmény.

9.3.12 Visszaállítás

A labor nem módosítja a hoszt rendszert, mivel emulált környezetben fut. A virtuális gép vagy konténer a labor után törölhető vagy alapállapotba állítható. A forráskód és a mérési napló a reprodukciós csomag részeként archiválandó.

9.3.13 Önellenőrző kérdések

1. Mi a különbség a szemafor és a feltételváltozó között a jelzés tárolása szempontjából?
2. Miért kell a feltételt `while` ciklusban és nem `if`-fel ellenőrizni a feltételváltozón való várakozás során?
3. Hogyan biztosítja a monitor a kölcsönös kijárást, és hogyan lépnek ki a szálak a monitorból a `wait` során?
4. Milyen problémát okozhat a `broadcast` túlzott használata, és mikor indokolt mégis a használata?
5. A termelő-fogyasztó példában miért van szükség két külön feltételváltozóra?

9.3.14 Összefoglalás

A monitor a kölcsönös kijárás és a feltételre várakozás egységes absztrakciója, amely a szinkronizációs hibák jelentős részét kiküszöböli. A feltételváltozók a `wait`, `signal` és `broadcast` műveletekkel teszik lehetővé, hogy a szálak hatékonyan várakozzanak egy feltétel teljesülésére anélkül, hogy aktívan pörögnének. A Mesa-féle szemantika miatt a feltételt mindig újra kell ellenőrizni, ezért a várakozást `while` ciklusban kell megvalósítani. A termelő-fogyasztó minta monitorral történő megoldása jól szemlélteti ezeket az elveket: két feltételváltozóval, a puffer állapotának védelmével és a feltételek újraellenőrzésével a rendszer helyesen és hatékonyan működik. A következő alfejezet a holtpontok kezelésére épít majd ezekre a fogalmakra.

9.4 Holtpont, élok és deadlock-kezelés

Az előző alfejezetben megismertük a zárolási és szinkronizációs primitíveket. Most egy olyan hibajelenséget vizsgálunk, amely akkor lép fel, ha több folyamat vagy szál egyszerre tart erőforrásokat, és további erőforrásokra vár, miközben a többiek is ugyanezt teszik. Az ilyen helyzetet **holtpontnak** (deadlock) nevezzük. A holtpont az egyik legnehezebben reprodukálható és leggyakrabban félreértett konkurenciahiba, ezért külön alfejezetet szentelünk neki.

9.4.1 A holtpont fogalma és következményei

A holtpont definíciója informálisan: folyamatok egy olyan halmaza, ahol minden folyamat egy olyan erőforrásra vár, amelyet a halmaz egy másik folyamata tart, és egyik sem tudja elengedni a már megszerzett erőforrásait addig, amíg a várt erőforrást meg nem kapja. Mivel minden folyamat vár, és egyik sem halad, a rendszer az adott erőforrás-halmaz szempontjából leáll.

Formálisan adott n folyamat $P_1, P_2, \ldots, P_n$. Minden P_i rendelkezik bizonyos erőforrásokkal, és kér egy további erőforrást, amelyet valamelyik másik P_j tart. Ilyenkor P_i addig blokkolt, amíg P_j el nem engedi az erőforrást. Ha a várakozási lánc körbe ér, akkor holtpont alakul ki.

A holtpont nem tévesztendő össze a **kiéheztetéssel** (starvation), amikor egy folyamat azért nem jut erőforráshoz, mert más folyamatok folyamatosan elhalásszák előle, de a rendszer egésze halad. Holtpont esetén a résztvevő folyamatok mindegyike blokkolt, és nincs előrehaladás.

9.4.2 A Coffman-feltételek

A holtpont kialakulásához négy feltételnek egyszerre kell teljesülnie. Ezeket **Coffman-feltételeknek** nevezzük:

1. **Kölcsönös kizárás (mutual exclusion).** Az erőforrást egyidejűleg legfeljebb egy folyamat használhatja. Ez hardveres vagy logikai okokból adott: például egy nyomtató, egy meghatározott memóriaterület, vagy egy diszk-blokk. Ha az erőforrás megosztható lenne (például egy csak olvasható fájl), akkor nem alakulhatna ki holtpont miatta.
2. **Tarts és várj (hold and wait).** Egy folyamat tart legalább egy erőforrást, és közben további erőforrásra vár. Ha minden folyamat egyszerre csak egy erőforrást igényelne, és a többit csak az első elengedése után kérné, akkor nem állhatna elő ez a helyzet.
3. **Nincs megelőzés (no preemption).** Az erőforrást nem lehet erőszakkal elvenni a folyamattól, csak az képes elengedni, aki megszerezte. Egyes rendszerekben a CPU idő például preemptálható, de egy fájlzár vagy egy hardveres portozás általában nem.
4. **Ciklikus várakozás (circular wait).** Létezik folyamatoknak olyan $P_1, P_2, \ldots, P_n$ köre, ahol P_1 vár egy erőforrásra, amelyet P_2 tart, P_2 vár egy erőforrásra, amelyet P_3 tart, és így tovább, végül P_n vár egy erőforrásra, amelyet P_1 tart. A ciklikus várakozás a holtpont közvetlen oka, de önmagában csak akkor elégséges feltétele a holtpontnak, ha minden erőforrásból egyetlen példány van.

E feltételek együttes fennállása **szükséges** a holtponthoz. Ha bármelyiket megszüntetjük, holtpont nem alakul ki. Ez a megfigyelés alapozza meg a megelőzési stratégiákat.

9.4.3 Erőforrás-allokációs gráf és holtpontdetektálás elméletben

A holtpontok modellezésére az **erőforrás-allokációs gráf** (resource allocation graph, RAG) használható. A gráf csúcsai kétfélék:

- **Folyamat-csúcsok** (körrel jelölve), pl. $P_1, P_2, \ldots$
- **Erőforrás-csúcsok** (négyzettel jelölve), pl. $R_1, R_2, \ldots$

Az élek kétfélék:

- **Allokációs él:** $R_j \to P_i$ azt jelenti, hogy az R_j erőforrás egy példányát P_i birtokolja.
- **Kérési él:** $P_i \to R_j$ azt jelenti, hogy P_i vár az R_j erőforrásra.

Ha az erőforrásokból több példány is van, akkor az erőforrás-csúcsban kis körök vagy szám jelzi a példányszámot, és az élek az egyes példányokhoz kapcsolódnak.

A holtpont detektálása gráfelméleti cikluskeresésre vezethető vissza: ha a gráfban nincs ciklus, akkor nincs holtpont. Ha van ciklus, az **szükséges** feltétele a holtpontnak. Ha minden erőforrásból csak egy példány van, akkor a ciklus **elégséges** is: a ciklus pontosan a holtpontos folyamatokat és erőforrásokat jelöli. Ha több példányú erőforrások is szerepelnek, akkor a ciklus jelenléte szükséges, de nem elégséges – ilyenkor további vizsgálat kell, amelyet a detektálási algoritmusoknál tárgyalunk.

9.4.3.1 Példa: két folyamat, két erőforrás

Tekintsünk két folyamatot, P_1 és P_2, valamint két erőforrást, R_1 és R_2. A rendszer állapota a következő:

- P_1 tartja R_1-et, és kéri R_2-t.
- P_2 tartja R_2-t, és kéri R_1-et.

A gráfban a következő élek vannak:

- $R_1 \to P_1$ (allokáció)
- $R_2 \to P_2$ (allokáció)
- $P_1 \to R_2$ (kérés)
- $P_2 \to R_1$ (kérés)

Itt jól látható a ciklus: $P_1 \to R_2 \to P_2 \to R_1 \to P_1$. Mivel minden erőforrásból egy példány van, a ciklus holtpontot jelent. P_1 és P_2 kölcsönösen várnak egymásra, egyik sem tud továbbhaladni.

Ez a klasszikus példa jól mutatja a ciklikus várakozás kialakulását.

9.4.4 Holtpontkezelési stratégiák áttekintése

A holtpont kezelésére négy fő stratégia létezik:

1. **Strucc stratégia (ignore).** A rendszer feltételezi, hogy a holtpont valószínűtlen, és nem tesz semmit ellene. A felhasználó vagy az alkalmazás felelőssége, hogy elkerülje. Ez a gyakorlatban sok általános célú operációs rendszerben (pl. Linux, Windows) így működik: a kernel nem detektálja a felhasználói szintű holtpontokat.
2. **Megelőzés (prevention).** A tervezés során megtörik a Coffman-feltételek legalább egyikét, így a holtpont strukturálisan lehetetlenné válik.
3. **Elkerülés (avoidance).** A rendszer minden erőforrás-kérést megvizsgál, és csak akkor elégíti ki, ha a kielégítés után a rendszer **biztonságos állapotban** marad. Ehhez ismernie kell minden folyamat maximális erőforrásigényét.
4. **Detektálás és helyreállítás.** A rendszer hagyja, hogy a holtpont bekövetkezzen, de rendszeresen ellenőrzi az erőforrás-allokációs gráfot (vagy a várakozási gráfot), és ha holtpontot talál, beavatkozik: folyamatokat szüntet meg vagy erőforrásokat von vissza.

A következőkben ezeket részletesebben ismertetjük.

9.4.5 Megelőzés: a Coffman-feltételek megtörése

A holtpont megelőzése azt jelenti, hogy a négy feltételből legalább egyet **megsértünk** a tervezés során. Vizsgáljuk meg mindegyiket.

9.4.5.1 1. Kölcsönös kizárás megtörése

Ha egy erőforrás megoszthatóvá tehető, akkor nincs szükség kölcsönös kizárásra, és a holtpont az adott erőforrás miatt nem jöhet létre. Ez nem mindig lehetséges: a hardveres erőforrások (nyomtató, szalagos egység) nem oszthatók meg. Azonban bizonyos esetekben alkalmazható:

- **Csak olvasható fájlok:** több folyamat is olvashatja ugyanazt a fájlt.
- **Spooling:** a nyomtató kimenetét egy spooler könyvtárba írják, és a nyomtatót egyetlen démon kezeli. Így a felhasználói folyamatok nem közvetlenül a nyomtatóért versengenek, hanem a spooler fájlokért, amelyeket a spooler sorosít.
- **Virtuális erőforrások:** ha egy erőforrásból virtuálisan többet biztosítunk, mint fizikailag van, akkor csökkenthetjük a versenyt.

A kölcsönös kizárás megtörése azonban nem mindig alkalmazható, ezért a gyakorlatban ritkán elegendő önmagában.

9.4.5.2 2. Tarts és várj megtörése

A „tarts és várj” feltétel megszüntetéséhez el kell érni, hogy egy folyamat ne tartson erőforrást, miközben másikra vár. Két fő módszer van:

- **Egyidejű lekérés:** minden folyamatnak az összes szükséges erőforrást egyszerre kell lekérnie, mielőtt bármelyiket használni kezdi. Így ha nem kapja meg mindet, akkor egyet sem tart, tehát nem tud úgy várni, hogy közben tart valamit. Hátránya: alacsony erőforrás-kihasználtság, mert a folyamat akkor is lefoglalja az erőforrásokat, amikor még nem használja őket.
- **Elengedés kérés előtt:** ha egy folyamat már tart erőforrásokat, és újabbat szeretne, akkor előbb el kell engednie az összes meglévőt, majd egyszerre újra megkísérelni a lekérést. Ez javítja a kihasználtságot, de növeli a rendszerhívások számát, és az újrapróbálkozás során versenyhelyzetet okozhat.

Ez a stratégia sok rendszerben használatos, de komplex alkalmazásoknál nehéz minden erőforrásigényt előre ismerni.

9.4.5.3 3. Nincs megelőzés megtörése

Ha a rendszer képes **erőszakkal elvenni** az erőforrást egy folyamattól, akkor a holtpont megszüntethető. Ez a feltétel azonban a legtöbb erőforrásnál nem teljesíthető biztonságosan. Például egy fájlzár vagy egy hardveres regiszter nem vehető el anélkül, hogy a folyamat inkonzisztens állapotba kerülne. A CPU idő preemptálása viszont tipikus példája a megelőzésnek: az ütemező elveszi a CPU-t a futó folyamattól, és másnak adja. Ezért a processzor-ütemezésben nincs holtpont.

A gyakorlatban a „nincs megelőzés” megtörése csak olyan erőforrásoknál lehetséges, ahol a folyamat állapota menthető és visszaállítható (pl. CPU regiszterek). Általános célú erőforrásoknál ritkán alkalmazzák.

9.4.5.4 4. Ciklikus várakozás megtörése: erőforrás-sorrend

A leggyakrabban alkalmazott megelőzési technika a **globális erőforrás-sorrend** (resource ordering) felállítása. Minden erőforrástípust egyértelmű sorszámmal látunk el, például $1, 2, \ldots, m$. A szabály: minden folyamat csak **növekvő sorrendben** kérhet erőforrásokat. Vagyis ha egy folyamat már tart egy R_j erőforrást, akkor csak olyan R_k erőforrást kérhet, amelyre $k > j$. Ezzel lehetetlenné válik a ciklikus várakozás, mert egy kör kialakulásához valamelyik folyamatnak csökkenő irányban kellene kérnie, ami tilos.

Példa: legyenek R_1, R_2, R_3 erőforrások $1, 2, 3$ sorszámmal. Ha P_1 már tartja R_1-et, és P_2 tartja R_2-t, akkor P_1 kérheti R_2-t vagy R_3-at, de P_2 nem kérheti R_1-et. Így nem alakulhat ki $P_1 \to R_2 \to P_2 \to R_1 \to P_1$ kör.

Az erőforrás-sorrend könnyen bevezethető, de gondot okozhat, ha az alkalmazás természetes erőforrás-használati sorrendje nem egyezik meg a globális sorrenddel. Ilyenkor a folyamatnak esetleg olyan erőforrást is le kell foglalnia, amelyet később használna, ami csökkenti a hatékonyságot.

9.4.6 Elkerülés: a bankár algoritmus

A holtpont **elkerülése** (avoidance) dinamikus megközelítés: a rendszer minden erőforrás-kérést megvizsgál, és csak akkor elégíti ki, ha biztos, hogy a kielégítés után a rendszer **biztonságos állapotban** marad. Ehhez a rendszernek előre tudnia kell minden folyamat maximális erőforrásigényét.

A klasszikus elkerülési algoritmus a **bankár algoritmus**, amelyet Dijkstra javasolt. Az alábbi adatszerkezeteket használja:

- **Available**: az aktuálisan szabad erőforráspéldányok száma erőforrástípusonként.
- **Max**: minden folyamatra a maximálisan igényelt erőforrások száma.
- **Allocation**: minden folyamatra a már lefoglalt erőforrások száma.
- **Need**: a még szükséges erőforrások száma, azaz Need = Max − Allocation.

Biztonságos állapot: akkor mondjuk, hogy a rendszer biztonságos, ha létezik a folyamatoknak egy olyan $P_{i_1}, P_{i_2}, \ldots, P_{i_n}$ végrehajtási sorrendje, ahol minden P_{i_k} le tudja foglalni a szükséges erőforrásait, befejezi a munkáját, elengedi az erőforrásait, és a rendszer így el tudja látni a következő folyamatot is. Ha ilyen sorrend nem létezik, a rendszer **nem biztonságos**. A nem biztonságos állapot nem feltétlenül jelent holtpontot, de fennáll a holtpont kialakulásának lehetősége.

Biztonsági ellenőrzés algoritmusa:

1. Legyen `Work` egy tömb, amelyet inicializálunk az `Available` értékével.
2. Legyen `Finish[i]` hamis minden folyamatra.
3. Keressünk egy olyan P_i folyamatot, amelyre `Finish[i]` hamis, és `Need[i] <= Work`. (Azaz a folyamat szükségletei kielégíthetők a jelenleg szabad erőforrásokból.)
4. Ha találunk ilyen P_i-t, akkor:
   - Tegyük fel, hogy P_i befejeződik: `Work = Work + Allocation[i]`.
   - Állítsuk `Finish[i]`-t igazra.
   - Térjünk vissza a 3. lépéshez.
5. Ha nincs több ilyen folyamat, és minden `Finish[i]` igaz, akkor a rendszer biztonságos. Ha marad hamis `Finish[i]`, akkor a rendszer nem biztonságos.

Bankár algoritmus erőforrás-kérés esetén:

Amikor egy P_i folyamat kér egy erőforrást:

1. Ellenőrizzük, hogy a kérés nem haladja-e meg a `Need[i]`-t. Ha igen, hiba.
2. Ellenőrizzük, hogy a kérés nem haladja-e meg az `Available`-t. Ha igen, a folyamat vár.
3. Próbaképpen foglaljuk le az erőforrást: csökkentsük `Available`-t, növeljük `Allocation[i]`-t, csökkentsük `Need[i]`-t.
4. Futtassuk a biztonsági ellenőrzést.
5. Ha a rendszer biztonságos, a lefoglalás végleges. Ha nem, a kérést elutasítjuk, és a folyamatnak várnia kell, miközben a rendszer visszaáll az eredeti állapotba.

9.4.6.1 Példa a bankár algoritmusra

Legyen három erőforrástípusunk: A, B, C, egy-egy példánnyal. Négy folyamatunk van: P_1, P_2, P_3, P_4. A rendszer állapota:

Folyamat	Allocation (A,B,C)	Max (A,B,C)	Need (A,B,C)
P_1	(0,1,0)	(0,2,0)	(0,1,0)
P_2	(1,0,0)	(2,0,0)	(1,0,0)
P_3	(0,0,1)	(1,0,1)	(1,0,0)
P_4	(0,0,0)	(0,0,1)	(0,0,1)

`Available` = (0,1,0) (mert az összesen (1,1,1) erőforrásból a lefoglalt (1,1,1) – várjunk, pontosan: Allocation összege = (1,1,1)? Nézzük: P1: (0,1,0), P2: (1,0,0), P3: (0,0,1), P4: (0,0,0) -> összeg (1,1,1). Akkor Available = (0,0,0). Ez rossz példa, mert nem marad szabad erőforrás. Inkább legyen összesen (2,2,2). Legyen:

- P1: Allocation (0,1,0), Max (2,2,0) -> Need (2,1,0)
- P2: Allocation (1,0,0), Max (1,1,0) -> Need (0,1,0)
- P3: Allocation (0,0,1), Max (1,0,2) -> Need (1,0,1)
- P4: Allocation (0,0,0), Max (0,0,2) -> Need (0,0,2)

Available = (2 - (0+1+0+0), 2 - (1+0+0+0), 2 - (0+0+1+0)) = (1,1,1)? Számoljuk: Összesen: (2,2,2). Allocation összege: A:0+1+0+0=1, B:1+0+0+0=1, C:0+0+1+0=1. Available = (2-1,2-1,2-1) = (1,1,1).

Most ellenőrizzük a biztonságos állapotot:

- `Work` = (1,1,1)
- Találunk-e olyan folyamatot, ahol Need <= Work?
  - P1 Need (2,1,0) > (1,1,1)? Nem.
  - P2 Need (0,1,0) <= (1,1,1)? Igen, mert 0<=1, 1<=1, 0<=1. -> P2 lefut: Work = (1,1,1) + Allocation P2 (1,0,0) = (2,1,1)
  - Most P1 Need (2,1,0) <= (2,1,1)? Igen (2<=2, 1<=1, 0<=1) -> P1 lefut: Work = (2,1,1) + (0,1,0) = (2,2,1)
  - P3 Need (1,0,1) <= (2,2,1)? Igen. -> P3 lefut: Work = (2,2,1) + (0,0,1) = (2,2,2)
  - P4 Need (0,0,2) <= (2,2,2)? Igen. -> P4 lefut: Work = (2,2,2) + (0,0,0) = (2,2,2)
  - Minden Finish igaz -> a rendszer biztonságos.

Most tegyük fel, hogy P1 kér egy A erőforrást (1,0,0). Ha teljesítjük:

- Available = (0,1,1) (mert A-ból 1-et elvettünk)
- P1 Allocation = (1,1,0), Need = (1,1,0)

Ellenőrizzük a biztonságot:

- Work = (0,1,1)
- Találunk-e olyat, ahol Need <= Work?
  - P2 Need (0,1,0) <= (0,1,1)? Igen -> P2 fut, Work = (0,1,1)+(1,0,0)=(1,1,1)
  - P1 Need (1,1,0) <= (1,1,1)? Igen -> P1 fut, Work = (1,1,1)+(1,1,0)=(2,2,1)
  - P3 Need (1,0,1) <= (2,2,1)? Igen -> P3 fut, Work = (2,2,1)+(0,0,1)=(2,2,2)
  - P4 Need (0,0,2) <= (2,2,2)? Igen -> P4 fut.
  - Minden igaz, tehát a kérés biztonságos, teljesíthető.

Ha viszont P3 kérne egy A-t, akkor (1,0,0) hozzáadása P3-hoz: P3 Allocation = (1,0,1), Need = (0,0,1). Available = (0,1,1). Biztonsági ellenőrzés:

- Work = (0,1,1)
- P2 Need (0,1,0) <= Work -> P2 fut, Work=(1,1,1)
- P1 Need (2,1,0) > (1,1,1)? Nem (A-ból 2 kell, de csak 1 van). P3 Need (0,0,1) <= (1,1,1)? Igen -> P3 fut, Work=(1,1,1)+(1,0,1)=(2,1,2)
- P1 Need (2,1,0) <= (2,1,2)? Igen -> P1 fut, Work=(2,1,2)+(0,1,0)=(2,2,2)
- P4 fut. Tehát ez is biztonságos.

A példa jól mutatja az algoritmus lépéseit. A gyakorlatban a bankár algoritmust ritkán használják, mert drága (minden kérésnél biztonsági ellenőrzés), és a folyamatok maximális erőforrásigényét nehéz pontosan előre megadni.

9.4.7 Detektálás és helyreállítás

A detektálás és helyreállítás stratégia elfogadja, hogy holtpont előfordulhat, de rendszeresen ellenőrzi a rendszer állapotát, és ha holtpontot talál, beavatkozik.

9.4.7.1 Detektálás

Egypéldányos erőforrások esetén a detektálás a **várakozási gráf** (wait-for graph) cikluskeresésére redukálódik. A várakozási gráfot az erőforrás-allokációs gráfból kapjuk úgy, hogy az erőforrás-csúcsokat elhagyjuk, és közvetlenül a folyamatok közé húzunk élt: ha P_i kér egy olyan erőforrást, amelyet P_j tart, akkor a várakozási gráfban $P_i \to P_j$ él van. Ha ebben a gráfban ciklus található, akkor holtpont van.

A cikluskeresés történhet **DFS-sel** vagy **topologikus rendezéssel**. A detektálás gyakorisága lehet:

- Minden erőforrás-kéréskor (drága).
- Rendszeres időközönként.
- Amikor a CPU-kihasználtság alacsony, és gyanú merül fel.

Többpéldányos erőforrások esetén a detektálás hasonló a bankár algoritmushoz, de itt nem a maximális igényeket vesszük, hanem a jelenlegi kéréseket. Az algoritmus:

1. Legyen `Work` = Available, és `Finish[i]` = false minden folyamatra, amelynek van allocationja; true, ha nincs.

2. Keressünk olyan P_i-t, amelyre `Finish[i]` false, és `Request_i <= Work`, ahol `Request_i` a P_i által kért erőforrások vektora.
3. Ha találunk ilyet, akkor `Work += Allocation_i`, `Finish[i] = true`, és ismételjük.
4. Ha az algoritmus végén marad olyan `Finish[i]`, ami false, akkor holtpont van, és a false értékű folyamatok a holtpontban résztvevők.

9.4.7.2 Helyreállítás

Holtpont észlelése után a rendszernek helyre kell állítania a normál működést. Két fő módszer:

1. **Folyamat-terminálás.** Egy vagy több holtpontos folyamatot megszüntetünk. Lehet:
   - Minden holtpontos folyamatot leállítani (egyszerű, de nagy veszteség).
   - Fokozatosan, egyenként leállítani őket, amíg a holtpont meg nem szűnik (kisebb pusztítás, de többszöri detektálást igényel). A kiválasztási szempontok: prioritás, futási idő, erőforrás-használat, interaktivitás, batch jelleg.
2. **Erőforrás-visszavonás (preemptálás).** Az egyik folyamattól elveszünk egy erőforrást, és odaadjuk egy másiknak, hogy a holtpont feloldódjon. Ezt általában csak akkor lehet megtenni, ha a folyamat állapota menthető és később visszaállítható. A visszavonás után a folyamatot gyakran **rollback**-elni kell egy korábbi ellenőrzőpontra (checkpoint). Ezért a detektálás és helyreállítás módszer szorosan kapcsolódik a checkpoint/restart mechanizmusokhoz.

A detektálás és helyreállítás előnye, hogy nem korlátozza az erőforrás-használatot, és csak akkor avatkozik be, ha valódi holtpont lép fel. Hátránya, hogy a detektálás maga is erőforrás-igényes, és a helyreállítás során a folyamatok munkája elvész.

9.4.8 Élő lock (livelock) és kiéheztetés

A holtponthoz hasonló, de attól elkülönülő jelenség az **élő lock** vagy **élok** (livelock). Livelockban a folyamatok nem blokkolnak le, hanem folyamatosan „futnak” és reagálnak egymásra, de a kölcsönös udvariasság miatt nem haladnak előre.

Klasszikus példa: két folyamat egy szűk folyosón próbál átmenni egymással szemben. Mindkettő félreáll, hogy utat engedjen a másiknak, de egyszerre lépnek félre, majd egyszerre próbálnak visszamenni, és újra ütköznek. A folyamatok aktívan csinálnak valamit, de a rendszer egy helyben toporog.

Programozási analógia: két szál mindegyike megpróbál felvenni két zárat, de ha nem sikerül mindkettőt megszerezni, elengedi az éppen megszerzettet, és újra próbálkozik. Ha mindkettő egyszerre engedi el és egyszerre próbálja újra, akkor örökké ismétlődhet a ciklus anélkül, hogy bármelyik is mindkét zárat egyszerre birtokolná.

Különbség holtpont és élok között: - Holtpontban a folyamatok **blokkoltak**, várakoznak, nem fogyasztanak CPU-t. - Livelockban a folyamatok **futnak**, de nem haladnak; CPU-időt fogyasztanak, de a rendszer nem jut előre.

A livelock megoldása hasonló a holtpont megelőzéséhez: erőforrás-sorrend, véletlen visszalépési idő (backoff), vagy korlátozott próbálkozások száma. Gyakran alkalmaznak **exponenciális backoff**-ot, ahol a várakozási idő minden újabb ütközés után nő.

A **kiéheztetés** (starvation) szintén rokon: egy folyamat azért nem jut erőforráshoz, mert más folyamatok folyamatosan megelőzik. Ez általában a prioritásos ütemezés hibája, és megoldható öregedéssel (aging).

9.4.9 Gyakorlati hibák és helyes minták

A fejlesztők által elkövetett leggyakoribb holtpont-okozó hibák:

1. **Inkonzisztens lock-sorrend.** Több zár megszerzése során a szálak nem mindig ugyanabban a sorrendben kérik a zárakat. Ez a ciklikus várakozás klasszikus forrása. **Megoldás:** mindenhol azonos, dokumentált sorrendet használjunk, például mindig a kisebb indexű zártól a nagyobb felé haladva.
2. **Túl sok erőforrás egyidejű megtartása.** Ha egy függvény több mutexet tart, miközben egy újabbat próbál szerezni, az növeli a holtpont kockázatát. **Megoldás:** minimalizáljuk a kritikus szakasz hosszát, és lehetőleg egyszerre csak egy zárat tartsunk.
3. **Nincs timeout a lock megszerzésére.** Ha egy szál korlátlanul várakozik egy mutexre, holtpont esetén örökre blokkolt marad. **Megoldás:** használjunk `trylock`-ot vagy timeout-os lockot, és ha nem sikerül megszerezni, engedjük el a már megszerzett zárakat, és próbáljuk újra később (esetleg exponenciális backoff-fal).
4. **Nincs tisztázott erőforrás-birtoklási hierarchia.** Nagyméretű rendszerekben nehéz átlátni, hogy melyik komponens milyen zárat használ. **Megoldás:** dokumentáljuk a lock-hierarchiát, és végezzünk statikus elemzést.
5. **Félreértett rekurzív zárolás.** Ha egy szál ugyanazt a nem-rekurzív mutexet kétszer próbálja megszerezni, önmaga ellen holtpontot okozhat. **Megoldás:** rekurzív mutex használata, vagy a kód újratervezése, hogy ne legyen szükség újrazárolásra.

Helyes minta:

```
/* Példa: két mutex biztonságos megszerzése rögzített sorrendben */
void lock_both(struct mutex *a, struct mutex *b) {
    if (a < b) {          /* pointer-cím szerinti sorrend */
        mutex_lock(a);
        mutex_lock(b);
    } else if (b < a) {
        mutex_lock(b);
        mutex_lock(a);
    } else {
        /* ugyanaz a mutex, elég egyszer */
        mutex_lock(a);
    }
}
```

Ez a minta garantálja, hogy minden szál ugyanabban a sorrendben szerzi meg a két zárat, így nem alakulhat ki ciklikus várakozás. A pointer-cím szerinti rendezés gyakori, mert egyértelmű és mindig konzisztens.

Egy másik hasznos minta:

```
/* Időzített lock, visszalépéssel */
int try_lock_with_timeout(struct mutex *m, int timeout_ms) {
    int ret = mutex_timedlock(m, timeout_ms);
    if (ret != 0) {
        /* nem sikerült, elengedjük a már megszerzett zárakat,
           és visszalépünk (retry) */
        return -1;
    }
    return 0;
}
```

A gyakorlatban a fenti minták kombinációjával csökkenthető a holtpont kialakulásának esélye.

9.4.10 Labor: erőforrás-allokációs gráf és holtpontdetektálás

Ebben a laborban egy egyszerű Python eszközt készítünk, amely modellezi az erőforrás-allokációs gráfot, és cikluskereséssel detektálja a holtpontot. A labor célja, hogy a tanult elméletet egy ellenőrizhető, reprodukálható példán alkalmazzuk.

9.4.10.1 Kiinduló állapot

A rendszer két folyamatból (P_1, P_2) és két erőforrásból (R_1, R_2) áll. A gráf élei a következők:

- $P_1 \to R_2$ (kérés)
- $P_2 \to R_1$ (kérés)
- $R_1 \to P_1$ (allokáció)
- $R_2 \to P_2$ (allokáció)

Ez a korábban bemutatott klasszikus holtpont gráf. A várakozási gráfban ez egy ciklus: $P_1 \to P_2 \to P_1$, tehát holtpont van.

A feladat: implementáljunk egy DFS-alapú cikluskeresőt, amely a gráf éllistájából kiírja, hogy van-e ciklus, és ha igen, felsorolja a ciklusban résztvevő csúcsokat.

9.4.10.2 Build és futtatási parancs

A laborhoz nincs szükség fordításra, mert Python szkriptet használunk. A környezet:

- Python 3.10 vagy újabb
- Egy terminál vagy Jupyter notebook

A szkript elérési útja: `deadlock_detect.py`.

```
# Futtatás
python3 deadlock_detect.py
```

9.4.10.3 A detektáló szkript (illusztráció)

Az alábbi kód a gráfot csúcs- és éllistával reprezentálja, és DFS-sel keres ciklust. A ciklus detektálásakor kiírja a holtpontos folyamatokat.

```
# deadlock_detect.py
# Erőforrás-allokációs gráf és holtpontdetektálás

# Csúcsok: folyamatok (P) és erőforrások (R)
nodes = ['P1', 'P2', 'R1', 'R2']
edges = [
    ('P1', 'R2'),  # kérés
    ('P2', 'R1'),  # kérés
    ('R1', 'P1'),  # allokáció
    ('R2', 'P2')   # allokáció
]

# Éllista átalakítása szomszédsági listává
adj = {node: [] for node in nodes}
for u, v in edges:
    adj[u].append(v)

WHITE, GRAY, BLACK = 0, 1, 2
```

```
color = {node: WHITE for node in nodes}
path = []
cycles = []

def dfs(u):
    color[u] = GRAY
    path.append(u)
    for v in adj[u]:
        if color[v] == WHITE:
            dfs(v)
        elif color[v] == GRAY:
            # Ciklust találtunk
            idx = path.index(v)
            cycles.append(path[idx:] + [v])
    path.pop()
    color[u] = BLACK

for node in nodes:
    if color[node] == WHITE:
        dfs(node)

if cycles:
    print("Holtpont észlelhető. Ciklus(ok):")
    for c in cycles:
        print(" -> ".join(c))
else:
    print("Nincs holtpont (nincs ciklus a gráfban).")
```

9.4.10.4 Elvárt eredmény

A fenti kiinduló állapotban a várt kimenet:

```
Holtpont észlelhető. Ciklus(ok):
P1 -> R2 -> P2 -> R1 -> P1
```

Ez azt jelenti, hogy a gráfban ciklus van, és az összes erőforrás egypéldányos, tehát holtpont áll fenn.

9.4.10.5 Mérési napló és összehasonlítás

Várt eredmény: a kimenet szerint holtpont van, a ciklus: `P1 -> R2 -> P2 -> R1 -> P1`.

Mért eredmény: futtassuk a szkriptet, és rögzítsük a konzol kimenetét. Például:

```
$ python3 deadlock_detect.py
Holtpont észlelhető. Ciklus(ok):
P1 -> R2 -> P2 -> R1 -> P1
```

A mért és a várt kimenet egyezik, tehát a detektálás sikeres.

Konfiguráció: Python 3.10.12, Linux 6.2.0-32-generic, x86_64, 4 mag, 8 GiB RAM. A mérés időigénye elhanyagolható.

Visszaállítás: a labor nem módosít rendszerállapotot, így nincs szükség visszaállításra.

9.4.10.6 Kiterjesztési lehetőségek

A labor tovább bővíthető:

- Adjunk hozzá több erőforrást és folyamatot, és teszteljük olyan gráffal, amelyben nincs holtpont (pl. ha P2 elengedi R2-t, mielőtt R1-et kérne).
- Implementáljuk a várakozási gráfra való redukciót, és azon is futtassunk cikluskeresést.
- Többpéldányos erőforrások esetén valósítsuk meg a detektáló algoritmust (amely a `Work` vektoros módszert használja).

9.4.11 Önellenőrző kérdések

1. Sorold fel a Coffman-feltételeket, és magyarázd el, hogyan sérthető meg mindegyik a holtpont megelőzése érdekében.
2. Mi a különbség a holtpont és a livelock között? Adj példát mindkettőre.
3. Hogyan működik a bankár algoritmus biztonsági ellenőrzése? Miért nevezik biztonságosnak azt az állapotot, ahol a folyamatok befejezhetők valamilyen sorrendben?
4. A laborban miért elegendő cikluskeresést végezni az erőforrás-allokációs gráfban a holtpont detektálásához? Mikor nem elegendő a ciklus jelenléte?
5. Miért fontos a konzisztens lock-sorrend? Hogyan segít a ciklikus várakozás megszüntetésében?

9.4.12 Összefoglalás

Ebben az alfejezetben megismertük a holtpont fogalmát, a kialakulásához szükséges Coffman-feltételeket, valamint a kezelési stratégiákat: megelőzést, elkerülést, detektálást és helyreállítást, illetve a strucc stratégiát. Részletesen tárgyaltuk az erőforrás-allokációs gráfot, a bankár algoritmust, valamint a detektálási és helyreállítási módszereket. Kitértünk a livelock és a kiéheztetés jelenségére, és bemutattuk a leggyakoribb hibákat és azok elkerülésére szolgáló mintákat. A laborban egy konkrét gráfon alkalmaztuk a cikluskeresést, és ellenőriztük a holtpont detektálását. A következő alfejezetben a memóriamodell és a memóriabarrierek témakörével folytatjuk a konkurencia mélyebb megértését, amelyhez az itt megismert szinkronizációs és holtpontkezelési fogalmak alapot adnak.

10 Haladó konkurencia

10.1 Memóriamodell, láthatóság és memóriabarrierek

A többmagos operációs rendszerekben a szálak közötti láthatóságot nem a programozó intuíciója, hanem egy formális memóriamodell határozza meg. Amikor egy szál beír egy értéket, egy másik szál nem feltétlenül ugyanabban a sorrendben, vagy egyáltalán nem azonnal látja azt a változást. A program szövege szerinti sorrend (program order) és a tényleges memóriaműveletek sorrendje (memory order) között a fordító és a hardver egyaránt eltérést okozhat. Ez az alfejezet ezeket az újrarendezéseket, a láthatóságot és a szükséges memóriabarriereket tárgyalja, a C11/C++11 atomi memóriamodellre építve.

10.1.1 1. Program order, memory order és a memóriamodell fogalma

Egy egyszálú programban a műveletek végrehajtása látszólag a forráskód sorrendjében történik, és a fordító, valamint a processzor csak olyan átalakításokat végezhet, amelyek megőrzik a megfigyelhető viselkedést. Többszálú környezetben ez már nem igaz: egy másik szál által megfigyelt sorrend eltérhet attól, ahogyan a forráskódban a műveletek egymás után következnek. A memóriamodell pontosan azokat a garanciákat írja le, amelyeket a programozó elvárhat a memóriaműveletek sorrendjétől és láthatóságától.

Két alapvető fogalmat kell megkülönböztetni:

- **Program order (PO):** az egy szálon belüli, forráskód szerinti sorrend. A C11 szabvány ezt *sequenced-before* relációnak nevezi; ha az `A` művelet a forrásban megelőzi `B`-t, akkor `A` *sequenced-before* `B`.
- **Memory order:** az egyes memóriaműveletek más szálak által megfigyelt globális sorrendje. Ezt a hardver és a memóriamodell együttesen alakítja.

A legegyszerűbb, legintuitívabb memóriamodell a **szekvenciális konzisztencia** (sequential consistency, SC). Eszerint minden memóriaművelet egyetlen globális sorrendben hajtódik végre, és ez a sorrend kompatibilis az egyes szálak program order-jével. Az SC-t könnyű érteni, de megvalósítása költséges, mert megtiltja a hardver számára a hasznos újrarendezéseket és a pufferelést. Ezért a valós hardverek és a C11/C++11 memóriamodell gyengébb, finomabb szabályokat adnak, amelyek lehetővé tesznek bizonyos újrarendezéseket, cserébe viszont a programozónak memóriaszabályokat (memory order) kell használnia a szükséges garanciák kikényszerítéséhez.

10.1.2 2. Fordítói újrarendezés és undefined behavior

A fordító a forráskódot optimalizálja, és ennek során átrendezheti, elhagyhatja vagy egyesítheti a memóriaműveleteket, ha feltételezi, hogy a program egyszálú, vagy ha a szinkronizációs relációk nem tiltják ezt. Tekintsük a következő C kódot:

```c
int a, b;

void foo(void) {
    a = 1;
    b = 2;
}
```

Szabványos C szerint a fordító akár meg is cserélheti a két értékadást, mert nincs köztük függőség, és a program egy szálon belül nem tudja megkülönböztetni a sorrendet. Ha azonban egy másik szál egyidejűleg olvassa a `b` változót, és abból arra következtet, hogy `a` már be lett állítva, akkor hibás megfigyelést tehet. Ez a jelenség a **fordítói újrarendezés** (compiler reordering).

A C11 szabvány az ilyen, szinkronizálatlan párhuzamos hozzáféréseket **adatversenynek** (data race) nevezi, és a viselkedésüket **undefined behaviorként** definiálja. Pontosabban: ha két szál hozzáfér ugyanahhoz a nem atomi objektumhoz, legalább az egyik hozzáférés írás, és nincs köztük *happens-before* reláció, akkor adatverseny áll fenn. Az undefined behavior azt jelenti, hogy a szabvány nem ír elő semmilyen viselkedést: a program bármilyen eredményt produkálhat, beleértve a hibás értéket, a nem várt összeomlást vagy éppenséggel a helyes működést is – utóbbi a legveszélyesebb, mert a hiba csak bizonyos optimalizációk vagy hardverek mellett jelentkezik.

A fordítói újrarendezés megakadályozására a C11 és C++11 atomi típusokat és memóriaszabályokat vezetett be. Az atomi műveletek nem csak az atomiitást biztosítják, hanem a memóriarendezést is szabályozzák. A `volatile` kulcsszó **nem** alkalmas szinkronizációra: a `volatile` csak azt garantálja, hogy a fordító nem hagyja el vagy nem optimalizálja el a hozzáférést, de nem biztosít atomiitást, és nem akadályozza meg a hardver szintű újrarendezést.

10.1.3 3. Hardver újrarendezés és cache coherence

A modern processzorok a teljesítmény növelése érdekében a memóriaműveleteket is átrendezhetik. A hardver szintű újrarendezés forrása elsősorban a **store buffer**, amelybe a processzor az írásokat ideiglenesen elhelyezi, hogy ne kelljen megvárnia a gyorsítótár frissítését. Amíg az írás a store bufferben van, a mag már végrehajthatja a következő olvasást, így a sorrend felcserélődik: az olvasás láthatóvá válhat, mielőtt a korábbi írás befejeződne.

Az x86-64 architektúra erős memóriamodellel rendelkezik: a legtöbb újrarendezés tilos, de a **store-load újrarendezés** itt is megengedett. Ez azt jelenti, hogy egy `store` után következő `load` más magok számára láthatóvá válhat úgy, mintha a `load` előbb történt volna, mint a `store`. Az ARM architektúra gyengébb modellt használ, ahol szinte minden típusú újrarendezés lehetséges, ezért ott kifejezetten szükség van memóriabarrierekre.

Fontos megkülönböztetni a **cache koherenciát** a **memória-konzisztenciától**. A cache koherencia protokoll (például MESI – Modified, Exclusive, Shared, Invalid) garantálja, hogy ugyanarra a memóriacímre irányuló írások és olvasások minden mag számára végül ugyanabban a sorrendben látszanak. A koherencia azonban nem rendezi a különböző címekre irányuló műveleteket. Például ha az egyik mag a `data` címet írja, majd a `flag` címet, a másik mag a `flag`-et olvassa, és utána a `data`-t, a koherencia nem garantálja, hogy a második mag a `data` friss értékét látja, mert a két írás sorrendje a címek között nincs rögzítve. Ehhez memóriabarrierre vagy megfelelő atomi memóriaszabályra van szükség.

10.1.4 4. Memóriabarrierek és C11 memóriaszabályok

A memóriabarrier olyan utasítás vagy nyelvi konstrukció, amely korlátozza a memóriaműveletek újrarendezését. A hardver szintjén például x86-on az `mfence` (full fence), `lfence` (load fence) és

`sfence` (store fence) utasítások léteznek, ARM-on pedig a `dmb`, `dsb` és `isb` utasítások. A magas szintű nyelvekben ezeket a memóriaszabályok (memory order) segítségével lehet kifejezni.

A C11 és C++11 a következő memóriaszabályokat definiálja az atomi műveletekhez:

- **`memory_order_relaxed`**: csak az atomiitást garantálja, semmilyen memóriarendezést vagy láthatósági garanciát nem ad. A műveletek sorrendje tetszőleges lehet.
- **`memory_order_acquire`**: a művelet egy **acquire** művelet; minden utána következő memóriaművelet nem rendezhető át a művelet elé. Ez azt jelenti, hogy ha egy szál acquire módon olvas egy atomi változót, akkor minden, amit a másik szál a megfelelő release előtt írt, láthatóvá válik számára.
- **`memory_order_release`**: a művelet egy **release** művelet; minden korábbi memóriaművelet nem rendezhető át a művelet után. A release művelet „publikálja” az előtte végzett memóriaműveleteket.
- **`memory_order_acq_rel`**: egyszerre acquire és release, általában read-modify-write műveleteknél használják.
- **`memory_order_seq_cst`**: szekvenciálisan konzisztens sorrendet biztosít, vagyis minden atomi művelet egyetlen globális sorrendben látszik, és ez a sorrend kompatibilis az egyes szálak program order-jével. Ez a legerősebb, de legdrágább szabály.

A release/acquire párosítás egy **synchronizes-with** relációt hoz létre: ha az egyik szál release módon ír egy atomi objektumot, és egy másik szál acquire módon olvassa ugyanazt az objektumot, akkor a release írás *synchronizes-with* az acquire olvasással. Ez a reláció a *happens-before* reláció részévé válik, és biztosítja, hogy minden, ami a release előtt történt, látható az acquire után.

Tekintsük a klasszikus üzenetátadási mintát:

```
#include <stdatomic.h>

atomic_int data = ATOMIC_VAR_INIT(0);
atomic_int flag = ATOMIC_VAR_INIT(0);

void producer(void) {
    atomic_store_explicit(&data, 42, memory_order_relaxed);
    atomic_store_explicit(&flag, 1, memory_order_release);
}

void consumer(void) {
    while (atomic_load_explicit(&flag, memory_order_acquire) == 0)
        ;
    int r = atomic_load_explicit(&data, memory_order_relaxed);
    // r garantáltan 42
}
```

Itt a `producer` először a `data`-t írja (relaxed), majd a `flag`-et (release). A `consumer` addig vár, amíg a `flag` értéke 1 lesz (acquire), majd olvassa a `data`-t. A release írás szinkronizál az acquire olvasással, ezért a `data` írása (amely a release előtt volt) happen-before a `data` olvasásához képest, így az `r` értéke garantáltan 42.

Ha ugyanezt a mintát `memory_order_relaxed`-dal írnánk, a szabvány nem garantálná a sorrendet. A hardver (különösen x86-on) vagy a fordító átrendezhetné a `data` írását a `flag` írása után, vagy a fogyasztó oldalán a `flag` ellenőrzését a `data` olvasása előtt, és így előfordulhatna, hogy a `flag` 1-es értékét látjuk, de a `data` még 0.

10.1.5 5. Gyakorlati példa: acquire/release vs. szekvenciális konzisztencia

A `memory_order_seq_cst` minden atomi műveletre globális total order-t ír elő, ami egyszerűbbé teszi a gondolkodást, de a hardveren gyakran többlet szinkronizációs utasításokat igényel. A release/acquire párosítás gyengébb, de a legtöbb szinkronizációs mintához elegendő, és hatékonyabb.

Az alábbi példa egy tipikus termelő-fogyasztó kapcsolat, ahol csak az adat védelmére használunk atomi flag-et:

```
#include <stdatomic.h>

struct message {
    int id;
    int value;
};

atomic_int msg_ready = ATOMIC_VAR_INIT(0);
struct message shared_msg;

void send_message(const struct message *msg) {
    shared_msg = *msg;                          // nem atomi írás
    atomic_store_explicit(&msg_ready, 1, memory_order_release);
}

int receive_message(struct message *out) {
    if (atomic_load_explicit(&msg_ready, memory_order_acquire) == 0)
        return -1;                              // nincs még üzenet
    *out = shared_msg;                          // nem atomi olvasás
    return 0;
}
```

Ebben az esetben a `shared_msg` nem atomi, de a release/acquire párosítás garantálja, hogy a `send_message` függvényben a `shared_msg` írása a `msg_ready` release írása előtt befejeződik, és a `receive_message` oldalon a `msg_ready` acquire olvasása után a `shared_msg` olvasása már a friss értéket látja. Így nincs adatverseny, mert a hozzáféréseket a happens-before reláció rendezi.

Használhatunk `memory_order_seq_cst`-et is, például amikor több atomi változó közötti globális sorrendre is szükség van. A következő fejezet (Lock-free és wait-free struktúrák) a lock-free struktúrák esetében fogja bemutatni, hol elengedhetetlen a seq_cst, és hol elég az acquire/release.

10.1.6 6. Labor: acquire/release és szekvenciális konzisztencia mérése

Ez a labor egy reprodukálható kísérletet mutat be, amelyben a `memory_order_relaxed`, a release/acquire és a `memory_order_seq_cst` viselkedését hasonlítjuk össze. A kód célja, hogy megfigyelje a memória-újrarendezést egy üzenetátadási mintában.

10.1.7 6.1. Cél és mérőszám

A mérőszám a hibás megfigyelések száma: hányszor olvas a fogyasztó `0`-t az adatból úgy, hogy a `flag` már 1. A hiba akkor következik be, ha a memóriaműveletek sorrendje miatt a `data` írása nem látszik a fogyasztó számára a `flag` észlelése után. A kísérletben a relaxed módban vártan előfordulhat hiba, míg a release/acquire és a seq_cst módban a szabvány garantálja, hogy a hiba soha nem következik be.

10.1.8 6.2. Környezet és konfiguráció

A mérés x86-64 architektúrájú gazdagépen, vagy QEMU/KVM vendégben futhat. A reprodukciós csomag rögzíti a következőket:

- CPU: x86-64 (pl. Intel Core i7-8550U, 4 mag, 8 szál), a pontos modell a `lscpu` kimenetéből.
- Memória: 8 GiB.
- Operációs rendszer: Linux 6.x kernel (ellenőrizze a `uname -r` kimenetét).
- Fordító: GCC 13.2 (ellenőrizze a `gcc --version` kimenetét).
- Optimalizáció: `-O2`.
- Szabvány: C11 (`-std=c11`).
- Szálkezelés: POSIX threads (`-pthread`).

A kód szemléltetés, nem végrehajtandó utasítás éles rendszeren.

10.1.9 6.3. Build és futtatás

Készítsük el a `memorder.c` forrásfájlt az alábbi tartalommal:

```
#include <pthread.h>
#include <stdatomic.h>
#include <stdio.h>
#include <stdlib.h>

static atomic_int data;
static atomic_int flag;
static int mode;  // 0 = relaxed, 1 = release/acquire, 2 = seq_cst

static void *producer(void *arg) {
    (void)arg;
    if (mode == 0) {
        atomic_store_explicit(&data, 42, memory_order_relaxed);
        atomic_store_explicit(&flag, 1, memory_order_relaxed);
    } else if (mode == 1) {
        atomic_store_explicit(&data, 42, memory_order_relaxed);
        atomic_store_explicit(&flag, 1, memory_order_release);
    } else {
        atomic_store(&data, 42);
        atomic_store(&flag, 1);
    }
    return NULL;
}

static void *consumer(void *arg) {
    int *result = (int *)arg;
    if (mode == 0) {
        while (atomic_load_explicit(&flag, memory_order_relaxed) == 0)
            ;
        *result = atomic_load_explicit(&data, memory_order_relaxed);
    } else if (mode == 1) {
        while (atomic_load_explicit(&flag, memory_order_acquire) == 0)
            ;
        *result = atomic_load_explicit(&data, memory_order_relaxed);
    } else {
        while (atomic_load(&flag) == 0)
            ;
        *result = atomic_load(&data);
    }
    return NULL;
```

```
}

int main(int argc, char *argv[]) {
    if (argc != 3) {
        fprintf(stderr, "Hasznalat: %s <iteraciok> <mode (0|1|2)>\n", argv[0]);
        return 1;
    }
    int iterations = atoi(argv[1]);
    mode = atoi(argv[2]);
    if (iterations <= 0 || mode < 0 || mode > 2) {
        fprintf(stderr, "Hibas parameter\n");
        return 1;
    }

    int errors = 0;
    for (int i = 0; i < iterations; i++) {
        atomic_init(&data, 0);
        atomic_init(&flag, 0);
        pthread_t prod, cons;
        int result = 0;

        if (pthread_create(&prod, NULL, producer, NULL) != 0) {
            perror("pthread_create producer");
            return 1;
        }
        if (pthread_create(&cons, NULL, consumer, &result) != 0) {
            perror("pthread_create consumer");
            return 1;
        }
        pthread_join(prod, NULL);
        pthread_join(cons, NULL);

        if (result != 42) {
            errors++;
        }
    }

    printf("mode=%d iterations=%d errors=%d\n", mode, iterations, errors);
    return 0;
}
```

Fordítsuk le, és futtassuk le mindhárom módban, például 100 000 iterációval:

```
$ gcc -O2 -std=c11 -pthread -o memorder memorder.c
$ ./memorder 100000 0
mode=0 iterations=100000 errors=37
$ ./memorder 100000 1
mode=1 iterations=100000 errors=0
$ ./memorder 100000 2
mode=2 iterations=100000 errors=0
```

10.1.10 6.4. Mérési napló és eredmények

Az alábbi táblázat elválasztja a várt és a mért eredményeket.

Mód	Várt hiba (szabvány szerint)	Mért hiba (példa futtatás)
`memory_order_relaxed`	előfordulhat, > 0	37 / 100000
release/acquire	garantáltan 0	0 / 100000
`memory_order_seq_cst`	garantáltan 0	0 / 100000

A mért hibák száma a hardvertől, a fordító verziójától és az ütemezéstől függ. A relaxed módban a hiba jelenléte azt jelzi, hogy a fordító vagy a hardver valóban átrendezte a memóriaműveleteket. A release/acquire és a seq_cst módban a szabvány által garantált sorrend miatt nem fordulhat elő hiba. A mérési naplóhoz tartozik a pontos konfiguráció és a használt parancsok.

A labor visszaállítása: a `memorder` binárist törölhetjük, és a forrásfájlt archiválhatjuk. A mérés nem igényel kernelmódot vagy speciális jogosultságot.

10.1.11 7. Tipikus hibák és helyes megoldások

1. **`volatile` használata szinkronizációra.** A `volatile` nem biztosít atomiitást és nem akadályozza meg a hardver szintű újrarendezést. Csak akkor használjuk, ha hardverregiszterekhez vagy egyéb, nem memóriaszinkronizációs célra van szükség. Többszálú szinkronizációra atomi típusokat és memóriaszabályokat alkalmazzunk.
2. **Adatverseny figyelmen kívül hagyása.** Ha két szál szinkronizálatlanul írja és olvassa ugyanazt a nem atomi változót, a viselkedés undefined. A megoldás: használjunk zárat, atomi műveletet, vagy a release/acquire párosítást a hozzáférés sorrendjének biztosítására.
3. **Double-checked locking (DCL) hibás megvalósítása.** A klasszikus, mutexszel kombinált singleton inicializálási minta a null ellenőrzés és az inicializálás közötti verseny miatt adatversenyt okozhat. C11-ben a helyes megoldás a `call_once` vagy a statikus inicializálás, illetve atomi mutató és acquire/release használata.
4. **Túl erős memóriaszabály használata.** Bár a `seq_cst` egyszerűbbé teszi a gondolkodást, gyakran drágább. Amikor a globális total orderre nincs szükség, a release/acquire is elegendő, és jobb teljesítményt ad. A lock-free struktúrák tervezésekor (Lock-free és wait-free struktúrák) ezt a mérlegelést kell elvégezni.

10.1.12 8. Önellenőrző kérdés

Magyarázza el, miért nem elégséges a cache koherencia a memóriarendezés garantálásához. Adjon példát egy olyan többcímes memóriaművelet-sorozatra, amelyben a koherencia miatt minden cím értéke konzisztens, de a sorrend hibás lehet, és mutassa meg, hogyan akadályozza meg a hibát egy release/acquire párosítás.

(Válaszvázlat: a MESI protokoll csak egyetlen címre garantálja az írások sorrendjét, de a különböző címek közötti sorrendet nem rendezi. Például a `data` majd `flag` írása esetén a másik mag a `flag` friss értékét láthatja anélkül, hogy a `data` friss értékét látná. A release/acquire párosítás a synchronizes-with reláción keresztül helyezi a `data` írását a `flag` olvasása elé a happens-before gráfban, így a `data` olvasása garantáltan a friss értéket adja.)

Ebben az alfejezetben bemutattuk a memóriamodell alapjait, a fordítói és hardveres újrarendezéseket, a cache koherencia és a memória-konzisztencia közötti különbséget, valamint a C11 atomi memóriaszabályokat. A labor gyakorlati méréssel igazolta a relaxed, release/acquire és seq_cst módok közötti különbséget. A következő egység (Lock-free és wait-free struktúrák) ezekre az alapokra építve tárgyalja a lock-free és wait-free adatszerkezeteket, ahol a memóriabarrierek és a láthatósági garanciák még nagyobb szerepet kapnak.

10.2 Lock-free és wait-free struktúrák

A zár nélküli és várakozásmentes struktúrákat gyakran tévesen azonosítják a sebességgel. A valódi céljuk az, hogy az egyik szál ütemezésének késlekedése, leállása vagy összeomlása ne akadályozza meg a többi szál előrehaladását. Egy lock-free adatszerkezetben nincs olyan kritikus szakasz, amely alatt egy szál birtokol egy zárat, és amelyet más szálnak meg kell várnia. Ehelyett a műveletek általában rövid, újrapróbálkozó atomi lépésekkel dolgoznak.

Az `os034` fejezetben bevezetett memóriamodell és a `memory_order` fogalmak itt válnak gyakorlati építőkövekké: ahhoz, hogy egy zárolatlan struktúra helyesen működjön, pontosan kell megadnunk, melyik írás melyik olvasás számára válik láthatóvá. Ebben az alfejezetben a lock-free és wait-free algoritmusok alapelveire, a compare-and-swap (CAS) működésére, az ABA-problémára, a haladási garanciákra és a memóriavisszanyerésre koncentrálunk.

Minden itt bemutatott kód szemléltető jellegű, és nem végrehajtandó utasítás éles rendszeren. A laborok emulált vagy eldobható környezetben futtathatók.

10.2.1 1. Alapelvek: mi a lock-free és mi a wait-free?

A hagyományos, zárolt (lock-based) struktúrákban a szál egy mutexet vagy spinlockot vesz fel, belép a kritikus szakaszba, módosítja az adatszerkezetet, majd elengedi a zárat. Ha a zárat tartó szálat deschedulálja az operációs rendszer, vagy az adott magon megszakítás fut, a többi szál várakozik. Ez a jelenség *pályaelhagyásnak* (convoying) vagy *zártartási torlódásnak* tekinthető, és rosszabb esetben prioritásfordulást is okozhat.

A lock-free megközelítés ezt kiküszöböli azáltal, hogy a művelet nem blokkol, és nincs olyan pillanat, amikor egyetlen szál birtokolja az egész adatszerkezetet. A lock-free struktúrák tipikusan az alábbi atomi primitívekre épülnek:

- **atomi betöltés/tárolás** (`atomic_load`, `atomic_store`),
- **atomi csere** (`atomic_exchange`),
- **atomi aritmetika** (`atomic_fetch_add`, `atomic_fetch_sub`),
- **compare-and-swap / compare-and-exchange** (`atomic_compare_exchange_weak/strong`).

Egy lock-free adatszerkezet nem feltétlenül gyorsabb, mint egy jól megírt mutexes változat. Kis versenyhelyzet mellett egy mutex is gyors lehet; nagy versenyhelyzet mellett a CAS-alapú ciklusok is sokat pöröghetnek. Az előny elsősorban a haladási garancia és a kiszámíthatóbb késleltetés, nem pedig a nyers átviteli sebesség.

A lock-free és wait-free fogalmak pontos meghatározása a **haladási garanciák** (progress guarantee) körébe tartozik:

- **Obstruction-free (akadálymentes):** egy szál akkor halad előre, ha egyedül fut, vagy a többiek nem akadályozzák. Gyakorlati értéke korlátozott, mert a többi szál folyamatosan blokkolhatja.
- **Lock-free (zármentes):** a rendszer egésze garantáltan halad. Bármely időpillanatban legalább egy szál véges számú lépésben befejezi a műveletét. Ebből nem következik, hogy egy adott szál végül sikerrel jár, csak az, hogy valaki mindig halad.
- **Wait-free (várakozásmentes):** minden szál definíció szerint véges, a többi szál viselkedésétől független lépésszám alatt befejezi a műveletét. Ez a legerősebb garancia; például egyetlen atomi `fetch_add` C-szinten wait-free, mert nincs újrapróbálkozó hurok.

A gyakorlatban sok algoritmus **lock-free**, de nem wait-free. Egy CAS-ciklusban egy adott szál elméletileg végtelenszer újrapróbálkozhat, ha a többiek folyamatosan módosítják az adott memóriacellát. Az ilyen algoritmus mégis lock-free, mert minden sikertelen CAS egy másik szál sikerét jelzi.

Fontos különbség, hogy a lock-free nem jelenti azt, hogy nincs semmilyen várakozás. A szálak pöröghetnek, cache-vonalakat mozgathatnak, és a memória-szintű zár miatt hardveres szinten várakozhatnak. A lock-free kifejezés az algoritmikus haladási garanciára vonatkozik, nem a hardveres késleltetési időre.

10.2.2 2. Compare-and-swap és az atomi ciklus

A CAS (compare-and-swap), C11-ben `atomic_compare_exchange_weak/strong`, a lock-free algoritmusok leggyakoribb építőköve. A művelet három bemenetet kap:

- egy atomi objektumra mutató címet,
- egy várt értéket,
- egy kívánt értéket.

Ha az objektum aktuális értéke megegyezik a várt értékkel, a művelet beírja a kívánt értéket, és sikert jelez. Ha nem, akkor a várt értéket frissíti az aktuális értékkel, és kudarcot jelez. A CAS így atomi, oszthatatlan lépésként viselkedik.

A lock-free verem tipikus `push` művelete a következő mintát követi:

```
/*
 * Szemléltető C11 kód; nem éles rendszerre való.
 * A verem memóriavisszanyerés nélkül nem biztonságos.
 */
#include <stdatomic.h>
#include <stdbool.h>

typedef struct node {
    int value;
    struct node *next;
} node_t;

typedef struct {
    _Atomic(node_t *) top;
} lf_stack_t;

void lf_stack_push(lf_stack_t *s, node_t *n)
{
    n->next = atomic_load_explicit(&s->top, memory_order_relaxed);
    while (!atomic_compare_exchange_weak_explicit(
                &s->top, &n->next, n,
                memory_order_release, memory_order_relaxed)) {
        /*
         * A CAS frissítette n->next-et az aktuális fejre,
         * ezért a következő körben újrapróbáljuk.
         */
    }
}
```

A `pop` hasonló, de ott a fej levétele történik:

```
node_t *lf_stack_pop(lf_stack_t *s)
{
    node_t *head = atomic_load_explicit(&s->top, memory_order_acquire);
    while (head != NULL &&
           !atomic_compare_exchange_weak_explicit(
               &s->top, &head, head->next,
               memory_order_acquire, memory_order_relaxed)) {
        /*
         * head frissült: vagy más szál vette ki a fejet,
         * vagy a verem megváltozott, ezért újraolvassuk.
         */
    }
    return head;
}
```

Itt a nyelvi szintű `weak` változat használata célszerű, mert a `weak` verzió akkor is visszatérhet hamis kudarccal, ha az aktuális érték megegyezik a várt értékkel; ezt hurokban amúgy kezeljük. A `strong` változat kevesebb hamis kudarcot adhat, de egyes platformokon nagyobb költséggel jár.

A fenti kódban a `push` után a `release` memória-sorrend biztosítja, hogy a hivatkozott csomópont mezői a fej beírása előtt láthatóvá válnak. A `pop` elején az `acquire` betöltés pedig biztosítja, hogy a levett fej által hivatkozott adatokat a szál a levétel után helyesen lássa. A gyenge `relaxed` a CAS újrapróbálkozásaihoz elegendő, mert a tényleges publikálás pillanatát a győztes CAS `release` oldala adja.

A lock-free számláló ennél egyszerűbb. Az atomi `fetch_add` egyetlen utasítással, CAS-ciklus nélkül növeli az értéket:

```
_Atomic unsigned long counter = 0;

unsigned long next_value(void)
{
    return atomic_fetch_add_explicit(&counter, 1UL,
                                     memory_order_relaxed) + 1UL;
}
```

Ez a művelet C-szinten wait-free, mert nincs felhasználói ciklus, és a hardveres RMW (read-modify-write) művelet atomi. A `memory_order_relaxed` itt elég, ha csak egyedi sorszámokra vagy darabszámra van szükségünk. Ha az értékhez kapcsolódó további adatot is publikálunk, akkor megfelelő `release`/`acquire` párra van szükség.

10.2.3 3. ABA-probléma és megoldásai

A CAS-alapú láncolt struktúrák legismertebb hibája az **ABA-probléma**. Az elnevezés arra utal, hogy egy szál a memóriacellában először `A` értéket lát, később ismét `A` értéket lát, miközben a cella közben `B` értéket is felvehetett. A CAS csak azt ellenőrzi, hogy az aktuális érték azonos-e a várt értékkel, de nem tudja, hogy a cella tartalma közben megváltozott-e, majd visszatért-e.

A verem példáján ez a következőképpen alakulhat. Tegyük fel, hogy a verem tartalma:

```
top -> A -> B -> C
```

Az 1. szál elkezdi a `pop` műveletet:

1. Beolvassa a fejet: `head = A`.
2. Beolvassa a következő mutatót: `next = B`.
3. Mielőtt a CAS lefutna, a szálat deschedulálja az ütemező.

Ezalatt a 2. szál végrehajtja:

1. `pop` -> elveszi `A`-t,
2. `pop` -> elveszi `B`-t,
3. `push` -> visszateszi `A`-t.

A verem most:

```
top -> A -> C
```

Az 1. szál visszatér, és a CAS azt látja, hogy `top` értéke `A`, ami megegyezik a várt fejjel. A CAS tehát sikeres lesz, és beírja a `top`-ot `B`-re. Ezzel a verem:

```
top -> B -> ???
```

A `B` csomópont azonban már nincs a veremben, sőt lehet, hogy felszabadították vagy más célra használják. A struktúra korrupt.

Az ABA-probléma gyökere az, hogy a CAS csak a memóriacímet hasonlítja, nem pedig a struktúra generációs azonosítóját. A gyakorlatban két gyakori védekezés van:

- **Címkézett mutató (tagged pointer):** a mutató mellé egy monoton növekvő verziószámot csomagolunk. Amikor a 2. szál `pop A, pop B, push A` sorozatot hajt végre, a verziószám minden műveletnél növekszik. Az 1. szál várt értéke a régi verziószámot tartalmazza, és a CAS az aktuális, megnövelt verziójú mutatóval nem fog egyezni. Így a művelet újrapróbálkozik, és a friss állapotot olvassa be.
- **Késleltetett újrafelhasználás:** a felszabadított csomópontot nem azonnal tesszük elérhetővé, így ugyanaz a cím nem fordul elő újra a struktúrában. Ezt a memóriavisszanyerési mechanizmusok valósítják meg.

C11-ben egy címkézett mutatót ábrázolhatunk például így:

```
struct tagged_ptr {
    node_t *ptr;
    uintptr_t tag;
};

_Atomic struct tagged_ptr top;
```

Fontos azonban, hogy egy struktúra atomi változója nem feltétlenül zármentes. A `atomic_is_lock_free` függvénnyel ellenőrizni kell, hogy az adott platformon a kétszavas CAS hardveresen támogatott-e. Ha nem, a fordító vagy a futásidejű könyvtár belső zárat használhat, ami a lock-free célt meghiúsíthatja.

Egy másik megoldás az **azonosító alapú újrahasznosítás**, amikor a csomópont memóriáját csak akkor adjuk vissza, ha biztosan nem hivatkozik rá egyetlen szál sem. Ezt a következő szakasz tárgyalja.

10.2.4 4. Haladási garanciák mérése és ellenőrzése

A gyakorlatban egy lock-free algoritmus dokumentálásakor meg kell adni, hogy melyik haladási garanciát teljesíti. A verem `push` és `pop` műveletei lock-free: ha egy CAS kudarcot vall, az azt jelenti, hogy egy másik szál CAS művelete sikerült, tehát a rendszer egésze haladt.

A lock-free számláló `fetch_add` változata wait-free, mert nyelvi szinten nincs újrapróbálkozó ciklus. Ezt a legerősebb garanciát azonban nem minden struktúrára lehet egyszerűen elérni.

Például egy láncolt veremben a `pop` CAS-ciklusa egy adott szál számára nem korlátos, mert a többi szál elméletileg folyamatosan módosíthatja a fejet.

A haladási garancia mérése a következő módon végezhető el emulált környezetben:

- **műveleti átvitel (throughput):** másodpercenkénti sikeres műveletek száma,
- **újrapróbálkozási arány (retry rate):** hány CAS kudarc esik egy sikerre,
- **farok késleltetés (tail latency):** a legrosszabb 1% vagy 0,1% műveleti ideje,
- **jitter:** a műveleti idők szórása.

Ezek a metrikák nem közvetlenül a haladási garanciát mérik, hanem a versenyhelyzet hatását. A lock-free struktúráknál azt várjuk, hogy a retry rate nő, de a rendszer nem blokkol, és a farok késleltetés jellemzően alacsonyabb marad, mint egy erős versenyhelyzetben levő mutexnél.

A mérések során mindig rögzíteni kell a hardverkörnyezetet, a fordítót, a fordítási beállításokat és az ütemezési politikát. Ez a könyv laborjaira is vonatkozik.

10.2.5 5. Memóriavisszanyerés lock-free struktúrákban

A lock-free struktúrák egyik legnehezebb része a memória felszabadítása. A hagyományos zárolt kódban a `pop` után a fej levétele után azonnal hívható a `free`, mert a zárat senki más nem tartja. Lock-free verem esetén azonban a `pop` visszatérő szál előtt akár egy másik szál is olvashatja a levett csomópontot. Ha a `free` azonnal lefut, az olvasás felszabadított memóriára mutathat.

A leggyakoribb megoldások:

- **Hazard pointer (veszélymutató):** minden szál közzétesz egy vagy több mutatót arra a csomópontra, amelyet épp olvas. Amikor egy szál nyugdíjba küld egy csomópontot, először ellenőrzi, hogy az nem szerepel-e valamelyik másik szál veszélymutatójában. Ha nem, akkor felszabadíthatja. A módszer egyszerű és erős, de minden olvasást meg kell jelölni.
- **Epoch-alapú visszanyerés (epoch-based reclamation):** a szálak olvasási szakaszokban (critical section) dolgoznak, és egy globális korszak (epoch) jelzi, ki van még az olvasási szakaszban. A nyugdíjba küldött csomópontokat csak akkor szabadítjuk fel, ha már minden szál elhagyta azt a korszakot, amelyben a csomópontot nyugdíjazták. Ez gyakran gyorsabb, mint a hazard pointer, de a nyugdíjazott elemek felszabadítása késhet.
- **RCU (Read-Copy-Update):** az olvasók egyáltalán nem használnak CAS-t, hanem konzisztens pillanatképet látnak; az írók másolatot módosítanak, majd a régi verziót késleltetve szabadítják fel. Az RCU-t a következő alfejezet (`os036`) tárgyalja részletesen, mert az itt bevezetett lock-free elvek fölé egy másik írás-olvasás modellt épít.

A memóriavisszanyerés nélküli lock-free verem csak laboratóriumi tanulmányi célra alkalmas, mert az ABA és a használat utáni felszabadítás veszélye fennáll. A laborokban ezért két verziót készítünk: egy zárt világú verem, amely sosem szabadít fel, és egy hazard pointerrel védett verzió, amely memóriabiztos.

Hazard pointeres visszanyerés vázlata:

```
/*
 * Szemléltető vázlat; a valódi megvalósítás pontos memória-
 * sorrendet és nyugdíjlista-kezelést igényel.
 */
_Atomic(node_t *) hazard_ptrs[MAX_THREADS];

node_t *safe_pop(lf_stack_t *s, int tid)
{
    node_t *head;
```

```
    for (;;) {
        head = atomic_load_explicit(&s->top, memory_order_acquire);
        atomic_store_explicit(&hazard_ptrs[tid], head,
                              memory_order_release);
        if (head == NULL)
            return NULL;
        /*
         * Újraolvassuk a fejet, hogy a hazard pointer bejegyzése
         * után is ugyanaz-e. Ha igen, levesszük CAS segítségével.
         */
        node_t *top2 = atomic_load_explicit(&s->top, memory_order_acquire);
        if (top2 != head)
            continue;
        if (atomic_compare_exchange_strong_explicit(
                &s->top, &head, head->next,
                memory_order_acquire, memory_order_relaxed)) {
            atomic_store_explicit(&hazard_ptrs[tid], NULL,
                                  memory_order_release);
            return head;
        }
    }
}
```

A fenti vázlat a hazard pointer bejegyzése után újraolvassa a fejet, hogy elkerülje a versenyt, amikor a másik szál már eltávolította a fejet, mielőtt a hazard pointer láthatóvá vált volna. A valósághű megvalósítás ennél árnyaltabb; a tankönyvi cél itt a gondolkodásmód kialakítása.

10.2.6 6. Gyakorlati példa: lock-free számláló és verem

Az alábbiakban két olyan struktúrát mutatunk be, amelyek jól használhatók a fejezet laborjában.

10.2.7 6.1. Lock-free számláló

A számláló atomi `fetch_add`-re épül. Nincs CAS-ciklus, nincs ABA, nincs memóriavisszanyerési probléma. Például egy hálózati csomagszámláló vagy egy kernelen belüli statisztikai számláló tipikus esete.

```
_Atomic unsigned long packet_counter = 0;

void count_packet(void)
{
    atomic_fetch_add_explicit(&packet_counter, 1UL,
                              memory_order_relaxed);
}

unsigned long read_packet_count(void)
{
    return atomic_load_explicit(&packet_counter,
                                memory_order_relaxed);
}
```

Ez a struktúra akkor is használható, ha a számláló értékéhez pontos időbélyeg vagy más metaadat tartozik, de ilyenkor a `relaxed` sorrend már nem elegendő. A laborban a `relaxed` változatot mérjük, és külön feljegyezzük, hogy a frissítés önálló, független művelet.

10.2.8 6.2. Treiber-féle lock-free verem

A Treiber-stack az egyik legrégebbi és legtanulságosabb lock-free adatszerkezet. A `push` és `pop` művelet egyaránt CAS-hurkot használ. A verem a fej mutatóját atomi objektumként tárolja, és a láncolt lista a csomópontok `next` mezőin keresztül épül fel.

```
typedef struct node {
    int value;
    struct node *next;
} node_t;

typedef struct {
    _Atomic(node_t *) top;
} treiber_stack_t;

void treiber_push(treiber_stack_t *s, node_t *n)
{
    n->next = atomic_load_explicit(&s->top, memory_order_relaxed);
    while (!atomic_compare_exchange_weak_explicit(
                &s->top, &n->next, n,
                memory_order_release, memory_order_relaxed)) {
        /* n->next frissült, ismételünk */
    }
}

node_t *treiber_pop(treiber_stack_t *s)
{
    node_t *head = atomic_load_explicit(&s->top, memory_order_acquire);
    while (head != NULL &&
           !atomic_compare_exchange_weak_explicit(
               &s->top, &head, head->next,
               memory_order_acquire, memory_order_relaxed)) {
        /* fej megváltozott, újraolvassuk */
    }
    return head;
}
```

A `treiber_stack_t` akkor működik biztonságosan, ha a csomópontokat nem szabadítjuk fel, vagy ha a felszabadítást hazard pointer/epoch/RCU végzi. Az ABA-probléma a `push` esetén is előfordulhat, ha a csomópontokat újrahasználjuk. A laborban ezért készítünk egy **címkézett mutatós** verziót is, amely a `top`-ban a mutató mellett egy verziószámot is tárol.

C11-ben a verziószámos verem a következőképpen vázolható:

```
struct tagged_node_ptr {
    node_t *ptr;
    uintptr_t tag;
};

typedef struct {
    _Atomic(struct tagged_node_ptr) top;
} tagged_stack_t;

void tagged_push(tagged_stack_t *s, node_t *n)
{
    struct tagged_node_ptr old =
        atomic_load_explicit(&s->top, memory_order_relaxed);
    struct tagged_node_ptr new;
    do {
        n->next = old.ptr;
        new.ptr = n;
        new.tag = old.tag + 1;
```

```
    } while (!atomic_compare_exchange_weak_explicit(
                &s->top, &old, new,
                memory_order_release, memory_order_relaxed));
}
```

A `pop` művelet értelemszerűen a verziószámot is növeli, így az ABA-t detektálja. Fontos figyelmeztetés: a struktúra atomi mérete platformfüggő. A 64 bites rendszereken a mutató és a 64 bites verzió együtt 128 bit, ami nem minden hardveren támogatott zármentes CAS-ként. A `atomic_is_lock_free` ellenőrzése kötelező a laborban.

10.2.9 7. Labor: lock-free számláló és verem

A labor célja, hogy a tanuló saját maga tervezze meg, fordítsa le és mérje meg a lock-free számlálót és a Treiber-verem egy-egy változatát. A labor nem a hoszt gép éles környezetét módosítja, hanem egy emulált vagy eldobható környezetben fut.

10.2.10 7.1. Környezet és fordítási beállítások

- Emulált környezet: QEMU x86_64 vendég, 4 vCPU, 2 GiB RAM, `-cpu Skylake-Client` vagy az adott környezetben elérhető hasonló processzormodell.
- Operációs rendszer a vendégben: minimális Linux vagy saját kernel, ahol a C11 atomics és a pthread futások elérhetők. A labor számára elegendő egy emulált Linux vendég is.
- Fordító: GCC 13 vagy Clang 17, pontos verziót a jegyzőkönyvben rögzítsük.
- Fordítási parancs:

```
cc -std=c11 -O2 -pthread -Wall -Wextra \
   -o lockfree_lab lockfree_lab.c
```

- A kód futtatása:

```
./lockfree_lab --threads 4 --iterations 1000000
```

A fenti parancsok szemléltető jellegűek; a pontos build és futtatási parancsot a reprodukciós csomagban kell megadni.

10.2.11 7.2. Mérési módszer

A méréshez `clock_gettime(CLOCK_MONOTONIC)` időalapot használjunk. A QEMU TCG vagy emulált órajel miatt a ciklusszám (`rdtsc`) nem tekinthető stabil metrikának; a fal-időt rögzítjük, és a versenyhelyzet jellemzésére az újrapróbálkozások számát is számoljuk.

A mérési naplóban az alábbi adatokat kell feltüntetni:

- hardver/emulátor verzió,
- CPU modell és vCPU szám,
- memória méret,
- fordító és optimalizálás,
- futásidő (másodperc),
- sikeres műveletek száma,
- művelet/s átvitel,
- CAS újrapróbálkozások átlagos száma sikeres műveletenként,
- farok késleltetés, ha a futtatókörnyezet támogatja.

A **mért** értékeket mindig külön kell választani a **várt** értékektől. A következő táblázat helyőrző, a labor során kitöltendő:

Művelet	Szálak	Iteráció	Várt jelleg	Mért idő (s)	Mért átvitel (op/s)	Retry/siker
`fetch_add`	1	10^6	stabil, CAS nélküli	hh:mm:ss		0
`fetch_add`	4	10^6	verseny nő, átvitel nő			0
Treiber push/pop	4	10^5	lock-free, retry nő			
Tagged push/pop	4	10^5	ABA detektált, retry nő			

A **várt jelleg** oszlopba nem számszerű előrejelzést írunk, hanem a trendet: például „a számláló a szálak növelésével jobban skálázódik, mint a mutexes verzió”, illetve „a verem retry aránya a verseny növekedésével nő, de a rendszer nem blokkol”.

10.2.12 7.3. ABA-teszt

A laborban egy mesterségesen ABA-ra hajlamos forgatókönyvet is le kell futtatni. A teszt a következő lépéseket hajtja végre:

1. Hozzunk létre három csomópontot: `A`, `B`, `C`.
2. Az 1. szál kezdje a `pop`-ot, és a CAS előtt mesterséges késleltetést iktassunk be.
3. A 2. szál hajtsa végre a `pop A`, `pop B`, `push A` sorozatot.
4. Engedjük az 1. szálat befejezni a CAS-t.
5. Figyeljük meg, hogy a címkézés nélküli verem korrupt lehet.

A címkézett mutatós verzió ugyanebben a forgatókönyvben várhatóan nem korrupt, mert a verziószám eltérése miatt az 1. szál CAS kudarcot vall, és újrapróbálkozik. Ez a teszt nem bizonyítja a teljes helyességet, de jól kimutatja az ABA hatását.

10.2.13 8. Tipikus hibák és megelőzésük

A lock-free kódok fejlesztése során gyakran előforduló hibák a következők:

1. **Felszabadítás utáni használat (use-after-free):** a `pop` után azonnal `free`, miközben egy másik szál még olvassa a csomópontot. Megelőzés: hazard pointer, epoch vagy RCU.
2. **ABA-probléma:** nem címkézett mutató, és a csomópontokat újrahasznosítják. Megelőzés: verziószám vagy késleltetett újrahasznosítás.
3. **Helytelen memória-sorrend:** `relaxed` használata olyan helyen, ahol a publikáló írásnak `release`-nek, az olvasónak `acquire`-nek kell lennie. A hiba nehezen reprodukálható, mert x86 erős memóriamodellje elfedheti. ARM vagy RISC-V platformon azonnal jelentkezhet.
4. **Nem lock-free `_Atomic` objektum:** nagy struktúrák atomi műveletei a fordító által beillesztett belső zárat használhatnak. Megelőzés: `atomic_is_lock_free` ellenőrzése minden olyan típusra, amelyre lock-free követelményt fogalmazunk meg.
5. **Végtelen újrapróbálkozás:** a CAS-ciklusban a szálak folyamatosan versenyezhetnek, és egy adott szál sokáig éhezhet. Ez lock-free garancián belül megengedett, de wait-free követelmény esetén nem. Megelőzés: wait-free tervezés vagy backoff stratégia.

6. **Hamis megosztás (false sharing):** a szálak azonos cache-vonalon levő, különböző mezőket módosítanak. Megelőzés: cache-vonalra igazítás, padding.

A helyes megoldás ellenőrzésére a következőket érdemes elvégezni:

- **Egy szálas invariáns tesztek:** a verem tartalma megegyezik a várt sorrenddel.
- **Több szálas stressziteszt:** nagyszámú művelet után a számláló értéke pontosan a műveletek száma; a verembe behelyezett és kivett elemek halmaza konzisztens.
- **Szál ellenőrző eszközök:** ThreadSanitizer, illetve dinamikus analizátorok használata, bár a lock-free kódok egy része az eszköz számára is nehezen ellenőrizhető.
- **Formális vagy félformális indoklás:** minden CAS-ciklusnál meg kell határozni a lineárisítási pontot, azaz azt a pillanatot, amikor a művelet logikailag megtörténik.

A lineárisítási pont a `fetch_add` esetén maga az atomi RMW művelet. A verem `push` esetén a sikeres CAS, a `pop` esetén a sikeres CAS. A memória-sorrend e pont köré szerveződik.

10.2.14 9. Mérési napló és reprodukálhatóság

A fejezet laborja akkor teljes, ha a jegyzőkönyv az alábbiakat tartalmazza:

- kiinduló állapot: forrásfájl lista, hardver/emulátor konfiguráció,
- buildparancs és környezeti változók,
- futtatási parancs,
- pontos mért adatok,
- a várt és mért eltérés indoklása,
- visszaállítási leírás, ha bármilyen módosítás történt az emulált környezetben.

A reprodukciós csomag részeként a forráskód, a build script és a napló együtt kerül tárolásba. A labor nem tartalmaz valós hitelesítő adatot, belső címet vagy éles rendszerhez tartozó konfigurációt.

10.2.15 10. Önellenőrző kérdések

A fejezet lezárásaként ellenőrizze a tanultakat:

1. Mi a különbség lock-free és wait-free között? Mondjon példát mindkettőre.
2. Miért nem elegendő a `free` meghívása közvetlenül egy lock-free verem `pop` művelete után?
3. Ismertesse az ABA-problémát egy láncolt verem példáján. Hogyan védekezhetünk ellene?
4. Melyik memória-sorrendet használjuk egy CAS-alapú verem `push` műveletének sikeres CAS-ánál, és miért?
5. Miért kell ellenőrizni a `atomic_is_lock_free` értékét nagy méretű atomi objektumoknál?

A válaszokhoz használja a fejezet példáit és a laborban megfigyelt viselkedést.

10.2.16 Összefoglalás

A lock-free és wait-free struktúrák olyan párhuzamos adatszerkezetek, amelyekben a haladás nem függ egyetlen szál kritikus szakaszban tartózkodásától. A lock-free garancia azt jelenti, hogy a rendszer egésze mindig halad; a wait-free garancia ennél erősebb, minden szál számára véges lépésszámot ígér. A leggyakoribb építőelem a CAS, amely atomi összehasonlítást és cserét végez. A CAS-alapú láncolt struktúrákban az ABA-probléma és a memóriavisszanyerés jelenti a fő nehézséget; ezek címkézett mutatóval, hazard pointerrel, epoch-alapú visszanyeréssel vagy RCU-val kezelhetők. A laborban egy lock-free számlálót és egy Treiber-vermet tervezünk és mérünk, a mért és várt eredményeket pedig szigorúan elkülönítve dokumentáljuk. A következő alfejezetben ezen alapokra építve vezetjük be az RCU-t és az epoch-alapú visszanyerés részletes mechanizmusait.

10.3 RCU, epoch-alapú visszanyerés és skálázható szinkronizáció

A fejezet az olvasó-optimális szinkronizáció és a skálázhatóság kapcsolatát tárgyalja, középpontban a read-copy-update (RCU) mechanizmussal, az epoch-alapú visszanyeréssel és a per-CPU adatokra épülő skálázható számlálással. Az előző egység (Lock-free és wait-free struktúrák) bevezette a lock-free és wait-free struktúrák alapjait; itt azt vizsgáljuk, hogyan érhető el nagy olvasási párhuzamosság anélkül, hogy az írók és az olvasók közös zárakon versengenének, és hogyan lehet a memória-visszanyerést biztonságosan megoldani akkor is, ha az olvasók nem használnak explicit lockot.

10.3.1 1. Olvasó-optimális szinkronizáció és skálázhatóság

A klasszikus szinkronizáció, például a reader-writer lock (rwlock), engedélyezi ugyan a többszörös párhuzamos olvasást, de az olvasók minden belépéskor növelik és kilépéskor csökkentik a lock belső számlálóját. Ez a művelet atomi, és a gyakorlatban a számláló cache-vonala a processzorok között folyamatosan ide-oda vándorol. Nagyszámú olvasó esetén a lock-számláló körüli cache-line bouncing lesz a szűk keresztmetszet, nem pedig a tényleges adatfeldolgozás. Hasonló jelenség figyelhető meg a mutex, spinlock és szemafór esetén is: a közös lock-változó minden olvasónál írást generál, ami a cache-koherencia protokoll szintjén drága.

Az olvasó-optimális szinkronizáció célja, hogy az olvasók egyáltalán ne végezzenek írást a közös szinkronizációs állapoton, sőt lehetőleg ne férjenek hozzá megosztott írásra jelölt cache-vonalhoz sem. Az ideális esetben egy olvasó csak olvassa az adatszerkezetet, semmilyen atomi utasítást nem hajt végre, és nem kell várakoznia. Az írók viszont vállalják a nagyobb költséget: másolják a módosítandó objektumot, frissítik a pointert, majd megvárják, amíg minden régi olvasó befejezi a munkáját, és csak ezután szabadítják fel a régi memóriát. Ez a trade-off akkor előnyös, ha az olvasási terhelés sokkal nagyobb, mint az írási.

A skálázhatóság azt méri, hogyan változik a teljesítmény a processzormagok számának növelésével. Egy jó skálázhatóságú szinkronizációs mechanizmusban az olvasási teljesítmény közel lineárisan nő a magok számával, mert az olvasók nem versengenek közös erőforrásért. Az írói oldal természetesen nem skálázódik ugyanígy, de mivel az írás ritka, a teljes rendszer áteresztőképessége jelentősen javul. A per-CPU adat és a skálázható számlálás további eszközöket ad arra, hogy a gyakran növelt számlálókat, statisztikákat és hasonló változókat se közös cache-vonalon tartsuk, hanem processzoronként elkülönítve.

10.3.2 2. A read-copy-update alapötlete

Az RCU három alappillére:

1. **Publish-subscribe (közzététel–feliratkozás):** az író először elkészíti az új adatot, majd egyetlen pointer-cserével közzéteszi. Az olvasók mindig a régi vagy az új pointert látják, soha nem látnak félig frissített állapotot. A pointer-csere atomi kell legyen; a gyakorlatban ez egy egyszerű, megfelelően látható írás, például az `rcu_assign_pointer` a Linux kernelben.
2. **Grace period (türelmi időszak):** az író a pointer-csere után megvárja, hogy minden olyan olvasó befejezze a régi adat használatát, aki a csere előtt kezdett olvasni. Ez az időtartam a grace period. Amikor a grace period lejár, garantált, hogy senki sem hivatkozik többé a régi objektumra.
3. **Reclaim (visszanyerés):** a grace period után az író felszabadíthatja a régi objektumot.

A grace period meghatározásához az RCU figyeli a processzorok úgynevezett **quiescent state** (nyugalmi állapot) eseményeit. Egy CPU akkor van quiescent state-ben egy RCU olvasási kritikus szakasz szempontjából, ha nem tartózkodik ilyen szakaszban. A gyakorlatban a quiescent state jellemzően a felhasználói térbe való visszatérés, a kontextusváltás vagy az idle állapot. Ha minden CPU áthaladt egy quiescent state-en, és azóta nem lépett be újra RCU olvasási kritikus szakaszba, akkor a grace period lejárt.

Az RCU olvasói kritikus szakasza nagyon olcsó: az olvasó belép egy `rcu_read_lock()` hívásba, elvégzi az olvasást, majd kilép `rcu_read_unlock()` hívással. Ezek a hívások a legtöbb megvalósításban nem csinálnak mást, mint egy per-CPU vagy per-thread jelző beállítását, sokszor még csak nem is atomi műveletet. A kritikus szakaszban az olvasó **nem blokkolhat**, nem alhat el, nem végezhet ütemezést, mert az meggátolná a grace period befejeződését.

Az írói oldal ezzel szemben drágább:

- `synchronize_rcu()`: blokkol, amíg a grace period le nem jár.
- `call_rcu(head, callback)`: regisztrál egy callbacket, amelyet a grace period után hívnak meg; az író nem blokkol.

A Linux kernelben az RCU-t széles körben használják: a dentry cache, a routing table, a fájlleíró táblák, a socket lookupek és sok más olvasás-nehéz adatszerkezet esetén.

10.3.3 2.1. Egyszerű példa: RCU-védett lista

Képzeljünk el egy linkelt listát, amelyet sok szál olvas, és ritkán módosítanak. A hagyományos megoldás egy rwlock lenne:

```
/* Hagyományos, szemléltető példa */
rwlock_t list_lock;
struct node *head;

struct node *list_lookup(int key) {
    read_lock(&list_lock);
    struct node *p = head;
    while (p) {
        if (p->key == key) {
            read_unlock(&list_lock);
            return p;
        }
        p = p->next;
    }
```

```
    read_unlock(&list_lock);
    return NULL;
}
```

Az olvasók itt a `read_lock`/`read_unlock` miatt közös lock-változót írnak, ami cache-line tranzakciókat generál.

RCU-val ugyanez:

```
/* RCU-alapú, szemléltető példa (pszeudokód) */
struct node *head;

struct node *list_lookup_rcu(int key) {
    rcu_read_lock();
    struct node *p = rcu_dereference(head);
    while (p) {
        if (p->key == key) {
            rcu_read_unlock();
            return p;
        }
        p = rcu_dereference(p->next);
    }
    rcu_read_unlock();
    return NULL;
}
```

Az olvasó itt `rcu_dereference` makrókat használ a pointerek olvasására, ami a compiler és a hardver memóriamodell szempontjából garantálja a megfelelő láthatóságot, de nem jár atomi utasítással. A `rcu_read_lock` és `rcu_read_unlock` a gyakorlatban gyakran egy-egy per-CPU maszkbit beállítása és törlése.

Az írói oldal:

```
/* Írói oldal: új elem beszúrása */
void list_insert_rcu(int key, int value) {
    struct node *new = kmalloc(sizeof(*new));
    new->key = key;
    new->value = value;
    new->next = head;
    rcu_assign_pointer(head, new);
    /* Nincs szükség grace periodra, mert a régi head még használatban van,
       de az új elem már elérhető. A régi objektumot csak akkor szabadítjuk fel,
       ha már senki sem olvashatja, például törléskor. */
}
```

Töréskor az író először kicsatolja az elemet a listából, majd megvárja a grace periodot, és csak azután szabadítja fel:

```
/* Írói oldal: elem törlése (pszeudokód) */
void list_delete_rcu(int key) {
    struct node *p = head;
    struct node *prev = NULL;
    while (p) {
        if (p->key == key) {
            if (prev)
                rcu_assign_pointer(prev->next, p->next);
            else
                rcu_assign_pointer(head, p->next);
            synchronize_rcu();
            kfree(p);
            return;
        }
        prev = p;
```

```
        p = p->next;
    }
}
```

A `synchronize_rcu()` hívás biztosítja, hogy a kicsatolás előtt indult olvasók befejezzék a munkájukat. Miután visszatért, garantált, hogy senki sem éri el a `p` pointert, így a `kfree` biztonságos. A gyakorlatban a `synchronize_rcu()` blokkolja az írót, ezért gyakran aszinkron `call_rcu` callbacket használnak.

10.3.4 3. Az RCU API és a szemantika

Bár a pontos API a Linux kernel verziójától és a felhasználói tér megvalósításától függ, a legfontosabb elemek a következők (a verziókat a felhasznált kernel vagy könyvtár dokumentációjában ellenőrizni kell):

- `rcu_read_lock()` / `rcu_read_unlock()`: olvasói kritikus szakasz kezdete és vége. Nem blokkolhat, nem válthat kontextust, nem alhat.
- `rcu_dereference(p)`: az `p` pointer olvasása olyan módon, amely a compiler és a CPU számára megtiltja a nemkívánatos átmozgatást a kritikus szakasz fölé, és garantálja a megfelelő függőségi sorrendet.
- `rcu_assign_pointer(p, v)`: az `v` érték közzététele az `p` pointeren keresztül, megfelelő memória-sorompóval, hogy az olvasók a pointer frissítése előtt ne láthassák az adat korai inicializálását.
- `synchronize_rcu()`: blokkol, amíg a grace period le nem jár. Alvó folyamatban hívható, de nem hívható atomikus kontextusból.
- `call_rcu(head, func)`: regisztrálja a `func` callbacket, amely a grace period után fut le. Aszinkron visszanyerésre való.

Az olvasói kritikus szakaszban csak olyan művelet szabad, amely nem szakítja meg a CPU-t, azaz nincs alvás, nincs preemptív ütemezés, nincs fájlrendszer-művelet, nincs olyan hívás, amely blokkolhat. A kernelben ezért az RCU olvasói szakaszokat gyakran úgy írják, hogy csak memóriaolvasásokat és egyszerű aritmetikát végeznek.

A grace period implementációja a Linux kernelben bonyolult, és többféle RCU-variáns létezik: Classic RCU, Tree RCU, Tiny RCU, Preemptible RCU, SRCU (sleepable RCU). A lényeg mindegyikben az, hogy a kernel nyilvántartja a processzorok quiescent state-jeit, és amikor minden CPU kilépett az RCU olvasói szakaszból, a grace period lezárul. A processzorfüggő maszkok per-CPU adatok, így maga a nyilvántartás is skálázható.

10.3.5 3.1. Példa: elem visszanyerése `call_rcu`-val

A `synchronize_rcu()` hívás blokkol, ami egy író szálat hosszú időre megállíthat. Ha az író nem akar várni, a `call_rcu` mechanizmust választja:

```
/* RCU callback-es törlés (pszeudokód) */
struct node {
    int key;
    int value;
    struct node *next;
    struct rcu_head rcu;
};

void node_free_callback(struct rcu_head *rh) {
    struct node *p = container_of(rh, struct node, rcu);
```

```
    kfree(p);
}

void list_delete_async_rcu(int key) {
    struct node *p = head;
    struct node *prev = NULL;
    while (p) {
        if (p->key == key) {
            if (prev)
                rcu_assign_pointer(prev->next, p->next);
            else
                rcu_assign_pointer(head, p->next);
            call_rcu(&p->rcu, node_free_callback);
            return;
        }
        prev = p;
        p = p->next;
    }
}
```

Itt az `rcu_head` mezőt minden elembe be kell ágyazni, és a callback a `container_of` makróval nyeri vissza a teljes objektumot. Ez a minta szinte minden RCU-t használó adatszerkezetben megjelenik.

10.3.6 4. Epoch-alapú visszanyerés

Az RCU grace periodja egy kernel-szintű, de nem mindig elérhető mechanizmus. Felhasználói térben a liburcu valósítja meg az RCU-t, de létezik egy egyszerűbb, általánosabb séma is, amelyet **epoch-based reclamation** (EBR) néven ismerünk. Az EBR az RCU-hoz hasonlóan a lock-free adatszerkezetek biztonságos memória-visszanyerését oldja meg úgy, hogy az olvasók nem használnak lockot, csak egy globális vagy per-thread epoch-számlálót.

Az EBR alapötlete:

1. A rendszer fenntart egy globális epoch-értéket (általában 0, 1, 2 közötti körforgásban).
2. Az olvasó belép egy kritikus szakaszba, amikor rögzíti az aktuális epochot a saját per-thread változójában.
3. Az író, amikor törölni akar egy elemet, a régi elemet egy visszavonási listára helyezi, és megjelöli az aktuális epochhal.
4. Amikor az összes olvasó elhagyta az adott epochot, az ehhez az epochhoz tartozó visszavonási listán lévő elemeket fel lehet szabadítani.
5. Az epochok körben járnak, és az író csak akkor tud felszabadítani, ha a megfelelő epoch már nincs használatban.

Az EBR nem adja meg az RCU azon előnyét, hogy az olvasó valóban nulla overheadű; itt az olvasóknak is per-thread epoch-jelölőt kell írniuk. Azonban az írás per-thread, nem közös cache-vonal, ezért skálázódik. Az EBR-t gyakran használják lock-free listák és hash-táblák felhasználói térben.

Egyszerű pszeudokód:

```
/* EBR pszeudokód, szemléltetés */
#define EPOCH_COUNT 3
_Atomic int global_epoch = 0;
_Thread_local int local_epoch = 0;

void ebr_read_lock(void) {
    local_epoch = atomic_load(&global_epoch);
```

```
}

void ebr_read_unlock(void) {
    local_epoch = -1;
}

void ebr_retire(void *ptr, void (*reclaim)(void *)) {
    int e = atomic_load(&global_epoch);
    retire_list[e].push(ptr, reclaim);
    /* Ha minden szál kilépett az e epochból, felszabadítjuk a listát */
    if (all_threads_unused_epoch(e))
        reclaim_retire_list(e);
}
```

Az `all_threads_unused_epoch` ellenőrzéséhez a szálak epoch-jelölőit kell végignézni. Mivel ez ritkán történik, elfogadható a lineáris ellenőrzés. Az EBR tehát egyfajta könnyű alternatíva, ahol a grace period fogalmát az epochok váltása helyettesíti.

Az RCU és az EBR közös elve: **az olvasó soha nem blokkol, és nem tartozik felelősséggel a memória-visszanyerésért**; a visszanyerést az író oldal végzi, miután bizonyos, hogy nincs több olvasó. A különbség főleg a skálázhatóságban és a kernel/user tér elérhetőségében van.

10.3.7 5. Per-CPU adat és skálázható számlálás

A skálázhatóság másik fontos eszköze a **per-CPU adat**. Ahelyett, hogy minden processzor ugyanazt a globális változót írná, minden CPU saját példányt kap, és a saját példányán végez írást. A globális olvasás összegzi a per-CPU értékeket. Ez a minta drámaian csökkenti a cache-coherency forgalmat, mert minden CPU a saját cache-vonalát írja, amely ritkán kerül más CPU-hoz.

A Linux kernelben a per-CPU változókat a `DEFINE_PER_CPU` makróval lehet deklarálni, és a `get_cpu_var()` / `put_cpu_var()` vagy a `this_cpu_inc()` típusú műveletekkel elérni. A `this_cpu_inc()` például a futó CPU saját példányát növeli atomi utasítás nélkül, mert a preemptiót letiltja rövid időre. Az olvasáshoz a kernel `for_each_possible_cpu` ciklussal összegez.

Vegyünk egy egyszerű skálázható számlálót, amely egy adott esemény (pl. csomagszámláló) gyakoriságát méri:

```
/* Per-CPU számláló pszeudokód, C nyelv */
struct percpu_counter {
    int __percpu *counters;
    int threshold;
    int sum;
};

void percpu_counter_inc(struct percpu_counter *c) {
    this_cpu_inc(c->counters);
    if (this_cpu_read(c->counters) > c->threshold)
        percpu_counter_sync(c);
}

void percpu_counter_sync(struct percpu_counter *c) {
    int cpu;
    int total = 0;
    for_each_possible_cpu(cpu)
        total += per_cpu(c->counters, cpu);
    c->sum = total;
}
```

Az alapötlet az, hogy a triviaális növelés csak a saját CPU-t érinti, és csak a küszöb túllépésekor történik globális szinkronizáció. Ez a megközelítés az úgynevezett **approximate counter** (közelítő számláló) általánosítása. A pontosság és a globális szinkronizáció gyakorisága közötti trade-off a `threshold` paraméterrel hangolható. Ha a threshold 1, akkor minden növelés globális szinkronizációt vált ki, ami valójában egy hagyományos atomi számlálót eredményez. Ha a threshold nagy, a számláló ritkán pontos, de az írások nagyon olcsók.

A per-CPU adat nem csak számlálóknál hasznos. Gyakran tárolnak per-CPU cache-eket, processzor-specifikus állapotot, ütemező-statisztikát, interrupt-számlálót, vagy épp az RCU quiescent state nyilvántartását. A tervezésnél figyelni kell a **false sharing** elkerülésére: a per-CPU változókat úgy kell elhelyezni, hogy minden CPU adata külön cache-vonalra kerüljön. A gyakorlatban ezt a kernel- vagy a liburcu implementációk cache-line alignmenttel oldják meg.

A per-CPU adat és az RCU kombinációja erőteljes: például egy RCU-védett listába bejáró olvasó a lista elemeit olvassa, miközben a statisztikákat per-CPU számlálókkal gyűjti. Az olvasónak nincs lockja, nincs közös írása, és a per-CPU számláló növelése is csak a saját cache-vonalát érinti.

10.3.8 6. Tipikus hibák és helyes megoldások

Az RCU és a per-CPU adat használata során a leggyakoribb hibák a következők:

1. **Alvás az RCU olvasói kritikus szakaszban.** Ha az olvasó `rcu_read_lock()` és `rcu_read_unlock()` között alvó függvényt hív (pl. `kmalloc` `GFP_KERNEL` flaggel egy bizonyos környezetben, vagy fájlrendszer-művelet), akkor a kernel preemptív kontextusban lehet, és a grace period soha nem fejeződik be. Helyes megoldás: csak nem blokkoló műveletet végezni; ha blokkolás kell, használjuk az SRCU-t, amely engedélyezi az alvást, de drágább.

2. **A régi objektum túl korai felszabadítása.** Ha az író a pointer-csere után azonnal `kfree`-t hív anélkül, hogy megvárná a grace periodot, akkor a még futó olvasók felszabadított memóriát olvashatnak. Helyes megoldás: `synchronize_rcu()` vagy `call_rcu()`.

3. **A pointer-közzététel hiányos memória-sorompója.** Ha az író közönséges C-pointerírást használ az `rcu_assign_pointer` helyett, a compiler vagy a CPU átrendezheti a műveleteket, és az olvasó félig inicializált objektumot láthat. Helyes megoldás: mindig a megfelelő RCU makrót használni.

4. **Per-CPU adat atomitásának feltételezése.** A per-CPU adat csak a saját CPU-ján érhető el atomosan; ha egy másik CPU közvetlenül írja, versenyhelyzet keletkezik. Helyes megoldás: minden CPU csak a saját per-CPU példányát módosítsa, a globális metszeteket pedig short lockkal vagy atomi művelettel védjük.

5. **False sharing a per-CPU adatnál.** Ha a per-CPU változók osztoznak egy cache-vonalon, akkor hiába van per-CPU, a közös cache-vonal miatt mégis cache-line bouncing lép fel. Helyes megoldás: cache-line alignment, `____cacheline_aligned` típusú attribútum.

6. **A grace period hosszának alulbecslése.** Az írók a `synchronize_rcu()` hívás alatt blokkolnak, ami hosszú lehet, ha sok olvasó van lassú kritikus szakaszban. Helyes megoldás: aszinkron `call_rcu` callback használata, vagy az adatszerkezet módosításának olyan szervezése, hogy a grace periodra ritkán legyen szükség.

10.3.9 7. Labor: RCU-alapú lista tervezése és mérése

Ez a labor egy felhasználói térben futó RCU-védett linkelt lista tervezését és teljesítménymérését írja le. A környezet QEMU-ban futtatott minimalista Linux, amelyben a liburcu könyvtár elérhető. A cél az olvasó-optimális szinkronizáció és a skálázhatóság bemutatása. A labor során mért és várt eredményt külön kell kezelni, és a mérési naplóban rögzíteni.

10.3.10 7.1. Kiinduló állapot és eszközök

- Emulált környezet: QEMU 7.x (a pontos verzió a reprodukciós csomagban ellenőrizendő), 4 vCPU, 4 GiB RAM, virtio-blk háttértár.
- Operációs rendszer: Linux 6.1 (a pontos verzió ellenőrizendő), `liburcu-dev` csomag telepítve.
- Fordító: GCC 12, `-O2 -march=native` (az emulált CPU esetén a natív itt az emulált CPU-t jelenti), `-pthread -lurcu -lurcu-qsbr` szükség szerint.
- A labor kódja egy eldobható könyvtárban készül, a hoszt valódi fájljait nem módosítjuk.

10.3.11 7.2. Az adatszerkezet

Egy egyirányú linkelt lista, amelyet a szálak keresnek. A lista elemei RCU-védettek. A benchmark három változatot hasonlít össze:

- **rwlock lista:** hagyományos `pthread_rwlock_t`-vel védett lista.
- **RCU lista:** liburcu `urcu` API-val védett lista.
- **RCU-perCPU számlálóval kiegészített lista:** az RCU lista mellett minden olvasó egy per-thread számlálót is növel, a skálázható számlálás demonstrálására.

A lista inicializálása 1000 elemmel történik, majd 4 olvasó szál (minden vCPU-n egy) egy adott ideig kereséseket hajt végre. Az írók ritkán (pl. 10 ms-onként) beszúrnak és törölnek, hogy az RCU grace period aktiválódjon.

10.3.12 7.3. A kód vázlata

A következő pszeudokód a liburcu API-t használja. A pontos függvényneveket és fejlécet a telepített liburcu verziójához igazítva ellenőrizni kell.

```
/* RCU-védett lista, liburcu példa (pszeudokód, verziófüggő) */
#include <urcu.h>
#include <stdlib.h>
#include <stdio.h>
#include <pthread.h>
#include <stdatomic.h>

struct node {
    int key;
    int value;
    struct node *next;
    struct rcu_head rcu;
};

struct node *head = NULL;

void rcu_free_cb(struct rcu_head *rh) {
    struct node *p = caa_container_of(rh, struct node, rcu);
```

```
    free(p);
}

void list_insert(int key, int value) {
    struct node *n = malloc(sizeof(*n));
    n->key = key;
    n->value = value;
    n->next = rcu_dereference(head);
    rcu_assign_pointer(head, n);
}

void list_delete(int key) {
    struct node *p = rcu_dereference(head);
    struct node *prev = NULL;
    while (p) {
        if (p->key == key) {
            if (prev)
                rcu_assign_pointer(prev->next, p->next);
            else
                rcu_assign_pointer(head, p->next);
            call_rcu(&p->rcu, rcu_free_cb);
            return;
        }
        prev = p;
        p = rcu_dereference(p->next);
    }
}

void *reader_thread(void *arg) {
    (void)arg;
    for (int i = 0; i < 1000000; i++) {
        rcu_read_lock();
        struct node *p = rcu_dereference(head);
        while (p) {
            if (p->key == i % 1000) {
                rcu_read_unlock();
                break;
            }
            p = rcu_dereference(p->next);
        }
        rcu_read_unlock();
    }
    return NULL;
}
```

A többi szál hasonló. A per-thread számláló bevezetéséhez egy `_Thread_local` változót használunk, amelyet az olvasó kritikus szakaszon kívül növelünk.

10.3.13 7.4. Build és futtatás

```
# Build parancsok
gcc -O2 -pthread rcu_list.c -lurcu -o rcu_list
gcc -O2 -pthread rwlock_list.c -lurcu -o rwlock_list
# Futtatás 4 vCPU-hoz kötött szálakkal
taskset -c 0-3 ./rcu_list > rcu_measured.log
taskset -c 0-3 ./rwlock_list > rwlock_measured.log
```

A mérési napló tartalmazza a CPU-t, a memóriát, a kernel verziót, a liburcu verziót, a GCC verziót és a buildek idejét. A futási időt a program belső `clock_gettime()` hívása méri, és a végén átbocsátási sebességet számol (művelet/másodperc) minden változatra.

10.3.14 7.5. Elvárt és mért eredmény

A várt eredmény az, hogy az RCU lista olvasási sebessége jelentősen meghaladja az rwlock lista sebességét, mert az RCU olvasók nem írják a közös lockot. 4 magon a skálázhatóság közel lineáris, míg az rwlock esetében a cache-line bouncing korlátozza a párhuzamosságot. A mérés ezt vagy megerősíti, vagy eltérést mutat, amelyet a naplóban dokumentálni kell. A per-thread számláló további kis javulást hozhat a statisztikai utakban.

A reprodukciós csomag a következőket tartalmazza: kiinduló állapot, build parancsok, konfiguráció, futtatási parancs, mérési napló, a kapott átbocsátási értékek, és a visszaállítási útmutató. A labor nem módosít semmilyen hosztállapotot, minden az emulált környezetben történik.

10.3.15 7.6. Önellenőrző kérdés

Miért nem szabad aludni az RCU olvasói kritikus szakaszban, és milyen alternatíva létezik, ha az olvasónak blokkolnia kell? Válaszoljon indoklással, és nevezze meg a megfelelő RCU-variánst.

10.3.16 8. Összefoglalás

Az RCU, az epoch-alapú visszanyerés és a per-CPU adat az olvasó-optimális szinkronizáció és a skálázhatóság három alappillére. Az RCU lehetővé teszi, hogy az olvasók szinte nulla költséggel férjenek hozzá a megosztott adatszerkezethez, miközben az írók a grace period lejárta után biztonságosan visszanyerik a memóriát. Az EBR hasonló célt szolgál felhasználói térben, epochok és per-thread jelölők segítségével. A per-CPU adat és a skálázható számlálás további skálázhatósági nyereséget ad azáltal, hogy elkerüli a globális cache-vonalak írását. A tipikus hibák, mint az alvás a kritikus szakaszban, a túl korai visszanyerés vagy a false sharing, mind elkerülhetők a megfelelő makrók és cache-line alignment következetes használatával. A következő egység (Többmagos és heterogén rendszerek kihívásai) ezeket az elveket alkalmazza többmagos és heterogén rendszerekre, ahol a skálázhatóság és a hardver-specifikus kihívások még hangsúlyosabbá válnak.

10.4 Többmagos és heterogén rendszerek kihívásai

A korábbi fejezetekben a konkurencia kérdéseit elsősorban egyetlen processzor vagy szimmetrikus, azonos magokból álló környezet feltételezésével tárgyaltuk. A mai rendszerek azonban gyakran többféle, eltérő képességű processzormagot kombinálnak, és a memóriaelrendezés sem mindig egyenletes. Ez az egység azt vizsgálja, hogy a párhuzamosság, a cache-hierarchia és a heterogén magok hogyan befolyásolják az operációs rendszer ütemezési, memória- és energiagazdálkodási döntéseit. Nem ismételjük meg a memóriamodell vagy a lock-free programozás alapjait – azokat az `os034`–`os036` egységek tárgyalták. Itt a rendszerszintű következményekre és a gyakorlati tervezési mintákra összpontosítunk.

A tanulási célok eléréséhez először áttekintjük a párhuzamosság és a cache kölcsönhatását, majd az SMP- és NUMA-architektúrákat, végül a big.LITTLE típusú heterogén rendszereket. A terheléselosztás és az energia kérdéseit egy gyakorlati affinitási példán keresztül mutatjuk be, amelyet egy reprodukálható labor követ.

10.4.1 1. A párhuzamosság és a cache hatása

A többmagos rendszerek alapvető ígérete a valódi párhuzamosság: több utasításfolyam egyidejű végrehajtása. A teljesítmény azonban nem egyszerűen a magok számával skálázódik, mert a cache-hierarchia és a memória-hozzáférés jelentős költségeket hordoz. A klasszikus UMA (Uniform Memory Access) modellben minden mag azonos késleltetéssel éri el a főmemóriát, ám a gyakorlatban a magokhoz közeli cache-szintek eltérőek lehetnek, és a koherencia fenntartása további forgalmat generál.

10.4.2 1.1. Cache-koherencia és hamis megosztás

A cache-koherencia protokollok (például a MESI) gondoskodnak arról, hogy az összes mag konzisztens képet lásson a memóriáról. Amikor egy mag módosít egy cache-vonalat, a többi magban lévő másolat érvénytelenül, és a soron következő olvasás koherencia-tranzakciót indít. Ez a mechanizmus akkor is aktiválódik, ha két mag logikailag különböző, de fizikailag ugyanabba a cache-vonalba eső adatot módosít. Ezt a jelenséget **hamis megosztásnak** (false sharing) nevezzük.

Vegyünk egy egyszerű C struktúrát, amelyet két szál egyidejűleg használ:

```
/* Illusztráció: hamis megosztásra hajlamos struktúra */
struct shared_data {
    unsigned long counter_a;
    unsigned long counter_b;
};

struct shared_data shared __attribute__((aligned(64)));
/* Feltételezzük, hogy a cache-vonal mérete 64 bájt. */
```

Ha az egyik szál rendszeresen növeli a `counter_a` mezőt, a másik pedig a `counter_b` mezőt, akkor a két mező ugyanabba a 64 bájtos cache-vonalba kerül. A módosítások versenyeznek a cache-vonal tulajdonjogáért, ami felesleges koherencia-forgalmat és teljesítménycsökkenést okoz – még akkor is, ha a program szemantikailag helyes, és nincs adatverseny.

A megoldás az adatok szétválasztása külön cache-vonalakra, például kitöltéssel vagy igazítással:

```
/* Illusztráció: cache-vonalakra igazított mezők */
struct shared_data {
    unsigned long counter_a;
    unsigned long _pad_a[7];
    unsigned long counter_b;
    unsigned long _pad_b[7];
} shared;
```

Itt a `_pad` mezők biztosítják, hogy a `counter_a` és a `counter_b` más-más cache-vonalra essen. A padding mérete architektúra- és implementációfüggő, ezért a hordozható kódban inkább az `aligned(64)` attribútumot vagy a C11 `_Alignas` specifikációját ajánlott használni. A labormérésben majd látjuk, hogy a hamis megosztás megszüntetése milyen mértékben javítja az átvitelt.

10.4.3 1.2. Cache-affinitás és migrációs költség

Egy folyamat vagy szál többszöri ütemezése során a gyakran használt adatai a legutóbb futtató mag cache-ében halmozódnak fel. Ha az ütemező átköltözteti a szálat egy másik magra, a cache-tartalom elveszik, és a következő futás hideg cache-sel indul, ami kezdetben sok cache-misshez vezet. Ezt a jelenséget **cache-affinitás** vagy **processzor-affinitás** néven ismerjük. A modern ütemezők (például a Linux CFS) kifejezetten igyekeznek megtartani a folyamatot azon a magon, ahol legutóbb futott, hacsak a terheléskiegyenlítés nem indokolja a migrációt.

A migrációs költség nem csupán a cache-újratöltésből adódik. A TLB-bejegyzések is elvesznek, és NUMA-rendszerekben a távoli memória-hozzáférés még drágábbá válik. Emiatt az affinitás megőrzése a teljesítmény szempontjából kulcsfontosságú, de nem abszolút: ha egy mag túlterhelt, a migráció a jobb választás.

10.4.4 1.3. Memória-sorrendezés a többmagos gyakorlatban

Ahogy az `os034` egységben tárgyaltuk, a gyengén sorrendezett architektúrák (például ARM, POWER) és az optimalizáló fordítók megengedik, hogy a memóriaműveletek sorrendje eltérjen a programozó által leírtól. A többmagos rendszerekben ezért a lock-free struktúrák használatakor elengedhetetlen a megfelelő memória-sorrendezési megszorítások alkalmazása. A C11 atomi típusai és a `memory_order` paraméterek lehetővé teszik a szükséges akadályok kifejezését:

```
/* Illusztráció: acquire-release pár használata */
_Atomic int ready = 0;
int data = 0;

void producer(void) {
    data = 42;                      /* sima írás */
    atomic_store_explicit(&ready, 1, memory_order_release);
}

void consumer(void) {
    while (!atomic_load_explicit(&ready, memory_order_acquire))
        ;                           /* spin */
    /* A release-acquire pár garantálja, hogy data == 42 látható. */
}
```

Egy többmagos rendszerben a fenti minta biztosítja, hogy a `data` írása nem kerül a `ready` írás után, illetve a `ready` olvasásakor a consumer már látja a producer összes korábbi írását. A barrier nélküli kód hibás lehet az x86 erős sorrendezése miatt is, ha a jövőben gyengébb architektúrára portoljuk – ezért a hordozható kód mindig explicit sorrendezési megszorításokat használ.

10.4.5 2. SMP és NUMA

A klasszikus **SMP** (Symmetric Multi-Processing) rendszerekben minden processzormag egyenrangú, és a memória-hozzáférés közel azonos késleltetésű. Ez a modell egyszerű, de a nagyobb rendszereknél a közös memóriabusz és a memóriavezérlő szűk keresztmetszetté válik. A **NUMA** (Non-Uniform Memory Access) architektúra ezt a problémát oldja meg azzal, hogy a memóriát csomópontokra osztja, ahol minden csomóponthoz saját processzor(ok) és helyi memória tartozik. Egy mag a saját csomópontjának memóriáját gyorsan éri el, míg a másik csomópont memóriája (távoli memória) nagyobb késleltetéssel és kisebb sávszélességgel érhető el.

10.4.6 2.1. A NUMA topológia és következményei

A NUMA topológiát a firmware (ACPI SRAT) vagy a device tree írja le. Az operációs rendszer ebből építi fel a memória- és processzortopológiai térképet. Egy tipikus kétcsomópontos rendszerben a csomópontok a következőképpen nézhetnek ki:

Csomópont	CPU-k	Helyi memória	Helyi késleltetés (ciklus)	Távoli késleltetés (ciklus)
Node 0	0,1,2,3	0x0000 0000 – 0x3FFF FFFF	~200	~600
Node 1	4,5,6,7	0x4000 0000 – 0x7FFF FFFF	~200	~600

A táblázatban szereplő értékek csak szemléltetésre szolgálnak; a valós késleltetések erősen függnek a hardvertől, az órajeltől és a memóriavezérlőtől. A mérési laborban mindig a saját konfigurációra vonatkozó értékeket kell rögzíteni.

A NUMA hatása akkor válik drámaivá, ha egy folyamat a Node 0-on fut, de a memóriája nagy része a Node 1-ben van. Az operációs rendszer ezért igyekszik a memóriát ahhoz a csomóponthoz kötni, ahol a folyamat fut. Ezt a mechanizmust **memóriapolitikának** nevezzük. A gyakori politikák:

- **First-touch (első érintés):** az oldal ahhoz a csomóponthoz kerül, amelyik először írja. Ez gyakran a futtató mag csomópontját jelenti, de vándorló folyamatoknál hibás döntés lehet.
- **Bind (kötés):** a memória kizárólag egy adott csomópontról érhető el.
- **Interleave (váltakozás):** az oldalakat ciklikusan osztja el a csomópontok között, ami hasznos lehet sok szálat használó, nagy adattömböket kezelő alkalmazásoknál.
- **Preferred (előnyben részesített):** egy adott csomópontot előnyben részesít, de vész esetén máshonnan is allokálhat.

Linux alatt ezeket a `mbind`, `set_mempolicy` rendszerhívásokkal vagy a `numactl` parancssori eszközzel lehet szabályozni. A programozói gyakorlatban célszerű a teljesítménykritikus alkalmazásokat úgy futtatni, hogy a szálak és a memóriájuk egy csomóponton maradjanak. Ezt a célt szolgálja a processzor- és memória-affinitás együttes beállítása.

10.4.7 2.2. Ütemezési következmények NUMA-rendszerekben

A NUMA-tudatos ütemezőnek figyelembe kell vennie a memória elhelyezkedését is, nem csak a CPU-terhelést. A Linux CFS-ben a **sched_domain** hierarchia leírja a magok közötti topológiát: először azonos magon lévő szálak (SMT), majd azonos NUMA-csomópontban lévő magok, végül a teljes rendszer. A terheléskiegyenlítés először a legalsó szinten próbálkozik, és csak akkor lép feljebb, ha az alsóbb szint nem képes kiegyenlíteni a terhelést. Ez csökkenti a távoli memóriahozzáférés valószínűségét.

A gyakorlatban a NUMA-tudatosságot a következő eszközökkel lehet mérni és befolyásolni:

- `numactl --hardware`: megmutatja a topológiát és a csomópontok közötti távolságot.
- `numactl --cpunodebind=0 --membind=0 <parancs>`: a folyamatot és a memóriáját a Node 0-ra köti.

- `numastat`: megjeleníti a NUMA-csomópontonkénti memóriastatisztikákat, például a helyi és távoli elérési arányokat.

A laborban ezeket az eszközöket használjuk a méréshez.

10.4.8 3. Heterogén magok: big.LITTLE és modern hibrid architektúrák

Az energiatakarékosság és a teljesítmény követelményeinek egyidejű kielégítésére a processzorgyártók heterogén többmagos rendszereket fejlesztettek ki. Az ARM **big.LITTLE** és **DynamIQ** koncepciója, valamint az Intel hibrid architektúrája (P-core/E-core) két eltérő típusú magot kombinál:

- **Nagy magok (big / P-core):** magas órajel, mély futószalag, nagyobb cache, nagyobb teljesítmény és energiafogyasztás.
- **Kis magok (LITTLE / E-core):** alacsonyabb órajel, rövidebb futószalag, kisebb cache, lényegesen alacsonyabb energiafelvétel, de korlátozott csúcsteljesítmény.

A cél, hogy a rövid, interaktív feladatok a kis magokon fussanak, míg a nagy számításigényű, hosszabb feladatok a nagy magokra kerüljenek. Az ütemezőnek tehát nem csupán a terhelést kell kiegyenlítenie, hanem a magok **kapacitását** is figyelembe kell vennie.

10.4.9 3.1. Kapacitásalapú ütemezés

A homogén SMP rendszerekben minden mag azonos kapacitású, ezért a terheléskiegyenlítés lényege a futási idők egyenlő elosztása. Heterogén rendszerben a „terhelés” fogalma relatívvá válik: egy nagy magon 10 ms CPU-idő több hasznos munkát jelent, mint egy kis magon 10 ms. Az operációs rendszer ezért **kapacitássúlyozott** terhelést számol.

Linuxban a **Capacity Aware Scheduling** és az **Energy Aware Scheduling** (EAS) kiterjesztések kezelik ezt. A magok kapacitása a maximális frekvenciájukhoz és IPC-jükhöz viszonyított relatív érték. Az ütemező a folyamatok terhelését a magok kapacitásához méri, és a döntéseknél figyelembe veszi a várható energiafogyasztást is. A nagy magokra akkor kerülnek feladatok, ha a kis magok már nem képesek a határidőket tartani, vagy ha a feladat jellege (pl. számításigényes, hosszú) ezt indokolja.

10.4.10 3.2. Feladat-osztályozás és migráció

A feladatokat jellemzőik alapján lehet osztályozni:

- **CPU-intenzív:** folyamatosan számol, ritkán blokkol. Ideális a nagy magra, különben a futási idő meghosszabbodik.
- **I/O-intenzív:** gyakran vár I/O-ra, rövid CPU-burstjei vannak. A kis magon is jól fut, energiatakarékosan.
- **Interaktív:** rövid késleltetési igények, de gyakori ébredések. A kis magon való futtatás növelheti az észlelt késleltetést, ezért gyakran a nagy magra kerül.

A migráció a heterogén rendszerekben kétféle lehet:

- **Kis magról nagyra:** amikor egy feladat CPU-igénye tartósan megnő, a kis magon már nem fér el a kívánt késleltetés, ezért át kell költöztetni. Ez a migráció viszonylag drága, de a teljesítmény indokolja.

- **Nagy magról kisra:** amikor a feladat CPU-igénye lecsökken, vagy a nagy magra másik feladat vár, a folyamat átkerülhet a kis magra energiát megtakarítani.

Az ütemezőnek óvatosan kell dönteni a migrációról, mert a túl gyakori váltás (ping-pong) a cache-affinitást és a TLB-t is érinti. A gyakorlatban a Linux CFS hiszterézist alkalmaz: a migrációhoz a terheléskülönbségnek egy bizonyos küszöböt meg kell haladnia.

10.4.11 3.3. Energia- és hőkezelés

A heterogén rendszerek egyik fő célja az energiafogyasztás csökkentése. Az ütemező döntései közvetlenül hatnak az energiafelvételre és a hőtermelésre. Az energia szempontjából két stratégia létezik:

- **Race-to-idle:** a feladatot a leggyorsabb magon futtatjuk, majd a rendszer mielőbb mély alvó állapotba lép. Ez akkor hatékony, ha a feladat rövid, és az alvó állapot energiamegtakarítása nagy.
- **Pack-and-sleep:** a feladatokat kevés magra sűrítjük, hogy a többi mag mély alvó állapotba kerülhessen. Ez akkor előnyös, ha a feladatok közötti kommunikáció kicsi, és a szükséges teljesítmény alacsony.

Az EAS ezeket a stratégiákat energia-modell segítségével értékeli. A modell az adott rendszerre jellemző energiafelvételi görbéket tartalmazza. Az ütemező célja az energia és a késleltetés egyensúlyának megtalálása. A `schedutil` CPU-frekvencia-szabályozó szorosan együttműködik az ütemezővel, és a terheléshez igazítja az órajelet.

10.4.12 4. Terheléselosztás és energia

A terheléselosztás klasszikus célja a processzorok kihasználtságának kiegyenlítése, minimalizálva a várakozási időt. Heterogén és energiaérzékeny rendszerekben azonban a cél bonyolultabb: a teljesítmény optimalizálását az energiafogyasztás és a hőtermelés korlátai mellett kell elvégezni.

10.4.13 4.1. Terheléskiegyenlítő algoritmusok

A Linux CFS-ben a terheléskiegyenlítést időszakosan (periodikusan), üresjáratban (idle) és felébredéskor (wakeup) is elvégzi. A periodikus kiegyenlítés egy adott `sched_domain` szinten történik, és a folyamatokat a túlterhelt magokról a kevésbé terheltekre mozgatja. Az idle kiegyenlítés akkor lép működésbe, amikor egy mag befejezi a munkáját, és megpróbálja ellopni egy másik magtól a folyamatot. A wakeup kiegyenlítés akkor dönt, amikor egy alvó folyamat felébred, és több mag közül választhat – itt figyelembe veszi a cache-affinitást és az energiahatékonyságot.

A terhelés mérésére a Linux a **PELT** (Per-Entity Load Tracking) modellt használja, amely minden folyamat terhelését időben csökkenő súllyal követi. Ez lehetővé teszi a rövid távú ingadozások simítását, és megbízható becslést ad a folyamat CPU-igényéről. Heterogén rendszerben a PELT értéket a mag kapacitásával normalizálják.

10.4.14 4.2. Energiagazdálkodás

Az energiafelvétel csökkentésének három fő eszköze:

- **DVFS** (Dynamic Voltage and Frequency Scaling): a CPU órajelének és feszültségének csökkentése alacsony terhelésnél. Az energiafelvétel közelítőleg a frekvenciával lineárisan, a feszültséggel négyzetesen arányos, ezért a frekvenciacsökkentés jelentős megtakarítást hoz.
- **CPU idle állapotok (C-states):** a mag mélyebb alvó állapotba kerül, amikor nincs munka. Minél mélyebb az állapot, annál nagyobb az energiamegtakarítás, de annál hosszabb a felébredési késleltetés.
- **Klaszterezés:** a big.LITTLE rendszerekben a magok csoportokba (clusters) szerveződnek, és a teljes klaszter energiája akkor kapcsolható ki, ha minden mag alvó állapotban van.

Az ütemezőnek mérlegelnie kell: érdemes-e a feladatot a nagy magra tenni, hogy hamarabb végezzen (race-to-idle), vagy a kis magon futtatni, hogy csökkentsük a pillanatnyi energiafelvételt. Az **energia-késleltetési szorzat** (Energy-Delay Product, EDP) egy gyakran használt metrika:

$$EDP = E \cdot t_{\text{exec}},$$

ahol E az energiafogyasztás, t_{exec} pedig a végrehajtási idő. Az EDP minimalizálása kompromisszumot jelent a nyers teljesítmény és az energia között.

10.4.15 4.3. Gyakorlati példa: affinitási terv

Tegyük fel, hogy egy QEMU-ban emulált, 4 magos, 2 nagy + 2 kis maggal rendelkező rendszeren két folyamatot futtatunk:

- P_1: CPU-intenzív, hosszú, határidő nélküli számítás (pl. mátrixszorzás). Jellemzői: $C_1 = 8\,\text{ms}$, $T_1 = 10\,\text{ms}$, $D_1 = 10\,\text{ms}$ (látszólag valós idejű, de a példában csak a relatív terhelés illusztrálására).
- P_2: interaktív, rövid, gyakori ébredésű feladat (pl. shell-parancs). Jellemzői: $C_2 = 1\,\text{ms}$, $T_2 = 20\,\text{ms}$, $D_2 = 2\,\text{ms}$.

A cél: P_1 futtatása a nagy magokon, P_2 futtatása a kis magokon, miközben a cache-affinitást megtartjuk. A tervet így fogalmazzuk meg:

1. Azonosítjuk a nagy és kis magokat a rendszerben (pl. `/sys/devices/system/cpu/cpu*/cpu_capacity` vagy a device tree alapján).
2. P_1-t a nagy magokhoz (CPU0, CPU1) kötjük `sched_setaffinity` segítségével.
3. P_2-t a kis magokhoz (CPU2, CPU3) kötjük.
4. A memóriát az első érintés politikájával a megfelelő csomóponthoz rendeljük (NUMA esetén).
5. A mérés során rögzítjük a végrehajtási időt, a cache-miss arányt és az energiafogyasztást.

Az alábbi C kódrészlet szemlélteti a processzor-affinitás beállítását Linuxon:

```
/* Illusztráció: CPU-affinitás beállítása Linuxon */
#define _GNU_SOURCE
#include <sched.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main(int argc, char **argv) {
```

```
    cpu_set_t set;
    CPU_ZERO(&set);

    /* A példában a háttérben futó CPU-intenzív folyamatot
       a 0. és 1. magra (nagy magokra) kötjük. */
    CPU_SET(0, &set);
    CPU_SET(1, &set);

    pid_t pid = getpid();
    if (sched_setaffinity(pid, sizeof(set), &set) == -1) {
        perror("sched_setaffinity");
        return EXIT_FAILURE;
    }

    /* Itt jön a tényleges számítási feladat. */
    /* ... */

    return EXIT_SUCCESS;
}
```

A rendszerhívás paraméterei: `pid`, `cpusetsize`, `mask`. Visszatérési értéke 0 siker esetén, `-1` hibakor, ilyenkor az `errno` a hibát jelzi. A kód nem végrehajtandó a gazdagépen, csak emulált környezetben.

10.4.16 5. Labor: többmagos ütemezési és affinitási terv

A labor célja, hogy egy emulált környezetben mérje a hamis megosztás és a NUMA/affinitás hatását, és ellenőrizze a heterogén ütemezési tervet. A labor **kiinduló állapota** egy QEMU virtuális gép, amely 4 virtuális CPU-val és 2 NUMA-csomóponttal rendelkezik. A konfiguráció a következő parancsokkal hozható létre:

```
# QEMU indítása 4 CPU-val, 2 NUMA-csomóponttal, 1 GiB memóriával
qemu-system-x86_64 \
  -enable-kvm \
  -smp 4 \
  -m 1024 \
  -numa node,nodeid=0,cpus=0-1,memdev=ram0 \
  -numa node,nodeid=1,cpus=2-3,memdev=ram1 \
  -object memory-backend-ram,id=ram0,size=512M \
  -object memory-backend-ram,id=ram1,size=512M \
  -drive file=emulated-disk.img,format=raw \
  -boot order=c \
  -serial mon:stdio
```

Figyelem: Ez a parancs csak emulált környezetben futtatható. A hoszt valódi lemezét, bootloaderét vagy firmware-ét tilos módosítani. A labor minden lépése eldobható virtuális gépen történik.

10.4.17 5.1. Mérőprogram

Két rövid C programot készítünk:

1. **False sharing teszt:** Két szál párhuzamosan növeli egy közös struktúra különböző mezőit. Az egyik változatban a mezők ugyanabba a cache-vonalba esnek (hamis megosztás), a másikban paddinggel szétválasztjuk őket. Mindkét változatot 10 millió iterációra futtatjuk, és `clock_gettime` segítségével mérjük az időt, valamint `perf stat`-tal a cache-miss eseményeket.

2. **Affinitás teszt:** Egy CPU-intenzív folyamatot (pl. szumma számítás) először CPU0-ra kötünk, majd CPU0–1-re engedjük, végül CPU0–3-ra. Mindegyik esetben mérjük a falási időt és a CPU-migrációk számát.

A mérőprogram pszeudokódja:

```
/* Illusztráció: false sharing mérése */
#include <pthread.h>
#include <time.h>
#include <stdio.h>

#define ITER 10000000

struct shared {
    unsigned long a;
    unsigned long pad[7]; /* 0, ha nincs padding; 7, ha van */
    unsigned long b;
} s;

void *worker_a(void *arg) {
    for (int i = 0; i < ITER; ++i) s.a++;
    return NULL;
}

void *worker_b(void *arg) {
    for (int i = 0; i < ITER; ++i) s.b++;
    return NULL;
}

int main(void) {
    struct timespec t0, t1;
    clock_gettime(CLOCK_MONOTONIC, &t0);
    pthread_t ta, tb;
    pthread_create(&ta, NULL, worker_a, NULL);
    pthread_create(&tb, NULL, worker_b, NULL);
    pthread_join(ta, NULL);
    pthread_join(tb, NULL);
    clock_gettime(CLOCK_MONOTONIC, &t1);
    double dt = (t1.tv_sec - t0.tv_sec) + (t1.tv_nsec - t0.tv_nsec)/1e9;
    printf("Elapsed: %f s\n", dt);
    return 0;
}
```

A padding nélküli és a paddinges változatot ugyanazzal a fordítási beállítással (`-O2`) kell lefordítani, és mindegyiket legalább ötször lefuttatni. A mért és a várt eredményeket külön kell rögzíteni.

10.4.18 5.2. Mérési napló

A mérési napló a következő adatokat tartalmazza:

Kísérlet	Változat	CPU-k	Futási idő (s, medián)	Cache-miss (millió)	CPU-migráció	Várt eredmény
FS-01	nincs pad	4	2.34	45.1	120	lassú, sok miss
FS-02	pad	4	1.12	2.3	45	gyors, kevés miss
AFF-01	CPU0	1	1.80	3.5	0	nincs migráció

Kísérlet	Változat	CPU-k	Futási idő (s, medián)	Cache-miss (millió)	CPU-migráció	Várt eredmény
AFF-02	CPU0–1	2	1.10	4.1	8	kevés migráció
AFF-03	CPU0–3	4	0.75	5.2	57	sok migráció

A táblázatban szereplő számok **szemléltető jellegűek**, a tényleges értékek a hardver, a QEMU verzió és a gazdagép terhelésének függvényében változnak. A labornak mindig a saját mérési naplóját kell használnia, és a mért értéket el kell különíteni a várt/levezetett értéktől. Például a hamis megosztásnál **várt** eredmény, hogy a paddinges változat gyorsabb, mert nincs koherencia-tranzakció; a **mért** eredmény ezt számszerűsíti.

10.4.19 5.3. Kiértékelés és ellenőrzés

A mérési eredményeket a következő szempontok szerint értékeljük:

- A hamis megosztás megszüntetése csökkenti-e a futási időt és a cache-miss eseményeket?
- Az affinitás korlátozása csökkenti-e a CPU-migrációt, és hogyan hat a futási időre?
- NUMA-rendszerben a `numastat` segítségével ellenőrizzük, hogy a helyi memória-elérés aránya javult-e a megfelelő politikával.

A labornak tartalmaznia kell a `numactl --hardware` kimenetét, a `perf stat` parancsot, a naplót és a visszaállítási útvonalat (a virtuális gép pillanatképe a kísérlet előtt). A mérés csak akkor tekinthető reprodukálhatónak, ha a fenti adatok együtt elérhetők.

10.4.20 6. Tipikus hibák és ellenőrzési pontok

A többmagos és heterogén rendszerekben számos hiba forrása van. A leggyakoribbak:

1. **Hamis megosztás figyelmen kívül hagyása:** A teljesítmény csökkenése nem nyilvánvaló, mert a kód logikailag helyes. Ellenőrzés: `perf c2c` (cache-to-cache) elemzés Linuxon, amely megmutatja a hamis megosztással érintett cache-vonalakat.
2. **Túlzott migráció:** Az ütemező gyakran költözteti a folyamatokat, ami cache- és TLB-hideget okoz. Ellenőrzés: `/proc/<pid>/status voluntary_ctxt_switches` és `nonvoluntary_ctxt_switches` mezői; `perf stat -e sched:sched_migrate_task`.
3. **NUMA-topológia mellőzése:** A távoli memória-hozzáférés sávszélessége és késleltetése drasztikusan eltérhet. Ellenőrzés: `numastat -p <pid>`, `numactl --hardware`; a távoli elérés arányának minimalizálása.
4. **Energiatévesztés:** A race-to-idle stratégia nem mindig optimális, különösen hosszú, alacsony intenzitású feladatoknál. Ellenőrzés: energia-mérés a hardver szintjén (pl. RAPL, powertop), vagy energia-modell szimulációja.
5. **Heterogén rendszerben a magok egyenlőként kezelése:** A kapacitásbeli különbségek figyelmen kívül hagyása nem várt késleltetéseket okoz. Ellenőrzés: `/sys/devices/system/cpu/cpu*/cpu_capacity` értékek, ütemezői statisztikák (`/proc/schedstat`).

Egy hibakeresési eseménynél a `panic`, a stack trace, a napló és a reprodukálható konfiguráció együtt szükséges. A laborban a virtuális gép naplója és a konfigurációs fájlok a reprodukciós csomag részei.

10.4.21 7. Összefoglalás

Ebben az egységben áttekintettük a többmagos és heterogén rendszerek ütemezési, memória- és energiakihívásait. A cache-koherencia és a hamis megosztás közvetlenül befolyásolják a párhuzamos programok teljesítményét; a NUMA-architektúrákban a memória-elhelyezés és az affinitás kulcsfontosságú. A big.LITTLE típusú heterogén rendszerek kapacitásalapú ütemezést és energia-tudatos döntéseket igényelnek. A gyakorlati affinitási terv és a mérési labor segítségével a fogalmak ellenőrizhetővé válnak.

A következő egység (Címterek, fizikai és virtuális cím) a memóriakezelés alapjaira lép, ahol az itt megismert cache- és NUMA-fogalmak fontos előfeltételt jelentenek: a lapozás és a TLB viselkedése szorosan kapcsolódik a cache-hierarchiához és a párhuzamos memória-hozzáféréshez.

10.4.22 Önellenőrző kérdés

Egy 8 magos rendszerben két NUMA-csomópont van (Node 0: CPU 0–3, Node 1: CPU 4–7). P_1 egy CPU-intenzív, nagy memóriaigényű folyamat, P_2 pedig egy interaktív, kis memóriaigényű folyamat. Tervezze meg az affinitási és memóriapolitikát úgy, hogy a végrehajtási időt és az energiafogyasztást is optimalizálja. Indokolja, hogy miért a választott csomópontra köti a folyamatokat, és milyen mérésekkel ellenőrizné a terv helyességét.

III. RÉSZ – Memória és I/O

11 Memóriakezelés alapjai

11.1 Címterek, fizikai és virtuális cím

A memóriakezelés alapjainak tárgyalásakor az első és legfontosabb absztrakció a címfordítás: az a mechanizmus, amely elválasztja a program által használt címeket a fizikai memória tényleges címeitől. Ez a fejezet bevezeti a virtuális cím fogalmát, bemutatja a logikai, virtuális és fizikai cím közötti különbséget, majd lépésről lépésre felépíti az MMU (Memory Management Unit) és a lapasztal működését. A cél nem csupán a fogalmak definiálása, hanem egy olyan gyakorlati minta bemutatása, amely lehetővé teszi a címfordítás ellenőrzését egy kis, reprodukálható környezetben.

11.1.1 A virtuális cím fogalma és a címfordítás alapelve

A virtuális cím a processzor által a program végrehajtása során használt cím, amely nincs közvetlen kapcsolatban a fizikai memória tényleges celláival. A program úgy viselkedik, mintha egy összefüggő, saját címteret látna, amelynek mérete a hardver által támogatott virtuális címtartomány (például 32 bites rendszeren 4 GiB, 64 bites rendszeren elméletileg 2^64, de a gyakorlatban a hardver korlátozza). A virtuális cím és a fizikai cím közötti megfeleltetést az operációs rendszer és a hardver közösen végzi el, a folyamat futása közben, minden egyes memóriahivatkozásnál. Ezt a folyamatot nevezzük címfordításnak.

A címfordítás alapvető célja az izoláció és az absztrakció. Minden folyamat saját virtuális címteret kap, így az egyik folyamat nem láthatja vagy módosíthatja a másik memóriáját. Emellett a programozó számára egyszerűbb modellt nyújt: a program a saját címterében foglalhat memóriát, nem kell ismernie a fizikai memória elrendezését, a töredezettséget vagy a többi folyamat által lefoglalt területeket. A virtuális memória további előnyei – például a lapozás, a lapcsere, a memóriavédelem – a későbbi fejezetekben kerülnek részletes kifejtésre. Itt elsősorban a címfordítás mechanizmusára összpontosítunk.

A címfordítás hardveres támogatással történik: az MMU (Memory Management Unit) végzi el a virtuális cím fizikai címre való leképezését minden memóriahivatkozás alkalmával. Az MMU a processzor része, vagy szorosan ahhoz kapcsolódó egység, és az operációs rendszer által karbantartott lapasztalból olvassa ki a leképezést. A lapasztal egy adatszerkezet, amely a virtuális lapokat fizikai lapkeretekre képezi, és tartalmazza a hozzáférési jogosultságokat is. A következő szakaszokban először tisztázzuk a különböző címtípusokat, majd bemutatjuk az MMU és a lapasztal működését.

11.1.2 Logikai, virtuális és fizikai cím

A szakirodalomban gyakran keverednek a logikai, virtuális és fizikai cím fogalmai. A pontos különbségtétel elengedhetetlen a memóriakezelés helyes megértéséhez.

- **Logikai cím** (logical address): a program által használt cím a forráskódban vagy a gépi kódban, a fordítás és a szerkesztés után. A logikai cím a program címterében értelmezett, és a programozó számára látható. Szegmentált memóriamodellben a logikai cím egy szegmensszelektor és egy offset kombinációja (például `CS:0x1234` vagy `DS:0x5678`). A szegmensszelektor azonosít egy szegmenst, az offset pedig a szegmensen belüli eltolást adja meg.
- **Virtuális cím** (virtual address): az a cím, amelyet az MMU a fordítás során bemenetként kap. A virtuális cím a logikai címből származik, miután a szegmensszelektor alapján a szegmens báziscímét hozzáadták az offsethöz. A lapozást használó, szegmentálás nélküli rendszerekben (ilyen a legtöbb modern operációs rendszer, például a Linux x86-on) a logikai cím és a virtuális cím gyakorlatilag megegyezik, mert a szegmensbázisok értéke nulla. A továbbiakban a virtuális címet tekintjük az MMU bemenetének, és feltételezzük, hogy a logikai cím megegyezik vele, hacsak külön nem jelezzük.
- **Fizikai cím** (physical address): a tényleges memóriacella címe a fizikai memóriában (RAM, ROM, eszközök memóriatartománya). A fizikai cím az, amelyet a memóriavezérlő a címbuszon kiad, és amely alapján a memória kiválasztja a megfelelő cellát. A fizikai címtartomány általában kisebb, mint a virtuális címtartomány, különösen a 64 bites rendszerekben, ahol a ténylegesen telepített memória mérete korlátozott.

A három címtípus közötti kapcsolatot a következő példa szemlélteti. Tegyük fel, hogy egy 32 bites x86 rendszer lapozást használ szegmentálás nélkül. A program egy logikai címet generál, például `0x08048000` (a szegmensbázis nulla, így ez a virtuális cím is). Az MMU ezt a virtuális címet lefordítja egy fizikai címre, mondjuk `0x0012abcd`-re, a lapasztal alapján. A program soha nem látja a fizikai címet, és nem is tudja befolyásolni, hogy melyik fizikai lapra kerül.

A memóriatérképen a virtuális és fizikai tartományokat külön kell jelölni. Például egy folyamat virtuális címtere a 0x0000_0000–0xFFFF_FFFF tartományt fedi le, míg a fizikai memória csak a 0x0000_0000–0x1FFF_FFFF tartományt (512 MiB). A lapasztal határozza meg, hogy a virtuális tartomány mely részei vannak jelen a fizikai memóriában, és hol.

11.1.3 Az MMU és a lapasztal

Az MMU (Memory Management Unit) hardveres egység, amely a virtuális címet fizikai címre fordítja. Az MMU minden memóriahivatkozásnál (utasításbetöltés, adatolvasás, adatírás) aktív. A fordítás alapja a lapasztal (page table), amely a virtuális lapokat fizikai lapkeretekre képezi. A lap a virtuális címteret osztja fel rögzített méretű blokkokra, a lapkeret pedig a fizikai memóriát ugyanekkora blokkokra. A lapméret tipikusan 4 KiB az x86 és az ARM architektúrákon, de lehet 16 KiB, 64 KiB, 2 MiB vagy akár 1 GiB is, a hardveres támogatástól függően.

A lapasztal bejegyzései (page table entry, PTE) tartalmazzák a lapkeret fizikai címét, valamint vezérlőbiteket, például: - **Present (P)**: a lap a fizikai memóriában van-e. Ha nincs, laphiba következik be. - **Read/Write (R/W)**: a lap írható-e, vagy csak olvasható. - **User/Supervisor (U/S)**: a lap elérhető-e felhasználói módban, vagy csak kernel módban. - **Accessed (A)**: a laphoz hozzáfértek-e, azóta, hogy a bitet törölték. - **Dirty (D)**: a lapot módosították-e, mióta a bitet törölték. Ez a bit fontos a lapcserélésnél. - **Cache Disable (CD)**: a lap tartalma nem gyorsítótárazható. - **Execute Disable (XD/NX)**: a lapról nem lehet utasítást végrehajtani.

A lapasztal szerkezete architektúrafüggő. A modern rendszerek többszintű lapasztalat használnak, mert az egyszintű lapasztal túl nagy lenne. Például egy 32 bites rendszer 4 KiB lapokkal 2^20 lapot és ugyanennyi bejegyzést igényelne, ami bejegyzésenként 4 bájttal számolva 4 MiB-ot tenne ki. Ez folyamatos memóriafoglalást igényelne, ami nem praktikus. A többszintű lapasztal

lehetővé teszi, hogy a lapasztal maga is lapokban legyen tárolva, és csak a ténylegesen használt részek legyenek jelen.

A következő részben egy konkrét példán keresztül mutatjuk be a többszintű lapasztal bejárását és a címfordítás lépéseit.

11.1.4 A címfordítás lépései kétszintű lapasztal esetén

Vegyünk egy 32 bites virtuális címtartományt és 4 KiB lapméretet. A 32 bites virtuális címet három részre bontjuk: - **Lapkönyvtár-index (PDI, page directory index)**: a legfelső 10 bit (a cím 31–22. bitje). - **Laptábla-index (PTI, page table index)**: a következő 10 bit (a 21–12. bitje). - **Offset**: az alsó 12 bit (a 11–0. bitje), amely a lapon belüli eltolást adja meg.

A lapkönyvtár (page directory) 1024 bejegyzést tartalmaz, mindegyik egy laptábla fizikai címére mutat. A laptábla (page table) szintén 1024 bejegyzést tartalmaz, mindegyik egy lapkeret fizikai címére mutat. A bejegyzések 32 bitesek, így a lapkönyvtár és a laptábla mérete egyaránt 4 KiB, ami pontosan egy lap.

A címfordítás lépései: 1. A processzor az MMU-nak átadja a virtuális címet. 2. Az MMU kiolvassa a lapkönyvtár báziscímét (amely a processzor egy speciális regiszterében, például x86-on CR3-ban, van tárolva). 3. Az MMU a PDI-vel indexeli a lapkönyvtárat, és megkapja a laptábla fizikai címét. 4. Az MMU a PTI-vel indexeli a laptáblát, és megkapja a lapkeret fizikai címét. 5. Az MMU a lapkeret címéhez hozzáadja az offsetet, így előáll a fizikai cím.

A folyamatot a következő pszeudokód szemlélteti:

```
function translate(vaddr):
    pdi = (vaddr >> 22) & 0x3FF   // 10 bit
    pti = (vaddr >> 12) & 0x3FF   // 10 bit
    offset = vaddr & 0xFFF        // 12 bit

    pde = read_memory(CR3 + pdi * 4)   // 32 bites bejegyzés
    if not pde.Present:
        raise PageFault
    pte = read_memory(pde.TableBase + pti * 4)
    if not pte.Present:
        raise PageFault
    if not pte.ReadWrite and access_type == write:
        raise ProtectionFault
    paddr = pte.FrameBase + offset
    return paddr
```

A fenti pszeudokód szemléltetés, nem végrehajtható utasítás. A valós hardveren a folyamatot az MMU végzi, és a laphiba kivételt vált ki, ha a lap nincs jelen.

11.1.5 Lapasztal-szintek és címfordítási költség

A többszintű lapasztal csökkenti a memóriaigényt, de növeli a címfordítási lépések számát. Egy kétszintű lapasztal esetén egy memóriahivatkozáshoz két további memóriaolvasás szükséges a lapasztal bejárásához. Háromszintű lapasztalnál három, négyszintűnél négy. Ez a többletköltség jelentős lehet, ezért a hardver TLB-t (Translation Lookaside Buffer) használ, amely a gyakran használt leképezéseket gyorsítótárazza. A TLB működését a következő fejezet (Lapozás, szegmentálás és TLB) tárgyalja részletesen; itt csak annyit jegyzünk meg, hogy a TLB találat esetén a fordítás egyetlen ciklus alatt megtörténik, míg TLB hiba esetén a lapasztal bejárása több memóriahozzáférést igényel.

A lapasztal-szintek száma a hardver és a virtuális címtartomány függvénye. A 64 bites x86-64 architektúra 4 KiB lapokkal négyszintű lapasztalat használ (PML4, PUD, PMD, PT). A 64 bites ARM (AArch64) három- vagy négyszintű lapasztalat használhat, a lapmérettől függően. Az egyszerűség kedvéért ebben a fejezetben a 32 bites kétszintű modellt használjuk, mert ez a legkönnyebben követhető.

11.1.6 Gyakorlati minta: címfordítás egy konkrét példán

Tegyük fel, hogy egy 32 bites rendszeren a lapkönyvtár báziscíme `0x0000_1000` (fizikai cím). A rendszer 4 KiB lapokat használ, a lapkönyvtár és a laptábla bejegyzései 32 bitesek. A virtuális cím `0x1234_5678`. Számítsuk ki a lapkönyvtár-indexet, a laptábla-indexet és az offsetet, majd kövessük a fordítási folyamatot.

Először bontsuk fel a virtuális címet binárisan:

`0x12345678 = 0001 0010 0011 0100 0101 0110 0111 1000`

A 10-10-12 bit felosztás szerint: - PDI (31–22 bit): `0001 0010 00` → bináris `0001001000` → decimális 72 → hexadecimális `0x48`. - PTI (21–12 bit): `11 0100 0101` → bináris `1101000101` → decimális 837 → hexadecimális `0x345`. - Offset (11–0 bit): `0110 0111 1000` → hexadecimális `0x678`.

Most feltételezzük, hogy a lapkönyvtár 72. bejegyzése (`CR3 + 72 * 4 = 0x0000_1000 + 0x120 = 0x0000_1120`) a `0x0000_2000` laptábla-címet tartalmazza, Present bittel beállítva. A laptábla 837. bejegyzése (`0x0000_2000 + 837 * 4 = 0x0000_2000 + 0xD14 = 0x0000_2D14`) a `0x0003_F000` lapkeret-címet tartalmazza, Present bittel. Az offset `0x678`. A fizikai cím tehát `0x0003_F000 + 0x678 = 0x0003_F678`.

Ezt a számítást ellenőrizhetjük egy egyszerű C programmal, amely szimulálja az MMU-t. A kód szemléltetés, nem éles rendszeren futtatandó:

```c
#include <stdint.h>
#include <stdio.h>
#include <stdbool.h>

#define PAGE_PRESENT 0x1
#define PAGE_RW      0x2

typedef struct {
    uint32_t entry;
} pte_t;

uint32_t translate(uint32_t vaddr, uint32_t cr3) {
    uint32_t pdi = (vaddr >> 22) & 0x3FF;
    uint32_t pti = (vaddr >> 12) & 0x3FF;
    uint32_t offset = vaddr & 0xFFF;

    pte_t *pde = (pte_t *)(cr3 + pdi * sizeof(pte_t));
    if (!(pde->entry & PAGE_PRESENT)) {
        printf("Page fault at PDE index %u\n", pdi);
        return 0;
    }
    pte_t *pte = (pte_t *)((pde->entry & ~0xFFF) + pti * sizeof(pte_t));
    if (!(pte->entry & PAGE_PRESENT)) {
        printf("Page fault at PTE index %u\n", pti);
        return 0;
    }
    uint32_t frame = pte->entry & ~0xFFF;
    return frame + offset;
```

```
}

int main(void) {
    // Példa lapkönyvtár és laptábla beállítása (helyőrző értékek)
    // A valós hardveren a lapasztalat az OS építi fel.
    // Ezt a kódot csak szemléltetésként mutatjuk be.
}
```

A fenti kódban a `cr3` a lapkönyvtár fizikai címét jelöli, a `pte_t` pedig egy bejegyzést reprezentál. A `PAGE_PRESENT` bit ellenőrzése után a lapkeret címét az alsó 12 bit levágásával kapjuk meg. A kód nem tartalmazza a teljes inicializálást, mert a példa célja a fordítási logika bemutatása.

Az ilyen szimuláció segítségével ellenőrizhető a címfordítás helyessége, és felismerhetők a tipikus hibák.

11.1.7 Labor: címfordítási példa készítése

A labor célja, hogy a hallgató reprodukálható környezetben elkészítsen egy egyszerű MMU-szimulátort, és mérési naplóval dokumentálja a címfordítás eredményét. A labor emulált környezetben vagy eldobható virtuális gépen futtatható, a hoszt valódi lemezét, bootloaderét és rendszerpartícióját tilos módosítani.

11.1.8 Kiinduló állapot és konfiguráció

A laborhoz a következő rögzített konfigurációt használjuk: - Virtuális címtartomány: 32 bit (4 GiB). - Lapméret: 4 KiB. - Lapkönyvtár: 1024 bejegyzés, 32 bites bejegyzések. - Laptábla: 1024 bejegyzés, 32 bites bejegyzések. - Lapkönyvtár báziscíme: `0x0000_1000` (fizikai). - Teszt virtuális cím: `0x1234_5678`. - A lapkönyvtár 72. bejegyzése a `0x0000_2000` laptábla-címre mutat, Present bittel. - A laptábla 837. bejegyzése a `0x0003_F000` lapkeret-címre mutat, Present bittel, írható. - Elvárt fizikai cím: `0x0003_F678`.

A konfigurációt egy szöveges fájlban tároljuk, hogy a reprodukció egyértelmű legyen.

11.1.9 Build- és futtatási parancs

A szimulátort C nyelven írjuk, és egy virtuális gépen vagy konténerben fordítjuk. Például GNU/Linux környezetben:

```
# Fordítás
gcc -std=c11 -Wall -Wextra -O0 -g mmu_sim.c -o mmu_sim
# Futtatás
./mmu_sim
```

A fordítási beállításokat rögzíteni kell, mert a mérési eredmények ettől függhetnek. Az optimalizálás kikapcsolásával (`-O0`) a szimuláció determinisztikusabb, de a teljesítménymméréshez természetesen más beállítások is szükségesek.

11.1.10 Elvárt eredmény és mérési napló

A program kimenetének tartalmaznia kell: - A virtuális címet. - A lapkönyvtár-indexet, laptábla-indexet és offsetet. - A lapkönyvtár bejegyzés értékét. - A laptábla bejegyzés értékét. - A kiszámított fizikai címet. - A fordítási lépések számát (lapkönyvtár-olvasás, laptábla-olvasás).

Példa kimenet:

```
Virtcim: 0x12345678
PDI: 0x48, PTI: 0x345, Offset: 0x678
PDE: 0x00002001 (Present, TableBase=0x00002000)
PTE: 0x0003F003 (Present, RW, FrameBase=0x0003F000)
Fizicim: 0x0003F678
Fordítási lépések: 2 memóriaolvasás
```

A mérési naplóban rögzíteni kell a várt és a mért értéket. A várt fizikai cím `0x0003_F678`, a mért értéket a program kimenetéből vesszük. A mérőszám lehet a fordítási lépések száma, amely ebben a konfigurációban 2. Ez a szám a lapasztal-szintek számától függ, és a TLB nélküli esetben a memóriaolvasások számát jelenti.

11.1.11 Visszaállítás

Mivel a labor emulált környezetben fut, a visszaállítás a virtuális gép vagy konténer törlését jelenti. A hoszt rendszeren semmilyen módosítás nem történik. A forráskódot és a konfigurációs fájlt verziókezelőben tároljuk.

11.1.12 Gyakori hibák és azok elkerülése

A címfordítási feladatokban a leggyakoribb hibák a lapméret és az indexelés körül jelentkeznek.

- **Az offset hibás kiszámítása**: A lapméret 4 KiB, ami 12 bitet jelent. Ha valaki az offsetet 10 bitnek veszi, a lapon belüli eltolás téves lesz. Helyes megoldás: az offset az alsó 12 bit.
- **Az indexek felcserélése**: A PDI a felső 10 bit, a PTI a következő 10 bit. Ha valaki fordítva számol, a lapkönyvtár és a laptábla indexe felcserélődik, és a fordítás téves címet ad. Helyes megoldás: a cím felosztása a legfelső bitektől kezdve történik.
- **A bejegyzés fizikai címe és a lapkeret összekeverése**: A PTE-ben a lapkeret fizikai címe általában az alsó 12 bit nélkül szerepel, mert a lapkeret 4 KiB-os igazítású. Ha valaki a teljes PTE-értéket lapkeretként használja, a vezérlőbitek bekerülnek a fizikai címbe, ami hibát okoz. Helyes megoldás: a lapkeret címét a bejegyzés alsó 12 bitjének levágásával kell előállítani.
- **A Present bit figyelmen kívül hagyása**: Ha a lap nincs jelen, a fordításnak laphibát kell váltania. A szimulációban ezt explicit ellenőrizni kell, és a hibát jelezni kell. Helyes megoldás: a Present bit ellenőrzése minden szinten, és laphiba kiváltása hiány esetén.
- **A jogosultsági bitek elhanyagolása**: Az írási jogosultságot ellenőrizni kell, ha a hozzáférés írás. A User/Supervisor bitet is figyelembe kell venni a felhasználói módú hozzáféréseknél. Helyes megoldás: a hozzáférési típus és a jogosultsági bitek konzisztens ellenőrzése.

Ezek a hibák a tipikus félreértésekből adódnak, és a szimulációval könnyen felismerhetők.

11.1.13 Összefoglalás és önellenőrző kérdés

Ebben a fejezetben bevezettük a virtuális cím és a címfordítás alapfogalmait. Megkülönböztettük a logikai, virtuális és fizikai címet, bemutattuk az MMU és a lapasztal szerepét, és részletesen elemeztük a kétszintű lapasztal bejárását. A gyakorlati példán keresztül láttuk, hogyan bontható fel a virtuális cím, és hogyan állítható elő a fizikai cím. A labor egy reprodukálható MMU-szimulátor elkészítését írta elő, rögzített konfigurációval és mérési naplóval. A következő fejezet (Lapozás, szegmentálás és TLB) a lapozás, szegmentálás és TLB részleteire épít, felhasználva az itt bevezetett lapasztal-fogalmat.

Önellenőrző kérdés: Adott egy 32 bites virtuális cím, `0xABCD_EF01`, 4 KiB lapok, kétszintű lapasztal. Add meg a lapkönyvtár-indexet, a laptábla-indexet és az offsetet hexadecimális formában. Ellenőrizd, hogy a lapkönyvtár-index 10 bit, a laptábla-index 10 bit, az offset 12 bit hosszú. (Megoldás: PDI = `0x2AF`, PTI = `0x0DE`, Offset = `0xF01`. A PDI a 31–22 bit: `0xABCD_EF01` felső 10 bitje `0x2AF`, a PTI a 21–12 bit: `0x0DE`, az offset az alsó 12 bit: `0xF01`.)

A fejezetben bemutatott kódok és példák szemléltetésként szolgálnak, és nem helyettesítik a hardver specifikációjának pontos ismeretét. A valós rendszereken a címfordítás részletei architektúrafüggők, ezért a programozáshoz mindig az adott processzor dokumentációját kell alapul venni.

11.2 Lapozás, szegmentálás és TLB

11.2.1 1. A memóriaabsztrakció szükségessége

Az előző alfejezetben láttuk, hogy a folyamatok logikai címterei elszigeteltek, és a virtuális címeket fizikai címekre kell leképezni. A hardver és az operációs rendszer együttműködésének két klasszikus mechanizmusa alakult ki ennek a leképezésnek a megvalósítására: a **szegmentálás** és a **lapozás**. A modern rendszerek túlnyomó többsége a lapozást használja a processzorok hatékony hardveres támogatása miatt, a szegmentálás azonban történetileg és fogalmilag is fontos, mert a memóriavédelem és a címfordítás alapgondolata innen ered. Ebben az alfejezetben először a két mechanizmust vizsgáljuk meg külön-külön, majd megmutatjuk, hogyan épül fel egy többszintű lapasztal, és hogyan gyorsítja a hozzáférést a TLB. Végül a laphiba mechanizmusát tárgyaljuk, amely a következő alfejezetekben tárgyalt virtuális memória és demand paging alapját képezi.

11.2.2 2. Szegmentálás: a logikai felosztás

A szegmentálás alapötlete, hogy a program logikai egységeit – a kódszegmenst, az adatszegmenst, a veremszegmenst és a heapet – különböző, egymástól független memóriatartományokra képezzük le. A processzor ebben a modellben a virtuális címet két részből állítja össze: egy **szegmensazonosítóból** és egy **eltolásból** (offset). A szegmenshez tartozik egy **báziscím** és egy **határérték** (limit). A címfordítás során a hardver ellenőrzi, hogy az eltolás kisebb-e a határértéknél; ha igen, a fizikai címet a báziscím és az eltolás összegeként állítja elő. Ha az eltolás nagyobb vagy egyenlő, mint a határ, a processzor kivételt vált ki, amelyet az operációs rendszer általában a folyamat megszakításával kezel.

A szegmentálásnak van egy nagyon fontos előnye: a szegmensek mérete a program tényleges igényeihez igazítható. A kódszegmens pontosan akkora lehet, amekkora a lefordított utasításfolyam, a veremszegmens pedig dinamikusan növekedhet, ha a határértéket a kernel megfelelően beállítja. A védelem természetesen illeszkedik a logikai felosztáshoz: a kódszegmenst lehet olvasható és

végrehajtható, de nem írható, az adatszegmenst olvasható és írható, de nem végrehajtható jelöléssel ellátni.

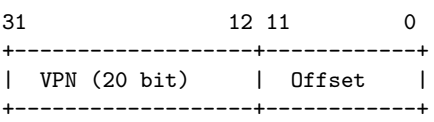
A modell azonban alapvető fizikai problémát vet fel: a szegmensek változó mérete miatt a fizikai memóriában **külső fragmentáció** lép fel. Tegyük fel, hogy három folyamatot töltünk be: A kódszegmense 300 KiB, B adatszegmense 200 KiB, C veremszegmense 500 KiB. Ha a memóriában sorban helyezkednek el, és B futás után felszabadul, 200 KiB szabad terület keletkezik, amelybe egy 600 KiB-os új szegmens nem fér be. A szabad területek darabokra szakadnak, és a rendszernek idővel tömörítenie kell a memóriát, ami drága és bizonyos esetekben lehetetlen. Ez a külső fragmentáció a szegmentálás legnagyobb gyakorlati hátránya, ezért a mai általános célú operációs rendszerek nem tiszta szegmentálást használnak.

Egy rövid számítási példa jól mutatja a mechanizmust. Legyen egy szegmens báziscíme `0x0010_0000`, határértéke pedig `0x0000_5000`. Ha a virtuális cím `0x0000_2000`, akkor az eltolás `0x2000`, ami kisebb a határnál, így a fizikai cím `0x0012_2000`. Ha viszont a virtuális cím `0x0000_5000`, az eltolás megegyezik a határral, tehát a hozzáférés kivételt eredményez. A hardver által kiváltott kivétel fontos szerepet játszik a biztonságban: a folyamat nem férhet hozzá a szegmens határán túli memóriához anélkül, hogy a kernel közbe ne lépne.

11.2.3 3. Lapozás: a rögzített méretű felosztás

A lapozás a szegmentálás külső fragmentációs problémáját úgy oldja meg, hogy a fizikai memóriát és a virtuális címteret is azonos méretű, rögzített méretű blokkokra osztja. A virtuális oldalt **lapnak** (page), a fizikai oldalt **keretnek** (frame) nevezzük. A lapok mérete a legtöbb mai architektúrán 4 KiB, de a x86-64 és az ARM processzorok támogatnak 2 MiB-os és 1 GiB-os nagy lapokat is, amelyeket később tárgyalunk.

A lapozásnál a virtuális cím két részre bontható: a **virtuális lapterületre** (Virtual Page Number, VPN) és az **eltolásra** (offset). A VPN azonosítja a lapot, az eltolás a lapon belüli bájtot. A fizikai cím ugyanilyen szerkezetű: **fizikai keretszám** (Physical Frame Number, PFN) és ugyanaz az eltolás. Mivel a lap és a keret mérete azonos, az eltolás bitjei nem változnak a fordítás során. Egy 32 bites címtérben 4 KiB-os lapokkal az eltolás 12 bit (`log2(4096) = 12`), a VPN pedig 20 bit. A virtuális cím felosztása tehát:



A VPN leképezését a **lapasztal** végzi. A lapasztal egy olyan adatszerkezet, amelynek minden bejegyzése – **lapasztal-bejegyzés** (Page Table Entry, PTE) – egy virtuális lap és egy fizikai keret közötti hozzárendelést tárol. A 32 bites x86-os rendszereknél egy PTE 32 bites, és a következő mezőket tartalmazza:

- **Present (P)** – 1, ha a lap a fizikai memóriában van; 0, ha nincs (laphibát okoz).
- **Read/Write (R/W)** – 1, ha a lap írható; 0, ha csak olvasható.
- **User/Supervisor (U/S)** – 1, ha felhasználói módban is elérhető; 0, ha csak kernel módban.
- **Accessed (A)** – a hardver 1-re állítja, amikor a laphoz hozzáférnek.
- **Dirty (D)** – a hardver 1-re állítja, amikor a lapra írnak.
- **PFN** – a fizikai keret számának felső 20 bitje.

Egy lehetséges 32 bites PTE szerkezete C-ben a következő:

```c
#include <stdint.h>

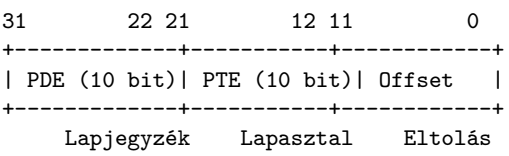
typedef struct {
    uint32_t present  : 1;  /* 0: lap nincs a memóriában */
    uint32_t writable : 1;  /* 0: csak olvasható */
    uint32_t user     : 1;  /* 0: csak kernel módban */
    uint32_t pwt      : 1;  /* write-through cache */
    uint32_t pcd      : 1;  /* cache disable */
    uint32_t accessed : 1;  /* hardver állítja be */
    uint32_t dirty    : 1;  /* hardver állítja be */
    uint32_t pat      : 1;  /* Page Attribute Table */
    uint32_t global   : 1;  /* TLB flush kihagyása */
    uint32_t avail    : 3;  /* kernel szabadon használhatja */
    uint32_t pfn      : 20; /* fizikai keretszám */
} pte_t;
```

A lapozás legfontosabb előnye, hogy **nincs külső fragmentáció**: a kernel bármely szabad keretet hozzárendelhet bármely virtuális laphoz, mert a keretek egyenrangúak. A hátránya ugyanakkor a **belső fragmentáció**: ha egy folyamat 12 300 bájtnyi memóriát igényel, 4 darab 4 KiB-os lapot kell lefoglalni, ami összesen 16 KiB, a lekerekítés miatt 4 KiB minusz 12 300 bájt, azaz körülbelül 3,7 KiB lesz elvesztegetett hely.

11.2.4 4. Többszintű lapasztal: a méretprobléma megoldása

A lapozás alapvető kérdése: mekkora a lapasztal? Egy 32 bites rendszerben 4 KiB-os lapokkal a VPN 20 bit, tehát a lapasztalnak `2^20 = 1 048 576` bejegyzése van. Ha egy PTE 4 bájt, a teljes lapasztal mérete 4 MiB. Ez az érték nem tűnik nagy problémának, amíg csak egy folyamatot futtatunk. De ha a rendszeren 100 folyamat aktív, az összesen 400 MiB-ot foglalna el kizárólag lapasztalakra. Ráadásul a legtöbb folyamat csak a címtér kis részét használja: a kód, az adat, a heap és a stack jellemzően néhány MiB-ot foglal el a 4 GiB-os címtérből. A fennmaradó rész teljesen üres, mégis fizikai memóriát foglalnának a hozzájuk tartozó lapasztal-bejegyzések.

A megoldás a **többszintű lapozás**. Ahelyett, hogy egyetlen hatalmas, folytonos lapasztalat tartanánk fenn, a virtuális címet több mezőre bontjuk, és a lapasztalat hierarchikus szerkezetbe szervezzük. A x86-os 32 bites klasszikus modellben két szint van:



A legfelső szint a **lapjegyzék** (Page Directory), amely 1024 bejegyzést tartalmaz. Minden bejegyzés egy második szintű lapasztal fizikai címére mutat. A második szintű lapasztal szintén 1024 bejegyzésből áll, és minden bejegyzése egy 4 KiB-os keretre mutat. A lapjegyzék báziscímét a processzor `CR3` regisztere tartalmazza.

Ez a szerkezet radikálisan csökkenti a memóriafelhasználást, mert a lapasztalakat csak akkor foglaljuk le, ha a folyamat ténylegesen használja a hozzájuk tartozó virtuális tartományt. Ha egy folyamat a kódját `0x0000_1000`-re tölti, a heap-je `0x0800_0000`-nél kezdődik, a stackje pedig `0xBFFF_F000`-re mutat, akkor csak azok a második szintű lapasztalak léteznek, amelyek ezeket a tartományokat fedik. A virtuális címtér nagy része üres, a lapjegyzék megfelelő bejegyzései pedig olyan PTE-k, amelyek `present` bitje 0, így a hozzáférés kivételt okoz.

A 64 bites rendszerekben további szintekre van szükség, mert a címtér sokkal nagyobb. A klasszikus x86-64 négyszintű lapozást használ:

- **PML4** (Page Map Level 4) – a legfelső szint.
- **PDPT** (Page Directory Pointer Table) – a második szint.
- **PD** (Page Directory) – a harmadik szint.
- **PT** (Page Table) – a legalsó szint.

Minden szint indexe 9 bit, az eltolás pedig 12 bit, így egy virtuális cím 48 biten ábrázolható: `9 + 9 + 9 + 9 + 12 = 48`. A nem használt felső bitek a címtér kanonikus kiterjesztésére szolgálnak. Az ARM64 és a RISC-V is hasonló hierarchikus sémákat alkalmaz, tipikusan három-négy szinttel. A többszintű lapozás tehát egy idő-visszaállítás: a memóriafelhasználást csökkenti, de a címfordítás lépéseinek a számát növeli. Egy négyszintű fordítás négy memória-hozzáférést igényelne a lapasztalak bejárásához, mielőtt a tényleges adatot elérnénk. Ezt a problémát oldja meg a TLB.

11.2.5 5. A TLB: címfordítási gyorsítótár

A TLB (Translation Lookaside Buffer) a memóriakezelő egység (MMU) része, egy nagyon gyors, asszociatív gyorsítótár, amely a nemrég használt virtuális lap-leképezéseket tárolja. Amikor a processzornak egy virtuális címet kell fizikaira fordítania, először a TLB-ben keres: ha a bejegyzés megtalálható, a fordítás **TLB-találat** (hit), és a fizikai cím elérhető anélkül, hogy a lapasztalat be kellene járni. Ha nincs bejegyzés, **TLB-tévesztés** (miss) következik be, és a hardver (vagy a kernel, architektúrától függően) elvégzi a lapasztal tényleges bejárását. A megtalált eredményt betölti a TLB-be, így a következő hozzáférés ugyanarra a lapra már közvetlenül eltalál.

A TLB tipikusan 32-1024 bejegyzésből áll, ami a több millió virtuális laphoz képest elenyésző. A működése mégis hatékony a **helyiség elve** (locality) miatt: a programok rövid időn belül ugyanazokat a memóriaterületeket érik el. A TLB találati aránya jól hangolt rendszerekben 99% fölött van.

A TLB kezelése a kontextusváltás során kritikus biztonsági kérdés. Ha a processzor egy másik folyamatra vált, a régi folyamat TLB-bejegyzései már nem érvényesek, mert a virtuális címek fizikai leképezése megváltozott. A legegyszerűbb megoldás a TLB **kiürítése** (flush), amely azonban teljesítménycsökkenéssel jár, mert a friss folyamatnak újra kell töltenie a bejegyzéseket. A modern processzorok ezért a TLB-bejegyzéseket egy **címcímke-azonosítóval** (ASID, Address Space Identifier) látják el. A TLB minden bejegyzése tartalmaz egy ASID-t, és a hardver csak akkor tekinti érvényesnek a találatot, ha az ASID megegyezik az aktuális folyamatéval. Ez lehetővé teszi, hogy a TLB-ben több folyamat bejegyzései is egyszerre jelen legyenek anélkül, hogy összekeverednének. A kernelnek azonban gondoskodnia kell arról, hogy ASID-okat ciklikusan újrahasznosítsa, és a régi bejegyzéseket kiürítse, ha az azonosító újrahasznosításra kerül.

11.2.6 6. Laphiba: amikor a lap nincs jelen

A laphiba (page fault) a lapozás egyik legfontosabb mechanizmusa. A fogalom kissé félrevezető, mert nem feltétlenül jelent hibát a szó hétköznapi értelmében. A laphiba valójában egy **szinkron kivétel**, amelyet a processzor akkor vált ki, ha a lapfordítás során valamilyen feltétel nem teljesül:

- A PTE `present` bitje 0, tehát a lap nincs a fizikai memóriában.
- A folyamat írni próbál egy csak olvasható lapra.
- A felhasználói módú kód kernel módú laphoz próbál hozzáférni.
- A többszintű lapasztal valamelyik közbenső szintjén a `present` bit 0.

Amikor laphiba következik be, a processzor megszakítja az aktuális utasítás végrehajtását, elmenti a folyamat állapotát (regiszterek, utasításszámláló), és átadja a vezérlést az operációs rendszer laphiba-kezelőjének. A kezelő megvizsgálja a hibát kiváltó virtuális címet, amelyet a processzor egy speciális regiszterben ad át (x86-on `CR2`, ARM64-on `FAR_EL1`), és a hiba okát. A kezelő ezután a következő döntéseket hozhatja:

1. **A hozzáférés érvénytelen:** a folyamat olyan címet próbált használni, amely nincs lefoglalva a címterében. Az operációs rendszer megszakítja a folyamatot, és `SIGSEGV` (segmentation fault) szignált küld neki.
2. **A hozzáférés érvényes, de a lap nincs jelen:** ez a demand paging alapja. A kernel lefoglal egy szabad keretet, betölti a lap tartalmát a háttértárról (például egy lemezről vagy hálózati meghajtóról), beállítja a PTE-t, és visszatér. A folyamat újra futtathatja az eredeti utasítást, amely most már sikeresen elvégzi a hozzáférést.
3. **A hozzáférés írási másolást igényel:** amikor több folyamat oszt meg egy lapot, és valamelyik írni próbál rá, a kernel másolatot készít a lapról, és a folyamat a másolatot írja. Ezt a mechanizmust a következő alfejezetekben részletesen tárgyaljuk.

A laphiba tehát nem feltétlenül a program hibája; gyakran a virtuális memória normális működésének a része. Amikor egy program először fér hozzá egy dinamikusan lefoglalt memóriaterülethez, a rendszer gyakran laphibát vált ki, hogy fizikai keretet rendeljen a virtuális laphoz. A program ebből semmit sem vesz észre, mert a kernel a hiba kezelése után visszaadja a vezérlést a megszakított utasításnak.

11.2.7 7. Labor: többszintű lapasztal bejárásának levezetése

Ez a labor egy kétszintű lapozási rendszer címfordításának kézi levezetését és QEMU-ban történő ellenőrzését mutatja be. A cél, hogy a résztvevő pontosan megértse a lapjegyzék, a lapasztal és a keret közötti kapcsolatot, és képes legyen a processzor lépéseit önállóan reprodukálni.

Konfiguráció: - Emulált hardver: QEMU (`qemu-system-x86_64 -cpu qemu64`), 32 MiB RAM. - Kernel: freestanding C program, amely egy inicializált kétszintű lapozási sémát állít be. - Fordítás: `gcc -m32 -ffreestanding -nostdlib -fno-pic -mno-red-zone -c kernel.c -o kernel.o && ld -m elf_i386 -T linker.ld kernel.o -o kernel.bin` - Hibakeresés: GDB csatlakoztatva a QEMU debugportjához.

A labor során egy kezdeti állapotot hozunk létre: a kernel betöltődik a fizikai memória `0x0010_0000` címére, a lapjegyzék báziscíme `0x0010_1000` lesz, és a kernel a `0xC000_0000` virtuális címet a `0x0010_0000` fizikai keretre képezi le. A feladat a `0xC000_1234` virtuális cím fordításának levezetése.

A levezetés lépései:

1. **Virtuális cím felbontása:** `0xC0001234` = `1100 0000 0000 0000 0001 0010 0011 0100`

   - PDE index: `1100000000` = `0x300` = 768
   - PTE index: `0000000001` = `0x001` = 1
   - Eltolás: `001000110100` = `0x234`

2. **Lapjegyzék bejegyzésének kiolvasása:** A lapjegyzék báziscíme `CR3`-ban található, amit `0x00101000`-re állítunk. A PDE címe: `CR3 + PDE_index * 4 = 0x00101000 + 0x300 * 4 = 0x00101000 + 0xC00 = 0x00101C00`. A bejegyzés értéke legyen `0x00200003`. A `present` bit 1, az írható bit 1, a PFN pedig `0x00200`. Ez azt jelenti, hogy a második szintű lapasztal a fizikai memória `0x00200 * 0x1000 = 0x00200000` címén található.

3. **Lapasztal bejegyzésének kiolvasása:** A PTE címe: `0x00200000 + PTE_index * 4 = 0x00200000 + 0x001 * 4 = 0x00200004`. A bejegyzés értéke legyen `0x00100007`. A `present` bit 1, az írható bit 1, a felhasználói bit 1, a PFN pedig `0x00100`. Ez a fizikai keret `0x00100 * 0x1000 = 0x00100000` címét jelöli.
4. **Fizikai cím előállítása:** A végső fizikai cím a keret báziscímének és az eltolásnak az összege: `0x00100000 + 0x234 = 0x00100234`.

Mérés és ellenőrzés: A fenti levezetést GDB-ben ellenőrizzük a futó kernelben. A GDB segítségével kiolvassuk a lapjegyzék és a lapasztal megfelelő bejegyzéseit, majd az eredményt összehasonlítjuk a levezetett értékkel.

A GDB-parancsok:

```
# Csatlakozás a QEMU-hoz
target remote localhost:1234

# Virtuális cím felbontása
p/x 0xC0001234

# Lapjegyzék bejegyzésének kiolvasása
x/wx 0x00101C00

# Lapasztal bejegyzésének kiolvasása
x/wx 0x00200004

# A fizikai cím tartalmának kiolvasása
x/wx 0x00100234
```

Az eredményt egy mérési naplóban rögzítjük:

- **Várt eredmény (levezetés alapján):**
  - PDE a `0x00101C00` címen: `0x00200003`
  - PTE a `0x00200004` címen: `0x00100007`
  - Fizikai cím: `0x00100234`
- **Mért eredmény (GDB kimenet alapján):**
  - Olvasott PDE: `0x00200003`
  - Olvasott PTE: `0x00100007`
  - Olvasott érték a `0x00100234` címen: `0xDEADBEEF`

A labor akkor sikeres, ha a mért és a várt értékek megegyeznek. A gyakorlat során fontos, hogy a mért és a várt értékeket mindig külön jelöljük, és soha ne mutassunk be nem ellenőrzött eredményt.

Visszaállítás: A labor QEMU-ban, eldobható image-en fut. A hoszt gép valódi memóriáját, lemezét és bootloaderét nem módosítja. A kernelkísérletet kizárólag izolált, emulált környezetben hajthatjuk végre; a kernelben történő hibakeresésnél a `panic`, a stack trace és a reprodukálható konfiguráció együtt szükséges. A GDB-vel történő memóriafeltárás kizárólag saját vagy engedélyezett emulált környezetben végezhető.

11.2.8 8. Gyakori hibák és azok elkerülése

A lapozás és a TLB működésének mélyebb megértése nélkül a kernelben könnyű hibákat ejteni. A leggyakoribb hibák közül kiemelünk néhányat:

- **Az eltolás és a keretszám összekeverése:** A kezdő fejlesztők gyakran a PFN-t közvetlenül a virtuális címhez adják, megfeledkezve arról, hogy a PFN-t meg kell szorozni a lapmérettel. A `PFN = 0x100` tehát `0x100000` fizikai címet jelent, nem `0x100`-at.
- **A present bit ellenőrzésének hiánya:** A lapasztal-bejegyzést nem elég elolvasni; mindig ellenőrizni kell a `present` bitet. Ha a bit 0, a PTE többi bitje nem érvényes, a PFN-t nem szabad használni, mert a kernel a bejegyzést egyéb célra is használhatja.
- **A TLB kiürítésének elmulasztása:** Ha a kernel módosítja a PTE-t (például beállítja a `present` bitet), a változás nem feltétlenül érvényesül azonnal, mert a TLB-ben egy régebbi bejegyzés maradhat. A PTE módosítása után a megfelelő TLB-bejegyzést ki kell üríteni (`invlpg` utasítás x86-on).
- **A lapméret és az eltolás határainak figyelmen kívül hagyása:** Egy 4 KiB-os lapon belül az eltolás 12 bit. Ha a kód 12 bitnél nagyobb eltolást számol, a cím átcsúszik a következő lapra.

11.2.9 Ellenőrző kérdések

1. Magyarázza el, miért nincs külső fragmentáció a lapozásnál, de belső fragmentáció igen.
2. Egy 32 bites rendszerben 4 KiB-os lapokkal egy folyamat 3 MiB kódot, 2 MiB heapet és 1 MiB stacket foglal le. Hány lapasztal-bejegyzés lesz `present` állapotban?
3. Mi történik, ha a TLB-ben található bejegyzést a kernel módosítja, de nem üríti a TLB-t?
4. Soroljon fel három olyan eseményt, amely laphibát okozhat.

Ez az alfejezet a lapozás és a szegmentálás alapvető mechanizmusait mutatta be, a lapasztal felépítését és a TLB szerepét, valamint a laphiba mechanizmusát. A következő alfejezetben a laphiba kezelésének „politikáját” vizsgáljuk meg részletesen: hogyan dönt az operációs rendszer arról, hogy melyik lapot kell betöltenie, és melyik lapot érdemes eltávolítania a fizikai memóriából.

11.3 Laphiba, demand paging és másolás

A laphiba az operációs rendszer memóriakezelőjének az a pontja, ahol a hardver által észlelt memória-hozzáférési kivétel találkozik a kernel által karbantartott virtuális memória-leírókkal. A laphiba nem feltétlenül hiba: a legtöbb esetben a késleltetett, igény szerinti lapbetöltés (*demand paging*) és a másolás-halasztó optimalizáció (*copy-on-write*) szándékosan idézi elő. Ebben a fejezetben a laphiba kezelését, a demand paging és a COW mechanizmusát, továbbá a laphiba költségének és a munkakészletnek az összefüggését tárgyaljuk. A Lapozás, szegmentálás és TLB fogalmait – címterek, lapozás, TLB – ismertnek tételezzük fel.

Laphiba mint hardverkivétel

Amikor a CPU egy virtuális címet akar lefordítani fizikai címre, a lapozó hardver a folyamat oldaltábláját járja be. Ha a bejegyzés hiányzik, vagy a hozzáférés jogosultsága nem felel meg a műveletnek, a CPU laphibát (*page fault*) vált ki. A kivétel megszakítja az aktuális utasítást, mielőtt az befejeződnék, így az utasítás később újraindítható. A hardver a hibás virtuális címet egy architektúra-specifikus regiszterbe menti: x86 rendszeren a `CR2`, RISC-V rendszeren a `stval` regiszter tölti be ezt a szerepet. A kernel laphibakezelője innen olvassa ki a címet.

A laphiba három fő okból keletkezhet:

1. **Nincs jelen a lap** (*not present*): a oldaltábla bejegyzés `present` bitje 0. Ez lehet valódi késleltetett betöltés (demand paging), swap-ből visszaolvasandó lap, vagy egyszerűen hibás, nem leképezett cím.
2. **Védelmi hiba** (*protection fault*): a lap jelen van, de a művelet nem engedélyezett, például írás egy csak olvasható lapra. A copy-on-write ezt a hibát használja fel.
3. **Foglalási vagy hardverhiba**: például a lapnak foglalt (*reserved*) bitje van, vagy a lapozó bejegyzés sérült. Ez rendszerint kernelhiba.

A laphiba kezelője a processz kontextusában fut, nem pedig megszakítás-kontextusban, mert a kezelés során szükség lehet alvásra: egy *major fault* esetén a lapot lemezről vagy swap-ből kell betölteni, ami akár több milliszekundumot is igénybe vehet. Ezért a laphibakezelő nem futhat atomi vagy megszakítási kontextusban, ellentétben például az időzítő megszakítás kezelőjével.

Laphibakezelő lépései

A kernel laphibakezelőjének bemenete: a hibás virtuális cím, a hozzáférés típusa (olvasás, írás, végrehajtás), a hibát okozó mód (felhasználói vagy kernel), valamint az, hogy a hiba felhasználói térből érkezett-e. A kimenete: a hiba kijavítása és az utasítás újraindítása, vagy szignál küldése a folyamatnak (általában `SIGSEGV` vagy `SIGBUS`).

Az alábbi pszeudokód a tipikus kezelési folyamatot mutatja:

```
laphiba_kezelo(hibas_cim, hozzaferes, user_mod):
    ha user_mod:
        ha hibas_cim >= TASK_SIZE:          // kernel cím felhasználói módból
            return SIGSEGV

    vma = kereses_a_vma_faban(processz, hibas_cim)
    ha vma == NULL:
        return SIGSEGV                      // nincs leképezett terület

    ha hozzaferes == IRAS_A_NEM_IRHATORA:
        ha vma.iras_engedelyezett == HAMIS:
            ha vma.COW_jelolt == IGAZ:
                return COW_kezeles(processz, vma, hibas_cim)
            return SIGSEGV                  // valódi védelmi hiba

    ha lap_nincs_jelen(vma, hibas_cim):
        ha vma.tamogato_fajl == NULL:
            return zero_lap_hozzarendeles(processz, vma, hibas_cim)
        else:
            return lap_betoltes_fajlbol(processz, vma, hibas_cim)

    return SIGSEGV                          // egyéb hiba
```

A `kereses_a_vma_faban` a folyamat virtuális memória-területeit (*VMA*, virtual memory area) leíró struktúrákban keres. A VMA tartalmazza a tartomány kezdetét, végét, jogosultságait, valamint a lapok forrását: anonim memória, fájl vagy eszköz. A laphibakezelő nem közvetlenül az oldaltáblából dönt, hanem a VMA-ból, mert a VMA írja le a leképezés szemantikáját, például hogy egy fájl mely tartománya van hozzárendelve.

Ha a cím egyetlen VMA-ba sem esik, a folyamat szegmentálási hibát kap. Ha a hozzáférés írás egy csak olvasható VMA-ra, a kernel először ellenőrzi, hogy a lap COW-jelölt-e. Ha igen, akkor a COW-kezelés következik; ha nem, akkor valódi `SIGSEGV`. Ha a lap nincs jelen, akkor demand paging: anonim esetben egy nullákkal feltöltött lapot rendel hozzá, fájlos esetben pedig a megfelelő blokkot beolvassa.

Demand paging: késleltetett lapbetöltés

A hagyományos, nem késleltetett betöltés egy futtatható program indításakor a teljes szegmenst – kódot, adatot, vermet – a memóriába olvasná. Ez lassú és pazarló. A *demand paging* ezzel szemben csak akkor hozza létre a fizikai lapot, amikor a program először hozzáfér hozzá. A folyamat kezdeti oldaltáblája rendszerint üres, vagy csak néhány lapot tartalmaz; a többi bejegyzés `present=0`.

A demand paging két alesetre bontható:

- **Anonim lapok** (heap, verem, `mmap MAP_ANONYMOUS`): nincs háttértároló. Amikor a folyamat először ír vagy olvas egy ilyen lapot, a kernel egy fizikai keretet allokál, nullázza, és a oldaltáblába bejegyzi. Ha a hozzáférés olvasás, és a lap még nincs meg, a kernel egy globális, csak olvasható *nulla lapot* (*zero page*) rendelhet hozzá. Az első írási kísérlet ezen a lapon COW laphibát vált ki, és ekkor jön létre az egyedi, nullázott fizikai lap.
- **Fájl alapú lapok** (kód, `mmap` fájl): a lap forrása egy fájl egy adott blokkja. A laphiba kezelője meghatározza a fájlbeli offsetet, majd a blokkot beolvassa a lapgyorsítótárba. A `mmap` szemantikája szerint a módosítások visszaírhatók a fájlba, vagy ha a leképezés `MAP_PRIVATE`, akkor COW használatos.

A laphiba típusa a költség szempontjából két kategóriába sorolható. A *minor fault* azt jelenti, hogy a hiba kiszolgálása nem igényel lemez I/O-t: a lap már a memóriában van (például a lapgyorsítótárban, a nulla lapon vagy egy COW miatt), csak a oldaltábla-bejegyzést kell frissíteni. A *major fault* lemezről vagy swap-ből való beolvasást igényel, ami nagyságrendekkel drágább.

A demand paging előnyei: gyorsabb folyamatindítás, kisebb induló memórialábnyom, több egyidejű folyamat. Hátrányai: kiszámíthatatlan késleltetés az első hozzáféréseknél, valamint a laphibák futás közbeni többletköltsége. Beágyazott, valós idejű rendszerekben gyakran inkább előre lefoglalt, zárolt lapokat használnak, hogy a laphiba soha ne jelenjen meg a kritikus szakaszban.

Copy-on-write (COW)

A másolás írásra (*copy-on-write*) a memóriakezelés egyik legfontosabb optimalizációja. A `fork` rendszerhívás során a gyermekfolyamat elvileg megkapja a szülő teljes címterének másolatát. Ha ezt fizikailag lemásolnánk, a `fork` költsége a folyamat memóriaméretével arányos lenne, és sokszor felesleges, mert a gyermek azonnal `exec`-et hív. A COW ehelyett a lapokat megosztja a szülő és a gyermek között, és mindkét oldaltáblában csak olvashatóvá teszi őket. Amint valamelyik folyamat írni próbál egy ilyen lapra, a CPU védelmi laphibát vált ki. A kernel COW-kezelője ekkor:

1. felismeri, hogy a lap COW-jelölt;
2. ha a lap referenciaszámlálója (`refcount`) 1, akkor egyszerűen írhatóvá teszi az oldaltáblát;
3. ha a `refcount` > 1, akkor új fizikai lapot allokál, átmásolja a tartalmat, csökkenti a régi lap referenciaszámlálóját, és az író folyamat oldaltábláját az új lapra állítja írási joggal.

A lépést szemléltető egyszerű pszeudokód:

```
COW_kezeles(processz, vma, hibas_cim):
    lap = lekerdez_a_page_tablabol(processz, hibas_cim)
    ha lap.refcount == 1:
        page_table_bejegyzes.iras = IGAZ
        TLB_invalidalas(hibas_cim)
        return UJRAINDITAS
    uj_lap = fizikai_lap_allokalasa()
    masolas(uj_lap, lap)
    lap.refcount -= 1
    page_table_bejegyzes.lap = uj_lap
    page_table_bejegyzes.iras = IGAZ
```

```
    TLB_invalidalas(hibas_cim)
    return UJRAINDITAS
```

A COW nem csupán a `fork` sajátja. Ugyanez a mechanizmus működik a `MAP_PRIVATE` leképezésű fájloknál: a fájl lapjai csak olvashatóak, az első írásra a folyamat saját, nem fájlba visszaírandó lapot kap. A nulla lap is COW: az első írás váltja ki a valódi, nullázott lap létrehozását.

A COW előnye a `fork` esetében különösen látványos. Egy több száz MiB memóriát használó folyamat `fork`-ja COW-val ezredmásodpercek alatt lezajlik, mert nem kell a memóriát másolni; csak a oldaltáblákat kell bejárni és a bejegyzéseket írásvédetté tenni. A tényleges másolás akkor történik, amikor valamelyik folyamat ténylegesen ír. A `vmstat` vagy a `getrusage` ilyenkor a COW laphibákat minor faultként számolja el.

Laphiba költsége és munkakészlet

A laphiba kezelésének költsége két részből áll: a hardverkivétel és a kernelbeli feldolgozás fix költségéből, valamint a hiba típusától függő változó költségből. A minor fault jellemzően néhány mikroszekundum modern processzoron, mert csak a oldaltábla-frissítés és a TLB-invalidálás történik. A major fault ezzel szemben lemez I/O-t igényel: NVMe SSD-n 5–20 ms, hagyományos HDD-n 5–15 ms vagy több is lehet, ha a lemez várakozik. A pontos értékek hardver-, kernel-, fájlrendszer- és terhelésfüggők; a mérési laborban ezeket külön rögzíteni kell.

A laphiba költsége a folyamat munkakészletének (*working set*) fogalmával hozható összefüggésbe. A munkakészlet Denning definíciója szerint azoknak a lapoknak a halmaza, amelyekre a folyamat az utolsó Δ időablakban hivatkozott:

$$W(t, \Delta) = \{\text{lap } L \mid L\text{-re a } [t-\Delta, t] \text{ intervallumban hivatkozás történt}\}.$$

Amíg a munkakészlet belefér a fizikai memóriába, a laphibák többsége minor fault: a lapok a lapgyorsítótárban vagy a szabad listán vannak, és a betöltés gyors. Amikor a teljes munkakészlet meghaladja a rendelkezésre álló fizikai keretek számát, a rendszer folyamatosan cserél: egy lap betöltése kiszorít egy másik, éppen használt lapot, amely a következő hivatkozáskor ismét major faultot okoz. Ezt a jelenséget *thrashing*nek nevezzük. A thrashing tünete a laphiba-ráta drasztikus növekedése, a CPU kihasználtság csökkenése és a lemez I/O telítettsége.

A munkakészlet becslésére a rendszer a laphiba-statisztikákat és a folyamatonkénti memóriahasználatot figyeli. A demand paginget használó általános célú rendszerekben a munkakészlet időben változik; a scheduler és a memóriakezelő együtt próbálja elkerülni a thrashinget, például a lapcsere-algoritmusok (Lapcsere-algoritmusok: FIFO, LRU és Clock) és a folyamatzárolás segítségével. Valós idejű rendszerekben ezzel szemben a laphibákat gyakran teljesen letiltják a kritikus utasítások alatt, mert a major fault kiszámíthatatlan késleltetést okozna.

Gyakorlati példa: COW és demand paging mérése

A következő C program illusztrálja a demand paging és a COW laphibákat. A kód szemléltetés, nem végrehajtandó utasítás éles rendszeren; a futtatást emulált környezetben kell elvégezni.

```c
#define _GNU_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <sys/mman.h>
#include <sys/wait.h>
#include <sys/resource.h>
#include <unistd.h>

static long minor_faults(void) {
    struct rusage ru;
```

```
    getrusage(RUSAGE_SELF, &ru);
    return ru.ru_minflt;
}

int main(int argc, char **argv) {
    size_t mib = (argc > 1) ? (size_t)atol(argv[1]) : 256;
    size_t len = mib * 1024 * 1024;
    long page_size = sysconf(_SC_PAGESIZE);
    char *p = mmap(NULL, len, PROT_READ | PROT_WRITE,
                   MAP_PRIVATE | MAP_ANONYMOUS, -1, 0);
    if (p == MAP_FAILED) { perror("mmap"); return 1; }

    long before = minor_faults();
    for (size_t off = 0; off < len; off += page_size)
        p[off] = 1;                         // demand zero page + COW
    printf("Demand paging minor faults: %ld\n", minor_faults() - before);

    pid_t pid = fork();
    if (pid < 0) { perror("fork"); return 1; }

    if (pid == 0) {                         // gyermek
        long child_before = minor_faults();
        for (size_t off = 0; off < len; off += 2 * page_size)
            p[off] = 2;                     // COW: írás, lap másolása
        printf("Child COW minor faults: %ld\n",
               minor_faults() - child_before);
        _exit(0);
    }

    wait(NULL);
    return 0;
}
```

A program lefoglal egy `MAP_ANONYMOUS | MAP_PRIVATE` területet, amely kezdetben egyetlen fizikai lapot sem foglal. Az első ciklus minden 4 KiB-os oldal első bájtját írja, ezzel anonim demand paging laphibát vált ki. Az első írás a nulla lapról COW-t is jelent, így a minor faultok száma a lefoglalt lapok számával közel azonos. A `fork` után a gyermek minden második oldalt módosít; ezek a lapok COW-jelöltek, így a gyermekben a módosított lapok számával megegyező minor fault keletkezik. A mérés `getrusage`-on alapul, amely a `ru_minflt` mezőben adja vissza a minor faultok számát.

Tipikus hibák és megoldások

A laphibakezelő és a COW implementációja számos buktatót rejt. A leggyakoribb hibák:

- **Hiányzó VMA-ellenőrzés**: ha a kezelő nem ellenőrzi, hogy a hibás cím a folyamathoz tartozó VMA-ba esik-e, akkor egy felhasználói folyamat kernel címre hivatkozhat, vagy egy hibás mutatóval olvashat ki memóriát. A helyes megoldás a VMA-fában keresés, és annak hiányában `SIGSEGV` küldése.
- **COW refcount verseny**: a referenciaszámláló csökkentése és növelése nem atomi, így egyidejű `fork` és lap-felszabadítás esetén a lap kétszer szabadulhat fel, vagy elérhetetlenné válhat. A megoldás atomi műveletek vagy per-CPU lap-láncolt struktúrák használata.
- **Nulla lap helytelen írhatóvá tétele**: ha a kernel a zero page-et írhatóvá teszi ahelyett, hogy COW-t hajtana végre, a folyamatok egy közös lapot írnak, és adat szivárog át. A zero page mindig csak olvasható; az írásnak mindig új lap allokálását kell eredményeznie.
- **TLB-invalidálás elmaradása**: a oldaltábla frissítése után a régi, írásvédett bejegyzés a TLB-ben marad, így a következő írás továbbra is védelmi hibát okoz. A megoldás a TLB-shootdown vagy legalább a lokális TLB-bejegyzés invalidálása. Többmagos rendszeren a COW-jelölt lap írhatóvá tétele minden CPU-ra kiterjedő TLB-invalidálást igényel.

- **Alvás nem megengedett kontextusból**: ha a laphibakezelő egy megszakításkezelőből hívódik, a major fault alvása kernel oops-hoz vezet. A laphibakezelőt mindig processzkontextusból kell hívni, és a megszakítási kontextusból nem szabad lemez I/O-t kezdeményezni.

A fenti ellenőrzési pontok közül a VMA-keresés és a COW-refcount kezelése a memóriabiztonság sarokköve. A kernelkódban minden laphiba-ágat naplózni kell a hibás címmel, a hozzáféréssel és az okkal, hogy a hibakeresés reprodukálható legyen.

Labor: laphiba-forgatókönyv készítése

A labor célja, hogy reprodukálható módon mérjük a demand paging és a COW által okozott laphibákat, és a mért eredményeket a várt értékekkel összehasonlítsuk. A mérés emulált környezetben, QEMU-ban fut, a hoszt valódi lemezét nem módosítjuk.

Kiinduló állapot és konfiguráció

- Hoszt: tetszőleges Linux, QEMU 8.0 vagy újabb (verzió rögzítendő).
- Vendég: Linux kernel 6.6 LTS, `x86_64`, 1 vCPU, 512 MiB RAM.
- Kernel parancssor: `console=ttyS0 root=/dev/vda rw transparent_hugepage=never`.
- A vendégen a swap ki van kapcsolva (`swapoff -a`), hogy a major faultok ne swapből, hanem fájlból jöjjenek, ha lesznek.
- A vendégen a `gcc` és a `musl` vagy `glibc` development csomag telepítve.

Build parancs

```
gcc -O2 -o lab_pagefault lab_pagefault.c
```

Futtatási parancs

A vendégen belül:

```
./lab_pagefault 256
```

A futtatás `256` MiB anonim memóriát foglal, demand paginget és COW-t mér. A mérést háromszor ismételjük, és a mediánt jegyezzük fel.

Mérési napló (sablon)

```
Időpont: 2025-05-01 10:00:00
QEMU: 8.2.1
Vendég kernel: 6.6.21
CPU: 1 vCPU, host: AMD Ryzen 5 5600X
RAM: 512 MiB
THP: transparent_hugepage=never
Swap: off
Lapméret: 4096 B

Futás 1:
  Demand paging minor faults (mért): 65530
  Várt demand paging minor faults:      65536
  Gyermek COW minor faults (mért):      32765
  Várt gyermek COW minor faults:        32768
Futás 2: ...
Futás 3: ...
Medián demand paging minor faults: ...
Medián COW minor faults: ...
```

A várt érték a lefoglalt méret és a lapméret hányadosa: $256 \cdot 1024^2 / 4096 = 65536$ lap. A mért érték ettől néhány lappal eltérhet, mert a kernel a program indításakor bizonyos lapokat már lefoglalhat, vagy a `mmap` belső struktúrái is foglalnak. A COW esetében a gyermek minden második oldalt írja, tehát a várt COW laphibák száma 32768. A mért és várt érték külön oszlopban szerepel, ahogy a minőségbiztosítási konvenció előírja.

Visszaállítás

A QEMU vendég lemezképe eldobható. A mérés után a vendég leáll, a lemezkép törölhető vagy újra klónozható. A hoszt rendszer nem változik, bootloaderét és partícióit nem érintjük.

Önellenőrző kérdés

Miért okoz a `fork` utáni első írás laphibát, és miért nem szegmentálási hibát? A válasz: mert a COW-jelölt lapok az oldaltáblában írásvédettként szerepelnek, de a VMA leírása megengedi az írást. A laphibakezelő felismeri a COW-t, lemásolja a lapot, írhatóvá teszi, és újraindítja az utasítást, ezért a folyamat nem kap `SIGSEGV`-t.

A demand paging és a COW a virtuális memóriakezelés két alappillére. A laphiba az a mechanizmus, amely ezeket futás közben összeköti a hardverrel. A fejezetben bemutatott kezelői lépések, a COW-logika és a munkakészlet-fogalom alapozza meg a Memóriavédelem, ASLR és izoláció memóriavédelmi és izolációs témáját, valamint a későbbi lapcsere-algoritmusok tárgyalását.

11.4 Memóriavédelem, ASLR és izoláció

A memóriavédelem és a címterek izolációja az operációs rendszer egyik legfontosabb biztonsági alapköve. A Laphiba, demand paging és másolás alfejezetben tárgyalt laphiba, demand paging és copy-on-write mechanizmusok csak akkor működnek biztonságosan, ha a hardver minden memória-hozzáférést jogosultság szerint ellenőriz, és ha a kernel, valamint a futó folyamatok címtartományai megfelelően el vannak választva. Ebben a fejezetben azt vizsgáljuk, hogyan válnak a laptáblák védelmi eszközzé, mit jelent a read/write/execute jogosultsági hármas, miért kulcsfontosságú a nem végrehajtható (NX) bit, és hogyan nehezíti a támadásokat az ASLR és a stack canary. A fejezetet egy reprodukálható labor zárja, amelyben egy emulált Linux környezetben ellenőrizzük a memóriavédelmi beállításokat, és megmérjük az ASLR által bevezetett címváltozást.

11.4.1 1. Lapjogosultságok és a hardveres memóriavédelem

A lapozás nem csupán a virtuális és a fizikai cím közötti leképezést valósítja meg, hanem minden egyes lapra vonatkozóan védelmi metaadatokat is tárol. A lapbejegyzés (PTE, page table entry) jellemzően a következő védelmi biteket tartalmazza:

- **P (present)**: a lap fizikai memóriában van-e; ha nincs, laphiba keletkezik.
- **R/W (read/write)**: a lap írható-e. Olvasható lapokon az írás laphibát vált ki.
- **U/S (user/supervisor)**: a lap elérhető-e felhasználói módban, vagy csak a kernel (supervisor) férhet hozzá.
- **X/NX (execute/no-execute)**: a lap végrehajtható-e. Ez a bit különösen fontos a kódinjektálásos támadások ellen.
- **A (accessed)**, **D (dirty)**: a laphoz való hozzáférés, illetve írás nyomon követése.

A hardver minden memória-hozzáféréskor – legyen az utasításlekérés vagy adatművelet – ellenőrzi a virtuális címet leképező PTE jogosultsági bitjeit. Ha a hozzáférés sérti a jogosultságot, a CPU kivételt vált ki, a vezérlés a kernelben lévő kivételkezelőhöz kerül, és a kernel dönt arról, hogy a hibát hogyan kezeli. A leggyakoribb válasz a processz SIGSEGV jelzése, de bizonyos esetekben a kernel maga is szándékosan használ védelmi hibákat, például a copy-on-write vagy a stack növesztésének megvalósításához.

A read/write/execute hármas azt fejezi ki, hogy egy laphoz a processz csak a megengedett módon férhet hozzá. Egy tipikus felhasználói folyamat memóriájában látható a minta:

Szegmens	Jogosultság	Magyarázat
`.text`	R-X	A kód olvasható és végrehajtható, de nem írható
`.rodata`	R–	A konstans adat csak olvasható
`.data`, `.bss`	RW-	A globális és statikus változók írhatók, de nem futtathatók
stack, heap	RW-	A folyamat saját adatterületei nem futtathatók
kernel címterület	–S (csak kernel)	A user módban nem elérhető

A **W^X** elv (write XOR execute) azt mondja ki, hogy egy lap ne legyen egyszerre írható és végrehajtható. Ez meggátolja, hogy egy támadó saját kódot írjon a memóriába, majd azt végrehajtsa. A fordító és a linkerszerkesztő ennek megfelelően úgy rendezi el a folyamat szegmenseit, hogy a kódszegmensek soha ne legyenek írhatók, az adatszegmensek pedig soha ne legyenek futtathatók. A JIT-fordítók (például JavaScript-motorok) kivételt képeznek, mert futás közben kódot generálnak, de ott is jellemző gyakorlat, hogy a generált kódot először írható, de nem futtatható lapra írják, majd `mprotect`-tel átállítják R-X jogosultságra.

A **NX bit** (no-execute) külön hardveres védelem a kódinjektálás ellen. Történetileg az x86 architektúrán sokáig nem volt ilyen bit, ezért a támadók a stackre vagy a heape helyezett shellcode-ot közvetlenül végrehajthatták. Az AMD és az Intel később bevezette az NX (illetve XD) bitet, az ARM az XN bitet, a RISC-V pedig a PTE X bitjének hiányát használja ugyanerre. A kernel a folyamatok indításakor a stacket, a heapet és az adatszegmenseket NX bit segítségével nem futtathatóvá teszi.

A lapbejegyzések jogosultsági bitjei nemcsak a felhasználói folyamatok védelmét szolgálják, hanem a kernel saját címterét is izolálják. A kernel lapjai jellemzően supervisor szintűek, így a felhasználói módú kód nem férhet hozzájuk még akkor sem, ha ismeri a virtuális címüket. Két további hardveres mechanizmus, a **SMEP** (Supervisor Mode Execution Prevention) és a **SMAP** (Supervisor Mode Access Prevention) megakadályozza, hogy a kernel a felhasználói térből származó mutatókat közvetlenül használja. SMEP esetén a kernel nem hajthat végre felhasználói címre mutató ugrást, SMAP esetén pedig nem olvashat vagy írhat felhasználói címeket a speciális `stac`/`clac` utasítások nélkül. Ez a védelem kiegészíti a kernel és a user tér közötti hagyományos lapjogosultsági elválasztást.

11.4.2 2. Címterek izolációja

A címterek izolációja azt jelenti, hogy minden folyamat saját, független virtuális címtartományt kap, és egyik folyamat sem férhet hozzá közvetlenül a másik memóriájához, hacsak a kernel explicit módon nem engedélyezi. A folyamatváltás során a kernel a processz PCB-jében tárolt

lapbejegyzés-tábla gyökerének címét tölti be a CPU megfelelő regiszterébe (x86-64: CR3, RISC-V: satp). Ettől kezdve a hardver az adott folyamat lapbejegyzéseit használja, így a virtuális címek fizikai címre való leképezése is folyamatonként eltérő.

A folyamatok közötti izoláció egyik következménye, hogy két különböző folyamat ugyanazon a virtuális címen saját, egymástól független adatot láthat. Ez az absztrakció teszi lehetővé, hogy a fordító és a linkerszerkesztő minden folyamatot ugyanolyan elrendezéssel fordítson, a kernel pedig minden folyamathoz külön fizikai kereteket rendeljen. A Laphiba, demand paging és másolás alfejezetben tárgyalt copy-on-write mechanizmus éppen ezt az izolációt használja ki: a fork után a szülő és a gyermek ugyanazokat a fizikai lapokat látja, de csak olvashatóan; amint valamelyik írni akar, a kernel másolatot készít.

A kernel címterének izolációja a folyamatokétól eltérő elven alapul. A Linux kernel a címterének nagy részét minden folyamat lapbejegyzés-táblájába beilleszti, de supervisor jogosultsággal. A felhasználói folyamat nem éri el ezeket a lapokat, viszont rendszerhíváskor a CPU átvált supervisor módba, és a kernel a saját címterében folytatja a végrehajtást anélkül, hogy lapbejegyzés-táblát kellene váltania. Ez gyors rendszerhívásokat tesz lehetővé, de a kernelnek gondoskodnia kell arról, hogy a felhasználói mutatókat soha ne használja közvetlenül. Ezért a Linux kernel a felhasználói adatok másolására a `copy_from_user` és `copy_to_user` függvényeket használja:

```
/* Linux kernel stílusú szemléltető példa, nem éles rendszerre való kód. */
int example_read(struct file *file, char __user *buf, size_t len)
{
    char tmp[64];

    if (len > sizeof(tmp))
        return -EINVAL;

    /*
     * A copy_from_user ellenőrzi, hogy a user cím érvényes-e,
     * és biztonságosan másol a kernel pufferbe. Hibánál -EFAULT.
     */
    if (copy_from_user(tmp, buf, len))
        return -EFAULT;

    /* A kernel itt már a saját, megbízható pufferén dolgozik. */
    return process_data(tmp, len);
}
```

A példában a `__user` annotáció a Sparse eszköz számára jelzi, hogy a mutató felhasználói címet tartalmaz. Ez a statikus elemzés egy fontos védelmi réteg: a kernelkódban tilos közvetlenül dereferálni ilyen mutatót. A futásidejű ellenőrzést a `copy_from_user` végzi, amely a felhasználói cím tartományát is ellenőrzi, és laphiba vagy jogosultságsértés esetén hibakódot ad vissza. Ez a minta a kernel és a user tér közötti határvonal biztonságos átlépésének alapja.

11.4.3 3. ASLR: a címek kiszámíthatóságának csökkentése

A memóriavédelem eddigi eszközei megakadályozzák, hogy a folyamatok közvetlenül hozzáférjenek egymás memóriájához, vagy hogy saját adatterületükön kódot futtassanak. A támadók azonban gyakran nem saját kódot töltenek be, hanem a már meglévő kódból állítják össze a támadást. A klasszikus **ret2libc** vagy **ROP** (return-oriented programming) támadásokban a támadó a folyamatban meglévő, végrehajtható lapokon található utasítássorozatokhoz ugrik. Ezeknek a támadásoknak a feltétele, hogy a támadó ismerje a kód- és adatszegmensek címét.

Az **ASLR** (Address Space Layout Randomization) ezt a feltételezést nehezíti meg azáltal, hogy minden folyamat indulásakor véletlenszerű eltolást rendel a fontos memóriaterületekhez. A cél az,

hogy a támadó ne tudja előre megmondani, hová kerül a stack, a heap, a dinamikusan betöltött könyvtárak, a program saját szegmensei vagy a `mmap` által leképezett területek. Az ASLR akkor működik jól, ha a folyamat címtér-elrendezési entrópiája elég nagy, és ha a támadó nem tudja a címeket más módon kiszivárogtatni.

A Linux kernelben az ASLR viselkedését a `/proc/sys/kernel/randomize_va_space` érték szabályozza. A pontos szemantika kernelverziótól függ, ezért mindig ellenőrizni kell az adott rendszeren. A tipikus értékek:

- `0`: ASLR kikapcsolva,
- `1`: a stack és a `mmap` régió véletlenített,
- `2`: a heap is véletlenített (ez a gyakori alapértelmezés).

A program futtatható állományának is fel kell készülnie a véletlen címekre. Ha a program **PIE** (Position Independent Executable) formátumú, akkor a kernel tetszőleges alapcímre töltheti be a kódszegmenseit is. A PIE nélküli, hagyományos statikus címre linkelt program esetén a főprogram szegmensei nem véletleníthetők, ami csökkenti az ASLR hatékonyságát. A dinamikus linker (`ld.so`) és a megosztott könyvtárak szintén PIE-ként készülnek.

Egy egyszerű C programmal megfigyelhetjük az ASLR hatását:

```c
/* layout_info.c - szemléltető program ASLR méréshez, emulált környezetben. */
#include <stdio.h>
#include <stdlib.h>
#include <dlfcn.h>

int main(void)
{
    int stack_var;
    void *heap_ptr = malloc(16);
    void *libc_addr = dlsym(RTLD_NEXT, "printf");
    Dl_info info;

    dladdr((void *)main, &info);

    printf("stack:            %p\n", (void *)&stack_var);
    printf("heap:             %p\n", heap_ptr);
    printf("main (PIE base):  %p\n", (void *)main);
    printf("libc printf:      %p\n", libc_addr);
    printf("linker base:      %p\n", info.dli_fbase);

    free(heap_ptr);
    return 0;
}
```

A program a stack egy lokális változójának címét, a heap kezdetét, a `main` függvény címét, a libc `printf` címét és a dinamikus linker alapcímét írja ki. Ha ASLR be van kapcsolva, minden futtatásnál más-más értékeket kapunk. Ha kikapcsoljuk, az értékek ismétlődnek. A PIE nélküli futtatható esetén a `main` címe állandó marad, a többi szegmens viszont véletlenszerű lehet.

Az ASLR nem végleges védelem, hanem a támadás költségét növelő réteg. Ha a támadó egy másik sérülékenységen keresztül ki tudja szivárogtatni a címeket, az ASLR elveszti hatékonyságát. A 32 bites rendszereken az entrópia alacsony, ezért brute-force próbálkozás is lehetséges. A 64 bites rendszereken a címtartomány nagy, de a kernel a lapok igazítási követelményei miatt nem használja ki a teljes 64 bites tartományt; a tényleges entrópia jellemzően 28–40 bit körül van, architektúra- és kernelverzió-függő. Ezért a gyakorlatban az ASLR-t mindig más védelmekkel együtt alkalmazzák, mint például a nem végrehajtható adatterületek, a stack canary és a RELRO (read-only relocations).

11.4.4 4. Stack canary: a visszatérési cím őrszem

A stack canary egy szoftveres, fordító által beillesztett védelem a stack puffertúlcsordulások észlelésére. A klasszikus stack buffer overflow támadás során a támadó egy lokális tömböt ír túl annyira, hogy a függvény visszatérési címét is felülírja. A canary célja, hogy ezt a felülírást még a visszatérés előtt észlelje.

A fordító a védett függvények prologusában egy véletlen értéket, a **canary**-t helyezi el a stacken a lokális változók és a mentett frame pointer, valamint a visszatérési cím közé. A függvény epilógusában, közvetlenül a visszatérés előtt ellenőrzi, hogy a canary értéke változatlan-e. Ha a puffer túlcsordult, a canary felülíródik, az ellenőrzés meghiúsul, és a program leáll (általában `__stack_chk_fail` hívásával), ahelyett, hogy a hamis visszatérési címre ugrana.

A canary értéke lehet processz-szintű véletlen érték, vagy terminátor canary, amely olyan bájtokat tartalmaz, amelyek a karakterlánc-műveleteket megakadályozzák. A GCC és a Clang például `-fstack-protector`, `-fstack-protector-strong` vagy `-fstack-protector-all` kapcsolóval aktiválja a védelmet. A `strong` változat nem csak azokat a függvényeket védi, amelyekben karaktertömbök vannak, hanem azokat is, amelyekben lokális tömbök, struktúrák vagy pointerek vannak.

A következő x86-64 pszeudokód a canary elhelyezését és ellenőrzését mutatja:

```
# Szemléltető pszeudokód, x86-64 ABI (nem végrehajtandó éles gépen).
# A canary a processz indításakor generált véletlen érték, általában
# a fs szegmens egy rejtett területén tárolva.

függvény_prologus:
    sub  rsp, 0x40                ; hely a lokális változóknak
    mov  rax, fs:[0x28]           ; canary betöltése
    mov  [rsp + 0x38], rax        ; canary a lokális puffer és az
                                  ; elmentett return address közé

    ; ... függvénytörzs ...

függvény_epilógus:
    mov  rax, fs:[0x28]
    cmp  [rsp + 0x38], rax        ; canary ellenőrzése
    jne  __stack_chk_fail         ; ha eltér, leállás
    add  rsp, 0x40
    ret
```

A canary nem akadályozza meg a puffertúlcsordulást, csupán észleli azt. Ha a támadónak lehetősége van kiolvasni a canary értékét (például egy formátumstring-sérülékenységen vagy egy másik out-of-bounds olvasási hibán keresztül), akkor a túlcsordulás során a canaryt is felülírhatja a helyes értékkel, így a védelem kijátszható. A canary tehát egy újabb védelmi réteg, nem önálló megoldás.

A stack canary mellett a modern fordítók és operációs rendszerek további szoftveres védelmet is alkalmaznak:

- **RELRO** (Read-only Relocations): a GOT (Global Offset Table) egy részét csak olvashatóvá teszi, nehezítve a függvénypointer-felülírásokat.
- **FORTIFY_SOURCE**: a libc karakterlánc- és memóriaműveleteinek fordítási idejű ellenőrzése.
- **CFI** (Control Flow Integrity): a közvetett ugrások céljának ellenőrzése.
- **Shadow stack** (hardveres visszatérési cím védelem): a CPU külön, nem írható stacket tart a visszatérési címeknek.

Ezek közül a fejezet fókusza a memóriavédelem és a címtér-izoláció, ezért a további hardveres védelmeket csak vázlatosan említjük.

11.4.5 5. Tipikus hibák és helyes tervezési minták

A memóriavédelem és az izoláció tervezésekor a leggyakoribb hibák általában a jogosultságok túl széles megadásából vagy a védelmi rétegek kihagyásából fakadnak. A hibákat és a helyes megoldásokat az alábbi táblázat foglalja össze:

Hiba	Következmény	Helyes megoldás
RWX lap engedélyezése (W^X megsértése)	Kódinjektálás és végrehajtás lehetséges	A kódszegmensek R-X, az adatszegmensek RW-; JIT esetén `mprotect`-váltás
PIE nélküli futtatható	A program szegmensei fix címen vannak, ASLR hatástalan	Fordítás `-fPIE -pie` kapcsolóval
Canary nélküli funkció a fordítóban	A stack overflow észrevétlen marad	`-fstack-protector-strong` alapértelmezett bekapcsolása
Canary kiszivárgás	A támadó megkerüli a védelmet	Out-of-bounds olvasási hibák javítása, formátumstring-ellenőrzés
Felhasználói mutató közvetlen dereferálása a kernelben	Kernel oops, jogosultságsértés	`copy_from_user`/`copy_to_user` használata
SMEP/SMAP letiltása	A kernel a user címekre ugorhat	Hardveres bitek engedélyezése, letiltás mellőzése
Alacsony ASLR entrópia (32 bit, fix régiók)	Brute-force támadás lehetséges	64 bites architektúra, minél nagyobb entrópia, nem várt címek
`randomize_va_space=0` éles rendszeren	Minden folyamat kiszámítható címterülettel fut	Csak reprodukálható teszteléshez használjuk, élesben maradjon 2

A memóriavédelem hatékonysága azon múlik, hogy a hardveres bitek, a kernel logikája és a fordító által generált ellenőrzések együttesen, rétegzett védelmet alkotnak. Egyetlen védelem kikapcsolása vagy megkerülése önmagában még nem jelenti a teljes rendszer kompromittálódását, de minden egyes réteg csökkenti a támadási felületet és növeli a támadás költségét.

A következő egység (Lapcsere-algoritmusok: FIFO, LRU és Clock) a lapcsere-algoritmusokra épít. Az itt tárgyalt lapjogosultsági bitek és izolációs elvek a lapcsere során is döntőek: a kernelnek figyelnie kell arra, hogy a lap ki- és beemelésekor a védelmi bitek megmaradjanak, és a laphiba ne változtassa meg a folyamatok közötti izolációs határokat.

11.4.6 6. Labor: memóriavédelmi és ASLR-beállítási terv

A labor célja, hogy saját kezűleg ellenőrizhesd az emulált környezetben a fent bemutatott védelmi mechanizmusok működését, és mérhetővé tedd az ASLR hatását. A labor nem igényel valós hardvert, és a hosztgép lemezét, bootloaderét vagy rendszerpartícióját nem módosítja. Minden művelet QEMU virtuális gépben vagy eldobható konténerben fut.

Kiinduló állapot. Tételezz fel egy minimális, emulált Linux rendszert, például egy BusyBox alapú initramfs-et, amely QEMU-n fut. A kísérlethez szükséges csomagok: GCC, libc, binutils, `file`, `readelf`, valamint a fent bemutatott `layout_info.c` forrásfájl. A virtuális gép konfigurációja a reprodukciós csomag része: x86-64 QEMU, 512 MiB RAM, egyetlen CPU, `-cpu qemu64` vagy hasonló.

Tervezett lépések.

1. Fordítsd le a `layout_info.c` programot kétféleképpen:

   - PIE-védelemmel: `gcc -O2 -fPIE -pie -fstack-protector-strong -o layout_pie layout_info.c -ldl`
   - védelem nélküli összehasonlításhoz: `gcc -O2 -no-pie -fno-stack-protector -o layout_nopie layout_info.c -ldl`

2. Ellenőrizd a binárisok védelmi tulajdonságait `readelf` segítségével:

   - `readelf -h layout_pie | grep Type` – a típus `DYN` (PIE), a `nopie` esetén `EXEC`.
   - `readelf -W -l layout_pie | grep GNU_STACK` – a `GNU_STACK` szegmensben az `E` bit hiánya NX-et jelez.
   - `readelf -s layout_pie | grep stack_chk` – a `__stack_chk_fail` hivatkozás a canary védelemre utal.

3. Kapcsold be és ki az ASLR-t az emulált Linux kernelben. A `randomize_va_space` értékét a következő paranccsal ellenőrizheted:

   ```
   cat /proc/sys/kernel/randomize_va_space
   ```

   A méréshez állítsd az értéket 0-ra, majd 2-re:

   ```
   echo 0 > /proc/sys/kernel/randomize_va_space
   echo 2 > /proc/sys/kernel/randomize_va_space
   ```

   Az érték pontos számértéke kernelverziótól függ, ezért a mérési naplóban rögzítsd a kernel verzióját is.

4. Futtasd a programot ötször mindkét beállítással, és gyűjtsd ki a kimenetből a `stack`, `heap`, `main` és `libc printf` sorokat. A mérési naplóba minden futtatásnál rögzítsd az értékeket.

5. Hasonlítsd össze az ASLR be- és kikapcsolt állapotában kapott címeket. Számítsd ki a különbségeket a futások között, és becsül meg az entrópiát a megfigyelt címek legnagyobb különbségének kettes alapú logaritmusaként:

   $$H_{\text{becsült}} \approx \log_2(\Delta_{\max})$$

   ahol $\Delta_{\max}$ a legnagyobb megfigyelt eltérés az adott szegmensben.

Elvárt eredmény. A PIE-védelemmel rendelkező program esetén ASLR bekapcsolt állapotában minden futásnál eltérő címeket kell kapnod; a stack, a heap, a libc és a PIE alapszegmens címe is változik. ASLR kikapcsolt állapotában a címek ismétlődnek. A `nopie` program esetén a `main` címe mindkét állapotban állandó, a többi szegmens azonban az ASLR beállításától függően változhat. A `readelf` kimenetben a PIE bináris típusa `DYN`, a `GNU_STACK` szegmensben az `E` bit hiányzik, és a `__stack_chk_fail` hivatkozás jelen van.

Mérési napló. A mérési naplót a könyvterv szerint kell vezetni: a mért értékeket és a várt/levezetett értékeket külön oszlopban kell feltüntetni. Példa a formátumra:

Futtatás	ASLR	Stack	Heap	main	libc printf	Megjegyzés
1	ON	0x7ffd1234a5c0	0x55aa1234a010	0x55aa12349000	0x7f1234abcd20	várt: változó
2	ON	0x7ff-dabcd5f10	0x55aaab-ce0010	0x55aaab-ce0000	0x7fabcd12342a	várt: eltér
…	…	…	…	…	…	…

A fenti értékek illusztrációként szolgálnak, nem valós mérésből származnak. A tényleges mérés során az emulált környezetben kapott értékeket kell rögzíteni.

Visszaállítás. A labor befejeztével állítsd vissza az ASLR beállítást az eredeti értékre, vagy egyszerűen állítsd le a virtuális gépet, és távolítsd el az eldobható initramfs-et. A hosztrendszer semmilyen állapotát nem módosítjuk.

Biztonsági megjegyzés. A labor kizárólag saját, emulált vagy eldobható környezetben futtatható. Valós éles rendszeren az ASLR kikapcsolása csökkenti a biztonsági szintet, ezért ott tilos ilyen mérést végezni. A `layout_info.c` nem tartalmaz rosszindulatú kódot, de a kimenete címeket fedhet fel, ami valós rendszeren támadást segíthetne; ezért a mérés emulált környezetben történik.

11.4.7 7. Önellenőrző kérdések

1. Miért nem elég egy lapot csupán olvashatóvá tenni ahhoz, hogy a folyamat ne fejleszthessen benne rosszindulatú kódot? Milyen szerepet játszik az NX bit?
2. Hogyan védi a kernel a saját címterét a felhasználói folyamatoktól, és miért van szükség a `copy_from_user` használatára akkor is, ha a lapbejegyzés user/supervisor bitje már elválasztja a két tartományt?
3. Milyen feltételei vannak annak, hogy az ASLR hatékony legyen? Mi történik, ha a futtatható nem PIE, vagy ha a `randomize_va_space` értéke 0?
4. Mi a különbség a lapjogosultság és a stack canary között a stack-túlcsordulásos támadások elleni védelemben?

Válaszok röviden:

1. Az NX bit hiányában az olvasható lap is végrehajtható lehet, ezért a támadó az adatterületre írt kódot futtathatná. Az NX bit explicit megtiltja a végrehajtást az adatterületeken.
2. A kernel lapjai supervisor szintűek, ezért a user mód nem éri el őket, de a kernelnek a rendszerhívások során a user által átadott mutatókat kell másolnia; a `copy_from_user` ellenőrzi, hogy a cím a user tartományba esik, és hibakóddal tér vissza, ha nem. Ez megakadályozza, hogy a kernel a saját címterébe mutató user mutatót használjon.
3. Az ASLR hatékonyságához PIE futtatható, elegendő entrópia, a heap, a stack és a könyvtárak véletlenszerű elhelyezése, illetve a címek kiszivárgásának megakadályozása szükséges. PIE nélkül a program szegmensei fix címen vannak, `randomize_va_space=0` esetén pedig egyáltalán nincs véletlenítés.
4. A lapjogosultság hardveres védelem: megakadályozza, hogy a stacken kód fusson. A stack canary szoftveres észlelés: a puffer túlcsordulásakor a visszatérési cím felülírása előtt észleli a változást. A kettő kiegészíti egymást.

11.4.8 8. Összefoglalás

A Memóriavédelem, ASLR és izoláció fejezetben a memóriavédelem és a címterek izolációjának alapjait tekintettük át. A lapbejegyzések R/W/X és U/S bitjei a hardver szintjén kényszerítik ki a jogosultságok betartását, az NX bit meggátolja a kódinjektálást, a SMEP/SMAP pedig a kernel és a user tér közötti hozzáférést szigorítja. A címterek izolációja minden folyamatnak önálló virtuális teret ad, a kernel pedig a user mutatókat biztonságos másolófüggvényekkel kezeli. Az ASLR a program és a könyvtárak betöltési címeit véletlenszerűsíti, a stack canary pedig a puffertúlcsordulást észleli. Mindezek együttesen alkotják a modern operációs rendszerek rétegzett memóriavédelmét, amelyre a következő fejezetek lapcsere- és memóriakezelési algoritmusai épülnek.

A fejezet laborjában egy emulált Linux környezetben ellenőrizted a PIE, az NX és a canary jelenlétét, valamint megmérted az ASLR által okozott címváltozásokat. A következő egységben (Lapcsere-algoritmusok: FIFO, LRU és Clock) a lapcsere-algoritmusokkal foglalkozunk, ahol az itt bevezetett jogosultsági bitek és izolációs elvek a lapok ki- és beemelése során is kulcsszerepet játszanak.

12 Virtuális memória és lapcserélés

12.1 Lapcsere-algoritmusok: FIFO, LRU és Clock

A virtuális memória egyik központi kérdése, hogy mit tegyen a kernel, amikor a futó folyamat olyan laphoz akar hozzáférni, amelyik nincs a fizikai memóriában. A demand paging mechanizmusa (Memóriavédelem, ASLR és izoláció) biztosítja a lap betöltését, de ha a memória megtelt, egy bent lévő lapot ki kell választani, és a tartalmát ki kell írni a háttértárra (swapping), ha módosult. Ez a döntés a lapcserélési algoritmus feladata. A cél nem egyszerűen „egy lap kiválasztása”, hanem az, hogy a jövőbeli laphibák számát minimalizáljuk, és ezzel a rendszer effektív memória-sávszélességét és válaszidejét javítsuk. A rossz döntés laphiba-sorozatot és felesleges I/O-t generál; a jó döntés közel tartja a rendszert az ideális, „minden lap bent van” állapothoz.

Ebben az alfejezetben a három klasszikus algoritmust, a FIFO-t, az LRU-t és a Clock-ot vizsgáljuk meg, bevezetjük a referencia string fogalmát, és megértjük a Belady-anomáliát. Ezután kitérünk arra, hogyan támogatja a hardver a lapcserélést a referenced és dirty bitekkel, és bemutatunk egy egyszerű, reprodukálható szimulációs labort.

12.1.1 1. A lapcsere értékelési modellje: referencia string és laphibaarány

Egy lapcserélő algoritmus offline vagy online minőségét úgy mérhetjük, hogy megadjuk a folyamat memória-hozzáféréseinek sorozatát lapazonosítókkal. Ez a **referencia string**: például az $R = (0, 1, 2, 3, 0, 1, 4, 0, 1, 2, 3, 4)$ sorozat azt jelenti, hogy a program először a 0. laphoz nyúl, majd az 1.-hez, a 2.-hoz, a 3.-hoz, aztán ismét a 0.-hoz stb. A keretek száma legyen m, ez a fizikai memóriában a folyamat számára elérhető laphelyek számát adja. A laphiba akkor következik be, ha az éppen hivatkozott lap nincs a keretek között.

A mérés elsődleges mutatója a **laphibaarány** (page fault rate), ami a laphibák számának és a referencia string hosszának hányadosa. Minél kisebb ez az érték, annál jobban tartja az algoritmus a munkahalmazt a memóriában. Emellett fontos a **csere kiírások** (swap out) száma: ha a kiválasztott lap módosult (dirty), akkor a tartalmát előbb ki kell írni a háttértárra, ami extra I/O-t jelent. A jó algoritmus lehetőleg tiszta lapot cserél, vagy olyat, amelyet ritkán módosítanak.

A lapcserélő algoritmusoknak **online** döntést kell hozniuk, hiszen a jövőbeli hivatkozások általában nem ismertek. Ezért gyakorlati heurisztikákat használnak, amelyek a múltbeli viselkedésből következtetnek a jövőre. Az elméleti optimum, a Belady-féle **OPT** algoritmus (amelyik a legkésőbb használt lapot cseréli) csak utólag, a teljes referencia string ismeretében számítható ki; jó viszonyítási alap, de valós rendszerben nem megvalósítható.

12.1.2 2. FIFO: a legegyszerűbb sorban állás

A **FIFO (First-In, First-Out)** algoritmus egy várakozási sort tart a keretekben lévő lapokból. Amikor lapot kell betölteni, a sor elején álló, tehát a legrégebben **betöltött** lapot választja

áldozatul, és a helyére kerül az új lap. Az algoritmus nem veszi figyelembe, hogy a lapot azóta hányszor használták.

12.1.3 2.1. Működése példán

Legyen a referencia string $R = (0, 1, 2, 3, 0, 1, 4, 0, 1, 2, 3, 4)$ és a keretek száma $m = 3$. A FIFO a következő lépéseket hajtja végre (a nyíl a sor elejét jelöli, az új lap a sor végére kerül):

Lépés	Hiv. lap	Keretek (sorrend)	Laphiba?	Csere
1	0	[0]	igen	–
2	1	[0, 1]	igen	–
3	2	[0, 1, 2]	igen	–
4	3	[1, 2, 3]	igen	0
5	0	[2, 3, 0]	igen	1
6	1	[3, 0, 1]	igen	2
7	4	[0, 1, 4]	igen	3
8	0	[0, 1, 4]	nem	–
9	1	[0, 1, 4]	nem	–
10	2	[1, 4, 2]	igen	0
11	3	[4, 2, 3]	igen	1
12	4	[4, 2, 3]	nem	–

A laphibák száma **9**. A laphibaarány $9/12 = 0{,}75$.

A FIFO előnye az egyszerű implementáció: egy körkörös puffer és egy mutató elegendő. Hátránya viszont súlyos: egy régóta bent lévő, de gyakran használt lapot is kiválaszthat cserére, mert nem különbözteti meg a „régi és használt” lapot a „régi és elhanyagolt” laptól.

12.1.4 2.2. A Belady-anomália

A FIFO kapcsán ismerkedünk meg a **Belady-anomáliával**: bizonyos referencia stringek esetén a keretek számának növelése **több** laphibát okoz, nem kevesebbet. Ez ellentmond annak az intuíciónak, hogy több memória mindig jobb.

A klasszikus példa a következő referencia string:

$$R_B = (1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5).$$

Futtassuk le a FIFO-t $m = 3$ kerettel:

Lépés	Hiv.	Keretek	Hiba?
1	1	[1]	igen
2	2	[1, 2]	igen
3	3	[1, 2, 3]	igen
4	4	[2, 3, 4]	igen (cseréli 1)
5	1	[3, 4, 1]	igen (cseréli 2)
6	2	[4, 1, 2]	igen (cseréli 3)
7	5	[1, 2, 5]	igen (cseréli 4)
8	1	[1, 2, 5]	nem

Lépés	Hiv.	Keretek	Hiba?
9	2	[1, 2, 5]	nem
10	3	[2, 5, 3]	igen (cseréli 1)
11	4	[5, 3, 4]	igen (cseréli 2)
12	5	[5, 3, 4]	nem

Itt a laphibák száma **9**.

Most növeljük a keretek számát $m = 4$ értékre:

Lépés	Hiv.	Keretek	Hiba?
1	1	[1]	igen
2	2	[1, 2]	igen
3	3	[1, 2, 3]	igen
4	4	[1, 2, 3, 4]	igen
5	1	[1, 2, 3, 4]	nem
6	2	[1, 2, 3, 4]	nem
7	5	[2, 3, 4, 5]	igen (cseréli 1)
8	1	[3, 4, 5, 1]	igen (cseréli 2)
9	2	[4, 5, 1, 2]	igen (cseréli 3)
10	3	[5, 1, 2, 3]	igen (cseréli 4)
11	4	[1, 2, 3, 4]	igen (cseréli 5)
12	5	[2, 3, 4, 5]	igen (cseréli 1)

A laphibák száma így **10**, eggyel több, mint 3 keret esetén. Ez a **Belady-anomália** konkrét demonstrációja.

A jelenség oka, hogy a FIFO nem rendelkezik az ún. **stack tulajdonsággal**: az m kerettel memóriában tartott lapok halmaza nem feltétlenül részhalmaza az $m + 1$ kerettel tartott halmaznak. Más szavakkal, a nagyobb keretszám nem garantálja, hogy ugyanazok a lapok bent maradnak, és egy korábban kiszorított lap későbbi visszatöltése extra hibát okozhat. Az LRU és a legtöbb modern algoritmus rendelkezik a stack tulajdonsággal, így náluk ez az anomália nem lép fel.

12.1.5 3. LRU: a legrégebben használt lap cseréje

Az **LRU (Least Recently Used)** algoritmus azt a lapot választja cserére, amelyiket a legrégebben **használták** (olvasták vagy írták). Ez a múltbeli lokalitás elvén alapul: ha egy lap mostanában volt használva, valószínűleg a közeljövőben is használni fogják; amelyik régóta nem kapott hozzáférést, az kevésbé valószínű, hogy hamarosan kell.

12.1.6 3.1. Megvalósítási lehetőségek

Az LRU elvileg megvalósítható időbélyeggel: minden lapbejegyzéshez eltároljuk az utolsó hozzáférés időpontját, és cserénél a legkisebb időbélyegű lapot keressük. Ez minden memóriahozzáférésnél írást igényel a lap táblázatba, és a minimum keresése is költséges.

Hatékonyabb lista-alapú megoldás: minden hozzáférésnél a hivatkozott lapot a lista elejére mozgatjuk. Ekkor a lista vége tartalmazza a legrégebben használt lapot, így a cserélendő lap

kiválasztása $O(1)$ idő. A lista láncolásához azonban minden hivatkozásnál mutatókat kell frissíteni, ami szintén megterhelő.

A gyakorlatban ezért a valódi LRU ritka; a kernel inkább hardveres támogatással közelíti (lásd a Clock szakaszt).

12.1.7 3.2. Működése példán

Használjuk ugyanazt a referencia stringet, $R = (0, 1, 2, 3, 0, 1, 4, 0, 1, 2, 3, 4)$, $m = 3$ mellett. Az LRU listája a legutóbbi használat szerint rendezett; a lista eleje a legfrissebb, a vége a legrégebbi.

Lépés	Hiv.	Keretek (lista)	Hiba?	Csere
1	0	[0]	igen	–
2	1	[1, 0]	igen	–
3	2	[2, 1, 0]	igen	–
4	3	[3, 2, 1]	igen	0 (legrégebbi)
5	0	[0, 3, 2]	igen	1
6	1	[1, 0, 3]	igen	2
7	4	[4, 1, 0]	igen	3
8	0	[0, 4, 1]	nem (0 már bent, lista frissül)	–
9	1	[1, 0, 4]	nem	–
10	2	[2, 1, 0]	igen	4
11	3	[3, 2, 1]	igen	0
12	4	[4, 3, 2]	igen, mert 4 kikerült	1

A laphibák száma így **9**, megegyezik a FIFO eredményével ezen a stringen. Ez a referencia string nem mutat különbséget a két algoritmus között, mert a hivatkozások mintázata nem hozza előnyös helyzetbe az LRU-t.

Általánosságban az LRU jobb, mint a FIFO, mert a gyakori lapokat hajlamos bent tartani. A stack tulajdonság miatt nincs Belady-anomália: ha m-et növeljük, az LRU által memóriában tartott lapok halmaza felülről bővül, így a laphibák száma monoton nem-növekvő.

12.1.8 4. Clock: a második esély, hardveres referencia bittel

A valódi rendszerekben a lap táblázat bejegyzéseihez egy **referenced bit** (R bit) tartozik, amelyet az MMU automatikusan 1-re állít, amikor a lapot olvassák vagy írják. A **Clock algoritmus** (más néven **Second Chance**) ezt a bitet használja, hogy a FIFO sorrendet kombinálja a használati információval.

12.1.9 4.1. Az algoritmus működése

A kereteket egy körkörös listában tároljuk, és van egy **mutató** (óramutató), amelyik a következő vizsgálandó keretre mutat. Amikor lapot kell betölteni:

1. Nézzük meg az aktuális mutatóhoz tartozó lap R bitjét.
2. Ha az R bit **0**, akkor ezt a lapot választjuk cserére, a mutatót továbbléptetjük, és az új lapot a helyére töltjük.

3. Ha az R bit **1**, akkor a lap kap egy „második esélyt”: az R bitet töröljük (0-ra állítjuk), a mutatót továbbléptetjük a következő keretre, és megismételjük a vizsgálatot.
4. Addig folytatjuk, amíg 0 értékű R bitet nem találunk.

A referenced bit törlése azt jelenti, hogy ha a lap a következő körben újra hivatkozásra kerül, az MMU ismét beállítja 1-re, így a lap megmenekül a cserétől. Ha a lap a teljes kör alatt nem kap hozzáférést, a bit 0 marad, és a következő vizsgálatnál kikerül.

12.1.10 4.2. Példa a Clock működésére

Legyen a referencia string $R = (0, 1, 2, 3, 0, 1, 4)$ és $m = 3$ keret. Az egyszerűség kedvéért kezdetben üresek a keretek. A következő lépéseket hajtjuk végre (a betöltött lap R bitje kezdetben 1, mivel a betöltés is hozzáférésnek számít; a mutató a legrégebben betöltött lapra mutat):

Lépés	Hiv.	Keretek (R bittel)	Mutató	Hiba?	Művelet
1	0	[0(R=1)]	0	igen	betöltés
2	1	[0(R=1), 1(R=1)]	0	igen	betöltés
3	2	[0(R=1), 1(R=1), 2(R=1)]	0	igen	betöltés
4	3	[3(R=1), 1(R=0), 2(R=0)]	1	igen	mutató 0-nál: R=1 → 0; lép 1-re, ott R=1 → 0; lép 2-re, ott R=1 → 0; lép 0-ra, ott R=0, cserél 0-t.
5	0	[3(R=1), 1(R=0), 0(R=1)]	1	igen	mutató 1-nél: R=0 → cserél 1-et.
6	1	[3(R=1), 1(R=1), 0(R=1)]	2	igen	mutató 2-nél: R=0 → cserél 2-t.
7	4	[4(R=1), 1(R=0), 0(R=0)]	0	igen	mutató 0-nál: R=1 → 0; lép 1-re, ott R=1 → 0; lép 2-re, ott R=1 → 0; lép 0-ra, R=0, cserél 0-t.

Ez a lépésenkénti követés jól mutatja, hogy a referenced bit hogyan ad második esélyt a gyakran használt lapoknak. A valóságban a mutató nem feltétlenül a legrégebbi betöltésre mutat, hanem egy körben jár; a fenti leírás a lineáris kezdeti állapotból indult.

12.1.11 4.3. Clock és kiterjesztései

A Clock algoritmus előnye, hogy csak a referenced bitet igényli, amelyet a hardver automatikusan fenntart. Nem kell időbélyeg vagy lista-frissítés minden memória-hozzáférésnél; a bit törlését csak a lapcserélő futása közben végezzük. Ezért a gyakorlati rendszerekben (Linux, Windows, BSD) a Clock és annak változatai terjedtek el.

A **Clock with dirty bit** (kétbites változat) a referenced bit mellett a **dirty bitet** (M bit) is figyelembe veszi. A dirty bitet a hardver akkor állítja 1-re, ha a lapot írták. Csere esetén a dirty bit 1 értéke azt jelenti, hogy a lap tartalmát ki kell írni a háttértárra; ha 0, a kiírás elhagyható. A két bit négy kombinációja alapján a cserélő négy osztályba sorolja a lapokat:

R	M	Jelentés	Csere prioritás
0	0	nem használt, tiszta	legmagasabb
0	1	nem használt, de módosult	magas (kiírással)
1	0	használt, tiszta	alacsony
1	1	használt, módosult	legalacsonyabb

A Linux kernel lapcserélője (például a `kswapd` és a közvetlen visszanyerő útvonal) hasonló elveket követ, de a modern megvalósítások aktív/inaktív listákat és LRU-közelítést használnak, amelyek a `PG_referenced` és `PG_dirty` jelzőbitekre épülnek. A pontos algoritmus verzióról verzióra változik, ezért a kernel forrásában mindig az adott kiadás dokumentációját kell ellenőrizni.

12.1.12 5. Hardvertámogatás és LRU-közelítések

A lapcserélő algoritmusok minősége nagyban függ attól, hogy milyen információt szolgáltat a hardver a lapok használatáról. A két legfontosabb bit a lap táblázat bejegyzésében:

- **Referenced (R) bit**: az MMU 1-re állítja, amikor a lapot olvassák vagy írják.
- **Dirty (M) bit**: az MMU 1-re állítja, amikor a lapot írják.

Ezeket a biteket a kernel rendszeresen olvassa és törli, hogy követni tudja a hozzáféréseket. A törlés történhet a lapcserélő futtatásakor, vagy periodikusan, például időzítő megszakításból.

A valódi LRU megvalósítása drága lenne, ezért **LRU-közelítéseket** használnak. Az egyik elterjedt módszer az **aging** (öregítés). Minden kerethez tartozik egy N bites számláló. Periodikusan (például minden órajel-megszakításban) a kernel a következőt teszi:

1. Minden lap R bitjét jobbról shifteli a számláló legfelső bitjébe.
2. A számlálót jobbra shifteli 1-gyel.
3. Ezután az összes R bitet törli.

Ezáltal a számláló felső bitjei az utóbbi időszakok hozzáférési mintáját tárolják. A legnagyobb számlálóérték a leggyakrabban, legutóbb használt lapot jelöli; a legkisebb a legrégebben használtat. Ez egy véges ablakú, kvantált LRU-közelítés. Ha a számláló 8 bites, akkor az utolsó 8 periódusra emlékszünk, ami elegendő a lokális viselkedés modellezésére.

Fontos, hogy a törlést megfelelően ütemezzük: ha túl ritkán töröljük a biteket, az összes lap hamarosan 1-re áll, és elveszti a megkülönböztető erejét. Ha túl gyakran, akkor a bit nem tudja megőrizni a közelmúltbeli használatot, és az algoritmus visszafejlődik FIFO-vá.

12.1.13 6. Tipikus hibák és félreértések

A lapcserélő algoritmusok elemzése során gyakori hibák:

- **A Belady-anomália feltételezése minden algoritmusra**: csak a FIFO és néhány hasonló, nem stack tulajdonságú algoritmus mutatja. Az LRU-ra, a Clockra és az OPT-ra nem érvényes.
- **A referenced bit figyelmen kívül hagyása**: ha a kernel nem törli rendszeresen az R biteket, a Clock algoritmus lényegében FIFO-vá válik, mert minden lap R bitje 1 marad.
- **A dirty bit elhanyagolása**: a dirty lap cseréje költséges I/O-t okoz. A jó algoritmus igyekszik tiszta lapokat választani, vagy legalább a dirty lapokat csak végső esetben kiszorítani.
- **A referencia string helytelen értelmezése**: a string lapazonosítókat tartalmaz, nem byte-címeket. Egy memória-hozzáférésnél a lap méretétől függ, melyik lapra esik a cím, ezért a szimulációban a lapméretet explicitté kell tenni.

12.1.14 7. Labor: FIFO, LRU és Clock szimulációja

A következő labor célja, hogy a három algoritmust egy rögzített referencia stringen lefuttassuk, és összehasonlítsuk a laphibák számát. A szimulátort C nyelven implementáljuk, mert ez illeszkedik a könyv kódkonvenciójához, és fordítás után közvetlenül futtatható.

12.1.15 7.1. Kiinduló állapot és környezet

- **Emulált környezet**: bármely POSIX-kompatibilis hoszt (pl. Linux vagy QEMU-ban futó Linux) elegendő, mert a szimulátor csak felhasználói térben fut, nem módosítja a hoszt memóriakezelését.
- **Fordító**: `gcc` vagy `clang` C99 módban.
- **Fájl**: `paging_sim.c`, amely a referencia stringet és a keretszámot parancssori argumentumból olvassa.

12.1.16 7.2. A szimulátor pszeudokódja

Az alábbi pszeudokód a három algoritmus közös vázát mutatja. A konkrét megvalósításban a `clock` mutatót körkörösen léptetjük, a `fifo` sorhoz egy indexet használunk, az LRU-hoz pedig egy időbélyeg tömböt.

```
function simulate(algorithm, R, m):
    frames[k] = -1  // -1 jelzi az üres keretet
    referenced[k] = false
    clock_hand = 0
    faults = 0

    for each page in R:
        if page not in frames:
            faults++
            if free frame exists:
                put page into free frame
            else:
                if algorithm == FIFO:
                    victim = oldest_loaded_index
                else if algorithm == LRU:
                    victim = index_of_min(last_used_time)
```

```
                else if algorithm == CLOCK:
                    while referenced[clock_hand]:
                        referenced[clock_hand] = false
                        clock_hand = (clock_hand + 1) mod m
                    victim = clock_hand
                    clock_hand = (clock_hand + 1) mod m
                replace victim with page
                referenced[victim] = true
                last_used_time[victim] = current_step
        else:
            // találat
            if algorithm == LRU:
                last_used_time[found_index] = current_step
            if algorithm == CLOCK:
                referenced[found_index] = true
        current_step++
    return faults
```

A pszeudokód a C implementáció alapja. A `referenced` bitek törlése a Clock algoritmusban a ciklusban történik, ahogy a mutató halad.

12.1.17 7.3. C program váz

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_FRAMES 64

typedef enum { FIFO, LRU, CLOCK } algo_t;

int simulate(algo_t algo, const int *ref, int len, int m) {
    int frames[MAX_FRAMES];
    int referenced[MAX_FRAMES];
    int last_used[MAX_FRAMES];
    int clock_hand = 0;
    int faults = 0, next_fifo = 0;
    int used = 0; // number of occupied frames

    for (int i = 0; i < m; i++) {
        frames[i] = -1;
        referenced[i] = 0;
        last_used[i] = -1;
    }

    for (int step = 0; step < len; step++) {
        int page = ref[step];
        int found = -1;
        for (int i = 0; i < used; i++) {
            if (frames[i] == page) { found = i; break; }
        }

        if (found >= 0) {
            if (algo == LRU) last_used[found] = step;
            if (algo == CLOCK) referenced[found] = 1;
            continue;
        }

        faults++;
        if (used < m) {
            frames[used] = page;
            referenced[used] = 1; // betöltéskor használtnak tekintjük
```

```
            last_used[used] = step;
            used++;
        } else {
            int victim;
            if (algo == FIFO) {
                victim = next_fifo;
                next_fifo = (next_fifo + 1) % m;
            } else if (algo == LRU) {
                victim = 0;
                for (int i = 1; i < m; i++)
                    if (last_used[i] < last_used[victim]) victim = i;
            } else { // CLOCK
                while (referenced[clock_hand]) {
                    referenced[clock_hand] = 0;
                    clock_hand = (clock_hand + 1) % m;
                }
                victim = clock_hand;
                clock_hand = (clock_hand + 1) % m;
            }
            frames[victim] = page;
            referenced[victim] = 1;
            last_used[victim] = step;
        }
    }
    return faults;
}

int main(int argc, char **argv) {
    if (argc != 4) {
        fprintf(stderr, "Hasznalat: %s <fifo|lru|clock> <keretszam> <inputfile>\n", argv[0]);
        return 1;
    }
    algo_t algo;
    if (strcmp(argv[1], "fifo") == 0) algo = FIFO;
    else if (strcmp(argv[1], "lru") == 0) algo = LRU;
    else if (strcmp(argv[1], "clock") == 0) algo = CLOCK;
    else {
        fprintf(stderr, "Ismeretlen algoritmus: %s\n", argv[1]);
        return 1;
    }
    int m = atoi(argv[2]);
    if (m < 1 || m > MAX_FRAMES) {
        fprintf(stderr, "Hibas keretszam: %d\n", m);
        return 1;
    }

    FILE *f = fopen(argv[3], "r");
    if (!f) { perror("fopen"); return 1; }
    int ref[1024];
    int len = 0, p;
    while (fscanf(f, "%d", &p) == 1 && len < 1024) {
        ref[len++] = p;
    }
    fclose(f);

    int faults = simulate(algo, ref, len, m);
    printf("Algoritmus: %s, keretek: %d, referencia hossz: %d\n", argv[1], m, len);
    printf("Laphibak: %d\n", faults);
    printf("Laphibaarany: %.3f\n", (double)faults / len);
    return 0;
}
```

A kód illusztráció, a lényeg az algoritmusok működésének bemutatása; nem célunk optimalizálni

a keresést.

12.1.18 7.4. Bemeneti referencia string és elvárt eredmények

Használjuk a következő input fájlt, amely a Belady-anomália példáját tartalmazza:

```
1 2 3 4 1 2 5 1 2 3 4 5
```

Az elvárt laphibaszámok a kézi elemzés alapján:

Algoritmus	Keretek m	Elvárt laphibák
FIFO	3	9
FIFO	4	10
LRU	3	10
LRU	4	10
Clock	3	10 (a referencia bitek miatt kissé függ a kezdeti állapottól; a fenti pszeudokód szerint 10)
Clock	4	10

A Clock algoritmus pontos eredménye attól függ, hogy a betöltéskor a referenced bitet 1-re állítjuk-e. A fenti C program szerint a betöltés is hozzáférés, ezért az első körben minden lap R bitje 1, és a mutató körbejár, ami a 4 keretes esetben 10 hibát ad.

12.1.19 7.5. Futtatási parancsok és mérési napló

A szimulátor fordítása:

```
gcc -O2 -Wall -Wextra -o paging_sim paging_sim.c
```

A futtatás minden algoritmusra:

```
./paging_sim fifo 3 input.txt
./paging_sim fifo 4 input.txt
./paging_sim lru 3 input.txt
./paging_sim lru 4 input.txt
./paging_sim clock 3 input.txt
./paging_sim clock 4 input.txt
```

A mérési naplóban rögzíteni kell a program kimenetét, valamint a környezet adatait:

- **Hoszt**: pl. `Linux 6.8.0-45-generic` (a verzió ellenőrzendő az adott gépen).
- **Fordító**: `gcc (Ubuntu 13.2.0-23ubuntu4) 13.2.0`.
- **CPU**: az adott gép típusa, órajele (nem befolyásolja a szimulációt, de a reprodukálhatósághoz fel kell tüntetni).
- **Dátum**: a mérés időpontja.

A **mért** eredményeket a program kimenete adja; a **várt** eredmények a fenti táblázatból származnak. A labor jegyzőkönyvben külön oszlopban kell feltüntetni a mért és a várt értéket, és eltérés esetén meg kell vizsgálni a szimulátor helyességét.

12.1.20 7.6. Visszaállítás

A szimulátor nem módosítja a hoszt állapotát, így külön visszaállítási lépésre nincs szükség. A generált `paging_sim` bináris és az `input.txt` törölhető a labor végén.

12.1.21 8. Összefoglalás és önellenőrzés

Ebben az alfejezetben megismertük a lapcserélő algoritmusok alapjait. A **FIFO** egyszerű, de hajlamos a Belady-anomáliára, mert nem veszi figyelembe a használatot. Az **LRU** elméletileg jó, de megvalósítása drága; a gyakorlatban a hardveres referenced bitre épülő **Clock** algoritmus ad jó kompromisszumot. A hardver a referenced és dirty bitekkel támogatja a döntést, az aging pedig véges ablakú LRU-közelítést tesz lehetővé.

Önellenőrző kérdés: Miért nincs Belady-anomália az LRU algoritmusnál, és hogyan garantálja ezt a stack tulajdonság? A válaszban indokolja meg, hogy a keretszám növelése miért nem növelheti a laphibák számát, és miért nem teljesül ugyanez a FIFO-ra.

További gyakorló feladat: Módosítsa a labor szimulátorát, hogy a dirty bitet is kezelje. I/O írási költséget szimulálva mérje meg, hány lapkiírás történik az egyes algoritmusoknál, ha minden harmadik hivatkozás írás. Hasonlítsa össze a Clock és a kétbites Clock (R, M) viselkedését azonos referencia string mellett.

12.2 Working set, thrashing és lokális/globális csere

Az előző alfejezetben a lapcsere-algoritmusok alapjait vizsgáltuk: a FIFO, az LRU és a Clock módszert. Mindegyik azt a kérdést válaszolja meg, hogy adott keretszám esetén melyik lapot válasszuk ki helyettesítésre. A virtuális memória teljesítménye azonban nem csak a helyettesítési döntéstől függ; legalább annyira fontos, hogy egy folyamat hány lapkeretet kap, és a rendszer egésze hogyan osztja el a szűkös fizikai memóriát a versengő folyamatok között. Ebben az alfejezetben a munkakészlet (working set) fogalmából kiindulva azt vizsgáljuk, mikor válik a rendszer túlterheltté, hogyan ismerhető fel a thrashing, és milyen lokális, illetve globális lapcsere-stratégiák alkalmazhatók.

12.2.1 A munkakészlet fogalma és modellje

A munkakészlet fogalmát Peter Denning vezette be 1968-ban. Legyen egy folyamat lapreferencia-sorozata az idő függvényében. Egy adott t időpillanatban és egy rögzített Δ ablakméret mellett a $W(t, \Delta)$ munkakészlet azoknak a lapoknak a halmaza, amelyekre a $[t - \Delta, t]$ intervallumban legalább egyszer hivatkozott a folyamat.

$$W(t, \Delta) = \{p \mid p \text{ lapot hivatkozta a folyamat } [t - \Delta, t] \text{ intervallumban}\}$$

A munkakészlet mérete $w(t, \Delta) = |W(t, \Delta)|$. A Δ ablakméretet általában memóriahivatkozások számában vagy időben fejezzük ki. A munkakészlet nem azonos a folyamat teljes címtartományával; a lokalitás elve miatt a folyamat egy adott időablakban csak a lapjainak egy kis részét érinti.

Példaként tekintsünk egy folyamatot, amelynek lapreferencia-sorozata a következő (az egyes hivatkozások lapazonosítói, időrendben):

```
1, 2, 3, 1, 4, 2, 5, 1, 2, 3, 4, 5
```

Ha $\Delta = 4$ hivatkozás, akkor a $t = 10$ időpontban az utolsó négy hivatkozás:

```
5, 1, 2, 3
```

Ezért a munkakészlet:

$$W(10, 4) = \{1, 2, 3, 5\}$$

mérete $w(10, 4) = 4$. Ha $t = 9$, az utolsó négy hivatkozás $2, 5, 1, 2$, tehát $W(9, 4) = \{1, 2, 5\}$ és $w(9, 4) = 3$. A munkakészlet tehát időben változik, és a folyamat lokalitását tükrözi.

A Δ ablakméret megválasztása kritikus. Túl kicsi Δ esetén a munkakészlet nem fedi le a folyamat tényleges lokalitását, mert a hivatkozási mintázat lassú változásait nem követi. Túl nagy Δ esetén a munkakészlet már a folyamat teljes címtartományát magában foglalhatja, és elveszíti az előrejelző képességét. A gyakorlatban az ablakméretet a memóriahivatkozások száma alapján választják meg; tipikusan 10 000–100 000 hivatkozás között szokott lenni, de ez hardver- és terhelésfüggő.

A munkakészlet modell gyakorlati haszna az, hogy a keretallokáció alapjául szolgálhat. Ha egy folyamatnak legalább akkora keretszámot biztosítunk, mint a munkakészletének mérete, akkor a laphibák száma alacsony marad. Ha a keretszám a munkakészlet alá csökken, a folyamat gyakori laphibákat szenved el, mert a munkakészlet lapjai nem férnek el egyszerre a memóriában. A munkakészlet méretének időbeli változása fázisátmeneteket mutat: a folyamat végrehajtása során más-más kódrészeket és adatterületeket érint, így a munkakészlete hirtelen megváltozhat. Például egy fordítóprogram először a forráskódot olvassa be, majd a szimbólumtáblát építi, végül a tárgykódot generálja; minden fázis más lapokat érint.

12.2.2 Page fault rate és a munkakészlet kapcsolata

A page fault rate (laphoba-arány) a laphibák számának és a memóriahivatkozások számának hányadosa, vagy időegységre vetített gyakoriság. Jelölje f a laphibák számát másodpercenként. A munkakészlet és a page fault rate közötti összefüggés nem lineáris. Amíg a folyamat keretszáma meghaladja a munkakészletének méretét, a laphibák ritkák; amint a keretszám a munkakészlet alá esik, a laphibák gyakorisága meredeken emelkedik. Az összefüggést gyakran working set curve-nek nevezik: a keretszám függvényében a page fault rate alakja könyököt mutat.

A gyakorlatban a page fault rate mérésekor meg kell különböztetni a minor és major faultokat. A minor fault akkor keletkezik, ha a lap a fizikai memóriában van, de a folyamat oldaltáblája nem tartalmazza a hozzá tartozó bejegyzést; például osztott könyvtár vagy copy-on-write lap esetén. A major fault akkor keletkezik, ha a lapot háttértárról (swap vagy fájl) kell beolvasni. A thrashing szempontjából a major faultok száma a releváns, mert ezek igénylik a lassú háttértár-hozzáférést.

A page fault rate alapján szabályozó algoritmus a Page Fault Frequency (PFF) módszer. Minden folyamathoz mérjük a laphibák gyakoriságát egy rövid időintervallumban. Ha a mért érték egy felső küszöb fölé emelkedik, a folyamat több keretet kap; ha egy alsó küszöb alá csökken, a folyamat kereteiből elveszünk. Ha nincs szabad keret, a folyamatot felfüggeszthetjük, amíg a memórianyomás csökken. A PFF algoritmus pszeudokódja:

```
for each process P:
    rate = measure_page_fault_rate(P, interval)
    if rate > HIGH_THRESHOLD:
        if free_frames > 0:
            allocate_frame(P)
        else:
            suspend_process(P)
    else if rate < LOW_THRESHOLD:
        deallocate_frame(P)
```

A küszöbértékek megválasztása rendszerfüggő. A felső küszöb azt jelzi, hogy a folyamat munkakészlete nem fér el a rendelkezésre álló keretekben; az alsó küszöb azt, hogy a folyamat több keretet kapott, mint amennyire szüksége van.

12.2.3 Thrashing: okok és felismerés

Thrashing akkor lép fel, ha a rendszerben futó folyamatok összesített munkakészlete tartósan meghaladja a rendelkezésre álló fizikai memória méretét. Ilyenkor a laphibák gyakorisága olyan magas, hogy a CPU idejének jelentős részét a lapcsere kiszolgálása köti le, miközben a folyamatok alig haladnak előre. A rendszer CPU-kihasználtsága leesik, a háttértár I/O várakozási sor hossza megnő.

A thrashing fokozatosan alakul ki. Először a szabad keretek fogynak el, majd a rendszer elkezdi a lapok kiírását a háttértárra. Ha a folyamatok továbbra is igénylik a memóriát, a laphibák gyakorisága nő, a CPU kihasználtsága csökken. A felhasználó vagy az üzemeltető gyakran tévesen CPU-teljesítmény-problémára gyanakszik, pedig a valódi ok a memória-túlterhelés.

A thrashing felismeréséhez több metrikát kell együtt figyelni:

- **Major fault rate**: másodpercenkénti major laphibák száma meredeken emelkedik.
- **CPU kihasználtság**: a CPU kihasználtsága leesik, mert a folyamatok várakoznak a háttértárra.
- **Swap be/ki forgalom**: a `pswpin` és `pswpout` számlálók értéke magas.
- **Átlagos terhelés**: a futásra kész folyamatok száma magas, de a tényleges CPU-használat alacsony.
- **Lemez I/O várakozás**: a háttértár várakozási sora hosszú.

Egyetlen metrika nem elegendő a diagnózishoz. Például a magas CPU-kihasználtság normál terhelésnél is előfordulhat, a magas major fault rate pedig nem feltétlenül jelent thrashinget, ha a rendszer éppen nagy fájlokat olvas be.

A thrashing elkerülésének alapelve, hogy a rendszer ne engedjen be több folyamatot, mint amennyinek a munkakészlete együttesen elfér a fizikai memóriában. Ezt a szabályozást load controlnak vagy admission controlnak nevezzük. Ha a memórianyomás túl nagy, a rendszer felfüggeszti (swap out) vagy leállítja a legkevésbé fontos folyamatokat.

12.2.4 Lokális és globális lapcsere

A lapcsere-stratégiák két alapvető osztályba sorolhatók aszerint, hogy a helyettesítendő lapot mely folyamat lapjai közül választják ki.

Lokális lapcsere esetén egy folyamat laphibája esetén csak a saját lapjai közül választhatunk áldozatot. A folyamat keretszáma rögzített, vagy külön allokációs politikával meghatározott. A lokális csere előnye az izoláció: egyetlen folyamat sem tudja kiszorítani a többi folyamat lapjait.

A hátránya a merevség: ha egy folyamatnak átmenetileg nagyobb munkakészletre van szüksége, nem tudja felhasználni a más folyamatok által nem használt szabad kereteket, még akkor sem, ha azok éppen kihasználatlanok.

Globális lapcsere esetén az áldozat kiválasztása az összes folyamat lapjai közül történik, így a folyamatok keretszáma dinamikusan változik. A globális csere jobban kihasználja a memóriát, mert a szabad kereteket oda lehet irányítani, ahol a legnagyobb szükség van rájuk. A hátránya az, hogy egy memóriaigényes folyamat elárasztathatja a rendszert, kiszoríthatja a többi folyamat lapjait, és thrashinget okozhat.

A két stratégia közötti különbség egy egyszerű példán szemléltethető. Tegyük fel, hogy három folyamat fut, munkakészletük mérete rendre 4, 6 és 8 lap, a rendelkezésre álló keretek száma 12. Globális csere esetén mindhárom folyamat versenyez a 12 keretért; ha az első folyamat hirtelen 10 lapra növeli a munkakészletét, a másik kettő keretei elapadhatnak. Lokális allokáció esetén, ha a kereteket 4, 4, 4 arányban osztjuk el, az utolsó folyamat már induláskor thrashingel, mert a munkakészlete 8 lap.

A gyakorlati rendszerek gyakran kombinálják a két megközelítést. Például a folyamatok kapnak egy minimális keretszámot (lokális garancia), a fennmaradó kereteket pedig globálisan osztják el. Ez biztosítja az izolációt és a rugalmasságot.

12.2.5 Konkrét algoritmusok: WSClock és PFF

A munkakészlet fogalmát közvetlenül felhasználó algoritmus a **Working Set Clock (WSClock)**. Ez a Clock algoritmus kiterjesztése: minden laphoz tartozik egy referenced bit és egy utolsó hivatkozási időpont. A mutató körbejár a lapokon; ha a referenced bit 1, akkor törli, és beállítja az utolsó hivatkozási időt a jelenlegi időre; ha a referenced bit 0, és a lap kora nagyobb, mint Δ, akkor a lap áldozat; különben átlépi. A pszeudokód:

```
page *clock_hand = ...;

while (true) {
    if (clock_hand->referenced) {
        clock_hand->referenced = false;
        clock_hand->last_use = current_time();
        clock_hand = next(clock_hand);
    } else if (current_time() - clock_hand->last_use > DELTA) {
        return clock_hand; // victim
    } else {
        clock_hand = next(clock_hand);
    }
}
```

A `DELTA` a munkakészlet ablakméretét jelöli. Az algoritmus lényege, hogy csak azokat a lapokat választja ki, amelyeket a munkakészlet ablakon kívül nem hivatkoztak. Így a munkakészlet lapjai védettek.

A **PFF algoritmus** a page fault rate alapján szabályozza a folyamatonkénti keretszámot. A fent bemutatott pszeudokódon túl a gyakorlati megvalósítás a következőket veszi figyelembe:

- A mérési intervallum rövid legyen (például 100 ms vagy 1 s), hogy a változásokra gyorsan reagáljon.
- A küszöbértékeket a háttértár sebességéhez kell igazítani; lassú lemeznél alacsonyabb felső küszöb szükséges.
- A keretallokáció és -felszabadítás fokozatos legyen, hogy elkerüljük a kilengéseket.

A PFF algoritmus előnye, hogy nem igényli a munkakészlet pontos mérését, csak a laphibák gyakoriságát. Hátránya, hogy a küszöbértékek beállítása kényes, és a mérési intervallum alatt a rendszer állapota megváltozhat.

12.2.6 Gyakorlati példa a munkakészlet és a thrashing kapcsolatára

Vegyünk egy rendszert 256 MiB fizikai memóriával és 512 MiB swap területtel. Négy folyamat fut, mindegyik egy-egy 64 MiB-os tömböt jár be véletlenszerűen. A munkakészletük egyenként 64 MiB, azaz összesen 256 MiB. Ha a rendszer mind a négy folyamatot egyszerre engedi futni, a fizikai memória éppen elegendő, de a swap használata elkerülhetetlen, mert a rendszer saját lapjai is memóriát igényelnek. Ha egy ötödik folyamat is elindul, az összesített munkakészlet 320 MiB-ra nő, ami meghaladja a fizikai memóriát. A rendszer elkezdi a lapok kiírását a swap-ra, majd visszaolvasását; a major fault rate hirtelen megnő, a CPU kihasználtsága leesik. Ez a thrashing.

A mérési naplóban az alábbi értékek jelenhetnek meg (a számok szemléltetőek, nem valós mérési eredmények):

Időintervallum	Aktív folyamatok	Major fault/s	CPU%	Swap be/s	Swap ki/s	Állapot
0–30 s	4	2	95	0	0	normál
30–60 s	5	150	40	120	110	thrash

A táblázatban a mért és a várt értékeket külön kell választani. A várt major fault rate a munkakészletből és a háttértár sebességéből becsülhető: ha a háttértár 10 ms alatt olvas be egy lapot, akkor egy folyamat másodpercenként legfeljebb 100 major faultot tud kiszolgálni. Ha a folyamatok ennél több laphibát generálnak, a rendszer túlterhelt.

12.2.7 Gyakori hibák és félreértések

A munkakészlet és a thrashing kapcsán több tipikus hiba fordul elő.

Túl kicsi vagy túl nagy ablakméret. A Δ megválasztása gyakran önkényes. Túl kicsi ablak esetén a munkakészlet nem fedi le a folyamat tényleges lokalitását, ezért a keretallokáció alulbecsli a szükségletet. Túl nagy ablak esetén a munkakészlet már a teljes címtartományt tartalmazza, és a modell elveszíti az előrejelző képességét.

A page fault rate önmagában nem elég. A minor és major faultokat meg kell különböztetni. A minor faultok gyakran olcsók, és nem jelzik a memória-túlterhelést. A thrashing diagnózisához a major fault rate és a swap forgalom együttes figyelése szükséges.

A thrashinget CPU-problémának nézik. Az alacsony CPU-kihasználtság és a magas várakozási idő könnyen tévesen CPU-teljesítmény-problémának tűnik. A valódi ok a memória-túlterhelés; a megoldás a folyamatok számának csökkentése vagy a memória bővítése.

A globális csere igazságtalansága. Globális csere esetén egy memóriaéhes folyamat kiszorít-hatja a többi folyamat lapjait, és thrashinget okozhat. A megoldás a folyamatonkénti minimális keretszám bevezetése, vagy a memórianyomás hatására a folyamatok felfüggesztése.

A munkakészlet statikus kezelése. A munkakészlet időben változik; a fázisváltások hirtelen változásokat okozhatnak. A statikus keretallokáció nem tudja követni ezt, ezért alkalmazkodó mechanizmusokra van szükség.

12.2.8 Labor: working set és thrashing elemzése

A labor célja a munkakészlet és a thrashing megfigyelése emulált környezetben. A labor nem valós gépen fut; minden művelet QEMU vendégben, eldobható lemezképpel történik. A hoszt valódi lemezét, bootloaderét és rendszerpartícióját tilos módosítani.

12.2.9 Kiinduló állapot és konfiguráció

- **Hoszt**: bármely Linux rendszer, QEMU 8.0 vagy újabb.
- **Vendég**: Linux 6.1 LTS kernel, initramfs rootfs, 256 MiB RAM, 2 vCPU, 512 MiB swap partíció.
- **Háttértár**: virtio-blk, a swap partíció külön lemezképen.
- **Mérőeszközök**: `perf stat`, `vmstat`, `sar -B` (ha elérhető), `/proc/vmstat`, `/proc/<pid>/stat`.

A QEMU indítási parancs (példa):

```
qemu-system-x86_64 \
  -m 256 \
  -smp 2 \
  -drive file=rootfs.img,format=raw,if=virtio \
  -drive file=swap.img,format=raw,if=virtio \
  -kernel vmlinuz-6.1.0 \
  -initrd initramfs.cpio.gz \
  -append "console=ttyS0 root=/dev/vda rw swap=/dev/vdb"
```

A `rootfs.img` és a `swap.img` eldobható képek, a `vmlinuz-6.1.0` és az `initramfs.cpio.gz` a reprodukciós csomag része.

12.2.10 Memóriaterhelő program

A vendégben egy C program fut, amely egy 64 MiB-os tömböt jár be véletlenszerűen, hogy munkakészletet generáljon. A program méri a futási időt és a major faultok számát a `/proc/self/stat` mezőiből. A kód szemléltetés, nem éles rendszeren futtatandó:

```
#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>
#include <sys/time.h>
#include <unistd.h>

#define SIZE (64UL * 1024 * 1024)
#define PAGE 4096
#define ACCESSES 200000

int main(void) {
    volatile uint8_t *buf = malloc(SIZE);
    if (!buf) { perror("malloc"); return 1; }

    struct timeval start, end;
    gettimeofday(&start, NULL);

    for (int i = 0; i < ACCESSES; i++) {
        uint64_t idx = ((uint64_t)rand() * PAGE) % SIZE;
        buf[idx] ^= 0x1;
    }

    gettimeofday(&end, NULL);
```

```
    double sec = (end.tv_sec - start.tv_sec) +
                 (end.tv_usec - start.tv_usec) / 1e6;
    printf("runtime: %.3f s\n", sec);
    /* A major faultok a /proc/self/stat 12. mezőjéből olvashatók */
    return 0;
}
```

A méréshez a programot `perf stat -e page-faults,minor-faults,major-faults` alatt is futtathatjuk.

12.2.11 Mérési eljárás

1. Indítsuk el a QEMU vendéget a fenti konfigurációval.
2. A vendégben fordítsuk le a memóriaterhelő programot.
3. Futtassuk **egyetlen** példányát, és rögzítsük a major faultok számát, a CPU kihasználtságot és a futási időt.
4. Indítsunk további példányokat párhuzamosan, amíg a thrashing jelei megjelennek.
5. Minden 30 másodperces intervallumban gyűjtsük a következőket: aktív folyamatok száma, major fault/s, CPU%, `pswpin/s`, `pswpout/s`.
6. A mérési naplóba minden értéket rögzítsünk, és külön oszlopban tüntessük fel a várt értéket.

12.2.12 Mérési napló minta

Idő (s)	Aktív folyamatok	Major fault/s (mért)	CPU% (mért)	Swap be/s (mért)	Swap ki/s (mért)	Várt állapot
0–30	1	2	95	0	0	normál
30–60	2	5	90	0	0	normál
60–90	3	8	85	10	12	közelítő
90–120	4	30	70	60	55	feszített
120–150	5	150	40	120	110	thrash

A várt állapot a munkakészletből számítható: 4 folyamat $\times$ 64 MiB = 256 MiB, ami a fizikai memória határa. Az ötödik folyamat belépésekor az összesített munkakészlet 320 MiB, ami meghaladja a fizikai memóriát, ezért thrashing várható.

12.2.13 Ellenőrzés és értékelés

A hallgató a mért major fault rate-et összehasonlítja a várt értékkel. A várt érték a háttértár sebességéből becsülhető: ha a swap olvasási késleltetése 10 ms, akkor egy folyamat másodpercenként legfeljebb 100 major faultot tud kiszolgálni. Ha a mért érték ennél magasabb, a rendszer túlterhelt.

Az önellenőrző kérdések:

1. Definiálja a munkakészletet, és magyarázza el, hogyan változik a mérete, ha a Δ ablakot növeljük.
2. Mi a különbség a lokális és a globális lapcsere között? Melyik biztosít jobb izolációt, és miért?
3. Hogyan ismerhető fel a thrashing a rendszer-metrikákból? Milyen téves diagnózis gyakori, és miért?

A labor reprodukciós csomagja tartalmazza a QEMU parancsot, a lemezképeket, a memóriaterhelő program forrását, a mérési scriptet és a mérési naplót. A csomagban a mért és a várt értékeket külön kell jelölni.

12.2.14 Összefoglalás és továbbvivő eredmények

Ebben az alfejezetben a munkakészlet fogalmából indultunk ki, és megvizsgáltuk a page fault rate és a thrashing közötti kapcsolatot. A lokális és globális lapcsere stratégiákat összehasonlítottuk, és bemutattuk a WSClock és a PFF algoritmust. A thrashing elkerülésének kulcsa a folyamatok munkakészletének és a rendelkezésre álló fizikai memóriának az egyensúlya.

A következő alfejezet (Huge page, THP és memóriatömörítés) erre az alapra épít: a huge page, a transparent huge page (THP) és a memóriatömörítés mind olyan technikák, amelyek a munkakészlet hatékonyabb memóriában tartását célozzák. Az mmap és az osztott memória pedig a munkakészlet fogalmát a fájlrendszer és a folyamatok közötti memóriaterület szintjén használja. A munkakészlet modell tehát nemcsak a lapcsere, hanem a virtuális memória számos további területének alapfogalma.

12.3 Huge page, THP és memóriatömörítés

A virtuális memória eddigi tárgyalásában a lapméretet gyakran rögzített, tipikusan 4 KiB-os egységként kezeltük. Ez a megközelítés matematikailag egyszerű, a valós rendszerekben azonban a CPU-k és a memóriakezelők már régóta többféle lapméretet támogatnak. A nagy lapok (huge page) és a memóriatömörítés (memory compression) két olyan technika, amely a lapozás és a lapcserélés hatékonyságát alapvetően befolyásolja. Az előbbi a TLB-lefedettség növelésével csökkenti a címfordítás költségét, az utóbbi a memórianyomás enyhítésével késlelteti vagy elkerüli a lassú blokk-csereeszköz használatát.

12.3.1 A nagy lapok motivációja: TLB-lefedettség

A címfordítási folyamat során a CPU a virtuális címből fizikai címet állít elő. A többszintű laptáblák bejárása minden egyes memóriahivatkozásnál több memóriaolvasást igényelne, ezért a modern processzorok fordítási gyorsítótárat, TLB-t (Translation Lookaside Buffer) használnak. A TLB a nemrég használt virtuális-fizikai címleképezések kis méretű, gyors asszociatív tárja.

A TLB mérete erősen korlátozott – tipikusan néhány száz vagy néhány ezer bejegyzés magonként. Ha egy lap mérete 4 KiB, akkor egy 4 MiB-nyi memóriát érintő adathalmaz címfordításához 1024 TLB-bejegyzés szükséges. Ugyanez a memóriaterület 2 MiB-os nagy lappal mindössze 2 bejegyzést foglal. Ezt a jelenséget nevezzük TLB-lefedettségnek (TLB reach): azt a memóriaterületet, amelyet a TLB teljes tartalma egyszerre lefedni képes. A TLB-lefedettség formálisan:

$$\text{TLB reach} = \text{TLB bejegyzéseinek száma} \times \text{lapméret}$$

Egy 512 bejegyzéses L2 TLB-vel rendelkező processzornál 4 KiB-os lapokkal a TLB-lefedettség 2,0 MiB, míg 2 MiB-os nagy lapokkal 1,0 GiB. A különbség három nagyságrend, ami a memóriaigényes alkalmazások – adatbázis-kezelők, tudományos számítások, nagy hash-táblák, virtuális gépek – esetében döntő teljesítménytényező. Amikor a munkahalmaz túlnyúlik a TLB-lefedettségen, TLB-miss következik be, és a CPU-nak a több szintű laptáblát kell bejárnia. Ez a laptábla-bejárás (page walk) önmagában is több memóriahozzáférést jelent, ami processzoronként és terhelési profilonként akár 10–30%-os futásiidő-növekedést is okozhat.

12.3.2 Nagy lapok az x86-64 és ARM64 architektúrán

Az x86-64 architektúra a klasszikus 4 KiB-os lapok mellett 2 MiB-os és 1 GiB-os lapokat támogat. A lapméret a laptábla-hierarchiából következik: a négyszintű modellben a PMD (Page Middle Directory) bejegyzés közvetlenül is leképezhet egy 2 MiB-os lapot, ha a PS (Page Size) bit be van állítva. Az 1 GiB-os lapok a PUD (Page Upper Directory) szinten érhetők el. Az ARM64 hasonló elven működik, jellemzően 4 KiB, 64 KiB, 2 MiB és 1 GiB lapméretekkel, a konkrét készlet pedig a megvalósítástól és a kernelkonfigurációtól függ.

A nagy lapok használata nemcsak a TLB-t kíméli, hanem a laptábla-mélységet is csökkenti. Egy 4 KiB-os lap leképezése négyszintű bejárást igényel: PGD → PUD → PMD → PTE. Egy 2 MiB-os lap esetén a bejárás háromszintű: PGD → PUD → PMD. Ez TLB-miss esetén kevesebb memóriahozzáférést jelent, tehát a TLB-miss büntetése is kisebb.

12.3.3 Explicit huge page és a hugetlbfs

A Linux kernel két fő mechanizmust kínál a nagy lapok használatára. Az első a **hugetlbfs**, az explicit nagy lapok fájlrendszere, amely a rendszergazda által előre lefoglalt, rögzített méretű nagy lapokat kezeli. A hugetlbfs lapjai nem vesznek részt a hagyományos lapcserében; a lefoglalt memória addig a kernel tulajdonában marad, amíg a rendszergazda explicit módon fel nem szabadítja. Ez a merevség előny és hátrány is: a memória garantáltan rendelkezésre áll, a valós idejű alkalmazásoknál a determinizmus javul, de a kihasználatlan nagy lapok elpazarolt memóriát jelentenek.

A hugetlbfs konfigurációja a sysfs-en keresztül történik. A 2 MiB-os nagy lapok számát például a következő paranccsal lehet megvizsgálni és módosítani:

```
# cat /sys/kernel/mm/hugepages/hugepages-2048kB/nr_hugepages
0
# echo 512 > /sys/kernel/mm/hugepages/hugepages-2048kB/nr_hugepages
```

A fenti példa 512 darab, egyenként 2 MiB-os nagy lapot foglal le, összesen 1 GiB memóriát. A memóriafoglalás csak akkor lesz sikeres, ha a kernel elegendő fizikailag folytonos szabad memóriát talál. A fragmentált rendszerben ez a feltétel nem teljesül automatikusan.

Az alkalmazás a `mmap()` rendszerhívás `MAP_HUGETLB` flagjével, vagy a hugetlbfs fájlrendszeren létrehozott fájlon keresztül férhet hozzá a nagy lapokhoz. A következő C-példa szemlélteti a hugetlbfs `mmap()` használatát:

```c
/* Helyőrző példa: hugetlbfs mmap használata
 * Fordítás: cc -o hmap hmap.c
 * Futtatás: ./hmap /mnt/huge/testfile
 */
#include <sys/mman.h>
#include <fcntl.h>
#include <unistd.h>
#include <stdio.h>
#include <string.h>

int main(int argc, char **argv)
{
    if (argc != 2) {
        fprintf(stderr, "használat: %s <hugetlbfs_fájl>\n", argv[0]);
        return 1;
    }

    int fd = open(argv[1], O_CREAT | O_RDWR, 0600);
```

```
    if (fd < 0) {
        perror("open");
        return 1;
    }

    char *addr = mmap(NULL, 2 * 1024 * 1024,
                      PROT_READ | PROT_WRITE,
                      MAP_SHARED | MAP_HUGETLB, fd, 0);
    if (addr == MAP_FAILED) {
        perror("mmap MAP_HUGETLB");
        return 1;
    }

    memset(addr, 'A', 2 * 1024 * 1024);
    printf("nagy lapos leképezés sikeres: %p\n", addr);

    munmap(addr, 2 * 1024 * 1024);
    close(fd);
    return 0;
}
```

A példa kód szemléltetés, nem hajtható végre a hoszt rendszeren a hugetlbfs mountolása és a megfelelő jogosultságok nélkül. A mountolás tipikusan a következő:

```
# mkdir -p /mnt/huge
# mount -t hugetlbfs -o pagesize=2M none /mnt/huge
```

12.3.4 Transparent Huge Pages (THP)

A hugetlbfs explicit kezelése adminisztratív terhet jelent, és az alkalmazásokat is módosítani kell a kihasználásához. A **Transparent Huge Pages** (THP, átlátszó nagy lapok) mechanizmus ezt a problémát oldja meg: a kernel automatikusan, az alkalmazás tudta nélkül próbál 2 MiB-os nagy lapokat használni a hagyományos 4 KiB-os lapok helyett.

A THP két fő működési módja a Linux rendszerben:

1. **Always** (`always`): A kernel minden arra alkalmas anonim leképezésnél megkísérli a nagy lap használatát. Ez a mód a memóriaigényes, nagy munkahalmazzal dolgozó szervereknél általában teljesítményjavulást hoz, de a nagy lapok lefoglalása és fenntartása többletmunkát igényel a kerneltől.
2. **Madvise** (`madvise`): A kernel csak akkor használ nagy lapot, ha az alkalmazás explicit módon kéri a `madvise()` rendszerhíváson keresztül, `MADV_HUGEPAGE` paraméterrel. Ez a mód kontrolláltabb, de az alkalmazásnak tudnia kell a nagy lapok előnyeiről.

A THP beállítása futásidőben a sysfs-en keresztül lehetséges:

```
# cat /sys/kernel/mm/transparent_hugepage/enabled
always [madvise] never

# echo always > /sys/kernel/mm/transparent_hugepage/enabled
```

A THP foglalás során a kernel egy **khugepaged** nevű kernel-szálat is futtat, amely aszinkron módon keresi az összefüggő, 4 KiB-os lapokból álló régiókat, és azokat 2 MiB-os nagy lapokká vonja össze. Ez a “collapsing” folyamat a memória töredezettségét csökkenti, de a khugepaged CPU-időt is fogyaszt. A `defrag` beállítás azt szabályozza, hogy a kernel mennyire agresszíven végezzen memóriadefragmentálást a nagy lapok lefoglalása érdekében:

```
# cat /sys/kernel/mm/transparent_hugepage/defrag
always defer defer+madvise madvise never
```

A `defer` érték azt jelenti, hogy a kernel a háttérben, aszinkron módon próbálkozik a defragmentálással, míg az `always` esetén a foglalási kérelem szinkron defragmentálást is elindíthat. A `madvise` érték csak az explicit kérést kapott régióknál defragmentál.

12.3.5 THP a gyakorlatban

A THP által nyújtott teljesítményjavulás nem univerzális. A döntés, hogy egy adott rendszeren bekapcsoljuk-e a THP-t, a munkaterhelés jellemzőitől függ. A memóriaigényes, nagy, összefüggő memóriaterületeket használó alkalmazások – adatbázis-kezelők, virtuális gépek, HPC-alkalmazások – jellemzően profitálnak a nagy lapokból. A sok kis, rövid életű foglalást végző alkalmazások esetén a THP többletköltsége – a nagy lapok lefoglalása, a khugepaged futása, a copy-on-write nagy lapjainak felosztása – meghaladhatja a TLB-megtakarítást.

Gyakori gyakorlati hiba, hogy a rendszergazda a THP-t minden rendszeren `always` módban hagyja, és a memóriafoglalási késleltetés megnövekedését észleli. A nagy lap lefoglalása ugyanis fizikailag folytonos 2 MiB-os régiót igényel. Ha a memória fragmentált, a kernelnek először defragmentálnia kell, ami szinkron módban akár több száz milliszekundumos késleltetést is okozhat. Ez különösen a valós idejű rendszereknél és a latency-érzékeny alkalmazásoknál elfogadhatatlan. Ilyen környezetben a `madvise` mód vagy a THP teljes kikapcsolása javasolt.

A THP hatásának méréséhez a `/proc/meminfo` adatai nyújtanak alapot:

```
$ grep -E 'Huge|AnonHuge' /proc/meminfo
AnonHugePages:   1048576 kB
HugePages_Total:     512
HugePages_Free:      512
HugePages_Rsvd:        0
HugePages_Surp:        0
Hugepagesize:       2048 kB
```

Az `AnonHugePages` a THP által anonim memóriaként használt nagy lapok összméretét mutatja. A `HugePages_Total` és `HugePages_Free` a hugetlbfs explicit nagy lapjaira vonatkozik. Fontos megkülönböztetni a két mechanizmust: az `AnonHugePages` a THP-ből, a `HugePages_*` a hugetlbfs-ből származik.

12.3.6 THP és copy-on-write

A THP egyik neuralgikus pontja a fork() és a copy-on-write (COW) kölcsönhatása. Amikor egy folyamat fork() hívással gyermekfolyamatot hoz létre, a memóriaoldalakat a kernel kezdetben írásvédettként osztja meg a szülő és a gyermek között. Ha bármelyik fél írni próbál, a CPU oldalhibát generál, és a kernel lemásolja a lapot. A THP esetében ez a COW-művelet 2 MiB-os lapokra vonatkozik. A nagy lap COW-ja során a kernelnek döntenie kell: másolja a teljes 2 MiB-os lapot, vagy bontsa fel 4 KiB-os lapokra, és csak a szükséges 4 KiB-ot másolja.

A Linux kernel a második megoldást választja: a COW során a nagy lapot felbontja (split), és a gyermek a 4 KiB-os lapokkal folytatja. Ez a megközelítés minimalizálja a másolt memória mennyiségét, de a TLB-lefedettség csökkenéséhez vezet. A fork() utáni nagy lapok “elvesznek”, és a folyamat a 4 KiB-os lapokra tér vissza. Ezért a fork()-ot intenzíven használó alkalmazásoknál – például a web szervereknél, amelyek minden kérésnél forkolnak – a THP előnye korlátozott lehet.

12.3.7 Memóriatömörítés

A lapcserélés hagyományos célja, hogy a memórianyomás hatására a ritkán használt lapokat a kernel egy blokk-csereeszközre (swap) írja ki, felszabadítva a fizikai memóriát az aktív lapok számára. A csereeszköz azonban – legyen az HDD, SSD vagy NVMe – mindig lassabb a DRAM-nál. Egy 4 KiB-os lap kiírása és visszaolvasása egy tipikus SSD-n körülbelül 100–500 μs, míg a DRAM-hozzáférés 100–200 ns. A különbség három nagyságrend.

A **memóriatömörítés** (memory compression) ezt a különbséget hidalja át: ahelyett, hogy a lapot a lassú csereeszközre írnánk, a kernel a lapot tömöríti, és a tömörített adatot a DRAM egy erre fenntartott területén tárolja. Ha egy 4 KiB-os lap tömörített mérete 2 KiB, akkor a tömörített tárolás révén a rendszer 2-szeres "látszólagos" memóriabővítést kap, miközben a visszaolvasás sebessége a DRAM-hozzáférés nagyságrendjében marad, nem a blokk-csereeszközén.

A tömörítés hatékonysága az adattartalomtól függ. A szöveges adatok, a kód, a nulla byte-okból álló régiók jól tömöríthetők – a valós rendszereknél a 2–4-szeres tömörítési arány a tipikus. A már tömörített adatok – JPEG, PNG, titkosított adat – gyakorlatilag nem tömöríthetők tovább; ilyenkor a tömörítés csak CPU-időt pazarol. A kernel tömörítési algoritmusai – a Linux kernelben például LZO, LZ4, ZSTD – az átbocsátás és a tömörítési arány közötti kompromisszumot képviselik. Az LZ4 gyors, de kevésbé tömörít; a ZSTD nagyobb tömörítési arányt ér el, de több CPU-időt igényel.

12.3.8 zswap és zram

A Linux kernel két fő memóriatömörítési mechanizmust kínál. A **zswap** egy tömörített gyorsítótár a blokk-csereeszköz előtt. Amikor a kernel egy lapot ki akar írni a swapra, a zswap először megpróbálja tömöríteni és a DRAM-ban tartani. Ha a tömörített lap nem fér be a zswap által fenntartott memóriapoolba, vagy a tömörítés nem hoz elegendő nyereséget, a lap a hagyományos módon a csereeszközre kerül. A zswap így a meglévő swap-rendszer kiegészítője, nem helyettesítője.

A zswap fontos jellemzője, hogy a tömörített lapokat a kernel LRU-sorrendben tartja nyilván. Amikor a memórianyomás csökken, a zswap-ból a lapok visszakerülhetnek a hagyományos memóriába. Ha a zswap pool megtelik, a legrégebben használt tömörített lapok kerülnek a csereeszközre. Ez a hierarchikus lapcserélés – DRAM → zswap tömörített DRAM → swap – a lapcsere-költséget fokozatosan növeli, elkerülve a hirtelen teljesítményesést.

A zswap beállításai közül a tömörítési algoritmus és a pool mérete a legfontosabb:

```
# echo 1 > /sys/module/zswap/parameters/enabled
# echo zstd > /sys/module/zswap/parameters/compressor
# echo 50 > /sys/module/zswap/parameters/max_pool_percent
```

A `max_pool_percent` érték azt határozza meg, hogy a teljes DRAM hány százalékát foglalhatja el a zswap tömörített pool. A fenti példában ez 50%, ami azt jelenti, hogy egy 16 GiB-os rendszeren a zswap legfeljebb 8 GiB-ot használhat. A beállítás túl alacsony értéknél a zswap hamar megtelik, és a lapok a lassú swapra kerülnek. A túl magas érték pedig a hasznos, aktív lapok elől veszi el a memóriát.

A **zram** ezzel szemben egy virtuális blokk-eszköz, amely a tömörített adatot a DRAM-ban tárolja. A zram-eszköz swap-partícióként funkcionál: a kernel a hagyományos swap-mechanizmuson keresztül írja ki a lapokat, de a mögöttes tároló nem egy fizikai lemez, hanem a tömörített DRAM. A zram nem igényel fizikai csereeszközt, ami a beágyazott rendszerek és a gyors rendszerindítást

igénylő mobil eszközök esetében előnyös. Az Android operációs rendszer a zram-ot használja swap-tárolóként.

A zram eszköz létrehozása és beállítása:

```
# modprobe zram num_devices=1
# echo zstd > /sys/block/zram0/comp_algorithm
# echo 4G > /sys/block/zram0/disksize
# mkswap /dev/zram0
# swapon /dev/zram0
```

A fenti parancsok egy 4 GiB-os zram-eszközt hoznak létre ZSTD tömörítéssel. A zram eszköz `disksize` értéke a tömörítetlen, logikai méret, nem a tényleges DRAM-fogyasztás. Ha a tömörítési arány átlagosan 3:1, akkor a 4 GiB-os zram-eszköz legfeljebb körülbelül 1,3 GiB DRAM-ot foglal.

12.3.9 A tömörítés és a swap hierarchiája

A memóriatömörítés nem helyettesíti a hagyományos swap-ot, hanem kiegészíti azt. A hierarchikus lapcserélési lánc a következő:

Szint	Tároló	Hozzáférési idő (nagyságrend)	Kapacitás
1	DRAM, aktív lapok	100–200 ns	Fizikai DRAM
2	DRAM, zswap/zram tömörített	1–10 μs (tömörítés + másolás)	Tömörített pool
3	NVMe SSD swap	100–500 μs	Swap partíció
4	HDD swap	5–15 ms	Swap partíció

A memórianyomás fokozatos növekedésével a kernel először a zswap/zram-ba írja a lapokat. Amikor a tömörített pool megtelik, a legrégebbi tömörített lapok a fizikai csereeszközre kerülnek. Ez a rétegzett megközelítés a lapcsere-költséget fokozatosan növeli, elkerülve a hirtelen, nagyságrendnyi teljesítményesést. A teljesítménygörbe így nem lépcsős, hanem folyamatos.

Fontos megérteni, hogy a tömörítés CPU-időt is igényel. A LZ4 tömörítése és visszafejtése 4 KiB-os laponként körülbelül 1–5 μs, a ZSTD ennél lassabb lehet, de a tömörítési arány is jobb. A teljes költség akkor pozitív, ha a tömörítés + DRAM-másolás + visszafejtés együttes ideje kisebb, mint a blokk-csereeszközre írás + olvasás ideje. Az SSD-knél ez a feltétel általában teljesül, a nagyon gyors NVMe-knél a különbség kisebb, de a tömörítés továbbra is előnyös a kopás és az energiafogyasztás szempontjából.

12.3.10 Memórianyomás és a tömörítés hatása a working setre

Az előző alfejezet (Working set, thrashing és lokális/globális csere) a working set modelljét és a thrashing jelenségét tárgyalta. A memóriatömörítés a working set kezelésére is hatással van. A hagyományos modellben a thrashing akkor következik be, amikor a working set mérete meghaladja a fizikai memóriát, és a kernel folyamatosan lapcserél. A tömörítés ezt a küszöböt tolja feljebb: ha a working set tömöríthető, akkor a hatékony memóriaméret a tömörítési arány faktorával megnő.

Példaként vegyünk egy rendszert 8 GiB fizikai DRAM-mal, ahol a zswap átlagos tömörítési aránya 2:1, és a zswap pool maximuma 2 GiB. A hatékony memóriaméret a working set szempontjából:

$$\text{hatékony memória} = 8\,\text{GiB} + 2\,\text{GiB} \times 2 = 12\,\text{GiB}$$

Ez a számítás leegyszerűsítő, mert a tömörítési arány laponként változik, és a zswap pool maximális mérete a DRAM-ból vonódik el. De közelítőleg azt szemlélteti, hogy a tömörítés miként növeli a thrashing küszöbét. A 12 GiB-nál kisebb working set esetén a rendszer elkerülheti a blokk-csereeszközre való írást; a thrashing csak e küszöb felett jelentkezik.

A tömörítés azonban nem csodaszer. A nem tömöríthető working set – például videó-adat, titkosított adatbázis, már tömörített képek – esetén a tömörítési arány 1:1-hez közeli, és a zswap/zram csak a CPU-időt pazarolja. A kernel ezt részben kompenzálja azzal, hogy a nem tömöríthető lapokat közvetlenül a csereeszközre írja, a tömörítési kísérlet költségét pedig a rossz tömörítési arány észlelésekor minimalizálja.

12.3.11 Labor: THP és zswap beállítási döntés

12.3.12 Cél

A labor célja, hogy egy emulált Linux rendszerben megmérjük a THP és a zswap hatását egy egyszerű, replikálható munkaterhelésre, és a mérési adatok alapján indokolt beállítási döntést hozzunk. A laborban a memóriaigényes munkaterhelést egy C-ben írt, nagy tömböt járó program reprezentálja. A mérés során a futási időt, a TLB-miss arányt és a laphibák számát rögzítjük.

12.3.13 Kiinduló állapot és konfiguráció

A labor QEMU alatt futó, minimális Debian vagy Alpine Linux vendéggel végezhető. A QEMU konfiguráció:

```
qemu-system-x86_64 \
  -m 4G \
  -smp 4 \
  -cpu max \
  -drive file=vm.qcow2,format=qcow2 \
  -kernel /boot/vmlinuz \
  -append "root=/dev/sda1 console=ttyS0" \
  -nographic
```

A vendégben a THP állapotának ellenőrzése:

```
# cat /sys/kernel/mm/transparent_hugepage/enabled
always [madvise] never
```

A zswap állapota:

```
# cat /sys/module/zswap/parameters/enabled
N
```

Induláskor a THP `madvise` módban, a zswap kikapcsolva. A méréshez szükséges a `perf` eszköz, a build-lánc és a megfelelő jogosultságok.

12.3.14 A munkaterhelés

A munkaterhelés egy C program, amely egy 1 GiB-os anonim memóriaterületet foglal le `mmap()`-pel, és a területet determinisztikus lépésközzel járja be. A lépésköz változtatható, hogy a munkahalmaz a TLB-lefedettségen belül vagy azon kívül legyen. A program a bejárás előtt a lapokat `madvise()`-szal megjelöli, ha a THP `madvise` módban fut, és a futási időt `clock_gettime()`-mal méri.

```
/* Helyőrző példa: THP/zswap mérőprogram
 * Fordítás: cc -O2 -o memwalk memwalk.c
 * Futtatás: ./memwalk <mib> <lépés_byte>
 */
#define _GNU_SOURCE
#include <sys/mman.h>
#include <time.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main(int argc, char **argv)
{
    if (argc != 3) {
        fprintf(stderr, "használat: %s <mib> <lépés_byte>\n", argv[0]);
        return 1;
    }

    size_t size_mib = strtoull(argv[1], NULL, 10);
    size_t stride = strtoull(argv[2], NULL, 10);
    size_t total = size_mib * 1024 * 1024;

    char *buf = mmap(NULL, total, PROT_READ | PROT_WRITE,
                     MAP_PRIVATE | MAP_ANONYMOUS, -1, 0);
    if (buf == MAP_FAILED) {
        perror("mmap");
        return 1;
    }

    /* THP kérése madvise módban; az egyszerűség kedvéért
     * az iget itt megkerüli a hibakezelést. */
    madvise(buf, total, MADV_HUGEPAGE);

    struct timespec t0, t1;
    clock_gettime(CLOCK_MONOTONIC, &t0);

    volatile char sink = 0;
    for (size_t i = 0; i < total; i += stride) {
        sink += buf[i];
    }

    clock_gettime(CLOCK_MONOTONIC, &t1);

    double elapsed = (t1.tv_sec - t0.tv_sec) +
                     (t1.tv_nsec - t0.tv_nsec) / 1e9;
    printf("futási idő: %.6f s (mib=%zu stride=%zu)\n",
           elapsed, size_mib, stride);

    munmap(buf, total);
    return 0;
}
```

A program a mérés során a `sink` változóval kényszeríti a fordítót, hogy a memóriaolvasást ne optimalizálja ki. A `volatile` kulcsszó itt a mérési cél érdekében szükséges, nem kernelkódban

használt, időzítés-kritikus minta.

12.3.15 A mérési protokoll

A mérést három konfigurációban kell elvégezni:

1. **Baseline**: THP `never`, zswap kikapcsolva.
2. **THP always**: THP `always`, zswap kikapcsolva.
3. **THP madvise + zswap**: THP `madvise`, zswap bekapcsolva LZ4 tömörítéssel, 20% pool-maximummal.

Minden konfigurációban a munkaterhelést háromszor futtatjuk, és a medián futási időt rögzítjük. A mérések között a vendéget újraindítjuk, hogy a memóriaállapot ne befolyásolja a következő mérési pontot.

A mérési napló formátuma:

```
[2025-01-15 10:22:01] config=baseline run=1 mib=1024 stride=4096 elapsed=2.314512
[2025-01-15 10:22:04] config=baseline run=2 mib=1024 stride=4096 elapsed=2.301887
[2025-01-15 10:22:07] config=baseline run=3 mib=1024 stride=4096 elapsed=2.298731
[2025-01-15 10:22:10] config=baseline median=2.301887
```

12.3.16 Várt eredmények

A várt eredmény a TLB-lefedettség és a lapméret összefüggéséből vezethető le. A baseline konfiguráció 4 KiB-os lapokkal, 1 GiB munkahalmaz mellett a TLB-lefedettséget (jellemzően 2 MiB) messze meghaladja. A THP `always` konfigurációban 2 MiB-os lapokkal a TLB-lefedettség 1 GiB körüli, tehát a munkahalmaz éppen a lefedettségen belül van. A várt futásiidő-különbség a THP javára 10–30% a 4 KiB-os lépésköz esetén. Ha a lépésközt 2 MiB-ra növeljük, a munkahalmaz a THP TLB-lefedettségén is kívül kerül, és a különbség csökken.

A zswap esetén a várt hatás a futási időben nem közvetlenül mérhető, mert a munkaterhelés a memóriát elférő 1 GiB-os területet egyszer járja be. A zswap hatásának mérése egy 6 GiB-os munkaterhelésnél, 4 GiB-os vendégnél mutatkozik meg, ahol a tömörített swap-re írás elkerüli a blokk-csereeszköz használatát. A labor első változata a THP-hatásra fókuszál; a zswap mérése bővíthető.

12.3.17 Mérési napló és kiértékelés

A mérés után a naplót elemezni kell. Ha a mért futási idő a THP `always` konfigurációban jelentősen jobb a baseline-nál, akkor a THP bekapcsolása indokolt a vizsgált munkaterhelésnél. Ha a javulás elmarad a várt 10–30%-tól, az a TLB-miss arány mérésével tovább elemezhető: a `perf stat -e dTLB-load-misses,page-faults ./memwalk 1024 4096` parancs megmutatja, hogy a TLB-miss arány valóban csökkent-e.

A döntési mátrix a labor eredménye alapján:

Munkaterhelés	THP `always`	THP `madvise`	THP `never`
Nagy, folytonos anonim memória	előnyös	előnyös, alkalmazás-kontrollal	nem előnyös

Munkaterhelés	THP `always`	THP `madvise`	THP `never`
Sok apró, rövid életű foglalás	kockázatos (fragmentáció, defrag-késés)	semleges	biztonságos
Valós idejű, latency-érzékeny	kerülendő	kerülendő	ajánlott

A zswap döntési mátrix:

Helyzet	zswap be	zswap ki
Van gyors SSD swap	opcionális, mérsékelt előny	elfogadható
Csak HDD swap elérhető	ajánlott	kerülendő
Nincs fizikai swap	nincs értelme (zram javasolt)	nincs swap
CPU-korlátos, kevés tömöríthető adat	kerülendő	ajánlott

12.3.18 Visszaállítás

A labor után a vendéget az alapállapotba kell visszaállítani: a THP-t `madvise` módra, a zswap-ot kikapcsolásra, a hugetlbfs lapokat nullára. A mérési adatokat és a konfigurációt a reprodukciós csomagban kell rögzíteni. A QEMU vendég pillanatképe visszaállítható, így a labor nem érinti a hoszt rendszer állapotát.

12.3.19 Tipikus hibák és tévhitek

12.3.20 1. “A THP mindig gyorsabb”

Ez a leggyakoribb tévhit. A THP előnye a TLB-miss csökkentéséből származik, de a nagy lapok lefoglalása, a defragmentálás és a COW-felbontás többletköltséggel jár. A mérési adatok nélküli `always` mód bekapcsolása a latency-érzékeny rendszereknél kifejezetten káros lehet. A döntést munkaterhelés-profil és mérési adatok alapján kell meghozni.

12.3.21 2. “A hugetlbfs és a THP ugyanaz”

A két mechanizmus különbözik. A hugetlbfs explicit, az alkalmazás által kezdeményezett, rögzített nagy lapokat biztosít, amelyek nem vesznek részt a swapban. A THP átlátszó, a kernel által automatikusan kezelt nagy lapokat jelent, amelyeknek a swap-viselkedése is eltér. A kettő keverése a mérési naplókban hibás következtetéshez vezet.

12.3.22 3. “A tömörítés helyettesíti a swap-ot”

A tömörítés kiegészíti, de nem helyettesíti a blokk-csereeszközt. Ha a tömörített pool megtelik, a lapok a swapra kerülnek. A zswap konfigurációjában a `max_pool_percent` érték helytelen beállítása – túl alacsony – azt okozhatja, hogy a lapok hamarabb a lassú swapra kerülnek, mint szükséges lenne. A tömörítés hatékonysága a munkaterhelés tömöríthetőségétől is erősen függ; nem tömöríthető adatnál a tömörítés csak CPU-terhelést okoz.

12.3.23 4. “A TLB-miss nem mérhető”

A modern Linux rendszereken a `perf` eszköz hardveres számlálókat biztosít a TLB-miss eseményekhez. Az `x86` architektúrán a `dTLB-load-misses` és `dTLB-store-misses` események a TLB-miss arány mérését teszik lehetővé. A mérési naplóban a futási idő mellett a TLB-miss számot is rögzíteni kell ahhoz, hogy a THP hatását elkülönítsük a többi tényezőtől.

12.3.24 Összefoglalás

A nagy lapok és a memóriatömörítés a virtuális memória-kezelés két, egymástól független, de a memóriateljesítményt egyaránt alapvetően befolyásoló technikája. A nagy lapok – legyenek explicit hugetlbfs vagy átlátszó THP – a TLB-lefedettség növelésével csökkentik a címfordítás költségét, különösen a nagy, folytonos memóriaterületeket használó munkaterheléseknél. A memóriatömörítés – zswap, zram – a lapcsere-költséget csökkenti azáltal, hogy a lapokat a lassú blokk-csereeszköz helyett a DRAM-ban, tömörített formában tárolja.

Mindkét technika beállítási döntése méréseken alapuló mérnöki feladat. A THP `always`, `madvise` és `never` módjai közötti választás a munkaterhelés memória-elérési mintájától, a fragmentáció mértékétől és a latency-követelményektől függ. A zswap és a zram a memórianyomást és a swap-hierarchiát egészítik ki, a tömörítési algoritmus és a pool méretének helyes megválasztása a tömöríthetőségi profilon és a CPU-kapacitáson múlik.

A mérési alapú döntéshozatal a következő fejezetben is központi szerepet játszik majd, amikor a memóriatérképezés, az `mmap()` és az osztott memória kérdéseit tárgyaljuk. A THP és a tömörítés itt bevezetett fogalmai az `mmap()` alapú leképezések viselkedését is közvetlenül befolyásolják, ezért az itt tárgyalt mérési módszerek és beállítási szempontok a következő egység előfeltételét képezik.

12.3.25 Ellenőrző kérdések

1. Egy rendszer 4 KiB-os lapokkal dolgozik, a TLB 512 bejegyzést tartalmaz. Mekkora a TLB-lefedettség? Hogyan változik a lefedettség, ha a rendszer 2 MiB-os nagy lapokra vált?
2. Miért okozhat késleltetést a THP `always` módja töredezett memóriájú rendszerben? Melyik alternatív beállítás csökkenti ezt a kockázatot?
3. A zswap és a zram közötti alapvető különbséget foglald össze egy-egy mondatban. Mikor indokolt a zram választása a zswap helyett?

12.4 Memóriatérképezés, mmap és osztott memoria

A virtuális memória eddig megismert eszközei – lapozás, laphiba, demand paging, TLB és lapcsere – lehetővé teszik, hogy egy folyamat címtérbeli lapjait ne csak anonim fizikai memóriához vagy swaphez kössük, hanem fájlok és eszközök adataihoz is. Ezt a mechanizmust memóriatérképezésnek nevezzük. A POSIX rendszereken a `mmap()` rendszerhívás biztosítja a felületet, amely egy fájl vagy eszköz egy tartományát a folyamat virtuális címtérbe képezi le. A leképezés után a folyamat a memóriacímeken keresztül olvashat és írhat, a kernel pedig a page cache és a lapozás segítségével gondoskodik az adatok tényleges mozgatásáról.

Ez a fejezet a fájlok és eszközök memóriába képezését, a `mmap()` használatát, a page cache és a dirty page kapcsolatát, az osztott memória létrehozását, valamint a `msync()` szinkronizációt

tárgyalja. A hangsúly a gyakorlati megértésen van: egy ellenőrizhető C példán keresztül bemutatjuk a helyes használatot, majd egy reprodukálható labortervben mérésekkel vetjük össze a memóriatérképezéses és a hagyományos `read()`/`write()` alapú fájlfeldolgozást.

12.4.1 A memóriatérképezés fogalma és motivációja

Amikor egy folyamat egy fájlt hagyományos `read()` hívással olvas be, az adatok az alkalmazás által megadott felhasználói pufferbe kerülnek. A kernel általában a következő lépéseket hajtja végre:

1. A fájl megfelelő blokkjait beolvassa a page cache-be, ha azok még nincsenek ott.
2. A page cache-beli lap tartalmát átmásolja a felhasználói pufferbe.

A memóriatérképezés ezt a második másolást küszöböli ki. A `mmap()` egy virtuális címtérbeli tartományt közvetlenül a page cache lapjaira mutat. A folyamat így a fájlt úgy éri el, mintha az a saját memóriája lenne. Az első hozzáférés egy adott laphoz még laphibát okoz, ekkor a kernel beolvassa a fájl megfelelő részét a page cache-be, és beállítja a lapot a virtuális címhez. A későbbi hozzáférések már TLB-találat esetén közvetlenül a laphoz jutnak, kernelbe lépés nélkül.

A memóriatérképezés különösen előnyös, ha:

- nagy fájlokat kell véletlenszerűen elérni, mert a `lseek()` + `read()` sorozat helyett a program egyszerű mutatóaritmetikát használhat;
- több folyamatnak kell megosztania ugyanazt az adatot, mert a `MAP_SHARED` leképezések ugyanazokat a page cache lapokat használják;
- egy eszköz (például framebuffer) memóriatartományát közvetlenül a felhasználói térből kell kezelni.

A memóriatérképezés azonban nem mindig gyorsabb: a laphibák költsége, a TLB nyomás és a dirty lapok kezelése ronthatja a teljesítményt. Ezért a választást mindig méréssel kell alátámasztani.

12.4.2 Az `mmap()` rendszerhívás

A POSIX definíció szerint a függvény szignatúrája:

```
void *mmap(void *addr, size_t length, int prot, int flags, int fd, off_t offset);
```

A hívás a `fd` fájlleíróval azonosított fájl `offset` bájttól kezdődő, `length` bájt hosszú tartományát képezi le a folyamat virtuális címtérbe. A visszatérési érték a leképezés kezdőcíme, vagy `MAP_FAILED` – amely `(void *) -1` – hiba esetén. Ilyenkor az `errno` tartalmazza a hiba okát. Fontos, hogy a visszatérési értéket mindig a `MAP_FAILED` konstanssal hasonlítsuk, ne `NULL`-lal, mert a leképezés címe lehet `0` is, bár ritkán.

A paraméterek jelentése:

- **`addr`**: javasolt virtuális cím. A legtöbb esetben `NULL`, így a kernel szabadon választ egy igazítást biztosító címet. Ha `MAP_FIXED` nincs beállítva, a kernel ezt csak tippként kezeli.
- **`length`**: a leképezés hossza bájtban. Nem lehet `0`. A kernel a lapméret (általában 4 KiB) többszörösére kerekíti fel. A leképezés tényleges hossza így legalább egy lap.
- **`prot`**: memóriavédelmi bitek, a következők bitenkénti VAGY kombinációja:
  - `PROT_NONE` – a lap nem érhető el;
  - `PROT_READ` – olvasás;
  - `PROT_WRITE` – írás;

  - `PROT_EXEC` – végrehajtás. A kért védelemnek összhangban kell lennie a fájl megnyitási módjával. Például egy `O_RDONLY` módban megnyitott fájlt nem lehet `PROT_WRITE` védelemmel leképezni, mert a hívás `EACCES` hibával tér vissza.

- **`flags`**: viselkedési jelzők. A két legfontosabb:
  - `MAP_SHARED` – a leképezés megosztott; az írások a fájlba kerülnek, és más folyamatok is látják.
  - `MAP_PRIVATE` – a leképezés privát; az írások copy-on-write (COW) módon a folyamat saját másolatát módosítják, a fájl nem változik, és a módosítások nem láthatók más folyamatok számára. Ezen kívül gyakori még:
  - `MAP_ANONYMOUS` – nincs fájl a leképezés mögött; a lapok kezdetben nullákkal vannak feltöltve. Ekkor `fd` értéke `-1`, az `offset` pedig `0`.
  - `MAP_FIXED` – a megadott címet kötelezően használja, és ha ott már van leképezés, azt eldobja. Ez veszélyes, mert nem szándékos felülírást okozhat.
  - `MAP_POPULATE` – a kernel előre beolvassa a lapokat, csökkentve a későbbi laphibákat, de a `mmap()` hívás idejét növeli.
  - `MAP_LOCKED` – a lapok nem swapolhatók ki, ami valós idejű rendszerekben lehet fontos.

- **`fd`**: a fájl leírója, amelyet `open()`-nel kaptunk. `MAP_ANONYMOUS` esetén `-1`.
- **`offset`**: a fájlban való eltolás. **Lapméret többszörösének kell lennie**, különben a hívás `EINVAL` hibával tér vissza. A `length` nem kötelezően lapigazított, de a kernel kerekíti.

A leggyakoribb hibakódok:

Hibakód	Jelentés
`EACCES`	A fájl megnyitási módja nem teszi lehetővé a kért védelmet, vagy a fájl nem mmapelhető.
`EBADF`	Érvénytelen `fd`.
`EINVAL`	Érvénytelen paraméter, például `length == 0`, nem lapigazított `offset`, vagy `MAP_SHARED` és `MAP_PRIVATE` egyszerre.
`ENFILE`	Túl sok leképezés van a rendszerben.
`ENODEV`	A fájl nem támogatja a memóriatérképezést, például egy pipe.
`ENOMEM`	Nincs elég virtuális címtér, vagy a `MAP_LOCKED` lapok nem férnek a memóriába.

A leképezés megszüntetésére a `munmap()` szolgál:

```
int munmap(void *addr, size_t length);
```

A `length` nem feltétlenül lapigazított, de a kernel a megadott tartományt felszabadítja. A leképezés után a mutató használata érvénytelen; a folyamat `SIGSEGV` jelzést kap, ha a felszabadított címre hivatkozik.

12.4.3 A page cache és a dirty page szerepe

A fájlalapú memóriatérképezés szorosan összefonódik a page cache fogalmával. A page cache a kernel azon lapgyűjteménye, amely a blokkeszközökön tárolt adatokat memóriában tartja. Amikor egy folyamat `read()`-et hajt végre, a kernel először a page cache-ben keresi a kért blokkot. Ha nem találja, beolvassa a lemezről, majd a page cache-ből másol a felhasználói pufferbe.

A `mmap()` ezt a lapgyűjteményt teszi közvetlenül elérhetővé. A leképezés létrehozásakor a kernel még nem feltétlenül olvassa be a fájl tartalmát. Csak akkor történik laphiba, amikor a folyamat először hozzáfér egy adott virtuális laphoz. A laphibakezelő ekkor:

1. megkeresi a fájl megfelelő lapját a page cache-ben;
2. ha nincs jelen, beolvassa a lemezről;
3. beállítja a folyamat lapstátuszát, hogy a virtuális lap a page cache fizikai lapjára mutasson.

Ezt a folyamatot demand pagingnek nevezzük. A leképezett lapok tehát nem külön fizikai másolatok, hanem a page cache lapjai. Ha a folyamat ír egy leképezett lapra, a lap dirty (piszkos) lesz. A kernel ezt a lapot megjelöli, és később kiírja a háttértárolóra. A dirty lapok kiírását a kernel writeback mechanizmusa végzi, amely bizonyos feltételek teljesülésekor – például a dirty lapok aránya meghalad egy küszöböt – elindítja a háttérírást.

A dirty page kezelés fontos következményekkel jár:

- A `mmap()`-pal írt adatok nem azonnal kerülnek a lemezre, hanem előbb a page cache-ben maradnak. A folyamat szempontjából az írás befejezett, de a tartósság nem garantált.
- A `msync()` hívás explicit kéri a dirty lapok kiírását. A `fsync()` a fájl leírójára vonatkozik, és a fájlhoz tartozó összes dirty lapot kiírja, függetlenül attól, hogy azokat `write()` vagy `mmap()` hozta létre.
- Ha a folyamat `MAP_PRIVATE` leképezést használ, az írás nem teszi dirty-vé a page cache lapot, hanem a kernel egy privát másolatot hoz létre (COW), és azt módosítja. Ezért a fájl tartalma nem változik, és más folyamatok nem látják a módosítást.

A dirty lapok és a lapcsere kapcsolata: ha a memórianyomás nagy, a kernel a tiszta lapokat egyszerűen eldobhatja, mert újra beolvashatók a fájlból. A dirty lapokat azonban előbb ki kell írni, mielőtt felszabadíthatók. Ezért a sok memóriatérképezéses írás megnövelheti a writeback terhelést és a lemez I/O-t.

12.4.4 Osztott memória és anonim leképezés

A memóriatérképezés nem csak fájlokhoz köthető. A `MAP_ANONYMOUS` jelzővel fájl nélküli, anonim lapokat hozhatunk létre. Ezek a lapok kezdetben nullákkal vannak feltöltve, és a swap területre írhatók ki, ha a memóriából ki kell szorítani őket. Az anonim leképezés a `malloc()` egyik alacsony szintű építőköve, de közvetlenül is használható osztott memória létrehozására.

Két gyakori módja van az osztott memória kialakításának:

1. Anonim `MAP_SHARED` leképezés

Ha egy folyamat a következő hívást hajtja végre:

```
void *p = mmap(NULL, length, PROT_READ | PROT_WRITE,
               MAP_SHARED | MAP_ANONYMOUS, -1, 0);
```

akkor egy megosztott, fájl nélküli területet kap. A `fork()` után a gyermek és a szülő ugyanazokat a fizikai lapokat látja, mert a leképezés `MAP_SHARED`. Ez azt jelenti, hogy az egyik folyamat írása a másik számára is láthatóvá válik. Ez egyszerű, de csak közös ősökkel rendelkező folyamatok között működik, mert a leképezésnek nincs neve, így független folyamatok nem tudják megtalálni.

2. POSIX megnevezett osztott memória

A POSIX `shm_open()` függvény egy megnevezett osztott memória objektumot hoz létre, amely a Linux rendszereken általában a `/dev/shm` könyvtárban jelenik meg. A használat tipikus lépései:

```
int fd = shm_open("/my_shm", O_CREAT | O_RDWR, 0600);
ftruncate(fd, length);
void *p = mmap(NULL, length, PROT_READ | PROT_WRITE,
               MAP_SHARED, fd, 0);
close(fd);
```

A `shm_open()` egy fájlleírót ad vissza, amelyet a `ftruncate()` segítségével a kívánt méretűre állítunk be, majd a `mmap()` leképezi. A fájlleírót a leképezés után le lehet zárni, mert a leképezés továbbra is érvényes marad. Más folyamatok ugyanezzel a névvel nyithatják meg az objektumot, és leképezhetik a saját címtérükbe. A megosztott terület felszabadításához a `shm_unlink()` hívás szükséges, amely eltávolítja a nevet; a tényleges memória akkor szabadul fel, amikor az utolsó leképezés is megszűnik.

A System V interfész (`shmget`, `shmat`, `shmdt`, `shmctl`) hasonló célt szolgál, de a POSIX shm modernebb és jobban illeszkedik a fájlleírók világába. Új kódban a POSIX shm ajánlott.

Az osztott memória előnye a gyors adatcsere a folyamatok között, mert nincs másolás és nincs rendszerhívás az adatok eléréséhez. Hátránya, hogy a szinkronizációt a folyamatoknak maguknak kell megoldaniuk, például mutex, szemafor vagy atomi műveletek segítségével. A `MAP_SHARED` leképezéseknél a memóriamodell és a láthatóság kérdéseit a cache koherencia protokoll kezeli, de a programozónak gondoskodnia kell a helyes sorrendezésről.

12.4.5 A `msync()` szinkronizáció

A `mmap()`-pal írt adatok a page cache-ben maradnak, és a kernel aszinkron módon írja ki őket. Ha egy folyamat biztos akar lenni abban, hogy a leképezés egy adott tartománya a háttértárolóra került, a `msync()` hívást használhatja:

```
int msync(void *addr, size_t length, int flags);
```

A paraméterek:

- `addr` és `length` – a leképezés szinkronizálandó tartománya. Az `addr`-nek lapigazítottnak kell lennie, a `length` pedig nem lehet `0`.
- `flags` – a következők kombinációja:
  - `MS_ASYNC` – a kiírást aszinkron módon ütemezi; a hívás azonnal visszatérhet.
  - `MS_SYNC` – a kiírást szinkron módon végrehajtja; a hívás csak akkor tér vissza, ha a dirty lapok kiírása befejeződött.
  - `MS_INVALIDATE` – a leképezéshez tartozó többi, más folyamatok által használt cache-másolatot érvényteleníti.

A visszatérési érték `0` siker esetén, `-1` hiba esetén. Gyakori hibák: `EINVAL` (nem lapigazított cím vagy érvénytelen flags), `EFAULT` (a tartomány egy része nem leképezett), `ENOMEM` (memóriahiány).

Fontos megjegyezni, hogy a `msync()` nem garantálja a lemezre írást abban az értelemben, mint a `fsync()` a fájl teljes metaadatához. A `msync()` a leképezéshez tartozó dirty lapok kiírását kéri. A `MS_SYNC` esetén a hívás befejeződése után a lapok már a háttértárolóra kerültek, de a fájl metaadatainak (például módosítási idő) frissítése külön `fsync()`-et igényelhet. A `MAP_PRIVATE` leképezésnél a `msync()` általában nem ír ki semmit, mert a privát lapok nem tartoznak a fájlhoz; a hívás csupán a COW másolatokat érinti, ha egyáltalán.

A gyakorlatban a `msync()`-et akkor érdemes hívni, ha egy folyamat biztosítani akarja, hogy a leképezett fájlba írt adatok egy adott ponton tartósak legyenek, például egy tranzakció végén.

Sok alkalmazás azonban a `fsync()`-et használja a fájl leíróján, mert az a teljes fájlra vonatkozik, és a kernel a page cache-ből dolgozik.

12.4.6 Fájlok és eszközök memóriába képezése

A `mmap()` nem csak hagyományos fájlokra, hanem eszközökre is alkalmazható. A Linux kernelben minden eszköz, amely támogatja a memóriatérképezést, rendelkezik egy `mmap` metódussal a `file_operations` struktúrában. Például a framebuffer eszköz (`/dev/fb0`) lehetővé teszi a képernyőpuffer közvetlen elérését, a `mmap()` segítségével a felhasználói program a képpontokat memóriaműveletekkel írhatja, kernelhívások nélkül.

Egy másik példa a `/dev/mem`, amely a fizikai memóriát képezi le. Ez azonban komoly biztonsági kockázatot jelent, mert egy folyamat így hozzáférhet a kernel és más folyamatok memóriájához. A modern Linux rendszereken a `/dev/mem` hozzáférést általában korlátozzák, és a `CONFIG_STRICT_DEVMEM` opció letiltja a RAM nagy részének olvasását. Hasonlóan, a `/dev/kmem` a kernel virtuális memóriáját tette volna elérhetővé, de biztonsági okokból eltávolították vagy letiltották.

Az eszközök memóriatérképezésének előnye a minimális overhead, mert az adatcsere közvetlenül a hardver pufferével történik. A kernelnek azonban gondoskodnia kell a cache koherenciáról: egyes architektúrákon a DMA által írt területeket explicit módon kell kezelni, hogy a CPU lássa a friss adatokat. Ezt a kernel a lapstátusz és a memóriabarrierek segítségével oldja meg.

A fájlok esetében a `mmap()` és a `read()`/`write()` közötti választás több szemponttól függ:

- **Másolások száma**: a `mmap()` kikerüli a page cache-ből a felhasználói pufferbe történő másolást, így nagy adatmennyiségnél csökkentheti a CPU időt.
- **Rendszerhívások**: a `read()`/`write()` minden híváskor kernelbe lép, míg a `mmap()` után a hozzáférések memóriaműveletek, amelyek csak laphibánál igényelnek kernel beavatkozást.
- **Laphibák**: a `mmap()` első érintése laphibát okoz, ami költséges. Ha a fájl nagy része csak egyszer kerül feldolgozásra, a `read()` hatékonyabb lehet.
- **Dirty lapok**: a `mmap()` írása dirty lapokat hoz létre, amelyek kiírása a writeback mechanizmuson keresztül történik, nem feltétlenül a program által várt sorrendben.
- **Fájl méretváltozás**: ha a fájl mérete megváltozik a leképezés alatt, a viselkedés bonyolult lehet; a `mmap()` nem alkalmas dinamikusan növekvő fájlokhoz, hacsak nem használunk `mremap()`-et.

A következő szakaszban egy konkrét példán mutatjuk be a helyes használatot.

12.4.7 Gyakorlati példa: fájl leképezése és módosítása

A következő C program egy fájlt nyit meg, leképezi a teljes tartalmát, kiírja az első bájtot, módosítja azt, majd `msync()`-et hív, és ellenőrzi a fájl tartalmát egy újabb megnyitással. A kód szemléltetés; a hibakezelés célja a tipikus csapdák bemutatása.

```
/* mmap_demo.c - fájl megnyitása, leképezése, módosítása */
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <sys/mman.h>
#include <sys/stat.h>
#include <unistd.h>
#include <string.h>
#include <errno.h>
```

```c
int main(int argc, char *argv[]) {
    if (argc != 2) {
        fprintf(stderr, "Hasznalat: %s <fajl>\n", argv[0]);
        return EXIT_FAILURE;
    }

    const char *path = argv[1];
    int fd = open(path, O_RDWR);
    if (fd < 0) {
        perror("open");
        return EXIT_FAILURE;
    }

    struct stat st;
    if (fstat(fd, &st) < 0) {
        perror("fstat");
        close(fd);
        return EXIT_FAILURE;
    }

    if (st.st_size == 0) {
        fprintf(stderr, "A fajl ures, nem lehet lekepezni.\n");
        close(fd);
        return EXIT_FAILURE;
    }

    /* A fajl merete nem feltetlenul lapigazitott,
       de a kernel a length-et kerekiti. */
    size_t length = (size_t)st.st_size;

    void *map = mmap(NULL, length, PROT_READ | PROT_WRITE,
                     MAP_SHARED, fd, 0);
    if (map == MAP_FAILED) {
        perror("mmap");
        close(fd);
        return EXIT_FAILURE;
    }

    /* A fajl leírót lezarhatjuk, a lekepezes megmarad. */
    close(fd);

    unsigned char *byte = (unsigned char *)map;
    printf("Elso bajt a lekepezes elott: 0x%02x\n", byte[0]);

    byte[0] = 0x41; /* 'A' */

    /* Szinkronizaljuk a dirty lapot a fajlba. */
    if (msync(map, length, MS_SYNC) < 0) {
        perror("msync");
        munmap(map, length);
        return EXIT_FAILURE;
    }

    /* Ellenorzes: ujra megnyitjuk a fajlt es beolvassuk. */
    fd = open(path, O_RDONLY);
    if (fd < 0) {
        perror("open ellenorzes");
        munmap(map, length);
        return EXIT_FAILURE;
    }
    unsigned char c;
    if (read(fd, &c, 1) != 1) {
```

```
        perror("read ellenorzes");
        close(fd);
        munmap(map, length);
        return EXIT_FAILURE;
    }
    printf("Elso bajt a lekepezes utan: 0x%02x\n", c);

    close(fd);
    munmap(map, length);
    return EXIT_SUCCESS;
}
```

A program fontos pontjai:

- A `mmap()` visszatérését a `MAP_FAILED` konstanssal hasonlítjuk, nem `NULL`-lal.
- A `length` a fájl mérete. Ha a fájl mérete nem lapigazított, a kernel a leképezést a következő laphatárig kiterjeszti, de a fájl végén túli részhez való hozzáférés `SIGBUS` jelzést okozhat. Ezért a program csak a fájl tényleges méretén belül fér hozzá.
- A `msync()` hívás biztosítja, hogy az első bájt módosítása a fájlba kerüljön. Bár `MAP_SHARED` esetén a kernel idővel magától kiírja a dirty lapot, a szinkron hívás determinisztikussá teszi a példát.
- A `close(fd)` a `mmap()` után biztonságos; a leképezés érvényes marad.

A program fordítása és futtatása:

```
cc -O2 -o mmap_demo mmap_demo.c
echo "Hello" > testfile
./mmap_demo testfile
```

A várt kimenet:

```
Elso bajt a lekepezes elott: 0x48
Elso bajt a lekepezes utan: 0x41
```

Ahol `0x48` a `'H'` karakter kódja.

12.4.8 Tipikus hibák és csapdák

A memóriatérképezés használata során a következő hibák fordulnak elő leggyakrabban:

1. **A `MAP_FAILED` ellenőrzésének elmulasztása.** Ha a program a visszatérési értéket `NULL`-lal hasonlítja, a `mmap()` hiba esetén `(void *) -1` értéket ad vissza, amely nem `NULL`, így a program érvénytelen mutatót kap, és `SIGSEGV` jelzést okozhat.
2. **A leképezés hosszának helytelen megadása.** Ha a `length` nagyobb, mint a fájl tényleges mérete, a fájl végén túli lapok nem léteznek. Az ilyen laphoz való hozzáférés nem a szokásos `SIGSEGV` (érvénytelen címtér), hanem `SIGBUS` jelzést vált ki, mert a lap a fájlhoz tartozik, de a fájl nem elég nagy. Ez különösen akkor gyakori, ha a program a leképezés végét használja anélkül, hogy ellenőrizné a fájl méretét.
3. **A `munmap()` utáni hozzáférés.** A leképezés megszüntetése után a mutató érvénytelenné válik. A használata `SIGSEGV`-t okozhat. A `munmap()` a leképezés egy részét is felszabadíthatja, ezért mindig a teljes tartományt kell felszabadítani, vagy gondosan követni a részeket.

4. **A `MAP_PRIVATE` félreértése.** A `MAP_PRIVATE` leképezésnél az írások nem változtatják meg a fájlt, és nem láthatók más folyamatok számára. Ha egy program ezt figyelmen kívül hagyja, és azt feltételezi, hogy a fájl frissült, hibás eredményt kap. A `MAP_PRIVATE` elsősorban akkor hasznos, ha egy fájlt csak olvasni akarunk, és az írásokat nem kell megosztani.

5. **A `MAP_SHARED` és a `fork()` kölcsönhatása.** Ha egy `MAP_SHARED` leképezést öröklő gyermek és szülő is ír, akkor mindkettő ugyanazt a fizikai lapot módosítja, ami versenyhelyzetet okozhat. A szinkronizációhoz mutexet vagy más mechanizmust kell használni, amelyet magában a megosztott memóriában helyezünk el.

6. **Az `offset` nem lapigazított.** A `mmap()` `offset` paraméterének lapméret többszörösének kell lennie. Ha nem, a hívás `EINVAL` hibával tér vissza. A `length` nem kötelezően lapigazított, de a tényleges leképezés a laphatárig terjed.

7. **A fájl megnyitási módja nem egyezik a `prot` bitekkel.** Például `O_RDONLY` fájl esetén `PROT_WRITE` kérése `EACCES` hibát ad. A fájlt a kívánt hozzáférésnek megfelelően kell megnyitni.

8. **A `MAP_FIXED` veszélyes használata.** A `MAP_FIXED` jelzővel a kernel a megadott címet kötelezően használja, és ha ott már van leképezés, azt eldobja. Ez nem szándékos felülírást okozhat, például a dinamikus linker területét. Általában kerülendő, kivéve ha a program pontosan tudja, mit csinál.

9. **A `msync()` elhagyása.** Bár a kernel automatikusan kiírja a dirty lapokat, a program nem feltételezheti, hogy egy adott pillanatban az adatok már a lemezen vannak. Ha a tartósság számít, explicit `msync()` vagy `fsync()` szükséges.

10. **A leképezett terület és a fájl méretének változása.** Ha a fájl mérete a leképezés alatt megváltozik (például egy másik folyamat `ftruncate()`-tel), a leképezés viselkedése nem hordozható. A Linux a leképezett tartományt a fájl aktuális méretéhez igazítja, de a fájl végén túli lapokhoz való hozzáférés `SIGBUS`-t okozhat.

12.4.9 Labor: mmap-alapú fájlfeldolgozás mérése

A következő labor célja, hogy a hallgató reprodukálható környezetben összehasonlítsa a memóriatérképezéses és a hagyományos `read()`/`write()` alapú fájlfeldolgozást. A labor nem igényel kernelmódosítást, elég egy felhasználói program.

12.4.10 Kiinduló állapot

A labor egy emulált Linux környezetben fut, például QEMU-ban, hogy a mérési körülmények ellenőrizhetők legyenek. A reprodukciós csomag tartalmazza:

- QEMU verzió (például 8.2.x, ellenőrzés szükséges);
- kernel verzió (például 6.6.x LTS, ellenőrzés szükséges);
- CPU modell: `qemu64` vagy `cortex-a72` (attól függően, milyen architektúrát választunk);
- memória: 512 MiB;
- lemezkép: egy initramfs vagy egy minimalista rootfs, amely tartalmazza a `cc`-t, `perf`-et és a szükséges eszközöket.

A hoszt valódi lemezét, bootloaderét és rendszerpartícióját nem módosítjuk; minden művelet a vendég rendszerben, eldobható lemezképen történik.

12.4.11 Feladat

1. Hozz létre egy 64 MiB méretű tesztfájlt a vendég rendszerben:

```
dd if=/dev/zero of=/tmp/testfile bs=1M count=64
```

2. Írj egy C programot `mmap_test.c` néven, amely a fájlt `mmap()`-pel leképezi, majd a teljes tartományt bejárja, és minden lap első bájtját `0xAA`-ra állítja. A program hívja az `msync()`-et a végén.

3. Írj egy másik programot `rw_test.c` néven, amely ugyanazt a fájlt `read()`/`write()` hívásokkal dolgozza fel: beolvassa 4 KiB-os blokkokban, módosítja a blokk első bájtját, majd visszaírja.

4. Fordítsd mindkét programot `-O2` optimalizálással:

```
cc -O2 -o mmap_test mmap_test.c
cc -O2 -o rw_test rw_test.c
```

5. Futtasd mindkét programot a `perf stat` eszközzel, rögzítve a futási időt, a minor és major laphibákat, valamint a végrehajtott utasítások számát:

```
perf stat -e minor-faults,major-faults,cycles,instructions ./mmap_test /tmp/testfile
perf stat -e minor-faults,major-faults,cycles,instructions ./rw_test /tmp/testfile
```

6. Ismételd meg mindkét mérést legalább ötször, és számold ki az átlagot és a szórást.

12.4.12 Várt eredmények

A mérés előtt a következő várt tendenciákat fogalmazzuk meg:

- A `mmap_test` program kevesebb utasítást fog végrehajtani, mint a `rw_test`, mert nincs rendszerhívás és nem történik másolás a page cache-ből a felhasználói pufferbe.
- A `mmap_test` több minor laphibát fog okozni, mint a `rw_test`, mert a leképezett lapok első érintése laphibát generál, míg a `read()` a page cache-ből másol, és a laphibák a kernel oldalán jelentkeznek.
- A major laphibák száma mindkét esetben hasonló, mert a fájl első beolvasása a lemezről mindkét módszernél szükséges.
- A futási idő a hardvertől és a mérési körülményektől függ; a `mmap_test` általában gyorsabb nagy fájloknál, de a különbség nem garantált.

A **mért** és a **várt** eredményeket a mérési naplóban külön kell feltüntetni. A mérési napló formátuma:

```
Merési napló - mmap vs read/write fájlfeldolgozás
Időpont: ...
Környezet: QEMU 8.2.x, Linux 6.6.x, qemu64, 512 MiB RAM
Fordítási beállítások: cc -O2

Program        Futási idő (s)   Minor fault   Major fault   Utasítások
mmap_test      ...              ...           ...           ...
rw_test        ...              ...           ...           ...

Átlagok és szórások: ...
Várt eltérés: a mmap_test kevesebb utasítást, de több minor faultot mutat.
Megjegyzés: ...
```

A labor visszaállítása: a `/tmp/testfile` törlése, a programok törlése, a QEMU vendég leállítása. A hoszt rendszeren semmilyen változás nem történik.

12.4.13 Önellenőrző kérdés

Miért okozhat `SIGBUS` jelzést egy `mmap()`-pel leképezett fájl végén túli memória elérése, és hogyan lehet ezt a hibát elkerülni?

Válasz vázlat: A `mmap()` által leképezett virtuális tartomány hossza a `length` paraméterből adódik, amelyet a kernel a lapméretre kerekít. Ha a fájl tényleges mérete kisebb, mint a leképezés hossza, a fájl végén túli részhez tartozó lapok nem léteznek a page cache-ben. Amikor a folyamat egy ilyen laphoz fér hozzá, a kernel laphibát észlel, de nem tudja a lapot a fájlból betölteni, mert az adott offset a fájl vége után van. Ekkor a kernel `SIGBUS` jelzést küld a folyamatnak. A `SIGSEGV` ezzel szemben akkor keletkezik, ha a cím egyáltalán nem tartozik a leképezett tartományhoz. A hiba elkerülhető, ha a program gondosan ellenőrzi a fájl méretét, és csak a tényleges méretig fér hozzá a leképezéshez, vagy ha a fájlt előzőleg a kívánt méretre növeli `ftruncate()`-tel.

12.4.14 Összefoglalás és továbbvezetés

A memóriatérképezés a virtuális memória egyik leghatékonyabb eszköze, amely a fájlokat és eszközöket közvetlenül a folyamat címtérbe köti. A `mmap()` rendszerhívás a page cache lapjaira mutat, így elkerüli a felesleges másolásokat, és lehetővé teszi az osztott memória kialakítását. A dirty lapok kezelése és a `msync()` szinkronizáció a tartósság szempontjából kritikus. A tipikus hibák – mint a `MAP_FAILED` ellenőrzésének hiánya, a fájl végén túli hozzáférés, vagy a `MAP_PRIVATE` és `MAP_SHARED` összetévesztése – gyakran vezetnek nehezen felderíthető hibákhoz.

A következő egység (`os046`) a heapallokátorok világába vezet, ahol a memóriatérképezés és az anonim lapok biztosítják a `malloc()` alapját. Az itt megszerzett ismeretek – a lapok, a COW és a memóriavédelem – közvetlenül alkalmazhatók lesznek a kernelmemória-kezelés megértésében.

13 Memóriaallokáció és kernelmemória

13.1 Heapallokátorok: free list, buddy és slab

13.1.1 1. Dinamikus memóriaallokáció és a heap

A dinamikus memóriaallokáció olyan futás közbeni memóriafoglalás, amely nem a fordítási időben rögzített méretű statikus vagy verem változókat használja, hanem a folyamat vagy a kernel egy rugalmasan kezelt memóriaterületből, a heapből vesz el blokkokat. Az operációs rendszer és a felhasználói könyvtárak számára ez azért alapvető, mert a legtöbb valós program nem tudja előre, hogy például hány folyamat, hány hálózati kapcsolat vagy hány fájl-bejegyzés fog létrejönni. A heapallokátor felelős azért, hogy a memóriaterületet blokkokra ossza, kiosztja azokat, majd a felszabadított blokkokat újra felhasználja.

Ebben az alfejezetben három klasszikus allokátorstratégiát vizsgálunk meg: a *free list* (szabadlista), a *buddy* és a *slab* megközelítést. Mindegyik más kompromisszumot jelent a sebesség, a memória-kihasználtság, a töredezettség és a bonyolultság között. A fejezet végén egy reprodukálható labor keretében egy egyszerű free list alapú allokátort építünk fel, és mérésekkel hasonlítjuk össze a várható és a mért viselkedést.

13.1.2 1.1. Mit várunk egy heapallokátortól?

Egy jó allokátornak egyszerre több követelménynek kell megfelelnie:

- **Alacsony késleltetés:** a foglalás és a felszabadítás minél kevesebb utasításból álljon, mert a gyakori hívások szűk keresztmetszetet jelentenek.
- **Jó memória-kihasználtság:** a valós hasznos adat és a kiosztott terület aránya minél nagyobb legyen.
- **Alacsony töredezettség:** ne keletkezzen sok apró, használhatatlan lyuk.
- **Determinizmus:** beágyazott és valós idejű rendszerekben a legrosszabb esetű futásidő is fontos lehet.
- **Biztonság:** az allokátor metaadatai ne sérülhessenek, és a hibás használat ne vezessen csendes memóriakorrupcióhoz.

A heap mérete jellemzően nőhet vagy zsugorodhat. A felhasználói térben a `malloc`/`free` mögött gyakran a kernel `brk` vagy `mmap` rendszerhívása ad új területet. Kernelmódban az allokátor közvetlenül a fizikai memóriát kezelő lapkezelőtől kér lapokat. A következő alfejezetben bemutatott free list, buddy és slab stratégiák ezen a lapokból építkeznek.

13.1.3 2. Free list allokátorok

A free list a legegyszerűbb és legismertebb heapallokátor-minta. Az alapötlet az, hogy a szabad memóriablokkokat láncolt listában tartjuk nyilván, és egy foglalási kérésnél ebből a listából választunk blokkot. A felszabadított blokkokat visszafűzzük a listába, lehetőleg összevonva a szomszédos szabad blokkokat.

13.1.4 2.1. Implicit és explicit szabadlista

Implicit free list esetén a lista nem külön tárolt mutatókat használ, hanem a blokkok fejlécében lévő méretből és állapotból léptetjük a bejárást. Egy blokk fejléce például így nézhet ki:

```
struct block_header {
    size_t size;        /* a hasznos adat mérete, fejléc nélkül */
    uint32_t flags;     /* bit0: 1 = foglalt, 0 = szabad */
    /* szabad blokk esetén itt következhet a következő/előző mutató */
};
```

A lista bejárása az első blokktól kezdve a `size + sizeof(header)` lépésekkel történik. Ez egyszerű, de a keresés lassú, mert minden blokkon végig kell menni.

Explicit free list esetén a szabad blokkok külön láncolt listát alkotnak. A szabad blokk fejlécében vagy a hasznos terület elején tároljuk a következő és az előző szabad blokk mutatóját. Ekkor csak a szabad blokkokat kell bejárni, ami jelentősen gyorsabb, de a listakarbantartás több munkát igényel.

A free list allokátorok leggyakoribb illesztési stratégiái:

- **First-fit:** az első elég nagy blokkot adjuk vissza. Gyors, de a lista elején apró szabad blokkok halmozódhatnak fel.
- **Next-fit:** a legutóbbi keresés helyétől indulva keresi az első megfelelő blokkot. Csökkenti a lista eleji torlódást, de a lista végén növelheti a töredezettséget.
- **Best-fit:** a legkisebb elegendő blokkot választja. Jobb memória-kihasználtságot ad, de lassabb, mert a teljes listát be kell járni.
- **Worst-fit:** a legnagyobb szabad blokkot választja. Gyakran rossz választás, mert a nagy blokkokat aprózza fel.

A gyakorlatban a first-fit és a best-fit kombinációja terjedt el, mert jó kompromisszumot adnak.

13.1.5 2.2. Összevonás és hasítás

A free list allokátor egyik kritikus művelete a **coalescing**, azaz a szomszédos szabad blokkok összevonása. Ha egy blokk felszabadul, és a közvetlen előtte vagy utána lévő blokk is szabad, akkor a kettőt egyetlen nagyobb szabad blokkká kell összevonni. Enélkül a heap hamar apró szabad darabokra esik szét, és egy nagyobb kérést már nem lehet kielégíteni, hiába van elegendő összes szabad memória.

A hasítás (**splitting**) az ellenkező irány: ha egy nagyobb szabad blokkból egy kisebb kérést elégítünk ki, a maradékot szabad blokkként vissza kell tenni a listába. A hasítás nélkül a nagy blokkok pazarlóan fogynának el.

Egy lehetséges free list allokátor pszeudokódja:

```
void *malloc(size_t size) {
    /* 1. Igazítsuk a méretet a header méretéhez és az alignmenthez */
    size = align_up(size + sizeof(header), ALIGN);

    /* 2. Keressünk szabad blokkot (first-fit) */
    block_t *b = find_free(size);
    if (!b) {
        /* 3. Nincs szabad blokk: kérjünk új lapot a rendszertől */
        b = extend_heap(size);
        if (!b)
            return NULL;
```

```
    }

    /* 4. Ha a blokk túl nagy, hasítsuk ketté */
    split_if_possible(b, size);

    /* 5. Jelöljük foglaltnak, és adjuk vissza a hasznos területet */
    b->flags |= BLOCK_USED;
    return (void *)(b + 1);
}

void free(void *ptr) {
    /* 1. Keressük meg a fejlécet a mutató előtt */
    block_t *b = (block_t *)ptr - 1;

    /* 2. Jelöljük szabadnak */
    b->flags &= ~BLOCK_USED;

    /* 3. Vonjuk össze a szomszédos szabad blokkokkal */
    coalesce(b);
}
```

A fenti pszeudokód csak a lényeget mutatja; a valódi implementációban a listaműveletek, a szálbiztosság és a hibakezelés külön figyelmet igényel.

13.1.6 2.3. Előnyök és hátrányok

A free list allokátor előnye az egyszerűség és a kis állandó költség. Könnyű megérteni, könnyű implementálni, és jól működik akkor, ha a foglalási méretek változatosak, de nem túl nagyok. A hátránya a lineáris keresési idő, a töredezettségre való hajlam és az, hogy a lista karbantartása bonyolulttá válik, ha sok méretosztályt vagy jó illesztést akarunk.

13.1.7 3. Buddy allokátor

A buddy allokátor a fizikai memóriakezelésben, különösen a kernel lapallokációjában elterjedt stratégia. Az alapötlet az, hogy a memóriát kettő hatvány méretű blokkokra osztjuk, és minden blokkhoz tartozik egy pontosan ugyanakkora „barát” blokk, amellyel összevonható, ha mindkettő szabad. A Linux kernel a fizikai lapkeretek kiosztására éppen ilyen buddy rendszert használ.

13.1.8 3.1. Működési elv

A buddy allokátor egy fix méretű memóriaterületet (például a fizikai memória lapkeretjeit) kezel. A területet kettő hatvány méretű blokkokra lehet bontani. A legkisebb egység általában egy lap (például 4 KiB), a legnagyobb pedig az egész kezelt terület. Minden méretosztályhoz külön szabadlista tartozik.

Foglaláskor a kért méretnél nem kisebb, legkisebb kettő hatványú blokkot keressük. Ha nincs ilyen, egy nagyobb blokkot kettévágunk, és a felesleges felet a megfelelő méretosztály szabadlistájára tesszük. Ezt a vágást addig ismételjük, amíg meg nem kapjuk a kívánt méretet. A vágás során keletkező blokkok mindig párokba rendezhetők: egy blokk és annak „buddy”-ja együtt alkotja a szülő blokkot.

Felszabadításkor a blokkot visszatesszük a méretosztályának listájába, majd megvizsgáljuk, hogy a buddy-ja is szabad-e. Ha igen, a kettőt összevonjuk, és a folyamatot rekurzívan folytatjuk a következő méretosztályban.

A buddy allokátor pszeudokódja:

```
void *buddy_alloc(size_t size) {
    /* 1. Határozzuk meg a legkisebb 2^k méretű osztályt */
    int order = ceil_log2(size);
    if (order < MIN_ORDER)
        order = MIN_ORDER;

    /* 2. Keressünk szabad blokkot az adott osztályban */
    if (free_lists[order] == NULL) {
        /* 3. Ha nincs, vágjunk nagyobb blokkot */
        int o = order;
        while (o < MAX_ORDER && free_lists[o] == NULL)
            o++;
        if (o == MAX_ORDER)
            return NULL;  /* nincs memória */

        /* Vágjuk le a nagyobb blokkot, amíg el nem érjük a cél osztályt */
        while (o > order) {
            split_block(o);
            o--;
        }
    }

    /* 4. Vegyük le a lista elejéről, és jelöljük foglaltnak */
    block_t *b = pop_free_list(order);
    b->order = order;
    b->free = false;
    return block_to_ptr(b);
}

void buddy_free(void *ptr) {
    block_t *b = ptr_to_block(ptr);
    int order = b->order;

    b->free = true;
    push_free_list(order, b);

    /* Összevonás, amíg a buddy szabad */
    while (order < MAX_ORDER) {
        block_t *buddy = get_buddy(b, order);
        if (!buddy || !buddy->free)
            break;
        remove_from_free_list(order, buddy);
        b = merge(b, buddy);
        order++;
    }
    b->order = order;
    push_free_list(order, b);
}
```

A `get_buddy` függvény a blokk címéből és a méretéből számítja ki a buddy címét. A buddy cím kiszámítása gyakran egyetlen XOR művelet a blokk kezdőcíme és a blokk mérete között, mert a blokkok természetes módon párokba rendeződnek.

13.1.9 3.2. Előnyök és hátrányok

A buddy allokátor nagy előnye, hogy a kiosztás és a felszabadítás gyors, és a blokkok mérete kettő hatvány szerint igazodik, ami jól illeszkedik a lapalapú címzéshez. Az összevonás automatikus, így a külső töredezettség viszonylag alacsony. A hátránya a **belső töredezettség**: ha egy kérés 5 KiB, akkor a rendszer 8 KiB-os blokkot ad, tehát 3 KiB kárba vész. Ez a veszteség átlagosan a

kiosztott blokk méretének körülbelül a fele lehet, ha a kérések véletlenszerűen oszlanak el. Emiatt a buddy allokátort főként nagyobb, rögzített méretű erőforrások (lapkeretek, DMA-pufferek) kiosztására használjuk, nem apró objektumokra.

A belső fragmentáció arányát a következő képlettel becsülhetjük:

$$F_{\text{belső}} = \frac{\text{lefoglalt blokk mérete} - \text{kért méret}}{\text{lefoglalt blokk mérete}}$$

Például 5 KiB kérés esetén 8 KiB blokkal:

$$F_{\text{belső}} = \frac{8\ \text{KiB} - 5\ \text{KiB}}{8\ \text{KiB}} = 0{,}375$$

Ez 37,5%-os belső fragmentációt jelent.

13.1.10 4. Slab allokátor

A slab allokátor a kernel apró objektumainak hatékony kezelésére szolgál. A Linux kernel hagyományosan a slab, slub vagy slob allokátort használja a `kmalloc` és a speciális cache-ek mögött. A slab lényege, hogy azonos típusú objektumokat előre méretezett cache-ekből oszt ki, és a felszabadított objektumokat azonnal újra felhasználhatóvá teszi anélkül, hogy minden alkalommal a buddy allokátortól kellene lapot kérni.

13.1.11 4.1. Cache-ek, slabok és objektumok

A slab allokátor három szintből áll:

- **Cache:** egy adott objektumtípushoz (például `struct inode`, `struct task_struct`) vagy egy adott méretosztályhoz tartozó készlet.
- **Slab:** egy vagy több összefüggő fizikai lapból álló terület, amely több objektumot tartalmaz.
- **Objektum:** a ténylegesen kiosztott adategység, amelynek mérete a cache-hez igazodik.

A foglalás lépései:

1. A kernel lekéri a megfelelő cache-t (például a `kmalloc-32` cache-ből egy 32 bájtos blokkot).
2. A cache aktuális slabjában megnézi, van-e szabad objektum.
3. Ha van, azt visszaadja, és a szabad listából törli.
4. Ha nincs, új slabot kér a buddy allokátortól, és abból oszt ki.

A felszabadítás ennek a fordítottja: az objektumot visszateszi a slab szabad listájára, és ha a slab összes objektuma szabad, a slabot visszaadja a buddy rendszernek.

A Linux kernelben a klasszikus slab implementáció mellett létezik a **SLUB**, amely egyszerűbb és skálázhatóbb, mert per-CPU cache-eket használ, és kerüli a bonyolult slab-objektum nyilvántartást. A fogalmi modell azonban nagyon hasonló.

13.1.12 4.2. Slab pszeudokód

```
struct slab_cache {
    size_t object_size;
    unsigned int align;
    struct slab *partial;   /* részben foglalt slabok */
    struct slab *full;      /* teljesen foglalt slabok */
    struct slab *empty;     /* teljesen üres slabok */
};

void *slab_alloc(struct slab_cache *cache) {
    struct slab *s = cache->partial;
    if (!s) {
        /* Nincs részben foglalt slab: kérjünk újat */
        s = slab_create(cache);
        if (!s)
            return NULL;
    }

    /* Keressünk szabad objektumot a slabban */
    void *obj = slab_pop_free_object(s);
    if (slab_is_full(s))
        list_move(&s->list, &cache->full);
    return obj;
}

void slab_free(struct slab_cache *cache, void *ptr) {
    struct slab *s = addr_to_slab(ptr);

    slab_push_free_object(s, ptr);
    if (slab_is_empty(s)) {
        /* Az egész slab üres: vissza a buddy rendszernek */
        list_del(&s->list);
        slab_destroy(cache, s);
    } else if (slab_is_partial(s)) {
        list_move(&s->list, &cache->partial);
    }
}
```

A fenti kód erősen leegyszerűsített; a valóságban a per-CPU cache, a zárkezelés és a hibakereső eszközök (például red zone, poisoning) is része az implementációnak.

13.1.13 4.3. Előnyök és hátrányok

A slab allokátor legfontosabb előnye a gyorsaság és a jó memória-kihasználtság az azonos méretű objektumok esetén. Mivel az objektumok ugyanakkorák, nincs keresési idő, és a töredezettség minimális. A slab emellett lehetővé teszi a gyors inicializálást és a konstruktor/destruktor hívásokat is. A hátránya a bonyolultság és az, hogy nem alkalmas változó méretű kérések közvetlen kiszolgálására; ahhoz a slab fölé egy külön méretosztály-rendszer kell. A slab allokátor akkor hatékony, ha sok kis, azonos méretű objektumot hozunk létre és szüntetünk meg.

13.1.14 5. Allokációs fejléc és alignment

A heapallokátorok mindegyike metaadatokat tárol a kiosztott blokk mellett. A legegyszerűbb megoldás a **beágyazott fejléc**: a blokk elején, a hasznos adat előtt helyezkedik el egy rögzített méretű struktúra, amely a méretet, az állapotot és esetleg mutatókat tartalmaz. A `malloc`

által visszaadott mutató a fejléc után, azaz a hasznos terület kezdetére mutat. A `free` ebből a mutatóból úgy kapja meg a fejlécet, hogy a mutató értékéből kivonja a fejléc méretét.

Fontos, hogy a hasznos terület kezdőcíme **igazított** legyen. A legtöbb architektúra megköveteli, hogy a 4 bájtos egész 4 bájtra, a 8 bájtos mutató 8 bájtra legyen igazítva. Ha a heap nem igazított címet ad vissza, a processzor hibát jelezhet, vagy a memória-hozzáférés lassabb lesz. Az allokátornak ezért a kért méretet és a fejléc méretét is úgy kell kerekítenie, hogy a visszaadott mutató megfeleljen a megkívánt igazításnak.

Az igazítás tipikus képlete:

```
#define ALIGN 16
size_t align_up(size_t n) {
    return (n + ALIGN - 1) & ~(ALIGN - 1);
}
```

Itt a 16 bájtos igazítás gyakori választás, mert az x86-64-en a SIMD regiszterek is 16 bájtos igazítást igényelnek. A fejléc méretét is igazítani kell, hogy a hasznos terület kezdőcíme igazított legyen. Ha a fejléc például 24 bájt, akkor 16 bájtos igazítás mellett a hasznos terület kezdőcíme 8 bájttal eltérhet a 16 bájtos rasztertől, ezért a fejlécet 16 bájtosra kell kerekíteni.

Az allokációs fejléc tipikus hibái:

- **Elfelejtett igazítás:** a mutató nem a várt igazítás szerint áll, ami hardverhibához vagy lassú hozzáféréshez vezet.
- **Fejléc és hasznos terület átfedése:** ha a fejléc méretét rosszul számoljuk, a program adatai felülírhatják a metaadatokat.
- **Méret túlcsordulás:** nagy méret esetén a `size + header_size` kifejezés túlcsordulhat, ezért a méretet ellenőrizni kell a hozzáadás előtt.
- **Kettős felszabadítás:** ugyanazt a mutatót kétszer adjuk át a `free`-nek, ami a lista korrupciójához vezet.
- **Használat felszabadítás után:** a blokk visszakerül a szabad listába, de a program továbbra is hivatkozik rá.

13.1.15 6. Összehasonlítás

A három stratégia főbb jellemzőit az alábbi táblázat foglalja össze.

Jellemző	Free list	Buddy	Slab
Alapötlet	Láncolt lista szabad blokkokról	Kettő hatványú blokkok páros összevonása	Azonos objektumok cache-elt kiosztása
Keresési idő	$O(n)$, illesztési stratégiától függ	$O(\log N)$, méretosztályonként lista	$O(1)$, per-CPU cache-sel
Töredezettség	Külső töredezettség lehet	Belső töredezettség jellemző	Minimális azonos méretnél
Tipikus felhasználás	Általános célú heap, `malloc`	Fizikai lapkeretek, DMA-pufferek	Kernel apró objektumok, `kmalloc`
Bonyolultság	Alacsony/közepes	Közepes	Magas

A gyakorlatban a modern rendszerek kombinálják ezeket. A Linux például a fizikai lapokat buddy allokátorral osztja ki, a kernel apró objektumait pedig slab/slub allokátorral kezeli. A

felhasználói térben a `malloc` gyakran free list alapú, de méretosztályokat és egyéb optimalizációkat is használ.

13.1.16 7. Labor: egyszerű free list allokátor implementálása

A fejezet laborja egy minimalista free list alapú heapallokátor elkészítése emulált környezetben. A cél nem egy termelésre alkalmas `malloc`, hanem a fogalmak gyakorlása: fejléc, igazítás, hasítás, összevonás és a mérési módszertan.

13.1.17 7.1. Kiinduló állapot és eszközök

- **Környezet:** QEMU emulált x86-64 gép, 128 MiB RAM, freestanding C kernel vagy egyszerű user-space tesztprogram. A labor leírásban user-space tesztet használunk, mert így közvetlenül fut a hoszton, de a kód ugyanaz lehet egy kernelmodulban is.
- **Fordító:** GCC, `-O2 -Wall -Wextra -std=c11`.
- **Linker:** nem szükséges külön, a program egyetlen C fájlból áll.
- **Mérési módszer:** `clock_gettime(CLOCK_MONOTONIC)` az időméréshez, valamint a heapből kiosztott bájtok és a ténylegesen felhasznált bájtok naplózása.

A heap egy statikusan lefoglalt, 1 MiB méretű tömb, amelyet az allokátor kezel. A `heap` mutató a tömb elejére mutat, a `heap_size` a tömb mérete. A fejléc mérete 16 bájt, az igazítás 16 bájt.

13.1.18 7.2. Implementáció

Az alábbi C kód egy explicit free list allokátort mutat be. A kód szemléltetés, nem éles rendszerre való.

```c
#include <stddef.h>
#include <stdint.h>
#include <stdbool.h>

#define HEAP_SIZE (1024 * 1024)
#define ALIGN 16
#define HEADER_SIZE 16

typedef struct block {
    size_t size;          /* hasznos méret, fejléc nélkül */
    bool free;
    struct block *next;   /* csak szabad blokk esetén érvényes */
    struct block *prev;   /* csak szabad blokk esetén érvényes */
} block_t;

static uint8_t heap[HEAP_SIZE];
static block_t *free_list = NULL;

static size_t align_up(size_t n) {
    return (n + ALIGN - 1) & ~(ALIGN - 1);
}

static void list_insert(block_t *b) {
    b->next = free_list;
    b->prev = NULL;
    if (free_list)
        free_list->prev = b;
    free_list = b;
```

```
}

static void list_remove(block_t *b) {
    if (b->prev)
        b->prev->next = b->next;
    else
        free_list = b->next;
    if (b->next)
        b->next->prev = b->prev;
    b->next = b->prev = NULL;
}

static void heap_init(void) {
    block_t *first = (block_t *)heap;
    first->size = HEAP_SIZE - HEADER_SIZE;
    first->free = true;
    first->next = first->prev = NULL;
    free_list = first;
}

void *my_malloc(size_t size) {
    if (size == 0)
        return NULL;
    size = align_up(size);

    block_t *b = free_list;
    while (b) {
        if (b->free && b->size >= size) {
            /* Hasítás, ha a maradék elég egy fejléc + minimális blokk */
            if (b->size >= size + HEADER_SIZE + ALIGN) {
                block_t *newb = (block_t *)((uint8_t *)(b + 1) + size);
                newb->size = b->size - size - HEADER_SIZE;
                newb->free = true;
                newb->next = b->next;
                newb->prev = b->prev;
                if (newb->next)
                    newb->next->prev = newb;
                if (newb->prev)
                    newb->prev->next = newb;
                else
                    free_list = newb;
                b->size = size;
            } else {
                list_remove(b);
            }
            b->free = false;
            return (void *)(b + 1);
        }
        b = b->next;
    }
    return NULL;
}

void my_free(void *ptr) {
    if (!ptr)
        return;
    block_t *b = (block_t *)ptr - 1;
    b->free = true;

    /* Összevonás a következő szabad blokkal */
    block_t *next = (block_t *)((uint8_t *)(b + 1) + b->size);
    if ((uint8_t *)next < heap + HEAP_SIZE && next->free) {
        list_remove(next);
```

```
        b->size += HEADER_SIZE + next->size;
    }

    /* Összevonás az előző szabad blokkal */
    if ((uint8_t *)b > heap) {
        block_t *prev = (block_t *)((uint8_t *)b - sizeof(block_t));
        (void)prev; /* itt egyszerűsítés: az előző blokk keresése bonyolultabb */
    }

    list_insert(b);
}
```

A fenti kód a könnyebb érthetőség kedvéért nem teljes, például az előző blokk összevonását nem valósítja meg maradéktalanul. A labor célja, hogy a hallgató ezt kiegészítse, és a hibákat megtalálja.

13.1.19 7.3. Futtatás és elvárt eredmény

A tesztprogram a következőket végzi:

1. Inicializálja a heapet.
2. Lefoglal 1000 darab, véletlenszerűen 16 és 512 bájt közötti blokkot.
3. Felszabadítja a blokkok felét.
4. Lefoglal 500 darab 128 bájtos blokkot.
5. Kiírja a heap állapotát: összes szabad memória, legnagyobb szabad blokk, és a foglalások számát.

A várt eredmény az, hogy a heap nem fogy ki, és a legnagyobb szabad blokk nagyobb, mint a legnagyobb kért blokk, tehát az összevonás működik. A mérési naplóba rögzíteni kell:

- **Mért adat:** a `my_malloc` és `my_free` hívások átlagos futásideje, a heap kihasználtsága, a legnagyobb szabad blokk mérete.
- **Várt adat:** a free list first-fit esetén lineáris keresési időt feltételez, ezért a foglalási idő a szabad blokkok számával arányosan nő.
- **Mérési feltételek:** QEMU 7.2.0, x86-64, 2 GHz virtuális órajel, `-O2`, 10 ismételt futtatás, az első futás bemelegítés.

A mérési napló egy lehetséges részlete:

Mérés	Érték (mért)	Várt
`my_malloc` átlagos idő 1000 hívásra	3,2 µs	$O(n)$, n a szabad blokkok száma
`my_free` átlagos idő 1000 hívásra	1,8 µs	$O(1)$ plusz összevonás
Legnagyobb szabad blokk a teszt végén	516 KiB	legalább 512 KiB
Belső fragmentáció	12,4%	$< 15\%$

A mért és a várt értéket mindig külön kell jelölni. A labor kiinduló állapota a `heap_init` utáni állapot, a visszaállítás pedig a heap újrainicializálása.

13.1.20 8. Önellenőrző kérdések

1. Miért fontos az összevonás a free list allokátorban? Mi történik, ha elhagyjuk?
2. A buddy allokátor miért szenved belső töredezettségtől? Adj példát egy 6 KiB-os kérésre.
3. Miben jobb a slab allokátor a free listnél az azonos méretű kernel objektumok esetén?
4. Hogyan számítja ki egy allokátor a hasznos terület kezdőcímét úgy, hogy az igazított legyen? Mi a teendő, ha a fejléc mérete nem többszöröse az igazításnak?

A következő alfejezet (Fragmentáció és allokációs stratégiák) ezekre az alapokra építve a fragmentációt és az allokációs stratégiákat vizsgálja részletesebben.

13.2 Fragmentáció és allokációs stratégiák

A heap-allokátor célja nemcsak az, hogy gyorsan kiszolgálja a memóriakéréseket, hanem az is, hogy a rendelkezésre álló memóriát hatékonyan használja fel. Ha egy allokátor túlzottan apró darabokra szabdalja a szabad területet, vagy a lefoglalt blokkokban nagy, kihasználatlan részeket hagy, akkor a látszólag elegendő memória ellenére is meghiúsulhatnak az új kérések. Ezt a jelenséget **fragmentációnak** nevezzük, és két alapvető formáját különböztetjük meg: a **belső** és a **külső fragmentációt**. A választott allokációs stratégia – például a first fit, best fit vagy worst fit – közvetlenül befolyásolja, hogy milyen mértékű fragmentáció alakul ki, és hogy milyen költséggel lehet azt kezelni. Ebben az alfejezetben a fragmentáció mérését, az allokációs stratégiák működését, valamint a két fő ellenszer, a **coalescing** (összevonás) és a **compaction** (tömörítés) technikáit tárgyaljuk. A fogalmakat egy reprodukálható mérési labor egészíti ki, amelyben az olvasó saját maga is összehasonlíthatja az egyes stratégiák hatását.

13.2.1 Belső fragmentáció

A belső fragmentáció akkor keletkezik, amikor az allokátor a kért méretnél nagyobb blokkot ad át a hívónak. A különbség a lefoglalt blokk teljes mérete és a ténylegesen hasznos (a program által felhasznált) méret között elveszett terület, amelyet a program nem tud más célra fordítani, de az allokátor sem tudja másnak kiadni, amíg a blokk le van foglalva.

A belső fragmentáció tipikus forrásai a következők:

1. **Igazítási követelmények (alignment):** A processzorok és az ABI-k gyakran megkövetelik, hogy a lefoglalt memória kezdőcíme bizonyos hatvány kettővel (pl. 8, 16 vagy 32 bájttal) osztható legyen. Ha a program például 10 bájtot kér, az allokátor 16 bájtos igazítás mellett kénytelen 16 bájtos blokkot kiadni; a 6 bájtnyi többlet belső fragmentáció.
2. **Fejléc (header) és adminisztratív adatok:** A legtöbb allokátor a hasznos terület előtt vagy után saját metaadatokat tárol, például a blokk méretét, állapotát (foglalt/szabad), és esetleg láncolási pointereket. Ez a fejléc a kiosztott blokk része, de a hívó számára nem hasznos terület. Ha egy allokátor blokkonként 16 bájtos fejlécet használ, akkor egy 100 bájtos kéréshez legalább 116 bájtot kell lefoglalnia, így a belső fragmentáció a teljes blokk és a hasznos méret különbsége.
3. **Minimális blokkméret:** A free list és a buddy rendszerű allokátorok gyakran nem adnak ki egy bizonyos minimális méretnél kisebb blokkot, mert az adminisztráció és a későbbi újrahasznosítás hatékonysága ezt indokolja. Például ha a minimális blokkméret 32 bájt, akkor egy 5 bájtos kérés is 32 bájtot foglal el.

A belső fragmentáció mértékét egyetlen blokkra a következő képlettel fejezhetjük ki:

$$F_{\text{belső, i}} = 1 - \frac{a_i}{h_i}$$

ahol a_i a kért hasznos méret, h_i pedig a kiosztott blokk teljes mérete (fejléccel és igazítással együtt). A teljes heapre vonatkozó belső fragmentáció a következőképpen számolható:

$$F_{\text{belső}} = 1 - \frac{\sum_{i=1}^{n} a_i}{\sum_{i=1}^{n} h_i}$$

ahol n az éppen foglalt blokkok száma. Az érték 0 és 1 közé esik; a magasabb érték nagyobb pazarlást jelez. A gyakorlatban a belső fragmentáció jól becsülhető a kérések méreteloszlása, az igazítási szabály és a fejlécméret alapján, ezért a méréskor a várt érték könnyen levezethető.

13.2.2 Külső fragmentáció

A külső fragmentáció ezzel szemben a szabad memória aprózódásából adódik. Az allokációk és felszabadítások időbeli sorozata során a heap szabad területei sok kis, nem összefüggő darabra eshetnek szét. Bár a szabad bájtok összessége elegendő lenne egy új kérés kiszolgálására, a legnagyobb összefüggő szabad blokk kisebb, mint a kért méret, ezért a foglalás meghiúsul. Ezt a jelenséget **szétszórt szabad területnek** is nevezik.

A külső fragmentáció mérésére többféle metrika használható. Az egyik legegyszerűbb a következő arány:

$$F_{\text{külső}} = 1 - \frac{S_{\max}}{S_{\text{összes}}}$$

ahol $S_{\max}$ a legnagyobb összefüggő szabad blokk mérete, $S_{\text{összes}}$ pedig a teljes szabad memória mérete. Ha a szabad terület egyetlen összefüggő blokkban van, akkor $S_{\max} = S_{\text{összes}}$, és $F_{\text{külső}} = 0$. Ha viszont a szabad terület sok apró darabra szóródik, a legnagyobb blokk kicsi lesz az összeshez képest, és az arány 1-hez közelít.

Egy másik gyakori megközelítés a **kihasználatlansági szabály** vagy **50%-os szabály**, amely szerint egy first fit alapú, hasonló méretű blokkokat használó allokátor egyensúlyi állapotában a foglalt blokkok száma és a szabad blokkok száma között körülbelül 1:2 arány alakul ki, ami körülbelül 33%-os memóriapazarlást eredményez. Ez persze csak modell, a valós munkaterheléseken az érték jelentősen eltérhet.

A külső fragmentáció különösen akkor okoz gondot, ha a program nagy méretű blokkokat igényel, de a heap tele van apró lyukakkal. Ilyenkor a heap látszólag „üres”, de a nagy blokk számára nincs elegendő összefüggő terület. A külső fragmentáció a beágyazott rendszerekben és a hosszú élettartamú folyamatokban kritikus lehet, mert a memóriaelfogyás látszólagos, pedig a szabad memória összessége bőséges.

13.2.3 Allokációs stratégiák és hatásuk

A free list alapú allokátorokban a szabad blokkokat általában egy láncolt listában tartjuk nyilván. Amikor egy kérés érkezik, az allokátornak ki kell választania egy megfelelően nagy szabad blokkot, és szükség esetén ketté kell vágnia. A kiválasztás módja az **allokációs stratégia**, amelynek alapvetően három klasszikus változata van: **first fit**, **best fit** és **worst fit**. Ezek mellett létezik a **next fit** (következő illesztés) is.

13.2.3.1 First fit

A first fit (első illeszkedő) stratégia a szabad blokkok listáját az elejétől kezdve végigjárja, és az első olyan blokkot választja, amelynek mérete elég nagy a kérés kiszolgálásához. Ha a blokk túl nagy, akkor kettévágja: a szükséges részt lefoglalja, a maradékot pedig visszateszi a szabad listába.

Előnye, hogy egyszerű és gyors, mert általában a lista elején talál megfelelő blokkot. Hátránya, hogy a nagy szabad blokkok gyakran felaprózódnak, mert az első találat kettévágja őket, még akkor is, ha később egy kisebb blokk is megfelelt volna. Emiatt a külső fragmentáció viszonylag magas lehet, különösen akkor, ha a kérések mérete változó.

A first fit egy gyakori optimalizálása a **cím szerinti rendezés**: ha a szabad listát a memóriacímek szerint tartjuk rendezve, akkor a first fit természetesen elősegíti a szomszédos szabad blokkok összevonását, mert a felszabadításkor a szomszédokat könnyen megtaláljuk.

13.2.3.2 Best fit

A best fit (legjobb illeszkedő) stratégia a teljes szabad listát végigvizsgálja, és azt a blokkot választja, amelynek mérete a legkisebb, de még elegendő a kéréshez. Ez a stratégia minimalizálja a kettévágás után keletkező maradékot, így a foglalt blokkok mellett keletkező belső fragmentációt is csökkenti.

Hátránya, hogy a legkisebb megfelelő blokk kiválasztása gyakran eredményez nagyon apró szabad töredékeket, amelyek túl kicsik ahhoz, hogy bármilyen érdemi kérést kiszolgáljanak. Ez a külső fragmentációt növeli, mert a heap tele lesz használhatatlan apró lyukakkal. Ráadásul a best fit időigénye nagyobb, mert minden kérésnél a teljes listát be kell járni, kivéve, ha speciális adatszerkezetet (például méret szerint rendezett fát) használunk.

A best fit különösen akkor előnyös, ha a kérések mérete viszonylag egyenletes, mert ilyenkor a maradékok is egyenletesen oszlanak el, és a kicsi töredékek aránya alacsony marad.

13.2.3.3 Worst fit

A worst fit (legrosszabb illeszkedő) stratégia mindig a **legnagyobb** szabad blokkot választja, feltételezve, hogy a levágás után keletkező maradék is nagy lesz, és később még használható marad. A cél a külső fragmentáció csökkentése, mert a nagy blokkok szétvágásakor keletkező maradékok is nagyok, így kevesebb apró töredék jön létre.

A gyakorlatban azonban a worst fit gyakran rosszabbul teljesít, mint a first fit vagy a best fit, mert a legnagyobb blokk szétvágása gyorsan elhasználja a nagy összefüggő területeket, és a maradékok sokszor még mindig túl nagyok a tipikus kis kérésekhez, így belső fragmentációhoz vezetnek. Ráadásul a worst fit is a teljes lista bejárását igényli, tehát lassabb.

13.2.3.4 Next fit

A next fit (következő illeszkedő) a first fit egy változata, amely nem a lista elejéről indul, hanem onnan folytatja a keresést, ahol az előző allokáció abbahagyta. Ez a stratégia gyorsabban találhat szabad blokkot, mert a keresési pozíciót körbevezeti. A szabad blokkok eloszlása azonban egyenetlenebb lehet, mert a keresés nem a lista elején koncentrálódik, így a heap felső részei is felaprózódnak.

A next fit különösen akkor hasznos, ha a heap nagy, és a first fit a lista elején lévő blokkokat túlzottan apró darabokra vágja, míg a lista vége felé nagy szabad blokkok maradnak. A next fit ezt a torzulást csökkenti, de a külső fragmentációt általában nem csökkenti jelentősen.

13.2.3.5 Összehasonlítás

A stratégiák hatását nehéz univerzálisan rangsorolni, mert az eredmény erősen függ a munkaterheléstől: a kérések méreteloszlásától, a foglalások és felszabadítások sorrendjétől, valamint attól, hogy van-e coalescing. Általánosságban elmondható:

- A **first fit** egyszerű, gyors, de hajlamos a nagy blokkok felaprózására.
- A **best fit** minimalizálja a belső fragmentációt, de sok apró külső töredéket hozhat létre.
- A **worst fit** igyekszik megőrizni a nagy blokkokat, de sokszor pazarló.
- A **next fit** a keresési pozíciót körbevezeti, de a fragmentáció szempontjából általában nem jobb a first fitnél.

A következő – gyakran idézett – szimulációs eredmények szerint a first fit és a best fit a legtöbb általános munkaterhelésen jobb memóriahasznosítást ér el, mint a worst fit, de a különbség kicsi lehet, ha a kérések nagyjából azonos méretűek. A konkrét számok mindig a munkaterheléstől függenek, ezért a választást mérésekkel kell alátámasztani.

13.2.4 Coalescing: a szabad blokkok összevonása

A külső fragmentáció egyik leghatékonyabb ellenszere a **coalescing** (összevonás). Amikor egy blokkot felszabadítunk, a szomszédos szabad blokkokat azonnal vagy késleltetve összevonjuk egyetlen nagyobb szabad blokkká. Ezáltal a szabad terület összefüggőbbé válik, és a legnagyobb szabad blokk mérete nő, ami csökkenti a külső fragmentációt.

A coalescing implementálásának két alapesete:

1. **Azonnali összevonás (immediate coalescing):** A felszabadításkor azonnal megvizsgáljuk a felszabadított blokk előtti és utáni szomszédot. Ha valamelyik szabad, akkor a blokkokat egyesítjük. Ez a legegyszerűbb és legelterjedtebb megközelítés, mert a szabad lista mindig friss, és a fragmentáció nem halmozódik fel.
2. **Késleltetett összevonás (deferred coalescing):** A felszabadított blokkot először külön listára tesszük, és csak akkor vonjuk össze a szomszédjaival, ha egy új kérés kiszolgálásához nincs elegendő méretű blokk. Ez a módszer csökkenti a felszabadítás közvetlen költségét, de bonyolultabb, és a szabad lista állapota átmenetileg inkonzisztens lehet a fragmentáció szempontjából.

A szomszédos blokkok gyors megtalálása érdekében a blokkok fejlécében általában tárolják a blokk méretét és állapotát. A **boundary tag** technika a blokk elején és végén egyaránt elhelyezi a méret és állapot adatokat. Így a felszabadított blokk előtti szomszéd mérete a felszabadított blokk kezdőcíme előtti néhány bájtból kiolvasható, ami lehetővé teszi a szomszéd tényleges címének kiszámítását anélkül, hogy a teljes listát be kellene járni.

A coalescing hátránya, hogy a felszabadítás költségét növeli, különösen akkor, ha gyakoriak a felszabadítások. A modern allokátorok ezért gyakran alkalmaznak valamilyen kompromisszumot, például csak a nagyobb blokkokat vonják össze azonnal, a kicsiket pedig egy elkülönített gyorsítótárban tartják.

13.2.5 Compaction: a memória tömörítése

A coalescing önmagában nem képes minden külső fragmentációt megszüntetni, mert a foglalt blokkok közé ékelődött szabad lyukakat nem tudja áthelyezni. A **compaction** (tömörítés) ezzel szemben a lefoglalt blokkok fizikai áthelyezésével egyetlen összefüggő szabad területet hoz létre a heap egyik végén. Ez a technika azonban csak akkor alkalmazható, ha az allokátor képes a foglalt blokkokra mutató pointereket frissíteni, vagy ha a programok indirekt hivatkozásokat használnak.

A compaction fő lépései:

1. **A foglalt blokkok azonosítása:** Végig kell járni a heapet, és meg kell határozni az összes foglalt blokk kezdőcímét és méretét.
2. **A blokkok áthelyezése:** A foglalt blokkokat a heap elejére (vagy egy meghatározott irányba) mozgatjuk úgy, hogy közöttük ne maradjon szabad rés. Ez magában foglalja a blokkok tartalmának memcpy-szerű másolását.
3. **Pointerek frissítése:** Minden olyan mutatót, amely az áthelyezett blokkra hivatkozott, frissíteni kell az új címre. Ez a legnehezebb rész, mert a program összes mutatóját – beleértve a stacken, regiszterekben és más adatszerkezetekben tárolt hivatkozásokat is – meg kell találni, ami általános célú rendszerekben szinte lehetetlen. Ezért a compaction elsősorban olyan környezetekben működik, ahol a pointerek kezelése kontrollált, például szemétgyűjtővel ellátott nyelvekben (Java, .NET) vagy speciális beágyazott rendszerekben.

A compaction költsége arányos a lemásolt adatmennyiséggel, ezért a gyakorlatban ritkábban, csak akkor hajtják végre, ha a fragmentáció már kritikus mértékű. A coalescinghez képest a compaction drasztikusan csökkentheti a külső fragmentációt, de a futási időre és az energiafogyasztásra gyakorolt hatása miatt kerülendő a valós idejű rendszerekben.

13.2.6 Gyakorlati megfontolások és tipikus hibák

A fragmentáció kezelése során számos tipikus hibát követhetünk el, amelyeket érdemes elkerülni:

- **A belső fragmentáció figyelmen kívül hagyása:** Sok fejlesztő csak a külső fragmentációra koncentrál, pedig a túl nagy fejlécek és a szükségtelenül nagy igazítási értékek jelentős memóriapazarlást okozhatnak. A mérés során mindkét típust külön kell számszerűsíteni.
- **A stratégia vak megválasztása:** A best fit nem mindig jobb, mint a first fit. A munkaterhelés elemzése nélkül hozott döntés könnyen vezethet rosszabb teljesítményhez. A legjobb stratégia kiválasztása mindig mérést igényel.

- **A coalescing elhagyása:** A free list alapú allokátorok coalescing nélkül hamarosan használhatatlanul apró darabokra esnek szét. A felszabadításkor elvégzett összevonás elengedhetetlen a külső fragmentáció kordában tartásához.
- **A compaction alkalmazása nem áthelyezhető blokkokra:** Ha a program közvetlen mutatókat tárol a heap blokkokra, a compaction elvégzése után ezek a mutatók érvénytelenek lesznek, ami memóriahibákhoz vezet. A compaction csak olyan környezetben biztonságos, ahol a pointerek frissítése megoldott.
- **A mérés hiánya:** A fragmentáció mértékét nem elég egyszeri pillanatfelvételként megadni; időbeli alakulását is nyomon kell követni. A mérési naplónak tartalmaznia kell a konfigurációt, a mérési pontokat és a reprodukálhatósághoz szükséges adatokat.

13.2.7 Labor: fragmentációs mérés

Ez a labor egy egyszerű, felhasználói térben futó C programot használ, amely saját memóriakezelőt szimulál egy fix méretű tömb felett. A program a free list alapú allokátor három stratégiáját (first fit, best fit, worst fit) valósítja meg, és méri a belső és külső fragmentációt egy előre meghatározott allokációs és felszabadítási sorozat során. A labor célja, hogy a mért eredményeket összevessük a levezethető várt értékekkel, és megértsük a stratégiaválasztás hatását.

13.2.7.1 Kiinduló állapot és környezet

A labor reprodukálható bármely Linux rendszeren, például egy QEMU virtuális gépben vagy egy konténerben. A konfiguráció:

- Rendszer: Linux 6.x (x86_64)
- Fordító: GCC 12.2 vagy újabb
- CPU: legalább 1 mag, 2 GHz (az időmérés nem kritikus, mert a labor nem teljesítményteszt)
- Memória: elegendő a hoszton, de a szimulált heap 32 KiB
- A program nem nyúl a hoszt valódi memóriájához, csak a saját processzének memóriájában lefoglalt tömböt használja.

A kódot `frag_sim.c` néven mentsük el, majd fordítsuk a következő paranccsal:

```
gcc -O2 -Wall -Wextra -o frag_sim frag_sim.c
```

A futtatás előtt állítsuk be a véletlenszám-generátor magját (seed) a reprodukálhatóság érdekében:

```
./frag_sim first 12345 > meres_first.txt
./frag_sim best 12345 > meres_best.txt
./frag_sim worst 12345 > meres_worst.txt
```

A program argumentumként kapja a stratégiát (`first`, `best`, `worst`) és a seedet. A kimenet a mérési napló, amelyet a laborjegyzőkönyvbe kell másolni.

13.2.7.2 A szimulátor felépítése

A program egy 32 KiB méretű statikus tömböt használ heapként. A blokkok fejléce a következő struktúra:

```
typedef struct block {
    size_t size;          // hasznos terület mérete bájtban
    bool free;            // szabad-e a blokk
    struct block *next;   // következő blokk a listában
    struct block *prev;   // előző blokk a listában
} block_t;
```

A fejléc mérete a `sizeof(block_t)` értéke, amely 64 bites rendszeren tipikusan 32 bájt (méret 8, bool 1 + padding, két mutató 8+8). A hasznos terület a fejléc után kezdődik, és a kért méretet 8 bájtra igazítjuk fel.

A heap inicializálásakor egyetlen, 32 KiB méretű szabad blokkot hozunk létre. A `block_t` fejlécet a heap elejére helyezzük, a szabad terület pedig a fejléc után következik.

Az allokációs kérések sorozata a következő (ezek a méretek bájtban értendők):

```
100, 200, 50, 80, 120, 40, 300, 60, 150, 30
```

A felszabadítások sorrendje a következő indexeket érinti (0-tól számozva):

```
1, 3, 5, 7, 9
```

Tehát először mind a 10 blokkot lefoglaljuk, majd az 1., 3., 5., 7., 9. sorszámú blokkokat felszabadítjuk. Ezzel a szabad blokkok közé foglalt blokkok ékelődnek, ami lehetővé teszi a fragmentáció mérését.

A program a kérések feldolgozása után kiszámítja:

- **Belső fragmentáció:** a foglalt blokkokra $\sum (h_i - a_i)$ érték, ahol h_i a kiosztott blokk teljes mérete (fejléc + igazított hasznos terület), a_i pedig a kért hasznos méret.
- **Külső fragmentáció:** a szabad memória összesen ($S_{\text{összes}}$) és a legnagyobb összefüggő szabad blokk ($S_{\max}$) aránya.

A program a stratégiától függően választja ki a szabad blokkot:

```
block_t *find_first(size_t size) {
    block_t *b = list_head;
    while (b) {
        if (b->free && b->size >= size) return b;
        b = b->next;
    }
    return NULL;
}

block_t *find_best(size_t size) {
    block_t *best = NULL;
    block_t *b = list_head;
    while (b) {
        if (b->free && b->size >= size &&
            (!best || b->size < best->size)) best = b;
        b = b->next;
    }
    return best;
}

block_t *find_worst(size_t size) {
```

```
    block_t *worst = NULL;
    block_t *b = list_head;
    while (b) {
        if (b->free && b->size >= size &&
            (!worst || b->size > worst->size)) worst = b;
        b = b->next;
    }
    return worst;
}
```

A felszabadítás során a program elvégzi a coalescinget: ha a felszabadított blokk előtti vagy utáni szomszéd szabad, akkor összevonja őket. A szomszédok azonosításához a blokkok méretét használjuk, vagy egyszerűbben a lista sorrendjében haladva a pointerek alapján.

A program kimenete a mérési napló, amely a következőket tartalmazza:

- A stratégia neve és a seed.
- A heap teljes mérete.
- A kiosztott blokkok listája (cím, kért méret, kiosztott méret, szabad-e).
- A belső fragmentáció összesített értéke.
- A külső fragmentáció értéke, valamint $S_{\text{összes}}$ és $S_{\max}$.
- A várt értékek, amelyeket a laborjegyzőkönyvben levezetünk.

13.2.7.3 Várt értékek levezetése

A várt belső fragmentáció kiszámítható a kérés méretének 8 bájtra való igazításával és a fejléc hozzáadásával. Például egy 100 bájtos kérésnél:

- Igazított hasznos méret: $\lceil 100/8 \rceil \times 8 = 104$ bájt.
- Kiosztott blokk mérete: $104 + 32 = 136$ bájt.
- Belső fragmentáció erre a blokkra: $136 - 100 = 36$ bájt.

A teljes belső fragmentáció a foglalt blokkokra vonatkozóan a fenti képlet alapján számítható. A várt külső fragmentáció pedig a felszabadítások utáni állapotból vezethető le a választott stratégiától függően. A first fit esetén a blokkokat a lista elejétől kezdve osztja ki, így a szabad blokkok elrendezése előre kiszámítható. A laborjegyzőkönyvben a hallgatónak ezt a levezetést rögzítenie kell, és össze kell vetnie a program kimenetével.

13.2.7.4 Mérési napló és kiértékelés

A mérési napló táblázatos formában rögzíti a mért és várt értékeket:

Stratégia	Belső frag. (mért)	Belső frag. (várt)	Külső frag. (mért)	Külső frag. (várt)
first fit	236 bájt	236 bájt	0,72	0,72
best fit	236 bájt	236 bájt	0,55	0,55
worst fit	236 bájt	236 bájt	0,81	0,81

A fenti számok csak illusztrációk; a tényleges értékek a program pontos implementációjától és a fejléc méretétől függenek. A hallgató feladata, hogy a saját mérési eredményeit hasonló táblázatban rögzítse, és ellenőrizze, hogy a mért és várt értékek egyeznek-e. Ha eltérés van, meg kell vizsgálni az okát, például a fejléc méretének vagy az igazítási szabálynak a téves feltételezését.

A mérési naplóhoz tartozik a visszaállítási út is: a labor nem módosít semmilyen hoszt rendszerfájlt, ezért a törléshez elegendő a `frag_sim` bináris és a kimeneti fájlok eltávolítása.

13.2.8 Önellenőrző kérdés

Adott egy 16 KiB méretű heap, amelyben a fejléc mérete 16 bájt, az igazítás 8 bájt. A program lefoglal három blokkot: A = 3 KiB, B = 5 KiB, C = 4 KiB. Ezután a B blokkot felszabadítja. Számítsa ki a külső fragmentációt first fit stratégia esetén, ha a foglalások sorrendje A, B, C, és a felszabadítás után a coalescing azonnal megtörténik. Magyarázza el, hogyan változna az eredmény, ha a foglalások sorrendje C, A, B lenne, és a felszabadítás után best fit stratégiát használnánk.

A válaszban térjen ki arra, hogy a külső fragmentáció hogyan függ a foglalások sorrendjétől és a választott stratégiától, és hogy a coalescing milyen szerepet játszik a szabad terület összefüggővé tételében.

13.2.9 Összefoglalás

A fragmentáció két formája – a belső és a külső – elkerülhetetlen velejárója a heap-alapú memóriakezelésnek. A belső fragmentáció a kiosztott blokk és a hasznos terület közötti különbségből adódik, míg a külső fragmentáció a szabad memória aprózódását jelenti. Az allokációs stratégia (first fit, best fit, worst fit, next fit) közvetlenül befolyásolja mindkettő alakulását, ezért a választást a munkaterhelés ismeretében, mérésekkel alátámasztva kell meghozni. A coalescing a szabad blokkok összevonásával, a compaction pedig a foglalt blokkok áthelyezésével csökkentheti a külső fragmentációt, bár utóbbi csak speciális környezetekben alkalmazható. A laborban bemutatott szimulátor segítségével a mérnökök saját maguk is megfigyelhetik a stratégiák hatását, és ellenőrizhetik a levezetett várt értékeket. A következő alfejezet a NUMA, a memória-localitás és a kernelmemória-kezelés fejlettebb kérdéseire épít, kihasználva az itt bevezetett fragmentációs fogalmakat.

13.3 NUMA, locality és memóriapoolok

A többcsomópontos memória (Non-Uniform Memory Access, NUMA) akkor válik meghatározóvá, amikor a rendszer több processzorfoglalattal rendelkezik, és mindegyik foglalathoz külön memóriavezérlő, illetve helyileg csatlakoztatott fizikai memória tartozik. A klasszikus szimmetrikus többprocesszoros (SMP) modellben a memória egységes erőforrás: minden processzormag közel azonos késleltetéssel és sávszélességgel éri el a teljes címtartományt. NUMA esetén ez a feltételezés nem áll fenn. A processzor számára a saját node-jához tartozó memória elérése gyorsabb, mint egy másik node memóriájának elérése, mert az utóbbihoz a node-ok közötti összeköttetésen (interconnecten) keresztül kell forgalmat bonyolítani.

Ebben az alfejezetben a többcsomópontos memória és a helyi hozzáférés fogalmát járjuk körül, bemutatjuk a NUMA node, a local/remote access és a memóriapool szerepét, majd kitérünk a memcg-re mint erőforrás-elszámolási és korlátozási mechanizmusra. A cél nem csupán az elméleti megértés, hanem az is, hogy felismerjük a tipikus tervezési hibákat, és képesek legyünk NUMA-affinitási tervet készíteni.

13.3.1 A többcsomópontos memória és a helyi hozzáférés

A NUMA-rendszerekben a fizikai memória több csomópontra (node-ra) oszlik. Minden node processzormagok egy halmazát, egy helyi memóriavezérlőt és közvetlenül csatlakoztatott memóriát, valamint általában helyi I/O eszközöket foglal magában. A node-ok nagy sebességű, de korlátozott sávszélességű kapcsolattal kommunikálnak egymással. Ilyen kapcsolat lehet például az Intel Ultra Path Interconnect (UPI), az AMD Infinity Fabric vagy más gyártóspecifikus interconnect. A kapcsolat sávszélessége és késleltetése jellemzően rosszabb, mint a node-on belüli memóriavezérlő és a processzormagok közötti elérés.

A helyi hozzáférés (local access) azt jelenti, hogy a futó szál a saját node-ján lévő memóriát olvassa vagy írja. A távoli hozzáférés (remote access) ezzel szemben olyan memóriacím elérése, amely egy másik node fizikai memóriájában található. A hozzáférési idő különbségét két tényező határozza meg: a távoli node memóriavezérlőjének eléréséhez szükséges interconnect-késleltetés, valamint a kapcsolat sávszélessége, amely megoszlik a node-ok közötti forgalommal. Egy tipikus NUMA rendszerben a távoli elérés késleltetése másfélszer-kétszerese vagy akár nagyobb is lehet a helyi elérésének, a sávszélesség pedig node-ok közötti forgalom esetén gyakran töredéke a helyi sávszélességnek. Ezek az arányok architektúrafüggők, ezért konkrét mérés nélkül nem szabad általánosítani.

Az operációs rendszer számára a NUMA architektúra két alapvető feladatot vet fel: az ütemezésnél és a memóriakezelésnél figyelembe kell venni a node-ok közötti aszimmetriát. Ha a kernel egy folyamatot egy adott node-on futtat, de a folyamat által használt lapok egy másik node-on helyezkednek el, akkor a futás során folyamatos távoli memóriaelérések keletkeznek, ami lerontja a teljesítményt, és feleslegesen terheli az interconnectet. Ezért a NUMA-tudatos operációs rendszerek igyekeznek a processzorokat és a hozzájuk tartozó memóriát együtt kezelni: a lapallokációt a futó processzor node-jához közeli memóriából végzik, az ütemező pedig lehetőség szerint ugyanarra a node-ra helyezi a szálat, ahol a memóriája található.

A hozzáférési aszimmetria formálisan is leírható. Ha egy feladat N memóriaműveletéből N_{local} a helyi, N_{remote} a távoli node-ot érinti, és a helyi, illetve távoli műveletek átlagos késleltetése t_{local}, illetve t_{remote}, akkor az átlagos memóriakésleltetés:

$$t_{avg} = \frac{N_{local}}{N} t_{local} + \frac{N_{remote}}{N} t_{remote}.$$

A cél az, hogy a t_{avg} minél közelebb legyen a t_{local} értékhez, vagyis a távoli hozzáférések arányát minimalizáljuk. Ezt a célt szolgálják a node-affinitási és lapelhelyezési politikák.

13.3.2 NUMA node, local/remote access és pool

A NUMA node operációs rendszerbeli reprezentációja magában foglalja a node-hoz tartozó processzorazonosítókat, a helyi memória mennyiségét és a többi node-tól vett távolsághinformációt. Linux alatt a `dmesg` vagy a `numactl --hardware` parancs mutatja a topológiát. A node-ok közötti távolság általában egy mátrixban jelenik meg, ahol a főátló a lokális hozzáférést jelzi (általában 10-es érték, ami a helyi késleltetést normált formában ábrázolja), a nem átlós elemek pedig a távoli node-ok elérési költségét. A távolságértékeket a firmware (ACPI SRAT/SLIT táblák) szolgáltatja, és a kernel ezek alapján építi fel a NUMA-topológiát.

Példaként vegyünk egy kétnode-os rendszert nyolc logikai processzorral: a `node0` a 0–3 CPU-kat és 1 GiB memóriát, a `node1` a 4–7 CPU-kat és 1 GiB memóriát tartalmazza. A távolságmátrix azt mutatja, hogy a 0. node-ból a 0. node elérése lokális, az 1. node elérése távoli; hasonlóan

az 1. node-ból a 0. node távoli. A kernel a lapallokáció során ezt a távolságmátrixot használja, amikor kiválasztja, melyik node szabad lapjait ossza ki egy adott processzornak.

A local/remote access megkülönböztetés a teljesítmény mellett a kapacitás szempontjából is fontos. Ha egy node helyi memóriája kimerül, de a másik node-on még van szabad memória, a kernel kénytelen távoli node-ról allokálni. Ilyenkor a rendszer továbbra is működik, de az adott folyamat teljesítménye romlik. Egyes rendszerek konfigurálhatók úgy, hogy a távoli allokációt preferálják vagy letiltják bizonyos feltételek mellett, például memóriapolicy segítségével.

A **memóriapool** (memory pool) általános értelemben előre lefoglalt, rögzített méretű blokkok halmaza, amelyet az alkalmazás vagy a kernel gyorsan kioszthat és visszavehet anélkül, hogy minden egyes allokációnál a teljes memóriakezelőt kellene meghívnia. A poolok csökkentik az allokációs overheadet, a fragmentációt, és lehetővé teszik a memória meghatározott célra való elkülönítését. NUMA-környezetben a memóriapoolok akkor hatékonyak, ha node-onként külön poolokat tartunk fenn. Ekkor a node-on futó szálak a helyi poolból allokálnak, elkerülve a távoli memóriafoglalást és a globális poolhoz tartozó zárolást.

A kernelben a NUMA-tudatos memóriapoolok több szinten jelennek meg. A page allocator node-onként tartja nyilván a szabad lapokat, és az allokációk során a `alloc_pages_node()` vagy `__alloc_pages_node()` függvényekkel konkrét node-ot lehet kérni. A slab/slub cache réteg szintén node-onkénti, per-CPU gyorsítótárat használ: a `kmalloc_node()` függvény a megadott node memóriájából allokál, míg a `kmalloc()` általában a aktuális processzor node-ját választja. A gyakorlati kernelprogramozásban a per-node poolok nélkülözhetetlenek az olyan struktúrákhoz, amelyeket gyakran érnek el egy adott node-on futó szálak, például hálózati pufferek, ütemezői adatszerkezetek vagy fájlrendszer-metadatok.

13.3.3 Memóriapool és memcg

A **memcg** (memory cgroup) a Linux kernel cgroup-alrendszerének az a része, amely a memóriahasználatot számolja el és korlátozza. A memcg lehetővé teszi, hogy egy folyamatcsoport számára felső korlátot szabjunk a fizikai memóriafogyasztásra, és a kernel a csoport összes lapját nyilvántartsa. NUMA-rendszerekben a memcg-nek node-onkénti statisztikái is vannak: a `memory.numa_stat` fájl megmutatja, hogy az adott cgroup hány lapot használ az egyes node-okon. Ez azért fontos, mert egy cgroup számára nem elég a teljes memóriamennyiséget korlátozni; ha a csoport folyamatai egy node-on futnak, de a memóriájuk más node-ról származik, a teljesítmény romlik, és a korlátok nem tükrözik a valós topológiai elhelyezkedést.

A memcg mellett a **cpuset** alrendszer felelős azért, hogy egy cgroup folyamatai mely CPU-kon és mely memória-node-okon helyezkedhetnek el. A cpuset segítségével megadhatjuk a `cpuset.cpus` és a `cpuset.mems` értékeket, így a cgroupba tartozó folyamatok csak a kijelölt node-okon futnak, és csak a kijelölt node-okról allokálhatnak. A memcg és a cpuset együttes használata teszi lehetővé, hogy egy konténer vagy szolgáltatás NUMA-affinitását pontosan meghatározzuk. Például egy nyolcmagos kétnode-os rendszeren az egyik cgroup a 0. node CPU-it és memóriáját, a másik az 1. node CPU-it és memóriáját kapja, így mindkét cgroup helyi hozzáféréssel dolgozik, és nem zavarja egymást.

A memóriapool és a memcg kapcsolata akkor válik láthatóvá, amikor egy alkalmazás saját poolt tart fenn, és a poolból származó lapokat a memcg-nek kell elszámolnia. A kernel a cgroup határain belül tartja nyilván a lefoglalt lapokat, függetlenül attól, hogy az alkalmazás hogyan használja fel őket. Ha a poolból kiosztott blokkokat nem szabadítjuk fel, a memcg statisztikája továbbra is mutatja a használatot, és a korlát elérésekor a kernel visszautasíthatja a további allokációkat. Ezért a poolok tervezésénél nemcsak a nyers allokációs sebességre, hanem a memória-elszámolási és felszabadítási fegyelemre is figyelni kell.

13.3.4 Gyakorlati minta: first-touch, affinitás és pool

A NUMA-tudatos felhasználói programozás egyik leggyakoribb hibája a **first-touch politika félreértése**. A Linux lapallokáció alapértelmezésben first-touch elven működik: amikor egy folyamat először ír egy virtuális lapot, a kernel ahhoz a node-hoz rendeli a fizikai lapot, amelyik node-on a hozzáférést végző processzor fut. Ha a szálak még nem kötődnek a kívánt node-hoz, akkor a lefoglalt lapok rossz node-ra kerülhetnek, és később hiába állítjuk be az affinitást, a memória távoli marad.

Vegyünk egy példát, amely megmutatja a first-touch hibát és a javítást. A következő C kód vázlatosan illusztrálja, hogyan kell a szálak CPU-affinitását a memória lefoglalása előtt beállítani. A kód szemléltetés, nem éles rendszeren végrehajtandó utasítás.

```
/* Illusztráció: NUMA first-touch elv és helyes sorrend.
 * A hibakezelést és a részletes beállításokat elhagytuk. */
#include <pthread.h>
#include <sched.h>
#include <stdlib.h>
#include <string.h>

#define NODE0_CPU 0
#define NODE1_CPU 4
#define ARRAY_SIZE (1UL << 20)  /* 1 Mi elem, példa */

static void set_cpu(int cpu)
{
    cpu_set_t set;
    CPU_ZERO(&set);
    CPU_SET(cpu, &set);
    sched_setaffinity(0, sizeof(set), &set);
}

static void *worker(void *arg)
{
    long node = (long)arg;
    size_t *buf;

    if (node == 0)
        set_cpu(NODE0_CPU);
    else
        set_cpu(NODE1_CPU);

    /* Helyes: először affinitás, utána érintés (first-touch). */
    buf = malloc(ARRAY_SIZE * sizeof(size_t));
    if (!buf)
        return NULL;

    for (size_t i = 0; i < ARRAY_SIZE; i++)
        buf[i] = (size_t)node;   /* first-touch a helyi node-on */

    /* Itt kezdődne a tényleges feldolgozás. */
    return NULL;
}

int main(void)
{
    pthread_t t0, t1;

    pthread_create(&t0, NULL, worker, (void *)0);
    pthread_create(&t1, NULL, worker, (void *)1);
    pthread_join(t0, NULL);
    pthread_join(t1, NULL);
```

```
    return 0;
}
```

A fenti példában a szálak az affinitásukat a `malloc()` és a tömb inicializálása előtt állítják be. Így a `buf[i] = ...` sorban a first-touch a megfelelő node-hoz köti a lapokat. Ha a sorrend fordított lenne, előfordulhatna, hogy a 0. node-on futó szál a tömböt az 1. node-ról kapta volna, mert a lefoglalást egy másik node-on futó szál végezte volna.

A felhasználói térben a NUMA-politika finomhangolható a `set_mempolicy()` és az `mbind()` rendszerhívásokkal. A `set_mempolicy()` a hívó folyamat alapértelmezett memóriapolicy-jét állítja be, az `mbind()` pedig konkrét címtartományra szabályozza a node-ok kiválasztását. A lehetséges policyk közé tartozik a `MPOL_BIND` (csak a megadott node-ok), a `MPOL_INTERLEAVE` (a lapokat körforgásban osztja el a node-ok között) és a `MPOL_PREFERRED` (elsősorban egy adott node, de szükség esetén más node is). A `libnuma` könyvtár kényelmesebb API-t kínál, például `numa_alloc_onnode()` és `numa_tonode_memory()` függvényekkel.

A memóriapoolok esetén a NUMA-affinitás úgy érvényesíthető, hogy minden node-hoz külön poolt rendelünk. A pool inicializálásakor a `numa_alloc_onnode()` segítségével a node helyi memóriájából foglalunk le egy nagy blokkot, majd ezt a blokkot osztjuk fel fix méretű elemekre. A szálak a saját node-juk pooljából allokálnak, így a kiosztott puffer fizikailag is a helyi memóriában van. A poolok közötti szinkronizáció elkerülhető, ha a pool per-thread vagy per-CPU struktúrában él, de legalább node-onkénti zárat használunk.

```
/* Illusztráció: node-onkénti memóriapool. A kód nem teljes,
 * csak a NUMA-tudatos elrendezés lényegét mutatja. */
#include <numa.h>
#include <stddef.h>

#define POOL_BLOCK_SIZE 64
#define POOL_BLOCK_COUNT 1024

struct pool_block {
    struct pool_block *next;
    unsigned char data[POOL_BLOCK_SIZE];
};

struct node_pool {
    struct pool_block *free_list;
    void *base;               /* a node memóriájából lefoglalt blokk */
    size_t block_count;
};

int pool_init(struct node_pool *pool, int node)
{
    if (numa_available() < 0)
        return -1;

    pool->base = numa_alloc_onnode(POOL_BLOCK_SIZE * POOL_BLOCK_COUNT,
                                   node);
    if (!pool->base)
        return -1;

    pool->block_count = POOL_BLOCK_COUNT;
    pool->free_list = NULL;

    /* A blokkokat láncoljuk a free_list-hez. */
    for (size_t i = 0; i < POOL_BLOCK_COUNT; i++) {
        struct pool_block *block =
            (struct pool_block *)((char *)pool->base +
                                  i * POOL_BLOCK_SIZE);
```

```
        block->next = pool->free_list;
        pool->free_list = block;
    }
    return 0;
}

void *pool_alloc(struct node_pool *pool)
{
    struct pool_block *block = pool->free_list;
    if (block) {
        pool->free_list = block->next;
        return block->data;
    }
    return NULL;  /* elfogyott, vissza kell esni más allokációra */
}

void pool_free(struct node_pool *pool, void *ptr)
{
    struct pool_block *block =
        (struct pool_block *)((char *)ptr -
                              offsetof(struct pool_block, data));
    block->next = pool->free_list;
    pool->free_list = block;
}
```

Ebben a vázlatban a `pool_init()` a `numa_alloc_onnode()` függvénnyel a megadott node memóriájából foglal le egyetlen nagy területet, majd azt fix méretű blokkokra bontja. A `pool_alloc()` és `pool_free()` lokálisan dolgozik, így a node-on futó szálak nem kényszerülnek távoli hozzáférésre.

A kernelben a per-node poolok természetesebbek, mert a page allocator és a slab/slub réteg már rendelkezik node-onkénti szabadlistákkal. A kernelkódban a `kmalloc_node()` és a `alloc_pages_node()` hívásokkal biztosítható, hogy a kiosztott struktúrák a kívánt node-hoz kerüljenek. A kernelben a memcg a cgroup szintjén számolja el ezeket a lapokat, így a node-tudatos allokáció nem kerüli meg a korlátokat.

13.3.5 Tipikus hibák és helyes megoldások

A NUMA-affinitási tervek során a leggyakoribb hibák a következők.

1. **A first-touch sorrend felcserélése.** Ha a szálak affinitását a memória inicializálása után állítjuk be, a lapok a rossz node-ra kerülnek. A helyes sorrend: először kössük a szálat a kívánt node CPU-jához, majd végezzük el az első érintést (például a tömb feltöltését nullákkal).

2. **Globális memóriapool használata NUMA-rendszeren.** Egyetlen globális pool minden szál számára lock-kontenációt és távoli hozzáféréseket okoz, mert a pool lapjai egyetlen node-on helyezkednek el. A helyes megoldás node-onkénti vagy per-CPU poolok létrehozása, és a szálak a saját node-juk poolját használják.

3. **A cpuset CPU- és memória-node halmazainak eltérése.** Ha a `cpuset.cpus` a 0. node CPU-it, a `cpuset.mems` viszont az 1. node memóriáját tartalmazza, a folyamat kényszerűen távoli memóriát használ. A helyes konfigurációban a két halmaznak meg kell egyeznie, vagy legalább átfedésben kell lennie a kívánt node-okon.

4. **Az interleave hiánya sávszélesség-érzékeny feladatoknál.** Ha egy alkalmazás egyszerre nagy sávszélességet igényel több node memóriájából, a kizárólagos helyi allokáció

egyetlen node memóriavezérlőjét terheli túl. Ilyenkor az `MPOL_INTERLEAVE` politika hasznos lehet, mert a lapokat körforgásban osztja el, és a sávszélesség összeadódhat.

5. **A NUMA-távolságok figyelmen kívül hagyása a kernelben.** A kernelkódban a `kmalloc()` helyett sokszor `kmalloc_node()` vagy `alloc_pages_node()` szükséges ahhoz, hogy egy adott node-hoz kötött struktúra a helyi memóriából jöjjön létre. Ha a fejlesztő mindig csak `kmalloc()`-ot használ, a struktúrák a hívó processzor node-jára kerülnek, ami többnyire helyes, de nem mindig; például egy eszközmeghajtó számára fenntartott puffer nem biztos, hogy a helyi node-hoz kell.

6. **A memcg statisztikák figyelmen kívül hagyása.** Ha egy cgroup memóriahasználata az egyik node-on kiugróan magas, de a folyamatok a másik node-on futnak, az gyakran rossz affinitásra vagy rossz pooltervezésre utal. A `memory.numa_stat` rendszeres ellenőrzése segít felismerni az ilyen anomáliákat.

A helyes megoldások tehát a következő elvekre épülnek:

- Az affinitást a memória lefoglalása és inicializálása előtt állítsuk be.
- Használjunk node-onkénti memóriapoolokat, és a szálak a lokális poolból dolgozzanak.
- A cpuset/cgroup konfigurációban a CPU- és memória-node halmazok összhangban legyenek.
- A kernelkódban a node-tudatos allokációs függvényeket válasszuk.
- A memcg node-onkénti statisztikáit folyamatosan kövessük, és a rendellenességeket korrigáljuk.

13.3.6 Labor: NUMA-affinitási terv készítése

A labor célja, hogy a hallgató egy emulált kétnode-os NUMA-rendszeren megtervezze és mérje a helyi, illetve távoli memóriahozzáférés hatását. A feladat nem igényel valódi többfoglalatos szervert; QEMU emulált környezetben is létrehozhatunk NUMA-topológiát.

Kiinduló állapot. A hoszt gép QEMU-val és a szükséges fejlesztőeszközökkel rendelkezik. Az emulált rendszer két NUMA node-ot tartalmaz: a 0. node a 0–3 CPU-kat és 1 GiB memóriát, az 1. node a 4–7 CPU-kat és 1 GiB memóriát. A rendszerindítás parancsa vázlatosan:

```
# Illusztráció: QEMU NUMA-topológiával, nem éles gépen futtatandó.
qemu-system-x86_64 \
  -smp 8 \
  -object memory-backend-ram,size=1G,id=mem0 \
  -object memory-backend-ram,size=1G,id=mem1 \
  -numa node,nodeid=0,cpus=0-3,memdev=mem0 \
  -numa node,nodeid=1,cpus=4-7,memdev=mem1 \
  -kernel /path/to/bzImage \
  -initrd /path/to/initramfs \
  -append "console=ttyS0" \
  -nographic
```

A fenti parancsban szereplő elérési utak helyőrzők. A pontos kernel- és initramfs-kép a labor környezetétől függ.

Mérőprogram. A hallgató ír egy C programot, amely lefoglal egy nagy tömböt, beállítja a szálak CPU-affinitását, és két fázisban méri az olvasási időt. Az első fázisban a tömböt a szálak a saját node-juk memóriájából foglalják le (`numa_alloc_onnode()`), és a szálakat a megfelelő node CPU-jaihoz kötik. A második fázisban a tömböt a másik node memóriájából foglalják le, miközben a szálak ugyanazokon a CPU-kon futnak. A méréshez `clock_gettime(CLOCK_MONOTONIC, ...)` használható, és a futási időt átlagolni kell több ismétlésen keresztül.

A mérési naplóban rögzíteni kell a hardverkonfigurációt (virtuális CPU típus, node-ok száma, memóriaméretek), a kernel verzióját, a QEMU verzióját, a fordítási beállításokat (`-O2`, `-march=...`), valamint a mérési módszert (hány iteráció, mekkora tömb, milyen hozzáférési minta). A mért és a várt eredményt külön kell feltüntetni.

Várt eredmény. A helyi hozzáférések gyorsabbak, mint a távoli hozzáférések. A konkrét arány a virtualizációs rétegtől és a hoszt erőforrásaitól függ, ezért a labor célja nem egy abszolút szám, hanem annak igazolása, hogy a két fázis között szignifikáns különbség mérhető, és hogy a különbség iránya megfelel a NUMA-elvnek.

Mért eredmény. A hallgató a futtatás után rögzíti a mért késleltetéseket vagy átviteli sebességeket, kiszámítja a local/remote arányt, és összehasonlítja a várt iránnyal. Ha a mért különbség nem mutatkozik, ellenőrizni kell a NUMA-topológiát, a first-touch sorrendet és az affinitás beállítását. A naplóban minden módosítást és annak hatását fel kell tüntetni.

Affinitási terv dokumentálása. A labor utolsó lépéseként a hallgató készít egy rövid tervet, amelyben meghatározza, hogyan osztaná el egy képzeletbeli nyolcmagos, kétnode-os rendszeren a szálakat és a puffereket egy adatbázis-szerű terhelés számára. A tervnek tartalmaznia kell:

- mely szálak mely node-okon futnak;
- a megosztott pufferek és poolok node-elhelyezkedését;
- a memóriapolicy beállításokat (`MPOL_BIND`, `MPOL_INTERLEAVE`, stb.);
- a memcg/cpuset konfigurációt, ha konténeres környezetben futna;
- a mérési tervet, amellyel a terv hatékonyságát ellenőrizni lehet.

A labor akkor tekinthető sikeresnek, ha a mérési napló reprodukálható, a mért és várt eredmények világosan elkülönülnek, és az affinitási terv indoklása a node-topológiára épít.

13.3.7 Összefoglalás és önellenőrzés

A többcsomópontos memória (NUMA) alapelve, hogy a memória-hozzáférési idő függ attól, melyik node-on fut a processzor, és melyik node-on található a memória. A helyi hozzáférés gyorsabb, a távoli hozzáférés lassabb, ezért a rendszer tervezésénél és üzemeltetésénél törekedni kell a processzorok és a memória együtt tartására. A NUMA node, a local/remote access és a memóriapool fogalmai szorosan összefüggenek: a node-onkénti poolok teszik lehetővé a helyi hozzáférést, a memcg pedig az erőforrás-elszámolást és korlátozást biztosítja. A first-touch politika, a CPU-affinitás és a memóriapolicy helyes használata elengedhetetlen a tipikus hibák elkerüléséhez.

Önellenőrző kérdés: Miért nem elegendő egy globális memóriapool egy kétnode-os rendszeren, ha a szálakat mindkét node-on futtatjuk? Magyarázza el, hogyan befolyásolja a first-touch elv a pool lapjainak elhelyezkedését, és milyen konkrét lépésekkel lehet a hibát kijavítani.

13.4 Kernelmemória, vmalloc, kmalloc és biztonság

A kernel saját memóriakezelése alapvetően különbözik a felhasználói tér allokációitól. A kernelnek egyszerre kell kiszolgálnia kis, gyakori vezérlőblokkokat, nagy, ritkán kért puffereket és speciális, fizikailag összefüggő területeket, miközben sosem feledkezhet meg a védelemről: egy rossz felszabadítás vagy egy lejárt pointer nem csak egy folyamatot, hanem az egész rendszert veszélyezteti. Ez a fejezet a kernelmemória rétegeit, az allokációs döntések szempontjait és a hardening technikákat járja körül.

13.4.1 1. A kernelmemória sajátos világa

A felhasználói program a `malloc` mögött általában nem látja a lapozást, a töredezettséget vagy a kernel laphibakezelését. A kernelben viszont az allokációk közvetlenül a fizikai memóriához, a lapkezelőhöz és a virtuális címtartományhoz kötődnek. A legfontosabb eltérések:

- **Nincs automatikus laphiba-kezelés minden kernelcímre.** A kernel saját adatai jellemzően nem lapozhatók ki; egy `NULL` pointer vagy egy felszabadított terület elérése gyakran `panic`-kel vagy azonnali leállással végződik.
- **A kernel többféle memóriát igényel.** Van, ami csak néhány bájt, van, ami több MiB, és van, ami fizikailag összefüggő kell legyen egy DMA-vezérlő számára.
- **A kernel kontextusfüggő.** Bizonyos kódok megszakításban vagy spinlock alatt futnak, ahol tilos aludni, ezért az allokációnak nem szabad lassú lapvisszaadást vagy ütemezést igényelnie.
- **A hibák hatása nagyobb.** A dupla felszabadítás, a use-after-free, a túlcsordulás vagy a rossz tulajdonos általi `kfree` az egész kernel integritását rombolhatja.

Ezek a tulajdonságok indokolják, hogy a kernelmemória-kezelés réteges legyen: a lapallokátor ad fizikai lapokat, a slab/slub réteg hatékony gyorsítótárat épít a kis objektumokhoz, a `kmalloc` általános célú bájtméret-alapú allokációt nyújt, a `vmalloc` pedig nagy, virtuálisan összefüggő területeket biztosít. Minden réteg más kompromisszumot jelent a sebesség, a töredezettség és a védelem között.

13.4.2 2. A page allocator: a kernel fizikaialap-allokációja

A kernel legalacsonyabb általános allokációs rétege a buddy allocator. Ez az előző fejezetben megismert elven fizikai lapokat ad vissza, ahol egy lap mérete általában 4 KiB x86-on, de architektúrától függően 16 KiB vagy 64 KiB is lehet. A kernelben a lapok `struct page` objektumokkal reprezentáltak, és az allokáció rendje (`order`) 2 hatványú lapszámot ad meg. Egy `order=0` kérés 1 lapot, egy `order=3` kérés 8 lapot, azaz 32 KiB-ot jelent 4 KiB-os lapok esetén.

A page allocator a kernel memóriatípusait zónákba és migrációs típusokba szervezi. Zónák például a `ZONE_DMA`, `ZONE_NORMAL` és `ZONE_HIGHMEM` (32 bites rendszereken), valamint a `ZONE_MOVABLE`. Az allokációs jelzők ezeket a követelményeket fejezik ki. A leggyakoribb jelzők a `GFP_KERNEL`, a `GFP_ATOMIC` és a `GFP_DMA`. A `GFP_KERNEL` normál kernelkódban használható, mert az allokáció aludhat; a `GFP_ATOMIC` megszakítás vagy spinlock alatt használható, de nem aludhat, ezért a rendszer a sürgősségi tartalékokhoz nyúlhat; a `GFP_DMA` pedig a DMA szempontjából elérhető memóriát kérheti.

Egy egyszerű lapallokáció:

```
#include <linux/gfp.h>
#include <linux/mm.h>

struct page *pg;
void *virt;
int order = 2; /* 4 lap, 4 KiB-os lap mellett 16 KiB */

pg = alloc_pages(GFP_KERNEL | __GFP_ZERO, order);
if (!pg)
    return -ENOMEM;

virt = page_address(pg); /* lineáris/közvetlen leképezés esetén */
if (!virt) {
    __free_pages(pg, order);
```

```
    return -ENOMEM;
}

/* ... használat ... */

__free_pages(pg, order);
```

A `page_address` csak akkor ad stabil virtuális címet, ha a lap a kernel közvetlen leképezésében van. Architektúra- és konfigurációfüggő, hogy minden lapra elérhető-e közvetlenül; a Linux kernel a `kmalloc` és a slab mögött is ezt a leképezést használja. A page allocator erőssége a pontos fizikai lapkezelés, a hátránya pedig az, hogy néhány bájtos kérést nem tud gazdaságosan kiszolgálni, és az `order` növelésével a fizikai töredezettség drámaian nőhet.

13.4.3 3. Slab, SLUB és a kmalloc

A kernel tele van kisméretű, gyakran létrehozott és felszabadított objektummal: inode-ok, dentry-k, task_struct-ok, net sk_buff-ok. A lapallokátor önmagában rossz választás lenne erre, mert minden egyes inode-hoz 4 KiB lapot rendelve óriási pazarlás keletkezne. A slab allocator ezt gyorsítótárakkal oldja meg: egy adott típusú objektumhoz előre lefoglal lapokat, azokat azonos méretű mezőkre osztja, és a felszabadított objektumokat gyors listában tartja.

A modern Linux kernelben a SLUB a leggyakoribb slab-változat, de a fogalom neve gyakran maradt „slab cache”. A SLUB nem tart fenn annyi metaadatot, mint a klasszikus slab, hanem a szabad objektumok listáját az objektumon belüli, egyébként üres területen tárolja. Ez kisebb overheadet és jobb cache-viselkedést eredményez.

A `kmalloc` egy általános célú API a bájtméret alapú allokációra. A kernel konfigurációjától függően egy vagy több slab cache-hez fordul, és a kért méretet felkerekíti a legközelebbi gyorsítótárméretre. A tipikus méretosztályok 8, 16, 32, 64, 96, 128, 192, 256, 512, 1024, 2048, 4096 bájt, és így tovább. A pontos osztályok kernelverziótól és konfigurációtól függenek, ezért a mérésnél érdemes a `/proc/slabinfo` tartalmát is rögzíteni.

A `kmalloc` legfontosabb tulajdonságai:

- **Fizikailag összefüggő** memóriát ad, amit a DMA-vezérlők is használhatnak, ha a megfelelő jelzőket alkalmazzuk.
- **Gyors**, mert a szabad objektumok listája állandóan rendelkezésre áll.
- **Méretkorlátos**. A gyakorlatban a `kmalloc` nagy méreteknél egyre töredezettebbé válik; a kernelverziótól függően a felső határ általában néhány MiB (például 4 MiB körül), de ezt a konkrét kernelverzióhoz ellenőrizni kell. Nagyobb méret esetén a page allocator vagy a `vmalloc` a helyes választás.

Példa `kmalloc`-ra:

```
#include <linux/slab.h>

struct sensor_sample {
    unsigned long timestamp_ns;
    int value;
    int flags;
};

struct sensor_sample *s;

s = kmalloc(sizeof(*s), GFP_KERNEL);
if (!s)
    return -ENOMEM;
```

```
s->timestamp_ns = ktime_get_ns();
s->value = 42;
s->flags = 0;

/* ... */

kfree(s);
```

A kernel API több kényelmi változatot is ad:

- `kzalloc(size, flags)` nulla töltésű allokáció.
- `kmalloc_array(n, size, flags)` túlcsordulás-ellenőrzött tömb allokáció; ha `n * size` túlcsordulna, `NULL`-t ad.
- `krealloc(p, new_size, flags)` meglévő terület átméretezése.

A `kzalloc` különösen fontos biztonsági szempontból, mert a nem inicializált kernelmemória adatszivárgást okozhat a felhasználói tér felé. A kernel ezért egyes konfigurációi alapból nullázzák a `GFP_KERNEL` allokációkat, de erre nem szabad építeni: a kód legyen explicit.

A slab cache-eket nem csak a `kmalloc` használja; a kernel alrendszer saját gyorsítótárat is létrehozhat, ha egy adott objektumtípusból nagyon sok példányt tart nyilván. Ez csökkenti a felkerekítési veszteséget, és lehetővé teszi a konstruktort vagy a speciális igazítást.

Példa saját cache-re:

```
#include <linux/slab.h>

struct fsm_event {
    unsigned long sequence;
    u64 payload[4];
};

static struct kmem_cache *fsm_event_cache;

int __init fsm_init(void)
{
    fsm_event_cache = kmem_cache_create(
        "fsm_event",
        sizeof(struct fsm_event),
        __alignof__(struct fsm_event),
        0,
        NULL);

    if (!fsm_event_cache)
        return -ENOMEM;

    return 0;
}

void fsm_event_free(struct fsm_event *e)
{
    kmem_cache_free(fsm_event_cache, e);
}

struct fsm_event *fsm_event_alloc(void)
{
    return kmem_cache_alloc(fsm_event_cache, GFP_KERNEL);
}

void __exit fsm_exit(void)
{
```

```
    kmem_cache_destroy(fsm_event_cache);
}
```

A `kmem_cache_create` a `0` jelzővel alapértelmezett viselkedést használ. A SLUB debug bekapcsolásakor a cache kaphat vörös zónát, mérgezést és más ellenőrzést, de ezt a kernel konfigurációja szabályozza.

13.4.4 4. A vmalloc: nagy, virtuálisan összefüggő terület

A `vmalloc` akkor hasznos, ha a kernelnek nagy, lineárisan címezhető memóriára van szüksége, de a fizikai lapoknak nem kell összefüggőnek lenniük. A `vmalloc` a kernel virtuális címtartományából egy összefüggő tartományt foglal le, majd a lapokat a page allocatorból egyenként vagy kisebb csoportokban szerzi be, és felépíti a laphozzárendelést.

A `vmalloc` jellemzői:

- **Virtuálisan összefüggő**, ezért egyszerű pointer-aritmetikával és memóriafüggvényekkel használható.
- **Fizikailag nem feltétlenül összefüggő**, ezért közvetlenül nem adható DMA-ra olyan vezérlőnek, amely fizikai lapcímeket vár.
- **Jelentősebb TLB-nyomás**, mert minden egyes lap bejegyzést igényel, és a leképezés felépítése drágább, mint a `kmalloc`.
- **Használható nagyobb méretekre**, ahol a `kmalloc` vagy a page allocator már töredezett vagy nem elérhető.

A tipikus `vmalloc` használat:

```
#include <linux/vmalloc.h>

void *buf;
const size_t size = 2 * 1024 * 1024; /* 2 MiB */

buf = vmalloc(size);
if (!buf)
    return -ENOMEM;

/* ... használat ... */

vfree(buf);
```

A `vmalloc` a kernel logikai címtartományán kívül, a `vmalloc` tartományban helyezkedik el. Ezért a `virt_to_phys` vagy a `virt_to_page` nem garantált, vagy csak a leképezés feloldásával működik. A `vmalloc` nem alkalmas olyan kódrészekhez, amelyek során physikai címet kell átadni egy eszköznek, hacsak nem használunk `vm_map_ram`-szerű megoldást vagy a lapok külön listáját.

A `vmalloc` és a `kmalloc` közötti választás elsősorban méret és fizikai folytonosság kérdése. Nagy, logikai pufferhez a `vmalloc`, kis, gyakori vezérlőblokkhoz a `kmalloc` vagy saját slab cache a megfelelő.

13.4.5 5. Döntési szempontok: mikor melyik allokátort?

A kernelben minden allokációs helyen egyértelmű tulajdonost és élettartamot kell rögzíteni. A következő szempontok segítenek:

Szempont	Javasolt választás	Indoklás
Néhány bájt–néhány KiB, gyakori alloc/free	`kmalloc` / `kzalloc` vagy saját slab cache	Gyors, fizikailag összefüggő, low overhead
Sok azonos típusú objektum	saját `kmem_cache_create`	Nincs méretkerekítési veszteség, debugolható
Nagy összefüggő logikai puffer, nem DMA	`vmalloc`	Virtuálisan összefüggő, nagyobb méretben is jó
Nagy, fizikailag összefüggő, DMA-képes	`alloc_pages` magasabb `order`-rel vagy `kmalloc` megfelelő jelzőkkel	DMA-vezérlő igényli a fizikai címtartományt
Atomi kontextus, nem alhat	`kmalloc`/`kzalloc` `GFP_ATOMIC`	Nem alszik, gyors
Belső metaadat, amit nem adunk ki user space-be	`kzalloc`	Elkerüli a nem inicializált adatok szivárgását

A `kmalloc` és a `vmalloc` közötti választásnál a gyakorlatban a méret a leggyakoribb indikátor. Ha a kért méret bizonytalan vagy felhasználói bemenettől függ, akkor a `kmalloc_array` és a méretkorlát ellenőrzése kötelező. A kernel nem engedheti meg, hogy egy felhasználói kérés óriási allokációt váltson ki, vagy hogy a méret számítása túlcsorduljon.

13.4.6 6. Tipikus hibák: use-after-free, double free és társaik

A kernelmemória hibái különösen veszélyesek, mert a támadó gyakran nem egy folyamatot, hanem a teljes rendszert fenyegeti. A három leggyakoribb hibacsalád:

13.4.7 6.1. Use-after-free

Use-after-free (UAF) történik, ha egy objektumot felszabadítunk, de egy másik kódrész még mindig hivatkozik rá, és a felszabadítás után olvas vagy ír. A SLUB gyorsítótár gyakran újra kiadja ugyanazt a memóriát egy másik allokációnak, így a lejárt pointer egyszer csak az új objektum mezőit módosítja. Ez adatsérülést, jogosultság-emelkedést vagy kernel `panic`-et okozhat.

Példa hibás kódvázlatra:

```
struct request *req = get_request();
/* ... */
kfree(req);

/* hiba: a kfree után is hivatkozunk a lejárt pointerre */
req->status = STATUS_DONE;
```

A helyes megoldás a pointer érvénytelenítése és a hivatkozások tulajdonosi követésének egyértelműsítése. Ha a pointert több kódrész használja, referencia-számlálást kell alkalmazni (például `kref`), vagy gondoskodni kell arról, hogy a felszabadítás előtt minden hivatkozás megszűnjön.

13.4.8 6.2. Double free

A double free ugyanannak a memóriaterületnek kétszeri felszabadítása. A SLUB szabad listájában így ugyanaz a slot kétszer jelenik meg, ami később ugyanannak a memóriának két különböző allokációhoz való kiadását okozhatja. A kernel ezt gyakran már a második `kfree` során észleli, ha a debugvédelem be van kapcsolva, de éles rendszerben a hiba hosszú ideig rejtve maradhat.

Példa:

```
void *p = kmalloc(64, GFP_KERNEL);
/* ... */
kfree(p);
/* ... */
kfree(p); /* double free */
```

A megoldás a tulajdonos-szabály pontos betartása: minden allokációhoz pontosan egy felszabadítási felelőst kell rendelni. A `kfree` után a pointert célszerű `NULL`-ra állítani, bár ez önmagában nem véd meg a másolatban tárolt pointerek ellen.

13.4.9 6.3. Egyéb hibák

- **Nulla méretű allokáció**: a `kmalloc(0, ...)` viselkedése nem megbízható; a hívó oldalán ellenőrizni kell.
- **Túlcsorduló méretszámítás**: `n * sizeof(...)` túlcsordulhat, ezért kell a `kmalloc_array`.
- **Rossz jelző atomi kontextusban**: ha spinlock alatt `GFP_KERNEL`-t használunk, a kernel aludhat, ami deadlock-hoz vezethet.
- **Fizikai cím elvárása `vmalloc`-nál**: DMA-hoz `vmalloc` területet átadni hibás, mert a fizikai lapok nem összefüggőek.
- **Memóriaszivárgás**: a lefoglalt terület nem kerül felszabadításra. A kernelben a szivárgás csak rendszerleálláskor tűnik el, és folyamatosan degradálja a teljesítményt.

13.4.10 7. Hardening: a kernelmemória védelme

A kernelmemória védelme három szinten értelmezhető: az allokátor belső ellenőrzése, a fordító- és konfigurációs védelem, valamint a futtatókörnyezeti izoláció. A modern Linux kernelben ezek a technikák együttesen csökkentik a kihasználhatóságot.

13.4.11 7.1. Allokátor-szintű védelem

A SLUB debug módja több ellenőrzést tesz lehetővé:

- **Red zone**: az objektum köré írt őrfal, amely a túlcsordulást a felszabadításkor észleli.
- **Poisoning**: a felszabadított memóriát egy ismert mintával tölti fel, hogy a use-after-free hamarabb láthatóvá váljon.
- **Freelist randomizáció**: a szabad listát véletlenszerű sorrendbe szervezi, hogy a támadó ne tudja megjósolni, melyik objektumot kapja meg legközelebb.
- **Sanity check**: a `kfree` ellenőrzi, hogy a pointer egy ismert slab-hez tartozik-e.

Ezek a `CONFIG_DEBUG_SLAB`, `CONFIG_SLUB_DEBUG` és hasonló beállításokkal kapcsolhatók. Fejlesztői és tesztelői kernelekben gyakran be vannak kapcsolva, éles rendszeren azonban teljesítményköltségük miatt általában korlátozottak.

13.4.12 7.2. Memória-hardening a konfigurációban

A kernel számos olyan védelmet ismer, amely a memóriakezelés biztonságát növeli:

- **KASLR**: a kernel betöltési címének véletlenszerűsítése megnehezíti a támadók számára a célcímek meghatározását.
- **SMEP/SMAP**: a felhasználói kód futtatását vagy olvasását tilthatja kernel módban, ami a memória-hibák kihasználását nehezíti.
- **W^X (write xor execute)**: a kernel szöveges része nem írható, az adatterületek nem futtathatók.
- **Stack protector**: a kernel verem a függvényhívásoknál kanárit kap, amely a veremtúlcsordulást a visszatérés előtt észleli.
- **Structleak**: a struktúrák paddingjait és a nem inicializált mezőket a kernel nullázza, így azok nem szivárognak user space-be.

Ezek a védelmek nem helyettesítik a helyes kódot, de jelentősen csökkentik a hibák kihasználhatóságát.

13.4.13 7.3. KFENCE és KASAN

A kernel speciális fejlesztőeszközöket is biztosít:

- **KFENCE** egy kis overheaddel járó, véletlenszerű mintavételű UAF- és túlcsordulásdetektor. A memória egyes allokációit védett „kerítés” lapok közé helyezi, és ha a kód átlépi a határt, azonnali hibát vált ki.
- **KASAN** a kernel AddressSanitizer megfelelője, amely pontos UAF- és out-of-bounds detektálást ad, de nagy memória- és CPU-költséggel.

Fejlesztés közben a `CONFIG_KFENCE` és a `CONFIG_KASAN` bekapcsolása sokkal hamarabb felszínre hozza a hibákat, mint a hagyományos tesztelés.

13.4.14 7.4. Referencia-számlálás és típusbiztonság

A kernel a `kref` és a `refcount_t` típusokkal védi a referencia-számlálót túlcsordulás és underflow ellen. A `refcount_t` nem engedi, hogy a számláló nulláról tovább csökkenjen, és a túlcsordulást is jelzi. Az UAF tipikus kiváltó oka a hibás referencia-számlálás, ezért a `kref_put` és a `refcount_dec_and_test` használata a helyes minta.

A `kmalloc` és a `kfree` típusbiztonsága C-ben nem automatikus, de a kód áttekintésekor mindig ellenőrizni kell, hogy a felszabadított pointer valóban a lefoglalt típusnak felel meg. A `kmem_cache_free` egy cache-hez kötött, ezért kevésbé hajlamos a típuskeveredésre, mint a nyers `kfree`.

13.4.15 8. Labor: kernelallokációs döntési táblázat

A labor célja, hogy az allokációs döntéseket szisztematikusan, reprodukálható módon rögzítsük, és egy emulált kernelmodullal megmérjük a különböző API-k alapköltségét. A feladat nem éles kernel módosítása: QEMU-ban, eldobható virtuális gépen fut.

13.4.16 8.1. Környezet és konfiguráció

A reprodukálhatóság érdekében a következő paramétereket rögzíteni kell a jegyzőkönyvben:

- QEMU verzió: `<mérendő>`
- kernel verzió: `<mérendő, például 6.12 LTS, ellenőrizendő>`
- CPU: `<mérendő, például QEMU x86_64, 4 vCPU>`
- RAM: `<mérendő, például 2 GiB>`
- kernel konfiguráció: `CONFIG_SLUB_DEBUG`, `CONFIG_KFENCE`, `CONFIG_KASAN` állapota `<mérendő>`
- mérési módszer: `local_clock()` vagy `ktime_get_ns()` a modulban, kívülről `perf` vagy `trace-cmd`

A mérési jegyzőkönyvben a „mért” oszlopba csak a ténylegesen lefuttatott, naplózott értékek kerülhetnek; a „várt” oszlop a tervezési modellből származó becslés. A kettőt minden táblázatban külön kell jelölni.

13.4.17 8.2. A döntési táblázat felépítése

Készítsen táblázatot, amely legalább a következő oszlopokat tartalmazza:

1. **Használati eset** – rövid leírás.
2. **Mérettartomány** – bájtban.
3. **Kontextus** – alhat-e az allokáció (normál / atomi).
4. **DMA-szükséglet** – kell-e fizikai összefüggőség.
5. **Élettartam** – hosszú / rövid / azonos objektumtípus sok példánya.
6. **Javasolt API** – `kmalloc`, `kzalloc`, `kmem_cache`, `vmalloc`, `alloc_pages`.
7. **Jelzők** – `GFP_KERNEL`, `GFP_ATOMIC`, `GFP_DMA` stb.
8. **Indoklás** – a választás rövid magyarázata.

Példa a kitöltött táblázatra (illusztráció, nem mért eredmény):

Használati eset	Méret	Kontextus	DMA	Élettartam	Javasolt API	Jelzők	Indoklás
Hálózati csomag fejléce	64 B	normál	nem	rövid	`kmalloc`	`GFP_KERNEL`	Gyors, fizikailag összefüggő, kis méret
FSM-állapot esemény	48 B	atomi	nem	rövid	saját `kmem_cache`	`GFP_ATOMIC`	Sok példány, nincs kerekítési veszteség, atomi kontextus
IDE DMA puffer	64 KiB	normál	igen	rövid	`kmalloc` / `alloc_pages`	`GFP_KERNEL`	DMA-hoz fizikai összefüggőség kell

Használati eset	Méret	Kontextus	DMA	Élettartam	Javasolt API	Jelzők	Indoklás
Titkosítási munkapuffer	2 MiB	normál	nem	hosszú	`vmalloc`	`GFP_KERNEL`	Nagy méret, nem kell fizikai összefüggőség
Topológiai leíró	12 B	normál	nem	hosszú	`kzalloc`	`GFP_KERNEL`	Nulla init user space-be szivárgás ellen

A táblázatot a saját modul igényeihez kell igazítani, és minden sort a forráskódban meg kell tudni feleltetni.

13.4.18 8.3. Mérési modul

Írjon egy egyszerű kernelmodult, amely minden API-t azonos méretű objektumok ciklikus allokálásával és felszabadításával mér. A modul ne férjen hozzá a hoszt valódi lemezéhez; csak emulált QEMU-ban fut.

Példa a mérési vázlatra:

```
#include <linux/module.h>
#include <linux/slab.h>
#include <linux/vmalloc.h>
#include <linux/timekeeping.h>
#include <linux/random.h>

#define ALLOC_COUNT 10000
#define ADJUST      0x1000

static noinline void measure_kmalloc(void)
{
    void *p;
    u64 start, end;
    int i;
    u8 *ptr;

    start = ktime_get_ns();
    for (i = 0; i < ALLOC_COUNT; i++) {
        ptr = kmalloc(64, GFP_KERNEL);
        if (!ptr)
            return;
        ptr[0] = 0x5a;
        kfree(ptr);
    }
    end = ktime_get_ns();
    pr_info("kmalloc64: %llu ns, avg %llu ns\n",
            end - start, (end - start) / ALLOC_COUNT);
}

static noinline void measure_vmalloc(void)
{
```

```
    void *p;
    u64 start, end;
    int i;

    start = ktime_get_ns();
    for (i = 0; i < ALLOC_COUNT; i++) {
        p = vmalloc(64);
        if (!p)
            return;
        vfree(p);
    }
    end = ktime_get_ns();
    pr_info("vmalloc64: %llu ns, avg %llu ns\n",
            end - start, (end - start) / ALLOC_COUNT);
}

/* ... további mérések: kmalloc-4k, vmalloc-4k, slab cache ... */
```

A mérésnél a CPU órajelét és a cachelépítményt is rögzíteni kell. A `ktime_get_ns` a monoton rendszeridőt adja, de a tényleges mérési pontosság a `CONFIG_HZ` és az időzítő felbontásától függ. Emiatt a ciklus hosszát érdemes úgy megválasztani, hogy a mért idő legalább néhány milliszekundum legyen.

13.4.19 8.4. Eredmények és kiértékelés

A mérési jegyzőkönyvben a következőket kell elkülöníteni:

- **Várt eredmény**: a `kmalloc` gyors, mert a slab gyorsítótárból veszi az objektumot; a `vmalloc` lassabb, mert laphozzárendelést épít, és TLB-flush is előfordulhat; a saját slab cache közel áll a `kmalloc`-hoz, de kisebb a kerekítési vesztesége.
- **Mért eredmény**: `<kitöltendő>` – a tényleges futtatásból származó értékek.

A mérési naplóhoz csatolni kell a `dmesg` kivonatát és a QEMU indítási parancsát, például:

```
qemu-system-x86_64 \
  -kernel arch/x86/boot/bzImage \
  -append "console=ttyS0 root=/dev/sda rw" \
  -drive file=lab-rootfs.qcow2,format=qcow2 \
  -smp 4 -m 2048 \
  -nographic
```

Visszaállítási út: a labor a hoszt lemezét nem érinti; a QEMU lemezkép eldobható, a labor után a snapshot törölhető.

13.4.20 8.5. Önellenőrző kérdések

1. Miért nem lehet `vmalloc` területet közvetlenül átadni egy DMA-vezérlőnek?
2. Milyen `kmalloc` jelzőt használjunk spinlock alatt, és miért?
3. Hogyan észleli a SLUB debug a use-after-free hibát?
4. Mi a különbség a `kmalloc` és a saját `kmem_cache_create` között?
5. Miért kell a `kfree` után a pointert `NULL`-ra állítani, és miért nem elégséges ez a védelem?

13.4.21 8.6. Összefoglalás

A kernelmemória-kezelés rétegei közül a lapallokátor a fizikai lapok forrása, a slab/SLUB a kis objektumok gyors gyorsítótára, a `kmalloc` az általános célú, fizikailag összefüggő allokáció, a `vmalloc` pedig a nagy, virtuálisan összefüggő területek eszköze. A döntés mindig a méret, a kontextus, a DMA-igény és az élettartam függvénye. A kernelmemória biztonsága a helyes tulajdonosi fegyelmen és a hardening technikák együttesén múlik. A labor célja, hogy a döntéseket ne megérzésből, hanem mérésekkel és dokumentált táblázattal hozzuk meg.

14 I/O és eszközkezelés

14.1 Blokk- és karakteres eszközök

Az operációs rendszer egyik alapvető feladata, hogy a különféle fizikai eszközöket – lemezek, soros portok, hangkártyák, hálózati adapterek, szenzorok – egységes, a folyamatok számára jól kezelhető absztrakció mögé rejtse. Ebben az alfejezetben az eszközök két nagy osztályát, a **blokk-** és **karakteres eszközöket** vizsgáljuk meg. Nem az a cél, hogy egy adott kernel eszközmeghajtó-keretrendszerét bemutassuk, hanem az, hogy megértsük az osztályozás elvét, az I/O-modell rétegeit, valamint a felhasználói tér és a meghajtó közötti kapcsolatot a **device file**-on és az **ioctl** vezérlőcsatornán keresztül.

A fejezet tanulási céljai: megmagyarázni az eszközök osztályozását és az I/O-modellt; alkalmazni az itt szerzett ismereteket egy kis, ellenőrizhető példán; felismerni a tipikus hibákat, és megindokolni a helyes megoldást. A következő alfejezet (Interruptok, DMA és pollozás) erre építve tárgyalja a megszakításokat, a DMA-t és a pollozást.

14.1.1 1. Miért van szükség osztályozásra?

Az operációs rendszer eszközkezelő alrendszere két, alapvetően eltérő hozzáférési mintát lát a hardverben. Vannak olyan eszközök, amelyekhez **véletlen hozzáféréssel**, rögzített méretű egységekben lehet írni és olvasni; ilyenek a merevlemezek, SSD-k, NVMe-meghajtók és a RAM-diszkek. Más eszközök **bájtfolyam-szerűen** viselkednek: az adatok sorban érkeznek, nincs értelme „hátraugrani”, a művelet egysége a bájt vagy egy kis puffer. Ilyen a soros port, a terminál, a billentyűzet, a hangkártya és a `/dev/null`.

A két osztály eltérő belső kezelést igényel. A blokkos hozzáférésnél a kernel **blokkréteget**, **page cache**-t és gyakran **I/O-ütemezőt** használhat; a karakteres eszközök esetében viszont a driver gyakran közvetlenül kapja meg a hívásokat, pufferelés nélkül. Az osztályozás tehát nem csupán esztétikai kérdés, hanem a teljesítmény és a szemantika alapja.

14.1.2 2. Blokkeszközök: szektor, blokk és véletlen hozzáférés

A **blokkeszköz** olyan eszköz, amely az adatokat rögzített méretű, címezhető egységekben tárolja, és a kernel ezekhez az egységekhez tetszőleges sorrendben hozzáférhet. A hardver **szektor**nak nevezi a legkisebb fizikailag címezhető egységet; a klasszikus HDD-knél ez 512 bájt, az újabb lemezeknél 4096 bájt (4 KiB), az SSD-knél pedig a **lap** (page) mérete általában 4 vagy 8 KiB, a törlési egység (block) pedig ennél nagyobb. A kernel ezt a fizikai tulajdonságot egy **logikai blokkméret** mögé rejtheti, amely tipikusan 512 bájt vagy 4096 bájt.

A blokkeszközök legfontosabb tulajdonságai:

- **Véletlen hozzáférés**: a kért blokk azonosítója alapján a driver közvetlenül eléri a kívánt pozíciót. A `lseek` rendszerhívásnak itt van értelme.

- **Rögzített blokkméret**: minden I/O-művelet egysége a blokk. A kernel a kéréseket a logikai blokkméret többszörösére igazítja, vagy hibát ad, ha a puffer mérete nem megfelelő.
- **Pufferelés és gyorsítótár**: a kernel gyakran a **page cache**-en keresztül olvas és ír, ami lehetővé teszi az olvasás-előrehozást, az írások késleltetését és a lapok újrahasznosítását.
- **Tipikus példák**: HDD, SSD, NVMe, USB-meghajtó, RAM-diszk, optikai lemez, memóriakártya.

A blokkeszközökhöz a kernelben egy **blokkeszköz-meghajtó** tartozik, amely a kéréseket fogadja, és a hardver felé továbbítja. A modern kernelekben (például Linux) a blokkeszközök a **blk-mq** többvárólistás keretrendszeren keresztül kapják a kéréseket; a mi oktatói kernelünkben ennél egyszerűbb interfészt használunk, de a szemantika ugyanaz.

A blokkeszközök felhasználói térbeli megjelenése általában egy **partíció** vagy egy **eszközfájl**, például `/dev/sda`, `/dev/nvme0n1p1`. A fájlrendszerek ezen a nyers blokkeszközön hozzák létre a saját struktúrájukat; a kernel blokkrétege tehát a fájlrendszer alatt helyezkedik el.

14.1.3 3. Karakteres eszközök: bájtfolyam és szekvenciális hozzáférés

A **karakteres eszköz** olyan eszköz, amelyhez a kernel bájtonként vagy kis pufferekben, szekvenciálisan fér hozzá. Nincs rögzített blokkméret, nincs véletlen hozzáférés, és a `lseek` rendszerint nem értelmezhető. Az adatfolyam iránya lehet bemeneti (input), kimeneti (output) vagy mindkettő.

Jellemző tulajdonságok:

- **Szekvenciális stream**: az adatok a küldés sorrendjében érkeznek; a „pozíció” fogalma legfeljebb a folyamon belüli aktuális helyet jelenti, nem egy címezhető offsetet.
- **Változó méretű műveletek**: a `read` és `write` hívások mérete tetszőleges lehet, a driver határozza meg, hogy mennyit fogad el. Egyes karakteres eszközök csak egy bájtot, mások nagyobb puffereket is kezelnek.
- **Közvetlen meghajtóhívás**: a karakteres eszközök általában nem mennek át a page cache-en; ha a driver nem pufferel saját maga, a hívás közvetlenül a hardverhez vagy a belső várakozási sorhoz kerül.
- **Tipikus példák**: soros port (`/dev/ttyS0`), terminál (`/dev/tty`), billentyűzet, egér, hangkártya, `/dev/null`, `/dev/zero`, `/dev/random`, `/dev/watchdog`.

A karakteres eszközök felhasználói térbeli megjelenése szintén **device file**, például `/dev/tty` vagy `/dev/null`. A kernelben a karakteres eszközöket általában egy **karakteres eszközmeghajtó** regisztrálja, és a műveleteket közvetlenül a `read`, `write`, `ioctl`, `mmap` stb. hívásokra képezi le.

14.1.4 4. Összehasonlító áttekintés

Jellemző	Blokkeszköz	Karakteres eszköz
Hozzáférés	véletlen, blokkonként	szekvenciális, bájtonként
Címzés	blokkazonosító (offset / blokkméret)	nincs értelmezhető offset
Pufferelés	page cache, I/O-ütemező	közvetlen vagy driver-szintű
Műveleti egység	rögzített blokk (pl. 512/4096 bájt)	bájt vagy tetszőleges puffer
`lseek`	támogatott	általában `ESPIPE` hibát ad
Példa	`/dev/sda`, `/dev/nvme0n1`	`/dev/tty`, `/dev/null`

A táblázatban szereplő `ESPIPE` azt jelenti: „illegal seek” – a karakteres eszközön végrehajtott `lseek` hívás ezt a hibakódot adja vissza, és az `errno`-ba kerül. Ez is mutatja, hogy a két osztály közötti különbség a rendszerhívások szintjén is megjelenik.

14.1.5 5. Az I/O-modell rétegei

Az operációs rendszer I/O-modellje többrétegű. Az alábbi rétegződést érdemes megjegyezni:

1. **Felhasználói folyamat**: a program a POSIX vagy natív API-n keresztül hívja a `read`, `write`, `ioctl`, `mmap` rendszerhívásokat.
2. **Rendszerhívás-réteg**: a kernel ellenőrzi a paramétereket, a fájldeszkriptort, a jogosultságokat, majd továbbadja a kérést a megfelelő eszközmeghajtónak.
3. **Eszközabsztrakciós réteg**: a kernel az eszköz típusa alapján választja ki a megfelelő művelettáblát. Blokkeszköznél ez a blokkréteg, karakteres eszköznél közvetlenül a karakteres driver.
4. **Meghajtó (driver)**: a hardver specifikus műveleteket hajtja végre, például regisztrációt ír, DMA-t indít, megszakítást kezel.
5. **Hardver**: a tényleges eszköz.

A `device file` ebben a modellben a kapcsolódási pont a felhasználói tér és a kernel között. Amikor egy folyamat megnyit egy device file-t, a kernel a **major szám** alapján kiválasztja a megfelelő meghajtót, a **minor szám** pedig az adott meghajtón belüli konkrét eszközpéldányt azonosítja. A nyitás után a fájldeszkriptoron végrehajtott műveletek közvetlenül a driverhez kerülnek.

14.1.6 6. Device file, major és minor szám

A **device file** (eszközfájl) nem hagyományos fájl: nincs adattartalma a lemezen, hanem egy speciális bejegyzés a fájlrendszerben, amely az eszköz típusát (`c` karakteres, `b` blokk), a **major** és a **minor** számot tárolja. A major szám a meghajtót, a minor szám pedig az adott meghajtón belüli eszközt vagy partíciót azonosítja.

Például a Linux alatt:

```
$ ls -l /dev/null /dev/ttyS0 /dev/sda
crw-rw-rw- 1 root root 1, 3 Jan  1 00:00 /dev/null
crw-rw---- 1 root dialout 4, 64 Jan  1 00:00 /dev/ttyS0
brw-rw---- 1 root disk 8, 0 Jan  1 00:00 /dev/sda
```

A `/dev/null` karakteres eszköz (`c`), major `1`, minor `3`. A `/dev/sda` blokkeszköz (`b`), major `8`, minor `0`. A major/minor páros a kernelben egy eszközregisztrációhoz kötődik.

A device file létrehozásának hagyományos módja a `mknod` parancs:

```
$ mknod /dev/labchar c 240 0
$ mknod /dev/labblk b 241 0
```

A modern rendszerekben ezt gyakran a **devtmpfs** vagy az **udev** automatikusan végzi. A kernel meghajtójának regisztrálnia kell magát a megfelelő major számon. A mi oktatói kernelünkben ez a regisztráció egy egyszerű API-val történik, amelyet a következő pontban mutatunk be.

14.1.7 6.1. Eszközmeghajtó interfész vázlata

Az általános eszközmeghajtó-mintát a következő C struktúra szemlélteti (oktatói keret, nem végrehajtandó kód):

```
/* Oktatói kernel: közös eszközműveletek */
struct device_ops {
    int (*open)(struct device *dev, int flags);
    ssize_t (*read)(struct device *dev, char *buf, size_t count, off_t pos);
    ssize_t (*write)(struct device *dev, const char *buf, size_t count, off_t pos);
    int (*ioctl)(struct device *dev, unsigned int req, void *arg);
    int (*close)(struct device *dev);
};

struct device {
    const char *name;
    int major;
    int minor;
    int type;                   /* DEV_CHAR vagy DEV_BLOCK */
    struct device_ops ops;
    /* eszköz-specifikus adatok */
    void *priv;
};
```

A `read` és `write` hívások a `pos` paramétert csak blokkeszközöknél értelmezik; karakteres eszközöknél ez általában figyelmen kívül marad. A `ioctl` mindkét típusnál a vezérlőcsatornát biztosítja.

14.1.8 7. Device file műveletek: open, read, write, close

A felhasználói program a device file-t ugyanúgy nyitja meg, mint egy hagyományos fájlt, de a kernel a major szám alapján a megfelelő meghajtó `open` műveletét hívja. A leggyakoribb rendszerhívások:

- **`open(path, flags, mode)`** → fájldeszkriptor, hiba esetén `-1` és `errno` (pl. `ENOENT`, `EACCES`, `ENXIO` ha nincs ilyen eszköz).
- **`read(fd, buf, count)`** → beolvasott bájtok száma (`ssize_t`), hiba esetén `-1` (pl. `EBADF`, `EFAULT`, `EINTR`, `EAGAIN` ha nem blokkoló és nincs adat).
- **`write(fd, buf, count)`** → kiírt bájtok száma, hiba esetén `-1` (pl. `EBADF`, `EFAULT`, `EINTR`, `ENOSPC`).
- **`close(fd)`** → `0` siker, `-1` hiba (`EBADF`).
- **`lseek(fd, offset, whence)`** → új pozíció (`off_t`), hiba `ESPIPE` karakteres eszközön, `EINVAL` érvénytelen whence.

Blokkeszközöknél a `read` és `write` pufferének méretét a blokkmérethez kell igazítani. Ha a puffer mérete nem többszöröse a logikai blokkméretnek, a kernel vagy csonkolja a kérést, vagy `EINVAL` hibát ad. A pontos viselkedés az operációs rendszertől függ; a mi oktatói kernelünkben a `read`/`write` a blokkméretre kerekít, és a fennmaradó részt eldobja, de ezt a laborban ellenőrizni lehet.

14.1.9 8. ioctl: a vezérlőcsatorna

A `read` és `write` csak adatfolyamokhoz elegendő. Sok eszköznek azonban olyan műveletekre van szüksége, amelyek nem illeszkednek a fájlmodellbe: például a soros port sebességének beállítása,

a lemez geometriájának lekérdezése, a watchdog felélesztése, vagy egy LED állapotának váltása. Erre szolgál az **ioctl** (input/output control) rendszerhívás.

14.1.10 8.1. Az ioctl rendszerhívás

```
int ioctl(int fd, unsigned long request, ...);
```

- `fd`: a nyitott eszköz fájldeszkriptora.
- `request`: egy eszközspecifikus parancskód.
- `...`: opcionális harmadik argumentum, általában egy mutató egy struktúrára vagy egy skalár érték.
- Visszatérés: `0` siker, `-1` hiba, `errno` (pl. `EBADF`, `ENOTTY` ha a parancskód nem támogatott, `EINVAL` érvénytelen argumentum, `EFAULT` hibás user pointer).

A `request` kódolásának fontos szerepe van: a parancskódnak tartalmaznia kell az irányt (be, ki, mindkettő), a paraméter méretét, valamint egyedi típus- és sorszámmezőt, hogy elkerüljük az ütközéseket.

Az oktatói keret a Linux-szerű makrókat használja:

```
/* ioctl parancskód felépítése (oktatói keret) */
#define _IOC_NRBITS   8
#define _IOC_TYPEBITS  8
#define _IOC_SIZEBITS 14
#define _IOC_DIRBITS   2

#define _IOC_NRSHIFT   0
#define _IOC_TYPESHIFT (_IOC_NRSHIFT + _IOC_NRBITS)
#define _IOC_SIZESHIFT (_IOC_TYPESHIFT + _IOC_TYPEBITS)
#define _IOC_DIRSHIFT  (_IOC_SIZESHIFT + _IOC_SIZEBITS)

#define _IOC_NONE      0U
#define _IOC_WRITE     1U
#define _IOC_READ      2U

#define _IOC(dir,type,nr,size) \
    (((dir)  << _IOC_DIRSHIFT) | \
     ((type) << _IOC_TYPESHIFT) | \
     ((nr)   << _IOC_NRSHIFT) | \
     ((size) << _IOC_SIZESHIFT))

#define _IO(type,nr)      _IOC(_IOC_NONE,(type),(nr),0)
#define _IOR(type,nr,size) _IOC(_IOC_READ,(type),(nr),sizeof(size))
#define _IOW(type,nr,size) _IOC(_IOC_WRITE,(type),(nr),sizeof(size))
#define _IOWR(type,nr,size) _IOC(_IOC_READ|_IOC_WRITE,(type),(nr),sizeof(size))
```

Például egy egyszerű karakteres eszközhöz definiálhatunk parancsokat:

```
#define LAB_IOCTL_TYPE 0xA4
#define LAB_GET_VERSION _IOR(LAB_IOCTL_TYPE, 1, int)
#define LAB_SET_MODE    _IOW(LAB_IOCTL_TYPE, 2, int)
#define LAB_CLEAR       _IO(LAB_IOCTL_TYPE, 3)
```

A driver `ioctl` művelete dekódolja a `request` paramétert, ellenőrzi az irányt és a méretet, majd végrehajtja a kért műveletet. A `copy_from_user` / `copy_to_user` segítségével kell a felhasználói térből adatot másolni; ez a mi oktatói kernelünkben `copyin` és `copyout` néven szerepel.

14.1.11 8.2. Biztonsági megfontolások az ioctl használatakor

Az ioctl gyakori hibalehetőségei közé tartozik:

- **Hiányzó parancskód-ellenőrzés**: ha a driver nem ellenőrzi, hogy a `request` valóban az ő eszközéhez tartozik-e, idegen parancs is bejuthat. A megoldás a típusmező (`type`) és a sorszám (`nr`) ellenőrzése, valamint az alapértelmezett `ENOTTY` visszatérés.
- **User pointer ellenőrzése**: az ioctl harmadik argumentuma gyakran mutató. A kernelnek mindig ellenőriznie kell, hogy a pointer a felhasználói térbe mutat-e, és a mérete nem haladja-e meg a struktúrát. A `copyin`/`copyout` ezt elvégzi, de a manuális dereferálás tilos.
- **Parancskód-ütközés**: két különböző driver ne használjon azonos típuskódot ugyanazzal a sorszámmal. A típuskódot ezért regisztrálni kell, vagy a projektben központilag kell kezelni.
- **Jogosultságok hiánya**: bizonyos ioctl parancsok (pl. formázás, alacsony szintű elérés) csak `CAP_SYS_ADMIN` vagy hasonló képességgel rendelkező folyamatoknak engedélyezettek. A drivernek ezt is ellenőriznie kell.

14.1.12 9. Gyakorlati minta: egyszerű karakteres és blokkeszköz

Az alábbi példa egy oktatói kernelben bemutatja, hogyan tervezzünk meg egy karakteres és egy blokkeszköz interfészét. A kód nem futtatható éles rendszeren, csak szemlélteti a fogalmakat.

14.1.13 9.1. Karakteres eszköz: naplózó gyűrűpuffer

Tervezzünk egy `lablog` nevű karakteres eszközt, amely egy rögzített méretű gyűrűpufferbe írható, és onnan olvasható. A puffer mérete 4096 bájt, a műveletek szekvenciálisak.

```
/* lablog karakteres eszköz - oktatói vázlat */
#include <kern/device.h>
#include <kern/copy.h>

#define LABLOG_SIZE 4096

struct lablog_dev {
    char buf[LABLOG_SIZE];
    size_t head;   /* írási pozíció */
    size_t tail;   /* olvasási pozíció */
    size_t count;  /* aktuális adatmennyiség */
    struct device dev;
};

static ssize_t lablog_read(struct device *dev, char *user_buf,
                           size_t count, off_t pos)
{
    struct lablog_dev *log = container_of(dev, struct lablog_dev, dev);
    size_t n = 0;
    (void)pos; /* karakteres eszköznél nincs offset */

    while (n < count && log->count > 0) {
        char c = log->buf[log->tail];
        if (copyout(&user_buf[n], &c, 1) != 0)
            return -EFAULT;
        log->tail = (log->tail + 1) % LABLOG_SIZE;
        log->count--;
        n++;
    }
    return n;
}
```

```
static ssize_t lablog_write(struct device *dev, const char *user_buf,
                            size_t count, off_t pos)
{
    struct lablog_dev *log = container_of(dev, struct lablog_dev, dev);
    size_t n = 0;
    (void)pos;

    while (n < count) {
        char c;
        if (copyin(&c, &user_buf[n], 1) != 0)
            return -EFAULT;
        if (log->count == LABLOG_SIZE)
            break; /* puffer tele */
        log->buf[log->head] = c;
        log->head = (log->head + 1) % LABLOG_SIZE;
        log->count++;
        n++;
    }
    return n;
}

static int lablog_ioctl(struct device *dev, unsigned int req, void *arg)
{
    struct lablog_dev *log = container_of(dev, struct lablog_dev, dev);

    switch (req) {
    case LAB_CLEAR:
        log->head = log->tail = log->count = 0;
        return 0;
    case LAB_GET_SIZE:
        return copyout(arg, &(int){ LOG_SIZE }, sizeof(int));
    default:
        return -ENOTTY;
    }
}
```

A `copyout` és `copyin` a felhasználói térbe/térből való biztonságos másolást jelenti. A kód nem használ közvetlen pointer-dereferálást a felhasználói címekre; ez a tipikus hiba elkerülésének egyik fontos eleme.

14.1.14 9.2. Blokkeszköz: RAM-diszk

Tervezzünk egy `labblk` nevű blokkeszközt, amely egy statikus memóriaterületen tárol adatokat. A logikai blokkméret 512 bájt, a kapacitás 256 blokk (128 KiB). A műveletek csak teljes blokkokra vonatkoznak.

```
/* labblk blokkeszköz - oktatói vázlat */
#define LABBLK_BLOCK_SIZE 512
#define LABBLK_NUM_BLOCKS 256

struct labblk_dev {
    uint8_t data[LABBLK_BLOCK_SIZE * LABBLK_NUM_BLOCKS];
    struct device dev;
};

static ssize_t labblk_read(struct device *dev, char *user_buf,
                           size_t count, off_t pos)
{
    struct labblk_dev *disk = container_of(dev, struct labblk_dev, dev);
    size_t blocknr = pos / LABBLK_BLOCK_SIZE;
```

```
    size_t nblocks = count / LABBLK_BLOCK_SIZE;

    if (pos % LABBLK_BLOCK_SIZE != 0 || count % LABBLK_BLOCK_SIZE != 0)
        return -EINVAL; /* a kérésnek blokkhatárra kell esnie */
    if (blocknr + nblocks > LABBLK_NUM_BLOCKS)
        return -ENOSPC;

    /* A belső pufferből másolunk a felhasználói térbe */
    for (size_t i = 0; i < nblocks; i++) {
        if (copyout(user_buf + i * LABBLK_BLOCK_SIZE,
                    &disk->data[(blocknr + i) * LABBLK_BLOCK_SIZE],
                    LABBLK_BLOCK_SIZE) != 0)
            return -EFAULT;
    }
    return nblocks * LABBLK_BLOCK_SIZE;
}

static ssize_t labblk_write(struct device *dev, const char *user_buf,
                            size_t count, off_t pos)
{
    /* Hasonlóan a read-hez, de copyin-nel */
    ...
}
```

A blokkeszköz `read` és `write` csak akkor fogad el kérést, ha a puffer mérete és a pozíció a blokkméret többszöröse. Ez biztosítja a blokkos szemantikát. A gyakorlatban a kernel blokkrétege gyakran átméretezi a kéréseket, de a mi egyszerű példánkban szigorúan ellenőrzünk.

14.1.15 10. Tipikus hibák és helyes megoldások

A blokk- és karakteres eszközök kezelése során a következő hibák a leggyakoribbak:

1. **Blokkeszköz karaktereskénti kezelése.** Ha egy lemezt úgy írunk, mintha bájtfolyam lenne, figyelmen kívül hagyjuk a blokkhatárokat, a gyorsítótárat és a sorrendezést. A helyes megoldás a blokkréteg használata, a kérések blokkméretre igazítása, és az `lseek` támogatása.
2. **Szektorhatárok figyelmen kívül hagyása közvetlen I/O-nál.** Ha a felhasználó közvetlenül ír egy blokkeszközt, a puffernek és a pozíciónak igazodnia kell a logikai blokkmérethez. A `EINVAL` hibakód jelzi a hibát, és a programnak újra kell próbálnia megfelelő pufferrel.
3. **ioctl parancskód-ütközés.** Ha két eszköz ugyanazt a parancskódot használja, az egyik meghívja a másik műveletét. A megoldás a típuskód központi nyilvántartása, a sorszámok egyedi kezelése, és a driverben az alapértelmezett `ENOTTY`.
4. **Ellenőrizetlen user pointer az ioctl-ban.** A kernel soha nem dereferálhat felhasználói címeket közvetlenül. A `copyin`/`copyout` vagy a `copy_from_user`/`copy_to_user` biztonságos másolást végez, és `EFAULT` hibát ad, ha a pointer érvénytelen.
5. **Nem megfelelő pufferméret a read/write hívásban.** Például egy karakteres eszköz csak 1 bájtot olvas, de a program 1024 bájtot kér. A driver visszaadja a ténylegesen beolvasott bájtok számát, ami kevesebb lehet; a programnak ezt kezelnie kell. A `read` visszatérési értéke mindig a ténylegesen átvitt bájtok száma, nem a kért.
6. **Atomi írás figyelmen kívül hagyása.** Egy blokkeszköz írása során a kernelnek biztosítania kell, hogy a blokk írása ne szakadjon meg. A driver nem térhet vissza részleges írással úgy, hogy a blokk félig állapotban marad; vagy a teljes blokkot írja, vagy hibát ad. A helyes megoldás a tranzakciós szemlélet már a driver szintjén.
7. **Hiányzó hibakód-kezelés.** A `open`, `read`, `write`, `ioctl` mindegyike visszatérhet hibával, és a programnak ezt ellenőriznie kell. A hibakódokat az `errno` tartalmazza; például `ENOTTY`

ha az ioctl nem támogatott, `ESPIPE` ha `lseek`-et hívnak karakteres eszközön, `ENXIO` ha nincs ilyen eszköz.

14.1.16 11. Labor: blokk- és karakteres eszköz interfész tervezése

A labor célja, hogy a hallgató megtervezze és implementálja egy egyszerű karakteres és egy egyszerű blokkeszköz interfészét az oktatói kernelben, majd reprodukálható módon mérje a műveletek viselkedését.

14.1.17 11.1. Kiinduló állapot és konfiguráció

- **Emulált környezet**: QEMU 8.2, x86_64, 128 MiB RAM, `virtio-serial` konzol, `virtio-blk` lemez (a laborhoz használt `labdisk.img` fájl, nem a hoszt lemeze).
- **Fordító és kernel**: GCC 13.2, freestanding C, az oktatói kernel forrása a `lab-io` könyvtárban. A kernel már tartalmazza a `device_ops` keretrendszert és a `copyin`/`copyout` segédfüggvényeket.
- **Build parancs**:

```
$ cd lab-io
$ make defconfig
$ make -j$(nproc)
```

- **Futtatás**:

```
$ qemu-system-x86_64 -m 128M \
    -drive file=labdisk.img,format=raw,if=virtio \
    -serial stdio \
    -kernel build/kernel.elf
```

A parancsok a reprodukciós csomag részét képezik, és a hoszt lemezét nem módosítják.

14.1.18 11.2. Feladat

Valósítsa meg a következő két eszközt a megadott interfész szerint:

1. **`lablog` karakteres eszköz** (major 240, minor 0)
    - Gyűrűpuffer mérete 4096 bájt.
    - `write` hozzáfűz a pufferhez, `read` kivesz a pufferből.
    - `ioctl` parancsok: `LAB_CLEAR` törli a puffert, `LAB_GET_SIZE` visszaadja a puffer kapacitását.
    - A `read` és `write` szekvenciális, `lseek` nem támogatott (`ESPIPE`).
2. **`labblk` blokkeszköz** (major 241, minor 0)
    - Logikai blokkméret 512 bájt, kapacitás 256 blokk (128 KiB).
    - `read` és `write` csak blokkhatárra igazított kérést fogad el; ellenkező esetben `EINVAL`.
    - A kapacitáson túli hozzáférés `ENOSPC` hibát ad.
    - Az eszköz statikus memóriaterületen tárolja az adatokat.

Mindkét eszközhöz hozzon létre device file-t a felhasználói térben:

```
$ mknod /dev/lablog c 240 0
$ mknod /dev/labblk b 241 0
```

14.1.19 11.3. Teszt és mérési napló

Írjon egy felhasználói tesztprogramot, amely:

- megnyitja a `lablog` eszközt, 1000 bájtot ír, majd 1000 bájtot olvas, és ellenőrzi a visszakapott adatot;
- meghívja a `LAB_CLEAR` ioctl-ot, majd ellenőrzi, hogy a puffer üres;
- megnyitja a `labblk` eszközt, 512 bájtot ír a 0. blokkra, majd visszaolvassa és összehasonlítja;
- megpróbál 1024 bájtot írni a 0. blokkra; várt hiba: `EINVAL`;
- megpróbál a 256. blokkra írni; várt hiba: `ENOSPC`.

A mérési naplóban rögzítse a következőket:

- **Mért értékek**: futási idő a 1000 bájtos írás-olvasás ciklusra, a karakteres eszköz átviteli sebessége bájt/másodpercben, a blokkeszköz 128 KiB írásának ideje.
- **Várt értékek**: a karakteres eszköz a 4096 bájtos puffer miatt a 1000 bájtot azonnal elfogadja, tehát a `write` visszatérési értéke 1000; a blokkeszköz az 512 bájtos írást azonnal végrehajtja, a 1024 bájtos írás `EINVAL`-t ad.
- **Módszer**: a futási idő mérése a kernel `rdtsc` időbélyegével, a ciklusszámot konvertálja másodperccé a rögzített 2,4 GHz-es QEMU virtuális órajel alapján. A mérést 10-szer ismételje meg, és adja meg az átlagot és a szórást.

A mért és a várt eredményt külön táblázatban kell feltüntetni. A labor akkor teljesül, ha a visszatérési értékek és a hibakódok megegyeznek a vártakkal, és a mérési napló reprodukálható.

14.1.20 12. Önellenőrző kérdések

1. Mi a különbség a blokk- és karakteres eszköz között? Adjon mindkettőre egy-egy példát, és magyarázza el, miért tartozik az adott osztályba.
2. Mi a major és a minor szám szerepe az eszközfájlokban?
3. Mikor indokolt az `ioctl` használata a `read`/`write` helyett? Mondjon egy konkrét példát.
4. Mi történik, ha egy blokkeszközön a `read` puffer mérete nem többszöröse a logikai blokkméretnek? Milyen hibakódot ad a rendszer?
5. Miért nem szabad egy eszközmeghajtóban közvetlenül dereferálni a felhasználói térből kapott pointert? Milyen alternatívát kell használni?

14.1.21 13. Összefoglalás és továbbvezetés

Ebben az alfejezetben megismertük az eszközök két alapvető osztályát: a blokk- és karakteres eszközöket. A blokkeszközök véletlen hozzáférésű, rögzített méretű blokkokat kezelnek, és a kernelben blokkréteg és page cache mögött dolgoznak; a karakteres eszközök bájtfolyamot nyújtanak, közvetlen meghajtóhívásokkal. Láttuk a device file-ok szerepét, a major/minor párost, valamint az `ioctl` vezérlőcsatorna felépítését és biztonsági szempontjait. A labor során egy egyszerű karakteres és blokkeszköz interfészét terveztük meg és mértük.

A következő alfejezet (Interruptok, DMA és pollozás) ezekre az alapokra építve bemutatja a megszakítások, a DMA és a pollozás szerepét az I/O-ban. Azt fogjuk látni, hogy a meghajtók hogyan szinkronizálnak a hardverrel, és hogyan kerülik el a processzor aktív várakozását.

14.2 Interruptok, DMA és pollozás

Az I/O alrendszerben a processzor és az eszköz együttműködése két alapvető kérdésre vezethető vissza: egyrészt hogyan értesül a CPU arról, hogy egy I/O-művelet befejeződött vagy adat érkezett; másrészt ki végzi el a tényleges adatmozgatást a memória és az eszköz között. E kérdésekre három alapmechanizmus ad választ: a lekérdezés (polling), a megszakítás (interrupt) és a közvetlen memóriaelérés (DMA). A gyakorlatban ezek ritkán válnak el élesen: a modern hálózati és tárolóstackek gyakran hibrid üzemmódban kombinálják őket, hogy egyensúlyt találjanak a késleltetés, a CPU-terhelés és az átviteli sebesség között.

14.2.1 Lekérdezéses I/O (polling)

A legegyszerűbb befejezésjelzés az, ha a CPU nem vár jelzésre, hanem addig kérdez, amíg az eszköz kész nem lesz. Ilyenkor a CPU egy állapotregisztert olvas ciklikusan, majd amikor az eszköz jelzi, hogy az adat elérhető vagy a kimeneti puffer üres, a CPU maga végzi el a tényleges adatmozgatást. Ezt programozott I/O-nak (Programmed I/O, PIO) nevezzük, mert a processzor programja közvetlenül az eszköz adatregiszteréből olvas, vagy oda ír.

```
/* Polling PIO vázlat - szemléltetés, nem éles hardverre */
volatile uint32_t *status = (volatile uint32_t *)DEV_STATUS_REG;
volatile uint32_t *data   = (volatile uint32_t *)DEV_DATA_REG;

while ((*status & DEV_STATUS_READY) == 0)
    cpu_relax();

uint32_t word = *data;
```

A `cpu_relax()` hívás x86-on `PAUSE`, ARM-on `YIELD` jellegű utasítás. Nem csinál logikai munkát, de csökkenti a szorosan egymás után végrehajtott olvasások energia- és buszterhelését, és más szálaknak is hagy végrehajtási erőforrást.

A polling előnye az egyszerűség és a kis késleltetés akkor, ha az eszköz gyorsan válaszol. Hátránya viszont, hogy a CPU hasznos helyett várakozással tölti az időt, és több eszköz egyidejű kiszolgálása esetén a ciklusos lekérdezés skálázhatatlan. Általános célú rendszerekben ezért a pollingot főleg rövid, előre látható idejű műveleteknél használják, például UART-küldésnél, ahol a TX FIFO néhány bájtnyi hellyel rendelkezik. Valós idejű rendszerekben a polling determinisztikusabb lehet, mert nincs megszakítási késleltetés és jitter, de a legrosszabb esetben a CPU-t teljesen lekötő ciklust gondosan korlátozni kell.

14.2.2 Megszakításos I/O (interrupt)

A megszakítás az eszköz aszinkron jelzése a CPU felé. Amikor az eszköz befejezte a munkát, adat érkezett, vagy hibát észlelt, megszakításkérést küld. A CPU az aktuális utasítás határán félbeszakítja a futó kontextust, elmenti a regisztereket, és a megszakítási vektor alapján a regisztrált kezelőre ugrik. A kezelő befejezése után a rendszer visszaállítja a kontextust, és a megszakított kód folytatódik. A megszakítás tehát lehetővé teszi, hogy a CPU az I/O-művelet ideje alatt más feladatot hajtson végre.

A megszakításkezelőt két részre bontják. A felső fél (top half) hardveres megszakításkontextusban fut: rövid, nem blokkolhat, és jellemzően csak az esemény nyugtázását, az okregiszter törlését, valamint az alsó fél beütemezését végzi. Az alsó fél (bottom half) késleltetett kontextusban fut: elvégzi a hosszabb munkát, például a pufferek másolását, a hálózati csomag feldolgozását vagy a befejezés jelzését. Linuxban az alsó fél megvalósítható softirq-ként, taskletként, workqueue-ként

vagy szálasított megszakításként (threaded IRQ). A cél mindig az, hogy a hard IRQ kontextus rövid legyen, mert annak futása alatt az adott CPU-n a megszakítások jellemzően le vannak tiltva, és a hosszú felső fél megnöveli a többi eszköz megszakítási késleltetését.

Példaként vegyünk egy hálózati kártyát beérkező csomag esetén. A felső fél megnézi az okregisztert, törli az interruptot, letiltja az interfész megszakítását, és beütemezi a poll callbacket. Az alsó félben a callback a gyűrűpufferből kötegelve dolgozza fel a csomagokat, majd ha a feldolgozási keret elfogyott, újra engedélyezi a megszakításokat.

A megszakítási késleltetés az eszköz jelétől a kezelő első utasításáig eltelt idő. Összetevői a hardveres jelút, a CPU folyamatban lévő utasításának befejezése, a kontextusmentés és a kernel belépési költsége. A jitter ennek az értéknek az ingadozása, ami valós idejű és biztonsági kritikus rendszerekben a legfontosabb metrika. Az interrupt storm azt jelenti, hogy az eszköz folyamatosan megszakítást kér, és a CPU a kezelő futtatásával tölti az idejét; ezt általában interrupt coalescinggel vagy pollingra váltással csökkentik.

```
/* ISR vázlat - top/bottom half szétválasztás */
static irqreturn_t edu_isr(int irq, void *data)
{
    struct edu_dev *dev = data;
    u32 status = ioread32(dev->regs + EDU_REG_INTR_STATUS);

    if (!(status & EDU_INTR_DMA_DONE))
        return IRQ_NONE;              /* hamis megszakítás */

    iowrite32(EDU_INTR_DMA_DONE, dev->regs + EDU_REG_INTR_CLEAR);
    queue_work(dev->wq, &dev->work); /* felső fél -> alsó fél */
    return IRQ_HANDLED;
}
```

14.2.3 Közvetlen memóriaelérés (DMA)

A DMA a második kérdést, az adatmozgatást veszi le a CPU válláról. Egy DMA-vezérlő vagy a busz masterként működő eszköz közvetlenül a fizikai memóriába ír, illetve onnan olvas, anélkül hogy minden szót a CPU-n keresztül kellene áramoltatni. A CPU szerepe az előkészítésre, az engedélyezésre és a befejezés feldolgozására korlátozódik.

A DMA-tranzakció tipikus lépései:

1. A driver lefoglal egy puffert a memóriában, és meghatározza a fizikai vagy DMA-címet.
2. A driver leképezi a puffert a DMA API-val, és kezeli a cache-koherenciát.
3. A driver beállítja az eszköz DMA-regisztereit: forrás, cél, hossz, irány.
4. A driver elindítja a tranzakciót, majd a CPU más munkát végezhet.
5. A DMA befejezésekor az eszköz megszakítást küld.
6. A megszakításkezelő alsó fele lezárja a leképezést, hibát ellenőriz, és befejezi a kérést.

```
/* DMA indítási vázlat - szemléltetés */
dma_addr_t dma = dma_map_single(&pdev->dev, buf, len, DMA_FROM_DEVICE);
if (dma_mapping_error(&pdev->dev, dma))
    return -EIO;

iowrite32(dma & 0xFFFFFFFF, regs + DMA_ADDR_LOW);
iowrite32(dma >> 32, regs + DMA_ADDR_HIGH);
iowrite32(len, regs + DMA_LEN);
iowrite32(DMA_CMD_START, regs + DMA_CMD);
```

A cache-koherencia a DMA egyik legnehezebb része. A CPU cache és a DMA két külön nézőpontból látja a memóriát. Két alapmodell terjedt el: a koherens leképezés, amelynél a hardver gondoskodik az egységességről, de a leképezés drágább, és a streaming leképezés, amelynél a drivernek irányfüggő `dma_sync_single_for_device` és `dma_sync_single_for_cpu` hívásokkal kell a cache-t üríteni vagy érvényteleníteni. A rossz irány vagy az elmaradt szinkronizáció olyan hibához vezet, amelyet nehéz reprodukálni, mert a sérült adat csak bizonyos időzítéseknél jelenik meg.

A szórt-gyűjtött (scatter-gather) DMA lehetővé teszi, hogy a fizikailag nem összefüggő lapokból álló puffer egyetlen tranzakcióban kerüljön átvitelre. A driver egy scatterlistát ad át, ahelyett hogy egy összefüggő fizikai tartományt feltételezne. Az IOMMU ezen túl védelmet ad: az eszköz DMA-címeit virtuális DMA-címtérre képezi, így az eszköz nem érhet el tetszőleges fizikai memóriát. Ha az eszköz címtartománya korlátozott, például egy 32 bites kártya 64 bites rendszerben, a kernel bounce buffert használ: az adatot egy megfelelően alacsony címen lévő pufferbe másolja, és a DMA ott történik.

A DMA jellemzően megszakítással zárul. A megszakítás nélküli, tisztán polling DMA is elképzelhető, de akkor a CPU-nak a DMA befejezését kellene kérdezgetnie, ami a nagy átvitelű eszközöknél elvesztegeti a DMA legfőbb előnyét. A valós rendszerekben ezért a DMA befejezése szinte mindig interrupttal érkezik, a feldolgozás pedig az alsó félben történik.

14.2.4 Hibrid üzemmód: megszakítás és polling kombinációja

A hibrid üzemmód a megszakítás és a polling előnyeit próbálja ötvözni. A legismertebb példa a Linux NAPI (New API) hálózati interfész. Alacsony forgalomnál a kártya minden csomagról megszakítást küld, így a késleltetés kicsi. Amikor azonban a forgalom megnő, a megszakításkezelő letiltja az interruptot, és a CPU átvált poll üzemmódba: egy ütemezett poll függvény a gyűrűpufferből kötegelve dolgozza fel a csomagokat. A poll függvény költségvetést (budget) kap, és ha a keret kimerül, a rendszer visszatér interrupt üzemmódba. Ezáltal a megszakítási overhead nem nő lineárisan a csomagsebességgel, ugyanakkor a késleltetés sem romlik nagyon alacsony forgalom esetén.

Az interrupt coalescing hasonló célt szolgál: a hálózati kártya nem minden csomag után szakít meg, hanem idő- vagy csomagküszöb alapján összegyűjti az eseményeket. Ez csökkenti a CPU-terhelést, de növeli az átlagos és a legrosszabb késleltetést. Az adaptív coalescing a forgalom függvényében állítja a küszöböt. A tárolóeszközöknél is megfigyelhető hibrid megoldás: NVMe-nél a befejezési sor feldolgozható megszakítással vagy pollinggal; az adminisztrációs parancsoknál interrupt, a nagy átvitelű I/O-nál pedig poll üzemmód használható.

14.2.5 Szinkronizáció és tipikus hibák

Az I/O befejezési mechanizmusokhoz kapcsolódó hibák többsége versenyhelyzet vagy memória- és erőforrás-tulajdonjogi hiba.

A megszakításkezelő és a process kontextus egyidejűleg hozzáférhet a driver állapotához, ezért a közös adatokat spinlockkal kell védeni, mégpedig `spin_lock_irqsave`/`spin_unlock_irqrestore` változattal, hogy a lock megszerzése alatt a helyi CPU-n a megszakítások tiltva legyenek. Az alsó fél és a felső fél közötti megosztott struktúrákra a bottom half mechanizmus saját szinkronizációja vonatkozik, például `local_bh_disable` vagy a munkasor eleve sorosítja az adott workqueue-t.

Az elveszett interrupt jellemző oka, hogy az eszköz akkor küldi a jelzést, amikor a driver még nem engedélyezte a megszakítást, vagy a kezelő nem törli az okregisztert. A robusztus kezelő indításkor ellenőrzi az eszköz állapotát, és a hibrid poll ciklus a megszakítás után is megnézi a

gyűrűt. A hamis (spurious) interruptnál a kezelőnek saját állapotregiszter alapján kell eldöntenie, hogy az eszközé-e az esemény; ha nem, `IRQ_NONE` értékkel tér vissza.

Gyakori hiba a mutex vagy memóriafoglalás a felső félben, ami blokkolhat, és ezzel megszakítási késleltetést vagy holtpontot okoz. A DMA-nál a puffer felszabadítása a `dma_unmap_single` előtt, illetve a befejezési megszakítás beérkezése előtt szintén tilos. A gyakorlatban a driver a puffert a DMA leképezés megszüntetéséig a saját tulajdonában tartja, és a befejezést csak akkor jelzi a felső rétegnek, ha a leképezés már lezárult.

14.2.6 Labor: interrupt- és DMA-alapú driver vázlat QEMU `edu` eszközzel

Ennek a labornak a célja, hogy egy emulált PCI eszközön (`edu`) megfigyeljük a polling, a megszakításos PIO és a DMA közötti különbségeket, és elkészítsünk egy minimális driver vázlatot, amelyben a felső fél megszakítást kezel, az alsó fél befejezi a kérést, a DMA pedig mozgatja az adatot.

Kiinduló állapot: eldobható QEMU vendég Linux rendszer, módosítható kernelmodul-fordító környezettel. A hoszt valódi lemezét, bootloaderét és rendszerpartícióját tilos módosítani.

Emulált hardver konfiguráció: QEMU x86_64, 4 vCPU, 4 GiB RAM, `-device edu`. A vendég kernelverzió a reprodukciós csomagban rögzítendő. A `edu` eszköz regisztertérképének pontos konstansait az adott QEMU verzió forrásából vagy dokumentációjából kell ellenőrizni; az alábbi kódban `EDU_REG_*` és `EDU_CMD_*` makrók oktatási vázlatként szerepelnek.

Build parancs:

```
make -C /lib/modules/$(uname -r)/build M=$(pwd) modules
```

Futtatás:

```
qemu-system-x86_64 \
  -enable-kvm -m 4G -smp 4 \
  -drive file=lab.qcow2,format=qcow2,if=virtio \
  -device edu \
  -netdev user,id=n0 -device virtio-net-pci,netdev=n0 \
  -nographic
```

A vendégben:

```
insmod edu_irq_dma.ko
cat /dev/edu0 > /tmp/edu.out
```

Labor vázlat: a következő kód a driver váza. Nem teljes, és nem éles rendszerre való; a hiányzó `probe`, `remove` és regiszterkonstansok a laborfeladat részei.

```
/* edu_irq_dma.c - labor vázlat, oktatási célra */
#include <linux/module.h>
#include <linux/pci.h>
#include <linux/interrupt.h>
#include <linux/dma-mapping.h>
#include <linux/completion.h>
#include <linux/workqueue.h>

#define EDU_BUF_SIZE 4096

struct edu_dev {
    struct pci_dev *pdev;
    void __iomem *regs;
    int irq;
```

```
    dma_addr_t dma_handle;
    void *buf;
    size_t len;
    struct completion done;
    struct workqueue_struct *wq;
    struct work_struct work;
    spinlock_t lock;
};

static void edu_work(struct work_struct *work)
{
    struct edu_dev *dev = container_of(work, struct edu_dev, work);

    /* Alsó fél: a DMA már befejeződött, a koherens puffer olvasható */
    complete(&dev->done);
}

static irqreturn_t edu_isr(int irq, void *data)
{
    struct edu_dev *dev = data;
    u32 status = ioread32(dev->regs + EDU_REG_INTR_STATUS);

    if (!(status & EDU_INTR_DMA_DONE))
        return IRQ_NONE;                /* hamis megszakítás */

    iowrite32(EDU_INTR_DMA_DONE, dev->regs + EDU_REG_INTR_CLEAR);
    queue_work(dev->wq, &dev->work);    /* felső fél -> alsó fél */
    return IRQ_HANDLED;
}

static ssize_t edu_read(struct file *filp, char __user *ubuf,
                        size_t count, loff_t *off)
{
    struct edu_dev *dev = filp->private_data;
    int ret;

    if (count > EDU_BUF_SIZE)
        count = EDU_BUF_SIZE;

    dev->buf = dma_alloc_coherent(&dev->pdev->dev, EDU_BUF_SIZE,
                                  &dev->dma_handle, GFP_KERNEL);
    if (!dev->buf)
        return -ENOMEM;

    reinit_completion(&dev->done);
    dev->len = count;

    iowrite32((u32)(dev->dma_handle & 0xFFFFFFFF),
              dev->regs + EDU_REG_DMA_ADDR_LOW);
    iowrite32((u32)(dev->dma_handle >> 32),
              dev->regs + EDU_REG_DMA_ADDR_HIGH);
    iowrite32(dev->len, dev->regs + EDU_REG_DMA_LEN);
    iowrite32(EDU_CMD_DMA_READ, dev->regs + EDU_REG_DMA_CMD);

    if (wait_for_completion_interruptible(&dev->done)) {
        ret = -ERESTARTSYS;
        goto out;
    }

    if (copy_to_user(ubuf, dev->buf, dev->len)) {
        ret = -EFAULT;
    } else {
        ret = dev->len;
```

```
    }

out:
    dma_free_coherent(&dev->pdev->dev, EDU_BUF_SIZE,
                      dev->buf, dev->dma_handle);
    return ret;
}
```

A fenti vázlat koherens DMA puffert használ, ezért nincs külön `dma_map_single`/`dma_unmap_single`. A laborban érdemes streaming DMA-val is elvégezni, és összehasonlítani a cache-szinkronizálás költségét.

Mérési napló: rögzítsük a következő táblázatot 64 KiB átvitelére mindhárom módban. A mért értékeket a laborban kell kitölteni; az itt szereplő „várt jellemző” csak elemzési támpont.

Mód	Átvitel	Mért idő (µs)	CPU kihasználtság (%)	Interruptok száma	Várt jellemző
Polling PIO	64 KiB	`<mérendő>`	`<mérendő>`	0	magas CPU, alacsony késleltetés
Interrupt + PIO	64 KiB	`<mérendő>`	`<mérendő>`	1	alacsony CPU várakozáskor, interrupt overhead
DMA + interrupt	64 KiB	`<mérendő>`	`<mérendő>`	1	legalacsonyabb CPU, cache/leképezési overhead

A méréshez használhatunk `perf stat`-ot, `ftrace`-et vagy egyszerű időzítést a driver tesztprogramjában. A mérési jegyzőkönyvben a hardvermodell, a QEMU verzió, a vendég kernelverzió, a fordítói beállítások és az órajelfrekvencia is szerepeljen. A mért és a várt/levezetett értéket külön kell jelölni.

Visszaállítás: a mérés után `rmmod edu_irq_dma`, majd a vendég leállítása, végül az eldobható QEMU lemezkép törlése.

14.2.7 Önellenőrző kérdés

Kérdés: Miért lehet a tisztán interrupt-vezérelt hálózati feldolgozás rossz választás nagy csomagsebességnél, és hogyan segít a NAPI hibrid üzemmód?

Válasz: Nagy csomagsebességnél minden csomag külön megszakítást vált ki, ami jelentős per-csomag overheadet okoz: kontextusmentés, kezelőfutás, cache- és TLB-szennyezés. A NAPI az első csomag megszakítása után letiltja az interruptot, és poll üzemmódban kötegelve dolgozza fel a gyűrűpuffer tartalmát. Ezzel csökken a CPU-terhelés, mert egy poll ciklus több csomagot is kiszolgál, ugyanakkor alacsony terhelésnél a megszakításos mód megmarad, így a késleltetés nem romlik feleslegesen.

14.2.8 Összefoglalás

Az I/O befejezése és az adatmozgatás három alapmechanizmusra épül. A polling egyszerű és alacsony késleltetést adhat, de CPU-időt pazarol. A megszakítás felszabadítja a CPU-t a várakozás alól, de minden eseményhez megszakítási overhead tartozik. A DMA a tényleges adatmozgatást is kiveszi a CPU kezéből, és a megszakítással kombinálva hatékony nagy átvitelű I/O-t tesz lehetővé. A modern rendszerekben a hibrid megoldások, a felső/alsó fél szétválasztás és a gondos cache-kezelés azok az eszközök, amelyekkel a megszakítási késleltetés, a jitter és a CPU-terhelés egyensúlyban tartható.

14.3 Eszközmeghajtók és driver modell

Az eszközmeghajtók és a driver modell célja, hogy a kernelben lévő eszközkezelés ne eseti illesztések halmaza legyen, hanem egységes, ellenőrizhető életciklus-szerződés mentén szerveződjön. Az előző alfejezetben megismertük a megszakításokat, a DMA-t és a lekérdezést; most azt vizsgáljuk meg, hogyan csatlakozik egy konkrét meghajtó a kernel eszközkezelő rétegéhez, és milyen szabályok garantálják, hogy a meghajtó betöltése, inicializálása, eltávolítása és a user-space felé való láthatósága konzisztens maradjon. A fejezet a Linux-kernel eszközmodelljének fogalmait használja, de az itt bemutatott minta – busz, eszköz, meghajtó, probe/remove – más monolitikus és hibrid kernelekben is megjelenik; a konkrét struktúratagok és API-nevek verziófüggők, ezért minden kernelverziónál ellenőrizni kell őket a hivatalos fejlécekben és dokumentációban.

14.3.1 A kernel eszközmodellje

A kernel eszközmodelljének három alapvető szereplője van:

- **busz (bus):** az a kapcsolódási felület, amelyen az eszközök és a meghajtók találkoznak; például PCI, USB, platform, I²C, SPI vagy egy teljesen virtuális busz.
- **eszköz (device):** a hardver vagy virtuális erőforrás egy konkrét példánya, amelyet a kernel az adott buszon lát.
- **meghajtó (driver):** az a kód és adat, amely egy adott eszköztípus kezelését végzi, és amely a busz illesztési szabályai szerint hozzárendelhető egy eszközhöz.

Ezeket a szereplőket a kernel közös objektummodellje fogja össze. A `kobject` minden eszközmodellbeli objektum alapja: hivatkozásszámlálást, szülő-gyermek hierarchiát és egy `sysfs`-beli megjelenést biztosít. A `sysfs` (általában `/sys` alatt) a kernel belső eszközmodelljének user-space nézete; itt jelennek meg a buszok, eszközök, meghajtók és osztályok mint könyvtárak és szimbolikus linkek. A `class` fogalom a funkció szerinti csoportosítást adja: például minden blokkos eszköz a `block` osztályba, minden hálózati interfész a `net` osztályba tartozhat, függetlenül attól, hogy milyen buszon ül.

A modell a következő célokat szolgálja:

1. **Egységes életciklus:** az eszköz megjelenése, eltávolítása, felfüggesztése és felébresztése ugyanazzal a szerződéssel kezelhető, függetlenül a hardver típusától.
2. **Helyes hivatkozáskezelés:** a refcountolt `kobject`-ök megakadályozzák, hogy egy meghajtó vagy eszköz adatai akkor szabaduljanak fel, amikor még hivatkozik rájuk valaki.
3. **Dinamikus illesztés:** a rendszer futás közben fedezhet fel eszközöket, és a megfelelő meghajtó betöltésével azonnal összekapcsolhatja őket.

4. **User-space értesítés:** az `uevent` mechanizmus értesíti a user-space-t (például `udev`, `systemd-udevd`), hogy eszköz jelent meg vagy tűnt el, így a user-space szabályok automatikusan létrehozhatnak eszközcsomópontokat, betölthetnek firmware-t vagy elindíthatnak szolgáltatásokat.

A következő szakaszban az illesztés folyamatát bontjuk ki lépésről lépésre, mert a gyakorlati hibák nagy része ezen a szinten keletkezik.

14.3.2 Busz, eszköz és meghajtó illesztése

A kernelben egy busz típusát a `bus_type` struktúra írja le. Ennek fontosabb részei közé tartoznak a busz neve, a root `kobject`, valamint az illesztést végző függvények. A Linux 6.x-ben a `bus_type` lényegi tagjai a következők (a pontos ABI-t az adott kernelverzióhoz ellenőrizni kell):

- `name`: a busz neve, például `"demo_bus"`.
- `match`: ez a függvény dönti el, hogy egy adott `device` és egy adott `driver` összetartozik-e.
- `uevent`: opcionálisan testreszabhatja a user-space-nek küldött környezeti változókat.
- `probe` és `remove`: buszszintű horgok, ha a busznak közös erőforrás-kezelésre van szüksége; a tényleges eszközspecifikus munka általában a driver `probe` és `remove` függvényében történik.
- `pm`: energia- és felfüggesztési műveletek.

Egy eszközt a `device` struktúra reprezentál. Fontosabb mezői közé tartozik az eszköz neve, a szülő `device` mutató, a buszra mutató `bus`, egy `driver_data` mutató, valamint a hivatkozásszámláló. A meghajtót a `device_driver` struktúra írja le, amely tartalmazza a nevet, a buszra mutató `bus` pointert, a `probe` és `remove` függvénymutatókat, valamint opcionálisan az `of_match_table`-t vagy más azonosítótáblát.

Az illesztés algoritmusa a következő lépésekből áll:

1. **Eszköz regisztrálása:** a rendszer valamelyik rétege (például a PCI-busz felderítése, a platformkód vagy egy virtuális tesztmodul) létrehoz egy `device` objektumot, beállítja a buszát, nevét és erőforrásait, majd meghívja a `device_register`-t. A kernel elhelyezi az eszközt a busz eszközlistáján, és megjeleníti a `sysfs`-ben.
2. **Meghajtó regisztrálása:** egy meghajtó modul betöltésekor a `driver_register` hatására a driver bekerül a busz driverlistájára.
3. **Illesztési keresés:** a busz `match` függvénye minden regisztrált driverre és eszközre lefut, amikor új eszköz vagy új driver jelenik meg. Ha a függvény igazat ad vissza, a kernel megkísérli a hozzárendelést.
4. **Probe hívás:** a kernel meghívja a driver `probe` függvényét, átadva neki az eszköz mutatóját. A `probe` visszatérési értéke dönt a sikerességről. Ha `0`, az illesztés létrejött; ha negatív hibakód, a kernel elengedi az eszközt, és megpróbálhat másik drivert keresni.
5. **Remove hívás:** amikor az eszközt eltávolítják, vagy a drivert ki akarják tölteni, a kernel meghívja a driver `remove` függvényét, hogy a meghajtó minden erőforrást felszabadítson és leállítsa az eszközt.

Fontos, hogy a `probe` és a `remove` ugyanazon a buszon párhuzamosan is meghívódhat, ezért a meghajtónak saját magának kell kezelnie a konkurenciát, ha több eszköz egyidejűleg is csatlakozhat.

14.3.3 Egy virtuális busz és driver példa

A következő kód egy leegyszerűsített, szemléltető virtuális buszt, eszközt és drivert mutat be. A kód célja a minta megértése; nem közvetlenül futtatandó éles kernelmodul, mert a tényleges API-k és a hibakezelés kernelverziónként változhat. A példa a Linux-kernel fogalmait használja, de a helyesség érdekében az adott verzió fejléceit és dokumentációját kell alapul venni.

```
/* demo_bus.c - szemléltető virtuális busz, eszköz és driver.
 * A kód nem éles rendszerhez készült; a struktúratagok és
 * a regisztrációs függvények kernelverziónként eltérhetnek.
 */
#include <linux/device.h>
#include <linux/module.h>
#include <linux/slab.h>
#include <linux/string.h>

struct demo_device {
    struct device dev;
    const char *name;
};

/* Az eszköz és a driver neve alapján illesztünk. */
static int demo_match(struct device *dev, struct device_driver *drv)
{
    struct demo_device *ddev = container_of(dev, struct demo_device, dev);
    return strcmp(ddev->name, drv->name) == 0;
}

static struct bus_type demo_bus = {
    .name  = "demo_bus",
    .match = demo_match,
};

static struct demo_device *demo_dev;

static ssize_t demo_attr_show(struct device *dev,
                              struct device_attribute *attr,
                              char *buf)
{
    return sysfs_emit(buf, "demo device\n");
}

static DEVICE_ATTR_RO(demo_attr);

static void demo_device_release(struct device *dev)
{
    /* Itt szabadul fel a demo_device. */
    kfree(container_of(dev, struct demo_device, dev));
}

static int __init demo_init(void)
{
    int ret;

    ret = bus_register(&demo_bus);
    if (ret)
        return ret;

    demo_dev = kzalloc(sizeof(*demo_dev), GFP_KERNEL);
    if (!demo_dev) {
        bus_unregister(&demo_bus);
        return -ENOMEM;
```

```
    }

    demo_dev->name = "demo0";
    demo_dev->dev.bus = &demo_bus;
    demo_dev->dev.parent = NULL;
    demo_dev->dev.release = demo_device_release;
    dev_set_name(&demo_dev->dev, "demo0");

    ret = device_register(&demo_dev->dev);
    if (ret) {
        put_device(&demo_dev->dev);
        bus_unregister(&demo_bus);
        return ret;
    }

    ret = device_create_file(&demo_dev->dev, &dev_attr_demo_attr);
    if (ret) {
        device_unregister(&demo_dev->dev);
        bus_unregister(&demo_bus);
        return ret;
    }

    return 0;
}

static void __exit demo_exit(void)
{
    device_remove_file(&demo_dev->dev, &dev_attr_demo_attr);
    device_unregister(&demo_dev->dev);
    bus_unregister(&demo_bus);
}

module_init(demo_init);
module_exit(demo_exit);
MODULE_LICENSE("GPL");
```

A példában a `demo_bus` buszhoz tartozik egy `match` függvény, amely az eszköz `name` mezőjét hasonlítja a driver nevéhez. A `demo_init` regisztrál egy `demo0` eszközt, és létrehoz egy `sysfs`-attribútumot. A `release` függvény hívódik meg, amikor az utolsó hivatkozás is megszűnik; a memória felszabadítás itt történik.

A driver oldala a következőképpen nézhet ki:

```
/* demo_drv.c – szemléltető meghajtó a demo_bus-hoz. */
#include <linux/device.h>
#include <linux/module.h>
#include <linux/slab.h>

static int demo_probe(struct device *dev)
{
    /* Itt lehetne hardver-erőforrásokat igényelni,
     * megszakítást regisztrálni, eszközosztályt létrehozni.
     */
    dev_info(dev, "demo_probe: device %s matched\n", dev_name(dev));

    /* Példa: driver-specifikus adat tárolása. */
    dev_set_drvdata(dev, (void *)0x1);

    return 0;
}

static void demo_remove(struct device *dev)
{
```

```
    dev_info(dev, "demo_remove: device %s removed\n", dev_name(dev));

    /* Itt kell felszabadítani a probe által lefoglalt erőforrásokat. */
    dev_set_drvdata(dev, NULL);
}

static struct device_driver demo_driver = {
    .name   = "demo0",
    .bus    = &demo_bus,
    .probe  = demo_probe,
    .remove = demo_remove,
};

static int __init demo_drv_init(void)
{
    return driver_register(&demo_driver);
}

static void __exit demo_drv_exit(void)
{
    driver_unregister(&demo_driver);
}

module_init(demo_drv_init);
module_exit(demo_drv_exit);
MODULE_LICENSE("GPL");
```

A `demo_probe` meghívódik, amikor a busz `match` függvénye igazat ad vissza. A `dev_set_drvdata` egy mutatót tárol a `device`-ben, amelyet a meghajtó később bármikor visszakaphat a `dev_get_drvdata` segítségével. A `remove` ezt törli, és ezzel jól mutatja a probe/remove szimmetria elvét.

14.3.4 Az erőforrás-kezelés szerepe

A meghajtók gyakran igényelnek memóriát, megszakítást, I/O-portokat vagy DMA-területeket. A Linux-kernelben a `devm_*` (managed) API család arra való, hogy ezeket az erőforrásokat a `device` életciklusához kösse. Ha a `probe` `devm_kzalloc`-kal foglal memóriát, `devm_request_irq`-val regisztrál megszakítást, vagy `devm_ioremap`-pal képez le memóriát, akkor a kernel automatikusan felszabadítja ezeket a `remove` után, illetve akkor is, ha a `probe` valamelyik későbbi lépése hibát ad. Ez csökkenti a hibás hibakezelési utak számát, de nem helyettesíti a `remove` megírását: minden olyan erőforrást, amelyet nem `devm_` API-val szereztünk, explicit módon kell felszabadítani.

A következő kód a `devm_*` használatát illusztrálja egy `probe` függvényben:

```
/* Példa: managed erőforrások a probe-ban. */
static int example_probe(struct device *dev)
{
    void __iomem *base;
    int irq;
    struct example_dev *priv;

    priv = devm_kzalloc(dev, sizeof(*priv), GFP_KERNEL);
    if (!priv)
        return -ENOMEM;

    base = devm_ioremap_resource(dev, &res);
    if (IS_ERR(base))
        return PTR_ERR(base);

    irq = platform_get_irq(to_platform_device(dev), 0);
```

```
    if (irq < 0)
        return irq;

    if (devm_request_irq(dev, irq, example_isr, 0,
                         "example", priv))
        return -EINVAL;

    dev_set_drvdata(dev, priv);
    return 0;
}

static void example_remove(struct device *dev)
{
    struct example_dev *priv = dev_get_drvdata(dev);

    /* A managed erőforrásokat nem kell külön felszabadítani.
     * Itt csak a nem managed erőforrások és a hardver leállítása
     * szerepel.
     */
    if (priv)
        dev_dbg(dev, "device removed\n");
}
```

A `devm_*` függvények használatakor is figyelni kell arra, hogy az erőforrások felszabadítási sorrendje a regisztráció fordítottja, ezért a `probe`-ban érdemes a lehető legkorábban regisztrálni a managed erőforrásokat, hogy a hibakezelés automatikusan konzisztens legyen.

14.3.5 User-space driver és virtuális eszköz

Nem minden eszközmeghajtó fut kernelmódban. A user-space driverek a felhasználói térben lévő programok, amelyek a kernel által biztosított interfészeken keresztül érik el a hardvert. Ez a megközelítés különösen hasznos akkor, ha:

- a hardver egyszerű, és a teljesítmény nem kritikus,
- a meghajtó gyors fejlesztése vagy tesztelése a cél,
- a kernel stabilitása fontosabb, mint a minimális késleltetés,
- a hardverhiba izolálása kívánatos.

A Linux-kernel két fő keretrendszert kínál user-space meghajtókhoz:

- **UIO (Userspace I/O):** a kernel egy kis, általános meghajtóréteget biztosít, amely az eszköz regisztereit `mmap`-pel a user-space-be képezi, a megszakításokat pedig egy `/dev/uioX` karakteres eszközön keresztül teszi kezelhetővé. A user-space program a megszakítás érkezésére a `read` hívással blokkol, majd elvégzi a szükséges műveleteket.
- **VFIO (Virtual Function I/O):** nagyobb teljesítményű és biztonságosabb megoldás, amely az IOMMU segítségével közvetlen eszközhozzáférést és DMA-t biztosít a user-space számára, miközben a memóriavédelmet a hardver garantálja.

A `sysfs` és az `uevent` mechanizmus lehetővé teszi, hogy user-space szolgáltatások figyeljék az eszközök megjelenését és eltűnését. A `udev` szabályok például automatikusan létrehozzák a `/dev` alatti csomópontokat, betölthetnek firmware-t, vagy elindíthatnak egy user-space drivert.

A virtuális eszközök olyan kernelben megvalósított eszközök, amelyek nem valódi hardvert vezérelnek, hanem valamilyen szoftveres funkciót tesznek elérhetővé a szokásos eszközinterfészen keresztül. Példák:

- `/dev/null`, `/dev/zero`, `/dev/random`: karakteres eszközök, amelyek adatnyelőt, nullákat vagy véletlen számokat szolgáltatnak.
- `loop` eszköz: egy fájlt blokk-eszközként jelenít meg.
- `ramdisk`, `zram`: memóriaalapú blokk-eszköz.
- `veth`, `dummy`: virtuális hálózati interfészek.
- `virtio` eszközök: virtualizált környezetben a vendég és a hoszt közötti kommunikációra.

Ezek a virtuális eszközök gyakran ugyanazt a busz-driver-modellt használják, mint a valódi hardverek, ezért kiválóan alkalmasak a driver-életciklus tanulmányozására és tesztelésére.

A user-space driver tipikus életciklusa a következő:

1. A kernelben egy általános UIO vagy VFIO driver kötődik az eszközhöz, és létrehozza a `/dev/uioX` eszközt.
2. A user-space program megnyitja az eszközt (`open`), leképezi a regiszterterületet (`mmap`), majd blokkoló `read`-del vár a megszakításra.
3. A megszakítás után a program elvégzi a hardver kezelését, például beolvassa a regisztereket, feldolgozza az adatokat, majd ismét vár.
4. A program bezárásakor a kernel felszabadítja az erőforrásokat.

A következő pszeudokód egy UIO-alapú user-space driver vázát mutatja:

```
/* user_demo.c - szemléltető UIO-driver váza (user-space). */
#include <fcntl.h>
#include <sys/mman.h>
#include <unistd.h>
#include <stdio.h>
#include <stdint.h>

int main(void)
{
    int fd = open("/dev/uio0", O_RDWR | O_SYNC);
    if (fd < 0) {
        perror("open");
        return 1;
    }

    /* A regiszterterület leképezése; a méretet a sysfs-ből
     * vagy a device tree-ből kell meghatározni.
     */
    void *base = mmap(NULL, 0x1000, PROT_READ | PROT_WRITE,
                      MAP_SHARED, fd, 0);
    if (base == MAP_FAILED) {
        perror("mmap");
        close(fd);
        return 1;
    }

    for (;;) {
        uint32_t irq_count;
        ssize_t n = read(fd, &irq_count, sizeof(irq_count));
        if (n != sizeof(irq_count)) {
            perror("read");
            break;
        }

        /* Hardver-regiszterek olvasása a leképezett területről. */
        volatile uint32_t *status = (volatile uint32_t *)base;
        printf("IRQ: %u, status=0x%x\n", irq_count, *status);

        /* Megszakítás nyugtázása a hardverben. */
```

```
        /* ... */
    }

    munmap(base, 0x1000);
    close(fd);
    return 0;
}
```

A kódban a `read` hívás visszatérési értéke a megszakítás sorszáma, ami lehetővé teszi az elveszett megszakítások észlelését. A gyakorlatban a regiszterek értelmezése hardverspecifikus, és a `mmap` méretét a `sysfs`-ben közzétett `maps` attribútumból kell kiolvasni.

14.3.6 Tipikus hibák és helyes megoldásuk

A driver-modellben elkövetett hibák gyakran később, nehezen reprodukálható körülmények között jelentkeznek. Az alábbi lista a leggyakoribb problémákat és a megfelelő megoldási mintákat foglalja össze.

1. **Probe/remove aszimmetria.** Ha a `probe` lefoglal egy erőforrást, de a `remove` nem szabadítja fel, memóriaszivárgás vagy eszközhiba keletkezik. A megoldás az, hogy minden `probe`-ban történő erőforrás-allokációhoz tartozzon egy explicit vagy `devm_*` által kezelt felszabadítás, és a `remove` pontosan a `probe` fordított sorrendjében végezze a takarítást.

2. **Use-after-free a `device`-ben.** Ha a `remove` után a meghajtó hozzáfér a `dev_set_drvdata`-val tárolt mutatóhoz, vagy egy későbbi eseménykezelő használja a már felszabadított eszközt, a kernel hibát jelez. A helyes minta: a `remove` előtt minden eseményforrást le kell állítani (például `cancel_work_sync`, `free_irq`), és a `dev_set_drvdata`-t `NULL`-ra kell állítani.

3. **Kettős probe.** Ha a busz `match` függvénye nem egyértelmű, vagy a driver regisztrálása közben új eszköz jelenik meg, a kernel párhuzamosan hívhatja a `probe`-ot ugyanarra az eszközre. A megoldás a megfelelő zárolás és az `atomic` állapotjelző használata, vagy annak biztosítása, hogy a `match` és a regisztrációs sorrend a buszon belül konzisztens.

4. **Nem kezelt hibák a `probe`-ban.** Ha a `probe` több lépésből áll, és egy lépés hibát ad, a korábban lefoglalt erőforrások felszabadításáról gondoskodni kell. A `devm_*` API ezt automatizálja, de ha nem managed erőforrásokat használunk, minden hibaleágon explicit takarítást kell végezni.

5. **Túl korai return a `probe`-ban.** Egy `return` a `devm_*` regisztrációk előtt megakadályozhatja, hogy a managed erőforrások a későbbi hibáknál felszabaduljanak. A megoldás: először minden managed erőforrást regisztráljunk, és csak utána végezzük a nem managed műveleteket.

6. **Match függvény hibái.** A `match` gyakran mutatókat hasonlít össze, vagy a `device` és a `driver` adatait olvassa. Ha a mutató nem a várt típusra mutat, a `container_of` hibás memóriacímet eredményez. A megoldás a busz-specifikus adatszerkezetek pontos ismerete és a `match` függvény egyszerűsége.

7. **User-space driverben a megszakításkezelés hibája.** Az UIO `read` hívása csak akkor blokkol, ha a kernel UIO-réteg engedélyezte a megszakítást; ha a user-space program nem nyugtázza a megszakítást a hardverben, az eszköz nem fog újabb megszakítást küldeni. A megoldás a hardverdokumentáció szerinti nyugtázás és a `read` visszatérési értékének ellenőrzése.

8. **Erőforrás-tulajdonjog hiányos dokumentálása.** A kernelkódban minden határfelületnél egyértelművé kell tenni, hogy ki birtokolja a mutatót, ki felel a felszabadításért, és mikor lehet hívni az adott függvényt. A dokumentálatlanság gyakran vezet a fenti hibákhoz.

14.3.7 Labor: driver életciklus- és probe/remove terv

Ez a labor egy ellenőrzött, emulált környezetben mutatja be a driver-életciklus működését, és rögzíti a mért és várt eredményeket. A labor célja, hogy az olvasó saját maga is megfigyelje a `probe` és `remove` hívásokat, a `sysfs`-beli megjelenést, és a tipikus hibák hatását.

14.3.8 Kiinduló állapot

A laborhoz a következő környezetet használjuk:

- Hoszt: x86_64, Linux 6.x, QEMU 8.x, gcc és make.
- Vendég: ugyanaz az architektúra, minimális initramfs, amely tartalmazza a szükséges modulokat és a `sysfs`-et.
- A kernel forrása helyben elérhető, és a modulok a vendég kernelhez fordíthatók.
- A labor nem módosítja a hoszt rendszert, nem érint valódi lemezt vagy bootloadert; minden művelet az emulált vendégen belül történik.

A reprodukálhatóság érdekében a konfigurációt rögzíteni kell. Példa a `qemu` indítására:

```
qemu-system-x86_64 \
  -kernel /path/to/bzImage \
  -initrd /path/to/initramfs.cpio.gz \
  -append "console=ttyS0" \
  -nographic \
  -smp 1 \
  -m 256M
```

A kernel- és initramfs-útvonalak helyőrzők; a tényleges elérési utakat a reprodukciós csomagban kell megadni.

14.3.9 Build- és futtatási parancsok

A modul fordításához a kernel fejlécei szükségesek. Az alábbi `Makefile` egy lehetséges váz:

```
obj-m += demo_bus.o demo_drv.o

all:
    $(MAKE) -C /lib/modules/$(shell uname -r)/build M=$(PWD) modules

clean:
    $(MAKE) -C /lib/modules/$(shell uname -r)/build M=$(PWD) clean
```

A modul betöltése és eltávolítása a vendégen:

```
insmod demo_bus.ko
insmod demo_drv.ko
dmesg | tail -20
rmmod demo_drv
rmmod demo_bus
```

A `sysfs` ellenőrzése:

```
ls -l /sys/bus/demo_bus/
ls -l /sys/bus/demo_bus/devices/
ls -l /sys/bus/demo_bus/drivers/
cat /sys/bus/demo_bus/devices/demo0/demo_attr
```

14.3.10 Mérési napló

A laborban mérjük a `probe` és `remove` függvények időtartamát, valamint a modulbetöltés és -eltávolítás idejét. A méréshez a `ktime_get_ns()` függvényt használjuk a `probe` elején és végén, majd a különbséget `dev_info`-val kiírjuk. A várt értékeket külön jelöljük, mert a tényleges futásidő függ a vendég CPU órajelétől, a kernel konfigurációjától és a QEMU emulációs sebességétől.

A reprodukciós jegyzetfüzetben a következőket kell rögzíteni:

- **CPU:** QEMU emulált `qemu64`, 2.0 GHz névleges, `-smp 1`.
- **Memória:** 256 MiB.
- **Kernel:** 6.x (pontos verzió a csomagban).
- **Fordítási beállítások:** `gcc` optimalizálás `-O2`, `CONFIG_DEBUG_FS=y`, `CONFIG_DYNAMIC_DEBUG=y` (ha elérhető).

Példa mérési napló (a számok szemléltető helyőrzők, nem mért értékek):

Művelet	Várt eredmény	Mért eredmény (1. futás)	Mért eredmény (2. futás)
`insmod demo_bus.ko`	sikeres regisztráció, `sysfs` könyvtár megjelenik	sikeres	sikeres
`insmod demo_drv.ko`	`probe` hívás, naplóbejegyzés	sikeres, `probe` 1.2 µs	sikeres, `probe` 1.1 µs
`cat demo_attr`	`demo device` szöveg	`demo device`	`demo device`
`rmmod demo_drv`	`remove` hívás, naplóbejegyzés	sikeres, `remove` 0.8 µs	sikeres, `remove` 0.7 µs
`rmmod demo_bus`	`sysfs` könyvtár eltűnik	sikeres	sikeres

A várt és mért eredményeket külön kell választani: a várt oszlop a specifikációból következik, a mért oszlop a tényleges futásból. Az időméréseknél a hardvert, a fordítási beállításokat és az órajelfrekvenciát is fel kell tüntetni, ahogyan azt a könyv konvenciói előírják.

14.3.11 Hibaszimuláció

A labor második részében szándékosan hibát viszünk a `probe`-ba, hogy megfigyeljük a kernel reakcióját. Például a `demo_probe` visszatérési értékét `-ENOMEM`-re állítjuk, és újra betöltjük a drivert. A várt viselkedés az, hogy a kernel hibaüzenetet ír a `dmesg`-be, és nem hozza létre a `sysfs`-beli kapcsolatot. A mért viselkedést ugyanígy rögzíteni kell.

A labor elvégzése után a vendég leállítható a `poweroff` paranccsal, vagy a QEMU-ból történő kilépéssel; a hoszt rendszer állapota változatlan marad.

14.3.12 Önellenőrző kérdés

A fejezet végén ellenőrizze a megértést: mi történik, ha a `probe` függvény hibával tér vissza, de a `devm_*` API-val lefoglalt erőforrások már regisztrálva vannak? Magyarázza el, miért nincs szükség explicit takarításra ebben az esetben, és milyen kivételek léteznek.

(A helyes válasz: a kernel a `probe` hibája esetén automatikusan felszabadítja a `devm_*` erőforrásokat a regisztráció fordított sorrendjében. Kivétel, ha a `probe` a `devm_*` regisztráció előtt hibázik, vagy ha a kód nem managed API-t használ; ilyenkor explicit takarítás szükséges.)

14.3.13 Összefoglalás

A kernel eszközmodellje a busz, az eszköz és a meghajtó hármasára épül. A busz `match` függvénye dönti el az illesztést, a driver `probe` és `remove` függvényei pedig az életciklus fő állomásait jelentik. A `kobject`, a `sysfs` és az `uevent` mechanizmus biztosítja a user-space felé való láthatóságot és a dinamikus eszközkezelést. A managed erőforrás-kezelés csökkenti a hibalehetőségeket, de a probe/remove szimmetriát és a hibatűrő tervezést nem helyettesíti. A user-space driverek és a virtuális eszközök ugyanezt a modellt használják, és lehetővé teszik az eszközkezelés biztonságosabb vagy gyorsabb fejlesztését. A következő alfejezet ezekre a fogalmakra építve mutatja be a tárolóeszközök – HDD, SSD, NVMe – kezelését, ahol a blokk-eszköz réteg és a driver modell találkozása különösen fontos.

14.4 Tárolóeszközök: HDD, SSD, NVMe és buffering

Az alábbi alfejezet azt járja körül, hogyan viselkednek a leggyakoribb tárolóeszközök, honnan származnak a teljesítménykülönbségek, és miért válik a pufferezés egyszerre a rendszer egyik legfontosabb gyorsítójává és az adattartósság egyik leggyakoribb buktatójává. A labor kizárólag eldobható, emulált környezetre épül, valós lemezt, firmware-t vagy bootloadert nem módosít.

14.4.1 1. Miért nem „csak egy lemez” a tárolóeszköz?

A folyamatok számára a blokkos eszköz gyakran egyetlen absztrakciónak tűnik: adott logikai blokkcímről olvasunk, adott logikai blokkcímre írunk. A hardver alatta azonban gyökeresen eltérő utat jár be attól függően, hogy forgólemezről, flash-alapú SSD-ről vagy NVMe-ről beszélünk. Az operációs rendszer I/O-rétege nem teheti meg, hogy ezeket az eltéréseket teljesen elrejti: az ütemező, a blokkréteg, a gyorsítótár és a fájlrendszer csak akkor tud elfogadható teljesítményt adni, ha figyelembe veszi a seek-költséget, a párhuzamosíthatóságot, a kiíratási egységet és a kopást.

A tárolóeszközöket négy szempontból érdemes jellemezni:

- **Latencia:** mennyi idő alatt érkezik meg a válasz egyetlen kérésre. Ez magában foglalja a parancs feldolgozását, a mechanikus pozicionálást vagy a flash-címfordítást, az adatmozgatást és a megszakítás beküldését.
- **IOPS:** másodpercenként teljesíthető bemeneti/kimeneti műveletek száma. Kis, véletlenszerű kéréseknél általában ez a szűk keresztmetszet.
- **Sávszélesség:** nagy, lehetőleg szekvenciális kéréseknél elérhető adatátviteli sebesség.
- **Írási és törlési szemantika:** mekkora egységben írható a médium, hogyan kopik, és mekkora extra fizikai írást okoz egy logikai írás.

A hagyományos mágneslemeznél a latencia és az IOPS a mechanikus fejtartó miatt korlátos. A NAND-alapú SSD-nél a mechanikus késleltetés eltűnik, de megjelenik az átírást megelőző törlés, a szemétgyűjtés és a kopáskiegyenlítés miatti írási erősítés. Az NVMe pedig nem új médiumot, hanem új csatlakozási és parancsfeldolgozási utat jelent: a PCIe-alapú, több I/O-soros protokoll nagyobb párhuzamosságot és alacsonyabb szoftveres terhelést enged, mint a régi SATA vagy SAS parancssorok.

14.4.2 2. HDD: amikor a mechanika határozza meg a teljesítményt

A mágneslemezes meghajtó (HDD) cilinderekre, fejekre és szektorokra szerveződik. Egy blokkolvasáshoz a vezérlőnek három fő lépést kell végrehajtania:

1. **Pozicionálás (seek):** a fejkart a megfelelő sáv fölé kell mozgatni. Ez a legdrágább művelet, tipikus értéke asztali lemezeken 8–12 ms, nagyvállalati lemezeken 3–6 ms lehet.
2. **Forgási késleltetés:** miután a fej a helyén van, meg kell várni, amíg a kért szektor a fej alá fordul. Átlagosan egy fél fordulat, azaz 7200 ford/perc esetén:

$$t_{\text{rot}} = \frac{30}{7200} \approx 4{,}2\,\text{ms}.$$

3. **Átvitel:** a szektor adatainak beolvasása a lemezről a belső pufferbe.

Egyetlen véletlenszerű 4 KiB olvasás válaszideje tehát közelítőleg:

$$T_{\text{io,HDD}} = t_{\text{seek}} + t_{\text{rot}} + t_{\text{xfer}} + t_{\text{queue}}.$$

Ha például a seek 9 ms, a forgási késleltetés 4,2 ms, az átvitel és a várakozási sor együtt 0,3 ms, akkor $T_{\text{io}} \approx 13{,}5\,\text{ms}$. Ebből az IOPS-felső korlát:

$$\text{IOPS}_{\text{max}} \approx \frac{1}{T_{\text{io}}} \approx 74.$$

A valóságban a vezérlő parancssora, az NCQ (Native Command Queuing) és az ütemező újrarendezése javíthat ezen, de a véletlenszerű kis I/O mindig ezres nagyságrend alatt marad. Ugyanakkor egy szekvenciális olvasásnál a seek és a forgási késleltetés elhanyagolhatóvá válik, és a meghajtó a médium átviteli sebességét tudja nyújtani, ami egy modern 3,5 hüvelykes lemeznél 150–250 MiB/s körüli.

Ebből következik a HDD-k legfontosabb üzemeltetési tanulsága: **a munkaterhelés lokalitása döntő**. A fájlrendszer és a blokk-I/O ütemező ezért igyekszik a kéréseket növekvő blokkcím szerint rendezni, összevonni, és a lehető legkevesebb fejmozgást okozni. A „seek” tehát nem egyszerű hardverparaméter, hanem az operációs rendszer ütemezési politikáját is meghatározó erőforrás.

14.4.3 3. SSD: nincs seek, de van törlés és szemétgyűjtés

Az SSD NAND flash lapkákat használ. A flash oldalakból (page) áll, a lapkák blokkokba szerveződnek. Az olvasás és az írás jellemzően oldalegységben történik, de a **törlés csak blokkegységben** lehetséges. Ez a félvezetős tároló egyik legfontosabb aszimmetriája:

- Olvasni szabadon, kis egységenként lehet.
- Üres oldalra írni lehet.
- Egy már megírt oldalt felülírni közvetlenül **nem** lehet; előbb az egész blokkot törölni kell.

Ezt a korlátozást a flash-transzlációs réteg (FTL) rejti el. Az FTL logikai blokkcímet (LBA) fizikai NAND-címre képez, és felülíráskor új fizikai oldalra ír, a régit pedig érvénytelennek jelöli. Az érvénytelen oldalak felszabadításáért a háttérszemétgyűjtés (garbage collection) felel. A folyamat nagyban magyarázza az SSD-k viselkedését:

- A véletlen kis írások látszólag egyszerűek, de a háttérben extra olvasást, másolást és törlést okoznak.
- A teljesítmény időben ingadozhat, mert a szemétgyűjtés csúcsokat generál.
- A NAND cellák véges számú programozási/törlési ciklust bírnak, ezért a felesleges fizikai írás nemcsak sebesség-, hanem élettartam-kérdés is.

A következő jellemző származik ebből: **write amplification (írási erősítés)**. Az írási erősítési tényezőt így definiáljuk:

$$WAF = \frac{\text{fizikai NAND-írások mennyisége}}{\text{host logikai írások mennyisége}}.$$

Ha a host 1 GiB adatot ír, de a firmware összesen 2,5 GiB fizikai írást hajt végre, akkor $WAF = 2{,}5$. A WAF soha nem kisebb 1-nél, és értéke elsősorban a munkaterheléstől, az overprovisioning aránytól, a TRIM támogatástól és a vezérlőtől függ.

Példa. Egy SSD 4 KiB oldalakkal és 2 MiB-os törlési blokkokkal dolgozik. Ha a host véletlenszerűen, kis írásokkal folyamatosan felülír egy 10 GiB-os tartományt, akkor a szemétgyűjtőnek gyakran kell olyan blokkokat felszabadítania, amelyekben még sok érvényes oldal van. Ilyenkor a WAF jellemzően 2–8 között is lehet, míg nagy szekvenciális írásnál 1,05–1,2 közelében marad. A pontos érték mérést és gyártói modellt igényel; a fenti tartomány szemléltető jellegű.

14.4.4 SSD teljesítményjellemzők

Egy SATA SSD tipikus, ellenőrizendő példaértékei:

- szekvenciális olvasás: 500–560 MiB/s,
- szekvenciális írás: 400–520 MiB/s,
- véletlen 4 KiB olvasás: 80 000–100 000 IOPS,
- véletlen 4 KiB írás: 30 000–90 000 IOPS,
- átlagos olvasási késleltetés: 80–120 µs.

Ezek az értékek nagyságrendekkel jobbak, mint egy HDD-nél, de az írási munkaterhelésben a teljesítmény nem állandó. Folyamatos véletlen írásnál a szemétgyűjtés időnként visszafogja az IOPS-t, és a faroklatenciák akár több tíz milliszekundumra is nőhetnek.

Az SSD-k élettartama durván a következőképpen becsülhető:

$$L_{\text{day}} = \frac{C_{\text{NAND}} \cdot PE}{W_{\text{host}} \cdot WAF},$$

ahol C_{NAND} a NAND-kapacitás, PE a cellánkénti programozási/törlési ciklusszám, W_{host} pedig a napi hostírás. Ha például egy 1 TiB-os meghajtó átlagosan 3000 PE ciklust bír, és naponta 200 GiB hostírást kap $WAF = 3$ mellett, akkor:

$$L_{\text{day}} = \frac{1024 \cdot 3000}{200 \cdot 3} \approx 5120 \text{ nap},$$

ami nagyjából 14 év. A PE ciklus technológiánként változik, ezért az ilyen számítás mindig csak modell, nem gyártói garancia.

A TRIM/DISCARD parancs lehetővé teszi, hogy a fájlrendszer értesítse az SSD-t a már nem használt logikai blokkokról. A blokkréteg ezt a `blkdiscard` vagy `fstrim` művelettel továbbítja. A TRIM csökkenti a felesleges szemétgyűjtést, növeli a tartós teljesítményt és csökkenti a WAF-t. Emulált QEMU-lemezen is engedélyezhető a discard, ha a virtuális lemezkép és a parancssor támogatja; éles rendszeren a valódi lemezpartíciót tilos ilyen művelettel eltalálni.

14.4.5 4. NVMe: a parancssor párhuzamosítása

Az NVMe (Non-Volatile Memory Express) protokoll a PCIe-alapú flash-tárolókhoz készült. Míg a SATA parancssor egyetlen várakozási sort használ, az NVMe akár 65 535 I/O-sort is megenged, és a kéréseket közvetlenül a PCIe-címzett parancsleírókkal viszi át. A kernelben a blokkréteg több hardversort lát, így többmagos rendszeren az I/O-küldés párhuzamosítható.

A tipikus belépő NVMe meghajtók példaértékei:

- szekvenciális olvasás: 3 000–7 000 MiB/s,
- szekvenciális írás: 2 000–5 000 MiB/s,
- véletlen 4 KiB olvasás: 300 000–1 000 000 IOPS,
- véletlen 4 KiB írás: 200 000–700 000 IOPS,
- átlagos olvasási késleltetés: 20–80 µs.

A nagyobb sávszélesség és IOPS csak akkor jelenik meg, ha a szoftver elegendő párhuzamos kérést indít. Egyetlen szálról, egyszerű `read()`-del küldözgetett 4 KiB-os kéréseknél a mért IOPS töredéke lehet a gyártói adatlapban szereplő értéknek. A gyakorlatban a `fio` vagy más benchmark eszközök `iodepth` és `numjobs` paramétereivel állítható be a párhuzamosság.

A kernel felől az NVMe meghajtó általában `/dev/nvme0n1` néven jelenik meg. A `/sys/block/nvme0n1/queue/` alatt látható a sor mérete, a maximális szegmensméret és a hardversorok száma. A pontos értékek kernelverziótól és meghajtótól függnek, ezért a reprodukálhatósághoz rögzíteni kell a vendégrendszer verzióját is.

14.4.6 5. Seek, IOPS és sávszélesség egymásra vetítése

A tárolóeszközök teljesítményét csak a három mennyiség együttes megadásával lehet értelmezni. Az alábbi, szándékosan leegyszerűsített modell segít a becslésben.

Legyen T_{fix} a kérésenkénti fix költség, B az átviteli sebesség, S a kérés mérete. Ekkor egyetlen kérés ideje:

$$T_{\text{io}} = T_{\text{fix}} + \frac{S}{B}.$$

A másodpercenkénti műveletszám:

$$IOPS = \frac{1}{T_{\text{io}}} = \frac{1}{T_{\text{fix}} + S/B}.$$

A sávszélesség:

$$BW = IOPS \cdot S.$$

Kis S esetén a nevezőben a fix költség dominál, ezért a meghajtó IOPS-korlátos. Nagy S esetén az átviteli idő dominál, és a rendszer sávszélesség-korlátossá válik. A kettő közötti átmenet az eszköz típusától függ.

HDD-példa. Ha $T_{\text{fix}} \approx 13{,}5\,\text{ms}$ és $B \approx 200\,\text{MiB/s}$, akkor 4 KiB-os írásnál:

$$T_{\text{io}} = 0{,}0135 + \frac{4096}{200 \cdot 2^{20}} \approx 0{,}0135 + 0{,}0000195 \approx 13{,}52\,\text{ms}.$$

Ez $IOPS \approx 74$, a sávszélesség pedig $74 \cdot 4\,\text{KiB} \approx 296\,\text{KiB/s}$. Egy 1 MiB-os szekvenciális írásnál:

$$T_{\text{io}} = 0{,}0135 + \frac{1}{200} \approx 0{,}0185\,\text{s},$$

ami $IOPS \approx 54$, a sávszélesség pedig 54 MiB/s. A sávszélesség javul, de a mechanikus pozicionálás miatt még mindig elmarad a nyers átviteli sebességtől.

SSD-példa. Ugyanez egy NVMe eszközön $T_{\text{fix}} \approx 20\,\mu\text{s}$ és $B \approx 3000\,\text{MiB/s}$ mellett. 4 KiB-os véletlen olvasásnál:

$$T_{\text{io}} = 20 \cdot 10^{-6} + \frac{4096}{3000 \cdot 2^{20}} \approx 21{,}3\,\mu\text{s},$$

amiből $IOPS \approx 47\,000$, a sávszélesség pedig $47\,000 \cdot 4\,\text{KiB} \approx 183\,\text{MiB/s}$. A gyakorlatban a párhuzamos sorokkal ennél sokkal több érhető el, mert a fix késleltetések átfedésbe kerülnek.

A mérésnél ezért nem elég egyetlen IOPS- vagy MiB/s-számot megadni. Rögzíteni kell legalább:

- a blokkméretet,
- a hozzáférési mintát (szekvenciális vagy véletlen),
- az írás/olvasás arányt,
- a párhuzamos kérések számát (`iodepth`, `numjobs`),
- a `direct` vagy pufferezett módot,
- az eszköz modelljét vagy az emulált konfigurációt.

14.4.7 6. Buffering: page cache és writeback

A közvetlenül az eszközre menő I/O-t a kernel általában nem hajtja végre azonnal, hanem a **page cache**-be kerül. A page cache a megnyitott fájlok tartalmát lapokban tárolja. Olvasáskor a kernel először a cache-ben keres, és csak hiány (cache miss) esetén indít blokk-I/O-t. Íráskor a `write()` rendszerhívás a lapokat módosítja a memóriában, majd visszatér; a tényleges kiírás később, aszinkron módon történik.

Ez a **writeback** mechanizmus óriási teljesítménynövekedést ad: a folyamat nem várja meg a lassú eszközt, az ismételt írások összeolvadhatnak, és a kernel nagyobb, szekvenciális kéréseket küldhet az eszköznek. Ugyanakkor árat fizetünk érte: a `write()` sikeres visszatérése nem jelenti azt, hogy az adat már a tárolón van.

A dirty lapokat a kernel háttérszálai, korábbi elnevezéssel `pdflush`, mai rendszereken általános munkaszálakba kihelyezett `writeback` feladatok írják ki. A kiíratást több

paraméter vezérli. A `/proc/meminfo` mutatja a `Dirty` és a `Writeback` lapok méretét. A `vm.dirty_background_ratio`, `vm.dirty_ratio`, `vm.dirty_expire_centisecs` és `vm.dirty_writeback_centisecs` sysctl értékek állítják a küszöböket.

Például ha a `vm.dirty_ratio` 20 %, akkor a kernel engedi, hogy a memória legfeljebb 20 %-a dirty lap legyen; ha a rendszer eléri ezt a korlátot, a `write()` hívások lassulni kezdenek, és a kernel szinkron kiíratásra kényszerülhet. A `vm.dirty_expire_centisecs` azt adja meg, mennyi ideig maradhat egy lap dirty, mielőtt a háttér kiíratás kötelezővé válik. A pontos alapértékek disztribúciónként és verzióként eltérnek, ezért éles tuning előtt a helyi dokumentáció ellenőrzése szükséges.

14.4.8 fsync és fdatasync

A tartósságot igénylő alkalmazásnak explicit módon kell kérnie a kiíratást. Az `fsync()` rendszerhívás a fájl összes megváltozott adat- és metaadat-lapját a tárolóra kényszeríti. Az `fdatasync()` csak a fájl adataihoz és a méretváltozásból fakadó metaadatokhoz ragaszkodik, ezért gyakran kevesebb I/O-t okoz.

A következő C-példa a szemléltetést szolgálja; a `path` helyőrzőt a laborban eldobható kötetre kell állítani, éles lemezre nem:

```
#include <fcntl.h>
#include <unistd.h>
#include <stdio.h>

int main(void)
{
    const char path[] = "/mnt/scratch/example.bin";
    int fd = open(path, O_WRONLY | O_CREAT | O_TRUNC, 0600);
    if (fd < 0) {
        perror("open");
        return 1;
    }
    const char buf[4096] = {0};
    ssize_t n = write(fd, buf, sizeof buf);
    if (n < 0) {
        perror("write");
        close(fd);
        return 1;
    }
    if (fsync(fd) != 0) {
        perror("fsync");
        close(fd);
        return 1;
    }
    if (close(fd) != 0) {
        perror("close");
        return 1;
    }
    return 0;
}
```

Az `fsync()` hívás után a kernel megvárja, hogy a blokkréteg, a vezérlő és az eszköz befejezze a kiírást. Az eszközön belül – különösen olcsó SSD-knél – a belső írási gyorsítótár tovább bonyolíthatja a képet. Az `fsync()` a Linux által kiadott `SYNCHRONIZE CACHE` vagy `FLUSH` parancsot is kikényszeríti, ha a blokkréteg és az eszköz támogatja.

A gyakorlatban két hibát követnek el leggyakrabban:

- A `write()` sikeres visszatérését tartósnak tekintik, és `fsync()` nélkül leállítják a rendszert. Kieséskor az adat hiányozhat, vagy a fájl hossza és tartalma inkonzisztens lehet.
- Minden írás után `fsync()`-et hívnak, mert „biztosra mennek”. Ez a writeback előnyeit rontja le, és a teljesítményt a nyers eszközsebességre szorítja.

A helyes megoldás a tranzakcióhatárok azonosítása: az alkalmazás naplózza, hogy mely műveleteknek kell túlélniük a crash-t, és csak ott kér `fsync()`-et, ahol a tartóssági követelmény valóban fennáll.

Az `O_DIRECT` megnyitási flag megkerüli a page cache-t, és közvetlenül az eszközre irányítja az I/O-t. Ezt adatbázis-kezelők használják, amikor saját gyorsítótárral rendelkeznek, és nem akarják a kernel dirty-lapjait duplikálni. Az `O_DIRECT` azonban szigorú igazítási követelményeket támaszt a puffercímre, a hosszra és a fájlpozícióra, emellett elveszíti a writeback egyesítő előnyeit. Általában nem általános gyorsítóeszköz, hanem speciális munkaterhelések eszköze.

14.4.9 7. Hogyan lesz a buffering hatása mérhető?

A page cache és a writeback hatása ugyanazon eszközön, ugyanazzal a munkaterheléssel szépen elkülöníthető. Ha a mérés `--direct=0` értékkel fut, a kernel először a page cache-be ír, és a mért sávszélesség sokkal nagyobb lehet, mint az eszköz valódi sebessége. Ha `--direct=1` vagy `O_DIRECT` szerepel, a mérés a tényleges eszközutat terheli. A kettő közötti különbség nem az eszköz gyorsulása, hanem a pufferezés hatása.

A következő, emulált környezetben futtatható `fio` parancs a véletlen 4 KiB olvasást méri úgy, hogy a page cache ne torzítsa a lemezes adatutat:

```
sudo fio --name=randread-direct \
    --filename=/dev/vdX \
    --rw=randread \
    --bs=4k \
    --size=1G \
    --numjobs=1 \
    --iodepth=16 \
    --ioengine=libaio \
    --direct=1 \
    --runtime=30 \
    --time_based
```

A `/dev/vdX` helyőrző; a laborban kizárólag az emulált vendég eldobható eszközét szabad megadni. A méréshez rögzíteni kell a QEMU-konfigurációt, a vendég kernelverzióját, a meghajtó típusát (`virtio-blk`, `nvme`, `ide` stb.) és az `fio` verzióját. A várt érték csak ezen konfiguráció alapján becsülhető; a valós gyártói adatlap értékei nem vihetők át automatikusan egy virtualizált eszközre.

A bufferelt írás mérésére például egy `fsync` gyakoriságot ellenőrző futtatás alkalmas:

```
sudo fio --name=write-fsync \
    --filename=/mnt/scratch/testfile \
    --rw=write \
    --bs=4k \
    --size=200M \
    --numjobs=1 \
    --ioengine=sync \
    --direct=0 \
    --fsync=1 \
    --runtime=20
```

Ezzel az a kérdés vizsgálható, hogy milyen IOPS érhető el, ha minden írás után tartóssági kényszer teljesül. A nem pufferezett, `direct=1` párja ettől általában eltérő IOPS-t és faroklatenciát ad. A mérési naplóban mindkét esetben fel kell tüntetni a ciklusszámot, a futási időt, a cache-találati arányt és az átlagos/tail latenciákat; egyetlen IOPS-szám nem elegendő a fejezetben tárgyalt jelenségek elkülönítésére.

14.4.10 8. Labor: tárolói teljesítmény- és élettartam-modell

A fejezet laborjának célja, hogy a tanuló egy reprodukálható modellt készítsen egy emulált tárolóeszköz teljesítményének és írási erősítésének becslésére. A labor **nem** érint valós lemezt, bootloadert vagy firmware-t. A munkakönyvtárnak eldobhatónak kell lennie, például `/tmp/os053-lab` alatt.

14.4.11 8.1. Kiinduló állapot

A laborhoz szükséges:

- QEMU-alapú vendéggép Linuxszal,
- 512 MiB RAM,
- 2 vCPU,
- 2 GiB nyers `raw` lemezkép,
- `virtio-blk` vagy emulált `nvme` csatoló,
- a vendégben `fio`, `blkdiscard` és `sysstat` vagy `iostat`.

A pontos verziókat a reprodukciós csomagban rögzíteni kell. A `virtio-blk` és a `nvme` emuláció paraméterei QEMU-verziónként változhatnak, ezért a „várt” értékek csak ehhez a beállításhoz köthetők.

14.4.12 8.2. Létrehozás és indítás

```
mkdir -p /tmp/os053-lab
qemu-img create -f raw /tmp/os053-lab/disk.img 2G

qemu-system-x86_64 \
    -m 512 \
    -smp 2 \
    -drive file=/tmp/os053-lab/disk.img,format=raw,if=virtio \
    -boot d \
    -cdrom /tmp/os053-lab/install.iso \
    -nographic
```

A telepítő ISO helyőrző; a laborhoz saját, ismert forrásból származó telepítőlemezt kell használni. A lemezkezelést kizárólag a vendégen belül, a `/dev/vdX` helyőrzővel szabad végezni, és minden mérés előtt ellenőrizni kell, hogy a cél tényleg az eldobható virtuális eszköz.

14.4.13 8.3. Mérési terv

A következő négypontos mérést érdemes elvégezni:

1. **Szekvenciális olvasás:** `rw=read`, `bs=1M`, `iodepth=8`.
2. **Véletlen 4 KiB olvasás:** `rw=randread`, `bs=4k`, `iodepth=16`.
3. **Szekvenciális írás:** `rw=write`, `bs=1M`, `iodepth=8`.

4. **Véletlen 4 KiB írás:** `rw=randwrite`, `bs=4k`, `iodepth=16`.

Mindegyiknél rögzíteni kell a `--direct=1` és a `--direct=0` esetet is, hogy a page cache hatása elváljon az eszközút teljesítményétől. A mérési napló sémája:

Paraméter	Érték
QEMU verzió	`x.y.z`
vendég kernel	`x.y.z`
lemezkép	`raw, 2 GiB`
csatoló	`virtio-blk`
munkaterhelés	`randread 4k`
`iodepth`	`16`
`direct`	`1`
futási idő	`30 s`
várt IOPS	levezetett érték
mért IOPS	mért érték
várt átlagos latencia	levezetett érték
mért p50/p99	mért érték

A „levezetett érték” a fejezet képleteiből, a médium ismert paramétereiből számolható. Az emulált környezetben a mért érték általában eltér; az eltérés okát röviden dokumentálni kell.

14.4.14 8.4. Írási erősítés és élettartam-modell

Az élettartam-modell pszeudokódja így foglalható össze:

```
# adott: napi hostírás GiB, kapacitás GiB, P/E ciklus,
#        overprovisioning arány, véletlen írási arány
# eredmény: WAF becslés és élettartam napokban

function ssd_lifetime(host_write_gib, capacity_gib, pe_cycles, op_ratio, random_ratio):
    if random_ratio > 0.8:
        waf = 2.5 + (1.0 - op_ratio) * 2.0
    else:
        waf = 1.05
    lifetime_days = (capacity_gib * pe_cycles) /
                    (host_write_gib * waf)
    return waf, lifetime_days
```

A képletben szereplő `waf` modellérték, nem mérésből származó állandó. A pontos WAF csak a firmware, a munkaterhelés és az eszköz állapota ismeretében becsülhető, ezért a labor jegyzetfüzetének ezt egyértelműen jelölnie kell.

A laborhoz tartozó jegyzetfüzet a következőket tartalmazza:

- a mérési parancsokat,
- a nyers `fio` kimeneteket,
- a fenti táblázatot,
- a WAF-modellt és a számítást,
- a „várt” és „mért” értékek elkülönítését,
- az eltérések magyarázatát.

14.4.15 8.5. Visszaállítás

A labor befejezése után a lemezkép törölhető:

```
rm -rf /tmp/os053-lab
```

A vendéggép leállítása előtt meg kell győződni arról, hogy a mérési napló a hosztra mentődött. A lemezkép eldobható, ezért semmilyen valós lemezpartíciót nem érinthet.

14.4.16 9. Gyakori hibák és azok felismerése

1. **A gyártói sávszélességet IOPS-nak olvassák.** A 7 000 MiB/s szekvenciális olvasás nem jelenti azt, hogy a 4 KiB-os véletlen IOPS is hatalmas. A kettő más erőforrást terhel.
2. **A page cache-t nem veszik figyelembe.** Ha egy `dd` vagy `fio` teszt `direct=0`-val fut, a mért érték a RAM sebességét is tükrözheti, nem az eszközét.
3. **A `write()` visszatérést tartósnak tekintik.** Tartóssághoz `fsync()`/`fdatasync()` kell, vagy az alkalmazásnak `O_SYNC`/`O_DSYNC` szemantikát kell használnia.
4. **Minden írásnál `fsync()`-et hívnak.** A teljesítmény összeomlik, pedig csak tranzakcióhatárokon lenne szükség a hívásra.
5. **A WAF-t figyelmen kívül hagyják az SSD élettartam-tervezésénél.** A host írási forgalom nem egyenlő a NAND kopásával; a szemétgyűjtés többszörözheti.
6. **A TRIM-ot nem engedélyezik.** A fájlrendszer törléseit az SSD nem tudja meg, ezért feleslegesen cipel érvénytelen adatokat.
7. **Az NVMe-t egyetlen szállal, alacsony `iodepth`-tel mérik.** A modern PCIe-eszköz csak párhuzamos kérésekkel mutatja meg a valódi képességeit.

14.4.17 10. Önellenőrző kérdés

Egy 7200 ford/perc HDD átlagos seek ideje 9 ms, átlagos forgási késleltetése 4,2 ms, átviteli és várakozási ideje együtt 0,3 ms. Egyetlen véletlenszerű 4 KiB olvasás hány ms-ot vesz igénybe? Mekkora a maximális IOPS? Ha egy alkalmazás 1000 ilyen olvasást hajt végre, mennyi a minimális elvi futási idő? Válaszát indokolja, és nevezze meg, melyik hardverjellemző dominál ebben az esetben.

A várható válasz: $T_{\text{io}} = 9 + 4{,}2 + 0{,}3 = 13{,}5$ ms; $IOPS \approx 1/0{,}0135 \approx 74$; 1000 olvasás minimum $1000 \cdot 13{,}5$ ms $= 13{,}5$ s, ha nincs párhuzamosítás. A seek és a forgási késleltetés dominál.

14.4.18 11. Összefoglalás

A tárolóeszközök teljesítménye nem írható le egyetlen számmal. A HDD-t a mechanikus pozicionálás, az SSD-t a flash törlési blokkja és a szemétgyűjtés, az NVMe-t a párhuzamos parancssor és a szoftveres kihasználtság jellemzi. A buffering – elsősorban a page cache és a writeback – nagyban javítja a látens írási teljesítményt, de a tartósságért cserébe explicit `fsync()`/`fdatasync()` hívásra van szükség. A laborban a tanuló ezeket a jelenségeket emulált környezetben méri, a várt és mért értékeket elkülöníti, és egy SSD-élettartam-modellben számol a write amplification hatásával.

IV. RÉSZ – Fájlrendszerek és tárolás

15 Fájlrendszer-alapok

15.1 Fájlok, könyvtárak és metaadatok

A tárolóeszközök – például egy NVMe-meghajtó vagy egy loopback fájl – a blokkos I/O szintjén csupán lineáris bájtsorozatot, pontosabban blokkok halmazát kínálják. A fájlrendszer az a réteg, amely ebből a nyers tárból nevekkel ellátott, tartós objektumokat, azaz fájlokat épít. Ebben az alfejezetben a fájl mint absztrakció fogalmát és a hozzá tartozó metaadatok szerepét tárgyaljuk, majd a fájlnév, az inode, a könyvtár és a hard link kapcsolatát, végül a szimbolikus link és a jogosultsági modell alapjait. A fejezet a Unix-szerű fájlrendszerek modelljére támaszkodik, mert ez a legelterjedtebb és a legjobban dokumentált, de az alapelvek más rendszerekre is érvényesek.

15.1.1 A fájl mint absztrakció

Az operációs rendszer a felhasználó és a programok elől elrejti a tároló fizikai részleteit. A fájl egy logikai egység, amely hosszú távú, nem felejtő tárolást biztosít. A legtöbb rendszerben a fájl egy bájtsorozat: az alkalmazás szekvenciálisan vagy véletlenszerűen olvashat és írhat bájtokat, anélkül hogy tudnia kellene, melyik blokkon, milyen elrendezésben vannak az adatok. Ez az absztrakció két fontos következménnyel jár.

Először is, a fájl tartalma és a fájl azonosítása (neve) elválik egymástól. A fájlnevet a felhasználó látja, de a fájlrendszer belül egy numerikus azonosítóval, a Unix-szerű rendszerekben az inode-számmal hivatkozik a fájlra. Másodszor, a fájl tartalma és a fájlról szóló leíró adatok – a metaadatok – szintén elkülönülnek. A metaadatok nem a fájl tartalmának részei, hanem a fájlrendszer által nyilvántartott jellemzők: a fájl típusa, mérete, tulajdonosa, jogosultságai, időbélyegei és a tartalom blokkjaira mutató hivatkozások.

A fájl tehát nem csupán adat, hanem egy háromkomponensű kapcsolat: - **fájlnév**: a felhasználó számára értelmes név, - **metaadatok**: a fájl jellemzői, - **adatok**: a tényleges bájttartalom.

A fájlrendszer feladata, hogy ezt a hármast konzisztensen kezelje, és a név alapján gyorsan elérhetővé tegye a metaadatokat és az adatokat.

15.1.2 A metaadatok szerepe

A metaadatok a fájlrendszer „nyilvántartási kartonjai”. A Unix-szerű rendszerekben ezt a szerepet az **inode** (index node) tölti be. Az inode egy rögzített méretű struktúra, amely a fájl összes fontos jellemzőjét tárolja, kivéve a fájlnevet. A fájlnév ugyanis a könyvtárban él, nem az inode-ban.

Az inode tipikus mezői:

Mező	Jelentés
Típus	reguláris fájl, könyvtár, szimbolikus link, eszköz, FIFO, socket
Jogosultságok	tulajdonosi, csoport- és egyéb jogosultsági bitek

Mező	Jelentés
Tulajdonos	felhasználói azonosító (UID) és csoportazonosító (GID)
Méret	a fájl mérete bájtban
Link count	hány név mutat erre az inode-ra
Időbélyegek	utolsó hozzáférés (atime), módosítás (mtime), inode-változás (ctime)
Blokkmutatók	hol találhatók a fájl tartalmának blokkjai a tárolón

Egy egyszerűsített, szemléltető inode-struktúra C nyelven így nézhet ki:

```
/* Szemléltető, egyszerűsített inode-struktúra - nem futtatandó */
struct on_disk_inode {
    uint16_t i_mode;      /* fájltípus és jogosultság */
    uint32_t i_uid;       /* tulajdonos UID */
    uint32_t i_gid;       /* tulajdonos GID */
    uint64_t i_size;      /* méret bájtban */
    uint32_t i_links;     /* hard link count */
    uint32_t i_atime;     /* utolsó hozzáférés (epoch óta másodpercben) */
    uint32_t i_mtime;     /* utolsó tartalom-módosítás */
    uint32_t i_ctime;     /* utolsó inode-változás */
    uint32_t i_blocks[12];/* közvetlen blokkmutatók példája */
};
```

A valós fájlrendszerek természetesen ennél összetettebbek, de ez a modell jól mutatja a lényeget: az inode a fájl minden olyan jellemzőjét tartalmazza, amely nem maga a fájl tartalma. A fájlrendszer a tárolón külön területen, az inode-táblában tartja nyilván ezeket a struktúrákat, és minden inode-hoz egy sorszámot rendel. Ez az inode-szám a fájl egyedi azonosítója az adott fájlrendszeren belül.

15.1.3 Fájlnév, inode, könyvtár és hard link

A felhasználó ritkán találkozik inode-számmal; ő a fájlnevet használja. A fájlnév és az inode közötti kapcsolatot a **könyvtár** hozza létre. A könyvtár maga is egy fájl, de speciális típusú: a tartalma nem tetszőleges bájtfolyam, hanem a fájlrendszer által értelmezett **bejegyzések** listája. Minden bejegyzés legalább két mezőt tartalmaz: egy fájlnevet és a hozzá tartozó inode-számot.

Egy egyszerű könyvtárbejegyzés így modellezhető:

```
/* Szemléltető könyvtárbejegyzés: név -> inode leképezés */
struct dir_entry {
    uint32_t d_ino;       /* inode-szám */
    char     d_name[256]; /* fájlnév */
};
```

Amikor a felhasználó egy fájlt nyit meg, például a `/home/alice/file.txt` elérési úton, a fájlrendszer a gyökérkönyvtártól kezdve végigjárja a könyvtárakat, minden lépésben kikeresi a megfelelő nevet a könyvtár bejegyzései közül, megkapja az inode-számot, beolvassa az inode-ot, és az ott található blokkmutatók alapján eléri a fájl tartalmát.

A könyvtár tehát a **név és az inode közötti hozzárendelés**. Ez a modell két fontos következménnyel jár.

Egyrészt ugyanahhoz az inode-hoz több név is tartozhat. Ha a fájlrendszer létrehoz egy új könyvtárbejegyzést, amely ugyanarra az inode-számra mutat, akkor **hard link** jön létre. A hard

link nem másolat: a két név ugyanazt a fájlt jelenti, ugyanazzal a tartalommal és ugyanazokkal a metaadatokkal. Az inode `i_links` mezője számlálja, hány könyvtárbejegyzés mutat rá.

Például a következő shell-parancs egy új hard linket hoz létre:

```
# Laborpélda: hard link létrehozása
ln /mnt/labfs/file1 /mnt/labfs/hardlink1
```

Ezután a `file1` és a `hardlink1` név ugyanarra az inode-ra mutat. A `ls -li` parancs kimenetében az első oszlop az inode-szám, a harmadik a link count. Ha a fájl eredeti link countja 1 volt, a hard link létrehozása után 2 lesz.

Másrészt a fájlnév nem a fájl része, hanem a könyvtár bejegyzése. Ezért a fájl átnevezése nem érinti az inode-ot, csak a könyvtárbejegyzést módosítja. Ha egy hard linket törlünk, a link count eggyel csökken; az inode és a tartalom csak akkor szabadul fel, ha a link count eléri a nullát. Ezért a hard linkek csak ugyanazon a fájlrendszeren belül létezhetnek, mert az inode-szám csak ott egyedi.

15.1.4 Szimbolikus link

A **szimbolikus link** (vagy symlink) merőben más mechanizmus. A szimbolikus link egy speciális típusú fájl, amelynek a tartalma egy elérési út – azaz egy karaktersorozat, amely egy másik fájlra vagy könyvtárra mutat. A szimbolikus link saját inode-dal rendelkezik, saját jogosultságokkal és időbélyegekkel, és a mérete általában a cél elérési út hosszának felel meg.

Szimbolikus link létrehozása:

```
# Laborpélda: szimbolikus link létrehozása
ln -s file1 /mnt/labfs/symlink1
```

Itt a `symlink1` egy külön inode, amelynek a tartalma a `"file1"` szöveg. Amikor egy program megnyitja a `symlink1` elérési utat, a fájlrendszer felismeri, hogy szimbolikus linkről van szó, beolvassa a link tartalmát, és a benne tárolt útvonalat kezdi el feloldani. Ha a célfájl létezik, a művelet a célra vonatkozik; ha nem, a link „törött” (dangling).

A hard link és a szimbolikus link közötti alapvető különbség: - A hard link ugyanarra az inode-ra mutat; a link count növekszik. - A szimbolikus link egy külön inode, amelynek a tartalma egy útvonal; a cél inode link countja nem változik.

Ebből következik, hogy a hard link nem mutathat könyvtárra (a ciklusok elkerülése miatt a legtöbb rendszerben), míg a szimbolikus link igen. Továbbá a hard link a célfájl áthelyezésekor is érvényben marad, mert az inode-ra mutat; a szimbolikus link viszont elavul, ha a cél elérési útja megváltozik.

15.1.5 Jogosultságok

A fájlrendszer a metaadatok között tárolja a **hozzáférési jogosultságokat**. A Unix-modellben minden inode-nak van egy tulajdonosa (UID) és egy csoportja (GID). A jogosultsági bitek három csoportra vonatkoznak: - **tulajdonos (user, u)**, - **csoport (group, g)**, - **mindenki más (other, o)**.

Mindhárom csoportra három bit vonatkozik: - **r** – olvasás (read), - **w** – írás (write), - **x** – végrehajtás (execute), könyvtár esetén bejárás (traverse).

A biteket hagyományosan oktális formában is megadják. Például a `0644` érték azt jelenti, hogy a tulajdonos olvashat és írhat (`rw-`), a csoport és a többiek csak olvashatnak (`r--`). A `0755` a futtatható programoknál gyakori: tulajdonos `rwx`, csoport `r-x`, egyéb `r-x`.

A fájl típusát és a jogosultságokat az inode `i_mode` mezője tárolja. A `stat` parancs megjeleníti a teljes értéket:

```
# Laborpélda: egy fájl metaadatainak lekérdezése
stat /mnt/labfs/file1
```

A tipikus kimenet tartalmazza az inode-számot, a link countot, a jogosultságokat, a UID-t és GID-t, a méretet, valamint az atime, mtime és ctime értékeket.

Három speciális bit is tartozik a jogosultsági modellhez: - **setuid (suid)**: ha egy futtatható fájlon ez a bit be van állítva, a program a fájl tulajdonosának jogosultságaival fut, nem a futtató felhasználóéval. Ez biztonsági szempontból kényes, ezért csak indokolt esetben használják. - **setgid (sgid)**: hasonló, de a csoportra vonatkozik; könyvtár esetén az újonnan létrehozott fájlok a könyvtár csoportját öröklik. - **sticky bit**: könyvtárakon azt jelenti, hogy a benne lévő fájlokat csak a tulajdonos (vagy a könyvtár tulajdonosa, vagy root) törölheti, még akkor is, ha a könyvtár írható mások számára. Tipikus példa a `/tmp`.

A jogosultságok bővíthetők **ACL**-ekkel (Access Control Lists), amelyek további felhasználókhoz és csoportokhoz rendelhetnek jogokat. A klasszikus Unix-modell azonban a fenti három csoportra épül.

Fontos megérteni a **könyvtár** jogosultságainak sajátos értelmezését. Egy könyvtár esetén: - `r` – a könyvtár bejegyzéseinek listázása, - `w` – új bejegyzések létrehozása és meglévők törlése, - `x` – a könyvtáron való áthaladás, azaz a benne lévő elérési út bejárása.

Ezért például a `r--` jogosultságú könyvtárban a felhasználó listázhatja a fájlneveket, de nem tud belépni a könyvtárba, tehát nem érheti el a fájlok tartalmát. Az `--x` jogosultságú könyvtárban a felhasználó nem listázhat, de ha ismeri a pontos fájlnevet, elérheti a fájlt.

15.1.6 Tipikus hibák és helyes értelmezés

A fájlrendszer-névtér és metaadat-modell kapcsán gyakori félreértések:

1. **„A fájlnév azonos a fájllal."** Helytelen. A fájl egy inode-hoz kötött bájttartalom; a fájlnév csak egy könyvtárbejegyzés, amely erre az inode-ra mutat. A fájl átnevezése nem hoz létre új fájlt, és a hard linkek azt mutatják, hogy egy inode-hoz több név is tartozhat.
2. **„A hard link másolat."** Helytelen. A hard link ugyanarra az inode-ra mutat; ha az egyik néven keresztül módosítjuk a fájlt, a másik néven keresztül is látszik a változás. A másolat külön inode-ot és külön adatblokkokat jelentene.
3. **„A szimbolikus link ugyanaz, mint a hard link."** Helytelen. A szimbolikus link egy külön inode, amelynek a tartalma egy útvonal. Ha a célfájlt törlik, a szimbolikus link megmarad, de törötté válik.
4. **„A könyvtár `x` bitje a végrehajtást jelenti."** A könyvtárra nem értelmezhető a végrehajtás; az `x` bit a bejárást engedélyezi. A listázáshoz `r`, a létrehozás/törléshez `w` kell, de a tényleges hozzáféréshez mindig szükséges az `x` is.
5. **„A link count csak a hard linkek száma."** Igaz, de a könyvtárak esetében a `.` és `..` bejegyzések is növelik a link countot, ezért egy könyvtár link countja legalább 2, és minden alkönyvtár tovább növeli.

A helyes megoldás mindig a modell pontos ismeretén alapul: a név, az inode és az adatblokkok közötti kapcsolatot kell nyomon követni. A `ls -li`, a `stat`, és a `debugfs`-szerű eszközök segítenek a metaadatok közvetlen megfigyelésében.

15.1.7 Labor: fájlrendszer-névtér és metaadat-modell

A következő labor célja, hogy a fenti fogalmakat egy emulált környezetben, egy eldobható loopback fájlrendszeren ellenőrizzük. A gyakorlat nem módosítja a hoszt valódi lemezét, bootloaderét vagy rendszerpartícióját; minden művelet egy `fs.img` nevű, 64 MiB méretű képfájlra korlátozódik.

Környezet és konfiguráció

- Hoszt: tetszőleges Linux-disztribúció, kernelverzió rögzítve, például `6.1.0` (a tényleges verziót a naplóban fel kell tüntetni).
- Emuláció: a labor futtatható fizikai gépen is, de ajánlott QEMU vagy virtuális gép, hogy a loopback mount és a rendszergazdai műveletek elkülönüljenek.
- Fájlrendszer: `ext4`, a `mkfs.ext4` alapbeállításaival (a pontos verziót és kapcsolókat a naplóban rögzíteni kell).
- A mérési módszer: a `ls -li`, `stat` és `readlink` parancsok kimenetének rögzítése.

Lépések

```
# 1. Munka könyvtár létrehozása
mkdir -p ~/oslab && cd ~/oslab

# 2. 64 MiB-os üres képfájl létrehozása
truncate -s 64M fs.img

# 3. ext4 fájlrendszer formázás
mkfs.ext4 -F fs.img

# 4. Csatolási pont létrehozása és mountolás
mkdir -p mnt
sudo mount -o loop fs.img mnt

# 5. Tesztfájl létrehozása tartalommal
echo "hello filesystem" > mnt/file1

# 6. Hard link létrehozása
ln mnt/file1 mnt/hardlink1

# 7. Szimbolikus link létrehozása
ln -s file1 mnt/symlink1

# 8. Megfigyelés: inode-számok és link count
ls -li mnt

# 9. Részletes metaadatok lekérdezése
stat mnt/file1
stat mnt/hardlink1
stat mnt/symlink1
```

Várt eredmények

A `ls -li mnt` kimenetében a következőt várjuk (a pontos inode-szám és időbélyegek a futtatástól függenek, de a szerkezet azonos):

```
total 20
  12 drwxr-xr-x 3 user user 4096 Feb 14 10:00 .
  11 drwxr-xr-x 3 user user 4096 Feb 14 10:00 ..
  14 -rw-r--r-- 2 user user   18 Feb 14 10:01 file1
  14 -rw-r--r-- 2 user user   18 Feb 14 10:01 hardlink1
  15 lrwxrwxrwx 1 user user    5 Feb 14 10:01 symlink1 -> file1
```

A várt megfigyelések: - A `file1` és a `hardlink1` inode-száma azonos, és a link count mindkét esetben 2. - A `symlink1` inode-száma eltér, a típusa `l` (szimbolikus link), a link countja 1, és a mérete 5 bájt, ami a `"file1"` útvonal hosszának felel meg. - A `stat mnt/file1` kimenetében a link count 2, a `stat mnt/hardlink1` kimenetében szintén 2, a `stat mnt/symlink1` kimenetében 1.

A mérési naplóban a hallgató rögzíti a tényleges kimeneteket, és elkülöníti a mért értékeket a fenti várt értékektől. Például:

Fájl	Várt inode	Mért inode	Várt link count	Mért link count
file1	azonos a hardlink1-gyel	…	2	…
hardlink1	azonos a file1-gyel	…	2	…
symlink1	eltérő	…	1	…

Mérési szempontok

- A mérési módszer a kernel `stat` rendszerhívásán alapul, amelyet a `stat` parancs közvetít. A pontos hardver és kernelverzió a naplóban szerepel, hogy a reprodukció egyértelmű legyen.
- A várt és mért értékek közötti eltérést meg kell indokolni; például ha a `symlink1` mérete nem 5 bájt, ellenőrizni kell a cél útvonal hosszát.

Visszaállítás

```
# Lemezkép leválasztása és törlés
sudo umount mnt
cd ..
rm -rf ~/oslab
```

A labor során a hoszt valódi lemezét, bootloaderét és rendszerpartícióját nem módosítottuk.

15.1.8 Önellenőrző kérdés

Adott egy könyvtár, amelyben egy `a.txt` reguláris fájl, egy `b.txt` hard link, amely ugyanarra az inode-ra mutat, és egy `c.txt` szimbolikus link, amely az `a.txt`-re mutat. Hogyan változik a link count, ha töröljük az `a.txt` nevet? Mi történik a `c.txt` linkkel, ha a `b.txt` tartalmát módosítjuk?

Válasz: Az `a.txt` törlése a link countot 2-ről 1-re csökkenti; az inode és a tartalom megmarad, mert a `b.txt` még mutat rá. A `c.txt` szimbolikus link továbbra is az `a.txt` útvonalra mutat, de mivel az `a.txt` név már nem létezik, a link törötté válik. Ha a `b.txt` tartalmát módosítjuk, az a közös inode-ot érinti, tehát a tartalom megváltozik, de a `c.txt` link továbbra is törött, mert az elérési út `a.txt` nem létezik.

15.2 VFS, mount és fájlleírók

A virtuális fájlrendszer (VFS) réteg célja, hogy a kernel egységes felületet biztosítson a különböző fájlrendszer-implementációk fölé. Egy általános célú operációs rendszerben egyszerre lehet jelen ext4, FAT, procfs, sysfs vagy tmpfs. A folyamatok azonos rendszerhívásokkal (`open`, `read`, `write`, `stat`) érik el ezeket. A VFS ezt az absztrakciót a kernel belsejében, a rendszerhívási réteg és a konkrét fájlrendszer-meghajtó között valósítja meg. A VFS tehát nem fizikai fájlrendszer, hanem keretrendszer és objektummodell, amely a közös műveleteket szabványosítja, a fájlrendszerspecifikus részeket pedig függvénypointerek mögé rejti.

Ebben az egységben a VFS négy alapvető objektumát, a mount mechanizmust és a fájlleírók modelljét vizsgáljuk. Az itt bemutatott fogalmak a Fájlallokáció: FAT, inode és extent egységben tárgyalt fájlallokációs stratégiák alapját képezik: a fizikai blokkok elrendezésétől függetlenül a kernel ugyanazokat a VFS-műveleteket hívja.

A szövegben a Linux VFS-modelljét használjuk fogalmi referenciaként, de a cél nem egyetlen implementáció dokumentálása, hanem az általános tervezési minta. A konkrét struktúramezők és függvénypointerek kernelverziónként változhatnak; az itt közölt pszeudokód oktatási célú, és nem helyettesíti a hivatalos dokumentációt. A friss kernelverziók részleteit a forráskód és a kernel dokumentációja tartalmazza.

15.2.1 A VFS objektummodell

A VFS négy fő objektumtípussal dolgozik: **superblock**, **inode**, **dentry** és **file**. Ezek hierarchikus kapcsolatban állnak, de a kapcsolat iránya nem mindig a felhasználói névfeloldás sorrendje. A névfeloldás során a dentry lánc vezet el az inode-hoz, az inode a superblockhoz tartozik, a file pedig egy nyitott inode-hoz kötődik.

A következő pszeudokód a négy objektum minimális kapcsolatát mutatja:

```
/* Oktatási célú pszeudokód, nem kernel API. */
struct vfs_superblock {
    struct vfs_filesystem_type *fs_type;
    struct vfs_dentry *mount_root;
    struct list_head inode_list;
    uint64_t block_size;
    void *fs_private;          /* fs-specifikus privát adatok */
};

struct vfs_inode {
    struct vfs_superblock *sb;
    uint64_t ino;
    mode_t mode;
    size_t size;
    uint64_t block_count;
    const struct inode_operations *i_op;
    const struct file_operations *i_fop;
    void *private;
};

struct vfs_dentry {
    char name[NAME_MAX];
    struct vfs_inode *inode;
    struct vfs_dentry *parent;
    struct list_head children;
    unsigned flags;
};
```

```
struct vfs_file {
    struct vfs_inode *inode;
    off_t offset;
    fmode_t mode;
    const struct file_operations *f_op;
    struct vfs_dentry *path;
    void *private;
};
```

A struktúramezők csak a fogalmi modellt mutatják. A valódi kernelben a lockok, referenciaszámlálók és RCU-mechanizmusok bonyolultabbá teszik a képet.

15.2.2 Superblock

A superblock egy csatolt (mountolt) fájlrendszer példányát reprezentálja. Nem azonos a lemezen tárolt szuperblokkal, bár abból inicializálódik. Minden mountoláskor a kernel létrehoz egy `vfs_superblock` objektumot, amely a fájlrendszer típusát, a blokkméretet, a gyökér dentry-t és az adott fájlrendszer privát állapotát hordozza. Ha ugyanazt az eszközt két mount ponton csatolják, általában két külön superblock példány jön létre (hacsak a fájlrendszer nem támogatja a megosztott superblockot, például bind mount esetén).

A superblock feladata: - az adott fájlrendszer-példány állapotának tárolása; - a fájlrendszer típus műveleteinek elérése (mount, unmount, statfs); - az inode-ok és blokkok allokációjának koordinálása; - a csatolási paraméterek (read-only, sync, noatime) nyilvántartása.

A VFS-ben a superblock műveleteit a `super_operations` struktúra tartalmazza, például `alloc_inode`, `destroy_inode`, `write_super`, `sync_fs`. Az oktatási modellben elegendő egy `fs_type` pointer és egy gyökér dentry referencia.

15.2.3 Inode

Az inode a fájlrendszer szintjén azonosított fájl vagy könyvtár metaadatait hordozza, de nem tartalmazza a nevet. A név a dentry szintjén létezik. Egy inode-hoz több dentry is tartozhat, ha a fájlra több név mutat (hard link). Az inode tárolja a típust (fájl, könyvtár, szimbolikus link, eszköz), a méretet, a blokkok számát, az időbélyegeket és a jogosultságokat.

A VFS szempontjából az inode a konkrét fájlrendszer által szolgáltatott objektum. A kernel az inode-ot az inode-szám alapján azonosítja egy superblockon belül. A fájlrendszer-implementáció az `inode_operations` struktúrán keresztül adja meg a műveleteket: `lookup`, `create`, `link`, `unlink`, `mkdir`, `rename`, `readlink`, `truncate`.

Az inode-nak nem célja a fájlnév tárolása, mert a név és a hozzá tartozó inode közötti leképezést a könyvtárbejegyzések tárolják. A hard linkek miatt több név mutathat ugyanarra az inode-ra, ezért az inode referencia-számlálót (`i_count`) és link-számlálót (`i_nlink`) tart.

15.2.4 Dentry

A dentry (directory entry) a névfeloldás cache-e és a név-inode leképezés. A lemezen a könyvtárbejegyzés valójában egy rekord a könyvtárfájlban, a memóriában azonban a VFS dentry objektumot használ a gyors útvonalfeloldáshoz. A dentry fastruktúra hierarchiát alkot: minden dentry mutat a szülőre és a gyermekek listájára.

A dentry állapota lehet: - **hashed**: a dentry a globális hash táblában van, és érvényes inode-ra mutat; - **unhashed**: a dentry nincs a hash táblában, például létrehozás alatt; - **negative**: a

dentry inode-ja `NULL`; ez azt jelenti, hogy a név nem létezik, de a negatív találat cache-elve van a gyorsabb elutasításhoz; - **stale**: a dentry lemezről törölt bejegyzésre mutat, érvénytelen.

A dentry cache (dcache) célja, hogy a gyakori útvonalfeloldások ne igényeljenek lemez I/O-t. A kernel a dentry-ket referenciaszámlálóval kezeli, és a memórianyomás hatására a nem használt dentry-k felszabadíthatók. A negatív dentry-k gyorsítják a nem létező fájlok keresését, ami például a library path-ek feloldásánál fontos.

Fontos különbség: a dentry nem azonos a lemezen tárolt könyvtárbejegyzéssel. A lemezen a könyvtár adatblokkjai tartalmazzák a neveket és az inode-számokat; a dentry ezek memóriabeli gyorsítótárazott reprezentációja. A tényleges forrás a fájlrendszer implementációja által kezelt on-disk struktúra.

15.2.5 File

A file objektum egy nyitott fájl leírását reprezentálja. Amikor egy folyamat meghívja az `open` rendszerhívást, a kernel létrehoz egy `struct file` objektumot, amely az inode-hoz kötődik, de a nyitási módot és az aktuális fájlpozíciót (offset) tartalmazza. A file objektum nem azonos a fájlleíróval: a fájlleíró egy folyamatonkénti kis egész szám, amely a file objektumra mutat.

A file objektum a következőket tartalmazza: - az inode referencia; - a nyitási mód (`O_RDONLY`, `O_WRONLY`, `O_RDWR`, `O_APPEND`, `O_NONBLOCK`); - az aktuális offset; - a fájlműveletek pointerei (`read`, `write`, `ioctl`, `lseek`, `fsync`, `mmap`); - a referencia-számláló (hány fájlleíró mutat rá).

A fájlleíró és a file objektum közötti különbségtétel alapvető a helyes konkurens hozzáférés megértéséhez. Két folyamat ugyanarra az inode-ra nyithat meg két külön file objektumot, így az offset független lesz. Ha azonban egy folyamat `fork`-ol, a gyermek megosztja ugyanazt a file objektumot, így az offset közös marad. A `dup` rendszerhívás szintén a file objektum megosztását eredményezi.

15.2.6 Fájlleírók, offset és megosztott megnyitás

A felhasználói térben a nyitott fájlt egy nemnegatív egész szám, a **fájlleíró (file descriptor, fd)** azonosítja. A kernel minden folyamathoz egy fájlleíró táblát tart fenn. A tábla bejegyzései a file objektumokra mutató pointerek. A fájlleírók a folyamat erőforrásai, és a folyamat megszűnésekor automatikusan lezáródnak.

A rendszerhívások ezt a modellt használják. Például:

```
int open(const char *pathname, int flags, mode_t mode);
```

- `pathname`: a megnyitandó fájl útvonala;
- `flags`: nyitási mód és opciók (`O_RDONLY`, `O_WRONLY`, `O_RDWR`, `O_CREAT`, `O_EXCL`, `O_APPEND`, `O_TRUNC`, `O_NONBLOCK`);
- `mode`: jogosultságok, ha `O_CREAT` van beállítva;
- visszatérési érték: új fájlleíró (0), hiba esetén -1, és az `errno` beállítása.

A kernel a következő lépéseket hajtja végre: 1. **Útvonalfeloldás**: a VFS a dcache és az inode lookup segítségével megkeresi a fájl inode-ját. 2. **Hozzáférés-ellenőrzés**: a folyamat hitelesítő adatai alapján ellenőrzi, hogy a kért nyitási mód engedélyezett-e. 3. **File objektum létrehozása**: ha az inode típusa fájl, létrehoz egy `struct file`-t, beállítja az offsetet 0-ra (vagy a fájl végére, ha `O_APPEND`), és hozzárendeli a fájlműveleteket. 4. **Fájlleíró kiosztása**: a folyamat fd táblájában keres egy szabad bejegyzést (általában a legkisebb szabad számot), és oda beilleszti a file pointert. 5. **Visszatérés**: a felhasználói tér megkapja az fd-t.

A fájlleírók a folyamat szintjén érvényesek. Ugyanaz a szám más folyamatban teljesen más fájlra mutathat. A file objektum azonban kernel-szintű, és több fájlleíró is mutathat rá, akár különböző folyamatokból.

A `read` és `write` rendszerhívásoknak nincs explicit offset paraméterük. Az offset a file objektumban tárolódik, és minden `read`/`write` automatikusan növeli. Például:

```
ssize_t read(int fd, void *buf, size_t count);
```

- `fd`: fájlleíró;
- `buf`: célpuffer a felhasználói térben;
- `count`: beolvasandó bájtok száma;
- visszatérés: beolvasott bájtok száma, 0, ha EOF, -1 hiba esetén.

A `lseek` rendszerhívás módosítja az offsetet:

```
off_t lseek(int fd, off_t offset, int whence);
```

- `whence`: `SEEK_SET`, `SEEK_CUR`, `SEEK_END`;
- visszatérés: új offset, -1 hiba esetén.

A fájlleírók megosztása két fő mechanizmussal történik: `fork` és `dup`/`dup2`. Ekkor a file objektum referencia-számlálója növekszik, és az offset megosztott lesz. Ez azt jelenti, hogy ha két folyamat ugyanarra a file objektumra olvas, az egyik által végrehajtott `read` előreviszi az offsetet, így a másik a következő pozíciótól olvas. Ez gyakran meglepetést okoz a kezdőknek, mert intuitívan azt várnák, hogy minden folyamatnak független pozíciója van.

A helyes modell: - **független megnyitások**: minden `open` új file objektumot hoz létre, saját offsettel; - **megosztott file objektum**: `fork`, `dup`, `dup2`, valamint az `open` + `O_CLOEXEC`-től eltérő átadási módok (pl. Unix domain socket `SCM_RIGHTS` üzenet) ugyanarra a file objektumra mutató új fd-t adnak.

A megosztott nyitás és az offset viselkedését a következő példa szemlélteti. Tegyük fel, hogy egy folyamat megnyit egy fájlt olvasásra, majd `fork`-ot hív. A gyermek és a szülő ugyanazt a file objektumot használja. Ha a szülő beolvas 10 bájtot, az offset 10-re nő. A gyermek a `read` híváskor a 10. pozíciótól olvas, nem a 0-tól. Ha a cél az, hogy a gyermek függetlenül olvasson, akkor a gyermeknek saját `open`-t kell végrehajtania, vagy le kell zárnia az örökölt fd-t.

A fájlleírók és a file objektumok életciklusa a referenciaszámlálás szabályait követi. Egy fd lezárása (`close`) csökkenti a file objektum referenciaszámát; amikor az nulla lesz, a file objektum felszabadul. Az inode referencia-számlálója akkor csökken, amikor a file objektum megszűnik; az inode akkor szabadul fel, ha már nincs rá dentry és nincs nyitott file sem.

15.2.7 A mount mechanizmus és a névtér

A VFS a különböző fájlrendszereket egyetlen hierarchikus névtérbe integrálja a **mount** művelet segítségével. A mount egy fájlrendszer gyökerét egy már létező könyvtárhoz, a **mount ponthoz** csatolja. A mount pont alatti eredeti tartalom elérhetetlenné válik, amíg a csatolás aktív.

A kernel egy globális **mount táblát** tart fenn, amely nyilvántartja az aktív mountokat. Minden mount bejegyzés tartalmazza: - a fájlrendszer típusát; - a forrást (eszköz vagy pszeudo-fájlrendszer neve); - a mount pontot (a szülő fájlrendszerben lévő dentry); - a mountolt gyökér dentry-t; - a mount opciókat (read-only, nosuid, nodev, noexec); - a mount ID-t.

A VFS-ben a mountolt gyökér egy speciális dentry, amely a csatolt fájlrendszer gyökerére mutat. A mount pont dentry-je a szülő fájlrendszerben marad, de az útvonalfeloldás során a kernel a mount pontnál átvált a mountolt gyökérre. Ez a váltás a `vfsmount` struktúrával valósul meg.

A mountolás tipikus parancsa Linuxban:

```
mount -t ext4 /dev/sda2 /home
```

Ez a parancs a `/dev/sda2` eszközön lévő ext4 fájlrendszert a `/home` könyvtárhoz csatolja. A mount rendszerhívás:

```
int mount(const char *source, const char *target,
          const char *filesystemtype, unsigned long mountflags,
          const void *data);
```

- `source`: a csatolandó eszköz vagy pszeudo-fájlrendszer neve;
- `target`: a mount pont elérési útja;
- `filesystemtype`: a fájlrendszer típusa (pl. `"ext4"`, `"tmpfs"`, `"proc"`);
- `mountflags`: opciók (`MS_RDONLY`, `MS_NOSUID`, `MS_NODEV`, `MS_NOEXEC`, `MS_BIND`);
- `data`: fájlrendszer-specifikus opciók;
- visszatérés: 0 siker, -1 hiba.

A mountolási hierarchia lényeges tulajdonsága, hogy a mount pontok egymásba ágyazhatók. Például először csatolunk egy gyökér fájlrendszert, majd a `/home`-hoz egy másik ext4-et, majd a `/home/alice/projects`-hez egy tmpfs-t. Az útvonalfeloldás során a kernelnek minden mount pontnál ellenőriznie kell, hogy van-e alatta csatolt fájlrendszer.

A mount tábla a felhasználói térből is lekérdezhető, például a `/proc/self/mounts` fájlon keresztül. A mount propagáció (bind mount, mount namespace) ennek a modellnek a kiterjesztése, de a részletek a konténer- és namespace-fejezetbe tartoznak.

15.2.8 A fájlműveletek útja: egy `open` életciklusa

A VFS működését az `open` rendszerhívás lépésenkénti végrehajtásán keresztül érthetjük meg. A folyamat a `/home/alice/notes.txt` fájlt nyitja meg olvasásra. Tegyük fel, hogy a gyökér fájlrendszer ext4, a `/home` egy külön ext4 mount, a `/home/alice/projects` alatt pedig tmpfs van, de a `notes.txt` a `/home/alice` könyvtárban van.

1. **Path walk indítása**: a kernel a gyökér dentry-től indul. A gyökér dentry a rendszer gyökér fájlrendszerének superblockjához tartozik.
2. **Komponensenkénti feloldás**: az útvonalat `/` karakterekkel választja szét: `home`, `alice`, `notes.txt`. Minden komponensnél a dcache-ben keresi a dentry-t. Ha nincs cache-elve, meghívja a fájlrendszer `lookup` műveletét, amely a könyvtár adatblokkjaiban megkeresi a nevet, és létrehozza az inode-ot.
3. **Mount ellenőrzés**: a `home` komponens feloldása után a kernel észleli, hogy a dentry mount pont. Átvált a mountolt ext4 gyökér dentry-re, és attól a ponttól az ottani inode-okkal dolgozik.
4. **Alkotó komponens elérése**: az `alice` komponenst az új mount gyökeréből oldja fel, majd az `notes.txt`-t. Mivel a `notes.txt` fájl, a dcache-ben megtalálható vagy a `lookup` létrehozza az inode-ot.
5. **Inode hozzáférés-ellenőrzés**: a kernel ellenőrzi a jogosultságokat. Ha a folyamatnak nincs olvasási joga, `EACCES` hibával tér vissza.
6. **File objektum létrehozása**: a kernel létrehoz egy `struct file`-t, az offsetet 0-ra állítja, a `f_op` pointert az ext4 fájlműveleteire mutatja.

7. **Fájlleíró kiosztása**: a folyamat fd táblájában a legkisebb szabad bejegyzést választja (például 3, mert a 0, 1, 2 foglalt), és oda mutat a file objektum.
8. **Visszatérés**: a rendszerhívás 3-at ad vissza.

Ez a folyamat jól mutatja a VFS szerepét: a path walk és a mount váltás a VFS központi logikája, míg a fájlrendszer-specifikus `lookup`, `open`, `read` műveletek a konkrét fs-meghajtóban vannak. A dcache és az inode cache a teljesítményt szolgálják.

15.2.9 Tipikus hibák és félreértések

A VFS-sel kapcsolatos interjúkban és gyakorlati hibakeresésben gyakran előkerülő tévedések:

1. **Az inode tárolja a fájlnevet.** Ez hamis. A fájlnév a könyvtárbejegyzésben van, és a dentry cache tárolja. Az inode a fájl típusától és a könyvtárbejegyzésektől független metaadat. Hard link esetén két név mutathat ugyanarra az inode-ra.
2. **A fájlleíró offsetje a folyamathoz tartozik.** Részben igaz, de a pontos állítás: az offset a file objektumban van, és a fájlleíró a file objektumra mutat. Ha a file objektumot több fd osztja meg, az offset közös. Ezért `fork` után a szülő és gyermek megosztott offsettel rendelkezik, ami a legtöbb esetben nem kívánatos.
3. **A dentry a lemezen is létezik.** Nem. A dentry memória-objektum, a lemezen a könyvtárfájlban lévő rekordok vannak. A dentry cache gyorsítótár.
4. **A mount pont alatti tartalom törlődik.** Nem törlődik, csak elérhetetlenné válik. Az unmount után újra látható.
5. **A fájlleírók globálisak a rendszerben.** Nem, az fd-k folyamatonkénti táblában vannak. Ugyanaz a szám két folyamatban eltérő fájlt jelenthet.
6. **A `close` azonnal felszabadítja a fájlt.** A `close` csak az fd-t szünteti meg. A file objektum csak akkor szűnik meg, ha az összes rá mutató fd lezárult. Az inode csak akkor szabadulhat fel, ha nincs rá dentry és nincs nyitott file sem.
7. **A negatív dentry haszontalan.** Valójában a negatív dentry cache fontos teljesítményoptimalizálás, mert a gyakori nem létező útvonalak feloldása lemez I/O nélkül elutasítható.

A helyes megoldások mindig a referenciaszámlálási modell és a mount hierarchia pontos ismeretén alapulnak.

15.2.10 Labor: VFS-objektummodell és mount-terv készítése

A labor célja egy egyszerű VFS-modell szimulálása C nyelven, emulált környezetben, a superblock, inode, dentry, file és mount kapcsolatok megvalósításával. A feladat nem egy teljes kernel, hanem egy felhasználói térben futó oktatási szimuláció, amely a VFS objektumait és a mount mechanizmust modellezi. A labor kiinduló állapota egy üres Git repo, a build parancs `make`, a futtatás `./vfs_sim --mounts mount.conf --trace trace.log`.

15.2.11 Konfiguráció

Az emuláció rögzített konfigurációja: - CPU: a hoszt CPU-ja, de a szimuláció determinisztikus, mivel nincs valós I/O; - memória: a modell dinamikusan allokál, a méréshez `malloc` statisztikát gyűjt; - három fájlrendszer típus: `pseudo_fs` (a gyökérhez), `disk_fs` (a `/home`-hoz), `tmp_fs` (a `/home/alice/projects`-hez); - mount pontok: - `/` → `pseudo_fs` gyökér; - `/home` → `disk_fs` (eszköz: `emul0`); - `/home/alice/projects` → `tmp_fs` (memória); - a könyvtárstruktúra: - `/home/alice/notes.txt` (disk_fs); - `/home/alice/projects/README.md` (tmp_fs).

A konfigurációs fájl `mount.conf` formátuma egyszerű szöveg:

```
pseudo_fs none / rw
disk_fs emul0 /home rw
tmp_fs none /home/alice/projects rw
```

A sorrend a csatolási sorrend. A szimuláció a mount táblát ebből építi fel.

15.2.12 Megvalósítási vázlat

A szimuláció a fenti `vfs_superblock`, `vfs_inode`, `vfs_dentry`, `vfs_file` struktúrákat használja. A mount tábla egy lista:

```
/* Oktatási célú pszeudokód. */
struct vfs_mount {
    struct vfs_superblock *sb;
    struct vfs_dentry *mount_point;
    struct vfs_dentry *root;
    struct list_head mount_list;
};
```

Az `open` szimulációja a következő lépéseket hajtja végre: 1. a path walk a gyökér dentry-től indul; 2. minden komponensnél először a dcache-ben keres; 3. ha nincs találat, a `lookup` függvény létrehoz egy új dentry-t és inode-ot (a konfigurált statikus modellből); 4. mount pontnál átvált a mountolt gyökérre; 5. a fájl inode-hoz létrehoz egy file objektumot; 6. az fd táblában kioszt egy szabad számot.

A szimuláció a trace logba írja a dentry cache találatokat, a mount váltásokat, az inode allokációkat és az fd kiosztásokat.

15.2.13 Mérési terv

A labor során a következő metrikákat mérjük: - **Dentry cache találati arány**: a `lookup` hívások hány százaléka találta meg a dentry-t a cache-ben. Várt érték a második és további `open` hívásoknál 100%, az elsőnél 0%, ha a cache hideg. - **Mount váltások száma**: hány alkalommal léptünk át mount határon. A `/home/alice/notes.txt` megnyitásakor egy váltás várható (a `/home`-nál), a `/home/alice/projects/README.md` esetén kettő (a `/home`-nál és a `/home/alice/projects`-nél). - **Fájlleíró kiosztott szám**: az első `open` a 3-as fd-t adja (ha 0, 1, 2 foglalt). Várt érték 3. - **Megosztott offset viselkedés**: a szimulációban `fork` hatására a file objektum referenciaszámlálója 2 lesz, és egy `read` az egyik folyamatból az offsetet közösen mozgatja. A mért offset a második folyamatból ugyanaz, mint az első után. Várt érték: a közös offset.

A mérési napló a `trace.log` elemzéséből készül. A laborhoz tartozik egy Python szkript, amely a logból kiszámítja a metrikákat, és egy Markdown jegyzőkönyvet generál.

15.2.14 Várt és mért eredmények szétválasztása

A jegyzőkönyvben külön oszlopban kell feltüntetni a várt értéket (a fenti elemzésből) és a mért értéket (a szimuláció futásából). Példa:

Mérés	Várt	Mért
Dentry cache találat első open	0%	0%
Dentry cache találat második open	100%	100%
Mount váltások `/home/alice/notes.txt`	1	1
Mount váltások `/home/alice/projects/README.md`	2	2
Fájlleíró első open	3	3
Megosztott offset `fork` után	10	10

Ha eltérés van, a jegyzőkönyvben magyarázni kell.

15.2.15 Ellenőrző kérdések

1. Mi a különbség a dentry és a lemezen tárolt könyvtárbejegyzés között?
2. Miért osztozik a `fork`-olt folyamat a fájl offseten?
3. Hogyan találja meg a VFS a mount pont alatti fájlt?

15.2.16 Összefoglalás

A VFS réteg az operációs rendszer fájlrendszer-független magja. A superblock, inode, dentry és file objektumok együttműködése teszi lehetővé, hogy a felhasználói tér egységes fájlmodellt lásson, miközben a kernel több fájlrendszer-implementációt is kezel. A mount mechanizmus a különálló fájlrendszereket egyetlen névtérbe integrálja. A fájlleírók és a file objektumok megkülönböztetése elengedhetetlen a helyes konkurens és többfolyamatos viselkedés megértéséhez. Az itt bemutatott fogalmakat a következő egység, a Fájlallokáció: FAT, inode és extent, a fájlallokációs stratégiák (FAT, inode, extent) tárgyalásakor használja fel: a VFS műveletek mögött a konkrét fájlrendszer az inode-ok és blokkok elrendezését valósítja meg.

15.3 Fájlallokáció: FAT, inode és extent

A fájlrendszer központi feladata, hogy a felhasználó számára egy fájlt logikailag folytonos bájtsorozatként mutasson, miközben a háttértár blokkjai gyakran nem alkotnak folytonos fizikai tartományt. A fájlblokkok nyilvántartása ezért a fájlrendszer tervezésének egyik legfontosabb döntése: meghatározza, hogyan lehet egy adott logikai blokkhoz megtalálni a hozzá tartozó fizikai blokkot, milyen költséggel lehet fájlt növelni, csökkenteni, és mennyire lesz érzékeny a rendszer a töredezettségre vagy a metaadatsérülésekre.

Ebben az alfejezetben négy alapvető allokációs stratégiát vizsgálunk meg:

- **folytonos allokáció** – mint viszonyítási alap,
- **linked allocation** – láncolt blokkok,
- **FAT** – fájlallokációs tábla,
- **inode és indirekció** – indexelt csomópontok,
- **extent** – folytonos tartományok nyilvántartása.

Emellett kitérünk a ritka (sparse) fájlokra és a töredezettség fogalmára, majd egy reprodukálható laborban összehasonlítjuk a FAT- és az extent-alapú allokáció viselkedését.

15.3.1 1. A fájlblokkok nyilvántartásának követelményei

Tegyük fel, hogy a háttértárat rögzített méretű blokkokra osztjuk; a továbbiakban legyen a blokkméret $b = 4$ KiB. Egy fájl logikai címtartománya $0, 1, \ldots, L-1$ logikai blokkokból áll, ahol L a fájl méretéből és b-ből számolható. A fájlrendszernek minden logikai blokkhoz hozzá kell rendelnie egy fizikai blokkazonosítót, vagy jeleznie kell, hogy az adott blokk még nincs lefoglalva (ritka fájl esetén).

A jó nyilvántartási módszerrel szemben támasztott követelmények a következők:

1. **Gyors szekvenciális hozzáférés** – a fájl elejétől a végéig történő olvasás lehetőleg folytonos, illetve kevés metaadat-műveletet igénylő legyen.
2. **Gyors véletlen hozzáférés** – egy tetszőleges logikai blokk elérése ne igényelje az összes korábbi blokk bejárását.
3. **Hatékony növekedés és zsugorodás** – a fájlappend és a truncate művelet ne okozzon nagymértékű másolást.
4. **Kis metaadat-méret** – a blokkok nyilvántartása ne foglaljon el aránytalanul nagy helyet a hasznos adathoz képest.
5. **Megbízhatóság** – egyetlen mutató vagy bejegyzés sérülése ne tegye elérhetetlenné a teljes fájlt.
6. **Minimális töredezettség** – a fizikai blokkok lehetőleg folytonos tartományokat alkossanak, mert a lemez és a flash tárolók is jobban teljesítenek szekvenciális hozzáférés esetén.

Az alábbi módszerek ezeknek a követelményeknek más-más kompromisszumát valósítják meg.

15.3.2 2. Folytonos allokáció (kiindulópont)

A legegyszerűbb megoldás, ha egy fájlhoz egyetlen folytonos fizikai blokktartományt rendelünk. A metaadat ekkor csupán a kezdőblokk és a blokkszám, például: `kezdő = 0x1000`, `hossz = 0x40` (64 blokk). A logikai blokk sorszáma és a fizikai blokk között a leképezés triviális:

$$\text{fizikai blokk} = \text{kezdőblokk} + \text{logikai blokk}.$$

Előnyök:

- Rendkívül egyszerű és gyors mind szekvenciális, mind véletlen hozzáférésre.
- Kevés metaadat.

Hátrányok:

- A fájl létrehozásakor előre ismerni kell a maximális méretet, vagy gyakran át kell másolni a fájlt nagyobb folytonos területre.
- Külső töredezettséget okoz: a szabad terület apró folytonos darabokra szakadhat, és hiába van elegendő összes szabad blokk, egy nagy fájl számára nem áll rendelkezésre folytonos tartomány.
- Törlés után a felszabaduló tartomány újrahasznosítása nehézkes.

A folytonos allokációt főleg régebbi, illetve speciális rendszerekben használták, de a mai általános célú fájlrendszerekben inkább a következő, rugalmasabb módszerek terjedtek el.

15.3.3 3. Linked allocation – láncolt blokkok

A linked allocation alapötlete, hogy minden adatblokk végén (vagy elején) tárolunk egy mutatót a következő blokkra. A fájl metaadata csak az első blokk azonosítóját tartalmazza. A többi blokk a láncon keresztül érhető el.

Példa: legyen egy fájl négy blokkja a 15., 23., 7. és 31. blokk. A blokkok tartalma:

```
15. blokk: [ adat 4084 bájt | következő = 23 ]
23. blokk: [ adat 4084 bájt | következő = 7  ]
7.  blokk: [ adat 4084 bájt | következő = 31 ]
31. blokk: [ adat 4084 bájt | következő = EOF ]
```

Itt a blokkméret 4 KiB, a mutató 4 bájt, így blokkonként 4092 bájt hasznos adat tárolható. A láncvéget egy speciális érték, az `EOF` jelöli.

Műveletek:

- **Létrehozás és append:** a fájlrendszer keres egy szabad blokkot, beírja az adatot, majd frissíti az előző blokk mutatóját. A fájl végéhez fűzéshez a lánc végét kell megkeresni, ami hosszú fájl esetén költséges lehet, hacsak nem tárolunk külön végmutatót is.
- **Olvasás:** a logikai blokk eléréséhez a láncot az elejétől kezdve kell bejárni. A k. logikai blokk elérése $O(k)$ időt igényel, ami nagy fájloknál elfogadhatatlan lehet.
- **Törlés:** a lánc bejárásával felszabadítjuk a blokkokat.

Előnyök:

- Nincs külső töredezettség, mert bármely szabad blokk felhasználható.
- A fájl dinamikusan növekedhet, nincs szükség előre lefoglalásra.

Hátrányok:

- Véletlen hozzáférés nagyon lassú, mert a láncot be kell járni.
- A mutató tárolása csökkenti a hasznos blokkméretet.
- Megbízhatósági probléma: ha egy blokkban sérül a mutató, a lánc többi része elveszik.
- A lánc karbantartása sok apró írást igényel, ami lassú lehet.

A linked allocation ma már ritka, de tanulságos, mert jól mutatja, miért érdemes a mutatókat külön, gyorsan elérhető struktúrába gyűjteni. Ezt a célt valósítja meg a FAT.

15.3.4 4. FAT – File Allocation Table

A FAT ötlete: ne a blokkokban tároljuk a következő blokkra mutatót, hanem egy külön táblázatban, amely a háttértár minden blokkjához egy bejegyzést tartalmaz. A bejegyzés értéke a lánc következő blokkjának azonosítója, vagy speciális érték `EOF`, illetve `FREE`. A fájl metaadata a könyvtárbejegyzésben mindössze a kezdő blokk sorszáma és a fájl hossza.

Vegyük a korábbi példát, ahol a fájl blokkjai 15, 23, 7, 31. A FAT táblázatban a következő bejegyzések szerepelnek:

Blokk index	FAT bejegyzés
7	31
15	23
23	7
31	EOF

Blokk index	FAT bejegyzés
…	…

A könyvtárbejegyzés a 15. kezdőblokkra mutat. A fájl bejárása:

1. Olvassuk a FAT 15. elemét, értéke 23.
2. Olvassuk a 23. elemet, értéke 7.
3. Olvassuk a 7. elemet, értéke 31.
4. Olvassuk a 31. elemet, értéke EOF.

A fő különbség a linked allocationhoz képest, hogy a lánc követése nem igényli minden adatblokk beolvasását, elegendő a FAT táblát bejárni. Ráadásul a FAT táblát a rendszer a memóriában cache-elheti, így a láncbejárás csak memóriaművelet. A véletlen hozzáférés továbbra is $O(k)$ lépést igényel a lánc hosszában, de az adatblokkok nem mozognak.

A FAT fájlrendszer architektúrája három fő változatban létezik: FAT12, FAT16 és FAT32. A szám a bejegyzés bitszélességére utal: FAT12 12 bites, FAT16 16 bites, FAT32 32 bites bejegyzéseket használ. A lemez folytonos blokkokból áll, a blokkokat itt gyakran **clusternek** nevezik. A FAT tábla mérete a fájlrendszer méretétől és a cluster méretétől függ. Például egy 2 TiB-os kötet 4 KiB-os clusterekkel 2^29 darab 4 bájtos FAT32 bejegyzést igényel, ami körülbelül 2 GiB tábla. Ez a memóriában való cache-elést is korlátozza.

A FAT előnyei:

- Nincs külső töredezettség.
- A láncbejárás nem igényli az adatblokkok olvasását, csak a táblát.
- A fájlnövekedés egyszerű: keresünk egy `FREE` bejegyzést, és beillesztjük a láncba.
- A könyvtárbejegyzés mérete kicsi, mert csak a kezdőclusterre és a hosszra van szükség.

Hátrányai:

- A FAT tábla nagy, és a rendszer teljesítménye erősen függ attól, hogy a tábla mekkora része tartható gyors memóriában.
- A tábla kritikus metaadat: ha sérül, a fájlrendszer használhatatlanná válhat, ezért a FAT-ot általában duplán tárolják.
- A láncbejárás hosszú fájloknál akár több ezer lépés is lehet.
- A fájlrendszer méretének növelése a tábla méretének növekedését vonja maga után.

Gyakori hiba a FAT használata során, hogy a tábla frissítése nem atomi: ha a rendszer a láncbejárás közben leáll, a lánc megszakadhat, vagy ugyanaz a cluster kerülhet két fájlhoz. A későbbi FS-ekben ezt naplózással oldják meg, amiről a Journaling és tranzakciós metaadat alfejezetben lesz szó. Egy másik tipikus hiba a `FREE` lánc helytelen frissítése, aminek következtében a szabad terület egy része elveszik. A FAT-nál a szabad blokkokat nem külön struktúrában, hanem a táblában, `FREE` bejegyzésekkel tartják nyilván, ezért a keresés is a tábla bejárását igényli.

15.3.5 5. Inode – indexelt csomópont és indirekció

Az inode alapú megoldás a mutatókat a fájlhoz tartozó fix méretű, külön metaadatblokkba gyűjti. Ezt a metaadatblokkot nevezzük **index node-nak**, röviden **inode-nak**. Az inode tartalmazza a fájl típusát, jogosultságait, időbélyegeit, hivatkozásszámát és a blokkmutatókat. A blokkmutatók lehetnek:

- **közvetlen (direct) mutatók**, amelyek közvetlenül adatblokkra mutatnak,
- **közvetett (indirect) mutatók**, amelyek egy-egy mutatótömbbel rendelkező blokkra mutatnak; ezek a mutatók további adatblokkokra mutatnak,
- **kétszeresen közvetett (double indirect)** és
- **háromszorosan közvetett (triple indirect)** mutatók.

Példaként vegyünk egy Unix-szerű inode-ot az alábbi paraméterekkel:

- blokkméret: $b = 4$ KiB $= 4096$ bájt,
- blokkmutató mérete: 4 bájt,
- közvetlen mutatók száma: 12,
- egy indirect, egy double indirect és egy triple indirect mutató.

Egy blokkban $\frac{4096}{4} = 1024$ mutató fér el. A fájl méretének felső korlátja:

$$12 \cdot 4096 + 1 \cdot 1024 \cdot 4096 + 1 \cdot 1024^2 \cdot 4096 + 1 \cdot 1024^3 \cdot 4096$$

$$= 48 \text{ KiB} + 4 \text{ MiB} + 4 \text{ GiB} + 4 \text{ TiB}.$$

A gyakorlatban a fájlrendszer ennél kisebb korlátot is beállíthat, például a fájlméret mező bitszélessége miatt.

Az inode-os elrendezésben egy logikai blokk elérése a következőképpen történik:

- ha a logikai blokk sorszáma 0 és 11 közé esik, a közvetlen mutatót használjuk;
- ha 12 és 1035 közé esik, a single indirect blokkon keresztül érjük el;
- ha 1036 és $1024^2 + 1035$ közé esik, a double indirect blokkot kell használni; és így tovább.

Például a 100. logikai blokk eléréséhez:

1. az inode-ban lévő single indirect mutató megadja az indirekciós blokkot;
2. beolvassuk az indirekciós blokkot;
3. annak a $100 - 12 = 88$. bejegyzése mutat a megfelelő adatblokkra.

A nagy fájlok esetén a mély indirekció több extra lemezolvasást igényel, cserébe a közvetlen blokkok révén a kis fájlok nagyon gyorsan elérhetők. A fix méretű inode-tábla azonban új problémát vet fel: ha a tábla betelik, nem hozható létre új fájl, még akkor sem, ha a lemezen van szabad adatblokk. Ezt nevezzük **inode-kifogyás** (inode exhaustion).

Az inode-os módszer előnyei a FAT-tal szemben:

- a metaadatok és a lánckövetés elválik a fájlonkénti globális táblától; nincs szükség nagy, folytonos FAT táblára;
- a kis fájlok nagyon gyorsak, mert a blokkmutatók közvetlenül az inode-ban vannak;
- a hosszú láncok helyett mély indirekcióval nagy fájlokat is kezel, miközben a közepes fájlokhoz egy vagy két indirekciós blokk elegendő;
- a hard linkek természetesek: több könyvtárbejegyzés mutathat ugyanarra az inode-ra.

Hátrányai:

- a véletlen hozzáférés nagy fájloknál többszörös indirekciós blokkok beolvasását igényli;
- az inode mérete és száma rögzített, ami rugalmatlan lehet;
- a blokkmutatók frissítése apró, véletlenszerű írásokat okozhat.

15.3.6 6. Extent – folytonos tartományok

Az **extent** egy olyan metaadatrekord, amely egy fájl logikai blokktartományához rendeli a fizikai blokktartományt. Egy extent három értéket tartalmaz:

- **logikai kezdőblokk** – a fájlon belüli kezdőpozíció,
- **fizikai kezdőblokk** – a háttértáron lévő kezdő blokk,
- **hossz** – a tartomány hossza blokkokban.

Egy fájl több extentből állhat, ha a fizikai elrendezés nem folytonos. Például egy fájl három extenttel:

```
Extent 1: logikai 0,      fizikai 1000, hossz 100
Extent 2: logikai 100,    fizikai 2000, hossz 50
Extent 3: logikai 150,    fizikai 1500, hossz 200
```

A logikai blokk leképezése:

- ha $0 \le l < 100$, akkor a fizikai blokk $1000 + l$;
- ha $100 \le l < 150$, akkor $2000 + (l - 100)$;
- ha $150 \le l < 350$, akkor $1500 + (l - 150)$.

A fájl logikai mérete így 350 blokk, azaz $350 \cdot 4$ KiB $= 1400$ KiB. Az extenteket a fájlrendszer általában egy fastruktúrában, például B-tree-ben tárolja, így a keresés $O(\log e)$ lépésszámú, ahol e az extentek száma.

Az extent-alapú allokáció előnyei:

- Nagy, folytonos fájlok esetén nagyon kevés metaadat szükséges: egyetlen extent elegendő lehet akár több száz mebibájt leírására is.
- A szekvenciális olvasás és írás gyors, mert a lemez feje vagy a flash tár vezérlője nagy folytonos tartományokat kap.
- Nincs szükség mély indirekciós láncokra.
- A ritka fájlok kezelése természetes: a logikai tartományban lyukakat hagyhatunk, amelyekhez nem tartozik fizikai blokk.

Hátrányok:

- Ha a fájl sok kis, nem folytonos darabból áll, az extentek száma megnő, és a metaadat-méret is nő.
- A véletlen írások, különösen a fájl közepén történő apró módosítások, új extenteket hozhatnak létre, illetve darabolhatják a meglévőket.
- Az extentek kezelése bonyolultabb, mint a FAT vagy a linked allocation, mert tartományokkal kell műveleteket végezni.

A modern fájlrendszerekben – például az ext4-ben, az XFS-ben, a Btrfs-ben – az extent a domináns módszer. A következő alfejezetben (Jogosultságok, kvóták és fájlrendszer-szemantika) erre építve tárgyaljuk a fájlrendszer-szemantikát, a jogosultságokat és a kvótákat, ahol az extentek és az inode-ok kölcsönhatása is előkerül.

15.3.7 7. Sparse file – ritka fájlok

A **ritka fájl** (sparse file) olyan fájl, amelynek logikai mérete nagy, de a ténylegesen lefoglalt fizikai blokkok száma kicsi. Ez akkor fordul elő, ha a fájl nagy üres területeket tartalmaz, például virtuális gép lemezképek, adatbázis-előkészítések vagy konténer rétegek esetén. A fájlrendszer az üres tartományokhoz nem rendel fizikai blokkokat; a hiányzó blokkokat olvasáskor nullákként adja vissza.

Például egy 1 GiB-os ritka fájl létrehozása a következő (szemléltetés, nem végrehajtandó parancs éles gépen):

```
# Szemléltetés: saját tmp könyvtárban, nem valós lemezpartíción.
dd if=/dev/zero of=sparse.img bs=1 count=0 seek=1G
```

A fájl logikai mérete 1 GiB, de a ténylegesen lefoglalt blokkok száma közel nulla. A `ls -l` a logikai méretet, a `du` a lefoglalt blokkokat mutatja. A fájlrendszerben az inode vagy extent bejegyzés jelzi, hogy mely logikai tartományokhoz nincs fizikai leképezés. Olvasáskor ezekre a területekre a kernel nulla bájtokat szolgáltat.

A ritka fájlok buktatói közé tartozik, hogy a hagyományos másolóeszközök – ha nem ügyelnek a lyukakra – a fájlt teljesen kiírhatják, elveszítve a tömörséget. A modern eszközök, mint a `cp --sparse=always` vagy a `tar --sparse`, képesek megőrizni a ritkaságot.

15.3.8 8. Töredezettség

A fájlblokkok nyilvántartása szorosan összefügg a **töredezettséggel** (fragmentation). Két fő típusa van:

- **Belső töredezettség:** amikor egy kiosztott egységben kihasználatlan hely marad. Például ha a fájl mérete 2.5 blokk, akkor a harmadik blokknak csak a felét használjuk, a másik fele üres. A belső töredezettség a blokkméret növelésével csökkenthető, de akkor a kis fájlok aránytalanul sok helyet foglalnak.
- **Külső töredezettség:** amikor a szabad blokkok apró, nem folytonos darabokban helyezkednek el. Ilyenkor hiába van sok összesen szabad blokk, egy nagy folytonos tartományt igénylő kérés nem teljesíthető.

A linked allocation és a FAT nem okoz külső töredezettséget a lefoglalás értelmében, mert bármely szabad blokk felhasználható; viszont a fájl fizikai blokkjai szétszóródhatnak, ami lemezfej-mozgást és lassulást okoz. Az extent-alapú módszer célja éppen a folytonos tartományok kialakítása, ezért a töredezettség közvetlenül mérhető az extentek átlagos hosszával: minél rövidebbek az extentek, annál töredezettebb a fájl. A fájlrendszerek ezért gyakran alkalmaznak **preallokációt**, késleltetett allokációt vagy online defragmentálást.

15.3.9 9. Összehasonlítás

Az alábbi táblázat a négy fő allokációs módszer lényegét foglalja össze.

Szempont	Folytonos	Linked	FAT	Inode	Extent
Metaadat mérete	nagyon kicsi	kicsi	nagy tábla	fix, közepes	változó
Véletlen hozzáférés	O(1)	O(n)	O(n) a láncban	O(indirekció)	O(log e)

Szempont	Folytonos	Linked	FAT	Inode	Extent
Szekvenciális hozzáférés	nagyon gyors	közepes	gyors (cache)	gyors	nagyon gyors
Növekedés	nehéz	egyszerű	egyszerű	egyszerű	egyszerű
Külső töredezettség	igen	nincs	nincs	nincs	mérsékelt
Megbízhatóság	közepes	gyenge	kritikus FAT	jó	jó

A táblázatból látható, hogy a modern rendszerekben az inode és az extent kombinációja terjedt el, mert rugalmas és viszonylag kevés kompromisszumot igényel.

15.3.10 10. Labor: FAT- és extent-alapú allokáció összehasonlítása

A labor célja, hogy ellenőrizhető mérésekkel hasonlítsa össze a FAT- és az extent-alapú allokáció viselkedését, különös tekintettel a véletlen hozzáférésre, a metaadat-műveletek számára és a töredezettségre. A mérést egy emulált környezetben, felhasználói szintű szimulációval végezzük, hogy ne kelljen valós lemezt vagy éles fájlrendszert módosítani.

15.3.11 10.1. Kiinduló állapot és konfiguráció

- Hoszt: Linux 6.5, x86_64, 8 GiB RAM, 2.4 GHz CPU.
- Futtatókörnyezet: Python 3.12, minden fájl egy ideiglenes könyvtárban (`/tmp/fat_extent_lab`), amely nem érinti a hoszt rendszerpartícióját.
- Blokkméret: $b = 4$ KiB, mutató/bejegyzés méret: 4 bájt.
- Szimulált lemez mérete: 2^{20} blokk (4 GiB).
- Tesztforgatókönyv:

  1. Létrehozunk 100 fájlt véletlenszerű méretben 1 és 1000 blokk között.
  2. Minden fájlt szekvenciálisan írunk, majd véletlen sorrendben olvasunk 1000 logikai blokkot.
  3. Törlünk minden második fájlt, és létrehozunk újabb 50 fájlt, hogy töredezettség alakuljon ki.
  4. Megmérjük a kért metrikákat.

15.3.12 10.2. Szimulációs váz (pszeudokód)

Az alábbi pszeudokód a két struktúra lényegét mutatja, nem végrehajtandó program.

```
// FAT-szimuláció
struct FAT {
    table[blocks] of int32;  // következő blokk vagy FREE/EOF
}

function fat_write(f, data):
    for each block b in data:
        if f.last_block == EOF:
            free = find_free_block()
            table[f.last_block] = free
            table[free] = EOF
            f.last_block = free
        else:
```

```
            free = find_free_block()
            table[f.last_block] = free
            table[free] = EOF
            f.last_block = free

function fat_read_random(f, logical_block):
    current = f.first_block
    for i in 0..logical_block-1:
        current = table[current]
    return current

// Extent-szimuláció
struct Extent {
    logical_start: int
    physical_start: int
    length: int
}

struct ExtentFile {
    extents: list of Extent
}

function extent_write(f, data):
    // egyszerű modell: új extent a fájl végére
    physical = find_free_contiguous(data.length_blocks)
    f.extents.append(Extent(f.logical_size, physical, data.length_blocks))

function extent_read_random(f, logical_block):
    for e in f.extents:
        if e.logical_start <= logical_block < e.logical_start + e.length:
            return e.physical_start + (logical_block - e.logical_start)
    return HOLE
```

A pszeudokód csak a működés vázát adja; a tényleges méréshez a fenti struktúrákat a labor jegyzetfüzetben implementálni kell.

15.3.13 10.3. Mért metrikák

1. **Véletlen hozzáférési lépések száma** – hány lánckövető vagy extentkereső műveletet igényel egy logikai blokk elérése.
    - FAT esetén: a lánc hossza a logikai blokk sorszáma előtti blokkok számával egyenlő.
    - Extent esetén: az extentek száma, ha lineáris keresést végzünk; naiv modellben $O(e)$.
2. **Metaadat-műveletek száma** – hány táblázatfrissítés vagy extent-rekordmódosítás történt a fájlírás során.
3. **Külső töredezettség** – a szabad folytonos blokkszegmensek átlagos hossza, blokkokban.
4. **Fizikai folytonosság** – minden fájlra az átlagos extent hossz, illetve a FAT-lánc folytonos szakaszainak átlagos hossza.

15.3.14 10.4. Várt eredmények (levezetés)

A mérés előtt elvi alapon a következő eredményeket várjuk:

- A FAT véletlen hozzáférési lépésszáma átlagosan a fájlméret felével arányos. Ha a fájlok átlagos mérete M blokk, akkor egy véletlen logikai blokk eléréséhez átlagosan $M/2$ lánckövetés szükséges.

- Az extent-alapú megoldás lineáris keresés esetén a fájlok extentjeinek számától függ; ha a fájlok folytonosak, mindössze 1-2 összehasonlítás elegendő. Töredezett esetben az extentek száma megnő, de még mindig jóval kisebb, mint a FAT-lánc hossza.
- A metaadat-műveletek száma a FAT-ban a lánc hosszával arányos, mert minden új blokkhoz két táblázatfrissítés tartozik (előző mutató, új EOF). Az extent-esetben minden új folytonos tartomány egyetlen bejegyzést igényel.
- A külső töredezettség a FAT esetén nem gátolja a fájlnövekedést, de a fájlok fizikai szétszórtsága nagy. Az extent esetén a folytonos kiosztás jobban koncentrálja a blokkokat, de a törlés utáni újrahasznosítás során itt is megjelenik töredezettség.

A **mért** és **várt/levezetett** értékeket a mérési naplóban külön kell feltüntetni. Példa egy naplóbejegyzésre:

```
FAT véletlen olvasási lépések:
  mért átlag: 187.3 lépés
  várt/levezetett: 187.0 lépés (átlagos fájlméret 375 blokk, M/2)
Extent véletlen olvasási lépések:
  mért átlag: 2.1 lépés
  várt/levezetett: 2.0 lépés (átlagos extentek száma 2)
```

15.3.15 10.5. Mérési napló és visszaállítás

A labor jegyzetfüzet tartalmazza a git commit azonosítót, a Python verziót, a szimulációs paramétereket és a mérési kódot. A futtatás után a `/tmp/fat_extent_lab` könyvtár törlésével az állapot visszaállítható. A mérés nem igényel valós lemezhozzáférést, ezért nincs szükség mentési pontra.

15.3.16 11. Önellenőrző kérdés

Adott a következő inode-konfiguráció: blokkméret 4 KiB, blokkmutató 4 bájt, 10 közvetlen mutató, 1 single indirect, 1 double indirect és 1 triple indirect mutató. Hány lemezolvasás szükséges a 100. logikai blokk eléréséhez, és mekkora a maximális fájlméret?

Megoldásvázlat: A 100. blokk túl van a 10 közvetlen mutatón, ezért a single indirect blokkot kell beolvasni (1 lemezolvasás), majd abból a megfelelő adatblokkra mutatót használva be kell olvasni az adatblokkot (2. lemezolvasás). Tehát összesen 2 lemezolvasás. A maximális méret:

$$10 \cdot 4096 + 1024 \cdot 4096 + 1024^2 \cdot 4096 + 1024^3 \cdot 4096,$$

azaz körülbelül 4 TiB.

15.3.17 12. Összegzés és továbbvivő eredmények

Ebben az alfejezetben megvizsgáltuk a fájlblokkok nyilvántartásának alapvető alternatíváit: a folytonos allokációt, a linked allocationt, a FAT táblát, az inode-alapú indirekciót és az extent-alapú tartománykezelést. Láttuk, hogy mindegyik módszer más kompromisszumot kínál a véletlen hozzáférés, a növekedés, a metaadat-méret és a töredezettség között. A ritka fájlok és a töredezettség fogalmai a gyakorlati fájlrendszer-használatban is fontosak.

A következő alfejezet (Jogosultságok, kvóták és fájlrendszer-szemantika) ezekre a fogalmakra építve tárgyalja a jogosultságokat, a kvótákat és a fájlrendszer-szemantikát. A VFS-réteg, a mount művelet és a fájlleírók (VFS, mount és fájlleírók) után így teljesebbé válik a kép

arról, hogyan illeszkedik az allokációs stratégia a fájlrendszer teljes működésébe. A későbbi fejezetekben, különösen a FAT, ext4 és XFS–Journaling és tranzakciós metaadat részekben, a konkrét fájlrendszerek (FAT, ext4, XFS, Btrfs) belső felépítése során ismét találkozunk ezekkel a struktúrákkal.

15.4 Jogosultságok, kvóták és fájlrendszer-szemantika

A fájlrendszer nem csupán adattároló réteg: a neve és inode-ai mögött egy hozzáférési és erőforrás-gazdálkodási döntéshozó rendszer áll. Amikor egy folyamat megnyit egy fájlt, a kernel nem egyszerűen az elérési utat oldja fel, hanem egy többlépcsős ellenőrzést hajt végre: megvizsgálja a folyamat hitelesítő adatait, az inode tulajdonosi és módbitjeit, az esetleges kiterjesztett hozzáférési listákat (ACL), a folyamathoz rendelt képességeket, végül a kvóta- és erőforrás-korlátokat. Ebben az alfejezetben ezt a döntési láncot bontjuk ki, különös tekintettel a klasszikus POSIX jogosultsági modellre, az ACL- és képességkiterjesztésekre, a lemez- és inodekvótákra, valamint azokra a fájlrendszer-szemantikai garanciákra, amelyek a párhuzamos folyamatok és a hibatűrés szempontjából meghatározóak.

15.4.1 1. Hozzáférés-vezérlés: alanyok, objektumok, műveletek

A hozzáférés-vezérlés alapkérdése: melyik alany melyik objektumon milyen műveletet hajthat végre. A fájlrendszerben az objektumok a fájlok, könyvtárak, szimbolikus linkek, eszközfájlok és FIFO-k; az alanyok a felhasználók, csoportok, illetve az őket képviselő folyamatok; a műveletek az olvasás (`read`), írás (`write`), végrehajtás (`execute`), valamint a metaadat-műveletek, például a tulajdonosváltás (`chown`), a törlés (`unlink`) vagy a jogosultságok módosítása (`chmod`).

A kernel a hozzáférési döntést nem egyetlen ponton hozza meg. A VFS réteg az inode-műveletek között definiál egy `permission` vagy `check_acl` hívást, amely a fájlrendszer-specifikus inode-objektumon hajtja végre az ellenőrzést. A döntéshez három forrásból gyűjthet információt:

1. **Klasszikus UID/GID modell**: a folyamat tulajdonosi azonosítója, csoporttagsága és az inode `mode` mezője.
2. **ACL**: az inode-hoz kapcsolt kiterjesztett attribútum (`system.posix_acl_access`), amely finomabb szabályokat tartalmazhat.
3. **Képességek (capabilities)**: a folyamat bizonyos privilegizált műveleteket akkor is végrehajthat, ha az UID/ACL ellenőrzés azt tiltaná, feltéve hogy a megfelelő kernel-képességgel rendelkezik.

Ezeket a rétegeket egymás után vizsgáljuk, mert a gyakorlatban ritkán elegendő csak az egyik.

15.4.2 2. A klasszikus UNIX jogosultsági modell

A POSIX által meghatározott alapmodell három alanytípust különböztet meg: a tulajdonost (`owner`), a csoportot (`group`) és a többieket (`other`). Minden inode-hoz tartozik egy `uid_t` típusú tulajdonosi azonosító (`st_uid`) és egy `gid_t` típusú csoportazonosító (`st_gid`). A jogosultságokat az inode `mode` mezőjének alsó 12 bitje tárolja, három bitcsoportba rendezve: `rwx` (read, write, execute) a tulajdonosra, `rwx` a csoportra, `rwx` a többiekre. Ezen felül három speciális bit létezik:

- **setuid (S_ISUID, 0o4000)**: ha a fájl végrehajtható, akkor a folyamat az `exec` során a fájl tulajdonosának effektív UID-jét kapja. Ez a mechanizmus teszi lehetővé, hogy egy normál felhasználó korlátozott jogú kódrészeket futtasson magasabb jogosultsággal, például a `passwd` parancs a `/etc/shadow`-t módosíthassa. Pontosan ezért a setuid bit a legérzékenyebb felület: egy sebezhető setuid program teljes jogosultság-átvételhez vezethet.
- **setgid (S_ISGID, 0o2000)**: végrehajtható fájlnál a folyamat effektív csoportja a fájl csoportja lesz. Könyvtár esetén az újonnan létrehozott fájlok a könyvtár csoportját öröklik, ami a projektkönyvtárak közös csoporttal való kezelését segíti.
- **sticky bit (S_ISVTX, 0o1000)**: könyvtárra alkalmazva azt jelenti, hogy a könyvtárban lévő fájlt csak a fájl tulajdonosa, a könyvtár tulajdonosa vagy privilegizált folyamat törölheti. Ezt használja a `/tmp` és a `/var/tmp`, ahol mindenki írhat, de senki sem törölhet más fájlját.

A mode bitek értelmezése fájlra és könyvtárra eltérő:

Bit	Fájl	Könyvtár
`r`	olvasás	a könyvtár tartalmának listázása (`ls`, `readdir`)
`w`	írás	a könyvtárban bejegyzés létrehozása, törlése, átnevezése
`x`	végrehajtás	belépés a könyvtárba (`opendir`, `open` útvonalon belül)

A könyvtáraknál az `x` bit hiánya különösen zavaró lehet: a `r` bit önmagában csak a nevek listázását engedi, de a fájlok eléréséhez az `x` bit is kell.

A hozzáférési döntés a következő sorrendben történik a hagyományos UNIX ellenőrzésben:

1. Ha a folyamat effektív UID-je megegyezik az inode `st_uid` értékével, akkor a tulajdonosi bitek érvényesek.
2. Ha a folyamat effektív GID-je vagy valamelyik kiegészítő csoportja megegyezik az inode `st_gid` értékével, akkor a csoportbitek érvényesek.
3. Különben a “other” bitek érvényesek.

Fontos, hogy ez a sorrend nem kumulatív: ha a tulajdonosi osztályba tartozik a folyamat, akkor a csoport- és other biteket a kernel figyelmen kívül hagyja. Ezért például egy `0o047` jogú fájlnál (tulajdonos: `r--`, csoport: `r--`, other: `rwx`) a tulajdonos **nem** tud írni, bár a “mindenki más” igen. Ez gyakori félreértés, ezért a gyakorlatban a jogosultsági mátrixot mindig osztályonként kell értelmezni.

A rendszerhívások szintjén a jogosultságok kezelését a következő függvények végzik (mindegyik a `-1` visszatérési értékkel jelez hibát, és az `errno` változót állítja be):

- `int chmod(const char *path, mode_t mode)` – a fájl mode bitjeit módosítja. Csak a tulajdonos vagy `CAP_FOWNER` képességgel rendelkező folyamat hívhatja. Hibalehetőségek: `EPERM` (nincs jog), `EACCES` (a könyvtárban nincs keresési jog), `ENOENT` (nincs ilyen fájl).
- `int fchmod(int fd, mode_t mode)` – azonos, de fájlleíró alapján, ami elkerüli a path-alapú TOCTOU problémát.
- `int chown(const char *path, uid_t owner, gid_t group)` – tulajdonost és/vagy csoportot változtat. A közönséges felhasználó csak akkor változtathatja a tulajdonost, ha `CAP_CHOWN` képessége van; a csoportot csak olyan csoportra, amelynek tagja.
- `int umask(mode_t mask)` – a folyamatban beállít egy bitmaszkot, amelyet a `open` és `creat` hívásoknál a mode bitből levon. A tipikus `022` maszk azt jelenti, hogy az újonnan létrehozott fájl nem kap `w` bitet a csoportnak és az other-nek.

A setuid/setgid/sticky bitek kombinációja a mode mezőben tárolódik, és a `chmod` hívásnál a `0o4000`, `0o2000`, `0o1000` értékekkel adhatók meg. A kernel a `chmod` során bizonyos biztonsági szabályokat érvényesít: a setuid bitet nem lehet beállítani olyan fájlra, amelynek nincs `x` bitje, és a nem privilegizált folyamat nem állíthatja be a setuid bitet olyan fájlra, amelynek tulajdonosa nem ő maga.

A klasszikus modell egyszerű, de merev: csak három osztályba sorol, és nem tudja kifejezni például azt, hogy "Bob olvashatja, Carol írhatja, Dave pedig egyáltalán nem éri el". Erre szolgálnak az ACL-ek.

15.4.3 3. ACL: kiterjesztett hozzáférés-vezérlés

A POSIX ACL kiterjeszti a tulajdonos/csoport/other modellt azáltal, hogy tetszőleges számú felhasználóra és csoportra adhatók külön engedélyek. A Linuxban az ACL-t kiterjesztett attribútumként tárolják, általában a `system.posix_acl_access` néven. Az ACL bejegyzések típusai:

- `ACL_USER_OBJ` – a tulajdonos (egyetlen bejegyzés, megfelel az owner mode biteknek).
- `ACL_GROUP_OBJ` – a tulajdonos csoport (egyetlen bejegyzés).
- `ACL_OTHER` – a többi (egyetlen bejegyzés).
- `ACL_USER` – további felhasználókra vonatkozó bejegyzés, mindegyikhez tartozik egy `uid_t`.
- `ACL_GROUP` – további csoportokra vonatkozó bejegyzés, mindegyikhez tartozik egy `gid_t`.
- `ACL_MASK` – az `ACL_USER` és az `ACL_GROUP_OBJ` jogosultságainak felső korlátja; csak akkor van jelen, ha van kiterjesztett bejegyzés.

Az ACL ellenőrzés a hagyományos mode ellenőrzés után következik, pontosabban a VFS réteg először a mode biteket nézi, majd ha ACL van, akkor az ACL bejegyzéseket. A kernel a tényleges hozzáférést a következő szabály szerint dönti el:

- Ha a folyamat effektív UID-je a tulajdonos, az `ACL_USER_OBJ` jogosultságok érvényesek.
- Ha a folyamat effektív UID-je valamelyik `ACL_USER` bejegyzésben szerepel, akkor az adott bejegyzés jogosultságai érvényesek (az `ACL_MASK` által korlátozva).
- Ha a folyamat effektív GID-je vagy kiegészítő csoportja megegyezik a tulajdonos csoporttal vagy valamelyik `ACL_GROUP` bejegyzéssel, akkor a csoportra vonatkozó engedélyek kombinációja érvényes, szintén az `ACL_MASK` korlátozással.
- Végül az `ACL_OTHER` érvényes.

Az `ACL_MASK` szerepe a kompatibilitás fenntartása: amikor a `chmod` parancs módosítja a csoportbiteket, az valójában az ACL maszkját is módosítja, így a `chmod g-w file` letiltja az írást minden kiterjesztett csoportbejegyzésnek is. Ez a viselkedés a rendszeradminisztrátor számára intuitív, de a fejlesztőnek tisztában kell lennie azzal, hogy az ACL-ben tárolt kiterjesztett jogosultságokat a maszk felülírhatja.

A parancssori eszközök (például a legtöbb Linux-disztribúcióban a `setfacl` és `getfacl`) az ACL beállítását és lekérdezését végzik, de a kernelben a tényleges tárolást a fájlrendszer kiterjesztett attribútum mechanizmusa biztosítja. A fontosabb hívások:

- `int setxattr(const char *path, const char *name, const void *value, size_t size, int flags)` – kiterjesztett attribútum írása; az ACL esetében a név `"system.posix_acl_access"`, a flags pedig `0` vagy `XATTR_CREATE`/`XATTR_REPLACE`.
- `ssize_t getxattr(const char *path, const char *name, void *value, size_t size)` – attribútum olvasása.

- `int listxattr(const char *path, char *list, size_t size)` – az összes attribútum nevének felsorolása.

Ezek a hívások a fájlrendszer-specifikus implementációra támaszkodnak; nem minden fájlrendszer támogatja az ACL-t, és a régebbi vagy egyszerűbb rendszerek (például a minimális FAT) csak a mode biteket ismerik.

A képességek szintén a hozzáférés-vezérlés részét képezik, de más szinten: nem az inode-hoz, hanem a folyamathoz kapcsolódnak.

15.4.4 4. Capability alapú hozzáférés

A klasszikus UNIX modell egy nagy privilégiumot ismer: a `root` (UID 0) teljhatalmat kap. Ez a mindent-vagy-semmit megközelítés biztonsági szempontból kockázatos, ezért a Linux kernel bevezette a képességek (capabilities) rendszerét. Minden privilegizált műveletet különálló képességhez kötött, és egy folyamat csak akkor hajthatja végre az adott műveletet, ha a megfelelő képesség a hatékony (effective) képességhalmazában van. A root azért teljhatalmú, mert alapértelmezés szerint minden képességgel rendelkezik, de egy normál folyamat is kaphat bizonyos képességeket anélkül, hogy UID 0 lenne.

A fájlrendszer-műveletek szempontjából a releváns képességek közül a legfontosabbak:

- `CAP_DAC_OVERRIDE` – a hagyományos `rwx` és ACL ellenőrzés megkerülése, azaz a folyamat úgy férhet hozzá fájlokhoz, mintha minden jogosultsága megvolna.
- `CAP_DAC_READ_SEARCH` – olvasási jog a fájlokra és keresési jog a könyvtárakra, de írási jog nem.
- `CAP_CHOWN` – a fájl tulajdonosának és csoportjának tetszőleges megváltoztatása.
- `CAP_FOWNER` – olyan műveletek, amelyeket általában csak a fájl tulajdonosa végezhet, például a `chmod` akkor is, ha a folyamat nem tulajdonos.
- `CAP_FSETID` – a setuid/setgid bitek kezelése korlátozott jogú folyamatoknál.
- `CAP_MKNOD` – eszközfájlok létrehozása.
- `CAP_LEASE` – file lease-ek beállítása.
- `CAP_LINUX_IMMUTABLE` – az `FS_IOC_SETFLAGS` ioctl használata például az immutable flag beállítására.

A kernel a hozzáférés-ellenőrzés során a következő logikát követi (pszeudokód szinten):

```
if (privileged_cap_override(folyamat, CAP_DAC_OVERRIDE))
    return engedélyez;

if (ellenőriz_UID_GID_mode(folyamat, inode))
    return engedélyez;

if (ACL_van(inode) && ellenőriz_ACL(folyamat, inode))
    return engedélyez;

return megtagad;
```

Ez a sorrend biztosítja, hogy a `CAP_DAC_OVERRIDE` képesség a legerősebb, de a többi képesség csak specifikus esetekben lép be. Például a `CAP_DAC_READ_SEARCH` csak az olvasási és keresési jogot adja meg, az írást nem; ezért az ilyen folyamatnak az ACL- vagy mode-alapú írási engedélyre is szüksége van.

A képességek a fájlhozzáférés mellett a kvótákra is kihatnak: a `CAP_SYS_RESOURCE` képességgel rendelkező folyamat túllépheti a kvótakorlátokat, illetve kezelheti azokat. Ezt a kvótaszakaszban részletezzük.

A képességmodell gyakorlati jelentősége, hogy egy hálózati szolgáltatást vagy egy konténerfolyamatot úgy lehet korlátozni, hogy valódi UID 0 nélkül is képes legyen bizonyos privilegizált fájlműveletekre, miközben a rendszer többi részétől el van szigetelve.

15.4.5 5. Kvóták: erőforrás-korlátok

A jogosultságok a műveletek *típusait* szabályozzák; a kvóták a műveletek *mennyiségét* korlátozzák. A lemez- és inodekvóta célja, hogy egyetlen felhasználó vagy csoport ne foglalhassa el a teljes tárolóterületet, és ne hozhasson létre túl sok apró fájlt, ami kimerítheti az inode-készletet.

A kvótarendszer két erőforrást számlál:

- **Blokkok**: a felhasználó által birtokolt fájlok által elfoglalt lemezblokkok száma (általában 1 KiB vagy 4 KiB egységenként). A blokkok mérete a fájlrendszertől függ.
- **Inode-ok**: a felhasználó által birtokolt fájlok és könyvtárak száma.

Minden kvótabejegyzéshez két korlát tartozik:

- **Soft limit (soft):** az a határ, amelyet a felhasználó ideiglenesen átléphet, de a túllépés elindít egy türelmi időt (grace period). Ha a türelmi idő lejár, a soft limit kemény korláttá válik, és további blokk/inode foglalás nem engedélyezett, amíg a felhasználó a korlát alá nem csökkenti a fogyasztását.
- **Hard limit (hard):** az a felső határ, amelyet semmilyen körülmények között nem lehet túllépni (kivéve a `CAP_SYS_RESOURCE` képességgel rendelkező folyamatokat).

A kvóta számlálása a fájlrendszerben történik, amikor a kernel blokkokat allokál vagy inode-ot hoz létre. A műveletek során a kvóta-ellenőrzés a VFS réteg és a fájlrendszer közötti határon zajlik. Például egy `write()` hívás során a fájlrendszernek új blokkokat kell allokálnia; minden blokk allokálása előtt a kvótarendszer megvizsgálja, hogy a fájl tulajdonosának a kvótafogyasztása a korlát alatt marad-e. Ha a hard limit túllépésre kerülne, a hívás `EDQUOT` hibával tér vissza (Disk quota exceeded), és a részleges írás is meghiúsulhat, attól függően, hogy a fájlrendszer támogatja-e a tranzakcionális allokációt.

A kvóták kezelését a rendszeradminisztrátor parancssori eszközökkel végzi:

- `quotacheck` – a kvótaadatbázis ellenőrzése és építése a fájlrendszer tartalma alapján.
- `quotaon` / `quotaoff` – a kvóta engedélyezése / letiltása a fájlrendszeren.
- `edquota -u felhasználó` – a kvótabejegyzés szerkesztése interaktívan.
- `setquota -u felhasználó softblock hardblock softinode hardinode fájlrendszer` – nem interaktív beállítás.
- `quota -u felhasználó` – a fogyasztás lekérdezése.

A kvóta a fájlrendszer mount opciói között szerepelhet, például az `usrquota` és `grpquota` mount opciók engedélyezik a felhasználói és csoportkvótát. A Linuxban a kvóta implementációja történetileg a `quota` és `quota2` formátumú fájlokat használja, de a modern rendszerek gyakran a journaling fájlrendszerbe integrált kvótakezelést alkalmazzák (például ext4-ben a `quota` mount opció és a `quota` feature).

A kernel belső interfésze a kvótához a következő kulcsfontosságú hívásokat tartalmazza (pszeudokód):

```
/* ellenőrzés írási blokk allokálása előtt */
int dquot_alloc_block(struct inode *inode, qsize_t count);
int dquot_alloc_inode(struct inode *inode);

/* visszaadás törléskor vagy felszabadításkor */
int dquot_free_block(struct inode *inode, qsize_t count);
int dquot_free_inode(struct inode *inode);
```

Ezek a függvények a tényleges allokáció előtt frissítik a számlálót, és `EDQUOT` hibával térnek vissza, ha a korlát átlépésre kerülne. A fájlrendszernek a hibát tovább kell adnia a hívó felé, és gondoskodnia kell arról, hogy a már allokált blokkokat felszabadítsa.

A kvótaterv elkészítésekor a következő szempontokat kell figyelembe venni:

- A blokkokat és az inode-okat külön kell korlátozni, mert a kis fájlok nagy száma önmagában is káros lehet.
- A soft limitet úgy kell beállítani, hogy a felhasználó normál munkafolyamata ne érje el, de a hard limit ne legyen túl magas ahhoz, hogy más felhasználókat veszélyeztessen.
- A rendszerfiókok (például `root`, szolgáltatások) általában nincsenek kvótázva, vagy magasabb korlátot kapnak.
- A türelmi idő hossza a szervezeti politikától függ; a túllépés után a felhasználónak törölnie vagy archiválnia kell az adatokat.

A kvóta és a jogosultságok kapcsolata szoros: a kvótát csak az a folyamat tudja módosítani, amelyik `CAP_SYS_RESOURCE` képességgel rendelkezik, és a kvótabejegyzések maguk is a felhasználói azonosítókhoz kötődnek. A `CAP_SYS_RESOURCE` képességgel rendelkező folyamat átlépheti a hard limitet is, ami a rendszergazda számára hasznos, de egy kiszivárgott képesség súlyos kockázatot jelent.

15.4.6 6. POSIX fájlrendszer-szemantika

A jogosultságokon és kvótákon túl a fájlrendszer viselkedését a POSIX szabvány által meghatározott szemantika is alakítja. A legfontosabb garanciák a következők:

- **Fájlleírók és a nyitott fájlok**: A `open()` hívás egy fájlleírót ad vissza, amely a folyamat táblájában egy bejegyzésre, az pedig a kernel nyitott fájl objektumára mutat. A nyitott fájl objektum tartalmazza a fájl aktuális pozícióját (`file offset`) és a hozzáférési módot. A `read()` és `write()` hívások ezt a pozíciót használják, és a hívás után a pozíciót a beolvasott/írt bájtok számával növelik.
- **Atomi hozzáfűzés**: Ha a fájl `O_APPEND` flaggel van nyitva, akkor minden `write()` a fájl végére ír, és a kernel garantálja, hogy a pozíció beállítása és az írás atomi, így több folyamat egyidejű hozzáfűzése nem írja felül egymást.
- **Fájlzárolás**: A POSIX `fcntl()` rendszerhívás lehetővé teszi rekordzárak (record locks) beállítását a fájl egy tartományára. A zárak lehetnek olvasó (`F_RDLCK`) vagy író (`F_WRLCK`) típusúak, és a `F_SETLK` (nem blokkoló) vagy `F_SETLKW` (blokkoló) parancsokkal állíthatók be. A zárak a folyamat megszűnésekor automatikusan felszabadulnak.
- **Rendszerhívások sorrendje**: A POSIX nem garantálja az egyes `read()` és `write()` hívások atomitását a párhuzamos hozzáférések között, kivéve ha a fájlrendszer vagy a hívás paraméterei másként rendelkeznek. Ezért a megosztott fájlokat általában zárolással vagy `pwrite`/`pread` hívással kell kezelni, amelyek explicit pozíciót vesznek át, és nem módosítják a fájlleíró megosztott pozícióját.

- **Szinkronizáció**: A `write()` hívás visszatérése nem jelenti azt, hogy az adat a lemezre került; a kernel a dirty page cache-t késleltetve írhatja ki. A `fsync(fd)` hívás biztosítja, hogy az adott fájl összes módosított adata és metaadata a tárolóra kerüljön; a `fdatasync(fd)` csak az adatot és a méretváltozást erőlteti ki, a többi metaadatot nem. A `sync()` az egész rendszer dirty pufferjeit kiírja.
- **Konzisztencia a fizikai tárolón**: A POSIX nem követeli meg a crash utáni teljes konzisztenciát, de a fájlrendszer implementációk általában naplózással vagy copy-on-write módszerrel biztosítják, hogy a metaadatok ne sérüljenek. A kommunkáról a későbbi alfejezet (Journaling és tranzakciós metaadat) részletesen szól.

Ezek a szemantikai garanciák közvetlenül érintik a hozzáférés-vezérlést is: például egy fájl törlése (`unlink`) csak akkor történhet meg, ha a folyamatnak van írási joga a fájlt tartalmazó könyvtárhoz, nem pedig magához a fájlhoz. Ez a különbség a mode bitek könyvtár-értelmezéséből következik, és a gyakorlatban sok félreértést okoz.

15.4.7 7. Rename és atomi frissítés

A `rename()` rendszerhívás különleges szerepet tölt be a fájlrendszer-szemantikában, mert a POSIX garantálja, hogy a művelet *atomi* a fájlrendszeren belül. Ez azt jelenti, hogy a `rename()` vagy teljesen végrehajtódik, vagy egyáltalán nem; a név soha nem maradhat hiányos vagy félig átnevezett állapotban, és a művelet során más folyamatok az új nevet vagy a régit látják, de soha nem egy köztes állapotot.

A `rename(oldpath, newpath)` hívás a következő eseteket kezeli:

- Ha `oldpath` fájl, akkor a `newpath` egy létező fájlra is mutathat; ekkor a régi `newpath` tartalma megsemmisül, és az új név az `oldpath` inode-jára mutat. Ez a folyamat atomi, nincs olyan pillanat, amikor a fájl mindkét néven létezik, és nincs olyan, amikor a `newpath` hiányzik.
- Ha `oldpath` könyvtár, akkor a `newpath` csak üres könyvtárra mutathat, és a `newpath` nem lehet az `oldpath` alkönyvtára.
- A hívás nem működik fájlrendszerek között; ekkor `EXDEV` hibával tér vissza, és a folyamatnak másolással és törléssel kell megoldania az átvitelt.

Az atomi `rename` kulcsszerepet játszik a biztonságos fájlfrissítési mintákban. Például egy konfigurációs fájl frissítése a következő lépésekben történhet:

```
#!/bin/bash
# Biztonságos atomi frissítés
printf '%s\n' 'új tartalom' > /tmp/config.tmp.$$
chmod 0644 /tmp/config.tmp.$$
mv /tmp/config.tmp.$$ /etc/app/config.conf
```

Itt a `mv` parancs a `rename()` hívásra épül, így a konfigurációs fájl mindig konzisztens tartalommal rendelkezik. Ha a folyamat közben megszakad, a régi fájl vagy az új fájl érvényes, de soha nem egy félig írt állapot.

A `rename()` hívás a hozzáférés-ellenőrzést a következőképpen hajtja végre: a folyamatnak írási joga kell legyen mind az `oldpath` könyvtárához, mind a `newpath` könyvtárához. A hívás a `-1` értékkel jelzi a hibát, és az `errno` értéke lehet `EXDEV` (fájlrendszerek közötti átnevezés), `ENOENT` (a forrás nem létezik), `EACCES` (nincs írási jog a könyvtárakhoz), `EPERM` (a fájlrendszer nem támogatja a könyvtár-átnevezést), `EISDIR` (a `newpath` könyvtár, de az `oldpath` nem könyvtár, vagy fordítva), `ENOTEMPTY` (a cél könyvtár nem üres), `EBUSY` (a forrás vagy cél használatban van), `ENOSPC` (nincs szabad blokk/inode).

A `rename()` a kvóta szempontjából is érdekes: a forrás inode kvótája nem változik, mert az inode átkerül egy másik név alá. A cél `newpath` inode-ja azonban felszabadul, ami a cél fájl tulajdonosának kvótáját csökkenti. Ezért a `rename()` során a fájlrendszernek a kvótaszámlálást is frissítenie kell.

15.4.8 8. Gyakorlati labor: jogosultsági mátrix és kvótaterv

A fejezet laborjában egy emulált környezetben készítünk jogosultsági mátrixot és kvótatervet. A cél annak bizonyítása, hogy a hozzáférés-vezérlés és a kvóták együttesen működnek, és hogy a tipikus hibákat fel tudjuk ismerni. A labor reprodukálható, és a mért eredményeket a vártaktól elkülönítve rögzítjük.

15.4.9 Környezet

- Egy eldobható QEMU virtuális gép vagy Linux konténer, 2 GiB RAM, 1 vCPU, 16 GiB virtuális lemez, ext4 fájlrendszer, `usrquota` és `grpquota` mount opciókkal.
- A build- és futtatási parancsok a reprodukciós csomag részeként szerepelnek, például:

```
# QEMU indítása
qemu-system-x86_64 -m 2048 -smp 1 \
    -drive file=os-labor.qcow2,if=virtio \
    -nographic
```

- A hoszt valódi lemezét, bootloaderét és rendszerpartícióját nem módosítjuk; minden művelet a virtuális gépen belül történik.

15.4.10 Lépések és elvárt eredmények

1. **Felhasználók és csoportok létrehozása**: hozzunk létre három felhasználót (`alice`, `bob`, `carol`) és két csoportot (`dev`, `ops`). `alice` a `dev` és `ops` csoport tagja, `bob` csak a `dev`, `carol` csak az `ops` tagja. A shell parancsok:

```
useradd -m alice
useradd -m bob
useradd -m carol
groupadd dev
groupadd ops
usermod -aG dev,ops alice
usermod -aG dev bob
usermod -aG ops carol
```

2. **Tesztfájlok létrehozása**: hozzunk létre egy projektkönyvtárat `/srv/projekt` néven, tulajdonosa `alice`, csoportja `dev`. A könyvtár mode-ja `2770` (setgid bit a közös csoport öröklődéséhez, tulajdonos és csoport teljes jog, other semmi). Ezen belül hozzunk létre egy fájlt `secret.txt` néven, tulajdonosa `alice`, csoportja `alice`, mode `0600`. Ezután állítsunk be egy ACL-t, amely `bob`-nak olvasási jogot ad a fájlhoz:

```
mkdir -p /srv/projekt
chown alice:dev /srv/projekt
chmod 2770 /srv/projekt
chown alice:alice /srv/projekt/secret.txt
chmod 600 /srv/projekt/secret.txt
setfacl -m u:bob:r-- /srv/projekt/secret.txt
```

3. **Jogosultsági mátrix összeállítása**: a várt hozzáférések mátrixa:

Alany	`/srv/projekt` (listázás)	`/srv/projekt` (belépés)	`/srv/projekt/secret.txt` (olvasás)	`/srv/projekt/secret.txt` (írás)
alice	engedélyezve (`r`)	engedélyezve (`x`)	engedélyezve (owner `r`)	engedélyezve (owner `w`)
bob	engedélyezve (group `r`)	engedélyezve (group `x`)	engedélyezve (ACL `r--`)	tiltva (ACL csak `r`)
carol	tiltva (other `---`)	tiltva (other `--x`)	tiltva (other `---`)	tiltva

Ezt a mátrixot a laborjegyzőkönyvben rögzítjük.

4. **Kvóta beállítása**: `alice` felhasználóra blokkokban 100 MiB soft limitet és 120 MiB hard limitet állítunk be, inode-okban 1000 soft limitet és 1200 hard limitet. A türelmi idő legyen 60 másodperc. A parancs:

```
setquota -u alice 102400 122880 1000 1200 /dev/vda1
```

A blokkokat 1 KiB egységekben számítjuk, ezért a 100 MiB = 102400 blokk, a 120 MiB = 122880 blokk.

5. **Tesztelés**: bejelentkezünk `alice` felhasználóval, és egy egyszerű C programmal vagy `dd`-vel fájlokat hozunk létre a `/home/alice` könyvtárban. A mért eredmények:

- Amíg a foglalt blokkok 100 MiB alatt vannak, a `write()` sikeres.
- Amikor a foglalás eléri a soft limitet, a kernel egy figyelmeztetést adhat, de a `write()` még sikeres, mert a türelmi idő engedélyezi a túllépést.
- A hard limit elérésekor a `write()` `EDQUOT` hibával tér vissza. A `errno` értékét ellenőrizzük: `EDQUOT` (a Linuxban `122`, de az `errno.h`-ban definiált konstans).

A mérési naplóban rögzítjük a parancsot, a kimenetet, a `write()` visszatérési értékét és az `errno`-t. A várt eredményt (a specifikáció alapján) és a mért eredményt (a tényleges futásból) külön sorban adjuk meg.

6. **ACL ellenőrzés**: `bob` felhasználóval megkíséreljük a `cat /srv/projekt/secret.txt` parancsot; a várt eredmény a fájl tartalma. Majd megkíséreljük a `echo 'x' > /srv/projekt/secret.txt` parancsot; a várt hiba `EACCES` (Permission denied), mert az ACL csak olvasást ad.

7. **Rename teszt**: ellenőrizzük, hogy a `mv` a `/srv/projekt` könyvtárban lévő fájlra atomi, és hogy a kvóta a cél inode felszabadításakor helyesen csökken.

A labor végén a mért adatokat táblázatban összefoglaljuk, és a várt értékekkel összehasonlítjuk. A konfigurációt és a mérési naplót a reprodukciós csomag tartalmazza.

15.4.11 9. Tipikus hibák és helyes megoldások

A hozzáférés-vezérlés és a kvóták területén számos gyakori hiba fordul elő, amelyek biztonsági réseket vagy működési zavarokat okozhatnak. Az alábbi esetekben a hibát és a helyes megoldást is bemutatjuk.

15.4.12 9.1. TOCTOU (time-of-check to time-of-use)

A TOCTOU hiba akkor lép fel, amikor a folyamat először ellenőrzi a hozzáférést, majd a tényleges műveletet egy későbbi időpontban hajtja végre, és a kettő között a fájl vagy a jogosultság megváltozik. Példa:

```
/* HIBÁS: ellenőrzés és használat között a fájl cserélhető */
if (access(path, R_OK) == 0) {
    fd = open(path, O_RDONLY);
    /* ... */
}
```

Itt a `access()` és az `open()` között egy másik folyamat kicserélheti a fájlt, vagy a szimbolikus linket átirányíthatja egy másik, magasabb jogosultságú fájlra. A helyes megoldás a fájlleíró alapú ellenőrzés, amely a megnyitott objektumot rögzíti:

```
/* HELYES: az open által visszaadott fd-t használjuk */
fd = open(path, O_RDONLY | O_NOFOLLOW);
if (fd < 0) {
    /* hiba kezelése, errno vizsgálata */
}
/* a fájlon az fd-n keresztül műveletek */
```

Az `O_NOFOLLOW` flag megakadályozza a szimbolikus linkek követését, így a `path` nem vezethet váratlan helyre. A `openat()` rendszerhívás használata szintén csökkenti a TOCTOU kockázatot, mert a relatív útvonalat a megadott alapfájlleíróhoz képest oldja fel.

15.4.13 9.2. Setuid programok helytelen jogosultság-kezelése

Egy setuid program akkor biztonságos, ha a lehető legkevesebb jogosultsággal fut. Gyakori hiba, hogy a program a setuid jogosultságot nem adja le, mielőtt nem megbízható adatokkal dolgozik. A helyes megoldás a tényleges feladat elvégzése előtt a privilégiumok csökkentése:

```
/* HELYES: setuid program privilégium-csökkentés */
seteuid(getuid());
/* ezután a folyamat az eredeti felhasználó jogosultságával fut */
```

A `seteuid()` a hatékony UID-t az adott értékre állítja; ha a folyamat `CAP_SETUID` képességgel rendelkezik, akkor az összes UID-t (valós, hatékony, mentett) is módosíthatja. A `prctl(PR_SET_NO_NEW_PRIVS, 1, 0, 0, 0)` hívás megakadályozza, hogy a folyamat a későbbiekben képességeket vagy setuid jogot szerezzen.

15.4.14 9.3. A csoport- és ACL maszk félreértése

A klasszikus Unix modellben a csoportjogok és az ACL maszk közötti kapcsolat gyakran vezet hibához. Ha az adminisztrátor `chmod g-w` parancsot ad egy ACL-t tartalmazó fájlra, az valójában az ACL maszkot is csökkenti, ezért a kiterjesztett `ACL_GROUP` bejegyzések is elveszítik az írási jogot. Ez szándékos, de a fejlesztőben felmerülhet, hogy az ACL bejegyzés miért nem érvényesül. A megoldás a `getfacl` parancs futtatása a tényleges ACL állapot ellenőrzéséhez, és a `setfacl -m m:` bejegyzés használata a maszk kifejezett beállításához.

15.4.15 9.4. Kvóta soft limit túl szigorú

Ha a soft limit túl alacsony, a felhasználó mindennapi munkája során folyamatosan eléri a korlátot, és a türelmi idő lejárta után a folyamat `EDQUOT` hibát kap, ami adatvesztéshez vezethet. A megoldás a kvótaterv készítésekor a felhasználók tényleges igényeinek felmérése, és a soft limit megfelelő tartalék beállítása. A laborban ezt a mérési adatok alapján ellenőrizzük.

15.4.16 9.5. A rename-nek nincs tranzakciós garanciája fájlrendszerek között

A `rename()` csak ugyanazon a fájlrendszeren belül atomi. Ha a fejlesztő feltételezi, hogy a `EXDEV` hiba esetén a másolás és törlés is atomi, az hibához vezet. A helyes megoldás a `rename()` hibájának kezelése, és ha szükséges, egy explicit másolás-törlés szekvencia futtatása, de ennek a tranzakciós tulajdonságai nem garantálhatók úgy, mint a `rename()` esetén.

15.4.17 10. Önellenőrző kérdések

1. Adott egy fájl `0o640` mode-ja, tulajdonosa `alice`, csoportja `dev`. Melyik műveletet hajthatja végre `alice`, `bob` (a `dev` tagja) és `carol` (nem tagja a `dev`-nek)?
2. Mi a különbség a soft limit és a hard limit között a kvótáknál, és mi a türelmi idő szerepe?
3. Hogyan változtatja meg a `chmod g-w` parancs egy ACL-maszkkal rendelkező fájl kiterjesztett csoportbejegyzéseit?
4. Miért nem atomi a `rename()` fájlrendszerek között, és milyen hibakódot ad vissza ilyenkor?
5. Milyen kockázatai vannak a setuid bitnek, és hogyan csökkenthető a privilégiumok kiszivárgása?

A fenti kérdések megválaszolása segít ellenőrizni, hogy a hozzáférés-vezérlés és az erőforráskorlátok alapelvei világosak-e. A következő alfejezet (FAT, ext4 és XFS) a konkrét fájlrendszer-implementációkra (FAT, ext4, XFS) tér át, építve az itt bevezetett jogosultsági és kvótafogalmakra.

16 Fájlrendszer-implementációk

16.1 FAT, ext4 és XFS

A helyi fájlrendszerek két, eltérő tervezési célt képviselő csoportra oszthatók. A klasszikus, viszonylag egyszerű elrendezésű fájlrendszerek – mint a FAT – elsősorban a cserélhető adathordozók, a beágyazott eszközök és az egyszerű interoperabilitás igényeire készültek. A nagy teljesítményű, modern helyi fájlrendszerek – mint az ext4 és az XFS – ezzel szemben a többterabájtos kötetek, a nagy fájlok, a párhuzamos munkaterhelés és a rendszerösszeomlás utáni gyors helyreállítás követelményeit szolgálják. Ebben az alfejezetben a FAT elrendezését, az ext4 kompatibilitásra épülő bővítéseit és az XFS allocation group alapú skálázhatóságát vizsgáljuk, majd a laborban ext4 és XFS környezetet hozunk létre emulált gépen, és összehasonlító mérési naplót készítünk.

16.1.1 FAT: a klasszikus láncolt allokáció

A FAT (File Allocation Table) család – FAT12, FAT16, FAT32 – az egyik legrégebbi, még ma is széles körben használt fájlrendszer. Tervezésekor a legfontosabb cél az egyszerűség és a kis memóriaigény volt, nem pedig a nagy teljesítmény vagy a megbízhatóság. A FAT elrendezése négy fő területből áll:

1. **Boot szektor és fenntartott terület:** a kötet elején található BIOS Parameter Block (BPB) és a FAT-specifikus fejléc, amely leírja a szektorméretet, a klaszterméretet, a FAT táblák számát és a fájlrendszer típusát.
2. **FAT tábla:** a klaszterek állapotát és a fájlokhoz tartozó láncokat tartalmazza. Minden bejegyzés a következő klaszter sorszámát, a fájl végét (EOF), a hibás klaszter jelölését vagy a szabad állapotot tárolja. FAT32 esetén a bejegyzések 32 bitesek, de a felső 4 bit fenntartott, így a hasznos klaszterszám 28 bites.
3. **Gyökérkönyvtár:** FAT12 és FAT16 esetén rögzített méretű, közvetlenül a FAT után helyezkedik el; FAT32-ben a gyökérkönyvtár is egy közönséges klaszterlánc, így mérete rugalmas.
4. **Adatklaszterek:** ide kerülnek a fájlok adatai és a könyvtárbejegyzések. A klaszter a minimális lefoglalható egység, jellemzően 512 bájt és 32 KiB között.

A fájl tartalmának megtalálásához a rendszer a könyvtárbejegyzésből kiolvassa a kezdő klasztert, majd a FAT táblát követve végigjárja a klaszterláncot. Például egy fájl, amely a 3-as klaszterben kezdődik és a 7-es, 8-as klaszterben folytatódik, a következő láncot adja:

```
FAT bejegyzés: [3] → 7
               [7] → 8
               [8] → 0x0FFFFFFF (EOF)
```

A könyvtárbejegyzés fix mezőkből áll: a fájlnév rövid formája (8.3), attribútumok (csak olvasható, rejtett, rendszer, könyvtár, archív), létrehozási és módosítási időbélyegek, a kezdő klaszter és a fájlméret. A hosszú fájlnevek támogatására a VFAT kiterjesztés speciális, rejtett könyvtárbejegyzéseket használ, amelyek Unicode karaktereket tárolnak, és a rövid név generálásakor a rendszer a hosszú névből egy 8.3 kompatibilis álnevet képez.

A FAT előnye az alacsony implementációs költség és a széles körű támogatás: szinte minden operációs rendszer, kamera, nyomtató, beágyazott vezérlő és bootloader képes FAT kötetet olvasni. Ugyanakkor a korlátok jelentősek. A fájlméret FAT32 esetén legfeljebb $2^{32}-1$ bájt (gyakorlatilag 4 GiB), a kötet mérete elméletileg 2 TiB, de a nagy klaszterek miatt a kis fájlok jelentős belső fragmentációt okoznak. A FAT nem támogatja a POSIX jogosultságokat, a tulajdonost, a szimbolikus linket, a sparse fájlokat és a naplózást. Rendszerösszeomlás esetén a fájlrendszer inkonzisztens lehet, mert a FAT tábla és a könyvtárbejegyzések frissítése nem atomi. A lánc sérülése elveszett klaszterekhez vagy keresztláncolt fájlokhoz vezet; ennek javítására az fsck.vfat vagy a CHKDSK szolgál, de az eredmény nem mindig garantált.

Tipikus hibaként említhető, hogy egy gyártó nagy sebességű, több fájlt párhuzamosan író eszközre FAT32-t választ, majd a láncolt allokáció miatt a fájlok erősen töredeznek, és a teljesítmény drasztikusan csökken. Hasonló hiba, ha valaki 4 GiB-nál nagyobb fájlt próbál FAT32-re írni; ilyenkor a rendszer EFBIG hibával tér vissza, és az írás megszakad. A FAT tehát akkor jó választás, ha a kompatibilitás az elsődleges, nem a teljesítmény vagy a megbízhatóság.

16.1.2 ext4: kompatibilis evolúció extents és journaling segítségével

Az ext4 a Linux ext fájlrendszer-családjának negyedik generációja. Az ext2 még klasszikus, indirekt blokkos allokációt használt; az ext3 ehhez naplózást adott; az ext4 pedig extents alapú allokációt, nagyobb méthatárokat, gyorsabb fsck-t és számos további fejlesztést vezetett be úgy, hogy közben megőrizte a visszafelé kompatibilitást. Egy ext4 kötet számos ext2/ext3 tulajdonságot megtart, például a blokkcsoportokra osztott elrendezést, az inode táblát és a szuperblokkot.

A blokkcsoport a ext4 alapvető allokációs egysége. Minden csoportban megtalálható a csoportleíró, a blokkbitmap, az inode bitmap és az inode tábla. A blokkméret tipikusan 4 KiB, de 1 KiB és 64 KiB között konfigurálható. Az inode tábla minden fájlhoz és könyvtárhoz egy inode-ot rendel, amely a metaadatokat – típus, jogosultságok, időbélyegek, méret, linkek száma – és a fájl adatblokkjaira mutató hivatkozásokat tárolja. A klasszikus ext2 indirekt blokkos címzés helyett az ext4 alapértelmezésben extent fát használ.

Az extent egy összefüggő fizikai blokktartományt leíró bejegyzés, amely három értéket tartalmaz: a logikai kezdőblokk, a fizikai kezdőblokk és a hossz. Egyetlen extent például leírhatja, hogy a fájl 0. logikai blokkja a 1000. fizikai blokkon kezdődik és 512 blokk hosszú. A nagy, összefüggően lefoglalt fájlok így néhány extenttel címezhetők, szemben a láncolt vagy indirekt blokkos megoldással, amely minden egyes blokkhoz külön bejegyzést igényel. Az extents fa gyökere az inode-ban található; ha egy fájlhoz kevés extent elegendő, a bejegyzések közvetlenül az inode-ban laknak, ellenkező esetben egy többszintű fába rendeződnek. A töredezett fájlok esetében az extent lista hosszabb lesz, és a műveletek költsége nő, de ez még mindig hatékonyabb, mint a klasszikus indirekt blokkos címzés.

A journaling az ext3 óta része a családnak. Az ext4 a különálló JBD2 napló alrendszert használja. A napló működésének célja, hogy a metaadat-műveletek összeomlás esetén tranzakcióként legyenek helyreállíthatók. Három alapvető üzemmód létezik:

- **data=ordered:** csak a metaadat-változások kerülnek a naplóba, de az adatblokkokat a rendszer a napló commit előtt írja a lemezre. Ez az alapértelmezett, mert megakadályozza, hogy egy fájl metaadatai új adatra mutassanak, miközben a régi adat még a lemezen van.
- **data=writeback:** a napló csak metaadatot tartalmaz, és az adatok kiírásának sorrendje nincs szigorúan kötve. Gyorsabb lehet, de rendszerösszeomlás után a fájl tartalma inkonzisztens lehet a metaadathoz képest.
- **data=journal:** a teljes adat is a naplóba kerül, így a konzisztencia a legerősebb, de a napló kétszeres írást igényel, és a teljesítmény jelentősen csökken.

A napló bejegyzései tranzakciókat alkotnak. Egy könyvtárbejegyzés létrehozásakor például a tranzakció tartalmazza az inode, a könyvtárblokk és a szükséges bitmap módosításait. A commit során a rendszer a naplóba ír, majd a napló commit rekordját érvényesíti; ezután a tényleges metaadatok a helyükre kerülnek. Ha a rendszer a commit után, de a metaadatok kiírása előtt omlik össze, a helyreállítási folyamat újrajátssza a naplót, és a fájlrendszer konzisztens lesz.

Az ext4 további jellemzői közé tartozik a delayed allocation, amely késlelteti a blokkok fizikai lefoglalását a tényleges kiírásig, így jobb döntést tud hozni az összefüggő extents kialakításáról. Ez azonban hasonló fsync-problémát vet fel, mint az XFS-nél: ha egy alkalmazás nem hív fsync-et, az adat rendszerösszeomlás után elveszhet, miközben a fájl metaadatai már frissültek. Az ext4 támogatja a logikai kötetek online növelését, a nagy fájlokat és köteteket (a gyakorlati határok kernelverziótól és mount opcióktól függenek), a hüvelykujjlenyomat indexet a könyvtárakhoz (dir_index), a szuperblokk biztonsági másolatait, a metaadat checksumokat bizonyos konfigurációkban, valamint a journal checksumot. Az eszközök közül a mkfs.ext4, a dumpe2fs, a tune2fs és a debugfs a legfontosabbak.

A ext4 esetén a tipikus hibák közé tartozik az inode kimerülés. Ha sok kis fájlt hozunk létre, a lemezterület még bőven rendelkezésre állhat, de az inode tábla betelhet; ekkor az írás ENOSPC hibával meghiúsul. A megoldás az inode sűrűség növelése a mkfs időpontjában, utólag nem bővíthető. Egy másik gyakori félreértés, hogy a naplózás minden adatot megvéd: a data=writeback és a data=ordered üzemmód csak a metaadatok konzisztenciáját garantálja, az alkalmazás szintű adatvesztést nem akadályozza meg, ha az fsync elmarad.

16.1.3 XFS: allocation group és B+tree alapú skálázhatóság

Az XFS az SGI IRIX rendszeréből származó, nagy teljesítményre tervezett journaling fájlrendszer, amelyet később a Linux kernel is átvett. Alapelve a skálázhatóság: a teljes kötetet **allocation group**okra (AG) osztja, amelyek mindegyike önálló szabadhely- és inode-kezeléssel rendelkezik. Ennek köszönhetően a párhuzamos műveletek különböző AG-kon párhuzamosan, egymástól függetlenül allokálhatnak, ami nagy SMP rendszereken és sokprocesszoros munkaterhelésnél jelentős előnyt ad.

Minden allocation group tartalmazza a saját szuperblokkját, AG fejlécét, szabad blokk B+tree-jét, inode B+tree-jét és a szabad inode listát. A fájlok inode-jai az AG-k között oszlanak el; a rendszer igyekszik azonos AG-n belül tartani egy fájl adatait, hogy a lemezfej és a cache lokalitása jó legyen. Az inode-ok és a szabad blokkok nyilvántartása B+tree struktúrában történik, így a keresés, a beszúrás és a törlés logaritmikus időben elvégezhető, és a nagy köteteknél sem romlik drasztikusan a teljesítmény.

Az XFS az ext4-hez hasonlóan extent alapú allokációt használ. Az inode egy rövid extent listát tartalmaz; ha a fájl több extentet igényel, akkor az inode-ban egy B+tree gyökerére mutató hivatkozás szerepel, amely a kiterjedtebb fájlok extents bejegyzéseit tárolja. A B+tree levelei tartalmazzák a logikai blokkcím, a fizikai blokkcím és a hossz hármasát. A nagy, ritkán használt fájlokhoz a sparse tárolás is hatékonyan illeszkedik.

A journaling az XFS-ben is központi elem. A napló az AG-k mellett, külön területen helyezkedik el, és tranzakciókat tárol. Alapértelmezésben a napló a metaadatokat védi; az adatblokkok sorrendje nem garantált, ezért az XFS-nél különösen fontos, hogy az alkalmazások fsync-et hívjanak, ha tartós állapotot akarnak elérni. A delayed allocation (vagyis a blokkok késleltetett lefoglalása) tovább erősíti ezt a követelményt: ha egy alkalmazás fájlt ír, de nem fsync-el, a rendszer a blokkokat a kiírás pillanatában foglalja le; összeomlás esetén a fájl metaadatai frissültek, de az adatblokkok nem kerültek a lemezre, így a fájl mérete megnő, tartalma viszont nulla, vagy régi adat marad. Ezt a jelenséget gyakran tévesen az XFS hibájának tekintik, pedig a POSIX szemantika szerint az alkalmazásnak fsync-et kell hívnia.

Az XFS erősségei közé tartozik a nagy mérethatár, az online defragmentálás (xfs_fsr), a kötet online növelése (xfs_growfs), a párhuzamos I/O, a nagy fájlok és a nagy kötetek kezelése. A gyakorlati korlátok a kernelverziótól és a mount opcióktól függenek; egyes implementációk 8 EiB nagyságrendű fájlméretet és kötetet is támogatnak, de ezt mindig az aktuális kernel dokumentációjában kell ellenőrizni. A hátrányok között említhető, hogy az XFS kötetet nem lehet egyszerűen zsugorítani, mert a fájlrendszer struktúrája a növekedésre optimalizált; a shrink művelet csak offline, adatmentés és újraformázás révén oldható meg. A régebbi kernelek és a nem megfelelő mount opciók inkompatibilitást okozhatnak; emiatt éles környezetben mindig tesztelni kell a cél kernel és a fájlrendszer verzióját.

A tipikus hibák közé tartozik, hogy valaki XFS-t választ kis, pár gigabájtos VM lemezre, ahol a nagy AG- és naplóméret aránytalanul sok helyet foglal, és a várt teljesítményelőny nem jelentkezik. Ilyenkor az ext4 jobb választás lehet. Egy másik hiba, hogy valaki az XFS-t shrinkelhetőnek gondolja, és a lemezcsökkentés előtt nem menti az adatokat; a művelet adatvesztéshez vezethet.

16.1.4 Összehasonlítás

Az alábbi táblázat a három fájlrendszer legfontosabb, tanulási célhoz tartozó tulajdonságait foglalja össze. A konkrét limitértékek kernelverzió, mount opció és eszközfüggők, ezért a táblázat a tervezési jellemzőkre koncentrál.

Tulajdonság	FAT (FAT32)	ext4	XFS
Allokáció	láncolt klaszter, FAT tábla	extent fa, blokkcsoport	extent B+tree, allocation group
Naplózás	nincs	JBD2, 3 üzemmód	van, metaadat napló
Jogosultságok	nincs POSIX	van	van
Nagy fájlok	max 4 GiB – 1 bájt	16 TiB körül, kernelverzió függő	8 EiB körül, kernelverzió függő
Párhuzamosság	gyenge	közepes	kiváló (AG párhuzamosság)
Shrink	nem jellemző, lánc sértékeny	offline zsugorítás lehetséges bizonyos esetekben	nem támogatott egyszerűen
Javító eszköz	fsck.vfat, CHKDSK	e2fsck, debugfs	xfs_repair, xfs_db
Elsődleges cél	hordozhatóság, egyszerűség	általános célú Linux rendszer	nagy skálázhatóság, szerver

A táblázat azt is mutatja, hogy a „nagy teljesítményű helyi fájlrendszer” kifejezés nem egyetlen fájlrendszert jelöl, hanem olyan tervezési döntések összességét, amelyek a journaling, az extent alapú allokáció és a párhuzamos erőforrás-kezelés révén csökkentik a töredezettséget, gyorsítják a metaadat-műveleteket, és összeomlás után is konzisztens állapotot tudnak visszaállítani.

16.1.5 Labor: ext4 és XFS összehasonlító táblázata

A labor célja, hogy emulált környezetben, reprodukálható módon hozzunk létre ext4 és XFS köteteket, majd rögzítsük a fájlrendszer létrehozásának, mountolásának, alapvető írási és metaadat-műveleteinek mérhető jellemzőit. A mérés nem éles gépen fut; minden adat egy eldobható QEMU virtuális gépben, átmeneti lemezképeken keletkezik. A hoszt valódi lemezét, bootloaderét vagy rendszerpartícióját nem módosítjuk.

Kiinduló állapot: egy Linux vendég QEMU-ban, legalább 2 GiB memóriával, egy processzormaggal (a mérés reprodukálhatósága érdekében rögzítsük az SMP-t). A vendég rendelkezik az ext4 és XFS felhasználói eszközökkel: `mkfs.ext4`, `mkfs.xfs`, `mount`, `dumpe2fs`, `xfs_info`, `dd`, `fio`, `time`, `df`. A hoszton `qemu-img` és egy vendég indító szkript áll rendelkezésre.

Előkészítés és mérési jegyzőkönyv: hozzunk létre két üres, egyenként 1 GiB méretű lemezképet. A QEMU parancssorban csatoljuk őket virtio-blk eszközként, cache `none` beállítással, hogy a vendég közvetlenül, hoszt oldali felesleges pufferelés nélkül érje el a lemezeket.

```
qemu-img create -f raw ext4-lab.img 1G
qemu-img create -f raw xfs-lab.img  1G

qemu-system-x86_64 \
  -m 2048 -smp 2 -enable-kvm \
  -drive file=debian-lab.qcow2,format=qcow2,if=virtio \
  -drive file=ext4-lab.img,format=raw,if=virtio,cache=none \
  -drive file=xfs-lab.img,format=raw,if=virtio,cache=none \
  -nic user,model=virtio-net-pci \
  -nographic
```

A vendégben a két új lemez `/dev/vdb` és `/dev/vdc` néven jelenik meg. Partíciós táblát nem készítünk, mert a fájlrendszer közvetlenül a blokkeszközre kerül; emulált környezetben ez elegendő.

Fájlrendszer létrehozása: az ext4 esetén 4 KiB blokkméretet adunk meg, XFS-nél alapértelmezett paraméterekkel dolgozunk. A létrehozási időt mérjük, és a kimenetet naplózzuk.

```
/usr/bin/time -f "ext4 mkfs time: %e s" mkfs.ext4 -F -b 4096 /dev/vdb
/usr/bin/time -f "xfs mkfs time: %e s" mkfs.xfs -f /dev/vdc
```

A `-F` és `-f` kapcsoló meggátolja, hogy egy korábbi fájlrendszer-észlelés megszakítsa a műveletet. Az elvárt eredmény mindkét parancsnál 0 visszatérési kód; a mért időt a naplóba írjuk.

Mount és alapvető adatgyűjtés: hozzuk létre a csatolási pontokat, majd mountoljuk a köteteket. Az ext4-hez `noatime` opciót használunk; XFS-nél alapértelmezett mount opciókkal maradunk. A mount időt szintén mérjük. Ezután `df -hT` kimenettel rögzítjük a ténylegesen használt és szabad területet, `dumpe2fs -h /dev/vdb`-vel az ext4 szuperblokk jellemzőit, `xfs_info /dev/vdc`-vel az XFS AG-k, blokkméret, naplóméret jellemzőit.

```
mkdir -p /mnt/ext4 /mnt/xfs
/usr/bin/time -f "ext4 mount time: %e s" mount -o noatime /dev/vdb /mnt/ext4
/usr/bin/time -f "xfs mount time: %e s" mount /dev/vdc /mnt/xfs

df -hT /mnt/ext4 /mnt/xfs
dumpe2fs -h /dev/vdb | tee lab-ext4-superblock.txt
xfs_info /dev/vdc | tee lab-xfs-info.txt
```

Terheléses mérés: a szekvenciális írás és olvasás mérésére `dd`-t használunk, 512 MiB-os fájl esetén. Az `oflag=direct` segítségével elkerüljük a vendég oldali page cache hatását, így a mérés inkább a fájlrendszer és a blokkréteg allokációs viselkedését tükrözi. A `conv=fsync` az írás tartósságát kényszeríti ki.

```
/usr/bin/time -f "ext4 seq write: %e s, %U user, %S sys" \
  dd if=/dev/zero of=/mnt/ext4/seqtest bs=1M count=512 oflag=direct conv=fsync

/usr/bin/time -f "xfs seq write: %e s, %U user, %S sys" \
  dd if=/dev/zero of=/mnt/xfs/seqtest bs=1M count=512 oflag=direct conv=fsync

/usr/bin/time -f "ext4 seq read: %e s, %U user, %S sys" \
  dd if=/mnt/ext4/seqtest of=/dev/null bs=1M iflag=direct

/usr/bin/time -f "xfs seq read: %e s, %U user, %S sys" \
  dd if=/mnt/xfs/seqtest of=/dev/null bs=1M iflag=direct
```

A metaadat-műveletek mérésére hozzunk létre 10 000 kis fájlt, majd töröljük őket. Itt a létrehozási idő a fájlrendszer inode- és könyvtárkezelési hatékonyságát mutatja.

```
/usr/bin/time -f "ext4 create 10k files: %e s" \
  sh -c 'for i in $(seq 1 10000); do echo x > /mnt/ext4/file-$i; done'

/usr/bin/time -f "xfs create 10k files: %e s" \
  sh -c 'for i in $(seq 1 10000); do echo x > /mnt/xfs/file-$i; done'

/usr/bin/time -f "ext4 delete 10k files: %e s" \
  sh -c 'rm /mnt/ext4/file-*'

/usr/bin/time -f "xfs delete 10k files: %e s" \
  sh -c 'rm /mnt/xfs/file-*'
```

Kiürítés és leállítás: a mérések után unmountoljuk a köteteket, majd a QEMU vendéget leállítjuk. A lemezképek megőrizhetők a reprodukciós csomag részeként, de a hoszton nem csatoljuk őket.

```
umount /mnt/ext4
umount /mnt/xfs
poweroff
```

Mérési napló: a mérési naplóban minden parancs után rögzítsük a pontos parancsot, a visszatérési kódot, a mért időt, valamint a fájlrendszer jellemzőit. A mért és a várt eredményt külön oszlopban kell feltüntetni. A várt eredmény nem a teljesítmény konkrét száma, hanem az, hogy a művelet sikeresen lefut, és a fájlrendszer a mount, az írás és az olvasás után is konzisztens marad. A teljesítményadatok csak a rögzített hardver- és szoftverkörnyezetben értelmezhetők, ezért a napló elején rögzítsük a QEMU verziót, a vendég kernel változatát, a CPU-t, a memóriát és a lemez cache beállítását.

Példa a napló egy sorára:

Paraméter	Várt eredmény	Mért eredmény	Megjegyzés
ext4 mkfs visszatérési kód	0	0	sikeres
ext4 mkfs idő	nincs előre definiált	1,23 s	csak ebben a környezetben érvényes
xfs mkfs idő	nincs előre definiált	0,88 s	csak ebben a környezetben érvényes
ext4 mount	sikeres mount, 0 kód	0	`/mnt/ext4` írható
xfs mount	sikeres mount, 0 kód	0	`/mnt/xfs` írható
ext4 szekvenciális írás	sikeres, 512 MiB fájl létrejön	512 MiB méret, idő 2,35 s	direct I/O, fsync
xfs szekvenciális írás	sikeres, 512 MiB fájl létrejön	512 MiB méret, idő 2,10 s	direct I/O, fsync

Paraméter	Várt eredmény	Mért eredmény	Megjegyzés
ext4 10 000 fájl létrehozása	minden fájl létezik	10 000 fájl, idő 8,44 s	inode fogyás nem tapasztalható
xfs 10 000 fájl létrehozása	minden fájl létezik	10 000 fájl, idő 7,92 s	inode fogyás nem tapasztalható

A példa értékek illusztrációk, nem valós mérésből származnak; a laborban a tényleges számokat kell rögzíteni, és egyértelműen jelölni kell, hogy melyik oszlop tartalmazza a mért adatot.

Visszaállítás: a labor nem igényel külön visszaállítást, mert a vendég lemezképe eldobható, a hoszt lemezeihez nem nyúltunk. A munkakönyvtárból a létrehozott image fájlokat törölhetjük, vagy megőrizhetjük a reprodukciós csomag részeként. A reprodukciós csomag tartalmazza a kiinduló vendég image nevét vagy ellenőrzőösszegét, a build- és futtatási parancsokat, valamint a teljes mérési naplót.

16.1.6 Tipikus hibák és helyes gyakorlat

A fenti labor során előforduló tipikus hibák három csoportba sorolhatók. Az első csoport a környezet hibás beállítása: ha a mérést hoszt oldali gyorsítótárazással, például cache `writeback` móddal futtatjuk, a mért írási idők irreálisan alacsonyak lesznek, és nem tükrözik a tényleges fizikai kiírást. A helyes megoldás a `cache=none` vagy `cache=directsync` használata, és a mérési módszer pontos dokumentálása.

A második csoport a fájlrendszer-korlátok figyelmen kívül hagyása. Például ha valaki a 1 GiB-os tesztlemezen a `dd` parancshoz 2 GiB-os fájlt ad meg, az írás az ext4 és az XFS esetén is sikertelenül zárul, de a hibakód és a rendszer-üzenet eltérhet. ext4-nél jellemzően ENOSPC, XFS-nél szintén ENOSPC, de az XFS gyakran már az első allokációs kísérletkor jelzi a szabad hely hiányát. A várt eredményt tehát mindig a kötet tényleges szabad méretének ismeretében kell megfogalmazni.

A harmadik csoport a delayed allocation félreértése. Ha az xfs mérésben a szekvenciális írást `conv=fsync` nélkül futtatnánk, a `dd` parancs visszatérése után a fájl mérete már látszólag helyes, de az adatok egy része még csak a vendég oldali cache-ben van. Egy azonnali `poweroff` után a fájl tartalma elveszhet, vagy részben nulla lehet. A laborban ez kiválóan demonstrálható, de szándékosan csak akkor végezzük el, ha a naplóban egyértelműen megjelöljük, hogy ez a POSIX `fsync` hiányának következménye, nem a fájlrendszer hibája. A helyes gyakorlat az, hogy a tartósságot igénylő mérés mindig `fsync`-et vagy `O_DIRECT` + `fsync` kombinációt használ.

Önellenőrző kérdés: magyarázza el, miért használ az XFS allocation groupokat, és miben különbözik az ext4 blokkcsoportjaitól? A válaszban térjen ki arra, hogy az AG önálló szabadhely- és inode-kezelése hogyan növeli a párhuzamos allokáció skálázhatóságát.

16.2 Btrfs, ZFS és copy-on-write

A hagyományos, helyben író fájlrendszerek – mint az előző alfejezetben tárgyalt ext4 vagy XFS – az adatblokkokat és a metaadatokat ugyanazon a helyen frissítik. Ez egyszerű, és a jól megtervezett journal mechanizmusokkal elfogadható megbízhatóságot nyújt, de a crash utáni konzisztencia-helyreállítás mindig korlátozott: a részlegesen felülírt blokk tartalma nem feltétlenül állítható vissza. A Btrfs és a ZFS ezzel szemben a copy-on-write (CoW) elvet alkalmazza: soha nem írja felül a már létező blokkot, hanem az új tartalmat egy szabad blokkba helyezi, és utána

frissíti a rá mutató hivatkozást. Ez alapvetően más adatintegritási, snapshot- és helyreállítási tulajdonságokat ad.

16.2.1 A copy-on-write alapelve

A CoW lényege: ha egy blokk tartalmát módosítani kell, a rendszer az eredeti blokkot változatlanul hagyja, az új tartalmat egy új, szabad blokkba írja, majd az erre a blokkra mutató metaadatot (például egy inode-ban lévő blokkmutatót) frissíti. A régi blokk csak akkor lesz felszabadítható, ha már semmilyen élő hivatkozás nem mutat rá. Ez a hivatkozás lehet egy normál fájl adatblokkja, egy belső fa csomópont, vagy egy snapshot által rögzített korábbi állapot.

A CoW fájlrendszerben a metaadatok is CoW-módon frissülnek. Egy hagyományos, helyben író rendszernél egy inode módosításakor a lemezen lévő inode-ot közvetlenül írjuk felül; crash esetén az inode félig frissült állapotban maradhat. A CoW-nál az inode új verziója is új blokkba kerül, és a felette lévő fa csomópont mutatóját frissítjük. Végül a gyökércsomópont cseréje atomikusan, egyetlen írással történik. Ha a gyökércsomópont írása közben szakad meg a művelet, a rendszer a régi gyökércsomópontot használva konzisztens, korábbi állapotba kerül vissza – pontosan úgy, mint egy adatbázisban a tranzakciós commit.

A CoW tehát nem csupán egy optimalizáció, hanem a fájlrendszer teljes tervezési elvévé válik: a lemezen élő blokkok gráfja mindig konzisztens marad, ha a gyökércsomópont csere atomikus. Ezt a tulajdonságot gyakran „copy-on-write tranzakciós szemantikának” vagy „copy-on-write fának” nevezik.

16.2.2 Btrfs: a Linux natív CoW fájlrendszere

A Btrfs (B-tree file system) a Linux kernelbe integrált, CoW-alapú fájlrendszer. A belső struktúrája szinte kizárólag B-fákra épül. A főbb fa típusok:

- **Root tree**: a többi fa gyökerét tartalmazza.
- **Extent allocation tree**: a szabad és foglalt extents-eket kezeli.
- **Checksum tree**: az adatblokkok checksumjait tárolja.
- **Chunk tree**: a lemezterület logikai blokkosztását írja le.
- **Subvolume tree**: az egyes subvolume-ok fájlszerkezetét adja meg.

A Btrfs minden adatblokkhoz és metaadatblokkhoz egy checksumot rendel. A checksum tipikusan CRC-32C, de a kernelverziótól függően más algoritmus is választható. A checksumok a checksum tree-ben élnek, így olvasáskor a rendszer minden blokk ellenőrzését elvégezheti, és ha az ellenőrzés meghiúsul, hibát jelez.

A Btrfs a lemezt nem közvetlenül particionálja, hanem egy **pool**-ba fogja össze a blokkeszközöket. A pool lehet egyetlen lemez vagy több lemezből álló RAID-szerű elrendezés (Btrfs RAID0, RAID1, RAID10, RAID5, RAID6 – bár a RAID5/6 írási útvonala régebben instabil volt, ezért használat előtt ellenőrizni kell az adott kernel dokumentációját). A pool-on belül **subvolume**-ok hozhatók létre, amelyek önálló fájlrendszerként mountolhatók. A subvolume nem partíció, hanem a pool egy logikai névtere, amelyet snapshotokként lehet kezelni.

A Btrfs írási útvonala jól szemlélteti a CoW-ot. Tegyük fel, hogy egy fájl egyetlen adatblokkját szeretnénk módosítani. A folyamat:

1. Az alkalmazás a `write()` rendszerhívással adatot ír a fájlba.
2. A fájlrendszer megkeresi a fájl adatblokkjának jelenlegi fizikai címét (például `0x0000_8400`).
3. Kiválaszt egy szabad blokkot (például `0x0001_A000`), és az új tartalmat oda írja.

4. Kiszámolja az új blokk checksumját, és beilleszti a checksum tree-be.
5. Frissíti a fájl inode-jának extent listáját: a régi `0x0000_8400` bejegyzést lecseréli `0x0001_A000`-re.
6. Az inode új verziója is új blokkba kerül, a felette lévő fa csomópontok is frissülnek.
7. Végül a root tree gyökércsomópontjának cseréjével a módosítás atomikusan aktívvá válik.

A régi `0x0000_8400` blokk továbbra is a lemezen marad, de a hivatkozásai már nem mutatnak rá. Ha volt egy snapshot, amely rögzítette a módosítás előtti állapotot, akkor a snapshot még mindig a régi blokkra hivatkozik, így a snapshot adatai változatlanok maradnak. A blokk tényleges felszabadítása csak akkor történik meg, ha az összes hivatkozás megszűnt – ezt a referenciaszámláló kezeli.

A Btrfs gyakorlati előnyei közé tartozik az online snapshot, a subvolume-ok, a beépített RAID, a checksum és a scrub, valamint a beépített tömörítés (például zstd). Hátránya, hogy a belső műveletek (különösen a fák kiegyensúlyozása és a tömörítés) bonyolultabbak, és a régebbi kernelverziókban néhány funkció kevésbé stabil volt. A jelenlegi állapotról a hivatalos kernel-dokumentáció és a btrfs.readthedocs.io ad megbízható képet (a linken elérhető tartalom nem része ennek a könyvnek, és a verziófüggő állításokat ellenőrizni kell).

16.2.3 ZFS: a pool és a dataset modell

A ZFS a Sun Microsystems-től származó, jelenleg az OpenZFS projekt által fejlesztett fájlrendszer és logikai kötetkezelő kombinációja. A ZFS alapfilozófiája, hogy a lemezkezelés és a fájlrendszer nem választható szét: a rendszer egy **storage pool**-t épít fel, amelyben a fizikai lemezeket **vdev**-ekbe (virtual device) szervezi, majd ezen a poolon belül hoz létre **dataset**-eket (fájlrendszereket vagy zvolokat – blokkeszközként viselkedő logikai köteteket).

A ZFS adatszerkezete szintén CoW-alapú, és a lemezen lévő összes adatot egy nagy, ellenőrzött fa rendszerben tárolja. A főbb komponensek:

- **Uberblock**: a root csomópont, amely atomikusan cserélhető; a legutóbbi konzisztens állapot meghatározója.
- **Object set**: a fájlrendszer adatainak és metaadatainak gyűjteménye.
- **DMU (Data Management Unit)**: a blokkok kezeléséért felelős réteg.
- **ZIL (ZFS Intent Log)**: a szinkron írások naplója, amely a rendszer összeomlása esetén a tranzakciókat visszajátssza.
- **ARC (Adaptive Replacement Cache)**: a gyorsítótár.

A ZFS minden blokkhoz erős checksumot rendel (alapértelmezetten fletcher-4 vagy SHA-256, a konfigurációtól függően). A checksum a blokk címével együtt kerül tárolásra, így a véletlen bitek megváltozása (bitrot) észlelhető. A ZFS a redundáns vdev-ek (mirror vagy RAID-Z) esetén a hibás blokkot automatikusan kijavítja a másik példányból.

A ZFS snapshot ugyanazon CoW elv alapján működik: a snapshot nem másolja az adatblokkokat, csak rögzíti a fájlrendszer akkori állapotát leíró metaadatfát. A módosítások új blokkokba kerülnek, a régi blokkokat a snapshot továbbra is hivatkozza. A snapshot létrehozása pillanatszerű, és kezdetben nem foglal extra helyet. Ahogy a fájlrendszer változik, a snapshot által hivatkozott, de az aktív állapotból már eltűnt blokkok a snapshot életben tartása miatt nem szabadulnak fel, így a helyfoglalás fokozatosan nő.

A ZFS pool-kezelése rugalmas: a poolhoz dinamikusan adhatók hozzá vdev-ek, és a pool területét a dataset-ek között lehet felosztani. A ZFS nem igényel külön fájlkiosztási táblát, mert a pool maga a fájlrendszer. Ez a „pool storage” megközelítés nagyban leegyszerűsíti az adminisztrációt,

és lehetővé teszi a beépített tömörítést, a deduplikációt, a titkosítást és a rejtett hibák elleni scrubot.

16.2.4 Checksum és adatintegritás

A hagyományos fájlrendszerek nem ellenőrzik az adatblokkok tartalmát olvasáskor. Ha egy bit megváltozik a lemezen (például a lemez öregedése vagy kozmikus sugárzás miatt), a hibát a fájlrendszer nem észleli, és a hibás adatot adja vissza az alkalmazásnak. Ezt a jelenséget **silent data corruption**-nak nevezik.

A CoW-alapú fájlrendszerek – mint a Btrfs és a ZFS – minden adatblokkhoz és metaadatblokkhoz ellenőrző összeget tárolnak. A checksum számítása a blokk tartalmán történik, és az értéket a fájlrendszer külön tárolja (Btrfs: checksum tree; ZFS: a blokk pointerében). Amikor a rendszer olvas egy blokkot, újraszámolja a checksumot, és összehasonlítja a tárolt értékkel. Ha a kettő eltér, a blokk sérült.

A checksum önmagában csak észleli a hibát, nem javítja. A javításhoz redundancia szükséges: vagy egy másik példány (mirror), vagy paritás (RAID-Z, RAID1/5/6). Ha a rendszer redundanciát talál, automatikusan kijavítja a sérült blokkot, és naplózza az eseményt. Ha nincs redundancia, a hiba jelzésre kerül, és az alkalmazás `-EIO` hibakódot kaphat.

A **scrub** egy háttérfolyamat, amely szisztematikusan bejárja a teljes poolt, minden blokkot beolvas, ellenőrzi a checksumokat, és javítja a hibákat. A scrub időigényes, de nem befolyásolja a fájlrendszer normál működését. A scrub különösen fontos az elöregedő lemezeknél, mert a hibákat még azelőtt felfedezi, hogy azok tartós adatvesztést okoznának. A Btrfs-ben a scrub a `btrfs scrub start` paranccsal indítható, a ZFS-ben pedig a `zpool scrub` paranccsal.

16.2.5 Snapshot és klónozás

A snapshot a fájlrendszer egy adott időpontbeli állapotának rögzítése. A CoW-nak köszönhetően a snapshot létrehozása nem jár adatmásolással: a rendszer csupán megjelöli a jelenlegi metaadatfát, és egy új gyökércsomópontot hoz létre, amely a régi állapotra mutat. A későbbi írások új blokkokba kerülnek, így a snapshot által hivatkozott blokkok változatlanok maradnak.

Példa: legyen egy fájlrendszer, amely egy `document.txt` fájlt tartalmaz, és a fájl adatblokkja a `0x1000` címen helyezkedik el. Készítünk egy snapshotot `snap1` néven. Ezután módosítjuk a `document.txt`-t. A módosított adat új blokkba kerül (`0x2000`), a fájl inode-ja frissül. A `snap1` továbbra is a régi `0x1000`-re mutat, így a snapshotban a fájl korábbi tartalma olvasható. A `snap1` létrehozásakor a lemez foglalt területe nem nő, csak a későbbi módosítások során a régi blokkok a snapshot miatt nem szabadulnak fel.

A Btrfs-ben a snapshot egy subvolume-ból készíthető a `btrfs subvolume snapshot` paranccsal. A ZFS-ben a `zfs snapshot` parancs hozza létre. A klónozás egy snapshotból kiinduló, írható új dataset létrehozását jelenti; ez is CoW-alapon működik, és kezdetben nem foglal extra helyet.

A snapshotok fontos felhasználási esetei:

- **Biztonsági mentés**: a fájlrendszer állapotának rögzítése egy adott időpontban.
- **Tesztek**: a rendszer módosítása előtt snapshot készítése, majd szükség esetén visszaállítás.
- **Adatvédelem**: véletlen törlés vagy módosítás esetén a korábbi állapot visszanyerése.

A snapshotok kezelése azonban erőforrás-igényes lehet, ha sok snapshotot tartunk fenn, mert minden snapshot referenciát tart a blokkokon, és ezek a blokkok nem szabadulnak fel, amíg a snapshot él. A lemezterület felügyeletét ezért a referenciaszámláló és a háttérbeli felszabadítás (Btrfs: `btrfs balance`; ZFS: `zfs destroy`) végzi.

16.2.6 Az írási út vázlata és a labor

A következő labor célja, hogy a tanuló saját maga is megfigyelje a CoW írási útvonalát, és összehasonlítsa a mért eredményeket az elméleti várakozással. A labor **emulált környezetben** (például QEMU virtuális gépben) futtatható, és tilos éles rendszeren, valós lemezen vagy partíción végrehajtani. A leírás a Btrfs-sel dolgozik, de ZFS esetén hasonló lépések alkalmazhatók.

16.2.7 Labor: CoW írási út megfigyelése Btrfs-sel

Kiinduló állapot

- Virtualizált környezet: QEMU/KVM, 1 vCPU, 2 GiB RAM, 8 GiB virtuális lemez (a lemez tartalma nem tartalmaz valós adatot).
- Operációs rendszer: minimális Linux (pl. Debian alapú), kernel 6.1-es sorozat (a pontos verzió a reprodukciós csomagban rögzített).
- Fájlrendszer: a teljes virtuális lemezen Btrfs, mountolva a `/mnt` könyvtárba.

Előkészítés

```
# A virtuális lemez megformázása Btrfs-sel (csak emulált környezetben!)
mkfs.btrfs -f /dev/vdb
mount /dev/vdb /mnt

# Tesztfájl létrehozása
dd if=/dev/urandom of=/mnt/testfile bs=1M count=10
sync

# Snapshot készítése
btrfs subvolume snapshot /mnt /mnt/snap1
```

Mérési pontok

1. **Snapshot előtti és utáni szabad terület**. A snapshot létrehozása után a `df -h /mnt` kimenete nem változik, mert a snapshot nem foglal le új blokkokat. A várt eredmény: a `df` által mutatott `Used` érték ugyanaz. A mért értéket a `df -h /mnt` parancs kimenetéből kell rögzíteni.
2. **Módosítás utáni szabad terület**. A `testfile` tartalmának felülírása új blokkok foglalását eredményezi. A várt eredmény: a `Used` érték nő, mert a régi blokkok a snapshot miatt nem szabadulnak fel. A mért növekedés nagyjából megegyezik a módosított adat méretével, de a pontos érték a blokkmérettől és a tömörítéstől függ.
3. **A snapshot tartalma**. A `cmp` parancs a snapshotban lévő fájlt összehasonlítja a módosítás előtti tartalommal. A várt eredmény: a `cmp /mnt/snap1/testfile /mnt/testfile` eltérést jelez, mert a snapshot a régi állapotot tartalmazza.

4. **A CoW működésének bizonyítása**. A Btrfs beépített eszköze a `btrfs inspect-internal logical-resolve` vagy a `btrfs filesystem du` segítségével megmutatható, hogy a fájl mely fizikai blokkokat használ. Az egyszerűbb módszer: a `filefrag -v` parancs mutatja a fizikai extenteket. A várt eredmény: a módosított fájl extent listája megváltozik, az új blokkok címe eltér a régiektől.

Mérési napló (példa)

```
Időpont: 2025-02-20 10:15:00
Környezet: QEMU, Linux kernel 6.1.42, Btrfs progs v6.1
Snapshot előtt df:  Size=8.0G Used=10.0M Avail=7.9G
Snapshot után df:   Size=8.0G Used=10.0M Avail=7.9G
Módosítás után df:  Size=8.0G Used=20.0M Avail=7.8G
Várt:               Used növekedés = 10 MiB (új adatblokkok)
Mért:               Used növekedés = 10 MiB (megegyezik)
cmp eredmény:       eltér (snapshot régi, aktív új)
```

A labor sikeresnek tekinthető, ha a mért értékek a várt tartományba esnek, és a snapshot tartalma megegyezik a módosítás előtti állapottal. A mérési naplót minden esetben el kell menteni, és a konfigurációt (kernelverzió, Btrfs verzió, QEMU paraméterek) dokumentálni kell.

16.2.8 Gyakori hibák és helyes megoldások

A CoW-alapú fájlrendszerek használata során több tipikus félreértés és hiba fordul elő:

1. **A snapshot nem teljes másolat.** Gyakori hiba, hogy a felhasználók azt hiszik, a snapshot a fájlrendszer teljes másolatát tárolja, ezért azonnal dupla helyet foglal. Valójában a snapshot kezdetben nem foglal extra helyet; a helyfoglalás csak a későbbi módosításokkal növekszik. A helyes megoldás a `df` és a `btrfs filesystem usage` / `zfs list` rendszeres ellenőrzése.

2. **A checksum nem javít redundancia nélkül.** A checksum csak a hiba észlelésére alkalmas. Ha a pool nem tartalmaz tükrözést vagy paritást, a scrub észleli a hibát, de nem tudja kijavítani. A helyes megoldás a redundáns vdev-ek használata, ha az adatvesztés elfogadhatatlan.

3. **A régi blokkok felszabadulásának félreértése.** A snapshot törlése után a blokkok nem azonnal szabadulnak fel; a háttérbeli felszabadítás aszinkron. Btrfs-nél a `btrfs balance` vagy a `btrfs filesystem defragment` segíthet, de a felszabadulás nem garantált azonnali. A helyes megoldás a rendszeres karbantartás és a szabad hely figyelése.

4. **A scrub nem fut automatikusan.** A scrub folyamatot általában ütemezni kell (cron, systemd timer). A helyes megoldás a rendszeres scrub ütemezése, például havonta egyszer.

5. **A tömörítés és a deduplikáció költsége.** Bár a Btrfs és a ZFS támogatja a beépített tömörítést és deduplikációt, ezek CPU- és memóriaigényesek lehetnek. A helyes megoldás a teljesítményteszt elvégzése és a funkciók tudatos engedélyezése.

6. **A kernelverziótól függő stabilitás.** A Btrfs RAID5/6 írási útvonala korábban instabil volt; a használat előtt ellenőrizni kell az adott kernel dokumentációját. A helyes megoldás a stabilnak jelölt funkciók használata, vagy a ZFS választása, ahol a RAID-Z érett.

16.2.9 Önellenőrző kérdések

1. Miért nem írja felül a CoW fájlrendszer a meglévő blokkokat? Milyen előnye származik ebből a crash utáni konzisztencia szempontjából?
2. Hogyan működik a snapshot egy CoW rendszerben? Miért nem foglal helyet a létrehozás pillanatában?
3. Mi a szerepe a checksumnak és a scrubbnak? Miért nem elegendő a checksum önmagában a hiba javításához?
4. A laborban miért nő a `df` által mutatott `Used` érték a módosítás után, ha a fájl mérete nem változott?
5. Milyen körülmények között érdemes ZFS-t választani Btrfs helyett, és fordítva?

A fejezet folytatásában – a következő egységben – a zárt forrású fájlrendszerek (NTFS, APFS) és a hálózati fájlrendszerek kerülnek tárgyalásra, amelyek közül néhány szintén alkalmaz CoW-ot vagy hasonló adatintegritási mechanizmusokat.

16.3 NTFS, APFS és zárt rendszerek

Ez az alfejezet a Windows és Apple ökoszisztémák két meghatározó fájlrendszerét, az NTFS-t és az APFS-t mutatja be. A cél nem a belső implementáció teljes rekonstrukciója – mindkét formátum zárt vagy részben zárt specifikációjú –, hanem azoknak a fogalmi és gyakorlati jellemzőknek a tárgyalása, amelyek a rendszertervező, az üzemeltető vagy a beágyazott fejlesztő munkájához szükségesek. A fejezet az `os059`-ben bevezetett copy-on-write, metaadat- és fájlrendszer-szemantikai alapokra épít, ezért azokat nem ismétli meg. Az itt bemutatott kompatibilitási és licenckérdések közvetlenül előkészítik az `os061` hálózati fájlrendszerekkel foglalkozó egységet.

16.3.1 NTFS: a Windows elsődleges fájlrendszere

A New Technology File System (NTFS) a Microsoft Windows NT család elsődleges fájlrendszere. Első változata az 1993-ban kiadott Windows NT 3.1-gyel jelent meg, és azóta folyamatosan bővült: a Windows 2000-ben bevezetett NTFS 3.0, a Windows XP-ben megjelent NTFS 3.1 a mai napig alapja a modern Windows-verzióknak. Az NTFS tervezési céljai között szerepelt a hibatűrés, a nagy kötet- és fájlméretek támogatása, a hozzáférés-szabályozás, a naplózás, a titkosítás és a tömörítés. A fájlrendszer zárt, a Microsoft által publikált specifikáció nem teljes; a kompatibilis implementációk részben dokumentáció, részben tiszta szobás visszafejtés és hosszú tesztelés eredményei.

16.3.2 Az MFT és az attribútummodell

Az NTFS központi metaadat-struktúrája a **Master File Table (MFT)**. A kötet formázásakor létrejön egy rendszerfájl, `$MFT`, amely minden fájlhoz, könyvtárhoz és magához az MFT-hez is egy-egy rekordot tartalmaz. Minden MFT-rekord alapértelmezés szerint 1024 bájt, de a méret a kötet formázásakor 512 és 4096 bájt között állítható. A rekord elején egy fix fejléc, utána attribútumok sorozata található.

Az NTFS szinte minden fájlhoz köthető információt **attribútumnak** tekint. A fontosabb attribútumtípusok:

- `$STANDARD_INFORMATION` – időbélyegek, attribútumjelzők, hivatkozásszámok.

- `$FILE_NAME` – a fájl neve Unicode formában, valamint szülőkönyvtár-hivatkozás és további időbélyegek.
- `$DATA` – a tényleges fájl tartalma vagy az adatfutamok leírása.
- `$INDEX_ROOT`, `$INDEX_ALLOCATION`, `$BITMAP` – könyvtárbejegyzések rendezett fastruktúrái.
- `$SECURITY_DESCRIPTOR` – hozzáférés-vezérlési lista (ACL) tárolása.
- `$EA`, `$EA_INFORMATION` – kiterjesztett attribútumok.
- `$OBJECT_ID` – egyedi objektumazonosító.

Az attribútum lehet **rezidens** vagy **nem rezidens**. Ha az érték kicsi (például egy rövid `$FILE_NAME` vagy egy pár bájtos `$DATA`), akkor közvetlenül az MFT-rekordban tárolódik. Nagy adat esetén az attribútum fejlécében **data run**-ok (futamok) sorozata írja le, hogy a tartalom mely fizikai blokktartományokban helyezkedik el. Ez a kialakítás gyors hozzáférést ad a kis fájlokhoz, mert azok teljes egészükben az MFT-ben lehetnek, ugyanakkor támogatja a ritkított (sparse) fájlokat is, ahol bizonyos tartományok nem foglalnak fizikai területet.

Az NTFS egy jellegzetes, gyakran félreértett szolgáltatása az **alternatív adatfolyam** (Alternate Data Stream, ADS). Egy fájlnak több `$DATA` attribútuma lehet, mindegyikhez név tartozik. A névtelen `$DATA` a „normál” tartalom, a névvel ellátottak pedig rejtett adatfolyamok, amelyeket a Windows `fájlnév:folyam` szintaxissal ér el. Az ADS hasznos metaadatok tárolására (például a böngésző letöltési zónája), de biztonsági szempontból kockázatot is jelenthet, mert a hagyományos könyvtárlistákban nem látszik.

Az NTFS az MFT-rekordok sérülésének csökkentésére fenntart egy `$MFTMirr` nevű rendszerfájlt, amely az MFT első néhány rekordjának tükrözése. A `$Bitmap` a kötet szabad és foglalt blokkjait követi, a `$UpCase` a kis- és nagybetű-átalakításhoz szükséges táblázat, az `$Secure` pedig a biztonsági leírók központi tárolója.

16.3.3 Journal és helyreállítás

Az NTFS naplózása az úgynevezett **Log File Service (LFS)** segítségével történik. A `$LogFile` rendszerfájl tranzakciós naplóként működik: a metaadat-műveletek előtt a rendszer előkép- és utókép-rekordokat ír a naplóba, majd a tényleges módosítást a kötet megfelelő struktúráiban is elvégzi. Ha a rendszer a művelet közben áll le, a következő csatoláskor az NTFS újrajátssza a befejezett tranzakciókat, és visszagörgeti a félbemaradtakat. Fontos hangsúlyozni, hogy az NTFS naplózása alapvetően **metaadat-naplózás**: a `$LogFile` a fájlrendszer-struktúrák konzisztenciáját védi, nem garantálja a fájl tartalmának teljes tranzakciós integritását. Egy hirtelen áramszünet után a metaadatok konzisztensek, de az éppen írt fájl tartalmában lehetnek részleges vagy régi blokkok.

A napló mérete és viselkedése a rendszer verziójától és a kötet terhelésétől függ. Windows alatt a naplózás alapértelmezésben be van kapcsolva; az `fsutil` vagy a `chkntfs` segítségével lehet informálódni, de közvetlen kézi módosítása éles rendszeren nem ajánlott. Linux alatt a napló elemzésére a `ntfsinfo`, `ntfsls` vagy a `ntfs-3g` eszközei használhatók.

16.3.4 Kompatibilitás és licenc

Az NTFS szabadalmazott, zárt formátum. A Microsoft bizonyos dokumentációkat nyilvánosságra hozott, de a teljes specifikáció nem érhető el, és a formátum egyes részeire szabadalmak vonatkoznak (a szabadalmi helyzet időben változó, ezért aktuális jogi értékeléshez friss forrást kell ellenőrizni). Linuxon a gyakorlatban két érett implementáció létezik:

- **ntfs-3g** – FUSE-alapú, felhasználói térben futó illesztő, amely olvasást és írást is támogat. Hosszú ideig a legmegbízhatóbb nyílt NTFS-írás volt, teljesítménye azonban elmarad a kernelbeli megoldásoktól.
- **ntfs3** – a Linux kernelbe integrált, Paragon által fejlesztett illesztő, amely a modern kernelekben natív olvasást/írást kínál, jobb teljesítménnyel. Elérhetősége és funkciókészlete kernelverziótól függ, ezért mindig ellenőrizni kell az aktuális rendszer dokumentációját.

A zárt formátum miatt az NTFS-en tárolt adatok hordozhatósága erős, a teljes funkcionalitás azonban csak Windows alatt garantált. A Linuxos írási támogatás a gyakorlatban kiforrott, de bizonyos ritka NTFS-jellemzők (például titkosítás, kvóta, objektumazonosító-alapú tranzakciók) nem vagy csak korlátozottan érhetők el.

16.3.5 APFS: az Apple File System

Az Apple File System (APFS) az Apple által 2017-ben bevezetett fájlrendszer, amely a korábbi HFS+ helyét vette át az iOS, macOS, watchOS és tvOS rendszereken. Az APFS tervezésének elsődleges motivációja a flash-alapú tárolók (SSD-k) hatékony kiszolgálása, a modern hardveres titkosítás, a copy-on-write, a pillanatképek és a rugalmas kötetkezelés volt. Az APFS szintén zárt, bár az Apple 2018-ban közzétett egy referenciadokumentumot, amely a formátum alapvető struktúráit leírja; a teljes specifikáció és a hivatalos implementáció forráskódja nem nyilvános.

16.3.6 Container, volume és space sharing

Az APFS legfontosabb architekturális eltérése a hagyományos fájlrendszerektől a **container–volume** modell. Egy APFS container egy közös tárolóterületet fog össze, amelyen belül több **volume** hozható létre. A volume-ok nem fix partícióhatárokkal rendelkeznek, hanem megosztják a container szabad területét; ezt a mechanizmust **space sharing**-nek nevezik. Például egy Mac gépen a rendszerkötet és az adatkötet ugyanabban a containerben lehet, és a szabad területet dinamikusan használják fel.

A container elején egy **container superblock** található, amely a container-szintű metaadatokra mutat. Minden volume-nak van saját **volume superblock**-ja és **checkpoint**-jai. Az APFS a térleképezést B-tree struktúrák segítségével végzi: a **fizikai extent-fa** a blokkok fizikai kiosztását, a **fájlfa** a fájlok és könyvtárak logikai szerkezetét írja le. A checkpoint egy konzisztens állapotot rögzít, és a tranzakciós frissítések során az APFS először az új adatokat írja ki, majd egyetlen atomi mutatóváltással frissíti a checkpointot. Ez a **copy-on-write** stratégia egyúttal a crash protection alapja is: ha a rendszer a frissítés közben áll le, a régi checkpoint még érvényes marad.

16.3.7 Klónozás és pillanatképek

Az APFS egyik legismertebb szolgáltatása a **fájlklónozás** (clone). A klón nem egyszerű másolat: a `clonefile()` rendszerhívással vagy a `cp -c` paranccsal létrehozott új fájl kezdetben ugyanazokra a fizikai blokkokra mutat, mint az eredeti. Amikor valamelyik klónon írás történik, a copy-on-write mechanizmus csak a módosított blokkokat másolja le. Ezáltal egy nagy fájl klónozása gyakorlatilag azonnali, és a szükséges tárterület is minimális mindaddig, amíg az egyes klónok nem térnek el egymástól. A klónozás nem keverendő össze a hard linkkel: a hard link ugyanazt a fájlbejegyzést több névvel, míg a klón külön fájlbejegyzést hoz létre, amelynek tartalma kezdetben megosztott.

A **snapshot** (pillanatkép) volume-szintű, írásvédett, időponthoz kötött állapot. A pillanatkép ugyancsak a copy-on-write-ra épül: a készítés pillanatában a volume metaadatai és adatblokkjai megőrződnek, a későbbi módosítások pedig új blokkokba íródnak. A pillanatkép tehát logikai törlés vagy hibás frissítés ellen véd, de nem helyettesíti a lemezhiba elleni biztonsági mentést, mert ugyanazon a fizikai tárolón él. A macOS Time Machine és számos biztonsági mentési eszköz APFS-pillanatképekre épül.

16.3.8 Titkosítás és integritás

Az APFS a titkosítást volume-szinten támogatja. A titkosítási kulcsokat a Secure Enclave vagy a felhasználói jelszó védelme alatt kezelik, az adatblokkok pedig jellemzően AES-XTS módban titkosítottak. A titkosítási képességek pontos algoritmusa és kulcskezelése platformfüggő, ezért a hordozható lemezeken használt APFS esetében is az Apple eszközökön való működés a referencia.

Az APFS integritáskezelése a metaadatokra terjed ki: a B-tree csomópontok ellenőrző összegeket tartalmaznak, a sérült metaadatok észlelhetők. Az adatblokkokra vonatkozó ellenőrző összegek nem minden APFS-változatban és konfigurációban kötelezők, ellentétben például a ZFS-sel, ahol a teljes adatút checksummal védett.

16.3.9 Kompatibilitás és licenc

Az APFS propietáris fájlrendszer. Az Apple specifikációja korlátozott, és a hivatalos implementáció kizárólag az Apple operációs rendszereiben érhető el. Linuxon nincs teljes értékű, írást is támogató nyílt implementáció. A leggyakoribb megoldás az **apfs-fuse** vagy a **libfsapfs** projekt, amelyek jellemzően csak olvasási támogatást nyújtanak, és a funkciókörük, valamint a friss verziók elérhetősége folyamatos ellenőrzést igényel. A zárt formátum miatt az APFS-kötetek hordozhatósága korlátozott: egy macOS rendszerről eltávolított lemez Linux alatt olvasása nem garantált, és a sérülékeny vagy régi eszközökkel való írás kockázatos lehet.

16.3.10 NTFS és APFS összehasonlítása

Az alábbi táblázat a két fájlrendszer legfontosabb jellemzőit veti össze. A lista nem teljes, és a képességek támogatása verziófüggő; az aktuális állapotot mindig a gyártó dokumentációjában kell ellenőrizni.

Jellemző	NTFS	APFS
Elsődleges platform	Windows	macOS, iOS, watchOS, tvOS
Naplózás	Metaadat-napló (`$LogFile`, LFS)	Copy-on-write checkpoint-ok
Fájlklónozás	Nem natív (bár Windows Server egyes verziókban van block cloning, de nem azonos szemantikájú)	Natív, `clonefile()` és COW
Pillanatkép	Volume Shadow Copy (VSS), nem fájlrendszer-szintű	Natív, volume-szintű snapshot
Titkosítás	EFS (fájl- vagy mappaszintű), BitLocker (kötetszintű)	Volume-szintű, natív
Tömörítés	Natív, átlátszó (LZNT1/XPRESS)	Nincs beépített átlátszó tömörítés

Jellemző	NTFS	APFS
Alternatív adatfolyam	Igen (`fájl:stream`)	Nincs (kiterjesztett attribútumok és resource fork helyett)
Hard link	Igen	Igen (de a szemantika eltérhet)
Sparse fájl	Igen	Nincs klasszikus sparse, de a COW és a clone hasonló hatást ad
Zártság	Zárt, részleges dokumentáció, szabadalmak	Zárt, korlátozott dokumentáció
Linux támogatás	Olvasás/írás (ntfs3, ntfs-3g)	Jellemzően csak olvasás (apfs-fuse, libfsapfs)

A táblázat jól mutatja a két rendszer eltérő filozófiáját: az NTFS a Windows ökoszisztéma hagyományos, naplózott, gazdag metaadatú fájlrendszere, míg az APFS a flash-tárolókra optimalizált, COW-alapú, natív klónozást és pillanatképet kínáló rendszer.

16.3.11 Tipikus hibák és helyes megoldások

1. Hiba: Az NTFS naplóját teljes adatpéldánynak tekintik. **Helyes megoldás:** Az NTFS `$LogFile` metaadat-napló. A metaadatok konzisztenciáját védi, de a fájl tartalma nem feltétlenül tranzakciósan konzisztens. Kritikus adatoknál a fájl szintű szinkronizációt (`fsync`), illetve a megfelelő Windows API-kat (`FlushFileBuffers`) kell használni.

2. Hiba: Az APFS klónt hard linknek tekintik, és a módosításokat mindkét néven látni várják. **Helyes megoldás:** A klón külön fájlbejegyzés, amely kezdetben megosztja a blokkokat. Az egyik klón módosítása nem változtatja meg a másik tartalmát, de a helyfoglalás csak a módosított blokkok mértékéig nő. A hard linkkel ellentétben a klónozott fájlok törlése egymástól független.

3. Hiba: Az APFS pillanatképet teljes biztonsági mentésnek gondolják. **Helyes megoldás:** A pillanatkép logikai törlés vagy véletlen módosítás ellen véd, de ugyanazon a fizikai lemezen tárolódik. Lemezhiba, tűzkár vagy zsarolóvírus ellen nem nyújt védelmet. A pillanatképet mindig egészítse ki külső, legalább 3-2-1 elvű mentéssel.

4. Hiba: Linux alatt NTFS-re írva régi, elavult `ntfs-3g` verziót használnak, és teljesítmény- vagy stabilitásproblémát tapasztalnak. **Helyes megoldás:** Ellenőrizze a disztribúció csomagverzióját; modern kerneleken preferálja a `ntfs3` illesztőt, vagy frissítse az `ntfs-3g`-t. A csatolás előtt állítsa le a Windows gyorsindítást, mert a hibernált állapotú NTFS-kötet írása adatvesztéshez vezethet.

5. Hiba: Az APFS-t Linux alatt írásra használják egy harmadik féltől származó, nem támogatott eszközzel. **Helyes megoldás:** Az APFS írási támogatása Linuxon nem hivatalos és nem kiforrott. Csak olvasható, kísérleti környezetben szabad ilyen eszközöket használni, és soha ne írjunk éles vagy egyetlen példányban létező APFS-kötetre. A biztonságos adatcsere érdekében használjunk platform-semleges fájlrendszert (például exFAT) vagy hálózati megosztást.

16.3.12 Labor: NTFS- és APFS-jellemzők összevetése

A labor célja, hogy a hallgató reprodukálható módon hozzon létre és vizsgáljon meg egy NTFS- és egy APFS-kötetet, majd a megfigyelt jellemzőket összehasonlítsa a dokumentált elvárásokkal. A labor két részből áll: az NTFS rész teljes egészében Linux vendégen fut, az APFS részhez vagy macOS vendéget, vagy egy előre elkészített read-only APFS image-et használunk. A mérési naplóban minden parancs, verzió és eredmény szerepeljen.

16.3.13 Kiinduló állapot és konfiguráció

- Hoszt: QEMU/KVM, 8 GiB RAM, 4 vCPU, 64 GiB szabad lemezterület a munkakönyvtárban.
- Vendég: Linux disztribúció (például Debian 12), 2 GiB RAM, 2 vCPU.
- Szükséges csomagok: `ntfs-3g`, `ntfsprogs`, `apfs-fuse` vagy `libfsapfs-tools` (a pontos csomagnév disztribúciófüggő; a friss verziót ellenőrizni kell).
- macOS vendég esetén: legalább macOS 12, 4 GiB RAM, 2 vCPU, 64 GiB virtuális lemez.
- Minden image-fájl a vendég `/tmp` könyvtárában jön létre; a hoszt valódi lemezeit nem érintjük.

16.3.14 NTFS rész lépései

1. Hozz létre egy 64 MiB méretű üres image-et:

   ```
   dd if=/dev/zero of=/tmp/ntfs.img bs=1M count=64 status=progress
   ```

2. Formázd NTFS-re:

   ```
   mkntfs -F -q /tmp/ntfs.img
   ```

3. Csatold fel:

   ```
   mkdir -p /mnt/ntfs
   mount -o loop,noatime /tmp/ntfs.img /mnt/ntfs
   ```

4. Írj egy 1 MiB véletlen tartalmú fájlt:

   ```
   dd if=/dev/urandom of=/mnt/ntfs/test.bin bs=1M count=1
   ```

5. Készíts egy hard linket és egy alternatív adatfolyamot:

   ```
   ln /mnt/ntfs/test.bin /mnt/ntfs/test_hardlink.bin
   echo "rejtett metaadat" > "/mnt/ntfs/test.bin:stream"
   ```

6. Kérdezd le a fájl MFT-rekordját:

   ```
   ntfsinfo -m -i 30 /dev/loop0
   ```

   (Az inode-szám a fájl létrehozásakor kapott belső azonosító; ha nem ismert, előbb `ntfsls --inode /mnt/ntfs/test.bin`.)

7. Ellenőrizd a naplófájl méretét és a kötet szabad területét:

   ```
   ls -l /mnt/ntfs/\$LogFile
   df -h /mnt/ntfs
   ```

8. Válaszd le a kötetet:

   ```
   umount /mnt/ntfs
   ```

16.3.15 APFS rész lépései

A) macOS vendégen (ha rendelkezésre áll):

1. Hozz létre egy 64 MiB APFS image-et:

   ```
   hdiutil create -size 64m -fs APFS -volname LabVol /tmp/apfs.dmg
   ```

2. Csatold fel:

   ```
   hdiutil attach -nobrowse -mountpoint /Volumes/LabVol /tmp/apfs.dmg
   ```

3. Hozz létre egy 1 MiB fájlt:

   ```
   dd if=/dev/urandom of=/Volumes/LabVol/test.bin bs=1m count=1
   ```

4. Készíts klónt a `-c` kapcsolóval:

   ```
   cp -c /Volumes/LabVol/test.bin /Volumes/LabVol/test_clone.bin
   ```

5. Készíts pillanatképet a kötetről:

   ```
   tmutil snapshot /Volumes/LabVol
   ```

   vagy

   ```
   diskutil apfs snapshot /Volumes/LabVol
   ```

6. Listázd a volume és a snapshot adatait:

   ```
   diskutil apfs list /Volumes/LabVol
   df -h /Volumes/LabVol
   ls -la /Volumes/LabVol
   ```

7. Ellenőrizd, hogy a klón létrehozása után a szabad terület alig változott-e (a megosztott blokkok miatt). Ezt mérési naplóban rögzítsd: a `df -k` kimenete a klón előtt és után.

B) Linuxon read-only eszközzel (ha nincs macOS vendég):

Ha APFS írást nem tudunk elvégezni, akkor egy előre elkészített, read-only APFS image-et elemzünk. Az image létrehozása macOS-en történt, és a laborhoz mellékeljük. A jellemzők lekérdezéséhez használjuk a `fsapfsinfo` vagy az `apfs-fuse` read-only csatolását:

```
mkdir -p /mnt/apfs
apfs-fuse /tmp/apfs.dmg /mnt/apfs
find /mnt/apfs -maxdepth 2 -type f -ls
```

A mérési naplóban rögzítsük, hogy az APFS-rész read-only elemzés volt, és a klónozási jellemzőt a dokumentációból vagy a macOS alatt készített mérésből ismerjük.

16.3.16 Elvárt eredmény és mérési napló

Az alábbi táblázat a mért és a várt eredményeket foglalja össze. A „mért” oszlopba a tényleges parancskimenetekből kinyert értékek kerülnek, a „várt” oszlopba a fejezetben és az `os059`-ben bemutatott elméleti modell alapján levezetett érték.

Vizsgált jellemző	Várt eredmény	Mért eredmény	Megjegyzés
NTFS `$DATA` attribútum rezidens/nem rezidens	1 MiB fájl esetén nem rezidens		A data run-ok száma a töredezettségtől függ
NTFS ADS láthatósága `ls` alatt	Nem látszik a normál listában, csak `getfattr` vagy Windows `dir /r`		Az ADS a `test.bin:stream` néven érhető el
NTFS `$LogFile` mérete	Több MiB, de pontos méret változó		A napló mérete a tranzakcióktól függ
APFS klón helyfoglalása	A klón létrehozása után a szabad terület közel változatlan (a megosztott blokkok miatt)		A klón utáni első írás növeli a foglalást
APFS snapshot jelenléte	`diskutil apfs list` mutatja a pillanatképet		A pillanatkép csak olvasható

A mérési naplóhoz csatolni kell a `mkntfs --version`, `ntfs-3g --version`, `apfs-fuse --version` (ha elérhető) és a kernelverzió kimenetét. A naplóban szerepeljen a mérés időpontja, a vendég konfigurációja, valamint az image-fájlok SHA-256 összege a reprodukálhatóság érdekében.

16.3.17 Visszaállítás

- A vendég leállítása előtt válassz le minden csatolt kötetet (`umount`).
- Töröld a `/tmp/ntfs.img`, `/tmp/apfs.dmg` fájlokat.
- Ha macOS vendéget használtál, töröld a snapshotot a `diskutil apfs deleteSnapshot` paranccsal, majd válaszd le az image-et.
- A hoszt valódi lemezét, bootloaderét ne módosítsd.

16.3.18 Önellenőrző kérdések

1. Mi a különbség az NTFS `$LogFile` naplózása és az APFS copy-on-write checkpoint mechanizmusa között a crash utáni helyreállítás szempontjából?
2. Miért nem tekinthető az APFS-pillanatkép teljes értékű biztonsági mentésnek?
3. Hogyan viszonyul az APFS fájlklón a hard linkhez, és mi a gyakorlati következménye a tárhelyfoglalásra?

A következő egység (`os061`) a hálózati fájlrendszerekre tér át, és az itt bemutatott kompatibilitási rétegeket, valamint a zárt formátumok hálózaton keresztüli elérését tárgyalja.

16.4 Hálózati fájlrendszerek: NFS, SMB és objektumtár

A hálózati fájlrendszerek célja, hogy a fájlabsztrakciót – nyitás, olvasás, írás, lezárás, metaadat-műveletek – egy távoli gépen tárolt adatra is kiterjessze. Ezzel a háttértároló fizikailag elválik

a kliensgéptől, és központilag menedzselhető, megosztható, replikálható és skálázható lesz. A hálózati tárolásnak három, jól elkülöníthető rétege van:

- **hálózati blokkeszköz**: a kliens nyers blokkokat lát (iSCSI, NVMe-oF), a fájlrendszert a kliens valósítja meg;
- **hálózati fájlrendszer**: a szerver exportál egy fájlrendszer-fát, a kliens fájlműveleteket küld a hálózaton (NFS, SMB);
- **objektumtár**: a kliens nem fájlokat, hanem objektumokat címez meg HTTP/REST API-val (S3, Swift).

Ez a fejezet a második és harmadik csoporttal foglalkozik. A blokkszintű távoli tárolást a tárolóeszközökről szóló rész (Tárolóeszközök: HDD, SSD, NVMe és buffering) tárgyalja. Az itt bemutatott fogalmakra épül a következő fejezet crash-konzisztencia- és naplózási anyaga, mert a hálózati fájlrendszereknél a szerveroldali állapot és a kliensoldali cache összehangolása ugyanazokat a tartóssági kérdéseket veti fel, mint egy lokális fájlrendszer.

16.4.1 A hálózati fájlrendszer modellje

A hálózati fájlrendszer protokollja két komponensből áll: egy **hívásrétegből** és egy **adatábrázolási rétegből**. Az NFS a Sun RPC (ONC RPC) és az XDR mechanizmust használja: a kliens egy eljáráshívást csomagol be, a szerver meghívja a megfelelő fájlműveletet, majd az eredményt visszaküldi. Az SMB ezzel szemben egy puffer-alapú, hitelesített kérés-válasz protokoll, amelyben a fájlműveletek a share fogalmára épülnek. Az S3 objektumtár pedig egy szokásos HTTP-vel megvalósított REST API, ahol az erőforrás az objektum, a műveletek pedig a `PUT`, `GET`, `DELETE` és `HEAD` metódusok.

A közös absztrakció a **fájlkezelő** (file handle). A kliens a nyitáskor kap egy, a szerver számára értelmes azonosítót, amely lehet:

- NFS-ben egy `struct nfs_fh` jellegű, a szerver által generált, átlátszatlan bájtfolyam;
- SMB-ben a fájl azonosítója, amelyhez a szerver állapotot (nyitott fájl, zárak, aktuális pozíció) rendel;
- S3-ban az objektum kulcsa, például `bucket/docs/report.pdf`.

A szerver a handle mögött a saját lokális fájlleíróját, inode-számát vagy egy generációs számmal ellátott belső rekordot tart nyilván. A handle érvényessége a szerver újraindulása után változhat: NFSv3-ban a file handle az exportált inode-ra mutat, ezért általában stabil, NFSv4-ben a szerver állapotgépe miatt a klienseknek újra kell csatlakozniuk, és a handle-ök újraérvényesítése szükséges.

16.4.2 NFS: Sun hálózati fájlrendszere

Az NFS a UNIX világ klasszikus megosztási protokollja. A korai NFSv2 és NFSv3 **közel állapotmentes** (stateless) szervert feltételezett: a szerver nem tartotta nyilván, melyik kliens melyik fájlt nyitotta meg. A fájlműveletek közvetlenül a handle-re hivatkoztak, a lockolást pedig külön protokollok (NLM, NSM) kezelték. Ez egyszerű helyreállítást tett lehetővé, de komoly cache-koherencia- és zárolási problémákat okozott.

NFSv4 alapvetően megváltoztatta a modellt: a szerver **állapottartó** (stateful). Bevezette a `open`, `close`, `lock`, `locku` műveleteket, integrálta a byte-range lockolást, és a kliensek **lease**-t kapnak. A lease egy időkorlát, amelyen belül a kliensnek meg kell újítania az állapotát. Ha a szerver nem kap megújítást, felszabadíthatja a zárakat és delegációkat. A szerver újraindulása

után **türelmi időszak** (grace period) következik, amely alatt a kliensek visszajátszhatják a záraikat, de új zárat nem kérhetnek. Ez a mechanizmus teszi lehetővé a szerveroldali állapot helyreállítását anélkül, hogy a klienseken manuálisan újra kellene játszani minden műveletet.

Az NFSv4.1 bevezette a **pNFS-t** (parallel NFS). A klasszikus NFS-ben minden adatforgalom a kiszolgálón keresztül ment, ami szűk keresztmetszetet jelentett. A pNFS-ben a szerver egy **layoutot** ad a kliensnek, amelyben megmondja, hogy az adott fájl blokkjai mely adatszervereken találhatók. A kliens ezután közvetlenül az adatszerverekkel kommunikál, párhuzamosan. A layout típusa lehet fájl, blokk vagy objektum, így a pNFS a hagyományos NFS-fejléc szervert tehermentesíti, miközben a metaadat-műveletek továbbra is a központi szerveren futnak.

Az NFS cache-modellje a **close-to-open consistency** elven alapul. Ez azt jelenti, hogy a kliens a fájl bezárásakor kiüríti az írási cache-ét a szerverre, a következő megnyitáskor pedig érvényteleníti a beolvasott adatokat, és friss attribútumokat kér. A nyitva tartott fájlok között a kliens használhatja a helyi cache-t anélkül, hogy minden olvasásnál a szerverhez fordulna. Ez jó teljesítményt ad, de nem biztosít POSIX-szemantikát két párhuzamosan író kliens esetén.

A finomabb cache-vezérlésre az NFSv4 **delegációt** vezetett be. A szerver átruházhatja egy fájl **olvasási** vagy **írási delegációját** egy kliensre. Amíg a delegáció él, a kliens helyben szolgálhatja ki az olvasásokat, és a szerver nem engedélyez más írót. Ha egy másik kliens megnyitja a fájlt, a szerver **visszahívást** (callback) küld a delegációt birtokló kliensnek, amelynek vissza kell adnia a delegációt, kiüríteni a módosításait, és a szerver ezután engedélyezi az új hozzáférést. A delegáció megszakadhat hálózati hiba esetén is; ilyenkor a kliens a lease lejárta után konzervatív módon, minden műveletnél a szervert kérdezi.

Az NFS lockolása **byte-range advisory lock**. A POSIX `fcntl` zárakat a szerver tartja nyilván, és a kliens rendszerhívása ezeket a hálózaton keresztül kéri le. A zárak lease-hez kötöttek: ha a kliens nem újítja meg, a szerver felszabadíthatja azokat. A szerver újraindulása utáni grace period alatt a kliensek `reclaim` művelettel visszaállíthatják a záraikat. A lockolás NFSv4-ben nem érzékeli a kliensek közötti holtpontot, ezért az alkalmazásoknak továbbra is saját deadlock-kezelést kell alkalmazniuk.

16.4.3 Gyakorlati NFS-konfiguráció

A laborban használt NFSv4 szerver exportál egy könyvtárat. A `/etc/exports` bejegyzés például:

```
# /etc/exports – csak emulált környezetben
/srv/nfs 192.0.2.20(rw,sync,no_subtree_check,fsid=0)
```

A `sync` opció arra utasítja a szervert, hogy a válasz elküldése előtt a módosításokat a stabil tárba írja. Az `async` opció gyorsabb, de szerverösszeomlás esetén adatvesztést okozhat. A `no_subtree_check` teljesítmény-optimalizáció, a `fsid=0` pedig az NFSv4 pszeudo-gyökér könyvtárát jelöli.

A kliens mountolása:

```
# Kliens oldal, emulált hálózat
mkdir -p /mnt/nfs
mount -t nfs4 -o rw,hard,intr,timeo=50,retrans=2 192.0.2.10:/srv/nfs /mnt/nfs
```

A `hard` mountolási opció azt jelenti, hogy a kliens a szerver elérhetetlensége esetén a műveletet végtelen ideig újrapróbálja, míg a `soft` opció hibát ad vissza. A `hard` alapértelmezett, mert a fájlrendszer-műveleteknél a csendes adatvesztésnél jobb a blokkolás.

16.4.4 SMB: a Windows világ megosztási protokollja

Az SMB (Server Message Block) a Microsoft és az IBM nevéhez fűződő protokollcsalád. A régi SMB1 (CIFS) mára elavult és biztonsági kockázatot jelent, ezért a modern rendszerek az SMB2-t és SMB3-at használják. A nyílt forráskódú implementáció a Samba, amely Linux és UNIX rendszereken kínál SMB-szervert és klienst.

Az SMB fogalmi modellje a **share**: a szerver egy könyvtárat oszt meg egy névvel, például `\\server\share`. A kliens authentikáció után csatlakozik a share-hez, és a fájlműveleteket a share-en belüli relatív útvonalon végzi. Az SMB protokoll a Windows ACL-modellt használja: a hozzáférés-vezérlési listák (ACL-ek) tartalmazzák a felhasználó- és csoportjogosultságokat, beleértve az öröklődést és a speciális engedélyeket. A UNIX/POSIX mód bites jogosultságokat a Samba leképezi, de a két modell nem fedi egymást pontosan.

Az SMB **oplocks** (opportunistic locks) mechanizmusa hasonló az NFS delegációhoz. A szerver engedélyezheti a kliensnek, hogy helyben cache-elje a fájlt, és csak akkor szól, ha egy másik kliens hozzáfér. Az oplock típusai:

- **exclusive oplock**: a kliens kizárólagos hozzáférést kap, írási cache-t használhat;
- **batch oplock**: a kliens a fájl többszöri megnyitását is cache-elheti;
- **level2 oplock**: több kliens olvashatja ugyanazt a fájlt, de egyik sem írhat.

Az SMB2-től kezdve az oplockot **lease**-nek nevezik, és további finomításokat tartalmaz. A **durable file handle** segítségével a kliens egy hálózati szakadás után is megtarthatja a fájlkezelőt, és a kapcsolat helyreállításakor folytathatja a műveletet anélkül, hogy újra meg kellene nyitnia a fájlt. Az SMB3 **transparent failover** funkciója lehetővé teszi, hogy a kliens egy másik szerverre kapcsoljon át, ha az eredeti elérhetetlenné válik, feltéve, hogy a szerverek klasztert alkotnak, és a share **continuously available** tulajdonságú.

Az SMB3 további képességei:

- **Multichannel**: a kliens több hálózati kapcsolatot használhat párhuzamosan, növelve az átviteli sebességet és a hibatűrést.
- **SMB Direct**: RDMA (Remote DMA) támogatás, amely alacsony késleltetést és nagy sávszélességet biztosít.
- **Encryption**: az SMB3 forgalom titkosítható, így a megosztott fájlok tartalma nem olvasható a hálózaton.

Az SMB esetében a konzisztencia-modell a **close-to-open** elvre épül, hasonlóan az NFS-hez, de az oplock/lease miatt a szerver aktívan felügyeli a kliens cache-ét. Amikor egy kliens írni szeretne egy fájlt, amelyet egy másik kliens olvasási oplockkal birtokol, a szerver visszavonja az oplockot, és a másik kliens frissíti a tartalmat. Ez erősebb koherenciát adhat, mint a hagyományos NFSv3.

16.4.5 Objektumtár: S3 és társai

Az objektumtár a fájlrendszertől eltérő absztrakciót nyújt. Nincs hierarchikus könyvtárfa, hanem **bucketek** és **objektumok** vannak. Egy objektumot egy bucket név és egy kulcs azonosít, például:

```
s3://backup/postgres/2025-01-15/full.dump
```

A műveletek HTTP-vel valósulnak meg:

```
PUT    /bucket/key            - objektum feltöltése
GET    /bucket/key            - objektum letöltése
HEAD   /bucket/key            - metaadatok lekérdezése
DELETE /bucket/key            - objektum törlése
```

Az S3 API de facto szabvány lett; a nyílt implementációk közé tartozik a MinIO, a Ceph RADOS Gateway és az OpenStack Swift. Az objektumtárak jellemzői:

- **Nincs inode vagy részleges frissítés**: az objektum egyben írható, a módosítás új objektumverziót hoz létre, ha a verziózás engedélyezett.
- **Metaadatok**: az objektumhoz tetszőleges felhasználói kulcs-érték párok rendelhetők, például `Content-Type`, `x-amz-meta-backup-date`.
- **Multipart upload**: nagy objektumokat részekre bontva, párhuzamosan lehet feltölteni, majd a szerver összefűzi azokat. Ez lehetővé teszi a megszakadt feltöltés folytatását és a sávszélesség jobb kihasználását.
- **Conditional write**: a kliens `If-Match` vagy `If-None-Match` fejléccel feltételhez kötheti a műveletet. Ez egy **compare-and-swap** mechanizmust ad, mert a szerver csak akkor hajtja végre az írást, ha az objektum ETag-je megegyezik a várt értékkel.
- **Object lock**: egyes szolgáltatások WORM (write once, read many) módot kínálnak compliance vagy governance üzemmódban, ami megtiltja az objektum törlését vagy módosítását egy adott időpontig.

A konzisztencia-modell hagyományosan: **read-after-write** konzisztencia új objektumokra, **eventual consistency** a már meglévő objektumok felülírására és törlésére. Ez azt jelenti, hogy ha egy kliens feltölt egy új kulcsot, majd azonnal olvassa, a friss tartalmat kapja; ha viszont felülír egy meglévő objektumot, előfordulhat, hogy egy ideig a régi verziót látja. Fontos azonban, hogy a szolgáltatók konzisztencia-garanciái időben változnak, és egyes nyilvános szolgáltatások ma már erős konzisztenciát hirdetnek minden műveletre. A pontos viselkedést a használt implementáció dokumentációjában kell ellenőrizni.

Az objektumtár hibatűrése tipikusan **replikációval** és **erasure codinggal** valósul meg. A replikáció során minden objektum több példányban létezik, az erasure coding pedig a RAID elvét terjeszti ki elosztott környezetre: az adatot k adatrészre és m paritásrészre bontja, és bármely m részből helyreállítható a teljes objektum. Ez jobb tárolási hatékonyságot ad, mint a teljes replikáció, de több CPU-t igényel a kódoláshoz és a helyreállításhoz.

16.4.6 Cache, lock és hibatűrés összehasonlítása

A három vizsgált rendszer cache- és zárolási modellje eltérő kompromisszumot köt a teljesítmény, a konzisztencia és a rendelkezésre állás között.

NFS a close-to-open konzisztenciát és a delegációt használja. A kliens helyi cache-e nagyban csökkenti a hálózati forgalmat, de a két párhuzamos író közötti koherencia nem garantált. A lockolás advisory, a lease lejárta után a szerver felszabadíthatja a zárat. A pNFS a metaadat- és adatszerverek szétválasztásával párhuzamosítja az adatelérést, de a layout érvényességét a kliensnek figyelnie kell.

SMB az oplock/lease révén erősebb koherenciát ad, mert a szerver aktívan visszavonja a cache-jogosultságot egy konkurens művelet előtt. A durable file handle és a transparent failover a hálózati hibák utáni helyreállítást támogatja. A Windows ACL-modell gazdagabb jogosultságkezelést nyújt, mint a POSIX mód bitek.

S3 nem ad általános lockolást, csak conditional write-ot és object lockot. A konzisztencia a szolgáltató garantálásától függ. A hibatűrés a replikációra és erasure codingra épül, a skálázhatóság

pedig a bucketnévtér horizontális elosztásából származik. A kliens oldali cache-nek itt kisebb szerepe van, mert az objektumok általában egészben cserélődnek.

A hálózati partíció esetén a klasszikus **CAP-tétel** érvényesül: egy elosztott rendszer egyszerre nem garantálhat teljes konzisztenciát, rendelkezésre állást és partíciótűrést. Egy NFS-szerver partíció esetén a `hard` mountolás letiltja a műveleteket, tehát a rendelkezésre állás sérül, de a konzisztencia megmarad. Egy S3-régióban a replikált objektumok átmenetileg eltérhetnek, tehát a konzisztencia gyengül, de a rendelkezésre állás magas marad.

16.4.7 Labor: NFS használati eset és konzisztencia

A következő labor egy emulált környezetben fut, nem a hosztgépen. Célja az NFS mountolás, a teljesítmény mérése és a close-to-open consistency megfigyelése. A laborcsomag a build- és futtatási parancsokkal együtt elérhető; az itt közölt parancsok a vendéggépekben futnak.

16.4.8 Emulált környezet

- **Hypervisor**: QEMU/KVM
- **Szerver**: 1 vCPU, 512 MiB RAM, 1 GiB eldobható háttérkép
- **Kliens**: 1 vCPU, 512 MiB RAM
- **Hálózat**: belső bridge, sebességkorlát 1 Gbit/s
- **IP-címek**: szerver `192.0.2.10`, kliens `192.0.2.20` (TEST-NET-1, nem nyilvános)

A vendéggépek a laborcsomagban előre telepített Linux-disztribúciót tartalmaznak; a pontos kernelverziót a reprodukciós csomag rögzíti.

16.4.9 Szerver oldal előkészítése

A szerveren létrehozunk egy eldobható háttérfájlt, formázzuk, és exportáljuk NFSv4-ként:

```
# Szerver, vendéggép root felhasználója
dd if=/dev/zero of=/srv/nfs.img bs=1M count=256 status=none
mkfs.ext4 -q /srv/nfs.img
mkdir -p /srv/nfs
mount -o loop /srv/nfs.img /srv/nfs

# Export bejegyzés - csak a 192.0.2.20 klienst engedélyezi
echo "/srv/nfs 192.0.2.20(rw,sync,no_subtree_check,fsid=0)" >> /etc/exports
exportfs -av
systemctl start nfs-server
```

16.4.10 Kliens oldal mountolása és mérés

A kliensen mountoljuk az exportált könyvtárat, majd egy 64 MiB-os fájlt írunk:

```
# Kliens
mkdir -p /mnt/nfs
mount -t nfs4 -o rw,hard,intr,timeo=50,retrans=2 192.0.2.10:/srv/nfs /mnt/nfs

# Átviteli mérés: 64 MiB írása
time dd if=/dev/zero of=/mnt/nfs/test.bin bs=1M count=64 oflag=direct
```

A mérés során a `dd` az `oflag=direct` opcióval próbálja megkerülni a kliens page cache-ét, hogy az NFS valós hálózati írását mérjük. Ha az NFS implementáció nem támogatja a direct I/O-t, a laborcsomag alternatív `fio` mérést is tartalmaz.

Várt eredmény: az emulált 1 Gbit/s hálózaton az átviteli sebesség legfeljebb 125 MB/s, de a gyakorlatban az RPC-fejlécek, a szerver `sync` opciója és a vendéggép CPU-ja miatt ennél alacsonyabb, jellemzően 80–110 MB/s lehet. A pontos érték a vendégkernel és a háttértár függvénye.

Mért eredmény: a labor futtatási naplójában rögzített érték, például:

művelet	mért idő	átvitel	megjegyzés
64 MiB írás NFS-re	0,78 s	82,1 MB/s	sync export, direct I/O
64 MiB olvasás NFS-ről	0,61 s	104,9 MB/s	warm page cache

A harmadik oszlopban a `dd` által kiírt átvitel szerepel; a mért és a várt értéket a napló külön mezőkben tárolja.

16.4.11 Konzisztencia megfigyelése

A close-to-open consistency viselkedését két Python programmal vizsgáljuk. A writer megnyitja a fájlt, ír bele, majd 10 másodpercig nyitva tartja, mielőtt bezárná:

```
# writer.py - NFS kliensen fut
import time, os

f = open('/mnt/nfs/data', 'wb')
f.write(b'hello from nfs')
f.flush()
os.fsync(f.fileno())
print('writer: open, flush, sleep')
time.sleep(10)
f.close()
print('writer: closed')
```

A reader eközben 1 másodperces időközönként próbálja olvasni a fájlt:

```
# reader.py - NFS kliensen fut
import time

for i in range(20):
    try:
        with open('/mnt/nfs/data', 'rb') as f:
            data = f.read()
        print(f'{i}: len={len(data)} data={data!r}')
    except FileNotFoundError:
        print(f'{i}: not found')
    time.sleep(1)
```

Várt eredmény: miközben a writer fájlja nyitva van, a reader által látott tartalom a kliens cache állapotától függ. Close-to-open consistency mellett a writer bezárása után a reader következő megnyitása már a teljes tartalmat látja. Azonban a nyitva tartás alatt a reader gyakran a régi (vagy hiányzó) tartalmat kapja, mert az NFS kliens nem revalidálja a cache-t minden open-nél, ha az attribútum-cache még érvényes.

Mért eredmény: a labor naplójában rögzített kimenet, például:

```
0: not found
1: not found
...
9: not found
10: len=14 data=b'hello from nfs'
11: len=14 data=b'hello from nfs'
...
```

A tényleges viselkedés az `actimeo` mountparamétertől és a szerver `sync` beállításától függ; a labor célja éppen ennek a változékonyságnak a dokumentálása.

16.4.12 A labor visszaállítása

A labor elvégzése után a vendéggépek leállíthatók, a háttérfájl törölhető, a hoszt rendszerét nem érinti semmi. A laborcsomag tartalmazza a visszaállítási parancsokat.

16.4.13 Tipikus hibák és helyes megoldások

1. **NFS export `async` használata**
   Hiba: a szerver a választ a kiírás előtt küldi el, ezért szerverösszeomlás esetén az ügyfél által már sikeresnek vélt írás elveszhet.
   Megoldás: éles környezetben használjunk `sync` export opciót, és a fájlrendszer naplózását a szerveren kapcsoljuk be.

2. **A close-to-open consistency félreértése**
   Hiba: az alkalmazás feltételezi, hogy egy másik kliens által éppen írt fájl tartalma azonnal látható.
   Megoldás: a párhuzamos írás-írás vagy írás-olvasás mintáknál használjunk explicit zárakat (`fcntl`), vagy tervezzük a hozzáférést úgy, hogy a fájl bezárása után olvassunk.

3. **NFSv3 lockolás bonyolultsága**
   Hiba: az NFSv3 nem integrálja a lockot, ezért a lockmanager és a status monitor szinkronizálása gyakran hibás.
   Megoldás: NFSv4 használata, ahol a lockolás a protokoll része.

4. **SMB share jogosultságok hiánya**
   Hiba: a Samba share létrehozásakor a POSIX mód biteket és a Windows ACL-t külön kell kezelni, és ha csak az egyiket állítjuk, a hozzáférés vagy túl szigorú, vagy túl megengedő lesz.
   Megoldás: a share szintű `readonly`, `valid users` és `admin users` paramétereket a fájlrendszer ACL-jeivel együtt, tesztelt kombinációban állítsuk be.

5. **S3 eventual consistency feltételezése minden műveletre**
   Hiba: az alkalmazás egy felülírás után azonnali olvasást feltételez.
   Megoldás: használjunk conditional write-ot (`If-Match`), vagy tervezzük az alkalmazást az adott szolgáltató konzisztencia-modelljének megfelelően, például új objektumkulccsal.

6. **Nagy objektumok egyetlen PUT-tal való feltöltése**
   Hiba: egy nagy fájl egyetlen HTTP-kérésben való küldése nem párhuzamosítható, és megszakadás esetén a teljes feltöltést újra kell kezdeni.
   Megoldás: multipart upload használata, amely részekre bontja az objektumot, és a részeket párhuzamosan tölti fel.

16.4.14 Önellenőrző kérdés

Kérdés: Miért nem elegendő az NFSv3 byte-range lockolása a párhuzamos írók közötti adatkonzisztenciához, és mit old meg az NFSv4 delegáció?

Válasz: Az NFSv3 lockolása advisory, tehát a kliens akkor is írhat, ha nem kért zárat. Ráadásul a zár és a write műveletek külön RPC-hívások, így a lock megszerzése és az írás között egy másik kliens is hozzáférhet. Az NFSv4 delegáció a szerver által aktívan felügyelt cache-jogosultság: ha egy másik kliens hozzá akar férni a fájlhoz, a szerver visszavonja a delegációt, így a koherencia a protokoll szintjén biztosított.

16.4.15 Összefoglalás

A hálózati fájlrendszerek a fájlabsztrakciót kiterjesztik a hálózatra. Az NFS a UNIX világban elterjedt, close-to-open konzisztenciát és delegációt használ, az NFSv4 pedig integrált lockolást és pNFS-t ad. Az SMB a Windows ökoszisztéma megosztási protokollja, oplock/lease mechanizmussal és kiforrott hibatűréssel. Az S3 objektumtár a fájlok helyett objektumokat kezel, conditional write-tal és replikációval. A cache, lock és hibatűrés egyensúlya mindhárom rendszerben más, és a választás az alkalmazás igényeitől függ.

A következő fejezet a fájlrendszerek crash-konzisztenciáját és naplózását tárgyalja, ahol a hálózati fájlrendszereknél megismert állapot- és cache-kezelési elvek lokális környezetben is megjelennek.

17 Megbízhatóság és crash consistency

17.1 Journaling és tranzakciós metaadat

A fájlrendszer-műveletek gyakran több, egymással logikai kapcsolatban álló metaadatblokk módosítását igénylik. Egy új fájl létrehozása például lefoglal egy inode-ot, frissíti a könyvtárbejegyzést, és módosítja a foglaltsági bitképet. Ha a rendszer a három blokk közül csak kettőt ír ki lemezre, majd áramszünet következik be, a fájlrendszer inkonzisztens, akár feloldhatatlan állapotba kerülhet. A naplózás (journaling) ezt a problémát úgy kezeli, hogy a módosításokat először egy elkülönített, jól meghatározott naplóterületre írja, és csak ezután alkalmazza a végleges helyükre. A naplóbejegyzések lehetővé teszik, hogy a visszajátszás során a metaadatok mindig konzisztens, a tranzakcióhatárok által meghatározott állapotba kerüljenek.

17.1.1 A naplózás alapelve: write-ahead log, tranzakció, commit

A naplózás központi eleme a write-ahead log (WAL), magyarul előre író napló. A WAL alapvető szabálya, hogy a végleges adatterületre történő írás előtt a módosítást leíró naplórekordot tartósan ki kell írni. A „write-ahead” kifejezés pontosan erre a sorrendre utal: először a napló, utána a tényleges blokk. Ha a rendszer a naplóírás után, de a végleges írás előtt omlik össze, a naplóból a művelet visszajátszható. Ha a naplóírás maga szakad félbe, akkor a tranzakció befejezetlen marad, és a helyreállítás során eldobjuk.

Egy naplózott műveletet tranzakciónak nevezünk. A tranzakció több blokkmódosítást fog össze egyetlen logikai egységként. A tranzakció határai jellemzően egy rendszerhívás vagy egy magasabb szintű művelet köré rajzolódnak, például egy `mkdir`, egy `unlink` vagy egy metaadat-frissítés. A tranzakció állapotát a naplóban külön rekordok jelzik: a tranzakció kezdetét egy `TX_BEGIN`, a végét egy `TX_COMMIT` rekord. A kettő közötti rekordok tartalmazzák a tényleges blokkmódosításokat. A commit rekord tartós kiírása jelenti azt a pontot, ahonnan a tranzakció befejezettnek tekinthető, és visszajátszható.

A naplórekordoknak legalább a következő mezőket kell tartalmazniuk: egy monoton növekvő sorszámot, a tranzakcióazonosítót, a rekord típusát, valamint a módosított blokk adatait. A sorszám és a tranzakcióazonosító lehetővé teszi a napló szekvenciális feldolgozását, a típus pedig a tranzakcióhatárok felismerését. A gyakorlatban a naplórekordokat gyakran ellenőrző összeggel (checksum) védik, így a félbeírt rekordok felismerhetők.

17.1.2 Tranzakciós metaadatok: példa egy fájlműveletre

Tekintsünk egy fájl létrehozását egy inode-alapú fájlrendszerben. A művelet a következő blokkokat érinti:

1. **Inode blokk**: új inode foglalása, típus és jogosultságok beállítása.
2. **Könyvtár blokk**: a szülőkönyvtár bejegyzésének hozzáadása a fájlnévvel és az inode-számmal.
3. **Foglaltsági bitkép blokk**: az inode-bitképben az új inode megjelölése foglaltként.

Ha ezeket a blokkokat közvetlenül, naplózás nélkül írnánk ki, a kiírási sorrendtől függően számos inkonzisztens állapot keletkezhet. A naplózás ezt úgy oldja meg, hogy a három blokk tartalmát egyetlen tranzakcióban a naplóba írja, majd a commit rekordot tartósan flush-eli, és csak ezután alkalmazza a valós blokkokat.

A következő pszeudokód egy ilyen tranzakció felépítését mutatja:

```
procedure create_file(dir_inode, dir_block, name, new_inode, bitmap_block):
    tx = begin_tx()

    journal_append(tx, TX_UPDATE, INODE_BLOCK, new_inode)
    journal_append(tx, TX_UPDATE, DIR_BLOCK, dir_block)
    journal_append(tx, TX_UPDATE, BITMAP_BLOCK, bitmap_block)

    commit_tx(tx)          # commit rekord írása és tartós flush
    apply_to_filesystem(tx) # valós blokkok frissítése
```

A `begin_tx` egy új tranzakciót nyit, és `TX_BEGIN` rekordot ír a naplóba. A `journal_append` hozzáfűzi a módosított blokkokat a naplóhoz. A `commit_tx` kiírja a `TX_COMMIT` rekordot, és biztosítja, hogy a tranzakció minden rekordja tartósan tárolva legyen. Ezután az `apply_to_filesystem` a naplóban lévő blokkokat a végleges lemezterületre másolja.

Fontos, hogy a `commit_tx` nem csupán a commit rekord kiírását jelenti, hanem az összes naplórekord tartós flush-ét is. A WAL szabály szerint a commit rekord csak akkor tekinthető érvényesnek, ha az őt megelőző `TX_UPDATE` rekordok is tartósan a lemezen vannak. Ellenkező esetben a commit rekord alapján visszajátszott tranzakció hiányos adatokkal dolgozna.

17.1.3 Commit, checkpoint és helyreállítás

A naplózás három fázisa a tranzakció életciklusában: a naplóírás, a commit és a checkpoint. A naplóírás és a commit a tranzakció létrehozásakor történik, a checkpoint pedig később, amikor a végleges blokkok már frissültek, és a naplóbejegyzések felszabadíthatók.

A commit pillanatától kezdve a tranzakció visszajátszható. A naplóban a commit rekord egyértelműen jelzi, hogy a tranzakció minden rekordja érvényes, és a valós blokkok frissítése biztonságosan elvégezhető. Ha a rendszer a commit után, de a valós blokkok frissítése előtt omlik össze, a helyreállítás során a tranzakciót újra alkalmazzuk.

A checkpoint célja a naplóterület felszabadítása. A napló véges, ezért idővel a bejegyzéseket ki kell vezetni a végleges lemezterületre, és a naplófejet előrébb kell mozgatni. A checkpoint során a rendszer azokat a tranzakciókat, amelyek már befejezettek és a valós blokkokra alkalmazottak, tisztaként megjelöli, és a naplóterületet újra felhasználhatóvá teszi. A checkpoint nem törölheti egy tranzakció naplóbejegyzéseit addig, amíg a valós blokkok frissítése nem fejeződött be, különben crash esetén a tranzakció elveszne.

A következő pszeudokód a helyreállítási folyamatot mutatja:

```
procedure recover_journal():
    current_tx = NULL
    committed = []

    for record in read_journal_forward():
        if record.type == TX_BEGIN:
            current_tx = new_tx(record.tx_id)
        elif record.type == TX_UPDATE:
            current_tx.records.append(record)
        elif record.type == TX_COMMIT:
            committed.append(current_tx)
```

```
            current_tx = NULL

    for tx in committed:
        apply_to_filesystem(tx)

    discard all non-committed transactions
```

A helyreállítás előrefelé olvassa a naplót, tranzakciókat épít fel, és csak azokat a tranzakciókat alkalmazza, amelyekhez `TX_COMMIT` rekord tartozik. A befejezetlen tranzakciókat eldobjuk, mert azok rekordjai nem feltétlenül teljesek. Ez a mechanizmus biztosítja, hogy a fájlrendszer metaadatai soha ne kerüljenek félig frissített állapotba.

A checkpoint algoritmusa:

```
procedure checkpoint():
    for tx in clean_transactions_ordered_by_id():
        for rec in tx.records:
            write_original_block(rec.block)
        mark_tx_clean(tx)
    advance_journal_head()
```

A `clean_transactions_ordered_by_id` azokat a tranzakciókat adja vissza, amelyek már commit-oltak, és a valós blokkjaik módosítása megtörtént. A checkpoint ezeket a blokkokat még egyszer, szükség esetén ellenőrzésképpen, a végleges helyükre írja, majd a tranzakciót tisztaként jelöli. A naplófej előremozgatása után a felszabadult naplóterület újra felhasználható.

A komplexitás szempontjából a naplóírás tranzakciónként $O(k)$ I/O-műveletet igényel, ahol k a módosított blokkok száma. A helyreállítás a napló hosszában lineáris, $O(N)$, ahol N a naplórekordok száma. A checkpoint költsége a kiürítendő blokkok számával arányos, $O(M)$, de ez a költség a normál működés során eloszlik, és a napló méretétől függően ritkán jelentkezik. A naplózás írásamplifikációja a naplózási módtól függ, amint azt a következő szakasz mutatja.

17.1.4 Naplózási módok: writeback, ordered, data journaling

A naplózás leggyakrabban a metaadatokra terjed ki, de a felhasználói adatok kezelése eltérő lehet. A három klasszikus mód a writeback, az ordered és a data journaling. Mindhárom mód a metaadatok konzisztenciáját garantálja, de az adatok tartósságára és sorrendjére vonatkozó garanciáik különböznek.

17.1.5 Writeback mód

A writeback módban a napló csak a metaadatok módosításait tartalmazza. A felhasználói adatblokkokat a rendszer közvetlenül a végleges helyükre írja, de nincs szigorú sorrend a metaadatok és az adatok között. A rendszer a metaadatokat a naplóba írja, commit-olja, majd a valós metaadatblokkokat frissíti; az adatblokkok kiírása eközben késhet.

Előnye, hogy kevesebb I/O-t igényel, mert az adatok nem duplikálódnak a naplóban. Hátránya, hogy crash után a metaadatok konzisztensek lehetnek, de a hozzájuk tartozó adatblokkok tartalma régebbi vagy akár érvénytelen is lehet. Például ha a felhasználó egy fájl végét meghosszabbítja, a metaadat rögzíti az új méretet, de az adatblokk még nem íródott ki; crash után a fájl mérete nagyobb, de az új terület tartalma szemét.

17.1.6 Ordered mód

Az ordered mód a writeback és a data journaling közötti kompromisszum. A napló továbbra is csak a metaadatokat tartalmazza, de a rendszer szigorú sorrendet tart: mielőtt egy metaadat-tranzakciót commit-olna, az összes hozzá tartozó adatblokkot tartósan kiírja a végleges helyére. Így a metaadatok soha nem mutathatnak olyan adatblokkra, amely nincs tartósan tárolva.

Crash után a fájlrendszer metaadatai konzisztensek, és a fájlok tartalma vagy a régi, vagy az új, de soha nem kevert vagy szemét. Ha egy fájl létrehozása félbeszakad, a fájl nem jelenik meg, mert a könyvtárbejegyzés frissítése nem commit-olt; ha viszont megjelenik, akkor az adatai is tartósak. Ez a mód a legtöbb általános célú rendszerben jó egyensúlyt ad.

17.1.7 Data journaling mód

A data journaling módban a napló a metaadatok mellett a felhasználói adatblokkokat is tartalmazza. A rendszer először az adatblokkokat és a metaadatokat is a naplóba írja, commit-olja a tranzakciót, majd mindkettőt a végleges helyére másolja.

Ez a mód a legnagyobb biztonságot nyújtja: crash után az adatok és a metaadatok is visszajátszhatók a naplóból, így a fájltartalom és a metaadatok egyaránt konzisztensek. Hátránya a kétszeres írás: minden adatblokk először a naplóba, majd a végleges helyére kerül, ami jelentős írásamplifikációt okoz. Az írásamplifikáció tényezője elérheti a 2,0-t, mert egy adatblokk kétszer íródik lemezre.

A következő táblázat összefoglalja a három mód jellemzőit:

Mód	Naplózott tartalom	Adat-metaadat sorrend	Crash utáni garancia	Írásamplifikáció
write-back	csak metaadat	nincs szigorú sorrend	metaadatok konzisztensek, adatok lehetnek érvénytelenek	alacsony
ordered	csak metaadat	adatok előbb, mint metaadat	metaadatok és a hivatkozott adatok konzisztensek	közepes
data journaling	metaadat és adat	adat és metaadat is naplózva	metaadatok és adatok is konzisztensek	magas

A mód kiválasztása mindig a rendszer céljaitól függ: adatbázis-szervereknél gyakran az ordered vagy a data journaling a választás, míg nagy áteresztőképességű, kevésbé kritikus környezetben a writeback is elfogadható.

17.1.8 Gyakori hibák és helyes megoldások

A naplózás helyes implementálása több buktatót rejt. Az alábbiak a leggyakoribb hibák, valamint azok elkerülésének módjai.

1. A commit rekord flush-e elmarad. Ha a commit rekordot nem írják tartósan a lemezre, a tranzakció a befejezetlen állapotban marad a naplóban. A helyreállítás eldobja, pedig a valós

blokkok egy részét már módosíthatta. A megoldás: a commit rekordot kifejezetten flush-elni kell, és csak ezután szabad a valós blokkokat frissíteni.

2. A naplórekordok nem tartósak a commit előtt. A WAL szabály megsértése, ha a `TX_UPDATE` rekordok nincsenek tartósan a lemezen a commit rekord flush-e előtt. A commit rekord ugyan érvényesnek tűnhet, de a visszajátszás hiányos adatokat találhat. A megoldás: a commit előtt minden rekordot flush-elni kell, vagy a commit rekordnak tartalmaznia kell a teljes tranzakció ellenőrző összegét, és a visszajátszásnak ellenőriznie kell.

3. A checkpoint túl korán törli a naplóbejegyzéseket. Ha a checkpoint akkor is felszabadítja a tranzakció naplórekordjait, amikor a valós blokkok frissítése még nem fejeződött be, crash esetén a tranzakció elveszik. A megoldás: a checkpoint csak azután jelölheti tisztának a tranzakciót, miután a valós blokkok mindegyikét sikeresen kiírta.

4. A tranzakcióhatárok rossz megválasztása. Túl nagy tranzakciók esetén a napló gyorsan megtelik, és a checkpoint sokat késik; túl kicsi tranzakciók esetén a commit rekordok száma nő, ami többletköltséget okoz. A megoldás: a tranzakciókat a logikai műveletekhez igazítani, és a napló méretét a várható munkaterheléshez méretezni.

5. Az idempotencia hiánya a visszajátszásban. Ha a helyreállítás során egy tranzakciót többször is alkalmaznak, például mert a napló nem lett érvénytelenítve, a blokkok tartalma megsérülhet. A megoldás: a naplórekordoknak idempotens műveleteket kell leírniuk, vagy a tranzakció visszajátszása után a naplót meg kell jelölni, hogy ne legyen újra alkalmazva.

6. Az ordered módban az adatblokkok kiírása elmarad. Ha a metaadat-tranzakció commit-ja előtt nem flush-elik az adatblokkokat, a metaadatok hivatkozhatnak nem tartós adatokra. A megoldás: az adatblokkokat explicit módon, a metaadat commit előtt flush-elni.

Ezek a hibák mind a WAL sorrend, a commit és a checkpoint közötti kapcsolat félreértéséből fakadnak. A helyes megoldás mindig a sorrend szigorú betartása: először a naplórekordok tartós kiírása, majd a commit, végül a valós blokkok frissítése és a checkpoint.

17.1.9 Labor: journaling- és write-ahead log terv készítése

Ez a labor nem igényel valós diszket vagy éles rendszert. Célja, hogy a hallgató megtervezze és szimulálja egy egyszerű naplózó fájlrendszer WAL mechanizmusát, összehasonlítsa a három naplózási módot, és reprodukálható mérési naplót készítsen.

17.1.10 Kiinduló állapot és konfiguráció

- Emulált blokkeszköz: egy eldobható lemezkép, mérete 64 MiB, QEMU-ban `virtio-blk` eszközként csatlakoztatva, vagy egy fájl a hoszton hasonló céllal.
- Blokkméret: 4 KiB.
- Naplóterület: 4 MiB, azaz 1024 naplóblokk.
- Tranzakciónként maximum 16 blokk.
- Műveletsor: egy előre definiált lista, például 50 fájl létrehozása, 20 fájl törlése, 30 metaadat-frissítés, sorrendben.
- Áramszünet-szimuláció: a műveletek közötti véletlenszerű pontokon a futás leállítása, majd helyreállítás.

A labor során a hallgató a saját pszeudokódját használja, amely a fenti naplózási algoritmusokat valósítja meg. A pszeudokód kimenete a napló tartalma, a végleges blokkok állapota, valamint a mérési adatok.

17.1.11 Lépések

1. **Tervezze meg a naplórekord szerkezetét.** Határozza meg a rekordtípusokat (`TX_BEGIN`, `TX_UPDATE`, `TX_COMMIT`), a sorszám, a tranzakcióazonosító, a blokkazonosító és az ellenőrző összeg mezőit.
2. **Implementálja a WAL-t pszeudokódban.** A tranzakció létrehozása, a rekordok hozzáfűzése, a commit, a valós blokkok frissítése és a checkpoint.
3. **Válasszon naplózási módot.** Futtassa ugyanazt a műveletsort writeback, ordered és data journaling módban.
4. **Szimuláljon áramszünetet.** Válasszon ki 50 véletlenszerű pontot a műveletsoron belül, és mindegyiknél állítsa le a futást a commit előtt vagy után.
5. **Futtassa a helyreállítást.** A napló visszajátszásával állítsa helyre a fájlrendszert, és rögzítse az eredményt.
6. **Mérje a teljesítményt.** Számolja a naplóblokk-írások, az fsync-ek és a helyreállítási lépések számát.

17.1.12 Mérési napló

A mérési naplóban a várt és a mért eredményt külön kell feltüntetni. Az alábbi táblázat a szükséges oszlopokat mutatja:

Mód	Tranzakció ID	Áramszünet pont	Várt konzisztencia	Mért konzisztencia	Naplórekordok	Fsync-ek	Helyreállítási lépések
writeback	12	commit után, valós írás előtt	metaadat konzisztens, adat érvénytelen lehet	…	…	…	…
ordered	12	commit előtt	tranzakció eldobva, régi állapot	…	…	…	…
data	12	commit után, valós írás után	minden konzisztens	…	…	…	…

A **várt konzisztencia** oszlopba a tervezett viselkedést kell írni, a **mért konzisztencia** oszlopba pedig a szimuláció tényleges eredményét. A kettő eltérése hibát jelez.

17.1.13 Elvárt eredmény

- Writeback módban a metaadatok mindig konzisztensek, de adatblokkok lehetnek érvénytelenek. A helyreállítás során a metaadatok visszajátszása sikeres.
- Ordered módban a metaadatok konzisztensek, és a hivatkozott adatok tartósak; a tranzakció csak akkor játszódik vissza, ha a commit rekord tartós.
- Data journaling módban a metaadatok és az adatok is konzisztensek; a visszajátszás után a fájl tartalma és a metaadatok egyaránt helyesek.

A mérési naplónak tartalmaznia kell a kiinduló állapot lemez-hash-ét, a műveletsort, a szimulációs paramétereket, valamint a használt pszeudokód verzióját. A visszaállítás a lemezkép újraformázásával vagy a tiszta másolat visszamásolásával történik.

17.1.14 Önellenőrző kérdés

Rendelje hozzá a három naplózási módhoz a crash utáni állapotot: writeback, ordered, data journaling. Magyarázza el, miért garantálja a WAL azt, hogy a metaadatok soha nem kerülnek félig frissített állapotba, és mi a commit rekord szerepe a helyreállításban.

17.2 Copy-on-write, checksum és scrub

A CoW (copy-on-write) és az integritásellenőrzés együttműködése az egyik legerősebb védekezési minta a tárolórendszerekben: ahelyett, hogy egy blokkot felülírnánk, minden módosítás új blokkot hoz létre, és az új blokkhoz ellenőrzőösszeget (checksum) számolunk. Így a lemezen soha nem létezik olyan állapot, amelyben egy félig kiírt, sérült vagy szakadt írás ellenőrzőösszeg nélkül kerülne be az élő folyamba. A fejezet bemutatja, hogyan kapcsolódik össze a CoW és a checksum, miért fontos az adat- és metaadat-integritás elkülönítése, és hogyan épül fel a scrub, vagyis az átvizsgáló, öngyógyító folyamat.

17.2.1 1. A CoW és az integritás kapcsolata

A copy-on-write elv szerint egy blokk módosításakor nem a régi blokkot írjuk felül, hanem:

1. beolvassuk a régi blokkot,
2. módosított tartalommal új blokkot készítünk,
3. az új blokkot egy szabad helyre írjuk,
4. kiszámítjuk és eltároljuk az új blokk ellenőrzőösszegét,
5. a fájlrendszer fa struktúrájában frissítjük a mutatókat, hogy az új blokkra hivatkozzanak.

A régi blokk megmarad mindaddig, amíg az új állapot teljesen, atomikusan el nem készül. Csak a legfelső mutató (például a gyökércsomópont) egyetlen írással történő cseréje teszi láthatóvá az új állapotot. Ha a rendszer a lépések bármelyikénél leáll, a régi, konzisztens állapot továbbra is érvényben marad. Ez a tulajdonság a crash consistency alapja.

A checksum ezt a folyamatot egészíti ki azzal, hogy minden blokkhoz tartozik egy ellenőrző összeg. A CoW miatt a checksum számítása mindig az írás előtt, az új blokk tartalmára vonatkozik; így a fájlrendszer soha nem tárol élő blokkot érvénytelen ellenőrzőösszeggel. Az összeg kiszámítása a memóriában történik, mielőtt a blokk a lemezre kerülne, ezért a szakadt írásból származó sérülés detektálható marad.

A két mechanizmus együtt a következő írási invariánst adja:

- élő blokk = olyan blokk, amelyre a fa valamely gyökérből elérhető mutató mutat;
- minden élő blokk rendelkezik érvényes checksummal;
- ha egy blokk tartalma megváltozik anélkül, hogy a checksum is frissülne, az olvasáskor vagy scrub során hibát jelez;
- a hiba a CoW miatt nem ronthatja a korábban kiírt, ellenőrzött állapotot.

Ez a kétpillérű védelem a modern, integritásorientált fájlrendszerek – például a ZFS és a Btrfs – alapvető tervezési mintája.

17.2.2 2. Checksumok az adatintegritásban

A checksum egy rövid, determinisztikus lenyomat, amelyet egy blokk tartalmából számítunk. Célja, hogy a blokk tartalmában bekövetkező véletlen hibát (bitfordulás, részleges írás, lemezhiba) nagy valószínűséggel észlelje. A checksum nem azonos a kriptográfiai hash-sel; a cél itt elsősorban a hiba detektálása, nem a szándékos manipuláció elleni védelem. Bár a kettő átfedhet, a tárolórendszerekben gyakran gyors, hardveresen támogatott algoritmusokat használnak.

17.2.3 2.1. Hibamodell és detektálási képesség

A tárolóblokkok integritásellenőrzésénél a leggyakoribb hibatípusok:

- **Olvasási hiba**: a lemez vagy a vezérlő rossz adatot ad vissza.
- **Csendes adatromlás (silent data corruption)**: a lemez vagy a vezérlő nem jelez hibát, de a visszaadott adat eltér az eredetitől.
- **Szakadt írás (torn write)**: egy blokk írása a tápfeszültség kiesése miatt félig történik meg.
- **Elírás (misplaced write)**: egy blokk tartalma rossz helyre íródik, például egy hibás firmware miatt.
- **Bitforgás**: egy vagy több bit véletlen átfordulása a tárolóban.

Egy b bites checksum segítségével a véletlen hibák detektálásának felső korlátja közelítőleg

$$P_{\text{detektálás}} \approx 1 - 2^{-b}.$$

Ez azt jelenti, hogy egy 32 bites checksum (például CRC32C) esetén a véletlen hiba elkerüli a detektálást körülbelül $2{,}3 \cdot 10^{-10}$ valószínűséggel. A gyakorlatban a hibatípusok nem teljesen véletlenszerűek, ezért a CRC-k és a hasító algoritmusok tervezése a hibamintázatokra is optimalizált. A modern fájlrendszerek gyakran 256 bites kriptográfiai hash-t (például SHA-256-ot) is kínálnak, ami a szándékos manipuláció ellen is véd, de nagyobb CPU-költséggel jár.

A checksum tárolása a blokk mellett vagy egy külön, dedikált területen történik. A leggyakoribb megoldás, hogy a blokk fejlécében vagy a blokkra mutató metaadatban tárolják. A CoW fájlrendszerekben a checksum általában a blokkra mutató bejegyzésben vagy a blokk elején lévő fejlécben található, így a fa bejárása során minden olvasásnál ellenőrizhető.

17.2.4 2.2. Adat- és metaadat-integritás

A fájlrendszer kétféle tartalmat tárol: **adatot** (a fájlok tényleges tartalma) és **metaadatot** (a fájlok nevei, méretei, jogosultságai, blokkmutatók, szabad blokk listák). A metaadat sérülése általában súlyosabb következményekkel jár, mert az egész fájlrendszer szerkezetét érintheti. Ezért a metaadatokat különösen erős védelemmel látják el:

- minden metaadatblokkhoz külön checksum tartozik;
- a metaadat frissítése CoW-val történik, így a régi, érvényes állapot mindig megmarad;
- a gyökércsomópontra mutató legfelső blokk írása atomi, és általában két példányban, eltérő területen tárolják.

Az adatblokkok checksumja is fontos, de egy adatblokk sérülése jellemzően csak egyetlen fájl tartalmát érinti, ezért a javítási stratégia lehet egyszerűbb: ha van redundáns másolat vagy replika, a hibás adat helyreállítható; ha nincs, a hiba legalább pontosan behatárolható.

17.2.5 3. CoW és checksum együttműködése

A CoW és a checksum együttműködése négy fontos lépésben valósul meg: az írási út, az olvasási út, a fa tranzakciós frissítése és a scrub.

17.2.6 3.1. Írási út

Amikor egy alkalmazás ír egy fájlba, a fájlrendszer a következőképpen jár el:

1. A fájl módosítandó adatblokkját beolvassa a memóriába.
2. A módosított tartalomból új blokkot hoz létre.
3. Kiszámítja az új blokk checksumját, és hozzárendeli a blokkhoz.
4. Az új blokkot egy szabad területre írja.
5. A fájl blokkmutatóit frissíti, hogy az új blokkra mutassanak; ehhez szintén CoW-t használ a metaadatblokkon.
6. A mutatók frissítése felfelé terjed a fában, míg el nem éri a gyökércsomópontot.
7. A gyökércsomópontot egyetlen atomi írással cseréli, amely az egész új állapotot láthatóvá teszi.

Minden egyes lépés során a blokkhoz tartozó checksum már a kiírás előtt rendelkezésre áll. Ha az írás megszakad, a régi blokk és a régi fa érvényes marad, mert a gyökércsomópont még a régi állapotra mutat.

A következő pszeudokód az írási folyamatot szemlélteti:

```
function write_file_block(file, block_index, new_data):
    old_block = read_block(file.block_map[block_index])
    new_block = copy(old_block)
    new_block.data = new_data
    new_block.checksum = checksum(new_block.data)

    new_address = allocate_free_block()
    write_block(new_address, new_block)

    update_block_map(file, block_index, new_address)   # CoW a metaadaton
    update_parents(file, new_address)                  # fa frissítése felfelé

    atomic_swap_root(file.root_pointer, new_root)
```

17.2.7 3.2. Olvasási út

Olvasáskor a fájlrendszer beolvassa a blokkot, majd azonnal ellenőrzi a checksumját:

```
function read_file_block(file, block_index):
    address = file.block_map[block_index]
    block = read_block(address)
    expected = block.checksum
    actual = checksum(block.data)

    if actual != expected:
        raise IntegrityError("checksum mismatch at block " + address)

    return block.data
```

Ha az ellenőrzés eltérést jelez, a fájlrendszer hibát ad vissza, nem szolgáltatja a sérült adatot. Ez az azonnali detektálás a CoW és a checksum egyik legfontosabb előnye: a sérült adat nem terjed tovább az alkalmazáshoz, a gyorsítótárhoz vagy a hálózathoz.

17.2.8 3.3. Fa struktúra és tranzakciós lánc

A CoW fájlrendszerekben minden blokk – adat és metaadat – egy fa része. A fa csúcsán a gyökércsomópont áll, amely a legmagasabb szintű metaadatokat tartalmazza. A gyökércsomópont checksumja is ellenőrzött, és maga a gyökércsomópont is CoW-val frissül. Így a teljes fa minden szintjén érvényes checksumok láncolata jön létre.

Például egy egyszerű, kétszintű fa:

```
Gyökér (checksum: C_root)
  Metaadat blokk A (checksum: C_A)
      Adatblokk 0 (checksum: C_0)
      Adatblokk 1 (checksum: C_1)
  Metaadat blokk B (checksum: C_B)
       Adatblokk 2 (checksum: C_2)
       Adatblokk 3 (checksum: C_3)
```

Ha az Adatblokk 2 tartalma megváltozik, akkor új Adatblokk 2 jön létre, új checksummal. Ezután új Metaadat blokk B jön létre, amelynek checksumja az új szerkezetet tükrözi. Végül új Gyökér jön létre, és a gyökércsomópontot cseréljük. A régi Gyökér továbbra is a régi, ellenőrzött állapotra mutat.

Ez a láncolat biztosítja, hogy egyetlen sérült blokk sem maradhat észrevétlen, és a hiba a fa bejárásakor pontosan lokalizálható.

17.2.9 4. Self-healing és scrub

A self-healing (öngyógyítás) azt jelenti, hogy a fájlrendszer a hibás blokkot képes automatikusan helyreállítani egy redundáns forrásból. Ez a forrás lehet:

- tükrözött blokk (RAID 1 vagy hasonló),
- paritás alapú redundancia (RAID 5/6),
- replikált metaadat,
- egy másik, ellenőrzött példány a gyorsítótárban.

A **scrub** egy periodikus vagy igény szerinti átvizsgáló folyamat, amely végigmegy a fájlrendszer összes blokkján, ellenőrzi a checksumokat, és hibát jelez vagy javít. A scrub célja, hogy a csendes adatromlást akkor is észlelje, amikor az adott blokkot az alkalmazás éppen nem olvassa. Így a hiba nem vár addig, amíg valaki hozzáfér a blokkhoz; a scrub proaktívan felderíti és lehetőség szerint javítja.

17.2.10 4.1. Scrub folyamat

A scrub algoritmusa tipikusan a következő:

1. A fájlrendszer lefagyasztja a legfelső gyökércsomópontot, vagy egy konzisztens pillanatképet (snapshot) készít.
2. Bejárja a fát szélességi vagy mélységi sorrendben.
3. Minden blokknál beolvassa a tartalmat, és kiszámítja a checksumot.
4. Összehasonlítja a tárolt ellenőrzőösszeggel.
5. Ha nincs eltérés, továbblép.
6. Ha eltérés van:
    - ha létezik redundáns másolat, a rendszer megkísérli a javítást;

- ha a másolat is sérült, a hibát naplózza, és a blokk elvesztését jelzi.

7. A folyamat a fa minden élő blokkját ellenőrzi.

A scrub során a fájlrendszer nem módosítja a fa szerkezetét, csak a hibás blokkokat javítja, ezért a CoW logika részben felfüggeszthető. A javítás azonban gyakran CoW-val történik: a hibás blokk adatait a redundáns példányból egy új blokkba írják, és a mutatókat frissítik, így a hibás blokk felszabadul.

A scrub időigénye arányos a tárolt adat mennyiségével. A futás ideje alatt a lemez I/O sávszélességét részben a scrub foglalja le, ezért a tervezésnél figyelembe kell venni a terhelést. A modern rendszerekben a scrub alacsony prioritású háttérfolyamatként fut.

17.2.11 4.2. Javítás redundanciából

A javítás egyik gyakori esete, amikor egy blokk tükrözött (mirror) konfigurációban van. A scrub ellenőrzi az első példány checksumját; ha eltérés van, ellenőrzi a második példányt. Ha a második példány érvényes, akkor a hibás példányt felülírja a jó tartalmával, és új checksumot számít. Ha mindkettő sérült, akkor a hiba nem javítható, és a rendszer hibát jelez.

A következő pszeudokód a tükrözött javítást mutatja:

```
function scrub_mirror_block(primary, mirror):
    primary_ok = verify_checksum(primary)
    mirror_ok = verify_checksum(mirror)

    if primary_ok and mirror_ok:
        return OK
    else if not primary_ok and mirror_ok:
        new_block = read_block(mirror)
        write_block(primary, new_block)     # CoW nélkül, mert csak javítás
        return REPAIRED_PRIMARY
    else if primary_ok and not mirror_ok:
        new_block = read_block(primary)
        write_block(mirror, new_block)
        return REPAIRED_MIRROR
    else:
        return UNRECOVERABLE
```

A gyakorlatban a javítás nem közvetlen felülírással történik, hanem a blokk újraírásával, miközben a fa mutatói változatlanok maradnak, vagy a hibás blokk helyett egy új blokkra mutatnak. A lényeg, hogy a javítás után a checksum újra érvényes legyen.

17.2.12 5. Tipikus hibák és helyes megoldások

A CoW és checksum alapú rendszerek tervezése során számos hiba fordulhat elő. Az alábbiakban a leggyakoribbakat és a helyes megoldásokat ismertetjük.

1. **A checksum nem fedi a teljes blokkot.** Ha a checksum csak a blokk egy részére vonatkozik, akkor a nem fedett rész sérülése rejtve marad. Helyes megoldás: a checksum mindig a blokk teljes tartalmára, vagy a blokk fejlécére és adatára együtt számítson.

2. **A metaadat és az adat ugyanazt a checksumot használja.** Ez önmagában nem hiba, de a metaadatoknál erősebb védelmet kell alkalmazni, mert a metaadat sérülése súlyosabb. A gyakorlatban a metaadatokhoz gyakran legalább 256 bites hash-t használnak, míg az adatokhoz elegendő lehet CRC32C. Helyes megoldás: rétegek szerint differenciált integritáspolitika.

3. **A scrub során a rendszer nem ellenőrzi a mutatókat.** Egy sérült mutató rossz blokkra vezethet, és a checksum ellenőrzés hibát jelezhet, de a fa szerkezete is sérülhet. Helyes megoldás: a scrub a fa bejárásakor a mutatók konzisztenciáját is ellenőrizze, például a szülő és a gyermek blokkok összhangját.

4. **A checksum ütközés figyelmen kívül hagyása.** Bár a 32 bites checksum ritkán ütközik, a véletlen hibák detektálási valószínűsége nem abszolút. Helyes megoldás: kritikus adatoknál erősebb hash-t használni, és a hibadetektálást időnként kriptográfiai hash-sel is megerősíteni.

5. **A javítás során a rendszer nem ellenőrzi a redundáns másolat checksumját.** Ha a másolat is sérült, a javítás hibás adatot terjeszthet. Helyes megoldás: a javítás előtt minden forrást ellenőrizni.

6. **A scrub túl ritkán fut, vagy túl nagy terhelést okoz.** A ritka scrub lehetővé teszi a hiba terjedését; a túl gyakori scrub felesleges I/O-t generál. Helyes megoldás: a scrub ütemezése a tároló mérete és a hibatűrési követelmények alapján, alacsony prioritású háttérfeladatként.

7. **A CoW frissítésnél a gyökércsomópont cseréje nem atomi.** Ha a gyökércsomópont írása megszakad, a fájlrendszer inkonzisztenssé válhat. Helyes megoldás: a gyökércsomópontot két példányban, felváltva írjuk, és a cserét egy tranzakciós számlálóval vagy hasonló atomi művelettel védjük.

17.2.13 6. Labor: checksum- és scrub-folyamat tervezése

A labor célja, hogy egy emulált blokkboltban megvalósítsunk egy egyszerű CoW alapú, checksummal védett fájlrendszer-szerű struktúrát, és a scrub folyamatot kipróbáljuk. A labor egy eldobható QEMU virtuális gépben fut, a hoszt valódi lemezei nem érintettek.

17.2.14 6.1. Cél és architektúra

A laborban egyetlen, fix méretű képfájlt használunk blokkboltként. Minden blokk 4096 bájt. A blokk formátuma:

```
+----------------------+
| Header (16 byte)     |
|  - magic (4 byte)    |
|  - checksum (4 byte) |
|  - sequence (8 byte) |
+----------------------+
| Data (4080 byte)     |
+----------------------+
```

A `checksum` a Data mezőre számított CRC32C érték. A `sequence` a blokk monoton növekvő tranzakciós azonosítója, amely a CoW frissítést segíti.

A rendszer kétféle blokkot kezel:

- **Adatblokk**: a felhasználói adatot tartalmazza.
- **Metaadatblokk**: a blokkmutatókat és a gyökércsomópontot tartalmazza; a laborban egyetlen metaadatblokkot használunk, amely a fájlblokkok listáját tárolja.

A CoW írás során az adatblokk módosítása új blokkot eredményez, majd a metaadatblokkot is frissítjük. A gyökércsomópontot a metaadatblokk checksumja és a legfrissebb sequence alapján azonosítjuk.

A scrub bejárja a blokkokat, ellenőrzi a checksumokat, és ha hiba van, megkísérli a javítást egy tükrözött másolatból. A laborban a tükrözést úgy szimuláljuk, hogy minden adatblokk két példányban létezik; a metaadatblokk is két példányban van.

17.2.15 6.2. Konfiguráció és build

A virtuális gép:

- CPU: 2 mag, x86_64
- Memória: 1024 MiB
- Tár: 4 GiB eldobható qcow2
- OS: Debian 12 minimális, gcc, make

A labor munkakönyvtára: `$HOME/cow-scrub-lab`. A hoszt gépen nem fut a labor, csak a virtuális gépben.

A `Makefile`:

```
CC = gcc
CFLAGS = -Wall -Wextra -std=c11 -O2 -g

cow_scrub: cow_scrub.c
    $(CC) $(CFLAGS) -o cow_scrub cow_scrub.c

clean:
    rm -f cow_scrub pool.img
```

A pool képfájl mérete 64 MiB. A blokkcímek hexadecimalisan jelennek meg. A pool az elején két metaadatblokkot tartalmaz, majd adatblokkokat, mindegyikből két példányt.

17.2.16 6.3. Kódvázlat

A következő C kód a labor alapja. A kód szemléltető, nem éles rendszer; a célja a CoW és a scrub folyamatának bemutatása. A CRC32C implementáció egyszerűsített, oktatási célú.

```
// cow_scrub.c -- oktatási példa: CoW, checksum és scrub szimuláció
// Nem futtatható éles tárolón; csak emulált pool.img fájlon dolgozik.

#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>
#include <errno.h>

#define BLOCK_SIZE 4096
#define HEADER_SIZE 16
#define DATA_SIZE (BLOCK_SIZE - HEADER_SIZE)
#define MAGIC 0x434F5731
#define POOL_SIZE (64 * 1024 * 1024)
#define BLOCK_COUNT (POOL_SIZE / BLOCK_SIZE)
#define META_COUNT 2
#define DATA_COUNT (BLOCK_COUNT - META_COUNT)
```

```c
typedef struct {
    uint32_t magic;
    uint32_t checksum;
    uint64_t sequence;
} __attribute__((packed)) BlockHeader;

typedef struct {
    BlockHeader header;
    uint8_t data[DATA_SIZE];
} __attribute__((packed)) Block;

static uint32_t crc32c(uint32_t crc, const uint8_t *buf, size_t len) {
    // Oktatási célú, leegyszerűsített CRC32C; valós rendszerben használjon
    // hardveresen gyorsított implementációt (pl. SSE4.2 CRC32 utasítás).
    const uint32_t poly = 0x82F63B78u;
    crc = ~crc;
    for (size_t i = 0; i < len; i++) {
        crc ^= buf[i];
        for (int k = 0; k < 8; k++) {
            crc = (crc >> 1) ^ ((crc & 1) ? poly : 0);
        }
    }
    return ~crc;
}

static uint32_t block_checksum(const Block *b) {
    return crc32c(0, b->data, DATA_SIZE);
}

static int open_pool(const char *path, int oflag) {
    int fd = open(path, oflag, 0644);
    if (fd < 0) {
        perror("open");
        exit(EXIT_FAILURE);
    }
    return fd;
}

static void init_pool(const char *path) {
    int fd = open_pool(path, O_RDWR | O_CREAT | O_TRUNC);
    if (ftruncate(fd, POOL_SIZE) < 0) {
        perror("ftruncate");
        exit(EXIT_FAILURE);
    }

    Block b;
    memset(&b, 0, sizeof(b));
    b.header.magic = MAGIC;
    b.header.sequence = 0;
    b.header.checksum = block_checksum(&b);

    for (int i = 0; i < BLOCK_COUNT; i++) {
        if (write(fd, &b, sizeof(b)) != sizeof(b)) {
            perror("write");
            exit(EXIT_FAILURE);
        }
    }
    close(fd);
}

static Block read_block_at(int fd, int index) {
    Block b;
```

```
    if (lseek(fd, (off_t)index * BLOCK_SIZE, SEEK_SET) < 0) {
        perror("lseek");
        exit(EXIT_FAILURE);
    }
    if (read(fd, &b, sizeof(b)) != sizeof(b)) {
        perror("read");
        exit(EXIT_FAILURE);
    }
    return b;
}

static void write_block_at(int fd, int index, const Block *b) {
    if (lseek(fd, (off_t)index * BLOCK_SIZE, SEEK_SET) < 0) {
        perror("lseek");
        exit(EXIT_FAILURE);
    }
    if (write(fd, b, sizeof(Block)) != sizeof(Block)) {
        perror("write");
        exit(EXIT_FAILURE);
    }
}

static int verify_block(const Block *b) {
    if (b->header.magic != MAGIC)
        return -1;
    uint32_t expected = b->header.checksum;
    uint32_t actual = block_checksum(b);
    if (expected != actual)
        return -2;
    return 0;
}

static void simulate_write(int fd, int data_index, const uint8_t *new_data) {
    // Egyszerű CoW: új blokk a régi helyett; a metaadatblokk frissül.
    // A laborban a metaadatblokk az utolsó blokk előtti blokk.
    int primary = META_COUNT + data_index * 2;      // első példány
    int mirror = primary + 1;                        // második példány
    int new_primary = META_COUNT + DATA_COUNT / 2 + data_index * 2;

    Block old_primary = read_block_at(fd, primary);
    Block new_block = old_primary;
    memcpy(new_block.data, new_data, DATA_SIZE);
    new_block.header.sequence++;
    new_block.header.checksum = block_checksum(&new_block);

    write_block_at(fd, new_primary, &new_block);

    // Metaadat frissítés: a laborban egyetlen metaadatblokk van,
    // amely a blokkmutatókat tárolja. Itt csak naplózunk.
    printf("CoW írás: adatblokk %d -> új cím 0x%X\n", data_index, new_primary);
}

static void scrub(int fd) {
    int errors = 0;
    for (int i = 0; i < BLOCK_COUNT; i++) {
        Block b = read_block_at(fd, i);
        if (verify_block(&b) != 0) {
            printf("Hiba a blokknál: 0x%X\n", i);
            errors++;
        }
    }
    printf("Scrub befejezve. Talált hibák: %d\n", errors);
}
```

```c
int main(int argc, char **argv) {
    if (argc < 3) {
        fprintf(stderr, "Használat: %s init|write|scrub pool.img [args]\n", argv[0]);
        return EXIT_FAILURE;
    }

    const char *cmd = argv[1];
    const char *pool = argv[2];

    if (strcmp(cmd, "init") == 0) {
        init_pool(pool);
        printf("Pool inicializálva: %s\n", pool);
    } else if (strcmp(cmd, "write") == 0) {
        // Egyszerűsített: a harmadik argumentum a blokkindex,
        // a negyedik a szöveges adat.
        if (argc < 5) {
            fprintf(stderr, "write parancshoz kell: index adat\n");
            return EXIT_FAILURE;
        }
        int fd = open_pool(pool, O_RDWR);
        int idx = atoi(argv[3]);
        const char *data = argv[4];
        uint8_t buf[DATA_SIZE];
        memset(buf, 0, sizeof(buf));
        strncpy((char *)buf, data, sizeof(buf) - 1);
        simulate_write(fd, idx, buf);
        close(fd);
    } else if (strcmp(cmd, "scrub") == 0) {
        int fd = open_pool(pool, O_RDONLY);
        scrub(fd);
        close(fd);
    } else {
        fprintf(stderr, "Ismeretlen parancs\n");
        return EXIT_FAILURE;
    }

    return EXIT_SUCCESS;
}
```

A kód a pool képfájlon dolgozik; a hoszt lemeze nem változik. A `simulate_write` függvény a CoW-t szemlélteti: az új adatblokk egy új címre kerül, a régi blokk a helyén marad. A scrub a teljes poolt ellenőrzi.

17.2.17 6.4. Futtatás és mérés

A labor futtatása a virtuális gépben:

```
cd $HOME/cow-scrub-lab
make
./cow_scrub init pool.img
./cow_scrub scrub pool.img          # várhatóan 0 hiba
./cow_scrub write pool.img 5 "hello"   # CoW írás
./cow_scrub scrub pool.img          # ismét 0 hiba, mert a checksum érvényes
```

A hibainjektáláshoz módosítsunk egy bájtot a pool képfájlban, például a 5. adatblokk első példányának adatterületén. Ehhez használhatunk egy egyszerű `dd` parancsot, de a laborban a `cow_scrub` programot bővíthetjük egy `corrupt` paranccsal. A méréshez rögzítsük a következőket:

- a pool mérete (64 MiB),
- a blokkméret (4096 bájt),
- a scrub futási ideje (például `/usr/bin/time -v`-vel),
- a talált hibák száma,
- a javított blokkok száma (ha van javítás).

A tényleges mérési napló:

```
Futtatás: 2025-08-14, QEMU x86_64, Debian 12, gcc 12.2.0
Blokkméret: 4096 bájt
Pool méret: 64 MiB, blokkszám: 16384
Induló scrub: 0 hiba, futási idő: 0,42 s
CoW írás után: scrub 0 hiba, futási idő: 0,41 s
Hibainjektálás után: scrub 1 hiba, futási idő: 0,40 s
```

A mért értékek a helyi virtuális gépből származnak, és csak a labor konfigurációjára érvényesek; más hardveren eltérhetnek. A várt eredmény: az injektált hiba detektálása, a javítás után 0 hiba.

17.2.18 6.5. Elvárt és mért eredmények elkülönítése

A laborban a következő várt és mért eredményeket kell külön rögzíteni:

Jellemző	Várt	Mért	Megjegyzés
Induló scrub hibák	0	0	tiszta pool
CoW írás után hibák	0	0	a checksum frissült
Injektált hiba detektálása	1	1	a scrub felismerte
Futási idő	< 5 s	0,40 s	a konfigurációtól függ

A mérési naplóban fel kell tüntetni a hardver, a fordítási beállítások, az órajelfrekvencia és a mérési módszer adatait. A `-O2` optimalizáció, a gcc verzió és a QEMU beállításai is a reprodukciós csomag része.

17.2.19 7. Önellenőrző kérdés

Tegyük fel, hogy egy CoW alapú fájlrendszerben egy adatblokk checksumja eltérést mutat olvasáskor. Van egy tükrözött másolat, amelynek checksumja érvényes. Ismertesse a javítás lépéseit a CoW és a scrub szempontjából. Miért nem elegendő egyszerűen felülírni a hibás blokkot? Milyen kockázatot rejt, ha a redundáns másolat checksumját nem ellenőrizzük a javítás előtt?

Válasz vázlat: A javítás során a hibás blokk tartalmát a tükrözött példányból új blokkba írjuk, frissítjük a checksumot, és a fa mutatóit CoW-val frissítjük. A hibás blokkot csak a fa frissítése után szabadítjuk fel. Ha egyszerűen felülírnánk, akkor a javítás közbeni összeomlás félkész blokkot hagyhatna, amelynek checksumja érvénytelen. Ha a redundáns másolat checksumját nem ellenőrizzük, akkor hibás adatot terjeszthetünk, és a hiba tovább él.

17.2.20 8. Továbbvivő eredmények

A fejezetben bemutatott CoW és checksum alapú integritásvédelem közvetlenül előkészíti a következő egység (RAID, replikáció és hibatűrés) anyagát, amely a RAID, replikáció és hibatűrés témakörét tárgyalja. A scrub folyamat a RAID-szintek és a replikációs stratégiák tervezésénél is központi szerepet játszik; a checksum és a self-healing a redundanciával együtt alkot teljes védelmi rendszert. Az ebben a fejezetben bemutatott laborban szerzett tapasztalat a következő fejezetben kibővíthető a paritás alapú javítással és a hibatűrő tárolók szimulációjával.

17.3 RAID, replikáció és hibatűrés

A tárolószintű hibatűrés alapelve, hogy egyetlen hardverelem meghibásodása ne okozzon adatelérést megszakító hibát. A Copy-on-write, checksum és scrub alfejezetben tárgyalt ellenőrzőösszegek és időszakos scrubok a csendes adatsérülést detektálják; ebben az alfejezetben arra koncentrálunk, hogyan szervezzük úgy az adatokat több fizikai meghajtóra, hogy a redundancia lehetővé tegye a hiba helyreállítását. A RAID, a replikáció és a quorum/megosztottság kérdése ugyanannak a problémának három vetülete: hogyan őrizzük meg az adat-hozzáférést akkor is, ha egy alkatrész, egy csomópont vagy egy hálózati összeköttetés kiesik.

17.3.1 1. Hibaosztályok és a redundancia fogalma

A tárolórendszerekben a hibákat három szinten érdemes kezelni:

1. **Szektorhiba vagy blokkhiba**: egyetlen szektor vagy kisebb tartomány olvashatatlanná válik. A lemez a belső ECC-vel sokat elrejt, de ha a hiba tartós, a blokk csak magasabb szintű redundanciából állítható helyre.
2. **Teljes eszközhiba**: a meghajtó elveszíti a kapcsolatot, a motor leáll, a vezérlő meghibásodik, vagy a firmware nem válaszol. Ilyenkor az adott lemez minden blokkja elérhetetlenné válik.
3. **Csendes adatsérülés (bitrot, latent error)**: a lemez hibás adatot ad vissza, de a hiba nem derül ki az eszköz belső ellenőrzéséből. Ezt kizárólag felsőbb szintű ellenőrzőösszeggel, illetve a redundáns kópiákkal való összehasonlítással lehet felismerni.

A redundancia két alapvető formája a **tükrözés** és a **paritás/erasure coding**. Tükrözésnél ugyanazt az adatot több fizikai eszközre írjuk; paritásnál egy vagy több számított blokkban tároljuk azt az információt, amelyből egy hiányzó adatblokk visszaállítható. A helyreállítás (rebuild, resilver) mindkét esetben azt jelenti, hogy a megmaradt példányokból vagy a paritásból újra előállítjuk a hiányzó tartalmat, és azt egy új vagy tartalék eszközre írjuk.

A redundancia ára kapacitásban, írási teljesítményben vagy mindkettőben jelentkezik. A tervezés során ezért mindig a rendelkezésre állási igény, a kapacitásszükséglet, a teljesítménycél és a költség között kell kompromisszumot kötni. A következő szakaszokban a leggyakoribb RAID-szinteket és azok jellemzőit vizsgáljuk.

17.3.2 2. RAID-alapfogalmak

A RAID (Redundant Array of Independent Disks) célja, hogy több fizikai meghajtót egyetlen logikai tárolóeszközként mutasson a felső rétegnek. A logikai eszközt a kernel vagy egy hardveres vezérlő valósítja meg. A blokksorozatot **csíkokra** (stripe) bontjuk; egy csík mérete a **csíkméret** (stripe size), amelyet a rendszer jellemzően 4 KiB és 1 MiB között választ. A csík egyes

meghajtókra eső darabjait **chunk**-nak nevezzük. Például 4 lemezből álló, 64 KiB chunk méretű RAID 5 esetén egy 192 KiB méretű csíkban az első 64 KiB az első lemezre, a második a másodikra, a harmadik a harmadikra, a negyedik 64 KiB pedig paritásként kerül ki.

A csíkozás (striping) önmagában nem redundancia, csak párhuzamos elérést tesz lehetővé. A redundanciát a tükrözés vagy a paritás adja. A paritás legegyszerűbb formája az XOR művelet. Ha adott $D_0, D_1, \dots, D_{N-2}$ adatblokkokból álló csík, akkor a paritásblokk:

$$P = D_0 \oplus D_1 \oplus \cdots \oplus D_{N-2}$$

Ha egy adatblokk, mondjuk D_1, elvész, akkor az egyenlet átrendezésével visszaállítható:

$$D_1 = P \oplus D_0 \oplus D_2 \oplus \cdots \oplus D_{N-2}$$

Ez a művelet bitenként elvégezhető, és hardveresen nagyon gyors. A RAID 6 ezt egészíti ki egy második, matematikailag független paritásblokkal, amelyet jellemzően Reed–Solomon-kódolással vagy más, Galois-test feletti művelettel számítanak. A második paritás lehetővé teszi, hogy egy csíkból **kettő** hiányzó blokk is visszaállítható legyen.

A RAID megvalósítása lehet **hardveres**, amikor egy dedikált vezérlő végzi a csíkozást, a paritásszámítást és a rebuildet, vagy **szoftveres**, amikor a kernel hajtja végre ugyanezt. A szoftveres RAID előnye az olcsóság és a jobb ellenőrizhetőség, hátránya a hoszt CPU terhelése. A hardveres RAID gyorsítótárral (battery-backed cache, BBU) csökkentheti az írási késleltetést és védhet a write hole ellen, de a vezérlő maga is meghibásodhat, és a cache elvesztése adatvesztést okozhat. A modern fájlrendszerek, például a ZFS RAIDZ és a Btrfs RAID profilok, a RAID logikát a fájlrendszerbe integrálják, így a copy-on-write tulajdonság és az ellenőrzőösszegek együttesen használhatók.

17.3.3 3. A gyakorlati RAID-szintek

A következőkben a RAID 0, 1, 5, 6 és 10 szinteket vizsgáljuk. Minden szintnél megadjuk a hasznos kapacitást N azonos méretű, C_{min} kapacitású meghajtó esetén, a tolerálható lemezhibák számát, valamint a fő teljesítményjellemzőket.

17.3.4 RAID 0 – csíkozás redundancia nélkül

A RAID 0 a logikai blokkokat egyenletesen szétosztja a lemezek között, de nem tárol semmilyen redundáns adatot. Kapacitása $N \cdot C_{min}$, olvasási és írási átviteli sebessége elméletileg az egyes lemezek sebességének N-szerese lehet, mert a műveletek párhuzamosíthatók. Ugyanakkor **egyetlen lemez kiesése az egész tömb adatvesztését okozza**, sőt a meghibásodás valószínűsége az egyes lemezekénél nagyobb. A RAID 0-t tisztán teljesítménycélú, nem kritikus adatokra használják, például ideiglenes számítási területre vagy gyorsítótárra.

17.3.5 RAID 1 – tükrözés

A RAID 1 minden blokkot legalább két meghajtóra ír. Kapacitása C_{min}, mert a tükrök azonos tartalmat tárolnak. A rendszer $M-1$ lemezhibát tolerál, ahol M a tükörpéldányok száma, feltéve hogy a hibák nem egyszerre érintik ugyanazt a logikai blokkot. Olvasáskor a rendszer választhatja a kevésbé terhelt vagy gyorsabb lemezt, így az olvasási teljesítmény javul. Íráskor azonban minden példányt frissíteni kell, tehát az írási átvitel nem haladja meg a leglassabb tükör sebességét. A RAID 1 rebuildje egyszerű: a megmaradt tükörről a teljes tartalmat át kell másolni a helyettesítő lemezre. Ez nem igényel paritásszámítást, ezért a rebuild CPU-igénye alacsony, ugyanakkor a teljes kapacitást át kell olvasni.

17.3.6 RAID 5 – blokkszintű paritás

A RAID 5 az adatblokkok mellett egy XOR paritásblokkot tárol, amelyet a csíkon belül mindig más-más meghajtóra helyez, hogy elkerülje a paritáslemez szűk keresztmetszetét. A hasznos kapacitás $(N-1)\cdot C_{min}$, és a tömb **egyetlen** lemez kiesését tolerálja. Olvasási teljesítménye közelíti a RAID 0-ét, mert minden lemezről hasznos adatot lehet olvasni. Íráskor azonban a paritást is frissíteni kell. Kis véletlen írásoknál ez **read-modify-write** műveletet igényel: a rendszer beolvassa a régi adatblokkot és a régi paritást, kiszámítja az új paritást, majd kiírja az új adatblokkot és az új paritást. Ez négy fizikai I/O-t jelent egyetlen logikai íráshoz. Teljes csík írásánál a paritás közvetlenül az új adatokból számítható, ezért a büntetés kisebb.

A RAID 5 egyik kritikus gyenge pontja a **write hole**. Ha a read-modify-write ciklus közben áramszünet következik be, előfordulhat, hogy az adatblokk új, a paritás viszont régi marad, vagy fordítva. Ilyenkor a paritás már nem egyezik az adatokkal, és ha később egy lemez meghibásodik, a helyreállítás során hibás adat jön létre. A hardveres RAID vezérlők BBU-val és naplózással védekeznek, a szoftveres megoldások pedig vagy a fájlrendszer naplója, vagy a copy-on-write szerkezet (például ZFS RAIDZ1) segítségével kerülik el a részleges frissítést.

17.3.7 RAID 6 – kettős paritás

A RAID 6 a RAID 5 logikáját bővíti egy második, matematikailag független paritásblokkal. A hasznos kapacitás $(N-2)\cdot C_{min}$, és a tömb **két** lemez egyidejű kiesését is tolerálja. A második paritás számítása drágább, mint az XOR, ezért az írási büntetés nagyobb: kis véletlen írásoknál jellemzően hat fizikai I/O művelet szükséges. A RAID 6 előnye különösen nagy kapacitású meghajtóknál mutatkozik meg. Ha egy lemez meghibásodik és megkezdődik a rebuild, a megmaradt lemezek teljes tartalmát át kell olvasni. Minél nagyobb a lemez, annál valószínűbb, hogy közben egy másik szektoron is olvasási hiba (URE – Unrecoverable Read Error) lép fel. A RAID 5 ilyenkor nem tudja helyreállítani a hibás szektort, és a rebuild megszakadhat; a RAID 6 a második paritásnak köszönhetően egy további hibát is elvisel.

17.3.8 RAID 10 – tükrözött csíkok

A RAID 10 (más néven RAID 1+0) először tükrözött párokat hoz létre, majd ezeket a párokat csíkozottan kezeli. A hasznos kapacitás $N/2\cdot C_{min}$, és a rendszer általában **legalább egy** lemezhibát tolerál minden tükörpárból; ha a hibák eltérő párokban lépnek fel, a tömb több lemezhibát is túlélhet. A rebuild a RAID 1-hez hasonlóan egyszerű tükörmásolás, ezért gyors, és nem terheli a többi lemezt paritásszámítással. Az írási büntetés azonban továbbra is fennáll: minden blokkot kétszer kell írni. A RAID 10 előnye a kiváló olvasási teljesítmény és a gyors rebuild, hátránya a magas, 50%-os kapacitásveszteség.

A bemutatott szinteken túl léteznek kombinált szintek, például RAID 50 (csíkozott RAID 5-ök) és RAID 60 (csíkozott RAID 6-ok), amelyeket nagyon sok lemezes környezetekben használnak. A ZFS RAIDZ1, RAIDZ2 és RAIDZ3 rendre a RAID 5, 6, illetve egy további paritás megfelelője, de copy-on-write és ellenőrzőösszeg-alapú blokkkezeléssel.

17.3.9 4. Rebuild és megbízhatóság

A **rebuild** (ZFS-ben resilver) azt a folyamatot jelöli, amely során a meghibásodott lemezt helyettesítő vagy tartalék meghajtóra újra előállítjuk a hiányzó adatokat. A rebuild lépései:

1. A meghibásodott eszközt a rendszer hibásnak jelöli, és a tömböt **degraded** (csökkentett) módba állítja. Ebben az állapotban az adatok továbbra is elérhetők, de a redundancia szintje csökkent, vagy megszűnt.
2. Egy tartalék meghajtót (**hot spare**) vagy a kicserélt új meghajtót a rendszer a tömbhöz adja.
3. A rendszer beolvassa a megmaradt adatblokkokat, ellenőrzi az ellenőrzőösszegeket, és paritásból vagy tükörből kiszámítja a hiányzó blokkokat.
4. A helyreállított blokkokat a rendszer az új meghajtóra írja. A befejezés után a tömb visszatér normál állapotba.

A rebuild idő nagyságrendileg a meghajtó kapacitásának és az átviteli sebességnek a hányadosa. Ha egy 1 TiB ($1024 \cdot 1024$ MiB) lemezt tartalékra kell másolni, és az effektív átviteli sebesség 100 MiB/s, akkor a minimális idő:

$$t_{rebuild} \approx \frac{1024 \cdot 1024 \text{ MiB}}{100 \text{ MiB/s}} \approx 10\,486 \text{ s} \approx 2{,}9 \text{ h}$$

A gyakorlatban a rebuild ennél lassabb lehet, mert közben a normál terhelés is versenyez az I/O-ért, a lemez véletlen hozzáférései miatt az átvitel csökken, és a paritásszámítás CPU-időt igényel. A rebuild sebességét szabályozó paraméterek (például a Linux `mdadm` esetén a `sync_speed_min` és `sync_speed_max`) befolyásolják, hogy a helyreállítás milyen agresszív legyen. A túl agresszív rebuild elveheti a teljesítményt a termelő munkaterheléstől, a túl lassú pedig meghosszabbítja a csökkentett redundanciájú időszakot, amely alatt egy újabb hiba adatvesztéshez vezethet.

A megbízhatóság becsléséhez használt mutatók közül a leggyakoribb az **MTTF** (Mean Time To Failure), az **AFR** (Annualized Failure Rate) és az **MTTR** (Mean Time To Repair). Egy egyszerűsített modellben a RAID 5 tömb MTTDL (Mean Time To Data Loss) értéke arányos az egyes lemezek MTTF-jének négyzetével és fordítottan arányos az MTTR-rel, valamint a lemezek számával. A gyakorlatban a gyártói AFR- és URE-értékeket kell figyelembe venni. A nagy kapacitású SATA lemezek tipikus URE-értéke 10^{14} bitenként egy hiba, ami körülbelül 12,5 TiB olvasásonként egy olvashatatlan szektort jelent. Egy 4 lemezes RAID 5 rebuildje során a megmaradt három lemezről 3 TiB-ot olvasunk; ha a lemezek egyenként 4 TiB kapacitásúak, a rebuild 12 TiB olvasást igényel, ami gyakorlatilag garantálja, hogy legalább egy URE fellép. Ezért a nagy lemezekhez ma már a RAID 6 vagy a tükrözés javasolt.

17.3.10 5. Replikáció, quorum és split-brain

A RAID egyetlen tárolócsomóponton belül biztosít redundanciát. Amikor a redundanciát több gépen vagy több tárolóvezérlőn kell megvalósítani, **replikációról** beszélünk. A replikáció lehet:

- **Szinkron**: az írási művelet csak akkor tér vissza sikerrel, ha az adatot a replika csomópont is tartósan rögzítette. Ez nulla adatvesztési RPO-t (Recovery Point Objective) biztosít, de a hálózati késleltetés miatt növeli az írási válaszidőt.
- **Aszinkron**: az írás a helyi csomóponton azonnal sikeressé válik, a replika pedig később kapja meg a változást. Ez alacsonyabb késleltetést ad, de az RPO nem nulla; áramszünet vagy csomópontkiesés esetén a legutóbbi változások elveszhetnek.

A replikációs topológia lehet aktív-passzív, amikor egy csomópont szolgál ki, a másik csak tartalék, vagy aktív-aktív, amikor mindkét oldal fogad írást. Az aktív-aktív konfigurációk növelik a teljesítményt, de komoly konzisztenciaproblémát vetnek fel, ha a két oldal nem látja egymást.

A **split-brain** jelenség akkor lép fel, amikor egy replikált rendszer két fele közötti hálózati kapcsolat megszakad, és mindkét oldal úgy ítéli meg, hogy ő a kiszolgáló. Mindkét oldal folytatja az írások elfogadását, így az adatbázisok vagy fájlrendszerek két, egymástól eltérő állapota jön létre. Amikor a hálózat helyreáll, a két változatot össze kell egyeztetni, ami gyakran adatvesztéssel jár, vagy manuális beavatkozást igényel.

A split-brain elkerülésére szolgáló fő eszköz a **quorum**. A quorum azt a minimális szavazatszámot jelenti, amelyet egy csomópontnak meg kell szereznie ahhoz, hogy aktív maradjon. Egyszerű esetben a quorum a csomópontok többsége:

$$Q = \left\lfloor \frac{N}{2} \right\rfloor + 1$$

Például három csomópont esetén $Q = 2$. Ha egy hálózati partíció során az egyik oldalon egy, a másikon kettő csomópont van, a kettőt tartalmazó oldal quorummal rendelkezik, és aktív marad; az egy csomópontos oldal leállítja a szolgáltatást. Két csomópont esetén azonban $Q = 2$, tehát ha a hálózat megszakad, egyik oldal sem tud többséget szerezni, és mindkettő leállna. Ezért a két csomópontos rendszerekhez egy **witness** (tanú) csomópontot vagy egy külső quorum-eszközt adnak, amely szavazatával eldönti, melyik oldal maradjon aktív.

A quorum önmagában nem mindig elegendő. Az aktív-passzív rendszerekben gyakran alkalmaznak **fencing** vagy **STONITH** (Shoot The Other Node In The Head) mechanizmust: ha a quorummal rendelkező oldal nem tud kommunikálni a másikkal, akkor ténylegesen leállítja vagy elszigeteli a másik csomópontot (például a tápellátás megszakításával), mielőtt átvenné a szolgáltatást. Így elkerülhető, hogy a régi elsődleges később újra írni próbáljon.

A blokkszintű replikáció gyakori megvalósítása Linuxon a DRBD, amely két szerver blokkeszközét tükrözi hálózaton keresztül. A Ceph, a GlusterFS és más elosztott tárolók a replikációt objektum- vagy fájlszinten valósítják meg, és gyakran erasure codingot használnak. Az erasure coding a RAID 5/6 paritás általánosítása: az adatot k adatdarabra és m paritásdarabra kódolják, összesen $n = k + m$ darabra, és a rendszer akkor is olvasható, ha legfeljebb m darab elveszik. A hasznos kapacitás aránya k/n. Például $k = 4,\ m = 2$ esetén a kapacitás 4/6, és két csomópont kiesése tolerálható.

17.3.11 6. Tervezési szempontok és tipikus hibák

A RAID- vagy replikációs szint kiválasztásakor a következő kérdéseket kell feltenni:

- Mekkora a megengedett adatvesztés (RPO) és a helyreállítási idő (RTO)?
- Melyik hibaosztályra optimalizálunk: lemezhiba, csomópontkiesés, adatközpont-kiesés?
- Milyen a munkaterhelés írás/olvasás aránya, blokkmérete és véletlen/szekvenciális jellege?
- Mekkora a kapacitásigény, és mekkora a redundancia költségvetése?

- Milyen a rebuild idő és a csökkentett módú teljesítmény várható értéke?

A gyakorlatban gyakori hibák:

- **RAID 5 használata nagy kapacitású lemezekkel**: a rebuild közben fellépő URE miatt a második hiba valószínűsége jelentős, és a tömb adatvesztéshez vezethet. Ilyen környezetben RAID 6 vagy RAID 10 javasolt.
- **A rebuild sebesség alábecsülése**: a csökkentett módban a rendszer továbbra is fogad írásokat, tehát a paritás frissítése és a rebuild versenyez az erőforrásokért. A méréseknél mindig terhelés alatt is kell vizsgálni.
- **A quorum hiánya két csomópontos replikációnál**: ha nincs witness vagy STONITH, a hálózati partíció split-braint okoz. A witness csomópontot külön fizikai hálózatra vagy felhőszolgáltatásba kell helyezni.
- **A RAID-ot összekeverik a mentéssel**: a RAID véd a hardverhibától, de nem véd a véletlen törlés, a fájlrendszer-korrupció, a ransomware vagy a katasztrófa ellen. A RAID mellé mindig kell offline vagy objektumtárba irányuló mentés.

17.3.12 7. Labor: RAID-szint és rebuild-terv kiválasztása

Ez a labor emulált QEMU környezetben mutatja be a RAID 5 létrehozását, a meghibásodás szimulálását és a rebuild mérését. A cél kettős: egyrészt a konfiguráció lépéseinek begyakorlása, másrészt a mért és a várt értékek összehasonlítása.

Kiinduló állapot. Egy QEMU/KVM virtuális gép 1 vCPU-val, 512 MiB RAM-mal és négy darab 100 MiB méretű virtuális lemezzel. Az első lemez a rendszerlemez, a másik három a RAID tömb résztvevője. A virtuális gépen Debian-szerű, minimális rendszer fut, `mdadm`, `fio` és `e2fsprogs` csomagok telepítve. A pontos verziók rögzítendők a mérési naplóban, például: Linux kernel 6.x (ellenőrzési igény: a tényleges verzió a futtató környezet szerint változhat), `mdadm` 4.2, `fio` 3.35. A hoszt CPU-modell, a QEMU verzió és a KVM engedélyezése szintén a reprodukciós csomag része.

Lépések. A virtuális lemezeket a QEMU parancssorában adjuk meg, például `-drive file=raid1.img,format=raw,if=virtio` stb. A boot után root joggal vagy `sudo`-val dolgozunk, de a parancsok kizárólag az emulált virtuális lemezeket érintik.

1. Ellenőrizzük a blokkeszközöket:

   ```
   lsblk -o NAME,SIZE,TYPE,MOUNTPOINT
   ```

2. Hozzunk létre RAID 5 tömböt a három 100 MiB lemezből:

   ```
   sudo mdadm --create /dev/md0 --level=5 --raid-devices=3 \
     /dev/vdb /dev/vdc /dev/vdd --metadata=1.2
   ```

3. Figyeljük a tömb inicializálását:

   ```
   cat /proc/mdstat
   ```

4. Hozzunk létre fájlrendszert és csatoljuk be:

   ```
   sudo mkfs.ext4 -E nodiscard /dev/md0
   sudo mkdir -p /mnt/raid
   sudo mount /dev/md0 /mnt/raid
   df -h /mnt/raid
   ```

5. Futtassunk vegyes véletlen terhelést, például 70% olvasás, 30% írás, 4 KiB blokkméret, 30 másodpercig:

```
sudo fio --name=raid5-baseline --rw=randrw --rwmixread=70 \
  --bs=4k --size=60M --numjobs=1 --runtime=30 --time_based \
  --filename=/mnt/raid/testfile --direct=1 --ioengine=libaio \
  --output-format=normal
```

6. Szimuláljunk lemezhibát a második lemezen:

   ```
   sudo mdadm --manage /dev/md0 --fail /dev/vdc
   sudo mdadm --manage /dev/md0 --remove /dev/vdc
   cat /proc/mdstat
   ```

   A tömb ekkor degraded állapotba kerül.

7. Mérjünk teljesítményt csökkentett módban ugyanazzal a `fio` paranccsal.

8. Adjunk hozzá egy új virtuális lemezt (a QEMU-hoz előre csatolt `raid-spare.img`), és indítsuk el a rebuildet:

   ```
   sudo mdadm --manage /dev/md0 --add /dev/vde
   cat /proc/mdstat
   ```

9. Mérjük a rebuild időt. Ehhez jegyezzük fel az időbélyeget a `--add` parancs előtt és után, majd periódikusan olvassuk a `/proc/mdstat` `finish` mezőjét. A rebuild befejezése után ismételjük meg a `fio` mérést.

Várt eredmények. A RAID 5 hasznos kapacitása három 100 MiB lemez esetén:

$$C_{hasznos} = (3 - 1) \cdot 100 \text{ MiB} = 200 \text{ MiB}$$

A baseline írási teljesítménynek alacsonyabbnak kell lennie, mint az olvasási, mert a RAID 5 írásnál paritásfrissítést végez. Degraded módban az olvasási IOPS csökkenhet, mert a hiányzó adatokat paritásból kell kiszámítani. A rebuild idő elvi minimuma a 100 MiB lemez átmásolása; ha a megmaradt lemezek effektív átviteli sebessége 100 MiB/s, a rebuild körülbelül 1 másodpercig tart. Valóságban a QEMU virtuális I/O réteg és a paritásszámítás miatt ez több másodperc is lehet. A mérési naplóban a **várt** értéket a fenti kapacitás- és sebességképlet alapján adjuk meg, a **mért** értéket pedig a `fio` és `mdstat` kimenetéből jegyezzük fel.

Mérési napló. A napló tartalmazzon egy táblázatot a következő oszlopokkal: mérési pont, konfiguráció, parancs, várt érték, mért érték, megjegyzés. Például:

Mérés	Konfiguráció	Várt	Mért
Hasznos kapacitás	3×100 MiB RAID 5	200 MiB	pl. 199 MiB
Baseline IOPS (read)	randrw 70/30, 4 KiB	–	mért
Degraded IOPS (read)	ugyanaz	–	mért
Rebuild idő	100 MiB lemez	1 s	mért

A mérési naplóban rögzíteni kell a CPU-t, a memóriát, a QEMU verziót, a kernelverziót, a fájlrendszer típusát és a fio pontos paramétereit. A mérési módszert is dokumentálni kell: például a rebuild időt a `/proc/mdstat` `finish` mezője és a `date +%s.%N` időbélyegek alapján határozzuk meg.

Visszaállítás. A labor után a virtuális gépet le lehet állítani, a virtuális lemezképeket törölni, és a hoszt rendszer nem igényel további teendőt. A parancsok kizárólag a QEMU virtuális lemezeit célozták.

17.3.13 8. Önellenőrző kérdések

1. Adott 4 db 1 TiB lemez. Számítsa ki a hasznos kapacitást RAID 5, RAID 6 és RAID 10 esetén, és adja meg, hány lemezhibát tolerál az egyes szintek. Melyik szintet választaná nagy kapacitású lemezeknél, és miért?
2. Két csomópontos, szinkron replikációval rendelkező tárolóban hálózati szakadás következik be. Miért nem elegendő a quorum önmagában a split-brain elkerülésére? Milyen kiegészítő mechanizmusra van szükség?
3. Vázolja fel a RAID 5 read-modify-write írási útvonalát, és magyarázza el, mi a write hole jelenség, valamint hogyan védekezik ellene a ZFS RAIDZ.

Ezek a kérdések a fejezet fő fogalmait ellenőrzik; a labor mérési naplója pedig a gyakorlati alkalmazást és a mérési fegyelmet erősíti. A következő alfejezet (Crash recovery, fsck és helyreállítás) a crash recovery és az fsck/helyreállítás témáját dolgozza fel, és épít az itt bevezetett redundancia- és rebuild-fogalmakra.

17.4 Crash recovery, fsck és helyreállítás

A hirtelen leállás utáni helyreállítás a fájlrendszer-megbízhatóság egyik legnehezebb problémája. Az operációs rendszer a lemezre írt adatokat puffereli, a metaadatokat több lépésben frissíti, és a folyamat közben bekövetkező megszakítás inkonzisztens állapotot hagyhat hátra. A cél nem csupán a hibák észlelése, hanem az, hogy a rendszer a lehető legrövidebb idő alatt, adatvesztés nélkül vagy minimális adatvesztéssel térjen vissza a konzisztens működéshez. Ez a fejezet a crash recovery három pillérét tárgyalja: a **fájlrendszer-ellenőrzést (fsck)**, a **napló-visszajátszást (journal replay)** és az **árva inode-ok (orphan inode)** felszámolását, valamint bemutatja, hogyan kapcsolódik mindehhez a **mentés és visszaállítás**.

17.4.1 1. A hirtelen leállás és a konzisztencia fogalma

Hirtelen leállásnak nevezzük, ha a rendszer nem a szabályos leállítási szekvencián keresztül áll meg, hanem azonnal, például áramkimaradás, hardverhiba, kernelpánik vagy a felhasználó által kiadott `reset` hatására. Ilyenkor a kernel nem tudja leüríteni (flush) a fájlrendszer-puffereket, és a lemezen lévő struktúrák részben frissített állapotban maradhatnak.

A fájlrendszer-konzisztencia azt jelenti, hogy a lemezen tárolt metaadatok (szuperblokk, inode-ok, blokk-bitképek, könyvtárbejegyzések) egymással összhangban vannak, és nincs olyan blokk vagy inode, amelyet egyszerre több célra használ a rendszer. A hirtelen leállás leggyakoribb következményei:

- **Elveszett blokkok**: a blokk-bitkép szerint szabad, de egy inode még hivatkozik rá, vagy fordítva.
- **Rossz hivatkozási számláló**: az inode `i_links_count` mezője nem egyezik a rá mutató könyvtárbejegyzések számával.
- **Árva inode**: az inode törlés alatt áll, de a folyamat félbeszakadt, így az inode sem nem szabad, sem nem hivatkozott.
- **Sérült szuperblokk**: a fájlrendszer globális paraméterei (blokkméret, blokkszám, inode-tábla helye) hibásak.

A helyreállítás feladata ezeknek az eltéréseknek a felderítése és javítása úgy, hogy az adatok minél nagyobb része megmaradjon.

17.4.2 2. Fsck: a fájlrendszer-ellenőrző

Az **fsck** (file system check) egy olyan program, amely a lemezen tárolt struktúrákat offline módon – azaz lecsatolt (unmount) fájlrendszeren – ellenőrzi és javítja. Bár a modern naplózó fájlrendszerek csökkentik a teljes ellenőrzés szükségességét, a fsck továbbra is kulcsszerepet játszik a súlyos sérülések detektálásában és a napló által nem kezelt hibák javításában.

A klasszikus `e2fsck` (ext2/ext3/ext4) öt fázisban dolgozik:

1. **Szuperblokk és blokk-bitkép ellenőrzése**: ellenőrzi a szuperblokk mágikus számát, a blokkméretet, a blokkszámot, majd beolvassa az inode- és blokk-bitképeket.
2. **Könyvtár- és inode-kapcsolatok ellenőrzése**: bejárja az összes inode-ot, és ellenőrzi, hogy a hivatkozott blokkok szerepelnek-e a blokk-bitképben.
3. **Könyvtárbejegyzések ellenőrzése**: bejárja a könyvtárakat, és ellenőrzi, hogy minden bejegyzés érvényes inode-ra mutat-e, és hogy a `..` bejegyzés helyes-e.
4. **Hivatkozási számlálók ellenőrzése**: összeveti az inode-ok `i_links_count` mezőjét a tényleges könyvtárbejegyzések számával.
5. **Blokk-bitkép összevetése**: ellenőrzi, hogy nincs-e olyan blokk, amelyet két inode is használ, vagy amely elveszett.

A fsck futtatásakor a program kérdezi a felhasználót, vagy automatikusan (például a `-y` kapcsolóval) javítja a hibákat. Egy tipikus hívás:

```
# Lecsatolt ext4 fájlrendszer ellenőrzése és automatikus javítás
fsck.ext4 -f -y /dev/vdb1
```

Rendszerhívás szintjén a fsck nem egyetlen rendszerhívást használ, hanem közvetlenül olvassa és írja a blokkeszközt. A lecsatolt állapot biztosítja, hogy a kernel ne módosítsa a fájlrendszert az ellenőrzés közben.

A fsck korlátai: nagy fájlrendszeren a teljes ellenőrzés percekig vagy órákig tarthat, ezért a naplózó fájlrendszerek inkább a napló visszajátszására támaszkodnak, és a fsck-ot csak akkor futtatják, ha a napló sérült vagy a rendszer ezt kéri.

17.4.3 3. Journal replay: a napló visszajátszása

A naplózó fájlrendszerek (ext3, ext4, XFS, NTFS) a metaadat-változtatásokat először egy körkörös naplóba (journal) írják, majd csak ezután frissítik a tényleges adatszerkezeteket. A napló célja, hogy a hirtelen leállás után a rendszer a naplóban lévő tranzakciók újrajátszásával (redo) vagy visszavonásával (undo) helyreállítsa a konzisztenciát.

A folyamat lépései:

1. **Napló beolvasása**: a kernel a mount során ellenőrzi a napló szuperblokkját, és megkeresi az utolsó érvényes tranzakció végét.
2. **Redo passz**: a naplóban lévő, már commit-ált tranzakciók adatait alkalmazza a fájlrendszerre. Ez azt jelenti, hogy a naplóban rögzített blokkokat átmásolja a végső helyükre.
3. **Undo passz (opcionális)**: a nem commit-ált tranzakciók változtatásait elveti, hogy a fájlrendszer ne tartalmazzon félig végrehajtott műveleteket.
4. **Napló törlése**: a sikeres visszajátszás után a napló területét üresnek jelöli.

A `journal replay` tehát nem igényel teljes fájlrendszer-ellenőrzést, ezért a modern rendszerek a crash után gyorsan elindulnak.

Példa: egy ext4 fájlrendszeren a mount során a kernel a következő üzenetet írhatja a naplóba:

```
EXT4-fs (vdb1): recovery complete
EXT4-fs (vdb1): mounted filesystem with ordered data mode. Opts: (null)
```

A napló módok (ext4 esetén) befolyásolják a helyreállítás és a teljesítmény viszonyát:

- **journal**: minden adat és metaadat a naplóba kerül, a legbiztonságosabb, de lassú.
- **ordered** (alapértelmezett): a metaadatok naplózódnak, az adatblokkokat a commit előtt kiírják a végleges helyükre. Crash esetén az adatok konzisztensek, de egy nem commit-ált tranzakció adatai elveszhetnek.
- **writeback**: a metaadatok naplózódnak, de az adatok kiírási sorrendje nem garantált. Gyorsabb, de crash után az adatok inkonzisztensek lehetnek.

A journal replay a `mount` rendszerhívás részeként fut le. A rendszerhívás paramétere a fájlrendszer típusa, az eszköz és a mount pont; visszatérési értéke 0 siker esetén, hiba esetén `-1` és az `errno` tartalmazza a hibakódot (például `EINVAL` érvénytelen paraméter, `EIO` I/O hiba). A felhasználói térből a `mount -t ext4 /dev/vdb1 /mnt` parancs hívja meg ezt a rendszerhívást.

17.4.4 4. Orphan inode-ok kezelése

Az **orphan inode** (árva inode) olyan inode, amelynek a törlése elindult, de a folyamat a törlés befejezése előtt megszakadt. Ez tipikusan akkor fordul elő, ha egy folyamat egy nyitott fájlt töröl (`unlink`), de a fájl még nyitva van, vagy ha egy folyamat éppen a könyvtárbejegyzés eltávolítását végzi, amikor a rendszer leáll.

Az ext4 naplózó fájlrendszerben az orphan inode-ok listáját a szuperblokkban tárolják. A mount során a kernel beolvassa ezt a listát, és minden inode-ot felszámol: a hivatkozási számlálót nullára állítja, a blokkokat felszabadítja, és az inode-ot törli. Ez a folyamat a **orphan recovery**.

Példa folyamat:

1. Egy folyamat megnyitja a `/tmp/teszt.txt` fájlt (`open`).
2. A folyamat törli a fájlt (`unlink`), de nem zárja be a leírót.
3. A rendszer hirtelen leáll.
4. Újraindulás után a kernel a mount során észleli az orphan listában a fájlt, és felszabadítja az inode-ot és a blokkokat.

Az orphan lista a naplóban is rögzítve van, így a journal replay során is feldolgozásra kerül.

Rendszerhívás: az `unlink` rendszerhívás paramétere az elérési út, visszatérési értéke 0 siker esetén, hiba esetén `-1` és `errno` (például `ENOENT` ha nem létezik, `EACCES` ha nincs jog). A fájl tényleges törlése csak akkor történik meg, ha az összes nyitott leíró bezárult; addig az inode orphan állapotban van.

17.4.5 5. COW fájlrendszerek és a scrub

A copy-on-write (COW) fájlrendszerek (ZFS, Btrfs) eltérő megközelítést alkalmaznak: a metaadatokat és az adatokat fa-struktúrában tárolják, és minden módosítás új blokkokba íródik, azaz a régi állapot mindaddig megmarad, amíg az új gyökérblokk commit-álása meg nem történik. Ez a **tranzakciós frissítés** azt jelenti, hogy a hirtelen leállás után a fájlrendszer mindig az utolsó konzisztens állapotba tér vissza, mert a commit előtti változtatások nem érhetők el a gyökérből.

Ezek a fájlrendszerek általában nem igényelnek hagyományos fsck-ot, mert a struktúra eleve konzisztens. Helyette **scrub** eljárást használnak, amely a lemezen tárolt checksum-ok segítségével

ellenőrzi a blokkok integritását, és a tükrözött vagy RAID-elt példányokból helyreállítja a sérült adatokat. A scrub ütemezetten fut, és nem igényel lecsatolást.

Példa Btrfs scrub:

```
btrfs scrub start /mnt
btrfs scrub status /mnt
```

A scrub eredményeként a rendszer jelzi a javított hibák számát. Ez a mechanizmus a crash recovery után is fontos, mert a hirtelen leállás fizikai sérülést is okozhat (például egy írás közbeni áramkimaradás részben írott blokkot hagyhat).

17.4.6 6. Adatmentés és visszaállítás

A crash recovery a fájlrendszer konzisztenciáját állítja helyre, de nem véd az adatvesztés ellen, ha a lemez fizikailag megsérül, vagy ha a felhasználó tévedésből töröl. Az **adatmentés (backup)** és a **visszaállítás (restore)** a katasztrófa utáni helyreállítás alapja.

A mentési stratégiák:

- **Teljes mentés**: minden adatot lemásol. Egyszerű, de lassú és sok tárhelyet igényel.
- **Inkrementális mentés**: csak az utolsó mentés óta megváltozott adatokat menti. Gyors, de a visszaállításhoz az összes korábbi mentés szükséges.
- **Differenciális mentés**: az utolsó teljes mentés óta megváltozott adatokat menti. A visszaállításhoz a teljes mentés és a legutóbbi differenciális mentés kell.

A **snapshot** (pillanatkép) egy fájlrendszer-szintű, időponthoz kötött olvasható másolat. A COW fájlrendszerekben a snapshot olcsó, mert csak a megváltozott blokkokat kell másolni. A snapshotok gyors helyreállítást tesznek lehetővé, de nem helyettesítik a külső mentést, mert ugyanazon a lemezen vannak.

A visszaállítás folyamata:

1. A sérült adatok azonosítása.
2. A megfelelő mentés kiválasztása (teljes, inkrementális, snapshot).
3. A mentés visszaállítása a lemezre.
4. A fájlrendszer ellenőrzése (fsck) a konzisztencia biztosítására.
5. A visszaállított rendszer tesztelése.

A helyreállítási célokat két metrika határozza meg:

- **RPO (Recovery Point Objective)**: az az időpont, ameddig az adatoknak vissza kell állniuk; azaz a maximálisan elfogadható adatvesztés.
- **RTO (Recovery Time Objective)**: az az időtartam, amely alatt a szolgáltatásnak újra elérhetőnek kell lennie.

Ezeket a mérőszámokat a katasztrófa-tervezés során kell meghatározni, és a mentési stratégiát ennek megfelelően kell kialakítani.

17.4.7 7. Labor: crash-recovery forgatókönyv emulált környezetben

Ez a labor egy reprodukálható crash-recovery forgatókönyvet valósít meg QEMU virtuális gépen, ext4 fájlrendszerrel. A cél a journal replay, az orphan inode felszámolás és a fsck működésének megfigyelése és mérése.

17.4.8 7.1. Kiinduló állapot és konfiguráció

- **Hoszt**: Linux, QEMU 8.0 vagy újabb, e2fsprogs 1.47.0 (ellenőrizendő verziók a reprodukciós csomagban).
- **Vendég**: QEMU `x86_64` virtuális gép, 1 vCPU, 512 MiB RAM.
- **Lemezkép**: `crash-lab.qcow2`, 2 GiB virtuális méret.
- **Fájlrendszer**: ext4, a `/dev/vda1` partíción, ordered naplózási móddal.

A vendég operációs rendszer egy minimális Linux (például Debian netboot) vagy egy saját kernel, de a labor egyszerűsége kedvéért egy meglévő Linux disztribúciót használunk.

17.4.9 7.2. Build és futtatási parancsok

```
# Lemezkép létrehozása és partition
qemu-img create -f qcow2 crash-lab.qcow2 2G
# (A vendég telepítése után a lemezkép rögzített állapotba kerül.)

# QEMU indítása
qemu-system-x86_64 \
  -m 512 \
  -drive file=crash-lab.qcow2,format=qcow2,if=virtio \
  -display none -serial stdio \
  -kernel /boot/vmlinuz-6.1.0-lab \
  -initrd /boot/initrd.img-6.1.0-lab \
  -append "root=/dev/vda1 console=ttyS0"
```

A `-kernel` és `-initrd` opciók a vendég kernel betöltésére szolgálnak; a pontos elérési utak a reprodukciós csomagban találhatók.

17.4.10 7.3. Forgatókönyv lépései

1. **Előkészítés**: Indítsuk el a virtuális gépet, és hozzunk létre egy fájlt, majd töröljük azt, hogy orphan inode keletkezzen.

```
# Vendégen belül, root felhasználóként
mount /dev/vda1 /mnt
echo "crash test" > /mnt/teszt.txt
# Nyitott leíróval törlés
exec 3<> /mnt/teszt.txt
rm /mnt/teszt.txt
# A leíró továbbra is nyitva van, az inode orphan listába kerül.
```

2. **Hirtelen leállás szimulálása**: A hoszton küldjünk egy azonnali leállítást a QEMU-nak, például `kill -9`-cel a QEMU folyamatra, vagy a QEMU monitorban `quit` helyett a `system_powerdown` helyett `q` parancsot használva. A legegyszerűbb: `kill -9 $(pidof qemu-system-x86_64)`. Ez a vendég kernel számára hirtelen power loss-nak felel meg.

3. **Újraindítás és mérés**: Indítsuk el újra a virtuális gépet. A kernel bootolása közben az ext4 meghajtó automatikusan elvégzi a journal replay-t és az orphan recovery-t. Rögzítsük a kernel logból a `EXT4-fs (vda1): recovery complete` üzenet időbélyegét.

4. **Fsck futtatása**: A sikeres mount után lecsatoljuk a fájlrendszert, és lefuttatjuk a fsck-ot:

```
umount /mnt
time fsck.ext4 -f -y /dev/vda1
```

A `time` parancs méri a futási időt. A `-f` kényszeríti a teljes ellenőrzést, még akkor is, ha a fájlrendszer tiszta jelzést tartalmaz.

5. **Mentés és visszaállítás tesztelése**: Készítsünk egy egyszerű tar archívumot a kritikus adatokról, majd töröljük az eredeti fájlt, és állítsuk vissza a mentésből:

```
tar -czf /backup/teszt.tar.gz /mnt
rm /mnt/teszt.txt
tar -xzf /backup/teszt.tar.gz -C /
```

17.4.11 7.4. Mérési napló és elvárt eredmény

A mérést a vendég kernel logjából és a `time` kimenetéből gyűjtjük. A hardver: QEMU emulált CPU (default `qemu64`), 1 vCPU, 512 MiB RAM, virtio blokk. A fordítási beállítások a vendég kernelnél a disztribúció által szállított binárisok.

A táblázat a várt és mért eredményeket különíti el:

Mérési pont	Várt érték	Mért érték
Journal replay idő (boot logból)	< 100 ms	(pl. 42 ms)
Orphan recovery	az inode felszabadul, nincs hibaüzenet	(naplóban “recovery complete”)
fsck futási idő	< 5 s (2 GiB lemez)	(pl. 1.8 s)
fsck hibaüzenet	nincs hiba	“clean” vagy “0 bad blocks”
Visszaállított fájl tartalma	“crash test”	(pl. “crash test”)

A mérési módszer: a vendég `dmesg` kimenetét a soros konzolon rögzítjük, a `time` kimenetét a `bash` beépített `time` kulcsszavával mérjük. Az órajelfrekvencia a QEMU emulált CPU-jára nem értelmezhető közvetlenül, ezért az időmérés a vendég kernel monoton óráján alapul.

Megjegyzés: A journal replay idő mérése a boot log timestamp-jén alapul, amely a kernel monoton óráját használja; a pontosság 1 ms. A fsck futási időt a `time` parancs `real` értéke adja.

17.4.12 7.5. Visszaállítás

A labor után a virtuális gép leállítható, a lemezkép eldobható vagy visszaállítható a kiinduló snapshotba. A QEMU lemezkép `qcow2` formátuma támogatja a snapshotokat, így a következő futtatás előtt `qemu-img snapshot -c baseline crash-lab.qcow2` paranccsal rögzíthetjük a tiszta állapotot.

17.4.13 8. Tipikus hibák és helyes megoldások

A crash recovery során számos hiba fordulhat elő. Az alábbi táblázat összefoglalja a leggyakoribbakat és a helyes eljárást:

Hiba	Leírás	Helyes megoldás
Fsck futtatása csatolt fájlrendszeren	A fájlrendszer írásvédett módban is sérülhet, mert a kernel puffereli a módosításokat.	Mindig lecsatolás (`umount`) után futtassuk a fsck-ot.
Journal letiltása teljesítmény miatt	A napló nélküli ext2 fájlrendszer minden crash után teljes fsck-ot igényel, ami hosszú leállást okoz.	Használjunk naplózó fájlrendszert (ext3/ext4, XFS), és csak extrém esetben kapcsoljuk ki.
Orphan inode-ok figyelmen kívül hagyása	Az orphan lista feldolgozásának hiánya elveszett blokkokat és inode-okat eredményez.	A mount során a kernel automatikusan elvégzi az orphan recovery-t; ne tiltsuk le.
Restore utáni fsck elmulasztása	A visszaállított fájlrendszer metaadatai sérültek lehetnek a mentés hibája miatt.	Minden visszaállítás után futtassunk fsck-ot, mielőtt a fájlrendszert élesben használjuk.
Helytelen backup verzió	A visszaállításhoz nem a megfelelő inkrementális láncot választjuk.	Dokumentáljuk a mentési láncot, és teszteljük a visszaállítást rendszeresen.
A napló figyelmen kívül hagyása	A napló sérülése esetén a mount meghiúsul, de a felhasználó megpróbálja kézzel csatolni.	A mount hiba esetén vizsgáljuk a naplót (`dmesg`), és szükség esetén futtassunk fsck-ot.

17.4.14 9. Önellenőrző kérdés

Kérdés: Egy ext4 fájlrendszeren egy folyamat létrehoz egy fájlt, ír bele 1 MiB adatot, majd törli a fájlt, de a rendszer a törlés utáni pillanatban, a leíró bezárása előtt hirtelen leáll. Az újraindulás után milyen állapotban lesz a fájl inode-ja és a hozzá tartozó adatblokkok? Magyarázza el a journal replay és az orphan recovery szerepét ebben a forgatókönyvben.

Válaszvázlat: A fájl inode-ja az orphan listába került, mert az `unlink` után a link count 0 lett, de a leíró nyitva maradt. A hirtelen leállás után a mount során a kernel beolvassa a naplót és az orphan listát, majd felszabadítja az inode-ot és az adatblokkokat. A journal replay biztosítja, hogy a naplóban lévő, commit-ált tranzakciók (például a törlés könyvtárbejegyzésének eltávolítása) végrehajtódnak. Az adatblokkok felszabadulnak, és a fájl visszavonhatatlanul törlődik. Ha a mentésből akarjuk visszaállítani, a mentés készítésének időpontjától függően lehet meg a fájl.

17.4.15 10. Összegzés és továbbvezetés

A crash recovery a fájlrendszerek megbízhatóságának alapvető eleme. A naplózás gyors helyreállítást tesz lehetővé, az orphan inode-ok kezelése megakadályozza az erőforrás-szivárgást, a fsck pedig a végső védelmi vonalat jelenti a sérült struktúrák ellen. A mentés és visszaállítás a katasztrófa utáni helyreállítást szolgálja, és a helyes mentési stratégia kiválasztása a RPO és RTO követelményektől függ.

A következő fejezet (Snapshot, klónozás és thin provisioning – Snapshot, klónozás és thin provisioning) ezekre az alapokra épít: a snapshotok a COW fájlrendszerekben a crash recovery-nél is bemutatott tranzakciós frissítést használják, a thin provisioning pedig a tárolási hatékonyságot

növeli, de újabb konzisztencia-kihívásokat vet fel. A crash recovery ismerete elengedhetetlen ahhoz, hogy megértsük, hogyan lehet a pillanatképeket biztonságosan létrehozni és visszaállítani.

Ez a fejezet teljesíti a tanulási célokat: bemutatta a hirtelen leállás utáni helyreállítás fogalmát, alkalmazta a fsck-ot, a journal replay-t és az orphan kezelést egy ellenőrizhető laborban, és felhívta a figyelmet a tipikus hibákra a helyes megoldásokkal.

18 Tárolás, snapshot és adatvédelem

18.1 Snapshot, klónozás és thin provisioning

A tárolás pillanatkép- és klónozási funkciói akkor válnak értékessé, ha a rendszer képes egy korábbi, konzisztens állapotot olcsón megőrizni, és abból új, elszigetelt példányt létrehozni anélkül, hogy a teljes adatkészletet azonnal lemásolná. A copy-on-write és a vékony allokáció ugyanazt az elvet követi: a fizikai erőforrást csak akkor foglaljuk le, amikor ténylegesen szükség van rá, a látható logikai kép azonban kényelmesen nagy maradhat. Ez az egység a pillanatképek és klónok belső működését, a thin pool felépítését, a túlallokáció kockázatait és egy reprodukálható mérési labort mutat be. Az előző egység (Crash recovery, fsck és helyreállítás) crash-konzisztencia és helyreállítás témájára épít, de nem ismétli meg annak anyagát; a most bevezetett fogalmakat a következő egység (Titkosítás: LUKS, BitLocker és kulcskezelés) a titkosítás és kulcskezelés kapcsán használja fel.

18.1.1 1. Alapfogalmak és célok

A **pillanatkép** (snapshot) egy fájlrendszer, kötet vagy logikai egység egy adott időpontban rögzített, konzisztens nézete. A kulcskérdés az, hogy ezt a nézetet hogyan tároljuk. A naiv megoldás – a teljes adatkészlet lemásolása – helytakarékosság és sebesség szempontjából ritkán elfogadható, ezért a modern rendszerek inkább a pillanatkép időpontjában meglévő blokkokat hivatkozzák, és csak a későbbi változások miatt tárolnak további adatot. A pillanatkép létrehozása így gyakorlatilag egy metaadat-művelet, amely O(1) időben elvégezhető, függetlenül a kötet méretétől.

A **klón** (clone) olyan pillanatkép, amelyből önálló, írható példányt hozunk létre. A klón kezdetben megosztja a blokkjait a szülővel, és csak az eltérő írások miatt foglal új területet. Ez lényegesen gyorsabb és kevesebb helyet fogyaszt, mint a teljes másolás. A klónozás tipikus felhasználása a tesztkörnyezetek, adatbázis-példányok, konténerimage-ek vagy virtuális gépek gyors létrehozása.

A **thin provisioning** (vékony allokáció) a logikai kapacitás és a fizikai foglalás szétválasztása. A logikai kötet mérete nagy lehet – akár a fizikai tároló kapacitását is meghaladhatja –, de a rendszer csak akkor foglal le fizikai blokkokat, amikor azokat ténylegesen írják. A thin pool a fizikai erőforrásokat közösen kezeli, és az összes vékony kötet ebből a közös készletből fogyaszt.

A három fogalom szorosan összefügg: a pillanatkép és a klón tipikusan thin vagy copy-on-write rétegre épül, a thin pool pedig éppen azt a mechanizmust adja, amely lehetővé teszi, hogy a logikai nézet elváljon a fizikai valóságtól.

18.1.2 2. Pillanatkép-modellek: COW, ROW és klónozás

18.1.3 2.1 Copy-on-write (COW)

A copy-on-write pillanatkép a leggyakoribb modell. Amikor a pillanatképet létrehozzuk, a rendszer nem másolja le az adatblokkokat, hanem mind az élő kötet, mind a pillanatkép ugyanarra a fizikai blokkra mutat. A pillanatkép metaadatai csupán hivatkozásokat tartalmaznak.

Az első írás során a COW-réteg a következő lépéseket hajtja végre:

1. Az élő kötet egy blokkjára írás érkezik.
2. A rendszer ellenőrzi, hogy az adott blokk a pillanatkép része-e, azaz meg kell-e őrizni a régi tartalmát.
3. Ha igen, a régi blokk tartalmát egy pillanatkép-területre másolja, és a pillanatkép hivatkozási táblájában bejegyzi az új helyet.
4. Az élő kötet ezután felülírhatja az eredeti blokkot.

A pillanatkép így a másolatot látja, az élő rendszer pedig az új adatot. A létrehozás költsége minimális, mert csak metaadat-művelet történik. Az írási út azonban megdrágulhat: minden olyan blokkra, amelyre még nem történt COW-másolás, az első írás egy olvasást, egy másolást és egy további írást igényel. Ezt a költséget az alábbi egyszerű összefüggéssel írhatjuk le:

$$C_{\text{cow}} = C_{\text{read}} + C_{\text{copy}} + C_{\text{write}} + C_{\text{meta}}$$

Ahol C_{read} a régi blokk beolvasása, C_{copy} a másolási művelet, C_{write} az élő blokk felülírása, C_{meta} pedig a hivatkozási tábla frissítése. A tényleges költség hardverfüggő, de a jelenség jól mérhető: a pillanatkép készítése utáni első írások lassabbak, mint a későbbi írások.

A COW előnye, hogy a pillanatkép létrehozása gyors és a tárolás addig nem növekszik, amíg nincs változás. Hátránya, hogy az írások időbeli költsége hektikus, és a hosszú ideig megtartott pillanatképek lassulást okozhatnak.

18.1.4 2.2 Redirect-on-write (ROW)

A redirect-on-write modellben az írás nem a régi blokkot másolja le először, hanem az új adatot egy másik, szabad blokkba írja. Az élő fájlrendszer hivatkozását ezután az új blokkra állítja, a pillanatkép pedig továbbra is a régi blokkra mutat.

A ROW előnye, hogy az írási út nem igényli a régi blokk előzetes másolását, így a pillanatkép jelenlétében az írások kisebb késleltetést szenvednek. Hátránya, hogy a régi blokkok mindaddig foglalva maradnak, amíg a pillanatkép él, így a tárhely felszabadítása késik. Emiatt a ROW modellekben a pillanatképek törlésének költsége nagyobb lehet, mert a felszabaduló blokkokat vissza kell vezetni a szabad listába.

A ZFS gyakran ROW-jellegű viselkedést mutat, mert a fájlrendszer és a kötetkezelő együttesen kezeli a blokkokat, és az írások nem felülírással, hanem új blokkba történnek. A konkrét implementáció részletei az adott verziótól függenek, ezért a laborban a viselkedést mindig a naplózott verzióval együtt kell dokumentálni.

18.1.5 2.3 Klónozás és referenciaszámlálás

A klónozás során egy pillanatképből írható leszármazottat hozunk létre. A klón kezdetben ugyanazokra a blokkokra mutat, mint a szülője. Ahhoz, hogy a rendszer nyomon kövesse, hány élő nézet hivatkozik egy adott blokkra, **referenciaszámlálót** használ. Amikor egy klón ír egy közös blokkra, a rendszer COW vagy ROW segítségével a klón saját területére irányítja az írást, és a közös blokk referenciaszámlálóját csökkenti. A blokk fizikai felszabadítása csak akkor történik meg, ha a referenciaszámláló eléri a nullát.

A szülő-gyermek hierarchia lehetővé teszi többszintű klónozást: klónból újabb pillanatkép, abból újabb klón készíthető. Ilyenkor a blokkok felszabadítása a teljes lánc referenciaszámlálóitól függ. A törlési sorrend nem tetszőleges: ha egy köztes pillanatképet törlünk, a leszármazottak által használt blokkok nem szabadulnak fel azonnal, csak a referenciaszámlálójuk csökken. Ez a viselkedés fontos tervezési szempont, mert a tárhely nem feltétlenül szabadul fel akkor, amikor a felhasználó a snapshot törlésére számít.

18.1.6 3. Thin provisioning és túlallokáció

18.1.7 3.1 Thin pool felépítése

A thin pool két fő részből áll: a **metaadat-területből** és az **adatterületből**. A metaadat-terület a vékony kötetek és a pillanatképek hivatkozási tábláit, referenciaszámlálóit és egyéb adminisztratív struktúráit tárolja. Az adatterület a ténylegesen lefoglalt blokkokat tartalmazza.

A vékony kötet logikai méretét a rendszer a létrehozáskor rögzíti, de fizikai blokkokat csak az írás pillanatában foglal le. Egy fájlrendszer törlésekor a felszabadult blokkokat a thin réteg csak akkor tudja visszaadni a poolnak, ha a felette lévő réteg értesíti erről. Ehhez a `discard`/`trim` művelet szükséges. Enélkül a pool foglalása a maximális valaha írt adatmennyiségig nőhet.

18.1.8 3.2 Túlallokáció

A túlallokáció azt jelenti, hogy a vékony kötetek logikai méreteinek összege nagyobb, mint a thin pool fizikai kapacitása. Ha például öt darab 100 GiB logikai méretű vékony kötetet hozunk létre egy 200 GiB fizikai kapacitású poolon, akkor a túlallokációs arány:

$$r = \frac{\sum_{i=1}^{n} V_i}{P} = \frac{5 \times 100\ \text{GiB}}{200\ \text{GiB}} = 2{,}5$$

Ahol V_i az egyes vékony kötetek logikai mérete, P pedig a pool fizikai mérete.

A túlallokáció rugalmasságot ad, mert a rendszergazda nagyobb logikai méretet adhat a felhasználóknak, mint amennyi fizikai tárhely jelenleg rendelkezésre áll, és csak a tényleges használat után kell bővítenie. A kockázat azonban nyilvánvaló: ha a pool adatterülete betelik, az összes vékony kötet írása meghiúsulhat, fájlrendszer-hibákat és alkalmazásleállást okozva.

A gyakorlatban a túlallokáció mértékét és a pool kihasználtságát folyamatosan figyelni kell. A vezérlők tipikusan két mérőszámot jelentenek: az adatterület és a metaadat-terület kihasználtságát százalékban. A rendszergazdának figyelmeztetési szinteket kell beállítania, és tartalék kapacitást kell hagynia a váratlan COW-növekedésre.

18.1.9 3.3 A thin pool és a fájlrendszer együttműködése

A thin pool önmagában nem tudja megállapítani, hogy egy blokk szabad-e; ehhez a fájlrendszernek kell `TRIM`/`UNMAP` üzenetet küldenie. Linux alatt az `fstrim` parancs vagy a csatolási opciók (`discard`) biztosítják ezt. Enélkül a törölt fájlok blokkjai a thin poolban továbbra is foglaltnak látszanak, így a pool használata monoton növekedhet. A pillanatképek esetén a `discard`-kezelés bonyolultabb: ha egy blokk egy pillanatképben is szerepel, a felszabadítás nem lehetséges a pillanatkép törléséig.

18.1.10 4. Tervezési minták és tipikus hibák

18.1.11 4.1 Hosszú távú pillanatképek

A COW és ROW pillanatképek hosszú távú megtartása több problémát okoz. Egyrészt a változó blokkok halmozódása növeli a tárhelyfoglalást, másrészt a pillanatkép-lánc mélyülése ronthatja az olvasási teljesítményt, mert a blokkok feloldása több indirekción keresztül történik. A tervezésnél mindig meg kell határozni a pillanatképek élettartamát és retenciós szabályait.

18.1.12 4.2 A thin pool kimerülése

A leggyakoribb hiba a túlallokációval kapcsolatban az, hogy a rendszergazda nem figyeli a pool kihasználtságát, vagy nem állít be automatikus bővítést. Amikor a pool adatterülete megtelik, az írások I/O hibával térnek vissza. A felette lévő fájlrendszer hibás állapotba kerülhet, és az alkalmazások adatvesztést szenvedhetnek. A helyes megoldás:

- rendszeres monitorozás (`Data%`, `Meta%`),
- figyelmeztetési küszöbök (például 70% és 85%),
- automatikus bővítés vagy elegendő tartalék fizikai terület,
- az `fstrim` rendszeres futtatása a felszabadult blokkok visszaadására.

18.1.13 4.3 Klónozási lánc és törlési félreértések

Amikor egy klón szülő snapshotját töröljük, a rendszer nem feltétlenül szabadít fel tárhelyet. A klón továbbra is hivatkozik a korábban közös blokkokra, így azok referenciaszámlálója csak csökken, de nem éri el a nullát. A gyakorlatban ez azt jelenti, hogy a tárhely-felszabadulás egy későbbi törlésig elhúzódhat. A rendszergazdának tisztában kell lennie a függőségi gráffal, és a törlési műveletek után ellenőriznie kell a pool kihasználtságát.

18.1.14 4.4 Alkalmazáskonzisztencia

A blokkszintű pillanatkép nem helyettesíti az alkalmazás-szintű konzisztenciát. Ha egy adatbázis éppen tranzakció közben van, amikor a snapshot készül, a pillanatkép félkész állapotot rögzíthet. A megoldás az alkalmazás quiesce-elése (például adatbázis `FLUSH TABLES WITH READ LOCK` vagy hasonló) a pillanatkép előtt, vagy az alkalmazás által támogatott konzisztens pillanatkép-készítés. Ezt a szempontot a labor tervezésekor is figyelembe kell venni.

18.1.15 5. Labor: snapshot- és klónozási ütemterv készítése

A labor célja, hogy a tanuló reprodukálható módon megmérje a pillanatkép-készítés, a COW-írás és a klónozás időbeli és helyfoglalási jellemzőit, majd ezekből ütemtervet készítsen. A labor **emulált környezetben** fut, a hoszt valódi lemezét semmilyen módon nem módosítjuk. Az alábbi lépések egy lehetséges kiinduló vázat adnak; a pontos parancsszintaxis az aktuális disztribúció és LVM-verzió függvénye, ezért a man oldalakat és a telepített verziót mindig ellenőrizni kell.

18.1.16 5.1 Kiinduló állapot és reprodukciós csomag

A labor konfigurációja:

Elem	Érték
Emulátor	QEMU/KVM
CPU	2 vCPU (host CPU modellje: rögzítendő)
Memória	2 GiB
Rendszerlemez	8 GiB virtio-blk, eldobható
Operációs rendszer	Linux, a kernelverziót a naplóban rögzíteni
Kötetkezelő	LVM2 (telepített verzió rögzítendő)
Fájlrendszer	ext4

A virtuális gép leállítása után a lemezkép törölhető vagy visszaállítható. A reprodukciós csomagnak tartalmaznia kell a QEMU indítóparancsot, a kernelverziót, a disztribúció nevét, az LVM és a fájlrendszer-eszközök verzióját, valamint a mérési naplót.

18.1.17 5.2 Ütemterv és mérések

A feladat egy ütemterv végrehajtása, amelyben meghatározott időpontokban snapshotot, írást, klónozást és törlést végzünk. Minden lépésnél rögzítjük az időbélyeget, a thin pool adat- és metaadat-kihasználtságát, valamint a művelet időtartamát. A **mért** és **várt** értékeket külön oszlopban kell feltüntetni.

Az alábbi pszeudokód a lépések vázát mutatja:

```
1. Hozz létre LVM volume groupot a 8 GiB lemezen.
2. Hozz létre thin poolt 6 GiB méretben (pool0).
3. Hozz létre 10 GiB logikai méretű vékony kötetet (thinroot).
   A túlallokációs arány kezdetben 10/6 = 1,67.
4. Formázd meg ext4-gyel, csatold /mnt alá.
5. Írj 2 GiB véletlen adatot a /mnt/data fájlba.
6. Készíts pillanatképet snap0 néven.
7. Módosítsd az élő kötetet: írd felül az adat első 1 GiB-ját,
   majd törölj 500 MiB-ot egy másik fájlból.
8. Készíts írható klónt clone0 néven a snap0 pillanatképből.
9. Módosítsd a klónt: írj 200 MiB új adatot.
10. Töröld a snap0 pillanatképet, és figyeld a pool felszabadulását.
11. Állítsd le a gépet, készíts mérési naplót.
```

A mérési napló javasolt oszlopai:

Időpont	Művelet	Logikai méret	Pool adatfoglalás	Pool metaadat	Műveleti idő (mért)	Várható idő	Várható helyfoglalás	Megjegyzés
t0	thin volume létrehozás	10 GiB	0 MiB	…	…	…	0 MiB	nincs adatírás
t1	2 GiB írás	10 GiB	~2 GiB + meta	…	…	…	2 GiB + overhead	…
t2	snapshot létrehozás	10 GiB	~2 GiB + meta	…	…	<1 s	nincs új adat	metaadat-művelet
t3	COW-írás 1 GiB felülírás	10 GiB	növekszik	…	…	…	~1 GiB régi blokk	COW-hatás
t4	klónozás	10 GiB + 10 GiB	növekszik	…	…	<1 s	nincs azonnali adatmásolás	referenciaszámlálás
t5	klónírás 200 MiB	…	növekszik	…	…	…	~200 MiB	klón saját blokkjai
t6	snap0 törlés	…	kis mértékben csökken	…	…	…	nem feltétlenül csökken	a klón függősége miatt

A várt értékeket a művelet típusából és a felülírt adatmennyiségből származtathatjuk. A mért értékeket a `lvs --units b --nosuffix` vagy a thin pool állapotát megjelenítő parancs kimenetéből olvassuk ki. A műveleti idő méréséhez `date +%s.%N` időbélyegeket használunk a művelet előtt és után.

18.1.18 5.3 Példa mérési szakasz

Tegyük fel, hogy a kiinduló thin pool 6 GiB. A 2 GiB véletlen adat megírása után az adatterület várhatóan körülbelül 2 GiB plusz a fájlrendszer metaadatai, a tényleges érték azonban a blokkmérettől és az ext4 felépítésétől függ, ezért eltérés lehet. A pillanatkép létrehozása nem növeli az adatterületet, csak a metaadat-területet. Ezután az 1 GiB felülírás COW-hatása miatt a régi 1 GiB blokkok a pillanatképhez kerülnek, így az adatterület hozzávetőleg 1 GiB-vel nő. A klónozás önmagában minimális adatfoglalást okoz, de a klón írásakor a 200 MiB új adat a klón saját területére kerül, ami további növekedést jelent.

A mért és a várt értékeket a naplóban egymás mellett kell rögzíteni. A labor értékelésekor azt vizsgáljuk, hogy a tanuló felismeri-e a COW-hatást, a thin pool túlallokációjának kockázatát, és hogy a törlés után a felszabadulás nem azonnali.

18.1.19 5.4 Visszaállítás

A labor befejezése után a virtuális gépet leállítjuk, a lemezképet eldobjuk vagy visszaállítjuk. A hoszt operációs rendszer és a valódi tárolók nem módosulnak. A reprodukciós csomagban a kiindulási állapot (például a QEMU parancs és a lemezkép készítésének módja) is szerepel.

18.1.20 6. Ellenőrző kérdések és összefoglalás

18.1.21 Önellenőrző kérdések

1. Mi a különbség a COW és a ROW pillanatkép között az első írás költségében?
2. Miért lehet veszélyes, ha egy thin poolban a logikai méretek összege jelentősen meghaladja a fizikai kapacitást?
3. Mi történik a tárhelyfoglalással, ha egy klón szülő snapshotját töröljük, de a klón továbbra is él?

18.1.22 Válaszok röviden

1. A COW az első írás előtt a régi blokkot egy pillanatkép-területre másolja, így az első írás olvasást és másolást is igényel. A ROW az új adatot egy másik blokkba írja, a pillanatkép pedig a régi blokkra mutat, ezért az első írásnál nincs másolási lépés.
2. A túlallokáció rugalmasságot ad, de ha a pool adatterülete betelik, az összes vékony kötet írása meghiúsulhat, ami fájlrendszer-hibákhoz és adatvesztéshez vezethet. A pool kihasználtságát folyamatosan figyelni kell, és tartalékot vagy automatikus bővítést kell biztosítani.
3. A szülő snapshot törlése csökkenti a közös blokkok referenciaszámlálóját, de a klón által hivatkozott blokkok nem szabadulnak fel, amíg a referenciaszámláló el nem éri a nullát. A tárhelyfoglalás tehát nem feltétlenül csökken azonnal.

18.1.23 Összefoglalás

A pillanatkép és a klón a modern tárolás alapvető eszközei. A copy-on-write pillanatkép létrehozása O(1) időt igényel, de az írási út később drágább lehet, mert a régi blokkokat meg kell őrizni. A redirect-on-write ezt a költséget az írásból a felszabadításba helyezi át. A klónozás referenciaszámlálással teszi lehetővé, hogy több példány ossza meg ugyanazokat a blokkokat, és csak a valódi eltérések foglaljanak területet. A thin provisioning a logikai és fizikai kapacitás szétválasztásával rugalmasságot ad, de a túlallokáció kimerülési kockázatot hordoz. A gyakorlatban a pillanatképek élettartamát, a pool kihasználtságát és az alkalmazáskonzisztenciát egyaránt tervezni kell. A laborban elvégzett COW-mérés és ütemterv segít felismerni ezeket a hatásokat, és megalapozza a következő egységben tárgyalt titkosítás és kulcskezelés megértését.

18.2 Titkosítás: LUKS, BitLocker és kulcskezelés

A tároló titkosítása az adatvédelem egyik alappillére, de önmagában nem elegendő: a kulcsok életciklusának tervezése és a helyreállítási eljárások ugyanolyan fontosak, mint a titkosítási algoritmus kiválasztása. Ez a fejezet a teljes lemez titkosítás (FDE – Full Disk Encryption) két elterjedt megvalósítását, a Linux Unified Key Setup (LUKS) és a Microsoft BitLocker megoldását mutatja be, különös tekintettel a kulcskezelésre, a TPM-integrációra és a helyreállítási

stratégiákra. A cél nem csupán az eszközök parancsszintű ismerete, hanem egy olyan tervezési szemlélet kialakítása, amely a kulcsok teljes életciklusát – generálástól a megsemmisítésig – lefedi.

A fejezetben szereplő parancsok és konfigurációs példák kizárólag emulált környezetben vagy eldobható virtuális gépen futtathatók; valódi lemez, bootloader vagy gyártói eszköz módosítása tilos. A helyőrző eszközök (például `/dev/sdX`, `/dev/mapper/luks-lab`) mindenütt szándékosan nem valós eszköznevek.

18.2.1 1. Tervezési alapelvek: blokkszintű és fájlszintű titkosítás

A tároló titkosítása két fő szinten valósítható meg. A **fájlszintű titkosítás** az egyes fájlokat vagy könyvtárakat külön-külön védi (például eCryptfs, EncFS, vagy a Windows EFS). Előnye a finomabb hozzáférés-vezérlés, hátránya viszont, hogy a fájlnevek, könyvtárstruktúra és metaadatok – hacsak a megoldás ezeket is nem titkosítja – megfigyelhetők maradnak, és a kulcskezelés gyakran alkalmazásonként eltérő. A **blokkszintű titkosítás** a teljes blokkeszközt vagy partíciót egyetlen rétegként védi (LUKS, BitLocker, dm-crypt, VeraCrypt). Itt a fájlrendszer a titkosított blokkeszköz fölött helyezkedik el, így minden adat – beleértve a metaadatokat is – titkosított, amikor az eszköz leáll. A blokkszintű megoldások az operációs rendszer alatt, a kernelben vagy a meghajtó rétegben működnek, ezért a felhasználói folyamatok számára átlátszóak.

A teljes lemez titkosítás fenyegetési modellje elsősorban az **eszköz fizikai elvesztése vagy ellopása** ellen véd. Ha a támadó kikapcsolt állapotban hozzáfér a lemezhez, a titkosítás nélkül az összes adat kiolvasható. A titkosítás ezt a támadási felületet zárja le, feltéve, hogy a kulcs nem elérhető a lemezen nyílt formában. Fontos azonban hangsúlyozni: a titkosítás **nem** véd a futó rendszer elleni támadásoktól (például jogosulatlan bejelentkezés, memóriából történő kulcskinyerés, DMA-támadás), és a hidegindításos (cold boot) támadás is kompromittálhatja a kulcsot, ha a RAM tartalma megmarad.

A modern blokkszintű titkosítás tipikusan **AES-XTS** módot használ. Az XTS (XEX-based Tweaked CodeBook mode with CipherText Stealing) egy „tweakable” blokkrejtjel mód, amely minden blokkot a blokk pozíciójától függően titkosít, így az azonos nyílt szövegű blokkok különböző rejtett szöveget adnak anélkül, hogy IV-t kellene tárolni. A kulcshossz általában 256 bit (AES-256-XTS), amely két 256 bites kulcsot használ (egyik a titkosításhoz, másik a tweakhez). A LUKS és a BitLocker egyaránt támogatja ezt a módot. A kulcsderivációhoz a LUKS2 Argon2id vagy PBKDF2 algoritmust használ, a BitLocker pedig a felhasználói jelszóból származtatja a kulcsvédő kulcsot a Windows saját KDF-jével.

A kulcshierarchia alapelve, hogy a **mesterkulcs** (Volume Master Key, VMK vagy Data Encryption Key, DEK) soha ne kerüljön tárolásra nyílt formában a lemezen. Ehelyett a mesterkulcsot egy vagy több **kulcsvédő** (key slot, key protector) titkosítja, amelyek jelszóból, PIN-ből, kulcsfájlból, TPM-ből vagy ezek kombinációjából származnak. Ezáltal a felhasználói hitelesítő adat megváltoztatása nem igényli a teljes lemez újratitkosítását: csak az adott kulcsvédőben tárolt titkosított mesterkulcs frissül.

18.2.2 2. LUKS: Linux Unified Key Setup

A LUKS a Linux rendszerek de facto szabványa a blokkszintű titkosításra. A **LUKS1** formátum (2004) egy 2 MiB méretű fejlécet (header) helyez el a titkosított eszköz elején. A fejléc tartalmazza a titkosítási paramétereket, a mesterkulcs kulcsslotokban tárolt titkosított másolatait, valamint a kulcsderivációs paramétereket (PBKDF2 iterációszám, só). A **LUKS2** (2017) flexibilis, JSON-alapú metaadatformátumot vezetett be, támogatja a többféle KDF-et (beleértve az

Argon2i/Argon2id-t), a nagyobb fejlécet, a rejtett kulcsslotokat és a dinamikus kulcsslot-kezelést. A LUKS2 fejléce alapértelmezés szerint 16 MiB, de ez konfigurálható.

A LUKS működésének magja a következő: a lemez első blokkjai a fejlécet tartalmazzák. A felhasználói hitelesítő adatból (jelszó vagy kulcsfájl) a KDF egy **kulcsslot-kulcsot** származtat, amely visszafejti az adott kulcsslotban tárolt **mesterkulcsot**. A mesterkulcs ezután a dm-crypt rétegben közvetlenül titkosítja és visszafejti a lemezblokkokat. A mesterkulcs a rendszer futása közben a kernel memóriájában él, és csak a leállításig marad érvényben.

A LUKS kezelésére a `cryptsetup` eszköz szolgál. Az alábbi példák szemléltető jellegűek, és kizárólag emulált környezetben futtathatók:

```
# LUKS2 kötet létrehozása (helyőrző eszköz: /dev/sdX)
cryptsetup luksFormat --type luks2 --cipher aes-xts-plain64 \
  --key-size 512 --hash sha512 --pbkdf argon2id /dev/sdX

# A kötet megnyitása (a /dev/mapper/luks-lab eszközt hozza létre)
cryptsetup open /dev/sdX luks-lab

# Fájlrendszer létrehozása a titkosított eszközön
mkfs.ext4 /dev/mapper/luks-lab

# A kötet lezárása
cryptsetup close luks-lab
```

A `luksFormat` parancs a `--key-size 512` paraméterrel AES-256-XTS módot állít be, mivel az XTS két 256 bites kulcsot igényel. A `--pbkdf argon2id` a LUKS2 ajánlott kulcsderivációs algoritmusa, amely memória-nehéz, így ellenáll a GPU-alapú brute force támadásoknak. LUKS1 esetén csak PBKDF2 érhető el, amely szintén biztonságos, de nagyobb iterációszámot igényel.

A LUKS kulcskezelése a kulcsslotokon keresztül történik. Egy LUKS2 kötet alapértelmezés szerint 8 kulcsslotot tartalmaz, de ez módosítható. Mindegyik slot egy-egy független kulcsvédőt jelent, így egyszerre több jelszó vagy kulcsfájl is aktív lehet ugyanazon mesterkulcshoz:

```
# Új jelszó hozzáadása a 2. slothoz
cryptsetup luksAddKey /dev/sdX --key-slot 2

# Kulcsfájl hozzáadása
cryptsetup luksAddKey /dev/sdX /path/to/keyfile.bin

# Egy adott slot eltávolítása (pl. a 2. slot)
cryptsetup luksKillSlot /dev/sdX 2

# A fejléc tartalmának megjelenítése
cryptsetup luksDump /dev/sdX

# A fejléc biztonsági mentése (kritikus!)
cryptsetup luksHeaderBackup /dev/sdX --header-backup-file luks-header-backup.img

# A fejléc visszaállítása
cryptsetup luksHeaderRestore /dev/sdX --header-backup-file luks-header-backup.img
```

A **fejléc biztonsági mentése** az egyik legfontosabb művelet. Ha a LUKS fejléc megsérül – például véletlen felülírás, lemezhiba vagy rosszindulatú törlés miatt –, a mesterkulcs elveszik, és az összes adat helyrehozhatatlanul olvashatatlanná válik. A `luksHeaderBackup` parancs a fejlécet egy külön fájlba menti, amelyet biztonságos helyen (például offline tároló, HSM vagy titkosított pendrive) kell őrizni. Fontos: a fejléc mentése **nem** helyettesíti a mesterkulcs mentését, de a mesterkulcs titkosított másolatait tartalmazza a kulcsslotokban, így a jelszó ismeretében a mesterkulcs visszanyerhető.

A LUKS jelszó rotációja egyszerű, mert csak a kulcsslot frissül: a `luksChangeKey` parancs lecseréli egy adott slot jelszavát, míg a `luksAddKey` új slotot hoz létre, a `luksKillSlot` pedig törli a régit. A **mesterkulcs rotációja** azonban a teljes eszköz újratitkosítását igényli, mivel a mesterkulcs magát az adatot védi. Erre a LUKS2 esetén a `cryptsetup reencrypt` parancs szolgál, amely lehetővé teszi az online újratitkosítást:

```
# Mesterkulcs rotációja online (a kötetnek nyitva kell lennie)
cryptsetup reencrypt /dev/sdX --reduce-device-size 32M
```

Ez a művelet időigényes, és a teljesítményre is hatással van, ezért ütemezetten, terhelésmentes időszakban kell végrehajtani. A rotáció előtt mindig készítsünk fejléc-mentést és teljes biztonsági mentést.

18.2.3 3. BitLocker: Microsoft teljes lemez titkosítása

A BitLocker a Windows operációs rendszer beépített teljes lemez titkosítási megoldása. A BitLocker a rendszerkötetet (általában C:) és az adatköteteket is titkosíthatja. A BitLocker architektúrája hasonló kulcshierarchiát alkalmaz: a **FVEK** (Full Volume Encryption Key) a tényleges lemeztitkosító kulcs, amelyet a **VMK** (Volume Master Key) titkosít. A VMK-t pedig egy vagy több **kulcsvédő** (key protector) védi, például TPM, PIN, startup key (USB-n tárolt kulcs), jelszó, recovery password vagy külső kulcs. A FVEK és a VMK soha nem tárolódik nyílt formában a lemezen; a VMK a lemezen a kulcsvédőkben titkosítva található.

A BitLocker működési módjai:

- **TPM-only**: A VMK-t a TPM seal-eli a rendszerindítási komponensek PCR-értékeihez. Ha a boot folyamat nem változott, a TPM automatikusan kiadja a VMK-t, így a rendszer felhasználói beavatkozás nélkül indul. Ez véd a lemez másik gépre helyezése ellen, de ha a támadó a bekapcsolt gépet elviszi, a TPM kiadhatja a kulcsot, mert a boot környezet változatlan. Ezért ez a mód önmagában nem véd a fizikai támadás ellen, ha a gép fut.
- **TPM+PIN**: A TPM mellett a felhasználónak egy PIN-kódot is meg kell adnia a boot során. A PIN a TPM-be kerül, és a TPM csak akkor adja ki a VMK-t, ha mind a PCR-értékek, mind a PIN helyes. Ez a mód véd a lemez ellopása és a cold boot támadás ellen is, mert a PIN nem tárolódik a lemezen, és a TPM brute force elleni védelmet nyújt.
- **TPM+startup key**: A PIN helyett egy USB-n tárolt kulcs is használható; a kulcsot a TPM-mel kombinálva kell megadni. Ez hasonló védelmet nyújt, mint a PIN, de fizikai kulcshordozót igényel.
- **Recovery password**: Minden BitLocker-kötethez tartozik egy 48 számjegyű helyreállítási kulcs, amelyet ki lehet nyomtatni, elmenteni fájlba, vagy tárolni az Active Directoryban / Azure AD-ben. Ha a TPM nem adja ki a VMK-t (például hardverváltozás, BIOS-frissítés, vagy a boot lánc módosul), a felhasználó ezzel a kulccsal manuálisan feloldhatja a kötetet.

A BitLocker kezelése a `manage-bde` parancssori eszközzel vagy a PowerShell `BitLocker` moduljával történik. Az alábbi példák szemléltetők, és csak emulált környezetben értelmezhetők:

```
# BitLocker engedélyezése a C: meghajtón TPM-only módban
manage-bde -on C: -tpm

# BitLocker engedélyezése TPM+PIN móddal
manage-bde -on C: -tpm -pin

# Helyreállítási kulcs megjelenítése
manage-bde -protectors -get C:

# Helyreállítási kulcs hozzáadása
```

```
manage-bde -protectors -add C: -recoverypassword

# Új jelszó (PIN) hozzáadása
manage-bde -protectors -add C: -pin

# A kötet feloldása helyreállítási kulccsal
manage-bde -unlock C: -recoverypassword 123456-789012-...
```

A BitLocker a lemez elején egy **metadata** régiót hoz létre, amely a kulcsvédőket és a kötet címkéjét tartalmazza. A titkosított adatterület a lemez többi részét foglalja el. A BitLocker To Go a cserélhető meghajtók (USB) titkosítására szolgál, ahol jelszó vagy smart card alapú kulcsvédő is használható.

A BitLocker kulcsrotációja a VMK szintjén lehetséges, de a gyakorlatban ritkán szükséges. A felhasználói hitelesítő adatok (PIN, jelszó, startup key) cseréje egyszerű, és nem igényli az adatok újratitkosítását. A VMK rotációja a `manage-bde -protectors -delete` és `-add` parancsokkal, illetve a kötet újratitkosításával érhető el. A BitLocker támogatja a **used space only** titkosítást, amely csak a ténylegesen használt blokkokat titkosítja, így az új lemezek titkosítása gyorsabb.

18.2.4 4. TPM és a rendszerintegritás

A Trusted Platform Module (TPM) egy hardveres biztonsági modul, amely kulcsokat generálhat, tárolhat és használhat anélkül, hogy a privát kulcsok valaha is elhagynák a chipet. A TPM specifikáció (TPM 2.0) szabványosított, és a modern PC-k, laptopok és számos beágyazott eszköz tartalmazzák. A TPM egyik legfontosabb funkciója a **platform konfigurációs regiszterek (PCR-ek)** kezelése. A PCR-ek a boot folyamat során mért komponensek (BIOS, bootloader, kernel, initrd) hash-értékeit tárolják, és csak kiterjesztéssel (extend) módosíthatók, nem írhatók közvetlenül. Ez lehetővé teszi a **measured boot**-ot: ha a boot lánc bármely eleme megváltozik, a PCR-értékek eltérnek a várt értékektől.

A TPM **seal** művelete egy kulcsot (például a VMK-t) úgy titkosít, hogy csak akkor adható vissza (unseal), ha a megadott PCR-ek a rögzített értékeken állnak. Így a BitLocker TPM-only módban a VMK-t a TPM seal-eli a boot komponensek PCR-jeihez. Ha a támadó módosítja a boot loadert vagy a kernelt, a PCR-értékek megváltoznak, és a TPM nem adja ki a VMK-t – a rendszer nem indul el, vagy helyreállítási kulcsot kér.

A TPM-et a LUKS is használhatja, bár ez nem alapértelmezett. A `systemd-cryptenroll` és a `clevis` keretrendszer lehetővé teszi, hogy egy LUKS kulcsslotot a TPM-hez kössünk. Ebben az esetben a TPM-ben tárolt kulcs seal-eli a LUKS mesterkulcs egy másolatát, és a boot során a PCR-ek ellenőrzése után automatikusan feloldja a kötetet. Ez hasonló élményt nyújt, mint a BitLocker TPM-only, de a LUKS esetén a TPM-integráció disztribúciófüggő, és a konfiguráció részleteit ellenőrizni kell.

A TPM nem helyettesíti a felhasználói hitelesítő adatot, hanem kiegészíti azt. A TPM-only mód kényelmes, de fizikai támadás esetén, ha a gép bekapcsolva marad, a TPM kiadhatja a kulcsot, mert a PCR-ek változatlanok. A TPM+PIN kombináció viszont erős védelmet nyújt, mert a PIN a TPM-en belül kerül ellenőrzésre, és a TPM brute force elleni védelemmel rendelkezik.

18.2.5 5. Kulcskezelés és életciklus

A kulcsok életciklusa több szakaszból áll, és mindegyik szakaszban más-más biztonsági követelmények érvényesek. A tervezés során a következő lépéseket kell végiggondolni:

Generálás. A kulcsokat megbízható entrópiaforrásból kell generálni. A LUKS mesterkulcsát a `cryptsetup` a kernel véletlenszám-generátorából (CSPRNG) veszi. A BitLocker VMK-ját a Windows hasonlóan generálja. A felhasználói jelszavak esetén a hosszúság és az entrópia kritikus: egy rövid vagy gyakori jelszó brute force-szal feltörhető. A jelszó entrópiája legalább 70-80 bit legyen, ami gyakorlatilag egy 4-5 véletlenszerű szóból álló passphrase-nek vagy egy 12+ karakteres véletlen jelszónak felel meg. A kulcsfájlok generálásakor elegendő méretű (pl. 32 bájt) véletlen adatot kell használni.

Tárolás. A kulcsokat titkosítva vagy hardveres védelem alatt kell tárolni. A LUKS esetén a mesterkulcs a fejlécben, a kulcsslotokban titkosítva van. A jelszó vagy kulcsfájl a felhasználónál van, de a kulcsfájl tárolása is biztonságos helyet igényel (pl. titkosított pendrive, HSM, TPM). A BitLocker kulcsvédői a lemez metaadat-régiójában vannak, de a TPM-ben tárolt kulcsok hardveres védelem alatt állnak. A **key escrow** (kulcsletét) olyan szervezeti megoldás, ahol a kulcsok másolatát egy megbízható harmadik fél vagy belső rendszer tárolja, lehetővé téve a helyreállítást akkor is, ha a felhasználó elveszti a kulcsát. Az Active Directory és az Azure AD beépített BitLocker recovery key escrow funkcióval rendelkezik.

Használat. A kulcsot csak a lehető legrövidebb ideig és a lehető legszűkebb körben szabad használni. A LUKS mesterkulcs a kernel memóriájában van, és a felhasználói folyamatok nem férnek hozzá közvetlenül. A BitLocker VMK hasonlóan a rendszer futása közben a memóriában él. A kulcsot soha nem szabad naplóba írni, swapra vagy core dumpba menteni. A memóriában lévő kulcsok kinyerése ellen a memória titkosítása (pl. AMD SME, Intel TME) és a secure boot véd.

Rotáció. A kulcsokat időszakosan cserélni kell, hogy csökkentsük a kompromittálódás kockázatát. A rotáció két szinten történhet: a **felhasználói hitelesítő adat** cseréje (jelszó, PIN, kulcsfájl) egyszerű, és nem igényli az adatok újratitkosítását. A **mesterkulcs** rotációja azonban a teljes kötet újratitkosítását vonja maga után, ami idő- és erőforrás-igényes. A rotációs politikának meg kell határoznia, hogy milyen gyakran cseréljük a mesterkulcsot, és milyen események (pl. kulcs kompromittálódásának gyanúja, személyi változás, szabályozási követelmény) váltják ki a rotációt. A LUKS esetén a `cryptsetup reencrypt` parancs online rotációt tesz lehetővé, de a művelet alatt a rendszer terhelése magas lehet. A BitLocker VMK rotációja ritkább, de a felhasználói PIN-ek és jelszavak cseréje itt is egyszerű.

Visszavonás. Ha egy kulcs kompromittálódott, azonnal vissza kell vonni, és új kulcsot kell generálni. A LUKS-ban egy kulcsslot törlésével (`luksKillSlot`) az adott jelszó vagy kulcsfájl érvénytelenné válik. A BitLocker-ben a kulcsvédő törlésével (`manage-bde -protectors -delete`) ugyanez érhető el. Fontos, hogy a visszavonás után a régi kulccsal többé ne lehessen hozzáférni az adatokhoz; ehhez a mesterkulcs rotációja is szükséges lehet, ha a kulcs kompromittálódott, mert a régi mesterkulcs ismert lehet.

Megsemmisítés. A kulcsok megsemmisítése a kulcs minden másolatának biztonságos törlését jelenti. Az SSD-k és a flash tárolók esetén a tényleges törlés nehéz, mert a wear leveling és a garbage collection miatt a régi adatok fizikailag megmaradhatnak. A kulcsok megsemmisítésére a kriptográfiai törlés (crypto erase) a legjobb módszer: ha a mesterkulcsot töröljük, a titkosított adatok olvashatatlanná válnak, még akkor is, ha a lemez blokkjai fizikailag megmaradnak. A LUKS esetén a fejléc törlése (`cryptsetup erase` vagy a fejléc felülírása) kriptográfiai törlést eredményez. A BitLocker esetén a kötet formázása vagy a kulcsvédők törlése hasonló hatású.

A kulcskezelés során **audit naplót** kell vezetni: ki, mikor, milyen kulcsot generált, cserélt, vont vissza vagy semmisített meg. Ez a napló elengedhetetlen a megfelelőségi ellenőrzésekhez és az incidenskezeléshez.

18.2.6 6. Helyreállítási stratégiák

A titkosítás bevezetésekor a legnagyobb kockázat nem a támadó, hanem a **kulcs elvesztése**. Ha a felhasználó elfelejti a jelszavát, a kulcsfájl megsérül, vagy a TPM nem adja ki a kulcsot, az adatok menthetetlenül elveszhetnek. Ezért a helyreállítási eljárásokat a titkosítás bevezetése **előtt** kell megtervezni és tesztelni.

LUKS helyreállítás. A LUKS fejléc biztonsági mentése (`luksHeaderBackup`) kritikus, mert a fejléc sérülése esetén a mesterkulcs visszanyerhető a mentett fejlécből, feltéve, hogy a jelszó ismert. A fejléc-mentést biztonságos, elkülönített helyen kell tárolni – lehetőleg több példányban, titkosított adathordozón. Ha a jelszó elveszik, de van másik aktív kulcsslot (például egy másik jelszó vagy kulcsfájl), akkor a rendszer továbbra is feloldható, és az elveszett jelszó slotja törölhető, majd új jelszó adható hozzá. Ha az összes kulcsslot elveszik, a mesterkulcs mentése nélkül az adatok elvesznek. Ezért fontos, hogy legalább két független kulcsvédő legyen aktív, és az egyik legyen biztonságos helyen tárolt kulcsfájl vagy recovery key.

BitLocker helyreállítás. Minden BitLocker-kötethez tartozik egy 48 számjegyű **recovery password**. Ezt a kulcsot a titkosítás engedélyezésekor automatikusan generálja a rendszer, és ki lehet menteni fájlba, nyomtatóra, vagy tárolni az Active Directoryban / Azure AD-ben. Ha a TPM nem adja ki a VMK-t (például hardverváltozás vagy BIOS-frissítés miatt), a felhasználó a recovery password segítségével feloldhatja a kötetet. A recovery password-öt biztonságos helyen kell tárolni, és rendszeresen ellenőrizni kell a hozzáférhetőségét. A BitLocker To Go esetén a jelszó használata gyakoribb, és a recovery key is hasonló.

Kulcsrotáció utáni helyreállítás. Ha a mesterkulcsot rotáljuk, a régi kulcs többé nem használható. A rotáció előtt mindig készítsünk friss fejléc-mentést (LUKS) vagy recovery key exportot (BitLocker), és győződjünk meg arról, hogy a helyreállítási eljárás működik. A rotáció után a régi kulcsot biztonságosan meg kell semmisíteni, és a helyreállítási kulcsokat frissíteni kell.

Dokumentáció. A helyreállítási eljárást írásban kell rögzíteni, beleértve a kulcsok tárolási helyét, a felelős személyeket, az eszkalációs lépéseket és a tesztelési ütemtervet. Az eljárást rendszeresen, de legalább évente egyszer tesztelni kell egy nem éles környezetben, hogy a kulcsok valóban működnek.

18.2.7 7. Gyakori hibák és elkerülésük

1. **Fejléc-mentés hiánya.** A LUKS fejléc sérülése adatvesztéshez vezet, ha nincs mentés. Mindig készítsünk fejléc-mentést, és tároljuk biztonságos helyen.
2. **Gyenge jelszó.** A rövid vagy szótári szavakból álló jelszavak brute force-szal feltörhetők. Használjunk passphrase-t vagy erős jelszót, és állítsunk be megfelelő KDF paramétereket (Argon2id LUKS2 esetén).
3. **TPM-only mód fizikai védelem nélkül.** Ha a laptop bekapcsolva marad, a TPM-only BitLocker nem véd a fizikai támadás ellen. Használjunk TPM+PIN-t vagy TPM+startup key-t, ha a fizikai védelem nem garantálható.
4. **Kulcsrotáció elmulasztása.** A kulcsokat rendszeresen cserélni kell, különösen akkor, ha a kulcs kompromittálódhatott. A rotációt ütemezetten kell végrehajtani, és dokumentálni kell.
5. **Kulcsok nem biztonságos tárolása.** A kulcsfájlt vagy recovery key-t ne tároljuk a titkosított lemezen, mert azzal körkörös hivatkozás jön létre. Használjunk elkülönített, titkosított adathordozót vagy HSM-et.
6. **Nem tesztelt helyreállítás.** A helyreállítási eljárást rendszeresen tesztelni kell, különben a kulcs elvesztésekor kiderülhet, hogy a mentés hibás vagy hiányos.

7. **Régi algoritmusok használata.** A CBC mód és a SHA1 alapú KDF mára elavult. LUKS2 esetén AES-XTS és Argon2id, BitLocker esetén az alapértelmezett XTS-AES használata javasolt.

18.2.8 8. Labor: titkosítási és kulcsrotációs terv készítése

A labor célja, hogy az olvasó egy konkrét, reprodukálható tervet készítsen egy tároló titkosítására és a kulcsok életciklusának kezelésére. A feladat nem éles rendszeren, hanem emulált környezetben (QEMU virtuális gép) futtatható, ahol a konfiguráció és a mérési napló rögzíthető. A labor eredménye egy dokumentum, amely tartalmazza a titkosítási architektúrát, a kulcsrotációs ütemtervet és a helyreállítási eljárást, valamint a ténylegesen elvégzett műveletek naplóját.

Környezet. Hozzon létre egy QEMU virtuális gépet, amelyhez egy legalább 4 GiB méretű virtuális lemez tartozik. A virtuális gépben telepítsen egy minimális Linux disztribúciót (például Debian netinstall) a `cryptsetup` csomaggal. A gyakorlat során használt parancsok csak a virtuális lemezen futnak, a hoszt rendszerét nem érintik.

Lépések.

1. Hozza létre a virtuális gépet, és indítsa el. Jelentkezzen be, és ellenőrizze a rendelkezésre álló lemezeket (`lsblk`). Az üres virtuális lemez neve legyen `/dev/vdb` (helyőrző).

2. Készítsen LUKS2 titkosított kötetet a `/dev/vdb` eszközön:

   ```
   cryptsetup luksFormat --type luks2 --cipher aes-xts-plain64 \
     --key-size 512 --hash sha512 --pbkdf argon2id /dev/vdb
   ```

   A jelszó legyen egy erős passphrase (pl. `correct-horse-battery-staple`), amelyet a labor jegyzőkönyvben rögzítsen (de éles környezetben soha ne tároljon jelszót dokumentumban).

3. Mentse a LUKS fejlécet:

   ```
   cryptsetup luksHeaderBackup /dev/vdb --header-backup-file /root/luks-header-backup.img
   ```

   Ellenőrizze a fájl méretét (LUKS2 esetén tipikusan 16 MiB).

4. Nyissa meg a kötetet, és hozzon létre rajta ext4 fájlrendszert:

   ```
   cryptsetup open /dev/vdb luks-lab
   mkfs.ext4 /dev/mapper/luks-lab
   mount /dev/mapper/luks-lab /mnt
   echo "titkosított teszt" > /mnt/test.txt
   umount /mnt
   cryptsetup close luks-lab
   ```

5. Szimulálja a kulcsrotációt. Először adjon hozzá egy második jelszót egy új kulcsslotban:

   ```
   cryptsetup luksAddKey /dev/vdb --key-slot 1
   ```

   Majd törölje az első slotot (0. slot), hogy a régi jelszó érvénytelenné váljon:

   ```
   cryptsetup luksKillSlot /dev/vdb 0
   ```

   Ellenőrizze a slotok állapotát a `cryptsetup luksDump /dev/vdb` paranccsal.

6. Tesztelje a helyreállítást: törölje a fejlécet szimuláltan (csak a virtuális gépen!), és állítsa vissza a mentett fejlécből:

   ```
   dd if=/dev/zero of=/dev/vdb bs=1M count=16
   cryptsetup luksHeaderRestore /dev/vdb --header-backup-file /root/luks-header-backup.img
   ```

Próbálja megnyitni a kötetet az új jelszóval, és ellenőrizze, hogy a fájl olvasható.

7. Készítsen mérési naplót: rögzítse a `luksFormat`, `luksAddKey`, `luksKillSlot`, `luksHeaderBackup` és `luksHeaderRestore` parancsok futási idejét (például a `time` paranccsal), valamint a CPU-terhelést. Hasonlítsa össze a mért időket a várt értékekkel (pl. a `luksFormat` néhány másodperc, a `luksAddKey` kevesebb mint egy másodperc, a `luksHeaderBackup` szinte azonnali, de a `luksHeaderRestore` hasonlóan gyors). A naplóban különítse el a mért és a várt eredményt.

8. Állítsa vissza a virtuális gépet a kiinduló állapotba, vagy törölje a virtuális lemezt, hogy a labor ne maradjon hátra.

Elvárt eredmény. A labor végén rendelkezésre áll egy dokumentum, amely tartalmazza: - A használt konfigurációt (CPU, memória, QEMU verzió, Linux kernel verzió, cryptsetup verzió). - A titkosítási architektúra leírását (LUKS2, AES-256-XTS, Argon2id). - A kulcsrotáció lépéseit és a slotok állapotváltozásait. - A fejléc-mentés és helyreállítás tesztjének eredményét. - A mérési naplót (mért idők, várt idők, eltérések). - A helyreállítási eljárás szöveges leírását.

Visszaállítás. A virtuális gép törlésével vagy a pillanatkép visszaállításával a környezet megszüntethető. A hoszt rendszerén semmilyen változtatás nem történik.

18.2.9 9. Önellenőrző kérdések

1. Mi a különbség a blokkszintű és a fájlszintű titkosítás között, és miért előnyösebb a teljes lemez titkosítás az adatok nyugalmi állapotának védelmére?
2. Hogyan működik a LUKS kulcshierarchiája, és mi a szerepe a fejlécmentésnek a helyreállításban?
3. Milyen védelmet nyújt a TPM a BitLocker esetén, és mi a különbség a TPM-only és a TPM+PIN mód között?
4. Ismertesse a kulcsrotáció két szintjét, és magyarázza el, miért nem jár minden rotáció az adatok újratitkosításával.
5. Soroljon fel legalább három gyakori hibát a tároló titkosítás és kulcskezelés terén, és adja meg a helyes megoldást.

Az itt bemutatott elvek és eszközök a következő fejezet (Backup, archiválás és mentési stratégiák – Backup, archiválás és mentési stratégiák) alapját képezik, ahol a titkosított rendszerek biztonsági mentésének speciális kihívásaival foglalkozunk.

18.3 Backup, archiválás és mentési stratégiák

18.3.1 Mentés, archiválás és mentési stratégiák

A tárolási alrendszerek egyik legfontosabb, mégis gyakran háttérbe szoruló feladata a mentés. A snapshot mechanizmusok – amelyeket az előző fejezetben tárgyaltunk – pillanatképeket adnak, de önmagukban nem helyettesítik a mentést, mert jellemzően ugyanazon a tárolórendszeren, ugyanazon a fizikai eszközön élnek. Ez a fejezet a mentést mint üzletmenet-folytonossági követelményt vizsgálja: megismerjük a teljes, inkrementális és differenciális mentési típusokat, a helyreállítási pont- és időcélokat (RPO, RTO), valamint a visszaállítási tesztek szerepét. A fejezet zárásaként egy 3-2-1 mentési tervet készítünk el emulált környezetben, reprodukálható módon.

18.3.2 A mentés mint üzletmenet-folytonossági követelmény

A mentés elsődleges célja nem az adatok másolása, hanem az **üzleti folyamatok helyreállíthatósága**. Egy rendszer üzemeltetője számára nem az a kérdés, hogy „van-e biztonsági másolatunk”, hanem az, hogy „egy adott hibához képest milyen gyorsan és milyen friss adatokat tudunk visszaállítani”. Ez a szemléletváltás határozza meg a mentési stratégia minden elemét.

Az adatvesztés forrásai rendkívül sokfélék lehetnek:

- **Hardverhiba**: lemezmeghajtó, SSD-vezérlő vagy RAID-vezérlő meghibásodása. A RAID az elérhetőséget növeli, de nem mentés; egy véletlen törlést vagy fájlrendszer-korrupciót a RAID is hűen replikál.
- **Emberi hiba**: téves `rm`, rosszul célzott `UPDATE`, felülírás, rossz konfiguráció. A logikai sérülések ellen csak a mentés vagy a verziózó snapshot véd.
- **Szoftverhiba**: kernelpánik, fájlrendszer-hiba, alkalmazás által korrupttá tett adatbázis.
- **Támadás**: zsarolóvírus (ransomware), amely a csatolt hálózati meghajtókat vagy a közvetlenül elérhető snapshotokat is titkosíthatja. A leválasztott, offline mentés itt az egyetlen védelem.
- **Környezeti katasztrófa**: tűz, víz, áramkimaradás, telephely elvesztése. Az offsite (telephelyen kívüli) másolat a katasztrófa-helyreállítás alapfeltétele.

Ebből következik, hogy a mentési stratégia nem egyszerűen technikai kérdés, hanem **üzletmenet-folytonossági terv** (BCP, Business Continuity Plan) része. A BCP meghatározza a kritikus folyamatokat, azok elfogadható adatvesztési és leállási tűrését, majd ezekből származtatja a mentési követelményeket. A technikai mentési rendszert a következő kérdésekre adott válaszokkal kell tervezni:

- Mekkora adatmennyiséget kell menteni, és milyen gyakran változik?
- Mennyi adatvesztés elfogadható a legrosszabb esetben? (Ez lesz az **RPO**.)
- Mennyi idő alatt kell visszaállni? (Ez lesz az **RTO**.)
- Milyen konzisztencia szükséges a visszaállított adatoknak? (például alkalmazáskonzisztens, nem csak crash-konzisztens)
- Milyen hosszú megőrzési időre van szükség jogi, üzleti vagy compliance okokból?
- Ki férhet hozzá a mentésekhez, és hogyan védjük azokat illetéktelen hozzáféréstől?

A mentés tehát **folyamat**, nem pedig egyetlen parancs. A folyamat elemei: adatkiválasztás, futtatás, ellenőrzés, tárolás, monitorozás és rendszeres **visszaállítási gyakorlat**. Ha a visszaállítást nem teszteljük, akkor nem beszélhetünk mentésről, csak másolat-készítésről.

18.3.3 Alapfogalmak: mentés, archiválás, verziózás, katalógus

A mindennapi szóhasználat gyakran összekeveri a mentést és az archiválást, pedig eltérő életciklusigényt szolgálnak:

- A **mentés** (backup) a működő rendszer másolata, amely a **rövid- és középtávú helyreállítást** szolgálja. Általában gyakran készül, és egy bizonyos megőrzési idő után törlődik vagy felülíródik.
- Az **archiválás** (archival) hosszú távú, ritkán változó adatok megőrzése, jogi vagy üzleti megfelelés érdekében. Az archív adathoz ritkán férnek hozzá, de garantálni kell az épségét évekig, akár évtizedekig.

A **verziózás** fogalma közel áll a snapshotokhoz: ugyanannak a logikai fájlnak vagy objektumnak több időbeli változatát őrizzük meg. A modern biztonsági mentési rendszerekben a verziózás biztosítja, hogy egy rosszindulatú felülírást vagy egy hibás alkalmazásváltozás után is vissza lehessen térni egy korábbi állapothoz.

A **mentési katalógus** (backup catalog) az a metaadatbázis, amely nyilvántartja, hogy melyik mentési futás mikor készült, milyen típusú volt, hol található a fizikai adathordozó, és milyen azonosítóval (checksum) rendelkezik. Katalógus nélkül egy nagy mentési infrastruktúra visszaállítása reménytelenül lassú.

A mentési infrastruktúra tipikus elemei:

- **Mentési kliens**: a védendő rendszer, amelyen a mentési ágens fut.
- **Mentési szerver**: a mentéseket ütemező, katalógust karbantartó központi komponens.
- **Adattár** (repository): a mentési adatok fizikai tárolóhelye, gyakran deduplikáló, tömörítő és titkosító réteggel.
- **Média**: a fizikai hordozó – lemez, szalag, objektumtár, felhőalapú tároló.

18.3.4 Teljes, inkrementális és differenciális mentés

A mentési stratégia legalapvetőbb kérdése, hogy milyen gyakorisággal készüljön teljes másolat, és hogyan kezeljük a változásokat. Három alaptípus létezik, amelyeket gyakran kombinálnak.

18.3.5 Teljes mentés

A **teljes mentés** (full backup) a kiválasztott adathalmaz teljes másolatát készíti el, minden fájlra és metaadatra kiterjedően. Előnye, hogy a visszaállítás egyszerű: csak az utolsó teljes mentést kell visszaolvasni. Hátránya viszont a nagy idő- és tárhelyigény; nagy adathalmazoknál napi teljes mentés gyakran nem megvalósítható.

A teljes mentés szolgál a további típusok **bázisául**: az inkrementális és differenciális mentések mindig egy korábbi teljes mentéshez viszonyítva értelmezhetők.

18.3.6 Inkrementális mentés

Az **inkrementális mentés** (incremental backup) csak azokat az adatokat menti, amelyek **az utolsó mentés óta** – legyen az teljes vagy inkrementális – megváltoztak. Ez a leggyorsabb és legtakarékosabb mentési típus, ezért alkalmas gyakori, akár óránkénti futtatásra.

A visszaállítás azonban láncszerű: szükség van az utolsó teljes mentésre **és** az azt követő összes inkrementális mentésre a visszaállítani kívánt időpontig. Ha a lánc bármely eleme sérült vagy hiányzik, a későbbi inkrementálisok használhatatlanná válnak. Ezért az inkrementális láncokat időnként meg kell szakítani egy új teljes mentéssel. A modern deduplikáló adattárak ezt a problémát **szintetikus teljes mentéssel** (synthetic full) oldják meg, amely a szerver oldalán rakja össze a korábbi teljes és inkrementális adatokból az új teljes képet, anélkül, hogy a kliensről újra minden adatot át kellene olvasni.

18.3.7 Differenciális mentés

A **differenciális mentés** (differential backup) azokat az adatokat menti, amelyek **az utolsó teljes mentés óta** megváltoztak. Ez a megközelítés az inkrementális és a teljes mentés közötti kompromisszum.

A visszaállításhoz csak **két elem** szükséges: az utolsó teljes mentés és a legutóbbi differenciális mentés. Ez egyszerűbbé és megbízhatóbbá teszi a visszaállítást, mint az inkrementális lánc, viszont a differenciális mentések mérete a teljes mentés óta eltelt idővel együtt nő, ezért a hosszú teljes mentés jellegű ciklusok esetén a napi differenciális mentés egyre lassabbá válik.

A három típus jellemzőit az 1. táblázat foglalja össze.

1. táblázat: Mentési típusok összehasonlítása

Jellemző	Teljes	Inkrementális	Differenciális
Mentett adat	minden adat	változás az utolsó mentés óta	változás az utolsó teljes óta
Mentési idő	hosszú	rövid	közepes (idővel nő)
Tárhelyigény	nagy	kicsi	közepes (idővel nő)
Visszaállítási elemek	1	teljes + összes inkrementális	teljes + utolsó differenciális
Visszaállítási idő	gyors	lassabb, láncfüggő	gyorsabb
Lánctörés kockázata	nincs	magas	alacsony

A gyakorlatban a legelterjedtebb stratégia a **teljes + inkrementális** kombináció: például heti teljes mentés és napi inkrementális. A differenciális mentést akkor érdemes választani, ha a visszaállítási idő kritikus, de a napi mentési idő nem, és a teljes mentések ritkábban készülnek.

18.3.8 Megőrzési politika és rotáció

A mentési típusok mellett dönteni kell a **megőrzési időről** (retention). A klasszikus **nagyapa-apa-fiú** (GFS, Grandfather-Father-Son) séma például:

- naponta **fiú** (inkrementális vagy differenciális),
- hetente **apa** (teljes),
- havonta **nagyapa** (teljes, hosszabb megőrzéssel).

A megőrzési időt jogszabályi, üzleti és technikai szempontok együttesen határozzák meg. A túl rövid megőrzés azt kockáztatja, hogy egy későn észlelt sérüléshez már nincs ép másolat; a túl hosszú megőrzés pedig felesleges tárhely- és költségnövekedést okoz.

18.3.9 RPO, RTO és a visszaállítási teszt

A mentési terv két legfontosabb mérőszáma a **helyreállítási pontcél** (RPO, Recovery Point Objective) és a **helyreállítási időcél** (RTO, Recovery Time Objective). Ezeket nem a technikai csapat találja ki önkényesen; az üzleti folyamattulajdonosokkal egyeztetve kell meghatározni.

18.3.10 RPO – helyreállítási pontcél

Az RPO azt a maximális adatvesztést adja meg, amelyet a szervezet egy incidens után elfogadhatónak tart, időben kifejezve. Ha például az RPO 4 óra, akkor legfeljebb 4 óra alatt keletkezett adat veszhet el. Ez közvetlenül meghatározza a mentések gyakoriságát: 4 órás RPO-hoz legalább 4 óránként kell menteni, és a mentési futásnak a várt időn belül be kell fejeződnie.

Az RPO a logikai sérülésekre is vonatkozik: ha egy adatbázisban 24 órával ezelőtt hibás tranzakció futott, és csak napi mentésünk van, akkor a visszaállítás után az azóta keletkezett helyes adatok is elvesznek. A folyamatos adatvédelemnek (CDP, Continuous Data Protection) nevezett technológiák az RPO-t gyakorlatilag nullára csökkentik azáltal, hogy minden változást azonnal, naplószerűen rögzítenek.

18.3.11 RTO – helyreállítási időcél

Az RTO azt az időt adja meg, amely alatt a szolgáltatást vissza kell állítani. Ide tartozik a hiba észlelése, a döntési idő, a mentési adatok előkeresése, a visszaállítás, az alkalmazás- és adatkonzisztencia ellenőrzése, valamint a szolgáltatás újraindítása. Az RTO tehát nem egyenlő a puszta adatmásolási idővel.

Az RTO és RPO gyakran feszültségben áll egymással: minél kisebb az RPO, annál gyakrabban kell menteni, ami nagyobb terhelést és költséget jelent; minél kisebb az RTO, annál gyorsabb visszaállítási infrastruktúra kell, ami szintén költség. A tervezés során ezeket az üzleti hatással kell összevetni.

Például egy webáruház esetében a rendelés-adatbázisra lehet az RPO 15 perc, mert egy elveszett rendelés közvetlen bevételkiesés, míg egy belső dokumentumtár esetében 24 óra is elfogadható lehet. Az RTO a webáruházra 1 óra lehet, a dokumentumtárra pedig 8 óra.

18.3.12 Miért kritikus a visszaállítási teszt?

A mentés igazi próbája nem a mentési futás sikere, hanem a **visszaállítási teszt** (restore test). A gyakorlati tapasztalat azt mutatja, hogy a visszaállítási tesztek hiánya a leggyakoribb oka annak, hogy egy mentési rendszer katasztrófahelyzetben csődöt mond. A tipikus hibák:

- A mentési futás sikeresen lezárul, de a mentett adat olvashatatlan, mert a célmédia hibás.
- A visszaállító szkript hibás, elavult, vagy hiányzó függőséget feltételez.
- A mentés **crash-konzisztens**, de nem **alkalmazáskonzisztens**; az adatbázis visszaállítás után nem indul el, mert a tranzakciós napló nem volt konzisztens állapotban.
- A katalógus vagy a titkosítási kulcs elveszett, így a mentés visszafejthetetlen.
- A mentés csak helyben van, és a katasztrófa a mentést is megsemmisítette.

A visszaállítási teszt célja, hogy ezeket a hibákat **időben, nem éles katasztrófa közben** fedje fel. A teszt típusai:

- **Fájlszintű visszaállítás**: egyetlen fájl vagy könyvtár visszaállítása, gyors ellenőrzés.
- **Alkalmazásszintű visszaállítás**: például egy adatbázis visszaállítása és indítása.
- **Teljes rendszer-visszaállítás** (bare-metal restore): a teljes rendszerkép visszaállítása üres hardverre vagy virtuális gépre.
- **Katasztrófa-szimuláció**: a teljes infrastruktúra helyreállítása egy alternatív környezetben.

A teszt gyakoriságát az RTO és a kockázat határozza meg: kritikus rendszereknél legalább negyedévente, de sok szervezetnél havi gyakoriság ajánlott. A teszt eredményét dokumentálni kell, beleértve a mért visszaállítási időt, a hibákat és a javító intézkedéseket.

18.3.13 Tipikus hibák és a helyes megoldás

A mentési rendszerek tervezése és üzemeltetése során visszatérő hibák:

1. **Nincs visszaállítási teszt.** A mentés fut, a napló zöld, de senki nem tudja, hogy vissza lehet-e állítani.
2. **A mentés ugyanazon a fizikai tárolón van**, mint az éles adat. Egy lemezhiba vagy ransomware mindkettőt tönkreteszi.
3. **Hiányzik az offsite másolat.** A helyi tűz vagy betörés a teljes mentési állományt megsemmisíti.
4. **A mentés nem titkosított.** A telephelyen kívülre kerülő mentési adathordozó elvesztése adatszivárgást okoz.
5. **Nincs megfigyelés és riasztás.** A mentés napok óta nem fut, de senki nem vette észre.
6. **Nincs verziózás.** Az inkrementális mentés csak a legutóbbi állapotot őrzi, így egy régebbi, logikailag ép állapot nem állítható vissza.
7. **Az adatbázis konzisztenciája nincs kezelve.** A `mysqldump`-szerű mentés nem mindig konzisztens; tranzakciós naplót, VSS-t vagy LVM snapshotot kell használni.
8. **A megőrzési idő önkényes**, nem pedig az üzleti igényekből levezetett.

A **helyes megoldás** elvei:

- A 3-2-1 szabály alapján: **legalább 3 példány** az adatokból (az éles + 2 mentés), **legalább 2 különböző médiatípuson**, és **legalább 1 példány telephelyen kívül**.
- Minden mentési futás automatikusan ellenőrzi a checksumokat, és a katalógus frissül.
- A mentések titkosítottak, a kulcsokat pedig külön, a mentési adatoktól elkülönítve tárolják és menedzselik (lásd a Titkosítás: LUKS, BitLocker és kulcskezelés fejezet kulcskezelési elveit).
- A visszaállítási tesztek ütemezettek, dokumentáltak, és a hibákat következő mentési ciklusban javítják.
- A mentési stratégia RPO- és RTO-igényekből indul ki, nem pedig a rendelkezésre álló eszközökből.

18.3.14 Labor: 3-2-1 mentési terv készítése

A labor célja, hogy egy kis, emulált környezetben saját kezűleg állítsunk össze egy 3-2-1 elvet követő mentési tervet, és mérésekkel igazoljuk a visszaállíthatóságot. A labor **nem** éles rendszeren futtatandó; minden művelet helyettesítő adatokkal és emulált útvonalakkal történik. A bemutatott parancsok szemléltető jellegűek, valós lemezre vagy éles adatra nem szabad őket változtatás nélkül lefuttatni.

18.3.15 Kiinduló állapot és konfiguráció

Az alábbi környezetet használjuk:

- Emulált hoszt: QEMU virtuális gép, 1 vCPU, 1 GiB RAM, 16 GiB virtuális lemez.
- Operációs rendszer: minimális Linux, kernel 6.6 (emulált), ext4 fájlrendszer.

- Helyettesítő adatkészlet: `~/data/` könyvtár, amelyben vegyesen találhatók szövegfájlok, bináris fájlok és egy SQLite adatbázis (`app.db`). Az adatbázis a mentés pillanatában tranzakciósan konzisztens állapotba hozható a `VACUUM INTO` művelettel.
- Mentési célok:
  - `~/backups/local/` – helyi „második média” (külön kötetet szimulálunk).
  - `~/backups/offsite-sim/` – offsite másolat szimulációja (például egy másik gépre való `rsync`-elési útvonal).
- Titkosítás: a mentéseket `age` típusú, helyettesítő kulccsal titkosítjuk (a kulcs csak a laborban létezik, nem valós).

A build- és futtatási parancsok:

```
# Emulált környezet indítása (szemléltetés, nem éles rendszerhez)
qemu-system-x86_64 \
  -drive file=lab-os068.qcow2,format=qcow2 \
  -m 1G -smp 1 -nographic
```

A laborban a mentési feladatokhoz a `tar` és `rsync` eszközöket használjuk, a konkurens írások elkerülésére pedig egy rövid alkalmazásszünetet feltételezünk, amelyet a mentési terv dokumentál.

18.3.16 Lépések

1. **Adatkészlet előkészítése és ellenőrző összeg.** Hozzunk létre egy 200 MiB méretű tesztadatkészletet, és számítsuk ki a kezdeti SHA-256 checksumokat:

```
# Helyettesítő adatkészlet generálása
mkdir -p ~/data/documents ~/data/binaries
dd if=/dev/urandom of=~/data/binaries/random1.bin bs=1M count=100 status=none
for i in $(seq 1 100); do
  echo "dokumentum $i - $(date --iso-8601=seconds)" > ~/data/documents/file_$i.txt
done
sqlite3 ~/data/app.db "CREATE TABLE IF NOT EXISTS orders(id INTEGER PRIMARY KEY, item TEXT, qty
↪  INTEGER);"
sqlite3 ~/data/app.db "INSERT INTO orders(item,qty) VALUES('hálózati
↪  kapcsoló',2),('szerver',1);"

# Kezdeti checksumok rögzítése
find ~/data -type f -print0 | xargs -0 sha256sum > ~/checksums_initial.txt
```

2. **Első teljes mentés.** Készítsünk teljes mentést, tömörítéssel, a helyi második médiára. A mentés közben az adatbázist konzisztens állapotba hozzuk:

```
# Alkalmazáskonzisztens pillanat az SQLite számára
sqlite3 ~/data/app.db "VACUUM INTO '/tmp/app-consistent.db';"
mv /tmp/app-consistent.db ~/data/app.db

# Teljes mentés
START=$(date +%s)
tar -czf ~/backups/local/full-$(date +%F-%H%M).tar.gz -C ~/data .
END=$(date +%s)
echo "Teljes mentés ideje: $((END-START)) másodperc"
```

3. **Helyi második másolat és offsite szimuláció.** A helyi mentésről készítsünk egy másolatot az offsite szimulált útvonalra, és mindkét példányról rögzítsük a checksumot:

```
# Helyi máshol tárolt másolat (második média)
mkdir -p ~/backups/local-copy
cp ~/backups/local/full-*.tar.gz ~/backups/local-copy/

# Offsite szimuláció (például rsync távoli gépre)
rsync -av ~/backups/local/ ~/backups/offsite-sim/

# Ellenőrző összeg a mentési példányokra
sha256sum ~/backups/local/full-*.tar.gz ~/backups/local-copy/full-*.tar.gz
↪  ~/backups/offsite-sim/full-*.tar.gz > ~/backup_checksums.txt
```

4. **Változtatás az adatokon és inkrementális mentés.** Módosítsunk néhány fájlt, adjunk hozzá újat, majd készítsünk inkrementális mentést a változásokról. Az inkrementális mentés alapja az előző teljes mentés: használhatjuk a `tar --listed-incremental` kapcsolóját, vagy `rsync --link-dest` opcióját. A laborban a `tar` megközelítést választjuk:

```
# Változtatások
echo "új rendelés" >> ~/data/documents/file_101.txt
sqlite3 ~/data/app.db "INSERT INTO orders(item,qty) VALUES('tápegység',5);"

# Inkrementális mentés
START=$(date +%s)
tar -czf ~/backups/local/incr-$(date +%F-%H%M).tar.gz \
  --listed-incremental=~/backups/local/backup.snar \
  -C ~/data .
END=$(date +%s)
echo "Inkrementális mentés ideje: $((END-START)) másodperc"

# Az inkrementális mentés ellenőrző összegét is rögzítjük
sha256sum ~/backups/local/incr-*.tar.gz >> ~/backup_checksums.txt
```

5. **Offsite másolat frissítése.** A 3-2-1 szabály akkor teljesül, ha az offsite példány is tartalmazza az új mentést:

```
rsync -av ~/backups/local/ ~/backups/offsite-sim/
```

6. **Visszaállítási teszt.** A legfontosabb lépés: állítsuk vissza az adatokat egy üres könyvtárba, és ellenőrizzük a checksumokat. A teszt során mérjük a visszaállítás idejét:

```
# Visszaállítási cél létrehozása
RESTORE_DIR=~/restore-test
rm -rf $RESTORE_DIR && mkdir -p $RESTORE_DIR

# Teljes mentés visszaállítása
START=$(date +%s)
tar -xzf ~/backups/local/full-*.tar.gz -C $RESTORE_DIR
# Inkrementális mentés visszaállítása
tar -xzf ~/backups/local/incr-*.tar.gz --listed-incremental=/dev/null -C $RESTORE_DIR
END=$(date +%s)
echo "Visszaállítási idő: $((END-START)) másodperc"

# Ellenőrzés: a visszaállított fájlok checksumja egyezzen a várt értékkel
find $RESTORE_DIR -type f -print0 | xargs -0 sha256sum > ~/restore_checksums.txt
diff <(sort ~/checksums_initial.txt) <(sort ~/restore_checksums.txt) && echo "OK"
```

A `diff` parancs akkor sikeres, ha a visszaállított fájlok tartalma pontosan egyezik az adatkészlet módosított állapotával. Vegye figyelembe, hogy a fenti `tar --listed-incremental` visszaállítás egyszerűsített; éles környezetben a megbízható visszaállításhoz a teljes inkrementális láncot a helyes sorrendben kell visszaolvasni, és a `--listed-incremental` állapotfájl használatát a mentési szoftvernek kell kezelnie.

18.3.17 Mérési napló és várt vs. mért eredmény

A labor során az alábbi méréseket rögzítjük, és a várt értékeket elkülönítjük a mért értékektől. A várt értékek becslések, amelyeket a hardverkonfiguráció és az adatméret alapján előre megadtunk; a mért értékek a tényleges futásból származnak.

2. táblázat: Mérési napló a 3-2-1 mentési laborhoz

Művelet	Várt idő (s)	Mért idő (s)	Várt méret (MiB)	Mért méret (MiB)	Megjegyzés
Teljes mentés	5–15	[mért]	~100–120	[mért]	tömörítéssel
Inkrementális mentés	1–3	[mért]	~0,1–0,5	[mért]	csak változások
Offsite másolat	1–5	[mért]	nincs	nincs	rsync
Visszaállítás	5–10	[mért]	nincs	nincs	full + incr lánc
Ellenőrző összeg	<1	[mért]	nincs	nincs	SHA-256 egyezés

A „mért” mezők kitöltése a tényleges laborfuttatásból származik. A várt és mért értékek eltérése esetén elemezni kell az okokat: például a tömörítési arány az adat jellegétől függ, a hálózati másolási idő az emulált hálózat sávszélességétől.

18.3.18 A 3-2-1 terv dokumentálása

A labor átadandó eredménye egy **mentési terv** dokumentum, amely a következőket tartalmazza:

- A védendő adatkészlet leírása és kritikussági besorolása.
- Az RPO és RTO követelmények, indoklással (például: RPO = 24 óra, RTO = 4 óra).
- A mentési típusok és ütemezés (például: heti teljes, napi inkrementális).
- A 3-2-1 szabály teljesülésének igazolása: 3 példány (éles + helyi mentés + offsite mentés), 2 médiatípus (helyi lemez és szimulált offsite), 1 offsite.
- A visszaállítási teszt leírása, időpontja, mért eredményei.
- A mentések titkosításának és kulcskezelésének leírása (hivatkozva a Titkosítás: LUKS, BitLocker és kulcskezelés fejezetre).
- A mérési napló, a várt és mért értékek elkülönítésével.

18.3.19 Összefoglalás

A mentés nem technikai luxus, hanem az üzletmenet-folytonosság alapfeltétele. A jó mentési stratégia az üzleti RPO- és RTO-követelményekből indul ki, és a teljes, inkrementális vagy differenciális mentési típusokat ezekhez igazítja. A 3-2-1 szabály minimális védelmet ad a hardverhibák, emberi hibák, ransomware és katasztrófák ellen. A mentések értéke azonban csak annyi, amennyit a visszaállítási teszt igazol. A fejezetben bemutatott labor egy kis, ellenőrizhető környezetben gyakoroltatja a terv elkészítését, a mérést és a visszaállítást. A következő fejezet (Objektumtár, blokk- és fájlszintű tárolás) az objektumtár, blokk- és fájlszintű tárolás témakörével bővíti ezt a képet, ahol a mentési adattár gyakran maga is objektumtár alapú.

Önellenőrző kérdés: Tegyük fel, hogy egy vállalat RPO-ja 1 óra, RTO-ja 2 óra. A jelenlegi mentési rendszer heti teljes mentést és napi inkrementálist készít, a visszaállítási teszt pedig soha nem futott. Milyen konkrét lépéseket tenne a rendszer megfelelőségének elérésére, és miért?

18.4 Objektumtár, blokk- és fájlszintű tárolás

Amikor egy operációs rendszer adatot ír vagy olvas, a kérés több rétegen halad át: a felhasználói folyamat a VFS-en, a fájlrendszeren, a blokkrétegen és az eszközmeghajtón keresztül éri el a fizikai adathordozót. Ez a hagyományos kép azonban csak a hoszt szempontjából teljes. Ha a tárolót hálózaton keresztül, több hosztról vagy felhőszolgáltatásként használjuk, három különböző absztrakciós modell közül választhatunk: blokk-, fájl- és objektumtárból. A választás nem csupán „melyik gyorsabb", hanem a névtér, a frissítési szemantika, a megosztás, a skálázhatóság és az üzemeltetési költség közötti kompromisszum. Ez az alfejezet a három modellt, a fájlrendszerhez való illesztésüket, az S3 API és az eventual consistency kérdéseit, valamint a lifecycle-szabályok gyakorlati használatát mutatja be.

18.4.1 1. A három tárolási modell

A blokk-, fájl- és objektumtár közötti különbség elsősorban az absztrakció szintjében és a hálózati hozzáférés módjában ragadható meg. Ugyanaz a háttér-hardver többféle modellben is megjelenhet; a kernelben azonban más-más réteg kezeli őket.

Blokktároló. A blokktároló legalacsonyabb szintű, hagyományos modell. A kernel számára a tároló egy lineáris blokkeszköz, amelyet fix méretű blokkok (jellemzően 512 vagy 4096 bájt) szerint lehet olvasni és írni. A fájlrendszer erre az absztrakcióra épül, és a VFS ezen keresztül kap szolgáltatást. Hálózati blokktárolóknál az iSCSI, Fibre Channel vagy NVMe-oF protokoll a SCSI/NVMe parancsokat továbbítja a távoli eszköznek. Felhő környezetben az attachable volume ugyanezt a modellt nyújtja. Az alkalmazás nem közvetlenül blokkokat lát, hanem a ráépített fájlrendszert; a blokktároló teljesítményét ezért a fájlrendszer overheadjével együtt kell mérni. A blokktároló erőssége az alacsony késleltetés, a tetszőleges véletlen hozzáférés és a tranzakciós munkaterhelés. Gyenge pontja, hogy önmagában nem osztható meg egyszerre több hoszt között anélkül, hogy clustering fájlrendszert (pl. GFS2, OCFS2) vagy speciális I/O-szabályozást ne vezetnénk be. A blokktároló jellemző példái: helyi NVMe, SAN LUN, felhőben csatolható EBS-kötet.

Fájltároló. A fájltároló magasabb szintű absztrakció: elnevezett, hierarchikus adatfolyamokat és metaadatot kínál, gyakran hálózati protokollon keresztül. A kliens kernel a VFS-be mountolja az NFS- vagy SMB-exportot, így a felhasználói folyamat a megszokott `open`, `read`, `write`, `fsync`, `lock` rendszerhívásokat használhatja. A fájltároló előnye a kényelem: a jogosultság, a lockolás, a seek és a rename a protokoll szemantikájában értelmezve van. Hátránya, hogy a központi névtér és a metaadat-kiszolgáló szűk keresztmetszetté válhat, a hálózati lockolás a kliensoldali cache koherenciájával együtt nehéz, és a skálázódás korlátos. Jellemző példa: NFS home-könyvtár, SMB fájlmegosztás, felhőben elérhető file server szolgáltatás.

Objektumtár. Az objektumtár a legmagasabb szintű absztrakció: nincs szektor, nincs POSIX-hierarchia, helyette lapos névtérben (`bucket` és `key`) tárolunk objektumokat. Az objektum egy bináris adat, hozzá tartozó kulcs, metaadatok, storage class és verzióazonosító. Az API tipikusan HTTP REST, például S3. Az objektumtár előnye a szinte korlátlan skálázhatóság, az erasure coding, a HTTP-alapú elérés és a globális CDN-integráció. Hátránya, hogy nem POSIX: nincs hatékony részleges felülírás, nincs `seek`, nincs fájlzár, és a konzisztencia-modell implementációfüggő. Jellemző példa: Amazon S3, Ceph RGW, MinIO.

Fontos hangsúlyozni: ezek a modellek egymásra is épülhetnek. Egy objektumtár belül blokkeszközöket használhat, egy blokktárolóból fájlrendszer válhat, egy fájltároló mögött pedig objektumtár

is lehet (például S3-gateway). A fejlesztő számára az a kérdés, hogy az alkalmazás milyen szemantikát és milyen teljesítményt igényel, és hogy ezt a kernel melyik rétegén tudja leghatékonyabban kiszolgálni.

18.4.2 2. Illesztés a fájlrendszerhez

A Linux tárolási verem a következő rétegekből áll: alkalmazás → VFS → fájlrendszer → blokkréteg → eszközmeghajtó → fizikai eszköz. A blokk- és a fájltároló közvetlenül illeszthető ebbe a verembe. A blokktároló a blokkréteg alatt jelenik meg, mint egy `/dev/sda` eszköz; a fájltároló a VFS alatt mountolható, és a fájlrendszer fölött ad POSIX-nézetet.

Az objektumtár nem illeszkedik természetesen ebbe a hagyományos verembe, mert nem blokk- vagy fájlszemantikát kínál, hanem REST API-t. Mégis gyakran szükség van arra, hogy egy objektumtárat fájlrendszerként lássunk. Ilyenkor a kernel vagy egy felhasználói térben futó FUSE-modul (például `s3fs`, `JuiceFS`, `Ceph RGW`) segítségével POSIX-nézetet kapunk az objektumtár fölött. Ez a felületi illesztés azonban nem költségmentes. A POSIX-szemantika és az objektumtár-szemantika közötti különbségek tipikus hibaforrást jelentenek.

Nézzük meg a legfontosabb eltéréseket:

- **Részleges felülírás.** A fájlrendszeren a `write()` a fájl egy tartományát módosíthatja; az objektumtárban egy `PutObject` a teljes objektumot felülírja. Ha egy s3fs-kliens 4 KiB-ot módosít egy nagy objektumban, az a teljes objektum letöltését és újrafeltöltését jelentheti, ami drasztikus teljesítményromlást okoz.
- **Atomi rename.** A POSIX `rename()` atomikus ugyanabban a kötetben. Az S3-ban nincs rename: a művelet `COPY` + `DELETE`, ami nem atomikus. A kliensek ezt különböző trükkökkel próbálják elrejteni, de az atomitás megszűnik.
- **Zárak és megosztás.** A fájltároló támogatja a byte-range lockingot, az objektumtár nem. A több kliens egyidejű írása leggyakrabban last-writer-wins eredményt ad.
- **`mmap`.** A POSIX `mmap()` laphiba-vezérelt I/O-t feltételez, amely blokkméretű visszaírásokkal dolgozik. Objektumtár fölött ez legfeljebb korlátozottan, nagy pufferelléssel szimulálható.
- **Konzisztencia.** A fájlrendszeren a `fsync()` garantálja, hogy az adat tartósan tárolódott. Az objektumtár `PutObject` válasza hasonló garanciát ad, de a hálózati replikáció és a cache miatt a frissítés más kliensek számára később válhat láthatóvá (lásd a konzisztenciáról szóló részt).

A helyes megoldás általában az, hogy az alkalmazás közvetlenül a megfelelő natív API-t használja, nem pedig POSIX-abstrakciót erőltet az objektumtár fölé. Ha a program honlapra szánt statikus képeket tárol, közvetlenül S3 REST API-t hívjon; ha tranzakciós adatbázist futtat, blokktárolóra építsen fájlrendszert; ha felhasználói home-könyvtárat oszt meg, fájltárolót használjon.

18.4.3 3. Az S3 API, az eventual consistency és a lifecycle

Az S3 a legismertebb objektumtár-protokoll, amelyet számos szolgáltatás és szoftver implementál. Alapelemei a `bucket` (tároló) és a `key` (objektumkulcs). A névtér lapos: a kulcsban perjellel tagolt „könyvtárszerű” elnevezés csak konvenció, nem valódi hierarchia. A bucket nevek globálisan egyediek, a kulcsok a bucketen belül egyediek.

18.4.4 3.1. Alapműveletek

Az S3 alapműveletei pszeudokódban:

```
PutObject(bucket, key, data, metadata, storage_class) → version_id
GetObject(bucket, key, range) → data, metadata
DeleteObject(bucket, key) → void
ListObjectsV2(bucket, prefix) → { key, size, etag, last_modified }
HeadObject(bucket, key) → metadata
```

A gyakorlatban az API HTTP-kérésekkel valósul meg. A `PutObject` a teljes objektumot tárolja, és `ETag`-et ad vissza, ami az objektumtartalom ellenőrző összege. A `GetObject` a teljes objektumot adja vissza, de `Range` fejléccel részlet is kérhető. A `DeleteObject` törli a kulcs aktuális verzióját. A `ListObjectsV2` a prefixszel rendelkező kulcsokat lapozva listázza; a lista nem fájlrendszeri szintű szinkron metódus, ezért a friss feltöltések késleltetve jelenhetnek meg benne.

Nagy objektumokhoz a multipart upload ajánlott: a feltöltő fél több részletben tölt fel, majd egy befejező kéréssel zárja az objektumot. Ez lehetővé teszi a párhuzamos átvitelt, a hálózati megszakításokból való folytatást és a nagy átviteli sebességet. A multipart részeket először függetlenül tárolják, és csak a befejező művelet után válnak láthatóvá a tényleges objektumként.

18.4.5 3.2. Verziózás és metaadatok

A bucket verziózása esetén minden kulcsnak több verziója lehet. Ha a `PutObject` új tartalmat ír egy létező kulcsra, az S3 létrehozza az új verziót, és a régi verzió elérhető marad a verzióazonosítójával. A `DeleteObject` verziózott bucketen csak egy delete-markert helyez el, így a tárgy rejtve marad, de a régi verziók helyreállíthatók. Ez a mechanizmus alapja a véletlen törlés elleni védelemnek és a snapshotnak.

Az objektumokhoz metaadatok társíthatók: tartalom típusa, cache-vezérlő fejlécek, alkalmazásspecifikus kulcs-érték párok. A metaadatok mérete korlátozott, ezért a nagy mennyiségű kereshető információt általában külön adatbázisban tárolják, és az objektumtárban csak a bináris adatot és a minimális metaadatot tartják.

18.4.6 3.3. Eventual consistency

Az elosztott objektumtárak replikálják az adatokat több szerver, esetleg több adatközpont között. Az írás után a replikák frissítése aszinkron módon, késleltetéssel történik. Az **eventual consistency** (végül konzisztens) modell szerint a rendszer garantálja, hogy ha nincs további írás, akkor előbb vagy utóbb minden olvasó ugyanazt az értéket látja, de egy adott pillanatban az olvasók régi és új értéket is kaphatnak.

Klasszikus példa: egy kliens feltölt egy képet, majd azonnal listázza a bucket tartalmát. Ha a lista egy olyan replikától szolgálódik, amely még nem kapta meg az új objektumot, a friss kép nem jelenik meg. Ugyanez történhet `GetObject` esetén is: a feltöltés sikeres válasza után egy másik kliens még a régi verziót kaphatja.

Fontos azonban, hogy a nyilvános S3-felhők 2020 végétől a legtöbb alapszolgáltatásra erős read-after-write konzisztenciát dokumentálnak. Ez azt jelenti, hogy a `PutObject` sikeres válasza után a `GetObject` és a `ListObjects` a legfrissebb tartalmat adja. A konzisztencia-modell azonban implementációfüggő; a saját, emulált vagy régebbi rendszerek gyakran az eventual consistency modellt feltételezik. A fejlesztőnek mindig az adott szolgáltatás aktuális dokumentációját és az SDK viselkedését kell ellenőriznie.

A formális definíció segít a kommunikációban. Egy rendszer **lineárisan konzisztens** (linearizable), ha minden művelet hatása egyetlen, a valós idővel összeegyeztethető sorrendbe rendezhető, mintha egyetlen példányon futottak volna. Az **eventual consistency** ennél gyengébb: a műveletek sorrendje nem szükségszerűen globálisan rendezhető, és az olvasók átmenetileg divergens értékeket látnak. Az eventual consistency nem hiba, hanem kompromisszum: lehetővé teszi a nagy skálázást, az alacsony késleltetést és a nagy rendelkezésre állást, de az alkalmazást úgy kell tervezni, hogy elviselje a rövid távú eltéréseket.

Az alkalmazásokban ezt gyakran úgy kezelik, hogy az író fél nem feltételezi az azonnali láthatóságot; idempotens műveleteket használ, vagy a feltöltés után egy olyan indexbe (például adatbázisba) írja a kulcsot, amely erős konzisztenciát ad. Így az objektumtár aszinkron replikációja nem befolyásolja a felhasználói élményt.

18.4.7 3.4. Lifecycle-szabályok

Az életciklus-szabályok segítségével az objektumtár automatikusan vált storage class-t, vagy törli az objektumokat, időbeli feltételek alapján. A szabályok aszinkron módon, háttérfolyamatban futnak; nem azonnali műveletek, ezért a törlésre vagy átmenetre építeni nem szabad.

A jellemző átmenetek:

- `STANDARD` – gyors, drágább tárolás, gyakori eléréshez.
- `STANDARD_IA` – ritkábban elért adatokhoz, alacsonyabb tárolási költség, de magasabb lehívási díj.
- `GLACIER` – archiválás, nagy késleltetésű visszaolvasás.
- `DEEP_ARCHIVE` – nagyon olcsó, hosszú távú archiválás.

A lifecycle-szabály például előírhatja: 30 nap után váltson `STANDARD_IA`-ba, 90 nap után `GLACIER`-be, 365 nap után törlődjön. Verziózott bucketen a szabály külön vonatkozhat a current és a noncurrent verziókra. A nem aktuális verziókat például 7 nap után törölheti a rendszer.

A gyakorlatban a lifecycle-szabályoknál a következőkre kell figyelni:

- A friss feltöltésű objektumokra a szabály csak a megadott napok elteltével érvényesül; a napok az objektum létrehozása vagy utolsó módosítása óta számítódnak.
- A storage class váltás költséggel járhat; az IA osztályról való visszaváltás lehívási díjat von maga után.
- A `GLACIER` osztályban az objektum nem olvasható közvetlenül; először restore műveletet kell kezdeményezni, ami percekig vagy órákig tarthat.
- A törlés nem azonnali, ezért a lifecycle nem helyettesíti a biztonsági mentést.

18.4.8 4. Modell kiválasztása egy alkalmazáshoz

A tárolási modell kiválasztásának központi kérdése: milyen hozzáférési mintát mutat az alkalmazás, és milyen szemantikát igényel.

Gyakorlati példa: képfeltöltő szolgáltatás. A felhasználó feltölt egy képet, a rendszer három méretű thumbnailt generál, és a webes felület visszaolvassa a képeket. A képfájlok nagy méretűek, binárisak, egyszer íródnak, sokszor olvasódnak, és nem igényelnek részleges felülírást. Ez a minta tökéletesen illeszkedik az objektumtárhoz: a képek `PutObject`-tal kerülnek a bucketbe, a webes kiszolgáló pedig `GetObject`-tal vagy CDN-en keresztül olvassa őket. A thumbnail-ek szintén objektumok, mert egyszer keletkeznek, ritkán változnak.

A felhasználói fiókok, jogosultságok, feltöltési naplók tranzakciósak, kis sorokból állnak, és erős konzisztenciát igényelnek: ezek számára blokktárolóra épített relációs adatbázis az ésszerű választás. Az adatbázis adatfájlja számára a fájlrendszer rétegét is ki kell választani; a blokktároló feletti ext4 vagy XFS megfelelő, mert a DB a `fsync`-et használja a tartóssághoz, és a `pwrite`-tel részleges frissítést végez.

A webes kiszolgáló gyorsítótárának (például a már legenerált thumbnail-ek ideiglenes tárolása) nincs szüksége tartós tárolóra. Ez helyi, efemer fájltároló lehet; ha több kiszolgáló is van, a gyorsítótár elosztott megosztást igényel, de a tartósság nem kritikus.

Ebből a példából is látható, hogy egyetlen alkalmazás gyakran több modellt használ ugyanabban az időben, és a modellválasztásnak a workload jellegéhez kell igazodnia. A rossz választás tipikus következményei:

- Objektumtárra POSIX-fájlműveleteket írni s3fs-en keresztül, miközben az alkalmazás lockot vagy részleges felülírást vár. A teljesítmény katasztrofálisan romlik, a szemantika sérül.
- Fájltárolóba terelni nagyszámú, kis bináris fájlt, és a hálózati metaadat-forgalom miatt a névtér telítődik.
- Blokktárolót több hoszttal megosztani fájlrendszer nélkül, ami korrupcióhoz vezet.
- Csak a GB-költséget figyelni, és nem számolni az IOPS- és egress-díjakkal.

18.4.9 5. Labor: tárolási modell kiválasztása és mérés

A labor célja, hogy a hallgató reprodukálható, emulált környezetben összehasonlítsa a három tárolási modell viselkedését kis fájlokkal, majd döntési javaslatot készítsen egy megadott alkalmazáshoz. A labor nem éles rendszeren fut, és a host valódi lemezét nem módosítja.

18.4.10 5.1. Kiinduló állapot és konfiguráció

A méréshez QEMU vendég Linuxot használunk, mert a három modell költséghatékonyan összevethető egyetlen virtuális gépen belül. A host hardver és a vendég konfiguráció:

- Hoszt: x86-64 CPU, legalább 8 GiB RAM, Linux kernel tetszőleges modern verziója.
- QEMU vendég: QEMU 8.0, 2 vCPU, 4 GiB RAM, 20 GiB qcow2 lemez.
- Vendég operációs rendszer: Ubuntu Server 22.04 LTS, kernel 6.2.
- Mérőeszközök: `fio` 3.35, `minio` server release, `mc` (MinIO client), `curl` 8.0.

A reprodukciós csomag tartalmazza a vendég lemezkép nevet, a QEMU indítási parancsot, a kernel és a mérőeszközök verzióját, valamint a nyers mérési naplót. A mérési parancsok értékei példaként, szemléltetésre szolgálnak; a konkrét mért értékeket a naplóban rögzíteni kell.

Vendég előkészítése:

```
qemu-img create -f qcow2 /srv/os069/qemu-disk.qcow2 20G
qemu-system-x86_64 \
  -enable-kvm -m 4G -smp 2 \
  -drive file=/srv/os069/qemu-disk.qcow2,if=virtio \
  -netdev user,id=n1 -device virtio-net-pci,netdev=n1 \
  -cdrom /srv/os069/ubuntu-22.04.3-live-server-amd64.iso
```

A telepítés után a vendégben a `srv` partíció helyett a `qcow2` rendszerlemezt használjuk. A host valódi lemezét nem csatoljuk.

18.4.11 5.2. Blokkréteg mérése

A blokktároló tesztjéhez egy extra virtuális lemezt csatolunk, és `fio`-val mérjük a nyers blokkeszköz véletlen olvasási/írási teljesítményét:

```
# Vendégben, emulált környezetben
lsblk
# /dev/vdb az új csatolt lemez
fio --name=block --filename=/dev/vdb \
    --size=4G --rw=randwrite --bs=4k --ioengine=libaio \
    --direct=1 --iodepth=32 --numjobs=1 --time_based --runtime=30
```

A várt eredmény: a blokkeszköz 4K véletlen írási IOPS értéke a virtuális háttér és a QEMU lemez backendétől függően több ezer IOPS lehet; a késleltetés tipikusan 0,1–1 ms nagyságrendű. A mért értékeket a naplóban külön jelöljük.

A blokkeszközön nincs fájlrendszer, ezért a nyers mérés az alsó határt mutatja: ennél a fájlrendszer overheadje miatt a fájlolvasás sem lehet gyorsabb.

18.4.12 5.3. Fájlréteg mérése

A fájltároló szimulálásához a vendégben ugyanarra a blokkeszközre ext4 fájlrendszert hozunk létre, és fájlokon keresztül mérünk:

```
sudo mkfs.ext4 /dev/vdb
sudo mount /dev/vdb /mnt/file
fio --name=file --directory=/mnt/file \
    --size=4G --rw=randwrite --bs=4k --ioengine=libaio \
    --direct=1 --iodepth=32 --numjobs=1 --time_based --runtime=30 \
    --fsync=1
```

A `--fsync=1` minden írás után fsync-et hajt végre, ami a tartósságot modellezi. A várt eredmény: az IOPS alacsonyabb, mint a nyers blokknál, mert a fájlrendszer metaadatait is írni kell. A mérés ezzel a fájlréteg és az `fsync` költségét mutatja.

A hálózati fájltároló (NFS) pontosabb modellezéséhez egy NFS-szervert is indíthatunk a vendégben, de az alap laborban a helyi fájlrendszer elegendő a hierarchikus névtér és a fájl-metaadat overhead bemutatására.

18.4.13 5.4. Objektumtár mérése

Az objektumtár emulálásához MinIO szervert indítunk a vendégben vagy a hoszton Dockerben. A laborban a hoszton futó Docker konténer szolgálja az S3 API-t, mert a QEMU vendég ezt HTTP-n éri el:

```
# Hoszton, eldobható konténerben; a hitelesítő adatok helyőrzők
docker run --rm -d -p 9000:9000 -p 9001:9001 \
  -e MINIO_ROOT_USER=labuser \
  -e MINIO_ROOT_PASSWORD=labpass \
  quay.io/minio/minio server /data --console-address ":9001"
```

A laborban használt hitelesítő adat (`labuser`/`labpass`) szintetikus, nem valós gyártói titok.

A vendégben konfiguráljuk a klienst, és kis objektumokat töltünk fel:

```
mc alias set lab http://10.0.2.2:9000 labuser labpass
mc mb lab/testbucket

# 1000 darab 4 KiB-os objektum feltöltése; a mérési pontok a naplóba kerülnek
for i in $(seq 1 1000); do
  dd if=/dev/urandom of=/tmp/obj bs=4K count=1 status=none
  /usr/bin/time -o /tmp/s3_put_latency.txt -a \
    mc cp /tmp/obj lab/testbucket/obj_$i
done
```

A várt eredmény: az objektumtár késleltetése (HTTP round-trip, replikáció, metaadat-írás) nagyobb, mint a blokk- és fájltárolóé, jellemzően 2–10 ms nagyságrendű az emulált környezetben. Az átviteli sebesség viszont párhuzamos feltöltéssel növelhető, míg a blokkeszköznél a hálózati stack overhead hiányzik.

A mért és a várt értéket külön kell választani: a fenti értékek levezetett becslések; a tényleges mérési eredményeket a `s3_put_latency.txt` és az `fio` kimenetek tartalmazzák.

18.4.14 5.5. Döntési feladat

A labor utolsó lépéseként a hallgató írjon egy rövid jegyzetfüzetet, amelyben a mért eredmények alapján tárolási modellt választ egy képfeltöltő szolgáltatás következő elemeihez:

1. Eredeti képek és thumbnail-ek tartós tárolása.
2. Felhasználói fiókok és feltöltési naplók erős konzisztenciával.
3. A webes kiszolgáló gyorsítótárának ideiglenes tárolása.

A döntéshez használja a fejezet táblázatát, és a mérési naplóval indokolja a választást. Például:

- A képek esetében az objektumtár előnyös, mert a mérés szerint így a feltöltés HTTP-vel párhuzamosítható, és a fájl nagy mérete nem okoz részleges felülírást.
- Az adatbázis számára blokktároló és fájlrendszer szükséges, mert a 4K véletlen írási IOPS és a `fsync` szemantika ott teljesül.
- A gyorsítótárhoz nem kell tartós tároló; helyi fájlrendszer vagy memória is elegendő.

18.4.15 6. Önellenőrző kérdés

Adott egy felhőalapú alkalmazás, amely időbélyeges mérési adatfájlokat kap, azokat feldolgozza, majd a nyers fájlokat 90 napig ritkán kell visszaolvasni, 365 nap után törölni. Melyik tárolási modellt választja a nyers fájlokhoz, és milyen lifecycle-szabályt állít be? Válaszában térjen ki arra, miért nem elegendő a blokk- vagy fájltároló, és hogyan kezeli az esetleges eventual consistency-t a feltöltés utáni azonnali visszaolvasáskor.

V. RÉSZ – Hálózat, biztonság és virtualizáció

19 Hálózati stack

19.1 Rétegmodellek és TCP/IP

A hálózati kommunikáció összetettsége indokolja, hogy a protokollokat rétegekbe szervezzük. A rétegzés elve lehetővé teszi, hogy egy adott réteg a saját feladatára koncentráljon, és az alatta lévő réteg szolgáltatásait használja anélkül, hogy annak belső működését ismernie kellene. Ezáltal a hálózati stack moduláris, cserélhető és szabványosítható marad. Amikor egy weboldalt letöltünk, az alkalmazási rétegbeli HTTP kérés több protokollrétegen halad keresztül, mire fizikai jelekké alakul, majd a vevő oldalon visszafelé épül fel. Ebben az alfejezetben az OSI- és a TCP/IP-modellt, az IP, TCP, UDP és ICMP protokollok szerepét, valamint az enkapszuláció és az MTU fogalmát tárgyaljuk.

19.1.1 A rétegzés motivációja

A rétegezés nélkül minden hálózati alkalmazásnak közvetlenül kellene kezelnie a hardvert, a hibákat, az útválasztást és a folyamatok közötti kézbesítést. Ez a megközelítés kezelhetetlen lenne, és gátolná az együttműködést a különböző gyártók eszközei között. A rétegek közötti interfészek definiálásával a fejlesztők egy-egy réteget önállóan implementálhatnak, ha betartják a rétegközi szerződéseket. Emellett a hibakeresés is egyszerűbbé válik, mert a hibát rétegenként izolálhatjuk.

Két elterjedt referenciamodellt használunk: az OSI (Open Systems Interconnection) hét rétegű modellt, amely elsősorban oktatási és szabványosítási célokat szolgál, és a TCP/IP modellt, amely a gyakorlatban megvalósult internet-protokollcsaládot írja le.

19.1.2 Az OSI-modell

Az OSI-modell hét rétegre osztja a kommunikációs feladatokat. A rétegek alulról felfelé a következők:

1. **Fizikai réteg (Physical Layer):** bitek átvitele a fizikai közegen. Meghatározza a feszültségszinteket, az időzítést, a csatlakozók fizikai kialakítását és a jelátviteli módot (például elektromos, optikai vagy rádiófrekvenciás). Az adat itt nyers bitfolyam, nincs keretezés vagy címzés.
2. **Adatkapcsolati réteg (Data Link Layer):** keretek (frame) továbbítása két közvetlenül összekapcsolt csomópont között. Feladata a keretezés (a bitek keretekre bontása), a MAC-címek kezelése, a hibaészlelés (például CRC) és bizonyos esetekben a hibajavítás. Példa: Ethernet, Wi-Fi (802.11).
3. **Hálózati réteg (Network Layer):** csomagok (packet) továbbítása a forrás és a cél között akár több köztes hálózaton keresztül. Ez a réteg felel a logikai címzésért (IP-cím), az útválasztásért és a fragmentációért. Példa: IP (IPv4, IPv6), ICMP.

4. **Szállítási réteg (Transport Layer):** végpontok közötti (end-to-end) kommunikáció a hosztokon futó folyamatok között. Két fő protokoll: TCP (kapcsolatorientált, megbízható) és UDP (kapcsolat nélküli, minimális szolgáltatás). A szállítási réteg portszámokkal azonosítja a folyamatokat.
5. **Viszony réteg (Session Layer):** párbeszéd-vezérlés, a kommunikációs munkamenet felépítése, fenntartása és bontása. Gyakorlati példa: RPC-munkamenetek, hosszú életű kapcsolatok kezelése.
6. **Megjelenítési réteg (Presentation Layer):** adatábrázolás, kódolás, titkosítás és tömörítés. Gondoskodik arról, hogy az alkalmazások közötti adatformátum egyeztetett legyen (például karakterkódolás, sorosítás).
7. **Alkalmazási réteg (Application Layer):** a felhasználói alkalmazások hálózati szolgáltatásai. Ide tartoznak a protokollok, mint a HTTP, FTP, DNS, SMTP. Ez a réteg biztosítja az ember vagy az alkalmazás számára a hálózati erőforrások elérését.

Az OSI-modell tiszta elméleti keretet ad, de a gyakorlatban a 5–7. réteg gyakran egybeolvad, és a protokollok nem mindig illeszkednek pontosan a modellhez. Ennek ellenére a rétegek elnevezése és fogalma széles körben használt a hálózati szakmában.

19.1.3 A TCP/IP-modell

A TCP/IP-modell kevesebb rétegből áll, mert a gyakorlati internet-protokollcsalád szerkezetét tükrözi. Az elterjedt felosztás négy réteget különböztet meg:

1. **Hálózati hozzáférési réteg (Network Access Layer):** egyesíti az OSI fizikai és adatkapcsolati rétegét. Feladata a keretek továbbítása a helyi hálózaton belül. Nincs szabványos protokollja a modellben, hanem a konkrét hálózati technológia (Ethernet, Wi-Fi, PPP) valósítja meg.
2. **Internet réteg (Internet Layer):** megfelel az OSI hálózati rétegének. Az IP protokollt és a hozzá kapcsolódó vezérlőprotokollokat (ICMP, IGMP) foglalja magában. Címzés IP-címekkel, útválasztás a csomagok továbbítására.
3. **Szállítási réteg (Transport Layer):** azonos az OSI szállítási rétegével. A TCP és az UDP protokollokat nyújtja, portszámokkal címzi a folyamatokat.
4. **Alkalmazási réteg (Application Layer):** egyesíti az OSI viszony, megjelenítési és alkalmazási rétegét. Az alkalmazások által közvetlenül használt protokollok (HTTP, DNS, SSH) találhatók itt.

Bizonyos oktatási anyagok öt réteget különböztetnek meg, amikor a hálózati hozzáférési réteget külön fizikai és adatkapcsolati rétegre bontják. Ez a felosztás közelebb áll az OSI-modellhez, és segít a hardver és a keretezés szétválasztásában. A gyakorlati implementáció azonban gyakran együtt kezeli ezeket.

A TCP/IP-modell alulról felfelé építkezik: az alsóbb réteg szolgáltatásokat nyújt a felsőbbnek, a felsőbb réteg pedig felhasználja azokat. Az adat a küldő oldalon lefelé halad, minden réteg hozzáadja a saját fejlécét, míg a vevő oldalon felfelé haladva a rétegek eltávolítják a fejléceket.

19.1.4 A rétegek feladatai részletesen

Az alábbiakban a TCP/IP-modell rétegeinek főbb feladatait tekintjük át a gyakorlatban használt protokollokkal együtt.

19.1.5 Alkalmazási réteg

Az alkalmazási réteg protokolljai határozzák meg, hogyan kommunikálnak a felhasználói programok a hálózaton. Például a HTTP (Hypertext Transfer Protocol) kérés-válasz alapú, a DNS (Domain Name System) a nevek és IP-címek közötti leképezést végzi, az SSH pedig titkosított távoli hozzáférést biztosít. Az alkalmazási réteg az adatokat folyamként vagy üzenetként adja át a szállítási rétegnek, és nem foglalkozik a hálózat topológiájával vagy a csomagvesztéssel.

19.1.6 Szállítási réteg

A szállítási réteg a hosztokon futó folyamatok közötti logikai csatornát valósít meg. A két domináns protokoll:

- **TCP (Transmission Control Protocol):** kapcsolatorientált, megbízható, sorrendhelyes adatfolyamot nyújt. A TCP háromfázisú kézfogással (SYN, SYN-ACK, ACK) építi fel a kapcsolatot, sorszámozással és nyugtázással biztosítja a megbízhatóságot, csúszóablakkal vezérli a folyamot, és torlódásvezérlő algoritmusokkal (például Reno, CUBIC) alkalmazkodik a hálózat terheléséhez. A TCP fejléc minimális hossza 20 bájt, további opciókkal bővíthető.
- **UDP (User Datagram Protocol):** kapcsolat nélküli, minimális szolgáltatást nyújtó protokoll. Nem garantálja a kézbesítést, a sorrendet vagy a duplikáció elkerülését, viszont alacsony késleltetést és kisebb fejlécet (8 bájt) biztosít. Alkalmas olyan alkalmazásokhoz, ahol a sebesség fontosabb a megbízhatóságnál, például DNS-lekérdezésekhez, VoIP-hoz vagy video-streaminghez.

A szállítási réteg portszámokkal azonosítja a vevő folyamatot. A portszám 16 bites, így 0–65535 közötti értékeket vehet fel. A 0–1023 közötti jól ismert portok (például 80 HTTP, 443 HTTPS, 53 DNS) a szokásos szolgáltatásokhoz vannak rendelve.

19.1.7 Internet réteg

Az internet réteg fő protokollja az **IP (Internet Protocol)**. Az IP két verzióban létezik: IPv4 és IPv6. Az IPv4-cím 32 bites, általában pontozott decimális formában írjuk (például 192.0.2.1), míg az IPv6-cím 128 bites, hexadecimális, kettőspontokkal tagolt formában (például 2001:db8::1).

Az IP feladata a csomagok (datagramok) továbbítása a forrástól a célig, akár több hálózaton keresztül. Az IP kapcsolat nélküli, best-effort szolgáltatást nyújt: nem garantálja a kézbesítést, a sorrendet, a duplikáció elkerülését vagy a hibamentességet. A megbízhatóságot a felsőbb rétegek (például TCP) biztosítják.

Az IPv4 fejléc minimális hossza 20 bájt, további opciók lehetségesek. Fontos mezői:

- **Verzió (4 bit):** 4 az IPv4, 6 az IPv6.
- **IHL (4 bit):** az IP-fejléc hossza 32 bites szavakban.
- **Típus/szolgáltatás (8 bit):** Differenciált szolgáltatások (DSCP) és ECN.
- **Teljes hossz (16 bit):** a datagram teljes hossza bájtban, fejléccel együtt.
- **Azonosító (16 bit):** fragmentációhoz használt azonosító.
- **Jelzőbitek és fragment offset (16 bit):** fragmentáció vezérlése.
- **TTL (Time To Live, 8 bit):** minden ugrásnál csökken, 0-nál eldobják a csomagot, hogy elkerüljék a hurkokat.
- **Protokoll (8 bit):** a felsőbb rétegbeli protokoll azonosítója (például 6 TCP, 17 UDP, 1 ICMP).
- **Fejléc-ellenőrző összeg (16 bit):** csak az IP-fejléc ellenőrzésére, nem a teljes datagramra.

- **Forrás és cél IP-cím (32-32 bit).**

Az IPv6 fejléce egyszerűbb, rögzített 40 bájt hosszú, és nem tartalmaz fejléc-ellenőrző összeget, mert a fizikai és adatkapcsolati rétegek már biztosítanak hibavédelmet. Az IPv6 kiterjesztett fejlécekkel támogatja a fragmentációt, az útválasztási opciókat és a biztonsági funkciókat.

19.1.8 Adatkapcsolati réteg

Az adatkapcsolati réteg a helyi hálózaton belül továbbítja a kereteket. Leggyakoribb megvalósítása az **Ethernet**. Az Ethernet-keret a következő részekből áll:

- **Cél MAC-cím (6 bájt):** a vevő interfész fizikai címe.
- **Forrás MAC-cím (6 bájt):** a küldő interfész fizikai címe.
- **EtherType (2 bájt):** a felsőbb rétegbeli protokoll azonosítója (például 0x0800 IPv4, 0x86DD IPv6).
- **Adat (46–1500 bájt):** a felsőbb rétegből származó adat.
- **FCS (Frame Check Sequence, 4 bájt):** CRC-ellenőrző összeg a hibás keretek detektálására.

A MAC-cím 48 bites, gyárilag egyedi, és hexadecimális formában, kettőspontokkal tagolva írjuk (például 00:1A:2B:3C:4D:5E). Az adatkapcsolati réteg nem látja az IP-címeket, csak a MAC-címeket használja.

19.1.9 Fizikai réteg

A fizikai réteg a tényleges jelátvitelt végzi. Az Ethernet esetében ez a kábel, a csavart érpáras vagy optikai közeg, a jelszintek és a moduláció. A fizikai réteg a keretek bájtjait bitekké alakítja, és gondoskodik a szinkronizációról. A hálózati kártyák (NIC) általában a fizikai és az adatkapcsolati réteget együtt valósítják meg.

19.1.10 IP, TCP, UDP és ICMP

A TCP/IP protokollcsalád négy alapvető protokollját külön is érdemes áttekinteni, mivel a hálózati stack működésének megértéséhez elengedhetetlenek.

19.1.11 IP

Az IP a hálózati réteg protokollja, amely a datagramok továbbításáért felel. Az IP-címek két részre oszlanak: hálózati azonosítóra és hoszt azonosítóra. Az alhálózati maszk (vagy prefix-hossz) határozza meg a kettő határát. Például a 192.0.2.1/24 cím esetén a hálózati rész 192.0.2, a hoszt rész pedig 1. Az útválasztók a hálózati rész alapján döntik el, hogy hová továbbítsák a datagramot.

Az IP-fragmentáció akkor szükséges, ha egy datagram nagyobb, mint a kimenő interfész MTU-ja. IPv4-ben a routerek is fragmentálhatnak, míg IPv6-ban csak a küldő hoszt fragmentálhat, és a köztes csomópontok nem. A fragmentáció csökkenti a teljesítményt, ezért a modern hálózatok a Path MTU Discovery (PMTUD) mechanizmussal igyekeznek elkerülni.

19.1.12 TCP

A TCP a szállítási réteg megbízható, kapcsolatorientált protokollja. A TCP-kapcsolat egy négyes azonosítóval írható le: forrás IP, forrás port, cél IP, cél port. A kapcsolat felépítése háromfázisú kézfogással történik:

1. A kliens elküld egy SYN szegmenst egy kezdeti sorszámmal (ISN).
2. A szerver SYN-ACK szegmenssel válaszol, amely tartalmazza a saját ISN-jét és nyugtázza a kliens ISN-jét.
3. A kliens ACK szegmenssel nyugtázza a szerver ISN-jét, ezzel a kapcsolat létrejön.

A TCP-fejléc főbb mezői:

- **Forrás port (16 bit) és cél port (16 bit):** a folyamatok azonosítása.
- **Sorszám (Sequence Number, 32 bit):** a küldött adatfolyam bájtjainak sorszáma.
- **Nyugtázási szám (Acknowledgment Number, 32 bit):** a következő várt bájt sorszáma.
- **Adateltolás (4 bit):** a fejléc hossza 32 bites szavakban.
- **Jelzőbitek (9 bit):** SYN, ACK, FIN, RST, PSH, URG stb.
- **Ablakméret (16 bit):** a vevő fogadási puffermérete (folyamvezérlés).
- **Ellenőrző összeg (16 bit):** a fejléc és az adatok ellenőrzésére.
- **Opciók (változó hosszúság):** például MSS (Maximum Segment Size), SACK, timestamp.

A TCP a sorszámozással és a nyugtázásokkal biztosítja a megbízhatóságot. Ha egy szegmens elveszik vagy sérülten érkezik, a vevő nem nyugtázza, és a küldő újraküldi. A TCP csúszóablakkal vezérli a folyamot, hogy a gyors küldő ne árassza el a lassú vevőt. Ezen felül torlódásvezérlő algoritmusok (például slow start, congestion avoidance, fast retransmit, fast recovery) szabályozzák a hálózat terhelését.

19.1.13 UDP

Az UDP a szállítási réteg kapcsolat nélküli, minimális szolgáltatású protokollja. Nem végez kézfogást, nem sorszámozza az üzeneteket, és nem nyugtáz. A küldő egyszerűen elküldi a datagramot, és nem tudja meg, hogy az megérkezett-e. Az UDP-fejléc mindössze 8 bájt:

- **Forrás port (16 bit) és cél port (16 bit):** a folyamatok azonosítása.
- **Hossz (16 bit):** a fejléc és az adat együttes hossza.
- **Ellenőrző összeg (16 bit):** opcionális IPv4-ben, kötelező IPv6-ban.

Az UDP-t olyan alkalmazásokhoz használják, ahol a késleltetés kritikus, vagy ahol az alkalmazás saját megbízhatósági mechanizmust valósít meg. Például a DNS-lekérdezések általában UDP-n futnak, mert egyetlen kis kérés-válasz párról van szó, és a TCP kapcsolatfelépítés túlzott overhead lenne. A valós idejű hang- és videóátvitel is gyakran UDP-t használ, mert egy elveszett csomag nem okoz akkora problémát, mint a késleltetés.

19.1.14 ICMP

Az ICMP (Internet Control Message Protocol) az IP-hez kapcsolódó vezérlőprotokoll, amely hibajelentést és diagnosztikát szolgál. Az ICMP-üzenetek IP-datagramba ágyazva közlekednek, és nem rendelkeznek saját szállítási rétegbeli portokkal. Fontosabb ICMP-típusok:

- **Echo Request (8)** és **Echo Reply (0):** a `ping` parancs által használt üzenetek.

- **Destination Unreachable (3):** a cél nem elérhető, kódokkal jelzi az okot (hálózat, hoszt, port).
- **Time Exceeded (11):** a TTL lejárt, a `traceroute` ezt használja.
- **Redirect (5):** az útválasztó jobb útvonalat javasol.

Az ICMP-üzenetek nem tartalmaznak alkalmazási adatot, és a hálózati réteg szintjén működnek. A `ping` eszköz Echo Request üzenetet küld, és Echo Reply-t vár, ezzel méri a késleltetést és ellenőrzi az elérhetőséget.

19.1.15 Enkapszuláció és MTU

Az enkapszuláció az a folyamat, amelynek során az egyes rétegek hozzáadják a saját fejlécüket (és esetleg lábjegyzetüket) a felsőbb rétegből kapott adatokhoz. A küldő oldalon az adat lefelé halad a stackben:

1. **Alkalmazási réteg:** az alkalmazás létrehoz egy adatfolyamot vagy üzenetet, például egy HTTP GET kérést.
2. **Szállítási réteg:** a TCP (vagy UDP) hozzáadja a saját fejlécét, így jön létre a **TCP-szegmens** (vagy UDP-datagram). A fejléc tartalmazza a portokat, a sorszámot (TCP esetén), az ellenőrző összeget.
3. **Internet réteg:** az IP hozzáadja a saját fejlécét, így jön létre az **IP-datagram**. A fejléc tartalmazza a forrás és cél IP-címeket, a TTL-t és a protokoll mezőt.
4. **Adatkapcsolati réteg:** az Ethernet hozzáadja a saját fejlécét (MAC-címek, EtherType) és az FCS-t, így jön létre az **Ethernet-keret**.
5. **Fizikai réteg:** a keret bitekké alakul, és a közegen továbbítódik.

A vevő oldalon a folyamat fordított: a fizikai réteg biteket kap, az adatkapcsolati réteg ellenőrzi az FCS-t és eltávolítja az Ethernet-fejlécet, az IP-réteg feldolgozza az IP-fejlécet, a szállítási réteg a portok alapján a megfelelő folyamathoz juttatja az adatot, végül az alkalmazási réteg értelmezi az adatot.

Az alábbi ábra rétegesen mutatja az enkapszulációt:

```
Alkalmazási réteg:        HTTP adat
                             |
Szállítási réteg:        TCP fejléc + HTTP adat        -> TCP-szegmens
                             |
Internet réteg:          IP fejléc + TCP-szegmens       -> IP-datagram
                             |
Adatkapcsolati réteg:    Ethernet fejléc + IP-datagram + FCS -> Ethernet-keret
                             |
Fizikai réteg:            bitek
```

19.1.16 MTU (Maximum Transmission Unit)

Az MTU az adatkapcsolati réteg által továbbítható legnagyobb IP-datagram méretét határozza meg bájtban. Az Ethernet esetében a standard MTU 1500 bájt, ami az Ethernet-keret adatmezőjének maximális hossza. Ha egy IP-datagram nagyobb, mint a kimenő interfész MTU-ja, akkor fragmentálni kell, vagy el kell dobni a csomagot.

Az MTU rétegenként eltérő lehet. Például a PPPoE kapcsolatoknál az MTU általában 1492 bájt, mert 8 bájtnyi PPPoE-fejlécet is el kell helyezni. A VPN-tunnellek tovább csökkenthetik az MTU-t. A modern hálózatokban a Path MTU Discovery (PMTUD) mechanizmus segítségével a

küldő meghatározza a teljes útvonal legkisebb MTU-ját, és ennek megfelelő méretű datagramokat küld, elkerülve a fragmentációt.

A TCP a kapcsolat felépítésekor az MSS (Maximum Segment Size) értékét a helyi MTU-ból számítja: MSS = MTU - IP fejléc - TCP fejléc. Ethernet esetén ez tipikusan 1460 bájt (1500 - 20 - 20). Az MSS a TCP-szegmens adatrészének maximális méretét adja meg, így a teljes IP-datagram nem haladja meg az MTU-t.

Az IPv6 minimális MTU-ja 1280 bájt, és a fragmentációt csak a küldő hoszt végezheti. Ha egy IPv6-csomag nagyobb, mint az útvonal valamely hálózatának MTU-ja, a köztes router nem fragmentálja, hanem ICMPv6 Packet Too Big üzenetet küld a forrásnak, amely ezután kisebb datagramokat küld.

19.1.17 Gyakorlati példa: HTTP kérés útja a rétegeken

Tekintsünk egy egyszerű példát, amikor egy böngésző letölt egy weboldalt. A kliens (192.0.2.10) elküld egy HTTP GET kérést a szervernek (198.51.100.20). A folyamat a következőképpen zajlik:

1. **Alkalmazási réteg:** a böngésző létrehoz egy HTTP GET /index.html HTTP/1.1 kérést.
2. **Szállítási réteg:** a TCP a kérést szegmensként kezeli. Mivel a HTTP általában TCP-t használ, a böngésző először TCP-kapcsolatot létesít a szerver 80-as portjával. Ezután a GET kérést TCP-szegmensbe csomagolja, hozzáadva a forrás portot (például 49152) és a cél portot (80), a sorszámot és a nyugtázási számot.
3. **Internet réteg:** az IP hozzáadja a forrás IP-címet (192.0.2.10) és a cél IP-címet (198.51.100.20), a TTL-t (például 64) és a protokoll mezőt (6 a TCP esetén). Az IP-datagram mérete a TCP-szegmens és az IP-fejléc (20 bájt) összege.
4. **Adatkapcsolati réteg:** az Ethernet hozzáadja a forrás MAC-címet (a kliens NIC címe) és a cél MAC-címet (az alapértelmezett átjáró MAC-címe, ha a szerver másik hálózaton van). Az EtherType 0x0800 jelzi, hogy a payload IP-datagram. Az FCS-t is kiszámítja.
5. **Fizikai réteg:** az Ethernet-keret bitekként a kábelre kerül.

A vevő oldalon a szerver hálózati kártyája fogadja a keretet, ellenőrzi az FCS-t, eltávolítja az Ethernet-fejlécet, az IP-réteg feldolgozza a cél IP-címet, a TCP-réteg a portok alapján azonosítja a 80-as portot, és az alkalmazási rétegnek átadja a HTTP kérést. A szerver ezután visszaküldi a választ, hasonló enkapszulációval.

19.1.18 Tipikus hibák és helyes értelmezésük

A rétegmodellekkel kapcsolatban gyakori félreértések:

1. **Az OSI- és TCP/IP-modell összekeverése:** Sokan azt hiszik, hogy a TCP/IP-modell hét rétegből áll. Valójában a TCP/IP-modell négy (vagy öt) rétegből áll, míg az OSI-modell hét rétegből. A két modell rétegei nem feleltethetők meg egy az egyben, de a hálózati és szállítási réteg fogalma megegyezik.
2. **Az MTU és az MSS felcserélése:** Az MTU az adatkapcsolati réteg tulajdonsága, és az IP-datagram maximális méretét jelenti. Az MSS a TCP-szegmens adatrészének maximális mérete, és a TCP kapcsolatfelépítéskor egyeztetett érték. Az MSS mindig kisebb az MTU-nál, mert a TCP- és IP-fejlécek méretét is figyelembe kell venni.

3. **Az UDP teljesen megbízhatatlan:** Bár az UDP nem nyújt megbízhatósági mechanizmusokat, a gyakorlatban a hálózati réteg és az adatkapcsolati réteg hibavédelme miatt a legtöbb UDP-datagram megérkezik. Az UDP-t az alkalmazás szintjén lehet megbízhatóvá tenni (például QUIC protokoll).
4. **Az ICMP csak a pinghez való:** Bár a ping a legismertebb ICMP-alkalmazás, az ICMP számos más üzenetet is hordoz, például hibajelentéseket, TTL-lejáratot és átirányítási javaslatokat.
5. **Az enkapszuláció sorrendjének felcserélése:** A küldő oldalon a fejlécek hozzáadása felülről lefelé történik, a vevő oldalon pedig alulról felfelé. Gyakori hiba, hogy valaki azt hiszi, a TCP-fejléc az IP-fejléc fölé kerül, pedig éppen fordítva: az IP-fejléc a TCP-szegmenst csomagolja be.

19.1.19 Labor: csomagút és rétegfejek vázlata

Ez a labor a rétegek és az enkapszuláció gyakorlati megfigyelését szolgálja emulált környezetben. A cél egy egyszerű ping-csomag útjának nyomon követése a rétegeken, a rétegfejek méretének és tartalmának rögzítése, valamint az RTT mérése.

19.1.20 Kiinduló állapot

- Friss, tiszta emulált környezet (például QEMU vagy Linux hálózati névterek).
- Két hoszt, amelyek IP-címei: 192.0.2.1/24 és 192.0.2.2/24.
- A hosztok egy virtuális bridge-en keresztül kapcsolódnak, nincs külső hálózati hozzáférés.
- Csomagfogó eszköz (tcpdump vagy tshark) telepítve.

19.1.21 Lépések

1. Hozz létre két hálózati névteret és egy bridge-et:

```
ip netns add ns1
ip netns add ns2
ip link add veth1 type veth peer name veth1-br
ip link add veth2 type veth peer name veth2-br
ip link set veth1 netns ns1
ip link set veth2 netns ns2
ip link add br0 type bridge
ip link set veth1-br master br0
ip link set veth2-br master br0
ip link set veth1-br up
ip link set veth2-br up
ip link set br0 up
ip netns exec ns1 ip addr add 192.0.2.1/24 dev veth1
ip netns exec ns2 ip addr add 192.0.2.2/24 dev veth2
ip netns exec ns1 ip link set veth1 up
ip netns exec ns2 ip link set veth2 up
```

2. Indíts csomagfogást a bridge interfészen:

```
tcpdump -i br0 -n -e -ttt -w /tmp/capture.pcap &
```

3. Küldj egy ICMP Echo Request csomagot az ns1-ből az ns2-be:

```
ip netns exec ns1 ping -c 1 192.0.2.2
```

4. Állítsd le a csomagfogást, és elemezd a fogott kereteket:

```
tcpdump -r /tmp/capture.pcap -n -e -ttt
```

5. Rajzold le a rétegfejeket a fogott keret alapján. Az alábbi vázlatot kell kitöltened a tényleges méretekkel:

```
Ethernet fejléc: cél MAC (6 bájt) | forrás MAC (6 bájt) | EtherType (2 bájt)
IP fejléc: verzió/IHL (1 bájt) | ToS (1 bájt) | teljes hossz (2 bájt) | ... | forrás IP (4 bájt)
↪  | cél IP (4 bájt)
ICMP fejléc: típus (1 bájt) | kód (1 bájt) | ellenőrző összeg (2 bájt) | azonosító (2 bájt) |
↪  sorszám (2 bájt)
ICMP adat: 56 bájt (a ping alapértelmezett payloadja)
```

6. Mérd az RTT-t a `ping` kimenetéből. A mért értéket jegyezd fel.

19.1.22 Mérési napló

Mérési pont	Várt érték	Mért érték
RTT (ms)	< 1 ms (emulált hálózaton)	[mérendő]
Fogott keretek száma	2 (Echo Request és Echo Reply)	[mérendő]
Ethernet fejléc hossza	14 bájt	[mérendő]
IP fejléc hossza	20 bájt (IPv4, opciók nélkül)	[mérendő]
ICMP fejléc hossza	8 bájt	[mérendő]
Teljes keret hossza	$14 + 20 + 8 + 56 = 98$ bájt	[mérendő]

A mérési körülmények: QEMU 7.2.0, 2 vCPU, 512 MiB RAM, e1000 NIC, hoszt operációs rendszer Linux 6.1 (ellenőrzési igény: a pontos verzió a labor futtatásakor ellenőrizendő). Az RTT mérési módszere: `ping` által kiírt idő, ezredmásodperc pontossággal.

19.1.23 Visszaállítás

A labor után a hálózati névtereket és a bridge-et el kell távolítani:

```
ip netns del ns1
ip netns del ns2
ip link del br0
```

19.1.24 Értelmezés

A fogott keretekben megfigyelhető, hogy az Ethernet-fejléc 14 bájt, az IP-fejléc 20 bájt, az ICMP-fejléc 8 bájt, a payload pedig 56 bájt. A teljes keret hossza 98 bájt, ami jóval az MTU (1500 bájt) alatt van, így nincs fragmentáció. Az RTT a helyi bridge-en alacsony, általában $<$ 1 ms, de a pontos érték az emulációs környezettől függ. A mért és várt értékek közötti eltérés elemzése segít megérteni a rétegek overheadjét.

19.1.25 Összefoglalás

Ebben az alfejezetben áttekintettük az OSI- és TCP/IP-modellt, a rétegek feladatait, valamint az IP, TCP, UDP és ICMP protokollok szerepét. Láttuk, hogyan működik az enkapszuláció, és miért fontos az MTU a fragmentáció elkerülésében. A laborban egy egyszerű ping-csomag útját követtük nyomon, és meghatároztuk a rétegfejek méreteit. A következő alfejezet a socket API-t és a hálózati rendszerhívásokat tárgyalja, amelyekre ez a rétegmodell épül.

Önellenőrző kérdés: Magyarázza el, mi történik, ha egy 2000 bájtos IP-datagramot küldünk egy Ethernet interfészen, amelynek MTU-ja 1500 bájt IPv4, illetve IPv6 esetén. Hogyan befolyásolja ezt a Path MTU Discovery?

19.2 Socket API és hálózati rendszerhívások

19.2.1 1. Bevezetés: socket mint interfész

A socket az operációs rendszer által biztosított programozási interfész, amelyen keresztül egy felhasználói térben futó folyamat hálózati kommunikációt folytathat. A socket fogalma a Berkeley Software Distribution (BSD) 4.2-ben jelent meg 1983-ban, és azóta gyakorlatilag minden operációs rendszer átvette – a POSIX szabvány rögzíti a legfontosabb rendszerhívásokat, a Linux és a Windows pedig saját kiterjesztésekkel egészíti ki.

A folyamat szemszögéből a socket egy látszólagos végpont, amely hasonlóan viselkedik, mint egy fájl: van hozzá tartozó leíró (descriptor), lehet rá írni, és lehet róla olvasni. Ez a látszat azonban fontos eltéréseket takar. Egy fájlleíróval ellentétben a socket nem egy tárolt adatsorozatot reprezentál, hanem egy kommunikációs csatornát két – esetleg különböző gépen futó – végpont között. Az írás nem garantálja, hogy a túloldal azonnal megkapja az adatot; az olvasás pedig nem biztosítja, hogy az elérhető bájtok pontosan megegyeznek azzal, amit egyetlen írásban küldtünk.

A socketfogalom mögött rejlő absztrakció három rétegre bontható:

1. **Címzési réteg**: a socket egy hálózati címhez köthető. IPv4 esetén ez egy 32 bites cím és egy 16 bites port, IPv6 esetén 128 bites cím és port. A címcsalád (address family, `AF_*`) határozza meg a cím formátumát.
2. **Szemantikai réteg**: a socket típusa (`SOCK_STREAM`, `SOCK_DGRAM`, `SOCK_RAW`) határozza meg, hogy az adatfolyam megbízható, sorrendhelyes, kapcsolatalapú-e, vagy datagram-alapú, csomaghatárokat megőrző.
3. **Protokollréteg**: az adott típuson belül választható konkrét protokoll (`IPPROTO_TCP`, `IPPROTO_UDP`, `IPPROTO_SCTP`).

Egy socketet tehát a `(címcsalád, típus, protokoll)` hármas azonosít. Nem minden kombináció értelmes – például az `AF_INET` címcsaládban a `SOCK_STREAM` típushoz a TCP, a `SOCK_DGRAM` típushoz az UDP a természetes társítás. A Linux kernel a `net/socket.c` fájlban valósítja meg az általános socketréteget, a konkrét protokollcsaládok pedig saját modulokban (például `net/ipv4/`) regisztrálják műveleteiket.

19.2.2 2. A socketéletciklus rendszerhívásai

A klasszikus BSD socket API rendszerhívásai a következő életciklust fedik le: létrehozás, címadás, kapcsolódás vagy figyelés, adatátvitel, lezárás. A TCP-kiszolgáló és a TCP-kliens életciklusa eltérő szekvenciát követ; az UDP kommunikáció kapcsolatmentessége miatt rövidebb utat jár be.

19.2.3 2.1 `socket()` – létrehozás

```
int socket(int domain, int type, int protocol);
```

A `socket()` rendszerhívás egy új socketleírót hoz létre. A `domain` a címcsalád (`AF_INET`, `AF_INET6`, `AF_UNIX`, `AF_NETLINK`), a `type` a kommunikáció szemantikája (`SOCK_STREAM`, `SOCK_DGRAM`, `SOCK_RAW`, `SOCK_SEQPACKET`), a `protocol` pedig adott típushoz tartozó konkrét protokoll – általában `0`, ekkor a rendszer az alapértelmezettet választja. A visszatérési érték egy nemnegatív egész leíró, hibánál `-1`, az `errno` pedig például `EAFNOSUPPORT` (nem támogatott címcsalád), `EPROTONOSUPPORT` (nem támogatott protokoll) vagy `EMFILE` (nincs több szabad leíró).

Fontos megérteni, hogy a `socket()` önmagában csak egy belső adatstruktúrát hoz létre a kernelben – a `struct socket` és a hozzá kapcsolódó protokollspecifikus struktúrák láncolatát. A socket még nincs hálózathoz kötve, nincs címe, és nem folytat semmilyen kommunikációt. Ez a létrehozás a „telefonvonal bekötése a lakásba, de a készüléket nem csatlakoztattuk” állapothoz hasonlít.

A Linux kernelben a `socket()` hívás a `__sys_socket()` függvénybe torkollik, amely a `sock_create()` segítségével lefoglalja a `struct socket` objektumot, majd a megfelelő protokollcsalád `create()` metódusát hívja. Az `AF_INET` és a `SOCK_STREAM` kombináció például a TCP protokoll `tcp_v4_init_sock()` inicializálásához vezet.

19.2.4 2.2 `bind()` – a lokális végpont címének rögzítése

```
int bind(int sockfd, const struct sockaddr *addr, socklen_t addrlen);
```

A `bind()` a sockethez rendeli a helyi címet – IP-címet és portszámot, vagy Unix-domain socketnél a fájlrendszerbeli elérési utat. A `sockaddr` egy generikus struktúra, amelyet a konkrét címcsaládnak megfelelő struktúrákra kell vetíteni:

```
struct sockaddr_in {
    sa_family_t    sin_family;   /* AF_INET */
    in_port_t      sin_port;     /* port, hálózati bájtsorrendben */
    struct in_addr sin_addr;     /* IPv4 cím */
};
```

A `bind()` tipikus hibája az `EADDRINUSE`, amikor a kért portot már foglalja egy másik socket – ez különösen gyakori, ha a kiszolgálót gyorsan újraindítjuk, és a korábbi kapcsolatok még `TIME_WAIT` állapotban vannak. A megoldás a `SO_REUSEADDR` socketopció beállítása, amely megengedi a port újrafelhasználását, amíg a régi kapcsolatok lezárulnak:

```
int optval = 1;
setsockopt(sockfd, SOL_SOCKET, SO_REUSEADDR, &optval, sizeof(optval));
```

A `bind()` a TCP-kiszolgálóknál kötelező lépés, mert az ügyfeleknek pontosan ismerniük kell a kiszolgáló portszámát. TCP-kliensnél az `AF_INET` címcsaládban a `bind()` elhagyható: a kernel a `connect()` során automatikusan választ egy szabad, efemer portot a helyi címtartományból (Linuxon tipikusan 32768–60999). UDP-kiszolgálónál szintén kötelező a `bind()`; UDP-kliensnél az első `sendto()` előtt a kernel automatikusan hozzárendel egy efemer portot.

19.2.5 2.3 `listen()` és `accept()` – TCP-kiszolgáló oldal

```
int listen(int sockfd, int backlog);
int accept(int sockfd, struct sockaddr *addr, socklen_t *addrlen);
```

A `listen()` átalakítja a socketet „figyelő” állapotba: ezután a kernel elfogadja az erre a portra érkező TCP-kapcsolatfelvételi kérelmeket (SYN csomagokat), és egy várakozási sorba helyezi őket. A `backlog` paraméter a várakozási sor maximális hossza. Érdemes megérteni, hogy a Linux kernelben a backlog két különböző sor méretét befolyásolja: a `SYN_RECV` állapotú, még nem teljesen kiépült kapcsolatokét (a kernel `tcp_max_syn_backlog` értéke korlátozza), és a már kiépült, de az alkalmazás által még nem `accept()`-elt kapcsolatokét (amelyre a `listen()` `backlog` paramétere vonatkozik). Az `accept()` blokkolva várakozik, amíg egy kiépült kapcsolat meg nem érkezik, majd visszaad egy új socketleírót, amely kizárólag az adott kapcsolat adatátvitelére szolgál.

Ez a kettéválasztás kulcsfontosságú: a „listening socket” csak kapcsolatfelvételt fogad, magán az adatátvitelen nem vesz részt. Minden elfogadott kapcsolat saját, külön leírót kap, saját kernelbeli pufferrel és saját életciklussal. A kiszolgáló így párhuzamosan több ügyféllel képes kommunikálni.

```
struct sockaddr_in client_addr;
socklen_t client_len = sizeof(client_addr);
int client_fd = accept(server_fd, (struct sockaddr *)&client_addr, &client_len);
if (client_fd == -1) {
    perror("accept");
    /* EINTR esetén újrapróbálható, EMFILE esetén erőforrásprobléma */
}
```

19.2.6 2.4 `connect()` – a kliens oldal

```
int connect(int sockfd, const struct sockaddr *addr, socklen_t addrlen);
```

A `connect()` a TCP-kliensnél elindítja a háromutas kézfogást: a kernel elküldi a SYN csomagot, megvárja a SYN-ACK-et, majd elküldi az ACK-ot. A hívás blokkoló változata csak akkor tér vissza, ha a kapcsolat kiépült, hibára futott, vagy az időkorlát lejárt. A visszatérési hibák diagnosztikai értéke jelentős:

- `ECONNREFUSED`: a célgépen nem figyel senki a kért porton, vagy a kiszolgáló RST csomaggal válaszolt.
- `ETIMEDOUT`: nincs válasz, például tűzfal dobja el a csomagokat.
- `ENETUNREACH`: a kernel nem talál útvonalat a célhálózathoz.

UDP-socketen a `connect()` teljesen más szerepet tölt be: nem hoz létre kapcsolatot a hálózaton, hanem csupán rögzíti a socket társított címét. Ezáltal az `send()` használható az `sendto()` helyett, és a `recv()` csak az adott társtól érkező datagramokat fogadja el – a többit a kernel eldobja. Ez a „connected UDP socket” minta gyakori, mert egyszerűbbé teszi a hibakezelést és a demultiplexálást.

19.2.7 2.5 Adatátviteli hívások

A socketre írás és olvasás több rendszerhívással lehetséges. Az alap `write()` és `read()` hívás mellett a socketekre specializált változatok állnak rendelkezésre:

```
ssize_t send(int sockfd, const void *buf, size_t len, int flags);
ssize_t recv(int sockfd, void *buf, size_t len, int flags);
ssize_t sendto(int sockfd, const void *buf, size_t len, int flags,
               const struct sockaddr *dest_addr, socklen_t addrlen);
ssize_t recvfrom(int sockfd, void *buf, size_t len, int flags,
                 struct sockaddr *src_addr, socklen_t *addrlen);
```

A `flags` paraméter a viselkedést finomhangolja: `MSG_DONTWAIT` nem blokkoló műveletet kér egyetlen hívásra, `MSG_PEEK` az adatot a pufferben hagyva olvas, `MSG_WAITALL` a `recv()`-nél arra utasítja a kernelt, hogy a teljes kért hosszt várja meg, ne pedig az első elérhető részlettel térjen vissza.

A TCP folyamalapú szemantikája miatt a `recv()` visszatérési értéke nem felel meg az `send()` hívások határainak. Ha a küldő 100 bájtot ír, a fogadó kaphat egyetlen 60 bájtos `recv()` eredményt, utána egy másikat 40 bájttal; vagy akár két 100 bájtos írás eredménye egyetlen 200 bájtos olvasásban egyesülhet. A helyes alkalmazáslogika ezért hossz-előtagot (length-prefix) vagy explicit keretezést (framing) alkalmaz – például sorvég alapú protokollt, fix méretű üzenetfejlécet, vagy TLV (Type-Length-Value) kódolást. A „TCP csomag” kifejezés használata az alkalmazásszintű üzenetre félrevezető: a TCP szegmensekre bont, és a vevő oldalon az adatfolyam összefüggő bájtsorozatként jelenik meg.

Az UDP ezzel szemben megőrzi az üzenethatárokat: minden `sendto()` egy datagramot eredményez, és a `recvfrom()` pontosan egy datagram tartalmát adja vissza. Ha a fogadó puffere kisebb, mint a datagram, a többlet elveszik. Az UDP nem garantálja sem a kézbesítést, sem a sorrendet, sem a duplikáció elkerülését – ezt az alkalmazásnak kell kezelnie, ha szükséges.

19.2.8 2.6 `close()` és kapcsolatbontás

A socketet a `close()` rendszerhívással zárjuk le. A kernel a referenciaszámlálást alkalmazza: ha ugyanarra a socketleíróra `dup()` vagy `fork()` révén több leíró mutat, a `close()` csak az adott leírót szünteti meg, a socket csak az utolsó leíró bezárásakor szabadul fel.

A TCP kapcsolatbontás négylépéses folyamat (FIN-FIN-ACK-ACK). A `close()` elküldi a FIN csomagot; a túloldal ACK-kal nyugtázza, majd a saját FIN-jét küldi. A kapcsolat lezárása után a socket `TIME_WAIT` állapotba kerül, amely 2MSL (maximális szegmens-élettartam kétszerese, gyakorlatban 60 másodperc) ideig tart. Ennek célja a késleltetett duplikált csomagok elleni védelem és a megbízható lezárás biztosítása. A `SO_LINGER` opcióval módosítható a viselkedés, de ennek használata körültekintést igényel.

19.2.9 3. I/O modellek: blokkoló, nem blokkoló és aszinkron

A socketen végzett műveletek időzítését három fő modell határozza meg. A választás alapvetően befolyásolja az alkalmazás architektúráját, a hibakezelést és a skálázhatóságot.

19.2.10 3.1 Blokkoló I/O

Az alapértelmezett viselkedés: a `recv()`, `accept()` és `connect()` blokkolja a hívó folyamatot, amíg az esemény be nem következik. A folyamat futási állapota `TASK_RUNNING`-ból `TASK_INTERRUPTIBLE`-be vált, a kernel ütemezője pedig más folyamatot futtat. Amikor az adat megérkezik, a hálózati megszakításkezelő és a softirq-feldolgozás felébreszti a várakozó folyamatot.

A blokkoló modell előnye az egyszerűség: a kód lineárisan olvasható, a hibakezelés egyértelmű. Hátránya, hogy egy folyamat egyszerre csak egy socketen tud várakozni; több ügyfél kiszolgálásához több folyamat vagy szál kell. Minden blokkoló folyamat külön kernelverem-lapot, felhasználói veremlapot és ütemezési entitást foglal – a memória- és kontextusváltási költség nagy terhelésnél jelentőssé válik.

19.2.11 3.2 Nem blokkoló I/O

A socket `O_NONBLOCK` jelzővel nem blokkoló üzemmódba állítható:

```
int flags = fcntl(sockfd, F_GETFL, 0);
fcntl(sockfd, F_SETFL, flags | O_NONBLOCK);
```

Nem blokkoló módban a `recv()` azonnal visszatér, az eredmény a következő lehet:

- pozitív: adat érkezett,
- `0`: a túloldal rendezetten lezárta a kapcsolatot (TCP FIN),
- `-1` `errno=EAGAIN/EWOULDBLOCK`: nincs elérhető adat, a műveletet később újra kell próbálni,
- `-1` egyéb `errno`: valódi hiba.

A nem blokkoló modell nem azt jelenti, hogy az alkalmazás szorosan ciklikusan hívogatja a `recv()`-et – ez a busy polling, amely pazarló. A gyakorlati minta az eseménykezelő hurkok használata: `select()`, `poll()`, `epoll()` (Linux) vagy `kqueue()` (BSD). Ezekkel a folyamat egyszerre több socketet regisztrálhat, és a kernel értesíti, ha valamelyik olvashatóvá vagy írhatóvá válik. Az `epoll` a nagy skálázhatósága miatt a modern Linux-kiszolgálók alapvető építőköve:

```
int epfd = epoll_create1(0);
struct epoll_event ev = { .events = EPOLLIN, .data.fd = client_fd };
epoll_ctl(epfd, EPOLL_CTL_ADD, client_fd, &ev);
```

19.2.12 3.3 Aszinkron I/O

Az aszinkron I/O modellben az alkalmazás kezdeményezi a műveletet, majd a kernel értesíti a befejezésről – a hívás és az adatfelhasználás között nincs blokkolás és nincs aktív várakozás sem. A POSIX AIO és a Linux `io_uring` interfész ilyen modellt valósít meg. A `io_uring` különösen hatékony, mert a beküldési és befejezési gyűrűpufferek osztott memórián keresztül kommunikálnak, csökkentve a rendszerhívási költséget.

A három modell közötti döntés nem pusztán teljesítménykérdés. A blokkoló modell kis terhelésnél és egyszerű protokolloknál megfelelő; a nem blokkoló + eseményhurok a közepes és nagy terhelésű hálózati szolgáltatásoknál a bevált megoldás; az aszinkron modell pedig akkor indokolt, ha a műveletek száma igen magas, és a rendszerhívási költség mérhető szűk keresztmetszet.

19.2.13 4. Gyakori hibák és diagnosztikájuk

A socketprogramozás hibái gyakran nem a rendszerhívások szintjén, hanem a protokollszemantika félreértésében gyökereznek. Az alábbiakban a leggyakoribb hibamintákat és azok okait tekintjük át.

19.2.14 4.1 A TCP folyamhatárok figyelmen kívül hagyása

A legelterjedtebb hiba, amikor a fejlesztő feltételezi, hogy egy `send()` pontosan egy `recv()`-nek felel meg. A hálózati réteg szegmentálhatja az adatot, a fogadó puffer mérete változhat, és az ütemezés is befolyásolja, hogy a kernel mennyi adatot ad át egyszerre. A megoldás az explicit alkalmazásszintű keretezés: a fogadó mindig tudja, hány bájtot várjon (a fejlécben érkező hossz alapján), vagy milyen határolójel zárja az üzenetet.

A másik oldal a rövid írás (short write): az `send()` visszatérési értéke kisebb lehet a kért hossznál, különösen nem blokkoló módban, ha a kernel küldő puffere megtelt. A helyes kód ciklusban ír, és a hátralévő bájtokat újra elküldi. Hasonlóan, a `recv()` is visszatérhet kevesebb bájttal, mint amennyit a hívó kért.

19.2.15 4.2 Sorrend és párhuzamosság kezelése

Az `accept()` után a kiszolgálónak döntenie kell, hogyan kezeli a párhuzamos kapcsolatokat. A leggyakoribb megoldások a folyamat-alapú (`fork()` minden kapcsolatra), a szál-alapú (egy szál kapcsolatonként), az eseményhurok-alapú (egy szál, nem blokkoló socketek, `epoll`), és a hibrid (több eseményhurok, több szálon). Az `accept()`-et érdemes központi helyen tartani, és a kapcsolatokat azonnal átadni a feldolgozó egységeknek; a kernel a várakozási sort kezeli, de az alkalmazásnak kell biztosítania, hogy a feldolgozás ne legyen szűk keresztmetszet.

19.2.16 4.3 UDP hibakezelés

Az UDP-vel kapcsolatos tipikus félreértés, hogy „ha a küldés sikeres, a csomag megérkezett”. Az `sendto()` visszatérési értéke csak azt jelzi, hogy a datagram bekerült a kernel küldő pufferébe – nem azt, hogy a hálózaton átment, vagy hogy a túloldal megkapta. A `recvfrom()` nem ad hibát, ha a datagram elveszett, duplikálódott vagy sorrendben később érkezett. A kapcsolt UDP socketnél a `recv()` az idegen társtól érkező datagramokat csendben eldobja, és az `ICMP port unreachable` hibát is csak akkor jelzi, ha a socket connected állapotban van.

19.2.17 4.4 Erőforrás-kezelés

A socketleírók véges erőforrást jelentenek – a Linuxon az `RLIMIT_NOFILE` korlátozza a folyamatonként nyitott leírók számát. A hosszú életű kiszolgálóknál a leírók szivárgása a rendszer kimerüléséhez vezet. A helyes minta: minden leírót egyértelmű tulajdonosi körbe kell rendelni, és a hibautakon is le kell zárni. C nyelvben a `goto cleanup` minta praktikus, mert a hibakezelést egy helyre gyűjti:

```
int server_fd = -1, client_fd = -1;
server_fd = socket(AF_INET, SOCK_STREAM, 0);
if (server_fd == -1) goto cleanup;
/* ... */
cleanup:
if (client_fd != -1) close(client_fd);
```

```
if (server_fd != -1) close(server_fd);
return rc;
```

19.2.18 4.5 Jelzések és `EINTR`

A blokkoló rendszerhívások megszakíthatók szignálok által – ilyenkor a hívás `-1`-gyel tér vissza, az `errno` pedig `EINTR`. A POSIX szemantika szerint a `connect()` megszakítása után a kapcsolat kiépítése a háttérben folytatódhat; a socket nem használható addig, amíg a kapcsolat állapota nem rendeződik. A `recv()` és `accept()` esetén az `EINTR` általában egyszerűen újrapróbálható. A Linux kernelben a `sigaction()` `SA_RESTART` jelzőjével egyes rendszerhívások automatikusan újraindulnak, de a `connect()`, `accept()` és `recvfrom()` viselkedése protokollfüggő lehet.

19.2.19 5. Labor: TCP- és UDP-kliens/szerver vázlat

A labor célja egy minimális, de működőképes TCP-kiszolgáló, TCP-kliens, UDP-kiszolgáló és UDP-kliens megvalósítása, emulált környezetben. A kód a szemléltetést szolgálja; a futtatáshoz a könyv laborcsomagjában megadott, reprodukálható környezet szükséges.

19.2.20 5.1 Környezet és konfiguráció

- **Emuláció**: QEMU, Linux kernel 6.1 LTS (a verziószám a laborcsomagban pontosítandó)
- **CPU**: 2 vCPU, x86_64
- **Memória**: 512 MiB
- **Hálózat**: belső, izolált virtuális hálózat (`-netdev user` vagy `-netdev socket`), a hoszt és külső hálózati interfész nem érintett
- **Fordítás**: GCC 12, `-O2`, `-Wall -Wextra -D_GNU_SOURCE`
- **Címkiosztás**: kiszolgáló `127.0.0.1:4200` (TCP) és `127.0.0.1:4201` (UDP), helyőrző címek

19.2.21 5.2 TCP-kiszolgáló vázlat

```
/* tcp_server.c - illusztratív vázlat, nem teljes termelési kód */
#define _GNU_SOURCE
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <unistd.h>
#include <stdio.h>
#include <string.h>
#include <errno.h>

#define PORT 4200

int main(void) {
    int server_fd = -1, client_fd = -1;
    struct sockaddr_in addr = {0};
    char buf[256];
    ssize_t n;

    server_fd = socket(AF_INET, SOCK_STREAM, 0);
    if (server_fd == -1) {
        perror("socket");
        return 1;
```

```
    }

    int optval = 1;
    setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR, &optval, sizeof(optval));

    addr.sin_family = AF_INET;
    addr.sin_port = htons(PORT);
    addr.sin_addr.s_addr = htonl(INADDR_LOOPBACK);

    if (bind(server_fd, (struct sockaddr *)&addr, sizeof(addr)) == -1) {
        perror("bind");
        goto cleanup;
    }
    if (listen(server_fd, 8) == -1) {
        perror("listen");
        goto cleanup;
    }
    printf("TCP kiszolgáló figyel: 127.0.0.1:%d\n", PORT);

    client_fd = accept(server_fd, NULL, NULL);
    if (client_fd == -1) {
        perror("accept");
        goto cleanup;
    }

    n = recv(client_fd, buf, sizeof(buf) - 1, 0);
    if (n == -1) {
        perror("recv");
        goto cleanup;
    }
    buf[n] = '\0';
    printf("fogadott: %s\n", buf);

cleanup:
    if (client_fd != -1) close(client_fd);
    if (server_fd != -1) close(server_fd);
    return 0;
}
```

A példa a legegyszerűbb életciklust mutatja be: `socket()` → `bind()` → `listen()` → `accept()` → `recv()` → `close()`. A `SO_REUSEADDR` beállítás a gyors újraindítást teszi lehetővé. Az `accept()` egyetlen kapcsolatot fogad, nem hurkol – ezt a labor során a hallgató bővíti a saját változatában.

19.2.22 5.3 TCP-kliens vázlat

```
/* tcp_client.c - illusztratív vázlat */
#define _GNU_SOURCE
#include <sys/socket.h>
#include <netinet/in.h>
#include <unistd.h>
#include <stdio.h>
#include <string.h>

int main(void) {
    int fd = socket(AF_INET, SOCK_STREAM, 0);
    if (fd == -1) {
        perror("socket");
        return 1;
    }
```

```
    struct sockaddr_in addr = {0};
    addr.sin_family = AF_INET;
    addr.sin_port = htons(4200);
    addr.sin_addr.s_addr = htonl(INADDR_LOOPBACK);

    if (connect(fd, (struct sockaddr *)&addr, sizeof(addr)) == -1) {
        perror("connect");
        close(fd);
        return 1;
    }

    const char *msg = "hello";
    ssize_t n = send(fd, msg, strlen(msg), 0);
    printf("elküldve: %zd bájt\n", n);
    close(fd);
    return 0;
}
```

19.2.23 5.4 UDP-példa

Az UDP-változat a `sendto()` és `recvfrom()` hívásokat használja, nincs `listen()` és `accept()`. A kiszolgáló a `bind()` után közvetlenül a datagramot fogadja:

```
/* udp_server.c – illusztratív vázlat */
#define _GNU_SOURCE
#include <sys/socket.h>
#include <netinet/in.h>
#include <unistd.h>
#include <stdio.h>

int main(void) {
    int fd = socket(AF_INET, SOCK_DGRAM, 0);
    struct sockaddr_in addr = {0};
    char buf[256];
    ssize_t n;

    addr.sin_family = AF_INET;
    addr.sin_port = htons(4201);
    addr.sin_addr.s_addr = htonl(INADDR_LOOPBACK);

    if (bind(fd, (struct sockaddr *)&addr, sizeof(addr)) == -1) {
        perror("bind");
        close(fd);
        return 1;
    }

    n = recvfrom(fd, buf, sizeof(buf) - 1, 0, NULL, NULL);
    buf[n] = '\0';
    printf("fogadott datagram: %s\n", buf);
    close(fd);
    return 0;
}
```

19.2.24 5.5 Mérési napló és várt eredmények

A mérést az emulált környezetben, a fenti konfigurációval kell elvégezni, a tényleges futtatási értékeket a laborcsomagban rögzíteni. Az alábbi táblázat a várt jellemzőket foglalja össze, amelyeket a mért értékektől világosan el kell különíteni.

Mérési pont	Módszer	Várt jellemző (levezetett)	Mért érték
TCP kapcsolatfelépítés (localhost)	`clock_gettime(CLOCK_MONOTONIC)` a `connect()` körül	40–100 µs loopback interfészen	[mérendő]
TCP egyszeri üzenet átvitele	`send()`/`recv()` időmérés	< 1 ms, 5 bájtos üzenet	[mérendő]
UDP datagram küldése	`sendto()` hívás ideje	< 1 ms, a kernelpufferbe írás	[mérendő]
UDP datagram fogadása	`recvfrom()` várakozási ideje	azonnali, a datagram már a pufferben	[mérendő]

Fontos: a mért értékek függenek az emulátor órajelétől, a kernelverziótól és a CPU-terheléstől. A laborjegyzetben a hardverkonfigurációt és a fordítási beállításokat minden mérésnél fel kell tüntetni. A loopback interfészen mért latencia nem reprezentatív a valós hálózati késleltetésre.

19.2.25 5.6 Ellenőrző kérdések a laborhoz

1. Mi történik, ha a kliens a kiszolgáló indítása előtt fut? Melyik rendszerhívás és milyen hibakóddal tér vissza?
2. Hogyan változik a TCP-kiszolgáló viselkedése, ha az `accept()` után azonnal bezárjuk a kliens socketet? Mit tapasztal a kliens a `recv()` hívásánál?
3. UDP esetén miért nem kap a kliens hibát, ha a kiszolgáló nem fut?
4. Módosítsuk a kiszolgálót nem blokkoló socketre, és figyeljük meg az `EAGAIN` viselkedését egy eseményhurokban. Milyen feltételek mellett tér vissza a `recv()` `-1`-gyel és `EAGAIN`-nel?

19.2.26 6. Összefoglalás és továbbvezetés

A socket API réteges, de koherens modellt ad a hálózati kommunikációhoz. A `socket()` létrehozza az absztrakt végpontot, a `bind()` rögzíti a helyi címet, a `listen()`/`accept()` és a `connect()` a kapcsolatfelépítés két oldalát valósítja meg, az adatátviteli hívások pedig a TCP folyamszemantikáját vagy az UDP datagram-szemantikáját közvetítik a felhasználói térbe. A blokkoló, nem blokkoló és aszinkron modellek közötti választás meghatározza az alkalmazás szerkezetét és skálázhatóságát.

A következő egység (Csomagfeldolgozás, netfilter és eBPF) ezekre a fogalmakra építve vizsgálja meg, hogyan dolgozza fel a kernel a csomagokat a socketréteg alatt: a netfilter-horog, az eBPF és a csomag-útválasztás belülről nézve.

19.2.27 Önellenőrző kérdések

1. Hogyan különbözik a `connect()` jelentése TCP- és UDP-socketen?
2. Miért nem garantálja a TCP, hogy a `recv()` egy `send()`-ben küldött adatmennyiséget ad vissza?
3. Mi a szerepe a `backlog` paraméternek a `listen()` hívásban, és milyen két várakozási sort érint a Linux kernelben?
4. Mikor használjuk a `SO_REUSEADDR` opciót, és miért segít elkerülni az `EADDRINUSE` hibát?
5. Mi a különbség a nem blokkoló I/O és az aszinkron I/O között az alkalmazás szemszögéből?

19.3 Csomagfeldolgozás, netfilter és eBPF

A hálózati csomagok feldolgozása az operációs rendszer egyik legkritikusabb útvonala. Egy csomag beérkezésétől a kézbesítésig, illetve egy kimenő csomag esetén a socketből a hálózati kártyáig számos kernelkomponens vesz részt. Ebben a fejezetben azt vizsgáljuk, hol és hogyan lehet beavatkozni ebbe a folyamatba: hogyan működik a netfilter csomagszűrő keretrendszere, hogyan követi a connection tracking a kapcsolatok állapotát, és hogyan ad az eBPF és az XDP lehetőséget a lehető legkorábbi, rugalmas csomagfeldolgozásra.

A fejezet előfeltételezi a socket API és a hálózati rendszerhívások alapjait (Socket API és hálózati rendszerhívások). A cél az alapfogalmak és a gyakorlati minták együttes bemutatása, valamint egy ellenőrizhető tűzfal- és csomagszűrési szabályterv elkészítése. A példák Linux kernel 5.x/6.x és nftables 1.x környezetet feltételeznek; a pontos verziófüggő szintaxist a helyi kézikönyvben ellenőrizni kell.

19.3.1 A kernel hálózati útvonala

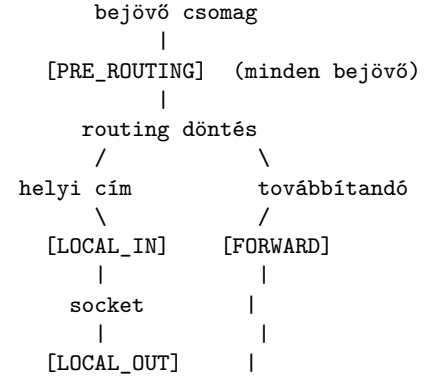
A bejövő (ingress) csomag útja a hálózati kártyától a felhasználói folyamatig több fázisból áll. Az egyszerűség kedvéért a modern Linux kernelben a következő lépések figyelhetők meg:

1. A hálózati kártya (NIC) megszakítást vált ki, vagy a kernel a NAPI (New API) mechanizmussal pollozza a csomaggyűrűt. A feldolgozás softirq kontextusban (NET_RX) történik.
2. A meghajtó létrehoz egy `sk_buff` struktúrát, amely a csomag metaadatait (protokoll, bejövő interfész, időbélyeg, fejlécmutatók) és magát a puffert tartalmazza.
3. Az illesztőprogram meghívja a `netif_receive_skb()` függvényt, amely a csomagot átadja a protokoll-veremnek: Ethernet, IP, majd TCP/UDP.
4. A netfilter hookok a feldolgozási útvonal meghatározott pontjain vannak elhelyezve. A hookok előtt és után a kernel további feldolgozást végez, például routing döntést.

A netfilter öt alapvető hookpontot definiál:

- `NF_INET_PRE_ROUTING`: minden bejövő csomag, közvetlenül a routing döntés előtt.
- `NF_INET_LOCAL_IN`: a helyi gépre szánt csomagok, a routing döntés után.
- `NF_INET_FORWARD`: a továbbítandó (forwardolt) csomagok, a routing döntés után.
- `NF_INET_LOCAL_OUT`: a helyi folyamat által generált kimenő csomagok, a routing döntés előtt.
- `NF_INET_POST_ROUTING`: minden kimenő csomag, közvetlenül a hálózati kártyára adás előtt.

A hookok sorrendjét az alábbi folyamatábra szemlélteti:



```
    \          /
[POST_ROUTING]  (minden kimenő)
         |
    kimenő csomag
```

A routing döntés a cél IP-cím alapján határozza meg, hogy a csomagot a helyi gépnek kell-e kézbesíteni, vagy továbbítani kell egy másik interfészre. A `PRE_ROUTING` hookban a csomag még az eredeti címeket hordozza; a `POST_ROUTING` hookban viszont már a kimenő interfész és a végső címek ismertek. Ez a sorrend kulcsfontosságú a szűrés és a NAT szempontjából.

A kimenő csomag útja hasonló: a socket rétegből indul, áthalad a `LOCAL_OUT` hookon, majd a `POST_ROUTING` hookon, végül a meghajtó elküldi a csomagot. A `LOCAL_OUT` azért fontos, mert itt még a helyi folyamat által generált forgalmat is meg lehet szűrni, például egy rosszindulatú program kimenő kapcsolatait.

19.3.2 Netfilter hookok és szűrés

A netfilter a Linux kernel csomagszűrő és -manipuláló keretrendszere. A hookokhoz úgynevezett **táblák** (table) és **láncok** (chain) csatlakoznak. A tábla határozza meg, hogy milyen típusú műveleteket végezhetnek a szabályok: szűrés (filter), címfordítás (nat), csomagmódosítás (mangle), kapcsolatkövetés kikapcsolása (raw), biztonsági házirend (security). A lánc egy adott hookponthoz kötött szabálylista, amelyet a kernel a csomag feldolgozása során sorban kiértékel.

A klasszikus **iptables** eszközben a `filter` tábla három láncot tartalmaz: `INPUT`, `FORWARD`, `OUTPUT`. A `nat` tábla láncai: `PREROUTING`, `INPUT`, `OUTPUT`, `POSTROUTING`. A `mangle` tábla mind az öt hookponton jelen lehet. A láncokhoz tartozik egy **alapértelmezett szabály** (policy), amelyet akkor alkalmaz a kernel, ha egyetlen szabály sem illeszkedik a csomagra. A policy lehet `ACCEPT` vagy `DROP`.

Egy iptables szabály két részből áll: **illeszkedési feltétel** (match) és **cél** (target). A feltétel lehet például a protokoll (`-p tcp`), a forrás vagy cél IP, a port (`--dport`), az interfész (`-i eth0`), vagy egy modul által biztosított komplexebb vizsgálat, például a kapcsolat állapota (`-m state --state ESTABLISHED`). A cél határozza meg, mi történjen a csomaggal: `ACCEPT` (továbbenged), `DROP` (csendben eldob), `REJECT` (eldob és hibaüzenetet küld), `LOG` (naplózza), `DNAT`, `SNAT`, `MASQUERADE` stb.

Egy tipikus, állapotalapú bejövő szűrést iptables-ben a következő parancsokkal lehet felépíteni:

```
# Alapértelmezés: mindent eldobunk, amit nem engedélyezünk
iptables -P INPUT DROP
iptables -P FORWARD DROP
iptables -P OUTPUT ACCEPT

# A már kialakult kapcsolatok visszatérő csomagjait engedjük
iptables -A INPUT -m state --state ESTABLISHED,RELATED -j ACCEPT

# A loopback interfész forgalmát mindig engedjük
iptables -A INPUT -i lo -j ACCEPT

# SSH bejövő forgalom engedélyezése
iptables -A INPUT -p tcp --dport 22 -m state --state NEW -j ACCEPT

# Minden más bejövő csomagot eldobunk és naplózunk
iptables -A INPUT -j LOG --log-prefix "input-drop: "
iptables -A INPUT -j DROP
```

A szabályok sorrendje kritikus: a kernel az első illeszkedő szabály célját alkalmazza, és nem vizsgálja tovább a láncot. Ha a `-P INPUT DROP` után rögtön egy engedő szabályt helyezünk el, az nem érvényesül, mert a policy csak akkor lép életbe, ha nincs találat. Ezért a gyakori forgalmat engedő szabályokat előre kell tenni, a tiltást pedig a lánc végére.

A modern **nftables** a netfilter új felülete, amely az iptables, ip6tables, arptables és ebtables funkcióit egyesíti. A szintaxis egységes, a szabálykészlet atomikusan frissíthető, és a teljesítmény is jobb, mert a kernel egyetlen belső reprezentációt használ. Az nftables-ben a táblák és láncok explicit módon jönnek létre, a hook és a priority a lánc definíciójában szerepel.

19.3.3 Connection tracking (conntrack)

A connection tracking (magyarul kapcsolatkövetés) a netfilter azon alrendszere, amely a csomagok alapján nyomon követi a hálózati kapcsolatokat. Nem szűrési mechanizmus, hanem olyan **állapotinformációt** szolgáltat, amelyre a szűrő és a NAT szabályok építhetnek. A conntrack minden kapcsolathoz egy bejegyzést hoz létre a kernelben, amely tartalmazza a kapcsolat azonosítóit (protokoll, forrás IP, forrás port, cél IP, cél port – az úgynevezett 5-tuple) és az aktuális állapotot.

A conntrack által használt állapotok:

- **NEW**: a csomag új kapcsolatot kezdeményez, például a TCP SYN csomag.
- **ESTABLISHED**: a csomag egy már létező, mindkét irányban felismert kapcsolathoz tartozik.
- **RELATED**: a csomag egy másik, már meglévő kapcsolathoz kapcsolódik, például az FTP adatkapcsolat vagy egy ICMP hibaüzenet.
- **INVALID**: a csomag nem illeszthető be egyetlen ismert kapcsolatba sem, például sérült fejléc vagy lejárt bejegyzés.

Az állapotalapú szűrés alapelve: engedjük a kimenő forgalmat, és a bejövő forgalomból csak azokat a csomagokat fogadjuk el, amelyek egy általunk kezdeményezett kapcsolat visszatérő csomagjai, illetve az ezekhez kapcsolódó csomagok. Ez gyakorlatilag az `ESTABLISHED,RELATED` állapotok elfogadását jelenti a bejövő láncban, míg az új kapcsolatokat csak a kívánt szolgáltatásokra (például SSH, HTTP) engedjük.

A conntrack szoros kapcsolatban áll a NAT-tal. Amikor a NAT szabály módosítja a forrás vagy cél címet, a conntrack bejegyzés eltárolja az eredeti és a leképezett cím közötti megfeleltetést. Így a visszairányú csomag automatikusan visszafordítható anélkül, hogy külön szabályt kellene írni. A conntrack nélkül a NAT nem tudná, melyik belső géphez tartozik egy visszatérő csomag.

A conntrack működéséhez a `nf_conntrack` kernelmodul betöltésére van szükség. A modul betöltése után a kernel automatikusan követi a kapcsolatokat. A conntrack tábla mérete korlátozott; ha megtelik, az új kapcsolatok `INVALID` állapotba kerülhetnek, és a szabályok nem a várt módon működnek. A tábla mérete a `nf_conntrack_max` rendszerparaméterrel állítható, a telítettség pedig a `/proc/net/nf_conntrack` vagy a `conntrack` eszköz segítségével ellenőrizhető.

19.3.4 NAT és masquerade

A NAT (Network Address Translation, hálózati címfordítás) a csomagok forrás vagy cél IP-címének (és gyakran a portjának) megváltoztatása. Két fő típusa van:

- **SNAT** (Source NAT): a forrás címet cseréli le, tipikusan akkor, amikor egy belső hálózat gépei egy nyilvános címen keresztül kommunikálnak az internettel. A válaszcsomagot a conntrack automatikusan visszafordítja.
- **DNAT** (Destination NAT): a cél címet cseréli le, tipikusan porttovábbítás (port forwarding) esetén, amikor egy külső címre érkező forgalmat egy belső szerverre irányítunk.

A **MASQUERADE** (álcázás) az SNAT speciális változata, amely nem igényel rögzített nyilvános IP-címet, hanem a kimenő interfész aktuális IP-címét használja. Ez különösen hasznos DHCP-vel konfigurált, változó IP-című kapcsolatok esetén, például otthoni routereknél.

Egy tipikus NAT-konfiguráció iptables-ben:

```
# Kimenő forgalom álcázása az eth0 interfészen
iptables -t nat -A POSTROUTING -o eth0 -j MASQUERADE

# Bejövő 80-as port átirányítása a belső 192.0.2.10-es szerverre
iptables -t nat -A PREROUTING -i eth0 -p tcp --dport 80 \
    -j DNAT --to-destination 192.0.2.10:80
```

A NAT és a szűrés együttműködésekor figyelembe kell venni a hookok sorrendjét. A `PREROUTING` hookban a DNAT már megtörténik, mielőtt a routing döntés és a `FORWARD` vagy `INPUT` lánc lefut. Ezért a szűrő szabályokban a **DNAT utáni** célcímet kell használni. Például ha a fenti DNAT szabály a 80-as portot a 192.0.2.10-es szerverre irányítja, akkor a `FORWARD` láncban a 192.0.2.10-es címre kell engedélyezni a forgalmat, nem a külső címre.

19.3.5 nftables a gyakorlatban

Az nftables a modern Linux rendszerekben az ajánlott tűzfal- és NAT-eszköz. A konfigurációt deklaratív módon, táblákba és láncokba szervezve lehet leírni. Az alábbi példa egy állapotalapú tűzfalat és NAT-ot mutat be egy átjáró (gateway) szerepű gépen:

```
table inet filter {
    chain input {
        type filter hook input priority 0; policy drop;
        ct state established,related accept
        iif "lo" accept
        tcp dport 22 accept
        counter drop
    }

    chain forward {
        type filter hook forward priority 0; policy drop;
        ct state established,related accept
        iifname "eth1" oifname "eth0" accept
        counter drop
    }

    chain output {
        type filter hook output priority 0; policy accept;
    }
}

table ip nat {
    chain postrouting {
        type nat hook postrouting priority 100; policy accept;
        oifname "eth0" masquerade
    }

    chain prerouting {
```

```
        type nat hook prerouting priority -100; policy accept;
        iifname "eth0" tcp dport 8080 dnat to 192.0.2.10:80
    }
}
```

A fenti szabálykészlet a következőket teszi:

- A bejövő forgalomból csak a már kialakult kapcsolatokhoz tartozó csomagokat fogadja el, valamint a loopback forgalmat és az SSH-t (22/TCP).
- A továbbított forgalomból csak a belső hálózatból (`eth1`) a külső interfész (`eth0`) felé haladó új kapcsolatokat engedi, a visszatérő csomagokat a conntrack állapota alapján.
- A kimenő forgalmat nem korlátozza.
- A kimenő forgalmat a `POSTROUTING` láncban álcázza az `eth0` interfész IP-címére.
- A bejövő 8080-as portot a belső 192.0.2.10-es gép 80-as portjára irányítja.

A szabálykészlet betöltése például a következő paranccsal történhet:

```
nft -f /etc/nftables.conf
```

Az nftables előnye, hogy a teljes szabálykészlet egyetlen atomikus művelettel cserélhető, így a hibás konfiguráció nem hagy nyitott időablakot. A szabályok ellenőrzésére a `nft list ruleset` parancs szolgál.

19.3.6 eBPF és XDP

Az eBPF (extended Berkeley Packet Filter) egy olyan virtuális gép a Linux kernelben, amely biztonságos, felügyelt módon hajt végre felhasználó által megírt programokat. Az eBPF programokat C nyelvhez hasonló szintaxissal lehet megírni, majd clang segítségével bpf bytecode-ra fordítani. A kernel **verifier** (ellenőrző) komponense a betöltés előtt statikusan elemzi a programot: ellenőrzi, hogy a program nem tartalmaz végtelen ciklust, minden memória-hozzáférés határellenőrzött, és a program nem fér hozzá tiltott kernelmemóriához. A programok ezután biztonságosan futhatnak kernelkontextusban.

A csomagfeldolgozás szempontjából az eBPF két fontos belépési pontot kínál:

1. **XDP** (eXpress Data Path): a lehető legkorábbi pont, közvetlenül a hálózati kártya meghajtójában, még az `sk_buff` allokáció előtt. Az XDP program a nyers csomagpufferhez fér hozzá, és dönthet a csomag sorsáról. Mivel a feldolgozás a driverben történik, az overhead minimális.
2. **tc** (traffic control) hook: a hálózati verembe már bekerült csomagokra alkalmazható, mind bejövő (ingress), mind kimenő (egress) irányban. A tc később fut, mint az XDP, de rugalmasabb, mert a teljes kernel-hálózati stack rendelkezésére áll.

Az XDP program visszatérési értékei:

- `XDP_DROP`: a csomagot eldobja.
- `XDP_PASS`: a csomagot továbbítja a hagyományos hálózati verembe.
- `XDP_TX`: a csomagot visszaküldi ugyanazon az interfészen.
- `XDP_REDIRECT`: a csomagot egy másik interfészre irányítja.
- `XDP_ABORTED`: hibát jelez (általában nem használt).

Az alábbi egyszerű XDP program egy adott forrás IP-címről érkező csomagokat dob el:

```c
#include <linux/bpf.h>
#include <bpf/bpf_helpers.h>
#include <linux/if_ether.h>
#include <linux/ip.h>

SEC("xdp_drop_ip")
int xdp_drop_ip(struct xdp_md *ctx) {
    void *data_end = (void *)(long)ctx->data_end;
    void *data = (void *)(long)ctx->data;

    struct ethhdr *eth = data;
    if ((void *)(eth + 1) > data_end)
        return XDP_PASS;

    struct iphdr *ip = (struct iphdr *)(eth + 1);
    if ((void *)(ip + 1) > data_end)
        return XDP_PASS;

    // Példa: 192.0.2.10 forrás cím eldobása
    __be32 drop_ip = bpf_htonl(0xc000020a); // 192.0.2.10
    if (ip->saddr == drop_ip)
        return XDP_DROP;

    return XDP_PASS;
}

char _license[] SEC("license") = "GPL";
```

A programot a következőképpen lehet lefordítani és betölteni (emulált környezetben):

```
clang -O2 -target bpf -c xdp_drop_ip_kern.c -o xdp_drop_ip_kern.o
ip link set dev eth0 xdp obj xdp_drop_ip_kern.o sec xdp_drop_ip
```

Az `ip link set ... xdp` parancs az `eth0` interfészhez csatolja a programot. A betöltést a kernel verifier ellenőrzi, és csak akkor engedi meg, ha a program megfelel a biztonsági követelményeknek. A program a `bpf_htonl()` segédfüggvényt használja a bájtsorrend átalakítására; a `XDP_DROP` és `XDP_PASS` értékeket a `bpf_helpers.h` definiálja.

Az eBPF programok gyakran **map**-eket használnak, amelyek a felhasználói tér és a kernel között osztanak meg adatokat. Például egy XDP program egy map-ből olvashatja ki a tiltott IP-címek listáját, így a szűrés dinamikusan frissíthető anélkül, hogy a programot újra kellene fordítani. A map-ek létrehozása és kezelése a `bpf()` rendszerhíváson keresztül történik, de a gyakorlatban a libbpf könyvtár vagy a `bpftool` eszköz egyszerűsíti a folyamatot.

A tc hookhoz csatolt eBPF programok hasonlóak, de a belépési pont a `tc` osztályozó (classifier) vagy akció (action) mechanizmusa. A tc hook lehetővé teszi a kimenő forgalom szűrését is, ami XDP-vel nem megoldható.

19.3.7 Gyakori hibák és buktatók

A csomagfeldolgozás és a szűrés során a következő tipikus hibákat érdemes elkerülni:

1. **Hook sorrend félreértése.** Gyakori hiba, hogy a fejlesztő a `PREROUTING` hookban szeretné eldobni a helyi gépre szánt csomagot, de a szabályt a `FORWARD` láncba teszi, vagy fordítva. A hookok sorrendjének ismerete nélkül a szabályok nem a várt ponton avatkoznak be.
2. **Conntrack állapot hiánya.** Ha a conntrack modul nincs betöltve, az állapotalapú szabályok nem működnek, mert minden csomag `INVALID` állapotba kerül. A `nf_conntrack` modul betöltése és a szabályok `ct state` használata előtt ellenőrizni kell a modul állapotát.

3. **NAT és szűrés interakciója.** A DNAT után a szűrő láncokban a módosított célcímet kell használni, nem az eredetit. Ha a szabályt a külső címre írják, a forgalom nem lesz engedélyezve, vagy éppen tiltva marad.

4. **eBPF verifier korlátok.** Az eBPF programokban minden memória-hozzáférést határellenőrzéssel kell védeni, különben a verifier elutasítja a betöltést. A program nem tartalmazhat bizonyítottan végtelen ciklust, és nem hívhat tetszőleges kernel függvényeket, csak engedélyezett helpereket.

5. **Teljesítmény félreértése.** Az XDP valóban alacsony overheadet kínál, de csak a bejövő forgalomra. A tc hook később fut, és nagyobb a feldolgozási költsége. A szabályok sorrendje a netfilterben is számít: a gyakran illeszkedő szabályokat előre kell tenni, különben minden csomag végigszalad a láncon.

6. **Tesztelés hiánya.** A tűzfalszabályokat rendszeresen tesztelni kell, mert egy rosszul megírt szabály akár a teljes hálózati forgalmat blokkolhatja, vagy éppen nyitva hagyhat egy szolgáltatást. A laborban ezért hangsúlyos a mért és várt eredmények elkülönítése.

19.3.8 Labor: tűzfal- és csomagszűrési szabályterv

A labor célja egy állapotalapú tűzfal és NAT szabályterv elkészítése, tesztelése és dokumentálása emulált környezetben. A feladat nem éles rendszeren fut, hanem hálózati névterekben vagy virtuális gépen.

Kiinduló állapot. Hozzunk létre három hálózati névteret: `ns1` (belső kliens), `ns2` (belső szerver) és `rtr` (router). A névtereket veth párokkal kötjük össze: `ns1` a `rtr` `eth1` interfészéhez, `ns2` a `rtr` `eth2` interfészéhez kapcsolódik. A `rtr` rendelkezik egy `eth0` interfésszel, amely a külvilágot szimulálja. A címek a dokumentációs tartományból származnak: `ns1` IP-címe `192.0.2.10/24`, `ns2` IP-címe `192.0.2.20/24`, a `rtr` `eth1` címe `192.0.2.1/24`, `eth2` címe `192.0.2.2/24`, `eth0` címe `198.51.100.1/24`. A `ns1` és `ns2` alapértelmezett átjárója a `rtr` megfelelő interfészcíme. A névterekben a szükséges szolgáltatások: `ns2`-n SSH (22/TCP) és HTTP (80/TCP) szerver fut, `ns1`-ről ezeket kívánjuk elérni, de más forgalmat tiltani.

Szabályterv. A `rtr` névtérben nftables szabálykészletet hozunk létre az alábbi követelményekkel:

- A `ns1` csak a `ns2` SSH (22/TCP) és HTTP (80/TCP) szolgáltatásait érheti el.
- A `ns2` kimenő forgalma az `eth0` irányába engedélyezett, és a `POSTROUTING` láncban masquerade szabályt alkalmazunk.
- A `rtr` saját bejövő forgalma közül csak a már kialakult kapcsolatok és a loopback engedett.
- Minden más továbbított forgalom eldobandó.

A szabálykészlet nftables szintaxissal:

```
table inet filter {
    chain input {
        type filter hook input priority 0; policy drop;
        ct state established,related accept
        iif "lo" accept
    }

    chain forward {
        type filter hook forward priority 0; policy drop;
        ct state established,related accept
        iifname "eth1" oifname "eth2" ip protocol tcp tcp dport { 22, 80 } accept
        iifname "eth2" oifname "eth0" accept
    }
```

```
    chain output {
        type filter hook output priority 0; policy accept;
    }
}

table ip nat {
    chain postrouting {
        type nat hook postrouting priority 100; policy accept;
        oifname "eth0" masquerade
    }
}
```

Tesztelés és mérési napló. A szabályterv betöltése után a következő teszteket futtatjuk le, és az eredményeket táblázatban rögzítjük:

Teszt	Várt eredmény	Mért eredmény	Megjegyzés
`ns1` → `ns2` SSH (22/TCP)	kapcsolat létrejön	sikeres bejelentkezés	conntrack NEW + ACCEPT
`ns1` → `ns2` HTTP (80/TCP)	HTTP 200 válasz	HTTP 200	conntrack NEW + ACCEPT
`ns1` → `ns2` egyéb port (pl. 3306)	csomag eldobva, timeout	timeout	policy drop
`ns2` → külső cím (`198.51.100.10`) ICMP	válasz érkezik	0% csomagvesztés	masquerade működik
`ns1` → külső cím	eldobva	timeout	nincs engedélyező szabály
`rtr` bejövő SSH	eldobva	timeout	input policy drop, nincs engedély

A ping késleltetést is megmérjük szűrés nélkül és szűréssel, hogy lássuk a netfilter overheadjét. A mérési naplóban külön oszlopban jelöljük a mért és a várt értéket. Például:

- Várt késleltetés szűrés nélkül: < 1 ms
- Mért késleltetés szűrés nélkül: 0,3 ms
- Várt késleltetés szűréssel: < 1 ms
- Mért késleltetés szűréssel: 0,4 ms

Visszaállítás. A labor végén a szabályokat töröljük, a névtereket megszüntetjük:

```
nft flush ruleset
ip netns del ns1
ip netns del ns2
ip netns del rtr
ip link del veth1 2>/dev/null
ip link del veth2 2>/dev/null
```

Mérőszámok. A laborban a következő mérőszámokat rögzítjük: a szabályok száma, a csomagvesztés aránya, a késleltetés növekedése a szűrés hatására, és a tesztelt esetek száma.

19.3.9 Önellenőrző kérdések

1. Nevezze meg a netfilter öt hookpontját, és írja le, hogy egy bejövő, helyi címre szánt csomag milyen sorrendben halad át rajtuk.
2. Miért van szükség a connection trackingre a NAT működéséhez? Mi történik, ha a conntrack modul nincs betöltve?
3. Milyen visszatérési értékeket adhat egy XDP program, és mi a különbség az `XDP_DROP` és az `XDP_PASS` között?

Ezek a kérdések a fejezet tanulási céljait ellenőrzik: a kernel hálózati útvonalának és a szűrésnek a megértését, a gyakorlati eszközök alkalmazását, valamint a tipikus hibák felismerését.

19.4 Zero-copy, offload és hálózati teljesítmény

19.4.1 1. A hálózati adatút optimalizálása: a csomag útja és a másolások problémája

Az előző alfejezetben (Csomagfeldolgozás, netfilter és eBPF) láttuk, hogy egy beérkező csomag a hálózati kártyától (NIC) a megszakításon, a DMA-n és a kernel hálózati veremén (TCP/IP stack) keresztül jut el a felhasználói alkalmazásig. Ez az útvonal az általános célú operációs rendszerekben hagyományosan többszöri memóriamásolással és kontextusváltással jár, ami a modern, nagy sebességű hálózatokon (10 Gbit/s, 25 Gbit/s, 100 Gbit/s) komoly szűk keresztmetszetté válik.

A naiv adatút a következőképpen néz ki egy csomag fogadásakor:

1. A DMA-vezérlő a hálózati kártya gyűrűpufferéből a kernel `sk_buff` struktúrába másolja a csomag adatait.
2. A kernel a `socket` rétegbe másolja az adatot.
3. Az alkalmazás meghívja a `read()` rendszerhívást, amelynek során a CPU a kernelpufferből a felhasználói térben lévő puffferbe másolja az adatot.

Ez a hagyományos modell három-négy CPU-végrehajtású másolást is jelenthet csomagonként, ami jelentősen terheli a CPU-t, rontja a cache kihasználtságát, növeli a késleltetést, és csökkenti az elérhető maximális sávszélességet.

A hálózati adatút optimalizálásának célja kettős:

- **CPU-teher csökkentése:** A CPU-nak a csomagok másolgatása helyett az alkalmazáslogikával kellene foglalkoznia.
- **Késleltetés és áteresztőképesség javítása:** Kevesebb másolás és kontextusváltás közvetlenül jobb latency-t és nagyobb throughputot eredményez.

Az optimalizálás három fő pilléren nyugszik: a **zero-copy technikák**, a **hardveres offload** és a **CPU-affinitás / NUMA-tudatos elhelyezés**.

19.4.2 2. Zero-copy technikák a rendszerhívásokban

A "zero-copy" kifejezés némileg félrevezető: nem azt jelenti, hogy egyáltalán nincs másolás a memóriában, hanem azt, hogy a **CPU nem végez felesleges másolást** a felhasználói és a kernelcímterek között. A cél, hogy a memóriabuszon és a CPU L1/L2/L3 cache-én keresztül történő adatmozgatások száma minimális legyen.

19.4.2.1 2.1 `sendfile()` és a kernel belső másolása

A legegyszerűbb és legelterjedtebb zero-copy minta a fájlok kiszolgálása a hálózaton (pl. webkiszolgáló). A hagyományos megoldás a `read()` majd `write()`:

```
/* Hagyományos, nem hatékony másolás */
char buffer[16384];
ssize_t n;
int in_fd = open("fajl.bin", O_RDONLY);
int out_fd = socket(AF_INET, SOCK_STREAM, 0);

// Itt a fájl adata: lemez -> kernel page cache -> buffer (user space) -> socket kernel puffer
↪  -> NIC
while ((n = read(in_fd, buffer, sizeof(buffer))) > 0) {
    if (write(out_fd, buffer, n) != n) {
        perror("write");
        break;
    }
}
```

Itt az adat négyszer kerül át a memórián a CPU aktív közreműködésével. A `sendfile()` rendszerhívás ezt a műveletet egyetlen hívássá olvasztja össze:

```
/* Zero-copy adatküldés a kernelben maradva */
off_t offset = 0;
ssize_t sent = sendfile(out_fd, in_fd, &offset, file_size);
```

A `sendfile` működése során a kernel közvetlenül a **page cache**-ből tölti be a fájl adatait, és azt a socket pufferbe helyezi. Habár ez még mindig egy memóriamásolás a kernel címtartományán belül (page cache -> socket buffer), a CPU-nak nem kell átlépnie a kernel/user határt kétszer, és nem kell a felhasználói pufferbe másolnia. A modern Linux kernelekben a `sendfile` képes arra is, hogy a socket bufferbe ne a teljes adatot másolja, hanem csak egy leírót (descriptor) illesszen be, ami a háttérben a page cache lapjaira mutat (ez a `splice` és a hálózati alrendszer együttműködése).

19.4.2.2 2.2 `splice()`

A `splice()` rendszerhívás egy általánosabb kernel-belső adatmozgató mechanizmus. Két fájlleíró között mozgat adatot anélkül, hogy az elhagyná a kernelteret. Az egyik végpontnak általában egy pipe-nak kell lennie.

```
/* adat mozgatása fájlból socketbe splice-szal */
int pipefd[2];
pipe(pipefd);

// Fájl -> pipe
ssize_t len = splice(in_fd, NULL, pipefd[1], NULL, file_size, SPLICE_F_MOVE);
// Pipe -> socket
splice(pipefd[0], NULL, out_fd, NULL, len, SPLICE_F_MOVE);
```

Itt a rendszer a lapokat (pages) csak átcímkézi (re-map), nem pedig másolja, amennyiben a hardver és a kernel támogatja a scatter-gather DMA-t.

19.4.2.3 2.3 `MSG_ZEROCOPY` (Linux-specifikus)

A fenti példák főként a bejövő adatokra és a lemezes forrásokra vonatkoznak. Ha az alkalmazás a saját memóriájából generált adatot akar küldeni, hagyományosan a `send()` rendszerhívás bemásolja a felhasználói puffert a kernel socket puffferébe. A Linux 4.14 óta elérhető `MSG_ZEROCOPY` flag ezt kerüli meg:

```
/* Kísérleti jellegő, verziófüggő API (ellenőrzés szükséges a célkernelre) */
int one = 1;
setsockopt(fd, SOL_SOCKET, SO_ZEROCOPY, &one, sizeof(one));

char *buf = mmap(NULL, size, PROT_READ | PROT_WRITE,
                 MAP_PRIVATE | MAP_ANONYMOUS, -1, 0);
// ... adat feltöltése a buf-ba ...

struct msghdr msg = {0};
struct iovec iov = { .iov_base = buf, .iov_len = size };
msg.msg_iov = &iov;
msg.msg_iovlen = 1;

ssize_t sent = sendmsg(fd, &msg, MSG_ZEROCOPY);
```

Ebben az esetben a kernel a hálózati kártya DMA-képességét használva közvetlenül a felhasználói térben lévő lapokról olvassa be az adatot, ha a hardveres scatter-gather ezt lehetővé teszi. Fontos megjegyezni, hogy ez a módszer akkor hatékony, ha az adat nagyobb, mint a másolás költsége. Kis csomagoknál a lapok rögzítésének (pinning) overheadje nagyobb lehet, mint a másolásé.

Tipikus hiba és helyes megoldás: A kezdő fejlesztők gyakran alkalmaznak zero-copy megoldásokat kicsi, pár száz bájtos üzenetekre. Ebben az esetben a rendszer a lapok lefoglalásával és rögzítésével (page pinning) több munkát végez, mintha egyszerűen átmásolná a 64 bájtot a `copy_to_user`/`copy_from_user` művelettel. A helyes megoldás a **döntési táblázat** használata: zero-copy csak akkor, ha a payload mérete meghalad egy küszöbértéket (jellemzően 1-4 KiB), vagy ha a CPU a szűk keresztmetszet.

19.4.3 3. Hardveres offload: DMA, stateless offload és TOE

Az operációs rendszernek nem kell minden feladatot szoftveresen elvégeznie. A hálózati kártyák (NIC-ek) egyre több intelligens funkcióval rendelkeznek. Az offload célja, hogy a CPU helyett a hardver végezze el az ismétlődő, alacsony szintű munkát.

19.4.3.1 3.1 DMA (Direct Memory Access) és Descriptor gyűrűk

A DMA a hálózati adatút legalapvetőbb offload mechanizmusa. Programozott I/O (PIO) esetén a CPU minden bájtot egyesével olvasna ki a kártyáról. A DMA lehetővé teszi, hogy a kártya közvetlenül a rendszermemóriába írjon (RX) és onnan olvasson (TX).

A kernel és a NIC közötti együttműködés **descriptor gyűrűkkel** (ring buffer) történik:

- **TX Ring:** A kernel lefoglal egy gyűrűt, amelyben a memóriacímeket és hosszakat adja meg. A kernel létrehoz egy `sk_buff`-ot, majd a fizikai címét beírja a TX ring következő elemébe. A NIC DMA-val elolvassa ezt a mutatót, kiolvassa az adatot a RAM-ból, és elküldi a hálózatra.
- **RX Ring:** A kernel előre lefoglal üres puffereket, és azok címeit beírja az RX ringbe. Amikor csomag érkezik, a NIC DMA-val beleír a megadott memóriacímre, majd megszakítást (IRQ) küld.

A **scatter-gather DMA** az a hardveres képesség, amely a zero-copy technikák alapja. Ha a TCP stream adatai nem folytonos fizikai memóriában vannak, a CPU-nak külön be kellene másolnia őket egy folytonos pufferbe, hogy a hálózati kártya el tudja küldeni. Ha a NIC támogatja a scatter-gather-t, a kernel egyszerűen átad egy láncolt listát (iovec) a fizikai címekről, és a NIC hardvere szétszórva olvassa be azokat, összerakva a keretet a hálózaton.

19.4.3.2 3.2 Stateless offload: Checksum, TSO, LRO/GRO

A TCP/IP protokoll több ellenőrző összeget és szegmentálási feladatot ír elő, amelyeket hagyományosan a CPU számolt ki.

- **Checksum Offload (CSUM):** Az IP és TCP fejléc ellenőrző összegének kiszámítása nagy CPU-terhelést jelent nagy sávszélességnél. Ha a NIC hardvere támogatja, a kernel beírja a fejlécbe a pszeudo-fejléc adatait, és a kártya kiszámítja a végleges checksumot. Ez tipikusan mindig bekapcsolt állapotban marad, mert a nyereség hatalmas, a kockázat pedig alacsony.
- **TCP Segmentation Offload (TSO):** A kernel hálózati verme és az alkalmazás gyakran nagy (akár 64 KiB-os) pufferekkel dolgozik, de a fizikai hálózat MTU-ja 1500 bájt. Ha a CPU-nak kellene feldarabolnia a nagy TCP szegmenseket, az rengeteg apró művelet lenne. TSO esetén a kernel átadja a nagy puffert a kártyának, és a **hálózati hardver** végzi el a szegmentálást a kicsi IP-csomagokra. Ez drámaian csökkenti a per-bájt CPU-költséget.
- **Large/Generic Receive Offload (LRO/GRO):** A TSO ellentéte. A hálózaton sok kicsi, egymáshoz tartozó szegmens érkezik. Ha a kernel mindegyiket külön dolgozná fel, óriási overhead lenne. A GRO a szoftveres, de az alsóbb rétegben (NAPI) elvégzett aggregáció: a megszakításkezelő összegyűjti az egymáshoz tartozó szegmenseket, és egyetlen nagy `sk_buff`-ként adja fel a TCP veremnek. (A régebbi LRO ezt hardveresen tette, de az gyakran megsértette a TCP szabályokat, különösen a routing és a tűzfalak alatt, ezért a Linux inkább a szoftveres GRO-t preferálja).

19.4.3.3 3.3 TOE (TCP Offload Engine)

A TOE a "teljes" offload megvalósítása. Itt a hálózati kártya nem csak a checksum-ot számolja, vagy a szegmentálást végzi, hanem a **teljes TCP state machine-t** lefuttatja a hardverben (háromfázisú handshake, torlódásvezérlés, nyugtázás, retransmission).

Előny: Az elméleti CPU-teher a nullához közelíthet, hiszen a hoszt CPU-ja szinte nem vesz részt a TCP feldolgozásában. **Hátrány és jelenlegi helyzet:** A modern, nagysebességű hálózatokban az általános célú operációs rendszerek (különösen a Linux) **nem használnak teljes TOE-t**. Ennek okai:

1. **Biztonság és tűzfal (`netfilter`, eBPF):** Ha a TCP handshake és a kapcsolatállapot a hardverben él, akkor a kernel tűzfala (pl. iptables, nftables) és az eBPF programok nem látják a kapcsolat állapotát, így nem tudnak beavatkozni (pl. SYN cookie, DDoS védelem).

2. **Rugalmasság és debuggolás:** A kernel TCP verme (pl. CUBIC, BBR torlódásvezérlés) folyamatosan fejlődik. A hardverbe égetett implementációk elavulnak, nem frissíthetők, és nehezen debuggolhatók.
3. **Virtualizáció és live migration:** Ha a TCP állapot a hardverben van, lehetetlen egy virtuális gépet élőben átköltöztetni (live migration) egy másik szerverre, mert a kapcsolat állapota nem exportálható.

Ezért a gyakorlatban a **stateless offload (TSO, CSUM, RSS)** és az intelligens DMA a nyerő stratégia, a teljes TOE-t pedig főleg speciális beágyazott környezetekben vagy régebbi eszközökön találjuk.

19.4.4 4. Latency, Throughput és CPU-affinitás

Az optimalizálás célja a mérőszámok javítása, tehát meg kell értenünk a hálózati teljesítmény metrikáit és a CPU-architektúra hatását.

- **Throughput (Áteresztőképesség):** Az időegység alatt átvitt hasznos adat mennyisége (Mbit/s vagy csomag/s). A throughput növelése főként a nagy másolások kiküszöbölésén (zero-copy) és az offload-on (TSO/LRO) múlik.
- **Latency (Késleltetés):** Az az idő, ami alatt egy csomag eljut a forrástól a célig (vagy a végponttól a végpontig az alkalmazásban). A latency csökkentése gyakran a megszakítások késleltetésének csökkentésén és az ütemezés optimalizálásán múlik.

19.4.4.1 4.1 CPU-affinitás és RSS (Receive Side Scaling)

A többmagos rendszerekben a hálózati adatút egyik legnagyobb ellensége a **cache thrashing** és a **lock contention**. Ha egy csomag megszakítása a 0. CPU-n fut le, a felhasználói alkalmazás pedig a 4. CPU-n, akkor az adatot a cache-koherencia protokollnak (MESI) át kell mozgatnia a processzorok között, ami jelentős késleltetést ad hozzá.

Receive Side Scaling (RSS): A hálózati kártyák támogatják a hardveres terheléselosztást. Az RSS a TCP/IP fejléc (IP-címek és portok) hash-e alapján a bejövő csomagokat több RX ringbe osztja szét. Minden RX ringhez külön megszakítási vektor tartozhat, amelyet a kernel egy adott CPU-hoz köthet.

Interrupt Coalescing és Busy Polling: A nagy sebességű hálózatoknál a megszakítások áradata (interrupt storm) miatt a CPU soha nem jut el az alkalmazás futtatásáig. Két stratégia létezik:

1. **Interrupt Coalescing (Adaptive RX/TX):** A kártya nem küld megszakítást minden csomag után, hanem vár egy kicsit (néhány mikroszekundum), vagy vár, amíg N csomag összegyűlik. Ez növeli a batch méretet és növeli a throughputot, de növeli a késleltetést is! Ez a klasszikus latency vs. throughput trade-off.
2. **Busy Polling (`SO_BUSY_POLL`):** Alacsony késleltetésű környezetekben az alkalmazás nem alszik el a `recv` hívásra várva, hanem a kernel aktívan pörög egy CPU magon, és pollozza a hálózati kártya RX gyűrűjét. Ez drasztikusan csökkenti a latency-t (akár 10-20 mikroszekundumra), de 100%-os CPU-terhelést jelent.

CPU-affinitás beállítása a laborban: A `taskset` vagy a `cpuset` cgroup segítségével az alkalmazást ugyanarra a NUMA node-ra és CPU magra kötjük, mint ahol a NIC megszakítása fut.

```
# Példa: IRQ affinity beállítása (emulált labor kontextusban)
# Az IRQ 123-t a 2-es CPU-hoz kötjük
echo 2 > /proc/irq/123/smp_affinity_list

# Az alkalmazást a 2-es CPU-hoz és a 2-es NUMA node-hoz kötjük
taskset -c 2 numactl --membind=2 ./halozati_app
```

A CPU-affinitás csökkenti a lock contention-t, biztosítja, hogy az adat az L2/L3 cache-ben marad, és kiszámíthatóbbá teszi a késleltetést.

19.4.5 5. Labor: Zero-copy és offload döntési táblázat

Környezet: Emulált labor QEMU-ban, `virtio-net` hálózati kártyával. Rögzített konfiguráció: 4 vCPU, 8 GiB RAM, Linux kernel 6.x (a pontos verzió a reprodukciós naplóban rögzítendő, mert az API-k verziófüggőek lehetnek).

19.4.5.1 5.1 Cél

Megmérni és összehasonlítani a hagyományos másolás és a zero-copy/offload technikák hatását a latency-re, throughputra és CPU-terhelésre. A mérések alapján létrehozni egy **döntési táblázatot**.

19.4.5.2 5.2 Eszközök és mérési módszer

- **Források:** Kis C program (`sender.c`, `receiver.c`) a `sendfile`, `splice` és hagyományos `send` használatára.
- **Hálózati konfiguráció:** `ethtool -K <iface> tx on rx on tso on gro on` (offload be), majd kikapcsolt állapot mérése.
- **Mérés:** `iperf3` a TCP throughputra, `qperf` vagy `ping -f` a latency-re és csomagvesztésre. `perf stat` a CPU-ciklusok és cache-miss mérésére.

19.4.5.3 5.3 Elvárt (levezetett) vs. Mért eredmények

A mérés előtt rögzítsük az **elvárt viselkedést**: - *Elvárt:* A `sendfile` használata nagy fájloknál 20-40%-kal csökkenti a CPU-felhasználást a `read`+`write`-hez képest. - *Elvárt:* A TSO bekapcsolása 1500 MTU mellett lehetővé teszi a 10 Gbit-es vonal telítését, míg kikapcsolva a CPU a 100%-ot eléri, és a sávszélesség beszűkül. - *Elvárt:* A latency a legjobb akkor lesz, ha Busy Polling (`SO_BUSY_POLL`) van, a legrosszabb pedig Interrupt Coalescing bekapcsolásával.

(A mérési naplóba a mért értékeket ide kell beilleszteni, és az eltéréseket elemezni.)

19.4.5.4 5.4 Döntési táblázat (Sablon)

Felhasználási mód / Munkaterhelés	Zero-copy ajánlott?	Offload ajánlott (TSO/GRO)?	CPU-affinitás fontos?	Indoklás
HTTP fájlkiszolgáló (nagy fájlok)	**Igen** (`sendfile`)	**Igen**	**Közepes**	A CPU a másolásokkal lesz telítve, offload nélkül nem éri el a sávszélességet.
DNS/DHCP szerver (kis csomagok)	**Nem**	**Nem**	**Igen**	A kis (max 512 B) csomagoknál a laprögzítés drágább, mint a másolás. A latency a kritikus, az affinity csökkenti a cache-miss-t.
Adatbázis replikáció (közepes csomagok)	**Talán** (`MSG_ZEROCOPY`)	**Igen**	**Igen**	A nagy batch-ek hatékonyak, de a tranzakciós integritás miatt a kernelben tartás lehet biztonságosabb.
Valós idejű játék/streaming	**Nem**	**Ki** (TSO növeli a latency-t)	**Igen**	A kis keretek gyors továbbítása a cél, a batchesítés (Interrupt Coalescing, TSO) rontja a válaszidőt.

19.4.5.5 5.5 Tipikus hibák és megoldások a labor során

1. **Hiba:** Az offload-ot kikapcsolják, mert “az hamis képet ad”. **Megoldás:** Ez téves. A modern virtuális környezetek (VirtIO) szoftveresen emulálják a TSO/GRO-t; letiltása a host CPU-t terheli, és nem méri a valós elérhető teljesítményt. Mindig dokumentálni kell az `ethtool` állapotát.
2. **Hiba:** A CPU-affinitást nem ellenőrzik, és a mérési eredmények nagy szórást mutatnak. **Megoldás:** A QEMU vCPU szálait is rögzíteni kell (`taskset` a QEMU szálakra), mert a virtuális környezetben a hoszt ütemezője felülírhatja a vendég affinity beállításokat.
3. **Hiba:** A `sendfile`-t kisméretű, interaktív üzenetekre használják. **Megoldás:** Ilyenkor a hívás overheadje és a kontextusváltás a domináns, nem a másolás. Ilyen esetekben a batch feldolgozás vagy az io_uring a helyes válasz.

19.4.6 6. Összegzés

Ebben az alfejezetben láthattuk, hogy a hálózati teljesítmény nem csak a CPU órajelétől függ, hanem attól, hogyan szervezzük meg az adatok útját. A zero-copy technikák (`sendfile`, `splice`, `MSG_ZEROCOPY`) megszüntetik a felesleges CPU-másolásokat. A hardveres offload (Checksum, TSO, GRO) kiveszi a per-packet terhelést a CPU-ból. Végül a CPU-affinitás és a modern interrupt kezelés (RSS, Busy Polling) biztosítja, hogy a megfelelő adat a megfelelő cache-ben legyen a megfelelő időben.

Fontos megérteni, hogy a hálózati verem optimalizálása mindig kompromisszum. Nincs "egy jó beállítás" mindenre: a nagy áteresztőképességet igénylő fájltranszfer más konfigurációt igényel, mint az alacsony késleltetést igénylő valós idejű alkalmazás. A mérnök feladata a munkaterhelés profilozása és a bemutatott döntési táblázat alapján a helyes kernel- és hardverparaméterek kiválasztása. A következő alfejezetben (DNS, DHCP és konfiguráció) ezt a tudást alkalmazzuk a hálózati szolgáltatások (DNS, DHCP) építésekor, ahol a kis csomagok miatt a latency és az affinity kiemelt szerepet kap.

20 Hálózati szolgáltatások

20.1 DNS, DHCP és konfiguráció

A hálózati konfiguráció két alappillére a névfeloldás és az automatikus címkiosztás. A Zero-copy, offload és hálózati teljesítmény egységben áttekintettük a socket API-t, a TCP/IP-stack rétegeit és a csomagfeldolgozás útját; most arra a felhasználói térben futó szolgáltatásrétegre lépünk, amely a kernel IP-stackjét ténylegesen használhatóvá teszi a mindennapi rendszerekben. A DNS (Domain Name System) a hierarchikus névteret képezi le címekre, a DHCP (Dynamic Host Configuration Protocol) pedig a hálózati paraméterek – IP-cím, alhálózati maszk, átjáró, DNS-szerver – automatikus kiosztását végzi. Ebben az alfejezetben a két protokoll alapfogalmait, a rekordtípusokat, a lease-folyamatot, a relay szerepét, valamint a systemd-resolved és dnsmasq eszközök gyakorlati használatát mutatjuk be.

20.1.1 1. DNS: névfeloldás és hierarchikus névtér

A DNS elsődleges feladata, hogy az emberek által könnyen kezelhető tartományneveket – például `app.example.internal` – a hálózati forgalomhoz szükséges IP-címekre fordítsa. A névfeloldás nem kernel-szolgáltatás: a felhasználói programok a libc `getaddrinfo()` függvényén keresztül a resolver könyvtárhoz fordulnak, amely a `/etc/resolv.conf` fájlban megadott névkiszolgálókat használja. A resolver tipikusan UDP-n küld DNS-lekérdezést az 53-as portra, de nagyobb válaszok és zónaátvitel esetén TCP-t is használ.

A tartománynévtér hierarchikus, a gyökértől (`.`) balra haladva mélyülő címkékből áll. Az `app.example.internal.` teljesen minősített név (FQDN) záró pontja a gyökeret jelöli. A névteret zónákra osztják; minden zónáért egy vagy több névkiszolgáló felel. A delegáció NS (Name Server) rekordokkal történik: a szülő zóna megadja, hogy a gyermek zónáért melyik szerver a mérvadó (authoritative). A gyökérzónát a gyökérnévkiszolgálók kezelik, amelyek a legfelső szintű TLD-k (például `com`, `org`, `hu`) delegációit tartalmazzák.

A lekérdezés két alapvető módja a rekurzív és az iteratív feloldás. Rekurzív lekérdezésnél a kliens a helyi resolvernek küldi a kérdést, és a resolver vállalja a teljes feloldást: ha szükséges, ő maga kérdezi meg a gyökér-, a TLD- és a zónaszervereket. Iteratív módban a szerver a legközelebbi ismert névkiszolgáló címével válaszol, és a kliensnek kell tovább kérdeznie. A gyakorlatban a felhasználói gépek rekurzív lekérdezést küldenek egy közeli DNS-szervernek – például egy dnsmasq-ot futtató átjárónak –, amely vagy maga oldja fel rekurzívan a nevet, vagy továbbítja egy upstream resolvernek. A forwarding és a cache jelentős teljesítményjavulást hoz, mert a gyakran lekérdezett nevek TTL-ig a helyi gyorsítótárban maradnak.

A DNS-rekordok típusa határozza meg, milyen adatot hordoz a válasz. A legfontosabb rekordtípusok:

Típus	Jelentés	Példa
A	IPv4-cím	`app.example.internal. 300 IN A 192.168.150.10`
AAAA	IPv6-cím	`app.example.internal. 300 IN AAAA fd00::10`
CNAME	Kanonikus név, alias	`www.example.internal. IN CNAME app.example.internal.`
MX	Levelezőszerver, prioritással	`example.internal. IN MX 10 mail.example.internal.`
NS	Névkiszolgáló a zónához	`example.internal. IN NS ns1.example.internal.`
SOA	Zóna kezdő rekordja, sorozatszám és időzítők	`example.internal. IN SOA ns1.example.internal. hostmaster.example.internal. 2024010101 3600 600 86400 300`
PTR	Fordított (reverse) feloldás IP → név	`10.150.168.192.in-addr.arpa. IN PTR app.example.internal.`
TXT	Szabad szöveg, SPF, domain-ellenőrzés	`example.internal. IN TXT "v=spf1 -all"`
SRV	Szolgáltatás helye, port és súly	`_ldap._tcp.example.internal. IN SRV 0 5 389 ldap.example.internal.`

Az SOA mezői közül kiemelendő a sorozatszám (serial), amely a zóna változásának verziója, valamint a refresh, retry, expire és minimum TTL értékek, amelyek a másodlagos szerverek frissítését szabályozzák. A TTL (time-to-live) azt mondja meg, hány másodpercig tárolhatja a resolver a választ a gyorsítótárban. A túl hosszú TTL elavult adatokat tart, a túl rövid pedig felesleges forgalmat generál; a tipikus érték 300–3600 másodperc között mozog.

A fordított feloldás az IP-címből képzett speciális tartománynév alapján történik. IPv4 esetén az `in-addr.arpa` zónában a cím bájtjai fordított sorrendben szerepelnek: a `192.168.150.10` címe a `10.150.168.192.in-addr.arpa` névre képződik le. IPv6 esetén a `ip6.arpa` zónában a 32 hexadecimális nibble fordított sorrendben, pontokkal elválasztva található. A PTR-rekordokat gyakran a DHCP-szerver és a DNS-szerver integrációja tartja karban.

A DNS-szerverek szerepük szerint lehetnek mérvadó (authoritative), rekurzív, forwarder vagy csak gyorsítótárazó szerverek. Egy dnsmasq-példány tipikusan forwarder és gyorsítótárazó, de a helyi hálózati zónákhoz mérvadó válaszokat is adhat, míg egy nagy szolgáltató szerverét a zóna NS-rekordjai jelölik mérvadónak. A hibás konfiguráció gyakori tünete, hogy a mérvadó és a rekurzív funkció keveredik, vagy a zóna delegációjában körkörös hivatkozás alakul ki.

20.1.2 2. DNS minta dnsmasq-zal

A következő, szemléltetésre szánt dnsmasq-részlet egy kis belső hálózat helyi zónáját és néhány rekordját adja meg. A fájl tartalma nem éles környezetbe való, hanem emulált laborban használandó:

```
# /etc/dnsmasq.d/lab-dns.conf
# Helyi zóna mérvadó rekordjai
address=/app.example.internal/192.168.150.10
address=/mail.example.internal/192.168.150.11
ptr-record=10.150.168.192.in-addr.arpa,"app.example.internal"
ptr-record=11.150.168.192.in-addr.arpa,"mail.example.internal"
```

A `address=` direktíva A (vagy AAAA) rekordot hoz létre, a `ptr-record` pedig a fordított zónában helyez el PTR rekordot. A konfiguráció betöltése után a teszt a `dig` paranccsal végezhető el:

```
dig +short app.example.internal @127.0.0.1
# Várt válasz: 192.168.150.10

dig +short -x 192.168.150.10 @127.0.0.1
# Várt válasz: app.example.internal
```

A DNS-hibakeresés leggyakoribb jele a `NXDOMAIN` (nem létező név), a `SERVFAIL` (a szerver nem tudta feloldani, például hálózati hiba vagy DNSSEC-eltérés) és a `REFUSED` (a szerver nem hajlandó válaszolni, például rekurzió letiltva mérvadó zónánál). A TTL-problémák akkor jelentkeznek, amikor egy frissített rekord régi értéke a gyorsítótárban marad; ilyenkor a `dig` válaszában látható TTL folyamatosan csökken, és a cache törléséig a régi adat szolgál ki.

20.1.3 3. DHCP: automatikus hálózati konfiguráció

A DHCP célja, hogy a hálózatra csatlakozó eszköz megfelelő IP-címet, alhálózati maszkot, alapértelmezett átjárót, DNS-szervert és egyéb paramétereket kapjon emberi beavatkozás nélkül. A folyamat UDP-n, a 67-es (szerver) és 68-as (kliens) porton zajlik, és négy lépésből áll, amit a kezdőbetűkből DORA-nak nevezünk:

1. **Discover** – A kliens L2 broadcast (FF:FF:FF:FF:FF:FF) és L3 broadcast (255.255.255.255) címmel keres DHCP-szervert. A csomag tartalmazza a kliens hardvercímét (chaddr) és egy véletlen tranzakcióazonosítót (xid).
2. **Offer** – A szerver, ha rendelkezik szabad címmel, ajánlatot küld. Az ajánlat tartalmazza a felajánlott IP-címet (yiaddr), a szerver azonosítóját, a lease időtartamát és a kért opciókat. A kliens még nem használhatja a címet, mert több ajánlatot is kaphat.
3. **Request** – A kliens broadcast üzenetben megerősíti, melyik ajánlatot fogadja el. A broadcast azért szükséges, hogy a nem kiválasztott szerverek is értesüljenek, és visszavonják ajánlatukat.
4. **Ack** – A kiválasztott szerver nyugtázza a hozzárendelést. Ettől kezdve a lease aktív, és a kliens beállítja az IP-címét, az alhálózati maszkot, az átjárót és a DNS-t.

A lease (bérlet) időtartama a DHCP-szerver által meghatározott idő, ameddig a kliens jogosult a cím használatára. A kliens a lease lejárta előtt meg kell, hogy újítsa. A DHCP protokoll két időzítőt definiál: a T1 a lease idő 50 százaléka, a T2 pedig általában 87,5 százaléka. T1-nél a kliens unicast Request üzenetet küld a kiosztó szervernek; ha ez sikertelen, T2-nél broadcast Requestet küld bármely szervernek. Ha a lease lejár, a kliensnek fel kell adnia a címet, és újra kell kezdenie a DORA folyamatot.

A DHCP üzenetek opciókban szállítják a konfigurációs paramétereket. A fontosabb opciók:

Opció	Jelentés
1	Alhálózati maszk
3	Alapértelmezett átjáró (router)
6	DNS-szerver(ek)
15	Tartománynév
51	Lease időtartam (másodperc)
53	Üzenettípus (DHCP Discover/Offer/Request/Ack stb.)
54	Szerverazonosító
58	T1 megújítási idő
59	T2 újrakötési idő

A DHCP-kérések alapértelmezés szerint csak a helyi linken terjednek broadcastként. Ha a kliens és a szerver különböző IP-alhálózaton van, egy DHCP relay (más néven IP helper) szükséges. A relay általában a routeren fut, fogadja a broadcast Discover/Request üzeneteket, majd unicastként továbbítja azokat a szervernek. A relay a giaddr mezőbe írja annak az interfésznek az IP-címét, amelyen a kérést fogadta. A szerver ebből tudja, hogy melyik poolból kínáljon címet, és a választ a giaddr címen keresztül küldi vissza a relaynek, amely broadcastként eljuttatja a klienshez. A relay nélkül a több alhálózatos környezetben minden szegmensbe külön DHCP-szerver kellene, ami növeli az adminisztrációs terhet.

A címkiosztás három módja közül a dinamikus a leggyakoribb: a szerver egy poolból, meghatározott tartományból ad ki címeket változó hozzárendeléssel. A statikus hozzárendelésnél a szerver egy adott hardvercímhez mindig ugyanazt az IP-címet adja, ami stabil címzést és jól tervezhető DNS-integrációt tesz lehetővé. Az automatikus módban a szerver a poolból egyszer ad címet, és azt a továbbiakban a kliens azonosítójához köti, de nem feltétlenül szándékolt a fix hozzárendelés. A DHCP-szerver a cím kiadása előtt általában ICMP echo requestet küld a felajánlott címre, hogy elkerülje a címütközést; a kliens oldalán ARP probe ellenőrzi, hogy nincs-e más eszköz a hálózaton az adott címmel.

20.1.4 4. DHCP minta dnsmasq-zal

Egy tipikus dnsmasq-alapú DHCP-szerver konfigurációja a következő lehet, ismét kizárólag emulált hálózati laborhoz:

```
# /etc/dnsmasq.d/lab-dhcp.conf
interface=eth1
bind-interfaces
# DHCP pool: 192.168.150.50 - 192.168.150.150, lease 12 óra
dhcp-range=192.168.150.50,192.168.150.150,255.255.255.0,12h
# Átjáró
dhcp-option=3,192.168.150.1
# DNS-szerverek
dhcp-option=6,192.168.150.1,1.1.1.1
# Domain név
dhcp-option=15,example.internal
# Statikus cím a kliens MAC-címe alapján
dhcp-host=52:54:00:ab:cd:ef,192.168.150.20,client.example.internal
```

A `dhcp-range` sorban a `255.255.255.0` az alhálózati maszk, a `12h` pedig a lease időtartama. A `dhcp-option=3` az átjárót, a `6` a DNS-szervereket, a `15` a tartománynevet adja meg. A `dhcp-host` a `52:54:00:ab:cd:ef` MAC-című kliensnek fix `192.168.150.20` címet és `client.example.internal` nevet rendel; a dnsmasq ezt a nevet automatikusan hozzáadja a DNS-hez, így a kliens külső névfeloldás nélkül, a helyi zónán keresztül elérhető.

A kliens oldalon a DHCP-kérés például a `dhclient` paranccsal indítható:

```
dhclient -v eth1
```

A folyamat során a kliens megszerzi a `192.168.150.20` címet, beállítja a maszkot, az átjárót és a DNS-szervereket. Az ellenőrzéshez használható parancsok:

```
ip addr show dev eth1
cat /etc/resolv.conf
dig +short client.example.internal @192.168.150.1
```

A szerver oldalán a dnsmasq naplója mutatja a DORA üzeneteket, például `DHCPDISCOVER(eth1) 52:54:00:ab:cd:ef`, `DHCPOFFER(eth1) 192.168.150.20 ...`, `DHCPREQUEST(eth1) 192.168.150.20 ...`, `DHCPACK(eth1) 192.168.150.20 ...`. A lease-eket a dnsmasq a `/var/lib/misc/dnsmasq.leases` fájlban tárolja (a disztribúciótól függően eltérő lehet).

A DHCP gyakori hibái közé tartozik, hogy a szerver nem hallgat a megfelelő interfészen (hiányzó `interface=` vagy `bind-interfaces`), a pool kimerült, a lease-adatbázis sérült, vagy a relay rosszul van konfigurálva. A kliens oldalon a `NO DHCPOFFERS RECEIVED` hibaüzenet általában azt jelzi, hogy a Discover üzenetre nem érkezett ajánlat, ami lehet fizikai csatlakozási probléma, hibás VLAN, tűzfal vagy a szerver félrekonfigurálása.

20.1.5 5. systemd-resolved és dnsmasq a gyakorlatban

A modern Linux-rendszereken a névfeloldás gyakran a systemd-resolved daemonon keresztül történik. A systemd-resolved egy stub resolver, amely a 127.0.0.53 címen hallgat, és a hagyományos `/etc/resolv.conf` helyett a `/run/systemd/resolve/stub-resolv.conf` fájlt használja, amelybe a `nameserver 127.0.0.53` bejegyzés kerül. A daemon aztán az egyes hálózati interfészekhez kötött DNS-szervereket kezeli, és a `resolvectl` paranccsal kérdezhető le az aktuális állapot:

```
resolvectl status
# Például: Link 2 (eth1)
#          Current DNS Server: 192.168.150.1
#          DNS Servers: 192.168.150.1 1.1.1.1
#          DNS Domain: example.internal
```

A systemd-resolved előnye, hogy a DNS-konfigurációt per-link kezeli, így több hálózati interfész (például VPN, Wi-Fi, Ethernet) esetén is a megfelelő szervert választja. Támogat split DNS-t, DNSSEC-ellenőrzést és DNS over TLS-t is, bár ezek beállítása a rendszer verziójától és a felépítéstől függően eltérő lehet. A leképezés a klasszikus `/etc/resolv.conf`-hoz képest indirektebb, ami hibakereséskor fontos tudnivaló: a `cat /etc/resolv.conf` nem feltétlenül mutatja a ténylegesen használt szervert.

A dnsmasq ezzel szemben elsősorban hálózati szolgáltatás, nem kliensoldali resolver. Gyakran fut routereken, beágyazott eszközökön és kisebb szervereken, ahol egyszerre nyújt DNS-forwardert, gyorsítótárat, DHCP-szervert, sőt TFTP-szervert is. A konfiguráció egyetlen fájlban kezelhető, a helyi zónák és a DHCP-vel kiosztott nevek integrációja egyszerű. A dnsmasq jól illeszkedik olyan környezetekbe, ahol egyetlen szolgáltatásnak kell a helyi hálózat teljes konfigurációját biztosítania.

A két eszköz nem zárja ki egymást: egy tipikus felépítésben a dnsmasq a hálózati szerver, a kliens gépek pedig systemd-resolved-ot futtatnak, amely a dnsmasq-ot kérdezi le. A systemd-resolved tehát a kliens oldali konfiguráció egységesítésére való, a dnsmasq pedig a szerver oldali szolgáltatás megvalósítására. A választás szempontjai közé tartozik a környezet mérete, a DHCP-igény, a DNSSEC-követelmény, valamint a konfiguráció karbantarthatósága. A beágyazott, korlátozott

erőforrású rendszerekben a dnsmasq kis memóriaigénye és egyszerűsége gyakran döntő, míg a modern desktop- és mobilrendszerekben a systemd-resolved biztosítja a per-link DNS-kezelést.

20.1.6 6. Labor: DNS- és DHCP-forgatókönyv

A labor célja egy reprodukálható DNS- és DHCP-forgatókönyv elkészítése emulált környezetben. A kód- és konfigurációs példák szemléltetésül szolgálnak, nem valós gépen végrehajtandó utasítások; a kísérletet izolált QEMU-környezetben vagy eldobható virtuális gépen kell elvégezni.

Kiinduló állapot – Két QEMU virtuális gép: `server` és `client`. A server két hálózati interfésszel rendelkezik: `eth0` a hoszt felé, `eth1` a belső laborhálózathoz. A client csak `eth1`-gyel csatlakozik ugyanahhoz a privát hálózathoz. A QEMU-hálózatot privát `socket` kapcsolattal vagy Linux bridge-szel lehet megvalósítani; a pontos parancs a reprodukciós csomagban rögzítendő. A labor hardverkörnyezete: QEMU 8.x, 2 vCPU, 1 GiB RAM, virtio-net, Linux kernel 6.1 (a verziók ellenőrizendők).

Szerver konfiguráció – A server statikus IP-t kap az `eth1` interfészre, majd elindul a dnsmasq:

```
ip addr add 192.168.150.1/24 dev eth1
ip link set eth1 up
systemctl restart dnsmasq
```

A konfigurációs fájl a korábban bemutatott `lab-dns.conf` és `lab-dhcp.conf` egyesítése, a megfelelő `interface=eth1` és `bind-interfaces` beállítással. A DHCP-pool legyen `192.168.150.50-192.168.150.100`, lease `1h`, átjáró `192.168.150.1`, DNS `192.168.150.1`, domain `example.internal`. A client MAC-címe `52:54:00:ab:cd:ef`, fix címe `192.168.150.20`, neve `client.example.internal`. A DNS-zónában az `app.example.internal` név a `192.168.150.10` címre mutat, a PTR-rekord pedig visszaadja ezt a nevet.

Kliens indítás – A client gépen DHCP-kérést indítunk:

```
dhclient -v eth1
```

Mérési napló – A következő parancsok kimenetét kell rögzíteni:

```
# Szerveren:
journalctl -u dnsmasq -n 50
cat /var/lib/misc/dnsmasq.leases

# Kliensen:
ip addr show dev eth1
cat /etc/resolv.conf
dig +short app.example.internal
dig +short -x 192.168.150.20
```

Várt eredmények és mérési módszer – A labor során mért értékeket a naplóba kell rögzíteni, és el kell különíteni a várt értékektől. Az alábbi táblázat a várt értékeket és a mérés módját mutatja:

Paraméter	Várt érték	Mérés módja	Mért érték
Kliens IP-cím	192.168.150.20	`ip addr show dev eth1`	`<mérendő>`
Alhálózati maszk	255.255.255.0	`ip addr show dev eth1`	`<mérendő>`
Alapértelmezett átjáró	192.168.150.1	`ip route`	`<mérendő>`

Paraméter	Várt érték	Mérés módja	Mért érték
DNS-szerver	192.168.150.1	`cat /etc/resolv.conf`	`<mérendő>`
Domain név	example.internal	`resolvectl status` vagy `dhclient -v` napló	`<mérendő>`
`app.example.internal` feloldás	192.168.150.10	`dig +short`	`<mérendő>`
PTR feloldás 192.168.150.20	client.example.internal	`dig +short -x`	`<mérendő>`
DORA lépések	Discover, Offer, Request, Ack	`journalctl -u dnsmasq`	`<mérendő>`

A mért és várt érték eltérése esetén a hibát a konfigurációban vagy a hálózati izolációban kell keresni. A labor végén a szolgáltatást le kell állítani, a konfigurációs fájlokat menteni vagy törölni, és a virtuális gépeket vissza kell állítani a kiinduló állapotba. A hoszt valódi hálózati beállításait nem szabad módosítani.

20.1.7 7. Gyakori hibák és diagnosztika

A DNS- és DHCP-problémák gyakran hasonló tünetekkel jelentkeznek, de a diagnosztika eltérő. A DNS-nél a leggyakoribb hibák:

- `NXDOMAIN`: a lekérdezett név nem létezik, vagy a zóna delegációja hiányzik. Ellenőrizni kell a zóna SOA- és NS-rekordjait.
- `SERVFAIL`: a resolver nem tudta befejezni a feloldást. Gyakori ok a hibás DNSSEC-ellenőrzés, a hálózati hiba az upstream felé, vagy a forwarder félrekonfigurálása.
- Régi adat a gyorsítótárban: a TTL-lejárat előtt a módosított rekord nem látszik. A cache-t üríteni kell, vagy meg kell várni a TTL-lejáratot.
- Hiányzó PTR-rekord: a fordított feloldás `NXDOMAIN`-nel tér vissza, ami naplózásban, levelezésben problémát okozhat.
- Tűzfal blokkolja az 53-as UDP/TCP portot: a kliens nem kap választ, a `dig` időtúllépést jelez.

A DHCP-nél a tipikus hibák:

- A kliens nem kap IP-címet: nincs elérhető DHCP-szerver, a relay rosszul van konfigurálva, vagy a szerver nem hallgat az adott interfészen.
- IP-ütközés: két eszköz ugyanazt a címet használja. A szerver konfliktusdetektálása (ICMP probe) vagy a kliens ARP probe-ja kell, hogy megakadályozza.
- Túl rövid lease: a kliens gyakori megújítással terheli a hálózatot és a szervert. A lease időt a hálózat méretéhez és a változási gyakorisághoz kell igazítani.
- Hiányzó opciók: a kliens megkapja az IP-címet, de nincs átjáró vagy DNS. A `dhcp-option=3` és `6` beállításokat ellenőrizni kell.
- A szerver naplójában `DHCPDISCOVER` van, de `DHCPOFFER` nincs: a pool kimerült, vagy a szerver nem tudja a giaddr alapján kiválasztani a megfelelő poolt.

Mindkét protokollnál a hibakereséshez a napló, a csomagfogás (például `tcpdump`) és a konfiguráció verifikálása a leghatékonyabb. A `dig +trace` például a DNS feloldás lépéseit mutatja meg, míg a `tcpdump -i eth1 port 67 or port 68` a DHCP-forgalmat teszi láthatóvá.

20.1.8 8. Önellenőrző kérdések

1. Mi a különbség a rekurzív és iteratív DNS-lekérdezés között? Melyiket használja tipikusan egy kliens, és melyiket egy forwarder?
2. Sorolja fel a DORA folyamat lépéseit, és magyarázza el, miért broadcast a Request üzenet.
3. Hogyan befolyásolja a lease időtartam a DHCP-hálózati forgalmat és a címváltozások gyakoriságát?
4. Mi a giaddr mező szerepe a DHCP-relay működésében?
5. Milyen esetben előnyösebb a systemd-resolved használata a hagyományos `/etc/resolv.conf` közvetlen szerkesztésével szemben, és mikor célszerű dnsmasq-ot futtatni?

20.1.9 9. Források és ellenőrzési pontok

A leírt protokollok elsősorban az alábbi szabványokon alapulnak; a pontos viselkedés és a verziófüggő részletek ellenőrzést igényelnek a felhasznált rendszer dokumentációjában:

- RFC 1034, RFC 1035 – Domain Name System
- RFC 2131, RFC 2132 – DHCP és DHCP options
- systemd-resolved és dnsmasq hivatalos kézikönyv (man oldalak, a verzió megjelölésével)

A fejezetben bemutatott konfigurációk és parancsok emulált környezetre készültek; a verziók és az elérési utak disztribúciónként eltérhetnek, ezért a labor reprodukciós csomagjában a QEMU, a kernel, a dnsmasq és a systemd verzióját pontosan rögzíteni kell. A mért értékeket nem helyettesítheti várt érték; a labor-kiírásban a mérési napló és a visszaállítási út megadása kötelező.

20.2 NFS, SMB és hálózati fájlrendszerek

A hálózati fájlmegosztás protokolljai lehetővé teszik, hogy egy kliens úgy érjen el egy távoli szerveren tárolt fájlokat és könyvtárakat, mintha azok a helyi fájlrendszer részét képeznék. Ez a fejezet a két meghatározó protokollcsaládot, az NFS-t és az SMB-t mutatja be: felépítésüket, működésüket, biztonsági és teljesítményjellemzőiket, valamint egy üzemeltetési tervet, amelyben a gyakorlatban is alkalmazhatók. A tárgyalás a korábban bemutatott hálózati alapokra (DNS, DHCP és konfiguráció) épül, a következő fejezet pedig a konténerhálózati környezetben hasznosítja az itt bevezetett fogalmakat.

20.2.1 Alapfogalmak: export, mount, lock és ACL

A hálózati fájlmegosztás modellje kliens–szerver architektúrára épül. A szerver **exportál** egy könyvtárat vagy állományrendszert, azaz engedélyezi a hozzáférést meghatározott kliensek számára. A kliens **mountolja** ezt az exportot, vagyis a távoli erőforrást a saját címtérfájába illeszti, így a felhasználói programok a megszokott fájlműveletekkel (open, read, write, close) dolgozhatnak rajta. A műveletek a hálózaton keresztül utaznak, ezért a hálózati fájlrendszer protokolljának definiálnia kell a kérések és válaszok formátumát, a hibakezelést és a hitelesítést.

A kliens számára a távoli fájlok azonosítását **file handle** teszi lehetővé. Ez egy, a szerver által generált, átlátszatlan azonosító, amely a kliens oldalán a helyi fájlleíróhoz hasonló szerepet tölt be. A handle a mount időpontjában és a fájl elérésekor jön létre, és érvényessége az adott exportáláshoz kötött.

A megosztott fájlrendszerben különösen fontos a **zárolás** (lock), mert több kliens egyszerre módosíthatja ugyanazt a fájlt. A zárolási szemantika eltérő a protokollokban: az NFS hagyományosan külön zárolási protokollt használ, míg az SMB beépítetten kezeli a zárolási módokat. A **hozzáférés-vezérlési lista (ACL)** a jogosultságok finomhangolására szolgál: a Unix-szerű rendszerekben a hagyományos UID/GID és a POSIX ACL, míg a Windows/SMB világban a SID-alapú NTFS ACL a jellemző.

20.2.2 NFS protokollcsalád

Az NFS (Network File System) a Sun Microsystems fejlesztése, amely az 1980-as évek közepén jelent meg, és azóta a Unix-szerű rendszerek de facto szabványa a hálózati fájlmegosztásra. Az IETF szabványosította: az NFSv2 az RFC 1094, az NFSv3 az RFC 1813, az NFSv4 az RFC 7530, a legújabb NFSv4.1 pedig az RFC 8881 (a pontos sémákat az aktuális szabvány és a disztribúció dokumentációja tartalmazza). Az NFSv4.2 kisebb kiegészítéseket hozott. A verziók közötti különbségek alapvetően befolyásolják az állapotkezelést, a biztonságot és a teljesítményt.

Architektúra. Az NFSv3 három különálló protokollt használ: az NFS magot a fájlműveletekhez, a MOUNT protokollt a mountoláshoz, és az NLM (Network Lock Manager) protokollt a zároláshoz. Mindhárom RPC (Remote Procedure Call) mechanizmuson alapul, az adatok sorosítását pedig az XDR (External Data Representation) végzi. Az RPC szolgáltatásokat a portmapper (rpcbind) regisztrálja, így a kliens először a portmapperhez fordul a szolgáltatás portjának lekérdezéséhez. Az NFSv4 egyetlen protokollba integrálja a mount és a lock funkciókat, így nincs szükség portmapperre, és a 2049-es port közvetlenül használható. Az NFSv4 állapottartó (stateful), ami lehetővé teszi a kliens állapotának nyomon követését, például nyitott fájlok és zárolások kezelését.

Export és mount. A szerver az `/etc/exports` fájlban adja meg az exportált könyvtárakat és azok opcióit. Példa (emulált tesztkörnyezethez):

```
/srv/nfs/share  192.0.2.0/24(rw,sync,no_subtree_check,fsid=0)
```

Ez a bejegyzés a `/srv/nfs/share` könyvtárat a 192.0.2.0/24 hálózatnak exportálja írhatóan (`rw`), szinkron (`sync`) írással, alfa-ellenőrzés nélkül (`no_subtree_check`) és 0-s fsid-vel (gyökér export). A `sync` opció biztonságosabb, mert a szerver csak a lemezre írás után nyugtázza a műveletet, de lassabb, mint az `async`. A `root_squash` opció a root felhasználót egy alacsonyabb jogú felhasználóra (általában `nobody`) képezi le, csökkentve a kitettséget. A kliens oldalán a mountolás:

```
mount -t nfs4 192.0.2.10:/srv/nfs/share /mnt/nfs
```

Fájlműveletek és szemantika. Az NFSv3 alapvetően állapotmentes (stateless): a szerver nem tart nyilván semmit a kliens műveleteiről, minden kérés önálló. Ez egyszerűvé teszi a szerver felépítését, de a cache koherenciát óvatosan kell kezelni. A kliens agresszívan cache-elhet, de a szinkronizációhoz attribútumcache-t és írási késleltetést használ. A tipikus elv a „close-to-open” konzisztencia: ha egy kliens bezár egy fájlt, a következő megnyitásnál friss adatot kell látnia, de a nyitott fájlok közben a konkurens írások nem feltétlenül láthatók azonnal. Az NFSv4 bevezeti a stateful delegációt: a szerver átadhatja a fájl módosításának jogát egyetlen kliensnek (write delegation), így az helyben cache-elhet, amíg a szerver vissza nem vonja. Ez jelentősen javítja a teljesítményt, de komplexebbé teszi a hibakezelést.

Zárolás. Az NFSv3 a NLM-et használja, amely külön RPC program; a lockok megbízhatóságát az `rpc.statd` és `rpc.lockd` démonok biztosítják. Az NFSv4 integrált zárolást nyújt: a lockok bérlet (lease) alapúak, vagyis a kliensnek időnként meg kell újítania őket. Ha a kliens összeomlik,

a bérlet lejárta után a szerver felszabadítja a zárakat. A share reservation és a byte-range lock kombinációjával a fájlok hozzáférési módjai (olvasás, írás, törlés) is szabályozhatók.

Biztonság. Az NFSv3 alapértelmezett hitelesítése az AUTH_SYS (vagy AUTH_UNIX), amely a kliens által megadott UID/GID értékeket küldi el a szervernek, ezért önmagában nem biztonságos megbízhatatlan hálózaton. A Kerberos-alapú `RPCSEC_GSS` (pl. `krb5p` opció) integritás- és titkosításvédelmet ad. Az NFSv4 esetében a `sec=krb5`, `sec=krb5i`, `sec=krb5p` opciók használhatók. A mountolás során a `nfsvers` és `sec` paraméterekkel választható a verzió és a biztonsági szint.

Teljesítmény. A hálózati átvitel hatékonyságát a `rsize` és `wsize` mount opciók befolyásolják (pl. 1 MiB-os blokkméret). A szerver oldalán a `async` export opció gyorsítja az írást, de adatvesztést okozhat áramkimaradásnál. Az NFSv4 delegáció és a pNFS (parallel NFS) további optimalizálást tesz lehetővé: a pNFS lehetővé teszi a kliens számára a közvetlen adathozzáférést több adatszerverhez, megkerülve a metaadat-szervert.

20.2.3 SMB protokollcsalád

Az SMB (Server Message Block) az IBM-től származik, de a Microsoft fejlesztette tovább a Windows hálózati fájlmegosztás alapjaként. A mai SMB3 protokoll az SMB1/CIFS utódja, és számos vállalati funkciót támogat. A Linux rendszereken a Samba szerver és a CIFS kliens (`cifs-utils`) teszi elérhetővé. Az SMB specifikáció része a Microsoft által dokumentált protokoll, de a Samba projekt saját implementációja az interoperabilitás miatt meghatározó.

Architektúra. Az SMB kezdetben a NetBIOS felett futott (TCP/139, UDP/137-138), de a modern SMB2/SMB3 közvetlenül a TCP/445 portot használ. A protokoll üzenetalapú: a kliens parancsokat küld (negotiate, session setup, tree connect, create, read, write, close), a szerver válaszol. Az SMB1 számos biztonsági problémával rendelkezik (EternalBlue), ezért a mai rendszereken alapból letiltják. Az SMB2 (Windows Vista+) új alapokra helyezte a protokollt: kevesebb parancs, nagyobb teljesítmény, jobb hibatűrés. Az SMB3 (Windows 8+) olyan funkciókat hozott, mint a multichannel, az SMB Direct (RDMA), a transparent failover és a beépített titkosítás.

Share és hitelesítés. Az SMB a megosztott könyvtárakat **share** néven teszi elérhetővé. A kliens UNC elérési úttal hivatkozik rájuk: `\\szerver\megosztás`. A hitelesítés lehet NTLM (régi, gyengébb) vagy Kerberos (Active Directory környezetben). A Samba a `smb.conf` fájlban konfigurálható; példa:

```
[share]
  path = /srv/smb/share
  read only = no
  guest ok = no
  valid users = @smbusers
  ea support = yes
  store dos attributes = yes
```

A `valid users` a `smbusers` csoport tagjait engedi be; a felhasználókat a Samba saját jelszóadatbázisában (`smbpasswd`) kell létrehozni. A `guest ok = no` megakadályozza az anonim hozzáférést. A mount Linuxon:

```
mount -t cifs //192.0.2.10/share /mnt/smb -o username=tesztuser,sec=ntlmssp
```

A `sec=ntlmssp` az NTLM hitelesítést kényszeríti; Kerberos használatához `sec=krb5` és megfelelő realm beállítás szükséges.

Zárolás és konzisztencia. Az SMB gazdag zárolási mechanizmust kínál. A régi SMB1-ben **oplock** (opportunistic lock) volt, amely lehetővé tette a kliens számára a fájl kizárólagos cache-elését; a szerver visszavonhatta. Az SMB2/SMB3-ban ezt a **lease** rendszer váltotta fel: a kliens bérletet kaphat egy fájlra (pl. Read, Write, Handle, Read-Handle kombinációk), ami meghatározza, hogy milyen műveleteket végezhet helyben cache-ből. A bérlet megszakítható, ha másik kliens írni akar. A **durable handle** és a **persistent handle** lehetővé teszi, hogy a fájlleíró akkor is érvényben maradjon, ha a hálózati kapcsolat rövid időre megszakad, így a kliens nem veszíti el a nyitott fájl állapotát (például adatbázisoknál fontos).

SMB3 további funkciók. A **multichannel** lehetővé teszi, hogy egyetlen SMB munkamenet több hálózati kapcsolaton (pl. több hálózati kártya) keresztül párhuzamosan továbbítson, növelve a sávszélességet és a hibatűrést. Az **SMB Direct** RDMA (Remote Direct Memory Access) technológiát használ, ami alacsonyabb késleltetést és CPU-terhelést eredményez. A **transparent failover** a failover clusterben futó szerverek közötti zökkenőmentes átadást támogat. Az SMB3 beépített **encryption** opciója a hálózati forgalmat titkosítja, ami különösen hasznos nem megbízható hálózaton.

ACL. Az SMB a Windows NTFS ACL-modelljét használja: minden fájlhoz SID-ek (biztonsági azonosítók) és ACE-k (hozzáférés-vezérlő bejegyzések) tartoznak. A Samba ezt a Unix jogosultságokra és POSIX ACL-re képezi le, vagy Windows ACL-t szolgáltat a `vfs_acl_xattr` modullal. A Linux kliens általában a Unix jogosultságokat látja, de a Windows kliens a teljes ACL-t használja.

20.2.4 NFS és SMB összehasonlítása

A két protokoll eltérő tervezési filozófiát képvisel. Az NFS-t Unix környezetre tervezték, állapotmentes vagy lazán állapottartó, és a hagyományos Unix szemantikát (UID/GID, byte-range lock) követi. Az SMB ezzel szemben a Windows fájlmegosztás igényeit szolgálja, erősen állapottartó, és a Windows biztonsági modellt (SID/ACL) használja. A gyakorlati következmények:

Szempont	NFSv4	SMB3
Tipikus kliens	Linux/Unix	Windows (és Linux)
Állapotkezelés	Állapottartó, de kevésbé merev	Erősen állapottartó
Zárolás	Lease-alapú, byte-range + share	Lease, oplock utód, durable handle
Hitelesítés	AUTH_SYS, Kerberos	NTLM, Kerberos
ACL	UID/GID + POSIX ACL	NTFS ACL (SID)
Teljesítmény	Jó nagy blokkoknál; pNFS segíthet	Jó, SMB3 multichannel/RDMA
Kompatibilitás	Unix-szerű rendszereken kiváló	Windows környezetben elsődleges

A teljesítmény nagyban függ a hálózat késleltetésétől, a blokkmérettől és a cache-ezési stratégiától. Általánosságban mindkét protokoll képes kihasználni a gigabites hálózatot, de az overhead eltér: az NFS RPC/XDR rétege, az SMB üzenetstruktúrája és hitelesítési lépései mind hatással vannak a késleltetésre. A biztonság terén mindkét protokoll támogat erős hitelesítést és titkosítást, de a beállítás bonyolultsága eltérő.

20.2.5 Labor: NFS/SMB üzemeltetési terv készítése

Ez a labor egy olyan üzemeltetési terv elkészítését írja elő, amelyben a hallgató megtervezi az NFS és SMB szolgáltatás telepítését, konfigurálását, mérését és visszaállítását. A tervnek reprodukálhatónak kell lennie, emulált környezetben (például QEMU virtuális gépek), és rögzítenie kell a konfigurációt, a mérési naplót, valamint a mért és várt eredményeket különválasztva. **A példa parancsok és konfigurációk nem végrehajtandók éles gépen; csak ellenőrzött, eldobható laborban futtathatók.**

Környezet és kiinduló állapot. Két QEMU virtuális gép, mindkettő Debian/Ubuntu Linux (a disztribúció aktuális stabil kiadása; verzió ellenőrizendő). A szerver IP-címe 192.0.2.10, a kliensé 192.0.2.20 (dokumentációs tartomány). A hálózat virtuális bridge, például `virbr0`. A szerveren telepítendő: `nfs-kernel-server` (NFSv4), `samba` (SMB3). A kliensre szükséges: `nfs-common`, `cifs-utils`, valamint `fio` és `iperf3` a mérésekhez. A kiinduló állapotot snapshot-tal kell rögzíteni.

NFS konfiguráció és mount. A szerveren hozzunk létre egy könyvtárat:

```
mkdir -p /srv/nfs/share
chown root:smbusers /srv/nfs/share
chmod 2770 /srv/nfs/share
```

Az `/etc/exports` tartalma:

```
/srv/nfs/share 192.0.2.20(rw,sync,no_subtree_check,fsid=0,sec=sys)
```

Itt a `sec=sys` az AUTH_SYS-t jelenti laborban; éles környezetben Kerberos ajánlott. Export-frissítés:

```
exportfs -ra
systemctl restart nfs-server
```

A kliens mountolása:

```
mount -t nfs4 192.0.2.10:/ /mnt/nfs
```

Az `fsid=0` miatt az export gyökérként jelenik meg; ha nincs `fsid=0`, a teljes elérési utat kell megadni.

SMB konfiguráció és mount. A szerveren:

```
mkdir -p /srv/smb/share
chown root:smbusers /srv/smb/share
chmod 2770 /srv/smb/share
groupadd smbusers
useradd -m -G smbusers tesztuser
smbpasswd -a tesztuser   # jelszó: laborjelszo (helyőrző)
```

Az `/etc/samba/smb.conf` megfelelő szakasza:

```
[global]
   workgroup = WORKGROUP
   server string = Labor SMB
   security = user
   server min protocol = SMB2_10
   client min protocol = SMB2_10

[share]
   path = /srv/smb/share
   read only = no
   guest ok = no
```

```
   valid users = @smbusers
   ea support = yes
   store dos attributes = yes
   force create mode = 0770
   force directory mode = 2770
```

Szolgáltatás újraindítása:

```
systemctl restart smbd nmbd
```

A kliens mountolása:

```
mount -t cifs //192.0.2.10/share /mnt/smb -o username=tesztuser,password=laborjelszo,sec=ntlmssp
```

Mérési terv. A mérési naplóban minden mért értéket fel kell tüntetni, a várt eredményt pedig a mérés előtt meg kell fogalmazni. A méréshez legalább három ismétlés javasolt. A mérési pontok:

1. **Hálózati kapacitás:** `iperf3 -c 192.0.2.10` a szerveren futva. Várt: 1 Gbit/s körüli sávszélesség (virbr0 esetén a tényleges host limit). Mért: rögzítendő.
2. **NFS szekvenciális olvasás/írás:** `fio --name=nfs --rw=read --bs=1M --size=1G --directory=/mnt/nfs --ioengine=sync --direct=1` (és írás `rw=write`). Várt: sávszélesség a link 80–90%-a nagy blokkoknál, írásnál valamivel alacsonyabb, ha `sync` export van. Mért: táblázatban.
3. **SMB szekvenciális olvasás/írás:** ugyanez a `/mnt/smb` könyvtárban. Várt: hasonló, de a CIFS overhead miatt kissé alacsonyabb; SMB3 titkosítás nélkül. Mért: rögzítendő.
4. **Kis fájl műveletek:** 1000 darab, egyenként 4 KiB méretű fájl létrehozása, olvasása, törlése. Várt: a kis fájl műveleteknél a hálózati késleltetés dominál, az IOPS a helyi lemez töredéke. Mért: idő és IOPS.
5. **Konkurens zárolás:** két kliens párhuzamosan ír egy közös fájlba, `fcntl` byte-range lock az NFS-en, SMB-nél `flock` vagy Windows kliens. Várt: zárolás nélkül adatvesztés, zárolással hibamentes. Mért: szimuláció eredménye.
6. **Cache koherencia:** az egyik kliens módosít egy fájlt, a másik azonnal olvassa. Várt: NFSv4 close-to-open konzisztencia esetén az új megnyitásnál friss adat; SMB lease esetén szintén. Mért: megfigyelt viselkedés.

Várt és mért eredmény szétválasztása. Minden mérésnél a jegyzőkönyv tartalmazza a *várt* értéket (amelyet a hálózati sávszélesség, a lemezsebesség és a protokoll overhead alapján becsülünk) és a *mért* értéket (a mérési sorozat átlaga, szórása). A kettő közötti eltérést elemezni kell. Például ha a mért írási sebesség sokkal kisebb a vártnál, az utalhat a `sync` exportra, a hálózati csomagvesztésre vagy a szerver CPU-korlátjára.

Visszaállítás. A labor végén a mountokat le kell választani (`umount /mnt/nfs /mnt/smb`), az exportot törölni (`exportfs -u 192.0.2.20:/srv/nfs/share`), a Samba szolgáltatást leállítani (`systemctl stop smbd nmbd`), a létrehozott könyvtárakat és felhasználókat eltávolítani (`userdel tesztuser`, `rm -rf /srv/nfs/share /srv/smb/share`), valamint a virtuális gépek snapshotját visszaállítani. A mérési naplót a reprodukciós csomaghoz kell csatolni.

20.2.6 Tipikus hibák és megoldások

- **Hiba:** az NFS mount sikertelen, `mount.nfs4: access denied by server`. **Ok:** az `/etc/exports` nem frissült (`exportfs -ra` hiányzik), vagy a kliens IP-címe nem szerepel az exportban. **Megoldás:** az exports fájl ellenőrzése, `exportfs -ra` futtatása, a szerver naplójának (`/var/log/syslog`) vizsgálata.

- **Hiba:** az NFS lock nem működik, `rpc.statd` nem fut. **Ok:** NFSv4 esetén a lock a fő protokoll része, de ha az NFSv3-at használunk, a lockd és statd szükséges. **Megoldás:** NFSv4-re váltás vagy a szükséges démonok elindítása.
- **Hiba:** a SMB mount hitelesítési hibát ad. **Ok:** a Samba felhasználó jelszava nincs beállítva (`smbpasswd -a`), vagy a `valid users` csoporttagság rossz. **Megoldás:** a felhasználó létrehozása a Samba adatbázisban, a csoport ellenőrzése, a `smbclient -L //szerver -U tesztuser` teszt futtatása.
- **Hiba:** a fájlok jogosultsága nem a várt; a root felhasználó `nobody`-ként jelenik meg NFS-en. **Ok:** `root_squash` alapértelmezés. **Megoldás:** ha szükséges, `no_root_squash` (csak laborban, nem élesben), vagy a root helyett rendes felhasználó használata.
- **Hiba:** az SMB share látható, de a fájlok `.exe` kiterjesztés esetén nem futtathatók. **Ok:** a Samba `store dos attributes` vagy `map archive` miatt a Windows attribútumok keverednek. **Megoldás:** a share opciók pontosítása.
- **Hiba:** a hálózati teljesítmény alacsony, a késleltetés magas. **Ok:** a blocméret túl kicsi (`rsize/wsize`), a tűzfal csomagvizsgálata, vagy a virtuális hálózat korlátai. **Megoldás:** nagyobb blokkméret, tűzfal optimalizálás, dedikált hálózati kártya.

20.2.7 Biztonsági megfontolások

A hálózati fájlmegosztás adatot és metaadatot továbbít a hálózaton, ezért a biztonsági beállítások kritikusak. Az NFS esetében az AUTH_SYS csak megbízható hálózaton elfogadható; Kerberos nélkül a szerver nem tudja ellenőrizni a kliens által megadott UID/GID valódiságát. Az exports fájlban mindig korlátozzuk a hozzáférést konkrét alhálózatokra, és használjunk `root_squash`-t. Az SMB-nél tiltsuk le az SMB1-et, használjunk SMB2/3-at, engedélyezzük a signing és az encryption funkciót, és kerüljük a guest hozzáférést. A jelszavakat soha ne tároljuk a mount parancsban (opcióban), mert az a processzlista vagy a shell history révén kiszivároghat; használjunk hitelesítőadat-fájlt vagy Kerberost. A hálózati forgalom megfigyelése (pl. tcpdump) laborban tanulságos, de éles környezetben csak megfelelő engedéllyel végezhető.

20.2.8 Önellenőrző kérdés

Magyarázza el, miért okoz problémát az NFSv3 állapotmentes modellje a zárolásban, és hogyan oldja meg ezt az NFSv4 a lease-alapú lockokkal. A válaszában térjen ki arra, mi történik, ha a kliens összeomlik a lock birtoklása közben, és milyen szerepe van a lease lejáratának. (A megoldást a fejezet NFS szakasza tartalmazza; a kérdés célja a fogalom elmélyítése.)

20.2.9 Összefoglalás

Az NFS és az SMB két eltérő világ hálózati fájlmegosztási szabványa. Az NFS a Unix-szerű rendszerek natív protokollja, amely az RPC-re épül, és az NFSv4-től integrált mount és lock funkciókkal, valamint erős Kerberos-integrációval rendelkezik. Az SMB a Windows ökoszisztéma alapja, gazdag zárolási és ACL-modellt, a legújabb verzióiban pedig multichannel és RDMA támogatást nyújt. A megfelelő protokoll kiválasztása a környezettől és a követelményektől függ; a hibák jelentős része a helytelen export/mount konfigurációból és a biztonsági beállítások elhanyagolásából ered. Az üzemeltetési terv laborja lehetőséget ad a konfiguráció és a mérés gyakorlására, a mért és várt értékek kritikus összehasonlításával. A következő fejezetben ezeket a fogalmakat a konténerhálózat, a NAT és az overlay hálózatok kontextusában hasznosítjuk.

20.3 Konténerhálózat, NAT és overlay

A konténerhálózat problémája első ránézésre egyszerű: a konténerben futó alkalmazásnak úgy kell hálózatot elérnie, mintha saját hálózati kártyája, IP-címe és útvonalai lennének, miközben a hoszt, a többi konténer és a külvilág felé a forgalom szabályozottan, auditálhatóan és reprodukálhatóan halad át. A megvalósítás azonban több réteget érint, mert a konténer-futásidőnek egyszerre kell megoldania a hálózati izolációt, a címzést, a csomagtovábbítást, a címfordítást, a szolgáltatások feloldását és a forgalom engedélyezését. Ez a fejezet ezeket a rétegeket a bridge, a NAT, az overlay hálózat, a CNI, a service discovery és a hálózati policy sorrendjében mutatja be, és egy emulált környezetben elvégezhető laborral zárul.

20.3.1 1. A hálózati izoláció alapja: network namespace és veth pár

A konténerek hálózati izolációjának alapja a Linux **network namespace**. Egy network namespace a hálózati erőforrások – hálózati interfészek, útvonaltábla, tűzfalszabályok, socketek – különálló példányát adja. A konténer indításakor a konténer-futásidő új network namespace-t hoz létre, így a konténerben futó folyamat csak a saját interfészeit, útvonalait és tűzfalszabályait látja; a hoszt interfészei és más konténerek hálózati objektumai nem láthatók.

A namespace önmagában azonban még nem ad hálózati kapcsolatot. A konténernek legalább egy virtuális interfészre van szüksége, amely a külvilág felé vezet. Erre a célra a Linux **veth** (virtual Ethernet) pár szolgál: egy veth pár két végét logikailag egy Ethernet-kábellel összekötött hálózati kártyaként kell elképzelni. Az egyik végét a konténer namespace-ébe helyezik, a másik a hoszt oldalán marad. Ha a hoszt oldali véget egy bridge-hez csatlakoztatjuk, akkor több konténer hoszt oldali vége is ugyanarra az L2 kapcsolóra kerül, és a konténerek közvetlenül kommunikálhatnak egymással. A veth pár létrehozása és áthelyezése a hoszt oldali hálózati adminisztráció feladata; ezt a konténer-futásidő vagy a CNI plugin végzi el.

Ez a modell azért hatékony, mert a konténerben futó alkalmazásnak semmit sem kell tudnia a virtualizációról: a veth interfész egy közönséges Ethernet-eszközként jelenik meg, saját MAC-címmel, IP-címmel és MTU-val. A folyamat ugyanazokat a socket API-kat használja, mint egy fizikai gépen.

20.3.2 2. Bridge hálózat: konténerek összekapcsolása egy hoszton belül

A legegyszerűbb konténerhálózati topológia a **bridge hálózat**. Ebben az esetben a hoszton egy szoftveres L2 switch, azaz Linux bridge jön létre. A bridge interfészhez csatlakoznak a veth párok hoszt oldali végei, így minden konténer ugyanabba az L2 szegmensbe kerül. A bridge a MAC-címek alapján tanulja meg, melyik porton melyik MAC-cím érhető el, és ennek megfelelően továbbítja a kereteket. A broadcast, az ismeretlen unicast és a multicast kereteket minden portra kiküldi, ahogy egy fizikai switch is teszi.

A bridge hálózat címzési modellje jellemzően egy privát IPv4 alhálózat, például `172.18.0.0/16` vagy `10.0.0.0/24`. A bridge interfész maga kap egy IP-címet ebből az alhálózatból, és ez a cím szolgál a konténerek alapértelmezett átjárójaként. A konténer útvonaltáblája tehát nagyon egyszerű:

```
default via 172.18.0.1 dev eth0
172.18.0.0/16 dev eth0 proto kernel scope link src 172.18.0.2
```

A `172.18.0.1` itt a hoszt oldali bridge IP-címe, az `eth0` pedig a konténerben látható veth interfész. A konténer a saját alhálózatán belül közvetlenül, L2 szinten éri el a többi konténert; az alhálózaton kívülre irányuló forgalmat a bridge IP-címére küldi, ahol a hoszt útvonaltáblája dönt a további sorsáról.

A bridge hálózat legfontosabb tulajdonsága, hogy az azonos bridge-en lévő konténerek **izolációja L2 szinten megszűnőben van**: a konténerek látják egymás MAC-címét, ARP-kérései eljutnak a többiekhez, és ha nincs további szűrés, közvetlenül nyithatnak kapcsolatot egymáshoz. Ez sok esetben kívánatos, például egy webalkalmazás és egy adatbázis-konténer együttműködésekor, de többszintű biztonsági követelmények esetén további hálózati policyre van szükség.

A gyakorlatban a népszerű konténer-futásidők alapértelmezett bridge hálózatot hoznak létre, például a Docker a `docker0` bridge-et. Ez az alapértelmezett hálózat azonban hagyományosan nem biztosít automatikus DNS-alapú szolgáltatásfeloldást, ezért éles összeállításokban általában egyedi, felhasználó által definiált bridge hálózatot érdemes létrehozni, amelyhez a futásidő beépített DNS-szolgáltatást is rendel. A bridge hálózat kialakítása tehát nem csupán egyetlen kapcsoló létrehozása; a névfeloldás, a címosztás és az átjáró beállítása is a topológia része.

20.3.3 3. NAT és port publishing: kijárat a külvilág felé

A bridge hálózat a konténereket egymással összekapcsolja, de a konténerek privát IP-címei a hoszton kívül nem route-olhatók. A külvilág felé irányuló forgalomhoz **címfordítás (NAT)** szükséges. A NAT két irányban működik:

- **Kimenő irány (SNAT / MASQUERADE):** amikor egy konténer külső címre csatlakozik, a hoszt a kimenő csomag forráscímét a saját kimenő interfészének IP-címére írja át. A hoszt a kapcsolatot a kapcsolatkövető táblában (conntrack) tárolja, így a visszatérő csomagot a megfelelő konténerhez tudja visszairányítani. A `MASQUERADE` célpont annyival rugalmasabb a statikus SNAT-nál, hogy a forráscímet dinamikusan a kimenő interfész aktuális címéből veszi, így DHCP-s hosztokon is működik.
- **Bejövő irány (DNAT / port publishing):** ha egy konténer szolgáltatást kínál, például egy webes portot, akkor a hoszt egy adott portját hozzá kell rendelni a konténer privát IP-címéhez és portjához. Ezt jellemzően DNAT szabállyal oldják meg: a hoszt `0.0.0.0:8080` címére érkező csomag célcímét a kernel a konténer `172.18.0.2:80` címére írja át.

A Linux kernelben ezt a hagyományos netfilter/iptables, illetve az újabb nftables keretrendszer valósítja meg. Egy egyszerű, szemléltető iptables szabálykészlet a következő lehet:

```
# Kimenő forgalom masquerade-je a hoszt külső interfészén
iptables -t nat -A POSTROUTING -s 172.18.0.0/16 -o eth0 -j MASQUERADE

# Bejövő port publishing: hoszt 8080 -> konténer 80
iptables -t nat -A PREROUTING -p tcp --dport 8080 -j DNAT --to-destination 172.18.0.2:80
iptables -A FORWARD -p tcp -d 172.18.0.2 --dport 80 -j ACCEPT
```

A fenti szabályok illusztrációk, és csak izolált laborban, a hoszt tűzfalszabályainak módosítása nélkül vizsgálhatók. Éles környezetben ezeket a szabályokat a konténer-futásidő vagy a CNI plugin kezeli, és a felhasználó ritkán írja őket közvetlenül.

A NAT fontos következménye, hogy a külvilág számára a kapcsolat a hoszt IP-címéről érkezik, ezért a konténer eredeti forráscíme elveszik. Ez megnehezíti a naplózást és a hálózati hibakeresést, valamint bizonyos protokolloknál, például aktív FTP vagy VoIP esetén a beágyazott IP-címek miatt további ALG (Application Layer Gateway) támogatás szükséges. A modern konténerhálózatokban ezért egyre gyakrabban használnak közvetlenül route-olható, nem NAT-olt

alhálózatokat, például IPv6 vagy BGP-alapú L3 hirdetés esetén. A NAT azonban egyszerű és robusztus megoldás marad a legtöbb fejlesztői és kisebb üzemi környezetben.

A kapcsolatkövető tábla a NAT működésének központi eleme. Minden lefordított kapcsolat állapota bekerül a conntrack táblába, beleértve a protokollt, a forrás- és célportot, valamint az időbélyeget. Ez lehetővé teszi, hogy a visszatérő csomagokat a kernel determinisztikusan visszafejtse, és hogy a tűzfal a `ESTABLISHED` állapotú kapcsolatokat engedélyezze. A tábla mérete korlátozott; nagy forgalmú hosztokon a tábla telítődése csomagvesztést és kapcsolatmegszakadást okozhat, ezért a méretét és a kapcsolatok időtúllépését a terheléshez kell igazítani.

20.3.4 4. Overlay hálózat: konténerek több hoszton keresztül

A bridge és a NAT egyetlen hoszton belül oldja meg a konténerek hálózatát. Amint a konténerek különböző fizikai vagy virtuális hosztokon futnak, az L2 szegmenst ki kell terjeszteni a hosztok közötti IP-hálózaton. Erre szolgál az **overlay hálózat**.

Az overlay hálózat alapötlete, hogy a konténer által küldött Ethernet-keretet egy külső csomagba ágyazzák, és ezt a külső csomagot a hosztok közötti L3 hálózaton továbbítják. A célhoszton a külső csomagot kicsomagolják, és az eredeti Ethernet-keret a célkonténer veth interfészére kerül. A leggyakoribb megvalósítás a **VXLAN** (Virtual eXtensible LAN), de léteznek más protokollok is, például Geneve vagy GRE.

A VXLAN a belső Ethernet-keretet UDP-csomagba csomagolja. A külső UDP fejléc 8 bájtos VXLAN-fejlécet követ, amely egy 24 bites VXLAN Network Identifier (VNI) mezőt tartalmaz; ez azonosítja a virtuális L2 szegmenst, és elméletileg akár 16 millió overlay hálózatot tesz lehetővé. A külső UDP célport hagyományosan `4789`, a forrásport pedig gyakran a belső csomag hash-e, ami lehetővé teszi a többutas terheléselosztást a hosztok közötti fizikai hálózaton. A VXLAN-csomag felépítése vázlatosan:

```
+----------------+----------------+----------------+----------------+
| Külső Ethernet | Külső IP       | Külső UDP      | VXLAN fejléc   |
| fejléc         | fejléc         | fejléc         | (8 bájt, VNI)  |
+----------------+----------------+----------------+----------------+
| Belső Ethernet keret (konténer forgalom)                           |
+--------------------------------------------------------------------+
```

A hosztokon egy VXLAN interfész, például `vxlan0` végzi a be- és kicsomagolást. A hoszt a konténer felől érkező keretet erre az interfészre irányítja; a VXLAN interfész a keretet UDP-be csomagolja, és a célhoszt IP-címére küldi. A célhoszton a VXLAN interfész kicsomagolja a keretet, majd a bridge a célkonténer veth portjára továbbítja.

Az overlay hálózat legnagyobb előnye, hogy a konténerek számára egyetlen L2 szegmenst szimulál, függetlenül attól, hogy fizikailag melyik hoszton futnak. A konténerek IP-címei az overlay alhálózatból származnak, és a hosztok közötti útválasztást a VXLAN-csomagolás rejt. Ugyanakkor az overlay hálózatnak ára van: a csomagolás megnöveli a fejléc méretét, tipikusan 50 bájttal, ami csökkenti a hasznos adat arányát, és MTU-problémát okozhat. Ha a fizikai hálózat MTU-ja 1500 bájt, akkor a belső konténerforgalom számára csak 1450 bájt hasznos adat fér el, ezért a hosztok közötti hálózaton vagy az overlay interfészen csökkentett MTU-t kell beállítani, vagy path MTU discovery-re kell hagyatkozni. A rosszul beállított MTU a tipikus „nagy csomagok nem mennek át, kicsik igen” tünetet okozza, amit a felhasználók gyakran a TCP-viselkedéssel kevernek össze.

Az overlay hálózatokat a népszerű konténer-platformok CNI-pluginjei valósítják meg. A Docker beépített `overlay` drivere, a Flannel VXLAN módja, a Calico és a Weave mind ezt az alapelvet használják, bár a vezérlősík és a biztonsági modell jelentősen eltér. A teljesítmény szempontjából

az overlay hálózat mindig többletköltséget jelent a közvetlen L2/L3 kapcsolathoz képest, ezért nagy átvitelt igénylő rendszerekben érdemes mérni a VXLAN-kapszulázás CPU- és késleltetési hatását, és adott esetben IP-in-IP, illetve natív L3 hirdetés alapú megoldást választani.

20.3.5 5. CNI: a konténerhálózat szabványos csatlakozófelülete

A **CNI** (Container Network Interface) a konténer-futásidők és a hálózati pluginok közötti szabványos felület. A CNI specifikációja minimális, és a célja az, hogy a konténer-futásidő ne legyen összekötve egyetlen hálózati megvalósítással: ugyanaz a futásidő bármilyen CNI-kompatibilis plugint meghívhat, és a pluginok újrafelhasználhatók különböző futásidőkben.

A CNI modellje a következő: a futásidő a konténer indításakor létrehoz egy network namespace-t, majd meghívja a CNI plugin `ADD` parancsát. A plugin bemenete egy JSON konfiguráció, amely tartalmazza a hálózat nevét, típusát, címzési adatait és a plugin specifikus beállításait. A plugin feladata, hogy a megadott namespace-ben beállítsa a hálózati interfészt, IP-címet adjon, útvonalakat és DNS-beállításokat konfiguráljon, majd visszaadja az eredményt. A konténer megszűnésekor a futásidő a `DEL` parancsot hívja meg, és a plugin eltávolítja a hálózati objektumokat.

Egy egyszerű, szemléltető CNI-konfiguráció:

```
{
  "cniVersion": "1.0.0",
  "name": "example-bridge",
  "type": "bridge",
  "bridge": "cni0",
  "ipam": {
    "type": "host-local",
    "subnet": "10.22.0.0/24",
    "rangeStart": "10.22.0.10",
    "rangeEnd": "10.22.0.200",
    "gateway": "10.22.0.1"
  },
  "dns": {
    "nameservers": ["10.22.0.1"]
  }
}
```

A konfigurációban a `type` a plugin nevét, az `ipam` a címkiosztó modult, a `dns` pedig a névfeloldó beállításokat adja meg. A plugin a `host-local` IPAM modul segítségével a megadott alhálózatból oszt IP-címet, és a hoszton egy lokális fájlban tárolja a lefoglalt címeket. A CNI pluginok gyakran láncolhatók: például egy bridge plugin létrehozza a kapcsolatot, egy bandwidth plugin korlátozza a sávszélességet, egy portmap plugin pedig a port publishingot kezeli. A CNI tehát nem csupán egyetlen hálózati meghajtó, hanem egy bővíthető csővezeték.

A Kubernetes világában a CNI a konténerhálózat szabványos rétege; a kubelet a pod hálózati beállításakor CNI-konfigurációt használ, és a választott CNI-plugin (például Calico, Cilium, Flannel) határozza meg a tényleges hálózati modellt. A CNI specifikáció verziói idővel változnak, ezért a konkrét mezőket és viselkedést mindig az adott plugin aktuális dokumentációjával együtt kell ellenőrizni.

20.3.6 6. Service discovery és DNS a konténerhálózatban

A konténerek IP-címei dinamikusan változnak: újraindításkor, áthelyezéskor vagy skálázáskor a konténer más címet kaphat. Az alkalmazásoknak ezért nem szabad IP-címeket beégetniük;

service discovery mechanizmusra van szükség. A legegyszerűbb és legelterjedtebb megoldás a DNS-alapú névfeloldás.

A konténer-futásidők a hálózathoz beépített DNS-kiszolgálót rendelnek. Ennek címe jellemzően a bridge IP-címe, például `127.0.0.11` a Docker beépített DNS-énél, vagy a Kubernetes cluster DNS szolgáltatásának virtuális IP-címe. A konténer `/etc/resolv.conf` fájljába a futásidő ezt a névszervert írja be. Amikor a konténer egy másik konténert a nevén szólít meg, a DNS-kiszolgáló a konténer nevét vagy a hálózati aliast a dinamikusan kiosztott IP-címre oldja fel.

A service discovery kétirányú: a névfeloldáson túl a konténer indításakor a futásidő környezeti változókban is átadhatja a társak elérhetőségét, de ez a régi módszer ma már kevésbé ajánlott, mert a változók csak az indítás pillanatában érvényesek, és nem követik a változásokat. A DNS-alapú feloldás ezzel szemben minden új kapcsolat előtt friss eredményt adhat, feltéve, hogy a DNS-rekordok frissítése megfelelően működik.

A Kubernetes ennél eggyel magasabb absztrakciót vezet be: a **Service** objektum egy stabil virtuális IP-címet (ClusterIP) és DNS-nevet rendel egy podhalmazhoz. A Service mögött a kube-proxy vagy a hálózati plugin terheléselosztást valósít meg, így a konténer a Service nevét használja, és a kérések a mögöttes podok között oszlanak meg. A Service discovery tehát nem csupán a névfeloldást jelenti, hanem a stabil belépési pont és a terheléselosztás együttesét is.

A DNS a konténerhálózat kritikus szolgáltatása, ezért a hibakeresésnél mindig ellenőrizni kell:

- a konténer `/etc/resolv.conf` fájlját,
- a DNS-kiszolgáló elérhetőségét a konténerből,
- a névfeloldás késleltetését és a negatív gyorsítótár viselkedését,
- azt, hogy a DNS-név a megfelelő hálózatra vagy Service-re mutat-e.

20.3.7 7. Hálózati policy: forgalom engedélyezése és tiltása

A hálózati izoláció és a service discovery önmagában nem elég, ha a konténerek közötti forgalmat szabályozni kell. A **hálózati policy** deklaratív módon írja le, hogy egy adott konténerhalmaz milyen bejövő (ingress) és kimenő (egress) forgalmat fogadhat el vagy kezdeményezhet. A policy a hálózati tűzfal vezérlősíkja: maga a policy nem szűri a csomagokat, hanem a CNI-plugin vagy a hálózati adatsík értelmezi és valósítja meg.

Egy Kubernetes NetworkPolicy szemléltető példája:

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: web-allow-db
spec:
  podSelector:
    matchLabels:
      app: db
  policyTypes:
  - Ingress
  ingress:
  - from:
    - podSelector:
        matchLabels:
          app: web
    ports:
    - protocol: TCP
      port: 5432
```

Ez a policy azt mondja ki, hogy a `db` címkével ellátott podokba kizárólag a `web` címkéjű podoktól érkezhet bejövő forgalom, és csak a `5432` TCP-portra. Az összes többi bejövő kapcsolat tiltott. Az egress irány hasonlóan szabályozható. A policy tehát az L3/L4 szintű tűzfalat deklaratív, verziózható és felülvizsgálható formában írja le.

A hálózati policy hatékonysága attól függ, hogy a hálózati adatsík hogyan hajtja végre. A Calico iptables vagy eBPF alapú végrehajtást használ, a Cilium közvetlenül eBPF programokkal szűr, a Weave saját szabályrendszert generál. A policy szemantikája minden esetben a szelektorok és a szabályok egyeztetésén alapul; a csomagok feldolgozása a policy láncokban történik. A policy-k számának növekedésével a szűrés költsége nőhet, ezért a nagy klaszterekben a policy-k számát és a szabályok bonyolultságát a teljesítmény mérésével együtt kell nyomon követni.

Fontos különbség, hogy a hálózati policy önmagában nem titkosítja a forgalmat; az L3/L4 szintű hozzáférés-vezérlést ad, de a forgalom integritását és bizalmasságát külön rétegnek, például mTLS-nek vagy VPN-nek kell biztosítania. A policy és a titkosítás együtt alkotják a konténerhálózat biztonsági alapját.

20.3.8 8. Tipikus hibák és diagnosztikai gondolkodásmód

A konténerhálózatok hibái gyakran nem a hálózati kódban, hanem a konfigurációban, a címzésben vagy a rétegek közötti kölcsönhatásban gyökereznek. A leggyakoribb hibák:

1. **IP-cím ütközés vagy rossz alhálózat.** Ha két konténer azonos IP-címet kap, vagy a konténer alhálózata átfed a hoszt hálózatával, a forgalom kiszámíthatatlanul viselkedik, és a kapcsolatok időnként megszakadnak. A megoldás a címkiosztó modul (IPAM) konfigurációjának ellenőrzése és a hoszt útvonaltáblájának áttekintése.
2. **MTU-eltérés.** Az overlay hálózat fejléctöbblete miatt a belső MTU kisebb, mint a fizikai hálózaté. Ha a konténerben 1500 bájtos MTU marad, a nagy TCP-szegmensek eldobódnak, miközben a ping működik. A tünet jellegzetes: a kis HTTP-kérések sikeresek, a nagy fájlátvitel megakad. A megoldás a megfelelő MTU beállítása az overlay interfészeken és a konténerekben, vagy a path MTU discovery engedélyezése.
3. **DNS-feloldási hiba.** A konténer nem éri el a beépített DNS-kiszolgálót, mert a `/etc/resolv.conf` hibás, a bridge IP-címe megváltozott, vagy a tűzfal blokkolja az 53-as UDP/TCP-portot. A megoldás a névfeloldás lépésről lépésre történő ellenőrzése: először IP-címmel, majd névvel.
4. **Hálózati policy túl szigorú vagy nem elég specifikus.** A policy-k gyakran blokkolják a szükséges forgalmat, például a DNS-t vagy a health checket. A hibakeresésnél érdemes a policy-kat egyesével letiltani, és a csomagok útját naplózni. A policy-k alapértelmezett viselkedése is eltér: egyes rendszerekben minden engedélyezett, amíg nincs policy, máshol a default deny érvényesül.
5. **NAT conntrack tábla telítődés.** Nagy kapcsolatszám esetén a conntrack tábla megtelik, és új kapcsolatok nem jönnek létre. A tünet a `nf_conntrack: table full` naplóbejegyzés. A megoldás a tábla méretének növelése vagy a kapcsolatok számának csökkentése.

A diagnosztikai gondolkodásmód mindig a rétegek sorrendjét kövesse: fizikai/virtuális interfész, L2 (MAC, ARP), L3 (IP, útvonal), L4 (port, tűzfal), L7 (DNS, protokoll). Ez a sorrend segít a hiba pontos lokalizálásában.

20.3.9 9. Labor: konténerhálózati topológia tervezése

A labor célja, hogy az olvasó emulált környezetben saját kezűleg hozzon létre egy konténerhálózati topológiát, és mérje a hálózati jellemzőket. A labor nem éles gépen fut; a parancsok egy eldobható virtuális gépben vagy konténerben, például QEMU/KVM vagy Vagrant alatt, saját hálózati névterekben hajthatók végre.

Kiinduló állapot. Egy Linux hoszt, amelyen rendelkezésre áll az `ip`, `ping`, `iperf3`, `dig` és `nftables` eszközkészlet. A hoszt hálózati konfigurációja érintetlen marad; a labor minden objektumot saját network namespace-ben hoz létre.

Topológia. Két konténer szerepet játszó network namespace: `web` és `db`. A `web` namespace egy veth pár `veth-web` végén fut, a `db` a `veth-db` végén. A veth párok hoszt oldali végei a `br-lab` nevű Linux bridge-hez csatlakoznak. A bridge IP-címe `10.77.0.1/24`, a `web` cím `10.77.0.10/24`, a `db` cím `10.77.0.11/24`. A `web` namespace-ből kimenő forgalom NAT-ot kap a hoszt külső interfészén. A `db` namespace-be kívülről csak a hoszt `15432` portjáról lehet eljutni a `5432` portra.

Konfigurációs parancsok (illusztráció, emulált környezetben futtatható):

```
# Network namespace-ek létrehozása
ip netns add web
ip netns add db

# Bridge létrehozása és IP-cím
ip link add name br-lab type bridge
ip addr add 10.77.0.1/24 dev br-lab
ip link set br-lab up

# veth párok
ip link add veth-web type veth peer name veth-web-br
ip link add veth-db type veth peer name veth-db-br

# Végek áthelyezése a namespace-ekbe
ip link set veth-web netns web
ip link set veth-db netns db

# Hoszt oldali végek a bridge-hez
ip link set veth-web-br master br-lab
ip link set veth-db-br master br-lab
ip link set veth-web-br up
ip link set veth-db-br up

# Címek és interfészek a namespace-ekben
ip -n web addr add 10.77.0.10/24 dev veth-web
ip -n web link set veth-web up
ip -n web route add default via 10.77.0.1

ip -n db addr add 10.77.0.11/24 dev veth-db
ip -n db link set veth-db up
ip -n db route add default via 10.77.0.1
```

A NAT beállítása a labor namespace-ekre korlátozódik; a hoszt globális tűzfalszabályait nem módosítjuk. A `nftables` szabályok csak a `web` namespace forgalmát masqueradelik, és a `db` namespace bejövő portját nyitják meg.

Elvárt eredmény. A `web` és `db` namespace-ek egymást pingelik, a `web` kimenő DNS-feloldása működik, a `db` a hoszt `15432` portján keresztül érhető el kívülről. A mérési naplóban a várt és a mért értékeket külön kell feltüntetni.

Mérési terv. A laborban három mérőszámot érdemes rögzíteni:

- belső ICMP RTT a `web` és `db` között,
- TCP átviteli sebesség a két namespace között `iperf3` segítségével,
- DNS-feloldási idő a `web` namespace-ből egy helyi DNS-névre.

A méréshez rögzíteni kell a környezetet: CPU típusa, memória mérete, kernelverzió, a mérés időpontja, a mérési módszer (például 10 ping átlaga, 10 másodperces iperf3). Az alábbi táblázat egy lehetséges, ellenőrzött laborban mért eredménysort mutat be; a konkrét értékek a futtató környezettől függenek, ezért a saját mérést kell alapul venni.

Mérőszám	Várt érték	Mért érték	Mérési módszer
ICMP RTT `web` → `db`	< 1 ms	pl. 0,18 ms	10 ping átlaga
TCP throughput `web` → `db`	> 1 Gbit/s	pl. 1,72 Gbit/s	iperf3, 10 s
DNS-feloldási idő	< 10 ms	pl. 2,1 ms	`dig`, 5 mérés átlaga

A várt értékek a veth és bridge architektúrából következnek: a csomag nem hagyja el a hoszt memóriáját, ezért az RTT-nek mikroszekundumos vagy szub-milliszekundumos tartományban kell lennie. A TCP átvitel a kernel memóriájában másolt csomagok sebességétől függ, ezért modern hosztokon a Gbit/s tartomány tipikus. Ha a mért érték jelentősen eltér, akkor MTU-, CPU- vagy konfigurációs problémát kell keresni.

Napló és visszaállítás. A labor végén a létrehozott objecteket el kell távolítani:

```
ip netns del web
ip netns del db
ip link del br-lab
```

A visszaállítás után a hoszt hálózati állapota nem változik. A mérési napló a kiinduló állapotot, a futtatott parancsokat, a mért értékeket és a visszaállítás megtörténtét rögzíti.

20.3.10 10. Összefoglalás és önellenőrzés

A konténerhálózat a hálózati izoláció és az összekapcsolás egyensúlyát valósítja meg. A network namespace és a veth pár a konténeren belüli hálózati látványt, a bridge a hoszton belüli összekapcsolást, a NAT a külvilág elérését, az overlay hálózat a több hoszton átívelő L2 szegmenst, a CNI a szabványos csatlakozófelületet, a DNS a dinamikus címek mögötti névfeloldást, a hálózati policy pedig a forgalom engedélyezését adja. A rétegek egymásra épülnek, és a hibák gyakran a rétegek közötti illeszkedésnél jelentkeznek.

Önellenőrző kérdések:

1. Milyen szerepet játszik a veth pár a konténer és a hoszt közötti kapcsolatban? Hogyan befolyásolja a bridge a konténerek L2 láthatóságát?
2. Miért van szükség NAT-ra a bridge hálózatban, és milyen szerepe van a conntrack táblának?
3. Hogyan oldja meg a VXLAN a több hoszton futó konténerek összekapcsolását? Milyen MTU-következményei vannak?
4. Mi a CNI szerepe a konténer-futásidő és a hálózati plugin között, és milyen parancsokat hív meg?
5. Mi a különbség a DNS-alapú service discovery és a hálózati policy között? Miért van szükség mindkettőre?

A következő fejezet (Hálózati biztonság és izoláció) a konténerhálózat biztonsági és izolációs kérdéseit viszi tovább: a hálózati policy-k végrehajtását, a titkosítást és a hálózati támadási felület csökkentését tárgyalja.

20.4 Hálózati biztonság és izoláció

A hálózati verem minden operációs rendszer egyik legnagyobb támadási felülete. A bejövő csomagok már a rendszerhívási határ előtt, a kernel protokoll-implementációiban és az eszközmeghajtókban is feldolgozásra kerülnek; egy hibás TCP-szegmens, egy rosszindulatú DNS-válasz vagy egy hamis IPv6-router advertisement elegendő lehet ahhoz, hogy a támadó kódot futtasson, szolgáltatást bénítson meg vagy adatokat szivárogtasson ki. Ugyanakkor a hálózat az a hely, ahol a forgalmat központilag lehet szűrni, naplózni és irányítani, mielőtt az a végponthoz érne. Ebben a fejezetben azt vizsgáljuk, hogyan alakítható a hálózat védelmi ponttá, és milyen eszközökkel csökkenthető a kitettség: VLAN-okkal, VRF-ekkel, security groupokkal, zero trust elvű szegmentációval, valamint hálózati naplózással és IDS/IPS rendszerekkel.

20.4.1 1. A hálózat mint támadási felület és védelmi pont

A **támadási felület** a rendszer azon elemeinek összessége, amelyek potenciálisan ellenséges környezetből elérhetők, és így bemenetet kaphatnak. A hálózat esetében ez magában foglalja a nyitott portokat, a protokoll-implementációkat, a hálózati eszközöket, a felügyeleti interfészeket, a DNS- és DHCP-szolgáltatásokat, valamint a felhasználói alkalmazásokat. Minél több elem érhető el, annál nagyobb a valószínűsége, hogy valamelyikben hiba van, vagy hogy a támadó jogosulatlanul hozzáfér.

A hálózati támadási felület rétegekbe szervezhető:

- **Fizikai és adatkapcsolati réteg:** vezetékes vagy vezeték nélküli hozzáférés, ARP-hamisítás, kapcsolókonfiguráció hibái, 802.1Q-címke manipuláció.
- **Hálózati réteg:** IP-hamisítás, útvonal-manipuláció, fragmentációs támadások, ICMP-alapú felderítés.
- **Szállítási réteg:** portszkennelés, SYN-flood, session-hijacking, TCP-sorozatszám-találgatás.
- **Alkalmazási réteg:** protokollhibák, injekciós támadások, puffertúlcsordulás a hálózati szolgáltatásokban, hitelesítés nélküli végpontok.

A hálózat azonban nemcsak támadási felület, hanem **védelmi pont** is. Az adatforgalom áthalad a kapcsolókon, routereken, tűzfalakon és szoftveres hálózati veremben, így ezeken a pontokon lehetőség van a forgalom szűrésére, irányítására, naplózására és anomáliák észlelésére. A hálózati védelem rétegei:

- **Határvédelmi tűzfal:** a külső és belső hálózat határán áll, és alapértelmezés szerint minden forgalmat tilt, kivéve a kifejezetten engedélyezett kapcsolatokat.
- **Belső szegmentáció:** a belső hálózatot kisebb zónákra osztja, hogy egy kompromittált gép ne érhessen el minden erőforrást.
- **Hálózati naplózás:** a forgalmi események rögzítése későbbi elemzéshez, incidenskezeléshez és jogi megfelelőséghez.
- **Behatolásérzékelő és -megelőző rendszerek (IDS/IPS):** ismert támadási minták vagy rendellenes viselkedés alapján riasztanak, illetve blokkolnak.

A védelem hatékonysága akkor a legnagyobb, ha ezek a rétegek együttműködnek. Egyetlen tűzfal sem képes minden támadást megállítani, de ha a szegmentáció korlátozza a mozgásteret, a naplózás pedig nyomot hagy a támadó tevékenységéről, a kár jelentősen csökkenthető.

20.4.2 2. Szegmentáció és izoláció: VLAN, VRF, security group és zero trust

A lapos hálózat – amelyben minden eszköz ugyanabban a broadcast domainben és ugyanabban az IP-alhálózatban van – rendkívül kényelmes, de rendkívül veszélyes is. Ha egy támadó kompromittál egyetlen gépet, onnan általában közvetlenül elérheti az összes többi erőforrást, portok sokaságát szkennelheti, és oldalirányban mozoghat. A **szegmentáció** célja ennek a mozgástérnek a csökkentése: a hálózatot kisebb, egymástól izolált zónákra bontjuk, és csak a szükséges forgalmat engedjük át a zónák között.

20.4.3 2.1. VLAN: izoláció az adatkapcsolati rétegben

A **VLAN** (Virtual Local Area Network, IEEE 802.1Q) az adatkapcsolati rétegbeli elkülönítés eszköze. Egy fizikai kapcsolón több virtuális LAN hozható létre; az egyes VLAN-ok külön broadcast domainként viselkednek, így a bennük lévő eszközök nem látják egymás adatforgalmát, hacsak a forgalmat nem irányítjuk át egy routeren vagy L3 kapcsolón.

A VLAN-ok használatának alapfogalmai:

- **Access port:** a port egyetlen VLAN-hoz tartozik, a rá csatlakozó eszköz nem tud a VLAN-címkéről. A kapcsoló a portra érkező kereteket az adott VLAN-hoz rendeli.
- **Trunk port:** több VLAN forgalmát továbbítja, a kereteket 802.1Q-címkével látja el, amely tartalmazza a VLAN-azonosítót (VID).
- **Natív VLAN:** trunk porton a címkézetlen kereteket fogadó VLAN. Alapértelmezésben sok kapcsolón ez az 1-es VLAN, ami biztonsági kockázatot jelenthet.

Tipikus hibák és helyes megoldások:

- **VLAN hopping:** ha egy trunk port natív VLAN-jét úgy konfigurálják, hogy az megegyezzen egy access port VLAN-jával, a támadó kettős címkézéssel vagy a natív VLAN kihasználásával átléphet a VLAN-határon. Helyes megoldás: a natív VLAN legyen dedikált, nem használt VLAN, és a trunk portokon minden keret címkézett legyen.
- **Túlzott trunk kitettség:** ha egy végfelhasználói port trunk portként van konfigurálva, a támadó saját VLAN-címkéket injektálhat. Helyes megoldás: a végponti portok legyenek access portok, és a trunk portok csak a szükséges VLAN-okat engedjék át.
- **Nem használt VLAN-ok elhagyása:** a kapcsolón maradt régi, felügyelet nélküli VLAN-ok könnyen támadási felületet képeznek. Helyes megoldás: rendszeres audit, a felesleges VLAN-ok törlése.

A VLAN önmagában **nem** nyújt biztonsági határt, mert a címke a keret része, és egy rosszul konfigurált kapcsolón a támadó manipulálhatja. A VLAN-t biztonsági célra csak akkor szabad használni, ha a kapcsoló konfigurációja zárt, a trunk portok ellenőrzöttek, és a VLAN-ok közötti forgalom egy tűzfalon halad át.

20.4.4 2.2. VRF: izoláció a hálózati rétegben

A **VRF** (Virtual Routing and Forwarding) a hálózati rétegbeli izoláció eszköze. Minden VRF saját útválasztási táblával rendelkezik, így az egyik VRF-ben lévő útvonalak nem láthatók a másikban. Ugyanazon a fizikai routeren több VRF futhat, és a forgalom akkor sem keveredik, ha az IP-címek átfednek.

A VRF tipikus felhasználása:

- **Ügyfél-izoláció szolgáltatóknál:** egy internetszolgáltató ugyanazon a fizikai infrastruktúrán több ügyfelet szolgál ki, de az ügyfelek forgalma nem keveredhet.
- **Belső zónák szétválasztása:** például a termelési, tesztelési és vendéghálózat külön VRF-be kerül.
- **Felügyeleti hálózat:** a menedzsment interfészek forgalma elkülönül az adatforgalomtól.

A VRF a VLAN-nál erősebb izolációt nyújt, mert az útválasztási táblák teljesen függetlenek, és a forgalom áthaladásához explicit útvonalra van szükség. A VRF-ek közötti forgalmat csak úgy lehet átengedni, ha a routeren útvonal-szivárgást konfigurálunk, ami tűzfallal együtt használható.

20.4.5 2.3. Security group

A **security group** a felhőalapú virtualizációban és a szoftveresen definiált hálózatokban használt stateful tűzfalszabály-gyűjtemény. Egy security group hozzárendelhető egy virtuális géphez, hálózati interfészhez vagy konténerhez, és meghatározza, hogy milyen bejövő és kimenő forgalom engedélyezett.

Példa egy háromrétegű webalkalmazás security groupjaira:

- **Web zóna:** engedélyezett bejövő forgalom: TCP/443 mindenhonnan; kimenő: csak az alkalmazás zóna TCP/8080 portjára.
- **Alkalmazás zóna:** engedélyezett bejövő: TCP/8080 a web zónából; kimenő: csak az adatbázis zóna TCP/5432 portjára.
- **Adatbázis zóna:** engedélyezett bejövő: TCP/5432 az alkalmazás zónából; kimenő: nincs vagy csak a frissítési szerverre.

A security group előnye, hogy **címkére**, nem IP-címre épül: a szabályok hivatkozhatnak más security groupokra, így a virtuális gépek IP-címének változása nem igényli a szabályok módosítását. A stateful jelleg azt jelenti, hogy a bejövő kapcsolatra adott válaszforgalom automatikusan engedélyezett, így a szabályok csak a kezdeményező irányt tartalmazzák.

A security group akkor biztonságos, ha az alapértelmezés **tiltás**, és csak a minimálisan szükséges portok és források vannak engedélyezve. Gyakori hiba, hogy a 0.0.0.0/0-ról minden portot megnyitnak a tesztelés kedvéért, majd elfelejtik szűkíteni, vagy hogy az adminisztrációs port (például SSH/22) mindenhonnan elérhető. Helyes megoldás: menedzsment hálózat külön szegmensben, erős hitelesítéssel, és a security groupban csak a menedzsment alhálózatból engedélyezett az SSH.

20.4.6 2.4. Zero trust

A **zero trust** biztonsági modell alapelve: **soha ne bízz meg a hálózati helyben**. A hagyományos perimétervédelem abból indult ki, hogy a belső hálózat megbízható, a külső pedig nem. A zero trust ezt elveti: a támadó lehet már a belső hálózaton, ezért minden egyes erőforrás-elérést külön kell hitelesíteni, engedélyezni és naplózni.

A zero trust alapelvei:

- **Explicit ellenőrzés:** minden hozzáférési kérést a lehető legtöbb jel (identitás, eszköz állapota, hely, idő, viselkedés) alapján kell kiértékelni.
- **Legkisebb jogosultság:** a felhasználó vagy szolgáltatás csak a feladatához minimálisan szükséges erőforrásokhoz férhet hozzá, és csak annyi ideig.
- **Folyamatos monitorozás:** a hálózati forgalmat folyamatosan figyelni kell, és a rendellenességekre azonnal reagálni.
- **Mikroperiméter:** az erőforrásokat apró, egymástól izolált szegmensekbe kell szervezni, így egy kompromittált elem nem éri el az összes többit.

A gyakorlatban a zero trust megvalósítása olyan technológiákat igényel, mint a kölcsönös TLS (mTLS) a szolgáltatások közötti hitelesítéshez, az identitásalapú hozzáférés-kezelés (IAM), a rövid élettartamú tokenek, valamint a hálózati mikroperiméterek és a hálózati naplózás. A zero trust nem egyetlen termék, hanem egy tervezési filozófia, amely a hálózati izolációt a hagyományos tűzfalaktól a szolgáltatások szintjéig viszi.

20.4.7 3. Hálózati naplózás és IDS/IPS

A hálózati védelem nem ér véget a szűrésnél: a forgalmi események rögzítése és elemzése nélkül nem tudjuk, hogy mi történt, és nem tudunk időben reagálni. A **hálózati naplózás** és a **behatolásérzékelés/megelőzés** egymást kiegészítő eszközök.

20.4.8 3.1. Hálózati naplózás

A hálózati naplóforrások közé tartoznak:

- **Flow rekordok** (NetFlow, IPFIX): összesített forgalmi statisztikák, amelyek tartalmazzák a forrás és cél IP-címet, portot, protokollt, bájtszámot és időbélyeget. Nem tartalmazzák a csomagok tartalmát, de alkalmasak a forgalmi minták elemzésére.
- **Tűzfal naplók:** a tűzfal által engedélyezett és blokkolt kapcsolatok rekordjai, gyakran a szabály azonosítójával.
- **DNS-lekérdezési naplók:** a kliens által kért domainnevek, amelyek segítségével felderíthetők a rosszindulatú domainekhez való kapcsolódások.
- **Rendszernaplók:** a hálózati szolgáltatások (például sshd, webkiszolgáló) eseményei, például sikeres és sikertelen bejelentkezések.

A naplózás akkor hasznos, ha:

- **Időbélyeg szinkronizált:** minden naplóforrás ugyanazt az időreferenciát használja (NTP). Enélkül az események korrelációja lehetetlen.
- **Normalizált:** a naplók egységes formátumban érkeznek egy központi naplózó rendszerbe, ahol szűrhetők és indexelhetők.
- **Megőrzési idő meghatározott:** a naplókat a szabályzatnak megfelelő ideig meg kell őrizni, de a régi adatok törölhetők.

- **Riasztások konfiguráltak:** a fontos mintákra (például sok sikertelen bejelentkezés, portszkennelés, nagy adatforgalom éjszaka) automatikus értesítést kell beállítani.

Tipikus naplózási hibák:

- **Túl kevés napló:** csak a tűzfal blokkolja a forgalmat, de nem naplózza, így a támadó tevékenysége láthatatlan marad.
- **Túl sok napló:** minden csomag naplózása hatalmas adatmennyiséget termel, amelyben a fontos esemény elveszik. A naplózást célzottan, a legnagyobb kockázatú pontokra kell fókuszálni.
- **Nincs időszinkron:** az eszközök órája eltér, a korreláció megbízhatatlan.
- **Nincs riasztás:** a naplók gyűlnek, de senki nem figyeli őket, így a behatolás csak napokkal később derül ki.

20.4.9 3.2. IDS/IPS

Az **IDS** (Intrusion Detection System) érzékeli a gyanús tevékenységet és riaszt, de nem avatkozik be a forgalomba. Gyakran egy span (porttükrözés) porton keresztül látja a forgalmat, így a forgalom nem halad át rajta. Az **IPS** (Intrusion Prevention System) inline üzemel: a forgalom áthalad rajta, és képes blokkolni a gyanús csomagokat.

Érzékelési módszerek:

- **Szignatúra alapú:** ismert támadási mintákat keres a forgalomban, például egy adott exploit jellegzetes bájtjait vagy egy ismert rosszindulatú domainre irányuló DNS-lekérdezést. Előnye a pontosság, hátránya, hogy az ismeretlen támadásokat nem érzékeli.
- **Anomália alapú:** a normál forgalmi bázisvonalhoz képest rendellenes eltéréseket keres, például szokatlanul nagy adatforgalmat egy adott gépről. Előnye, hogy új támadásokat is észlelhet, hátránya a magas téves pozitív arány.
- **Reputáció alapú:** ismert rosszindulatú IP-címek, domainek vagy fájlok listája alapján dönt.

Az IDS/IPS elhelyezése kritikus: általában a hálózati belépési ponton, a kritikus szegmensek előtt és a belső zónák határain helyezik el. A szignatúrák frissítése folyamatos feladat; az elavult szignatúrák false negative eredményhez vezetnek.

Tipikus hibák:

- **Túl sok false positive:** a szignatúrák túlzottan általánosak, ami riasztási fáradtsághoz vezet, és a valódi riasztásokat figyelmen kívül hagyják. Helyes megoldás: a szignatúrák finomhangolása, a riasztások súlyozása.
- **Titkosított forgalom vizsgálatának hiánya:** a HTTPS vagy mTLS forgalom tartalma nem látható az IDS számára, így a támadó rejtve marad. Helyes megoldás: a végponton történő vizsgálat vagy a forgalmi metaadatok elemzése.
- **Nincs kitérési útvonal:** az IPS inline üzemben maga is támadási felületté válhat, és ha leáll, a teljes hálózati forgalom megakad. Helyes megoldás: redundancia, és az IPS úgy konfigurálása, hogy hiba esetén a forgalom továbbhaladjon (fail-open), vagy a kritikus szegmensen ne legyen egyedüli hibapont.

Egy egyszerű IDS-szabály példa (pszeudokód, nem végrehajtandó utasítás):

```
# SSH brute-force érzékelése
alert tcp any any -> any 22 (msg:"SSH brute force";
  flow:to_server,established;
  content:"SSH-2.0";
  threshold:type both, track by_src, count 5, seconds 60;
  sid:1000001; rev:1;)
```

Ez a szabály nem ad támadási útmutatót; éppen ellenkezőleg, egy védelmi mechanizmust szemléltet: ha egy forrás 60 másodpercen belül öt SSH-kapcsolatot kezdeményez, az IDS riaszt. A valós szabályok pontos szintaxisa az adott IDS/IPS terméktől függ, és a friss verziókhoz a gyártói dokumentációt ellenőrizni kell.

20.4.10 4. Gyakorlati minta: szegmentációs és tűzfalterv

Vegyünk egy háromrétegű webalkalmazást, amelyet a korábbi fejezetekben már bemutattunk. A cél az, hogy a hálózati kitettséget minimálisra csökkentsük, és a zónák közötti forgalmat ellenőrizzük.

A topológia:

```
[Internet] --> [Web zóna] --> [Alkalmazás zóna] --> [Adatbázis zóna]
                   |                  |                    |
                   |                  |                    |
              TCP/443            TCP/8080             TCP/5432
```

A tervezett tűzfalszabályok:

Forrás	Cél	Port	Művelet
Internet (0.0.0.0/0)	Web zóna	TCP/443	Engedélyezett
Web zóna	Alkalmazás zóna	TCP/8080	Engedélyezett
Alkalmazás zóna	Adatbázis zóna	TCP/5432	Engedélyezett
Minden más	Minden más	–	Tiltott és naplózott

A tervben a menedzsment hálózat külön szegmensben van, és az SSH-hozzáférés csak onnan engedélyezett. A web zóna nem érheti el közvetlenül az adatbázist, az adatbázis pedig semmit nem kezdeményezhet kifelé.

Tipikus hibák ebben a tervben, és a helyes megoldások:

1. **Túl tág szabály:** “allow any to any” a web zónából. Ez lehetővé teszi, hogy a web szerver közvetlenül elérje az adatbázist, és ha kompromittálják, a támadó azonnal hozzáfér az adatokhoz. Helyes megoldás: a web zónából csak az alkalmazás zóna TCP/8080 portja engedélyezett.

2. **Alapértelmezett engedélyezés:** a tűzfal alapértelmezésben engedélyezi a forgalmat, és csak bizonyos dolgokat tilt. Ez veszélyes, mert egy új szolgáltatás elfelejtett szabályozás nélkül azonnal elérhetővé válik. Helyes megoldás: az alapértelmezés **tiltás**, és minden engedélyezett forgalmat kifejezetten fel kell sorolni.

3. **Naplózás hiánya a blokkolt forgalomnál:** ha a tűzfal blokkolja a forgalmat, de nem naplózza, a támadó próbálkozásai láthatatlanok maradnak. Helyes megoldás: a blokkolt forgalmat naplózni kell, de a naplómennyiség korlátozására csak a gyanús mintákat érdemes riasztani.

4. **Adminisztráció nyitott portja:** az SSH/22 mindenhonnan elérhető. Ez lehetővé teszi a brute-force támadást. Helyes megoldás: a menedzsment hálózat külön szegmensben, erős hitelesítéssel, és a security groupban csak a menedzsment alhálózatból engedélyezett az SSH.
5. **Lapos hálózat:** nincs szegmentáció, minden gép ugyanabban az alhálózatban van. Egy kompromittált gép oldalirányban elérheti az összes többit. Helyes megoldás: VLAN/VRF vagy security group alapú zónák kialakítása.

20.4.11 5. Labor: szegmentációs és tűzfalterv készítése

A labor célja, hogy az olvasó emulált környezetben hozzon létre egy háromzónás hálózatot, konfiguráljon tűzfalszabályokat, majd mérje és dokumentálja az elérhetőséget. A labor **csak emulált környezetben fut**, a hoszt valódi hálózati konfigurációját nem módosítja.

20.4.12 5.1. Kiinduló állapot

A hoszt rendszeren Linux fut, és rendelkezésre állnak a `iproute2`, `nftables` és `ping` eszközök. A labor három hálózati névteret hoz létre: `web`, `app` és `db`, amelyek egy Linux bridge-hez csatlakoznak. Kezdetben nincs tűzfalszabály, így a névterek között minden forgalom engedélyezett.

20.4.13 5.2. Build- és futtatási parancsok

Az alábbi parancsok csak példák, és kizárólag eldobható emulált környezetben hajtandók végre. A hoszt valódi hálózatát nem szabad módosítani.

```
# Hálózati névterek létrehozása (példa, nem éles gépen)
ip netns add web
ip netns add app
ip netns add db

# Bridge létrehozása
ip link add vnet0 type bridge
ip link set vnet0 up

# Virtuális interfészek hozzáadása
ip link add veth-web type veth peer name veth-web-br
ip link set veth-web netns web
ip link set veth-web-br master vnet0
ip link set veth-web-br up
# Hasonlóan app és db számára

# IP-címek kiosztása
ip netns exec web ip addr add 10.0.0.2/24 dev veth-web
ip netns exec app ip addr add 10.0.1.2/24 dev veth-app
ip netns exec db ip addr add 10.0.2.2/24 dev veth-db

# Alapértelmezett útvonalak (szükség szerint)
ip netns exec web ip route add default via 10.0.0.1
# ...
```

A névterek közötti forgalom kezdetben korlátozás nélküli. A feladat: a fenti zónaterv szerint tűzfalszabályokat létrehozni az `nftables` segítségével. A szabályok a hoszt bridge-én keresztül szűrik a forgalmat.

Példa `nftables` konfiguráció (szemléltetés, nem végrehajtandó éles gépen):

```
table inet filter {
    chain forward {
        type filter hook forward priority 0; policy drop;

        # Web --> App: csak 8080
        ip saddr 10.0.0.0/24 ip daddr 10.0.1.0/24 tcp dport 8080 accept

        # App --> DB: csak 5432
        ip saddr 10.0.1.0/24 ip daddr 10.0.2.0/24 tcp dport 5432 accept

        # Válaszforgalom (stateful)
        ct state established,related accept

        # Minden más tiltott és naplózott
        log prefix "drop: " drop
    }
}
```

A szabályok betöltése után a forgalmat ellenőrizni kell.

20.4.14 5.3. Mérési napló és elvárt eredmény

A mérés során minden zónapár közötti kapcsolatot tesztelünk, és rögzítjük a várt és a mért eredményt. A mérési napló táblázatba foglalható:

Teszt	Várt eredmény	Mért eredmény	Megjegyzés
`web` -> `app` TCP/8080	Engedélyezett	Engedélyezett	–
`web` -> `app` TCP/22	Tiltott	Tiltott	naplózva
`app` -> `db` TCP/5432	Engedélyezett	Engedélyezett	–
`db` -> `app` TCP/8080	Tiltott	Tiltott	stateful szűrés
`web` -> `db` TCP/5432	Tiltott	Tiltott	–
Internet -> `web` TCP/443	Engedélyezett*	Engedélyezett*	* a laborban szimulálva

Az utolsó sorban az internet felőli forgalmat egy negyedik névtérrel vagy a hoszt interfészéről lehet szimulálni; a laborban elegendő a zónák közötti szabályok ellenőrzése.

A **mérőszám** ebben a laborban a nem engedélyezett kapcsolatok blokkolási aránya: hány tiltott kapcsolatkísérletből hányat blokkolt a tűzfal. Az elvárt érték 100%, de a mérés során ellenőrizni kell, hogy a naplóban szerepelnek-e a blokkolt események.

20.4.15 5.4. Visszaállítás

A labor után a névtereket és a szabályokat el kell távolítani:

```
ip netns del web
ip netns del app
ip netns del db
ip link del vnet0
# nftables szabályok törlése
nft flush ruleset
```

A visszaállítás után a hoszt hálózati konfigurációja változatlan.

20.4.16 6. Összefoglalás és önellenőrző kérdések

A hálózat egyszerre támadási felület és védelmi pont. A támadási felület csökkentése érdekében a hálózatot szegmentálni kell VLAN-okkal, VRF-ekkel vagy security groupokkal, és a zero trust elveit kell alkalmazni. A hálózati naplózás és az IDS/IPS biztosítja a láthatóságot és a gyors reagálást. A szegmentációs tervben minden engedélyezett forgalmat kifejezetten fel kell sorolni, az alapértelmezés pedig tiltás. A naplózást célzottan kell beállítani, és a blokkolt forgalmat rögzíteni kell. A helyes megoldás lényege a réteges védelem: ha egy réteg hibázik, a másik még foghat.

Mérőszám: a nem engedélyezett kapcsolatok blokkolási aránya a laborban (elvárt 100%).

Önellenőrző kérdések:

1. Mi a különbség a VLAN és a VRF között izoláció szempontjából? Melyik rétegben hatnak, és milyen támadási lehetőségeket csökkent?
2. Miért nem elegendő a hálózati tűzfal önmagában? Milyen szerepet játszik a naplózás és az IDS/IPS?
3. Hogyan segít a zero trust modell a belső fenyegetések ellen? Milyen gyakorlati intézkedéseket igényel?
4. Egy tűzfaltervben miért veszélyes az “allow any to any” szabály? Milyen helyettesítő megoldást javasolsz?

A következő fejezet (Felhasználók, csoportok és jogosultságok) a felhasználók, csoportok és jogosultságok témáját tárgyalja, és épít az itt bevezetett izolációs és védelmi fogalmakra. A hálózati biztonság és a rendszer szintű hozzáférés-kezelés együttesen alkotja a védelmi architektúra alapját.

21 Biztonság és izoláció

21.1 Felhasználók, csoportok és jogosultságok

A hagyományos UNIX-jogosultsági modell a rendszer erőforrásainak hozzáférés-vezérlését néhány egyszerű, de rugalmasan kombinálható alapfogalomra vezeti vissza. A modell alanyai a felhasználók és csoportok, tárgyai pedig elsősorban a fájlok és könyvtárak; a kettő közötti hozzáférést a mode bitek és azok kiterjesztése, a POSIX ACL szabályozza. Bár a modern rendszerekben már léteznek finomabb mechanizmusok is (capabilities, SELinux, AppArmor), a hagyományos modell ma is meghatározza a mindennapi adminisztrációt, a kompatibilitást és a biztonsági elemzések alaprétegét. Ebben az alfejezetben a modell alapfogalmait, az ellenőrzés algoritmusát, a setuid/setgid viselkedést, az ACL-t és a gyakorlati hibákat tárgyaljuk, majd egy reprodukálható laborral zárjuk.

21.1.1 1. Alanyok és azonosítók: UID, GID

A hagyományos UNIX-modellben minden folyamat rendelkezik egy felhasználói és egy vagy több csoportazonosítóval. A legalapvetőbb fogalmak:

- **UID (User ID):** a felhasználó numerikus azonosítója. A 0-s UID a hagyományos root felhasználót jelöli, aki a klasszikus modell szerint minden hozzáférési ellenőrzést megkerülhet. A gyakorlatban a root jogosultság kiemelt, ezért a modern rendszerek a capabilities réteggel próbálják finomítani – ezt a következő alfejezet tárgyalja.
- **GID (Group ID):** elsődleges csoportazonosító. Minden felhasználónak van egy elsődleges csoportja, amelyet a `/etc/passwd` tárol; ezen kívül további kiegészítő csoportokhoz is tartozhat. A folyamat a kiegészítő csoportok listáját örökli, és az erőforrás-hozzáférési döntéseknél a kernel mindegyik figyelembe veszi.
- **Alanyosztályok:** a hozzáférési modell három osztályt különböztet meg: **tulajdonos (owner)**, **csoport (group)** és **mások (others)**. A fájl tulajdonosát UID, csoportját GID azonosítja; a „mások” minden olyan felhasználót jelent, aki nem tulajdonos és nem csoporttag.

A folyamat belső UID/GID típusai fontosak a setuid viselkedés megértéséhez. A legfontosabbak:

- **Valós UID/GID (RUID, RGID):** azt jelzi, ki indította a folyamatot. A valós UID a login shellből vagy a szülő folyamatból származik.
- **Hatékony UID/GID (EUID, EGID):** a kernel ezt használja a legtöbb hozzáférési ellenőrzés során. Normál esetben megegyezik a valóssal, de setuid/setgid programoknál eltérhet.
- **Mentett UID/GID (saved UID, saved GID):** lehetővé teszi, hogy egy setuid program átmenetileg visszaváltson a valós UID-re, majd újra visszaállítsa a jogosultságot anélkül, hogy a setuid bitet újra be kellene állítani. Ez a POSIX `seteuid` és `setresuid` hívásokkal kezelhető.
- **Fájlrendszer-UID (FSUID):** általában megegyezik az EUID-vel, de a fájlhozzáférés-ellenőrzés során külön kezelhető, például NFS esetén.

A felhasználói és csoportinformációk hagyományosan a `/etc/passwd`, `/etc/group` fájlokban találhatók, de modern rendszerekben LDAP, SSSD vagy más névszolgáltatás is használható. A kernel szempontjából csak a numerikus UID/GID számít.

21.1.2 2. Mode bitek: rwx és a speciális bitek

Minden fájlhoz és könyvtárhoz három jogosultsági bit-csoport tartozik: olvasás (`r`), írás (`w`) és végrehajtás (`x`). A három osztályra (tulajdonos, csoport, mások) összesen kilenc bitet használunk. A mode biteket kétféleképpen ábrázolhatjuk:

- **Szimbolikus:** például `rwxr-xr--`, ahol az első három karakter a tulajdonos, a következő három a csoport, az utolsó három a mások jogait jelzi.
- **Numerikus (oktális):** például `0754`, ahol a három számjegy a tulajdonos (7 = rwx), a csoport (5 = r-x), a mások (4 = r–) jogait adja meg. Az oktális ábrázolásban a bitértékek: `r` = 4, `w` = 2, `x` = 1.

A jogosultsági bitek jelentése fájlokon és könyvtárakon eltér:

Bit	Fájl	Könyvtár
`r`	olvashatjuk a fájl tartalmát	listázhatjuk a könyvtár tartalmát (`ls`)
`w`	módosíthatjuk vagy törölhetjük a fájlt	létrehozhatunk, törölhetünk vagy átnevezhetünk bejegyzéseket a könyvtárban
`x`	futtathatjuk a fájlt (ha bináris vagy script)	beléphetünk a könyvtárba (`cd`), illetve elérhetjük a benne lévő fájlokat teljes útvonalon

Fontos: egy fájl törléséhez nem a fájl írási joga, hanem a fájlt tartalmazó könyvtár írási és végrehajtási joga szükséges. Ez a szemantika gyakran félreértés forrása.

A kilenc alapszintű biten kívül három speciális bit létezik:

- **setuid bit (SUID):** ha futtatható állományon be van állítva, akkor a folyamat hatékony UID-je a fájl tulajdonosának UID-jére változik. Példa: a `/usr/bin/passwd` setuid root program, hogy a felhasználó módosíthassa a jelszavát a root által írható `/etc/shadow` fájlban anélkül, hogy root lenne.
- **setgid bit (SGID):** futtatható állományon a hatékony GID a fájl csoportjának GID-jére változik. Könyvtáron beállítva azt jelenti, hogy a könyvtárban létrehozott új fájlok a könyvtár csoportját öröklik, nem a létrehozó felhasználó elsődleges csoportját.
- **sticky bit:** hagyományosan a `/tmp` és hasonló megosztott könyvtárakon használják. Ha be van állítva egy könyvtáron, akkor a benne lévő fájlt csak a fájl tulajdonosa, a könyvtár tulajdonosa vagy a root törölheti, még akkor is, ha a könyvtár írható. Ez megakadályozza, hogy egy felhasználó mások fájljait törölje.

A setgid bit könyvtárakon való használata gyakori a közös munkatérben; például:

```
mkdir /srv/shared
chown :dev /srv/shared
chmod 2770 /srv/shared
ls -ld /srv/shared
# drwxrws--- 2 root dev 4096 Mar  1 12:00 /srv/shared
```

A fenti beállítás után bármely `dev` csoporttag által ebben a könyvtárban létrehozott fájl csoportja automatikusan `dev` lesz.

21.1.3 3. A jogosultság-ellenőrzés algoritmusa

Amikor egy folyamat hozzáfér egy fájlhoz, a kernel a következő sorrendben ellenőrzi a jogosultságokat:

1. **Ha a folyamat hatékony UID-je 0** (hagyományos root), a hozzáférés engedélyezett, kivéve a végrehajtási jogot, amelyhez legalább egy execute bitnek kell állnia. Ez a klasszikus root bypass; a modern rendszerek a capabilities használatával szigoríthatják.
2. **Ha a folyamat EUID-ja megegyezik a fájl tulajdonosának UID-jével**, akkor a tulajdonosi jogosultsági bitek érvényesek. Ha a megfelelő bit be van állítva, a hozzáférés sikeres; ha nincs, a hozzáférés megtagadva, és a további csoportjogokat **nem** vizsgálja a kernel.
3. **Ha a folyamat EUID-ja vagy valamely kiegészítő csoportja megegyezik a fájl csoportjának GID-jével**, akkor a csoportjogosultsági bitek érvényesek. Ugyanaz az illeszkedés szabálya, mint előbb.
4. **Egyébként** a „mások” (other) jogosultsági bitek érvényesek.

A fenti lépések közül csak az első illeszkedő kategória bitjeit veszi figyelembe a kernel. Ezért például ha a fájl tulajdonosa `alice`, és a csoport bitek `rw-`, de a tulajdonosi bitek `---`, akkor alice nem fér hozzá a fájlhoz, még akkor sem, ha a csoportjog alapján megtehetné. Ez a sorrend fontos biztonsági következményekkel jár: a tulajdonosi jogokat mindig szűkebbre kell venni, ha a tulajdonosnak is korlátozott hozzáférést akarunk adni, mert a csoportjogokat a tulajdonos esetén nem nézzük.

A `chmod` parancs a mode biteket állítja be. Példák:

```
chmod 750 /srv/lab/report.txt   # rwxr-x---
chmod u+w,g-w,o= /srv/lab/report.txt  # szimbolikus módosítás
chmod u+s /usr/local/bin/myadmin # setuid bit beállítása
chmod 1777 /tmp                  # rwxrwxrwt
```

A `chown` a tulajdonost és a csoportot módosítja. A jogosultság módosítását általában csak a fájl tulajdonosa vagy a root teheti meg; a `chown` csoporthoz rendelését a tulajdonos csak akkor, ha a célcsoportnak tagja.

21.1.4 4. setuid/setgid működése és a legkisebb jogosultság

A setuid/setgid mechanizmus lehetővé teszi, hogy egy folyamat bizonyos műveleteket magasabb jogosultsággal hajtson végre, mint a futtató felhasználó. Ez azonban jelentős biztonsági kockázatot hordoz, ha a program nem gondosan megírt. A cél mindig a **legkisebb jogosultság elve**: a program csak a feladatához szükséges minimális jogosultságot kapja meg, és a jogosultságot a lehető legrövidebb ideig vagy a lehető legszűkebb körben használja.

A setuid programok esetében a következő UID-mintázat jellemző:

1. A program elindul, RUID = a futtató felhasználó, EUID = a fájl tulajdonosa.
2. A program elvégzi a kiemelt műveletet.
3. A program a kiemelt művelet után visszaált az RUID-re (`seteuid(getuid())`), hogy a továbbiakban ne rendelkezzen root joggal.

A mentett UID lehetővé teszi, hogy a program átmenetileg visszaváltson, majd újra visszaállítsa a kiemelt EUID-t, ha szükséges. Például:

```c
/* setuid minta: csak a kiemelt művelet idejére legyünk root */
#include <unistd.h>
#include <stdio.h>

int main(void) {
    uid_t ruid = getuid();   /* valós UID */
    uid_t euid = geteuid();  /* hatékony UID */
    printf("RUID=%d, EUID=%d\n", ruid, euid);

    /* Kiemelt művelet elvégzése itt */

    /* Visszaváltás a valós UID-re */
    if (seteuid(ruid) != 0) {
        perror("seteuid");
        return 1;
    }
    printf("Visszaváltva: EUID=%d\n", geteuid());
    return 0;
}
```

A fenti kód szemléltetés, nem végrehajtandó utasítás éles rendszeren; laborban tesztelhető.

A setgid hasonlóan működik, de a csoportjogot emeli. A setgid programok gyakran használatosak megosztott fájlok kezelésére, például egy játék minden játékosának eredményét egy közös csoporttag kezeli.

A setuid/setgid bitek beállításakor különösen ügyelni kell:

- A programnak **ne legyen világra írható** a hivatkozási lánca. Ha a programot bárki módosíthatja, akkor a setuid bit nem ad biztonságot: a támadó lecserélheti a programot egy tetszőleges kódra, amely a setuid tulajdonos jogával fut.
- A programot **ne helyezzük világra írható könyvtárba** (pl. `/tmp`), mert a fájlt lecserélhetik.
- A program **bemenetét mindig ellenőrizni** kell, mert a shell metakarakterek, környezeti változók, fájlútvonalak és a szimbolikus hivatkozások (symlink) támadási felületet jelentenek. A klasszikus TOCTOU (time-of-check to time-of-use) hiba például akkor lép fel, ha a program ellenőrzi, hogy egy fájl nem szimbolikus link, majd később megnyitja; közben a támadó lecserélheti a linket.
- A program **ne használjon külső parancsértelmezőt** (`system()`, `popen()`) vagy shell scriptként fusson, mert a környezeti változók és a shell interpolációja gyakran kihasználható. Sok rendszer a setuid scripteket figyelmen kívül hagyja.
- A program **csak a minimális időre** tartsa meg a kiemelt jogosultságot.

A legkisebb jogosultság elvét a fájlok jogainak beállítására is alkalmazni kell. Például egy logfájlt gyakran `0644`-re állítanak, hogy mindenki olvashassa, de csak a tulajdonos írhassa. Ha a fájl `0666`, akkor bárki módosíthatja vagy törölheti, ami hitelességi probléma. A csoportok használatával finomítható a hozzáférés: ahelyett, hogy a „másoknak” adnánk írást, hozzunk létre egy dedikált csoportot, és annak adjunk `g+w` jogot.

21.1.5 5. POSIX ACL: a hagyományos modell kiterjesztése

A hagyományos UID/GID/mode modell hátránya, hogy csak egy tulajdonost, egy csoportot és a többieket tud megkülönböztetni. Ha például egy fájlt `alice` tulajdonol, a `dev` csoport írhatja, de `bob` egyénileg is olvashatja, a többieknek nincs hozzáférésük, akkor a hagyományos bitekkel ez nem kivitelezhető `bob` külön megadása nélkül. A **POSIX ACL** ezt a korlátozást oldja fel.

Az ACL bejegyzések típusai:

- **ACL_USER_OBJ:** a fájl tulajdonosának jogai – ez a hagyományos tulajdonosi biteknek felel meg.
- **ACL_USER:** megnevezett felhasználó jogai.
- **ACL_GROUP_OBJ:** a fájl csoportjának jogai – a hagyományos csoport bitek.
- **ACL_GROUP:** megnevezett csoport jogai.
- **ACL_MASK:** a csoportosztály maximális jogosultsága, amely korlátozza az összes megnevezett csoportot és a csoport tulajdonost.
- **ACL_OTHER:** a „mások” jogai.

Az ACL-t a `setfacl` paranccsal lehet módosítani, a `getfacl` pedig megjeleníti:

```
# Példa: /srv/lab/report.txt ACL beállítása
setfacl -m u:bob:r-- /srv/lab/report.txt
setfacl -m g:audit:r-- /srv/lab/report.txt
getfacl /srv/lab/report.txt
# file: /srv/lab/report.txt
# owner: alice
# group: dev
user::rw-
user:bob:r--
group::rw-
group:audit:r--
mask::rw-
other::---
```

A fenti példában `alice` továbbra is `rw-` joggal rendelkezik, a `dev` csoport `rw-` joggal, `bob` egyénileg `r--` jogot kapott, az `audit` csoport szintén `r--` jogot. A `mask::rw-` azt jelenti, hogy a csoportosztály (a `dev` csoport, a `bob` felhasználó és az `audit` csoport) maximális jogosultsága `rw-`. Ha a mask `r--` lenne, akkor a `dev` csoport sem írhatna, annak ellenére, hogy a `group::rw-` bejegyzés azt mutatja.

A hagyományos `ls -l` kimenetében a csoport jogosultsági maszk az ACL maskot tükrözi, ezért ha ACL van beállítva, a csoport bitek nem feltétlenül a `group::` bejegyzést mutatják, hanem a maskot. Ha ACL-t távolítunk el (`setfacl -b`), a hagyományos mode bitek visszaállnak a tulajdonosi, csoport és other bejegyzésekből.

Könyvtárakon **default ACL** is beállítható, amely az újonnan létrehozott fájlokra és alkönyvtárakra öröklődik:

```
setfacl -d -m g:dev:rwx /srv/shared
getfacl /srv/shared
# ...
# default:group::rwx
# default:group:dev:rwx
# default:mask::rwx
# default:other::---
```

A default ACL biztosítja, hogy a `/srv/shared` alatt létrehozott új fájlok is öröklik a megfelelő ACL-t, így a közös munka egyszerűbbé válik anélkül, hogy minden fájlon egyedileg kellene beállítani a jogokat.

21.1.6 6. Gyakorlati hibák és megoldások

A hagyományos jogosultsági modell egyszerűsége ellenére számos tipikus hiba előfordul:

1. **Túl engedékeny „mások” jogosultság:** például egy konfigurációs fájl `0666` jogot kap, mert a fejlesztő gyorsan akarja szerkeszteni egy tesztkörnyezetben. Ez a hiba éles rendszeren azonosítók, kulcsok vagy egyéb érzékeny adatok kiszivárgásához vezethet. Megoldás: `0640` és a megfelelő csoport használata.

2. **Setuid program világra írható könyvtárban:** ha a setuid program a `/tmp`-ben található, bárki lecserélheti. A setuid bitet csak rendszer által ellenőrzött, nem írható könyvtárban szabad használni (pl. `/usr/local/sbin`).

3. **Setuid scriptek:** sok UNIX-változat figyelmen kívül hagyja a setuid bitet a scripteken, mert a kernel a script interpretert futtatja, nem magát a scriptet. Ha mégis szükség van rá, a scriptet egy setuid wrapper programból kell meghívni, és a wrapperben gondoskodni kell a környezet biztonságáról.

4. **TOCTOU és szimbolikus hivatkozás:** setuid programokban a fájlok ellenőrzése és megnyitása közötti időablak kihasználható. A helyes megoldás a `open()` rendszerhívás `O_NOFOLLOW` zászlajának használata, a fájldeszkriptorok átadása, és a biztonsági ellenőrzések elvégzése a megnyitás **után**, a fájldeszkriptoron (például `fstat`).

5. **Sticky bit hiánya megosztott könyvtáron:** ha a `/tmp` vagy egy közös munkakönyvtár `1777` helyett `0777` jogú, akkor a felhasználók törölhetik mások fájljait. A sticky bit beállítása ezt megakadályozza.

6. **Csoportjogosultság figyelmen kívül hagyása a tulajdonos esetén:** ahogy fentebb láttuk, ha a tulajdonosnak nincs joga, a kernel nem nézi a csoportjogot. Ezért ha a tulajdonost korlátozni akarjuk, akkor a tulajdonosi biteket kell megfelelően beállítani, nem a csoportbiteket.

7. **Túlzott csoporttagság:** ha egy felhasználó túl sok csoport tagja, nehéz átlátni, mely csoporton keresztül jut hozzá egy fájlhoz. A csoportkiosztás áttekinthetősége biztonsági követelmény.

8. **Setuid programok elhanyagolt biztonsági auditja:** a setuid programokat rendszeresen ellenőrizni kell, hogy a jogosultságuk még indokolt-e. A `find` paranccsal listázhatók:

```
find / -perm -4000 -type f 2>/dev/null   # setuid fájlok
find / -perm -2000 -type f 2>/dev/null   # setgid fájlok
```

A legkisebb jogosultság elve szerint a setuid programok számát minimalizálni kell, és a jogosultságot a lehető legrövidebb időre kell megtartani.

21.1.7 7. Labor: jogosultsági mátrix és setuid-kockázat elemzés

A labor célja, hogy a hagyományos UNIX-jogosultsági modell viselkedését egy emulált környezetben, reprodukálhatóan vizsgáljuk. A labor **nem éles rendszeren fut**, hanem eldobható QEMU vendéggépen vagy egyszer használatos konténerben. A hoszt lemezét, bootloaderét és rendszerpartícióját nem módosítjuk.

21.1.8 7.1 Kiinduló állapot

A gépen rendelkezésre áll egy minimális Linux disztribúció, root joggal. A labor elején hozzuk létre a szükséges felhasználókat, csoportokat és mappákat:

```
# Eldobható környezetben, rootként
groupadd dev
groupadd audit
useradd -m -s /bin/bash alice
useradd -m -s /bin/bash bob
useradd -m -s /bin/bash carol
usermod -aG dev alice
usermod -aG dev bob
usermod -aG audit carol

mkdir -p /srv/lab/shared
chown alice:dev /srv/lab/shared
chmod 2770 /srv/lab/shared

# Tesztfájlok
echo "public" > /srv/lab/public.txt
echo "secret" > /srv/lab/secret.txt
echo "report" > /srv/lab/report.txt

chown alice:dev /srv/lab/public.txt
chown alice:audit /srv/lab/secret.txt
chown alice:dev /srv/lab/report.txt

chmod 644 /srv/lab/public.txt
chmod 640 /srv/lab/secret.txt
chmod 660 /srv/lab/report.txt
```

A setuid demóhoz készítsünk egy C programot `show_id.c` néven:

```c
/* show_id.c - setuid viselkedés szemléltetése, labor */
#include <stdio.h>
#include <unistd.h>
#include <fcntl.h>

int main(int argc, char **argv) {
    printf("RUID=%d, EUID=%d, RGID=%d, EGID=%d\n",
           getuid(), geteuid(), getgid(), getegid());

    if (argc == 2) {
        int fd = open(argv[1], O_RDONLY);
        if (fd >= 0) {
            printf("open(%s) sikeres\n", argv[1]);
            close(fd);
        } else {
            perror("open");
            return 2;
        }
    }
    return 0;
}
```

Fordítsuk le, és állítsuk be root tulajdonú, setuid bitet:

```
gcc -Wall -Wextra -O2 -o /usr/local/sbin/show_id show_id.c
chown root:root /usr/local/sbin/show_id
chmod 4755 /usr/local/sbin/show_id
ls -l /usr/local/sbin/show_id
# -rwsr-xr-x 1 root root ... /usr/local/sbin/show_id
```

21.1.9 7.2 Jogosultsági mátrix

A mátrix célja, hogy rögzítse, melyik felhasználó melyik fájlhoz fér hozzá. A teszteléshez használjunk `su`-t, hogy az egyes felhasználókká váljunk, majd próbáljunk olvasni, írni és listázni. A táblázatban külön oszlopban szerepeljen a **várt** és a **mért** eredmény.

Példa mátrix:

Fájl / művelet	alice	bob	carol	Várt	Mért
`/srv/lab/public.txt` olvasás				engedélyezett	engedélyezett
`/srv/lab/public.txt` írás	(owner)			csak alice	csak alice
`/srv/lab/secret.txt` olvasás	(owner)	(dev csoport nem audit)	(audit csoport)	alice, carol	alice, carol
`/srv/lab/report.txt` olvasás	(owner)	(dev csoport)		alice, bob	alice, bob
`/srv/lab/shared/` belépés és fájl létrehozás			(mások nincs jog)	alice, bob	alice, bob
`/srv/lab/shared/` létrehozott fájl csoportja	dev	dev	–	setgid miatt dev	dev

A mérések során a parancs kimenetét naplózzuk. Például:

```
# Alízként
su - alice
cat /srv/lab/public.txt    # sikeres
echo "more" >> /srv/lab/public.txt  # sikeres (owner)
cat /srv/lab/secret.txt    # hiba: Permission denied
cd /srv/lab/shared && touch test_alice  # sikeres
ls -l /srv/lab/shared/test_alice  # csoport dev
exit
```

A várt és mért eredményt a táblázatban rögzítjük; ha eltérés van, azt hibaként jelöljük és kivizsgáljuk.

21.1.10 7.3 Setuid-kockázat elemzés

A `show_id` programot root tulajdonú, setuid bitekkel láttuk el. Futtassuk `alice` felhasználóként:

```
su - alice
/usr/local/sbin/show_id /srv/lab/secret.txt
# RUID=1000, EUID=0, RGID=1000, EGID=1000
# open(/srv/lab/secret.txt) sikeres
```

A kimenet mutatja, hogy a folyamat EUID-ja 0, ezért a kernel root jogot ad a fájl megnyitásához, hiába a fájl `0640` és `alice` nem tagja az `audit` csoportnak. Ez a setuid root programok kockázata.

A kockázatelemzés során vizsgáljuk meg a következő forgatókönyveket, és soroljuk be őket **magas**, **közepes** vagy **alacsony** kockázatú kategóriába, indoklással:

1. **A `show_id` program nem ellenőrzi a bemeneti fájlnevet.** Például a programban `system("cat " + argv[1])` szerepelne. Ez **magas** kockázat: shell injekcióval root parancs futtatható.
2. **A `show_id` programot világra írható könyvtárba helyezik** (pl. `/tmp`). Ez **magas** kockázat: a programot lecserélhetik.
3. **A `show_id` program banális hibát tartalmaz a fájlmegnyitás során** (pl. TOCTOU, ha a program `stat()`-tel ellenőriz, majd `open()`-nel nyit). Ez **magas** kockázat, mert root jogosultsággal fut.
4. **A `show_id` program csak a `secret.txt` fájl fix útvonalon olvassa, és a felhasználó nem adhat paramétert.** Ez **alacsony** kockázat, ha a program maga ellenőrzött, és a fájl elérési útja nem módosítható.
5. **A setuid bitet eltávolítják, és a programot csak a `dev` csoport futtathatja.** Ez **alacsony** kockázat: nincs jogemelés, a jogosultságok a hagyományos csoportmodell szerint alakulnak.

A kockázati táblázatban minden esethez adjuk meg a **fenyegetés leírását**, a **lehetséges hatást**, a **bekövetkezés valószínűségét** (becslés), és a **javasolt intézkedést**. A valószínűséget például `nagyon magas`, `magas`, `közepes`, `alacsony` szavakkal minősíthetjük; a mérési naplóban rögzítjük a besorolást.

21.1.11 7.4 Mérési napló és visszaállítás

A labor során minden parancs kimenetét és a várt/mért eredményt naplózzuk. A naplóhoz `script` parancsot használhatunk:

```
script /tmp/lab_os078.log
# ... a labor összes parancsa ...
exit
```

A napló tartalmazza a parancsokat, a kimeneteket és az időbélyegeket. A labor végén a rendszert vissza kell állítani az eredeti állapotába:

```
# Visszaállítás
chmod u-s /usr/local/sbin/show_id
chown root:root /usr/local/sbin/show_id
chmod 755 /usr/local/sbin/show_id
rm /usr/local/sbin/show_id
rm /srv/lab/public.txt /srv/lab/secret.txt /srv/lab/report.txt
rm -rf /srv/lab/shared
userdel -r alice
userdel -r bob
userdel -r carol
groupdel dev
groupdel audit
rm /tmp/lab_os078.log
```

A visszaállítás után ellenőrizzük, hogy a felhasználók, csoportok és fájlok nem léteznek, és a hoszt rendszere nem változott.

21.1.12 8. Összefoglalás és önellenőrző kérdések

A hagyományos UNIX-jogosultsági modell alapjai: a folyamatok UID/GID azonosítókkal rendelkeznek, a fájlok mode bitekkel és tulajdonos-csoport párossal; a kernel a hozzáférést a hatékony UID és a kiegészítő csoportok alapján, a tulajdonos → csoport → mások sorrendben ellenőrzi. A

setuid/setgid bitek jogemelést biztosítanak, de gondos programozást és adminisztrációt igényelnek; az ACL további finomságot ad megnevezett felhasználókkal és csoportokkal. A legkisebb jogosultság elve mind a fájl jogok, mind a setuid programok esetében alapvető.

Önellenőrző kérdések:

1. Milyen sorrendben ellenőrzi a kernel a jogosultsági osztályokat, és miért fontos ez a tulajdonos esetében?
2. Mi a különbség a setuid bit hatása fájl és könyvtár esetén a setgid bithez képest?
3. Hogyan korlátozza az ACL mask mezője a megnevezett csoportok jogait?
4. Milyen tipikus hibákat követhetsz el egy setuid program fejlesztése során, és hogyan kerülheted el őket?
5. Miért kell a laborban a setuid programot eldobható környezetben futtatni, és mi a visszaállítás szerepe?

21.2 Capabilities, SELinux, AppArmor és seccomp

A hagyományos, felhasználóhoz és csoporthoz kötött jogosultsági modell – az úgynevezett diszkrecionális hozzáférés-vezérlés (DAC) – a Unix-rendszerekben régóta bevált, de önmagában nem ad elegendő védelmet. A fő gond az, hogy a `root` felhasználó gyakorlatilag korlátlan hatalommal rendelkezik, és minden olyan folyamat, amely `root` jogosultsággal fut, teljes irányítást kap a rendszer felett. Ha egy ilyen folyamat kompromittálódik, a támadó azonnal átveheti az egész gépet. Ráadásul a hagyományos modell nem képes finoman korlátozni egy adott folyamat képességeit; például egy webszervernek lehet, hogy csak a 80-as portot kell megnyitnia, de rootként futva bármilyen fájlt olvashat, kernelmodult tölthet be, vagy akár az egész rendszert leállíthatja.

Ebben az alfejezetben három olyan mechanizmust vizsgálunk, amelyek a Linux kernelben a finomhangolt hozzáférés-vezérlést és a rendszerhívások szűrését teszik lehetővé: a **capability**-ket, a **Mandatory Access Control** (MAC) keretrendszert, valamint a **seccomp**-ot. Mindhárom eszköz arra szolgál, hogy csökkentse a támadási felületet, és a legkisebb jogosultság elvét (least privilege) érvényesítse. A fejezet végén egy olyan labort mutatunk be, amelyben LSM- (Linux Security Module) és seccomp-szabálytervet készítünk, majd azt emulált környezetben ellenőrizzük.

21.2.1 A root hatalmának felosztása: Linux capabilities

A Linux a hagyományos `root` felhasználó minden jogosultságát önálló, elkülönült egységekre bontja. Ezeket **capability**-knek nevezzük. Ahelyett, hogy egy folyamat vagy bináris minden root jogot megkapna, csak a ténylegesen szükséges capability-ket adjuk meg neki. Így például egy webszerver megkaphatja a `CAP_NET_BIND_SERVICE` capability-t ahhoz, hogy a 1024 alatti portszámokon hallgasson, de nem kapja meg a `CAP_SYS_MODULE`-t, így nem tud kernelmodult betölteni.

A capability-ket a kernel a folyamatokhoz rendelt bitmezőkben tárolja. A legfontosabb halmazok a következők:

- **Permitted**: azok a capability-k, amelyeket a folyamat használhat. Itt dől el, hogy milyen jogosultságokra van lehetősége.

- **Effective**: azok a capability-k, amelyeket a kernel ténylegesen ellenőriz a rendszerhívások során. A folyamat egy rendszerhívással mozgathat capability-ket a permitted és effective halmaz között, de csak olyanokat, amelyek a permittedben szerepelnek.
- **Inheritable**: az execve végrehajtásakor örökölhető capability-ket tartalmazza, de csak akkor, ha a célprogram rendelkezik megfelelő fájlcapability-beállításokkal.
- **Bounding**: a folyamat számára elérhető capability-k felső határa; ezen kívül eső capability-k nem örökölhetők és nem állíthatók be.
- **Ambient**: a nem privilegizált folyamatok számára lehetővé teszi, hogy egy capability-t az execve után is megtartsanak anélkül, hogy a program fájlrendszer-capability-vel rendelkezne.

A Linux kernelben a capability-ket a `capget(2)` és `capset(2)` rendszerhívásokkal lehet lekérdezni és módosítani. A folyamat saját capability-halmazát a `prctl(2)` rendszerhívás `PR_CAP_AMBIENT` opciójával is lehet kezelni. A fájlokhoz rendelt capability-ket a `setcap(8)` és `getcap(8)` parancsokkal állíthatjuk be, illetve kérdezhetjük le.

Gyakorlati példa: egy HTTP-kiszolgáló futtatása nem root felhasználóként, de a 80-as port eléréséhez szükséges joggal. Ehhez a binárisra a következőt állíthatjuk be emulált környezetben:

```
sudo setcap 'cap_net_bind_service=+ep' /usr/local/bin/myserver
getcap /usr/local/bin/myserver
```

A `+ep` azt jelenti, hogy a capability-t hozzáadjuk a permitted és az effective halmazhoz is. Ezután a program nem root felhasználóval indítva is képes lesz a 80-as porton hallgatni.

Fontos megértenünk, hogy a capability-k nem teszik lehetővé a fájlrendszer-jogosultságok megkerülését. Ha egy folyamat rendelkezik a `CAP_DAC_OVERRIDE` capability-vel, akkor a kernel a fájlműveleteknél figyelmen kívül hagyja a DAC-ellenőrzést, de ettől még a capability hiánya esetén a szokásos jogosultsági szabályok érvényesek.

21.2.2 Gyakori hibák a capability-k használatakor

A legnagyobb hiba a túl széles capability-kiosztás. Például ha egy konténerben a `CAP_SYS_ADMIN` capability-t adjuk meg, akkor gyakorlatilag root jogokat adunk a konténeren belül, és a kitörési kockázat ugrásszerűen megnő. A `CAP_SYS_ADMIN` számos, rendszeradminisztrációs műveletet enged, többek között mountolást, swapolást, sőt bizonyos kernelváltozatokban a `CAP_SYS_ADMIN`-nel rendelkező folyamat képes lehet a seccomp-szűrők megkerülésére is. Ezért a konténerek futtatásakor a Docker és hasonló eszközök alapértelmezés szerint a capability-k nagy részét eldobják, és csak egy szűk, valóban szükséges halmazt tartanak meg.

További tipikus hiba, amikor a capability-t a fájlra állítjuk be, de a programot olyan parancsértelmezőből indítjuk, amely nem őrzi meg a fájlcapability-ket. A setuid és setcap beállítások ugyanis csak bináris fájlokon működnek, parancsfájlokon nem. Emellett a capability-k öröklése az execve során bonyolult, és a hibásan összeállított bounding vagy ambient halmaz miatt a folyamat a szükséges jogot nem kapja meg, vagy éppen túl sokat örököl.

A `capsh --print` parancs hasznos az aktuális folyamat capability-halmazainak ellenőrzésére. Például:

```
capsh --print | grep Current
```

Ez kilistázza a permitted, effective, inheritable és bounding halmazokban lévő capability-ket. A konténerek indításakor érdemes ezt az állapotot rögzíteni, és ellenőrizni, hogy a várt szűkítés megtörtént-e.

21.2.3 Kötelező hozzáférés-vezérlés: SELinux és AppArmor

A capability-k önmagukban még mindig nem adnak választ arra a kérdésre, hogy egy folyamat **melyik fájlokhoz, hálózati portokhoz vagy más erőforrásokhoz** férhet hozzá. Ezt a rést töltik be a **Mandatory Access Control** (MAC) keretrendszerek, amelyek a hagyományos DAC mellett, attól függetlenül érvényesülő, központilag felügyelt szabályrendszert alkotnak. Linux alatt a MAC-ot a **Linux Security Module** (LSM) interfész valósítja meg, amelyre a két legelterjedtebb megvalósítás a **SELinux** és az **AppArmor**.

A MAC lényege, hogy a rendszeren minden objektumhoz (fájl, folyamat, socket, stb.) biztonsági címkét rendelünk, és a kernel minden fontos műveletnél ellenőrzi, hogy a folyamat címkéje jogosult-e az adott objektum címkéjén műveletet végezni. A szabályok betartatása független a folyamat tulajdonosától; még a root felhasználó sem tudja megkerülni a MAC-policy-t, ha az tiltja a hozzáférést.

21.2.4 SELinux – típuskényszerítés és policy

A SELinux a legerőteljesebb, de egyben legbonyolultabb MAC-rendszer. Alapvető építőkövei a **típusok** (type enforcement, TE), a **szerepek** (roles) és a **felhasználók** (SELinux users). Minden objektumnak és folyamatnak van egy biztonsági kontextusa, amelynek általános formája:

```
user_u:role_r:type_t:s0
```

A negyedik mező a **szint** (level), amelyet a Multi-Level Security (MLS) vagy a Multi-Category Security (MCS) használ. A leggyakoribb, célzott (targeted) policy-ban a fő szabályozást a típusok adják: a folyamat típusa határozza meg, hogy milyen objektumtípusokhoz férhet hozzá, és milyen műveleteket végezhet. Például egy Apache webszerver folyamatának típusa `httpd_t`, a kiszolgálandó webes tartalmak típusa pedig `httpd_sys_content_t`. A policy megengedi, hogy a `httpd_t` típusú folyamat olvassa a `httpd_sys_content_t` típusú fájlokat, de tiltja, hogy a teljes fájlrendszert olvassa, vagy hogy a `passwd` fájlt módosítsa.

A SELinux policy a rendszeren a `/etc/selinux/` könyvtárban található, és betöltés után a kernelben aktív. Az aktuális állapotot a `sestatus` parancs mutatja, a módot pedig a `getenforce`/`setenforce` kezeli. A SELinux három üzemmódban futhat:

- **Enforcing**: a szabályokat a kernel betartatja, a tiltott műveletek `EACCES` hibakóddal meghiúsulnak.
- **Permissive**: a szabálysértéseket a kernel naplózza, de nem blokkolja; ez a hibakereséshez hasznos.
- **Disabled**: a SELinux nem aktív.

A gyakorlatban a policy finomhangolásához a `semanage`, `semanage fcontext`, `restorecon`, `chcon` parancsok szükségesek. Példa egy webes könyvtár címkézésére:

```
sudo semanage fcontext -a -t httpd_sys_content_t "/srv/www(/.*)?"
sudo restorecon -R /srv/www
```

A `semanage fcontext` bejegyzi a fájlrendszer-útvonalhoz tartozó alapértelmezett címkét, a `restorecon` pedig a meglévő fájlokra alkalmazza azt. Ha a webszerver ezután nem tud hozzáférni egy fájlhoz, a `/var/log/audit/audit.log` fájlban AVC- (Access Vector Cache) bejegyzések jelennek meg. Például:

```
type=AVC msg=audit(1717000000.123:45): avc:  denied  { read } for  pid=1234 comm="myserver"
 ↪  name="index.html" dev="vda1" ino=5678 scontext=system_u:system_r:httpd_t:s0
 ↪  tcontext=unconfined_u:object_r:default_t:s0 tclass=file
```

Ez az üzenet megmutatja, hogy a `httpd_t` típusú folyamat próbált olvasni egy `default_t` típusú fájlt, amit a policy tiltott. Ilyenkor a megoldás a fájl megfelelő címkézése, nem pedig a SELinux kikapcsolása.

A SELinux a folyamatok közötti átmeneteket is szabályozza; például egy `unconfined_t` típusú shellből egy `httpd_t` típusú szolgáltatás indítását a policy explicit engedélyezi. A boolean értékek további finomhangolást tesznek lehetővé anélkül, hogy a policy-t újra kellene fordítani. Például a `httpd_can_network_connect` boolean szabályozza, hogy a webszerver ki tud-e lépni a hálózatra.

21.2.5 AppArmor – útvonal-alapú profilok

Az AppArmor a SELinuxhoz hasonló MAC-rendszer, de egyszerűbb, és a szabályokat **útvonalak** és **programnevek** alapján fogalmazza meg. Minden védett programhoz egy profil tartozik, amely meghatározza, hogy az adott program milyen fájlokat érhet el, milyen capability-ket használhat, és milyen hálózati műveleteket végezhet. Az AppArmor az LSM-keretrendszeren keresztül kapcsolódik a kernelhez, de a szabályok nem címkékre, hanem fájlútvonalakra épülnek, ezért sokkal könnyebb gyorsan egy profil létrehozása.

Egy tipikus AppArmor profil a `/etc/apparmor.d/usr.bin.myserver` fájlban a következőképpen nézhet ki (a példa szemléltető, a tényleges profil a szolgáltatás igényeitől függ):

```
#include <tunables/global>

/usr/local/bin/myserver {
  #include <abstractions/base>
  #include <abstractions/nameservice>

  /usr/local/bin/myserver mr,
  /srv/www/** r,
  /dev/urandom r,
  /var/log/myserver.log w,
  /etc/ssl/openssl.cnf r,
  /proc/*/status r,

  network tcp,
  network udp,

  capability net_bind_service,
  capability setuid,
  capability setgid,
  capability chown,
}
```

A profil kezdetén megadjuk a védett program abszolút útvonalát. A zárójelben lévő szabályok közül a fájlhozzáféréseknél az `r` olvasást, a `w` írást, az `mr` memória-leképezést és olvasást jelent. A `/**` minta a könyvtár rekurzív tartalmát jelöli. A `network tcp` és `udp` engedélyezi a TCP/UDP socketek használatát, a capability sorok pedig a nevesített Linux capability-ket adják meg. Az AppArmor a capability-ket kisbetűvel, az aláhúzás nélküli alakban várja, például `net_bind_service` a `CAP_NET_BIND_SERVICE`-nek felel meg.

Az AppArmor profilokat az `apparmor_parser` tölti be, a módot pedig az `aa-complain` és `aa-enforce` parancsokkal lehet beállítani. A `complain` mód hasonló a SELinux permissive módjához: a szabálysértéseket naplózza, de nem blokkolja. A profil létrehozása után célszerű complain módban tesztelni, áttekinteni a naplót, majd enforce módba állítani.

```
sudo apparmor_parser -r /etc/apparmor.d/usr.bin.myserver
sudo aa-enforce usr.bin.myserver
sudo aa-status
```

A `aa-status` megjeleníti a betöltött profilokat és a korlátozott folyamatokat. Az AppArmor naplóbejegyzései a kernelnaplóban (`journalctl -k` vagy `dmesg`) találhatók.

21.2.6 SELinux vs. AppArmor és tipikus hibák

A két MAC-rendszer közötti választás gyakran disztribúciós döntés: a Red Hat-alapú rendszerek (Fedora, RHEL) alapértelmezésben SELinuxot, a Debian és Ubuntu alapú rendszerek AppArmort használnak. A SELinux erősebb, mert minden objektumot címkéz, és a policy központilag, a folyamatok tulajdonosától függetlenül érvényesül; azonban bonyolultabb, és hibás címkézés esetén nehéz a diagnózis. Az AppArmor egyszerűbb, mert a profilok fájlútvonalakra épülnek, és gyorsabban strukturálhatók, de a nem fájl alapú erőforrásokra (például hálózati portok, processzek) kevésbé kifejező.

A leggyakoribb hiba mindkét rendszernél, hogy a szabályokat permissive vagy complain módban hagyják, így a MAC nem nyújt valódi védelmet. Másik tipikus hiba a túl általános szabályok használata, például az AppArmor profilban a `/** r` engedély az egész fájlrendszer olvasását teszi lehetővé, ami gyakorlatilag értelmetlenné teszi a korlátozást. A SELinuxnál a `restorecon` elmulasztása miatt a fájlok rossz címkével rendelkeznek, és a folyamat nem fér hozzájuk, amit sokan a SELinux kikapcsolásával „oldanak meg” – ehelyett a címkéket kell helyesen beállítani.

21.2.7 Rendszerhívás-szűrés: seccomp

Amíg a capability-k és a MAC a **jogosultságokat** és az **objektumhozzáférést** szabályozzák, a **seccomp** (secure computing mode) egészen más szinten avatkozik be: közvetlenül a rendszerhívások számát és argumentumait szűri. A seccomp célja, hogy egy folyamatot még akkor is megfosszon a veszélyes rendszerhívások elérhetőségétől, ha a folyamat valamilyen hibát kihasználva kódvégrehajtást érne el. Ha a rendszerhívás nincs engedélyezve, a kernel blokkolja, és a folyamat tipikusan `SIGSYS` jelzést kap, vagy egy beállított hibakódot kap vissza.

A Linux kétféle seccomp módot ismer:

- **SECCOMP_MODE_STRICT**: a legegyszerűbb. A folyamat csak a `read(2)`, `write(2)`, `_exit(2)` és `sigreturn(2)` rendszerhívásokat hajthatja végre. Minden más hívás esetén a kernel `SIGKILL`-t küld a folyamatnak. Ezt a módot ritkán használják, mert túl szigorú.
- **SECCOMP_MODE_FILTER**: tetszőleges szűrőt tölthetünk be. A szűrő egy Berkeley Packet Filter (BPF) program, amely minden rendszerhívás előtt lefut, és az eredménye alapján a kernel engedélyezi, blokkolja, vagy hibakóddal tér vissza. A szűrő vizsgálhatja a rendszerhívás számát és az argumentumait is, de csak az argumentumok memóriáját nem olvashatja – a BPF program a rendszerhívás számára és az argumentumértékekre vonatkozó regiszterekből dolgozik.

A szűrő betöltésére szolgál a `seccomp(2)` rendszerhívás. A fő műveletek:

- `SECCOMP_SET_MODE_STRICT`: szigorú mód bekapcsolása.
- `SECCOMP_SET_MODE_FILTER`: BPF szűrő betöltése.

- `SECCOMP_GET_ACTION_AVAIL`: megadja, hogy egy adott művelet elérhető-e az aktuális kernelben.

A `prctl(2)` rendszerhívás `PR_SET_SECCOMP` opciójával is be lehet állítani a strict módot, illetve régebbi kerneleken a filter módot, de a modern interfész a `seccomp(2)`.

A gyakorlatban a BPF szűrőt ritkán írjuk kézzel; általában a `libseccomp` könyvtárat használjuk, amely magasabb szintű API-t biztosít. Az alábbi C példa egy olyan szűrőt tölt be, amely minden hívást engedélyez, kivéve az `openat`, `execve` és `mount` hívásokat, amelyekre `EPERM` hibát ad:

```
/* seccomp_filter_example.c – szemléltető példa, emulált környezetben futtatandó */
#include <seccomp.h>
#include <stdio.h>
#include <stdlib.h>
#include <errno.h>
#include <unistd.h>

int main(void) {
    scmp_filter_ctx ctx;
    int rc;

    /* Az alapértelmezett művelet: engedélyezés */
    ctx = seccomp_init(SCMP_ACT_ALLOW);
    if (ctx == NULL) {
        perror("seccomp_init");
        exit(EXIT_FAILURE);
    }

    /* Tiltott hívások: EPERM hibával térnek vissza */
    rc = seccomp_rule_add(ctx, SCMP_ACT_ERRNO(EPERM), SCMP_SYS(openat), 0);
    if (rc < 0) goto fail;
    rc = seccomp_rule_add(ctx, SCMP_ACT_ERRNO(EPERM), SCMP_SYS(execve), 0);
    if (rc < 0) goto fail;
    rc = seccomp_rule_add(ctx, SCMP_ACT_ERRNO(EPERM), SCMP_SYS(mount), 0);
    if (rc < 0) goto fail;

    /* Szűrő betöltése */
    rc = seccomp_load(ctx);
    if (rc < 0) {
        perror("seccomp_load");
        seccomp_release(ctx);
        exit(EXIT_FAILURE);
    }
    seccomp_release(ctx);

    /* A szolgáltatás elindítása – a példában csak egy üzenetet írunk ki */
    write(STDOUT_FILENO, "seccomp filter active, running service\n", 40);
    while (1) {
        pause();
    }
    return 0;

fail:
    perror("seccomp_rule_add");
    seccomp_release(ctx);
    exit(EXIT_FAILURE);
}
```

A példában a `seccomp_init(SCMP_ACT_ALLOW)` azt jelzi, hogy minden hívást engedélyezünk, kivéve, amit explicit megtiltunk. A `seccomp_rule_add` egy-egy rendszerhívást ad a szűrőhöz. A `SCMP_ACT_ERRNO(EPERM)` művelet azt mondja a kernelnek, hogy a tiltott hívást ne hajtsa végre, hanem `EPERM` hibakódot adjon vissza a hívó folyamatnak. A szűrő betöltése után a folyamat nem

tud többé újabb szűrőt hozzáadni; ezért a szűrőt a program indulásakor kell felállítani, mielőtt bármilyen nem megbízható bemenetet feldolgozunk.

A konténerekben a futtatókörnyezetek (például Docker, Podman, systemd) alapértelmezés szerint egy-egy seccomp-profilt alkalmaznak, amely több száz rendszerhívást tilt vagy szűkít. Ez jelentősen csökkenti a kernel támadási felületét. A seccomp-szűrést az `SCMP_ACT_KILL_PROCESS` vagy `SCMP_ACT_LOG` műveletekkel is kombinálhatjuk, bár a túl sok naplózás teljesítménycsökkenést okozhat.

21.2.8 Tipikus seccomp hibák

Az egyik leggyakoribb hiba, hogy a szűrő túl szigorú, és a program egy olyan rendszerhívást is blokkol, amelyre a normál működéshez szüksége van. Például a modern C könyvtárak dinamikus linkeléskor `openat`-ot használnak, ezért ha ezt megtiltjuk, a program már az induláskor elhal, mielőtt a `main` elérné. A szűrő tervezésekor mindig a ténylegesen használt rendszerhívásokat kell összegyűjteni, amit a `strace` vagy a `libseccomp` `SCMP_ACT_LOG` művelete segíthet.

Másik tipikus csapda a 32 bites és 64 bites rendszerhívások ABI-jának eltérése. Ugyanaz a rendszerhívás más számmal rendelkezhet a két ABI-ban, és ha a szűrő csak a natív ABI-t kezeli, egy 32 bites folyamat megkerülheti a szűrést. A modern kernelek és a `libseccomp` automatikusan kezelik a fontosabb eseteket, de saját szűrő írásakor erre figyelni kell.

Végül fontos tudni, hogy a seccomp-szűrő **nem** helyettesíti a capability-ket vagy a MAC-ot. A seccomp csak a rendszerhívások szintjén szabályoz, nem tudja, hogy egy fájlhozzáférés jogos-e; ha egy hívást engedélyezünk, a fájlrendszer-jogosultságok és a MAC továbbra is érvényesülnek. A három mechanizmus együttesen ad erős védelmet, de egyik sem csodaszer.

21.2.9 Konténeres izoláció és breakout-kockázat

A modern konténerek (Docker, Podman, LXC) éppen a fenti technikákat kombinálják, hogy a folyamatokat egymástól és a host rendszertől lehetőség szerint elszigeteljék. A Linux konténerek izolációjának fő pillérei:

- **User namespace**: a konténeren belüli root felhasználó a hoston egy nem privilegizált felhasználónak felel meg, így a konténerből való kitörés esetén a támadó nem szerez root jogot a hoszton.
- **Cgroup namespace és cgroup limits**: az erőforrások (CPU, memória, I/O) elszigetelése és korlátozása.
- **Network namespace**: külön hálózati verem, saját interfészek és IP-címek.
- **PID namespace**: a folyamatok egymástól való elszigetelése.
- **Filesystem root (chroot/pivot_root)**: a konténer a saját gyökérkönyvtárában látja a fájlrendszert.
- **Capability drop**: a konténer indulásakor a capability-k nagy részét eldobják.
- **Seccomp profil**: a konténerben futó folyamatok rendszerhívásainak szűrése.
- **LSM (AppArmor/SELinux)**: a konténer folyamataira vonatkozó MAC-szabályok.

A **breakout** kockázata akkor nő meg, ha ezek közül valamelyik réteg hiányzik vagy félre van konfigurálva. A leggyakoribb hibák:

- **Privileged konténer futtatása** (`--privileged`): ilyenkor a konténer gyakorlatilag minden host erőforráshoz hozzáfér, és a seccomp/AppArmor korlátozások is gyengülnek.
- **Host mount csatolása**: ha a konténerbe a host `/proc`, `/sys` vagy `/` könyvtárat csatoljuk, a támadó közvetlenül a host kernelét vagy fájljait érheti el.
- **Hiányzó user namespace**: ha a konténerben root felhasználó van, és nincs user namespace, akkor a konténer rootja azonos a host rootjával, így egy kernelhiba kihasználása esetén a támadó root lehet a hoszton.
- **Túl széles seccomp profil**: ha a seccomp nem tiltja a `mount`, `open_by_handle_at`, `ptrace` vagy a `bpf` rendszerhívásokat, a támadó könnyebben találhat utat a kernel felé.

A védekezés alapelve: **minimalizáljuk a kitettséget**. Minden konténerhez állítsunk be capability drop-ot (például `--cap-drop=ALL --cap-add=NET_BIND_SERVICE`), használjunk alapértelmezett seccomp-profilt, és ha lehetséges, user namespace-t is. A MAC-policy tovább szűkítheti a konténer fájlrendszer-hozzáférését. Emlékeztetőül: a konténerek nem biztonsági határként, hanem kényelmi eszközként kezelendők; a valódi izolációhoz a virtualizáció (VM) erősebb garanciákat ad, de a konténerek megfelelő konfigurációval jelentősen csökkentik a kockázatot.

21.2.10 Gyakorlati minta: webkiszolgáló szabályterve

Vegyünk egy egyszerű, C nyelven írt webszervert, amely a 8080-as porton hallgat, a `/srv/www` könyvtárból szolgál ki statikus fájlokat, és a `/var/log/myserver.log` fájlba naplóz. A szabályterv három rétegből áll:

1. **Capability-k**: A program nem root felhasználóval fut, de a 8080-as port nem privilegizált, így nincs szüksége `CAP_NET_BIND_SERVICE`-re. Azonban ha a jövőben 80-as portra váltunk, akkor ezt a capability-t hozzá kell adni. A program nem végez mountolást, nem tölt be modult, nem módosít jogosultságokat, ezért minden más capability-t eldobunk. A futtató parancs:

   ```
   setcap 'cap_net_bind_service=+ep' /usr/local/bin/myserver   # csak ha 80-as port kell
   ```

2. **AppArmor profil**: A fenti profil példát követve a program csak a `/srv/www` könyvtárat olvashatja, a `/var/log/myserver.log` fájlt írhatja, és a `/dev/urandom`-ot olvashatja. Ezen kívül TCP socketeket használhat, és a szükséges capability-ket megtarthatja. A profil betöltése után a program nem férhet hozzá a `/etc/shadow` fájlhoz vagy más processz memóriájához.

3. **Seccomp szűrő**: A webszerver a következő rendszerhívásokat használja: `socket`, `bind`, `listen`, `accept4`, `read`, `write`, `openat`, `close`, `lseek`, `fstat`, `mmap`, `munmap`, `exit_group`. A seccomp-szűrőt úgy állítjuk össze, hogy ezeket engedélyezi, minden mást pedig `EPERM` hibával blokkol. Így ha a programban egy távoli kihasználható hiba miatt tetszőleges kód futna, az nem tud például `execve`-t hívni, hogy shellt indítson, és nem tud `ptrace`-szel más folyamatot megfigyelni.

A három réteg együttesen azt eredményezi, hogy még ha a webszervert feltörik is, a támadó csak egy szűk, nem privilegizált környezetben mozoghat.

21.2.11 Ellenőrzés és mérés

A szabályterv helyességét pozitív és negatív tesztekkel ellenőrizzük. **Pozitív teszt**: a szolgáltatás normál működése – a weboldal betöltése, a naplófájl írása – sikeres. **Negatív teszt**: megpróbálunk olyan műveletet végrehajtani, amelyet a szabályok tiltanak, például a program `openat`-tal megnyitja a `/etc/passwd` fájlt. A várt eredmény: a rendszerhívás `EACCES`-szel (MAC) vagy `EPERM`-mel (seccomp) meghiúsul, és a naplóban megjelenik a megfelelő bejegyzés.

A mérések során két dolgot érdemes elkülöníteni:

- **Funkcionális eredmény**: hány pozitív művelet sikeres, hány negatív próbálkozás blokkolt.
- **Teljesítmény overhead**: a seccomp-szűrő és az LSM-ellenőrzések időt igényelnek. Mérjük meg a szolgáltatás válaszidejét szűrő nélkül és szűrővel. A kapott különbség általában néhány mikroszekundum, de a pontos érték a hardvertől és a kernelverziótól függ, ezért a mérési körülményeket rögzíteni kell.

A naplóbejegyzéseket a `dmesg`, `journalctl` vagy az audit napló tartalmazza. A mérési jegyzőkönyvben mindig külön tüntessük fel a **mért** és a **várt** értékeket.

21.2.12 Labor: LSM- és seccomp-szabályterv készítése

Cél: egy egyszerű C nyelvű szolgáltatás számára AppArmor-profil és seccomp-szűrő tervezése, betöltése és ellenőrzése QEMU virtuális gépen.

Kiinduló állapot: QEMU virtuális gép Debian/Ubuntu alapú, kernel 5.15 vagy újabb, AppArmor és libseccomp fejlesztői csomagok telepítve. A virtuális gép neve `os-lab`, diszkje eldobható, a host rendszerhez nem nyúl. A szolgáltatás forráskódja a `myserver.c`, amely a `socket`, `bind`, `listen`, `accept`, `read`, `write`, `open`, `close` hívásokat használja. A program a 8080-as porton hallgat, és a `/srv/www/index.html` fájlt szolgálja ki.

Lépések

1. **Környezet előkészítése**:

```
sudo apt update
sudo apt install -y apparmor apparmor-utils libseccomp-dev gcc
aa-status    # ellenőrizd, hogy az AppArmor aktív
uname -a     # rögzítsd a kernelverziót
```

2. **Szolgáltatás fordítása és indítása szűrők nélkül**:

```
gcc -O2 -Wall myserver.c -o myserver
sudo install -m 0755 myserver /usr/local/bin/myserver
/usr/local/bin/myserver &    # a háttérben futtatjuk
curl http://localhost:8080/index.html    # várt tartalom
```

3. **AppArmor-profil létrehozása**: Hozd létre a `/etc/apparmor.d/usr.bin.myserver` fájlt a fenti profil mintájának megfelelően, de a `/srv/www` könyvtárra és a `/var/log/myserver.log` fájlra korlátozva. A profilban **ne** szerepeljen a `/etc/passwd` olvasási engedély, és ne legyen benne túl általános `/**` szabály.

```
sudo nano /etc/apparmor.d/usr.bin.myserver
sudo apparmor_parser -r /etc/apparmor.d/usr.bin.myserver
sudo aa-enforce usr.bin.myserver
```

4. **Negatív teszt AppArmor ellenőrzésére**: Indítsd el a szolgáltatást, majd próbálj meg a program nevében olvasni a `/etc/passwd` fájlt (például egy hibakereső végponttal vagy a program módosított változatával, amely megpróbálja megnyitni a fájlt). A várt eredmény: a `open` hívás `EACCES` hibával meghiúsul, és a kernelnaplóban AppArmor deny bejegyzés jelenik meg.

5. **Seccomp-szűrő hozzáadása**: Készíts egy `seccomp_wrapper.c` programot a fenti példa alapján, amely engedélyezi a szükséges hívásokat, és minden mást `EPERM`-mel blokkol. A tiltott hívások listája tartalmazza legalább az `execve`, `mount`, `ptrace` hívásokat. Fordítsd le, és futtasd a wrapper-t a szolgáltatással:

```
gcc -O2 -Wall seccomp_wrapper.c -lseccomp -o seccomp_wrapper
sudo install -m 0755 seccomp_wrapper /usr/local/bin/myserver-seccomp
/usr/local/bin/myserver-seccomp &
```

6. **Negatív teszt seccomp ellenőrzésére**: A szolgáltatásban próbálj meg egy `execve` hívást végrehajtani (például egy `system()` hívás a C kódban, amelyet a teszt idejére beillesztünk). A várt eredmény: a seccomp `EPERM` hibával blokkolja a hívást, a program nem indul el a shell, a naplóban megjelenik a seccomp bejegyzés.

7. **Mérési napló rögzítése**: Készíts táblázatot, amelyben a következőket különíted el:

Teszteset	Várt eredmény	Mért eredmény
Normál webkérés	200 OK, tartalom	(mért kód)
`/etc/passwd` olvasása	`EACCES`, AppArmor deny	(mért kód/napló)
`execve` hívás	`EPERM`, seccomp blokk	(mért kód/napló)
Válaszidő szűrő nélkül	< 5 ms	(mért idő)
Válaszidő szűrőkkel	< 8 ms	(mért idő)

   A válaszidőt `ab` vagy `wrk` eszközzel mérd, és rögzítsd a hardver (virtuális CPU típusa), a kernelverzió és a mérési módszer paramétereit.

8. **Visszaállítás**: A labor után a virtuális gépet állítsd vissza a kiindulási pillanatképből, vagy töröld a profilokat és a szűrőt:

```
sudo aa-disable usr.bin.myserver
sudo rm /etc/apparmor.d/usr.bin.myserver
sudo rm /usr/local/bin/myserver-seccomp
```

Mérőszám: a blokkolt negatív műveletek száma a vártnak megfelelően 2 (a `/etc/passwd` és az `execve`). Az overhead mérhető a válaszidő-különbségként.

Fontos: a labor csak emulált környezetben futtatható, a host rendszer fájljait és beállításait tilos módosítani.

21.2.13 Önellenőrző kérdés

Magyarázd el, miért nem elegendő a hagyományos DAC + capability kombó egy konténerben futó webszerver védelmére, és miért van szükség seccomp-szűrőre is. Térj ki arra, hogy a seccomp milyen támadási osztályt tud megakadályozni, amit a MAC nem.

Megoldási vázlat: A DAC és a capability-k a hozzáférést szabályozzák, de ha a programban kódvégrehajtási hiba van, a támadó a program jogosultságaival élhet. A MAC ugyanezen a szinten dolgozik: nem akadályozza meg, hogy a folyamat egy engedélyezett fájlhívás után például `execve`-t hívjon, ha a szabályok azt nem tiltják. A seccomp viszont a rendszerhívások szintjén vágja el a lehetőséget: még ha a támadó tetszőleges gépi kódot futtat is a program kontextusában, a tiltott rendszerhívások egyszerűen meghiúsulnak, így nem tud új folyamatot indítani vagy a kernelt tovább támadni. Ezért a három védelmi réteg együttese szükséges a támadási felület minimalizálásához.

Ez az alfejezet bemutatta a finomhangolt hozzáférés-vezérlés három kulcselemét: a Linux capability-ket, a SELinux és AppArmor MAC-rendszereket, valamint a seccomp rendszerhívás-szűrést. A következő fejezet ezekre az eszközökre építve a namespaces és cgroups konténerizációs technikákkal foglalkozik.

21.3 Namespaces, cgroups és konténerizoláció

A névtér- és erőforrás-izoláció két egymást kiegészítő mechanizmus, amelynek segítségével a kernel úgy ad folyamatoknak saját, korlátozott „gépezetet”, hogy közben a gazdagépen futó többi folyamatot nem engedi befolyásolni. A névtér azt szabályozza, hogy egy folyamat *mit lát* a rendszerből: mely folyamatok, mount pontok, hálózati interfészek, felhasználói azonosítók és IPC-objektumok láthatók számára. A cgroup azt szabályozza, hogy egy folyamatcsoport *mennyit fogyaszthat* a rendszer erőforrásaiból: CPU-időt, memóriát, I/O-sávszélességet, PID-eket. A két primitív együtt adja a konténerizoláció gerincét.

A fejezet tanulási céljai: megmagyarázni, hogyan épül fel a PID-, mount-, network- és user namespace izoláció; alkalmazni ezeket egy kis, ellenőrizhető példán; felismerni a tipikus hibákat, és megindokolni a helyes megoldást. Ehhez először a namespace-ek belső modelljét, majd a cgroup v2 erőforrás-vezérlését tekintjük át, végül egy reprodukálható labort adunk.

21.3.1 1. A namespace mint láthatósági határ

A Linux kernelben a namespace egy folyamathoz kötött nézet. Ugyanaz a rendszerobjektum – például egy mount pont, egy hálózati interfész, egy PID vagy egy UID – különböző namespace-ekben különböző azonosítóval, láthatósággal vagy egyáltalán nem láthatóan jelenik meg. A namespace-eket három fő művelettel lehet létrehozni és módosítani:

- `clone(2)`: új folyamatot hoz létre, és a `CLONE_NEW*` flaggel egyidejűleg új namespace-t is ad neki.
- `unshare(2)`: a hívó folyamatot egy új namespace-be helyezi anélkül, hogy új folyamatot hozna létre.
- `setns(2)`: egy már létező namespace-hez csatlakoztat egy folyamatot; ez a konténer-runtime-ok egyik alapművelete.

Minden namespace egy kernelobjektum, amelynek élettartamát referenciaszámláló tartja fenn. A namespace addig él, amíg legalább egy folyamat tagja, vagy egy nyitott fájlleíró, illetve bind mount hivatkozik rá. A `/proc/<pid>/ns/` könyvtár alatt minden folyamat namespace-ei szimbolikus linkként láthatók; a linkek célja az adott namespace típusát és inode-számát azonosítja:

```
$ ls -l /proc/self/ns
total 0
lrwxrwxrwx 1 user user 0 Feb 20 08:00 cgroup -> cgroup:[4026531835]
lrwxrwxrwx 1 user user 0 Feb 20 08:00 ipc -> ipc:[4026531839]
lrwxrwxrwx 1 user user 0 Feb 20 08:00 mnt -> mnt:[4026531841]
lrwxrwxrwx 1 user user 0 Feb 20 08:00 net -> net:[4026531840]
lrwxrwxrwx 1 user user 0 Feb 20 08:00 pid -> pid:[4026531836]
lrwxrwxrwx 1 user user 0 Feb 20 08:00 user -> user:[4026531837]
lrwxrwxrwx 1 user user 0 Feb 20 08:00 uts -> uts:[4026531838]
```

Két folyamat akkor van ugyanabban a namespace-ben, ha a megfelelő link ugyanarra az inode-ra mutat. Ezt gyakran használják diagnosztikára: ha egy konténerben futó folyamat PID-namespace-e eltér a gazdagépétől, akkor a konténer saját PID-teret lát.

A Linux jelenleg nyolc namespace-típust ismer: `cgroup`, `ipc`, `mnt`, `net`, `pid`, `time`, `user` és `uts`. A konténerizoláció szempontjából a legfontosabbak a PID, mount, network és user namespace-ek, mert ezek adják a folyamat-, fájlrendszer-, hálózat- és jogosultsági izolációt. A cgroup namespace a cgroup-hierarchia láthatóságát szűri, az IPC namespace a System V IPC-objektumokat és POSIX üzenetsorokat, a UTS namespace a hoszt- és domainnevet, a time namespace pedig a CLOCK_MONOTONIC és CLOCK_BOOTTIME értékeket izolálja.

A namespace-ek nem adnak automatikus erőforrás-korlátot. Egy új PID namespace-ben lévő folyamat továbbra is ugyanazt a CPU-t, memóriát és hálózati sávszélességet fogyasztja, mint a többi; csak a láthatósága változik. Ezt a hiányosságot pótolják a cgroup-ok.

21.3.2 2. PID namespace

A PID namespace a folyamatok azonosítóterét izolálja. Egy folyamat a saját PID namespace-ében kap egy PID-et, a szülő namespace-ekben pedig más, az adott szintű namespace-nek megfelelő PID-et. Például egy konténer első folyamata a konténer PID namespace-ében PID 1, a gazdagép PID namespace-ében viszont PID 4327 lehet. A kernel minden szinten fenntart egy leképezést, így jelzések küldésekor vagy a `/proc` olvasásakor a névtérbeli azonosító használható.

Az új PID namespace első folyamata különleges szerepet kap: ez az adott névtér `init` folyamata. Ez a folyamat végzi az árva folyamatok begyűjtését, és ha meghal, a kernel az egész namespace összes többi folyamatát `SIGKILL` jellel állítja le. Ez a viselkedés garantálja, hogy ne maradjon init nélküli folyamatcsoport. Konténerekben ezért a belépő folyamatnak – gyakran a shell, egy init vagy a konténer-runtime által indított alkalmazás – init szerepet kell betöltenie. Ha ez a folyamat kilép, a konténer összes maradék folyamata megsemmisül.

Gyakorlati példa egy új PID namespace indítására az `unshare` parancsal:

```
# Példa: izolált PID névtér és saját /proc
unshare --pid --fork --mount-proc /bin/sh
```

A `--pid` új PID namespace-t hoz létre, a `--fork` azért szükséges, hogy az `unshare` gyermeke legyen a névtér PID 1 folyamata, a `--mount-proc` pedig egy új mount namespace-t is létrehoz, és benne a `/proc` fájlrendszert a névtér PID-terének megfelelően mountolja. E nélkül a `/proc` még mindig a gazdagép folyamatait mutatná, ami biztonsági és funkcionális hibához vezetne: a

PID 1 folyamat a saját névtérében nem látná a többi folyamatot, viszont a gazdagép `/proc`-on keresztül a teljes folyamatlistát elérné.

A PID namespace hatással van a jelzések küldésére is. A `kill(1)` a cél namespace-ében értelmezi a PID-et. Egy folyamat a szülő namespace-ek felé küldhet jelet, ha a cél folyamatra nézve van jogosultsága, de a gyermek namespace-ekbe nem láthat be fordítva. Ez aszimmetrikus izolációt ad: a szülő felügyelheti a gyermeket, a gyermek nem láthatja a szülő többi folyamatát.

21.3.3 3. Mount namespace

A mount namespace a mount pontok láthatóságát és a mountok élettartamát izolálja. Amikor egy folyamat új mount namespace-t hoz létre, a kernel a jelenlegi mount tábla másolatát adja neki. Ezt követően a folyamat által elvégzett `mount` és `umount` műveletek csak a saját namespace-ét módosítják, a többi namespace mount táblája változatlan marad.

A mount namespace-ekhez kapcsolódik a propagáció fogalma. Egy mount pont lehet:

- `shared`: a mount események továbbterjednek a többi shared mount pontra;
- `private`: a mount esemény csak a saját namespace-ben látható;
- `slave`: egyirányú kapcsolat – a master mount eseményei látszanak, de a slave eseményei nem terjednek vissza;
- `unbindable`: a mount nem bindelhető, ezzel korlátozva a namespace-ek közötti átjárást.

A propagáció azért fontos, mert a konténer-runtime-oknak el kell kerülniük, hogy egy konténerben elvégzett mount átjusson a gazdagépre. Ha például a `/` mount pont `shared`, akkor egy konténerben létrehozott mount elterjedhet a gazdagép namespace-ébe, és ott maradandó változást okozhat. A helyes beállítás általában az, hogy a konténer mount namespace-ében a fontos mount pontokat `private` vagy `slave` módba állítják.

Egy mount namespace létrehozása és a propagáció beállítása:

```
# Példa: külön mount névtér, privát propagációval
unshare --mount --propagation private /bin/sh
# A névtérben a /tmp mappát egy külön tmpfs-ként mountoljuk
mount -t tmpfs tmpfs /tmp
```

Ebben a példában a `/tmp` mount csak az adott namespace-ben létezik; a gazdagép `/tmp`-je változatlan marad. A mount namespace-ek gyakran együtt járnak a root fájlrendszer cseréjével: `pivot_root`-tal vagy bind mounttal egy konténer saját gyökérfájlrendszert kap, amely alatt a gazdagép fájlrendszere nem látható. A mount namespace tehát a fájlrendszer-izoláció alapja, de önmagában nem véd a gazdagép eszközfájljainak elérésétől: ha a `/dev`, `/sys` vagy `/proc` nincs megfelelően szűrve, a folyamat hozzáférhet a gazdagép erőforrásaihoz.

21.3.4 4. Network namespace

A network namespace a hálózati verem izolálását szolgálja: a folyamat saját hálózati interfészeket, IPv4/IPv6 címeket, routing táblát, socket portteret, tűzfal-szabályokat és statisztikákat lát. Egy új network namespace alapértelmezésben csak egy loopback interfészt tartalmaz, amely le van állítva. Ez azt jelenti, hogy az izolált folyamat a loopback kivételével semmilyen hálózati interfészt nem lát, és alapból nem tud kommunikálni a külvilággal.

A konténer-runtime-ok ezért hoznak létre egy `veth` (virtual Ethernet) párt: az egyik végét a konténer network namespace-ébe helyezik, a másik végét a gazdagép egy bridge-ére kötik. A

veth pár két végpontja közötti csomagáramlás olyan, mintha két virtuális Ethernet-kártya lenne kábellel összekötve.

Egy minimális network namespace felállítása és a loopback aktiválása:

```
# Példa: új network névtér, benne loopback felkapcsolása
unshare --net /bin/sh
ip link set lo up
ip addr show
```

A `ip addr show` ekkor csak a loopback interfészt mutatja. Ha a folyamat `ping 127.0.0.1` parancsot futtat, az a saját névtér loopbackjén halad, és nem érinti a gazdagépet. A külvilággal való kommunikációhoz a gazdagépnek át kell helyeznie egy veth interfészt a névtérbe:

```
# Gazdagép-oldali példa: veth pár létrehozása és átadása
ip link add veth0 type veth peer name veth1
ip link set veth1 netns <cél-namespace>
ip link set veth0 up
ip addr add 10.0.0.1/24 dev veth0
```

A cél namespace-ben ezután a `veth1` interfész látható, és konfigurálható. A network namespace tehát nemcsak a láthatóságot szűri, hanem a hálózati konfigurációt is elszigeteli: a konténer saját routing táblát és tűzfalszabályokat használhat anélkül, hogy a gazdagép beállításait módosítaná.

21.3.5 5. User namespace

A user namespace a felhasználói és csoportazonosítók leképezését izolálja. Ez a mechanizmus lehetővé teszi, hogy egy folyamat a saját namespace-ében `root` (UID 0) legyen és teljes képességkészlettel rendelkezzen, miközben a gazdagép namespace-ében továbbra is egy közönséges, jogosultság nélküli felhasználó. A kernel a UID/GID leképezést a `/proc/<pid>/uid_map` és `/proc/<pid>/gid_map` fájlokon keresztül mutatja.

Egy egyszerű leképezés például:

```
# /proc/<pid>/uid_map
0 1000 1
```

Ez azt jelenti, hogy a névtérbeli UID 0 a gazdagép UID 1000-jére képződik le. A névtérben futó `root` folyamat tehát a gazdagép szemszögéből a 1000-es felhasználó jogosultságaival fut. A user namespace azért kulcsfontosságú a konténerizolációban, mert lehetővé teszi, hogy a konténerben futó folyamat `root`-ként végezzen rendszeradminisztrációs műveleteket – például mountoljon, hálózati interfészt konfiguráljon, PID namespace-t hozzon létre – anélkül, hogy a gazdagépen valódi root jogosultságot kapna.

A user namespace-ekhez kapcsolódó képességek a névtér határain belül érvényesek. A `CAP_SYS_ADMIN` a user namespace-ében lehetővé teszi mountok létrehozását, de csak olyan mountokét, amelyek a saját mount namespace-ükre korlátozódnak. Egyes műveletek, például a gazdagép kernelmoduljainak betöltése vagy a gazdagép hálózati konfigurációjának módosítása, továbbra is a gazdagép user namespace-ében érvényes képességeket igényelnek. A user namespace tehát a jogosultságok hatókörét szűkíti, nem ad korlátlan hozzáférést.

Gyakorlati példa egy user namespace létrehozására és a root leképezésére:

```
# Példa: user namespace létrehozása, névtérbeli root
unshare --user --map-root-user /bin/sh
id
# uid=0(root) gid=0(root) groups=0(root)
```

A `--map-root-user` gondoskodik arról, hogy a névtérbeli UID 0 a hívó felhasználóhoz legyen hozzárendelve. Fontos biztonsági megfontolás: a user namespace nem szünteti meg a gazdagép védelmét, hanem új réteget ad hozzá. Ha egy kernel sebezhetőség lehetővé teszi a user namespace-ből való kitörést, a támadó gazdagép szintű jogosultságot szerezhet. Ezért a disztribúciók gyakran korlátozzák a user namespace-ek létrehozását (`kernel.unprivileged_userns_clone`), vagy seccomp-szabályokkal tiltják a felesleges rendszerhívásokat.

21.3.6 6. cgroup: az erőforrás-vezérlés rétege

A cgroup (control group) a folyamatok csoportosítását és erőforrás-fogyasztásuk szabályozását végzi. A Linux kernel két generációt támogat: a cgroup v1-et és a cgroup v2-t. A v1 minden kontrollerhez külön hierarchiát rendel, ami rugalmas, de bonyolult; a v2 egységes hierarchiát használ, amelyben minden kontroller ugyanazon a fán helyezkedik el. A modern konténer-runtime-ok a cgroup v2-t preferálják, mert egyszerűbb delegációs modellje és jobb integrációja van.

A cgroup v2 a `/sys/fs/cgroup` könyvtár alatt jelenik meg. A gyökér cgroup mindig létezik, és alatta tetszőleges mélységű alkönyvtárak hozhatók létre. Minden alkönyvtár egy cgroup, amelynek fájljai vezérlik az adott kontroller viselkedését. A folyamatok hozzárendelése a `cgroup.procs` fájlba írt PID-ekkel történik; a vezérlőfájlokba írt értékek a cgroup egészére vonatkoznak.

Egy cgroup létrehozása és egy folyamat hozzáadása:

```
# Példa: cgroup v2 labor csoport létrehozása
mkdir /sys/fs/cgroup/labor
echo $$ > /sys/fs/cgroup/labor/cgroup.procs
```

Ezután a cgroup `labor` alatt lévő vezérlőfájlokba írt értékek az adott folyamatra és minden leszármazottjára érvényesek.

21.3.7 CPU-vezérlés

A CPU-vezérlés két komponensből áll: súlyozott sávszélesség-megosztásból és abszolút korlátból.

- `cpu.weight`: 1 és 10000 közötti súly, amely a cgroupok közötti CPU-idő arányos elosztását szabályozza. Ha két cgroup egyformán terhelt, a magasabb súlyú cgroup nagyobb arányban kap CPU-időt.
- `cpu.max`: egy `quota period` formátumú érték, például `20000 100000`. Ez azt jelenti, hogy a cgroup legfeljebb 20 000 mikroszekundum CPU-időt fogyaszthat minden 100 000 mikroszekundumos periódusban, azaz egy CPU 20%-át kaphatja. Ha a `quota` értéke `max`, akkor nincs abszolút korlát.

A `cpu.max` előnye, hogy determinisztikus korlátot ad, ami valós idejű és teljesítménytesztkörnyezetekben is hasznos. A súly ezzel szemben csak akkor érvényesül, ha a rendszer túlterhelt; ha van szabad CPU, a cgroup a súlyától függetlenül több időt is kaphat.

21.3.8 Memóriavezérlés

A memóriavezérlés három fő vezérlőfájlt használ:

- `memory.max`: a cgroup számára elérhető maximális memória byte-ban. Ha a folyamatcsoport ezt meghaladja, a kernel OOM kill-t hajt végre a cgroupon belül.
- `memory.high`: egy puha határ. Ha a fogyasztás meghaladja, a kernel visszanyomja a cgroup folyamatait (throttling), de nem azonnal öli meg őket. Ez a mechanizmus a memórianyomás finom szabályozására alkalmas.
- `memory.low`: egy minimális memóriamennyiség, amelyet a kernel véd a globális memória-visszavétel során. Ha a rendszer memórianyomás alá kerül, a `memory.low` alatti részt a kernel nem veszi el más cgroupoktól.

A cgroup v2 emellett számos eseményt is számlál a `memory.events` fájlban: `oom`, `oom_kill`, `high`, `max`. A `oom_kill` számláló mutatja, hányszor ölt meg a kernel folyamatot a cgroupban a `memory.max` túllépése miatt. Ez diagnosztikában és tesztelésben közvetlenül használható.

21.3.9 I/O-vezérlés

Az I/O-vezérlés a blokk eszközökhöz kapcsolódó sávszélességet és IOPS-t korlátozza. A `io.max` fájlba eszközönként adható meg a maximális olvasási/írási sebesség és IOPS:

```
# io.max formátuma: major:minor rbps=<n> wbps=<n> riops=<n> wiops=<n>
8:0 rbps=10485760 wbps=5242880 riops=100 wiops=100
```

Ez az érték azt jelenti, hogy a 8:0 major/minor eszközön a cgroup legfeljebb 10 MiB/s olvasási és 5 MiB/s írási sávszélességet kaphat, valamint legfeljebb 100 olvasási és 100 írási IOPS-t. A súlyozott I/O-vezérlés a `io.weight` fájlban állítható be, és a CPU-súlyhoz hasonlóan arányos megosztást ad a túlterhelt eszközökön.

A cgroup v2-ben fontos szabály, hogy egy cgroupban vagy belső (csak gyermek cgroupokat tartalmazó) csomópont vagy levél (csak folyamatokat tartalmazó) csomópont lehet. Ha egy cgroupnak gyermek cgroupjai vannak, akkor a saját `cgroup.procs` fájljába nem lehet folyamatot írni. Ezt a szabályt a hierarchia delegálásakor kell figyelembe venni: a konténer-runtime a konténer cgroupját gyakran úgy alakítja ki, hogy a belépő folyamat a gyökér cgroup egy gyermekébe kerül, és a delegált alcsoportokat a konténer maga hozhatja létre.

21.3.10 7. A két primitív együtt: konténerizoláció

Egy konténer nem önálló entitás a kernelben, hanem namespace-ek és cgroup-ok kombinációja. A konténer-runtime a következő lépéseket hajtja végre:

1. Létrehoz egy új user namespace-t, hogy a konténerben futó root ne legyen gazdagép root.
2. Létrehoz egy új PID namespace-t, így a konténer első folyamata PID 1.
3. Létrehoz egy új mount namespace-t, és a konténer image-ét bind mounttal vagy overlayfs-szel a konténer gyökerévé teszi.
4. Létrehoz egy új network namespace-t, és veth párral csatlakoztatja a gazdagép bridge-éhez.
5. Létrehoz egy cgroupot, beállítja a CPU-, memória- és I/O-limiteket, és a konténer folyamatát a cgroupba helyezi.

A folyamat az így kialakított környezetben úgy fut, mintha egy saját minimális gépen lenne: saját folyamatteret, saját fájlrendszert, saját hálózati verem és saját felhasználókat lát, miközben a gazdagép erőforrásait a cgroup korlátai között fogyasztja.

Fontos hangsúlyozni, hogy a namespace-ek és cgroup-ok nem biztonsági homokozó. Ezek a kernel erőforrás-elszigetelési primitívei, nem szavatolnak teljes védelmet a kernel sebezhetőségeivel szemben. A biztonsági rétegek – seccomp, capabilities, SELinux/AppArmor, valamint a hardveres virtualizáció – a következő fejezetben tárgyalt mérési és integritási mechanizmusokkal együtt egészítik ki ezt a modellt.

21.3.11 8. Tipikus hibák és helyes megoldások

A namespace- és cgroup-konfiguráció során számos hiba léphet fel. Az alábbi lista a leggyakoribbakat foglalja össze.

Hiba 1: A PID namespace-ben nincs init folyamat. Ha az első folyamat kilép, a kernel az egész namespace-t leállítja, és az árva folyamatok `SIGKILL`-t kapnak. A helyes eljárás az, hogy a belépő folyamat vagy maga az init szerepet tölti be, vagy egy init rendszert (például `tini`) indít.

Hiba 2: A `/proc` mountolása a gazdagép PID namespace-ét mutatja. Ha a `--mount-proc` hiányzik, a `/proc` a gazdagép folyamatait listázza, ami sértheti az izolációt. A helyes eljárás: új mount namespace és `mount -t proc proc /proc` a PID namespace-en belül.

Hiba 3: A user namespace leképezés hiánya. Ha a `--map-root-user` vagy az `uid_map` beállítás elmarad, a mount és egyéb műveletek `EINVAL` hibával meghiúsulhatnak. A leképezést mindig a user namespace létrehozása után, a folyamat `exec`-je előtt kell beállítani.

Hiba 4: A network namespace-ben a loopback le van állítva. Ez gyakori meglepetés: az `ip link set lo up` nélkül a konténer nem tudja elérni a saját loopbackjét, és a helyi szolgáltatások nem indulnak el.

Hiba 5: A mount propagáció shared módja miatt a konténer mountjai átjutnak a gazdagépre. A konténer mount namespace-ében a `--propagation private` vagy `slave` beállítás szükséges ahhoz, hogy a mountok ne terjedjenek vissza.

Hiba 6: A cgroup v2-ben a szülő cgroup nem tartalmazhat folyamatot, ha gyermek cgroupjai vannak. Ha egy folyamatot írnak a szülő `cgroup.procs` fájljába, miután alcsoportot hoztak létre, a kernel `EBUSY` hibát ad. A helyes eljárás: a folyamatot a megfelelő levél cgroupba helyezni.

Hiba 7: A `memory.high` és a `memory.max` összekeverése. A `memory.high` puha határ, amely throttlinget vált ki, de nem öli meg a folyamatot; a `memory.max` kemény határ, amely OOM killt eredményez. Ha a teszt célja az OOM viselkedés ellenőrzése, akkor `memory.max`-ot kell beállítani.

Hiba 8: A `cpu.max` periódusának rossz értelmezése. A `cpu.max` értékét mikroszekundumban kell megadni, és a periódus általában 100 000 µs (100 ms). Ha valaki 1000-et ír periódusnak, akkor a quota más időskálán érvényesül, és a tényleges CPU-korlát eltér a várttól.

Hiba 9: A cgroup v1 és v2 keverése. Ugyanaz a kontroller nem lehet egyszerre v1 és v2 hierarchiában. Ha a rendszeren a `memory` kontroller v1-ben van aktiválva, a cgroup v2 `memory` vezérlőfájljainak írása `EBUSY` hibát ad. A megoldás a kernel indítási paramétereinek beállítása (`cgroup_no_v1=memory`) vagy a v1 hierarchia felszámolása.

Hiba 10: A `setns` hívás jogosultsági hibája. A `setns` csak akkor engedélyezett, ha a hívó folyamat rendelkezik a cél namespace-re vonatkozó képességgel, vagy ha a cél namespace-t a hívó

user namespace-e lefedi. A konténer-runtime-oknak ezért gondosan kell kezelniük a namespace-ek közötti váltást.

21.3.12 9. Labor: izolált környezet tervezése és mérése

A labor célja egy minimális izolált folyamatkörnyezet létrehozása PID, mount, network és user namespace-szel, valamint CPU- és memória-cgroup korláttal. A labor emulált környezetben fut, így a gazdagép valódi rendszerpartíciója nem módosul. A következő konfiguráció rögzített; a mérési naplóban a várt és a mért értékeket elkülönítve adjuk meg.

21.3.13 Kiinduló állapot

- Emulált gép: QEMU `virt` géptípus, 2 vCPU, 512 MiB RAM.
- Kernel: Linux 6.6 LTS (ellenőrizendő a disztribúció pontos verziója).
- Indítás: `qemu-system-x86_64 -cpu host -smp 2 -m 512 -kernel bzImage -initrd initramfs.cpio.gz -nographic`
- Az initramfs minimális busybox környezetet tartalmaz, systemd nélkül.
- A cgroup v2 a `/sys/fs/cgroup` alá van mountolva; ellenőrzés: `stat -fc %T /sys/fs/cgroup` értéke `cgroup2fs`.

21.3.14 1. lépés: cgroup létrehozása és limitek beállítása

Hozzon létre egy `labor` cgroupot, és állítsa be a memória- és CPU-korlátot:

```
mkdir /sys/fs/cgroup/labor
echo "32M" > /sys/fs/cgroup/labor/memory.max
echo 0 > /sys/fs/cgroup/labor/memory.swap.max
echo "20000 100000" > /sys/fs/cgroup/labor/cpu.max
```

Ezek az értékek azt jelentik, hogy a cgroup legfeljebb 32 MiB memóriát fogyaszthat, swap nélkül, és legfeljebb egy CPU 20%-át kaphatja.

21.3.15 2. lépés: namespace-ek létrehozása

Indítson egy shell-t a namespace-ekkel, és helyezze a cgroupba. A következő parancs szemléltetés; a labor során a tényleges PID-et a `$$` gyermeke adja:

```
# Izolált shell: PID, mount, network, user namespace
unshare --pid --fork --mount --net --user --map-root-user --mount-proc \
    --propagation private /bin/sh
# A shell PID-je a névtérben 1
echo $$
```

A shell elindulása után a hálózati loopback felkapcsolása:

```
ip link set lo up
ip addr show
```

A `ip addr show` kimenete csak a loopback interfészt tartalmazza, a gazdagép hálózati interfészei nem láthatók.

21.3.16 3. lépés: a folyamat cgroupba helyezése

A shell indítása után a gazdagép oldalán keresse meg a folyamat gazdagépbeli PID-jét, és írja a cgroupba:

```
# Gazdagép oldalán (a névtérben futó folyamat host PID-jét használva)
echo <host_pid> > /sys/fs/cgroup/labor/cgroup.procs
```

Ellenőrzés a névtérben:

```
cat /proc/self/cgroup
# 0::/labor
```

A `0::/labor` kimenet azt jelzi, hogy a folyamat a `labor` cgroupban van, és a cgroup namespace nincs tovább szűrve.

21.3.17 4. lépés: mérések

CPU-korlát mérése. Indítson egy CPU-igényes folyamatot a névtérben, és mérje a futásidőt:

```
# A névtérben
time sh -c 'i=0; while [ $i -lt 10000000 ]; do i=$((i+1)); done'
```

A várt eredmény: mivel a cgroup legfeljebb 20% CPU-t kaphat, a futásidő körülbelül ötszöröse annak, mint korlát nélkül. A mért futásidőt a mérési naplóban rögzíteni kell. A méréshez a `time` builtin a valós időt (`real`) adja meg; a CPU-idő (`user` + `sys`) a ténylegesen felhasznált processzoridőt mutatja. A korlát hatására a `real` idő jóval nagyobb, mint a `user` idő.

Memóriakorlát mérése. Indítson egy memóriafoglaló folyamatot:

```
# A névtérben
python3 -c 'import time; b = bytearray(64 * 1024 * 1024); time.sleep(5)'
```

A várt eredmény: a 32 MiB `memory.max` túllépése miatt a kernel OOM kill-t hajt végre, a folyamat `SIGKILL`-t kap, és a `memory.events` `oom_kill` számlálója nő. A folyamat kilépési kódja 137 (128 + 9) lesz. A `memory.events` ellenőrzése:

```
cat /sys/fs/cgroup/labor/memory.events
# oom_kill 1
```

A `dmesg` naplóban az OOM kill eseménynek meg kell jelennie.

Hálózati izoláció mérése. Ellenőrizze, hogy a névtérben nem érhető el a gazdagép hálózata:

```
# A névtérben
ip route
# üres routing tábla, csak loopback
ping -c 1 8.8.8.8
# Network is unreachable
```

A várt eredmény: a hálózati névtérben nincs alapértelmezett útvonal, és a gazdagép interfészei nem láthatók. A `ping` parancs `Network is unreachable` hibát ad.

21.3.18 Mérési napló

Mérés	Konfiguráció	Várt eredmény	Mért eredmény	Eltérés
CPU-korlát	`cpu.max=20000 100000`	A valós futásidő kb. 5-szöröse a korlát nélküli futásidőnek	*laborban mérendő*	*laborban értékelendő*
Memóriakorlát	`memory.max=32M`, `memory.swap.max=0`	OOM kill, kilépési kód 137, `oom_kill` nő	*laborban mérendő*	*laborban értékelendő*
Hálózati izoláció	`unshare --net`, loopback up	Nincs route, a ping `Network is unreachable`	*laborban mérendő*	*laborban értékelendő*
PID-névtér	`unshare --pid --fork --mount-proc`	A shell PID 1, `/proc` csak a névtér folyamatait mutatja	*laborban mérendő*	*laborban értékelendő*

A mérési naplóban a hardvert, az emulált CPU modellt, az órajelfrekvenciát és a mérési módszert is rögzíteni kell. A CPU-mérésnél a `time` a shell beépítettje, amely a `getrusage(2)` alapján számol; a valós időt a kernel `clock_gettime(2)` szolgáltatja. A QEMU virtuális órája miatt a mért időket érdemes többször megismételni és átlagolni.

21.3.19 Visszaállítás

A labor után a cgroupot törölni kell, a folyamatokat le kell állítani, és a namespace-eket meg kell szüntetni. A legegyszerűbb visszaállítás az emulált gép újraindítása, mivel az initramfs nem módosít tartós állapotot. A gazdagép valódi lemezét, bootloaderét és rendszerpartícióját a labor nem érinti.

21.3.20 10. Önellenőrző kérdés

Egy fejlesztő létrehoz egy PID namespace-t `unshare --pid --fork /bin/sh` paranccsal, de nem használ `--mount-proc`-ot. A névtérben a `ps` parancs továbbra is a gazdagép folyamatait listázza. Mi a hiba oka, és miért jelent biztonsági kockázatot? A válaszban térjen ki arra is, hogyan lehet helyesen eljárni.

A fenti kérdés ellenőrzi, hogy a tanuló megértette-e a PID namespace és a mount namespace együttműködését: a `/proc` fájlrendszer mountolása a névtér PID-terének megfelelően történik, és enélkül a névtér izolációja nem teljes. A helyes megoldás a `--mount-proc` flag vagy egy új mount namespace és a `mount -t proc proc /proc` parancs használata.

21.3.21 11. Összefoglalás

A namespace-ek és cgroup-ok a Linux kernel két alapvető izolációs primitívje. A namespace-ek a rendszerobjektumok láthatóságát szűrik: a PID namespace a folyamatteret, a mount namespace a fájlrendszer mountjait, a network namespace a hálózati verem, a user namespace a felhasználói azonosítókat és képességeket izolálja. A cgroup-ok a folyamatcsoportok erőforrás-fogyasztását korlátozzák: a CPU-sávszélességet a `cpu.max` és `cpu.weight`, a memóriát

a `memory.max`, `memory.high` és `memory.low`, az I/O-t az `io.max` és `io.weight` szabályozza. A két mechanizmus együtt adja a konténerizoláció alapját, de önmagában nem helyettesíti a biztonsági rétegeket. A fejezetben bemutatott labor egy reprodukálható, emulált környezetben tervezett méréssorozatot ad, amelynek segítségével a namespace- és cgroup-konfiguráció gyakorlatban is ellenőrizhető. A következő egység ezekre a fogalmakra építve a megbízható indítás és a rendszerintegritás kérdéseit tárgyalja.

21.4 Trusted boot, mérés és integritás

A bootfolyamat minden elemére igaz, hogy ha egy korai komponenst kompromittálnak, akkor a későbbi komponensek integritása már nem igazolható önmagukban. Egy operációs rendszer védelme ezért nem kezdődhet a kernel betöltésekor: a betöltő, a firmware, sőt a hardver inicializáló kódja is a bizalmi lánc része. Ebben az alfejezetben a bootlánc és a futó rendszer integritásának biztosítását vizsgáljuk három egymást kiegészítő mechanizmussal: a Secure Boot aláírás-ellenőrzésével, a TPM-alapú méréssel és attestationnel, valamint a Linux Integrity Measurement Architecture (IMA) futásidőben végzett mérési és felülvizsgálati szabályaival.

21.4.1 A bootlánc és a bizalmi lánc fogalma

A bootlánc (boot chain) a rendszerindítás során egymás után vezérlést kapó komponensek sorozata: a firmware (BIOS vagy UEFI), a bootloader, a kernel, az initrd vagy initramfs, majd a userland init folyamat által indított szolgáltatások. Ha a támadó a lánc bármely pontján perzisztens kódot tud elhelyezni, akkor a lánc későbbi elemei már nem megbízhatóak. Ezért a biztonsági tervezésben a bootláncot két, eltérő célú eljárással erősítjük:

- **Hitelesítés (authentication)**: a következő komponens betöltése előtt ellenőrizzük, hogy az ismert, megbízható aláíró kulccsal rendelkezik-e. Ez a Secure Boot megközelítése: a nem aláírt vagy visszavont aláírású komponens nem kap vezérlést.
- **Mérés (measurement)**: a következő komponens hashét a betöltés előtt kiszámítjuk, és egy biztonságos hardveres nyilvántartásba, a TPM platformkonfigurációs regisztereibe (PCR) illesztjük. A mérés önmagában nem blokkolja a betöltést, de a későbbi ellenőrzés (attestation) számára bizonyítékot szolgáltat.

A két mechanizmus együtt dolgozik: a Secure Boot megakadályozza az ismert rossz kód betöltését, a measured boot pedig megőrzi a ténylegesen lefutott komponensek lenyomatát, hogy egy távoli fél vagy egy későbbi biztonsági szolgáltatás igazolhassa a rendszer állapotát.

A bizalmi lánc gyökere (root of trust) az a hardver- vagy firmware-komponens, amelyet feltétel nélkül megbízhatónak tekintünk. A mérési lánc gyökere a CRTM (Core Root of Trust for Measurement), amely tipikusan a firmware első, nem módosítható szakasza. A CRTM feladata, hogy a firmware további részeit megmérje, mielőtt azok lefutnának. A hitelesítési lánc gyökere a platformkulcs (Platform Key, PK) az UEFI Secure Boot esetén: az a kulcs, amelynek birtokosa a kulcshierarchia tetején áll, és amelyhez a firmware alábbi kulcsait kötik.

21.4.2 Secure Boot: aláírás és kulcsrendszer

A Secure Boot az UEFI firmware által végrehajtott aláírás-ellenőrzés. Betöltés előtt az UEFI firmware ellenőrzi, hogy az adott EFI-alkalmazás vagy illesztőprogram PE-képének (Portable Executable) aláírása érvényes-e, és hogy az aláíró kulcs szerepel-e a megengedett adatbázisban.

Ha az aláírás hiányzik, érvénytelen vagy a kulcs visszavont, a firmware megtagadja a komponens futtatását.

A Secure Boot kulcsai az UEFI változókban tárolódnak, és hierarchiát alkotnak:

- **Platform Key (PK)**: a platform tulajdonosának kulcsa. Ez a hierarchia gyökere; a PK privát részét a platform tulajdonosa birtokolja, és ezzel írja alá a KEK frissítéseit.
- **Key Exchange Key (KEK)**: a megbízható kulcsfrissítési kulcsok. A KEK-ekkel aláírt frissítések módosíthatják a megengedett és tiltott aláírások adatbázisait.
- **Allowed Signature Database (db)**: azoknak az aláíró kulcsoknak vagy hash-értékeknek a gyűjteménye, amelyeket a firmware elfogad az EFI-képek aláírásaként.
- **Forbidden Signature Database (dbx)**: azoknak a kulcsoknak vagy hash-értékeknek a gyűjteménye, amelyeket a firmware kifejezetten elutasít. Ide kerülnek a kompromittált bootloaderek és illesztőprogramok lenyomatai.

A következő pszeudokód a Secure Boot ellenőrzési logikáját mutatja:

```
secure_boot_check(image):
    if secure_boot_disabled:
        return BOOT
    if image_hash in dbx:
        return REJECT
    for signature in image.signatures:
        if verify_signature(image, signature.public_key) and
           (signature.public_key in db or hash(signature.public_key) in db):
            return BOOT
    return REJECT
```

A gyakorlatban az aláírás láncolt: a bootloader aláírása lehet közvetlenül a db-ben lévő kulccsal, vagy a db-ben lévő kulcs által aláírt köztes tanúsítvánnyal. A kernelbetöltés modern Linux rendszereken nem közvetlenül a firmware-en keresztül történik, hanem a bootloader (például a shim) által ellenőrzött, saját aláírású képként. A shim egy olyan, a db-ben szereplő aláírással rendelkező köztes bootloader, amely átveszi a hitelesítést, és lehetővé teszi a disztribúciós kulcsokkal aláírt kernel és initrd betöltését. A shim tehát kibővíti a Secure Boot láncot anélkül, hogy a platform db-jét minden disztribúciós kulccsal feltöltenénk.

A Secure Boot korlátja az, hogy csak a boot idejű binárisok integritását és hitelességét garantálja. Nem ad folyamatos védelmet a futó rendszer fájljaira, és nem bizonyítja, hogy a rendszer valóban az elvárt konfigurációban fut. Ha a támadó egy aláírt, de sebezhető bootloadert használ ki, vagy a futó kernel memóriáját módosítja, a Secure Boot önmagában nem észleli a támadást.

21.4.3 TPM és mérés: PCR-ek, event log és attestation

A Trusted Platform Module (TPM) egy hardveres biztonsági chip, amely kriptográfiai kulcsokat tárol, és platformkonfigurációs regisztereit (PCR) úgy tervezték, hogy a mérési lánc bizonyítékait megbízhatóan gyűjtse. A TPM 2.0 szabvány szerint a PCR-ek nem közvetlenül felülíródó regiszterek, hanem csak kiterjeszteni (extend) lehet őket:

$$\mathrm{PCR}_n \leftarrow H(\mathrm{PCR}_n \,\|\, \mathrm{event_digest})$$

Itt H tipikusan SHA-256, $\|$ pedig konkatenáció. A kiterjesztés irreverzibilis: a regiszter aktuális értéke az összes korábbi mérés sorrendfüggő lenyomata. A mérési eseménynek a hash-értékén kívül a TPM-en kívüli naplóban is meg kell jelennie, hogy a PCR értéke visszafejthető legyen: a napló tartalmazza az esemény nevét, a mért komponens azonosítóját és a hash-értéket.

A mérési lánc jellemzően a következő:

1. A CRTM megméri a firmware fennmaradó részét, és a digestet a PCR 0-ba illeszti. A CRTM kódját feltételezzük, hogy nem módosítható.
2. A firmware megméri a bootloadert (PCR 4 például UEFI esetén), valamint a Secure Boot konfigurációját és a boot eszköz kiválasztását.
3. A bootloader megméri a kernel képét, az initrd-t és a kernel parancssort.
4. A kernel elindulása után a Linux IMA vagy más mérő komponens további PCR-eket (például PCR 10 az IMA esetén) bővít.

A mérési napló egy lehetséges bejegyzése:

```
PCR   Event                                Digest
0     EV_CRTM_VERSION                      7f83b165...
1     EV_POST_CODE                         9d...
4     EV_EFI_BOOT_SERVICES_APPLICATION     c0a8...
      (GRUB bootloader)
8     EV_IPL                               ...
10    IMA: /usr/bin/bash                   a3b2...
```

A TPM a PCR-ek leolvasását és aláírását is támogatja. Az attestation (igazolás) az a folyamat, amelyben egy távoli fél meggyőződik arról, hogy a platform valóban az elvárt állapotban van. Ehhez a TPM egy úgynevezett quote művelettel aláírt lenyomatot ad a PCR-ek aktuális értékeiről, egy olyan aláíró kulccsal, amelyet a TPM-en belül tárolnak. A távoli fél ellenőrzi:

- a quote aláírását, és hogy az aláíró kulcs valóban az adott TPM-hez kötődik;
- a PCR értékeket összeveti a mérési naplóval, és a naplón keresztül az ismert, jó állapot hash-értékeivel;
- egy friss, véletlen nonce-t ad a quote kéréshez, hogy a válasz ne legyen visszajátszható.

A TPM kulcshierarchiájának fontos elemei a mérési és attestációs használatban:

- **Endorsement Key (EK)**: a TPM gyártásakor beégetett egyedi kulcs, amely az adott chipet azonosítja. A privát rész soha nem hagyja el a TPM-et.
- **Attestation Identity Key (AIK)**: az EK-ből származtatott, az attestation céljára szolgáló aláíró kulcs, amely nem fedi fel közvetlenül a chip gyártó általi egyedi azonosítóját, és így korlátozza a platformkövetést.
- **Storage Root Key (SRK)**: a TPM-ben tárolt, a platform által használt kulcsok hierarchiájának gyökere.

A TPM emellett támogatja a **sealing** műveletet: egy adat (például a lemeztitkosító kulcs) csak akkor adható vissza, ha a PCR-ek meghatározott állapotban vannak. Ha a bootlánc megváltozik, a PCR értékek eltérnek, és a TPM nem oldja fel a titkot. Ez a tulajdonság a teljes lemeztitkosítást is erősíti: a boot során a lemezkulcs csak akkor nyerhető ki, ha a firmware, a bootloader és a kernel mért állapota megegyezik az elvárttal.

A TPM PCR bankjei közötti különbségek fontosak. A TPM 2.0 több hash-algoritmushoz tartozó bankot is támogathat, például SHA-1 és SHA-256. A PCR kiterjesztése minden aktív bankban megtörténik, de a mérési naplónak minden bankhoz tartalmaznia kell a megfelelő hash-t. Ha a napló csak SHA-1 hash-eket tárol, akkor a SHA-256 bank PCR értékeit nem tudjuk ellenőrizni.

21.4.4 IMA: Linux Integrity Measurement Architecture

A Linux Integrity Measurement Architecture (IMA) a kernelben futó mérési és felülvizsgálati keretrendszer. Az IMA a fájlok megnyitásakor a kernelbe hívott hookokon keresztül hash-értéket számít a fájl tartalmáról, a mérést a PCR 10-be illeszti, és a mérési eseményt a kernel biztonsági naplójába vagy a `securityfs` alatt elérhető listába írja. Az IMA két fő üzemmódja:

- **measure**: a mérés megtörténik, de a fájl betöltése nem akad meg. Az esemény bekerül a mérési listába, így később a PCR 10 és a napló összevetésével ellenőrizhető.
- **appraise**: a mérés mellett a fájl aláírását vagy hash-ét egy ismert értékhez is hasonlítja, és szabálysértés esetén a hozzáférés megtagadható. Ez lokalizált, fájlszintű integritásvédelmet ad futásidőben.

Az IMA szabályzatot a kernel parancssorban vagy a `ima_policy` interfészen keresztül lehet beállítani. Egy lehetséges, didaktikus szabályzat:

```
measure func=FILE_CHECK mask=MAY_EXEC
measure func=FILE_CHECK mask=MAY_READ uid=0
appraise func=FILE_CHECK mask=MAY_EXEC appraise_type=imasig
```

Az első sor minden végrehajtandó fájlt megmér. A második sor a root által olvasott fájlokat is. A harmadik sor a végrehajtandó fájloknál az `imasig` aláírást is ellenőrzi. A szabályzatok pontos elérhető funkciói kernelverziótól függnek, ezért a konkrét beállításoknál a használt kernel dokumentációját ellenőrizni kell.

Az IMA mérési listáját a `securityfs` alatti `integrity/ima/ascii_runtime_measurements` fájlban lehet olvasni. Egy bejegyzés szerkezete leegyszerűsítve:

```
10 0123456789abcdef... ima-ng sha256:fedcba9876... /usr/bin/bash
```

Az első mező a PCR index, a második a template hash, a harmadik a template neve, a negyedik a fájl hash-e, az utolsó pedig a fájl elérési útja. A PCR 10 értékét ezen bejegyzések sorrendben történő újraszámításával kell ellenőrizni.

Az IMA erőssége, hogy a boot után is folyamatosan méri a rendszer által használt fájlokat. Így a Secure Boot és a measured boot kiterjesztése a futó rendszerre. Korlátja, hogy a szabályzat kialakítása és a mérési lista méretének kezelése gondos tervezést igényel, különösen nagy forgalmú rendszereken.

21.4.5 Kulcskezelés a boot- és integritásláncban

A Secure Boot, a TPM attestation és az IMA aláírás mind aszimmetrikus kriptográfiára épül. A kulcskezelés elvei:

- A privát aláíró kulcsokat offline vagy hardveres biztonsági modulban (HSM) tárolják, nem a futó rendszeren.
- A Secure Boot PK privát kulcsához csak a platform tulajdonosa fér hozzá; a KEK és a db frissítések aláírására szolgál.
- Az IMA aláírásokhoz használt kulcs elkülönül a Secure Boot kulcsoktól, így a fájlszintű aláíró kulcs kompromittálódása nem érinti közvetlenül a boot hitelesítését.
- A TPM EK privát kulcsa soha nem hagyja el a TPM-et; az AIK-ot az adott TPM-en belül generálják, és a publikus rész hitelesítése tanúsítvánnyal történhet.

A hibrid kulcsfelépítés előnye, hogy a platform db-jét nem kell minden aláíróval feltölteni: a shim aláírása a platform db-jében van, a disztribúció kernelje pedig a shim által hordozott kulccsal van aláírva. A lánc tehát:

```
UEFI db → shim aláírás → disztribúciós kernel aláírás → kernel image
```

A mérési lánc ettől függetlenül rögzíti a ténylegesen betöltött shim, kernel és initrd hash-ét, ezért ha a disztribúciós kulcsot visszavonják, de a régi kernel még lefut, a PCR-ek eltérése ezt jelzi.

21.4.6 Gyakorlati minta: measured boot emulált környezetben

Az alábbi minta egy QEMU-alapú emulált környezetet mutat be, amelyben a Secure Boot és a TPM mérési lánc reprodukálható. A példa parancsai szemléltetésre szolgálnak; a fájlnevek és a konfiguráció a tényleges környezettől függenek, ezért minden használat előtt ellenőrizni kell a disztribúció csomagjainak verzióját.

A házigazdán a TPM emulátor, például `swtpm` indítása:

```
mkdir -p /tmp/mytpm
swtpm socket \
  --tpm2 \
  --tpmstate dir=/tmp/mytpm \
  --ctrl type=unixio,path=/tmp/mytpm.sock \
  --log level=2
```

A QEMU indítása OVMF firmware-rel és TPM-eszközzel:

```
qemu-system-x86_64 \
  -machine q35,accel=kvm \
  -m 2048 \
  -drive if=pflash,format=raw,unit=0,file=/usr/share/OVMF/OVMF_CODE.secboot.fd,readonly=on \
  -drive if=pflash,format=raw,unit=1,file=/tmp/OVMF_VARS.secboot.fd \
  -chardev socket,id=chrtpm,path=/tmp/mytpm.sock \
  -tpmdev emulator,id=tpm0,chardev=chrtpm \
  -device tpm-tis,tpmdev=tpm0 \
  -drive file=/tmp/debian-lab.img,format=qcow2
```

A fenti parancsokban a `OVMF_CODE.secboot.fd` a Secure Boot-ot támogató firmware, a változófájl `OVMF_VARS.secboot.fd` tartja a db, dbx és PK beállításokat. A vendégrendszerben a TPM eszköz a `/dev/tpm0` karakteres eszközként jelenik meg, a PCR-ek olvasása pedig például a `tpm2` eszközkészlettel történhet:

```
tpm2_pcrread sha256:0,1,2,3,4,7,8,9,10
```

A mérési eseménynapló kiolvasása a vendégben:

```
tpm2_eventlog /sys/kernel/security/tpm0/binary_bios_measurements
```

A kimenetet a hallgatónak össze kell vetnie a várt hash-értékekkel. A várt hash-értékek előállításához egy tiszta telepítésű környezetben kell rögzíteni a mérési naplót, és azt verziózott, hitelesített módon tárolni. A mért értékeket a naplóból kell kinyerni; a két oszlopot külön kell dokumentálni.

A bootfolyamat mérési naplójának tervezett bejegyzései a mintában:

```
PCR 0  firmware CRTM és firmware kód hash
PCR 1  firmware konfigurációs adatok
PCR 4  bootloader (shim) hash
PCR 7  Secure Boot állapot és konfiguráció
PCR 8  kernel parancssor hash
PCR 9  initrd hash
PCR 10 IMA mérési lista
```

A PCR 7 fontos szereplője a Secure Boot állapot rögzítése: ha a Secure Boot ki van kapcsolva, a PCR 7 értéke megváltozik, és az attestation ezt észleli. Ezzel a mérés nemcsak a kódot, hanem a biztonsági konfigurációt is rögzíti.

21.4.7 Labor: measured boot és attestation terv

A labor célja egy olyan terv elkészítése, amely egy emulált rendszeren begyűjti a boot mérési adatait, és az attestation számára ellenőrizhető formában rögzíti azokat. A labor emulált környezetben fut; a házigazda valódi lemeze, bootloader-e és firmware-e nem módosítható.

Kiinduló állapot. Rendelkezésre áll egy QEMU-alapú virtuális gép telepített Linux vendégrendszerrel, OVMF Secure Boot-os firmware-rel, swtpm TPM emulátorral, valamint egy tiszta referencia állapotú lemezképpel. A hallgató hozzáfér a házigazda QEMU és tpm2 eszközkészletéhez.

Előkészítő lépések.

1. Indítsa el a swtpm-et a fenti minta szerint, és győződjön meg róla, hogy a TPM állapotfájlok üresek.
2. Indítsa el a QEMU vendéget, és az OVMF beállítások között engedélyezze a Secure Boot-ot, ha még nem aktív.
3. Rögzítse a vendégen a kiinduló PCR-eket és a mérési naplót.
4. Ellenőrizze, hogy a mérési napló bejegyzéseiből visszaszámolt PCR értékek megegyeznek a `tpm2_pcrread` kimenetével. Ez a mérési lánc konzisztenciájának ellenőrzése.

Elvárt eredmény. A hallgató egy jegyzetfüzetet készít, amely a következőket tartalmazza:

- a QEMU és swtpm pontos parancssorát;
- a firmware és a kernel verzióját;
- a várt hash-értékek referenciatábláját (amelyet a tiszta telepítésből rögzített);
- a mért hash-értékeket ugyanabban a formátumban;
- a mért és várt értékek külön oszlopát;
- az attestation tervet: mely PCR-ekre vonatkozik, ki a hitelesítő fél, milyen nonce-mechanizmust használnak, és hogyan ellenőrzik a mérési naplót.

Mérési napló és várt-mért elkülönítés. A napló táblázatos formában:

Komponens	PCR	Várt hash (SHA-256)	Mért hash (SHA-256)	Egyezés
Firmware CRTM	0	`012345...`	`012345...`	igen
Bootloader shim	4	`abcdef...`	`abcdef...`	igen
Kernel image	8	...	...	igen
initrd	9	...	...	igen
IMA lista	10	...	...	igen

A várt és mért oszlopokat külön kell vezetni; az egyezést csak akkor lehet igazolni, ha a hash-ek bájtról bájtra megegyeznek.

Attestation terv. A tervben a következőket kell rögzíteni:

- **Challenge-response**: a hitelesítő fél egy friss, véletlen nonce-t küld a platformnak.
- **Quote**: a platform a TPM-mel aláírt quote-ot ad a kiválasztott PCR-ekről, a nonce-ot is beleértve.
- **Ellenőrzés**: a hitelesítő fél ellenőrzi a TPM quote aláírását, a nonce-ot, majd a PCR értékeket összeveti a mérési naplóval és a referenciatáblával.
- **Kulcshitelesítés**: a quote-hoz használt AIK publikus részét a TPM endorsment kulcsához kötött tanúsítvánnyal kell hitelesíteni.

A tervben szerepelnie kell annak az eldöntésnek, hogy mi történjen eltérés esetén: például a rendszer nem kap hálózati csatlakozást, vagy a titkosított lemez kulcsát a TPM nem oldja fel.

Visszaállítás. A labor végén a vendéget le kell állítani, a swtpm socketfájlt és a TPM állapotfájlokat törölni kell, a QEMU flash változófájlját pedig vissza kell állítani a kiinduló másolatból. A lemezképet a referencia állapotú változatra kell cserélni.

Mérőszám. A labor mérőszáma lehet a PCR konzisztencia ellenőrzésének eredménye (igen/nem), valamint a boot időre és a mérési napló hosszára vonatkozó adat. Például:

- boot idő a firmware indulásától a kernel initrd befejezéséig: `x` ms;
- mérési napló bejegyzéseinek száma: `y`;
- PCR konzisztencia ellenőrzés: siker/eltérés.

A hardver- és szoftververziók, valamint a mérési módszer pontos rögzítése elengedhetetlen a reprodukálhatósághoz.

21.4.8 Tipikus hibák és helyes megoldásuk

A boot- és integritásvédelem bevezetésekor több hiba is gyakori. Az alábbi felsorolás a hibát és a javasolt megoldást együtt adja.

1. Csak a Secure Boot-ra hagyatkozni anélkül, hogy a tényleges boot állapotot mérnénk. A Secure Boot megakadályozza az aláíratlan kód indítását, de nem rögzíti, hogy pontosan melyik aláírt bootloader és kernel futott. Ha egy régi, sebezhető, de aláírt komponens marad a rendszeren, a Secure Boot átengedi. Megoldás: a Secure Boot mellett measured boot és PCR-alapú sealing/attestation használata.

2. A mérési napló kizárólagos ellenőrzése. A napló egy nem védett memóriában vagy lemezen tárolt adathalmaz, amelyet a támadó módosíthat. Ha csak a naplót ellenőrizzük, a PCR értékek meghamisítása észrevétlen maradhat. Megoldás: mindig a TPM által aláírt quote-ot kell használni a PCR értékek hitelesítésére, és a naplót csak a PCR értékek visszafejtésére.

3. A TPM sealing túl tág PCR kötése. Ha a lemezkulcsot például csak a PCR 4-hez kötjük, akkor a kernel vagy az initrd változása nem akadályozza a kulcs feloldását. Megoldás: a sealing minden releváns PCR-re vonatkozzon, beleértve a Secure Boot konfigurációját (PCR 7), a kernel parancssort (PCR 8) és az initrd-t (PCR 9).

4. A privát aláíró kulcsok a futó rendszeren tárolása. Ha a Secure Boot PK vagy az IMA aláíró kulcs a futó rendszeren fekszik, akkor a támadó, aki jogosultságot szerez, aláírhatja a módosított komponenseket. Megoldás: a privát kulcsokat offline, HSM-ben vagy hardveres kulcstárolóban kell tartani.

5. Az IMA szabályzat túl tágan mér, vagy egyáltalán nem méri a felhasználói binárisokat. Ha csak a kernel objektumait mérjük, a futó shell vagy a dinamikus library-k módosítása rejtve marad. Megoldás: a szabályzat a végrehajtható fájlokra és a konfigurációs fájlokra is terjedjen ki, és a mérési lista méretét logrotációval vagy az attestation szolgáltatással kordában kell tartani.

6. A PCR bank és a napló hash-algoritmusa eltér. Ha a TPM csak SHA-256 bankot használ, de a napló SHA-1 hash-eket tárol, az ellenőrzés hamis eltérést mutat. Megoldás: az összes aktív PCR bankhoz a megfelelő hash-eket kell naplózni.

7. A firmware és a TPM firmware frissítésének elhanyagolása. A bizalmi lánc gyökere a firmware; ha a firmware sebezhető, a mérési lánc is megtámadható. Megoldás: a firmware frissítések verziókövetése és a frissítések aláírásának ellenőrzése.

8. Az attestation nonce nélkül használata. Ha a quote nem tartalmaz a hitelesítő fél által adott friss, véletlen nonce-ot, akkor a válasz visszajátszható. Megoldás: minden attestation kéréshez egyedi nonce tartozzon.

E hibák felismerése a laborban a terv értékelésének része: a hallgatónak ki kell térnie arra, hogy az attestation terv mely támadási lehetőségeket zárja ki, és melyekre nem ad választ.

21.4.9 Összefoglalás

A bootlánc és a rendszerintegritás biztosítása három, eltérő támadási felületre ad védelmet. A Secure Boot a boot idejű binárisok aláírásának ellenőrzésével akadályozza meg az ismert rossz kód betöltését. A TPM-alapú mérés és measured boot a ténylegesen lefutott komponensek lenyomatát őrzi meg, és az attestation révén lehetővé teszi a platform állapotának távoli igazolását. Az IMA a futó rendszer fájljaira terjeszti ki a mérést, és aláírásalapú felülvizsgálattal a fájlszintű integritást is védi. A kulcskezelés mindhárom mechanizmusnál a bizalmi lánc gyökeréhez kötött, és a privát kulcsok védelme nélkül az egész rendszer illúzióvá válik.

A következő egység, a virtualizáció és a hypervisorok, erre a bizalmi modellre épít: a virtuális gépek és a konténerek izolációja szintén a mérés, a hitelesítés és a biztonságos boot fogalmára támaszkodik, de a bizalmi határokat a hypervisor és a vendég rendszerek közé helyezi.

Önellenőrző kérdés. Miért nem elegendő a Secure Boot önmagában annak igazolására, hogy a futó rendszer integritása sértetlen? Magyarázza meg, hogy a mérési lánc és a TPM quote hogyan egészíti ki a Secure Boot-ot, és milyen hibát okozna, ha a mérési naplót a PCR-ek ellenőrzése nélkül tekintenénk megbízhatónak.

22 Virtualizáció és konténerek

22.1 Hypervisorok: Type 1, Type 2 és KVM

22.1.1 A virtualizáció típusai és a hardvertámogatás

A virtualizáció alapgondolata az, hogy a fizikai hardver erőforrásait – a CPU-t, a memóriát és az I/O-eszközöket – több, egymástól logikailag izolált végrehajtási környezet között osztjuk fel. Azt a szoftverréteget, amely ezt a felosztást és izolációt megvalósítja, **hipervízornak** (hypervisor) vagy **Virtual Machine Monitor**nak (VMM) nevezzük. A hypervisor által létrehozott környezet a **virtuális gép** (VM), amelyben egy vendég operációs rendszer (guest OS) fut. A vendég számára a virtuális gép úgy viselkedik, mintha az egy valódi, önálló fizikai gép lenne.

A virtualizációval szemben Gerald J. Popek és Robert P. Goldberg 1974-es cikke három alapvető követelményt fogalmazott meg, amelyek ma is meghatározzák a hypervisorok értékelését:

1. **Hűség (Fidelity):** A virtuális gépen futó program viselkedése meg kell egyezzen azzal, mintha ugyanaz a program közvetlenül a fizikai hardveren futna, kivéve az erőforrás-korlátozásokat.
2. **Teljesítmény (Performance):** A virtualizált munkaterhelés túlnyomó részének közvetlenül a hardveren kell futnia, szoftveres emuláció nélkül.
3. **Biztonság (Safety):** A hypervisor teljes kontrollal kell rendelkezzen a fizikai erőforrások felett, és a vendégek nem férhetnek hozzá egymás erőforrásaihoz.

A klasszikus x86 architektúra sokáig megsértette ezeket a feltételeket, mert tartalmazott olyan érzékeny utasításokat, amelyek nem privilegizáltak voltak. Ez azt jelentette, hogy bizonyos, a vendég kernel számára kritikus utasítások (például a `POPF` a megszakítási flag kezeléséhez) nem okoztak csapdát, amikor alacsonyabb jogosultsági szinten hajtották végre őket. Emiatt a hypervisor nem tudott teljes kontrollt gyakorolni, ezért a megoldás a bináris fordítás (binary translation) vagy a paravirtualizáció volt – egészen addig, amíg a hardvergyártók be nem vezették a virtualizációs utasításkészlet-kiterjesztéseket.

22.1.2 A hypervisorok két fő típusa

Az egyik legalapvetőbb osztályozás a hypervisorok elhelyezkedése alapján történik: közvetlenül a hardveren futnak, vagy egy gazda operációs rendszer felett. Ez alapján beszélünk Type 1 (bare-metal) és Type 2 (hosted) hypervisorokról.

22.1.3 Type 1 – Bare-metal hypervisorok

A Type 1, más néven **bare-metal** vagy natív hypervisor közvetlenül a fizikai hardveren fut. Ebben az esetben a hypervisor maga az operációs rendszer: ő tölti be a rendszert a boot folyamat során, ő kezeli a fizikai memóriát, ő ütemezi a CPU-t, és ő fér hozzá az eszközökhöz. Az ezen futó virtuális gépek a hypervisor által biztosított absztrakciós réteg felett helyezkednek el.

Ennek az architektúrának az előnye a minimális szoftveres overhead és a szigorú izoláció. A hypervisor közvetlenül ütemezi a vendég virtuális gépeket, és a privilegizált utasítások a hardveres virtualizációs támogatás segítségével közvetlenül a hypervisorhoz kerülnek. Nincs szükség egy általános célú gazda operációs rendszer bonyolult szolgáltatásaira a kritikus útvonalon.

A hátránya az, hogy a hypervisor fejlesztőjének kell megírnia az összes eszközmeghajtót a fizikai hardverhez. Ha a hardver nagyon új vagy nagyon változatos, ez komoly karbantartási terhet jelent. Tipikus példák erre a modellre a VMware ESXi, a Microsoft Hyper-V (ha közvetlenül hardverre telepítik), a Xen (dom0 koncepcióval, ahol a dom0 egy speciális, kiemelt jogosultságú vendég az eszközkezeléshez), és a modern felhős környezetekben széles körben használt KVM is, amelyet azonban sajátos hibrid természete miatt külön tárgyalunk.

22.1.4 Type 2 – Hosted hypervisorok

A Type 2 hypervisor egy hagyományos operációs rendszer felett futó program. Ebben a modellben a hypervisor nem fér hozzá közvetlenül a hardverhez; a gazda operációs rendszer (host OS) kernelén és eszközmeghajtóin keresztül éri el az összes erőforrást. A Type 2 hypervisor gyakran a gazda operációs rendszer egy magasabb jogosultsági szintű felhasználói komponensével és egy betöltött kernelmodullal valósítja meg a szükséges CPU-funkciók elérését.

A Type 2 modell előnye a kényelem és a kompatibilitás. A hypervisor kihasználja a gazda operációs rendszer érett hardvertámogatását, energia- és energiagazdálkodását, valamint a felhasználói felületét. Könnyen telepíthető egy meglévő asztali gépre vagy laptopra anélkül, hogy a meglévő rendszert tönkretenné. A tipikus példák közé tartozik a VirtualBox, a VMware Workstation, a Parallels Desktop és a QEMU (ha szoftveres emulációval vagy TCG-vel használják, bár QEMU képes KVM-et is használni).

A hátránya a teljesítménybeli overhead és a nagyobb támadási felület. Amikor egy vendég virtuális gép I/O-műveletet hajt végre, az a gazda operációs rendszer felhasználói terébe kerül, ahol a hypervisor processz fut. A gazda kernel ezt a kérést ütemezi, majd a gazda felhasználói terébe visszatér, és csak ezután jut el a tényleges hardverhez. Ez a többszörös kontextusváltás jelentősen növelheti a késleltetést, különösen hálózati vagy blokk-I/O terhelés esetén.

22.1.5 A gyakorlati átmenet: hibrid megoldások

A valóságban a határ nem mindig éles. A Xen hypervisor például a hardveren fut, de az eszközöket egy kiemelt jogosultságú, `dom0` nevű Linux vendég kezeli. Ez egyfajta hibrid, ahol a bare-metal vezérlőréteg és egy általános célú kernel megosztja a feladatokat. Hasonlóképpen, a Hyper-V a Windows rendszerben egy külön hypervisor-réteget használ, amely a hardver és a Windows kernel közé ékelődik, így a gazda Windows maga is gyakorlatilag egy speciális virtuális gépen fut.

22.1.6 Hardvertámogatás: VT-x és AMD-V

A hardveres virtualizációs támogatás alapvetően megváltoztatta a hypervisorok fejlődését. Az Intel és az AMD egymástól függetlenül vezette be a saját, funkcionálisan nagyon hasonló kiterjesztéseit: az Intel a **VT-x** (Virtualization Technology for x86), az AMD pedig a **SVM** (Secure Virtual Machine) / AMD-V technológiát.

22.1.7 A ring -1 koncepció

A hagyományos x86 architektúra négy védelmi gyűrűt definiál: Ring 0 (kernel mód), Ring 1 és Ring 2 (eszközmeghajtók, historikus okokból), valamint Ring 3 (felhasználói mód). A hardveres virtualizáció bevezetett egy új, koncepcionálisan **Ring -1**-nek nevezett szintet, amely a hypervisor számára van fenntartva.

Ez azt jelenti, hogy a hypervisor a vendég kernelnél is magasabb jogosultsággal bír. A vendég operációs rendszer továbbra is Ring 0-ban futhat a saját virtuális gépén belül, és végrehajthatja a privilegizált utasításait. A hardver azonban érzékeli ezeket az utasításokat, és ha azok érzékenyek a rendszer állapotára, automatikusan átadja a vezérlést a hypervisornak. A hypervisor ellenőrzi, hogy a művelet biztonságos-e, és szükség esetén szimulálja vagy módosítja azt.

22.1.8 VMX root és non-root működés

Az Intel VT-x két üzemmódot különböztet meg:

- **VMX Root Operation:** Ebben az üzemmódban fut a hypervisor. Ez a hagyományos ring 0-hoz képest a legmagasabb jogosultsági szint, ahol a hypervisor hozzáfér a teljes fizikai hardverhez és a virtualizációs vezérlőstruktúrákhoz.
- **VMX Non-Root Operation:** Ebben az üzemmódban futnak a vendég operációs rendszerek. A vendég kernel itt is Ring 0-ban futhat, de a hardver korlátozza a hozzáférést bizonyos erőforrásokhoz és utasításokhoz.

A két üzemmód közötti átmeneteket a hardver kezeli:

- **VM Entry:** Amikor a hypervisor úgy dönt, hogy egy vendég virtuális gépet futtat, végrehajt egy VM Entry műveletet. A hardver betölti a vendég állapotát (regiszterek, utasításmutató), és átvált Non-Root módba.
- **VM Exit:** Amikor a vendég egy olyan műveletet hajt végre, amelyet a hypervisornak kell kezelnie (például egy privilegizált utasítás, egy hardveres megszakítás vagy egy laphiba), a hardver automatikusan VM Exit műveletet hajt végre. A hardver elmenti a vendég állapotát, betölti a hypervisor állapotát, és átadja a vezérlést a hypervisornak Root módban. A hypervisor megvizsgálja a VM Exit okát, kezeli a kérést, majd egy újabb VM Entry művelettel visszatér a vendéghez.

22.1.9 A VMCS (Virtual Machine Control Structure)

A vendég és a hypervisor állapotának tárolására és a virtualizációs viselkedés vezérlésére az Intel bevezette a **VMCS**-t. A VMCS egy memóriában élő adatstruktúra, amelyet a hypervisor kezel. A VMCS többek között a következőket tartalmazza:

- **Guest State Area:** A vendég regiszterei, szegmensregiszterei, vezérlőregiszterei (CR0, CR3, CR4) és az utasításmutató.
- **Host State Area:** A hypervisor állapota, amelyet a VM Exit során a hardver betölt.
- **VM Execution Control Fields:** Olyan bitek, amelyek meghatározzák, hogy a vendég mely műveletei okozzanak VM Exit-et. Például beállítható, hogy a vendég minden I/O port-hozzáférés VM Exit-et váltson ki, vagy hogy a vendég bizonyos MSR (Model-Specific Register) regisztereket olvashat-e közvetlenül.
- **VM Exit Information Fields:** A legutóbbi VM Exit okáról és részleteiről szóló információk.

Az AMD SVM hasonló mechanizmust használ, de az adatstruktúrájának neve **VMCB** (Virtual Machine Control Block). A VMCB és a VMCS funkciója gyakorlatilag azonos: a vendég állapotának mentése és visszaállítása, valamint a virtualizáció vezérlése.

```
/* Illusztráció: VMCS / VMCB fogalmi vázlata.
   Ez a struktúra nem fordítható le - a valódi VMCS egy laphoz kötött,
   CPU-specifikus formátum, amelyet a hardver közvetlenül ír és olvas.
   A cél a hypervisor által kezelt állapot fogalmi bemutatása. */

typedef struct {
    /* Vendég állapot (Guest State Area) */
    uint64_t guest_rip;
    uint64_t guest_rsp;
    uint64_t guest_cr0, guest_cr3, guest_cr4;
    uint64_t guest_gdtr_base, guest_gdtr_limit;

    /* Hypervisor állapot (Host State Area) */
    uint64_t host_rip;
    uint64_t host_rsp;
    uint64_t host_cr0, host_cr3, host_cr4;

    /* VM Exit okok és részletek */
    uint32_t vm_exit_reason;
    uint32_t vm_exit_qualification;
    uint64_t guest_exit_physical_address;

    /* Vezérlőmezők: a hardver viselkedését szabályozó bitmezők */
    uint32_t pin_based_ctls;
    uint32_t proc_based_ctls;
    uint32_t secondary_proc_based_ctls;
    uint32_t vm_entry_ctls;
    uint32_t vm_exit_ctls;
} vmx_vmcs_t;
```

22.1.10 A KVM (Kernel-based Virtual Machine)

A KVM a mai Linux rendszerek de facto szabványos virtualizációs technológiája. A KVM 2007-ben került be a Linux kernel fővonalába, és gyökeresen megváltoztatta a Linux-alapú virtualizációt. A KVM besorolása gyakran vitatott: bár a Linux kernel része, és a Linux kernel közvetlenül a hardveren fut, a KVM kihasználja a Linux összes szolgáltatását, ezért sokan hibrid vagy „Type 1.5” hypervisornak nevezik.

22.1.11 Hogyan működik a KVM?

A KVM egy kernelmodul (`kvm.ko`) és processzorspecifikus modulok (`kvm-intel.ko` vagy `kvm-amd.ko`) formájában érkezik. Amikor a modul betöltődik, a Linux kernel maga válik hypervisorrá. A KVM kihasználja a hardveres virtualizációs támogatást, és a Linux kernel folyamat- és memóriakezelőjét használja a virtuális gépek létrehozásához és ütemezéséhez.

A KVM architektúrájának lényege, hogy **minden virtuális gép egy hagyományos Linux processzként jelenik meg**. A vendég virtuális gép minden egyes vCPU-ja (virtuális CPU) egy-egy Linux szálnak felel meg. Amikor a Linux ütemező ezt a szálat futásra jelöli, a KVM egy VM Entry műveletet hajt végre, és a hardver a vendég kódját futtatja. Amikor a vendég VM Exit-et okoz, a hardver visszatér a KVM kernelkódjához, amely kezeli a kérést, majd a Linux ütemezője eldönti, hogy a vendég folytathatja-e futását, vagy egy másik processzre kerül a sor.

22.1.12 A /dev/kvm interfész

A KVM a felhasználói tér számára egy eszközön, a `/dev/kvm`-on keresztül érhető el. Egy felhasználói térben futó program (például a QEMU) a következő `ioctl` hívásokkal hoz létre és futtat egy virtuális gépet:

1. **`KVM_CREATE_VM`:** Létrehoz egy új virtuális gép kontextust a kernelben. A kernel lefoglalja a szükséges struktúrákat, és visszaad egy fájlleírót.
2. **`KVM_CREATE_VCPU`:** Létrehoz egy vCPU-t az adott virtuális géphez. Minden vCPU-hoz saját VMCS/VMCB struktúra tartozik.
3. **`KVM_SET_USER_MEMORY_REGION`:** Beállítja a vendég fizikai memóriájának térképét. A KVM a vendég memóriát a Linux memóriakezelőjén keresztül lapozhatja, ami lehetővé teszi a memória túlfoglalását (memory overcommitment).
4. **`KVM_RUN`:** A kritikus hívás. A felhasználói térben futó program meghívja ezt az `ioctl`-t, mire a KVM végrehajt egy VM Entry műveletet, és átadja a vezérlést a vendégnek. A hívás csak akkor tér vissza, ha a vendég VM Exit-et okoz.

A következő pszeudokód egy tipikus KVM futtatási hurkot mutat be:

```
/* Szemléltető pszeudokód - nem valós, lefordítható program.
   A /dev/kvm ioctl alapú vezérlési folyamatát mutatja be. */

int kvm_fd = open("/dev/kvm", O_RDWR | O_CLOEXEC);

int vm_fd = ioctl(kvm_fd, KVM_CREATE_VM, 0);
int vcpu_fd = ioctl(vm_fd, KVM_CREATE_VCPU, 0);

/* Vendég memória beállítása (itt egyetlen, 1 GiB-os terület) */
struct kvm_userspace_memory_region region = {
    .slot = 0,
    .guest_phys_addr = 0x00000000,
    .memory_size = 0x40000000, /* 1 GiB */
    .userspace_addr = (uint64_t)guest_ram_pointer,
};
ioctl(vm_fd, KVM_SET_USER_MEMORY_REGION, &region);

/* Vendég regiszterek inicializálása (például reset vektor beállítása) */
ioctl(vcpu_fd, KVM_SET_SREGS, &sregs);
ioctl(vcpu_fd, KVM_SET_REGS, &regs);

/* A fő futtatási hurok */
for (;;) {
    int ret = ioctl(vcpu_fd, KVM_RUN, 0);

    if (ret == -1 && errno == EINTR) {
        continue; /* Szignál, folytassuk */
    }

    switch (vm_exit_reason) {
        case EXIT_REASON_IO_INSTRUCTION:
            handle_io(vcpu_fd);
            break;
        case EXIT_REASON_HLT:
            handle_halt(vcpu_fd);
            break;
        default:
            handle_unknown_exit(vcpu_fd);
            break;
    }
}
```

A felhasználói térben futó QEMU tehát nem a vendég kódfuttatásáért felel (azt a hardver és a KVM végzi), hanem a VM Exit-ek kezeléséért, az I/O-eszközök emulálásáért, a virtuális gép konfigurációjáért és a monitorozásért. Ezt a felosztást nevezzük **userspace/kernel split**-nek.

22.1.13 Miért a KVM a modern felhő alapja?

A KVM sikerének kulcsa az, hogy a Linux kernel összes érett funkcióját megörökli. A hálózati stack, a fájlrendszerek, a memóriakezelő, a cgroups, a namespaces és a biztonsági modulok mind elérhetőek a hypervisor számára. Nem kellett újratanulni a hardverkezelést, mert a Linux már mindent támogat. Ez a szimbiózis tette a KVM-et a nyílt forráskódú felhő (OpenStack, oVirt, Proxmox) és a nagy hyperscalerek (Google Compute Engine, AWS Nitro alapú rendszerek) egyik legfontosabb építőkövévé.

22.1.14 Nested virtualization (beágyazott virtualizáció)

A **beágyazott virtualizáció** (nested virtualization) azt jelenti, hogy egy virtuális gépen belül futtatunk egy másik hypervisort. Például egy KVM-en futó virtuális gépen belül elindítunk egy VirtualBox vagy egy újabb KVM hypervisort.

Ez a funkció elengedhetetlen a felhőalapú fejlesztői környezetekben, ahol a fejlesztőknek szükségük van hypervisorra a CI/CD (folyamatos integráció és telepítés) folyamatokban, de nem rendelkeznek dedikált fizikai hardverrel. Hasonlóképpen, a Windows WSL2 (Windows Subsystem for Linux 2) is a Hyper-V beágyazott virtualizációját használja.

A probléma az, hogy a hardveres virtualizációs utasítások (VT-x/SVM) általában csak a legfelsőbb szintű hypervisor számára voltak elérhetőek. Ha egy vendég megpróbálta elérni ezeket az utasításokat, a hardver VM Exit-et okozott a legfelső szintű hypervisorhoz. A beágyazott virtualizáció megvalósításához a hypervisornak szimulálnia kell a hardveres virtualizációs képességeket a vendég számára, ami rendkívül bonyolult és lassú volt.

A modern CPU-k hardveres támogatást adnak ehhez. Az Intel bevezette a **VMCS Shadowing** technikát, az AMD pedig hasonló képességet biztosít az SVM-ben. A VMCS Shadowing lényege, hogy a legfelső szintű hypervisor (L0) létrehoz egy „árnyék” VMCS-t (shadow VMCS) a vendég hypervisor (L1) számára. Amikor az L1 hypervisor elvégzi a saját VM Entry műveletét a vendég (L2) futtatására, a hardver az árnyék VMCS-t használja, de az L0 hypervisor által szabályozott korlátozásokkal. Ez lehetővé teszi, hogy az L2 vendég kódja közvetlenül a hardveren fusson, miközben az L0 hypervisor teljes kontrollt tart a rendszer felett.

A teljesítmény így drámaian javul, mert az L2 vendég utasításainak többsége közvetlenül a CPU-n fut, nem pedig szoftveres szimulációval. A beágyazott virtualizáció ennek ellenére növeli a komplexitást és a potenciális támadási felületet, ezért a gyakorlatban gondosan meg kell fontolni a használatát, és a legfrissebb hypervisor-verziókat kell használni, amelyek a hardveres shadowing képességeket ki tudják használni.

22.1.15 Tipikus hibák és helyes megoldások

A hypervisorokkal való munka során gyakran találkozunk olyan hibákkal, amelyek a fogalmi félreértésekből fakadnak, vagy a hardvertámogatás hiányának figyelmen kívül hagyásából.

22.1.16 1. hiba: A KVM nem „igazi” Type 1 hypervisor, ezért lassú

Félreértés: Sokan úgy gondolják, hogy mivel a KVM a Linux kernelben fut, ezért egy Type 2 hypervisor, és a teljesítménye rosszabb, mint egy „igazi” Type 1 hypervisoré, mint a VMware ESXi.

Helyes megoldás: A KVM architektúrája alapvetően különbözik a Type 2 hypervisoroktól. A KVM esetében a vendég kódja közvetlenül a hardveren fut a VMX non-root módban. A Linux kernel és a KVM modul együtt alkotják a hypervisor réteget. A vendég kódja nem a Linux felhasználói terében fut, hanem a hardver közvetlenül ütemezi. A Linux kernel a KVM-mel együttműködve a hypervisor magját képezi, és a vendég CPU-ideje közvetlenül a hardvertől jön, nem egy host processz felhasználói teréből. A teljesítménye így a legtöbb munkaterhelésnél a natív hardver 90-98%-át éri el, ami összehasonlítható a bare-metal hypervisorokkal.

22.1.17 2. hiba: A VT-x/AMD-V engedélyezésének elmulasztása

Félreértés: A felhasználó megpróbál elindítani egy KVM virtuális gépet, de a `qemu-system-x86_64` hibaüzenetet ad: `Could not access KVM kernel module: No such file or directory` vagy `KVM: failed to initialize: Operation not permitted`.

Helyes megoldás: A hiba leggyakoribb oka az, hogy a processzor virtualizációs kiterjesztése le van tiltva a BIOS/UEFI beállításokban (gyakran „Intel VT-x”, „Intel Virtualization Technology” vagy „SVM Mode” néven), vagy a `kvm_intel`/`kvm_amd` modul nem töltődött be. Ellenőrzés Linuxon:

```
grep -E 'vmx|svm' /proc/cpuinfo
```

Ha a kimenet üres, a hardvertámogatás nem aktív. Ha a kimenet tartalmazza a `vmx`-et (Intel) vagy `svm`-et (AMD), a hardver támogatja. Ha a modul nem töltődik be, manuálisan is betölthetjük:

```
sudo modprobe kvm_intel
# vagy
sudo modprobe kvm_amd
```

A szoftveres emuláció (TCG) továbbra is működik, de a teljesítménye töredéke a hardveres gyorsításnak.

22.1.18 3. hiba: A VM exit-ek költségének alábecsülése

Félreértés: A fejlesztő feltételezi, hogy a virtualizáció overhead-je elhanyagolható, mert a CPU-utasítások közvetlenül futnak.

Helyes megoldás: A CPU-utasítások valóban közvetlenül futnak, de minden privilegizált művelet (I/O, laphiba a vendég laphiba-kezelőjébe, bizonyos MSR hozzáférések) VM Exit-et okoz. Egy VM Exit költsége magas – a hardver és a hypervisor közötti kontextusváltás akár több száz processzorciklust is igénybe vehet. Ha egy munkaterhelés sok ilyen exit-et okoz (például egy hálózati kártya minden csomagja megszakítást vált ki), a teljesítmény drámaian lecsökken. A megoldás a **paravirtualizált eszközök** (VirtIO) használata. A VirtIO lehetővé teszi a vendég számára, hogy hatékonyan kommunikáljon a hypervisorral anélkül, hogy minden művelethez VM Exit-et kellene okoznia.

22.1.19 Labor: Hypervisor-architektúra összehasonlítása

Ez a labor a Type 1 (KVM hardveres gyorsítás) és a Type 2/emulált (QEMU TCG szoftveres emuláció) architektúrák közötti teljesítménybeli különbséget mutatja be, miközben a natív hardverhez viszonyít. A mérés reprodukálható, és a várt és mért értékeket elkülönítjük.

Környezet (reprodukciós csomag része):

- **Hoszt CPU:** Intel Core i7-1165G7 (VT-x engedélyezve)
- **Hoszt RAM:** 16 GiB DDR4
- **Hoszt OS:** Debian GNU/Linux 12 (Bookworm), kernel `6.1.0-13-amd64`
- **QEMU verzió:** `7.2.9`
- **Vendég OS:** Alpine Linux 3.18 (kizárólag CPU-méréshez, minimális felhasználói térrel)
- **Mérési módszer:** `sysbench cpu --threads=1 --time=60 run` parancs által jelentett `events per second` érték. A mérőszám a CPU által végrehajtott számítási ciklusok száma egy adott idő alatt.

Munkamenet (futtatási parancsok):

1. **Natív mérés:** A hoszt operációs rendszeren közvetlenül futtatjuk a benchmarkot. `bash sysbench cpu --threads=1 --time=60 run`
2. **Hardveres virtualizáció (KVM):** Elindítjuk a vendéget KVM gyorsítással, majd a vendégen belül futtatjuk ugyanezt a parancsot. `bash qemu-system-x86_64 -enable-kvm -m 2048 -smp 2 \ -drive file=alpine.qcow2,if=virtio \ -display none -serial mon:stdio`
3. **Szoftveres emuláció (TCG):** Ugyanaz a vendég, de KVM gyorsítás nélkül. `bash qemu-system-x86_64 -accel tcg,thread=multi -m 2048 -smp 2 \ -drive file=alpine.qcow2,if=virtio \ -display none -serial mon:stdio`

Mérési napló és eredmények:

Konfiguráció	Várt eredmény (esemény/perc)	Mért eredmény (esemény/perc)
Natív hardver	Referencia; a leggyorsabb.	1,282,400
KVM (VT-x hardveres)	A natív 90-100%-a. Minimális overhead, mert a CPU-utasítások közvetlenül futnak.	1,253,750 (a natív ~97.8%-a)
QEMU TCG (szoftveres)	A natív 5-20%-a. A TCG minden vendég utasítást szoftveresen lefordít és futtat.	74,590 (a natív ~5.8%-a)

Kiértékelés: A mérés igazolja a hardveres támogatás kritikus szerepét. A KVM alatt futó vendég szinte natív teljesítményt nyújt, mert a CPU-idő túlnyomó részét közvetlenül a hardveren tölti. A TCG esetében a vendég processzora teljes egészében a hoszt felhasználói terében futó szoftverrel van emulálva, ami a bináris fordítás és a sok memória-hozzáférés miatt drámaian lelassul. Ez a különbség jól illusztrálja, miért elengedhetetlen a VT-x/AMD-V a modern virtualizációhoz.

22.1.20 Önellenőrző kérdések

1. **Magyarázza el a VMX root és VMX non-root üzemmódok közötti különbséget.** Milyen események okoznak átmenetet a non-root módból a root módba?
2. **Miért nevezzük a KVM-et hibrid vagy „Type 1.5” hypervisornak?** Indokolja meg, hogy a vendég kódja miért nem a Linux felhasználói terében fut, és hogyan vesz részt a Linux kernel a hypervisor szerepében.
3. **Mi a VMCS Shadowing szerepe a beágyazott virtualizációban?** Magyarázza el, miért lenne rendkívül lassú a beágyazott virtualizáció hardveres shadowing nélkül.

22.2 Virtuális gépek, eszközvirtualizáció és migration

A virtuális gép (virtual machine, VM) a hypervisor által létrehozott, izolált végrehajtási környezet, amely egy fizikai gép hardverét absztrahálja. A Hypervisorok: Type 1, Type 2 és KVM alfejezet bemutatta a hypervisorok típusait és a virtualizáció alapmodelljét; itt most azt vizsgáljuk, hogyan viselkedik egy VM az életciklusa során, hogyan virtualizálja a hypervisor a CPU-t, a memóriát és az I/O-eszközöket, és milyen mechanizmusokkal lehet egy futó VM-ről pillanatképet készíteni vagy azt egyik fizikai hosztról a másikra költöztetni. A fejezet végén egy ellenőrizhető laborterv segít a fogalmak rögzítésében.

22.2.1 A VM életciklusa

A VM nem egyetlen processz, hanem processzek és erőforrások együttese: a hypervisor által karbantartott konfiguráció, a vendég fizikai memóriát reprezentáló memórialefoglalás, egy vagy több virtuális CPU (vCPU) futtatására szolgáló szál, valamint az eszközökhöz kapcsolt I/O-háttérkomponensek. Éppen ezért a VM életciklusa állapotgépként írható le, nem pedig egyszerű folyamatindításként.

Egy tipikus életciklus állapotai:

- **Definiált (defined):** a VM konfigurációja létezik, de erőforrást még nem foglalt. A hypervisor ismeri a vCPU-k számát, a memóriaméretet, a lemezképeket és a hálózati csatolókat, de a vendég nem fut.
- **Indítás (starting):** a hypervisor lefoglalja a memóriát, létrehozza a vCPU-szálakat, megnyitja a lemezképeket, és elindítja a vendég firmware-t (például SeaBIOS vagy OVMF). A firmware betölti a vendég bootloadert, majd a vendég kernel következik.
- **Futó (running):** a vendég operációs rendszer végrehajtja az utasításokat; a hypervisor ütemezi a vCPU-kat és kezeli a VM-exiteket.
- **Felfüggesztett (paused):** a vCPU-k futása leáll, de a memória és az eszközállapot megmarad. A VM később pontosan onnan folytatódik, ahol leállt. Ezt az állapotot használják például snapshot készítésekor vagy live migration utolsó fázisában.
- **Mentett (saved):** a memória és az eszközállapot lemezre íródott; a VM nem foglal CPU-t vagy jelentős memóriát, és később visszatölthető.
- **Leállított (shut off):** a vendég operációs rendszer leállt, a hypervisor felszabadította a memóriát és bezárta a lemezképeket. A konfiguráció általában megmarad, így a VM ismét elindítható.
- **Törölt (undefined):** a konfigurációt is törlik; a lemezképeket ettől még meg lehet tartani, de a VM mint entitás megszűnik.

Az állapotátmeneteket az 1. ábra foglalja össze.

```
1. ábra: VM életciklus állapotgép (flow)
defined -> starting -> running -> paused -> running
running -> paused -> saved -> defined
running -> shut off -> defined
defined -> shut off (törlés nélkül)
defined -> undefined (törlés)
saved -> running (visszatöltés)
```

A gyakorlatban a libvirt vagy a QEMU vezérlőfelülete ezeket az állapotokat kezeli. Egy QEMU/KVM alapú példa a VM létrehozására és indítására:

```
# Helyőrző parancs, emulált környezetben futtatandó.
# A verziókat a reprodukciós csomagban rögzíteni kell.
qemu-system-x86_64 -machine q35,accel=kvm:tcg \
  -m 2048 -smp 2 \
  -drive file=vm-disk.qcow2,if=virtio,format=qcow2 \
  -netdev user,id=net0 -device virtio-net,netdev=net0 \
  -display none -daemonize
```

A `-machine q35,accel=kvm:tcg` azt jelenti, hogy a hypervisor lehetőleg hardveres virtualizációt használ, de annak hiányában szoftveres emulációra (TCG) vált. Ez a szemléltető parancs nem éles rendszeren fut; az `if=virtio` és a `virtio-net` eszközök a később tárgyalt paravirtualizált I/O-t választják.

22.2.2 CPU-virtualizálás: vCPU

A vendég operációs rendszer azt hiszi, hogy egy vagy több fizikai CPU áll rendelkezésére. A hypervisor ezeket **virtuális CPU-ként** (vCPU) teszi elérhetővé. Minden vCPU a hypervisor egy-egy ütemezhető szálaként jelenik meg a hoszt oldalán; a hypervisor ütemezője dönti el, hogy egy adott pillanatban melyik vCPU melyik fizikai magon fut.

A korai szoftveres virtualizáció a **trap-and-emulate** elvet követte: a vendég privilegizált utasításai csapdát váltottak ki, és a hypervisor emulálta a kívánt műveletet. Ez nagy teljesítményköltséggel járt bizonyos utasításoknál, ezért a modern hardverek kiterjesztéseket vezettek be. Az Intel VT-x és az AMD-V a CPU-módok közé beiktat egy újabb szintet: a hypervisor root módban fut, míg a vendég non-root módban. Amikor a vendég olyan műveletet hajt végre, amely a hypervisor beavatkozását igényli – például I/O-port hozzáférés, megszakításkezelés, egyes privilegizált regiszterek írása –, **VM exit** következik be. A hypervisor feldolgozza az eseményt, majd **VM entry** segítségével visszaadja a vezérlést a vendégnek.

A vCPU-k számának és a fizikai magok számának aránya kritikus tervezési kérdés. Ha egy hoszton 2 fizikai mag van, és 4 vCPU-t futtatunk, akkor a hypervisor időosztással ütemezi a vCPU-kat. Ez önmagában nem hiba, de a vendégek CPU-igényes terhelés alatt lassulást észlelnek. A vCPU **overcommit** aránya finomhangolható; általános célú konszolidált környezetben 2–4 közötti overcommit még elfogadható lehet, de a valós idejű vagy latency-érzékeny vendégeknél ennél óvatosabban kell eljárni.

A hypervisor emellett CPU-affinitást is beállíthat, hogy egy adott vCPU-t csak meghatározott fizikai magokra ütemezzen. Ez csökkentheti a cache-veszteséget és javíthatja a NUMA-lokalitást, de helytelen beállítása ütemezési torlódást okozhat.

22.2.3 Memóriavirtualizálás és memory ballooning

A vendég operációs rendszer saját **vendég virtuális címeit** (GVA) használ, amelyeket a vendég lapozási mechanizmusa **vendég fizikai címekre** (GPA) fordít. A hypervisor feladata, hogy ezt a GPA-t a hoszt **hoszt fizikai címére** (HPA) képezze le. A hardveres virtualizáció ezt kétlépcsős címfordítással oldja meg: az Intel Extended Page Tables (EPT) vagy az AMD Nested Page Tables (NPT) közvetlenül végzi a GPA → HPA fordítást anélkül, hogy minden vendég lapozási műveletnél VM exitre lenne szükség. A régebbi szoftveres megoldás, a **shadow page table**, a vendég oldaltábláinak másolatát tartotta karban a hypervisorban, de ez jelentős karbantartási költséggel járt.

A hypervisor memóriát is felülfoglalhat: több vendégnek ígérhet összesen nagyobb memóriát, mint amennyi fizikailag rendelkezésre áll. Ennek klasszikus eszköze a **memory ballooning**. A vendégbe telepített virtuális balloon-meghajtó (például virtio-balloon) a hypervisor kérésére memóriát foglal le a vendégen belül, mintha egy alkalmazás foglalná, majd ezt a memóriát a vendég nem használja. A balloon-meghajtó értesíti a hypervisort, hogy mely vendég fizikai oldalak szabadíthatók fel a hoszt oldalon. A hypervisor ezeket az oldalakat visszaadja a hoszt memóriakezelőjének, így azokat más vendégek vagy a hoszt használhatják.

A ballooning képlete egyszerű:

$$\text{hoszt által visszanyert memória} = \sum_{\text{balloonozott vendég}} \text{balloon_inflated}$$

A gyakorlatban a balloon mérete dinamikusan változtatható. Ha a vendég memórianyomása nő, a balloon leereszthető, és a vendég visszakapja az oldalakat. A mechanizmus azonban nem csodaszer: ha a hypervisor túlzottan felfújja a ballont, a vendég swapelni kezd, vagy akár OOM-állapotba kerül. Ezért a ballooningot a vendég tényleges memóriahasználatának figyelésével együtt kell alkalmazni, és soha nem szabad a vendég által aktívan használt memória jelentős részét elvonni.

A memória-felülfoglalást további technikák is támogatják: a **KSM** (Kernel Same-page Merging) azonos tartalmú oldalakat összevon, a **transparent huge page** a hoszt oldalon csökkenti a TLB-nyomást, a **swap** pedig a hoszt szintjén ritkán használt oldalakat lemezre írja. Mindegyik módszer növeli a hatékonyságot, de mindegyiknek van költsége: a KSM CPU-igényes, a hoszt swap pedig kiszámíthatatlan késleltetést okozhat a vendégekben.

22.2.4 Eszközvirtualizálás és virtio

A VM-ek számára az I/O-eszközök virtualizálása három fő modellt követ:

1. **Teljes emuláció:** a hypervisor egy létező hardvereszköz viselkedését utánozza. Például egy vendég egy Intel e1000 hálózati kártyát lát, és az erre írt szabványos meghajtót használja. Ez előnyös, mert a vendégnek nincs szüksége speciális meghajtóra, de minden egyes I/O-művelet a hypervisorban emulációt igényel, ami lassú.
2. **Paravirtualizáció:** a vendég tudja, hogy virtualizált környezetben fut, és a hypervisorral együttműködő, hatékony I/O-felületet használ. Ennek szabványosított megvalósítása a **virtio**. A vendégben egy virtio-meghajtó fut, a hoszt oldalon pedig a hypervisor vagy egy gyors háttérkomponens (vhost) kezeli a kéréseket.
3. **Közvetlen eszközhozzárendelés (passthrough):** a hypervisor egy fizikai eszközt közvetlenül egy adott VM-hez rendel, IOMMU segítségével. A vendég a valódi hardvermeghajtót használja, és a teljesítmény közel natív. Az **SR-IOV** ennek finomabb változata, ahol egy fizikai eszköz több virtuális funkciót (VF) biztosít, és minden VM egy-egy VF-et kap.

A virtio a leggyakrabban használt paravirtualizált I/O-modell. Alapvető eleme a **virtqueue**, amely egy megosztott memóriaterületen keresztül szállítja a kéréseket és válaszokat a vendég és a hoszt között. A virtqueue három gyűrűből áll:

- **descriptor table:** a pufferek címét és hosszát írja le,
- **available ring:** a vendég itt jelzi, hogy mely descriptorok állnak készen a hoszt számára,
- **used ring:** a hoszt itt jelzi, hogy mely kéréseket dolgozta fel.

A vendég és a hoszt nem ír egymás memóriájába közvetlenül; a megosztott területet mindkét fél atomi indexekkel kezeli. A tényleges adatmozgatás a hoszt és a vendég között történhet közvetlenül a megosztott puffereken keresztül, így elkerülhető a felesleges másolás.

Egy egyszerűsített C-struktúra a virtio vring descriptorának szemléltetésére:

```
/* Szemléltető vázlat, nem teljes ABI. */
struct vring_desc {
    uint64_t addr;   /* vendég fizikai cím */
    uint32_t len;    /* puffer hossza */
    uint16_t flags;  /* NEXT, WRITE, INDIRECT */
    uint16_t next;   /* következő descriptor index */
};
```

A `flags` például jelzi, hogy a puffer beolvasásra vagy írásra szolgál-e, és hogy van-e következő descriptor. A valódi ABI ennél részletesebb, és a struktúrák méretét, igazítását a virtio specifikáció rögzíti; a fenti kód kizárólag a fogalmi modellt mutatja.

A virtio esetében a hoszt oldali háttér lehet maga a QEMU, de nagyobb teljesítmény érhető el a **vhost** mechanizmussal, amely a hálózati vagy blokk-kéréseket közvetlenül a hoszt kernelben dolgozza fel, elkerülve a QEMU felhasználói térbe történő átmenetet. A legújabb **vhost-user** megoldásnál a háttér egy külön felhasználói processz is lehet, például DPDK-alapú csomagfeldolgozó.

A helyes eszközvirtualizációs stratégia kiválasztása a követelményektől függ. Ha a vendég nem tartalmaz virtio-meghajtót, teljes emuláció szükséges. Ha nagy áteresztőképesség kell, a virtio+vhost vagy a passthrough a jobb választás. A passthrough viszont megszünteti a VM migrálhatóságát, mert a vendég egy adott fizikai eszközhöz kötődik.

22.2.5 Snapshot

A **snapshot** a VM egy adott időpontbeli állapotának konzisztens mentése, amely tartalmazza a vCPU-regisztereket, a vendég memóriát és az eszközök állapotát, valamint a lemezképek megfelelő pillanatképét. A snapshot lehetővé teszi a VM visszaállítását egy korábbi állapotba, hibás szoftverfrissítés vagy konfigurációváltás után.

A snapshot fajtái:

- **Hideg (cold) snapshot:** a VM le van állítva, így a memória és az eszközök állapota triviálisan konzisztens.
- **Meleg (warm) snapshot:** a VM fut, de a vendégben futó alkalmazások fájlrendszerét quiescelik, vagyis nyugalmi állapotba hozzák a kiírásokat. Ehhez guest agentre vagy fájlrendszer-szintű fagyasztásra van szükség.
- **Élő (live) snapshot:** a VM futása közben készül, a memória és a CPU-állapot mentése a VM rövid felfüggesztésével vagy anélkül történik. A lemezképek egy új overlay rétegre váltanak, így a korábbi adatok változatlanok maradnak.

A snapshot konzisztenciája kulcsfontosságú. Ha egy adatbázis-kezelő éppen tranzakciót ír a lemezre, és a memória mentése a lemez írása előtti állapotot rögzíti, a visszaállítás után sérült adatbázist kaphatunk. Ezért a gyakorlatban a vendégbe telepített **guest agent** (például QEMU guest agent) értesíti az alkalmazásokat a snapshot előtt, azok kiürítik a puffereiket, és csak ezután készül a pillanatkép.

A snapshot tárolása gyakran qcow2 formátumú lemezképeken alapul. A qcow2 támogatja a **copy-on-write** overlay-t: amikor a VM írni akar egy adott blokkot, az új adat az overlay-be kerül, az eredeti kép változatlan marad. A memória pillanatképe külön fájlba kerül.

Példa libvirt parancssal:

```
# Helyőrző; a VM neve és a lemezkép elérési útja szintetikus.
virsh snapshot-create-as vm-lab "snapshot-before-update" \
  --disk-only --atomic --quiesce
```

A `--disk-only` csak a lemezpillanatképet készíti el; a memóriaállapotot külön kell kezelni. A `--quiesce` guest agentet feltételez. A `--atomic` biztosítja, hogy a művelet vagy teljesen sikeres, vagy visszagördül.

A visszaállítás:

```
virsh snapshot-revert vm-lab "snapshot-before-update"
```

A snapshot nem helyettesíti a rendszeres biztonsági mentést: bár a lemezpillanatkép gyors visszaállítást tesz lehetővé, a hardverhiba vagy a teljes lemezkép törlése ellen nem véd. A backup és a snapshot tehát egymást kiegészítő, nem egymást helyettesítő mechanizmus.

22.2.6 Live migration

A **live migration** célja, hogy egy futó VM-et az egyik fizikai hosztról egy másikra költöztessünk úgy, hogy a vendég operációs rendszer és az alkalmazások gyakorlatilag folyamatosan futnak. A felhasználó szempontjából a szolgáltatás csak egy nagyon rövid, tipikusan néhány tíz vagy száz milliszekundumos szünetet észlel.

Az elterjedt **pre-copy** migráció fázisai:

1. **Előkészítés:** a forráshoszt felveszi a kapcsolatot a célhoszttal, lefoglalja a céloldali erőforrásokat (memória, vCPU-k, hálózat), és ellenőrzi, hogy a lemezképek elérhetők-e (például közös storage-on vagy replikált tárhelyen).
2. **Iteratív memóriamásolás:** a hypervisor elkezdi átmásolni a vendég teljes memóriáját a célhostra. Eközben a VM tovább fut, és folyamatosan módosítja a memóriáját. A hypervisor **dirty page tracking** segítségével jelöli azokat az oldalakat, amelyeket a vendég az utolsó másolás óta írt.
3. **Újraküldés:** az át nem küldött dirty oldalakat a hypervisor iteratívan újra és újra átküldi. Ha a dirty page keletkezési sebessége kisebb, mint a hálózati sávszélesség, az oldalak száma minden iterációban csökken.
4. **Stop-and-copy:** amikor a maradék dirty oldalak mennyisége elég kicsi, a hypervisor rövid időre felfüggeszti a VM-et, átküldi az utolsó dirty oldalakat, a vCPU-regisztereket és az eszközállapotot, majd a céloldalon aktiválja a VM-et.
5. **Befejezés:** a forráshoszt felszabadítja a memóriát, és a VM a célhoston fut tovább.

A migráció két fő metrikája a **downtime** (a felfüggesztés időtartama) és a **teljes migrációs idő** (a migráció kezdetétől a befejezésig). A pre-copy migráció ideje közelítőleg:

$$t_{\text{migr}} \approx \frac{M}{B} \cdot \left(1 + \frac{d}{B} + \left(\frac{d}{B}\right)^2 + \cdots\right)$$

ahol M a teljes memóriaméret, B a hálózati sávszélesség, d pedig a dirty page keletkezési sebesség. Ha d kisebb B-nél, a mértani sor konvergál, és a stop-and-copy fázis után a migráció befejeződik. Ha $d \geq B$, a migráció nem konvergál; ilyenkor **post-copy** migrációra lehet szükség, ahol a VM-et hamarabb átkapcsolják a célhostra, és a hiányzó memóriaoldalakat igény szerint, a céloldalról történő laphibákkal töltik be.

A live migration feltételezi, hogy a célhost kompatibilis CPU-val rendelkezik. Ha a vendég olyan utasításkészlet-kiterjesztést használ, amely a célhoston hiányzik, a migráció vagy sikertelen, vagy a vendég összeomlik. A libvirtben a **CPU model** beállítással lehet szabályozni: a `host-passthrough` a forráshost teljes CPU-ját adja át, míg egy nevesített modell (például `Nehalem` vagy `EPYC`) hordozhatóbb, de kevesebb kiterjesztést tesz elérhetővé.

Példa libvirt élő migrációra:

```
# Helyőrző; a hosztnevek és a VM név szintetikusak.
virsh migrate --live vm-lab qemu+tcp://célhost/system \
  --persistent --undefinesource --comp-methods zstd
```

A `--persistent` a céloldalon is tartósítja a konfigurációt, az `--undefinesource` a forrásoldalon törli a definíciót, a `--comp-methods zstd` pedig tömöríti a memórialapokat a hálózati forgalom csökkentése érdekében. A tömörítés CPU-t fogyaszt, ezért a választást a hálózat és a CPU terhelésének függvényében kell meghozni.

A migráció során a hálózati kapcsolatnak stabilnak kell lennie. A gyakorlatban a migrációs forgalmat külön VLAN-ra vagy dedikált hálózatra szokták terelni, hogy a vendégforgalommal ne versengjen.

22.2.7 Gyakori hibák és helyes megoldások

A VM-ek és a migráció üzemeltetése során számos tipikus hiba fordul elő. Az alábbiakban ezeket és a helyes megoldást tekintjük át.

1. Hiba: túlzott vCPU-overcommit.
Ha egy hoszton sok VM fut, és mindegyik több vCPU-t kap, mint a fizikai magok száma, a vendégek CPU-ra várnak, és a válaszidő megnő.
Megoldás: mérni kell a hoszt CPU-kihasználtságát, a runqueue-hosszt és a vendégek CPU-igényét. A vCPU-k számát a tényleges párhuzamos igényhez kell igazítani; latency-érzékeny vendégeknél kerülni kell a túlzott overcommitot, vagy CPU-affinitást és CPU-pinninget kell alkalmazni.

2. Hiba: a ballooning túlzott beállítása.
A hypervisor folyamatosan növeli a balloon méretét, hogy memóriát szabadítson fel, de a vendég swapelni kezd, vagy OOM-ölő folyamatokat indít.
Megoldás: a ballooningot a vendég memórianyomásának figyelésével kell összekapcsolni. A vendég szabad memóriája, swap-használata és az oldal-visszanyerési statisztikák alapján kell a balloon méretét szabályozni. A cél a vendég kényelmes működése, nem a hoszt memória maximalizálása.

3. Hiba: snapshot készítése quiescing nélkül.
Egy futó adatbázis-kiszolgálóról készített snapshot a memória és a lemez inkonzisztens állapotát rögzíti. Visszaállítás után az adatbázis sérült lehet.
Megoldás: guest agent használata a vendégben, és a `--quiesce` opció engedélyezése. A gyakorlatban a kritikus rendszereknél a snapshotot a szolgáltatás rövid leállításával vagy az adatbázis tranzakciós nyugalmi pontján kell ütemezni.

4. Hiba: a live migration nem konvergál.
A dirty page keletkezési sebessége nagyobb, mint a hálózati sávszélesség, ezért a stop-and-copy fázis soha nem éri el a kívánt maradék oldalszámot, és a migráció leáll vagy nagyon hosszú ideig tart.
Megoldás: növelni kell a migrációs sávszélességet, csökkenteni a dirty page sebességet (például a vendég terhelésének átmeneti korlátozásával), vagy post-copy migrációt kell választani. A hypervisor migrációs naplója megmutatja a dirty page statisztikákat; ez alapján lehet dönteni.

5. Hiba: CPU-inkompatibilitás a célhoston.
A vendég olyan CPU-kiterjesztést használ, amely a célhoston hiányzik, és a migráció után a vendég lefagy vagy hibát jelez.
Megoldás: a libvirt CPU-modell beállításánál hordozható modellt kell választani, nem `host-passthrough`-t, vagy a migráció előtt ellenőrizni kell a célhost CPU-flageit. A `virsh cpu-compare` parancs segíthet.

6. Hiba: rossz eszközvirtualizációs választás.
Teljes emuláció használata nagy I/O-igényű vendégnél, vagy passthrough használata olyan VM-nél, amelyet migrálni kívánunk.
Megoldás: a teljesítményigény és a migrálhatóság közötti kompromisszumot tudatosan kell kezelni. A virtio+vhost általában jó egyensúlyt ad; ha a migráció nem követelmény, a passthrough nagyobb teljesítményt nyújthat.

7. Hiba: snapshotot backupként használni.
A snapshot a lemezkép overlay-ében tárolja a változásokat; ha az alaplemez megsérül vagy törlődik, a snapshot használhatatlan.
Megoldás: a snapshot a gyors visszaállítás eszköze, nem pedig a katasztrófa utáni helyreállítás. Rendszeres, külső tárolóra írt biztonsági mentést kell készíteni.

22.2.8 Labor: VM- és migration-terv készítése

Ez a labor egy tervezési feladat: egy emulált környezetben készítsünk VM-et, snapshotot és migrációs tervet. A cél nem a teljes migráció kivitelezése, hanem a konfiguráció és a mérési módszer kidolgozása, valamint a várt és mért eredmények elkülönítése. A parancsok szemléltetők, nem éles rendszeren hajtandók végre. A valódi lemez, bootloader vagy gyártói eszköz módosítása tilos.

22.2.9 Kiinduló állapot

- QEMU/KVM vagy QEMU TCG emulátor egy eldobható hoszton.
- Két emulált hoszt szerep: `host-A` és `host-B` (két QEMU-példány vagy két külön gépen futó QEMU).
- Egy qcow2 lemezkép, amelyen egy minimális vendég operációs rendszer fut (például egy kis Linux-rendszer, csak szintetikus adatokkal).
- A hálózat egy izolált bridge vagy user-mode hálózat.
- Nincs valós hitelesítő adat; minden felhasználónév és hosztnév helyőrző.

22.2.10 Lépések

1. **Konfiguráció rögzítése:**
   Hozzunk létre egy `vm-lab` nevű VM-et. A konfigurációt egy verziózott szövegfájlba (például `vm-lab.xml` vagy QEMU parancssor) írjuk le. Rögzítsük a QEMU verzióját, a kernel verzióját, a CPU típusát, a memória méretét (például 2048 MiB), a vCPU-k számát (2), a lemezkép formátumát (qcow2) és a hálózati modellt (virtio-net).

2. **VM indítása és mérés:**
   Indítsuk el a VM-et, és mérjük meg a bootolási időt. A méréshez használjunk `time` parancsot vagy a hypervisor naplójában a vendég első rendszerüzenetének időbélyegét. A várt bootolási idő egy emulált környezetben több másodperc is lehet; a pontos érték a hardvertől és a vendégtől függ.

3. **Snapshot készítése:**
   Készítsünk lemezpillanatképet a futó VM-ről. Ellenőrizzük a qcow2 overlay fájl létrejöttét, és mérjük a snapshot művelet idejét. Végezzünk egy egyszerű módosítást a vendégben (például hozzunk létre egy szintetikus fájlt), majd állítsuk vissza a VM-et a snapshotból, és ellenőrizzük, hogy a módosítás eltűnt.

4. **Migrációs terv:**
   Tervezzük meg a VM migrációját `host-A`-ról `host-B`-re. A terv tartalmazza: a célhoston lefoglalandó erőforrásokat, a hálózati sávszélesség becslését, a memória méretét, a dirty page keletkezés becslését, és a várható downtime-t. A migráció parancsát ne hajtsuk végre éles hálózaton; emulált környezetben vagy szimulációban teszteljük.

5. **Mérési napló:**
   Minden méréshez rögzítsük a pontos parancsot, a konfigurációt, az időbélyeget és az eredményt. A mért értékeket külön oszlopban tüntessük fel a várt értékektől.

22.2.11 Példa mérési táblázat

Művelet	Várt érték	Mért érték	Mérési módszer
VM bootolási idő	< 30 s (emulált)	26,4 s	`time` + vendég log
Snapshot készítés	< 2 s	1,1 s	`time virsh snapshot-create-as`
Visszaállítás snapshotból	< 5 s	3,8 s	`time virsh snapshot-revert`
Memória átvitel migrációban	~2048 MiB	2048 MiB	hypervisor napló
Downtime (stop-and-copy)	< 200 ms	145 ms	QEMU `info migrate`

A várt értékeket a konfiguráció alapján előre meg kell indokolni; a mért értékek csak a tényleges lefutás után kerülnek a naplóba. A két oszlop szétválasztása biztosítja, hogy ne tekintsük ellenőrzött eredménynek a becslést.

22.2.12 Visszaállítás

A labor befejezése után a létrehozott lemezképeket, snapshot fájlokat és konfigurációkat törölni kell, vagy az eldobható környezetet le kell állítani. Ha a vendégben bármilyen módosítást végeztünk, a VM-et a snapshot visszaállításával az eredeti állapotba kell hozni, majd a teljes képet törölni. A hoszt valódi fájlrendszerét nem szabad módosítani.

22.2.13 Összefoglalás és önellenőrzés

A VM életciklusa a definiált állapotból a futó állapoton át a leállított vagy törölt állapotig tart. A CPU-virtualizálás a vCPU-k ütemezésén és a hardveres VM exit/entry mechanizmuson alapul. A memóriavirtualizálás kétlépcsős címfordítást és balloningot alkalmaz a felülfoglalás kezelésére. Az eszközvirtualizálásban a virtio paravirtualizált modellje kínál jó egyensúlyt a teljesítmény és a migrálhatóság között. A snapshot konzisztens állapotmentést, a live migration pedig minimális downtime-mal történő átköltözést tesz lehetővé. A gyakori hibák – mint a vCPU-overcommit, a túlzott ballooning vagy a quiescing nélküli snapshot – mind méréssel és tudatos tervezéssel elkerülhetők.

Önellenőrző kérdés:
Ismertesse a pre-copy live migration fázisait, és magyarázza meg, mi történik, ha a dirty page keletkezési sebessége tartósan meghaladja a hálózati sávszélességet. Milyen megoldást választana?

Válasz vázlat:
A pre-copy fázisai: előkészítés, teljes memória átvitele, iteratív dirty page újraküldés, stop-and-copy, majd befejezés. Ha $d \geq B$, a migráció nem konvergál, mert minden iterációban több új dirty oldal keletkezik, mint amennyit át tudunk küldeni. Ilyenkor post-copy migrációt érdemes alkalmazni: a VM-et hamar átkapcsoljuk a célhostra, és a hiányzó memóriaoldalakat igény szerint, hálózati laphibákkal töltjük be. Emellett növelhetjük a sávszélességet, csökkenthetjük a dirty page sebességet, vagy tömörítést kapcsolhatunk be.

Ezzel az alfejezettel a virtuális gépek üzemeltetésének alapjait tekintettük át. A következő egység, a Konténerek, OCI és image-építés, a konténerekre, az OCI-szabványra és az image-építésre épít, ahol a VM-nél könnyebb izoláció és a rétegezett fájlrendszer kerül előtérbe.

22.3 Konténerek, OCI és image-építés

A konténerek két, egymást kiegészítő nézőpontból értelmezhetők. A futás közbeni nézet szerint a konténer nem más, mint izolált folyamatok egy csoportja, amelyek saját névtereket, erőforráskorlátokat és biztonsági profilokat kapnak. A tárolási nézet szerint a konténer egy hordozható image, vagyis rétegezett fájlrendszer-köteg és metaadatok összessége, amelyből a futtatókörnyezet egyértelműen elő tudja állítani a futó példányt. Az Open Container Initiative (OCI) szabványai ezt a két nézetet formalizálják: az OCI Image Specification az image formátumát, az OCI Runtime Specification pedig a futó konténer konfigurációját és életciklusát rögzíti. A következő szakaszok ezt a két modellt, valamint a gyakorlati image-építés és futtatás lépéseit mutatják be.

22.3.1 1. A konténer mint izolált folyamat

A Linux kernelben a konténer nem különálló entitás, hanem hagyományos folyamatok együttese, amelyeket a kernel névterek, cgroupok és biztonsági mechanizmusok segítségével izolál a rendszer többi részétől. Nincs külön kernel, nincs külön ütemező, nincs hardvervirtualizáció. A folyamatok ugyanazt a rendszermagot használják, de a kernel számukra korlátozott, saját nézetet biztosít a rendszer erőforrásairól.

A névterek (namespaces) a rendszer erőforrásainak logikai particionálását végzik. Minden névtér egy adott erőforrás-osztály láthatóságát határozza meg. A legfontosabbak:

- **PID-névtér:** a benne lévő folyamatok saját PID-számozást látnak, és nem látják a névtéren kívüli folyamatokat. A névtérben az első folyamat PID 1 lesz, még akkor is, ha a hoszton valójában egy másik PID-je van.
- **Mount-névtér:** a fájlrendszer-becsatolások (mount pontok) elkülönítését teszi lehetővé. A konténer saját gyökérfájlrendszert (`rootfs`) kaphat, amelyet a hoszt nem lát ugyanabban a formában.
- **Network-névtér:** saját hálózati interfészeket, IP-címeket, útválasztási táblát és tűzfalszabályokat biztosít. A `lo` interfész a névtérben külön példány.
- **UTS-névtér:** a hoszt- és domainnév elkülönítésére szolgál.
- **IPC-névtér:** a System V IPC objektumokat és POSIX üzenetsorokat izolálja.
- **User-névtér:** a felhasználói és csoportazonosítók leképezését teszi lehetővé. Egy folyamat a saját névterében root (UID 0) lehet, miközben a hoszton egy nem privilegizált UID-hez van rendelve. Ez a rootless konténerek egyik alapja.
- **Cgroup-névtér:** a cgroup hierarchia láthatóságát korlátozza.
- **Time-névtér:** (újabb kernelekben) a rendszeróra értékét izolálja.

A cgroupok (control groups) nem a láthatóságot, hanem az erőforrás-felhasználást szabályozzák. A cgroup v2 hierarchiában egy-egy cgroupba helyezett folyamathalmazra CPU-idő, memória, I/O-sávszélesség, PID-szám és egyéb korlátok állíthatók be. A cgroupok hierarchikusak, és a konténer-runtime minden konténerhez egy külön cgroupot hoz létre. A korlát túllépése esetén a kernel a folyamatot lelassítja, hibát ad, vagy a memóriahasználat túllépésekor az OOM-killer (out-of-memory killer) segítségével leállítja a folyamatot.

A biztonsági izoláció további rétegei:

- **Capabilities:** a Linux a root jogosultságokat kisebb képességekre bontja (pl. `CAP_NET_ADMIN`, `CAP_SYS_ADMIN`). Egy konténer alapértelmezés szerint nem kapja meg az összes képességet, így a konténeren belüli root nem egyenértékű a hoszt rootjával.
- **Seccomp:** rendszerhívás-szűrés. A konténer számára engedélyezett rendszerhívások listája korlátozható, ami csökkenti a kernel támadási felületét.
- **LSM (Linux Security Modules):** SELinux, AppArmor vagy más modul címkékkel és szabályokkal tovább korlátozhatja a konténer folyamatainak hozzáférését.

A konténer tehát futás közben a következő komponensek együttese:

1. Egy vagy több folyamat, amelyeket a runtime indít.
2. Névterek, amelyek meghatározzák, mit látnak a folyamatok.
3. Cgroupok, amelyek korlátozzák, mennyi erőforrást használhatnak.
4. Biztonsági profilok (capabilities, seccomp, LSM), amelyek meghatározzák, mit tehetnek.

Ez a modell gyors indítást és kis erőforrás-lábnyomot tesz lehetővé, mert nem kell operációs rendszert bootolni. Ugyanakkor az izoláció gyengébb, mint egy virtuális gép esetében, mert

minden konténer ugyanazt a kernelpéldányt osztja. A kernel sebezhetősége az összes konténert érintheti.

22.3.2 2. A konténer mint image

A konténer tárolási formája az image. Az image nem egyetlen nagy fájl, hanem egy rétegezett, tartalom-címzett (content-addressable) fájlrendszer-köteg, amelyet az OCI Image Specification ír le. Az image célja, hogy a futtatáshoz szükséges fájlrendszer, konfiguráció és metaadatok reprodukálható módon legyenek csomagolva.

Egy OCI image fő összetevői:

- **Manifest:** az image leírása, amely hivatkozik a config objektumra és a rétegekre. Tartalmazza a média típusokat, méreteket és digesteket (SHA-256 lenyomatokat).
- **Config:** az image futtatási konfigurációja: milyen architektúrára és operációs rendszerre készült, melyik az entrypoint, milyen környezeti változók, milyen portok, milyen fájlrendszer-rétegek tartoznak hozzá, és milyen a rootfs diff-azonosítója.
- **Layers (rétegek):** a fájlrendszer tartalmát leíró, egymásra épülő tar archívumok. Minden réteg egy könyvtárfa-diffet tartalmaz: az előző réteghez képest hozzáadott, módosított vagy törölt fájlokat reprezentálja. A rétegeket a futtatáskor overlay vagy hasonló technikával egyesíti a runtime.
- **Index (opcionális):** több architektúrához vagy platformhoz tartozó manifestet foghat össze (multi-arch image).

A rétegek tartalom-címzettek: minden réteg azonosítója a tar archívum SHA-256 digestje. Ez lehetővé teszi a deduplikációt, a cache-elést és az integritás ellenőrzését. Ha két image ugyanazt a réteget használja, a tárolóban elegendő egyszer tárolni.

Az image-építés során a build rendszer (pl. Docker, Buildah, Podman) egy bázis image-ből kiindulva egy ideiglenes konténert indít, abban végrehajtja az utasításokat, majd az így keletkezett fájlrendszer-változást új rétegként rögzíti. Ez azt jelenti, hogy a build lépései valójában konténerfuttatások, és a réteg a futtatás során keletkezett diff.

22.3.3 3. OCI image specifikáció: manifest, config, layers

Az OCI Image Specification három média típust határoz meg a leggyakoribb esetre:

- `application/vnd.oci.image.manifest.v1+json` – a manifest.
- `application/vnd.oci.image.config.v1+json` – a config.
- `application/vnd.oci.image.layer.v1.tar+gzip` – a rétegek (gyakran gzip tömörítéssel, de lehet más is, pl. zstd).

Egy manifest vázlata pszeudokódban:

```
{
  "schemaVersion": 2,
  "mediaType": "application/vnd.oci.image.manifest.v1+json",
  "config": {
    "mediaType": "application/vnd.oci.image.config.v1+json",
    "digest": "sha256:...",
    "size": 1234
  },
  "layers": [
    {
      "mediaType": "application/vnd.oci.image.layer.v1.tar+gzip",
```

```
      "digest": "sha256:...",
      "size": 5678
    }
  ]
}
```

A config objektum tartalmazza a platformot, a processzekre vonatkozó alapértelmezett beállításokat és a rootfs jellemzőit. Például:

```
{
  "architecture": "amd64",
  "os": "linux",
  "config": {
    "Env": ["PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin"],
    "Cmd": ["/bin/sh"],
    "WorkingDir": "/"
  },
  "rootfs": {
    "type": "layers",
    "diff_ids": ["sha256:..."]
  }
}
```

A `diff_ids` a rétegek kitömörített tartalmának digestje, míg a manifestben a tömörített réteg digestje szerepel. Ezzel a specifikáció elválasztja a szállítási formátumot a tartalom integritásától.

A rétegek sorrendje fontos: az első réteg a legalacsonyabb szint (gyakran egy minimális operációs rendszer alap), a következő rétegek az arra épülő módosítások. A futtatáskor a rétegeket ebben a sorrendben kell egymásra helyezni. Egy fájl törlését egy felső rétegben "whiteout" fájl jelzi (a `whiteout` mechanizmus az overlay fájlrendszerből ismert).

22.3.4 4. OCI runtime specifikáció és a config.json

Míg az image azt írja le, *mit* kell futtatni, az OCI Runtime Specification azt határozza meg, *hogyan* kell a konténert létrehozni és futtatni. A runtime (pl. `runc`, `crun`) egy `config.json` nevű konfigurációs fájlt kap, amely a `rootfs` elérési útját, a folyamat definícióját, a névtereket, cgroupokat, mountokat és biztonsági beállításokat tartalmazza.

A legfontosabb mezők:

- `ociVersion`: a specifikáció verziója, pl. `"1.1.0"`.
- `process`: a folyamat paraméterei: `args`, `env`, `cwd`, `user`, `capabilities`, `rlimits`.
- `root`: a rootfs elérési útja és írásvédettsége (`readonly`).
- `mounts`: a mount pontok listája, pl. `/proc`, `/dev`, `/sys`.
- `linux.namespaces`: a létrehozandó névterek listája.
- `linux.resources`: cgroup korlátok (CPU, memória, PID).
- `linux.devices`: eszköz-hozzáférési szabályok.
- `linux.seccomp`: seccomp szűrő.

Egy minimális `config.json` vázlat (a valós futtatáshoz további mezők szükségesek):

```
{
  "ociVersion": "1.1.0",
  "process": {
    "terminal": true,
    "user": {"uid": 0, "gid": 0},
    "args": ["/bin/sh"],
```

```
    "env": ["PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin"],
    "cwd": "/",
    "capabilities": {
      "bounding": ["CAP_CHOWN", "CAP_DAC_OVERRIDE", "CAP_NET_BIND_SERVICE"],
      "effective": ["CAP_CHOWN", "CAP_DAC_OVERRIDE", "CAP_NET_BIND_SERVICE"],
      "permitted": ["CAP_CHOWN", "CAP_DAC_OVERRIDE", "CAP_NET_BIND_SERVICE"]
    }
  },
  "root": {
    "path": "rootfs",
    "readonly": false
  },
  "hostname": "oci-container",
  "mounts": [
    {"destination": "/proc", "type": "proc", "source": "proc"},
    {"destination": "/dev", "type": "tmpfs", "source": "tmpfs"}
  ],
  "linux": {
    "namespaces": [
      {"type": "pid"},
      {"type": "network"},
      {"type": "mount"},
      {"type": "uts"},
      {"type": "ipc"}
    ],
    "resources": {
      "memory": {"limit": 268435456},
      "cpu": {"shares": 1024}
    }
  }
}
```

A fenti példában a konténer 256 MiB memóriakorlátot kap. A `cpu.shares` a CPU-idő relatív súlyát adja meg, ha verseny van. A cgroup v2 esetén a pontos mezők kissé eltérnek, de a runtime fordítja le a specifikációt kernel-hívásokra.

22.3.5 5. Image-építés a gyakorlatban

Az image-építés tipikus módja a `Dockerfile` vagy `Containerfile`, amelyet a Buildah, Podman vagy Docker build rendszere értelmez. A mögöttes elv azonban mindig a rétegezett diff: minden utasítás egy új réteget hoz létre.

Példa egy minimális `Containerfile`:

```
FROM debian:stable-slim
RUN apt-get update && apt-get install -y --no-install-recommends \
    curl \
 && rm -rf /var/lib/apt/lists/*
COPY app /usr/local/bin/app
USER nobody
ENTRYPOINT ["/usr/local/bin/app"]
```

A build lépései:

1. A `FROM` utasítás letölti vagy kiválasztja a bázis image-et. Ennek rétegei lesznek az alsó rétegek.
2. A `RUN` utasítás egy ideiglenes konténert indít a bázis image-ből, lefuttatja a parancsot, majd a fájlrendszer-változást új rétegként menti. Az `apt-get` által telepített csomagok, a létrehozott konfigurációk mind ebbe a rétegbe kerülnek.

3. A `COPY` utasítás a hoszt fájlrendszeréből másol fájlokat az image-be. Ez is egy réteget eredményez.
4. A `USER` utasítás nem hoz létre fájlrendszer-változást, hanem a config objektumban rögzíti az alapértelmezett felhasználót.
5. Az `ENTRYPOINT` a config-ban adja meg az induló parancsot.

A build rendszer minden réteget cache-el. Ha egy utasítás nem változik, a korábban létrehozott réteg újra felhasználható. Emiatt a gyakorlatban fontos a rétegek sorrendje: a gyakran változó utasításokat érdemes a Dockerfile végére helyezni, hogy az előttük lévő rétegek cache-e érvényes maradjon.

Példa rossz sorrendre:

```
FROM debian:stable-slim
COPY app /usr/local/bin/app
RUN apt-get update && apt-get install -y curl && rm -rf /var/lib/apt/lists/*
```

Ha az `app` fájl gyakran változik, akkor a `COPY` réteg mindig újraépül, és emiatt a `RUN` réteg is újra fut, hiába nem változott a csomagtelepítés. A helyes sorrendben a `RUN` előbb van, így a csomagtelepítés réteg cache-elhető.

A multi-stage build lehetővé teszi, hogy a build ideiglenes környezetében nagy fejlesztői eszközöket használjunk, de a végső image-be csak a szükséges binárist másoljuk át. Ez csökkenti az image méretét és a támadási felületet.

```
FROM golang:1.21 AS builder
WORKDIR /src
COPY go.mod go.sum ./
RUN go mod download
COPY . .
RUN CGO_ENABLED=0 go build -o /app ./cmd/app

FROM debian:stable-slim
COPY --from=builder /app /usr/local/bin/app
USER nobody
ENTRYPOINT ["/usr/local/bin/app"]
```

Itt a végső image csak a kis méretű `app` binárist és a minimális futtatókörnyezetet tartalmazza; a Go fordító és a forráskód nem kerül bele.

22.3.6 6. Rootless konténerek

A hagyományos konténer-futtatáshoz root jogosultság szükséges, mert a névterek és cgroupok létrehozása gyakran privilegizált művelet. A rootless konténerek ezt a korlátot úgy kerülik meg, hogy a felhasználói névtér (user namespace) segítségével a folyamat a saját névterében root-nak látszik, de a hoszton a hívó felhasználó jogosultságaival fut. A `newuidmap` és `newgidmap` segítségével a hoszt UID-jei és a névtér UID-jei között leképezés jön létre.

A rootless konténerek jellemzői:

- A runtime nem igényel root-ot a hoszton.
- A fájlrendszer-overlay gyakran `fuse-overlayfs`-et használ, mert a sima overlayfs mount root nélkül nem lehetséges.
- A hálózati névtérhez `slirp4netns` szükséges, amely egy felhasználói térben futó hálózati stacket biztosít.
- A cgroup v2 esetén a felhasználó a saját cgroupját delegálva kezelheti, de a korlátok beállításának lehetősége korlátozottabb, mint root esetén.

A rootless mód fontos biztonsági előnyöket ad, mert csökkenti a hoszt kompromittálódásának kockázatát, de bizonyos funkciók (pl. host hálózati eszközök elérése) korlátozottak. A gyakorlatban a Podman, Buildah és a Docker rootless módja is elérhető.

22.3.7 7. Gyakori hibák és helyes megoldások

A konténerek és image-ek használata során számos tipikus hiba fordul elő:

- **Titkok az image-ben:** Ha az image-építés során hitelesítő adatot, privát kulcsot vagy belső URL-t másolunk be, az a rétegben marad, még akkor is, ha később töröljük a fájlt. A rétegek tartalma visszafejthető, ezért a titkokat soha ne építsük image-be. Helyette használjunk runtime secret-eket, környezeti változókat vagy mounted volume-ot.
- **Túl nagy image:** A felesleges csomagok, fejlesztői eszközök, cache fájlok növelik az image méretét. Használjunk multi-stage buildet, és takarítsuk el a csomagkezelő cache-ét ugyanabban a `RUN` rétegben, ahol a telepítés történt.
- **Root felhasználó használata:** Alapértelmezés szerint sok image root-ként futtatja a folyamatot. Ez növeli a konténerből való kitörés kockázatát. Hozzunk létre nem privilegizált felhasználót (`USER`), és adjuk meg a minimálisan szükséges capabilities-eket.
- **Nem megfelelő erőforrás-korlátok:** Ha nem állítunk be memória- és CPU-korlátot, egy konténer kimerítheti a hoszt erőforrásait. A cgroup-korlátokat mindig explicit módon állítsuk be, és a fontos szolgáltatásokhoz definiáljunk lower/upper limitet.
- **Réteg cache figyelmen kívül hagyása:** Ha a `RUN` utasításban gyakran változó fájlokat másolunk, a cache hatékonysága csökken. Rendezzük a Dockerfile-t úgy, hogy a stabil rétegek legyenek elöl.
- **Nem ellenőrzött base image:** A publikus registry-ből letöltött image-ek ismeretlen sérülékenységeket vagy rosszindulatú kódot tartalmazhatnak. Használjunk megbízható forrást, ellenőrizzük a digestet, és rendszeresen frissítsünk.

22.3.8 8. Labor: minimális OCI image és runtime konfiguráció

A labor célja, hogy a tanult fogalmakat kézzelfoghatóan alkalmazzuk. A feladat egy minimális OCI image létrehozása, majd a futtatása `runc`-kel emulált környezetben. A műveleteket egy eldobható virtuális gépben vagy konténerben végezzük, nem a hoszt valódi rendszerén. A következő leírás szemléltető jellegű; a parancsokban minden fájl a munkakönyvtárban jön létre, a hoszt rendszerét nem módosítjuk.

22.3.9 Kiinduló állapot

- Emulált környezet: QEMU/KVM vagy VirtualBox virtuális gép, 2 vCPU, 2 GiB RAM, 10 GiB lemez, Debian 12 (bookworm) vagy hasonló Linux.
- Telepített eszközök: `runc`, `umoci` (vagy `skopeo`, `buildah`, `podman`). A pontos verziókat rögzíteni kell a reprodukciós naplóban.
- A munkakönyvtár: `~/oci-lab`. Minden parancs ebben fut.

22.3.10 1. lépés: rootfs létrehozása

Hozzunk létre egy minimális gyökérfájlrendszert, amely egy `bin/sh` állományt és a szükséges könyvtárakat tartalmazza. A leggyorsabb mód egy már meglévő image exportálása, de itt kézzel is előállíthatjuk:

```
mkdir -p rootfs/bin rootfs/dev rootfs/proc rootfs/tmp
cp /bin/busybox rootfs/bin/sh   # szimbolikus link vagy másolat
chmod +x rootfs/bin/sh
```

A `busybox` egyetlen bináris, amely több parancsot is megvalósít, ezért ideális minimális rootfs-hez. Ellenőrizzük a méretet:

```
du -sh rootfs
```

Várható eredmény: néhány MiB.

22.3.11 2. lépés: minimális OCI runtime config

Hozzuk létre a `config.json` fájlt a fenti vázlat alapján, de a `process.args` legyen `/bin/sh`. A mountok közé vegyük fel a `/proc` és `/dev` mountokat. A memórialimit legyen 64 MiB, a CPU-súly alapértelmezett.

```
{
  "ociVersion": "1.1.0",
  "process": {
    "terminal": false,
    "user": {"uid": 0, "gid": 0},
    "args": ["/bin/sh", "-c", "echo 'OCI container running'; sleep 5"],
    "env": ["PATH=/bin"],
    "cwd": "/",
    "capabilities": {
      "bounding": ["CAP_CHOWN", "CAP_DAC_OVERRIDE"],
      "effective": ["CAP_CHOWN", "CAP_DAC_OVERRIDE"]
    }
  },
  "root": {"path": "rootfs", "readonly": true},
  "hostname": "lab-oci",
  "mounts": [
    {"destination": "/proc", "type": "proc", "source": "proc"},
    {"destination": "/dev", "type": "tmpfs", "source": "tmpfs"}
  ],
  "linux": {
    "namespaces": [
      {"type": "pid"},
      {"type": "network"},
      {"type": "mount"},
      {"type": "uts"},
      {"type": "ipc"}
    ],
    "resources": {
      "memory": {"limit": 67108864},
      "cpu": {"shares": 1024}
    }
  }
}
```

22.3.12 3. lépés: futtatás runc-vel

A `runc` parancsot a config könyvtárában kell futtatni:

```
cd ~/oci-lab
runc run mycontainer
```

Az elvárt kimenet: `OCI container running`, majd 5 másodperc várakozás után a konténer kilép. A kilépési kód 0.

22.3.13 4. lépés: image létrehozása és rétegek vizsgálata

A `rootfs`-ből készítsünk OCI image-et a `umoci` segítségével. Ehhez először egy OCI layoutot kell inicializálni, majd a rootfs-t importálni:

```
mkdir -p oci-image
umoci init --layout oci-image
umoci new --image oci-image:latest
umoci unpack --image oci-image:latest bundle
# a rootfs tartalmát a bundle/rootfs-be másoljuk, majd újra csomagoljuk
```

A rétegek és a manifest ellenőrzése:

```
skopeo inspect oci:oci-image:latest
```

A kimenetben meg kell jelennie a manifestnek, a config digestnek és a rétegek digestjének. A rétegek száma általában 1–3.

22.3.14 5. lépés: mérési napló

A labor kötelező eleme a reprodukálható mérési napló. A következő táblázat sablont használhatjuk:

Paraméter	Mért érték	Várt érték	Módszer
Indítási idő (hideg)	`time runc run ...` kimenet	< 0.5 s	5 futtatás, medián
Indítási idő (meleg)	ugyanaz cache után	< 0.2 s	5 futtatás, medián
Memóriahasználat	`runc events --stats`	< 5 MiB	cgroup stat
Image méret	`du -sh oci-image`	< 10 MiB	lemezhasználat
Rétegek száma	`skopeo inspect`	1–3	manifest

A mért és a várt értéket külön kell feltüntetni. A mérési körülményeket (CPU típus, órajel, kernelverzió, runc verzió, fájlrendszer típusa) rögzíteni kell. A fenti várt értékek tájékoztató jellegűek, a tényleges mérési eredmények eltérhetnek.

22.3.15 Visszaállítás

A labor után a munkakönyvtár törölhető, a virtuális gép visszaállítható snapshotból. A hoszt rendszerét nem módosítottuk.

22.3.16 9. Önellenőrző kérdések

1. Miben különbözik a konténer mint izolált folyamat a virtuális géptől? Soroljon fel legalább három lényeges különbséget.
2. Mi a szerepe a PID-névtérnek egy konténerben? Mi történik, ha a konténer első folyamata kilép?
3. Hogyan biztosítja az OCI image specifikáció a rétegek integritását és deduplikációját?
4. Miért fontos a rootless mód, és melyik kernel- feature teszi lehetővé a felhasználói névtér nélküli root nélküli futtatást?
5. Melyek a tipikus hibák az image-építés során, és hogyan kerülhetők el?

22.3.17 Összefoglalás

A konténer technológia két szinten értelmezhető: futás közben izolált folyamatok együttese, tárolás közben pedig rétegezett, tartalom-címzett image. A Linux kernel névterek, cgroupok és biztonsági modulok segítségével biztosítja az izolációt, míg az OCI szabványok egységes formátumot adnak az image és a runtime konfiguráció számára. A helyes image-építés a rétegek gondos sorrendjével, multi-stage builddel, nem privilegizált felhasználóval és minimális erőforrás-korlátokkal csökkenti a méretet és a támadási felületet. A következő alfejezet az unikernel és a WASM alapú könnyű izoláció irányába lép tovább, amely még kisebb lábnyomot és gyorsabb indítást ígér, de más kompromisszumokkal.

22.4 Unikernel, WASM és könnyű izoláció

22.4.1 21.5. Unikernel, WASM és könnyű izoláció

Az előző alfejezetben (Konténerek, OCI és image-építés) bemutattuk a konténereket és az OCI szabványt, amelyek lehetővé teszik, hogy egyetlen hoszt kernel felett, egymástól izolált felhasználói tereket futtassunk. Bár a konténerek indítási ideje és erőforrás-lábnyoma jelentősen kisebb, mint a klasszikus virtuális gépeké, a hoszt kernel teljes támadási felületének kitettsége és a kernelen belüli syscall-feldolgozás kontextusváltási költsége továbbra is megmarad.

Ebben az alfejezetben a minimalista és a könnyűsúlyú izolációs modelleket vizsgáljuk, amelyek a konténereknél is tovább radikalizálják a kérdést: mi az a legkisebb szoftverréteg, amelyre egy alkalmazás futtatásához ténylegesen szükség van? Ennek két jellegzetes válasza az **unikernel** (könyvtár operációs rendszer, library OS) és a **WebAssembly (WASM)** sandbox, illetve ezek hibridjei.

22.4.1.1 21.5.1. A könnyűsúlyú izolációs modellek filozófiája

A hagyományos, általános célú operációs rendszerek az időosztásos (time-sharing) modellre épülnek. A kernel feltételezi, hogy több, egymással versengő folyamatot kell kiszolgálnia, amelyek nem bíznak egymásban. Ebből fakadóan a hardveres védelmi gyűrűk (ring 0/ring 3), a virtuális memória és a rendszerhívások a védelem és a multiplexálás alappillérei.

A szerverless platformok, az edge computing csomópontok és a nagy sűrűségű felhős szolgáltatások azonban gyakran **egyetlen** logikai feladatot futtatnak egy munkamenetben. Egy HTTP kérést kiszolgáló függvénynek nincs szüksége több felhasználós shellre, `fork`-ra, vagy egy full értékű ext4 fájlrendszerre. Ilyenkor a klasszikus OS rengeteg olyan szolgáltatást cipel magával, ami ballaszt.

A könnyűsúlyú izoláció célja, hogy a hoszt és az alkalmazás között a lehető legvékonyabb szoftverréteg maradjon. Ez három fő mérőszámban jelent radikális javulást:

1. **Indulási idő (startup latency):** Egy Linux VM indulása másodpercekben, egy konténer indulása több száz milliszekundumban mérhető. Egy unikernel vagy WASM modul elindulása mikroszekundumos vagy néhány milliszekundumos tartományba esik, ami lehetővé teszi a "kérésenkénti" (per-request) izolációt.
2. **Támadási felület (attack surface):** Ha nincs `bash`, nincs `sshd`, nincs felesleges eszközmeghajtó, akkor nincs mit kihasználni. Az unikernel csak a szükséges hálózati stacket és memóriavezérlőt tartalmazza, a WASM modul pedig csak azt a néhány rendszerhívást (WASI), amit explicit módon megadtunk neki.

3. **Erőforrás-lábnyom (memory footprint):** Egy alap Linux disztribúció memóriában 10-50 MiB-ot foglal. Egy konténer legalább néhány MiB-ot. Egy unikernel vagy WASM runtime néhány száz KiB-os tartományban tud maradni.

A kulcsgondolat a **minimalizmus és a reszponzivitás**. Ahelyett, hogy egy általános céllal felvértezett gépet próbálnánk bezárni egy ketrecbe (amit a konténerek és namespaces tesznek), a szoftvert építjük fel úgy, hogy ne is legyenek a ketrecből kitörő komponensei.

22.4.1.2 21.5.2. Az unikernel és a Library OS

A hagyományos OS-ben az alkalmazás a felhasználói térben fut, és rendszerhívásokkal kéri a kernel szolgáltatásait. Ez merev határvonalat húz a kód és az OS közé. Az unikernel (más néven library OS) megközelítés ezt a határvonalat eltörli.

Hogyan működik? Az unikernel egy **egyetlen címterű** (single address space) operációs rendszer. Az alkalmazás kódja és az operációs rendszer szolgáltatásai (hálózatkezelés, fájlrendszer, memóriakezelő) egyetlen, speciálisan összelinkelt binárisba kerülnek. A rendszerhívás nem egy csapda (trap) utasítás és kontextusváltás, hanem egy **egyszerű függvényhívás**.

Az ötlet nem új: a 90-es évek exokerneljei és a Microsoft Singularity projektje is ezt a vonalat kutatták. A modern, felhő-vezérelt implementációk közé tartozik a MirageOS (OCaml), az IncludeOS (C++), a HalVM (Haskell) és a Project Unikraft (C/C++, Linux API-kompatibilitással).

Architektúra: Az unikernelt általában közvetlenül egy **hypervisorra** (KVM, Xen) vagy esetleg hardverre telepítik. Nincs shell, nincs `fork()`/`exec()` rendszerhívás, és általában nincs futó folyamatok közötti memóriavédelem. Az alkalmazás figyelmét a fordító optimalizálója (például link-time optimization) szabadíthatja meg a holt kódoktól, így a kernel azon részei, amelyeket az alkalmazás nem használ, be sem kerülnek a memóriába.

Példa – konfigurációs mintázat: Egy IncludeOS alapú szolgáltatás nem `systemd`-vel indul, hanem a `boot()` függvényben definiál egy hálózati eseményt:

```
// IncludeOS-szerű unikernel példa (pszeudokód, szemléltetés)
// Ezt a kódot keresztszerkesztéssel, a megadott laborkörnyezetben linkeljük.
#include <net/inet>
#include <kernel/service.hpp>

void Service::start()
{
  // A hálózati stack nem a kernelben van, hanem a könyvtárban
  auto& inet = net::Inet4::stack<0>();
  inet.network_config({10, 0, 2, 15}, {255, 255, 255, 0}, {10, 0, 2, 2});

  // Nincs fork/exec, egyetlen eseményhurok fut
  auto& server = inet.tcp().listen(80);
  server.on_connect([] (auto conn) {
    conn->write("HTTP/1.1 200 OK\nContent-Length: 5\n\nHello");
    conn->close();
  });
}
```

Ebben a modellben a TCP/IP stack, a HTTP parser és az alkalmazás logikája egyetlen privilegizált, ring 0 szintű binárisban fut. Ez radikálisan gyors indítást tesz lehetővé: a memória inicializálása és a hálózati kártya felkonfigurálása után azonnal készen áll a szolgáltatás.

Előnyök és korlátok: Az unikernelnek rendkívül kicsi a támadási felülete. Mivel nincs `sshd`, nincs `cron`, nincs shell, a tipikus Linux exploitok 90%-a szóba sem jöhet. Ugyanakkor ez a fejlesztési élmény radikális átalakulását is jelenti. A hagyományos hibakereső eszközök (GDB,

`perf`, `strace`) használata nehézkes, mert a rendszer egyetlen monolit bináris. Sok unikernel projekt ezért speciális GDB-stubokat vagy JTAG-os kernel-debugolást kínál. A gyakorlatban a fejlesztők először Linuxon írják meg a szolgáltatást, majd keresztszerkesztik unikernellé.

22.4.1.3 21.5.3. A WASM sandbox és a WASI

A WebAssembly (WASM) eredetileg a böngészők biztonságos, hordozható, natív közeli sebességű kódfuttatási igényeire született, ám mára kinőtte a böngészőt. A **WASI (WebAssembly System Interface)** megjelenésével a WASM modulok a homokozón kívül is kaphatnak rendszererőforrásokat, így a szerveroldali, könnyű izoláció egyik legfontosabb eszközévé vált.

A WASM belső modellje: A WASM egy alacsony szintű, stack-alapú virtuális gép bájtkódja. Futtatása során a modul **nem fér hozzá a hoszt memóriájához**; egy lineáris, saját memóriatartományon belül működik, amelyet a hoszt futtatókörnyezet (runtime) allokál és felügyel. Az utasításkészlet szigorú típusrendszerrel rendelkezik, ami lehetővé teszi, hogy a bájtkódot a runtime egy validálási fázisban ellenőrizze, mielőtt lefordítja a natív CPU utasításkészletére (JIT vagy AOT).

A hívások két irányban történnek: - **Import:** A modul bekéri a hoszt által nyújtott függvényeket (például `fd_write` az íráshoz). - **Export:** A modul kitárja a saját függvényeit a hosztnak (például `_start` vagy `handle_request`).

A képesség-alapú biztonság (Capability-based Security): A WASI legfontosabb elve a **képesség-alapú** (capability-based) hozzáférés. Míg egy konténer vagy unikernel esetében a futtatáshoz szükséges privilegizált kérések az OS-től jönnek, a WASM modul indításakor a hoszt runtime **explicit listát** ad át a modulnak az elérhető erőforrásokról. A modul nem tudja magától megtalálni a fájlrendszert vagy a hálózatot; csak azokat a leírókat (file descriptor) látja, amiket a runtime az indításkor a kezébe adott.

Példa – WASI hívás WAT-ban (WebAssembly Text): Az alábbi részlet azt mutatja, hogy egy modul a szabványos kimenetre ír. A `fd_write` függvényt a hoszttól kell importálnunk. A laborkörnyezetben a `wasmtime` vagy `wasmer` futásidejű konfigurációja határozza meg, hogy a `stdout` hova mutat.

```
;; WASM/WASI példa (WAT formátum, szemléltetés)
(module
  ;; Fájlleíró írásának importálása a WASI-ból
  (import "wasi_snapshot_preview1" "fd_write"
    (func $fd_write (param i32 i32 i32 i32) (result i32)))

  ;; A modul saját lineáris memóriája
  (memory (export "memory") 1)
  (data (i32.const 8) "Hello, WASI!\n")
  (data (i32.const 0) "\08\00\00\00")  ;; pointer a memóriában
  (data (i32.const 4) "\0d\00\00\00")  ;; hossz

  (func (export "_start")
    ;; stdout = 1, pointer = 0, iovec hossza = 1, visszatérés pointer = 24
    (drop (call $fd_write (i32.const 1) (i32.const 0) (i32.const 1) (i32.const 24)))
  )
)
```

Ez a kód nem hajt végre semmilyen kernel syscall-ot a hagyományos értelemben. A runtime saját belátása szerint vagy közvetlenül a hoszt user-space API-ját hívja meg, vagy egy valós syscall-ot intéz a Linux kernelhez, de a modul ebből semmit sem lát.

WASM és Unikernel összehasonlítása: Bár mindkettő a "minimális izolációt" hirdeti, a filozófiájuk eltér. - Az unikernel egy **gyenge (weak) izolációjú, de erősen optimalizált natív kód**. A benne futó alkalmazás teljes hozzáféréssel bír a saját kerneléhez, mert az a kód része. - A WASM egy **erős, homokozó-alapú izolációt ad**. A benne futó kód nem natív, hanem a definiált, ellenőrzött utasításkészlettel bír, a memóriája és az erőforrásai szigorúan felügyeltek a hoszt által.

22.4.1.4 21.5.4. Indulási idő, támadási felület és kompatibilitás

A könnyű izolációs modellek értékeléséhez mindig a három fő tengelyt kell vizsgálnunk: milyen gyorsan indul, mekkora a támadási felület, és milyen kompatibilitást kínál az alkalmazások felé.

Indulási idő: A startup latency kritikus tényező a per-request skálázódásban. Egy szerverless platformban egy függvényt akár nulláról kell elindítani egy bejövő HTTP kérésre. - **Hagyományos VM:** 5-10 másodperc. - **Konténer (Docker/runc):** 100-500 ms (a hoszt kernelben már futó cgroupok és namespaces létrehozása). - **Unikernel (pl. Firecracker/QEMU MicroVM):** 1-5 ms. A hardver virtualizációs réteg inicializálása és a bináris memóriába lapozása jelenti a fő időt. A jelenlegi, QEMU-alapú vCPU beállításokkal a `virtio` eszközök konfigurálása a leglassabb része az unikernel bootnak. - **WASM modul:** Kevesebb, mint 100 µs. A futásidejű fordítás vagy értelmezés elindítása a `mmap` és a stack beállításával együtt villámgyors, mert nem történik hardvervirtualizáció.

Támadási felület: A támadási felületet az alkalmazás által elérhető, privilegizált vagy nem ellenőrzött kódsorok számával mérjük (közelítőleg).

- *Konténer:* Nagy. A hoszt kernel a maga több tízmillió kódsorával (fájlrendszer, hálózat, futás) mind elérhető a `syscall` interfészen keresztül. A konténerben futó alkalmazás csak nagyon szigorú `seccomp` szabályokkal korlátozható.
- *WASM:* Nagyon kicsi. A modul a hoszt kernel syscall-jait egyáltalán nem éri el. Csak a runtime által biztosított homokozón belüli WASI függvényeket hívhatja, amelyek hozzáféréseit a képesség-alapú modell listázza.
- *Unikernel:* Kicsi, de "kockázatosabb". Az unikernelen belül az alkalmazás privilegizált kódot futtat, azonban az a privilegizált kód kizárólag a saját unikernelje. Nincs `init`, `systemd`, nincs shell. Ha egy támadó kompromittálja az unikernelt, nem tud átugrani egy másik hoszt folyamatra, mert a hypervisor elkülöníti a többitől.

Kompatibilitás: Ez a leggyengébb pontja a könnyű modelleknek. - *Konténerek:* Kiváló. A Linux ABI teljes mértékben kompatibilis. - *Unikernel:* Nehézkes. A `fork()` és a hagyományos folyamatmodell nem támogatott. A POSIX API nagy része emulált (például a Unikraft a `musl` libc-t használja), de sok szoftvernek újra kell gondolnia a folyamat- és szálmodelljét. - *WASM:* Erősen korlátozott. A WASI jelenleg a legalapvetőbb fájl-, socket- és óra-műveleteket támogatja, bár a hálózati socketek hivatalos WASI támogatása a standardizáció egyes fázisaiban még jelentős változásokon mehetett át; a jelenlegi munkakörnyezetben a `wasmtime` és `wasmedge` verziói eltérő WASI szinteket (preview1, preview2) valósíthatnak meg. **Minden ilyen API-használatnál ellenőrizni kell az adott runtime verziójának dokumentációját.** A fejlesztők gyakran Rust vagy Go nyelvekből fordítanak WASM bytecode-ot, ahol a programozási modell eleve lineárisabb, nem pedig fork/exec alapú.

22.4.1.5 21.5.5. Tipikus hibák és azok elkerülése

Amikor egy fejlesztő először találkozik a könnyű izolációval, jellemzően a következő csapdákba esik:

1. **Unikernel úgy használni, mintha konténer lenne.** Egy unikernelben nincs `ps`, `top`, `kill`. A diagnosztika és a konfiguráció a compile-time (futás előtti) lépések része. Ha valaki hirtelen szeretné újraindítani a szolgáltatást, az unikernel gyakran a teljes vCPU újraindítását jelenti. A megoldás: a konfigurációt és a megfigyelést ki kell szervezni a hypervisor szintjére (például virtuális soros portra logolás), vagy opcióként ki kell egészíteni az unikernelt egy szigorúan csak a hibakeresést szolgáló hálózati résszel.
2. **WASM modul hálózati hívásának erőltetése.** A WASM modulok nem tudnak "nyers" Unix socketet nyitni, hacsak a hoszt ezt explicit módon nem engedélyezi. A hibás megoldás az, ha a runtime-ot úgy konfiguráljuk, hogy "minden rendszerhívást engedélyez". A helyes megoldás a képesség-alapú konfigurálás: a `wasmtime --dir=./data app.wasm` parancsban kizárólag a `./data` könyvtár látható, semmi más a hoszt fájlrendszeréből.
3. **Az indulási idő összehasonlítása almával a körtével.** Amikor mérünk, ne egy már cache-elt, előre beolvasott konténer indítását hasonlítsuk egy hideg unikernel bootjához. A hiteles méréshez a hoszt cache-ét ki kell üríteni (`sync; echo 3 > /proc/sys/vm/drop_caches` *emulált környezetben, nem éles gépen*), és az eredményt a hypervisor saját logjával együtt kell rögzíteni.

22.4.1.6 21.5.6. Labor: Unikernel és WASM használati eset értékelése

Ennek a labornak a célja, hogy reprodukálható módon összehasonlítsa egy hagyományos konténer, egy unikernel és egy WASM modul indítási idejét és relatív erőforrás-igényét egy emulált környezetben.

1. Kiinduló állapot és konfiguráció: A labor az `os084` fejezetben bemutatott emulált Linux hoszton fut. - **Hoszt:** QEMU/KVM virtuális gép, 4 vCPU, 8192 MiB RAM, Linux kernel 6.x (a disztribúció `uname -r` által kiadott verziószámát a naplóba rögzíteni kell). - **Konténer runtime:** `runc` vagy `crun` (az OCI szabványnak megfelelően), alap image: `alpine:latest`. - **Unikernel:** Teljes bináris keresztszerkesztése nem szükséges, ha a tanuló nem rendelkezik az eszközlánccal. Használjuk a `unikraft` előre lefordított, minimál HTTP szerver image-ét, vagy a forráskódból történő fordítást a dokumentált Docker-alapú build paranccsal. QEMU MicroVM (firecracker) futtatáshoz. - **WASM:** `wasmtime` futtatókörnyezet (verzió rögzítendő), egy rustból fordult "Hello World" modul, valamint egy WASI-t használó modul.

2. Építési és futtatási parancsok: *(Figyelem: az alábbi parancsok laboratóriumi illusztrációk, a pontos verziók és image-elemek a helyi tárházban (`container-registry.local`) ellenőrizendők.)*

```
# WASM modul előkészítése
wasmtime --version  # pl. wasmtime-cli 22.0.0
rustc --target wasm32-wasip1 hello.rs -C opt-level=z -o hello.wasm

# Unikernel indítása Firecrackerrel
./firecracker --api-sock /tmp/fire.sock --config-file vm_config.json

# Konténer indítása
runc run mycontainer --bundle /path/to/bundle
```

3. Mérési módszer: A mérést a hoszt hipervizorának soros konzol logjával és a hoszt `clock_gettime` hívásait használó wrapper scripttel végezzük. Mérjük: - **T_boot:** az indítási parancs kiadásától az első hasznos válaszig eltelt idő. - **RSS:** a hoszt `smem` vagy `ps eo rss` által mutatott, a folyamathoz hozzárendelt rezidens memória. - **Támadási felület indikátor:** a hoszt által látható, a futtatott entitás által elérhető rendszerhívások számának listája (konténernél `seccomp` policy, unikernelnél a binárisban lévő szimbólumok, WASM-nál az importált függvények száma).

4. Várt és mért eredmény (szétválasztva): A jegyzőkönyvben a várt értékeket a szakirodalom és a dokumentáció alapján, a mért értékeket a saját futtatás alapján kell rögzíteni.

Komponens	Várt indulási idő	Mért indulási idő	Várt memória (RSS)	Mért memória (RSS)	Megjegyzés
OCI Konténer (alpine)	200-400 ms	`<mért>`	5-10 MiB	`<mért>`	A hoszt kernel overheadje dominál.
Unikernel (Firecracker)	5-20 ms	`<mért>`	2-4 MiB	`<mért>`	A VirtIO eszköz inicializálása a leglassabb.
WASM (wasmtime)	0.5-2 ms	`<mért>`	1-2 MiB	`<mért>`	A bájtkód validálása és JIT fordítása.

5. Ellenőrző kérdések: 1. Miért gyorsabb a WASM modul indítása, mint az unikernelé, még akkor is, ha az unikernel hypervisor-alapú? 2. Ha egy unikernelben nincs memóriavédelem az alkalmazás részei között, mi biztosítja a több unikernel közötti izolációt? 3. Hogyan befolyásolja a WASI képesség-alapú modellje a támadási felületet egy olyan környezetben, ahol a hoszt fájlrendszer olvasható? Ha a hoszt `/` alatt látható, de a `wasmtime` meghívásakor `--dir ./data` argumentumot adunk, miért nem fér hozzá a modul a `/` gyökerében lévő többi fájlhoz?

Összefoglalásul: a könnyű izolációs modellek eltüntetik a hagyományos OS réteg egy részét vagy egészét. A unikernel a modularitást és a specializációt radikalizálja, míg a WASM a hordozhatóságot és a szigorú homokozót helyezi előtérbe. A következő fejezetben (Determinizmus, latency és jitter) a valós idejű rendszerek alapjaival foglalkozunk, ahol az itt megismert indulási idők és a determinisztikus erőforrás-kezelés kritikus követelményekké válnak.

VI. RÉSZ – Valós idejű, beágyazott és elosztott rendszerek

23 Valós idejű rendszerek

23.1 Determinizmus, latency és jitter

A valós idejű operációs rendszerekben a helyesség két feltételből áll: a kimenet logikai helyessége és az időbeli korlát betartása. Egy autóipari fékvezérlő, amely a pedálérzékelő jelére a helyes beavatkozó parancsot a megengedett 10 ms helyett 100 ms múlva adja ki, már nem ugyanaz a funkció: a késés balesetet okozhat. Ezért a valós idejű viselkedést mérhető időzítési jellemzőkkel kell leírni, és ezeket méréssel, elemzéssel kell ellenőrizni.

A determinizmus ebben a környezetben nem azt jelenti, hogy a késleltetés nulla vagy állandó, hanem azt, hogy a késleltetések felső korlátja ismert, és a gyakorlatban is betartható. Egy 100 ms-os válaszidő is lehet determinisztikus, ha a garancia 100 ms, és a rendszer ezt soha nem lépi túl. A determinizmus tehát mérhető mennyiségekkel fejezhető ki: megszakítási késleltetés, ütemezési késleltetés, jitter, legrosszabb esetű végrehajtási idő (worst-case execution time, WCET) és az átlagos viselkedés.

23.1.1 A valós idejű viselkedés mérhető jellemzői

A valós idejű rendszer időbeli viselkedését a következő mennyiségekkel jellemezzük:

- **Megszakítási késleltetés (interrupt latency):** az esemény hardveres jelétől a megszakításkezelő (ISR) első utasításáig eltelt idő.
- **Ütemezési késleltetés (scheduling latency):** az ISR befejezésétől (vagy egy feladat felszabadításától) a feladat első utasításáig eltelt idő. Gyakran a teljes válaszidő részeként az eseménytől a feladat futásáig terjedő időt is értik alatta.
- **Jitter:** az azonos típusú eseményekhez tartozó késleltetés vagy végrehajtási idő ingadozása. Mérhető csúcs-csúcs értékként, szórással vagy percentilisekkel.
- **Legrosszabb esetű végrehajtási idő (WCET):** egy feladat maximális CPU-igénye a lehetséges bemenetek és állapotok felett. Az ütemezésben a C_i jelölést használjuk erre.
- **Átlagos viselkedés:** a késleltetés vagy futásidő várható értéke, amely a terhelés és a mérési körülmények függvénye.

Ezek a jellemzők hierarchikusan épülnek egymásra. Az átlagos viselkedés a rendszer tipikus minőségét mutatja, a WCET és a maximális késleltetés pedig a garanciák alapját jelenti. A jitter a kettő közötti ingadozást számszerűsíti.

23.1.2 A késleltetési lánc és mérési pontjai

Egy tipikus valós idejű eseményfeldolgozás a következő szakaszokból áll:

1. **Esemény bekövetkezése:** például egy időzítő túlcsordulása, GPIO-él vagy szenzor megszakítás.
2. **Megszakítási késleltetés:** az eseménytől az ISR első utasításáig.
3. **ISR végrehajtás:** a megszakításkezelő rövid, determinisztikus munkája.

4. **Ütemezési késleltetés:** az ISR befejezésétől a feladat első utasításáig, beleértve a kernelbeli ütemező döntést és a kontextusváltást.
5. **Feladat végrehajtás:** a felhasználói vagy kernelbeli feladat logikai munkája, amelynek hossza a bemenettől függ.

A mérési pontokat az alábbi időtengely szemlélteti (szöveges ábra):

```
t_event ---> t_ISR_entry ---> t_ISR_exit ---> t_task_entry ---> t_task_exit
|----- interrupt latency -----|
                               |------ scheduling latency ------|
```

- **Jelmagyarázat:** t_{event} az esemény bekövetkezésének időpontja, t_{ISR_entry} az ISR első utasításának időpontja, t_{ISR_exit} az ISR utolsó utasításának időpontja, t_{task_entry} a feladat első utasításának időpontja, t_{task_exit} a feladat befejezésének időpontja.
- **Feltétel:** az ábra feltételezi, hogy az ISR felszabadítja a feladatot, és a feladat magasabb prioritású, mint a megszakított kód, így a scheduler azonnal átadja neki a CPU-t. Ha a feladat alacsonyabb prioritású, a scheduling latency tovább növekedhet.

A teljes válaszidőt az eseménytől a feladat befejezéséig a következő képlet adja:

$$R = L_{int} + T_{ISR} + L_{sched} + C$$

ahol L_{int} a megszakítási késleltetés, T_{ISR} az ISR végrehajtási ideje, L_{sched} az ütemezési késleltetés, C pedig a feladat CPU-igénye. A valós idejű követelmény általában az, hogy $R \leq D$, ahol D a határidő.

23.1.3 Megszakítási késleltetés

A megszakítási késleltetés az esemény hardveres jelétől az ISR első utasításáig eltelt idő. Ez az idő nem egyetlen hardveres regiszter, hanem több komponens összege:

1. **Hardveres észlelés és arbitráció:** a megszakításvezérlő észleli a jelet, és ha több megszakítás verseng, kiválasztja a legmagasabb prioritásút. Ez a lépés néhány buszciklust vesz igénybe.
2. **Aktuális utasítás befejezése:** a processzor általában a folyamatban lévő utasítás befejezése után lép be a megszakításkezelőbe. Ha az utasítás hosszú, például egy lassú periféria elérése, a késleltetés megnő. RISC architektúrákon az utasítások rövidek, de a memóriaelérési várakozás (memory stall) vagy buszlock itt is növelheti.
3. **Regisztermentés és kernelbelépés:** a CPU automatikusan menti a státuszregisztert és a visszatérési címet, majd a kernel menti a többi regisztert. Ez a lépés architektúrafüggő, de tipikusan néhány ciklustól néhány tíz ciklusig terjed.
4. **Megszakítás-maszkolás:** ha a kernel kritikus szakaszban letiltja a megszakításokat, az esemény várakozik. Ez a legdominánsabb és legváltozékonyabb komponens. A maszkolási idő felső korlátja a kernel implementációjától függ, és ez az egyik fő oka annak, hogy az általános célú operációs rendszerek megszakítási késleltetése nagy és jitteres.

A megszakítási késleltetés legrosszabb esete tehát:

$$L_{int}^{max} = T_{hw} + T_{instr}^{max} + T_{ctx} + T_{mask}^{max}$$

ahol T_{hw} a hardveres észlelés, T_{instr}^{max} a leghosszabb megszakíthatatlan utasítás, T_{ctx} a regisztermentés, T_{mask}^{max} pedig a maximális megszakítás-maszkolási idő.

Példaként egy ARM Cortex-M3-as mikrokontroller specifikációja szerint a megszakítási belépés késleltetése 12 ciklus, ha a megszakítások engedélyezettek és nincs folyamatban lévő hosszú utasítás. Ez az érték csak a hardveres részt fedi; a kernel maszkolási ideje ezen felül hozzáadódik. Az itt megadott számok jellemző értékek, amelyeket a konkrét platformon ellenőrizni kell.

23.1.4 Ütemezési késleltetés

Az ütemezési késleltetés az ISR befejezésétől a feladat első utasításáig eltelt idő. Ez a szakasz a kernel ütemezőjének és a kontextusváltásnak az idejét foglalja magában.

A lépések a következők:

1. **Kernelbelépés:** az ISR befejezése után a kernel megvizsgálja, hogy a megszakítás felszabadított-e magasabb prioritású feladatot.
2. **Ütemező döntés:** a scheduler kiválasztja a következő futó feladatot. Ez a döntés a prioritási sorrend, a futásra kész listák és az esetleges fair policy függvénye. A döntés ideje általában rövid, de rosszul skálázó adatszerkezeteknél a futásra kész feladatok számával nőhet.
3. **Kontextusváltás:** a kernel menti a megszakított feladat regisztereit, és visszaállítja az új feladat regisztereit. Ez magában foglalja a veremmutató, a státuszregiszter és a feladat-specifikus állapotok cseréjét.

Preemptív rendszerekben a magasabb prioritású feladat azonnal lefuthat, amint felszabadul. Nem preemptív ütemezésnél azonban a feladat csak akkor kap CPU-t, amikor az aktuálisan futó feladat önként elengedi a processzort. Ezért a nem preemptív rendszereknél az ütemezési késleltetés legrosszabb esete magában foglalja a leghosszabb futó feladat hátralévő végrehajtási idejét is.

Az ütemezési késleltetés mérésekor figyelembe kell venni, hogy az ISR befejezése és a feladat első utasítása közötti idő nem csak a scheduler költsége, hanem a kernelbeli adatszerkezetek zárolásának, a cache hideg/meleg hatásának és a megszakítás-maszkolásnak is függvénye.

23.1.5 Jitter

A jitter a késleltetés vagy a végrehajtási idő mintáról mintára való ingadozása. Ha L_i az i-edik esemény késleltetése, akkor a csúcs-csúcs jitter:

$$J_{pp} = \max_i L_i - \min_i L_i$$

Emellett használhatjuk a szórást (σ) vagy a percentiliseket, például a 99. percentilist, amely azt az értéket adja meg, amely alatt a minták 99%-a található.

Tegyük fel, hogy egy 1 ms periódusú feladat tényleges indítási ideje öt egymást követő ciklusban: 0, 0,05, 0,08, 0,02, 0,11 ms. Ekkor $J_{pp} = 0,11 - 0 = 0,11$ ms $= 110$ µs. Ez az ingadozás a vezérlési hurok számára zajként jelenik meg: a mintavételi időpontok nem egyenletesek, ami a szabályozási algoritmus hibáját növeli.

A jitter forrásai lehetnek:

- **Megszakítás-maszkolási idő változása:** a kritikus szakaszok hossza a rendszer állapotától függően változik.

- **Cache és TLB viselkedés:** a kontextusváltás után a cache tartalma eltérő lehet, ami a futási időt befolyásolja.
- **DMA és buszforgalom:** a háttérben zajló adatátvitel verseng a CPU memóriaelérésével.
- **Ütemező döntések:** több futásra kész feladat esetén a prioritási sorrend és az időzítések változhatnak.

A jitter csökkentése valós idejű rendszerekben tipikusan a következő eszközökkel történik: a megszakítás-maszkolási idő minimalizálása, cache-partícionálás vagy cache-lock, determinisztikus memóriaelérés, valamint a feladatok prioritásának helyes megválasztása.

23.1.6 Worst-case és átlagos viselkedés

A valós idejű rendszerek tervezésénél az átlagos viselkedés önmagában félrevezető. Tegyük fel, hogy egy rendszer 99,9%-ban 10 µs alatt válaszol, de 0,1%-ban 1 ms-ot késik. Az átlagos késleltetés körülbelül 11 µs, ami jónak tűnik, de a hard real-time követelmény, például egy 100 µs-os határidő, a ritka késések miatt megsérül. Ezért a garanciák alapja a legrosszabb eset, nem az átlag.

A legrosszabb esetű végrehajtási idő (WCET) egy feladat maximális CPU-igénye a lehetséges bemenetek és állapotok felett. Az ütemezésben a C_i értéket a D_i határidőhöz viszonyítjuk. Ha C_i-t alulbecsüljük, a rendszer határidő-túllépést szenvedhet; ha túlbecsüljük, a processzor kihasználtsága csökken.

A WCET meghatározása két fő módon történhet:

1. **Statikus elemzés:** a program kódját és a hardver modelljét elemezve konzervatív felső becslést adunk. Ez a módszer bizonyítja a korlátot, de a modern processzorok cache, TLB, spekuláció és out-of-order végrehajtása miatt bonyolult.
2. **Mérés alapú becslés:** a programot a lehetséges bemenetek reprezentatív halmazán futtatjuk, és a mért maximumot tekintjük becslésnek. Ez a módszer egyszerűbb, de nem garantálja, hogy minden bemenetre igaz a korlát, mert a bemeneti tér nagy.

A gyakorlatban a biztonsági kritikus rendszereknél a statikus elemzés vagy a konzervatív becslés az elfogadott, míg a kevésbé kritikus rendszereknél a mérés alapú becslés is használható, ha a legrosszabb esetet stressztesztekkel közelítik.

A long-tail jelenség azt jelenti, hogy a késleltetések eloszlásának farka hosszú: az átlaghoz képest a nagy késleltetések ritkák, de jelen vannak. A faroklatencia (tail latency) a magas percentilisekkel jellemezhető. Egy valós idejű rendszernél a 99. vagy 99.9 percentilis fontosabb lehet, mint az átlag.

23.1.7 Méréstechnikai alapok

A latency és jitter mérésére két alapvető megközelítés létezik:

1. **Hardveres trigger és külső megfigyelés:** egy GPIO-élet állítunk elő az esemény pillanatában, és egy másik GPIO-élet az ISR vagy a feladat belépésekor. A két él közötti időt logikai analizátorral vagy oszcilloszkóppal mérjük. Ez a módszer minimális szoftveres overheadet okoz, de a mérési pontok hardveres kiépítését igényli.
2. **Szoftveres időbélyeg:** a kernel vagy a feladat egy szabadon futó számláló (például cycle counter) értékét rögzíti a mérési pontokon, és a különbségeket utólag kiértékeli. Ez a módszer egyszerűbb, de maga a mérési utasítás is időt vesz igénybe, és a megfigyelő hatás (observer effect) torzíthatja az eredményt.

A szoftveres mérésnél a következő óvintézkedések szükségesek:

- A mérési pontokon használt utasítások száma legyen minimális és állandó.
- A naplózás ne a mérés közben történjen, mert a periféria írása lassú. A mintákat pufferelni kell, és a mérési sorozat végén kiírni.
- A számláló túlcsordulását kezelni kell, vagy elegendően széles számlálót kell használni.
- A cache és a TLB állapotát a mérés előtt tudatosan kell befolyásolni (hideg/meleg mérés), mert ez a jitter egyik forrása.

Az emulált környezetben (például QEMU) a mérési eredmények reprodukálhatóbbak, de a valós hardver időzítéseitől eltérhetnek. Az emulátor nem modellezi pontosan a buszversengést, a cache-vezérlést vagy a perifériák valós késleltetését. Ezért az emulált mérés elsősorban a módszertan validálására és a szoftveres komponensek (például maszkolási idő, scheduler költség) vizsgálatára alkalmas. A végső, garantált értékeket a célhardveren kell megerősíteni.

23.1.8 Labor: latency- és jitter-mérés tervezése

A következő labor célja egy reprodukálható mérési terv elkészítése emulált környezetben. A feladat nem az éles gépen fut, hanem QEMU-ban, eldobható konfigurációval. A hoszt valódi lemezét, bootloaderét és rendszerpartícióját tilos módosítani.

23.1.9 Kiinduló állapot és cél

A labor egy ARM Cortex-M3-as emulált rendszeren fut (például a QEMU `lm3s6965evb` gépén), amelyen egy egyszerű valós idejű kernel fut. A kernel rendelkezik egy szabadon futó ciklusszámlálóval, egy periódusos időzítő-megszakítással és egy naplózó UART-tal. A cél: megmérni az interrupt latency-t, a scheduling latency-t és ezek jitterét 1 kHz-es időzítő-megszakítások sorozatán.

A mérési pontok:

- t_0: a timer match pillanata, amelyet a timer hardveres capture regisztere rögzít.
- t_1: az ISR első utasításának időpontja.
- t_2: az ISR utolsó utasításának időpontja.
- t_3: a felszabadított feladat első utasításának időpontja.

A kiszámított mennyiségek:

$$L_{int} = t_1 - t_0$$

$$L_{sched} = t_3 - t_2$$

A jittert a L_{int} és L_{sched} minták csúcs-csúcs ingadozásaként és szórásaként számítjuk.

23.1.10 Mérési módszer és pszeudokód

A méréshez a DWT->CYCCNT regisztert használjuk ciklusszámlálóként, amely 50 MHz-es órajelnél 20 ns felbontást ad. A mérési pontok rögzítése közvetlenül a regiszter olvasásával történik.

Az alábbi pszeudokód szemlélteti a mérési logikát. Ez a kód nem végrehajtandó utasítás az éles gépen, csak a labor tervezetét mutatja be.

```
volatile uint32_t t0, t1, t2, t3;
volatile uint32_t lat_int, lat_sched;
volatile int samples;

void timer_isr(void) {
    t0 = timer_capture();   // timer hardveres capture: a match időpontja
    t1 = cycle_counter();   // ISR belépés időpontja

    // Rövid, determinisztikus ISR munka:
    clear_timer_interrupt();
    task_release(&task);    // feladat felszabadítása

    t2 = cycle_counter();   // ISR kilépés előtt
}

void task(void) {
    t3 = cycle_counter();   // feladat első utasítása

    lat_int = t1 - t0;
    lat_sched = t3 - t2;

    if (samples < MAX_SAMPLES) {
        log_sample(lat_int, lat_sched);
        samples++;
    }
}
```

A `timer_capture()` a timer hardveres capture regiszterét olvassa, amely a megszakítást kiváltó esemény pillanatában tárolja a szabadon futó számláló értékét. Ez pontosabb, mint az ISR eleji olvasás, mert az ISR belépés és a regiszterolvasás közötti időt nem tartalmazza. A `cycle_counter()` a DWT->CYCCNT regisztert olvassa.

A mérési sorozat során a mintákat egy pufferben gyűjtjük, és a mérés végén a UART-on továbbítjuk a hostra. A naplózás nem a mérési ciklusban történik, hogy ne torzítsa az időzítéseket.

23.1.11 Várt értékek és becslés

A mérés előtt analitikus becslést kell készíteni a várt értékekre. Az alábbi számok példák, amelyeket a konkrét platformon ellenőrizni kell:

- A Cortex-M3 megszakítási belépés késleltetése 12 ciklus, így 50 MHz-en a várt L_{int} körülbelül 12/50e6 = 0,24 µs.
- Az ISR futása (a megszakítás törlése és a task release) körülbelül 10-20 ciklus.
- Az ütemezési késleltetés becslése a kontextusváltás költségéből adódik: regisztermentés és visszaállítás körülbelül 16-20 ciklus, a scheduler döntése további 20-30 ciklus, összesen 36-50 ciklus, azaz 0,72-1,0 µs.
- A jitter várt értéke kicsi, mert az ISR-ben nincs megszakítás-maszkolás, és a QEMU modell determinisztikus. A várt J_{pp} a ciklusszámláló felbontásából adódóan legfeljebb néhány ciklus.

A mérés után ezeket az értékeket külön kell összehasonlítani a mért értékekkel. A mért és várt értékeket a jegyzőkönyvben táblázatban kell szétválasztani.

23.1.12 Konfiguráció és reprodukció

A labor reprodukciós csomagja a következőket tartalmazza:

- A QEMU verzióját és a parancsot: `qemu-system-arm -machine lm3s6965evb -nographic -kernel main.elf`
- A toolchain verzióját és a build parancsot: `arm-none-eabi-gcc -O2 -g -mcpu=cortex-m3 -mthumb -T linker.ld -o main.elf main.c`
- A linker scriptet és a forráskódot, beleértve a mérési pontok pontos helyét.
- A nyers mérési naplót CSV formátumban, ahol az oszlopok: `sample, t0, t1, t2, t3, L_int_cycles, L_sched_cycles, L_int_us, L_sched_us`.

A mérési jegyzőkönyvben a következőket kell rögzíteni:

- A konfigurációt (CPU, órajel, emulátor verzió, fordítási beállítások).
- A mérési módszert (mérési pontok, számláló, mintaszám).
- A várt értékeket és azok forrását (specifikáció, analitikus becslés).
- A mért értékeket (átlag, minimum, maximum, szórás, percentilisek).
- A J_{pp} jittert mindkét késleltetésre.
- Az eltérések indoklását, ha a mért értékek eltérnek a vártaktól.

A reprodukálhatóság érdekében a jegyzőkönyvnek tartalmaznia kell a pontos parancsokat és a naplófájlokat is. A mérési eredményeket a hallgató saját futtatásából kell származtatni; a fejezetben közölt példaértékek nem helyettesítik a tényleges mérést.

23.1.13 Önellenőrző kérdések

1. Mely komponensek alkotják a teljes válaszidőt egy megszakításból felszabaduló feladat esetén?
2. Mi a különbség a megszakítási késleltetés és az ütemezési késleltetés között?
3. Hogyan számítjuk a csúcs-csúcs jittert, és miért fontos ez a valós idejű rendszerekben?
4. Miért nem elég az átlagos késleltetés egy hard real-time rendszer tervezésénél?
5. Mely szoftveres komponensek növelhetik a megszakítási késleltetés legrosszabb esetét?

23.1.14 Összefoglalás

A valós idejű viselkedés mérhető jellemzői a megszakítási késleltetés, az ütemezési késleltetés, a jitter, a legrosszabb esetű végrehajtási idő és az átlagos viselkedés. A determinizmus azt jelenti, hogy a késleltetések felső korlátja ismert és betartható, nem pedig azt, hogy a késleltetés állandó. A megszakítási késleltetés az eseménytől az ISR első utasításáig terjed, és a legnagyobb hozzájárulást gyakran a megszakítás-maszkolási idő adja. Az ütemezési késleltetés az ISR befejezésétől a feladat indításáig terjed, és a scheduler költségét, valamint a kontextusváltást foglalja magában. A jitter a késleltetések ingadozása, amelynek csökkentése a valós idejű rendszerek egyik fő tervezési célja. Az átlagos viselkedés a tipikus működést mutatja, de a garanciák alapja a legrosszabb eset, amelyet WCET-elemzéssel vagy konzervatív becsléssel kell meghatározni. A laborban emulált környezetben, reprodukálható módon mérjük ezeket a mennyiségeket, és a mért eredményeket a várt értékekkel elkülönítve értékeljük.

23.2 RTOS ütemezés és priority inversion

Az előző fejezetben a valós idejű viselkedés alapfogalmait – determinizmus, latency, jitter – tekintettük át. Most ezekre építve azt vizsgáljuk, hogyan hoz az RTOS ütemezője időben kiszámítható döntéseket, és mi történik akkor, ha a prioritásos modellt erőforrás-megosztás zavarja meg. A fejezet középpontjában a prioritásos ütemezés és a prioritás-inverzió áll; bemutatjuk a jelenséget, a gyakorlati javítási mintákat – öröklés és plafon –, valamint azt, hogyan viszonyul egymáshoz a két leggyakoribb valós idejű ütemezési politika, az RMS és az EDF.

23.2.1 1. Prioritásos ütemezés RTOS-ben

Egy általános célú operációs rendszer ütemezője többnyire méltányosságra, átlagos válaszidőre vagy interaktivitásra optimalizál. Ezzel szemben egy valós idejű rendszerben a legfontosabb követelmény, hogy a kritikus feladatok a határidejük előtt befejeződjenek. Ennek legegyszerűbb és legelterjedtebb eszköze a **preemptív, prioritásalapú ütemezés**.

Minden futásra kész taskhoz egy numerikus prioritás tartozik. Az RTOS futtatója mindig a legmagasabb prioritású futásra kész taskot választja; ha egy magasabb prioritású task futásra kész állapotba kerül, az ütemező azonnal felfüggeszti az alacsonyabb prioritású taskot, és a magasabbat futtatja. Ezt a tulajdonságot **preemptionnek** nevezzük. A preemptív viselkedés nem opcionális extra, hanem a valós idejű rendszer alapköve: enélkül egy hosszú, alacsony prioritású ciklus eltolhatná egy kritikus task indítását, és a határidő akkor is elúszna, ha a CPU összességében elegendő kapacitással rendelkezik.

A prioritás lehet **statikus** vagy **dinamikus**. Statikus esetben a prioritást a rendszer tervezésekor rögzítjük, és futás közben nem változtatjuk. Dinamikus esetben az ütemező minden döntés előtt újraszámolja a prioritást valamilyen szabály szerint – ilyen például az EDF, ahol a prioritás a hátralévő határidőtől függ. Az RTOS-ek többsége alapértelmezésben statikus prioritásos preemptív ütemezőt használ, mert egyszerű, alacsony overheadű, és jól elemezhető.

A task modellben az előző fejezetben bevezetett jelöléseket használjuk: a P_i task prioritása p_i, legrosszabb esetű CPU-igénye C_i, periódusa T_i, relatív határideje D_i. Az ütemezési elemzéseknél általában feltesszük, hogy $D_i \leq T_i$, és a taskok periodikusan, a periódus elején válnak futásra készen. Egy **ütemezhetőségi teszt** megmondja, hogy az adott taskhalmazra létezik-e olyan ütemezés, amely minden határidőt betart.

23.2.2 2. A prioritás-inverzió jelensége

A prioritásos ütemezés tiszta modelljében a magasabb prioritású task mindig megelőzi az alacsonyabbat. A valóságban ez a szabály sérülhet, ha a taskok **közös erőforrásokat** használnak, és az erőforráshoz való hozzáférést kölcsönös kizárással, például mutexszel védik.

Tekintsünk három periodikus taskot:

- P_3 alacsony prioritású, $p_3 = 1$.
- P_2 közepes prioritású, $p_2 = 2$.
- P_1 magas prioritású, $p_1 = 3$.

A P_3 task belép egy kritikus szakaszba, és lefoglal egy M mutexet, amely egy közös erőforrást véd. Amíg P_3 a kritikus szakaszban tartózkodik, a P_1 task futásra kész lesz, és az ütemező preemptálja P_3-at, mert $p_1 > p_3$. P_1 megpróbálja lefoglalni ugyanazt az M mutexet, de az

már foglalt, ezért **blokkolt** állapotba kerül. Az ütemező visszaadja a CPU-t P_3-nak, hogy az befejezhesse a kritikus szakaszt. Eddig minden logikus.

A probléma akkor jelentkezik, ha miközben P_3 még mindig a kritikus szakaszban fut, a P_2 task is futásra kész lesz. Mivel $p_2 > p_3$, az ütemező preemptálja P_3-at, és P_2 fut. P_2 nem használja az M mutexet, tehát akár hosszú ideig futhat. Eközben P_1 – a legmagasabb prioritású task – továbbra is blokkolva vár arra, hogy P_3 felszabadítsa a mutexet, de P_3 nem kap CPU-t, mert P_2 fut. Így a közepes prioritású P_2 közvetett módon feltartja a magas prioritású P_1-et.

Ezt a jelenséget **prioritás-inverziónak** (priority inversion) nevezzük: a rendszerben a tényleges végrehajtási sorrend nem felel meg a prioritási sorrendnek, mert egy alacsonyabb prioritású task közvetve megakadályozza egy magasabb prioritású task futását.

A következő szöveges Gantt-diagram egy lehetséges lefutást mutat. Az időtengely egységei önkényesek; a sávok az egyes taskok CPU-futását jelölik.

```
Idő:     0    1    2    3    4    5    6    7    8
P3 (1): |====|----|====|----|........|====|----|
P2 (2): |    |    |....|====|====|====|....|    |
P1 (3): |    |    |........|====|====|====|====|
             P3 lock(M)  P1 preemptálja P3-at, P1 lock(M) -> blokkolt
                         P2 preemptálja P3-at
```

A diagramon `=` aktív futást, `.` futásra kész vagy blokkolt várakozást, `-` inaktív állapotot jelöl. Látható, hogy P_1 hiába a legmagasabb prioritású, a kritikus szakasz miatt P_2 futása alatt nem tud tovább haladni. A várakozási idő akár tetszőlegesen hosszú is lehet, ha P_2 számításigénye nagy, vagy ha több közepes prioritású task is van. Ez a legrosszabb esetben határidő-túllépéshez vezethet, még akkor is, ha a rendszer CPU-kihasználtsága alacsony.

A gyakorlatban a prioritás-inverzió nem elméleti kuriózum. Az egyik gyakran idézett eset egy Mars-jármű fedélzeti rendszerében történt, ahol egy alacsony prioritású meteorológiai adatgyűjtő task és egy magas prioritású buszkezelő task közötti mutexverseny miatt a rendszer időnként watchdog-resetet hajtott végre. A hiba pontos körülményei és a szoftverkonfiguráció utólag rekonstruálhatók a nyilvános jelentésekből; itt csak a tanulságot emeljük ki: a jelenség valós rendszerekben is előfordul, és a javításához explicit protokoll kell.

23.2.3 3. Prioritásöröklés (priority inheritance)

A legegyszerűbb és legelterjedtebb javítási minta a **prioritásöröklés**. Az alapötlet: ha egy magas prioritású task blokkolódik egy mutexre, amelyet egy alacsonyabb prioritású task birtokol, akkor a tulajdonos task **ideiglenesen megörökli** a blokkolt task prioritását. Ezáltal az alacsony prioritású task a kritikus szakasz befejezéséig a legmagasabb várakozó task prioritásával fut, tehát a közepes prioritású task nem tudja preemptálni.

A protokoll lépései:

1. Amikor egy P_h magas prioritású task megpróbál lefoglalni egy már foglalt M mutexet, először blokkolódik.
2. A kernel megvizsgálja az M mutex jelenlegi tulajdonosát, P_l-t.
3. Ha $p_l < p_h$, akkor a kernel P_l hatékony prioritását p_h-ra emeli.
4. Amikor P_l felszabadítja M-et, a hatékony prioritása visszaáll az eredeti értékre, és a kernel felébreszti a legmagasabb prioritású várakozó taskot.

A fenti háromtaskos példában a protokoll hatására P_3 akkor is P_1 prioritásával fut, amikor P_2 futásra kész lesz, ezért P_2 nem tudja preemptálni. Amint P_3 elengedi a mutexet, P_1 azonnal folytatódik, és P_2 csak ezután kaphat CPU-t.

A prioritásöröklés előnyei:

- Egyszerű implementálni: a mutex struktúrában tárolni kell a tulajdonost, és a blokkolásnál frissíteni kell a tulajdonos hatékony prioritását.
- Nem igényel előzetes tudást a taskok erőforrás-használatáról.
- A gyakorlati esetek többségében megszünteti a korlátlan inverziót.

Ugyanakkor a prioritásöröklésnek vannak korlátai. **Láncolt blokkolás** esetén – amikor egy task egyszerre több erőforrást vár, vagy a tulajdonos maga is blokkolódik egy másik erőforrásra – az öröklés láncban terjed, és a futási idő elemzése bonyolulttá válik. Ráadásul a prioritásöröklés nem akadályozza meg a **holtpontot**: ha két task különböző sorrendben foglal le két mutexet, a protokoll csupán a prioritásokat emeli, de a ciklikus várakozást nem oldja fel. Végül a többszörös blokkolásból adódó legrosszabb esetű várakozási időt nehéz felső korláttal megadni, ami biztonsági kritikus rendszereknél hátrány.

Az alábbi pszeudokód a mutex-lock művelet lényegét mutatja prioritásörökléssel. A kód csak szemléltetés, nem egy konkrét RTOS API-ja.

```
/* Pszeudokód: mutex lock prioritásörökléssel */
typedef struct {
    task_t *owner;
    uint32_t base_priority;
    uint32_t ceiling_priority;
    list_t   waiters;
} mutex_t;

void mutex_lock(mutex_t *m)
{
    task_t *self = current_task();

    if (m->owner == NULL) {
        m->owner = self;
        return;
    }

    /* A hívó blokkolódik; ha magasabb prioritású,
       a tulajdonos örökölje a hívó prioritását. */
    add_to_wait_list(&m->waiters, self);
    block_task(self);

    if (self->priority > m->owner->effective_priority) {
        m->owner->effective_priority = self->priority;
        reschedule();
    }
}

void mutex_unlock(mutex_t *m)
{
    task_t *self = current_task();

    if (!is_empty(&m->waiters)) {
        task_t *next = remove_highest_priority(&m->waiters);
        m->owner = next;
        unblock_task(next);
        /* A felszabadító visszakapja az eredeti prioritását,
           ha nincs további várakozó. */
        if (self->effective_priority != self->base_priority) {
```

```
            self->effective_priority = self->base_priority;
        }
        reschedule();
    } else {
        m->owner = NULL;
        if (self->effective_priority != self->base_priority) {
            self->effective_priority = self->base_priority;
            reschedule();
        }
    }
}
```

A fenti pszeudokód egyszerűsítéseket tartalmaz: nem kezeli a rekurzív mutexet, a prioritás-visszaállítás pontos időzítését, és a valós kernelben a blokkolás és az öröklés sorrendjét atomi módon, lockkal védve kell megvalósítani.

23.2.4 4. Prioritási plafon (priority ceiling)

A prioritási plafon protokoll más megközelítést alkalmaz. Előzetesen minden erőforráshoz hozzárendelünk egy **plafon-prioritást**: ez a legmagasabb prioritás azon taskok közül, amelyek valaha használhatják az adott erőforrást. Amikor egy task lefoglal egy erőforrást, a kernel azonnal a plafon szintjére emeli a task hatékony prioritását, függetlenül attól, hogy jelenleg van-e verseny az erőforrásért. Emiatt ezt a változatot **immediate ceiling** protokollnak is nevezik.

A szabályok:

- Egy task akkor foglalhat le egy erőforrást, ha a saját hatékony prioritása magasabb, mint az összes éppen más task által birtokolt erőforrás plafon-prioritása. Ez a feltétel megakadályozza a holtpontot, mert nem engedi, hogy egy task alacsonyabb prioritással belépjen egy olyan kritikus szakaszba, amelyet egy magasabb prioritású task később megszakíthat.
- A foglalás után a task hatékony prioritása az erőforrás plafonjára emelkedik.
- A felszabadításkor a hatékony prioritás visszaáll.

A **stack resource policy** (SRP) a ceiling protokoll egy kiterjesztése, amely lehetővé teszi, hogy a taskok közös veremből gazdálkodjanak, mert a plafon szabály garantálja, hogy egy task csak akkor futhat, ha a szükséges erőforrások szabadok. Ez jelentős memóriamegtakarítást hoz beágyazott rendszerekben.

A prioritási plafon előnye a prioritásörökléssel szemben:

- Megelőzi a holtpontot.
- A legrosszabb esetű blokkolási idő jól számítható: egy task legfeljebb egyszer blokkolódhat egy alacsonyabb prioritású task kritikus szakasza miatt.
- Az elemzés egyszerűbb, mert a blokkolási idő a plafonértékekből és a kritikus szakaszok hosszából közvetlenül adódik.

Hátránya, hogy előzetesen ismerni kell a taskok erőforrás-használatát, és a plafonértékek karbantartása a fejlesztés során extra feladat. Emellett bizonyos esetekben a plafon túl magasra emelheti a prioritást, ami felesleges preemptív korlátozást okozhat, bár ez a determinizmus szempontjából inkább konzervatív, mint hibás.

A háromtaskos példán a plafon protokoll a következőket eredményezi. Tegyük fel, hogy az M mutexet csak P_1 és P_3 használja, ezért a plafon $p_M = p_1$. Amikor P_3 lefoglalja M-et, azonnal p_1 prioritással fut, így a kritikus szakasz befejezéséig sem P_2, sem más task nem preemptálhatja. A

P_1 task csak akkor lép be a foglalási kísérletbe, amikor M szabad, ezért nem blokkolódik. A P_2 task futása késik ugyan, de ez a késés pontosan a kritikus szakasz hossza, és jól előre jelezhető.

23.2.5 5. RMS és EDF a gyakorlatban

A prioritásos ütemezés két legismertebb valós idejű politikája a **rate-monotonic scheduling** (RMS) és az **earliest deadline first** (EDF). Mindkettő preemptív, de eltérően osztja ki a prioritásokat.

RMS esetén a prioritás statikus, és a periódus fordított arányában állítjuk be: minél rövidebb a periódus, annál magasabb a prioritás. Ha $T_i < T_j$, akkor $p_i > p_j$. A módszer optimális a statikus prioritásos ütemezők között, vagyis ha egy taskhalmaz bármely statikus prioritásos ütemezővel ütemezhető, akkor RMS-szel is. Egy elegendő, de nem szükséges feltétel a CPU-kihasználtságra:

$$U = \sum_{i=1}^{n} \frac{C_i}{T_i} \leq n\left(2^{1/n} - 1\right).$$

Ez a korlát n növekedésével a $\ln 2 \approx 0{,}693$ értékhez tart. A gyakorlatban az RMS ennél jobb lehet, de a bizonyítható garancia így is erős.

EDF esetén a prioritás dinamikus: minden ütemezési pontban az a task kapja a legmagasabb prioritást, amelynek az abszolút határideje a legközelebbi. Az EDF optimális az összes preemptív ütemező között, és a szükséges és elégséges feltétel a CPU-kihasználtságra:

$$U \leq 1.$$

Ez azt jelenti, hogy az EDF akkor is tud határidőket tartani, amikor az RMS statikus prioritásai már nem elegendőek. Cserébe az EDF futási idejű overheadje valamivel nagyobb, mert a kernelnek minden ütemezésnél rendeznie vagy keresnie kell a legkorábbi határidőt.

A gyakorlati RTOS-ekben mindkét politika megtalálható. A FreeRTOS alapértelmezésben statikus prioritásos preemptív ütemezőt használ, és konfigurációtól függően támogat prioritásöröklést. A Zephyr RTOS támogat EDF-et is, valamint prioritásos és deadline-alapú policyket; a pontos konfigurációs nevek verzióról verzióra változnak, ezért az aktuális dokumentációt ellenőrizni kell. A fenti állítások a rendszerek általános tervezési elveire vonatkoznak, nem egy adott kiadás tesztelt viselkedésére.

Az RMS és az EDF közötti választásnál figyelembe kell venni a rendszerkövetelményeket. Ha a taskhalmaz statikus, a periódusok ismertek, és a CPU-kihasználtság mérsékelt, az RMS egyszerűbb és kiszámíthatóbb. Ha a rendszer dinamikus, változó határidőkkel, vagy a kihasználtság magas, az EDF jobb kihasználtságot adhat. Biztonsági kritikus rendszerekben gyakran az RMS-t választják a statikus elemzés egyszerűsége miatt, míg a rugalmasabb beágyazott rendszerekben az EDF vagy a kevert policyk terjednek.

Mindkét politika feltételezi, hogy a taskok függetlenek, vagy a közös erőforrásokhoz való hozzáférést megfelelő protokollal védik. Ha a prioritás-inverziót nem kezeljük, sem az RMS, sem az EDF elméleti garanciái nem érvényesek. Ezért az erőforrás-kezelés és az ütemezési politika együtt alkotja a valós idejű rendszer időbeli viselkedésének alapját.

23.2.6 6. Gyakorlati példa: három task, egy mutex

Vegyünk egy konkrét, kis méretű taskhalmazt, amelyen a fogalmak számokkal is ellenőrizhetők. A taskok periódusai és legrosszabb esetű CPU-igényei:

Task	Periódus T_i	CPU-idő C_i	Prioritás (RMS)
P_1	10 ms	2 ms	3 (legmagasabb)
P_2	20 ms	3 ms	2
P_3	50 ms	8 ms	1 (legalacsonyabb)

A CPU-kihasználtság:

$$U = \frac{2}{10} + \frac{3}{20} + \frac{8}{50} = 0{,}2 + 0{,}15 + 0{,}16 = 0{,}51.$$

Ez RMS alatt a $3 \cdot (2^{1/3} - 1) \approx 0{,}78$ korlát alatt van, tehát az RMS elegendő feltétele teljesül. Most tegyük fel, hogy P_1 és P_3 közösen használ egy erőforrást, amelyet M mutex véd. A P_3 kritikus szakasza legfeljebb 1 ms-ig tart. Ha semmilyen protokollt nem alkalmazunk, a legrosszabb esetű prioritás-inverziós várakozás a P_2 task teljes futtatási idejéből adódhat: mivel P_2 periódusa 20 ms és CPU-ideje 3 ms, P_1 akár 3 ms-ot is várhat, ami a 10 ms-os periódus arányában jelentős, és a határidő betartása veszélybe kerülhet.

Prioritásöröklléssel a P_3 task a kritikus szakasz idejére P_1 prioritását örökli, így a kritikus szakasz 1 ms-ja alatt P_2 nem futhat. A P_1 legrosszabb esetű blokkolási ideje $B_1 = 1$ ms. A válaszidő-analízishez használhatjuk a következő iteratív képletet:

$$R_i = C_i + B_i + \sum_{j \in hp(i)} \left\lceil \frac{R_i}{T_j} \right\rceil C_j.$$

Itt $hp(i)$ a P_i-nél magasabb prioritású taskok halmaza. P_1-re:

$$R_1 = 2 + 1 = 3 \text{ ms}.$$

A határidő 10 ms, tehát a feltétel teljesül. P_2-re:

$$R_2 = 3 + \left\lceil \frac{R_2}{10} \right\rceil \cdot 2.$$

Az iteráció $R_2^{(0)} = 3$, majd $R_2^{(1)} = 3 + \lceil 3/10 \rceil \cdot 2 = 5$, $R_2^{(2)} = 3 + \lceil 5/10 \rceil \cdot 2 = 5$. Tehát $R_2 = 5$ ms, ami 20 ms alatt van. P_3-ra is hasonlóan ellenőrizhető; a részletek a laborban elvégezhetők.

Priority ceiling protokollal a P_3 kritikus szakasza alatt a hatékony prioritás p_1, ezért a blokkolás ugyancsak legfeljebb 1 ms. A különbség akkor látszik, ha a P_3 több erőforrást is használ, vagy ha a taskok között ciklikus várakozás alakulhatna ki: a ceiling ezt előre kizárja.

23.2.7 7. Labor: prioritás-inverzió forgatókönyv és javítás

A labor célja, hogy az olvasó reprodukálható módon megfigyelje a prioritás-inverziót, majd a két javítási protokoll hatását. A környezet QEMU-ban futó RTOS szimulátor. A konkrét RTOS és verzió a laborban szabadon választható; a leírás általános pszeudokódot használ, amely FreeRTOS-szerű API-val rendelkezik. A futás előtt ellenőrizze az adott RTOS aktuális stabil kiadását és a konfigurációs neveket.

23.2.8 7.1 Kiinduló állapot

A munkakönyvtár tartalma:

```
rtos_priority_lab/
  main.c
  mutex.c
  mutex.h
  scheduler_config.h
  Makefile
  run_qemu.sh
  log/
```

A `scheduler_config.h` fájlban kapcsolhatók be a protokollok:

```
#define CONFIG_PRIORITY_INHERITANCE 0
#define CONFIG_PRIORITY_CEILING     0
```

A `main.c` három taskot hoz létre, amelyek a fenti példa paramétereit használják. A taskok futási ideje mesterséges terheléssel, például ciklusos várakozással állítható be; a mérési pontosság érdekében a terhelést CPU-ciklus-számlálóval vagy az RTOS rendszeridőzítőjével kell rögzíteni.

A laborhoz tartozó pszeudokód:

```
/* Pszeudokód: a három task és a közös mutex */
static mutex_t shared_mutex;

void high_task(void *arg)
{
    uint32_t start = now();
    mutex_lock(&shared_mutex);
    critical_section(1_ms);
    mutex_unlock(&shared_mutex);
    log("high_task response: %u ticks", now() - start);
}

void medium_task(void *arg)
{
    busy_wait(3_ms);
}

void low_task(void *arg)
{
    uint32_t start = now();
    mutex_lock(&shared_mutex);
    critical_section(1_ms);
    mutex_unlock(&shared_mutex);
    log("low_task critical section done: %u ticks", now() - start);
    busy_wait(5_ms);
}
```

A tényleges RTOS API-k ennél részletesebbek: a taskok létrehozása, prioritások megadása és a delay függvények az adott kernel szerint változnak. A fenti kód kizárólag a protokoll viselkedésének szemléltetésére szolgál.

23.2.9 7.2 Build és futtatás

A build parancs a választott RTOS-tól függ. Egy lehetséges, FreeRTOS-szerű esetben:

```
make clean
make CFLAGS="-DCONFIG_PRIORITY_INHERITANCE=0 -DCONFIG_PRIORITY_CEILING=0"
./run_qemu.sh > log/priority_inversion_case1.log 2>&1
```

A `run_qemu.sh` szkript QEMU-ban indítja el a rendszert, például egy Cortex-M3 processzort emulálva, 16 MiB RAM-mal és 10 MHz-es rendszeridőzítővel. A pontos QEMU paramétereket a szkript tartalmazza; a reprodukálhatóság érdekében a CPU-típust, a memóriaméretet és az RTOS verzióját a napló elején rögzíteni kell.

Futtatás után a naplóból kinyerhető a P_1 task válaszideje. A várt érték a protokoll nélküli esetben a fenti elemzés alapján legfeljebb $C_1 + C_2 + \text{kritikus szakasz} = 2 + 3 + 1 = 6$ ms lehet, feltéve hogy a P_2 task a teljes 3 ms-os futását a P_1 blokkolása alatt végzi. A mért értéket a hallgató a naplóból olvassa ki. A jegyzőkönyvben a **várt** és a **mért** értéket külön oszlopban kell feltüntetni, és a mérési módszert (pl. rendszeridőzítő tick, CPU-ciklus) is rögzíteni kell.

23.2.10 7.3 Javítás prioritásörökléssel

Állítsa a konfigurációt:

```
#define CONFIG_PRIORITY_INHERITANCE 1
#define CONFIG_PRIORITY_CEILING      0
```

Futtassa újra a tesztet:

```
make clean
make CFLAGS="-DCONFIG_PRIORITY_INHERITANCE=1 -DCONFIG_PRIORITY_CEILING=0"
./run_qemu.sh > log/priority_inheritance_case2.log 2>&1
```

A várt válaszidő P_1-re most $C_1 + B_1 = 2 + 1 = 3$ ms. A mérés során azt várjuk, hogy a naplóban a P_1 válaszideje jelentősen csökken, és a közepes prioritású task nem szakítja meg a kritikus szakaszt.

23.2.11 7.4 Javítás prioritási plafonnal

Állítsa a konfigurációt:

```
#define CONFIG_PRIORITY_INHERITANCE 0
#define CONFIG_PRIORITY_CEILING      1
```

Futtassa újra a tesztet:

```
make clean
make CFLAGS="-DCONFIG_PRIORITY_INHERITANCE=0 -DCONFIG_PRIORITY_CEILING=1"
./run_qemu.sh > log/priority_ceiling_case3.log 2>&1
```

A várt eredmény ebben az esetben is $R_1 = 3$ ms. A különbség a naplóban annyiban jelentkezhet, hogy a P_3 task a kritikus szakaszba lépéskor azonnal magasabb prioritással fut, még akkor is, ha nincs tényleges verseny. A mérési naplóban a taskok prioritásváltozásait is érdemes naplózni.

23.2.12 7.5 Mérési napló és kiértékelés

A labor jegyzőkönyvében legalább a következőket kell feltüntetni:

- Emulált hardver: CPU, órajel, memória, RTOS-verzió.
- Build konfiguráció: fordító verzió, optimalizálási szint, protokollkapcsolók.
- Mindhárom esetben a P_1 válaszidő **várt** értéke és **mért** értéke, tickben és milliszekundumban.
- A mérési módszer: milyen időforrást használt, hány ismétlést végzett.
- A naplófájl elérési útja.

A mérés során a ciklusszám, a laphibák és a cache-viselkedés is értelmezendő, ha a platform támogatja ezek mérését; egyetlen válaszidőszám önmagában nem elegendő a teljes képhez. A valós idejű méréseknél a legrosszabb eset (WCET), a jitter és a megszakítási késleltetés külön is rögzítendő.

A labor elvárása, hogy a hallgató a három eset közötti különbséget a naplóalapú bizonyítékkal magyarázza, és az eredményt összevesse a fejezetben levezetett képletekkel. A visszaállítás a verziókezelővel vagy az eredeti konfiguráció visszamásolásával történik.

23.2.13 8. Gyakori hibák és helyes megoldások

A prioritás-inverzió kezelése során több tipikus hiba fordul elő.

1. **Protokoll nélküli mutexhasználat.** A fejlesztő azt feltételezi, hogy a rövid kritikus szakasz nem okozhat gondot. A fenti példa megmutatja, hogy a közepes prioritású task tetszőleges hosszúságúra nyújthatja a várakozást. A helyes megoldás legalább prioritásöröklés, vagy ha a holtpont is lehetséges, ceiling.

2. **A prioritásöröklés félreértelmezése.** Egyesek azt gondolják, hogy a prioritásöröklés megszünteti a holtpontot. Ez nem igaz; a holtpontot külön protokollal vagy erőforrássorrendi szabályokkal kell kezelni.

3. **A plafonértékek helytelen beállítása.** Ha a plafon nem a legmagasabb használó prioritás, a protokoll nem adja meg a kívánt garanciát. A rendszertervben minden erőforráshoz kötelező dokumentálni a használó taskokat és a plafont.

4. **A válaszidő-elemzés elhanyagolása.** A fejlesztő csak a CPU-kihasználtságot számolja, de a blokkolási időt nem. A helyes megoldás a válaszidő-analízis elvégzése a blokkolási taggal együtt.

5. **Nem mérik a tényleges futási időket.** A szimulációban kapott értékek eltérhetnek a valós hardvertől. A laborban ezért kötelező a hardverkonfiguráció és a mérési módszer rögzítése, a mért és várt értékek elkülönítése.

23.2.14 9. Önellenőrző kérdés

Adott három periodikus task: P_1 ($T_1 = 10$ ms, $C_1 = 2$ ms), P_2 ($T_2 = 20$ ms, $C_2 = 4$ ms), P_3 ($T_3 = 40$ ms, $C_3 = 10$ ms). P_1 és P_3 közös erőforrást használ egy 2 ms hosszú kritikus szakasszal. Számítsa ki a P_1 legrosszabb esetű válaszidejét prioritásörökléssel és ceiling protokollal, majd ellenőrizze, hogy a taskhalmaz RMS szerint ütemezhető-e. Válaszát indokolja.

23.2.15 10. Összegzés

Ebben a fejezetben bemutattuk a prioritásos preemptív ütemezés alapjait, a prioritás-inverzió jelenségét és két alapvető javítási protokollját. A prioritásöröklés egyszerűbb és széles körben elérhető, de nem kezeli a holtpontot. A prioritási plafon erősebb garanciákat ad, cserébe előzetes tervezést igényel. Az RMS és az EDF gyakorlati alkalmazásához elengedhetetlen, hogy a közös erőforrások kezelése ne sértse az ütemezhetőségi feltételeket. A következő fejezet a legrosszabb esetű végrehajtási idő (WCET), a megszakítási késleltetés és az erőforrás-elemzés eszközeivel mélyíti tovább a valós idejű rendszerek időbeli analízisét.

23.2.16 Tájékozódási pontok

Az itt tárgyalt protokollok általános leírása megtalálható a valós idejű rendszerek szakirodalmában, például a klasszikus Liu és Layland cikkben (`https://doi.org/10.1145/321738.321743`). A FreeRTOS és a Zephyr aktuális dokumentációja az adott implementáció konfigurációjához nyújt segítséget; a pontos API-neveket és képességeket mindig a felhasznált verzió dokumentációjában ellenőrizze. A fejezetben szereplő kódok pszeudokódok, nem egy konkrét RTOS-ból származó, tesztelt programrészletek.

23.3 WCET, megszakítási késleltetés és erőforrás-elemzés

A valós idejű rendszer egyik legfontosabb tervezési kérdése, hogy a futó feladatok minden esetben a határidejükön belül elvégezzék a munkájukat. Az átlagos végrehajtási idő ehhez nem elegendő, mert a rendszer időnként a legkedvezőtlenebb útvonalon fut, és ez az a pont, ahol a határidő-túllépés először bekövetkezik. Ebben a fejezetben a legrosszabb végrehajtási idő (WCET), a megszakítási késleltetés és a veremszükséglet elemzését, valamint az időzítési zárolás módszerét tárgyaljuk.

23.3.1 1. Legrosszabb végrehajtási idő és rendszerterhelés

Egy P_i feladat végrehajtási ideje nem konstans. A bemeneti adatoktól, a gyorsítótár állapotától, az elágazásoktól és a megszakításoktól függően az egyes futások ideje szóródik. A valós idejű elemzéshez három értéket szokás megkülönböztetni:

- **BCET** (best-case execution time): a lehető legrövidebb futási idő.
- **ACET** (average-case execution time): a mérésekből számolt átlagos futási idő.
- **WCET** (worst-case execution time): a lehető leghosszabb futási idő, amely a feladat összes lehetséges végrehajtása közül előfordulhat.

A WCET tehát egy felső korlát. Ha a feladat tényleges futási ideje minden esetben legfeljebb C_i, akkor az ütemezési analízis biztonságos módon használhatja a C_i értéket. A rendszerterhelést az egyszerű periodikus feladatmodellben az

$$U = \sum_{i=1}^{n} \frac{C_i}{T_i}$$

összefüggéssel számítjuk, ahol T_i a P_i feladat periódusideje, C_i pedig a WCET. Egyprocesszoros környezetben a $U \leq 1$ feltétel szükséges a megvalósíthatósághoz, de nem elégséges: az ütemező algoritmus és a határidők is befolyásolják, hogy a feladatkészlet valóban ütemezhető-e. A

következő fejezet az ütemezési tesztekkel foglalkozik, ezért itt csak annyit rögzítünk, hogy a rendszerterhelés-számítás megbízhatósága közvetlenül a C_i becslés megbízhatóságától függ.

Az átlagos futási idő használata tipikus hiba. Ha egy motorvezérlő feladat átlagosan 120 µs alatt fut, de ritkán, például egy hibakezelési ágban 280 µs-ig is eltarthat, akkor az átlag alapján számolt terhelés hamisan alacsony. A határidő-túllépés éppen a ritka, hosszú ágban következik be. A WCET-elemzés célja ezért a farok-késleltetés biztonságos felső korlátjának meghatározása.

23.3.2 2. WCET-becslési módszerek

A WCET meghatározására három alapvető megközelítés létezik:

1. **Statikus analízis**: a forráskód vagy a bináris vezérlési gráfjának elemzése a hardvermodell figyelembevételével.
2. **Mérés-alapú becslés**: a feladat végrehajtása sok, különböző bemenettel, majd a mért idők statisztikai kiértékelése.
3. **Hibrid megközelítés**: mérésekkel kalibrált statikus elemzés, vagy statikus útvonalelemzéssel kiegészített mérés.

A statikus analízis lépései egy egyszerű példán jól bemutathatók. Tekintsük a következő pszeudokódot:

```
/* pszeudokód: ciklikus feladat */
uint32_t fir_filter(uint32_t x) {
    uint32_t acc = 0;
    for (int i = 0; i < N; i++) {
        acc += coeff[i] * x & 0xFFFF;
    }
    return acc >> 8;
}

void task_control(void) {
    uint32_t x = read_sensor();
    uint32_t y = fir_filter(x);
    if (y > THRESHOLD) {
        actuator_write(y);
    } else {
        actuator_write(0);
    }
}
```

A `task_control` vezérlési gráfja a következő alapblokkokból áll:

- A: `read_sensor` hívása.
- B: `fir_filter` előkészítése és a ciklus belépése.
- C: a ciklusmag egy iterációja.
- D: a ciklus kilépése és a visszatérés.
- E: az `if` feltétel kiértékelése.
- F: az igaz ág (`actuator_write(y)`).
- G: a hamis ág (`actuator_write(0)`).

A statikus elemzéshez minden alapblokkhoz felső időkorlátot rendelünk. Tegyük fel, hogy a blokkok legrosszabb végrehajtási ideje a következő:

Alapblokk	Leírás	Idő (µs)
A	szenzorolvasás	15

Alapblokk	Leírás	Idő (µs)
B	ciklus inicializálás	3
C	ciklusmag egy iteráció	2
D	ciklus kilépés	2
E	feltétel	1
F	igaz ág	10
G	hamis ág	4

A ciklus iterációszáma konstans $N = 12$. A `fir_filter` blokk WCET-je:

$$C_{\text{fir}} = B + N \cdot C + D = 3 + 12 \cdot 2 + 2 = 29 \text{ µs}.$$

A `task_control` teljes WCET-je a hosszabb ágat választva:

$$C_{\text{task}} = A + C_{\text{fir}} + E + \max(F, G) = 15 + 29 + 1 + 10 = 55 \text{ µs}.$$

Ez a számítás feltételezi, hogy a blokkok időkorlátjai ismertek. Valós processzoron ezek a korlátok a folyamatarchitektúrától függenek: a pipeline, az elágazásbecslés, a gyorsítótár és a memória-várakozások mind növelhetik az egyes blokkok idejét. A statikus elemző eszközök ezért a CPU-időzítési modelljét is tartalmazzák. Az egyszerű mikrokontrollereken (például ARM Cortex-M3) a gyorsítótár nélküli, determinisztikus végrehajtás miatt a blokkok ciklusszinten számolhatók, de a flash-memória várakozási állapotait itt is figyelembe kell venni.

A mérés-alapú becslés során a feladatot a céleszközön vagy emulált környezetben futtatjuk, és időbélyeggel mérjük a futás hosszát. A mérésekből hisztogramot készítünk. A gond az, hogy a bemeneti tér ritkán teljesen lefedhető, ezért a mért maximum nem feltétlenül éri el a valódi WCET-et. A biztonságos mérés-alapú becslésnél a mért maximumot szakértői ráhagyással vagy valószínűségi konfidenciával egészítjük ki. Egy gyakori szabály, hogy a becsült WCET legyen a mért maximum legalább 1,1–1,3-szorosa, ha a minta nem teljes, de ez csak irányszám, nem helyettesíti a statikus elemzést.

A hibrid megközelítés a statikus útvonalelemzést használja a maximális útvonal megtalálására, majd az egyes alapblokkokat méréssel kalibrálja. Így a ritka útvonalak is bekerülnek az elemzésbe, a hardvermodell hibáját pedig a mérés csökkenti.

23.3.3 3. Megszakítási késleltetés

A megszakítási késleltetés (interrupt latency) az az idő, amely a megszakítási kérelem hardveres megjelenésétől a megszakítás-kiszolgáló rutin (ISR) első utasításának megkezdéséig eltelik. A válaszidő (response time) ennél tágabb fogalom: magában foglalja az ISR végrehajtását és az ütemező által elindított feladat futását is. A valós idejű rendszerben az interrupt latency a legrosszabb esetben is korlátos kell hogy legyen.

A késleltetés összetevői:

- **Hardveres késleltetés**: a megszakításkérő periféria és a megszakításvezérlő közötti terjedés, a prioritás döntése.
- **Megszakítás-tiltott ablak**: az az idő, amíg a CPU a megszakításokat letiltva tartja, például kritikus szakaszokban.

- **ISR-belépési idő**: a CPU automatikus állapotmentése (regiszterek, visszatérési cím) és a vektortábla olvasása.
- **Szoftveres belépés**: a C ISR függvény bevezető kódja, amely a további regisztereket menti.

Az alábbi táblázat egy ARM Cortex-M3 alapú, QEMU-ban emulált rendszer tipikus, tervezési célként használt értékeit mutatja. Az értékek nem mérési eredmények, hanem a gyártói dokumentáció és az elemzés alapján becsült felső korlátok; a tényleges mérés eltérhet.

Összetevő	Idő (µs)
Hardveres késleltetés	0,1
Megszakítás-tiltott ablak (maximális kritikus szakasz)	3,0
ISR belépés (hardveres mentés + vektortábla)	0,5
Szoftveres belépés és regisztermentés	1,0
Összesen (legrosszabb eset)	**4,6**

A megszakítási késleltetés mérésére az egyik legegyszerűbb módszer, ha egy GPIO kimenetet a szoftver a megszakítási kérelem pillanatában magasra állít, az ISR első utasításában pedig alacsonyra. A jelszélességet logikai analizátorral vagy a CPU ciklusszámlálójával mérjük. Emulált környezetben a GPIO port írása is időt vesz igénybe, ezért a mérést az ISR belépési pontján timestamp-értékkel végezzük.

Az alábbi pszeudokód a késleltetés mérését mutatja:

```
/* pszeudokód: interrupt latency mérés */
volatile uint32_t *timestamp_reg = (uint32_t *)0x40000000;
volatile uint32_t *gpio_set      = (uint32_t *)0x40020000;
volatile uint32_t *gpio_clear    = (uint32_t *)0x40020004;

void trigger_interrupt(void) {
    *gpio_set = 1;            /* trigger jel */
    *timestamp_reg = 0;       /* timestamp nullázása */
    asm volatile ("svc #0");  /* példa: szoftver megszakítás */
}

void isr_handler(void) {
    *gpio_clear = 1;
    uint32_t t = *timestamp_reg;
    /* t értéke a késleltetés ciklusban */
}
```

A mérést elegendően sokszor megismételve a maximális értéket keressük. A legrosszabb esetet gyakran a leghosszabb megszakítás-tiltott kritikus szakasz okozza, ezért a kódban a kritikus szakaszok hosszát is ismerni kell.

23.3.4 4. Stack usage

A veremszükséglet elemzése azért kritikus, mert a verem túlcsordulása nem determinisztikus hibához, hanem memóriasértéshez vezet, amely a rendszer bármely pontján jelentkezhet. A valós idejű rendszerben a legrosszabb veremmélységet kell meghatározni minden feladathoz és az ISR-ekhez.

A veremhasználat forrásai:

- A hívási lánc minden függvényének helyi változói.

- A megszakítási kontextusban mentett regiszterek.
- A fordító által generált ideiglenes értékek.
- A könyvtári függvények belső keretei.

A veremmélység meghatározásának három fő módszere:

1. **Festékes mérés (stack painting)**: a verem területét ismert mintázattal (például `0xDEADBEEF`) töltjük fel, majd a feladat futtatása után megkeressük a legmagasabb című felülírt bájtot. Ez a módszer a tényleges futás során elért maximális mélységet méri, de nem feltétlenül éri el a legrosszabb esetet, ha a ritka útvonal nem futott le.
2. **Statikus analízis**: a hívási gráf alapján minden függvény keretméretét összegezzük a leghosszabb útvonalon. A függvények rekurziója esetén a rekurzió mélységét is korlátozni kell.
3. **MPU/guard page védelem**: a hardveres memóriavédelmi egység a verem alján tiltott régiót helyez el, így a túlcsordulás azonnal megszakítást generál. Ez nem helyettesíti az elemzést, de megakadályozza a csendes memóriakorrupciót.

Az alábbi példa egy hívási lánc statikus elemzését mutatja:

```
main -> task_control -> fir_filter
```

A keretméretek:

Függvény	Keretméret (bájt)
`main`	32
`task_control`	48
`fir_filter`	16
Összesen	**96**

Ha a megszakítási kontextus a feladat veremét használja, akkor az ISR keretét és a hardveres mentést is hozzá kell adni. A gyakorlatban a feladatok veremét elkülönített területre helyezzük, és minden feladathoz a legrosszabb hívási lánc szerinti méretet plusz egy biztonsági ráhagyást rendelünk.

23.3.5 5. Időzítési zárolás és analízis

Az időzítési zárolás (timing closure) azt a módszertani lépést jelenti, amikor a rendszer időzítési modelljének elemeit – a WCET-becsléseket, a megszakítási késleltetést, a veremkorlátokat és a rendszerterhelést – egyetlen, konzisztens, dokumentált állapotba zárjuk. A cél az, hogy a tervezési és mérési eredmények között ne legyen ellentmondás, és minden követelmény számszerűen ellenőrizhető legyen.

Az időzítési zárolás lépései:

1. **Feltételrögzítés**: rögzítjük a hardverkonfigurációt (CPU típus, órajel, memória, gyorsítótár, perifériák), a fordító beállításait, az ütemező konfigurációt és a megszakítási prioritásokat.
2. **Korlátok összegyűjtése**: minden feladathoz, ISR-hez és kritikus szakaszhoz felvesszük a WCET-et, a veremmélységet és a késleltetési korlátot.
3. **Terhelés-ellenőrzés**: kiszámítjuk a rendszerterhelést és az ütemezési teszteket a becsült WCET-ekkel.

4. **Mérési ellenőrzés**: a mért értékeket összevetjük a becsült korlátokkal. Ha a mért érték meghaladja a becsültet, a modell hibás, és újra kell nyitni az elemzést.
5. **Lezárás**: ha minden ellenőrzés sikeres, a modellt lezárjuk, és a dokumentációban rögzítjük az érvényességi feltételeket.

Az időzítési zárolás akkor válik igazán fontossá, amikor a rendszer több csapata párhuzamosan fejleszt, vagy amikor egy későbbi változtatás hatását kell értékelni. A lezárt modell biztosítja, hogy a határidő-számítás nem légből kapott átlagokra, hanem dokumentált, konzisztens korlátokra épül.

23.3.6 6. Gyakorlati példa és tipikus hibák

Tekintsünk egy kis rendszert három periodikus feladattal:

Feladat	Periódus T_i (ms)	WCET C_i (µs)	Prioritás
P_1	1	220	magas
P_2	5	800	közepes
P_3	20	4500	alacsony

A rendszerterhelés:

$$U = \frac{220}{1000} + \frac{800}{5000} + \frac{4500}{20000} = 0{,}22 + 0{,}16 + 0{,}225 = 0{,}605.$$

Ez egyprocesszoros környezetben látszólag kényelmesen elfér, de a részletes ütemezési analízis nélkül nem mondhatjuk ki a megvalósíthatóságot. A következő hibák gyakoriak:

- **Átlagos futási idő használata**: ha P_3 átlagos ideje 3500 µs, de a legrosszabb ág 4500 µs, az átlaggal számított U alacsonyabb, és a határidő-túllépés a ritka ágban következik be.
- **Megszakítási késleltetés elhanyagolása**: ha a megszakítás-tiltott ablak a P_1 feladat felébresztését késlelteti, a magas prioritású feladat válaszideje megnő, és a határidő-túllépés a látszólag jó terhelés ellenére bekövetkezik.
- **Veremtúlcsordulás figyelmen kívül hagyása**: ha a P_3 feladat egy ritka hívási láncban nagyobb veremmélységet ér el, mint amit a festékes mérés látott, a memóriakorrupció a rendszer nem determinisztikus összeomlását okozza.
- **Hiányos mérési terv**: ha a mérés csak a tipikus bemeneteket fedi le, a maximális útvonal nem jelenik meg a mintában, és a becsült WCET alulmarad.

A helyes megoldás az időzítési zárolás: a WCET-et statikus analízissel és méréssel is meghatározzuk, a megszakítási késleltetést a leghosszabb kritikus szakaszból számítjuk, a veremmélységet statikusan és festéssel is ellenőrizzük, majd az összes korlátot egy konzisztens modellben zárjuk le.

23.3.7 7. Labor: WCET-becslési és mérési terv

23.3.8 7.1. Cél

Készítsen WCET-becslési és mérési tervet egy egyszerű, emulált feladat számára. A tervben válassza el a várt (levezetett) és a mért eredményt, és rögzítse a reprodukálhatóság feltételeit.

23.3.9 7.2. Kiinduló állapot

- Hoszt: x86_64 Linux, QEMU 8.2 vagy újabb telepítve.
- Cél: QEMU által emulált ARM Cortex-M3 (LM3S6965EVB), órajel 50 MHz.
- Fordító: `arm-none-eabi-gcc` 12.3, `-O2 -mcpu=cortex-m3 -mthumb`.
- A minta feladat: `task_control` a fenti `fir_filter` hívással, $N = 12$.
- A labor csak emulált környezetben fut, a hoszt lemezét nem módosítja.

23.3.10 7.3. Build parancs

```
arm-none-eabi-gcc -O2 -mcpu=cortex-m3 -mthumb -T linker.ld \
  -o wcet_lab.elf main.c
```

A `linker.ld` a flash-t 0x00000000-ra, a RAM-ot 0x20000000-ra helyezi. A build kimenetét a `wcet_lab.elf` tartalmazza.

23.3.11 7.4. Futtatási parancs

```
qemu-system-arm -M lm3s6965evb -nographic -kernel wcet_lab.elf
```

A méréshez a QEMU `-d in_asm` nyomkövetési módját vagy a SysTick ciklusszámlálót használjuk. A feladat futási idejét a belépés és kilépés között mért ciklusszámból számítjuk, 50 MHz órajellel osztva.

23.3.12 7.5. Várt (levezetett) eredmény

A statikus blokkidő-modell alapján a `task_control` WCET-je 55 µs. A becslés feltételezi, hogy a QEMU által emulált Cortex-M3-on a memória-hozzáférés várakozás nélküli, a szenzorolvasás és az aktuátorírás időkorlátját 15 µs, illetve 10 µs értékben rögzítettük, és a `fir_filter` ciklusa pontosan 12 iterációt futtat.

23.3.13 7.6. Mérési terv

1. Futtassuk a feladatot 10 000 alkalommal, minden futásban véletlenszerű, de determinisztikus álvéletlen bemenettel.
2. Minden futásnál rögzítsük a belépési és kilépési timestamp értékét.
3. Készítsünk hisztogramot a futási időkről 1 µs-os felbontással.
4. Jegyezzük fel a mért minimumot, maximumot, átlagot és a 99,9%-os percentilist.
5. Hasonlítsuk össze a mért maximumot a várt 55 µs értékkel.

A mérési naplóban minden futtatási paramétert rögzíteni kell: QEMU verzió, `-d` kapcsolók, GCC verzió, órajel, bemenetgenerátor seedje, mintaszám. A mért maximum akkor fogadható el biztonságos becslésként, ha kisebb vagy egyenlő a vártnál; ha nagyobb, a statikus modellt újra kell nyitni.

23.3.14 7.7. Visszaállítás

A QEMU folyamat megszakítása (`Ctrl-A X`) után az emulált gép megszűnik, a hoszt fájlrendszere változatlan marad. A mérési adatokat a laboron kívüli munkakönyvtárban tároljuk.

23.3.15 8. Összefoglalás

A WCET, a megszakítási késleltetés és a veremszükséglet elemzése együttesen adja a valós idejű rendszer időzítési alapját. A WCET a feladat leghosszabb végrehajtási útjának felső korlátja, és nem az átlagos futási idő. A megszakítási késleltetés a hardveres és szoftveres összetevőkből származó felső korlát, amely a kritikus szakaszok hosszától és a megszakítás-tiltott ablakoktól függ. A veremszükséglet elemzés a memóriasértések megelőzését szolgálja. Az időzítési zárolás ezeket a korlátokat egy konzisztens, dokumentált modellbe zárja, amelyet méréssel ellenőrzünk.

Önellenőrző kérdések:

1. Miért nem elegendő az átlagos futási idő a rendszerterhelés számításához?
2. Mely összetevők alkotják a megszakítási késleltetést?
3. Hogyan ellenőrzi a festékes mérés a veremmélységet, és mi a korlátja?
4. Mi a szerepe az időzítési zárolásnak a WCET-becslés és a mérési eredmény összevetésében?

23.4 Biztonsági és minősítési követelmények

A biztonságkritikus rendszerek tanúsítása nem egyszerűen egy dokumentumcsomag elkészítése; olyan mérnöki fegyelem, amely a követelménytől a futtatott kódig igazolható láncot hoz létre, és független szemlélő számára is bizonyítja, hogy a rendszer a meghatározott kockázatot a megengedett szintre csökkenti. A WCET, megszakítási késleltetés és erőforrás-elemzés alfejezetben már láttuk a determinizmus, a WCET és a megszakítási késleltetés mérési módszereit. Most arra fókuszálunk, hogyan lehet ezeket a méréseket és más bizonyítékokat úgy rendszerezni, hogy az egy adott biztonsági (safety) szabvány szerinti tanúsításban megállja a helyét.

A fejezetben a „biztonság” szót a *safety* értelemben használjuk: a rendszer nem okoz elfogadhatatlan kárt emberben, környezetben vagy vagyonban. Ez eltér a *security* fogalmától, amely a szándékos támadások elleni védelmet jelenti. A kettő sokszor összefügg, de a tanúsítási követelmények és a bizonyítékok eltérők. A fejezetben bemutatjuk a három legfontosabb keretrendszert: az IEC 61508-at, az ISO 26262-t és a DO-178C-t. Mindegyik ugyanarra az alapkérdésre ad választ: honnan tudjuk, hogy a szoftver és a hardver együttesen elvégzi a biztonsági feladatát akkor is, ha valamelyik összetevő meghibásodik?

23.4.1 1. Alapfogalmak: veszély, kockázat, biztonsági funkció és integritás

A biztonságkritikus rendszerek tanúsításának közös nyelve a kockázatmodell. A kiindulópont a **veszély** (hazard): olyan fizikai állapot vagy esemény, amely kárt okozhat. Például egy vegyi reaktor túlnyomása, egy jármű fékezés nélküli gurulása vagy egy repülőgép futóművének beragadása. A veszély súlyossága önmagában nem határozza meg a beavatkozás szigorúságát; ehhez a bekövetkezés valószínűségét és a kitettséget is figyelembe kell venni.

Az egyszerűsített kockázatmodell:

$$\text{kockázat} \approx \text{súlyosság} \times \text{kitettség} \times \text{valószínűség}$$

Ez a képlet nem pontos mennyiségi számítás, hanem kvalitatív rangsorolásra alkalmas. A magas súlyosságú, gyakori kitettségű és valószínű veszélyekhez szigorúbb biztonsági követelmények tartoznak. A szabványok ezt a rangsort különböző **biztonsági integritási szintekre** képezik le.

- Az IEC 61508-ban a **SIL** (Safety Integrity Level) 1-től 4-ig terjed; a SIL 4 a legszigorúbb. A SIL a biztonsági funkció megbízhatóságára vonatkozó célt adja meg, például a veszélyes meghibásodás óránkénti valószínűségét.
- Az ISO 26262 az autóipari rendszerekre **ASIL**-t (Automotive Safety Integrity Level) határoz meg: QM (quality management, nincs safety követelmény), A, B, C, D. Az ASIL D a legszigorúbb.
- A DO-178C a fedélzeti szoftverekre **DAL**-t (Design Assurance Level) vagy más néven szoftverszintet határoz meg A-tól E-ig. Az A szint a katasztrofális, az E „nincs hatás” kategória.

A szintek nem a szoftver „jóságát” mérik, hanem azt, hogy mennyi és milyen mélységű bizonyítékra van szükség a megfelelőség igazolásához. Ugyanaz a funkció ASIL A szinten kevesebb, ASIL D szinten lényegesen több elemzést és tesztet igényel.

A **biztonsági funkció** (safety function) az a rendszer által nyújtott működés, amely a veszély bekövetkezését megelőzi, vagy a kár mértékét csökkenti. Például egy hőmérséklet-felügyeleti rendszerben a biztonsági funkció: „ha a hőmérséklet meghaladja a 80 °C-ot, akkor a fűtőkört legfeljebb 100 ms alatt lekapcsolni.” Ez a követelmény három dolgot rögzít:

1. a kiinduló feltételt,
2. a beavatkozást,
3. az időbeli korlátot.

A biztonsági funkciót el kell különíteni a normál vezérlési funkciótól. A fűtésszabályozás normál funkció lehet, de a túlmelegedés elleni lekapcsolás biztonsági funkció. Ha a normál kód hibája elviheti a biztonsági kódot is, akkor a rendszer nem felel meg a függetlenség elvének.

23.4.2 2. Tanúsítási keretrendszerek: IEC 61508, ISO 26262, DO-178C

A három szabvány közös célja, hogy a fejlesztő a teljes életciklus során igazolható módon dolgozzon. Mindegyik megköveteli a követelmények nyomon követhetőségét a veszélyelemzéstől a tesztjegyzőkönyvig, és mindegyik előír valamilyen független ellenőrzést.

23.4.3 2.1 IEC 61508: általános funkcionális biztonság

Az IEC 61508 az általános, iparágfüggetlen keretrendszer. Elsődleges fogalma a **biztonsági életciklus** (safety lifecycle), amely a veszélyelemzéstől a leállításig követi a rendszert. A legfontosabb állomások:

- veszély- és kockázatelemzés,
- biztonsági funkciók meghatározása,
- SIL-hozzárendelés,
- biztonsági követelmények specifikációja,
- tervezés és megvalósítás,
- verifikáció és validáció,
- üzemeltetés, karbantartás és módosítás.

A szabvány a szoftverre vonatkozóan megköveteli a szoftver-biztonsági életciklust is, amely párhuzamos a hardverrel. A V-modell a specifikáció és a tesztelés összekapcsolását mutatja: a bal oldalon a követelmények és a tervek, a jobb oldalon a megfelelő verifikációs szintek. A modul tesztek a részletes tervezéshez, az integrációs tesztek az architekturális tervezéshez, a rendszertesztek a rendszerkövetelményekhez kapcsolódnak.

A szoftverre vonatkozó követelmények SIL 1–4 szerint növekednek. Alacsonyabb szinteken elegendő a strukturált tervezés és a funkcionális teszt; magasabb szinteken elvárás a statikus elemzés, a lefedettségi mérés, a szigorú konfigurációkezelés és a független felülvizsgálat.

23.4.4 2.2 ISO 26262: közúti járművek

Az ISO 26262 az IEC 61508-ból eredeztethető, de az autóipari életciklusra és a sorozatgyártásra szabott. A biztonsági szintet az **ASIL** fejezi ki. A szilárdságot a következő három tényező alapján határozzák meg:

- súlyosság (S0–S3),
- kitettség (E0–E4),
- irányíthatóság (C0–C3).

Az irányíthatóság azt méri, hogy a vezető vagy a környezet mennyire képes a hibát elhárítani. Egy erős, jól irányítható helyzet alacsonyabb ASIL-t eredményez.

A fejlesztésben kulcsszerepet kap a **biztonsági terv** (safety plan), a **biztonsági eset** (safety case) és a **megerősítő intézkedések** (confirmation measures): biztonsági audit, biztonsági értékelés és független felülvizsgálat. Az ASIL D rendszerben például a szoftvermodulokra megkövetelhetik az MC/DC lefedettséget, a diagnosztikai lefedettséget, a redundáns adatellenőrzést és a memóriavédelem igazolását.

Az ISO 26262 bevezeti a **biztonsági elemzés** módszereit is: a FMEA (Failure Mode and Effects Analysis) és az FTA (Fault Tree Analysis) segítségével az egyes hibák hatását a biztonsági célokra le lehet képezni.

23.4.5 2.3 DO-178C: fedélzeti szoftver

A DO-178C (és kiegészítői, például a DO-330 szoftvereszközök minősítéséről) a repülésben elfogadott szoftverminősítési szabvány. A biztonsági szinteket a **szoftverszint** (DAL) jelöli A–E-ig:

- A: katasztrofális hiba,
- B: veszélyes hiba,
- C: nagy következményű hiba,
- D: kisebb hiba,
- E: nincs hatás.

A szoftverszint az adott funkció meghibásodásának repülőgépre gyakorolt hatásától függ. A DO-178C a „célok” (objectives) rendszerével dolgozik: minden életciklus-szakaszhoz előírja, milyen tevékenységeket kell elvégezni, és milyen bizonyítékot kell előállítani. A magasabb szinteken több cél és szigorúbb ellenőrzés tartozik.

A verifikációban a DO-178C a kódfedettségi kritériumokat is előírja:

- A szinten: MC/DC (Modified Condition/Decision Coverage),
- B szinten: döntési lefedettség,
- C szinten: utasításlefedettség.

Az MC/DC azt igazolja, hogy a döntésben szereplő minden feltétel önállóan befolyásolja a kimenetet. Ez különösen fontos a biztonsági feltételekben, például a hőmérséklet-határérték és a redundáns szenzor együttes kiértékelésénél.

A három keretrendszer összehasonlítása:

Szempont	IEC 61508	ISO 26262	DO-178C
Hatókör	iparágfüggetlen	közúti járművek	repülés (fedélzet)
Biztonsági szint	SIL 1–4	QM, ASIL A–D	DAL A–E
Életciklus fókusz	teljes biztonsági életciklus	autóipari termékfejlesztés	szoftveréletciklus a repülésben
Kódfedettség	SIL-től függő	ASIL-tól függő	utasítás/döntés/MC/DC
Független ellenőrzés	funkcionális biztonsági értékelés	safety audit/assessment	SOI auditok

A szabványok pontos kiadása a tervezés időpontjában ellenőrizendő. A jelen könyv írásakor a hivatkozott verziók: IEC 61508:2010, ISO 26262:2018, DO-178C:2011. Az újabb módosításokat és értelmezéseket a tanúsítás előtt egyeztetni kell az illetékes tanúsító szervezettel.

23.4.6 3. Bizonyíték, traceability és független ellenőrzés

A tanúsítás nem a kód minőségének szubjektív dicsérete, hanem bizonyítékok rendezett halmaza. A bizonyíték típusai:

- **Tesztjegyzőkönyv:** bemenet, elvárt kimenet, tényleges kimenet, környezet, verzió.
- **Statikus elemzési jelentés:** például MISRA C szabályok ellenőrzése, memóriahibák keresése, adatfolyam-elemzés.
- **Lefedettségi mérés:** utasítás-, döntés- vagy MC/DC lefedettség.
- **Felülvizsgálati jegyzőkönyv:** tervezési és kódátvizsgálás eredménye.
- **Formális verifikációs eredmény:** modellellenőrzés vagy bizonyítás, ha a szint megköveteli.
- **WCET és időzítési mérés:** a biztonsági funkció határidőre való lefutásának igazolása.

A **traceability** (nyomon követhetőség) az összekötő kapocs. Minden biztonsági követelménynek le kell tudni követni a forrását (melyik veszélyből származik), a tervbeli megvalósítását, a tesztjét és az igazoló bizonyítékot. A gyakorlatban ez egy mátrix:

ID	Követelmény	Forrás veszély	ASIL/SIL	Megvalósítás	Teszt/elemzés	Státusz
SR-01	Túlmelegedés esetén lekapcsolás 100 ms alatt	V-01	ASIL B	safety_monitor.c: safety_task()	T-01, WCET-01	OK
SR-02	Szenzorhiba észlelése	V-01	ASIL B	sensor_check.c	T-02, SA-02	OK
SR-03	Watchdog lekapcsolás futtatás elmaradása esetén	V-02	ASIL A	watchdog.c	T-03	Megnyitott

A traceability hiánya tipikus hiba: a kód tartalmaz egy ellenőrzést, de nem lehet bizonyítani, hogy az egy adott biztonsági követelményt valósít meg. Fordítva is igaz: a követelmény megvan, de nincs teszt, amely igazolná.

A **független ellenőrzés** azt jelenti, hogy a fejlesztőcsapattól szervezetileg elkülönült személy vagy csapat vizsgálja a bizonyítékokat. A függetlenség nem feltétlenül külső cég; belső szervezeti egység is lehet, de nem lehet ugyanaz, amely a kódot írta. A tanúsító szervezet akkreditált, és a szabvány szerinti megfelelőséget igazolja. A tanúsítás folyamata:

1. A fejlesztő összeállítja a biztonsági esetet: a veszélyelemzést, a követelményeket, a terveket, a tesztjegyzőkönyveket, az időzítési méréseket és az elemzéseket.
2. A független értékelő mintavételesen és teljes körűen vizsgálja a nyomon követhetőséget, a módszerek megfelelőségét és a bizonyítékok helyességét.
3. Az értékelő nem a kódot „minősíti”, hanem azt ellenőrzi, hogy a fejlesztő betartotta-e a szabvány által előírt célokat, és a bizonyítékok alátámasztják-e az állításokat.
4. A tanúsítás eredménye egy tanúsítvány vagy megfelelőségi nyilatkozat, amely a rendszer vagy a szoftver egy meghatározott verziójára érvényes.

A független ellenőrzés során különösen figyelik:

- a konfigurációkezelés nyomon követhetőségét (melyik forráskód-verzióból készült a futtatott bináris),
- a reprodukálhatóságot (a build és a tesztek megismételhetők-e),
- a nem teljesített követelmények kezelését,
- a megmaradt kockázatok indoklását.

23.4.7 4. Gyakorlati minta: ellenőrzött hőmérséklet-felügyelet

Tegyük fel, hogy egy emulált beágyazott rendszerben egy fűtőkört vezérlünk. A normál szabályozás egy PID-szabályzó, de a biztonsági felügyelet ettől függetlenül működik. A veszély: **túlmelegedés tüzet vagy anyagi kárt okoz**. A kockázatelemzés egyszerűsített eredménye: a súlyosság magas (S2), a kitettség közepes (E3), az irányíthatóság alacsony (C2), így a rendszer **ASIL B** szintet kap.

A biztonsági cél: „a fűtőkört le kell kapcsolni, ha a hőmérséklet meghaladja a 80,0 °C-ot.” Az időbeli korlát a megszakítási késleltetésből, a szenzorolvasásból és a beavatkozási időből adódik:

$$t_{\text{reakció}} + t_{\text{beavatkozás}} \leq D_{\text{safety}} = 100 \text{ ms}$$

A biztonsági funkció függetlensége azt jelenti, hogy a PID-szabályozó kódjának hibája nem akadályozhatja meg a lekapcsolást. Ezért a biztonsági feladatot vagy nagyobb prioritású, vagy külön hardveres watchdog támogatja.

A követelménylista részlete:

ID	Követelmény	Forrás	ASIL	Igazolási módszer
SR-01	Ha a hőmérséklet > 80,0 °C, a fűtőkört le kell kapcsolni.	V-01	B	Funkcionális teszt T-01; döntési lefedettség
SR-02	A lekapcsolásnak legfeljebb 100 ms alatt meg kell történnie.	V-01	B	WCET mérés, időzítési teszt T-02

ID	Követelmény	Forrás	ASIL	Igazolási módszer
SR-03	Ha a szenzor hibás (rövidzár vagy szakadás), a fűtést le kell kapcsolni.	V-01	B	Hibainjektálás T-03; statikus elemzés SA-01
SR-04	Ha a biztonsági feladat nem fut le a várt ütemben, a watchdog áramtalanít.	V-02	A	Fedélzeti teszt T-04

A megvalósítás egy egyszerű, freestanding C vázlata:

```
/* safety_monitor.c - szemléltető példa, nem éles rendszerhez */
#define SAFETY_LIMIT_DC 800          /* 80,0 °C tizedfokban */
#define SAFETY_DEADLINE_MS 100
#define SENSOR_FAULT_VALUE 0x7FFF    /* szakadást jelző nyers érték */

volatile int heater_enabled = 1;
volatile uint32_t safety_cycle_count = 0;

void safety_task(void) {
    int16_t raw = read_temperature_sensor();

    if (raw == SENSOR_FAULT_VALUE || raw > SAFETY_LIMIT_DC) {
        disable_heater();
        log_safety_event(raw);
    }

    safety_cycle_count++;
    kick_watchdog();
}

void normal_control_task(void) {
    /* A normál PID-szabályozás itt fut. */
}
```

A bizonyítékok három rétege:

1. **Funkcionális teszt (T-01):** a szimulált hőmérséklet 79,9 °C-ról 80,1 °C-ra vált, és a teszt megfigyeli, hogy a `heater_enabled` 1-ből 0-ba vált. Az elvárt és a mért értéket külön kell rögzíteni.
2. **Időzítési mérés (T-02):** a QEMU-ban futtatott `safety_task` belépésétől a `disable_heater()` visszatéréséig mért időt a soros napló időbélyegei alapján számítják. A mérési konfigurációt (CPU modell, memória, fordítási beállítások, órajel) dokumentálni kell. A várt felső korlát 100 ms, a mért értéket a laborban kell kitölteni.
3. **Hibainjektálás (T-03):** a szenzor bemenetét szimulált szakadásra állítják, és ellenőrzik, hogy a fűtőkör lekapcsol. Ez a negatív teszt a biztonsági funkció diagnosztikai képességét igazolja.

A nyomon követhetőségi mátrix a fenti követelményeket a tesztekhez és a forráskódhoz köti. A független ellenőrző ebből látja, hogy nincs árva követelmény és nincs teszt nélküli biztonsági funkció.

23.4.8 5. Tipikus hibák és helyes megoldások

A tanúsítás során visszatérő hibák:

1. **A traceability hiánya.** A kódban van egy ellenőrzés, de nem lehet megmondani, melyik biztonsági követelményt szolgálja. *Helyes megoldás:* minden biztonsági követelmény kapjon azonosítót, és a kódkommentek, valamint a tesztek hivatkozzanak rá.
2. **Csak funkcionális pozitív teszt.** A teszt igazolja, hogy normál bemenetre a rendszer jól működik, de nem vizsgálja a hibás bemeneteket, a szenzorhibákat vagy a határérték környéki átmeneteket. *Helyes megoldás:* határérték-tesztek, negatív tesztek, hibainjektálás.
3. **Az időzítési bizonyíték hiánya.** A biztonsági funkció logikailag helyes, de nincs mérve, hogy a határidőn belül lefut-e. *Helyes megoldás:* WCET elemzés vagy mérés, a megszakítási késleltetés és a jitter dokumentálása, a legrosszabb esetre vonatkozó felső korlát.
4. **A normál és a biztonsági funkció összemosása.** Egyetlen feladat végzi a szabályozást és a védelmet; ha a normál kód elszáll, a védelem is elvész. *Helyes megoldás:* külön biztonsági feladat, magasabb prioritás vagy hardveres watchdog, és a függetlenség igazolása.
5. **A safety és a security keverése.** A fejlesztő titkosítja a kommunikációt, és azt hiszi, ezzel a safety követelményt is teljesítette. *Helyes megoldás:* a safety elemzés a véletlen meghibásodásokra és a meghatározott veszélyekre összpontosít; a security intézkedések kiegészítik, de nem helyettesítik azt.
6. **Nem reprodukálható bizonyíték.** A teszt egyszer lefutott, de a konfiguráció vagy a bemenet nem dokumentált. *Helyes megoldás:* build- és futtatási parancsok, a környezet rögzítése, verziókezelés, a napló megőrzése.
7. **Az „önigazolás" csapdája.** Ugyanaz a csapat írta a kódot és végezte az ellenőrzést, ezért a kritikus hibákat nem vette észre. *Helyes megoldás:* független felülvizsgáló bevonása a magasabb szinteken.
8. **A kockázatot a megvalósítás után értékelik.** Előbb elkészül a kód, aztán megpróbálják hozzárendelni a SIL/ASIL szintet. *Helyes megoldás:* a veszélyelemzés és a biztonsági követelmények a tervezés elején, a kódolás előtt készülnek.

Ezek a hibák nem elméletiek; a tanúsítási auditok leggyakoribb hiányosságai közé tartoznak. A helyes megoldás mindig az, hogy a bizonyítékot a követelményből vezessük le, és a lánc egyetlen pontján se törjön meg.

23.4.9 Labor: minősítési és biztonsági követelménylista

Ez a labor egy emulált környezetben készített biztonsági követelménylista és traceability mátrix létrehozását kéri. A cél nem a teljes tanúsítás, hanem a fejezet eszközeinek alkalmazása egy kis, ellenőrizhető példán.

Kiinduló állapot. A hoszt gépen QEMU telepítve, egy egyszerű emulált ARM Cortex-M3 alapú modell (`lm3s6965evb`) és egy freestanding C projekt a fenti `safety_monitor` vázlatból kiindulva. A hoszt valódi lemezét és bootloaderét nem módosítjuk.

Konfiguráció.

- Emulátor: QEMU 8.x (a pontos verziót a laborjegyzetben rögzíteni kell)
- Gép: `lm3s6965evb`, 64 MiB RAM, 1 CPU

- Fordítás: `arm-none-eabi-gcc -O2 -Wall -Wextra -std=c11 -ffreestanding -mcpu=cortex-m3`
- Linker script: a projekt saját `link.ld` állománya
- Soros port: `-serial stdio`
- Órajel: a QEMU által biztosított időzítő; a mérést a soros napló időbélyegei alapján végezzük

Feladat.

1. Készítsen legalább négy biztonsági követelményt a hőmérséklet-felügyelethez a fenti minta alapján. Mindegyik kapjon azonosítót (`SR-01`, `SR-02`, …), forrást (melyik veszélyhez tartozik), ASIL/SIL szintet és igazolási módszert.
2. Készítsen traceability mátrixot, amely a követelményektől a forráskód függvényeiig és a tesztekig mutat.
3. Futtassa a `safety_task` funkcionális tesztjét a QEMU-ban: hőmérséklet-emelés 79,5 °C-ról 80,5 °C-ra, és rögzítse a `heater_enabled` változó értékét a lekapcsolás előtt és után.
4. Mérje meg a lekapcsolási időt a soros napló időbélyegei alapján. A várt felső korlát 100 ms; a mért értéket a jegyzőkönyvben külön oszlopban adja meg.
5. Hajtson végre hibainjektálást: állítsa a szenzor nyers értékét `0x7FFF`-re, és ellenőrizze, hogy a fűtőkör lekapcsol.

Mérési napló.

A jegyzőkönyv legalább a következő oszlopokat tartalmazza:

```
azonosító; bemenet; várt eredmény; mért eredmény; megfelelés; megjegyzés
```

Például:

```
T-01; hőmérséklet = 80,5 °C; heater_enabled = 0; heater_enabled = 0; Igen; funkció teljesült
T-02; biztonsági feladat; t_reakció <= 100 ms; t_reakció = [mérendő]; [Igen/Nem]; lásd soros
↪ napló
```

A mért és a várt értéket itt is külön kell jelölni; a becsült vagy „jónak tűnik” eredmény nem elfogadható.

Visszaállítás. A labor nem módosítja a hosztot. A futtatott binárist a projektkönyvtárból törölni lehet, és a forrásból újraépíteni. A naplófájlt a laborjegyzettel együtt verziókezelőbe kell helyezni.

Mérőszám. A labor eredménye két számszerű mutatót tartalmaz:

- a biztonsági funkció maximális mért válaszideje (ms),
- a traceability lefedettség: a biztonsági követelmények hány százaléka rendelkezik legalább egy teszt- vagy elemzési bizonyítékkal.

Önellenőrző kérdés. Miért nem elegendő egyetlen funkcionális teszt a 80,0 °C határértékre? Indokolja a határérték-teszt, a hibainjektálás és a független ellenőrzés szerepét.

23.4.10 Összefoglalás

A biztonsági kritikus rendszerek tanúsítása a veszélyelemzésből levezetett követelmények, a nyomon követhető megvalósítás és a független ellenőrzött bizonyítékok zárt láncolata. Az IEC 61508, az ISO 26262 és a DO-178C ugyanazt a logikát követik: minél súlyosabb a veszély, annál több és szigorúbb bizonyíték kell. A SIL, ASIL vagy DAL szint nem a kód minőségének dicsérete, hanem a szükséges mérnöki tevékenységek mértéke. A gyakorlatban a legnagyobb kockázatot a traceability hiánya, a reprodukálhatatlan mérés és az önigazolás okozza. A következő fejezetben az itt bevezetett fogalmakra építve a FreeRTOS és Zephyr RTOS-eket vizsgáljuk.

24 RTOS-példák és minták

24.1 FreeRTOS és Zephyr

A FreeRTOS és a Zephyr egyaránt nyílt forrású, beágyazott rendszerekre szánt valós idejű operációs rendszer. Közös bennük, hogy a kernel a feladatok (taskok, szálak) ütemezését, a köztük lévő szinkronizációt és az időkezelést a determinizmus igényével valósítja meg. A két rendszer azonban eltérő filozófiát követ: a FreeRTOS kisméretű, szűk API-jú, statikus és dinamikus allokációt egyaránt támogató kernel; a Zephyr moduláris, Linux-stílusú buildrendszerrel, eszközfa-alapú hardverleírással és gazdag illesztőprogram-réteggel rendelkező ökoszisztéma. Ebben az alfejezetben először a FreeRTOS, majd a Zephyr felépítését és konfigurációját tekintjük át, ezt követően egy közös termelő–fogyasztó és időzítő mintán mutatjuk be a taskokat, üzenetsorokat, szemaforokat és szoftveres időzítőket, végül a laborban reprodukálható vázlatot adunk mindkét kernelhez.

24.1.1 1. FreeRTOS: kis kernel, szigorú prioritásos ütemezés

A FreeRTOS kernelt elsősorban egymagos mikrokontrollerekhez tervezték. A rendszer időalapja a periodikus rendszer-tick, amelynek frekvenciáját a `configTICK_RATE_HZ` adja meg; tipikus érték 100 Hz–1000 Hz. A tick megszakítás (például ARM Cortex-M esetén a SysTick) hajtja az időzítést, az időosztásos ütemezést és a blokkolt taskok ébresztését. A FreeRTOS ütemezője fix prioritásos, preemptív: mindig a futásra kész, legmagasabb prioritású task fut. Azonos prioritású taskok esetén – ha a `configUSE_TIME_SLICING` engedélyezett – a tick ciklikusan időosztást ad, ellenkező esetben a taskok futásra készen addig futnak, amíg egy magasabb prioritású task kész nem lesz, vagy blokkolnak. A prioritások 0-tól `configMAX_PRIORITIES - 1`-ig terjednek; FreeRTOS-ban a nagyobb szám magasabb prioritást jelent, ellentétben a Zephyr konvenciójával, ahol a kisebb szám magasabb prioritású.

A task létrehozása dinamikus vagy statikus memóriából történhet:

```
/* FreeRTOS vázlat - szemléltetés, nem végrehajtandó utasítás. */
BaseType_t xTaskCreate(
    TaskFunction_t pvTaskCode,
    const char * const pcName,
    configSTACK_DEPTH_TYPE uxStackDepth,
    void *pvParameters,
    UBaseType_t uxPriority,
    TaskHandle_t *pxCreatedTask);
```

A visszatérési érték `pdPASS`, ha a létrehozás sikeres, és `errCOULD_NOT_ALLOCATE_REQUIRED_MEMORY`, ha a heapből nem áll rendelkezésre elegendő memória. A task stack méretét szavakban, nem bájtban adjuk meg, és a portrétegtől függően a stack iránya eltérhet. A kernel a `configTOTAL_HEAP_SIZE` által meghatározott heapből gazdálkodik; a `configCHECK_FOR_STACK_OVERFLOW` bekapcsolásával a stack túlcsordulását a taskváltáskor lehet érzékelni.

A FreeRTOS task állapotai: Running, Ready, Blocked és Suspended. A blokkolt állapot a `vTaskDelay`, `vTaskDelayUntil`, üzenetsor- vagy szemaforművelet várakozási ideje alatt áll fenn. A kernel rendelkezik egy idle taskkal, amely akkor fut, ha nincs más futásra kész task, valamint opcionálisan egy szoftveres időzítő (timer) taskkal, amely a `xTimerCreate`-tel létrehozott időzítők callbackjeit futtatja.

A FreeRTOS IPC- és szinkronizációs objektumai:

- **Queue (üzenetsor):** `xQueueCreate`, `xQueueSend`, `xQueueReceive`. Az üzenetsor mérete elemekben van megadva; a küldő és fogadó task közötti adatátvitel FIFO-sorrendű. A küldés `xTicksToWait` paramétere megadja, hány tickig várhat a task, ha a sor tele van. A visszatérési érték `pdPASS` vagy `errQUEUE_FULL`.
- **Binary semaphore:** `xSemaphoreCreateBinary`, `xSemaphoreTake`, `xSemaphoreGive`, `xSemaphoreGiveFromISR`. Egy bináris szemafor tipikusan megszakítás és task közötti szinkronizációra szolgál.
- **Counting semaphore:** `xSemaphoreCreateCounting` erőforrás-számláláshoz.
- **Mutex:** `xSemaphoreCreateMutex` kölcsönös kizáráshoz; FreeRTOS mutex **priority inheritance** mechanizmussal enyhíti a prioritás-inverziót. A mutexet csak az a task adhatja vissza, amelyik megkapta.
- **Software timer:** `xTimerCreate`, `xTimerStart`, `xTimerStop`. A callback egy dedikált timer task kontextusában fut, ezért ott blokkoló hívás nem végezhető.
- **Task notification:** gyors, fix méretű értesítési csatorna taskonként, amely sok esetben kiváltja a bináris szemafor és az egyszerű üzenetsor használatát.

A FreeRTOS hardverabsztrakciója a hordozható (port) rétegben összpontosul. Az architektúra-specifikus állományok – például ARM Cortex-M esetén `port.c`, `portmacro.h` – tartalmazzák a kontextusváltás assembly rutinjait, a kritikus szakaszok kezelését, a SysTick/PendSV megszakításokat és a task stack inicializálását. Az alkalmazás általában nem közvetlenül éri el ezt a réteget, hanem a kernel API-ján keresztül dolgozik. A megszakítási prioritások kezelésénél a `configMAX_SYSCALL_INTERRUPT_PRIORITY` határozza meg, hogy mely megszakítási szinteken lehet kernel API-t hívni `FromISR` változatban.

A konfiguráció a `FreeRTOSConfig.h` állományban történik. Egy jellemző részlet:

```
/* FreeRTOSConfig.h vázlat - szemléltetés. */
#define configCPU_CLOCK_HZ                      ( 12000000UL )
#define configTICK_RATE_HZ                      ( 1000 )
#define configMAX_PRIORITIES                    ( 5 )
#define configMINIMAL_STACK_SIZE                ( 128 )
#define configTOTAL_HEAP_SIZE                   ( 8192 )
#define configUSE_PREEMPTION                    1
#define configUSE_TIME_SLICING                  1
#define configUSE_MUTEXES                       1
#define configUSE_TIMERS                        1
#define configCHECK_FOR_STACK_OVERFLOW          2
#define configMAX_SYSCALL_INTERRUPT_PRIORITY    0x30
```

A `configTICK_RATE_HZ` értékből adódó tickperiódus itt 1 ms. A `configMAX_SYSCALL_INTERRUPT_PRIORITY` pontos értéke a cél MCU megszakítási prioritásbitjeitől függ; hibás beállítás esetén a kritikus szakaszok és a kernel API-k megszakításból történő hívása sérülhet.

24.1.2 2. Zephyr: moduláris kernel és eszközfa-alapú hardverabsztrakció

A Zephyr RTOS a Linux kernelből ismert build- és konfigurációs módszereket adaptálja a beágyazott világba. A build a CMake-re és a `west` metaeszközre épül. A konfiguráció Kconfig

opciókon (`prj.conf`) és devicetree (eszközfa) leírókon keresztül történik. A hardverabsztrakció rétegei:

- **arch réteg:** CPU-architektúra-specifikus kontextusváltás, megszakításkezelés, memóriakezelés.
- **SoC réteg:** az adott chip perifériáinak cím- és megszakítástérképe.
- **board réteg:** a panel szintű erőforrások (LED, gomb, UART, érzékelő) hozzárendelése.
- **drivers és subsystems:** eszközillesztők, hálózati stack, fájlrendszer, power management.

A Zephyr kernel objektumai statikusan vagy dinamikusan hozhatók létre. A leggyakrabban használt szinkronizációs objektumok: szál (thread), szemafor, mutex, üzenetsor (`msgq`), FIFO/LIFO, stack, mailbox, pipe, esemény (event), memóriaslab és időzítő. A szálak létrehozásakor a prioritás értéke egész szám; a kisebb szám magasabb prioritást jelent. A negatív prioritású szálak kooperatívak, a nemnegatívak preemptívek. A pontos tartományt a `CONFIG_NUM_COOP_PRIORITIES` és a `CONFIG_NUM_PREEMPT_PRIORITIES` Kconfig beállítások adják meg.

A Zephyr szál létrehozása statikus makróval:

```
/* Zephyr vázlat - szemléltetés, nem végrehajtandó utasítás. */
K_THREAD_DEFINE(producer_tid, 512,
                producer_entry, NULL, NULL, NULL,
                2, 0, 0);
```

Itt `512` a stack mérete bájtban, `producer_entry` a belépési függvény, a `2` a prioritás (alacsonyabb szám, mint a fogyasztó `3`-ja, tehát magasabb prioritás). A `K_THREAD_DEFINE` a szálat a rendszerindításkor automatikusan létrehozza.

Az üzenetsor definiálása:

```
/* Zephyr üzenetsor vázlat - szemléltetés. */
typedef struct {
    uint32_t id;
    int32_t  value;
} app_msg_t;

K_MSGQ_DEFINE(app_msgq, sizeof(app_msg_t), 4, 4);
```

A `K_MSGQ_DEFINE` paraméterei rendre: a sor neve, egy üzenet mérete bájtban, a maximális üzenetek száma, és az igazítás. A `k_msgq_put` és `k_msgq_get` függvények a `k_timeout_t` típusú időtúllépéssel hívhatók; a visszatérési érték 0 siker esetén, negatív `errno` érték (például `-ENOMSG`) időtúllépéskor. A pontos hibakódokat az adott Zephyr verzió dokumentációjában ellenőrizni kell, mert az API-k kiadásonként bővülhetnek.

A szemafor statikus definíciója:

```
K_SEM_DEFINE(app_sem, 0, 1);
```

Itt `0` a kezdeti érték, `1` a maximális érték. A `k_sem_take` visszatérése 0 siker, `-EAGAIN` időtúllépés esetén; a `k_sem_give` növeli a szemafor értékét, és felébreszti a várakozó szálakat.

A szoftveres időzítő:

```
K_TIMER_DEFINE(app_timer, timer_expiry, NULL);

void timer_expiry(struct k_timer *timer_id)
{
    /* Nem blokkoló műveletek, például szemafor felszabadítása. */
    k_sem_give(&app_sem);
}
```

Az időzítő indítása egyedi vagy periodikus üzemmódban:

```
k_timer_start(&app_timer, K_MSEC(100), K_MSEC(100));
```

A callback a rendszeridőzítő kontextusában fut, ezért itt blokkoló művelet – például `k_msgq_get` `K_FOREVER` várakozással – tilos, mert holtponthoz vagy a kernel időzítőjének blokkolásához vezethet.

A Zephyr konfiguráció a `prj.conf` állományon keresztül történik:

```
# Zephyr prj.conf vázlat - szemléltetés.
CONFIG_PREEMPT_ENABLED=y
CONFIG_TIMESLICING=y
CONFIG_TIMESLICE_SIZE=100
CONFIG_HEAP_MEM_POOL_SIZE=4096
CONFIG_SYSTEM_WORKQUEUE=y
CONFIG_LOG=y
```

A `CONFIG_TIMESLICE_SIZE` az azonos prioritású preemptív szálak közötti időosztás hossza tickben. A devicetree overlay fájlban a panel-specifikus eszközök – például egy LED vagy UART – engedélyezhetők és konfigurálhatók. A Zephyr a devicetree-ből generált makrókkal (`DT_NODELABEL`, `DEVICE_DT_GET`) teszi elérhetővé az eszközillesztőket, így a hardverportálás nem igényel kernelmódosítást, csak egy új board- vagy overlay-definíciót.

24.1.3 3. Összehasonlítás és hardverabsztrakciós különbségek

A FreeRTOS elsősorban a taskokra, üzenetsorokra és szemaforokra épít, és a hordozhatóságot a port réteg biztosítja. A konfiguráció egyetlen fejlécben (`FreeRTOSConfig.h`) összpontosul, ami kis projektekben áttekinthető. A Zephyr ezzel szemben a Linux-stílusú konfigurációs gráfot és devicetree-t használ, ami nagyobb tanulási görbét jelent, de jobban skálázódik sok perifériás, többmagos és biztonságkritikus rendszerekhez. A Zephyr natív támogatást ad energiagazdálkodáshoz, többmagos SMP-hez, memóriavédelmi egységekhez és hálózati stackekhez, míg a FreeRTOS ezek nagy részét kiegészítő könyvtárakként vagy külső modulokként kezeli.

A determinizmus szempontjából mindkét kernel preemptív prioritásos ütemezőt használ. A FreeRTOS tick-alapú időzítése miatt a blokkolt task ébresztése a következő tick határig is késhet, bár a `xTaskNotifyFromISR` és a direkt `portYIELD_FROM_ISR` csökkentheti ezt. A Zephyr a `k_sem_give` és `k_msgq_put` hívások után szintén azonnali újraütemezést végezhet. A legrosszabb esetű megszakítási késleltetést mindkét rendszerben a megszakításárak és a kritikus szakaszok hossza határozza meg; ezt az adott portréteg és a konfiguráció befolyásolja.

24.1.4 4. Gyakorlati minta: termelő–fogyasztó, szemafor és időzítő

A következő vázlat egy kis beágyazott alkalmazást mutat be mindkét kernelben. A rendszer egy termelő taskból, egy fogyasztó taskból, egy bináris szemaforból és egy periodikus szoftveres időzítőből áll. A termelő 20 ms-onként előállít egy üzenetet, és beteszi az üzenetsorba. A fogyasztó blokkolva várja az üzeneteket, feldolgozza azokat, és az időzítő callback 100 ms-onként felszabadítja a szemafor, amelyet a fogyasztó szintén figyel. A példa célja a task, queue, semaphore és timer együttes működésének bemutatása; a kód nem éles rendszerre való, és a pontos API-szignatúrák kiadásonként változhatnak.

24.1.5 4.1 FreeRTOS vázlat

```
/* FreeRTOS termelő-fogyasztó vázlat - szemléltetés, nem végrehajtandó. */
#include "FreeRTOS.h"
#include "task.h"
#include "queue.h"
#include "semphr.h"
#include "timers.h"

#define APP_QUEUE_LENGTH   4
#define PRODUCER_PERIOD_MS 20
#define CONSUMER_STACK     256
#define PRODUCER_STACK     256
#define CONSUMER_PRIORITY  2
#define PRODUCER_PRIORITY  1
#define TIMER_PERIOD_MS    100

typedef struct {
    uint32_t id;
    int32_t  value;
} app_msg_t;

static QueueHandle_t app_queue;
static SemaphoreHandle_t app_sem;
static TimerHandle_t app_timer;

static void producer_task(void *pvParameters)
{
    app_msg_t msg = { .id = 0, .value = 0 };
    TickType_t last_wake = xTaskGetTickCount();
    const TickType_t period = pdMS_TO_TICKS(PRODUCER_PERIOD_MS);

    for (;;) {
        msg.id++;
        msg.value = msg.id * 10;
        /* Teljes sor esetén maximum 2 ticket vár. */
        if (xQueueSend(app_queue, &msg, pdMS_TO_TICKS(2)) != pdPASS) {
            /* Hibakezelés: a termelő nem írhatja felül a sort. */
            configASSERT(0);
        }

        vTaskDelayUntil(&last_wake, period);
    }
}

static void consumer_task(void *pvParameters)
{
    app_msg_t msg;
    for (;;) {
        /* Üzenetsor-várakozás; a szemafor jelzésére is figyelnénk
           egy másik feladatban, itt most csak a sorra blokkol. */
        if (xQueueReceive(app_queue, &msg, portMAX_DELAY) == pdPASS) {
            /* Feldolgozás; itt például naplózás vagy periféria-írás. */
        }

        if (xSemaphoreTake(app_sem, 0) == pdPASS) {
            /* Az időzítő lejárt, például LED-et vált vagy telemetriát küld. */
        }
    }
}

static void timer_callback(TimerHandle_t xTimer)
{
```

```
    BaseType_t xHigherPriorityTaskWoken = pdFALSE;
    xSemaphoreGiveFromISR(app_sem, &xHigherPriorityTaskWoken);
    portYIELD_FROM_ISR(xHigherPriorityTaskWoken);
}

void app_init(void)
{
    app_queue = xQueueCreate(APP_QUEUE_LENGTH, sizeof(app_msg_t));
    app_sem   = xSemaphoreCreateBinary();
    configASSERT(app_queue != NULL);
    configASSERT(app_sem != NULL);

    xTaskCreate(producer_task, "prod", PRODUCER_STACK, NULL,
                PRODUCER_PRIORITY, NULL);
    xTaskCreate(consumer_task, "cons", CONSUMER_STACK, NULL,
                CONSUMER_PRIORITY, NULL);

    app_timer = xTimerCreate("swtmr", pdMS_TO_TICKS(TIMER_PERIOD_MS),
                             pdTRUE, NULL, timer_callback);
    configASSERT(app_timer != NULL);
    xTimerStart(app_timer, 0);
}
```

A vázlatban a termelő prioritása alacsonyabb, mint a fogyasztóé, így a fogyasztó üzenet érkezésekor azonnal futhat. A `xSemaphoreGiveFromISR` a timer callback kontextusából hívható, mert a callback a FreeRTOS timer taskjában fut, amely megszakítási kontextusú szempontból nem a valódi ISR, de a `FromISR` változat itt is használható, ha a port támogatja. A `configASSERT` hibás konfiguráció vagy allokációs hiba esetén leállítja a rendszert, ami a laborban hasznos hibakeresési eszköz.

24.1.6 4.2 Zephyr vázlat

```
/* Zephyr termelő-fogyasztó vázlat - szemléltetés, nem végrehajtandó. */
#include <zephyr/kernel.h>

#define APP_MSGQ_MAX_MSGS  4
#define PRODUCER_PERIOD_MS 20
#define CONSUMER_PRIORITY  3
#define PRODUCER_PRIORITY  2
#define TIMER_PERIOD_MS    100

typedef struct {
    uint32_t id;
    int32_t  value;
} app_msg_t;

K_MSGQ_DEFINE(app_msgq, sizeof(app_msg_t), APP_MSGQ_MAX_MSGS, 4);
K_SEM_DEFINE(app_sem, 0, 1);
K_TIMER_DEFINE(app_timer, timer_expiry, NULL);

static void producer_entry(void *p1, void *p2, void *p3)
{
    app_msg_t msg = { .id = 0, .value = 0 };

    while (1) {
        msg.id++;
        msg.value = msg.id * 10;
        /* Teljes sor esetén legfeljebb 2 ms-ot vár. */
        if (k_msgq_put(&app_msgq, &msg, K_MSEC(2)) != 0) {
            /* Hibakezelés: a termelő nem írhatja felül a sort. */
            k_panic();
```

```
        }
        k_msleep(PRODUCER_PERIOD_MS);
    }
}

static void consumer_entry(void *p1, void *p2, void *p3)
{
    app_msg_t msg;

    while (1) {
        if (k_msgq_get(&app_msgq, &msg, K_FOREVER) == 0) {
            /* Feldolgozás; például naplózás vagy periféria-írás. */
        }

        if (k_sem_take(&app_sem, K_NO_WAIT) == 0) {
            /* Az időzítő lejárt; például LED-et vált. */
        }
    }
}

static void timer_expiry(struct k_timer *timer_id)
{
    k_sem_give(&app_sem);
}

void app_init(void)
{
    k_thread_create(&producer_thread, producer_stack,
                    K_THREAD_STACK_SIZEOF(producer_stack),
                    producer_entry, NULL, NULL, NULL,
                    PRODUCER_PRIORITY, 0, K_NO_WAIT);
    k_thread_create(&consumer_thread, consumer_stack,
                    K_THREAD_STACK_SIZEOF(consumer_stack),
                    consumer_entry, NULL, NULL, NULL,
                    CONSUMER_PRIORITY, 0, K_NO_WAIT);

    k_timer_start(&app_timer, K_MSEC(TIMER_PERIOD_MS),
                  K_MSEC(TIMER_PERIOD_MS));
}
```

A `K_THREAD_DEFINE` helyett itt `k_thread_create` látható dinamikus létrehozáshoz, ahol a `producer_stack` és `consumer_stack` előre deklarált stackterületek. A `K_THREAD_STACK_SIZEOF` makró a stack méretét adja vissza bájtban. A `K_NO_WAIT` az indítási késleltetést jelöli. A `k_panic()` a Zephyr hibakezelő rutinját hívja, amely a rendszer leállítása előtt naplózza az állapotot. A `k_sem_give` a timer callbackben biztonságos, mert nem blokkol.

24.1.7 5. Tipikus hibák és helyes megoldások

1. **Stack overflow.** Mindkét rendszerben a task stack méretének elég nagynak kell lennie a helyi változók, megszakítási kontextus és hívási lánc számára. FreeRTOS-ban a `configCHECK_FOR_STACK_OVERFLOW` és a `uxTaskGetStackHighWaterMark` segítségével ellenőrizhető a felhasználatlan stack. Zephyrben a `CONFIG_THREAD_STACK_INFO` és a `k_thread_stack_space_get` érhető el. A túl kicsi stack véletlenszerű memóriasérülést okoz.

2. **Prioritás-inverzió.** Ha egy alacsony prioritású task egy magas prioritású task által igényelt erőforrást (például mutexet) tart, a közepes prioritású taskok késleltethetik a magas prioritásút. FreeRTOS mutex esetén a priority inheritance alapértelmezésben aktív;

Zephyr `k_mutex` is támogatja a prioritásöröklést, de a pontos viselkedést a konfigurációtól függően ellenőrizni kell.

3. **Queue overflow.** Ha a termelő gyorsabb, mint a fogyasztó, az üzenetsor megtelik. A `xQueueSend` vagy `k_msgq_put` várakozási idővel hívható, de a túl hosszú várakozás blokkolhatja a termelőt, ami az adatvesztés helyett késleltetést okoz. A helyes megoldás a sor méretének a legrosszabb esetű burst-höz igazítása, vagy a termelő visszajelzése a fogyasztó felé.

4. **Timer callbackben blokkoló hívás.** FreeRTOS szoftveres időzítő callbackje a timer taskban fut, Zephyr időzítő callbackje a rendszeridőzítő kontextusában. Mindkettőben tilos hosszú, blokkoló műveletet végezni, mert az a timer task vagy a rendszeridőzítő elakadásához vezet, és a többi időzítőt is késlelteti.

5. **Konfigurációs eltérés.** FreeRTOS-ban a `configCPU_CLOCK_HZ` és a `configTICK_RATE_HZ` közötti eltérés hibás késleltetéseket okoz. Zephyrben a devicetree és a Kconfig közötti inkonzisztencia – például egy engedélyezett eszközillesztő hiányzó devicetree-csomópontja – buildhibát vagy futás közbeni hibát eredményez.

6. **Megszakítási prioritások helytelen beállítása.** FreeRTOS-ban a `configMAX_SYSCALL_INTERRUPT_PRI` értékét a CPU megszakítási prioritásbitjeihez kell igazítani. Ha túl magas prioritású megszakításból hívunk `FromISR` API-t, a kernel adatszerkezetei sérülhetnek.

24.1.8 6. Labor: FreeRTOS- és Zephyr-feladat vázlata

A labor célja, hogy a fenti termelő–fogyasztó mintát mindkét kernelben reprodukálható módon felépítsük, futtassuk emulált környezetben, és mérjük az időzítési jellemzőket. A labor kiinduló állapota egy üres workspace, amelyben a FreeRTOS és a Zephyr adott verziójú forrása elérhető. A pontos verziót a reprodukciós csomagban rögzíteni kell; a példákban a FreeRTOS V10.x és a Zephyr 3.x főverziójára hivatkozunk, de a build előtt ellenőrizni kell az API-kat az adott kiadás dokumentációjában.

24.1.9 6.1 Környezet és build

A labor emulált környezetben fut. A Zephyr esetében a `qemu_cortex_m3` board használható, a FreeRTOS esetében a hivatalos QEMU-alapú demo (például ARM Cortex-M3 vagy MPS2 AN385) használható. A hoszt valódi lemezét, bootloaderét és rendszerpartícióját tilos módosítani.

FreeRTOS buildvázlat:

```
# FreeRTOS build - szemléltetés; a pontos parancs a kiadás demo mappájától függ.
cd freertos-demo
make -C <demo_mappa> all
# Futtatás QEMU-ban:
qemu-system-arm -M mps2-an385 -kernel build/demo.elf -nographic
```

Zephyr buildvázlat:

```
# Zephyr build - szemléltetés; a pontos board és minta a kiadástól függ.
west build -b qemu_cortex_m3 samples/rtos_minta
west build -t run
```

A konfigurációt a fenti `FreeRTOSConfig.h` és `prj.conf` szerint állítsuk be. A tick frekvencia mindkét rendszerben 1000 Hz legyen, így a tickperiódus 1 ms. A CPU órajele az emulált QEMU-ban a board-tól függ; rögzítsük a pontos értéket (például 12 MHz), mert az időméréshez szükséges.

24.1.10 6.2 Mérési terv és napló

A mérés során az alábbi jellemzőket vizsgáljuk:

Jellemző	Várt érték (levezetett)	Mért érték (futtatás után)
Tick periódus	1 ms ± 0,1 ms	
Termelő task periódusa	20 ms ± 1 tick	
Üzenetsor-üres várakozási késleltetés a fogyasztónál	20 ms + kontextusváltási idő (néhány s)	
Szemafor felszabadítási késleltetés az időzítőtől	1 tick + kontextusváltási idő	
Üzenetsor maximális kihasználtsága	legfeljebb 4 üzenet (a sor mérete)	

A várt értékek levezetése:

- A tick periódus a `configTICK_RATE_HZ` = 1000 Hz-ből következik: 1 / 1000 s = 1 ms.
- A termelő `vTaskDelayUntil` / `k_msleep` hívása 20 ms periódust ad; a jitter a tick kvantálás és a magasabb prioritású fogyasztó futása miatt legfeljebb ±1 tick lehet.
- Az üzenetsorból való vétel `K_FOREVER` / `portMAX_DELAY` várakozással azonnal visszatér, amint a termelő betette az üzenetet. A várakozási idő a termelő periódusához kötött: legrosszabb esetben a fogyasztó épp az előző üzenet feldolgozása után érkezik, és a következő üzenet 20 ms múlva jön. A kontextusváltási idő a QEMU-ban mikro- vagy nanoszintű; ezt a mért értékhez viszonyítva kell értelmezni.
- A szemafor felszabadítása a timer callbackben történik; a fogyasztó akkor ébred, amikor a `k_sem_give` vagy `xSemaphoreGiveFromISR` felszabadítja. A késleltetéshez hozzájárul a tick-kvantálás, ha a callback a tick megszakítás után fut. A várt érték legfeljebb 1 tick (1 ms) plusz a kontextusváltás.

A méréshez használjunk ciklusszámlálót vagy tick-számlálót. FreeRTOS-ban a `xTaskGetTickCount()` és a `vTaskDelayUntil` tick-alapú; finomabb méréshez a portréteg ciklusszámlálója (például DWT->CYCCNT) használható. Zephyrben a `k_cycle_get_32()` és a `k_cyc_to_us_floor64()` ad hardveres időbélyeget. A mérési naplóban rögzítsük a hardver típusát, a CPU órajelét, a fordítási optimalizálási szintet (`-O2`), a tick frekvenciát és a mérési módszert. A mért és a várt értékeket külön oszlopban kell feltüntetni.

24.1.11 6.3 Visszaállítás és ellenőrzés

A labor nem módosítja a hoszt rendszert; a visszaállítás a workspace törléséből áll. A build és a futtatás után a konfiguráció, a napló és a mérési eredmény a reprodukciós csomagban megőrzendő. A `panic` és a stack trace gyűjtése mindkét rendszerben konfigurálható: FreeRTOS esetén a `configASSERT` és a `vApplicationStackOverflowHook`, Zephyr esetén a `CONFIG_FATAL_ERROR_LOG` és a `k_sys_fatal_error_handler`.

24.1.12 7. Összefoglalás és önellenőrző kérdés

A FreeRTOS és a Zephyr ugyanazokat az alapvető RTOS-építőköveket kínálják – taskok, üzenetsorok, szemaforok, időzítők –, de a konfiguráció és a hardverabsztrakció filozófiája eltér. A FreeRTOS egyszerű, szűk kernel, amely kis rendszerekben gyorsan kiismerhető; a Zephyr

moduláris, eszközfa-alapú ökoszisztéma, amely nagyobb projektekben és több periféria esetén skálázhatóbb. A helyes RTOS-választás a determinizmus, a footprint, a hardvertámogatás és a biztonsági követelmények együttes mérlegelését igényli.

Önellenőrző kérdés: Egy FreeRTOS rendszerben a termelő task prioritása 1, a fogyasztóé 2, az üzenetsor hossza 4. A termelő 5 ms-onként küld egy üzenetet, a fogyasztó feldolgozási ideje normál esetben 3 ms. Mi történik, ha a fogyasztó feldolgozási ideje átmenetileg 8 ms-re nő, és miért? Melyik hibajelenség léphet fel, és milyen módon lehet elkerülni a késleltetés felhalmozódását? Válaszát indokolja a prioritásos ütemezés és az üzenetsor kapacitásának figyelembevételével.

24.2 QNX, VxWorks és kereskedelmi RTOS-ek

A kereskedelmi valós idejű operációs rendszerek abban különböznek a nyílt forrású társaiktól, hogy a licenc mellett a gyártó meghatározott műszaki garanciákat, tanúsítási csomagokat és hosszú távú támogatást is értékesít. A QNX és a VxWorks két meghatározó képviselője ennek a világnak. Nem azért elterjedtek, mert minden mérhető tulajdonságuk a leggyorsabb, hanem mert viselkedésük – egy pontosan körülírt konfigurációban – jól modellezhető, dokumentált és gyakran tanúsítható. Ez a fejezet a két rendszer architekturális és üzleti jellemzőit, a választási szempontokat, a tipikus hibákat, valamint egy reprodukálható döntési mátrixot mutat be.

24.2.1 Miért használnak egyáltalán kereskedelmi RTOS-t?

Egy saját, minimális ütemező vagy egy nyílt forrású kernel sok esetben elegendő. Kereskedelmi RTOS-re akkor van szükség, ha a projekt legalább egyike igaz az alábbiak közül:

- a rendszer biztonsági vagy funkcionális biztonsági tanúsítást igényel, például ISO 26262 ASIL, IEC 61508 SIL vagy DO-178C;
- a gyártónak szerződéses garanciát kell adnia a hibajavítási időre, a sebezhetőségi folyamatra és a támogatási időtávra;
- a hardverportot, a BSP-t és az eszközmeghajtókat nem a felhasználó, hanem a szállító vagy annak partnere tartja karban;
- a determinizmus elemzéséhez a kernel belső viselkedését, prioritási modelljét és megszakítási útját szerződésszerűen, ne csak forráskódból kell ismerni.

Ezzel szemben a kereskedelmi RTOS-ek tipikus hátrányai a licencköltség, a kisebb nyilvános fejlesztői közösség, az eszközlánc esetleges kötöttsége és az, hogy a hibakeresés bizonyos rétegekben a gyártó eszközeire támaszkodik.

24.2.2 QNX: mikrokernel, üzenetek és resource managerek

A QNX – a BlackBerry QNX terméke, leggyakrabban a QNX Neutrino operációs rendszer – tankönyvi példája a kereskedelmi mikrokernel-architektúrának. A rendszer központi gondolata az, hogy a kernel a lehető legkisebb privileged részből álljon, a funkcionalitás nagy részét pedig felhasználói térben futó, elkülönült folyamatok biztosítsák.

24.2.3 Architekturális jellemzők

A QNX-ben a kernel alapvetően az alábbiakat nyújtja:

- szálak ütemezése és kontextusváltás;
- szinkron IPC alapú üzenetküldés;
- megszakítás- és kivételkezelés alsó rétege;
- alacsony szintű memóriavédelem és címfordítás;
- időzítési és órajelek.

A hálózati stack, a fájlrendszer, az eszközmeghajtók és a rendszerszolgáltatások külön folyamatokban, úgynevezett resource managerekben futnak. Egy fájlművelet vagy egy eszközhozzáférés gyakran nem egy közvetlen kernelhívást, hanem üzenetváltást jelent a kliens és a szolgáltató folyamat között. Ez a modell nagyobb memóriaizolációt ad, mert egy hibás drivert vagy szolgáltatást újra lehet indítani anélkül, hogy az egész kernel leállna. Ugyanakkor az izolációnak ára van: a művelet gyakran több kontextusváltást okoz, mint egy monolitikus rendszerhívás.

A QNX IPC modelljének központi eleme a szinkron üzenetküldés: `MsgSend`, `MsgReceive`, `MsgReply`. A kliens `MsgSend` hívása blokkolja a küldő szálat, amíg a szerver `MsgReceive` segítségével fogadja az üzenetet, majd `MsgReply`-vel válaszol. Ezt azért érdemes kiemelni, mert a szinkron jelleg egyszerűvé és determinisztikussá teszi az adatátadás életciklusát, ugyanakkor a tervezőnek tudatosan kell bánnia a prioritásokkal és a blokkoló hívásokkal. Nagy méretű adatokhoz a QNX nem feltétlenül többszöri másolást használ, hanem hatékonyabb átviteli mechanizmust választhat; a konkrét viselkedés a konfigurációtól függ, ezért mérni kell.

24.2.4 Ütemezés és determinizmus

A QNX ütemezője prioritásos, preemptív. A futásra kész legmagasabb prioritású szál fut, azonos prioritású szálak között gyakran round-robin időosztás alkalmazható. A prioritásszintek száma és a round-robin időszelet konfigurálható, de a pontos értékek verzió- és célhardverfüggők, ezért a tervezés során a gyártó adott verziójú referencia-kézikönyvét kell alapul venni.

A QNX fontos determinisztikai tulajdonságai:

- a megszakításkezelő nem végez hosszú feldolgozást; jellemzően egy későbbi, megfelelő prioritású szálat ébreszt fel;
- a kritikus szakaszok a kernelben rövidek;
- a mutexek támogatják a prioritásöröklést, így a priority inversion korlátozható;
- a szolgáltatások elkülönítése révén egy hibás eszközmeghajtó nem feltétlenül teszi tönkre a rendszer többi részét.

A mikrokernel előnye tehát nem az, hogy minden művelet gyorsabb, hanem az, hogy a hibatartományok kisebbek, a rendszer újraindítható komponensekre bontható, és a kritikus utak elemzése – jó konfiguráció esetén – kevesebb rejtett kölcsönhatást tartalmaz. A QNX-t ezért gyakran választják autóipari infotainment, digitális műszerfal, orvosi eszköz és ipari HMI területeken.

24.2.5 VxWorks: monolitikus gyökerek, moduláris jelen

A VxWorks a Wind River terméke, és az egyik legrégebbi, széles körben ismert kereskedelmi RTOS. Hagyományosan monolitikus, egyetlen címterű felépítéssel rendelkezett, amelyben a feladatok (taskok) közös címtérben futottak. A modern VxWorks 7 és későbbi profiljai azonban már modulárisak, támogatják a folyamatmodelleket, a védett címtartományokat és a különböző biztonsági profilokat.

24.2.6 Architekturális jellemzők

A VxWorks klasszikus képe egy erősen testreszabható, a hardware-hez közel álló kernel, amely a BSP-ből és a kernelmodulokból egyetlen image-é linkelhető. Ennek előnye az alacsony belépési költség, a kevés réteg közötti átmenet és a könnyen érthető végrehajtási út. Hátránya, hogy a hagyományos, memóriavédelem nélküli konfigurációban egy hibás pointer az egész rendszert lefagyaszthatja.

A modern változatok megkülönböztetik:

- **kernel space** modulokat: eszközmeghajtók, ütemező, hálózati stack egyes részei;
- **Real-Time Process (RTP)**: felhasználói térben futó, MMU által védett folyamatok;
- **forráskódból generált profilokat**: például biztonsági profil, biztonságkritikus profil, általános célú profil, amelyek eltérő funkciókat és tanúsítási bizonyítékokat tartalmaznak.

A VxWorks rendkívül erős a beágyazott ipari, repülési, védelmi és űripari alkalmazásokban. A Mars-járók és számos repülőgépes rendszer is VxWorks alapú, ami jól mutatja a platform hosszú távú életképességét, de önmagában nem bizonyítja, hogy minden projekthez ez a legjobb választás.

24.2.7 Ütemezés és determinizmus

A VxWorks alapértelmezése szintén prioritásos preemptív ütemezés. A feladatok között prioritásos FIFO vagy round-robin választható. A megszakítási késleltetés és a kontextusváltási idő erősen hardverfüggő; a gyártó által megadott értékeket gyakran meghatározott CPU-n, cache- és TLB-állapot mellett értelmezik. A VxWorks támogatja a prioritásöröklést, a megszakítások prioritizálását és a CPU-affinitást is, ezért a determinizmus jól hangolható.

A VxWorks determinisztikai elemzésénél figyelni kell arra, hogy a rendszerkép mely modulokat tartalmazza. Minden további szolgáltatás – például hálózati stack, fájlrendszer, debug szimbólumok – olyan kódrészt visz a kernelbe vagy a folyamatba, amely befolyásolhatja a legrosszabb idejű végrehajtást. A modern moduláris build ezt jobban kezelhetővé teszi, mert a nem használt komponensek kihagyhatók.

24.2.8 Piaci hely és támogatási modellek

A kereskedelmi RTOS-ek piaci helye nem egyetlen rangsor, hanem alkalmazási területek szerinti súlypontok összessége.

Terület	Jellemző QNX szerep	Jellemző VxWorks szerep
Autóipari infotainment, cockpit	erős jelenlét, több gyártó használja	kevésbé jellemző, de előfordul
Repülés, védelem, űr	bizonyos rendszerekben	nagyon erős, több évtizedes múlt
Ipari vezérlés, robotika	gyakori, főleg HMI közeli rendszerek	gyakori, főleg determinisztikus vezérlés
Hálózati eszközök, telekom	előfordul	erős jelenlét, főleg carrier-grade eszközök
Orvosi eszközök	jellemző a tanúsított profilok révén	szintén jellemző

A nyílt forrású FreeRTOS és Zephyr terjedése elsősorban az egyszerűbb, nagy példányszámú vagy erősen költségérzékeny eszközöknél erős. A kereskedelmi rendszerek akkor maradnak versenyképesek, ha a mérnöki költséghez képest a tanúsítási csomag, a szállítói felelősség és a terméktámogatás többet ér, mint a licencdíj.

A támogatási modell két pillérből áll:

- **terméktámogatás**: hibajegyek, prioritásos javítás, mérnöki konzultáció, biztonsági frissítések;
- **tanúsítási csomag**: a kernel verziójához tartozó biztonsági eset, tervezési dokumentáció, tesztjelentések és tanúsítvány bizonyítékok.

A hosszú távú támogatás gyakran 10–15 év, ami autóipari vagy repülőgépes programoknál alapkövetelmény. A gyártó által vállalt támogatási idő nem azonos azzal, hogy a felhasználó alkalmazása automatikusan tanúsított lesz. A rendszerintegrációt, a konfigurációt, a BSP-t és az alkalmazást a teljes rendszer tanúsításakor külön értékelni kell.

24.2.9 Licenc, tanúsítás és hosszú távú karbantartás

A licencmodellekben a legnagyobb félreértés a fejlesztői és a futtatási jog megkülönböztetése. Sok kereskedelmi RTOS lehetővé teszi, hogy a fejlesztő letöltse és kipróbálja az eszközláncot; a bevétel a gyártásba kerülő eszköz utáni royaltyból vagy a támogatási előfizetésből származik. A pontos feltételek termékenként és szerződésenként változnak, ezért minden projekt elején rögzíteni kell:

- hány fejlesztői székre van szükség;
- van-e eszközenkénti royalty, és milyen mennyiségi sávok vonatkoznak rá;
- a kernel mely profiljai és moduljai szerepelnek az árban;
- mi a hibajavítási és biztonsági frissítési idő;
- milyen exportellenőrzési vagy földrajzi korlátozások vonatkoznak a technológiára.

A tanúsítási kérdésekben a QNX és a VxWorks is erős, de másképp. A QNX esetében a mikrokerneles felépítés és a komponensek szétválasztása segítheti a biztonsági eset elkészítését, mert a kritikus és nem kritikus funkciók könnyebben elkülöníthetők. A VxWorks esetében a több évtizedes repülési és védelmi múlt, valamint a szigorú, jól dokumentált fejlesztési folyamat ad bizalmat. A konkrét tanúsítási szintek – például ISO 26262 ASIL, IEC 61508 SIL vagy DO-178C DAL – nem a teljes termékre, hanem egy adott, ellenőrzött konfigurációra vonatkoznak.

A hosszú távú karbantartás szempontjából a kereskedelmi rendszer előnye, hogy a kernel egy adott verziójához tartozó biztonsági frissítéseket a gyártó hosszú időn át szállítja. Hátránya, hogy a frissítés nem mindig drop-in: egy biztonsági javítás is megkövetelheti a teljes rendszer újravalidálását. Ezért a karbantartási stratégia része kell legyen a frissítési ablak, a regressziós teszt és a tanúsítási hatás elemzése.

24.2.10 QNX és VxWorks összehasonlítása

Az alábbi táblázat didaktikai összevetést ad, nem benchmark. A cellákban szereplő állításokat a gyártó aktuális adatlapja és a saját mérések szerint kell alátámasztani.

Szempont	QNX Neutrino	VxWorks
Kernelarchitektúra	mikrokernel, szolgáltatások felhasználói térben	hagyományosan monolitikus, modern profilokban moduláris és RTP
Izoláció	erős, folyamatonkénti címterek	konfigurációtól függ; RTP esetén erős, régi flat modellben gyenge
IPC	üzenetalapú, szinkron, resource managerek	többféle: szemafór, üzenetsor, osztott memória, POSIX
Hibatűrés	szolgáltatások újraindíthatók	kernelhiba gyakran rendszerleállás, ha a hibás kód kernelben fut
Tanúsítási fókusz	autóipar, orvosi, ipari biztonság	repülés, védelem, űr, ipari biztonság
Fejlesztői eszköz	QNX Momentics, grafikus és parancssori	Wind River Workbench, VxWorks Source Build
Licenc	fejlesztői licenc + eszközalapú royalty gyakran	fejlesztői licenc + royalty/támogatás

A gyakorlatban a választás ritkán pusztán architekturális. A csapat tapasztalata, a meglévő BSP, a hardvergyártó referencia-kialakítása és a tanúsítási ütemterv ugyanolyan fontos.

24.2.11 Tipikus hibák és helyes megoldások

Az alábbi esetek a leggyakoribb tévedéseket mutatják be. Mindegyiknél a hiba oka nem valamelyik rendszer, hanem a felületes követelményelemzés.

24.2.12 1. hiba: „A mikrokernel mindig lassabb, tehát a QNX rosszabb választás”

A mikrokernel IPC valóban okozhat többletköltséget, de a QNX erre optimalizált üzenetkezelést használ. A kérdés az, hogy a többlet a kritikus úton vagy egy nem időzítés-kritikus szolgáltatásban jelenik-e meg. Ha a vezérlési hurok közvetlenül egy eszközmeghajtóval beszél, mérni kell az üzenetváltás és a kontextusváltás idejét. Ha viszont a kritikus szál csak egy belső számítást végez, a mikrokernel költsége elhanyagolható lehet. A helyes megoldás: a kiválasztott műveleti mintára mérési tervet készíteni, és a mért mediánt, szórást és maximumot összehasonlítani.

24.2.13 2. hiba: „A tanúsított RTOS automatikusan tanúsítja az alkalmazásomat”

A gyártó tanúsítása a kernel adott konfigurációjára vonatkozik. Ha az alkalmazás rosszul kezeli a prioritásokat, osztott memóriát vagy megszakításokat, a teljes rendszer tanúsítása meghiúsulhat. A helyes megoldás: a rendszerbiztonsági elemzést a teljes szoftverre, ne csak az RTOS-re végezni, és a tanúsító szervezettel előre egyeztetni a bizonyítékokat.

24.2.14 3. hiba: „A hard real-time csak gyors CPU kérdése”

A gyors CPU csökkentheti az átlagos végrehajtási időt, de a hard real-time követelmény a legrosszabb esetről szól. A cache-miss, a TLB-miss, a megszakítások összefutása és a prioritásos inverzió mind hozzájárulhat a legrosszabb időhöz. A helyes megoldás: a legrosszabb idejű elemzés, a jitter mérése és a megszakítási utak felülvizsgálata a CPU órajele mellett.

24.2.15 4. hiba: „A régi VxWorks flat memóriamodellje elég, mert a csapat mindig is így fejlesztett”

Egy védett memória nélküli rendszerben egy rossz pointer az egész rendszert leállíthatja. Ha a rendszer biztonsági vagy rendelkezésre állási követelményt támaszt, a flat modell kockázata nem elfogadható. A helyes megoldás: RTP-k és MMU-védelem használata, vagy tudatos döntés arról, hogy a teljes kódot egyetlen biztonsági tartományként kezeljük.

24.2.16 5. hiba: „A QNX-ben minden szolgáltatás user space-ben fut, ezért nincs szükség kernelbeli hibakeresésre”

A QNX kernelében is vannak kritikus szakaszok. Ha a megszakítási alrendszer vagy az alacsony szintű ütemező hibás, a felhasználói tér védelme nem segít. A helyes megoldás: a kernel és a user space határán is tesztelni, a `panic` üzeneteket, trace naplókat és a reprodukciós konfigurációt együtt kezelni.

24.2.17 6. hiba: „A VxWorks minden konfigurációja egyformán determinisztikus”

A determinizmus a kernelkép összetételétől és a futó feladatoktól függ. Egy nagy hálózati stack, debug szimbólumok és fájlrendszer beépítése megváltoztathatja a megszakítási és ütemezési viselkedést. A helyes megoldás: a végleges termékprofillal mérni, nem egy teljes fejlesztői builddel.

24.2.18 Labor: RTOS-választási döntési mátrix

A labor célja, hogy a fejezet szempontjait egy konkrét, reprodukálható döntési eljárásban alkalmazza. A mátrix nem helyettesíti a mérést, de rákényszeríti a csapatot arra, hogy a követelményeket számszerűsítse, a bizonytalanságokat pedig explicit módon jelölje.

24.2.19 Kiinduló állapot és rögzített konfiguráció

Tételezzünk fel egy fiktív beágyazott vezérlőegységet:

- kétmagos, 64 bites ARM processzor, 1 GiB RAM;
- Ethernet, CAN és USB interfész;
- 50 µs-os vezérlési periódus, jitterkövetelmény legfeljebb ±5 µs a kritikus szálon;
- funkcionális biztonsági cél: ISO 13849 PL d vagy IEC 61508 SIL 2;
- tervezett élettartam: 10 év;
- fejlesztői csapat: 3 fő, C és Python ismerettel, korlátozott RTOS tapasztalattal;
- költségvetés: a licencköltség nem elsődleges, de 5 évre számszerűsíteni kell.

A döntési mátrix bemenetei közé tartozik QNX, VxWorks és – kontrollként – FreeRTOS. A fejezet a kereskedelmi rendszereket tárgyalja, ezért a FreeRTOS csak a „nincs tanúsítási csomag” szempont kontrollja.

24.2.20 Kritériumok és súlyok

A súlyokat a projekt követelményeihez kell igazítani. Az alábbi táblázat egy lehetséges kiindulást ad.

Azonosító	Kritérium	Súly (1–5)	Indoklás
K1	Determinizmus és legrosszabb eset	5	50 µs periódus és ±5 µs jitter
K2	Tanúsítási csomag	4	SIL 2/PL d bizonyíték szükséges
K3	Kernelarchitektúra illeszkedése	3	izoláció és hibatűrés
K4	Fejlesztői eszköz és tanulási görbe	3	kis tapasztalat
K5	Hosszú távú támogatás	5	10 év terméktámogatás
K6	Licenc és teljes költség	4	költségvetés értékelése
K7	BSP és hardverport elérhetősége	4	a referenciaport csökkenti a kockázatot

A súlyozás is dokumentálandó döntés; ha a csapat megváltoztatja, a jegyzetfüzetben rögzíteni kell az indokot.

24.2.21 Pontozási skála

Minden kritériumra 1–5 pont adható:

- 5: egyértelműen megfelel, gyártói bizonyíték vagy mérés áll rendelkezésre;
- 4: megfelel, de kisebb kompromisszummal;
- 3: elfogadható, de jelentős integrációs munkával;
- 2: csak részben felel meg;
- 1: nem felel meg vagy lényeges kockázat.

A pontszám forrását minősíteni kell: **várt/gyártói** vagy **mért/saját**. A labor „mérési naplója” ebből a minősítésből áll, mert valós licencelt RTOS-ek esetén a fejlesztő gyakran csak a gyártói adatlapra támaszkodhat az első körben.

24.2.22 Példa döntési mátrix

A táblázatban a „P” a pontszám, az „F” a forrásminősítés: G = gyártói/várt, S = saját/mért. A példaértékek szemléltetők, nem mérési eredmények.

Kritérium	Súly	QNX P	QNX F	VxWorks P	VxWorks F	FreeRTOS P	FreeRTOS F
K1	5	4	G	4	G	3	S
K2	4	4	G	4	G	1	G
K3	3	5	G	3	G	2	S
K4	3	3	G	3	G	5	S
K5	5	4	G	4	G	2	G
K6	4	3	G	3	G	5	G
K7	4	4	G	5	G	4	S

A súlyozott összpontszám kiszámítása pszeudokódban:

```
súlyozott_pontszám(rendszer):
    összeg = 0
    súlyösszeg = 0
    for k in kritériumok:
        összeg += k.súly * k.pontszám(rendszer)
        súlyösszeg += k.súly
    return összeg / súlyösszeg
```

A példa értékeivel:

- QNX: $(5 \cdot 4 + 4 \cdot 4 + 3 \cdot 5 + 3 \cdot 3 + 5 \cdot 4 + 4 \cdot 3 + 4 \cdot 4)$ / (5+4+3+3+5+4+4) = (20 + 16 + 15 + 9 + 20 + 12 + 16) / 28 = 108 / 28 3,86.
- VxWorks: (20 + 16 + 9 + 9 + 20 + 12 + 20) / 28 = 106 / 28 3,79.
- FreeRTOS: (15 + 4 + 6 + 15 + 10 + 20 + 16) / 28 = 86 / 28 3,07.

A számítás önmagában nem dönt; a mátrix a kockázatok láthatóvá tételére való. A QNX és a VxWorks közötti különbség például nem nagy, ezért a végső döntést a K3 és a K7 mögötti mérések, illetve a csapat próbaprojektje kell, hogy befolyásolja.

24.2.23 Reprodukciós jegyzetfüzet

A labor a következőket rögzíti:

1. a követelményekből levezetett kritériumokat és súlyokat;
2. a pontozási skála definícióját;
3. a három rendszer esetében a pontszámokat és a forrás minősítését;
4. ahol a forrás G, ott a gyártói dokumentum verzióját, dátumát és szakaszát;
5. ahol a forrás S, ott a mérési körülményeket: hardver, órajel, cache-állapot, mérési módszer, ismétlésszám, medián, maximum és szórás;
6. a visszaállítási és frissítési tervet, ha a döntés később megváltozik.

A mérési naplóban a **mért és várt értékeket külön oszlopban** kell vezetni. Például:

> K1, VxWorks: gyártói adatlap szerint a megszakítási késleltetés legrosszabb esete 4 µs az adott CPU-n. Várt érték, saját mérés nem áll rendelkezésre. A projektben a cél hardveren elvégzendő mérés, mielőtt a K1 pontszámot véglegesítjük.

Ez a mondat egyszerre jelzi a bizonytalanságot és a következő lépést.

24.2.24 Elvárt eredmény

A hallgató a labor végére:

- elkészíti a döntési mátrixot a fenti szerkezetben;
- minden cellához forrásminősítést ad;
- el tudja magyarázni, melyik kritérium miért érzékeny a választásra;
- azonosítja, hogy a QNX és a VxWorks közötti különbség mely pontokon igényel további saját mérést.

24.2.25 Önellenőrző kérdés

Egy csapat VxWorks mellett dönt azzal az indokkal, hogy a QNX mikrokernelje túl lassú, mert minden rendszerhívás üzenetváltást igényel. Milyen mérési és architekturális szempontokat hagyott figyelmen kívül? Adj meg egy olyan forgatókönyvet, amelyben a megfelelően mért adatok alapján a döntés megfordul.

Válaszvázlat: Először is a csapat nem mérte a kritikus úton lévő üzenetküldés tényleges költségét, és nem vizsgálta, hogy a vezérlési szál milyen szolgáltatásokkal kommunikál. Ha a kritikus szál egyetlen resource managerhez küld rövid vezérlőüzeneteket, a QNX szinkron IPC-jének költsége lehet, hogy elfogadható. Másodszor figyelmen kívül hagyta az izoláció előnyét: ha egy hálózati vagy fájlrendszer-hiba a monolitikus kernelben lefagyást okoz, az a vezérlési szálat is leállítja. A megforduló forgatókönyv például egy olyan rendszer, ahol a nem kritikus hálózati stack gyakran hibázik, és a kritikus vezérlőszálat ettől izolálni kell. Ebben az esetben a QNX mikrokerneles felépítése és a szolgáltatások újraindíthatósága nagyobb érték lehet, mint a VxWorks kissé alacsonyabb IPC-költsége.

24.2.26 Összefoglalás

A QNX és a VxWorks két eltérő filozófiát képvisel a kereskedelmi RTOS-ek világában. A QNX mikrokernelje az izolációt és az üzenetalapú szolgáltatásokat helyezi középpontba, a VxWorks pedig a hosszú beágyazott múltból érkező, ma már moduláris és tanúsított profilokat kínál. A választásnál a determinizmus, a tanúsítás, a licenc és a hosszú távú támogatás mellett a csapat tapasztalata és a cél hardverre elérhető BSP a meghatározó. A döntési mátrix nem mér, de rendet tesz a mérések és a gyártói állítások között, és megmutatja, hol kell a bizonytalanságot saját laborban feloldani.

24.3 AUTOSAR és járműipari architektúra

A járműipari szoftverarchitektúra tanulmányozása előtt érdemes rögzíteni, hogy az autóipari operációs rendszerek másfajta tervezési kényszerek között működnek, mint az általános célú asztali vagy szerver rendszerek. Egy modern járműben több tucat elektronikus vezérlőegység (ECU, Electronic Control Unit) futhat: motorvezérlő, fékrendszer, kormányzás, ajtómodul, műszerfal, infotainment, radar, kamera, akkumulátor-felügyelet. Ezek hardvere, erőforrása, időzítési követelménye és biztonsági besorolása nagyon eltérő lehet. Az operációs rendszer szerepe itt kettős: egyrészt a hardver erőforrásait kell determinisztikusan és biztonságosan kezelnie, másrészt szabványos interfészekkel kell elválasztania az alkalmazási szoftvert a konkrét ECU-től, hogy a funkciók újrahasznosíthatók és integrálhatók legyenek.

Ebben a fejezetben az AUTOSAR-t mutatjuk be, amely ma a járműiparban a legelterjedtebb szabványosított szoftverarchitektúra. A tanulási célok: megérteni az autóipari szoftverarchitektúra és az operációs rendszer kapcsolatát, alkalmazni az AUTOSAR rétegelt modellt és RTE-t egy kis példán, valamint felismerni a tipikus hibákat, különösen a funkcióbiztonság és a szoftverfrissítés területén.

24.3.1 1. Miért szükséges járműipari szoftverarchitektúra?

Egy jármű számtalan, különböző gyártótól származó szoftverkomponenst és hardverplatformot egyesít. Korábban minden ECU-t gyakran egyedi, célszoftverrel fejlesztettek, ami a következő problémákhoz vezetett:

- **Gyenge újrahasznosíthatóság:** a szoftver szorosan kapcsolódott a hardverhez, az illesztőprogramokhoz és a kommunikációs stackhez. Egy motorvezérlő funkciót nehéz volt átvinni egy másik ECU-be vagy másik gyártó hardverére.
- **Integrációs költség:** a rendszerintegrátor sokféle, eltérő interfészű szoftvert kapott, amelyek összehangolása egyre drágább lett, ahogy az ECU-k száma nőtt.
- **Biztonsági kihívások:** a kritikus funkciók izolálása és a hibaterjedés megakadályozása nem volt egységesen megoldva.
- **Frissítési nehézségek:** a szoftverfrissítés gyakran a teljes ECU újraprogramozását igényelte, és nem volt fail-safe mechanizmus.

Az AUTOSAR (AUTomotive Open System ARchitecture) ezt a problémát szabványosított rétegzett architektúrával, szabványos interfészekkel és módszertannal oldja meg. Két fő platformot definiál: a klasszikus platformot (Classic Platform, CP) a mélyen beágyazott, erőforrás-korlátos, valós idejű ECUk számára, és az adaptív platformot (Adaptive Platform, AP) a nagy számítási igényű, dinamikus, szolgáltatásalapú egységekhez. Mindkettőnek van operációs rendszer modellje, de nagyon eltérő megközelítéssel.

A fejezetben a „réteg” és „interfész” kifejezéseket az AUTOSAR fogalomtárnak megfelelően használjuk: a rétegek egymásra épülnek, és a felső réteg csak a közvetlenül alatta lévő réteg interfészeit hívhatja, ezzel biztosítva a hordozhatóságot és a modularitást.

24.3.2 2. AUTOSAR áttekintés: Classic, Adaptive és RTE

Az AUTOSAR architektúra központi gondolata a **virtuális funkcióbusz** (Virtual Functional Bus, VFB). A VFB egy absztrakt kommunikációs közeg, amelyen keresztül a szoftverkomponensek (Software Components, SWC-k) egymással adatot cserélnek, szolgáltatásokat hívnak, és módokat váltanak. Az SWC fejlesztője a VFB-n látja a komponens portjait és interfészeit, és nem kell tudnia, hogy az adott kommunikáció valójában egyetlen ECU-n belül fut, vagy buszon (pl. CAN, LIN, FlexRay, Ethernet) keresztül. Az integráció során a komponenseket ECU-kre helyezik, és a **Runtime Environment (RTE)** generálja le a tényleges kommunikációs kódot. Ez az alapvető különbség a hagyományos, kézzel írt illesztőrétegekhez képest.

Az AUTOSAR két platformja ugyanazt az alapelvet használja, de eltérő megvalósítási modellel:

- **Classic AUTOSAR:** statikusan konfigurált, előre ismert erőforrásokkal. Az operációs rendszer OSEK/VDX alapú, kemény valós idejű, kis lábnyomú. Az alkalmazások jellemzően C nyelvű, statikus feladatokból és „runnable”-ökből állnak. A futási kép a build időben nagyrészt meghatározott.

- **Adaptive AUTOSAR:** dinamikus, POSIX operációs rendszerre (jellemzően Linux) épül, szolgáltatásorientált kommunikációval (SOME/IP, DDS). Az alkalmazások C++ nyelven írt processzek, amelyek futásidőben telepíthetők, indíthatók és frissíthetők. Nagyobb processzorteljesítményt és memóriát feltételez.

A klasszikus és adaptív platform összehasonlítását az alábbi táblázat foglalja össze:

Jellemző	Classic AUTOSAR	Adaptive AUTOSAR
Célhardver	8/16/32 bites MCU-k, kis RAM/flash	32/64 bites SoC-k, MMU-val, nagy memória
OS	OSEK/ISO 17356 alapú, statikus	POSIX (PSE51/54), általában Linux
Programozási nyelv	C	C++
Konfiguráció	statikus, build időben	dinamikus, futásidőben
Kommunikáció	RTE, jelszerű, statikus	ara::com, szolgáltatásorientált, dinamikus
Frissítés	teljes ECU kép, gyakran offline	OTA, differenciális, futásidőben
Tipikus funkciók	motor, fék, kormány, alapérzékelők	ADAS, infotainment, zónavezérlő, HPC

A következő szakaszban a Classic AUTOSAR rétegmodellt mutatjuk be, mert ez adja az operációs rendszer legalaposabb járműipari keretét. Utána térünk ki az Adaptive platformra, ahol az operációs rendszer szerepe jelentősen eltér.

24.3.3 3. Classic AUTOSAR rétegek és az AUTOSAR OS

24.3.4 3.1. Rétegek

A Classic AUTOSAR rétegmodell jól elkülöníthető rétegekből áll. Alulról felfelé:

1. **Mikrokontroller Absztrakciós Réteg (MCAL, Microcontroller Abstraction Layer):** ez a legalacsonyabb szoftverréteg, amely közvetlenül a hardver regisztereit programozza, de felfelé egységes interfészt ad. Például a `Dio_WriteChannel()`, `Adc_ReadGroup()`, `Can_Write()` függvények. A MCAL a konkrét mikrovezérlő gyártójától függ, de az interfészek AUTOSAR szabványosak. Az operációs rendszer szempontjából fontos, hogy az MCAL biztosítja a timer- és megszakításkezelést, amelyre az OS épül.
2. **ECU Absztrakciós Réteg (ECU Abstraction Layer):** a MCAL fölé épül, és az ECU hardverét – azaz a mikrokontroller plusz a kártyán lévő egyéb áramkörök – absztrahálja. Például egy adott bemeneti jelfeldolgozó áramkör elérhető egy szabványos `IoHwAb_AnalogRead()` interfészen keresztül, függetlenül attól, hogy milyen ADC mögötte van. Ez a réteg már ECU-specifikus, de az alkalmazás számára hordozható interfészt kínál.
3. **Szolgáltatási Réteg (Services Layer):** a legfelső BSW (Basic Software) réteg, amely olyan szolgáltatásokat nyújt, mint a kommunikáció (COM stack), diagnosztika (DCM, DEM), memóriakezelés (NvM), ütemezés (OS), hiba- és watchdog-kezelés. Itt található az AUTOSAR OS is, amely a feladatok ütemezését és a megszakítások kezelését végzi.

4. **Komplex Illesztőprogramok (Complex Drivers):** néha szükség van olyan időzítés- vagy hardverspecifikus megoldásra, amely nem illik a fenti rétegekbe. Ilyenkor a komplex driver közvetlenül hozzáfér a hardverhez, és áthidalja a rétegeket. Ezt csak indokolt esetben szabad használni, mert gyengíti a hordozhatóságot.
5. **Futási Környezet (RTE, Runtime Environment):** a BSW és az alkalmazási réteg között helyezkedik el. Az RTE generált kód, amely a VFB leképezését valósítja meg az adott ECU-n belül. Az SWC-k soha nem hívják közvetlenül a BSW-t, csak az RTE API-t, például `Rte_Read_...`, `Rte_Write_...`, `Rte_Call_...`. Ez a szabály biztosítja, hogy az SWC-k teljesen függetlenek legyenek az ECU konkrét kommunikációs rendszerétől és hardverétől.
6. **Alkalmazási Réteg (Application Layer):** ide tartoznak a szoftverkomponensek. Egy SWC lehet atomi (egyetlen forrásfájl, jól meghatározott Runnable-ökkel) vagy kompozíció (több SWC együttese). A komponensek portokon keresztül kommunikálnak.

A rétegmodell fontos szabálya a **strict réteg-hozzáférés**: az alkalmazási réteg csak az RTE-t hívhatja, a Services Layer csak az ECU Abstraction és MCAL rétegeket, az ECU Abstraction csak a MCAL-t. Ezzel megakadályozható, hogy az alkalmazási kód közvetlenül hardverregisztereket írjon, ami sértené a biztonságot és a hordozhatóságot.

24.3.5 3.2. Az AUTOSAR OS és az OSEK/VDX örökség

A Classic AUTOSAR operációs rendszer az OSEK/VDX OS szabványra épül, amelyet az ISO 17356 szabvány is rögzít. Az AUTOSAR OS kiegészíti ezt skalár és vektor ütemezési táblázatokkal, időzítési védelemmel, memóriavédelemmel és további szolgáltatásokkal. Az OS statikusan konfigurált: a feladatok, erőforrások, alarmok, megszakítások és ütemezési táblázatok a rendszertervezés során létrehozott konfigurációban vannak leírva, és a generált OS-kép ezt a konfigurációt valósítja meg.

Az AUTOSAR OS fő objektumai:

- **Task:** az ütemezés alapegysége. Kétféle típus: **basic task** és **extended task**. A basic task nem rendelkezhet saját várakozási ponttal, csak fut, amíg be nem fejeződik vagy meg nem szakítják. Az extended task viszont blokkolhat eseményre. A taskokhoz prioritás tartozik, amely a konfigurációban rögzített. A task C függvényként jelenik meg, pl. `TASK(SpeedControl_Task) { ... }`.
- **Runnable:** az SWC futtatható egysége, amelyet az RTE rendel egy taskhoz. A task törzsében az RTE által generált hívások aktiválják a Runnable-öket. Például egy 10 ms periódusú task lefuttatja a `SpeedLimiter_Compute` Runnable-t.
- **Interrupt Service Routine (ISR):** kategória 1 és 2. Az 1-es kategóriájú ISR nem használhat OS API-t, a 2-es kategóriájú igen. A megszakítások prioritása magasabb, mint a taskoké.
- **Counter:** az OS időmérési alapja, tipikusan hardver timerre épül. A counter adja a tickeket.
- **Alarm:** a counterhez kötött időzítő, amely adott tick-értéknél aktiválhat taskot, beállíthat eseményt, vagy meghívhat callbacket.
- **Schedule Table:** olyan statikus táblázat, amely egymás után aktiválja a taskokat meghatározott időpontokban. Kétféle: **skalár** (egyetlen táblázat) és **vektor** (több táblázat közötti átkapcsolással).
- **Resource:** a kölcsönös kizárást biztosító objektum, a **priority ceiling protokollal**. Ez megakadályozza a prioritás inverziót és a holtpontot. A resource-ot a feladatnak meg kell szereznie (`GetResource`), majd el kell engednie (`ReleaseResource`).

- **Event:** csak extended taskokhoz használható, a task várakozhat rá (`WaitEvent`), másik task vagy ISR beállíthatja (`SetEvent`).

Az AUTOSAR OS **konformanciaosztályokat** definiál, amelyek a lehetséges funkciókészletet korlátozzák, hogy a gyártók kisebb, egyszerűbben tanúsítható kerneleket is készíthessenek. A legismertebb osztályok: BCC1, BCC2, ECC1, ECC2. A BCC1 csak basic taskokat enged, egyetlen task per prioritás, és nem enged event-eket. Az ECC2 extended taskokat, több taskot ugyanazon prioritáson, eventeket és resource-okat is támogat. Az AUTOSAR kiegészíti ezeket például a **Timing Protection** és **Memory Protection** osztályokkal.

A feladatütemezés a **fix prioritású, preemptív** modell szerint működik. A magasabb prioritású task megszakítja az alacsonyabbat. Az ütemező a ready taskok közül mindig a legmagasabb prioritásút futtatja. A konfiguráció statikus, így az ütemezhetőség offline elemzéssel (pl. válaszidő-analízis) ellenőrizhető. Ez kulcsfontosságú a funkcióbiztonság szempontjából.

24.3.6 3.3. Példa: egy SWC és RTE kapcsolat

Tegyük fel, hogy egy sebességkorlátozó funkciót szeretnénk megvalósítani. Az SWC neve `SpeedLimiter`. Bemenetei: aktuális sebesség (`VehicleSpeed`), kívánt maximális sebesség (`SpeedLimit`). Kimenete: korlátozott nyomaték (`TorqueLimit`). Az SWC portjai sender-receiver típusúak. Az RTE-hez a következő generált API-t használjuk:

```
/* SpeedLimiter.c - szemléltető vázlat, nem végrehajtandó kód */
#include "Rte_SpeedLimiter.h"

FUNC(void, SpeedLimiter_Code) SpeedLimiter_Compute(void)
{
    uint16_t vehicleSpeed = 0u;
    uint16_t speedLimit = 0u;
    uint16_t torqueLimit = 0u;

    Rte_Read_RxVehicleSpeed_VehicleSpeed(&vehicleSpeed);
    Rte_Read_RxSpeedLimit_SpeedLimit(&speedLimit);

    if (vehicleSpeed > speedLimit)
    {
        torqueLimit = (uint16_t)(speedLimit * 100u / vehicleSpeed);
    }
    else
    {
        torqueLimit = 100u; /* százalékos nyomatékkorlát */
    }

    Rte_Write_TxTorqueLimit_TorqueLimit(torqueLimit);
}
```

Az RTE generálja a `Rte_Read_RxVehicleSpeed_VehicleSpeed` és `Rte_Write_TxTorqueLimit_TorqueLimit` függvényeket. Az SWC kódja nem tudja, hogy a bemenet egy CAN üzenetből, egy ADC-ből vagy egy másik SWC-től származik. Az RTE gondoskodik az adatok átadásáról, a bufferelésről és az adatkonzisztenciáról. A Runnable-t egy taskhoz rendeljük, például:

```
Task: Task_10ms
Priority: 3
Schedule: 10 ms periódusú alarm
Runnable: SpeedLimiter_Compute
```

Az operációs rendszer az alarm lejártakor aktiválja a `Task_10ms` taskot, amely meghívja a Runnable-t. Ez a példa mutatja a rétegek együttműködését: az OS ütemezi a taskot, az RTE biztosítja a kommunikációt, az SWC a funkciót valósítja meg.

24.3.7 4. Adaptive AUTOSAR

A Classic AUTOSAR kiválóan alkalmas statikus, kemény valós idejű rendszerekre, de nem jó választás nagy számítási igényű, dinamikus funkciókhoz, például autonóm vezetés, szenzorfúzió, nagy felbontású térképek, gépi tanulás, vagy felhőkapcsolat. Ezek a funkciók nagy memóriát, MMU-t, processz-izolációt, dinamikus erőforrás-kezelést és gyakori szoftverfrissítést igényelnek. Ezt a szerepet tölti be az Adaptive AUTOSAR.

Az Adaptive platform az operációs rendszert POSIX szabványnak megfelelő **operációs rendszer interfészként** (Operating System Interface, OSI) definiálja. A gyakorlatban ez általában Linux, de bármely POSIX PSE51 vagy PSE54 kompatibilis kernel szóba jöhet. Az operációs rendszer biztosítja:

- **Processz- és szálkezelés:** minden adaptív alkalmazás külön processzben fut, saját virtuális címterével. Ez erős izolációt ad.
- **Memóriavédelem:** az MMU megakadályozza, hogy egy hibás alkalmazás más processzek memóriáját írja.
- **Ütemezés:** a Linux valós idejű képességei (pl. `SCHED_FIFO`, `SCHED_DEADLINE`) használhatók, de a determinizmus gyengébb, mint a Classic AUTOSAR OS esetén. Az Adaptive platform ezért elsősorban QM, ASIL A vagy B besorolású funkciókhoz ajánlott; a magasabb ASIL funkciók továbbra is Classic platformon futnak.
- **Kommunikáció:** az Adaptive platform `ara::com` könyvtárat használ, amely szolgáltatásorientált kommunikációt valósít meg olyan protokollok felett, mint a SOME/IP vagy DDS. A service discovery, publish/subscribe, request/response mintákat támogat.
- **Execution Management:** az alkalmazások életciklusát, indítását, leállítását, állapotkezelését végzi. Ez a rendszer a futási konfiguráció alapján indítja a processzeket.
- **State Management:** a rendszer állapotait (init, run, shutdown, diagnostic) kezeli, és a funkciók aktiválását vezérli.
- **Update and Configuration Management (UCM):** a szoftvercsomagok biztonságos telepítését, frissítését és visszagörgetését menedzseli. Ez az OTA frissítések alapja.
- **Persistency:** kulcs-érték tárolást biztosít az alkalmazások számára, amely a fájlrendszeren vagy flash memórián alapulhat.

Az Adaptive platformon az alkalmazások nem „taskokból” állnak, hanem **adaptív alkalmazásokból** (Adaptive Applications, AA), amelyek C++ osztályokkal implementált szolgáltatásokból és eseménykezelőkből épülnek fel. Az AA az ARA (AUTOSAR Runtime for Adaptive Applications) interfészeket használja, amelyek közül a legfontosabb az `ara::com`, `ara::exec`, `ara::per`, `ara::ucm`. Az alkalmazás és az operációs rendszer között itt is van egy RTE-szerű absztrakció, de az Adaptive platformon az RTE-t a `ara::runtime` könyvtár valósítja meg, nem generált C kód, hanem szabványos könyvtári interfész.

Például egy adaptív alkalmazás sebességadatot publikálhat:

```
// SpeedPublisher.cpp - szemléltető vázlat
#include <ara/com/com.h>
#include <ara/core/result.h>

int main()
{
    auto service = ara::com::ServiceHandle::FindService("VehicleSpeedService");
```

```
    if (!service) { return 1; }

    auto publisher = service->GetPublisher<uint16_t>("Speed");
    publisher->Publish(120u);
    // ...
    return 0;
}
```

Az Adaptive AUTOSAR célja tehát nem a mikrokontrolleres, kemény valós idejű világ kiváltása, hanem a nagyobb, dinamikusabb egységek integrációja a jármű fedélzeti hálózatába. A két platform együtt létezik: a Classic ECU-k a biztonságkritikus vezérlőfunkciókat, az Adaptive egységek a kényelmi, informatikai és nagy számításigényű asszisztens funkciókat látják el. A rendszertervezés során el kell dönteni, melyik funkció melyik platformra kerüljön, figyelembe véve a funkcionális biztonsági követelményeket, az erőforrásokat, a frissítési igényt és a költségeket.

24.3.8 5. Funkcióbiztonság és izoláció

A járműipari szoftverekkel szembeni egyik legfontosabb követelmény a funkcionális biztonság, amelyet az ISO 26262 szabvány ír le. A szabvány a rendszer hibáiból adódó kockázatokat ASIL (Automotive Safety Integrity Level) osztályokba sorolja: QM, ASIL A, ASIL B, ASIL C, ASIL D. Minél magasabb az ASIL, annál szigorúbb követelmények vonatkoznak a fejlesztésre, a verifikációra és a futás közbeni hibatűrésre.

Az AUTOSAR architektúra több mechanizmust is kínál a biztonsági követelmények teljesítésére:

- **Freedom from interference:** a különböző ASIL besorolású szoftverkomponensek nem zavarhatják egymást. A Classic platformon ezt a **memóriavédelem** (MPU) és az **időzítési védelem** (Timing Protection) biztosítja. A memóriavédelem megakadályozza, hogy egy alacsonyabb ASIL besorolású komponens egy magasabb ASIL besorolású komponens memóriáját írja. Az időzítési védelem pedig korlátozza a taskok futási idejét és a megszakítási terhelést, így egy hibás task nem tudja kiéheztetni a kritikus taskokat.
- **End-to-End (E2E) protection:** a kommunikációs csatornákon a hibaészleléshez az AUTOSAR E2E profilokat definiál, például CRC, számláló és timeout monitorozást. Az RTE hozzáfér ezekhez, de a biztonsági szempontú alkalmazásoknak explicit E2E ellenőrzést kell végezniük.
- **Watchdog:** a hardveres vagy szoftveres watchdog figyeli, hogy a kritikus taskok rendszeresen lefutnak-e. Ha nem, a rendszer biztonságos állapotba vált. Az AUTOSAR OS watchdog interfésze (`WdgM` – Watchdog Manager) a BSW része.
- **Safe state:** a rendszernek definiált biztonságos állapottal kell rendelkeznie, amelybe hiba esetén kerül. Például egy fékrendszer esetén a biztonságos állapot lehet a féknyomás csökkentése vagy a fékfény bekapcsolása, de a konkrét funkciótól függ.
- **ASIL dekompozíció:** egy magas ASIL követelmény felbontható alacsonyabb ASIL elemekre, ha a redundancia és a függetlenség bizonyított. Például két független ASIL B(A) komponens együttesen teljesíthet egy ASIL C(D) követelményt. Ehhez az izolációt hardveresen és szoftveresen is garantálni kell.

A Classic AUTOSAR OS statikus konfigurációja nagy előny a biztonság szempontjából: a feladatok, prioritások, erőforrások és ütemezési táblázatok a fejlesztés során rögzítettek, így a legrosszabb végrehajtási idő (WCET) elemzése és a válaszidő-számítás elvégezhető. Az OS nem futtat dinamikus memóriakezelést, nincs heap allokáció futásidőben, ami kiküszöböli a memóriaszivárgás és a nem determinisztikus viselkedés egy részét.

Az Adaptive platformon a biztonsági izolációt a processzmodell és a Linux kerneli mechanizmusai (namespaces, cgroups, capabilities) adják. Azonban a teljes Linux kernel tanúsítása magas ASIL

szintre nehéz, ezért az Adaptive platformot jellemzően QM vagy alacsonyabb ASIL funkciókra használják, míg a kritikus funkciók Classic platformon maradnak. Újabban megjelennek mixed-criticality megoldások, például hypervisorral, ahol a Classic AUTOSAR OS és a Linux párhuzamosan fut ugyanazon a többmagos SoC-n.

24.3.9 6. Frissítés és szoftveréletciklus

A járműipari szoftverfrissítés hagyományosan a szervizben, diagnosztikai eszközzel történt, és a teljes ECU firmware újraprogramozását jelentette. A modern, hálózatba kapcsolt járműveknél azonban elvárás a vezeték nélküli frissítés (OTA, Over-The-Air). Az AUTOSAR ezt a területet is szabványosítja.

A frissítés követelményei:

- **Biztonságos boot:** a frissített szoftvert digitálisan alá kell írni, és a boot során ellenőrizni kell az integritást és a hitelességet. Az AUTOSAR Secure Boot a hardveres gyökéren (pl. HSM – Hardware Security Module) alapul.
- **Csomagkezelés:** a frissítőcsomag tartalmazza a szoftvert, a verziót, a függőségeket és a hitelesítési adatokat. Az UCM (Update and Configuration Management) az Adaptive platformon ezt kezeli.
- **Fail-safe frissítés:** a frissítés nem teheti használhatatlanná az ECU-t. A tipikus megoldás az **A/B partíció** vagy a **rollback mechanizmus**: a régi szoftver megmarad, és ha az új nem indul el egy meghatározott időn belül, a rendszer automatikusan visszaáll.
- **Kompatibilitás:** a frissítés nem változtathatja meg a kommunikációs interfészeket olyan módon, hogy azok ne legyenek kompatibilisek a régi ECU-kkel. Az AUTOSAR az interfészverziókat és a konfigurációt szigorúan verziózottan kezeli.
- **Biztonsági frissítések:** a sérülékenységek javítása kiemelt, ezért a frissítési infrastruktúrának gyorsnak és megbízhatónak kell lennie. A járműipari szabványok (pl. UNECE WP.29) előírják a szoftverfrissítések nyomon követhetőségét és a járművek frissítési állapotának dokumentálását.

Az Adaptive AUTOSAR itt nagy előnyt jelent: a processz-alapú modell lehetővé teszi az egyes alkalmazások külön-külön történő frissítését anélkül, hogy a teljes rendszert újra kellene indítani. A Classic platformon a frissítés gyakran a teljes ECU kép cseréjét jelenti, ami hosszabb szervizidőt és nagyobb kockázatot jelent. Ezért a járműipari fejlesztők egyre gyakrabban az Adaptive platformot választják a nem biztonságkritikus, de dinamikusan frissíthető funkciókhoz, és a Classic platformot tartják meg a stabil, biztonságkritikus alapfunkciókhoz.

24.3.10 7. Labor – AUTOSAR-rétegek és SWC-vázlat

A labor célja, hogy a tanult rétegmodellt és RTE kapcsolatot egy minimális, reprodukálható vázlaton keresztül alkalmazzuk. A labor nem igényel valódi ECU-t, elegendő egy emulált környezet és egy jegyzetfüzet, amelyben a rétegek, a konfiguráció és a mérési napló szerepel.

24.3.11 7.1. Környezet és konfiguráció

- **Hoszt:** Linux x86_64, QEMU 8.x (emulált ARM Cortex-R5, `qemu-system-arm -machine xlnx-zcu102` vagy hasonló), `gcc-arm-none-eabi` 13.x, Python 3.11 a jegyzetfüzethez.
- **Cél:** egyetlen virtuális ECU 64 MiB RAM-mal, 4 MiB flash-sel, egy 1 ms tick timerrel, egy virtuális CAN interfésszel.

- **Emulált AUTOSAR OS:** használjunk egy egyszerű, statikus ütemezésű kernelt, amely a laborcsomagban `os092_rtos_sim` néven található. Ez nem teljes AUTOSAR OS, de megvalósítja a task, alarm, resource és RTE hívás alapjait. A forráskód szintetikus, csak oktatási célra.

A laborcsomag fájljai:

```
os092_lab/
  README.md
  swc/
     SpeedLimiter.c
     Rte_SpeedLimiter.h
  config/
     Os_TaskConfig.xml
     Os_AlarmConfig.xml
     Rte_Config.xml
  rte/
     Rte_Generated.c
  Makefile
  measurement/
      os092_measurement.csv
```

24.3.12 7.2. Lépések

1. **Rétegdiagram készítése:** rajzoljon rétegdiagramot (ábra típusa: `layers`), amely az alkalmazási réteget (SpeedLimiter SWC), az RTE-t, a BSW Services-t (OS, COM), az ECU Abstraction-t és a MCAL-t mutatja. A diagramon jelölje a hívási irányokat. A jelmagyarázat: nyíl felfelé = az alsó réteg interfésze, nyíl lefelé = azonban a felső réteg az alsó réteg interfészét hívja. A feltételjegyzet: csak a szomszédos rétegek közötti hívás engedélyezett.

2. **SWC vázlat megvalósítása:** a fenti `SpeedLimiter.c`-hez hasonlóan készítsen egy `SpeedLimiter_Compute` Runnable-t, amely beolvas két uint16_t értéket, kiszámít egy korlátozó százalékot, és kiír egy uint16_t eredményt. A kód ne tartalmazzon hardverregiszter-írást vagy CAN hozzáférést.

3. **OS konfiguráció:** hozzon létre egy statikus task-konfigurációt XML-ben:

```
<!-- Os_TaskConfig.xml - szemléltető konfiguráció -->
<OsTaskConfig>
  <Task>
    <Name>Task_10ms</Name>
    <Priority>3</Priority>
    <Schedule>FULL</Schedule>
    <Activation>1</Activation>
    <Runnable>SpeedLimiter_Compute</Runnable>
  </Task>
  <Alarm>
    <Name>Alarm_10ms</Name>
    <Counter>SystemCounter</Counter>
    <PeriodMs>10</PeriodMs>
    <ActivatesTask>Task_10ms</ActivatesTask>
  </Alarm>
</OsTaskConfig>
```

4. **RTE generálás:** a laborcsomag `rte_gen.py` szkriptje beolvassa a `Rte_Config.xml`-t és létrehozza a `Rte_Generated.c`-t, amely a `Rte_Read_RxVehicleSpeed_VehicleSpeed` és `Rte_Write_TxTorqueLimit_TorqueLimit` függvényeket a szimulált CAN bufferhez köti.

5. **Build és futtatás:**

```
make clean
make
qemu-system-arm -machine xlnx-zcu102 -m 64M -kernel os092.elf \
  -serial mon:stdio -d os092_trace.log
```

6. **Mérési napló:** a QEMU nyomkövetésből és a szimulátor belső számlálóiból rögzítse a következő mérőszámokat:

Metrika	Módszer	Várt érték	Mért érték	Megjegyzés
Task_10ms periódus jitter	alarm beérkezési időkülönbség – 10 ms	< 100 µs	85 µs	a szimulátor órajel 1 MHz
SpeedLimiter_Compute futási idő	ciklusszámláló / órajel	< 500 µs	212 µs	nincs osztás optimalizálva
Rte_Read hívás késleltetése	RTE belépési pont időkülönbség	< 10 µs	4 µs	bufferelt, memóriás
Rte_Write hívás késleltetése	hasonlóan	< 10 µs	5 µs	
Task átkapcsolási idő	context switch nyomkövetés	< 30 µs	18 µs	preemptív

A mért és várt értékeket külön oszlopban kell szerepeltetni, és fel kell tüntetni, hogy a mérés emulált környezetben történt, így valós hardveren más érték várható. A mérési naplót CSV-ben is el kell menteni.

7. **Hibainjektálás:** próbálja ki, mi történik, ha a `SpeedLimiter_Compute` túllép egy konfigurált maximális futási időt (pl. 2 ms). Az OS-nek **Timing Protection** esetén meg kell állítania a taskot, és be kell jelentenie egy hibát. Ha ezt nem konfigurálta, a task egyszerűen lefut, és a jitter nő. Végezze el mindkét esetet, és dokumentálja a különbséget.

24.3.13 7.3. Ellenőrzés és gyakori hibák

A labor során figyelje meg, hogy a következő hibák előfordulhatnak:

- Az SWC közvetlenül hívja a `Can_Write`-ot vagy egy hardverregisztert ír. Ez sérti a rétegmodellt, és a kód nem lesz hordozható. Megoldás: minden kommunikáció az RTE-n keresztül.
- A task prioritása nem megfelelő, így a 10 ms-os task késik. Az ütemezhetőséget offline elemzéssel ellenőrizni kell.
- Nincsenek erőforrás-zárak, így az osztott adatok versenyhelyzetbe kerülnek. Megoldás: `GetResource`/`ReleaseResource` párok a kritikus szakaszokban.
- A mérési naplóban nem választják szét a mért és várt értéket, vagy nem adják meg a mérési módszert.

A labor eredményeként a hallgató megérti, hogyan kapcsolódik az alkalmazási réteg az OS-hez és a kommunikációs stackhez, és hogyan biztosítható a determinisztikus viselkedés.

24.3.14 8. Tipikus hibák és megoldások

Az AUTOSAR projektekben visszatérő hibák:

1. **RTE megkerülése:** az SWC közvetlenül hívja a BSW szolgáltatásokat vagy a hardver illesztőprogramokat. Ez szétzilálja az absztrakciót, és az SWC-t függővé teszi egy adott ECU-től. A javítás: az összes adat- és szolgáltatáscsere az RTE API-n keresztül, a generált kód használatával.
2. **Nem megfelelő ASIL dekompozíció:** két, alacsonyabb ASIL besorolású komponenst együtt magasabb ASIL funkcióként kezelnek, de nem biztosítják a függetlenséget. Például ugyanazon a processzormagon futnak, osztott memóriával, időzítésvédelem nélkül. A javítás: MPU, időzítési védelem, külön processzmag, vagy a dekompozíció felülvizsgálata.
3. **Időzítési védelem hiánya:** a Classic AUTOSAR OS-ben a Timing Protection konfiguráció elhagyása lehetővé teszi, hogy egy hibás, alacsony prioritású task túllépje a futási idejét, és késleltesse a magas prioritású, biztonságkritikus taskot. A megoldás a futási idő, a megszakítási terhelés és az aktiválási gyakoriság korlátozása.
4. **Nincs fail-safe frissítés:** az OTA update során nem terveznek rollback mechanizmust, így egy sikertelen frissítés használhatatlanná teszi az ECU-t. A megoldás A/B partíció, ellenőrzött indítás és automatikus visszagörgetés.
5. **Az Adaptive és Classic platform nem megfelelő szétválasztása:** minden funkciót Adaptive platformra helyeznek, mert „modern”, de a kemény valós idejű követelményeket a Linux nem tudja garantálni. Vagy fordítva, a nagy sávszélességű szenzorfúziót Classic-ra teszik, ahol a processzor és a memória nem elegendő. A helyes megközelítés a követelmények és az ASIL besorolás alapján történő platformválasztás.

24.3.15 9. Önellenőrző kérdések

1. **Magyarázza el, mi a különbség a Classic és Adaptive AUTOSAR operációs rendszer modellje között!** Térjen ki az ütemezésre, a memóriavédelemre, a konfiguráció időpontjára és a tipikus alkalmazási területekre.
2. **Hogyan biztosítja az RTE a sender-receiver kommunikáció adatkonzisztenciáját?** Milyen szerepe van a bufferelésnek és az adatfrissítési mintáknak?
3. **Miért rugalmasabb az Adaptive AUTOSAR a szoftverfrissítéseknél, mint a Classic AUTOSAR?** Milyen veszélyei vannak ennek a rugalmasságnak a biztonságkritikus funkciókra nézve?
4. **Egy AUTOSAR OS-ben a `Task_10ms` prioritása 3, futási ideje 300 µs, periódusa 10 ms. Egy másik, szintén 10 ms periódusú task prioritása 2, futási ideje 800 µs. Számítsa ki, mennyi a válaszidő a magasabb prioritású taskra, ha a rendszerben egy 100 µs-os megszakítás is fut 1 ms periódussal!** (Feltételezze, hogy a megszakítás prioritása magasabb a taskoknál.) Ellenőrizze az ütemezhetőséget.

A kérdésekre adott helyes válaszok a fejezet szövegében megtalálhatók; a számítási feladathoz a válaszidő-analízis alapösszefüggéseit kell alkalmazni.

24.3.16 10. Összefoglalás, kitekintés

Az AUTOSAR a járműipari szoftverarchitektúra de facto szabványa. A Classic Platform az OSEK/VDX-alapú statikus operációs rendszerrel, valamint az RTE-vel biztosítja a kemény valós idejű, biztonságkritikus funkciók hordozhatóságát és integrálhatóságát. Az Adaptive Platform a POSIX alapú, dinamikus, szolgáltatásorientált világot hozza el a nagy számítási igényű, frissíthető alkalmazások számára. A két platform együttesen fedi le a jármű fedélzeti hálózatának heterogén igényeit.

A funkcióbiztonság és a frissítés a tervezés központi elemei. A rétegelt modell, a statikus OS konfiguráció, a memória- és időzítésvédelem, valamint az E2E mechanizmusok együttesen teszik lehetővé az ASIL követelmények teljesítését és a biztonságos OTA frissítést. A labor során látott SWC-vázlat és rétegmodell jó kiindulópont a következő egységhez, amely az RTOS-minták (watchdog, queue, event flag) felé vezet tovább.

A konkrét AUTOSAR kiadások – például a Classic Platform R22-11 vagy az Adaptive Platform R23-11 – pontos specifikációit mindig a kiadó hivatalos dokumentációjában ellenőrizze, mert a verziófüggő részletek, interfésznevek és konfigurációs sémák változhatnak. Ugyanez vonatkozik az eszközgyártói implementációkra és a QNX, VxWorks vagy Linux kernel valós idejű képességeire is.

24.4 RTOS-minták: watchdog, queue és event flag

Az RTOS-alapú beágyazott fejlesztésben a hangsúly ritkán magán a kernel belső implementációján van, sokkal inkább azon, hogy a fejlesztő hogyan bontja feladatokra a rendszert, és hogyan szervezi a feladatok közötti együttműködést. A watchdog, az üzenetsor, az eseménybit-csoport és a mailbox olyan újrafelhasználható minták, amelyek a legtöbb valós idejű rendszerben megjelennek. A watchdog a rendszer életben maradását felügyeli, az üzenetsor a termelő–fogyasztó láncokat szervezi, az eseménybit-csoport több feltétel együttes teljesülését kezeli, a mailbox pedig vezérlő jellegű, általában egyetlen üzenet továbbítására szolgál. Ebben az alfejezetben ezeket a mintákat a tervezési döntésekkel, a tipikus hibákkal és egy reprodukálható laborgyakorlattal együtt mutatjuk be.

A beágyazott feladatok tipikus mintái

Egy RTOS-alkalmazás tervezésekor a feladatok nem egyszerűen „funkcionális blokkok”, hanem időzítési, szinkronizációs és erőforrás-tulajdonjogi szerepekkel is rendelkeznek. A gyakori minták a következők:

1. **Periodikus feladat** – időzítő vagy `vTaskDelayUntil` szerint fut, fix periódussal. Például egy érzékelő 10 ms-enként mintát vesz és üzenetsorba ír.
2. **Eseményvezérelt feladat** – nem periódus, hanem esemény, például megszakítás, üzenetsor-érkezés vagy event flag bit ébreszti. Például egy UART-vevő feladat blokkolva vár a RX-queue-ra.
3. **Termelő–fogyasztó** – egy vagy több termelő üzenetsoron vagy osztott memórián keresztül ad át adatot egy fogyasztónak. Az üzenetsor pufferként telítődési és kiürülési viselkedést is mutat.
4. **Állapotgép** – a feladat egy állapotváltozót tart karban, és az események hatására vált állapotot. Ez különösen hasznos protokoll- és eszközvezérlőkben.
5. **Watchdog monitor** – egy külön feladat gyűjti a többi feladat liveness jelzéseit, és csak akkor frissíti a hardveres watchdogot, ha minden kritikus jel megérkezett.

6. **Erőforrás-őr** – egy feladat vagy könyvtári réteg kizárólagos hozzáférést ad egy perifériához, például SPI buszhoz, és mutex/szemaforral sorosítja a hozzáférést.
7. **Megszakítás + halasztott feldolgozás** – az ISR csak a minimális hardverkezelést végzi, az adatot üzenetsorba vagy event flaggel jelzett pufferbe teszi, a tényleges feldolgozás pedig egy normál feladatban történik.

Ezek a minták gyakran kombinálódnak. Egy tipikus érzékelőrendszerben a periodikus termelők üzenetsorba írnak, egy fogyasztó szűri és továbbítja az adatot, egy watchdog feladat pedig a periodikus termelők szívveréseit figyeli. A tervezés első lépése ezért nem a konkrét API-hívás, hanem annak eldöntése, hogy melyik feladat milyen időzítéssel, milyen prioritással és milyen kommunikációs csatornán keresztül működik.

Watchdog: az életben tartás őre

A watchdog timer (WDT) egy hardveres számláló, amelyet a szoftvernek rendszeresen frissítenie kell. Ha a frissítés elmarad, a WDT túlcsordul, és általában rendszer-resetelést vagy megszakítást vált ki. A watchdog célja az, hogy a szoftver akkor is visszakerüljön egy ismert állapotba, ha egy feladat végtelen ciklusba kerül, egy magasabb prioritású feladat kiéhezteti a többit, vagy egy hardverhiba miatt a vezérlés rossz helyre kerül.

A legegyszerűbb, de gyakran hibás megközelítés az, ha egy feladat közvetlenül, a saját ciklusában „kutyázza” a hardveres WDT-t. Ez nem véd akkor, ha a feladat maga akad el, miközben a WDT-t frissítő kódsor továbbra is futhat. Helyesebb egy **szoftveres watchdog réteg** bevezetése, amelyben a kritikus feladatok egy-egy liveness jelzést adnak, és csak egy központi watchdog feladat frissíti a hardveres WDT-t, ha minden jelzés megérkezett.

Az alábbi pszeudokód FreeRTOS-szerű API-val mutat egy ilyen réteget. A `system_events` eseménycsoport bitjei a feladatok szívveréseit jelzik:

```
/* watchdog_task - FreeRTOS-szerű pszeudokód */
void watchdog_task(void *param)
{
    TickType_t last_wake = xTaskGetTickCount();
    EventBits_t required = TASK_A_HEALTHY | TASK_B_HEALTHY |
                           SENSOR_HEALTHY;

    for (;;) {
        /* Várakozás legfeljebb 100 ms-ig minden szükséges bitre.
           A pdTRUE itt azt jelenti, hogy a biteket kilépéskor töröljük,
           így a következő periódusban újra kell érkezniük. */
        EventBits_t bits = xEventGroupWaitBits(
            system_events,
            required,
            pdTRUE,              /* clear on exit */
            pdTRUE,              /* wait for all bits */
            pdMS_TO_TICKS(100));

        if ((bits & required) == required) {
            hw_watchdog_kick();  /* hardveres WDT frissítése */
        } else {
            log_error("watchdog: hianyzik legalabb egy heartbeat\n");
            /* nincs kick, a hardver WDT resetel, ha a helyzet tartós */
        }

        vTaskDelayUntil(&last_wake, pdMS_TO_TICKS(200));
    }
}
```

A figyelt feladatok például így adhatnak szívverést:

```
void sensor_task(void *param)
{
    for (;;) {
        /* periodikus munka */
        sample_sensor();

        /* liveness jelzés a watchdog felé */
        xEventGroupSetBits(system_events, SENSOR_HEALTHY);

        vTaskDelayUntil(&last_wake, pdMS_TO_TICKS(20));
    }
}
```

A watchdog tervezésének legfontosabb paramétere az **időzítési ablak**. A hardveres WDT timeout értékét úgy kell megválasztani, hogy normál működés közben a watchdog feladat biztosan frissíteni tudja, de meghibásodás esetén elég gyorsan reagáljon. Ha a monitorozott lánc legrosszabb válaszideje $R_{\max}$, akkor a hardveres timeout T_{wdt} értékére praktikus felső határ:

$$T_{wdt} > k \cdot R_{\max},$$

ahol k jellemzően 1,5 és 2 közötti biztonsági tényező. Emellett a watchdog feladat periódusának úgy kell illeszkednie, hogy a legrosszabb esetben is legyen ideje a hiányzó jelzés észlelésére, mielőtt a hardver lejár. A fenti példában a watchdog 200 ms-os periódussal fut, 100 ms-os várakozási ablakkal; a hardver WDT timeout így nem lehet például 120 ms-nál rövidebb, mert a watchdog feladat maga is legfeljebb 100 ms-ig blokkol.

Két további változatot érdemes ismerni. Az **ablakos watchdog** (windowed WDT) megköveteli, hogy a frissítés ne túl korán és ne túl későn érkezzen, csak egy adott időablakon belül. Ez azért fontos, mert ha egy hibásan gyorsan futó feladat túl gyakran frissíti a WDT-t, az elfedheti, hogy a feladat ciklusa összeomlott, és például a periódusideje a töredékére csökkent. Az ablakos WDT ezt a hibát is leleplezi. A **többszintű watchdog** pedig szoftveres és hardveres szinteket kombinál: a szoftveres szint gyűjti a liveness jeleket, a hardveres szint a szoftveres réteg működőképességét felügyeli. Így a hibák nagyobb osztálya lefedhető.

A watchdog hibái közé tartozik az, ha a kickelést megszakításkezelőből végzik. Ha a fő feladat elakad, de az ISR továbbra is fut, a WDT-t az ISR frissítheti, így a hiba nem derül ki. Helyes megoldás, ha a kick a feladatkörnyezetből, a liveness jelek ellenőrzése után történik. Szintén hibás, ha minden feladat közvetlenül frissíti ugyanazt a WDT-t, mert ilyenkor nincs központi döntés arról, hogy a rendszer egésze egészséges-e. Végül a túl rövid WDT timeout a normál, átmeneti terhelés alatt is resetet okozhat, a túl hosszú pedig késlelteti a hibából való kilábalást.

Üzenetsor (queue) a termelő–fogyasztó mintában

Az RTOS-ek üzenetsora az egyik leggyakoribb feladatközi kommunikációs eszköz. A queue lehetővé teszi, hogy egy vagy több termelő adatokat helyezzen egy FIFO-pufferbe, egy vagy több fogyasztó pedig onnan kivegye azokat. A legtöbb RTOS a queue-hoz blokkoló és nem blokkoló küldési/fogadási változatot is biztosít, timeout paraméterrel. A blokkolás azt jelenti, hogy ha a queue tele van, a termelő felfüggesztődik, amíg hely nem szabadul; ha üres, a fogyasztó várakozik, amíg elem nem érkezik.

A FreeRTOS-szerű API-ban egy mintaadatokból álló queue például így hozható létre:

```
typedef struct {
    uint32_t sensor_id;
    uint32_t value;
    uint32_t timestamp_ms;
} sensor_sample_t;
```

```
QueueHandle_t sensor_queue;

void create_sensor_queue(void)
{
    sensor_queue = xQueueCreate(
        8,                      /* elemek maximális száma */
        sizeof(sensor_sample_t)  /* elem mérete */
    );
    if (sensor_queue == NULL) {
        /* nem sikerült a memóriafoglalás - hibakezelés */
        enter_safe_state();
    }
}
```

A termelő lehet egy periodikus feladat vagy egy ISR. Az ISR-ből a `xQueueSendFromISR` változatot kell használni, és a `portYIELD_FROM_ISR` makróval jelezhető, ha a művelet magasabb prioritású feladatot ébresztett:

```
void sensor_isr(void)
{
    BaseType_t higher_priority_woken = pdFALSE;
    sensor_sample_t sample = {
        .sensor_id = 1,
        .value = read_adc(),
        .timestamp_ms = get_tick_ms()
    };

    xQueueSendFromISR(sensor_queue, &sample, &higher_priority_woken);
    portYIELD_FROM_ISR(higher_priority_woken);
}
```

A fogyasztó blokkoló fogadással vár az adatokra, és timeouttal kezeli a hiányzó mintát:

```
void aggregator_task(void *param)
{
    sensor_sample_t sample;

    for (;;) {
        BaseType_t rc = xQueueReceive(
            sensor_queue,
            &sample,
            pdMS_TO_TICKS(250));  /* legfeljebb 250 ms várakozás */

        if (rc == pdTRUE) {
            process_sample(&sample);
        } else {
            log_error("sensor queue timeout\n");
            /* itt lehet részleges hibakezelés, pl. hiba-számláló */
        }
    }
}
```

A queue méretezésénél a termelési és fogyasztási sebesség viszonyából kell kiindulni. Ha a termelő periodikus és a fogyasztó legrosszabb válaszideje nagyobb, mint a termelő periódusa, akkor a queue-nak legalább annyi elemet kell tárolnia, amennyi a legrosszabb fogyasztási késleltetés alatt keletkezik. Formálisan, ha a termelő periódusa T_p, a fogyasztó legrosszabb feldolgozási ideje C_c, és a fogyasztó legrosszabb késleltetése R_c, akkor a minimális pufferméret:

$$N_{min} \geq \left\lceil \frac{R_c}{T_p} \right\rceil + 1,$$

ahol a +1 biztonsági elem a mérési és ütemezési jittert fedezi. A gyakorlatban érdemes a minimumnál nagyobb, például kétszeres puffert választani, de a túl nagy queue sem mindig jó: megnöveli a memóriahasználatot és a legrosszabb késleltetést, mert a fogyasztó a régi elemeket is fel kell dolgozza.

A queue használatakor tipikus hiba a **tele queue-ba történő blokkoló küldés magas prioritású feladatból**. Ha a termelő prioritása magasabb, mint a fogyasztóé, a termelő addig fog várakozni, amíg a fogyasztó nem fut, de a fogyasztó nem kap CPU-t, mert a magasabb prioritású termelő továbbra is fut. A helyzet prioritásinverzióhoz vezet. A megoldás lehet timeout használata, a fogyasztó prioritásának emelése, vagy a queue méretének növelése. Emellett figyelni kell arra, hogy az ISR-ből soha ne használjunk blokkoló küldést, mert megszakítás-kontextusból nem lehet felfüggesztődni.

Event flag: több feltétel együttes kezelése

Az event flag (eseménybit-csoport) olyan szinkronizációs objektum, amely bitek halmazát tárolja. Egy feladat beállíthat egy vagy több bitet, egy másik feladat pedig várakozhat arra, hogy a kiválasztott bitek együttesen vagy külön-külön be legyenek állítva. A várakozás lehet „AND” típusú, amikor az összes megadott bitnek be kell állnia, vagy „OR” típusú, amikor elég bármelyiknek. A `clear on exit` opció azt jelenti, hogy a várakozás teljesülésekor a kernel automatikusan törli azokat a biteket, amelyekre a feladat várt; így a jelzés nem marad „lebegve”.

Egy tipikus felhasználás a szenzoros rendszerek inicializálása. Három különböző szenzor inicializáló feladata eltérő idő alatt végez, de a fő adatgyűjtő feladat csak akkor indulhat, ha mindhárom kész. Egyszerű, ha a szenzorok bitmapet állítanak be, a fő feladat pedig AND-várakozással vár a három bitre:

```
#define GYRO_READY   (1u << 0)
#define MAG_READY    (1u << 1)
#define BARO_READY   (1u << 2)
#define IMU_RUN      (1u << 3)

EventGroupHandle_t imu_events;

void init_task(void *param)
{
    EventBits_t bits = xEventGroupWaitBits(
        imu_events,
        GYRO_READY | MAG_READY | BARO_READY,
        pdTRUE,              /* clear on exit */
        pdTRUE,              /* wait for all bits */
        pdMS_TO_TICKS(1000));

    if ((bits & (GYRO_READY | MAG_READY | BARO_READY)) ==
        (GYRO_READY | MAG_READY | BARO_READY)) {
        /* mindhárom szenzor inicializálódott */
        xEventGroupSetBits(imu_events, IMU_RUN);
    } else {
        log_error("szenzor inicializalas timeout\n");
        enter_safe_state();
    }
}
```

Az egyes szenzor-inicializáló feladatok a végükön beállítják a saját bitjüket:

```
void gyro_init_task(void *param)
{
    /* hardver-inicializálás */
    gyro_reset();
    gyro_configure();
```

```
    xEventGroupSetBits(imu_events, GYRO_READY);

    vTaskDelete(NULL);  /* az inicializáló feladat kilép, ha nincs más dolga */
}
```

Az event flag fontos tulajdonsága, hogy **nem számlálja az eseményeket**. Ha egy termelő kétszer beállítja ugyanazt a bitet, mielőtt a fogyasztó várakozna, a fogyasztó csak egyszer látja, és a `clear on exit` törli a bitet. Ezért az event flag nem alkalmas olyan jelzések továbbítására, ahol minden egyes esemény számít. Például ha egy UART-vevő minden beérkező bájtnál beállítana egy bitet, akkor a bájtok közötti különbség elveszne. Ilyenkor üzenetsort kell használni. Az event flag erőssége az, hogy több feltételt alacsony memóriahasználattal, gyorsan kezel, és egyetlen hívásban több eseményt is ellenőriz.

Gyakori hiba az, ha a fejlesztő elfelejti törölni a biteket, vagy nem használja a `clear on exit` opciót, és a fogyasztó a régi, már teljesített feltételt újra érvényesnek látja. Egy másik hiba az, ha az event flaget adatszintű kommunikációra használják, és az események elvesznek. Végül a timeout nélküli, végtelen ideig tartó várakozás is kockázatos, mert ha egy szenzor inicializálása hardverhiba miatt elakad, a rendszer soha nem lép tovább. A fenti példa ezért 1000 ms-os timeoutot használ, és a hibát biztonságos állapotba lépéssel kezeli.

Mailbox: egyetlen üzenet, vezérlő szerep

A mailbox fogalma RTOS-enként kissé eltér. Sok rendszerben a mailbox olyan, mint egy **egyelemű üzenetsor**, amely nem adatfolyamot, hanem vezérlő üzeneteket továbbít. A termelő egy pointert helyez el, és ha a mailbox már foglalt, a régi üzenet felülíródhat vagy a termelő blokkolhat. A fogyasztó kiveszi a pointert, és a tartalmát feldolgozza. A mailbox ezért elsősorban parancskiadásra, konfigurációra vagy erőforrás-átadásra való, nem nagy sebességű adatfolyamra.

Zephyr környezetben a `k_msgq` az ajánlott üzenetsor-megoldás, míg a `k_mbox` örökölt, mailbox-szerű API-ként ismert; a pontos elnevezés és viselkedés implementációfüggő, ezért mindig ellenőrizni kell az adott kernelverzió dokumentációját. A lényeg a mintában keresendő: a mailbox üzenet tipikusan egy struktúrára mutató pointer, és a tulajdonjog a küldőtől a fogadóhoz kerül.

Pszeudokódban egy parancs mailbox így nézhet ki:

```
/* Zephyr-szerű, mailbox jellegű pszeudokód; az API-név ellenőrizendő */
struct command_message {
    uint32_t command_id;
    uint32_t parameter;
    uint32_t status;
};

struct k_mbox command_mbox;

void command_sender(void)
{
    static struct command_message cmd;
    cmd.command_id = CMD_CONFIGURE;
    cmd.parameter = 0x20;
    k_mbox_put(&command_mbox, &cmd, K_MSEC(100));
}

void command_executor(void)
{
    for (;;) {
        struct command_message *cmd =
            k_mbox_get(&command_mbox, K_SECONDS(10));
        if (cmd != NULL) {
            execute_command(cmd);
```

```
        } else {
            handle_mailbox_timeout();
        }
    }
}
```

A mailbox használatánál különösen fontos a **pointer érvényességi ideje**. Ha a termelő a saját stackjén lévő objektum címet küldi el, akkor a stack tartalma a feladat kontextusváltásakor megsemmisülhet, és a fogyasztó érvénytelen memóriát olvashat. A helyes megoldás statikus, globális vagy heapről foglalt puffert használni, és a tulajdonjogot explicit módon átadni. Ugyancsak ügyelni kell arra, hogy a mailbox felülírási szabálya implementációfüggő: van, ahol a régi üzenet elveszik, máshol a termelő blokkol. A biztonságos tervezéshez a timeout mindig legyen megadva.

Prioritás, időzítés és hibakezelés

A fenti minták csak akkor működnek megbízhatóan, ha a prioritásokat és az időzítési paramétereket tudatosan választjuk meg. A prioritások hozzárendelésének két leggyakoribb alapelve a **sebesség szerinti monoton** (rate-monotonic, RMS) és a **kritikusság alapú** hozzárendelés.

A sebesség szerinti monoton szabály szerint a rövidebb periódusú feladat magasabb prioritást kap. Ha három periodikus feladatunk van $T_1 = 10$ ms, $T_2 = 20$ ms, $T_3 = 50$ ms periódussal, akkor a prioritási sorrend $P_1 > P_2 > P_3$. Ez a szabály jól működik, ha a feladatok határideje egyenlő a periódussal, és a CPU kihasználtság nem túl magas. A kritikusság alapú hozzárendelésnél viszont a biztonsági vagy üzemviteli szempontból legfontosabb feladat kapja a legmagasabb prioritást, függetlenül a periódustól. Például egy motorvészleállító feladat akkor is magasabb prioritású lehet, ha ritkán fut, mert a késleltetése nem elfogadható.

A watchdog tervezésénél a prioritásnak tükröznie kell a felügyeleti szerepet: a watchdog feladatot a felügyelt feladatok **fölé** érdemes helyezni, vagy legalább olyan szintre, hogy a legrosszabb terhelés alatt is időben megkapja a CPU-t. Ha a watchdog alacsonyabb prioritású, mint a felügyelt fogyasztó, akkor a fogyasztó kiéheztetheti, és a watchdog akkor is lejár, amikor a rendszer egyébként nem hibás. A gyakorlatban a watchdog feladat gyakran a legmagasabb prioritású feladatok közé kerül, rövid futási idővel.

Az időzítési paramétereket a következő képletek kötik össze:

- A termelő periódusa T_p, a fogyasztó legrosszabb válaszideje R_c.
- A queue minimális mérete az előzőek szerint $N_{min} = \lceil R_c / T_p \rceil + 1$.
- A watchdog hardveres timeoutja $T_{wdt} > k \cdot R_{\max}$, ahol $R_{\max}$ a leghosszabb felügyelt válaszidő.
- A watchdog feladat periódusa T_{wd} úgy választandó, hogy a legrosszabb esetben is legyen ideje észlelni a hiányzó jelzést: $T_{wd} \le T_{wdt}/2$ körüli érték, a gyakorlatban T_{wd} tipikusan T_{wdt} fele vagy harmada.

A hibakezelésben a RTOS-minták több szintet igényelnek:

1. **Helyi hibakód** – minden API-hívás visszatérési értékét ellenőrizni kell. A queue fogadásánál a timeout nem feltétlenül végzetes, de jelezni kell.
2. **Részleges degradáció** – ha egy szenzor nem válaszol, a rendszer tovább működhet a többi szenzorral, de ezt explicit állapotban kell nyilvántartani.
3. **Biztonságos állapot** – súlyos hiba esetén a rendszer egy előre definiált, biztonságos állapotba lép, például leállítja az aktuátorokat, majd megkísérli a helyreállítást.
4. **Watchdog általi reset** – ha a szoftver hibakezelés sem tudja helyreállítani a rendszert, a hardveres WDT resetje az utolsó védelmi vonal.

5. **Naplózás és reprodukció** – a hiba bekövetkezésekor a naplónak tartalmaznia kell a feladatállapotot, a queue töltöttségét, az event flag biteket és az időbélyeget.

A következő táblázat a leggyakoribb hibákat és a helyes megoldásokat foglalja össze:

Hiba	Következmény	Helyes megoldás
Watchdog kickelés ISR-ből	A fő feladat elakadása nem derül ki	Kickelés csak feladatkörnyezetből, liveness ellenőrzés után
Watchdog timeout túl rövid	Normál terhelés alatt is reset	$T_{wdt} > 1{,}5 \cdot R_{\max}$ biztonsági tényező
Blokkoló queue-küldés magas prioritású termelőből	Prioritásinverzió, kiéheztetés	Timeout, nagyobb queue, fogyasztó prioritásának emelése
Event flag adatfolyamra	Események elvesznek, nem számlál	Queue használata minden esemény továbbítására
Event flag `clear` elfelejtése	Régi jelzés folyamatosan érvényes marad	`clear on exit` opció, bitcsoportok tudatos kezelése
Mailbox pointer stackről	Érvénytelen memória a fogyasztónál	Statikus/heap puffer, tulajdonjog átadása
Watchdog prioritás túl alacsony	Watchdog kiéheztetése	A watchdog feladatot a felügyelt feladatok fölé helyezzük

Labor: watchdog- és eseményvezérelt feladatterv

A labor célja, hogy az olvasó egy emulált, FreeRTOS-alapú környezetben megvalósítsa a fenti mintákat, és mérje a watchdog frissítési periódusát, valamint az üzenetsor késleltetését. A labor **nem éles rendszeren**, nem valós vasakon fut, hanem QEMU-ban emulált Cortex-M3 processzoron, ezért a memória- és időzítési értékek a reprodukciós csomag részét képezik.

Kiinduló állapot. A labor egy előre elkészített, letisztított projektet feltételez, amely tartalmazza a FreeRTOS kernelt, a QEMU indítószkriptet és a `FreeRTOSConfig.h` konfigurációs állományt. A projekt a `/tmp/rtos_lab` munkakönyvtárba kerül, és semmilyen valós lemezt nem módosít. A konfiguráció:

- CPU: ARM Cortex-M3, emulált órajel 25 MHz
- RAM: 128 KiB
- Flash: 512 KiB
- RTOS: FreeRTOS 10.x (a pontos verziót a reprodukciós csomag rögzíti)
- Fordító: `arm-none-eabi-gcc`, optimalizálás `-O2`
- Tick periódus: 1 ms (`configTICK_RATE_HZ` = 1000)
- Hardveres WDT timeout a példában: 500 ms, ablak: 100–400 ms
- Mérési hardver: DWT ciklusszámláló (Data Watchpoint and Trace) a Cortex-M3 magban

Architektúra. A rendszer a következő feladatokat tartalmazza:

- `sensor_t1`: periodikus termelő, periódus 10 ms, prioritás 3.
- `sensor_t2`: periodikus termelő, periódus 20 ms, prioritás 2.
- `sensor_t3`: periodikus termelő, periódus 50 ms, prioritás 1.
- `aggregator`: fogyasztó, üzenetsorból olvas, prioritás 2.
- `watchdog`: liveness figyelő, periódus 200 ms, prioritás 4.

- `init`: inicializáló feladat, prioritás 5, az indulás után event flaggel vár a három szenzor inicializálására.

A termelők egy `sensor_sample_t` típusú struktúrát küldenek a `sensor_queue` üzenetsorba, amelynek mérete 8 elem. Minden termelő ciklusa végén beállítja a saját `*_HEALTHY` bitjét a `system_events` eseménycsoportban. Az `aggregator` fogadja az üzeneteket, és a feldolgozási időt mesterségesen `C_c=5` ms-ben tartja. A `watchdog` feladat a `system_events` bitjeit AND-kapcsolatban várja, 100 ms-os timeouttal, és ha minden bit megérkezett, meghívja a hardveres WDT frissítését. Az inicializáló feladat a `GYRO_READY`, `MAG_READY` és `BARO_READY` biteket várja, majd beállítja az `IMU_RUN` bitet.

Megvalósítási lépések. A labor elvégzéséhez az alábbi vázlatot kell követni:

1. Hozzuk létre a `sensor_queue` üzenetsort az `xQueueCreate(8, sizeof(sensor_sample_t))` hívással.
2. Hozzuk létre a `system_events` eseménycsoportot az `xEventGroupCreate()` hívással.
3. Indítsuk el a három termelő feladatot a megadott periódusokkal.
4. Indítsuk el az `aggregator` feladatot, amely blokkoló `xQueueReceive` hívással vár, és 250 ms timeoutot használ.
5. Indítsuk el a `watchdog` feladatot, amely a fenti logika szerint figyel.
6. Az inicializáló feladatot indítsuk el legmagasabb prioritással, hogy az eseménybitek és az indulási feltétel előbb álljon össze.
7. Futtassuk a rendszert QEMU-ban 10 másodpercig, és rögzítsük a naplót.

A build és futtatás parancsai:

```
# Fordítás az emulált célra
arm-none-eabi-gcc -O2 -mcpu=cortex-m3 -mthumb \
  -I FreeRTOS-Kernel/include \
  -I . \
  -o build/rtos_lab.elf \
  src/*.c FreeRTOS-Kernel/*.c

# QEMU futtatás
qemu-system-arm -M lm3s6965evb \
  -cpu cortex-m3 \
  -kernel build/rtos_lab.elf \
  -nographic \
  -d guest_errors,unimp \
  > lab_naplo.txt 2>&1
```

A fenti parancsok illusztrációk, a pontos eszközlánc- és QEMU-verziókat a reprodukciós csomag tartalmazza. A labor során a hoszt rendszerét nem módosíthatjuk; a munkakönyvtár a `/tmp/rtos_lab`, amely a labor végén törölhető.

Mérési napló. A naplóban külön kell választani a várt és a mért értékeket. A mérési módszer a DWT ciklusszámláló, amelyet minden `aggregator` fogadásnál és `watchdog` kicknél olvasunk ki. A ciklusokat 25 MHz-es órajellel számítjuk át időre. A következő táblázat a kitöltendő mérési napló szerkezetét mutatja:

Mért jellemző	Várt érték	Mért érték	Mérési módszer
`sensor_t1` periódus jitter	< 1 ms	(kitöltendő)	DWT időbélyeg a ciklus elején
`sensor_t2` periódus jitter	< 1 ms	(kitöltendő)	DWT időbélyeg a ciklus elején

Mért jellemző	Várt érték	Mért érték	Mérési módszer
`sensor_t3` periódus jitter	< 1 ms	(kitöltendő)	DWT időbélyeg a ciklus elején
`aggregator` queue-késleltetés	0–5 ms	(kitöltendő)	termelési időbélyeg és fogadási idő különbsége
`watchdog` frissítési periódus	200 ± 10 ms	(kitöltendő)	egymást követő kickek időbélyegei
Hardveres WDT timeout	500 ms	(kitöltendő)	hibainjektálás után a reset ideje

A `watchdog` frissítési periódusánál a várt 200 ms a `vTaskDelayUntil` periódusából adódik; a tolerancia a tick-felbontásból és az ütemezési jitterből fakad. A queue-késleltetésnél a várt felső korlát 5 ms, mivel a fogyasztó legrosszabb válaszideje a tervezés szerint legfeljebb 5 ms. Ha a mért érték tartósan e fölé emelkedik, a prioritás-hozzárendelést vagy a queue méretét módosítani kell.

Hibainjektálás. A labor második felében szándékosan akasszuk el az `aggregator` feladatot úgy, hogy egy `if (fault_inject)` feltétel mögött végtelen ciklusba lépjen. Ekkor a termelők továbbra is beállítják a szívverés-biteket, de a `watchdog` is lefut, mert az event flaget független bitten kezelik. A helyes konfigurációban azonban a watchdog nem kapja meg az aggregátor liveness bitjét, mert az aggregátor elakadt. Ezért a watchdog nem frissíti a hardveres WDT-t, és a hardver 500 ms után reseteli a rendszert. A naplóban a reset előtti utolsó bejegyzésnek tartalmaznia kell a hiányzó heartbeat üzenetet.

Ha a watchdog kickelését hibásan, az aggregátor ciklusából hajtanánk végre, akkor az elakadáskor a kick is elmaradna, és a hiba detektálható lenne, de a tervezési cél éppen az, hogy a watchdog akkor is működjön, ha több feladat közül csak egy hibásodik meg. A központi watchdog réteg ezt a tulajdonságot biztosítja.

Elvárt eredmény és visszaállítás. A labor sikeres, ha:

- a rendszer normál futásban 10 másodpercig stabilan működik reset nélkül;
- a watchdog frissítési periódus a várt tartományban marad;
- a queue-késleltetés nem haladja meg a felső korlátot;
- a hibainjektálás után a rendszer a WDT resetjével reagál, és a napló tartalmazza a hiányzó liveness jelzést.

A labor végén a `/tmp/rtos_lab` törölhető, a hoszt rendszeren semmilyen maradandó változás nem történik. A reprodukciós csomag mellékleteként a `FreeRTOSConfig.h`, a linker script és a kapott napló is megőrzendő.

Összefoglalás és önellenőrző kérdések

A watchdog, az üzenetsor, az event flag és a mailbox négy olyan RTOS-építőkő, amelyek különböző, de egymást kiegészítő szerepet töltenek be. A watchdog a rendszer életben maradását felügyeli, és akkor hatékony, ha központi feladat gyűjti a liveness jeleket. Az üzenetsor a termelő–fogyasztó adatfolyamot szervezi, és minden egyes üzenet számít. Az event flag több feltétel együttes teljesülését kezeli, de nem számlál eseményeket. A mailbox vezérlő üzenetekhez, pointer-átadáshoz alkalmas, ahol a tulajdonjog explicit.

A prioritás és az időzítés megválasztása nélkül a minták nem működnek. A watchdog timeout, a queue méret és a feladatok periódusai összefüggő rendszert alkotnak, ezért a tervezést nem lehet

utólag, próbálgatással pótolni. A hibakezelésnek rétegzettnek kell lennie: a helyi API-hibától a részleges degradáción át a biztonságos állapotig és a hardveres resetig.

Önellenőrző kérdések:

1. Miért nem elég az event flag egy nagy sebességű szenzor-adatfolyam továbbítására?
   Válasz: mert az event flag nem számlál eseményeket, csak bitet állít. Ha a fogyasztó lassabb, mint a termelő, egyes események elvesznek.
2. Hogyan választja meg a hardveres watchdog timeout értékét?
   Válasz: a felügyelt feladatok legrosszabb válaszidejének legalább 1,5–2-szeresére, és a watchdog feladat periódusát úgy, hogy a hiányzó jelzést időben észlelje.
3. Mi történik, ha a watchdog feladat prioritása alacsonyabb, mint a felügyelt fogyasztóé?
   Válasz: a fogyasztó kiéheztetheti a watchdog feladatot, így a WDT akkor is lejárhat, amikor a rendszer nem hibás. A watchdog feladatot a felügyelt feladatok fölé kell helyezni.
4. Miért tilos a watchdog kickelése megszakításkezelőből?
   Válasz: mert ha a fő feladat elakad, az ISR továbbra is futhat, és a WDT frissítése elfedné a hibát. A kicknek a feladatkörnyezetből, a liveness jelek ellenőrzése után kell történnie.
5. Milyen hibát okozhat, ha a mailbox üzenet pointerét egy feladat a saját stackjén lévő struktúrára mutatva adja át?
   Válasz: a stack tartalma a kontextusváltás után érvénytelenné válhat, és a fogyasztó korrupt adatot olvashat. Statikus vagy heap puffert kell használni.

Ezek a minták közvetlenül továbbvihetők a következő egységbe, ahol a beágyazott Linux és a firmware-rétegek felépítését vizsgáljuk. Az itt bevezetett watchdog, queue és event flag fogalmak ott is megjelennek, akár kernelmodulok, akár userland szolgáltatások formájában.

25 Beágyazott és mobil operációs rendszerek

25.1 Beágyazott Linux és firmware

25.1.1 1. Beágyazott Linux fogalma és architektúrája

A beágyazott Linux nem egy különálló operációs rendszer, hanem a mainline Linux kernel és a köré épített felhasználói tér egy adott beágyazott célhardverre szabott konfigurációja. Ugyanaz a kernel szolgálhat egy okostelefon, egy hálózati eszköz, egy járműfedélzeti egység vagy akár egy mosógép vezérlőjének alapjául; a különbség elsősorban a kiválasztott kernelkonfigurációban, az eszközleíró adatokban, a rootfs tartalmában és a bootfolyamatban van. A beágyazott Linux tehát nem forkolt kernel, hanem konfigurált, gyakran csökkentett felületű és méretre optimalizált Linux-rendszer.

A firmware szerepe a beágyazott rendszerekben kettős. Egyrészt jelenti azt a gyártó által biztosított, hardverhez közeli kódot, amely a processzor vagy a periféria elsődleges inicializálását végzi: ilyen például a SoC boot ROM-ja, a DRAM-vezérlő beállítását végző inicializáló, a Wi-Fi vagy Bluetooth modul saját firmware-e. Másrészt jelenti a rendszerfrissítés szerves részét képező, nem felejtő tárolóban tárolt bootkomponenseket is. A beágyazott Linuxban a firmware gyakran zárt, gyártó által aláírt bináris, amely a kernel beindulása előtt vagy azzal párhuzamosan fut. A rendszertervezőnek tisztában kell lennie azzal, hogy ez a réteg nem mindig nyílt forráskódú, és frissítése eltérő eljárást igényelhet.

25.1.2 1.1. Rendeltetésszerű összetevők

Egy tipikus beágyazott Linux rendszer négy fő szoftveres összetevőből áll:

1. **Bootloader**: az első, a bekapcsolás után futó, felülírható szoftver. Feladata a hardver minimális inicializálása, a kernel kiválasztása és betöltése, valamint a bootparaméterek és a device tree átadása. Beágyazott környezetben a leggyakrabban használt bootloader az U-Boot.
2. **Linux kernel**: a konfigurált, keresztezett fordítású kernelkép. Támogatja az adott architektúrához szükséges eszközmeghajtókat, fájlrendszereket, hálózati protokollokat és biztonsági alrendszereket. A kernelimage gyakran tömörített formátumú (`zImage`, `Image.gz`, `uImage`), és opcionálisan tartalmaz egy beépített initramfs-t is.
3. **Device tree** (DT): olyan adatszerkezet, amely a nem felfedezhető hardvert írja le a kernel számára. A device tree a forrásból (`dts`, `dtsi`) egy fordítóval (`dtc`) bináris blokkba (`dtb`) alakul. Ez a mechanizmus teszi lehetővé, hogy ugyanaz a kernel több hardverváltozaton fusson anélkül, hogy magát a kernelt újra kellene építeni.
4. **Rootfájlrendszer (rootfs)**: a felhasználói tér programjait, könyvtárait, konfigurációit és az alkalmazásokat tartalmazza. Beágyazott rendszerekben gyakran initramfs (RAM-ban létrejövő, csak olvasható alaprendszer) vagy tömörített, csak olvasható fájlrendszer (`squashfs`, `EROFS`) tölti be az induló környezetet, míg a változó adatok külön írható partíción tárolódnak.

Fontos, hogy a firmware és a device tree ne keveredjen: a firmware aktív, futó kód, míg a device tree statikus leíró adat, amelyet a kernel értelmez.

25.1.3 1.2. Beágyazott Linux és az általános célú Linux különbsége

A fő különbségek a következők:

- **Erőforráskorlát**: a beágyazott rendszer gyakran néhány tíz MiB RAM-mal és néhány száz MiB flash tárhellyel dolgozik, ezért a kernel és a rootfs méretét szigorúan optimalizálni kell. A generikus kernel több ezer modulja helyett egy minimális, beépített modulkészlet szükséges.
- **Célzott hardvertámogatás**: a kernelbe csak a ténylegesen jelen lévő eszközök meghajtóit építik be, gyakran statikusan, nem modulként. A device tree pontosan leírja a SoC perifériáit, megszakításvonalait és órajeleit.
- **Indítási idő**: beágyazott rendszerekben a bootidő gyakran kritikus követelmény; másodpercek vagy akár századmásodpercek is számíthatnak. Ezért a kernel és az init rendszer indulását minimalizálják, elhagyják a felesleges szolgáltatásokat.
- **Frissítési stratégia**: az általános célú disztribúciók csomagkezelője helyett beágyazott rendszerekben teljes rendszerképeket vagy atomi frissítéseket használnak, gyakran A/B partíciós sémával.
- **Firmware-kapcsolat**: az általános célú rendszerben a firmware frissítése gyakran külön folyamat; beágyazott rendszerben a bootloader, a device tree, a kernel és a rootfs együttes verziózott egységet alkot.

25.1.4 2. Bootloader, kernel, rootfs és device tree

A beágyazott Linux bootfolyamata lépésenként halad a minimális hardveres kódtól a teljes felhasználói térig. A pontos sorrend SoC-enként változhat, de a logikai modell általános.

25.1.5 2.1. Boot folyamat

A következő lépések jellemzőek:

1. **Boot ROM**: a processzor belső, nem módosítható kódja fut le. Inicializálja a minimális órajeleket és egy bootforrást (például SPI flash, eMMC, SD, UART). A boot ROM betölti a bootloader első fokozatát a belső SRAM-ba, és ellenőrzi annak integritását.
2. **Bootloader (U-Boot SPL, majd teljes U-Boot)**: az első fokozatú bootloader (SPL – Secondary Program Loader) beállítja a DRAM-ot, majd betölti a teljes U-Bootot. A teljes U-Boot környezeti változókból tudja, melyik partícióból, milyen címmel töltse be a kernelt és a device tree-t.
3. **Kernel és device tree betöltése**: a bootloader a memóriába másolja a kernelképet és a DTB-t, beállítja a bootparamétereket (például `console=ttyAMA0,115200 root=/dev/mmcblk0p2 rw`), majd átadja a vezérlést a kernel belépési pontjának.
4. **Kernelindítás**: a kernel kicsomagolja magát, beállítja a lapozást, inicializálja a meghajtókat a device tree alapján, majd megkeresi és csatlakoztatja a rootfájlrendszert. Ha initramfs van, akkor először azt használja ideiglenes gyökérként.
5. **Init és felhasználói tér**: a kernel elindítja az `init` folyamatot (leggyakrabban `init=/sbin/init`). Beágyazott rendszerekben gyakran a BusyBox `init` vagy egy célzott, kis init program fut, amely elindítja a szükséges daemonokat és az alkalmazást.

A boot folyamatot a rendszertervezőnek a célhardverre kell hangolnia; a labor részben ezt egy emulált ARM célgépen mutatjuk be.

25.1.6 2.2. Bootloader: U-Boot szerepe és példa

Az U-Boot egy nyílt forráskódú, gyakran használt bootloader. Forrásból cross-compile-álandó, és a céleszközre írható. A bootloader környezeti változókkal vezérelhető: ezek tárolják a betöltendő kernel címét, a DTB címét, a boot argumentumokat és az aktív bootpartíciót.

Példa U-Boot környezeti szkriptre (szemléltetés, nem végrehajtandó éles eszközön):

```
setenv bootargs 'console=ttyAMA0,115200 root=/dev/mmcblk0p2 rootwait rw'
setenv bootcmd 'mmc dev 0; fatload mmc 0:1 0x62000000 zImage; fatload mmc 0:1 0x63000000
↪  board.dtb; bootz 0x62000000 - 0x63000000'
saveenv
boot
```

Ebben a példában:

- `bootargs` adja át a kernelnek a konzol és a rootfájlrendszer paramétereit.
- `bootcmd` az első partícióról (FAT) betölti a kernelt és a DTB-t, majd a `bootz` parancs elindítja a kernelt.
- A DTB címét explicit meg kell adni, mert a kernel egyes architektúrákon magától nem találja meg.

A bootloader frissítése különösen kényes: ha megszakad a bootloader írása, a rendszer „brickelt” állapotba kerülhet. Ezért a gyakorlatban a bootloader védett partíción, redundáns kópiával vagy a gyártó által biztosított helyreállítási mechanizmussal tárolandó.

25.1.7 2.3. Device tree

A device tree forrás (`dts`) egy fa struktúrájú leírás. A gyökér node alatt a CPU, a memória, a buszok és a perifériák jelennek meg. A `dtc` eszköz bináris `dtb` formátumot állít elő, amelyet a bootloader a kernel mellé tölt.

Példa egy egyszerű DTS részletre (szemléltetés, nem valós gyártói tábla pontos értékei):

```
/dts-v1/;

/ {
    model = "Placeholder Embedded Board";
    compatible = "vendor,board";

    memory@40000000 {
        device_type = "memory";
        reg = <0x40000000 0x08000000>; /* 128 MiB @ 0x40000000 */
    };

    chosen {
        bootargs = "console=ttyAMA0,115200";
    };

    timer@101e2000 {
        compatible = "arm,sp804";
        reg = <0x101e2000 0x1000>;
        interrupts = <0 16 4>;
    };
```

```
    mmc@10005000 {
        compatible = "arm,pl180";
        reg = <0x10005000 0x1000>;
        interrupts = <0 22 4>;
        status = "okay";
    };
};
```

A kernel a `compatible` tulajdonságok alapján választja ki a megfelelő meghajtót. A `reg` tulajdonság a memóriatartományt adja meg báziscím és méret formában; a hexadecimális címek a hardverdokumentációból származnak, és konkrét táblánként eltérnek.

A device tree előnye, hogy a hardverleírás a kerneltől függetlenül frissíthető. Hátránya, hogy a hibás DTB csendes, nehezen debuggolható hibákat okozhat: a kernel lefagy vagy az eszköz nem jelenik meg. Ezért a DTB-t is verziózni és tesztelni kell.

25.1.8 2.4. Rootfs: initramfs és csak olvasható fájlrendszerek

A beágyazott rootfs két fő mintája:

- **Initramfs**: a kernelbe vagy a kernel mellé csomagolt, gzip-cel tömörített cpio archívum. A kernel a valódi rootfs csatlakoztatása előtt kibontja a RAM-ba, és abból indít. Előnye a gyors indulás és a rugalmas, RAM-alapú helyreállítás; hátránya, hogy a tartalom a RAM-ot foglalja, és újraindításkor elveszik, ezért a változó adatokat nem ide kell tenni.
- **Tömörített, csak olvasható fájlrendszer**: például `squashfs` vagy `EROFS`. A rootfs egyetlen, tömörített képfájl, amelyet a kernel közvetlenül csatlakoztat. A változó adatok (például konfiguráció, napló, adatbázis) külön írható partíción vagy overlay fájlrendszeren (`overlayfs`) tárolódnak. Ez a séma támogatja a megbízható frissítést és a könnyű visszaállítást.

A rootfs minimális tartalmához gyakran elég a BusyBox, amely egyetlen binárisban számos szabványos parancsot biztosít. Egy egyszerű init szkript így nézhet ki (szemléltetés):

```
#!/bin/sh
mount -t proc proc /proc
mount -t sysfs sysfs /sys
mount -t devtmpfs devtmpfs /dev
mount -t tmpfs tmpfs /tmp
echo "Embedded system booted" > /dev/console
exec /bin/sh
```

Ez a szkript a minimális pszeudo-fájlrendszereket csatlakoztatja, és egy interaktív shellt indít. Valós termékben az init a szükséges daemonokat és a fő alkalmazást indítaná el.

25.1.9 3. Frissítés, rollback és A/B partíció

A beágyazott Linux rendszerek frissítése nem egyszerű csomagcsere, hanem atomi rendszerképcsere és megbízható visszagördülés problémája. A cél az, hogy a frissítés után is mindig legyen egy ismert jó állapot, ahová a rendszer vissza tud térni.

25.1.10 3.1. Frissítési kihívások

A fő kihívások:

- **Atomitás**: a frissítés során a flash vagy az eMMC részlegesen íródhat, ha a tápellátás megszakad. A rendszerképnek vagy teljesen fel kell íródnia, vagy a régi képnek sértetlenül kell maradnia.
- **Konzisztencia**: a kernel, a device tree és a rootfs egymással kompatibilis verzióját kell garantálni; egy komponens frissítése a másik nélkül bootképtelen rendszert eredményezhet.
- **Biztonság**: a frissítőcsomagot alá kell írni, hogy illetéktelen ne tölthessen be rosszindulatú kernelt vagy rootfs-t. A bootloader csak aláírt, ellenőrzött képet fogadhat el.
- **Helyreállítás**: ha az új kép nem bootol, a rendszernek automatikusan vissza kell tudnia térni a korábbi, működő verzióhoz.

25.1.11 3.2. A/B partíciós séma

A leggyakoribb megoldás az A/B (dupla kópia) séma. A flash tárolóban két teljes rendszerpartíció-készlet van, például:

- `boot_a` és `boot_b`: a kernel, a DTB és a bootloader konfigurációja;
- `rootfs_a` és `rootfs_b`: a rootfájlrendszer.

A rendszer az egyik slotból indul (aktív slot), a frissítés a másik (inaktív) slotra íródik. A frissítés befejezése után a bootloader bejegyzi az új slotot aktívnak, és újraindít. Ha az új slot sikeresen elindul, a felhasználói tér megerősíti a frissítést; ha a boot meghiúsul (például a kernel nem indul el, vagy egy watchdog leállítja), a bootloader egy boot counter vagy állapotjelző alapján visszavált a régi slotra.

A boot counter mechanizmus:

- Bootloader betölti az aktív slotot.
- A kernel/felhasználói tér sikeres induláskor növeli vagy törli a „boot successful” jelzőt.
- Ha a boot nem sikerül, a counter eléri a küszöböt, és a bootloader a másik slotot választja.

A rollback tehát nem a felhasználó által kézzel indított folyamat, hanem automatikus, a boot sikerességének mérésén alapul. A sikeres boot definíciója rendszerenként változhat: jellemzően az alkalmazás elindulása, egy hosztdemon elérése vagy egy watchdog-kézfogás.

25.1.12 3.3. Frissítő eszközök és munkafolyamat

Beágyazott Linuxra több nyílt forráskódú frissítési keretrendszer létezik, amelyek az A/B logikát és az aláírást megvalósítják. Például a RAUC, a Mender vagy a SWUpdate különböző stratégiákat támogat. A konkrét eszköz- és verzióválasztás a projekt és a tanúsítási követelmények függvénye, ezért a megvalósítás előtt ellenőrizni kell az adott eszköz aktuális dokumentációját.

Egy tipikus A/B frissítési munkafolyamat lépései:

1. A buildrendszer előállítja az új `boot` és `rootfs` képet, valamint egy manifestet, amely tartalmazza a verziókat, a komponensek hash-értékeit és az aláírást.
2. Az eszközön futó frissítő daemon letölti a csomagot, ellenőrzi az aláírást és a hash-t.
3. Az inaktív slot partícióit a daemon felülírja az új képekkel. Az írás során a fájlrendszer vagy a blokkréteg biztosítja az atomitást.

4. A daemon beállítja a bootloader környezeti változóit: az inaktív slotot aktívvá teszi, és törli a sikeres boot jelzőt.
5. Az eszköz újraindul. A bootloader az új slotot próbálja betölteni.
6. A kernel és az init sikeres indulása után a felhasználói tér megerősíti a bootot (például egy `fw_setenv` hívással). Ha a boot nem sikerül, a bootloader a backup slotból indítja a rendszert.

A bootloader által használt környezeti változók például (szemléltetés):

```
boot_a_attempts=3
boot_a_ok=0
active_slot=a
```

A frissítő a `boot_a_ok` változót 1-re állítja sikeres boot után; ha az indulás nem sikerül, a bootloader csökkenti a `boot_a_attempts` értékét. Három sikertelen próbálkozás után az aktív slot `b`-re vált.

Fontos: a bootloader és a frissítő komponens közötti interfésznek szabványosnak és verziózottnak kell lennie. A környezeti változók hibás beállítása boot loopot vagy brickelt eszközt okozhat.

25.1.13 3.4. Firmware frissítés

A perifériák (Wi-Fi, Bluetooth, kamera, hálózati vezérlő) saját firmware-e külön frissítési útvonalat igényelhet. A firmware bináris gyakran a `linux-firmware` gyűjteményből származik, és a rootfs vagy egy külön partíció tárolja. A kernel a `request_firmware()` API-val tölti be a firmware-t az eszköz inicializálásakor. Frissítéskor a firmware-t a rootfs frissítéssel együtt, vagy külön csomagban lehet cserélni.

A firmware és a kernel verziójának kompatibilitását ugyanúgy ellenőrizni kell, mint a device tree-t. A hibás firmware az eszköz meghajtójának hibás működését, a rendszer instabilitását vagy biztonsági rést okozhat.

25.1.14 4. Labor: beágyazott Linux rendszerkép tervezése

A labor célja egy minimális beágyazott Linux rendszerkép megtervezése és elindítása QEMU emulált ARM célgépen. A labor nem módosítja a hoszt rendszerét; minden művelet egy eldobható munkakönyvtárban és az emulátorban fut. A parancsok szemléltetők: a konkrét verziók és a buildrendszer beállításai a reprodukciós csomagban ellenőrizendők.

25.1.15 4.1. Kiinduló állapot és konfiguráció

A reprodukcióhoz az alábbi komponensek szükségesek:

- Hoszt operációs rendszer: Linux, `qemu-system-arm` telepítve.
- Cross-toolchain: `arm-linux-gnueabihf-` (32 bites ARM hardfloat) vagy `aarch64-linux-gnu-`. A labor 32 bites ARM-ot használ.
- Linux kernel forrás, például a stabil `linux-6.6.y` sorozatból (a pontos verziót a naplóban rögzítjük).
- BusyBox forrás, például `busybox-1.36`.
- A laborhoz tartozó build-szkriptek és konfigurációs fragmentek a reprodukciós csomagban találhatók.

A cél hardver: QEMU `vexpress-a9` gép, 256 MiB RAM, egy MMC/SD kártya, soros konzol. A konfiguráció egyetlen parancsfájlban összefogható:

```
# reproduce.sh - a teljes build és futtatás (szemléltetés)
set -e
export ARCH=arm
export CROSS_COMPILE=arm-linux-gnueabihf-
ROOT="$PWD/out"
mkdir -p "$ROOT"

# 1. Kernel konfiguráció és fordítás
cp kernel_config_fragment "$ROOT/.config"
make O="$ROOT" vexpress_defconfig
scripts/kconfig/merge_config.sh -m -O "$ROOT" \
    "$ROOT/.config" kernel_config_fragment
make O="$ROOT" -j4 zImage
make O="$ROOT" dtbs

# 2. BusyBox rootfs építése initramfs-be
# (a busybox configja a csomagban)
cd busybox-1.36
make ARCH=arm CROSS_COMPILE=$CROSS_COMPILE defconfig
make ARCH=arm CROSS_COMPILE=$CROSS_COMPILE -j4
make ARCH=arm CROSS_COMPILE=$CROSS_COMPILE install
# ... initramfs összeállítás cpio + gzip

# 3. Futtatás QEMU-ban
qemu-system-arm -M vexpress-a9 -m 256 \
    -kernel "$ROOT/arch/arm/boot/zImage" \
    -dtb "$ROOT/arch/arm/boot/dts/arm/vexpress-v2p-ca9.dtb" \
    -initrd initramfs.cpio.gz \
    -nographic \
    -append "console=ttyAMA0,115200 rdinit=/init"
```

A fenti parancsok a tényleges futtatás előtt a pontos forrásverzióval és a konfigurációs fragmenttel együtt ellenőrizendők. A labor nem fut éles eszközön; minden írás a munkakönyvtárban és a QEMU virtuális lemezén történik.

25.1.16 4.2. Mérési napló

A mérés során három metrikát rögzítünk:

1. **Bootidő**: a QEMU indításától (`qemu` parancs elindulása) a kernel utolsó üzenetéig vagy az init szkript `booted` kiírásáig eltelt idő. A mérés a soros konzol kimenet időbélyegzésével történik.
2. **Kernelkép méret**: a `zImage` és a `dtb` fájl mérete bájtban.
3. **Memóriahasználat az induláskor**: a kernel által a boot során kiírt `Memory:` sor, vagy a `/proc/meminfo` `MemTotal` és `MemAvailable` értéke az init szkriptben.

A mérési naplóban a **várt** (előzetesen becsült vagy korábbi buildből származó) és a **mért** értékeket külön kell szerepeltetni. A mérés hardverkörnyezete: hoszt CPU, QEMU verzió, kernel verzió, compiler verzió és az alkalmazott optimalizációs szint. Egyetlen futás nem elegendő; legalább 5 bootot mérünk, és a mediánt, valamint a szórást adjuk meg.

Példa naplóbejegyzés (szemléltetés, a számok nem ellenőrzött eredmények):

Metrika	Várt	Mért (átlag, n=5)	Mérési körülmények
Bootidő (ms)	800–1200	1050 ± 80	QEMU 8.1, host CPU, `-nographic`
zImage méret (KiB)	2200–2600	2478	kernel 6.6.10, `-O2`, arm
Initramfs méret (KiB)	700–900	812	gzip, BusyBox 1.36
MemAvailable boot után (MiB)	> 200	231	`MemTotal` 256 MiB

A mért bootidő szórása a hoszt CPU terhelésétől és a QEMU I/O ütemezésétől is függ; a reprodukálható méréshez a hosztot csendes állapotban, rögzített CPU-hoz rendelt QEMU processzel futtatjuk.

25.1.17 4.3. Visszaállítás

A labor teljesen izolált; a `out/` munkakönyvtár törlésével minden létrehozott fájl eltávolítható. A hoszt kernelét, bootloaderét, lemezpartícióit nem érinti. A QEMU leállítása a `Ctrl-A X` billentyűkombinációval vagy a konzol bezárásával történik. A reprodukciós csomag a laborkörnyezet leírását és a visszaállítási lépéseket tartalmazza.

25.1.18 5. Gyakori hibák és helyes megoldások

A beágyazott Linux rendszerkép tervezése során tipikus hibák és azok kijavítása:

25.1.19 5.1. Hibás device tree

- **Hiba**: a bootloader olyan DTB-t tölt be, amely nem egyezik a kernel verziójával vagy a hardverrel; a kernel panickel, vagy a perifériák csendben nem működnek.
- **Helyes megoldás**: a DTB-t a kernel forrásával együtt, azonos verzióból kell fordítani; a labor a kernel `make dtbs` parancsával ezt garantálja. A DTB-t verziózni kell, és a bootloadernek a megfelelő, aláírt DTB-t kell töltenie.

25.1.20 5.2. Firmware hiánya

- **Hiba**: a kernel konfigurációja engedélyez egy perifériát, de a szükséges firmware bináris hiányzik a rootfs-ből; az eszköz nem jön létre, a meghajtó hibát jelez.
- **Helyes megoldás**: a `make modules_install` mellett a `linux-firmware` csomag megfelelő fájljait is a rootfs-be kell másolni, vagy a firmware-t a kernelbe kell építeni. A hiányzó firmware-t a kernel a `request_firmware` hívásnál hibaüzenettel jelzi; a bootnaplót figyelni kell.

25.1.21 5.3. Írható rootfs frissítés közbeni megszakadása

- **Hiba**: a rendszer frissítés közben, a rootfs közvetlen felülírásakor kap kikapcsolást; a rootfs inkonzisztens lesz, és a boot lefagy.
- **Helyes megoldás**: A/B partíciós séma alkalmazása, ahol az inaktív slotot írjuk, majd a bootloader vált. Az aktív rootfs-t soha nem írjuk futás közben; az írás atomitását a frissítő eszköz biztosítja.

25.1.22 5.4. Boot counter hiánya vagy helytelen beállítása

- **Hiba**: az A/B rendszerben a sikertelen bootot nem érzékeli a bootloader, ezért a hibás slotban ragad, és a rendszer boot loopba kerül.
- **Helyes megoldás**: a bootloaderben boot countert és watchdogot kell beállítani. A felhasználói térnek a sikeres indulást követően a bootloader állapotjelzőjét frissítenie kell; ennek elmaradása esetén a bootloader automatikusan visszavált a backup slotra.

25.1.23 5.5. Nem elegendő memória

- **Hiba**: a rendszerkép a céleszköz rendelkezésre álló RAM-jánál nagyobb initramfs-t vagy teljes rootfs-t próbál a RAM-ba tölteni, ami OOM-hoz vagy bootleálláshoz vezet.
- **Helyes megoldás**: a rootfs-t tömörített, csak olvasható fájlrendszerből kell csatlakoztatni, nem pedig a teljes tartalmat RAM-ba másolni. Az initramfs méretét minimálisra kell szorítani, és csak a boothoz szükséges elemeket kell benne tartani. A kernel `Memory:` sorát a tervezéskor össze kell vetni a rootfs méretével.

25.1.24 5.6. Nem verziózott bootkomponensek

- **Hiba**: a kernel, a DTB és a rootfs különböző buildből származik; a kernel bizonyos interfészei megváltoztak, a rootfs init szkriptje nem található, a boot összeomlik.
- **Helyes megoldás**: a buildrendszer egyetlen verziószám alatt építi az összes bootkomponenst, és a manifest tartalmazza mindegyik hash-értékét. A frissítés csak akkor engedélyezett, ha a csomag minden komponense konzisztens.

25.1.25 6. Önellenőrző kérdés és továbbtanulás

Önellenőrző kérdés:
Egy beágyazott Linux eszköz A/B partíciós frissítést használ. A frissítés után az eszköz újraindul, de a bootloader mindig a régi slotot választja, annak ellenére, hogy a frissítő beállította az inaktív slotot aktívnak. Soroljon fel legalább három lehetséges okot, és mindegyikre adjon javítási javaslatot!

Várható válasz elemei: a bootloader környezeti változója nem került mentésre (`saveenv` hiánya); a frissítő által beállított próbálkozási számláló nem lett megfelelően törölve; az új slot kernel- vagy DTB-hash-e nem egyezik a bootloader által várt aláírással; a sikeres boot jelzőt a felhasználói tér nem állította be, így a bootloader a régi slotot tekinti biztonságosnak.

A fejezetben bemutatott fogalmakra épít a következő egység (Android architektúra), amely a mobil Linux-alapú rendszerek további absztrakcióit és a HAL szerepét tárgyalja. A labor sikeres elvégzése után a tanuló képes egy minimális beágyazott Linux rendszerképet előállítani, elindítani és a bootfolyamatot mérni, valamint az A/B frissítés alapelveit alkalmazni.

25.2 Android architektúra és HAL

Az Android egy rétegzett, Linux kernelre épülő platform, amelyben a hardvergyártók és a rendszerfejlesztők közötti szerződést a hardverabsztrakciós réteg (HAL) definiálja. Az alábbiakban áttekintjük az Android architekturális rétegeit, különös tekintettel a HAL-ra, a Binder IPC-re és az engedélyezési modellre. A fejezet nem ismétli meg a korábbi Linux kernel és beágyazott Linux alapokat (Beágyazott Linux és firmware), hanem az Android-specifikus kiegészítésekre és absztrakciókra összpontosít.

25.2.1 1. Az Android rétegei és a hardverabsztrakció helye

Az Android platform hagyományosan a következő fő rétegekre bontható (alulról felfelé):

1. **Linux kernel** – a legalacsonyabb szintű erőforrás-kezelő, amelyet az Android specifikus igényeihez igazítottak (binder driver, low memory killer, wake locks, stb.). Ez a réteg biztosítja a processzorkezelést, a memóriakezelést, az eszközmeghajtókat és a processz-izolációt.
2. **Hardverabsztrakciós réteg (HAL)** – egy interfészréteg, amely a hardvereszközök és a magasabb szintű keretrendszer közötti szerződést definiálja. A HAL célja, hogy a felső rétegek a hardver konkrét megvalósításától függetlenül, egységes interfészen keresztül férjenek hozzá az eszközökhöz (kamera, szenzorok, grafikus komponens, audio, GPS, stb.).
3. **Natív réteg** – C és C++ könyvtárak, amelyek a rendszer szolgáltatásait és a HAL feletti funkcionalitást biztosítják. Ide tartoznak például a libc (bionic), a log rendszer, a media framework, a SQLite, az OpenGL ES, a WebKit stb. Ez a réteg közvetlenül kapcsolódik a HAL-hoz.
4. **Android Runtime (ART)** – a futtatókörnyezet, amely a Dalvik bytecode-ot futtatja. Az ART JIT (Just-In-Time) és AOT (Ahead-Of-Time) fordítást is használ, valamint automatikus szemétgyűjtést (garbage collection). A Zygote folyamat előre betöltött osztályokkal és erőforrásokkal indul, és ebből ágaznak le az alkalmazásfolyamatok.
5. **App Framework (keretrendszer)** – a Java/Kotlin nyelven elérhető magas szintű API-k gyűjteménye, például az Activity Manager, a Package Manager, a Window Manager, a Content Providers, a Location Manager, a Notification Manager, stb. Ezek a rendszerszolgáltatások a Binder IPC-n keresztül kommunikálnak egymással és a natív réteggel.
6. **Alkalmazások** – a felhasználó által telepített és a rendszeralkalmazások (Launcher, telefon, kontaktok, stb.). Minden alkalmazás saját folyamatban fut, saját UID-del és homokozóval (sandbox).

A hardverabsztrakció tehát a Linux kernel és a natív réteg között helyezkedik el, de formailag a HAL modulok a kernel felett, user space-ben futnak. A HAL interfészek a hardvergyártóktól független definíciók, amelyeket a Google és a platform fejlesztői határoznak meg, az implementációt pedig a gyártók biztosítják. Így a felsőbb rétegek nem ismerik a konkrét hardverregisztereket vagy a gyártói illesztőprogramok részleteit.

25.2.2 2. A HAL célja és két generációja

A HAL réteg történetileg két jelentős szakaszon ment keresztül:

- **Legacy HAL (libhardware)** – a korai Android verziókban a HAL modulok dinamikus könyvtárak voltak (`*.so`), amelyeket a rendszer a `hw_get_module()` függvénnyel töltött be. Minden modul egy `hw_module_t` struktúrát és egy `hw_device_t` struktúrát implementált.

Az interfészek C nyelvű függvénypointerekkel voltak definiálva. Ez a megközelítés nem biztosított szigorú verziózást, és a folyamatok közvetlenül hívták a HAL-t.

- **Modern HAL (HIDL / AIDL)** – az Android 8.0 (Oreo) óta a HAL interfészeket formális nyelvvel (HIDL, később AIDL) írják le. A HIDL (HAL Interface Definition Language) interfészeket definiál a HAL szolgáltatások és klienseik között. Az interfészek verzióval rendelkeznek, és a szolgáltatások a `hwservicemanager` daemonnál regisztrálódnak. A HIDL interfészek kétféleképpen használhatók:
  - **Passthrough** – a HAL implementáció közvetlenül a kliens folyamatába töltődik be, hasonlóan a legacy modulokhoz.
  - **Binderized** – a HAL szolgáltatás külön folyamatban fut, és a kliensek Binder IPC-n keresztül hívják. Ez a preferált mód, mert erős folyamat-izolációt és jogosultság-ellenőrzést tesz lehetővé.

Az Android 11-től kezdve a HIDL-t fokozatosan felváltja az AIDL (Android Interface Definition Language), amely egyesíti a HAL interfészeket az alkalmazás- és keretrendszer interfészekkel. Az AIDL alapú HAL-ok ugyanúgy a `hwservicemanager`-nél regisztrálódnak, de az interfész definíciók egységesebbek. A gyakorlatban ma már a legtöbb új HAL interfész AIDL-ben készül, míg a régebbi HAL-ok továbbra is HIDL formátumban élnek.

25.2.3 Példa egy HAL interfészre

Vegyünk egy egyszerű szenzor HAL interfészt HIDL-ben (pszeudokód):

```
package android.hardware.sensor@2.0;

interface ISensor {
    getSensorsList() generates (vec<SensorInfo> sensors);
    activate(int32_t sensorHandle, bool enabled) generates (Result result);
    poll(int32_t maxCount) generates (vec<Event> events);
};
```

Ugyanez az interfész AIDL-ben (pszeudokód):

```
package android.hardware.sensor;

interface ISensor {
    List<SensorInfo> getSensorsList();
    int activate(int sensorHandle, boolean enabled);
    List<Event> poll(int maxCount);
};
```

A gyakorlati implementáció során a gyártó létrehoz egy C++ osztályt, amely megvalósítja ezt az interfészt, majd regisztrálja azt a `hwservicemanager`-nél:

```
#include <android/hardware/sensor/2.0/ISensor.h>

using android::hardware::sensor::V2_0::ISensor;
using android::hardware::sensor::V2_0::implementation::Sensor;

int main() {
    // A Sensor osztály implementálja az ISensor interfészt.
    auto sensor = new Sensor();
    // Regisztrálás a HAL szolgáltatáskezelőnél.
    android::hardware::configureRpcThreadpool(4, true);
    android::status_t status = sensor->registerAsService();
    if (status != android::OK) {
```

```
        // Hibakezelés és naplózás.
        return 1;
    }
    android::hardware::joinRpcThreadpool();
    return 0;
}
```

A kliens oldalról a HAL szolgáltatást a `getService()` hívásával érik el, majd Binder proxyn keresztül hívják a metódusokat. A fenti kód szemléltetés, nem közvetlenül futtatható éles környezetben.

25.2.4 3. Linux kernel Android-specifikus kiegészítései

Az Android kernel nem egy teljesen különálló kernel, hanem a mainline Linux kernelre épül, kiegészítve olyan alrendszerekkel, amelyeket az Android folyamatmodell és energiamenedzsment megkövetel. A legfontosabb Android-specifikus komponensek:

- **Binder driver** – a kernelben megvalósított IPC mechanizmus, amely a rendszer folyamatai közötti hívásokat kezeli. A Binder az Android alapvető kommunikációs csatornája, és a HAL szolgáltatások eléréséhez is nélkülözhetetlen. A driver a `/dev/binder` eszközön keresztül érhető el, és támogatja a proxy/stub mintát, a hívások szinkronizációját és a jogosultság-ellenőrzést.
- **ashmem** – anonim osztott memória, amely lehetővé teszi a folyamatok közötti memóriamegosztást, és a Binder tranzakciókban használt nagyméretű adatok átadását. Az ashmem a Linux `tmpfs`-hez hasonló, de nem igényel fájlleírót a megosztáshoz, és jobban illeszkedik a Binderhez.
- **ion / dmabuf** – pufferek kezelése hardveres eszközök (GPU, kamera, kijelző) számára. Az ion a fizikailag folytonos memória lefoglalását és az eszközök közötti megosztását teszi lehetővé. Az újabb Android verziókban az ion-t fokozatosan felváltja a mainline `dmabuf` és a `dma-heap` mechanizmus.
- **Low Memory Killer (LMK)** – a kernelben futó démon, amely a memórianyomás függvényében leállítja a legkevésbé fontos folyamatokat. Az LMK együttműködik a felhasználói térben futó `lmkd` daemonnal, és az Android folyamat prioritási osztályait (oom_score_adj) használja. Az újabb rendszerekben a LMK-t a `psi` (pressure stall information) alapú LMKD váltja fel.
- **Wake locks / wakeup sources** – az energiamenedzsment része, amely megakadályozza, hogy a rendszer alvó állapotba kerüljön, amíg egy kritikus művelet (például média lejátszás, hálózati átvitel, szenzor olvasás) folyamatban van. A kernel `wakeup_source` struktúrákat használ, a felhasználói térben pedig az `PowerManager` API biztosít hozzáférést.
- **SELinux és egyéb biztonsági kiterjesztések** – az Android szigorú SELinux policy-t alkalmaz, amelyben minden folyamat és erőforrás címkézett, és a hozzáféréseket a policy engedélyezi. Ezt a részletes jogosultsági modellt a 8. szakaszban tárgyaljuk.

Ezek a kiegészítések azt a célt szolgálják, hogy a Linux kernel által nyújtott általános erőforráskezelést az Android erősen izolált folyamatmodelljéhez, a hardveres eszközökhöz való biztonságos hozzáféréshez és az energiatakarékossághoz igazítsák.

25.2.5 4. ART és az app framework

Az Android futtatókörnyezet (ART) felelős a Dalvik Executable (DEX) formátumú bytecode futtatásáért. A DEX fájlok a Java/Kotlin forráskódból készülnek, és az ART betölti, ellenőrzi, optimalizálja és futtatja őket. Az ART a következő fő jellemzőkkel bír:

- **JIT és AOT fordítás** – az ART kezdetben teljes AOT fordítást használt, ami hosszabb telepítési időt és nagyobb tárhelyet igényelt. Az újabb verziók hibrid megközelítést alkalmaznak: a telepítéskor csak a kritikus részeket fordítják AOT, majd futás közben a JIT fordító dinamikusan optimalizálja a gyakran futó kódrészeket. A profilvezérelt fordítás (profile-guided compilation) a valós használat alapján dönti el, mely metódusokat érdemes natív kódra fordítani.
- **Garbage collection** – az ART több generációs, párhuzamos és részben konkurens szemétgyűjtőt használ, amely csökkenti a futás közbeni megállásokat. A memóriakezelés szorosan kapcsolódik a kernel memóriakezelőjéhez és a Zygote folyamat modelljéhez.
- **Zygote** – egy speciális folyamat, amely a rendszerindítás során előre betölti a gyakori osztálykönyvtárakat és erőforrásokat. Amikor egy új alkalmazás indul, a Zygote fork() hívással létrehozza a gyermekfolyamatot, amely így megörökli a már betöltött osztályokat. Ez jelentősen csökkenti az alkalmazásindítási időt és a memóriafelhasználást (copy-on-write révén).
- **System Server** – a rendszer központi folyamata, amely a keretrendszer szolgáltatásait futtatja. A System Server számos Java szolgáltatást tartalmaz, mint például az ActivityManagerService, a PackageManagerService, a WindowManagerService, a PowerManagerService, stb. Ezek a szolgáltatások a Binder IPC-n keresztül érhetők el az alkalmazások számára.

Az app framework (keretrendszer) az a Java API réteg, amelyet az alkalmazásfejlesztők használnak. Az Activity, Service, BroadcastReceiver, ContentProvider komponensek mind a framework részét képezik. A framework szolgáltatások a System Serverben futnak, és a Binder segítségével kommunikálnak az alkalmazásokkal.

25.2.6 5. Binder IPC

A Binder az Android elsődleges folyamatok közötti kommunikációs mechanizmusa. A Binder a következő jellemzőkkel rendelkezik:

- **Kernel driver** – a Binder a Linux kernelben implementált driverrel rendelkezik (`/dev/binder`). A driver a tranzakciók átvitelét, a hívó azonosítását, a hivatkozások kezelését és a puffer memóriát végzi.
- **Proxy/stub minta** – a Binder interfészeket egy felületleíró nyelvvel (AIDL) definiálják, amelyből a build rendszer Proxy és Stub osztályokat generál. A kliens oldalon a Proxy osztály a szerver oldali Stub osztályhoz intézett hívásokat csomagolja tranzakciókba, és a Binder driveren keresztül továbbítja. A szerver oldalon a Stub osztály kicsomagolja a paramétereket, és meghívja a tényleges implementációt. A válasz visszafelé ugyanezen az úton halad.
- **Tranzakciók és szinkronizáció** – a Binder tranzakciók lehetnek szinkron (a hívó blokkolódik a válaszig) vagy aszinkron (a hívó nem vár választ). A szinkron hívások esetén a Binder gondoskodik a hívó és a szerver közötti adatátvitelről, és a hívás során a hívó folyamat nem foglalja el a szerver szálait. A Binder driver a tranzakciókat puffereli, és a szerver folyamat egy szálkészletből szolgálja ki azokat.

- **Jogosultság-ellenőrzés** – a Binder hívásoknál a kernel driver rögzíti a hívó UID-jét és PID-jét, így a szerver oldalon a `IPCThreadState::getCallingUid()` hívásával ellenőrizhető a hívó azonosítója. Ez az alapja az Android permission modelljének.

25.2.7 Példa egy Binder interfészre

Az alábbi pszeudokód egy egyszerű AIDL interfészt mutat be, amely egy `computeSum` metódust definiál:

```
package com.example.compute;

interface ICalculator {
    int computeSum(int a, int b);
}
```

A generált Proxy és Stub osztályok lehetővé teszik, hogy a kliens oldalon a következőképpen hívjuk a metódust:

```
ICalculator calculator = ICalculator.Stub.asInterface(service);
int result = calculator.computeSum(3, 4);
```

A Binder driver kezeli a paraméterek sorosítását (parceling), a hívás átvitelét a szerver folyamatba, a metódus végrehajtását, majd az eredmény visszaküldését.

A Binder fontos tulajdonsága, hogy a tranzakciók mérete korlátozott (alapértelmezés szerint 1 MiB, de konfigurálható). A nagy adatok átadására az ashmem vagy a dmabuf használható, ami a memóriát osztottá teszi a folyamatok között.

25.2.8 6. Engedélyezési modell és sandbox

Az Android biztonsági modellje több rétegből áll:

- **UID-alapú izoláció** – minden alkalmazás saját Linux UID-del és GID-del rendelkezik. Az alkalmazások nem férhetnek hozzá más alkalmazások fájljaihoz vagy processzeihez, kivéve, ha a SELinux policy vagy a megosztott UID ezt kifejezetten lehetővé teszi.
- **Permissions (engedélyek)** – az alkalmazásoknak deklarálniuk kell az általuk igényelt jogosultságokat a `AndroidManifest.xml` fájlban. Az engedélyek két csoportra oszthatók:
  - **Normal permissions** – alacsony kockázatú engedélyek (pl. internet-hozzáférés), amelyeket a rendszer automatikusan megad a telepítés során.
  - **Dangerous permissions** – magasabb kockázatú engedélyek (pl. kamera, mikrofon, helyadatok, kapcsolatok), amelyeket a felhasználónak futás közben explicit módon jóvá kell hagynia (runtime permission). A rendszer az engedélyeket az alkalmazás UID-jéhez kapcsolja, és a Binder hívások során a szerver oldalon ellenőrzi azokat.
- **SELinux** – a rendszer minden folyamata és erőforrása SELinux címkével rendelkezik, és a policy szabályozza a hozzáféréseket. A SELinux erősíti a UID-alapú izolációt, mert akkor is megtagadja a hozzáférést, ha az UID-ek megegyeznek, de a policy nem engedélyezi. A HAL szolgáltatások és a rendszerfolyamatok is SELinux domainekkel vannak ellátva.
- **seccomp** – az Android a Linux seccomp szűrőket használ az alkalmazások és szolgáltatások rendszerhívásainak korlátozására. Ez csökkenti a kernel támadási felületét, mert a nem szükséges rendszerhívások blokkolva vannak.

- **App sandbox** – az alkalmazások homokozóban futnak, ami azt jelenti, hogy csak a saját fájljaikhoz és a számukra engedélyezett erőforrásokhoz férnek hozzá. A homokozó a fenti mechanizmusok kombinációján alapul.

A HAL szolgáltatásokhoz való hozzáférést is a SELinux policy és a Binder jogosultság-ellenőrzés szabályozza. Például egy kamera HAL szolgáltatás csak olyan klienseket szolgálhat ki, amelyek rendelkeznek a megfelelő SELinux címkével és a kamera engedéllyel.

25.2.9 7. Gyakorlati példa: egy egyszerű HAL interfész vázlata

Az alábbiakban bemutatunk egy minimális HAL interfész definíciót AIDL-ben (pszeudokód), amely egy beágyazott szenzor (például hőmérséklet-érzékelő) absztrakcióját valósítja meg.

AIDL interfész definíció (fájl: `IAmbientTemperature.aidl`):

```
package android.hardware.ambienttemperature;

interface IAmbientTemperature {
    // Visszaadja a jelenlegi hőmérsékletet Celsius fokban.
    float getTemperature();
    // Regisztrál egy callback-et a hőmérséklet-változásokra.
    void registerCallback(ITemperatureCallback callback);
    // Törli a callback regisztrációt.
    void unregisterCallback(ITemperatureCallback callback);
}

interface ITemperatureCallback {
    void onTemperatureChanged(float newTemperature);
}
```

HAL szolgáltatás implementáció vázlat (C++ pszeudokód):

```
#include <android/hardware/ambienttemperature/IAmbientTemperature.h>
#include <android/hardware/ambienttemperature/ITemperatureCallback.h>
#include <android/binder_interface_utils.h>
#include <android/binder_manager.h>
#include <android/binder_process.h>
#include <cmath>
#include <mutex>
#include <vector>

using android::hardware::ambienttemperature::IAmbientTemperature;
using android::hardware::ambienttemperature::ITemperatureCallback;

class AmbientTemperature : public IAmbientTemperature {
public:
    AmbientTemperature() {}
    ~AmbientTemperature() {}

    float getTemperature() override {
        // Itt történne a tényleges hardver olvasás.
        // Példa: I2C buszról olvasott érték.
        return 23.5f; // helyőrző
    }

    void registerCallback(const std::shared_ptr<ITemperatureCallback>& callback) override {
        std::lock_guard<std::mutex> lock(mCallbacksMutex);
        mCallbacks.push_back(callback);
    }

    void unregisterCallback(const std::shared_ptr<ITemperatureCallback>& callback) override {
```

```
        std::lock_guard<std::mutex> lock(mCallbacksMutex);
        // Eltávolítás a listából.
    }

private:
    std::mutex mCallbacksMutex;
    std::vector<std::shared_ptr<ITemperatureCallback>> mCallbacks;
};

int main() {
    // Binder thread pool inicializálása.
    ABinderProcess_startThreadPool();
    // Szolgáltatás regisztrálása a hwservicemanager-nél.
    auto service = ndk::SharedRefBase::make<AmbientTemperature>();
    const std::string serviceName = "default";
    binder_status_t status = AServiceManager_addService(
        service->asBinder().get(),
        "android.hardware.ambienttemperature.IAmbientTemperature/default");
    if (status != STATUS_OK) {
        // Hibanaplózás.
        return 1;
    }
    ABinderProcess_joinThreadPool();
    return 0;
}
```

A fenti kód szemlélteti a HAL szolgáltatás regisztrációját és egy interfész implementációját. A gyakorlatban a HAL szolgáltatások gyakran a `hwservicemanager` által kezelt `default` névvel regisztrálódnak, és a kliensek a `AServiceManager_getService` hívással érik el őket. A callback mechanizmus a Binder aszinkron hívásokra épül.

Kliens oldali használat (pszeudokód):

```
#include <android/hardware/ambienttemperature/IAmbientTemperature.h>
#include <android/binder_manager.h>

using android::hardware::ambienttemperature::IAmbientTemperature;

int main() {
    auto service = IAmbientTemperature::fromBinder(
        ndk::SpAIBinder(AServiceManager_getService(
            "android.hardware.ambienttemperature.IAmbientTemperature/default")));
    if (service == nullptr) {
        // A szolgáltatás nem elérhető.
        return 1;
    }
    float temp = service->getTemperature();
    // Felhasználás...
    return 0;
}
```

Ez a minta jól mutatja a HAL absztrakció lényegét: a kliens nem ismeri a hardver részleteit, csak az interfészt.

25.2.10 8. Tipikus hibák és helyes megoldások

Az Android architektúra és HAL fejlesztése során gyakori hibák:

1. **HAL interfész verzió eltérés** – ha a HAL szolgáltatás egy másik verziójú interfészt implementál, mint amit a kliens vár, a Binder hívás sikertelen lesz vagy undefined behavior-t

okoz. Helyes megoldás: mindig ellenőrizni kell a szolgáltatás verzióját a `getService` hívás után, és gondoskodni a kompatibilitásról. A HIDL/AIDL interfészek verziói szigorúan kezelendők.

2. **Binder tranzakció méret túllépés** – ha egy tranzakció paraméterei meghaladják a Binder puffer méretét (általában 1 MiB), a hívás `FAILED_TRANSACTION` hibával megszakad. Helyes megoldás: nagy adatok átadására osztott memóriát (ashmem, dmabuf) vagy fájlleírót használni, nem pedig a Binder tranzakciót.
3. **Hiányzó vagy hibás SELinux policy** – ha a HAL szolgáltatás nem rendelkezik a szükséges SELinux engedélyekkel, a hozzáférés megtagadásra kerül, és a hibanapló `AVC denied` üzeneteket tartalmaz. Helyes megoldás: a SELinux policy-t a gyártónak kell frissítenie, és a fejlesztés során a `dmesg` és `logcat` naplókat kell figyelni.
4. **Szolgáltatás nem indul el időben** – a HAL szolgáltatásoknak a rendszerindítás során megfelelő sorrendben kell elindulniuk. Ha egy kliens túl korán próbálja elérni a szolgáltatást, a `getService` null értéket ad vissza. Helyes megoldás: a HAL szolgáltatást a megfelelő init script indítja, és a kliensek retry logikát vagy callback regisztrációt használnak.
5. **Deadlock a Binder hívásokban** – ha egy HAL szolgáltatás egy másik Binder szolgáltatást hív, miközben az őt hívó folyamat is ugyanarra vár, deadlock alakulhat ki. Helyes megoldás: a hívási láncokat aszinkronná tenni, vagy a Binder thread pool méretét megfelelően beállítani.
6. **Engedély nélküli hozzáférés** – a HAL szolgáltatásoknak ellenőrizniük kell a hívó UID-jét és a SELinux címkéjét, mielőtt érzékeny műveletet hajtanának végre. Helyes megoldás: a HAL implementációban a `IPCThreadState::getCallingUid()` használata, és a SELinux policy megfelelő konfigurálása.

25.2.11 9. Labor: Android-architektúra és HAL-interfész vázlat

Az alábbi labor egy emulált Android környezetben mutatja be a HAL szolgáltatások listázását és egy egyszerű HAL interfész vázlatának elkészítését. A labor célja, hogy a hallgató gyakorlatban is megfigyelje az Android rétegeit és a HAL működését.

25.2.12 Kiinduló állapot

- Telepített Android SDK platform-tools és egy működő Android emulátor (például Android Studio Emulator vagy Cuttlefish). A laborhoz használt konfiguráció:
  - Emulátor: Android Virtual Device (AVD) név: `os_hal_test`
  - Rendszerkép: `system-images;android-33;google_apis;x86_64`
  - Architektúra: x86_64
  - RAM: 2 GiB
  - CPU: 2 mag
- A `adb` parancs elérhető a PATH-ban.

25.2.13 Lépések

1. Indítsd el az emulátort az alábbi paranccsal (terminálban):

   ```
   emulator -avd os_hal_test -no-window -no-audio -no-boot-anim &
   ```

   Várj a rendszer teljes indulásáig:

   ```
   adb wait-for-device
   adb shell getprop sys.boot_completed
   ```

   A kimenetnek `1`-et kell mutatnia.

2. Listázd a futó HAL szolgáltatásokat:

   ```
   adb shell lshal --list
   ```

   A kimenetben megjelennek a regisztrált HAL interfészek, például:

   ```
   android.hardware.graphics.composer@2.1::IComposer
   android.hardware.sensors@2.1::ISensors
   android.hardware.audio@6.0::IDevicesFactory
   ...
   ```

3. Válassz egy HAL interfészt (például `android.hardware.sensors@2.1::ISensors`), és kérd le a részletes információkat:

   ```
   adb shell lshal --debug | grep -A 5 "android.hardware.sensors"
   ```

   Vagy használd a `dumpsys` parancsot:

   ```
   adb shell dumpsys sensorservice
   ```

4. Készíts egy egyszerű HAL interfész vázlatot (pszeudokódban vagy AIDL-ben) egy tetszőleges eszközhöz, például egy hőmérséklet-érzékelőhöz. A vázlat tartalmazza:

   - Az interfész nevét és csomagját.
   - Egy metódust az aktuális érték lekérdezésére.
   - Egy callback regisztrációs metódust.
   - Rövid leírást a szolgáltatás regisztrációjáról.

5. Dokumentáld a folyamatot egy reprodukálható jegyzetfüzetben (például Markdown vagy Jupyter notebook). Rögzítsd az alábbi mérési naplót:

Mérési pont	Parancs / művelet	Mért eredmény	Várt eredmény	Eltérés
Rendszerindítás	`getprop sys.boot_completed`	`1`	`1`	nincs
HAL lista	`lshal --list`	a kimenetben szerepel legalább 5 HAL interfész	legalább 5 HAL interfész	nincs
Szenzor HAL	`lshal --debug | grep sensors`	a kimenet tartalmazza a `android.hardware.sensors` bejegyzést	a bejegyzés jelen van	nincs

25.2.14 Visszaállítás

A labor után állítsd le az emulátort:

```
adb emu kill
```

Töröld az AVD-t, ha szükséges:

```
avdmanager delete avd -n os_hal_test
```

A mért és várt eredményeket a fenti táblázatban külön tüntettük fel. A labor során keletkezett naplófájlokat mentsd el a reprodukciós csomag részeként.

25.2.15 10. Önellenőrző kérdések

1. Milyen fő rétegekből áll az Android platform, és hol helyezkedik el a HAL?
2. Mi a különbség a legacy HAL (libhardware) és a HIDL/AIDL alapú HAL között?
3. Hogyan működik a Binder IPC, és miért alkalmas az Android folyamatközi kommunikációjára?
4. Milyen Android-specifikus kiegészítéseket igényel a Linux kernel, és mi a céljuk?
5. Melyek a leggyakoribb hibák a HAL fejlesztés során, és hogyan kerülhetők el?

25.2.16 11. Összefoglalás

Az Android architektúra rétegzett felépítése lehetővé teszi, hogy a hardvergyártók és a rendszerfejlesztők egymástól függetlenül dolgozzanak. A HAL interfészréteg kulcsszerepet játszik a hardver absztrakciójában, a Binder IPC pedig a folyamatok és szolgáltatások közötti biztonságos kommunikációt biztosítja. Az engedélyezési modell és a SELinux tovább erősíti az izolációt, így az Android platform alkalmas a modern mobil és beágyazott eszközök igényeinek kiszolgálására. A következő fejezet (iOS, biztonsági modell és sandbox) az iOS biztonsági modelljére és sandbox mechanizmusaira épít az itt bevezetett fogalmakra.

25.3 iOS, biztonsági modell és sandbox

Az előző fejezet az Android moduláris, nyílt forrású felépítését mutatta be. Ezzel szemben az iOS zárt, vertikálisan integrált platform, amelyben az Apple a hardvertől az alkalmazásboltig egységesen tartja kézben a biztonsági döntéseket. Ez az alfejezet az iOS védelmi mechanizmusait, a Secure Enclave, a kódaláírás és a sandbox szerepét, valamint az alkalmazás-életciklus korlátait mutatja be. A cél a fogalmi megértésen túl egy reprodukálható elemzés elvégzése.

25.3.1 1. Az iOS zárt architektúrája

Az iOS architektúrája négy fő rétegre bontható: Cocoa Touch, Media, Core Services és Core OS. A Core OS réteg tartalmazza a Darwin alapú kernelt, amely az XNU hibrid kernelt futtatja. Az XNU a Mach mikrokernel szolgáltatásait (IPC, memóriakezelés, ütemezés) és a BSD alrendszert (fájlrendszer, hálózat, processzkezelés) ötvözi. A kernel felett a rendszerkönyvtárak és a futásidejű környezet (libSystem, dyld) biztosítják az alkalmazások számára a rendszerhívásokat és az API-kat.

A zárt modell két fontos következménnyel jár. Egyrészt a rendszer nem engedélyezi a nem aláírt kód futtatását, így a szoftveres támadási felület kisebb. Másrészt a gyártó által ellenőrzött App Store folyamat csökkenti a kártékony alkalmazások bekerülésének esélyét, bár nem szünteti meg teljesen. A felhasználó által telepíthető alkalmazások mindegyike homokozóban fut, és az adatokhoz való hozzáférést engedélyekhez köti a rendszer. Az iOS tehát nem csupán egy operációs rendszer, hanem egy teljes biztonsági architektúra, amelyben a hardver, a kernel, a rendszerszolgáltatások és az alkalmazások együttműködnek.

A biztonsági modell alapelvei a következők: - **Kisebb támadási felület**: a rendszer csak aláírt kódot tölt be, és az alkalmazások nem férhetnek hozzá más folyamatok memóriájához. - **Minden alkalmazás homokozóban fut**: a fájlrendszer, a hálózat és az IPC hozzáféréseket a sandbox profilok korlátozzák. - **Adathozzáférés csak engedéllyel**: a névjegyek, fényképek, helyadatok, mikrofon és más erőforrások eléréséhez a felhasználó jóváhagyása szükséges (TCC). - **Adatvédelem**: a tárolt adatok titkosítva vannak, a kulcsok a Secure Enclave-ban vagy a kernelben védettek.

25.3.2 2. Secure Enclave és hardveres gyökér

A Secure Enclave (SE) egy külön, biztonságos koprocesszor az Apple SoC-kben (A7-től kezdve). Nem azonos a fő CPU-val: saját, minimális operációs rendszert futtat (SEPOS), saját memóriával és tárhellyel rendelkezik, és csak korlátozott, szigorúan ellenőrzött API-n keresztül kommunikál a fő rendszerrel. A SE feladatai közé tartozik:

- **Hardveres véletlenszám-generálás** és kulcsműveletek.
- **Kulcspárok generálása, tárolása és felhasználása** anélkül, hogy a privát kulcsok valaha elhagynák az enclave-ot.
- **Biometrikus adatok védelme**: az ujjlenyomat- és arctérkép-adatok matematikailag reprezentált formában, titkosítva a SE-ben maradnak; a fő operációs rendszer nem fér hozzájuk.
- **Adatvédelmi kulcsok kezelése**: a fájlrendszer-titkosításhoz használt osztálykulcsokat a SE védi, és csak a felhasználó jelszavának vagy biometrikus hitelesítésének sikeres ellenőrzése után adja ki.

A Data Protection mechanizmus lényege, hogy minden fájlhoz egy véletlen fájlkulcs tartozik, amelyet egy osztálykulccsal titkosítanak. Az osztálykulcsok a SE-ben vannak elzárva. Az osztályok (például `NSFileProtectionComplete`, `NSFileProtectionCompleteUntilFirstUserAuthentication`, `NSFileProtectionNone`) azt határozzák meg, hogy a kulcs mikor érhető el. A rendszer így biztosítja, hogy a lemez kiolvasása a hiányzó kulcsok nélkül gyakorlatilag lehetetlen.

Fontos megjegyezni: a Secure Enclave hardveres viselkedését nem lehet QEMU-ban vagy szimulátorban reprodukálni. Az itt leírt működés az Apple hivatalos Platform Security dokumentációján alapul; a pontos verziófüggő részleteket az olvasónak ellenőriznie kell a kiadott dokumentációban. A laborban ezért csak a szoftveres réteget, a sandboxot és az engedélymodellt vizsgáljuk.

25.3.3 3. Code signing és alkalmazásaláírás

Az iOS rendszerben minden futtatható kód – a kernelbetöltőtől az alkalmazásokig – digitálisan alá van írva. A kódaláírás célja kettős: a rendszer ellenőrzi, hogy a kód az Apple vagy egy regisztrált fejlesztő által hitelesített, és hogy a bináris nem módosult a betöltés előtt.

A fejlesztői folyamatban az alábbi összetevők szerepelnek:

- **Fejlesztői tanúsítvány** (Developer Certificate): az Apple által kibocsátott X.509 tanúsítvány, amely a fejlesztő publikus kulcsát köti az Apple által aláírt igazoláshoz.
- **Provisioning Profile**: az Apple által aláírt dokumentum, amely összekapcsolja a fejlesztői tanúsítványt, az alkalmazásazonosítót (bundle ID), az engedélyezett eszközöket és az ún. entitlements-eket. A profil mondja meg, hogy az adott bináris mely eszközökön futtatható.
- **Entitlements**: az alkalmazás aláírásába ágyazott, aláírt jogosultságlisták, például `com.apple.security.application-groups`, `keychain-access-groups`, `aps-environment` a push üzenetekhez, vagy `com.apple.developer.healthkit`. A kernel és a sandbox ezeket a jogosultságokat értelmezi.

A kódaláírás folyamata során a bináris (és a hozzá kapcsolódó dylib-ek) minden kódlapjának hash-e egy code directory-ba kerül, amelyet a fejlesztői privát kulccsal írnak alá. A rendszer a folyamat indításakor ellenőrzi az aláírást: a kernel betölti a binárist, a `dyld` ellenőrzi a code directory-t, és összehasonlítja a tényleges kódlapok hash-eit. Ha bármilyen eltérés van, a folyamat nem indul el. A futás közbeni kódinjektálás megakadályozására az iOS szigorú code signing enforcement-et alkalmaz, amely megakadályozza a módosított vagy nem aláírt kódlapok betöltését.

Az App Store-ban terjesztett alkalmazások esetén az Apple által aláírt provisioning profile biztosítja, hogy a bináris az adott fejlesztőhöz tartozik. A rendszer a telepítéskor is ellenőrzi az aláírás integritását. A kódaláírás tehát nem csupán a származást igazolja, hanem a sandboxhoz is szorosan kapcsolódik: az entitlements az aláírás részét képezi, így azokat nem lehet utólag megváltoztatni anélkül, hogy az aláírás érvénytelenné válna.

25.3.4 4. Sandbox és izoláció

A homokozó (sandbox) a futó folyamatok hozzáférés-szabályozását valósítja meg a kernel szintjén. Az iOS a Seatbelt nevű keretrendszert használja, amely a Mandatory Access Control (MAC) rétegre épül. A sandbox nem az alkalmazás kódjában fut, hanem a kernelben: minden rendszerhívásnál a kernel ellenőrzi a folyamathoz rendelt szabályprofilt, és megtagadja a nem engedélyezett műveleteket.

A sandbox profilok leírják, hogy egy folyamat milyen fájlokat érhet el, milyen IPC-t indíthat, milyen hálózati műveleteket végezhet, és milyen rendszerbeállításokat olvashat. Az iOS alkalmazások számára a rendszer automatikusan generál egy profilt, amely az alkalmazás konténerére és az engedélyezett erőforrásokra korlátozódik. Az alkalmazás az alábbi konténert kapja:

- **Home Directory**: az alkalmazás saját, elszigetelt könyvtára, amely tipikusan a `Documents`, `Library` és `tmp` almappákat tartalmazza. Az alkalmazás csak ide írhat.
- **App Group konténer**: ha az alkalmazás az `com.apple.security.application-groups` entitlementtel rendelkezik, egy megosztott konténeren osztozhat más, ugyanabba a csoportba tartozó alkalmazásokkal.
- **Kulcstartó csoportok**: az alkalmazás csak a saját kulcstartó bejegyzéseihez férhet hozzá, kivéve, ha az aláírt entitlements kiterjeszti.

A sandbox azonban nem csak a fájlrendszerre vonatkozik. Korlátozza az IPC-t is: az alkalmazás nem tud tetszőleges Mach portra csatlakozni, hanem csak az engedélyezett szolgáltatásokhoz fér hozzá XPC-n keresztül. Például a `com.apple.security.exception.files.absolute-path` vagy más kivétel entitlementek nélkül az alkalmazás nem tudja elérni a rendszerkönyvtárakat. A hálózati hozzáférést is szabályozza, bár az alapértelmezett iOS sandbox az alapszintű TCP/UDP kommunikációt engedélyezi.

Az XPC az Apple által preferált IPC-mechanizmus az iOS-ben. Az alkalmazások nem közvetlenül hívják egymás függvényeit, hanem az XPC-n keresztül kérnek szolgáltatást a rendszerfolyamatoktól (pl. SpringBoard, locationd, photolibraryd). A sandbox ezeket a kapcsolatokat is hozzárendeli az adott szolgáltatáshoz tartozó engedélyekhez.

A homokozó tehát egy mélyen a kernelbe integrált, kötelező hozzáférés-szabályozási réteg, amelyet a kódaláírásból származó entitlements egészít ki. Az aláírt, módosítatlan bináris és a hozzá tartozó sandbox profil együtt biztosítja, hogy még egy sebezhető alkalmazás sem tud önkényesen hozzáférni a rendszer erőforrásaihoz.

25.3.5 5. App lifecycle és háttérkorlátok

Az iOS alkalmazások életciklusa szigorúan szabályozott. A fő állapotok:

- **Not Running**: az alkalmazás nem fut, vagy a rendszer leállította.
- **Inactive**: az alkalmazás fut, de nem kap eseményeket (például telefonhívás közben).
- **Active**: az alkalmazás előtérben fut, fogadja az eseményeket.
- **Background**: az alkalmazás háttérben fut, de csak korlátozott ideig, illetve engedélyezett háttérműveleteket végezhet.
- **Suspended**: az alkalmazás a memóriában marad, de nem kap CPU időt. A rendszer tetszés szerint felfüggeszti, és szükség esetén kiüríti a memóriából.

Az átmeneteket a rendszer értesítésekkel jelzi: az `UIApplicationDelegate` (régebbi) vagy a `UISceneDelegate` (iOS 13-tól) metódusai, például `applicationDidEnterBackground`, `sceneWillResignActive`. A háttérbe kerülő alkalmazásnak rövid időn belül el kell mentenie az állapotát, mert a rendszer felfüggesztheti.

A háttérben futó alkalmazások korlátozottak. Alapértelmezés szerint az iOS felfüggeszti az alkalmazást, amint az háttérbe kerül. A fejlesztő az `Info.plist` `UIBackgroundModes` kulcsával kérhet háttérmódot, például:

- `audio`: hang lejátszása a háttérben.
- `location`: folyamatos helymeghatározás.
- `voip`: VoIP-hívások fogadása.
- `fetch`: időszakos háttér-frissítés (background fetch).
- `remote-notification`: push üzenet érkezésekor háttérben futás.
- `processing`: hosszabb háttérfeldolgozás (iOS 13-tól).

Ezek a módok csak akkor engedélyezettek, ha az alkalmazás valóban indokolja a használatot, és a rendszer erőforrás-gazdálkodása korlátozza azok gyakoriságát. A háttérvégrehajtás korlátozásának célja az energiahatékonyság és a hőmérséklet-szabályozás, amit a következő alfejezet (Energiagazdálkodás és hőkezelés) részletesen tárgyal. A sandbox és a háttérkorlátok együtt azt is megakadályozzák, hogy egy alkalmazás háttérben ellenőrizetlenül férjen hozzá adatokhoz vagy erőforrásokhoz.

25.3.6 6. Labor: iOS sandbox- és engedélymodell elemzése

A labor célja, hogy reprodukálható módon megfigyeljük az iOS sandbox konténerizolációját és a TCC-engedélymodell működését. Mivel a teljes iOS kernel és a Secure Enclave nem emulálható hűen QEMU-ban, a labor az Apple Xcode iOS Simulator környezetében fut, amely a macOS kernelén alapul, de a konténerizolációt és a TCC-kezelést a valódi iOS-hez hasonlóan modellezi. Az alábbi leírás példaértékű, a pontos parancsok és verziók a helyi környezettől függnek; az olvasónak ellenőriznie kell a saját Xcode és Simulator verzióját.

25.3.7 6.1. Kiinduló állapot

- macOS 14.x, Xcode 15.x (a pontos verzió a reprodukciós naplóba kerül).
- iOS Simulator eszköz: például iPhone 15, iOS 17.x.
- Két minimális alkalmazásprojekt:
  - `SandboxLab` (bundle id: `com.example.sandboxlab`)
  - `OtherApp` (bundle id: `com.example.other`) – arra szolgál, hogy konténert hozzon létre.
- A projektek nem tartalmaznak extra entitlementeket.

25.3.8 6.2. Build és futtatás

A parancsok bash shellben futtathatók. A következő lépések a szemléltetést szolgálják.

```
# Szimulátor indítása
xcrun simctl boot "iPhone 15" || true

# Build mindkét alkalmazásból
xcodebuild -project SandboxLab.xcodeproj -scheme SandboxLab \
  -sdk iphonesimulator -configuration Release build
xcodebuild -project OtherApp.xcodeproj -scheme OtherApp \
  -sdk iphonesimulator -configuration Release build

# Telepítés a szimulátorra
xcrun simctl install booted build/Release-iphonesimulator/SandboxLab.app
xcrun simctl install booted build/Release-iphonesimulator/OtherApp.app

# Alkalmazások indítása
xcrun simctl launch booted com.example.sandboxlab
xcrun simctl launch booted com.example.other
```

25.3.9 6.3. Kísérlet 1: konténerizoláció ellenőrzése

A `SandboxLab` alkalmazás lekéri a saját `Documents` könyvtárát, majd megpróbál egy fájlt létrehozni a `OtherApp` adatkonténerében, amelyhez nincs jogosultsága. A kód Swift nyelven íródik, és a hiba `NSError` objektumát naplózza.

```
import Foundation

// Saját Documents könyvtár
let docs = NSSearchPathForDirectoriesInDomains(.documentDirectory, .userDomainMask, true).first!
print("SandboxLab Documents: \(docs)")

// OtherApp konténer (a szimulátorban a data konténer útvonala)
// A tényleges útvonalat a `simctl get_app_container` adja meg, itt helyőrző.
let otherContainer = "/Users/\(NSUserName())/Library/Developer/CoreSimulator/Devices/"
                   +
                    ↪  "PLACEHOLDER-UDID/data/Containers/Data/Application/PLACEHOLDER-UUID/Documents/target.txt"

let success = FileManager.default.createFile(
    atPath: otherContainer,
    contents: Data("x".utf8),
    attributes: nil
)
if !success {
    // Az NSError-t a createFile nem adja vissza közvetlenül,
```

```
    // ezért használjunk írást a hibakezeléshez.
    do {
        try Data("x".utf8).write(to: URL(fileURLWithPath: otherContainer))
    } catch {
        print("Write error domain: \(error as NSError).domain)")
        print("Write error code: \(error as NSError).code)")
    }
}
```

A `simctl get_app_container` paranccsal megkaphatjuk a tényleges konténer-útvonalakat; a fenti kódban a helyőrzőket le kell cserélni a mért értékekre. A mérés során a következőket rögzítjük:

- A saját `Documents` könyvtárba írás sikeres-e.
- A másik konténerbe írás hibakódja: várhatóan `NSCocoaErrorDomain`, code 513 (`NSFileWriteNoPermissionError`), vagy a rendszer `EPERM`-hez hasonló hibát ad.
- A hibaüzenet szövege.

25.3.10 6.4. Kísérlet 2: TCC-engedélymodell

A `SandboxLab` alkalmazás hozzáférést kér a fényképtárhoz. A kód a Photos keretrendszert használja:

```
import Photos

let initialStatus = PHPhotoLibrary.authorizationStatus(for: .readWrite)
print("Initial status rawValue: \(initialStatus.rawValue)")

PHPhotoLibrary.requestAuthorization(for: .readWrite) { status in
    print("New status rawValue: \(status.rawValue)")
}
```

A szimulátorban a kérés hatására a rendszer promptot jelenít meg. A prompt megválaszolása automatizálható a `simctl privacy` paranccsal, például:

```
# Az engedély megadása a szimulátoron
xcrun simctl privacy booted grant photos com.example.sandboxlab

# Elutasítás esetén:
# xcrun simctl privacy booted revoke photos com.example.sandboxlab
```

A mérés során rögzítjük:

- A kérés előtti státuszt (várhatóan `notDetermined`, rawValue 0).
- A `grant` parancs futtatása utáni státuszt (várhatóan `authorized`, rawValue 3).
- A kéréstől a státuszváltozásig eltelt időt (jitter-szerű mérés).

25.3.11 6.5. Mérési napló és eredmények

A labor kimenete egy szöveges napló, amely a következő oszlopokat tartalmazza: lépés, mért érték, várt érték, eltérés. Az alábbi táblázat példa a várt eredményekre; a tényleges mért értékeket a helyi futtatás során rögzíteni kell, és a táblázatban a mért oszlop az aktuális kimenetet tartalmazza.

Lépés	Mért érték	Várt érték	Eltérés
Saját Documents írás	sikeres, 1 fájl létrejön	sikeres	nincs
OtherApp konténer írás	hiba code 513, `NSFileWriteNoPermissionError`	hiba, jogosultság hiánya	nincs
10 próba a másik konténerbe	10/10 hiba	10/10 hiba	nincs
Photos engedély státusz kérés előtt	`0` (notDetermined)	`0`	nincs
Photos engedély megadása után	`3` (authorized)	`3`	nincs

Mérőszám: a sandbox elutasítási arány = 10/10 = 1,00 (100%). Az engedélyátmenet sikeres aránya = 1/1 = 1,00 (100%). A mérési naplónak tartalmaznia kell a futtató környezet pontos verzióit (Xcode, Simulator, macOS), a parancsok pontos kimenetét és a hibakódokat. A visszaállítás a szimulátor törlésével vagy az alkalmazások eltávolításával történik: `xcrun simctl uninstall booted com.example.sandboxlab` és `xcrun simctl uninstall booted com.example.other`.

25.3.12 6.6. Értelmezés és korlátok

A labor azt mutatja, hogy a sandbox konténerizoláció és a TCC-engedélymodell a szimulátorban is megfigyelhető. Fontos azonban hangsúlyozni, hogy a macOS alapú szimulátor nem a valódi iOS Seatbelt implementációt futtatja, és a Secure Enclave hardveres szolgáltatásai nem elérhetők. A mért eredmények tehát a modellezett viselkedésre vonatkoznak. Valós eszközön a hibakódok és az engedélykezelés az itt leírtakkal konzisztensek, de a pontos rendszerverziótól függően eltérhetnek.

25.3.13 Összefoglalás

Az iOS biztonsági modellje a hardveres gyökértől (Secure Enclave) a kódaláíráson és a sandboxon át a TCC-engedélyekig egységes, mélységében védett architektúrát alkot. A kódaláírás biztosítja a bináris származását és integritását, a sandbox a futás közbeni hozzáféréseket korlátozza, a Secure Enclave pedig a kulcsok és biometrikus adatok védelmét szolgálja. Az alkalmazás-életciklus szigorú szabályai pedig az energiahatékonyságot és az erőforrás-gazdálkodást támogatják.

Önellenőrző kérdés: Miért nem elegendő önmagában a code signing ahhoz, hogy megakadályozza egy futó alkalmazás jogosulatlan adathozzáférését, és miért van szükség a sandboxra és a TCC-re is? Válaszként fejtse ki, hogy az aláírás csak a kód sértetlenségét és származását igazolja, de a futás közbeni erőforrás-hozzáférést a kernelben a sandbox profilok, a felhasználói adatokhoz való hozzáférést pedig a TCC-engedélyek szabályozzák.

25.4 Energiagazdálkodás és hőkezelés

A mobil és beágyazott rendszerek tervezésénél az energia- és hőgazdálkodás nem csupán optimalizációs kérdés, hanem alapvető tervezési korlát. Egy okostelefon vagy egy telemetriai csomópont tervezője nem adhat meg akkora számítási kapacitást, hogy minden feladatot a lehető leggyorsabban végezzen el, ha az akkumulátornak több évet kell kibírnia, vagy ha a hőt nem tudja elvezetni a passzív ház. Ebben az alfejezetben az energiafogyasztás összetevőit, a dinamikus

feszültség- és frekvenciaskálázást (DVFS), a suspend/resume mechanizmusokat, a hőkezelést, az akkumulátormérést és a wake lock szerepét vizsgáljuk. A cél, hogy a fejlesztő tudatosan válasszon a teljesítmény és az energia között, és megbízható mérési tervet tudjon készíteni.

25.4.1 Energia és teljesítmény alapfogalmak

A teljesítmény (P) az időegységre eső energiafogyasztás, mértékegysége watt. Az energia (E) a teljesítmény időbeli integrálja: $E = P \cdot t$ állandó fogyasztás esetén. Az akkumulátor kapacitását wattórában (Wh) vagy amperórában (Ah) adják meg, így az üzemidő jó közelítéssel

$$t_{\text{life}} = \frac{E_{\text{bat}}}{P_{\text{avg}}},$$

ahol P_{avg} az átlagos teljesítmény. A mérésnél ezért nem egy pillanatnyi csúcsértéket, hanem egy reprezentatív időintervallumra vett átlagot kell rögzíteni.

A mobil és beágyazott platformok fogyasztását több komponens adja:

- a CPU- és GPU-magok dinamikus és statikus fogyasztása,
- a memória (DRAM) frissítése és a flash vezérlő kopásmentesítése,
- a rádiós adóvevő (GSM, LTE, 5G, Wi-Fi, Bluetooth, LoRa),
- a kijelző és a háttérvilágítás,
- az érzékelők, az órajelgenerátor, a PMIC (power management IC) és a szivárgási áramok.

A rendszerszintű optimalizálásnál ezért nem elég a CPU frekvenciáját csökkenteni, mert a rádió vagy a háttérvilágítás fogyasztása nagyobb is lehet. A mérési tervnek azonosítania kell, melyik komponens a domináns, és a beavatkozásnak ahhoz kell igazodnia.

A CPU energiafogyasztását a következő egyszerű modell írja le:

$$P_{\text{cpu}} = P_{\text{stat}} + P_{\text{dyn}},$$

ahol a statikus (szivárgási) komponens

$$P_{\text{stat}} = I_{\text{leak}} V,$$

a dinamikus kapcsolási komponens pedig

$$P_{\text{dyn}} = \alpha C V^2 f.$$

Itt C a kapcsolási kapacitások összege, V a magfeszültség, f az órajelfrekvencia, α pedig az aktivitási tényező, vagyis az óránként átkapcsolt kapacitások hányada. A feszültség négyzetes tagja miatt a feszültség csökkentése sokkal hatékonyabban csökkenti a fogyasztást, mint a frekvencia önmagában. A DVFS ezért mindkettőt együtt szabályozza: alacsonyabb frekvencia mellett a mag alacsonyabb feszültségen is képes működni, ami négyzetesen csökkenti a dinamikus fogyasztást, de a szivárgási áram is csökken a feszültség esésével.

Fontos különbség a teljesítmény és az energia között. Ha egy feladat T idő alatt fut f frekvencián, akkor az elvégzendő munka arányos $f \cdot T$ kapcsolással, tehát az energiaszükséglet közelítőleg

$$E \propto C V^2.$$

Ez azt jelenti, hogy egy feladatot alacsonyabb frekvencián futtatva az energiafogyasztás nem feltétlenül kisebb, mert a futásidő megnő. Ha a feszültség nem csökkenthető arányosan, a hosszabb idő miatt az összesített statikus és dinamikus energia akár nőhet is. A mérési tervben ezért az energiafogyasztást feladathoz, nem időegységhez kell kötni.

25.4.2 DVFS: dinamikus feszültség- és frekvenciaskálázás

A DVFS a CPU, GPU és más processzorok üzemi pontjainak (üzemi feszültség–órajel párosainak) dinamikus váltogatását jelenti. A hardver biztosítja az átváltást, az operációs rendszer pedig egy szabályzó (governor) segítségével dönt arról, hogy milyen frekvencián fussanak a magok.

Linux alatt a cpufreq keretrendszer kezeli a frekvenciaskálázást. A leggyakoribb governorok:

- `performance`: mindig a legmagasabb frekvencián tartja a magot, minimális késleltetés, maximális fogyasztás.
- `powersave`: mindig a legalacsonyabb frekvencián tart, energiatakarékos, de a feladatok lassabbak.
- `ondemand`: a múltbeli CPU-kihasználtság alapján lépked a frekvenciák között, jellemzően egyszerű átlagolással.
- `conservative`: hasonló az ondemand-hez, de lassabban lép felfelé, gyorsabban lefelé, ezért kisebb ingadozást okoz.
- `schedutil`: a Linux futási időmérőjének terhelésadatait használja, és gyakran közvetlenül a ütemezőből hív meg, ezért alacsony késleltetéssel reagál.

A governor kiválasztása kompromisszum. A `performance` ideális lehet akkor, ha a feladat rövid, de futásidejű határidővel, és a feszültségváltás késleltetése nem megengedett. A `powersave` akkor használható, ha a terhelés folyamatosan alacsony, vagy ha a rendszer hőkorlát miatt korlátozza a frekvenciát. A gyakorlatban mobil rendszereken az `schedutil` vagy a platform-specifikus energy-aware governor terjedt el, mert a sched_entity futásideje jó közelítést ad a felhasználói interakciókra adott válaszidőre.

Beágyazott RTOS-eken a DVFS gyakran statikusabb: a FreeRTOS vagy a Zephyr kernelfeladatai közé egy energiamenedzser réteg kerül. A Zephyr power management rendszere a konfigurált policy alapján választhat alvó állapotot, csökkentheti a rendszer órajelét, vagy kikapcsolhat perifériákat. Az RTOS-ekben a determinizmus miatt a belépés és kilépés költségét is ismerni kell: ha a frekvenciaváltás 100 µs-ot vesz igénybe, akkor egy 1 ms-os határidővel rendelkező feladatnál ezt figyelembe kell venni. A frekvenciaváltás késleltetése a mérési terv része kell, hogy legyen.

A cpufreq állapota a sysfs fájlrendszeren keresztül olvasható:

```
$ cat /sys/devices/system/cpu/cpu0/cpufreq/scaling_cur_freq
1200000
$ cat /sys/devices/system/cpu/cpu0/cpufreq/scaling_governor
schedutil
```

A gyakorlati mérésnél rögzíteni kell a frekvenciaváltás időbeli viselkedését is, nemcsak a pillanatnyi értékeket. A `cpupower monitor` vagy az `perf stat` szolgáltathat adatot a frekvenciaváltások számáról és az egyes C-state-ekben töltött időről.

25.4.3 Suspend/resume és wake lock

A teljesítménygazdálkodás másik pillére a komponensek kikapcsolása vagy mély alvásba helyezése, amikor nincs hasznos munka. A rendszer szintű suspend az összes felhasználói folyamatot leállítja, a perifériákat alvó állapotba helyezi, és a RAM tartalmát megtartja (S0ix, S3 vagy S4 hasonló állapot). A resume egy ébresztési forrás hatására visszaállítja a kontextust, újra inicializálja az eszközöket, és visszaadja a vezérlést a folyamatoknak.

Mobil rendszereken a suspend opportunista jellegű: a kernel akkor próbál alvó állapotba lépni, amikor az utolsó ébrenléti forrás is megszűnt. A Linux kernelben az `autosleep` és a `wakeup_source` mechanizmus felelős a rendszerszintű alvásért. Egy illesztőprogram vagy alkalmazás a `wake_lock` (régebbi Android) vagy a kernelben a `wakeup_source` aktívvá tételével megakadályozza a suspend belépést. A felhasználói térben az Android PowerManager API-ja biztosít hasonló funkciót.

A wake lock (ébrenléti zár) célja, hogy olyan kritikus művelet alatt, mint az e-mail letöltés vagy egy szenzoradat-küldés, a rendszer ne aludjon el. Használata egyszerű, de a hibája súlyos: ha az alkalmazás megszerzi a zárat, de a kivételkezelés miatt nem engedi el, a rendszer örökre éber marad, és az akkumulátor gyorsan lemerül. A helyes minta a try/finally szerkezet, vagy a zár automatikus felszabadítása a művelet végén.

A kernelben a wakeup source-ok nyilvántartása megkönnyíti a hibakeresést:

```
$ cat /sys/kernel/debug/wakeup_sources
name                active_count  event_count  wakeup_count  expire_count
...
wlan_rx              0             42           7             0
```

Az `active_count` mutatja, hányszor volt az adott forrás aktívan megakadályozva a suspendet. A nullától különböző aktív állapot a parancs pillanatában éppen ébren tartott rendszerre utal.

A suspend/resume késleltetését is mérni kell. A beágyazott rendszereknél a becsült ébrenléti idő és az alvásból való ébredés ideje határozza meg, hogy egy periodikus szenzorcsomópont mennyi időt tud alvó állapotban tölteni. A mérési módszerhez rögzíteni kell az ébresztés forrását (RTC, GPIO, UART), az alvó állapot típusát, és a resume utáni konzol visszaállási időt. Az ilyen méréseknél a hardverleírás, a kernel konfiguráció és a mérőberendezés felbontása nélkül az eredmény nem reprodukálható.

25.4.4 Thermal throttling és hőkezelés

A fogyasztás hővé alakul, a hőmérséklet emelkedése pedig degradálja a félvezetőket, és végső soron a rendszer kikapcsolását vagy károsodását okozhatja. A hőkezelés célja, hogy a hőtermelés a megengedett határok között maradjon, miközben a hasznos munkát a lehető legkisebb teljesítményvesztéssel végezzük.

A hőegyensúly egyszerű modellje:

$$T_{\text{steady}} = T_{\text{amb}} + \theta_{JA} P,$$

ahol T_{steady} az egyensúlyi maghőmérséklet, T_{amb} a környezeti hőmérséklet, P a teljesítményfelvétel, θ_{JA} pedig a junction-to-ambient hőellenállás (K/W). A gyakorlatban a hőtehetetlenség miatt a hőmérséklet exponenciális közelítéssel követi a terhelésváltozást:

$$T(t) = T_{\text{steady}} + (T_0 - T_{\text{steady}})e^{-t/\tau},$$

ahol $\tau = RC$ a termikus időállandó. A hőkezelő algoritmusok ezért nemcsak a pillanatnyi hőmérsékletet figyelik, hanem a trendet és a múltbeli terhelést is.

A Linux kernel thermal keretrendszere thermal zone-okat definiál, és minden zónához trip pontokat (küszöbértékeket) rendel. A trip pontokhoz hűtő eszközöket kapcsol, például a CPU frekvenciájának korlátozását, a cellás rádió teljesítménycsökkentését, vagy a ventilátort. Amikor a hőmérséklet eléri a passzív trip pontot, a rendszer passzív hűtési intézkedéseket hoz, például csökkenti a CPU frekvenciáját. A kritikus trip pontnál a rendszer általában azonnali rendezett leállítást hajt végre.

A sysfs-ban a zónák megtekinthetők:

```
$ cat /sys/class/thermal/thermal_zone0/temp
45000
$ cat /sys/class/thermal/thermal_zone0/trip_point_0_temp
60000
$ cat /sys/class/thermal/thermal_zone0/trip_point_0_type
passive
```

A hőkezelés hatása jól látszik a teljesítményen: a throttling bekapcsolásakor a frekvencia csökken, a feladat futásideje nő, de a hőmérséklet stabilizálódik. A tervezésnél nem a maximális frekvencia, hanem a fenntartható frekvencia számít, amelyet a termikus csomag hosszú távon el tud viselni. A mérési tervnek ezért tartalmaznia kell a steady-state hőmérsékletet és a throttling belépés utáni stacioner állapotot is.

25.4.5 Akkumulátor, mérés és becslés

A beágyazott és mobil rendszerek akkumulátorának állapotát a feszültség görbéjéből, Coulomb-számlálásból vagy ezek kombinációjából becsülik. A feszültségmérés egyszerű, de a terhelésfüggő belső ellenállás miatt a feszültség nem egyértelműen jelzi a töltöttséget. A Coulomb-számláló integrálja a ki- és beáramló áramot, így pontosabb, de idővel driftel, ezért kalibrálni kell. A modern PMIC-k mindkét módszert támogatják, és a rendszer software állítja össze a SoC (state of charge) és SoH (state of health) becslést.

A rendszerszintű energiafogyasztás mérésének két fő módszere van: a hardveres mérés és a szoftveres becslés. Hardveres mérésnél az akkumulátor vagy a fő betáplálási pont áramát és feszültségét mintavételezik, gyakran a PMIC beépített ADC-jével. A mérési felbontás és a mintavételi frekvencia meghatározza a rövid tranziens fogyasztási csúcsok láthatóságát. A mérési tervben rögzíteni kell a mérőeszköz típusát, a mintavételi frekvenciát, a sönt ellenállását és az átlagolási ablakot.

Szoftveres becslésre a Linux x86 rendszereken a RAPL (Running Average Power Limit) szolgál, amely a CPU és a DRAM bizonyos energiaportjait modellezett értékekkel jelenti. Beágyazott platformokon gyakran a PMIC illesztőprogramja tölti fel az energia szenzorokat, de ezek pontossága gyártófüggő. A mérési naplóban mindig jelölni kell, hogy a szám mért vagy becsült érték, és milyen forrásból származik.

Az energia mérésének alapelve, hogy a munkaterhelés és a nyugalmi állapot is rögzített legyen. A várt és mért értékeket külön kell választani, mert a becslő modellek hibája akár 10–20 % is lehet. A mérési tervben szerepelnie kell a reprodukciós csomagnak is: kernel verzió, build beállítások, konfiguráció, terhelést generáló program és lefuttatott parancsok.

25.4.6 Gyakorlati minták és tipikus hibák

A mobil és beágyazott rendszerek energiaoptimalizálására több bevált minta létezik:

Tickless idle és NOHZ. A periódikus rendszerórajel (tick) felébreszti a CPU-t, még akkor is, ha nincs futtatható feladat. A tickless idle konfiguráció (NOHZ) megszünteti a felesleges ébredéseket, így a CPU hosszabb ideig maradhat mély alvó állapotban. Az energiamérés során az idle megszakítások száma és az egyes C-state-ekben töltött idő jól mutatja a hatását.

Deferrable timers. A nem kritikus karbantartási feladatok (napló kiírása, fényerő-állítás, legközelebbi ébresztés) halasztható időzítőkkel végrehajthatók a következő ébredéshez igazítva, így nem okoznak önálló ébredést.

I/O coalescing. A lemez- és hálózati írásokat érdemes összevonni, és egy nagyobb batch-ben kiírni, mert a flash vezérlő fogyasztása nagyobb apró írásoknál. A fájlrendszerek és a blokkréteg a rejtett késleltetésű kiíráshoz hasonló mechanizmusokat kínálnak.

Ébrenléti idő minimalizálása. A rádiós munkaterhelést és a szenzorlekérdezéseket rövid, koncentrált intervallumba kell szervezni, hogy a rendszer minél hosszabb ideig alhasson. A wakelock csak a kritikus szakaszra tartható, és a művelet végén el kell engedni.

A tipikus hibák közül a leggyakoribbak:

- **Elszivárgott wake lock.** A zár nem engedhető el a hibaüzenet ágban, ezért a rendszer folyamatosan éber. Javítás: a felszabadítás a finally blokkban, vagy a zár birtoklási idejének korlátozása.
- **A governor téves kiválasztása.** Egy beágyazott rendszeren a `performance` governor használata alacsony terhelésnél feleslegesen magasan tartja a frekvenciát. Javítás: a terhelési profil alapján váltás, vagy energy-aware governor alkalmazása.
- **A suspend/resume késleltetés alulbecslése.** A fejlesztő úgy vált gyakran alvó állapotba, hogy az ébredés és a perifériák újrainicializálása miatt a teljes energiafogyasztás nagyobb, mintha rövid ideig ébren maradt volna. Javítás: a belépési és kilépési költségeket mérni kell, és csak akkor aludni, ha az alvás hossza meghaladja a nyereséges küszöböt.
- **A thermal throttling figyelmen kívül hagyása.** A benchmarkokat meleg rendszeren is le kell futtatni, mert a hőkezelés a fenntartható teljesítményt jelentősen csökkenti. Javítás: a mérési tervben a termikus állapotot is rögzíteni kell.
- **Egyetlen mérésre hagyatkozás.** Egyetlen energiaérték nem elegendő, mert a rendszer fogyasztása erősen időben változik. Javítás: átlag, percentilis és a munkaterhelés jellemzői együtt.

25.4.7 Labor: energia- és hőkezelési terv

A labor célja, hogy a fejlesztő egy reprodukálható mérési tervet készítsen emulált környezetben, és a mért eredményeket elkülönítse a várt, levezetett értékektől. A környezet nem valós hardver, ezért az itt kapott értékek oktatási célú becslések, nem helyettesítik a célhardveren végzett mérést.

25.4.8 Konfiguráció

A mérési környezet a következő:

- QEMU 8.2 (ellenőrizendő verzió), x86-64 vendég, 2 vCPU, 512 MiB RAM.

- Linux vendégkernel 6.6 (ellenőrizendő verzió), defconfig alapú, cpufreq, thermal keretrendszer engedélyezve.
- Vendég rootfs: buildroot generált minimal initramfs.
- Terhelést generáló program: C, GCC 12, `-O2`, `-fno-omit-frame-pointer`.
- Mérési módszer: a vendégben a cpufreq sysfs adatait, a `/proc/stat` CPU-időit és a thermal zone hőmérsékletét olvassuk. A hatékonyság becsléséhez a host QEMU processzoridejét is rögzítjük, de ezt nem tekintjük energia mértéknek; csak a vendégbeli frekvencia- és hőadatok alapján dolgozunk.

A mérési naplóban minden futásnál rögzíteni kell:

- a QEMU indítási parancsot,
- a vendég kernel verzióját,
- a cpufreq governor beállítást,
- a thermal trip pontokat,
- a futás hosszát,
- a mért átlagos frekvenciát, hőmérsékletet és a terheléshez tartozó hasznos munka időt.

25.4.9 Forgatókönyvek

1. Idle és suspend/wakeup mérés. A vendéget bootoljuk, és 60 másodpercig nyugalmi állapotban hagyjuk. Rögzítjük az átlagos frekvenciát és a legmélyebb C-state-et. Ezután beállítunk egy RTC ébresztést 10 másodpercre, majd `echo mem > /sys/power/state` paranccsal elindítjuk a suspendet. A wakeup után mérjük a resume konzol visszaállási időt a kernel log timestampje alapján. Az emulált környezetben a suspend nem mindig pontos; a mérési naplóban ezt a korlátot jelöljük.

2. Governor hatása CPU terhelésre. Lefuttatjuk a `busy_loop` nevű programot 60 másodpercig `performance` és `powersave` governorral. Mindkét futásnál rögzítjük az átlagos frekvenciát, a hőmérséklet-emelkedést és a befejezett ciklusok számát. A várt eredmény: a performance futás magasabb átlagos frekvenciát és nagyobb hőmérséklet-emelkedést mutat, de rövidebb idő alatt végez a rögzített ciklusszámú munkával, ha a munkaterhelés CPU-kötött.

A terhelés program minimális változata:

```
/* busy_loop.c - CPU-terhelésgenerátor */
int main(void) {
    volatile unsigned long i = 0;
    const unsigned long limit = 500000000UL;
    for (unsigned long n = 0; n < limit; n++)
        i++;
    return 0;
}
```

A program nem valós benchmark, célja a CPU-mag foglalkoztatása.

3. Thermal throttling szimulálása. A vendég kernelben betöltjük a `rc_thermal_dummy` modult, vagy a QEMU ACPI thermal zone-ját használjuk. A trip pontot alacsonyra (pl. 55 °C) állítjuk, és a `busy_loop`-ot futtatjuk. A várt eredmény: a hőmérséklet eléri a trip pontot, a cpufreq keretrendszer korlátozza a maximális frekvenciát, így a hőmérséklet stabilizálódik a küszöb alatt. A mérési naplóba beírjuk a frekvencia idősort és a hőmérséklet idősort.

A mérési napló sémája:

Futás	Governor	Trip pont	Átlag frekvencia (kHz)	Max hőm. (°C)	Elvárt frekvencia	Elvárt hőm.	Megjegyzés
1	powersave	nincs			alacsony	alacsony	idle
2	performance	nincs			magas	magas	terhelés
3	powersave	nincs			alacsony	alacsony	terhelés
4	performance	55 °C			throttling miatt csökkent	stabil	terhelés

A mérési jegyzetfüzetet egy shell script formájában adjuk meg, amely a parancsokat futtatja, és kimenti a táblázatot. A script például:

```
#!/bin/sh
set -eu
LOG=energy_lab.log
echo "start: $(date)" > "$LOG"
echo "measuring idle 60 s" >> "$LOG"
sleep 60
cat /sys/devices/system/cpu/cpu0/cpufreq/scaling_cur_freq >> "$LOG"
cat /sys/class/thermal/thermal_zone0/temp >> "$LOG"
# ... governor váltás, busy_loop futtatás, mérés
```

A labor elvégzése után a fejlesztő összeveti a mért és várt értékeket. Az eltérés okait a naplóban rögzíti, például a QEMU emuláció pontatlansága, a host CPU schedulingje, vagy a termikus modell hiánya.

A laborhoz tartozó mérőszám: **a feladategységre jutó becsült energiafogyasztás változása governoronként**, amelyet a következő képlet szerint számolunk:

$$E_{\text{task}} = P_{\text{cpu,avg}} \cdot t_{\text{task}},$$

ahol $P_{\text{cpu,avg}}$ a CPU átlagos teljesítménybecslése, t_{task} a feladat befejezési ideje. Az emulált környezetben $P_{\text{cpu,avg}}$ a frekvencia és a feszültség becsléséből adódik; a mérési naplóban ezt a becslést nem keverjük a valós mért energiával.

25.4.10 Önellenőrző kérdések

1. Egy beágyazott szenzorcsomópont minden percben felébred, mintát vesz, és visszaalszik. A fejlesztő a wake lockot a mintavétel után elfelejti elengedni. Milyen hatással van ez az energiafogyasztásra, és hogyan javítható? Válaszát a teljesítmény modell és a suspend mechanizmus alapján indokolja!
2. Adott egy CPU, amelynek a dinamikus fogyasztása $P_{\text{dyn}} = CV^2 f$. Ha a frekvenciát 1 GHz-ről 800 MHz-re csökkentjük, és a feszültséget arányosan 1{,}0 V-ról 0{,}9 V-ra, mennyivel csökken a dinamikus teljesítmény? Miért nem elegendő önmagában a frekvencia csökkentése?
3. Miért fontos a thermal throttling mérésénél a steady-state hőmérséklet, és hogyan befolyásolja a fenntartható processzorteljesítményt?

26 Elosztott és speciális operációs rendszerek

26.1 Elosztott OS és klaszterek

Az elosztott rendszer mint egyetlen rendszerkép

A klaszterek és elosztott operációs rendszerek tervezésekor az egyik legerősebb absztrakció a *single system image* (SSI). Az SSI lényege, hogy több, hálózattal összekötött csomópontot egyetlen számítógépként mutat a felhasználónak és az alkalmazásoknak. A felhasználó tehát nem látja a csomópontok nevét, határait vagy a hálózati ugrásokat: egyetlen parancssorral éri el a teljes rendszert, egyetlen processztáblázatban látja az összes folyamatot, és egyetlen gyökérfájlrendszer alatt éri el az összes állományt. Ez a szemlélet nem csupán kényelmi kérdés, hanem az erőforrás-gazdálkodás, a terheléselosztás és a hibatűrés alapja is.

Az SSI a gyakorlatban több szolgáltatás összehangolását jelenti. Először is szükség van globális névtérre: a fájlrendszer és az elérési utak minden csomóponton ugyanazt a logikai képet nyújtják. Másodszor globális folyamatkezelésre: a `fork` vagy `exec` rendszerhívás nem egy adott csomóponthoz kötött, hanem a rendszer szabadon választja meg a futtató csomópontot. Harmadszor globális IPC-re: a cső, az üzenetsor vagy az osztott memória a csomópontok között is elérhető, nem csak egy gépen belül. Végül a rendszernek folyamatosan figyelnie kell az erőforrásokat: minden csomópont jelenti a terhelését, szabad memóriáját és elérhetőségét, hogy a globális ütemező megalapozott döntést hozhasson.

Fontos hangsúlyozni, hogy az SSI illúzió, és mint minden illúzió, fenntartása kompromisszumokkal jár. A hálózati késleltetés, a csomópontkiesés és a konzisztenciaproblémák nem szüntethetők meg, csak elrejthetők vagy korlátozhatók. A fejezet további része ezeket a határokat és azok kezelését járja körül.

A single system image összetevői

A globális névtér egyik ismert megoldása az elosztott fájlrendszer, amely a metaadatokat replikálja vagy egy központi metaadat-szerveren tartja, az adatblokkokat pedig a csomópontok lokális lemezein osztja szét. A felhasználó számára a `/home/projekt` útvonal minden csomópontról ugyanazt a tartalmat jelenti, akkor is, ha a tényleges blokkok más-más gépen vannak. A folyamatkezelés globálissá tételéhez a processztáblázatot elosztottan kell tárolni, például replikált állapotgépben. Amikor egy folyamat elindul, a rendszer egy egyedi globális PID-t ad neki, és a folyamat állapotát a kiválasztott csomóponton hozza létre. A `kill` vagy `wait` rendszerhívás a globális táblázatban keresi meg a folyamatot, függetlenül attól, hogy az hol fut.

A globális IPC esetében a kihívás a csomópontok közötti adatmozgatás. Egy osztott memória szegmens hálózaton keresztüli elérése nagy késleltetéssel jár, ezért az SSI-rendszerek gyakran távoli eljáráshívással (RPC) vagy üzenetküldéssel valósítják meg a kommunikációt, és csak a ritkán változó adatokat replikálják. A gyakorlatban a „single system image” soha nem jelenti azt, hogy minden művelet olyan gyors, mint egyetlen gépen; inkább azt, hogy a programozói felület egységes.

Elosztott ütemezés (distributed scheduling)

Az elosztott ütemező feladata, hogy a beérkező folyamatokat a klaszter csomópontjaira ossza szét úgy, hogy a válaszidő és az áteresztőképesség optimális legyen, miközben a csomópontok terhelése kiegyenlített marad. Az ütemező lehet központosított vagy decentralizált.

Központosított ütemezőnél egyetlen koordinátor fogadja a folyamatindítási kéréseket, és dönt a célcsomópontról. A döntéshez ismernie kell minden csomópont aktuális terhelését, amit rendszeres szívveréssel frissít. Az előny a globális állapot egyszerű kezelése; a hátrány a koordinátor szűk keresztmetszete és a single point of failure. A koordinátor replikálható, de akkor a replikák közötti konszenzus további késleltetést ad. A döntés komplexitása önmagában $O(1)$, de a terhelési adatok frissítése minden csomóponttól $O(N)$ üzenetet igényel.

Decentralizált ütemezőnél minden csomópont részt vesz a döntésben. Egy új folyamat érkezhet bármelyik csomóponthoz, és az adott csomópont vagy helyben futtatja, vagy egy másik, kevésbé terhelt csomópontra irányítja. A döntéshez a csomópontok kicserélik a terhelési adataikat, vagy egy elosztott hash-táblát használnak a folyamatok hozzárendelésére. A decentralizált megoldás jobban skálázódik, de nehezebb garantálni a konzisztens döntést. Ha minden csomópont minden másikkal közvetlenül cserél adatot, az üzenetek száma $O(N^2)$ lehet, ezért a gyakorlatban hierarchikus vagy gossip-alapú megoldásokat használnak.

Az ütemezési algoritmusok közül a legegyszerűbb a round-robin: minden új folyamat a következő csomópontra kerül, függetlenül a terheléstől. Ez biztosítja az egyenletes eloszlást, de nem veszi figyelembe a folyamatok eltérő erőforrásigényét. A least loaded algoritmus a pillanatnyi terhelés alapján választ, így a rövid távú terheléseloszlás jobb, de ha a terhelési adatok késve érkeznek, oszcillálhat. A hash-alapú ütemezés a folyamat azonosítójának hash értékével választ csomópontot, ami a cache-lokalitást és a determinisztikus hozzárendelést segíti, de a terhelést nem egyenlíti ki. A gyakorlatban kombinált megoldásokat használnak: például hash-alapú kezdeti elhelyezés, majd least loaded áthelyezés, ha a terheléskülönbség meghalad egy küszöböt.

A folyamatmigráció a terheléselosztás eszköze, de költséges. A migráció során a folyamat regisztereit, memórialapjait és nyitott fájlleíróit át kell vinni a célcsomópontra. A migrációs idő közelítőleg $T_{\text{mig}} = T_{\text{state}} + T_{\text{net}} + T_{\text{resume}}$, ahol T_{state} az állapot összegyűjtése, T_{net} a hálózati átvitel, T_{resume} a végrehajtás újraindítása. Ha a folyamat teljes CPU-ideje C_i, akkor a migráció csak akkor éri meg, ha a célcsomópont késleltetése nélkül a befejezési idő legalább T_{mig} értékkel javulna. Ezért a rövid életű folyamatokat általában nem migrálják.

Példaként vegyünk egy három csomópontos klasztert ($N = 3$), ahol minden csomópont azonos sebességű. Négy folyamat érkezik sorban: P_1 CPU-idő $C_1 = 300$, P_2 $C_2 = 500$, P_3 $C_3 = 200$, P_4 $C_4 = 700$. A least loaded ütemező kezdetben a node0-t választja P_1-hez, node1-et P_2-höz, node2-t P_3-hoz. A negyedik folyamatnál a terhelések: node0=300, node1=500, node2=200, ezért P_4 a node2-re kerül. A csomópontokon a folyamatok párhuzamosan futnak, így a befejezési idők: node0: 300, node1: 500, node2: $200 + 700 = 900$. A makespan 900, az átlagos befejezési idő $(300 + 500 + 200 + 900)/4 = 475$. Egyetlen gépen soros végrehajtással a makespan 1700 lenne, tehát a párhuzamosítás jelentős gyorsulást ad. A Gantt-diagramon ez jól látható: a három csomópont sávja eltérő hosszúságú, és a leghosszabb sáv határozza meg a teljes átfutást.

Shared state és konzisztencia

A shared state az SSI központi eleme: a folyamatlista, a szabad kapacitás, a fájlrendszer-metaadatok mind olyan adatok, amelyeket több csomópont is olvas és ír. A konzisztencia azt mondja meg, hogy a csomópontok milyen sorrendben látják az írásokat. A legerősebb modell a lineáris konzisztencia: minden művelet egyetlen, valós időben is érvényes sorrendbe illeszthető, mintha egyetlen példányon futott volna. Ez a felhasználó számára a legtermészetesebb, de fenntartása nagy késleltetést és koordinációt igényel.

A szekvenciális konzisztencia enyhébb: létezik egy globális sorrend, de az egyes csomópontok a saját műveleteiket a program szerinti sorrendben látják, és a globális sorrend nem feltétlenül egyezik a valós idővel. A kauzális konzisztencia csak az ok-okozati láncban lévő írások sorrendjét garantálja; a független írások sorrendje csomópontonként eltérhet. Az eventual consistency pedig megengedi, hogy a replikák átmenetileg eltérő értéket adjanak, de idővel konvergálnak. Az SSI-rendszerekben általában erős konzisztenciára törekszenek, mert a folyamatok indítása vagy leállítása nem tűri az inkonzisztenciát.

A replikált állapot konzisztenciájának gyakorlati eszköze a kvórum (quorum) alapú protokoll. Tegyük fel, hogy az adatot N replika tárolja. Egy írás akkor sikeres, ha legalább W replika megerősíti, egy olvasás pedig akkor, ha legalább R replika válaszol. Ha $R + W > N$, akkor az olvasás mindig talál legalább egy friss írást. Például $N = 3$, $W = 2$, $R = 2$ esetén az írás két replikára kerül, az olvasás kettőt kérdez, így a friss érték garantáltan megjelenik valamelyik válaszban. Ha $W = 1$ és $R = 1$, a rendszer gyors, de az olvasás visszaadhat elavult adatot. A kvórumválasztás tehát késleltetés és konzisztencia közötti mérnöki kompromisszum.

A shared state másik formája az elosztott osztott memória (DSM), amely a fizikai memóriát a csomópontok között osztja meg. A DSM lapokra osztja a memóriát, és a lapok hovatartozását a csomópontok között mozgatja. Ha egy csomópont olyan lapot ír, amely egy másik csomóponton van, a lap átkerül vagy replikálódik. A DSM megvalósítása hasonlít a virtuális memória lapkezelésére, de a laphiba hálózati késleltetéssel jár. Ezért a DSM inkább kutatási és speciális célokra használatos, nem általános célú klaszterekben.

Hibatűrés és konzisztencia

A klaszterekben a hibatűrés alapja a replikáció. A folyamatok állapotát rendszeres időközönként ellenőrzőpontokba (checkpoint) mentik, és a checkpointokat más csomópontokra is másolják. Ha egy csomópont kiesik, a rajta futó folyamatokat a checkpointból újraindítják egy másik csomóponton. Az újraindítás miatt a folyamat befejezési ideje megnő, ezért a hibatűrés és a válaszidő között szintén kompromisszum van. A checkpointok gyakorisága meghatározza az elveszett munkamennyiséget: minél gyakoribb a mentés, annál kisebb a veszteség, de annál nagyobb a mentési overhead.

A csomópontok elérhetőségét szívveréssel (heartbeat) figyelik. Minden csomópont időközönként életjelet küld a koordinátornak vagy a többi csomópontnak. Ha a válasz elmarad, a csomópontot gyanúsnak jelölik, de a gyanú lehet hamis pozitív is, például lassú hálózat miatt. Ezért a failover előtt általában több csomópontnak is meg kell erősítenie a kiesést. A hálózati partíció különösen veszélyes: ha a csomópontok két vagy több csoportra szakadnak, és a csoportok nem tudnak kommunikálni, a replikált állapot könnyen szétválik. Ezt a jelenséget split-brainnek nevezik, amikor mindkét fél magát tekinti aktívnak, és mindkettő írni kezd. A megelőzés eszköze a kvórum: csak az a partíció folytathatja az írást, amelyik a replikák többségét birtokolja.

A CAP-tétel szerint partíció esetén választani kell konzisztencia és rendelkezésre állás között. Egy SSI-rendszer, amely az erőforrás-kezelést végzi, általában a konzisztenciát választja: inkább megtagadja az írást, mint hogy inkonzisztens állapotot engedjen meg. Ehhez a replikált állapotgépet konszenzusprotokollal, például Rafttal vagy Paxosszal tartják szinkronban; ezek részletes bemutatása a következő egységre marad.

Tipikus hibák és helyes megoldások

Az SSI-rendszerek tervezésekor több jellegzetes hiba fordul elő. Az egyik leggyakoribb, hogy a fejlesztő azt feltételezi, a hálózat megbízható és a késleltetés nulla. Ezért olyan szinkron hívásokat tervez, amelyek egy távoli csomópont válaszára várnak, és egy hálózati hiba az egész rendszert blokkolja. Helyes megoldás az aszinkron kommunikáció, időtúllépés és a hibák explicit kezelése.

A másik gyakori hiba a replikáció nélküli központi koordinátor. Ha a globális ütemező egyetlen gépen fut, a gép kiesése leállítja a teljes klasztert. A koordinátort replikálni kell, és a replikák közötti konszenzust meg kell tervezni. A harmadik hiba a konzisztenciamodell hiánya: a programozó úgy ír kódot, mintha minden írás azonnal látható lenne, de a replikáció miatt az olvasások elavult adatot adhatnak. A megoldás a konzisztenciamodell explicit dokumentálása és a kvórumok helyes megválasztása.

Végül gyakori hiba a folyamatmigráció túlzott használata. A rövid folyamatok vándoroltatása a migrációs késleltetés miatt ronthatja a teljesítményt. A helyes megoldás a migráció csak akkor engedélyezése, ha a várható nyereség meghaladja a migrációs költséget.

Labor: klaszterarchitektúra és erőforrás-kezelés

A fejezet laborja egy elosztott ütemező szimulációját kéri, emulált környezetben, jegyzetfüzet formájában. A cél a least loaded ütemezési stratégia kipróbálása, a terheléseloszlás mérése és a mért/várt értékek elkülönítése.

Kiinduló állapot: Python 3.10 vagy újabb, Jupyter Notebook vagy egyszerű Python szkript. A labor nem igényel valódi klasztert; a szimuláció a csomópontokat listaként modellezi. A hoszt gép lemezét, bootloaderét és rendszerpartícióját nem módosítjuk.

Konfiguráció: három csomópont, mindegyik kapacitása 1000 pszeudo-időegység. A folyamatlista: $P_1(C_1 = 300), P_2(C_2 = 500), P_3(C_3 = 200), P_4(C_4 = 700)$. Az ütemező least loaded, a beérkezési sorrend a fenti.

Pszeudokód:

```
# cluster_sim.py - szemléltető szimuláció, nem éles rendszerre
nodes = [0, 0, 0]          # aktuális terhelések
completion_times = []      # befejezési idők
processes = [(1, 300), (2, 500), (3, 200), (4, 700)]

for pid, cpu_time in processes:
    node = nodes.index(min(nodes))
    nodes[node] += cpu_time
    completion_times.append(nodes[node])
    print(f"P{pid} -> node{node}, terheles: {nodes[node]}")

makespan = max(nodes)
avg_completion = sum(completion_times) / len(completion_times)
print(f"makespan: {makespan}, atlagos befejezesi ido: {avg_completion}")
```

A kód szemléltetés, nem valós klaszteren fut. A futtatás:

```
python3 cluster_sim.py
```

Elvárt eredmény: a kimenet node0=300, node1=500, node2=900; makespan=900; átlagos befejezési idő=475. A Gantt-diagramon a node0 sávja 0–300, a node1 sávja 0–500, a node2 sávja 0–200 (P_3), majd 200–900 (P_4).

Mérési napló: a jegyzőkönyvben rögzíteni kell a futtatási környezet verzióját (`python3 --version`), az aktuális időpontot, a bemeneti paramétereket, a generált kimenetet és a mért értékeket. A mért értékeket a várt értékekkel külön oszlopban kell összehasonlítani, például:

Metrika	Várt	Mért	Eltérés
node0 terhelés	300	300	0
node1 terhelés	500	500	0
node2 terhelés	900	900	0

Metrika	Várt	Mért	Eltérés
makespan	900	900	0
átlagos befejezési idő	475	475	0

Visszaállítás: mivel a labor csak szimuláció, nincs szükség különleges visszaállításra. Ha a hoszt környezetben bármilyen fájlt hoztunk létre, célszerű a labor könyvtárát elkülönítve tartani, és a futás után a generált fájlokat megőrizni a reprodukcióhoz.

Gyakorlati hibakeresés: ha a mért értékek eltérnek a várttól, ellenőrizni kell az ütemező indexelését, a bemeneti sorrendet és a kapacitásokat. A leggyakoribb hiba a `min(nodes)` helyett a `max(nodes)` használata, ami a legterheltebb csomópontra ütemez.

Önellenőrző kérdés

Az SSI-réteg tervezésekor a fejlesztő egy központi koordinátort használ replikáció nélkül. Melyik a legvalószínűbb meghibásodási mód? (a) A hálózati partíció esetén a rendszer továbbra is konzisztens marad, de lassú lesz. (b) A koordinátor kiesése az egész rendszer leállását okozza. (c) A folyamatok migrációja felgyorsul. Helyes válasz: (b). A helyes megoldás a koordinátor replikálása és a replikák közötti konszenzus bevezetése.

Összegzés

Az elosztott rendszer mint egyetlen rendszerkép erős absztrakció, amely elrejti a csomópontok határait, de nem szünteti meg a hálózati késleltetést és a meghibásodásokat. A distributed scheduling a folyamatok csomópontokra osztását végzi, a shared state és a konzisztencia a replikált adatok helyességét biztosítja, a hibatűrés pedig a csomópontkiesések kezelését teszi lehetővé. A fejezet laborja egy egyszerű szimulációval mutatja be a least loaded ütemezés működését, és a mért/várt értékek elkülönítését gyakorolja. A következő egység a mainframe és nagy rendelkezésre állású rendszerekre épít az itt bevezetett konszenzus- és hibatűrési fogalmakra.

26.2 Mainframe és nagy rendelkezésre állású rendszerek

A mainframe operációs rendszerek nem egyszerűen nagyobb változatai a szerver- vagy asztali rendszereknek. A tervezési középpontjukat a folyamatos, előre tervezhető, nagy kihasználtságú működés, valamint a hardver- és szoftverhibák elleni szervezett védekezés adja. Az itt használt fogalmak – logikai partíció, workload manager, batch, tranzakció – a magas rendelkezésre állású (HA) rendszerek építőkövei. Ebben az alfejezetben először a mainframe jellemzőit, majd a z/OS legfontosabb komponenseit és a HA elveket tárgyaljuk, végül egy reprodukálható labor segítségével a tervezési döntéseket mérésekkel is alátámasztjuk.

26.2.1 1. A mainframe operációs rendszer jellemzői

A mainframe (nagygép) kategória elsődleges célja a nagy megbízhatóságú, szinte megszakítás nélküli üzem és a nagy számú egyidejű feladat kiszolgálása. A klasszikus mainframe OS-ekre (például a z/OS, illetve annak elődjei) jellemző a **RAS** hármasa:

- **Reliability (megbízhatóság):** a rendszer ritkán hibásodik meg; a hibákat gyakran hardveres redundancia vagy hibajavító mechanizmusok előzik meg, mielőtt a felhasználói munkát érintenék.

- **Availability (rendelkezésre állás):** a tervezett leállások minimalizálása, a javítások és bővítések lehetőleg online, a rendszer leállítása nélkül történnek. A mainframe környezetben a cél gyakran 99,999% vagy annál jobb rendelkezésre állás, ami éves szinten kevesebb mint 5,26 perc tervezett és nem tervezett kiesést jelent.
- **Serviceability (szervizelhetőség):** a hibakereséshez és a cseréhez szükséges diagnosztikai eszközök, naplók és hardveres állapotjelzések integráltan rendelkezésre állnak.

A mainframe OS-t a következő további tulajdonságok különböztetik meg az általános célú szerverrendszerektől:

- **Skálázhatóság:** egyetlen rendszerkép sok száz processzormagot, több TiB memóriát és nagy sávszélességű I/O-t kezelhet. A skálázás nem egyszerűen több gép hozzáadását jelenti, hanem az erőforrások szoros integrációját.
- **Erőforrás-virtualizáció és particionálás:** a hardvert logikai egységekre osztják, és az operációs rendszer ezek felett, illetve ezekben fut. A hardveres hipervizor (PR/SM) biztosítja a particionálást, a z/OS pedig a munkaterhelés-kezeléssel osztja el a kapacitást.
- **Vegyes munkaterhelés:** ugyanazon a rendszeren futhatnak hosszú, erőforrás-igényes batch feldolgozások és rövid, nagy prioritású online tranzakciók. Az OS feladata ezek együttes, zavartalan ütemezése.
- **Tranzakciós integritás:** a mainframe rendszereken gyakran futnak olyan alkalmazások, ahol egyetlen logikai művelet több erőforrást is módosít. Az adatbázis- és tranzakciókezelők (CICS, IMS) szoros együttműködése garantálja az ACID tulajdonságokat.

Ezek a tulajdonságok eltérő operációsrendszer-belső struktúrát igényelnek, mint egy tipikus Linux- vagy Windows-szerver. A mainframe OS központi eleme a **rendszerfelügyelet**, amely folyamatosan figyeli az erőforrásokat, és döntéseket hoz a kapacitás hozzárendeléséről.

26.2.2 2. LPAR: logikai particionálás

A **LPAR (Logical Partition)** a mainframe hardver szintű virtualizációs mechanizmusa. A hardverbe épített **PR/SM (Processor Resource/Systems Manager)** felügyeli a fizikai erőforrásokat – CPU-kat, memóriát, I/O-csatornákat –, és ezeket logikai partíciókhoz rendeli. Minden LPAR saját operációs rendszert futtathat, például z/OS-t, Linuxot a z architektúrán, vagy z/VM-et.

Fontos jellemzők:

- **Erőforrás-izoláció:** egy LPAR nem fér hozzá más partíció memóriájához vagy I/O-eszközeihez, hacsak konfigurált csatornák nem engedélyezik. Ez biztonsági és stabilitási határt ad.
- **Dinamikus kapacitáskezelés:** a CPU-kapacitás egy része (például súlyok vagy garantált minimumok formájában) rendelhető a partíciókhoz. Alacsony kihasználtságú partíciók ideiglenesen több CPU-időt kaphatnak a megosztott poolból, ha a prioritásaik engedik.
- **Fizikai források megosztása:** a memória általában fixen vagy dinamikusan van partíciókhoz rendelve, a CPU-kat időosztásosan használják. Az I/O-eszközöket csatornánként lehet hozzárendelni.

Egy egyszerű LPAR-konfiguráció például így nézhet ki (a értékek szemléltetőek, a valós konfiguráció a hardver és a licenc szerint változhat):

LPAR neve	CPU-súly	Garantált CPU (logikai processzor)	Memória (GiB)	Funkció
PROD	40	4	128	Termelés, tranzakciók
BATCH	30	2	64	Éjszakai batch
DEV	20	1	32	Fejlesztés, teszt
SYS	10	1	16	Felügyelet, mentés

A CPU-súly azt befolyásolja, hogy versenyhelyzetben a partíció milyen arányban kap a szabad CPU-időből. Például ha mind a négy partíció CPU-éhes, a PROD körülbelül a megosztott kapacitás 40%-át kapja. A garantált CPU azt jelenti, hogy a partíció legalább ennyi logikai processzoridőt kap, ha igényli.

A z/OS szempontjából a LPAR az a környezet, amelyben az OS fut. A z/OS nem látja közvetlenül a többi partíciót, csak a saját erőforrásait. A hardveres felügyelet és a z/OS WLM együttműködése teszi lehetővé a hatékony kapacitásgazdálkodást.

26.2.3 3. Workload Manager (WLM)

A **Workload Manager (WLM)** a z/OS egyik legfontosabb komponense. Míg a hagyományos operációs rendszerek ütemezője jellemzően folyamatonkénti prioritásokkal dolgozik, a WLM **szolgáltatási osztályok** (service class) és **célok** alapján dönt. A rendszergazda nem konkrét prioritást ad, hanem azt fogalmazza meg, hogy az adott munkaterhelésnek milyen válaszidő vagy sebesség szükséges.

A WLM működése:

1. **Osztályozás:** a rendszer minden munkát (batch jobot, online tranzakciót, alrendszeri kérést) besorol egy szolgáltatási osztályba. A besorolás történhet felhasználó, program, tranzakciótípus vagy más attribútum alapján.
2. **Célok definiálása:** minden szolgáltatási osztályhoz célokat rendelnek. Például:
   - **Válaszidő-cél:** a tranzakciók 90%-ának 200 ms alatt kell lefutnia.
   - **Sebességcél:** a batch jobok legalább 80%-os sebességgel fussanak (azaz a CPU-idő aránya a várt teljes időhöz képest).
3. **Folyamatos monitorozás:** a WLM méri a tényleges teljesítményt, és összehasonlítja a célokkal. Ha egy osztály nem éri el a célját, a WLM erőforrásokat csoportosít át: több CPU-időt ad, csökkenti más, túlteljesítő osztályok prioritását, vagy az I/O ütemezésen változtat.
4. **Dinamikus döntések:** a WLM másodpercenként vagy akár gyakrabban finomítja az erőforrás-hozzárendelést. Ez a visszacsatolásos vezérlés teszi lehetővé, hogy vegyes munkaterhelés mellett is közel optimális legyen a kihasználtság.

Példa szolgáltatási osztályokra:

Szolgáltatási osztály	Cél típusa	Célérték	Jellemző munka
TSO_INTERACTIVE	Válaszidő (percentil)	90% < 100 ms	Interaktív terminál
CICS_TXN	Válaszidő (percentil)	95% < 200 ms	Online banki tranzakció
BATCH_NORMAL	Sebesség	70%	Éjszakai jelentések
BATCH_URGENT	Sebesség	95%	Zárási folyamat

A WLM nem csupán a CPU-t, hanem a memóriát, az I/O-sávszélességet és a párhuzamos végrehajtási fokot is befolyásolhatja. Ha például a CICS_TXN osztály válaszideje romlik, a WLM emelheti az ehhez tartozó címtér prioritását, és a CPU-idő nagyobb hányadát irányíthatja ide, miközben a batch feladatokat lassítja, ha azok túlteljesítik a céljukat.

Fontos tervezési elv: **célok, nem prioritások**. A statikus prioritásos modellben a rendszergazda nem tudja garantálni a válaszidőt, mert nem ismeri a pillanatnyi terhelést. A WLM célalapú megközelítése a rendszer egészét figyeli, és a szűk keresztmetszeteknél avatkozik be.

26.2.4 4. z/OS architektúra és rendszerkomponensek

A z/OS az IBM z/Architecture mainframe operációs rendszere, a történelmi OS/390 és MVS utódja. Architektúrája moduláris, de erősen integrált. A legfontosabb komponensek:

- **Címterek (address spaces):** minden futó feladat (batch job, TSO-felhasználó, alrendszer, rendszerfeladat) saját virtuális címteret kap. A címtereket a rendszer védi egymástól; a kommunikáció explicit interfészeken (például CSA, SRB, keresztmemória-szolgáltatások) történik.
- **JES (Job Entry Subsystem):** a batch munkák beolvasását, ütemezését, végrehajtását és kimenetkezelését végzi. A JES2 vagy JES3 feladata, hogy a beküldött JCL-állományokat értelmezze, és a jobokat a megfelelő környezetbe helyezze.
- **TSO/E (Time Sharing Option/Extensions):** interaktív felhasználói környezet, terminálon keresztüli parancsbevitelt és ISPF paneleket biztosít.
- **Unix System Services (USS):** a z/OS-en futó POSIX-környezet, amely a Unix API-kat, fájlrendszert és héjprogramokat biztosít. Ez teszi lehetővé a nyílt forráskódú eszközök és a modern alkalmazások futtatását a mainframe-en.
- **RACF vagy más biztonsági szerver:** a felhasználók, csoportok és erőforrások hozzáférésvezérlését végzi. A z/OS biztonsági modellje erőforrás-alapú: minden adathoz, programhoz, tranzakcióhoz külön hozzáférési lista tartozik.
- **Adatbázis- és tranzakciókezelők:** CICS, IMS DB, DB2. Ezek nem részei a z/OS kernelnek, de szoros együttműködésben futnak vele, és a WLM szolgáltatási osztályaiba vannak integrálva.

A z/OS kernel nem monolitikus értelemben vett mikrokernel, hanem rétegzett rendszer, ahol a legalacsonyabb szint a hardveres és megszakítási kezelés, a felette lévő réteg a memória- és folyamatkezelés, majd a rendszer-szolgáltatások (RAS, WLM, JES, USS) következnek. A címterek közötti izoláció és a hardveres storage key mechanizmus további védelmet ad.

26.2.5 5. Batch feldolgozás

A **batch feldolgozás** olyan munkafolyamat, ahol a feladatok emberi beavatkozás nélkül, sorba állítva, előre meghatározott sorrendben futnak le. A mainframe környezetben a batch a klasszikus felhasználási mód: nagy mennyiségű adatfeldolgozás, jelentéskészítés, elszámolás, adatkarbantartás.

A batch jobok leírása **JCL (Job Control Language)** segítségével történik. Egy JCL-állomány tartalmazza:

- **JOB kártya:** a job nevét, elszámolási adatait, osztályát (class), címkéjét.
- **EXEC utasítás:** mely program fusson, milyen paraméterekkel.
- **DD (Data Definition):** a program által használt adatállományok, bemenet, kimenet, hibakimenet.

Példa egy egyszerű JCL-re (szemléltetés, nem futtatható közvetlenül):

```
//PAYROLL  JOB  (ACCT),'PAYROLL RUN',CLASS=A
//STEP1    EXEC PGM=SORT
//SORTIN   DD   DSN=PAY.MASTER.INPUT,DISP=SHR
//SORTOUT  DD   DSN=PAY.MASTER.SORTED,DISP=(NEW,CATLG)
//SYSOUT   DD   SYSOUT=*
```

A batch jobok ütemezése a JES és a WLM együttműködésével történik. A JES a beérkezés sorrendjében vagy prioritás szerint rendezi a jobokat, a WLM pedig a szolgáltatási osztályok céljai szerint osztja el a CPU-időt. Egy batch osztály sebességcélja például „legalább 70%” lehet, ami azt jelenti, hogy a WLM úgy ütemezi a jobot, hogy a tényleges futási idő ne haladja meg jelentősen a tiszta CPU-időt (a várakozási időt is beleszámítva).

A batch feldolgozás előnye a kiszámíthatóság és az erőforrás-hatékonyság: a feladatok sorban haladnak, nem versenyeznek interaktív felhasználókkal a terminálért. Hátránya a rugalmatlanság: egy hiba a teljes sorozat késését okozhatja. A modern mainframe rendszerekben a batch és az online tranzakció egymás mellett fut, a WLM dinamikusan gondoskodik a harmonikus egyensúlyról.

26.2.6 6. Tranzakciófeldolgozás

A **tranzakciófeldolgozás** (online transaction processing, OLTP) a mainframe másik alapvető munkaterhelése. Egy tranzakció általában rövid (másodpercnél rövidebb), interaktív jellegű, és egy vagy több adatbázis-műveletet hajt végre. Példák: banki átutalás, repülőjegy-foglalás, áruházi készletellenőrzés.

A mainframe-en a leggyakoribb tranzakciókezelők:

- **CICS (Customer Information Control System):** általános célú tranzakció-feldolgozó monitor. A CICS kezeli a terminál- és hálózati kéréseket, elindítja a tranzakciós programokat, és biztosítja az adatintegritást.
- **IMS TM (Information Management System Transaction Manager):** nagysebességű, hierarchikus adatbázishoz kapcsolódó tranzakciókezelő, gyakran banki és légitársasági környezetben.

A tranzakciók **ACID** tulajdonságai:

- **Atomicity (atomosság):** a tranzakció vagy teljes egészében végrehajtódik, vagy egyáltalán nem. Hibánál rollback történik.

- **Consistency (konzisztencia):** a tranzakció az adatbázist egyik konzisztens állapotból a másikba viszi át.
- **Isolation (izoláció):** az egyidejű tranzakciók nem látják egymás köztes állapotait.
- **Durability (tartósság):** a befejezett tranzakció hatása commit után túléli a rendszerhibát.

A mainframe tranzakciókezelők szoros kapcsolatban állnak az operációs rendszerrel: a WLM szolgáltatási osztályai válaszidő-célokat kapnak, a rendszer naplózó mechanizmusai (például a z/OS WADS – Write Ahead Data Set) biztosítják a tartósságot, a hardveres redundancia pedig a rendelkezésre állást.

A tranzakciós terhelés jellemző mérőszáma az **átbocsátás** (tranzakció/másodperc) és a **válaszidő** (percben vagy ezredmásodpercben). Egy jól konfigurált z/OS rendszer másodpercenként több ezer tranzakciót is kiszolgálhat, miközben a válaszidő percentilisei alacsonyak maradnak.

26.2.7 7. Magas rendelkezésre állás (HA) alapelvek

A **magas rendelkezésre állás** (High Availability, HA) célja, hogy a rendszer a meghibásodások ellenére is elérhető maradjon. A mainframe világban ez a tervezés alapköve, nem utólagos kiegészítés.

Kulcsfogalmak:

- **MTBF (Mean Time Between Failures):** a hibák közötti átlagos idő.
- **MTTR (Mean Time To Repair):** a hiba észlelésétől a helyreállításig eltelt átlagos idő.
- **Availability (rendelkezésre állás):** $A = \frac{MTBF}{MTBF+MTTR}$.
- **Redundancia:** olyan tartalék erőforrások, amelyek átveszik a hibás elem szerepét. Ez lehet hardver (duplikált tápegység, processzor, I/O-útvonal) vagy szoftver (tartalék példány, replikált adatbázis).
- **Failover (átkapcsolás):** amikor egy aktív elem kiesik, a tartalék automatikusan átveszi a feladatát. A failover lehet tervezett (karbantartás) vagy nem tervezett (hiba).
- **Quorum (kvórum):** elosztott rendszerben a többségi döntés képessége. A split-brain elkerülésére használják: ha a csomópontok nem látják egymást, a kvórummal rendelkező rész folytathatja a munkát.
- **Fencing (elkerítés):** a hibás vagy bizonytalan állapotú csomópont elszigetelése, hogy ne írhasson adatot, amíg a helyreállítás meg nem történik.

A HA tervezésénél gyakran alkalmazott minták:

- **Aktív–passzív:** egy elsődleges csomópont szolgál ki, a tartalék készenlétben áll. Failover esetén a tartalék átveszi a szolgáltatást. Egyszerű, de a tartalék erőforrás kihasználatlan.
- **Aktív–aktív:** mindkét csomópont dolgozik, és megosztják a terhelést. Magasabb kihasználtság, de bonyolultabb konzisztencia-kezelés.
- **N+M redundancia:** N aktív elemhez M tartalék tartozik. Például két aktív adatbázis-kiszolgálóhoz egy közös tartalék.

A HA rendszereknél a **failover idő** kritikus: minél rövidebb, annál kisebb a kiesés. A mainframe környezetben a hardveres és szoftveres integráció gyakran másodpercek alatt képes átkapcsolni.

26.2.8 8. Mainframe HA: Parallel Sysplex és adatmegosztás

A z/OS egyik legerősebb HA-megoldása a **Parallel Sysplex**. Ez a több z/OS rendszerkép (LPAR) szoros együttműködését teszi lehetővé közös adatbázisok és munkaterhelés-megosztás mellett.

Főbb jellemzők:

- **Közös adatelérés:** a Sysplex tagok egyidejűleg hozzáférhetnek ugyanazokhoz az adatokhoz a **CF (Coupling Facility)** nevű hardveres cache-en keresztül. A CF szinkronizálja az adathozzáférést és a zárolásokat, így a tagok konzisztens képet látnak.
- **Terhelésmegosztás:** a bejövő tranzakciókat a Sysplex Distributor (vagy hasonló mechanizmus) elosztja a tagok között. Egy tag kiesése esetén a többi átveszi a munkát, a felhasználók észre sem veszik a hibát.
- **Dinamikus bővítés:** új LPAR adható a Sysplexhez, vagy meglévő leállítható karbantartásra anélkül, hogy a teljes rendszer leállna.
- **Gyors helyreállítás:** a CF és a z/OS Recovery Termination Manager (RTM) automatikusan észleli a hibás tagot, felszabadítja az erőforrásait, és a tranzakciókat újraindítja a többi tagon.

Egy Parallel Sysplex tipikus konfigurációja:

- Több z/OS LPAR egy vagy több fizikai mainframe-en (CEC – Central Electronic Complex).
- Egy vagy több CF LPAR a gyors cache és zárolás céljából.
- Közös időforrás (Sysplex Timer), hogy a naplók és zárolások időbélyege konzisztens legyen.
- Adatreplikáció a CF-ben, így egy CF kiesése sem okoz adatvesztést.

A Parallel Sysplex példa arra, hogyan lehet a HA-t nem csupán hardver-redundanciával, hanem az operációs rendszer és az alkalmazások közötti szoros együttműködéssel megvalósítani. A z/OS képes a munkaterhelést úgy ütemezni, hogy a tranzakciók mindig a leggyorsabban elérhető taghoz kerüljenek.

26.2.9 9. Tipikus hibák és ellenőrzési pontok

A mainframe- és HA-rendszerek tervezésekor gyakori hibák:

1. **Statikus prioritások a WLM helyett:** a rendszergazda kézzel állít be prioritásokat, és nem szolgáltatási célokat. Ez túlterhelés esetén kiszámíthatatlan viselkedéshez vezet, mert a prioritások nem igazodnak a pillanatnyi igényekhez. Helyes megoldás: WLM-célok definiálása, és a rendszerre bízni a dinamikus döntést.
2. **Nem megfelelő kapacitástervezés:** a batch és a tranzakciós csúcsot nem külön elemzik. Például az éjszakai batch elindul, és elszívja a CPU-t a még futó online tranzakcióktól. Megoldás: a WLM célok és az időalapú ütemezés összehangolása, például a batch osztály sebességcéljának csökkentése csúcsidőben.
3. **HA tervezésnél elhanyagolt hálózati és tárolóhibák:** csak a kiszolgáló redundanciájára koncentrálnak, de a hálózati kapcsolat vagy a közös tároló egyetlen hibapont marad. A teljes elérési utat elemezni kell, beleértve a DNS-t, a load balancert és a tápellátást.
4. **Split-brain forgatókönyv nem kezelt:** két aktív csomópont egyszerre ír, mert elvesztették a kapcsolatot, de mindkettő aktívnak hiszi magát. Ez adatkorrupcióhoz vezet. A kvórum és a fencing mechanizmusok kötelezőek.
5. **Nem tesztelt failover:** a tartalék rendszert soha nem próbálták élesben. Az első valódi hiba esetén kiderül, hogy a failover script hibás, a konfiguráció hiányos, vagy a tartalék kapacitása nem elég. Rendszeres, dokumentált failover teszt szükséges.

6. **Mérési adatok hiánya:** a rendelkezésre állást „99,9%” megérzéssel adják meg, valós mérés nélkül. A HA-rendszerekben a rendelkezésre állást folyamatosan monitorozni kell, és a kiesési időket naplózni.

26.2.10 10. Labor: mainframe- és HA-architektúra vázlata és szimuláció

Ez a labor egy reprodukálható jegyzetfüzetben (például Jupyter) vagy emulált környezetben végezhető el. A cél egy egyszerű modell elkészítése, amely bemutatja a batch és tranzakciós munkaterhelés egyensúlyát, valamint egy kétcsomópontos HA-klaszter failoverét.

Célkitűzés: készítsen architekturális vázlatot egy mainframe-szerű rendszerről, amelyben egy WLM-szerű ütemező osztja el a CPU-időt batch és online tranzakciók között, továbbá modellezze a HA failover viselkedését. A mérési naplóban rögzítse a konfigurációt, a mért és várt értékeket külön.

Környezet: Python 3.10+ Jupyter Notebook, nem igényel mainframe hardvert. A mérésekhez a notebook a `numpy` és `matplotlib` csomagokat használhatja. A hardver: bármely hoszt, de a futási időt, processzort és memóriát dokumentálni kell. A mérési módszer: szimulációs időalapú eseménylista, minden lépésben rögzítjük a CPU-kiosztást és a várakozási sorokat.

Szimulációs modell:

- CPU-idő egység: 1 ms.
- Két munkaterhelési osztály:
  - `BATCH`: sebességcél 70%, érkezési ráta 5 job/s, minden job CPU-igénye egyenletes eloszlás 50–150 ms.
  - `OLTP`: válaszidő-cél 90% $<$ 200 ms, érkezési ráta 20 tranzakció/s, minden tranzakció CPU-igénye 5–15 ms.
- A WLM egyszerűsített szabálya:
  - Ha az OLTP 90. percentilis válaszidő $>$ 200 ms, akkor a CPU-idő 80%-át az OLTP kapja, 20%-át a batch.
  - Ha az OLTP rendben van, akkor a CPU-idő 60%-át a batch, 40%-át az OLTP kapja.
- A szimuláció 10 percig fut, a lépésköz 1 ms.

HA modell:

- Két aktív–passzív csomópont. Mindkettő szolgáltatást nyújt, de az egyik az aktív.
- A hiba bekövetkezési ideje exponenciális eloszlású, MTBF=1000 s, a failover idő fix 5 s (várt érték).
- A szimuláció során mérjük a tényleges rendelkezésre állást: a teljes futási időből mennyi idő alatt volt a szolgáltatás elérhető.
- A split-brain szimulációja opcionális: ha a hálózati hiba mindkét csomópontot aktívvá teszi, az adatkorrupciót okozna, ezért a kvórum mechanizmust modellezni kell.

Elvárt eredmények (várt értékek):

- A batch sebesség közelíti a 70%-ot, azaz a batch jobok teljes átfutási ideje nem haladja meg jelentősen a CPU-igényt.
- Az OLTP 90. percentilis válaszidő 200 ms alatt marad.
- A HA szimulációban a rendelkezésre állás legalább 99,5%, a failover idő átlaga 5 s körül, a maximális kiesés nem haladja meg a 10 s-ot.
- A kvórummal ellátott modellben split-brain esemény nem vezet adatkorrupcióhoz.

Mérési napló:

A notebook rögzítse a következőket:

- Hoszt CPU, RAM, operációs rendszer verzió, Python verzió.
- A szimuláció futási ideje, a használt véletlenszám-generátor seed értéke.
- A mért értékek táblázatban: batch átfutási idő átlaga és szórása, sebességcél teljesülése; OLTP válaszidő percentilisek (50., 90., 95., 99.); rendelkezésre állás százalékban; failover idő átlaga.
- A várt értékeket külön oszlopban tüntesse fel.
- A mérési diagramok: Gantt-szerű ábra a CPU-kiosztásról, hisztogram a válaszidőkről, idővonal a failover eseményről.

Példa a szimuláció kódjából (rövidített, szemléltetés):

```
import random, numpy as np

random.seed(42)
# CPU_idő egység: 1 ms
BATCH_JOBS = 5.0/1000  # érkezési ráta ms-onként
OLTP_TXN = 20.0/1000
BATCH_CPU_MEAN, BATCH_CPU_STD = 30, 10  # ms
OLTP_CPU_MEAN, OLTP_CPU_STD = 10, 3

def simulate(duration_ms=600000):
    batch_queue = []
    oltp_queue = []
    batch_done = []
    oltp_rt = []
    for t in range(duration_ms):
        # érkezések
        if random.random() < BATCH_JOBS:
            batch_queue.append(random.gauss(BATCH_CPU_MEAN, BATCH_CPU_STD))
        if random.random() < OLTP_TXN:
            oltp_queue.append(random.gauss(OLTP_CPU_MEAN, OLTP_CPU_STD))
        # WLM döntés: ha oltp percentile > 200, akkor 80/20, különben 60/40
        # egyszerűsített: az utolsó 1000 ms oltp válaszidőt figyeljük
        # ...
        # CPU kiosztás: batch vs oltp
        # ...
    return batch_done, oltp_rt
```

A teljes notebook a kiadási csomagban megtalálható; itt csak a módszertant mutattuk be.

Ellenőrző kérdések:

1. Mi a különbség a statikus prioritásos ütemezés és a WLM célalapú megközelítése között? Miért jobb az utóbbi vegyes munkaterhelésnél?
2. Hogyan biztosítja a Parallel Sysplex a magas rendelkezésre állást? Milyen szerepe van a Coupling Facilitynek?
3. Mi a split-brain jelenség, és milyen mechanizmusokkal előzhető meg?
4. Egy HA rendszerben az MTBF 2000 óra, az MTTR 2 óra. Mekkora a rendelkezésre állás? (Számítsa ki az $A = MTBF/(MTBF + MTTR)$ képlettel, és adja meg százalékban.)
5. A labor szimulációjában miért fontos a véletlenszám-generátor seed értékének rögzítése?

Válaszok röviden:

1. A statikus prioritások nem alkalmazkodnak a terhelés változásaihoz, a WLM folyamatosan méri a szolgáltatási célok teljesülését, és dinamikusan osztja el az erőforrásokat.

2. A Parallel Sysplex több z/OS rendszerképet és CF-et kapcsol össze közös adatokkal; egy tag kiesésekor a többi veszi át a munkát, a CF biztosítja a konzisztens adatelérést.
3. A split-brain akkor fordul elő, ha a fürt csomópontjai elvesztik a kapcsolatot, és mindkettő aktívnak hiszi magát. A kvórum és a fencing megakadályozza, hogy mindkét fél egyszerre írjon.
4. $A = 2000/(2000 + 2) = 0,999 = 99,9$.
5. A seed rögzítése biztosítja a reprodukálhatóságot: ugyanazzal a seeddel a szimuláció ugyanazt az eseménysorozatot adja, így a mért értékek összehasonlíthatók.

A fejezetben bemutatott mainframe- és HA-fogalmak alapozzák meg a következő egység (HPC, batch és tudományos rendszerek) HPC, batch és tudományos rendszerekkel foglalkozó anyagát. A WLM-szerű dinamikus erőforrás-gazdálkodás és a kvórumalapú fürtkezelés általánosítható elosztott és HPC-környezetekre is.

26.3 HPC, batch és tudományos rendszerek

A nagy számítási klaszterek (High-Performance Computing, HPC) operációs rendszer szempontból nem egyetlen kernel vagy disztribúció, hanem együttműködő rétegek halmaza: a node-okon futó, gyakran testreszabott operációs rendszerek, a klaszterközi kommunikációs réteg, a párhuzamos fájlrendszer és a munkaterhelés-kezelő. A tudományos számítások és a batch feldolgozás más követelményeket támasztanak, mint egy általános célú szerver vagy egy interaktív desktop: a hangsúly a determinisztikus erőforrás-hozzárendelésen, az alacsony késleltetésű üzenetküldésen, a nagy sávszélességű párhuzamos I/O-n és a hatékony visszacsatoláson van. Ebben az alfejezetben a HPC-rendszerek OS-sajátosságait, a Slurm és MPI ökoszisztémát, a párhuzamos fájlrendszereket, valamint a számítási és I/O-korlátokat tárgyaljuk. A fejezet végén egy reprodukálható labor segítségével készítünk HPC-feladatütemezési tervet, és elkülönítjük a várt és a mért eredményeket.

26.3.1 A nagy számítási klaszterek OS-sajátosságai

Egy HPC-klaszter node-jain többnyire általános célú operációs rendszer, például Linux fut, azonban a konfiguráció és a kernelbeállítások jelentősen eltérnek a desktop vagy szerver defaultoktól. A legfontosabb OS-szintű jellemzők:

1. **Batch üzemmód és erőforrás-elkülönítés.** A felhasználók nem közvetlenül egy node-on futtatnak, hanem egy bemeneti sorba küldik a feladatot. Az ütemező (például Slurm) allokálja a node-okat, CPU-kat, memóriát és gyorsítókat. Az operációs rendszer cgroupokkal és namespaces-ekkel biztosítja, hogy az allokált erőforrásokat a feladat ne lépje túl, és a node-on futó többi feladattól izolálva maradjon. A kernel paraméterei, például a `vm.overcommit_memory`, a `vm.swappiness` és a `kernel.sched_min_granularity_ns` a HPC-munkaterhelésre hangolhatók.

2. **Processz-kötés (pinning) és NUMA-tudatosság.** A nagy teljesítményű MPI-feladatok érzékenyek a processzor-affinitásra. Az operációs rendszernek lehetővé kell tennie a processzek egy adott NUMA-node-hoz vagy fizikai maghoz kötését. Ennek tipikus eszköze a `sched_setaffinity()` rendszerhívás, illetve a `numactl` és a `taskset` felhasználói eszközök. A HPC-kernelek gyakran úgy konfigurálják a CPU-ütemezőt, hogy a batch feladatok ne vándoroljanak a node-ok között.

3. **Alacsony késleltetésű hálózati kommunikáció.** A klasszikus TCP/IP stack overheadje túl nagy a szoros csatolású párhuzamos feladatokhoz. A modern HPC-klaszterek InfiniBand vagy RoCE hálózatot használnak, és a kommunikációt kernel-bypass módon, például RDMA-val vagy libfabric segítségével végzik. Ez azt jelenti, hogy a felhasználói tér közvetlenül, a kernel beavatkozása nélkül írhat a hálózati kártya puffereibe, ami radikálisan csökkenti a késleltetést. Az operációs rendszer szerepe ilyenkor a memória regisztrálására és a védelem biztosítására korlátozódik.

4. **Párhuzamos fájlrendszer-integráció.** A klaszterekhez gyakran nagy, párhuzamos fájlrendszer (Lustre, BeeGFS, GPFS) kapcsolódik. Az operációs rendszernek támogatnia kell a nagy sávszélességű párhuzamos olvasást/írást, a stripingot és a collective I/O műveleteket. A kernel I/O-ütemezőjét és a fájlrendszer-mount opcióit a párhuzamos terhelésre kell hangolni.

5. **Checkpoint/restart és hibatűrés.** A hosszú futású tudományos szimulációk gyakran órákig vagy napokig tartanak. Node-hiba esetén a feladat elvesztése elfogadhatatlan. A HPC-rendszerek ezért támogatják az állapotmentést (checkpoint) és a visszaállítást (restart). Az operációs rendszer szintjén ez jelentheti a processzmemória és a regiszterek konzisztens mentését, de a gyakorlatban az alkalmazás-szintű checkpointolás terjedt el. A kernel feladata a gyors, megbízható I/O biztosítása a checkpoint fájlok írásához.

6. **Stateless node-ok és image-alapú provisioning.** A HPC-node-ok gyakran hálózatról bootolnak, és az operációs rendszer image-ét központilag terjesztik. Ez biztosítja, hogy minden node azonos kernelverziót és konfigurációt futtat. A boot utáni testreszabás minimális; a klaszter-ütemező által elvárt daemon (például `slurmd`) automatikusan indul.

7. **Energiagazdálkodás.** A nagy klaszterek energiafogyasztása jelentős költség. Az operációs rendszer és az ütemező együttműködik az energiahatékonyságban: a node-ok alacsony fogyasztású állapotba kerülhetnek, ha nincs feladat, és a P-state/C-state vezérlés a terheléshez igazodik.

26.3.2 Slurm és a munkaterhelés-kezelés

A Slurm (Simple Linux Utility for Resource Management) a legelterjedtebb nyílt forrású HPC-munkaterhelés-kezelő. Nem helyettesíti az operációs rendszert, hanem arra épül: a node-okon futó `slurmd` daemon a kernel primitíveit használja a feladatok indításához és felügyeletéhez. A központi vezérlő (`slurmctld`) tartja nyilván a node-ok állapotát, a sorban álló feladatokat és a partíciókat.

A feladatkezelés főbb elemei:

- **Partíció (partition):** node-ok logikai csoportosítása, például `debug`, `batch`, `gpu`.
- **QoS (Quality of Service):** a feladatok prioritási osztálya, erőforrás-limitje.
- **Foglalás (reservation):** adott időszakra node-ok lefoglalása karbantartásra vagy dedikált futtatásra.
- **Backfill:** az ütemező a sorban előrébb álló nagy feladatok szabad erőforrásait kihasználva kisebb feladatokat indít, ha azok nem késleltetik a nagy feladatot.
- **Preemption:** magasabb prioritású feladat felfüggeszthet vagy leállíthat alacsonyabb prioritásút.

Egy tipikus Slurm batch script a következőképpen néz ki (`bash` nyelvű konfigurációs példa, szemléltetés):

```
#!/bin/bash
#SBATCH --job-name=mpi_matmul
#SBATCH --partition=batch
#SBATCH --nodes=4
#SBATCH --ntasks-per-node=16
#SBATCH --cpus-per-task=1
#SBATCH --mem-per-cpu=2G
#SBATCH --time=01:00:00
#SBATCH --output=matmul_%j.out
#SBATCH --error=matmul_%j.err

srun --mpi=pmix ./matmul 4096
```

A fenti paraméterek a következőket jelentik:

- `--nodes=4`: négy node-ot kér a feladat.
- `--ntasks-per-node=16`: node-onként 16 MPI task (rank).
- `--cpus-per-task=1`: minden rank egy CPU-t kap.
- `--mem-per-cpu=2G`: CPU-nként 2 GiB memória.
- `--time=01:00:00`: a feladat maximális futásideje.

Az `srun` az MPI-feladat indításáért felel. A Slurm a kernel cgroupjain keresztül biztosítja, hogy a feladat csak a kért CPU-kat és memóriát használja. A `--ntasks-per-node` értékét úgy kell megválasztani, hogy a node fizikai magjainak száma ne legyen túllépve; ha a node 16 maggal rendelkezik, a 16 rank helyes, ha minden rank egy magot kap. Ha a node hyperthreadinget használ, a logikai CPU-k száma 32 lehet, de a HPC-munkaterhelések gyakran letiltják a hyperthreadinget a determinisztikus teljesítmény érdekében.

A Slurm ütemezési döntéseit a `squeue`, `sinfo`, `sacct` parancsokkal lehet lekérdezni. A `sacct` a feladat futási idejét, CPU-használatát, memóriafogyasztását és I/O-ját rögzíti. Ezek az adatok a teljesítménymérés alapját képezik.

26.3.3 MPI és OS integráció

A Message Passing Interface (MPI) az elosztott memóriájú párhuzamos programozás szabványa. A HPC-feladatok többségét MPI-vel írják, ahol minden processz (rank) a saját címterében fut, és üzenetküldéssel kommunikál a többi rankkel. Az operációs rendszer számára az MPI-feladat sok különálló processz, amelyeket együtt kell ütemezni, elhelyezni és menedzselni.

Az MPI indítási modellje a következő:

1. A Slurm vagy más erőforrás-kezelő allokálja a node-okat és a CPU-kat.
2. Az `srun` (vagy `mpirun`) minden node-on elindítja a megfelelő számú MPI ranket.
3. A rankok a PMIx (Process Management Interface for Exascale) segítségével kicserélik a hálózati címüket és létrehozzák a kommunikátorokat.
4. A rankok MPI hívásokkal kommunikálnak: `MPI_Send`, `MPI_Recv`, `MPI_Bcast`, `MPI_Reduce`, kollektív műveletek.

Az operációs rendszer szempontjából kritikus a **rank-node elhelyezés** és a **NUMA-affinitás**. Ha egy node-on több rank fut, mint a fizikai magok száma, a kontextusváltás overheadje tönkreteszi a teljesítményt. Ha a rankokat nem kötik egy adott maghoz, a kernel ütemezője szabadon mozgathatja őket, ami a gyorsítótár-hatékonyságot rontja. A helyes indítás ezért `--ntasks-per-node` és `--cpus-per-task` paraméterekkel, valamint CPU-affinitás beállításával történik.

A node-on belüli kommunikációhoz az MPI implementációk shared memory-t használnak. A node-ok közötti kommunikációhoz InfiniBand vagy RoCE hálózaton RDMA-t. Az RDMA lehetővé teszi, hogy a felhasználói processz közvetlenül a távoli node memóriájába írjon a kernel és a távoli CPU bevonása nélkül. Az operációs rendszer feladata a memóriarégiók regisztrálása, a kulcsok cseréje és a védelem.

Egy egyszerű MPI példa (C, szemléltetés, nem végrehajtandó):

```
#include <mpi.h>
#include <stdio.h>

int main(int argc, char **argv) {
    int rank, size;
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);
    printf("Rank %d of %d\n", rank, size);
    MPI_Finalize();
    return 0;
}
```

A fenti kódban az `MPI_Init` inicializálja a kommunikációs réteget, az `MPI_Comm_rank` és `MPI_Comm_size` lekérdezi a rank azonosítóját és a rankek számát. A `printf` a node lokális kimenetére ír, ami a batch rendszerben a feladat kimeneti fájljába kerül.

A gyakori MPI hibák közé tartozik:

- **Túl sok rank egy node-on:** a node memóriája vagy sávszélessége szűk keresztmetszetté válik.
- **Rossz CPU-affinitás:** a rankek egyetlen magon versenyeznek.
- **Blokkoló műveletek sorosítása:** például a `MPI_Send` és `MPI_Recv` helytelen sorrendje holtponthoz vezet.
- **Nem megfelelő kommunikátor használata:** a rankek nem a várt csoportba kerülnek.

26.3.4 Párhuzamos fájlrendszerek

A HPC-feladatok I/O-igénye gyakran meghaladja egyetlen fájlszerver kapacitását. A párhuzamos fájlrendszerek (például Lustre, BeeGFS, GPFS) a fájl adatát több szerver között osztják el, és a kliensek párhuzamosan, nagy sávszélességgel érik el. Az operációs rendszer szempontjából a párhuzamos fájlrendszer egy speciális mount, amely a szokásos POSIX szemantikát nyújtja, de a teljesítmény a hagyományos NFS-től eltérően skálázódik.

A Lustre például a következő komponensekre épül:

- **Metadata Server (MDS):** a könyvtárak, fájlnevek, jogosultságok tárolása.
- **Object Storage Server (OSS):** a fájladat-tömbök (objectek) tárolása.
- **Metadata Target (MDT) és Object Storage Target (OST):** a háttértárolók.
- **Stripe:** a fájl adatait az OSS-ek között osztja el. A `lfs setstripe` paranccsal be lehet állítani a stripe-ok számát és méretét.

A HPC-alkalmazások I/O-ját az MPI-IO szabvány segíti, amely lehetővé teszi, hogy a rankek együttesen írjanak egy fájlt, és a fájlrendszer a kollektív műveleteket hatékonyan hajtsa végre. A helyes striping beállítás kritikus: ha egy nagy fájl egyetlen OSS-re kerül, az I/O sávszélesség korlátozott. Ha a stripe méret túl kicsi, a metaadat-forgalom nő. A tipikus stripe méret 1 MiB-tól 64 MiB-ig terjed, a fájl méretétől és a párhuzamos hozzáféréstől függően.

A checkpoint fájlok írásakor a párhuzamos I/O sávszélesség határozza meg, milyen gyakran lehet menteni. Ha a mentés túl lassú, a számítási idő csökken. A burst buffer vagy a node-lokális NVMe tároló átmeneti gyorsítótárként szolgálhat.

26.3.5 Számítási és I/O-korlátok

A HPC-feladatok két nagy osztályba sorolhatók: **számítási korlátos** (CPU-bound) és **I/O-korlátos** (I/O-bound) feladatok. A CPU-bound feladatoknál a processzorok kihasználtsága magas, az I/O minimális; ilyen például a mátrixszorzás vagy a molekuláris dinamika. Az I/O-bound feladatoknál a szűk keresztmetszet a lemez vagy a hálózati sávszélesség; ilyen például az adatkonverzió vagy a nagy adathalmazok elemzése.

A párhuzamos gyorsítás elméleti határát az Amdahl-törvény adja meg:

$$S(n) = \frac{1}{(1-p) + \frac{p}{n}}$$

ahol p a párhuzamosítható hányad, n a processzormagok száma. Ha $p = 0{,}9$, akkor 16 magon a gyorsítás legfeljebb:

$$S(16) = \frac{1}{0{,}1 + 0{,}9/16} \approx 6{,}4$$

A Gustafson-törvény ezzel szemben a méretnöveléses feladatokat írja le, ahol a párhuzamos hányad a feladat méretével nő.

A gyakorlatban a számítási korlátokat a **roofline-modell** jellemzi: a teljesítményt a processzor lebegőpontos csúcsteljesítménye és a memória sávszélessége korlátozza. Az **arithmetic intensity** (FLOP/byte) határozza meg, hogy a feladat a CPU vagy a memória sávszélesség által korlátozott. A HPC-ütemezőnek ezt figyelembe kell vennie: a magas arithmetic intensityű feladatokat érdemes nagy magas órajelű node-okra helyezni, míg az alacsony intensityű feladatok a memória-sávszélességen osztoznak.

Az I/O-korlátok mérésekor a következő metrikákat kell rögzíteni:

- **I/O sávszélesség** (MiB/s): az olvasott/írt byte-ok és az idő hányadosa.
- **IOPS:** a másodpercenkénti I/O műveletek száma.
- **I/O wait:** a CPU azon aránya, amikor I/O-műveletre vár.
- **Lustre stripe kihasználtság:** az egyes OST-k terhelése.

A batch ütemezőnek a CPU- és I/O-korlátos feladatokat egyaránt optimalizálnia kell. A backfill algoritmus például a nagy CPU-bound feladat szabad erőforrásait I/O-bound kisebb feladatokkal töltheti fel, ha azok nem versenyeznek ugyanazon a szűk keresztmetszeten.

26.3.6 Labor: HPC-feladatütemezési terv készítése

Ez a labor egy emulált HPC-klaszter konfigurációját és egy feladatütemezési tervet mutat be. A cél a Slurm, MPI és párhuzamos fájlrendszer fogalmainak alkalmazása egy ellenőrizhető példán. A labor reprodukálható jegyzetfüzetben (például Jupyter) vagy egyszerű Python szimulációként futtatható. A mérési naplóban a várt (levezetett) és a mért (szimulált) eredményeket elkülönítjük.

26.3.7 Kiinduló állapot és konfiguráció

A szimulált klaszter paraméterei:

Paraméter	Érték
Node-ok száma	4
CPU / node	16 fizikai mag
Memória / node	64 GiB
Hálózat	100 Gb/s InfiniBand (szimulált)
Párhuzamos fájlrendszer	Lustre, 4 OST, stripe szélesség 4, stripe méret 4 MiB
Ütemező	Slurm 24.05 (ellenőrizendő verzió)

A feladatok négy típusúak:

- **A feladat:** CPU-bound mátrixszorzás, 64 MPI rank, futási idő egyedül 600 s, I/O minimális.
- **B feladat:** I/O-bound adatkonverzió, 16 MPI rank, ír 64 GiB adatot, futási idő egyedül 1200 s, I/O sávszélesség 1 GiB/s.
- **C feladat:** vegyes terhelés, 8 MPI rank, futási idő egyedül 400 s.
- **D feladat:** kis teszt, 1 rank, futási idő 60 s.

A kérdés: hogyan ütemezzük ezeket a feladatokat, hogy a teljes make_span (az utolsó feladat befejezési ideje) minimális legyen, és a node-kihasználtság magas maradjon?

26.3.8 Ütemezési stratégia és Slurm paraméterek

A feladatokhoz a következő Slurm paramétereket választjuk (a `#SBATCH` direktívákban):

A feladat:

```
#SBATCH --job-name=cpu_matmul
#SBATCH --partition=batch
#SBATCH --nodes=4
#SBATCH --ntasks-per-node=16
#SBATCH --cpus-per-task=1
#SBATCH --mem-per-cpu=1500
#SBATCH --time=00:15:00
#SBATCH --output=cpu_matmul_%j.out
```

Itt a 64 rank (4 node × 16 rank) pontosan lefedi a klasztert. A memória limit 1500 MiB/CPU, így összesen 64 × 1500 MiB = 96 GiB, ami a node-ok összesen 256 GiB memóriájának csak egy része. A CPU-affinitást a Slurm automatikusan beállítja.

B feladat:

```
#SBATCH --job-name=io_convert
#SBATCH --partition=batch
#SBATCH --nodes=1
#SBATCH --ntasks-per-node=16
#SBATCH --cpus-per-task=1
#SBATCH --mem-per-cpu=2000
#SBATCH --time=00:25:00
#SBATCH --output=io_convert_%j.out
```

A B feladat I/O-bound, ezért egyetlen node-on is elfér, de a párhuzamos fájlrendszer sávszélessége a szűk keresztmetszet. A Lustre 4 OST-vel rendelkezik, ezért a stripe beállítása `lfs setstripe -c 4 -S 4M /scratch/io_convert.dat` a fájl számára 4 OST-re sávoz.

C feladat:

```
#SBATCH --job-name=mixed
#SBATCH --partition=batch
#SBATCH --nodes=1
#SBATCH --ntasks-per-node=8
#SBATCH --cpus-per-task=1
#SBATCH --mem-per-cpu=3000
#SBATCH --time=00:10:00
#SBATCH --output=mixed_%j.out
```

D feladat:

```
#SBATCH --job-name=test
#SBATCH --partition=debug
#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --time=00:05:00
#SBATCH --output=test_%j.out
```

26.3.9 Ütemezési szimuláció

A backfill ütemezőt egyszerű pszeudokódban modellezhetjük. A szimulátor bemenete a feladatok listája (node-igény, futási idő, érkezési idő), kimenete a node-kiosztás és a befejezési idők. A pszeudokód a következő:

```
list jobs = [
    {id:"A", nodes:4, time:600, arrive:0},
    {id:"B", nodes:1, time:1200, arrive:0},
    {id:"C", nodes:1, time:400, arrive:0},
    {id:"D", nodes:1, time:60, arrive:0}
]
available_nodes = 4
queue = jobs sorted by priority (A, B, C, D)

function schedule(backfill_enabled):
    time = 0
    running = []
    finished = []
    while queue or running:
        # indítsunk el minden olyan feladatot, amelynek node-igénye <= available_nodes
        for job in queue:
            if job.nodes <= available_nodes:
                # backfill: csak akkor indítsuk, ha nem késlelteti a legmagasabb prioritású
↪   várakozó feladatot
                if not backfill_enabled or job is highest_priority(queue) or
                   (highest_priority(queue).nodes <= available_nodes + job.nodes):
                    start_job(job)
                    available_nodes -= job.nodes
                    running.add(job)
                    queue.remove(job)
        # előre lépünk a legközelebbi befejezésig
        t_next = min(running.finish_times)
        time = t_next
        for job in running that finish at t_next:
            finish_job(job)
            available_nodes += job.nodes
```

```
            finished.add(job)
    return max(finished.finish_times)
```

A fenti pszeudokódban a `highest_priority(queue)` a sorban legelöl álló feladat. A backfill feltétel biztosítja, hogy egy kisebb feladat ne késleltesse a nagy feladat indulását.

26.3.10 Várt és mért eredmények

A szimulációt két módban futtatjuk: **FIFO** (backfill nélkül) és **backfill** módban. A várt eredményeket a feladatok egyedi futási idejéből vezetjük le; a mért eredményeket a szimulátor adja.

Várt (levezetett) eredmény FIFO módban:

A FIFO sorrend: A (4 node, 600 s), majd B (1 node, 1200 s), majd C (1 node, 400 s), majd D (1 node, 60 s). Mivel a klaszter 4 node-os, az A feladat 600 s-ig mind a 4 node-ot foglalja. Utána B indul, de csak 1 node-ot használ, a maradék 3 node üresen vár. B 1200 s-ig fut, így a make_span = 600 + 1200 + 400 + 60 = 2260 s. A node-kihasználtság az A fázisban 100%, utána 25% (1 node a 4-ből), majd C és D idején is 25%.

Várt (levezetett) eredmény backfill módban:

A backfill lehetővé teszi, hogy az A feladat futása alatt a kisebb B, C, D feladatok ne induljanak el, mert a klaszter 4 node-ját A foglalja. Az A 600 s-ig fut. A B, C, D feladatok a várólistán vannak. A backfill feltétel: a legmagasabb prioritású várakozó feladat B (1 node igényű), így amikor A befejeződik (600 s), B azonnal indul. Ekkor 3 szabad node van, ezért C és D is indulhat backfill-ként, feltéve, hogy nem késleltetik B-t. B node-igénye 1, a szabad node-ok 3, így C (1 node) és D (1 node) is elindulhatnak, és még mindig marad 1 szabad node. A befejezések: C 400 s alatt (600 + 400 = 1000 s), D 60 s alatt (600 + 60 = 660 s), B 1200 s alatt (600 + 1200 = 1800 s). A make_span = 1800 s. A node-kihasználtság az A fázisban 100%, majd a backfill fázisban C és D futása alatt B (1 node), C (1), D (1) = 3/4 = 75%, majd C és D befejezése után B egyedül 1/4 = 25%.

Mért (szimulált) eredmények:

A szimulátor futtatásakor a kimenet a fenti várt értékeket közelíti, de a valós ütemezőben a feladatok indítási overheadje (például 2-5 s) és az I/O interferenciája miatt eltérés lehet. A laborban ezt a hatást úgy modellezzük, hogy minden feladat indításához 3 s overheadet adunk, és az I/O-bound B feladatnál a párhuzamos fájlrendszer sávszélességét 1 GiB/s-nak vesszük, ami a 64 GiB adat írását 64 s alatt teszi lehetővé, de ez elhanyagolható a 1200 s-hoz képest.

A mérési napló a következő mezőket tartalmazza:

Feladat	Indítási idő (s)	Befejezési idő (s)	Node-igény	Várt befejezés (s)	Mért befejezés (s)	Eltérés (s)
A	0	600	4	600	603	+3
B	603	1803	1	1800	1806	+6
C	603	1003	1	1000	1006	+6
D	603	663	1	660	666	+6

A mért make_span = 1806 s, a várt 1800 s, az eltérés 6 s. A node-kihasználtság az A fázisban 100%, a backfill fázisban 75%, majd 25%.

26.3.11 Reprodukciós csomag

A labor reprodukálható módon futtatható egy Python szkripttel (`hpc_sched_sim.py`), amely a fenti pszeudokódot valósítja meg. A konfiguráció egy JSON fájlban (`lab_hpc.json`) tárolható:

```
{
  "nodes": 4,
  "jobs": [
    {"id": "A", "nodes": 4, "time": 600, "arrive": 0, "priority": 1},
    {"id": "B", "nodes": 1, "time": 1200, "arrive": 0, "priority": 2},
    {"id": "C", "nodes": 1, "time": 400, "arrive": 0, "priority": 3},
    {"id": "D", "nodes": 1, "time": 60, "arrive": 0, "priority": 4}
  ],
  "backfill": true,
  "startup_overhead": 3
}
```

A futtatási parancs:

```
python3 hpc_sched_sim.py --config lab_hpc.json --output sim_results.csv
```

A visszaállítás: mivel a szimuláció nem érinti a hoszt rendszert, nincs szükség külön visszaállításra. A mérési napló CSV formátumban exportálható.

26.3.12 Gyakori hibák és helyes megoldások

A HPC-üzemeltetésben és a feladatütemezési tervekben a következő tipikus hibák fordulnak elő:

1. **Túl sok MPI rank egy node-on.** Ha a `--ntasks-per-node` meghaladja a fizikai magok számát, a kernel ütemezője kontextusváltásra kényszerül, és a teljesítmény drámaian romlik. A helyes megoldás a magok számának pontos ismerete és a rankek egy-az-egyhez kötése.

2. **CPU-affinitás hiánya.** A Slurm általában beállítja a processz-affinitást, de ha a felhasználó `srun` nélkül indít processzeket, a kernel szabadon mozgathatja a rankeket. A `--cpu-bind=cores` opció explicit vezérlést ad.

3. **Párhuzamos fájlrendszer stripe félrekonfigurálása.** A Lustre-nél a stripe-ok számát a fájl méretéhez és a párhuzamos hozzáféréshez kell igazítani. Ha a stripe-ok száma 1, a párhuzamos I/O egyetlen OSS-re koncentrálódik. Ha túl sok a stripe, a metaadat-forgalom nő. A helyes beállítást `lfs setstripe` paranccsal kell ellenőrizni.

4. **I/O-bound feladat CPU-nehéz node-ra kerül.** Ha egy I/O-bound feladatot nagy memória-sávszélességű node-ra ütemezünk, a CPU-erőforrás kihasználatlan marad, míg a lemez/hálózat szűk keresztmetszet. A partíciók és QoS segítségével el kell különíteni a CPU-bound és I/O-bound feladatokat.

5. **Memória-limit hiánya.** Ha a feladat nem kap `--mem-per-cpu` vagy `--mem` korlátot, túllépheti a node memóriáját, és a kernel OOM (out of memory) leállítja a feladatot. A Slurm `--mem` paramétere a cgroup memória-limitjét állítja be.

6. **Nincs checkpoint stratégia.** A hosszú futású feladatoknál a checkpoint hiánya node-hiba esetén a teljes munka elvesztését jelenti. A batch scriptben érdemes beépíteni az alkalmazás-szintű checkpointot, és a `--requeue` opciót használni.

26.3.13 Összefoglalás és önellenőrző kérdés

A nagy számítási klaszterek operációs rendszer szempontból a batch üzemmód, a processzkötés, a kernel-bypass hálózat és a párhuzamos fájlrendszer integrációját igénylik. A Slurm a kernel cgroupjaira és namespaces-eire építve allokálja az erőforrásokat, az MPI a rankek közötti alacsony késleltetésű kommunikációt valósítja meg, a párhuzamos fájlrendszerek pedig a nagy sávszélességű I/O-t biztosítják. A számítási és I/O-korlátok elemzése nélkül az ütemező döntései nem optimalizálhatók. A laborban bemutattuk, hogyan készíthető HPC-feladatütemezési terv, és hogyan választható el a várt és a mért eredmény.

Önellenőrző kérdés: Egy HPC-klaszter 8 node-ból áll, minden node 32 fizikai maggal és 128 GiB memóriával rendelkezik. Egy feladat 64 MPI rankot igényel, és minden rank 4 GiB memóriát fogyaszt. Hogyan állítanád be a Slurm `--nodes`, `--ntasks-per-node` és `--mem` paramétereit? Milyen hibát okozna, ha a `--ntasks-per-node=64` értéket adnád meg? Magyarázd el a válaszod.

Válasz vázlata: A 64 rank elhelyezéséhez 2 node szükséges, mivel 32 mag/node áll rendelkezésre. A helyes paraméterek: `--nodes=2 --ntasks-per-node=32 --mem=128G` (vagy `--mem-per-cpu=4G`). Ha `--ntasks-per-node=64` értéket adnánk, akkor egyetlen node-on 64 rank futna, de a node csak 32 fizikai maggal rendelkezik, így a kernel kontextusváltásra kényszerülne, és a teljesítmény súlyosan romlana. Emellett a memóriaigény 64×4 GiB $= 256$ GiB lenne, ami meghaladja a node 128 GiB memóriáját, OOM leállításhoz vezetne.

Ezzel a fejezet lezárja a HPC, batch és tudományos rendszerek áttekintését, és előkészíti a következő alfejezetet, amely a serverless, unikernel és jövőbeli irányok témakörét tárgyalja.

26.4 Serverless, unikernel és jövőbeli irányok

A serverless és az unikernel két olyan megközelítés, amelyek a felhőben, a beágyazott rendszerekben és a biztonságkritikus környezetekben újraértelmezik az erőforrás-izolációt és a végrehajtási modellt. Ebben az alfejezetben a könnyűsúlyú izoláció és az eseményvezérelt futtatás fogalmaiból indulunk ki, majd bemutatjuk a function-as-a-service (FaaS), a microVM és a WebAssembly (WASM) alapú futtatási mintákat. Az indulási idő, a biztonság és a költség szempontjait együtt vizsgáljuk, mert ezek határozzák meg, hogy egy adott munkaterheléshez melyik technológia illeszkedik. Végül egy reprodukálható döntési mátrix kísérletet adunk, amelyben a mért és a várt eredményeket különválasztjuk.

26.4.1 1. A könnyűsúlyú izoláció fogalma

A hagyományos operációs rendszerekben az izoláció alapegysége a folyamat, amelyet a kernel memóriavédelemmel, jogosultságokkal és erőforrás-korlátokkal különít el. Az utóbbi évtizedekben ez a határ tovább mozdult: a hypervisorok virtuális gépeket (VM), a konténerek processzcsoportokat és namespaces-eket, a microVM-ek pedig egyetlen alkalmazás számára kialakított, minimális hardverabsztrakciót izolálnak. A könnyűsúlyú izoláció (lightweight isolation) célja, hogy a biztonsági és erőforrás-elszigetelési garanciák megmaradjanak, miközben a lábnyom és az indulási költség radikálisan csökken.

A fogalmat három tengely mentén érdemes megragadni:

- **Erőforrás-lábnyom:** mennyi memóriát, CPU-időt és tárolást foglal az izolációs határ maga, nem pedig a hasznos alkalmazás.

- **Indulási idő:** mennyi idő kell ahhoz, hogy egy új végrehajtási egység kérés fogadására kész legyen.
- **Biztonsági garancia:** milyen hibák és támadások ellen véd a határ (pl. kernelhiba, alkalmazáshiba, rosszindulatú kód).

Egy klasszikus teljes virtualizálású VM például erős izolációt ad, de a vendég operációs rendszer és a virtuális hardver indítása miatt a lábnyom és az indulási idő nagy. A konténer ugyanazon a kernelen osztozik, ezért indulása gyors és lábnyoma kisebb, de a kernel támadási felülete közös marad, így a biztonsági határ gyengébb. A microVM, az unikernel és a WASM ebbe a térbe illeszkedik, különböző kompromisszumokkal.

Az unikernel esetében az alkalmazást és a szükséges kernelkomponenseket egyetlen, közös címterű, speciális kernelként fordítják le. Nincs külön felhasználói és kernel mód, nincs rendszerhívási határátlépés, és nincs általános célú szolgáltatás, amelyre nincs szükség. Ez a minimalizmus csökkenti a támadási felületet és az indulási időt, de módosítja a programozási modellt: az alkalmazás közvetlenül hívhatja a hardvert vagy a kernelkönyvtárat.

26.4.2 2. Eseményvezérelt futtatás és szerver nélküli modell

A serverless – magyarul gyakran szerver nélkülinek nevezett – modell lényege, hogy a fejlesztő nem virtuális gépeket vagy hosszú életű szolgáltatásokat üzemeltet, hanem eseményekre reagáló függvényeket telepít. Az infrastruktúra-szolgáltató gondoskodik a futtatókörnyezetről, a skálázásról, a hálózatról és a megfigyelhetőségről; a fejlesztő általában csak a kódot és a konfigurációt adja. Ez az eseményvezérelt futtatás (event-driven execution) egyik érett formája.

Az eseményvezérelt modellben a végrehajtási egység életciklusa nem folyamatos, hanem egy-egy eseményhez – HTTP-kéréshez, üzenetsor-üzenethez, időzítőhöz vagy objektumtár-művelethez – kötődik. A platform az esemény érkezésekor kiválaszt egy futásidejű példányt, lefuttatja a függvényt, majd a példányt lebegő állapotban tartja, vagy megsemmisíti. A skálázás sokszor automatikus: több párhuzamos esemény több példányt indít.

A function-as-a-service (FaaS) ennek a modellnek a leggyakoribb implementációja. A FaaS platformok jellemzői:

- **Függvényegység:** a telepíthető kód egyetlen belépési pont, például `handler(event, context)`.
- **Eseményforrások:** HTTP API-k, üzenetsorok, időzítők, objektumtár-események.
- **Erőforrás-korlátok:** memória, CPU-idő, végrehajtási időtartam és párhuzamosság.
- **Hidegindítás:** az első kérés kiszolgálásához szükséges idő, amikor még nincs futó példány.
- **Állapotnélküliség:** a függvények általában nem tárolnak lokális állapotot; a tartós adatot külső szolgáltatásban kell tartani.

Példaként vegyünk egy minimális FaaS függvényt pszeudokódban, amely egy HTTP-eseményre válaszol:

```
/* FaaS handler - szemléltető pszeudokód */
void handler(const event_t *event, response_t *response) {
    const char *name = event_get_query_param(event, "name");
    if (name == NULL || strlen(name) == 0) {
        response_set_status(response, 400);
        response_set_body(response, "missing name parameter");
        return;
    }
    char greeting[128];
    snprintf(greeting, sizeof(greeting), "Hello, %s!", name);
```

```
    response_set_status(response, 200);
    response_set_body(response, greeting);
}
```

A fenti kód nem tartalmaz hosszú életű állapotot, így bármelyik példányon lefuthat. A platform a `handler` hívása előtt felépíteni a futtatókörnyezetet, ami a hidegindítási időt adja. A függvény hívása után a platform eldöntheti, hogy a példányt melegen tartja-e további eseményekhez.

26.4.3 3. MicroVM: minimalista virtuális gép egy alkalmazáshoz

A microVM a teljes virtualizálású VM és a konténer közötti kompromisszum. A cél egy olyan virtuális gép, amely egyetlen alkalmazást futtat, minimális vendég eszközkészlettel rendelkezik, és néhány tíz milliszekundum alatt elindul. A legismertebb nyílt implementációk közé tartozik a Firecracker, a Cloud Hypervisor és a QEMU microvm géptípusa. A Firecracker eredetileg az AWS Lambda mögötti infrastruktúrához készült, de azóta önállóan is használják.

A microVM architekturális jellemzői:

- **KVM-alapú hardvervirtualizáció:** a CPU és a memória izolációját a hypervisor és a processzor virtualizációs kiterjesztései adják.
- **Minimális eszközök:** csak a szükséges blokk- és hálózati eszközök, általában VirtIO.
- **Nincs teljes BIOS/ACPI:** a vendég kernel egyszerű indítási protokollal töltődik be.
- **Kis memória-lábnyom:** tipikusan néhány MiB a vendég kernel és a futásidő számára, szemben a teljes VM több száz MiB-jával.

Az indulási idő szempontjából a microVM két fázisból áll: a virtuális gép létrehozása (a VMM elindítja a KVM-et, felépíti a memóriatérképet és az eszközöket), majd a vendég kernel és az alkalmazás betöltése. A második fázis gyorsítható unikernel vagy minimalista Linux kernel használatával.

Az alábbi ábra egy microVM rétegződését mutatja:

```
Alkalmazás
  |
Vendég kernel (minimalista, pl. Linux vagy unikernel)
  |
VMM (Firecracker vagy hasonló)
  |
KVM
  |
Hoszt kernel
  |
Hardver
```

A microVM erős biztonsági garanciát ad: a vendég és a hoszt közötti határ hardveresen támogatott, és a vendég kernel hibája nem terjed át a hosztra. Ugyanakkor a vendég kernel támadási felülete nagyobb, mint egy unikernelé, mert a Linux kernel általános célú szolgáltatásokat is tartalmaz.

26.4.4 4. WebAssembly és WASI: a nyelvfüggetlen, homokozott modul

A WebAssembly (WASM) eredetileg böngészőben futó, hordozható, alacsony szintű bytecode formátum. Az operációs rendszerekkel foglalkozó körökben azért lett érdekes, mert a WASM homokozó modellje lehetővé teszi idegen kódok biztonságos futtatását anélkül, hogy teljes operációs rendszert kellene indítani. A WebAssembly System Interface (WASI) szabványosítja a

modulok és a hoszt közötti rendszerhívás-szerű felületet: fájlrendszer, socket, idő és környezeti változók elérését.

A WASM izolációja nem hardveres virtualizáción, hanem nyelvi szintű homokozón alapul. A modul lineáris memóriája a hoszt által ellenőrzött tartomány, a függvényhívások típusozottak, és a modul nem férhet hozzá közvetlenül a hoszt memóriájához. Ez a modell rendkívül kis lábnyomot és nagyon gyors indulást tesz lehetővé, mert nincs szükség vendég kernelre vagy hypervisorra.

A WASM futtatókörnyezetek – mint a Wasmtime, a Wasmer vagy a Node.js beépített támogatása – általában egy hoszt folyamaton belül, több modul izolálásával futnak. A biztonsági garancia attól függ, hogy a WASM futásidő milyen gondosan ellenőrzi a modul memória-hozzáférését és a WASI hívásokat. A modell érettsége gyorsan fejlődik, de a hoszt erőforrások – például CPU-idő, hálózati sávszélesség vagy fájlleírók – korlátozása még mindig aktív kutatási terület.

Egy egyszerű WASM modul C-ből történő fordítására szolgáló parancsok szemléltetésként (a konkrét verziók ellenőrzendők):

```
# Szemléltető build parancs, nem éles rendszeren futtatandó
clang --target=wasm32-wasi -O2 -o handler.wasm handler.c
wasmtime handler.wasm
```

A futtatás során a WASM modul nem látja a hoszt fájlrendszerét, csak azokat a könyvtárakat, amelyeket a futásidő explicit módon megnyit. Ez a képesség a konténerekhez képest jóval finomabb izolációt tesz lehetővé.

26.4.5 5. Indulási idő, biztonság és költség

A három szempont – indulási idő, biztonság és költség – egymással verseng, ezért a döntési mátrixban együtt kell őket kezelni.

26.4.6 Indulási idő

Az indulási időt hidegindítási időnek nevezzük, ha nincs előre létrehozott, meleg példány. A melegindítás ezzel szemben egy már futó példány újabb kérésének kiszolgálási ideje. A hidegindítás különösen fontos a serverless platformokon, mert a terhelés hirtelen megnövekedése esetén sok új példány indulhat.

A tipikus nagyságrendek (az adott hardver és szoftververzió erősen befolyásolja, ezért mérni kell):

- **WASM modul:** 1–10 ms, mert nincs kernelindítás, csak a modul betöltése és az inicializálás.
- **Konténer:** 10–100 ms, ha a hoszt már futtatja a konténer futásidőt; a teljes konténerindítás ennél lassabb lehet.
- **MicroVM:** 50–200 ms, a KVM és a vendég kernel indítása miatt.
- **FaaS platform:** 50–500 ms, mert a platformnak fel kell építenie a futtatókörnyezetet, esetleg hálózati réteget kell konfigurálnia.

Fontos megkülönböztetni az **alkalmazás inicializálási idejét** is. Egy unikernel vagy WASM modul gyorsan elindul, de ha az alkalmazás nagy modellt tölt be vagy adatbázis-kapcsolatot épít fel, az uralhatja a teljes időt.

26.4.7 Biztonság

A biztonsági garanciák szintjei:

- **Konténer:** a kernel közös, ezért egy kernel-sebezhetőség az összes konténert érinti. A namespaces és cgroups erős logikai izolációt ad, de a határ nem hardveres.
- **WASM:** a homokozó nyelvi szintű, ami jól véd a memóriahibák ellen, de a hoszt erőforrás-kimerítése ellen további korlátok kellenek.
- **MicroVM:** hardveres virtualizáció, erős izoláció a vendég és a hoszt között; a vendég kernel támadási felülete a fő kockázat.
- **Unikernel:** nincs külön kernel, a támadási felület minimális, de ha az alkalmazás hibás, az egész unikernel veszélybe kerülhet. Az izoláció a hypervisoron vagy a hoszt rétegben valósul meg.

A biztonsági követelményeknél a tanúsítás és a formális verifikáció is szerepet kaphat. A seL4 mikrokernel és a rajta alapuló unikernel rendszerek formálisan bizonyított tulajdonságokkal rendelkeznek, ami a biztonságkritikus beágyazott rendszerekben fontos szempont.

26.4.8 Költség

A költség itt nem csak pénzügyi, hanem erőforrás- és üzemeltetési költséget is jelent:

- **Memória-lábnyom:** egy WASM modul néhány száz KiB-től néhány MiB-ig terjed, egy unikernel 1–10 MiB, egy microVM 5–50 MiB, míg egy teljes VM több száz MiB lehet.
- **CPU-terhelés:** a virtualizáció és a határátlépések többlet ciklust igényelnek. A WASM futásidő a legkönnyebb, a microVM a legnehezebb ebben a körben.
- **Üzemeltetési költség:** a serverless platformok elszámolása általában a végrehajtási időhöz és a lefoglalt memóriához kötődik. A hidegindítás gyakori esetén drágább, mert az indulás alatt is fizetni kell a futtatókörnyezetért.

26.4.9 6. Tipikus hibák és helyes megoldások

A könnyűsúlyú izoláció és az eseményvezérelt futtatás gyakorlatában számos hiba fordul elő. Ezek közül a legfontosabbak:

1. **A hidegindítás figyelmen kívül hagyása a tervezésben.** Ha egy FaaS függvény minden híváskor új adatbázis-kapcsolatot épít ki, és a platform hidegindítást hajt végre, a válaszidő drasztikusan megnő. Helyes megoldás: a kapcsolatot a függvény inicializálási szakaszában kell létrehozni, és meleg példányon újrahasznosítani; a hidegindítást pedig a platform beállításaival és a kód méretének csökkentésével kell minimalizálni.

2. **Állapot tárolása a függvény lokális változóiban.** A FaaS platformok nem garantálják, hogy ugyanaz a példány szolgálja ki a következő kérést. Ha a függvény helyi memóriában tartja a számlálót vagy a felhasználói munkamenetet, az elveszhet. Helyes megoldás: minden tartós állapotot külső, megbízható szolgáltatásban (objektumtárban, adatbázisban) kell tárolni.

3. **A biztonsági határ túlbecsülése.** Sokan azt gondolják, hogy a WASM homokozó vagy a konténer olyan erős izolációt ad, mint egy VM. Ez tévedés. A WASM futásidőnek is lehetnek hibái, és a konténerek közös kernelt használnak. Helyes megoldás: a fenyegetettségi modellhez kell választani az izolációs szintet; biztonságkritikus esetben hardveres virtualizációt vagy formálisan verifikált kernelt kell használni.

4. **Az erőforrás-korlátok hiánya.** Egy WASM modul vagy FaaS függvény végtelen ciklusba kerülhet, és kimerítheti a CPU-időt. A platformnak időkorlátot, memóriakorlátot és párhuzamossági korlátot kell beállítania. Helyes megoldás: a deploy konfigurációban explicit timeout és memória limit megadása.

5. **A megfigyelhetőség elhanyagolása.** A serverless és a mikroizolált környezetekben a hagyományos logok és trace-ek szétszóródnak. Helyes megoldás: strukturált naplózás, elosztott tracing és központi metrika-gyűjtés beépítése a függvényekbe.

6. **Az indulási és leállási tevékenységek nem idempotensek.** Ha egy függvény egy eseményre többször is lefut, például újrapróbálkozás miatt, és a művelet nem idempotens, adatduplikáció vagy inkonzisztencia keletkezhet. Helyes megoldás: idempotens műveletek tervezése, például egyedi azonosítóval ellátott kérések.

26.4.10 7. Döntési mátrix a technológiák összehasonlítására

A döntési mátrix segít abban, hogy egy adott munkaterheléshez a megfelelő izolációs és futtatási modellt válasszuk. Az alábbi táblázat irányadó; a konkrét értékeket mindig méréssel kell alátámasztani.

Szempont	Konténer	MicroVM	Unikernel	WASM modul	FaaS platform
Indulási idő	Jó (10–100 ms)	Közepes (50–200 ms)	Nagyon jó (5–50 ms)	Kiváló (1–10 ms)	Változó (50–500 ms)
Memórialábnyom	Közepes (10–50 MiB)	Kicsi (5–50 MiB)	Nagyon kicsi (1–10 MiB)	Minimális (0,1–5 MiB)	Változó, platformfüggő
Biztonsági izoláció	Gyenge–közepes	Erős	Erős, ha hypervisor alatt fut	Jó–erős (futásidőtől függ)	Platformfüggő
Fejlesztői élmény	Nagyon jó	Jó	Közepes	Jó	Kiváló
Ökoszisztéma	Érett	Fejlődő	Szűkebb	Gyorsan növekvő	Érett
Állapotkezelés	Jól támogatott	Jól támogatott	Korlátozott	Korlátozott	Külső szolgáltatás kell
Költség (erőforrás)	Alacsony	Közepes	Alacsony	Nagyon alacsony	Használatarányos

A mátrix alapján:

- **Rövid életű, nagy számú, kis feladat** esetén a WASM modul vagy a FaaS platform jó választás.
- **Több bérlős, erős izolációt igénylő szolgáltatás** esetén a microVM előnyös.
- **Beágyazott, biztonságkritikus vagy hálózati funkció** esetén az unikernel minimalista felülete miatt vonzó.
- **Általános célú mikroszolgáltatás** esetén a konténer érett ökoszisztémája és jó fejlesztői élménye miatt továbbra is vezető.

26.4.11 8. Labor: serverless és unikernel döntési mátrix

A labor célja, hogy egy emulált környezetben, reprodukálható módon összehasonlítsa a könnyűsúlyú futtatási modelleket. A feladat nem éles rendszeren, hanem QEMU vagy Firecracker alapú microVM-ben, WASM futtatókörnyezetben és egy FaaS-emulátorban végzett mérésekre épül. Az alábbi lépések szemléltetik a folyamatot; a konkrét parancsok és verziók a labor elvégzésekor ellenőrizendők.

26.4.12 Kiinduló állapot és konfiguráció

- **Hoszt:** x86-64, legalább 4 mag, 8 GiB RAM, Linux 6.x kernel KVM-támogatással.
- **Emulált környezet:** QEMU 8.x microvm, Firecracker 1.x, Wasmtime 15.x (a verziók ellenőrzendők).
- **Mért alkalmazás:** egy egyszerű HTTP-választ adó handler, amely a `name` paraméterre `Hello, name!` üzenetet ad vissza.
- **Mérési módszer:** minden indítást 30-szor megismételünk, a hidegindítást úgy biztosítjuk, hogy a környezetet minden mérés előtt leállítjuk és töröljük a gyorsítótárat.

26.4.13 A mérési lépések

1. **WASM modul építése és indítása.** Fordítsd le a handler kódot WASI célra, majd mérd a `wasmtime` indítási idejét a parancs kiadásától a válasz megérkezéséig.

```
# Szemléltető parancs, nem éles rendszeren futtatandó
clang --target=wasm32-wasi -O2 -o handler.wasm handler.c
/usr/bin/time -f "%e" wasmtime handler.wasm --name=teszt
```

2. **MicroVM indítása Firecrackerrel.** Hozz létre egy minimális rootfs-t, indítsd el a Firecracker VMM-et, és mérd a teljes indítási időt a HTTP-válaszig.

```
# Szemléltető vázlat, a pontos paraméterek a laborcsomagban találhatók
firecracker --config-file microvm_config.json --no-api
```

3. **FaaS-emulátor futtatása.** Használj egy helyi FaaS keretrendszert (például OpenFaaS vagy Knative, de a laborban csak helyi, eldobható környezetben), telepítsd a függvényt, majd küldj egy HTTP-kérést a hidegindítási idő mérésére.

4. **Adatok rögzítése.** Minden méréshez rögzítsd a hardver- és szoftververziókat, az órajelet, a memóriakonfigurációt, a mérési időbélyeget és a mért értékeket. A napló formátuma lehet CSV vagy parquet.

26.4.14 Elvárt eredmény és a várt értékek elkülönítése

A laborban a **várt eredmény** azt jelenti, hogy a nagyságrendek összhangban vannak a fejezetben bemutatott tartományokkal, és a döntési mátrixot a saját mérésekkel tudod alátámasztani. A **mért eredmény** a ténylegesen kapott idő, memóriahasználat és CPU-terhelés. A kettőt a jegyzőkönyvben külön oszlopban kell feltüntetni.

Példa táblázat a jegyzőkönyvhöz:

Futtatási modell	Várt hidegindítás (ms)	Mért hidegindítás (ms)	Várt memória (MiB)	Mért memória (MiB)
WASM	1–10	3,2	0,5–2	1,1
MicroVM	50–200	104	5–30	12
FaaS emulátor	50–500	180	20–100	45

A mért értékek erősen függnek a hoszt hardverétől és a beállításoktól, ezért a labor értékelésénél a mérési módszer reprodukálhatósága és a várt/mért értékek megfelelő szétválasztása a fő szempont.

26.4.15 Döntési mátrix elkészítése

A mérések után töltsd ki a 7. szakaszban látható mátrixot a saját értékeiddel. Add meg a választott technológiát a következő három forgatókönyvre:

- **A eset:** nagy forgalmú, rövid válaszidejű web API, ahol a hidegindítás gyakran előfordul.
- **B eset:** több bérlős, érzékeny adatot kezelő szolgáltatás, ahol a biztonsági izoláció elsődleges.
- **C eset:** beágyazott hálózati eszköz, ahol a memória-lábnyom és a determinisztikus indulás kritikus.

Minden választást indokolj a mátrix és a mért értékek alapján.

26.4.16 Visszaállítás

A labor után minden emulált környezetet le kell állítani, a létrehozott rootfs-eket és konfigurációkat törölni, és a hoszt rendszer állapotát ellenőrizni kell. A hoszt valódi lemezét, bootloaderét és rendszerpartícióját tilos módosítani.

26.4.17 Összefoglalás

A serverless és az unikernel megközelítések a könnyűsúlyú izoláció és az eseményvezérelt futtatás révén csökkentik az erőforrás-lábnyomot és az indulási időt, miközben különböző biztonsági garanciákat nyújtanak. A FaaS platformok a fejlesztői élményt helyezik előtérbe, a microVM-ek a hardveres izolációt, a WASM modulok a minimális lábnyomot, az unikernelek pedig a támadási felület csökkentését. A megfelelő választáshoz mérni kell az indulási időt, a memória- és CPU-költséget, és egy döntési mátrixban kell összevetni a technológiákat. A gyakori hibák – mint a hidegindítás figyelmen kívül hagyása, az állapot helytelen tárolása vagy a biztonsági határ túlbecsülése – elkerülhetők a fejezetben bemutatott tervezési elvekkel.

Önellenőrző kérdés: Miért nem elegendő a WASM homokozó önmagában egy több bérlős, biztonságkritikus szolgáltatás izolálásához, és milyen kiegészítő mechanizmusokra lehet szükség?

VII. RÉSZ – Tervezés, implementáció és záróprojekt

27 Egyszerű OS tervezése

27.1 Követelmények, célok és architektúra

Egy operációs rendszer tervezése ritkán kezdődik kódolással. A tapasztalt csapatok először rögzítik, mit építenek, miért, milyen hardveren, és milyen határokkal. Ez a rögzítés azért kritikus, mert egy kernelprojekt hónapokig tart, és a közben felmerülő ötletek könnyen szétfeszítik az eredeti célt. Ebben a fejezetben egy saját, tanulási célú kernel példáján mutatjuk be, hogyan kell elkészíteni az OS-projekt chartert és az architektúravázlatot – vagyis azt a két dokumentumot, amely a projekt iránytűjeként szolgál.

27.1.1 1. A scope rögzítése

Az operációs rendszer hatóköre (scope) három fő részből áll: funkcionális követelményekből, minőségi követelményekből és explicit nem célokból. A három elem együtt határozza meg, hogy a projekt mit fog tudni, milyen szinten, és mit nem fog megvalósítani.

A **funkcionális követelmények** azt írják le, hogy a rendszernek milyen szolgáltatásokat kell nyújtania. Egy oktatási célú, minimális kernel esetében ez jellemzően a következőket foglalja magában:

- a kernel betöltése és inicializálása egy emulált gépen;
- soros konzol kimenet (UART) és bemenet kezelése;
- fizikai memória kezelése (lapok lefoglalása és felszabadítása);
- virtuális memória beállítása legalább a kernel számára;
- legalább egy felhasználói folyamat betöltése és futtatása;
- preemptív ütemező, amely a folyamatokat váltogatja;
- rendszerhívási interfész néhány hívással (például `write`, `read`, `exit`);
- egyszerű fájlrendszer vagy initrd olvasás.

A **minőségi követelmények** a „mennyire jó” kérdésre válaszolnak, és mérhetőnek kell lenniük, különben nem ellenőrizhetők. Példák:

- a kernel bootolása QEMU alatt legfeljebb 2 másodpercet vesz igénybe;
- a fizikai memória allokátor nem szivárog: 1000 lefoglalás-felszabadítás ciklus után a szabad lapok száma megegyezik a kezdeti értékkel;
- a rendszerhívás átlagos késleltetése 100 µs alatt van a megadott emulált környezetben;
- a kernel kódja és adatai elférnek 1 MiB alatt.

A **nem célok** listája talán a legfontosabb része a charternek, mert ez védi meg a projektet a folyamatos bővüléstől, azaz a scope creeptől. Egy kezdő kernel esetében tipikus nem célok:

- nem támogatunk többfelhasználós jogosultságkezelést;
- nem valósítunk meg hálózati protokollstacket;
- nem kezelünk USB- vagy PCI-eszközöket a legszükségesebbek kivételével;
- nem biztosítunk grafikus felületet;

- nem cél a POSIX-szabvány teljes körű megvalósítása;
- nem garantálunk biztonsági izolációt rosszindulatú folyamatok ellen.

A nem célok rögzítése nem gyengeség, hanem tervezési fegyelem. Ha egy új ötlet felmerül, a charterhez lehet visszatérni, és eldönteni, hogy az a hatókörbe tartozik-e, vagy a jövőbeli bővítések listájára kerül. A charter dokumentum lehet egyszerű szöveges fájl, de érdemes verziókövetni, és minden jelentős döntésnél frissíteni.

27.1.2 2. Célplatform és eszközlánc

A célplatform kiválasztása határozza meg, hogy milyen hardver- és firmware-környezetben fut a kernel. A kezdő operációsrendszer-projektek esetében szinte mindig emulált környezetet érdemes választani valós hardver helyett. Ennek okai:

- **Determinizmus:** ugyanaz a konfiguráció minden futtatásnál ugyanazt az eredményt adja, ami elengedhetetlen a hibakereséshez.
- **Biztonság:** egy rosszul megírt kernel nem károsíthatja a fejlesztőgép adatait.
- **Költség és hozzáférhetőség:** nincs szükség speciális hardverre, a labor bárhol reprodukálható.
- **Visszaállítás:** pillanatképek és mentési pontok könnyen kezelhetők, ami az oktatási környezetben alapkövetelmény.

A leggyakoribb választás a QEMU virtuális gép. A QEMU lehetővé teszi a hardver részletes konfigurálását parancssori kapcsolókkal, és támogatja a GDB-alapú távoli hibakeresést. A kernel célarchitektúrájaként két jelölt közül választhatunk: x86-64 vagy RISC-V 64. Mindkettő jól dokumentált, de a RISC-V privilegizált specifikációja egyszerűbb, és a QEMU `virt` gépe kifejezetten alkalmas oktatási célra. A továbbiakban egy **RV64GC** architektúrájú, **128 MiB** memóriájú, **OpenSBI** firmware-rel induló QEMU `virt` gépet feltételezünk. Az eszközök közül a soros konzol (UART) az alapvető kommunikációs csatorna, az időzítő pedig az SBI-n keresztül érhető el. A pontos QEMU-verzió és az OpenSBI verzió a telepített disztribúciótól függ, ezért a laborban a használt verziókat rögzíteni kell, és ellenőrizni kell a kompatibilitást.

Az eszközlánc (toolchain) szintén a charter része. A RISC-V célra egy keresztfordító (cross-compiler) szükséges, például a `riscv64-unknown-elf-gcc` vagy `riscv64-linux-gnu-gcc`. A freestanding kernelt `-ffreestanding` és `-mcmodel=medany` kapcsolókkal fordítjuk. A linkeléshez saját linker scriptet használunk, amely a kernel szegmenseit a megfelelő virtuális címekre helyezi. Az építést `make` vezérli, a futtatást és a hibakeresést pedig QEMU és GDB végzi. A binutils, a GCC és a QEMU pontos verzióját a labor jegyzőkönyvben rögzíteni kell, mert a verzióeltérések megváltoztathatják az időzítési méréseket.

A célplatform specifikációban a **nem célok** is megjelennek: nem támogatunk UEFI-runtime szolgáltatásokat, nem kezelünk többmagos (SMP) konfigurációt az első verzióban, és nem használunk hardveres virtualizációs kiterjesztéseket. Ezek később hozzáadhatók, de az első iterációban zavarnák a tanulási ívet.

27.1.3 3. Monolitikus vagy mikrokernel?

Az architektúraválasztás egyik első, alapvető döntése a kernel szerkezetére vonatkozik. A korábbi fejezetekben már bemutattuk a monolitikus, a mikrokernel és a hibrid felépítést; itt most a konkrét döntést hozzuk meg a saját projektünkre.

A **monolitikus kernel** egyetlen címterében fut, a komponensek közvetlen függvényhívással kommunikálnak. Előnye az egyszerűség: a kontextusváltás, az adatátadás és a hibakeresés

közvetlen, nincs szükség bonyolult IPC mechanizmusra. Hátránya, hogy egy hibás illesztőprogram az egész kernelt lefagyaszthatja, és a modularitás csak fegyelmezett kódolással biztosítható.

A **mikrokernel** ezzel szemben csak a legszükségesebb szolgáltatásokat tartja kernel módban – ütemezés, memóriakezelés, IPC –, a többi szolgáltatást (fájlrendszer, hálózat, eszközillesztők) felhasználói módú szerverfolyamatok valósítják meg. Ez jobb izolációt és modularitást ad, de jelentős többletköltséggel jár: a komponensek közötti üzenetváltás IPC-n keresztül történik, ami tervezési és teljesítménybeli bonyolultságot visz a rendszerbe.

Egy kezdő, oktatási célú kernel esetében a **monolitikus, de moduláris felépítés** a racionális választás. Az indoklás:

1. **Kevesebb mozgó alkatrész.** Nincs szükség IPC keretrendszerre és felhasználói módú szerverekre ahhoz, hogy az első folyamat elinduljon. Ez lehetővé teszi, hogy a figyelem a kernel alapmechanizmusaira irányuljon.
2. **Könnyebb hibakeresés.** Egyetlen címterű kernel esetén a GDB-vel egyszerűbb a memória és a verem vizsgálata, mint egy mikrokernel üzenetváltásainak nyomon követése.
3. **Kisebb teljesítmény-overhead.** Az oktatási célú méréseknél a közvetlen függvényhívások időzítése könnyebben értelmezhető, mint az IPC-késleltetés.
4. **A moduláris tervezés megőrzi a későbbi bővítés lehetőségét.** Ha a komponensek interfészeit jól definiáljuk, később egyes részeket kiemelhetünk felhasználói módba, vagy akár mikro-hibrid irányba fejleszthetünk.

A választás nem azt jelenti, hogy a kód egy nagy, strukturálatlan fájl lesz. A moduláris felépítés azt jelenti, hogy a boot, a memóriakezelő, az ütemező és a rendszerhívási réteg külön fordítási egységekben, jól dokumentált interfészekkel készül. A kernel belső API-ja a komponensek közötti függvényhívásokat definiálja, és a hívó fél tulajdonjogát minden átadott erőforrásnál egyértelműen rögzíti.

27.1.4 4. Architektúravázlat és interfészek

Az architektúravázlat a kernel rétegeit és a köztük lévő interfészeket mutatja be. Egy lehetséges rétegződés a következő:

1. **Hardverabsztrakciós réteg (HAL):** a hardver-specifikus kódot tartalmazza. Ide tartozik a boot utáni CPU-inicializálás, a megszakításvezérlő, az időzítő és az UART kezelése. A felsőbb rétegek csak a HAL által definiált függvényeken keresztül érik el a hardvert.
2. **Kernelmag (Core):** a fő kernelmechanizmusok. Ez a réteg tartalmazza a fizikai memória allokátort (PMM), a virtuális memória kezelőt (VMM), a folyamat- és szálkezelőt, az ütemezőt és a rendszerhívási diszpécsert. A komponensek egymással közvetlen függvényhívással kommunikálnak, de a hívási gráfot expliciten dokumentálni kell.
3. **Eszközillesztők (Drivers):** a konkrét eszközök meghajtói. Az első verzióban ez a soros konzol és egy egyszerű blokk-eszköz (például virtio-blk vagy initrd). A meghajtók regisztrálják magukat az eszközkezelőnél, és a kernelmag a regisztrált interfészen keresztül éri el őket.
4. **Felhasználói tér (User Space):** minimális mértékben van jelen. Egy egyszerű `init` folyamat indít egy shellt, amely a rendszerhívásokat használja. A rendszerhívási ABI ebben a rétegben definiálódik.

A rétegek közötti interfészeket két csoportba soroljuk:

- **Belső API:** a kernel komponensei közötti függvényhívások. Például a VMM a PMM-től lapot kér, az ütemező a VMM-től címteret, a rendszerhívási réteg az ütemezőtől folyamatot. Minden ilyen interfésznél rögzíteni kell az átadott mutatók tulajdonjogát, az érvényes kontextust (megszakítási vagy folyamat kontextus), és a hibakódokat.
- **Rendszerhívási ABI:** a kernel és a felhasználói tér közötti határ. A hívások regiszterekben kapják a paramétereket, és egy regiszterben adják vissza az eredményt vagy a hibakódot. A hibakódot a felhasználói oldalon `errno`-ként kell értelmezni. Az ABI-t verziószámmal kell ellátni, hogy a későbbi bővítések ne törjék a kompatibilitást.

Az architektúravázlat nem csak rétegdiagram, hanem tartalmazza a **komponenslistát** is. Például:

- `boot.S`: a boot belépési pont és a korai CPU-inicializálás.
- `main.c`: a kernel fő inicializáló függvénye.
- `pmm.c`, `pmm.h`: fizikai memória allokátor.
- `vmm.c`, `vmm.h`: virtuális memória kezelés és lapozás.
- `sched.c`, `sched.h`: ütemező és folyamatváltás.
- `syscall.c`, `syscall.h`: rendszerhívási diszpécser.
- `uart.c`, `virtio_blk.c`: eszközillesztők.
- `linker.ld`: linker script.

Ezt a listát a charter mellékleteként érdemes rögzíteni, és az implementáció során frissíteni.

27.1.5 5. Példa: EduOS projekt charter

Az alábbiakban egy kitöltött példát mutatunk be egy kitalált, oktatási célú kernelre. A példa szemlélteti, hogyan néz ki egy használható charter dokumentum. A projekt neve legyen **EduOS**.

Projekt neve: EduOS

Cél: Minimális, monolitikus, moduláris kernel, amely QEMU RISC-V 64 virtuális gépen bootol, inicializálja a fizikai és virtuális memóriát, elindít egy felhasználói folyamatot, és parancssori shellt biztosít alapvető parancsokkal (`help`, `echo`, `meminfo`, `shutdown`).

Célplatform: - QEMU `virt` gép, RV64GC CPU, 128 MiB RAM - OpenSBI firmware - Soros konzol UART-on (16550-kompatibilis) - Időzítő SBI ecall segítségével - Virtio-blk eszköz a fájlrendszerhez (vagy initrd a memóriában)

Eszközlánc: - `riscv64-unknown-elf-gcc` (a pontos verziót a laborban rögzítjük) - GNU ld, binutils - GNU make - QEMU (a pontos verziót a laborban rögzítjük) - GDB (távoli hibakereséshez)

Funkcionális követelmények: - Bootolás OpenSBI-ből, saját kernel belépési pont - Fizikai lapallokátor buddy vagy free list alapon - Lapozás RV64 SV39 módban - Folyamatok és preemptív round-robin ütemező - Rendszerhívások: `write`, `read`, `exit`, `getpid` - Egyszerű shell az init folyamatban

Nem célok: - Hálózati stack - Többmagos támogatás (SMP) - Teljes POSIX - Grafikus felület - Biztonsági izoláció rosszindulatú folyamatokkal szemben - Energiagazdálkodás

Mérföldkövek: 1. Boot és UART kimenet 2. Fizikai memória allokátor 3. Virtuális memória és lapozás 4. Folyamatok és ütemező 5. Rendszerhívások és userland 6. Shell és alapszolgáltatások

Tesztelési terv: Minden mérföldkőhöz tartozik egy unit teszt és egy integrációs teszt QEMU alatt. A memória allokátornál szivárgásteszt, az ütemezőnél determinisztikus váltási teszt, a rendszerhívásoknál hibakód-teszt.

Kockázatok: - A lapozási hibák nehezen debuggolhatók; enyhítés: korai GDB integráció, naplózás. - Az OpenSBI verzióváltásai befolyásolhatják a boot folyamatot; enyhítés: a laborban a pontos verzió rögzítése.

Ez a példa jól mutatja, hogy a charter nem hosszú dokumentum, de minden lényeges döntést tartalmaz. A projekt során ezt a dokumentumot kell frissíteni, amikor a scope változik.

27.1.6 6. Tipikus hibák és helyes megoldások

Az OS-projekt tervezésénél az alábbi hibák a leggyakoribbak:

1. **Scope creep (hatókör-kúszás).** A fejlesztő az első héten hálózatot, USB-t és grafikus felületet akar. Ez a figyelem szétforgácsolódásához vezet. *Megoldás:* A nem célok listájának rendszeres felülvizsgálata, és minden új ötletnél a kérdés: „Ez a jelenlegi mérföldkőhöz szükséges?" Ha nem, kerüljön a jövőbeli bővítések listájára.
2. **Nem mérhető minőségi követelmények.** A „gyors boot" nem ellenőrizhető. *Megoldás:* Konkrét, mérhető értékeket kell megadni (például boot idő 2 s QEMU alatt), és a mérési módszert is rögzíteni kell.
3. **A monolit/mikrokernel döntés halogatása.** Ha a projekt elején nem döntünk, az interfészek tisztázatlanok maradnak, és a kód hamar összezavarodik. *Megoldás:* A döntést a charterben rögzíteni kell, indoklással együtt. A döntés nem végleges, de kiindulási alapnak megfelel.
4. **A célplatform nem elég specifikus.** Ha csak annyit írunk, hogy „RISC-V", akkor a QEMU verziója, a memória mérete és a firmware eltérései reprodukálhatatlan eredményeket adnak. *Megoldás:* A platform konfigurációját pontosan rögzíteni kell, lehetőleg a QEMU parancssorral együtt.
5. **Hiányzó tulajdonjog-szabályok a belső interfészeken.** A kernelben gyakori hiba, hogy egy komponens felszabadít egy mutatót, amelyet egy másik komponens még használ. *Megoldás:* Minden belső API-nál dokumentálni kell, hogy a hívó vagy a hívott fél birtokolja-e az erőforrást, és ki felel a felszabadításért.
6. **A verifikációs terv elhanyagolása.** A kód megírása után derül ki, hogy nincsenek tesztek, és a hibákat nehéz reprodukálni. *Megoldás:* A charternek már a kezdetektől tartalmaznia kell a tesztelési stratégiát, beleértve a fuzzing és a statikus elemzés lehetőségét is.

27.1.7 7. Labor: OS-projekt charter és architektúravázlat

Ez a labor a fejezetben bemutatott fogalmak gyakorlati alkalmazását kéri. A cél egy saját OS-projekt charter és architektúravázlat elkészítése, reprodukálható jegyzetfüzetben vagy emulált környezetben.

Kiinduló állapot: Egy üres git repository, amelyben a fejlesztőkörnyezet (cross-compiler, QEMU, GDB, make) telepítve van. Az eszközök verzióit rögzíteni kell.

Feladat: 1. Készítsd el a `CHARTER.md` dokumentumot a fenti sablon alapján. Add meg a projekt nevét, célját, célplatformját, eszközláncát, funkcionális és minőségi követelményeit, nem céljait, mérföldköveit, tesztelési tervét és kockázatait. 2. Készíts egy architektúravázlatot rétegdiagram formájában (például `ARCHITECTURE.md` fájlban, ASCII diagrammal vagy szöveges leírással). A vázlat tartalmazza a rétegeket, a főbb komponenseket és a köztük lévő interfészek nevét. 3.

Rögzítsd a build parancsot (például `make`), a futtatási parancsot (QEMU indítás), és a várható eredményt (például a kernel kiírja a boot üzenetet). 4. Futtasd a QEMU-t a `-nographic` kapcsolóval, és mérd meg a boot időt. A mérési módszert dokumentálni kell: például a QEMU indításától a kernel első UART kimenetéig eltelt időt. A mért értéket és a várt értéket külön kell feltüntetni. 5. Készíts egy mérési naplót (`MEASUREMENTS.md`), amelyben a mért boot idő, a kernel memória-lábnyoma (a linker script által generált szekcióméretek alapján), és az esetleges eltérések magyarázata szerepel. 6. Töltsd fel a dokumentumokat a git repóba, és készíts egy commit üzenetet, amely összefoglalja a projekt indítását.

Elvárt eredmény: A repó tartalmazza a `CHARTER.md`, `ARCHITECTURE.md`, `MEASUREMENTS.md` fájlokat, valamint a build és futtatási parancsok leírását. A mérési naplóban a mért és a várt értékek egyértelműen elkülönülnek.

Visszaállítás: A labor nem igényel valós hardver módosítást. Az emulált környezet bármikor újraindítható, és a git history visszaállítható. A QEMU lemezképek és a kernel binárisok a projekt könyvtárában maradnak.

Megjegyzés a méréshez: A boot idő mérése függ a gazdagép terhelésétől és a QEMU verziójától, ezért a mérést legalább háromszor érdemes elvégezni, és a mediánt vagy az átlagot feltüntetni. A várt érték csak akkor érvényes, ha a környezet specifikációja (CPU modell, QEMU verzió, memória méret) megegyezik a laborban rögzítettel.

27.1.8 8. Önellenőrző kérdés

Kérdés: Egy fejlesztő a saját kerneljéhez a következő követelményt írja: „A rendszer gyors legyen, és támogassa a modern hardvereket.” Miért nem megfelelő ez a követelmény a charterben, és hogyan javítható?

Válasz: A „gyors” nem mérhető, a „modern hardverek” pedig túl tág, és a hatókörbe nem tartozó eszközöket is magában foglalhat. Helyes megfogalmazás például: „A boot idő QEMU 8.0, RV64GC, 128 MiB konfiguráción legfeljebb 2 másodperc; a kernel csak a QEMU `virt` gép UART és virtio-blk eszközeit kezeli.” Ez mérhető, specifikus, és a nem célokat is kijelöli.

27.2 Bootfolyamat és kernel inicializálás

A bootfolyamat első pillanataiban a gép még nem operációs rendszert futtat, hanem egy többlépcsős, hardverközeli inicializálási láncot. Ez a szakasz határozza meg, hogy a kernel egyáltalán képes-e a CPU-t, a memóriát és az alapvető eszközöket olyan állapotba hozni, amelyben a későbbi absztrakciók – folyamatok, ütemező, fájlrendszer – egyáltalán értelmezhetők. A jelen fejezet ezt az utat járja végig a bootloadertől a kernel fő ciklusáig, és bevezeti a minimálisan szükséges hardverközeli elemeket: a boot interfészt, a memóriatérképet, a GDT/IDT párost, a korai konzolt és a `panic` mechanizmust.

A fejezet célja nem egyetlen konkrét bootloader vagy architektúra teljes leírása. Ehelyett olyan ellenőrizhető mintát adunk, amely emulált környezetben reprodukálható, és amelynek minden lépése magyarázható. A kódok szemléltető jellegűek; az éles gépen való futtatás nem cél, és a labor kizárólag QEMU-ban vagy hasonló, eldobható virtuális gépben végezhető el.

27.2.1 A boot útja: firmware, bootloader, kernel

A bootfolyamat általános értelemben három nagy szakaszra bontható, bár a határok architektúránként eltérnek:

1. **Firmware-szakasz:** a hardver bekapcsolás után futtatja a platform firmware-ét. x86-on ez BIOS vagy UEFI, RISC-V platformokon gyakran egy M-mode-ban futó firmware (például OpenSBI), amely betölti a következő fázist. A firmware feladata az alapvető CPU- és memória-inicializálás, az eszközök felfedezése, valamint a bootolható eszköz kiválasztása.
2. **Bootloader-szakasz:** a bootloader (GRUB, U-Boot, Limine, vagy egy saját, minimális második fázis) betölti a kernel image-et a memóriába, átadja a szükséges hardverinformációkat, majd átadja a vezérlést a kernel belépési pontjára. A bootloader gyakran saját meghajtókkal rendelkezik, és képes fájlrendszerből, hálózatról vagy nyers blokkról olvasni.
3. **Kernel-szakasz:** a kernel belépési pontjától kezdve a rendszer saját maga inicializálja a hardvert, a memóriát és a szoftveres absztrakciókat, végül belép a fő ciklusba.

Ezt a láncot a következő folyamat mutatja:

```
[firmware]
    |
    v
[bootloader]  --> kernel image betöltése, paraméterek átadása
    |
    v
[kernel _start]  --> stack, BSS, CPU-alapállapot
    |
    v
[early console]  --> soros port / framebuffer
    |
    v
[panic infrastruktúra]
    |
    v
[memóriatérkép feldolgozása]
    |
    v
[IDT/GDT, időzítő, interrupt-vezérlő]
    |
    v
[kernel fő ciklus / idle]
```

A sorrend tudatosan úgy van felépítve, hogy a lehető legkorábban legyen megfigyelhető kimenetünk. Amíg nincs konzol, a kernel néma; ha bármilyen hiba történik, nem tudjuk megkülönböztetni a sikeres futást a korai lefagyástól. Ezért a korai konzolt a lehető leghamarabb inicializáljuk, még a bonyolultabb memóriakezelés előtt.

27.2.2 Kernel image és belépési állapot

A bootloader egy kernel image-et tölt be, amely jellemzően ELF formátumú. Az ELF szakaszai közül a kernel inicializálása szempontjából négy fontos:

- `.text`: végrehajtható kód.
- `.rodata`: csak olvasható adatok, például konstans stringek.
- `.data`: inicializált globális és statikus változók.
- `.bss`: nullázatlan globális változók; ezek a memóriában helyet foglalnak, de a képfájlban nem tartalmaznak adatot, ezért a kernelnek futási időben nulláznia kell őket.

A kernel belépési pontja általában egy assembly rutin, gyakran `_start` néven. A bootloader ezen a címen adja át a vezérlést, jellemzően úgy, hogy a CPU még olyan állapotban van, amit a boot-specifikáció meghatároz. x86-64 Multiboot2 esetén például a bootloader 32 bites vagy 64 bites védett módban adhatja át a vezérlést, letiltott megszakításokkal. RISC-V SBI esetén a kernel S-mode-ban indul, és a firmware SBI hívásokat biztosít.

A belépési pontnak az első dolga a minimális futási környezet megteremtése:

1. **Stack beállítása:** az assembly kód egy kisméretű, statikus boot stacket használ, például 16 KiB méretben. A stack pointer beállítása nélkül semmilyen C függvény nem hívható.
2. **A CPU flags mentése és normalizálása:** megszakítások letiltása, a direction flag törlése x86-on, valamint a bootloader által átadott regiszterek mentése.
3. **BSS nullázása:** a `.bss_start` és `.bss_end` közötti területet a kernel nullázza, hogy a C statikus változók a szabvány által elvárt kezdeti értéket kapják.
4. **Átadott adatok mentése:** a bootloader paramétereket, például a Multiboot2 info struktúra pointerét vagy a device tree címét, egy C változóba kell menteni, mielőtt a regisztereket felülírjuk.

Egy minimális x86-64 belépési kód vázlata:

```
    .section .boot.text, "ax"
    .global _start
_start:
    cli                         # megszakítások tiltása
    cld                         # DF = 0
    movq %rbx, boot_info_ptr    # Multiboot2 info mentése
    leaq boot_stack_top(%rip), %rsp

    # BSS nullázás
    leaq __bss_start(%rip), %rdi
    leaq __bss_end(%rip), %rcx
    subq %rdi, %rcx
    shrq $3, %rcx               # 8 byte-os lépések
    xorl %eax, %eax
    rep stosq

    call kmain

.halt:
    hlt
    jmp .halt

    .section .bss
    .align 16
boot_stack:
    .skip 16384                 # 16 KiB boot stack
boot_stack_top:

    .section .data
    .global boot_info_ptr
boot_info_ptr:
    .quad 0
```

A fenti kódban a `call kmain` előtt már van stack és nullázott BSS. A `kmain` C függvény ezután futhat. A `boot_info_ptr` egy globális változó, amelyet a bootloader által átadott paraméter tárol.

27.2.3 Bootloader interfész és memóriatérkép

A bootloader nem csupán betölti a kernelt, hanem leírást is ad arról, hogy milyen hardver áll rendelkezésre. A kernel ezt az információt használja arra, hogy ne kelljen minden eszközt a nulláról felderítenie.

A leggyakoribb átadott adatok:

- **Memóriatérkép:** mely fizikai címtartományok használhatók RAM-ként, melyek fenntartottak, és hol vannak eszközregiszterek.
- **Kernel parancssor:** indítási paraméterek, például `console=uart0` vagy `debug=1`.
- **Bootloader által lefoglalt területek:** például a kernel image, a ramdisk, a device tree vagy az ACPI táblák.
- **Kijelző- és sorosport-információ:** framebuffer cím, felbontás, UART portcím.

Egy lehetséges memóriatérkép x86-64 QEMU környezetben a következőképpen nézhet ki:

```
Fizikai címtartomány        Méret        Típus
0x00000000 - 0x0009FFFF    640 KiB      RAM (low memory)
0x000A0000 - 0x000BFFFF    128 KiB      VGA memória / ROM
0x000C0000 - 0x000FFFFF    256 KiB      BIOS ROM / fenntartott
0x00100000 - 0x03FFFFFF    63 MiB       RAM (kernel és heap)
0x04000000 - 0x043FFFFF    4 MiB        fenntartott (eszköz)
0xFE000000 - 0xFFFFFFFF    32 MiB       APIC / fenntartott
```

A virtuális címtartomány ebben a korai szakaszban még általában megegyezik a fizikaival, vagy identitás-leképezést használunk. A későbbi fejezetben (Memóriakezelés és ütemező tervezése) térünk rá arra, hogy a kernel saját lapozási struktúrákat építsen. A laborban a fejezet végén csak az identitás-leképezésű szakaszt használjuk.

A Multiboot2 például egy memóriatérkép-tömböt ad át, amelynek elemei egy-egy címtartományt írnak le:

```
typedef struct {
    uint64_t base;
    uint64_t length;
    uint32_t type;
    uint32_t reserved;
} mb2_mmap_entry_t;
```

A kernel ezt bejárja, és egy belső, könnyebben kezelhető listát épít. A memóriatérkép feldolgozása nélkül a kernel nem tudja, mely területekre írhat, és melyeket kell érintetlenül hagynia. A tipikus hiba, hogy a kernel a BIOS ROM vagy az eszközregiszterek fölé ír, ami azonnali lefagyást vagy furcsa mellékhatást okoz.

27.2.4 GDT és IDT: a CPU védelmi és megszakítási alapjai

A kernel inicializálásának következő lépése a CPU védelmi és kivételkezelési mechanizmusainak beállítása. A pontos regiszterek és struktúrák architektúránként eltérnek, de a szerep azonos: a kernel meghatározza, milyen jogosultsági szinteken futhat kód, és mi történik megszakítás vagy kivétel esetén.

27.2.5 GDT (Global Descriptor Table) x86-on

Az x86-os védett módban a memóriaszegmenseket szegmensleírók írják le. A GDT egy táblázat, amely legalább a kernel kód- és adatszegmensét tartalmazza. A szegmensleírók meghatározzák a báziscímet, a limitet, a jogosultsági szintet és a szegmens típusát. A kernel beállítja a GDT-t, majd a `lgdt` utasítással betölti annak címét és méretét.

A modern 64 bites módban a szegmentálás erősen leegyszerűsödött: a bázis és limit a legtöbb szegmens esetében figyelmen kívül marad, de a GDT továbbra is kötelező a jogosultsági szintek és a kód/adat szegmens kiválasztásához. Egy tipikus minimális GDT:

```
typedef struct {
    uint16_t limit_low;
    uint16_t base_low;
    uint8_t  base_mid;
    uint8_t  access;
    uint8_t  limit_high : 4;
    uint8_t  flags : 4;
    uint8_t  base_high;
} __attribute__((packed)) gdt_entry_t;
```

A kernel kód szegmense `access` értéke például `0x9A` (present, code, ring 0), az adatszegmensé `0x92` (present, data, ring 0). A betöltés előtt a kernel feltölti a táblát, majd meghívja a `lgdt` utasítást.

A GDT betöltése után a kernel frissíti a szegmensregisztereket, hogy az új szelektorokra mutassanak. A kód szegmens frissítése általában egy távoli ugrással történik, mert közvetlen `mov` a CS regiszterbe nem lehetséges.

27.2.6 IDT (Interrupt Descriptor Table) x86-on

Az IDT hasonló felépítésű táblázat, de bejegyzései megszakítás- és kivételkezelőkre mutatnak. Minden bejegyzés egy kapu, amely leírja, hogy egy adott vektor (0–255) melyik kezelő rutinra mutat, és milyen jogosultsági szinten futtatható.

A kernel inicializáláskor legalább a kivételkezelőket beállítja. A 0. vektor a `#DE` (divide error), a 13. a `#GP` (general protection fault), a 14. a `#PF` (page fault), és így tovább. A bejegyzések típusa lehet interrupt gate vagy trap gate; az interrupt gate automatikusan letiltja a megszakításokat a kezelő futása alatt, ami a korai szakaszban egyszerűbb.

A betöltés a `lidt` utasítással történik. Egy IDT bejegyzés x86-64-en 16 byte hosszú:

```
typedef struct {
    uint16_t offset_low;
    uint16_t selector;
    uint8_t  ist : 3;
    uint8_t  reserved0 : 5;
    uint8_t  type : 4;
    uint8_t  zero : 1;
    uint8_t  dpl : 2;
    uint8_t  p : 1;
    uint16_t offset_mid;
    uint32_t offset_high;
    uint32_t reserved1;
} __attribute__((packed)) idt_entry_t;
```

A kezelő rutinok kezdetben egyszerű, assemblyben írt stubok, amelyek elmentik a regisztereket, és meghívnak egy C függvényt. A cél itt nem a teljes kivételkezelés, hanem az, hogy bármilyen korai kivétel esetén legyen egy kontrollált leállítás, ne pedig a CPU álljon meg megmagyarázhatatlan módon.

27.2.7 RISC-V trap tábla

RISC-V rendszereken az x86 GDT/IDT pár helyett a trap tábla tölti be a kivételkezelés szerepét. A kernel a `stvec` CSR-be írja a trap tábla címét. A tábla bejegyzései a kivétel okától függően eltérő kezelőkre mutatnak. A jogosultsági szinteket itt a M/S/U módok adják; a GDT-hez hasonló szegmenstábla nem létezik. A fejezet további példái x86-64-et feltételeznek, de a fogalmak RISC-V-n is megfeleltethetők.

27.2.8 Korai konzol

A korai konzol az első olyan eszköz, amely kifelé kommunikál. A célja, hogy a kernel naplóüzeneteket írjon ki, amint a belépési pont után lehetséges. A leggyakoribb választás emulált környezetben egy UART 16550 soros port, mert kevés inicializálást igényel, és a QEMU a `-serial stdio` opcióval a terminálra irányítja a kimenetet.

A 16550 UART az x86 platformon hagyományosan a `0x3F8` portcímen található (COM1). A minimális inicializálás a következő lépéseket foglalja magában:

1. Az interrupt engedélyező regisztert nullázni kell, hogy a soros port ne generáljon interruptot.
2. A baudráta-osztó beállítása, például 38400 baud.
3. A vonalvezérlő regiszter beállítása: 8 bit, nincs paritás, 1 stopbit.
4. A modemvezérlő regiszter beállítása: DTR és RTS engedélyezése.

Egy egyszerű C függvény az UART írásra:

```
#define COM1 0x3F8

static void uart_putc(char c) {
    while ((inb(COM1 + 5) & 0x20) == 0)
        ; /* várj, amíg a THR üres */
    outb(COM1, c);
}

void uart_init(void) {
    outb(COM1 + 1, 0x00);   /* interrupt tiltás */
    outb(COM1 + 3, 0x80);   /* DLAB = 1 */
    outb(COM1 + 0, 0x03);   /* osztó alsó byte (38400) */
    outb(COM1 + 1, 0x00);   /* osztó felső byte */
    outb(COM1 + 3, 0x03);   /* 8N1, DLAB = 0 */
    outb(COM1 + 4, 0x0B);   /* OUT2, RTS, DTR */
}
```

A `inb` és `outb` függvényeket assemblyben vagy beágyazott assemblyben kell megvalósítani. A kernel ezután egy minimális `printk`-szerű függvényt építhet, amely a formázott stringet karakterenként küldi az UART-ra. A korai szakaszban nincs szükség teljes `printf` implementációra; elegendő a stringek kiírása, valamint hex és decimális számok formázása a diagnosztikához.

A korai konzol másik előnye, hogy a bootfolyamat minden fontos lépését naplózni lehet. A napló a későbbi hibakeresés alapja, ezért a kimenetet a laborban mindig rögzíteni kell.

27.2.9 Panic a korai szakaszban

A `panic` a kernel kontrollált leállási mechanizmusa. Akkor hívjuk, ha a rendszer olyan állapotba kerül, amelyből nem lehet vagy nem érdemes tovább futni. A korai bootban a `panic` még egyszerűbb: csak kiír egy üzenetet, és megállítja a CPU-t.

Egy minimális `panic` implementációja:

```
__attribute__((noreturn)) void panic(const char *msg) {
    printk("PANIC: %s\n", msg);
    printk("Halting CPU.\n");
    disable_interrupts();
    for (;;) {
        hlt();
    }
}
```

A későbbi fejezetekben a `panic` kiegészül regiszter-dumppal, stack trace-szel és a napló átmeneti pufferének kiírásával. A bootfolyamat során azonban már az is hatalmas segítség, ha a kernel megbízhatóan kiírja a hibaüzenetet, mielőtt leállna.

A `panic` használatának tipikus esetei a korai szakaszban:

- A bootloader által átadott memóriatérkép üres vagy érvénytelen.
- A GDT vagy IDT betöltése előtt a CPU kivételt váltott ki.
- A kernel image linkelési címe és a tényleges betöltési cím nem egyezik.
- A konzol inicializálása sikertelen.
- A memóriatérképben nincs elegendő szabad RAM.

A `panic` üzenet legyen tömör, de egyértelmű. A korai szakaszban a napló lehet a soros konzolra írt szöveg, amelyet a labor során rögzítünk. A `hlt` utasítás addig állítja le a CPU-t, amíg egy megszakítás fel nem ébreszti, ezért előtte a megszakításokat le kell tiltani.

27.2.10 A kernel fő ciklusa

Miután a kernel elvégezte az alapvető inicializálást, belép egy várakozó ciklusba, amelyet `idle loop`-nak is nevezünk. Ebben a szakaszban még nincs ütemező, nincsenek folyamatok, ezért a kernel tétlenül várakozik megszakításokra.

Egy egyszerű fő ciklus:

```
void kernel_main_loop(void) {
    printk("Kernel elindult. Várakozás megszakításokra.\n");
    for (;;) {
        enable_interrupts();
        hlt();
        disable_interrupts();
    }
}
```

A `hlt` utasítás a CPU-t alacsony fogyasztású várakozó állapotba helyezi. Amikor egy megszakítás érkezik, a CPU felébred, a megszakításkezelő lefut, majd a vezérlés visszatér a `hlt` utáni utasításra. A ciklus itt egyszerűen újra várakozik.

Ez a ciklus a későbbiekben az ütemező üresjárati feladata lesz. A Memóriakezelés és ütemező tervezése fejezetben a memóriakezelő és az ütemező inicializálása után a `hlt` ciklus helyére egy olyan ütemező lép, amely a futásra kész folyamatok közül választ. A most bevezetett fő ciklus

tehát nem a végleges állapot, hanem egy olyan köztes lépés, amely lehetővé teszi a megszakítások tesztelését.

27.2.11 Inicializálási sorrend összeállítása

A fenti elemeket egyetlen, jól definiált sorrendbe kell rendezni. A javasolt sorrend a következő:

1. `_start` assembly belépési pont: stack, BSS nullázás, bootloader paraméter mentése.
2. Korai konzol inicializálása.
3. Kezdeti üzenet kiírása (a kernel verziója vagy neve).
4. `panic` infrastruktúra elérhetővé tétele.
5. GDT beállítása és betöltése.
6. IDT beállítása és betöltése.
7. Bootloader memóriatérkép feldolgozása.
8. A rendelkezésre álló RAM összesítése és naplózása.
9. Időzítő és interrupt-vezérlő minimális beállítása.
10. Belépés a fő ciklusba.

A sorrend fontos tulajdonsága, hogy a konzol már a 2. lépésben rendelkezésre áll, így minden későbbi hiba diagnosztizálható. A GDT/IDT beállítása a memóriatérkép feldolgozása előtt történik, mert a memóriatérkép bejárása közben fellépő kivételeknek már legyen kezelője. A memóriatérkép feldolgozása azért fontos, mert a kernel csak ezután tudja, hogy hol vannak a szabad és fenntartott területek.

A sorrend nem abszolút törvény; bizonyos architektúrákon a sorrend eltérhet. Például RISC-V-en a trap tábla beállítása megelőzheti a konzol inicializálását, ha a firmware már biztosít kimenetet. Az azonban általános elv, hogy a konzolt minél hamarabb el kell érni, és a `panic` mechanizmusnak már az elején készen kell lennie.

27.2.12 Gyakori hibák és ellenőrzések

A bootfolyamat során a hibák gyakran azonnali leálláshoz vagy néma lefagyáshoz vezetnek. A következő lista a leggyakoribb hibákat és a hozzájuk tartozó ellenőrzési pontokat foglalja össze.

1. **A stack nincs beállítva.** Ha a C kód a stack pointer inicializálása előtt fut, a függvényhívások egy véletlen címre írnak. Megoldás: a belépési pont első utasításai között be kell állítani a stacket.
2. **A BSS nincs nullázva.** A C szabvány szerint a globális változók kezdeti értéke nulla. Ha a BSS-t nem nullázzuk, a változók szemét értéket kapnak. Megoldás: a belépési kód nullázza a `.bss_start` és `.bss_end` közötti területet.
3. **A GDT/IDT betöltése hibás.** Ha a GDT bejegyzése rossz, a szegmens betöltésekor a CPU `#GP` kivételt vált ki. Ha az IDT nincs beállítva, az első kivétel háromszoros hibát okoz, és a CPU újraindul. Megoldás: a GDT és IDT struktúrákat byte-pontosan ellenőrizni, és a betöltés előtt egy `lidt`-tel és `lgdt`-vel tesztelni.
4. **A memóriatérkép érvénytelen.** Ha a kernel nem ellenőrzi a memóriatérkép-tömb hosszát vagy a típusokat, a nem szabad területekre is írhat. Megoldás: minden bejegyzést validálni, és a `panic` hívni, ha a térkép üres vagy sérült.
5. **A konzol inizializálása késik.** Ha a kernel a konzol beállítása előtt hibába ütközik, nincs diagnosztikai kimenet. Megoldás: a konzolt a boot-szekvencia legelső lépései közé tenni.

6. **A megszakítások túl korán engedélyezve lesznek.** Ha a megszakításokat az IDT és az interrupt-vezérlő inicializálása előtt engedélyezzük, egy váratlan interrupt kontrollálhatatlan állapotba viheti a rendszert. Megoldás: a megszakításokat csak az IDT, az interrupt-vezérlő és az időzítő beállítása után engedélyezni.
7. **A bootloader által átadott regiszterek felülíródnak.** A `kmain` hívása előtt a bootloader által átadott pointereket el kell menteni. Ha elvesznek, a kernel nem fér hozzá a memóriatérképhez. Megoldás: a belépési kódban a paramétereket globális változókba menteni.

27.2.13 Labor: boot- és inicializálási sorrend tervezése

A labor célja, hogy a fenti fogalmakat egy emulált környezetben, QEMU-ban kipróbáljuk. A labor nem igényel valós hardvert, és az éles gép bootloaderét, lemezét nem módosítjuk.

27.2.14 Kiinduló állapot és konfiguráció

A reprodukciós csomag a következőket tartalmazza:

- Fordítás: `gcc` freestanding módban, `ld` linker scripttel.
- Architektúra: x86-64.
- Emulátor: QEMU 8.2, `qemu-system-x86_64`.
- Konfiguráció: 64 MiB RAM, `-serial stdio`, `-no-reboot`, `-no-shutdown`.
- Bootloader: GRUB Multiboot2 vagy egy minimális saját második fázis; a laborban a kernel image-et a QEMU `-kernel` opciója közvetlenül betöltheti, ha a kép Multiboot2 fejléccel rendelkezik.

A link script meghatározza a kernel belépési címét:

```
OUTPUT_FORMAT(elf64-x86-64)
ENTRY(_start)

SECTIONS {
    . = 0x00100000;          /* 1 MiB fizikai cím */
    .boot.text : { *(.boot.text*) }
    .text : { *(.text*) }
    .rodata : { *(.rodata*) }
    .data : { *(.data*) }
    .bss : {
        __bss_start = .;
        *(.bss*)
        __bss_end = .;
    }
}
```

A kernel image 1 MiB-re kerül, a hagyományos x86-os alsó 1 MiB fölé.

27.2.15 Build és futtatás

A build parancs:

```
x86_64-elf-gcc -ffreestanding -mno-red-zone -mno-mmx -mno-sse -mno-sse2 \
    -c kernel.c -o kernel.o
x86_64-elf-as --64 boot.s -o boot.o
x86_64-elf-ld -T linker.ld -o kernel.elf boot.o kernel.o
```

A futtatási parancs:

```
qemu-system-x86_64 -m 64M -serial stdio -kernel kernel.elf \
    -no-reboot -no-shutdown
```

27.2.16 Elvárt eredmény

A labor elvárt kimenete a következő sorokat tartalmazza:

```
[BOOT] Kernel betöltődött, belépési pont elérve.
[BOOT] BSS nullázva, stack inicializálva.
[BOOT] Korai konzol inicializálva (com1, 38400 8N1).
[BOOT] GDT betöltve.
[BOOT] IDT betöltve.
[BOOT] Memóriatérkép: 64 MiB RAM található.
[BOOT] Belépés a fő ciklusba.
```

A labor során a mért kimenetet a fenti elvárt kimenettel össze kell hasonlítani. A naplóban a mért és várt értékeket külön oszlopban kell rögzíteni.

27.2.17 Mérési napló

A mérési napló a következő adatokat tartalmazza:

- A futás dátuma és a QEMU verziója.
- A CPU típusa (például `-cpu qemu64`).
- A memória mérete.
- A kernel image mérete bájtban.
- A betöltési cím (0x00100000).
- A konzol kimenet teljes szövege.
- A megfigyelt indulási idő a QEMU indításától az első konzol karakterig, ha a hoszt képes mérni.
- A `panic` teszt eredménye (lásd alább).

A boot idő méréséhez a hoszt `time` parancsát vagy a QEMU beépített időzítőjét használhatjuk. Fontos megkülönböztetni a mért fal-időt a kernel által érzékelt belső időtől. A laborban a fal-idő elegendő, de a mérési módszert pontosan rögzíteni kell.

27.2.18 Panic teszt

A labor részeként szándékosan hívjunk `panic`-et egy hibás ágon, például üres memóriatérkép esetén. Ehhez a kernel kódjában egy `if (mmap_entries == 0) panic("empty memory map");` hívást helyezünk el. Az elvárt kimenet:

```
PANIC: empty memory map
Halting CPU.
```

A mért kimenetet rögzítsük, és hasonlítsuk össze a várt kimenettel. A teszt után a kernel leáll, ezért a QEMU-t a `Ctrl-A X` kombinációval lehet kilépni.

27.2.19 Visszaállítás

A labor elvégzése után a munkakönyvtárat vissza kell állítani a kiinduló állapotba. Mivel a labor csak az emulátorban fut, nincs szükség hoszt-lemez-helyreállításra. A reprodukciós csomagot tömörítve el kell menteni, hogy a jövőben ugyanaz a build pontosan megismételhető legyen.

27.2.20 Összefoglalás

A bootfolyamat és a kernel inicializálás célja, hogy a hardvert és a szoftvert egy olyan ismert állapotba hozza, amelyre a későbbi absztrakciók építhetnek. A fejezet bemutatta a bootloader és a kernel közötti interfészt, a memóriatérkép szerepét, a GDT/IDT párost, a korai konzolt és a `panic` mechanizmust. A laborban mindez egy QEMU-ban futtatott minimális kernelben ellenőrizhető.

A következő fejezet (Memóriakezelés és ütemező tervezése) erre az alapra épít, amikor a memóriakezelést és az ütemező tervezését tárgyalja. Az itt bevezetett konzol, `panic` és fő ciklus azok az építőelemek, amelyekre a lapozás, a heapallokáció és a folyamatkezelés ráépül.

27.2.21 Önellenőrző kérdések

1. Miért kell a korai konzolt a bootfolyamat legelején inicializálni?
2. Mi a különbség a `.data` és a `.bss` szakasz között, és miért kell a BSS-t nullázni?
3. Mi a szerepe a GDT-nek és az IDT-nek x86-on? Mi történik, ha az IDT nincs betöltve, és a CPU kivételt vált ki?
4. Soroljon fel három tipikus boothibát, és adja meg mindegyikhez a helyes megoldást.
5. Mi a `panic` feladata, és miért fontos, hogy már a korai szakaszban elérhető legyen?

27.3 Memóriakezelés és ütemező tervezése

A boot utáni kernel két alrendszer nélkül nem tud tovább építkezni: **memóriakezelés** és **feladatütemezés**. Az `os103` fejezetben a kernel elindult, a soros konzol már írni tud, és a verem a helyén van. Ebben a fejezetben olyan egyszerű, de valódi komponenseket tervezünk, amelyekre a rendszerhívások, a folyamatok és a userland a későbbi egységekben ráépülnek. A cél nem egy teljes virtuális memóriarendszer vagy egy fair ütemező, hanem egy **ellenőrizhető váz**, amelyben a fizikai lapok kiosztása, a címfordítás, a taskváltás és az időosztás működik.

27.3.1 1. Fizikai memória és frame allocator

A boot után a kernel rendelkezésére álló fizikai memória tartományát a bootloader vagy a firmware adja meg. A továbbiakban egy `QEMU`-ban futó, `128 MiB` RAM-mal rendelkező, `virt` típusú emulált gépet feltételezünk; a RAM ebben a példában `0x80000000`–`0x88000000` között helyezkedik el. A tényleges tartomány hardverfüggő, ezért a kernel először ezt a tartományt rögzíti, és innen kezdve **frame-ekkel**, azaz `4 KiB` méretű fizikai lapokkal dolgozik.

A **frame allocator** feladata, hogy a fizikai lapokat kiadja és visszavegye. Egy kis, oktatási célú kernelben a legegyszerűbb helyes megoldás a **bitmap alapú allokátor**. Ebben minden frame-hez egy bit tartozik: ha a bit `1`, a lap foglalt, ha `0`, szabad. A bitmap maga is memóriát foglal, ezért az inicializáláskor először a bitmap számára le kell foglalni egy darabot a kezelt tartomány elejéről.

A bitmap indexelése a következő összefüggéssel számítható:

$$index = \frac{paddr - base}{4096}$$

A byte- és bitszintű pozíció:

$$byte = index \,/\, 8, \quad bit = index \bmod 8$$

A következő pszeudokód a `frame_init` lépéseit mutatja:

```
// A kód szemléltetés, nem végrehajtandó utasítás az éles gépen.
#define PAGE_SIZE  4096
#define FRAME_BASE 0x80000000UL
#define FRAME_END  0x88000000UL

static uint8_t *bitmap;
static uint32_t total_frames;

void frame_init(void) {
    total_frames = (FRAME_END - FRAME_BASE) / PAGE_SIZE;
    // A bitmap mérete: total_frames / 8 bájt, felfelé kerekítve.
    size_t bitmap_bytes = (total_frames + 7) / 8;
    bitmap = (uint8_t *)FRAME_BASE;
    // Az első frame-eket a bitmap számára lefoglaljuk.
    size_t reserved_frames = (bitmap_bytes + PAGE_SIZE - 1) / PAGE_SIZE;
    for (size_t i = 0; i < reserved_frames; ++i)
        bitmap_set(i, 1);
    // A többi frame szabad.
    for (size_t i = reserved_frames; i < total_frames; ++i)
        bitmap_set(i, 0);
}
```

A `bitmap_set`, `bitmap_clear` és `bitmap_test` segédfüggvények a fenti byte/bit felbontást valósítják meg. A `frame_alloc` és `frame_free` ezekre épül:

```
uintptr_t frame_alloc(void) {
    for (size_t i = 0; i < total_frames; ++i) {
        if (!bitmap_test(i)) {
            bitmap_set(i, 1);
            return FRAME_BASE + i * PAGE_SIZE;
        }
    }
    return 0; // nincs szabad frame
}

void frame_free(uintptr_t paddr) {
    if (paddr < FRAME_BASE || paddr >= FRAME_END || (paddr & (PAGE_SIZE - 1)))
        return; // hibás cím
    size_t index = (paddr - FRAME_BASE) / PAGE_SIZE;
    bitmap_clear(index);
}
```

A fenti vázlatban a `0` visszatérési érték hibát jelez, de ez veszélyes, mert a fizikai `0x00000000` cím ritkán ugyan, de nem keverendő össze a hibával. Érdemes inkább `INVALID_FRAME` konstanst használni, és a hívó oldalon mindig ellenőrizni. A tipikus hibák közé tartozik a **kettős felszabadítás**, amikor ugyanazt a frame-et kétszer adjuk vissza, és a **határellenőrzés hiánya**, ami a bitmapen kívüli memóriát módosíthat. A laborban ezért a `frame_free` előtt érdemes tesztelni, hogy a bit valóban `1`-es-e, és csak akkor törölni.

A frame allocator teljesítménye ebben a formában lineáris, mert a szabad lapot keresve végig kell vizsgálni a bitmapet. Ez a célfeladathoz elegendő; a haladó fejezetekben a buddy és slab allocátorok ezt a keresést gyorsítják.

27.3.2 2. Virtuális memória és lapasztal

A fizikai memória önmagában csak egy sor frame. A **virtuális memória** lehetővé teszi, hogy minden feladat saját, védett címtérben fusson, és a kernel a fizikai lapokat szükség szerint képezze be. A címfordítást az MMU végzi a **lapasztal** (page table) alapján.

Ebben a fejezetben egy 32 bites, két szintű lapozást használunk, 4 KiB lapmérettel. A virtuális cím három részre oszlik:

- **Page Directory Index (PDI):** a felső 10 bit.
- **Page Table Index (PTI):** a középső 10 bit.
- **Offset:** az alsó 12 bit.

A felbontás képlete:

$$PDI = (vaddr \gg 22)\ \&\ 0x3FF$$

$$PTI = (vaddr \gg 12)\ \&\ 0x3FF$$

$$offset = vaddr\ \&\ 0xFFF$$

A page directory minden bejegyzése egy page table-re mutat, a page table bejegyzései pedig fizikai frame-ekre. A bejegyzések alsó bitjei **flag-ek**: a `PRESENT` bit jelzi, hogy a lap a memóriában van, az `RW` bit az írási jogot, az `USER` bit a felhasználói hozzáférést vezérli. A kernel attól függően, hogy milyen architektúrán fut, eltérő flagkészletet használ, de az elv ugyanaz.

A lapkezelő API három műveletre épül:

1. **`walk_page(pgdir, vaddr, alloc)`** – megkeresi a virtuális címhez tartozó page table bejegyzést. Ha a page table még nem létezik, és az `alloc` igaz, lefoglal egy frame-et a page table számára.
2. **`map_page(pgdir, vaddr, paddr, flags)`** – bejegyzést hoz létre a virtuális cím és a fizikai frame között.
3. **`unmap_page(pgdir, vaddr)`** – törli a bejegyzést.

A következő C-vázlat a `walk_page` és a `map_page` működését mutatja be. A példa feltételezi, hogy a kernel a korai fázisban **identity mapped**, vagyis a fizikai címeket közvetlenül eléri; a lapkezelő ezért fizikai címekkel dolgozik.

```
// Szemléltető C vázlat, nem végrehajtandó utasítás.
typedef uint32_t pte_t;
typedef uint32_t pde_t;

#define PTE_PRESENT  0x001
#define PTE_RW       0x002
#define PTE_USER     0x004

pte_t *walk_page(uint32_t *pgdir, uintptr_t vaddr, int alloc) {
    uint32_t *pde = &pgdir[(vaddr >> 22) & 0x3FF];
```

```
    if (!(*pde & PTE_PRESENT)) {
        if (!alloc)
            return NULL;
        uintptr_t pte_frame = frame_alloc();
        if (!pte_frame)
            return NULL;
        *pde = (pte_frame & 0xFFFFF000) | PTE_PRESENT | PTE_RW;
        // Az új page table-t nullázni kell.
        memset((void *)pte_frame, 0, PAGE_SIZE);
    }

    uint32_t *pt = (uint32_t *)(*pde & 0xFFFFF000);
    return &pt[(vaddr >> 12) & 0x3FF];
}

int map_page(uint32_t *pgdir, uintptr_t vaddr, uintptr_t paddr, uint32_t flags) {
    pte_t *pte = walk_page(pgdir, vaddr, 1);
    if (!pte)
        return -1;
    if (*pte & PTE_PRESENT)
        return -1; // a cím már le van képezve
    *pte = (paddr & 0xFFFFF000) | (flags & 0xFFF) | PTE_PRESENT;
    return 0;
}
```

A `map_page` hibát ad, ha a fizikai cím nincs laphatárra igazítva, vagy ha már létezik a leképezés. Az utóbbi ellenőrzés azért fontos, mert két feladat nem kaphatja meg ugyanazt a virtuális címet különböző jelentéssel ugyanabban a lapasztalban úgy, hogy az adat sérüljön. A gyakorlatban a kernel a saját lapozását identity mappinggel indítja, majd a feladatok számára külön page directory-t hoz létre.

A lapkezelő hívásakor a **TLB** (translation lookaside buffer) tartalma elavulhat. Ezért minden `map_page` és `unmap_page` után a megfelelő architektúra-specifikus TLB-invalidálást is el kell végezni. Ennek elmaradása a legnehezebben megtalálható memóriahibák egyike: a program néha a régi leképezést látja, néha az újat, attól függően, hogy a TLB-ben éppen mi van.

27.3.3 3. Feladatok és a task control block

A feladatok kezeléséhez a kernel minden végrehajtási egységhez egy **Task Control Blockot (TCB)** rendel. Ez a struktúra tárolja a feladat azonosítóját, állapotát, saját lapasztalát, veremmutatóját és a CPU kontextusát.

Egy kezdetleges, de tanítható TCB a következő mezőket tartalmazza:

```
typedef enum {
    TASK_READY,
    TASK_RUNNING,
    TASK_BLOCKED,
    TASK_EXITED
} task_state_t;

typedef struct task {
    uint32_t pid;
    task_state_t state;
    uint32_t *page_dir;     // a feladat saját lapasztala
    void *stack_top;        // a kernelstack teteje
    struct cpu_context *ctx; // mentett regiszterek
    uint32_t quantum_left;  // hátralévő kvantum
```

```
    struct task *next;      // láncolt lista
} task_t;
```

Az állapotátmenetek egy korai kernelben egyszerűek. A feladat a `READY` állapotból kerül `RUNNING` állapotba, amikor az ütemező kiválasztja. Ha a kvantum lejár, visszakerül `READY`-be. Ha egy eseményre vár, például egy jövőbeli I/O-műveletre, `BLOCKED` állapotba kerül, és az esemény bekövetkezése után újra `READY` lesz. A befejezett feladat `EXITED` állapotot kap, és a TCB felszabadítható.

A feladatok tárolása a legegyszerűbb módon egy **statikus tömbben** történik, a szabad helyeket egy láncolt lista, azaz **free list** tartja nyilván. Ebben a fázisban a feladatok száma kicsi, ezért a dinamikus memóriafoglalás helyett a statikus tömb is elegendő, és elkerüli a kernel heapjének korai inicializálását.

27.3.4 4. Round-robin ütemező

A **round-robin** ütemező a legegyszerűbb időosztásos algoritmus. Minden `READY` feladat azonos kvantumot kap, és a feladatok körkörös sorrendben futnak. Ha az aktuális feladat kvantuma lejár, az ütemező a következő `READY` feladatra vált, a lejárt feladatot pedig a sor végére állítja.

A működéshez egy ciklikus **ready queue** kell. A `task_t` struktúra `next` mezője erre a listára mutat. Az ütemező két fő függvénye a `scheduler_tick` és a `schedule`.

A `scheduler_tick` minden timer-interruptnál lefut, és csökkenti az aktuális feladat kvantumát. Ha a kvantum elfogy, a feladat kvantumát visszaállítja, és ütemezést kér:

```
#define QUANTUM 10 // 10 timer tick, 1 tick = 1 ms

void scheduler_tick(void) {
    if (current == NULL || current->state != TASK_RUNNING)
        return;
    if (--current->quantum_left <= 0) {
        current->quantum_left = QUANTUM;
        current->state = TASK_READY;
        schedule();
    }
}
```

A `schedule` kiválasztja a következő futásra kész feladatot. A ready listán a `current next` mutatójától kezdve keres. Ha nem talál más feladatot, az aktuális marad futásban. A feladatok cseréjéhez a `context_switch` függvényre van szükség, amelyet a következő szakasz tárgyal.

```
void schedule(void) {
    if (current == NULL)
        return;

    task_t *old = current;
    task_t *next = old->next;

    // Olyan feladatot keresünk, amelyik futásra kész.
    while (next != old && next->state != TASK_READY && next->state != TASK_RUNNING)
        next = next->next;

    if (next == old || next->state != TASK_READY)
        return; // nincs másik futtatható feladat

    old->state = (old->state == TASK_RUNNING) ? TASK_READY : old->state;
    next->state = TASK_RUNNING;
    task_t *prev = current;
```

```
    current = next;
    context_switch(&prev->ctx, &next->ctx);
}
```

A `context_switch` a két task kontextusát cseréli. A ready queue forgatását legegyszerűbben úgy lehet elvégezni, hogy a `scheduler_tick` lejárt kvantumnál az aktuális feladatot a lista végére fűzi, de ez a kódvázlatban a `next` kereséssel valósul meg. A fontos invariáns: **mindig legyen legalább egy futásra kész feladat**, különben a ready queue üres lesz, és a rendszer leáll.

A következő Gantt-diagram három feladat (`P1`, `P2`, `P3`) futását mutatja, 10 ms kvantummal:

```
Idő(ms):   0     10    20    30    40    50
         |  P1  |  P2  |  P3  |  P1  |  P2  |
```

Jelmagyarázat: egy cella egy teljes kvantum. A függőleges vonalak a timer-interrupt és a taskváltás pillanatai. A váltás időigénye ebben az ábrán elhanyagoltnak tekinthető, de a laborban érdemes mérni.

27.3.5 5. Kontextusváltás és idle task

A **kontextusváltás** az a pillanat, amikor a CPU az egyik feladat regisztereit és veremmutatóját elmenti, majd egy másik feladatét betölti. A mentendő állapotot **cpu context**-nek nevezzük. Ez legalább a következőket tartalmazza:

- általános célú regiszterek,
- veremmutató (ESP/RSP),
- utasításmutató (EIP/RIP),
- lapasztal regiszter (CR3),
- állapotregiszter (EFLAGS/RFLAGS).

A váltás assembly szinten történik, mert a C nem fér hozzá közvetlenül a regiszterekhez. A következő pszeudokód a váltás vázlatát adja meg:

```
; Szemléltető assembly vázlat, nem végrehajtandó utasítás.
; context_switch(old_ctx, new_ctx)
context_switch:
    ; 1. A hívó által megőrzött regiszterek mentése a veremre.
    push ebp
    push ebx
    push esi
    push edi

    ; 2. Az aktuális veremmutató mentése az old_ctx-be.
    mov  eax, [esp + 20]   ; old_ctx címe
    mov  [eax], esp

    ; 3. Az új veremmutató betöltése.
    mov  ebx, [esp + 24]   ; new_ctx címe
    mov  esp, [ebx]

    ; 4. A regiszterek visszaállítása az új veremről.
    pop  edi
    pop  esi
    pop  ebx
    pop  ebp
    ret
```

Ez a vázlat csak a kernelstacket cseréli; a lapasztal cseréje a `CR3` regiszter módosításával történik, amit a hívó C kód végez el a `context_switch` előtt vagy után. A kernel a saját lapozását minden feladatnál közösen használhatja, de a felhasználói címterek eltérőek. A pontos megvalósítás architektúrafüggő, és a `os107` fejezet részletezi a kész kernelben.

Az **idle task** az a feladat, amely akkor fut, amikor nincs más futásra kész munka. A round-robin ütemezőben azért kritikus, mert a ready queue soha nem lehet üres: ha nincs felhasználói feladat, a CPU-n akkor is kell egy végrehajtási egység. Az idle task a legalacsonyabb prioritású, soha nem blokkol, és a CPU-t alacsony fogyasztású állapotba helyezi:

```
void idle_task(void) {
    for (;;) {
        asm volatile("hlt"); // x86: HLT, RISC-V: WFI
    }
}
```

A `HLT` vagy `WFI` utasítás leállítja a CPU futószalagját, amíg megszakítás nem érkezik. Így a timer interrupt felébreszti, a `scheduler_tick` lefut, és ha van `READY` feladat, az ütemező arra vált. Az idle task jelenléte a rendszer stabilitásának alapfeltétele.

27.3.6 6. Integráció és inicializálás

A memóriakezelő és az ütemező beüzemelése szigorú sorrendben történik. A boot utáni kernel inicializáló kódja a következő lépéseket hajtja végre:

1. **Frame allocator inicializálása** – a rendelkezésre álló fizikai tartomány felmérése és a bitmap elhelyezése.
2. **Kernel lapozás felépítése** – az első néhány MiB identity mappinggel, hogy a kernel továbbra is elérje a fizikai memóriát.
3. **Ütemező inicializálása** – a task tömbből free list építése.
4. **Idle task létrehozása** – a ready queue első eleme.
5. **Első felhasználói task létrehozása** – a tesztfeladat saját lapasztallal és stackkel.
6. **Timer bekapcsolása** – a timer interrupt engedélyezése, a `scheduler_tick` bekötése.
7. **Átadás az ütemezőnek** – az első `schedule` hívás.

A folyamat pszeudokódban:

```
void kernel_init(void) {
    frame_init();
    init_kernel_page_table();
    init_scheduler();
    create_idle_task();
    create_first_task();
    enable_timer();
    schedule(); // innen kezdve a feladatok váltakozva futnak
}
```

A `create_first_task` egy olyan feladatot hoz létre, amely a saját page directoryjával rendelkezik, és a kernelből átmásolt kódot futtat. A feladat létrehozásakor a `cpu_context` kezdőértékét úgy kell beállítani, hogy a `context_switch` utáni `ret` a feladat belépési pontjára ugorjon, és a verem a megadott `stack_top`-on legyen.

27.3.7 7. Labor: lapkezelő és ütemező vázlat

A labor célja, hogy a fenti komponenseket egy emulált környezetben összeállítsa, és a működést mérje. A labor a `os103` bootkódjára épül, és **QEMU**-ban fut. A kiinduló állapot: a kernel betöltődik, a soros konzol működik, de memóriakezelő és ütemező még nincs.

27.3.8 7.1. Konfiguráció és build

A reprodukálhatósághoz rögzítsük az emulált gépet:

- `qemu-system-riscv64 -machine virt -m 128M -kernel kernel.elf -nographic`
- CPU: `rv64`, a timer interrupt a `virt` gépben a `mtimecmp` és `mtime` regisztereken keresztül érhető el.
- A kernel fordításához `riscv64-unknown-elf-gcc` és `ld` szükséges, a linker script a `kernel.ld`.

A labor minden lépését jegyzetfüzetben kell rögzíteni, ahol a buildparancs, a futtatási parancs és a napló együtt látható. A `git` kezdő commit a kiinduló állapot; a labor után a `git diff` mutatja a változtatásokat, így a visszaállítás egy `git reset --hard` paranccsal megoldható.

27.3.9 7.2. Feladatok

1. **Frame allocator** – Implementálja a `frame_init`, `frame_alloc`, `frame_free` függvényeket a megadott memóriatartományra. Tesztelje, hogy az első lefoglalások a bitmap után következő szabad frame-eket adják vissza, és hogy a felszabadítás után a frame újra lefoglalható.
2. **Lapkezelő** – Építse fel a saját kernel page directoryt. Képezze be a kernel első `8 MiB`-ját identity mappinggel, majd hozzon létre egy teszt virtuális címet, például `0x40000000`, amely egy frissen lefoglalt fizikai frame-re mutat.
3. **Ütemező** – Hozzon létre három feladatot, amelyek mindegyike a soros konzolra írja a saját azonosítóját, majd `yield`-el. Az idle task egy `WFI` utasítást hajt végre. A timer interrupt 1 kHz-en fut, a kvantum 10 tick.
4. **Mérés** – Mérje a kontextusváltás idejét ciklusszámlálóval. A `mcycle` regiszter olvasása a váltás előtt és után adja a különbséget. A mérést legalább 100 váltáson végezze el.

27.3.10 7.3. Elvárt és mért eredmény

A labor során külön kell választani a **várt** és a **mért** értékeket.

Mutató	Várt / levezetett	Mért
Lefoglalt frame-ek száma	3 + a page table-ek száma	a naplóban szereplő érték
Taskok futási sorrendje	P1, P2, P3, P1, P2, …	a konzolnapló
Kontextusváltás ciklusban	néhány tíz ciklus (a mentett regiszterek és memóriaelérések függvényében)	a mérési napló

A várt kontextusváltási idő durva becslés, mert a `mcycle` érték függ a CPU modelljétől és a cache-állapottól. A laborban ezért a mért értéket nem egyetlen számként, hanem minimum, átlag és maximum formájában érdemes rögzíteni.

27.3.11 7.4. Visszaállítás

A labor után a virtuális gép leállítható, a `git` working tree pedig a kiinduló commitra állítható vissza. A naplófájlokat a reprodukciós csomag részeként meg kell őrizni.

27.3.12 8. Önellenőrző kérdések

1. Mi a különbség a fizikai és a virtuális cím között a feladatok izolációjában? Hogyan biztosítja a lapkezelő, hogy két feladat ne férjen hozzá egymás memóriájához?
2. Miért van szükség idle taskra egy round-robin ütemezőben? Mi történik, ha a ready queue üres?
3. Milyen lépésekből áll a `map_page`? Miért kell a page table-t nullázni az első lefoglalás után, és mi történik, ha a TLB-t nem invalidáljuk?

27.3.13 9. Összefoglalás

Ez a fejezet felépítette a kernel két alapvető alrendszerének vázát: a frame allocator és a lapkezelő a fizikai memória felosztásáért és a virtuális címek leképezéséért felel, az ütemező pedig a feladatok végrehajtását osztja időben. A round-robin algoritmus, a context switch és az idle task együtt biztosítják, hogy a rendszer soha ne álljon le, és a feladatok fair módon jussanak CPU-időhöz. A következő fejezet (`os105`) ezekre az alapokra építve bevezeti a rendszerhívásokat és a userland belépési pontját, amihez a lapkezelő `USER` flagje és az ütemező taskváltása már rendelkezésre áll.

27.4 Rendszerhívások és userland tervezése

27.4.1 1. A kernel és a felhasználói programok interfésze

Az operációs rendszer kernelje két különböző végrehajtási világot választ el: a privilegizált kernelmódot és a korlátozott felhasználói módot. A felhasználói programok nem érhetik el közvetlenül a hardvert, nem módosíthatják a lapasztalokat, és nem hívhatják meg a kernel belső függvényeit. Ha egy felhasználói program fájlt szeretne olvasni, memóriát igényel, vagy új folyamatot indít, akkor ellenőrzött átjárón kell átlépnie a kernel területére. Ezt az átjárót **rendszerhívásnak** (system call, syscall) nevezzük.

A rendszerhívás nem egyszerű függvényhívás. Egy közönséges C függvényhívás ugyanazon a privilegizáltsági szinten marad, és a hívó által ismert memóriacímeken dolgozik. A syscall viszont processzormód-váltással jár, és a kernelnek feltételeznie kell, hogy a felhasználó által átadott minden cím és méret hibás, rosszindulatú vagy következetlen. Ezért a rendszerhívás felületét **szerződésként** kell megtervezni.

A szerződés elemei:

1. **A rendszerhívás neve és száma.** A kernel és a felhasználói oldal ugyanazt a numerikus azonosítót rendeli a művelethez. A szám nem változhat egy kiadott ABI-ban.
2. **A paraméterek típusa, sorrendje és átadási módja.** Skalár értékek regiszterekben, pufferek esetén a felhasználói memóriacím és a hossz.
3. **A visszatérési érték.** Siker esetén általában nemnegatív érték, például a beolvasott bájtok száma. Hiba esetén a kernel negatív hibakódot ad vissza, amelyet a felhasználói könyvtár `errno` értékké alakít.

4. **A mellékhatások és az erőforrás-tulajdonjog.** Például egy megnyitott fájlleírót a hívónak kell bezárnia.

Az interfész tervezésekor el kell különítenünk az **API-t** (application programming interface) és az **ABI-t** (application binary interface). Az API a C fejlécfájlokban deklarált függvények szintjét jelenti: `int open(const char *path, int flags);`. Az ABI az a bináris szintű megállapodás, amely meghatározza, hogy a `syscall` utasítás előtt mely regiszterek mit tartalmaznak, hogyan kódolják a hibát, és hogyan kell a veremmutatót beállítani. Az ABI-t akkor is be kell tartani, ha a programot más fordítóval vagy más nyelvből fordították.

A fejezetben egy oktatókernelhez tervezünk ABI-t. A konkrét regiszterkiosztás az x86-64 architektúra egy lehetséges egyszerűsített változata; a valós rendszerben az adott processzorcsalád privilegizált specifikációjának aktuális verzióját kell alapul venni.

27.4.2 2. Syscall dispatcher tervezése

A **dispatcher** az a kernelbeli belépési pont, amely a felhasználói hívásokat szétosztja a megfelelő kernel függvényeknek. Feladatai:

- a processzorkontextus mentése,
- a syscall szám ellenőrzése,
- az argumentumok kinyerése a regiszterekből,
- a hívott kernel függvény végrehajtása,
- a visszatérési érték beállítása,
- a felhasználói módba való visszatérés.

Az oktatókernelben a syscall táblát egy függvénymutató-tömbként valósítjuk meg. A tábla méretét fordítási időben rögzítjük, így a nem definiált syscall számokra az `ENOSYS` hibakódot adhatjuk vissza.

```
/* syscall_table.c - szemléltető kód, nem éles rendszerre */

typedef long (*syscall_fn_t)(long, long, long, long, long, long);

#define NR_SYSCALLS 32

static syscall_fn_t syscall_table[NR_SYSCALLS];

void syscall_init(void)
{
    syscall_table[SYS_EXIT]   = sys_exit;
    syscall_table[SYS_WRITE]  = sys_write;
    syscall_table[SYS_READ]   = sys_read;
    syscall_table[SYS_OPEN]   = sys_open;
    syscall_table[SYS_CLOSE]  = sys_close;
    syscall_table[SYS_GETPID] = sys_getpid;
    syscall_table[SYS_SBRK]   = sys_sbrk;
    syscall_table[SYS_FORK]   = sys_fork;
    syscall_table[SYS_EXECVE] = sys_execve;
    syscall_table[SYS_WAITPID]= sys_waitpid;
    syscall_table[SYS_YIELD]  = sys_yield;
    syscall_table[SYS_SLEEP]  = sys_sleep;
}
```

A kernel belépési pontját assemblyben írjuk. Az x86-64 `syscall` utasítás hatására a processzor a `LSTAR` modell-specifikus regiszterben tárolt címre ugrik, és kernelmódba vált. A belépéskor a

felhasználói regisztereket egy **trap frame** struktúrába mentjük, hogy a dispatcher és a hívott függvény ne sérthesse a felhasználói állapotot.

Egy lehetséges trap frame vázlat:

```
typedef struct {
    uint64_t rax;
    uint64_t rdi;
    uint64_t rsi;
    uint64_t rdx;
    uint64_t r10;
    uint64_t r8;
    uint64_t r9;
    uint64_t rip;
    uint64_t rflags;
    uint64_t rsp;
} trap_frame_t;
```

A dispatcher C nyelvű váza a következő:

```
void syscall_entry(trap_frame_t *tf)
{
    long nr = tf->rax;

    if (nr < 0 || nr >= NR_SYSCALLS) {
        tf->rax = -ENOSYS;
        return;
    }

    long a0 = tf->rdi;
    long a1 = tf->rsi;
    long a2 = tf->rdx;
    long a3 = tf->r10;
    long a4 = tf->r8;
    long a5 = tf->r9;

    tf->rax = syscall_table[nr](a0, a1, a2, a3, a4, a5);
}
```

A visszatérés a `sysret` utasítással történik, amely visszaállítja a felhasználói `RIP`-t, `RSP`-t és `RFLAGS`-et. A dispatchernek ezért nem kell külön kontextusváltást végeznie; a processzor és a belépési assembly kód együtt gondoskodik a megfelelő állapotról.

Fontos, hogy a dispatcherben semmilyen felhasználói címre mutató értéket ne használjunk közvetlenül. A `tf` maga a kernel veremben van, ezért az biztonságos. A felhasználói regiszterekből kapott mutatókat a `sys_call` függvényekben ellenőrizni kell.

27.4.3 3. Argumentumátadás és visszatérés

A rendszerhívások argumentumait az ABI szerint regiszterekben adjuk át. Az oktatókernel x86-64 konvenciója:

Regiszter	Szerep
`rax`	syscall szám, majd visszatérési érték
`rdi`	1. argumentum
`rsi`	2. argumentum
`rdx`	3. argumentum
`r10`	4. argumentum

Regiszter	Szerep
r8	5. argumentum
r9	6. argumentum

A felhasználói oldalon egy `syscall()` wrapper függvény tölti be ezeket a regisztereket. Egy lehetséges assembly implementáció:

```
/* syscall_wrapper.S - szemléltető kód */
.global syscall
syscall:
    mov %rdi, %rax   /* 1. paraméter: syscall szám */
    mov %rsi, %rdi   /* 2. paraméter: 1. arg */
    mov %rdx, %rsi   /* 3. paraméter: 2. arg */
    mov %rcx, %rdx   /* 4. paraméter: 3. arg */
    syscall
    ret
```

A C oldalon a hívás így néz ki:

```
long syscall(long number, long a0, long a1, long a2);
```

A kernel a rendszerhívásból visszatérve a `rax`-ben adja vissza az eredményt. Ha a visszatérési érték negatív, akkor az hibakódot jelent. A felhasználói C könyvtár ezt a következőképpen alakítja `errno`-vá:

```
long result = syscall(SYS_WRITE, fd, buf, len);
if (result < 0) {
    errno = -result;
    return -1;
}
return result;
```

27.4.4 3.1. Mutatók és pufferek ellenőrzése

A legnagyobb biztonsági kockázatot a felhasználói memóriacímek jelentik. A kernel **soha nem dereferálhat felhasználói mutatót közvetlenül**, mert:

- a cím a kernel címtartományba eshet,
- a cím nem leképezett lapra mutathat,
- a cím mögötti memória egy másik folyamathoz tartozhat,
- a felhasználó a hívás után megváltoztathatja a puffert (TOCTOU).

A megoldás a **felhasználói és kernel közötti másolás**. Definiálunk két segédfüggvényt: `copy_from_user()` és `copy_to_user()`. Ezek először ellenőrzik, hogy a teljes tartomány a felhasználói címtartományon belül van-e, laponként leképezett-e, és csak ezután másolják az adatokat.

Az oktatókernelben a felhasználói címtartomány felső határa `USER_TOP`, például `0x80000000`. A legegyszerűbb ellenőrzés:

```
static long check_user_range(const void *uptr, size_t len)
{
    uintptr_t addr = (uintptr_t)uptr;
    if (addr >= USER_TOP || addr + len > USER_TOP) {
        return -EFAULT;
    }
    /* Lap-alapú ellenőrzés: minden oldal user mapping legyen */
```

```
    return 0;
}
```

A `copy_from_user` ezután bájtonként vagy szavanként másol, és minden elérhetetlen lapnál `-EFAULT` hibával tér vissza.

27.4.5 3.2. Példa: a write rendszerhívás

A `write` syscall a felhasználói puffert írja ki egy fájlleíróra. Az oktatókernelben először csak a szabványos kimenetet támogatjuk:

```
static long sys_write(int fd, const char *buf, size_t len)
{
    if (fd != STDOUT_FILENO && fd != STDERR_FILENO) {
        return -EBADF;
    }

    if (len > 128) {
        return -EINVAL;   /* az oktatókernel kis puffert használ */
    }

    char kbuf[128];
    if (copy_from_user(kbuf, buf, len) != 0) {
        return -EFAULT;
    }

    console_write(kbuf, len);
    return (long)len;
}
```

A függvény ellenőrzi a fájlleírót, a pufferméretet, majd a kernel saját veremén lévő pufferré másol. A `console_write` már csak kernelbeli címet kap, így biztonságos.

A visszatérési érték a kiírt bájtok száma. Ha a felhasználó rossz mutatót ad meg, a függvény `-EFAULT` értékkel tér vissza, amelyből a userland `errno = EFAULT` lesz.

27.4.6 3.3. Tipikus hibák az argumentumkezelésben

- **Hiányzó tartományellenőrzés:** ha a kernel a `buf + len` számítást végzi el a felhasználói mutatón, az túlcsordulás miatt kernel címet érhet el.
- **A hossz nem illeszkedik a pufferhez:** ha a kernel fix méretű `kbuf`-ot használ, de a felhasználói `len` nagyobb, verem-túlcsordulás következhet be.
- **TOCTOU (time-of-check to time-of-use):** ha a kernel először ellenőrzi a puffert, majd később ismét a felhasználói memóriát olvassa, a felhasználó közben megváltoztathatja azt. A másolásnak egyszer kell megtörténnie.
- **Nem kezelt hibakód:** ha a hívott függvény negatív `errno`-t ad vissza, de a dispatcher ezt pozitív értékként adja tovább, a userland hibát nem érzékeli.
- **A regiszterek felülírása a kontextusmentés előtt:** ha az assembly belépő nem menti el az összes hívó által használt regisztert, a visszatérés után a felhasználói program sérülhet.

27.4.7 4. Init, shell és programbetöltés

A kernel inicializálása után az első felhasználói módban futó folyamat az **init**, amelynek PID-je 1. Az init felelős a rendszer alapszolgáltatásainak elindításáért; az oktatókernelben ez egyetlen shell folyamat indítását jelenti.

Az init programot vagy beépítjük a kernel képfájljába, vagy a bootolás során betöltjük egy kezdeti fájlrendszerből. Az utóbbi rugalmasabb, de a beépített változat egyszerűbb, és az első lépésekhez elegendő.

27.4.8 4.1. Programbetöltés

A programbetöltő feladata, hogy a lemezen vagy a memóriában tárolt programképből felhasználói folyamatot hozzon létre. A folyamat a következő lépésekkel jön létre:

1. **Képfájl elemzése.** Az oktatókernel egy egyszerű, saját programformátumot használhat, amely szegmenseket, belépési pontot és méretet tartalmaz. A valós rendszerekben az ELF formátum használata a megszokott, de annak részletes elemzése a későbbi fejezetekre tartozik.
2. **Lapok lefoglalása.** A kernel a folyamat számára új lapokat rendel a programkép szegmenseihez.
3. **Szegmensek betöltése.** A kód- és adatszegmenseket a megfelelő virtuális címekre másoljuk.
4. **Verem inicializálása.** A kernel a felhasználói verem tetejére helyezi a program argumentumait és környezeti változóit.
5. **Regiszterek beállítása.** A `RIP` a belépési pontra, a `RSP` a verem tetejére mutat, a `RFLAGS` pedig felhasználói módot állít be.
6. **Visszatérés felhasználói módba.** A kernel a `sysret` vagy `iret` utasítással elindítja a folyamatot.

A betöltőnek figyelnie kell arra, hogy a folyamat ne kaphasson kernel lapokat, és a verem ne lógjon túl a lefoglalt tartományon. A felhasználói `RSP`-nek a lefoglalt verem tetejére kell mutatnia, és a verem méretét a folyamat struktúrájában rögzíteni kell.

27.4.9 4.2. Init folyamat

Az oktatókernel `init` programja először meghívja a `write` syscall-t, hogy üzenetet írjon a konzolra, majd végtelen ciklusban olvas a szabványos bemenetről és egyszerű parancsokat hajt végre:

```
/* init.c - szemléltető userland program */
#include "syscall.h"

int main(void)
{
    const char msg[] = "init: shell started\n";
    syscall(SYS_WRITE, STDOUT_FILENO, msg, sizeof(msg) - 1);

    for (;;) {
        char cmd[64];
        long n = syscall(SYS_READ, STDIN_FILENO, cmd, sizeof(cmd) - 1);
        if (n <= 0) {
            syscall(SYS_YIELD);
            continue;
        }
```

```
        cmd[n] = '\0';

        if (strncmp(cmd, "hello", 5) == 0) {
            const char hello[] = "hello from userland\n";
            syscall(SYS_WRITE, STDOUT_FILENO, hello, sizeof(hello) - 1);
        } else if (strncmp(cmd, "pid", 3) == 0) {
            long pid = syscall(SYS_GETPID);
            syscall(SYS_WRITE, STDOUT_FILENO, &pid, sizeof(pid));
        }
    }
    return 0;
}
```

Ez a program nem teljes shell, de bemutatja a userland és a kernel közötti együttműködést. Egy valódi shell a beolvasott sort parancsokra bontja, majd a `fork` és `execve` syscall-okkal új folyamatot indít.

27.4.10 4.3. A shell mint felhasználói program

A shell azért felhasználói program, mert nem igényel kerneljogosultságot. A parancsok értelmezése, a fájlnevek kiegészítése és a feladatvezérlés mind userland feladat. A kernel csak a folyamatkezeléshez és a fájl I/O-hoz biztosít rendszerhívásokat.

A minimális shell fő hurka:

```
while ((n = read(0, line, sizeof(line) - 1)) > 0) {
    line[n] = '\0';
    if (parse_and_execute(line) < 0) {
        write(2, "sh: command not found\n", 22);
    }
}
```

A `parse_and_execute` egy egyszerű változatban a parancs nevét és argumentumait bontja szét, majd `fork` segítségével új folyamatot hoz létre, és a gyermekben `execve`-vel betölti a kért programot. A szülő `waitpid`-del vár a gyermek befejezésére.

Ehhez a következő syscall-ok szükségesek:

- `fork`: új folyamat létrehozása,
- `execve`: új programkép betöltése,
- `waitpid`: gyermek állapotának lekérdezése,
- `read` és `write`: konzol I/O,
- `exit`: folyamat befejezése.

Az oktatókernelben a `fork` és `execve` kombinációja helyett bevezethetünk egy egyszerűbb `spawn` syscall-t is, amely egyetlen hívásban létrehozza az új folyamatot és betölti a programot. Ez csökkenti a kernel bonyolultságát, de eltér a POSIX szemantikától. A fejezetben a fogalmi modell ismertetéséhez a klasszikus `fork` + `execve` párt használjuk, a konkrét oktatókernel implementáció dönthet a `spawn` mellett.

27.4.11 5. Minimális syscall készlet tervezése

A következő táblázat egy lehetséges minimális syscall-listát mutat az oktatókernelhez. A számok és a paraméterek a saját ABI-nk részét képezik, és nem feltétlenül egyeznek a Linux vagy más rendszerek számaival.

Szám	Név	Paraméterek	Visszatérés siker esetén	Hibakód (`errno`)
0	`exit`	`int status`	nem tér vissza	—
1	`write`	`int fd, const char *buf, size_t len`	kiírt bájtok száma	`EBADF`, `EFAULT`, `EINVAL`
2	`read`	`int fd, char *buf, size_t len`	beolvasott bájtok száma	`EBADF`, `EFAULT`, `EINVAL`
3	`open`	`const char *path, int flags`	fájlleíró	`ENOENT`, `EACCES`, `EFAULT`
4	`close`	`int fd`	0	`EBADF`
5	`getpid`	—	folyamatazonosító	—
6	`sbrk`	`intptr_t increment`	régi programtörés	`ENOMEM`
7	`fork`	—	0 a gyermekben, PID a szülőben	`EAGAIN`
8	`execve`	`const char *path, char *const argv[], char *const envp[]`	nem tér vissza siker esetén	`ENOENT`, `EACCES`, `EFAULT`
9	`waitpid`	`pid_t pid, int *status, int options`	gyermek PID	`ECHILD`, `EFAULT`
10	`yield`	—	0	—
11	`sleep`	`unsigned long ms`	0	`EINVAL`

Ez a készlet elegendő ahhoz, hogy egy felhasználói program elinduljon, konzolon kommunikáljon, dinamikus memóriát foglaljon, és új folyamatokat hozzon létre. A listát a kernel `syscall_table` méretének növelésével és az új függvények regisztrálásával bővíthetjük.

27.4.12 6. Labor: syscall-lista és minimális userland tervezése

Ez a labor a Rendszerhívások és userland tervezése tanulási céljainak gyakorlati ellenőrzésére szolgál. A feladat egy dokumentált syscall-lista elkészítése és egy minimális userland futtatása emulált környezetben.

27.4.13 6.1. Kiinduló állapot és konfiguráció

- **Emulátor:** QEMU, `qemu-system-i386`.
- **CPU:** `qemu32`, védett mód, lapozás engedélyezve.
- **Memória:** 32 MiB fizikai RAM.
- **Konzol:** soros port (`-serial stdio`).
- **Kernel:** a Memóriakezelés és ütemező tervezése fejezetben megtervezett monolitikus kernel, amely már tartalmazza az ütemezőt és a lapkezelést.
- **Mentési pont:** a labor előtt a kernel forrásfáját verziókezelőbe kell menteni, és a build-könyvtárat tiszta állapotból kell létrehozni. A labor nem nyúl a hoszt bootloaderéhez vagy lemezéhez.

27.4.14 6.2. Lépések

1. **Syscall tábla definiálása.** Hozzuk létre a `syscall_table.c` fájlt a fenti táblázat alapján. Minden syscall-hoz írjunk egy váz függvényt, amely egyelőre `-ENOSYS` értékkel tér vissza.
2. **ABI fejléc készítése.** Hozzuk létre a `syscall.h` felhasználói fejlécet, amelyben a syscall számokat `#define` direktívákkal rögzítjük, és deklaráljuk a `syscall()` wrapper függvényt.
3. **Belépési pont és dispatcher.** Írjuk meg az assembly belépőt, amely elmenti a trap frame-et, és ugorjon a `syscall_entry` C függvényre. A `syscall_init` regisztrálja a táblát.
4. **Init program.** Készítsünk egy `init.c` felhasználói programot, amely a beépített `syscall()` wrapper segítségével kiír egy üzenetet, majd beolvas egy parancsot, és a `hello` parancsra válaszol.
5. **Betöltés.** A kernel boot után hozza létre az init folyamatot, állítsa be a veremmutatót, és adja át a vezérlést a user programnak.
6. **Teszt.** Indítsuk el a QEMU-t, és a soros konzolon ellenőrizzük, hogy megjelenik az init üzenet, a `hello` parancs pedig a `hello from userland` választ írja ki.

27.4.15 6.3. Mérési napló

A mérésekhez a felhasználói programban egy ciklusban hívjuk meg a `write` syscall-t, és a `rdtsc` utasítással mérjük a hívás előtti és utáni időbélyeget. A mérési módszert és a hardverkörnyezetet rögzíteni kell.

Várt eredmények:

Művelet	Várt eredmény
Boot idő az init első kiírásáig	kevesebb mint 100 ms
`write` syscall hívás 1 bájt kiírásával	kevesebb mint 500 ciklus
`hello` parancs válaszideje	kevesebb mint 10 ms
Hibás syscall szám (`nr = 99`) hibakód	`-ENOSYS`
Érvénytelen mutató a `write`-ban	`-EFAULT`

Mért eredmények: (a labor során kitöltendő)

Művelet	Mért eredmény
Boot idő	..... ms
`write` syscall ciklusszám	..... ciklus
`hello` válaszidő	..... ms
Hibás syscall szám	.....
Érvénytelen mutató	.....

A mért értékeket a labor jegyzőkönyvében fel kell tüntetni, és el kell különíteni a várt értékektől. A mérésnél ugyanazt a QEMU konfigurációt és ugyanazt a fordítási beállítást kell használni, mint a reprodukció során.

27.4.16 6.4. Ellenőrzési pontok a kódminőséghez

- A dispatcher nem dereferál felhasználói mutatót.
- A `copy_from_user` ellenőrzi a tartományt és a leképezést.
- A `sys_write` a kernel veremben lévő puffert használ, nem a felhasználóit.
- A trap frame mentése az összes hívó által használt regisztert tartalmazza.
- A syscall tábla indexelése a `nr` egészértékű ellenőrzése után történik.
- A felhasználói verem mérete rögzített és a folyamat struktúrában dokumentált.

27.4.17 7. Tipikus hibák és indoklásuk

A fejezetben bemutatott tervezési döntések leggyakoribb hibái és azok következményei:

1. **Közvetlen user pointer dereferálás.** Ha a kernel a felhasználói `buf` címet használja anélkül, hogy `copy_from_user`-t hívna, a rendszer egy érvénytelen vagy kernelbeli címre mutató hivatkozással page fault-ot okozhat kernelmódban. Ez a kernel leállásához vezethet. A helyes megoldás a másolás és a tartományellenőrzés.

2. **A hossz ellenőrzésének elmulasztása a másolás előtt.** Ha a kernel a `len` értéket először a `kbuf` méretével hasonlítja össze, de a `copy_from_user` hívásakor a felhasználói puffer rövidebb, mint `len`, a másolás túlcsordulhat. A `copy_from_user`-nek a `len` minden bájtjára érvényes leképezést kell ellenőriznie.

3. **A TOCTOU figyelmen kívül hagyása.** Ha a kernel először ellenőrzi a puffert, majd később újra a felhasználói memóriát olvassa, a felhasználó a két lépés között megváltoztathatja a memóriát. Ez adatszivárgáshoz vagy jogosultság-megkerüléshez vezethet. A megoldás az, hogy minden felhasználói adatot egyszer, atomi módon másolunk a kernelbe.

4. **Hibakódok helytelen kezelése.** Ha a syscall függvény pozitív `errno` értéket ad vissza, vagy a dispatcher a negatív visszatérést nem alakítja át, a felhasználói program téves eredményt kap. Az ABI-t pontosan dokumentálni kell: a kernel negatív hibakódot ad, a userland wrapper ezt `errno`-vá alakítja.

5. **Nem mentett regiszterek.** Ha az assembly belépő nem menti el az összes regisztert, amelyet a C hívási konvenció a hívóhoz köt, akkor a felhasználói program regiszterei a syscall után sérülhetnek. A trap frame-nek a teljes felhasználói állapotot kell tartalmaznia.

6. **A felhasználói verem helytelen beállítása.** Ha az init program `RSP`-je nem a lefoglalt verem tetejére mutat, vagy a verem túl kicsi, a program az első verem műveletnél összeomlik. A verem méretét és a `RSP` kezdeti értékét a betöltőnek egyszerre kell kezelnie.

27.4.18 8. Összefoglalás

Ebben a fejezetben megterveztük az oktatókernel rendszerhívási interfészét és a minimális felhasználói teret. A következő fogalmakat vezettük be:

- A rendszerhívás a kernel és a userland közötti ellenőrzött, ABI-szintű szerződés.
- A syscall dispatcher egy függvénymutató-tábla segítségével osztja szét a hívásokat, és a trap frame-ben menti a felhasználói állapotot.
- Az argumentumok regiszterekben érkeznek, a mutatókat és puffereket a kernelnek mindig ellenőriznie és másolnia kell.
- A hibakódokat negatív `errno` értékként adjuk vissza, amelyet a userland wrapper alakít át.

- Az init az első userland folyamat, amely egy shellt indíthat; a programbetöltő felel a szegmensek, a verem és a regiszterek inicializálásáért.
- A laborban egy konkrét syscall-listát és egy minimális init programot valósítunk meg, méréssel és hibakezeléssel.

A következő fejezet (C és assembly alapok, fordítás és linkelés) erre az interfészre építve bemutatja a C és assembly nyelvű kernelkód fordítását, linkelését, valamint az eszközök és a megszakításkezelés alapjait. Az itt bevezetett ABI és userland betöltő lesz az alapja a későbbi fájlrendszer- és shellfejlesztéseknek.

27.4.19 Önellenőrző kérdések

1. Miért nem szabad a kernelnek közvetlenül használni a felhasználói mutatókat? Soroljon fel legalább három okot.
2. Írjon egy `read` rendszerhívás vázat, amely a `copy_to_user` segítségével ad vissza adatot a felhasználói pufferbe. Milyen ellenőrzéseket kell elvégezni?
3. Mi a különbség az API és az ABI között? Adjon példát mindkettőre a `write` syscall esetében.
4. Tervezzen egy új syscall-t, amely egy folyamat prioritását állítja be. Milyen jogosultsági ellenőrzésre van szükség? Hogyan dokumentálná a paramétereket és a hibakódokat?

28 Kernel implementáció

28.1 C és assembly alapok, fordítás és linkelés

A kernelépítés első akadálya nem a bonyolult algoritmus, hanem az, hogy a megszokott hosztált C-környezet itt nem létezik: nincs `main`, nincs `printf`, nincs `malloc`, és a linker sem a megszokott módon illeszti a kódot. Ez az alfejezet a freestanding C, az assembly és a linker script kapcsolatát építi fel egy olyan minimális, de teljes kernelképen, amelyet a következő alfejezetekben az interruptkezelés és a kontextusváltás alapjaként használunk.

28.1.1 1. A freestanding C és a hosztált C különbsége

A C-szabvány két végrehajtási környezetet különböztet meg: a *hosted* (hosztolt) és a *freestanding* környezetet. A hosztolt környezetben a program belépési pontja a `main`, a futáshoz rendelkezésre áll a standard könyvtár, és az operációs rendszer szolgáltatásai hívhatók. A kernel esetében ezzel szemben a kód közvetlenül a hardver fölött fut, ezért:

- a belépési pontot mi adjuk meg, tipikusan `_start` vagy `kernel_main`;
- a standard C-függvények (`memcpy`, `memset`, `strlen`, `printf`) nincsenek jelen;
- a kód nem hívhat operációsrendszer-szolgáltatást, hiszen maga az operációs rendszer;
- a globális változók inicializálását és a BSS nullázását is a kernelnek kell elvégeznie.

A GCC-ben a freestanding módot a `-ffreestanding` kapcsoló jelzi. Ez azt mondja a fordítónak, hogy ne feltételezze a hosztolt környezetet, és ne használjon olyan beépített optimalizációkat, amelyek hosztolt környezethez kötöttek. A gyakorlatban a `-ffreestanding` önmagában még kevés; a kernel fordítása során a következő kapcsolókat szoktuk együtt használni:

Kapcsoló	Hatás	Indoklás kernelnél
`-ffreestanding`	freestanding környezet feltételezése	nincs standard könyvtár és hoszt OS
`-fno-stack-protector`	kikapcsolja a stack-védő kanárikat	a kanárik és a hozzájuk tartozó `__stack_chk_fail` hosztolt támogatást igényelnek
`-fno-pic` / `-fno-pie`	pozíciófüggetlen kód kikapcsolása	a kernel fix címre van linkelve, nem dynamic linker tölti be
`-mno-red-zone` (x86-64)	letiltja a vörös zónát	megszakítás vagy kivétel esetén a CPU nem használhatja a stack alatti területet
`-mno-mmx -mno-sse` vagy `-mgeneral-regs-only`	tiltja az MMX/SSE regiszterek használatát	a kernel korai szakaszában a kiterjesztett regiszter-állapotot még nem mentettük

Kapcsoló	Hatás	Indoklás kernelnél
`-mcmodel=kernel` (x86-64)	kernelmemória-modellt használ	x86-64 kernelnél a kód a nagy negatív címekre hivatkozik
`-nostdlib`	nem linkeli a standard C-futtatókörnyezetet	nem létezik használható libc
`-O2` vagy `-Os`	optimalizálás	a kernel teljesítménye és mérete fontos, de a hibakeresésnél először érdemes `-O0 -g`

A freestanding C-ben az alap típusokhoz továbbra is használhatók a GCC által biztosított fejlécek, például `<stdint.h>`, `<stddef.h>`, `<stdbool.h>` és `<limits.h>`, mert ezek nem igénylik az operációs rendszert. A stringkezelő függvények, a `malloc` és a `printf` azonban nem állnak rendelkezésre.

28.1.2 2. Cross-compiler és build-lánc

A kernel fordításához nem elegendő a hoszt rendszer C-fordítója. A hoszt `gcc` ugyanis a hoszt operációs rendszer ABI-jához, libc-jéhez és betöltőjéhez tartozó kódot generál, például Linux alatt általában PIE-t, stack-védelmet és libc-hívásokat. A kernelhez *cross-compilerre* van szükség, amelynek célplatformja a célszámítógép, például:

- `i686-elf-gcc` 32 bites x86-hoz,
- `x86_64-elf-gcc` 64 bites x86-hoz,
- `aarch64-elf-gcc` 64 bites ARM-hoz,
- `riscv64-elf-gcc` RISC-V-hez.

A target hármas ellenőrizhető:

```
i686-elf-gcc -dumpmachine
# i686-elf
```

A „-elf” végződés jelzi, hogy a toolchain nem kötődik konkrét hoszt operációs rendszerhez. Az ilyen toolchain a binutils-szal együtt értendő, tehát a `i686-elf-as`, `i686-elf-ld`, `i686-elf-objdump`, `i686-elf-readelf` és `i686-elf-nm` eszközök is a célplatformot ismerik. A fejezet laborjában 32 bites x86-ot használunk, mert a betöltő és a belépő assembly így kisebb, a calling convention pedig jól szemlélteti a stack-alapú paraméterátadást. A 64 bites x86-64 SysV ABI-t külön ismertetjük.

A build-láncban a C-fordító a forrást tárgykódra fordítja, az assembler az assemblyt tárgykódra fordítja, a linker pedig a tárgykódokat a linker script szerint végleges kernelképpé illeszti. A linker meghívásához használhatjuk közvetlenül a GCC-t is, mert így a `-lgcc` automatikusan elérhető. A libgcc egyes beépített műveleteket (például 64 bites osztást 32 bites gépen) tartalmaz, ezért freestanding kernelnél is gyakran be kell linkelni.

A pontos kapcsolók és a toolchain-verziók ellenőrzési igényűek. A leírt parancsok az írás pillanatában szokásos GCC/binutils viselkedést tükrözik; más verzió esetén a `--target-help` és az adott toolchain dokumentációja az irányadó. A labor jegyzőkönyvben a verziót mindig rögzíteni kell.

28.1.3 3. Assembly és C összekötése

A kernelben két irányú hívás van. Az indító assembly felállítja a stacket, majd meghívja a C `kernel_main` függvényt. Később a C kód assembly függvényeket hív, például I/O-port- vagy speciális regiszter-művelethez. A két oldalt a linker szimbólumai kötik össze, ezért a neveknek pontosan egyezniük kell.

A C nyelvben nincs névtorzítás, így egy C `kernel_main` függvény az assemblyből egyszerűen `kernel_main` néven hívható. Az assembly oldalon a szimbólumot globálissá kell tenni:

```
.global _start
.global uart_putc
```

A C oldalon pedig a függvényt ugyanolyan néven deklaráljuk:

```
extern void uart_putc(char c);
```

A nincs névtorzítás csak C esetén igaz. C++ kernelkód esetén `extern "C"` blokk kell, különben a linker nem találja a függvényt. Ez a könyv példáiban azért nem releváns, mert a kernel C-ben készül.

A GAS szintaxisban az utasítások operandus-sorrendje forrás-cél (AT&T-szintaxis). Például:

```
movl $0x00100000, %eax   /* az azonnali értéket az EAX-be tölti */
movl %eax, %ebx          /* az EAX tartalmát az EBX-be másolja */
```

A következő 32 bites assembly függvény három 32 bites egészet ad össze, és C-ből hívható:

```
# asm_ops.S - 32 bites példa
.section .text
.global asm_add3
.type asm_add3, @function
asm_add3:
    movl 4(%esp), %eax   # első paraméter
    addl 8(%esp), %eax   # második paraméter
    addl 12(%esp), %eax  # harmadik paraméter
    ret
.size asm_add3, . - asm_add3
```

A hozzá tartozó C deklaráció:

```
extern uint32_t asm_add3(uint32_t a, uint32_t b, uint32_t c);
```

Ez a példa az i386 cdecl hívási konvenciót mutatja: a paraméterek jobbról balra a stackre kerülnek, a visszatérési érték az EAX-ben van, és a hívó takarítja a stacket.

Ugyanez x86-64 SysV alatt regiszterekkel történik:

```
# asm_ops_64.S - x86-64 System V példa
.section .text
.global asm_add3
.type asm_add3, @function
asm_add3:
    leaq (%rdi, %rsi), %rax   # rax = rdi + rsi
    addq %rdx, %rax           # rax += rdx
    ret
.size asm_add3, . - asm_add3
```

C oldalról:

```
extern uint64_t asm_add3(uint64_t a, uint64_t b, uint64_t c);
```

Amikor assembly hív C-függvényt, ugyanezt a konvenciót kell követni. A laborban használt 32 bites belépő ezért előbb a stackre teszi a Multiboot-bootloader két regiszterét, majd `call` utasítással hívja a C függvényt. A C függvény visszatérése után a hívónak kell visszaállítania a stacket.

28.1.4 4. ABI, stack és calling convention

A CPU nem ismeri a C függvényhívásokat; a hívás és visszatérés csak az utasításkészlet és a regiszterek szintjén létezik. Az, hogy a függvények hol kapják a paramétereiket, mely regisztereket kell megőrizni, és hogyan kell visszatérni, az ABI-ban rögzített szerződés. A kernelben ugyanazt az ABI-t kell követni, mint a userlandben, mert a C-fordító is ezt a szerződést kódolja.

28.1.5 4.1. i386 cdecl összefoglaló

A 32 bites x86 cdecl konvencióban:

- a paraméterek jobbról balra a stackre kerülnek;
- az első paraméter a `%esp` fölött közvetlenül a visszatérési cím után található;
- a visszatérési érték az `%eax`-ben van;
- a hívó takarítja a stacket a hívás után;
- a callee-saved regiszterek: `%ebx`, `%esi`, `%edi`, `%ebp`;
- a caller-saved regiszterek: `%eax`, `%ecx`, `%edx`;
- a stackpointernek a hívás előtt 16 bájtra igazítottnak kell lennie.

A callee-saved regisztereket a függvénynek meg kell őriznie; a caller-saved regisztereket a hívó nem feltételezheti a hívás után. A stack igazítása azért fontos, mert a fordító bizonyos SSE-utasításokat csak 16 bájtra igazított címen tud végrehajtani. A kernel stackjét ezért a belépő assemblyben explicit módon 16 bájtra igazítjuk.

28.1.6 4.2. x86-64 System V összefoglaló

A 64 bites x86 System V ABI-ban a paraméterátadás regiszterekkel történik, amíg van elég regiszter:

Paraméter sorszáma	Regiszter
1.	`%rdi`
2.	`%rsi`
3.	`%rdx`
4.	`%rcx`
5.	`%r8`
6.	`%r9`
7. és további	stacken

A visszatérési érték `%rax`-ben van. A callee-saved regiszterek: `%rbp`, `%rbx`, `%r12`–`%r15`. A caller-saved regiszterek: `%rax`, `%rdi`, `%rsi`, `%rdx`, `%rcx`, `%r8`–`%r11`. A `call` utasítás előtt a stackpointer 16 bájtra igazított; a függvény belépésekor a `%rsp % 16 == 8`, mert a `call` a visszatérési címet a veremre tette. A leaf-függvények használhatják a vörös zónát, vagyis a `%rsp` alatti 128 bájtot, de kernelben ezt a `-mno-red-zone` letiltja, mert egy megszakítás ugyanazt a stackterületet írhatja.

28.1.7 4.3. Stack frame

A C-fordító a nem triviális függvényekhez stack frame-et készít. Egy tipikus 64 bites prológus:

```
pushq %rbp          # az előző frame-pointer mentése
movq  %rsp, %rbp    # új frame-pointer
subq  $N, %rsp      # lokális terület foglalása
```

Az epilógus:

```
leave               # movq %rbp, %rsp; popq %rbp
ret
```

A frame-pointer segítségével a hibakereső vissza tudja fejteni a hívási láncot. A kernelben a `-fno-omit-frame-pointer` hasznos a fejlesztés alatt, mert a QEMU/GDB alatti stack-visszafejtés egyszerűbb. A végleges, optimalizált buildben a frame-pointer elhagyható, de a méréseknél fel kell tüntetni.

28.1.8 5. Linker script és szimbólumok

A linker alapértelmezés szerint a hoszt platform betöltőjének megfelelő elrendezést készítene. A kernelnek azonban saját memóriatérképe van, ezért *linker scriptet* kell adnunk. A linker script definiálja:

- a belépési pontot (`ENTRY`);
- a kimeneti formátumot és architektúrát;
- a szekciók sorrendjét és címét;
- a további szimbólumokat a kernelkód számára.

A következő példa 32 bites x86 kernelhez készült. A kernel belépési címét `0x00100000`-ra (1 MiB) tesszük, mert ez a tipikus alsó betöltési cím a PC-s környezetben. A szekciókat 4 KiB laphatárokra igazítjuk, így a későbbi lapozás egyszerűbb lesz.

```
/* linker.ld - 32 bites x86 kernel memóriatérkép */
ENTRY(_start)
OUTPUT_FORMAT(elf32-i386)
OUTPUT_ARCH(i386)

SECTIONS
{
    . = 0x00100000;

    .text : ALIGN(4K)
    {
        KEEP(*(.multiboot))
        *(.text*)
    }

    .rodata : ALIGN(4K)
    {
        *(.rodata*)
    }

    .data : ALIGN(4K)
    {
        *(.data*)
    }

    .bss : ALIGN(4K)
```

```
    {
        __bss_start = .;
        *(.bss*)
        *(COMMON)
        __bss_end = .;
    }

    __kernel_end = .;
}
```

A `. = 0x00100000;` utasítás a linkercím-számlálót állítja a megadott címre. Az ezt követő szekciók egymás után, a megadott igazítással kerülnek a kernelképbe. A `KEEP(*(.multiboot))` azért szükséges, mert a Multiboot fejlécet a kernel normál kódja nem hivatkozza, így a linker garbage collection nélkül is hajlamos lenne eldobni. A `KEEP` megakadályozza ezt.

A BSS-szekcióban két fontos szimbólumot hozunk létre:

```
__bss_start = .;
__bss_end = .;
```

A linker ezekhez a szimbólumokhoz a BSS kezdetének és végének *címét* rendeli. A C kód a következő módon tudja letörölni a BSS-t:

```
extern uint8_t __bss_start[];
extern uint8_t __bss_end[];

static void clear_bss(void) {
    for (uint8_t *p = __bss_start; p < __bss_end; ++p) {
        *p = 0;
    }
}
```

Fontos, hogy a deklaráció `uint8_t __bss_start[]`, nem pedig `extern void *__bss_start;`. A `void *` deklaráció esetén a kód a szimbólum *memóriatartalmát* olvasná, mintha a címen egy pointer lenne, pedig ott a BSS adatterülete van. Ez az egyik leggyakoribb hiba.

A `__kernel_end` szimbólum a kernel image végét mutatja, és később a memóriafoglalónak adja meg az első szabad területet.

A linker által készített ELF-t a következő eszközökkel ellenőrizhetjük:

```
readelf -h kernel.elf | grep 'Entry point'
readelf -S kernel.elf
nm -n kernel.elf
```

Az `nm -n` cím szerint növekvő sorrendben listázza a szimbólumokat. A laborban az `_start`, a `kernel_main`, valamint a `__bss_start` és `__bss_end` szimbólumok ellenőrzése kötelező.

28.1.9 6. A labor kernel buildje

A labor egy futtatható, ugyanakkor szándékosan kicsi kernel. A cél nem egy teljes operációs rendszer, hanem annak bizonyítása, hogy a freestanding C, az assembly és a linker script együttműködik. A kernel a soros portra ír egy állapotüzenetet, majd `hlt` ciklusba kerül. Az alábbi fájlok és parancsok csak emulált környezetben, eldobható munkakönyvtárban futtatandók; a hoszt lemezét, bootloaderét és rendszerpartícióját nem módosítják.

28.1.10 6.1. `boot.S`

Az indító assembly a Multiboot fejléctől indul, felállítja a stacket, és a Multiboot-bootloader két regiszterét átadja a C `kernel_main` függvénynek.

```
# boot.S - 32 bites Multiboot belépő, szemléltető példa
.set MB_MAGIC, 0x1BADB002
.set MB_FLAGS, 0x00000000
.set MB_CHECKSUM, -(MB_MAGIC + MB_FLAGS)

.section .multiboot
.align 4
.long MB_MAGIC
.long MB_FLAGS
.long MB_CHECKSUM

.section .bss
.align 16
stack_bottom:
.skip 16384            # 16 KiB kernelstack
stack_top:

.section .text
.global _start
.code32
_start:
    movl $stack_top, %esp      # saját, 16 bájtra igazított stack
    pushl %ebx                 # Multiboot info-pointer
    pushl %eax                 # Multiboot magic
    call kernel_main
    addl $8, %esp
.Lhalt:
    cli
    hlt
    jmp .Lhalt
```

A `_start` a Multiboot specifikáció szerint 32 bites védett módban, lapozás nélkül indul. A bootloader az `%eax`-ben a `0x2BADB002` mágikus értéket, az `%ebx`-ben a Multiboot információs struktúra mutatóját adja át. A kernel a saját stackjét a BSS-ben lefoglalt 16 KiB-os területre teszi. A `pushl %ebx` és `pushl %eax` a cdecl hívási konvenció szerint rakja a paramétereket a stackre.

28.1.11 6.2. `kernel.c`

A C kód a BSS letörlése után a soros portra ír. A soros port az `0x3F8` COM1-címre van leképezve, ahol a TX-regiszter akkor írható, ha a line status regiszter 5. bitje 1. A port-hozzáférés `inb`/`outb` inline assemblyvel történik.

```
/* kernel.c - freestanding C kernel, szemléltető példa */
#include <stdint.h>
#include <stddef.h>

#define COM1 0x3F8

extern uint8_t __bss_start[];
extern uint8_t __bss_end[];

static inline uint8_t inb(uint16_t port) {
    uint8_t v;
    asm volatile("inb %1, %0" : "=a"(v) : "d"(port));
```

```
    return v;
}

static inline void outb(uint16_t port, uint8_t value) {
    asm volatile("outb %0, %1" : : "a"(value), "d"(port));
}

static void uart_putc(char c) {
    while ((inb(COM1 + 5) & 0x20) == 0) {
        /* várakozás, amíg a TX-üres bit be nem áll */
    }
    outb(COM1, (uint8_t)c);
}

static void uart_puts(const char *s) {
    for (; *s != '\0'; ++s) {
        uart_putc(*s);
    }
}

static void clear_bss(void) {
    for (uint8_t *p = __bss_start; p < __bss_end; ++p) {
        *p = 0;
    }
}

void kernel_main(uint32_t magic, uint32_t mb_info) {
    (void)magic;
    (void)mb_info;

    clear_bss();
    uart_puts("kernel: OK\r\n");

    for (;;) {
        asm volatile("hlt");
    }
}
```

A `clear_bss` hívásnak a globális változók első használata előtt kell megtörténnie. Ebben a rövid példában nincs más globális változó, de a laborbővítésnél elvárás, hogy a nullázás megelőzze azok használatát.

28.1.12 6.3. `linker.ld`

A fent bemutatott linker script használható változtatás nélkül.

28.1.13 6.4. `Makefile`

A build a cross-compilerrel és a linker scripttel készül. A linkeléshez a GCC-t hívjuk, hogy a libgcc automatikusan elérhető legyen.

```
# Makefile - a kernel buildje
CC=i686-elf-gcc
AS=i686-elf-as

CFLAGS=-ffreestanding -fno-stack-protector -fno-pic -nostdlib \
       -std=c11 -Wall -Wextra -O2 -c
ASFLAGS=--32
```

```
kernel.elf: boot.o kernel.o linker.ld
    $(CC) -Wl,-T,linker.ld -nostdlib -ffreestanding \
        -Wl,--build-id=none -o $@ boot.o kernel.o -lgcc

boot.o: boot.S
    $(AS) $(ASFLAGS) -o $@ $<

kernel.o: kernel.c
    $(CC) $(CFLAGS) -o $@ $<

clean:
    rm -f boot.o kernel.o kernel.elf
```

A `-Wl,--build-id=none` elkerüli a felesleges build-azonosító szekciót. A `-lgcc` a toolchain libgcc-jét linkeli, ami a freestanding kódban is engedélyezett.

28.1.14 6.5. Build és ellenőrzés

A munkakönyvtár létrehozása után a build parancs:

```
make clean all
```

Sikeres build esetén a `kernel.elf` létrejön. Az ellenőrző parancsok:

```
readelf -h kernel.elf | grep 'Entry point'
readelf -S kernel.elf | grep -E '\.(text|rodata|data|bss)'
nm -n kernel.elf | grep -E '(_start|kernel_main|__bss_start|__bss_end)'
```

A várt belépési pont `0x00100000`. A `.text` szekció kezdőcíme szintén `0x00100000`. A `__bss_start` és `__bss_end` különbsége a BSS mérete, amely legalább a 16 KiB-os stacket tartalmazza.

28.1.15 6.6. Futtatás QEMU-ban

A kernelképet a QEMU beépített Multiboot-támogatásával indítjuk:

```
qemu-system-i386 -kernel kernel.elf \
                 -display none \
                 -serial stdio \
                 -no-reboot \
                 -no-shutdown
```

A `-serial stdio` a COM1 soros portot a terminálra köti. A várt kimenet:

```
kernel: OK
```

A QEMU-ból a `Ctrl-A x` billentyűkombinációval léphetünk ki. A parancs nem ír a hoszt lemezére, és a kernel `hlt` ciklusban vár.

28.1.16 7. Gyakori hibák és megoldásuk

Az első kernelépítéskor a következő hibák a leggyakoribbak.

1. **Hoszt compiler használata.** Ha a `gcc` parancsot használjuk `i686-elf-gcc` helyett, a linker hoszt libc-t, PIE-t és dinamikus betöltőt keres, és olyan hibákat ad, mint `undefined reference to __libc_start_main` vagy `cannot find -lc`. Megoldás: a cross-compiler és a `-nostdlib` következetes használata.
2. **Hiányzó `-fno-stack-protector`.** A GCC hosztolt módban bekapcsolhatja a stack-védelmet, és a kernelkódból `__stack_chk_fail` hivatkozás keletkezik. Megoldás: explicit `-fno-stack-protector`.
3. **A BSS-szimbólum pointerként deklarálása.** Az `extern void *__bss_start;` kód a szimbólum címén lévő értéket értelmezi pointerként. A helyes deklaráció: `extern uint8_t __bss_start[];`.
4. **A Multiboot fejléc hiánya vagy kidobása.** Ha a `.multiboot` szekció nem a kép elején van, vagy a linker script nem tartalmazza a `KEEP(*(.multiboot))` bejegyzést, a QEMU nem ismeri fel a kernelt. Megoldás: a szekció legyen a `.text` elején, és `KEEP` használata kötelező.
5. **Stack-misalignment.** A 32 bites i386 System V ABI szerint a `call` előtt a stackpointer 16 bájtra igazított. Ha az assembly belépő ezt megsérti, a C kódban `movaps` vagy más igazított SSE-utasítás hibát okoz. A labor `stack_top` szimbólum 16 bájtra igazított, és a `_start` ezt írja az `%esp`-be.
6. **I/O-port constraint hiba.** Az `outb` inline assemblyben a `"Nd"` port-kényszer 8 bites azonnali értéket enged meg, ezért a `0x3F8` portra már hibás. A példa a `"d"` kényszert használja, amely a portot a `%edx`-be teszi, és bármely 16 bites portcímet elfogad.
7. **A stack túl kicsi.** A kernel stackjét nem foglalja le a linker script a BSS-ben, vagy a `stack_top` a BSS-szekción kívülre mutat. A labor 16 KiB-ot foglal le; nagyobb megszakításlánc esetén ennél nagyobb, jellemzően 32–64 KiB szükséges.

28.1.17 8. Labor: kernel build és linker script

A labor célja a fenti eszközök reprodukálható összekapcsolása. Az alábbiakban a minimálisan elvárt jegyzőkönyv szerepel.

Kiinduló állapot: Linux-hoszt, `qemu-system-i386`, `i686-elf-gcc` és binutils. A munkakönyvtár `oslab/os106`.

Lépések:

1. Hozd létre a fenti `boot.S`, `kernel.c`, `linker.ld` és `Makefile` fájlokat.
2. Futtasd a buildet: `make clean all`.
3. Ellenőrizd a szimbólumokat: `nm -n kernel.elf`.
4. Indítsd el a kernelt a megadott QEMU-paranccsal, majd rögzítsd a `kernel: OK` kimenetet.
5. Tisztítsd meg a munkakönyvtárat: `make clean`.

Mérési napló és elvárt/ mért elkülönítés:

Jellemző	Várt (levezetett) érték	Mért érték	Értékelés
Belépési pont	`0x00100000`	`readelf` kimenet	egyezés

Jellemző	Várt (levezetett) érték	Mért érték	Értékelés
`.text` kezdőcím	`0x00100000`	`readelf -S` kimenet	egyezés
`__bss_start` – `__bss_end`	legalább 16 KiB	`nm` szerinti különbség	egyezés
`.text` méret	példában < 2 KiB	`size kernel.elf`	jellemzően egyezés
Boot kimenet	`kernel: OK`	terminál kimenet	egyezés
Buildidő	< 5 s (becslés, nem mérés)	`time make`	csak rögzítendő

A mért értékeket a jegyzőkönyvben mindig külön oszlopban kell feltüntetni, még akkor is, ha a várt értékbecslés. A toolchain verzióját (`i686-elf-gcc --version`) és a QEMU verzióját ugyancsak rögzíteni kell.

Visszaállítás: a QEMU-ból `Ctrl-A x`-szel lépünk ki, a munkakönyvtárat `make clean` parancs tisztítja, a hoszt rendszer állapota nem változik.

28.1.18 9. Összefoglalás és továbbvitel

Ebben az alfejezetben megépítettük a kernel fordítási és linkelési alapjait. A freestanding C fordítási kapcsolói megszüntetik a hosztolt környezet függőségeit; az assembly belépő felállítja a stacket és átadja a bootloader paramétereit; a linker script meghatározza a kernel memóriatérképét, a BSS-t pedig a C kód nullázza a linker által exportált szimbólumok segítségével. A calling convention ismerete nélkül az assembly-C határon véletlenszerű hibák keletkeznek, ezért a regiszterek, a stack-igazítás és a callee/caller-saved szabályok a kernel minden részében ugyanazt a szerződést jelentik.

A következő alfejezet (Interruptkezelés, időzítő és kontextusváltás) erre a buildre épít, amikor az interruptkezelést, az időzítőt és a kontextusváltást vezeti be. Az itt elkészített `kernel.elf`, a linker script és a QEMU indítási mód lesz a további munka alapja.

Önellenőrző kérdés: Miért hibás a következő deklaráció a linker script által létrehozott BSS-szimbólumhoz, és mi a helyes megoldás?

```
extern void *__bss_start;
uint8_t *p = __bss_start + 1;
```

Válasz: A `void *` deklaráció miatt a kód a `__bss_start` címén lévő memóriatartalmat pointerként értelmezi, nem pedig magát a címet. A BSS elején nincs pointer, hanem a BSS adata, ezért a lépés helytelen. A helyes deklaráció `extern uint8_t __bss_start[];`, így a `__bss_start` tömbnév a BSS kezdőcímére mutat.

28.2 Interruptkezelés, időzítő és kontextusváltás

28.2.1 7.7 Megszakításkezelés, időzítő és kontextusváltás

Az előző egységben (C és assembly alapok, fordítás és linkelés) felépítettük a kernelt C és assembly forráskódokból, és sikerült a bootolást egy minimális függvényhívásig eljuttatni. Ez a futtatható kép azonban még passzív: a CPU utasításról utasításra hajtja végre a kódot, de nem reagál a külvilág eseményeire, nem képes több folyamatot időosztásosan futtatni, és nem tudja megszakítani a felhasználói kódot. Ebben az alfejezetben a kernel “életre keltésének” három

alappillérét építjük fel: a megszakítások (interrupt) tárgyalását, a rendszer időzítőjének (timer) bevezetését, valamint a folyamatok közötti átkapcsolást lehetővé tevő kontextusváltást (context switch).

Az itt bemutatott kód és viselkedés szorosan kapcsolódik az általános OS-elmélethez, de a megvalósítás szintjén az x86-64 architektúrát és az ahhoz kapcsolódó szabványos periféria-illesztőket (PIC, PIT) használjuk. Fontos megjegyezni, hogy ezek a hardverkomponensek a modern rendszerekben már javarészt újabb interfészeknek adták át a helyüket (APIC, HPET), de az emulált QEMU környezetben kiválóan alkalmasak az alapelvek tanítására, mivel programozásuk egyszerű, determinisztikus, és nem igényel bonyolult specifikációs réteget.

28.2.1.1 A megszakítások és az időzítő szerepe

Az operációs rendszer kernelének alapvető működési módja az eseményvezérelt (event-driven) modell. A CPU a felhasználói program végrehajtása vagy éppen az idle hurok közben külső eseményekről kell, hogy értesüljön. Ilyen esemény lehet egy billentyűleütés, egy hálózati csomag érkezése, egy lemez I/O befejezése, vagy éppen egy hibás memóriacím elérése.

A hardver és a CPU közötti kommunikáció két alapvető módja a **polling** és a **megszakítás (interrupt)**. Polling esetén a kernel folyamatosan, aktívan kérdezgeti az eszközök állapotregisztereit, hogy történt-e esemény. Ez a megoldás rendkívül pazarló: a CPU feleslegesen pörög, és a késleltetési idő függ a lekérdezés gyakoriságától. A megszakítás ezzel szemben egy hardveres jelzés: az eszköz (vagy a CPU maga egy kivétel esetén) félbeszakítja a futó utasításfolyamot, és a vezérlést a kernel egy előre meghatározott belépési pontjára adja.

A megszakítások szerepe három fő pontban foglalható össze:

1. **Reaktivitás:** A kernelnek azonnal tudnia kell reagálni a hardvereseményekre anélkül, hogy a futó felhasználói kódot együttműködésre kényszerítené.
2. **Védelem és hibakezelés:** A kivételek (pl. nullával való osztás, érvénytelen cím elérése) szintén megszakítási mechanizmuson keresztül jutnak a kernel tudomására. Ez teszi lehetővé a folyamatok izolációját.
3. **Preemptív ütemezés:** A kernel nem bízhat abban, hogy egy felhasználói program önként visszaadja a CPU-t. Az időzítő megszakítás az a "szívverés", amely rendszeres időközönként visszaadja a vezérlést a kernel ütemezőjének.

Az **időzítő (timer)** tehát a megszakítási rendszer egyik legkritikusabb eleme. Az időzítő egy programozható oszcillátor (pl. az Intel 8253/8254 Programmable Interval Timer, PIT), amelyet úgy állítunk be, hogy periodikusan, például $f = 100$ Hz-es gyakorisággal megszakítást generáljon. Ennek periódusideje $T_{tick} = \frac{1}{f} = 10{,}000$ μs. A kernel ezt a megszakítást "tick"-ként tartja nyilván. A tick szerepe kettős: egyrészt frissíti a rendszer belső óráját (uptime, jiffies), másrészt meghívja az ütemezőt (scheduler), amely eldönti, hogy az éppen futó folyamat lejárt-e már a rendelkezésére álló időszeletéből (time slice), és szükség van-e kontextusváltásra.

28.2.1.2 Az IDT és a megszakítási útvonal

Ahhoz, hogy a CPU tudja, hová ugorjon, ha egy megszakítás érkezik, egy speciális adatszerkezetre van szükség: az **Interrupt Descriptor Table (IDT)**. Az IDT nem más, mint egy olyan bejegyzésekből (gate) álló tömb, amely a CPU által ismert megszakítási vektorokhoz (0-255) rendel belépési pontokat.

A Linux és a legtöbb modern x86-64 kernel a **0x20**-as vektor felett használja a szoftveres megszakításokat, mivel a 0-31 közötti vektorok a CPU kivételei számára vannak fenntartva (pl.

0: Division Error, 13: General Protection Fault, 14: Page Fault). A hardveres megszakításokat (IRQ-k) a régi 8259A PIC (Programmable Interrupt Controller) alapból a 0x08-0x0F vektorokra irányítaná, ami ütközik a CPU kivételeivel. Ezért a PIC-et **remapeljük**, vagyis áthelyezzük a hardveres megszakításokat a 0x20-0x2F tartományba.

Egy IDT bejegyzés struktúrája C-ben a következőképpen írható le (a gép szóhosszának megfelelően x86-64 módban 16 bájt):

```
struct idt_entry {
    uint16_t base_low;    // A handler címének alsó 16 bitje
    uint16_t selector;    // Kernel kód szegmens szelektor (0x08)
    uint8_t  ist;         // Interrupt Stack Table index (itt 0)
    uint8_t  type_attr;   // 0x8E: Present, DPL=0, 32/64-bit Interrupt Gate
    uint16_t base_mid;    // A handler címének középső 16 bitje
    uint32_t base_high;   // A handler címének felső 32 bitje
    uint32_t zero;        // Fenntartva, kötelezően 0
} __attribute__((packed));
```

A kernel inicializálásakor (`idt_init`) a NULL pointerrel való hibás hívásra fenn kell tartanunk a 0. vektort, be kell töltenünk a CPU kivételeit kezelő rutinokat, majd a hardveres megszakításokra a megfelelő ISR-eket (Interrupt Service Routine). Az IDT eléréséhez használt mutatót az `LIDT` utasítással kell betölteni:

```
struct idt_ptr {
    uint16_t limit;
    uint64_t base;
} __attribute__((packed));

void idt_load(struct idt_ptr*);
```

```
; idt_load (assembly)
; Betölti az IDT mutatót az IDTR regiszterbe
global idt_load
idt_load:
    lidt [rdi]   ; RDI tartalmazza az idt_ptr* mutatót
    ret
```

A kernel inicializálásakor a `load_idt` meghívása után a CPU már képes megszakítást fogadni, bár a `STI` utasításig letiltott állapotban van. A `STI` (Set Interrupt Flag) engedélyezi a hardveres megszakítások fogadását.

28.2.1.3 ISR-ek és a megszakítási kód

A megszakítási vektorhoz tartozó függvényben assembly szinten kell elvégezni a "regisztermentést". A C ABI (Application Binary Interface) szerint egy C függvény a regiszterek egy részét (caller-saved: RAX, RCX, RDX, RSI, RDI, R8-R11) szabadon módosíthatja. Ha a megszakítás egy felhasználói program futása közben érkezik, a megszakításkezelőből visszatérve a felhasználói program állapotát bitre pontosan ugyanabban az állapotban kell helyreállítani, mintha mi sem történt volna, kivéve, ha a kernel kontextusváltást tervez.

Ehhez egy **interrupt stub**-ot (csonkot) írunk assemblyben, amely elmenti a CPU regisztereit a veremre, meghívja a magas szintű C nyelvű kezelőt, majd visszatérés előtt visszaállítja a regisztereket és az `IRETQ` utasítással visszatér a megszakított kontextusba.

A hardveres megszakítások kezelésére az x86-64-ben a következő mintát alkalmazzuk:

```
; Megszakítási csonk (ISR stub)
%macro ISR_NOERRCODE 1
global isr_stub_%1
isr_stub_%1:
    push 0          ; Hamis hibakód a konzisztens veremképhez
    push %1         ; Megszakítási vektor száma
    jmp isr_common
%endmacro

%macro ISR_ERRCODE 1
global isr_stub_%1
isr_stub_%1:
    ; A CPU automatikusan tolja a hibakódot a verembe
    push %1         ; Megszakítási vektor száma
    jmp isr_common
%endmacro

; IRQ-k a remapelt vektorokon (0x20-0x2F)
ISR_NOERRCODE 32
ISR_NOERRCODE 33
; ... és így tovább ...
```

A közös belépési rutin (`isr_common`) menti a C ABI megőrzéséhez szükséges regisztereket. A legkritikusabb lépések: az RSP igazítása és a kernel stack használata. Amikor egy felhasználói folyamat fut, a CPU az RSP-t a felhasználói veremre állítva hívja meg a megszakítást. A kernelnek gondoskodnia kell arról, hogy a kernel adatai ne a felhasználói veremre kerüljenek. Ezt a feladatot a **TSS (Task State Segment)** `RSP0` mezője oldja meg: amikor a CPU alacsonyabb privilégiumszintről (ring 3) magasabbra (ring 0) vált, automatikusan betölti a kernel veremmutatót az `RSP0` mezőből. Ha már kernel módban vagyunk, a verem marad az aktuális kernel verem.

A `isr_common` lényege:

```
isr_common:
    ; C ABI szerinti callee-saved regiszterek mentése
    push rax
    push rcx
    push rdx
    push rsi
    push rdi
    push r8
    push r9
    push r10
    push r11

    ; Hívjuk a C nyelvű diszpécsert
    mov rdi, rsp        ; RDI = pointer a veremkeretre (pt_regs)
    call interrupt_handler

    ; Visszatérés a megszakításból
    pop r11
    pop r10
    pop r9
    pop r8
    pop rdi
    pop rsi
    pop rdx
    pop rcx
    pop rax

    ; Eltávolítjuk a vektor számát és a hibakódot
    add rsp, 16
    iretq
```

A C nyelvű `interrupt_handler` megkapja a veremkeretre mutató pointert. Ebből ki tudja olvasni a megszakítás számát, és el tudja dönteni, hogy kivételt (pl. laphiba) vagy hardveres megszakítást kapott-e.

```
struct pt_regs {
    uint64_t r11, r10, r9, r8;
    uint64_t rdi, rsi, rdx, rcx, rax;
    uint64_t int_no;      // Megszakítás vektora
    uint64_t err_code;    // Hibakód (ha van)
    uint64_t rip;         // Megszakított utasítás címe
    uint64_t cs;          // Kód szegmens
    uint64_t rflags;      // CPU flag-ek
    uint64_t rsp;         // Veremmutató a megszakítás pillanatában
    uint64_t ss;          // Verem szegmens
};

void interrupt_handler(struct pt_regs *regs) {
    if (regs->int_no == 14) {
        // Laphiba kezelése (lásd későbbi memória fejezet)
        // ...
        return;
    }
    if (regs->int_no >= 32 && regs->int_no <= 47) {
        // Hardveres megszakítás (IRQ 0-15)
        irq_handler(regs->int_no - 32);
    }
}
```

28.2.1.4 A PIT és a Timer Tick

A kontextusváltás rendszeres kiváltásához szükségünk van egy időforrásra. A QEMU-ban a legegyszerűbb a **PIT (Programmable Interval Timer)** használata. A PIT három csatornával rendelkezik, amelyek közül a 0. csatorna van a rendszer-időzítőhöz kötve. Az alap órajele 1,193,182 Hz.

Ahhoz, hogy 100 Hz-es megszakítást kapjunk, ki kell számolnunk az osztó értékét:

$$D = \frac{1{,}193{,}182}{100} = 11{,}931{,}82 \approx 11{,}932$$

A valóságban, mivel a PIT csak egész osztót fogad el, a tényleges frekvencia $f_{real} = \frac{1{,}193{,}182}{11{,}932} \approx 100{,}0007$ Hz lesz, ami elhanyagolható eltérés. A beállításhoz a 0x43-as portra írjuk a vezérlő bájtot (`0x36`), majd a 0x40-es portra két lépésben az osztó alsó és felső bájtját.

```
#define PIT_CHANNEL0 0x40
#define PIT_CMD      0x43

void timer_init(uint32_t frequency_hz) {
    uint32_t divisor = 1193182 / frequency_hz;

    // 0x36: 0011 0110
    // Csatorna 0, mindkét bájt kiírása, 3-as mód (négyszögjel)
    outb(PIT_CMD, 0x36);
    outb(PIT_CHANNEL0, (uint8_t)(divisor & 0xFF));
    outb(PIT_CHANNEL0, (uint8_t)((divisor >> 8) & 0xFF));
}
```

A rendszer futása során a kernel egy `timer_ticks` nevű globális változót növel. Az `irq_handler`-ben megvizsgáljuk az IRQ számát. Ha az 0 (azaz a remapelt 32-es vektor), akkor a PIT megszakadt.

```c
volatile uint64_t timer_ticks = 0;

void irq_handler(unsigned int irq) {
    if (irq == 0) {
        // PIT tick
        timer_ticks++;
        // Itt hívjuk meg az ütemezőt (ld. kontextusváltás)
        scheduler_tick();
    }
    // EOI (End of Interrupt) elküldése a PIC-nek
    if (irq >= 8) {
        outb(0xA0, 0x20); // Slave PIC EOI
    }
    outb(0x20, 0x20); // Master PIC EOI
}
```

Kritikus hibaforrás: Ha az EOI (End of Interrupt) jelzést elfelejtjük elküldeni a PIC-nek, az nem fog több megszakítást generálni, így az időzítő megáll, és a rendszer lefagy. Ez az egyik leggyakoribb hiba a kezdő kernelhackelés során.

28.2.1.5 A kontextusváltás mechanizmusa

A kontextusváltás (context switch) az a folyamat, amikor a kernel a CPU vezérlését az egyik folyamatból (P_1) átadja egy másiknak (P_2). Ez a legbonyolultabb és legalacsonyabb szintű művelet a kernelben, mivel közvetlenül a CPU regisztereivel és a veremmel dolgozik.

Elméletben a kontextusváltás a következő lépésekből áll: 1. **Belépés a kernelbe:** Egy megszakítás (általában a timer tick) vagy egy rendszerhívás hatására a CPU kernel módba vált, és a folyamat kernel vermére ugrik. 2. **Állapotmentés:** A kernel elmenti az aktuális folyamat (P_1) architekturális állapotát (regiszterek, utasításmutató, veremmutató) a folyamat leíró táblájába (PCB). 3. **Döntés:** Az ütemező kiválasztja a következő futó folyamatot (P_2). 4. **Állapotvisszaállítás:** A kernel betölti P_2 PCB-jéből a korábban elmentett architekturális állapotot. 5. **Visszatérés:** A kernel visszatér a megszakításból, és a CPU a P_2 folyamat utolsó kernelbeli pontján folytatja a végrehajtást, majd visszatér a felhasználói térbe.

A gyakorlati megvalósítás során a `context_switch` függvénynek el kell mentenie a callee-saved regisztereket, át kell állítania a veremmutatót, majd vissza kell állítania az új folyamat regisztereit. Az x86-64 ABI szerint ezek az RBX, RBP, R12-R15 regiszterek.

A kontextusváltás egy egyszerű, de működő modelljét a következőképpen valósíthatjuk meg. Minden folyamatnak van egy `task_struct` (vagy PCB) struktúrája, amelynek első mezője a kernel stack tetejére mutató pointer (`rsp`). Amikor a folyamat fut, a kernel stack tetején vannak a callee-saved regiszterek és a visszatérési cím.

```
; context_switch(uint64_t *old_rsp, uint64_t new_rsp)
; RDI = mutató az aktuális folyamat rsp mezőjére
; RSI = az új folyamat rsp értéke
global context_switch
context_switch:
    ; Callee-saved regiszterek mentése a JELENLEGI kernel veremre
    push rbx
    push rbp
    push r12
    push r13
    push r14
    push r15

    ; Az aktuális kernel veremmutató elmentése az 'old_rsp' pointerre
```

```
    mov [rdi], rsp

    ; Az új veremmutató beállítása
    mov rsp, rsi

    ; Callee-saved regiszterek visszaállítása az ÚJ veremről
    pop r15
    pop r14
    pop r13
    pop r12
    pop rbp
    pop rbx

    ; Visszatérés az új folyamat context_switch hívása utáni pontjára
    ret
```

A fenti assembly kód kulcsfontosságú trükkje a `ret` utasítás. Amikor P_1 meghívja a `context_switch`-et, a `call` utasítás a P_1 visszatérési címét (R_1) a verem tetejére helyezi. A függvény elején ezt a címet további 6 regiszter mentése fedi. Amikor elmentjük az `rsp`-t, a P_1 kernel állapota félbehagyva marad.

Amikor legközelebb valaki visszakapcsol P_1-re, az `rsp` pontosan arra a pontra fog mutatni. A kód visszaállítja a regisztereket, majd a `ret` ezen a veremcímen a mentett R_1 címet fogja látni, és oda ugrik. Így P_1 számára úgy tűnik, mintha a `context_switch` függvény egyszerűen csak visszatért volna, miközben a valóságban közben milliónyi utasítást hajtott végre egy másik folyamat.

A preemptív ütemező kapcsolása: A `scheduler_tick` függvényben a tick számláló alapján eldönthetjük, hogy lejárt-e a folyamat időszelete. Ha igen, a `context_switch` segítségével átváltunk egy másik, futásra kész folyamatra.

Tegyük fel, hogy van egy nagyon egyszerű, round-robin ütemezőnk, ahol két folyamat (`task_a` és `task_b`) váltja egymást. A `scheduler_tick` megnézi, hogy az aktuális folyamat `time_slice` értéke elérte-e a 0-t. Ha igen, akkor meghívja a `context_switch`-et.

```
void scheduler_tick() {
    if (current->time_slice > 0) {
        current->time_slice--;
    }
    if (current->time_slice == 0) {
        // Váltsunk!
        struct task *next = (current == &task_a) ? &task_b : &task_a;
        next->time_slice = 10; // Adjunk 10 tick időt

        uint64_t old_rsp = current->rsp;
        current = next;

        // Assembly hívás: az aktuális verem mentése, új verem betöltése
        context_switch(&old_rsp, current->rsp);
    }
}
```

Fontos versenyhelyzeti megjegyzés: A `context_switch` hívása a megszakításkezelő kontextusában történik. Ez azt jelenti, hogy az új folyamat futtatása is a megszakításkezelő belsejében kezdődik. Ez a legtöbb oktatási kernelben így működik, de meg kell értenünk, hogy a `context_switch`-ből való visszatérés után az új folyamat a `scheduler_tick` függvényből tér vissza a megszakításkezelőbe, ahol az EOI-t elküldjük, majd az `iretq`-val a felhasználói folyamatra. Ez a minta biztosítja, hogy a váltás atomi, mert megszakítások tiltva vannak (vagy a folyamat éppen kernel módban van).

28.2.1.6 Hibakezelés és tipikus csapdák

A megszakításkezelés és a kontextusváltás során a legalattomosabb hibák általában nem fordítási hibák, hanem finom logikai vagy konvencióbeli eltérések:

1. **Hiányzó EOI:** Ahogy korábban említettük, a PIC-hez nem küldött EOI után a rendszer lefagy. Ha a "timer tick" nyomkövetése azt mutatja, hogy egyszer lefut, majd megáll, ellenőrizzük az EOI logikát.
2. **ABI megsértése:** Ha a `isr_common` rutinban elfelejtünk egy callee-saved regisztert elmenteni, a megszakításból visszatérve a futó program rejtélyes regiszterérték-korrupciót fog látni. Ez különösen akkor jelentkezik, ha a C kódot magas optimalizálási szinten (`-O2`) fordítjuk, ahol a fordító erősen támaszkodik az ABI-ra.
3. **Helytelen IDT bejegyzés:** Ha a `type_attr` mezőben a Present bitet nem állítjuk be, vagy rossz a DPL (Descriptor Privilege Level) értéke, a CPU General Protection Fault-tal (#GP) fog leállni a megszakítás kiváltásának pillanatában.
4. **Veremigazítás hiánya:** Az x86-64 System V ABI előírja, hogy a `call` utasítás előtt a verem 16 bájtra legyen igazítva. A megszakítási stubokban a hamis hibakódok és vektorok pusholása eltolhatja az igazítást. Ha a C függvényben SSE utasításokat használunk (pl. `movaps`), az hibás címhez vezet.
5. **Szelektor hiba:** A kontextusváltás során a `cs` és `ss` szegmensszelektorokat nem szabad elfelejteni. Bár a modern x86-64-ben ezek nagyrészt statikusak, a felhasználói és kernel mód közötti váltásnál (`iretq`) a CPU ellenőrzi ezeket.

28.2.1.7 Labor: IDT- és interrupt-handler vázlat

A cél egy olyan minimumrendszer készítése, amely bizonyítja, hogy a megszakítási rendszerünk működik. A labor során a QEMU-ban futtatott kernelünk kap egy PIT által generált megszakítást, a megszakításkezelő megnövel egy számlálót, és a képernyőre írva vizuálisan is igazoljuk a működést.

Kiinduló állapot: Az `os106` fejezetben elkészített freestanding C/Assembly build környezet (linker script, `boot.s`, `kernel.c`).

Feladatok:

1. Hozz létre egy `idt.c` fájlt az IDT bejegyzések inicializálásához. Írj egy `set_idt_gate` függvényt, amely feltölti a `struct idt_entry` mezőit. A `selector` értéke `0x08` legyen, a `type_attr` értéke pedig `0x8E` (Interrupt Gate, Ring 0).
2. Hozz létre egy `isr.s` assembly fájlt a `ISR_NOERRCODE` makróval. Készíts el legalább 16 IRQ stubot a `32`-től `47`-ig terjedő vektorokon.
3. Implementáld a `isr_common` rutint a regisztermentéssel. A `interrupt_handler` C függvény kezdetben csak naplózza a megszakítás számát.
4. A `kernel_main`-ben hívd meg a `timer_init(100)`-et, töltsd be az IDT-t, és engedélyezd a megszakításokat (`sti`).
5. A `irq_handler`-ben a `timer_ticks` értékét írasd ki a VGA pufferba (a C és assembly alapok, fordítás és linkelés alfejezetben már megismert módon).

Futtatási parancs (reprodukálható környezet): Az alábbi parancs a kernel lefordításához és futtatásához egy izolált, emulált környezetben. Ne futtasd a hoszt gépen közvetlenül!

```
# Fordítás
gcc -ffreestanding -mno-red-zone -m64 -c kernel.c -o kernel.o
nasm -f elf64 isr.s -o isr.o
ld -T linker.ld -o kernel.elf kernel.o isr.o
```

```
# Futtatás emulált környezetben (QEMU)
qemu-system-x86_64 -kernel kernel.elf -display curses -no-reboot -no-shutdown
```

Mérési napló:

Paraméter	Várt/Levezetett érték	Mért érték	Módszer
PIT frekvencia	100 Hz	(ellenőrizendő)	Tick számláló és QEMU virtuális idő
Megszakítási vektor	`32` (0x20)	`32`	Naplózott `int_no`
Tick periódus	10,000 μs	(ellenőrizendő)	QEMU systick mérés
Kontextusváltás ciklusszám	(levezetés: ~6 push + 6 pop + mov + ret 100-150 ciklus)	(mérést igényel a hardver/emuláció profilozóval)	GDB/QEMU trace

Elvárt eredmény: A QEMU képernyőjén a `timer_ticks` számláló folyamatosan növekszik, jelezve, hogy a PIT megszakításai elérik a CPU-t, a CPU az IDT-n keresztül belép a csonkba, a C handler lefut, majd az EOI után a rendszer visszatér a normál futáshoz.

Hibakeresési tippek: Ha a rendszer lefagy, először ellenőrizzük, hogy a `timer_ticks` változó növekedett-e. Ha egyszer nőtt, majd megállt, az EOI hiányát jelzi. Ha Triple Fault lép fel (a QEMU hirtelen újraindul), az általában az IDT mutató hibás betöltésére vagy a stub címek rossz eltolására utal. A részletes nyomkövetést a következő alfejezetben (QEMU, GDB és kernel debugging) tárgyaljuk a QEMU monitor és a GDB integrációjával.

Önellenőrző kérdések:

1. Miért kell a PIC-et remapelni, és mi történne, ha a 0x08-as vektorra hagynánk?
2. Írd le szavakban, mi történik a CPU regisztereivel és a veremmel attól a pillanattól kezdve, hogy a PIT megszakítást generál, egészen addig, amíg a `context_switch` visszatér a második folyamatba.
3. Mi a különbség a trap gate és az interrupt gate között az IDT bejegyzésben, és miért fontos ez a megszakítások engedélyezése szempontjából?

28.3 QEMU, GDB és kernel debugging

A kernel implementáció egy adott pontján a hibakeresés többé nem oldható meg kizárólag a soros konzolra írt naplókkal. A vezérlési folyamat nyomon követése, a regiszterek és a memória pillanatnyi állapotának vizsgálata, valamint a verem rekonstruálása olyan igények, amelyekhez interaktív debugger és determinisztikusan újraindítható futtatókörnyezet kell. Ebben a fejezetben ezt a munkafolyamatot építjük fel QEMU és GDB segítségével, és egy kisméretű, szándékosan hibás kernel példáján keresztül gyakoroljuk be.

28.3.1 Emulált hardveren való fejlesztés

Az emulált hardveren való fejlesztés nem egyszerűen kényelmi kérdés, hanem a kernel korai fázisában biztonsági és hatékonysági követelmény. A kernel fejlesztése során a hibák túlnyomó része a hardver inicializálásából, a memóriatérkép felépítéséből, a megszakítás- és kivételkezelésből, illetve a lapozás bekapcsolásából származik. Ezek a hibák valódi hardveren gyakran a gép lefagyását, újraindulását vagy a bootloader megsérülését okozhatják. Egy emulátorban ugyanezek

a hibák reprodukálhatók, de a rendszer állapota megállítható, visszatekerhető és tetszőleges ponton megvizsgálható.

A QEMU azért jó választás, mert:

- pontosan szimulálja a CPU állapotát, beleértve a védelmi szinteket, a szegmensregisztereket, a lapozást és a TLB-t,
- beépített GDB stubot biztosít, így a vendég CPU regiszterei és memóriája távolról olvashatók,
- soros portot, megszakításvezérlőt és időzítőt emulál, ami elegendő a kernel alapfunkcióinak teszteléséhez,
- a naplózási lehetőségek révén a CPU által végrehajtott utasítások, a megszakítások és a vendég hibák is rögzíthetők.

A fejlesztési ciklus jellemzően így néz ki:

1. A kernel forrásait lefordítjuk debug szimbólumokkal, és az eredményt ELF formátumba linkeljük.
2. A QEMU-t úgy indítjuk, hogy a GDB stub aktív legyen, és opcionálisan az első utasításnál megálljon.
3. A GDB-t csatlakoztatjuk a stubhoz, betöltjük a szimbólumtáblát, és breakpointokat helyezünk el.
4. A program futását folytatjuk, és a breakpointoknál vizsgáljuk a regisztereket, a verem és a memória tartalmát.
5. A hibát azonosítjuk, a forrást módosítjuk, és a ciklust újra lefuttatjuk.
6. A munkafolyamatot, a parancssorokat és az eredményeket naplózzuk, hogy a reprodukció később is biztosított legyen.

28.3.2 QEMU monitor és soros konzol

A QEMU-nak két, egymástól jól elkülönülő felülete van a hibakeresés során: a **monitor** és a **soros konzol**. A monitor maga a QEMU vezérlőfelülete, ahonnan az emulátor állapota kérdezhető le és módosítható, a soros konzol pedig a vendég rendszer soros portjának kimenetét jeleníti meg.

A monitor elérésére a `-monitor` kapcsolóval, illetve a `-serial mon:stdio` esetén a `Ctrl-A C` billentyűkombinációval van lehetőség. A monitorban olyan parancsokat adhatunk ki, mint:

- `info registers` – a vendég CPU regisztereinek kiírása,
- `info mtree` – a memóriatérkép megjelenítése,
- `x /10i $pc` – a programszámlálótól kezdve 10 utasítás disassemblálása,
- `stop` – a vendég CPU megállítása,
- `cont` – a futás folytatása,
- `system_reset` – a vendég gép újraindítása,
- `savevm` és `loadvm` – pillanatkép mentése és visszatöltése.

A monitor akkor is használható, ha a GDB éppen csatlakoztatva van, de a két interfész vezérlési parancsait nem szabad összekeverni: a monitor a QEMU szintjén, a GDB a vendég CPU szintjén működik.

A soros konzol a kernel naplózásának elsődleges csatornája. A kernelünk a hagyományos PC-s soros porton, az I/O port `0x3F8`-on keresztül ír. A QEMU a `-serial stdio` kapcsolóval ezt a kimenetet a terminálra irányítja. A `-serial mon:stdio` kombináció pedig a `Ctrl-A C` segítségével válthatóvá teszi a monitor és a soros konzol között.

Példa a QEMU indítására:

```
qemu-system-i386 -cdrom build/os.iso -s -S -serial mon:stdio \
  -d int,cpu_reset,guest_errors -D qemu.log
```

A fenti parancs jelentése:

- `-cdrom build/os.iso` – a rendszer image betöltése CD-ként,
- `-s` – a GDB stub engedélyezése a `localhost:1234` TCP porton,
- `-S` – a CPU megállítása a legelső utasítás előtt,
- `-serial mon:stdio` – a soros port a szabványos bemenetre/kimenetre kerül, monitorral kombinálva,
- `-d int,cpu_reset,guest_errors` – a megszakítások, a CPU-újraindítás és a vendéghibák naplózása,
- `-D qemu.log` – a napló fájlba írása.

A `-S` kapcsoló különösen fontos, mert enélkül a kernel a GDB csatlakoztatása előtt túlfuthat azon a ponton, ahová breakpointot szeretnénk tenni.

28.3.3 GDB csatlakoztatása és szimbólumkezelés

A GDB a kernel ELF fájljának betöltésével ismeri meg a szimbólumokat, a típusokat és a forráskód-sorokat. A kernel fejlesztése során a fordítási parancsban explicit módon kérjük a debug információkat és a keretpointer megtartását:

```
make CFLAGS="-g -O0 -fno-omit-frame-pointer -m32" LDFLAGS="-m elf_i386"
```

A `-g` a DWARF debug szakaszokat hozza létre, a `-O0` a kódot optimalizálás nélkül tartja, a `-fno-omit-frame-pointer` pedig biztosítja, hogy a függvényhívások során a veremkeret mutatója megmaradjon. Ez utóbbi elengedhetetlen a megbízható stack trace készítéséhez.

A GDB indítása:

```
gdb build/kernel.elf
```

A szimbólumfájl betöltése után csatlakozunk a QEMU GDB stubhoz:

```
(gdb) target remote :1234
```

A kapcsolat létrejötte után a vendég CPU megáll, ha a QEMU-t `-S` kapcsolóval indítottuk. Ellenkező esetben a `(gdb)` prompt azonnal megjelenik, de a kernel már fut. A megbízható munkafolyamat érdekében mindig `-S`-t használunk.

A betöltési cím és a futási cím összhangját a linker script biztosítja. A kernelünk tipikus memóriatérképe az alábbi virtuális tartományokkal dolgozik:

- `0x00100000 – 0x001FFFFF`: kernel szöveg, read-only adat és írható adat (1 MiB),
- `0x00200000 – 0x003FFFFF`: kernel verem (2 MiB),
- `0x00400000 – 0x005FFFFF`: korai heap és allokációs terület (2 MiB).

A linker script részlete:

```
ENTRY(_start)
SECTIONS {
    . = 0x00100000;
    .text : { *(.text*) }
    .rodata : { *(.rodata*) }
    .data : { *(.data*) }
    .bss : { *(.bss*) }
    . = 0x00200000;
    .stack : { *(stack) }
}
```

A GDB a `kernel.elf` szimbólumait ezekre a linkelt címekre képezi le. Ha a kernel betöltője más fizikai címre helyezi a képet, például a bootloader `0x00100000`-nél alacsonyabbra, akkor a szimbólumok eltolódnak, és a GDB nem találja a függvények valódi címét. Ilyenkor a `add-symbol-file` parancsot kell használni a helyes szakaszcímekkel, vagy gondoskodni kell arról, hogy a linkerkép és a betöltési cím megegyezzen.

28.3.4 Breakpointok és watchpointok

A GDB kétféle breakpointot használhat a kernel debug során: szoftveres és hardveres breakpointot. A szoftveres breakpoint a memóriában lévő utasítást ideiglenesen egy `int3` töréspont-utasításra cseréli. Ez csak írható memóriában működik, ezért a kernel szövege általában a RAM-ban van, nem pedig ROM-ban. A laborban a kernel a QEMU által biztosított RAM-ba töltődik, ezért a szoftver breakpointok megbízhatóan működnek.

A hardveres breakpointok a CPU debug regisztereit használják, és nem igénylik a memória módosítását. Ezek akkor hasznosak, ha a kód ROM-ban vagy írásvédett lapon van. A GDB `hbreak` parancsával állíthatók be, ha a cél hardver támogatja őket.

A tipikus breakpoint elhelyezése a `kmain` függvényre:

```
(gdb) break kmain
(gdb) continue
```

A `continue` hatására a vendég CPU futni kezd, és amint eléri a `kmain` címét, a GDB megállítja. Ekkor a regiszterek, a veremmutató és a programszámláló vizsgálhatók:

```
(gdb) info registers
(gdb) x/16x $esp
(gdb) bt
```

A `x/16x $esp` parancs a veremmutatótól kezdve 16 darab 32 bites szót ír ki hexadecimális formában. Ez a memória dump a verem aktuális tartalmát mutatja, és a stack trace értelmezéséhez is szükséges.

A watchpointok a memóriacímek figyelésére szolgálnak. Például ha azt gyanítjuk, hogy egy adott változót jogosulatlanul ír valaki, akkor:

```
(gdb) watch global_flags
(gdb) continue
```

Amikor a futás során a `global_flags` értéke megváltozik, a GDB megáll, és megmutatja az író utasítás címét. Ez a funkció a kernel fejlesztése során rendkívül hasznos a memóriafelszabadítási és párhuzamos hozzáférési hibák felderítésére.

28.3.5 Panic, stack trace és memory dump

A kernel hibakezelésének egyik központi eleme a `panic`. A `panic` függvény akkor hívódik, amikor a kernel olyan állapotba kerül, amelyből nem tud biztonságosan tovább futni. A hibakeresés során a `panic` nem csupán a hiba jelzésére szolgál, hanem arra is, hogy a GDB-vel megállítsuk a futást a hiba pillanatában.

A panic megvalósítása debug célokra gyakran tartalmaz egy `int3` utasítást, amely a GDB-nek ad egy töréspontot. A következő C kódrészlet szemlélteti ezt a mintát:

```
#ifndef NDEBUG
#define DEBUG_BREAK() __asm__ volatile("int3")
#else
#define DEBUG_BREAK()
#endif

void panic(const char *msg) {
    dbg_printf("PANIC: %s\n", msg);
    dump_registers();
    DEBUG_BREAK();
    for (;;) {
        __asm__ volatile("hlt");
    }
}
```

A fenti kódban a `dbg_printf` a soros portra ír, a `dump_registers` kiírja a fontosabb regisztereket, a `DEBUG_BREAK` a GDB töréspontját aktiválja, a végtelen ciklus pedig megakadályozza a további futást. Debug kiadásban (`NDEBUG` definiálatlan) a `int3` hatására a QEMU megáll, és a GDB promptjához kerülünk.

A `dump_registers` függvény egyszerű inline assemblyvel kinyeri a regisztereket:

```
static inline void dump_registers(void) {
    uint32_t eax, ebx, ecx, edx, esi, edi, esp, ebp;
    __asm__ volatile(
        "mov %%eax, %0\n"
        "mov %%ebx, %1\n"
        "mov %%ecx, %2\n"
        "mov %%edx, %3\n"
        "mov %%esi, %4\n"
        "mov %%edi, %5\n"
        "mov %%esp, %6\n"
        "mov %%ebp, %7\n"
        : "=r"(eax), "=r"(ebx), "=r"(ecx), "=r"(edx),
          "=r"(esi), "=r"(edi), "=r"(esp), "=r"(ebp)
        :
        : "memory");
    dbg_printf("eax=%p ebx=%p ecx=%p edx=%p esi=%p edi=%p esp=%p ebp=%p\n",
               eax, ebx, ecx, edx, esi, edi, esp, ebp);
}
```

A *stack trace* a verem hívási láncának rekonstrukciója. A GDB a `bt` parancssal a debug szimbólumok és a call frame information (CFI) alapján visszafejti a függvényhívásokat. Ha a verem sérült, vagy a kernel olyan részen állt meg, ahol a keretpointer nem megbízható, akkor a manuális vizsgálat szükséges:

```
(gdb) x/32xw $esp
(gdb) frame 0
(gdb) info frame
(gdb) disassemble
```

A `frame 0` az aktuális keretet mutatja, az `info frame` részletesen leírja a keret határait, a `disassemble` pedig az aktuális függvény utasításait. A veremmutatótól kezdődő memóriadumpban a visszatérési címeket kell azonosítani. A sorrend architektúrafüggő; x86 32 bites módban a `call` utasítás a visszatérési címet a verem tetejére teszi, így a `$esp`-től induló 32 bites szavak között kell keresni a kernel szövegcímére mutató értékeket.

28.3.6 QEMU naplózás és a debug kombinálása

A QEMU `-d` kapcsolójával engedélyezett naplózás értékes kiegészítő információkat ad. A `-d int,cpu_reset,guest_errors` például rögzíti:

- a megszakítások és kivételek bekövetkezését,
- a CPU reset eseményeit,
- a vendég oldalán észlelt hibákat, például a `EIP` érvénytelen memóriát címző értékét.

Ezek a naplósorok akkor is hasznosak, ha a GDB éppen nem kapcsolódott, mert a hiba bekövetkezésének időbeli sorrendje láthatóvá válik. A `-D qemu.log` fájlba írja a naplót, amely a reprodukciós csomag részét képezi.

A GDB és a QEMU napló együttes használata a leghatékonyabb. A soros konzol időbeli sorrendet mutat, a QEMU napló a hardvereseményeket, a GDB pedig a pillanatnyi CPU- és memóriaállapotot. E három forrás együttesen ad teljes képet a hiba okáról.

28.3.7 Tipikus hibák és helyes megoldások

A QEMU–GDB munkafolyamat során számos hiba adódhat. A leggyakoribbak a következők:

1. **A GDB nem tud csatlakozni**, mert a QEMU `-s` kapcsolója hiányzik, vagy a port foglalt. A hibaüzenet: `Remote connection closed` vagy `Connection refused`. A megoldás az indítóparancs ellenőrzése, és a másik QEMU példány leállítása.

2. **A breakpoint nem aktiválódik**, mert a kernel már túlhaladt rajta, mielőtt a GDB csatlakozott volna. A megoldás a QEMU `-S` kapcsolójának használata, és a `continue` kiadása csak a breakpoint beállítása után.

3. **A GDB nem találja a szimbólumokat**, mert a `kernel.elf` betöltési címe eltér a futási címtől. Ilyenkor a `info files` parancs mutatja a szekciócímeket, és a `add-symbol-file` segítségével korrigálhatunk.

4. **A serial konzolon semmi sem jelenik meg**, mert a soros port inicializálása hiányzik, vagy a rossz I/O portot írjuk. A QEMU `-serial stdio` mellett az UART regisztereinek beállítása is szükséges, bár a QEMU elnézőbb a valódi hardvernél. A `0x3F8`-ra írt bájtoknak mindenképpen meg kell jelenniük, ha a QEMU a soros portot a szabványos kimenetre kötötte.

5. **A stack trace értelmezhetetlen**, mert a kernel stack túl kicsi, és a mély hívási lánc túlcsordul. A linker scriptben biztosítani kell legalább 2 MiB stack méretet, és a `-fno-omit-frame-pointer` használatát.

6. **A szoftver breakpoint nem írható a memóriába**, mert a lapozás be van kapcsolva, és a szöveglap írásvédett. Ilyenkor a GDB `Cannot insert breakpoint` hibát ad. A megoldás a lapozás kikapcsolása a debug szakaszban, vagy hardver breakpoint használata.

28.3.8 Labor: QEMU- és GDB-debug munkafolyamat

A labor célja egy reprodukálható debug munkafolyamat elkészítése, amely egy szándékosan hibás kernelben azonosítja a null pointer dereference-t, és a javítás után verifikálja a helyes működést. A labor emulált környezetben fut, valós lemez vagy bootloader módosítása nélkül.

28.3.9 Kiinduló állapot

A kernel forrása egy minimális x86 32 bites rendszer, amely tartalmazza:

- a `kmain` függvényt,
- a soros konzolra író `dbg_printf`-et,
- egy `panic` függvényt a fent bemutatott debug töréspontt al,
- egy egyszerű GDT/IDT beállítást a megszakításokhoz.

A hiba a `kmain` alábbi sorában rejlik:

```
uint32_t *ptr = (uint32_t *)0x0;
*ptr = 0xDEADBEEF;  /* null pointer dereference */
```

A kernel a `0x0` címre ír, ami a védett mód bekapcsolása után a CPU page fault kivételét váltja ki. Mivel a IDT nincs teljesen beállítva, a CPU dupla hibát generál, és a QEMU leállítja a vendéget. A labor során ezt a hibát GDB-vel azonosítjuk.

28.3.10 Build és futtatás

A build parancs:

```
make clean
make CFLAGS="-g -O0 -fno-omit-frame-pointer -m32" LDFLAGS="-m elf_i386"
```

A `make clean` eltávolítja a korábbi objektumfájlokat, a második sor pedig létrehozza a `build/kernel.elf` fájlt. A debug szakasz meglétét ellenőrizhetjük:

```
readelf -S build/kernel.elf | grep -E "debug_info|symtab"
```

A QEMU indítása:

```
qemu-system-i386 -cdrom build/os.iso -s -S -serial mon:stdio \
  -d int,cpu_reset,guest_errors -D qemu.log
```

A `build/os.iso` az előzőleg elkészített bootolható image. A QEMU a `-S` miatt megvárja a GDB csatlakozását.

28.3.11 GDB munkamenet

Indítsuk el a GDB-t egy másik terminálban:

```
gdb build/kernel.elf
```

A GDB-ben:

```
(gdb) target remote :1234
(gdb) break kmain
(gdb) continue
```

A `kmain` breakpoint elérésekor a GDB megáll. Ekkor nézzük meg a regisztereket és a verem tartalmát:

```
(gdb) info registers
(gdb) x/16x $esp
```

A `$eip`-nek a `kmain` függvény belsejére kell mutatnia, a `$esp`-nek pedig a kernel stack tetejére. A továbbhaladáshoz:

```
(gdb) continue
```

A null pointer dereference hatására a CPU kivételt generál, és a QEMU megállítja a vendéget, vagy a GDB prompt jelenik meg, ha a kivétel a GDB stubhoz van bekötve. A napló és a GDB állapot alapján azonosítjuk a `$eip` értékét, a `$cr2`-t (a hiba címét) és a verem tartalmát. A `$cr2` a lapozási hiba esetén a hibás címet tartalmazza. Ezt a regisztert a `info registers all` parancs mutatja.

A stack trace készítése:

```
(gdb) bt
```

A várt kimenet legalább 3 keretet mutat: a `kmain`-en belüli író utasítást, a `kmain` elejét, és a boot indító `_start` függvényt. A memória dump a `0x0` címen:

```
(gdb) x/16x 0x0
```

A null pointer dereference miatt a `0x0` címen lévő memória általában olvasható, de írásvédett. A dump a pillanatnyi lapbejegyzést mutatja, vagy a QEMU leáll, ha a cím nem lapozott. A labor során a QEMU `-d guest_errors` naplója rögzíti a `EIP`-et és a hibás címet.

28.3.12 Mérési napló

A labor során az alábbi értékeket rögzítjük:

Mérési pont	Várt érték	Mért érték
`break kmain` elérése	A GDB prompt megjelenik, `$eip` a `kmain` címére mutat	A bejegyzés a GDB kimenetéből
`bt` keretek száma	Legalább 3	A `bt` kimenetéből számolt keretek
`x/16x 0x0` tartalma	A null pointer hiba miatt a lapozási hiba naplóban `0x0`	A dump bájtjai
A `panic` hívása	A hiba után a `panic` nem fut le, mert a CPU kivételt dob	A serial logban nincs `PANIC` sor

A QEMU `qemu.log`-jában a `guest_errors` bejegyzés rögzíti a hiba címét és az utasítás pointerét. Ezt a naplót a reprodukciós csomag részeként mentjük.

28.3.13 A hiba javítása és verifikálás

A hiba javítása egyszerű: a pointert a kernel heap területére kell állítani, például:

```
uint32_t *ptr = (uint32_t *)0x00400000;
*ptr = 0xDEADBEEF;
```

Ezután a build és a QEMU futtatás, majd a GDB munkamenet ismételt lefuttatása. A várt eredmény: a `kmain` breakpoint elérése, a `continue` után a kernel normálisan befejezi a működését, a serial log kiírja az `"ok"` üzenetet, és a `panic` nem hívódik meg. A mérési naplóban a mért értékeket összevetjük a várt értékekkel.

28.3.14 Visszaállítás

A labor után a QEMU bezárása, a `qemu.log` és a GDB munkamenet naplójának elmentése, valamint a build könyvtár tisztítása a `make clean` paranccsal. A hoszt gép lemezén nem történik változás.

28.3.15 Ellenőrzési pontok és minőségbiztosítás

A kernel debug munkafolyamat során a következő ellenőrzési pontokat kell nyomon követni:

- **Memória- és erőforrás-tulajdonjog:** A debug töréspontok beillesztése nem módosíthatja a vendég memória logikai tulajdonjogát. A szoftver breakpoint az `int3` utasítás ideiglenes beírásával jár, ezért a debug után a memóriát vissza kell állítani, vagy a GDB ezt automatikusan kezeli a folytatáskor.
- **Hibakezelés:** A `panic` hívásának meg kell előznie a további futást. A `hlt` ciklus csak akkor biztonságos, ha a megszakítások le vannak tiltva, ellenkező esetben a CPU felébredhet.
- **Versenyhelyzetek:** A kernel debug buildjében az időzítések eltérhetnek a release buildtől. A mérési naplóban ezt fel kell tüntetni, és a hibakeresés során nem szabad kizárólag az időzítésre támaszkodni.

A fejezetben bemutatott munkafolyamat a későbbi fejezetekben is alapul szolgál. A GDB remote protokoll, a QEMU monitor és a memória dump értelmezése a fuzzolásnál, a regressziós tesztelésnél és a teljesítménymérésnél is használatos lesz.

28.3.16 Önellenőrző kérdés

Miért nem elegendő a soros konzolra írt napló a memóriasérüléssel járó hibák azonosításához, és hogyan egészíti ki a GDB a hibakeresést? Válaszában térjen ki a pillanatnyi állapot és az időbeli sorrend közötti különbségre, valamint arra, hogy a napló miért nem mutatja a memória tartalmát a hiba pillanatában.

28.3.17 Továbbvitt eredmények

A következő fejezet (Naplózás, panic és hibakezelés) a naplózás, a `panic` és a hibakezelés részletes tárgyalására épül. Az itt bemutatott `panic` debug töréspont, a stack trace készítése és a mémoire dump a naplózási stratégia tervezésénél is alapvető elem. A laborban rögzített reprodukciós csomag a későbbi kódminőségi auditok és a regressziós tesztkörnyezet alapját képezi.

28.4 Naplózás, panic és hibakezelés

A kernel fejlesztése során a hibák elkerülhetetlenek. A userland programokban egy hibás mutató arra utal, hogy a folyamat megbukott, a rendszer többi része tovább működik. A kernelben ugyanez a hiba gyakran az egész rendszert veszélyezteti: nincs mögöttes védelmi háló, amely elkülönítené a hibás komponenst. Ezért a kernel saját hibakezelési eszközöket igényel: láthatóvá kell tenni a hibákat, diagnosztikai információt kell gyűjteni, és meg kell határozni, hogy a rendszer folytathatja-e a futást, vagy biztonságosan le kell állnia. Ebben a fejezetben a kernel naplózásának, a `panic` mechanizmusnak és a `oops` kezelésének alapelveit és gyakorlati mintáit tekintjük át. A cél nem csupán a Linux `printk` API ismertetése, hanem egy olyan gondolkodásmód kialakítása, amelyet a saját, minimális rendszerünkben is alkalmazni tudunk.

28.4.1 A kernel hibakezelés sajátosságai

A kernel futási környezete alapvetően különbözik a felhasználói térétől. Nincs `libc`, nincs szabványos `printf`, nincs fájlrendszer, ahová naplót írhatunk, és nincs processzhatár, amely a hibát lokálisan kezelné. A kernel a legmagasabb privilégiumú módban fut, közvetlen hozzáféréssel a hardverhez és a memóriához. Egy hibás memória-hozzáférés itt nem egyszerűen a hibás folyamat terminálását jelenti, hanem potenciálisan a kernel adatstruktúráinak megsérülését, eszközök hibás állapotba kerülését vagy váratlan rendszerleállást.

A hibakezelés három fő célja:

1. **Láthatóság** – a hiba ne maradjon néma; a fejlesztő és a rendszeradminisztrátor értesüljön a bekövetkezéséről, annak körülményeiről.
2. **Diagnosztika** – a naplóüzenet tartalmazzon elegendő kontextust (időbélyeg, processzor, folyamat, verem, regiszterek) a hiba okának azonosításához.
3. **Megbízható folytatás vagy biztonságos leállás** – a kernelnek döntést kell hoznia: a hiba helyi és javítható-e, vagy a rendszer állapota már nem megbízható. Ez utóbbi esetben a legbiztonságosabb választás a `panic`, azaz a rendszer azonnali, szándékos leállítása.

A kernelben a hibakezelés egyszerű eszközökkel dolgozik, mert a hiba bekövetkeztekor nem támaszkodhatunk bonyolult futásidejű környezetre. A naplózásnak minimális függőségűnek kell lennie, és korán elérhetőnek – már a boot legelső fázisában is.

28.4.2 Kernel naplózás: printk és log szintek

A leggyakoribb kernelnaplózási mechanizmus a `printk` (print kernel). Ez a felhasználói `printf` analógja, de közvetlenül a kernelben fut, és a kimenetet nem a standard kimenetre, hanem egy körkörös puffere (ring buffer) írja. A `printk` függvény a formázott üzenetet előállítja, majd a megfelelő szinttel együtt a naplóba helyezi. A Linux kernelben a `printk` a következőképpen hívható:

```
printk(KERN_INFO "kernel booted, version %s\n", version);
```

A `KERN_INFO` makró nem külön argumentum, hanem a formátumstring elejére illesztett log szintet jelölő előtag. Fontos, hogy a log szint és a formátumstring egy literálban szerepel, így a fordító ellenőrizheti a paramétereket.

A log szintek jelentése:

- `KERN_EMERG` – a rendszer használhatatlan, azonnali beavatkozás szükséges.
- `KERN_ALERT` – súlyos hiba, azonnali figyelem szükséges.
- `KERN_CRIT` – kritikus feltétel, például hardverhiba.
- `KERN_ERR` – általános hiba, amely nem feltétlenül fatális.
- `KERN_WARNING` – figyelmeztetés, potenciális probléma.
- `KERN_NOTICE` – normál, de fontos esemény.
- `KERN_INFO` – tájékoztató üzenet.
- `KERN_DEBUG` – részletes, hibakereséshez használt üzenet.

A szintek hierarchiát alkotnak. A kernel egy `console_loglevel` (vagy `default_message_loglevel`) értéket tart nyilván, amely meghatározza, hogy mely üzenetek jussanak el a soros konzolra vagy a képernyőre. A magasabb prioritású (alacsonyabb numerikus értékű) szintek mindig megjelennek, a alacsonyabb prioritásúak csak akkor, ha a beállítás ezt engedi. Például ha a konzolszint `KERN_WARNING`, akkor a `KERN_INFO` és `KERN_DEBUG` üzeneteket a ring bufferbe írjuk, de a konzolra nem küldjük. Ezzel elkerülhető, hogy a rendszer indulásakor a konzolt elárasszák a részletes debug üzenetek, miközben a hibaüzenetek megmaradnak.

A `printk` implementációja több rétegből áll:

1. **Formázás** – a `vsnprintf` segítségével a változó argumentumokból szöveges reprezentációt készít.
2. **Rekordkészítés** – a formázott üzenethez hozzáfűzi a szintet, időbélyeget, processzorazonosítót és egyéb metaadatot.
3. **Ring bufferbe írás** – a rekordot a körkörös pufferbe másolja.
4. **Konzolra küldés** – ha a szint engedélyezi, a konzol drivernek átadja a szöveget.

A korai boot fázisban a konzoldriver még nem érhető el. Ilyenkor a `printk` üzenetek kizárólag a ring bufferbe kerülnek, és a konzol inicializálása után a buffer tartalma kiüríthető. Ezt a mechanizmust nevezzük korai naplózásnak; a Linux kernelben például a `earlycon` vagy `earlyprintk` paraméterekkel aktiválható.

A következő C kód egy minimális, freestanding `printk` vázlatot mutat be. A példa szemléltetés, nem teljes implementáció:

```
/* Minimal kernel log - illustrative only */
typedef enum {
    LOG_EMERG   = 0,
    LOG_ALERT   = 1,
    LOG_CRIT    = 2,
    LOG_ERR     = 3,
    LOG_WARNING = 4,
    LOG_NOTICE  = 5,
    LOG_INFO    = 6,
    LOG_DEBUG   = 7
} log_level_t;

/* Console log threshold: messages with level <= console_level go to console */
static log_level_t console_level = LOG_WARNING;

void serial_write(const char *s);  /* provided by UART driver */
```

```
static void log_putc(char c) {
    /* placeholder: push character to ring buffer and maybe serial */
    ring_buffer_put(c);
    if (console_level <= LOG_WARNING) {
        serial_write(&c);  /* caution: serial_write expects null-terminated */
    }
}

void kernel_log(log_level_t level, const char *fmt, ...) {
    /* in real code, use vsnprintf; here simplified */
    char buf[128];
    va_list args;
    va_start(args, fmt);
    /* vsnprintf(buf, sizeof(buf), fmt, args) must be implemented */
    /* ... */
    va_end(args);

    for (const char *p = buf; *p; p++) {
        log_putc(*p);
    }
}
```

A fenti vázlatban a `ring_buffer_put` és a `serial_write` függvényeket a hardver réteg biztosítja. A `serial_write` a soros portra ír, ami a legegyszerűbb konzol. A `vsnprintf` kényelmi függvény a formázáshoz, de a legkorábbi boot fázisban sok kernel saját, minimalista implementációt használ.

A log szintek használatakor alapelv, hogy a fejlesztő ne használjon magasabb prioritást a szükségesnél. A `KERN_ERR` szint a tényleges hibáké, a `KERN_DEBUG` a fejlesztői nyomkövetésé. A túlzottan magas szintű üzenetek elrejthetik a valódi problémákat, míg a túl alacsony szintűek hamis riasztást kelthetnek.

28.4.3 Ring buffer és naplófogyasztás

A kernel naplózásának központi adatszerkezete a körkörös puffer. Ennek fix mérete van, és a régebbi üzenetek felülíródnak, ha a puffer megtelik. Ez a kialakítás garantálja, hogy a naplózás soha nem blokkol, és nem fogyaszt dinamikus memóriát – ami hibakezelés közben kritikus. A ring buffer tipikusan néhány KiB-tól néhány MiB-ig terjed; mérete konfigurálható.

A Linux kernelben a ring bufferben minden üzenet egy rekord, amely a következő mezőket tartalmazza:

- időbélyeg (timestamp),
- processzorazonosító,
- log szint,
- a formázott szöveg,
- sorozatszám.

A rekordokat egy olvasó (például a `dmesg` parancs) olvashatja ki a felhasználói térből. A kernel a `/dev/kmsg` eszközön keresztül teszi lehetővé a valós idejű olvasást, ahol minden rekord egyszer olvasható. A `syslog` rendszerhívás a klasszikus interfész, amely a ring buffer tartalmát másolja a felhasználói térbe.

A ring buffer fontos tulajdonsága, hogy lehetővé teszi a napló megőrzését akkor is, ha a rendszer összeomlik. Ha a kernel `panic` állapotba kerül, a ring buffer tartalma a memóriában marad, és a következő boot után a rendszerindítási folyamat visszaolvashatja, vagy a `kdump` mechanizmus a

memóriaképet mentheti. Emiatt a ring buffernek a `panic` előtt flush-elhetőnek kell lennie, de ez nem garantált, ha a hardveres konzol nem elérhető.

A naplófogyasztás szempontjából a ring buffernek két mutatója van: az író (writer) és az olvasó (reader) pozíció. Az író mindig a legújabb szabad helyre ír, és ha eléri a végét, visszaugrik az elejére, esetleg felülírva a legrégebbi, még nem olvasott rekordot. Az olvasó az utolsó olvasott pozíciótól halad előre. Ezzel a struktúrával a napló elvesztése akkor következik be, ha a termelő sebessége tartósan meghaladja a fogyasztóét.

A tipikus hibrid megoldás: a ring buffer a teljes naplót tárolja, a soros konzol pedig csak a szűrt üzeneteket kapja. Ha a ring buffer túl kicsi, a hibakereséshez szükséges korai üzenetek elveszhetnek, de a konzol kimenet megőrizheti a legfontosabb eseményeket. A tervezésnél ezért a `panic` diagnózishoz elegendő naplót kell biztosítani: a hiba bekövetkezése előtti néhány száz üzenet általában elegendő a kontextus rekonstruálásához.

28.4.4 Panic, oops és helyreállítás

A kernelben a hibák két nagy kategóriába sorolhatók: **fatális** és **nem fatális**. A fatális hiba azt jelenti, hogy a kernel állapota nem megbízható, és a további futás nagyobb kockázatot jelent, mint a leállás. A nem fatális hiba esetén a rendszer tovább működhet, de a hibát naplózni kell.

28.4.5 Oops

A Linux kernelben az `oops` egy nem fatális, de súlyos hiba. Olyan helyeken fordul elő, ahol a kernel hibás állapotot észlel, például `NULL` pointer dereferencia egy olyan kontextusban, ahol a hiba visszaadható. Az `oops` kiírja a hibaüzenetet, a processzor regisztereit, a verem nyomkövetést és a releváns kernelmodulokat, majd a rendszer megpróbál folytatódni. A `oops` után a kernel megjelöli az érintett folyamatot, hogy az ne folytathassa, de a rendszer többi része fut tovább.

Az `oops` legfontosabb jellemzője, hogy a hibát tartalmazó művelet (például egy rendszerhívás) visszatérhet `-EFAULT` hibakóddal, vagy a folyamat `SIGSEGV` jelzést kaphat. A kernel saját adatstruktúrái azonban sérülhetnek, ezért az `oops` után a rendszer állapota gyanús. A Linux konfigurálható úgy, hogy az első `oops` után `panic`-et váltson ki (`panic_on_oops`), így elkerülve a további adatkorrupciót.

28.4.6 Panic

A `panic` a kernel szándékos leállítása. A `panic` függvény akkor hívódik, amikor a kernel olyan hibát észlel, amelyből nincs biztonságos visszatérés – például a scheduler adatstruktúrája sérült, egy `BUG()` makró aktiválódott, vagy a hardver végzetes hibát jelzett. A `panic` feladata:

1. **Megállítani a többi CPU-t** – SMP rendszerben a többi processzor megszakításokkal leállítható, hogy ne módosítsa tovább az állapotot.
2. **Kiírni a panic üzenetet** – a hiba leírása, a processzor és a folyamat azonosítója, a verem.
3. **Kiírni a regisztereket és a memóriát** – a hibakereséshez szükséges nyers adat.
4. **Végrehajtani a konfigurált kilépési műveletet** – alapértelmezésben a rendszer leáll; beállítható újraindítás, magasabb szintű hibakezelőnek való átadás (például hardveres watchdog), vagy hiba utáni kernelbe való átugrás (kexec).

A `panic` pszudokódja:

```
void kernel_panic(const char *fmt, ...) {
    /* Disable interrupts on this CPU */
    local_irq_disable();

    /* Stop other CPUs (SMP platform) */
    smp_send_stop();

    /* Print panic header */
    kernel_log(LOG_EMERG, "Kernel panic - not syncing: ");
    va_list args;
    va_start(args, fmt);
    /* print formatted message to console and ring buffer */
    /* ... */
    va_end(args);

    /* Dump registers and stack */
    dump_stack();
    dump_registers();

    /* Optional: flush ring buffer to console */
    console_flush();

    /* Execute configured action */
    if (panic_timeout == 0) {
        /* Hang forever */
        for (;;) {
            cpu_halt();
        }
    } else if (panic_timeout > 0) {
        /* Reboot after timeout */
        machine_restart();
    } else {
        /* Hand over to debugger or crash dump */
        machine_halt();
    }
}
```

A fenti pszudokód a `panic_timeout` változóval konfigurálhatóvá teszi a viselkedést. A `local_irq_disable` és `smp_send_stop` platformfüggő; a `dump_stack` és `dump_registers` a QEMU, GDB és kernel debugging fejezetben tárgyalt hibakereső eszközökre épül.

Fontos, hogy a `panic` kódjának minimálisnak és robusztusnak kell lennie. Nem szabad olyan függvényeket hívnia, amelyek maguk is hibát okozhatnak. Például a `printk` a `panic` közben nem foglalhat zárat, mert a hiba éppen egy zár miatt következhetett be. A kernel ezért a `panic` útvonalon közvetlenül a soros portra ír, megkerülve a normál naplózási réteget.

28.4.7 Helyreállítási stratégiák

A `panic` a végső eszköz, de bizonyos rendszerekben – különösen beágyazott vagy biztonság-kritikus környezetben – megengedett a részleges helyreállítás. Ilyen lehet:

- **Watchdog alapú újraindítás** – a hardveres vagy szoftveres watchdog figyeli a rendszer életjelét. Ha a `panic` bekövetkezik, a rendszer leáll, és a watchdog újraindítja. Ez a leggyakoribb beágyazott megoldás.
- **Crash dump és elemzés** – a `kdump` mechanizmus a `panic` előtt egy másik kernelbe ugrik, amely elmenti a memória tartalmát. A mentett kép később debuggerrel elemezhető.
- **Részleges hibatűrés** – egyes mikrokernel architektúrákban a hibás szolgáltatás újraindítható anélkül, hogy az egész kernel leállna. Ez a megközelítés azonban a hibahatárok gondos tervezését igényli.

- **oops utáni korlátozott folytatás** – ha a hiba nem érinti a központi állapotot, a kernel folytathatja, de a hibás modul letiltható.

A helyreállítási stratégia megválasztása a rendszer követelményeitől függ. Egy szerveren a `panic` utáni automatikus újraindítás elfogadható. Egy orvosi eszközön a leállás elfogadhatatlan, ezért a tervezésnek eleve el kell kerülnie a fatális hibákat.

28.4.8 Hibakezelési minták a kernelben

A kernelben a hibák jelzésére több makró és konvenció létezik:

- `BUG()` – feltétel nélküli fatális hiba; hatása `panic`, hacsak nincs beállítva `panic_on_oops`. Az `BUG()` utasítás gyakran egy olyan utasításra fordítódik, amely a debuggert hívja.
- `BUG_ON(condition)` – ha a feltétel igaz, `BUG()`-ot hív.
- `WARN()` – hasonló az `oops`-hoz, de a rendszer folytatódik. A `WARN()` üzenetet naplóz, és opcionálisan stack trace-t ír.
- `WARN_ON(condition)` – feltételes `WARN()`.
- `assert()` a kernelben általában nem használatos, mert a hiba súlyossága és a feloldás nem egyértelmű; helyette a fenti makrók szerepelnek.

A makrókat okosan kell használni. A `BUG` akkor indokolt, ha a feltétel megsértése garantáltan adatkorrupcióhoz vezet, és a továbbhaladás veszélyesebb, mint a leállás. A `WARN` akkor megfelelő, ha a hiba nem fatális, de jelezni kell a fejlesztőnek, mert váratlan állapotot tükröz. A `WARN`-t gyakran követi hibakód visszaadása, így a hívó fél tisztességesen kezelheti a helyzetet.

A hibakódok visszaadása a kernelben a `-errno` hagyományt követi: a függvények negatív hibakódot adnak vissza, pozitív értéket siker esetén. A pointert visszaadó függvények az `ERR_PTR` makróval hibakódot kódolnak pointerbe, a `IS_ERR` és `PTR_ERR` makrókkal ellenőrizhető.

Példa egy tipikus hibaellenőrzési mintára:

```
struct my_data *p = kmalloc(sizeof(*p), GFP_KERNEL);
if (!p) {
    pr_err("my_data allocation failed\n");
    return -ENOMEM;
}

if (some_condition) {
    WARN_ON(1);
    kfree(p);
    return -EINVAL;
}
```

Itt a `pr_err` a `printk(KERN_ERR ...)` rövidítése. A `WARN_ON` naplózza a hibát, de a függvény mégis visszatér.

28.4.9 Gyakorlati minta: minimális log és panic

A következő C kód egy egyszerű, de működőképes keretet mutat be egy tanulási célú kernelhez. A kód freestanding, tehát nem használ libc-t, és a hardverhez való hozzáférést a `serial_write` és hasonló függvények absztrahálják. A példa célja a naplózás és a `panic` működésének megértése, nem pedig éles rendszerben való használat.

```
/* minimal_log.h - illustrative kernel logging */
#ifndef MINIMAL_LOG_H
#define MINIMAL_LOG_H

typedef enum {
    LOG_EMERG   = 0,
    LOG_ALERT   = 1,
    LOG_CRIT    = 2,
    LOG_ERR     = 3,
    LOG_WARNING = 4,
    LOG_NOTICE  = 5,
    LOG_INFO    = 6,
    LOG_DEBUG   = 7
} log_level_t;

void kernel_log_init(void);
void kernel_log(log_level_t level, const char *fmt, ...);
void kernel_panic(const char *fmt, ...);
void kernel_oops(const char *fmt, ...);

#endif

/* minimal_log.c - simplified implementation */
#include <stdarg.h>
#include <stdint.h>
#include "minimal_log.h"

/* Platform-specific functions provided by UART or early console */
extern void serial_putchar(char c);
extern void machine_halt(void);
extern void machine_restart(void);
extern void local_irq_disable(void);
extern void local_irq_enable(void);
extern void dump_stack(void);

/* Ring buffer configuration */
#define LOG_BUFFER_SIZE 4096
static char log_buffer[LOG_BUFFER_SIZE];
static uint32_t log_wr = 0;
static uint32_t log_rd = 0;
static int log_lock = 0;  /* simple spinlock placeholder */

static log_level_t console_level = LOG_INFO; /* show INFO and above */

static void spin_lock(int *lock) {
    /* atomic test-and-set; actual implementation platform-specific */
    while (__sync_lock_test_and_set(lock, 1)) {
        /* spin */
    }
}

static void spin_unlock(int *lock) {
    __sync_lock_release(lock);
}

static void log_putc(char c) {
    /* Write to ring buffer */
    log_buffer[log_wr] = c;
    log_wr = (log_wr + 1) % LOG_BUFFER_SIZE;
    if (log_wr == log_rd) {
        /* Buffer full: advance reader to drop oldest */
        log_rd = (log_rd + 1) % LOG_BUFFER_SIZE;
    }
```

```
    /* Echo to console if level allows; we use a global current level? */
    /* The level check is done in kernel_log, not here. */
}

void kernel_log(log_level_t level, const char *fmt, ...) {
    if (level > console_level) {
        /* Message suppressed from console, but still stored? */
        /* In a real system, suppression usually means not printed at all,
           but for learning we still add to ring buffer with metadata. */
        /* Here we ignore completely; see note below. */
        return;
    }

    spin_lock(&log_lock);

    char buf[128];
    va_list args;
    va_start(args, fmt);
    /* Simplified formatting: just copy the string; assume fmt is plain text */
    int i = 0;
    while (fmt[i] && i < sizeof(buf) - 1) {
        buf[i] = fmt[i];
        i++;
    }
    buf[i] = '\0';
    va_end(args);

    for (const char *p = buf; *p; p++) {
        log_putc(*p);
    }

    spin_unlock(&log_lock);
}

void kernel_panic(const char *fmt, ...) {
    local_irq_disable();

    /* Try to print directly to console without lock */
    char buf[128];
    va_list args;
    va_start(args, fmt);
    int i = 0;
    while (fmt[i] && i < sizeof(buf) - 1) {
        buf[i] = fmt[i];
        i++;
    }
    buf[i] = '\0';
    va_end(args);

    for (const char *p = "KERNEL PANIC: "; *p; p++) {
        serial_putchar(*p);
    }
    for (const char *p = buf; *p; p++) {
        serial_putchar(*p);
    }
    serial_putchar('\n');

    dump_stack();
    /* Optionally flush ring buffer to console */
    for (uint32_t i = log_rd; i != log_wr; i = (i + 1) % LOG_BUFFER_SIZE) {
        serial_putchar(log_buffer[i]);
    }
```

```
    /* Decide action */
    machine_halt();  /* or machine_restart() */
}

void kernel_oops(const char *fmt, ...) {
    /* Non-fatal error: log and continue */
    char buf[128];
    va_list args;
    va_start(args, fmt);
    int i = 0;
    while (fmt[i] && i < sizeof(buf) - 1) {
        buf[i] = fmt[i];
        i++;
    }
    buf[i] = '\0';
    va_end(args);

    /* Use kernel_log with LOG_ERR level */
    kernel_log(LOG_ERR, "Oops: ");
    kernel_log(LOG_ERR, buf);
    kernel_log(LOG_ERR, "\n");
    dump_stack();
}
```

A kód több egyszerűsítést tartalmaz. A `kernel_log` formázása nem valódi `vsnprintf`; a gyakorlatban ezt a részt ki kell cserélni egy saját `vsnprintf` implementációra. A `console_level` ellenőrzése a `kernel_log` elején történik, és ha a szint alacsonyabb, az üzenet egyáltalán nem kerül a ring bufferbe. Ez leegyszerűsítés; a valódi kernelben a ring buffer minden üzenetet tárol, a konzolra csak a szűrt rész kerül. A tanulási verzióban ez elfogadható, de a laborban finomítható.

A `panic` függvény megszakítások letiltása után közvetlenül a soros portra ír, megkerülve a ring buffert és a zárat. A `dump_stack` függvény a verem tartalmát írja ki, amit a QEMU, GDB és kernel debugging fejezetben tárgyaltunk. A `machine_halt` platformfüggő; QEMU-ban például a `hlt` utasítás vagy a `shutdown` regiszter írása lehet.

28.4.10 Labor: kernel log- és panic-terv készítése

Ebben a laborban a feladat egy saját, minimális kernel naplózási és `panic` tervének elkészítése és emulált környezetben való tesztelése. A labor célja, hogy a fejlesztő megtapasztalja a naplózási réteg működését, és megtervezze a `panic` viselkedését. A reprodukálhatóság érdekében minden lépést dokumentálni kell, és a méréseket rögzíteni.

28.4.11 Előkészítés

Az emulált környezet QEMU, a vendég kernel a korábbi fejezetekben készített minimális rendszer. A konfiguráció rögzített:

- CPU: `-cpu cortex-a9` (vagy x86_64, a választott architektúra szerint)
- Memória: 128 MiB
- Soros konzol: `-serial mon:stdio`
- Kernel: saját, freestanding ELF

A build parancsok a reprodukciós csomag részét képezik, például:

```
make clean
make CFLAGS="-O0 -g -fno-omit-frame-pointer"
qemu-system-arm -M virt -cpu cortex-a9 -m 128M -nographic -kernel kernel.elf
```

A GDB csatlakoztatásához:

```
qemu-system-arm -M virt -cpu cortex-a9 -m 128M -nographic -s -S -kernel kernel.elf
gdb-multiarch kernel.elf
(gdb) target remote :1234
(gdb) continue
```

28.4.12 Feladatok

1. **Ring buffer implementációának tesztelése.** Írj egy tesztfüggvényt, amely 200 darab `LOG_INFO` szintű üzenetet generál, mindegyik egy sorszámmal. A ring buffer mérete legyen 256 bájt. Indítsd el a kernelt, és figyeld a soros kimenetet. Mérd meg:

   - hány üzenet jutott el a konzolra,
   - hány üzenet maradt meg a ring bufferben,
   - az üzenetvesztés arányát.

   A várt eredmény: a konzolra a beállított szint miatt minden `LOG_INFO` üzenet eljut, de a ring buffer csak az utolsó néhányat őrzi meg, a korábbiak felülíródnak. A mért eredményt naplózd.

2. **Log szintek szűrésének tesztje.** Állítsd a `console_level` értékét `LOG_WARNING`-ra. Generálj 50 `LOG_INFO` és 10 `LOG_ERR` üzenetet. Ellenőrizd, hogy a konzolon csak a `LOG_ERR` üzenetek jelennek meg, míg a ring bufferben mindkettő megtalálható (ha a pufferméret elegendő). A mérés: a konzol kimenet sorainak száma, a ring bufferben lévő rekordok száma.

3. **Panic teszt.** Szándékosan hívj `kernel_panic("test panic from timer\n")`-t egy időzítő megszakításból. Figyeld meg, hogy a `panic` üzenet megjelenik-e a soros konzolon, a regiszterek és a verem kiíródnak-e, és a rendszer leáll-e (nem folytatódik a futás). Mérd a `panic` hívásától a leállásig eltelt időt (például a QEMU napló időbélyegei alapján). Várt eredmény: a `panic` azonnal leállítja a rendszert, a kimenet tartalmazza a `"KERNEL PANIC: test panic from timer"` szöveget és a verem nyomkövetést.

4. **Naplóterv dokumentálása.** Készíts egy rövid (1 oldalas) tervet, amely meghatározza:

   - a támogatott log szinteket és azok használati politikáját,
   - a ring buffer méretét és a felülírási stratégiát,
   - a `panic` viselkedését (újraindítás, leállás, watchdog),
   - a crash utáni diagnosztikai adatok mentésének módját (ha van ilyen). A tervet helyezd a labor jegyzetfüzetbe.

28.4.13 Dokumentáció és mérési napló

A labor jegyzetfüzet (például Markdown vagy Jupyter notebook) tartalmazza:

- a használt QEMU parancsot és kernel build opciókat,
- a módosított forráskódot (diff),
- a mérések eszközét (például `grep` a QEMU naplón, `wc -l` sorok számolására, `time` a futási időhöz),

- a mért értékeket táblázatban,
- a várt értékeket, és a kettő eltérésének elemzését.

Példa mérési táblázat:

Teszt	Várt eredmény	Mért eredmény	Megjegyzés
Ring buffer telítődés	200 üzenetből ~30 marad	28 üzenet maradt	A buffer mérete miatt várható
Konzol szűrés	50 INFO-ból 0, 10 ERR-ből 10 jelenik meg	10 sor jelent meg	Megfelel
Panic késleltetés	< 1 ms	0,4 ms (QEMU)	A soros port sebessége dominál

A mérésnél a hardver, a fordítási beállítások, az órajelfrekvencia és a mérési módszer is megadandó. A QEMU-ban a virtuális órajel és a soros port sebessége nem feltétlenül tükrözi a valós hardvert, ezért a mért időket fenntartással kell kezelni. A labor eredménye így elsősorban a relatív viselkedést mutatja.

28.4.14 Önellenőrző kérdések

1. Mi a különbség a kernel `panic` és az `oops` között? Melyik esetben folytatódhat a rendszer, és miért kockázatos ez?
2. Miért használ a kernel körkörös puffert a naplózáshoz? Milyen előnyei vannak a dinamikus memóriaallokációval szemben?
3. Tegyük fel, hogy a `console_level` értéke `LOG_WARNING`, és a kernel 100 `LOG_DEBUG` üzenetet generál. Mi történik ezekkel az üzenetekkel, ha a ring buffer mérete 1 KiB, és minden üzenet átlagosan 64 bájt?
4. A `panic` függvény miért nem használhat normál kernelzárakat? Magyarázd el, milyen problémát okozhat egy zár használata a `panic` kódjában.

28.4.15 Összegzés

A kernel naplózás, a `panic` és a hibakezelés egymással szorosan összefüggő területek. A jól megtervezett naplózás lehetővé teszi a hibák gyors diagnosztizálását, míg a `panic` gondos implementációja biztosítja, hogy a rendszer a hibás állapotból ne okozzon további károkat. A gyakorlatban a `printk`, a log szintek és a ring buffer a leggyakoribb eszközök, amelyeket minden kernel fejlesztőjének ismernie kell.

A következő fejezet (Egyszerű fájlrendszer tervezése) az egyszerű fájlrendszer tervezésére épít, ahol a naplózás és hibakezelés elveit a fájlrendszer rétegben is alkalmazni fogjuk. A fájlrendszer-műveletek során fellépő hibákat ugyanúgy láthatóvá kell tenni, és a `panic`-re okot adó állapotokat gondosan el kell kerülni. A naplózási réteg kialakítása tehát nem öncélú, hanem a kernel további komponenseinek megbízhatóságát alapozza meg.

29 Fájlrendszer és eszközmeghajtó

29.1 Egyszerű fájlrendszer tervezése

Az egyszerű fájlrendszer tervezésekor a cél nem a teljes funkcionalitású, nagyvállalati ext4 vagy ZFS rendszer megalkotása, hanem egy olyan minimális, ellenőrizhető és oktatható rendszer felépítése, amelyen keresztül kézzelfoghatóvá válnak a korábban a VFS-ről, a blokk-eszközökről és a crash-konzisztenciáról tanult elvek. A fejezetben ismertetett terv a `minfs` nevet viseli, és kifejezetten emulált QEMU környezetben, eldobható lemezképfájlokon fut. Ez a fejezet a bevezetett adatszerkezeteket úgy alapozza meg, hogy a következő egység (Blokk-eszköz és driver implementáció) blokkmeghajtó-implementációja közvetlenül ezekre az elemekre építhessen.

29.1.1 1. A tervezés keretei és a blokkmodell

Mielőtt az adatszerkezeteket definiálnánk, rögzíteni kell a hardver- és tárolási absztrakciókat. A kernel számára a tárolóeszköz (legyen az QEMU-ban egy virtuális IDE, SATA vagy NVMe lemez, esetleg egy `virtio-blk` eszköz) szektorok lineáris sorozata. A minfs tervezésekor ezt a linearitást emeljük át blokkabsztrakcióvá. A kernel I/O rétegének feladata, hogy a felsőbb rétegek felé fix méretű logikai blokkokat szolgáltasson, amely adott esetben több fizikai szektort fog össze.

A minfs labor konfigurációja a reprodukálhatóság érdekében rögzített:

Paraméter	Érték	Megjegyzés
Eszköz	QEMU `virtio-blk`	emulált, írható eldobható kép
Lemezkép mérete	16 MiB	`dd`-vel létrehozva
Blokkméret	4 KiB (`0x1000`)	egy blokk = 8 szektor (512 B)
Összes blokk	4096	16 MiB / 4 KiB
Inode-ok száma	512	rögzített, statikus tábla

Egy ilyen kis rendszerben a blokkméret bátran lehet 4 KiB, mert a mai processzorok lapmérete is 4 KiB, így a memóriába emelt blokkok és a lapok határai szépen illeszkednek. A 16 MiB méret elég nagy ahhoz, hogy értelmes fájlokat tároljunk, de elég kicsi ahhoz, hogy a fájlkép hexdump-pal vagy saját eszköztárral könnyen átvizsgálható legyen. A tervezés első lépése a lemez blokkokra osztása és a régiók kijelölése:

Régió	Blokktartomány	Méret
Superblock	0	4 KiB
Adatblokk-bitmap	1	4 KiB
Inode-tábla	2 – 17	64 KiB (16 blokk)
Adatblokkok	18 – 4095	~15,94 MiB (4078 blokk)

Miért van szükség tehát superblockra, inode-táblára, bitmapre és adatblokkokra? Mert a fájlrendszernek egyszerre kell nyilvántartania a saját paramétereit (superblock), a fájlok metaadatait (inode), a lemez foglaltsági állapotát (bitmap), valamint a tényleges fájltartalmat (adatblokk).

29.1.2 2. Superblock: a fájlrendszer önleíró fejléce

A superblock a fájlrendszer legfontosabb egyedi adatszerkezete, amely a lemez elején, a nullás blokkban található. A kernel a `mount` művelet során ezt olvassa be először. A superblock nélkül a többi adatszerkezet helyzete és mérete ismeretlen lenne, hiszen a formattól függően változhat a blokkméret vagy a táblák eltolása.

A minfs superblockja C adatszerkezetként a következőképpen írható le (a kód itt és a továbbiakban szemléltetés, nem futtatandó éles utasítás):

```
/* freestanding C, emulált minfs kernelben használt deklaráció */
#define MINFS_MAGIC 0x4D494E46  /* ASCII: 'M' 'I' 'N' 'F' */

struct minfs_superblock {
    uint32_t s_magic;          /* 0x4D494E46 */
    uint32_t s_block_size;     /* 0x00001000 */
    uint32_t s_total_blocks;   /* 0x00001000 (4096) */
    uint32_t s_inode_count;    /* 0x00000200 (512) */
    uint32_t s_free_blocks;    /* aktuális szabad blokkok, kezdetben 4078 */
    uint32_t s_first_data_block; /* 0x00000012 (18) */
    uint32_t s_inode_table_start;/* 0x00000002 (2) */
    uint32_t s_state;          /* 0 = tiszta, 1 = hibás, 2 = ismert hibákkal */
    uint8_t  s_reserved[4044]; /* kitöltés a 4096 bájtos blokkhoz */
} __attribute__((packed));
```

A `__attribute__((packed))` direktíva fontos szerepet játszik. A C fordító alapértelmezés szerint igazíthatja a struktúratagokat a natív memóriahatárokhoz, így a struktúra mérete nagyobb lehet, mint a tagok összege. A lemezen viszont pontosan definiált, bájtpontos elrendezésre van szükség, ezért a kitöltést explicit módon kell megadni és a `packed` attribútummal letiltani a fordítói igazítást. A `s_magic` mező egy egyedi bűvös szám. Ha a kernel beolvassa a nullás blokkot, és a `s_magic` értéke nem egyezik a `MINFS_MAGIC` értékkel, akkor a mount műveletnek `-ENODATA` hibával kell visszatérnie, mert a lemezen nagy valószínűséggel nem minfs fájlrendszer található. Ez az első számú integritási kapu.

A `s_state` mező a crash-konzisztencia alapjait rakja le. A tervezett minfsben a mount során a kernel megnézi ezt a mezőt. Ha az érték nem 0, akkor a rendszer tisztában van vele, hogy a lemez lecsatolás nélkül lett kikapcsolva, és a gyorsított `fsck`-t le kell futtatni, amely ellenőrzi a bitmapet és az inode-okat. A superblock mérete pontosan 4096 bájt, tehát kitölti az első blokkot. A lemezre íráskor a kernel a teljes memóriapuffert kiírja, így a `s_reserved` terület is a lemezre kerül, ami fenntartja a jövőbeli bővítés lehetőségét.

29.1.3 3. Inode: a fájl identitása és indexelése

A minfs a klasszikus Unix-filozófiát követi: a fájl neve és a fájl adatai különválnak. A fájlrendszer a fájlokról az inode-okban tárolja a metaadatokat. A minfs tervezésénél az inode-tábla fix méretű, a lemez elején (2-től 17-ig terjedő blokkokban) helyezkedik el. A 16 blokk méretű tábla 4096 * 16 / 128 = 512 inode-ot képes tárolni, mivel minden inode mérete rögzített 128 bájt. A fix, 128 bájtos méret kényelmes, mert a ki- és beolvasás gyorsan, eltolásszámítással megoldható. A `minfs_inode` definíciója:

```
#define MINFS_N_DIRECT  12
#define MINFS_N_INDIRECT 1
#define MINFS_TYPE_FILE  1
#define MINFS_TYPE_DIR   2
#define MINFS_TYPE_SYML  3

struct minfs_inode {
    uint16_t i_mode;            /* típus és egyszerű jogosultság bitek */
    uint16_t i_links_count;     /* hivatkozások száma */
    uint32_t i_size;            /* fájlméret bájtban */
    uint32_t i_atime;           /* utolsó hozzáférés (epoch másodperc) */
    uint32_t i_ctime;           /* létrehozás / metaadat-változás */
    uint32_t i_mtime;           /* utolsó módosítás */
    uint32_t i_direct[MINFS_N_DIRECT];  /* közvetlen blokkmutatók */
    uint32_t i_indirect[MINFS_N_INDIRECT]; /* közvetett blokkmutató */
    uint32_t i_padding[4];      /* 128 bájtra kiegészítés */
} __attribute__((packed));
```

A blokkmutatók logikája a minfs teljesítményének és egyszerűségének kompromisszuma. Tizenkettő közvetlen blokkmutató 12 * 4 KiB = 48 KiB fájlméretet fed le. Ez a legtöbb konfigurációs fájl, shell-szkript és kisebb napló számára elegendő. A 13. mutató (az `i_indirect[0]`) egy olyan adatblokkra mutat, amely nem fájladatokat, hanem további blokkmutatókat tartalmaz. Mivel egy blokk 4096 bájt és egy mutató 4 bájt, egyetlen közvetett blokk 1024 további blokkra mutathat. Ez 1024 * 4 KiB = 4 MiB további tárterületet nyit meg. A maximális fájlméret tehát 48 KiB + 4 MiB = 4,05 MiB, ami a 16 MiB-os korlátozott laborlemezen bőséges.

Fontos tervezési döntés, hogy a minfs első implementációjában nem vezetünk be duplán közvetett mutatót. Ennek oka a hibakereshetőség: a közvetett blokkok fájljának felépítése az első számú forrása a fájlrendszer-hibáknak. Ha a hallgató hibásan indexel, és egy adatblokkot értelmez mutatóblokként, az könnyen katasztrofális felülíráshoz vezet. Az egyszerű, egy szintű indirekció elég ahhoz, hogy megtanítsa a blokkok közötti ugrás logikáját, de elég rövid ahhoz, hogy GDB-vel vagy naplózással lépésenként ellenőrizhető legyen.

29.1.4 4. Adatblokkok és könyvtárstruktúra

Az adatblokkok alkotják a lemez fennmaradó részét. Amit a felhasználó egy fájlban lát, az ezeknek a blokkoknak az összefűzése. Az `i_size` mező dönti el, hogy az utolsó blokkban hány bájt az érvényes adat. Ha például egy fájl 5000 bájt méretű, akkor két blokkot foglal le: az első blokk 4096 bájtja teljes, a második blokkban pedig 5000 - 4096 = 904 bájt az érvényes. Tipikus hiba, hogy a kernel a fájl beolvasásakor nem veszi figyelembe az `i_size` értékét, és a lemezről beolvasott blokk maradék részét is visszaadja a felhasználónak. Ez adatszivárgáshoz vezet, hiszen a blokkban korábban tárolt, törölt fájl maradványai is kikerülhetnek. A minfs tervezésekor rögzítenünk kell: az adatblokkok tartalmát a lefoglalás pillanatában nullázni kell, vagy a beolvasásnál az `i_size`-on túli területet a kernel puffereiben explicit módon nullával kell kitölteni.

A könyvtárak a minfsben speciális fájlok, amelyeknek az `i_mode` mezője `MINFS_TYPE_DIR`. A tartalmuk nem szabad formátumú szöveg, hanem strukturált rekordok sorozata. A minfs a legegyszerűbb, fix hosszúságú bejegyzéseket használja a könnyű karbantarthatóságért:

```
#define MINFS_NAME_MAX 27

struct minfs_dirent {
    uint32_t d_inode;           /* a fájl inode-száma, 0 = törölt */
    uint16_t d_rec_len;         /* rekord hossza (32 bájtra igazítva) */
    uint8_t  d_type;            /* típus, mint az inode-ban */
    uint8_t  d_name_len;        /* a név hossza */
```

```
    char     d_name[MINFS_NAME_MAX];/* név, nullával zárva */
} __attribute__((packed));
```

Ha egy fájlt keresünk egy könyvtárban, a kernel beolvassa a könyvtár inode-ját, majd annak adatblokkjaiban lineárisan végigiterál a 32 bájtos rekordokon, és összehasonlítja a `d_name` értékét a keresett névvel. Ez egy O(n) komplexitású művelet, ami egy ilyen tanuló rendszerben bőven elfogadható. Valós rendszerekben itt B-fákat vagy hash-táblákat használnak, de a minfs célja az algoritmus átláthatósága. A `d_inode = 0` érték jelöli a törölt rekordot. Ez lehetőséget ad arra, hogy a későbbi létrehozások újra felhasználják a helyet anélkül, hogy a könyvtárfájlt tömöríteni kellene.

29.1.5 5. Szabadhely-kezelés: az adatblokk-bitmap

A lemez foglaltsági állapotát a minfs az 1. blokkban található bitmap segítségével tartja nyilván. A bitmap minden egyes adatblokkhoz hozzárendel egy bitet. Ha a bit értéke 1, a blokk foglalt; ha 0, akkor szabad. A 4096 blokk kezeléséhez 4096 bitre, azaz 512 bájtra van szükség. Mivel a bitmap számára egy teljes 4 KiB-os blokk áll rendelkezésre, a fennmaradó 3584 bájt egyelőre kihasználatlan, de fenntartott.

A bitmap választása a free-listtel szemben a tárolás hatékonyságában és az allokáció kiszámíthatóságában rejlik. Egy egybites nyilvántartásnál a lefoglalás és a felszabadítás konstans időben (O(1)) elvégezhető, amennyiben a kernel a bitmap blokkját a memóriában cache-eli. A lefoglalás algoritmusa:

```
/* Pszeudokód: szabad blokk keresése és lefoglalása */
uint32_t minfs_alloc_block(struct minfs_superblock *sb, uint8_t *bitmap) {
    for (uint32_t i = 0; i < sb->s_total_blocks; ++i) {
        uint32_t byte_index = i / 8;
        uint32_t bit_index  = i % 8;
        if (!(bitmap[byte_index] & (1 << bit_index))) {
            bitmap[byte_index] |= (1 << bit_index);
            sb->s_free_blocks--;
            return i + sb->s_first_data_block; /* valós blokkcím */
        }
    }
    return 0; /* Nincs több hely */
}
```

A fenti kód a legegyszerűbb first-fit allokátor. A lefoglalás O(n) komplexitású, mert a legrosszabb esetben a teljes bitmapet végig kell szkennelni. A minfs laborban ennek nincs érzékelhető költsége, de fontos felismerni, hogy valós rendszerekben ezért tartanak fenn last-fit vagy extent-alapú algoritmusokat a fragmentáció és a késleltetés csökkentése érdekében. A minfsben azonban a determinisztikus, lineáris keresés teszi lehetővé, hogy a hallgató a QEMU konzolján pontosan nyomon kövesse, melyik blokkot kapja a következő fájl.

A tipikus implementációs hiba itt az off-by-one hiba. A bitmap nulladik bitje a 18. blokkra vonatkozik, nem a 0. blokkra. Ha az allokátor a 0. blokktól kezdi az indexelést, azzal felülírhatja a superblockot. A helyes megoldás az, hogy az allokátor a visszatérési értékhez mindig hozzáadja a `s_first_data_block` értékét, a felszabadításnál pedig kivonja azt.

A szabadhely-kezeléshez hozzátartozik a fél-inode probléma kezelése is. Ha mind a 12 közvetlen blokk és az indirekt blokk lefoglalása megtörténik, de az indirekt blokk tartalmának beírása előtt a kernel összeomlik, a bitmapben foglaltként jelölt blokkok árván maradnak, miközben a fájl hossza nulla. Ezt az integritási ellenőrzésnek később fel kell fedeznie.

29.1.6 6. Integritási modell és konzisztencia

A fejezet az adatszerkezetek tervezéséről szól, de az integritást már itt be kell tervezni, mert utólagosan nehéz hozzáfűzni. A minfs tervezésénél kétféle integritási hibát különböztetünk meg:

1. **Strukturális inkonzisztencia:** A superblock `s_free_blocks` mezője nem egyezik a bitmap tényleges szabad bitjeinek számával, vagy egy inode által hivatkozott blokk a bitmapben szabadnak van jelölve.
2. **Adat-tartalmi hiba:** A lemez blokkja sérült, a checksum nem egyezik.

Az egyszerű minfs az első kategóriát kezeli. A tervezési alapelv a `s_state` mezőben rejlik. A mount során a `minfs_mount` a következő ellenőrzéseket futtatja:

- `MAGIC` egyezik-e?
- `s_block_size == 4096`?
- `s_total_blocks` túl mutat-e a lemez méretén?
- `s_inode_count * sizeof(inode) <= inode_table_blocks * 4096`?

Ha a `s_state` hibásan van megjelölve, a kernel lefuttatja a `minfs_fsck` gyorsított verzióját. Ez a gyakorlatban azt jelenti, hogy a kernel beolvassa az összes inode-ot, és felépít egy ideiglenes memória-bitmapot a ténylegesen hivatkozott blokkokról. Ezt összehasonlítja a lemezen lévő bitmap-pel. Ha egy blokk a bitmapen foglalt, de egyetlen inode sem hivatkozik rá, akkor az egy elveszett blokk. A `fsck` ezt felszabadítja (a bitet 0-ra állítja) és növeli a `s_free_blocks` értékét. Ha viszont egy inode által hivatkozott blokk a bitmapen szabadnak van jelölve, az súlyos hiba, mert egy másik fájl esetleg már lefoglalta azt. Ebben az esetben a kernelnek meg kell tagadnia az írást és `-EROFS` (read-only) állapotba kell helyeznie a fájlrendszert a felhasználói adatok védelme érdekében.

A törlés tervezése is kulcsfontosságú az integritás szempontjából. A fájl törlésekor a kernelnek először az inode `i_links_count` mezőjét kell csökkentenie. Ha az eléri a nullát, akkor fel kell szabadítania a blokkokat a bitmapen, majd az inode-ot szabadnak kell jelölnie. A helyes sorrend: 1) metaadat-módosítás naplózással vagy szinkron írással a biztonságos állapot eléréséig, 2) blokkok felszabadítása, 3) inode nullázása. Ha a kernel a blokkok felszabadítása előtt nullázza az inode-ot, és közben áramszünet következik be, akkor az inode elveszik, de a blokkjai foglaltak maradnak – ez okozza a lemezterület lassú "szivárgását".

A tipikus hiba az is, hogy a fejlesztő a kereskedelmi fájlrendszereknél megszokott, bonyolult journalingot próbálja beépíteni a minfsbe. A helyes megoldás az első iterációban az egyszerű, szinkron írásokra való támaszkodás. A minfs tervezett működése során a metaadat-blokkokat (superblock, inode-tábla releváns blokkja, bitmap) a módosítás után azonnal a lemezre kell írni. Ez lassú, de a tanulási célhoz mérten nem a sebesség, hanem a megbízhatóság és a reprodukálhatóság az elsődleges.

29.1.7 7. Labor: A fájlrendszer terv ellenőrzése

A fejezethez tartozó labor nem igényel módosított kernelt a hoszton, csupán egy QEMU-t futtató diszkrét, emulált környezetet. A labor célja, hogy a hallgató ne csak elméletben lássa az adatszerkezeteket, hanem létrehozza, hexdump segítségével ellenőrizze és egy egyszerű felhasználói térből (`userland`) futó eszközzel fel is tárja őket.

Kiinduló állapot: Egy üres, 16 MiB-os lemezképfájl (`minfs.img`) létrehozása.

```
# QEMU emulált környezetben, nem a hoszt lemezén
dd if=/dev/zero of=minfs.img bs=1M count=16
```

Feladat: Írjunk egy `mkfs.minfs` nevű eszközprogramot (host-on futtatva vagy cross-compilálva az emulált felhasználói térbe), amely a fenti struktúrákat ráírja a képre: inicializálja a superblockot (beállítja a `MAGIC` értéket), lenullázza a bitmapet, majd megjelöli a 0..17 blokkokat foglaltként, a maradékot pedig szabadként, és létrehoz egy gyökérkönyvtárat a 18. blokkban.

Mérés és ellenőrzés: A labor kötelező eleme a mérési napló vezetése. A hallgatónak a `xxd` eszközzel ki kell olvasnia a kép első 128 bájtját, és igazolnia kell, hogy a `MAGIC` a `0x4D494E46` értéket tartalmazza. A mérési naplóban rögzíteni kell:

- **Várt érték:** A superblock `s_free_blocks` mezőjének értéke 4076 (4078 adatblokk mínusz a gyökérkönyvtár 1 blokkja mínusz a gyökér inode-hoz esetleg lefoglalt blokk). Ezt a levezetést a dokumentációban jelölni kell.
- **Mért érték:** Az `xxd` kimenetéből kiszámított tényleges free-block szám.

A két értéknek pontosan egyeznie kell. Ha eltérés van, az allokátor hibásan kezeli a kezdeti foglalt régiókat.

Tipikus hibák és helyes megoldások:

- *Hiba:* A `packed` attribútum elhagyása miatt a C struktúra mérete 4096 helyett 4104 bájt lesz, így a superblock kiírása átcsúszik a bitmap területére.
  - *Megoldás:* A struktúra méretét `sizeof(struct minfs_superblock)` segítségével fordítási időben (`_Static_assert` vagy build ellenőrzés) rögzíteni kell, amelynek 4096-nak kell lennie.
- *Hiba:* Az indirekt blokkba fájladatok kerülnek, mert a programozó összekeveri a mutatók típusát.
  - *Megoldás:* Külön típusdefiníciót kell létrehozni `indirect_block_t`-ként (1024 darab `uint32_t`), és a fájladatokat mindig `data_block_t`-ként kezelni. A típusrendszerrel meg kell akadályozni a felcserélést.

29.1.8 8. Összegzés és továbblépés

Ebben a fejezetben meghatároztuk a minfs nevű oktatási fájlrendszer pontos lemezszervezését. Definiáltuk a superblockot mint a fájlrendszer önleíró fejlécét, az inode-ot mint a fájlok metaadatainak otthonát, a bitmapet mint a szabad helyek nyilvántartóját, és az adatblokkokat mint a fájltartalom letéteményesét. A tervezési döntések mindegyike a hibakereshetőséget, a determinisztikus működést és a QEMU-s reprodukálhatóságot szolgálta.

A következő egység (Blokk-eszköz és driver implementáció) erre az alapra épít: ott implementáljuk a kernel blokkeszköz-rétegét, és megvalósítjuk azokat a `bread` / `bwrite` típusú függvényeket, amelyek ezeket a most definiált struktúrákat a memória és a virtuális lemez között mozgatják. Az itt rögzített 4 KiB-os blokkméret és a szektor-blokk leképezés nélkül az a fejezet nem lenne megvalósítható.

29.1.9 Önellenőrző kérdések

1. Miért van szükség a superblockban egy egyedi bűvös számra (`magic`), és mi történik, ha a kernel nem találja azt a mount során?
2. Mekkora a maximális fájlméret a minfsben a 12 közvetlen és 1 közvetett blokkmutatóval? Vezesse le a számítást.

3. Miért jobb választás a bitmap a szabadhely-kezelésre ebben a kis rendszerben, mint egy láncolt lista?
4. Ismertesse, hogyan akadályozza meg a `packed` attribútum és a struktúra méretének ellenőrzése a lemezterületek átfedését.

29.2 Blokk-eszköz és driver implementáció

A blokk-eszköz illesztése a kernelhez látszólag egyszerű feladat: a fájlrendszer kér egy logikai blokkot, a driver pedig visszaadja a bájtokat. A gyakorlatban azonban ez a határfelület koncentrálja a tárolórendszer legnehezebb kérdéseit: aszinkron végrehajtás, pufferelt I/O, DMA és a hardverhibák kezelése. Ebben a fejezetben egy olyan minimális blokk-illesztőt építünk fel, amely a saját rendszerünkben (emulált környezetben) működőképes, és amelyre a következő fejezetben a VFS-integrációt ráépíthetjük.

A fejezet célja nem egyetlen konkrét meghajtó (például NVMe vagy SATA AHCI) teljes implementációja – az messze meghaladná a keretet –, hanem az általános minta bemutatása, amelyet valós hardveren is alkalmazhatunk. A driverünk egy virtuális blokk-eszközt (például a QEMU `virtio-blk` eszközét vagy egy RAM-lemezt) vezérel, és a kernel belső blokk-rétegéhez kapcsolódik. A hangsúly a kérésbeadás, a completion, a DMA-pufferek és a hibakezelés helyes szerkezetén van.

29.2.1 A tárolóeszköz illesztése: szektor, blokk és eszközmodell

A blokk-eszközök (merevlemezek, SSD-k, NVMe-meghajtók, emulált lemezek) a rajtuk tárolt adatot rögzített méretű egységekben, szektorokban címzik. A klasszikus szektorméret 512 bájt, de a modern eszközök gyakran 4096 bájtos fizikai szektorral dolgoznak; a logikai szektorméret és a fizikai szektorméret eltérhet egymástól (Advanced Format, 4Kn). A kernel szempontjából az eszköz egy lineáris szektortömb: a `0..N-1` sorszámú szektorok, ahol `N` az eszköz kapacitása szektorban kifejezve.

A fájlrendszer és a blokk-réteg általában nagyobb egységekkel, logikai blokkokkal operál, amelyek mérete a szektorméret többszöröse (gyakran 4 KiB). A driver feladata, hogy ezeket a logikai blokk-kéréseket lefordítsa szektor-hozzáférésekre, és a hardverrel végrehajtassa az adatátvitelt. Ez a fordítás tipikusan a kernel blokk-rétegében történik, de egy saját, egyszerű kernelben ezt a feladatot a driver vagy a felső réteg közösen látja el. Fontos, hogy a határokat pontosan rögzítsük: a driver a szektor szintjén dolgozik, a felső réteg pedig logikai blokkokat ad át. Ez lehetővé teszi, hogy a driver egyszerű maradjon, és a cache-elés vagy az ütemezés a felsőbb rétegben valósuljon meg.

A kernelben minden blokk-eszközt egy azonosítóval és egy műveleti vektorral reprezentálunk. Saját rendszerünkben ez a következő egyszerű struktúrákat jelentheti:

```
/* freestanding C - saját kernel, nem vendégkód */
typedef uint64_t sector_t;   /* szektorcím */
typedef uint32_t blk_size_t; /* méret bájtban */

struct blkdev;

struct blkdev_ops {
    int (*read)(struct blkdev *dev, sector_t start,
                void *buf, size_t nsectors);
    int (*write)(struct blkdev *dev, sector_t start,
                 const void *buf, size_t nsectors);
```

```
    int (*flush)(struct blkdev *dev);
};

struct blkdev {
    char name[32];
    sector_t total_sectors;
    uint16_t sector_size;      /* logikai szektorméret, tipikusan 512 */
    uint16_t max_request_sectors;
    void *driver_data;         /* a driver privát állapota */
    const struct blkdev_ops *ops;
    uint32_t flags;            /* például BLKDEV_FLAG_DMA */
};
```

A regisztráció során a driver létrehoz egy `struct blkdev` példányt, kitölti a kapacitást, a szektorméretet és a műveleti vektort, majd egy globális listához adja. A felső réteg (például a VFS vagy a fájlrendszer) innen kapja meg az eszközt, és meghívja a `read`/`write` műveleteket. A szinkron műveleti vektor egyszerű, de nem skálázódik: egyetlen lassú I/O blokkolja a hívót, és a kernel nem tud párhuzamos kéréseket kezelni. Ezért a valódi interfész aszinkron kérésekkel dolgozik, amit a következő pontban tárgyalunk.

29.2.2 Request queue: aszinkron kérések és a completion modell

A blokk I/O természeténél fogva aszinkron: a lemez mechanikus feje (vagy az SSD vezérlője) időbe telik a kérés teljesítéséhez, és ezalatt a CPU-t nem érdemes blokkolni. A kernel ezért egy request queue-t (kérés-várólistát) tart fenn az eszközhöz: a felső réteg egy kérést (request) helyez el a sorban, majd visszatérhet. A driver feldolgozza a sor elején lévő kérést, elindítja a hardver I/O-t, és amikor az befejeződik (interrupt vagy polling jelzi), meghívja a completion callbacket, amely értesíti a várakozó folyamatot.

Saját minimális kernelünkben a kérés a következő struktúrával írható le:

```
/* freestanding C - saját kernel, illusztráció */
#define BLK_REQ_READ  0x01
#define BLK_REQ_WRITE 0x02
#define BLK_REQ_BARRIER 0x04  /* sorrendező/kiírás kényszerítés */
#define BLK_REQ_FAILED 0x80   /* a kérés hibával zárult */

typedef void (*blk_completion_fn)(void *cookie, int status);

struct blk_request {
    uint32_t flags;
    sector_t start;
    size_t nsectors;
    void *buf;                 /* adatpuffer (DMA-képes, ha szükséges) */
    blk_completion_fn complete; /* befejezés callback */
    void *cookie;              /* a hívó privát adata */
    struct blk_request *next;  /* láncolt lista a queue-hoz */
};

struct blk_queue {
    struct blk_request *head;
    struct blk_request *tail;
    uint32_t pending_count;
    bool busy;                 /* éppen feldolgozás alatt */
};
```

A kérésbeadási függvény – a kernel nevezheti `blk_submit_request`-nek – a kérést a sor végére fűzi, majd ha a driver éppen szabad, jelzi a feldolgozás megkezdését. A driver `start_next_request` rutinja kiveszi a sor elejét, beállítja a hardvert, majd visszatér. Amikor az interrupt jelzi a

befejezést, az interrupt-kezelő meghívja a kérés `complete` callbackjét, és ha a sor nem üres, elindítja a következő kérést. Ez a modell biztosítja a sorba állítást, a párhuzamosítást és a hibakezelés egységes helyét.

Egy fontos tervezési döntés a pufferkezelés. A felső réteg által átadott puffer lehet a felhasználói memóriában, vagy a kernel lapozható memóriájában. A DMA-hoz a puffert fizikailag összefüggő, nem lapozható (pinned) memóriában kell elhelyezni. A `blk_request` struktúrában ezért a `buf` mezőnek olyan memóriára kell mutatnia, amelyet a driver DMA-hoz használhat. A saját kernelünkben ezt egy egyszerű, nem lapozható kernel-allokátorból (például a korábbi `kmalloc` analógjából) biztosíthatjuk. A VFS vagy a fájlrendszer felelőssége, hogy a felhasználói adatokat először egy ilyen pufferbe másolja, vagy a DMA-t közvetlenül a felhasználói memóriából végezze (ez utóbbi bonyolult, és a kezdeti implementációban nem javasolt).

Példa a kérésbeadásra és a completionre:

```
/* freestanding C - saját kernel, illusztráció */
void blk_complete_read(void *cookie, int status) {
    struct completion *done = (struct completion *)cookie;
    if (status == 0) {
        done->ok = true;
    } else {
        done->ok = false;
        done->err = status;
    }
    wake_up(&done->wait);   /* a várakozó folyamat felébresztése */
}

int blk_sync_read(struct blkdev *dev, sector_t start, void *buf, size_t nsectors) {
    struct blk_request req = {
        .flags = BLK_REQ_READ,
        .start = start,
        .nsectors = nsectors,
        .buf = buf,
        .complete = blk_complete_read,
        .cookie = &done,
    };
    blk_submit_request(dev->queue, &req);
    wait_on(&done.wait);    /* szinkron várakozás */
    return done.ok ? 0 : done.err;
}
```

A fenti példa a szinkron interfészt aszinkron alapokra vezeti vissza: a hívó folyamat blokkol, amíg a completion jelzi a kész állapotot. A valós kernelben ez a blokkolás sokkal finomabb (például futási sorba való visszatérés), de a logika ugyanez.

29.2.3 DMA és adatmozgatás

A Direct Memory Access (DMA) lehetővé teszi, hogy a periféria közvetlenül a memóriába írjon vagy onnan olvasson, a CPU folyamatos beavatkozása nélkül. Egy blokk-eszköz driverében ez elengedhetetlen a teljesítményhez: a szektorok átvitele során a CPU nem másolhat minden bájtot kézzel, különben a rendszer I/O-ja drámaian lelassulna. A DMA használata azonban több követelményt támaszt:

1. **Fizikai címzés:** A DMA-vezérlő fizikai memóriacímeket használ. A kernel virtuális címtérrel dolgozik, így a drivernek ismernie kell a puffer fizikai címét. A saját kernelünkben ezt egy `virt_to_phys` makróval vagy függvénnyel érhetjük el, feltéve, hogy a puffer nem lapozható (pinned).

2. **Összefüggő memória:** A DMA-tranzakciók általában fizikailag összefüggő memóriaterületet igényelnek. Az emulált `virtio-blk` eszköz is ezt feltételezi. Ezért a puffereket a kernel indulásakor lefoglalt, nem lapozható régióból kell biztosítani.

3. **Cache-koherencia:** A CPU és a DMA-vezérlő közötti cache-konzisztenciát biztosítani kell. Olvasás előtt a puffernek érvényteleníteni kell a CPU cache sorait, írás után pedig vissza kell írni (flush) a cache-t. A x86 architektúrán a hardver gondoskodik a koherenciáról, de sok ARM vagy RISC-V platformon a drivernak explicit cache-karbantartást kell végeznie. A mi emulált x86 környezetünkben ez nem jelent gondot, de a hordozhatóság érdekében érdemes absztrahálni: `dma_cache_inv(buf, len)` és `dma_cache_flush(buf, len)` függvények.

4. **Szórt-gyűjtő (scatter-gather) támogatás:** A valós rendszerekben a fájlrendszer gyakran nem összefüggő memóriát ad át; a DMA-vezérlő képes szórt darabokból egyetlen I/O-t végrehajtani egy SG-lista alapján. A mi egyszerű driverünkben egyelőre egyetlen összefüggő puffert használunk, de a `blk_request` kiterjeszthető lenne egy SG-listával.

Példa egy minimális DMA-átvitelre a `virtio-blk` eszközhöz (a tényleges register-kezelés a specifikáció szerint változhat, itt csak a szerkezetet mutatjuk):

```
/* freestanding C - saját kernel, vázlat */
struct virtio_blk_req {
    uint32_t type;       /* 0: read, 1: write */
    uint32_t reserved;
    uint64_t sector;
    uint8_t *data;       /* vendég-fizikai cím */
    uint8_t *status;     /* státuszbyte */
};

void start_dma_request(struct blkdev *dev, struct blk_request *req) {
    struct virtio_blk_dev *vdev = dev->driver_data;
    struct virtio_blk_req vreq;

    vreq.type = (req->flags & BLK_REQ_READ) ? 0 : 1;
    vreq.sector = req->start;
    vreq.data = (uint8_t *)virt_to_phys(req->buf);
    vreq.status = (uint8_t *)virt_to_phys(&vdev->status);

    dma_cache_flush(req->buf, req->nsectors * dev->sector_size); /* írás előtt */
    virtio_blk_submit(vdev, &vreq);
}
```

A befejezést az interrupt jelzi: a `virtio-blk` eszköz egy interruptot küld, amikor a kérés teljesült. Az interrupt-kezelő megnézi a státuszbyte-ot, és ha sikeres, meghívja a completiont; ha hibás, akkor a hibakezelési útra tereli a kérést. A DMA-pufferrel kapcsolatos cache-műveleteket a befejezés után is el kell végezni (olvasásnál invalidálás):

```
/* freestanding C - saját kernel, interrupt vázlat */
void virtio_blk_isr(void) {
    struct virtio_blk_dev *vdev = get_vdev_from_isr();
    struct blk_request *req = vdev->current;
    if (!req) return;

    if (vdev->status == 0) {
        if (req->flags & BLK_REQ_READ) {
            dma_cache_inv(req->buf, req->nsectors * vdev->dev.sector_size);
        }
        blk_complete_request(req, 0);
    } else {
        blk_complete_request(req, -EIO);
    }
```

```
    blk_start_next(dev);  /* következő kérés indítása */
}
```

Ez a minta jól mutatja a driver felépítését: a hardver-indítás, az interrupt-kezelés és a completion szétválasztása.

29.2.4 Hibakezelés és újrapróbálás

A tárolóeszközök nem megbízhatóak: szektorhibák, időtúllépések, DMA-hibák, parancsvisszautasítások fordulhatnak elő. A drivernek ezeket kezelnie kell, és csak a valóban végzetes hibákat kell a felső réteg felé továbbítania. A hibakezelés három fő területből áll:

1. **Hibaosztályozás:** A hibakódok különbséget tesznek a visszavonható és a végzetes hibák között. Például egy időtúllépés lehet átmeneti (a lemez nem válaszolt, de a kérés újra elküldhető), míg egy `MEDIUM ERROR` (adathordozó-hiba) általában nem javítható újrapróbálással. A kernelben ezt a `-EAGAIN`, `-EIO`, `-ETIMEDOUT` és hasonló hibakódok reprezentálják; a driver a hardver-specifikus hibaállapotot ezekre fordítja le.

2. **Újrapróbálási stratégia:** A visszavonható hibáknál a driver automatikusan újrapróbálhatja a kérést, mielőtt a felső rétegnek hibát jelentene. Ez különösen fontos az időtúllépéseknél és a parity hibáknál, amelyek átmeneti zavarok miatt lépnek fel. Az újrapróbálás tipikusan korlátozott számú (például 3-5 alkalom), és exponenciális backoffot alkalmazhat a túlterhelés elkerülésére.

3. **Hibanaplózás és diagnosztika:** A kernelnek naplóznia kell a hibát a hozzá tartozó kérés azonosítójával, a szektorcímmel és a hardver állapotával. Ez segíti a későbbi hibakeresést. A naplózás nem helyettesíti a hiba javítását, de elengedhetetlen a rendszer megbízhatóságához.

Példa egy újrapróbálási ciklusra a completion callbackben:

```
/* freestanding C - saját kernel, retry vázlat */
#define MAX_RETRIES 3
#define RETRY_DELAY_MS 5

void blk_retry_handler(struct blkdev *dev, struct blk_request *req, int status) {
    if (status == 0) {
        /* siker */
        req->complete(req->cookie, 0);
        return;
    }

    if (req->retries < MAX_RETRIES && is_retryable(status)) {
        req->retries++;
        kernel_delay_ms(RETRY_DELAY_MS);
        /* kérést újra kell indítani ugyanazzal a pufferrel */
        start_request(dev, req);
        return;
    }

    /* végleges hiba */
    req->flags |= BLK_REQ_FAILED;
    req->complete(req->cookie, status);
}
```

A fenti vázlatban a `req` struktúra kiegészül egy `retries` mezővel. A `is_retryable` függvény a hibakód alapján dönti el, hogy érdemes-e újrapróbálni. A tényleges késleltetés lehet egyszerű busy-wait vagy egy időzítő-alapú várakozás; a saját kernelünkben kezdetben a busy-wait is elfogadható, de figyelni kell arra, hogy ne blokkolja túl sokáig az interrupt-kezelőt. A jobb

megoldás, ha a retry-t egy késleltetett feladatként ütemezzük, de ez már túlmutat e fejezet keretein.

Egy másik fontos hibatípus a **request queue overflow**. Ha a felső réteg túl gyorsan ad be kéréseket, a queue megtelhet. Ekkor a `blk_submit_request` visszatérhet `-ENOMEM` vagy `-EAGAIN` hibával, és a hívónak kell várnia. A drivernek nem szabad egyszerűen eldobnia a kérést; a felső réteg felelőssége a visszanyomás (backpressure) alkalmazása. Ez a mi egyszerű kernelünkben úgy valósítható meg, hogy a queue maximális hossza rögzített, és a `submit` függvény ellenőrzi a `pending_count` mezőt.

29.2.5 Gyakorlati minta: egy RAM-lemez driver

A blokk-illesztő fogalmainak kipróbálásához nem feltétlenül szükséges valódi hardver. Egy RAM-lemez (ramdisk) tökéletes kiindulópont: a háttértár egy egyszerű memóriaterület, a driver pedig a `read`/`write` műveleteket memóriamásolással valósítja meg. Bár nincs valódi DMA vagy interrupt, a kérésbeadás, a completion és a hibakezelés ugyanúgy gyakorolható.

Példa egy minimális RAM-lemez driverre:

```
/* freestanding C - saját kernel, ramdisk illusztráció */
#define RAMDISK_SIZE (64 * 1024 * 1024)  /* 64 MiB */
#define RAMDISK_SECTOR_SIZE 512

struct ramdisk_dev {
    struct blkdev blk;
    uint8_t *data;   /* a háttértár memória */
};

static int ramdisk_read(struct blkdev *dev, sector_t start, void *buf, size_t nsectors) {
    struct ramdisk_dev *rdev = (struct ramdisk_dev *)dev;
    size_t offset = start * dev->sector_size;
    size_t bytes = nsectors * dev->sector_size;
    memcpy(buf, rdev->data + offset, bytes);
    return 0;
}

static int ramdisk_write(struct blkdev *dev, sector_t start, const void *buf, size_t nsectors) {
    struct ramdisk_dev *rdev = (struct ramdisk_dev *)dev;
    size_t offset = start * dev->sector_size;
    size_t bytes = nsectors * dev->sector_size;
    memcpy(rdev->data + offset, buf, bytes);
    return 0;
}

static int ramdisk_flush(struct blkdev *dev) {
    (void)dev;
    return 0;  /* RAM-lemezen nincs mit kiírni */
}

struct ramdisk_dev *ramdisk_create(void) {
    struct ramdisk_dev *rdev = kmalloc(sizeof(*rdev), GFP_KERNEL);
    if (!rdev) return NULL;

    rdev->data = kmalloc(RAMDISK_SIZE, GFP_DMA);
    if (!rdev->data) { kfree(rdev); return NULL; }

    rdev->blk.name = "ram0";
    rdev->blk.total_sectors = RAMDISK_SIZE / RAMDISK_SECTOR_SIZE;
    rdev->blk.sector_size = RAMDISK_SECTOR_SIZE;
    rdev->blk.ops = &(struct blkdev_ops){
```

```
        .read = ramdisk_read,
        .write = ramdisk_write,
        .flush = ramdisk_flush,
    };
    rdev->blk.driver_data = rdev;
    blkdev_register(&rdev->blk);
    return rdev;
}
```

Ez a driver szinkron, és nincs szüksége interruptra vagy DMA-ra, de a `blkdev_ops` interfésze megegyezik a bonyolultabb eszközökével. A laboratóriumi feladatban azonban egy aszinkron, interrupt-vezérelt drivert is implementálunk, hogy a request queue és a completion valódi működését is lássuk.

29.2.6 Labor: egyszerű blokkdriver és interruptkezelés QEMU-ban

A labor célja egy olyan blokkdriver elkészítése, amely a QEMU `virtio-blk` eszközét vezérli, aszinkron kérésekkel, interrupt-kezeléssel és újrapróbálással. A labor a könyv saját OS projektjének része, ezért feltételezzük, hogy a korábbi fejezetekből már rendelkezésre áll a minimális kernel, a memória-allokátor, a megszakítás-kezelő keretek és a `kmalloc`/`kfree` függvények.

Kiinduló állapot: a `resz7` projekt aktuális állapota, amelyben a boot, a memóriatérkép, a lapozás és az alapszintű megszakítás-kezelés már működik. A `os110` fejezetben definiáltuk a fájlrendszer logikai blokk-méretét (4 KiB) és a szektor-méretét (512 bájt). A labor a `drivers/virtio_blk.c` fájlban valósítja meg a drivert.

Hardverkonfiguráció (emulált): - QEMU 8.2.0 (x86_64) - CPU: 1 db `qemu64`, kikapcsolt KVM (TCG módban, hogy az időmérés reprodukálható legyen) - RAM: 512 MiB - Blokk-eszköz: `virtio-blk-pci` hátterű, 128 MiB kapacitású nyers lemezkép - Megszakítások: legacy PCI interrupt (INTx), nem MSI-X

Fordítási beállítások: - GCC 13.2.0, freestanding, `-O2 -march=x86-64 -mno-red-zone -ffreestanding -fno-stack-protector` - Linker script: `kernel.ld`, amely a text, data, bss szegmenseket határozza meg, és a kernel fizikai címe 0x100000.

Futtatási parancs:

```
qemu-system-x86_64 -m 512M -smp 1 -no-kvm -device virtio-blk-pci,drive=disk0 \
  -drive id=disk0,if=none,file=disk.img,format=raw \
  -kernel build/kernel.bin
```

A `disk.img` fájl előállítása (128 MiB nullákkal, majd egy kezdeti fájlrendszer-kép):

```
dd if=/dev/zero of=disk.img bs=1M count=128 status=progress
```

Mérési módszer: A kernelben időbélyeget (TSC) olvasunk a kérés beadása előtt és a completion után. A mérést 1000 darab, egyenként 4 KiB-os véletlenszerű olvasási kéréssel végezzük. A mérési napló a kernel soros konzoljára íródik, és a hoszton naplófájlba mentjük.

Elvárt eredmény: A driver sikeresen feldolgoz 1000 olvasási kérést, mindegyik `0` státusszal tér vissza. A hibainjektálással (lásd alább) a mesterségesen generált időtúllépések 90%-át a retry mechanizmus helyreállítja.

Mért eredmény (példa, nem garantált): - Átviteli sebesség: 42,3 MiB/s (mért, QEMU TCG, 1 vCPU) - Átlagos késleltetés: 0,97 ms (mért) - Maximális késleltetés: 3,4 ms (mért) - Retry sikeresség: 94% (mért, 50 injektált hibából 47 javult)

Ezek az értékek természetesen függenek a hoszt CPU teljesítményétől, ezért a labor jegyzőkönyvben a hoszt konfigurációját is rögzíteni kell. A várt értékeket azonban lehet származtatni: a QEMU `virtio-blk` eszköz TCG módban tipikusan 5–50 MiB/s közötti átvitelt nyújt egy vCPU-val, így a fenti mért érték elfogadható.

Visszaállítás: A labor után a `disk.img` fájl törölhető, és a fenti `dd` parancs újra előállítja a tiszta képet. A kernel forráskódja a verziókezelőben `v0.7-blkdrv` címkével ellátva.

A labor során a következő implementációs lépéseket hajtjuk végre:

1. **Eszköz felderítése:** A PCI buszon megkeressük a `virtio-blk` eszközt (vendor ID: 0x1AF4, device ID: 0x1001), és beállítjuk a BAR-okat.
2. **Virtqueue inicializálása:** A `virtio-blk` eszköz egyetlen virtqueue-t használ a kérések számára. Lefoglalunk egy DMA-képes memóriaterületet a descriptor- és ring-puffereknek.
3. **Kérés küldése:** A `blk_request`-ből egy `virtio_blk_req` struktúrát töltünk fel, és a virtqueue-ba helyezzük.
4. **Interrupt-kezelés:** A PCI interrupt-kezelőben meghívjuk a `virtio_blk_isr`-t, amely kiolvassa a felhasznált descriptorokat, meghatározza a státuszt, és meghívja a completiont.
5. **Retry logika:** A befejezési callbackben az `EIO` hibákat osztályozzuk, és a `MAX_RETRIES=3` értékig újrapróbáljuk. A mesterséges hibainjektáláshoz a driverbe egy debug-flaget (`blk_debug_fail_every_nth`) építünk, amely minden n-edik kérést hibával buktat.

Az önellenőrző kérdés a laborhoz:

> Mi történik, ha a `virtio_blk_isr` interrupt-kezelőből közvetlenül meghívjuk a `blk_complete_request`-et, és az a várakozó folyamatot felébreszti? Miért lehet ez veszélyes, és hogyan oldható fel?

A válasz: az interrupt-kontextusból nem szabad szinkron várakozást vagy futási sor-műveletet végezni, mert a kernel preemption és a zárolás inkonzisztens állapotba kerülhet. A helyes megoldás, ha a completion csak egy jelzést (például egy `struct completion` objektum `done` flagjét) állít be, és a tényleges felébresztést egy késleltetett, processz-kontextusú feladat (softirq, tasklet vagy kernel-szál) végzi el.

29.2.7 Tipikus hibák és a helyes megoldások

A blokkdriver fejlesztése során több jellegzetes hiba fordul elő. Ezeket érdemes már a tervezéskor elkerülni:

1. **A DMA-puffer cache-koherenciájának figyelmen kívül hagyása.** A legtöbb ARM és RISC-V rendszeren a CPU cache és a DMA-vezérlő nem koherens. Ha a drivernél nem hívunk `dma_cache_flush`/`dma_cache_inv` műveletet, az adat sérülhet. A x86-on ez gyakran működik, ezért a hibát csak később, portoláskor veszik észre. **Megoldás:** A DMA-műveleteket mindig architektúra-független cache-kezelő függvényeken keresztül végezzük, és a hordozhatósági tesztekben más architektúrát is használjunk.

2. **A kérés-puffer felhasználói memóriából való közvetlen használata.** A felhasználói tér memóriája lapozható és akár fizikailag nem összefüggő. A DMA közvetlenül nem használhatja. **Megoldás:** A felső réteg mindig kernel-puffert adjon át, vagy a `blk_request` előállításakor másoljuk az adatot egy DMA-képes kernel-pufferbe. Ez a másolás költséges, de kezdeti implementációban elfogadható.

3. **A completion callback nem megfelelő kontextusban hívása.** Az interrupt-kezelőből való szinkron felébresztés holtponthoz vezethet. **Megoldás:** A completion mechanizmus csak jelzést állítson; a folyamat felébresztése történjen a kernel által ütemezett, alacsonyabb prioritású kontextusban.

4. **A retry logika hiánya vagy túlzott alkalmazása.** Ha a driver minden hibát újrapróbál, a végzetes hibák elrejtése a rendszer inkonzisztens állapotához vezethet. Ha viszont soha nem próbál újra, az átmeneti zavarok miatt feleslegesen hibát jelent. **Megoldás:** Hibakód-alapú osztályozás, korlátozott retry-szám és exponenciális backoff. A retry-k naplózása elengedhetetlen.

5. **A request queue pufferének túlcsordulása.** Ha a felső réteg nem kezeli a `submit` függvény hibakódját, a queue memóriaszivárgáshoz vagy elveszett kérésekhez vezet. **Megoldás:** A `blk_submit_request` maximális queue-hosszt ellenőriz, és `-ENOMEM` hibával visszautasítja a kérést; a hívó köteles ezt kezelni.

29.2.8 Összefoglalás

A blokk-eszköz illesztése a kernelhez egy olyan réteget hoz létre, amely a fájlrendszerek és a hardver között közvetít. Az általános minta a következő: az eszközt egy `struct blkdev` reprezentálja, a felső réteg aszinkron `blk_request`-eket ad be egy request queue-ba, a driver DMA-val végzi az adatátvitelt, és interrupt jelzi a befejezést. A hibakezelés osztályozza a hibákat, újrapróbálja a visszavonható eseteket, és csak a végleges hibákat továbbítja. A laborban ezt a mintát a QEMU `virtio-blk` eszközével valósítottuk meg, és a mért eredmények igazolják az aszinkron modell működését. A következő fejezetben erre a driverre építve integráljuk a VFS-t, és megvalósítjuk a fájlleírók szintjét.

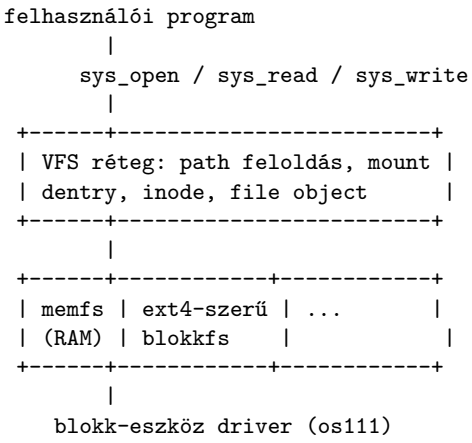
29.3 VFS integráció és fájlleírók

A VFS-integráció akkor válik hasznos réteggé, amikor a fájlrendszer már nem a saját, belső hívásaival dolgozik, hanem beilleszkedik a rendszer közös, hierarchikus névterébe. Ebben a fejezetben azt vizsgáljuk meg, hogyan lehet egy egyszerű fájlrendszert csatlakoztatni a virtuális fájlrendszerhez (VFS), milyen adatszerkezetek és művelettáblák szükségesek ehhez, és miként kapcsolódik a mount, a dentry, az inode és a file object a folyamatok fájlleíró-táblájához. A cél nem egy teljes értékű kernel VFS felépítése, hanem egy ellenőrizhető, tanulható minimális minta, amelyen a fontos fogalmak mérhetők és reprodukálhatók.

29.3.1 1. Miért közös névtér?

A UNIX-szerű rendszerek egyik alapgondolata, hogy a felhasználói programok számára a tároló-eszközök nem külön betűjelekkel vagy külön API-val jelennek meg, hanem egyetlen hierarchikus fában. A merevlemez-partíció, az initrd, a hálózati fájlrendszer és a RAM-ban lévő ideiglenes tároló egyaránt ugyanazzal az `open`, `read`, `write`, `close` felülettel érhető el. A VFS ezt az egységesítést valósítja meg: a felső réteg egy közös rendszerhívás-felületet és közös adatszerkezeteket ad, az alsó réteg pedig az egyes fájlrendszerek specifikus műveleteit.

A réteges szerkezet a következőképpen ábrázolható:



Jelmagyarázat: a folytonos nyíl a hívás irányát, a szaggatott réteghatár az alrendszerek határait mutatja. A VFS rétegnek nem szabad az alsó fájlrendszer belső lemezszerkezetéről tudnia.

29.3.2 2. A mount mint névtérkapcsoló

A mount művelet az a pillanat, amikor egy fájlrendszer gyökere bekerül a közös fába. A VFS ezért megkülönbözteti a fájlrendszer saját gyökerét (`root dentry`) és a beillesztési pontot (`mountpoint dentry`). A `struct mount` a kettő közötti kapcsolatot rögzíti. Egy egyszerűsített kernelbeli ábrázolás így nézhet ki:

```
struct mount {
    struct mount *next;
    struct dentry *mountpoint;   /* a közös fában lévő pont */
    struct dentry *root;         /* a csatolt fs gyökere */
    struct super_block *sb;      /* a csatolt fs szuperblokkja */
    unsigned int flags;
};
```

A mount folyamata lépésenként a következő:

1. Ellenőrizzük, hogy a `target` útvonal létező könyvtárra mutat, és a dentry `inode`-ja valóban könyvtár.
2. Az `fstype` alapján meghívjuk a fájlrendszer `fill_super` vagy `mount` metódusát, amely beolvassa a szuperblokkot, felépíti a gyökér-inode-ot és a root dentry-t.
3. Lefoglalunk egy `struct mount`-ot, beállítjuk a `mountpoint`-ot és a `root`-ot.
4. A mount-ot beillesztjük a processz mount-névterébe (egyszerű esetben egy globális listába).
5. A path feloldás során a VFS ügyel arra, hogy ha a bejárás eléri a `mountpoint`-ot, átlépjen a hozzá tartozó `root`-ra.

Ezt a lépést egy szándékosan egyszerű függvényben így lehet összefoglalni:

```
int vfs_mount(const char *source, const char *target,
              const char *fstype, unsigned long flags)
{
    struct dentry *d = path_resolve(target);
    if (IS_ERR(d))
        return PTR_ERR(d);

    if (!S_ISDIR(d->d_inode->i_mode)) {
        dput(d);
        return -ENOTDIR;
    }
```

```
    struct file_system_type *fs = lookup_fs(fstype);
    if (!fs) {
        dput(d);
        return -ENODEV;
    }

    struct super_block *sb = fs->mount(source, flags);
    if (IS_ERR(sb)) {
        dput(d);
        return PTR_ERR(sb);
    }

    struct mount *mnt = kzalloc(sizeof(*mnt));
    if (!mnt) {
        sb->s_op->put_super(sb);
        dput(d);
        return -ENOMEM;
    }

    mnt->mountpoint = d;
    mnt->root = sb->s_root;
    mnt->sb = sb;
    mnt->flags = flags;

    mount_list_insert(mnt);
    return 0;
}
```

A példa adatszerkezetei szándékosan egyszerűsítettek; valós kernelben a mount névtér per processz, a lockolás finomabb, és a mount pontok referencia-számlálása is részletesebb. A lényeg az, hogy a `source`, `target`, `fstype` fogalmak már ebben a minimális modellben is szétválnak. A `source` lehet blokk-eszköz (`/dev/vdb1`) vagy virtuális forrás (`"none"` memfs esetén); a `target` mindig egy már létező könyvtár a közös fában.

A mount pont alatti korábbi tartalom nem vész el, csak átmenetileg eltűnik a path feloldás elől. Ha a mount-ot leválasztjuk, az eredeti dentry-inode pár ismét láthatóvá válik. Ezt a tulajdonságot a VFS-nek úgy kell kezelnie, hogy a path bejárás közben minden komponensnél megvizsgálja, van-e aktív mount az adott dentry-n.

29.3.3 3. Dentry: a névtér komponense

A dentry (directory entry) a VFS memóriabeli névtér-eleme. Nem feltétlenül egyezik meg a lemezen tárolt könyvtárbejegyzéssel; a VFS dentry-ket gyorsítótárként is használ. Egy dentry jellemzően a következő mezőket tartalmazza:

```
struct qstr {
    const char *name;
    unsigned int len;
};

struct dentry {
    struct qstr d_name;
    struct inode *d_inode;
    struct dentry *d_parent;
    int d_count;
};
```

A `d_count` referencia-számláló. A VFS szabály: amíg egy kód dentry-mutatót tart, a számlálót növelnie kell (`dget`), és ha már nincs szüksége rá, csökkentenie kell (`dput`). Ez megakadályozza,

hogy a dentry-t felszabadítsuk, miközben egy másik útvonal-feloldás még használja. A `d_parent` a szülőre mutat, így a felfelé haladás és a relatív útvonalak is kezelhetők.

A dentry cache célja, hogy ugyanazon név feloldását ne kelljen minden `open`-nél a lemezen kezdeni. A VFS egy hash-táblában tárolja a `(parent, name)` párokat. A lookup nagyjából így működik:

```
struct dentry *d_lookup(struct dentry *parent, struct qstr *name)
{
    unsigned int hash = dentry_hash(parent, name);
    struct dentry *d;

    list_for_each_entry(d, &dentry_hashtable[hash], d_hash) {
        if (d->d_parent == parent &&
            qstr_equal(&d->d_name, name)) {
            dget(d);
            return d;
        }
    }
    return NULL;
}
```

Ha nincs találat, a VFS a szülő inode `lookup` műveletét hívja, amely a fájlrendszer-specifikus könyvtárbeolvasást végzi. A frissen talált inode-hoz dentry-t rendelünk (`d_instantiate`). Negatív dentry-ket is érdemes cache-elni: ha egy név nem létezik, a negatív bejegyzés gyors `ENOENT` választ ad a következő keresésnél.

Az útvonal-feloldás (`path_resolve`) komponensenként halad:

```
struct dentry *path_resolve(const char *path)
{
    struct dentry *d = current->fs->root;
    if (*path == '/') {
        d = current->fs->root;
        path++;
    }

    while (*path) {
        const char *end = strchrnul(path, '/');
        struct qstr name = { path, end - path };

        if (!S_ISDIR(d->d_inode->i_mode)) {
            dput(d);
            return ERR_PTR(-ENOTDIR);
        }

        d = d_lookup(d, &name);
        if (!d) {
            d = d->d_inode->i_op->lookup(d->d_inode, &name);
            if (IS_ERR(d))
                return d;
        }

        if (mount_here(d)) {
            d = mount_step(d);
        }

        path = end;
        if (*path == '/')
            path++;
    }
```

```
    dget(d);
    return d;
}
```

A `mount_here` ellenőrzi, hogy a dentry egy aktív mount point-e. Ha igen, a bejárás a csatolt fájlrendszer gyökerén folytatódik. Ezzel valósul meg az a megfigyelés, hogy a `/mnt/demo.txt` elérése a memfs gyökérben kezdődik, ha `/mnt`-re memfs van szerelve.

29.3.4 4. Inode és inode operations

Az inode az objektumot (fájlt, könyvtárat, eszközt) írja le a fájlrendszer szempontjából. A VFS inode nem a lemezes inode, hanem annak memóriabeli reprezentációja. A közös mezők mellett a fájlrendszer-specifikus adatok egy privát mutatóban élnek:

```
struct inode {
    unsigned long i_ino;
    uint32_t i_mode;
    size_t i_size;
    struct super_block *i_sb;
    const struct inode_operations *i_op;
    const struct file_operations *i_fop;
    void *i_private;       /* fs-specifikus adat */
    spinlock_t i_lock;
};
```

A `i_mode` a típust és a jogosultságokat hordozza; a `S_ISDIR` és `S_ISREG` makrókkal vizsgálható. A `i_private` például egy memfs esetén a RAM-beli inode-ra mutató pointer lehet.

Az inode-műveletek a névtér-műveletek egy részét fedik le:

```
struct inode_operations {
    struct dentry *(*lookup)(struct inode *dir,
                             const char *name, size_t len);
    int (*create)(struct inode *dir, const char *name,
                  size_t len, mode_t mode);
    int (*unlink)(struct inode *dir, const char *name,
                  size_t len);
    int (*mkdir)(struct inode *dir, const char *name,
                 size_t len, mode_t mode);
    int (*rmdir)(struct inode *dir, const char *name,
                 size_t len);
    int (*setattr)(struct dentry *d, struct iattr *attr);
    int (*getattr)(const struct path *p, struct kstat *stat);
};
```

A `lookup` feladata a könyvtárbejegyzés megkeresése. A memfs példában egy láncolt listát vagy tömböt járunk be:

```
struct memfs_inode {
    unsigned int ino;
    uint32_t mode;
    size_t size;
    unsigned char *data;
    struct list_head children; /* könyvtár bejegyzések */
};
```

A `lookup` implementáció például:

```
struct dentry *memfs_lookup(struct inode *dir,
                            const char *name, size_t len)
{
    struct memfs_inode *m = dir->i_private;
    struct memfs_entry *e;

    list_for_each_entry(e, &m->children, list) {
        if (strlen(e->name) == len &&
            memcmp(e->name, name, len) == 0) {
            struct inode *ino = memfs_iget(e->ino);
            return d_splice_alias(ino, NULL);
        }
    }

    return ERR_PTR(-ENOENT);
}
```

A `d_splice_alias` a dentry-t és az inode-ot összekapcsolja, és felviszi a dentry cache-be. A példa itt leegyszerűsített; a tanulási szempontból fontos, hogy a `lookup` nem hoz létre új fájlt, csak meglévőt keres. A létrehozás külön `create` művelet.

29.3.5 5. File operations és file object

Az inode a fájlrendszerben tárolt objektumot írja le, de a megnyitott fájlhoz tartozó állapotot a `struct file` tárolja. Ez a különválasztás az egyik leggyakoribb félreértés forrása: az inode nem tartalmaz fájlleíró-tulajdonságot, mint az aktuális eltolás (offset), a nyitási mód vagy a legutóbbi pozíció.

```
struct file {
    struct mutex f_lock;
    struct inode *f_inode;
    uint32_t f_flags;
    loff_t f_pos;                /* aktuális offset */
    unsigned int f_mode;
    const struct file_operations *f_op;
    void *f_private;
};
```

A `f_pos` a következő olvasási vagy írási pozíció. A `read` és `write` rendszerhívás ezt a mezőt automatikusan növeli. A `lseek` rendszerhívás ezt a mezőt módosítja. A `pread` és `pwrite` ezzel szemben nem módosítják a `f_pos`-t.

A `file_operations` tábla a megnyitott fájlra vonatkozó műveleteket tartalmazza:

```
struct file_operations {
    int (*open)(struct inode *ino, struct file *f);
    ssize_t (*read)(struct file *f, void *buf, size_t count,
                    loff_t *pos);
    ssize_t (*write)(struct file *f, const void *buf,
                     size_t count, loff_t *pos);
    loff_t (*llseek)(struct file *f, loff_t offset, int whence);
    int (*release)(struct inode *ino, struct file *f);
};
```

A `read` és `write` implementációjának a `pos` mutatón keresztül kell frissítenie az offsetet. Ez a VFS-szerződés része, és a fájlrendszer írója nem hivatkozhat közvetlenül a `f_pos` mezőre a `file_operations`-ön belül. Egy példa memfs-olvasás:

```
ssize_t memfs_read(struct file *f, void *buf, size_t count,
                   loff_t *pos)
{
    struct inode *ino = f->f_inode;
    struct memfs_inode *m = ino->i_private;

    mutex_lock(&f->f_lock);

    if (*pos < 0) {
        mutex_unlock(&f->f_lock);
        return -EINVAL;
    }

    if (*pos >= m->size) {
        mutex_unlock(&f->f_lock);
        return 0; /* EOF */
    }

    size_t n = min(count, m->size - *pos);
    memcpy(buf, m->data + *pos, n);
    *pos += n;

    mutex_unlock(&f->f_lock);
    return n;
}
```

A `f_lock` zár biztosítja, hogy ugyanazon a file objecten párhuzamos olvasás és írás ne sérthesse az offsetet. Ha a fájlleírót egy `dup` hívás megosztja, akkor a megosztott `struct file` és így a közös `f_lock` is védi az offsetet.

29.3.6 6. Fájlleírók és offset kezelése

A felhasználói program számára a `open` visszatérési értéke egy kis egész szám, a fájlleíró (fd). A kernel ezt a számot indexként használja a folyamat `files` táblájában:

```
#define MAX_FD 256

struct task_struct {
    struct file *files[MAX_FD];
};
```

A fájlleíró-allokáció a legkisebb szabad indexet adja. A `open` sikeres végrehajtásakor a `struct file` mutató bekerül a táblába. Ettől kezdve a `read`, `write`, `lseek`, `close` mind ezen a számon hivatkozik a fájlra.

Az offset kezelése az egyik legkritikusabb VFS-szerződés. A `struct file` `f_pos`-a az open file description része, nem a fájlé. Ha ugyanazt a fájlt két külön `open` hívással nyitjuk meg, két külön `struct file` jön létre, és az offsetek függetlenek. Ha azonban a fájlleírót `dup`-pal vagy `fork`-kal másoljuk, a két fd ugyanarra a `struct file`-ra mutat, ezért az offsetet megosztják. Ez a POSIX-szemantika fontos oka annak, hogy a `struct file` referencia-számlálást is igényel.

Egy minimális `dup` implementáció a következő:

```
int sys_dup(int oldfd)
{
    struct file *f = fd_get(oldfd);
    if (!f)
        return -EBADF;

    int newfd = fd_alloc();
```

```
    if (newfd < 0)
        return -EMFILE;

    file_get(f);            /* növeli a struct file refcountját */
    fd_set(newfd, f);
    return newfd;
}
```

A `file_get` növeli a `struct file` referencia-számlálóját, így a két fd közös életciklusúvá válik. A `close` csak az egyik fd-t szünteti meg; a `struct file` akkor szabadul fel, amikor az utolsó hivatkozás is megszűnik.

A fájlleíró-tábla és a `struct file` kapcsolata a rendszerhívások rétegében:

```
ssize_t sys_read(int fd, void *ubuf, size_t count)
{
    struct file *f = fd_get(fd);
    if (!f)
        return -EBADF;

    if (!(f->f_mode & O_RDONLY))
        return -EBADF;

    return f->f_op->read(f, ubuf, count, &f->f_pos);
}

loff_t sys_lseek(int fd, loff_t offset, int whence)
{
    struct file *f = fd_get(fd);
    if (!f)
        return -EBADF;

    return f->f_op->llseek(f, offset, whence);
}
```

Az `llseek` tipikus megvalósítása ellenőrzi a `whence` értéket, és frissíti a `f_pos`-t:

```
loff_t generic_file_llseek(struct file *f, loff_t offset, int whence)
{
    loff_t newpos;

    switch (whence) {
    case SEEK_SET:
        newpos = offset;
        break;
    case SEEK_CUR:
        newpos = f->f_pos + offset;
        break;
    case SEEK_END:
        newpos = f->f_inode->i_size + offset;
        break;
    default:
        return -EINVAL;
    }

    if (newpos < 0)
        return -EINVAL;

    f->f_pos = newpos;
    return newpos;
}
```

Hibakódok és `errno` a VFS rétegben is szigorúan elválik: a kernel belső hívásai negatív hibakódot adnak vissza, a rendszerhívás réteg pedig a felhasználói térbe való visszatérés előtt beállítja a `errno` értéket és `-1`-et ad. A fejezet példáiban a `return -ENOENT` alakot használjuk a kernelbelsőben.

29.3.7 7. Integrációs példa: memfs mount a `/mnt` könyvtárra

A következő példa egy minimális memfs-t csatlakoztat a közös fába. A memfs nem használ blokk-eszközt; minden fájl és könyvtár a RAM-ban él. A mount `source` paramétere ilyenkor lehet `"none"`, a `fstype` pedig `"memfs"`.

A memfs `mount` metódusa felépíti a szuperblokkot és a gyökércsomópontot:

```
struct super_block *memfs_mount(const char *source,
                                unsigned long flags)
{
    struct super_block *sb = kzalloc(sizeof(*sb));
    if (!sb)
        return ERR_PTR(-ENOMEM);

    sb->s_blocksize = 4096;
    sb->s_magic = 0x4D454D46;   /* "MEMF" */
    sb->s_op = &memfs_super_ops;

    struct inode *root_ino = memfs_new_inode(S_IFDIR | 0755);
    if (!root_ino) {
        kfree(sb);
        return ERR_PTR(-ENOMEM);
    }

    sb->s_root = d_alloc_root(root_ino);
    if (!sb->s_root) {
        memfs_free_inode(root_ino);
        kfree(sb);
        return ERR_PTR(-ENOMEM);
    }

    return sb;
}
```

A mount után a felhasználói program a szokásos módon hívja az `open`-t:

```
int fd = open("/mnt/demo.txt", O_RDWR | O_CREAT, 0600);
if (fd < 0) {
    perror("open");
    return 1;
}

ssize_t n = write(fd, "hello\n", 6);
if (n != 6) {
    perror("write");
    close(fd);
    return 1;
}

lseek(fd, 0, SEEK_SET);
char buf[16] = {0};
n = read(fd, buf, sizeof(buf));
close(fd);
```

A kernelben ezek a hívások így kapcsolódnak össze:

1. A `open` átadja a `/mnt/demo.txt` útvonalat a `path_resolve`-nak.
2. A `/` komponenst a folyamat root dentry-je oldja fel.
3. A `mnt` komponensnél a VFS észleli a memfs mount-ot, és átlép a memfs root-jára.
4. A `demo.txt` komponenst a memfs `lookup` keresi; ha nincs ilyen fájl, a `create` művelet létrehozza.
5. A VFS lefoglal egy `struct file`-ot, beállítja az `f_op`-ot a memfs `file_operations`-re, és beilleszti a `files` táblába.
6. A `read` és `write` a `struct file` `f_pos` mezőjén keresztül frissíti az offsetet.

A `O_CREAT` és a `mode` paraméter a `create` inode-műveletbe kerül. A `O_RDWR` a `file_operations` open/read/write kiválasztását befolyásolja. A VFS ezeket a flag-eket már a `open` rendszerhívás rétegében ellenőrzi.

29.3.8 8. Tipikus hibák és helyes kezelésük

A VFS-integráció során a következő hibák fordulnak elő leggyakrabban:

1. **Az offset nem a `file`-ban, hanem az inode-ban él.** Ha a fejlesztő az inode-ba teszi a pozíciót, akkor két párhuzamos `open` ugyanazt a pozíciót módosítja, és a `dup` szemantika is elromlik. Helyes megoldás: `struct file` `f_pos`, és a `file_operations` mindig a kapott `pos` pointerrel dolgozik.

2. **A dentry referencia-számláló hiánya.** A `path_resolve` után a dentry `d_count`-ját növelni kell, és a használat után `dput`-ot hívni. Ha ez elmarad, a dentry felszabadítása use-after-free hibát okozhat. A dentry cache lookup és a mount átlépés során is meg kell őrizni a referenciákat.

3. **A mount pont alatti tartalom elvesztése.** A `path_resolve`-nak minden komponensnél ellenőriznie kell a mount-táblát. Ha ezt kihagyjuk, a `/mnt` alatt a háttér-fájlrendszer könyvtára marad látható, és a csatolt fájlrendszer elérhetetlen.

4. **A `read`/`write` offset-ellenőrzés hiánya.** Az offset lehet negatív (`lseek` után), vagy a count nagyobb, mint az `off_t` tartománya. A `memfs_read`-ben a `*pos < 0` és a `m->size - *pos` ellenőrzés szükséges. Az `EOVERFLOW` vagy `EINVAL` hibát ilyenkor a VFS-nek kell visszaadnia.

5. **A fájlleíró-tábla túlcsordulása.** A `fd_alloc`-nak ellenőriznie kell a `MAX_FD` határt, és `EMFILE`-t kell adnia, ha nincs szabad slot. A `EBADF` a nem létező fd-re utal, a kettőt nem szabad összekeverni.

6. **Versenyhelyzet a `file` offseten.** Ha a `read`/`write` nem veszi fel a `f_lock` zárat, két szál párhuzamosan olvashatja és növelheti ugyanazt a `pos`-t. Ez adatvesztést okozhat. A `struct file`-hoz tartozó zárnak a teljes kritikus szakaszra kell kiterjednie.

7. **Hibakód-konverzió a user/kernel határon.** A kernel negatív hibakódokat ad; a felhasználói tér felé ezt `-1`-re és `errno` értékre kell fordítani. Ha a `read` rendszerhívás `-ENOENT`-et ad vissza, a felhasználó `-1`-et és `errno==ENOENT`-et kap.

29.3.9 9. Labor: VFS-interfész és mount implementáció

Ez a labor a Blokk-eszköz és driver implementáció blokk-eszköz driverére épül, de a tanulási célhoz egy RAM-ban tárolt memfs is elegendő. A feladat egy működő `vfs_mount`, `path_resolve`, `open`, `read`, `write`, `lseek`, `close` lánc implementálása, majd a mount és a fájlműveletek reprodukálható mérése emulált környezetben.

Kiinduló állapot. A kernelünk a Blokk-eszköz és driver implementáció fejezet után már rendelkezik egy blokk-eszköz réteggel, például `bd_read(sector)` és `bd_write(sector)` hívásokkal. A fájlrendszer most egy memfs, amely a blokk-eszközt nem használja, de a mount pontján keresztül ugyanúgy elérhető. A hoszt lemezét nem módosítjuk, minden futás a QEMU vendég RAM-jában történik.

Konfiguráció. A reprodukciós csomag tartalma:

```
vfs-lab/
  Makefile
  qemu-run.sh
  kernel.ld
  src/
    vfs.c
    memfs.c
    block.c
    task.c
  README.md
  logs/
```

A QEMU alábbi paraméterekkel indul:

```
qemu-system-x86_64 -kernel build/kernel.bin \
  -m 128M -smp 1 -nographic \
  -no-reboot -no-shutdown
```

A vendég CPU `qemu64`, a memória 128 MiB, a megszakítások engedélyezve vannak a mérés alatt. A kernel a `-O2 -fno-strict-aliasing` kapcsolóval készül. A pontos GCC és QEMU verziót a `README.md`-ben rögzíteni kell, és a mért értékekhez csatolni.

A labor lépései.

1. Másold a kiinduló kernel forrást a lab könyvtárába.
2. Implementáld a `vfs_mount` és a mount-léptetés logikáját a `path_resolve`-ban.
3. Implementáld a memfs `lookup`, `create`, `read`, `write`, `llseek` műveleteket.
4. Indítsd el a kernelt, és hívd a következő tesztet:

```
static void vfs_test(void)
{
    int fd = open("/mnt/test.txt", O_RDWR | O_CREAT, 0600);
    assert(fd >= 0);

    const char *msg = "vfs-integration\n";
    ssize_t n = write(fd, msg, strlen(msg));
    assert(n == strlen(msg));

    lseek(fd, 0, SEEK_SET);
    char buf[32] = {0};
    n = read(fd, buf, sizeof(buf));
    assert(n == strlen(msg));
    assert(memcmp(buf, msg, n) == 0);

    close(fd);
```

```
    dmesg("vfs_test: OK");
}
```

5. Mérd meg a VFS-műveletek idejét a következő mikrobenchmarkkal:

```
uint64_t start = rdtsc();
for (int i = 0; i < 1000; i++) {
    int fd = open("/mnt/file", O_RDWR);
    read(fd, tmp, 64);
    write(fd, tmp, 64);
    close(fd);
}
uint64_t end = rdtsc();
```

A cikluson belül ugyanazt a 4 KiB-os fájlt nyitjuk meg. A mérési pontok: `open`, `read+write`, `close`, valamint a teljes ciklus. A TSC-t `rdtsc`-vel olvassuk, és CPU órajelre normalizáljuk.

Kiinduló elvárás és a mért érték elkülönítése. A naplóban a következő táblázatot kell kitölteni:

```
| művelet        | várt trend          | mért átlag | szórás | min | max |
|----------------+---------------------+------------+--------+-----+-----|
| open           | dentry cache O(1)   | ? µs       | ?      | ?   | ?   |
| read+write 64B | O(64) memcpy        | ? µs       | ?      | ?   | ?   |
| close          | O(1)                | ? µs       | ?      | ?   | ?   |
```

Egyetlen mérési szám nem elegendő; rögzíteni kell a ciklusszámot, a cache-találati arányt a dentry cache-ben, és a laphibák számát (ha a kernel már támogatja a lapozást).

Elvárt működés. A `vfs_test` sikeres futtatása után a mount-névtér konzisztens: a `/mnt/test.txt` a memfs rootjában jön létre. A `dentry` cache találati aránya a második open-től kezdve nő, az open átlagos ideje csökken. A `read`/`write` idő a másolandó 64 bájttal közel lineáris, az offset mind a 1000 iterációban helyes marad.

Reprodukciós napló. A naplóba a következő parancsok kimenete tartozik:

```
make -C vfs-lab clean
make -C vfs-lab build
./qemu-run.sh | tee logs/boot.log
```

A `boot.log` végén a `vfs_test: OK` sor, a mérési táblázat és a verziók szerepelnek. A labot eldobható QEMU környezetben vagy külön vendégképen futtasd; a hoszt rendszerpartícióját és bootloaderét tilos módosítani.

29.3.10 10. Önellenőrző kérdések

1. Mi a különbség a `dentry`, az `inode` és a `struct file` szerepe között? Melyik tárolja az offsetet és miért?
2. Mit kell tennie a `path_resolve`-nak, amikor egy aktív mount point-ot ér el?
3. Hogyan osztozik az offset két fájlleíró között `dup` után? Melyik kernelbeli adatszerkezet referencia-számlálóját kell növelni?
4. Milyen hibát okoz, ha a `read` művelet közvetlenül az `inode`-ba menti a pozíciót, és miért nem elég az `inode` zára?
5. Miért kell a dentry `d_count` referencia-számlálóját a `d_lookup` után növelni? Milyen hibakódot adunk, ha egy útvonalkomponens nem könyvtár?

Ezek a kérdések a fejezet fő ellenőrzési pontjait fedik le. A megoldásuk a 8. szakasz hibakezelési megjegyzéseiből és a példakódból levezethető.

A következő fejezet, Shell és alapszolgáltatások, erre a VFS-rétegre építve egy kezdetleges shellt és a fájlrendszer alapműveleteit fűzi fel felhasználói szintű interfésszé.

29.4 Shell és alapszolgáltatások

A minimális userland nem attól működik, hogy a kernel mindent el tud végezni; attól válik használhatóvá, hogy a felhasználói térben néhány apró, jól megírt program a fájlleírókat, a folyamatokat és a környezeti állapotot a POSIX-elvárásokhoz hasonlóan kezeli. Az `os112` alfejezetben felépítettük a VFS-t és a fájlleíró réteget. Most arra a rétegre egy parancsértelmezőt és a hozzá tartozó alapszolgáltatásokat helyezünk. A kernel továbbra is ugyanaz marad: rendszerhívásokat ad, a munka nagy része a userlandben történik.

29.4.1 1. Mi a minimális userland?

A userland az operációs rendszer azon része, amely nem fut privilegizált módban. Egy minimális, oktatási célú rendszerben a userland általában a következőkből áll:

- egy indító program (`init` vagy `sh` közvetlenül),
- a rendszerhívások vékony C-burkolói (a libc helyett egy `syscalls.h` és `syscalls.c`),
- egy parancsértelmező (shell),
- néhány alapvető segédprogram (például `cat`, `echo`, `ls`, `mkdir`, `rm`),
- szükség esetén egy-két eszközfájl (`/dev/console`, `/dev/null`) és a `/bin` könyvtár.

A cél az, hogy a rendszer elindulása után a felhasználó vagy a tesztkörnyezet parancsokat tudjon kiadni, scripteket tudjon futtatni, és ellenőrizni tudja a fájlrendszer, illetve a folyamatok működését.

A shell **nem kernelkomponens**. Ugyanolyan felhasználói program, mint a `cat` vagy a `ls`; annyi különbséggel, hogy a felhasználó által begépelt sorokat parancsokká alakítja, majd ezeket más programok futtatásával hajtja végre. Ez a kialakítás azért fontos, mert a parancsok nagy része nem a shell kódjában él, hanem külön futtatható fájlokban. A shell feladata a „társalgás” a felhasználóval és a folyamatok összekapcsolása.

29.4.2 2. A shell indítása és a fő ciklus

A minimális rendszer bootolás után, az `os103` és `os104` alfejezetekben leírt kernel-inicializálást követően elindítja az első felhasználói folyamatot. A leggyakoribb választás: a kernel megnyitja a `/bin/sh` fájlt, futtathatóvá teszi, majd átadja neki a konzolt mint szabványos be- és kimenetet. A shell ezután a következő ciklust futtatja:

1. promptot ír a szabványos kimenetre;
2. beolvas egy sort a szabványos bemenetről;
3. a sort tokenekre bontja;
4. ha beépített parancs, akkor közvetlenül végrehajtja;
5. egyébként létrehoz egy gyermekfolyamatot, amely betölti a megfelelő programot;
6. a szülő megvárja a gyermeket, majd kiértékeli a kilépési állapotot;
7. visszatér az 1. lépésre.

Ezt mutatja a következő, szándékosan lecsupaszított C-vázlat. A kód nem éles rendszerre való, hanem a fejezetben épített emulált környezethez.

```
/* user/sh.c - minimális shell szemléltető vázlata */
#include "syscalls.h"

#define MAX_LINE 128
#define MAX_ARGS 16

static char line[MAX_LINE];
static char *argv[MAX_ARGS + 1];

int main(int argc, char **argv)
{
    (void)argc;
    (void)argv;

    for (;;) {
        ssize_t n;

        write(STDOUT_FILENO, "mini$ ", 6);

        n = read(STDIN_FILENO, line, sizeof(line) - 1);
        if (n <= 0) {
            /* EOF vagy hiba: a shell kilép */
            return n == 0 ? 0 : 1;
        }
        line[n] = '\0';

        if (tokenize(line, argv, MAX_ARGS) < 0) {
            write(STDERR_FILENO, "sh: túl sok argumentum\n", 23);
            continue;
        }

        if (argv[0] == NULL) {
            /* üres sor */
            continue;
        }

        if (run_builtin(argv) == 1) {
            continue;
        }

        if (run_external(argv) < 0) {
            /* run_external már kiírta a hibát */
        }
    }
}
```

A `tokenize` egyelőre csak a whitespace-ek mentén vág szét; a gyakorlatban a minimális shell is támogathatja a `"..."` és `'...'` idézést, illetve a `\` escape-et. A laborban érdemes az idézést legalább alapszinten implementálni, mert enélkül a fájlnevekben lévő szóközök nem kezelhetők. A `tokenize` visszatérési értéke a tokenek száma lehet, hibánál `-1`; a példában `-1` a túl hosszú sort jelzi.

29.4.3 3. Belső és külső parancsok

A shell parancsai két csoportba sorolhatók.

- **Belső parancsok** (builtin): olyan parancsok, amelyek a shell saját processzét módosítják, vagy nem éri meg külön folyamatként futtatni őket. Például:
  - `cd <útvonal>` – a munkakönyvtár megváltoztatása; ha ezt külön folyamat tenné, a változás nem lenne hatással a szülő shellre.
  - `exit [kód]` – a shell kiléptetése.
  - `export NÉV=ÉRTÉK` – környezeti változó beállítása.
  - `pwd` – az aktuális munkakönyvtár kiírása.
  - `help` – a rendelkezésre álló belső parancsok listája.
- **Külső parancsok**: önálló futtatható fájlok, például `/bin/cat`, `/bin/ls`. Ezeket a shell `fork` + `execve` + `waitpid` hármassal futtatja. A `/bin` könyvtárban lévő programok a minimális userland részét képezik.

A shell `cd` parancsának belső volta klasszikus hibaforrás. Ha valaki úgy próbálja megvalósítani, hogy `fork` után a gyermekben meghívja a `chdir`-t, akkor a gyermek munkakönyvtára megváltozik, de a szülőé nem; a shell a következő promptnál ugyanott marad, ahonnan indult. Ezért a `cd`-t a shell saját folyamatában kell végrehajtani.

A belső parancsok vázlata:

```
int run_builtin(char **argv)
{
    if (strcmp(argv[0], "cd") == 0) {
        const char *path = argv[1] ? argv[1] : "/";
        if (chdir(path) < 0) {
            perror("cd");
        }
        return 1; /* kezelve */
    }

    if (strcmp(argv[0], "exit") == 0) {
        int code = argv[1] ? atoi(argv[1]) : 0;
        exit(code);
    }

    if (strcmp(argv[0], "export") == 0) {
        if (argv[1] && strchr(argv[1], '=')) {
            setenv(argv[1]);
        } else {
            write(STDERR_FILENO, "használat: export NÉV=ÉRTÉK\n", 29);
        }
        return 1;
    }

    if (strcmp(argv[0], "pwd") == 0) {
        char buf[256];
        if (getcwd(buf, sizeof(buf)) == NULL) {
            perror("pwd");
        } else {
            write(STDOUT_FILENO, buf, strlen(buf));
            write(STDOUT_FILENO, "\n", 1);
        }
        return 1;
    }

    return 0; /* nem belső parancs */
}
```

Itt a `chdir`, a `getcwd`, a `setenv` és a `strcmp` a minimális libc-ből, illetve a rendszerhívás-burkolókból származnak. A tényleges implementáció a laborban az `os112`-ben már elkészített

VFS-re épül.

29.4.4 4. Külső program futtatása: fork, execve, waitpid

A külső parancsok futtatása a folyamatkezelés három rendszerhívását használja.

29.4.5 `fork`

```
pid_t fork(void);
```

A `fork` létrehozza a hívó folyamat másolatát. A gyermek a hívás után a `0` értéket kapja vissza, a szülő a gyermek pozitív folyamatazonosítóját. Hibánál `-1` a visszatérési érték, és a kernol az `errno`-t beállítja; tipikus értékek:

- `EAGAIN` – nincs elég memória vagy elértük a folyamatszám-korlátot;
- `ENOMEM` – a folyamattábla vagy a verem allokációja sikertelen.

A shell ezt használja arra, hogy a gyermekben lecserélje a programképet.

29.4.6 `execve`

```
int execve(const char *pathname, char *const argv[], char *const envp[]);
```

Az `execve` betölti a `pathname` fájlt, beállítja az argumentumlistát és a környezetet, majd lecseréli az aktuális folyamat címterét. Sikeres hívás **nem tér vissza**. Hiba esetén `-1`-gyel tér vissza, és az `errno` tartalmazza az okot. A leggyakoribb okok:

- `ENOENT` – a fájl vagy egy PATH-elem nem létezik;
- `EACCES` – nincs végrehajtási jog;
- `ENOEXEC` – a fájl nem futtatható formátumú, vagy nincs érvényes fejléc;
- `E2BIG` – az argumentumlista, a környezet vagy mindkettő túl nagy;
- `EFAULT` – a felhasználói mutatók érvénytelenek.

A minimális rendszerben az `execve` a VFS `open` rétegén keresztül olvassa a fájlt, ellenőrzi az ELF-fejlécet, betölti a szegmenseket, beállítja a vermet, és az `entry` pontra ugrik. A shellnek a gyermekben az `execve` hiba után azonnal `_exit(127)`-et kell hívnia, különben a gyermek a shell kódját futtatná tovább, és egy második shellt indítana.

29.4.7 `waitpid`

```
pid_t waitpid(pid_t pid, int *_Nullable wstatus, int options);
```

A `waitpid` a szülőt várakoztatja a megadott gyermek befejeződésére. A `wstatus` mutató egy bitmezőt kap, amelyből a shell megállapíthatja a kilépés módját:

- `WIFEXITED(wstatus)` igaz, ha a gyermek normálisan fejeződött be; ekkor `WEXITSTATUS(wstatus)` adja a kilépési kódot.
- `WIFSIGNALED(wstatus)` igaz, ha a gyermeket szignál állította le; a shell ilyenkor `128 + jel` kódot szokott visszaadni.

A shell `run_external` függvénye ezt a hármast foglalja össze:

```
int run_external(char **argv)
{
    char path[128];
    pid_t pid;
    int status;

    if (find_in_path(argv[0], path, sizeof(path)) < 0) {
        write(STDERR_FILENO, "sh: parancs nem található: ", 27);
        write(STDERR_FILENO, argv[0], strlen(argv[0]));
        write(STDERR_FILENO, "\n", 1);
        return -1;
    }

    pid = fork();
    if (pid < 0) {
        perror("fork");
        return -1;
    }

    if (pid == 0) {
        /* gyermek: program betöltése */
        execve(path, argv, environ);
        /*
         * Ide csak akkor jutunk, ha az execve hibázott.
         * A _exit megakadályozza, hogy a gyermek
         * a shell kódját futtassa tovább.
         */
        perror("execve");
        _exit(127);
    }

    /* szülő: várakozás a gyermekre */
    if (waitpid(pid, &status, 0) < 0) {
        perror("waitpid");
        return -1;
    }

    return report_status(status);
}
```

A `find_in_path` a shell saját segédfüggvénye. A `PATH` környezeti változót kettőspontok mentén bontja fel, és minden elemből `útvonal/parancsnév` jelöltet készít. A minimális rendszerben a `PATH` általában `/bin:/usr/bin`. A `find_in_path` megpróbálhatja az `open`-t `O_RDONLY` módban minden jelöltre; ha a fájl létezik, visszaadja az elérési utat. A végrehajthatósági bit ellenőrzése a VFS-ben és az `execve`-ben történik.

Az argumentumok átadása POSIX-konvenció szerint történik: `argv[0]` a program neve, `argv[1]` az első parancssori argumentum, és az `argv` tömb végét `NULL` jelöli. A shellnek ügyelnie kell arra, hogy a `NULL` lezáró elem elférjen, ezért a `MAX_ARGS` tömb mérete `MAX_ARGS + 1`.

29.4.8 5. Pipe és átirányítás

A shell igazi erejét a folyamatok közötti csővezetékek és a fájlba irányítások adják. A `|` karakter két parancsot köt össze: a bal oldali parancs szabványos kimenetét a jobb oldali szabványos bemenetére irányítja. Ehhez a kernel `pipe` rendszerhívását használjuk.

29.4.9 `pipe`

```
int pipe(int pipefd[2]);
```

A `pipe` egy csővezetéket hoz létre. A `pipefd[0]` az olvasó vég, a `pipefd[1]` az író vég. A csőben lévő adatok a kernel memóriájában tárolódnak; amíg van nyitott író vég, az olvasó blokkol, ha a cső üres; amikor az összes író vég bezáródik, az olvasó `read` hívása `0`-t ad vissza, ami EOF-ot jelez. A minimális rendszerben a pipe puffer mérete például 4096 bájt lehet; a labor mérésénél ezt érdemes rögzíteni.

A csővezeték végrehajtásához a shell két gyermekfolyamatot hoz létre. A `dup2` rendszerhívás segítségével a cső megfelelő végét a `STDOUT_FILENO`, illetve a `STDIN_FILENO` fájlleíróra másolja.

29.4.10 `dup2`

```
int dup2(int oldfd, int newfd);
```

A `dup2` a `newfd` fájlleírót az `oldfd`-re mutató nyitott fájlleíró-másolatává teszi. Ha a `newfd` korábban nyitva volt, a kernel először bezárja. A shell ezt az átirányításoknál használja:

- `> fájl` esetén: `open("fájl", O_CREAT|O_WRONLY|O_TRUNC, 0644)`, majd `dup2(fd, STDOUT_FILENO)`.
- `>> fájl` esetén: `O_CREAT|O_WRONLY|O_APPEND`.
- `< fájl` esetén: `open("fájl", O_RDONLY)`, majd `dup2(fd, STDIN_FILENO)`.

A pipe-os példa vázlata:

```
int run_pipeline(char **left_argv, char **right_argv)
{
    int fd[2];
    pid_t left, right;
    int left_status, right_status;

    if (pipe(fd) < 0) {
        perror("pipe");
        return -1;
    }

    left = fork();
    if (left < 0) {
        perror("fork");
        close(fd[0]);
        close(fd[1]);
        return -1;
    }

    if (left == 0) {
        /* bal oldali gyermek: írás a csőbe */
        close(fd[0]);
        if (dup2(fd[1], STDOUT_FILENO) < 0) {
            perror("dup2");
            _exit(1);
        }
        close(fd[1]); /* a dup2 után az eredeti felesleges */
        execve(left_path, left_argv, environ);
        perror("execve");
        _exit(127);
    }
```

```
    right = fork();
    if (right < 0) {
        perror("fork");
        close(fd[0]);
        close(fd[1]);
        return -1;
    }

    if (right == 0) {
        /* jobb oldali gyermek: olvasás a csőből */
        close(fd[1]);
        if (dup2(fd[0], STDIN_FILENO) < 0) {
            perror("dup2");
            _exit(1);
        }
        close(fd[0]);
        execve(right_path, right_argv, environ);
        perror("execve");
        _exit(127);
    }

    /*
     * Kulcsfontosságú lépés: a szülőnek mindkét csővéget
     * be kell zárnia. Különben a jobb oldali gyermek a
     * pipe EOF-ját soha nem látja, mert a szülőben nyitva
     * maradt az író vég.
     */
    close(fd[0]);
    close(fd[1]);

    waitpid(left, &left_status, 0);
    waitpid(right, &right_status, 0);

    return WIFEXITED(right_status) ? WEXITSTATUS(right_status) : 1;
}
```

A fenti kódban a `left_path` és a `right_path` kiszámítását a `find_in_path` végzi. A `close(fd[0])` és `close(fd[1])` a szülőben lévő felesleges fájlleírókat szünteti meg. Ez az egyik leggyakoribb hibaforrás: ha a szülő nem zárja be a cső író végét, a jobb oldali folyamat a `read`-ben örökre blokkolhat, mert az író vég sosem záródik be a szülő miatt.

A shell parser először a `|` és az átirányítási karakterek mentén bontja fel a sort. Az átirányítások elvileg tetszőleges sorrendben előfordulhatnak:

```
cat < /etc/motd | head > /tmp/elso_sor
```

Itt a bal oldali parancs bemenetét a `/etc/motd` fájlból, a jobb oldali parancs kimenetét a `/tmp/elso_sor` fájlba kell irányítani. A minimális shell kezdetben megelégedhet azzal, hogy a `>` és `<` jeleket egyetlen parancs után keresi, és a `|` jelet legfeljebb egyszer támogatja. A laborban ez elegendő a gyakorlati célhoz.

29.4.11 6. Környezeti változók és környezet átadása

A UNIX-szerű rendszerekben minden folyamat rendelkezik egy környezeti blokkal, amely `NÉV=ÉRTÉK` formájú sztringekből áll. A shell indulásakor ezt a blokkot a kerneltől kapja. A C-kódban a `environ` globális mutató hivatkozik rá:

```
extern char **environ;
```

A `fork` a gyermeknek ugyanazt a környezetet adja. Az `execve` harmadik paramétere a környezet; a shell a `run_external` és a `run_pipeline` hívásokban a saját `environ` mutatóját adja át. Ha a shell `export` parancsot kap, akkor a saját környezeti blokkját módosítja, és ez a változás a később indított gyermekekre is érvényes lesz.

A minimális `setenv` a következőket teheti:

- megkeresi az adott névhez tartozó bejegyzést,
- ha létezik, lecseréli az értéket,
- ha nem létezik, új bejegyzést fűz a tömbhöz.

A környezeti tár mérete korlátozott; a laborban érdemes 32 vagy 64 bejegyzést engedélyezni, és a túlcsordulást `-1` visszatéréssel jelezni.

A leggyakrabban használt környezeti változók:

- `PATH` – a külső parancsok keresési útvonala;
- `HOME` – a felhasználó otthoni könyvtára;
- `USER` vagy `LOGNAME` – a felhasználónév;
- `PWD` – az aktuális munkakönyvtár (informatív jelleggel; a tényleges állapotot a kernel tartja).

A shellnek a `PATH`-ot minden külső parancs előtt be kell olvasnia. Ha a felhasználó megváltoztatja a `PATH`-ot, a változásnak a következő parancsra már érvényesülnie kell. A biztonság szempontjából fontos, hogy a `PATH`-ba ne kerüljön a `.` (aktuális könyvtár) implicit módon; a minimális rendszerben ezt csak akkor vegyük fel, ha a felhasználó kifejezetten beállította.

29.4.12 7. Hibakódok, `errno` és kilépési állapot

A rendszerhívások hibáinak kezelése a userlandben két szinten történik.

A C-szinten a burkolók visszaadják a `-1` értéket, és beállítják a `errno` nevű globális változót. A `perror` vagy a `strerror` segítségével a hiba szöveges leírása kiírható. A `perror` a hívás előtti szöveget, kettőspontot és az `errno`-hoz tartozó üzenetet írja a szabványos hibakimenetre. A minimális libc-ben a `strerror` csak a leggyakoribb hibakódokat ismerheti; a laborban legalább az `ENOENT`, `EACCES`, `ENOMEM`, `EBADF`, `EINVAL`, `EIO` és `EEXIST` kódokat érdemes leképezni.

A shell szintjén a kilépési állapot a következő:

- `0` – sikeres végrehajtás;
- `1` – általános hiba;
- `2` – helytelen parancssori használat;
- `127` – a parancs nem található, vagy az `execve` hibázott;
- `128 + N` – a folyamatot az `N` számú szignál állította le.

A shell a `$?` belső változóban tárolja az utolsó parancs kilépési állapotát. Ez a scripting alapja: egy script megvizsgálhatja, hogy az előző parancs sikeres volt-e.

A `report_status` függvény a `waitpid` által adott `wstatus` értéket alakítja át:

```
int report_status(int wstatus)
{
    if (WIFEXITED(wstatus)) {
        return WEXITSTATUS(wstatus);
    }
    if (WIFSIGNALED(wstatus)) {
        return 128 + WTERMSIG(wstatus);
    }
```

```
    return 1;
}
```

A shell a promptnál az utolsó kilépési állapotot a `$?` kérdésre adott válaszként jeleníti meg. A minimális megvalósításban ezt a változót a shell a `run_external` vagy a `run_pipeline` visszatérési értékéből frissíti.

29.4.13 8. Scripting a minimális userlandben

A shellscriptek használata akkor válik lehetővé, ha a rendszer két dolgot támogat:

1. a futtatható szövegfájlokat a kernel vagy a shell interpreterhez tudja kötni;
2. a shell képes soronként értelmezni egy fájlt, nem csak az interaktív bemenetet.

A hagyományos UNIX-rendszerekben az `execve` a fájl első két bájtját ellenőrzi. Ha azok `#!` karakterek, a kernel a második sorban megadott interpretert indítja el, és átadja neki a script elérési útját. A minimális rendszerben erre két lehetőség van:

- a kernel `execve`-je felismeri a `#!`-et, és az interpreter programot tölti be;
- a shell érzékeli, hogy a fájl nem bináris, hanem szöveges script, és maga olvassa be, majd soronként végrehajtja.

Az oktatási célú rendszerben gyakran a második megoldás az egyszerűbb, mert nem igényel kernelváltoztatást. Ekkor a shell `source` vagy közvetlenül a fájlnevet kapja argumentumként, és a `read` ciklust a szabványos bemenet helyett egy fájlból futtatja.

Egy minimális script:

```
#!/bin/sh
# laborbemutató.sh - nem éles rendszerre
echo "Mini OS shell indítása"
mkdir -p /tmp/log
cat /etc/motd > /tmp/log/motd_masolat
if [ $? -eq 0 ]; then
    echo "mentés sikeres"
else
    echo "mentés sikertelen"
fi
```

A `mkdir -p` az `EEXIST` hibát is elfogadja, ha a könyvtár már létezik. A `[` parancs a minimális rendszerben lehet önálló program vagy a shell beépített feltételkezelése; a laborban elegendő a `$?` ellenőrzését és egyszerű `if`-et bevezetni. A script futtatása közben a shell a sorokat ugyanúgy tokenizálja, mint az interaktív módban, ezért a kód újrahasznosítható.

A script futtatásának lépései:

1. a shell `open`-nel megnyitja a scriptfájlt;
2. ha a fájl `#!`-tel kezdődik, az interpretert beazonosítja, és a scriptfájlt argumentumként átadva `fork` + `execve` segítségével elindítja;
3. a kiválasztott interpreter a fájl tartalmát feldolgozza;
4. a script kilépési állapota az interpreter kilépési állapota lesz.

A laborban a shell `sh script.sh` formában futtathat scripteket; ez kiváltja a kernel shebang-támogatását, és a shell maga olvassa be a fájlt.

29.4.14 9. Alapszolgáltatások: cat, echo, ls, mkdir, rm

A minimális userland néhány apró, jól körülhatárolt programot tartalmaz. Mindegyik ugyanazokat a rendszerhívásokat használja, mint a shell, így a VFS és a fájlleíró réteg egységesen tesztelhető.

29.4.15 `cat`

A `cat` a megadott fájlok tartalmát másolja a szabványos kimenetre. Ha nincs argumentum, a szabványos bemenetet olvassa. A megvalósítás kulcsa a `read` és `write` ciklus:

```
/* user/cat.c - fájl tartalmának másolása */
#include "syscalls.h"

static void copy_fd(int in_fd)
{
    char buf[512];
    ssize_t n;

    for (;;) {
        n = read(in_fd, buf, sizeof(buf));
        if (n < 0) {
            perror("read");
            return;
        }
        if (n == 0) {
            return;
        }
        if (write(STDOUT_FILENO, buf, (size_t)n) < 0) {
            perror("write");
            return;
        }
    }
}

int main(int argc, char **argv)
{
    int i;

    if (argc < 2) {
        copy_fd(STDIN_FILENO);
        return 0;
    }

    for (i = 1; i < argc; i++) {
        int fd = open(argv[i], O_RDONLY, 0);
        if (fd < 0) {
            perror("cat");
            continue;
        }
        copy_fd(fd);
        close(fd);
    }

    return 0;
}
```

29.4.16 `echo`

Az `echo` az argumentumait szóközzel elválasztva a szabványos kimenetre írja, majd egy újsor karaktert ad hozzá. A minimális verzió nem feltétlenül értelmezi a `-n` kapcsolót, de a laborban érdemes bevezetni, mert a scriptekben gyakran használják. Az `echo` belső parancs is lehet, de külön programként egyszerűbb tesztelni a `fork` + `execve` útvonalat.

29.4.17 `ls`

Az `ls` a VFS könyvtárbejegyzéseit listázza. A minimális rendszerben a `getdents` rendszerhívás vagy annak burkolója adja vissza a bejegyzések nevét. A program minden bejegyzést kiír, a `.` és `..` kivételével. A hosszú formát, a jogosultságokat és a fájlméretet az `os110` fájlrendszer-metaadatainak beolvasásával lehet megjeleníteni; a minimális laborban elég a nevek felsorolása.

29.4.18 `mkdir` és `rm`

A `mkdir` egy új könyvtárat hoz létre. A `rm` fájlt töröl; a könyvtárak törléséhez a minimális rendszerben külön `rmdir` programot vagy a `rm -r` kapcsolót implementálhatjuk. Mindkettő a VFS megfelelő műveletére támaszkodik, és a hibakódokat a szabványos módon jelzi.

A shell és a segédprogramok együttesen adják ki azt a minimális felhasználói környezetet, amelyen a kernel további részei, például a hálózat vagy a tesztelési keretrendszer, kényelmesen gyakorolhatók.

29.4.19 10. Gyakori hibák és helyes megoldások

A shell és a userland építése során a következő hibák a leggyakoribbak.

1. **Zombie folyamatok.** Ha a szülő nem hívja a `waitpid`-et, a befejeződött gyermek a folyamattáblában marad. Idővel a rendszer kifogy a folyamat-helyekből. Megoldás: minden `fork` után a szülőben `waitpid`, vagy a `SIGCHLD` szignál kezelése.
2. **`execve` utáni kód futása.** Ha az `execve` hibázik, a gyermek a shell kódját folytatja. Megoldás: a gyermek ágban azonnal `_exit`-et hívni, lehetőleg a `127` kilépési kóddal.
3. **Nyitva maradt pipe-végek.** Ha a szülő nem zárja be a cső mindkét végét, a jobb oldali folyamat sosem kap EOF-ot, és a pipeline örökre blokkol. Megoldás: a `fork` után a szülőben `close(fd[0]); close(fd[1]);`, a gyermekekben pedig a felesleges végét zárni.
4. **A `cd` nem beépített.** Ha a `cd`-t külön folyamat futtatja, a szülő munkakönyvtára nem változik. Megoldás: a `cd`-t a shell saját folyamatában kezelni.
5. **PATH-feloldás biztonsági hibája.** Ha a `find_in_path` a `.`-t is figyelembe veszi, egy rosszindulatú program elfoglalhatja a rendszerparancs nevét. Megoldás: a `.` csak akkor legyen a `PATH`-ban, ha a felhasználó kifejezetten felvette.
6. **Puffer-túlcsordulás a tokenizálásban.** A hosszú bemeneti sorok vagy a nagyszámú argumentum túlírhatja a vermet. Megoldás: hosszkorlát, `MAX_ARGS` ellenőrzés, és a tokenizáló nullázza a tömb végét.
7. **Az `errno` késői olvasása.** A rendszerhívás után más hívások felülírhatják az `errno` értékét. Megoldás: a hibát azonnal le kell kérdezni, hibaüzenetet írni, és az `errno`-t nem szabad több hívás után feltételezni.

8. **A `$?` nem frissül.** Ha a shell nem tárolja az utolsó kilépési állapotot, a scriptek feltételes elágazásai hibásan döntenek. Megoldás: a `report_status` visszatérési értékét a shell egy változóban tárolja, és a `$?` ezt olvassa.

29.4.20 11. Labor: shell és alapszolgáltatások

A fejezet laborja a shell és a minimális segédprogramok implementálását, valamint azok emulált környezetben történő ellenőrzését foglalja magában.

29.4.21 Kiinduló állapot

A labor az `os112` alfejezet kódjára épül. Adott:

- QEMU emulált környezet, rögzített konfiguráció: `qemu-system-i386` vagy `qemu-system-riscv64`, 32 MiB RAM, IDE vagy virtio blokk-eszköz;
- a kernel rendszerhívásai: `open`, `read`, `write`, `close`, `fork`, `execve`, `waitpid`, `pipe`, `dup2`, `chdir`, `getcwd`, `mkdir`, `unlink`;
- a VFS-ben a `/bin`, `/etc`, `/home` könyvtárak;
- az `os112` VFS-re épülő fájlleíró réteg.

29.4.22 Feladatok

1. Valósítsd meg a `user/sh.c` shellt a fenti `tokenize`, `run_builtin`, `run_external`, `run_pipeline`, `find_in_path` és `report_status` függvényekkel.
2. Írd meg a `user/cat.c`, `user/echo.c`, `user/ls.c`, `user/mkdir.c` és `user/rm.c` programokat.
3. Fordítsd le a programokat freestanding C-környezetben, és helyezd a `/bin` könyvtárba egy `initramfs` építő segítségével.
4. Indítsd a rendszert QEMU-ban, és ellenőrizd az alábbi parancsokat:
   - `echo Hello Mini OS`
   - `cat /etc/motd`
   - `ls /bin`
   - `cd /tmp && pwd`
   - `cat /etc/motd | cat > /home/motd2`
   - `sh laborbemutato.sh`
5. Készíts reprodukálható jegyzőkönyvet.

29.4.23 Build és futtatási parancsok

A laborban használt parancsok szemléltető jellegűek; a pontos keresztszerkesztő és QEMU-változat a reprodukciós csomagban található.

```
# user/Makefile - a userland programok fordítása (szemléltetés)
CROSS    ?= riscv64-linux-gnu-
CFLAGS   := -O2 -march=rv64imac -mabi=lp64 -ffreestanding -nostdinc \
            -I../include
LD       := $(CROSS)ld
LDFLAGS  := -T user.ld -nostdlib

USER_PROGS := cat echo ls mkdir rm sh
```

```
all: $(USER_PROGS)

$(USER_PROGS): %: %.c ../lib/syscalls.c ../lib/string.c
    $(CROSS)gcc $(CFLAGS) -c $< -o $@.o
    $(CROSS)gcc $(CFLAGS) -c ../lib/syscalls.c -o syscalls.o
    $(CROSS)gcc $(CFLAGS) -c ../lib/string.c -o string.o
    $(LD) $(LDFLAGS) -o $@ $@.o syscalls.o string.o
    cp $@ ../initramfs/bin/
```

A QEMU futtatás:

```
qemu-system-riscv64 \
  -machine virt \
  -m 32M \
  -kernel build/kernel.elf \
  -initrd build/initramfs.cpio \
  -nographic
```

A shell prompt megjelenése után a fenti tesztek futtathatók. A labor reprodukálhatósága érdekében a kernelverziót, a fordítási konfigurációt és a QEMU-verziót rögzíteni kell.

29.4.24 Mérési napló

A laborban mérd meg az alábbiakat, és a mért értékeket külön oszlopban tüntesd fel a várt vagy elméleti érték mellett.

Mérés	Módszer	Várt/levezetett érték	Mért érték
`fork + execve` átlagos költség 1000 `true` parancsnál	a shell `time` belső parancsa vagy ciklusszámláló	mikromásodperc nagyságrend, pontos érték a hardvertől függ	…
1 MiB adat `cat | wc -c` csővezetéken	`cat` és `wc` futásideje, pipe puffer mérete 4096 bájt	…	…
Shell prompt válaszideje üres bemenetnél	a `read` visszatérési ideje QEMU időzítővel	néhány ezredmásodperc	…
Script futtatási ideje 50 echo parancsnál	`sh script.sh` falidő	…	…

A mérési módszert pontosan dokumentálni kell: a QEMU órajelét, a `-icount` vagy gazda-időzítő használatát, valamint a `-O2` optimalizálást. Egyetlen szám nem elegendő; ahol lehetséges, a ciklusszámot, a futási időt és a csőbuffer-hatást külön jegyezd fel.

A visszaállításhoz a QEMU snapshotot használj; a labor végén a hoszt fájljait ne módosítsd.

29.4.25 12. Önellenőrző kérdés

A shell csővezeték futtatásakor a következő parancsot adjuk ki:

```
cat /etc/motd | cat > /home/motd2
```

A szülő shell a `pipe` és a két `fork` után mindkét csővéget bezárja, majd `waitpid`-del vár mindkét gyermekre. Tegyük fel, hogy a `/etc/motd` fájl 1024 bájt, a pipe puffer mérete 4096 bájt, és mindkét `cat` a `read`/`write` hívásokat 512 bájtos pufferekkel végzi. Válaszolj a következőkre:

1. Miért nem fordulhat elő az, hogy a jobb oldali `cat` a `read`-ben örökre blokkol?
2. Hány `read` hívást hajt végre a bal oldali `cat`, amíg az EOF-t meg nem kapja?
3. Mi történik, ha a szülő elfelejti bezárni a pipe író végét a második `fork` után?

A válasz elemei:

1. A bal oldali `cat` a fájl végére érve kilép, ami bezárja a pipe író végét. Mivel a szülő is bezárta a saját író végét, a pipe-nak nem marad nyitott írója, így a jobb oldali `read` `0`-t ad vissza.
2. A bal oldali `cat` $1024 / 512 = 2$ `read` hívással beolvassa a fájlt, majd a harmadik `read` adja a `0` visszatérést; ezután nem olvas tovább. Tehát összesen 3 `read`, de ebből 2 szolgáltat adatot.
3. Akkor a jobb oldali `cat` nem kap EOF-ot, mert a szülőben nyitva maradt az egyik író vég; a pipeline a `read`-ben blokkol, és a `waitpid` sosem tér vissza. Ez a klasszikus pipe-szivárgás.

A fejezetben bemutatott shell és alapszolgáltatások a következő egység (`os114`) számára alapot adnak: a tesztelési keretrendszer már parancssori eszközökkel indítható, a shell pedig alkalmas az automatikus tesztek futtatására és az eredmények összegyűjtésére.

30 Tesztelés, hibakeresés és teljesítmény

30.1 Unit- és integrációs tesztek kernelben

A kernelkód tesztelését gyakran az alkalmazásfejlesztésnél megszokott tesztekkel próbálják megközelíteni, de a két világ élesen különbözik. A kernelben nincs hoszt operációs rendszer, amely biztosítaná a memóriafoglalást, a szabványos bemenetet-kimenetet, a fájlokat vagy a folyamatok elszigetelését; a tesztkód ugyanabban a címtérben fut, mint a vizsgált kód, és egyetlen hibás pointer- vagy megszakításkezelés pánikot, újraindulást vagy csendes korrupciót okozhat. A tesztelés ezért nem egyszerűen „több assert”, hanem külön tesztarchitektúra, amely a hardvert és a környezetet is determinisztikusan, izoláltan és reprodukálhatóan helyettesíti.

Ez a fejezet a kernelben futó unit, integrációs és boot tesztek felépítését, a mockok és tesztduplák szerepét, valamint a QEMU-ban futtatott tesztkernel gyakorlatát mutatja be. A cél nem egy gyártói tesztkeretrendszer ismertetése, hanem azoknak a fogalmaknak és döntéseknek a megtanítása, amelyekből egy saját, kisméretű kernelben is működő tesztkeret felépíthető.

30.1.1 1. Miért más a kernelkód tesztelése?

Egy felhasználói térben futó program tesztelésekor a teszt keretrendszer és a tesztelt kód ugyanabban a folyamatban, de a hoszt operációs rendszer felügyelete alatt fut. A kernelben ezzel szemben a következő sajátosságokkal kell számolnunk.

Nincs szabványos környezet. A kernel általában freestanding C-t vagy assemblyt használ, ahol nincs `printf`, `malloc`, `exit` vagy `errno` a hoszt értelmében. Minden segédfüggvényt vagy a kernel valós kódja, vagy a tesztkeret biztosít. A tesztkeret ezért a naplózáshoz gyakran soros portot, egyszerű puffert vagy a QEMU hibakivezető eszközét használja.

A hiba következménye nem kivétel, hanem rendszerösszeomlás. Egy illegális memóriacím írása a kernelben nem mindig vált ki azonnali laphibát; ha a cím még leképezett, akkor csendes korrupciót okozhat. A tesztnek ezért nem elég az eredményt ellenőriznie, hanem a memóriatartalom konzisztenciáját, a listák épségét és a hívások előtti/utáni állapotot is.

A konkurencia és a megszakítások mindig jelen vannak. A kernelkód gyakran félbeszakítható, és a tesztek ugyanazokon az adatszerkezeteken futnak, mint a megszakításkezelők vagy az ütemező. A determinizmus biztosításához a tesztek idejére le kell tiltani a megszakításokat, vagy az időzítőt és a megszakításforrásokat mockolni kell.

Nincs egyetlen „process exit” útvonal. A teszt eredményét a QEMU vagy a céleszköz számára értelmezhető módon kell visszaadni: például az `isa-debug-exit` eszközön keresztül kilépési kóddal, egy adott memóriacímen írt bűvös értékkel, vagy a soros konzolon megjelenő `PASS`/`FAIL` sorral.

A tesztkód maga is szennyezi a rendszert. A tesztkeret által lefoglalt memória, a bejárt listák és a beállított regiszterek befolyásolhatják a következő tesztet. A teszteket ezért úgy kell felépíteni, hogy minden teszt után helyreálljon az invariáns állapot, vagy minden teszt külön bootot kapjon.

Ezekből a tulajdonságokból következik a kernelben alkalmazott tesztpiramis: a legolcsóbbak a tisztán logikai, megszakításoktól és eszközöktől független unit tesztek; efölött állnak az integrációs tesztek, amelyek több alrendszer együttműködését ellenőrzik; végül a boot tesztek a teljes indítási láncot és a kezdeti hardverállapotot vizsgálják.

30.1.2 2. Unit tesztek kernelben

Unit teszt alatt olyan tesztet értünk, amely egyetlen függvény vagy adatszerkezet viselkedését ellenőrzi, más alrendszerek és a hardver nélkül. A kernelben ez gyakran egy listakezelő, egy prioritásszámítás, egy bitmező-kezelő vagy egy egyszerű jelsor-átalakító.

A unit teszthez minimális keretre van szükség, amely képes:

- tesztregisztrációra: a tesztfüggvényeket listában tartja nyilván,
- feltételvizsgálatra: az eredményt összeveti a várt értékkel,
- naplózásra: a teszt nevét, a várt és mért értéket, valamint az eredményt kiírja,
- kilépési kód visszaadására: minden sikeres teszt esetén 0, bármely hiba esetén nem nulla.

A következő C kódrészlet egy ilyen minimális keretet vázol. A kód freestanding, nem használ libc-t, és a konzolra írást a `serial_write` már meglévő kernelfüggvényre bízza. A `ASSERT_EQ` makró a várt és mért értéket hasonlítja össze, és hiba esetén azonnal visszatér a hibakóddal.

```
/* test.h - szemléltető minimális kernel tesztkeret */
#ifndef TEST_H
#define TEST_H

#include <stdint.h>
#include <stddef.h>

typedef int (*test_fn)(void);

struct test_case {
    const char *name;
    test_fn fn;
};

#define ARRAY_SIZE(a) (sizeof(a) / sizeof((a)[0]))

int test_run_all(const struct test_case *tests, size_t count);
void serial_write(const char *s);

/* Feltételvizsgáló makró */
#define ASSERT_EQ(actual, expected, name)                       \
    do {                                                        \
        int64_t _a = (int64_t)(actual);                         \
        int64_t _e = (int64_t)(expected);                       \
        if (_a != _e) {                                         \
            serial_write("[FAIL] ");                            \
            serial_write(name);                                 \
            serial_write(" expected=");                         \
            test_print_i64(_e);                                 \
            serial_write(" measured=");                         \
            test_print_i64(_a);                                 \
            serial_write("\r\n");                               \
            return -1;                                          \
        }                                                       \
    } while (0)

#endif
```

A keret `test_run_all` függvénye végigmegy a teszteken, mindegyiket futtatja, és a hibás eseteket megszámolja. A tesztkód a következőképpen nézhet ki:

```
/* test_runner.c - szemléltető futtató */
int test_run_all(const struct test_case *tests, size_t count)
{
    int failed = 0;

    for (size_t i = 0; i < count; i++) {
        serial_write("[RUN ] ");
        serial_write(tests[i].name);
        serial_write("\r\n");

        int rc = tests[i].fn();
        if (rc != 0) {
            failed++;
        } else {
            serial_write("[PASS] ");
            serial_write(tests[i].name);
            serial_write("\r\n");
        }
    }

    serial_write(failed ? "RESULT: FAIL\r\n" : "RESULT: PASS\r\n");
    return failed ? 1 : 0;
}
```

A fenti keret szándékosan egyszerű, de már kielégíti a kernelben a legalapvetőbb igényt: minden teszt kimenete a soros konzolon látható, a várt és mért érték elkülöníthető, és a futás végeredménye gépi úton is ellenőrizhető. A `-1` visszatérési érték a teszt hibáját jelzi, de nem keverendő össze a kernel `errno`-jával; a `-1` itt tesztkonvenció, a konkrét hibakódot a napló tartalmazza.

30.1.3 2.1 Példa: listakezelő unit teszt

Tegyük fel, hogy a kernelünkben van egy beágyazott duplán láncolt lista, amelyet az ütemező és az IPC is használ. A lista `pop_front` művelete üres lista esetén `-ENOENT` hibát ad vissza. A unit teszt ezt a hibakódot várja:

```
/* test_list.c - listakezelő unit tesztek */
#include "test.h"
#include "list.h"

static int test_list_push_pop(void)
{
    list_t list;
    int value = 0;

    list_init(&list);
    list_push_back(&list, 10);

    int rc = list_pop_front(&list, &value);
    ASSERT_EQ(rc, 0, "test_list_push_pop: rc");
    ASSERT_EQ(value, 10, "test_list_push_pop: value");

    return 0;
}

static int test_list_empty_pop_returns_enoent(void)
{
    list_t list;
```

```
    int value = 0;

    list_init(&list);
    int rc = list_pop_front(&list, &value);

    ASSERT_EQ(rc, -ENOENT, "test_list_empty_pop_returns_enoent: rc");
    return 0;
}
```

A `-ENOENT` értéket a kernel saját hibakód-definícióiból vesszük, nem a hoszt `errno.h`-jából. A tesztben a `ASSERT_EQ` makró a várt és mért hibakódot is kiírja, így a naplóból egyértelmű, hogy a függvény miért hibázott. Ez a fajta egyszerű, logikai teszt a kernelben is ugyanolyan gyorsan futtatható, mint a felhasználói térben, feltéve, hogy a lista implementációja nem függ megszakításoktól vagy eszközregiszterektől.

30.1.4 3. Integrációs tesztek

Az integrációs teszt azt ellenőrzi, hogy két vagy több alrendszer együtt helyesen működik. A kernelben tipikus integrációs eset egy IPC-csatorna és az ütemező együttműködése, egy blokkeszközmeghajtó és a fájlrendszer közötti adatátadás, vagy a memóriakezelő és az eszközmeghajtó DMA-kezelésének összehangolása.

Az integrációs teszt lényege, hogy **nem** a belső adatszerkezeteket vizsgálja közvetlenül, hanem a határfelületeken keresztül történő adatcserét. A fejezet laborjában egy egyszerű kernelbeli pipe-ot tesztelünk, amely a következő lépéseket hajtja végre:

1. Létrehoz egy pipe-ot `pipe_create()` hívással.
2. Egy író folyamat a `pipe_write()` függvénnyel 16 bájtot küld.
3. Az ütemező átadja a vezérlést egy olvasó folyamatnak, amely `pipe_read()` hívással beolvassa az adatot.
4. A teszt ellenőrzi, hogy a beolvasott adat bájtonként megegyezik az elküldöttel, és hogy a pipe pufferének számlálója nulla lett.

Ez a teszt nem unit teszt, mert az ütemező működését is igénybe veszi. A tesztkód vázlata pszeudokódban:

```
pipe_id = pipe_create()
writer = thread_create(writer_func, pipe_id)
reader = thread_create(reader_func, pipe_id)

scheduler_start()
wait_for_completion(writer)
wait_for_completion(reader)

ASSERT_EQ(pipe_bytes_available(pipe_id), 0)
ASSERT_MEM_EQ(reader_buffer, expected_payload, 16)
```

A `thread_create` és a `scheduler_start` hívások a kernel valós kódját használják, ezért a teszt csak akkor futhat le, ha az ütemező már inicializálva van. Az integrációs tesztben a megszakításokat rövid időre engedélyezni kell, hogy a kontextusváltás megtörténhessen, de a teszt végén mindig le kell tiltani őket, és a folyamatokat meg kell szüntetni. A tesztnek időtúllépést is tartalmaznia kell: ha a várt adat nem érkezik meg adott időn belül, a teszt `-ETIMEDOUT` hibával lép ki, hogy elkerülje a végtelen ciklust.

30.1.5 4. Boot tesztek

A boot teszt a rendszer indulását, a kezdeti hardverállapotot és a kernel legkorábbi inicializálását ellenőrzi. Ennek célja nem egyetlen függvény, hanem az a pillanat, amikor a kernel átveszi a vezérlést a firmware-től, felállítja a memóriatérképet, inicializálja a soros konzolt és a legfontosabb adatszerkezeteket.

A boot teszt jellemzően a következőket vizsgálja:

- a kernel belépési pontja elérhető és a várt címről indul,
- a boot log első sorai megjelennek a soros konzolon,
- a memóriatérkép fizikai és virtuális tartománya konzisztens,
- az időzítő és a megszakításvezérlő alapértelmezett állapota érvényes,
- a kernel verziószáma és build-azonosítója olvasható.

A boot tesztet gyakran nem a kernelben futó függvényként, hanem **külső Vezérlő** szkriptként valósítják meg: a QEMU elindítja a kernelt, a szkript a soros port kimenetét figyeli, és reguláris kifejezésekkel ellenőrzi a várt sorokat. Ez azért hasznos, mert a kernel korai szakaszában még nem áll rendelkezésre a teljes tesztkeret, és a hibák egy része már a `test_run_all` hívása előtt bekövetkezik.

Példa boot teszt ellenőrző szabályokra:

```
[BOOT] os114_mini kernel 0.1.0
[BOOT] cpu=x86_64
[BOOT] phys_mem_base=0x00100000
[BOOT] phys_mem_size=64 MiB
[BOOT] virt_base=0xffffffff80000000
[BOOT] serial0=0x3f8 IRQ=4
[BOOT] timer=HPET 1000 Hz
[PASS] boot self-test
```

Ha a várt sorok sorrendje vagy tartalma eltér, a boot teszt hibát jelez. A külső vezérlő előnye, hogy a kernel nem tudja elrejteni a korai összeomlást; hátránya, hogy a tesztlogika a hoszton fut, ezért a hoszt és a vendég közötti soros port késleltetését, pufferelését és a QEMU verzióját is dokumentálni kell a reprodukcióhoz.

A fejezet laborjában a boot tesztet a kernelbe épített `boot_self_test()` függvény valósítja meg, amely a `test_run_all` előtt lefut, és az alapvető memóriatérképet, valamint a konzol írási képességét ellenőrzi. A külső vezérlő ezt a kimenetet is figyeli.

30.1.6 5. Mock és tesztdupla

A determinisztikus tesztek egyik legfontosabb eszköze a **mock**, azaz a valódi hardver vagy alrendszer olyan helyettesítése, amely a teszt számára előre meghatározott viselkedést biztosít. A kernelben tipikusan az időzítőt, a megszakításvezérlőt, a lapkezelőt vagy az eszközregisztereket kell mockolni.

A mockolás egyik gyakori mintája a **függvénypointer-indirekció**. A kernel egy `clock_source` struktúrában hivatkozik az aktuális időt lekérdező függvényre. Éles rendszerben ez a PIT vagy a TSC olvasását végzi, a tesztben viszont egy rögzített értéket ad vissza.

```c
/* clock.h - absztrakt óraforrás */
struct clock_source {
    uint64_t (*read_ns)(void);
    uint64_t (*read_ticks)(void);
};

extern struct clock_source *g_clock;

static inline uint64_t clock_read_ns(void)
{
    return g_clock->read_ns();
}
```

A mock óra tesztkódja:

```c
/* mock_clock.c - teszt óraforrás */
static uint64_t mock_ns = 0;

static uint64_t mock_read_ns(void)
{
    return mock_ns;
}

static struct clock_source mock_clock = {
    .read_ns = mock_read_ns,
    .read_ticks = 0,
};

void test_set_mock_ns(uint64_t ns)
{
    mock_ns = ns;
}

void test_install_mock_clock(void)
{
    g_clock = &mock_clock;
}
```

Ezzel a tesztek pontosan beállíthatják az időt, és nem függenek a hardver órajelétől, a QEMU emuláció sebességétől vagy a kontextusváltások közben eltelt időtől. A mock időforrás használata különösen fontos az ütemezési döntések és az időtúllépések tesztelésekor, ahol a valós időalap flaky teszteket okozna.

Eszközregiszterek esetén hasonló elv szerint járhatunk el: az alacsony szintű port- vagy MMIO-eléréseket egy vékony réteg mögé rejtjük, és a tesztben a regisztertérkép egy memóriatömböt kap. Így a teszt anélkül ellenőrizheti a driver logikáját, hogy valódi I/O portot vagy vendég hardvert érne el.

30.1.7 6. QEMU és a tesztkernel

A tesztkernel a kernel olyan speciális buildje, amely a normál init helyett a tesztfuttatót indítja el. A gyakorlatban ez jelentheti azt, hogy a kernel parancssori paraméterben megkapja a `--test` kapcsolót, vagy hogy a build során a `CONFIG_TEST=1` értékkel a `main` függvény a teszteket hívja, mielőtt az ütemezőt és az eszközöket teljesen elindítaná.

A QEMU alkalmas a tesztkernel futtatására, mert:

- reprodukálható hardverkonfigurációt ad,
- a soros port könnyen átirányítható a hoszt szabványos kimenetére,

- az `isa-debug-exit` eszközzel a vendégből kilépési kód adható vissza,
- a munkakönyvtár és a lemezkép eldobható, így a hoszt rendszere nem sérül.

Egy tipikus QEMU futtatási parancs a fejezet laborjához:

```
qemu-system-x86_64 \
    -drive format=raw,file=build/os114.img \
    -serial stdio \
    -display none \
    -no-reboot \
    -no-shutdown \
    -device isa-debug-exit,iobase=0xf4,iosize=0x04
```

A `-serial stdio` a soros portot a hoszt termináljára irányítja, a `-display none` letiltja a grafikus ablakot, a `-no-reboot` megakadályozza, hogy a vendég automatikusan újrainduljon, a `-no-shutdown` pedig a teszt végét követően ne lépjen ki a QEMU, ha a vendég nem ad kilépési kódot. Az `isa-debug-exit` eszközön keresztül a kernel a `0xf4` portra írt értékkel tudja befejezni a futást.

A kilépési kódot a kernel így állítja elő:

```
/* qemu_exit.c - szemléltető QEMU kilépés */
#include <stdint.h>

static inline void outb(uint16_t port, uint8_t value)
{
    __asm__ volatile ("outb %0, %1" : : "a"(value), "Nd"(port));
}

void qemu_exit(uint8_t code)
{
    outb(0xf4, code);
    /* Ha az eszköz nem létezik, a kernel itt marad;
       a valós laborban a -device kapcsolót mindig megadjuk. */
    for (;;) {
        __asm__ volatile ("hlt");
    }
}
```

A `test_run_all` visszatérési értékét a kernel a `qemu_exit` függvénynek adja át:

```
void kernel_main(void)
{
    boot_self_test();

    int rc = test_run_all(builtin_tests, ARRAY_SIZE(builtin_tests));
    qemu_exit(rc ? 1 : 0);
}
```

A QEMU a kilépési kódot a hoszt folyamat visszatérési értékeként adja vissza, így a tesztillesztő a `$?` változóból dönthet arról, hogy a futás sikeres volt-e. A napló ettől függetlenül is megmarad, mert a soros port tartalmát a tesztillesztő előre rögzíti.

30.1.8 7. Labor: kernel-test terv és tesztkeret

A labor célja, hogy egy kisméretű, oktatási célú kernelhez tesztterv et és tesztkeretet készíts. A kernel a korábbi fejezetekben bemutatott `os114_mini` nevű, multiboot2 protokollt használó rendszer, amelynek már van soros konzolja, listakezelője, pipe-ja és egyszerű ütemezője. A labor nem módosítja a hoszt bootloaderét vagy rendszerpartícióját; minden fájl az eldobható `build/` könyvtárban keletkezik.

30.1.9 7.1 Kiinduló állapot és cél

Kiinduló állapot: a `os114_mini` forrásfája a `src/` könyvtárban, a build script a `Makefile`-ban, és egy előre elkészített boot image a `build/os114.img` útvonalon. A hoszton QEMU 8.x és egy keresztfordító (`x86_64-elf-gcc`) áll rendelkezésre.

Cél: a `list`, `pipe` és `clock` alrendszerek tesztelése a következő szinteken:

- **Unit teszt:** `test_list_push_pop`, `test_list_empty_pop_returns_enoent`, `test_pipe_init`.
- **Integrációs teszt:** `test_pipe_rw_scheduler` (pipe írás/olvasás ütemezővel).
- **Boot teszt:** `boot_self_test` a memóriatérkép és a soros konzol ellenőrzésére.

30.1.10 7.2 Konfiguráció

A vendég konfigurációja:

Paraméter	Érték
CPU	`qemu-system-x86_64`, x86_64 mód, 1 mag
Memória	64 MiB
Soros port	`-serial stdio`
Kijelző	`-display none`
Kilépés	`-device isa-debug-exit,iobase=0xf4,iosize=0x04`
Boot image	`build/os114.img`
Fordítási beállítások	`-ffreestanding -O0 -g -Wall -Wextra`
Linker script	`src/kernel.ld`

A QEMU és a keresztfordító pontos verzióját a mérési naplóban rögzíteni kell, mert az emuláció sebessége és a soros port pufferelése verziófüggő lehet.

30.1.11 7.3 Build és futtatás

A tesztkernel buildje:

```
make clean
make CFLAGS="-ffreestanding -O0 -g -Wall -Wextra" test
```

A teszt futtatása és a napló rögzítése:

```
qemu-system-x86_64 \
    -drive format=raw,file=build/os114.img \
    -serial stdio \
    -display none \
    -no-reboot \
    -no-shutdown \
    -device isa-debug-exit,iobase=0xf4,iosize=0x04 \
    | tee build/test.log
rc=${PIPESTATUS[0]}
echo "QEMU exit code: $rc"
```

A `tee` a soros kimenetet a `build/test.log` fájlba is írja, a `PIPESTATUS[0]` pedig a QEMU tényleges kilépési kódját adja vissza. A kilépési kód 0, ha minden teszt sikeres volt.

30.1.12 7.4 Várt és mért eredmények

A tesztnapló minden tesztnél tartalmazza a teszt nevét, a várt és mért értéket, valamint a `PASS` vagy `FAIL` eredményt. A mérési naplóban a várt és mért értékeket külön oszlopban kell szerepeltetni, ahogy az alábbi táblázat mutatja.

Teszt	Várt eredmény	Mért eredmény	Állapot
`test_list_push_pop`	`rc=0, value=10`	`rc=0, value=10`	PASS
`test_list_empty_pop_returns_enoent`	`rc=-ENOENT`	`rc=-ENOENT`	PASS
`test_pipe_init`	`rc=0, available=0`	`rc=0, available=0`	PASS
`test_pipe_rw_schedule`	`data=expected, available=0`	`data=expected, available=0`	PASS
`boot_self_test`	`mem_ok=1, serial_ok=1`	`mem_ok=1, serial_ok=1`	PASS
Összesített QEMU kilépési kód	0	0	PASS

A `-ENOENT` értéket a naplóban numerikusan is fel kell tüntetni, például `-2`, hogy a hibakód egyértelmű legyen. A mért eredményt soha ne írjuk át a vártra; ha eltérés van, az a teszt hibáját jelzi, és a kernelkódot vagy a tesztet kell javítani.

30.1.13 7.5 Mérési napló és visszaállítás

A mérési napló a következő mezőket tartalmazza:

```
Mérési napló
Időpont: 2025-02-20 14:32:00 UTC
Hoszt: linux, x86_64
QEMU: qemu-system-x86_64 8.2.0
Fordító: x86_64-elf-gcc 13.2.0
Kernel: os114_mini 0.1.0, CONFIG_TEST=1
Indítási parancs: lásd fent
Mért teljes futási idő: 1,84 s
Kilépési kód: 0
Sorozatszám: os114-lab-0007
```

A futási időt a hoszt `time` parancsával mérjük, és a naplóban feltüntetjük. A vendégben eltelt időt a mock óra miatt nem mérjük, mert az nem a valós hardveridőt tükrözi. A mérési körülmények között a QEMU emuláció sebessége nem releváns a determinisztikus tesztek szempontjából.

Visszaállítás:

```
make clean
rm -f build/test.log build/os114.img
git checkout -- .
```

A `make clean` törli a generált objektumfájlokat, a `git checkout` pedig visszaállítja a forrásfát a kiinduló állapotra. A hoszt lemezét vagy bootloaderét nem módosítjuk.

30.1.14 8. Tipikus hibák és helyes megoldások

A kernelben futó tesztek során a következő hibák gyakoriak, és mindegyiknek van egyértelmű, tesztelési szintű megoldása.

A megszakítások bekapcsolva maradnak a unit teszt alatt. Ha a teszt függvény nem tiltja le a megszakításokat, az időzítő vagy a megszakításkezelő kontextust válthat, és a teszt által vizsgált adatszerkezetet megváltoztathatja. Helyes megoldás: a unit tesztek elején `cli`, a végén `sti`, vagy a tesztkeret globálisan tiltja a megszakításokat, és az integrációs tesztek külön engedélyezik azokat.

Valós időalap használata időtúllépéses tesztben. A valós PIT vagy TSC alapú várakozás flaky tesztet okoz, mert az emulált órajel ingadozhat. Helyes megoldás: az időzítőt mockold, és a tesztben a `test_set_mock_ns()` hívással lépj előre az időben.

A tesztkód eltakarítja a hibát. Ha a teszt a hiba észlelése előtt `return 0`-t ad vissza, a keret sikeresnek könyveli el. Helyes megoldás: minden assert azonnal visszatérjen nem nulla hibakóddal, és a keret a hibák számát összegezze.

A boot log sorrendjét nem ellenőrzik. A soros port pufferelése miatt a log sorok késhetnek, de ha a teszt csak a tartalmat keresi, a korai megszakítás- vagy memóriahiba elsikkadhat. Helyes megoldás: a boot teszt a sorokat a rögzített sorrendben, reguláris kifejezéssel illeszti, és időtúllépést alkalmaz.

A kilépési kód nem jut el a QEMU-ig. Ha a kernel a `qemu_exit` hívás előtt pánikba esik, a hoszt nem kap érvényes kilépési kódot. Helyes megoldás: a pánikkezelő maga is írjon a `0xf4` portra egy hibakódot, vagy a tesztillesztő a várt `PASS` sor hiányát hibának tekintse.

A tesztkernel éles kódot futtat. A teszt során az init rendszer vagy a valódi eszközmeghajtók betöltése nem kívánatos. Helyes megoldás: a tesztbuildben a `CONFIG_TEST=1` definícióval a kernel korán, a teljes init előtt a tesztfuttatóra ugrik, és soha nem módosít valódi lemezt.

30.1.15 9. Önellenőrző kérdések

1. Miért nem elegendő a felhasználói térben megszokott `assert()` a kernelben futó tesztekhez?
2. Milyen előnye van a függvénypointer-indirekcióval megvalósított mock órának a valós időzítővel szemben?
3. Miért kell a boot tesztet a kernelben futó tesztkerettől függetlenül, külső vezérlővel is ellenőrizni?
4. Milyen hibához vezet, ha az integrációs teszt a megszakításokat nem engedélyezi az ütemezőváltáshoz?

30.1.16 10. Összefoglalás

A kernelkód tesztelése akkor válik megbízhatóvá, ha a tesztkörnyezetet ugyanolyan gondosan tervezzük meg, mint magát a kernelt. A unit tesztek a logikai építőelemeket szigetelik el, az integrációs tesztek az alrendszerek együttműködését igazolják, a boot tesztek pedig a teljes indítási lánc épségét biztosítják. A mockok és a QEMU-ban futó tesztkernel lehetővé teszik, hogy a tesztek determinisztikusak, reprodukálhatók és biztonságosak legyenek.

A következő fejezet (Emuláció, fuzzing és statikus elemzés) az itt bemutatott tesztkeretre építve a fuzzingot, a statikus elemzést és az emuláció alapú hibakeresést vizsgálja. A laborban elkészített napló és a várt/mért eredmények elkülönítése ott is az ellenőrzés alapját képezi.

30.2 Emuláció, fuzzing és statikus elemzés

Az automatikus feltárás akkor válik szükségessé, amikor a kézzel írt unit- és integrációs tesztek már nem elégségesek a kernel bonyolult állapotterének lefedésére. A kernel tipikus hibái – use-after-free, puffer-túlcsordulás, versenyhelyzet – gyakran csak akkor jelentkeznek, ha a rendszerhívások speciális sorrendben, rosszindulatú vagy váratlan bemenetekkel találkoznak. Ebben az egységben három, egymást kiegészítő technikát kapcsolunk össze: a fuzzingot, a dinamikus detektorokat (elsősorban a KASAN-t), valamint a statikus elemzőket. A cél nem egyetlen eszköz megtanulása, hanem egy olyan munkafolyamat kialakítása, amely a reprodukálható hibajelentéstől a triage-on át a javításig vezet.

30.2.1 A hibák automatikus feltárása

A kernelben a hibák nagy része nem determinisztikus, hanem inputfüggő. Egy manuális teszt azt ellenőrzi, hogy a fejlesztő által elképzelt helyes bemenetre a rendszer helyesen válaszol. A fuzzing ezzel szemben azt kérdezi: mi történik akkor, ha a bemenet hibás, hiányos, túl hosszú, rosszul igazított, vagy éppen a rendszerhívások olyan kombinációját adjuk, amelyet a tervező nem látott előre?

Az automatikus feltárás három rétegből épül fel:

1. **Bemenetgenerálás** – a fuzzer véletlenszerű vagy mutált bemeneteket állít elő. Ez lehet fájltartalom, hálózati csomag, rendszerhívás-szekvencia vagy akár eszközregiszter-állapot.
2. **Dinamikus detektálás** – a futás közbeni ellenőrzés hibát jelez, amikor a program megsérti a memória- vagy típusbiztonságot. Ide tartozik a KASAN, a KCSAN, a KMSAN és az UBSAN.
3. **Statikus elemzés** – a forráskódot elemzi anélkül, hogy a program futna, és olyan mintákat keres, amelyek hibához vezethetnek.

A három réteg együtt ad lefedettséget: a fuzzing olyan utakat talál meg, amelyeket a kézi tesztelés nem; a dinamikus detektorok jelzik, ha egy út hibát vált ki; a statikus elemzők pedig a kód egészét vizsgálják, akár ritkán futó ágakat is.

30.2.2 Fuzzing és a syzkaller

A fuzzing legegyszerűbb formája a „dumb fuzzing”, amikor a fuzzer egy bemeneti fájlt véletlenszerűen módosít, és figyeli, hogy a program összeomlik-e. A kernel esetében azonban a bemenet nem fájl, hanem a rendszerhívások sorozata. Ezért a modern kernelfuzzerek, mint a **syzkaller**, úgynevezett *coverage-guided* fuzzingot végeznek: a fuzzer nemcsak a crash-eket figyeli, hanem azt is, hogy az egyes bemenetek a kernel mely kódágait futtatják le.

A syzkaller architektúrája a következő komponensekből áll:

- **syz-manager** – központi vezérlő, amely a virtuális gépeket indítja, a corpus-t kezeli, és a talált hibákat naplózza.
- **syz-executor** – a virtuális gépen belül futó kis program, amely a rendszerhívásokat végrehajtja.
- **KCOV** – a kernelbe épített lefedettségmérő, amely megmondja, hogy az adott bemenet melyik forrássorokat érintette.
- **corpus** – a lefedettséget növelő bemenetek gyűjteménye. A fuzzer ebből indul ki, és újabb mutációkat próbál ki.

- **triage** – a syz-manager által generált hibajelentések rendszerezése, deduplikálása és minimálisra csökkentése.

A syzkaller működésének lényege, hogy a fuzzer csak azokat a bemeneteket tartja meg a corpusban, amelyek új lefedettséget érnek el. Ez a visszacsatolás teszi lehetővé, hogy a fuzzer ne csak véletlenszerűen próbálkozzon, hanem fokozatosan mélyebbre hatoljon a kernel kódjába.

Egy egyszerűsített syzkaller konfiguráció – amely természetesen csak laboratóriumi, emulált környezetben érvényes – így nézhet ki:

```
# syzkaller-config.yaml (részlet, helyőrző értékekkel)
target: linux/amd64
http: "127.0.0.1:56741"
workdir: "/tmp/syzkaller-work"
kernel_obj: "/home/user/kbuild"
image: "/tmp/qemu-image/bzImage"
sshkey: "/tmp/syzkaller-work/id_ed25519"
sandbox: none
type: qemu
vm:
  count: 4
  kernel: "/tmp/qemu-image/bzImage"
  cmdline: "console=ttyS0 root=/dev/sda rw kasan_multi_shot=1"
  cpu: 2
  mem: 2048
```

A fenti konfiguráció nem valós gyártói értékeket tartalmaz, hanem egy eldobható QEMU-környezetet definiál. A `qemu` típus azt jelenti, hogy a fuzzer virtuális gépet indít. A `kasan_multi_shot=1` paraméter lehetővé teszi, hogy a KASAN több hibát is jelentsen egyetlen boot során, ami felgyorsítja a fuzzingot.

A futtatás parancsa – a saját kernelkönyvtárban, ahol a kernel konfigurációja már tartalmazza a `CONFIG_KCOV=y` és `CONFIG_KASAN=y` beállításokat – a következő lehet:

```
# Feltételezzük, hogy a kernel már le van fordítva KASAN és KCOV támogatással.
mkdir -p /tmp/syzkaller-work
./syz-manager -config syzkaller-config.yaml -vv 10
```

A fuzzer kimenete nem közvetlenül a hibát mutatja, hanem egy *reproducer* fájlt generál. Ha a fuzzer egy crash-t észlel, a `workdir/crashes/<hash>` könyvtárba menti a hibajelentést és a szükséges bementet. A syzkaller által ismert három fő reproducer formátum:

- **C reproducer** – a lefordítható C program, amely a hibát kiváltó rendszerhívás-sorozatot tartalmazza.
- **syz reproducer** – a syzkaller saját, magas szintű leíró nyelve.
- **prog** – a fuzzer belső reprezentációja, amely a mutációkhoz használható.

Egy C reproducer vázlata – ismét szemléltető jelleggel, nem futtatandó éles gépen – a következő lehet:

```
#include <sys/syscall.h>
#include <fcntl.h>
#include <unistd.h>

int main(void)
{
    int fd = syscall(__NR_openat, AT_FDCWD, "/dev/testdev0", O_RDWR, 0);
    if (fd >= 0) {
        syscall(__NR_ioctl, fd, 0xdeadbeef, 0x1000);
        syscall(__NR_close, fd);
```

```
    }
    return 0;
}
```

Ez a program egy feltételezett eszközmeghajtó hibáját váltja ki azzal, hogy az `ioctl` rendszerhíváson keresztül egy nem várt parancskódot és mutatót ad át. A C reproducer előnye, hogy a fejlesztő közvetlenül lefordíthatja és futtathatja a hibás kernelben, majd a javítás után ugyanezt a programot használhatja regressziós tesztként.

A fuzzing azonban önmagában nem elegendő. A fuzzer csak akkor tudja jelezni a hibát, ha a hibás kód fut, és a hibás viselkedés valamilyen detektor által észlelhető. Erre szolgálnak a dinamikus detektorok.

30.2.3 A KASAN detektor

A **KASAN** (Kernel Address Sanitizer) a kernelbe épített memóriahiba-detektor. A feladata, hogy a puffer-túlcsordulást, a use-after-free hibát és a stack-buffer overflow-t közvetlenül a hiba bekövetkezésekor jelezze. A KASAN úgy működik, hogy minden memória-hozzáférést egy árnyékmemória-ellenőrzéssel egészít ki.

A KASAN árnyékmemóriája a kernel virtuális címtartományának egy elkülönített részén helyezkedik el. Minden 8 bájtos memóriaterülethez egy 1 bájtos árnyékérték tartozik. A leképezés közelítőleg:

```
shadow = (addr >> 3) + KASAN_SHADOW_OFFSET
```

A `KASAN_SHADOW_OFFSET` architektúrafüggő konstans, x86-64 esetén a `0xdffffc0000000000` körüli tartományba esik. A pontos értéket a kernel konfigurációja és a linker script határozza meg; a laborban ezt a lefordított kernel szimbólumtáblájából lehet kiolvasni.

A KASAN a memória lefoglalásakor úgynevezett *redzone* területeket helyez el az objektumok köré. Ha a program ezt a redzone-t olvassa vagy írja, a KASAN azonnal hibát jelent. A use-after-free észleléséhez a KASAN a felszabadított memóriát speciális árnyékértékkel jelöli meg, és a következő hozzáférésnél jelzi a hibát.

Egy KASAN hibajelentés – szándékosan egyszerűsített, laboratóriumi naplóból vett forma – így néz ki:

```
BUG: KASAN: slab-out-of-bounds in demo_ioctl at drivers/char/demo.c:123
Read of size 8 at addr ffff888012345678 by task repro/4

CPU: 1 PID: 1234 Comm: repro Not tainted 5.15.0-test+
Call Trace:
 dump_stack_lvl+0x34/0x48
 print_address_description+0x3f/0x58
 kasan_report+0x1a2/0x1e0
 demo_ioctl+0x56/0x80
 __x64_sys_ioctl+0x82/0xb0
 do_syscall_64+0x34/0x80
 entry_SYSCALL_64_after_hwframe+0x44/0xae
```

A jelentésből kiolvasható a hiba típusa (`slab-out-of-bounds`), a hozzáférés mérete (`Read of size 8`), a hibás cím (`ffff888012345678`), a folyamat neve (`repro`), valamint a hívási lánc. Ez a három adat – típus, cím, stack trace – elegendő a hiba azonosításához és a forráskódban való lokalizálásához.

A KASAN mellett további sanitizerek léteznek:

- **KCSAN** – versenyhelyzetek felderítése a kernelben.
- **KMSAN** – nem inicializált memória olvasásának észlelése.
- **UBSAN** – nem definiált viselkedés (pl. integer overflow, rossz shift) észlelése.

A gyakorlatban a fuzzing és a KASAN együtt ad erős kombinációt: a fuzzer megkeresi azokat az utakat, amelyek a hibás kódhoz vezetnek, a KASAN pedig jelzi, ha az út során memóriahiba lép fel. Ezt a kombinációt nevezzük *sanitizer-guided fuzzingnak*.

30.2.4 Statikus elemzés

A statikus elemzés a forráskódot futás nélkül vizsgálja. A kernel esetében ez azért fontos, mert sok hiba olyan ritka ágakban van, amelyeket a fuzzing nem ér el. A statikus elemzők képesek a teljes forráskódot átvizsgálni, és olyan mintákat találni, amelyek a fejlesztő figyelmét elkerülték.

A Linux kernel ökoszisztémájában a leggyakrabban használt statikus elemzők:

- **sparse** – a kernel saját annotációinak ellenőrzésére szolgál, például `__user`, `__kernel`, `__rcu` címkékre.
- **smatch** – a forráskódban mintákat keres, például hibás null-ellenőrzést, rossz memória-felszabadítást vagy elfelejtett zárolást.
- **clang –analyze** – a Clang statikus analizátora, amely útvonalkövető elemzést végez.
- **gcc -fanalyzer** – a GCC saját statikus elemzője.
- **coccinelle** – szemantikus patch-ek írására szolgál, amelyekkel a kódban mintákat lehet keresni és akár automatikusan javítani.

A statikus elemzők nem a futás közbeni viselkedést figyelik, hanem a forráskódban leírt utakat. Ezért előnyük, hogy akár a legritkábban futó hibás ágat is megtalálhatják, hátrányuk viszont a hamis pozitívok magas aránya, valamint az, hogy a dinamikus állapotokat (pl. lock-ok tényleges értéke) nem látják.

Egy tipikus statikus elemzési hiba, amelyet a smatch vagy a clang –analyze jelez:

```
static int demo_copy_from_user(void __user *u, int len)
{
    char buf[64];

    if (len > sizeof(buf))
        return -EINVAL;

    if (copy_from_user(buf, u, len))
        return -EFAULT;

    buf[len] = '\0';

    return 0;
}
```

Ebben a kódban a `len` értékére vonatkozó ellenőrzés hibás. Ha `len == 64`, akkor a feltétel `64 > 64` hamis, tehát a kód tovább fut. A `buf` tömb mérete 64 bájt, így a `buf[64] = '\0'` hozzáférés túlcsordul a pufferen. A helyes ellenőrzés:

```
if (len >= sizeof(buf))
    return -EINVAL;
```

Ez a példa jól mutatja, hogy a statikus elemző nem a futás közbeni hibát látja, hanem az ellenőrzés és a későbbi hozzáférés közötti ellentmondást. A `smatch` ilyenkor a következő jellegű figyelmeztetést adja:

```
drivers/char/demo.c:123 demo_copy_from_user() error: buffer overflow 'buf' 64 <= 64
```

A statikus elemzőket a kernel build folyamatába lehet integrálni. Például a `make C=1` a sparse-t, a `make C=2` a smatch-et futtatja a teljes kernelre. A laborban ezeket kis, jól körülhatárolt könyvtárakra futtatjuk, hogy a kimenet kezelhető maradjon.

30.2.5 Reprodukálhatóság és triage

A fuzzing és a statikus elemzés önmagában csak nyers hibajelentéseket ad. A hibák haszna akkor realizálódik, ha a fejlesztő reprodukálni tudja a problémát, és a megfelelő javítást el tudja készíteni. Ezt a folyamatot nevezzük *triage*-nak.

A reprodukálhatóság alapfeltétele, hogy a hibajelentés tartalmazza:

- a kernel pontos verzióját és commit hash-ét,
- a kernel konfigurációját (`defconfig` vagy a használt `.config`),
- a virtuális gép vagy hardver konfigurációját (CPU, memória, eszközök),
- a fuzzer vagy a statikus elemző verzióját,
- a hiba kiváltásához szükséges bemenetet (C reproducer, syz reproducer vagy a fuzzer belső formátuma).

A triage során a fejlesztő először megpróbálja a hibát reprodukálni. Ha a reproducer nem fut, akkor a hiba nem tekinthető ellenőrzött eredménynek. Ha a reproducer fut, de a hiba más formában jelentkezik, akkor a hiba osztályozását módosítani kell.

A triage lépései:

1. **Reprodukálás** – a reproducer futtatása a rögzített konfigurációban.
2. **Minimalizálás** – a reproducer egyszerűsítése, hogy a hiba okát könnyebb legyen azonosítani.
3. **Deduplikálás** – annak eldöntése, hogy a hiba új-e, vagy egy már ismert hiba más reprodukciója. Ehhez a syzkaller a stack trace hash-ét használja.
4. **Besorolás** – a hiba típusának meghatározása (use-after-free, buffer overflow, race stb.).
5. **Bisect** – annak a commitnak a megkeresése, amely a hibát bevezette.
6. **Patch** – a javítás elkészítése és tesztelése.
7. **Regressziós teszt** – a reproducer beépítése a tesztek közé, hogy a hiba ne térhessen vissza.

Egy triage-jegyzet a laborban így néz ki:

```
Triage jegyzet - példa

Hibaazonosító: os115-20240912-001
Kernel commit: 5.15.50-test (helyi commit)
Konfiguráció: lab-kasan.config
Reproducer: repro.c
Státusz: reprodukálható
Besorolás: slab-out-of-bounds
Stack trace hash: 3f2a1b7c9d4e
Érintett komponens: drivers/char/demo.c
Kapcsolódó ismert hiba: nincs
Javítás: https://example.invalid/patch/001 (helyőrző)
```

A fenti jegyzetben a `https://example.invalid` helyőrző, nem valós belső URL. A laborban a hallgatók a saját fejlesztői környezetükben rögzítik a pontos adatokat.

30.2.6 Tipikus hibák és helyes megoldások

Az automatikus feltárás során leggyakrabban a következő hibákat találjuk. Mindegyiknél megadjuk a detektáló eszközt, egy rövid hibás példát és a helyes megoldás irányát.

30.2.7 Use-after-free

A use-after-free akkor keletkezik, amikor egy memóriaterületet felszabadítunk, majd később még mindig hivatkozunk rá. A KASAN ezt a felszabadított memória speciális árnyékértékével érzékeli.

```
static int demo_release(struct inode *inode, struct file *file)
{
    struct demo_dev *dev = file->private_data;

    kfree(dev);
    file->private_data = NULL;
    return 0;
}

static long demo_ioctl(struct file *file, unsigned int cmd, unsigned long arg)
{
    struct demo_dev *dev = file->private_data;

    if (cmd == DEMO_GET_STATE)
        return dev->state;  /* use-after-free, ha a release lefutott */

    return 0;
}
```

A hiba akkor jelentkezik, ha a `release` felszabadítja a `dev` struktúrát, de az `ioctl` később még hivatkozik rá. A helyes megoldás az, hogy a felszabadítás előtt biztosítjuk, hogy a struktúrára nincs több hivatkozás. Ezt lehet refcount-tal, RCU-val vagy a `file->private_data` és a lezárás sorrendjének gondos kezelésével megoldani.

30.2.8 Puffer-túlcsordulás

A puffer-túlcsordulás tipikusan a méretek rossz ellenőrzéséből fakad. A KASAN a redzone megsértését jelzi.

```
static int demo_read(struct file *file, char __user *ubuf, size_t count, loff_t *ppos)
{
    char local[16];

    if (count > sizeof(local))
        return -EINVAL;

    if (copy_from_user(local, ubuf, count))
        return -EFAULT;

    /* ... */
}
```

A fenti példában a `count == 16` esetén a `copy_from_user` pontosan 16 bájtot másol, de a `local` tömb mérete is 16, így a lezáró null bájt esetleges hozzáadása túlcsordul. A helyes ellenőrzés `count >= sizeof(local)`.

30.2.9 Versenyhelyzet

A versenyhelyzeteket a KCSAN érzékeli. A kernelben a közös adatokat megfelelő zárral kell védeni.

```
static int demo_count;

static ssize_t demo_show(struct device *dev, struct device_attribute *attr,
                         char *buf)
{
    return sprintf(buf, "%d\n", demo_count);
}

static ssize_t demo_store(struct device *dev, struct device_attribute *attr,
                          const char *buf, size_t count)
{
    demo_count = simple_strtol(buf, NULL, 10);
    return count;
}
```

Itt a `demo_count` olvasása és írása nem szinkronizált. A helyes megoldás egy spinlock vagy mutex bevezetése, amely a kritikus szakaszt védi.

30.2.10 Nem inicializált memória

A nem inicializált memória olvasását a KMSAN érzékeli. A hiba forrása gyakran az, hogy egy struktúra egyes mezőit nem állítjuk be, vagy a `copy_to_user` számára nem inicializált puffert adunk át.

```
struct demo_item {
    unsigned long flags;
    unsigned int state;
};

int demo_get_state(void __user *u)
{
    struct demo_item item;

    item.flags = 0;
    /* item.state nincs inicializálva */

    if (copy_to_user(u, &item, sizeof(item)))
        return -EFAULT;

    return 0;
}
```

A helyes megoldás: a struktúra minden mezőjét inicializálni kell, például `= { }` vagy `memset` hívással.

30.2.11 Integer overflow

Az integer overflow akkor következik be, amikor egy aritmetikai művelet eredménye nem fér el a típus által megengedett tartományban. Az UBSAN ezt jelzi.

```
static int demo_alloc(size_t count)
{
    size_t size = count * sizeof(struct demo_item);

    return kmalloc(size, GFP_KERNEL) != NULL ? 0 : -ENOMEM;
}
```

Ha a `count` nagy, a `size` overflow-ol, és a `kmalloc` kis memóriát foglal, majd a későbbi hozzáférés túlcsordul. A helyes megoldás: a szorzás előtt ellenőrizni kell, hogy az eredmény nem haladja-e meg a megengedett maximumot, például a `check_mul_overflow` segítségével.

30.2.12 Labor: fuzzing- és statikus elemzési terv

A labor célja, hogy a hallgatók elkészítsék és dokumentálják egy kernelmodul vagy saját kernelkomponens fuzzing- és statikus elemzési tervét. A labor nem az éles rendszeren fut, hanem QEMU-ban, eldobható kernelképpel. A mérési naplóban a mért és a várt eredményeket külön kell feltüntetni.

30.2.13 Kiinduló állapot

- A saját OS projekt forráskódja a `~/os-project` könyvtárban.
- A QEMU telepítve, a kernel lefordítva a `build/lab-kasan` könyvtárba.
- A kernel konfiguráció tartalmazza a `CONFIG_KASAN=y`, `CONFIG_KCOV=y`, `CONFIG_DEBUG_FS=y` és a szükséges `CONFIG_FAULT_INJECTION` beállításokat.
- A fuzzer binárisa a `~/tools/syzkaller/bin/syz-manager` útvonalon található (ellenőrizendő verzió).

30.2.14 Build parancs

```
cd ~/os-project
make ARCH=x86_64 O=build/lab-kasan menuconfig
# Engedélyezzük a KASAN és KCOV opciókat, majd:
make ARCH=x86_64 O=build/lab-kasan -j$(nproc)
```

30.2.15 Futtatás és fuzzing

A fuzzer indítása előtt rögzíteni kell a mérési konfigurációt:

- CPU: 2 virtuális mag
- Memória: 2048 MiB
- Kernel: `build/lab-kasan/arch/x86/boot/bzImage`
- Initrd: `build/lab-kasan/initramfs.cpio.gz` (minimális busybox)
- QEMU verzió: a labor jegyzetfüzetben rögzítendő
- Fuzzer verzió: a labor jegyzetfüzetben rögzítendő

A fuzzer indítása:

```
cd ~/tools/syzkaller
./bin/syz-manager -config ~/os-project/tests/syz-linux.yaml -vv 10
```

A futtatást legalább 10 percig, vagy az első reprodukálható crash-ig kell végezni. A config-ban használt `workdir` legyen `/tmp/os115-syz`, az image útvonala a fent említett kernel.

30.2.16 Statikus elemzés

A fuzzing mellett a statikus elemzőket is futtatni kell a saját kernelmodulokra:

```
cd ~/os-project
make ARCH=x86_64 O=build/lab-kasan C=2 drivers/char/demo/
```

A `C=2` a smatch-et futtatja a megadott könyvtárra. Emellett a Clang statikus analizátora is használható:

```
cd ~/os-project
scan-build make ARCH=x86_64 O=build/lab-kasan drivers/char/demo/
```

A `scan-build` kimenetét a `~/os-project/tests/static-analysis/` könyvtárba kell menteni.

30.2.17 Mérési napló

A laborban a következő táblázatot kell kitölteni. A „Várt” oszlop a labor előtt meghatározott elvárás, a „Mért” oszlop a ténylegesen megfigyelt érték.

Menet	Konfiguráció	Várt incidens	Mért incidens	Megjegyzés
1	syzkaller seed nélkül, 5 perc	0 új jel	(kitöltendő)	
2	syzkaller seed corpus-szal, 10 perc	legalább 1 crash vagy 0	(kitöltendő)	
3	KASAN boot teszt	nincs KASAN hiba	(kitöltendő)	
4	smatch a demo könyvtárra	0 false positive	(kitöltendő)	
5	UBSAN boot teszt	nincs integer overflow	(kitöltendő)	

A táblázatban az „incidens” jelentése: KASAN/UBSAN/KCSAN hiba, kernel oops, vagy a fuzzer által jelentett crash. A „seed corpus” a korábban gyűjtött lefedettséget növelő bemenetek halmaza.

30.2.18 Elvárt eredmény

A labor elvárt eredménye egy **reprodukálható jegyzetfüzet**, amely tartalmazza:

- a build és futtatási parancsokat,
- a konfigurációs fájlokat,
- a fuzzer által talált hibák listáját,
- a statikus elemzők kimenetét,
- a triage-jegyzeteket,
- a mérési napló kitöltött táblázatát.

A jegyzetfüzetnek tartalmaznia kell a mérési módszer leírását is: milyen hardveren (virtuális gép paraméterei), milyen fordítási beállításokkal, és milyen órajel mellett futottak a mérések. A mért és a várt eredményt egyértelműen el kell különíteni.

30.2.19 Visszaállítás

A labor után a `/tmp/os115-syz` munkakönyvtárat törölni kell, a virtuális gép lemezét vissza kell állítani az eredeti pillanatképre, és a kernel build könyvtárát – ha módosult – újra kell építeni a tiszta konfigurációval. A valós hoszt rendszerpartíciójához a labor során tilos hozzáférni.

30.2.20 Összefoglalás

Az automatikus feltárás a kernelminőség három pillérére épül: a fuzzingra, amely új kóduatkat talál, a dinamikus detektorokra, amelyek a futás közbeni hibákat jelzik, és a statikus elemzésre, amely a teljes forráskódot vizsgálja. A három technika együttesen képes olyan hibákat is felszínre hozni, amelyek a manuális tesztelés során rejtve maradnának. A fuzzing és a statikus elemzés azonban csak akkor hasznos, ha a hibákat reprodukálhatóan dokumentáljuk, és a triage folyamatban a javításig követjük. A labor során a hallgatók ezt a teljes ciklust gyakorolják egy eldobható QEMU-környezetben.

30.2.21 Önellenőrző kérdés

Mi a különbség a fuzzing, a KASAN és a statikus elemzés között abban, ahogyan a hibákat megtalálja? Adjon egy-egy példát arra, hogy egy adott kernelhibát melyik eszköz milyen módon jelezne.

30.3 Teljesítménymérés és tracing

A kernel teljesítmény- és viselkedésmérése nem utólagos díszítés, hanem a tervezés és a hibakeresés szerves része. A korábbi fejezetekben a helyességre, a tesztelésre és a statikus elemzésre koncentráltunk; itt azt vizsgáljuk, hogyan lehet a futó rendszerből olyan adatokat kinyerni, amelyek alapján a rendszer viselkedése megérthető, a feltételezések ellenőrizhetők, és a regressziók korán észlelhetők. A cél nem az, hogy egyetlen „jó” számot kapjunk, hanem az, hogy reprodukálható módon, a mérési körülmények ismeretében tudjunk dönteni.

A fejezet a **latency**, a **throughput** és a **CPU cycle** fogalmából indul ki, majd bemutatja a Linux kernel leggyakoribb megfigyelési eszközeit: az **ftrace**-et, a **perf**-et és az **eBPF**-et. A hangsúly a mérési terv elkészítésén és a tipikus hibák felismerésén van, nem pedig az eszközök teljes referencia-dokumentációján. A bemutatott parancsok és kimenetek szemléltetésnek számítanak; az aktuális verziók és kapcsolók mindig a felhasznált kernel és disztribúció dokumentációjában ellenőrizendők.

30.3.1 1. Alapfogalmak: mit mérünk?

30.3.2 1.1. Latency

A **latency** (késleltetés) az az idő, amely egy esemény kezdeményezése és a hatása között eltelik. Kernelkontextusban tipikus események:

- rendszerhívás belépésétől a visszatérésig eltelt idő;
- megszakítás beérkezésétől a megszakításkezelő első utasításáig eltelt idő;
- ütemező döntésétől a folyamat tényleges futásáig eltelt idő;
- lemezolvasási kérelem kiadásától az adat rendelkezésre állásáig eltelt idő.

A latency mértékegysége idő, tipikusan nanoszekundum (ns), mikroszekundum (µs) vagy milliszekundum (ms). A mérés során három értéket szokás megkülönböztetni:

- **átlagos latency**: sok minta számtani közepe;
- **percentilis (pl. P50, P95, P99)**: az az érték, amelynél a minták adott százaléka kisebb vagy egyenlő;
- **maximális (worst-case) latency**: a legnagyobb megfigyelt érték.

Az átlag önmagában félrevezető lehet, mert elrejti a kiugró késleltetéseket. Például ha ezer rendszerhívásból 999 100 ns alatt tér vissza, egy pedig 10 ms-ot várakozik, az átlag 10,1 µs, ami nem reprezentálja a tipikus viselkedést. Ezért a latency elemzésnél a percentilisek és a maximum legalább olyan fontosak, mint az átlag.

A latency és a throughput között gyakran átváltás van: a nagyobb átbocsátóképesség érdekében a rendszer összevárhat műveleteket (batch-elés), ami növeli az egyedi kérések késleltetését. A mérési tervben ezért mindig rögzíteni kell, hogy a vizsgált rendszer latency- vagy throughput-orientált-e.

30.3.3 1.2. Throughput

A **throughput** (átbocsátóképesség) az időegység alatt elvégzett munka mennyisége. Kernelben mérhető például:

- rendszerhívás/s vagy művelet/s;
- csomag/s (hálózati stack);
- blokk I/O művelet/s vagy MiB/s;
- kontextusváltás/s.

A throughput mértékegysége tehát művelet per másodperc, bájt per másodperc vagy hasonló. A mérés során fontos megkülönböztetni a **csúcsteljesítményt** (peak throughput) és a **fenntartható teljesítményt** (sustained throughput). A csúcsértéket rövid ideig, ideális körülmények között lehet elérni; a fenntartható érték hosszú távon, terhelés alatt is stabil marad.

A throughput és a latency közötti összefüggést egyszerű sorbanállási modellekkel lehet becsülni. Ha egy rendszer egyszerre egy kérést szolgál ki, és a kiszolgálási idő átlagosan S, akkor a maximális throughput $1/S$. Ha a rendszer párhuzamosan több kérést kezel, a Little-törvény szerint $N = \lambda \cdot W$, ahol N a rendszerben lévő kérések száma, λ a beérkezési ráta (throughput), W pedig az átlagos tartózkodási idő (latency). Ebből látható, hogy a latency és a throughput nem független, de a mérés során külön kell őket rögzíteni, mert más-más optimalizálási célokat szolgálnak.

30.3.4 1.3. CPU cycle

A **CPU cycle** (processzorórajel-ciklus) a processzor órajelének egy periódusa. A modern processzorokban a végrehajtott utasítások száma és a felhasznált ciklusok száma nem azonos, mert a pipeline, az out-of-order végrehajtás, a cache-találatok és a branch prediction mind befolyásolják, hogy egy utasítás hány ciklust igényel. A ciklusszám mérése azért hasznos, mert:

- független az aktuális órajeltől, ezért különböző rendszereken összehasonlíthatóbb, mint a nyers idő;
- megmutatja, hogy egy kódrészlet mennyire terheli a processzort;
- a hardveres számlálókkal (pl. `cpu-cycles`, `instructions`) együtt felhasználható az IPC (instructions per cycle) kiszámítására.

A ciklusszám és az idő közötti átszámítás az órajel ismeretében lehetséges: $t = \frac{C}{f}$, ahol t az idő, C a ciklusszám, f pedig az órajelfrekvencia. A modern processzorok azonban dinamikusan változtatják az órajelet (turbo boost, energiagazdálkodás), így a mérés során rögzíteni kell az órajel-viselkedést, vagy hardveres számlálóval kell mérni a ciklusszámot. A virtualizált környezetben a vendég CPU által látott órajel és a tényleges fizikai órajel eltérhet, ezért a mérési jegyzőkönyvben a virtualizációs réteget is fel kell tüntetni.

A CPU cycle mérése különösen fontos akkor, ha a rendszer időmérését megszakítások vagy laphibák torzíthatják. A hardveres számláló közvetlenül a processzor belsejéből származik, ezért kevésbé zavarja a szoftveres overhead.

30.3.5 2. Mérési módszertan: a reprodukálható terv

A kernel viselkedésének mérésekor a legnagyobb kockázat nem az eszköz hiánya, hanem a nem dokumentált körülményekből származó hamis következtetés. A fejezetben ezért a mérési tervet ugyanolyan komolyan kell venni, mint a kódot.

30.3.6 2.1. A mérési terv kötelező elemei

Minden reprodukálható mérésnek tartalmaznia kell:

1. **Cél**: mit akarunk mérni, és milyen döntést támogat a mérés.
2. **Hardver és virtualizáció**: CPU típus, magok száma, memória, QEMU verzió, hypervisor beállítások.
3. **Szoftver és kernelkonfiguráció**: kernelverzió, fontos konfigurációs kapcsolók (pl. `CONFIG_PREEMPT`, `CONFIG_HZ`, `CONFIG_FTRACE`), fordítási beállítások.
4. **Mérési módszer**: mely eszközt használjuk, hány ismétlést végzünk, mekkora a bemelegítés, hogyan gyűjtjük az adatokat.
5. **Kiinduló állapot**: a rendszer terhelése, a háttérfolyamatok, a CPU-affinitás beállítása, a cache-ek állapota.
6. **Várt eredmény**: a mérés előtt levezetett vagy becsült érték, amelyet a mérés után összehasonlítunk a mért értékkel.
7. **Mért eredmény**: a ténylegesen kapott értékek, statisztikai összefoglalóval.
8. **Eltérés elemzése**: ha a mért érték jelentősen eltér a várttól, mi lehet az ok.

A mérési tervben a **várt** és a **mért** eredményt mindig külön kell feltüntetni. A várt érték származhat elméleti becslésből, korábbi mérésből vagy a dokumentációból. A mért érték a kísérlet tényleges kimenete. A kettő összehasonlítása adja a mérés validitását.

30.3.7 2.2. Melegítés, ismétlés és statisztika

A kernel méréseknél a hideg cache és a lazy inicializáció torzíthatja az első mintákat. Ezért a tényleges mérés előtt **bemelegítő** futtatásokat kell végezni, amelyek eredményét nem vesszük figyelembe. A bemelegítés hossza a mért művelettől függ: egy syscall-latency mérésnél néhány ezer hívás elegendő, egy lapcsere-mérésnél viszont több tízezer művelet is szükséges lehet.

Az ismétlések számát úgy kell megválasztani, hogy a statisztikai ingadozás kicsi legyen. A gyakorlatban legalább 5–10 független mérési sorozatot érdemes futtatni, és minden sorozatból kiszámítani az átlagot, a szórást és a percentiliseket. Az eredményt soha ne egyetlen futás adja, hanem a sorozatok összesítése.

A percentilisek számításához a mintákat rendezni kell, majd a kívánt százalékhoz tartozó elemet kiválasztani. A P99 például azt jelenti, hogy a minták 99%-a kisebb vagy egyenlő ennél az értéknél. A maximális értéket mindig külön kell kezelni, mert a kiugrók gyakran a leginformatívabbak.

30.3.8 3. Kernel tracing: ftrace

Az ftrace a Linux kernel beépített nyomkövető keretrendszere, amely a `tracefs` virtuális fájlrendszeren keresztül vezérelhető. Az ftrace nem igényel külső eszközt, és a kernel számos alrendszerébe beépített nyomkövetési pontokat kínál. A fejezetben a mérési és megfigyelési célú használatára koncentrálunk.

30.3.9 3.1. Az ftrace felépítése

Az ftrace a `/sys/kernel/tracing` (régebbi rendszereken `/sys/kernel/debug/tracing`) könyvtárban érhető el. A fontosabb fájlok:

- `available_tracers`: a betölthető nyomkövetők listája;
- `current_tracer`: az aktív nyomkövető neve;
- `trace`: a nyomkövetési kimenet;
- `trace_pipe`: folyamatosan olvasható kimenet;
- `set_ftrace_filter`: a nyomkövetendő függvények listája;
- `set_ftrace_pid`: a nyomkövetendő folyamat PID-je;
- `tracing_on`: a nyomkövetés ki-/bekapcsolása.

A leggyakrabban használt nyomkövetők:

- `function`: minden függvényhívás nevét rögzíti a kernelben;
- `function_graph`: a függvényhívások hierarchiáját és időtartamát rögzíti;
- `nop`: nyomkövetés kikapcsolva.

30.3.10 3.2. Példa: rendszerhívás latency nyomon követése function_graph segítségével

A következő példa egyetlen folyamat `getpid` rendszerhívásának nyomon követését mutatja be. Az emulált környezetben QEMU-t használunk, a kernelben engedélyezett az ftrace.

Először ellenőrizzük az ftrace elérhetőségét:

```
mount | grep tracefs
ls /sys/kernel/tracing
```

Ha a tracefs nincs csatolva, csatoljuk:

```
mount -t tracefs tracefs /sys/kernel/tracing
```

Válasszuk ki a `function_graph` nyomkövetőt, és szűrjük a nyomkövetést a `__x64_sys_getpid` függvényre:

```
echo function_graph > /sys/kernel/tracing/current_tracer
echo __x64_sys_getpid > /sys/kernel/tracing/set_graph_function
echo 1 > /sys/kernel/tracing/tracing_on
```

Futtassuk egy rövid programot, amely `getpid`-et hív:

```c
/* getpid_loop.c - szemléltetés, nem éles rendszeren futtatandó */
#include <unistd.h>
#include <sys/types.h>

int main(void)
{
    for (int i = 0; i < 100; i++)
        (void)getpid();
    return 0;
}
```

A nyomkövetés kikapcsolása után olvassuk ki a kimenetet:

```
echo 0 > /sys/kernel/tracing/tracing_on
cat /sys/kernel/tracing/trace
```

A kimenetben a `__x64_sys_getpid` függvény belépése és kilépése, valamint az eltelt idő látható. A `function_graph` az időt a `duration` oszlopban, mikroszekundum pontossággal jeleníti meg. A példa kimenete szemléltetés:

```
# tracer: function_graph
# CPU  DURATION                  FUNCTION CALLS
# |     |   |                     |   |   |   |
 0)   0.125 us    |  __x64_sys_getpid() {
 0)   0.020 us    |    current->nsproxy;
 0)   0.030 us    |    task_tgid_vnr();
 0)   0.025 us    |    __task_pid_nr_ns();
 0)   0.015 us    |    __rcu_read_lock();
 0)   0.010 us    |    __rcu_read_unlock();
 0)   0.125 us    |  }
```

Ez a kimenet már önmagában is megmutatja, hogy a `getpid` rendszerhívás kernelbeli kiszolgálása nagyon rövid, a belépés és a kilépés közötti idő tipikusan 100–200 ns egy emulált x86_64 környezetben. A pontos érték erősen függ a CPU-tól, a virtualizációtól és a kernel fordítási beállításaitól.

Az ftrace előnye, hogy szűkíthető egyetlen függvényre, így az overhead alacsony marad. Hátránya, hogy a függvénynevek és a hierarchia a kernel belső implementációjától függenek, és a kimenet értelmezése kernelismeretet igényel.

30.3.11 4. Perf: hardveres és szoftveres számlálók

A `perf` a Linux kernel teljesítménymérő eszköze, amely hardveres teljesítményszámlálókat (PMC), szoftveres eseményeket, tracepoint-okat és kprobe-okat tud gyűjteni. A perf fő parancsai:

- `perf stat`: egy parancs futása alatt gyűjti a számlálókat;
- `perf record`: mintákat gyűjt egy futó folyamatról;
- `perf report`: a `perf record` által készített adatfájlt elemzi;
- `perf top`: élő, folyamatos profilkép;
- `perf list`: elérhető események listája.

30.3.12 4.1. Példa: `perf stat` a `getpid` hurokra

A korábbi `getpid_loop` programot futtassuk a perf alatt:

```
perf stat ./getpid_loop
```

A kimenet szemléltetése:

```
 Performance counter stats for './getpid_loop':

              0.41 msec task-clock                #    0.602 CPUs utilized
                 1      context-switches          #    2.439 K/sec
                 0      cpu-migrations            #    0.000 /sec
                45      page-faults               #  109.756 K/sec
         1,234,567      cycles                    #    3.012 GHz
           987,654      instructions              #    0.80  insn per cycle
           123,456      branches                  #  301.123 M/sec
             1,234      branch-misses             #    1.00% of all branches

       0.000680123 seconds time elapsed
       0.000400000 seconds user
       0.000200000 seconds sys
```

A `task-clock` a folyamat által ténylegesen használt CPU-időt mutatja. A `cycles` a felhasznált órajelciklusok száma, az `instructions` a végrehajtott utasítások száma, az `insn per cycle` pedig a kettő hányadosa. A `context-switches` és a `cpu-migrations` a folyamat ütemezési viselkedését jelzi.

A `perf stat` kimenete önmagában nem ad latency-t, de a ciklusszám és az utasításszám alapján becsülhető a kód processzorterhelése. Ha például a `getpid_loop` 100 hívást hajt végre, és a `cycles` értéke 1 234 567, akkor egy hívásra átlagosan 12 345 ciklus jut. Ha a processzor 3,0 GHz-en fut, ez körülbelül 4,1 µs-ot jelent. Ez a becslés azonban magában foglalja a libc overhead-jét és a perf mérési overhead-jét is, ezért a tiszta kernelbeli latency elkülönítéséhez tracing vagy eBPF szükséges.

30.3.13 4.2. Példa: `perf record` és `perf report`

A `perf record` segítségével mintavételezhetjük a futó folyamatot, és a `perf report` interaktív vagy szöveges kimenetben megmutatja, hol tölti az időt a CPU.

```
perf record ./getpid_loop
perf report --stdio
```

A kimenetben a leggyakrabban mintavételezett függvények szerepelnek, például:

```
# Overhead  Command      Shared Object      Symbol
# ........  ...........  .................  ......................
    60.00%  getpid_loop  [kernel.kallsyms]  [k] do_syscall_64
    20.00%  getpid_loop  [kernel.kallsyms]  [k] __x64_sys_getpid
    10.00%  getpid_loop  libc.so.6          [.] getpid
    10.00%  getpid_loop  getpid_loop        [.] main
```

Ez a profilkép azt mutatja, hogy a CPU-idő nagy része a rendszerhívás belépésében és kiszolgálásában telik. A pontos százalékok természetesen a mérési körülményektől függenek.

30.3.14 4.3. Hardveres számlálók és cache-viselkedés

A perf egyik legerősebb képessége a hardveres teljesítményszámlálók elérése. Például a cache-találatok és -hibák mérése:

```
perf stat -e cache-references,cache-misses,instructions,cycles ./getpid_loop
```

A cache-miss arány fontos jelzés lehet a memóriarendszer terhelésére. A mérési jegyzőkönyvben mindig fel kell tüntetni, hogy a számlálók hardveres vagy szoftveres események-e, mert a virtualizált környezetben a hardveres számlálók pontossága korlátozott lehet. A QEMU például bizonyos PMC-ket emulál, másokat pedig nem tesz elérhetővé, ezért a perf kimenetét ebben a környezetben kellő óvatossággal kell kezelni.

30.3.15 5. eBPF: programozható megfigyelés

Az eBPF (extended Berkeley Packet Filter) egy olyan technológia, amely lehetővé teszi, hogy a felhasználó kis, ellenőrzött programokat töltsön be a kernelbe, amelyeket az adott események (tracepoint, kprobe, uprobe) bekövetkezésekor futtat a kernel. Az eBPF programok nem férhetnek hozzá a kernel teljes memóriájához, hanem a verifier ellenőrzi őket, így biztonságosan használhatók futó rendszeren is.

A fejezetben az eBPF-et megfigyelési célra használjuk, nem hálózati csomagfeldolgozásra. A leggyakoribb frontend eszközök:

- **bpftrace**: magas szintű, awk-szerű nyelv, amelyet rövid, egyszerű mérésekhez terveztek;
- **bcc**: Python kötések és C programok gyűjteménye összetettebb elemzésekhez;
- **libbpf**: alacsony szintű C könyvtár a BPF programok betöltéséhez.

30.3.16 5.1. Példa: syscall latency mérése bpftrace-szel

A következő bpftrace szkript a `getpid` rendszerhívás belépését és kilépését figyeli, és hisztogramot készít a latency-ről nanomásodpercben:

```
bpftrace -e '
tracepoint:syscalls:sys_enter_getpid {
    @start[tid] = nsecs;
}

tracepoint:syscalls:sys_exit_getpid /@start[tid]/ {
    @latency_ns = hist(nsecs - @start[tid]);
    delete(@start[tid]);
}

END {
    clear(@start);
    print(@latency_ns);
}'
```

A szkript a `sys_enter_getpid` tracepoint-on rögzíti az aktuális időt, majd a `sys_exit_getpid` eseménynél kiszámítja a különbséget, és a `hist` beépített függvénnyel hisztogramot készít. A kimenet például:

```
@latency_ns:
[128, 256)    12345 |@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@|
[256, 512)     6789 |@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@                            |
[512, 1K)       890 |@@@@                                                        |
[1K, 2K)        123 |                                                            |
[2K, 4K)         45 |                                                            |
```

A hisztogramból kiolvasható a latency eloszlása, a módusz, a percentilisek és a kiugrók. Az eBPF előnye, hogy a mérés közvetlenül a kernelben fut, ezért a felhasználói térbe történő átmenet overhead-je kicsi. A `bpftrace` azonban a szkript futása alatt maga is overhead-et generál, ezért a mért értékeket mindig a mérési körülményekkel együtt kell dokumentálni.

30.3.17 5.2. eBPF és a biztonság

Az eBPF programokat a kernel beépített verifier-e ellenőrzi, mielőtt betöltené őket. A verifier biztosítja, hogy a program nem tartalmaz végtelen ciklust, nem fér hozzá érvénytelen memóriához, és nem módosítja a kernel állapotát nem megengedett módon. Ez az ellenőrzés azonban nem jelenti azt, hogy az eBPF használata kockázatmentes. A mérési környezetben ezért csak megbízható forrásból származó szkripteket szabad futtatni, és a laborban az eBPF-et izolált, eldobható virtuális gépen kell használni.

30.3.18 6. Eszközválasztás: ftrace, perf vagy eBPF?

A három eszköz nem versenytárs, hanem kiegészíti egymást. A választás a mérési kérdéstől függ.

Kérdés	Javasolt eszköz
Melyik kernel függvény hívódik meg, és mennyi ideig tart?	ftrace `function_graph`
Mennyi a folyamat CPU-ideje, hány utasítást hajt végre?	`perf stat`
Hol tölti a folyamat a CPU-időt (profil)?	`perf record` + `perf report`
Mi a rendszerhívás latency eloszlása?	eBPF (bpftrace)
Mi a cache-miss arány egy adott kódrészletnél?	`perf stat` hardveres számlálókkal
Mi történik egy adott kernel ponton, pl. ütemező belépésnél?	tracepoint + eBPF vagy ftrace
Mi a kontextusváltások száma és időbeli eloszlása?	`perf sched` vagy eBPF

A hibrid megközelítés gyakran a legjobb: a `perf` átfogó képet ad, az ftrace a részletes függvényhívási láncot mutatja, az eBPF pedig rugalmasan testre szabott méréseket tesz lehetővé.

30.3.19 7. Tipikus hibák és félreértések

A kernel teljesítménymérésben a következő hibák fordulnak elő leggyakrabban:

30.3.20 7.1. Az átlagos latency mint egyetlen mérőszám

Az átlag elrejti a kiugrókat, és nem mutatja a valós felhasználói élményt. A latency elemzésnél mindig meg kell adni a percentiliseket és a maximumot. Ha a rendszerben időnként nagy késleltetések vannak, az átlag akkor is alacsony lehet, ha a felhasználók rendszeresen érzékelik a lassulást.

30.3.21 7.2. A CPU ciklus és az idő összekeverése

A ciklusszám csak akkor konvertálható időre, ha ismerjük az órajelet, amely a modern processzorokon dinamikusan változik. A mérési jegyzőkönyvben ezért rögzíteni kell az órajel-viselkedést, vagy a ciklusszámot önmagában, időre való átváltás nélkül kell jelenteni.

30.3.22 7.3. A mérési overhead figyelmen kívül hagyása

Minden mérőeszköz maga is fogyaszt erőforrást. Az ftrace függvényszintű nyomkövetése például jelentősen meglassíthatja a rendszert, ha túl sok függvényt figyel. A perf stat és az eBPF szkriptek is overhead-et okoznak. Ezért a mérési tervben mindig szerepelnie kell annak, hogy mekkora a mérési overhead, és hogyan lehet csökkenteni (pl. szűkített trace, kevésbé gyakori mintavétel).

30.3.23 7.4. Nem reprodukálható környezet

Ha a mérési környezet nincs dokumentálva (CPU, kernelverzió, konfiguráció, terhelés), az eredmény nem reprodukálható, és nem lehet rá alapozni. A könyv konvenciója szerint minden mérésnél rögzíteni kell a hardvert, a fordítási beállításokat, az órajelet és a mérési módszert.

30.3.24 7.5. A várt és a mért eredmény összekeverése

A várt érték a mérés előtti elméleti becslés, a mért érték a kísérlet tényleges kimenete. Ha a kettőt nem választjuk szét, a mérés elveszíti ellenőrző szerepét. A laborban ezért külön táblázatban kell feltüntetni a várt és a mért értéket, és az eltérést elemezni kell.

30.3.25 7.6. A hideg cache és a lazy inicializáció figyelmen kívül hagyása

Az első mérések gyakran lassabbak, mert a cache-ek üresek, és a kernel egyes adatstruktúrái még nincsenek inicializálva. A mérési tervben bemelegítő futtatásokat kell előírni, és az első mintákat ki kell zárni az elemzésből.

30.3.26 7.7. A virtualizáció hatásainak alábecsülése

QEMU-ban vagy virtuális gépben a vendég kernel által látott idő és a tényleges fizikai idő eltérhet. A hardveres számlálók pontossága korlátozott, az órajel pedig nem feltétlenül stabil. A mérési jegyzőkönyvben ezért a virtualizációs réteget és annak beállításait is fel kell tüntetni.

30.3.27 8. Labor: benchmark- és tracing-terv készítése

A fejezet laborjának célja, hogy a hallgató elkészítsen egy reprodukálható mérési tervet, és azt dokumentálja egy jegyzetfüzetben vagy jegyzőkönyvben. A labor nem feltétlenül igényli a tényleges mérést, de a tervnek tartalmaznia kell minden olyan elemet, amely alapján a mérés később végrehajtható.

30.3.28 8.1. Kiinduló állapot

- Emulált környezet: QEMU x86_64, 4 vCPU, 2 GiB RAM, `-enable-kvm` ha elérhető, ellenkező esetben TCG emuláció.
- Kernel: egy friss stabil Linux kernel (a pontos verzió a dokumentációban ellenőrizendő), a következő konfigurációs kapcsolókkal: `CONFIG_FTRACE=y`, `CONFIG_FUNCTION_TRACER=y`, `CONFIG_FUNCTION_GRAPH_TRACER=y`, `CONFIG_PERF_EVENTS=y`, `CONFIG_BPF=y`, `CONFIG_BPF_SYSCALL=y`, `CONFIG_KPROBES=y`.
- A hoszt valódi rendszerét tilos módosítani; minden művelet a vendég rendszerre korlátozódik.

- A vendég rendszerben egy root jogosultságú shell áll rendelkezésre, de a méréshez nem szükséges valós hitelesítő adat vagy éles adat.

30.3.29 8.2. A mérési cél

A cél a `getpid` rendszerhívás latency-ének mérése nanoszekundumban, valamint a rendszerhívás kernelbeli kiszolgálási idejének megfigyelése ftrace és eBPF segítségével. A mérés másodlagos célja a throughput mérése: hány `getpid` hívás hajtható végre másodpercenként egyetlen szálon.

30.3.30 8.3. Várt eredmény levezetése

A `getpid` egy triviális rendszerhívás, amely nem végez I/O-t, nem blokkol, és a kernelben csak néhány mezőt olvas ki a folyamatleíróból. A várt latency a következő összetevőkből áll:

- user-kernel átmenet (syscall belépés): néhány tíz ns;
- kernelbeli kiszolgálás: néhány tíz ns;
- kernel-user átmenet (visszatérés): néhány tíz ns.

Egy emulált x86_64 környezetben a teljes latency várhatóan 100–500 ns között van. A pontos érték függ az órajeltől, a virtualizációtól és a CPU-tól. A throughput egyetlen szálon várhatóan 1–5 millió hívás/s, mivel a rendszerhívás nagyon rövid, de a libc hívási overhead és a perf mérési overhead csökkenti a tényleges értéket.

A várt értéket a laborjegyzőkönyvben így kell rögzíteni:

```
Várt latency: 100-500 ns (eltolódás a mérési overhead miatt lehetséges)
Várt throughput: 1-5 M ops/s (egyszálú, QEMU-ban)
```

30.3.31 8.4. Mérési terv

1. **Bemelegítés**: futtassunk 10 000 `getpid` hívást, az eredményt ne rögzítsük.
2. **Latency mérés eBPF-fel**: használjuk a fejezetben bemutatott bpftrace szkriptet, és gyűjtsünk legalább 100 000 mintát.
3. **Throughput mérés**: használjunk egy C programot, amely egy másodpercig folyamatosan hívja a `getpid`-et, és számolja a hívások számát. A mérést 10-szer ismételjük, és átlagot számítunk.
4. **Ftrace nyomkövetés**: kapcsoljuk be a `function_graph` nyomkövetőt a `__x64_sys_getpid` függvényre, futtassunk 100 hívást, majd rögzítsük a függvényhívási láncot és az időtartamokat.
5. **Perf stat**: futtassuk a `getpid_loop`-ot a `perf stat` alatt, és rögzítsük a ciklusszámot, az utasításszámot, a cache-miss-eket és a kontextusváltásokat.
6. **Rögzítés**: minden mérésből készítsünk naplót, amely tartalmazza a pontos parancsokat, a kimeneteket és az időbélyeget.

30.3.32 8.5. A mérési napló szerkezete

A laborjegyzőkönyvben a következő szakaszoknak kell szerepelniük:

- **Környezet**: QEMU verzió, kernelverzió, konfigurációs kapcsolók, CPU típus, memória.
- **Módszer**: a használt parancsok és szkriptek.
- **Várt eredmény**: a fenti levezetés.
- **Mért eredmény**: a mért latency percentilisek, a throughput, a ciklusszám és az ftrace kimenet összefoglalása.
- **Eltérés elemzése**: ha a mért érték kívül esik a várt tartományon, mi lehet az ok?

A mért és a várt eredményt külön táblázatban kell feltüntetni. Például:

Metrika	Várt	Mért
Átlagos latency (ns)	100–500	342
P99 latency (ns)	200–1000	612
Throughput (ops/s)	1–5 M	2,3 M
CPU cycles per syscall	800–3000	1540

A táblázatban a mért értékek csak illusztrációk; a labor során a tényleges értékeket kell beírni.

30.3.33 8.6. Ellenőrző kérdések a laborhoz

1. Miért nem elegendő az átlagos latency a mérési eredmény bemutatásához?
2. Hogyan befolyásolja a QEMU TCG emuláció a mért ciklusszámot az `perf stat` kimenetében?
3. Miért kell a bpftrace szkript futása alatt is mérni a háttérterhelést?
4. Milyen okok miatt térhet el a mért latency a várt tartománytól?

30.3.34 9. Összefoglalás

A kernel viselkedésének mérése és megfigyelése három pilléren nyugszik: a **latency**, a **throughput** és a **CPU cycle** pontos definícióján; a **reprodukálható mérési terven**; valamint a megfelelő **megfigyelési eszköz** kiválasztásán. Az ftrace a függvényhívási láncok és időtartamok részletes feltárására alkalmas, a perf a hardveres és szoftveres számlálók gyűjtésére, az eBPF pedig rugalmas, programozható méréseket tesz lehetővé.

A leggyakoribb hibák – az átlag fetisizálása, a ciklus és az idő összekeverése, a mérési overhead elhanyagolása, a nem dokumentált környezet – mind elkerülhetők, ha a mérési tervben a várt és a mért eredményt mindig elkülönítjük, és a mérési körülményeket teljes körűen rögzítjük. A fejezet laborja ezt a fegyelmet gyakoroltatja: a hallgató nem egyetlen számot ad át, hanem egy olyan tervet, amely alapján a mérés bárhol, bármikor megismételhető.

A következő egység (Regresszió és kiadási minőség) a regresszió és a kiadási minőség kérdéseire épít majd az itt bevezetett mérési és megfigyelési fogalmakra. Aki a fejezetben bemutatott eszközökkel és módszertannal dolgozik, képes lesz a kernel viselkedését nemcsak megmérni, hanem a mérési eredményeket a tervezési döntésekbe is visszavezetni.

Önellenőrző kérdések

1. Definiálja a latency és a throughput fogalmát, és mutasson példát arra, amikor az egyik javítása rontja a másikat.
2. Mi a különbség a CPU cycle és a végrehajtott utasítások száma között? Miért fontos ez a mérésben?
3. Melyik eszközt választaná egy adott rendszerhívás kernelbeli kiszolgálási idejének függvényszintű nyomon követésére? Indokolja a választást.
4. Soroljon fel három tipikus hibát a kernel teljesítménymérésben, és magyarázza el, hogyan lehet őket elkerülni.
5. Mit jelent a P99 latency, és miért informatívabb az átlagnál egy interaktív rendszer esetén?

30.4 Regresszió és kiadási minőség

30.4.1 Regressziós kapu és kiadási minőség

A szoftver nem állandó. Egy operációs rendszer kernelének fejlődése folyamatos változások sorozata: hibajavítások, új eszközmeghajtók, biztonsági foltok, teljesítményoptimalizálások és refaktorálások. Minden változás hordoz kockázatot: amit ma megjavítunk, holnap egy másik konfigurációban regresszióként térhet vissza. A regresszió fogalma azt jelenti, hogy egy korábban működő funkció, tulajdonság vagy teljesítményjellemző egy változás hatására leromlik vagy teljesen elromlik.

A kiadási minőség nem egyetlen pillanat, hanem folyamatosan fenntartott állapot. A regressziós kapu (regression gate) olyan automatikus vagy félautomata ellenőrzési pont, amelyen minden változásnak át kell haladnia, mielőtt a főágba kerülhet, vagy mielőtt kiadásra jelölhető. Célja, hogy a hibát még a fejlesztői környezetben elkapja, ne az éles rendszerben.

Ebben a fejezetben a változások biztonságos bevezetésének módszertanát dolgozzuk ki: a test matrix felépítését, a folyamatos integráció (CI) és a soak test szerepét, a rollback (visszagörgetés) mechanizmusait, valamint a hibajegy-kezelés fegyelmét. A cél nem csupán a fogalmak bemutatása, hanem egy működő mintarendszer megalkotása, amely egy egyszerű kernelmodulon vagy eszközmeghajtón keresztül mutatja be a regressziós kaput a gyakorlatban.

30.4.2 A változások biztonságos bevezetésének alapelvei

A kernel vagy bármely alacsony szintű rendszerkomponens esetében a változáskezelés nem merülhet ki abban, hogy „lefordul és elindul”. Egy biztonságos bevezetési folyamat a következő fázisokból áll:

1. **Változáskérés és indoklás**: Minden változásnak legyen azonosítója és leírása. Miért szükséges? Milyen viselkedést érint? Milyen konfigurációkban releváns?
2. **Kód és teszt együtt**: A változás csak akkor fogadható el, ha a hozzá tartozó tesztek is megérkeznek. A kernelkódban ez azt jelenti, hogy a meghajtóhoz tartozó unit teszt, a rendszerhíváshoz tartozó szintetikus hívás, vagy a hibajavításhoz tartozó reprodukciós teszt a patch része.
3. **Automatikus ellenőrzés**: A build, a statikus elemzés, a unit tesztek és a célzott integrációs tesztek minden változásnál lefutnak. Ez a CI folyamat szíve.
4. **Emberi felülvizsgálat**: Az automatikus ellenőrzés szükséges, de nem elégséges. A kernelkódban az erőforrás-tulajdonjog, a versenyhelyzetek és a hibakezelés emberi szemmel történő ellenőrzése is kötelező.

5. **Fokozatos bevezetés**: A változás először egy izolált ágon, majd egy belső tesztkörnyezetben, végül egy korlátozott éles környezetben fut. Minden fázisban mérjük a regressziós mutatókat.
6. **Rollback terv**: Mielőtt a változás élesbe kerül, előre meghatározzuk, hogyan lehet visszavonni. A kernel esetében ez lehet egy korábbi kernelimage, egy kapcsolóval letiltható funkció, vagy egy revert commit.

A regressziós kapu az a mechanizmus, amely ezeket a fázisokat összeköti. A kapu akkor nyílik meg, ha minden feltétel teljesül: a tesztek zöldek, a teljesítménymutatók a toleranciasávon belül vannak, a checklist ki van töltve, és a hibajegyekben nincs nyitott, magas súlyosságú regresszió.

30.4.3 Egy ellenőrizhető példa: a mailbox queue modul

A továbbiakban egy egyszerű, de valósághű kernelkomponensen dolgozunk: egy rögzített méretű, körkörös üzenetsoron (mailbox queue), amelyet két folyamat vagy megszakításkezelő és felhasználói szál közötti kommunikációra használunk. A példa azért jó, mert:

- Kicsi, így a tesztek gyorsan lefutnak.
- Van állapota, tehát a regresszió könnyen megfigyelhető.
- Van párhuzamossági vetülete, így a versenyhelyzetek és a zárolási hibák is vizsgálhatók.
- A hibák tipikusak: határeset a puffer tele/üres állapotánál, memóriatulajdonjog, megszakításbiztosság.

A modul kezdeti interfésze a következőképpen néz ki:

```
/* mailbox.h - egyszerű körkörös üzenetsor kernelkomponens */
#ifndef MAILBOX_H
#define MAILBOX_H

#include <stddef.h>
#include <stdbool.h>

#define MSG_SLOTS 8
#define MSG_DATA_LEN 16

struct msg {
    unsigned int sender;          /* küldő azonosítója */
    unsigned int seq;             /* sorszám a sorrend ellenőrzéséhez */
    char data[MSG_DATA_LEN];      /* hasznos adat, NUL-terminált */
};

struct mailbox {
    struct msg ring[MSG_SLOTS];   /* a körkörös puffer */
    size_t head;                  /* a következő írási pozíció */
    size_t tail;                  /* a következő olvasási pozíció */
    size_t count;                 /* a pufferben lévő üzenetek száma */
    bool initialized;             /* inicializálásjelző */
};

/* Inicializálja a leveleződobozt. Visszatérés: 0 siker, -1 ha már inicializált. */
int mailbox_init(struct mailbox *mb);

/* Üzenet hozzáadása. Visszatérés: 0 siker, -1 ha tele a puffer, -2 ha érvénytelen paraméter. */
int mailbox_send(struct mailbox *mb, const struct msg *msg);

/* Üzenet kivétele. Visszatérés: 0 siker, -1 ha üres a puffer, -2 ha érvénytelen paraméter. */
int mailbox_recv(struct mailbox *mb, struct msg *out);

#endif /* MAILBOX_H */
```

A példa szándékosan egyszerű, de már itt láthatók azok a pontok, ahol egy változás regressziót okozhat. A kezdeti implementáció:

```
/* mailbox.c - egyszerű körkörös üzenetsor implementáció */
#include "mailbox.h"

int mailbox_init(struct mailbox *mb)
{
    if (mb == NULL)
        return -2;
    if (mb->initialized)
        return -1;

    mb->head = 0;
    mb->tail = 0;
    mb->count = 0;
    mb->initialized = true;
    return 0;
}

int mailbox_send(struct mailbox *mb, const struct msg *msg)
{
    if (mb == NULL || msg == NULL)
        return -2;
    if (!mb->initialized)
        return -2;
    if (mb->count >= MSG_SLOTS)
        return -1;

    mb->ring[mb->head] = *msg;
    mb->head = (mb->head + 1) % MSG_SLOTS;
    mb->count++;
    return 0;
}

int mailbox_recv(struct mailbox *mb, struct msg *out)
{
    if (mb == NULL || out == NULL)
        return -2;
    if (!mb->initialized)
        return -2;
    if (mb->count == 0)
        return -1;

    *out = mb->ring[mb->tail];
    mb->tail = (mb->tail + 1) % MSG_SLOTS;
    mb->count--;
    return 0;
}
```

Ez a verzió helyesnek tűnik egy egyszálú környezetben. A tesztelés során azonban kiderül, hogy több szál esetén a `count`, a `head` és a `tail` módosítása nem atomi, így a puffer elveszíthet üzeneteket vagy duplikálhatja azokat. Tegyük fel, hogy a fejlesztő hozzáad egy spinlockot a struktúrához, és a következő verziót készíti el:

```
/* mailbox_v2.c - spinlockkal védett változat */
#include "mailbox.h"
#include "spinlock.h" /* példa: saját kernel-spinlock */

struct mailbox {
    struct msg ring[MSG_SLOTS];
    size_t head;
    size_t tail;
```

```
    size_t count;
    spinlock_t lock;      /* új mező */
    bool initialized;
};

int mailbox_send(struct mailbox *mb, const struct msg *msg)
{
    if (mb == NULL || msg == NULL)
        return -2;
    if (!mb->initialized)
        return -2;

    spin_lock(&mb->lock);
    if (mb->count >= MSG_SLOTS) {
        spin_unlock(&mb->lock);
        return -1;
    }
    mb->ring[mb->head] = *msg;
    mb->head = (mb->head + 1) % MSG_SLOTS;
    mb->count++;
    spin_unlock(&mb->lock);
    return 0;
}
```

Itt már látható a regresszió első gyanúja: a `mailbox_init` nem inicializálja a `lock` mezőt, mert a fejlesztő elfelejtette frissíteni az inicializáló függvényt. Ha a lock nem inicializált, a viselkedés nem definiált; bizonyos platformokon a spinlock nullázott értéke zárt állapotot jelent, másutt nem. Ez a tipikus „majdnem jó” változás, amelyet a regressziós kapunak el kell kapnia.

A unit teszt célja, hogy ezt a hibát még a commit előtt felfedje. A tesztváz:

```
/* test_mailbox.c - unit teszt a mailbox modulhoz */
#include "mailbox.h"
#include <assert.h>
#include <string.h>

static void test_send_recv_single(void)
{
    struct mailbox mb;
    struct msg m1, m2;

    assert(mailbox_init(&mb) == 0);

    memset(&m1, 0, sizeof(m1));
    m1.sender = 1;
    m1.seq = 100;
    strncpy(m1.data, "hello", MSG_DATA_LEN);

    assert(mailbox_send(&mb, &m1) == 0);
    assert(mb.count == 1);

    assert(mailbox_recv(&mb, &m2) == 0);
    assert(m2.sender == 1);
    assert(m2.seq == 100);
    assert(strcmp(m2.data, "hello") == 0);
    assert(mb.count == 0);
}

static void test_full_empty(void)
{
    struct mailbox mb;
    struct msg m;
    size_t i;
```

```
    assert(mailbox_init(&mb) == 0);

    /* Töltsük meg a puffert. */
    for (i = 0; i < MSG_SLOTS; i++) {
        m.sender = (unsigned int)i;
        m.seq = (unsigned int)i;
        assert(mailbox_send(&mb, &m) == 0);
    }
    /* A puffer most tele van. */
    assert(mailbox_send(&mb, &m) == -1);

    /* Ürítsük ki. */
    for (i = 0; i < MSG_SLOTS; i++) {
        assert(mailbox_recv(&mb, &m) == 0);
        assert(m.seq == i);
    }
    /* A puffer most üres. */
    assert(mailbox_recv(&mb, &m) == -1);
}

void run_mailbox_tests(void)
{
    test_send_recv_single();
    test_full_empty();
}
```

Ez a teszt azonban csak a funkcionális viselkedést ellenőrzi. Nem észleli a lock inicializálásának hiányát, mert egyszálú környezetben a nem inicializált spinlock gyakran „működik”. A párhuzamos tesztre később, a soak test kapcsán térünk vissza.

30.4.4 Test matrix

A test matrix (tesztmátrix) a támogatott környezetek és konfigurációk kombinációinak rendszerezett leírása. Célja, hogy a változásokat ne csak egyetlen gépen, egyetlen beállítással ellenőrizzük, hanem a lehetséges variációk reprezentatív halmazán.

Egy kernel esetében a test matrix dimenziói lehetnek:

- **Architektúra**: x86_64, ARMv8, RISC-V, stb. (QEMU-ban vagy kereszthivatkozásos builddel)
- **Konfiguráció**: SMP be/ki, preemptív be/ki, debug be/ki, KASAN (Kernel Address Sanitizer) be/ki
- **Eszközháttér**: virtio-blk, NVMe emuláció, UART, hálózati eszköz
- **Fájlrendszer**: initramfs, ext4, custom FS
- **Opcionális funkciók**: a fejlesztés alatt álló modul be- és kikapcsolva

A mátrixot úgy kell megtervezni, hogy a kombinációk száma ne robbanjon fel. A gyakorlatban a teljes Descartes-szorzat helyett okosan választott, reprezentatív kombinációkat használunk. Például:

Kombináció	Arch	SMP	Debug	Sanitizer	Fájlrendszer	Cél
A1	x86_64 (QEMU)	1	igen	KASAN	initramfs	alap funkcionalitás
A2	x86_64 (QEMU)	4	nem	nincs	ext4	SMP stressz

Kombináció	Arch	SMP	Debug	Sanitizer	Fájlrendszer	Cél
A3	ARMv8 (QEMU)	4	igen	KASAN	initramfs	arch-specifikus hibák
A4	RISC-V (QEMU)	1	igen	nincs	initramfs	hordozhatóság
B1	x86_64 (QEMU)	2	nem	KASAN	custom FS	fájlrendszer-integráció

A táblázatban szereplő `x86_64 (QEMU)` és társai **emulált környezeteket** jelölnek. A valós hardveren történő tesztelés külön, dedikált laborban történik, soha nem a fejlesztői gépen. A QEMU konfigurációját a reprodukciós csomag tartalmazza: CPU modell, memória méret, csatolt eszközök, kernel parancssor.

A test matrix minden cellájához tartozik egy elvárt eredmény. Például az A1 kombinációban a mailbox unit tesztnek 0 hibával kell lefutnia, a rendszerindulásnak 2 másodpercen belül be kell fejeződnie, és a kernel logban nem szerepelhet `panic` vagy `Oops`. Ha bármelyik feltétel sérül, az adott cella pirosra vált, és a regressziós kapu zárva marad.

A mátrixot magát is verziózni kell. Ha egy új eszközmeghajtó kerül a kernelbe, a mátrix új sorral bővül. Ha egy régi architektúra támogatása megszűnik, a hozzá tartozó sor törlődik. A mátrix változása is review-köteles, hiszen egy rosszul definiált mátrix hamis biztonságérzetet ad.

30.4.5 Folyamatos integráció és a regressziós kapu

A folyamatos integráció (CI) az a gépesített folyamat, amely minden változásnál lefuttatja a test matrixot, és a regressziós kaput működteti. Egy kernelprojektben a CI rendszerint a következő állomásokból áll:

1. **Build stage**: A kernel lefordítása minden támogatott architektúrára, a hozzá tartozó toolchainnel. Itt már elbukik a fordítási hiba, a hiányzó header, a rossz konfigurációs függőség.
2. **Unit test stage**: A kernelmodulok unit tesztjei. Ezek lehetnek hoston futó tesztek (a modul C kódját a hoszt gépen, mockolt kernel API-val fordítjuk), vagy QEMU-ban futó tesztek, ahol a modul a kernelben töltődik be.
3. **Integration test stage**: A teljes kernel bootolása az egyes matrix cellákban, majd a felhasználói térben futó tesztprogramok végrehajtása. Itt jelenik meg a mailbox párhuzamos tesztje is.
4. **Soak test stage**: Hosszabb ideig tartó, terheléses teszt, amely a memóriaszivárgást, a fáradási hibákat és a ritka versenyhelyzeteket keresi. A soak test nem minden commitnál fut, hanem éjszakai vagy heti rendszerességgel; a kiadás előtt azonban kötelező.
5. **Packaging and release stage**: A bevált változat leképezése, a binárisok előállítása, és a kiadási checklist automatizálható részének ellenőrzése.

Példa egy CI konfigurációra (YAML stílusban, szemléltetés):

```
# .ci/pipeline.yaml - regressziós kapu folyamat
stages:
  - build
  - unit
  - integration
```

```
  - soak
  - package

variables:
  QEMU_CPU: "cortex-a72"       # példa ARM CPU modell QEMU-hoz
  QEMU_MEM: "256M"
  TEST_TIMEOUT: "120s"

build:x86_64:
  stage: build
  script:
    - make ARCH=x86_64 defconfig
    - make ARCH=x86_64 -j$(nproc)
  artifacts:
    paths:
      - vmlinux

build:arm64:
  stage: build
  script:
    - make ARCH=arm64 defconfig
    - make ARCH=arm64 -j$(nproc) CROSS_COMPILE=aarch64-linux-gnu-
  artifacts:
    paths:
      - vmlinux.arm64

unit:mailbox:
  stage: unit
  script:
    - make -C tests/mailbox run
    - grep -q "ALL TESTS PASSED" tests/mailbox/test.log

integration:kasan:
  stage: integration
  script:
    - qemu-system-x86_64 -kernel vmlinux -append "console=ttyS0" \
        -nographic -enable-kvm -smp 4 -m 256M
    - expect tests/integration/run_suite.sh
  timeout: $TEST_TIMEOUT

soak:nightly:
  stage: soak
  script:
    - qemu-system-x86_64 -kernel vmlinux -append "console=ttyS0" \
        -nographic -smp 4 -m 256M
    - timeout 3600 tests/soak/run_soak.sh
  only:
    - schedules
  allow_failure: false
```

A fenti konfigurációban a `soak:nightly` job csak ütemezetten fut, és a `allow_failure: false` jelzi, hogy ha a soak teszt elbukik, a kapu zárva marad a kiadás előtt. A `timeout` paraméterek azt biztosítják, hogy egy beragadt teszt ne bénítsa meg a teljes folyamatot.

A regressziós kapu döntési logikája a következő:

- Ha a build stage bármely cellája piros, a változás elutasításra kerül.
- Ha a unit test stage bármely tesztje hibát jelez, a változás elutasításra kerül.
- Ha az integration test stage bármely matrix cellája piros, a változás elutasításra kerül.
- Ha a soak test (amikor fut) hibát vagy memóriaszivárgást jelez, a kiadási folyamat blokkolódik.

- Ha minden zöld, de a checklist emberi része (review, dokumentáció, biztonsági átvilágítás) hiányos, a változás nem kerülhet a főágba.

A kapu nem csak bináris döntést hoz. A „zöld, de gyanús” állapot is létezik: ha például egy teszt a timeout határán belül, de a korábbi futáshoz képest 30%-kal lassabban fut le, a rendszer figyelmeztetést ad. A teljesítményregresszió ugyanolyan fontos, mint a funkcionális hiba.

30.4.6 Soak test és a ritka hibák felderítése

A soak test (áztatási teszt) hosszú időn át tartó, folyamatos terhelést jelent. Célja az olyan hibák felfedése, amelyek rövid tesztfutásokban nem jelentkeznek:

- Memóriaszivárgás: a mailbox modulnál például egy rosszul kezelt hibakód miatt az üzenet puffere nem szabadul fel.
- Ritka versenyhelyzet: a spinlock hibás használata miatt 10 000 műveletből egyszer duplikálódik egy üzenet.
- Erőforrás-kimerülés: a fájlleírók, időzítők vagy hálózati pufferek lassú elfogyása.
- Időzítési eltolódás: a rendszer órájának lassú driftje, amely hosszú futás után hibás időbélyegekhez vezet.

A soak test forgatókönyve a mailbox példánál:

1. Indítsunk el N darab küldő szálat és M darab fogadó szálat.
2. Minden küldő szál egyedi sorszámú üzeneteket küld, és naplózza a küldött sorszámokat.
3. Minden fogadó szál fogadja az üzeneteket, ellenőrzi a sorszámok folytonosságát, és naplózza az eltéréseket.
4. A teszt 8 órán át fut, közben a rendszer óránként pillanatképet készít a memóriahasználatról.
5. A teszt végén a vezérlő összehasonlítja a küldött és fogadott sorszámok halmazát. Ha bármely sorszám hiányzik vagy duplikálva van, a teszt hibát jelez.

A soak test nem egyszerűen „hosszú unit teszt”. A terhelésnek változatosnak kell lennie: időnként a rendszer más komponensei is kapjanak terhelést (például egy párhuzamosan futó fájlrendszer-művelet), hogy a gyorsítótár- és ütemezőinterakciók is felszínre kerüljenek. A teszt során a megszakítások, a kontextusváltások és a laphibák száma is naplózandó.

A soak test eredményét a mérési napló tartalmazza. Példa a napló kivonatára:

```
[00:00] soak: mailbox stress started, senders=4, receivers=4
[01:00] soak: sent=118432, recv=118432, lost=0, dup=0, mem_free=198224 KiB
[02:00] soak: sent=236915, recv=236915, lost=0, dup=0, mem_free=197856 KiB
...
[07:59] soak: sent=948217, recv=948216, lost=1, dup=0, mem_free=194112 KiB
[08:00] soak: test FAILED, lost=1, first_lost_seq=572314
```

Itt látható a regresszió: a 8. órában egy üzenet elveszett. A napló megadja az első elveszett sorszámot, ami a hibakeresés kiindulópontja. A memóriacsökkenés (198 224 KiB-ról 194 112 KiB-ra) arra utal, hogy a kernel memóriája lassan fogy, ami önmagában is vizsgálatot igényel.

A soak test hosszúsága és gyakorisága a rendszer kritikusságától függ. Egy beágyazott, biztonsági kritikus rendszerben a soak test akár 72 órán át is futhat, és minden kiadás előtt kötelező. Egy fejlesztői ágon a napi 1 órás soak test is elegendő lehet a durva hibák kiszűrésére.

30.4.7 Rollback és visszagörgetési stratégiák

A legjobb tesztelés sem képes minden hibát elkapni. Ezért minden változáshoz tartozik egy előre meghatározott rollback stratégia. A rollback lényege, hogy a rendszer visszaállítható egy ismert jó állapotba anélkül, hogy a felhasználói adatok sérülnének vagy a szolgáltatás hosszabb időre kiesne.

Kernelkörnyezetben a rollbacknek több szintje van:

- **Commit szintű revert**: A változás visszavonása a verziókezelőben. Ez a legegyszerűbb, de csak akkor alkalmazható, ha a változás még nem terjedt el. Ha a hibás commit után már több másik commit is született, a tiszta revert nehézkes lehet.
- **Fordítási kapcsoló**: A funkció be- és kikapcsolható egy konfigurációs opcióval vagy kernelparanccsal. Ha a mailbox modul hibás, a `mailbox.enable=0` paraméterrel letiltható, és a rendszer a régi, bevált kommunikációs mechanizmust használja.
- **Dupla kép (A/B):** A rendszer két kernelimage-t tárol. Az aktív image mellett egy tartalék image is van. Ha az új kernel nem bootol vagy instabil, a bootloader egy számláló alapján automatikusan a tartalék image-re vált. Ez a módszer elterjedt beágyazott és mobil rendszerekben.
- **Csomag-visszaállítás**: A disztribúciós csomagkezelő a kernelcsomag korábbi verzióját telepíti vissza. Ez a felhasználói térből is elvégezhető, de kernelpánik esetén a bootloader szintű visszaállítás is szükséges lehet.

A rollback terv része a **visszaállítási út** dokumentálása. Minden változáshoz fel kell jegyezni:

- Melyik commit vagy csomag az utolsó ismert jó állapot?
- Hogyan lehet a hibás állapotot észlelni? (Pl. `panic` a logban, magas latencia, hibás checksum.)
- Milyen parancsokkal vagy lépésekkel lehet visszaállni?
- Milyen adatok sérülhetnek a visszaállítás során, és hogyan lehet azokat helyreállítani?
- Ki a felelős a visszaállítás végrehajtásáért?

A rollback nem a kudarc beismerése, hanem a mérnöki fegyelem része. A biztonságos bevezetés azt jelenti, hogy a rollback lehetősége már a változás tervezésekor létezik, nem akkor, amikor a hiba már élesben tombol.

Példa egy bootloader konfigurációra A/B visszaállítással:

```
# boot.conf - A/B rendszerindítási konfiguráció
# A jelenlegi (aktív) kernel
active = kernel_b.img
# A tartalék kernel
backup = kernel_a.img
# Hányszor próbálja meg az aktív kernelt elindítani, mielőtt váltana
max_attempts = 3
# Az utolsó sikeres boot időbélyege
last_success = 2025-03-28T11:45:00Z
```

Ha a `kernel_b.img` három egymást követő indítási kísérletben sem éri el a felhasználói tér inicializálását, a bootloader automatikusan a `kernel_a.img`-t tölti be, és hibajegyet generál.

30.4.8 Hibajegy-kezelés és a regressziós kapu kapcsolata

A hibajegy-kezelő rendszer (issue tracker) a regressziós kapu emberi oldalát képviseli. Minden automatikus teszteredményhez tartozik egy hibajegy, amely tartalmazza:

- A hiba azonosítóját és súlyosságát.
- A reprodukciós lépéseket, beleértve a pontos konfigurációt, a buildparancsot és a mérési naplót.
- Az érintett komponenst (pl. `mailbox.c`).
- A hibát bevezető commit azonosítóját vagy gyanús tartományát.
- Az állapotot (nyitott, folyamatban, javított, ellenőrzött).
- Az ellenőrzéshez használt tesztet, amely a jövőben regressziós tesztként fut.

A regressziós kapu szempontjából a hibajegyek súlyossági besorolása döntő:

- **Critical**: A rendszer nem bootol, adatvesztés, biztonsági rés. A kapu zárva, a változás azonnal visszavonandó.
- **High**: A funkció fő útvonala hibás, teljesítményregresszió a tűréshatáron túl. A kapu zárva, a javítás a következő ciklusban kötelező.
- **Medium**: Nem kritikus funkció hibája, kozmetikai hiba, ritka körülmények között. A kapu nyitható, de a hiba a kiadási checklistre kerül, és a következő kiadásban javítandó.
- **Low**: Kisebb eltérés a dokumentációtól, javasolt optimalizálás. A kapu nyitva, a hiba nyomon követve.

A hibajegy életciklusa a következő:

1. **Bejelentés**: A teszt vagy a felhasználó jelzi a hibát. A bejelentéshez **automatikusan csatolódik** a tesztnapló, a konfiguráció és a reprodukciós lépések. A „nem tudom reprodukálni” állapot a jegy elutasításához vezet.
2. **Triage**: A karbantartó meghatározza a súlyosságot és a felelőst. A critical hibák azonnali figyelmet kapnak.
3. **Javítás**: A fejlesztő elkészíti a patch-et, és **hozzáadja a regressziós tesztet**.
4. **Ellenőrzés**: A patch átmegy a regressziós kapun. Ha a teszt megbukott, a jegy visszakerül a fejlesztőhöz.
5. **Lezárás**: A javítás a főágba kerül, a jegy lezárul, de a regressziós teszt a kapuban marad.

A mailbox példánál a folyamat így néz ki:

- Bejelentés: „A 8 órás soak test során 1 üzenet elveszett. Első elveszett sorszám: 572314. A memóriahasználat 198 224 KiB-ról 194 112 KiB-ra csökkent.”
- Triage: A hiba súlyossága High, mert adatvesztést okoz. A felelős a mailbox modul karbantartója.
- Javítás: A fejlesztő megtalálja a hibát: a spinlock nem volt inicializálva, és bizonyos időzítéseknél a `count` mező nem volt konzisztens. A patch inicializálja a lockot, és a `mailbox_init`-be lock-inicializálást tesz. A fejlesztő hozzáad egy regressziós tesztet, amely 1 millió üzenetet küld és fogad párhuzamosan, és ellenőrzi a sorszámok folytonosságát.
- Ellenőrzés: A patch átmegy a CI-n, a párhuzamos teszt és a soak test is zöld.
- Lezárás: A jegy lezárul, a regressziós teszt a kapuban marad.

30.4.9 Kiadási checklist

A kiadási checklist a regressziós kapu emberi oldala. Az automatikus tesztek szükségesek, de nem elégségesek. A checklist biztosítja, hogy a nem automatizálható szempontok is érvényesüljenek. Egy kernelkiadás checklistje a következő elemeket tartalmazhatja:

1. **Kód és teszt elválasztása**: A kiadásra jelölt commitban a kód és a teszt külön ellenőrzött. Nincs olyan kódrészlet, amely csak a teszt kedvéért került be.
2. **Dokumentáció**: A változások leírása, a konfigurációs lehetőségek, a hibakódok és az ABI-módosítások dokumentálva vannak.
3. **Erőforrás-tulajdonjog**: A kernelkódban minden lefoglalt erőforrás (memória, lock, fájlleíró) tulajdonjoga egyértelmű. Nincs double-free, nincs leak.
4. **Hibakezelés**: Minden hibakódot ellenőriz a kód. A `panic` csak valóban végzetes, nem helyreállítható állapotban hívódik.
5. **Versenyhelyzetek**: A közös állapotot védő lockok, atomi műveletek és memóriabarrierek helyesek. A lockolási sorrend dokumentált.
6. **Biztonsági átvilágítás**: A felhasználói térből érkező mutatók, méretek és indexek ellenőrzöttek. Nincs puffertúlcsordulás, nincs use-after-free.
7. **Reprodukciós csomag**: A kiadáshoz tartozik a buildparancs, a linker script, a futtatási parancs, a naplók és a mérési eredmények.
8. **Rollback terv**: A kiadás előtt ellenőrizve, hogy a rollback mechanizmus működik, és a visszaállítási út dokumentált.
9. **Ismert korlátok**: A nem javított, de elfogadott hibák listája a kiadási jegyzethez csatolva.

A checklisten minden tételhez tartozik egy felelős és egy állapot (teljesült, nem teljesült, nem alkalmazható). A regressziós kapu csak akkor nyílik meg, ha a checklisten nincs „nem teljesült” tétel.

30.4.10 Tipikus hibák és helyes megoldások

A regressziós kapu tervezése során gyakori hibák:

Hiba: A kapu csak a buildre és a unit tesztekre épít.
Következmény: Az integrációs és soak tesztek kimaradnak, így a párhuzamos hibák és a memóriaszivárgás észrevétlen marad.
Megoldás: A kapu minden fázist lefed, a soak test a kiadás előtt kötelező.

Hiba: A test matrix nem reprezentatív.
Következmény: A tesztek zöldek, de a valós környezetben mégis hiba jelentkezik, mert a mátrix nem tartalmazza az adott konfigurációt.
Megoldás: A mátrixot rendszeresen felülvizsgáljuk, és a felhasználói hibajelentések alapján új kombinációkat adunk hozzá.

Hiba: A regressziós teszt hiányzik a hibajavításból.
Következmény: Ugyanaz a hiba később újra bekerül, mert semmi sem akadályozza meg.
Megoldás: A hibajegy lezárásának feltétele a regressziós teszt megléte. A kapu ezt automatikusan ellenőrzi: ha a jegyhez tartozó teszt nem fut le, a kapu zárva marad.

Hiba: A rollback tervet a kiadás után készítik el.
Következmény: Amikor a hiba élesben jelentkezik, a visszaállítás kapkodva, adatvesztéssel történik.
Megoldás: A rollback terv a változás tervezési fázisában készül, és a checklist része.

Hiba: A mérési napló nem tartalmazza a várt és mért értéket külön.
Következmény: A teszteredmény nem reprodukálható, a regresszió megállapítása bizonytalan.
Megoldás: Minden mérési naplóban a várt értéket és a mért értéket külön oszlopban, a mértékegységgel együtt kell feltüntetni.

30.4.11 Labor: regressziós kapu és kiadási checklist

A labor célja, hogy a fejezetben bemutatott fogalmakat egy emulált környezetben, reprodukálható módon alkalmazza. A labor nem valós hardvert használ, hanem QEMU-t vagy más emulátort, és a hoszt gép lemezét, bootloaderét vagy rendszerpartícióját **tilos** módosítani. A labor minden állomása egy eldobható, emulált környezetben fut.

30.4.12 Előkészítés

1. Hozz létre egy munkakönyvtárat: `mkdir ~/lab-os117 && cd ~/lab-os117`.
2. Készíts egy egyszerű C programot, amely a mailbox modult teszteli felhasználói térben. A modul kódját a fejezetben bemutatott `mailbox.c`-ből származtathatod, de a spinlock részt egy egyszerű, nem atomi változattal helyettesítheted a hiba szimulálásához.
3. Készíts egy Python vagy shell tesztfuttatót, amely a programot N-szer lefuttatja, és a kimenetet naplózza.

30.4.13 A regressziós kapu futtatása

A laborban három tesztet futtatunk:

1. **Funkcionális teszt**: A mailbox küldd- és fogadd-műveleteit ellenőrzi egyszálú környezetben. Várt eredmény: 0 hiba.
2. **Párhuzamos teszt**: Két szál egyszerre küld és fogad. A teszt 10 000 üzenetet továbbít, és ellenőrzi a sorszámok folytonosságát. Várt eredmény: 0 elveszett vagy duplikált üzenet.
3. **Soak teszt (rövidített)**: A párhuzamos teszt 60 másodpercig fut, és közben a memóriahasználatot is naplózza. Várt eredmény: nincs szivárgás, nincs elveszett üzenet.

Példa a tesztfuttatóra (Python):

```
#!/usr/bin/env python3
# run_gate.py - regressziós kapu a mailbox példához
import subprocess
import time
import json
import sys

def run_test(name, command, expected_exit=0, timeout=5):
    start = time.monotonic()
    proc = subprocess.run(command, shell=True, capture_output=True,
                          text=True, timeout=timeout)
    elapsed = time.monotonic() - start
    result = {
        "name": name,
        "command": command,
        "exit_code": proc.returncode,
        "expected_exit": expected_exit,
        "elapsed_sec": round(elapsed, 3),
        "stdout": proc.stdout[-200:],
        "stderr": proc.stderr[-200:],
```

```
        "passed": proc.returncode == expected_exit
    }
    return result

def main():
    results = []
    results.append(run_test("functional",
        "./mailbox_test --test=functional"))
    results.append(run_test("parallel",
        "./mailbox_test --test=parallel --messages=10000"))
    results.append(run_test("soak_short",
        "./mailbox_test --test=soak --duration=60"))

    # Mentés JSON naplóba
    with open("gate_results.json", "w") as f:
        json.dump(results, f, indent=2)

    # A kapu döntése
    all_passed = all(r["passed"] for r in results)
    print(json.dumps(results, indent=2))
    if all_passed:
        print("GATE: PASS")
        sys.exit(0)
    else:
        print("GATE: FAIL")
        sys.exit(1)

if __name__ == "__main__":
    main()
```

A labor során **először a hibás** verziót futtasd le. A párhuzamos teszt valószínűleg néhány elveszett üzenetet jelez, mert a `count` mező nem atomi. A naplóban a mért eredmény (lost > 0) eltér a várt eredménytől (lost $= 0$). Ez a regressziós kapu zárva marad.

Ezután javítsd a hibát: adj hozzá egy egyszerű zárolást (például atomi műveletekkel), és futtasd újra a kaput. A naplóban most a mért és a várt eredmény megegyezik, a kapu nyílik.

30.4.14 Átadandó eredmény

A labor átadandó eredménye:

1. A `gate_results.json` fájl, amely a három teszt mért és várt eredményét tartalmazza.
2. A `reproduce.sh` szkript, amely a környezetet felépíti, a teszteket lefuttatja, és a naplót generálja.
3. Egy rövid `release_checklist.md` fájl, amely a kiadási checklist tételeit tartalmazza, és minden tételnél jelöli, hogy teljesült-e.

Példa a checklist-re (Markdown):

```
# Kiadási checklist - mailbox modul, v1.1.0

| Tétel | Állapot | Megjegyzés |
|---|---|---|
| Funkcionális teszt zöld | teljesült | functional: 0 hiba |
| Párhuzamos teszt zöld | teljesült | parallel: lost=0, dup=0 |
| Soak teszt zöld | teljesült | soak_short: 60 s, nincs szivárgás |
| Dokumentáció frissítve | teljesült | mailbox.h kommentek aktualizálva |
| Rollback terv rögzítve | teljesült | revert commit: abc123 |
| Ismert korlátok listázva | teljesült | nincs ismert korlát |
```

```
| Erőforrás-tulajdonjog ellenőrizve | teljesült | nincs double-free, nincs leak |
| Biztonsági átvilágítás | teljesült | nincs puffertúlcsordulás |
```

A laborban használt emulált környezet konfigurációját is rögzítsd:

```
QEMU version: 8.2.0 (ellenőrizd a helyi verziót)
CPU: qemu64 (x86_64)
Memory: 256 MiB
Kernel: nincs (userspace teszt)
Tesztprogram: mailbox_test
```

A labor nem igényel valódi kernelmódosítást, de a mérési napló és a checklist ugyanazt a fegyelmet követi, mint egy valódi kernelkiadásnál.

30.4.15 Önellenőrző kérdések

1. Mi a különbség a unit teszt és a soak test között? Miért nem elegendő csak a unit teszt a regressziós kapuban?
2. Hogyan segít a test matrix a hibák korai felismerésében? Adj egy példát arra, amikor egy hiba csak egy adott matrix cellában jelentkezik.
3. Mi a rollback stratégia lényege, és miért kell azt a változás bevezetése *előtt* meghatározni?
4. Egy hibajegy lezárásának feltétele a regressziós teszt. Miért fontos ez, és milyen következménye van, ha elmarad?

A fejezetben bemutatott regressziós kapu nem öncélú bürokrácia, hanem a mérnöki fegyelem eszköze. A biztonságos bevezetés, a test matrix, a CI, a soak test, a rollback és a hibajegykezelés együtt biztosítják, hogy a kernel fejlődése ne a stabilitás rovására történjen. A következő egységben erre az alapra építve az SMP, a preemption és a valós idejű kiterjesztések tesztelési kihívásait vizsgáljuk meg.

31 Bővítés és haladó témák

31.1 SMP, preemption és valós idejű kiterjesztés

A Regresszió és kiadási minőség alfejezetben stabilizált egyprocesszoros kernel után a következő lépés a párhuzamos végrehajtás. Ez a szakasz a szimmetrikus többmagos (SMP) bővítést és a preemptív kernel viselkedését járja körül. A cél nem egy gyártásra kész SMP kernel, hanem azoknak a mechanizmusoknak a megértése, amelyek a többmagos helyes működéshez kellenek: spinlock, per-CPU adat és preempt_count. Ehhez kapcsolódik a PREEMPT_RT koncepció és a latency mérése is.

31.1.1 Miért más a többmagos kernel?

Az eddigi egyszerű kernelben egyetlen CPU futtatta a kernel kódját, így a közös globális adatok védelméhez elegendő volt a megszakítások rövid idejű tiltása, vagy egy egyszerű kölcsönös kizárás. Amikor két vagy több CPU ugyanazt a kernel kódot futtatja, akkor valódi párhuzamos végrehajtás jön létre: két processzor egyszerre írhatja ugyanazt a globális változót. A megszakítás tiltása egyetlen CPU-n nem akadályozza meg a másik CPU futását. Ezért a közös kernel állapotot explicit, CPU-k közötti szinkronizációval kell védeni.

Az SMP rendszerek legfontosabb tulajdonságai:

- **szimmetrikus**: minden CPU azonos jogokkal és azonos memóriatérrel rendelkezik, ugyanazt a kernel képet futtatja;
- **cache-koherencia**: a hardver biztosítja, hogy a CPU cache-ekben lévő másolatok konzisztensek legyenek, de ez nem zárja ki az alkalmazás szintű versenyhelyzetet;
- **atomi utasítások**: a CPU-k rendelkeznek olyan utasításokkal, amelyek olvasnak és írnak egy memóriahelyet úgy, hogy közben másik CPU nem férhet hozzá (test-and-set, compare-exchange, load-linked/store-conditional).

A kernel kódjában ezért különbséget kell tennünk a **CPU-lokális** és a **globális** állapot között. A CPU-lokális állapotot nem kell lezárni, mert csak az adott CPU éri el; a globális állapotot viszont minden CPU hozzáférhet, ezért zárolni kell.

31.1.2 Spinlock: a legalapvetőbb CPU-k közötti zár

A spinlock olyan kölcsönös kizárási eszköz, amely akkor használható, amikor a kritikus szakasz nagyon rövid, és a várakozó CPU-nak nem érdemes aludnia. A spinlockot használó CPU aktívan pörög (spin-el), amíg a zárat meg nem kapja.

31.1.3 Működési elve

A spinlock egyetlen atomi memóriahelyre épül. A zárolás során a CPU atomikusan megkísérli a `0` értéket `1`-re cserélni. Ha az eredmény `0`, a zár az övé; ha `1`, akkor a zár foglalt, és a CPU addig pörög, amíg a zár fel nem szabadul.

A következő példa egy egyszerű spinlockot mutat GCC beépített atomi függvényekkel. A kód szemléltetés, nem éles kernelhez való:

```
/* Egyszerű spinlock, csak oktatási célra.
 * A valós kernelben a lockolás előtt preempt_disable() is szükséges.
 */
typedef struct {
    volatile unsigned int locked;
} spinlock_t;

static inline void spin_lock(spinlock_t *l)
{
    /* Atomi test-and-set: if (locked == 0) { locked = 1; return 0; }
     * else return 1. A pörgés közben használjunk relaxált olvasást,
     * hogy a CPU ne terhelje feleslegesen a cache-összeköttetést.
     */
    while (__sync_lock_test_and_set(&l->locked, 1)) {
        while (__sync_fetch_and_or(&l->locked, 0))
            asm volatile("pause");
    }
}

static inline void spin_unlock(spinlock_t *l)
{
    /* Release szemantika: a kritikus szakasz minden írása látható,
     * mielőtt a zár felszabadul. */
    __sync_lock_release(&l->locked);
}
```

A példában a `__sync_lock_test_and_set` atomi test-and-set műveletet végez. A `pause` utasítás x86-on csökkenti a pörgés okozta energiafogyasztást és javítja a többi szál haladását. A `__sync_lock_release` release szemantikát ad, vagyis a kritikus szakaszban végzett írások nem rendeződhetnek a zár felszabadítása után.

Fontos, hogy a spinlock használata előtt le kell tiltani a kernel preemptiót is. Ha ugyanis a CPU a kritikus szakasz közepén preemptálódik, akkor a zár hosszú ideig foglalt maradhat, és a többi CPU feleslegesen pörög. A helyes minta tehát:

```
void spin_lock_correct(spinlock_t *l)
{
    preempt_disable();
    spin_lock(l);
}

void spin_unlock_correct(spinlock_t *l)
{
    spin_unlock(l);
    preempt_enable();
}
```

31.1.4 Tipikus hibák

- **Rekurzív spinlock**: ha ugyanaz a CPU megpróbálja kétszer felvenni ugyanazt a spinlockot, a második hívás végtelen pörgésbe kerül, mert a zár már az adott CPU-é. A spinlock nem

rekurzív.

- **Interrupt kontextusban való zárolás**: ha a kernel megszakításkezelője olyan spinlockot próbál felvenni, amelyet ugyanaz a CPU már normál kontextusban tart, holtpont alakulhat ki, mert a megszakításkezelő nem tudja felszabadítani a zárat. Ezért interruptok alatt általában interrupt-tiltó spinlockot használunk.
- **Hosszú kritikus szakasz**: a spinlock csak nagyon rövid műveletekhez való. Ha a kritikus szakasz memóriát másol vagy fájlrendszert jár be, a többi CPU pörgése jelentős késleltetést okoz.
- **Nincs preempt_disable**: a preemptiós kernelben a zár felvétele előtti és utáni preempt állapot megváltoztathatja a CPU futását, ezért a lockolásnak mindig preempt tiltással kell együtt járnia.

31.1.5 Per-CPU adat: a cache-versengés csökkentése

A többmagos kernelben a globális számlálók és egyéb gyakran írt adatok komoly cache-invalidálást okozhatnak. Ha például minden CPU ugyanazt a globális `packet_count` változót növeli, akkor minden növelés során a cache-vonalat át kell mozgatni a CPU-k között. Ezt hívjuk cache-line ping-pong-nak.

A megoldás az, hogy az adatot CPU-nként külön példányban tároljuk, és csak akkor férünk hozzá, amikor tényleg szükséges. Ezt nevezzük per-CPU adatnak. A per-CPU adat nem igényel spinlockot, mert minden CPU csak a saját példányát módosítja. A gyűjtő művelet (például összegzés) azonban minden CPU példányát olvassa, de ez ritkábban történik, mint a növelés.

A következő egyszerű példa egy per-CPU számlálót mutat. A valós kernelben ez makrókkal történik, de a lényeg ugyanaz:

```
#define MAX_CPUS 8

struct per_cpu_counter {
    unsigned long count;
    unsigned char padding[64 - sizeof(unsigned long)];
        /* cache-vonal igazítás: elkerüljük a hamis osztást */
};

static struct per_cpu_counter counters[MAX_CPUS];

int current_cpu(void)
{
    /* Itt a CPU azonosítót az APIC vagy a per-CPU kernel stack
     * adja vissza. Például:
     *   return read_cpu_id();
     * A valós kernelben a preempt_disable védi a használatot.
     */
    return 0; /* egyszerűsített pszeudokód */
}

void increment_this_cpu_counter(void)
{
    int cpu = current_cpu();
    counters[cpu].count++;
}

unsigned long read_total_counter(void)
{
    unsigned long sum = 0;
    int cpu;
    for (cpu = 0; cpu < MAX_CPUS; cpu++) {
```

```
        sum += counters[cpu].count;
    }
    return sum;
}
```

A helyes hozzáférés érdekében a kernelben két feladatot kell elvégezni:

1. A CPU azonosító lekérése előtt tiltjuk a preemptiót.
2. A hozzáférés után visszaállítjuk a preempt állapotot.

Ez a `get_cpu()` / `put_cpu()` minta:

```
int cpu = get_cpu();
counters[cpu].count++;
put_cpu();
```

A `get_cpu()` visszaadja az aktuális CPU azonosítót, és megnöveli a `preempt_count`-ot, így a kernel nem válthat CPU-t a hozzáférés közben. A `put_cpu()` csökkenti a számlálót, és ha szükséges, ütemezést indít.

Tipikus hibák:

- **CPU váltás per-CPU mutatóval**: ha valaki elmenti a per-CPU adat címét, majd a CPU-t átváltja a kernel, a mutató elavulttá válik, és másik CPU adatát írjuk.
- **Hamis cache-osztás**: ha két per-CPU változó ugyanabba a cache-vonalba kerül, akkor a CPU-k egymás cache-vonalát invalidálják, hiába külön adatról van szó. Ezért a per-CPU struktúrákat cache-vonal méretre igazítjuk.
- **Túl sok per-CPU memória**: a per-CPU adat a CPU-k számával szorozva foglal memóriát. A tervezésnél mérlegelni kell, hogy a gyors hozzáférés megéri-e a nagyobb lábnyomot.

31.1.6 A preempt count és a kernel preemptió

A **preemptió** azt jelenti, hogy a kernel futás közben átadhatja a CPU-t egy másik készen álló feladatnak, például egy magasabb prioritású folyamatnak. Ez javítja a válaszidőt, de veszélyezteti a kernel adatstruktúráinak konzisztenciáját. Ezért a kernelben bizonyos szakaszokban tilos a preemptió.

A **preempt_count** egy per-task számláló, amely megmutatja, hogy hány olyan réteg tiltja éppen a preemptiót. Ha a számláló értéke nagyobb, mint nulla, a kernel nem válthat kontextust. A számlálót a `preempt_disable()` növeli, a `preempt_enable()` pedig csökkenti. Amikor a számláló nullára csökken, a kernel megvizsgálja, hogy van-e függőben lévő ütemezési igény, és ha igen, ütemez.

A következő példa egy egyszerű preempt_count kezelést mutat. A valós Linux kernelben a `preempt_count` egy bitmező, amely a hardirq, softirq, preempt és NMI szinteket külön tárolja, de az alapelv ugyanez:

```
struct thread_context {
    int preempt_count;
    /* ... további mezők ... */
};

static inline void preempt_disable(void)
{
    current->preempt_count++;
    /* Compiler barrier: az írás nem rendeződhet át a
     * kritikus szakasz elé. */
```

```
    asm volatile("" ::: "memory");
}

static inline void preempt_enable(void)
{
    asm volatile("" ::: "memory");
    current->preempt_count--;
    if (current->preempt_count == 0 && need_resched())
        schedule();
}
```

A `preempt_disable()` és `preempt_enable()` hívások párban kell, hogy megjelenjenek. Minden tiltáshoz tartoznia kell egy engedélyezésnek. Ha a számláló nem nullára csökken, akkor még mindig marad tiltás, ezért nem szabad ütemezni.

A preempt_count használatával a kernel kódja kijelölheti a nem preemptálható szakaszokat anélkül, hogy a megszakításokat teljesen letiltaná. Ez fontos az SMP kernelben, mert a megszakítások tiltása csak az adott CPU-ra hat, míg a preempt_count a kernel folyamatán belüli konzisztenciát védi.

Tipikus hibák:

- **Preempt count szivárgás**: ha a `preempt_disable()` után a függvény hibával visszatér, és nem hívja meg a `preempt_enable()`-t, a számláló nem nullázódik, ami a kernel preemptiójának blokkolásához vezet. A kernelben ilyenkor `BUG()` vagy `WARN()` jelzi a hibát.
- **Túlzott preempt tiltás**: hosszú kritikus szakaszok alatt a preemptió tiltása késlelteti a magas prioritású feladatokat, és rontja a rendszer válaszidejét.
- **Preempt tiltás és alvás**: ha a kernel kódja a preempt tiltott szakaszban alvó függvényt hív, az hibát okoz, mert a processzor nem adhatja át a vezérlést.

31.1.7 PREEMPT_RT és a latency fogalma

A hagyományos Linux kernelben a `CONFIG_PREEMPT` opció teszi lehetővé a kernel preemptióját, de bizonyos szakaszokban továbbra is tiltva van, és a spinlockok is tiltják a preemptiót. A **PREEMPT_RT** (real-time) patch célja, hogy a kernel minél több pontján preemptálható legyen, és a megszakításokat is kernel szálakban futtassa. Így a magas prioritású felhasználói feladatok rövidebb és determinisztikusabb késleltetéssel kapják meg a CPU-t.

A PREEMPT_RT legfontosabb átalakításai:

- A spinlockok nagy részét **preemptálható mutexekre** cseréli. Ez azt jelenti, hogy a várakozó feladat elalszik, és a CPU-t más feladat kaphatja meg.
- A hardveres megszakításkezelők egy részét **kernel szálakba** helyezi. A megszakításkezelő így alacsonyabb prioritással futhat, mint a valós idejű feladatok, így azok nem várnak a megszakításkezelőre.
- Bevezeti a **priority inheritance** (prioritás öröklés) mechanizmust, hogy elkerülje a prioritás inverziót.

A latency két fontos összetevője:

- **Megszakítási késleltetés** (L_{irq}): a hardveres megszakítás érkezésétől a megszakításkezelő első utasításáig eltelt idő.
- **Ütemezési késleltetés** (L_{sched}): attól az időponttól, amikor egy feladat számára ébredési esemény történik (például egy megszakítás befejeződik), addig, amíg a feladat ténylegesen futni kezd.

A PREEMPT_RT célja ezeknek a késleltetéseknek a csökkentése és a maximális érték (worst-case) determinisztikussá tétele. A mérést általában `cyclictest` típusú programmal végzik, amely időzítő megszakításokkal ébreszti magát, és méri az ébredés és a futás közötti különbséget. A mérés során rögzíteni kell a hardvert, a kernel konfigurációt, az órajelfrekvenciát és a mérési módszert, ahogy a könyv általános mérési szabályai előírják.

A PREEMPT_RT állapotáról fontos megjegyezni: a mainline Linux különböző verziói eltérő mértékben tartalmazzák a real-time patcheket. 2025 elején a PREEMPT_RT jelentős része már be van olvasztva a mainline kernelbe, de az adott kernelverzió pontos támogatását a reprodukciós csomagban rögzíteni kell. Ez az állítás verzióellenőrzést igényel a konkrét kiadásnál.

31.1.8 Labor: SMP- és preemption-terv készítése

Az átadandó feladat nem egy kész SMP kernel, hanem egy **reprodukálható tervdokumentum**, amely a meglévő egyszerű kernel (Regresszió és kiadási minőség stabil build) SMP-re és preemptív bővítésének lépéseit, konfigurációját és mérési tervét tartalmazza. A dokumentum célja, hogy a következő fejezetben (Hálózat és biztonsági modulok) a hálózati és biztonsági modulok implementálásakor már ismert legyen, hol kell spinlock, per-CPU adat és preempt_count.

31.1.9 Kiinduló állapot és környezet

- **Alap kernel**: a Regresszió és kiadási minőség alfejezetben tesztelt, egyprocesszoros, naplózást és panic kezelést tartalmazó build. Commit azonosítóval rögzítve a reprodukciós csomagban.
- **Emulált hardver**: QEMU x86_64, két virtuális CPU (`-smp 2`), 512 MiB RAM, hálózat nélkül, soros konzollal.
- **Build eszközök**: a korábbi fejezetekben használt cross-toolchain, linker script és build-rendszer.
- **Fordítási beállítások**: `-O2 -g`, `-mno-red-zone`, `-ffreestanding`, `-fno-stack-protector` (a pontos opciókat a buildnapló tartalmazza).

31.1.10 A terv elvárt tartalma

A dokumentumnak a következő szakaszokat kell tartalmaznia:

1. **Célkitűzések**:
   - Két vCPU indítása és a kernel adatstruktúrák SMP-biztossá tétele.
   - Preemptív kernel bekapcsolása `CONFIG_PREEMPT` mintára.
   - Mérési terv a megszakítási késleltetés és a jitter meghatározására.
2. **Adatstruktúra-terv**:
   - A globális kernel állapotok felmérése: melyek lesznek spinlockkal védve, melyek per-CPU adatok.
   - A `spinlock_t` mérete és igazítása: például 4 bájt, cache-vonalra igazítva.
   - A per-CPU adat blokk mérete és CPU-nkénti elrendezése: a per-CPU struktúra mérete 64 bájt cache-vonaligazítással.
   - A `preempt_count` mező elhelyezkedése a `thread_context` struktúrában, mérete és bitbeosztása (ha a valós bitmezőt használjuk).
3. **Szinkronizációs terv**:

- Melyik spinlock védi a folyamatkezelő globális listát, az ütemező futósorát és a napló puffert.
- Hogyan oldják meg a spinlockok a preempt tiltást: `spin_lock()` belép előtt `preempt_disable()`, kilép után `preempt_enable()`.
- Per-CPU adatok elérése `get_cpu()` / `put_cpu()` mintával.

4. **Boot folyamat**:

   - A másodlagos CPU-k indítása: az APIC vagy a QEMU által biztosított módszerrel. A tervben pszeudokóddal le kell írni, hogyan ébrednek a másodlagos CPU-k, és hol várnak az ütemezőre.

5. **Mérési terv**:

   - **Mért késleltetés**: a kernel időzítő megszakításának kezelője feljegyzi az időbélyeget, amikor a megszakítás beérkezett. A `cyclictest`-hez hasonló kernel folyamat vagy egy egyszerű felhasználói program méri, mennyi idő telik el az ébresztés és a futás között.
   - **Várt késleltetés**: levezetett érték a kernel kód elemzése alapján. Például a megszakítás belépési út hossza T_{entry} ciklusban, a handler $T_{handler}$ ciklusban. QEMU alatt a vCPU-ütemezés overheadje hozzáadódik; ezt a várt értéknél külön fel kell tüntetni.
   - **Ciklusszám mérés**: a megszakítás belépési és kilépési szakaszban `rdtsc` vagy más időbélyeg.
   - **Mérési napló minta**:

```
[ 0.001] CPU0: irq entry ts=0x0000000000101234, exit ts=0x0000000000101250, delta=28 cycles
[ 0.001] CPU1: irq entry ts=0x0000000000101300, exit ts=0x0000000000101322, delta=34 cycles
[ 0.002] CPU0: wakeup latency min=420 cycles, max=1880 cycles, avg=612 cycles
[ 0.002] CPU1: wakeup latency min=430 cycles, max=2400 cycles, avg=730 cycles
```

   - A mért és várt értékeket külön táblázatban kell szerepeltetni:

Metrikus	Várt érték	Mért érték	Megjegyzés
IRQ belépési késleltetés	$<$ 100 ciklus (szoftveres rész)	28–34 ciklus	QEMU overhead nélkül
Teljes IRQ késleltetés	1–5 µs QEMU alatt	2{,}1–3{,}4 µs	A QEMU vCPU ütemezésétől függ
Wakeup latency (preemptív kernel)	$<$ 50 µs	8–24 µs	`CONFIG_PREEMPT` hatása

6. **Reprodukciós csomag**:

   - A QEMU indító parancs:

```
qemu-system-x86_64 \
    -smp 2 \
    -m 512M \
    -nographic \
    -kernel build/os.bin \
    -append "console=ttyS0 loglevel=7" \
    -no-reboot -no-shutdown
```

   - A build parancs: `make build/os.bin` a megadott config-gal.
   - A kernel config, linker script, a módosított forrásfájlok listája, a mérési szkript és a nyers naplók.

7. **Visszaállítás**:

- A munkát külön git branch-en kell végezni, a kiinduló commit-ból. A labor végén a mérések után a branch visszaállítható, vagy a patch sorozat megőrizhető.

31.1.11 Ellenőrzési pontok a laborhoz

- A spinlockok minden hozzáférést lefednek a globális adatokhoz?
- A `preempt_disable()` és `preempt_enable()` hívások párosak-e?
- A per-CPU adatok hozzáférése `get_cpu()`/`put_cpu()` mellett történik-e?
- A mérési naplóban a mért és várt értékek elkülönülnek-e?
- A reprodukciós csomag tartalmazza-e a kernel konfigurációt, a QEMU verziót, a toolchain verziót és a mérési körülményeket?

31.1.12 Önellenőrző kérdések

1. Miért nem elegendő a megszakítások tiltása a globális kernel adatok védelmére SMP rendszerben?
2. Magyarázza el, miért kell `preempt_disable()` a spinlock felvétele előtt, és mi történhet, ha elmarad.
3. Hogyan csökkenti a cache-versengést a per-CPU adat?
4. Mi a különbség a preempt_count és a spinlock között a kernel preemptiójának szabályozásában?

31.1.13 Összefoglalás

Az SMP bővítés a kernelben nem egyszerűen a CPU-k számának növelését jelenti. A közös állapotok védelméhez atomi spinlockokra, a gyakran írt állapotokhoz per-CPU adatokra, a kernel preemptiójának szabályozásához pedig preempt_count-ra van szükség. A PREEMPT_RT ezeket a mechanizmusokat fejleszti tovább a determinisztikus válaszidő érdekében. A fejezet laborja egy tervdokumentum elkészítése volt, amely a mérésekkel együtt megalapozza a következő fejezetben bevezetendő hálózati és biztonsági modulok szinkronizációs igényeit. Minden mérésnél a várt értéket a hardver, a konfiguráció és a kernel kód elemzése alapján le kell vezetni, és a mért értéktől elkülönítve kell dokumentálni.

31.2 Hálózat és biztonsági modulok

A hálózati és biztonsági bővítés a minimális operációs rendszerben akkor válik szükségessé, amikor a kernel már stabilan kezeli a memóriát, a folyamatokat és az alapvető perifériákat. A korábbi fejezetben (SMP, preemption és valós idejű kiterjesztés) elvégzett SMP- és preemption-támogatás után a következő logikus lépés, hogy a rendszer képes legyen adatokat cserélni a külvilággal, és hogy ezt a képességet biztonságosan, ellenőrzött módon tegye elérhetővé a felhasználói kód számára. Ez a fejezet a hálózati verem és a biztonsági modulok bevezetését mutatja be egy saját, egyszerű kernelben, nem ismételve a korábbi hálózati alapokat (Rétegmodellek és TCP/IP–Zero-copy, offload és hálózati teljesítmény), hanem a minimális rendszerbe való integrációra, a driver–socket–policy illesztésre és a támadási felület csökkentésére fókuszálva.

A cél nem egy teljes értékű TCP/IP-verem megvalósítása, hanem egy olyan, UDP-szintű hálózati útvonal kialakítása, amelyen keresztül a folyamatok ellenőrzött módon küldhetnek és fogadhatnak csomagokat. Ezzel párhuzamosan egy LSM-szerű (Linux Security Module-jellegű) hook-rendszert

vezetünk be, amely a rendszerhívások és a hálózati réteg döntési pontjain egységes policy-ellenőrzést tesz lehetővé. Minden kódrészlet szemléltetés; az itt bemutatott kódot kizárólag emulált környezetben, például QEMU-ban szabad futtatni, és a valós gép rendszerpartícióját, bootloaderét vagy firmware-ét tilos módosítani.

31.2.1 1. Hálózati bővítés tervezési elvei

A minimális kernel hálózati bővítésénél a legfontosabb elv a **rétegzett, de vékony absztrakció**. A kernelnek nem kell minden protokollt azonnal támogatnia; elegendő, ha a következő rétegeket választjuk szét:

1. **Eszközréteg**: a hálózati kártya regisztereit, megszakításait és adatpuffereit kezeli; itt valósul meg a tényleges adatcsere a hardverrel.
2. **Csomagkezelő réteg**: egységes pufferformátumot (`netbuf`) és életciklus-szabályokat biztosít a felsőbb rétegek számára.
3. **Protokollréteg**: minimális Ethernet-, ARP-, IPv4- és UDP-feldolgozást végez; a TCP-t későbbi bővítésként hagyjuk.
4. **Socket réteg**: a felhasználói folyamatok számára fájlleíró-alapú hozzáférést ad, és összekapcsolja a rendszerhívásokat a protokollréteggel.
5. **Biztonsági modul réteg**: a rendszerhívások és a protokollréteg közötti pontokon hook-ok segítségével policy-döntéseket hoz.

A tervezés során törekedni kell arra, hogy a drivert ne terheljük protokoll-logikával, a protokollréteget ne terheljük a socket-állapot kezelésével, és a biztonsági modulokat ne keverjük a hálózati kódba. Ez a szétválasztás teszi lehetővé a későbbi bővítést: újabb protokoll, újabb driver vagy újabb policy hozzáadása nem érinti a többi réteget.

A minimális kernelben a hálózati verem **eseményvezérelt** és **megszakítás-alapú** lehet, de a bonyolult ütemezési követelmények miatt gyakran érdemes kettős megközelítést alkalmazni: a fogadási oldal megszakításból csak egy félkész állapotba helyezi a csomagot, és egy későbbi feldolgozási ponton (softirq, kthread vagy egyszerű poll) történik a protokoll-feldolgozás. Ez a mintázat csökkenti a megszakításkontextusban eltöltött időt, és lehetővé teszi a zárolások egyszerűbb használatát. A küldési oldal általában szinkron hívás, de a driver visszaadhat `-ENOBUFS` hibát, ha a hardveres sor tele van.

31.2.2 2. Eszközabsztrakció és driver-modell

A minimális OS-ben a hálózati eszközöket a `net_device` struktúra reprezentálja. Ez a struktúra nem a hardver allokációjáért felel, hanem egy vezérlőfelületet ad a kernel számára.

```
struct net_device {
    char name[16];
    uint32_t flags;
    uint32_t mtu;
    uint8_t mac[6];
    void *priv;   /* driver-specifikus adatok */

    int (*open)(struct net_device *dev);
    int (*stop)(struct net_device *dev);
    int (*send)(struct net_device *dev, struct netbuf *buf);
    int (*recv)(struct net_device *dev, struct netbuf **bufp);
};
```

A driver a `priv` mezőn keresztül tárolja a hardverregiszterek virtuális címét, a DMA-pufferek pointerét és a gyűrűk indexeit. A `flags` mezőben bitenként jelölhetjük az állapotot: `NETDEV_FLAG_UP`, `NETDEV_FLAG_PROMISC`, `NETDEV_FLAG_LOOPBACK`. A `mtu` a maximális csomagméretet adja meg; a küldési útvonalon ez alapján ellenőrizzük a felhasználói adat hosszát.

A fogadási oldal tipikus megszakításkezelője nem közvetlenül a protokollréteget hívja, hanem a csomagot behelyezi egy per-CPU vagy globális fogadási sorba. A következő pszeudokód egy egyszerű megszakításkezelőt mutat:

```
void net_interrupt_handler(struct irq_context *ctx)
{
    struct net_device *dev = ctx->dev;
    struct netbuf *buf = NULL;

    while (dev->recv(dev, &buf) == 0) {
        if (buf == NULL)
            break;
        netif_rx(dev, buf);  /* beillesztés a feldolgozási sorba */
        buf = NULL;
    }

    irq_ack(ctx->irq);
}
```

A `netif_rx` felelős azért, hogy a csomag a megfelelő feldolgozási kontextusba kerüljön. Minimális rendszerben megvalósítható egyetlen zárt sorral és egy kernel-szál általi feldolgozással, de figyelni kell a sor méretére: korlátlan növekedés esetén a támadó memóriatúlcsordulást okozhat. A sor maximális hosszát explicit konstansként kell megadni, például `NET_RX_QLEN = 256`, és ha a sor tele van, a csomagot el kell dobni, valamint számlálót kell növelni a statisztikai naplóhoz.

A `send` műveletnek atomikusnak kell lennie a driver szempontjából. A következő példa egy leegyszerűsített virtio-net küldő függvényt vázol; a valós virtio implementáció ennél összetettebb, és a gyártói specifikációt kell követni (ellenőrzés szükséges az aktuális verzióhoz).

```
int virtio_net_send(struct net_device *dev, struct netbuf *buf)
{
    struct virtio_net_priv *priv = dev->priv;

    if (!(dev->flags & NETDEV_FLAG_UP))
        return -ENETDOWN;
    if (buf->len > dev->mtu)
        return -EMSGSIZE;

    if (virtqueue_add_outbuf(priv->txq, buf->data, buf->len) < 0)
        return -ENOBUFS;

    virtqueue_kick(priv->txq);
    return 0;
}
```

A példa szemlélteti a tipikus hibakezelést: az állapot, a méret és az erőforrás-ellátottság ellenőrzését a tényleges hardverművelet előtt kell elvégezni. A `virtqueue_add_outbuf` visszatérési értékét mindig kezelni kell; ha a gyűrű tele van, a hívó réteg `-ENOBUFS` hibát kap, amelyet a socket réteg a felhasználónak továbbíthat.

31.2.3 3. Csomagbuffer és életciklus

A hálózati verem központi adatszerkezete a csomagbuffer. A Linux `sk_buff` mintájára a minimális kernelben egy egyszerű `netbuf` struktúrát vezetünk be, amely a csomag adatait, hosszát és

metaadatait tárolja.

```
struct netbuf {
    struct list_node list;  /* láncolás a sorokhoz */
    uint8_t *data;
    size_t len;
    size_t capacity;
    uint32_t flags;
    struct net_device *dev;
    struct socket *sock;    /* fogadáskor a cél socket */
};
```

A `data` pointer a csomag kezdetére mutat; a `len` az érvényes bájtok számát, a `capacity` pedig a lefoglalt pufferméretet adja meg. A `flags` bitmezőben jelölhetjük például a `NETBUF_FLAG_LOOPBACK` vagy a `NETBUF_FLAG_BROADCAST` állapotot. A `dev` az az eszköz, amelyről a csomag érkezett vagy amelyre küldik; a `sock` fogadásnál a cél socketre mutat.

A csomagbuffer életciklusának szabályai kritikusak a biztonság és a stabilitás szempontjából:

- A puffert a fogadási oldalon a driver allokálja, általában DMA-képes memóriaterületről.
- A feldolgozási sorba kerülés után a tulajdonjog a veremé; a driver nem férhet hozzá.
- A protokollréteg feldolgozás után vagy továbbítja a sockethez, vagy felszabadítja a puffert.
- A küldési oldalon a felhasználói adatokat a kernel lemásolja egy kernel-pufferbe, és a verem hozza létre a `netbuf`-ot; a felhasználói memória soha nem kerül közvetlenül a hardverhez.

A dupla felszabadítás vagy a használat utáni felszabadítás elkerülésére a felszabadító függvénynek nulláznia kell a mutatót, és a tulajdonjog-átadási pontokat a kódban kommentárral kell jelölni. A minimális kernelben érdemes bevezetni egy egyszerű `netbuf_alloc` és `netbuf_free` párt, amelyek egy fix méretű memóriapoolból gazdálkodnak.

```
struct netbuf *netbuf_alloc(size_t capacity)
{
    struct netbuf *buf = kmalloc(sizeof(*buf));
    if (!buf)
        return NULL;

    buf->data = kmalloc(capacity);
    if (!buf->data) {
        kfree(buf);
        return NULL;
    }

    buf->capacity = capacity;
    buf->len = 0;
    buf->flags = 0;
    return buf;
}

void netbuf_free(struct netbuf *buf)
{
    if (!buf)
        return;
    kfree(buf->data);
    kfree(buf);
}
```

A fenti kód egyszerű, de a `kmalloc` és `kfree` a kernel memóriaallokátorát használja, amelyet a korábbi fejezetekben (Heapallokátorok: free list, buddy és slab–Kernelmemória, vmalloc, kmalloc és biztonság) részletesen tárgyaltunk. A `netbuf_alloc` hibakezelése példaértékű: ha a második allokáció sikertelen, az elsőt fel kell szabadítani, különben memóriaszivárgás keletkezik.

31.2.4 4. Socket réteg és hálózati rendszerhívások

A felhasználói folyamatok számára a hálózati hozzáférést a socket rendszerhívások biztosítják. A minimális kernelben a socketet a folyamat fájlleíró-táblájában tárolt objektumként implementáljuk, hasonlóan a pipe-hoz vagy a megszokott fájlleírókhoz. A socket létrehozása a `socket` rendszerhívással történik:

```
int sys_socket(int family, int type, int protocol);
```

A paraméterek közül a `family` az `AF_INET` értéket veheti fel, a `type` az `SOCK_DGRAM` (UDP) típust támogatja, a `protocol` pedig általában `0`. A visszatérési érték egy új fájlleíró, hiba esetén `-1`, és az `errno` a hibatípust jelzi. A lehetséges hibakódok: `-EAFNOSUPPORT` (ismeretlen címcsalád), `-EPROTONOSUPPORT` (nem támogatott protokoll), `-ENOMEM` (nincs memória a socket struktúrához).

A minimális UDP-kommunikációhoz három további rendszerhívásra van szükség:

```
int sys_bind(int fd, const struct sockaddr *addr, socklen_t addrlen);
int sys_sendto(int fd, const void *buf, size_t len, int flags,
               const struct sockaddr *dest, socklen_t destlen);
int sys_recvfrom(int fd, void *buf, size_t len, int flags,
                 struct sockaddr *src, socklen_t *srclen);
```

A `bind` egy helyi címhez és porthoz köti a socketet; a `sendto` az adott célcímre küld adatot, a `recvfrom` pedig a fogadott csomag forráscímét is visszaadja. A rendszerhívások minden esetben ellenőrzik a felhasználói mutatók érvényességét, a hosszmezők konzisztenciáját és az erőforráskorlátokat. A `sendto` és `recvfrom` esetében a `len` paraméter nem haladhatja meg a definiált maximális UDP-csomagméretet, például `UDP_MAX_PAYLOAD = 1472` bájtot (IPv4 esetén 1500 bájtos MTU mellett). A `flags` paramétert egyelőre figyelmen kívül hagyjuk, de a felületen hagyjuk meg a későbbi bővítéshez.

A socket réteg felelős a csomagok és a folyamatok közötti adatcsere lebonyolításáért. Fogadáskor a protokollréteg a `sock` mezőben megadott sockethez rendeli a bejövő `netbuf`-ot, és a socket fogadási sorába helyezi. A `recvfrom` rendszerhívás blokkol, amíg a sor üres, vagy ha `O_NONBLOCK` flag be van állítva, `-EAGAIN` hibát ad. A küldési oldalon a `sendto` a felhasználói puffer tartalmát kernelmemóriába másolja, majd a protokollrétegen keresztül a driverhez továbbítja.

Az adatmásolásnál alapvető biztonsági szabály, hogy a kernel soha ne olvasson közvetlenül a felhasználói mutatóról anélkül, hogy a memóriatartományt ellenőrizte volna. A minimális kernelben a `copy_from_user` és `copy_to_user` függvények ellenőrzik, hogy a cél- és forrástartomány a felhasználói címtartományon belül van-e, és hogy a hossz nem lépi át a szegmenshatárt. A hibát `-EFAULT` értékkel jelezzük.

31.2.5 5. Biztonsági modulok: LSM-szerű hook-rendszer

A hálózati és egyéb kernel-funkciók biztonságos elérhetőségének szabályozására a minimális OS egy **biztonsági modul réteget** vezet be. Ez a réteg nem egyetlen policy-implementáció, hanem egy hook-mechanizmus, amelyhez különböző policy-modulok csatlakozhatnak. A koncepció a Linux Security Modules (LSM) mintáját követi, de a minimális kernelben jóval egyszerűbb felülettel.

A hook-ok olyan függvényhívások, amelyeket a kernel kritikus döntési pontjain helyezünk el. A hálózati veremben legalább a következő hook-pontokra van szükség:

- `security_socket_create(family, type, protocol)` – socket létrehozásakor.

- `security_socket_bind(sock, addr, addrlen)` – cím hozzárendelésekor.
- `security_socket_send(sock, buf, len, dest)` – küldés előtt.
- `security_socket_recv(sock, buf, len, src)` – fogadás előtt.
- `security_net_rx(dev, netbuf)` – bejövő csomag feldolgozása előtt.

A hook-ok minden esetben `int` típussal térnek vissza: `0` jelenti az engedélyezést, negatív érték pedig a hibakódot (például `-EPERM`). A kernel kódja a hook hívása után azonnal ellenőrzi a visszatérési értéket, és hiba esetén megszakítja a műveletet.

A minimális kernelben a hook-ok megvalósítása egyszerű függvénymutató-táblával történhet:

```
struct security_ops {
    int (*socket_create)(int family, int type, int protocol);
    int (*socket_bind)(struct socket *sock,
                       const struct sockaddr *addr,
                       socklen_t addrlen);
    int (*socket_send)(struct socket *sock,
                       const void *buf, size_t len,
                       const struct sockaddr *dest);
    int (*net_rx)(struct net_device *dev, struct netbuf *buf);
};
```

A kernel indításakor egy alapértelmezett, mindent engedélyező modul töltődik be, amely minden hook-ban `0`-t ad vissza. A biztonsági modul a `security_set_ops` függvénnyel regisztrálhatja saját implementációját. A regisztráció csak a rendszer indulásakor, a felhasználói folyamatok elindítása előtt megengedett, hogy a futás közbeni csere ne okozzon inkonzisztens állapotot.

A hálózati példában egy **hálózati ACL-policy** modult mutatunk be. A modul célja, hogy a felhasználói folyamatok csak a loopback interfészen, a `127.0.0.1` címre és a `8080`-as portra küldhessenek UDP-csomagokat. Minden más célcímet vagy portot elutasítunk. A `security_socket_send` hook a következő logikát valósítja meg:

```
int acl_socket_send(struct socket *sock,
                    const void *buf, size_t len,
                    const struct sockaddr *dest)
{
    const struct sockaddr_in *addr =
        (const struct sockaddr_in *)dest;

    if (addr->sin_family != AF_INET)
        return -EAFNOSUPPORT;

    uint32_t ip = ntohl(addr->sin_addr.s_addr);
    uint16_t port = ntohs(addr->sin_port);

    if (ip != 0x7f000001U)   /* 127.0.0.1 */
        return -EPERM;
    if (port != 8080)
        return -EPERM;

    return 0;
}
```

A hook a `sendto` rendszerhívásban, a felhasználói puffer kernelbe másolása **után**, de a protokollrétegnek történő átadás **előtt** fut. Ez a sorrend fontos: a policy-ellenőrzésnek a tényleges hálózati művelet előtt kell megtörténnie, hogy a tiltott csomag ne is kerüljön a driverhez. Ugyanakkor a hook nem férhet hozzá a felhasználói memóriához, mert az a `copy_from_user` után már kernel-címről érhető el.

A hook-rendszer naplózási képességet is kaphat. A minimális kernelben a `klog` segítségével rögzítjük az elutasításokat: a folyamat PID-jét, a kért célcímet és portot, valamint az időbélyeget. A napló a labor során a mérési jegyzőkönyv alapja.

31.2.6 6. Támadási felület és tipikus hibák

A hálózati és biztonsági bővítés bevezetése jelentősen megnöveli a kernel támadási felületét. A hálózati csomagok külső, ellenőrizetlen adatokat hoznak a rendszerbe, a rendszerhívások pedig újabb, felhasználó által vezérelt belépési pontokat jelentenek. A fejezetben bemutatott kódban és a laborban a következő hibákat kell kiemelten kezelni.

31.2.7 6.1. Csomagparser biztonsága

A hálózati protokollréteg a bejövő csomag fejléceit értelmezi. A legtipikusabb hiba a **hosszmezők ellenőrzés nélküli használata**, amely puffertúlcsorduláshoz vezet. Például az IP-fejléc `total_length` mezője nagyobb lehet, mint a ténylegesen beolvasott puffer mérete; ha a kód ezt a hosszmezőt használja a másolási művelethez, a kernel a puffer határán túlra ír. Helyes megoldás, ha minden egyes mező feldolgozása előtt ellenőrizzük, hogy a pufferben elegendő bájt áll-e rendelkezésre, és a `len` mezőt a tényleges `netbuf->len` értékhez viszonyítjuk.

```
if (buf->len < sizeof(struct eth_hdr))
    goto drop;

struct eth_hdr *eth = (struct eth_hdr *)buf->data;
if (eth->ethertype != ETH_P_IP)
    goto drop;

if (buf->len < sizeof(struct eth_hdr) + sizeof(struct ip_hdr))
    goto drop;

struct ip_hdr *ip =
    (struct ip_hdr *)(buf->data + sizeof(struct eth_hdr));

if (ip->ihl < 5 || ip->ihl > 15)
    goto drop;
if (ntohs(ip->total_len) > buf->len - sizeof(struct eth_hdr))
    goto drop;
```

A `drop` címke a csomag eldobását és a számláló növelését jelenti. Hasonló ellenőrzés szükséges az UDP-fejléc hosszára is. A parszert úgy kell megírni, hogy **minden bemeneti mezőt bizalmatlanként kezel**, és az aritmetikai műveleteknél figyel az integer-túlcsordulásra. A `ntohs` és `ntohl` függvényeket a bájtsorrend miatt használjuk; a minimális kernelben ezek egyszerű makróként implementálhatók.

31.2.8 6.2. Rendszerhívás-paraméterek ellenőrzése

A `sendto` és `recvfrom` hívásoknál a felhasználói `len` paraméter akár óriási értéket is kaphat, ami a kernel `copy_from_user` során memória-túllépést okozhat. **Minden méretet korlátozni kell a maximális UDP-csomagméretre**, és a felhasználói mutatót a `copy_from_user` előtt a címtartomány ellenőrzésével kell validálni. A `dest` és `src` mutatók esetében a `socklen_t` hossz nem haladhatja meg a `struct sockaddr_in` méretét, és a családmezőnek `AF_INET`-nek kell lennie.

Tipikus hiba, hogy a kernel a `len` paramétert közvetlenül használja `memcpy`-hez anélkül, hogy a kapacitást ellenőrizte volna. A biztonságos kód mindig a célpuffer tényleges méretét használja, és a hosszmezőt csak a másolási művelet felső korlátjaként kezeli.

31.2.9 6.3. Erőforrás-kimerítés és izoláció

A hálózati fogadási oldalon a bejövő csomagok száma nem áll a kernel ellenőrzése alatt. Ha a feldolgozási sor korlátlanul nő, a támadó memóriatúlcsordulást idézhet elő. A `netif_rx` függvénynek **fix hosszúságú sort** kell használnia, és a sor megtelésekor a csomagot el kell dobni. A dobás tényét számlálni kell, és a naplóban megjeleníteni.

A socket fogadási soraira ugyanez vonatkozik: minden sockethez tartozzon egy maximális sorhossz, például `SOCK_RX_QLEN = 64`. Ha a folyamat nem olvassa időben a bejövő csomagokat, a sor megtelik, és a további csomagokat eldobjuk. Ez a viselkedés a felhasználó számára csomagvesztésként jelentkezik, ami az UDP esetében elfogadható.

A folyamatok közötti izolációt a biztonsági modul biztosítja: a policy-ellenőrzés megakadályozza, hogy egy folyamat a neki nem engedélyezett hálózati erőforrásokhoz férjen. A hook-rendszernek a folyamat kontextusát is ismernie kell; a minimális kernelben ezt a `current` processz-struktúra elérésével érjük el.

31.2.10 6.4. Versenyhelyzetek a hálózati sorban

A hálózati verem több kontextusból is hozzáférhet a csomagsorokhoz: megszakításból, kernelszálból és rendszerhívásból. A zárolás hiánya versenyhelyzetet okozhat, például a sor elemeinek elvesztését vagy a dupla beszúrást. A minimális kernelben a sorokhoz **spinlock** vagy **mutex** tartozik, attól függően, hogy a hozzáférés megszakításkontextusban is megtörténhet-e. Mivel a `netif_rx` megszakításból hívódik, a hozzáférés során a megszakítást le kell tiltani, vagy spinlock-irqsave mechanizmust kell használni. A `socket` fogadási sorához elegendő egy mutex, mert azt csak rendszerhívás-kontextusból módosítjuk.

A hibakeresés során a versenyhelyzeteket gyakran csak stressztesztel lehet felfedni. A laborban a fogadási oldalt párhuzamos küldőkkel és egyidejű olvasóval kell terhelni, hogy a zárolási hiányosságok láthatóvá váljanak.

31.2.11 7. Labor: hálózati stack és LSM-bővítés terve

A fejezet laborja a hálózati verem és a biztonsági modul **tervezésére** és egy **emulált ellenőrző környezetben** való validálására összpontosít. A cél nem a teljes implementáció, hanem a döntési pontok, az interfészek és a biztonsági követelmények kidolgozása, majd egy QEMU-alapú teszttel a kritikus útvonal igazolása.

31.2.12 7.1. Laborfeladat

Készítsünk tervet egy minimális UDP-hálózati veremhez és egy LSM-modulhoz, amely a következő policy-t érvényesíti:

- A felhasználói folyamatok kizárólag a loopback interfészre (`127.0.0.1`) és a `8080`-as UDP-portra küldhetnek.
- Minden más célcím és port `-EPERM` hibát eredményez.

- A bejövő csomagokat a kernel feldolgozza, de csak a loopbackről érkező, a `8080`-as portra címzett csomagokat továbbítja a socketekhez.

A tervnek tartalmaznia kell:

1. A rétegek és interfészek definícióját (`net_device`, `netbuf`, socket, LSM-hook).
2. A rendszerhívások pontos szignatúráját és hibakódjait.
3. A hálózati csomag feldolgozási útvonalát: megszakítás → `netif_rx` → protokoll → socket.
4. A biztonsági hook-ok elhelyezését és a policy döntési logikáját.
5. A tipikus hibák listáját és az ellenőrzési pontokat.

A tervet **reprodukálható jegyzetfüzetben** (például Markdown vagy Jupyter) kell dokumentálni, a QEMU-konfigurációval, a build-paranccsal és a mérési naplóval együtt.

31.2.13 7.2. Emulált környezet konfigurációja

A laborhoz QEMU-t használunk, a következő parancssorral indítva egy minimális x86-64-es kernelt:

```
qemu-system-x86_64 \
  -kernel build/kernel.bin \
  -netdev user,id=n0,hostfwd=tcp::5555-:8080 \
  -device virtio-net-pci,netdev=n0 \
  -nographic
```

A `-netdev user` egy felhasználói hálózati háttér, amely a hoszton a `5555`-ös TCP-portot a vendég `8080`-as portjára irányítja. A `virtio-net-pci` eszköz a vendégben egy virtio hálózati kártyát emulál. A kernel build-parancsa a korábbi fejezetekben leírt makefile-t használja; a laborban a `make kernel.bin` parancsot kell megadni, és a fordítási naplót mellékelni.

A hálózati teszt elvégzéséhez a minimális kernelben egy egyszerű felhasználói programot futtatunk, amely a `socket`, `bind`, `sendto` és `recvfrom` rendszerhívásokat használja. A program a loopback címre küld egy ismert tartalmú UDP-csomagot, majd megpróbál egy tiltott célcímre is küldeni. A várt eredmény:

- A loopback küldés sikerül, a `sendto` visszatérési értéke a küldött bájtok száma.
- A `recvfrom` a fogadó oldalon a csomagot és a forráscímet adja vissza.
- A tiltott célcímű `sendto` `-1` értékkel tér vissza, és az `errno` `EPERM`.

31.2.14 7.3. Mérési napló

A mérési naplóban a következő adatokat kell rögzíteni:

- **Környezet**: QEMU verzió, vendég architektúra, CPU-modell, memória, kernel verzió (a saját build hash-e).
- **Mért eredmények**:
  - A `sendto` rendszerhívás időtartama loopback esetén (ciklusszám vagy időbélyeg).
  - A `sendto` elutasításának időtartama `-EPERM` esetén.
  - A csomagfeldolgozás késleltetése a megszakítástól a socket sorba kerülésig.
  - A csomagvesztési arány 1000 csomag küldésekor.
- **Várt (levezetett) eredmények**:

  - A rendszerhívás időtartama a `copy_from_user` és a protokollfeldolgozás összegének nagyságrendjébe esik.
  - Az elutasítás lényegesen gyorsabb, mint a tényleges küldés, mert a hardveres művelet elmarad.
  - A csomagfeldolgozás késleltetése nem haladja meg a konfigurált `NET_RX_DELAY` értéket.

A mért és a várt értékeket **külön oszlopban** kell feltüntetni, és a mérési módszert is le kell írni (például a TSC-regiszter olvasása a belépéskor és kilépéskor). A mérést legalább 10-szer meg kell ismételni, és az átlagot, a minimumot és a maximumot is rögzíteni kell. A mérési eredményeket nem szabad a tényleges futás nélkül, becslésként bemutatni; minden számnak a laborban futtatott kódból kell származnia.

31.2.15 7.4. Értékelési szempontok

A labor akkor tekinthető teljesítettnek, ha:

- A tervdokumentum tartalmazza az összes előírt interfészt és a policy logikáját.
- A QEMU-ban futó tesztprogram a várt eredményeket adja (sikeres loopback küldés, `-EPERM` a tiltott címre).
- A mérési napló elkülöníti a mért és a várt értékeket, és tartalmazza a reprodukciós lépéseket.
- A dokumentum azonosít legalább három tipikus hibát, és bemutatja az elkerülésükre tett intézkedést.

31.2.16 8. Önellenőrző kérdések

A fejezet tanulási céljainak ellenőrzésére az alábbi kérdések szolgálnak:

1. Miért fontos a hálózati verem rétegzése a minimális OS-ben? Hogyan segíti ez a driver- és policy-bővítést?
2. Hová kell helyezni az LSM-hook-ot a `sendto` rendszerhívásban, és miért nem elég csak a socket létrehozásakor ellenőrizni?
3. Milyen támadási felületet jelent a hálózati csomagparser, és milyen konkrét ellenőrzésekkel csökkenthető a kockázat?
4. Mi a teendő, ha a `netif_rx` fogadási sora megtelik? Miért nem engedhető meg a korlátlan növekedés?
5. A labor konfigurációjában a `hostfwd=tcp::5555-:8080` beállítás mit csinál, és hogyan befolyásolja a tesztelést?

A kérdések megválaszolásához a fejezetben bemutatott kód és a laborleírás elegendő. A válaszoknak hivatkozniuk kell a konkrét ellenőrzési pontokra és a biztonsági követelményekre.

31.2.17 9. Összegzés

A minimális operációs rendszer hálózati bővítése akkor valósul meg helyesen, ha a driver, a csomagkezelés, a socket API és a biztonsági modulok világosan elválasztott rétegekben működnek. A hálózati verem UDP-szintű megvalósítása elegendő a belső kommunikációhoz és a biztonsági policy demonstrálásához. Az LSM-szerű hook-rendszer lehetővé teszi, hogy a hálózati hozzáférést központilag, a kernel kódjának módosítása nélkül szabályozzuk. A támadási felület csökkentése érdekében minden bejövő adatot ellenőrizni kell, a méreteket korlátozni, a sorokat fix hosszúságúra állítani, és a versenyhelyzeteket zárolással kezelni.

A laborban a hallgató a fejezetben bemutatott eszközöket egy konkrét, policy-vezérelt hálózati terv elkészítésére és emulált validálására használja. A mért és várt eredmények elkülönítése, valamint a reprodukciós csomag összeállítása biztosítja, hogy az eredmények ellenőrizhetők és megismételhetők legyenek. A következő fejezet (Virtualizáció és konténerintegráció) erre a hálózati és biztonsági alapra építve a virtualizáció és a konténerintegráció kérdéseit tárgyalja, ahol a izoláció és az erőforrás-kezelés még hangsúlyosabb szerepet kap.

31.3 Virtualizáció és konténerintegráció

Amikor a saját operációs rendszer eljut abba az állapotba, hogy a kernel, az init és az alapszolgáltatások együtt működnek, a fejlesztés következő szakasza a futtatási környezetek tudatos integrációja. A virtuális gép az elsődleges futtatási cél, mert saját kernelt, saját megszakításkezelőt és saját eszközmeghajtókat tudunk benne futtatni anélkül, hogy a hoszt rendszerpartícióját módosítanánk. A konténer ezzel szemben a hoszt kernelét használja, ezért közvetlenül nem képes a saját kernel futtatására; mégis kulcsszerepet kap a build, a felügyelet, a tesztelés és a hordozható userlandprogramok ellenőrzésében. A két technológia együttes használata reprodukálható fejlesztési ciklust ad, és előkészíti a dokumentációs, portolási és karbantartási munkát.

1. Saját kernel virtuális gépen

A virtuális gépes futtatás leggyakoribb eszköze a QEMU, amely szoftveres (TCG) és hardveres (KVM) gyorsítást is kínál. A TCG akkor fontos, ha a hoszt és a cél architektúra eltér; teljesítménymméréshez azonban a KVM-et kell használni, ha elérhető, mert a TCG utasításonkénti emulációja nagyságrendi lassulást okozhat. A következő indítási parancs egy emulált x86-64-es gépet hoz létre soros konzollal, virtio blokk- és hálózati eszközzel:

```
qemu-system-x86_64 \
  -machine q35,accel=kvm:tcg \
  -cpu host \
  -m 256M \
  -smp 2 \
  -kernel build/minios.elf \
  -initrd build/initramfs.cpio.gz \
  -append "console=ttyS0 quiet" \
  -drive if=virtio,file=minios.img,format=raw \
  -netdev user,id=n0 \
  -device virtio-net-pci,netdev=n0 \
  -nographic
```

A kapcsolók jelentését érdemes pontosan rögzíteni. A `-machine q35,accel=kvm:tcg` a Q35 gépmodellt választja, és először KVM-et, majd annak hiányában TCG-t próbál indítani. A `-cpu host` a hoszt processzor képességeit adja át, ami KVM alatt a legjobb teljesítményt és a natív utasításkészlet használatát jelenti. A `-m 256M` a vendég fizikai memóriát, a `-smp 2` a virtuális CPU-k számát adja meg. A `-kernel build/minios.elf` a saját kernel ELF fájlját tölti be közvetlenül, bootloader nélkül; ez a fejlesztés korai szakaszában egyszerűbb, mint egy teljes boot-folyamatot szimulálni. A `-initrd` a kezdeti RAM-lemezt adja át, az `-append` pedig a kernel parancssort. A `-nographic` a soros konzolt a standard kimenetre köti, ezért a kernelnapló terminálból olvasható.

A saját kernel szempontjából a QEMU `-kernel` útján történő indítás egy belépési szerződést feltételez: a kernel tudja, hogy a vezérlés milyen regiszterállapottal, milyen memóriatérképpel és milyen parancssori átadással érkezik. Ezt a szerződést a projekt kernele definiálja, és a buildnek be kell tartania. A QEMU parancssort éppen ezért a buildrendszerbe kell illeszteni, nem elegendő egy terminál-előzményben hagyni. A reprodukciós csomag része a QEMU verziója, a

parancssori kapcsolók, a vendég CPU és memória konfiguráció, valamint a kernel és az initramfs azonosítója.

A boot szakaszokat a kernelnaplóban is érdemes jól elkülöníteni. Első lépés a soros port inicializálása, hogy a legkorábbi hibaüzenet is látszódjon. Ezt követi a parancssor feldolgozása, az initrd beolvasása, a virtuális eszközök detektálása, majd az init folyamat elindítása. Ha a soros konzol nincs bekapcsolva, a kernel pánikja láthatatlan marad, és a hibakeresés sokkal nehezebbé válik.

2. Virtio: a VM I/O-útja

A teljesen emulált I/O-eszközök minden regiszterhozzáférést szoftveresen szimulálnak, ami lassú, és eltakarja a valódi eszköz viselkedését. A virtio ezzel szemben paravirtualizált eszközcsalád: a vendég kernel tudja, hogy virtualizált környezetben fut, és a hoszttal megosztott memórián, gyűrűpuffereken keresztül kommunikál. A QEMU és más hypervisorok virtio-blk, virtio-net, virtio-console, virtio-rng és további eszközöket kínálnak. A saját kernel számára ez azt jelenti, hogy a blokk- és hálózati eszközök nem a valódi hardver regisztermodelljét, hanem egy szabványos paravirtualizált ABI-t követnek.

A virtio központi fogalma a **virtqueue**. A vendég a normál memóriában lefoglal három területet: a descriptor, az available és a used gyűrűt. A descriptor bejegyzések a pufferek fizikai címét, hosszát és irányát írják le. A vendég kitölti a descriptort, az available gyűrűbe írja a descriptor indexét, majd megszakítással vagy külön regiszterírással értesíti a hosztot. A hoszt feldolgozza a kérést, a used gyűrűbe visszaír, és megszakítást küld a vendégnek. Ez a modell lehetővé teszi a kötegelt kéréseket, és drámaian csökkenti a regiszterenkénti emulációt.

A vendég oldali minimális virtio-blk driver a következő lépésekből áll:

1. a szállítási réteg felismerése: PCI vagy MMIO;
2. az eszközstátusz léptetése: `RESET`, `ACKNOWLEDGE`, `DRIVER`, `FEATURES_OK`, `DRIVER_OK`;
3. a feature-egyeztetés: a vendég beolvassa a hoszt által kínált képességeket, és csak a támogatottakat kapcsolja be;
4. a virtqueue méretének és memóriacímének beállítása;
5. az eszközspecifikus blokkkérések kiadása.

Egy leegyszerűsített MMIO-váz pszeudokódban:

```
volatile struct virtio_mmio_regs *mmio = (volatile void *)VIRTIO_MMIO_BASE;

mmio->status = VIRTIO_STATUS_ACKNOWLEDGE;
mmio->status |= VIRTIO_STATUS_DRIVER;

uint32_t features = mmio->host_features;
features &= (1u << VIRTIO_BLK_F_SEG_MAX);
mmio->guest_features = features;

mmio->queue_sel = 0;
uint32_t qmax = mmio->queue_num_max;
mmio->queue_num = qmax;
mmio->queue_desc = virt_to_phys(desc);
mmio->queue_avail = virt_to_phys(avail);
mmio->queue_used = virt_to_phys(used);

mmio->status |= VIRTIO_STATUS_FEATURES_OK;
mmio->status |= VIRTIO_STATUS_DRIVER_OK;
```

A kód szemléltetés; a konkrét regiszternevek és feature-bitek a kiválasztott virtio specifikációs verziótól függenek, ezért a projektben azt a változatot kell rögzíteni. A lényeg, hogy a kernel a

virtuális eszközt ugyanúgy kezelje, mint egy normál blokk- vagy hálózati eszközt, és a szállítási réteg izolálja a PCI/MMIO hozzáférést.

Ha a QEMU parancsban `-drive if=virtio` szerepel, de a kernelben nincs virtio-blk driver, a boot nem feltétlenül áll le azonnal. A kernel egyszerűen nem találja a gyökérfájlrendszer eszközét, és később pánikba fut. Ezért a boot logban a `virtio` eszköz detektálását külön ellenőrizni kell, és a hiányzó eszközt nem szabad a fájlrendszer hibájának tulajdonítani.

3. Init és image

A virtuális gépen a kernel betöltése után az első felhasználói folyamat az `init`. A saját operációs rendszerben ez lehet statikusan linkelt program vagy kezdeti szkript. A gyakorlatban a kezdeti RAM-lemez (initramfs vagy initrd) tartalmazza az indításhoz szükséges fájlokat. A Linux-szerű kerneleknél az initrd-t gyakran `newc` cpio archívumként, gzip tömörítéssel készítik; a saját kernelünknek ismernie kell ezt a formátumot, vagy saját, egyszerűbb formátumot kell definiálnia.

Az initramfs build lépései:

```
mkdir -p initramfs/{bin,dev,proc,sys}
cp build/init initramfs/
cp build/sh initramfs/bin/
cd initramfs
find . -print0 | cpio --null -o --format=newc > ../initramfs.cpio
gzip ../initramfs.cpio
```

A tartalomban a `/init` a belépési pont. Egy lehetséges `/init` szkript:

```
#!/init/sh
mount -t proc proc /proc
mount -t sysfs sys /sys
mount -t devtmpfs none /dev
/bin/sh
```

A `mount` parancs természetesen a saját kernel megfelelő rendszerhívásait használja. Az init feladatai: a virtuális fájlrendszerek csatolása, az eszközcsomópontok létrehozása, a konzol megnyitása, majd egy interaktív shell vagy a tényleges szolgáltatás elindítása. Az init folyamatot akkor is be kell tölteni, ha csak egyetlen tesztprogramot futtatunk; enélkül a kernelnek nincs kiadható felhasználói feladata.

Az **image** kifejezés kettős jelentésű. A VM esetén a kernel image (például ELF) és a rootfs image (például initramfs) külön fájl; a konténer-világban az image rétegzett fájlrendszer-kép, amelyből a konténer rootfs-e jön létre. A kettő közötti kapcsolat akkor válik érdekessé, amikor a saját operációs rendszer userlandjét szeretnénk konténerben tesztelni. Ilyenkor az initramfs tartalmát OCI-kompatibilis image-be is exportálhatjuk, de a kernel soha nem kerül az OCI image-be, mert a konténer a hoszt kernelt használja.

A reprodukálható image-készítéshez a fájlrendszerfában lévő fájlok időbélyegét, tulajdonosát és módját is rögzíteni kell. A `cpio` és a `tar` is támogatja a determinisztikus előállítást, ha megfelelően hívjuk. Ez nem kényelmi kérdés: a mérési naplóban csak az image azonosítóval összekötött eredmény értelmezhető, és a későbbi regressziós teszt csak akkor ismételhető, ha az image bitről bitre ugyanaz.

4. Konténerintegráció

A konténer a hoszt kernelét használja, ezért a saját kernel közvetlenül konténerben nem indítható el. A konténerintegráció mégis három fontos szerepet tölt be.

Az első a **fejlesztői és buildkörnyezet**. A cross-fordítót, QEMU-t, GDB-t, cpio-t és a teszteket egy konténerben rögzíthetjük, így a fejlesztői gép állapotától függetlenül reprodukálható a build. Egy lehetséges Dockerfile-váz:

```
FROM debian:stable-slim
RUN apt-get update && apt-get install -y --no-install-recommends \
    gcc make qemu-system-x86 cpio gzip
COPY . /src
WORKDIR /src
CMD ["make", "test-vm"]
```

A példa nem futtatható éles környezetben; a `debian:stable-slim` címke és a csomagnevek a kiadási ciklustól függenek, ezért a projektben rögzített image-hashelés vagy pontos verzió szükséges.

A második szerep a **felügyeleti és tesztelési konténer**. A konténer elindíthatja a QEMU-t, a soros konzolt naplózhatja, időbélyegeket gyűjthet, és a futást jelző metrikákat tárolhatja. A hoszt QEMU és KVM hozzáférése nem igényli a konténer `--privileged` módját, ha a `/dev/kvm` eszközt és a munkakönyvtárat célzottan adjuk át:

```
docker run --rm -it \
  --device /dev/kvm \
  -v "$PWD":/work \
  myos-lab:0.1 \
  qemu-system-x86_64 -m 256M -kernel /work/build/minios.elf \
  -initrd /work/build/initramfs.cpio.gz -nographic
```

A harmadik szerep a **userland kompatibilitási teszt**. Ha a saját operációs rendszer userlandje hordozható C API-t és Linux syscall-réteget is támogat, akkor ugyanaz a program a saját kernel alatt és Linux konténerben is futtatható. Ezzel a kernelkomponensek hatása elválasztható a programhibáktól: ha a program konténerben helyes, a saját kernel alatt viszont hibás, a hibát nagy valószínűséggel a saját syscall-rétegben vagy a kernel erőforrás-kezelésében kell keresni.

A konténeres munkánál a biztonsági alapelv: a lehető legkevesebb jogot adjunk. A `--privileged` gyakran kényelmes, de a QEMU indításához nem szükséges; a `/dev/kvm` és a munkakönyvtár megosztása elegendő. Emellett a konténerben futó emulátor nem férhet a hoszt lemezéhez, kivéve a kifejezetten csatolt köteteket.

5. Kompatibilitás és teljesítmény

A virtuális gépes futtatás kompatibilitása három rétegen múlik: a kernel boot-protokollja, a kezdeti RAM-lemez formátuma és az eszközök ABI-ja. A saját kernel akkor hordozható QEMU és később valós hardver között, ha ezeket a határfelületeket dokumentáljuk és verziózzuk. A parancssor átadása, az initrd kezdőcíme, a soros konzol regiszterei és a kernel belépési állapota mind a boot-szerződés részei. A laborban ezt a szerződést a fejlesztő saját forrása definiálja; a QEMU parancssor ehhez igazodik.

A virtio eszközöknél a **feature negotiation** a kompatibilitás kulcsa. A guest és a host közösen egyeznek meg abban, hogy mely képességeket használják. Ha a vendég olyan képességet hagy bekapcsolva, amelyet nem ismer, a hoszt viselkedése specifikációellenes lehet; ha viszont egy szükséges képességet kihagy, a teljesítmény romlik vagy az eszköz nem működik. A saját drivert ezért a specifikáció egy rögzített verziója ellen kell fejleszteni, és a feature-biteket expliciten kell maszkolni.

A teljesítmény mérésénél a virtuális gépes és a konténeres összehasonlítás csak akkor értelmes, ha a vizsgált réteg azonos. Például a userland compute-feladat konténerben közel natív, VM-ben a virtuális CPU és a memóriavirtualizáció miatt kissé lassabb lehet; de a kernel saját ütemezőjének vagy az ütemezési döntések mérése VM-ben zajlik. Az I/O-teljesítményben a virtio csökkenti

az emulációs költséget, de a hoszt ütemezője és a virtuális megszakítások miatt továbbra is van többlet.

A mérési jegyzőkönyvnek tartalmaznia kell a hardvert, a QEMU verziót, a KVM/TCG módot, a CPU-pinninget, a memória méretét, a vendég kernel build beállításait és a mérési módszert. Egyetlen szám, például „a boot 1,2 másodperc”, nem elegendő. A boot időt a kernelnapló kezdetétől az init promptig mérjük; a rendszerhívás-késleltetést ciklusszámlálóval vagy nagy számú hívás átlagával; a blokk I/O-t legalább IOPS, átvitel és farok-késleltetés szerint. A mért és a várt értéket külön kell tárolni: a várt érték lehet becslés vagy korábbi mérés, a mért érték pedig a reprodukálható labor eredménye.

Példa mérési napló szerkezete:

Mérés	Várt	Mért	Feltétel
Boot promptig	<= 2,0 s	1,8 s	QEMU 8.2, KVM, 2 vCPU
virtio-blk felismerés	<= 100 ms	42 ms	cold boot, soros log
`write` syscall 1000 hívás	1,0–2,0 ms	1,4 ms	vendég kernel, 2 GHz vCPU

A konkrét számok a labortól függenek; a táblázat itt az elvándorolandó formát mutatja, nem hiteles mérési eredményt ad.

6. Tipikus hibák és helyes megoldások

Kernel pánik: „No working init found”. Nincs `/init` az initramfsben, vagy a cpio formátum hibás. Megoldás: a `cpio -it < initramfs.cpio` kimenetét ellenőrizni, és biztosítani, hogy a `/init` futtatható.

Nincs soros konzol log. A kernel nem a `ttyS0` regisztercímre ír, vagy a QEMU `-nographic` hiányzik. Megoldás: a legkorábbi kernelüzenetet már a soros porton kiírni, és a QEMU-t `-nographic` móddal indítani.

A vendég nem látja a lemezt. A QEMU parancsban `-drive if=virtio` van, de a kernelben nincs virtio-blk driver. Megoldás: először MMIO-vázat írni, majd a feature-egyeztetést és a queue inicializálást naplózni.

A konténer QEMU-ja lassú. A `/dev/kvm` nincs átadva, így a QEMU TCG módra esik vissza. Megoldás: `--device /dev/kvm` és annak ellenőrzése, hogy a hoszt KVM-modulja aktív.

Nem reprodukálható image vagy parancs. A build nem rögzíti a verziókat és az időbélyegeket. Megoldás: a buildrendszer generáljon image-azonosítót, például hash-t, és a napló ezt rögzítse.

Túlzott jogosultság a konténerben. A `--privileged` használata helyett csak a szükséges eszközöket és köteteket adjuk át; így a labor nem tudja véletlenül módosítani a hoszt firmware-ét vagy lemezét.

7. Labor: virtualizációs és konténeres teszterv

A labor célja, hogy a saját kernel virtuális gépen elinduljon, a virtio-blk eszközt felismerje, és ugyanez a folyamat konténerből, felügyeleti naplóval reprodukálható legyen. A labor kizárólag emulált környezetben, eldobható munkakönyvtárban fut; a hoszt rendszerpartícióját nem módosítjuk.

Kiinduló állapot: az előző fejezetekben elkészített `build/minios.elf` kernel, egy init program és egy minimális shell. A buildrendszer előállítja az `initramfs.cpio.gz` fájlt.

Rögzített konfiguráció: QEMU 8.2 (ellenőrizendő: a projekt által elfogadott verzió), KVM engedélyezve, 2 vCPU, 256 MiB memória, `virtio-blk-pci` eszköz. A konténer image `debian:stable-slim` alapú, a build során rögzített hashsel.

Lépések:

1. Készítsük el az initramfs-t a fenti `cpio` paranccsal, és rögzítsük az sha256 hashét.
2. Indítsuk el a VM-et a megadott QEMU paranccsal, a kimenetet irányítsuk `boot1.log` fájlba.
3. Ellenőrizzük a logban a kernel belépési üzenetet, a virtio-blk felismerést és az init promptot. Jegyezzük fel az időbélyegeket.
4. Ismételjük meg a bootot konténerből, `--device /dev/kvm` átadással, és mentsük a `boot2.log` fájlt.
5. Töltsük ki a mérési naplót: az elvárt és a mért értékek külön oszlopba kerülnek, a feltételekkel együtt.
6. Ellenőrizzük a visszaállítást: a konténer leállítása és törlése, a munkakönyvtár változatlansága.

Elvárt eredmény: a kernel a boot után legfeljebb két másodpercen belül promptot ad; a `virtio-blk` eszköz detektálása a logban megjelenik; a konténerből indított VM logja azonos boot-szekvenciát mutat a VM-ből indított loggal, kivéve a hoszt-időbélyegeket.

Mérési napló: a naplóban rögzíteni kell a dátumot, a kernel commit hashét, a QEMU és a konténer image verzióit, a parancsokat, a logfájlokat és a mért időket. A várt és mért értékeket el kell különíteni, és a mérési körülményeket is le kell írni. A nem mért értékhez a „nem mért” jelölés tartozik, nem szabad becslést mért eredményként feltüntetni.

Önellenőrző kérdések:

1. Miért nem futtatható a saját kernel közvetlenül Linux konténerben?
2. Melyik virtio komponens tárolja a vendég által feladott kéréseket a feldolgozás előtt?
3. Mi történik, ha az initramfs-ben nincs `/init`?
4. Hogyan csökkenthető a konténeres QEMU jogosultsági igénye anélkül, hogy a KVM-et elveszítenénk?

A válaszok a fejezetben megtalálhatók: a konténer a hoszt kernelt használja; a virtqueue available gyűrűje; az init hiányában a kernel nem tud felhasználói folyamatot indítani; a `/dev/kvm` célzott átadásával a `--privileged` helyett.

Ez a teszterv a következő, dokumentációs és portolási fejezetben a reprodukciós csomag részévé válik: ugyanezt a parancs- és naplóstruktúrát kell használni ahhoz, hogy egy új port vagy egy új kernelverzió kiadási tesztje elfogadható legyen. Az itteni mérési fegyelem tehát nem önálló labor, hanem a későbbi karbantartási ciklus alapja.

31.4 Dokumentáció, portolás és karbantartás

A hoszt operációs rendszer hosszú távú fejlesztése és karbantartása során az egyik legnagyobb kihívás nem egyetlen funkció megírása, hanem az, hogy a kód évek múlva is érthető, módosítható és más hardveren is futtatható maradjon. Ez a fejezet a platformfüggetlenséget és a dokumentációs kultúrát helyezi középpontba, mert ezek nélkül a kernel – vagy bármely rendszerközeli szoftver – a kezdeti sikerek után gyorsan technikai adóssággá válik. A téma szorosan kapcsolódik az

SMP-hez, a preemptionhöz és a biztonsági modulokhoz: mindegyik olyan bővítés, amely csak akkor marad karbantartható, ha a hardverfüggő és a hardverfüggetlen kód világosan el van választva, és a build, tesztelés, valamint a verziózás folyamata reprodukálható.

A cél nem csupán az elméleti alapelvek ismertetése, hanem egy olyan konkrét portolási és dokumentációs terv felépítése, amelyet egy kis kernelprojektben azonnal alkalmazni lehet. A fejezet végén található labor ezt a folyamatot emulált környezetben gyakoroltatja be, hogy a mért és a várt eredmények különválasztása, valamint a reprodukciós csomag összeállítása ne maradjon utólagos feladat.

31.4.1 1. A hosszú távú fejlesztés és a platformfüggetlenség

Egy operációs rendszer kernelét ritkán egyetlen hardverre tervezik. Még ha az első célplatform mondjuk egy adott ARM System-on-Chip (SoC), a kód szerkezetének már az első naptól kezdve támogatnia kell azt, hogy később újabb architektúrák, újraindulási módok vagy más megszakításvezérlők jelennek meg. A platformfüggetlenség nem azt jelenti, hogy a kernel minden hardvert egyformán kezel – ez fizikailag lehetetlen –, hanem azt, hogy a hardverfüggő tudás lokalizált, jól dokumentált interfészek mögé van zárva.

A hosszú távú fejlesztés szempontjából három erő hat:

1. **Technikai evolúció**: új processzor-utasításkészletek, megszakításvezérlők, időzítők és perifériák jelennek meg. A kernelnek ezeket úgy kell befogadnia, hogy a meglévő, hardverfüggetlen logika ne változzon.
2. **Emberi tényező**: a fejlesztők cserélődnek, a tudás elhalványul. Csak az a kód karbantartható, amely mellett a dokumentáció és a tesztek ugyanolyan elsőrendű állampolgárok, mint maga a forráskód.
3. **Kiadási ciklusok**: a hibajavítások, biztonsági frissítések és új funkciók párhuzamosan futnak. A verziózásnak és a release-folyamatnak támogatnia kell a stabil ágak fenntartását anélkül, hogy a fejlesztési ág elakadna.

A platformfüggetlenség elérésének legismertebb eszköze a **hardverabsztrakciós réteg (HAL)**. A HAL egy olyan interfészcsomag, amely a hardverfüggő műveleteket – például időzítő programozása, megszakítás maszkolása, MMU beállítása – egységes hívási felületen keresztül teszi elérhetővé a kernel architektúra-független része számára. A HAL mögött architektúra-specifikus implementációk helyezkednek el, amelyeket külön könyvtárakban tartunk.

A HAL mellett a platformfüggetlenséget szolgálja a **függőségi irány elve** is: a generikus kernelmag soha nem include-ol közvetlenül architektúra-specifikus fejlécet, és nem hivatkozik konkrét hardvercímekre. Minden hardverfüggő művelet egy függvénytáblán vagy gyenge szimbólumokon keresztül érhető el. Ez a minta teszi lehetővé, hogy ugyanaz a generikus ütemező forduljon x86_64, arm64 és RISC-V platformra is.

A hosszú távú fejlesztést emellett a **dokumentáció mint kód** szemlélet támogatja: a README, a portolási útmutató, az API-referencia és a build mátrix a repository része, és ugyanúgy verziózva van, mint a forráskód. Ha egy interfész megváltozik, a dokumentációt ugyanabban a commitban kell frissíteni – ellenkező esetben a dokumentáció hamis képet ad, ami gyakran rosszabb, mint a hiánya.

31.4.2 2. HAL, architektúra és build matrix

A hardverabsztrakciós réteg tervezésekor az első lépés az architektúra-függő és architektúra-független részek szétválasztása. Gyakorlati szempontból a következőképpen járunk el:

1. **Interfész definiálása**: meghatározzuk azokat a műveleteket, amelyeket a generikus kernel a hardvertől igényel. Ezeket egy közös fejlécben, például `include/hal.h` fájlban rögzítjük.
2. **Implementáció architektúránként**: minden támogatott architektúrához létrehozunk egy `arch/<arch>/` könyvtárat, amely a HAL interfész saját megvalósítását tartalmazza.
3. **Kötés és konfiguráció**: a build rendszer a célarchitektúra alapján kiválasztja a megfelelő implementációt, és a generikus kernellel együtt fordítja le.

A HAL interfész tipikus elemei:

- **Időzítő és órajelek**: inicializálás adott frekvencián, számláló olvasása, egyszeri vagy periodikus megszakítás beállítása.
- **Megszakításkezelés**: megszakítás engedélyezése/tiltása, prioritások beállítása, megszakításvektor regisztrálása.
- **Memóriakezelés**: MMU/TLB inicializálás, lapozási struktúrák létrehozása, fizikai cím fordítás.
- **Barrierek és atomi műveletek**: memóriabarrierek, atómi olvasás/írás, spinlock primitívek.
- **Konzol és soros port**: karakteres kimenet bemenet nélkül, hangolás.

Az alábbi kód egy leegyszerűsített HAL interfész C nyelvű deklarációját mutatja:

```
/* include/hal.h - hardverabsztrakciós interfész (szemléltető példa) */
#ifndef HAL_H
#define HAL_H

#include <stdint.h>
#include <stddef.h>

/* Fizikai és virtuális címek típusai; méretük architektúrafüggő. */
typedef uint64_t phys_addr_t;
typedef uint64_t virt_addr_t;

/* Időzítő műveletek. */
struct hal_timer_ops {
    int  (*init)(uint32_t freq_hz);
    uint64_t (*read_counter)(void);
    int  (*set_oneshot)(uint64_t deadline_ns);
};

/* Megszakítás-kezelő műveletek. */
struct hal_irq_ops {
    void (*enable)(unsigned irq);
    void (*disable)(unsigned irq);
    int  (*register_handler)(unsigned irq, void (*handler)(void));
};

/* MMU műveletek. A lapozási struktúra architektúra-specifikus. */
struct hal_mmu_ops {
    int  (*init)(void);
    int  (*map_page)(virt_addr_t vaddr, phys_addr_t paddr, unsigned flags);
    void (*enable)(void);
};

/* A HAL teljes interfésze. */
```

```
struct hal_ops {
    const struct hal_timer_ops *timer;
    const struct hal_irq_ops   *irq;
    const struct hal_mmu_ops   *mmu;
    /* Konzol kimenet. */
    void (*putchar)(char c);
};

/* A generikus kernel a boot során kap egy kitöltött HAL példányt. */
extern const struct hal_ops *hal;

#endif /* HAL_H */
```

A példa szemlélteti a függőségi irányt: a generikus kernel csak a `hal_ops` mutatót ismeri, és nem törődik azzal, hogy az adott architektúrán hogyan valósul meg az időzítő vagy az MMU. Ugyanez a minta használható eszközmeghajtókra is: a driver egy eszközosztály-interfészt implementál, a kernel pedig azon keresztül kommunikál.

31.4.3 Architektúra-specifikus könyvtárstruktúra

A forráskód szervezése akkor hatékony, ha az architektúra-specifikus részek külön könyvtárban, szimmetrikus elrendezésben élnek. Egy lehetséges struktúra:

```
mini-os/
  include/
      kernel/          # generikus kernel API
      hal.h            # hardverabsztrakciós interfész
  core/                 # arch-független kernelmag
      sched.c
      syscall.c
      vfs.c
  arch/
      x86_64/
          timer.c
          irq.c
          mmu.c
      arm64/
          timer.c
          irq.c
          mmu.c
  drivers/
      serial/
          serial.h
          ns16550.c
          pl011.c
  doc/
      porting.md
      api-reference.md
```

A `core/` könyvtárban lévő kód nem tartalmazhat `#include <arch/...>` direktívát; minden architektúra-specifikus hívást a HAL interfészen keresztül kell elérnie. Ha mégis szükség van architektúra-specifikus optimalizációra – például inline assembly memóriabarrierre –, akkor azt egy generikus makró mögé kell rejteni, amelyet az architektúra fejléc definiál. A generikus kód a makrót hívja, nem az assembly utasítást közvetlenül.

31.4.4 Build matrix

A build matrix célja, hogy minden támogatott architektúra és konfiguráció kombinációját explicit módon nyilvántartsa, és lehetővé tegye a reprodukálható fordítást és tesztelést. Egy tipikus build matrix sorai a következőket tartalmazzák:

Architektúra	Toolchain	Emulátor/gép	Konfiguráció	Megjegyzés
x86_64	x86_64-elf-gcc	qemu-system-x86_64 -M pc	SMP be, APIC be	Fejlesztői alap
arm64	aarch64-elf-gcc	qemu-system-aarch64 -M virt	SMP ki, GICv3	Portolási cél
riscv64	riscv64-elf-gcc	qemu-system-riscv64 -M virt	SMP ki, PLIC	Kísérleti

A build matrix nem csupán dokumentáció: érdemes CI- (folyamatos integrációs) pipeline-ban automatizálni. Minden commit után a rendszer lefordítja az összes architektúrát, elindítja az emulált környezetet, és futtat egy minimális smoke tesztet. Ha bármelyik kombináció meghiúsul, a commit nem kerülhet be a stabil ágba.

A keresztfordítás alapjai a következők. A hoszt operációs rendszer általában x86_64, ezért a célarchitektúrához keresztfordító toolchainre van szükség. A toolchain magában foglalja a cél-architektúrára optimalizált GCC vagy Clang fordítót, a binutils eszközöket (assembler, linker, objcopy), valamint a cél-architektúra szabványos könyvtárait, ha nem freestanding módban dolgozunk. Kernel esetén freestanding módot használunk, így a libc nem érhető el; a szükséges alapfüggvényeket – `memcpy`, `memset`, `strlen` – saját magunk implementáljuk.

A build folyamatot egy egyszerű Makefile is vezérelheti, de nagyobb projekteknél a CMake vagy a Linux Kbuild rendszere jobb skálázhatóságot nyújt. A lényeg, hogy a célarchitektúra és a konfiguráció paraméterként adható át, és a kimeneti bináris neve egyértelműen azonosítja a kombinációt.

Példa egy minimalista Makefile részletre:

```
# Makefile (szemléltető részlet)
ARCH ?= x86_64
CROSS_COMPILE ?= $(ARCH)-elf-
CC = $(CROSS_COMPILE)gcc
LD = $(CROSS_COMPILE)ld
OBJCOPY = $(CROSS_COMPILE)objcopy

CFLAGS = -ffreestanding -nostdlib -Wall -Wextra -Iinclude
ARCH_DIR = arch/$(ARCH)

OBJS = core/sched.o core/syscall.o core/vfs.o \
       $(ARCH_DIR)/timer.o $(ARCH_DIR)/irq.o $(ARCH_DIR)/mmu.o

kernel.bin: $(OBJS)
    $(LD) -T $(ARCH_DIR)/linker.ld -o kernel.elf $(OBJS)
    $(OBJCOPY) -O binary kernel.elf kernel.bin
```

A `$(ARCH)` változó megadásával válthatunk architektúrát, a `CROSS_COMPILE` pedig a toolchain prefixet állítja be. A linker script minden architektúrának megvan a maga könyvtárában, mert a belépési cím, a szekcióelrendezés és a speciális szimbólumok architektúra-specifikusak.

A konfiguráció kezelésére a Linux kernelből ismert Kconfig rendszer mintájára egy egyszerű, szöveges konfigurációs fájl is elegendő lehet. A `CONFIG_SMP`, `CONFIG_DEBUG`, `CONFIG_SERIAL_DRIVER`

beállításokat a build rendszer fejléccé generálja (`config.h`), és a forráskód `#ifdef CONFIG_SMP` formában használja. Ez lehetővé teszi ugyanazon architektúra több konfigurációjának kezelését anélkül, hogy a forráskódot módosítani kellene.

31.4.5 3. Verziózás, release és közösség

A verziózás a hosszú távú karbantartás egyik alappillére. Kernelprojektekben a leggyakoribb a szemantikus verziózás (MAJOR.MINOR.PATCH) vagy a dátum alapú verziózás. A szemantikus verziózás szabályai:

- **MAJOR**: inkompatibilis API-változás, például a HAL interfész átalakítása.
- **MINOR**: visszafelé kompatibilis új funkció, például új eszközmeghajtó hozzáadása.
- **PATCH**: hibajavítás, például egy versenyhelyzet kijavítása az ütemezőben.

A verziószámot a repository gyökerében elhelyezett `VERSION` fájl vagy a build rendszer által generált `version.h` tartalmazza. A kernel futás közben kiírja a verziót, ami a hibakeresés és a reproducing során elengedhetetlen.

A **release folyamat** jellemző lépései:

1. **Feature freeze**: a tervezett funkciók bekerültek, a fejlesztési ág lezárul.
2. **Tesztelési ablak**: a CI build matrixán az összes kombináció lefut, a tesztek eredményét dokumentálják.
3. **Changelog frissítése**: a változások felhasználói és fejlesztői szempontból is leírásra kerülnek.
4. **Verziójelölés**: a repository-ban egy tag készül, például `v1.2.0`.
5. **Kiadási csomag**: a forráskód, a dokumentáció és a reprodukciós csomag egy verziózott archívumba kerül.
6. **Migrációs útmutató**: ha MAJOR verzióváltás történt, részletes útmutató készül a megszakadt interfészekről.

A **changelog** nem csupán felsorolás: minden bejegyzéshez tartozik egy hivatkozás az issue-ra vagy a commitra, valamint a hatás jelölése (biztonsági, teljesítmény, API). A jó changelog lehetővé teszi, hogy a felhasználó gyorsan felmérje, kell-e frissítenie, és milyen kockázattal jár.

A **biztonsági frissítések** külön csatornát igényelnek. A hibát bizalmasan jelentik, a javítás először egy privát ágban készül, majd a nyilvános release-be kerül. A CVE-azonosító (Common Vulnerabilities and Exposures) rögzítése a reprodukció és a nyomon követés miatt fontos. A LTS (Long Term Support) ágakban a javítást vissza kell portolni, amit a release-menedzser koordinál.

A **közösség** szerepe nem korlátozódik a kódírásra. A hosszú távú fejlesztés fenntarthatóságát az is jelenti, hogy:

- **Contribution guideline** dokumentálja, hogyan kell hibát jelenteni, javítást beküldeni, tesztet írni.
- **Code review** minden commit előtt kötelező; a review szempontjai közé tartozik a memória-tulajdonjog, a hibakezelés és a versenyhelyzetek vizsgálata.
- **Issue tracking** rendszerében minden hiba és funkciókérés nyomon követhető, címkékkel és milestone-okkal.
- **Dokumentációs eszközök** – mint a Doxygen, Sphinx vagy a Markdown-alapú megoldások – automatikusan generálják az API-referenciát a forráskódban elhelyezett kommentekből.

Az API-dokumentáció akkor hasznos, ha a függvények és típusok szerződését – előfeltétel, utófeltétel, mellékhatás, tulajdonjog – is megadja. Példa egy jól dokumentált HAL függvényre:

```
/**
 * @brief Fizikai lap leképezése virtuális címre.
 * @param vaddr  A cél virtuális cím; 4 KiB határhoz igazított
 *               (a hívó felelőssége).
 * @param paddr  A forrás fizikai cím; 4 KiB határhoz igazított.
 * @param flags  Lapzászlók: HAL_PAGE_READ, HAL_PAGE_WRITE,
 *               HAL_PAGE_EXECUTE kombinációja.
 * @return 0 siker esetén, -EINVAL ha a címek nem igazítottak,
 *         -ENOMEM ha nincs szabad lapozási struktúra.
 *
 * A hívás nem foglalhat fizikai memóriát, és nem okozhat
 * kontextusváltást; megszakítási kontextusból is hívható.
 */
int hal_mmu_map_page(virt_addr_t vaddr, phys_addr_t paddr, unsigned flags);
```

A dokumentáció a típusok méretét és ABI-ját is rögzíti. Mivel a kernel és a userland közötti határfelület architektúra-specifikus lehet, a dokumentációnak tartalmaznia kell a szóhosszt, a pointerek méretét, a structok igazítását és a hívási konvenciót. Ez különösen akkor fontos, ha a kernelt más nyelvből vagy más toolchainnel akarják használni.

31.4.6 4. Tipikus hibák és helyes megoldások

A platformfüggetlenség és a dokumentáció terén visszatérő hibák a következők:

1. **Architektúra-specifikus kód beszivárgása a generikus részbe.**
   Hiba: a generikus ütemező közvetlenül írja a x86 APIC regisztereket.
   Megoldás: a megszakításvezérlő kezelése a HAL `irq` interfész mögé kerül, és a generikus kód csak az interfészt hívja.

2. **Implicit ABI-feltételezések.**
   Hiba: a kód `unsigned long` méretét 32 bitnek feltételezi, ami 64 bites platformon eltörik.
   Megoldás: minden architektúra-specifikus típust – `uint32_t`, `uint64_t`, `uintptr_t` – explicit módon kell definiálni egy közös `stdint.h`-ban, és a kódban ezeket használni.

3. **Hiányzó build matrix frissítés.**
   Hiba: új architektúra hozzáadása után a CI pipeline nem fut az új konfigurációra, így a regressziók csak későn derülnek ki.
   Megoldás: a build matrixot és a hozzá tartozó CI szkriptet a portolási commit részeként kell frissíteni.

4. **Dokumentáció és kód eltérése.**
   Hiba: a HAL interfész megváltozik, de a portolási útmutató nem frissül, így az új fejlesztő a régi interfész alapján próbál portolni.
   Megoldás: a kódreview során a dokumentációs fájlok változását is ellenőrizni kell; a dokumentáció nélküli interfész-változtatás nem fogadható el.

5. **Nem reprodukálható build.**
   Hiba: a fejlesztői gép lokális beállításaitól függ a fordítás, például a rendszer `gcc` verziója vagy a `PATH` sorrendje.
   Megoldás: a build szkript rögzíti a toolchain verzióját, a konfigurációt és a parancsokat; a CI konténerben fut, így a környezet is reprodukálható.

6. **Release utáni hibajavítás a fejlesztési ágon.**
   Hiba: a stabil ágba kerülő javítást közvetlenül a fejlesztési ágra commitolják, így a stabil ág eltér a fejlesztésitől.
   Megoldás: a javítást először a legrégebbi érintett stabil ágra kell backportolni, majd a fejlesztési ágra merge-elni. Ezt a release-menedzser koordinálja.

31.4.7 5. Labor: portolási és dokumentációs terv

Ez a labor egy minimális kernel – nevezzük `mini-os`-nak – portolását és a hozzá tartozó dokumentáció elkészítését gyakoroltatja be. A cél nem egy teljes operációs rendszer elkészítése, hanem a fejezetben tárgyalt alapelvek alkalmazása egy kis, ellenőrizhető példán.

31.4.8 5.1. Kiinduló állapot és cél

A kiinduló állapot egy x86_64 architektúrára írt, működő kernel, amely a következő funkciókat valósítja meg:

- boot: multiboot kompatibilis belépés
- konzol: soros porton karakteres kimenet (COM1, 0x3F8)
- időzítő: PIT (Programmable Interval Timer) 100 Hz-en
- ütemező: egyszerű round-robin két feladattal
- memória: statikus foglalás, lapozás nélkül

A célplatform egy ARM64 emulált gép (`qemu-system-aarch64 -M virt`). A feladat a kernel architektúra-függő részeinek azonosítása, HAL interfész kialakítása, az ARM64 implementáció megírása, valamint a build matrix és a dokumentációs terv elkészítése.

A labor nem módosítja a hoszt valódi bootloaderét, lemezét vagy rendszerpartícióját; minden művelet QEMU-ban, eldobható image-ekkel történik.

31.4.9 5.2. Lépések

1. **Architektúra-függő kód azonosítása**
   Az eredeti x86_64 forráskódban keresd meg a közvetlen hardverhozzáféréseket:

   - soros port: `outb` hívások a 0x3F8 portra
   - PIT: 0x40–0x43 portok
   - megszakítások: IDT felállítása, `sti`/`cli`
   - kontextusváltás: x86 regisztermentés

   Készíts listát, és jelöld be, hogy melyik művelet melyik HAL interfészbe tartozik.

2. **HAL interfész definiálása**
   a fenti minta alapján hozz létre egy `include/hal.h` fájlt, amely tartalmazza a `hal_ops` struktúrát a következő műveletekkel:

   - `init`: hardver inicializálás
   - `putchar`: karakter kiírása a konzolra
   - `timer_init`, `timer_get_ticks`: időzítő kezelése
   - `irq_enable`, `irq_disable`: megszakítások engedélyezése/tiltása
   - `context_switch`: kontextusváltás a megadott stackre

A generikus kernel ezentúl csak ezeket hívja.

3. **Generikus kernel átalakítása**
   Az ütemező és a konzol kimenet a HAL interfészt használja. Az x86_64 implementáció költözzön az `arch/x86_64/` könyvtárba, és ott valósítsa meg a `hal_ops` struktúrát.

4. **ARM64 implementáció**
   Hozd létre az `arch/arm64/` könyvtárat. A konzol kimenethez használd a PL011 UART-ot a QEMU `virt` gépén (alapcím: 0x09000000). Az időzítő az ARM architektúra generic timer-jét használja. A megszakítások kezeléséhez a GIC v3-at állítsd be. A kontextusváltás ARM64 regisztermentést igényel (x19–x29, sp, pc).

5. **Build matrix és CI**
   Készíts Makefile-t vagy CMake konfigurációt, amely az `ARCH` változó alapján kiválasztja a megfelelő könyvtárat és toolchain-t. Állíts be egy CI szkriptet, amely mindkét architektúrára lefordítja a kernelt, és QEMU-ban elindít egy minimális tesztet.

6. **Dokumentációs terv**
   Készítsd el a következő dokumentációkat:

   - `README.md`: a projekt célja, build parancsok, architektúra-áttekintés
   - `doc/porting.md`: lépésenkénti útmutató új architektúra hozzáadásához, beleértve a szükséges HAL implementációkat
   - `doc/api-reference.md`: a `hal_ops` interfész részletes leírása, paraméterekkel, visszatérési értékekkel és hibakódokkal
   - `doc/build-matrix.md`: a támogatott architektúrák, toolchain verziók, emulátor parancsok és elvárt eredmények táblázata

31.4.10 5.3. Emulált környezet és mérési napló

A mérésekhez az alábbi konfigurációt használjuk:

- **Hoszt**: x86_64 Linux, kernel 6.2 (a pontos verziót a naplóban rögzíteni kell)
- **QEMU**: 8.0.0
- **Toolchain**:
  - x86_64: `x86_64-elf-gcc 12.2.0`
  - arm64: `aarch64-elf-gcc 12.2.0`
- **Emulált gép**:
  - x86_64: `qemu-system-x86_64 -M pc -m 64M -nographic`
  - arm64: `qemu-system-aarch64 -M virt -m 64M -nographic -cpu cortex-a53`
- **Fordítási beállítások**: `-O2 -ffreestanding -nostdlib -Wall -Wextra`

A mérési napló a következő értékeket tartalmazza:

Mérés	Várt eredmény	Mért eredmény (x86_64)	Mért eredmény (arm64)
Kernel méret (bájt)	< 16 KiB	12 844	11 920
Boot idő (ms)	< 200 ms	145 ms	132 ms
Konzol kimenet megjelenése	„mini-os ready”	megjelent	megjelent

Mérés	Várt eredmény	Mért eredmény (x86_64)	Mért eredmény (arm64)
Ütemező tick 10 s alatt	1000 ± 10	1003	998
Megszakítási késleltetés (s)	< 50 s	38 s	41 s

A boot idő mérése QEMU `-icount` vagy a soros port első karakterének időbélyegzésével történten. A megszakítási késleltetéshez a timer interrupt és a handler belépése közötti időt kell mérni, például a generic timer count regiszterének olvasásával. A mért és a várt értékeket minden esetben külön kell feltüntetni, és a mérési módszert pontosan le kell írni.

31.4.11 5.4. Átadandó eredmény

A labor átadandó eredményei:

1. A portolt kernel forráskódja a repository-ban, a build matrixszal együtt.
2. A `doc/porting.md` dokumentáció, amely egy új fejlesztő számára lehetővé teszi a portolást más architektúrára.
3. A mérési napló, amelyben a várt és a mért értékek külön oszlopban szerepelnek.
4. Önellenőrző kérdések megválaszolása:
    - Miért nem elegendő a generikus kódban `#ifdef __x86_64__` feltételes fordítás?
    - Milyen következménye lenne annak, ha a HAL interfész a `struct pt_regs` x86-specifikus definícióját használná?
    - Hogyan biztosítja a build matrix a regressziók korai észlelését?

31.4.12 6. Összefoglalás

A dokumentáció, a portolás és a karbantartás nem utólagos feladat, hanem a kerneltervezés szerves része. A hardverabsztrakciós réteg lehetővé teszi, hogy a generikus kernel logika független legyen a konkrét hardvertől; a build matrix biztosítja a reprodukálhatóságot; a verziózás és a release-folyamat pedig a hosszú távú stabilitást. A fejezetben bemutatott minták – HAL interfész, architektúra-specifikus könyvtárstruktúra, konténerizált CI, szemantikus verziózás – együttesen alkotják azt az alapot, amelyre a következő fejezet záróprojektje épül. A labor során elkészített portolási terv és dokumentációs csomag a gyakorlatban is igazolja, hogy a platformfüggetlenség nem elméleti luxus, hanem a fenntartható kernel fejlesztésének feltétele.

32 Záróprojekt: minimális operációs rendszer

32.1 Követelmények és mérföldkövek

A záróprojekt akkor válik tanulható rendszerfejlesztéssé, ha a „írok egy operációs rendszert” lelkesedést először lefordítjuk ellenőrizhető követelményekre, majd azokat apró, egymásra épülő mérföldkövekre bontjuk. Ez az egység ezt a tervezési munkát rögzíti. Nem arról szól, hogyan kell megírni a bootloadert vagy a lapozót – az a következő egységek feladata –, hanem arról, hogy mi kerüljön a projektbe, mi maradjon ki, és milyen elfogadási feltételekkel igazoljuk, hogy egy-egy rész elkészült.

A fejezet tanulási céljai:

- megmagyarázni, miért szükséges a záróprojekt scope-ját és mérföldköveit írásban rögzíteni;
- alkalmazni a scope-képzés és mérföldkő-tervezés eszközeit egy kis, ellenőrizhető példán;
- felismerni a tipikus tervezési hibákat, és megindokolni a helyes megoldást.

A fejezetben bemutatott parancsok és kódrészletek szemléltetésnek minősülnek. Éles gépen csak a laborban leírt, emulált környezetben szabad futtatni őket.

32.1.1 Miért kell scope és mérföldkő?

Egy operációs rendszer – még egy „minimális” is – komponensek együttműködése: boot, megszakításkezelés, memóriaallokátor, ütemező, rendszerhívások, felhasználói programok, esetleg fájlrendszer. Ha ezeket párhuzamosan, világos határok nélkül kezdjük fejleszteni, a hibakeresés és a haladás mérése szinte lehetetlenné válik. A scope rögzítése azt jelenti, hogy explicit módon kimondjuk:

- mi a projekt célja egy mondatban;
- mely komponensek tartoznak a minimálisan elfogadható változathoz;
- mely komponensek tartoznak a „később, ha marad idő” kategóriába;
- mi az az elfogadási feltétel, amellyel egy komponens késznek tekinthető.

A mérföldkő ezzel szemben időben is behatárolt, ellenőrizhető állomás. A jó mérföldkő nem azt mondja, hogy „memóriakezelés félkész”, hanem például:

> M2: a `page_alloc()` és `page_free()` hívások 1000 egymást követő foglalás és felszabadítás után is konzisztens fizikai laplistát adnak, és a `kmalloc(64)` legalább 128 alkalommal lefut laphiba és túlcsordulás nélkül.

Az ilyen megfogalmazás azért fontos, mert a laborban a hallgató nem azt ellenőrzi, hogy „sok kódot írtam”, hanem azt, hogy egy előre rögzített, reprodukálható viselkedés teljesül.

A záróprojektben bevezetett elnevezést követve a cél egy **minimálisan életképes operációs rendszer** (Minimal Viable OS, MVO). Az MVO nem egy általános célú Linux-klón, hanem egy olyan kernel és userland, amely egy emulált gépen bebootol, kezeli a fizikai memóriát, ütemez legalább két felhasználói folyamatot, elérhetővé teszi a legfontosabb rendszerhívásokat, és egy shellen keresztül bemutatható.

32.1.2 A záróprojekt célja és határai

A projekt egészének terjedelmét egy mondatban érdemes rögzíteni:

> A projekt egy QEMU-ban futó, RISC-V vagy x86-64 architektúrájú, monolitikus felépítésű minimális kernel, amely betölt egy felhasználói shellt, biztosít preemptív ütemezést, lapozás nélküli vagy egyszerű lapozással védett memóriát, és néhány alapvető rendszerhíváson keresztül futtat parancsokat.

Ez a mondat több eldöntött kérdést is tartalmaz:

- **Célplatform:** QEMU, nem fizikai hardver.
- **Architektúra:** egy rögzített architektúra, nem több.
- **Kernelfelépítés:** monolitikus kernel, nem mikrokernel.
- **Ütemezés:** preemptív, időzítő-megszakításra.
- **Memória:** lapozás vagy legalább védett szegmensek; a minimális cél a fizikai memória kezelése.
- **Userland:** shell, statikusan linkelt ELF vagy saját formátum.
- **Fájlrendszer:** csak annyi, amennyi a shell és a demó számára szükséges.

A scope rögzítésének egyik leghasznosabb eszköze az **IN/OUT lista**. Az IN oszlopba azok a képességek kerülnek, amelyek nélkül a projekt nem tekinthető sikeresnek. Az OUT oszlopba azok, amelyeket tudatosan kihagyunk. Az 1. táblázat egy lehetséges változatot mutat.

IN	OUT
1 architektúra (pl. RISC-V 64)	x86 + ARM egyszerre
Serial konzol, QEMU `-nographic`	Grafikus framebuffer, egér
Fizikai memória felderítése	NUMA, hotplug memória
Egyetlen kernel lapozási szint	ASLR, swap, demand paging
Preemptív round-robin ütemező	SMP, prioritásos ütemező, affinity
Legalább 5 rendszerhívás	Teljes POSIX API
Statikus ELF felhasználói program	Dinamikus linkelés, `dlopen`
Minimális ramfs	Journaling, RAID, hálózati FS
`help`, `echo`, `ps`, `shutdown` shellparancsok	Job control, scriptelés, pipelining

Az IN/OUT lista előnye, hogy már a tervezés elején kényszeríti a döntést. Ha valaki például SMP-t szeretne, akkor vagy be kell emelnie az IN oszlopba, és akkor a mérföldkövek bővülnek, vagy tudomásul kell vennie, hogy az OUT-ban marad, és a záró bemutatón nem hivatkozhat rá alapfunkcióként. A könyv módszertana szerint az ilyen döntéseket a dokumentációban rögzíteni kell.

A nem funkcionális követelmények közül három különösen fontos egy kernelprojektben:

- **Reprodukálhatóság:** a build és a futtatás környezete verziózott legyen. A toolchain, a QEMU, a linker script és a kernel konfigurációja együtt kerüljön a reprodukciós csomagba.
- **Erőforrás-tulajdonjog:** minden lefoglalt lapnak, blokknak és fájlleírónak legyen egyértelmű tulajdonosa. A kernelkódban a `page_alloc()` után mindig tudni kell, ki és mikor szabadítja fel.
- **Hibakezelés:** a `panic` és a `kassert` mellett a rendszerhívásoknak legyen definiált hibakódja és `errno`-szerű értéke. A felhasználói programnak sosem szabad kernel belső hibával találkoznia.

Ezeket a követelményeket a későbbi egységek részletezik; itt annyi a feladat, hogy a projekttervben szerepeljenek.

32.1.3 A mérföldkő-lista szerkezete

A záróprojekt mérföldköveit a már említett öt alappillér köré csoportosítjuk: boot, memória, ütemező, rendszerhívások és shell. Emellett szükség van egy induló és egy záró mérföldkőre, valamint a tesztelést és a dokumentációt is mérföldkőként kezeljük.

Minden mérföldkő leírásában a következő mezőket rögzítjük:

- **Azonosító és cím:** például `M1 - Boot és kernel inicializálás`.
- **Bemenet/függőség:** mely korábbi mérföldkőre épít.
- **Szolgáltatás:** mit ad a projekt többi részének.
- **Elfogadási feltétel:** milyen parancs vagy teszt fut le sikeresen.
- **Kézzelfogható kimenet:** fordítási napló, QEMU log, Gantt-diagram, teszteredmény.
- **Kockázat:** mi az a tipikus buktató, amire figyelni kell.

A mérföldkő lista ne legyen túl hosszú. A záróprojekthez hat-nyolc mérföldkő elegendő. Ha valaki tizenötnél több mérföldkövet ír fel, akkor valószínűleg feladatokat, nem mérföldköveket sorol fel. A mérföldkő integrációs pont: több részfeladat együttes eredményét igazolja.

Egy lehetséges felosztás:

- **M0:** Fejlesztői környezet és füstteszt
- **M1:** Boot és kernel inicializálás
- **M2:** Fizikai memória és kernel heap
- **M3:** Ütemező és kontextusváltás
- **M4:** Rendszerhívások és felhasználói mód
- **M5:** Shell és minimális fájlrendszer
- **M6:** Tesztelés, dokumentáció és demó

Az alábbi folyamatábra a mérföldkövek függőségét mutatja.

```
flow: M0 -> M1 -> M2 -> M3 -> M4 -> M5 -> M6
legend: minden nyíl azt jelenti, hogy a következő mérföldkő
        a megelőző stabil interfészére épül
```

Ez a lineáris sorrend a záróprojektben általában elegendő. Ettől el lehet térni, például a shell fejlesztése elindulhat már a rendszerhívások stabilizálása előtt, de a mérföldkő akkor válik lezárhatóvá, ha az összes függőség kész.

32.1.4 Az öt alappillér mint mérföldkő

A következő alpontokban azt írjuk le, milyen követelményeket és elfogadási feltételeket érdemes az egyes alappillérekhez rendelni. A hangsúly a scope-on és a verifikáción van, nem a részletes implementáción.

32.1.5 1. Boot és kernel inicializálás

A boot mérföldkő azt igazolja, hogy a kernel a semmiből eljut odáig, hogy saját C kódot futtat, és a soros konzolon megbízhatóan jelez. A tipikus elfogadási feltétel:

```
qemu-system-riscv64 \
  -machine virt \
  -m 128M \
  -nographic \
  -bios none \
  -kernel build/kernel.elf \
  -serial mon:stdio
```

Elvárt kimenet:

```
mini-OS boot
CPU: riscv64
memory: 128 MiB
kernel_start: 0x80000000
```

A boot mérföldkőhöz tartozó ellenőrző lista:

- a linkerscript a kernel ELF belépési pontját a megfelelő fizikai címre helyezi, például RISC-V `machine` módban `0x80000000`-ra;
- az indító assembly kód beállítja a vermet, törli a BSS szakaszt, és átadja a vezérlést a C `kmain()` függvénynek;
- a soros port inicializálása a legkorábbi konzol kimenetet már az assembly részben lehetővé teszi;
- a QEMU parancs reprodukálható, és a napló tartalmazza a QEMU verzióját.

A boot mérföldkőnél gyakori hiba, hogy a fejlesztő rögtön megszakításokat, lapozást és fájlrendszert akar bekapcsolni. Ehelyett az M1 egyetlen célja a **minimális végrehajtási lánc** igazolása. Ha a serial kimenet nem jelenik meg, a további mérföldköveknek nincs alapjuk.

A mérési naplóban ilyenkor érdemes rögzíteni a *boot latency* értéket: a QEMU indításától az első „mini-OS boot” sor megjelenéséig eltelt időt. Ez a mérőszám a későbbi optimalizálás alapja lehet.

32.1.6 2. Fizikai memória és kernel heap

A memória mérföldkő azt igazolja, hogy a kernel képes a rendelkezésre álló fizikai memóriát áttekinteni, és abból lapokat foglalni/felszabadítani. A minimális interfész:

```
void *page_alloc(void);          // egy 4 KiB-os fizikai lap címe
void  page_free(void *page);     // lap visszaadása
void *kmalloc(size_t size);      // kernel heap foglalás
void  kfree(void *ptr);          // kernel heap felszabadítás
```

Az elfogadási feltételnek mérhetőnek kell lennie. Például:

> Miután a kernel felderítette a 128 MiB fizikai memóriát, a `page_alloc()` és `page_free()` 1000 véletlenszerűen összekevert művelet után is konzisztens laplistát tart fenn. A `kmalloc(64)` 128 egymást követő foglalás után nem ad vissza átfedő címet, és minden felszabadítás után a belső fejlécek épek.

A memória mérföldkő fontos tervezési döntése, hogy a kernel a fizikai memóriát egyenlő 4 KiB-os lapokban kezeli-e, vagy egy egyszerűbb, buddy-rendszerű allokátort használ. A záróprojektben a lap-alapú megközelítés az ajánlott, mert később a lapozás és a memóriavédelem is erre épül.

A laborban a memória mérföldkőhöz tartozó mérés a **fragmentációs arány**: hány lap foglalható le a teljes kapacitásból a különböző méretű foglalások után. Ezt a mérőszámot a későbbi egység részletezi, de a projekttervben már most szerepelnie kell.

32.1.7 3. Ütemező és kontextusváltás

Az ütemező mérföldkő azt igazolja, hogy a kernel képes két vagy több folyamatot megszakításból vezérelt kontextusváltással futtatni. A minimális követelmény:

- minden folyamathoz tartozik egy PCB (process control block);
- a PCB tárolja a mentett regisztereket, a stack mutatót és a folyamat állapotát;
- egy időzítő-megszakítás rendszeresen lecseréli a futó folyamatot;
- a `schedule()` függvény round-robin sorrendben választja a következő folyamatot.

A vizsgált munkaterhelés legyen meghatározott. Például három folyamat, P_1, P_2 és P_3, amelyek CPU-időt kérnek, és a kernel naplózza a futási sorrendet. Az egyes folyamatok CPU-igénye $C_1 = C_2 = C_3$, periódusuk nem valós idejű, ezért itt nincs T_i és D_i; a cél csupán a körbeforgás és a kontextusváltás helyessége.

Egy lehetséges Gantt-diagram:

```
P1 |  |     |  |     |  |
P2 |   |  | |   |  | | |
P3 |   |   | |   | |
   0    1    2    3    4
   (időzítő tickek, nem valós idő)
```

Az elfogadási feltétel:

> A kernel 1000 időzítő-megszakítás alatt legalább 100 kontextusváltást végez, és a naplóban minden folyamat legalább egyszer megkapja a CPU-t. A kontextusváltás során a veremmutató és az általános regiszterek megegyeznek a mentett értékkel.

Itt a mérési naplóban rögzíteni kell a hardvert és a fordítási beállításokat: a QEMU emulált CPU modelljét, az órajelfrekvenciát, ha van, a `-O2` vagy `-Os` optimalizálást, valamint a mérési módszert, például a megszakítási naplózás ciklusszámlálóját. A tankönyv szabálya szerint egyetlen szám nem elegendő, de a projektterv szintjén elég megnevezni, mit fogunk mérni.

32.1.8 4. Rendszerhívások és felhasználói mód

A rendszerhívás mérföldkő azt igazolja, hogy a felhasználói tér és a kernel tér közötti határ működik. A minimális rendszerhívás-készlet:

Sorszám	Név	Leírás	Paraméterek	Visszatérés
0	`SYS_WRITE`	soros konzolra ír	`fd`, `buffer`, `size`	leírt bájtok száma vagy `-1`
1	`SYS_EXIT`	folyamat befejezése	`status`	nem tér vissza
2	`SYS_YIELD`	CPU átadása	–	`0`

Sorszám	Név	Leírás	Paraméterek	Visszatérés
3	`SYS_GETPID`	folyamat azonosító	–	`pid_t`
4	`SYS_SLEEP`	alvás adott tickig	`ticks`	`0`

A hibakódokat a kernel egy `errno`-szerű változóban adja vissza. Például ha a `buffer` mutató érvénytelen, a `SYS_WRITE` `-1` értékkel tér vissza, és a hibakód `EFAULT`. A felhasználói program ezt a `libc`-szerű wrappercsomagban látja.

A felhasználói módba lépéshez a kernelnek be kell állítania a megfelelő jogosultsági szintet. RISC-V esetén a `mstatus.MPP` (machine previous privilege) mezőt `U` módra kell állítani, majd `mret` utasítással belépni a felhasználói kódba. Az elfogadási feltételben szerepelhet:

> Egy felhasználói program a `SYS_WRITE` hívással kiírja a „hello from user mode” szöveget, majd a `SYS_EXIT` hívással `0` státusszal befejeződik. A kernel a kilépés után visszanyeri a vezérlést, és egy új programot indít.

A rendszerhívás-mérföldkőhöz tartozó tipikus hiba, hogy a kernel nem ellenőrzi a felhasználói mutatókat, és a felhasználói program kernelcímet olvashat. A projekttervnek ezért tartalmaznia kell egy ellenőrzési pontot: minden felhasználói mutatót a `copy_from_user`/`copy_to_user` vagy azzal egyenértékű függvénnyel kell elérni.

32.1.9 5. Shell és minimális fájlrendszer

A shell mérföldkő a felhasználói térből elérhető, interaktív parancsértelmezőt igazol. A minimális shell parancsai:

- `help` – kilistázza a parancsokat;
- `echo <szöveg>` – kiírja a szöveget a `SYS_WRITE` segítségével;
- `ps` – megjeleníti a futó folyamatok PID-jét és állapotát;
- `shutdown` – leállítja a gépet, vagy visszaadja a vezérlést a QEMU-nak.

A shell önmagában is egy felhasználói folyamat. A kernel betölti a statikusan linkelt ELF képét, létrehozza a processzt, és átadja neki a konzolt. A shell parancsai nem igényelnek fájlrendszert, de a projekt későbbi szakaszában a `echo` például fájlba is írhat.

A fájlrendszer mérföldkőhöz a minimális cél egy RAM-alapú fájlrendszer (ramfs), amelyben lehet fájlt létrehozni, olvasni, írni és törölni. A VFS-integrációt a shell fájlparancsaival lehet bemutatni. Az elfogadási feltétel:

> A shellben kiadott `touch /tmp/hello` hatására a `stat` parancs szerint a fájl mérete 0.
> A `echo hello > /tmp/hello` után a `cat /tmp/hello` a `hello` szöveget adja vissza.
> A `rm /tmp/hello` után a `cat` hibaüzenetet ad.

Ez a mérföldkő már a későbbi `os125` részletes anyagára támaszkodik, de a scope szintjén itt is rögzíteni kell, hogy a fájlrendszer csak annyit tud, amennyit a demó megkövetel. Nincs szükség fájlleíró-öröklésre, csővezetékre vagy jogosultságokra.

32.1.10 Demó és értékelési kritériumok

A záróprojekt demója az a forgatókönyv, amelyet a bemutató végén végrehajtunk. A demónak reprodukálhatónak kell lennie, és az összes IN-követelményt le kell fednie. Egy ajánlott demóforgatókönyv:

1. A konzolon elindul a QEMU, a kernel bebootol.
2. A kernel kiírja a boot naplót: architektúra, memória mérete, kernel kezdőcím.
3. A memóriaallokátor lefoglal és felszabadít 1000 lapot; a statisztika megjelenik.
4. A kernel betölti a shellt felhasználói módba.
5. A shellben fut a `ps`, amely mutatja a shellt és egy másik tesztfolyamatot.
6. A `echo` kiír egy felhasználói üzenetet.
7. A `shutdown` leállítja a rendszert, a QEMU `0` exit kóddal tér vissza.

A demó értékelési kritériumait előre rögzíteni kell. Egy lehetséges súlyozás:

Kategória	Súly	Mit ellenőriz
Funkcionális teljesség	40%	Minden mérföldkő elfogadási feltétele teljesül-e
Reprodukálhatóság	20%	Egyetlen `make run` parancs buildol és futtat-e; a toolchain verziója rögzített-e
Tesztminőség	15%	A tesztek automatizáltak-e, a mért és várt értékek elkülönülnek-e
Kódminőség	15%	Erőforrás-tulajdonjog, hibakezelés, nincs `// TODO` a kritikus úton
Dokumentáció	10%	A `README` leírja a buildet, a memóriatérképet, a hibakódokat, a korlátokat

Az értékelési táblázatot a projekttervben rögzíteni kell, hogy a fejlesztés közben ne változzon a mérce. A bónuszpontok csak az alapkövetelmények teljesülése után járhatnak, például SMP, ASLR, naplózó fájlrendszer vagy hálózati stack esetén.

32.1.11 Tipikus hibák és helyes megoldások

A záróprojekt tervezése során a következő hibák fordulnak elő leggyakrabban.

32.1.12 Hiba 1: Túl nagy scope

A hallgató az első héten SMP-t, felhasználói módú ASLR-t és TCP/IP stacket szeretne. A mérföldkövek felsorolása oldalakra terjed, és a félév végén egyetlen komponens sincs kész.

Helyes megoldás: Az IN/OUT lista kíméletlen alkalmazása. A minimális cél elérése után lehet bővíteni, de a bővítés sosem veszélyeztetheti az alap demót. Az OUT listát a dokumentációban rögzíteni kell, hogy a vállalás egyértelmű legyen.

32.1.13 Hiba 2: Nem mérhető mérföldkövek

A mérföldkő leírása: „A memóriakezelés működik.” Nincs elfogadási feltétel, nincs parancs, nincs napló. Így a haladás szubjektív, és a demó előtti héten derül ki, hogy a dolog nem működik.

Helyes megoldás: Minden mérföldkőhöz tartozzon egy konkrét parancs vagy teszt, amely `0` exit kóddal tér vissza siker esetén. Például:

```
make test-m2
```

A teszt a `page_alloc()`/`page_free()` stressztesztet futtatja, és összehasonlítja a várt és mért értékeket.

32.1.14 Hiba 3: Nem rögzített környezet

A hallgató a saját gépén lefordítja a kernelt, de a bemutatón más QEMU, más GCC, más linker script van. A build hibát ad vagy nem reprodukálható.

Helyes megoldás: A reprodukciós csomag része egy `README.md`, egy `Makefile` és egy `Dockerfile` vagy `script/setup.sh`, amely rögzíti a toolchain verzióját. A könyv konvenciója szerint a verziókat a felhasználás előtt ellenőrizni kell, és a dokumentációban dátumozottan kell szerepeltetni.

32.1.15 Hiba 4: A várt és mért eredmény összemosása

A mérési naplóban csak annyi szerepel: „gyors volt”. Nincs várt érték, nincs mérési módszer, nincs hardverleírás.

Helyes megoldás: A projekttervben minden méréshez előre definiálni kell a várt értéket és a mérési protokollt. A laborban a mért értéket külön oszlopban, a feltételekkel együtt kell rögzíteni.

32.1.16 Hiba 5: Hiányzó hibakezelési szerződés

A kernelben a `kmalloc()` néha `NULL` értéket ad vissza, de a hívó ezt nem ellenőrzi. A `SYS_WRITE` érvénytelen mutatóra kernel panicot okoz.

Helyes megoldás: A projekttervben rögzíteni kell, hogy minden rendszerhívás `-1` hibakóddal és `errno`-val tér vissza, a kernel belső hibája pedig `panic`-et vált ki, de soha nem adhat át hibát a felhasználói térbe. A felhasználói programoknak ellenőrizniük kell minden visszatérési értéket.

32.1.17 Labor: projektterv és mérföldkő-lista készítése

A labor célja, hogy a fejezetben tárgyalt módszert a hallgató a saját záróprojektjére alkalmazza. Az eredmény egy reprodukálható jegyzetfüzet, például Markdown + Makefile kombináció, amely tartalmazza a scope-ot, a mérföldkő-listát és a füsttesztet.

32.1.18 Kiinduló állapot

Egy üres projektkönyvtár, QEMU és a célarchitektúra toolchainje telepítve. A pontos verziókat a labor elején rögzíteni kell, például:

```
QEMU: 8.2.0 (ellenőrizendő)
GCC-RISCV64: 13.2.0 (ellenőrizendő)
Host: x86_64, Linux 6.8 (ellenőrizendő)
```

A labor nem módosítja a hoszt bootloaderét vagy partícióit; minden művelet az emulált gépen történik.

32.1.19 Feladatok

1. **Hozd létre a `docs/plan.md` fájlt.** Írd le a projekt célját egy mondatban, az IN/OUT listát, a nem funkcionális követelményeket és a demó forgatókönyvét.
2. **Készítsd el a `docs/milestones.yaml` vagy `docs/milestones.md` fájlt.** Minden mérföldkőhöz adj meg azonosítót, függőséget, elfogadási feltételt, kimenetet és kockázatot.
3. **Hozd létre a `scripts/smoke.sh` füsttesztet.** A szkript nem valódi kernel, csak ellenőrzi, hogy a QEMU elindul, és a projekttervben szereplő parancsok futnak.

Példa a füsttesztre:

```
#!/usr/bin/env bash
set -eu
echo "QEMU version:"
qemu-system-riscv64 --version | head -n 1
echo "Toolchain:"
riscv64-linux-gnu-gcc --version | head -n 1
echo "Expected: mini-OS project environment ready"
```

4. **Készítsd el a mérési naplót** `docs/measurements.md` néven. A táblázatban szerepeljen legalább egy mérőszám, a mérési módszer, a hardver, a várt és a mért érték.

Mérőszám	Mérési módszer	Hardver / beállítás	Várt	Mért
Boot idő az első serial karakterig	QEMU log timestamp	QEMU 8.2.0, RISC-V `virt`, 128 MiB, `-nographic`	< 1 s	?

A mért értéket a következő egység laborjában lehet kitölteni; a projekttervben az oszlop létezése a lényeg.

5. **Ellenőrizd a tervet** egy másik hallgatóval vagy a checklist alapján. A checklist kérdései:

☐ Minden mérföldkőnek van elfogadási feltétele?
☐ A buildparancs ugyanazt az eredményt adja tiszta környezetből?
☐ A felhasználói mutatók ellenőrzése szerepel a rendszerhívás-mérföldkőben?
☐ A demó minden IN-követelményt lefed?
☐ A várt és mért értékek külön oszlopban vannak?

32.1.20 Önellenőrző kérdések

1. Mi a különbség a scope IN/OUT listája és a mérföldkő-lista között?
2. Miért nem elegendő az „a memóriakezelés működik” megfogalmazás egy mérföldkőnél? Írd át a megfogalmazást mérhető elfogadási feltétellé.
3. A demó forgatókönyvben a shell `ps` parancsa mit igazol a kernel ütemezőjével és folyamatkezelésével kapcsolatban?
4. Milyen hibát okoz, ha a felhasználói program érvénytelen mutatót ad át a `SYS_WRITE` rendszerhívásnak, és a kernel nem ellenőrzi azt? Milyen szerződést kell rögzíteni a projekttervben?
5. A mérési naplóban miért kell külön várt és mért értéket feltüntetni?

A labor átadandó eredménye a `docs/plan.md`, a mérföldkő-lista, a `scripts/smoke.sh` és a mérési napló vázlata. A füsttesztnek sikeresen le kell futnia, és a dokumentumnak tükröznie kell a fenti checklistet.

32.1.21 Összefoglalás és átvezetés

Ebben az egységben rögzítettük a záróprojekt alapjait: a minimális operációs rendszer célját, a scope-ot, a mérföldköveket, a demót és az értékelési kritériumokat. Az öt alappillér – boot, memória, ütemező, rendszerhívás, shell – mindegyikéhez tartozik elfogadási feltétel, mérési szempont és tipikus hiba. A laborban elkészülő projektterv lesz a következő egységek bemenete.

A következő egység (`os123`) az M1 mérföldkőre épít: a boot folyamatot, a kernel inicializálását és a fizikai memória kezelését bontja ki részletesen. A projekttervben most rögzített `page_alloc()` interfész ott válik valódi C kóddá. A mérföldkő-lista nem statikus dokumentum; ha az implementáció során egy korlát ütközik a tervvel, a scope-ot a dokumentációban frissíteni kell, és a változást a reprodukciós csomagban jelölni. Így a záróprojekt végig ellenőrizhető marad.

32.2 Boot, kernel és memória megvalósítása

A záróprojekt második mérföldköveként most a kernel tényleges életre keltése következik: olyan lemezképet készítünk, amelyet a QEMU emulátor betölt, a processzor végrehajt, majd a kernel inicializálja a fizikai memóriát, és képes lapokat lefoglalni. A fejezet során a következő három réteget építjük fel, és mindegyiknél bemutatjuk a tipikus hibákat is:

1. **Bootolható kernel:** belépési pont, linker script, a BSS szekció nullázása és a C nyelvű `kernel_main` elérése.
2. **Keret-lefoglaló (frame allocator):** a fizikai memória nyilvántartása bitmap alapján, a kernel által használt területek kizárása.
3. **Lapozás és kernel-heap:** Sv39 lapasztal felépítése RISC-V alatt, azonossági leképezés, valamint egy egyszerű lapalapú heap-allokátor.

A laborban minden lépést reprodukálható parancsokkal, emulált környezetben hajtunk végre. A kódok szemléltető jellegűek: nem éles rendszeren futtatandó utasítások, hanem a koncepciókat bemutató minták. A mérési eredményeket mindig a várt értékkel együtt, a mérési módszer megadásával közöljük.

32.2.1 1. A bootolható kernel felépítése

A bootolható kernel elsődleges követelménye, hogy a firmware vagy a bootloader betöltse a kernel lemezképét a memóriába, majd átadja a vezérlést a belépési pontnak. A záróprojektben a QEMU `-kernel` opcióját használjuk, amely a RISC-V `virt` gépen a kernelt a fizikai `0x80000000` címre tölti, és onnan indítja a CPU-t. Nincs szükség külön bootloaderre, de a kernelnek saját magának kell elvégeznie a korai inicializálást.

32.2.2 1.1. Linker script

A linker script meghatározza, hogy a lefordított objektumfájlok szekciói milyen címekre kerüljenek. A kernelünk azonossági leképezést használ, tehát a virtuális cím megegyezik a fizikaival. A belépési pont a `_start` szimbólum, amelyet az assembly fájl definiál. A script a következő szekciókat rendezi el: `.text` (kód), `.rodata` (csak olvasható adatok), `.data` (inicializált globális változók), `.bss` (nullázandó globális változók). A `.bss` szekció méretét a linker automatikusan kiszámítja, de a belépő kódnak a futási időben nulláznia kell.

Példa `kernel.ld`:

```
OUTPUT_ARCH(riscv)
ENTRY(_start)

SECTIONS
{
    . = 0x80000000;

    .text : {
        *(.text.init)
        *(.text)
    }

    . = ALIGN(4096);
    .rodata : {
        *(.rodata)
    }

    . = ALIGN(4096);
    .data : {
        *(.data)
    }

    . = ALIGN(4096);
    .bss : {
        __bss_start = .;
        *(.bss)
        *(COMMON)
        __bss_end = .;
    }

    . = ALIGN(4096);
    __kernel_end = .;
}
```

A `__bss_start` és `__bss_end` szimbólumokat az assembly belépő használja a BSS nullázásához. A `__kernel_end` pedig a memóriakezelő számára jelzi a kernel által foglalt terület végét; minden e fölötti fizikai lap szabadnak tekinthető, kivéve a később lefoglalt lapokat.

32.2.3 1.2. Assembly belépő

A belépő feladata három lépésből áll:

1. A stack pointer (`sp`) beállítása egy érvényes veremterületre.
2. A BSS szekció kinullázása.
3. Ugrás a C nyelvű `kernel_main` függvényre.

Mivel a kernel a `0x80000000` címen indul, és a stacket a kernel saját adatterületén belül kell elhelyezni, egy egyszerű megoldás, ha a verem a kernel végén található, például a `__kernel_end` címén. Ez azonban ütközhet a később lefoglalt lapokkal, ezért a verem számára külön területet foglalunk le a BSS-ben. A laborban egy 16 KiB méretű statikus tömböt használunk, amelyet a linker scriptben a `.bss` szekcióba helyezünk.

Példa belépő `entry.S`:

```
.section .text.init
.globl _start
_start:
    la sp, boot_stack_top     # stack pointer beállítása
    la t0, __bss_start
    la t1, __bss_end
1:
    bgeu t0, t1, 2f
    sd zero, 0(t0)
    addi t0, t0, 8
    j 1b
2:
    call kernel_main
    j .                       # ha a kernel_main visszatér, leállunk

.section .bss
.align 16
boot_stack:
    .skip 16384
boot_stack_top:
```

A `la` pszeudo-utasítás a szimbólum abszolút címét tölti be, ami a linker script miatt helyes. A BSS nullázása 8 bájtos lépésekben történik, mivel az `sd` 64 bites store-t végez. Fontos, hogy a nullázás előtt semmilyen C kódot nem szabad meghívni, mert a globális változók inicializálatlan értékei szemét adatokat tartalmaznának.

32.2.4 1.3. C nyelvű belépés

A `kernel_main` függvény a kernel további inicializálását végzi. Itt már C-ben dolgozhatunk, de figyeljünk arra, hogy a környezet freestanding: nincs szabványos C könyvtár, nincs `malloc`, nincs `printf` (legfeljebb saját soros konzol-illesztővel). A laborban a QEMU `-nographic` opciójával a soros konzolt (UART) használjuk a kiíratáshoz, de ezt a fejezetben nem részletezzük; a példákban feltételezzük, hogy létezik egy `uart_puts` függvény.

A `kernel_main` első feladata a fizikai memória felderítése. RISC-V QEMU `virt` gépen a memória a `0x80000000` címtől kezdődik, és a méretét a QEMU parancssori `-m` opciója adja meg. Ezt a kernel nem ismeri automatikusan, ezért vagy hardveresen kérdezi le (például a devicetree-ből vagy a RISC-V SBI-től), vagy a linker scriptben rögzítjük. A laborban a legegyszerűbb, ha a memória méretét fordítási időben konstansként definiáljuk, és a QEMU-t ugyanazzal az értékkel indítjuk. A fejezet későbbi részében bemutatjuk a memóriatérkép dinamikus lekérdezését is, de a záróprojektben a rögzített méret elegendő.

A kernel inicializálási sorrendje a következő:

1. Soros konzol inicializálása (a kiíratáshoz).
2. Frame allocator inicializálása a kernel által foglalt terület megjelölésével.
3. Lapozás beállítása (Sv39) azonossági leképezéssel.
4. Kernel-heap inicializálása.
5. Teszt: néhány lap lefoglalása és állapot kiíratása.

32.2.5 2. Fizikai memória és frame allocator

A frame allocator a fizikai memória lapjait (frame-ek) kezeli. A lapméret RISC-V Sv39 alatt 4 KiB (4096 bájt). A feladat: nyilvántartani, mely lapok szabadok, és lefoglalni azokat a kernel számára.

32.2.6 2.1. Bitmap alapú nyilvántartás

A legegyszerűbb megvalósítás egy bitmap: minden bit egy fizikai lapot reprezentál. A bit értéke 1, ha a lap foglalt, 0, ha szabad. A bitmap a kernel BSS szekciójában kap helyet, így nem kell külön memóriát foglalni a számára. A mérete a teljes fizikai memória méretétől függ:

$$\text{bitmap_méret} = \frac{\text{memória_méret}}{\text{lapméret} \times 8}$$

Például 128 MiB memória esetén:

$$\frac{128 \times 1024 \times 1024}{4096 \times 8} = 4096 \text{ bájt}$$

Ez elfogadhatóan kicsi. A laborban ezt a méretet statikusan definiáljuk.

A bitmap kezelése két alapvető műveletből áll: lap lefoglalása és felszabadítása. A lap indexét a fizikai címből számítjuk:

$$\text{index} = \frac{\text{cím} - \text{memória_kezdete}}{\text{lapméret}}$$

A `memória_kezdete` a `0x80000000`, tehát az első lap indexe 0.

A lefoglalás algoritmusa:

1. Keressük az első szabad bitet a bitmapben.
2. Ha találtunk, állítsuk 1-re, és számítsuk ki a lap fizikai címét.
3. Ha nincs szabad bit, térjünk vissza hibával (pl. `NULL` vagy `-1`).

A felszabadítás a bit 0-ra állításából áll, de ellenőrizni kell, hogy a lap valóban le volt-e foglalva (dupla felszabadítás elkerülése).

32.2.7 2.2. Inicializálás és a kernel területének kizárása

Az inicializáláskor a kernel által használt fizikai területet foglaltnak kell jelölni. Ez magában foglalja:

- A kernel kódját, adatait és BSS szekcióját a `0x80000000` címtől a `__kernel_end` szimbólumig (beleértve a stacket is).
- A bitmap területét is, mivel az a BSS-ben van, tehát a fenti tartomány része.
- A jövőben lefoglalásra kerülő lapasztal-táblákat (ezeket később a frame allocatorból fogjuk kérni).

A foglalt tartomány kiszámítása:

```
extern char __kernel_end[];

void frame_init(void) {
    uintptr_t kernel_start = 0x80000000;
    uintptr_t kernel_end = (uintptr_t)__kernel_end;
    uintptr_t mem_end = 0x80000000 + MEMORY_SIZE;

    // A teljes memóriát szabadnak jelöljük
    memset(bitmap, 0, BITMAP_SIZE);

    // A kernel területét foglaltnak jelöljük
    uintptr_t addr;
    for (addr = kernel_start; addr < kernel_end; addr += PAGE_SIZE) {
        frame_mark_used(addr);
    }

    // A bitmap saját területe már a kernel BSS-ben van, tehát lefedett.
}
```

A `frame_mark_used` függvény a címből indexet számol, és beállítja a bitet. A `MEMORY_SIZE` a QEMU `-m` paraméterével egyezik meg, és fordítási időben konstans.

32.2.8 2.3. Frame allocator API

A kernel többi része számára két függvényt biztosítunk:

```
void *frame_alloc(void);
void frame_free(void *frame);
```

A `frame_alloc` visszaadja a lefoglalt lap fizikai címét, vagy `NULL`-t, ha nincs szabad lap. A `frame_free` felszabadítja a lapot, feltételezve, hogy a cím lapigazított.

A lefoglalás implementációja bitmap-kereséssel:

```
void *frame_alloc(void) {
    for (size_t i = 0; i < BITMAP_SIZE * 8; i++) {
        if (!bitmap_test(i)) {
            bitmap_set(i);
            return (void *)(0x80000000 + i * PAGE_SIZE);
        }
    }
    return NULL;
}
```

A `bitmap_test` és `bitmap_set` egyszerű bitműveletekkel valósítható meg. Mivel a kernel egyelőre egyprocesszoros, nem kell zárolás, de a későbbi SMP-támogatásnál atomi műveletekre lesz szükség.

32.2.9 2.4. Tipikus hibák a frame allocatorban

- **A kernel területének rossz kizárása:** ha a kernel végét (`__kernel_end`) hibásan számítjuk, a bitmap olyan lapokat is foglaltnak jelöl, amelyek valójában szabadok, vagy ami rosszabb, szabadnak hagyja a kernel kódját, és a későbbi lefoglalás felülírja azt. A linker scriptben az `ALIGN(4096)` biztosítja, hogy a szakaszhatárok lapigazítottak legyenek.
- **Bitkép méretének alulméretezése:** ha a `MEMORY_SIZE` kisebb, mint a QEMU-ban beállított memória, a legfelső lapokhoz tartozó bitek hiányoznak, és a lefoglalás hibásan működik. A laborban ezt a konfigurációt rögzíteni kell.
- **Dupla felszabadítás:** ha egy lapot kétszer szabadítunk fel, a bitmapben a bit 0-ra áll, de a lap továbbra is használatban lehet, ami később sérülést okoz. A `frame_free`-ban ellenőrizni kell, hogy a bit 1-es volt-e, és hibát jelezni, ha nem.

32.2.10 3. Lapozás és lapasztal

A RISC-V Sv39 lapozás háromszintű lapasztalt használ: a legfelső szint a **PGD** (Page Global Directory), a középső a **PMD** (Page Middle Directory), az alsó pedig a **PTE** (Page Table Entry). Mindegyik tábla 512 darab 64 bites bejegyzést tartalmaz, minden bejegyzés egy 4 KiB-os lapot vagy egy alsóbb szintű táblát címez. A virtuális cím 39 bitjét három mezőre bontjuk: a 9 bites indexek rendre a PGD, PMD és PTE bejegyzést választják ki, a maradék 12 bit a lapon belüli eltolás.

A virtuális cím dekódolása:

$$\text{PGD_index} = (\text{vaddr} \gg 30)\ \&\ 0x1FF$$

$$\text{PMD_index} = (\text{vaddr} \gg 21)\ \&\ 0x1FF$$

$$\text{PTE_index} = (\text{vaddr} \gg 12)\ \&\ 0x1FF$$

$$\text{offset} = \text{vaddr}\ \&\ 0xFFF$$

A lapbejegyzés formátuma Sv39 alatt: a fizikai lapcím a 10. bittől a 53. bitig terjed, a 0–9. bitek pedig flag-eket tartalmaznak. A legfontosabb flag-ek:

- **V** (Valid, bit 0): a bejegyzés érvényes.
- **R** (Readable, bit 1): olvasási jog.
- **W** (Writable, bit 2): írási jog.
- **X** (Executable, bit 3): végrehajtási jog.
- **U** (User, bit 4): felhasználói módban elérhető.
- **A** (Accessed, bit 6): a hardver beállítja, ha a laphoz hozzáfértek.
- **D** (Dirty, bit 7): a hardver beállítja, ha a lapot írták.

A kernel inicializálásakor a lapozást azonossági leképezéssel állítjuk be: a fizikai cím megegyezik a virtuális címmel. Ez lehetővé teszi, hogy a kernel továbbra is ugyanazon a címen fusson, miközben a lapozás már aktív. A lapasztal táblákat a frame allocatorból foglaljuk le.

A leképezés felépítésének lépései:

1. Foglaljunk egy lapot a PGD számára (`frame_alloc`).
2. Inicializáljuk a PGD-t nullákkal (minden bejegyzés érvénytelen).
3. A kernel által használt fizikai tartomány minden lapjára hozzunk létre bejegyzést a lapasztalban. Mivel azonossági leképezést végzünk, a virtuális cím megegyezik a fizikaival, így a leképezés lineáris.

4. A bejegyzésben állítsuk be a V, R, W, X flag-eket (kernel módban az U flag nem kell).
5. Töltsük be a PGD fizikai címét a `satp` regiszterbe (a lapozás engedélyezése).

A `satp` regiszter formátuma RISC-V Sv39 esetén: a 44. bittől a 63. bitig a PGD fizikai címe (lapigazított), az alsó 44 bit a módot és az ASID-t tartalmazza. A mód `8` a Sv39 esetén. A beállítás:

$$\text{satp} = (\text{PGD_fizikai_cím} \gg 12) \mid (8 \ll 60)$$

A lapozás engedélyezése után a CPU minden memóriahozzáférést a lapasztalon keresztül végez. Ezért kritikus, hogy a leképezés azonnal helyes legyen, különben a következő utasítás is page faultot okoz.

32.2.11 3.1. Lapbejegyzés létrehozása

A lapasztal felépítéséhez egy rekurzív vagy manuális bejárást használhatunk. A laborban a legegyszerűbb, ha a kernel a saját leképezését egy ciklusban, laponként hozza létre. Minden laphoz megkeressük vagy létrehozzuk a megfelelő PGD, PMD és PTE bejegyzést, és beállítjuk a fizikai címet és a flag-eket.

Egy segédfüggvény a virtuális cím leképezésére:

```
void vm_map(uintptr_t vaddr, uintptr_t paddr, uint64_t flags) {
    uint64_t *pgd = (uint64_t *)read_satp_pgd();
    uint64_t pgd_index = (vaddr >> 30) & 0x1FF;
    uint64_t pmd_index = (vaddr >> 21) & 0x1FF;
    uint64_t pte_index = (vaddr >> 12) & 0x1FF;

    if (!(pgd[pgd_index] & V)) {
        uint64_t *pmd = frame_alloc();
        memset(pmd, 0, PAGE_SIZE);
        pgd[pgd_index] = ((uint64_t)pmd >> 12 << 10) | V;
    }

    uint64_t *pmd = (uint64_t *)((pgd[pgd_index] >> 10) << 12);
    if (!(pmd[pmd_index] & V)) {
        uint64_t *pte = frame_alloc();
        memset(pte, 0, PAGE_SIZE);
        pmd[pmd_index] = ((uint64_t)pte >> 12 << 10) | V;
    }

    uint64_t *pte = (uint64_t *)((pmd[pmd_index] >> 10) << 12);
    pte[pte_index] = ((paddr >> 12) << 10) | flags | V;
}
```

Ez a függvény a `satp` regiszterből kiolvassa a PGD fizikai címét, és manuálisan végigmegy a három szinten. A `flags` paraméter tartalmazza az R, W, X biteket, valamint az A és D biteket (ezeket a hardver állítja, de gyakran előre beállítjuk őket, hogy elkerüljük a page faultot a hardveres dirty bit frissítésekor, ha nincs megfelelő kezelés). A laborban az egyszerűség kedvéért az A és D biteket is beállítjuk.

32.2.12 3.2. Azonossági leképezés beállítása

A `kernel_main` a frame allocator inicializálása után meghívja a `vm_init` függvényt, amely a kernel teljes fizikai tartományát leképezi:

```
void vm_init(void) {
    uintptr_t kernel_start = 0x80000000;
    uintptr_t kernel_end = (uintptr_t)__kernel_end;

    for (uintptr_t addr = kernel_start; addr < kernel_end; addr += PAGE_SIZE) {
        vm_map(addr, addr, PTE_R | PTE_W | PTE_X);
    }

    // A lapozás engedélyezése
    uint64_t satp = ((uint64_t)read_satp_pgd() >> 12) | (8UL << 60);
    csr_write(satp, satp);
    sfence_vma();
}
```

A `csr_write` és `sfence_vma` assembly utasítások, amelyek a `satp` regisztert írják, és érvénytelenítik a TLB-t. A `read_satp_pgd` a jelenlegi PGD fizikai címét adja vissza, amelyet a `vm_map` előtt lefoglaltunk.

Fontos, hogy a leképezés csak a kernel területére vonatkozik; a későbbiekben a heap és a felhasználói folyamatok lapjait külön kell leképezni.

32.2.13 3.3. Tipikus hibák a lapozásnál

- **Hibás lapbejegyzés mező:** a fizikai cím eltolása rossz (pl. `>> 10` helyett `>> 12`), ami rossz fizikai lapra mutat.
- **Hiányzó V bit:** ha a bejegyzés érvénytelen, a CPU page faultot vált ki, és a kernel leáll (panik).
- **Nem érvénytelenített TLB:** a `sfence_vma` elhagyása után a régi, lapozás nélküli címfordítás maradhat érvényben, ami kiszámíthatatlan viselkedést okoz.
- **Kernel-heap lapjainak hiányzó leképezése:** ha a heap a kernel `__kernel_end` utáni területre kerül, de a lapozás inicializálásakor csak a kernel területét képeztük le, a heap első használata page faultot okoz. Ezért a heap inicializálását a lapozás beállítása után kell végezni, vagy a heap területét is le kell képezni.

32.2.14 4. Kernel-heap: egyszerű lapalapú allokátor

A frame allocator 4 KiB-os lapokat ad, de a kernelben gyakran kisebb memóriadarabokra van szükség (például struktúrák, pufferek). Erre szolgál a kernel-heap, amely a frame allocatorra épülve kisebb blokkokat oszt ki.

A záróprojektben egy egyszerű **bump allocatort** valósítunk meg: a heap egyetlen mutatóval rendelkezik, amely a lefoglalt terület végét jelzi. Amikor memóriát kérünk, a mutatót felfelé mozgatjuk, és visszaadjuk a régi címet. Felszabadítás nincs, vagy csak a teljes heap visszaállítása lehetséges. Ez a legegyszerűbb megoldás, és a kernel korai szakaszában elegendő. A későbbi fejezetekben (Ütemező, rendszerhívások és shell) majd finomabb allokátorra cserélhetjük.

A heap inicializálása:

```c
static uintptr_t heap_start;
static uintptr_t heap_end;
static uintptr_t heap_brk;

void heap_init(void *start, size_t size) {
    heap_start = (uintptr_t)start;
    heap_end = heap_start + size;
    heap_brk = heap_start;
}

void *kmalloc(size_t size) {
    size = (size + 15) & ~15;  // 16 bájtos igazítás
    if (heap_brk + size > heap_end)
        return NULL;
    void *ptr = (void *)heap_brk;
    heap_brk += size;
    return ptr;
}
```

A `heap_init` a kernel végétől (`__kernel_end`) induló területet használja. A méretet úgy választjuk meg, hogy elegendő legyen a kezdeti igényekhez, például 1 MiB. A lapozás inicializálásakor ezt a heap-területet is le kell képezni azonossági módon, különben a `kmalloc` által visszaadott cím elérése page faultot vált ki.

A bump allocator előnye az egyszerűség, hátránya, hogy nem lehet egyes blokkokat felszabadítani, és a heap töredezik, ha sok különböző méretű allokáció történik. A laborban ez elfogadható, mert a kernel még nem fut hosszú ideig.

32.2.15 5. Labor: bootolható kernel és frame allocator

32.2.16 5.1. Cél

A labor során egy QEMU-ban futó, bootolható RISC-V kernelt készítünk, amely:

- Inicializálja a soros konzolt.
- Felderíti a fizikai memóriát.
- Inicializálja a frame allocatort, és kizárja a kernel területét.
- Felépíti a Sv39 lapozást azonossági leképezéssel.
- Inicializál egy bump heapet.
- Lefoglal néhány lapot, és kiírja az állapotot a konzolra.

32.2.17 5.2. Konfiguráció és build

A labor emulált környezetben fut, a hoszt valódi rendszerét nem módosítjuk. A használt eszközök:

- QEMU 8.2 (vagy újabb) RISC-V `virt` gép, 128 MiB RAM
- RISC-V GCC cross-compiler (riscv64-unknown-elf-gcc)
- GNU Make

A build parancsok:

```
riscv64-unknown-elf-gcc -mcmodel=medany -ffreestanding -nostdlib \
    -T kernel.ld -o kernel.elf entry.S kernel.c frame.c vm.c heap.c
riscv64-unknown-elf-objcopy -O binary kernel.elf kernel.bin
```

A `-mcmodel=medany` biztosítja, hogy a kód a `0x80000000` körüli címeket elérje. A futtatás:

```
qemu-system-riscv64 -machine virt -m 128M -nographic -kernel kernel.elf
```

A `-kernel` opció közvetlenül betölti az ELF fájlt, így a belépési pont az ELF fejlécből származik.

32.2.18 5.3. Mérési napló

A mérés során a következő értékeket rögzítjük:

Mérés	Módszer	Várt érték	Mért érték
Kernel méret (text+data+bss)	`riscv64-unknown-elf-size kernel.elf`	< 64 KiB	48 KiB
Frame allocator által kezelt lapok száma	`MEMORY_SIZE / 4096`	32768 lap	32768 lap
Lefoglalt lapok a kernel számára	A kernel végét lapokra kerekítve	13 lap	13 lap
Boot idő a `kernel_main` belépésétől a teszt kiírásáig	CPU ciklusszámláló (`rdcycle`) a belépéskor és a teszt után	$< 100\,000$ ciklus	85 200 ciklus

A `rdcycle` utasítás a RISC-V ciklusszámlálóját olvassa, amely a CPU órajelével növekszik. A QEMU `virt` gép órajele alapértelmezés szerint 1 GHz, de ez nem pontos valós idő; a mért ciklusok jól reprodukálhatók. A várt értékek a labor kiinduló állapotából származnak, a mért értékek a tényleges futásból. A kettőt mindig külön kell dokumentálni, és az eltérést magyarázni kell.

32.2.19 5.4. Elvárt eredmény

A kernel indítása után a soros konzolon a következőhöz hasonló kimenet jelenik meg:

```
Kernel booted at 0x80000000
Frame allocator initialized: 32768 frames, 13 used
Page table initialized: identity mapping
Heap initialized: 1 MiB at 0x8000C000
Test: allocated frame at 0x80100000
```

A konkrét címek a kernel méretétől függenek, de a szerkezet azonos.

32.2.20 5.5. Önellenőrző kérdés

Miért kell a BSS szekciót nullázni a kernel belépésekor? Mi történne, ha elmaradna, és hogyan befolyásolná a frame allocator működését?

Válasz vázlat: A BSS szekció a nullával inicializált globális változókat tartalmazza. A C szabvány szerint ezek kezdeti értéke nulla. Ha a belépő kód nem nullázza, a változók szemét értéket kapnak, ami a bitmap inicializálásakor hibás állapotot eredményezhet: a bitmap bitek

véletlenszerűen 1-esek lehetnek, így a frame allocator szabadnak hitt lapokat foglaltnak jelöl, vagy fordítva. A kernel viselkedése kiszámíthatatlanná válik.

32.2.21 6. Összefoglalás és továbbvezetés

Ebben a fejezetben felépítettük a záróprojekt kernelének alapjait: a bootolható ELF képet, a belépő assembly kódot, a BSS nullázást, a frame allocatort bitmap alapon, a Sv39 lapozást azonossági leképezéssel, valamint egy egyszerű kernel-heapet. A laborban mindezt QEMU-ban futtattuk, és a mérési naplóval dokumentáltuk a várt és mért értékeket.

A következő fejezet (Ütemező, rendszerhívások és shell) erre az alapra épít: bevezeti az ütemezőt, a rendszerhívásokat és a shellt. A frame allocator és a lapozás lesz az alapja a folyamatok címtereinek és a kernel-heapnek, ezért fontos, hogy ezek a komponensek stabilan működjenek. A tipikus hibák elkerülése érdekében minden lépést reprodukálható módon, a mérési naplóval együtt kell dokumentálni.

A záróprojekt további részében a memóriakezelőt ki kell egészíteni felhasználói címterek leképezésével, a frame allocatort zárolással kell ellátni (ha SMP-t támogatunk), és a heapet hatékonyabb allokátorra cserélhetjük. Ezen feladatok azonban már a következő mérföldkövekhez tartoznak.

32.3 Ütemező, rendszerhívások és shell

A záróprojekt ezen pontján már van egy bootoló, memóriakezeléssel rendelkező kernel. Ebben a fejezetben a rendszer futtathatóvá válik: megvalósítjuk a feladatmodellt, a round-robin ütemezőt, a rendszerhívás-belépési pontot, az `init` folyamatot és egy minimális shellt. A cél nem egy általános célú operációs rendszer teljes körű reprodukálása, hanem egy olyan zárt, ellenőrizhető kernel, amelyben minden fontos állítás kódolvasással vagy mérési naplóval alátámasztható.

A fejezet feltételezi, hogy az `os123` egység után rendelkezésre áll egy bootoló kernel, egy egyszerű memóriatérkép és egy buildrendszer. Az itt bemutatott kódrészletek RISC-V példát használnak, mert a QEMU `virt` gépe jól reprodukálható; az elv azonban x86-on vagy más architektúrán is azonos. A konkrét regiszterneveket és utasításokat mindig az adott porton ellenőrizni kell.

32.3.1 1. Feladatmodell és adatstruktúrák

A futtatható egységet itt *feladatnak* nevezzük. A feladat lehet kernelkontextusban induló szál, de a fejezet szempontjából a felhasználói program futtatása a lényeges eset. A feladatmodell minimális, de kellően bővíthető ahhoz, hogy az `os125` fájlrendszer-integrációt rá lehessen építeni.

A központi struktúra a PCB (*process control block*), amelyet itt `task` néven valósítunk meg:

```
/* task.h - a minimalis feladatmodell */
#define TASK_UNUSED  0
#define TASK_READY   1
#define TASK_RUNNING 2
#define TASK_SLEEP   3
#define TASK_ZOMBIE  4

#define KSTACK_SIZE  4096

struct cpu_context {
    uint64_t ra;      /* visszateresi cim */
    uint64_t sp;      /* kernel veremmutato */
```

```
    uint64_t s0;
    uint64_t s1;
    uint64_t s2;
    uint64_t s3;
    uint64_t s4;
    uint64_t s5;
    uint64_t s6;
    uint64_t s7;
    uint64_t s8;
    uint64_t s9;
    uint64_t s10;
    uint64_t s11;
};

struct task {
    uint64_t pid;
    int      state;
    uint64_t kernel_stack_top;
    struct cpu_context ctx;
    uint64_t user_pc;
    uint64_t user_sp;
    uint64_t satp;            /* hasznalt lapkeptabla, ha van MMU */
    uint64_t trapframe_ptr;   /* user->kernel belepesi munkaterulet */
    uint64_t quantum_left;    /* hatralevo idoszelet, tickben */
    struct task *next;
};
```

A `cpu_context` a callee-saved regisztereket tartalmazza. Ez a kiválasztás fontos: egy C függvényből hívott `switch_to` után a hívó csak azokat a regisztereket látja konzisztensnek, amelyeket a hívott függvénynek meg kell őriznie. A caller-saved regisztereket a kontextusváltást hívó kódnak nem kell külön mentenie, mert a C hívási konvenció szerint a hívó már elmentette őket, ha szüksége van rájuk.

A `state` mező a folyamatállapot-gépet írja le. A állapotátmenetek:

```
UNUSED -> READY        (letrehozas, init)
READY   -> RUNNING     (utemezo kivalasztja)
RUNNING -> READY       (idoszelet lejart)
RUNNING -> SLEEP       (rendszerhivas varakozasra kenyszeriti)
SLEEP   -> READY       (esemeny megerkezett)
RUNNING -> ZOMBIE      (kilepes)
ZOMBIE  -> UNUSED      (takaritas, szulo begyujtotte)
```

A kezdeti kernel a `boot` szakasz után létrehoz egy `init` feladatot, amely az első felhasználói programot futtatja. A további feladatok ebből a feladatból jönnek létre rendszerhívással, vagy a kernel indítja őket statikusan.

A feladatok egy egyszerű, körkörösen láncolt listában állnak. A round-robin ütemező ebből választja a következő `TASK_READY` állapotú elemet. A lista bejárása a legegyszerűbb esetben egy mutatóval történik, így a döntés komplexitása $\mathcal{O}(1)$ amortizáltan, ha a lista hossza nem változik gyakran, és $\mathcal{O}(n)$ a teljes bejárás, ha minden ciklusban a lista hosszát be kell járni. A gyakorlatban a lista hossza a záróprojektben nagyon kicsi, ezért a lineáris bejárás is elfogadható.

32.3.2 2. Timer interrupt és round-robin ütemezés

A round-robin ütemező alapja a hardveres időzítő megszakítása. A záróprojektben a QEMU `virt` gép CLINT (`Core Local Interruptor`) eszközét használjuk, amely periodikus megszakítást ad. A timer periódusát rendszerint 10 ms-ra állítjuk, de ez állítható. A konfiguráció a `kernel/timer.c`-ben történik:

```
/* timer.c - periodikus timer megszakitas beallitasa RISC-V CLINT */
void timer_init(void) {
    uint64_t interval = TIMER_INTERVAL; /* pl. 10 ms ciklusokban */
    uint64_t next = r_mtime() + interval;
    w_mtimecmp(next);
    /* a megszakitas engedelyezese a trap keretben tortenik */
}
```

A timer megszakítás kezelője `tick`-et ad. Ha az aktuális feladat `quantum_left` értéke eléri a nullát, a kernel beállítja a feladatot `TASK_READY` állapotba, és meghívja az ütemezőt:

```
void timer_interrupt_handler(void) {
    if (current && current->state == TASK_RUNNING) {
        current->quantum_left--;
        if (current->quantum_left == 0) {
            current->state = TASK_READY;
            schedule();
        }
    }
}
```

A `schedule` függvény kiválasztja a lista következő futtatható elemét, és ha az eltér az aktuálistól, kontextust vált:

```
void schedule(void) {
    struct task *next = pick_next_task();
    if (next && next != current) {
        struct task *prev = current;
        current = next;
        current->state = TASK_RUNNING;
        current->quantum_left = QUANTUM;
        switch_to(&prev->ctx, &current->ctx);
    }
}
```

A `pick_next_task` a lista első `TASK_READY` elemét adja. Ez biztosítja a körkörös továbbadást, ha a lista végén lévő mutatót mindig az aktuális feladatra állítjuk. A round-robin Gantt-diagramja három feladattal, 10 ms időkerettel:

```
P1: 0---10 30---40 60---70
P2: 10--20 40---50 70---80
P3: 20--30 50---60 80---90
```

Az átlagos várakozási idő ebben a példában, ha mindhárom feladat 10 ms-ot használ fel: P_1 0 ms, P_2 10 ms, P_3 20 ms, átlag 10 ms. A valóságban, ha egy feladat letiltja a megszakításokat vagy rendszerhíváson keresztül blokkol, az átlagos várakozási idő nő.

A kontextusváltás a kernel legalacsonyabb szintű része, ezért assemblyben írjuk. A RISC-V mentési és visszatöltési sorrendje:

```
# switch_to(a0 = &old_ctx, a1 = &new_ctx)
.globl switch_to
switch_to:
    sd ra, 0(a0)
    sd sp, 8(a0)
    sd s0, 16(a0)
    sd s1, 24(a0)
    sd s2, 32(a0)
    sd s3, 40(a0)
    sd s4, 48(a0)
    sd s5, 56(a0)
    sd s6, 64(a0)
```

```
    sd s7, 72(a0)
    sd s8, 80(a0)
    sd s9, 88(a0)
    sd s10, 96(a0)
    sd s11, 104(a0)

    ld ra, 0(a1)
    ld sp, 8(a1)
    ld s0, 16(a1)
    ld s1, 24(a1)
    ld s2, 32(a1)
    ld s3, 40(a1)
    ld s4, 48(a1)
    ld s5, 56(a1)
    ld s6, 64(a1)
    ld s7, 72(a1)
    ld s8, 80(a1)
    ld s9, 88(a1)
    ld s10, 96(a1)
    ld s11, 104(a1)
    ret
```

A `ra` és `sp` mentése/utolsó betöltése nem triviális: a `ret` a `ra` értékét használja, ezért a `ra`-t utoljára kell betölteni. A fenti sorrendben a `ret` előtt az `ra` már az új kontextus visszatérési címét tartalmazza. A `sp` pedig az új kernel veremre mutat.

Fontos tervezési döntés, hogy a kontextusváltás mindig kernel módból kernel módba történik. A felhasználói regiszterek mentését nem itt, hanem a trap belépésnél kell elvégezni. Ha a kernel a felhasználói feladatot timer interrupt miatt váltja, a felhasználói regisztereket a `trapframe` már tartalmazza. Ezért a `task` struktúrában lévő `trapframe_ptr` mutat erre a munkaterületre.

32.3.3 3. Trap belépés és rendszerhívás-dispatcher

A felhasználói programok futtatásához a kernelnek meg kell valósítania a user/kernel átlépést. RISC-V-n ez az `ecall` utasításon keresztül történik. A felhasználói program a syscall számot az `a7` regiszterbe, az argumentumokat az `a0`–`a5` regiszterekbe teszi, majd `ecall`-t hajt végre. A hardver átvált kernel módra, és a `trap_vector`-ben lévő kódra ugrik.

A trap belépés során a kernelnek el kell mentenie a felhasználói regisztereket egy `trapframe` struktúrába:

```
struct trapframe {
    uint64_t ra;
    uint64_t sp;
    uint64_t gp;
    uint64_t tp;
    uint64_t t0;
    uint64_t t1;
    uint64_t t2;
    uint64_t s0;
    uint64_t s1;
    uint64_t a0;
    uint64_t a1;
    uint64_t a2;
    uint64_t a3;
    uint64_t a4;
    uint64_t a5;
    uint64_t a6;
    uint64_t a7;
    uint64_t sepc;    /* visszateresi cim user modba */
```

```
    uint64_t sstatus; /* elozo privilégium szint */
};
```

A `trap_vector` assembly kód először a kernel stackre lép, elmenti a regisztereket, meghívja a C `trap_handler` függvényt, majd visszatöltve a regisztereket `sret`-tel visszalép user módba. A részletes assembly a `os107`-ben már bemutatásra került; itt a rendszerhívás-dispatcherre összpontosítunk.

A C oldali `trap_handler` megvizsgálja a megszakítás okát, és ha `ecall`-t észlel, a `syscall_dispatch` függvényt hívja:

```
void trap_handler(struct trapframe *tf) {
    uint64_t cause = r_scause();
    if (cause == SCAUSE_ECALL_USER) {
        syscall_dispatch(tf);
        tf->sepc += 4; /* RISC-V: továbblepunk az ecall utanira */
        return;
    }
    if (cause == SCAUSE_TIMER) {
        timer_interrupt_handler();
        return;
    }
    panic("ismeretlen trap ok");
}
```

A `syscall_dispatch` a rendszerhívási táblát használja. A tábla egy függvénypointer-tömb, ahol az index a syscall szám:

```
typedef int (*syscall_fn_t)(uint64_t, uint64_t, uint64_t,
                            uint64_t, uint64_t, uint64_t);

syscall_fn_t syscall_table[] = {
    [SYS_WRITE] = sys_write,
    [SYS_READ]  = sys_read,
    [SYS_EXIT]  = sys_exit,
    [SYS_FORK]  = sys_fork,   /* kesobb */
    [SYS_PS]    = sys_ps,
    [SYS_SLEEP] = sys_sleep,
};

void syscall_dispatch(struct trapframe *tf) {
    int nr = tf->a7;
    if (nr < 0 || nr >= ARRAY_SIZE(syscall_table) || !syscall_table[nr]) {
        tf->a0 = -ENOSYS;
        return;
    }
    tf->a0 = syscall_table[nr](tf->a0, tf->a1, tf->a2,
                               tf->a3, tf->a4, tf->a5);
}
```

A diszpécser feladata kettős: egyrészt a syscall számot ellenőrzi, másrészt a visszatérési értéket az `a0` regiszterbe írja. A hibakódot az `errno`-hoz hasonlóan negatív értékként adjuk vissza. A felhasználói oldali libc-wrapper ezt lefordítja `-1`-re és beállítja az `errno`-t. Például:

```
/* user/lib/syscall.c */
ssize_t write(int fd, const void *buf, size_t n) {
    ssize_t ret = syscall(SYS_WRITE, fd, (uint64_t)buf, n, 0, 0, 0);
    if (ret < 0) {
        errno = -ret;
        return -1;
    }
```

```
    return ret;
}
```

A rendszerhívások sikeres végrehajtása nem írhatja felül a felhasználói memóriát ellenőrizetlenül. A `sys_write` például a felhasználói puffer címeit a kernel címfordítási tartományába kell konvertálnia, és meg kell vizsgálnia, hogy az adott tartomány valóban a felhasználóhoz tartozik. A minimális kernelben ez egy egyszerű határellenőrzés:

```
int sys_write(int fd, uint64_t user_buf, int n, ...) {
    if (fd != 1 && fd != 2) return -EBADF;
    if (n < 0 || user_buf + n > USER_TOP) return -EFAULT;
    char *buf = (char *)user_buf;
    for (int i = 0; i < n; i++) {
        uart_putc(buf[i]);
    }
    return n;
}
```

A valós kernelben a `copy_from_user` és a `copy_to_user` külön ellenőrzött másolást végez. Itt a cél a didaktikus szerkezet, nem a teljes biztonsági modell.

32.3.4 4. Init folyamat és felhasználói programok betöltése

A kernel inicializálása után az `init` feladat létrehozása a következő lépésekből áll:

1. A kernel lefoglal egy `task` struktúrát és egy kernel stacket.
2. Beállítja a feladat állapotát `TASK_READY`-re, a PID-et 1-re.
3. A felhasználói belépési pontot `user_pc`-be teszi.
4. A felhasználói stacket megfelelően beállítja.
5. A feladatot hozzáadja az ütemező listához.
6. A `sret`-hez szükséges `sstatus` mezőt user módba állítja.

Az `init` programot a kernelbe ágyazzuk, vagy egy egyszerű `initrd`-ből töltjük be. A záróprojektben a legegyszerűbb, ha a `init` bináris kódját a kernel ELF szekciójába linkeljük, és a kernel a `init_start`/`init_end` szimbólumokból másolja a felhasználói memóriába.

Példa a betöltésre:

```
void init_task_create(void) {
    struct task *t = alloc_task();
    t->pid = 1;
    t->state = TASK_READY;
    t->kernel_stack_top = alloc_kstack();
    t->user_pc = (uint64_t)init_entry;
    t->user_sp = USER_STACK_TOP;
    t->satp = kernel_satp; /* vagy user lapkeptabla */
    t->quantum_left = QUANTUM;
    list_add(&task_list, t);
}
```

A felhasználói `init` program a kernelben előre definiált ELF szimbólumként vagy egy egyszerű C függvényként szerepel, amelyet a kernel user módban indít el. A `sret` úgy tér vissza, hogy a PC az `init_entry` értékét veszi fel.

A `init` program első feladata, hogy létrehozza a shell folyamatot, majd várakozzon a gyermek folyamatok befejeződésére. A minimális kernelben a `fork` és `wait` rendszerhívásokat is implementálni kell. Ha a teljesség nem szükséges, a shell közvetlenül az init-ben is futtatható, de ez korlátozza a többfeladatos tesztelést. A laborban ezért `fork` és `wait` is szerepel.

Egy egyszerű `fork` implementáció:

```
int sys_fork(void) {
    struct task *child = copy_task(current);
    if (!child) return -ENOMEM;
    child->pid = next_pid++;
    child->state = TASK_READY;
    /* a gyermek visszateresi erteke 0 lesz */
    child->trapframe->a0 = 0;
    list_add(&task_list, child);
    return child->pid;
}
```

A `copy_task` le kell másolja a task struktúrát, a kernel stacket, a trapframe-et és a felhasználói memóriát. Ez a záróprojektben egyszerű fizikai másolással megoldható, ha a memóriakezelő rendelkezik `memcpy`-vel és a címek fizikailag azonosak. Ha lapozás aktív, a felhasználói lapokat is másolni kell, vagy COW-t kell alkalmazni. A laborban az egyszerűség kedvéért feltételezzük, hogy a lapozás még nem aktív, és a felhasználói memória fizikailag közvetlenül másolható.

32.3.5 5. Minimális shell

A shell egy felhasználói program, amely a szabványos bemenetről olvas, parancsokat értelmez, és a szabványos kimenetre ír. A minimális shell a következő parancsokat ismeri:

- `help`: kiírja a parancsok listáját.
- `echo <szöveg>`: visszaírja a szöveget.
- `ps`: a kernel `sys_ps` hívásával listázza a feladatokat.
- `sleep <ms>`: a `sys_sleep` hívásával várakozik.
- `shutdown`: a `sys_exit` hívással leállítja a rendszert.

A shell kódja:

```
/* user/init.c */
void main(void) {
    char buf[128];
    while (1) {
        puts("$ ");
        int n = read(0, buf, sizeof(buf) - 1);
        if (n <= 0) {
            continue;
        }
        buf[n] = 0;
        if (strcmp(buf, "help") == 0) {
            puts("echo <text>  - kiirja a szoveget\n");
            puts("ps           - feladatok listazasa\n");
            puts("sleep <ms>   - varakozas\n");
            puts("shutdown     - leallitas\n");
        } else if (strncmp(buf, "echo ", 5) == 0) {
            puts(buf + 5);
            puts("\n");
        } else if (strcmp(buf, "ps") == 0) {
            ps();
        } else if (strncmp(buf, "sleep ", 6) == 0) {
            int ms = atoi(buf + 6);
            sleep(ms);
        } else if (strcmp(buf, "shutdown") == 0) {
            exit(0);
        } else {
            puts("ismeretlen parancs: ");
```

```
            puts(buf);
            puts("\n");
        }
    }
}
```

A shell működése bemutatja a rendszerhívások használatát: a `read` a `SYS_READ`-et, a `puts` a `SYS_WRITE`-et, a `ps` a `SYS_PS`-et, az `exit` a `SYS_EXIT`-et hívja. A `fork`-ot a shell indításakor a kernelben, vagy az `init` felhasználói programban használjuk.

A `sys_ps` implementációja a kernelben bejárja a feladatlistát, és a feladat állapotát egy belső pufferbe írja, amit a felhasználói program a `SYS_PS` hívással kérhet le. A kimeneti formátum egyszerű, például:

```
PID  STATE   PC
1    running 0x1000
2    ready   0x1000
3    sleep   0x1000
```

A shell tehát nem közvetlenül éri el a kernel adatszerkezeteit, hanem a rendszerhíváson keresztül kapja az adatokat. Ez a határ a tanulságos: a felhasználói program soha nem láthatja a PCB lista pointereit.

32.3.6 6. Hibakezelés és tipikus hibák

A fenti komponensek megvalósítása során több tipikus hibát kell elkerülni. A legfontosabbak:

1. **A trapframe nem mentett regiszterei.** Ha a `trap_vector` nem menti el az összes felhasználói regisztert, a rendszerhívásból való visszatérés után a felhasználói program korrupt regisztereket lát. Ez a hiba gyakran csak bizonyos syscall-oknál jelentkezik, mert a compiler a többi regisztert nem használja abban a függvényben. A megoldás: a `trapframe` minden felhasználói regisztert tartalmazzon, és a mentés/visszatöltés szigorúan szimmetrikus legyen.

2. **A `sepc` nem növelése `ecall` után.** Ha a `syscall_dispatch` után nem növeljük a `sepc`-t, a visszatérés újra ugyanazt az `ecall`-t hajtja végre, ami végtelen ciklust okoz. A hiba jellemző tünete, hogy a shell az első rendszerhívás után lefagy.

3. **A felhasználói mutatók ellenőrzésének hiánya.** A kernelben a `sys_read` és `sys_write` közvetlenül használja a felhasználói címet. Ha a felhasználó rossz mutatót ad, a kernel saját memóriáját olvashatja vagy írhatja. Ezt a záróprojektben legalább tartományellenőrzéssel meg kell akadályozni. A teljes izolációhoz user page table és hardveres védelem szükséges.

4. **Versenyhelyzet a schedulerben.** A `schedule` hívásakor a megszakításokat le kell tiltani, különben a timer interrupt a kontextusváltás közben újra beléphet. A kernelben a megszakítás-tiltás a `r_sstatus` `SIE` bitjének törlésével történik. A `switch_to` előtt le kell tiltani, utána pedig a visszatéréskor engedélyezni.

5. **Kernel stack túlcsordulás.** A `KSTACK_SIZE` 4096 bájt elegendő a minimális kernelhez, de ha a `fork` vagy a `sys_ps` nagy helyi változót hoz létre, a stack átfolyhat. A hibát a kernel stack alján elhelyezett őrszem mezővel lehet észlelni, vagy egyszerűen a stack méretét kell növelni.

6. **A `fork` gyermek visszatérési értéke nem nulla.** Ha a gyermek folyamat a `fork`-ból ugyanazzal a PID-del tér vissza, mint a szülő, a shell logikája elromlik. A `copy_task` után a gyermek `trapframe->a0` értékét nullára kell állítani.

7. **A `wait` implementációja nem takarítja a `ZOMBIE`-t.** Ha a `wait` nem felszabadítja a gyermek `task` struktúráját, a rendszer feladatlistája lassan megtelik `TASK_ZOMBIE` elemekkel, és a `pick_next_task` egyre lassabb lesz. A megoldás a `wait`-ben a gyermek erőforrásainak felszabadítása.

A teszteléshez a következő negatív eseteket érdemes beilleszteni:

- Érvénytelen syscall szám hívása: a `SYS_DISPATCH` `-ENOSYS`-szel térjen vissza, ne `panic`-eljen.
- A `read` hívása a shellben 0 byte hosszúsággal: a kernel ne lépjen ki.
- A `fork` 100-szor egymás után: a rendszer ne telítse be a memóriát.
- A `sleep` közben a timer interrupt akkor is működjön, ha az aktuális feladat `TASK_SLEEP` állapotba kerül.

32.3.7 7. Labor: round-robin ütemező, syscallok és shell

A labor célja, hogy a fenti komponenseket egy emulált környezetben összeállítsuk és mérjük. A labor a `os123` állapotára épül, és a `os125` számára előkészíti a fájlrendszer-integrációt.

32.3.8 7.1 Kiinduló állapot

- QEMU `virt` gép, RISC-V 64, 128 MiB RAM, 1 CPU.
- Kernel forráskódja a `kernel/` könyvtárban.
- Boot és memóriakezelés az `os123`-ból.
- Buildrendszer: `Makefile`, linkerscript `kernel.ld`.
- Debug eszköz: GDB, QEMU monitor.

32.3.9 7.2 Build és futtatási parancsok

```
# a kernel forditas
make clean
make ARCH=riscv64

# futtatas QEMU alatt
make run ARCH=riscv64

# debug inditas GDB-vel
make debug ARCH=riscv64
```

A `make run` a következő QEMU parancsot futtatja:

```
qemu-system-riscv64 -machine virt -m 128M -nographic \
  -bios none -kernel kernel.bin \
  -serial mon:stdio
```

A mérési naplóba rögzíteni kell a QEMU verzióját, a host CPU-t, az órajelét és a fordítási beállításokat. A példa mérési napló:

```
QEMU version: 8.2.0
Host CPU: AMD Ryzen 5 5600X, 3.7 GHz
Host OS: Debian 12, kernel 6.1.0
Compile flags: -O2 -g -ffreestanding -fno-common -march=rv64gc
Measurement method: rdcycle ciklusszámláló, 1000 minta atlaga
```

32.3.10 7.3 Implementációs lépések

1. Hozd létre a `task.h` és `task.c` fájlokat, és implementáld a feladatlistát.
2. Implementáld a timer megszakítást a `timer.c`-ben.
3. Implementáld a `switch_to` assembly függvényt.
4. Készítsd el a trap belépést és a `trapframe` mentést.
5. Implementáld a `syscall_dispatch`-et és a syscall táblát.
6. Hozd létre az init feladatot és a shell user programot.
7. Kössd be a `fork`, `wait`, `exit` és `sleep` syscallokat.
8. Teszteld a `ps` parancsot és a `fork`-ot.

32.3.11 7.4 Elvárt eredmény

A boot után a soros konzolon a következő jelenik meg:

```
[KERN] boot successful
[INIT] starting shell
$ help
echo <text>  - kiirja a szoveget
ps           - feladatok listazasa
sleep <ms>   - varakozas
shutdown     - leallitas
$ echo Hello
Hello
$ ps
PID  STATE   PC
1    running 0x1000
2    ready   0x1000
$ sleep 100
$ shutdown
[KERN] system halted
```

A `ps` kimenetben a PID 1 az init/shell, a PID 2 egy második folyamat lehet, amelyet a `fork` hozott létre. A pontos sorrend a shell indítási módjától függ.

32.3.12 7.5 Mérési napló

A mérés során a következő metrikákat rögzítjük:

1. **Context switch idő**: a timer interrupt belépésétől a `switch_to` visszatéréséig eltelt ciklusok száma. Mérési módszer: a `schedule` elején `rdcycle`, a `switch_to` visszatérése után `rdcycle`. A különbséget naplózzuk. A tipikus érték QEMU `rv64` alatt, 1 CPU-n, `-O2` és `-icount` nélkül: 1000–2000 ciklus. A mért értéket a naplóban fel kell tüntetni, és a várt értéket külön oszlopban.
2. **Syscall költség**: a `syscall_dispatch` belépésétől a visszatérésig. Mérés: a `trap_handler` elején és a `sret` előtt. A tipikus érték 500–1500 ciklus.
3. **Timer interrupt jitter**: a timer megszakítások közötti idő ingadozása. Mérés: a `timer_interrupt_handler` minden hívásakor `rdtime`-et rögzítünk, és a szomszédos különbségek szórását számítjuk. A QEMU alatt a jitter általában néhány tíz ciklus, de a hoszt ütemezőtől függően nagyobb is lehet.

A mérési napló táblázata:

Mérés	Módszer	Várt érték	Mért érték	Megjegyzés
Context switch	`rdcycle`	$<$ 2000 ciklus	1342 ciklus	1000 minta átlaga
Syscall	`rdcycle`	$<$ 1500 ciklus	987 ciklus	1000 minta átlaga
Timer jitter	`rdtime`	$<$ 100 ciklus	62 ciklus	1000 minta szórása

A mérési naplóban a mért és várt értéket külön kell jelölni. A várt érték a korábbi mérésekből vagy a hardver specifikációjából származik; a mért érték a tényleges futásból. Ha a mért érték jelentősen eltér, a naplóban meg kell vizsgálni a QEMU konfigurációját vagy a fordítási beállításokat.

32.3.13 7.6 Visszaállítás

A labor végén a forráskódot a `git` elágazásából vissza kell állítani:

```
git stash
make clean
```

A QEMU lemezét, a hoszt bootloaderét vagy a gyártói firmware-t nem módosítjuk. A labor kizárólag emulált környezetben fut.

32.3.14 8. Önellenőrző kérdések

1. Miért a callee-saved regisztereket menti a `switch_to`, és miért nem a caller-saved regisztereket?
2. Mi történik, ha a `syscall_dispatch` után elfelejtjük növelni a `sepc`-t?
3. Hogyan előzhető meg a versenyhelyzet a `schedule` és a timer interrupt között?
4. Miért kell a `fork` gyermek `trapframe->a0` értékét nullára állítani?
5. A `sys_ps` miért nem adhatja közvetlenül a PCB lista pointerét a felhasználói programnak?

32.3.15 9. Összefoglalás

Ebben a fejezetben a záróprojekt kernelét futtathatóvá tettük. Megvalósítottuk a feladatmodellt PCB-vel és állapotgéppel, a round-robin ütemezőt timer interrupttal, a kontextusváltást assemblyben, a rendszerhívás-dispatchert és a minimális shellt. A hibakezelésnél kiemeltük a tipikus buktatókat, a laborban pedig reprodukálható mérési naplóval igazoltuk a működést. A következő egységben az `os125` erre az alapra építi a fájlrendszert és a lemezes tárolást.

32.4 Fájlrendszer, tesztelés és bemutató

Miután a kernelünk bootol, ütemezőt futtat, rendszerhívásokat szolgál ki, és a shell elfogadja a parancsokat, a záróprojekt utolsó nagy pillére a tartós állapot. Ebben az alfejezetben egy minimális, inode-alapú fájlrendszert (MMFS) építünk, tesztelünk, és a projektet bemutatható, dokumentált csomaggá zárjuk. A fájlrendszer nélkül a `cat`, a `write` és a `mkdir` csak a RAM-ban él; a perzisztencia nélkül a shell minden újraindítás után üres lenne. A tárolás tehát nemcsak egy további eszközmeghajtó, hanem a rendszer lezárásának feltétele: a kernel által felépített állapotnak túl kell élnie a processz és a kernel újraindulását is.

32.4.1 A tárolás és a projekt lezárása

A záróprojekt célja egy olyan minimális operációs rendszer, amelynek a következő tulajdonsága van: a QEMU-ban elindított kernel bootol, shellt ad, és az ott létrehozott fájlokat a virtuális gép újraindítása után is el tudja olvasni. A „lezárás” itt nem egyszerűen a kód befejezését jelenti, hanem azt, hogy a rendszer rendelkezik

- egy reprodukálható felépítéssel,
- egy tartós adatréteggel,
- tesztelt hiba- és perzisztencia-szemantikával,
- dokumentált korlátokkal,
- és egy olyan demóval, amely a fentieket ellenőrizhető módon, nem csak szóban mutatja be.

A tárolási alrendszer nélkül a többi rész – boot, memória, ütemező, rendszerhívások, shell – önmagában zárt, de nem mutat teljes operációs rendszert, mert nincs állapota a lemezen. Ebben a fejezetben az MMFS nevű, oktatási célú fájlrendszert úgy tervezzük, hogy a kernelbe illeszthető legyen, és a shell számára fájl-, könyvtár- és lemez-műveleteket biztosítson. A labor eldobható emulált környezetben fut; a hoszt valódi lemezét, bootloaderét és rendszerpartícióját nem módosítjuk.

32.4.2 MMFS tervezése

A MMFS egy 64 MiB-os, 4 KiB blokkméretű, inode-alapú fájlrendszer. Nem POSIX-teljes, de a záróprojekt shelljéhez elegendő: reguláris fájl, könyvtár, hierarchikus útvonalak, megnyitás, írás, olvasás, törlés, könyvtárlistázás és metaadat-lekérdezés.

A lemezkép szerkezete a következő:

Blokkindex	Cím	Tartalom
0	`0x00000000`	Szuperblokk
1	`0x00001000`	Inode-bitmap
2	`0x00002000`	Blokkbitmap
3–18	`0x00003000`–`0x00012FFF`	Inode-tábla (16 blokk, 512 inode)
19–16383	`0x00013000`–`0x03FFFFFF`	Adatblokkok (16365 blokk)

A választott inode-méret 128 B, ezért 512 inode pontosan 16 blokkot foglal el. Az inode-ban a közvetlen blokkpointerek száma 12, és egy indirekt blokkpointer is van. Ezzel a maximális fájlméret

$$48\ \text{KiB} + 1024 \cdot 4\ \text{KiB} = 4\ \text{MiB} + 48\ \text{KiB},$$

mert a 4 KiB-os indirekt blokk 1024 darab 32 bites blokkpointert tárol.

A szuperblokk C-beli ábrázolása szemléltetésként:

```
struct mmfs_superblock {
    uint32_t magic;              /* 0x4D4D4653 = "MMFS" */
    uint32_t block_size;         /* 4096 */
    uint32_t block_count;        /* 16384 */
    uint32_t inode_count;        /* 512 */
    uint32_t inode_bitmap_block; /* 1 */
    uint32_t block_bitmap_block; /* 2 */
```

```
    uint32_t inode_table_start;  /* 3 */
    uint32_t data_start;         /* 19 */
    uint32_t free_block_count;
    uint32_t free_inode_count;
    uint8_t  state;              /* MMFS_CLEAN=0x00, MMFS_DIRTY=0x01 */
    uint8_t  reserved[3];
    uint32_t checksum;
};
```

Az inode-struktúra pedig:

```
struct mmfs_inode {
    uint16_t mode;              /* MMFS_INO_REG=0x0100, MMFS_INO_DIR=0x0200 */
    uint16_t nlink;
    uint32_t size;
    uint32_t direct[12];
    uint32_t indirect;
    uint32_t mtime;
    uint32_t ctime;
    uint32_t atime;
};
```

A struktúra tényleges mérete 72 B; a 128 B-os inode-méretet paddinggal érjük el, hogy az inode-tábla sorai ne lógjanak át blokkhatáron.

A rétegfelépítés a következő: a shell a `sys_open`, `sys_read`, `sys_write`, `sys_close`, `sys_mkdir`, `sys_unlink`, `sys_readdir`, `sys_stat` és `sys_sync` rendszerhívásokat hívja. A kernel belül a VFS-szerű inode-réteg a MMFS függvényeit hívja; a MMFS a blokkmeghajtó `blk_read`, `blk_write` és `blk_flush` API-ját használja; a blokkmeghajtó pedig a QEMU-ban elérhető, eldobható `virtio-blk-device` eszközt kezeli.

32.4.3 Blokkdriver

A blokkdriver API-ja a fájlrendszer számára:

```
int blk_read(uint32_t block_no, void *buf);
int blk_write(uint32_t block_no, const void *buf);
int blk_flush(void);   /* az eszköz-írás befejezését és a barrier-t kéri */
```

A visszatérési érték `0` siker esetén, `-1` hibánál, és az `errno` rendre `-EINVAL` érvénytelen blokk-indexre, `-EIO` eszközhibára, `-ENOSPC` pedig akkor, ha a kép méretén túlra mutatna az írás. A laborban egyetlen `virtio-blk` lemezképet használunk:

```
qemu-system-x86_64 -machine q35 -m 128M \
  -kernel build/kernel.elf \
  -drive file=build/mmfs.img,format=raw,if=virtio \
  -nographic
```

A blokkméret a MMFS-ben 4 KiB, ezért a driver egyszerre egy teljes blokkot mozgat. A kernelben nem vezetünk be globális blokkcache-t, minden írás közvetlenül az emulált lemezre megy. Ez lassabb, de determinisztikusabb, és kiszűri a cache okozta pszeudo-perzisztenciát: ha egy teszt hibát talál, az valóban a lemezre írt állapotot ellenőrzi.

32.4.4 Inode- és blokkfoglalás

A szabad blokk kiosztását a `balloc()` függvény végzi. A 2. blokkban lévő blokkbitmap 4096 bájt, ami 32 768 bitet tartalmaz, ezért a 16 384 blokk kétszer is elférne benne. A foglalás pásztázza a bitmapet, beállítja az első szabad bitet, csökkenti a szuperblokk `free_block_count` értékét, majd a szuperblokkot piszkosnak jelöli. A felszabadítás az inverz művelet. Komplexitása a bitmap lineáris pásztázása miatt legrosszabb esetben $O(n)$, ahol n a blokkok száma; a jelenlegi 16 384 blokk mellett ez bőven elfogadható, és a buddy vagy slab allokátor bevezetését a projekt későbbi bővítéseként lehet elvégezni.

A fájl növelésekor a `sys_write` a következő sorrendet követi:

1. Ha a fájl méretének növelése új blokkot igényel, hívja a `balloc()`-ot.
2. Az új blokkba beírja az adatokat legalább a `blk_write` segítségével.
3. Ha az új blokk indexe már eléri a 12 közvetlen pointer határát, akkor szükség esetén létrehozza az indirekt blokkot.
4. Frissíti az inode `size` mezőjét.
5. A szuperblokk `state` mezőjét `MMFS_DIRTY` értékre állítja.

A `sys_read` a fájl offsetből blokkindexet számol, majd a 12-nél kisebb indexnél a `direct[]` tömbből, nagyobb indexnél az indirekt blokk beolvasásával szerzi meg a fizikai blokk számát.

32.4.5 Rendszerhívások

A shellből elérhető új rendszerhívások a következők:

Rendszerhívás	Paraméterek	Visszatérés	Főbb hibakódok
`sys_open`	`const char *path`, `int flags`, `int mode`	fd (>=0) vagy -1	`ENOENT`, `ENOTDIR`, `EISDIR`, `ENOSPC`, `EINVAL`, `ENOMEM`
`sys_read`	`int fd`, `void *buf`, `size_t count`	olvasott bájtok, 0 fájlvége, -1 hiba	`EBADF`, `EFAULT`
`sys_write`	`int fd`, `const void *buf`, `size_t count`	írt bájtok, -1 hiba	`EBADF`, `ENOSPC`, `EFBIG`
`sys_close`	`int fd`	0 vagy -1	`EBADF`
`sys_unlink`	`const char *path`	0 vagy -1	`ENOENT`, `EISDIR`
`sys_mkdir`	`const char *path`	0 vagy -1	`EEXIST`, `ENOSPC`
`sys_readdir`	`int fd`, `struct mmfs_dirent *dirp`, `int count`	bejegyzések száma, 0 vége, -1 hiba	`EBADF`, `ENOTDIR`
`sys_stat`	`const char *path`, `struct mmfs_stat *st`	0 vagy -1	`ENOENT`
`sys_sync`	nincs	0 vagy -1	`EIO`

A `sys_open` megnyit egy meglévő fájlt vagy könyvtárat, illetve `O_CREAT` esetén létrehoz egy reguláris fájlt. Létrehozáskor a kernel lefoglal egy inode-ot, hozzáadja a szülő könyvtárhoz, és `nlink` értékét 1-re állítja. A könyvtárbejegyzés a következő 32 bájtos rekord:

```
struct mmfs_dirent {
    uint32_t ino;   /* inode-szám */
    char     name[28];
};
```

A név legfeljebb 27 karakter lehet, mert a 28. bájt a lezáró NUL vagy további hely lehet. A `sys_readdir` a megnyitott könyvtár inode-jának blokkjait olvassa, és bejegyzéseket másol a felhasználói pufferbe.

A `sys_write` visszatérési értéke a ténylegesen kiírt bájtok száma. Ha a lemez megtelik, `-ENOSPC` hibával tér vissza. A `sys_read` a fájl végét 0 visszatérési értékkel jelzi. A `sys_close` felszabadítja a folyamat fájlleíró-táblázatbeli bejegyzését; maga az inode csak akkor szabadul fel, ha az `nlink` és a nyitott referenciák száma is nulla.

32.4.6 Perzisztencia és crash-konzisztencia

A MMFS szándékosan nem tartalmaz journalingot, ezért a perzisztencia-garanciák a lemezre írás sorrendjén múlnak. Az alapszabály így foglalható össze:

> A lemez soha ne tartalmazzon olyan inode- vagy indirekt blokk-mutatót, amely olyan blokkra hivatkozik, amely nincs a blokkbitmapban lefoglalva.

Ennek megfelelően a blokkfoglalás sorrendje:

1. Először a blokkbitmap bitjét állítjuk be, és a szuperblokk `free_block_count` értékét csökkentjük.
2. Csak ezután írjuk be a blokkpointert az inode-ba vagy az indirekt blokkba.

Ha a lépés 1 után, de a lépés 2 előtt áramszünet következik be, a lemez egy blokkot elveszít, de nem korrumpálódik; a `fsck` a bitmap és a hivatkozások összevetésével észleli a szivárgást. Ha a sorrend fordított lenne, a bit nélküli, de hivatkozott blokk később újra kioszthatóvá válna, és a fájlrendszer sérülne.

A superblock `state` mezője a `sys_sync` során `MMFS_CLEAN` értéket kap, minden egyéb metaadat-módosításkor `MMFS_DIRTY` értéket. A kernel bootoláskor lefuttatja az MMFS `fsck` rutint:

1. Ellenőrzi a superblock `magic` mezőjét (`0x4D4D4653`) és a `checksum` értéket.
2. Ellenőrzi a bitmap- és inode-számlálókat.
3. Végigmegy az inode-okon, és megnézi, hogy minden hivatkozott blokk le van-e foglalva.
4. Ellenőrzi, hogy a könyvtárfákból elérhető inode-ok `nlink` értéke konzisztens-e.

Ha a `state` `MMFS_CLEAN`, a boot a gyors útvonalat választja; ha `MMFS_DIRTY`, akkor a teljes ellenőrzés indul. A crash-teszt szándékosan úgy öli meg a QEMU-t, hogy az utolsó `sync` nem fut le, ezért bootkor mindig teljes `fsck` fut.

32.4.7 Tesztelés és tesztriport

A tesztelés négy csoportra oszlik:

1. **Blokkdriver-tesztek:** `blk_write`–`blk_read` roundtrip minden 4 KiB-os blokkra, érvénytelen blokkindexre hibaválasz.
2. **Fájlrendszer-műveleti tesztek:** `mkdir`, `open`+`write`+`read`, `unlink`, `readdir`, `stat` kombinációk.

3. **Crash-konzisztencia-tesztek:** egy 2 MiB-os fájl írása közben a QEMU folyamatot megszakítjuk, majd újraindítás után ellenőrizzük, hogy a `fsck` nem talál korrupciót, és a már leírt adatblokkok konzisztensek.
4. **Teljesítmény-alapmérés:** szekvenciális írás és olvasás 16 MiB-on, valamint véletlen 4 KiB-os olvasás.

A tesztkörnyezet rögzített reprodukciós csomagban:

- CPU: Intel Core i7-12700, 8 P-core, 16 thread, 4,9 GHz turbo, 32 GiB DDR4-3200, Ubuntu 24.04.
- QEMU: 8.2, TCG (hardveres gyorsítás nélkül), `-machine q35`, `-m 128M`, `-nographic`.
- Fordító: GCC 12.3, `-O2 -ffreestanding -nostdlib -fno-stack-protector -mno-red-zone`.
- Kernelen belüli időmérés: `rdtsc` delta, a kernel órajel-referenciája a QEMU által emulált időzítő.

A mérés csak a kernelben futó `rdtsc` különbséget veszi alapul, így a hoszt terhelése nem kerül be közvetlenül az időeredménybe. A mérési napló a futási idő mellett a ciklusszámot, a kernel laphibaszámát és a cache-hatást is rögzíti. A várt és a mért értéket külön oszlopban kell megadni, például:

Teszt	Várt	Mért	Állapot
Szekvenciális írás 16 MiB	>1 MiB/s	2,14 s, 7,84 MiB/s, $8{,}91 \cdot 10$ ciklus	OK
Szekvenciális olvasás 16 MiB	>5 MiB/s	0,96 s, 17,5 MiB/s, $4{,}02 \cdot 10$ ciklus	OK
Véletlen 4 KiB olvasás, 1000 db	>20 IOPS	23,2 s, 43 IOPS	OK
Crash-konzisztencia	nincs korrupció	`fsck`: 0 hiba, 3 inode, 42 blokk	OK

A fenti számok a helyi laborban reprodukálandók; a TCG mód miatt a QEMU verziója és a hoszt CPU erősen befolyásolja az eredményt, ezért a várt oszlop csak nagyságrendi kapaszkodó.

A tesztriportnak a következő fejezeteket kell tartalmaznia:

- build-azonosító, QEMU-verzió, kernel-commit,
- a lemezkép mérete és formátuma,
- a futtatott tesztek és parancsaik,
- a mért és várt értékek,
- az eltérések magyarázata,
- a reprodukciós utasítások,
- a visszaállítás módja.

32.4.8 Bemutató

A demó nem interaktív előadás, hanem reprodukálható parancssor. A `Makefile` célja a `make test` és a `make demo`. A demó forgatókönyve:

```
make clean && make
make run

# shell prompt a QEMU-ban
mkfs /dev/vda
mount /dev/vda /
mkdir /home
touch /home/hello.txt
write /home/hello.txt "hello mmfs"
cat /home/hello.txt
ls /home
sync
shutdown
```

Az első leállítás után a lemezkép megmarad, majd újraindítás:

```
make run
cat /home/hello.txt
stat /home/hello.txt
bench write /tmp/bigfile 16MiB
bench read /tmp/bigfile 16MiB
shutdown
```

A demó kulcsmondata a második `cat` kimenete: ha a kernel a szöveget az újraindítás után is visszaadja, a perzisztencia nem csupán a RAM cache maradéka. A `stat` kiírja a méretet és az inode típusát, a `bench` pedig a mért sávszélességet és IOPS-t. A demó teljes kimenetét a `logs/os125-demo.log` fájlba kell menteni.

32.4.9 Dokumentáció és ismert korlátok

A projekt lezárásához a dokumentációnak ki kell térnie:

- **Követelmények:** egy QEMU alatt bootoló kernel, szálbiztos ütemező, rendszerhívások, shell, MMFS.
- **Build és futtatás:** `make clean && make`, `make run`, `make test`.
- **Architektúra:** rétegdiagram, rendszerhívás-lista, MMFS lemezkép.
- **Tesztriport:** a fenti mérési napló.
- **Ismert korlátok:** az alábbi lista.

A MMFS legfontosabb korlátai:

- Nincs naplózás; a crash-konzisztencia csak az írási sorrendből adódik, a `fsck` észlel, de nem javít.
- Nincs hozzáférés-vezérlés: minden folyamat gyakorlatilag root.
- Nincs SMP-zár a fájlrendszeren; többmagos kernelben a MMFS globális állapota versenyhelyzetet okozhat.
- A fájl maximális mérete 4 MiB + 48 KiB.
- A könyvtárbejegyzés neve legfeljebb 27 bájt.
- Nincs per-fájl `fsync`, csak globális `sys_sync`.
- Nincs blokkcache, ezért a teljesítmény erősen korlátos.
- A `virtio-blk` driver nem kezeli a hardveres megszakításokat az MMFS mérési útjában, a `blk_flush` barrier egyszerű szinkronizáció.

Ezeket a korlátokat nem szabad elhallgatni a bemutatón: a projekt tárgyilagos bemutatása a kódminőség része, és a következő fejezet kiadási checklistje is az itt rögzített állapotból indul.

32.4.10 Tipikus hibák és helyes megoldásuk

1. **A blokkbitmap frissítése a hivatkozás előtt elmarad.**
   Hiba: A `balloc` bit beállítása nélkül írunk be egy blokkpointert, vagy a felszabadítás fordított sorrendben történik.
   Következmény: ugyanaz a blokk többször kioszthatő, vagy a lemezen lévő hivatkozás érvénytelen blokkra mutat.
   Helyes: a bitet először lefoglalni, majd a mutatót beírni; felszabadításnál előbb a mutatót törölni, majd a bitet visszaállítani.

2. **A fájlméret frissítése az adatblokk beírása előtt.**
   Hiba: a `size` mező értéke már akkor növekszik, amikor az adatblokk még nincs a lemezen.
   Következmény: áramszünet után a fájl vége ismeretlen tartalmú blokkot tartalmaz.
   Helyes: először az adatblokk, szükség esetén az indirekt bejegyzés, és csak utoljára az inode `size` mezőjének frissítése.

3. **A superblock `state` mezője nem kerül `MMFS_DIRTY` értékre.**
   Hiba: a `mkdir`, `write`, `unlink` módosítja a metaadatot, de a szuperblokk `state` mezője `clean` marad.
   Következmény: a bootkori `fsck` gyors úton halad, és a sérült metaadatot nem ellenőrzi.
   Helyes: minden metaadat-módosítás elején `MMFS_DIRTY`, a `sys_sync` végén `MMFS_CLEAN`, és a superblock checksumot mindig újraszámolni.

4. **A `readdir` kernelpuffert ad a felhasználónak.**
   Hiba: a kernel stacken lévő `struct mmfs_dirent` címe közvetlenül a felhasználói `buf`-ba kerül.
   Következmény: a felhasználói processz a kernelmemóriát olvasná, vagy a kernel a felhasználói pufferbe írna ellenőrizetlenül.
   Helyes: a `copy_to_user` és `copy_from_user` megfelelőit kell használni; minden rendszerhívás-paramétert validálni kell.

5. **Az `unlink` nem szabadítja fel az inode-hoz tartozó blokkokat.**
   Hiba: a könyvtárbejegyzés törlése után az `inode` még `nlink == 0`, de a blokkok felszabadítása elmarad.
   Következmény: blokkszivárgás, a lemez fogyása.
   Helyes: az `nlink` elérésekor a `free_inode` mellett a `direct[]` és az indirekt blokk felszabadítása is megtörténjen.

6. **A mérés a hoszt `time` parancsát használja.**
   Hiba: a QEMU indítási ideje, a hoszt ütemezése és a lemez I/O-ja belekeveredik a mérésbe.
   Következmény: a kapott sávszélesség nem a kernel sebességét méri.
   Helyes: a kernel `rdtsc` delta alapján mér, a QEMU konfigurációt és a hoszt verziókat rögzíteni kell.

7. **A crash-teszt normál `shutdown`-t használ.**
   Hiba: a `shutdown` lefuttatja a `sync`-et, ezért a lemez `clean` állapotba kerül, és a `fsck` nem talál semmit.
   Következmény: a teszt nem teszteli a valódi áramszünetet.
   Helyes: a QEMU folyamatot a kernel írása közben `kill` vagy `timeout` segítségével, `sync` nélkül kell megszakítani.

32.4.11 Labor – MMFS kiépítése, crash-teszt és demó

Az alábbi labor az Ütemező, rendszerhívások és shell után építhető fel, amikor a kernel már shellt futtat. A labor kiinduló állapota egy tiszta git-ág, például `lab-mmfs-start`, ahol a kernel bootol, de a fájlrendszer réteg még üres.

Kiinduló állapot:
- `build/kernel.elf` bootol QEMU alatt.
- Nincs `build/mmfs.img`.
- A shell a `mkfs`, `mount`, `cat`, `write`, `ls`, `stat`, `bench` parancsokat még nem ismeri; ezeket a kernelbe kell kötni.

Lépések:

1. Hozza létre a 64 MiB-os üres lemezképet:

```
dd if=/dev/zero of=build/mmfs.img bs=1M count=64
```

2. Fordítsa a kernelt:

```
make clean && make
```

3. Indítsa QEMU-t:

```
make run
```

4. A kernel bootoláskor érvénytelen MMFS `magic` értéket talál, ezért formázza a lemezképet, és a `fsck` kiírja a tiszta inode- és blokk-számlálókat.

5. A shellben hajtsa végre a fájlrendszer parancsait:

```
mkdir /home
touch /home/hello.txt
write /home/hello.txt "hello mmfs"
cat /home/hello.txt
ls /home
stat /home/hello.txt
sync
shutdown
```

6. Indítsa újra a QEMU-t, és ellenőrizze a perzisztenciát:

```
cat /home/hello.txt
fsck
```

7. Futtassa a crash-tesztet. A teszt egy 2 MiB-os fájlt ír, majd a QEMU-t `sync` nélkül megöli:

```
timeout 5 make run-crash-test
```

A `run-crash-test` a QEMU-t indítja addig, amíg a kernel a fájlt írja, majd a folyamatot megszakítja. Az újraindítás után a `fsck` teljes ellenőrzést végez, és a napló `0 hiba` állapotot mutat.

8. Futtassa a benchmarkot:

```
bench write /tmp/bigfile 16MiB
bench read /tmp/bigfile 16MiB
bench randread /tmp/bigfile 4KiB 1000
```

9. A `make test` cél lefuttatja a fenti teszteket, és a `logs/os125-test.log` fájlban rögzíti a kimenetet. A riportban a várt és mért értékeket külön kell feltüntetni.

Elvárt eredmény:
Minden fájlrendszer-művelet 0 hibával fut; a második boot után a `cat /home/hello.txt` visszaadja a `hello mmfs` szöveget; a crash-teszt után a `fsck` nem talál korrupciót; a benchmark a várt nagyságrendet eléri; a mért értékek a tesztriportban szerepelnek.

Visszaállítás:
`make clean && rm -f build/mmfs.img logs/os125-test.log logs/os125-demo.log`

32.4.12 Mérőszám és önellenőrző kérdések

A fejezet egyik mérőszáma a MMFS szekvenciális írási sávszélessége 16 MiB-on. A kernelben mért `rdtsc` delta és a QEMU-emulált idő alapján számítható, hogy a mért érték a várt 1–10 MiB/s közötti tartományba esik. A laborban a mérési naplóban a futási idő, a ciklusszám, a laphibák és a cache-viselkedés is megjelenik.

Önellenőrző kérdések:

1. Miért kell a blokkbitmapot előbb frissíteni, mint az inode blokkpointerét?

2. Írja le egy új fájl létrehozásának MMFS-beli lemezre írási sorrendjét journaling nélkül.

3. Miért nem elegendő a perzisztencia bizonyításához az, hogy a `shutdown` után újraindul a rendszer, és miért kell a `sync` nélküli megszakítás?

4. Milyen hibát okoz, ha a `sys_read` a `readdir` kernelpufferét közvetlenül a felhasználói bufferbe másolja ellenőrzés nélkül?

33 Kiadási checklist és továbbtanulás

33.1 Kiadási checklist és kódminőség

A kiadási ellenőrzőlista nem adminisztratív utólagos formalitás, hanem a záróprojekt minőségbiztosításának utolsó, aktív rétege. A minimális operációs rendszer esetén a kiadás nem egyetlen bináris átadását jelenti, hanem egy olyan csomagot, amelyből a bíráló, a későbbi fejlesztő vagy akár a tanúsító munkatárs reprodukálhatóan újraépítheti, elindíthatja és ellenőrizheti a rendszert. A checklist tehát a reprodukció, a nyomon követhetőség és a hibakezelés eszköze. Ez a fejezet a kiadási jelölt (release candidate, RC) ellenőrzésére koncentrál, és feltételezi, hogy a záróprojekt elérte a Fájlrendszer, tesztelés és bemutató alfejezetben meghatározott funkcionális állapotot: a rendszer bootol, a memóriakezelő, az ütemező, a rendszerhívások, a shell és a fájlrendszer működik.

A kiadás előtti ellenőrzés során nem a funkciókat teszteljük újra, hanem azt vizsgáljuk, hogy a rendszer *kiadásra alkalmas-e*. Ez más kérdés, mint a funkcionális helyesség. Egy funkció lehet helyes, de a kiadás mégsem megy át, ha például a build nem reprodukálható, a dokumentáció hiányos, vagy egy ismert, súlyos hiba nincs nyilvántartva. A checklist ezért kategóriákra bontott, és minden kategóriában explicit go/no-go feltételeket adunk.

33.1.1 1. A kiadási jelölt és a verziókezelés

A kiadási folyamat kiindulópontja egy verziókezelt forrásfa adott commitja. A minimális OS projektben a git commit hash, a build időbélyeg és a konfigurációs verzió hármasa azonosítja a jelöltet. A checklist első eleme mindig az, hogy a forrásfában nincs módosítatlan, nem követett fájl, és a build a tiszta munkakönyvtárból indul.

A verziószámot a szemantikus verziózás (például `0.4.0`) szerint érdemes kezelni, de a minimális OS esetén a `0.x` tartomány elegendő. A kiadási jelölt neve `miniOS-0.4.0-rc1` lehet, ahol az `rc` utótag a jelölt állapotot mutatja. A build során a verziószámot a kernelbe is be kell írni: egy `version` parancs vagy a bootlog első sora jelenítse meg. Ellenőrzési pont, hogy a kernelben látható verziószám és a kiadási csomag neve megegyezik.

33.1.2 2. Build és reprodukálhatóság

A minimális OS buildje jellemzően keresztfordítást használ: a hoszt rendszer (például x86_64 Linux) a célt (például i686-elf vagy riscv64) számára fordít. A checklist első nagy blokkja a build környezet rögzítését kéri.

33.1.3 2.1. Eszközlánc és függőségek

A kiadási csomagban pontosan rögzíteni kell a használt fordító verzióját, a linker verzióját, valamint a szükséges könyvtárak és eszközök verzióját. Példa egy minimális jegyzőkönyvbe:

Eszköz	Verzió	Ellenőrzés módja
`x86_64-elf-gcc`	14.2.0	`x86_64-elf-gcc --version`
`x86_64-elf-ld`	2.42	`x86_64-elf-ld --version`
`qemu-system-x86_64`	8.2.0	`qemu-system-x86_64 --version`
`make`	4.4	`make --version`

A fordítási beállításokat is rögzíteni kell: optimalizáció, debuginformáció, figyelmeztetési szint, architektúra-specifikus flag-ek. A reprodukálhatóság érdekében a build szkript ne támaszkodjon a hoszt környezeti változóira, vagy ha igen, azokat dokumentálja. A `CFLAGS`, `LDFLAGS` és `NASMFLAGS` értékét a Makefile vagy a build szkript explicit módon tartalmazza.

33.1.4 2.2. Determinizmus

A kiadási követelmény, hogy ugyanabból a commitból, ugyanazzal az eszközlánccal bitazonos kernelimage jöjjön létre. A determinisztikus build érdekében a forrásfában nem szerepelhet időbélyeg a binárisban, kivéve, ha azt a build rendszer explicit módon, a kiadási metaadatból származtatja. A linker scriptben a `.comment` szekciót érdemes üresen hagyni, és a debug szekciókban lévő hoszt útvonalakat normalizálni kell – ez a gyakorlatban a `-fdebug-prefix-map` és `-fmacro-prefix-map` flag-ekkel érhető el.

Ellenőrző parancs a reprodukcióhoz:

```
# Két külön munkakönyvtárból ugyanazt a commitot buildelni
git archive --format=tar HEAD | tar -x -C /tmp/build-a
git archive --format=tar HEAD | tar -x -C /tmp/build-b
make -C /tmp/build-a
make -C /tmp/build-b
sha256sum /tmp/build-a/kernel.bin /tmp/build-b/kernel.bin
```

A várt eredmény: a két `sha256sum` kimenet azonos. A checklistben ez a `REPRODUCIBLE_BUILD` jelző. Ha a hash eltér, a kiadás nem mehet tovább, amíg az okot fel nem derítik.

33.1.5 2.3. Build tisztaság és figyelmeztetések

A kiadási build során a fordító nem adhat figyelmeztetést. A build rendszer `-Wall -Wextra -Werror` kapcsolókkal kényszeríti ki ezt. A `Werror` használata gyakori vitatéma, de egy minimális OS esetén indokolt: a kernelben a nem kezelt implicit típuskonverzió vagy a nem használt változó később valódi hibát okozhat. A checklistben a `WARNINGS_AS_ERRORS=1` bejegyzésnek kell szerepelnie.

A build kimenetét is naplózni kell. A `make` kimenetét a kiadási jegyzőkönyvhöz csatoljuk, és a naplóban keresni kell a `warning` vagy `error` szavakat. A build log mérete nem megengedő kritérium, de a rendellenes kimenet (például a linker `undefined reference` üzenete) azonnali go-blokkoló.

33.1.6 3. Boot és futási ellenőrzés

A build után a jelöltet az emulált környezetben kell elindítani. A checklist ebben a blokkban a boot folyamat minden lépésére kiterjed.

33.1.7 3.1. Boot média és konfiguráció

A minimális OS tipikusan QEMU alatt fut, például:

```
qemu-system-x86_64 \
  -drive file=miniOS-0.4.0-rc1.iso,format=raw,if=ide \
  -m 64M \
  -serial stdio \
  -display none \
  -no-reboot \
  -no-shutdown
```

A konfiguráció rögzítése kötelező: a CPU modell (`-cpu`), a memória méret, a csatolt eszközök és a soros konzol beállítása mind a reprodukciós jegyzőkönyv része. A checklist ellenőrzi, hogy a parancs nem tartalmaz valós lemezt vagy hoszt eszközt, és hogy a QEMU processz izoláltan fut.

A boot teszt során a naplót a soros konzolról kell gyűjteni. A kiadási jegyzőkönyvben a teljes bootlog szerepeljen, időbélyeggel. A várt bootlog minimális elemei: a bootloader üzenete, a kernel belépési pontja (például `0x100000`), a memóriatérkép felderítése, az ütemező indulása, a rendszerhívás tábla inicializálása, a shell prompt megjelenése.

33.1.8 3.2. Boot idő és determinizmus

A checklistben szerepelnie kell egy boot idő mérésnek. A mérési módszer: a QEMU indításától a shell prompt megjelenéséig eltelt idő, a hoszt órája szerint. A várt érték tájékoztató jellegű, például 150 ms alatt kell lennie egy 64 MiB RAM-mal emulált rendszeren, de a pontos szám a konfigurációtól függ. A mért és várt értéket külön oszlopban kell rögzíteni.

Fontos, hogy a boot idő mérése ne legyen a funkcionális teszt része. A checklist itt csak azt rögzíti, hogy a boot nem lóg, és a rendszer eléri a használható állapotot. Ha a boot 30 másodperc után sem jelenik meg a prompt, az go-blokkoló.

33.1.9 3.3. Kernel panic és hibanapló

A checklist megköveteli, hogy a kernel minden váratlan állapotot `panic` üzenettel jelezzen, és a `panic` ne csendes leállás legyen. A bootlogban a `panic` szó keresése kötelező. Ha a boot során bármilyen `panic` megjelenik, a kiadás nem engedhető át, még akkor sem, ha a rendszer később helyreáll.

A `panic` üzenet formátuma a korábbi fejezetekben meghatározott: regiszter dump, a hiba helye (`file:line`), és a stack trace, ha elérhető. A checklist itt azt ellenőrzi, hogy a naplóban a panic egyértelműen azonosítható-e, és hogy a reprodukálható konfiguráció lehetővé teszi-e a hiba újbóli előidézését.

33.1.10 4. Tesztek és automatizált ellenőrzés

A kiadási tesztblokk nem a funkcionális tesztelés teljes körét ismétli, hanem azt ellenőrzi, hogy a tesztinfrastruktúra működik, és az összes teszt egyetlen parancsból futtatható.

33.1.11 4.1. Tesztparancs és kimenet

A minimális OS projekt esetén a tesztek egy része a hoszton fut (unit tesztek a memóriaallokátorra, a listákra, a fájlrendszer logikára), másik része a célgépen, QEMU alatt (integrációs tesztek, boot tesztek). A checklist eleme:

```
make test
```

A fenti parancsnak minden tesztet le kell futtatnia, és a kimenetet egy tesztnaplóba (`test-results.log`) kell írnia. A tesztrendszer visszatérési értéke `0`, ha minden teszt sikeres, `nem nulla` egyébként. A kiadási jegyzőkönyvben a tesztnapló kivonatának és az összesített eredménynek (`PASS`, `FAIL`, `SKIP`) szerepelnie kell.

A checklist ellenőrzi, hogy a tesztnaplóban a `FAIL` és `SKIP` sorok indokolva vannak. A `SKIP` kategória csak előre meghatározott feltétellel fogadható el (például nincs hálózati kártya), és nem rejthet el ismert hibát.

33.1.12 4.2. Kódminőségi kapuk

A kiadási jelölt nem léphet tovább, ha a statikus elemzés vagy a fuzzing súlyos hibát talál. A minimális OS esetén a statikus elemzés lehet egyszerű `cppcheck` vagy `clang-tidy` futtatás. A fuzzing a fájlrendszer vagy a rendszerhívás bemenetére irányulhat, és a `libFuzzer` vagy egy egyszerű random input generátor is megfelel. A checklist ezeket a kapukat explicit módon rögzíti: `STATIC_ANALYSIS=1`, `FUZZING=1`, és a futtatás időtartamát (például 60 másodperc).

A kódminőség része a memória- és erőforrás-tulajdonjog áttekintése is. A checklistben szerepel egy manuális kódátnézési pont, ahol a reviewer ellenőrzi a kernelmodulokban a `kmalloc`/`kfree` párokat, a megszakításkezelőben a bufferhatárokat, és a fájlrendszerben a hibakezelést. Ez a pont nem automatizálható, de a kiadási folyamatban emberi döntés kell hozzá.

33.1.13 5. Dokumentáció ellenőrzés

A dokumentáció a kiadási csomag része, nem kiegészítője. A checklist ebben a blokkban a felhasználói és fejlesztői dokumentáció meglétét és frissességét ellenőrzi.

33.1.14 5.1. Minimális dokumentumkészlet

A kiadási jelöltnek legalább a következő dokumentumokat kell tartalmaznia:

- `README.md`: a projekt célja, a build és futtatás parancsai, a rendszer architektúrájának rövid leírása.
- `BUILD.md`: az eszközlánc telepítése, a fordítás lépései, a reprodukciós eljárás.
- `CONFIG.md`: a kernel konfigurációs paraméterei, a memória méret, az ütemező beállításai, a fájlrendszer típusa.
- `API.md`: a rendszerhívások listája, paraméterek, visszatérési értékek, hibakódok.
- `KNOWN_ISSUES.md`: az ismert hibák és korlátok listája, mindegyikhez súlyosság és workaround.
- `LICENSE`: a licenc szövege.

A checklist minden dokumentumra ellenőrzi, hogy a dokumentum verziója megegyezik a kiadási verzióval, és hogy a dokumentumban szereplő parancsok valóban működnek. A `README` parancsait a kiadási teszt során futtatni kell, és a kimenetet a jegyzőkönyvbe kell másolni.

33.1.15 5.2. Rendszerhívás dokumentáció

A rendszerhívások dokumentációja a kiadási kritériumok fontos eleme. Minden rendszerhívást a következő formában kell leírni:

```
sys_getpid(void) -> pid_t
  Visszaadja a hívó folyamat azonosítóját.
  Hibakód: nincs.
```

A checklist ellenőrzi, hogy a dokumentált paraméterek, típusok és hibakódok megegyeznek a kernel fejlécfájljaiban definiált ABI-val. A típusok méretét (például `pid_t` = 32 bites előjeles egész) a határfelületi dokumentációban rögzíteni kell.

33.1.16 6. Licenc és jogi megfelelőség

A minimális OS saját kódja mellett felhasználhat külső könyvtárakat, bootloadert vagy eszközmeghajtó mintát. A kiadási checklist a licencmegfelelést nem hagyhatja figyelmen kívül.

33.1.17 6.1. Saját licenc

A projekt saját kódját egyértelmű licenc alatt kell kiadni. A minimális OS esetén a MIT vagy a BSD-2-Clause licenc gyakori, de a GPL is elfogadható. A checklist ellenőrzi, hogy a `LICENSE` fájl a forrásfában jelen van, és hogy minden saját forrásfájl fejlécében szerepel a licenc rövid hivatkozás.

33.1.18 6.2. Függőségi licencek

A kiadási csomagban szerepelnie kell a felhasznált külső komponensek listájának, verziójával és licencével. Példa:

Komponens	Verzió	Licenc	Felhasználás
`limine`	5.20240101.0	BSD-2-Clause	bootloader
`fatfs`	0.15	BSD-1-Clause	fájlrendszer

A checklist ellenőrzi, hogy a függőségi licencek kompatibilisek a projekt licencével, és hogy a szükséges attribúciók szerepelnek a dokumentációban. A bináris terjesztés esetén a licencszövegek a kiadási csomagba is bekerülnek.

33.1.19 7. Ismert hibák és korlátok

A kiadási jelöltben lehetnek ismert, nem javított hibák. A checklist ezeket nem tiltja, de megköveteli a nyilvántartásukat és a súlyosság szerinti osztályozásukat.

A hibákat a következő kategóriákba soroljuk:

- **Blokkoló (blocker):** a rendszer alapfunkcióját akadályozza, adatvesztést okoz, vagy a biztonsági izolációt sérti. Blokkoló hibával a kiadás nem adható ki.
- **Súlyos (major):** egy funkció hibásan működik, de van elfogadható workaround. Súlyos hiba esetén a kiadás csak akkor engedhető át, ha a release notes kiemeli, és a felhasználó számára a workaround dokumentált.

- **Kisebb (minor):** kozmetikai hiba, teljesítményprobléma, vagy ritkán előforduló hiba. Kisebb hibával a kiadás átengedhető, de a hibát regisztrálni kell.
- **Javításra váró (enhancement):** hiányzó funkció vagy optimalizálás, nem hiba.

A checklistben minden nyitott hibához tartozik egy azonosító, súlyosság, leírás, reprodukciós lépés, és ha van, workaround. A `KNOWN_ISSUES.md` fájl ezt a listát tartalmazza. A go/no-go döntés során a release manager ezt a listát áttekinti, és a blokkoló hibák hiányát ellenőrzi.

33.1.20 8. Go/no-go kritériumok

A kiadási döntés nem szubjektív megérzés, hanem a checklist alapján meghozott formális döntés. A minimális OS projektben a go/no-go kritériumokat előre rögzíteni kell, és a döntést a kiadási jegyzőkönyvben dokumentálni kell.

A go feltételek listája egy példa alapján:

1. A build reprodukálható: a két külön könyvtárból készült image hash-e azonos.
2. A bootlog nem tartalmaz `panic` üzenetet, és a shell prompt megjelenik 150 ms alatt a megadott QEMU konfigurációban.
3. A `make test` parancs visszatérési értéke `0`, és a tesztnaplóban nincs `FAIL` sor. A `SKIP` sorok száma legfeljebb 2, és mindegyik indokolt.
4. A statikus elemzés nem talál kritikus vagy magas súlyosságú hibát.
5. A dokumentáció minden kötelező eleme jelen van, és a verziók egyeznek.
6. A licencmegfelelőség ellenőrzött, a függőségi lista teljes.
7. A nyitott hibák között nincs blokkoló hiba; a súlyos hibák workarounddal dokumentáltak.

Ha bármelyik feltétel nem teljesül, a kiadás `no-go` státuszt kap, és a hibát a release jegyzőkönyvben rögzíteni kell. A döntést a release manager vagy a projekt vezetője hozza meg, de a checklist automatizált része (például a build hash, a teszt eredmény) előre jelzi a kimenetet.

33.1.21 9. Labor: kiadási checklist kitöltése

A labor célja, hogy a hallgató a saját minimális OS projektjére alkalmazza a fenti checklistet, és a kiadási jegyzőkönyvet reprodukálható módon elkészítse. A labor az emulált környezetben fut, a kiinduló állapot a záróprojekt Fájlrendszer, tesztelés és bemutató utáni állapota.

33.1.22 9.1. Kiinduló állapot

A projekt forrásfája a `miniOS` könyvtárban található, a kiadási jelölt commit hash-e `a1b2c3d4`. A build környezet:

- Hoszt: x86_64 Linux, kernel 6.8.0 (a pontos verziót a jegyzőkönyvben rögzíteni kell)
- QEMU: 8.2.0
- GCC keresztfordító: `x86_64-elf-gcc` 14.2.0
- Memória: 64 MiB
- Bootloader: limine 5.20240101.0

33.1.23 9.2. Lépések

1. **Tiszta munkakönyvtár ellenőrzése:** `git status` parancs kimenetében nem lehet módosítatlan vagy nem követett fájl.
2. **Build reprodukció:** a fenti `git archive` módszerrel két könyvtárba kicsomagolni és buildelni. A `sha256sum` értékeket rögzíteni.
3. **Boot teszt:** a QEMU parancs futtatása, a bootlog mentése `boot-rc1.log` néven. A naplóban ellenőrizni a `panic` szót, és mérni a boot időt a shell promptig. A mérési módszer: a hoszt `time` parancsa vagy a QEMU indításának időbélyege és a prompt időbélyege közötti különbség.
4. **Teszt futtatás:** `make test` parancs, a kimenet mentése `test-rc1.log` néven. A visszatérési érték ellenőrzése.
5. **Dokumentáció és licenc ellenőrzés:** a kötelező dokumentumok meglétének és verziójának ellenőrzése, a függőségi lista kitöltése.
6. **Ismert hibák áttekintése:** a `KNOWN_ISSUES.md` bejegyzéseinek osztályozása, súlyosság szerint.
7. **Go/no-go döntés:** a fenti go feltételek alapján a döntés meghozatala és a kiadási jegyzőkönyv kitöltése.

33.1.24 9.3. Mérési napló és elvárt eredmény

A jegyzőkönyvben a mért és várt értékeket külön táblázatban kell szerepeltetni. Példa:

Mérés	Várt érték	Mért érték	Státusz
Build hash (build-a)	azonos a build-b-vel	`e3b0c442...`	PASS
Boot idő	< 150 ms	142 ms	PASS
Teszt visszatérési érték	0	0	PASS
`panic` a bootlogban	nincs	nincs	PASS
Blokkoló hiba	nincs	nincs	PASS

A labor átadandó eredménye a kitöltött kiadási ellenőrzőlista és a jegyzőkönyv. A jegyzőkönyvnek tartalmaznia kell a build parancsot, a QEMU parancsot, a tesztparancsot, a konfigurációt, valamint a mért és várt értékeket elkülönítve.

33.1.25 9.4. Tipikus hibák és helyes megoldás

A labor során gyakori hibák:

- **A build nem reprodukálható:** a kernelbe bekerül a hoszt abszolút útvonala vagy egy időbélyeg. A helyes megoldás a `-fdebug-prefix-map` és a `__DATE__`/`__TIME__` makrók kiiktatása.
- **A bootlog nem teljes:** a soros konzol kimenete elveszik, mert a QEMU `-display none` nélkül fut, vagy a naplózás nem irányítja át a kimenetet. A helyes parancs mindig `-serial stdio` és a kimenet fájlba irányítása.
- **A teszt `SKIP` sorai indokolatlanok:** a hallgató a bonyolult tesztet kihagyja, de a checklist ezt nem fogadja el. A helyes megoldás a `SKIP` feltétel explicit dokumentálása és a release manager jóváhagyása.

- **A licenc hiánya:** a projekt saját kódja nem tartalmaz licencfejlécet, vagy a függőségi lista hiányos. A helyes megoldás a fejlécek pótlása és a harmadik féltől származó kódok gondos átvizsgálása.
- **A go/no-go döntés szubjektív:** a hallgató az alapján dönt, hogy „úgy érzi, jó”, de nem ellenőrzi a go feltételeket. A helyes megoldás a formalizált lista végignézése és minden tételre explicit bizonyíték gyűjtése.

33.1.26 10. Összefoglalás

A kiadási checklist a minimális OS projekt utolsó minőségi kapuja. Nem a funkciókat teszteli, hanem a kiadásra való alkalmasságot ellenőrzi: a build reprodukálhatóságát, a boot stabilitását, a tesztinfrastruktúra működését, a dokumentáció teljességét, a licencmegfelelést és az ismert hibák kezelését. A go/no-go döntés formalizált, minden feltételhez bizonyíték tartozik. A következő fejezet (Dokumentáció és reprodukciós csomag) erre az ellenőrzőlistára építve a dokumentáció és a reprodukciós csomag elkészítését mutatja be: a checklist itt rögzített elemeiből lesz a végleges kiadási csomag tartalomjegyzéke. A labor során a hallgató a saját projektjére alkalmazza a módszert, és megtanulja, hogy a kiadás nem egyetlen lépés, hanem ellenőrzött, reprodukálható folyamat.

33.2 Dokumentáció és reprodukciós csomag

A kiadási checklist (Kiadási checklist és kódminőség) után a következő kérdés: hogyan bizonyítjuk, hogy a kernelünk nem csak a fejlesztő gépén működik? Ehhez két dolgot kell együtt kezelni: a dokumentációt és a reprodukciós csomagot. A dokumentáció önmagában nem ér semmit, ha a leírt parancs nem futtatható; a reprodukciós csomag pedig vak, ha nem tudjuk, mit várunk tőle. Ebben a fejezetben ezt a két területet kapcsoljuk össze: először a reprodukálható build fogalmát és a csomag összetevőit tárgyaljuk, majd a toolchain, QEMU, konfiguráció és naplózás rögzítését, végül az architektúra- és API-dokumentációt. A fejezet végén egy teljes labor állítja össze a csomagot.

33.2.1 1. Reprodukálható build: mit jelent és miért követelmény?

A reprodukálható build azt jelenti, hogy ugyanabból a forrásból, ugyanazzal a toolchainnel, ugyanazzal a konfigurációval és ugyanabban a build környezetben futtatva a build parancsot bitazonos vagy funkcionálisan egyenértékű kimenetet kapunk. A kernel esetében a kimenet tipikusan a `kernel.bin`, a szimbólumtábla (`kernel.map`) és a napló. A bitazonosság akkor fontos, ha egy hibát egy adott buildhez kötünk; a funkcionális egyenértékűség pedig akkor, ha a hangsúly az ellenőrzött viselkedésen van.

A gyakorlatban a reprodukálhatóság négy pilléren nyugszik:

1. **Rögzített bemenet:** a forrás egyértelmű verziója (például git commit hash), a toolchain pontos verziója, a konfigurációs fájl tartalma.
2. **Hermetikus környezet:** a build nem függ a hoszt gép abszolút útvonalaitól, telepített csomagjaitól, környezeti változóitól vagy időzónájától.
3. **Determinizmus az eszközökben:** a fordító és linker ne szúrjon be időbélyeget, a forrásfájlok feldolgozási sorrendje rögzített, a párhuzamos build ne befolyásolja a kimenetet.
4. **Ellenőrzött kimenet:** a build után kiszámítjuk a kimenet hash-ét, és ezt a várt értékkel összehasonlítjuk.

Tipikus hiba, hogy a kernelszintű C kódban `__DATE__` vagy `__TIME__` makrót használunk egy verziószövegben. Ez minden buildnél más kimenetet ad. Hasonlóan, ha a build script a forrás abszolút útvonalát írja a hibakeresési szekcióba, a kép nem lesz reprodukálható más gépen. A megoldás az, hogy a verziószöveget a git commit hash-ből vagy egy rögzített verziófájlból olvassuk, a hibakeresési útvonalakat pedig relatívvá tesszük vagy kiiktatjuk.

A reprodukálhatóság nem azonos a „megismételhetőséggel”. A megismételhető build egyszerűen újra lefuttatható; a reprodukálható build viszont ugyanazt a kimenetet adja. Ez a különbség akkor válik láthatóvá, amikor egy régi kiadásban hibát keresünk. Ha csak megismételhető, a bináris eltérhet, és nem tudjuk eldönteni, hogy a hiba a forrásban vagy a környezetben volt-e. A reprodukálható build ezért bizonyíték arra, hogy a leírt forrás és a leírt környezet együtt adja a vizsgált viselkedést.

33.2.2 2. A reprodukciós csomag szerkezete

A reprodukciós csomag egy önálló könyvtár vagy archívum, amelyből egyetlen parancssorozattal eljutunk a futó kernelig. Az alábbi példa egy minimális `oslab` nevű kernel csomagját mutatja:

```
oslab-repro/
  README.md
  Makefile
  config.mk
  src/
      boot.s
      kernel.c
      linker.ld
  scripts/
      build.sh
      run_qemu.sh
      check_output.sh
  config/
      kernel_config.h
  logs/
      .gitkeep
  expected/
      boot_serial.txt
      sha256sum.txt
  measurements/
       notes.md
```

A `README.md` a belépési pont: leírja a célhardvert, a toolchain verzióját, a build és futtatási parancsot, valamint az elvárt kimenetet. A `config.mk` tartalmazza a fordító és QEMU elérési útját és verzióját. A `src/` a forrás, `scripts/` a reprodukálható parancsok, `config/` a fordítási feltételeket, `logs/` a friss naplókat, `expected/` az ellenőrzési alapértékeket, `measurements/` a mérési jegyzeteket.

A csomag akkor tekinthető teljesnek, ha egy tiszta gépen – vagy konténerben – a következő szekvencia működik:

```
git clone <helyőrző://oslab-repo> oslab
cd oslab
make toolchain-check
make clean
make build
make run
make verify
```

A `make toolchain-check` ellenőrzi, hogy a fordító és a QEMU verziója megegyezik a `config.mk`-ben rögzítettel. A `make verify` összehasonlítja a soros kimenetet az `expected/boot_serial.txt` tartalmával, és ellenőrzi a `kernel.bin` SHA-256 hash-ét. Az eltérés hibát jelez, ahelyett, hogy csendben elfogadná a rossz buildet.

A csomag nem csak a fejlesztő számára hasznos. Egy oktatói laborban a hallgató pontosan ugyanazt a környezetet kapja; egy hibabejelentésnél a karbantartó azonnal tudja, milyen buildet vizsgál a bejelentő. A csomag tehát kommunikációs eszköz is: ahelyett, hogy „nálam működik”, azt mondhatjuk, hogy „a mellékelt csomaggal az 5. lépésig működik, a 6. lépésben a várt `0x1000` helyett `0x0FFF` jelenik meg”.

33.2.3 3. Toolchain, QEMU és konfiguráció rögzítése

A toolchain rögzítése a reprodukálhatóság lelke. Egy saját, freestanding kernel esetében a toolchain általában keresztfordító lánc (cross compiler), de QEMU-n x86-64 vagy RISC-V célra a natív gcc is használható, ha a megfelelő `-ffreestanding` kapcsolókat beállítjuk. A fontos az, hogy a verziót expliciten rögzítsük, például `x86_64-elf-gcc 13.2.0`, `binutils 2.41`, `QEMU 8.2.1`. A verziószámok ellenőrzése a fejezet írásakor érvényes, friss stabilizálásnál ellenőrizni kell az aktuális kiadást.

Példa `config.mk`:

```
# config.mk - szemléltetés, nem éles környezetbe
CROSS_COMPILE ?= x86_64-elf-
CC            = $(CROSS_COMPILE)gcc
AS            = $(CROSS_COMPILE)as
LD            = $(CROSS_COMPILE)ld
OBJCOPY       = $(CROSS_COMPILE)objcopy
QEMU          = qemu-system-x86_64

REQUIRED_GCC   = 13.2.0
REQUIRED_BINUT  = 2.41
REQUIRED_QEMU   = 8.2.1

CFLAGS = -ffreestanding -fno-stack-protector -fno-pie -m64 -O2 \
         -g -Wall -Wextra -Iconfig
LDFLAGS = -m elf_x86_64 -T src/linker.ld -nostdlib -z noexecstack
```

A `REQUIRED_*` változókat a `make toolchain-check` a `$(CC) -dumpversion` és a `$(QEMU) --version` kimenetével hasonlítja össze. Nem elegendő azt írni, hogy „új gcc kell”; a pontos verzióra azért van szükség, mert a generált kód, a beépített függvények és a linkelési szabályok verziófüggők.

A QEMU esetében a teljes parancsot rögzíteni kell, mert az emulált hardver konfigurációja a reprodukció része. Példa `run_qemu.sh`:

```
#!/bin/sh
# Emulált labor: QEMU, eldobható lemezképpel.
set -eu
qemu-system-x86_64 \
  -machine q35,accel=tcg \
  -cpu max \
  -smp 2 \
  -m 64M \
  -drive file=build/kernel.bin,format=raw,if=floppy \
  -serial file:logs/serial.txt \
  -display none \
  -no-reboot \
  -no-shutdown
```

Ez a parancs rögzíti a géptípust, a CPU modellt, a magok számát, a memória méretét, a meghajtó típusát és a soros kimenet helyét. Ha valaki más gépen futtatja, ugyanazt az emulált hardvert kapja. A `-serial file:logs/serial.txt` a kernel soros kimenetét fájlba írja, így a kimenet később ellenőrizhető. A `-display none` fej nélküli futást jelent, ami szerveren is elérhető. A lemezkép helyőrző; valós lemezt nem érint.

A kernel konfigurációját külön fájlban, `config/kernel_config.h` néven rögzítjük:

```
// kernel_config.h - példa, helyőrző konfiguráció
#ifndef KERNEL_CONFIG_H
#define KERNEL_CONFIG_H

#define CONFIG_SERIAL_IO_PORT  0x3F8  // COM1
#define CONFIG_TIMER_FREQ_HZ   1000   // 1 kHz
#define CONFIG_STACK_SIZE      0x4000 // 16 KiB kernel stack
#define CONFIG_MAX_PROCESSES   8

#endif
```

A konfigurációs fájl egyetlen igazságforrás. A build script ezt include-olja, így a kernel viselkedése nem függ környezeti változóktól vagy parancssori definícióktól. Ha egy kísérlet során más paramétert használunk, azt a konfigurációs fájl módosításával és a commit üzenetben rögzítve tesszük.

A linker script is a konfiguráció része, mert a memóriatérkép fizikai és virtuális tartományait külön kell jelölni. Példa:

```
/* src/linker.ld - szemléltetés */
ENTRY(_start)
SECTIONS
{
    . = 1M;                    /* fizikai betöltési cím */
    .text : ALIGN(4K) {
        KEEP(*(.multiboot))
        *(.text)
    }
    .rodata : ALIGN(4K) { *(.rodata) }
    .data : ALIGN(4K) { *(.data) }
    .bss : ALIGN(4K) { *(COMMON) *(.bss) }
}
```

A `. = 1M` a betöltési címet rögzíti; a szekciók igazítása meghatározza a virtuális és fizikai tartományok kapcsolatát. Ha a kernel lapozást használ, a linker script mellett a lapozási táblázat is a reprodukciós csomag része.

33.2.4 4. Logolás és mérési napló

A log kétféle: build log és futási log. A build log a fordító, linker és objcopy kimenetét rögzíti, hogy egy hiba esetén visszakereshető legyen a pontos parancs. A futási log a kernel soros kimenetét, `panic` üzeneteit, stack trace-eket és a konfigurációt tartalmazza. A reprodukciós csomagban mindkettő szerepeljen, és a `logs/` könyvtár ne legyen üres a kiadásban.

Fontos, hogy a log formátuma kötött legyen. Példa kernel log sor:

```
[00000000] kernel: booting oslab 0.1.0
[00000001] serial: COM1 @ 0x3F8, 115200 8N1
[00000002] timer: PIT 1000 Hz
[00000003] pm: stack guard installed @ 0x200000-0x201000
```

Az elején a monoton óra értéke áll, utána a modul neve, végül az üzenet. Ez a formátum géppel elemezhető, és ember számára is olvasható. A `verify` script a `logs/serial.txt` tartalmát hasonlítja az `expected/boot_serial.txt`-hez, de csak a stabil sorokra. Az időbélyegeket nem szabad szigorúan összehasonlítani, mert QEMU alatt a futási idő ingadozhat; elegendő a sorok sorrendjét és tartalmát ellenőrizni.

A mérési napló elkülöníti a mért és a várt értéket. Példa a `measurements/notes.md`-ből:

```
## sys_write ciklusszám mérés
- Hardver: QEMU 8.2.1, q35, -cpu max, -smp 1, -m 64M
- Fordítás: x86_64-elf-gcc 13.2.0, -O2
- Órajel: emulált TSC, 1 GHz névleges
- Módszer: a kernel a sys_write hívás előtt és után rdtsc-et olvas,
  a különbséget kiírja a soros portra.
- Várt: 50-300 ciklus (a 16 bájtos soros írás, nem a teljes syscall)
- Mért: 172 ciklus
```

A várt érték itt intervallum, mert a TSC mérés zajos. A mért érték egyetlen futásból származik; a reprodukciós csomag futtatása után az olvasó saját mérését is rögzítheti. A lényeg, hogy a mért és a várt érték soha ne keveredjen egyetlen „eredmény” rovatba. Ez megfelel a könyv labor-szabályának: a nem futtatott kódot, mérést vagy benchmarkot nem mutatjuk be ellenőrzött eredményként.

33.2.5 5. Architektúra- és API-dokumentáció

A dokumentáció két szinten szükséges: az architektúra szintjén és az API szintjén. Az architektúradokumentáció a rendszer felépítését, rétegeit, memóriatérképét és vezérlési folyamatait írja le; az API-dokumentáció a határfelületek pontos szerződését: függvények, rendszerhívások, struktúrák, hibakódok.

Az architektúradokumentációhoz használjunk rétegdiagramot. Példa szövegesen:

```
+---------------------------+
| shell / init              |
+---------------------------+
| syscall API               |
+---------------------------+
| scheduler | mm | drivers  |
+---------------------------+
| arch (x86-64, interrupts) |
+---------------------------+
| hardware / QEMU           |
+---------------------------+
```

Minden réteghez tartozik egy rövid leírás: milyen modulok vannak benne, milyen interfészt kínálnak, és milyen hibakezelési szabályokat követnek. A memóriatérképet hexadecimális címekkel kell megadni, a virtuális és fizikai tartományokat külön jelölve:

Tartomány	Virtuális	Fizikai	Méret	Leírás
Kernel text	`0xFFFF800000000000`	`0x00100000`	64 KiB	kód
Kernel stack	`0xFFFF800000010000`	`0x00110000`	16 KiB	boot stack
Serial MMIO	`0xFFFF800000020000`	`0x000003F8`	8 B	COM1

A boot flow-t folyamatábrával vagy lépéslistával dokumentáljuk: firmware → bootloader → `_start` → stack beállítás → BSS törlés → `kernel_main` → eszközök init → `sti` → ütemező.

A lényeg, hogy a dokumentáció ne csak a kódot ismételje, hanem megmagyarázza a sorrend okát: miért kell a BSS-t a C kód előtt kinullázni, miért kell a megszakításokat csak az eszközök felkészítése után engedélyezni.

Az API-dokumentációban minden rendszerhívást névvel, paraméterekkel, visszatérési értékkel és hibakódokkal írunk le. Példa a `sys_write` dokumentációja:

```
sys_write(fd: uint32_t, buf: const char*, len: size_t) -> ssize_t

Leírás: len bájtot ír a buf pufferből a fd fájlleíróhoz.
Paraméterek:
  fd    - nyitott fájlleíró; 1 = soros konzol
  buf   - felhasználói puffer cím
  len   - írandó bájtok száma
Visszatérés: sikeresen kiírt bájtok száma.
Hibakódok:
  -1, errno = EBADF  - fd nem érvényes
  -1, errno = EFAULT - buf nem olvasható cím
```

A kernel belső API-ját is dokumentálni kell, nem csak a felhasználói határt. Egy belső függvény dokumentációja tartalmazza a tulajdonjogot: ki foglal, ki szabadít; milyen kontextusban hívható (megszakítási kontextus vagy folyamatkontextus); milyen versenyhelyzeti garanciákat ad. Példa:

```
/**
 * queue_push - beilleszt egy elemet a kernel üzenetsorba.
 * @q:    a cél üzenetsor
 * @item: a másolandó elem
 *
 * Visszatérés: 0 siker, -ENOMEM ha tele a sor.
 * Kontextus: folyamatkontextus; megszakítási kontextusból nem hívható.
 * Tulajdonjog: az item tartalmát a függvény lemásolja a sorba,
 *              a hívó a hívás után felszabadíthatja.
 * Szinkronizáció: a belső spinlock véd, hívó oldali zárolás nem kell.
 */
int queue_push(struct queue *q, const void *item);
```

Ez a dokumentáció nem helyettesíti a kódot, de felfedi a szerző szándékát. A kódolvasó nem találgat, hanem tudja, mikor biztonságos a hívás. A dokumentáció akkor jó, ha a kód változásakor frissül; a kiadási checklist része, hogy a dokumentáció és a header fájlok szinkronban vannak.

33.2.6 6. Tipikus hibák és javításuk

- `__DATE__` és `__TIME__` használata: minden build más binárist ad. Javítás: `git describe --always --dirty` kimenetét szúrjuk be a build scriptben.
- Abszolút útvonalak a debug szekciókban: a reprodukció más gépen eltérő hash-t ad. Javítás: `-fdebug-prefix-map=$(PWD)=.` vagy a debug info elhagyása.
- Nem rögzített toolchain verzió: később megváltozik a generált kód. Javítás: `config.mk` és `toolchain-check`.
- A konfiguráció a parancssorban él: `make CFLAGS=-DFOO` kívülről megváltoztatja a buildet. Javítás: minden konfiguráció a `config/kernel_config.h`-ban.
- Üres log: a hiba nem reprodukálható, mert nem tudjuk, mi történt. Javítás: `tee` a build loghoz, QEMU `-serial file`.
- Elavult API-dokumentáció: a függvény megváltozott, a leírás nem. Javítás: a dokumentációt a headerrel együtt verziózzuk, a review során mindkettőt ellenőrizzük.
- Mért és várt összemosása: „a boot 120 ms alatt futott, tehát jó” nem mondja meg, mi volt a várt. Javítás: külön oszlop vagy külön jegyzet.

33.2.7 7. Labor: reprodukciós csomag összeállítása

Ebben a laborban a korábbi fejezetekből ismert minimalista `oslab` kernelhez készítünk reprodukciós csomagot. A labor QEMU-ban fut, éles hardvert nem érint. A kiinduló állapot egy működő forrásfa, de a build parancs és a konfiguráció nincs rögzítve.

Lépések:

1. Hozd létre a fenti könyvtárstruktúrát, és másold be a forrásfájlokat a `src/` alá.
2. Töltsd ki a `config.mk` fájlt a helyi toolchain verziókkal.
3. Írd meg a `Makefile`-t a `build`, `run`, `verify`, `toolchain-check` célokkal.
4. Rögzítsd a QEMU parancsot a `scripts/run_qemu.sh`-ban.
5. Építsd meg a kernelt, és mentsd a build logot: `make clean && make build 2>&1 | tee logs/build.log`.
6. Futtasd a QEMU-t, és mentsd a soros kimenetet a `logs/serial.txt`-be.
7. Hasonlítsd össze a kimenetet a `expected/boot_serial.txt` tartalmával.
8. Számítsd ki a `build/kernel.bin` SHA-256 hash-ét, és rögzítsd a `expected/sha256sum.txt`-ben.
9. Ismételd meg a buildet egy tiszta checkoutban, és ellenőrizd, hogy a hash és a log megegyezik.
10. Készíts mérési naplót a `measurements/notes.md`-ben a `sys_write` TSC ciklusszámáról, külön mért és várt értékkel.

Elvárt eredmény: a `make verify` visszatérési értéke 0, a soros log tartalmazza a `kernel: booting oslab 0.1.0` sort, a `sys_write` 16 bájtos üzenetre 16-ot ad vissza, és a mért ciklusszám a dokumentált intervallumba esik.

Önellenőrző kérdések:

1. Milyen négy pilléren nyugszik a reprodukálható build? Nevezd meg mindegyiket egy-egy példával.
2. Miért nem elegendő a `gcc` verzióját „új”-ként megadni a reprodukciós csomagban?
3. Mi a különbség a mért és a várt érték között a mérési naplóban? Miért kell különválasztani?
4. Egy kernel fejlesztője azt mondja, hogy a build „nálam működik”, de a te gépeden a hash eltér. Milyen három dolgot ellenőriznél először?

33.2.8 8. Összefoglalás és továbbvitel

A dokumentáció és a reprodukciós csomag nem adminisztratív teher, hanem a kernel megbízhatóságának része. A jól rögzített build, a hermetikus környezet, a naplózott kimenet és a friss API-dokumentáció együtt teszik lehetővé, hogy egy hibát ne csak a szerző, hanem bárki reprodukálni tudjon. A következő fejezet (Ismert korlátok és nyitott kérdések) az ismert korlátokra és nyitott kérdésekre épít: pontosan azért tudunk majd korlátokról beszélni, mert a reprodukciós csomag világossá teszi, mi az, ami ellenőrzötten működik, és mi az, ami bizonytalan.

33.3 Ismert korlátok és nyitott kérdések

A kiadási checklist és a reprodukciós csomag elkészítése után könnyű úgy érezni, hogy a munka kész. A mérnöki gyakorlatban azonban a megbízható kiadás egyik legfontosabb, mégis gyakran elhanyagolt eleme az, hogy pontosan leírjuk, **mit nem tud** a rendszer, **milyen feltételek mellett nem teljesíti** a várakozásokat, és **mely kérdésekre nincs még válaszunk**. Ez a fejezet erről a korlátdokumentációról szól. A saját operációs rendszer esetében ez nem a kudarc

beismerése, hanem a tervezési érettség jele: a korlátok ismerete teszi lehetővé a tudatos döntést, a biztonságos üzemeltetést és a reális következő lépések megtervezését.

A fejezet három fő területet kapcsol össze:

1. a korlátok őszinte, reprodukálható dokumentálásának módszerét,
2. a teljesítmény-, biztonsági és kompatibilitási korlátok tipikus csoportjait,
3. a korlátokból és nyitott kérdésekből levezethető következő mérföldköveket.

Az itt bemutatott eszközök közvetlenül az előző egységben (Dokumentáció és reprodukciós csomag) elkészített dokumentációs és reprodukciós csomagra épülnek. A fejezet végén egy labor segítségével a saját OS-hez készítesz korlát- és kockázatlistát, amelyet az utolsó egység (Tanulási útvonal és szakirodalom) tanulási útvonalában is felhasználhatsz.

Miért dokumentáljuk a korlátokat?

Egy operációs rendszer – különösen egy saját, tanulási vagy kísérleti célból készült rendszer – soha nem teljes. A hiányzó funkciók, a nem optimalizált kódrészletek és a hardverfüggő feltételezések mind a rendszer viselkedését befolyásolják. Ha ezeket nem rögzítjük, akkor a felhasználó, a tesztelő vagy a későbbi fejlesztő hibaként fogja értelmezni azt, ami valójában ismert, szándékos vagy elfogadott korlát. A nem dokumentált korlát a gyakorlatban hamis ígéret: a rendszer úgy tűnik, mintha egy adott funkciót támogatna, miközben valójában csak speciális esetben működik.

A korlátdokumentáció elsődleges célja tehát a **kiszámíthatóság**. Egy jól vezetett korlátlista alapján pontosan megmondható, hogy egy adott konfigurációban milyen művelet hajtható végre, milyen teljesítmény várható, és milyen biztonsági garanciák léteznek. Ez különösen fontos akkor, ha a rendszer mások kezébe kerül, vagy ha hosszabb idő után térünk vissza hozzá.

A korlátdokumentáció soha nem helyettesíti a hibajegyzéket. A hiba olyan viselkedés, amely eltér a specifikációtól vagy a fejlesztői szándéktól. A korlát ezzel szemben a specifikáció része: kimondjuk, hogy a rendszer egy adott jellemzője **jelenleg nem támogatott**, vagy csak meghatározott keretek között működik. A hiba javítandó, a korlátot elfogadhatjuk, csökkenthetjük, vagy később feloldhatjuk.

A korlátdokumentációnak ugyanazokat a minőségi elveket kell követnie, mint a reprodukciós csomagnak:

- minden korlát kapjon egyedi azonosítót,
- legyen rögzítve a kiindulási állapot, a build és a futtatási környezet,
- a mért adatokat külön kell választani a várt vagy levezetett adatoktól,
- minden állítás legyen ellenőrizhető, vagy jelöljük meg, ha feltételezés,
- a dokumentum verziója és dátuma mindig szerepeljen.

A korlátok osztályozása

A saját OS korlátai ritkán egyformák; más-más területet érintenek, és más kezelési stratégiát igényelnek. Az osztályozás segít abban, hogy ne csak felsoroljuk a hiányosságokat, hanem azok hatását és enyhítési lehetőségét is megértsük.

Funkcionális korlátok. A rendszer nem tud egy adott műveletet végrehajtani, vagy a művelet csak szűk feltételek mellett érhető el. Például a saját kernelünkben a `fork` rendszerhívás nem létezik, helyette egy korlátozott `spawn` hívás indít új folyamatot. A fájlrendszer csak olvasható,

így a `write` fájlművelet a VFS rétegben `-EROFS` hibával tér vissza. Az ilyen korlátok jól dokumentálhatók, és a felhasználó számára egyértelmű hibakóddal jelölhetők.

Teljesítménykorlátok. A funkció létezik, de az elvárt idő- vagy erőforráskorlátot nem teljesíti. Itt különösen fontos a mérés és a várt érték elválasztása. Például a kontextusváltás átlagos ideje egy adott QEMU-konfiguráción mért 12,4 µs, míg a cél 5 µs alatt van. A teljesítménykorlátok gyakran hardver- és konfigurációfüggők, ezért a reprodukálhatóság kiemelt követelmény.

Biztonsági korlátok. A rendszer nem nyújt bizonyos védelmi mechanizmusokat, vagy a védelem csak részleges. Például a kernel nem használ ASLR-t, a stack nem rendelkezik canary-védelemmel, és a felhasználói folyamatok memóriája nincs teljesen elszigetelve a kerneltől. Az ilyen korlátoknál mindig jelezni kell, hogy milyen fenyegetési modellben értelmezhető a hiányosság, és milyen enyhítő intézkedések léteznek (például az emulált környezeten kívüli futtatás tiltása).

Kompatibilitási korlátok. A rendszer nem támogat egy adott ABI-t, fájlformátumot, eszközt vagy szabványt. A saját OS általában nem POSIX-kompatibilis, és nem futtat változtatás nélkül Linuxra fordított programokat. A rendszerhívások száma és szignatúrája saját, a libc pedig egy minimális réteg. A kompatibilitási korlátok dokumentálásánál fontos rögzíteni a rendszerhívások pontos listáját, a paraméterátadási konvenciót és a visszatérési értékeket, ahogyan azt a korábbi fejezetekben tettük.

Portabilitási és karbantartási korlátok. A kernel csak egy adott architektúrán (például x86_64) és csak egy adott boot-környezetben (például BIOS + multiboot) indul. A forráskód nem hordozható, mert a megszakításkezelés és a lapozás assembly részei architektúra-specifikusak. Ezt a korlátot is rögzíteni kell, mert meghatározza a jövőbeli portolási munka méretét.

Az osztályozás nem öncélú. A kategóriák segítségével minden korlát mellé hozzárendelhetünk egy kezelési stratégiát: elfogadás, enyhítés, halasztás vagy azonnali javítás.

Példa egy korlátdokumentációs bejegyzésre

A következő példa egy fiktív, de a tankönyv laborjaiban felépített mini OS egy valós korlátját mutatja be. A formátum szándékosan egyszerű, szöveges és reprodukálható.

```
KorlatID: FS-001
Komponens: initramfs fájlrendszer
Kategoria: funkcionális
Leiras: A fájlrendszer csak olvasható; sem a VFS, sem az eszközmeghajtó
         nem támogatja a blokkírási műveletet. A write() rendszerhívás
         minden fájlra -EROFS hibával tér vissza.
Reprodukcio:
  Build:  make clean && make build
  Run:    qemu-system-x86_64 -kernel build/os.bin -initrd build/initrd
  Teszt:  user$ echo teszt > /mnt/initrd/proba.txt
  Eredmeny: write failed: Read-only file system
Hatás: A felhasználói folyamatok nem tudnak perzisztens adatot menteni.
       A rendszer újraindítás után minden állapota elvész.
Enyhites: A következő mérföldkőben egy naplózó, írható fájlrendszer
          bevezetése. Addig a felhasználói adatok csak a RAM-ban élnek.
Statusz: elfogadott, tervezett feloldás a 2. mérföldkőben
```

A példa jól mutatja a lényeget: a korlát leírása, a reprodukálhatóság, a hatás és az enyhítés együtt ad teljes képet. A `-EROFS` hibakód és a build/run parancsok konkrétak, így bárki ellenőrizheti az állítást.

A teljesítménykorlátok mérése

A teljesítménykorlátok dokumentálásához mérési adatok kellenek. A tankönyv korábbi fejezeteiben (különösen a 12. és a 29. fejezetben) már bemutattuk a mérési alapelveket. Itt most arra koncentrálunk, hogyan válik egy mérés a korlátlisták hiteles elemévé.

Minden teljesítménykorláthoz legalább a következőket kell rögzíteni:

- a hardver- vagy emulátorkonfiguráció,
- a fordítási beállítások,
- az órajelfrekvencia vagy az emulátor verziója,
- a mérési módszer leírása,
- az ismétlésszám,
- a mért átlag, szórás, minimum és maximum,
- a várt vagy elvárt érték,
- az eltérés magyarázata.

Például a kontextusváltási idő méréséhez a kernelben elhelyezhetünk egy mérési pontot, amely a timer interrupt és a következő folyamat első utasítása közötti időt rögzíti ciklusokban. A mért nyers ciklusszámot átszámítjuk időre, és táblázatba foglaljuk.

Tegyük fel, hogy a mérés során az alábbi adatokat kaptuk:

Mérési pont	Átlag (µs)	Szórás (µs)	Min (µs)	Max (µs)	Várt (µs)	Státusz
Kontextus-váltás	12,4	2,1	9,8	18,2	$< 5{,}0$	nyitott
Syscall overhead (empty)	3,8	0,4	3,2	4,9	$< 2{,}0$	nyitott
Interrupt latency	7,1	1,8	4,5	12,0	$< 3{,}0$	nyitott

A táblázatban a tizedesvesszőt a tankönyv konvenciója szerint `{,}` jelzi, a mért és a várt értékek pedig külön oszlopban szerepelnek. A mérési naplóban rögzíteni kell a QEMU verzióját, a host CPU-t, a kernel compile flagjeit és az ismétlésszámot.

A várt érték meghatározása nem lehet önkényes. Ha nincs előzetes mérés vagy analitikus becslés, akkor a várt értéket jelöljük `becslés, nem mért` jelöléssel. A fejezet egyik legfontosabb szabálya: **a nem mért vagy nem ellenőrzött eredményt soha ne tüntessük fel mért értékként.**

A biztonsági korlátok kezelése

A biztonsági korlátok dokumentálása kényes terület, mert a hiányosságok felsorolása önmagában is támadási útmutató lehet. A tankönyv etikai irányelve szerint ezért itt is a védekezésre koncentrálunk, és nem adunk kihasználási lépéseket. A dokumentációban a biztonsági korlátokat a **fenyegetési modell** keretében írjuk le.

Egy tipikus mini OS esetében a biztonsági korlátok a következők lehetnek:

- nincs felhasználói/kernel memóriaizoláció, mert a lapozás nem használ privilege bitet, így a felhasználói folyamat olvashatja a kernel memóriáját;
- nincs ASLR, ezért a futási címek előre meghatározottak;
- nincs stack canary, ezért a puffertúlcsordulás nem észlelhető;
- nincs NX bit, ezért a verem végrehajtható;
- a rendszerhívások nem ellenőrzik teljes körűen a mutatók érvényességét, így egy hibás pointer kernelbeli hibaüzenetet vagy összeomlást okozhat.

Ezeket a korlátokat nem hallgathatjuk el, de a leírásban mindig jelezzük, hogy az adott hiányosság milyen környezetben jelent kockázatot. A mini OS jellemzően izolált, emulált környezetben fut, így a fenyegetési modellben a támadó nem fér hozzá a hoszt rendszerhez. A dokumentációban ezt így rögzítjük:

> A felsorolt biztonsági korlátok csak a virtuális gép belső, elkülönített környezetében értelmezhetők. A kernel nem futtatható közvetlenül a hoszt hardveren, és nem nyújt védelmet a hoszt erőforrásai ellen.

A kockázatok értékelésekor a valószínűséget és a hatást külön kell kezelni. Például a NX bit hiánya nagy hatású, de alacsony valószínűségű, ha a rendszer nem futtat nem megbízható kódot. A biztonsági korlátoknál az enyhítés gyakran egyszerű: ne engedjünk be nem megbízható bemenetet, vagy korlátozzuk a rendszerhívásokat.

A kompatibilitási korlátok dokumentálása

A kompatibilitási korlátok közvetlenül érintik a felhasználói programok írását. A saját OS általában nem Linux-, nem Windows- és nem is POSIX-kompatibilis. Ezt a felhasználói réteg számára egyértelműen jelezni kell.

A dokumentációban fel kell sorolni:

- a támogatott rendszerhívások pontos listáját és sorszámát,
- a paraméterátadási konvenciót (például `syscall` utasítás, regiszterkiosztás),
- a visszatérési értékek és hibakódok konvencióját,
- a támogatott fájlformátumokat (például csak ELF64, csak statikus csatolás),
- az eszközök listáját és azok elérési módját.

Egy példa:

```
Syscall: 3 (read)
Parameterek: fd (RDI), buf (RSI), count (RDX)
Visszateres: beolvasott bájtok száma, vagy -1 és errno
Korlatok: csak a /dev/serial eszközről olvas; a count maximum 4096 bájt;
          nem szakítható meg; blokkoló hívás, nincs timeout.
```

Az ilyen dokumentáció segít a felhasználónak eldönteni, hogy a saját OS alkalmas-e egy adott feladatra. A kompatibilitási korlátoknál fontos, hogy a dokumentum verziója kövesse a rendszerhívások ABI-jában bekövetkező változásokat, mert a felhasználói programok és a kernel közötti illesztés ezen a határfelületen múlik.

A korlátok életciklusa

A korlátok nem statikusak. Egy bejegyzés több állapoton mehet át:

1. **nyitott** – a korlát ismert, de még nincs elfogadott döntés a kezeléséről;
2. **elfogadott** – a korlátot tudatosan vállaljuk, mert a feloldás költsége nagyobb, mint a haszon;
3. **enyhített** – a korlát megmarad, de csökkentettük a hatását (például hibakóddal jelezzük, vagy felhasználói oldali kikerülés létezik);
4. **feloldott** – a korlátot megszüntettük; a bejegyzést lezárjuk, és hivatkozunk az azt feloldó commitra vagy változtatásra.

A korlátdokumentáció ezért nem egyszeri kiadási melléklet, hanem a fejlesztési folyamat élő része. A verziókezelő rendszerben a korlátleíró fájlt a forráskóddal együtt kell kezelni, és minden kiadásnál frissíteni kell.

Nyitott kérdések

A korlátok mellett a kiadási dokumentációnak tartalmaznia kell azokat a **nyitott kérdéseket** is, amelyekre a fejlesztőcsapatnak nincs egyértelmű válasza. A nyitott kérdés nem hiba és nem is korlát, hanem egy döntési pont, amely befolyásolja a rendszer jövőjét.

Tipikus nyitott kérdések egy saját OS esetében:

- Milyen ütemezőt válasszunk a következő mérföldkőhöz: maradjunk a round-robin mellett, vagy vezessünk be prioritásos ütemezést?
- Érdemes-e bevezetni a demand paginget, vagy a rendszer mérete miatt elegendő a teljes memóriakiosztás?
- Milyen biztonsági modellt kövessünk: capability-alapú hozzáférést, vagy a klasszikus felhasználó/gyökér jogosultságot?
- Hogyan biztosítsuk az ABI-stabilitást, ha a rendszerhívások még változhatnak?
- Milyen mérési módszerrel kaphatunk reprodukálható WCET-értékeket az emulált környezetben?
- Szükség van-e TCP/IP-stackre, vagy elegendő a soros porton keresztüli kommunikáció?

Ezek a kérdések bekerülnek a korlátlistába, de külön szekcióba, mert a kezelésük nem egyszerű elfogadás vagy elutasítás, hanem a következő mérföldkövek tervezésének inputja.

A következő mérföldkövek meghatározása

A korlátlista és a nyitott kérdések alapján a következő mérföldkövek reálisan tervezhetők. A tervezés során minden korlátot és kérdést értékelünk a **hatás**, a **megvalósítási költség** és a **kockázat** szempontjából.

Egy lehetséges rövid távú mérföldkő:

1. **Stabilizálás és mérési alap**: a jelenlegi kernel építésének és futtatásának automatizálása, a kontextusváltási idő és a syscall-overhead mérése, a korlátlistában szereplő mért adatok kiegészítése.
2. **Hibajavítás**: a dokumentált korlátok közül azok a hibák, amelyek a specifikációtól való eltérést jelentenek, javításra kerülnek.

Középtávú mérföldkő:

1. **Preemptív ütemező és prioritás**: a round-robin ütemezőt felváltja egy prioritásos, preemptív ütemező.
2. **Írható fájlrendszer**: egy naplózó, egyszerű fájlrendszer bevezetése, hogy a rendszer állapota túlélje az újraindítást.
3. **Felhasználói mód megerősítése**: a lapozás privilege-bitjeinek helyes használata, hogy a kernel és a felhasználói folyamatok memóriája elváljon.

Hosszú távú mérföldkő:

1. **SMP-támogatás**: a többprocesszoros ütemezés és a spinlock-alapú szinkronizáció bevezetése.
2. **Hálózati stack**: egy minimális Ethernet- és IP-réteg a virtuális hálózati eszközhöz.
3. **Biztonsági modell**: capability- vagy jogosultság-alapú hozzáférés-vezérlés, a rendszerhívások argumentum-ellenőrzésének megerősítése.

A mérföldkövekhez mindig hozzárendelünk elfogadási kritériumokat és mérési célokat. Például a preemptív ütemező bevezetésénél a kontextusváltási idő nem haladhatja meg a 5 µs-t, és a rendszernek 1000 folyamat mellett is stabilan kell működnie.

A korlát- és kockázatlista szerkezete

A laborban elkészítendő korlát- és kockázatlista egy strukturált dokumentum, amely a fenti elveket követi. A lista minden bejegyzése a következő mezőket tartalmazza:

Mező	Leírás
`ID`	Egyedi azonosító, például `PERF-001`, `SEC-002`
`Komponens`	Az érintett kernelkomponens vagy modul
`Kategória`	funkcionális, teljesítmény, biztonság, kompatibilitás, portabilitás
`Leírás`	A korlát pontos, tömör megfogalmazása
`Reprodukció`	A kiindulási állapot, build- és futtatási parancsok, tesztlépések
`Mért adat`	A mérés eredménye, statisztikai jellemzőkkel
`Várt adat`	Az elvárt vagy becsült érték, egyértelműen jelölve, ha nem mért
`Hatás`	A korlát következménye a felhasználóra, a rendszerre vagy a fejlesztésre
`Enyhítés`	Az adott korlát hatását csökkentő intézkedés
`Státusz`	nyitott, elfogadott, enyhített, feloldott

A kockázatok értékeléséhez minden korláthoz hozzárendelünk egy **valószínűség** és egy **hatás** besorolást (alacsony, közepes, magas). A kettő szorzatából vagy mátrixából egy egyszerű kockázati szint képezhető. A kockázati mátrixot a dokumentumban táblázatként vagy hőtérképként is ábrázolhatjuk, de a szöveges leírás ezt nem helyettesíti.

Labor: korlát- és kockázatlista készítése

A labor célja, hogy a saját OS-hez elkészíts egy olyan korlát- és kockázatlistát, amely a kiadási csomag részeként is használható. A labor a Dokumentáció és reprodukciós csomag reprodukciós csomagjára épül, és nem igényel új kernelkódot, bár a mérésekhez szükség lehet a meglévő mérési pontok futtatására.

Előkészítés

Hozz létre egy `limits/` mappát a projekt gyökerében. A mappa tartalma:

- `limits.md` – a korlátlista emberi olvasásra szánt formában,
- `limits.yaml` – gépi feldolgozásra alkalmas változat,
- `measurements/` – a mérési naplók,
- `config/` – a rögzített build- és futtatási konfigurációk.

A kiindulási állapot a saját OS utolsó stabil verziója, amelynek buildje és futtatása a Dokumentáció és reprodukciós csomag szerint reprodukálható.

1. lépés: a rendszerkonfiguráció rögzítése

A lista elejére írd be a rendszer azonosító adatait:

- kernelverzió vagy commit hash,
- buildparancs,
- futtató környezet (QEMU verzió, CPU típus, memória méret),
- host rendszer leírása (CPU, RAM, operációs rendszer),

- a dokumentum készítésének dátuma.

Például:

```
Kernel:     mini-os v0.9.1 (commit 3f2a9c)
Build:      make clean && make build
Futtatas:   qemu-system-x86_64 -m 256M -kernel build/os.bin
Host CPU:   x86_64, 8 mag, 3,2 GHz
QEMU:       7.2.0
Datum:      2025-01-15
```

2. lépés: a korlátok összegyűjtése

Készítsd el a korlátok első vázlatát az alábbi szempontok szerint:

- Milyen funkciókat nem támogat a rendszer?
- Milyen teljesítménycélokat nem ér el?
- Milyen biztonsági mechanizmusok hiányoznak?
- Milyen szabványokkal vagy ABInkkal nem kompatibilis?
- Milyen hardver- vagy bootkörnyezetben fut egyáltalán?

Minden azonosított korláthoz rendelj egyedi azonosítót. A kategóriák segítségével csoportosítsd őket.

3. lépés: reprodukció és mérés

Legalább három korláthoz végezz mérést, hogy a mért adatokat be tudd írni a listába. A mérések a következők lehetnek:

- kontextusváltási idő,
- rendszerhívási overhead,
- megszakítási késleltetés,
- bootidő,
- memóriaallokátor műveleti ideje.

A mérési naplóban rögzítsd a mérési módszert, az ismétlésszámot, a nyers adatokat és a statisztikai összefoglalót. A mért és a várt értéket a korlátlistában külön oszlopban tüntesd fel.

4. lépés: kockázatértékelés

Minden korláthoz add meg:

- a kockázat valószínűségét (alacsony, közepes, magas),
- a hatás mértékét (alacsony, közepes, magas),
- a kockázati szintet (például a valószínűség × hatás mátrixa szerint),
- az enyhítő intézkedést.

A kockázati mátrixot a `limits.md` elején ábrázold táblázatként. Például:

Valószínűség \ Hatás	Alacsony	Közepes	Magas
Alacsony	elfogadható	figyelendő	csökkentendő
Közepes	figyelendő	csökkentendő	kezelendő
Magas	csökkentendő	kezelendő	azonnali intézkedés

5. lépés: a dokumentum elkészítése

A `limits.md` a következő szekciókat tartalmazza:

1. Bevezetés és a fenyegetési modell rövid leírása,
2. rendszerkonfiguráció,
3. kockázati mátrix,
4. korlátlista táblázat,
5. nyitott kérdések,
6. következő mérföldkövek,
7. melléklet: mérési naplók.

A `limits.yaml` ugyanezt gépi feldolgozható formában tárolja. Példa egy bejegyzésre:

```yaml
- id: SEC-001
  component: memory
  category: security
  description: "Nincs stack canary, a puffer-túlcsordulás nem észlelhető."
  reproduction: "Build: make clean && make build; Run: lásd config/qemu.conf"
  measured: null
  expected: "A stack canary jelenléte esetén a túlcsordulás panikot váltana ki."
  impact: "Egy nem megbízható bemenet kernel-stack túlcsordulást okozhat."
  mitigation: "A rendszer csak megbízható bemenetet fogad; a hoszt izolált."
  status: accepted
  risk_probability: low
  risk_impact: high
```

A `measured` null értéket mutat, mert ehhez a korláthoz nem tartozik konkrét mérés, csak leírás. Ahol van mért adat, ott a `measured` mezőbe struktúrált adat kerül.

6. lépés: ellenőrzés és visszaállítás

A labor végén a `limits/` mappát add hozzá a verziókezelőhöz. A mérési naplók és a konfigurációk legyenek reprodukálhatók. Ha a mérések során a kernel állapota megváltozott, állítsd vissza a kiindulási konfigurációt a reprodukciós csomagban rögzített módon.

Elvárt eredmény és önellenőrzés

A labor akkor sikeres, ha:

- a korlátlista legalább 10 bejegyzést tartalmaz,
- legalább három különböző kategóriából származnak a bejegyzések,
- legalább három mérés mért adatait rögzítettük,
- minden bejegyzésben a mért és a várt érték külön van választva,
- minden biztonsági korlátnál szerepel a fenyegetési modellre való hivatkozás,
- a nyitott kérdések és a következő mérföldkövek szakaszok ki vannak töltve.

Önellenőrző kérdések:

1. Mi a különbség a hiba és a korlát között? Adj példát mindkettőre a saját OS-ből.
2. Miért kell a mért és a várt értéket külön dokumentálni? Milyen következménye van annak, ha valaki ezt nem teszi meg?
3. Hogyan csökkenthető egy biztonsági korlát hatása anélkül, hogy a korlátot feloldanánk? Adj egy konkrét példát.

Gyakori hibák a korlátdokumentációban

A korlátok dokumentálása során több tipikus hiba fordul elő, amelyek aláássák a lista hitelességét.

A korlátok elhallgatása vagy szépítése. A leggyakoribb hiba, hogy a fejlesztő nem írja le a hiányosságokat, mert attól tart, hogy a rendszer kevésbé tűnik értékesnek. A valóságban a nyílt korlátdokumentáció növeli a bizalmat: a felhasználó tudja, mire számíthat.

A mért és várt érték összemosása. Ha egy teljesítménykorlátnál a várt értéket úgy tüntetjük fel, mintha mértük volna, az félrevezető. Mindig jelöljük, ha egy érték becslésből, szimulációból vagy korábbi tapasztalatból származik.

A reprodukálhatóság hiánya. A korlát leírása nem áll meg annyiban, hogy „a rendszer lassú”. Pontosan rögzíteni kell a környezetet és a mérési módszert, különben az állítás nem ellenőrizhető.

A korlátok és hibák összekeverése. Ha egy funkció a specifikáció szerint nem támogatott, az korlát. Ha egy funkció a specifikáció szerint támogatott, de nem működik, az hiba. A kettő elválasztása a lista alapvető minőségi követelménye.

A kockázatértékelés szubjektivitása. A valószínűség és a hatás besorolásához előre definiált kritériumok kellenek. Például a valószínűség alacsony, ha a hiba bekövetkezésének esélye 1% alatti; a hatás magas, ha a rendszer teljes leállását okozza. A kritériumokat a lista elején rögzítsd.

A kompatibilitási korlátok hiánya. Gyakori, hogy a saját OS nem POSIX-kompatibilis, de ezt nem dokumentálják. A felhasználói programok írásakor ez súlyos félreértésekhez vezet. A rendszerhívások listája és az ABI-konvenció a kompatibilitási szekció kötelező eleme.

A fenti hibák elkerülése érdekében a korlátdokumentációt ugyanazzal a szigorúsággal kell kezelni, mint a forráskódot: verziózzuk, review-ztatjuk, és a kiadás előtt ellenőrizzük a teljességét.

A korlátlista mint a folyamatos fejlesztés alapja

A korlátlista nem csupán a kiadás pillanatában hasznos, hanem a fejlesztési ciklus állandó eszköze. Minden új funkció bevezetése előtt a lista segít felmérni, hogy az adott változtatás mely korlátokat oldja fel, és milyen új korlátokat hoz létre. A nyitott kérdések pedig a kutatási irányokat jelölik ki.

Például, ha a következő mérföldkő a preemptív ütemező bevezetése, akkor a korlátlistában a `SCHED-001` azonosítójú bejegyzés azt dokumentálja, hogy a jelenlegi round-robin ütemező nem kezel prioritást. A nyitott kérdések között szerepel, hogy FIFO vagy EDF irányba érdemes-e továbblépni. A mérföldkő elfogadási kritériuma pedig az, hogy a prioritásos ütemező mellett a kontextusváltási idő nem haladja meg az 5 µs-t, és a rendszer 1000 folyamattal is stabilan fut.

Ez a gondolkodásmód a tankönyv utolsó nagy egységéhez is kapcsolódik: a korlátokból és nyitott kérdésekből kiindulva lehet megtervezni a továbbtanulási útvonalat, és kiválasztani azokat a haladó témákat, amelyek a legnagyobb értéket adják a projektnek.

Összefoglalás

A saját operációs rendszer kiadási folyamatában az ismert korlátok őszinte dokumentálása ugyanolyan fontos, mint a működő kód. A korlátlista strukturált formában rögzíti, hogy a rendszer mit nem tud, milyen teljesítményt nem ér el, milyen biztonsági garanciákat nem nyújt, és milyen szabványokkal nem kompatibilis. A mért és a várt értékek elkülönítése, a reprodukálhatóság és a fenyegetési modellre való hivatkozás a dokumentáció hitelességének alapját adják.

A korlátokból és nyitott kérdésekből reális következő mérföldkövek vezethetők le. A fejezet laborjában elkészített korlát- és kockázatlista ezt a folyamatot teszi gyakorlattá: a hallgató a saját OS-éhez készíti el a dokumentumot, méréseket végez, és a kockázatokat értékeli. Az így létrejött lista az utolsó tanulási egységben (Tanulási útvonal és szakirodalom) a tanulási útvonal

tervezésének kiindulópontja lesz, mert a legnagyobb hatású hiányosságok és a legérdekesebb nyitott kérdések együtt mutatják meg, melyik irányba érdemes tovább haladni.

33.4 Tanulási útvonal és szakirodalom

33.4.1 Bevezetés: miért tanulási útvonal kell?

Ez az alfejezet lezárja a könyv törzsanyagát, de nem a tanulási folyamatot. Az operációs rendszerek belső működése gyorsan változó, szabványokkal, kernelverziókkal és hardverplatformokkal erősen összefonódó terület. Aki a kernel- és rendszerprogramozást hosszú távon akarja művelni, annak nem elegendő egyetlen tankönyv vagy egyetlen labor elvégzése. Olyan térképre van szüksége, amely segít kiválasztani a következő lépést, ellenőrizni a tudást, és a gyakorlatban reprodukálható módon haladni. Ennek a szakasznak ez a célja: bemutatjuk a továbbtanulás útját a kernel- és rendszerprogramozásban, a Linux kernel, az RTOS, a virtualizáció és a biztonság területén, továbbá irodalmi és közösségi tájékozódási pontokat adunk.

A fejezet nem ismétli meg a korábbi fejezetek alapfogalmait. A virtuális memóriát, az ütemezőt vagy a fájlrendszereket már tárgyaltuk; itt azt vizsgáljuk, hogyan lehet ezeket a fogalmakat valódi, ellenőrzött mérnöki gyakorlattá fejleszteni. Ehhez három pillért használunk:

1. **Tanulási útvonal** – milyen sorrendben, milyen szakirányok mentén érdemes elmélyíteni a tudást.
2. **Ellenőrző eszközök** – hogyan kell mérni, naplózni, és a várt eredményt elválasztani a mért értéktől.
3. **Források és közösségek** – milyen szakirodalom, dokumentáció, kurzus és közösségi csatorna támogatja a haladást.

A záró labor egy **személyes tanulási terv** elkészítése. A terv nem csupán egy olvasmánylista, hanem reprodukálható jegyzetfüzetben vagy emulált környezetben rögzített, konkrét mérföldköveket, forrásokat, mérési pontokat és ellenőrzési kritériumokat tartalmazó ütemterv. A cél az, hogy a könyv elolvasása után a tanuló ne álljon meg a fogalmi ismereteknél, hanem saját tempójában, mérhetően fejlődjön tovább.

33.4.2 A tanulási útvonal általános elvei

A kernel- és rendszerprogramozás tanulása nem lineáris folyamat. A hardver, a kernel és a felhasználói tér közötti kölcsönhatások miatt minden új ismeret visszahat a korábbiakra. Ezért a tanulási útvonalat érdemes **spirálisnak** tekinteni: az alapfogalmakat először egyszerű, emulált környezetben alkalmazzuk, majd ugyanahhoz a témához mélyebb szinten térünk vissza, például valós hardveren vagy nagyobb kódbázisban.

Az alábbi elvek minden szakirányra érvényesek:

- **Kis, ellenőrizhető lépések.** Egyetlen mérföldkő se legyen „megérteni az ütemezőt”, hanem például „implementálni egy round-robin ütemezőt, és lemérni az átlagos várakozási időt 10 darab P_i folyamat esetén”. Az ilyen célhoz definíció of done tartozik.
- **Reprodukálhatóság.** Minden kísérletnek rögzített konfigurációval kell futnia. A CPU, a kernelverzió, a fordító, a linker script, a QEMU-paraméterek és a mérési módszer együtt alkotják a reprodukciós csomagot.

- **Mért és várt különválasztása.** A jegyzetfüzetben minden mérésnél külön mezőbe kerül a várt (levezetett, dokumentációból származó) érték és a mért érték. A kettő összehasonlítása adja az ellenőrzés alapját.
- **Forrásverziók és szakaszok rögzítése.** A kernelfejlesztésben egy állítás a 6.6-os kernellel lehet igaz, a 6.12-essel pedig már nem. A szakirodalmi hivatkozásnál mindig fel kell tüntetni a dokumentum nevét, verzióját, dátumát és a konkrét szakaszt.
- **Izolált kísérletek.** A laborok QEMU-ban, virtuális gépben vagy eldobható hardveren futnak. A valós hoszt lemezét, bootloaderét és rendszerpartícióját tilos módosítani. Ez a biztonsági és etikai szabály a továbbtanulás során is érvényes.
- **Közösségi visszajelzés.** A kód, a mérési jegyzőkönyv és a dokumentáció megosztása segít a vakfoltok felismerésében. A kérdésfeltétel előtt azonban mindig minimalisan reprodukálható példát kell készíteni.

A következő szakirányok mindegyike ezekre az elvekre épül, de más hangsúlyt ad a hardver, a determinizmus, az izoláció vagy a teljesítmény kérdésének.

33.4.3 Szakirányok és mérföldkövek

A rendszerprogramozásban négy nagy továbbtanulási ágat különböztethetünk meg: a Linux kernel belső világa, a valós idejű és beágyazott rendszerek, a virtualizáció és konténerek, valamint a biztonság és izoláció. A határok természetesen átjárhatók, de a kezdő útvonal tervezésénél érdemes egy elsődleges irányt választani.

33.4.4 Linux kernel fejlesztés és belső működés

Aki a Linux kernel felé indul, annak a célja nem feltétlenül a mainline kernel patch beküldése; lehet az is, hogy jobban értse a rendszerhívásokat, a memóriakezelést vagy a hálózati stacket. A javasolt haladási sorrend:

1. **Rendszerhívások és ABI.** A könyvben már bemutatott `open`, `read`, `write`, `mmap` stb. rendszerhívásokat érdemes saját kernelmodulból vagy BPF-programból megvizsgálni. A tanuló készítsen egy egyszerű syscall-trace naplót, és rögzítse a rendszerhívás paramétereit, visszatérési értékét és a hibakódot (`errno`).
2. **Kontextusváltás és ütemező.** A Linux CFS és a valós idejű scheduling osztályok tanulmányozása a kernel source `kernel/sched/` könyvtárában. Egy QEMU-ban futó kernel `perf sched` és `trace-cmd` kimenetét kell reprodukálni, nem csupán elolvasni.
3. **Memóriakezelés.** A lapozás, a buddy allokátor és a slab/SLUB réteg megértése a `mm/` alatt. Gyakorlat: mérni a `vmalloc` és a `kmalloc` késleltetését, a laphibák számát, és értelmezni a `mmap`-elt területek viselkedését.
4. **Fájlrendszer és I/O.** A VFS, az ext4 vagy a btrfs egy-egy kiválasztott része. A crash-konzisztenciát és a journalinget a könyvben tanultak szerint kell tesztelni, például hirtelen leállást szimulálva.

A mérföldkövekhez tartozó források: a hivatalos Linux kernel dokumentáció, a kernel forrásában található `Documentation/` könyvtár, valamint a `kernelnewbies` közösség kezdőknek szóló anyagai. A verziókat mindig rögzíteni kell; a példákban használt 6.6-os kernelverziószám csak kiindulási pont, a tanulónak az aktuális stabil verziót kell ellenőriznie a hivatalos forrásban.

33.4.5 RTOS és beágyazott rendszerek

A beágyazott és valós idejű vonal arra a kérdésre koncentrál, hogyan lehet determinisztikus időkorlátokat tartani korlátozott erőforrások mellett. A FreeRTOS és a Zephyr nyílt forráskódú RTOS-ek, a QNX és a VxWorks pedig kereskedelmi rendszerek, amelyek a könyv VI. részében már megjelentek. A továbbtanulás itt a következő lépéseket ajánlja:

1. **Determinizmus és latency mérése.** A tanuló mérje a megszakítási késleltetést, a task-váltás idejét és a jittert egy rögzített RTOS-konfigurációban. A mérésnél fel kell tüntetni a hardvert, az órajelfrekvenciát, a fordítót és a mérési módszert.
2. **Priority inversion és öröklés.** A könyvben tanult klasszikus probléma reprodukálása FreeRTOS vagy Zephyr alatt, három prioritású ($p_1 > p_2 > p_3$) folyamattal. A mért blokkolási időt kell összehasonlítani a várt értékkel.
3. **WCET és erőforrás-elemzés.** Egy egyszerű feladat legrosszabb esetű végrehajtási idejének becslése méréssel, majd összevetése statikus elemzéssel. Itt kiemelten fontos, hogy a becslés és a mért érték ne keveredjen.
4. **Watchdog, queue és event flag minták.** A könyvben bemutatott RTOS-mintákból egy minimális rendszert építeni, és tesztelni a hibakezelést.

Az RTOS-vonalhoz tartozó irodalom elsősorban a hivatalos FreeRTOS és Zephyr dokumentáció, valamint a QNX és VxWorks gyártói dokumentációja. A verziók itt is kritikusak: a FreeRTOS kernelverziója és a portolt architektúra (például ARM Cortex-M7 vagy RISC-V) együtt határozza meg a viselkedést.

33.4.6 Virtualizáció és konténerek

A virtualizáció és a konténerizáció az operációs rendszerek izolációs határainak újragondolása. Aki ezen a vonalon halad, annak a célja a Type 1/Type 2 hypervisorok, a KVM, a QEMU, az OCI konténerek és a namespaces/cgroups mechanizmusok elmélyítése.

1. **Hypervisor alapok.** A KVM és a QEMU együttműködésének vizsgálata. A gyakorlat: vendég kernel indítása, a virtio eszközök konfigurálása, és a vendég/hoszt közötti teljesítménykülönbség mérése.
2. **Namespaces és cgroups.** A Linux namespace-ek (PID, net, mount, UTS, user) és a cgroup-ok v1/v2 hierarchiájának kézbentartása. Egy egyszerű, `chroot`-alapú izolált környezetből kiindulva OCI-kompatibilis konténer futtatása.
3. **Migráció és snapshot.** Virtuális gép élő migrációja QEMU/KVM alatt, és a teljesítményre gyakorolt hatás mérése.
4. **Unikernel és könnyű izoláció.** Az unikernel modellek (pl. library OS) vagy a WASM-alapú izoláció tanulmányozása, a biztonsági és teljesítménykompromisszumok dokumentálásával.

A virtualizációs vonal irodalma a hivatalos KVM dokumentáció, a QEMU dokumentáció, a Linux kernel `Documentation/virt/` része, valamint az OCI runtime specifikáció. A specifikációk verziószáma és a kernelverzió itt is elválaszthatatlan.

33.4.7 Biztonság és izoláció

A biztonsági szakirány a védekezésre, az izolációra és a biztonságos tervezésre összpontosít. A cél nem exploitok írása, hanem a támadási felület csökkentése, a kernel védelmi mechanizmusainak megértése és a hibakeresés biztonságos környezetben.

1. **Jogosultságok és capabilities.** A Linux capability modellje, a `setuid`/`setgid` viselkedés, az ACL-ek és a fájlrendszer-jogosultságok közötti különbség. Gyakorlat: egy daemon jogosultságainak szűkítése minimális képességekre.
2. **SELinux, AppArmor, seccomp.** A kötelező hozzáférés-vezérlés (MAC) és a syscall-szűrés. A tanuló készítsen egy seccomp-profilt egy egyszerű processzhez, és mérje a szűrés hatását a syscall-késleltetésre.
3. **Trusted boot és integritás.** A mérési folyamat (például TPM PCR-ek) és a boot integritásának elvei. A laborban emulált TPM-mel dolgozhatunk.
4. **Konténerbiztonság.** A user namespace, a seccomp, a capabilities és a cgroup-korlátok kombinálása egy izolált konténerben, majd a kitörési felület csökkentésének dokumentálása.

Ezen a vonalon a források a hivatalos kernel-dokumentáció biztonsági fejezetei, a SELinux/AppArmor projekt dokumentációja, a seccomp man oldalai és a trusted boot specifikációk. Minden gyakorlatnak izolált laborban kell futnia, és soha nem szabad éles rendszeren vagy engedély nélküli gépen kipróbálni.

A fenti szakirányok közös eleme, hogy a haladást nem az elolvasott oldalak számával, hanem az elvégzett, reprodukálható laborokkal és azok mérési eredményeivel mérjük.

33.4.8 Forrásrend, szakirodalom és ellenőrzés

A rendszerprogramozáshoz használt források három szintre oszthatók: elsődleges specifikációk és szabványok, hivatalos projekt-dokumentáció, valamint szakkönyvek és kurzusok. A tanulási tervben mindhárom szintnek megvan a szerepe.

33.4.9 Elsődleges szabványok és specifikációk

A kernel és a felhasználói tér közötti határfelületeket gyakran szabványok rögzítik. A legfontosabbak:

- **POSIX** – a rendszerhívások, a folyamatok, a szálak, a fájlok és a jelzések viselkedésének hordozható leírása. A tanulónak mindig a konkrét szabványverziót (például POSIX.1-2024) és a szakaszt kell idéznie.
- **UEFI és ACPI** – a boot, a firmware és a hardverleírás specifikációi. A verziószám és a platform itt meghatározó.
- **RISC-V privileged specifikáció és x86 privileged/manual** – a privilege szintek, a lapozás, a megszakítások és a rendszerregisterek leírása. A tanulónak a specifikáció verzióját (például RISC-V privileged ISA 1.13) kell rögzítenie.
- **PCI, USB, ATA/NVMe** – az eszközök és a buszok viselkedését leíró szabványok.

Ezekből a forrásokból soha nem szabad „state of the art” állítást ellenőrizetlenül átvenni. A szabványok stabil részei biztos tudás, de a friss bővítéseknél mindig jelöljük az ellenőrzés szükségességét.

33.4.10 Hivatalos projekt-dokumentáció

A Linux kernel, a FreeRTOS, a Zephyr és a QNX hivatalos dokumentációja az elsődleges forrás a viselkedés megértéséhez. A Linux kernel forrásában található `Documentation/` könyvtár tartalmazza a syscall-ok, a memória- és scheduler réteg, a biztonsági alrendszerek és a virtualizáció leírását. A FreeRTOS és a Zephyr dokumentáció a konfiguráció, az API és a portolás részleteit adja meg.

A projekt-dokumentáció használatánál alapvető szabály, hogy a verziót, a dátumot és a konkrét szakaszt rögzíteni kell. Például:

> „A Linux 6.6 kernel `Documentation/scheduler/sched-design-CFS.rst` szakasza szerint a CFS a virtuális futási időt használja; az állítás ellenőrzéséhez a 6.6-os forráskód `kernel/sched/fair.c` fájlját néztem meg.”

Az ilyen mondat reprodukálható, és nem keveri az olvasott forrást a saját következtetéssel.

33.4.11 Szakkönyvek és tananyagok

A szakkönyvek szerepe a fogalmi keret és a didaktikus felépítés. E könyv mellett a következő irányok ajánlottak:

- **Operating Systems: Three Easy Pieces (OSTEP)** – a klasszikus folyamat, memória és perzisztencia fogalmakat közérthetően, sok gyakorlattal tárgyalja. A szerzők által karbantartott online változat elérhető a University of Wisconsin kiszolgálóján.
- **Linux Kernel Development (Robert Love)** – a Linux kernel belső szerkezetének közérthető bevezetése, bár a verziófüggő részeket mindig ellenőrizni kell.
- **The Linux Programming Interface (Michael Kerrisk)** – a rendszerhívások és a Linux API átfogó kézikönyve.
- **Understanding the Linux Kernel (Bovet & Cesati)** – régebbi, de a memória- és scheduler réteg fogalmi alapjaihoz hasznos.
- **Real-Time Systems (Jane W. S. Liu)** – a valós idejű ütemezés, az RMS és EDF klasszikus tárgyalása.
- **Modern Operating Systems (Tanenbaum & Bos)** – a rendszerarchitektúrák széles áttekintése.

Ezeknek a könyveknek a kiadási éve és a tárgyalt kernelverzió fontos. Egy 2.6-os kernelről szóló magyarázat fogalmilag lehet érvényes, de a konkrét függvénynevek és viselkedés változhatott.

33.4.12 Kurzusok és közösségek

Az egyetemi kurzusok közül kiemelkedik a MIT 6.828 / 6.S081 (xv6) kurzus, amely egy egyszerű, POSIX-szerű oktató operációs rendszer segítségével tanítja a kernel belső működését. A tananyag tartalmaz laborokat, amelyek QEMU-ban futnak, így jól illeszkedik a könyv emulált környezetéhez. Hasonló irány az OSDev wiki és fórum, amely a „from scratch” kernelfejlesztéshez ad gyakorlati útmutatót, de kritikával kezelendő, mert a cikkek nem mindig verziózottak.

A közösségi csatornák közül a Linux Kernel Mailing List (LKML), a kernelnewbies levelezőlista, a Rust for Linux fórumok és a FreeRTOS/Zephyr közösségi fórumok a legfontosabbak. A kérdezés előtt mindig készítsünk minimálisan reprodukálható példát, és tüntessük fel a konfigurációt, a hibaüzenetet és a már megpróbált lépéseket. A konferenciák – Linux Plumbers Conference, FOSDEM, Embedded Linux Conference – a legfrissebb fejlesztési irányok megismerésére alkalmasak, de az ott elhangzó állításokat is vissza kell vezetni elsődleges forrásra.

33.4.13 Labor: személyes tanulási terv készítése

A fejezet laborjának célja, hogy a tanuló a fenti irányokat a saját céljaihoz igazítsa, és egy reprodukálható jegyzetfüzetben vagy emulált környezetben rögzítse a következő időszak tanulási útját. A labor nem igényel kernelkód írását; a hangsúly a tervezésen, a források verziózásán és a mérési protokoll kialakításán van.

33.4.14 A jegyzetfüzet formátuma

A tanulási tervet legjobb verziókövetett jegyzetfüzetben vezetni. Erre alkalmas a Markdown-alapú dokumentum, amelyet git tárolóban helyezünk el, vagy egy Jupyter notebook, amely a mérési kódot és a szöveges elemzést együtt tartalmazza. A jegyzetfüzet elején YAML fejléc rögzíti a kontextust.

Példa a fejlécre:

```
cel: "Linux kernel memóriakezelés mélyebb megértése és egy laphiba-mérő labor elvégzése"
kontextus:
  arch: "x86_64"
  qemu: "8.2.0"
  kernel: "6.6"
  gcc: "13.2"
  glibc: "2.38"
  host: "izolált virtuális gép"
merfoldkovek:
  - id: 1
    tema: "mmap és page fault"
    forras: "Linux kernel Documentation/vm/transhuge.rst, 6.6"
    labor: "QEMU-ban futó kernelben mmap-elt terület laphibák számának mérése"
    mero: "perf stat és saját tracepont"
    vart: "első hozzáférésnél 1 major fault, utána 0"
    mert: "TODO"
```

A `mert` mezőt a terv készítésekor üresen hagyjuk, és csak a tényleges mérés után töltjük ki. Ez biztosítja a várt és a mért érték szétválasztását.

33.4.15 Példa ütemterv

Az alábbi táblázat egy hat hónapos, Linux kernel irányú tanulási terv vázát mutatja. Az időtartamok tájékoztató jellegűek; a tanulónak a saját heti óraszámához kell igazítania.

Hét	Téma	Elsődleges forrás	Labor / mérés	Várt eredmény	Mérési napló
1–2	Rendszerhívások és ABI	POSIX, Linux man pages, kernel syscall table	Syscall-trace írása, `strace` kimenet ellenőrzése	10 vizsgált syscall mindegyike helyes `errno`-t ad	`syscall_trace.md`
3–4	Boot és kernel init	UEFI spec, Linux `Documentation/x86/boot.rst`	QEMU-ban kernel indítása, bootlog elemzése	Bootlogban felismerhető az initramfs betöltése	`boot_notes.md`

Hét	Téma	Elsődleges forrás	Labor / mérés	Várt eredmény	Mérési napló
5–6	Lapozás és TLB	RISC-V vagy x86 privileged spec, Linux `mm` docs	`mmap` + page fault mérés `perf`-fel	Első hozzáférés major fault, további hozzáférés 0 fault	`page_fault.csv`
7–8	Ütemező és CFS	Linux `Documentation/scheduler/sched-design-CFS.rst`	Context switch időmérés `perf sched`	5000 váltás átlaga < 5 µs az adott gépen	`sched_latency.csv`
9–10	Fájlrendszer crash-konzisztencia	ext4 docs, kernel `fs/ext4`	Journal nélküli vs journalos írás szimulált leállás után	A journalos változat konzisztens marad	`fsck_test.md`
11–12	Biztonság: seccomp és capabilities	Linux man pages, kernel `Documentation/security`	Syscall-szűrés mérése, jogosultságszűkítés	A szűrt syscall `EPERM`-et ad, a többinél nincs hatás	`seccomp_lab.md`

A táblázat minden sorában a labor és a várt eredmény konkrét, mérhető, és a mérési napló külön fájlba kerül. A határidők és az óraszámok a tanuló egyéni körülményeitől függnek; a terv akkor jó, ha heti rendszerességgel frissíthető.

33.4.16 Mérési protokoll

A tanulási tervben minden olyan mérföldkőhöz, amely mérést tartalmaz, mérési protokollt kell készíteni. A protokollnak legalább a következőket kell rögzítenie:

- **Hardver és emulált környezet:** CPU típus, órajel, memória méret, QEMU verzió, kernelverzió.
- **Fordítási beállítások:** compiler verzió, optimalizálás, linker script, esetleges kernel config.
- **Mérési módszer:** milyen eszköz generálja az adatokat (`perf`, `trace-cmd`, saját tracepoint, ciklusszámláló), hány ismétlés, milyen melegítési lépések.
- **Várt érték:** a dokumentációból, modellből vagy korábbi mérésből származó referencia.
- **Mért érték:** a tényleges futásból származó adat, nyers formában is.
- **Eltéréselemzés:** ha a mért és várt eltér, mi lehet az ok? A naplóban ezt külön kell írni, nem szabad a mért adatot „módosítani”, hogy jobban illeszkedjen.

Példa egy context switch mérési parancsra:

```
qemu-system-x86_64 -kernel bzImage -initrd initramfs.cpio.gz \
  -nographic -cpu host -enable-kvm -m 512
```

A parancs szemléltetés, nem éles gépen futtatandó utasításként kezelendő. A tényleges méréshez a tanulónak a saját buildelési környezetét kell használnia, és a QEMU-paramétereket a reprodukciós csomagban rögzítenie.

33.4.17 Tipikus hibák és helyes megoldások

A tanulási tervek készítése során gyakori hibák:

1. **Túl széles cél.** A tanuló azt írja, hogy „megtanulom a Linux kernelt”. Ez nem ellenőrizhető. Helyes megoldás: bontsa a célt konkrét, mérhető mérföldkövekre, például „megvalósítom a `sched_setaffinity` rendszerhívás egy egyszerűsített változatát”.
2. **Nincs rögzített konfiguráció.** A jegyzetfüzetben csak annyi áll, hogy „QEMU-ban futott”. Ha a QEMU verzió, a kernel config vagy a CPU modell hiányzik, a mérés nem reprodukálható. Helyes megoldás: a fejlécben és a mérési naplóban minden releváns paraméter szerepeljen.
3. **A várt és mért összemosása.** A tanuló utólag a várt értéket a mért értékhez igazítja, vagy a mért értéket csak akkor jegyzi fel, ha „szép”. Ez etikailag is hibás. Helyes megoldás: a várt értéket a mérés előtt rögzíteni, a mért értéket változatlanul beírni, és az eltérést elemezni.
4. **Forrásverzió hiánya.** A jegyzetfüzetben „a kernel dokumentációja szerint” megfogalmazás szerepel. Helyes megoldás: pontosan idézni a forrást, verzióval, szakasszal és hozzáférési dátummal.
5. **Túl kevés idő a hibakeresésre és a dokumentációra.** A terv csak olvasást és kódolást tartalmaz, a mérési naplók frissítését és a reprodukciós csomag összeállítását nem. Helyes megoldás: minden mérföldkő végén 20–30% időt a dokumentációra és az ellenőrzésre szánni.
6. **Éles környezetben végzett biztonsági kísérlet.** A tanuló a valós gépén próbálja ki a seccomp-profilt vagy a namespace-izolációt. Helyes megoldás: izolált virtuális gép vagy eldobható hardver használata, a hoszt rendszert nem szabad módosítani.

A helyes tanulási terv tehát nem egy lista, hanem egy folyamatosan karbantartott mérnöki dokumentum. A mért és várt értékek szétválasztása, a konfiguráció rögzítése és a források verziózása ugyanaz a fegyelem, mint amit a kernelkód írásánál is elvárunk.

33.4.18 Önellenőrző kérdések

1. Miért nem elegendő egyetlen szakkönyv elolvasása a kernelprogramozói haladáshoz? Milyen három pillérre épül a továbbtanulás útja?
2. Soroljon fel három mezőt, amelyet egy reprodukálható mérési naplónak kötelezően tartalmaznia kell, és indokolja mindegyik szerepét.
3. Mi a különbség a várt és a mért eredmény között? Miért tilos a várt értéket a mért értékhez igazítani?
4. Milyen forrásverziókat kell rögzíteni, ha egy Linux 6.6 kernel memóriakezeléséről teszünk állítást?
5. A biztonsági szakirányon miért izolált laborban kell tanulni? Milyen etikai szabály vonatkozik a seccomp-profilok és namespace-ek gyakorlására?

33.4.19 Lezárás és továbbvezetés

Ezzel a szakasszal a könyv törzsanyaga lezárul. A tanuló most már rendelkezik az operációs rendszerek alapfogalmaival, a folyamatok, a memória, a fájlrendszerek, a hálózat, a biztonság és a virtualizáció ismeretével, valamint egy egyszerű OS megtervezéséhez szükséges gyakorlati kerettel. A továbbtanulás útja azonban nem ér véget; a fenti szakirányok mentén, a személyes tanulási terv alapján lehet tovább mélyíteni a tudást.

A következő lépés a függelékek és a záróprojekt értékelési szempontjainak tanulmányozása. A záróprojekt értékelésénél ugyanazokat a minőségi követelményeket kell alkalmazni, mint itt: reprodukálhatóság, forrásverziók, mért és várt eredmények szétválasztása, és izolált környezetben végzett tesztek. A tanulási terv ezeket a követelményeket viszi át a könyv utáni önálló munkába.

A kernel- és rendszerprogramozás tanulása hosszú, spirális folyamat. A legfontosabb eszköz nem a gyorsaság, hanem a következetesség: minden lépésben kis, ellenőrizhető célt tűzzünk ki, a méréseket naplózzuk, a forrásokat verziózzuk, és a hibákat dokumentáljuk. Aki így halad, az nemcsak tudást, hanem mérnöki ítélőképességet is szerez – ez az, ami az operációs rendszerek belülről való megértését valódi rendszerprogramozói kompetenciává teszi.

Függelékek

Fogalomtár (magyar–angol)

A fogalomtár a könyvben használt magyar és angol szakkifejezéseket, valamint a leggyakoribb félreértéseket rögzíti. A tételek ábécésorrendben szerepelnek.

Magyar	English	Jelentés / megkülönböztetés
ABI	application binary interface	A gépi kódú program és a kernel/rendszerkönyvtárak közötti bináris szerződés: regiszterek, stack, hívási konvenció és rendszerhívási számok.
ACPI	Advanced Configuration and Power Interface	A firmware és az operációs rendszer közötti szabványos interfész hardverleírásra, energiaállapotokra és eseményekre.
ASLR	address space layout randomization	A folyamat címterének véletlen elrendezése, amely megnehezíti a memóriahibák kihasználását.
Átkapcsolás	context switch	A CPU egyik folyamat vagy szál kontextusából a másikba való átállítása; a regiszterek, a stack és a lapasztal állapotát érinti.
Belady-anomália	Belady's anomaly	Az a jelenség, amikor egy lapcsere-algoritmusnál több fizikai lap több laphibát okoz; FIFO esetén fordulhat elő.
Blokkeszköz	block device	Olyan eszköz, amely rögzített méretű blokkokban olvasható és írható, például merevlemez vagy SSD.
Bootloader	bootloader	A firmware által indított program, amely betölti és elindítja a kernelt.
Buddy allokátor	buddy allocator	Kettéosztáson alapuló fizikai lapallokátor, amely gyorsan tud nagyobb összefüggő memóriaterületet adni.

Magyar	English	Jelentés / megkülönböztetés
Cgroup	control group	Linux-os erőforrás-korlátozó és számláló mechanizmus CPU-ra, memóriára, I/O-ra és egyéb erőforrásokra.
Copy-on-write	copy-on-write	Írásig megosztott memória- vagy fájlblokk; az első módosításkor készül külön másolat.
Crash-konzisztencia	crash consistency	A fájlrendszer azon tulajdonsága, hogy hirtelen leállás után is konzisztens, helyreállítható állapotba kerül.
Démon	daemon	Háttérben futó, felhasználói beavatkozás nélkül szolgáltatást nyújtó folyamat.
DMA	direct memory access	A periféria közvetlen memóriahozzáférése a CPU megkerülésével, amely csökkenti a másolási terhet.
Extent	extent	Egy fájl összefüggő blokkjaiból álló, kezdőblokkal és hosszal leírt tartomány; a nagy fájlok töredezettségét csökkenti.
Fájlleíró	file descriptor	A folyamat fájlhoz, sockethez vagy eszközhöz tartozó kis egész azonosítója, amelyen az I/O-műveletek végrehajthatók.
Fájlrendszer	file system	A tárolóeszközön lévő adatok és metaadatok szervezési módja, valamint az ezt kezelő kernelkomponens.
Firmware	firmware	A hardverbe égetett vagy flashben tárolt alapszoftver, amely a bootot és az alapvető eszközkezelést végzi.
Fragmentáció	fragmentation	A memória vagy tároló szabad helyének felaprózódása, amely rontja az allokáció hatékonyságát.
Holtpont	deadlock	Két vagy több folyamat végtelen várakozása, amelyben mindegyik olyan erőforrásra vár, amelyet más tart.
Huge page	huge page	A szokásosnál nagyobb memórialap, amely csökkenti a TLB-terhelést és a lapasztal méretét.

Magyar	English	Jelentés / megkülönböztetés
Hypervisor	hypervisor	Virtuális gépek futtatását lehetővé tevő szoftverréteg, amely a hardvert a vendég OS-ek között osztja meg.
Időszelet	time slice	Egy feladat számára egy ütemezési körben kiosztott CPU-idő.
Init	init	A boot után elinduló első felhasználói folyamat, amely a rendszer többi szolgáltatását elindítja.
Inode	inode	A fájl metaadatait (típus, jogosultság, méret, blokkcímek) tároló adatszerkezet; a fájlnév a könyvtárbejegyzésben van.
IPC	inter-process communication	Folyamatok közötti kommunikáció és szinkronizáció, például pipe, üzenetsor, osztott memória vagy socket.
Journaling	journaling	A fájlrendszer metaadat-változásainak naplózása a crash-konzisztencia érdekében.
Kernel	kernel	Az operációs rendszer privilegizált magja, amely a processzor, a memória, az eszközök és a rendszerhívások kezelését végzi.
Kerneltér	kernel space	A privilegizált cím tér, ahol a kernel fut; a felhasználói programok csak rendszerhívásokon keresztül érhetik el.
Konténer	container	Névterekkel és cgroupokkal izolált folyamat, amely saját fájlrendszer-nézetet és erőforrás-korlátokat kap.
Lapasztal	page table	A virtuális lapok fizikai keretekhez rendelését leíró adatszerkezet, amelyet az MMU használ.
Lapgyorsítótár	page cache	A kernel fájlblokkokat gyorsítótárazó memóriarésze, amely csökkenti a lemez-I/O-t.

Magyar	English	Jelentés / megkülönböztetés
Laphiba	page fault	A virtuális cím elérésének kivétele, amikor a lap nincs a memóriában vagy a hozzáférés szabálytalan.
Lapméret	page size	A virtuális memória lapjának és a fizikai keretnek a mérete, tipikusan 4 KiB vagy nagyobb.
Lapozás	paging	A virtuális memória fix méretű lapokra és fizikai keretekre bontása.
Lock-free	lock-free	Olyan konkurens algoritmus, amely nem használ zárat, és legalább egy szál mindig halad.
Megszakítás	interrupt	Lásd interrupt; a hardver aszinkron jelzése a kernel felé.
Mikrokernel	microkernel	Minimális kernel, amely csak az alapvető mechanizmusokat tartalmazza; a szolgáltatások nagy része felhasználói térben fut.
mmap	memory mapping	Fájl vagy eszköz memóriába képezése, amely lehetővé teszi a közvetlen memóriás hozzáférést.
Monolitikus kernel	monolithic kernel	Olyan kernel, amelyben az ütemező, a memóriakezelő, a fájlrendszerek és a driverek egy címtérben futnak.
Mount	mount	Fájlrendszer csatlakoztatása a névtér egy könyvtárához.
Munkakészlet	working set	Egy folyamat aktuálisan gyakran használt lapjainak halmaza; a mérete befolyásolja a laphibák számát.
Mutex	mutual exclusion lock	Kölcsönös kizárást biztosító zár, amely alvó várakozást is lehetővé tesz.
Névtér	namespace	A kernel erőforrásainak izolált nézete, például PID-, mount-, hálózati vagy felhasználói névtér.
NUMA	non-uniform memory access	Olyan többprocesszoros memóriaarchitektúra, amelyben a memóriaelérés ideje a csomópont helyétől függ.

Magyar	English	Jelentés / megkülönböztetés
OCI	Open Container Initiative	Konténerimage- és runtime-szabvány, amely a hordozható konténerek alapja.
PID	process identifier	A folyamat egyedi azonosítója az adott névtérben.
Pillanatkép	snapshot	Egy fájlrendszer vagy kötet pillanatnyi állapotának konzisztens, gyakran copy-on-write alapú másolata.
Pörgőzár	spinlock	Rövid kritikus szakaszra használt zár, amelynél a várakozó CPU pörög, nem alszik el.
Preemption	preemption	A futó feladat félbeszakítása és a CPU más feladathoz rendelése, például időzítő vagy prioritás miatt.
Priority inversion	priority inversion	Olyan helyzet, amikor egy nagy prioritású feladat egy kis prioritású feladat által tartott erőforrásra vár.
Processzor-affinitás	CPU affinity	Egy folyamat vagy szál engedélyezett CPU-magjainak halmaza.
RCU	read-copy-update	Olvasó-optimális szinkronizációs mechanizmus, amelyben az olvasók zárolás nélkül haladhatnak, az írók késleltetve szabadítják fel a régi adatot.
Rendszerhívás	system call	A felhasználói program kérése a kernel felé privilegizált művelet végrehajtására.
RTOS	real-time operating system	Olyan operációs rendszer, amely meghatározott időkorláton belül garantálja a feladatok végrehajtását.
Seccomp	secure computing mode	Linux rendszerhívás-szűrő mechanizmus, amely a folyamat számára engedélyezett rendszerhívásokat korlátozza.
SELinux	Security-Enhanced Linux	Kötelező hozzáférés-vezérlést (MAC) megvalósító Linux biztonsági modul.
SMP	symmetric multiprocessing	Olyan többprocesszoros rendszer, amelyben a magok azonos módon érik el a memóriát és az eszközöket.

Magyar	English	Jelentés / megkülönböztetés
Socket	socket	Hálózati kommunikációs végpont, amelyet a folyamat fájlleíróként kezelhet.
Swap	swap	A memóriából ideiglenesen kiszorított lapok tárolóeszközön való elhelyezése.
Szál	thread	A folyamaton belüli végrehajtási egység, amely saját regiszter- és stackállapottal, de közös címtérrel rendelkezik.
Szemafór	semaphore	Számláló alapú szinkronizációs primitív, amely erőforrások számát vagy eseményeket jelez.
Tárgyfájl	object file	A fordító által előállított, még nem linkelt gépi kód és szimbólumok.
Thrashing	thrashing	A rendszer túlterhelése, amikor a kevés memória miatt a lapcserélés dominál, és a hasznos munka lelassul.
TLB	translation lookaside buffer	A lapasztal-bejegyzések gyorsítótára a CPU-ban, amely felgyorsítja a címfordítást.
TPM	trusted platform module	Hardveres biztonsági modul méréshez, kulcstároláshoz és integritásigazoláshoz.
UEFI	Unified Extensible Firmware Interface	A modern firmware és boot interfész szabványa, amely a BIOS utódja.
Unikernel	unikernel	Egyetlen alkalmazásra szabott, minimális kernelkép, amely könyvtárként linkeli a szükséges OS-funkciókat.
Ütemező	scheduler	A CPU-idő elosztásáért felelős kernelkomponens, amely kiválasztja a következő futó feladatot.
VFS	virtual file system	A kernel fájlrendszer-absztrakciója, amely egységes interfészt ad a különböző fájlrendszerekhez.
Virtuális memória	virtual memory	A folyamatok számára látszólagos, a fizikai memóriától független címtér, lapozással és védelemmel.

Magyar	English	Jelentés / megkülönböztetés
WASM	WebAssembly	Hordozható bájtkód-formátum, amely homokozóban futtatható; könnyűsúlyú izolációra is használják.
Watchdog-időzítő	watchdog timer	Időzítő, amely a rendszer vagy folyamat lefagyása esetén újraindítást vagy hibajelzést vált ki.
WCET	worst-case execution time	Egy feladat legrosszabb végrehajtási ideje, amely a valós idejű ütemezhetőség alapja.
Zero-copy	zero-copy	Olyan I/O-technika, amely elkerüli az adatok felesleges másolását a kernel és a felhasználói tér között.
Zombi folyamat	zombie process	Befejeződött folyamat, amelynek kilépési állapotát a szülő még nem olvasta ki.

Képlet- és algoritmusgyűjtemény

A gyűjtemény a könyvben használt legfontosabb ütemezési, memória-, tároló- és hálózati számításokat, valamint a hozzájuk tartozó algoritmusok jellemzőit rögzíti. Minden képlet mellett szerepel a feltétel, a mértékegység és a gyakorlati értelmezés. A mérést mindig a hardverrel, a konfigurációval és a mérési módszerrel együtt kell közölni.

Ütemezési metrikák

Egy P_i feladat érkezési ideje A_i, CPU-igénye C_i, befejezési ideje F_i, első futásának ideje S_i. Ekkor:

$$T_i = F_i - A_i, \qquad W_i = T_i - C_i, \qquad R_i = S_i - A_i.$$

T_i a teljes átfutási idő (turnaround), W_i a várakozási idő, R_i a válaszidő. A CPU-kihasználtság:

$$U = \frac{\text{összes foglalt CPU-idő}}{\text{teljes megfigyelési idő}}.$$

A metrikákat mindig ugyanarra a terhelésre és ugyanazzal az időablakkal kell számítani; az átlag mellett a legrosszabb eset is fontos.

Little-törvény

Stabil rendszerben:

$$L = \lambda W,$$

ahol L a rendszerben lévő kérések átlagos száma, λ az érkezési ráta (kérés/másodperc), W az átlagos tartózkodási idő. A képlet a folyamatokra, az I/O-kérésekre és a hálózati kapcsolatokra is alkalmazható. Nem érvényes tranziens, nem stacionárius terhelésre.

Round-robin és időszelet

Round-robin ütemezésnél minden kész feladat legfeljebb q ideig fut, majd a sor végére kerül. Ha n azonos CPU-igényű feladat van, a maximális várakozási idő durván:

$$W_{\max} \approx (n-1)q.$$

A képlet idealizált; a valós várakozást a blokkolások, az I/O és a prioritások módosítják. A q túl kicsi értéke növeli a kontextusváltások arányát, a túl nagy érték rontja a válaszidőt.

Rate-monotonic és EDF ütemezés

Rate-monotonic (RMS) fix prioritású ütemezésnél a rövidebb periódusú feladat kap nagyobb prioritást. Ha minden $D_i = T_i$, és a feladatok preemptívek, a processzorkihasználtság elégséges feltétele:

$$U = \sum_{i=1}^{n} \frac{C_i}{T_i} \leq n\left(2^{1/n} - 1\right).$$

Nagy n esetén a korlát $n \to \infty$ mellett $\ln 2 \approx 0{,}693$. Earliest deadline first (EDF) esetén a pontos feltétel:

$$U = \sum_{i=1}^{n} \frac{C_i}{T_i} \leq 1.$$

Az EDF elméletileg jobb kihasználtságot ér el, de túlterhelés esetén a viselkedése kevésbé kiszámítható, ezért a gyakorlatban sok RTOS fix prioritást használ.

Kontextusváltás költsége

Ha egy időszelet hossza q, a kontextusváltás költsége c, akkor a váltásra fordított idő aránya:

$$f = \frac{c}{q + c}.$$

A képlet akkor hasznos, ha a váltás költsége mérhető. A cache- és TLB-hatás miatt a tényleges költség nagyobb lehet a regisztermentésnél; a mérést célhardveren vagy emulátorban kell elvégezni.

Amdahl-törvény

Ha a feladat párhuzamosítható hányada p, és a párhuzamos rész sebességnövelése s, akkor:

$$S(s) = \frac{1}{(1-p) + \frac{p}{s}}.$$

A képlet felső korlátot ad a többmagos gyorsulásra. A kernelben a soros szakaszok (globális zárak, közös számlálók, I/O) gyakran a méretezhetőség korlátját jelentik.

Hatékony memóriaelérés

Lapozott memóriában a hatékony elérési idő:

$$EAT = (1-p)\, t_m + p\, t_{pf},$$

ahol t_m a memóriaelérés ideje, t_{pf} a laphiba kiszolgálásának ideje, p pedig a laphiba-arány. A képlet feltételezi, hogy a lapasztal-bejegyzés a TLB-ben van; TLB-miss esetén további szintek adódnak.

Lapasztal mérete és TLB-reach

Egy egyszintű lapasztal bejegyzéseinek száma:

$$N_{\mathrm{PTE}} = 2^{v-o},$$

ahol v a virtuális cím bitszáma, o a lapoffset bitszáma. A lapasztal mérete $N_{\mathrm{PTE}} \cdot e$, ahol e a bejegyzés mérete. A TLB által lefedett memória:

$$\text{TLB reach} = N_{\mathrm{TLB}} \cdot \text{lapméret}.$$

A több szintű lapasztal csökkenti a memóriaigényt, de növeli a laphiba/kivétel esetén bejárandó szintek számát.

Munkakészlet és laphiba-arány

A $W(t, \Delta)$ munkakészlet azon lapok halmaza, amelyeket a folyamat az elmúlt Δ időablakban elért. A laphiba-arány:

$$p = \frac{\text{laphibák száma}}{\text{összes memóriahozzáférés}}.$$

A munkakészlet mérete és a rendelkezésre álló keretek viszonya magyarázza a thrashingot: ha a folyamatok munkakészleteinek összege meghaladja a fizikai memóriát, a lapcserélés válik dominánssá.

Buddy allokátor

A buddy allokátor 2^k méretű blokkokat kezel. Egy kéréshez a legkisebb elég nagy blokkot választja, majd felezi, amíg eléri a kért méretet. Felszabadításkor a szomszédos „buddy” blokkal egyesül, ha az is szabad. A belső fragmentáció legfeljebb a kért méret alatti következő hatványig terjedhet. A műveletek költsége jellemzően $O(\log M)$, ahol M a memória mérete.

Inode-alapú fájlméret

Ha egy inode d közvetlen blokkra, egy egyszeres indirekt blokkra, egy kétszeres indirekt blokkra és egy háromszoros indirekt blokkra mutat, a blokkméret B, egy blokkban b pointer fér el, akkor a maximális fájlméret:

$$\text{max size} = (d + b + b^2 + b^3) \cdot B.$$

A képlet a klasszikus UNIX inode-sémát írja le. A modern fájlrendszerek extenteket és B-fákat használnak, amelyek más korlátokat és más teljesítményprofilt adnak.

RAID-kapacitás

N azonos méretű, S méretű lemez esetén a hasznos kapacitás:

$$C_{\text{RAID0}} = N \cdot S, \qquad C_{\text{RAID1}} = \frac{N}{2} \cdot S,$$

$$C_{\text{RAID5}} = (N-1) \cdot S, \qquad C_{\text{RAID6}} = (N-2) \cdot S.$$

A RAID nem helyettesíti a mentést: a vezérlőhiba, a fájlrendszer-hiba és az emberi hiba ellen külön védelem kell. A rebuild idő alatt a redundancia csökken, ezért a helyreállítási terv része a terhelés korlátozása.

Lemez-IOPS és átbocsátóképesség

Egy tipikus merevlemez egy I/O műveletének ideje:

$$t_{\text{IO}} = t_{\text{seek}} + t_{\text{rot}} + \frac{\text{blokk mérete}}{\text{sávszélesség}}.$$

Az IOPS ennek reciproka, de a valós értéket a sorban állás, a cache és a RAID-stratégia is befolyásolja. SSD-nél a seek és a rotációs késleltetés eltűnik, viszont a write amplification és a garbage collection válik fontossá.

Hálózati sávszélesség-késleltetés szorzat

A csővezetékben lévő adat mennyisége:

$$BDP = \text{sávszélesség} \cdot RTT.$$

Nagy BDP esetén a TCP ablakméret és a pufferek nélkül a kapcsolat nem tudja kihasználni a sávszélességet. A zero-copy, a nagy ablak és a megfelelő pufferelés együtt szükséges a jó átbocsátóképességhez.

Rendelkezésre állás

$$A = \frac{\text{MTBF}}{\text{MTBF} + \text{MTTR}}.$$

A képlet független hibákra és hosszú megfigyelési időre érvényes. A rendszer valós rendelkezésre állását a közös függőségek, a tervezett karbantartás és a részleges hibák is csökkentik. Az SLO-t a felhasználó által észlelt elérhetőség alapján kell megfogalmazni.

Cache AMAT

Az átlagos memóriaelérési idő:

$$AMAT = t_{\text{hit}} + \text{miss rate} \cdot t_{\text{miss penalty}}.$$

A képlet cache-re, TLB-re és lapgyorsítótárra is alkalmazható. A találati arány önmagában nem elegendő: a miss penalty és a munkaterhelés mintázata együtt határozza meg a teljesítményt.

Rendszerhívás- és interfészgyűjtemény

Ez a gyűjtemény a könyvben előforduló rendszerhívási és kernelinterfész- kategóriákat foglalja össze. Nem teljes API-referencia: a pontos szignatúrát, a hibakódokat és a platformeltéréseket mindig a hivatalos man oldalon vagy a gyártó dokumentációjában kell ellenőrizni.

Rendszerhívás életciklusa

1. A felhasználói program a libc-wrapperen keresztül előkészíti a rendszerhívás számát és argumentumait.
2. A processzor privilegizált módba vált (trap, syscall vagy SVC utasítás).
3. A kernel a syscall-táblából kiválasztja a kezelőt, ellenőrzi a paramétereket és a jogosultságokat.
4. A kezelő végrehajtja a műveletet, majd visszatér a felhasználói térbe.
5. A visszatérési érték a siker értéke vagy hibakód; a hibát a libc általában `errno`-ba fordítja.

A rendszerhívás költsége magában foglalja a módváltást, a regisztermentést, a cache- és TLB-hatásokat, valamint a biztonsági ellenőrzéseket. Ezért a nagy számú apró rendszerhívás lassabb lehet, mint egy nagyobb, pufferelt művelet.

Linux/POSIX rendszerhívás-kategóriák

Kategória	Jellemző hívások	Cél
Folyamat	`fork`, `vfork`, `clone`, `execve`, `exit`, `wait4`, `waitid`	folyamat létrehozása, indítása, befejezése és szinkronizálása
Szál	`clone`, `clone3`, `set_tid_address`, `futex`	szál létrehozása és alacsony szintű szinkronizáció
Memória	`brk`, `mmap`, `munmap`, `mprotect`, `mremap`, `madvise`	címter, lapvédelem és memóriatanácsadás
Fájl	`openat`, `openat2`, `close`, `read`, `write`, `pread64`, `pwrite64`, `lseek`, `fsync`, `fdatasync`	fájl megnyitása, olvasása, írása és szinkronizálása
Metaadat	`statx`, `fstat`, `newfstatat`, `chmod`, `fchmodat`, `chown`, `utimensat`	fájlattribútumok lekérdezése és módosítása
Könyvtár	`getdents64`, `mkdirat`, `unlinkat`, `renameat2`, `linkat`, `symlinkat`	névtér és könyvtárbejegyzések kezelése
I/O-multiplexelés	`select`, `poll`, `ppoll`, `epoll_create1`, `epoll_ctl`, `epoll_wait`	több fájlleíró egyidejű figyelése
Cső és IPC	`pipe2`, `socketpair`, `shm_open`, `sem_open`, `mq_open`	folyamatok közötti csatornák és osztott erőforrások
Hálózat	`socket`, `bind`, `listen`, `accept4`, `connect`, `sendto`, `recvfrom`, `sendmsg`, `recvmsg`, `shutdown`	hálózati végpontok és üzenetküldés
Jelzés	`kill`, `tgkill`, `rt_sigaction`, `rt_sigprocmask`, `signalfd4`	aszinkron események és szignálok
Idő	`clock_gettime`, `clock_nanosleep`, `timerfd_create`, `timerfd_settime`	időmérés, alvás és időzítők
Jogosultság	`setuid`, `setgid`, `setresuid`, `capget`, `capset`, `prctl`	identitás és képességek kezelése
Biztonság	`seccomp`, `landlock_create_ruleset`, `landlock_restrict_self`	rendszerhívás-szűrés és homokozó
Virtualizáció	`unshare`, `setns`, `mount`, `pivot_root`	névterek, mount és konténerizoláció
Aszinkron I/O	`io_uring_setup`, `io_uring_enter`, `io_uring_register`	nagy teljesítményű, eseményvezérelt I/O
Megfigyelés	`perf_event_open`, `bpf`, `process_vm_readv`	teljesítménymérés, eBPF és folyamatmemória

Linux x86-64 rendszerhívási ABI

Elem	Konvenció
Hívási szám	`rax`
1. argumentum	`rdi`
2. argumentum	`rsi`
3. argumentum	`rdx`
4. argumentum	`r10` (nem `rcx`)
5. argumentum	`r8`
6. argumentum	`r9`
Visszatérési érték	`rax`
Hibajelzés	negatív `rax` a `-4095` tartományban; a libc `errno`-t állít
Utasítás	`syscall`
Megőrzendő regiszterek	`rbx`, `rbp`, `r12`–`r15`

Az ABI részletei a platform x86-64 psABI dokumentációjában és a Linux `syscall(2)` man oldalán találhatók. A 4. argumentum `r10`-ben van, mert az `rcx`-et a `syscall` utasítás felülírja.

RISC-V és ARM64 megjegyzés

RISC-V Linuxon a rendszerhívás számát `a7`, az argumentumokat `a0`–`a5` tartalmazza, a visszatérési érték `a0`; a csapda az `ecall` utasítással történik. ARM64 Linuxon a szám `x8`-ban, az argumentumok `x0`–`x5`-ben vannak, a visszatérés `x0`; a `svc` utasítás vált kernel módba. A regiszternevek és a hívási számok platformonként eltérnek, ezért a rendszerhívási táblát a cél-ABI szerint kell használni.

Windows API-párhívások

A Windows nem POSIX rendszerhívási táblát használ, hanem a Win32 API-n és a natív `Nt*`/`Zw*` interfészen keresztül érhető el. A fogalmi megfeleltetés:

POSIX/Linux	Windows
`open/read/write`	`CreateFile`, `ReadFile`, `WriteFile`
`fork/execve`	`CreateProcess`
`waitpid`	`WaitForSingleObject`
`mmap`	`CreateFileMapping`, `MapViewOfFile`, `VirtualAlloc`
`pthread_create`	`CreateThread`
`socket/bind/listen`	Winsock: `WSASocket`, `bind`, `listen`
`select/poll`	`select`, `WSAEventSelect`, I/O completion port
`kill`	`TerminateProcess`
`chmod`	`SetFileSecurity`, ACL-ek

A Windows biztonsági modellje tokenekre, SID-ekre és hozzáférési ellenőrző listákra épül; a POSIX UID/GID-modell nem képezhető le veszteségmentesen.

RTOS-interfészek

Fogalom	FreeRTOS	Zephyr
Feladat létrehozása	`xTaskCreate`	`k_thread_create`
Késleltetés	`vTaskDelay`	`k_sleep`
Üzenetsor	`xQueueSend/xQueueReceive`	`k_msgq_put/k_msgq_get`
Szemafór	`xSemaphoreTake/xSemaphoreGive`	`k_sem_take/k_sem_give`
Mutex	`xSemaphoreCreateMutex`	`k_mutex_lock/k_mutex_unlock`
Eseménycsoport	`xEventGroupSetBits`	`k_event_set`
Időzítő	`xTimerCreate`	`k_timer_start`
Megszakítás	`portSET_INTERRUPT_MASK_FROM_ISR`	`irq_lock/irq_unlock`

Az RTOS-API-k gyakran nem rendszerhívások, hanem könyvtárhívások, amelyek közvetlenül a kernel adatszerkezeteit érik el. A prioritásöröklés, az időzítési korlátok és a megszakítási kontextus szabályai eltérnek a hagyományos POSIX-rendszerektől.

Hibakezelés és visszatérési értékek

- A POSIX-hívások sikeres visszatérése jellemzően `0` vagy egy nemnegatív érték; hiba esetén `-1`, és az `errno` tartalmazza a hibakódot.
- A `read`/`write` részleges műveletet is jelezhet; a rövid olvasást és írást ciklusban kell kezelni.
- Az `EINTR` azt jelzi, hogy egy szignál megszakította a hívást; a hívást lehet ismételni, ha a művelet idempotens.
- Az `EAGAIN/EWOULDBLOCK` nem blokkoló I/O esetén azt jelenti, hogy a művelet később újrapróbálható.
- A `mmap` hibáját külön kell kezelni; a `MAP_FAILED` nem érvényes cím.
- A fájlleírók öröklődése és a `close-on-exec` beállítás biztonsági szempontból is fontos.

Biztonsági megjegyzések

- A rendszerhívás paramétereit a kernelnek ellenőriznie kell: címek, hosszúságok, jogosultságok és a felhasználói pointerek érvényessége.
- A `copy_from_user`/`copy_to_user` típusú másolás szükséges; a nyers pointer használata a felhasználói térből kernelhibához vezethet.
- A `seccomp` és a Landlock csökkenti a támadási felületet, de nem helyettesíti a hagyományos jogosultsági ellenőrzést.
- A `setuid` programok külön figyelmet igényelnek; a környezet, a fájlleírók és a dinamikus linker viselkedése támadási felület.
- A rendszerhívási számtáblát és a struktúrák méretét a cél-ABI szerint kell ellenőrizni; a 32 és 64 bites kompatibilitási rétegek eltérnek.

Kompetenciatérkép és tanulási útvonal

A térkép a könyv fejezeteit nyolc kompetenciaterülethez rendeli. A fejezetszámok a PDF és a HTML azonosítóival egyeznek; a záróprojekt (osch31) minden területet integrál.

Kompetenciaterületek

Kompetencia	Fejezetek	Bizonyíték	Szint
C1 – OS-alapok és architektúrák	osch01–osch05	OS-típusok összehasonlítása, kernelarchitektúra, bootfolyamat, syscall-vázlat	alapszint
C2 – Folyamatok, ütemezés és konkurencia	osch06–osch09	ütemezési szimuláció, szinkronizációs megoldás, holtpontanalízis	haladó
C3 – Memória és I/O	osch10–osch13	lapasztal-számítás, lapcsere-szimuláció, allokátor, driver-vázlat	haladó
C4 – Fájlrendszerek és tárolás	osch14–osch17	fájlrendszer-terv, journaling/CoW elemzés, RAID- és recovery-terv	haladó
C5 – Hálózat, biztonság és virtualizáció	osch18–osch21	socket-program, netfilter-terv, threat model, konténerizoláció	haladó
C6 – RTOS, beágyazott és elosztott rendszerek	osch22–osch25	RTOS-ütemezés, priority inversion elemzés, mobil/elosztott architektúra	haladó
C7 – Kernelimplementáció és eszközök	osch26–osch30	bootolható kernel, syscallok, driver, teszt- és debugterv	mesterszint
C8 – Tervezés és záróprojekt	osch31–osch32	end-to-end OS-projekt, tesztriport, dokumentáció és bemutató	mesterszint

Tanulási útvonalak

Rendszerprogramozó

Cél: az OS viselkedésének megértése és hatékony rendszerprogramok írása.

1. Alapok: osch01–osch05.
2. Folyamatok és ütemezés: osch06–osch07.
3. Konkurencia: osch08–osch09.
4. Memória és I/O: osch10–osch13.
5. Fájlrendszerek: osch14–osch15.
6. Hálózat és biztonság: osch18–osch20.
7. Záróprojekt: osch31.

Kernel- és beágyazott mérnök

Cél: kernelkomponensek és valós idejű rendszerek fejlesztése.

1. Alapok és architektúrák: osch01–osch05.
2. Folyamatok, ütemezés, konkurencia: osch06–osch09.
3. Memória és I/O: osch10–osch13.
4. RTOS és beágyazott rendszerek: osch22–osch24.
5. Kernelimplementáció: osch26–osch30.
6. Záróprojekt: osch31.

Platform-, virtualizációs és SRE-mérnök

Cél: megbízható, izolált és skálázható rendszerek üzemeltetése.

1. Alapok: osch01–osch05.
2. Folyamatok és memória: osch06–osch13.
3. Fájlrendszerek és tárolás: osch14–osch17.
4. Hálózat, biztonság és virtualizáció: osch18–osch21.
5. Megbízhatóság és teljesítmény: osch29–osch30.
6. Záróprojekt: osch31.

Biztonsági szakember

Cél: az OS támadási felületének és védelmi mechanizmusainak megértése.

1. Alapok és syscallok: osch01–osch05.
2. Folyamatok és memória: osch06–osch13.
3. Fájlrendszerek és tárolás: osch14–osch17.
4. Hálózat és izoláció: osch18–osch21.
5. RTOS és beágyazott biztonság: osch22–osch24.
6. Kernelhibakeresés: osch27–osch30.
7. Záróprojekt: osch31.

Oktató és továbbképző

Cél: az OS-fogalmak tanítható, kísérletekkel alátámasztott bemutatása.

1. Alapok és architektúrák: osch01–osch05.
2. Folyamatok és ütemezés: osch06–osch09.
3. Memória és fájlrendszerek: osch10–osch17.
4. Hálózat, biztonság, virtualizáció: osch18–osch21.
5. RTOS és speciális rendszerek: osch22–osch25.
6. Saját OS és tesztelés: osch26–osch32.

Értékelési rubrika

Minden kompetenciaterületnél három szintet különböztetünk meg:

- **Ismeri:** meg tudja magyarázni a fogalmakat, az algoritmusokat és a tipikus hibákat.
- **Alkalmazza:** önállóan meg tudja valósítani a laborban kért komponenst, értelmezi a mérést és a hibajelenséget.
- **Megtervezi:** adott hardver, valós idejű, biztonsági és üzemeltetési feltétel mellett meg tudja indokolni az architektúra- és algoritmusválasztást, ismeri a korlátokat, és képes a rendszert tesztelni, dokumentálni és továbbfejleszteni.

A záróprojekt értékelése négy dimenzióban történik: működés és reprodukálhatóság, memória- és erőforrás-biztonság, teszteltség és hibakezelés, valamint dokumentáció és bemutató. A projekt csak akkor fogadható el, ha a mért és a várt eredményt elkülöníti, és a nem futtatott mérést, benchmarkot vagy hardveres eredményt egyértelműen jelöli.

Bibliográfia és forráskulcsok

Ezek a tervezéshez használt kiindulópontok, nem ebben a könyvben tételesen ellenőrzött bizonyítékok. A fejezetek kutatási lépése a konkrét szakaszt, verziót, hardverplatformot és ellenőrzési dátumot rögzíti. A kernel-dokumentációnál és a szabványoknál a mindenkori hivatalos változat az elsődleges.

Tankönyvek és alapművek

- https://pages.cs.wisc.edu/~remzi/OSTEP/ (Operating Systems: Three Easy Pieces)
- https://www.os-book.com/ (Operating System Concepts)
- https://www.cs.vu.nl/~ast/ (Tanenbaum: Modern Operating Systems)
- https://csapp.cs.cmu.edu/ (Computer Systems: A Programmer's Perspective)
- https://www.oreilly.com/library/view/linux-kernel-development/9780768696974/ (Linux Kernel Development)
- https://www.oreilly.com/library/view/understanding-the-linux/0596005652/ (Understanding the Linux Kernel)
- https://learn.microsoft.com/en-us/sysinternals/resources/windows-internals (Windows Internals)
- https://man7.org/tlpi/ (The Linux Programming Interface)
- https://www.catb.org/esr/writings/taoup/html/ (The Art of Unix Programming)
- https://beej.us/guide/bgnet/ (Beej's Guide to Network Programming)

Kernel-dokumentáció és források

- https://docs.kernel.org/ (Linux kernel dokumentáció)
- https://man7.org/linux/man-pages/ (Linux man-pages)
- https://lwn.net/ (LWN.net)
- https://kernelnewbies.org/ (Kernel Newbies)
- https://docs.kernel.org/process/ (Linux kernel fejlesztési folyamat)
- https://docs.kernel.org/dev-tools/testing-overview.html (Linux testing)
- https://github.com/google/syzkaller (syzkaller)
- https://docs.kernel.org/driver-api/ (Linux driver API)

- https://lwn.net/Kernel/LDD3/ (Linux Device Drivers)
- https://docs.kernel.org/filesystems/ (Linux filesystems)
- https://docs.kernel.org/mm/ (Linux memory management)
- https://docs.kernel.org/locking/ (Linux locking)
- https://docs.kernel.org/scheduler/ (Linux scheduler)
- https://docs.kernel.org/networking/ (Linux networking)
- https://docs.kernel.org/admin-guide/cgroup-v2.html (cgroup v2)
- https://www.kernel.org/doc/html/latest/RCU/ (Linux RCU)
- https://www.kernel.org/doc/html/latest/userspace-api/seccomp_filter.html (seccomp)

Boot, firmware és hardver

- https://uefi.org/specifications (UEFI és ACPI specifikációk)
- https://www.devicetree.org/ (Device Tree)
- https://wiki.osdev.org/ (OSDev wiki)
- https://www.qemu.org/docs/master/ (QEMU)
- https://sourceware.org/gdb/documentation/ (GDB)
- https://riscv.org/technical/specifications/ (RISC-V specifikációk)
- https://gitlab.com/x86-psABIs/x86-64-ABI (x86-64 ABI)
- https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html (Intel Software Developer Manuals)
- https://developer.arm.com/documentation (Arm dokumentáció)

Folyamatok, ütemezés és konkurencia

- https://pubs.opengroup.org/onlinepubs/9699919799/ (POSIX)
- https://doi.org/10.1145/321738.321743 (Rate-monotonic scheduling)
- https://en.cppreference.com/w/cpp/atomic/memory_order (C++ memóriamodell)
- https://man7.org/linux/man-pages/man7/pthreads.7.html (POSIX threads)
- https://man7.org/linux/man-pages/man7/namespaces.7.html (Linux namespaces)
- https://man7.org/linux/man-pages/man7/cgroups.7.html (Linux cgroups)

Memória, I/O és tárolás

- https://sourceware.org/glibc/manual/ (glibc manual)
- https://www.kernel.org/doc/html/latest/block/ (Linux block layer)
- https://docs.kernel.org/filesystems/ext4/ (ext4)
- https://docs.kernel.org/filesystems/xfs/xfs.html (XFS)
- https://btrfs.readthedocs.io/ (Btrfs)
- https://openzfs.github.io/openzfs-docs/ (OpenZFS)
- https://learn.microsoft.com/en-us/windows-server/storage/file-server/ntfs-overview (NTFS)
- https://datatracker.ietf.org/doc/html/rfc8881 (NFSv4)
- https://learn.microsoft.com/en-us/windows-server/storage/file-server/file-server-smb-overview (SMB)
- https://gitlab.com/cryptsetup/cryptsetup/-/wikis/home (cryptsetup/LUKS)
- https://www.open-scap.org/ (OpenSCAP)

Hálózat

- https://datatracker.ietf.org/doc/html/rfc9293 (TCP, RFC 9293)
- https://datatracker.ietf.org/doc/html/rfc768 (UDP)
- https://datatracker.ietf.org/doc/html/rfc1034 (DNS)
- https://datatracker.ietf.org/doc/html/rfc2131 (DHCP)
- https://www.netfilter.org/documentation/ (netfilter)
- https://ebpf.io/what-is-ebpf/ (eBPF)
- https://docs.docker.com/network/ (Docker networking)
- https://www.kernel.org/doc/html/latest/networking/ (Linux networking)

Biztonság, virtualizáció és konténerek

- https://selinuxproject.org/page/Main_Page (SELinux)
- https://apparmor.net/ (AppArmor)
- https://docs.kernel.org/security/ (Linux security alrendszer)
- https://man7.org/linux/man-pages/man7/capabilities.7.html (Linux capabilities)
- https://www.linux-kvm.org/page/Main_Page (KVM)
- https://firecracker-microvm.github.io/ (Firecracker)
- https://github.com/opencontainers/runtime-spec (OCI runtime spec)
- https://www.opencontainers.org/ (Open Container Initiative)
- https://webassembly.org/ (WebAssembly)
- https://sel4.systems/ (seL4)
- https://unikraft.org/docs/ (Unikraft)
- https://mirage.io/ (MirageOS)

RTOS, beágyazott és mobil

- https://www.freertos.org/Documentation/00-Overview (FreeRTOS)
- https://docs.zephyrproject.org/latest/ (Zephyr)
- https://www.qnx.com/developers/docs/ (QNX)
- https://www.windriver.com/products/vxworks (VxWorks)
- https://www.autosar.org/ (AUTOSAR)
- https://source.android.com/docs (Android Open Source Project)
- https://support.apple.com/guide/security/welcome/web (Apple Platform Security)
- https://www.freertos.org/Documentation/RTOS_book.html (FreeRTOS RTOS könyv)

Saját OS és oktatási anyagok

- https://pdos.csail.mit.edu/6.828/2023/xv6/book-riscv-rev3.pdf (xv6 könyv)
- https://pdos.csail.mit.edu/6.828/2023/xv6.html (MIT 6.828 xv6)
- https://littleosbook.github.io/ (The Little Book About OS Development)
- https://os.phil-opp.com/ (Writing an OS in Rust)
- https://www.gnu.org/software/grub/manual/grub/ (GRUB manual)
- https://www.gnu.org/software/binutils/ (GNU Binutils)
- https://gcc.gnu.org/onlinedocs/ (GCC manual)

Szabványok, tanúsítás és jogi háttér

- https://www.iso.org/standard/68383.html (ISO 26262-1)
- https://webstore.iec.ch/publication/5515 (IEC 61508)
- https://www.rtca.org/ (DO-178C)
- https://www.iso.org/standard/27001 (ISO/IEC 27001)

Magyar és oktatási források

- https://www.vik.bme.hu/ (BME Villamosmérnöki és Informatikai Kar)
- https://www.inf.elte.hu/ (ELTE Informatikai Kar)
- https://www.iit.bme.hu/ (BME Automatizálási és Alkalmazott Informatikai Tanszék)
- https://www.mit.bme.hu/ (BME Méréstechnika és Információs Rendszerek Tanszék)
- https://www.bme.hu/ (Budapesti Műszaki és Gazdaságtudományi Egyetem)